

kcc

A Formal C17 Compiler in ARM64 Assembly

that cross-compiles to x86-64

documentation is the source.

Cuauhtémoc Pacheco
co-authored with an AI · Claude

kcc 1.0.0

kcc

LITERATE DRIVEN DEVELOPMENT

Volume 1

Literate Driven Development, Volume 1.
kcc: A Formal C17 Compiler in ARM64 Assembly that cross-compiles to x86-64.

First edition, 2026.

Copyright © 2026 Cuauhtémoc Pacheco.
Co-authored with an artificial-intelligence collaborator (Claude, Anthropic).

This book and the kcc source it contains are released under the **MIT License**.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

ISBN: *to be assigned*.
Legal deposit / Depósito legal: *to be assigned*.

Independently published.

Learn more about Literate Driven Development at https://literatedriven.com.

Typeset with LuaLaTeX from a weft literate program. The cover is generated from `assets/cover.svg`. The complete source, including this book, lives in the kcc repository.

kcc

A Formal C17 Compiler in ARM64 Assembly

that cross-compiles to x86-64

A Literate Program

LITERATE DRIVEN DEVELOPMENT ▪ Volume 1

Cuauhtémoc Pacheco

co-authored with an AI ▪ Claude

Independently published ▪ 2026

literatedriven.com

For everyone who believes
that a program should be able
to explain itself.

And for the friends, colleagues,
and all those who make programming,
artificial intelligence, and everything
we achieve with them into tools
for a better society.

Acknowledgments

This book stands on Donald Knuth's single idea: that a program can be written to be read by people first and machines second. Every page here is an attempt to honour it.

I am grateful to the weft literate-programming tool, without which the source and the prose could not have been kept as one document; to the free software that builds and checks kcc—the assembler, the linker, LaTeX, and the reference compilers that serve as its oracle; and to the artificial intelligence that worked alongside me on this volume, turning long stretches of assembly and exposition into a dialogue rather than a solitary labour.

Finally, to the readers who open a compiler expecting to be told how it works, and are willing to be shown: this book is for you.

Preface

kcc is a C compiler. It is also an argument.

The compiler is real. It reads C17, checks it against the Standard, optimises it, allocates its registers, and emits native ARM64 machine code and a Mach-O object that Apple's linker accepts; it compiles itself to a byte-identical binary; it passes an exhaustive battery of conformance tests against a reference compiler. It is also a genuine cross-compiler: from that same Apple-silicon binary it *generates* x86-64 code—a different instruction set, object format, and calling convention—that runs on Linux/x86-64 and, on the Mac itself, under Apple's *Rosetta* translator. The distinction matters and is worth stating plainly: the code generation is entirely kcc's work; Rosetta (and, for Linux binaries, QEMU) only *runs* the result on the ARM machine, so that x86-64 output can be executed and checked against a reference without a second computer. It generates x86-64 the way `gcc` does—selecting the instructions, laying out the stack, obeying the System V ABI, and printing assembly for the system assembler to encode—not by translating one machine's assembly into another's. The argument is this: that literate programming, practised with an artificial-intelligence collaborator, is a way of building software that is at once more *certain*, more *tractable*, *faster*, and *cheaper* than the way most software is built today—and that the result can aspire to be not merely correct but beautiful.

Literate Programming: Narrator and Engineer at Once

> *Let us* ***change our traditional attitude to the construction of programs****. Instead of imagining that our main task is to instruct a computer what to do, let us concentrate rather on* ***explaining to human beings what we want a computer to do****.*
>
> —Donald Knuth, *Literate Programming*, 1984

This is the key to everything that follows. Donald Knuth's literate programming inverted the relationship between a program and its explanation. A literate program is not code with comments; it is an essay—a human narrative—with code woven into it, presented in whatever order best serves understanding and mechanically extracted into something a machine can run. Knuth's aim was frankly aesthetic: to produce "works of literature," programs that could be read for pleasure and understood as a whole rather than reverse-engineered a function at a time.

Literate programming is remembered as expensive, and that memory is a misjudgement. The narrative is real work, but it is not *extra* work. A program that is correct was understood before it was written: someone decided, in some order, what each part must do and why. Literate programming does not add that thinking; it insists only that the thinking be written down in the one place where the code can be checked against it. What the method truly asked was not more labour but a rarer discipline—to explain a thing clearly *before* implementing it, and to keep the explanation true as the code moved—and that discipline was hard to sustain for a single reason: one author had to be narrator and engineer at once, holding both in step by hand, with no partner to catch the prose when it drifted from the code. The barrier was never the price of the writing. It was the loneliness of the writing. So the method was admired, and seldom used.

Literate Driven Development

A capable AI collaborator supplies the missing half. When the narrative and the code can be authored, revised, and kept synchronised *together*—the prose stating the intent, the code realising it, each continually checked against the other and against the world—the loneliness that made the discipline hard simply lifts, and its benefits, never really bought at a premium, compound freely. We call this practice **Literate Driven Development**: literate programming as the medium, an AI as the co-author, and the woven document as the single source of truth from which both the understanding and the executable follow. The prose is not documentation bolted onto a finished program; it is the place the program is thought, and a change is not finished until the story and the code agree.

The practice brings its own instruments, made for this way of working. weft[1], the literate metalanguage in which this book is composed, is built to be authored and read by human and artificial collaborators alike—prose and code fragments woven into one text that either kind of author can follow and revise. And a growing repertoire of *skills*—reusable, codified procedures the AI co-author can invoke—carries the recurring work of the craft, from keeping the prose and the code in agreement to running the verifications that check both against the world, so that the collaboration is guided by method rather than left to improvisation. The tools are new; what they make practical is the old discipline, finally without its loneliness.

[1] weft is open-source software: https://github.com/temo-pacheco/weft.

Four Things It Did

This book is the demonstration. In building kcc, Literate Driven Development did four things, and the reader can check each of them against the chapters that follow.

It eliminated uncertainty. Every decision is written down before it is coded, in prose that must name the clause of the Standard it obeys. Every change is then verified three ways: against the ISO standard the prose cites, against a reference compiler run over a large corpus of programs, and against the compiler's own ability to recompile itself unchanged. Nothing is taken on faith. The document records *why*; the test suites record *that*; and when the two disagree, the disagreement is visible instead of latent.

It managed complexity. A compiler is a large machine of interlocking parts—lexer, preprocessor, parser, type system, intermediate representation, optimiser, register allocator, code generator, object writer—and the classic failure mode is that no one can hold the whole of it in mind at once. The literate web dissolves that: each concept is told as its own story, at its own natural size, while the narrative keeps the whole in view. More than once a defect here spanned three of those layers at the same time; the way to it was to *read the story*, not to grep the code.

It was fast and cheap. Work that would occupy a team for weeks landed in hours: an optimiser with constant folding, dead-code and common-subexpression elimination, and the promotion of local variables into registers; a full register allocator with a dedicated floating-point bank; the difficult tail corners of C17—`_Generic`, correctly typed integer constants, `volatile` semantics, `long double`. The AI moves through the literate structure—its machine-readable map of fragments, its reverse mapping from a line of generated assembly back to the prose that produced it—so that each change stays local, and each local change stays testable.

It produced something useful. What you hold is not a toy. kcc self-hosts. It passes its full suite. It ships with a user manual for the person who wants to *use* it and a porting playbook for the person who wants to *extend* it to a new architecture. It is documentation and program at once, and—this is the point—neither is free to drift from the other.

Toward Works of Art

Knuth wanted programs that were works of literature. Literate Driven Development lets us reach for that ambition again, and reach further. The union of the literate ideal with an AI's tireless attention to *both* the prose and the proof makes it possible to build real, production-grade software that reads like a book and behaves like an engineering artifact—to stop treating correctness and beauty as a trade-off and to pursue them at once. That is the thesis of this book, and kcc is offered as its proof.

How to Read This Book

The chapters immediately after the table of contents are arranged for the reader who wants to arrive quickly. A *user manual* (Chapter 1) explains how to obtain, build, and run the compiler. A *porting playbook* (Chapter 2) explains how to add a new target architecture, so that the effort of supporting many machines becomes transcription rather than invention. And the *output-routing* chapter—the "assembly" of the program—shows how the fragments told throughout the book are finally gathered into the physical files the machine requires. After this orientation the book proper begins, and proceeds, chapter by chapter, from the language standard down to the running binary.

Contents

Acknowledgments vi

Preface vii

I. Orientation 1

1. User Manual 5
- 1.1. What kcc Is . . . 5
- 1.2. Requirements . . . 5
- 1.3. Building kcc . . . 5
- 1.4. Your First Program . . . 6
- 1.5. Command Reference . . . 6
- 1.6. Compiling and Linking in Separate Steps . . . 7
- 1.7. Warnings . . . 7
- 1.8. Debug Information . . . 7
- 1.9. Inline Assembly . . . 7
- 1.10. Generating x86-64 Code . . . 8
- 1.11. Reading Diagnostics . . . 9
- 1.12. Looking Inside the Compiler . . . 9
- 1.13. What kcc Supports . . . 9
- 1.14. Compiling a Real Program: Lua . . . 9
  - 1.14.1. On macOS (native ARM64) . . . 10
  - 1.14.2. On Linux (x86-64) . . . 10
- 1.15. Compiling SQLite . . . 10
  - 1.15.1. On macOS (native ARM64) . . . 11
  - 1.15.2. On Linux (x86-64) . . . 11
- 1.16. Playing DOOM . . . 11
- 1.17. Booting a Linux Kernel Compiled by kcc . . . 12
  - 1.17.1. The build image . . . 12
  - 1.17.2. Bootstrapping kcc for Linux . . . 13
  - 1.17.3. Compiling and linking the kernel . . . 15
  - 1.17.4. An initramfs with login and a shell . . . 15
  - 1.17.5. Booting it . . . 16
- 1.18. Building This Book . . . 17
- 1.19. For Contributors . . . 17
- 1.20. Troubleshooting . . . 17

2. Retargeting kcc: The Porting Playbook 19
- 2.1. The Principle: One Front End, Many Back Ends . . . 19
- 2.2. Two Ways to Reach Metal . . . 19
- 2.3. The Target Interface . . . 20
- 2.4. The Register Allocator Is Already Portable . . . 21
- 2.5. The Porting Playbook . . . 21
- 2.6. A Target Skeleton . . . 22
- 2.7. A Porting Checklist . . . 23
- 2.8. Testing a Cross Target . . . 23
- 2.9. What Stays Owned, What Is Borrowed . . . 23

3. Known Limitations 25
- 3.1. Language Features . . . 25
- 3.2. Standard Library and Headers . . . 26
- 3.3. Code Generation and Targets . . . 26
- 3.4. The Standing Rule . . . 26

The Assembly of the Program 27

II. Foundations 43

4. The Normative Corpus 45
- 4.1. Primary Standard . . . 45
- 4.2. Secondary References . . . 45
- 4.3. Citation Conventions . . . 45

5. Conformance Model 47
- 5.1. Hosted vs. Freestanding . . . 47
- 5.2. Translation Limits . . . 47
- 5.3. Implementation-Defined Behavior . . . 47
- 5.4. Undefined Behavior Policy . . . 47

III. The Machine 49

6. ARM64 Conventions 51
- 6.1. Register Assignment . . . 51
- 6.2. Apple Platform Constraints . . . 51
- 6.3. Common Macros . . . 51

7. System Calls 53
- 7.1. Syscall Number Constants . . . 53
- 7.2. Process Control . . . 53
- 7.3. File I/O . . . 53
  - 7.3.1. write . . . 53
  - 7.3.2. read . . . 53
  - 7.3.3. open . . . 53
  - 7.3.4. close . . . 54
- 7.4. Memory Management . . . 54
  - 7.4.1. mmap . . . 54
  - 7.4.2. lseek . . . 54
- 7.5. Process Creation . . . 54

8. Memory Management: The Arena 57
- 8.1. Design Rationale . . . 57
- 8.2. Chunk Layout . . . 57
- 8.3. Initialization . . . 57
- 8.4. Allocation: Fast Path . . . 58
- 8.5. Allocation: Slow Path . . . 58
- 8.6. Global State . . . 59

9. Data Structures 61
- 9.1. FNV-1a Hash Function . . . 61
- 9.2. Hash Table . . . 61
  - 9.2.1. Initialization . . . 61
  - 9.2.2. Lookup . . . 62
  - 9.2.3. Insertion . . . 63
- 9.3. Dynamic Vector . . . 64
  - 9.3.1. Initialization . . . 64
  - 9.3.2. Push . . . 65
  - 9.3.3. Element Access . . . 65
- 9.4. String Interning . . . 66
  - 9.4.1. Global State . . . 66

10. String Operations 67
- 10.1. String Length . . . 67
- 10.2. Memory Copy . . . 67
- 10.3. Memory Compare . . . 67

10.4. Memory Set . . . 67
10.5. String Compare . . . 68

11. Buffered I/O **69**
11.1. Buffer Layout . . . 69
11.2. Flush . . . 69
11.3. Put Character . . . 69
11.4. Put String . . . 70
11.5. Write N Bytes . . . 70
11.6. Print Unsigned Integer . . . 71
11.7. Stderr Output . . . 71

12. Diagnostic Engine **73**
12.1. Severity Levels . . . 73
12.2. Stderr Integer Output . . . 73
12.3. Core Emission . . . 73
12.4. Source Snippet and Caret . . . 74
12.5. Convenience Wrappers . . . 76
12.6. Error Count Query . . . 76
12.7. Reset . . . 76
12.8. Global State . . . 77
12.9. Diagnostic Tests . . . 77
12.10. Warning Tests . . . 78

13. Source File Management **81**
13.1. Design . . . 81
13.2. Opening a Source File . . . 81
13.3. Global State . . . 82

IV. The Frontend **83**

14. Source Encoding — Translation Phase 1 **85**
14.1. Trigraph Sequences . . . 85
14.2. Implementation . . . 85

15. Line Splicing — Translation Phase 2 **87**
15.1. Design . . . 87

16. Lexical Analysis **89**
16.1. Token Types . . . 89
16.2. Lexer Initialization . . . 90
16.3. Next Token . . . 90
16.4. Location Recording . . . 92
16.5. Identifier Scanner . . . 93
16.6. Number Scanner . . . 93
16.7. String Literal Scanner . . . 94
16.8. Character Literal Scanner . . . 95
16.9. Punctuator Scanner . . . 96
16.10. Token Dump . . . 99
16.11. Token Type Names . . . 100
16.12. Keyword Table Initialization . . . 103
16.13. Lexer Global State . . . 104

17. The Preprocessor **107**
17.1. Data Structures . . . 107
17.2. Initialization . . . 107
17.3. The Preprocessor Loop . . . 108
17.4. Directive Dispatch . . . 109
17.5. Context Stack . . . 111
17.6. Helper Functions . . . 113
17.7. Preprocessed Output (`-E` Mode) . . . 115
17.8. Global State . . . 115
17.9. Macro Definition (`#define`) . . . 117
17.10. Macro Undefine (`#undef`) . . . 119
17.11. The `push_macro` and `pop_macro` Pragmas . . . 119
17.12. Macro Expansion . . . 121
17.12.1. Argument Prescan . . . 125
17.12.2. Parameter Substitution . . . 127
17.13. Predefined Macros . . . 132
17.14. Conditional Compilation . . . 135
17.15. Expression Evaluation for `#if` . . . 141
17.16. File Inclusion (`#include`) . . . 148
17.16.1. The include search path . . . 150
17.16.2. Built-in standard headers . . . 152
17.16.3. Hosted library headers (shipped as files) . . . 154
17.17. Error Directive . . . 159

18. String Literal Processing **161**
18.1. Escape Sequence Table . . . 161
18.2. Implementation . . . 161
18.3. Processing All Tokens . . . 163

19. String Literal Concatenation **165**
19.1. Algorithm . . . 165
19.2. Data . . . 166

20. Recursive-Descent Parser **167**
20.1. AST Node Layout . . . 167
20.2. AST Node Allocation . . . 168
20.3. Token Consumption Helpers . . . 168
20.4. Error Recovery . . . 169
20.5. Typedef Disambiguation . . . 170
20.6. Translation Unit . . . 172
20.7. AST Dump . . . 173
20.8. Dump Helpers . . . 177
20.9. Parser Data . . . 178

21. Declaration Parsing **181**
21.1. Declaration . . . 181
21.2. Declaration Specifiers . . . 184
21.3. Declarator . . . 190
21.4. Direct Declarator . . . 191
21.5. Abstract Declarator . . . 195
21.6. Struct and Union . . . 198
21.7. Enum . . . 200
21.8. Type Name . . . 201
21.9. Is Type Token . . . 202
21.10. Initializer . . . 203
21.11. Static Assert . . . 205
21.12. Declaration Data . . . 205

22. Expression Parsing **207**
22.1. Precedence Table . . . 207
22.2. Expression Entry Point . . . 207
22.3. Assignment Expression . . . 208
22.4. Conditional Expression . . . 208
22.5. Binary Expression (Precedence Climbing) . . . 209
22.6. Unary Expression . . . 209
22.7. Cast Expression . . . 211
22.8. Postfix Expression . . . 212
22.9. Primary Expression . . . 215
22.10. Floating Constants . . . 220

22.11 Argument List . . . 223
22.12 Expression Data . . . 223

23. Statement Parsing 225
23.1. Statement Dispatch . . . 225
23.2. Compound Statement . . . 229
23.3. If Statement . . . 231
23.4. Switch Statement . . . 231
23.5. While Statement . . . 232
23.6. Do-While Statement . . . 232
23.7. For Statement . . . 233
23.8. Goto Statement . . . 234
23.9. Return Statement . . . 234
23.10 Labeled Statements . . . 234
23.11 Statement Data . . . 235

V. Semantics 237

24. Type System 239
24.1. Type Descriptor Layout . . . 239
24.2. Type Initialization . . . 239
24.3. Type Constructors . . . 242
24.3.1. Pointer Type . . . 242
24.3.2. Array Type . . . 243
24.3.3. Function Type . . . 243
24.3.4. Struct/Union Type . . . 243
24.3.5. Enum Type . . . 244
24.4. Specifier Bitmask Resolution . . . 245
24.5. Type Predicates . . . 247
24.6. Integer Conversion Rank . . . 248
24.7. Type Dump . . . 249
24.8. Type Data . . . 250

25. Symbol Table 253
25.1. Symbol Record Layout . . . 253
25.2. Initialization . . . 253
25.3. Scope Management . . . 253
25.4. Ordinary Name Space . . . 254
25.5. Tag Name Space . . . 256
25.6. Label Scope . . . 257
25.7. Symbol Table Data . . . 258

26. Semantic Analysis 259
26.1. Entry Point . . . 259
26.2. Type Resolution . . . 259
26.3. Dump Types . . . 262
26.4. Semantic Analysis Data . . . 263
26.5. Declaration Checking . . . 264
26.6. Statement Checking . . . 271
26.7. Expression Type-Checking . . . 274
26.7.1. Identifier Expression . . . 277
26.7.2. Reaching the End of a Non-Void Function . . . 281
26.7.3. Literal Expressions . . . 283
26.7.4. Binary Expression . . . 284
26.7.5. Unary Expression . . . 286
26.7.6. Assignment Expression . . . 287
26.7.7. Function Call . . . 289
26.7.8. Member Access . . . 290
26.7.9. Cast Expression . . . 292
26.7.10 Sizeof Expression . . . 292

27. Type Conversions 295
27.1. Integer Promotions . . . 295
27.2. Usual Arithmetic Conversions . . . 295
27.3. Array and Function Decay . . . 296
27.4. Lvalue Conversion . . . 297
27.5. Implicit Conversion Check . . . 297

VI. Intermediate Representation 299

28. Intermediate Representation 301
28.1. IR Instruction Layout . . . 301
28.2. IR Initialization . . . 302
28.3. Allocating IR Nodes . . . 303
28.4. Emitting Instructions . . . 304
28.5. Virtual Register Allocation . . . 305
28.6. Switching Basic Blocks . . . 305
28.7. IR Module Data . . . 305

29. IR Builder 307
29.1. IR Builder Entry Point . . . 307
29.1.1. Global Variable Initializers . . . 308
29.2. Building a Function . . . 317
29.3. Building Parameters . . . 319
29.4. Labels and Goto . . . 321
29.5. Building Statements . . . 322
29.6. Building Return . . . 326
29.7. String Constants . . . 328
29.8. IR Dump . . . 329
29.8.1. Dumping a Single Instruction . . . 330
29.8.2. Short Type Name . . . 334
29.9. Building Expressions . . . 335
29.9.1. Statement Expression . . . 337
29.9.2. Integer Literal . . . 337
29.9.3. Sizeof Expression . . . 341
29.9.4. Array Subscript . . . 343
29.9.5. Struct Member Access . . . 349
29.9.6. Bit-field Access . . . 353
29.9.7. Identifier (Variable Load) . . . 361
29.9.8. Binary Expression . . . 363
29.9.9. Unary Expression . . . 373
29.9.10 Increment and Decrement . . . 376
29.9.11 Assignment . . . 381
29.9.12 Function Call . . . 388
29.9.13 Variadic Function Definitions . . . 392
29.9.14 String Literal . . . 395
29.9.15 Conditional Expression . . . 395
29.10 Building Cast Expressions . . . 397
29.11 Building Variable Declarations . . . 398
29.11.1 Static Locals . . . 411
29.12 Building Control Flow . . . 416
29.12.1 If Statement . . . 416
29.12.2 While Statement . . . 417
29.12.3 Do-While Statement . . . 418
29.12.4 For Statement . . . 419
29.12.5 Switch Statement . . . 421
29.12.6 Break and Continue . . . 429
29.12.7 Pointer Arrays . . . 431
29.12.8 Initializer Lists . . . 432
29.12.9 Static Locals . . . 432

30. IR Validation **435**
30.1. What the Verifier Checks . . . 435
30.2. The Verifier . . . 435

VII. Optimization **437**

31. Optimization Passes **439**
31.1. Pass Manager . . . 439
31.2. Per-Function Iteration . . . 439

32. Peephole Optimization **441**
32.1. Constant Folding . . . 441
32.1.1. Building the Constant Map . . . 441
32.1.2. Folding Binary Operations . . . 442
32.1.3. Arithmetic Dispatch . . . 443
32.2. Strength Reduction and Algebraic Simplification . . . 444

33. Dead Code Elimination **447**
33.1. Dead Code Elimination . . . 447
33.1.1. Setup and Bitmap Allocation . . . 447
33.1.2. Mark Pass . . . 447
33.1.3. Sweep Pass . . . 448

34. Common Subexpression Elimination **451**

VIII. The Backend **455**

35. Code Generation **457**
35.1. Strategy . . . 457
35.2. Constants . . . 457
35.3. Initialization . . . 457
35.4. Pre-register Global Sizes . . . 458
35.4.1. Read-Only Const Globals . . . 462
35.5. Compile All Functions . . . 466
35.6. Branch Patching . . . 468
35.7. String Literal Codegen . . . 469
35.8. CodeGen Data . . . 473
35.9. Register Allocation . . . 474
35.10 Spill-Slot Colouring . . . 484
35.11 Compile Function . . . 486
35.12 Vreg Slot Offset . . . 493
35.13 AAPCS64 Argument Marshaling . . . 496
35.14 Instruction Dispatch . . . 497
35.14.1 Loading Constants . . . 499
35.14.2 Arithmetic and Bitwise Operations . . . 500
35.14.3 Comparison Operations . . . 503
35.14.4 Type Conversions . . . 504
35.14.5 Floating-Point Operations . . . 505
35.14.6 Memory Operations . . . 506
35.14.7 Control Flow . . . 510
35.14.8 Function Calls and External References . . . 512
35.15 Extended Inline Assembly . . . 516

36. ARM64 Encoding **519**
36.1. Code Buffer . . . 519
36.2. Arithmetic Encoding (Register) . . . 520
36.3. Arithmetic Encoding (Immediate) . . . 521
36.4. Logical Encoding . . . 521
36.5. Compare and Conditional Set . . . 522
36.6. Memory Encoding . . . 523
36.7. Floating-Point Encoding . . . 524
36.8. Branch Encoding . . . 526
36.9. System Instructions . . . 527
36.10 ADRP and Page-Offset Encoding . . . 527
36.11 Code Buffer Data . . . 527

37. Textual Assembly Emission **529**
37.1. The Output Buffer . . . 529
37.2. Rendering the Text Section . . . 531
37.3. Writing the File . . . 537

38. A Second Target: x86-64 by Assembly Emission **539**
38.1. Strategy: an All-Spill Frame . . . 539
38.2. Target State and Assembly Vocabulary . . . 539
38.2.1. Vocabulary for Control Flow and Calls (Rung 3) . . . 540
38.2.2. Vocabulary for Floating Point (Rung 4) . . . 541
38.3. Two Object-Format Flavours: Darwin and Linux . . . 542
38.4. Text-Buffer Emission Helpers . . . 542
38.5. Instruction Selection . . . 543
38.5.1. Binary, Unary, and Shift Shapes . . . 543
38.5.2. Division and Remainder . . . 544
38.5.3. Width-Changing Conversions . . . 545
38.5.4. Memory Operations . . . 545
38.5.5. Comparison, Label, and Call Helpers (Rung 3) . . . 546
38.5.6. The Dispatch . . . 547
38.6. The Frame: Pre-scan, Prologue, Epilogue . . . 553
38.7. Spilling Incoming Parameters . . . 554
38.8. Walking the Module . . . 554
38.9. Emitting Global Variable Storage . . . 556
38.10 Static Locals . . . 557
38.11 Read-Only Initialised Data: const Globals . . . 558
38.12 Registering the Module Initialiser . . . 561
38.13 Emitting String-Literal Data . . . 562
38.14 Rendering to a File . . . 562
38.15 Rung One in Retrospect . . . 563

39. The Linux Runtime **565**
39.1. Calling Convention and Syscall ABI . . . 565
39.2. The ELF Entry Point . . . 565
39.3. Process Control . . . 565
39.4. File I/O . . . 566
39.5. Memory Mapping . . . 566
39.6. Assembling the Runtime . . . 567
39.7. Verification: A Cross-Compiled, Native Linux Run . . . 567
39.8. kcc Self-Hosts on Linux . . . 568
39.9. Verification: The Automated Self-Host . . . 568

40. Mach-O Emission **571**
40.1. Constants . . . 571
40.2. Output Buffer Management . . . 571
40.3. Finalize and Write . . . 573
40.3.1. Layout Computation . . . 574
40.3.2. Header and Load Commands . . . 575

40.3.3. Code, Relocations, and Symbol Remapping . . . 579
40.3.4. Symbol Table and String Table . . . 582
40.4. Mach-O Data . . . 584

41. Linker Information **585**
41.1. Data Structures . . . 585
41.2. Initialization . . . 585
41.3. Adding Symbols . . . 586
41.4. Adding Relocations . . . 586
41.5. External Symbol Lookup . . . 586
41.6. Global Symbol Registration . . . 587
41.7. Linker Data . . . 588

42. DWARF Debug Information **589**
42.1. Constants . . . 589
42.2. Byte Emitters . . . 589
42.3. Building the Sections . . . 590
42.4. Mach-O Emission . . . 593
42.5. Global State . . . 595

43. Compilation Driver **597**
43.1. Full Compilation Pipeline . . . 597
43.2. Compiling Several Translation Units . . . 599
43.3. Automatic Linking . . . 605
43.4. Multi-file Tests . . . 610
43.5. End-to-End Driver Tests . . . 610

IX. The Driver **613**

44. The Driver **615**
44.1. Entry Point . . . 615
44.2. String Constants . . . 619

X. Verification **621**

45. Test Harness **623**
45.1. Runner Entry Point . . . 623
45.2. Test Assertion Helpers . . . 624
45.3. Runner Data . . . 624

46. Conformance Tests **627**
46.1. Keyword Recognition . . . 627
46.2. Numeric Literals . . . 627
46.3. String and Character Literals . . . 627
46.4. Punctuators . . . 628
46.5. Comments . . . 628
46.6. Trigraph Replacement . . . 629
46.7. Line Splicing . . . 629
46.8. Test Registration Table . . . 629
46.9. Test Source Data . . . 630

47. Code Generation Tests **631**
47.1. Test Runner . . . 631
47.2. Results . . . 632

48. Self-Hosting: The Arena in C **633**
48.1. Interface and Chunk Layout . . . 633
48.2. Initialization . . . 633
48.3. Allocation . . . 633
48.4. Assembling the Translation Unit . . . 634
48.5. Verification . . . 634

49. Self-Hosting: A Tokenizer in C **635**
49.1. Token Codes . . . 635
49.2. State and the Keyword Table . . . 635
49.3. Character Classes and Keyword Lookup . . . 635
49.4. The Cursor . . . 636
49.5. Scanning One Token . . . 636
49.6. Tokenizing a Buffer . . . 637
49.7. The Self-Test . . . 637
49.8. Assembling the Translation Unit . . . 637
49.9. Verification . . . 638

50. Self-Hosting: The Vector in C **639**
50.1. Interface and Layout . . . 639
50.2. Initialization . . . 639
50.3. Push . . . 639
50.4. Element Access . . . 639
50.5. Assembling the Translation Unit . . . 640
50.6. Verification . . . 640

51. Self-Hosting: The String Primitives in C **641**
51.1. The Byte-Exact Contract . . . 641
51.2. Length . . . 641
51.3. Copy and Fill . . . 641
51.4. Comparison . . . 641
51.5. Assembling the Translation Unit . . . 642
51.6. Verification . . . 642

52. Self-Hosting: The Hash Table in C **643**
52.1. The Byte-Exact Contract . . . 643
52.2. FNV-1a . . . 643
52.3. Initialization . . . 643
52.4. Lookup . . . 643
52.5. Growth . . . 644
52.6. Insertion . . . 644
52.7. Assembling the Translation Unit . . . 644
52.8. Verification . . . 644

53. Self-Hosting: The String Interner in C **647**
53.1. State and Contract . . . 647
53.2. Verification . . . 647

54. Self-Hosting: Buffered Output in C **649**
54.1. State and the Flush Contract . . . 649
54.2. Flush and Put . . . 649
54.3. Write, Format, and Error . . . 649
54.4. Assembling the Translation Unit . . . 650
54.5. Verification . . . 650

55. Self-Hosting: The Source Manager in C **651**
55.1. Opening the File . . . 651
55.2. Phase 1: Trigraphs . . . 652
55.3. Phases 2 and 3: Line Splicing and Comment Removal . . . 652
55.4. Assembling the Translation Unit . . . 653
55.5. Verification . . . 653

56. Self-Hosting: The Diagnostic Engine in C **655**
56.1. State and Dependencies . . . 655
56.2. The Caret . . . 655
56.3. Emit and Wrappers . . . 655

56.4. Assembling the Translation Unit 656
56.5. Verification 656

57. Self-Hosting: The Symbol Table in C 657
57.1. Layout and State 657
57.2. Scopes 657
57.3. Ordinary Names 657
57.4. Tags 658
57.5. Labels 658
57.6. Assembling the Translation Unit 658
57.7. Verification 659

58. Self-Hosting: The Standard Conversions in C 661
58.1. Reading a Type 661
58.2. Integer Promotions and Usual Arithmetic . 661
58.3. Decay and Lvalue 661
58.4. Implicit Conversion Check 662
58.5. Assembling the Translation Unit 662
58.6. Verification 662

59. Self-Hosting: The Type System in C 663
59.1. The Descriptor 663
59.2. Initialising the Canonical Descriptors 663
59.3. Constructors 664
59.4. Aggregate Constructors 664
59.5. Specifier Bitmask Resolution 665
59.6. Predicates and Rank 665
59.7. Dump 666
59.8. Assembling the Translation Unit 666
59.9. Verification 666

60. Self-Hosting: The Optimizer in C 669
60.1. Reading the IR 669
60.2. Constant Folding 670
60.3. Strength Reduction 671
60.4. Common Subexpression Elimination 672
60.5. Store-to-Load Forwarding 673
60.6. Dead-Code Elimination 674
60.7. The Pass Manager 675
60.8. Assembling the Translation Unit 675
60.9. Verification 675

61. Self-Hosting: The Lexer Tables in C 677
61.1. Keyword Registration 677
61.2. Token Names 677
61.3. Assembling the Translation Unit 678
61.4. Verification 678

62. Self-Hosting: The Linker Information in C 679
62.1. Tables and State 679
62.2. Initialization 679
62.3. Adding Symbols and Relocations 679
62.4. External and Global Symbols 680
62.5. Assembling the Translation Unit 680
62.6. Verification 680

63. Self-Hosting: The Statement Parser in C 683
63.1. Nodes, Tokens, and the Parser API 683
63.2. The Dispatcher 684
63.3. Expression Continuation 685
63.4. Inline Assembly 685
63.5. Compound Statements and the Control Forms 686
63.6. Jumps and Labels 688
63.7. Assembling the Translation Unit 688
63.8. Verification 688

64. Self-Hosting: The Mach-O Writer in C 691
64.1. Buffer, State, and the Primitive Writers . . 691
64.2. Layout Computation 692
64.3. Header, Segment, and Sections 693
64.4. Auxiliary Load Commands 694
64.5. Code, Relocations, Symbols, and Flush . . . 695
64.6. Assembling the Translation Unit 696
64.7. Verification 696

65. Self-Hosting: The Expression Parser in C 697
65.1. Nodes, Tokens, and the Parser API 697
65.2. Precedence, Comma, Assignment, Conditional 698
65.3. Unary, Cast, Postfix 698
65.4. Primary Expressions 701
65.5. Literal Conversion 702
65.6. Assembling the Translation Unit 704
65.7. Verification 704

66. Self-Hosting: The Parser Spine in C 707
66.1. Nodes, State, and the Tag Table 707
66.2. Allocation, Token Helpers, Errors 708
66.3. Typedef Scopes and the Translation Unit . . 708
66.4. The AST Dumper 709
66.5. Assembling the Translation Unit 711
66.6. Verification 711

67. Self-Hosting: The Compilation Driver in C 713
67.1. State, Strings, and the Pipeline 713
67.2. Argument Helpers and Object Naming . . . 715
67.3. Compiling Several Files 716
67.4. Environment, SDK, and Spawning 717
67.5. Automatic Linking 718
67.6. Assembling the Translation Unit 719
67.7. Verification 719

68. Self-Hosting: The Declaration Parser in C 721
68.1. Nodes, Tokens, and Specifier Bits 721
68.2. Attributes and `typeof` 722
68.3. Declaration Specifiers 723
68.4. Declarators 725
68.5. Abstract Declarators 727
68.6. Struct, Union, and Enum 728
68.7. Type Name, Predicate, Initializer, and Dispatch 729
68.8. Assembling the Translation Unit 732
68.9. Verification 732

69. Self-Hosting: The DWARF Emitter in C 733
69.1. Buffer State, Constants, and Byte Emitters 733
69.2. The Abbreviation Table 734
69.3. The Info Section 734
69.4. The Line-Number Program 735
69.5. The Entry Point and Mach-O Emission . . . 735
69.6. Assembling the Translation Unit 736
69.7. Verification 736

70. Self-Hosting: The ARM64 Encoder in C 739
70.1. Externs, Constants, and Module State . . . 739
70.2. Instruction Encoders . . . 739
70.3. The Code Buffer . . . 741
70.4. The Text Buffer . . . 742
70.5. Rendering the Text Section . . . 742
70.6. Globals, Module Init, and File Output . . . 744
70.7. Assembling the Translation Unit . . . 744
70.8. Verification . . . 745

71. Self-Hosting: The Semantic Analyzer in C 747
71.1. Nodes, Types, Symbols, and Constants . . . 747
71.2. Entry Point and Type Resolution . . . 749
71.3. Statement Checking . . . 750
71.4. Declaration Checking and Aggregate Layout 751
71.5. Expression Type-Checking . . . 754
71.6. Type Dumping and Assembly . . . 761
71.7. Verification . . . 762

72. Self-Hosting: The x86-64 Back End in C 763
72.1. State, Constants, and the AT&T Vocabulary 763
72.2. Symbol and Operand Helpers . . . 765
72.3. Instruction Shapes . . . 765
72.4. The Dispatch . . . 767
72.5. Frame, Parameters, and the Function Walk 775
72.5.1. Dead static-function elimination . . 775
72.6. Global, Static, and Read-Only Data . . . 778
72.7. Read-Only Byte Images . . . 781
72.8. General Static Initializers . . . 783
72.9. Module Init, String Literals, and the Render Entry . . . 785
72.10. Verification . . . 786

73. Self-Hosting: The Preprocessor in C 789
73.1. State, Constants, and Externs . . . 789
73.2. Initialization, the Main Loop, and Contexts 790
73.3. Helpers: Skip-Line and Rest-of-Line . . . 792
73.4. Directive Dispatch . . . 793
73.5. `#define`, `#undef`, and Pragmas . . . 793
73.6. Macro Expansion . . . 794
73.7. Argument Prescan and Substitution . . . 797
73.8. Predefined Macros and Command-Line Defines . . . 800
73.9. Conditionals and the `#if` Evaluator . . . 801
73.10. File Inclusion and Built-in Headers . . . 804
73.11. Phase 5 and 6: String-Literal Escapes and Concatenation . . . 807
73.12. The Compiler-Resident Built-in Headers . . 808
73.13. Assembling the Translation Unit . . . 812
73.14. Verification . . . 812

74. Self-Hosting: The ARM64 Code Generator in C 813
74.1. State, Constants, Externs . . . 813
74.2. Initialization and the Frame Constants . . . 815
74.3. Read-Only Constant Globals . . . 815
74.4. Pre-registering Globals and Functions . . . 817
74.5. Register Allocation . . . 818
74.6. Spill-Slot Colouring . . . 821
74.7. Frame, Slots, Transfers, and Marshalling . . 821
74.8. The Function Driver . . . 823
74.9. Branch Patching and String Literals . . . 824
74.10. Per-Opcode Instruction Selection . . . 825
74.11. Global Reference to a Defined Symbol, and Extended Asm . . . 829
74.12. Verification . . . 830

75. Self-Hosting: The IR Builder in C 831
75.1. State, Constants, Externs . . . 831
75.2. Initialisation and Node Allocation . . . 834
75.3. The Builder Spine . . . 835
75.4. Expressions . . . 850
75.5. Member Access and Bit-fields . . . 854
75.6. Identifiers, Operators, and Unary Expressions 858
75.7. Aggregates and Declarations . . . 867
75.8. Control Flow . . . 873
75.9. Assignment, Calls, Casts . . . 880
75.10. Dump and Validation . . . 894
75.11. Composition and Verification . . . 896

76. Self-Hosting: The Entry Point in C 899
76.1. Entry Point and Mode Dispatch . . . 899
76.2. Composition and Verification . . . 903

77. Self-Hosting: The Lexer in C 905
77.1. State, Init, and the Scanners . . . 905
77.2. Verification . . . 908

78. Differential Testing 911
78.1. The Runner . . . 911
78.2. Arithmetic and Bit Manipulation . . . 912
78.3. Control Flow . . . 912
78.4. Functions and Recursion . . . 913
78.5. Pointers, Arrays, and Structs . . . 913
78.6. Struct Layout . . . 915
78.7. Operators and Edge Cases . . . 917
78.8. Deeper Combinations . . . 924
78.9. Language Features . . . 924
78.10. Integer Width . . . 927
78.11. Known Divergences . . . 928
78.12. Results . . . 928

79. ABI Interoperability Testing 929

A. Development Roadmap 931
A.1. Phase 0 — Project Skeleton . . . 931
A.2. Phase 1 — Data Structures and Lexer Foundation . . . 931
A.3. Phase 2 — Complete Lexer and Diagnostics 931
A.4. Phase 3 — Preprocessor . . . 932
A.5. Phase 4 — Parser . . . 932
A.6. Phase 5 — Type System and Semantic Analysis . . . 932
A.7. Phase 6 — Intermediate Representation and Optimization . . . 933
A.8. Phase 7 — ARM64 Backend . . . 933
A.9. Phase 8 — Self-Hosting . . . 934
A.10. Phase 9 — Production Compiler . . . 936
A.10.1. Reprioritized Go-Forward Plan . . . 937
A.10.2. Optional Features — The Four Conditional Headers . . . 939
A.10.3. Stage 9a — Complete Type System 940
A.10.4. Stage 9b — Full ABI Compliance . . 941
A.10.5. Stage 9c — Register Allocation . . . 941

A.10.6. Stage 9d — Optimization 942
A.10.7. Stage 9e — Tooling and Diagnostics 942
A.11. Summary 943
A.12. Production Readiness 943

B. Roadmap: The Road to a Second Target 945
B.1. The Guiding Principle 945
B.2. Milestones at a Glance 945
B.3. M1 — Finish the C17 Frontend 945
B.4. M2 — Harden the IR Verifier 946
B.5. M3 — Systematic Conformance Battery . . 946
B.6. M4 — Broaden the IR Optimizer 946
B.7. M5 — Write-Protect Read-Only Data (Make Stray Writes Fault) 947
B.8. M6 — Carve the Target Interface 948
B.9. M7 — The Second Target (Hand-off) . . . 949
B.9.1. Two Milestones for Linux 949
B.10. Summary 949

The compiler's version is defined once, here, and expanded into every place the build emits it:

⟨ *version* ? ⟩ ≡
```
1.0.0
```
□

? · used in 38i, 625, 905

Part I.

Orientation

Before the book builds the compiler from the language standard down to the running binary, three chapters orient the reader who wants to arrive quickly: how to *use* kcc, how to *extend* it to a new architecture, and how its many fragments are *assembled* into the physical files the machine requires.

1. User Manual

This appendix is the practical guide to *using* kcc: how to obtain and build it, how to compile and run a program, what every command-line option does, how to read its diagnostics, and what it does and does not yet support. It assumes only that you are comfortable at a terminal; it does not assume you have read the rest of this book.

1.1. What kcc Is

kcc is a C compiler for the C17 language. It takes C source files and produces native machine code—either a relocatable object file (`.o`) or a finished executable that you can run directly. It is built from scratch: it has its own preprocessor, parser, type checker, optimizer, register allocator, and code generator, and it emits Mach-O object code itself rather than delegating to another assembler. Its target is **Apple silicon**: 64-bit ARM (AArch64) running macOS. The compiler produced by this book, and every example in this manual, runs on that platform.

1.2. Requirements

To *build* kcc from source you need:

- A Mac with Apple silicon (an M-series processor) running macOS.
- The Xcode Command Line Tools, which provide the system assembler, the linker `ld`, and the macOS SDK. Install them with `xcode-select --install` if you have not already.
- `weft`, the literate-programming tool that turns the `.weft` sources into assembly.
- `make`, which drives the project's build recipes; it comes with the Xcode Command Line Tools listed above.
- Optionally, `lualatex`, only if you want to typeset this book itself into a PDF. It is not needed to build or use the compiler.

To *use* the compiler once it is built you need only the Mac and the Xcode Command Line Tools: kcc invokes the system linker to produce executables and reads the SDK to find the C runtime startup, so the Command Line Tools must be present.

1.3. Building kcc

From the top of the project, a single command builds the compiler:

```
make build
```

This tangles the literate sources into assembly, assembles each into an object file, and links them into the compiler binary at `gen/kcc`. That path is the compiler; everything below refers to it as `kcc` for brevity. To run it from anywhere, either use the full path `gen/kcc` or copy the binary onto your `PATH`, for example:

```
cp gen/kcc /usr/local/bin/kcc
```

There is nothing to configure: kcc has no configuration file and no environment variables to set. Its entire behaviour is controlled by the command-line options described in §1.5.

To run the complete verification suite in one step:

```
make ci
```

This runs the fast native checks first — the unit tests, the differential tests against `clang`, ABI interop, end-to-end compile-and-run, diagnostics, warnings, and the code-generation suite — and finishes with the self-host milestones `meta-a` and `meta-c`. The last two cross-compile kcc to Linux and run it there, so they need Docker and `qemu-user`; the individual targets (for example `make difftest`) can be run on their own when that toolchain is not available.

1.4. Your First Program

The bootstrap tangles a first program for you, so it is ready the moment the compiler is built. Its source is this very fragment, emitted to `gen/hello.c`:

⟨ *Tutorial: hello world* 6 ⟩ ≡

```
#include <stdio.h>

int main(void) {
    printf("Hello from kcc!\n");
    return 0;
}
```

□

6 · used in 38j

Compile and link it with the compiler you just built, then run it:

```
./gen/kcc gen/hello.c -o gen/hello
./gen/hello
```

You should see `Hello from kcc!`. The same holds for any C file of your own; with kcc on your `PATH` the workflow is just as short, and with no `-o` option the executable is named `a.out`, following long tradition:

```
kcc hello.c -o hello      # or simply: kcc hello.c  ->  ./a.out
./hello
```

That is the whole workflow for a simple program: one command to build, one to run.

1.5. Command Reference

kcc is invoked as `kcc [mode] source.c ... [options]`. The first argument selects what the compiler does with the source; if it is not one of the recognised mode flags, kcc performs the default action—compile and link into an executable. Each mode and option is listed below.

Modes.

Invocation	What it does
`kcc a.c ... [options]`	compile *and link* into an executable (default)
`kcc -c a.c ... [options]`	compile to object file(s) (`.o`), no linking
`kcc -E a.c`	run only the preprocessor; print the result
`kcc --check a.c`	parse and type-check only; report errors, produce nothing
`kcc --dump-tokens a.c`	print the token stream from the lexer
`kcc --dump-ast a.c`	print the abstract syntax tree from the parser
`kcc --dump-types a.c`	print the resolved types
`kcc --dump-ir a.c`	print the optimized intermediate representation

The default mode and the `-c` mode both accept several source files at once and compile each; the default mode then links all of the resulting objects together. The inspection modes (`-E` and the `--dump` family, and `--check`) take a single source file and print their result to standard output, which makes them convenient to pipe into a pager or a file.

Options.

Option	Effect
`-o` *name*	name the output; default `a.out` when linking, or the object name when compiling with `-c`
`-I`*dir*	add *dir* to the `#include` search path (attached `-Idir` or separated `-I dir`)
`-D`*name*	define *name* as `1`; `-D`*name*`=`*val* defines it as *val* (also separated `-D` *name*)
`-Wall`	enable the warning family (§1.7)
`-g`	emit DWARF debug information for a debugger
`--emit-asm`	build the object through textual assembly: emit a `.s` file and run the platform as on it
`--target=x86_64`	generate x86-64 code instead of the native ARM64 (currently a scalar-only subset; see §1.10)

Options may appear in any order relative to the source files. A word that is not an option and not a mode flag is treated as a source file.

`--emit-asm` selects kcc's second code-emission strategy. The default back end encodes each instruction and writes the Mach-O object directly; with `--emit-asm` kcc instead renders the generated code as assembly text (`.s`) and invokes the system assembler to produce the object. The two produce equivalent objects for ordinary code, but the textual path is what makes *inline assembly* (§1.9) possible, because the direct encoder cannot assemble an arbitrary `asm` template. It works in both the default (link) and `-c` modes.

1.6. Compiling and Linking in Separate Steps

For a project of more than one file you can compile each translation unit to an object and link them later, exactly as with any C compiler:

```
kcc -c util.c   -o util.o
kcc -c main.c   -o main.o
kcc util.o main.o -o program
./program
```

Or you can let kcc do it all in one step by naming every source on the command line:

```
kcc util.c main.c -o program
```

1.7. Warnings

By default kcc reports only errors—conditions that make a program ill-formed. Passing `-Wall` additionally enables a family of warnings for suspicious but legal code: calling a function with no visible declaration, a local variable that is never used, a non-`void` function that can fall off its end without returning, a comparison that mixes signed and unsigned operands, and an implicit conversion between an integer and a pointer. Warnings do not stop compilation; they are advice.

1.8. Debug Information

The `-g` option makes kcc emit DWARF debugging information alongside the code, so that a debugger can map machine addresses back to functions and source lines. The information is function-granular: it records where each function begins, which is enough for a backtrace to name the right functions.

1.9. Inline Assembly

kcc accepts basic GNU inline assembly—`asm("`*`template`*`");` (also spelled `__asm__`, and with an ignored `volatile`, `inline`, or `goto` qualifier)—where *template* is a string literal containing one or more assembler instructions. Because the default back end encodes only the fixed instruction set kcc itself selects, inline assembly requires the textual back end: compile with `--emit-asm` (§1.5), which renders the code as `.s` and lets the system assembler assemble the template verbatim.

```
int main(void) {
    asm("mov x16, #1");   /* SYS_exit */
    asm("mov x0, #7");
    asm("svc #0x80");     /* exit(7) */
    return 0;
}
```

Extended assembly with operand constraints is supported in a common subset: register outputs (`=r`), inputs (`r`), read-write operands (`+r`), and integer immediates (`i`), with up to two operands and `%0`/`%1` (or `%w0`/`%x0`) placeholders. A `"memory"` or `"cc"` clobber is accepted; the compiler treats the asm as a barrier so cached values are re-read afterwards.

```
int y = 5, x;
asm("mov %0, %1" : "=r"(x) : "r"(y));   /* x = y  */
asm("add %0, %0, #1" : "+r"(x));        /* x += 1 */
```

Forms outside this subset—three or more operands, memory (`m`) or specific-register constraints, and `asm goto`—are rejected with a diagnostic rather than miscompiled.

1.10. Generating x86-64 Code

kcc's native output is ARM64 for Apple silicon, but it can also generate x86-64 code with the `--target=x86_64` option. This selects a second back end that renders the program as x86-64 assembly, assembles it with the system `as -arch x86_64`, and links a Mach-O executable that runs on Apple silicon under the Rosetta translator:

```
kcc --target=x86_64 add.c -o add
./add                  # runs under Rosetta; echo $? shows the exit status
```

Rosetta runs the code; kcc generates it. It is worth being precise about the division of labour, because "runs under Rosetta" can be misread as though Rosetta did the porting. It does not. *All* of the x86-64 code generation is kcc's: from the target-neutral IR it selects the x86-64 instructions, lays out the stack frame, and obeys the calling convention, then prints x86-64 assembly for the system `as` to encode into an object and `ld` to link—the same "emit assembly, hand it to the assembler" model `gcc` uses, and genuine code generation for a second instruction set, not a translation of ARM64 into x86-64. Rosetta enters only *after* the executable exists: kcc runs on Apple silicon (ARM64) but produces an x86-64 binary, and Rosetta—Apple's x86-64-to-ARM64 binary translator—is what lets that binary *execute* on the same machine, so its output and exit status can be checked against a mainstream x86-64 compiler without a separate Intel box. For the Linux/ELF binaries (§1.15, §1.17) that execution layer is QEMU instead; either way it is a runtime convenience, invisible to and independent of how the code was produced.

The x86-64 back end has grown well past its scalar beginnings. It handles integer, pointer, and *floating-point* arithmetic, the full set of operators and comparisons, control flow (`if`, `while`, `for`, `?:`, `&&`/`||`, and GNU computed `goto`), local variables and arrays, structs (addressed by member and passed by value in the classifiable System V cases), file-scope globals including `const` and symbol-address data, block-scoped `static` locals, string literals emitted as static data, extended inline assembly, and function calls following the System V AMD64 calling convention—recursion, stack-passed and floating/variadic arguments, and library calls such as `printf`. It is complete enough to compile kcc's own sources, a Linux kernel object, and programs like SQLite and Lua. A whole program such as

```
int printf(const char *, ...);
int count = 0;
int main(void) {
    for (int i = 1; i <= 5; i++)
        if (i % 2 == 0) { printf("%d even\n", i); count++; }
    printf("evens=%d\n", count);
    return count;
}
```

compiles, links, and runs in one step with output identical to a mainstream x86-64 compiler.

Targeting Linux (ELF). The same instruction selection can emit x86-64 assembly in the Linux/ELF dialect with `--target=x86_64-linux`: symbols are un-prefixed, read-only data goes in `.rodata`, zeroed globals use `.comm`, and an external reference resolves directly (for a static link) rather than through the Mach-O GOT. Because kcc runs on macOS—whose system assembler emits only Mach-O—this mode *writes the assembly to the output path and stops* rather than assembling it; you finish the build on a Linux host (or in a container):

```
kcc --target=x86_64-linux -c prog.c -o prog.s   # emits ELF-dialect asm
# then, on a Linux/x86-64 host:
as -o prog.o prog.s  &&  ld -o prog prog.o ...   # assemble + link + run
```

This "emit and stop" behaviour is a property of the *host*, not the compiler: a kcc built for and running on Linux/x86-64 has a native assembler and linker available and drives `as` and `ld` itself, finishing an executable in one command—which is exactly how the kernel-boot reproduction (§1.17) builds its objects.

What the x86-64 back end does *not* yet do is allocate registers: it uses an all-spill model, where every value lives in a stack slot, which is correct but makes the emitted code large and slow—an x86 register allocator is the main piece of remaining work. It also declines, cleanly rather than mis-lowering, a few by-value aggregate-argument shapes it cannot classify and most `__int128` operations; a program that hits one of these gets a clear refusal, never wrong code. Keep such a program on the default ARM64 path, which supports the whole language. That native path is entirely unaffected by this option—omitting `--target` (or writing `--target=arm64`) compiles for ARM64 exactly as before.

1.11. Reading Diagnostics

When kcc rejects a program it prints a diagnostic that names the file, the line and column, the nature of the problem, and—like a modern compiler—a caret pointing at the offending token in a copy of the source line. It does not stop at the first error: it recovers and continues so that a single run reports as many independent problems as it can find, which saves the edit-compile cycle of fixing them one at a time. A run that prints diagnostics and produces no output has failed; a run that prints nothing (or only warnings, under `-Wall`) and produces the requested file has succeeded.

1.12. Looking Inside the Compiler

The inspection modes are there for the curious and for anyone debugging a surprising result. `-E` shows exactly what the preprocessor produced—useful when a macro does not expand as expected. `--dump-ast` shows how the parser understood the program's structure. `--dump-ir` shows the optimized intermediate representation, the form on which the optimizer and code generator work; it is the clearest window on what code kcc will actually generate, and on what the optimizer did or did not simplify. None of these modes produces an object file; they print and exit.

1.13. What kcc Supports

kcc implements the C17 language broadly: the full expression and statement grammar, the complete type system with structs, unions, enums, bit-fields and function pointers, the preprocessor with macros and conditional inclusion, and the harder corners of the standard such as designated initializers, `_Generic` selection, `_Bool` normalization, `volatile` semantics, and correctly typed integer constants. It provides a small set of freestanding standard headers so that ordinary programs using `printf` and friends compile without an external installation. It optimizes—constant folding, dead-code elimination, common-subexpression elimination, and keeping locals in registers—and allocates both integer and floating values to registers.

Its two targets differ in maturity, and it is worth stating this plainly so you are not surprised. The native Apple-silicon (ARM64) back end supports the whole language. The x86-64 back end (§1.10; the framework behind it is Chapter 2) began as a scalar-only prototype but has outgrown that description: it now compiles kcc's own sources, the `init/main.o` of an unmodified Linux 6.6 kernel, and whole applications such as SQLite and Lua. Floating-point arithmetic (with the System V FP calling convention, including variadics), `const` globals in read-only data, block-scoped `static` locals, and symbol-address and string-literal initializers emitted as static data all work. What it does *not* yet do is allocate registers—it uses a simple all-spill model that is correct but emits verbose, slow code (an x86 register allocator is future work)—and it declines, rather than mis-lowers, a few by-value aggregate-argument shapes and most `__int128` operations. One tail corner of C17 remains on both targets: propagating `volatile` from a qualified *object* through member access (a member or bit-field declared `volatile` is already honoured). Nested designator chains, brace elision, and per-operation single-precision rounding for `float` arithmetic are fully supported. Appendix A tracks the state of every feature in detail.

Cross-compilation. Because both back ends lower the *same* target-neutral IR, kcc on Apple silicon is a genuine cross-compiler: `--target=x86_64-linux` makes an ARM64/macOS kcc generate x86-64/Linux code—a different instruction set *and* a different object format and ABI—not a translation of one assembly into another. The two targets complete the toolchain differently. For its native ARM64/Mach-O target kcc is self-contained: it emits machine code and links the executable itself. For x86-64 it follows the classic model that `gcc` itself uses—it generates assembly and hands it to the system `as` and `ld`; it does not carry its own x86 assembler or ELF linker. On macOS, whose assembler speaks only Mach-O, a Linux-dialect compile therefore writes the `.s` and stops (§1.10), to be finished on an x86-64 host or container. This is exactly the chain §1.17 and `make meta-c` exercise: kcc on ARM64 cross-compiles kcc's own source into a working Linux/x86-64 kcc, which in turn compiles the kernel object—and, run against its own source again, reproduces itself byte-for-byte.

1.14. Compiling a Real Program: Lua

Before the kernel, a gentler capstone: kcc compiles the **Lua 5.4.7** interpreter—about fifteen thousand lines of portable C across thirty-three translation units—into a working `lua` you can run scripts with, on both of kcc's targets. Lua is a good proof because it leans on the whole language: a bytecode virtual machine, a precise mark-and-sweep collector, `setjmp`-based error handling, its own `printf`-style formatter, and a numeric library full of `double` arithmetic. Nothing here is patched; it is the unmodified upstream tarball.

Get the sources once:

```
curl -LO https://www.lua.org/ftp/lua-5.4.7.tar.gz
tar xf lua-5.4.7.tar.gz && cd lua-5.4.7/src
```

The interpreter is every `.c` in `src` *except* `luac.c`, which is the standalone bytecode compiler and carries its own `main`. kcc needs no `-I` flag: the standard headers Lua includes—`<math.h>`, `<string.h>`, `<setjmp.h>`, `<locale.h>`, and the rest—are the compiler-resident freestanding headers (§17.16.2), so the only search path required is the current directory for Lua's own `lua.h`.

1.14.1. On macOS (native ARM64)

Here kcc is self-contained: it emits ARM64 machine code and links the executable itself, so a single tool takes the sources all the way to a binary.

```
KCC=/path/to/kcc/gen/kcc
for f in $(ls *.c | grep -v '^luac.c$'); do
    "$KCC" -c "$f" -o "${f%.c}.o"
done
"$KCC" *.o -o lua -lm          # kcc drives the Mach-O link itself
./lua -e 'print(_VERSION, math.pi)'
```

which prints `Lua 5.4  3.1415926535898`. A real script runs as you would expect:

```
$ ./lua
> local function fib(n) return n<2 and n or fib(n-1)+fib(n-2) end
> for i=1,10 do io.write(fib(i)," ") end print()
1 1 2 3 5 8 13 21 34 55
```

1.14.2. On Linux (x86-64)

The x86-64 target follows the gcc model (§1.10): kcc generates the object code and the system linker finishes the job. Because Lua uses `malloc`, `printf`, and `libm`, the final link is the hosted one, through the system compiler driver:

```
for f in $(ls *.c | grep -v '^luac.c$'); do
    kcc_linux -c "$f" --target=x86_64-linux -o "${f%.c}.o"
done
cc *.o -o lua -lm              # system ld pulls in libc + libm
./lua -e 'print(_VERSION, math.pi)'
```

The `kcc_linux` used here is the self-hosted Linux compiler that `make meta-c` builds (§1.19); on an Apple-silicon machine you can equally cross-compile each unit with the native kcc and `--target=x86_64-linux`, then assemble and link on the x86-64 host. Either way the result is byte-for-byte the same interpreter, and it prints the same `3.1415926535898`.

One bug worth naming. That `math.pi` value is not free. Lua defines π as a thirty-four-digit literal, `3.14159265358979323`8 and kcc's compile-time decimal-to-`double` conversion originally folded *every* digit into a 64-bit integer mantissa—which overflows past nineteen digits and yielded a nonsensical 8.6×10^{-18}. The fix, on both back ends, is to accumulate only the first eighteen significant digits (a `double` carries about seventeen) and let any further integer digits raise the power-of-ten exponent instead of wrapping the mantissa (§22.10). It is the kind of defect only a real program surfaces: no hand-written test reaches for a thirty-four-digit constant, but a numeric library does.

1.15. Compiling SQLite

The next rung up from an interpreter is a database engine. **SQLite 3.45.0** ships as a single 250 000-line *amalgamation*, `sqlite3.c`, and kcc compiles it—on *both* targets—into a working library: the demo below opens an in-memory database, creates a table, inserts rows, and runs a `SELECT`, and the rows come back byte-for-byte as a `clang`-built SQLite prints them. SQLite is a harder test than Lua because it reaches for the operating system directly—its Unix VFS calls `mmap`, `fcntl` file locks, `fstat`, and the `errno` constants—so unlike the freestanding Lua build it needs a few POSIX headers on the search path, gathered in a small `sysinc/` directory alongside the source: `<sys/mman.h>`, `<sys/stat.h>`, `<sys/types.h>`, `<fcntl.h>` (with `struct flock` and the `F_*LCK` constants), and `<errno.h>`. Two of those—`<fcntl.h>` and `<errno.h>`—are *platform-aware*: `errno` numbers, the `F_GETLK`/`F_SETLK` values, and even the field order of `struct flock` differ between Linux and macOS, so the shim wraps them in `#ifdef __linux__` and kcc predefines `__linux__` only for the Linux target. One shim, correct on both. Fetch the amalgamation once:

```
curl -LO https://sqlite.org/2024/sqlite-amalgamation-3450000.zip
unzip sqlite-amalgamation-3450000.zip && cd sqlite-amalgamation-3450000
FLAGS="-DSQLITE_THREADSAFE=0 -DSQLITE_OMIT_LOAD_EXTENSION \
       -DSQLITE_DISABLE_LFS -DSQLITE_OMIT_WAL -DNDEBUG \
       -DSQLITE_DISABLE_INTRINSIC"
```

1.15.1. On macOS (native ARM64)

kcc emits ARM64 machine code and the system linker binds the two objects to `libSystem`; `main.c` opens `":memory:"` and drives `CREATE TABLE/INSERT/SELECT` through `sqlite3_exec`.

```
kcc -c sqlite3.c -Isysinc -I. $FLAGS -o sqlite3.o
kcc -c main.c    -Isysinc -I. $FLAGS -o main.o
cc sqlite3.o main.o -o sqltest         # clang links against libSystem
./sqltest
```

which prints the two selected rows and `sqlite ok`.

1.15.2. On Linux (x86-64)

Identical, driven by the self-hosted `kcc_linux` that `make meta-c` builds (§1.19); kcc emits the objects and the system `cc` links them against glibc. On an Apple-silicon machine you can equally cross-compile each unit with `--target=x86_64-linux` and finish on an x86-64 host.

```
kcc_linux -c sqlite3.c --target=x86_64-linux -Isysinc -I. $FLAGS -o sqlite3.o
kcc_linux -c main.c    --target=x86_64-linux -Isysinc -I. $FLAGS -o main.o
cc -static sqlite3.o main.o -o sqltest -lm
./sqltest
```

Why the shim reaches the VFS at all. kcc searches the `-I` directories *before* both its own resident built-in headers and the `include/` folder it ships (§17.16.2), so the `sysinc/` shim's fuller `<errno.h>` and `<fcntl.h>`—the ones carrying `ETIMEDOUT`, `ENOLCK`, and `struct flock`—win over the leaner shipped copies. Getting that precedence wrong (the default `include/` sitting ahead of `-I`) is one of two compiler bugs SQLite surfaced; the other is subtler: SQLite defines each VFS's I/O-method table through a long, comment-laden, multi-line `IOMETHODS(...)` macro invocation, and kcc's line counter drifted across the argument prescan, so the `#if` directive on the line just after the invocation stopped being recognised and its whole block leaked into the parser. Both are fixed; the VFS now compiles verbatim.

Flags. The `SQLITE_OMIT_*` flags trim optional subsystems (write-ahead logging, extension loading) a minimal build does not need; the core SQL engine is fully present. The one non-obvious flag is `-DSQLITE_DISABLE_INTRINSIC`: because kcc defines `__GNUC__` (the kernel and much else test it), SQLite would otherwise reach for GCC's `__atomic_load_n` builtins with `__ATOMIC_RELAXED`, which kcc does not implement; the flag selects SQLite's portable plain-load fallback, exactly right for the single-threaded (`THREADSAFE=0`) build here.

1.16. Playing DOOM

The final demonstration is graphical: kcc compiles **DOOM**—the 1993 id Software engine, by way of the portable `doomgeneric` tree—into a playable, windowed game on Apple silicon. All eighty-one C files are compiled by kcc: the entire engine (fixed-point renderer, BSP tree walk, WAD loader, sound and game logic) *and* the SDL platform shim that opens a window and blits the framebuffer, which pulls in the large `<SDL2/SDL.h>`.

```
brew install sdl2                       # the one external dependency
# doomgeneric sources + a Freedoom IWAD in place:
KCC=/path/to/kcc/gen/kcc
for f in <every engine .c> doomgeneric.c doomgeneric_sdl.c; do
    "$KCC" -O0 -DNORMALUNIX -DLINUX -I/opt/homebrew/include/SDL2 \
           -c "$f" -o "${f%.c}.o"
done
clang *.o -L/opt/homebrew/lib -lSDL2 -lm -o doom   # clang == the linker only
./doom -iwad freedoom1.wad              # a window opens; DOOM runs
```

`clang` here is the linker, not a compiler. Every `.o` above is emitted by kcc; the final `clang` line compiles nothing—it is invoked purely as a *link driver*, a convenient stand-in for `ld`. kcc on its native ARM64 target is perfectly capable of driving the link itself (the Lua build above ends with plain `kcc *.o -o lua -lm`), and it would do so for a self-contained program. DOOM is not self-contained: it links against Homebrew's SDL2 (`-L/opt/homebrew/lib -lSDL2`), which in turn drags in a constellation of macOS system frameworks and SDK stubs. `clang` already knows the SDK paths, the C runtime startup object, the default `-lSystem`, and the framework search rules, so handing it the objects is simply less fuss than reconstructing all of that on kcc's own `ld` invocation. The division of labour is exactly the one this manual draws throughout—kcc is the compiler, the system `ld` (however it is invoked) is the linker—and it is what makes the demonstration meaningful: because kcc, not `clang`, compiled all eighty-one files, the game running is a statement about kcc's code generation, right down to the software renderer that computes every pixel.

DOOM earns its place because it is unforgiving of a miscompiled arithmetic detail. One bug surfaced only here: the engine's wall renderer does `worldbottom >>= 4` on a negative file-scope `fixed_t`, and kcc was lowering the compound right-shift of a 4-byte global as an un-sign-extended 8-byte load—a *logical* shift where the source meant an arithmetic one. The wrong value overshot a floor span below the viewport into a null `ylookup[]` entry and crashed the span drawer. No unit test in the suite exercised a signed compound-assign shift through a global; a first-person renderer drawing every pixel of every frame did, immediately. The renderer is byte-exact against a `clang` build once that access carries its true size.

1.17. Booting a Linux Kernel Compiled by kcc

This is the project's capstone demonstration: kcc compiles `init/main.o` of a real, unmodified Linux 6.6 kernel—the file that holds `start_kernel`, `do_initcalls`, and `kernel_init_freeable`—and that kernel boots, on QEMU, all the way to a BusyBox `login:` prompt where you get a shell and can halt the machine. Every other object is the stock `gcc` build; swapping in kcc's `init/main.o` isolates the compiler under test. The boot exercises two fixes worth naming, because they are exactly the kind of bug a kernel surfaces that a test suite does not: kcc must compute `sizeof(struct { int : 0; })` as 0 (a zero-width bit-field adds no storage), or the kernel's `ARRAY_SIZE` macro miscounts and `do_initcalls` walks off its table; and it must emit a file-scope `char *p = "/init"` as *static data*, not a run-time store into a constructor the kernel never calls, or `ramdisk_execute_command` stays `NULL` and PID 1 dereferences it.

The pipeline is five small scripts under `gen/repro/`, tangled from the fragments below, plus `run-all.sh` that chains them. They drive a Docker image (a Debian toolchain with the Linux 6.6 tree, QEMU, and a static BusyBox baked in) so the build is reproducible on macOS, Linux, or Windows; a *Native Linux* note after each step gives the Docker-free equivalent. The only host prerequisites are Docker and `weft` (to tangle kcc's own sources). Run the whole thing with:

```
bash gen/repro/run-all.sh        # build image, kcc, kernel, initramfs
bash gen/repro/05-boot.sh        # boot to the login: prompt (root, no password)
```

A note on authenticity: these scripts let `gcc` compile kcc's C sources into the `kcc_linux` cross-compiler, so the demo runs on any x86-64 host without an Apple-silicon machine. The *pure* self-host—where kcc itself compiles kcc—is `make meta-c` on ARM64 (§1.19); the `init/main.o` produced either way is byte-identical.

1.17.1. The build image

The image bakes the slow, cacheable prerequisites: a kernel toolchain, the Linux 6.6 source configured for a tiny bootable kernel (serial console, ELF binaries, an initramfs), a first `gcc vmlinux` so later relinks are fast, the `gen_init_cpio` helper, and a static BusyBox.

⟨ *Repro: Dockerfile* 12 ⟩ ≡

```
FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
      build-essential bc bison flex libssl-dev libelf-dev cpio xz-utils \
      wget ca-certificates file qemu-system-x86 && rm -rf /var/lib/apt/lists/*
WORKDIR /root
RUN wget -q --tries=3 --timeout=60 \
      https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.6.tar.xz \
 && tar xf linux-6.6.tar.xz && rm linux-6.6.tar.xz
RUN wget -q --tries=3 --timeout=60 -O /root/busybox \
      https://busybox.net/downloads/binaries/1.35.0-x86_64-linux-musl/busybox \
 && chmod +x /root/busybox
WORKDIR /root/linux-6.6
RUN make tinyconfig \
 && ./scripts/config --enable 64BIT --enable PRINTK --enable TTY \
      --enable SERIAL_8250 --enable SERIAL_8250_CONSOLE --enable BINFMT_ELF \
```

```
      --enable PRINTK_TIME --enable SECTION_MISMATCH_WARN_ONLY \
      --enable BOOT_CONFIG --enable BLK_DEV_INITRD --enable RD_GZIP \
 && make olddefconfig && make -j"$(nproc)" prepare \
 && make -j"$(nproc)" vmlinux && make usr/gen_init_cpio
```

□

12 · used in 38b

⟨ *Repro: 01 build image* 13a ⟩ ≡

```
#!/usr/bin/env bash
# Build the reproducible kernel-build image (Debian + linux-6.6 + QEMU + BusyBox).
# ~10 min and ~2 GB the first time; cached afterwards.
set -euo pipefail
IMG=${KCC_IMG:-kcc-kbuild:linux66}
here=$(cd "$(dirname "$0")" && pwd)
echo ">> building $IMG"
docker build --platform=linux/amd64 -t "$IMG" -f "$here/Dockerfile" "$here"
echo ">> done: $IMG"
```

□

13a · used in 38c

Native Linux: install `build-essential bc bison flex libssl-dev libelf-dev cpio qemu-system-x86`, unpack the Linux 6.6 source, apply the same `scripts/config` lines, and run `make olddefconfig prepare vmlinux usr/gen_init_cpio`; then skip the `docker run` wrappers below and run each inner block directly in the kernel tree.

1.17.2. Bootstrapping kcc for Linux

The x86-64/ELF compiler that will build the kernel object, `kcc_linux`, can be produced two ways, and `02-build-kcc.sh` picks one by looking at the host. On **Apple silicon** (`Darwin/arm64`) it takes the *authentic* route: it builds the native kcc (`make build`) and has *kcc cross-compile its own sources* to x86-64 assembly (`--target=x86_64-linux`)—the same first step as `make meta-c` (§1.19)—so the compiler that builds the kernel was itself built by kcc. On **any other host** (a Linux x86-64 box) there is no native kcc to run, so it takes the *portable* route: `gcc` compiles kcc's C sources, with three freestanding `mem*` shims a `-ffreestanding` build needs. Either way the modules and the Linux runtime (`os_linux.s`) are assembled and linked into `kcc_linux` inside the image; the `init/main.o` it later produces is byte-identical.

⟨ *Repro: 02 build kcc* 13b ⟩ ≡

```
#!/usr/bin/env bash
# Tangle kcc's sources and link the Linux-native compiler kcc_linux.
# Apple silicon: kcc(ARM) cross-compiles kcc -> x86-64 (authentic self-host).
#   How the emitted .s becomes the ELF is chosen by the LINK mode -- the first
#   argument, or $KCC_LINK, default "docker":
#     docker  assemble+link in the amd64 image (no local cross-tools needed)
#     cross   assemble+link on macOS with x86-64 cross-binutils (no container)
#     emit    only write the .s; finish it on the destination x86-64 host
# Other hosts: gcc bootstraps kcc's C sources -> x86-64 (portable).
set -euo pipefail
IMG=${KCC_IMG:-kcc-kbuild:linux66}
LINK="${1:-${KCC_LINK:-docker}}"
root=$(cd "$(dirname "$0")/../.." && pwd)
work="$root/gen/repro/work"; mkdir -p "$work"
echo ">> tangling literate sources (needs weft)"
( cd "$root" && make tangle >/dev/null )
MODS="arena strings io hashtab vec strtab source lexer_tables diag preprocessor parser expressions
   statements declarations types symtab conversions sema ir opt emit_arm64 emit_x86 linker codegen
   emit_macho dwarf driver main"

if [ "$(uname -s)" = Darwin ] && [ "$(uname -m)" = arm64 ]; then
  echo ">> host Apple silicon -- kcc(ARM) cross-compiles kcc -> x86-64 (link: $LINK)"
  [ -x "$root/gen/kcc" ] || ( cd "$root" && make build >/dev/null )
  for m in $MODS; do "$root/gen/kcc" -c "$root/gen/selfhost/$m.c" --target=x86_64-linux -o "$work/$m.s
   " >/dev/null 2>&1; done
  "$root/gen/kcc" -c "$root/gen/selfhost/lexer_core.c" --target=x86_64-linux -o "$work/lexer.s" >/dev/
   null 2>&1
  cp "$root/gen/runtime/os_linux.s" "$work/os_linux.s"
  # shared assemble+link step, honouring $AS/$LD (default GNU as/ld)
  cat > "$work/_link_kcc.sh" <<'INNER'
set -e
```

```
AS="${AS:-as}"; LD="${LD:-ld}"
ALL="os_linux arena strings io hashtab vec strtab source lexer lexer_tables diag preprocessor parser
   expressions statements declarations types symtab conversions sema ir opt emit_arm64 emit_x86 linker
   codegen emit_macho dwarf driver main"
for m in $ALL; do "$AS" --64 -o "$m.o" "$m.s"; done
OBJ=""; for m in $ALL; do [ "$m" = os_linux ] || OBJ="$OBJ $m.o"; done
"$LD" -static -nostdlib -e _start -o kcc_linux os_linux.o $OBJ
echo "  built kcc_linux: $(file -b kcc_linux | cut -d, -f1-2)"
INNER
  case "$LINK" in
    docker)
      docker run --rm --platform=linux/amd64 -v "$work":/w -w /w "$IMG" bash /w/_link_kcc.sh ;;
    cross)
      asx= ; ldx=
      for a in x86_64-linux-gnu-as x86_64-unknown-linux-gnu-as x86_64-elf-as; do if command -v "$a" >/
   dev/null 2>&1; then asx=$a; break; fi; done
      for l in x86_64-linux-gnu-ld x86_64-unknown-linux-gnu-ld x86_64-elf-ld; do if command -v "$l" >/
   dev/null 2>&1; then ldx=$l; break; fi; done
      [ -n "$asx" ] && [ -n "$ldx" ] || { echo "!! no x86-64 cross-binutils on PATH; install e.g. '
   brew install x86_64-elf-binutils', or use LINK=docker"; exit 1; }
      ( cd "$work" && AS="$asx" LD="$ldx" bash _link_kcc.sh ) ;;
    emit)
      echo ">> wrote x86-64 assembly to $work (no local link). Finish on an x86-64 Linux host:"
      echo "     scp $work/*.s $work/_link_kcc.sh user@HOST:/gen/kccx/"
      echo "     ssh user@HOST 'cd /gen/kccx && bash _link_kcc.sh'   # runs as/ld -> kcc_linux"
      exit 0 ;;
    *) echo "!! unknown LINK mode '$LINK' (use: docker | cross | emit)"; exit 1 ;;
  esac
else
  echo ">> host $(uname -s)/$(uname -m) -- gcc bootstraps kcc -> x86-64 (portable, LINK ignored)"
  cp "$root"/gen/selfhost/*.c "$work"/
  cp "$root"/gen/runtime/os_linux.s "$work"/os_linux.s
  cat > "$work/memglue.c" <<'MEM'
void *memcpy(void*d,const void*s,unsigned long n){char*a=d;const char*b=s;while(n--)*a++=*b++;return d
   ;}
void *memset(void*d,int c,unsigned long n){char*a=d;while(n--)*a++=c;return d;}
void *memmove(void*d,const void*s,unsigned long n){char*a=d;const char*b=s;if(a<b)while(n--)*a++=*b++;
   else{a+=n;b+=n;while(n--)*--a=*--b;}return d;}
MEM
  cat > "$work/_gcc_link.sh" <<'INNER'
set -e
G="gcc -O0 -w -std=gnu99 -ffreestanding -fno-stack-protector -fno-builtin -Wno-error=implicit-function
   -declaration"
as --64 -o os_linux.o os_linux.s
$G -c memglue.c -o memglue.o
MODS="arena strings io hashtab vec strtab source lexer_core lexer_tables diag preprocessor parser
   expressions statements declarations types symtab conversions sema ir opt emit_arm64 emit_x86 linker
   codegen emit_macho dwarf driver main"
OBJ="os_linux.o memglue.o"
for m in $MODS; do $G -c $m.c -o $m.o; OBJ="$OBJ $m.o"; done
ld -static -nostdlib -e _start -o kcc_linux $OBJ
echo "  built kcc_linux (gcc bootstrap): $(file -b kcc_linux | cut -d, -f1-2)"
INNER
  docker run --rm --platform=linux/amd64 -v "$work":/w -w /w "$IMG" bash /w/_gcc_link.sh
fi
```

□

13b · used in 38d

Choosing how the x86-64 binary is finished (`LINK`). On Apple silicon kcc *generates* the x86-64 assembly natively, but turning that `.s` into the `kcc_linux` ELF needs an x86-64 assembler and linker, which macOS does not provide. The script's first argument (or the `KCC_LINK` environment variable) picks how to finish, so you can build a `kcc_linux` to copy to any x86-64 Linux box:

```
bash gen/repro/02-build-kcc.sh docker   # (default) assemble+link in the image
bash gen/repro/02-build-kcc.sh cross    # use x86-64 cross-binutils on macOS
bash gen/repro/02-build-kcc.sh emit     # only emit .s; finish on the x86-64 host
```

`docker` needs no local cross-tools; `cross` wants an `x86_64-linux-gnu-` or `x86_64-elf-` binutils on `PATH` (e.g. `brew install x86_64-elf-binutils`) and produces the ELF without a container; `emit` stops after the assembly and prints

the two commands to finish the build on the target machine. All three yield the same static `kcc_linux`, which runs natively on x86-64 Linux—so `scp gen/repro/work/kcc_linux` to an x86-64 host and it compiles the kernel there directly (do *not* copy the ARM64 `gen/kcc`, which will not execute on x86).

Native Linux: run `make tangle`, then the `else`-branch's `_gcc_link.sh` body in `gen/repro/work`; the result is `kcc_linux`. (The `Darwin` branch is what an Apple-silicon developer gets automatically; the detection is a plain `uname` test you can edit to force a route.)

1.17.3. Compiling and linking the kernel

We keep every object from the image's `gcc` build except `init/main.o`, which `kcc_linux` compiles from `init/main.c` using `gcc`'s own flags (recovered from the recorded `.main.o.cmd`). kcc emits out-of-line copies of unused `static inline` bodies whose callees a tiny kernel lacks; they are dead, so each undefined symbol is stubbed to `0` and the link is retried until it succeeds—then `bzImage` is packed.

⟨ *Repro: 03 build kernel* 15a ⟩ ≡

```
#!/usr/bin/env bash
# Rebuild the kernel with init/main.o compiled by kcc.
set -euo pipefail
IMG=${KCC_IMG:-kcc-kbuild:linux66}
work=$(cd "$(dirname "$0")/../.." && pwd)/gen/repro/work
cat > "$work/_build_kernel.sh" <<'INNER'
set -e
cd /root/linux-6.6
make -j"$(nproc)" vmlinux >/dev/null 2>&1 || true
make init/main.o >/dev/null 2>&1
line=$(sed -n 's/^savedcmd_[^:]*:= //p;s/^cmd_[^:]*:= //p' init/.main.o.cmd | head -1)
FLAGS=""; prev=""
for tok in $line; do case "$tok" in
  -I*|-D*|-U*|-nostdinc|-std=*) FLAGS="$FLAGS $tok";;
  -include) FLAGS="$FLAGS -include";;
  *) [ "$prev" = "-include" ] && FLAGS="$FLAGS $tok";; esac; prev="$tok"; done
/w/kcc_linux --target=x86_64-linux $FLAGS -c -o init/main.o init/main.c
touch init/main.o
mkdir -p /w/ldwrap; : > /w/defsyms.txt
printf '#!/bin/bash\nexec /usr/bin/ld "$@" $(cat /w/defsyms.txt 2>/dev/null)\n' > /w/ldwrap/ld
chmod +x /w/ldwrap/ld; export PATH=/w/ldwrap:$PATH
for pass in 1 2 3 4 5 6; do
  if make -j"$(nproc)" vmlinux >/gen/vm 2>&1; then linked=1; break; fi
  grep -oE "undefined reference to \`[^']+'" /gen/vm | sed "s/.*to .//;s/.$//" | sort -u > /gen/u
  add=0; while read -r s; do grep -qx -- "--defsym $s=0" /w/defsyms.txt || { echo "--defsym $s=0" >> /
   w/defsyms.txt; add=$((add+1)); }; done < /gen/u
  [ "$add" -eq 0 ] && { echo "link stuck:"; tail -8 /gen/vm; exit 1; }
done
[ "${linked:-0}" = 1 ] || { echo "did not link"; exit 1; }
make -j2 bzImage >/dev/null 2>&1
cp arch/x86/boot/bzImage /w/bzImage
echo "  bzImage: $(stat -c%s /w/bzImage) bytes (init/main.o by kcc, $(wc -l </w/defsyms.txt) dead
   stubs)"
INNER
echo ">> building kernel (init/main.o = kcc)"
docker run --rm --platform=linux/amd64 -v "$work":/w "$IMG" bash /w/_build_kernel.sh
```

□

15a · used in 38e

Native Linux: run `_build_kernel.sh`'s body in your prepared `linux-6.6` tree with `/w` pointing at `gen/repro/work`.

1.17.4. An initramfs with login and a shell

BusyBox is the userland: one static binary, symlinked to `init`, `getty`, `login`, `sh`, `halt`, and the usual commands. `gen_init_cpio` builds the archive—including the `/dev/console` and `/dev/ttyS0` nodes—without needing root. An `/etc/inittab` spawns a serial `getty`; `/etc/passwd` gives `root` an empty password.

⟨ *Repro: 04 initramfs* 15b ⟩ ≡

```
#!/usr/bin/env bash
# Build a BusyBox initramfs that boots to a serial login shell.
set -euo pipefail
IMG=${KCC_IMG:-kcc-kbuild:linux66}
```

```
work=$(cd "$(dirname "$0")/../.." && pwd)/gen/repro/work
cat > "$work/inittab" <<'T'
::sysinit:/bin/busybox --install -s
::sysinit:/bin/hostname kcc
ttyS0::respawn:/sbin/getty -L ttyS0 115200 vt100
::ctrlaltdel:/sbin/reboot
::shutdown:/bin/umount -a -r
T
echo 'root::0:0:root:/root:/bin/sh' > "$work/passwd"
echo 'root:x:0:' > "$work/group"
printf '\nkcc-built Linux 6.6 (init/main.o by kcc). Login: root (no password).\n\n' > "$work/issue"
cat > "$work/cpio.list" <<'L'
dir /bin 755 0 0
dir /sbin 755 0 0
dir /etc 755 0 0
dir /dev 755 0 0
dir /proc 755 0 0
dir /sys 755 0 0
dir /root 755 0 0
file /bin/busybox /root/busybox 755 0 0
slink /init /bin/busybox 777 0 0
slink /sbin/init /bin/busybox 777 0 0
slink /bin/sh busybox 777 0 0
slink /bin/login busybox 777 0 0
slink /sbin/getty /bin/busybox 777 0 0
slink /sbin/halt /bin/busybox 777 0 0
slink /sbin/poweroff /bin/busybox 777 0 0
slink /sbin/reboot /bin/busybox 777 0 0
slink /bin/hostname busybox 777 0 0
slink /bin/mount busybox 777 0 0
slink /bin/umount busybox 777 0 0
file /etc/inittab /w/inittab 644 0 0
file /etc/passwd /w/passwd 644 0 0
file /etc/group /w/group 644 0 0
file /etc/issue /w/issue 644 0 0
nod /dev/console 600 0 0 c 5 1
nod /dev/ttyS0 600 0 0 c 4 64
nod /dev/null 666 0 0 c 1 3
nod /dev/tty 666 0 0 c 5 0
L
echo ">> packing initramfs (BusyBox login)"
docker run --rm --platform=linux/amd64 -v "$work":/w "$IMG" bash -c \
  'cp /root/busybox /w/busybox; /root/linux-6.6/usr/gen_init_cpio /w/cpio.list > /w/initramfs.cpio'
echo "  initramfs.cpio: $(stat -f%z "$work/initramfs.cpio" 2>/dev/null || stat -c%s "$work/initramfs.
   cpio") bytes"
```

□

15b · used in 38f

1.17.5. Booting it

If your host has `qemu-system-x86_64`, we boot there for speed (hardware acceleration: `hvf` on macOS, `kvm` on Linux); otherwise we boot inside the image with software emulation. At `kcc login:` type `root`; the shell is `/root #`. Type `uname -a`, `ls`, `cat /etc/issue`; `poweroff -f` halts. `random.trust_cpu=on` seeds the RNG so early boot never stalls waiting for entropy.

⟨ *Repro: 05 boot* 16 ⟩ ≡

```
#!/usr/bin/env bash
# Boot the kcc-compiled kernel to a BusyBox login shell.
set -euo pipefail
IMG=${KCC_IMG:-kcc-kbuild:linux66}
work=$(cd "$(dirname "$0")/../.." && pwd)/gen/repro/work
APPEND="console=ttyS0,115200 earlycon=uart8250,io,0x3f8 keep_bootcon panic=-1 random.trust_cpu=on"
if command -v qemu-system-x86_64 >/dev/null 2>&1; then
  # hardware acceleration only when the host arch matches the x86-64 guest
  case "$(uname -s)/$(uname -m)" in
    Darwin/x86_64)          ACCEL="-accel hvf";;
    Linux/x86_64)           [ -e /dev/kvm ] && ACCEL="-enable-kvm -cpu host" || ACCEL="-accel tcg";;
    *)                      ACCEL="-accel tcg";;   # e.g. Apple silicon: x86 guest -> software
  esac
```

```
  echo ">> booting on host qemu ($ACCEL) --- login: root / halt: poweroff -f"
  exec qemu-system-x86_64 $ACCEL -m 512M -kernel "$work/bzImage" \
    -initrd "$work/initramfs.cpio" -append "$APPEND" -nographic -no-reboot
fi
echo ">> no host qemu; booting in container (TCG, slower)"
exec docker run --rm -it --platform=linux/amd64 -v "$work":/w -w /w "$IMG" \
  qemu-system-x86_64 -accel tcg -m 512M -kernel bzImage -initrd initramfs.cpio \
  -append "$APPEND" -nographic -no-reboot
```

□

16 · used in 38g

⟨ *Repro: run all* 17 ⟩ ≡

```
#!/usr/bin/env bash
# Build everything (image, kcc, kernel, initramfs); then run 05-boot.sh yourself.
set -euo pipefail
d=$(cd "$(dirname "$0")" && pwd)
bash "$d/01-image.sh"
bash "$d/02-build-kcc.sh"
bash "$d/03-kernel.sh"
bash "$d/04-initramfs.sh"
echo
echo ">> all built. Boot the kernel with:  bash $d/05-boot.sh"
```

□

17 · used in 38h

On the acceleration: `05-boot.sh` needs no configuration—on a Linux host it finds `/dev/kvm` and the boot is nearly instant, on macOS it uses `hvf`, and where neither host `qemu` nor hardware acceleration is available it falls back to software emulation inside the image, which works everywhere but takes about half a minute to reach the prompt.

1.18. Building This Book

Two targets build the document, mirroring how `make tangle` extracts the code: `make weave` runs weft to produce the LaTeX source (`kcc.tex`), and `make pdf` compiles that `.tex` into the typeset `kcc.pdf` (with `lualatex`, for native Unicode). So for the finished book you want:

```
make pdf        # weaves kcc.tex, then compiles kcc.pdf
```

(`make pdf` depends on `make weave`, so it runs both; use `make weave` alone if you only want the LaTeX source.) The PDF step needs a TeX installation with `lualatex`, `latexmk`, and the `underscore` package (which lets the woven fragment names that contain underscores typeset in text mode); on a minimal distribution such as TinyTeX, `tlmgr install underscore` provides it. This is purely optional and unrelated to using the compiler.

1.19. For Contributors

The project's `make` recipes cover the full development loop. The most useful are `make build` (produce the compiler), `make test` (build and run the unit tests), and the conformance suites that guard against regressions: `make test-codegen` (end-to-end code-generation tests), `make difftest` (compile a corpus with both kcc and the reference compiler and compare their output), `make selfhost` (compile a piece of kcc's own C with kcc and check the result), `make diagtest` and `make warntest` (check that invalid programs are rejected and that warnings fire as expected), `make interop` (validate the calling convention against the reference compiler), and `make e2e` (the one-step compile-and-run path). All changes go through the `.weft` sources; the tangled assembly under `tmp` is generated and must never be edited by hand.

1.20. Troubleshooting

- **Linking fails, or the executable will not run.** kcc calls the system linker and needs the macOS SDK. Confirm the Xcode Command Line Tools are installed (`xcode-select -p` should print a path) and that `xcrun --show-sdk-path` prints a valid SDK location.
- **A program compiles but behaves unexpectedly.** Use `--dump-ir` to see the code kcc generated, and check the feature against the limits in §1.13; a divergence from another compiler on a corner feature is more likely a known gap than a miscompilation of ordinary code, which the difference suite guards closely.

- **A macro does not expand as you expect.** Run `kcc -E` on the file to see the preprocessed text.
- **`kcc` is not found.** The built compiler lives at `gen/kcc`; invoke it by that path, or copy it onto your `PATH` as shown in §1.3.

2. Retargeting kcc: The Porting Playbook

This chapter is a promise to a future contributor. When a new architecture is to be supported—x86-64, RISC-V, a second ARM ABI, a future ISA that does not yet exist—this is the document to read first. It explains *why* kcc is organised so that a new back end is cheap, *what* exactly must be supplied, and *in what order* to build it so that every step is small and testable. The intent is blunt: after reading this chapter, a competent implementer should be able to open a processor manual and an ABI document and start writing the new target the same afternoon. Breadth—supporting as many architectures as possible—is a first-class goal, and the only way to reach it is to make each additional architecture a matter of *filling in a form*, not inventing a compiler.

2.1. The Principle: One Front End, Many Back Ends

Everything in kcc is arranged around a single dividing line: the intermediate representation (Chapter 28). Above that line the compiler is *target-neutral*; below it, target-specific. This is not an accident of style—it is the entire strategy for portability.

Above the IR sits the whole language: the lexer, the preprocessor, the parser, semantic analysis, and IR construction. None of it knows or cares what machine will eventually run the program. It is written once and never touched again when a target is added. So is the optimizer (Chapters 31–**??**): constant folding, dead-code elimination, common-subexpression elimination, and scalar-local promotion all operate on the IR, and are therefore shared by every target for free. So, crucially, is the *logic* of register allocation (§2.4 below): linear scan, liveness, the loop and call analyses, the mem2reg reinterpretation—all of it is neutral, and only the *register banks* it draws from are target data.

Below the IR sits exactly what differs between machines: how an IR operation becomes instructions, which registers exist and how they are partitioned, how arguments are passed, how a stack frame is laid out, and how the result is turned into an object file the platform's linker accepts. A new target supplies these—and nothing else.

Written once (target-neutral)	Written per target
lexer, preprocessor, parser	instruction selection (IR op → asm)
semantic analysis, type system	register banks (names, counts, roles)
IR construction and validation	ABI (argument/return, alignment)
optimizer (fold, DCE, CSE, mem2reg)	prologue/epilogue and frame layout
register-allocation *logic*	assembler and linker invocation
diagnostics, driver skeleton	runtime (syscalls, `_start`)

The left column is roughly seventy percent of the compiler and never grows when architectures are added. The right column is what this chapter is about.

2.2. Two Ways to Reach Metal

There are two honest ways to turn a stream of instructions into an object file, and kcc deliberately supports both.

The first is *direct emission*: the back end computes the exact bytes of each instruction and writes the object file itself. This is what the ARM64 back end does—Chapter 36 encodes every instruction class into its 32-bit word, and Chapter 40 lays out the Mach-O container by hand. It owns everything down to the last bit, depends on nothing but the system linker, and is the truest expression of kcc's "build your own floor" character. Its price is that each instruction-set needs a full encoder—an intricate, error-prone module—and each object format needs its own writer.

The second is *assembly emission*: the back end prints textual assembly (`.s`) and hands it to the platform's assembler, which produces the object file. The assembler does the encoding *and* emits the native object format—ELF on Linux, Mach-O on macOS, COFF on Windows—so a target that emits assembly needs *neither* an instruction encoder *nor* an object writer. What remains is only the interesting, machine-defining work: choosing which instructions to emit, allocating registers, and obeying the ABI.

For *growing to many architectures*, the second way is decisively more economical, and the reason is structural, not merely a matter of typing less. An instruction encoder is the part of a back end most likely to harbour subtle, hard-to-find bugs (a wrong bit in a rarely-used addressing mode), and it must be written and hardened *per instruction set*. An object writer must be written *per object format*. Delegating both to the system assembler means inheriting decades of hardened correctness instead of maintaining a new bug surface for every ISA and every format. That is not a compromise for a production tool; it is the sensible choice. Many production compilers emit assembly for exactly this reason.

It is worth being precise about what is delegated and what is kept, because the distinction is the whole argument. kcc keeps everything that makes it a compiler: the front end, the IR, the optimizer, register allocation, instruction *selection*, and the ABI. It delegates only the two most mechanical, most target-multiplied, least conceptually interesting steps: turning chosen instructions into bytes, and packing bytes into an object file. A target built this way is not "a front end to the assembler"—it is a complete compiler that uses the platform assembler as its code-object back end, which is a respectable and common architecture.

The flagship ARM64 target remains direct: it already exists, it is the proof that kcc can go all the way to the metal from scratch, and it is what makes the self-hosting build byte-identical. We do not rewrite it. The target interface (§2.3) is what lets the two coexist: each target chooses its own emission in its own *emit* operation. ARM64 emits bytes and a Mach-O; a new target emits text and invokes `as`. Nothing above the emit operation knows the difference. Should a direct encoder for a new ISA ever be wanted (for the same from-scratch pride, or to drop the assembler dependency), it can be added later as a different *emit* without disturbing selection, the allocator, or the ABI—so choosing assembly now forecloses nothing.

2.3. The Target Interface

A target is a *descriptor* plus a set of *operations*. The descriptor is data; the operations are the code a new back end supplies. Adding an architecture means creating one target module that provides all of them, and registering it so that `--target` can select it.

Identity and toolchain. The descriptor first fixes the basics: a name (`arm64`, `x86_64`, …), the pointer width and endianness, the symbol-name convention (the Mach-O world prefixes an underscore; ELF does not), the assembler command and flags to run on the emitted `.s`, the object format it produces, and the linker command and startup files used to produce an executable. For a direct target the assembler command is empty and the object writer is used instead.

Register banks. For each class of register—general-purpose and floating-point—the target declares, as plain data: the register names (or numbers) available for allocation, how many there are, which are *caller-saved* (free across a call only if the value is dead there) versus *callee-saved* (preserved across calls, but must be spilled and restored by the prologue/epilogue), and which are reserved by the ABI or used as fixed scratch by the emitter (the stack pointer, the frame pointer, the argument registers during a call, any hardwired zero register). These banks are the *only* machine-specific input the register allocator needs; see §2.4.

The calling convention (ABI). The target supplies the rules for passing and returning values: which registers carry the first several integer and pointer arguments, which carry floating arguments, how an argument too large for a register or beyond the register budget is passed on the stack, how an aggregate is classified (the System V AMD64 eightbyte rules; the AArch64 homogeneous-aggregate and by-reference rules), where the return value lands, the required stack alignment at a call, and any red zone. In kcc this is expressed as a small classification routine plus a table—not scattered through the code—so that a new ABI is read out of its specification and transcribed in one place.

Instruction selection. This is the heart of a new target: for each IR operation the emitter produces the equivalent instruction or short sequence for this machine. A constant becomes a move-immediate; an `IR_ADD` becomes the machine's add; a `IR_LOAD` becomes a load from the addressing mode the target favours; a comparison plus conditional branch becomes the machine's compare-and-branch idiom; a call becomes the argument marshaling from the ABI followed by the call instruction. For an assembly target each case simply prints the mnemonic and operands; for a direct target it encodes the bytes. Selection is where the bulk of a new target's lines live, but each case is small, local, and independently testable.

Prologue, epilogue, and frame. The target lays out the stack frame (spill slots for un-allocated virtual registers, `alloca` storage, the callee-saved save area, incoming stack arguments) and emits the entry sequence that establishes it and the exit sequence that tears it down and returns. The frame *shape* is target-specific; the *contents* (how many vreg slots, which callee-saved registers were claimed) come from the shared register allocator.

Emission. Finally the target says how assembly text is spelled for this machine—the operand syntax, the directives that open the code and data sections, how a label and a global symbol are declared—and how to drive the assembler and linker. A direct target instead encodes bytes and writes the object; either way, the operation above it is the same "produce an object file from this function's instructions."

2.4. The Register Allocator Is Already Portable

The most valuable thing to understand before porting is that register allocation does *not* need to be rewritten per target. The allocator of Chapter 35 (§35.9) is a linear scan over virtual-register live ranges; its liveness computation, its back-edge and call analyses, its loop-carried reservation, and the mem2reg reinterpretation of scalar-local allocas are all expressed in terms of the IR and of *abstract* register slots, not of any particular machine's registers. What differs between machines is purely the *banks*: how many caller-saved and callee-saved integer registers exist, the same for floating-point, and which physical register each abstract slot maps to.

A new target therefore does not touch the allocator. It supplies its bank descriptors—"these four registers are caller-saved scratch, these nine are callee-saved, the floating bank is these"—and the allocator's existing machinery produces a map from virtual register to physical register for that bank. The two banks (integer and floating) are allocated by the same logic run over different descriptors, exactly as the ARM64 back end already does with its general and `d`-register banks. This is the single largest reason a new architecture is cheap: the hardest part of a back end to get right is written once and reused.

2.5. The Porting Playbook

The recipe below is ordered so that each step ends at a running program that does slightly more than the last. Never build the whole target before testing any of it; build the smallest thing that runs, confirm it against the reference compiler, and grow.

1. **Register the target and its `--target` dispatch.** Create the target descriptor with its identity and toolchain fields, and wire `--target=`*name* in the driver to select it while leaving ARM64/Mach-O the default and untouched. *Milestone:* `kcc --target=`*name* runs and reaches the (still empty) back end without disturbing the existing target.

2. **Declare the register banks.** Read the ABI's register roles and transcribe them: the caller-saved and callee-saved general registers, the floating registers, and the reserved ones (stack pointer, frame pointer, the argument registers, any zero register). Nothing runs yet; this is data the allocator will consume.

3. **Stand up emission.** Emit the assembly-file preamble—the directives for the code and data sections, the symbol syntax, the label-and-global declarations—and wire the call to the assembler and linker. *Milestone:* kcc emits a syntactically valid, empty `.s` that the platform assembler accepts and the linker turns into an (empty) object.

4. **Select the minimal instruction set.** Implement just enough selection to compile `int main(){ return N; }`: materialise an integer constant, and the function return. Add the prologue/epilogue in its simplest form. *Milestone:* the program builds, runs (under the chosen execution method—§2.8), and exits with `N`. This is the moment the whole framework is proven end to end.

5. **Grow selection, one IR opcode at a time.** Add integer arithmetic and bitwise operations, then loads and stores, then comparisons and branches, then unconditional jumps. After each opcode, compile a small program exercising it and difftest it against the reference compiler. A divergence now is localised to the one opcode just added, which is why the order matters.

6. **Implement the calling convention.** Marshal arguments into their registers and stack slots, and read the return value from its register, per the ABI declared in step 2. *Milestone:* multi-argument calls work, and `printf` produces correct output—the first programs that look like real code.

7. **Complete the frame and the wide cases.** Handle `alloca` storage, spilled virtual registers, more than the register budget of arguments (stack arguments), and large frames. *Milestone:* functions with many locals and deep expressions match the reference compiler.

8. **Add floating point.** Declare the floating register bank (the allocator handles it with the same logic), select the floating arithmetic and conversion instructions, and implement the floating half of the ABI. *Milestone:* the floating-point programs in the difference corpus pass.

9. **Provide the runtime and finish the toolchain.** Supply the target's system-call convention and `_start` for freestanding programs, and the exact linker invocation (startup files, libraries) for the platform. *Milestone:* `kcc file.c -o out` produces a runnable executable in one step, as on the flagship target.

At the end of this ladder the new target passes the same difference corpus the flagship does. Each rung was small, and each was verified before the next was begun.

As realized for x86-64. This ladder is not hypothetical: it is exactly the path the x86-64 back end of Chapter 38 walked—and walked to the end. Rung one stood up emission and the minimal instruction set together (`const` + `ret`, proving the pipeline with `int main(void){return N;}`); rung two grew selection across the whole integer ALU and the memory operations over an all-spill frame; rung three added comparisons, branches, and the System V calling convention, including stack-passed arguments; the later rungs completed the frame, the data section, floating point, and the runtime. Each was committed only once its programs ran exit-identically to `clang -arch x86_64` and the ARM64 oracle stayed byte-identical to the last bit.

A complete second target. What began as one rung is now a full second target. x86-64 emits AT&T assembly from the same neutral IR and lets the system `as` and `ld` encode and link it, on both Mach-O (macOS) and ELF (Linux); it implements the whole System V AMD64 ABI—integer and floating arguments, variadic `va_list`, stack-passed and by-value aggregate arguments, computed `goto`—and passes the difference corpus. It compiles real, unmodified software: Lua, SQLite, and Linux-kernel translation units—one of which, a kcc-compiled `init/main.o`, boots a tiny-config Linux 6.6 to the point of executing `init`. On Linux the target *self-hosts*: kcc compiled by kcc reproduces every one of its own modules to a three-way byte-identical fixpoint, and drives `as` and `ld` itself, so that `kcc file.c -o out` yields an ELF executable in a single command. Throughout, the code generation is entirely kcc's work; Rosetta on the Mac and a user-mode emulator on Linux only *run* the output so it can be checked against the oracle (§2.8), and the ARM64 target stayed byte-identical to the bit. The playbook is therefore not a hope but a demonstrated path: a back end built to its shape reaches all the way to real, running software—not a subset.

2.6. A Target Skeleton

The interface of §2.3 and the ladder of §2.5 describe the work; this section names it as a concrete list of artifacts to create, in the shape they take inside kcc. Treat the x86-64 back end (Chapter 38) as the worked template: every item below already exists there in full, and the fastest way to write a new target is to read x86-64's version of each and transcribe it for the new machine. Nothing above the emitter is touched.

1. Selection and identity. Add the architecture to the `_kcc_target` selector and route `--target=`*name* to it in the driver, leaving ARM64/Mach-O the default. Fix the symbol convention (an underscore prefix for the Mach-O world, none for ELF) and, if the target serves two object formats, an OS selector—`_kcc_x86_os` is the precedent for a single emitter that produces both Mach-O and ELF.

2. Register banks (pure data). Declare, as arrays, the allocatable general registers split into caller-saved and callee-saved, the floating bank likewise, and the reserved registers (stack pointer, frame pointer, the argument registers, any zero register). A legal shortcut for bring-up is to declare *no* allocatable registers and spill every value to the frame; this is exactly how x86-64 first came up, and it lets selection be written and tested before the banks are tuned.

3. ABI classifier and argument marshaller. Extend the type classifier (`_kcc_ir_classify_type`) with this machine's rules for where each value is passed and returned, and the marshaller (`_kcc_cg_marshal_args`) that places arguments in their registers and stack slots and reads the result back. This is the ABI, transcribed from its specification in one place rather than scattered through selection.

4. Instruction selection. One case per IR opcode, printing the mnemonic and operands for this machine. This is the bulk of the new lines but each case is small and independently testable; x86-64's per-opcode emitter is the direct model.

5. Prologue, epilogue, and frame. Lay out the stack frame—spill slots, `alloca` storage, the callee-saved save area, incoming stack arguments—and emit the entry and exit sequences. The *shape* is yours; the *contents* (slot count, which callee-saved registers were claimed) come from the shared allocator (§2.4).

6. Emission and toolchain. Spell the assembly preamble (section directives, label and global syntax), then invoke the system `as` to encode it and `ld` to link it, with the platform's startup files. A direct target instead encodes bytes and writes the object itself, as ARM64 does (Chapters 36–40).

7. Runtime. Supply the target's system-call convention and a `_start` for freestanding programs, so the driver can turn a source file into a runnable executable in one command.

2.7. A Porting Checklist

Each rung of the playbook pairs one deliverable with one proof. Do them in order, and never begin a rung until the previous one runs exit-identically to the oracle (§2.8); a divergence is then localised to the single rung just added.

Rung	What you write	How you prove it
1. Dispatch	target added to `_kcc_target`; `--target` wired	`kcc --target=`*name* reaches the back end; ARM64 default unchanged
2. Banks	register-bank arrays (or none, all-spill)	compiles; the allocator consumes the banks
3. Emission	`.s` preamble + `as`/`ld` wiring	an empty `.s` assembles and links to an object
4. Minimal ISA	materialise a constant, the return, the simplest prologue	`int main(void){return N;}` exits with `N`
5. Grow ISA	ALU, then loads/stores, then compares/branches, then jumps	one difftest program per opcode passes
6. Calls	argument marshalling and the return value	multi-argument calls and `printf` match the oracle
7. Frame	`alloca`, spills, stack arguments, large frames	many-locals and deep-expression programs match
8. Floating	FP bank, FP selection, the FP half of the ABI	the floating-point difference corpus passes
9. Runtime	syscalls, `_start`, the exact link line	`kcc file.c -o out` runs in one step

When the last row is green the new target passes the same difference corpus the flagship does, and every row before it was verified in isolation. That is the whole promise of the seam: retargeting is transcription against a checklist, not invention.

2.8. Testing a Cross Target

kcc itself is an ARM64 executable; a new target's output runs somewhere else, so testing is cross-compilation checked against an oracle. The oracle is the reference compiler (`clang`) asked to cross-compile the same source for the same target; the difference harness compiles a program with both kcc and the oracle, runs both outputs, and compares. Only the *execution* of the output differs from the native flow: an x86-64 Mach-O output runs directly under the Apple-silicon binary translator; an x86-64 Linux/ELF output runs under a user-mode emulator or on real hardware. Choosing a first target whose output can be executed on the development machine—so the existing difference harness applies with only the execution command changed—removes an entire category of friction from the initial bring-up and is strongly preferred for the first new target.

The milestone ladder doubles as the test plan: return value, then arithmetic, then `printf`, then loops, then aggregates, then the full corpus. Because the front end, IR, and optimizer are shared and already verified, a divergence on a new target is almost always in the one back-end step most recently added—which is exactly why the playbook adds one thing at a time.

2.9. What Stays Owned, What Is Borrowed

To close, the boundary restated, because it is the point of the whole design. kcc owns the compiler: the front end that turns C into an IR, the optimizer that improves the IR, the allocator that places values in registers, the selection that chooses instructions, and the ABI that makes functions interoperate. For non-flagship targets it borrows two things from the platform assembler—the encoding of instructions into bytes and the layout of the object file—because those are the parts that multiply per architecture and per format and that the system toolchain already does perfectly. The flagship target owns even those, and stands as the proof that kcc can build the whole floor when it chooses to. This split keeps the from-scratch claim exactly where it is most meaningful and makes breadth across architectures a matter of transcription rather than invention—which is what a compiler meant for the present and the future needs.

3. Known Limitations

kcc compiles the large majority of real C17, but it is not complete. This appendix catalogues what it does *not* do. The list is deliberately exhaustive and blunt: a compiler is only trustworthy if its boundary is stated as plainly as its capabilities. The invariant behind every entry is that kcc *refuses* an unsupported construct with a diagnostic rather than emitting wrong code—a clean error is a feature, silent miscompilation never is.

3.1. Language Features

Variable-length arrays (VLAs). An array whose size is a run-time value, `int a[n]`, is not supported. VLAs require the object's size to be computed at run time and the stack frame to grow dynamically (a run-time `alloca`), plus run-time `sizeof` and pointer arithmetic against a size that is not a compile-time constant. kcc's frame layout is fixed at compile time, so a VLA declaration is rejected. Fixed-size and flexible-array-member forms are fully supported; only the run-time-bound automatic array is not.

`_Atomic` and `_Complex`. The complex type specifiers `_Complex`/`_Imaginary` (§ 6.2.5) are not implemented and are rejected in the declaration specifiers: they need a two-component value representation and complex arithmetic lowering. The atomic qualifier `_Atomic` (§ 6.7.2.4) *is* accepted as a type qualifier, but with *relaxed*, non-atomic semantics—it is recorded and then ignored, so accesses compile to ordinary loads and stores rather than lock-free atomic sequences (LDXR/STXR loops). The `_Atomic(`*`type-name`*`)` specifier form and the `<stdatomic.h>` generic machinery are not modelled. This is safe for single-threaded programs; it is *not* a correct concurrency primitive, which is why it is called out here rather than claimed.

`__int128` arithmetic. The GNU 128-bit integer type `__int128` (and `unsigned __int128`) *is* recognised as a type: it has size and alignment 16, lays out correctly inside structs and unions, and—because kcc classifies a 16-byte value like a two-eightbyte aggregate—can be copied, assigned with plain =, passed and returned by value, and read or written through a union of two `unsigned long` halves. That is exactly the surface the Linux kernel's `cmpxchg16b` path uses (`union __u128_halves` plus inline assembly on the halves).

The self-hosting (Linux/x86-64) port additionally lowers the 128-bit operations the kernel needs—an unsigned multiply and a logical right shift (`math64.h`'s `mul_u64_u32_shr` / `mul_u64_u64_shr`), plus equality and inequality (==, !=)—by materialising a `__int128` rvalue as a sixteen-byte stack slot and computing on the two halves with ordinary 64-bit operations (Section 75.6). Everything else—add, subtract, divide, modulo, left shift, *ordering* comparison ($<$, $>$, $\leq$, $\geq$), compound assignment, unary negation, and a *signed* right shift—still needs carry-aware two-word arithmetic that is not yet emitted, so the semantic analyser rejects it rather than silently drop the high 64 bits. The assembly reference goes further and rejects *all* `__int128` arithmetic, as it does the other kernel-only builtins: register-pair lowering with carry (ADDS/ADC) in the direct back ends remains future work.

Fully general extended inline assembly. Inline assembly works through the textual back end (§1.9)—including register, immediate, memory, and per-CPU symbolic operands and the static-key `asm goto` pattern—but not in full generality: inline asm with more live operands than the back end can place, and `asm goto` with arbitrary outputs, are refused with a diagnostic. The obstacle is structural: an asm operand's virtual register is carried on the AST node rather than in an instruction's operand fields, so it is invisible to the middle-end passes (copy propagation, common-subexpression elimination, dead-code elimination) and to the register allocator. Making `IR_ASM` a first-class multi-operand instruction—operand vregs stored in an array that every pass iterates, with real use/def/clobber sets—is the future project that lifts this ceiling (and also fixes the IR dumper, which faults on `IR_ASM`).

Attribute layout at member granularity. `__attribute__((packed))` and `aligned(N)` are honoured at the aggregate level, but not on an individual member (`int x __attribute__((packed));`). Other attributes are parsed and ignored, which is correct for the hint attributes but means member-level layout control is absent.

`volatile` through an aggregate object. A `volatile`-qualified *scalar* object is honoured, and a member or bit-field declared `volatile` is honoured, but `volatile` applied to an aggregate *object* does not yet propagate to a member access on it (`vs.x`, where `vs` is a `volatile struct`): that access is currently treated as non-volatile. This is the one remaining tail corner of C17's type-qualifier semantics; every other use of `volatile`—the CSE barrier, the guarantee that a volatile load or store is neither elided nor reordered away—is in place.

3.2. Standard Library and Headers

kcc ships a set of freestanding built-in headers (`stddef.h`, `stdint.h`, `stdarg.h`, `stdbool.h`, `limits.h`, `float.h`) and enough of `stdio.h`, `stdlib.h`, `string.h`, `math.h`, `fcntl.h`, and `unistd.h` to build real programs against the system libraries. It does *not* provide a full hosted standard library: many `<...>` headers are incomplete or absent, and POSIX headers beyond the handful above are not modelled. A program that includes an unmodelled header, or uses a declaration kcc's headers do not declare, will not compile until the missing declarations are supplied (via `-I` and the real system headers, or by extending kcc's built-in set).

3.3. Code Generation and Targets

No target beyond ARM64 and x86-64. kcc targets ARM64 (Apple-silicon macOS / Mach-O) and x86-64 (macOS and Linux/ELF); no other architecture or object format is implemented. The two are not equally mature. The x86-64 back end does *not* allocate registers—an all-spill model puts every value in a stack slot, which is correct but makes the generated code large and slow (an x86 register allocator is the main outstanding piece)—and it refuses, cleanly, a few by-value aggregate-argument shapes it cannot classify and most `__int128` operations (§3.1).

Optimisation scope. The optimiser performs constant folding, dead-code elimination, common-subexpression elimination, a scalar mem2reg promotion, strength reduction, and copy propagation. It does *not* perform loop optimisations, inlining, vectorisation, or global (inter-procedural) analysis. Register allocation is a linear scan on the ARM64 target; the x86-64 target does not allocate registers at all (see above). Generated code is correct but not aggressively optimised.

Debug information granularity. Under `-g`, DWARF is emitted at function granularity (each function's start), which is enough to name functions in a backtrace but not to map individual statements or to describe local variables.

3.4. The Standing Rule

Every item above shares one property: when kcc cannot compile a construct, it says so. It does not guess, it does not emit a plausible-looking wrong instruction, and it does not silently drop the construct. The boundary moves outward as features land; until then, an honest refusal keeps every program kcc *does* accept trustworthy.

The Assembly of the Program

Throughout this book each idea is told as its own story, and its code is introduced as a *named fragment* wherever the narrative is clearest—the lexer's fragments in the lexer chapter, the register allocator's in the backend, and so on, in the order that best serves a reader rather than a machine. This chapter is where those scattered fragments are finally *assembled*: it is the single place that says, for each physical file the computer requires, which fragments compose it and in what order.

The word "assembly" carries both of its meanings here. These are the *assembly-language* files—one `.s` per module—that the system assembler turns into object code; and this chapter is the *assembling* of the program from its parts, the bill of materials that gathers a book's worth of prose-embedded fragments into the concrete artifacts a build produces. Reading a single entry below tells you exactly what a given output file is made of: its components, listed by name, resolved (through the fragment definitions in the chapters) down to the last instruction.

Because a literate program separates *where a fragment is explained* from *where it is emitted*, this is the only file in kcc that uses output directives (`@o`); every other file merely *defines* fragments (`@d`). That separation is what lets the narrative flow by concept while the build still receives well-formed, correctly ordered assembly files. It also makes the manifest a practical checklist: to add a new module to the compiler, add its output directive here and its fragments to the appropriate chapter—the two never drift, because the file the machine consumes is generated from exactly these lists.

The directives below route code fragments to their physical assembly files under `gen/src/`.

main.s — the compiler entry point:

⟨ `src/main.s` 27a ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Main: entry point* 621, … ⟩
⟨ *Main: data* 625 ⟩
□
27a

syscalls.s — macOS system call wrappers:

⟨ `src/syscalls.s` 27b ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Syscall: constants* 53a ⟩
⟨ *Syscall: exit* 53b ⟩
⟨ *Syscall: write* 53c ⟩
⟨ *Syscall: read* 53d ⟩
⟨ *Syscall: open* 54a ⟩
⟨ *Syscall: close* 54b ⟩
⟨ *Syscall: mmap* 54c ⟩
⟨ *Syscall: lseek* 54d ⟩
⟨ *Syscall: process* 54e ⟩
□
27b

arena.s — arena memory allocator:

⟨ `src/arena.s` 27c ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Syscall: constants* 53a ⟩
⟨ *Arena: constants* 57a ⟩
⟨ *Arena: init* 57b ⟩
⟨ *Arena: alloc fast path* 58a ⟩
⟨ *Arena: alloc slow path* 58b ⟩
⟨ *Arena: data* 59 ⟩
□
27c

strings.s — string and memory operations:

⟨ `src/strings.s` 27d ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *String: strlen* 67a ⟩
⟨ *String: memcpy* 67b ⟩
⟨ *String: memcmp* 67c ⟩
⟨ *String: memset* 67d ⟩
⟨ *String: strcmp* 68 ⟩
□
27d

io.s — buffered file I/O:

⟨ `src/io.s` 27e ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *IO: flush* 69b ⟩
⟨ *IO: putc* 69c ⟩
⟨ *IO: puts* 70a ⟩
⟨ *IO: write* 70b ⟩
⟨ *IO: putu* 71a ⟩
⟨ *IO: err* 71b ⟩
⟨ *IO: data* 69a ⟩
□
27e

hashtab.s — hash table (FNV-1a, open addressing):

⟨ `src/hashtab.s` 27f ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Hash: constants* 61b ⟩
⟨ *Hash: FNV-1a* 61a ⟩
⟨ *Hash: init* 61c ⟩
⟨ *Hash: get* 62 ⟩
⟨ *Hash: put* 63 ⟩
□
27f

vec.s — dynamic vector:

⟨ src/vec.s 27g ⟩ ≡

⟨ ARM64: common macros 51 ⟩
⟨ Vec: constants 64a ⟩
⟨ Vec: init 64b ⟩
⟨ Vec: push 65a ⟩
⟨ Vec: get 65b ⟩

□

27g

strtab.s — string interning:

⟨ src/strtab.s 28a ⟩ ≡

⟨ ARM64: common macros 51 ⟩
⟨ Strtab: init 66a ⟩
⟨ Strtab: intern 66b ⟩
⟨ Strtab: data 66c ⟩

□

28a

diag.s — diagnostic engine:

⟨ src/diag.s 28b ⟩ ≡

⟨ ARM64: common macros 51 ⟩
⟨ Diag: constants 73a ⟩
⟨ Diag: putu stderr 73b ⟩
⟨ Diag: emit 73c ⟩
⟨ Diag: caret 75 ⟩
⟨ Diag: error 76a ⟩
⟨ Diag: warning 76b ⟩
⟨ Diag: note 76c ⟩
⟨ Diag: had errors 76d ⟩
⟨ Diag: reset 76e ⟩
⟨ Diag: data 77a ⟩

□

28b

source.s — source file management:

⟨ src/source.s 28c ⟩ ≡

⟨ ARM64: common macros 51 ⟩
⟨ Syscall: constants 53a ⟩
⟨ Source: constants 81a ⟩
⟨ Source: open file 81b ⟩
⟨ Encoding: translate 85 ⟩
⟨ Splice: translate 87 ⟩
⟨ Source: data 82 ⟩

□

28c

lexer.s — C17 tokenizer:

⟨ src/lexer.s 28d ⟩ ≡

⟨ ARM64: common macros 51 ⟩
⟨ Lexer: token constants 89 ⟩
⟨ Lexer: init 90a ⟩
⟨ Lexer: record location 92 ⟩
⟨ Lexer: next token 90b, … ⟩
⟨ Lexer: scan identifier 93 ⟩
⟨ Lexer: scan number 94a ⟩
⟨ Lexer: scan string 94b ⟩
⟨ Lexer: scan char 95 ⟩
⟨ Lexer: scan punctuator 96, … ⟩
⟨ Lexer: dump token 99 ⟩
⟨ Lexer: dump all 100a ⟩
⟨ Lexer: data 104b ⟩

□

28d

lexer_tables.s — keyword table and token names:

⟨ src/lexer_tables.s 28e ⟩ ≡

⟨ ARM64: common macros 51 ⟩
⟨ Lexer: keyword data 103 ⟩
⟨ Lexer: keyword init 104a ⟩
⟨ Lexer: token names 100b, … ⟩

□

28e

preprocessor.s — C17 preprocessor (Phase 4):

⟨ src/preprocessor.s 28f ⟩ ≡

⟨ ARM64: common macros 51 ⟩
⟨ Lexer: token constants 89 ⟩
⟨ PP: constants 107a ⟩
⟨ PP: init 107b ⟩
⟨ PP: helpers 113 ⟩
⟨ PP: context 111, … ⟩
⟨ PP: eval 141, … ⟩
⟨ PP: predefined 132 ⟩
⟨ PP: expand 121, … ⟩
⟨ PP: define 117, … ⟩
⟨ PP: undef 119a ⟩
⟨ PP: pragma 119b ⟩
⟨ PP: conditional 135, … ⟩
⟨ PP: resolve include 148 ⟩
⟨ PP: include path 150 ⟩
⟨ PP: include 151 ⟩
⟨ Preprocessor: builtin headers 153 ⟩
⟨ PP: error directive 159b ⟩
⟨ PP: directive dispatch 109, … ⟩
⟨ PP: next token 108 ⟩
⟨ PP: dump 115 ⟩
⟨ StringLit: process 161, … ⟩
⟨ Concat: adjacent strings 165 ⟩
⟨ PP: data 116 ⟩
⟨ StringLit: data 163 ⟩
⟨ Concat: data 166 ⟩

□

28f

parser.s — recursive-descent parser core:

⟨ src/parser.s 28g ⟩ ≡

⟨ ARM64: common macros 51 ⟩
⟨ Lexer: token constants 89 ⟩
⟨ Diag: constants 73a ⟩
⟨ Vec: constants 64a ⟩
⟨ Parser: AST constants 167 ⟩
⟨ Parser: AST alloc 168a ⟩
⟨ Parser: helpers 168b ⟩

⟨ *Parser: error recovery* 169 ⟩
⟨ *Parser: typedef table* 170a, … ⟩
⟨ *Parser: translation unit* 172 ⟩
⟨ *Parser: dump AST* 173, … ⟩
⟨ *Parser: dump helpers* 177 ⟩
⟨ *Parser: data* 178a, … ⟩

□

28g

expressions.s — expression parsing:

⟨ src/expressions.s 29a ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Lexer: token constants* 89 ⟩
⟨ *Parser: AST constants* 167 ⟩
⟨ *Expr: precedence table* 207a ⟩
⟨ *Expr: parse expression* 207b ⟩
⟨ *Expr: parse assign* 208a ⟩
⟨ *Expr: parse conditional* 208b ⟩
⟨ *Expr: parse binary* 209a ⟩
⟨ *Expr: parse unary* 209b, … ⟩
⟨ *Expr: parse cast* 211 ⟩
⟨ *Expr: parse postfix* 212, … ⟩
⟨ *Expr: parse primary* 215, … ⟩
⟨ *Expr: parse float* 220 ⟩
⟨ *Expr: parse arg list* 223a ⟩
⟨ *Expr: data* 223b ⟩

□

29a

statements.s — statement parsing:

⟨ src/statements.s 29b ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Lexer: token constants* 89 ⟩
⟨ *Parser: AST constants* 167 ⟩
⟨ *Stmt: parse statement* 225, … ⟩
⟨ *Stmt: parse asm* 227 ⟩
⟨ *Stmt: parse compound* 230 ⟩
⟨ *Stmt: parse if* 231a ⟩
⟨ *Stmt: parse switch* 231b ⟩
⟨ *Stmt: parse while* 232a ⟩
⟨ *Stmt: parse do* 232b ⟩
⟨ *Stmt: parse for* 233 ⟩
⟨ *Stmt: parse goto* 234a ⟩
⟨ *Stmt: parse return* 234b ⟩
⟨ *Stmt: parse labeled* 234c ⟩
⟨ *Stmt: data* 235 ⟩

□

29b

declarations.s — declaration parsing:

⟨ src/declarations.s 29c ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Lexer: token constants* 89 ⟩
⟨ *Parser: AST constants* 167 ⟩
⟨ *Decl: parse declaration* 181, … ⟩
⟨ *Decl: parse specifiers* 184b, … ⟩
⟨ *Decl: parse declarator* 190 ⟩
⟨ *Decl: parse direct declarator* 191b, … ⟩
⟨ *Decl: parse abstract* 196 ⟩
⟨ *Decl: parse struct* 198, … ⟩
⟨ *Decl: parse enum* 200b ⟩
⟨ *Decl: parse type name* 201 ⟩
⟨ *Decl: is type token* 202 ⟩
⟨ *Decl: parse init* 203 ⟩
⟨ *Decl: parse static assert* 205a ⟩
⟨ *Decl: data* 205b ⟩

□

29c

types.s — type system:

⟨ src/types.s 29d ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Lexer: token constants* 89 ⟩
⟨ *Parser: AST constants* 167 ⟩
⟨ *Types: constants* 239a ⟩
⟨ *Types: init* 239b, … ⟩
⟨ *Types: make pointer* 242a, … ⟩
⟨ *Types: make array* 243a ⟩
⟨ *Types: make function* 243b ⟩
⟨ *Types: make struct* 243c ⟩
⟨ *Types: make enum* 244 ⟩
⟨ *Types: resolve specmask* 245a, … ⟩
⟨ *Types: compatible* 247a ⟩
⟨ *Types: is integer* 247b ⟩
⟨ *Types: is arithmetic* 248a ⟩
⟨ *Types: is scalar* 248b ⟩
⟨ *Types: rank* 248c ⟩
⟨ *Types: dump* 249a, … ⟩
⟨ *Types: data* 250b ⟩

□

29d

symtab.s — symbol table:

⟨ src/symtab.s 29e ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Lexer: token constants* 89 ⟩
⟨ *Parser: AST constants* 167 ⟩
⟨ *Diag: constants* 73a ⟩
⟨ *Vec: constants* 64a ⟩
⟨ *Symtab: constants* 253a ⟩
⟨ *Symtab: init* 253b ⟩
⟨ *Symtab: push scope* 253c ⟩
⟨ *Symtab: pop scope* 254a ⟩
⟨ *Symtab: declare* 254b ⟩
⟨ *Symtab: lookup* 255 ⟩
⟨ *Symtab: declare tag* 256a ⟩
⟨ *Symtab: lookup tag* 256b ⟩
⟨ *Symtab: label scope* 257a, … ⟩
⟨ *Symtab: data* 258 ⟩

□

29e

conversions.s — type conversions:

⟨ src/conversions.s 29f ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Lexer: token constants* 89 ⟩

⟨ *Parser: AST constants* 167 ⟩
⟨ *Types: constants* 239a ⟩
⟨ *Conv: integer promotions* 295a ⟩
⟨ *Conv: usual arithmetic* 295b ⟩
⟨ *Conv: array decay* 296a ⟩
⟨ *Conv: func decay* 296b ⟩
⟨ *Conv: lvalue* 297a ⟩
⟨ *Conv: implicit* 297b ⟩
□
29f

sema.s — semantic analysis:

⟨ `src/sema.s` 30a ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Lexer: token constants* 89 ⟩
⟨ *Diag: constants* 73a ⟩
⟨ *Parser: AST constants* 167 ⟩
⟨ *Types: constants* 239a ⟩
⟨ *Symtab: constants* 253a ⟩
⟨ *Vec: constants* 64a ⟩
⟨ *Hash: constants* 61b ⟩
⟨ *Sema: check* 259a ⟩
⟨ *Sema: resolve type* 259b ⟩
⟨ *Sema: check declaration* 264, … ⟩
⟨ *Sema: struct layout* 267 ⟩
⟨ *Sema: check statement* 271, … ⟩
⟨ *Sema: return path* 281 ⟩
⟨ *Sema: check expression* 274, … ⟩
⟨ *Sema: expr ident* 277 ⟩
⟨ *Sema: expr literal* 283 ⟩
⟨ *Sema: expr binary* 284 ⟩
⟨ *Sema: expr unary* 286 ⟩
⟨ *Sema: expr assign* 287 ⟩
⟨ *Sema: expr call* 289 ⟩
⟨ *Sema: expr member* 290 ⟩
⟨ *Sema: expr cast* 292a ⟩
⟨ *Sema: expr sizeof* 292b ⟩
⟨ *Sema: dump types* 262 ⟩
⟨ *Sema: data* 263 ⟩
□
30a

ir.s — intermediate representation:

⟨ `src/ir.s` 30b ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Lexer: token constants* 89 ⟩
⟨ *Parser: AST constants* 167 ⟩
⟨ *Types: constants* 239a ⟩
⟨ *Vec: constants* 64a ⟩
⟨ *Symtab: constants* 253a ⟩
⟨ *IR: constants* 301 ⟩
⟨ *IR: init* 302 ⟩
⟨ *IR: alloc instruction* 303a ⟩
⟨ *IR: alloc basic block* 303b ⟩
⟨ *IR: alloc function* 303c ⟩
⟨ *IR: emit* 304 ⟩
⟨ *IR: next vreg* 305a ⟩
⟨ *IR: set current bb* 305b ⟩
⟨ *IR: build* 307, … ⟩
⟨ *IR: build function* 317 ⟩
⟨ *IR: build param* 320 ⟩
⟨ *IR: build goto* 321 ⟩
⟨ *IR: build statement* 322 ⟩
⟨ *IR: build asm operands* 325 ⟩
⟨ *IR: build return* 326 ⟩
⟨ *IR: build expression* 340, … ⟩
⟨ *IR: build int literal* 343 ⟩
⟨ *IR: build sizeof* 347 ⟩
⟨ *IR: access width* 349b ⟩
⟨ *IR: build subscript* 349a, … ⟩
⟨ *IR: build member* 355, … ⟩
⟨ *IR: variadic builtins* 398 ⟩
⟨ *IR: build ident* 367 ⟩
⟨ *IR: build binary* 369, … ⟩
⟨ *IR: build unary* 378b ⟩
⟨ *IR: build increment/decrement* 382, … ⟩
⟨ *IR: build assign* 387 ⟩
⟨ *IR: build call* 393 ⟩
⟨ *IR: build string literal* 400b ⟩
⟨ *IR: build conditional* 400c ⟩
⟨ *IR: build cast* 402b ⟩
⟨ *IR: struct copy helper* 404 ⟩
⟨ *IR: struct address helper* 405 ⟩
⟨ *IR: aggregate init helpers* 407, … ⟩
⟨ *IR: bitfield access* 359 ⟩
⟨ *IR: static local helper* 417a ⟩
⟨ *IR: build var decl* 417b ⟩
⟨ *IR: build if* 421c ⟩
⟨ *IR: build while* 423a ⟩
⟨ *IR: build do* 423b ⟩
⟨ *IR: build for* 424 ⟩
⟨ *IR: build switch* 426 ⟩
⟨ *IR: build break* 434b, … ⟩
⟨ *IR: build continue* 435b ⟩
⟨ *IR: validate* 441 ⟩
⟨ *IR: dump* 334 ⟩
⟨ *IR: dump instruction* 335, … ⟩
⟨ *IR: dump type short* 339 ⟩
⟨ *IR: data* 305c ⟩
⟨ *IR: builder data* 328 ⟩
□
30b

opt.s — optimization passes:

⟨ `src/opt.s` 30c ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *IR: constants* 301 ⟩
⟨ *Vec: constants* 64a ⟩
⟨ *Types: constants* 239a ⟩
⟨ *Opt: constant fold* 447, … ⟩
⟨ *Opt: strength reduction* 450 ⟩
⟨ *Opt: dead code elimination* 453a, … ⟩
⟨ *Opt: common subexpression elimination* 457 ⟩
⟨ *Opt: pass manager* 445a, … ⟩
□
30c

emit_arm64.s — ARM64 instruction binary encoding:

⟨ `src/emit_arm64.s` 30d ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Backend: constants* 525a ⟩
⟨ *Backend: code buffer* 525b, … ⟩

⟨ *Backend: encode arithmetic reg* 526b ⟩
⟨ *Backend: encode arithmetic imm* 527a ⟩
⟨ *Backend: encode logical* 527b, … ⟩
⟨ *Backend: encode compare* 528b ⟩
⟨ *Backend: encode memory* 529a, … ⟩
⟨ *Backend: encode floating point* 530b ⟩
⟨ *Backend: encode branch* 532a, … ⟩
⟨ *Backend: encode system* 533a ⟩
⟨ *Backend: encode adrp* 533b ⟩
⟨ *Backend: emit data* 533c ⟩
⟨ *Vec: constants* 64a ⟩
⟨ *Linker: constants* 591a ⟩
⟨ *Emit-asm: data* 535a ⟩
⟨ *Emit-asm: buffer* 535b ⟩
⟨ *Emit-asm: render* 537, … ⟩

□

30d

emit_x86.s — the second target: an x86-64 back end that emits textual assembly for the system assembler. It reuses the growable text buffer of **emit_arm64.s** (the `_kcc_as_*` helpers), so it only supplies the IR walk and x86-64 instruction selection:

⟨ `src/emit_x86.s` 31a ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *IR: constants* 301 ⟩
⟨ *Vec: constants* 64a ⟩
⟨ *Types: constants* 239a ⟩
⟨ *Parser: AST constants* 167 ⟩
⟨ *X86: data* 546, … ⟩
⟨ *X86: render* 548a, … ⟩

□

31a

linker.s — relocation and symbol table management:

⟨ `src/linker.s` 31b ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Vec: constants* 64a ⟩
⟨ *Linker: constants* 591a ⟩
⟨ *Linker: init* 591b ⟩
⟨ *Linker: add symbol* 592a ⟩
⟨ *Linker: add reloc* 592b ⟩
⟨ *Linker: lookup or add extern* 592c ⟩
⟨ *Linker: global sym* 593 ⟩
⟨ *Linker: data* 594 ⟩

□

31b

codegen.s — IR to ARM64 code generation:

⟨ `src/codegen.s` 31c ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *IR: constants* 301 ⟩
⟨ *Vec: constants* 64a ⟩
⟨ *Linker: constants* 591a ⟩
⟨ *Types: constants* 239a ⟩
⟨ *Parser: AST constants* 167 ⟩
⟨ *CodeGen: constants* 463a ⟩
⟨ *CodeGen: init* 463b ⟩
⟨ *CodeGen: preregister globals* 465, … ⟩
⟨ *CodeGen: compile all* 472, … ⟩
⟨ *CodeGen: register allocation* 481 ⟩
⟨ *CodeGen: slot colouring* 490 ⟩
⟨ *CodeGen: compile function* 493, … ⟩
⟨ *CodeGen: large immediate adjust* 495 ⟩
⟨ *CodeGen: vreg slot offset* 499a, … ⟩
⟨ *CodeGen: marshal args* 502 ⟩
⟨ *CodeGen: instruction dispatch* 504, … ⟩
⟨ *CodeGen: asm* 522b ⟩
⟨ *CodeGen: floating point* 511 ⟩
⟨ *CodeGen: patch branches* 474 ⟩
⟨ *CodeGen: emit strings* 475, … ⟩
⟨ *CodeGen: data* 479 ⟩

□

31c

emit_macho.s — Mach-O object file writer:

⟨ `src/emit_macho.s` 31d ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Vec: constants* 64a ⟩
⟨ *Linker: constants* 591a ⟩
⟨ *MachO: constants* 577a ⟩
⟨ *MachO: output buffer* 577b, … ⟩
⟨ *MachO: finalize* 580, … ⟩
⟨ *MachO: data* 590 ⟩

□

31d

⟨ `src/dwarf.s` 31e ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Vec: constants* 64a ⟩
⟨ *IR: constants* 301 ⟩
⟨ *DWARF: constants* 595a ⟩
⟨ *DWARF: emit* 595b ⟩
⟨ *DWARF: build* 596 ⟩
⟨ *DWARF: macho* 600 ⟩
⟨ *DWARF: data* 601 ⟩

□

31e

driver.s — compilation driver:

⟨ `src/driver.s` 31f ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Driver: compile* 603, … ⟩
⟨ *Driver: link* 611 ⟩
⟨ *Driver: data* 615 ⟩

□

31f

include/*.h — hosted library header bridges. Unlike every other `@o` target these are not assembly: they are C headers tangled to `gen/include/` beside the compiler binary, where the default include directory finds them (§17.16.3). Section markers are suppressed (`-s`) so the shipped headers read cleanly:

⟨ include/stdio.h 31g ⟩ ≡

⟨ Header: stdio 154 ⟩

□

31g

⟨ include/stdlib.h 32a ⟩ ≡

⟨ Header: stdlib 155a ⟩

□

32a

⟨ include/string.h 32b ⟩ ≡

⟨ Header: string 155b ⟩

□

32b

⟨ include/ctype.h 32c ⟩ ≡

⟨ Header: ctype 155c ⟩

□

32c

⟨ include/math.h 32d ⟩ ≡

⟨ Header: math 156a ⟩

□

32d

⟨ include/assert.h 32e ⟩ ≡

⟨ Header: assert 156b ⟩

□

32e

⟨ include/time.h 32f ⟩ ≡

⟨ Header: time 156c ⟩

□

32f

⟨ include/errno.h 32g ⟩ ≡

⟨ Header: errno 156d ⟩

□

32g

⟨ include/inttypes.h 32h ⟩ ≡

⟨ Header: inttypes 157a ⟩

□

32h

⟨ include/signal.h 32i ⟩ ≡

⟨ Header: signal 157b ⟩

□

32i

⟨ include/setjmp.h 32j ⟩ ≡

⟨ Header: setjmp 157c ⟩

□

32j

⟨ include/locale.h 32k ⟩ ≡

⟨ Header: locale 157d ⟩

□

32k

⟨ include/fenv.h 32l ⟩ ≡

⟨ Header: fenv 158a ⟩

□

32l

⟨ include/wchar.h 32m ⟩ ≡

⟨ Header: wchar 158b ⟩

□

32m

⟨ include/wctype.h 32n ⟩ ≡

⟨ Header: wctype 158c ⟩

□

32n

⟨ include/uchar.h 32o ⟩ ≡

⟨ Header: uchar 158d ⟩

□

32o

⟨ include/fcntl.h 32p ⟩ ≡

⟨ Header: fcntl 158e ⟩

□

32p

⟨ include/unistd.h 32q ⟩ ≡

⟨ Header: unistd 158f ⟩

□

32q

Test binaries — test runner and conformance tests (assembled separately; linked into `gen/test_kcc`):

test_runner.s — test harness entry point:

⟨ tests/test_runner.s 32r ⟩ ≡

⟨ ARM64: common macros 51 ⟩
⟨ Test: runner entry 629 ⟩
⟨ Test: assert helpers 630a ⟩
⟨ Test: runner data 630b ⟩

□

32r

test_lexer.s — lexer conformance tests:

⟨ tests/test_lexer.s 33a ⟩ ≡

⟨ *ARM64: common macros* 51 ⟩
⟨ *Lexer: token constants* 89 ⟩
⟨ *Test: lex keywords* 633a ⟩
⟨ *Test: lex numbers* 633b ⟩
⟨ *Test: lex strings* 633c ⟩
⟨ *Test: lex punctuators* 634a ⟩
⟨ *Test: lex comments* 634b ⟩
⟨ *Test: lex trigraphs* 635a ⟩
⟨ *Test: lex line splice* 635b ⟩
⟨ *Test: registration* 636a ⟩
⟨ *Test: lex data* 636b ⟩

□

33a

test_codegen.sh — end-to-end code generation tests:

⟨ tests/test_codegen.sh 33b ⟩ ≡

⟨ *Test: codegen runner* 637 ⟩
⟨ *Test: codegen constants* 522a ⟩
⟨ *Test: codegen locals* 420 ⟩
⟨ *Test: codegen control flow* 425 ⟩
⟨ *Test: codegen increment* 386 ⟩
⟨ *Test: codegen compound* 392 ⟩
⟨ *Test: codegen logical* 378a ⟩
⟨ *Test: codegen functions* 400a ⟩
⟨ *Test: codegen pointers* 381 ⟩
⟨ *Test: codegen misc* 435c ⟩
⟨ *Test: codegen ternary* 402a ⟩
⟨ *Test: codegen switch* 434a ⟩
⟨ *Test: codegen sizeof* 348 ⟩
⟨ *Test: codegen arrays* 353b ⟩
⟨ *Test: codegen structs* 366 ⟩
⟨ *Test: codegen multi-var* 421a ⟩
⟨ *Test: codegen unsigned* 421b ⟩
⟨ *Test: codegen enums* 435d ⟩
⟨ *Test: codegen nested structs* 436a ⟩
⟨ *Test: codegen struct arrays* 436b ⟩
⟨ *Test: codegen global init* 436c ⟩
⟨ *Test: codegen ptr arithmetic* 436d ⟩
⟨ *Test: codegen pointer arrays* 437a ⟩
⟨ *Test: codegen init lists* 437b ⟩
⟨ *Test: codegen static locals* 438a ⟩
⟨ *Test: codegen preprocessor* 159a ⟩
⟨ *Test: codegen multifile* 616a ⟩
⟨ *Test: codegen self-hosting* 438b ⟩
⟨ *Test: codegen fnptr* 439 ⟩
⟨ *Test: codegen results* 638 ⟩

□

33b

arena.c — kcc's arena allocator, ported to C (Chapter 48):

⟨ selfhost/arena.c 33c ⟩ ≡

⟨ *Selfhost: arena* 640a ⟩

□

33c

selfhost.sh — compile it with kcc, relink, and re-run the codegen suite:

⟨ selfhost/selfhost.sh 33d ⟩ ≡

⟨ *Selfhost: verify* 640b ⟩

□

33d

lexer.c — a tokenizer in C (Chapter 77), with its self-test:

⟨ selfhost/lexer.c 33e ⟩ ≡

⟨ *Selfhost: lexer* 643c ⟩

□

33e

lexer_verify.sh — compile it with kcc, run the self-test, and diff against clang:

⟨ selfhost/lexer_verify.sh 33f ⟩ ≡

⟨ *Selfhost: lexer verify* 644 ⟩

□

33f

vec.c — the growable vector in C (Chapter 50), a drop-in replacement:

⟨ selfhost/vec.c 33g ⟩ ≡

⟨ *Selfhost: vec* 646a ⟩

□

33g

vec_verify.sh — compile it with kcc, relink the compiler with it, and re-run the code-generation suite:

⟨ selfhost/vec_verify.sh 33h ⟩ ≡

⟨ *Selfhost: vec verify* 646b ⟩

□

33h

strings.c — the string/memory primitives in C (Chapter 51), a drop-in replacement:

⟨ selfhost/strings.c 33i ⟩ ≡

⟨ *Selfhost: strings* 648a ⟩

□

33i

strings_verify.sh — compile them with kcc, relink the compiler with them, and re-run the code-generation suite:

⟨ selfhost/strings_verify.sh 33j ⟩ ≡

⟨ *Selfhost: strings verify* 648b ⟩

□

33j

hashtab.c — the FNV-1a hash table in C (Chapter 52), a drop-in replacement:

⟨ `selfhost/hashtab.c` 34a ⟩ ≡

⟨ *Selfhost: hashtab* 650c ⟩

□

34a

hashtab_verify.sh — compile it with kcc, relink the compiler with it, and re-run the code-generation suite:

⟨ `selfhost/hashtab_verify.sh` 34b ⟩ ≡

⟨ *Selfhost: hashtab verify* 651 ⟩

□

34b

strtab.c — the string interner in C (Chapter 53), a drop-in replacement:

⟨ `selfhost/strtab.c` 34c ⟩ ≡

⟨ *Selfhost: strtab* 653a ⟩

□

34c

⟨ `selfhost/strtab_verify.sh` 34d ⟩ ≡

⟨ *Selfhost: strtab verify* 653b ⟩

□

34d

io.c — the buffered output layer in C (Chapter 54), a drop-in replacement:

⟨ `selfhost/io.c` 34e ⟩ ≡

⟨ *Selfhost: io* 656b ⟩

□

34e

⟨ `selfhost/io_verify.sh` 34f ⟩ ≡

⟨ *Selfhost: io verify* 656c ⟩

□

34f

source.c — the source manager and translation phases 1–2 in C (Chapter 55), a drop-in replacement:

⟨ `selfhost/source.c` 34g ⟩ ≡

⟨ *Selfhost: source* 659a ⟩

□

34g

⟨ `selfhost/source_verify.sh` 34h ⟩ ≡

⟨ *Selfhost: source verify* 659b ⟩

□

34h

diag.c — the diagnostic engine in C (Chapter 56), a drop-in replacement:

⟨ `selfhost/diag.c` 34i ⟩ ≡

⟨ *Selfhost: diag* 662b ⟩

□

34i

⟨ `selfhost/diag_verify.sh` 34j ⟩ ≡

⟨ *Selfhost: diag verify* 662c ⟩

□

34j

symtab.c — the symbol table in C (Chapter 57), a drop-in replacement:

⟨ `selfhost/symtab.c` 34k ⟩ ≡

⟨ *Selfhost: symtab* 664c ⟩

□

34k

⟨ `selfhost/symtab_verify.sh` 34l ⟩ ≡

⟨ *Selfhost: symtab verify* 665 ⟩

□

34l

conversions.c — the standard conversions in C (Chapter 58), a drop-in replacement:

⟨ `selfhost/conversions.c` 34m ⟩ ≡

⟨ *Selfhost: conversions* 668b ⟩

□

34m

⟨ `selfhost/conversions_verify.sh` 34n ⟩ ≡

⟨ *Selfhost: conversions verify* 668c ⟩

□

34n

The type system itself, ported to C (Chapter 59), a drop-in replacement:

⟨ `selfhost/types.c` 34o ⟩ ≡

⟨ *Selfhost: types* 672b ⟩

□
34o

⟨ selfhost/types_verify.sh 35a ⟩ ≡

⟨ *Selfhost: types verify* 673 ⟩
□
35a

The optimizer, ported to C (Chapter 60), a drop-in replacement:

⟨ selfhost/opt.c 35b ⟩ ≡

⟨ *Selfhost: opt* 681b ⟩
□
35b

⟨ selfhost/opt_verify.sh 35c ⟩ ≡

⟨ *Selfhost: opt verify* 681c ⟩
□
35c

The lexer's keyword and token-name tables, ported to C (Chapter 61), a drop-in replacement:

⟨ selfhost/lexer_tables.c 35d ⟩ ≡

⟨ *Selfhost: lexer-tables* 684a ⟩
□
35d

⟨ selfhost/lexer_tables_verify.sh 35e ⟩ ≡

⟨ *Selfhost: lexer-tables verify* 684b ⟩
□
35e

The linker-information tables, ported to C (Chapter 62), a drop-in replacement:

⟨ selfhost/linker.c 35f ⟩ ≡

⟨ *Selfhost: linker* 686b ⟩
□
35f

⟨ selfhost/linker_verify.sh 35g ⟩ ≡

⟨ *Selfhost: linker verify* 686c ⟩
□
35g

The statement parser, ported to C (Chapter 63), a drop-in replacement:

⟨ selfhost/statements.c 35h ⟩ ≡

⟨ *Selfhost: statements* 694b ⟩
□
35h

⟨ selfhost/statements_verify.sh 35i ⟩ ≡

⟨ *Selfhost: statements verify* 694c ⟩
□
35i

The Mach-O object writer, ported to C (Chapter 64), a drop-in replacement:

⟨ selfhost/emit_macho.c 35j ⟩ ≡

⟨ *Selfhost: emit-macho* 702a ⟩
□
35j

⟨ selfhost/emit_macho_verify.sh 35k ⟩ ≡

⟨ *Selfhost: emit-macho verify* 702b ⟩
□
35k

The expression parser, ported to C (Chapter 65), a drop-in replacement:

⟨ selfhost/expressions.c 35l ⟩ ≡

⟨ *Selfhost: expressions* 710a ⟩
□
35l

⟨ selfhost/expressions_verify.sh 35m ⟩ ≡

⟨ *Selfhost: expressions verify* 710b ⟩
□
35m

The parser spine, ported to C (Chapter 66), a drop-in replacement:

⟨ selfhost/parser.c 35n ⟩ ≡

⟨ *Selfhost: parser* 717a ⟩
□
35n

⟨ selfhost/parser_verify.sh 35o ⟩ ≡

⟨ *Selfhost: parser verify* 717b ⟩
□
35o

The compilation driver, ported to C (Chapter 67), a drop-in replacement:

⟨ selfhost/driver.c 35p ⟩ ≡

⟨ *Selfhost: driver* 725a ⟩

□
35p

⟨ selfhost/driver_verify.sh 36a ⟩ ≡

⟨ *Selfhost: driver verify* 725b ⟩
□
36a

The declaration parser, ported to C (Chapter 68), a drop-in replacement:

⟨ selfhost/declarations.c 36b ⟩ ≡

⟨ *Selfhost: declarations* 738a ⟩
□
36b

⟨ selfhost/declarations_verify.sh 36c ⟩ ≡

⟨ *Selfhost: declarations verify* 738b ⟩
□
36c

The DWARF debug-info emitter, ported to C (Chapter 69), a drop-in replacement:

⟨ selfhost/dwarf.c 36d ⟩ ≡

⟨ *Selfhost: dwarf* 742a ⟩
□
36d

⟨ selfhost/dwarf_verify.sh 36e ⟩ ≡

⟨ *Selfhost: dwarf verify* 742b ⟩
□
36e

The ARM64 encoder and textual-`.s` renderer, ported to C (Chapter 70), a drop-in replacement:

⟨ selfhost/emit_arm64.c 36f ⟩ ≡

⟨ *Selfhost: emit_arm64* 750b ⟩
□
36f

⟨ selfhost/emit_arm64_verify.sh 36g ⟩ ≡

⟨ *Selfhost: emit_arm64 verify* 751 ⟩
□
36g

The semantic analyzer, ported to C (Chapter 71), a drop-in replacement:

⟨ selfhost/sema.c 36h ⟩ ≡

⟨ *Selfhost: sema* 768a ⟩
□
36h

⟨ selfhost/sema_verify.sh 36i ⟩ ≡

⟨ *Selfhost: sema verify* 768b ⟩
□
36i

The x86-64 back end, ported to C (Chapter 72), a drop-in replacement:

⟨ selfhost/emit_x86.c 36j ⟩ ≡

⟨ *Selfhost: emit_x86* 792a ⟩
□
36j

⟨ selfhost/emit_x86_verify.sh 36k ⟩ ≡

⟨ *Selfhost: emit_x86 verify* 792b ⟩
□
36k

The preprocessor, ported to C (Chapter 73), a drop-in replacement:

⟨ selfhost/preprocessor.c 36l ⟩ ≡

⟨ *Selfhost: preprocessor* 818a ⟩
□
36l

⟨ selfhost/preprocessor_verify.sh 36m ⟩ ≡

⟨ *Selfhost: preprocessor verify* 818b ⟩
□
36m

The ARM64 code generator, ported to C (Chapter 74), a drop-in replacement:

⟨ selfhost/codegen.c 36n ⟩ ≡

⟨ *Selfhost: codegen* 836a ⟩
□
36n

⟨ selfhost/codegen_verify.sh 36o ⟩ ≡

⟨ *Selfhost: codegen verify* 836b ⟩
□
36o

The middle-end IR builder, ported to C (Chapter 75), a drop-in replacement — the last subsystem to leave assembly:

⟨ selfhost/ir.c 36p ⟩ ≡

⟨ *Selfhost: ir* 902a ⟩

□

36p

⟨ selfhost/ir_verify.sh 37a ⟩ ≡

⟨ *Selfhost: ir verify* 902b ⟩

□

37a

The entry point and CLI dispatch, ported to C (Chapter 76) — the last compiler code to leave assembly, and the shared entry for both targets:

⟨ selfhost/main.c 37b ⟩ ≡

⟨ *Selfhost: main* 905 ⟩

□

37b

⟨ selfhost/main_verify.sh 37c ⟩ ≡

⟨ *Selfhost: main verify* 909 ⟩

□

37c

The compiler's lexer, ported to C (Chapter 77) — a drop-in for the module whose token stream the whole front end consumes:

⟨ selfhost/lexer_core.c 37d ⟩ ≡

⟨ *Selfhost: lexcore* 911 ⟩

□

37d

⟨ selfhost/lexer_core_verify.sh 37e ⟩ ≡

⟨ *Selfhost: lexcore verify* 914 ⟩

□

37e

The second target's OS layer (Chapter 39): the ELF entry point and Linux syscall wrappers, assembled on Linux, that kcc's `--target=x86_64-linux` output links against. It is not part of the ARM64 compiler, so it routes to its own directory:

⟨ runtime/os_linux.s 37f ⟩ ≡

⟨ *Runtime-linux: file* 573a ⟩

□

37f

⟨ runtime/demo_util.c 37g ⟩ ≡

⟨ *Runtime-linux: demo util* 573b ⟩

□

37g

⟨ runtime/demo_main.c 37h ⟩ ≡

⟨ *Runtime-linux: demo main* 573c ⟩

□

37h

⟨ runtime/meta_a_verify.sh 37i ⟩ ≡

⟨ *Runtime-linux: meta-a verify* 573d ⟩

□

37i

⟨ runtime/meta_c_verify.sh 37j ⟩ ≡

⟨ *Runtime-linux: meta-c verify* 574 ⟩

□

37j

difftest.sh — compile each program with kcc and `clang`, run both, compare (Chapter 78):

⟨ tests/difftest.sh 37k ⟩ ≡

⟨ *Difftest: runner* 917 ⟩
⟨ *Difftest: arithmetic* 918a ⟩
⟨ *Difftest: control* 918b ⟩
⟨ *Difftest: functions* 919a ⟩
⟨ *Difftest: aggregates* 919b ⟩
⟨ *Difftest: struct layout* 921 ⟩
⟨ *Difftest: floating point* 922 ⟩
⟨ *Difftest: operators* 923 ⟩
⟨ *Difftest: combinations* 930 ⟩
⟨ *Difftest: language* 931 ⟩
⟨ *Difftest: divergences* 933 ⟩
⟨ *Difftest: gaps* 934a ⟩
⟨ *Difftest: results* 934b ⟩

□

37k

interop.sh — compile one translation unit with kcc and the other with `clang`, link, run, compare to the all-`clang` oracle (Chapter 79):

⟨ tests/interop.sh 37l ⟩ ≡

⟨ *Interop: runner* 935a ⟩
⟨ *Interop: cases* 935b ⟩
⟨ *Interop: results* 936 ⟩

□

37l

e2e.sh — drive the whole toolchain like a user: `kcc src.c -o out` then run `out` (Section 43.5):

⟨ tests/e2e.sh 37m ⟩ ≡

⟨ *E2E: runner* 616b ⟩

□

37m

diagtest.sh — feed the compiler invalid programs and check the diagnostics (Section 12.9):

⟨ tests/diagtest.sh 37n ⟩ ≡

⟨ *Diag: tests* 77b ⟩
□
37n

warntest.sh — feed the compiler valid programs and check that `-Wall` warns while the default build stays silent (Section 12.10):

⟨ tests/warntest.sh 38a ⟩ ≡

⟨ *Warn: tests* 78 ⟩
□
38a

Kernel-boot reproduction

The capstone demonstration of §1.17—kcc compiling `init/main.o` of a Linux 6.6 kernel that boots to a BusyBox login shell—tangles to a Docker build recipe and five numbered scripts under `gen/repro/`, with `run-all.sh` to chain the build steps. The prose for each lives with the manual section; here we only route the output.

⟨ repro/Dockerfile 38b ⟩ ≡
⟨ *Repro: Dockerfile* 12 ⟩
□
38b

⟨ repro/01-image.sh 38c ⟩ ≡
⟨ *Repro: 01 build image* 13a ⟩
□
38c

⟨ repro/02-build-kcc.sh 38d ⟩ ≡
⟨ *Repro: 02 build kcc* 13b ⟩
□
38d

⟨ repro/03-kernel.sh 38e ⟩ ≡
⟨ *Repro: 03 build kernel* 15a ⟩
□
38e

⟨ repro/04-initramfs.sh 38f ⟩ ≡
⟨ *Repro: 04 initramfs* 15b ⟩
□
38f

⟨ repro/05-boot.sh 38g ⟩ ≡
⟨ *Repro: 05 boot* 16 ⟩
□
38g

⟨ repro/run-all.sh 38h ⟩ ≡
⟨ *Repro: run all* 17 ⟩
□
38h

The build recipe

The build is not described in a file kept separately from the book—it is itself a tangled artifact. The chunk below emits the project `Makefile` into the repository root (the `../` escapes the `gen/` prefix that `-p` prepends to every other output), so the one command

```
weft -p gen/ --mkdirs kcc.weft
```

tangles the compiler's sources, the verification scripts, and the `Makefile` that drives them, all into `gen/` (and the root `Makefile`). From there, ordinary `make` targets do the rest. The generated sources live under `gen/` and are never edited by hand; `make clean` removes the whole directory, and the command above regenerates it.

The version string *itself* is single-sourced from the `version` fragment defined in the master document. Tangling it to a plain `gen/VERSION` file lets non-weft artifacts share that one value: the `pdf` target below substitutes it into the cover `.svg` templates (which carry a `{{VERSION}}` placeholder) before rasterising them, so the covers, the compiler's banner, and this book never disagree about which version they describe.

⟨ VERSION 38i ⟩ ≡
⟨ *version* ? ⟩
□
38i

The tutorial's first program (§1.4) is tangled here too, so a freshly built compiler has something to compile: `gen/hello.c`.

⟨ hello.c 38j ⟩ ≡
⟨ *Tutorial: hello world* 6 ⟩
□
38j

⟨ ../Makefile 38k ⟩ ≡

```
# kcc - A Formal C17 Compiler in ARM64 Assembly
# Build pipeline: .weft → weft tangle → .s → as →
   .o → ld → kcc
#
# This Makefile is itself a tangled artifact: it
   is generated into the repo
# root by `weft -p gen/ --mkdirs kcc.weft` (the @o
    ../Makefile chunk in
# literate/assembly.weft). Do not edit it here;
   edit the book.
#
# Written for the system make (GNU Make 3.81),
   which
# lacks .ONESHELL/.SHELLFLAGS, so each multi-
   command recipe is a single logical
# line (joined with \) that begins with `set -e`
   for fail-fast behaviour.
# Command echo is silenced (.SILENT) to match the
   original `set quiet`.

SHELL := /bin/bash
.SILENT:

# All assembly source basenames (order doesn't
   matter for linking)
```

```
sources := main syscalls arena strings io hashtab
   vec strtab source lexer lexer_tables diag
   preprocessor parser expressions statements
   declarations types symtab conversions sema ir
   opt emit_arm64 emit_x86 linker codegen
   emit_macho dwarf driver

# Library sources (everything except main - shared
    with the test binary)
lib_sources := syscalls arena strings io hashtab
   vec strtab source lexer lexer_tables diag
   preprocessor parser expressions statements
   declarations types symtab conversions sema ir
   opt emit_arm64 emit_x86 linker codegen
   emit_macho dwarf driver

# Test source basenames (in gen/tests/)
test_sources := test_runner test_lexer

# SDK root for the linker
SDKROOT := $(shell xcrun --show-sdk-path)
LDFLAGS := -lSystem -syslibroot $(SDKROOT) -e
   _main -arch arm64

.DEFAULT_GOAL := build

.PHONY: all default ci tangle assemble build test
   selfhost selfhost-lexer \
        selfhost-types selfhost-opt selfhost-lexer
   -tables selfhost-linker \
        selfhost-statements selfhost-emit-macho
   selfhost-expressions selfhost-parser selfhost-
   driver \
        selfhost-declarations selfhost-dwarf
   selfhost-emit-arm64 selfhost-sema \
        selfhost-emit-x86 selfhost-preprocessor
   selfhost-codegen selfhost-ir selfhost-main
   selfhost-lexer-core meta-a meta-c \
        difftest interop e2e diagtest warntest
   test-codegen weave pdf clean \
        run map

# Default: build everything
all default: build

# CI: the complete verification suite, in one
   entry point. Fast native checks
# run first (fail fast); the meta-* self-host
   milestones run last and need
# Docker + qemu-user. Prerequisites share `build`,
    so it is assembled once.
ci: build test difftest interop e2e diagtest
   warntest test-codegen meta-a meta-c
        echo "=== ci: all suites passed ==="

# Tangle: weave the literate sources into gen/ (
   sources, scripts, and this
# Makefile itself). -p prepends gen/ to every @o
   path; --mkdirs creates the
# output directories. This target regenerates the
   very Makefile that defines
# it, so the build description never drifts from
   the book.
tangle:
        weft -p gen/ --mkdirs kcc.weft

# Assemble: compile each .s to .o
assemble: tangle
        set -e; for f in $(sources); do as -o gen/
   src/$${f}.o gen/src/$${f}.s; done

# Link: produce the kcc binary
build: assemble
        set -e; OBJ=""; for f in $(sources); do
   OBJ="$$OBJ gen/src/$${f}.o"; done; \
        ld -o gen/kcc $$OBJ $(LDFLAGS)

# Test: build and run the unit-test suite
test: tangle
        set -e; \
        for f in $(lib_sources); do as -o gen/src/
   $${f}.o gen/src/$${f}.s; done; \
        for f in $(test_sources); do as -o gen/
   tests/$${f}.o gen/tests/$${f}.s; done; \
        OBJ=""; \
        for f in $(lib_sources); do OBJ="$$OBJ gen
   /src/$${f}.o"; done; \
        for f in $(test_sources); do OBJ="$$OBJ
   gen/tests/$${f}.o"; done; \
        ld -o gen/test_kcc $$OBJ $(LDFLAGS); \
        echo ""; \
        ./gen/test_kcc

# Self-hosting (Stage 8b): compile kcc's arena in
   C with kcc, relink, run suite
selfhost: build
        chmod +x gen/selfhost/selfhost.sh
        gen/selfhost/selfhost.sh

# Self-hosting (Stage 8b): compile a C tokenizer
   with kcc, self-test, diff clang
selfhost-lexer: build
        chmod +x gen/selfhost/lexer_verify.sh
        gen/selfhost/lexer_verify.sh

# Self-hosting (Stage 8c): compile kcc's vector in
    C with kcc, relink, run suite
selfhost-vec: build
        chmod +x gen/selfhost/vec_verify.sh
        gen/selfhost/vec_verify.sh

# Self-hosting (Stage 8d): compile kcc's string
   primitives in C, relink, run suite
selfhost-strings: build
        chmod +x gen/selfhost/strings_verify.sh
        gen/selfhost/strings_verify.sh

# Self-hosting (Stage 8e): compile kcc's hash
   table in C, relink, run suite
selfhost-hashtab: build
        chmod +x gen/selfhost/hashtab_verify.sh
        gen/selfhost/hashtab_verify.sh

# Self-hosting (Stage 8f): compile kcc's string
   interner in C, relink, run suite
selfhost-strtab: build
        chmod +x gen/selfhost/strtab_verify.sh
        gen/selfhost/strtab_verify.sh

# Self-hosting (Stage 8g): compile kcc's buffered
   output in C, relink, run suite
selfhost-io: build
        chmod +x gen/selfhost/io_verify.sh
        gen/selfhost/io_verify.sh

# Self-hosting (Stage 8h): compile kcc's source
   manager in C, relink, run suite
selfhost-source: build
        chmod +x gen/selfhost/source_verify.sh
        gen/selfhost/source_verify.sh

# Self-hosting (Stage 8i): compile kcc's
   diagnostic engine in C, relink, run suites
selfhost-diag: build
        chmod +x gen/selfhost/diag_verify.sh
        gen/selfhost/diag_verify.sh

# Self-hosting (Stage 8j): compile kcc's symbol
```

```
   table in C, relink, run suite
selfhost-symtab: build
        chmod +x gen/selfhost/symtab_verify.sh
        gen/selfhost/symtab_verify.sh

# Self-hosting (Stage 8k): compile kcc's
   conversions in C, relink, run suite
selfhost-conversions: build
        chmod +x gen/selfhost/conversions_verify.
   sh
        gen/selfhost/conversions_verify.sh

# Self-hosting (Stage 8l): compile kcc's type
   system in C, relink, run suite
selfhost-types: build
        chmod +x gen/selfhost/types_verify.sh
        gen/selfhost/types_verify.sh

# Self-hosting (Stage 8m): compile kcc's optimizer
    in C, relink, run suite
selfhost-opt: build
        chmod +x gen/selfhost/opt_verify.sh
        gen/selfhost/opt_verify.sh

# Self-hosting (Stage 8n): compile kcc's lexer
   tables in C, relink, run suites
selfhost-lexer-tables: build
        chmod +x gen/selfhost/lexer_tables_verify.
   sh
        gen/selfhost/lexer_tables_verify.sh

# Self-hosting (Stage 8o): compile kcc's linker
   info in C, relink, run suite
selfhost-linker: build
        chmod +x gen/selfhost/linker_verify.sh
        gen/selfhost/linker_verify.sh

# Self-hosting (Stage 8p): compile kcc's statement
    parser in C, relink, run suite
selfhost-statements: build
        chmod +x gen/selfhost/statements_verify.sh
        gen/selfhost/statements_verify.sh

# Self-hosting (Stage 8q): compile kcc's Mach-O
   writer in C, relink, run suite
selfhost-emit-macho: build
        chmod +x gen/selfhost/emit_macho_verify.sh
        gen/selfhost/emit_macho_verify.sh

# Self-hosting (Stage 8r): compile kcc's
   expression parser in C, relink, run suite
selfhost-expressions: build
        chmod +x gen/selfhost/expressions_verify.
   sh
        gen/selfhost/expressions_verify.sh

# Self-hosting (Stage 8s): compile kcc's parser
   spine in C, relink, run suite
selfhost-parser: build
        chmod +x gen/selfhost/parser_verify.sh
        gen/selfhost/parser_verify.sh

# Self-hosting (Stage 8t): compile kcc's driver in
    C, relink, run codegen+e2e
selfhost-driver: build
        chmod +x gen/selfhost/driver_verify.sh
        gen/selfhost/driver_verify.sh

# Self-hosting (Stage 8u): compile kcc's
   declaration parser in C, relink, run codegen
selfhost-declarations: build
        chmod +x gen/selfhost/declarations_verify.
   sh
        gen/selfhost/declarations_verify.sh

# Self-hosting (Stage 8v): compile kcc's DWARF
   emitter in C, relink,
# run codegen + a -g byte-compare against the asm
   build
selfhost-dwarf: build
        chmod +x gen/selfhost/dwarf_verify.sh
        gen/selfhost/dwarf_verify.sh

# Self-hosting (Stage 8w): compile kcc's ARM64
   encoder + .s renderer in C,
# relink, verify both halves (difftest + --emit-
   asm functional check)
selfhost-emit-arm64: build
        chmod +x gen/selfhost/emit_arm64_verify.sh
        gen/selfhost/emit_arm64_verify.sh

# Self-hosting (Stage 8x): compile kcc's semantic
   analyzer in C, relink,
# verify with difftest + diagtest + warntest +
   codegen
selfhost-sema: build
        chmod +x gen/selfhost/sema_verify.sh
        gen/selfhost/sema_verify.sh

# Self-hosting (Stage 8y): compile kcc's x86-64
   back end in C, relink,
# verify functionally (arm64 codegen + x86
   programs run under Rosetta)
selfhost-emit-x86: build
        chmod +x gen/selfhost/emit_x86_verify.sh
        gen/selfhost/emit_x86_verify.sh

# Self-hosting (Stage 8z): compile kcc's
   preprocessor in C, relink,
# verify with difftest + diagtest + codegen
selfhost-preprocessor: build
        chmod +x gen/selfhost/preprocessor_verify.
   sh
        gen/selfhost/preprocessor_verify.sh

# Self-hosting: compile kcc's ARM64 code generator
    in C, relink, verify
selfhost-codegen: build
        chmod +x gen/selfhost/codegen_verify.sh
        gen/selfhost/codegen_verify.sh

# Self-hosting: compile kcc's middle-end IR
   builder in C, relink, verify
selfhost-ir: build
        chmod +x gen/selfhost/ir_verify.sh
        gen/selfhost/ir_verify.sh

# Self-hosting: compile kcc's entry point in C,
   relink in place of main.o, verify
selfhost-main: build
        chmod +x gen/selfhost/main_verify.sh
        gen/selfhost/main_verify.sh

# Self-hosting: compile the compiler's lexer in C,
    relink in place of lexer.o, verify
selfhost-lexer-core: build
        chmod +x gen/selfhost/lexer_core_verify.sh
        gen/selfhost/lexer_core_verify.sh

# Milestone A: cross-compile a multi-TU C program
   to Linux/x86-64 with kcc,
# link against the Linux runtime (no libc), run it
    natively (Chapter: Runtime).
meta-a: build
        chmod +x gen/runtime/meta_a_verify.sh
        gen/runtime/meta_a_verify.sh
```

```
# Milestone C: cross-compile kcc's own source to
   Linux, link it, and prove the
# Linux-native compiler self-hosts (and re-hosts
   itself) in a single command.
meta-c: build
        chmod +x gen/runtime/meta_c_verify.sh
        gen/runtime/meta_c_verify.sh

# Differential tests: compile each program with
   kcc AND clang, compare
difftest: build
        chmod +x gen/tests/difftest.sh
        gen/tests/difftest.sh

# ABI interop: compile one TU with kcc and the
   other with clang, link, run
interop: build
        chmod +x gen/tests/interop.sh
        gen/tests/interop.sh

# End-to-end: kcc compiles AND links to an
   executable, then runs it
e2e: build
        chmod +x gen/tests/e2e.sh
        gen/tests/e2e.sh

# Diagnostics: invalid programs must be rejected
   with clear errors (no hang)
diagtest: build
        chmod +x gen/tests/diagtest.sh
        gen/tests/diagtest.sh

# Warnings: -Wall must warn while the default
   build stays silent
warntest: build
        chmod +x gen/tests/warntest.sh
        gen/tests/warntest.sh

# Code generation tests: compile with kcc, link,
   run, check exit codes
test-codegen: build
        chmod +x gen/tests/test_codegen.sh
        gen/tests/test_codegen.sh

# Weave: literate .weft sources -> LaTeX (kcc.tex)
   . Mirrors `tangle`, which
# produces the code to compile; `weave` produces
   the document source.
weave:
        weft -w tex -o -d -r kcc.weft

# PDF: compile the woven LaTeX to kcc.pdf (
   lualatex for native Unicode).
# The covers are edited as assets/cover.svg and
   assets/back-cover.svg, which
# carry a {{VERSION}} placeholder; substitute the
   single-sourced version from
# gen/VERSION, rasterise to assets/cover.pdf +
   assets/back-cover.png (when
# rsvg-convert is available), then typeset.
pdf: weave
        test -f gen/VERSION || weft -p gen/ --
   mkdirs kcc.weft
        if command -v rsvg-convert >/dev/null
   2>&1; then VER=$$(cat gen/VERSION); sed "s/{{
   VERSION}}/$$VER/g" assets/cover.svg > gen/cover
   .svg && rsvg-convert -f pdf -o assets/cover.pdf
    gen/cover.svg; sed "s/{{VERSION}}/$$VER/g"
   assets/back-cover.svg > gen/back-cover.svg &&
   rsvg-convert -w 2480 -o assets/back-cover.png
   gen/back-cover.svg; fi
        latexmk -lualatex -interaction=nonstopmode
    -f kcc.tex

# Clean generated files
clean:
        rm -rf gen/

# Run the compiler:  make run ARGS="file.c -o out"
run: build
        ./gen/kcc $(ARGS)

# Structure map (JSON)
map:
        weft -m kcc.weft
```

□

38k · uses _main 621, 629

Part II.

Foundations

Every compiler is an implementation of a language standard, and the standard is a legal document: ambiguous, self-referential, and full of subtle constraints that only become visible when you try to encode them as executable rules. This part pins down the normative corpus that `kcc` implements — the final draft of ISO/IEC 9899:2018 (C17) — and the conformance model that governs how a compiler may interpret the standard's requirements, undefined behaviour, and implementation-defined choices. No code appears here; the chapters that follow lay the contractual ground on which every subsequent line of assembly rests.

4. The Normative Corpus

4.1. Primary Standard

The sole governing document for the C language accepted by kcc is:

> **ISO/IEC 9899:2018** — *Information technology — Programming languages — C*. Fourth edition, 2018-06-01.

This document, hereafter cited as **C17** or simply "the Standard," supersedes all prior editions (C89, C99, C11). Where the Standard is silent, kcc treats the behavior as implementation-defined and documents it in Appendix A (§ J.3).

4.2. Secondary References

The following documents inform the implementation but do not override the Standard:

1. **ARM Architecture Reference Manual (ARM ARM), DDI 0487** — defines the A64 instruction set, encoding rules, and system registers. All instruction mnemonics and encodings cite this document by section number.
2. **Procedure Call Standard for the Arm 64-bit Architecture (AAPCS64)**, ARM IHI 0055 — defines the calling convention, register usage, and stack layout.
3. **Apple ARM64 ABI documentation** — describes platform-specific deviations from AAPCS64: mandatory frame pointer, reserved `x18`, no red zone, Mach-O symbol conventions.
4. **Mach-O file format reference** — governs the binary format emitted by the backend.
5. **macOS system call interface** — the `svc #0x80` convention with syscall number in `x16`.

4.3. Citation Conventions

Throughout this document, we use the following shorthand:

- § 6.5.3.1 refers to ISO 9899:2018 section 6.5.3.1.
- **ARM ARM C6.2.5** refers to the ARM Architecture Reference Manual, section C6.2.5.
- **AAPCS64 §5.4** refers to the Procedure Call Standard, section 5.4.

The keywords SHALL, SHALL NOT, MAY, and UNDEFINED BEHAVIOR carry their ISO meanings. We abbreviate undefined behavior as UB and implementation-defined behavior as IB.

5. Conformance Model

5.1. Hosted vs. Freestanding

The Standard (§ 5.1.2) distinguishes two execution environments:

- **Hosted** (§ 5.1.2.2) — the full C library is available; program entry is `main`.
- **Freestanding** (§ 5.1.2.1) — a minimal library subset; entry point is implementation-defined.

kcc targets a *hosted* environment on macOS/ARM64. The program entry point is `_main` per Mach-O convention, which corresponds to `main` as defined in § 5.1.2.2.1.

5.2. Translation Limits

Section § 5.2.4.1 specifies minimum translation limits that every conforming implementation must support. kcc meets or exceeds all limits listed therein. Notable limits:

- 127 nesting levels of blocks
- 63 nesting levels of conditional inclusion
- 4095 characters in a logical source line (after line splicing)
- 4095 characters in a string literal
- 511 identifiers with block scope in one block
- 127 parameters in one function definition
- 63 levels of nested structure or union definitions

These limits are enforced through the diagnostic engine (§12).

5.3. Implementation-Defined Behavior

The Standard identifies approximately 200 implementation-defined behaviors (§ J.3). Each is documented in Appendix A of this document with:

1. The § J.3 clause number.
2. The choice made by kcc.
3. The rationale for that choice.

5.4. Undefined Behavior Policy

kcc adopts a *diagnostic-first* policy for undefined behavior:

Requirement 5.1 (UB Policy)**.** Where kcc can detect undefined behavior at compile time with reasonable cost, it shall emit a diagnostic. Where detection is infeasible or unreasonably expensive, kcc documents the treatment in Appendix B (UB Policy Matrix) and proceeds with the most natural ARM64 behavior.

This policy balances strict conformance with the practical reality of writing a compiler in assembly language. The UB matrix is the authoritative reference for each specific case.

Part III.

The Machine

A compiler that targets bare metal must build its own floor. This part constructs the runtime substrate on which the rest of kcc stands: the ARM64 calling conventions and macro vocabulary (Chapter 6), thin wrappers around the Darwin `svc #0x80` system-call interface (Chapter 7), a bump-pointer arena allocator backed by `mmap` (Chapter 8), the three fundamental data structures every compiler needs — hash table, dynamic vector, and string intern table (Chapter 9) — plus byte-level string operations (Chapter 10), buffered I/O (Chapter 11), a diagnostic engine that formats `file:line:col: severity: message` errors to stderr (Chapter 12), and the source file manager that `mmap`s C source files for zero-copy lexing (Chapter 13). None of these modules depend on any C library; every byte of output reaches the kernel through `svc`.

6. ARM64 Conventions

Before any code can be written, the compiler must establish a contract with the hardware: which registers hold persistent state, how the stack is managed, and what Apple's platform deviates from the standard AAPCS64 ABI. This chapter codifies those decisions—the register map that threads global compiler state through callee-saved registers, the Apple-specific constraints that every function must obey, and the `FUNC`/`ENDFUNC`/`SYSCALL` macros that enforce them uniformly across the entire codebase.

6.1. Register Assignment

The AAPCS64 (ARM IHI 0055) divides the 31 general-purpose registers into well-defined roles. kcc further refines the callee-saved registers for persistent compiler state:

Register(s)	AAPCS64 Role	kcc Usage
x0–x7	Arguments/results	Per AAPCS64
x8	Indirect result	Scratch
x9–x15	Temporaries	Scratch (caller-saved)
x16–x17	Intra-procedure scratch	Syscall number, linker veneers
x18	Platform register	**Reserved** (Apple)
x19	Callee-saved	Arena pointer (global)
x20	Callee-saved	String table pointer (global)
x21	Callee-saved	Current source buffer
x22	Callee-saved	Current token pointer
x23–x28	Callee-saved	Available for local use
x29	Frame pointer	**Mandatory** (Apple)
x30	Link register	Return address
sp	Stack pointer	16-byte aligned always

6.2. Apple Platform Constraints

Apple ARM64 deviates from vanilla AAPCS64 in several critical ways:

1. **Frame pointer is mandatory.** Every function must establish a frame record: `stp x29, x30, [sp, #-N]!` followed by `mov x29, sp` (or `add x29, sp, #off`).
2. **`x18` is reserved.** The platform register must never be read or written by user code.
3. **No red zone.** Unlike x86-64, there is no guaranteed scratch space below `sp`. All local storage must be explicitly allocated by decrementing `sp`.
4. **Stack alignment.** `sp` must be 16-byte aligned at every instruction that accesses memory through `sp`.
5. **Mach-O symbol prefix.** All external symbols carry a leading underscore: `_main`, `_kcc_arena_init`.
6. **Syscall convention.** System calls use `svc #0x80` with the syscall number in `x16`, arguments in `x0`–`x5`, and the return value in `x0`. The carry flag indicates error.

6.3. Common Macros

We define three macros used throughout the codebase. Every tangled `.s` file includes these at the top.

`FUNC` establishes a standard function prologue with the mandatory frame pointer. `ENDFUNC` tears down the frame and returns. `SYSCALL` invokes a macOS system call.

⟨ *ARM64: common macros* 51 ⟩ ≡

```
// -- kcc common macros ---
// FUNC name - standard function prologue (Apple
   ARM64 ABI)
// Saves frame pointer (x29) and link register (
   x30).
// The function is exported with the Mach-O
   underscore prefix.
.macro FUNC name
    .globl _\name
    .p2align 2
_\name:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
.endm

// ENDFUNC - standard function epilogue
// Restores x29/x30 and returns to caller.
.macro ENDFUNC
    ldp x29, x30, [sp], #16
    ret
.endm

// SYSCALL num - invoke macOS syscall
// Places the syscall number in x16 and traps to
   kernel.
// Arguments must already be in x0-x5 per
   convention.
// Returns result in x0; carry flag set on error.
.macro SYSCALL num
    mov x16, #\num
    svc #0x80
.endm
```

□

51 · used in 27abcdefg, 28abcdefg, 29abcdef, 30abcd, 31abcdef, 32r, 33a

7. System Calls

Because kcc links against no C library, every interaction with the operating system must go through the Darwin kernel's `svc #0x80` interface directly. This chapter provides thin, AAPCS64-conforming wrappers for the seven syscalls the compiler requires: process termination, file I/O (`open`, `read`, `write`, `close`), memory mapping (`mmap`), and file seeking (`lseek`). Each wrapper establishes a proper frame record and translates the kernel convention—number in `x16`, arguments in `x0`–`x5`—into ordinary function calls.

7.1. Syscall Number Constants

These constants come from the macOS/XNU kernel headers (`<sys/syscall.h>`). On ARM64, syscall numbers are placed in `x16` before the `svc` instruction.

⟨ *Syscall: constants* 53a ⟩ ≡

```
// -- macOS ARM64 syscall numbers ---
// Source: XNU bsd/kern/syscalls.master
.equ SYS_exit,      1
.equ SYS_fork,      2
.equ SYS_read,      3
.equ SYS_write,     4
.equ SYS_open,      5
.equ SYS_close,     6
.equ SYS_wait4,     7
.equ SYS_unlink,   10
.equ SYS_execve,   59
.equ SYS_mmap,    197           // 0xC5
.equ SYS_munmap,   73           // 0x49
.equ SYS_lseek,   199           // 0xC7

// lseek whence constants
.equ SEEK_SET,      0
.equ SEEK_CUR,      1
.equ SEEK_END,      2

// mmap constants for file mapping
.equ PROT_READ,     0x1         // PROT_READ only
.equ MAP_PRIVATE,   0x2         // MAP_PRIVATE (no
   MAP_ANON)
```

□

53a · used in 27bc, 28c

7.2. Process Control

The simplest syscall: terminate the process. `SYS_exit` takes a single argument in `x0` (the exit status) and does not return.

⟨ *Syscall: exit* 53b ⟩ ≡

```
// -- SYS_exit ---
// void kcc_sys_exit(int status)
//   x0 = exit status
// Does not return.
FUNC kcc_sys_exit
    SYSCALL SYS_exit
    brk #1                      // unreachable; trap
    if kernel returns
ENDFUNC
```

□

53b · used in 27b

7.3. File I/O

The core I/O syscalls. Each wrapper preserves the AAPCS64 calling convention: arguments arrive in `x0`–`x5`, the result is returned in `x0`, and the carry flag indicates an error (in which case `x0` holds the `errno` value).

7.3.1. write

Write bytes to a file descriptor. This is the fundamental output primitive; all formatted output in kcc ultimately reduces to calls to `kcc_sys_write`.

⟨ *Syscall: write* 53c ⟩ ≡

```
// -- SYS_write ---
// ssize_t kcc_sys_write(int fd, const void *buf,
   size_t len)
//   x0 = file descriptor
//   x1 = buffer pointer
//   x2 = byte count
// Returns bytes written in x0, or (uint64_t)-1
   with carry set.
FUNC kcc_sys_write
    SYSCALL SYS_write
ENDFUNC
```

□

53c · used in 27b

7.3.2. read

Read bytes from a file descriptor. Used by the source manager to read source files when `mmap` is unsuitable.

⟨ *Syscall: read* 53d ⟩ ≡

```
// -- SYS_read ---
// ssize_t kcc_sys_read(int fd, void *buf, size_t
   len)
//   x0 = file descriptor
//   x1 = buffer pointer
//   x2 = maximum bytes to read
// Returns bytes read in x0, 0 at EOF, or -1 on
   error.
FUNC kcc_sys_read
    SYSCALL SYS_read
ENDFUNC
```

□

53d · used in 27b

7.3.3. open

Open a file by pathname. Returns a file descriptor or -1 on error.

Unlike the other wrappers, `open` inspects the carry flag the kernel sets on failure and collapses the raw `errno` it leaves in `x0` to a uniform -1. Callers—`source_open` and

the preprocessor's include probing—test for -1, and a bare `errno` (a small positive integer) would otherwise be mistaken for a valid descriptor, so this normalisation is what lets a missing `#include` surface as a clean diagnostic instead of a later crash on a bogus file descriptor.

⟨ *Syscall: open* 54a ⟩ ≡

```
// -- SYS_open ---
// int kcc_sys_open(const char *path, int flags,
   mode_t mode)
//   x0 = path (null-terminated)
//   x1 = flags (O_RDONLY=0, O_WRONLY=1, O_RDWR=2,
   ...)
//   x2 = mode (permissions, used with O_CREAT)
// Returns fd in x0, or -1 on error.
FUNC kcc_sys_open
    SYSCALL SYS_open
    b.cs .Lsys_open_err          // carry set →
   errno in x0, normalise to -1
    ldp x29, x30, [sp], #16
    ret
.Lsys_open_err:
    mov x0, #-1
    ldp x29, x30, [sp], #16
    ret
```

□

54a · used in 27b

7.3.4. close

Close a file descriptor.

⟨ *Syscall: close* 54b ⟩ ≡

```
// -- SYS_close ---
// int kcc_sys_close(int fd)
//   x0 = file descriptor
// Returns 0 on success, -1 on error.
FUNC kcc_sys_close
    SYSCALL SYS_close
ENDFUNC
```

□

54b · used in 27b

7.4. Memory Management

7.4.1. mmap

Map pages of memory. This is how kcc obtains all dynamic memory: the arena allocator (Chapter 8) calls `mmap` with `MAP_PRIVATE | MAP_ANON` to get fresh pages from the kernel.

⟨ *Syscall: mmap* 54c ⟩ ≡

```
// -- SYS_mmap ---
// void *kcc_sys_mmap(void *addr, size_t len, int
   prot,
//                    int flags, int fd, off_t
   offset)
//   x0 = requested address (0 for kernel choice)
//   x1 = length in bytes
//   x2 = protection (PROT_READ|PROT_WRITE = 0x3)
//   x3 = flags (MAP_PRIVATE|MAP_ANON = 0x1002)
//   x4 = file descriptor (-1 for anonymous)
//   x5 = offset
// Returns mapped address in x0, or MAP_FAILED ((
   void *)-1).
FUNC kcc_sys_mmap
    SYSCALL SYS_mmap
ENDFUNC
```

□

54c · used in 27b

7.4.2. lseek

Reposition the file offset. Used by the source manager to determine file size before `mmap`: `lseek(fd, 0, SEEK_END)` returns the file length.

⟨ *Syscall: lseek* 54d ⟩ ≡

```
// -- SYS_lseek ---
// off_t kcc_sys_lseek(int fd, off_t offset, int
   whence)
//   x0 = file descriptor
//   x1 = offset
//   x2 = whence (SEEK_SET=0, SEEK_CUR=1, SEEK_END
   =2)
// Returns new file offset in x0, or -1 on error.
FUNC kcc_sys_lseek
    SYSCALL SYS_lseek
ENDFUNC
```

□

54d · used in 27b

7.5. Process Creation

Automatic linking (Chapter 44) requires kcc to run the system linker as a child process. Three syscalls suffice: `fork` duplicates the process, `execve` replaces the child's image with `/usr/bin/ld`, and `wait4` lets the parent collect the linker's exit status. `unlink` removes the temporary object files afterwards.

On Darwin ARM64 the `fork` syscall has an unusual convention: it returns in *both* processes at the instruction after `svc`, with `x1` set to 1 in the child and 0 in the parent (and `x0` holding the child's PID in the parent). The wrapper normalises this to the POSIX contract—the child sees a return value of 0, the parent sees the child PID, and -1 signals an error (carry set).

⟨ *Syscall: process* 54e ⟩ ≡

```
// -- SYS_fork ---
// pid_t kcc_sys_fork(void)
// Returns 0 in the child, the child PID in the
   parent, or -1 on error.
FUNC kcc_sys_fork
    SYSCALL SYS_fork
    b.cs .Lfork_err
    cmp x1, #1                   // Darwin: x1 == 1
   in the child
    b.ne .Lfork_ret
    mov x0, #0                   // child → 0
    b .Lfork_ret
.Lfork_err:
    mov x0, #-1
.Lfork_ret:
ENDFUNC
```

```
// -- SYS_execve ---
// int kcc_sys_execve(const char *path, char *
   const argv[], char *const envp[])
//   x0 = path, x1 = argv, x2 = envp
// Only returns on failure; then returns -1.
FUNC kcc_sys_execve
    SYSCALL SYS_execve
    mov x0, #-1
ENDFUNC

// -- SYS_wait4 ---
// pid_t kcc_sys_wait4(pid_t pid, int *status, int
    options, void *rusage)
//   x0 = pid, x1 = status ptr, x2 = options, x3 =
    rusage ptr
// Returns the reaped PID in x0.
FUNC kcc_sys_wait4
    SYSCALL SYS_wait4
ENDFUNC

// -- SYS_unlink ---
// int kcc_sys_unlink(const char *path)
//   x0 = path
FUNC kcc_sys_unlink
    SYSCALL SYS_unlink
ENDFUNC
```

□

54e · used in 27b

8. Memory Management: The Arena

A compiler allocates prolifically—tokens, AST nodes, type descriptors, symbol entries—yet all of these objects share a single lifetime: one translation unit. The arena allocator exploits this by reducing every allocation to a pointer bump and every deallocation to a single `munmap`. This chapter describes the chunk layout, the initialization and allocation routines, and the aligned-allocation variant used for structures with alignment requirements beyond a byte.

8.1. Design Rationale

A compiler allocates thousands of small objects (tokens, AST nodes, type descriptors, symbol entries) that all live until the end of compilation. A general-purpose allocator (malloc/free) would waste time tracking individual lifetimes. An arena allocator reduces allocation to a pointer bump — a single add instruction — and deallocation to a single `munmap` of the entire arena.

8.2. Chunk Layout

Each arena chunk is a contiguous region obtained from `mmap`. The first 24 bytes form a header:

Offset	Field	Description
0	`next`	Pointer to the next chunk (linked list)
8	`limit`	One past the last usable byte
16	`avail`	Next free byte (bump pointer)

⟨ *Arena: constants* 57a ⟩ ≡

```
// -- Arena chunk layout ---
.equ CHUNK_NEXT,     0          // pointer to next
   chunk
.equ CHUNK_LIMIT,    8          // end of usable
   region
.equ CHUNK_AVAIL,   16          // next free byte (
   bump pointer)
.equ CHUNK_HEADER, 24           // total header size

// Default chunk size: 64 KiB (16 pages of 4 KiB)
.equ ARENA_CHUNK_SIZE, 65536

// mmap constants (macOS)
.equ PROT_RW,        0x3        // PROT_READ |
   PROT_WRITE
.equ MAP_PRIV_ANON, 0x1002      // MAP_PRIVATE |
   MAP_ANON
.equ MAP_FAILED,     -1
```

□

57a · used in 27c

8.3. Initialization

`kcc_arena_init` obtains the first chunk from the kernel and initializes the global arena state. The arena pointer is stored in a global variable; in later phases, the main processing loop will cache it in `x19`.

⟨ *Arena: init* 57b ⟩ ≡

```
// -- kcc_arena_init ---
// void kcc_arena_init(void)
// Allocates the initial arena chunk via mmap.
// On failure (mmap returns MAP_FAILED), exits
   with status 1.
.globl _kcc_arena_init
.p2align 2
_kcc_arena_init:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
⟨ Arena: init mmap call 57c ⟩
⟨ Arena: init chunk header 57d ⟩
    ldp x29, x30, [sp], #16
    ret

.Larena_init_fail:
    mov x0, #1                      // exit status =
    1
    bl _kcc_sys_exit
```

□

57b · used in 27c · defines `_kcc_arena_init` 621, 629

The initialization proceeds in two steps. First, we request a 64 KiB anonymous mapping from the kernel:

⟨ *Arena: init mmap call* 57c ⟩ ≡

```
    // mmap(NULL, ARENA_CHUNK_SIZE, PROT_RW,
   MAP_PRIV_ANON, -1, 0)
    mov x0, #0                      // addr = NULL (
   kernel chooses)
    mov x1, #ARENA_CHUNK_SIZE       // len  = 64 KiB
    mov x2, #PROT_RW                // prot = read|
   write
    mov x3, #MAP_PRIV_ANON          // flags =
   private|anonymous
    mov x4, #-1                     // fd   = -1 (no
    file)
    mov x5, #0                      // offset = 0
    bl _kcc_sys_mmap

    // Check for MAP_FAILED
    cmn x0, #1                      // compare with
   (uint64_t)-1
    b.eq .Larena_init_fail
```

□

57c · used in 57b

Then we write the three header fields and record the chunk as current:

⟨ *Arena: init chunk header* 57d ⟩ ≡

```
    // Initialize chunk header (x0 = base address)
    str xzr, [x0, #CHUNK_NEXT]    // next = NULL

    mov x1, x0
    add x1, x1, #ARENA_CHUNK_SIZE
```

```
    str x1, [x0, #CHUNK_LIMIT]    // limit = base +
    size

    mov x1, x0
    add x1, x1, #CHUNK_HEADER
    str x1, [x0, #CHUNK_AVAIL]    // avail = base +
    header

    // Store as the current chunk
    adrp x1, _kcc_arena_current@PAGE
    add  x1, x1, _kcc_arena_current@PAGEOFF
    str  x0, [x1]
```

□

57d · used in 57b · uses _kcc_arena_current 59

8.4. Allocation: Fast Path

kcc_arena_alloc is the primary allocation function. It takes a size in bytes (in x0), rounds it up to 8-byte alignment, and returns a pointer to the allocated region.

The fast path is a single comparison and pointer bump — three instructions after the alignment arithmetic:

⟨ *Arena: alloc fast path* 58a ⟩ ≡

```
// -- kcc_arena_alloc ---
// void *kcc_arena_alloc(size_t size)
//   x0 = number of bytes to allocate
// Returns pointer to allocated memory in x0.
// All allocations are 8-byte aligned.
.globl _kcc_arena_alloc
.p2align 2
_kcc_arena_alloc:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Round size up to 8-byte alignment: (size +
   7) & ~7
    add x0, x0, #7
    and x0, x0, #~7
    mov x19, x0                    // x19 = aligned
    size

    // Load current chunk pointer
    adrp x1, _kcc_arena_current@PAGE
    add  x1, x1, _kcc_arena_current@PAGEOFF
    ldr  x2, [x1]                  // x2 = current
   chunk

    // Load avail and limit
    ldr x3, [x2, #CHUNK_AVAIL]    // x3 = avail
    ldr x4, [x2, #CHUNK_LIMIT]    // x4 = limit

    // Check: avail + size <= limit?
    add x5, x3, x19
    cmp x5, x4
    b.hi .Larena_alloc_slow        // overflow:
   need new chunk

    // Fast path: bump the pointer
    str x5, [x2, #CHUNK_AVAIL]    // avail += size
    mov x0, x3                     // return old
   avail

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

58a · used in 27c · defines _kcc_arena_alloc 61c, 63, 64b, 65a, 66b, 85, 113, 117, 118, 119b, 121, 123, 125b, 127, 132, 136, 148, 150, 151, 159b, 161, 165, 168a, 239b, 240, 241, 242a, 243abc, 244, 254b, 256a, 259b, 277, 302, 303abc, 343, 359, 393, 417a, 447, 450, 453a, 457, 463b, 465, 472, 475, 476, 477b, 478, 481, 490, 493, 498a, 518, 521, 522b, 525b, 535b, 537, 577b, 586, 591b, 596, 603, 609, 611 · uses _kcc_arena_current 59

8.5. Allocation: Slow Path

When the current chunk is exhausted, we allocate a new one via mmap, link it to the chain, and fulfill the request from the fresh chunk.

⟨ *Arena: alloc slow path* 58b ⟩ ≡

```
.Larena_alloc_slow:
    // Chunk size = max(ARENA_CHUNK_SIZE,
   CHUNK_HEADER + size)
    add x0, x19, #CHUNK_HEADER
    mov x1, #ARENA_CHUNK_SIZE
    cmp x0, x1
    csel x1, x0, x1, hi           // x1 = max(
   needed, default)

    // mmap the new chunk
    mov x0, #0
    mov x2, #PROT_RW
    mov x3, #MAP_PRIV_ANON
    mov x4, #-1
    mov x5, #0
    bl _kcc_sys_mmap

    cmn x0, #1
    b.eq .Larena_alloc_oom
⟨ Arena: alloc link new chunk 58c ⟩

.Larena_alloc_oom:
    mov x0, #2                    // exit status =
   2
    bl _kcc_sys_exit
```

□

58b · used in 27c

Once the new chunk is mapped, we link it into the chain, initialize its header, and return the first allocation from it:

⟨ *Arena: alloc link new chunk* 58c ⟩ ≡

```
    // x0 = new chunk base; link to previous chunk
    adrp x1, _kcc_arena_current@PAGE
    add  x1, x1, _kcc_arena_current@PAGEOFF
    ldr  x2, [x1]                  // x2 = old
   chunk

    str x2, [x0, #CHUNK_NEXT]     // new->next =
   old

    mov x3, x0
    add x3, x3, #ARENA_CHUNK_SIZE
    str x3, [x0, #CHUNK_LIMIT]    // limit (
   conservative)

    mov x3, x0
    add x3, x3, #CHUNK_HEADER
    add x4, x3, x19               // new avail
   after allocation
    str x4, [x0, #CHUNK_AVAIL]
```

```
    str x0, [x1]                    // update
    arena_current
    mov x0, x3                      // return =
    first byte after header

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

58c · used in 58b · uses _kcc_arena_current 59

8.6. Global State

The arena's mutable state is a single pointer to the current (most recently allocated) chunk. We store it in the __DATA segment.

⟨ *Arena: data* 59 ⟩ ≡

```
// -- Arena global state ---
.data
.p2align 3
_kcc_arena_current: .quad 0    // pointer to
    current arena chunk
```

□

59 · used in 27c · defines _kcc_arena_current 57d, 58ac

9. Data Structures

Three data structures recur throughout the compiler: a hash table for $O(1)$ lookups (keyword recognition, symbol resolution), a dynamic vector for growable sequences (token lists, IR instruction arrays), and a string intern table for canonicalizing identifiers so that pointer equality suffices for comparison. All three are arena-allocated, so they never individually free memory. This chapter presents the FNV-1a hash function, the open-addressed hash table built on top of it, the resizable vector, and the string interning layer.

9.1. FNV-1a Hash Function

FNV-1a (Fowler/Noll/Vo, variant "a") is a non-cryptographic hash function with excellent distribution for short strings — exactly the workload a compiler presents (identifiers average 6–12 bytes). The algorithm is simple: start from an offset basis, and for each byte XOR it into the hash then multiply by the FNV prime.

The 64-bit constants are:

- Offset basis: 0xCBF29CE484222325 (= 14695981039346656037)
- Prime: 0x00000100000001B3 (= 1099511628211)

⟨ *Hash: FNV-1a* 61a ⟩ ≡

```
// -- kcc_fnv1a ---
// uint64_t kcc_fnv1a(const void *data, size_t len
   )
//   x0 = pointer to data
//   x1 = byte count
// Returns 64-bit FNV-1a hash in x0.
.globl _kcc_fnv1a
.p2align 2
_kcc_fnv1a:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Load FNV-1a offset basis: 0
   xCBF29CE484222325
    movz x2, #0xCBF2, lsl #48
    movk x2, #0x9CE4, lsl #32
    movk x2, #0x8422, lsl #16
    movk x2, #0x2325

    // Load FNV-1a prime: 0x00000100000001B3
    movz x3, #0x0100, lsl #32
    movk x3, #0x01B3

    cbz x1, .Lfnv_done

.Lfnv_loop:
    ldrb w4, [x0], #1           // load byte
    eor  x2, x2, x4             // hash ^= byte
    mul  x2, x2, x3             // hash *= prime
    subs x1, x1, #1
    b.ne .Lfnv_loop

.Lfnv_done:
    mov x0, x2                  // return hash
    ldp x29, x30, [sp], #16
    ret
```

□

61a · used in 27f · defines _kcc_fnv1a 62, 63

9.2. Hash Table

The hash table uses open addressing with linear probing. Each entry stores the cached hash, a pointer to the key, the key length, and a 64-bit value. Empty slots are indicated by a null key pointer.

Offset	Field	Description
0	hash	Cached FNV-1a hash
8	key	Pointer to key bytes
16	keylen	Key length in bytes
24	value	Associated 64-bit value

⟨ *Hash: constants* 61b ⟩ ≡

```
// -- Hash table layout ---
.equ HT_ENTRIES,        0     // pointer to entries
    array
.equ HT_CAP,            8     // capacity (power of
    2)
.equ HT_COUNT,         16     // number of occupied
    slots
.equ HT_SIZE,          24     // total header size

.equ HTE_HASH,          0     // cached hash value
.equ HTE_KEY,           8     // pointer to key
.equ HTE_KEYLEN,       16     // key length
.equ HTE_VALUE,        24     // associated value
.equ HTE_SIZE,         32     // entry size
```

□

61b · used in 27f, 30a

9.2.1. Initialization

kcc_ht_init creates a hash table with the given capacity (which must be a power of 2). The entries array is arena-allocated and zero-filled, so all key pointers start as NULL.

⟨ *Hash: init* 61c ⟩ ≡

```
// -- kcc_ht_init ---
// void *kcc_ht_init(uint64_t capacity)
//   x0 = capacity (must be power of 2)
// Returns pointer to hash table header.
.globl _kcc_ht_init
.p2align 2
_kcc_ht_init:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                 // x19 = capacity

    // Allocate the header (HT_SIZE = 24 bytes)
    mov x0, #HT_SIZE
    bl _kcc_arena_alloc
    mov x20, x0                 // x20 = header
   pointer
```

```
    // Allocate entries array (capacity * HTE_SIZE
    )
    mov x0, x19
    mov x1, #HTE_SIZE
    mul x0, x0, x1
    mov x21, x0                    // x21 = total
    bytes
    bl _kcc_arena_alloc
    // x0 = entries pointer

    // Zero-fill entries (arena doesn't guarantee
    zeroed memory)
    mov x1, #0                     // fill byte
    mov x2, x21                    // byte count
    // x0 already = entries ptr, save it first
    str x0, [x20, #HT_ENTRIES]   // store entries
    pointer
    mov x3, x0                     // save entries
    ptr
    bl _kcc_memset

    // Fill header
    str x19, [x20, #HT_CAP]      // capacity
    str xzr, [x20, #HT_COUNT]    // count = 0

    mov x0, x20                    // return header
    pointer

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

61c · used in 27f · defines _kcc_ht_init 66a, 104a, 107b, 170ab, 253c, 257a, 302, 312, 316, 317, 463b, 493, 591b · uses _kcc_arena_alloc 58a

9.2.2. Lookup

kcc_ht_get searches for a key in the table. It returns the associated value if found, or 0 if not. Since 0 is a valid "not found" sentinel for our use cases (keyword token types start at 10), this is sufficient.

⟨ Hash: get 62 ⟩ ≡

```
// -- kcc_ht_get ---
// uint64_t kcc_ht_get(void *ht, const void *key,
   uint64_t keylen)
//   x0 = hash table pointer
//   x1 = key pointer
//   x2 = key length
// Returns value if found, 0 if not found.
.globl _kcc_ht_get
.p2align 2
_kcc_ht_get:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]

    mov x19, x0                    // x19 = ht
    mov x20, x1                    // x20 = key
    mov x21, x2                    // x21 = keylen

    // Hash the key
    mov x0, x1
    mov x1, x2
    bl _kcc_fnv1a
    mov x22, x0                    // x22 = hash

    // Load table fields
    ldr x23, [x19, #HT_ENTRIES] // x23 = entries
    base
    ldr x24, [x19, #HT_CAP]     // x24 = capacity
    sub x3, x24, #1              // x3 = mask (cap
    - 1)
    and x25, x22, x3             // x25 = index (
    callee-saved!)

.Lht_get_probe:
    // Compute entry address: entries + index *
    HTE_SIZE
    mov x5, #HTE_SIZE
    mul x6, x25, x5
    add x6, x23, x6              // x6 = entry
    pointer

    // Check if slot is empty (key == NULL)
    ldr x7, [x6, #HTE_KEY]
    cbz x7, .Lht_get_not_found

    // Check hash match first (fast reject)
    ldr x8, [x6, #HTE_HASH]
    cmp x8, x22
    b.ne .Lht_get_next

    // Check key length
    ldr x8, [x6, #HTE_KEYLEN]
    cmp x8, x21
    b.ne .Lht_get_next

    // Compare key bytes
    mov x0, x7                     // entry key
    mov x1, x20                    // search key
    mov x2, x21                    // length
    bl _kcc_memcmp
    cbnz x0, .Lht_get_next

    // Found! Return value (x25 = index, callee-
    saved across memcmp)
    mov x5, #HTE_SIZE
    mul x6, x25, x5
    add x6, x23, x6
    ldr x0, [x6, #HTE_VALUE]
    b .Lht_get_done

.Lht_get_next:
    add x25, x25, #1
    sub x3, x24, #1
    and x25, x25, x3             // wrap around
    b .Lht_get_probe

.Lht_get_not_found:
    mov x0, #0

.Lht_get_done:
    ldr x25, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

62 · used in 27f · defines _kcc_ht_get 66b, 93, 119b, 121, 135, 136, 146, 171, 255, 256b, 257b, 321, 322, 350, 355, 367, 378b, 382, 383, 387, 393, 398, 405, 474, 516, 521, 592c, 593 · uses _kcc_fnv1a 61a

9.2.3. Insertion

`kcc_ht_put` inserts a key-value pair, updating the value if the key already exists. Because the table uses open addressing, its performance collapses as it fills—probe chains lengthen, and a table that fills completely would loop forever hunting for an empty slot. A fixed capacity is therefore only safe for a table with a known, bounded population (the keyword table's 44 entries in 128 slots). Tables whose size tracks the input—the string interner, the symbol tables, the per-function local and label maps—must grow, or a large translation unit degrades to $O(n^2)$ and then hangs. So `kcc_ht_put` first calls `kcc_ht_grow`, which doubles the capacity and rehashes every entry once the load factor would exceed three-quarters. Growth mutates the header's entries pointer and capacity in place, so every caller's handle to the table stays valid; keeping the load factor bounded turns insertion and lookup back into amortised $O(1)$.

⟨ *Hash: put* 63 ⟩ ≡

```
// -- kcc_ht_grow ---
// void kcc_ht_grow(void *ht)
// Double the table's capacity and rehash all
   entries into the new array,
// updating HT_ENTRIES/HT_CAP in the header.  Keys
    are already unique, so each
// rehash insertion only needs to find the first
   empty slot.
.globl _kcc_ht_grow
.p2align 2
_kcc_ht_grow:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0                    // x19 = ht
   header
    ldr x20, [x19, #HT_ENTRIES] // x20 = old
   entries
    ldr x21, [x19, #HT_CAP]     // x21 = old
   capacity
    lsl x22, x21, #1               // x22 = new
   capacity = old * 2

    // Allocate and zero the new entries array.
    mov x0, x22
    mov x1, #HTE_SIZE
    mul x0, x0, x1
    mov x24, x0                    // x24 = new byte
    size
    bl _kcc_arena_alloc
    mov x23, x0                    // x23 = new
   entries base
    mov x1, #0
    mov x2, x24
    bl _kcc_memset

    // Rehash: walk the old entries, reinsert each
    occupied one.
    mov x24, #0                    // x24 = old
   index
.Lht_grow_scan:
    cmp x24, x21
    b.hs .Lht_grow_publish
    mov x0, #HTE_SIZE
    mul x1, x24, x0
    add x1, x20, x1                // x1 = old entry
   ptr
    ldr x2, [x1, #HTE_KEY]
    cbz x2, .Lht_grow_next       // empty slot →
   skip
    // Probe the new array from hash & (new_cap-1)
    for the first empty slot.
    ldr x3, [x1, #HTE_HASH]
    sub x4, x22, #1              // new mask
    and x5, x3, x4               // start index
.Lht_grow_probe:
    mov x0, #HTE_SIZE
    mul x6, x5, x0
    add x6, x23, x6              // new entry ptr
    ldr x7, [x6, #HTE_KEY]
    cbz x7, .Lht_grow_place
    add x5, x5, #1
    and x5, x5, x4
    b .Lht_grow_probe
.Lht_grow_place:
    ldr x7, [x1, #HTE_HASH]
    str x7, [x6, #HTE_HASH]
    ldr x7, [x1, #HTE_KEY]
    str x7, [x6, #HTE_KEY]
    ldr x7, [x1, #HTE_KEYLEN]
    str x7, [x6, #HTE_KEYLEN]
    ldr x7, [x1, #HTE_VALUE]
    str x7, [x6, #HTE_VALUE]
.Lht_grow_next:
    add x24, x24, #1
    b .Lht_grow_scan
.Lht_grow_publish:
    str x23, [x19, #HT_ENTRIES]
    str x22, [x19, #HT_CAP]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_ht_put ---
// void kcc_ht_put(void *ht, const void *key,
   uint64_t keylen, uint64_t value)
//   x0 = hash table pointer
//   x1 = key pointer
//   x2 = key length
//   x3 = value to store
// Inserts or updates the key-value pair.
.globl _kcc_ht_put
.p2align 2
_kcc_ht_put:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str x27, [sp, #80]

    mov x19, x0                    // x19 = ht
    mov x20, x1                    // x20 = key
    mov x21, x2                    // x21 = keylen
    mov x25, x3                    // x25 = value

    // Grow before inserting if the load factor
   would exceed 3/4.  A pure update
    // grows a touch early, which is harmless.
   The header pointer is unchanged,
    // so x19 stays valid; the entries/capacity
   are reloaded below.
    ldr x0, [x19, #HT_COUNT]
    add x0, x0, #1
    lsl x0, x0, #2               // (count + 1) * 4
    ldr x1, [x19, #HT_CAP]
```

```
    add x2, x1, x1, lsl #1      // capacity * 3
    cmp x0, x2
    b.le .Lht_put_nogrow
    mov x0, x19
    bl _kcc_ht_grow
.Lht_put_nogrow:

    // Hash the key
    mov x0, x20
    mov x1, x21
    bl _kcc_fnv1a
    mov x22, x0                  // x22 = hash

    // Load table fields
    ldr x23, [x19, #HT_ENTRIES] // x23 = entries
   base
    ldr x24, [x19, #HT_CAP]     // x24 = capacity
    sub x3, x24, #1              // x3 = mask
    and x27, x22, x3             // x27 = index (
   callee-saved!)

.Lht_put_probe:
    mov x5, #HTE_SIZE
    mul x6, x27, x5
    add x6, x23, x6              // x6 = entry
   pointer

    // Empty slot? Insert here
    ldr x7, [x6, #HTE_KEY]
    cbz x7, .Lht_put_insert

    // Check if same key (update case)
    ldr x8, [x6, #HTE_HASH]
    cmp x8, x22
    b.ne .Lht_put_next

    ldr x8, [x6, #HTE_KEYLEN]
    cmp x8, x21
    b.ne .Lht_put_next

    mov x26, x6                  // save entry ptr
   (callee-saved)
    mov x0, x7
    mov x1, x20
    mov x2, x21
    bl _kcc_memcmp
    cbnz x0, .Lht_put_next

    // Key exists: update value
    str x25, [x26, #HTE_VALUE]
    b .Lht_put_done

.Lht_put_next:
    add x27, x27, #1
    sub x3, x24, #1
    and x27, x27, x3
    b .Lht_put_probe

.Lht_put_insert:
    // x6 = empty entry pointer (freshly computed,
    no call since)
    str x22, [x6, #HTE_HASH]
    str x20, [x6, #HTE_KEY]
    str x21, [x6, #HTE_KEYLEN]
    str x25, [x6, #HTE_VALUE]

    // Increment count
    ldr x0, [x19, #HT_COUNT]
    add x0, x0, #1
    str x0, [x19, #HT_COUNT]

.Lht_put_done:
    ldr x27, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

63 · used in 27f · defines _kcc_ht_put 66b, 104a, 118, 119ab, 132, 171, 254b, 256a, 257b, 312, 320, 321, 322, 417b, 465, 469, 472, 498a, 592c, 593 · uses _kcc_arena_alloc 58a, _kcc_fnv1a 61a

9.3. Dynamic Vector

A vector is a growable array of fixed-size elements. When the vector is full, it doubles in capacity by allocating a new array from the arena and copying the old contents (the old array is wasted, as is typical with arena allocators).

Offset	Field	Description
0	data	Pointer to element array
8	count	Number of elements stored
16	cap	Capacity (number of elements)
24	elem_size	Size of each element in bytes

⟨ *Vec: constants* 64a ⟩ ≡

```
// -- Vector layout ---
.equ VEC_DATA,      0          // pointer to
   element array
.equ VEC_COUNT,     8          // number of
   elements
.equ VEC_CAP,       16         // capacity
.equ VEC_ELEMSIZE, 24          // element size in
   bytes
.equ VEC_SIZE,      32         // total header size
```

□

64a · used in 27g, 28g, 29e, 30abcd, 31abcde

9.3.1. Initialization

kcc_vec_init creates a vector for elements of the given size. The header (32 bytes) and an initial backing array for 16 elements are both arena-allocated. The count starts at zero; the caller never needs to specify an initial capacity.

⟨ *Vec: init* 64b ⟩ ≡

```
// -- kcc_vec_init ---
// void *kcc_vec_init(uint64_t elem_size)
//   x0 = element size in bytes
// Returns pointer to vector header.  Initial
   capacity is 16.
.globl _kcc_vec_init
.p2align 2
_kcc_vec_init:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                  // x19 =
   elem_size

    // Allocate header
    mov x0, #VEC_SIZE
    bl _kcc_arena_alloc
```

```
    mov x20, x0                     // x20 = header

    // Allocate initial data (16 * elem_size)
    mov x21, #16                    // initial
   capacity
    mov x0, x21
    mul x0, x0, x19
    bl _kcc_arena_alloc

    // Fill header
    str  x0, [x20, #VEC_DATA]
    str xzr, [x20, #VEC_COUNT]
    str x21, [x20, #VEC_CAP]
    str x19, [x20, #VEC_ELEMSIZE]

    mov x0, x20                     // return header

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

64b · used in 27g · defines _kcc_vec_init 170a, 253b, 568b · uses _kcc_arena_alloc 58a

9.3.2. Push

Append an element. If the vector is full, a new array of double capacity is allocated from the arena, and the old contents are copied.

⟨ *Vec: push* 65a ⟩ ≡

```
// -- kcc_vec_push ---
// void kcc_vec_push(void *vec, const void *elem)
//   x0 = vector pointer
//   x1 = pointer to element to copy
// Appends a copy of elem to the vector.
.globl _kcc_vec_push
.p2align 2
_kcc_vec_push:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0                     // x19 = vec
    mov x20, x1                     // x20 = elem

    ldr x21, [x19, #VEC_COUNT]  // x21 = count
    ldr x22, [x19, #VEC_CAP]    // x22 = cap

    cmp x21, x22
    b.lo .Lvec_push_store

    // Grow: double capacity
    lsl x22, x22, #1                // new_cap = cap *
    2
    ldr x0, [x19, #VEC_ELEMSIZE]
    mul x0, x22, x0                 // new_cap *
   elem_size
    bl _kcc_arena_alloc
    // x0 = new array

    // Copy old data
    mov x3, x0                      // save new array
    ldr x1, [x19, #VEC_DATA]    // src = old array
    mov x4, x1                      // save old for
   memcpy dst/src swap
    ldr x2, [x19, #VEC_ELEMSIZE]
    mul x2, x21, x2                 // old count *
   elem_size
    mov x0, x3                      // dst = new array
    mov x1, x4                      // src = old array
    bl _kcc_memcpy

    // Update vec header
    str x3, [x19, #VEC_DATA]
    str x22, [x19, #VEC_CAP]

.Lvec_push_store:
    // Compute dest: data + count * elem_size
    ldr x0, [x19, #VEC_DATA]
    ldr x2, [x19, #VEC_ELEMSIZE]
    mul x3, x21, x2
    add x0, x0, x3                  // dst

    mov x1, x20                     // src = elem
    // x2 = elem_size (still loaded)
    bl _kcc_memcpy

    // Increment count
    add x21, x21, #1
    str x21, [x19, #VEC_COUNT]

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

65a · used in 27g · defines _kcc_vec_push 170ab, 253c, 257a, 305b, 316, 317, 469, 476, 477b, 516, 554, 592ab · uses _kcc_arena_alloc 58a

9.3.3. Element Access

kcc_vec_get returns a pointer to the element at a given index, computed as `data + index * elem_size`. No bounds checking is performed — the caller must ensure the index is less than the current count.

⟨ *Vec: get* 65b ⟩ ≡

```
// -- kcc_vec_get ---
// void *kcc_vec_get(void *vec, uint64_t index)
//   x0 = vector pointer
//   x1 = element index
// Returns pointer to element at the given index.
// No bounds checking (caller's responsibility).
.globl _kcc_vec_get
.p2align 2
_kcc_vec_get:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    ldr x2, [x0, #VEC_ELEMSIZE]
    ldr x3, [x0, #VEC_DATA]
    mul x1, x1, x2
    add x0, x3, x1                  // data + index *
   elem_size

    ldp x29, x30, [sp], #16
    ret
```

□

65b · used in 27g · defines _kcc_vec_get 171, 254b, 255, 256ab, 257b, 334, 441, 445a, 447, 448, 453b, 454, 457, 472, 473, 474, 478, 481, 490, 493, 498a, 559, 560b, 568a, 580, 581, 586, 587, 588, 589ab, 596

9.4. String Interning

String interning ensures that each unique string is stored exactly once in the arena. Two interned strings can be compared for equality by pointer comparison rather than byte-by-byte — a critical optimization for the symbol table.

The intern table is a hash table mapping string contents to their canonical arena-allocated copy.

`kcc_strtab_init` creates the global intern table as a 256-slot hash table and stores its pointer in `_kcc_strtab`. The capacity is generous for typical translation units; since the hash table never resizes, choosing a power of two large enough to keep the load factor low is important.

⟨ *Strtab: init* 66a ⟩ ≡

```
// -- kcc_strtab_init ---
// void kcc_strtab_init(void)
// Creates the global string intern table.
.globl _kcc_strtab_init
.p2align 2
_kcc_strtab_init:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    mov x0, #256                    // initial
   capacity
    bl _kcc_ht_init

    adrp x1, _kcc_strtab@PAGE
    add  x1, x1, _kcc_strtab@PAGEOFF
    str  x0, [x1]                   // store table
   pointer

    ldp x29, x30, [sp], #16
    ret
```

□

66a · used in 28a · defines _kcc_strtab_init 603, 622, 623 · uses _kcc_ht_init 61c, _kcc_strtab 66c

`kcc_strtab_intern` looks up a string in the table. If found, it returns the existing interned pointer. If not, it copies the string into the arena, inserts it, and returns the new pointer.

⟨ *Strtab: intern* 66b ⟩ ≡

```
// -- kcc_strtab_intern ---
// const char *kcc_strtab_intern(const char *s,
   uint64_t len)
//   x0 = pointer to string (not necessarily null-
   terminated)
//   x1 = string length
// Returns pointer to the interned (null-
   terminated) copy.
.globl _kcc_strtab_intern
.p2align 2
_kcc_strtab_intern:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                     // x19 = string
    mov x20, x1                     // x20 = length

    // Look up in the table
    adrp x0, _kcc_strtab@PAGE
    add  x0, x0, _kcc_strtab@PAGEOFF
    ldr  x0, [x0]                   // x0 = table
   pointer
    mov  x1, x19
    mov  x2, x20
    bl _kcc_ht_get
    cbnz x0, .Lstrtab_found         // already
   interned

    // Not found: allocate len+1 bytes and copy
    add x0, x20, #1
    bl _kcc_arena_alloc
    mov x21, x0                     // x21 = new
   buffer

    // Copy string bytes
    mov x1, x19                     // src
    mov x2, x20                     // count
    bl _kcc_memcpy

    // Null-terminate
    strb wzr, [x21, x20]

    // Insert into table
    adrp x0, _kcc_strtab@PAGE
    add  x0, x0, _kcc_strtab@PAGEOFF
    ldr  x0, [x0]
    mov  x1, x21                    // key = interned
    copy
    mov  x2, x20                    // keylen
    mov  x3, x21                    // value = same
   pointer
    bl _kcc_ht_put

    mov x0, x21                     // return
   interned pointer

.Lstrtab_found:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

66b · used in 28a · defines _kcc_strtab_intern 188, 191b, 198, 200b, 203, 205a, 212, 214, 215, 226, 234a · uses _kcc_arena_alloc 58a, _kcc_ht_get 62, _kcc_ht_put 63, _kcc_strtab 66c

9.4.1. Global State

The string intern table maintains a single global pointer to its underlying hash table, initialized to zero (no table allocated) at program start. The table is created lazily on the first call to `kcc_strtab_intern`.

⟨ *Strtab: data* 66c ⟩ ≡

```
// -- String intern table global state ---
.data
.p2align 3
_kcc_strtab: .quad 0                // pointer to
   hash table
```

□

66c · used in 28a · defines _kcc_strtab 66ab

10. String Operations

Without a C library, the compiler must supply its own implementations of the fundamental memory and string primitives. This chapter provides `strlen`, `memcpy`, `memcmp`, `memset`, and `strcmp`—five routines used pervasively by the lexer, the arena, the I/O layer, and the string intern table. Each function follows AAPCS64 conventions and establishes the mandatory Apple frame pointer.

10.1. String Length

Compute the length of a null-terminated string by scanning for the zero byte. This is the moral equivalent of `strlen(3)`.

⟨ *String: strlen* 67a ⟩ ≡

```
// -- kcc_strlen ---
// size_t kcc_strlen(const char *s)
//   x0 = pointer to null-terminated string
// Returns length (not counting the null
   terminator) in x0.
FUNC kcc_strlen
    mov x1, x0                  // x1 = scan
   pointer
.Lstrlen_loop:
    ldrb w2, [x1], #1           // load byte,
   advance
    cbnz w2, .Lstrlen_loop      // loop until
   null
    sub x0, x1, x0              // length = end -
    start
    sub x0, x0, #1              // exclude the
   null byte itself
ENDFUNC
```

□

67a · used in 27d

10.2. Memory Copy

Copy `n` bytes from source to destination. Overlapping regions produce undefined results (as with `memcpy`, not `memmove`).

⟨ *String: memcpy* 67b ⟩ ≡

```
// -- kcc_memcpy ---
// void *kcc_memcpy(void *dst, const void *src,
   size_t n)
//   x0 = destination
//   x1 = source
//   x2 = byte count
// Returns dst in x0.
FUNC kcc_memcpy
    mov x3, x0                  // save dst for
   return value
    cbz x2, .Lmemcpy_done       // nothing to
   copy
.Lmemcpy_loop:
    ldrb w4, [x1], #1           // load from src
    strb w4, [x0], #1           // store to dst
    subs x2, x2, #1
    b.ne .Lmemcpy_loop
.Lmemcpy_done:
    mov x0, x3                  // return
   original dst
ENDFUNC
```

□

67b · used in 27d

10.3. Memory Compare

Compare `n` bytes. Returns 0 if equal, negative if the first differing byte in `s1` is less than in `s2`, positive otherwise.

⟨ *String: memcmp* 67c ⟩ ≡

```
// -- kcc_memcmp ---
// int kcc_memcmp(const void *s1, const void *s2,
   size_t n)
//   x0 = first buffer
//   x1 = second buffer
//   x2 = byte count
// Returns comparison result in x0.
FUNC kcc_memcmp
    cbz x2, .Lmemcmp_equal      // zero length →
   equal
.Lmemcmp_loop:
    ldrb w3, [x0], #1
    ldrb w4, [x1], #1
    cmp w3, w4
    b.ne .Lmemcmp_diff
    subs x2, x2, #1
    b.ne .Lmemcmp_loop
.Lmemcmp_equal:
    mov x0, #0
    b .Lmemcmp_ret
.Lmemcmp_diff:
    sub x0, x3, x4              // s1[i] - s2[i]
.Lmemcmp_ret:
ENDFUNC
```

□

67c · used in 27d

10.4. Memory Set

Fill `n` bytes of memory with the value `c`.

⟨ *String: memset* 67d ⟩ ≡

```
// -- kcc_memset ---
// void *kcc_memset(void *s, int c, size_t n)
//   x0 = destination
//   x1 = fill byte (only low 8 bits used)
//   x2 = byte count
// Returns s in x0.
FUNC kcc_memset
    mov x3, x0                  // save dst for
   return value
    cbz x2, .Lmemset_done
.Lmemset_loop:
    strb w1, [x0], #1
    subs x2, x2, #1
    b.ne .Lmemset_loop
.Lmemset_done:
    mov x0, x3                  // return
   original dst
```

```
ENDFUNC
```

□

67d · used in 27d

10.5. String Compare

Compare two null-terminated strings byte by byte. Returns 0 if equal, a negative value if the first differing byte in `s1` is less than in `s2`, a positive value otherwise — the same contract as `strcmp(3)`.

⟨ *String: strcmp* 68 ⟩ ≡

```
// -- kcc_strcmp ---
// int kcc_strcmp(const char *s1, const char *s2)
//   x0 = first string (null-terminated)
//   x1 = second string (null-terminated)
// Returns comparison result in x0.
FUNC kcc_strcmp
.Lstrcmp_loop:
    ldrb w2, [x0], #1
    ldrb w3, [x1], #1
    cmp w2, w3
    b.ne .Lstrcmp_diff
    cbz w2, .Lstrcmp_equal          // both reached
   null -> equal
    b .Lstrcmp_loop
.Lstrcmp_equal:
    mov x0, #0
    b .Lstrcmp_ret
.Lstrcmp_diff:
    sub x0, x2, x3                  // s1[i] - s2[i]
.Lstrcmp_ret:
ENDFUNC
```

□

68 · used in 27d

11. Buffered I/O

Issuing a syscall for every output character would be ruinously expensive; instead, all compiler output—diagnostics, generated code, debug traces—passes through a buffered I/O layer that accumulates data in a 4 KiB static buffer and flushes it in bulk. This chapter defines the buffer layout and global state, the flush routine, and the individual output primitives: `putc` for single bytes, `puts` for strings, `putu` for unsigned integers, and `putx` for hexadecimal.

11.1. Buffer Layout

The I/O state consists of four values: the file descriptor, the buffer base address, the current write position, and the capacity. For Phase 0, we use a single static buffer writing to `stdout`.

⟨ *IO: data* 69a ⟩ ≡

```
// -- I/O global state ---
.data
.p2align 3
_kcc_io_fd:    .quad 1             // file
   descriptor (1 = stdout)
_kcc_io_pos:   .quad 0             // current write
   position
_kcc_io_cap:   .quad 4096          // buffer
   capacity

.bss
.p2align 4
_kcc_io_buf:   .space 4096         // output buffer
```

□

69a · used in 27e · defines _kcc_io_buf 69bc, 70ab, _kcc_io_cap 69c, _kcc_io_fd 69b, _kcc_io_pos 69bc, 70ab

11.2. Flush

`kcc_io_flush` writes the buffered contents to the file descriptor and resets the position to zero. This is called automatically when the buffer fills, and explicitly before program exit.

⟨ *IO: flush* 69b ⟩ ≡

```
// -- kcc_io_flush ---
// void kcc_io_flush(void)
// Writes buffered data to the file descriptor,
   resets position.
.globl _kcc_io_flush
.p2align 2
_kcc_io_flush:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Load position - if zero, nothing to flush
    adrp x19, _kcc_io_pos@PAGE
    add  x19, x19, _kcc_io_pos@PAGEOFF
    ldr  x2, [x19]                 // x2 = bytes to
   write
    cbz  x2, .Lflush_done

    // Load fd
    adrp x0, _kcc_io_fd@PAGE
    add  x0, x0, _kcc_io_fd@PAGEOFF
    ldr  x0, [x0]                  // x0 = fd

    // Load buffer base
    adrp x1, _kcc_io_buf@PAGE
    add  x1, x1, _kcc_io_buf@PAGEOFF  // x1 = buf

    // x2 already has the byte count
    bl _kcc_sys_write

    // Reset position to 0
    str xzr, [x19]

.Lflush_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

69b · used in 27e · defines _kcc_io_flush 69c, 70ab, 262, 334, 554, 603, 607, 609, 611, 622, 623, 624, 629 · uses _kcc_io_buf 69a, _kcc_io_fd 69a, _kcc_io_pos 69a

11.3. Put Character

`kcc_io_putc` appends a single byte to the buffer. If the buffer is full, it flushes first.

⟨ *IO: putc* 69c ⟩ ≡

```
// -- kcc_io_putc ---
// void kcc_io_putc(int c)
//   x0 = character to write (low 8 bits)
FUNC kcc_io_putc
    // Load buffer state
    adrp x1, _kcc_io_pos@PAGE
    add  x1, x1, _kcc_io_pos@PAGEOFF
    ldr  x2, [x1]                  // x2 = current
   position

    adrp x3, _kcc_io_cap@PAGE
    add  x3, x3, _kcc_io_cap@PAGEOFF
    ldr  x3, [x3]                  // x3 = capacity

    // Flush if buffer is full
    cmp x2, x3
    b.lo .Lputc_store

    // Save the character across the flush call
    str x0, [sp, #-16]!
    bl _kcc_io_flush
    ldr x0, [sp], #16

    // Reload position (should be 0 after flush)
    adrp x1, _kcc_io_pos@PAGE
    add  x1, x1, _kcc_io_pos@PAGEOFF
    ldr  x2, [x1]

.Lputc_store:
    // Store the byte
    adrp x3, _kcc_io_buf@PAGE
    add  x3, x3, _kcc_io_buf@PAGEOFF
    strb w0, [x3, x2]

    // Increment position
    add x2, x2, #1
```

```
    str x2, [x1]
ENDFUNC
```

□

69c · used in 27e · uses _kcc_io_buf 69a, _kcc_io_cap 69a, _kcc_io_flush 69b, _kcc_io_pos 69a

11.4. Put String

kcc_io_puts writes a null-terminated string to the buffer. It processes the string byte by byte, flushing the buffer whenever it fills. The null terminator is not written.

This is the primary output function used by the compiler driver, the diagnostic engine, and (eventually) the code emitter.

⟨ *IO: puts* 70a ⟩ ≡

```
// -- kcc_io_puts ---
// void kcc_io_puts(const char *s)
//   x0 = pointer to null-terminated string
// Writes all bytes (excluding the null terminator
   ) to the buffer.
.globl _kcc_io_puts
.p2align 2
_kcc_io_puts:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0                 // x19 = string
   pointer

    // Cache buffer state in callee-saved
   registers
    adrp x20, _kcc_io_pos@PAGE
    add  x20, x20, _kcc_io_pos@PAGEOFF
    ldr  x21, [x20]             // x21 = current
   position

    adrp x22, _kcc_io_buf@PAGE
    add  x22, x22, _kcc_io_buf@PAGEOFF  // x22 =
   buffer base

.Lputs_loop:
    ldrb w0, [x19], #1          // load byte,
   advance string ptr
    cbz w0, .Lputs_done         // null
   terminator → stop

    // Store byte in buffer
    strb w0, [x22, x21]
    add x21, x21, #1

    // Check if buffer is full (4096 bytes)
    cmp x21, #4096
    b.lo .Lputs_loop

    // Buffer full - flush
    str x21, [x20]              // save pos
   before flush
    bl _kcc_io_flush
    ldr x21, [x20]              // reload pos (
   should be 0)
    b .Lputs_loop

.Lputs_done:
    str x21, [x20]              // save final
   position
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

70a · used in 27e · defines _kcc_io_puts 99, 173, 175ab, 177, 249ab, 250a, 262, 334, 335, 336, 337, 338, 339, 441, 621, 623, 624, 629 · uses _kcc_io_buf 69a, _kcc_io_flush 69b, _kcc_io_pos 69a

11.5. Write N Bytes

kcc_io_write writes exactly n bytes from a pointer to the output buffer. Unlike puts, it does not look for a null terminator — the caller supplies the byte count. This is essential for printing token text, which is a (pointer, length) pair into the source buffer rather than a null-terminated string.

⟨ *IO: write* 70b ⟩ ≡

```
// -- kcc_io_write ---
// void kcc_io_write(const void *ptr, size_t len)
//   x0 = pointer to data
//   x1 = byte count
// Writes exactly len bytes to the output buffer.
.globl _kcc_io_write
.p2align 2
_kcc_io_write:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0                  // x19 = source
   pointer
    mov x20, x1                  // x20 =
   remaining count

    // Cache buffer state
    adrp x21, _kcc_io_pos@PAGE
    add  x21, x21, _kcc_io_pos@PAGEOFF

    adrp x22, _kcc_io_buf@PAGE
    add  x22, x22, _kcc_io_buf@PAGEOFF

    cbz x20, .Lwrite_done

.Lwrite_loop:
    ldr  x1, [x21]               // x1 = current
   pos
    ldrb w2, [x19], #1           // load byte from
   source
    strb w2, [x22, x1]           // store in buffer
    add  x1, x1, #1
    str  x1, [x21]               // update pos

    cmp x1, #4096
    b.lo .Lwrite_no_flush
    bl _kcc_io_flush
.Lwrite_no_flush:
    subs x20, x20, #1
    b.ne .Lwrite_loop

.Lwrite_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

70b · used in 27e · defines _kcc_io_write 71a, 99, 175b · uses _kcc_io_buf 69a, _kcc_io_flush 69b, _kcc_io_pos 69a

11.6. Print Unsigned Integer

kcc_io_putu converts an unsigned 64-bit integer to decimal ASCII and writes it to the output buffer. This is used by --dump-tokens to print line and column numbers.

The algorithm divides repeatedly by 10, storing digits in reverse order in a small stack buffer, then writes them out with kcc_io_write.

⟨ *IO: putu* 71a ⟩ ≡

```
// -- kcc_io_putu ---
// void kcc_io_putu(uint64_t n)
//   x0 = unsigned integer to print
// Outputs the decimal representation to the
   buffered output.
.globl _kcc_io_putu
.p2align 2
_kcc_io_putu:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    str x19, [sp, #16]
    // digit buffer: sp+24 through sp+47 (24 bytes
   , room for 20 digits)

    mov x19, x0                  // x19 = number
    add x3, sp, #48              // x3 = one past
   end of buffer
    mov x4, x3                   // x4 = saved end
   pointer

    // Handle zero specially
    cbnz x19, .Lputu_nonzero
    mov w1, #'0'
    sub x3, x3, #1
    strb w1, [x3]
    b .Lputu_print

.Lputu_nonzero:
    mov x5, #10
.Lputu_loop:
    udiv x1, x19, x5             // x1 = n / 10
    msub x2, x1, x5, x19         // x2 = n % 10
    add  w2, w2, #'0'            // convert to
   ASCII
    sub  x3, x3, #1
    strb w2, [x3]
    mov  x19, x1
    cbnz x19, .Lputu_loop

.Lputu_print:
    mov x0, x3                    // pointer to
   first digit
    sub x1, x4, x3               // length
    bl _kcc_io_write

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

71a · used in 27e · defines _kcc_io_putu 99, 173, 175b, 250a, 334, 335, 336, 337, 338, 629 · uses _kcc_io_write 70b

11.7. Stderr Output

kcc_io_err writes a null-terminated string directly to stderr (file descriptor 2) via a raw syscall. No buffering is used — diagnostic messages should appear immediately, even if the program crashes afterwards.

⟨ *IO: err* 71b ⟩ ≡

```
// -- kcc_io_err ---
// void kcc_io_err(const char *s)
//   x0 = null-terminated string
// Writes directly to stderr (fd=2), unbuffered.
.globl _kcc_io_err
.p2align 2
_kcc_io_err:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                   // save string
   pointer
    bl _kcc_strlen                // x0 = length

    mov x2, x0                   // x2 = byte count
    mov x1, x19                  // x1 = buffer
    mov x0, #2                   // x0 = fd (stderr
   )
    bl _kcc_sys_write

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

71b · used in 27e · defines _kcc_io_err 73c, 75, 81b, 522b, 554, 577b, 579, 607, 609, 611, 624

12. Diagnostic Engine

The C standard (§5.1.1.3) requires that a conforming implementation issue at least one diagnostic for any syntax rule or constraint violation. This chapter implements a structured diagnostic engine that formats messages in the conventional `file:line:col: severity: message` form, supports three severity levels (error, warning, note), and enforces a fatal error limit to prevent cascading output from overwhelming the user.

12.1. Severity Levels

The C standard distinguishes only "diagnostic message" from other output, but modern practice recognizes three levels: errors that prevent translation, warnings that flag suspicious code, and notes that provide supplementary context. We encode these as integer constants.

The fatal error limit prevents the compiler from drowning the user in cascading errors. After `DIAG_MAX_ERRORS` errors, the engine prints a summary and calls `sys_exit(1)`.

⟨ *Diag: constants* 73a ⟩ ≡

```
// -- Diagnostic severity codes ---
.equ DIAG_ERROR,       0
.equ DIAG_WARNING,     1
.equ DIAG_NOTE,        2

// Maximum errors before aborting
.equ DIAG_MAX_ERRORS, 20
```

□

73a · used in 28bg, 29e, 30a

12.2. Stderr Integer Output

The diagnostic engine needs to print line and column numbers to stderr. This helper converts a `uint64_t` to decimal ASCII and writes it directly to file descriptor 2 via `sys_write`. The algorithm mirrors `kcc_io_putu` (Chapter 11) but bypasses the stdout buffer.

⟨ *Diag: putu stderr* 73b ⟩ ≡

```
// -- _kcc_diag_putu ---
// void _kcc_diag_putu(uint64_t n)
//   x0 = unsigned integer to print
// Outputs the decimal representation directly to
   stderr (fd=2).
.p2align 2
_kcc_diag_putu:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                    // x19 = number
    add x3, sp, #48                // x3 = one past
   end of digit buffer
    mov x4, x3                     // x4 = saved end
    pointer

    // Handle zero specially
    cbnz x19, .Ldiag_putu_nonzero
    mov w1, #'0'
    sub x3, x3, #1
    strb w1, [x3]
    b .Ldiag_putu_print

.Ldiag_putu_nonzero:
    mov x5, #10
.Ldiag_putu_loop:
    udiv x1, x19, x5               // x1 = n / 10
    msub x2, x1, x5, x19           // x2 = n % 10
    add  w2, w2, #'0'              // convert to
   ASCII digit
    sub  x3, x3, #1
    strb w2, [x3]
    mov  x19, x1
    cbnz x19, .Ldiag_putu_loop

.Ldiag_putu_print:
    // sys_write(2, digits, count)
    mov x0, #2                     // fd = stderr
    mov x1, x3                     // buffer
    sub x2, x4, x3                 // length
    bl _kcc_sys_write

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

73b · used in 28b

12.3. Core Emission

The heart of the diagnostic engine is `kcc_diag_emit`. Given a severity level, source location, and message string, it writes the full diagnostic to stderr in the standard format.

Each component (file path, colon, number, severity label, message) is written as a separate string to stderr. This avoids the need for `sprintf`-style formatting. Stderr writes are unbuffered, so diagnostics appear immediately even if the program crashes.

After an error diagnostic, the error counter is incremented. If it reaches `DIAG_MAX_ERRORS`, a "too many errors" message is printed and the process exits with status 1.

⟨ *Diag: emit* 73c ⟩ ≡

```
// -- kcc_diag_emit ---
// void kcc_diag_emit(int severity, uint64_t line,
    uint64_t col,
//                    const char *msg)
//   x0 = severity (DIAG_ERROR/DIAG_WARNING/
   DIAG_NOTE)
//   x1 = line number (1-based)
//   x2 = column number (1-based)
//   x3 = pointer to null-terminated message
   string
// Writes: file:line:col: severity: message\n   to
    stderr.
.globl _kcc_diag_emit
.p2align 2
_kcc_diag_emit:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
```

```
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    mov x19, x0                 // x19 = severity
    mov x20, x1                 // x20 = line
    mov x21, x2                 // x21 = column
    mov x22, x3                 // x22 = message

    // Print file path (from source manager)
    adrp x0, _kcc_src_path@PAGE
    add  x0, x0, _kcc_src_path@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Ldiag_no_path
    bl _kcc_io_err
    b .Ldiag_colon1
.Ldiag_no_path:
    adrp x0, _kcc_diag_unknown_file@PAGE
    add  x0, x0, _kcc_diag_unknown_file@PAGEOFF
    bl _kcc_io_err

.Ldiag_colon1:
    // Print ":"
    adrp x0, _kcc_diag_colon@PAGE
    add  x0, x0, _kcc_diag_colon@PAGEOFF
    bl _kcc_io_err

    // Print line number
    mov x0, x20
    bl _kcc_diag_putu

    // Print ":"
    adrp x0, _kcc_diag_colon@PAGE
    add  x0, x0, _kcc_diag_colon@PAGEOFF
    bl _kcc_io_err

    // Print column number
    mov x0, x21
    bl _kcc_diag_putu

    // Print ": "
    adrp x0, _kcc_diag_colon_space@PAGE
    add  x0, x0, _kcc_diag_colon_space@PAGEOFF
    bl _kcc_io_err

    // Print severity string based on level
    cmp x19, #DIAG_WARNING
    b.eq .Ldiag_sev_warn
    cmp x19, #DIAG_NOTE
    b.eq .Ldiag_sev_note
    // Default: error
    adrp x0, _kcc_diag_str_error@PAGE
    add  x0, x0, _kcc_diag_str_error@PAGEOFF
    b .Ldiag_sev_done
.Ldiag_sev_warn:
    adrp x0, _kcc_diag_str_warning@PAGE
    add  x0, x0, _kcc_diag_str_warning@PAGEOFF
    b .Ldiag_sev_done
.Ldiag_sev_note:
    adrp x0, _kcc_diag_str_note@PAGE
    add  x0, x0, _kcc_diag_str_note@PAGEOFF
.Ldiag_sev_done:
    bl _kcc_io_err

    // Print ": "
    adrp x0, _kcc_diag_colon_space@PAGE
    add  x0, x0, _kcc_diag_colon_space@PAGEOFF
    bl _kcc_io_err

    // Print message
    mov x0, x22
    bl _kcc_io_err

    // Print newline
    adrp x0, _kcc_diag_newline@PAGE
    add  x0, x0, _kcc_diag_newline@PAGEOFF
    bl _kcc_io_err

    // Print the offending source line and a caret
    under the column
    mov x0, x20                 // line
    mov x1, x21                 // column
    bl _kcc_diag_caret

    // Update counters based on severity
    cmp x19, #DIAG_ERROR
    b.ne .Ldiag_check_warn

    // Increment error count
    adrp x0, _kcc_diag_error_count@PAGE
    add  x0, x0, _kcc_diag_error_count@PAGEOFF
    ldr  x1, [x0]
    add  x1, x1, #1
    str  x1, [x0]

    // Check fatal error limit
    cmp x1, #DIAG_MAX_ERRORS
    b.lt .Ldiag_done

    // Too many errors - abort
    adrp x0, _kcc_diag_too_many@PAGE
    add  x0, x0, _kcc_diag_too_many@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Ldiag_check_warn:
    cmp x19, #DIAG_WARNING
    b.ne .Ldiag_done

    // Increment warning count
    adrp x0, _kcc_diag_warn_count@PAGE
    add  x0, x0, _kcc_diag_warn_count@PAGEOFF
    ldr  x1, [x0]
    add  x1, x1, #1
    str  x1, [x0]

.Ldiag_done:
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

73c · used in 28b · defines _kcc_diag_emit 76abc, 169 · uses _kcc_diag_caret 75, _kcc_diag_colon 77a, _kcc_diag_colon_space 77a, _kcc_diag_error_count 77a, _kcc_diag_newline 77a, _kcc_diag_str_error 77a, _kcc_diag_str_note 77a, _kcc_diag_str_warning 77a, _kcc_diag_too_many 77a, _kcc_diag_unknown_file 77a, _kcc_diag_warn_count 77a, _kcc_io_err 71b, _kcc_src_path 82

12.4. Source Snippet and Caret

A location alone (`file:line:col`) tells the reader *where* the problem is; a caret shows it. Immediately after the message line, kcc reproduces the offending source line verbatim and, on the line below, a single ^ positioned under the reported column — exactly the presentation `clang` and `gcc` use. This turns an abstract coordinate into something the eye lands on directly.

The routine reads straight from the live source buffer (`_kcc_src_base`/`_kcc_src_size`), which the driver leaves pointing at the current translation unit and the preproces-

sor restores to the includer on every #include pop. It is written to be *unfailing*: a null buffer, a zero line, or a line number past the end of the file simply suppresses the snippet rather than risking a bad read — diagnostics must never themselves crash. Because carets only render on the error path, and every valid program never reaches it, they add nothing to the output of correct code.

Two details make the caret land in the right place. First, the column is 1-based, so the indent is `col` − 1 characters. Second, to stay aligned when the line contains tabs, the indent is copied *from the source line itself* — a tab in the source yields a tab in the caret line, a non-tab yields a space — so the terminal's own tab stops keep the ^ under the right glyph. The whole caret line is assembled in a stack buffer and written in one syscall; an over-long indent is clamped to the buffer, which at worst nudges the caret left on a pathologically wide line.

⟨ *Diag: caret* 75 ⟩ ≡

```
// -- _kcc_diag_caret ---
// void _kcc_diag_caret(uint64_t line, uint64_t
   col)
//   x0 = line (1-based), x1 = column (1-based)
// Prints the source line for `line`, then a caret
    under `col`.
// No-op if the source buffer is absent or the
   line is out of range.
.equ CARET_MAX, 500
.p2align 2
_kcc_diag_caret:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    sub sp, sp, #512              // scratch buffer
    for the caret line

    mov x24, x0                   // x24 = line
    mov x19, x1                   // x19 = col

    adrp x2, _kcc_src_base@PAGE
    add  x2, x2, _kcc_src_base@PAGEOFF
    ldr  x2, [x2]                 // x2 = base
    cbz  x2, .Lcaret_done
    cbz  x24, .Lcaret_done        // line 0 →
   nothing to show
    adrp x3, _kcc_src_size@PAGE
    add  x3, x3, _kcc_src_size@PAGEOFF
    ldr  x3, [x3]
    add  x22, x2, x3              // x22 = end
   pointer

    // Walk to the start of the target line
    mov  x4, x2                   // p = base
    mov  x5, #1                   // curline
.Lcaret_findline:
    cmp  x5, x24
    b.ge .Lcaret_found
.Lcaret_scan:
    cmp  x4, x22
    b.hs .Lcaret_done             // ran off the end
    before reaching line
    ldrb w6, [x4], #1
    cmp  w6, #10                  // '\n'
    b.ne .Lcaret_scan
    add  x5, x5, #1
    b    .Lcaret_findline
.Lcaret_found:
    mov  x20, x4                  // x20 = line
   start
    mov  x21, x4                  // x21 = line end
    (scan forward)
.Lcaret_endscan:
    cmp  x21, x22
    b.hs .Lcaret_printline
    ldrb w6, [x21]
    cmp  w6, #10
    b.eq .Lcaret_printline
    add  x21, x21, #1
    b    .Lcaret_endscan

.Lcaret_printline:
    // Write the source line [x20, x21) to stderr,
    then a newline
    mov  x0, #2
    mov  x1, x20
    sub  x2, x21, x20
    bl   _kcc_sys_write
    adrp x0, _kcc_diag_newline@PAGE
    add  x0, x0, _kcc_diag_newline@PAGEOFF
    bl   _kcc_io_err

    // Build the caret line: (col-1) indent chars,
    then '^', then '\n'
    mov  x23, sp                  // x23 = stack
   buffer
    mov  x9, #0                   // x9 = bytes in
   buffer
    subs x10, x19, #1             // x10 = indent
   count = col-1
    b.le .Lcaret_put              // col <= 1 → no
   indent
    mov  x11, x20                 // x11 = source
   cursor (for tab copy)
.Lcaret_indent:
    cmp  x10, #0
    b.eq .Lcaret_put
    cmp  x9, #CARET_MAX
    b.hs .Lcaret_put              // clamp
   pathologically wide lines
    mov  w12, #32                 // default: space
    cmp  x11, x21
    b.hs .Lcaret_store
    ldrb w13, [x11]
    cmp  w13, #9                  // tab → copy tab
   for alignment
    b.ne .Lcaret_store
    mov  w12, #9
.Lcaret_store:
    strb w12, [x23, x9]
    add  x9, x9, #1
    add  x11, x11, #1
    sub  x10, x10, #1
    b    .Lcaret_indent
.Lcaret_put:
    mov  w12, #0x5E               // '^'
    strb w12, [x23, x9]
    add  x9, x9, #1
    mov  w12, #10                 // '\n'
    strb w12, [x23, x9]
    add  x9, x9, #1
    mov  x0, #2
    mov  x1, x23
    mov  x2, x9
    bl   _kcc_sys_write

.Lcaret_done:
    add sp, sp, #512
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

75 · used in 28b · defines _kcc_diag_caret 73c · uses _kcc_diag_newline 77a, _kcc_io_err 71b, _kcc_src_base 82, _kcc_src_size 82

12.5. Convenience Wrappers

Three wrappers provide a simpler calling convention for the common case where the caller already has line, column, and message:

`kcc_diag_error(line, col, msg)`	emits an error
`kcc_diag_warning(line, col, msg)`	emits a warning
`kcc_diag_note(line, col, msg)`	emits a note

These are tail-call wrappers: they rearrange the arguments and branch (not `bl`) to `kcc_diag_emit`, so the return address in x30 passes through untouched.

⟨ *Diag: error* 76a ⟩ ≡

```
// -- kcc_diag_error ---
// void kcc_diag_error(uint64_t line, uint64_t col
   , const char *msg)
//   x0 = line, x1 = col, x2 = msg
.globl _kcc_diag_error
.p2align 2
_kcc_diag_error:
    mov x3, x2                  // msg → x3
    mov x2, x1                  // col → x2
    mov x1, x0                  // line → x1
    mov x0, #DIAG_ERROR         // severity → x0
    b _kcc_diag_emit            // tail call
```

□

76a · used in 28b · defines _kcc_diag_error 91a, 94b, 95, 108, 111, 118, 125a, 135, 139ab, 151, 159b, 188, 259b, 273, 276, 277, 284, 286, 287, 289, 290 · uses _kcc_diag_emit 73c

The warning wrapper is identical in structure, substituting `DIAG_WARNING` for the severity argument.

⟨ *Diag: warning* 76b ⟩ ≡

```
// -- kcc_diag_warning ---
// void kcc_diag_warning(uint64_t line, uint64_t
   col, const char *msg)
.globl _kcc_diag_warning
.p2align 2
_kcc_diag_warning:
    mov x3, x2
    mov x2, x1
    mov x1, x0
    mov x0, #DIAG_WARNING
    b _kcc_diag_emit
```

□

76b · used in 28b · defines _kcc_diag_warning 110, 277, 281, 284, 287 · uses _kcc_diag_emit 73c

The note wrapper completes the set, using `DIAG_NOTE` as the severity. Notes are typically emitted immediately after an error to provide additional context (e.g., "previous definition was here").

⟨ *Diag: note* 76c ⟩ ≡

```
// -- kcc_diag_note ---
// void kcc_diag_note(uint64_t line, uint64_t col,
    const char *msg)
.globl _kcc_diag_note
.p2align 2
_kcc_diag_note:
    mov x3, x2
    mov x2, x1
    mov x1, x0
    mov x0, #DIAG_NOTE
    b _kcc_diag_emit
```

□

76c · used in 28b · defines _kcc_diag_note never used · uses _kcc_diag_emit 73c

12.6. Error Count Query

`kcc_diag_had_errors` returns the current error count. The driver uses this after tokenization to decide whether to continue with subsequent phases or exit with a failure status.

⟨ *Diag: had errors* 76d ⟩ ≡

```
// -- kcc_diag_had_errors ---
// uint64_t kcc_diag_had_errors(void)
// Returns the error count in x0.
.globl _kcc_diag_had_errors
.p2align 2
_kcc_diag_had_errors:
    adrp x0, _kcc_diag_error_count@PAGE
    add  x0, x0, _kcc_diag_error_count@PAGEOFF
    ldr  x0, [x0]
    ret
```

□

76d · used in 28b · defines _kcc_diag_had_errors 603, 622, 623 · uses _kcc_diag_error_count 77a

12.7. Reset

`kcc_diag_reset` clears both error and warning counters. This is used by the test harness between individual tests to prevent counters from one test affecting the next.

⟨ *Diag: reset* 76e ⟩ ≡

```
// -- kcc_diag_reset ---
// void kcc_diag_reset(void)
// Resets error and warning counts to zero.
.globl _kcc_diag_reset
.p2align 2
_kcc_diag_reset:
    adrp x0, _kcc_diag_error_count@PAGE
    add  x0, x0, _kcc_diag_error_count@PAGEOFF
    str  xzr, [x0]
    adrp x0, _kcc_diag_warn_count@PAGE
    add  x0, x0, _kcc_diag_warn_count@PAGEOFF
    str  xzr, [x0]
    ret
```

□

76e · used in 28b · defines _kcc_diag_reset 629 · uses _kcc_diag_error_count 77a, _kcc_diag_warn_count 77a

12.8. Global State

The diagnostic engine maintains two counters, a set of format strings, and a single policy flag. The counters are initialized to zero by the `.data` section. `_kcc_warn_all` records whether `-Wall` was passed on the command line; the driver sets it while scanning arguments, and the warning checks scattered through semantic analysis consult it before emitting. Keeping the whole `-Wall` family behind one default-off flag means a compile without it produces exactly the output it did before warnings existed, so no existing test's stderr shifts.

⟨ *Diag: data* 77a ⟩ ≡

```
// -- Diagnostic global state ---
.data
.p2align 3
_kcc_diag_error_count:  .quad 0     // running
   error count
_kcc_diag_warn_count:   .quad 0     // running
   warning count
.globl _kcc_warn_all
_kcc_warn_all:          .quad 0     // -Wall
   enabled (set by the driver from argv)

// Format strings (read-only)
.section __TEXT,__cstring,cstring_literals
_kcc_diag_colon:          .asciz ":"
_kcc_diag_colon_space:    .asciz ": "
_kcc_diag_newline:        .asciz "\n"
_kcc_diag_str_error:      .asciz "error"
_kcc_diag_str_warning:    .asciz "warning"
_kcc_diag_str_note:       .asciz "note"
_kcc_diag_unknown_file:   .asciz "<input>"
_kcc_diag_too_many:       .asciz "kcc: fatal: too
   many errors emitted, stopping\n"
```

□

77a · used in 28b · defines _kcc_diag_colon 73c, _kcc_diag_colon_space 73c, _kcc_diag_error_count 73c, 76de, _kcc_diag_newline 73c, 75, _kcc_diag_str_error 73c, _kcc_diag_str_note 73c, _kcc_diag_str_warning 73c, _kcc_diag_too_many 73c, _kcc_diag_unknown_file 73c, _kcc_diag_warn_count 73c, 76e, _kcc_warn_all 277, 281, 284, 287, 608b, 609, 611

12.9. Diagnostic Tests

Correct diagnostics are as much a feature as correct code generation, and they need their own kind of test: not a program that compiles and runs, but one that *must not*. The harness below feeds kcc a series of invalid translation units and checks that the compiler (a) exits non-zero, (b) prints the expected error text, and (c) does not hang—each run is wrapped in an `alarm` so a runaway recovery loop fails loudly instead of stalling the suite. A final "clean" case guards the other direction: an implicit `printf` call must still compile without an undeclared-identifier error, so the leniency the call path depends on is not lost. `make diagtest` runs it.

⟨ *Diag: tests* 77b ⟩ ≡

```
#!/bin/bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"

# kcc diagnostic tests (invalid inputs must be
   rejected clearly) - generated by weft
set -u
PASS=0
FAIL=0
KCC="$BASE/kcc"
TMP="/$BASE/kcc_diag_$$"
mkdir -p "$TMP"
trap 'rm -rf "$TMP"' EXIT

# run_diag name mode pattern src
#   mode=err   : expect a non-zero exit whose
   output contains <pattern>
#   mode=clean : expect a zero exit (no
   diagnostics)
run_diag() {
    local name="$1"; local mode="$2"; local
   pattern="$3"; local src="$4"
    printf '%s\n' "$src" > "$TMP/d.c"
    local out rc
    out=$(perl -e 'alarm 8; exec @ARGV' "$KCC" -c
   "$TMP/d.c" -o "$TMP/d.o" 2>&1)
    rc=$?
    if [ "$rc" = "142" ] || [ "$rc" = "137" ];
   then
        printf "  %-26s FAIL (hang/timeout)\n" "
   $name"; FAIL=$((FAIL+1)); return
    fi
    if [ "$mode" = "clean" ]; then
        if [ "$rc" -eq 0 ]; then printf "  %-26s
   ok (clean)\n" "$name"; PASS=$((PASS+1))
        else printf "  %-26s FAIL (wanted clean,
   rc=%s)\n" "$name" "$rc"; FAIL=$((FAIL+1)); fi
        return
    fi
    if [ "$rc" -ne 0 ] && printf '%s' "$out" |
   grep -qF "$pattern"; then
        printf "  %-26s ok\n" "$name"; PASS=$((
   PASS+1))
    else
        printf "  %-26s FAIL (rc=%s, wanted '%s')\
   n" "$name" "$rc" "$pattern"; FAIL=$((FAIL+1))
    fi
}

echo "=== diagnostics (invalid inputs must be
   rejected) ==="
run_diag "undeclared-var"    err "use of
   undeclared identifier 'nope'" \
    "int main(void){ return nope; }"
run_diag "undeclared-two"    err "use of
   undeclared identifier 'b'" \
    "int main(void){ return a + b; }"
run_diag "missing-semicolon" err "expected ';'" \
    "int main(void){ return 0 }"
run_diag "missing-expression" err "expected
   expression" \
    "int x = ;"
run_diag "missing-rparen"    err "expected ')'" \
    "int f(void){ return (1 + 2; }"
run_diag "recover-past-syntax" err "use of
   undeclared identifier" \
    "int a = ;
int main(void){ return undeclared_name; }"
run_diag "implicit-printf-ok" clean "" \
    "int main(void){ printf(\"hi\"); return 0; }"
run_diag "call-non-function"  err "called object
   is not a function" \
    "int main(void){ int x = 5; return x(); }"
run_diag "member-non-struct"  err "member access
   on non-struct/union type" \
    "int main(void){ int x = 5; return x.f; }"
```

```
run_diag "no-such-member"     err "no member named
    'zzz'" \
    "struct P { int a; }; int main(void){ struct P
    p; return p.zzz; }"
run_diag "redefinition-local" err "redefinition of
    'x'" \
    "int main(void){ int x; int x; return 0; }"
run_diag "redefinition-param" err "redefinition of
    'x'" \
    "int f(int x, int x){ return x; }"
run_diag "tentative-ok"       clean "" \
    "int x;
int x;
int main(void){ return x; }"
run_diag "shadow-ok"          clean "" \
    "int main(void){ int x = 1; { int x = 2;
   return x; } }"
run_diag "void-returns-value" err "void function
   should not return a value" \
    "void f(void){ return 5; }
int main(void){ f(); return 0; }"
run_diag "nonvoid-bare-return" err "non-void
   function should return a value" \
    "int f(void){ return; }
int main(void){ return f(); }"
run_diag "void-bare-return-ok" clean "" \
    "void f(void){ return; }
int main(void){ f(); return 0; }"
run_diag "int-returns-value-ok" clean "" \
    "int f(void){ return 5; }
int main(void){ return f(); }"
run_diag "vla-local"          err "variable-length
    arrays are not supported" \
    "int main(void){ int n = 4; int a[n]; a[0]=1;
   return a[0]; }"
run_diag "vla-param"          err "variable-length
    arrays are not supported" \
    "int f(int n){ char b[n]; b[0]=1; return b[0];
    }
int main(void){ return f(3); }"
run_diag "fixed-array-ok"     clean "" \
    "enum { N = 4 }; int g[N]; int main(void){ int
    a[10]; a[0]=1; return a[0]+g[0]+(int)sizeof(a)
   ; }"
run_diag "complex-type"       err "complex types (
   _Complex/_Imaginary) are not supported" \
    "int main(void){ double _Complex z; return 0;
   }"
run_diag "atomic-qual-ok"     clean "" \
    "_Atomic int x; int main(void){ x = 5; return
   x; }"
run_diag "int128-arith"       err "128-bit integer
    arithmetic (__int128) is not yet supported" \
    "unsigned __int128 f(unsigned __int128 a,
   unsigned __int128 b){ return a + b; }
int main(void){ return (int)f(1,2); }"
run_diag "int128-decl-ok"     clean "" \
    "typedef unsigned __int128 u128; union h {
   u128 full; struct { unsigned long lo, hi; }; };
int main(void){ union h x; x.lo=5; x.hi=0; return
   (int)(sizeof(u128) + x.lo - 16); }"

echo ""
echo "=== diagtest: $PASS ok, $FAIL fail ==="
[ "$FAIL" -eq 0 ]
```

□

77b · used in 37n

12.10. Warning Tests

Warnings are the mirror image of the diagnostics above: the program is *valid*, so it must still compile (exit 0, object file produced), yet under `-Wall` kcc must *say something*. The harness below therefore checks three things at once for each case—that the compile succeeds, that the expected warning text appears, and, in the `gated` cases, that the very same program is *silent* without `-Wall`. That last check is the load-bearing one: it is what proves the whole warning family stays behind its flag and leaves the default build byte-for-byte as it was. `make warntest` runs it.

⟨ *Warn: tests* 78 ⟩ ≡

```
#!/bin/bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# kcc warning tests (valid programs; -Wall must
   warn, default must not) - generated by weft
set -u
PASS=0
FAIL=0
KCC="$BASE/kcc"
TMP="/$BASE/kcc_warn_$$"
mkdir -p "$TMP"
trap 'rm -rf "$TMP"' EXIT

# run_warn name mode pattern src
#   mode=warn  : with -Wall, expect exit 0, object
    produced, output contains <pattern>
#   mode=clean : with -Wall, expect exit 0 and NO
   "warning:" in the output
#   mode=gated : WITHOUT -Wall, expect exit 0 and
   NO "warning:" in the output
run_warn() {
    local name="$1"; local mode="$2"; local
   pattern="$3"; local src="$4"
    printf '%s\n' "$src" > "$TMP/w.c"
    rm -f "$TMP/w.o"
    local wall="-Wall"
    [ "$mode" = "gated" ] && wall=""
    local out rc
    out=$(perl -e 'alarm 8; exec @ARGV' "$KCC" -c
   "$TMP/w.c" -o "$TMP/w.o" $wall 2>&1)
    rc=$?
    if [ "$rc" != "0" ]; then
        printf "  %-26s FAIL (compile rc=%s, want
   0)\n" "$name" "$rc"; FAIL=$((FAIL+1)); return
    fi
    if [ ! -f "$TMP/w.o" ]; then
        printf "  %-26s FAIL (no object produced)\
   n" "$name"; FAIL=$((FAIL+1)); return
    fi
    if [ "$mode" = "warn" ]; then
        if printf '%s' "$out" | grep -qF "$pattern
   "; then
            printf "  %-26s ok\n" "$name"; PASS=$
   ((PASS+1))
        else
            printf "  %-26s FAIL (wanted warning
   '%s')\n" "$name" "$pattern"; FAIL=$((FAIL+1))
        fi
    else
        # clean / gated: no warning at all
        if printf '%s' "$out" | grep -q "warning:"
   ; then
            printf "  %-26s FAIL (unexpected
   warning)\n" "$name"; FAIL=$((FAIL+1))
        else
```

```
            printf "  %-26s ok (%s)\n" "$name" "
   $mode"; PASS=$((PASS+1))
        fi
    fi
}

echo "=== warnings (-Wall on valid programs) ==="
run_warn "implicit-func"       warn "implicit
   declaration of function 'foo'" \
    "int main(void){ return foo(3); }"
run_warn "unused-var"          warn "unused
   variable 'x'" \
    "int main(void){ int x = 5; int y = 3; return
   y; }"
run_warn "unused-nested"       warn "unused
   variable 'z'" \
    "int main(void){ { int z = 9; } return 0; }"
run_warn "used-var-clean"      clean "" \
    "int main(void){ int x = 5; return x; }"
run_warn "write-counts-clean"  clean "" \
    "int main(void){ int x; x = 5; return 0; }"
run_warn "nested-use-clean"    clean "" \
    "int main(void){ int x = 5; { return x; } }"
run_warn "param-not-warned"    clean "" \
    "int f(int p){ return 0; }
int main(void){ return f(1); }"
run_warn "unused-gated"        gated "" \
    "int main(void){ int x = 5; int y = 3; return
   y; }"
run_warn "implicit-gated"      gated "" \
    "int main(void){ return foo(3); }"
run_warn "reach-end-plain"     warn "control
   reaches end of non-void function" \
    "int f(void){ int x = 1; }
int main(void){ return f(); }"
run_warn "reach-end-if"        warn "control
   reaches end of non-void function" \
    "int f(int c){ if(c) return 1; }
int main(void){ return f(0); }"
run_warn "return-ok-clean"     clean "" \
    "int f(int c){ if(c) return 1; else return 2;
   }
int main(void){ return f(1); }"
run_warn "infinite-loop-clean" clean "" \
    "int f(void){ for(;;){ } }
int main(void){ return 0; }"
run_warn "noreturn-call-clean" clean "" \
    "void abort(void);
int f(void){ abort(); }
int main(void){ return 0; }"
run_warn "switch-return-clean" clean "" \
    "int f(int c){ switch(c){ default: return 0; }
    }
int main(void){ return f(1); }"
run_warn "main-fallthrough-clean" clean "" \
    "int main(void){ }"
run_warn "reach-end-gated"     gated "" \
    "int f(void){ int x = 1; }
int main(void){ return f(); }"
run_warn "sign-compare-var"    warn "comparison of
    integers of different signedness" \
    "int f(int a, unsigned b){ return a < b; }
int main(void){ return f(1, 2); }"
run_warn "sign-compare-neg"    warn "comparison of
    integers of different signedness" \
    "int f(unsigned u){ return u < -1; }
int main(void){ return f(1); }"
run_warn "sign-const-clean"    clean "" \
    "int f(unsigned u){ return u < 10; }
int main(void){ return f(1); }"
run_warn "sign-samesign-clean" clean "" \
    "int f(unsigned a, unsigned b){ return a < b;
   }
int main(void){ return f(1, 2); }"
run_warn "sign-ptr-clean"      clean "" \
    "int f(int *p, int *q){ return p < q; }
int main(void){ int x; return f(&x, &x); }"
run_warn "sign-compare-gated"  gated "" \
    "int f(int a, unsigned b){ return a < b; }
int main(void){ return f(1, 2); }"
run_warn "int-to-ptr-init"     warn "incompatible
   integer to pointer conversion" \
    "int *p = 5;
int main(void){ return 0; }"
run_warn "int-to-ptr-assign"   warn "incompatible
   integer to pointer conversion" \
    "int main(void){ int *p; int n = 5; p = n;
   return 0; }"
run_warn "ptr-to-int-return"   warn "incompatible
   pointer to integer conversion" \
    "int f(void){ int x; return &x; }
int main(void){ return 0; }"
run_warn "null-const-clean"    clean "" \
    "int *p = 0;
int main(void){ int *q; q = 0; return 0; }"
run_warn "ptr-to-ptr-clean"    clean "" \
    "int main(void){ int x; int *p = &x; int *q =
   p; return *q; }"
run_warn "cast-clean"          clean "" \
    "int main(void){ int *p = (int*)5; return p !=
    0; }"
run_warn "compound-ptr-clean"  clean "" \
    "int main(void){ int a[3]; int *p = a; p += 1;
    return 0; }"
run_warn "int-conv-gated"      gated "" \
    "int *p = 5;
int main(void){ return 0; }"

echo ""
echo "=== warntest: $PASS ok, $FAIL fail ==="
[ "$FAIL" -eq 0 ]
```

□

78 · used in 38a

13. Source File Management

Before the lexer can scan a single token, the source file must be loaded into memory. The source manager handles this by memory-mapping each file with `mmap`, giving the lexer a contiguous, zero-copy view of the entire translation unit. This chapter describes the source buffer layout, the file-opening sequence, and the global state that tracks the currently active source.

13.1. Design

A compiler must read source files efficiently. The simplest approach is to `mmap` the entire file: the kernel provides a read-only view of the file's contents, paged in on demand. The lexer then scans directly through the mapped memory without any copying.

For each source file we track:

- The base pointer (start of mapped region)
- The file size (length of mapped region)
- The file path (for diagnostic messages)

Offset	Field	Description
0	`base`	Pointer to start of mapped file
8	`size`	File size in bytes
16	`path`	Pointer to file path string

⟨ *Source: constants* 81a ⟩ ≡

```
// -- Source buffer layout ---
.equ SRC_BASE,      0         // pointer to mmap'd
    file data
.equ SRC_SIZE,      8         // file size in
   bytes
.equ SRC_PATH,     16         // pointer to file
   path (null-terminated)
.equ SRC_BUFSIZE,  24         // structure size
```

□

81a · used in 28c

13.2. Opening a Source File

`kcc_source_open` opens a C source file by path. The sequence is:

1. open the file read-only.
2. `lseek(fd, 0, SEEK_END)` to determine the file size.
3. `mmap` the file with `PROT_READ | MAP_PRIVATE`.
4. `close` the file descriptor (the mapping persists).
5. Store the base pointer, size, and path in the global source state.

If any step fails, the function prints an error message to stderr and exits with status 1.

⟨ *Source: open file* 81b ⟩ ≡

```
// -- kcc_source_open ---
// void kcc_source_open(const char *path)
//   x0 = null-terminated file path
// Opens and mmaps the file, stores result in
   global state.
// On failure, prints error to stderr and exits.
.globl _kcc_source_open
.p2align 2
_kcc_source_open:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                    // x19 = path

    // Store path in global state
    adrp x1, _kcc_src_path@PAGE
    add  x1, x1, _kcc_src_path@PAGEOFF
    str  x19, [x1]

    // open(path, O_RDONLY, 0)
    mov x0, x19                    // path
    mov x1, #0                     // O_RDONLY = 0
    mov x2, #0                     // mode (unused
  for O_RDONLY)
    bl _kcc_sys_open

    // Check for error
    cmn x0, #1
    b.eq .Lsource_open_fail
    mov x20, x0                    // x20 = fd

    // lseek(fd, 0, SEEK_END) to get file size
    mov x0, x20
    mov x1, #0
    mov x2, #SEEK_END
    bl _kcc_sys_lseek

    cmn x0, #1
    b.eq .Lsource_seek_fail
    mov x21, x0                    // x21 = file
  size

    // Reject empty files
    cbz x21, .Lsource_empty

    // mmap(NULL, size, PROT_READ, MAP_PRIVATE, fd
  , 0)
    mov x0, #0                     // addr = NULL
    mov x1, x21                    // len = file
  size
    mov x2, #PROT_READ             // prot = read
  only
    mov x3, #MAP_PRIVATE           // flags =
  private
    mov x4, x20                    // fd
    mov x5, #0                     // offset = 0
    bl _kcc_sys_mmap

    cmn x0, #1
    b.eq .Lsource_mmap_fail

    // Store base and size in global state
    adrp x1, _kcc_src_base@PAGE
```

```
    add  x1, x1, _kcc_src_base@PAGEOFF
    str  x0, [x1]

    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    str  x21, [x1]

    // Close the file descriptor (mapping persists)
    mov x0, x20
    bl _kcc_sys_close

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lsource_open_fail:
    adrp x0, _kcc_err_open@PAGE
    add  x0, x0, _kcc_err_open@PAGEOFF
    bl _kcc_io_err
    mov x0, x19
    bl _kcc_io_err
    adrp x0, _kcc_src_newline@PAGE
    add  x0, x0, _kcc_src_newline@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lsource_seek_fail:
    mov x0, x20
    bl _kcc_sys_close
    adrp x0, _kcc_err_read@PAGE
    add  x0, x0, _kcc_err_read@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lsource_empty:
    mov x0, x20
    bl _kcc_sys_close
    adrp x0, _kcc_err_empty@PAGE
    add  x0, x0, _kcc_err_empty@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lsource_mmap_fail:
    mov x0, x20
    bl _kcc_sys_close
    adrp x0, _kcc_err_mmap@PAGE
    add  x0, x0, _kcc_err_mmap@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit
```

□

81b · used in 28c · defines _kcc_source_open 151, 603, 622, 623 · uses _kcc_err_empty 82, _kcc_err_mmap 82, _kcc_err_open 82, _kcc_err_read 82, _kcc_io_err 71b, _kcc_src_base 82, _kcc_src_newline 82, _kcc_src_path 82, _kcc_src_size 82

13.3. Global State

The source manager's state consists of the mapped buffer, its size, and the file path. These are accessed by the lexer when scanning.

⟨ *Source: data* 82 ⟩ ≡

```
// -- Source manager global state ---
.data
.p2align 3
.globl _kcc_src_base
_kcc_src_base:  .quad 0       // pointer to mmap'd file contents
.globl _kcc_src_size
_kcc_src_size:  .quad 0       // file size in bytes
.globl _kcc_src_path
_kcc_src_path:  .quad 0       // pointer to file path string

.section __TEXT,__cstring,cstring_literals
_kcc_err_open:   .asciz "kcc: error: cannot open file: "
_kcc_err_read:   .asciz "kcc: error: cannot read file\n"
_kcc_err_empty:  .asciz "kcc: error: source file is empty\n"
_kcc_err_mmap:   .asciz "kcc: error: cannot map file\n"
_kcc_src_newline: .asciz "\n"
```

□

82 · used in 28c · defines _kcc_err_empty 81b, _kcc_err_mmap 81b, _kcc_err_open 81b, _kcc_err_read 81b, _kcc_src_base 75, 81b, 85, 87, 107a, 111, 112, 151, 603, 622, 623, 630a, _kcc_src_newline 81b, _kcc_src_path 73c, 81b, 107a, 111, 112, 121, 136, 148, 151, 596, 630a, _kcc_src_size 75, 81b, 85, 87, 107a, 111, 112, 151, 603, 622, 623, 630a

Part IV.

The Frontend

The frontend transforms a stream of bytes into a structured representation of the program. It follows the C17 translation phases almost to the letter: trigraph replacement and line splicing (Chapters 14–15), tokenisation into the roughly one hundred token types defined by the grammar (Chapter 16), preprocessing with macro expansion and conditional compilation (Chapter 17), string literal processing and adjacent-literal concatenation (Chapters 18–19), and finally recursive-descent parsing of declarations, expressions, and statements into an abstract syntax tree (Chapters 20–23). The parser is hand-written — no generator tools — so every production rule is visible in the assembly, and error recovery follows a simple synchronisation-point strategy.

14. Source Encoding — Translation Phase 1

The first phase of C17 translation (§5.1.1.2, paragraph 1) maps physical source characters to the source character set and replaces trigraph sequences with their single-character equivalents. Although trigraphs are a historical relic from terminals that lacked certain keys, a conforming compiler must handle all nine sequences before any other processing occurs. This chapter defines the trigraph mapping table and implements the `kcc_source_encode` function that performs the replacement into a writable arena buffer.

14.1. Trigraph Sequences

§5.2.1.1 defines exactly nine trigraph sequences:

Trigraph	Replacement	Trigraph	Replacement	Trigraph	Replacement
??=	#	??(	[	??/	\
??)	]	??'	^	??<	{
??>	}	??!	|	??-	~

The replacement must occur before any other processing. In particular, ??/ produces a backslash that participates in line splicing (Phase 2), so Phase 1 must complete before Phase 2 begins.

14.2. Implementation

`kcc_source_encode` performs trigraph replacement on the source buffer. Because the mmap'd file is read-only (`PROT_READ`), we always copy the source into a writable arena-allocated buffer. This copy also prepares the buffer for in-place line splicing in Phase 2.

The function scans the source with a read pointer and writes to the new buffer. When ?? is encountered, the third character is checked against the nine trigraph mappings. If a match is found, the single replacement character is written and the read pointer advances by 3. Otherwise, a single ? is written and the read pointer advances by 1.

⟨ *Encoding: translate* 85 ⟩ ≡

```
// -- kcc_source_encode ---
// void kcc_source_encode(void)
// Translation Phase 1: replaces trigraphs in the
   source buffer.
// Allocates a writable arena copy of the mmap'd
   source, performs
// trigraph replacement during the copy, and
   updates _kcc_src_base
// and _kcc_src_size to point to the new buffer.
.globl _kcc_source_encode
.p2align 2
_kcc_source_encode:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    // Load source base and size
    adrp x19, _kcc_src_base@PAGE
    add  x19, x19, _kcc_src_base@PAGEOFF
    ldr  x20, [x19]              // x20 = read
   pointer (old base)

    adrp x21, _kcc_src_size@PAGE
    add  x21, x21, _kcc_src_size@PAGEOFF
    ldr  x22, [x21]              // x22 = source
   size

    // Allocate writable buffer from arena (same
   size as original)
    mov x0, x22
    bl _kcc_arena_alloc
    mov x23, x0                  // x23 = write
   base (new buffer)
    mov x24, x0                  // x24 = write
   pointer

    // Compute end of source
    add x22, x20, x22            // x22 = read end

.Lenc_loop:
    cmp x20, x22
    b.hs .Lenc_done

    ldrb w0, [x20]

    // Check for '?' - potential trigraph start
    cmp w0, #'?'
    b.ne .Lenc_copy_byte

    // Need at least 2 more characters for a
   trigraph
    add x1, x20, #2
    cmp x1, x22
    b.hs .Lenc_copy_byte

    // Check second '?'
    ldrb w1, [x20, #1]
    cmp w1, #'?'
    b.ne .Lenc_copy_byte

    // Check third character for trigraph match
    ldrb w1, [x20, #2]

    cmp w1, #'='
    b.eq .Lenc_trig_hash         // ??= → #
    cmp w1, #'('
```

```
    b.eq .Lenc_trig_lbracket    // ??( → [
    cmp w1, #'/'
    b.eq .Lenc_trig_backslash   // ??/ → backslash
    cmp w1, #')'
    b.eq .Lenc_trig_rbracket    // ??) → ]
    cmp w1, #'\''
    b.eq .Lenc_trig_caret       // ??' → ^
    cmp w1, #'<'
    b.eq .Lenc_trig_lbrace      // ??< → {
    cmp w1, #'!'
    b.eq .Lenc_trig_pipe        // ??! → |
    cmp w1, #'>'
    b.eq .Lenc_trig_rbrace      // ??> → }
    cmp w1, #'-'
    b.eq .Lenc_trig_tilde       // ??- → ~

    // Not a valid trigraph - copy the first '?'
   and advance by 1
    b .Lenc_copy_byte

.Lenc_trig_hash:
    mov w0, #'#'
    b .Lenc_write_trig
.Lenc_trig_lbracket:
    mov w0, #'['
    b .Lenc_write_trig
.Lenc_trig_backslash:
    mov w0, #'\\'
    b .Lenc_write_trig
.Lenc_trig_rbracket:
    mov w0, #']'
    b .Lenc_write_trig
.Lenc_trig_caret:
    mov w0, #'^'
    b .Lenc_write_trig
.Lenc_trig_lbrace:
    mov w0, #'{'
    b .Lenc_write_trig
.Lenc_trig_pipe:
    mov w0, #'|'
    b .Lenc_write_trig
.Lenc_trig_rbrace:
    mov w0, #'}'
    b .Lenc_write_trig
.Lenc_trig_tilde:
    mov w0, #'~'
    // fall through to write_trig

.Lenc_write_trig:
    strb w0, [x24], #1          // write
   replacement character
    add x20, x20, #3            // skip 3 source
   characters
    b .Lenc_loop

.Lenc_copy_byte:
    ldrb w0, [x20], #1          // read and
   advance
    strb w0, [x24], #1          // write and
   advance
    b .Lenc_loop

.Lenc_done:
    // Update global source state
    str x23, [x19]               // _kcc_src_base
   = new buffer
    sub x24, x24, x23           // compute actual
   output size
    str x24, [x21]               // _kcc_src_size
   = output size

    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

85 · used in 28c · defines _kcc_source_encode 151, 603, 622, 623, 630a · uses _kcc_arena_alloc 58a, _kcc_src_base 82, _kcc_src_size 82

15. Line Splicing — Translation Phase 2

The second phase of C17 translation (§5.1.1.2, paragraph 2) deletes every backslash-newline pair, splicing physical source lines into logical lines. This enables macros and string literals to span multiple lines in the source file. Because Phase 1 has already replaced the trigraph ??/ with a real backslash, the splice pass sees all backslash characters uniformly and operates in-place on the writable arena buffer.

15.1. Design

Line splicing removes every \
n pair from the source. Since this operation can only *shrink* the buffer (deleting 2 characters per splice), it can be performed in-place on the writable arena buffer produced by Phase 1.

The function uses a read pointer and a write pointer, both starting at `src_base`. When the read pointer encounters \ followed by
n (0x0A), both characters are skipped. Otherwise, the byte is copied from read to write position. The final buffer size is the write pointer's offset from the base.

⟨ *Splice: translate* 87 ⟩ ≡

```
// -- kcc_source_splice ---
// void kcc_source_splice(void)
// Translation Phase 2: removes backslash-newline
   sequences in-place.
// Requires that _kcc_src_base points to a
   writable buffer (as
// guaranteed by Phase 1's arena copy).
// Updates _kcc_src_size to reflect the (possibly
   shorter) buffer.
.globl _kcc_source_splice
.p2align 2
_kcc_source_splice:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Load source base and size
    adrp x19, _kcc_src_base@PAGE
    add  x19, x19, _kcc_src_base@PAGEOFF
    ldr  x20, [x19]              // x20 = base (
   read pointer)

    adrp x21, _kcc_src_size@PAGE
    add  x21, x21, _kcc_src_size@PAGEOFF
    ldr  x22, [x21]              // x22 = size

    // Compute read end
    add x1, x20, x22             // x1 = read end
    mov x2, x20                  // x2 = write
   pointer
    mov x3, x20                  // x3 = read
   pointer

.Lspl_loop:
    cmp x3, x1
    b.hs .Lspl_done

    ldrb w4, [x3]

    // Check for backslash
    cmp w4, #'\\'
    b.ne .Lspl_copy

    // Check if next character is newline
    add x5, x3, #1
    cmp x5, x1
    b.hs .Lspl_copy              // backslash at
   very end - keep it

    ldrb w6, [x5]
    cmp w6, #0x0A                // newline?
    b.ne .Lspl_copy

    // Skip the backslash-newline pair
    add x3, x3, #2
    b .Lspl_loop

.Lspl_copy:
    strb w4, [x2], #1            // copy byte,
   advance write pointer
    add x3, x3, #1               // advance read
   pointer
    b .Lspl_loop

.Lspl_done:
    // Update size = write pointer - base
    sub x2, x2, x20
    str x2, [x21]                // _kcc_src_size
   = new size

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

87 · used in 28c · defines _kcc_source_splice 151, 603, 622, 623, 630a · uses _kcc_src_base 82, _kcc_src_size 82

16. Lexical Analysis

Lexical analysis is Phase 3 of the C17 translation pipeline (§ 5.1.1.2, paragraph 3): the source buffer is decomposed into preprocessing tokens according to the longest-match rule of § 6.4. The lexer scans identifiers, numeric constants, string and character literals, and punctuators, while a hash table maps identifier text to keyword token codes. This chapter begins with the token type constants, then covers initialization, the main `kcc_lex_next` dispatch loop, individual scanners for each token category, keyword recognition, and the global state that connects the lexer to the rest of the compiler.

16.1. Token Types

Every C17 token falls into one of the categories defined in § 6.4: keywords, identifiers, constants, string literals, and punctuators. We assign a unique integer code to each. Codes 0–9 are structural, 10–53 are the 44 keywords, and 60+ are punctuators.

⟨ *Lexer: token constants* 89 ⟩ ≡

```
// -- Token type codes ---
// Structural tokens
.equ TOK_EOF,           0
.equ TOK_IDENT,         1
.equ TOK_NUMBER,        2       // pp-number (
   integer or float)
.equ TOK_CHAR_LIT,      3       // character
   constant
.equ TOK_STRING_LIT,    4       // string literal
.equ TOK_UNKNOWN,       5       // unrecognized
   character

// C17 keywords (section 6.4.1) - 44 keywords,
   codes 10-53
.equ TOK_AUTO,          10
.equ TOK_BREAK,         11
.equ TOK_CASE,          12
.equ TOK_CHAR,          13
.equ TOK_CONST,         14
.equ TOK_CONTINUE,      15
.equ TOK_DEFAULT,       16
.equ TOK_DO,            17
.equ TOK_DOUBLE,        18
.equ TOK_ELSE,          19
.equ TOK_ENUM,          20
.equ TOK_EXTERN,        21
.equ TOK_FLOAT,         22
.equ TOK_FOR,           23
.equ TOK_GOTO,          24
.equ TOK_IF,            25
.equ TOK_INLINE,        26
.equ TOK_INT,           27
.equ TOK_LONG,          28
.equ TOK_REGISTER,      29
.equ TOK_RESTRICT,      30
.equ TOK_RETURN,        31
.equ TOK_SHORT,         32
.equ TOK_SIGNED,        33
.equ TOK_SIZEOF,        34
.equ TOK_STATIC,        35
.equ TOK_STRUCT,        36
.equ TOK_SWITCH,        37
.equ TOK_TYPEDEF,       38
.equ TOK_UNION,         39
.equ TOK_UNSIGNED,      40
.equ TOK_VOID,          41
.equ TOK_VOLATILE,      42
.equ TOK_WHILE,         43
.equ TOK__ALIGNAS,      44
.equ TOK__ALIGNOF,      45
.equ TOK__ATOMIC,       46
.equ TOK__BOOL,         47
.equ TOK__COMPLEX,      48
.equ TOK__GENERIC,      49
.equ TOK__IMAGINARY,    50
.equ TOK__NORETURN,     51
.equ TOK__STATIC_ASSERT, 52
.equ TOK__THREAD_LOCAL,  53

// GNU/common extension keywords (codes 54-59),
   recognised so real system
// headers---which lean on them heavily---parse.
   `__inline`, `__const`, and the
// other `__x__` spellings of standard keywords
   are registered as ALIASES that
// lex to the standard keyword's own code, so no
   parser change is needed for them.
.equ TOK_ATTRIBUTE,     54      // __attribute__ /
   __attribute
.equ TOK_TYPEOF,        55      // typeof /
   __typeof__ / __typeof
.equ TOK_ASM,           56      // asm / __asm__ /
   __asm
.equ TOK_EXTENSION,     57      // __extension__ (a
   no-op prefix)
.equ TOK_INT128,        58      // __int128 (GNU
   128-bit integer type specifier)

// Punctuators (section 6.4.6) - codes 60+
.equ TOK_LPAREN,        60      // (
.equ TOK_RPAREN,        61      // )
.equ TOK_LBRACE,        62      // {
.equ TOK_RBRACE,        63      // }
.equ TOK_LBRACKET,      64      // [
.equ TOK_RBRACKET,      65      // ]
.equ TOK_SEMI,          66      // ;
.equ TOK_COMMA,         67      // ,
.equ TOK_DOT,           68      // .
.equ TOK_ARROW,         69      // ->
.equ TOK_INC,           70      // ++
.equ TOK_DEC,           71      // --
.equ TOK_AMP,           72      // &
.equ TOK_STAR,          73      // *
.equ TOK_PLUS,          74      // +
.equ TOK_MINUS,         75      // -
.equ TOK_TILDE,         76      // ~
.equ TOK_BANG,          77      // !
.equ TOK_SLASH,         78      // /
.equ TOK_PERCENT,       79      // %
.equ TOK_SHL,           80      // <<
.equ TOK_SHR,           81      // >>
.equ TOK_LT,            82      // <
.equ TOK_GT,            83      // >
.equ TOK_LE,            84      // <=
.equ TOK_GE,            85      // >=
.equ TOK_EQ,            86      // ==
.equ TOK_NE,            87      // !=
.equ TOK_CARET,         88      // ^
.equ TOK_PIPE,          89      // |
.equ TOK_AND,           90      // &&
.equ TOK_OR,            91      // ||
.equ TOK_QUESTION,      92      // ?
```

```
.equ TOK_COLON,       93      // :
.equ TOK_ASSIGN,      94      // =
.equ TOK_MUL_ASSIGN,  95      // *=
.equ TOK_DIV_ASSIGN,  96      // /=
.equ TOK_MOD_ASSIGN,  97      // %=
.equ TOK_ADD_ASSIGN,  98      // +=
.equ TOK_SUB_ASSIGN,  99      // -=
.equ TOK_SHL_ASSIGN, 100      // <<=
.equ TOK_SHR_ASSIGN, 101      // >>=
.equ TOK_AND_ASSIGN, 102      // &=
.equ TOK_XOR_ASSIGN, 103      // ^=
.equ TOK_OR_ASSIGN,  104      // |=
.equ TOK_HASH,       105      // #
.equ TOK_DHASH,      106      // ##
.equ TOK_ELLIPSIS,   107      // ...
```

□

89 · used in 28dfg, 29abcdef, 30ab, 33a

16.2. Lexer Initialization

kcc_lex_init prepares the lexer to scan a source buffer. It sets the cursor to the start of the buffer and initializes the line counter to 1.

⟨ *Lexer: init* 90a ⟩ ≡

```
// -- kcc_lex_init ---
// void kcc_lex_init(const char *base, uint64_t
   size)
//   x0 = pointer to source buffer
//   x1 = buffer size in bytes
// Initializes lexer state to scan the given
   buffer.
.globl _kcc_lex_init
.p2align 2
_kcc_lex_init:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Set cursor and end
    adrp x2, _kcc_lex_cursor@PAGE
    add  x2, x2, _kcc_lex_cursor@PAGEOFF
    str  x0, [x2]

    add  x1, x0, x1             // end = base +
   size
    adrp x2, _kcc_lex_end@PAGE
    add  x2, x2, _kcc_lex_end@PAGEOFF
    str  x1, [x2]

    // Line starts at 1, line_start = base
    mov  x1, #1
    adrp x2, _kcc_lex_line@PAGE
    add  x2, x2, _kcc_lex_line@PAGEOFF
    str  x1, [x2]

    adrp x2, _kcc_lex_line_start@PAGE
    add  x2, x2, _kcc_lex_line_start@PAGEOFF
    str  x0, [x2]

    ldp x29, x30, [sp], #16
    ret
```

□

90a · used in 28d · defines _kcc_lex_init 151, 603, 622, 623, 630a · uses _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_line 104b, _kcc_lex_line_start 104b

16.3. Next Token

kcc_lex_next is the main entry point. It skips whitespace and comments, then dispatches to the appropriate scanner based on the first character. The result is stored in the global token variables.

⟨ *Lexer: next token* 90b ⟩ ≡

```
// -- kcc_lex_next ---
// void kcc_lex_next(void)
// Advances to the next token.  Stores result in
   _kcc_tok_* globals.
.globl _kcc_lex_next
.p2align 2
_kcc_lex_next:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Load cursor and end
    adrp x19, _kcc_lex_cursor@PAGE
    add  x19, x19, _kcc_lex_cursor@PAGEOFF
    ldr  x20, [x19]             // x20 = cursor

    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
    ldr  x21, [x1]              // x21 = end

.Llex_skip_ws:
    // Check EOF
    cmp x20, x21
    b.hs .Llex_eof

    ldrb w0, [x20]

    // Check whitespace (space, tab, \r, \v, \f)
    cmp w0, #' '
    b.eq .Llex_ws_advance
    cmp w0, #0x09               // \t
    b.eq .Llex_ws_advance
    cmp w0, #0x0D               // \r
    b.eq .Llex_ws_advance
    cmp w0, #0x0B               // \v
    b.eq .Llex_ws_advance
    cmp w0, #0x0C               // \f
    b.eq .Llex_ws_advance

    // Check newline
    cmp w0, #0x0A               // \n
    b.eq .Llex_newline

    // Check for comments: // or /*
    cmp w0, #'/'
    b.eq .Llex_maybe_comment

    // Not whitespace; proceed to character
   dispatch
    b .Llex_dispatch

.Llex_ws_advance:
    add x20, x20, #1
    b .Llex_skip_ws

.Llex_newline:
    add x20, x20, #1
    // Increment line counter, update line_start
    adrp x1, _kcc_lex_line@PAGE
    add  x1, x1, _kcc_lex_line@PAGEOFF
    ldr  x2, [x1]
    add  x2, x2, #1
    str  x2, [x1]
```

```
    adrp x1, _kcc_lex_line_start@PAGE
    add  x1, x1, _kcc_lex_line_start@PAGEOFF
    str  x20, [x1]
    b .Llex_skip_ws

.Llex_maybe_comment:
    // Check next char
    add x1, x20, #1
    cmp x1, x21
    b.hs .Llex_dispatch          // just a '/' at
   EOF
    ldrb w2, [x1]
    cmp w2, #'/'
    b.eq .Llex_line_comment
    cmp w2, #'*'
    b.eq .Llex_block_comment
    b .Llex_dispatch             // just a '/'
   punctuator
```

□

90b · defined at 90b, 91ab · used in 28d · defines _kcc_lex_next 100a, 108, 109, 117, 119ab, 123, 135, 136, 139a, 140, 141, 142, 143, 144, 145ab, 146, 147a, 151, 630a · uses _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_line 104b, _kcc_lex_line_start 104b

Comment handling. When the whitespace skipper detects a / character, it peeks one byte ahead. A second / starts a line comment that runs to the next newline; a * starts a block comment that runs to the closing */ sequence. Block comments may span multiple lines, so the newline counter is updated inside the loop.

⟨ *Lexer: next token* 91a ⟩ + ≡

```
.Llex_line_comment:
    // Skip until end of line
    add x20, x20, #2             // skip "//"
.Llex_lc_loop:
    cmp x20, x21
    b.hs .Llex_skip_ws
    ldrb w0, [x20]
    cmp w0, #0x0A
    b.eq .Llex_skip_ws           // newline ends
   comment (don't skip it)
    add x20, x20, #1
    b .Llex_lc_loop

.Llex_block_comment:
    // Skip until */
    add x20, x20, #2             // skip "/*"
.Llex_bc_loop:
    cmp x20, x21
    b.hs .Llex_bc_unterminated
    ldrb w0, [x20]
    cmp w0, #0x0A
    b.eq .Llex_bc_newline
    cmp w0, #'*'
    b.ne .Llex_bc_advance
    // Check for '/' after '*'
    add x1, x20, #1
    cmp x1, x21
    b.hs .Llex_bc_advance
    ldrb w2, [x1]
    cmp w2, #'/'
    b.ne .Llex_bc_advance
    add x20, x1, #1              // skip past "*/"
    b .Llex_skip_ws

.Llex_bc_newline:
    add x20, x20, #1
    adrp x1, _kcc_lex_line@PAGE
    add  x1, x1, _kcc_lex_line@PAGEOFF
    ldr  x2, [x1]
    add  x2, x2, #1
    str  x2, [x1]
    adrp x1, _kcc_lex_line_start@PAGE
    add  x1, x1, _kcc_lex_line_start@PAGEOFF
    str  x20, [x1]
    b .Llex_bc_loop

.Llex_bc_advance:
    add x20, x20, #1
    b .Llex_bc_loop

.Llex_bc_unterminated:
    // Report error at current position via
   diagnostic engine
    adrp x0, _kcc_lex_line@PAGE
    add  x0, x0, _kcc_lex_line@PAGEOFF
    ldr  x0, [x0]                // line
    mov  x1, #1                  // col (
   approximate - comment start unknown)
    adrp x2, _kcc_err_unterm_comment@PAGE
    add  x2, x2, _kcc_err_unterm_comment@PAGEOFF
    bl _kcc_diag_error
    // Treat as EOF
    b .Llex_eof
```

□

91a · defined at 90b, 91ab · used in 28d · uses _kcc_diag_error 76a, _kcc_err_unterm_comment 104b, _kcc_lex_line 104b, _kcc_lex_line_start 104b

EOF and character dispatch. Once whitespace and comments have been consumed, we either hit EOF or a meaningful character. At EOF we emit TOK_EOF and, crucially, *pin the cursor to end* rather than clearing it. A null cursor would be a trap: the preprocessor's _kcc_pp_skip_line and the lexer's own re-entry both bound their scan by comparing the cursor against end, and a null (zero) cursor sits *below* end, so the bound check passes and the very next byte load dereferences address 0. Leaving the cursor at end makes every such check terminate immediately, so a truncated source—# at end of file, an unterminated comment, a directive that runs off the buffer—degrades to a clean diagnostic instead of a segfault. The dispatcher then classifies the first byte into one of four categories—identifier start (letter or underscore), digit, string or character delimiter, or punctuator—and branches to the corresponding scanner subroutine.

⟨ *Lexer: next token* 91b ⟩ + ≡

```
.Llex_eof:
    // Record token location before setting type
    bl _kcc_lex_record_loc
    mov w0, #TOK_EOF
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
    str  x0, [x1]
    str  x21, [x19]              // cursor stays at
    end (NOT null - see prose)
    // tok_start = cursor, tok_len = 0
    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    str  x20, [x1]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    str  xzr, [x1]
    b .Llex_done
```

```
.Llex_dispatch:
    // Save cursor state
    str x20, [x19]

    // Record location (line:col) for this token
    bl _kcc_lex_record_loc

    // Reload cursor
    ldr x20, [x19]
    ldrb w0, [x20]

    // Dispatch based on character class
    // Identifier start: a-z, A-Z, _
    cmp w0, #'_'
    b.eq .Llex_go_ident
    cmp w0, #'a'
    b.hs .Llex_check_lower
    cmp w0, #'A'
    b.hs .Llex_check_upper
    b .Llex_check_digit

.Llex_check_lower:
    cmp w0, #'z'
    b.ls .Llex_go_ident
    b .Llex_check_digit

.Llex_check_upper:
    cmp w0, #'Z'
    b.ls .Llex_go_ident
    b .Llex_check_digit

.Llex_go_ident:
    // A prefix letter (L, u, U) immediately
   followed by a single quote is a
    // prefixed character constant (L'x', u'x', U'
   x'), not an identifier.  kcc
    // represents every character constant as an
   int-valued token, so the prefix
    // only affects the declared type---which the
   char literal path already treats
    // as int---and can be dropped: consume it and
    scan the quote as a char
    // literal.  (Prefixed *string* literals L"/u
   "/U" still need wide elements and
    // are handled elsewhere.)
    cmp w0, #'L'
    b.eq .Llex_pfx_char
    cmp w0, #'u'
    b.eq .Llex_pfx_char
    cmp w0, #'U'
    b.eq .Llex_pfx_char
    b .Llex_ident_scan
.Llex_pfx_char:
    add x1, x20, #1
    cmp x1, x21
    b.hs .Llex_ident_scan        // prefix at end
   of buffer
    ldrb w2, [x1]
    cmp w2, #'\''
    b.ne .Llex_ident_scan        // not a quote →
   ordinary identifier
    add x20, x20, #1             // consume the
   prefix letter
    adrp x1, _kcc_lex_cursor@PAGE
    add  x1, x1, _kcc_lex_cursor@PAGEOFF
    str x20, [x1]                // publish so
   scan_char sees the quote
    b .Llex_go_char
.Llex_ident_scan:
    bl _kcc_lex_scan_ident
    b .Llex_done

.Llex_check_digit:
    cmp w0, #'0'
    b.lo .Llex_check_string
    cmp w0, #'9'
    b.ls .Llex_go_number

    // Check for '.' followed by digit (float like
    .5)
    b .Llex_check_string

.Llex_go_number:
    bl _kcc_lex_scan_number
    b .Llex_done

.Llex_check_string:
    cmp w0, #'"'
    b.eq .Llex_go_string
    cmp w0, #'\''
    b.eq .Llex_go_char
    cmp w0, #'.'
    b.eq .Llex_check_dot_number
    b .Llex_go_punct

.Llex_check_dot_number:
    // '.' might start a float (.5) or be a
   punctuator
    add x1, x20, #1
    cmp x1, x21
    b.hs .Llex_go_punct          // '.' at EOF
    ldrb w2, [x1]
    cmp w2, #'0'
    b.lo .Llex_go_punct
    cmp w2, #'9'
    b.ls .Llex_go_number         // '.digit' ->
   float
    b .Llex_go_punct

.Llex_go_string:
    bl _kcc_lex_scan_string
    b .Llex_done

.Llex_go_char:
    bl _kcc_lex_scan_char
    b .Llex_done

.Llex_go_punct:
    bl _kcc_lex_scan_punct
    b .Llex_done

.Llex_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

91b · defined at 90b, 91ab · used in 28d · uses _kcc_lex_cursor 104b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

16.4. Location Recording

Record the current line and column into the token globals. Column is computed as `cursor − line_start + 1`.

⟨ *Lexer: record location* 92 ⟩ ≡

```
// -- kcc_lex_record_loc ---
// Internal: records current line/col into tok
   globals.
.p2align 2
_kcc_lex_record_loc:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _kcc_lex_line@PAGE
```

```
    add  x0, x0, _kcc_lex_line@PAGEOFF
    ldr  x0, [x0]

    adrp x1, _kcc_tok_line@PAGE
    add  x1, x1, _kcc_tok_line@PAGEOFF
    str  x0, [x1]

    // col = cursor - line_start + 1
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x0, [x0]

    adrp x1, _kcc_lex_line_start@PAGE
    add  x1, x1, _kcc_lex_line_start@PAGEOFF
    ldr  x1, [x1]

    sub x0, x0, x1
    add x0, x0, #1

    adrp x1, _kcc_tok_col@PAGE
    add  x1, x1, _kcc_tok_col@PAGEOFF
    str  x0, [x1]

    ldp x29, x30, [sp], #16
    ret
```

□

92 · used in 28d · uses _kcc_lex_cursor 104b, _kcc_lex_line 104b, _kcc_lex_line_start 104b, _kcc_tok_col 104b, _kcc_tok_line 104b

16.5. Identifier Scanner

An identifier starts with a letter or underscore and continues with letters, digits, or underscores (§6.4.2.1). After scanning, we look up the identifier text in the keyword hash table; if found, the token type is changed to the corresponding keyword code.

⟨ *Lexer: scan identifier* 93 ⟩ ≡

```
// -- kcc_lex_scan_ident ---
// Internal: scans an identifier or keyword.
// Cursor is at the first identifier character.
.p2align 2
_kcc_lex_scan_ident:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Load cursor and end
    adrp x19, _kcc_lex_cursor@PAGE
    add  x19, x19, _kcc_lex_cursor@PAGEOFF
    ldr  x20, [x19]              // cursor

    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
    ldr  x21, [x1]               // end

    mov x22, x20                 // x22 = token
   start

    // Advance past identifier characters
.Lident_loop:
    add x20, x20, #1
    cmp x20, x21
    b.hs .Lident_end
    ldrb w0, [x20]
    // Check: a-z, A-Z, 0-9, _
    cmp w0, #'_'
    b.eq .Lident_loop
    cmp w0, #'a'
    b.lo .Lident_not_lower
    cmp w0, #'z'
    b.ls .Lident_loop
.Lident_not_lower:
    cmp w0, #'A'
    b.lo .Lident_not_upper
    cmp w0, #'Z'
    b.ls .Lident_loop
.Lident_not_upper:
    cmp w0, #'0'
    b.lo .Lident_end
    cmp w0, #'9'
    b.ls .Lident_loop

.Lident_end:
    // Store cursor
    str x20, [x19]

    // Compute length
    sub x2, x20, x22             // x2 = length

    // Store token: start and length
    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    str  x22, [x1]

    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    str  x2, [x1]

    // Default to identifier
    mov x0, #TOK_IDENT
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
    str  x0, [x1]

    // Check keyword table
    adrp x0, _kcc_kw_table@PAGE
    add  x0, x0, _kcc_kw_table@PAGEOFF
    ldr  x0, [x0]                // keyword hash
   table
    cbz  x0, .Lident_done        // not initialized
    yet

    mov  x1, x22                  // key = token
   text
    sub  x2, x20, x22            // keylen = token
   length
    bl _kcc_ht_get
    cbz x0, .Lident_done         // not a keyword

    // It's a keyword: update token type
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
    str  x0, [x1]

.Lident_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

93 · used in 28d · uses _kcc_ht_get 62, _kcc_kw_table 104b, _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

16.6. Number Scanner

A preprocessing number (§6.4.8) is surprisingly broad: it starts with a digit (or . followed by a digit) and continues

with digits, letters, underscores, dots, and e/E/p/P signs. We scan the full pp-number here; value parsing is deferred to Phase 5.

⟨ *Lexer: scan number* 94a ⟩ ≡

```
// -- kcc_lex_scan_number ---
// Internal: scans a preprocessing number.
.p2align 2
_kcc_lex_scan_number:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    adrp x19, _kcc_lex_cursor@PAGE
    add  x19, x19, _kcc_lex_cursor@PAGEOFF
    ldr  x20, [x19]

    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
    ldr  x21, [x1]

    mov x22, x20                // token start

.Lnum_loop:
    add x20, x20, #1
    cmp x20, x21
    b.hs .Lnum_end
    ldrb w0, [x20]

    // digits
    cmp w0, #'0'
    b.lo .Lnum_check_alpha
    cmp w0, #'9'
    b.ls .Lnum_loop

.Lnum_check_alpha:
    // letters (part of pp-number: suffixes, hex
   digits)
    cmp w0, #'a'
    b.lo .Lnum_check_upper
    cmp w0, #'z'
    b.ls .Lnum_check_exp
    b .Lnum_check_dot

.Lnum_check_upper:
    cmp w0, #'A'
    b.lo .Lnum_check_dot
    cmp w0, #'Z'
    b.ls .Lnum_check_exp
    b .Lnum_check_dot

.Lnum_check_exp:
    // If e/E/p/P, next char might be +/-
    cmp w0, #'e'
    b.eq .Lnum_maybe_sign
    cmp w0, #'E'
    b.eq .Lnum_maybe_sign
    cmp w0, #'p'
    b.eq .Lnum_maybe_sign
    cmp w0, #'P'
    b.eq .Lnum_maybe_sign
    b .Lnum_loop                // other letter,
   still pp-number

.Lnum_maybe_sign:
    // Peek at next char for +/-
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lnum_loop
    ldrb w2, [x1]
    cmp w2, #'+'
    b.eq .Lnum_skip_sign
    cmp w2, #'-'
    b.eq .Lnum_skip_sign
    b .Lnum_loop

.Lnum_skip_sign:
    add x20, x20, #1            // consume the
   letter
    b .Lnum_loop                // sign consumed
   in next iteration

.Lnum_check_dot:
    cmp w0, #'.'
    b.eq .Lnum_loop
    cmp w0, #'_'
    b.eq .Lnum_loop

.Lnum_end:
    str x20, [x19]

    // Store token
    sub x2, x20, x22
    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    str  x22, [x1]

    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    str  x2, [x1]

    mov x0, #TOK_NUMBER
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
    str  x0, [x1]

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

94a · used in 28d · uses _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

16.7. String Literal Scanner

A string literal starts and ends with " (§6.4.5). Within the string, backslash escapes are recognized (the actual escape processing is deferred to Phase 5). An unescaped newline or end-of-file before the closing quote is an error.

⟨ *Lexer: scan string* 94b ⟩ ≡

```
// -- kcc_lex_scan_string ---
// Internal: scans a string literal (opening '"'
   at cursor).
.p2align 2
_kcc_lex_scan_string:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    adrp x19, _kcc_lex_cursor@PAGE
    add  x19, x19, _kcc_lex_cursor@PAGEOFF
    ldr  x20, [x19]

    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
    ldr  x21, [x1]
```

```
    mov x22, x20                // token start (
   the opening ")
    add x20, x20, #1            // skip opening "

.Lstr_loop:
    cmp x20, x21
    b.hs .Lstr_unterminated
    ldrb w0, [x20]
    cmp w0, #'"'
    b.eq .Lstr_end
    cmp w0, #'\\'
    b.eq .Lstr_escape
    cmp w0, #0x0A               // newline
    b.eq .Lstr_unterminated
    add x20, x20, #1
    b .Lstr_loop

.Lstr_escape:
    add x20, x20, #2            // skip backslash +
    escaped char
    b .Lstr_loop

.Lstr_end:
    add x20, x20, #1            // skip closing "
    str x20, [x19]

    sub x2, x20, x22
    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    str  x22, [x1]

    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    str  x2, [x1]

    mov x0, #TOK_STRING_LIT
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
    str  x0, [x1]

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lstr_unterminated:
    // Error: unterminated string - report via
   diagnostic engine
    str x20, [x19]
    adrp x0, _kcc_tok_line@PAGE
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x0, [x0]               // line (recorded
    at opening quote)
    adrp x1, _kcc_tok_col@PAGE
    add  x1, x1, _kcc_tok_col@PAGEOFF
    ldr  x1, [x1]               // col (recorded
   at opening quote)
    adrp x2, _kcc_err_unterm_string@PAGE
    add  x2, x2, _kcc_err_unterm_string@PAGEOFF
    bl _kcc_diag_error

    sub x2, x20, x22
    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    str  x22, [x1]

    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    str  x2, [x1]

    mov x0, #TOK_STRING_LIT
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
    str  x0, [x1]

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

94b · used in 28d · uses _kcc_diag_error 76a, _kcc_err_unterm_string 104b, _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_tok_col 104b, _kcc_tok_len 104b, _kcc_tok_line 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

16.8. Character Literal Scanner

Character literals are similar to strings but delimited by single quotes (§6.4.4.4).

⟨ *Lexer: scan char* 95 ⟩ ≡

```
// -- kcc_lex_scan_char ---
// Internal: scans a character constant (opening
   '\'' at cursor).
.p2align 2
_kcc_lex_scan_char:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    adrp x19, _kcc_lex_cursor@PAGE
    add  x19, x19, _kcc_lex_cursor@PAGEOFF
    ldr  x20, [x19]

    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
    ldr  x21, [x1]

    mov x22, x20                // token start
    add x20, x20, #1            // skip opening '

.Lchr_loop:
    cmp x20, x21
    b.hs .Lchr_unterminated
    ldrb w0, [x20]
    cmp w0, #'\''
    b.eq .Lchr_end
    cmp w0, #'\\'
    b.eq .Lchr_escape
    cmp w0, #0x0A
    b.eq .Lchr_unterminated
    add x20, x20, #1
    b .Lchr_loop

.Lchr_escape:
    add x20, x20, #2
    b .Lchr_loop

.Lchr_end:
    add x20, x20, #1            // skip closing '
    str x20, [x19]

    sub x2, x20, x22
    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    str  x22, [x1]

    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    str  x2, [x1]

    mov x0, #TOK_CHAR_LIT
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
```

```
    str  x0, [x1]

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lchr_unterminated:
    str x20, [x19]
    adrp x0, _kcc_tok_line@PAGE
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x0, [x0]                // line (recorded
    at opening quote)
    adrp x1, _kcc_tok_col@PAGE
    add  x1, x1, _kcc_tok_col@PAGEOFF
    ldr  x1, [x1]                // col (recorded
   at opening quote)
    adrp x2, _kcc_err_unterm_char@PAGE
    add  x2, x2, _kcc_err_unterm_char@PAGEOFF
    bl _kcc_diag_error

    sub x2, x20, x22
    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    str  x22, [x1]

    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    str  x2, [x1]

    mov x0, #TOK_CHAR_LIT
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
    str  x0, [x1]

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

95 · used in 28d · uses _kcc_diag_error 76a, _kcc_err_unterm_char 104b, _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_tok_col 104b, _kcc_tok_len 104b, _kcc_tok_line 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

16.9. Punctuator Scanner

The punctuator scanner implements longest-match for all C17 punctuators (§6.4.6). Multi-character punctuators like >>= or ... are handled by peeking ahead.

⟨ *Lexer: scan punctuator* 96 ⟩ ≡

```
// -- kcc_lex_scan_punct ---
// Internal: scans a punctuator.
.p2align 2
_kcc_lex_scan_punct:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    adrp x19, _kcc_lex_cursor@PAGE
    add  x19, x19, _kcc_lex_cursor@PAGEOFF
    ldr  x20, [x19]

    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
    ldr  x21, [x1]

    mov x22, x20                 // token start
    ldrb w0, [x20]

    // Single-character dispatch
    cmp w0, #'('
    b.eq .Lpunct_1_lparen
    cmp w0, #')'
    b.eq .Lpunct_1_rparen
    cmp w0, #'{'
    b.eq .Lpunct_1_lbrace
    cmp w0, #'}'
    b.eq .Lpunct_1_rbrace
    cmp w0, #'['
    b.eq .Lpunct_1_lbracket
    cmp w0, #']'
    b.eq .Lpunct_1_rbracket
    cmp w0, #';'
    b.eq .Lpunct_1_semi
    cmp w0, #','
    b.eq .Lpunct_1_comma
    cmp w0, #'~'
    b.eq .Lpunct_1_tilde
    cmp w0, #'?'
    b.eq .Lpunct_1_question
    cmp w0, #':'
    b.eq .Lpunct_1_colon

    // Two-or-more character punctuators
    cmp w0, #'+'
    b.eq .Lpunct_plus
    cmp w0, #'-'
    b.eq .Lpunct_minus
    cmp w0, #'*'
    b.eq .Lpunct_star
    cmp w0, #'/'
    b.eq .Lpunct_slash
    cmp w0, #'%'
    b.eq .Lpunct_percent
    cmp w0, #'<'
    b.eq .Lpunct_lt
    cmp w0, #'>'
    b.eq .Lpunct_gt
    cmp w0, #'='
    b.eq .Lpunct_eq
    cmp w0, #'!'
    b.eq .Lpunct_bang
    cmp w0, #'&'
    b.eq .Lpunct_amp
    cmp w0, #'|'
    b.eq .Lpunct_pipe
    cmp w0, #'^'
    b.eq .Lpunct_caret
    cmp w0, #'#'
    b.eq .Lpunct_hash
    cmp w0, #'.'
    b.eq .Lpunct_dot

    // Unrecognized character
    add x20, x20, #1
    str x20, [x19]
    mov x0, #TOK_UNKNOWN
    b .Lpunct_store

// -- Simple single-character punctuators ---
.Lpunct_1_lparen:
    mov x0, #TOK_LPAREN
    b .Lpunct_single
.Lpunct_1_rparen:
    mov x0, #TOK_RPAREN
    b .Lpunct_single
.Lpunct_1_lbrace:
    mov x0, #TOK_LBRACE
    b .Lpunct_single
.Lpunct_1_rbrace:
    mov x0, #TOK_RBRACE
    b .Lpunct_single
```

```
.Lpunct_1_lbracket:
    mov x0, #TOK_LBRACKET
    b .Lpunct_single
.Lpunct_1_rbracket:
    mov x0, #TOK_RBRACKET
    b .Lpunct_single
.Lpunct_1_semi:
    mov x0, #TOK_SEMI
    b .Lpunct_single
.Lpunct_1_comma:
    mov x0, #TOK_COMMA
    b .Lpunct_single
.Lpunct_1_tilde:
    mov x0, #TOK_TILDE
    b .Lpunct_single
.Lpunct_1_question:
    mov x0, #TOK_QUESTION
    b .Lpunct_single
.Lpunct_1_colon:
    mov x0, #TOK_COLON
    b .Lpunct_single

.Lpunct_single:
    add x20, x20, #1
    str x20, [x19]
    b .Lpunct_store
```

□

96 · defined at 96, 97ab, 98 · used in 28d · uses _kcc_lex_cursor 104b, _kcc_lex_end 104b

Arithmetic operators. The five arithmetic operators (+, −, *, /, %) each have compound-assignment forms (+=, −=, etc.). Additionally, + and − form increment/decrement (++, −−), and − forms the arrow operator (->). Each group peeks at the next character to decide between the single and compound forms.

⟨ *Lexer: scan punctuator* 97a ⟩ +≡

```
// -- Multi-character punctuators ---
.Lpunct_plus:                       // + ++ +=
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_plus_single
    ldrb w2, [x1]
    cmp w2, #'+'
    b.eq .Lpunct_inc
    cmp w2, #'='
    b.eq .Lpunct_add_assign
.Lpunct_plus_single:
    mov x0, #TOK_PLUS
    b .Lpunct_single
.Lpunct_inc:
    mov x0, #TOK_INC
    b .Lpunct_double
.Lpunct_add_assign:
    mov x0, #TOK_ADD_ASSIGN
    b .Lpunct_double

.Lpunct_minus:                      // - -- -= ->
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_minus_single
    ldrb w2, [x1]
    cmp w2, #'-'
    b.eq .Lpunct_dec
    cmp w2, #'='
    b.eq .Lpunct_sub_assign
    cmp w2, #'>'
    b.eq .Lpunct_arrow
.Lpunct_minus_single:
    mov x0, #TOK_MINUS
    b .Lpunct_single
.Lpunct_dec:
    mov x0, #TOK_DEC
    b .Lpunct_double
.Lpunct_sub_assign:
    mov x0, #TOK_SUB_ASSIGN
    b .Lpunct_double
.Lpunct_arrow:
    mov x0, #TOK_ARROW
    b .Lpunct_double

.Lpunct_star:                       // * *=
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_star_single
    ldrb w2, [x1]
    cmp w2, #'='
    b.eq .Lpunct_mul_assign
.Lpunct_star_single:
    mov x0, #TOK_STAR
    b .Lpunct_single
.Lpunct_mul_assign:
    mov x0, #TOK_MUL_ASSIGN
    b .Lpunct_double

.Lpunct_slash:                      // /  /= (
   comments already handled)
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_slash_single
    ldrb w2, [x1]
    cmp w2, #'='
    b.eq .Lpunct_div_assign
.Lpunct_slash_single:
    mov x0, #TOK_SLASH
    b .Lpunct_single
.Lpunct_div_assign:
    mov x0, #TOK_DIV_ASSIGN
    b .Lpunct_double

.Lpunct_percent:                    // % %=
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_pct_single
    ldrb w2, [x1]
    cmp w2, #'='
    b.eq .Lpunct_mod_assign
.Lpunct_pct_single:
    mov x0, #TOK_PERCENT
    b .Lpunct_single
.Lpunct_mod_assign:
    mov x0, #TOK_MOD_ASSIGN
    b .Lpunct_double
```

□

97a · defined at 96, 97ab, 98 · used in 28d

Comparison, logical, and bitwise operators. The relational operators < and > introduce the richest lookahead: each can form a simple comparison, a shift (<<, >>), a compound relation (<=, >=), or a shift-assign (<<=, >>=)—requiring up to two bytes of lookahead. The remaining operators in this group (=, !, &, |, ^) each have at most one compound form.

⟨ *Lexer: scan punctuator* 97b ⟩ +≡

```
.Lpunct_lt:                         // < << <= <<=
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_lt_single
    ldrb w2, [x1]
    cmp w2, #'<'
    b.eq .Lpunct_shl_check
    cmp w2, #'='
```

```
    b.eq .Lpunct_le
.Lpunct_lt_single:
    mov x0, #TOK_LT
    b .Lpunct_single
.Lpunct_le:
    mov x0, #TOK_LE
    b .Lpunct_double
.Lpunct_shl_check:                  // << or <<=
    add x2, x20, #2
    cmp x2, x21
    b.hs .Lpunct_shl
    ldrb w3, [x2]
    cmp w3, #'='
    b.eq .Lpunct_shl_assign
.Lpunct_shl:
    mov x0, #TOK_SHL
    b .Lpunct_double
.Lpunct_shl_assign:
    mov x0, #TOK_SHL_ASSIGN
    b .Lpunct_triple

.Lpunct_gt:                         // > >> >= >>=
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_gt_single
    ldrb w2, [x1]
    cmp w2, #'>'
    b.eq .Lpunct_shr_check
    cmp w2, #'='
    b.eq .Lpunct_ge
.Lpunct_gt_single:
    mov x0, #TOK_GT
    b .Lpunct_single
.Lpunct_ge:
    mov x0, #TOK_GE
    b .Lpunct_double
.Lpunct_shr_check:
    add x2, x20, #2
    cmp x2, x21
    b.hs .Lpunct_shr
    ldrb w3, [x2]
    cmp w3, #'='
    b.eq .Lpunct_shr_assign
.Lpunct_shr:
    mov x0, #TOK_SHR
    b .Lpunct_double
.Lpunct_shr_assign:
    mov x0, #TOK_SHR_ASSIGN
    b .Lpunct_triple

.Lpunct_eq:                         // = ==
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_assign
    ldrb w2, [x1]
    cmp w2, #'='
    b.eq .Lpunct_eq_eq
.Lpunct_assign:
    mov x0, #TOK_ASSIGN
    b .Lpunct_single
.Lpunct_eq_eq:
    mov x0, #TOK_EQ
    b .Lpunct_double

.Lpunct_bang:                       // ! !=
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_bang_single
    ldrb w2, [x1]
    cmp w2, #'='
    b.eq .Lpunct_ne
.Lpunct_bang_single:
    mov x0, #TOK_BANG
    b .Lpunct_single
.Lpunct_ne:
    mov x0, #TOK_NE
    b .Lpunct_double

.Lpunct_amp:                        // & && &=
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_amp_single
    ldrb w2, [x1]
    cmp w2, #'&'
    b.eq .Lpunct_and
    cmp w2, #'='
    b.eq .Lpunct_and_assign
.Lpunct_amp_single:
    mov x0, #TOK_AMP
    b .Lpunct_single
.Lpunct_and:
    mov x0, #TOK_AND
    b .Lpunct_double
.Lpunct_and_assign:
    mov x0, #TOK_AND_ASSIGN
    b .Lpunct_double

.Lpunct_pipe:                       // | || |=
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_pipe_single
    ldrb w2, [x1]
    cmp w2, #'|'
    b.eq .Lpunct_or
    cmp w2, #'='
    b.eq .Lpunct_or_assign
.Lpunct_pipe_single:
    mov x0, #TOK_PIPE
    b .Lpunct_single
.Lpunct_or:
    mov x0, #TOK_OR
    b .Lpunct_double
.Lpunct_or_assign:
    mov x0, #TOK_OR_ASSIGN
    b .Lpunct_double

.Lpunct_caret:                      // ^ ^=
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_caret_single
    ldrb w2, [x1]
    cmp w2, #'='
    b.eq .Lpunct_xor_assign
.Lpunct_caret_single:
    mov x0, #TOK_CARET
    b .Lpunct_single
.Lpunct_xor_assign:
    mov x0, #TOK_XOR_ASSIGN
    b .Lpunct_double
```

□

97b · defined at 96, 97ab, 98 · used in 28d

Remaining punctuators and shared helpers. The hash (#) and dot (.) punctuators round out the set. Hash can double to form the token-pasting operator ##, while three consecutive dots form the ellipsis .... The shared helpers .Lpunct_double and .Lpunct_triple advance the cursor by two or three bytes, and .Lpunct_store writes the final token type, start pointer, and length into the global variables.

⟨ *Lexer: scan punctuator* 98 ⟩ + ≡

```
.Lpunct_hash:                       // # ##
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_hash_single
    ldrb w2, [x1]
```

```
    cmp w2, #'#'
    b.eq .Lpunct_dhash
.Lpunct_hash_single:
    mov x0, #TOK_HASH
    b .Lpunct_single
.Lpunct_dhash:
    mov x0, #TOK_DHASH
    b .Lpunct_double

.Lpunct_dot:                            // . ...
    add x1, x20, #1
    cmp x1, x21
    b.hs .Lpunct_dot_single
    ldrb w2, [x1]
    cmp w2, #'.'
    b.ne .Lpunct_dot_single
    add x2, x20, #2
    cmp x2, x21
    b.hs .Lpunct_dot_single
    ldrb w3, [x2]
    cmp w3, #'.'
    b.ne .Lpunct_dot_single
    mov x0, #TOK_ELLIPSIS
    b .Lpunct_triple
.Lpunct_dot_single:
    mov x0, #TOK_DOT
    b .Lpunct_single

// -- Length helpers ---
.Lpunct_double:
    add x20, x20, #2
    str x20, [x19]
    b .Lpunct_store

.Lpunct_triple:
    add x20, x20, #3
    str x20, [x19]
    b .Lpunct_store

.Lpunct_store:
    // Store token type, start, length
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
    str  x0, [x1]

    // Compute length from start
    ldr  x20, [x19]
    sub  x2, x20, x22

    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    str  x22, [x1]

    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    str  x2, [x1]

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

98 · defined at 96, 97ab, 98 · used in 28d · uses _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

16.10. Token Dump

The `--dump-tokens` mode prints each token on a separate line in the format:

```
<line>:<col> <type> '<text>'
```

`kcc_lex_dump_token` prints one token; `kcc_lex_dump_all` loops through the entire source.

⟨ *Lexer: dump token* 99 ⟩ ≡

```
// -- kcc_lex_dump_token ---
// void kcc_lex_dump_token(void)
// Prints the current token in dump format.
.globl _kcc_lex_dump_token
.p2align 2
_kcc_lex_dump_token:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Print line number
    adrp x0, _kcc_tok_line@PAGE
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x0, [x0]
    bl _kcc_io_putu

    // Print ':'
    mov x0, #':'
    bl _kcc_io_putc

    // Print column
    adrp x0, _kcc_tok_col@PAGE
    add  x0, x0, _kcc_tok_col@PAGEOFF
    ldr  x0, [x0]
    bl _kcc_io_putu

    // Print ' '
    mov x0, #' '
    bl _kcc_io_putc

    // Print token type name
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    bl _kcc_lex_type_name        // returns
  pointer to name string
    bl _kcc_io_puts

    // Print " '"
    mov x0, #' '
    bl _kcc_io_putc
    mov x0, #'\''
    bl _kcc_io_putc

    // Print token text
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]

    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]

    bl _kcc_io_write

    // Print "'\n"
    mov x0, #'\''
    bl _kcc_io_putc
    mov x0, #0x0A
    bl _kcc_io_putc

    ldp x29, x30, [sp], #16
    ret
```

□

99 · used in 28d · defines _kcc_lex_dump_token 100a, 115 · uses _kcc_io_puts 70a, _kcc_io_putu 71a, _kcc_io_write 70b, _kcc_lex_type_name 100b, _kcc_tok_col 104b, _kcc_tok_len 104b, _kcc_tok_line 104b, _kcc_tok_start 104b,

_kcc_tok_type 104b

The dump-all loop repeatedly calls kcc_lex_next and kcc_lex_dump_token until EOF.

⟨ *Lexer: dump all* 100a ⟩ ≡

```
// -- kcc_lex_dump_all ---
// void kcc_lex_dump_all(void)
// Tokenizes entire source and dumps each token.
.globl _kcc_lex_dump_all
.p2align 2
_kcc_lex_dump_all:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

.Ldump_loop:
    bl _kcc_lex_next

    // Check for EOF
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Ldump_done          // TOK_EOF = 0

    bl _kcc_lex_dump_token
    b .Ldump_loop

.Ldump_done:
    ldp x29, x30, [sp], #16
    ret
```

□

100a · used in 28d · defines _kcc_lex_dump_all 622 · uses _kcc_lex_dump_token 99, _kcc_lex_next 90b, _kcc_tok_type 104b

16.11. Token Type Names

A table mapping token type codes to human-readable names for the --dump-tokens output. kcc_lex_type_name returns a pointer to the name string for a given token type.

⟨ *Lexer: token names* 100b ⟩ ≡

```
.text
// -- kcc_lex_type_name ---
// const char *kcc_lex_type_name(uint64_t type)
//   x0 = token type code
// Returns pointer to a null-terminated type name
   string.
.globl _kcc_lex_type_name
.p2align 2
_kcc_lex_type_name:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Bounds check
    cmp x0, #108
    b.hs .Ltype_unknown_name

    // Load from pointer table
    adrp x1, _kcc_tok_names@PAGE
    add  x1, x1, _kcc_tok_names@PAGEOFF
    ldr  x0, [x1, x0, lsl #3]   // names[type]
    cbnz x0, .Ltype_name_done

.Ltype_unknown_name:
    adrp x0, _kcc_tn_unknown@PAGE
    add  x0, x0, _kcc_tn_unknown@PAGEOFF
.Ltype_name_done:
    ldp x29, x30, [sp], #16
    ret

// -- Token name strings ---
.section __TEXT,__cstring,cstring_literals
_kcc_tn_eof:          .asciz "eof"
_kcc_tn_identifier:   .asciz "identifier"
_kcc_tn_number:       .asciz "number"
_kcc_tn_char_lit:     .asciz "char_constant"
_kcc_tn_string_lit:   .asciz "string_literal"
_kcc_tn_unknown:      .asciz "unknown"
// Keywords
_kcc_tn_auto:         .asciz "auto"
_kcc_tn_break:        .asciz "break"
_kcc_tn_case:         .asciz "case"
_kcc_tn_char:         .asciz "char"
_kcc_tn_const:        .asciz "const"
_kcc_tn_continue:     .asciz "continue"
_kcc_tn_default:      .asciz "default"
_kcc_tn_do:           .asciz "do"
_kcc_tn_double:       .asciz "double"
_kcc_tn_else:         .asciz "else"
_kcc_tn_enum:         .asciz "enum"
_kcc_tn_extern:       .asciz "extern"
_kcc_tn_float:        .asciz "float"
_kcc_tn_for:          .asciz "for"
_kcc_tn_goto:         .asciz "goto"
_kcc_tn_if:           .asciz "if"
_kcc_tn_inline:       .asciz "inline"
_kcc_tn_int:          .asciz "int"
_kcc_tn_long:         .asciz "long"
_kcc_tn_register:     .asciz "register"
_kcc_tn_restrict:     .asciz "restrict"
_kcc_tn_return:       .asciz "return"
_kcc_tn_short:        .asciz "short"
_kcc_tn_signed:       .asciz "signed"
_kcc_tn_sizeof:       .asciz "sizeof"
_kcc_tn_static:       .asciz "static"
_kcc_tn_struct:       .asciz "struct"
_kcc_tn_switch:       .asciz "switch"
_kcc_tn_typedef:      .asciz "typedef"
_kcc_tn_union:        .asciz "union"
_kcc_tn_unsigned:     .asciz "unsigned"
_kcc_tn_void:         .asciz "void"
_kcc_tn_volatile:     .asciz "volatile"
_kcc_tn_while:        .asciz "while"
_kcc_tn_alignas:      .asciz "_Alignas"
_kcc_tn_alignof:      .asciz "_Alignof"
_kcc_tn_atomic:       .asciz "_Atomic"
_kcc_tn_bool:         .asciz "_Bool"
_kcc_tn_complex:      .asciz "_Complex"
_kcc_tn_generic:      .asciz "_Generic"
_kcc_tn_imaginary:    .asciz "_Imaginary"
_kcc_tn_noreturn:     .asciz "_Noreturn"
_kcc_tn_static_assert: .asciz "_Static_assert"
_kcc_tn_thread_local: .asciz "_Thread_local"
_kcc_tn_int128:       .asciz "__int128"
```

□

100b · defined at 100b, 101ab · used in 28e · defines _kcc_lex_type_name 99, 175ab, _kcc_tn_alignas 101b, _kcc_tn_alignof 101b, _kcc_tn_atomic 101b, _kcc_tn_auto 101b, _kcc_tn_bool 101b, _kcc_tn_break 101b, _kcc_tn_case 101b, _kcc_tn_char 101b, _kcc_tn_char_lit 101b, _kcc_tn_complex 101b, _kcc_tn_const 101b, _kcc_tn_continue 101b, _kcc_tn_default 101b, _kcc_tn_do 101b, _kcc_tn_double 101b, _kcc_tn_else 101b, _kcc_tn_enum 101b, _kcc_tn_eof 101b, _kcc_tn_extern 101b, _kcc_tn_float 101b, _kcc_tn_for 101b, _kcc_tn_generic 101b, _kcc_tn_goto 101b, _kcc_tn_identifier 101b, _kcc_tn_if 101b, _kcc_tn_imaginary 101b, _kcc_tn_inline 101b,

_kcc_tn_int 101b, _kcc_tn_int128 101b, _kcc_tn_long 101b, _kcc_tn_noreturn 101b, _kcc_tn_number 101b, _kcc_tn_register 101b, _kcc_tn_restrict 101b, _kcc_tn_return 101b, _kcc_tn_short 101b, _kcc_tn_signed 101b, _kcc_tn_sizeof 101b, _kcc_tn_static 101b, _kcc_tn_static_assert 101b, _kcc_tn_string_lit 101b, _kcc_tn_struct 101b, _kcc_tn_switch 101b, _kcc_tn_thread_local 101b, _kcc_tn_typedef 101b, _kcc_tn_union 101b, _kcc_tn_unknown 101b, _kcc_tn_unsigned 101b, _kcc_tn_void 101b, _kcc_tn_volatile 101b, _kcc_tn_while 101b

Punctuator name strings. Each punctuator token type has a corresponding human-readable name used in the `--dump-tokens` output. The names follow the convention used by Clang's token dump format: `l_paren`, `semi`, `plusplus`, `lessequal`, and so on.

⟨ *Lexer: token names* 101a ⟩ + ≡

```
// Punctuators
_kcc_tn_lparen:       .asciz "l_paren"
_kcc_tn_rparen:       .asciz "r_paren"
_kcc_tn_lbrace:       .asciz "l_brace"
_kcc_tn_rbrace:       .asciz "r_brace"
_kcc_tn_lbracket:     .asciz "l_bracket"
_kcc_tn_rbracket:     .asciz "r_bracket"
_kcc_tn_semi:         .asciz "semi"
_kcc_tn_comma:        .asciz "comma"
_kcc_tn_dot:          .asciz "dot"
_kcc_tn_arrow:        .asciz "arrow"
_kcc_tn_plusplus:     .asciz "plusplus"
_kcc_tn_minusminus:   .asciz "minusminus"
_kcc_tn_amp:          .asciz "amp"
_kcc_tn_star:         .asciz "star"
_kcc_tn_plus:         .asciz "plus"
_kcc_tn_minus:        .asciz "minus"
_kcc_tn_tilde:        .asciz "tilde"
_kcc_tn_exclaim:      .asciz "exclaim"
_kcc_tn_slash:        .asciz "slash"
_kcc_tn_percent:      .asciz "percent"
_kcc_tn_lessless:     .asciz "lessless"
_kcc_tn_greatergreater: .asciz "greatergreater"
_kcc_tn_less:         .asciz "less"
_kcc_tn_greater:      .asciz "greater"
_kcc_tn_lessequal:    .asciz "lessequal"
_kcc_tn_greaterequal: .asciz "greaterequal"
_kcc_tn_equalequal:   .asciz "equalequal"
_kcc_tn_exclaimequal: .asciz "exclaimequal"
_kcc_tn_caret:        .asciz "caret"
_kcc_tn_pipe:         .asciz "pipe"
_kcc_tn_ampamp:       .asciz "ampamp"
_kcc_tn_pipepipe:     .asciz "pipepipe"
_kcc_tn_question:     .asciz "question"
_kcc_tn_colon:        .asciz "colon"
_kcc_tn_equal:        .asciz "equal"
_kcc_tn_starequal:    .asciz "starequal"
_kcc_tn_slashequal:   .asciz "slashequal"
_kcc_tn_percentequal: .asciz "percentequal"
_kcc_tn_plusequal:    .asciz "plusequal"
_kcc_tn_minusequal:   .asciz "minusequal"
_kcc_tn_lesslessequal: .asciz "lesslessequal"
_kcc_tn_greatergreaterequal: .asciz "
  greatergreaterequal"
_kcc_tn_ampequal:     .asciz "ampequal"
_kcc_tn_caretequal:   .asciz "caretequal"
_kcc_tn_pipeequal:    .asciz "pipeequal"
_kcc_tn_hash:         .asciz "hash"
_kcc_tn_hashhash:     .asciz "hashhash"
_kcc_tn_ellipsis:     .asciz "ellipsis"
```

□

101a · defined at 100b, 101ab · used in 28e · defines _kcc_tn_amp 101b, _kcc_tn_ampamp 101b, _kcc_tn_ampequal 101b, _kcc_tn_arrow 101b, _kcc_tn_caret 101b, _kcc_tn_caretequal 101b, _kcc_tn_colon 101b, _kcc_tn_comma 101b, _kcc_tn_dot 101b, _kcc_tn_ellipsis 101b, _kcc_tn_equal 101b, _kcc_tn_equalequal 101b, _kcc_tn_exclaim 101b, _kcc_tn_exclaimequal 101b, _kcc_tn_greater 101b, _kcc_tn_greaterequal 101b, _kcc_tn_greatergreater 101b, _kcc_tn_greatergreaterequal 101b, _kcc_tn_hash 101b, _kcc_tn_hashhash 101b, _kcc_tn_lbrace 101b, _kcc_tn_lbracket 101b, _kcc_tn_less 101b, _kcc_tn_lessequal 101b, _kcc_tn_lessless 101b, _kcc_tn_lesslessequal 101b, _kcc_tn_lparen 101b, _kcc_tn_minus 101b, _kcc_tn_minusequal 101b, _kcc_tn_minusminus 101b, _kcc_tn_percent 101b, _kcc_tn_percentequal 101b, _kcc_tn_pipe 101b, _kcc_tn_pipeequal 101b, _kcc_tn_pipepipe 101b, _kcc_tn_plus 101b, _kcc_tn_plusequal 101b, _kcc_tn_plusplus 101b, _kcc_tn_question 101b, _kcc_tn_rbrace 101b, _kcc_tn_rbracket 101b, _kcc_tn_rparen 101b, _kcc_tn_semi 101b, _kcc_tn_slash 101b, _kcc_tn_slashequal 101b, _kcc_tn_star 101b, _kcc_tn_starequal 101b, _kcc_tn_tilde 101b

Token name pointer table. A flat array of 108 pointers, indexed by token type code, enables constant-time lookup in `kcc_lex_type_name`. Gaps in the code space (6–9 for future structural tokens, 54–59 reserved between keywords and punctuators) are filled with null entries.

⟨ *Lexer: token names* 101b ⟩ + ≡

```
// -- Token name pointer table ---
// Indexed by token type code.  Entries 6-9, 54-59
    are NULL (unused).
.data
.p2align 3
_kcc_tok_names:
    .quad _kcc_tn_eof              // 0  TOK_EOF
    .quad _kcc_tn_identifier       // 1  TOK_IDENT
    .quad _kcc_tn_number           // 2  TOK_NUMBER
    .quad _kcc_tn_char_lit         // 3
  TOK_CHAR_LIT
    .quad _kcc_tn_string_lit       // 4
  TOK_STRING_LIT
    .quad _kcc_tn_unknown          // 5
  TOK_UNKNOWN
    .quad 0, 0, 0, 0              // 6-9  reserved
    // Keywords 10-53
    .quad _kcc_tn_auto             // 10
    .quad _kcc_tn_break            // 11
    .quad _kcc_tn_case             // 12
    .quad _kcc_tn_char             // 13
    .quad _kcc_tn_const            // 14
    .quad _kcc_tn_continue         // 15
    .quad _kcc_tn_default          // 16
    .quad _kcc_tn_do               // 17
    .quad _kcc_tn_double           // 18
    .quad _kcc_tn_else             // 19
    .quad _kcc_tn_enum             // 20
    .quad _kcc_tn_extern           // 21
    .quad _kcc_tn_float            // 22
    .quad _kcc_tn_for              // 23
    .quad _kcc_tn_goto             // 24
    .quad _kcc_tn_if               // 25
```

```
    .quad _kcc_tn_inline          // 26
    .quad _kcc_tn_int             // 27
    .quad _kcc_tn_long            // 28
    .quad _kcc_tn_register        // 29
    .quad _kcc_tn_restrict        // 30
    .quad _kcc_tn_return          // 31
    .quad _kcc_tn_short           // 32
    .quad _kcc_tn_signed          // 33
    .quad _kcc_tn_sizeof          // 34
    .quad _kcc_tn_static          // 35
    .quad _kcc_tn_struct          // 36
    .quad _kcc_tn_switch          // 37
    .quad _kcc_tn_typedef         // 38
    .quad _kcc_tn_union           // 39
    .quad _kcc_tn_unsigned        // 40
    .quad _kcc_tn_void            // 41
    .quad _kcc_tn_volatile        // 42
    .quad _kcc_tn_while           // 43
    .quad _kcc_tn_alignas         // 44
    .quad _kcc_tn_alignof         // 45
    .quad _kcc_tn_atomic          // 46
    .quad _kcc_tn_bool            // 47
    .quad _kcc_tn_complex         // 48
    .quad _kcc_tn_generic         // 49
    .quad _kcc_tn_imaginary       // 50
    .quad _kcc_tn_noreturn        // 51
    .quad _kcc_tn_static_assert   // 52
    .quad _kcc_tn_thread_local    // 53
    .quad 0, 0, 0, 0              // 54-57 reserved
    .quad _kcc_tn_int128          // 58  __int128
    .quad 0                       // 59  reserved
    // Punctuators 60-107
    .quad _kcc_tn_lparen          // 60
    .quad _kcc_tn_rparen          // 61
    .quad _kcc_tn_lbrace          // 62
    .quad _kcc_tn_rbrace          // 63
    .quad _kcc_tn_lbracket        // 64
    .quad _kcc_tn_rbracket        // 65
    .quad _kcc_tn_semi            // 66
    .quad _kcc_tn_comma           // 67
    .quad _kcc_tn_dot             // 68
    .quad _kcc_tn_arrow           // 69
    .quad _kcc_tn_plusplus        // 70
    .quad _kcc_tn_minusminus      // 71
    .quad _kcc_tn_amp             // 72
    .quad _kcc_tn_star            // 73
    .quad _kcc_tn_plus            // 74
    .quad _kcc_tn_minus           // 75
    .quad _kcc_tn_tilde           // 76
    .quad _kcc_tn_exclaim         // 77
    .quad _kcc_tn_slash           // 78
    .quad _kcc_tn_percent         // 79
    .quad _kcc_tn_lessless        // 80
    .quad _kcc_tn_greatergreater  // 81
    .quad _kcc_tn_less            // 82
    .quad _kcc_tn_greater         // 83
    .quad _kcc_tn_lessequal       // 84
    .quad _kcc_tn_greaterequal    // 85
    .quad _kcc_tn_equalequal      // 86
    .quad _kcc_tn_exclaimequal    // 87
    .quad _kcc_tn_caret           // 88
    .quad _kcc_tn_pipe            // 89
    .quad _kcc_tn_ampamp          // 90
    .quad _kcc_tn_pipepipe        // 91
    .quad _kcc_tn_question        // 92
    .quad _kcc_tn_colon           // 93
    .quad _kcc_tn_equal           // 94
    .quad _kcc_tn_starequal       // 95
    .quad _kcc_tn_slashequal      // 96
    .quad _kcc_tn_percentequal    // 97
    .quad _kcc_tn_plusequal       // 98
    .quad _kcc_tn_minusequal      // 99
    .quad _kcc_tn_lesslessequal   // 100
    .quad _kcc_tn_greatergreaterequal // 101
    .quad _kcc_tn_ampequal        // 102
    .quad _kcc_tn_caretequal      // 103
    .quad _kcc_tn_pipeequal       // 104
    .quad _kcc_tn_hash            // 105
    .quad _kcc_tn_hashhash        // 106
    .quad _kcc_tn_ellipsis        // 107
```

□

101b · defined at 100b, 101ab · used in 28e ·
uses _kcc_tn_alignas 100b, _kcc_tn_alignof 100b,
_kcc_tn_amp 101a, _kcc_tn_ampamp 101a,
_kcc_tn_ampequal 101a, _kcc_tn_arrow 101a,
_kcc_tn_atomic 100b, _kcc_tn_auto 100b, _kcc_tn_bool 100b,
_kcc_tn_break 100b, _kcc_tn_caret 101a,
_kcc_tn_caretequal 101a, _kcc_tn_case 100b,
_kcc_tn_char 100b, _kcc_tn_char_lit 100b,
_kcc_tn_colon 101a, _kcc_tn_comma 101a,
_kcc_tn_complex 100b, _kcc_tn_const 100b,
_kcc_tn_continue 100b, _kcc_tn_default 100b,
_kcc_tn_do 100b, _kcc_tn_dot 101a, _kcc_tn_double 100b,
_kcc_tn_ellipsis 101a, _kcc_tn_else 100b,
_kcc_tn_enum 100b, _kcc_tn_eof 100b, _kcc_tn_equal 101a,
_kcc_tn_equalequal 101a, _kcc_tn_exclaim 101a,
_kcc_tn_exclaimequal 101a, _kcc_tn_extern 100b,
_kcc_tn_float 100b, _kcc_tn_for 100b, _kcc_tn_generic 100b,
_kcc_tn_goto 100b, _kcc_tn_greater 101a,
_kcc_tn_greaterequal 101a, _kcc_tn_greatergreater 101a,
_kcc_tn_greatergreaterequal 101a, _kcc_tn_hash 101a,
_kcc_tn_hashhash 101a, _kcc_tn_identifier 100b,
_kcc_tn_if 100b, _kcc_tn_imaginary 100b,
_kcc_tn_inline 100b, _kcc_tn_int 100b, _kcc_tn_int128 100b,
_kcc_tn_lbrace 101a, _kcc_tn_lbracket 101a,
_kcc_tn_less 101a, _kcc_tn_lessequal 101a,
_kcc_tn_lessless 101a, _kcc_tn_lesslessequal 101a,
_kcc_tn_long 100b, _kcc_tn_lparen 101a, _kcc_tn_minus 101a,
_kcc_tn_minusequal 101a, _kcc_tn_minusminus 101a,
_kcc_tn_noreturn 100b, _kcc_tn_number 100b,
_kcc_tn_percent 101a, _kcc_tn_percentequal 101a,
_kcc_tn_pipe 101a, _kcc_tn_pipeequal 101a,
_kcc_tn_pipepipe 101a, _kcc_tn_plus 101a,
_kcc_tn_plusequal 101a, _kcc_tn_plusplus 101a,
_kcc_tn_question 101a, _kcc_tn_rbrace 101a,
_kcc_tn_rbracket 101a, _kcc_tn_register 100b,
_kcc_tn_restrict 100b, _kcc_tn_return 100b,
_kcc_tn_rparen 101a, _kcc_tn_semi 101a, _kcc_tn_short 100b,
_kcc_tn_signed 100b, _kcc_tn_sizeof 100b,
_kcc_tn_slash 101a, _kcc_tn_slashequal 101a,
_kcc_tn_star 101a, _kcc_tn_starequal 101a,
_kcc_tn_static 100b, _kcc_tn_static_assert 100b,
_kcc_tn_string_lit 100b, _kcc_tn_struct 100b,
_kcc_tn_switch 100b, _kcc_tn_thread_local 100b,
_kcc_tn_tilde 101a, _kcc_tn_typedef 100b,
_kcc_tn_union 100b, _kcc_tn_unknown 100b,
_kcc_tn_unsigned 100b, _kcc_tn_void 100b,
_kcc_tn_volatile 100b, _kcc_tn_while 100b

16.12. Keyword Table Initialization

At startup, we populate a hash table with all 44 C17 keywords. Each entry maps the keyword text to its token type code.

⟨ *Lexer: keyword data* 103 ⟩ ≡

```
// -- Keyword strings ---
.section __TEXT,__cstring,cstring_literals
_kcc_kw_s_auto:       .asciz "auto"
_kcc_kw_s_break:      .asciz "break"
_kcc_kw_s_case:       .asciz "case"
_kcc_kw_s_char:       .asciz "char"
_kcc_kw_s_const:      .asciz "const"
_kcc_kw_s_continue:   .asciz "continue"
_kcc_kw_s_default:    .asciz "default"
_kcc_kw_s_do:         .asciz "do"
_kcc_kw_s_double:     .asciz "double"
_kcc_kw_s_else:       .asciz "else"
_kcc_kw_s_enum:       .asciz "enum"
_kcc_kw_s_extern:     .asciz "extern"
_kcc_kw_s_float:      .asciz "float"
_kcc_kw_s_for:        .asciz "for"
_kcc_kw_s_goto:       .asciz "goto"
_kcc_kw_s_if:         .asciz "if"
_kcc_kw_s_inline:     .asciz "inline"
_kcc_kw_s_int:        .asciz "int"
_kcc_kw_s_long:       .asciz "long"
_kcc_kw_s_register:   .asciz "register"
_kcc_kw_s_restrict:   .asciz "restrict"
_kcc_kw_s_return:     .asciz "return"
_kcc_kw_s_short:      .asciz "short"
_kcc_kw_s_signed:     .asciz "signed"
_kcc_kw_s_sizeof:     .asciz "sizeof"
_kcc_kw_s_static:     .asciz "static"
_kcc_kw_s_struct:     .asciz "struct"
_kcc_kw_s_switch:     .asciz "switch"
_kcc_kw_s_typedef:    .asciz "typedef"
_kcc_kw_s_union:      .asciz "union"
_kcc_kw_s_unsigned:   .asciz "unsigned"
_kcc_kw_s_void:       .asciz "void"
_kcc_kw_s_volatile:   .asciz "volatile"
_kcc_kw_s_while:      .asciz "while"
_kcc_kw_s_alignas:    .asciz "_Alignas"
_kcc_kw_s_alignof:    .asciz "_Alignof"
_kcc_kw_s_atomic:     .asciz "_Atomic"
_kcc_kw_s_bool:       .asciz "_Bool"
_kcc_kw_s_complex:    .asciz "_Complex"
_kcc_kw_s_generic:    .asciz "_Generic"
_kcc_kw_s_imaginary:  .asciz "_Imaginary"
_kcc_kw_s_noreturn:   .asciz "_Noreturn"
_kcc_kw_s_sassert:    .asciz "_Static_assert"
_kcc_kw_s_tlocal:     .asciz "_Thread_local"
// GNU/common extension spellings.  The `__x__`/`__x` forms alias standard
// keywords (same token code); the rest get their own codes above.
_kcc_kw_s_attr1:      .asciz "__attribute__"
_kcc_kw_s_attr2:      .asciz "__attribute"
_kcc_kw_s_typeof1:    .asciz "typeof"
_kcc_kw_s_typeof2:    .asciz "__typeof__"
_kcc_kw_s_typeof3:    .asciz "__typeof"
_kcc_kw_s_asm1:       .asciz "asm"
_kcc_kw_s_asm2:       .asciz "__asm__"
_kcc_kw_s_asm3:       .asciz "__asm"
_kcc_kw_s_ext:        .asciz "__extension__"
_kcc_kw_s_inline1:    .asciz "__inline"
_kcc_kw_s_inline2:    .asciz "__inline__"
_kcc_kw_s_const1:     .asciz "__const"
_kcc_kw_s_const2:     .asciz "__const__"
_kcc_kw_s_signed1:    .asciz "__signed"
_kcc_kw_s_signed2:    .asciz "__signed__"
_kcc_kw_s_vol1:       .asciz "__volatile"
_kcc_kw_s_vol2:       .asciz "__volatile__"
_kcc_kw_s_restr1:     .asciz "__restrict"
_kcc_kw_s_restr2:     .asciz "__restrict__"
_kcc_kw_s_int128:     .asciz "__int128"
_kcc_kw_s_alignof1:   .asciz "__alignof__"
_kcc_kw_s_alignof2:   .asciz "__alignof"

// -- Keyword registration table ---
// Each entry: (string_ptr, string_len, token_type) = 24 bytes
.data
.p2align 3
_kcc_kw_reg_table:
    .quad _kcc_kw_s_auto,       4,  10
    .quad _kcc_kw_s_break,      5,  11
    .quad _kcc_kw_s_case,       4,  12
    .quad _kcc_kw_s_char,       4,  13
    .quad _kcc_kw_s_const,      5,  14
    .quad _kcc_kw_s_continue,   8,  15
    .quad _kcc_kw_s_default,    7,  16
    .quad _kcc_kw_s_do,         2,  17
    .quad _kcc_kw_s_double,     6,  18
    .quad _kcc_kw_s_else,       4,  19
    .quad _kcc_kw_s_enum,       4,  20
    .quad _kcc_kw_s_extern,     6,  21
    .quad _kcc_kw_s_float,      5,  22
    .quad _kcc_kw_s_for,        3,  23
    .quad _kcc_kw_s_goto,       4,  24
    .quad _kcc_kw_s_if,         2,  25
    .quad _kcc_kw_s_inline,     6,  26
    .quad _kcc_kw_s_int,        3,  27
    .quad _kcc_kw_s_long,       4,  28
    .quad _kcc_kw_s_register,   8,  29
    .quad _kcc_kw_s_restrict,   8,  30
    .quad _kcc_kw_s_return,     6,  31
    .quad _kcc_kw_s_short,      5,  32
    .quad _kcc_kw_s_signed,     6,  33
    .quad _kcc_kw_s_sizeof,     6,  34
    .quad _kcc_kw_s_static,     6,  35
    .quad _kcc_kw_s_struct,     6,  36
    .quad _kcc_kw_s_switch,     6,  37
    .quad _kcc_kw_s_typedef,    7,  38
    .quad _kcc_kw_s_union,      5,  39
    .quad _kcc_kw_s_unsigned,   8,  40
    .quad _kcc_kw_s_void,       4,  41
    .quad _kcc_kw_s_volatile,   8,  42
    .quad _kcc_kw_s_while,      5,  43
    .quad _kcc_kw_s_alignas,    8,  44
    .quad _kcc_kw_s_alignof,    8,  45
    .quad _kcc_kw_s_atomic,     7,  46
    .quad _kcc_kw_s_bool,       5,  47
    .quad _kcc_kw_s_complex,    8,  48
    .quad _kcc_kw_s_generic,    8,  49
    .quad _kcc_kw_s_imaginary,  10, 50
    .quad _kcc_kw_s_noreturn,   9,  51
    .quad _kcc_kw_s_sassert,    14, 52
    .quad _kcc_kw_s_tlocal,     13, 53
    .quad _kcc_kw_s_attr1,      13, 54    // __attribute__ → TOK_ATTRIBUTE
    .quad _kcc_kw_s_attr2,      11, 54    // __attribute
    .quad _kcc_kw_s_typeof1,     6, 55    // typeof → TOK_TYPEOF
    .quad _kcc_kw_s_typeof2,    10, 55    // __typeof__
    .quad _kcc_kw_s_typeof3,     8, 55    // __typeof
    .quad _kcc_kw_s_asm1,        3, 56    // asm → TOK_ASM
    .quad _kcc_kw_s_asm2,        7, 56    // __asm__
    .quad _kcc_kw_s_asm3,        5, 56    // __asm
    .quad _kcc_kw_s_ext,        13, 57    // __extension__ → TOK_EXTENSION
    .quad _kcc_kw_s_inline1,     8, 26    // __inline → TOK_INLINE
    .quad _kcc_kw_s_inline2,    10, 26    // __inline__
```

```
    .quad _kcc_kw_s_const1,      7, 14    //
  __const → TOK_CONST
    .quad _kcc_kw_s_const2,      9, 14    //
  __const__
    .quad _kcc_kw_s_signed1,     8, 33    //
  __signed → TOK_SIGNED
    .quad _kcc_kw_s_signed2,    10, 33    //
  __signed__
    .quad _kcc_kw_s_vol1,       10, 42    //
  __volatile → TOK_VOLATILE
    .quad _kcc_kw_s_vol2,       12, 42    //
  __volatile__
    .quad _kcc_kw_s_restr1,     10, 30    //
  __restrict → TOK_RESTRICT
    .quad _kcc_kw_s_restr2,     12, 30    //
  __restrict__
    .quad _kcc_kw_s_int128,      8, 58    //
  __int128 → TOK_INT128
    .quad _kcc_kw_s_alignof1,   11, 45    //
  __alignof__ → TOK__ALIGNOF
    .quad _kcc_kw_s_alignof2,    9, 45    //
  __alignof

.equ KW_COUNT, 66
.equ KW_ENTRY_BYTES, 24       // 3 quads per entry
```

□

103 · used in 28e · defines _kcc_kw_s_alignas never used, _kcc_kw_s_alignof never used, _kcc_kw_s_atomic never used, _kcc_kw_s_auto never used, _kcc_kw_s_bool never used, _kcc_kw_s_break never used, _kcc_kw_s_case never used, _kcc_kw_s_char never used, _kcc_kw_s_complex never used, _kcc_kw_s_const never used, _kcc_kw_s_continue never used, _kcc_kw_s_default never used, _kcc_kw_s_do never used, _kcc_kw_s_double never used, _kcc_kw_s_else never used, _kcc_kw_s_enum never used, _kcc_kw_s_extern never used, _kcc_kw_s_float never used, _kcc_kw_s_for never used, _kcc_kw_s_generic never used, _kcc_kw_s_goto never used, _kcc_kw_s_if never used, _kcc_kw_s_imaginary never used, _kcc_kw_s_inline never used, _kcc_kw_s_int never used, _kcc_kw_s_long never used, _kcc_kw_s_noreturn never used, _kcc_kw_s_register never used, _kcc_kw_s_restrict never used, _kcc_kw_s_return never used, _kcc_kw_s_sassert never used, _kcc_kw_s_short never used, _kcc_kw_s_signed never used, _kcc_kw_s_sizeof never used, _kcc_kw_s_static never used, _kcc_kw_s_struct never used, _kcc_kw_s_switch never used, _kcc_kw_s_tlocal never used, _kcc_kw_s_typedef never used, _kcc_kw_s_union never used, _kcc_kw_s_unsigned never used, _kcc_kw_s_void never used, _kcc_kw_s_volatile never used, _kcc_kw_s_while never used

The initialization function creates a hash table and populates it by looping through the registration table.

⟨ *Lexer: keyword init* 104a ⟩ ≡

```
.text
// -- kcc_lex_kw_init ---
// void kcc_lex_kw_init(void)
// Builds the keyword hash table from the
   registration table.
.globl _kcc_lex_kw_init
.p2align 2
_kcc_lex_kw_init:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Create hash table with capacity 128 (power
   of 2, >2x keywords)
    mov x0, #128
    bl _kcc_ht_init
    mov x19, x0                  // x19 = hash
   table

    // Store in global
    adrp x1, _kcc_kw_table@PAGE
    add  x1, x1, _kcc_kw_table@PAGEOFF
    str  x19, [x1]

    // Loop through registration table
    adrp x20, _kcc_kw_reg_table@PAGE
    add  x20, x20, _kcc_kw_reg_table@PAGEOFF
    mov  x21, #0                 // counter
    mov  x22, #KW_COUNT

.Lkw_init_loop:
    cmp x21, x22
    b.hs .Lkw_init_done

    // Load entry: [string_ptr, string_len,
   token_type]
    ldr x1, [x20, #0]            // key = string
   ptr
    ldr x2, [x20, #8]            // keylen = string
    length
    ldr x3, [x20, #16]           // value = token
   type

    mov x0, x19                  // ht
    bl _kcc_ht_put

    add x20, x20, #KW_ENTRY_BYTES
    add x21, x21, #1
    b .Lkw_init_loop

.Lkw_init_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

104a · used in 28e · defines _kcc_lex_kw_init 621, 629 · uses _kcc_ht_init 61c, _kcc_ht_put 63, _kcc_kw_table 104b

16.13. Lexer Global State

The lexer maintains its state in global variables: the cursor position, end of buffer, current line/column tracking, and the current token.

⟨ *Lexer: data* 104b ⟩ ≡

```
// -- Lexer global state ---
.data
.p2align 3
.globl _kcc_lex_cursor
_kcc_lex_cursor:     .quad 0  // current position
   in source buffer
.globl _kcc_lex_end
_kcc_lex_end:        .quad 0  // one past end of
   source buffer
.globl _kcc_lex_line
_kcc_lex_line:       .quad 1  // current line
   number (1-based)
.globl _kcc_lex_line_start
_kcc_lex_line_start: .quad 0  // pointer to start
   of current line
```

```
// Current token (exported for test harness access)
.globl _kcc_tok_type
_kcc_tok_type:        .quad 0  // token type code
.globl _kcc_tok_start
_kcc_tok_start:       .quad 0  // pointer to token text in source
.globl _kcc_tok_len
_kcc_tok_len:         .quad 0  // token text length
.globl _kcc_tok_line
_kcc_tok_line:        .quad 0  // line where token starts
.globl _kcc_tok_col
_kcc_tok_col:         .quad 0  // column where token starts

// Keyword hash table
.globl _kcc_kw_table
_kcc_kw_table:        .quad 0  // pointer to keyword hash table

// Error messages (bare text - diagnostic engine adds file:line:col prefix)
.section __TEXT,__cstring,cstring_literals
_kcc_err_unterm_string:  .asciz "unterminated string literal"
_kcc_err_unterm_char:    .asciz "unterminated character constant"
_kcc_err_unterm_comment: .asciz "unterminated block comment"
```

□

104b · used in 28d · defines _kcc_err_unterm_char 95, _kcc_err_unterm_comment 91a, _kcc_err_unterm_string 94b, _kcc_kw_table 93, 104a, _kcc_lex_cursor 90ab, 91b, 92, 93, 94ab, 95, 96, 107a, 111, 112, 113, 117, 123, 136, 139a, 147a, 151, _kcc_lex_end 90ab, 93, 94ab, 95, 96, 107a, 111, 112, 113, 117, 123, 136, 139a, 147a, 151, _kcc_lex_line 90ab, 91a, 92, 107a, 111, 112, 113, 116, 121, 136, _kcc_lex_line_start 90ab, 91a, 92, 107a, 111, 112, 113, _kcc_tok_col 92, 94b, 95, 99, 168a, 169, 188, _kcc_tok_len 91b, 93, 94ab, 95, 98, 99, 109, 117, 119ab, 121, 125b, 135, 136, 140, 145b, 146, 151, 165, 184b, 186a, 188, 191b, 198, 200b, 202, 203, 205a, 212, 214, 215, 226, 230, 234a, _kcc_tok_line 92, 94b, 95, 99, 108, 109, 111, 140, 168a, 169, 188, 254b, _kcc_tok_start 91b, 93, 94ab, 95, 98, 99, 109, 117, 119ab, 121, 123, 125b, 135, 136, 140, 145b, 146, 151, 165, 184b, 186a, 188, 191b, 198, 200b, 202, 203, 205a, 212, 214, 215, 226, 230, 234a, _kcc_tok_type 91b, 93, 94ab, 95, 98, 99, 100a, 108, 109, 115, 117, 119ab, 123, 125b, 135, 136, 140, 141, 142, 143, 144, 145ab, 146, 151, 165, 168b, 169, 172, 181, 182, 184ab, 186b, 190, 191b, 192, 195, 196, 198, 199a, 200b, 201, 202, 203, 207b, 208ab, 209ab, 210, 211, 212, 215, 218, 223a, 225, 226, 227, 228, 230, 231a, 233, 234bc, 630a

17. The Preprocessor

The C preprocessor is a text-transformation engine defined by §6.10. It processes directives (`#define`, `#include`, conditionals) and expands macros before the translation unit reaches the parser.

Our preprocessor wraps the Phase 3 lexer: it calls `kcc_lex_next` internally, processes any directive or macro it encounters, and presents the caller with a fully preprocessed token stream.

17.1. Data Structures

A *macro definition* records the macro's name, replacement body, and (for function-like macros) parameter list. Macros are stored in a hash table mapping name text to a pointer to the definition structure.

The *context stack* supports nested `#include` and macro expansion by saving and restoring the lexer's scanning state. The *conditional stack* tracks nested `#if`/`#endif` groups.

⟨ *PP: constants* 107a ⟩ ≡

```
// -- Preprocessor constants ---

// Macro definition layout (arena-allocated)
.equ PP_MACRO_NAME,      0    // pointer to macro
   name text
.equ PP_MACRO_NAMELEN,   8    // length of name
.equ PP_MACRO_FLAGS,     16   // bit flags (see
   below)
.equ PP_MACRO_NPARAMS,   24   // parameter count (
   func-like)
.equ PP_MACRO_BODY,      32   // pointer to
   replacement text
.equ PP_MACRO_BODYLEN,   40   // replacement text
   length
.equ PP_MACRO_PARAMS,    48   // pointer to param
   name array
.equ PP_MACRO_SIZE,      56

// Macro flags
.equ PP_FLAG_FUNCLIKE,   1    // function-like
   macro
.equ PP_FLAG_EXPANDING,  2    // currently being
   expanded (anti-recursion)
.equ PP_FLAG_BUILTIN,    4    // predefined, cannot
    #undef
.equ PP_FLAG_VARIADIC,   8    // function-like
   macro with a trailing ... (__VA_ARGS__)

// Parameter entry (16 bytes each)
.equ PP_PARAM_NAME,      0
.equ PP_PARAM_LEN,       8
.equ PP_PARAM_SIZE,      16

// Context stack entry (for #include and macro
   expansion)
.equ PP_CTX_TYPE,        0    // 0=include, 1=
   expand
.equ PP_CTX_CURSOR,      8    // saved
   _kcc_lex_cursor
.equ PP_CTX_END,         16   // saved _kcc_lex_end
.equ PP_CTX_LINE,        24   // saved
   _kcc_lex_line
.equ PP_CTX_LSTART,      32   // saved
   _kcc_lex_line_start
.equ PP_CTX_SRC_BASE,    40   // saved
   _kcc_src_base (include only)
.equ PP_CTX_SRC_SIZE,    48   // saved
   _kcc_src_size (include only)
.equ PP_CTX_SRC_PATH,    56   // saved
   _kcc_src_path (include only)
.equ PP_CTX_MACRO,       64   // macro_def ptr to
   unmark (expand only)
.equ PP_CTX_PREVLINE,    72   // saved prev_line
.equ PP_CTX_SIZE,        80

.equ PP_CTX_INCLUDE,     0
.equ PP_CTX_EXPAND,      1

// Limits
.equ PP_MAX_CTX_DEPTH, 256    // max include +
   expansion depth
.equ PP_MAX_COND,        64   // max conditional
   nesting
.equ PP_MAX_ARGS,        64   // max function-like
   macro args

// Conditional stack states
.equ PP_COND_ACTIVE,     0    // in the active
   branch
.equ PP_COND_SKIP,       1    // skipping (branch
   not taken)
.equ PP_COND_DONE,       2    // active branch
   already found
```

□

107a · used in 28f · uses _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_line 104b, _kcc_lex_line_start 104b, _kcc_src_base 82, _kcc_src_path 82, _kcc_src_size 82

17.2. Initialization

`kcc_pp_init` creates the macro hash table and registers the predefined macros required by §6.10.8.

⟨ *PP: init* 107b ⟩ ≡

```
.text
// -- kcc_pp_init ---
// void kcc_pp_init(void)
// Initializes the preprocessor: creates macro
   table, sets up
// predefined macros, clears all state.
.globl _kcc_pp_init
.p2align 2
_kcc_pp_init:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Create macro hash table (capacity 256)
    mov x0, #256
    bl _kcc_ht_init
    adrp x1, _kcc_pp_macros@PAGE
    add  x1, x1, _kcc_pp_macros@PAGEOFF
    str  x0, [x1]

    // Clear context depth
    adrp x0, _kcc_pp_ctx_depth@PAGE
```

```
    add  x0, x0, _kcc_pp_ctx_depth@PAGEOFF
    str  xzr, [x0]

    // Clear conditional depth
    adrp x0, _kcc_pp_cond_depth@PAGE
    add  x0, x0, _kcc_pp_cond_depth@PAGEOFF
    str  xzr, [x0]

    // Clear prev_line (0 means "beginning of file
   " → first # is BOL)
    adrp x0, _kcc_pp_prev_line@PAGE
    add  x0, x0, _kcc_pp_prev_line@PAGEOFF
    str  xzr, [x0]

    // Clear has_pending
    adrp x0, _kcc_pp_has_pending@PAGE
    add  x0, x0, _kcc_pp_has_pending@PAGEOFF
    str  xzr, [x0]

    // Register predefined macros
    bl _kcc_pp_predefine

    ldp x29, x30, [sp], #16
    ret
```

□

107b · used in 28f · defines _kcc_pp_init 603, 622, 623 · uses _kcc_ht_init 61c, _kcc_pp_cond_depth 116, _kcc_pp_ctx_depth 116, _kcc_pp_has_pending 116, _kcc_pp_macros 116, _kcc_pp_prev_line 116

17.3. The Preprocessor Loop

kcc_pp_next is the main entry point. It replaces kcc_lex_next in the compilation pipeline. Each call returns one preprocessed token in the _kcc_tok_* globals.

The loop handles three concerns:

1. **Conditional skip.** If we are inside a false #if/#ifdef branch, all tokens are discarded except directive-introducing # at the start of a line, which may toggle the conditional state.
2. **Directive processing.** A # token at the beginning of a line is a preprocessing directive. The dispatcher consumes the entire directive and loops.
3. **Macro expansion.** An identifier token is looked up in the macro table; if found, its expansion is pushed onto the context stack and the loop restarts to lex from the expansion.

"Beginning of line" is detected by comparing the token's line number with the line of the previous token returned to the caller.

⟨ *PP: next token* 108 ⟩ ≡

```
.text
// -- kcc_pp_next ---
// void kcc_pp_next(void)
// Returns next preprocessed token in _kcc_tok_*
   globals.
.globl _kcc_pp_next
.p2align 2
_kcc_pp_next:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

.Lpp_loop:
    // Check for pending token (set by directive/
   expansion overflow)
    adrp x0, _kcc_pp_has_pending@PAGE
    add  x0, x0, _kcc_pp_has_pending@PAGEOFF
    ldr  x1, [x0]
    cbz  x1, .Lpp_call_lex
    str  xzr, [x0]            // clear pending
   flag
    b .Lpp_got_token

.Lpp_call_lex:
    bl _kcc_lex_next

.Lpp_got_token:
    // Load token type and line
    adrp x19, _kcc_tok_type@PAGE
    add  x19, x19, _kcc_tok_type@PAGEOFF
    ldr  x20, [x19]           // x20 = tok_type

    adrp x0, _kcc_tok_line@PAGE
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x21, [x0]            // x21 = tok_line

    // --- Handle EOF ---
    cbz  x20, .Lpp_eof

    // --- Conditional skip ---
    adrp x0, _kcc_pp_cond_depth@PAGE
    add  x0, x0, _kcc_pp_cond_depth@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lpp_active      // no conditionals
   → always active

    // Check top of conditional stack
    sub  x1, x0, #1
    adrp x2, _kcc_pp_cond_stack@PAGE
    add  x2, x2, _kcc_pp_cond_stack@PAGEOFF
    ldrb w3, [x2, x1]
    cmp  w3, #PP_COND_ACTIVE
    b.eq .Lpp_active

    // --- Skipping: only look for # at BOL ---
    cmp  x20, #TOK_HASH
    b.ne .Lpp_skip_tok
    // Check BOL
    adrp x0, _kcc_pp_prev_line@PAGE
    add  x0, x0, _kcc_pp_prev_line@PAGEOFF
    ldr  x0, [x0]
    cmp  x21, x0
    b.ls .Lpp_skip_tok        // same or earlier
   line → not BOL
    // Directive while skipping - update prev_line
    adrp x0, _kcc_pp_prev_line@PAGE
    add  x0, x0, _kcc_pp_prev_line@PAGEOFF
    str  x21, [x0]
    bl _kcc_pp_skip_directive
    b .Lpp_loop

.Lpp_skip_tok:
    adrp x0, _kcc_pp_prev_line@PAGE
    add  x0, x0, _kcc_pp_prev_line@PAGEOFF
    str  x21, [x0]
    b .Lpp_loop

    // --- Active processing ---
.Lpp_active:
    // Check for # directive at BOL
    cmp  x20, #TOK_HASH
    b.ne .Lpp_not_directive
    adrp x0, _kcc_pp_prev_line@PAGE
    add  x0, x0, _kcc_pp_prev_line@PAGEOFF
    ldr  x0, [x0]
```

```
    cmp  x21, x0
    b.ls .Lpp_not_directive     // not BOL
    // Process directive (consumes the line)
    adrp x0, _kcc_pp_prev_line@PAGE
    add  x0, x0, _kcc_pp_prev_line@PAGEOFF
    str  x21, [x0]
    bl _kcc_pp_dispatch
    b .Lpp_loop

.Lpp_not_directive:
    // Check for macro expansion.  A macro name
   may have been #defined over a
    // keyword spelling (e.g. `#define __inline__`
    in system headers), so any
    // keyword-class token [10, TOK_LPAREN) is
   also a candidate; try_expand's
    // hash lookup harmlessly returns 0 for the
   overwhelming majority that are
    // not macros.
    cmp  x20, #TOK_IDENT
    b.eq .Lpp_maybe_expand
    sub  x0, x20, #10           // keyword-class
   tokens occupy [10, 60)
    cmp  x0, #50                // 60-10; hs → not
   expandable
    b.hs .Lpp_return
.Lpp_maybe_expand:
    bl _kcc_pp_try_expand
    cbnz x0, .Lpp_loop          // expanded → loop
   to get token from expansion

.Lpp_return:
    // Update prev_line and return
    adrp x0, _kcc_pp_prev_line@PAGE
    add  x0, x0, _kcc_pp_prev_line@PAGEOFF
    str  x21, [x0]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lpp_eof:
    // Pop context stack if non-empty
    adrp x0, _kcc_pp_ctx_depth@PAGE
    add  x0, x0, _kcc_pp_ctx_depth@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lpp_real_eof
    bl _kcc_pp_ctx_pop
    // Argument prescan boundary: if a drain floor
    is active and the stack has
    // dropped below it, the argument being
   expanded is exhausted---return this
    // TOK_EOF rather than reading on into the
   enclosing text (which would
    // over-read, and could even expand an
   enclosing macro).
    adrp x0, _kcc_pp_drain_floor@PAGE
    add  x0, x0, _kcc_pp_drain_floor@PAGEOFF
    ldr  x1, [x0]
    cbz  x1, .Lpp_loop
    adrp x0, _kcc_pp_ctx_depth@PAGE
    add  x0, x0, _kcc_pp_ctx_depth@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, x1
    b.lt .Lpp_return            // popped below the
   floor → stop at EOF
    b .Lpp_loop

.Lpp_real_eof:
    // Check for unterminated conditionals
    adrp x0, _kcc_pp_cond_depth@PAGE
    add  x0, x0, _kcc_pp_cond_depth@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lpp_eof_ok
    // Emit warning
    adrp x0, _kcc_tok_line@PAGE
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_unterm_cond@PAGE
    add  x2, x2, _kcc_pp_err_unterm_cond@PAGEOFF
    bl _kcc_diag_error
.Lpp_eof_ok:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

108 · used in 28f · defines _kcc_pp_next 115, 125b, 136, 165, 168b, 169, 172, 181, 182, 184ab, 186b, 188, 190, 191b, 192, 196, 198, 199a, 200b, 203, 205a, 207b, 208ab, 209ab, 210, 211, 212, 214, 215, 218, 223a, 225, 226, 227, 228, 230, 231ab, 232ab, 233, 234abc · uses _kcc_diag_error 76a, _kcc_lex_next 90b, _kcc_pp_cond_depth 116, _kcc_pp_cond_stack 116, _kcc_pp_ctx_depth 116, _kcc_pp_err_unterm_cond 116, _kcc_pp_has_pending 116, _kcc_pp_prev_line 116, _kcc_pp_try_expand 121, _kcc_tok_line 104b, _kcc_tok_type 104b

17.4. Directive Dispatch

When we encounter # at the beginning of a line, the next token is the directive name. We dispatch to the appropriate handler by comparing the name against the twelve C17 directives.

⟨ *PP: directive dispatch* 109 ⟩ ≡

```
.text
// -- _kcc_pp_dispatch ---
// Internal: dispatches a preprocessing directive.
// Called after '#' has been consumed. Reads
   directive name and routes.
.p2align 2
_kcc_pp_dispatch:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Save directive line number
    adrp x0, _kcc_tok_line@PAGE
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x19, [x0]
    adrp x0, _kcc_pp_dir_line@PAGE
    add  x0, x0, _kcc_pp_dir_line@PAGEOFF
    str  x19, [x0]

    // Read directive name
    bl _kcc_lex_next

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    // Directive name can be TOK_IDENT(1) or a
   keyword(10-53)
    // since "if" and "else" are C keywords the
   lexer recognizes
    cmp  x0, #TOK_IDENT
    b.eq .Ldir_has_name
    cmp  x0, #10                // TOK_AUTO (first
   keyword)
    b.lt .Ldir_null
```

```
    cmp  x0, #53                //
   TOK__THREAD_LOCAL (last keyword)
    b.gt .Ldir_null
.Ldir_has_name:

    // Load name ptr and len into callee-saved
   regs
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x20, [x0]              // x20 = name ptr
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr  x21, [x0]              // x21 = name len

    // Dispatch by length, then memcmp
    cmp x21, #2
    b.eq .Ldir_l2
    cmp x21, #4
    b.eq .Ldir_l4
    cmp x21, #5
    b.eq .Ldir_l5
    cmp x21, #6
    b.eq .Ldir_l6
    cmp x21, #7
    b.eq .Ldir_l7
    b .Ldir_unknown

.Ldir_l2:  // "if"
    mov x0, x20
    adrp x1, _kcc_pp_s_if@PAGE
    add  x1, x1, _kcc_pp_s_if@PAGEOFF
    mov x2, #2
    bl _kcc_memcmp
    cbz x0, .Ldir_if
    b .Ldir_unknown

.Ldir_l4:  // elif, else, line
    mov x0, x20
    adrp x1, _kcc_pp_s_elif@PAGE
    add  x1, x1, _kcc_pp_s_elif@PAGEOFF
    mov x2, #4
    bl _kcc_memcmp
    cbz x0, .Ldir_elif

    mov x0, x20
    adrp x1, _kcc_pp_s_else@PAGE
    add  x1, x1, _kcc_pp_s_else@PAGEOFF
    mov x2, #4
    bl _kcc_memcmp
    cbz x0, .Ldir_else

    mov x0, x20
    adrp x1, _kcc_pp_s_line@PAGE
    add  x1, x1, _kcc_pp_s_line@PAGEOFF
    mov x2, #4
    bl _kcc_memcmp
    cbz x0, .Ldir_line_ctrl
    b .Ldir_unknown

.Ldir_l5:  // undef, ifdef, endif, error
    mov x0, x20
    adrp x1, _kcc_pp_s_ifdef@PAGE
    add  x1, x1, _kcc_pp_s_ifdef@PAGEOFF
    mov x2, #5
    bl _kcc_memcmp
    cbz x0, .Ldir_ifdef

    mov x0, x20
    adrp x1, _kcc_pp_s_undef@PAGE
    add  x1, x1, _kcc_pp_s_undef@PAGEOFF
    mov x2, #5
    bl _kcc_memcmp
    cbz x0, .Ldir_undef

    mov x0, x20
    adrp x1, _kcc_pp_s_endif@PAGE
    add  x1, x1, _kcc_pp_s_endif@PAGEOFF
    mov x2, #5
    bl _kcc_memcmp
    cbz x0, .Ldir_endif

    mov x0, x20
    adrp x1, _kcc_pp_s_error@PAGE
    add  x1, x1, _kcc_pp_s_error@PAGEOFF
    mov x2, #5
    bl _kcc_memcmp
    cbz x0, .Ldir_error
    b .Ldir_unknown

.Ldir_l6:  // define, ifndef, pragma
    mov x0, x20
    adrp x1, _kcc_pp_s_define@PAGE
    add  x1, x1, _kcc_pp_s_define@PAGEOFF
    mov x2, #6
    bl _kcc_memcmp
    cbz x0, .Ldir_define

    mov x0, x20
    adrp x1, _kcc_pp_s_ifndef@PAGE
    add  x1, x1, _kcc_pp_s_ifndef@PAGEOFF
    mov x2, #6
    bl _kcc_memcmp
    cbz x0, .Ldir_ifndef

    mov x0, x20
    adrp x1, _kcc_pp_s_pragma@PAGE
    add  x1, x1, _kcc_pp_s_pragma@PAGEOFF
    mov x2, #6
    bl _kcc_memcmp
    cbz x0, .Ldir_pragma
    b .Ldir_unknown

.Ldir_l7:  // include
    mov x0, x20
    adrp x1, _kcc_pp_s_include@PAGE
    add  x1, x1, _kcc_pp_s_include@PAGEOFF
    mov x2, #7
    bl _kcc_memcmp
    cbz x0, .Ldir_include
    b .Ldir_unknown
```

□

109 · defined at 109, 110 · used in 28f · uses _kcc_lex_next 90b, _kcc_pp_dir_line 116, _kcc_pp_s_define 116, _kcc_pp_s_elif 116, _kcc_pp_s_else 116, _kcc_pp_s_endif 116, _kcc_pp_s_error 116, _kcc_pp_s_if 116, _kcc_pp_s_ifdef 116, _kcc_pp_s_ifndef 116, _kcc_pp_s_include 116, _kcc_pp_s_line 116, _kcc_pp_s_pragma 116, _kcc_pp_s_undef 116, _kcc_tok_len 104b, _kcc_tok_line 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

Directive handlers. Once the name has been identified by length and content, control falls through to a dispatch target that calls the appropriate handler function. Each handler is a standalone routine that consumes the rest of the directive line and returns to the dispatcher, which then restores callee-saved registers and returns to the preprocessor loop.

⟨ *PP: directive dispatch* 110 ⟩ +≡

```
// Dispatch targets
.Ldir_define:
    bl _kcc_pp_do_define
    b .Ldir_done
.Ldir_undef:
    bl _kcc_pp_do_undef
    b .Ldir_done
```

```
.Ldir_ifdef:
    mov x0, #0                  // negate=0 → ifdef
    bl _kcc_pp_do_ifdef
    b .Ldir_done
.Ldir_ifndef:
    mov x0, #1                  // negate=1 →
   ifndef
    bl _kcc_pp_do_ifdef
    b .Ldir_done
.Ldir_if:
    bl _kcc_pp_do_if
    b .Ldir_done
.Ldir_elif:
    bl _kcc_pp_do_elif
    b .Ldir_done
.Ldir_else:
    bl _kcc_pp_do_else
    b .Ldir_done
.Ldir_endif:
    bl _kcc_pp_do_endif
    b .Ldir_done
.Ldir_include:
    bl _kcc_pp_do_include
    b .Ldir_done
.Ldir_error:
    bl _kcc_pp_do_error
    b .Ldir_done
.Ldir_line_ctrl:
    bl _kcc_pp_do_line          // #line N: adjust
   the reported line number
    b .Ldir_done
.Ldir_pragma:
    bl _kcc_pp_do_pragma        // push_macro/
   pop_macro; other pragmas ignored
    b .Ldir_done
.Ldir_null:
    bl _kcc_pp_skip_line
    b .Ldir_done
.Ldir_unknown:
    mov x0, x19
    mov x1, #1
    adrp x2, _kcc_pp_err_unknown_dir@PAGE
    add  x2, x2, _kcc_pp_err_unknown_dir@PAGEOFF
    bl _kcc_diag_warning
    bl _kcc_pp_skip_line

.Ldir_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

110 · defined at 109, 110 · used in 28f · uses _kcc_diag_warning 76b, _kcc_pp_do_pragma 119b, _kcc_pp_err_unknown_dir 116

17.5. Context Stack

The context stack saves and restores lexer state for macro expansion and #include. Push saves the current state and points the lexer at a new buffer. Pop restores the saved state and (for expansion) clears the anti-recursion flag on the expanded macro.

⟨ *PP: context* 111 ⟩ ≡

```
.text
// -- _kcc_pp_ctx_push ---
// Push include context. x0 = type (PP_CTX_INCLUDE
   )
.p2align 2
_kcc_pp_ctx_push:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x1, _kcc_pp_ctx_depth@PAGE
    add  x1, x1, _kcc_pp_ctx_depth@PAGEOFF
    ldr  x2, [x1]
    cmp  x2, #PP_MAX_CTX_DEPTH
    b.hs .Lctx_overflow

    // Calculate stack entry address
    mov  x3, #PP_CTX_SIZE
    mul  x4, x2, x3
    adrp x5, _kcc_pp_ctx_stack@PAGE
    add  x5, x5, _kcc_pp_ctx_stack@PAGEOFF
    add  x5, x5, x4             // x5 = entry ptr

    // Save type
    str  x0, [x5, #PP_CTX_TYPE]

    // Save lexer state
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x5, #PP_CTX_CURSOR]

    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x5, #PP_CTX_END]

    adrp x0, _kcc_lex_line@PAGE
    add  x0, x0, _kcc_lex_line@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x5, #PP_CTX_LINE]

    adrp x0, _kcc_lex_line_start@PAGE
    add  x0, x0, _kcc_lex_line_start@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x5, #PP_CTX_LSTART]

    // Save source state (for include)
    adrp x0, _kcc_src_base@PAGE
    add  x0, x0, _kcc_src_base@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x5, #PP_CTX_SRC_BASE]

    adrp x0, _kcc_src_size@PAGE
    add  x0, x0, _kcc_src_size@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x5, #PP_CTX_SRC_SIZE]

    adrp x0, _kcc_src_path@PAGE
    add  x0, x0, _kcc_src_path@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x5, #PP_CTX_SRC_PATH]

    // Save prev_line
    adrp x0, _kcc_pp_prev_line@PAGE
    add  x0, x0, _kcc_pp_prev_line@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x5, #PP_CTX_PREVLINE]

    // Clear macro pointer
    str  xzr, [x5, #PP_CTX_MACRO]

    // Increment depth
    add  x2, x2, #1
    str  x2, [x1]

    ldp x29, x30, [sp], #16
    ret

.Lctx_overflow:
    adrp x0, _kcc_tok_line@PAGE
```

```
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_depth@PAGE
    add  x2, x2, _kcc_pp_err_depth@PAGEOFF
    bl _kcc_diag_error
    mov  x0, #1
    bl _kcc_sys_exit

// -- _kcc_pp_ctx_push_expand ---
// Push expansion context.
// x0=type(PP_CTX_EXPAND), x1=macro_def, x2=buffer
   , x3=buflen
.p2align 2
_kcc_pp_ctx_push_expand:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x1                 // macro_def
    mov x20, x2                 // buffer
    mov x21, x3                 // buflen

    // Push the base context (saves lexer state)
    mov x0, #PP_CTX_EXPAND
    bl _kcc_pp_ctx_push

    // Now patch the entry with macro pointer
    adrp x0, _kcc_pp_ctx_depth@PAGE
    add  x0, x0, _kcc_pp_ctx_depth@PAGEOFF
    ldr  x0, [x0]
    sub  x0, x0, #1             // just-pushed entry
    mov  x1, #PP_CTX_SIZE
    mul  x0, x0, x1
    adrp x1, _kcc_pp_ctx_stack@PAGE
    add  x1, x1, _kcc_pp_ctx_stack@PAGEOFF
    add  x1, x1, x0
    str  x19, [x1, #PP_CTX_MACRO]

    // Point lexer at expansion buffer
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x20, [x0]

    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    add  x1, x20, x21
    str  x1, [x0]

    // Set line to current token line (for error
   reporting)
    // line_start = buffer start
    adrp x0, _kcc_lex_line_start@PAGE
    add  x0, x0, _kcc_lex_line_start@PAGEOFF
    str  x20, [x0]

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

111 · defined at 111, 112 · used in 28f · uses _kcc_diag_error 76a, _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_line 104b, _kcc_lex_line_start 104b, _kcc_pp_ctx_depth 116, _kcc_pp_ctx_stack 116, _kcc_pp_err_depth 116, _kcc_pp_prev_line 116, _kcc_src_base 82, _kcc_src_path 82, _kcc_src_size 82, _kcc_tok_line 104b

When an included file or macro expansion reaches TOK_EOF, the preprocessor loop calls _kcc_pp_ctx_pop to restore the previous lexer state. For expansion contexts, the macro's anti-recursion flag (PP_FLAG_EXPANDING) is cleared so that later occurrences of the same macro name can be expanded again. For include contexts, the saved source buffer and path are restored in addition to the lexer cursor.

⟨ *PP: context* 112 ⟩ +≡

```
// -- _kcc_pp_ctx_pop ---
// Pop context stack. Restores lexer state.
   Unmarks macro if expand.
.p2align 2
_kcc_pp_ctx_pop:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _kcc_pp_ctx_depth@PAGE
    add  x0, x0, _kcc_pp_ctx_depth@PAGEOFF
    ldr  x1, [x0]
    cbz  x1, .Lctx_pop_done    // empty stack (
   shouldn't happen)
    sub  x1, x1, #1
    str  x1, [x0]

    // Get entry
    mov  x2, #PP_CTX_SIZE
    mul  x3, x1, x2
    adrp x4, _kcc_pp_ctx_stack@PAGE
    add  x4, x4, _kcc_pp_ctx_stack@PAGEOFF
    add  x4, x4, x3            // x4 = entry ptr

    // Check type
    ldr  x0, [x4, #PP_CTX_TYPE]
    cmp  x0, #PP_CTX_EXPAND
    b.eq .Lctx_pop_expand

    // -- Include pop: restore source state too --
    ldr  x0, [x4, #PP_CTX_SRC_BASE]
    adrp x1, _kcc_src_base@PAGE
    add  x1, x1, _kcc_src_base@PAGEOFF
    str  x0, [x1]

    ldr  x0, [x4, #PP_CTX_SRC_SIZE]
    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    str  x0, [x1]

    ldr  x0, [x4, #PP_CTX_SRC_PATH]
    adrp x1, _kcc_src_path@PAGE
    add  x1, x1, _kcc_src_path@PAGEOFF
    str  x0, [x1]
    b .Lctx_pop_restore

.Lctx_pop_expand:
    // Unmark macro anti-recursion flag
    ldr  x0, [x4, #PP_CTX_MACRO]
    cbz  x0, .Lctx_pop_restore
    ldr  x1, [x0, #PP_MACRO_FLAGS]
    bic  x1, x1, #PP_FLAG_EXPANDING
    str  x1, [x0, #PP_MACRO_FLAGS]

.Lctx_pop_restore:
    // Restore lexer state
    ldr  x0, [x4, #PP_CTX_CURSOR]
    adrp x1, _kcc_lex_cursor@PAGE
    add  x1, x1, _kcc_lex_cursor@PAGEOFF
    str  x0, [x1]

    ldr  x0, [x4, #PP_CTX_END]
    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
```

```
    str  x0, [x1]

    ldr  x0, [x4, #PP_CTX_LINE]
    adrp x1, _kcc_lex_line@PAGE
    add  x1, x1, _kcc_lex_line@PAGEOFF
    str  x0, [x1]

    ldr  x0, [x4, #PP_CTX_LSTART]
    adrp x1, _kcc_lex_line_start@PAGE
    add  x1, x1, _kcc_lex_line_start@PAGEOFF
    str  x0, [x1]

    // Restore prev_line
    ldr  x0, [x4, #PP_CTX_PREVLINE]
    adrp x1, _kcc_pp_prev_line@PAGE
    add  x1, x1, _kcc_pp_prev_line@PAGEOFF
    str  x0, [x1]

.Lctx_pop_done:
    ldp x29, x30, [sp], #16
    ret
```

□

112 · defined at 111, 112 · used in 28f · uses _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_line 104b, _kcc_lex_line_start 104b, _kcc_pp_ctx_depth 116, _kcc_pp_ctx_stack 116, _kcc_pp_prev_line 116, _kcc_src_base 82, _kcc_src_path 82, _kcc_src_size 82

17.6. Helper Functions

Two utility routines support the directive handlers. _kcc_pp_skip_line advances the lexer cursor past the end of the current line, updating the line counter when it crosses a newline. Directives call it to discard trailing tokens after the significant part has been processed. _kcc_pp_rest_of_line returns a pointer and length for the remaining text on the directive line, with leading and trailing whitespace trimmed, so that #define bodies and #if expressions can be extracted cleanly.

⟨ PP: helpers 113 ⟩ ≡

```
.text
// -- _kcc_pp_skip_line ---
// Advances the lexer cursor past the end of the
   current line, skipping any
// block comment that spans the newline so the
   logical directive line ends
// where the comment does (not at the first
   physical newline inside it).
.p2align 2
_kcc_pp_skip_line:
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x1, [x0]
    adrp x2, _kcc_lex_end@PAGE
    add  x2, x2, _kcc_lex_end@PAGEOFF
    ldr  x2, [x2]
.Lsl_loop:
    cmp  x1, x2
    b.hs .Lsl_done
    ldrb w3, [x1]
    // A block comment is whitespace that may span
    newlines, so a directive's
    // trailing `/ * ... * /` extends its logical
   line.  Skip the whole comment;
    // otherwise a skipped multi-line `#define X 0
   x20 / * ... * /` would leave
    // the cursor mid-comment and an apostrophe in
    a word like "that's" would be
    // lexed as a character constant (kernel uapi/
   linux/openat2.h re-include).
    cmp  w3, #'/'
    b.ne .Lsl_notcmt
    add  x5, x1, #1
    cmp  x5, x2
    b.hs .Lsl_notcmt
    ldrb w4, [x5]
    cmp  w4, #'*'
    b.ne .Lsl_notcmt
    add  x1, x1, #2                    // enter
   comment, past the opening star
.Lsl_cmt:
    cmp  x1, x2
    b.hs .Lsl_done                     // EOF inside
   comment
    ldrb w3, [x1]
    cmp  w3, #'*'
    b.ne .Lsl_cmt_nl
    add  x5, x1, #1
    cmp  x5, x2
    b.hs .Lsl_cmt_adv
    ldrb w4, [x5]
    cmp  w4, #'/'
    b.ne .Lsl_cmt_nl
    add  x1, x1, #2                    // consume
   closing star-slash
    b .Lsl_loop
.Lsl_cmt_nl:
    cmp  w3, #0x0A
    b.ne .Lsl_cmt_adv
    adrp x4, _kcc_lex_line@PAGE
    add  x4, x4, _kcc_lex_line@PAGEOFF
    ldr  x5, [x4]
    add  x5, x5, #1
    str  x5, [x4]
    add  x5, x1, #1
    adrp x4, _kcc_lex_line_start@PAGE
    add  x4, x4, _kcc_lex_line_start@PAGEOFF
    str  x5, [x4]
.Lsl_cmt_adv:
    add  x1, x1, #1
    b .Lsl_cmt
.Lsl_notcmt:
    add  x1, x1, #1
    cmp  w3, #0x0A
    b.ne .Lsl_loop
    // Found newline: update line counter
    adrp x3, _kcc_lex_line@PAGE
    add  x3, x3, _kcc_lex_line@PAGEOFF
    ldr  x4, [x3]
    add  x4, x4, #1
    str  x4, [x3]
    adrp x3, _kcc_lex_line_start@PAGE
    add  x3, x3, _kcc_lex_line_start@PAGEOFF
    str  x1, [x3]
.Lsl_done:
    str  x1, [x0]
    ret

// -- _kcc_pp_rest_of_line ---
// Returns rest of directive line from cursor.
// x0 = text start (after leading whitespace), x1
   = text length
// Advances cursor to the newline (lexer will skip
    it).
.p2align 2
_kcc_pp_rest_of_line:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
```

```
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]

    adrp x23, _kcc_lex_cursor@PAGE
    add  x23, x23, _kcc_lex_cursor@PAGEOFF
    ldr  x19, [x23]          // cursor (scan
   pointer)
    adrp x2, _kcc_lex_end@PAGE
    add  x2, x2, _kcc_lex_end@PAGEOFF
    ldr  x20, [x2]           // end

    // Skip leading spaces/tabs
.Lrol_skip:
    cmp  x19, x20
    b.hs .Lrol_empty
    ldrb w3, [x19]
    cmp  w3, #' '
    b.eq .Lrol_sk
    cmp  w3, #0x09
    b.eq .Lrol_sk
    b .Lrol_start
.Lrol_sk:
    add  x19, x19, #1
    b .Lrol_skip

.Lrol_start:
    mov  x25, x19            // text start
    // Return a comment-stripped copy of the body
   (\stdref{5.1.1.2} phase 3,
    // which precedes macro definition): a body
   must not retain comments, or an
    // apostrophe or quote inside one---as in the
   kernel's "a node's interval"
    // doc comment---would later flip the
   substitutor into char/string mode and
    // copy the rest of the body verbatim, leaving
    every parameter and \#\# past
    // that point unexpanded.  Each comment
   becomes one space so adjacent tokens
    // stay separated; string and character
   literals are copied intact.  The
    // output never exceeds the input, so (end -
   start + 2) bytes suffice.
    sub  x0, x20, x25
    add  x0, x0, #2
    bl   _kcc_arena_alloc
    mov  x21, x0             // output buffer base
    mov  x22, #0             // op = output length
    mov  x24, #0             // state: 0=normal 1=
   block-comment 2=string 3=char
.Lrol_scan:
    cmp  x19, x20
    b.hs .Lrol_at_end
    ldrb w3, [x19]
    cmp  x24, #1
    b.eq .Lrol_st_block
    cmp  x24, #2
    b.eq .Lrol_st_str
    cmp  x24, #3
    b.eq .Lrol_st_char
    // normal state
    cmp  w3, #0x0A
    b.eq .Lrol_found_nl
    cmp  w3, #'"'
    b.eq .Lrol_enter_str
    cmp  w3, #0x27           // '
    b.eq .Lrol_enter_char
    cmp  w3, #'/'
    b.ne .Lrol_emit
    add  x8, x19, #1
    cmp  x8, x20
    b.hs .Lrol_emit
    ldrb w9, [x8]
    cmp  w9, #'*'
    b.eq .Lrol_enter_block
    cmp  w9, #'/'
    b.eq .Lrol_line_comment
    b .Lrol_emit
.Lrol_emit:
    strb w3, [x21, x22]
    add  x22, x22, #1
    add  x19, x19, #1
    b .Lrol_scan
.Lrol_enter_str:
    strb w3, [x21, x22]
    add  x22, x22, #1
    mov  x24, #2
    add  x19, x19, #1
    b .Lrol_scan
.Lrol_enter_char:
    strb w3, [x21, x22]
    add  x22, x22, #1
    mov  x24, #3
    add  x19, x19, #1
    b .Lrol_scan
.Lrol_enter_block:
    mov  x24, #1             // enter block comment
    (copy nothing)
    add  x19, x19, #2
    b .Lrol_scan
.Lrol_line_comment:
    add  x19, x19, #2        // // comment: skip to
    the end of the logical line
.Lrol_lc:
    cmp  x19, x20
    b.hs .Lrol_at_end
    ldrb w3, [x19]
    cmp  w3, #0x0A
    b.eq .Lrol_found_nl
    add  x19, x19, #1
    b .Lrol_lc
.Lrol_st_block:
    cmp  w3, #'*'
    b.ne .Lrol_block_adv
    add  x8, x19, #1
    cmp  x8, x20
    b.hs .Lrol_block_adv
    ldrb w9, [x8]
    cmp  w9, #'/'
    b.ne .Lrol_block_adv
    mov  x24, #0             // close block comment:
    emit a single space
    mov  w3, #' '
    strb w3, [x21, x22]
    add  x22, x22, #1
    add  x19, x19, #2
    b .Lrol_scan
.Lrol_block_adv:
    cmp  w3, #0x0A           // newline inside
   comment: keep line count accurate
    b.ne .Lrol_block_next
    adrp x8, _kcc_lex_line@PAGE
    add  x8, x8, _kcc_lex_line@PAGEOFF
    ldr  x9, [x8]
    add  x9, x9, #1
    str  x9, [x8]
.Lrol_block_next:
    add  x19, x19, #1
    b .Lrol_scan
.Lrol_st_str:
    cmp  w3, #0x5C           // backslash: copy it
   and the escaped character
    b.ne .Lrol_str_q
    strb w3, [x21, x22]
    add  x22, x22, #1
    add  x8, x19, #1
    cmp  x8, x20
    b.hs .Lrol_str_adv2
    ldrb w9, [x8]
```

```
    strb w9, [x21, x22]
    add  x22, x22, #1
.Lrol_str_adv2:
    add  x19, x19, #2
    b .Lrol_scan
.Lrol_str_q:
    cmp  w3, #'"'
    b.ne .Lrol_str_nl
    strb w3, [x21, x22]
    add  x22, x22, #1
    mov  x24, #0
    add  x19, x19, #1
    b .Lrol_scan
.Lrol_str_nl:
    cmp  w3, #0x0A          // an unterminated
   string still ends at the newline
    b.eq .Lrol_found_nl
    strb w3, [x21, x22]
    add  x22, x22, #1
    add  x19, x19, #1
    b .Lrol_scan
.Lrol_st_char:
    cmp  w3, #0x5C
    b.ne .Lrol_char_q
    strb w3, [x21, x22]
    add  x22, x22, #1
    add  x8, x19, #1
    cmp  x8, x20
    b.hs .Lrol_char_adv2
    ldrb w9, [x8]
    strb w9, [x21, x22]
    add  x22, x22, #1
.Lrol_char_adv2:
    add  x19, x19, #2
    b .Lrol_scan
.Lrol_char_q:
    cmp  w3, #0x27
    b.ne .Lrol_char_nl
    strb w3, [x21, x22]
    add  x22, x22, #1
    mov  x24, #0
    add  x19, x19, #1
    b .Lrol_scan
.Lrol_char_nl:
    cmp  w3, #0x0A
    b.eq .Lrol_found_nl
    strb w3, [x21, x22]
    add  x22, x22, #1
    add  x19, x19, #1
    b .Lrol_scan

.Lrol_found_nl:
    str  x19, [x23]           // cursor = newline (
   lexer will skip it later)
    b .Lrol_trim
.Lrol_at_end:
    str  x20, [x23]           // cursor = end
.Lrol_trim:
    // Trim trailing whitespace from the output
   buffer.
    cbz  x22, .Lrol_ret
    sub  x8, x22, #1
    ldrb w3, [x21, x8]
    cmp  w3, #' '
    b.eq .Lrol_trim_dec
    cmp  w3, #0x09
    b.ne .Lrol_ret
.Lrol_trim_dec:
    sub  x22, x22, #1
    b .Lrol_trim
.Lrol_ret:
    mov  x0, x21              // stripped body
   start
    mov  x1, x22              // stripped body
   length
    ldr  x25, [sp, #64]
    ldp  x23, x24, [sp, #48]
    ldp  x21, x22, [sp, #32]
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #80
    ret

.Lrol_empty:
    str  x20, [x23]           // cursor = end
    mov  x0, x19              // start == end:
   empty body
    mov  x1, #0
    ldr  x25, [sp, #64]
    ldp  x23, x24, [sp, #48]
    ldp  x21, x22, [sp, #32]
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #80
    ret
```

□

113 · used in 28f · uses _kcc_arena_alloc 58a, _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_line 104b, _kcc_lex_line_start 104b

17.7. Preprocessed Output (-E Mode)

kcc_pp_dump_all emits the entire preprocessed token stream to stdout, one token per line, in the same format as kcc_lex_dump_all.

⟨ PP: dump 115 ⟩ ≡

```
.text
// -- kcc_pp_dump_all ---
// void kcc_pp_dump_all(void)
// Calls kcc_pp_next in a loop and dumps each
   token.
.globl _kcc_pp_dump_all
.p2align 2
_kcc_pp_dump_all:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

.Lppdump_loop:
    bl _kcc_pp_next

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lppdump_done    // TOK_EOF

    bl _kcc_lex_dump_token
    b .Lppdump_loop

.Lppdump_done:
    ldp x29, x30, [sp], #16
    ret
```

□

115 · used in 28f · defines _kcc_pp_dump_all 622 · uses _kcc_lex_dump_token 99, _kcc_pp_next 108, _kcc_tok_type 104b

17.8. Global State

The preprocessor keeps its state in a set of global variables. The macro hash table (_kcc_pp_macros) maps macro names to definition structures. The context stack records saved lexer states for nested #include files and macro expansions, while the conditional stack tracks #if/#endif

nesting depth and branch state. The prev_line variable enables beginning-of-line detection for directive recognition.

⟨ PP: data 116 ⟩ ≡

```
// -- Preprocessor global state ---
.data
.p2align 3
.globl _kcc_pp_macros
_kcc_pp_macros:       .quad 0     // pointer to
   macro hash table
_kcc_pp_macro_stack: .quad 0      // head of the #
   pragma push_macro save stack
_kcc_pp_eval_depth:   .quad 0     // #if-expression
    macro recursion guard
.globl _kcc_line_delta
_kcc_line_delta:      .quad 0     // #line
   adjustment: __LINE__ = _kcc_lex_line + this
_kcc_pp_counter:      .quad 0     // __COUNTER__:
   next value to emit, incremented per use
_kcc_pp_drain_floor: .quad 0      // >0: pp_next
   stops when the ctx stack drops below it (
   argument prescan)
.globl _kcc_pp_ctx_depth
_kcc_pp_ctx_depth:    .quad 0     // context stack
   depth
_kcc_pp_cond_depth:   .quad 0     // conditional
   stack depth
_kcc_pp_prev_line:    .quad 0     // line of
   previous token (BOL detection)
_kcc_pp_has_pending: .quad 0      // pending token
   flag
_kcc_pp_dir_line:     .quad 0     // line of
   current directive

// Context stack (PP_MAX_CTX_DEPTH * PP_CTX_SIZE
   bytes)
// 256 * 80 = 20480 bytes
.p2align 3
_kcc_pp_ctx_stack:    .space 20480

// Conditional stack (PP_MAX_COND bytes)
.p2align 3
_kcc_pp_cond_stack:   .space 64

// -- Directive name strings ---
.section __TEXT,__cstring,cstring_literals
_kcc_pp_s_define:     .asciz "define"
_kcc_pp_s_undef:      .asciz "undef"
_kcc_pp_s_ifdef:      .asciz "ifdef"
_kcc_pp_s_ifndef:     .asciz "ifndef"
_kcc_pp_s_if:         .asciz "if"
_kcc_pp_s_elif:       .asciz "elif"
_kcc_pp_s_else:       .asciz "else"
_kcc_pp_s_endif:      .asciz "endif"
_kcc_pp_s_include:    .asciz "include"
_kcc_pp_s_error:      .asciz "error"
_kcc_pp_s_line:       .asciz "line"
_kcc_pp_s_pragma:     .asciz "pragma"
_kcc_pp_s_push_macro: .asciz "push_macro"
_kcc_pp_s_pop_macro:  .asciz "pop_macro"
_kcc_pp_s_defined:    .asciz "defined"

// -- Predefined macro name strings ---
_kcc_pp_pre_stdc:     .asciz "__STDC__"
_kcc_pp_pre_stdc_ver: .asciz "__STDC_VERSION__"
_kcc_pp_pre_hosted:   .asciz "__STDC_HOSTED__"
_kcc_pp_pre_no_atomics: .asciz "
   __STDC_NO_ATOMICS__"
_kcc_pp_pre_no_threads: .asciz "
   __STDC_NO_THREADS__"
_kcc_pp_pre_no_complex: .asciz "
   __STDC_NO_COMPLEX__"
_kcc_pp_pre_no_vla:   .asciz "__STDC_NO_VLA__"
_kcc_pp_pre_lp64:     .asciz "__LP64__"
_kcc_pp_pre_lp64b:    .asciz "_LP64"
_kcc_pp_va_args_str:  .asciz "__VA_ARGS__"
_kcc_pp_pre_linem:    .asciz "__LINE__"
_kcc_pp_pre_filem:    .asciz "__FILE__"
_kcc_pp_pre_counterm: .asciz "__COUNTER__"
_kcc_pp_file_unknown: .asciz "<stdin>"
_kcc_pp_pre_one:      .asciz "1"
_kcc_pp_pre_c17val:   .asciz "201710L"
// Byte-order macros (gcc/clang spelling).  Both
   kcc targets are little-endian.
_kcc_pp_pre_ord_le:   .asciz "
   __ORDER_LITTLE_ENDIAN__"
_kcc_pp_pre_ord_be:   .asciz "__ORDER_BIG_ENDIAN__
   "
_kcc_pp_pre_ord_pdp:  .asciz "__ORDER_PDP_ENDIAN__
   "
_kcc_pp_pre_byteord:  .asciz "__BYTE_ORDER__"
_kcc_pp_pre_le:       .asciz "__LITTLE_ENDIAN__"
_kcc_pp_pre_v1234:    .asciz "1234"
_kcc_pp_pre_v4321:    .asciz "4321"
_kcc_pp_pre_v3412:    .asciz "3412"

// -- Error messages ---
_kcc_pp_err_unknown_dir:  .asciz "unknown
   preprocessing directive"
_kcc_pp_err_def_name:     .asciz "expected
   identifier after #define"
_kcc_pp_err_def_param:    .asciz "expected
   parameter name or ')' in macro parameter list"
_kcc_pp_err_ifdef_name:   .asciz "expected
   identifier after #ifdef/#ifndef"
_kcc_pp_err_stray_elif:   .asciz "#elif without
   matching #if"
_kcc_pp_err_stray_else:   .asciz "#else without
   matching #if"
_kcc_pp_err_stray_endif:  .asciz "#endif without
   matching #if"
_kcc_pp_err_include:      .asciz "#include expects
    \"file\" or <file>"
_kcc_pp_err_notfound:     .asciz "#include file
   not found"
_kcc_pp_err_unterm_cond:  .asciz "unterminated
   conditional directive"
_kcc_pp_err_unterm_args:  .asciz "unterminated
   macro argument list"
_kcc_pp_err_depth:        .asciz "maximum include/
   expansion depth exceeded"
```

□

116 · used in 28f · defines _kcc_pp_cond_depth 107b, 108, 136, 139ab, _kcc_pp_cond_stack 108, 139ab, _kcc_pp_ctx_depth 107b, 108, 111, 112, 123, 125b, _kcc_pp_ctx_stack 111, 112, _kcc_pp_dir_line 109, 118, 125a, 135, 139ab, 140, 151, 159b, _kcc_pp_err_def_name 118, _kcc_pp_err_def_param 118, _kcc_pp_err_depth 111, _kcc_pp_err_ifdef_name 135, _kcc_pp_err_include 151, _kcc_pp_err_stray_elif 139a, _kcc_pp_err_stray_else 139b, _kcc_pp_err_stray_endif 139b, _kcc_pp_err_unknown_dir 110, _kcc_pp_err_unterm_args 125a, _kcc_pp_err_unterm_cond 108, _kcc_pp_has_pending 107b, 108, 125b, _kcc_pp_macros 107b, 118, 119ab, 121, 132, 135, 136, 146, _kcc_pp_prev_line 107b, 108, 111, 112, 125b, 151, _kcc_pp_pre_c17val 132, _kcc_pp_pre_hosted 132, _kcc_pp_pre_one 132, _kcc_pp_pre_stdc 132, _kcc_pp_pre_stdc_ver 132, _kcc_pp_s_define 109, _kcc_pp_s_defined 136, 146, _kcc_pp_s_elif 109, 140, _kcc_pp_s_else 109, 140, _kcc_pp_s_endif 109, 140, _kcc_pp_s_error 109, _kcc_pp_s_if 109, 140, _kcc_pp_s_ifdef 109, 140, _kcc_pp_s_ifndef 109, 140, _kcc_pp_s_include 109, _kcc_pp_s_line 109, _kcc_pp_s_pragma 109,

_kcc_pp_s_undef 109 · uses _kcc_lex_line 104b

17.9. Macro Definition (#define)

A #define directive creates a macro. If the identifier is immediately followed by ((no intervening whitespace), it is a function-like macro with parameters (§6.10.3). Otherwise it is an object-like macro.

The replacement body is the text from after the macro name (or parameter list) to the end of the directive line.

Each parameter is an identifier—or, as with the macro name, a token the lexer classified as a keyword, since translation phase 4 has no keywords (§6.4.1) and the kernel's TRACE_EVENT names a parameter struct. A bare ... in final position makes the macro variadic, binding the trailing arguments to __VA_ARGS__ (§6.10.3). The parser also accepts the GNU *named* variadic form name...—an identifier immediately followed by ...—which names the rest parameter so the body refers to the trailing arguments as name instead of __VA_ARGS__; the Linux kernel headers use this spelling heavily. Either way the binding name is written into the variadic slot params[x22] at definition time—__VA_ARGS__ for the bare form, the identifier for the named form—and the fixed-parameter count in x22 excludes it, so expansion need only collapse the trailing arguments into that already-named slot.

⟨ *PP: define* 117 ⟩ ≡

```
.text
// -- _kcc_pp_do_define ---
// Handles #define: reads macro name, optional
   params, body.
.p2align 2
_kcc_pp_do_define:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]

    // Read macro name.  During preprocessing (
   translation phase 4) there are
    // no keywords yet (\stdref{6.4.1}): `int`, `
   inline`, even `__attribute__`
    // are ordinary identifiers that a portability
    header may legally #define.
    // The lexer, however, eagerly classifies them
    as keyword tokens, so we
    // accept both TOK_IDENT and any keyword-class
    code [10, TOK_LPAREN) here.
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_IDENT
    b.eq .Ldef_name_ok
    sub  x1, x0, #10            // keyword-class
   tokens occupy [10, 60)
    cmp  x1, #50                // 60-10; hs →
   neither identifier nor keyword
    b.hs .Ldef_err_name
.Ldef_name_ok:

    // Save macro name (callee-saved)
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x19, [x0]              // x19 = name ptr
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr  x20, [x0]              // x20 = name len

    // Check if function-like: '(' immediately
   after name (no space)
    // Peek at the character right after the
   identifier in source
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
    ldr  x1, [x1]
    cmp  x0, x1
    b.hs .Ldef_object           // at end → object-
   like
    ldrb w2, [x0]
    cmp  w2, #'('
    b.eq .Ldef_funclike

.Ldef_object:
    // Object-like macro
    mov x21, #0                 // x21 = flags (not
    function-like)
    mov x22, #0                 // x22 = param
   count
    mov x23, #0                 // x23 = params ptr
    (none)

    // Get rest of line as replacement body
    bl _kcc_pp_rest_of_line
    mov x24, x0                 // x24 = body ptr
    mov x25, x1                 // x25 = body len
    b .Ldef_store

.Ldef_funclike:
    // Function-like macro
    mov x21, #PP_FLAG_FUNCLIKE
    mov x22, #0                 // param count
    mov x23, #0                 // params ptr

    // Consume the '(' by calling lex_next, then
   read first param/rparen
    bl _kcc_lex_next            // reads '(' token
    bl _kcc_lex_next            // reads first
   param or ')'

    // Allocate param array (max PP_MAX_ARGS
   entries)
    mov x0, #PP_MAX_ARGS
    mov x1, #PP_PARAM_SIZE
    mul x0, x0, x1
    bl _kcc_arena_alloc
    mov x23, x0                 // x23 = params
   array

    // Read parameters
    // Check for empty parameter list: )
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RPAREN
    b.eq .Ldef_params_done

.Ldef_param_loop:
    // A `...` here marks a variadic macro: the
   trailing arguments bind to
    // __VA_ARGS__ (\stdref{6.10.3}).  It is not a
    named parameter and must be
    // the last item, so the next token has to be
```

```
    ')'.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ELLIPSIS
    b.eq .Ldef_variadic
    // Expect a parameter name.  As with the macro
    name (translation phase 4 has
    // no keywords, \stdref{6.4.1}), the lexer may
    have classified it as a
    // keyword token, so accept TOK_IDENT or any
    keyword-class code [10, 60):
    // the kernel's TRACE_EVENT names a parameter
    `struct`.
    cmp  x0, #TOK_IDENT
    b.eq .Ldef_param_name_ok
    sub  x1, x0, #10
    cmp  x1, #50
    b.hs .Ldef_err_param
.Ldef_param_name_ok:

    // Store parameter name
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]

    // params[param_count] = {name, len}.  Do not
    count it yet: a trailing
    // `...` would make it a GNU named-variadic
    parameter (see .Ldef_named_variadic).
    mov x2, #PP_PARAM_SIZE
    mul x3, x22, x2
    add x3, x23, x3
    str x0, [x3, #PP_PARAM_NAME]
    str x1, [x3, #PP_PARAM_LEN]

    // Read next: '...' (named variadic), ',' or
    ')'
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ELLIPSIS
    b.eq .Ldef_named_variadic
    add x22, x22, #1
    cmp  x0, #TOK_COMMA
    b.ne .Ldef_check_rparen

    // Comma: read next parameter
    bl _kcc_lex_next
    b .Ldef_param_loop

.Ldef_check_rparen:
    cmp  x0, #TOK_RPAREN
    b.ne .Ldef_err_param
    b .Ldef_params_done

.Ldef_named_variadic:
    // GNU "name...": the identifier just stored
    in params[x22] binds the
    // trailing arguments (\stdref{6.10.3}, GNU
    extension).  Mark variadic and
    // leave x22 uncounted so that slot stays the
    variadic slot the expander
    // collapses into.
    orr x21, x21, #PP_FLAG_VARIADIC
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RPAREN
    b.ne .Ldef_err_param
    b .Ldef_params_done

.Ldef_variadic:
    orr x21, x21, #PP_FLAG_VARIADIC   // mark
    variadic; ... consumes no param slot
    // Bind the variadic slot params[x22] to
    __VA_ARGS__ at definition time
    // (the named-variadic path above binds it to
    an identifier instead), so
    // the expander's collapse need not choose the
    name.
    mov x2, #PP_PARAM_SIZE
    mul x3, x22, x2
    add x3, x23, x3
    adrp x4, _kcc_pp_va_args_str@PAGE
    add  x4, x4, _kcc_pp_va_args_str@PAGEOFF
    str  x4, [x3, #PP_PARAM_NAME]
    mov  x4, #11
    str  x4, [x3, #PP_PARAM_LEN]
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RPAREN              // ... must
    be the final parameter
    b.ne .Ldef_err_param

.Ldef_params_done:
    // Get rest of line as replacement body
    bl _kcc_pp_rest_of_line
    mov x24, x0                 // x24 = body ptr
    mov x25, x1                 // x25 = body len
    b .Ldef_store
```

□

117 · defined at 117, 118 · used in 28f · uses _kcc_arena_alloc 58a, _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_next 90b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

Storage and error reporting. Both the object-like and function-like paths converge at `.Ldef_store`, which allocates the macro definition structure in the arena, fills in every field, and inserts the definition into the macro hash table. If the directive was malformed—a missing name or an invalid parameter list—the corresponding error label emits a diagnostic and skips to the end of the line.

⟨ *PP: define* 118 ⟩ +≡

```
.Ldef_store:
    // Allocate macro definition in arena
    mov x0, #PP_MACRO_SIZE
    bl _kcc_arena_alloc
    mov x26, x0                 // x26 = macro_def
    ptr

    // Fill in the definition
    str x19, [x26, #PP_MACRO_NAME]
    str x20, [x26, #PP_MACRO_NAMELEN]
    str x21, [x26, #PP_MACRO_FLAGS]
    str x22, [x26, #PP_MACRO_NPARAMS]
    str x24, [x26, #PP_MACRO_BODY]
    str x25, [x26, #PP_MACRO_BODYLEN]
    str x23, [x26, #PP_MACRO_PARAMS]

    // Store in macro hash table
    adrp x0, _kcc_pp_macros@PAGE
    add  x0, x0, _kcc_pp_macros@PAGEOFF
    ldr  x0, [x0]               // hash table
```

```
    mov  x1, x19              // key = name
    mov  x2, x20              // keylen
    mov  x3, x26              // value = macro_def
    ptr
    bl _kcc_ht_put

    // Skip remaining text on directive line
    bl _kcc_pp_skip_line

    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret

.Ldef_err_name:
    adrp x0, _kcc_pp_dir_line@PAGE
    add  x0, x0, _kcc_pp_dir_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_def_name@PAGE
    add  x2, x2, _kcc_pp_err_def_name@PAGEOFF
    bl _kcc_diag_error
    bl _kcc_pp_skip_line
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret

.Ldef_err_param:
    adrp x0, _kcc_pp_dir_line@PAGE
    add  x0, x0, _kcc_pp_dir_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_def_param@PAGE
    add  x2, x2, _kcc_pp_err_def_param@PAGEOFF
    bl _kcc_diag_error
    bl _kcc_pp_skip_line
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

118 · defined at 117, 118 · used in 28f · uses _kcc_arena_alloc 58a, _kcc_diag_error 76a, _kcc_ht_put 63, _kcc_pp_dir_line 116, _kcc_pp_err_def_name 116, _kcc_pp_err_def_param 116, _kcc_pp_macros 116

17.10. Macro Undefine (#undef)

#undef removes a macro from the table by storing zero as its value.

⟨ *PP: undef* 119a ⟩ ≡

```
.text
// -- _kcc_pp_do_undef ---
.p2align 2
_kcc_pp_do_undef:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Read macro name
    bl _kcc_lex_next

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_IDENT
    b.ne .Lundef_done

    // Put 0 in the hash table for this name
    adrp x0, _kcc_pp_macros@PAGE
    add  x0, x0, _kcc_pp_macros@PAGEOFF
    ldr  x0, [x0]

    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    ldr  x1, [x1]

    adrp x2, _kcc_tok_len@PAGE
    add  x2, x2, _kcc_tok_len@PAGEOFF
    ldr  x2, [x2]

    mov  x3, #0                 // value = 0 (
    removed)
    bl _kcc_ht_put

.Lundef_done:
    bl _kcc_pp_skip_line
    ldp x29, x30, [sp], #16
    ret
```

□

119a · used in 28f · uses _kcc_ht_put 63, _kcc_lex_next 90b, _kcc_pp_macros 116, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

17.11. The push_macro and pop_macro Pragmas

Most #pragma directives are implementation-defined and kcc ignores them, but two are worth honouring because real headers lean on them to save and restore a macro around a local redefinition: #pragma push_macro("NAME") remembers NAME's current definition, and a later #pragma pop_macro("NAME") restores it, undoing any intervening #undef/#define. Because a redefinition never mutates the old macro object—#define installs a *fresh* entry and the arena keeps the previous one alive—saving is just capturing the current hash-table value (a macro pointer, possibly null for an undefined name) and restoring is putting it back.

The save records live on a small stack of arena nodes, each holding the next link, the macro name and length, and the saved macro pointer. A push prepends a node; a pop scans from the head for the newest node whose name matches and reinstalls its saved pointer, unlinking that node. A single global head, _kcc_pp_macro_stack, roots the stack, and the whole mechanism reuses the same _kcc_pp_macros table that #define and #undef manipulate.

⟨ *PP: pragma* 119b ⟩ ≡

```
// -- _kcc_pp_do_pragma ---
// Handles `#pragma push_macro("X")` / `#pragma
   pop_macro("X")`; any other
// pragma is skipped.  Called with the `pragma`
   keyword already consumed.
.text
.p2align 2
```

```
_kcc_pp_do_pragma:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    // Read the pragma verb.
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr x0, [x0]
    cmp x0, #TOK_IDENT
    b.ne .Lprag_skip
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr x19, [x0]                   // verb start
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr x20, [x0]                   // verb length
    // push_macro?  (10 characters)
    cmp x20, #10
    b.ne .Lprag_try_pop
    mov x0, x19
    adrp x1, _kcc_pp_s_push_macro@PAGE
    add  x1, x1, _kcc_pp_s_push_macro@PAGEOFF
    mov x2, #10
    bl _kcc_memcmp
    cbz x0, .Lprag_push
.Lprag_try_pop:
    cmp x20, #9                     // pop_macro (9
  characters)
    b.ne .Lprag_skip
    mov x0, x19
    adrp x1, _kcc_pp_s_pop_macro@PAGE
    add  x1, x1, _kcc_pp_s_pop_macro@PAGEOFF
    mov x2, #9
    bl _kcc_memcmp
    cbz x0, .Lprag_pop
    b .Lprag_skip

.Lprag_push:
    bl _kcc_pp_pragma_name          // x0=name ptr,
  x1=len (0 on parse failure)
    cbz x1, .Lprag_skip
    mov x19, x0                     // name
    mov x20, x1                     // len
    // saved = ht_get(macros, name, len)
    adrp x0, _kcc_pp_macros@PAGE
    add  x0, x0, _kcc_pp_macros@PAGEOFF
    ldr x0, [x0]
    mov x1, x19
    mov x2, x20
    bl _kcc_ht_get                  // x0 = current
  macro (or 0)
    str x0, [sp, #-16]!             // stash saved
  across arena_alloc (x2 is caller-saved)
    // node = arena_alloc(32); {next, name, len,
  saved}
    mov x0, #32
    bl _kcc_arena_alloc
    ldr x2, [sp], #16               // reload saved
    str x19, [x0, #8]               // name
    str x20, [x0, #16]              // len
    str x2, [x0, #24]               // saved macro
  pointer
    adrp x1, _kcc_pp_macro_stack@PAGE
    add  x1, x1, _kcc_pp_macro_stack@PAGEOFF
    ldr x2, [x1]                    // old head
    str x2, [x0, #0]                // node.next =
  head
    str x0, [x1]                    // head = node
    b .Lprag_skip

.Lprag_pop:
    bl _kcc_pp_pragma_name          // x0=name, x1=
  len
    cbz x1, .Lprag_skip
    mov x19, x0                     // name
    mov x20, x1                     // len
    // Scan the stack for the newest node with a
  matching name.
    adrp x0, _kcc_pp_macro_stack@PAGE
    add  x0, x0, _kcc_pp_macro_stack@PAGEOFF
    mov x3, x0                      // x3 = address
  of the link to update (head first)
.Lprag_pop_scan:
    ldr x4, [x3]                    // x4 = current
  node
    cbz x4, .Lprag_skip             // no matching
  push: nothing to restore
    ldr x1, [x4, #16]               // node.len
    cmp x1, x20
    b.ne .Lprag_pop_next
    mov x0, x19                     // name
    ldr x1, [x4, #8]                // node.name
    mov x2, x20                     // len
    stp x3, x4, [sp, #-16]!         // preserve link
  addr + node across memcmp
    bl _kcc_memcmp
    ldp x3, x4, [sp], #16
    cbz x0, .Lprag_pop_found
.Lprag_pop_next:
    add x3, x4, #0                  // link to update
    = &node.next (offset 0)
    b .Lprag_pop_scan
.Lprag_pop_found:
    // Restore: ht_put(macros, name, len, node.
  saved).
    adrp x0, _kcc_pp_macros@PAGE
    add  x0, x0, _kcc_pp_macros@PAGEOFF
    ldr x0, [x0]
    mov x1, x19
    mov x2, x20
    ldr x5, [x4, #24]               // saved macro
  pointer
    stp x3, x4, [sp, #-16]!
    mov x3, x5
    bl _kcc_ht_put
    ldp x3, x4, [sp], #16
    // Unlink node: *link = node.next.
    ldr x0, [x4, #0]
    str x0, [x3]
    b .Lprag_skip

.Lprag_skip:
    bl _kcc_pp_skip_line
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- _kcc_pp_pragma_name ---
// Parse `( "NAME" )` following a push_macro/
  pop_macro verb.  Returns
// x0 = pointer to NAME's bytes (inside the string
    literal), x1 = its length,
// or x1 = 0 on any parse failure.  The macro name
    is the string-literal
// content without its surrounding quotes.
.text
.p2align 2
_kcc_pp_pragma_name:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    bl _kcc_lex_next                // (
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr x0, [x0]
    cmp x0, #TOK_LPAREN
    b.ne .Lpname_fail
```

```
    bl _kcc_lex_next              // "NAME"
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr x0, [x0]
    cmp x0, #TOK_STRING_LIT
    b.ne .Lpname_fail
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr x1, [x1]
    add x0, x0, #1                // skip opening
   quote
    sub x1, x1, #2                // drop both
   quotes
    ldp x29, x30, [sp], #16
    ret
.Lpname_fail:
    mov x1, #0
    ldp x29, x30, [sp], #16
    ret
```

□

119b · used in 28f · defines _kcc_pp_do_pragma 110, _kcc_pp_pragma_name never used · uses _kcc_arena_alloc 58a, _kcc_ht_get 62, _kcc_ht_put 63, _kcc_lex_next 90b, _kcc_pp_macros 116, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

17.12. Macro Expansion

When kcc_pp_next encounters an identifier, it checks the macro table. If a macro is found and is not currently being expanded (anti-recursion per §6.10.3.4), the replacement text is pushed onto the context stack.

For function-like macros, arguments are collected first, and parameter names in the replacement text are substituted with the corresponding argument text.

⟨ PP: expand 121 ⟩ ≡

```
.text
// -- _kcc_pp_try_expand ---
// Checks current identifier token against macro
   table.
// Returns x0 = 1 if expanded (pushed context), 0
   if not.
.globl _kcc_pp_try_expand
.p2align 2
_kcc_pp_try_expand:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    // Get token start and length
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x19, [x0]              // x19 = name ptr
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr  x20, [x0]              // x20 = name len

    // Look up in macro table
    adrp x0, _kcc_pp_macros@PAGE
    add  x0, x0, _kcc_pp_macros@PAGEOFF
    ldr  x0, [x0]
    mov  x1, x19
    mov  x2, x20
    bl _kcc_ht_get
    cbz  x0, .Lexp_no           // not a macro
    mov  x21, x0                // x21 = macro_def
   ptr

    // Check anti-recursion flag
    ldr  x0, [x21, #PP_MACRO_FLAGS]
    tst  x0, #PP_FLAG_EXPANDING
    b.ne .Lexp_no               // already expanding
   → don't recurse

    // Dynamic built-in __LINE__: expands to the
   current line number, formatted
    // fresh at each use, so a static macro body
   cannot represent it.
    tst  x0, #PP_FLAG_BUILTIN
    b.eq .Lexp_not_dyn
    cmp  x20, #8                // "__LINE__" and "
   __FILE__" are both length 8
    b.ne .Lexp_chk_counter
    mov  x0, x19
    adrp x1, _kcc_pp_pre_linem@PAGE
    add  x1, x1, _kcc_pp_pre_linem@PAGEOFF
    mov  x2, #8
    bl _kcc_memcmp
    cbz  x0, .Lexp_line_dyn
    mov  x0, x19
    adrp x1, _kcc_pp_pre_filem@PAGE
    add  x1, x1, _kcc_pp_pre_filem@PAGEOFF
    mov  x2, #8
    bl _kcc_memcmp
    cbz  x0, .Lexp_file_dyn
    b .Lexp_not_dyn
.Lexp_chk_counter:
    // __COUNTER__ (length 11): a fresh sequential
    integer at each use.
    cmp  x20, #11
    b.ne .Lexp_not_dyn
    mov  x0, x19
    adrp x1, _kcc_pp_pre_counterm@PAGE
    add  x1, x1, _kcc_pp_pre_counterm@PAGEOFF
    mov  x2, #11
    bl _kcc_memcmp
    cbz  x0, .Lexp_counter_dyn
.Lexp_not_dyn:

    // Check if function-like
    ldr  x0, [x21, #PP_MACRO_FLAGS]
    tst  x0, #PP_FLAG_FUNCLIKE
    b.ne .Lexp_funclike

    // --- Object-like expansion ---
    ldr  x22, [x21, #PP_MACRO_BODY]     // body
   ptr
    ldr  x23, [x21, #PP_MACRO_BODYLEN]  // body
   len
    cbz  x23, .Lexp_empty       // empty replacement

    // Copy body to arena buffer (for lexer to
   scan)
    mov  x0, x23
    bl _kcc_arena_alloc
    mov  x24, x0                // x24 = buffer
    mov  x0, x24
    mov  x1, x22
    mov  x2, x23
    bl _kcc_memcpy

    // Mark macro as expanding
    ldr  x0, [x21, #PP_MACRO_FLAGS]
    orr  x0, x0, #PP_FLAG_EXPANDING
    str  x0, [x21, #PP_MACRO_FLAGS]

    // Push expansion context
```

```
    mov  x0, #PP_CTX_EXPAND
    mov  x1, x21              // macro_def for
   unmark on pop
    mov  x2, x24              // buffer start
    mov  x3, x23              // buffer len
    bl _kcc_pp_ctx_push_expand

    mov  x0, #1               // expanded
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.Lexp_empty:
    // Empty macro body - nothing to expand,
   return 1 to skip the token
    mov  x0, #1
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.Lexp_counter_dyn:
    // __COUNTER__ - expand to the current value
   of a monotonic counter, then
    // increment it so the next use sees the
   successor (\stdref{gcc __COUNTER__}).
    // The value is formatted through the same
   decimal path as __LINE__.
    adrp x0, _kcc_pp_counter@PAGE
    add  x0, x0, _kcc_pp_counter@PAGEOFF
    ldr  x22, [x0]            // value to emit
   = current counter
    add  x1, x22, #1
    str  x1, [x0]             // counter++ for
   the next expansion
    b .Lexp_num_format

.Lexp_line_dyn:
    // Format the current line number as a decimal
    token and push it as the
    // expansion.  Digits are written from the end
    of a small arena buffer.
    // The physical line is offset by any #line
   adjustment.
    adrp x0, _kcc_lex_line@PAGE
    add  x0, x0, _kcc_lex_line@PAGEOFF
    ldr  x22, [x0]            // physical line
   number
    adrp x0, _kcc_line_delta@PAGE
    add  x0, x0, _kcc_line_delta@PAGEOFF
    ldr  x0, [x0]
    add  x22, x22, x0         // presumed line
   = physical + #line delta
.Lexp_num_format:
    mov  x0, #24
    bl _kcc_arena_alloc
    mov  x24, x0              // buffer
    add  x23, x24, #24        // one past the
   end
    mov  x5, #10
    cbnz x22, .Lexp_line_loop
    mov  w1, #'0'
    sub  x23, x23, #1
    strb w1, [x23]
    b .Lexp_line_push
.Lexp_line_loop:
    udiv x1, x22, x5
    msub x2, x1, x5, x22      // x2 = line % 10
    add  w2, w2, #'0'
    sub  x23, x23, #1
    strb w2, [x23]
    mov  x22, x1
    cbnz x22, .Lexp_line_loop
.Lexp_line_push:
    add  x0, x24, #24
    sub  x22, x0, x23         // x22 = digit
   count
    // Mark builtin expanding and push, symmetric
   with the object-like path.
    ldr  x0, [x21, #PP_MACRO_FLAGS]
    orr  x0, x0, #PP_FLAG_EXPANDING
    str  x0, [x21, #PP_MACRO_FLAGS]
    mov  x0, #PP_CTX_EXPAND
    mov  x1, x21              // macro_def for
   unmark on pop
    mov  x2, x23              // digit buffer
    mov  x3, x22              // length
    bl _kcc_pp_ctx_push_expand
    mov  x0, #1
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.Lexp_file_dyn:
    // __FILE__ - expand to the current source
   path as a string-literal token.
    // _kcc_src_path tracks the file being read (
   the include machinery saves and
    // restores it, §6.10.8.1), so this is the
   header's own name inside a header.
    adrp x0, _kcc_src_path@PAGE
    add  x0, x0, _kcc_src_path@PAGEOFF
    ldr  x22, [x0]            // x22 = path
   pointer (0 before any file)
    cbnz x22, .Lexp_file_have
    adrp x22, _kcc_pp_file_unknown@PAGE
    add  x22, x22, _kcc_pp_file_unknown@PAGEOFF
.Lexp_file_have:
    mov  x0, x22
    bl _kcc_strlen
    mov  x23, x0              // x23 = path
   length
    add  x0, x23, #3          // 2 quotes + 1
   slack
    bl _kcc_arena_alloc
    mov  x24, x0              // x24 = buffer
    mov  w1, #'"'
    strb w1, [x24]            // opening quote
    add  x0, x24, #1
    mov  x1, x22
    mov  x2, x23
    bl _kcc_memcpy            // path bytes
   after the quote
    add  x0, x24, #1
    add  x0, x0, x23
    mov  w1, #'"'
    strb w1, [x0]             // closing quote
    add  x22, x23, #2         // token length =
    path + 2 quotes
    ldr  x0, [x21, #PP_MACRO_FLAGS]
    orr  x0, x0, #PP_FLAG_EXPANDING
    str  x0, [x21, #PP_MACRO_FLAGS]
    mov  x0, #PP_CTX_EXPAND
    mov  x1, x21              // macro_def for
   unmark on pop
    mov  x2, x24              // buffer
    mov  x3, x22              // length
    bl _kcc_pp_ctx_push_expand
    mov  x0, #1
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
```

```
    ret
□
```

121 · defined at 121, 123, 125ab, 127, 130, 131 · used in 28f · defines _kcc_pp_try_expand 108, 132 · uses _kcc_arena_alloc 58a, _kcc_ht_get 62, _kcc_lex_line 104b, _kcc_pp_macros 116, _kcc_src_path 82, _kcc_tok_len 104b, _kcc_tok_start 104b

Function-like expansion. When a function-like macro is invoked, the preprocessor must first locate the opening parenthesis (skipping whitespace), then collect the argument list by tracking parenthesis depth across nested calls and splitting on commas at the top level. Once all arguments are gathered, the substitution engine replaces each parameter name in the replacement body with the corresponding argument text before pushing the result onto the context stack.

The depth counter must survive the argument loop's own calls. Each iteration invokes `_kcc_lex_next`, which for an identifier argument calls `_kcc_ht_get` to test it against the keyword table—and any such call is free to clobber the caller-saved scratch registers. A depth kept in a scratch register therefore held only by luck for numeric arguments and was corrupted the moment an argument was an identifier or keyword, so a call like `FOO(int)` or `CJSON_PUBLIC(cJSON *)` never found its closing parenthesis and reported an "unterminated macro argument list." The depth lives in a callee-saved register (x20, dead in this path once the macro has been looked up), which the called routines are obliged to preserve.

⟨ *PP: expand* 123 ⟩ + ≡

```
.Lexp_funclike:
    // Function-like: peek for '(' after the
   identifier.  A function-like macro
    // invocation may span a context boundary---an
    object-like macro can expand
    // to a function-like macro name whose '('
   lies in the enclosing text
    // (`#define outer inner` used as `outer(a,b)
   `).  So first pop any expansion
    // context we have run to the end of, exposing
    that enclosing text; each
    // such context is exhausted and would be
   popped by the next read anyway.
.Lexp_fl_droptail:
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
    ldr  x1, [x1]
    cmp  x0, x1
    b.lo .Lexp_peek_ws          // current buffer
    still has text
    adrp x2, _kcc_pp_ctx_depth@PAGE
    add  x2, x2, _kcc_pp_ctx_depth@PAGEOFF
    ldr  x2, [x2]
    cbz  x2, .Lexp_peek_ws      // no enclosing
   context → genuinely at end
    // Never pop past the argument-prescan
   boundary.  While expanding a macro
    // argument, drain_floor marks the pushed
   argument context; popping it to
    // hunt for '(' would reach into the enclosing
    macro body and wrongly treat
    // a bare function-like macro name passed as
   an argument (say `l_addi` in
    // `op_arithI(l_addi)`, whose body is `iop
   (0,5,1)`) as an invocation that
    // consumes the body's '(', dropping the name.
    A function-like macro name
    // with no '(' inside its own argument is just
    an identifier (\stdref{6.10.3}).
    adrp x3, _kcc_pp_drain_floor@PAGE
    add  x3, x3, _kcc_pp_drain_floor@PAGEOFF
    ldr  x3, [x3]
    cmp  x2, x3
    b.ls .Lexp_peek_ws          // at/below the
   drain floor → don't cross it
    bl _kcc_pp_ctx_pop
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
    ldr  x1, [x1]
    b .Lexp_fl_droptail

    // Skip whitespace looking for '('
.Lexp_peek_ws:
    cmp  x0, x1
    b.hs .Lexp_no               // EOF → not a
   function call
    ldrb w2, [x0]
    cmp  w2, #' '
    b.eq .Lexp_peek_next
    cmp  w2, #0x09
    b.eq .Lexp_peek_next
    cmp  w2, #0x0A
    b.eq .Lexp_peek_next
    cmp  w2, #0x0D
    b.eq .Lexp_peek_next
    // Non-whitespace found
    cmp  w2, #'('
    b.ne .Lexp_no               // not '(' → not a
   function call
    b .Lexp_func_call
.Lexp_peek_next:
    add  x0, x0, #1
    b .Lexp_peek_ws

.Lexp_func_call:
    // Consume the '(' via lexer
    bl _kcc_lex_next            // should yield
   TOK_LPAREN

    // Collect arguments
    ldr  x22, [x21, #PP_MACRO_NPARAMS]  //
   expected param count

    // Allocate argument array: PP_MAX_ARGS * 16
   bytes
    mov  x0, #PP_MAX_ARGS
    lsl  x0, x0, #4             // * 16
    bl _kcc_arena_alloc
    mov  x23, x0                // x23 = arg array

    // Collect arguments: track paren depth in a
   callee-saved register (x20,
    // dead here) so _kcc_lex_next's calls cannot
   clobber it.
    mov  x24, #0                // arg count
    mov  x20, #1               // paren depth (we
   already consumed '(')

    // Mark start of first argument
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x0, [x0]
```

```
    // Save arg start in arg array
    lsl  x1, x24, #4
    add  x1, x23, x1
    str  x0, [x1, #0]          // arg[0].start =
   cursor
    str  xzr, [x1, #8]         // arg[0].len = 0 (
   placeholder)

.Lexp_arg_loop:
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    cmp  x0, #TOK_EOF
    b.eq .Lexp_arg_err          // unterminated
   args

    cmp  x0, #TOK_LPAREN
    b.ne .Lexp_arg_not_lp
    add  x20, x20, #1          // depth++
    b .Lexp_arg_loop

.Lexp_arg_not_lp:
    cmp  x0, #TOK_RPAREN
    b.ne .Lexp_arg_not_rp
    sub  x20, x20, #1          // depth--
    cbz  x20, .Lexp_arg_end    // matched → done
    b .Lexp_arg_loop

.Lexp_arg_not_rp:
    cmp  x0, #TOK_COMMA
    b.ne .Lexp_arg_loop
    cmp  x20, #1
    b.ne .Lexp_arg_loop         // comma inside
   nested parens

    // End of argument: compute length
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]              // comma position
    lsl  x1, x24, #4
    add  x1, x23, x1
    ldr  x2, [x1, #0]          // arg start
    sub  x2, x0, x2             // length =
   comma_start - arg_start
    str  x2, [x1, #8]

    // Start next argument
    add  x24, x24, #1
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x0, [x0]
    lsl  x1, x24, #4
    add  x1, x23, x1
    str  x0, [x1, #0]          // next arg start
    str  xzr, [x1, #8]
    b .Lexp_arg_loop

.Lexp_arg_end:
    // Finalize last argument
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]              // ')' position
    lsl  x1, x24, #4
    add  x1, x23, x1
    ldr  x2, [x1, #0]          // arg start
    sub  x2, x0, x2             // length
    str  x2, [x1, #8]
    add  x24, x24, #1          // total arg count

    // Variadic macro (\stdref{6.10.3.1}):
   collapse the trailing arguments past
    // the fixed parameters into a single
   __VA_ARGS__ argument---the raw source
    // span from the first trailing arg to the
   last, commas and all.  The
    // variadic slot params[NPARAMS] was already
   bound to its name at definition
    // time---"__VA_ARGS__" for a plain `...`, or
   the identifier for a GNU
    // "name..." parameter---so the ordinary
   substitution engine replaces that
    // name with the joined text.  Too few args (
   only fixed ones) yield an empty
    // variadic argument.
    ldr  x0, [x21, #PP_MACRO_FLAGS]
    tst  x0, #PP_FLAG_VARIADIC
    b.eq .Lexp_arg_done_va
    ldr  x2, [x21, #PP_MACRO_NPARAMS]   // fixed
   parameter count
    lsl  x3, x2, #4
    add  x3, x23, x3                    // &arg[
   NPARAMS]
    cmp  x24, x2
    b.le .Lexp_va_empty                 // no
   trailing args → empty __VA_ARGS__
    ldr  x4, [x3, #0]                   // first
   trailing arg's start
    sub  x5, x24, #1                    // last
   arg index
    lsl  x6, x5, #4
    add  x6, x23, x6                    // &arg[
   last]
    ldr  x7, [x6, #0]                   // last
   arg start
    ldr  x9, [x6, #8]                   // last
   arg len
    add  x7, x7, x9                     // last
   arg end
    sub  x7, x7, x4                     // joined
   length
    str  x7, [x3, #8]                   // arg[
   NPARAMS].len = joined (start kept)
    b .Lexp_va_setcount
.Lexp_va_empty:
    ldr  x4, [x23, #0]                  // any
   valid readable start; length 0
    str  x4, [x3, #0]
    str  xzr, [x3, #8]
.Lexp_va_setcount:
    add  x2, x2, #1                     // params/
   args now include __VA_ARGS__
    mov  x22, x2                        // param
   count for substitute
    mov  x24, x2                        // arg
   count
.Lexp_arg_done_va:
```

□

123 · defined at 121, 123, 125ab, 127, 130, 131 · used in 28f · uses _kcc_arena_alloc 58a, _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_next 90b, _kcc_pp_ctx_depth 116, _kcc_tok_start 104b, _kcc_tok_type 104b

Substitution and expansion context. With all arguments collected in the argument array, the expansion proceeds to the substitution phase. The macro's replacement body, parameter list, and the freshly gathered argument texts are passed to `_kcc_pp_substitute`, which performs textual replacement of parameter names with argument text. If the result is non-empty, the macro is marked with `PP_FLAG_EXPANDING` to prevent recursive re-expansion, and the substituted text is pushed onto the con-

text stack.

⟨ *PP: expand* 125a ⟩ + ≡

```
    // Argument prescan (6.10.3.1): fully expand
   each argument before it is
    // substituted, while this macro's
   PP_FLAG_EXPANDING is still clear, so a
    // nested invocation of the same macro inside
   an argument expands.
    mov  x0, x23              // arg array
    mov  x1, x24              // arg count
    bl _kcc_pp_expand_all_args

    // Now substitute: build expanded text
    // For each identifier in body matching a
   param name, replace with arg text
    ldr  x0, [x21, #PP_MACRO_BODY]
    ldr  x1, [x21, #PP_MACRO_BODYLEN]
    ldr  x2, [x21, #PP_MACRO_PARAMS]
    mov  x3, x22              // param count
    mov  x4, x23              // arg array
    mov  x5, x24              // arg count
    bl _kcc_pp_substitute
    // Returns x0 = buffer ptr, x1 = buffer len

    cbz  x1, .Lexp_empty_func

    mov  x22, x0              // buffer
    mov  x23, x1              // len

    // Mark macro as expanding
    ldr  x0, [x21, #PP_MACRO_FLAGS]
    orr  x0, x0, #PP_FLAG_EXPANDING
    str  x0, [x21, #PP_MACRO_FLAGS]

    // Push expansion context
    mov  x0, #PP_CTX_EXPAND
    mov  x1, x21
    mov  x2, x22
    mov  x3, x23
    bl _kcc_pp_ctx_push_expand

    mov  x0, #1
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.Lexp_empty_func:
    mov  x0, #1
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.Lexp_arg_err:
    adrp x0, _kcc_pp_dir_line@PAGE
    add  x0, x0, _kcc_pp_dir_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_unterm_args@PAGE
    add  x2, x2, _kcc_pp_err_unterm_args@PAGEOFF
    bl _kcc_diag_error
    mov  x0, #0
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.Lexp_no:
    mov  x0, #0               // not expanded
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

125a · defined at 121, 123, 125ab, 127, 130, 131 · used in 28f · uses _kcc_diag_error 76a, _kcc_pp_dir_line 116, _kcc_pp_err_unterm_args 116, _kcc_pp_expand_all_args 125b

17.12.1. Argument Prescan

§6.10.3.1 requires each macro argument to be *completely macro-expanded* before it is substituted into the replacement list. Our expander previously skipped this step, substituting the raw argument text and relying on the later rescan to expand everything. That fails for a function-like macro nested inside its own argument—`cast(StkId, cast_charp(p))` where `cast_charp` is itself `cast(char *, …)`, or the minimal `F(1, F(2, 3))`: the single per-macro `PP_FLAG_EXPANDING` bit paints the macro across the entire rescan, including the substituted argument, so the nested invocation reaches the parser unexpanded ("expected expression").

`_kcc_pp_expand_arg` fixes this by expanding an argument to text up front, while the outer macro's flag is not yet set. It pushes the argument text as an expansion context carrying *no* macro (so nothing is painted on its account) and drains `_kcc_pp_next`, appending each fully-expanded token to a growing buffer. The subtle part is knowing when the argument ends: `_kcc_pp_next` normally pops an exhausted context and reads straight on into the enclosing text, and that trailing token could itself be a macro it would expand—silently pulling surrounding tokens into the argument. So the drain installs a *drain floor*: `_kcc_pp_next` returns its `TOK_EOF` the moment the context stack drops below the floor, one above the depth at entry, rather than reading further. The floor is saved and restored so a nested prescan (an argument that invokes this same macro) installs its own deeper floor. Tokens are joined with single spaces, which never changes meaning because kcc implements neither # nor ##—there are no operands whose spelling or pasting the spacing could disturb.

⟨ *PP: expand* 125b ⟩ + ≡

```
// -- _kcc_pp_expand_arg ---
// (const char *text, uint64_t len) → x0 =
   expanded buf, x1 = expanded len
// Fully macro-expands one argument's text (
   argument prescan, 6.10.3.1).
.p2align 2
_kcc_pp_expand_arg:
    stp x29, x30, [sp, #-144]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str x27, [sp, #80]

    mov x19, x0                   // text
```

```
    mov x20, x1                     // len

    // Save the source beginning-of-line watermark
    across the argument prescan.
    // Lexing an argument runs through the pp loop
, which updates _kcc_pp_prev_line
    // for each token it reads; an argument whose
source span carries embedded
    // newlines (a multi-line invocation like
SQLite's per-VFS IOMETHODS(...)
    // spread across lines) advances the line
counter and leaves prev_line ABOVE
    // the invocation's line.  That leak makes the
    next real directive appear to
    // be on a line <= prev_line, so it fails the
beginning-of-line test and is
    // silently mistaken for ordinary tokens.  The
    lexer's own line counter is
    // restored by the context pop; prev_line is a
    separate global, so save and
    // restore it here too.
    adrp x2, _kcc_pp_prev_line@PAGE
    add  x2, x2, _kcc_pp_prev_line@PAGEOFF
    ldr  x2, [x2]
    str  x2, [sp, #112]             // saved
prev_line

    cbz x20, .Lea_empty

    // base_depth = current stack depth.  Set the
drain floor one above it, so
    // _kcc_pp_next stops (returns EOF) the
instant the argument context we are
    // about to push is popped---never reading on
into the enclosing text.  The
    // floor is saved/restored so nested prescans
(an argument that itself calls
    // this same macro) each install their own,
deeper floor.
    adrp x0, _kcc_pp_ctx_depth@PAGE
    add  x0, x0, _kcc_pp_ctx_depth@PAGEOFF
    ldr  x24, [x0]                  // base_depth
    adrp x0, _kcc_pp_drain_floor@PAGE
    add  x0, x0, _kcc_pp_drain_floor@PAGEOFF
    ldr  x1, [x0]
    str  x1, [sp, #96]              // saved
drain_floor
    add  x1, x24, #1
    str  x1, [x0]                   // floor =
base_depth + 1
    adrp x0, _kcc_pp_has_pending@PAGE
    add  x0, x0, _kcc_pp_has_pending@PAGEOFF
    ldr  x1, [x0]
    str  x1, [sp, #104]             // saved
has_pending
    str  xzr, [x0]                  // clear so the
drain reads the arg buffer

    // Output buffer, initial cap = len*4 + 256.
    lsl x0, x20, #2
    add x23, x0, #256               // cap
    mov x0, x23
    bl _kcc_arena_alloc
    mov x21, x0                     // out_buf
    mov x22, #0                     // out_len
    mov x27, #0                     // prev_end: end
of the last token emitted

    // Push the argument text as an expand context
    with no macro.
    mov x0, #PP_CTX_EXPAND
    mov x1, #0
    mov x2, x19
    mov x3, x20
    bl _kcc_pp_ctx_push_expand

.Lea_loop:
    bl _kcc_pp_next
    // Argument done once the stack falls back to
    base_depth.
    adrp x0, _kcc_pp_ctx_depth@PAGE
    add  x0, x0, _kcc_pp_ctx_depth@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, x24
    b.le .Lea_done
    // EOF safety
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lea_done
    // Token text → x25 = start, x26 = len.
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x25, [x0]
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr  x26, [x0]
    // Ensure room for a space + the token; grow (
    alloc+copy) if needed.
    add x0, x22, x26
    add x0, x0, #1
    cmp x0, x23
    b.ls .Lea_do_append
    lsl x0, x0, #1
    mov x23, x0                     // new cap
    bl _kcc_arena_alloc
    str x0, [sp, #136]              // new buf
    mov x1, x21
    mov x2, x22
    bl _kcc_memcpy                  // memcpy(new,
    old, out_len)
    ldr x21, [sp, #136]
.Lea_do_append:
    // Separate consecutive tokens with a single
    space ONLY where the source had
    // whitespace: two tokens adjacent in the same
    buffer (`.` then `section` in
    // `.section`) must stay joined, or a later
    `#`-stringify would produce
    // `. section` and break asm templates like
    EXPORT_SYMBOL's `.section`.  A
    // buffer change (macro-expansion boundary)
    leaves the token start != prev_end
    // and gets a space, which also blocks
    accidental token paste.  No separator
    // precedes the first token (a leading space
    would defeat a `##` paste).
    cbz x22, .Lea_no_sep
    cmp x25, x27                    // token adjacent
    to the previous one?
    b.eq .Lea_no_sep
    mov w0, #0x20                   // separator
    space
    strb w0, [x21, x22]
    add x22, x22, #1
.Lea_no_sep:
    add x0, x21, x22
    mov x1, x25
    mov x2, x26
    bl _kcc_memcpy
    add x22, x22, x26
    add x27, x25, x26               // prev_end =
    token start + length
    b .Lea_loop

.Lea_done:
    // Restore the drain floor and pending flag.
    The argument context has
```

```
    // already popped cleanly (the floor stopped
   pp_next at its EOF), so the
    // lexer is back at the enclosing position
   with nothing to undo.
    ldr x0, [sp, #96]
    adrp x1, _kcc_pp_drain_floor@PAGE
    add  x1, x1, _kcc_pp_drain_floor@PAGEOFF
    str  x0, [x1]
    ldr x0, [sp, #104]
    adrp x1, _kcc_pp_has_pending@PAGE
    add  x1, x1, _kcc_pp_has_pending@PAGEOFF
    str  x0, [x1]
    ldr x0, [sp, #112]          // restore
   prev_line (see prologue)
    adrp x1, _kcc_pp_prev_line@PAGE
    add  x1, x1, _kcc_pp_prev_line@PAGEOFF
    str  x0, [x1]
    mov x0, x21                 // buf
    mov x1, x22                 // len
    ldr x27, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #144
    ret

.Lea_empty:
    ldr x2, [sp, #112]          // restore
   prev_line (no lexing happened, but keep
   symmetric)
    adrp x3, _kcc_pp_prev_line@PAGE
    add  x3, x3, _kcc_pp_prev_line@PAGEOFF
    str  x2, [x3]
    mov x0, x19                 // original (
   empty) text
    mov x1, #0
    ldr x27, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #144
    ret

// -- _kcc_pp_expand_all_args ---
// (arg_array, count): replaces each 16-byte {
   start,len} entry with its
// fully-expanded {buf,len} in place, ahead of
   substitution.
.p2align 2
_kcc_pp_expand_all_args:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                 // arg array
    mov x20, x1                 // count
    mov x21, #0                 // index
.Leaa_loop:
    cmp x21, x20
    b.ge .Leaa_done
    add x2, x19, x21, lsl #4    // entry = args +
    i*16
    ldr x0, [x2, #0]            // start
    ldr x1, [x2, #8]            // len
    bl _kcc_pp_expand_arg       // → x0 = buf, x1
    = len
    add x2, x19, x21, lsl #4    // recompute (x2
   was clobbered)
    str x0, [x2, #0]
    str x1, [x2, #8]
    add x21, x21, #1
    b .Leaa_loop
.Leaa_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

125b · defined at 121, 123, 125ab, 127, 130, 131 · used in 28f · defines _kcc_pp_expand_all_args 125a, _kcc_pp_expand_arg 136 · uses _kcc_arena_alloc 58a, _kcc_pp_ctx_depth 116, _kcc_pp_has_pending 116, _kcc_pp_next 108, _kcc_pp_prev_line 116, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

17.12.2. Parameter Substitution

For function-like macros, parameter names appearing in the replacement body must be replaced with the corresponding argument text collected from the call site. `_kcc_pp_substitute` walks the body character by character; when it encounters an identifier, it compares it against each parameter name using `_kcc_memcmp`. On a match, the argument text is copied to the output buffer instead of the parameter name. Non-identifier characters and unmatched identifiers are copied verbatim.

A string literal or character constant is lexically a *single* preprocessing token, so a parameter name that happens to appear inside quotes is not a parameter at all and must never be replaced (§6.10.3.1). The walker therefore recognises a leading `"` or `'` and copies the entire literal—through its matching close delimiter, honouring backslash escapes—straight to the output before the identifier logic can see it. Without this, a body such as `puts("value x here")` would wrongly expand `x`, and an inline-asm template `"...%[ptr]..."` whose operand-name letters collide with a macro parameter would be silently rewritten.

The `##` token-paste operator (§6.10.3.3) is handled in the same pass. Pasting two tokens is, at the textual level, simply making them *adjacent*: when the concatenated output is rescanned the lexer reads `foo` immediately followed by `bar` as the single token `foobar`. So on seeing `##` the walker trims any whitespace it has already emitted for the left operand, skips the operator and any whitespace before the right operand, and continues—the next token, be it a substituted argument or a literal, lands flush against the left one. Operands are taken from the substituted argument text; because kcc's arguments are already prescan-expanded and a paste operand is normally a bare identifier or number, this matches the standard in every case a real header exercises. One GNU special case is handled first: in `, ## __VA_ARGS__`, when the right operand is a parameter whose argument is empty, the paste deletes the operand *and* a preceding comma (peeking the operand identifier and consulting its argument length before the ordinary trim). This is what makes the kernel's `static_assert(expr, ...)` macro—and hence the `<linux/bitops.h>` assertions—expand correctly. A lone `#` is the stringisation operator, handled just below.

⟨ *PP: expand* 127 ⟩ +≡

```
// -- _kcc_pp_substitute ---
```

```
// Substitutes parameters in macro body with argument text.
// x0=body_ptr, x1=body_len, x2=params, x3=nparam, x4=args, x5=nargs
// Returns x0=result_ptr, x1=result_len (arena-allocated)
.p2align 2
_kcc_pp_substitute:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    stp x27, x28, [sp, #80]

    mov x19, x0                 // body ptr
    mov x20, x1                 // body len
    mov x21, x2                 // params array
    mov x22, x3                 // param count
    mov x23, x4                 // args array
    mov x24, x5                 // arg count

    // Allocate the output buffer sized to the inputs rather than a fixed 4 KB:
    // a macro that substitutes several large arguments (e.g. the kernel's
    // `pr_alert(fmt, data_race(x), data_race(y), ...)`, whose arguments are big
    // _Generic expansions) can otherwise overflow and truncate mid-token.  The
    // bound is body length + 4x the total argument length (covering a parameter
    // repeated a few times) + slack.  (The C preprocessor port grows on demand;
    // this reference binary never preprocesses such macros, so the generous
    // input-sized bound suffices.)
    mov x9, #0                  // k
    mov x10, x20                // total = body len
.Lsub_cap_loop:
    cmp x9, x24
    b.hs .Lsub_cap_done
    lsl x11, x9, #4
    add x11, x23, x11           // &args[k]
    ldr x12, [x11, #8]          // args[k].len
    add x10, x10, x12, lsl #2 // total += 4 * len
    add x9, x9, #1
    b .Lsub_cap_loop
.Lsub_cap_done:
    add x0, x10, #4096          // + slack
    bl _kcc_arena_alloc
    mov x25, x0                 // x25 = output buffer
    mov x26, #0                  // x26 = output position

    // Walk through body text
    mov x27, #0                  // x27 = body position
.Lsub_loop:
    cmp x27, x20
    b.hs .Lsub_done

    // Check for identifier start
    add x0, x19, x27
    ldrb w0, [x0]

    // A string literal or character constant is a single token whose contents
    // are never subject to parameter replacement (\stdref{6.10.3.1}); copy it
    // through verbatim so a parameter name inside quotes---`x` in
    // "value x here", or `%[ptr]` in an inline-asm template---is not rewritten.
    cmp w0, #0x22               // "
    b.eq .Lsub_qlit
    cmp w0, #0x27               // '
    b.eq .Lsub_qlit

    // Token paste: a `##` in the body concatenates the token to its left with
    // the token to its right by making them textually adjacent in the output,
    // so the rescan lexes them as one token.
    cmp w0, #'#'
    b.ne .Lsub_dispatch
    add x1, x27, #1
    cmp x1, x20
    b.hs .Lsub_dispatch         // trailing '#' → verbatim
    add x2, x19, x1
    ldrb w2, [x2]
    cmp w2, #'#'
    b.eq .Lsub_paste
    b .Lsub_stringize           // single '#' → stringise the next parameter

.Lsub_dispatch:
    // Is it an identifier char? (a-z, A-Z, _)
    cmp w0, #'_'
    b.eq .Lsub_ident
    cmp w0, #'a'
    b.lo .Lsub_not_lower
    cmp w0, #'z'
    b.ls .Lsub_ident
.Lsub_not_lower:
    cmp w0, #'A'
    b.lo .Lsub_copy_char
    cmp w0, #'Z'
    b.ls .Lsub_ident

.Lsub_copy_char:
    // Not an identifier: copy byte
    add x1, x25, x26
    strb w0, [x1]
    add x26, x26, #1
    add x27, x27, #1
    b .Lsub_loop

.Lsub_qlit:
    // Copy a "..." or '...' literal verbatim through its matching close
    // delimiter, honouring backslash escapes.  w1 holds the delimiter.
    mov w1, w0
    add x2, x25, x26            // emit opening delimiter
    strb w0, [x2]
    add x26, x26, #1
    add x27, x27, #1
.Lsub_qlit_body:
    cmp x27, x20
    b.hs .Lsub_loop             // unterminated at body end: stop, resume loop
    add x2, x19, x27
    ldrb w0, [x2]
    add x2, x25, x26            // emit this byte
    strb w0, [x2]
    add x26, x26, #1
    add x27, x27, #1
    cmp w0, #0x5c               // backslash: next byte is escaped, copy verbatim
    b.ne .Lsub_qlit_notesc
    cmp x27, x20
    b.hs .Lsub_loop
    add x2, x19, x27
```

```
    ldrb w0, [x2]
    add x2, x25, x26
    strb w0, [x2]
    add x26, x26, #1
    add x27, x27, #1
    b .Lsub_qlit_body
.Lsub_qlit_notesc:
    cmp w0, w1                      // matching close
    delimiter ends the literal
    b.eq .Lsub_loop
    b .Lsub_qlit_body

.Lsub_stringize:
    // `# param` → a string literal of the
   argument's spelling.  Consume the
    // `#` and any whitespace, extract the
   parameter name, and emit its argument
    // quoted (via _kcc_pp_emit_string).  A `#`
   not followed by a parameter is
    // copied verbatim.
    add x27, x27, #1                // skip '#'
.Lsub_str_ws:
    cmp x27, x20
    b.hs .Lsub_str_hash
    add x0, x19, x27
    ldrb w0, [x0]
    cmp w0, #0x20
    b.eq .Lsub_str_ws_adv
    cmp w0, #0x09
    b.ne .Lsub_str_id
.Lsub_str_ws_adv:
    add x27, x27, #1
    b .Lsub_str_ws
.Lsub_str_id:
    mov x28, x27                    // identifier
   start
.Lsub_str_id_cont:
    add x27, x27, #1
    cmp x27, x20
    b.hs .Lsub_str_id_end
    add x0, x19, x27
    ldrb w0, [x0]
    cmp w0, #'_'
    b.eq .Lsub_str_id_cont
    cmp w0, #'a'
    b.lo .Lsub_str_id_nl
    cmp w0, #'z'
    b.ls .Lsub_str_id_cont
.Lsub_str_id_nl:
    cmp w0, #'A'
    b.lo .Lsub_str_id_nu
    cmp w0, #'Z'
    b.ls .Lsub_str_id_cont
.Lsub_str_id_nu:
    cmp w0, #'0'
    b.lo .Lsub_str_id_end
    cmp w0, #'9'
    b.ls .Lsub_str_id_cont
.Lsub_str_id_end:
    // Match identifier body[x28..x27) against the
    parameter names.
    add x0, x19, x28
    sub x1, x27, x28
    mov x2, #0
.Lsub_str_pcheck:
    cmp x2, x22
    b.hs .Lsub_str_notparam
    mov x3, #PP_PARAM_SIZE
    mul x4, x2, x3
    add x4, x21, x4
    ldr x5, [x4, #PP_PARAM_NAME]
    ldr x6, [x4, #PP_PARAM_LEN]
    cmp x1, x6
    b.ne .Lsub_str_pnext
    stp x0, x1, [sp, #-16]!
    mov x7, x2
    mov x2, x1
    mov x1, x5
    bl _kcc_memcmp
    ldp x8, x1, [sp], #16
    mov x2, x7
    cbz x0, .Lsub_str_found
    mov x0, x8
.Lsub_str_pnext:
    add x2, x2, #1
    b .Lsub_str_pcheck
.Lsub_str_found:
    // Emit the argument as a quoted string
   literal.
    lsl x3, x2, #4
    add x3, x23, x3
    ldr x0, [x3, #0]                // arg text
    ldr x1, [x3, #8]                // arg len
    add x2, x25, x26                // dest
    bl _kcc_pp_emit_string          // → x0 = bytes
   written
    add x26, x26, x0
    b .Lsub_loop
.Lsub_str_notparam:
    // '#' followed by a non-parameter identifier:
    emit '#' then the identifier.
    add x1, x25, x26
    mov w0, #'#'
    strb w0, [x1]
    add x26, x26, #1
    mov x3, #0
.Lsub_str_np_copy:
    sub x1, x27, x28
    cmp x3, x1
    b.hs .Lsub_loop
    add x4, x19, x28
    ldrb w5, [x4, x3]
    add x4, x25, x26
    strb w5, [x4]
    add x26, x26, #1
    add x3, x3, #1
    b .Lsub_str_np_copy
.Lsub_str_hash:
    // Trailing '#': emit verbatim.
    add x1, x25, x26
    mov w0, #'#'
    strb w0, [x1]
    add x26, x26, #1
    b .Lsub_loop

.Lsub_paste:
    // GNU comma elision `, ## PARAM`: if the ##
   right operand is a parameter
    // whose argument is empty, the ## deletes it
   and a preceding comma (the
    // kernel's `static_assert(expr, ...)` macro
   relies on this).  Peek the
    // operand identifier; if it names a parameter
    with an empty argument, drop
    // the comma and consume both the ## and the
   operand.
    add x9, x27, #2                 // past the ##
.Lsub_ce_ws:
    cmp x9, x20
    b.hs .Lsub_paste_trim
    add x0, x19, x9
    ldrb w0, [x0]
    cmp w0, #0x20
    b.eq .Lsub_ce_ws_adv
    cmp w0, #0x09
    b.ne .Lsub_ce_id0
.Lsub_ce_ws_adv:
    add x9, x9, #1
    b .Lsub_ce_ws
.Lsub_ce_id0:
    mov x10, x9                     // operand
```

```
    identifier start
.Lsub_ce_idc:
    cmp x9, x20
    b.hs .Lsub_ce_idend
    add x0, x19, x9
    ldrb w0, [x0]
    cmp w0, #'_'
    b.eq .Lsub_ce_idadv
    cmp w0, #'0'
    b.lo .Lsub_ce_idend
    cmp w0, #'9'
    b.ls .Lsub_ce_idadv
    cmp w0, #'a'
    b.lo .Lsub_ce_idup
    cmp w0, #'z'
    b.ls .Lsub_ce_idadv
.Lsub_ce_idup:
    cmp w0, #'A'
    b.lo .Lsub_ce_idend
    cmp w0, #'Z'
    b.hi .Lsub_ce_idend
.Lsub_ce_idadv:
    add x9, x9, #1
    b .Lsub_ce_idc
.Lsub_ce_idend:
    sub x11, x9, x10            // operand
    identifier length
    cbz x11, .Lsub_paste_trim
    mov x12, #0                 // param index k
.Lsub_ce_match:
    cmp x12, x22
    b.hs .Lsub_paste_trim       // no parameter
    matched → normal paste
    mov x13, #PP_PARAM_SIZE
    mul x13, x12, x13
    add x13, x21, x13           // &params[k]
    ldr x14, [x13, #PP_PARAM_LEN]
    cmp x14, x11
    b.ne .Lsub_ce_next
    stp x10, x11, [sp, #-16]!
    stp x12, x13, [sp, #-16]!
    add x0, x19, x10
    ldr x1, [x13, #PP_PARAM_NAME]
    mov x2, x11
    bl _kcc_memcmp
    ldp x12, x13, [sp], #16
    ldp x10, x11, [sp], #16
    cbnz x0, .Lsub_ce_next
    // Parameter k matched.  Is its argument empty
    ? (args[k].len at offset 8)
    lsl x13, x12, #4
    add x13, x23, x13
    ldr x14, [x13, #8]
    cbnz x14, .Lsub_paste_trim  // non-empty
    operand → normal paste
    // Empty operand: trim trailing whitespace,
    then drop a trailing comma.
.Lsub_ce_trim:
    cbz x26, .Lsub_ce_done
    sub x1, x26, #1
    add x3, x25, x1
    ldrb w4, [x3]
    cmp w4, #0x20
    b.eq .Lsub_ce_trim_adv
    cmp w4, #0x09
    b.ne .Lsub_ce_comma
.Lsub_ce_trim_adv:
    mov x26, x1
    b .Lsub_ce_trim
.Lsub_ce_comma:
    sub x1, x26, #1
    add x3, x25, x1
    ldrb w4, [x3]
    cmp w4, #','
    b.ne .Lsub_ce_done
    mov x26, x1
.Lsub_ce_done:
    mov x27, x9                 // consume the ##
    and the empty operand
    b .Lsub_loop
.Lsub_ce_next:
    add x12, x12, #1
    b .Lsub_ce_match

    // Drop any whitespace already emitted for the
    left operand, so it abuts.
.Lsub_paste_trim:
    cbz x26, .Lsub_paste_skip
    sub x1, x26, #1
    add x3, x25, x1
    ldrb w4, [x3]
    cmp w4, #0x20               // space
    b.eq .Lsub_paste_drop
    cmp w4, #0x09               // tab
    b.ne .Lsub_paste_skip
.Lsub_paste_drop:
    mov x26, x1
    b .Lsub_paste_trim
.Lsub_paste_skip:
    add x27, x27, #2            // skip the ##
    // Skip whitespace before the right operand so
    it, too, abuts.
.Lsub_paste_ws:
    cmp x27, x20
    b.hs .Lsub_loop
    add x0, x19, x27
    ldrb w0, [x0]
    cmp w0, #0x20
    b.eq .Lsub_paste_ws_adv
    cmp w0, #0x09
    b.ne .Lsub_loop
.Lsub_paste_ws_adv:
    add x27, x27, #1
    b .Lsub_paste_ws
```

□

127 · defined at 121, 123, 125ab, 127, 130, 131 · used in 28f · uses _kcc_arena_alloc 58a, _kcc_pp_emit_string 131

Identifier extraction and parameter matching. When the body walker encounters the start of an identifier (a letter or underscore), it enters the extraction path. The scanner advances through subsequent identifier characters to delimit the full name, then compares it against every parameter in the macro's parameter list using _kcc_memcmp. A match causes the corresponding argument text to be copied into the output buffer; an unmatched identifier is copied verbatim.

⟨ PP: expand 130 ⟩ +≡

```
.Lsub_ident:
    // Found identifier start. Extract the full
    identifier
    mov x28, x27                // x28 = ident
    start position
.Lsub_ident_cont:
    add x27, x27, #1
    cmp x27, x20
    b.hs .Lsub_ident_end
    add x0, x19, x27
    ldrb w0, [x0]
    cmp w0, #'_'
    b.eq .Lsub_ident_cont
    cmp w0, #'a'
    b.lo .Lsub_id_not_lower
    cmp w0, #'z'
    b.ls .Lsub_ident_cont
```

```
.Lsub_id_not_lower:
    cmp w0, #'A'
    b.lo .Lsub_id_not_upper
    cmp w0, #'Z'
    b.ls .Lsub_ident_cont
.Lsub_id_not_upper:
    cmp w0, #'0'
    b.lo .Lsub_ident_end
    cmp w0, #'9'
    b.ls .Lsub_ident_cont

.Lsub_ident_end:
    // Identifier is body[x28..x27)
    add  x0, x19, x28          // ident text ptr
    sub  x1, x27, x28          // ident len

    // Check against each parameter name
    mov  x2, #0                // param index
.Lsub_param_check:
    cmp  x2, x22
    b.hs .Lsub_not_param       // no match

    // params[x2]
    mov  x3, #PP_PARAM_SIZE
    mul  x4, x2, x3
    add  x4, x21, x4
    ldr  x5, [x4, #PP_PARAM_NAME]
    ldr  x6, [x4, #PP_PARAM_LEN]

    // Compare lengths first
    cmp  x1, x6
    b.ne .Lsub_param_next

    // Compare text
    // Save registers we need across memcmp
    stp  x0, x1, [sp, #-16]!  // push ident ptr, len
    mov  x7, x2                // save param index
    mov  x2, x1                // len for memcmp
    mov  x1, x5                // param name
    // x0 already = ident text
    bl _kcc_memcmp
    ldp  x8, x1, [sp], #16     // restore ident ptr, len
    mov  x2, x7                // restore param index

    cbz  x0, .Lsub_found_param
    mov  x0, x8                // restore ident ptr
    b .Lsub_param_next

.Lsub_param_next:
    add  x2, x2, #1
    b .Lsub_param_check

.Lsub_not_param:
    // Not a parameter: copy identifier text as-is
    sub  x1, x27, x28          // ident len
    mov  x2, #0
.Lsub_copy_ident:
    cmp  x2, x1
    b.hs .Lsub_loop
    add  x3, x19, x28
    ldrb w4, [x3, x2]
    add  x3, x25, x26
    strb w4, [x3]
    add  x26, x26, #1
    add  x2, x2, #1
    b .Lsub_copy_ident

.Lsub_found_param:
    // Substitute: copy argument text instead
    // x2 = param index (same as arg index)
    lsl  x3, x2, #4            // * 16
    add  x3, x23, x3
    ldr  x4, [x3, #0]          // arg text ptr
    ldr  x5, [x3, #8]          // arg text len

    mov  x2, #0
.Lsub_copy_arg:
    cmp  x2, x5
    b.hs .Lsub_loop
    ldrb w6, [x4, x2]
    add  x3, x25, x26
    strb w6, [x3]
    add  x26, x26, #1
    add  x2, x2, #1
    b .Lsub_copy_arg

.Lsub_done:
    mov  x0, x25               // result ptr
    mov  x1, x26               // result len

    ldp x27, x28, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

130 · defined at 121, 123, 125ab, 127, 130, 131 · used in 28f

Stringising an argument. The # operator (§6.10.3.2) turns an argument into a string literal. `_kcc_pp_emit_string` writes the quoted form: an opening ", the argument with leading and trailing whitespace stripped and every internal run of whitespace collapsed to a single space, each embedded " or \ preceded by a backslash, then a closing ". It operates on the substituted argument text; since kcc's arguments are already prescan-expanded this differs from the strict standard only when a # operand is itself a macro, which real code stringising a type or token name never does.

⟨ *PP: expand* 131 ⟩ +≡

```
// -- _kcc_pp_emit_string ---
// (const char *src, uint64_t len, char *dest) → x0 = bytes written.
// Emits a "..." string literal of src with whitespace collapsed and " / \ escaped.
.p2align 2
_kcc_pp_emit_string:
    mov x3, x2                 // dest cursor
    mov w4, #0x22              // opening quote
    strb w4, [x3], #1
    mov x5, #0                 // index
.Les_lead:
    cmp x5, x1
    b.hs .Les_close
    ldrb w6, [x0, x5]
    cmp w6, #0x20
    b.eq .Les_lead_adv
    cmp w6, #0x09
    b.ne .Les_body
.Les_lead_adv:
    add x5, x5, #1
    b .Les_lead
.Les_body:
    mov x7, #0                 // pending-space flag (collapses whitespace runs)
.Les_loop:
```

```
    cmp x5, x1
    b.hs .Les_close
    ldrb w6, [x0, x5]
    cmp w6, #0x20
    b.eq .Les_ws
    cmp w6, #0x09
    b.eq .Les_ws
    cbz x7, .Les_noflush
    mov w8, #0x20
    strb w8, [x3], #1           // flush one
   collapsed space
    mov x7, #0
.Les_noflush:
    cmp w6, #0x22               // escape " and \
    b.eq .Les_esc
    cmp w6, #0x5C
    b.eq .Les_esc
    strb w6, [x3], #1
    b .Les_adv
.Les_esc:
    mov w8, #0x5C
    strb w8, [x3], #1
    strb w6, [x3], #1
    b .Les_adv
.Les_ws:
    mov x7, #1                  // remember;
   trailing whitespace is never flushed
.Les_adv:
    add x5, x5, #1
    b .Les_loop
.Les_close:
    mov w4, #0x22               // closing quote
    strb w4, [x3], #1
    sub x0, x3, x2              // bytes written
    ret
```

□

131 · defined at 121, 123, 125ab, 127, 130, 131 · used in 28f · defines _kcc_pp_emit_string 127

17.13. Predefined Macros

ISO C17 requires several predefined macros (§6.10.8). We define __STDC__ (1), __STDC_VERSION__ (201710L), and __STDC_HOSTED__ (1). __LINE__ is also registered, but with a placeholder body: because its value is the line of each *use*, _kcc_pp_try_expand recognises it and substitutes the current _kcc_lex_line formatted as a decimal token, rather than expanding a fixed body. Registering it still lets #ifdef __LINE__ see it and prevents #undef. __FILE__ works the same way: recognised in _kcc_pp_try_expand, it substitutes the current source path (_kcc_src_path, which the #include machinery saves and restores per file, so it names the header when used inside one) wrapped in double quotes as a fresh string-literal token. The kernel reaches __FILE__ through WARN()/BUG()/pr_* in roughly half of all source files. __COUNTER__ is dynamic in the same family, but its value depends on nothing in the source: each expansion yields the next integer of a monotonically increasing counter (_kcc_pp_counter, starting at 0), which _kcc_pp_try_expand formats through the very decimal path __LINE__ uses and then increments. The kernel builds unique identifiers with it (__UNIQUE_ID, lockdep key names).

Four further macros advertise the *conditional features* (§6.10.8.3) kcc does not implement. Atomics, threads, complex arithmetic, and variable-length arrays are the only optional parts of the language; an implementation that omits one must say so by defining the matching __STDC_NO_* macro, so that a program can test for the feature with #ifdef and choose a fallback rather than fail at a missing header or type. Declaring the absence is what makes the omission *conforming* rather than a gap: kcc defines __STDC_NO_ATOMICS__, __STDC_NO_THREADS__, __STDC_NO_COMPLEX__, and __STDC_NO_VLA__, each to 1. With them the compiler is fully conforming in the freestanding sense without ever supplying <stdatomic.h>, <threads.h>, <complex.h>, or <tgmath.h>.

Five more announce the target's *byte order*, using gcc and clang's own spelling: __ORDER_LITTLE_ENDIAN__ (1234), __ORDER_BIG_ENDIAN__ (4321), __ORDER_PDP_ENDIAN__ (3412), __BYTE_ORDER__ (set to the little-endian value, since both Apple ARM64 and Linux x86-64 are little-endian), and the older bare __LITTLE_ENDIAN__ flag. Portable code selects its endian-swap routines with #if __BYTE_ORDER__ == __ORDER_LITTLE_ENDIAN__ and #errors when no order name resolves—*doomgeneric*'s i_swap.h is exactly this shape. All three order names *must* be defined together: an undefined __BYTE_ORDER__ is 0 inside a #if, which would compare equal to a likewise-undefined __ORDER_LITTLE_ENDIAN__ and silently pick the wrong branch.

⟨ *PP: predefined* 132 ⟩ ≡

```
.text
// -- _kcc_pp_predefine ---
// Registers predefined macros in the macro table.
.p2align 2
_kcc_pp_predefine:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // __STDC__ = 1
    adrp x0, _kcc_pp_pre_stdc@PAGE
    add  x0, x0, _kcc_pp_pre_stdc@PAGEOFF
    mov  x1, #8                 // len("__STDC__")
    adrp x2, _kcc_pp_pre_one@PAGE
    add  x2, x2, _kcc_pp_pre_one@PAGEOFF
    mov  x3, #1                 // len("1")
    bl _kcc_pp_def_simple

    // __STDC_VERSION__ = 201710L
    adrp x0, _kcc_pp_pre_stdc_ver@PAGE
    add  x0, x0, _kcc_pp_pre_stdc_ver@PAGEOFF
    mov  x1, #16                // len("
  __STDC_VERSION__")
    adrp x2, _kcc_pp_pre_c17val@PAGE
    add  x2, x2, _kcc_pp_pre_c17val@PAGEOFF
    mov  x3, #7                 // len("201710L")
    bl _kcc_pp_def_simple

    // __STDC_HOSTED__ = 1
    adrp x0, _kcc_pp_pre_hosted@PAGE
    add  x0, x0, _kcc_pp_pre_hosted@PAGEOFF
    mov  x1, #15                // len("
  __STDC_HOSTED__")
    adrp x2, _kcc_pp_pre_one@PAGE
    add  x2, x2, _kcc_pp_pre_one@PAGEOFF
    mov  x3, #1
    bl _kcc_pp_def_simple

    // Conditional-feature macros (\stdref
  {6.10.8.3}): kcc supports none of
    // atomics, threads, complex arithmetic, or
  variable-length arrays, so it
```

```
 // declares each absent (= 1) rather than
leaving it undefined.
 adrp x0, _kcc_pp_pre_no_atomics@PAGE
 add  x0, x0, _kcc_pp_pre_no_atomics@PAGEOFF
 mov  x1, #19                // len("
__STDC_NO_ATOMICS__")
 adrp x2, _kcc_pp_pre_one@PAGE
 add  x2, x2, _kcc_pp_pre_one@PAGEOFF
 mov  x3, #1
 bl _kcc_pp_def_simple

 adrp x0, _kcc_pp_pre_no_threads@PAGE
 add  x0, x0, _kcc_pp_pre_no_threads@PAGEOFF
 mov  x1, #19                // len("
__STDC_NO_THREADS__")
 adrp x2, _kcc_pp_pre_one@PAGE
 add  x2, x2, _kcc_pp_pre_one@PAGEOFF
 mov  x3, #1
 bl _kcc_pp_def_simple

 adrp x0, _kcc_pp_pre_no_complex@PAGE
 add  x0, x0, _kcc_pp_pre_no_complex@PAGEOFF
 mov  x1, #19                // len("
__STDC_NO_COMPLEX__")
 adrp x2, _kcc_pp_pre_one@PAGE
 add  x2, x2, _kcc_pp_pre_one@PAGEOFF
 mov  x3, #1
 bl _kcc_pp_def_simple

 adrp x0, _kcc_pp_pre_no_vla@PAGE
 add  x0, x0, _kcc_pp_pre_no_vla@PAGEOFF
 mov  x1, #15                // len("
__STDC_NO_VLA__")
 adrp x2, _kcc_pp_pre_one@PAGE
 add  x2, x2, _kcc_pp_pre_one@PAGEOFF
 mov  x3, #1
 bl _kcc_pp_def_simple

 // Data-model macros.  Apple ARM64 is LP64---
long and pointer are 64-bit---so
 // __LP64__/_LP64 are defined; portable code
selects type widths from them.
 // Arch/OS identity macros (__APPLE__,
__aarch64__) are deliberately NOT
 // defined: they gate platform-specific
headers, intrinsics, and inline asm
 // that kcc does not support, so announcing
them would break otherwise
 // portable code.  Data-model macros only
steer type choices kcc handles.
 adrp x0, _kcc_pp_pre_lp64@PAGE
 add  x0, x0, _kcc_pp_pre_lp64@PAGEOFF
 mov  x1, #8                 // len("__LP64__")
 adrp x2, _kcc_pp_pre_one@PAGE
 add  x2, x2, _kcc_pp_pre_one@PAGEOFF
 mov  x3, #1
 bl _kcc_pp_def_simple

 adrp x0, _kcc_pp_pre_lp64b@PAGE
 add  x0, x0, _kcc_pp_pre_lp64b@PAGEOFF
 mov  x1, #5                 // len("_LP64")
 adrp x2, _kcc_pp_pre_one@PAGE
 add  x2, x2, _kcc_pp_pre_one@PAGEOFF
 mov  x3, #1
 bl _kcc_pp_def_simple

 // __LINE__ - registered so `#ifdef __LINE__`
sees it and it cannot be
 // #undef'd; the body here is a placeholder,
since _kcc_pp_try_expand
 // substitutes the current line number
dynamically at each use (§6.10.8.1).
 adrp x0, _kcc_pp_pre_linem@PAGE
 add  x0, x0, _kcc_pp_pre_linem@PAGEOFF
 mov  x1, #8                 // len("__LINE__")
 adrp x2, _kcc_pp_pre_one@PAGE
 add  x2, x2, _kcc_pp_pre_one@PAGEOFF
 mov  x3, #1
 bl _kcc_pp_def_simple

 // __FILE__ - like __LINE__, registered so `#
ifdef __FILE__` sees it and it
 // cannot be #undef'd; _kcc_pp_try_expand
substitutes the current source path
 // as a string literal dynamically at each use
 (§6.10.8.1).
 adrp x0, _kcc_pp_pre_filem@PAGE
 add  x0, x0, _kcc_pp_pre_filem@PAGEOFF
 mov  x1, #8                 // len("__FILE__")
 adrp x2, _kcc_pp_pre_one@PAGE
 add  x2, x2, _kcc_pp_pre_one@PAGEOFF
 mov  x3, #1
 bl _kcc_pp_def_simple

 // __COUNTER__ - like __LINE__, registered so
`#ifdef __COUNTER__` sees it and
 // it cannot be #undef'd; _kcc_pp_try_expand
substitutes a fresh incrementing
 // integer at each use.  The kernel's `unique
id` idioms (e.g. lock names,
 // __UNIQUE_ID) rely on it.
 adrp x0, _kcc_pp_pre_counterm@PAGE
 add  x0, x0, _kcc_pp_pre_counterm@PAGEOFF
 mov  x1, #11                // len("__COUNTER__
")
 adrp x2, _kcc_pp_pre_one@PAGE
 add  x2, x2, _kcc_pp_pre_one@PAGEOFF
 mov  x3, #1
 bl _kcc_pp_def_simple

 // Byte-order macros, exactly as gcc/clang
publish them.  Real code selects
 // its endian-swap routines with `#if
__BYTE_ORDER__ == __ORDER_LITTLE_ENDIAN__'
 // and `#error's when none of the three order
names resolve (doomgeneric's
 // i_swap.h does this); an undefined `
__BYTE_ORDER__' is 0 in a `#if', which
 // would compare equal to a likewise-undefined
 `__ORDER_*' and take the wrong
 // branch, so all must be defined.  Both kcc
targets are little-endian.
 adrp x0, _kcc_pp_pre_ord_le@PAGE
 add  x0, x0, _kcc_pp_pre_ord_le@PAGEOFF
 mov  x1, #23                // len("
__ORDER_LITTLE_ENDIAN__")
 adrp x2, _kcc_pp_pre_v1234@PAGE
 add  x2, x2, _kcc_pp_pre_v1234@PAGEOFF
 mov  x3, #4
 bl _kcc_pp_def_simple

 adrp x0, _kcc_pp_pre_ord_be@PAGE
 add  x0, x0, _kcc_pp_pre_ord_be@PAGEOFF
 mov  x1, #20                // len("
__ORDER_BIG_ENDIAN__")
 adrp x2, _kcc_pp_pre_v4321@PAGE
 add  x2, x2, _kcc_pp_pre_v4321@PAGEOFF
 mov  x3, #4
 bl _kcc_pp_def_simple

 adrp x0, _kcc_pp_pre_ord_pdp@PAGE
 add  x0, x0, _kcc_pp_pre_ord_pdp@PAGEOFF
 mov  x1, #20                // len("
__ORDER_PDP_ENDIAN__")
 adrp x2, _kcc_pp_pre_v3412@PAGE
 add  x2, x2, _kcc_pp_pre_v3412@PAGEOFF
 mov  x3, #4
 bl _kcc_pp_def_simple

 adrp x0, _kcc_pp_pre_byteord@PAGE
```

```
    add  x0, x0, _kcc_pp_pre_byteord@PAGEOFF
    mov  x1, #14                // len("__BYTE_ORDER__")
    adrp x2, _kcc_pp_pre_v1234@PAGE
    add  x2, x2, _kcc_pp_pre_v1234@PAGEOFF
    mov  x3, #4
    bl _kcc_pp_def_simple

    adrp x0, _kcc_pp_pre_le@PAGE
    add  x0, x0, _kcc_pp_pre_le@PAGEOFF
    mov  x1, #17                // len("__LITTLE_ENDIAN__")
    adrp x2, _kcc_pp_pre_one@PAGE
    add  x2, x2, _kcc_pp_pre_one@PAGEOFF
    mov  x3, #1
    bl _kcc_pp_def_simple

    // Replay any command-line -D definitions on top of the built-ins.  The
    // macro table is rebuilt for every translation unit, so this runs each time
    // the preprocessor is (re)initialised, keeping -D macros defined per source.
    bl _kcc_pp_apply_cmdline_defines

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- _kcc_pp_def_simple ---
// Helper: define a simple object-like macro.
// x0=name, x1=namelen, x2=body, x3=bodylen
.p2align 2
_kcc_pp_def_simple:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0                 // name
    mov x20, x1                 // namelen
    mov x21, x2                 // body
    mov x22, x3                 // bodylen

    // Allocate macro definition
    mov x0, #PP_MACRO_SIZE
    bl _kcc_arena_alloc
    mov x23, x0

    str x19, [x23, #PP_MACRO_NAME]
    str x20, [x23, #PP_MACRO_NAMELEN]
    mov x0, #PP_FLAG_BUILTIN
    str x0, [x23, #PP_MACRO_FLAGS]
    str xzr, [x23, #PP_MACRO_NPARAMS]
    str x21, [x23, #PP_MACRO_BODY]
    str x22, [x23, #PP_MACRO_BODYLEN]
    str xzr, [x23, #PP_MACRO_PARAMS]

    // Store in hash table
    adrp x0, _kcc_pp_macros@PAGE
    add  x0, x0, _kcc_pp_macros@PAGEOFF
    ldr  x0, [x0]
    mov  x1, x19
    mov  x2, x20
    mov  x3, x23
    bl _kcc_ht_put

    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// Command-line -D definitions.  The driver records each -D argument's payload
// ("NAME" or "NAME=VALUE") here during argument scanning; the list is replayed
// into the macro table by _kcc_pp_apply_cmdline_defines whenever the
// preprocessor is initialised, so the macros are present for every source.
.data
.p2align 3
_kcc_cmdline_defs:  .space 512    // up to 64 payload pointers
_kcc_cmdline_ndefs: .quad 0
.text

// -- _kcc_pp_add_cmdline_define ---
// void _kcc_pp_add_cmdline_define(const char *payload)
//   Records a -D payload ("NAME" or "NAME=VALUE"); ignored once full.
.globl _kcc_pp_add_cmdline_define
.p2align 2
_kcc_pp_add_cmdline_define:
    adrp x1, _kcc_cmdline_ndefs@PAGE
    add  x1, x1, _kcc_cmdline_ndefs@PAGEOFF
    ldr  x2, [x1]
    cmp  x2, #64
    b.hs .Lacd_done
    adrp x3, _kcc_cmdline_defs@PAGE
    add  x3, x3, _kcc_cmdline_defs@PAGEOFF
    str  x0, [x3, x2, lsl #3]
    add  x2, x2, #1
    str  x2, [x1]
.Lacd_done:
    ret

// -- _kcc_pp_apply_cmdline_defines ---
// void _kcc_pp_apply_cmdline_defines(void)
//   Defines each recorded -D macro.  "NAME" defines NAME as 1; "NAME=VALUE"
//   splits at the first '=' and defines NAME as VALUE (a bare NAME with no
//   value takes 1, matching cc's convention).  Names and bodies point into the
//   original argv strings and are used by length, so no copy is needed.
.globl _kcc_pp_apply_cmdline_defines
.p2align 2
_kcc_pp_apply_cmdline_defines:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    adrp x0, _kcc_cmdline_ndefs@PAGE
    add  x0, x0, _kcc_cmdline_ndefs@PAGEOFF
    ldr  x20, [x0]                  // count
    mov  x19, #0                    // i
.Lapd_loop:
    cmp  x19, x20
    b.hs .Lapd_done
    adrp x0, _kcc_cmdline_defs@PAGE
    add  x0, x0, _kcc_cmdline_defs@PAGEOFF
    ldr  x21, [x0, x19, lsl #3]     // payload ptr
    // Scan for '=' → x22 = name length (or full length when absent).
    mov  x22, #0
.Lapd_scan:
    ldrb w0, [x21, x22]
    cbz  w0, .Lapd_no_eq
    cmp  w0, #'='
    b.eq .Lapd_have_eq
    add  x22, x22, #1
    b .Lapd_scan
.Lapd_have_eq:
```

```
    add  x2, x21, x22
    add  x2, x2, #1                // value =
   payload + namelen + 1
    mov  x0, x2
    bl _kcc_strlen                  // x0 = value
   length
    mov  x3, x0
    add  x2, x21, x22
    add  x2, x2, #1                // value ptr (
   recompute; strlen clobbers)
    mov  x0, x21                    // name
    mov  x1, x22                    // namelen
    bl _kcc_pp_def_simple
    b .Lapd_next
.Lapd_no_eq:
    // Bare NAME: define as 1.
    mov  x0, x21                    // name
    mov  x1, x22                    // namelen
    adrp x2, _kcc_pp_pre_one@PAGE
    add  x2, x2, _kcc_pp_pre_one@PAGEOFF
    mov  x3, #1
    bl _kcc_pp_def_simple
.Lapd_next:
    add  x19, x19, #1
    b .Lapd_loop
.Lapd_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

132 · used in 28f · defines _kcc_pp_add_cmdline_define 606b, _kcc_pp_apply_cmdline_defines never used · uses _kcc_arena_alloc 58a, _kcc_ht_put 63, _kcc_pp_macros 116, _kcc_pp_pre_c17val 116, _kcc_pp_pre_hosted 116, _kcc_pp_pre_one 116, _kcc_pp_pre_stdc 116, _kcc_pp_pre_stdc_ver 116, _kcc_pp_try_expand 121

17.14. Conditional Compilation

The conditional directives `#ifdef`, `#ifndef`, `#if`, `#elif`, `#else`, and `#endif` control which sections of the source are processed.

The name following `#ifdef`/`#ifndef` is any identifier *or keyword-class* token. In phase-4 translation no keywords exist yet—they are recognised only later, in the parser—so a header that tests a spelling the lexer happens to tag as a keyword, such as SDL's `#ifndef __inline__` in `begin_code.h`, still names a macro. Because the macro lookup keys on the token's source text, the directive treats the whole identifier-or-keyword range uniformly, exactly as `#define` does when it accepts such a spelling as a macro name.

A conditional stack records the state of each nested group. States are `ACTIVE` (processing tokens), `SKIP` (discarding tokens), and `DONE` (active branch already seen, skip rest).

⟨ *PP: conditional* 135 ⟩ ≡

```
.text
// -- _kcc_pp_do_ifdef ---
// x0 = negate (0 for ifdef, 1 for ifndef)
.p2align 2
_kcc_pp_do_ifdef:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                 // x19 = negate flag

    // Read macro name.  Accept keyword-class
   tokens too: in phase-4 translation
    // no keywords exist yet, so `#ifndef
   __inline__` (SDL's begin_code.h) names a
    // would-be macro whose spelling the lexer
   eagerly tagged as a keyword.  The
    // lookup below keys on the source text, so
   any identifier-or-keyword token
    // resolves correctly.
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_IDENT
    b.eq .Lifdef_name_ok
    sub  x1, x0, #10            // keyword-class
   tokens occupy [10, 60)
    cmp  x1, #50                // 60-10; hs → 
   neither identifier nor keyword
    b.hs .Lifdef_err
.Lifdef_name_ok:

    // Look up in macro table
    adrp x0, _kcc_pp_macros@PAGE
    add  x0, x0, _kcc_pp_macros@PAGEOFF
    ldr  x0, [x0]

    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    ldr  x1, [x1]

    adrp x2, _kcc_tok_len@PAGE
    add  x2, x2, _kcc_tok_len@PAGEOFF
    ldr  x2, [x2]

    bl _kcc_ht_get              // x0 = value (0 if
    not found)

    // Determine condition: ifdef → (value != 0),
   ifndef → (value == 0)
    cmp  x0, #0
    cset x0, ne                 // x0 = 1 if
   defined, 0 if not
    eor  x0, x0, x19            // negate if ifndef

    // Push conditional: ACTIVE if true, SKIP if
   false
    cmp  x0, #0
    mov  x0, #PP_COND_SKIP
    mov  x1, #PP_COND_ACTIVE
    csel x0, x1, x0, ne         // x0 = state

    bl _kcc_pp_cond_push
    bl _kcc_pp_skip_line

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

.Lifdef_err:
    adrp x0, _kcc_pp_dir_line@PAGE
    add  x0, x0, _kcc_pp_dir_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_ifdef_name@PAGE
    add  x2, x2, _kcc_pp_err_ifdef_name@PAGEOFF
    bl _kcc_diag_error
    // Push SKIP so we don't process the body
    mov x0, #PP_COND_SKIP
```

```
    bl _kcc_pp_cond_push
    bl _kcc_pp_skip_line
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

135 · defined at 135, 136, 139ab, 140 · used in 28f · uses _kcc_diag_error 76a, _kcc_ht_get 62, _kcc_lex_next 90b, _kcc_pp_dir_line 116, _kcc_pp_err_ifdef_name 116, _kcc_pp_macros 116, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

The `#if` and `#elif` directives evaluate a constant integer expression to decide whether to process the enclosed source. §6.10.1/4 requires the controlling line to undergo *full macro replacement*—after the `defined` operator is resolved—before it is parsed as an expression. `_kcc_pp_do_if` extracts the rest of the directive line, hands it to `_kcc_pp_prepare_cond` (which expands it, described next), temporarily redirects the lexer to scan the resulting text, calls the recursive-descent evaluator, and pushes an `ACTIVE` or `SKIP` state accordingly. `_kcc_pp_do_elif` follows the same pattern but must also account for whether a previous branch in the same group was already taken—if so, the state becomes `DONE` regardless of the expression's value.

Expanding the controlling line. The recursive-descent evaluator reads raw lexer tokens and looks up each bare identifier as a macro on the fly. That shortcut is correct for an object-like macro or a chain of them, but it cannot bind the parameters of a *function-like* macro: evaluating the body of `L_INTHASBITS(b)` directly leaves `b` unbound (hence 0), so a real header's `#if L_INTHASBITS(SIZE_Bx)` collapses to `((UINT_MAX >> -1) >= 1)`, takes the wrong branch, and silently mis-sizes a bit-field constant. The cure is to obey the standard literally: fully expand the line first, then evaluate a stream that holds only numbers and operators. `_kcc_pp_prepare_cond` does this in two passes. `_kcc_pp_protect_defined` raw-scans the line and rewrites each `defined NAME` / `defined(NAME)` to the literal `1` or `0`—looking the operand up *without* expanding it, as §6.10.1 demands—while copying every other token verbatim. The result is then run through `_kcc_pp_expand_arg`, the same argument-prescan engine the macro expander already uses, which performs a complete macro replacement (function-like invocations included) and returns re-lexable text.

⟨ *PP: conditional* 136 ⟩ +≡

```
// -- _kcc_pp_protect_defined ---
// (const char *src, uint64_t len) -> x0 = buf, x1
    = len
// Raw-scans one #if/#elif controlling line,
   copying every token verbatim
// EXCEPT `defined NAME` / `defined(NAME)`, which
   become the literal 1 or 0.
// The operand is looked up but never expanded
   (6.10.1): expansion happens
// only afterwards, on the rewritten line.  Output
    is at most 2*len bytes
// (each token plus one separating space; `defined
   ` only ever shrinks), so
// the buffer never needs to grow.
.p2align 2
_kcc_pp_protect_defined:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str x27, [sp, #80]
    mov x23, x0                 // src
    mov x24, x1                 // len

    // Save lexer state, then aim it at the
   controlling text.
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x21, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    ldr  x22, [x0]
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x23, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    add  x1, x23, x24
    str  x1, [x0]

    // Output buffer: 2*len + 64 bytes (never
   grows; see header note).
    lsl  x0, x24, #1
    add  x0, x0, #64
    bl _kcc_arena_alloc
    mov  x19, x0                // out_buf
    mov  x20, #0                // out_len

    bl _kcc_lex_next
.Lpd_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lpd_done          // TOK_EOF

    // Is this token the identifier `defined` (
   length 7, exact match)?
    cmp  x0, #TOK_IDENT
    b.ne .Lpd_copy
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #7
    b.ne .Lpd_copy
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_pp_s_defined@PAGE
    add  x1, x1, _kcc_pp_s_defined@PAGEOFF
    mov  x2, #7
    bl _kcc_memcmp
    cbnz x0, .Lpd_copy

    // `defined` seen.  Next token is either `(` (
   parenthesised form) or the
    // operand name itself.  Record which, and
   land on the name token.
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LPAREN
    cset w27, eq                // w27 = 1 if
   parenthesised
    cbz  w27, .Lpd_have_name
```

```
    bl _kcc_lex_next              // step onto the
   name inside the parens
.Lpd_have_name:
    // Look the operand up in the macro table
   WITHOUT expanding it.
    adrp x0, _kcc_pp_macros@PAGE
    add  x0, x0, _kcc_pp_macros@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    ldr  x1, [x1]
    adrp x2, _kcc_tok_len@PAGE
    add  x2, x2, _kcc_tok_len@PAGEOFF
    ldr  x2, [x2]
    bl _kcc_ht_get
    cmp  x0, #0
    cset x25, ne                  // x25 = 1 if
   defined, else 0

    // Advance past the name; for the
   parenthesised form also past the `)`.
    bl _kcc_lex_next
    cbz  w27, .Lpd_emit_bit
    bl _kcc_lex_next              // consume `)`
.Lpd_emit_bit:
    cbz  x20, .Lpd_bit_nosep
    mov  w0, #0x20                // space
   separator
    strb w0, [x19, x20]
    add  x20, x20, #1
.Lpd_bit_nosep:
    add  w1, w25, #0x30           // '0' + bit
    strb w1, [x19, x20]
    add  x20, x20, #1
    b .Lpd_loop

.Lpd_copy:
    // Copy the current token verbatim, space-
   separated from the previous one.
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x25, [x0]                // tok_start
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr  x26, [x0]                // tok_len
    cbz  x20, .Lpd_copy_nosep
    mov  w0, #0x20
    strb w0, [x19, x20]
    add  x20, x20, #1
.Lpd_copy_nosep:
    add  x0, x19, x20
    mov  x1, x25
    mov  x2, x26
    bl _kcc_memcpy
    add  x20, x20, x26
    bl _kcc_lex_next
    b .Lpd_loop

.Lpd_done:
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x21, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    str  x22, [x0]
    mov  x0, x19
    mov  x1, x20
    ldr  x27, [sp, #80]
    ldp  x25, x26, [sp, #64]
    ldp  x23, x24, [sp, #48]
    ldp  x21, x22, [sp, #32]
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #96
    ret

// -- _kcc_pp_prepare_cond ---
// (const char *src, uint64_t len) -> x0 = buf, x1
    = len
// Resolves `defined`, then fully macro-expands
   the controlling line, so the
// evaluator downstream sees only literals and
   operators.
//
// The expansion pass drives _kcc_pp_next, which
   normally suppresses tokens
// while the current conditional branch is being
   skipped.  A `#elif` is
// evaluated with its group's state still SKIP, so
    that suppression would
// silently swallow the whole (correct) expansion
   and make every `#elif`
// read as false.  We therefore zero the
   conditional depth across the
// expansion---the pushed expand context is self-
   contained, so treating it
// as unconditionally active is exactly right---
   and restore it afterwards.
.p2align 2
_kcc_pp_prepare_cond:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    bl _kcc_pp_protect_defined    // (src,len) -> (
   bufA,lenA)
    mov x19, x0
    mov x20, x1
    cbz x20, .Lpc_ret             // nothing to
   expand

    // Suspend conditional skipping for the
   duration of the expansion.
    adrp x21, _kcc_pp_cond_depth@PAGE
    add  x21, x21, _kcc_pp_cond_depth@PAGEOFF
    ldr  x22, [x21]               // saved
   cond_depth
    str  xzr, [x21]

    mov x0, x19
    mov x1, x20
    bl _kcc_pp_expand_arg         // full macro
   replacement -> (bufB,lenB)
    mov x19, x0
    mov x20, x1

    str  x22, [x21]              // restore
   cond_depth
.Lpc_ret:
    mov x0, x19
    mov x1, x20
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- _kcc_pp_do_if ---
// Handles #if with basic expression evaluation.
.p2align 2
_kcc_pp_do_if:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Get rest of line as expression text, then
   fully expand it (resolving
```

```
    // `defined` and every macro, including
   function-like ones) so the
    // evaluator below sees only literals and
   operators.
    bl _kcc_pp_rest_of_line
    bl _kcc_pp_prepare_cond
    mov x19, x0                // x19 = expanded
   expr text
    mov x20, x1                // x20 = expanded
   expr len

    // Save lexer state
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x21, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    ldr  x22, [x0]

    // Point lexer at expression text
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x19, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    add  x1, x19, x20
    str  x1, [x0]

    // Evaluate
    bl _kcc_lex_next           // get first token
    bl _kcc_pp_eval_expr       // x0 = result
    mov x19, x0                // save result

    // Restore lexer state
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x21, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    str  x22, [x0]

    // Push conditional
    cmp  x19, #0
    mov  x0, #PP_COND_SKIP
    mov  x1, #PP_COND_ACTIVE
    csel x0, x1, x0, ne

    bl _kcc_pp_cond_push

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- _kcc_pp_do_line ---
// #line N ["file"] (\stdref{6.10.4}): set the
   presumed line number of the
// following source line to N, which may itself
   come from a macro
// (`#line line`).  We evaluate the operand
   exactly as an #if expression --
// macro-expanded, then parsed as an integer --
   and store the difference from
// the physical line in _kcc_line_delta, which
   __LINE__ then adds.  The optional
// file-name string is ignored; __FILE__ tracks
   _kcc_src_path directly.
.p2align 2
.globl _kcc_pp_do_line
_kcc_pp_do_line:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    // Physical line of this #line directive,
   captured before reading on.
    adrp x0, _kcc_lex_line@PAGE
    add  x0, x0, _kcc_lex_line@PAGEOFF
    ldr  x23, [x0]

    // Expand and evaluate the operand as an
   integer, reusing the #if path.
    bl _kcc_pp_rest_of_line
    bl _kcc_pp_prepare_cond
    mov x19, x0                    // expanded text
    mov x20, x1                    // length

    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x21, [x0]                 // save cursor
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    ldr  x22, [x0]                 // save end

    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x19, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    add  x1, x19, x20
    str  x1, [x0]

    bl _kcc_lex_next
    bl _kcc_pp_eval_expr           // x0 = N (the
   presumed line)
    mov x19, x0

    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x21, [x0]                 // restore
   cursor
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    str  x22, [x0]                 // restore end

    // delta so the NEXT physical line (x23 + 1)
   reports N: N - (x23 + 1).
    sub  x0, x19, x23
    sub  x0, x0, #1
    adrp x1, _kcc_line_delta@PAGE
    add  x1, x1, _kcc_line_delta@PAGEOFF
    str  x0, [x1]

    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

136 · defined at 135, 136, 139ab, 140 · used in 28f · uses _kcc_arena_alloc 58a, _kcc_ht_get 62, _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_line 104b, _kcc_lex_next 90b, _kcc_pp_cond_depth 116, _kcc_pp_eval_expr 141, _kcc_pp_expand_arg 125b, _kcc_pp_macros 116, _kcc_pp_next 108, _kcc_pp_s_defined 116, _kcc_src_path 82, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

The `#elif` directive. The `#elif` handler must determine whether a previous branch in the same conditional group was already taken. If the current state is `ACTIVE`, the branch just processed was chosen, so the state transitions to `DONE` and the rest of the group is skipped. If the

state is already DONE, the directive line is consumed without evaluation. Only when the state is SKIP—meaning no branch has yet matched—does the handler evaluate the expression and potentially activate the branch.

⟨ *PP: conditional* 139a ⟩ + ≡

```
// -- _kcc_pp_do_elif ---
.p2align 2
_kcc_pp_do_elif:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Check conditional depth
    adrp x0, _kcc_pp_cond_depth@PAGE
    add  x0, x0, _kcc_pp_cond_depth@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lelif_stray

    // Check current state
    sub  x1, x0, #1
    adrp x2, _kcc_pp_cond_stack@PAGE
    add  x2, x2, _kcc_pp_cond_stack@PAGEOFF
    ldrb w3, [x2, x1]

    cmp  w3, #PP_COND_ACTIVE
    b.eq .Lelif_was_active      // previous branch
  was active → DONE
    cmp  w3, #PP_COND_DONE
    b.eq .Lelif_done_skip       // already found
  active → stay DONE

    // State is SKIP: evaluate expression to see
  if this branch is active.
    // Expand the controlling line first (see
  _kcc_pp_do_if).
    bl _kcc_pp_rest_of_line
    bl _kcc_pp_prepare_cond
    mov x19, x0
    mov x20, x1

    // Save/restore lexer, evaluate
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x21, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    ldr  x22, [x0]

    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x19, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    add  x1, x19, x20
    str  x1, [x0]

    bl _kcc_lex_next
    bl _kcc_pp_eval_expr
    mov x19, x0

    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x21, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    str  x22, [x0]

    // Update conditional stack top
    adrp x0, _kcc_pp_cond_depth@PAGE
    add  x0, x0, _kcc_pp_cond_depth@PAGEOFF
    ldr  x0, [x0]
    sub  x1, x0, #1
    adrp x2, _kcc_pp_cond_stack@PAGE
    add  x2, x2, _kcc_pp_cond_stack@PAGEOFF

    cmp  x19, #0
    mov  w3, #PP_COND_SKIP
    mov  w4, #PP_COND_ACTIVE
    csel w3, w4, w3, ne
    strb w3, [x2, x1]

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lelif_was_active:
    // Previous branch was active → mark DONE,
  skip rest
    adrp x0, _kcc_pp_cond_depth@PAGE
    add  x0, x0, _kcc_pp_cond_depth@PAGEOFF
    ldr  x0, [x0]
    sub  x1, x0, #1
    adrp x2, _kcc_pp_cond_stack@PAGE
    add  x2, x2, _kcc_pp_cond_stack@PAGEOFF
    mov  w3, #PP_COND_DONE
    strb w3, [x2, x1]
    bl _kcc_pp_skip_line
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lelif_done_skip:
    bl _kcc_pp_skip_line
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lelif_stray:
    adrp x0, _kcc_pp_dir_line@PAGE
    add  x0, x0, _kcc_pp_dir_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_stray_elif@PAGE
    add  x2, x2, _kcc_pp_err_stray_elif@PAGEOFF
    bl _kcc_diag_error
    bl _kcc_pp_skip_line
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

139a · defined at 135, 136, 139ab, 140 · used in 28f · uses _kcc_diag_error 76a, _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_next 90b, _kcc_pp_cond_depth 116, _kcc_pp_cond_stack 116, _kcc_pp_dir_line 116, _kcc_pp_err_stray_elif 116, _kcc_pp_eval_expr 141

#else, #endif, and the conditional push helper. The #else directive toggles the conditional stack: if the current branch was ACTIVE it becomes DONE, and if it was SKIP it becomes ACTIVE. #endif simply pops the stack by decrementing the depth counter. _kcc_pp_cond_push is the shared helper that stores a new state byte onto the stack and increments the depth.

⟨ *PP: conditional* 139b ⟩ + ≡

```
// -- _kcc_pp_do_else ---
.p2align 2
_kcc_pp_do_else:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _kcc_pp_cond_depth@PAGE
    add  x0, x0, _kcc_pp_cond_depth@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lelse_stray

    sub  x1, x0, #1
    adrp x2, _kcc_pp_cond_stack@PAGE
    add  x2, x2, _kcc_pp_cond_stack@PAGEOFF
    ldrb w3, [x2, x1]

    cmp  w3, #PP_COND_ACTIVE
    b.eq .Lelse_was_active
    cmp  w3, #PP_COND_DONE
    b.eq .Lelse_keep_done

    // Was SKIP → now ACTIVE
    mov  w3, #PP_COND_ACTIVE
    strb w3, [x2, x1]
    bl _kcc_pp_skip_line
    ldp x29, x30, [sp], #16
    ret

.Lelse_was_active:
    mov  w3, #PP_COND_DONE
    strb w3, [x2, x1]
    bl _kcc_pp_skip_line
    ldp x29, x30, [sp], #16
    ret

.Lelse_keep_done:
    bl _kcc_pp_skip_line
    ldp x29, x30, [sp], #16
    ret

.Lelse_stray:
    adrp x0, _kcc_pp_dir_line@PAGE
    add  x0, x0, _kcc_pp_dir_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_stray_else@PAGE
    add  x2, x2, _kcc_pp_err_stray_else@PAGEOFF
    bl _kcc_diag_error
    bl _kcc_pp_skip_line
    ldp x29, x30, [sp], #16
    ret

// -- _kcc_pp_do_endif ---
.p2align 2
_kcc_pp_do_endif:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _kcc_pp_cond_depth@PAGE
    add  x0, x0, _kcc_pp_cond_depth@PAGEOFF
    ldr  x1, [x0]
    cbz  x1, .Lendif_stray

    sub  x1, x1, #1
    str  x1, [x0]
    bl _kcc_pp_skip_line
    ldp x29, x30, [sp], #16
    ret

.Lendif_stray:
    adrp x0, _kcc_pp_dir_line@PAGE
    add  x0, x0, _kcc_pp_dir_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_stray_endif@PAGE
    add  x2, x2, _kcc_pp_err_stray_endif@PAGEOFF
    bl _kcc_diag_error
    bl _kcc_pp_skip_line
    ldp x29, x30, [sp], #16
    ret

// -- _kcc_pp_cond_push ---
// x0 = state to push
.p2align 2
_kcc_pp_cond_push:
    adrp x1, _kcc_pp_cond_depth@PAGE
    add  x1, x1, _kcc_pp_cond_depth@PAGEOFF
    ldr  x2, [x1]
    cmp  x2, #PP_MAX_COND
    b.hs .Lcpush_overflow
    adrp x3, _kcc_pp_cond_stack@PAGE
    add  x3, x3, _kcc_pp_cond_stack@PAGEOFF
    strb w0, [x3, x2]
    add  x2, x2, #1
    str  x2, [x1]
    ret
.Lcpush_overflow:
    ret                          // silently cap
   depth
```

□

139b · defined at 135, 136, 139ab, 140 · used in 28f · uses _kcc_diag_error 76a, _kcc_pp_cond_depth 116, _kcc_pp_cond_stack 116, _kcc_pp_dir_line 116, _kcc_pp_err_stray_else 116, _kcc_pp_err_stray_endif 116

When the preprocessor is skipping tokens inside a false conditional branch, it must still track nested #if/#endif pairs so that a matching #endif pops the correct level. _kcc_pp_skip_directive examines directives encountered during skipping: #if, #ifdef, and #ifndef push a new DONE state, #endif pops a level, and #elif/#else toggle the top of the conditional stack when the depth matches. The pushed state is DONE rather than SKIP precisely because the nested conditional is already dead: DONE keeps every one of its branches skipped, whereas SKIP would treat a nested #else as the branch to take and wrongly resume compilation inside the dead region.

⟨ *PP: conditional* 140 ⟩ + ≡

```
// -- _kcc_pp_skip_directive ---
// Handle directives encountered while skipping (
   conditional false).
// Only #if/#ifdef/#ifndef (push), #elif/#else (
   toggle), #endif (pop)
// are meaningful; all others are ignored.
.p2align 2
_kcc_pp_skip_directive:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    // Save directive line
    adrp x0, _kcc_tok_line@PAGE
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_pp_dir_line@PAGE
    add  x1, x1, _kcc_pp_dir_line@PAGEOFF
    str  x0, [x1]

    // Read directive name
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
```

```
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    // Directive name can be TOK_IDENT(1) or a
   keyword(10-53)
    cmp  x0, #TOK_IDENT
    b.eq .Lskipd_has_name
    cmp  x0, #10
    b.lt .Lskipd_done
    cmp  x0, #53
    b.gt .Lskipd_done
.Lskipd_has_name:

    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x19, [x0]
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr  x20, [x0]

    // Check for "if" (2), "ifdef" (5), "ifndef"
   (6) → push SKIP
    cmp x20, #2
    b.ne .Lskipd_not2
    mov x0, x19
    adrp x1, _kcc_pp_s_if@PAGE
    add  x1, x1, _kcc_pp_s_if@PAGEOFF
    mov x2, #2
    bl _kcc_memcmp
    cbz x0, .Lskipd_push
    b .Lskipd_done
.Lskipd_not2:

    cmp x20, #5
    b.ne .Lskipd_not5
    mov x0, x19
    adrp x1, _kcc_pp_s_ifdef@PAGE
    add  x1, x1, _kcc_pp_s_ifdef@PAGEOFF
    mov x2, #5
    bl _kcc_memcmp
    cbz x0, .Lskipd_push
    mov x0, x19
    adrp x1, _kcc_pp_s_endif@PAGE
    add  x1, x1, _kcc_pp_s_endif@PAGEOFF
    mov x2, #5
    bl _kcc_memcmp
    cbz x0, .Lskipd_endif
    b .Lskipd_done
.Lskipd_not5:

    cmp x20, #6
    b.ne .Lskipd_not6
    mov x0, x19
    adrp x1, _kcc_pp_s_ifndef@PAGE
    add  x1, x1, _kcc_pp_s_ifndef@PAGEOFF
    mov x2, #6
    bl _kcc_memcmp
    cbz x0, .Lskipd_push
    b .Lskipd_check46
.Lskipd_not6:

.Lskipd_check46:
    // Check "elif" (4), "else" (4)
    cmp x20, #4
    b.ne .Lskipd_done
    mov x0, x19
    adrp x1, _kcc_pp_s_elif@PAGE
    add  x1, x1, _kcc_pp_s_elif@PAGEOFF
    mov x2, #4
    bl _kcc_memcmp
    cbz x0, .Lskipd_elif
    mov x0, x19
    adrp x1, _kcc_pp_s_else@PAGE
    add  x1, x1, _kcc_pp_s_else@PAGEOFF
    mov x2, #4
    bl _kcc_memcmp
    cbz x0, .Lskipd_else
    b .Lskipd_done

.Lskipd_push:
    // Nested #if/#ifdef/#ifndef while skipping:
   the whole nested conditional
    // lies in a dead region, so push DONE, not
   SKIP.  DONE makes a nested
    // #else/#elif keep skipping; SKIP would let a
    nested #else \emph{activate}
    // and resume processing code inside the dead
   branch (the bug that made a
    // 64-bit build wrongly compile Lua's 32-bit
   `#else` fallback).
    mov x0, #PP_COND_DONE
    bl _kcc_pp_cond_push
    b .Lskipd_done

.Lskipd_endif:
    bl _kcc_pp_do_endif
    b .Lskipd_ret

.Lskipd_elif:
    bl _kcc_pp_do_elif
    b .Lskipd_ret

.Lskipd_else:
    bl _kcc_pp_do_else
    b .Lskipd_ret

.Lskipd_done:
    bl _kcc_pp_skip_line
.Lskipd_ret:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

140 · defined at 135, 136, 139ab, 140 · used in 28f · uses _kcc_lex_next 90b, _kcc_pp_dir_line 116, _kcc_pp_s_elif 116, _kcc_pp_s_else 116, _kcc_pp_s_endif 116, _kcc_pp_s_if 116, _kcc_pp_s_ifdef 116, _kcc_pp_s_ifndef 116, _kcc_tok_len 104b, _kcc_tok_line 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

17.15. Expression Evaluation for `#if`

The `#if` and `#elif` directives require evaluating a constant integer expression (§6.6). We implement the full integer operator set as a recursive-descent cascade, one function per precedence level: logical `||`, `&&`; bitwise `|`, `^`, `&`; equality and relational comparisons; the shifts `<<`, `>>`; additive `+`, `-`; multiplicative `*`, `/`, `%`; the unary operators `!`, `~`, `-`, `+`; and primaries—integer literals, parenthesised sub-expressions, `defined(NAME)`, and macro names (fully expanded, then 0 for any identifier left undefined).

Any identifier not recognized as a macro evaluates to 0 per §6.10.1 paragraph 4.

⟨ *PP: eval* 141 ⟩ ≡

```
.text
// -- _kcc_pp_eval_expr ---
// Evaluates #if expression. Token stream already
   set up.
// Returns x0 = integer result.
.globl _kcc_pp_eval_expr
.p2align 2
_kcc_pp_eval_expr:
    // Entry point: delegate to the conditional (
   ternary) operator
```

```
    b _kcc_pp_eval_ternary

// -- Conditional: cond ? then : else  (right-
   associative, lowest precedence)
.p2align 2
_kcc_pp_eval_ternary:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    bl _kcc_pp_eval_lor
    mov x19, x0                 // condition

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_QUESTION
    b.ne .Leval_ter_done

    bl _kcc_lex_next            // consume ?
    bl _kcc_pp_eval_expr        // then-branch (
   runs up to the :)
    mov x20, x0                 // then value

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COLON
    b.ne .Leval_ter_noelse
    bl _kcc_lex_next            // consume :
.Leval_ter_noelse:
    bl _kcc_pp_eval_ternary     // else-branch (
   right-associative)

    // Both branches are side-effect-free constant
    expressions, so evaluating
    // both to advance the token stream and
   selecting is safe.
    cmp  x19, #0
    csel x0, x20, x0, ne        // cond nonzero →
   then, else → else
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

.Leval_ter_done:
    mov  x0, x19
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

// -- Logical OR: expr || expr
.p2align 2
_kcc_pp_eval_lor:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    bl _kcc_pp_eval_land
    mov x19, x0

.Leval_lor_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_OR            // ||
    b.ne .Leval_lor_done

    bl _kcc_lex_next            // consume ||
    bl _kcc_pp_eval_land
    orr  x19, x19, x0
    cmp  x19, #0
    cset x19, ne
    b .Leval_lor_loop

.Leval_lor_done:
    mov  x0, x19
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

// -- Logical AND: expr && expr
.p2align 2
_kcc_pp_eval_land:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    bl _kcc_pp_eval_bor
    mov x19, x0

.Leval_land_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_AND           // &&
    b.ne .Leval_land_done

    bl _kcc_lex_next
    bl _kcc_pp_eval_bor
    and  x0, x19, x0
    cmp  x0, #0
    cset x19, ne
    b .Leval_land_loop

.Leval_land_done:
    mov  x0, x19
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret
```

□

141 · defined at 141, 142, 143, 144, 145ab, 146, 147ab · used in 28f · defines _kcc_pp_eval_expr 136, 139a, 145b, 147a, _kcc_pp_eval_ternary never used · uses _kcc_lex_next 90b, _kcc_pp_eval_bor 142, _kcc_tok_type 104b

Below the logical operators sit the three bitwise levels—|, then ^, then &—each a left-associative fold over the level beneath it. A preprocessor constant expression is a full integer constant expression (§6.6), so these are the same operators C allows outside `#if`; real headers use them to test feature bitmasks (`#if (FLAGS & X)`).

⟨ *PP: eval* 142 ⟩ +≡

```
// -- Bitwise OR: expr | expr
.p2align 2
_kcc_pp_eval_bor:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    bl _kcc_pp_eval_bxor
    mov x19, x0
.Leval_bor_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_PIPE
    b.ne .Leval_bor_done
    bl _kcc_lex_next
    bl _kcc_pp_eval_bxor
    orr  x19, x19, x0
    b .Leval_bor_loop
.Leval_bor_done:
    mov  x0, x19
    ldr  x19, [sp, #16]
```

```
    ldp  x29, x30, [sp], #32
    ret

// -- Bitwise XOR: expr ^ expr
.p2align 2
_kcc_pp_eval_bxor:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    bl _kcc_pp_eval_band
    mov x19, x0
.Leval_bxor_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_CARET
    b.ne .Leval_bxor_done
    bl _kcc_lex_next
    bl _kcc_pp_eval_band
    eor  x19, x19, x0
    b .Leval_bxor_loop
.Leval_bxor_done:
    mov  x0, x19
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

// -- Bitwise AND: expr & expr
.p2align 2
_kcc_pp_eval_band:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    bl _kcc_pp_eval_eq
    mov x19, x0
.Leval_band_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_AMP
    b.ne .Leval_band_done
    bl _kcc_lex_next
    bl _kcc_pp_eval_eq
    and  x19, x19, x0
    b .Leval_band_loop
.Leval_band_done:
    mov  x0, x19
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret
```

□

142 · defined at 141, 142, 143, 144, 145ab, 146, 147ab · used in 28f · defines _kcc_pp_eval_band never used, _kcc_pp_eval_bor 141, _kcc_pp_eval_bxor never used · uses _kcc_lex_next 90b, _kcc_tok_type 104b

The equality and relational operators form the next two precedence levels. _kcc_pp_eval_eq handles == and != by comparing the integer values returned from _kcc_pp_eval_rel, which in turn handles <, >, <=, and >=. All comparisons yield either 0 or 1 using the ARM64 cset instruction with the appropriate condition code.

⟨ PP: eval 143 ⟩ +≡

```
// -- Equality: expr == expr, expr != expr
.p2align 2
_kcc_pp_eval_eq:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    bl _kcc_pp_eval_rel
    mov x19, x0

.Leval_eq_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x20, [x0]
    cmp  x20, #TOK_EQ
    b.eq .Leval_eq_op
    cmp  x20, #TOK_NE
    b.eq .Leval_eq_op
    b .Leval_eq_done

.Leval_eq_op:
    bl _kcc_lex_next
    bl _kcc_pp_eval_rel
    cmp  x20, #TOK_EQ
    b.eq .Leval_do_eq
    // !=
    cmp  x19, x0
    cset x19, ne
    b .Leval_eq_loop
.Leval_do_eq:
    cmp  x19, x0
    cset x19, eq
    b .Leval_eq_loop

.Leval_eq_done:
    mov  x0, x19
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

// -- Relational: < > <= >=
.p2align 2
_kcc_pp_eval_rel:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    bl _kcc_pp_eval_shift
    mov x19, x0

.Leval_rel_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x20, [x0]
    cmp  x20, #TOK_LT
    b.eq .Leval_rel_op
    cmp  x20, #TOK_GT
    b.eq .Leval_rel_op
    cmp  x20, #TOK_LE
    b.eq .Leval_rel_op
    cmp  x20, #TOK_GE
    b.eq .Leval_rel_op
    b .Leval_rel_done

.Leval_rel_op:
    bl _kcc_lex_next
    bl _kcc_pp_eval_shift
    cmp  x20, #TOK_LT
    b.eq .Leval_do_lt
    cmp  x20, #TOK_GT
    b.eq .Leval_do_gt
    cmp  x20, #TOK_LE
    b.eq .Leval_do_le
    // >=
    cmp  x19, x0
    cset x19, ge
    b .Leval_rel_loop
.Leval_do_lt:
    cmp  x19, x0
    cset x19, lt
    b .Leval_rel_loop
.Leval_do_gt:
    cmp  x19, x0
```

```
    cset x19, gt
    b .Leval_rel_loop
.Leval_do_le:
    cmp  x19, x0
    cset x19, le
    b .Leval_rel_loop

.Leval_rel_done:
    mov  x0, x19
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret
```

□

143 · defined at 141, 142, 143, 144, 145ab, 146, 147ab · used in 28f · uses _kcc_lex_next 90b, _kcc_pp_eval_shift 144, _kcc_tok_type 104b

Beneath the relational operators come the shift, additive, and multiplicative levels—the arithmetic a real configuration header relies on (`#if VERSION >= 5 * 100 + 4`, `#if (N >> 3) == 1`). Each is a left-associative fold; division and remainder use signed `sdiv`, and because ARM64 `sdiv` yields zero rather than trapping on a zero divisor, an ill-formed `#if X / 0` degrades to zero instead of crashing the compiler. Right shift uses the *logical* `lsr`: preprocessor `#if` shifts are practically always bit-width probes of an unsigned maximum—`#if ((ULONG_MAX >> 31) >> 31) >= 3`, the idiom by which a header asks "is this type at least 64 bits?"—where an arithmetic shift of the all-ones `ULONG_MAX` (which reads as `-1`) would stay `-1` and pick the wrong branch. Logical shift is also correct for any non-negative signed operand; only a negative signed operand would differ, and that is both rare and implementation-defined.

⟨ *PP: eval* 144 ⟩ + ≡

```
// -- Shift: expr << expr, expr >> expr
.p2align 2
_kcc_pp_eval_shift:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    bl _kcc_pp_eval_add
    mov x19, x0
.Leval_shift_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x20, [x0]
    cmp  x20, #TOK_SHL
    b.eq .Leval_shift_op
    cmp  x20, #TOK_SHR
    b.eq .Leval_shift_op
    b .Leval_shift_done
.Leval_shift_op:
    bl _kcc_lex_next
    bl _kcc_pp_eval_add
    cmp  x20, #TOK_SHL
    b.ne .Leval_do_shr
    lsl  x19, x19, x0
    b .Leval_shift_loop
.Leval_do_shr:
    lsr  x19, x19, x0          // logical: correct
    for unsigned (and non-negative
                               // signed) operands
   , which is every real #if use
    b .Leval_shift_loop
.Leval_shift_done:
    mov  x0, x19
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

// -- Additive: expr + expr, expr - expr
.p2align 2
_kcc_pp_eval_add:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    bl _kcc_pp_eval_mul
    mov x19, x0
.Leval_add_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x20, [x0]
    cmp  x20, #TOK_PLUS
    b.eq .Leval_add_op
    cmp  x20, #TOK_MINUS
    b.eq .Leval_add_op
    b .Leval_add_done
.Leval_add_op:
    bl _kcc_lex_next
    bl _kcc_pp_eval_mul
    cmp  x20, #TOK_PLUS
    b.ne .Leval_do_sub
    add  x19, x19, x0
    b .Leval_add_loop
.Leval_do_sub:
    sub  x19, x19, x0
    b .Leval_add_loop
.Leval_add_done:
    mov  x0, x19
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

// -- Multiplicative: expr * expr, expr / expr,
   expr % expr
.p2align 2
_kcc_pp_eval_mul:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    bl _kcc_pp_eval_unary
    mov x19, x0
.Leval_mul_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x20, [x0]
    cmp  x20, #TOK_STAR
    b.eq .Leval_mul_op
    cmp  x20, #TOK_SLASH
    b.eq .Leval_mul_op
    cmp  x20, #TOK_PERCENT
    b.eq .Leval_mul_op
    b .Leval_mul_done
.Leval_mul_op:
    bl _kcc_lex_next
    bl _kcc_pp_eval_unary
    cmp  x20, #TOK_STAR
    b.eq .Leval_do_mul
    cmp  x20, #TOK_SLASH
    b.eq .Leval_do_div
    // %  → x19 - (x19 / x0) * x0
    sdiv x1, x19, x0
    msub x19, x1, x0, x19
    b .Leval_mul_loop
.Leval_do_mul:
    mul  x19, x19, x0
    b .Leval_mul_loop
.Leval_do_div:
    sdiv x19, x19, x0
    b .Leval_mul_loop
.Leval_mul_done:
    mov  x0, x19
    ldp  x19, x20, [sp, #16]
```

```
    ldp  x29, x30, [sp], #32
    ret
```

□

144 · defined at 141, 142, 143, 144, 145ab, 146, 147ab · used in 28f · defines _kcc_pp_eval_add never used, _kcc_pp_eval_mul never used, _kcc_pp_eval_shift 143 · uses _kcc_lex_next 90b, _kcc_tok_type 104b

At the lowest precedence tier sit the unary and primary expressions. _kcc_pp_eval_unary handles the unary operators !, ~, -, and + by recursing and then applying the corresponding operation (a leading + is a no-op). _kcc_pp_eval_primary recognizes four forms: integer literals (parsed via _kcc_pp_atoi), parenthesized sub-expressions, the defined operator (with or without parentheses), and bare identifiers—which evaluate to their macro body as an integer, or to 0 if undefined per §6.10.1/4.

⟨ PP: eval 145a ⟩ + ≡

```
// -- Unary: !expr, primary
.p2align 2
_kcc_pp_eval_unary:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_BANG
    b.eq .Leval_un_not
    cmp  x0, #TOK_TILDE
    b.eq .Leval_un_bnot
    cmp  x0, #TOK_MINUS
    b.eq .Leval_un_neg
    cmp  x0, #TOK_PLUS
    b.eq .Leval_un_pos
    b .Leval_primary

.Leval_un_not:
    bl _kcc_lex_next
    bl _kcc_pp_eval_unary
    cmp  x0, #0
    cset x0, eq                 // !x
    ldp  x29, x30, [sp], #16
    ret

.Leval_un_bnot:
    bl _kcc_lex_next
    bl _kcc_pp_eval_unary
    mvn  x0, x0                 // ~x
    ldp  x29, x30, [sp], #16
    ret

.Leval_un_neg:
    bl _kcc_lex_next
    bl _kcc_pp_eval_unary
    neg  x0, x0                 // -x
    ldp  x29, x30, [sp], #16
    ret

.Leval_un_pos:
    bl _kcc_lex_next
    bl _kcc_pp_eval_unary       // +x is a no-op
    ldp  x29, x30, [sp], #16
    ret

.Leval_primary:
    bl _kcc_pp_eval_primary
    ldp  x29, x30, [sp], #16
    ret
```

□

145a · defined at 141, 142, 143, 144, 145ab, 146, 147ab · used in 28f · uses _kcc_lex_next 90b, _kcc_tok_type 104b

Primary expressions and integer parsing. A primary expression in a preprocessor #if context is one of four forms: an integer literal, a parenthesized sub-expression, the defined operator (with or without parentheses around the macro name), or a bare identifier that is looked up as a macro and converted to an integer—defaulting to 0 for undefined names per §6.10.1 paragraph 4. The helper _kcc_pp_atoi performs simple decimal ASCII-to-integer conversion.

⟨ PP: eval 145b ⟩ + ≡

```
// -- Primary: number, defined(X), defined X, (
   expr), →ident0
.p2align 2
_kcc_pp_eval_primary:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    // Number literal
    cmp  x0, #TOK_NUMBER
    b.eq .Leval_number

    // Parenthesized expression
    cmp  x0, #TOK_LPAREN
    b.eq .Leval_paren

    // Identifier
    cmp  x0, #TOK_IDENT
    b.eq .Leval_ident

    // EOF or unknown → 0
    mov  x0, #0
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

.Leval_number:
    // Parse integer from token text
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_pp_atoi
    mov  x19, x0
    bl _kcc_lex_next
    mov  x0, x19
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

.Leval_paren:
    bl _kcc_lex_next            // consume (
    bl _kcc_pp_eval_expr
    mov  x19, x0
    // Expect )
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RPAREN
```

```
    b.ne .Leval_paren_done
    bl _kcc_lex_next            // consume )
.Leval_paren_done:
    mov  x0, x19
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret
```

□

145b · defined at 141, 142, 143, 144, 145ab, 146, 147ab · used in 28f · uses _kcc_lex_next 90b, _kcc_pp_eval_expr 141, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

Identifier and `defined` operator handling. Identifiers in a preprocessor constant expression require special treatment. The `defined` operator tests whether a macro name exists in the macro table, returning 1 or 0 without expanding it; both `defined(NAME)` and `defined NAME` forms are accepted per §6.10.1. Any other identifier is looked up as a macro and its body parsed as an integer; undefined names evaluate to 0. One subtlety in the zero-returning paths: the primary must *advance* past the identifier with `_kcc_lex_next` before loading the constant 0, because the lexer call returns its own value in x0 and would otherwise overwrite the zero. Getting this backwards makes an undefined name evaluate to the leftover token value—so a bound like `#if SQLITE_OS_OTHER + 1 > 1` (with `SQLITE_OS_OTHER` undefined) folds to *true* instead of `0 + 1 > 1`, silently picking the wrong configuration branch.

⟨ *PP: eval* 146 ⟩ + ≡

```
.Leval_ident:
    // Check for "defined"
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    cmp  x1, #7
    b.ne .Leval_ident_macro

    adrp x1, _kcc_pp_s_defined@PAGE
    add  x1, x1, _kcc_pp_s_defined@PAGEOFF
    mov  x2, #7
    bl _kcc_memcmp
    cbnz x0, .Leval_ident_macro

    // "defined" operator
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LPAREN
    b.eq .Leval_defined_paren

    // defined IDENT (without parens)
    cmp  x0, #TOK_IDENT
    b.ne .Leval_defined_zero
    b .Leval_defined_check

.Leval_defined_paren:
    bl _kcc_lex_next            // get IDENT
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_IDENT
    b.ne .Leval_defined_zero

.Leval_defined_check:
    // Look up macro
    adrp x0, _kcc_pp_macros@PAGE
    add  x0, x0, _kcc_pp_macros@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    ldr  x1, [x1]
    adrp x2, _kcc_tok_len@PAGE
    add  x2, x2, _kcc_tok_len@PAGEOFF
    ldr  x2, [x2]
    bl _kcc_ht_get
    cmp  x0, #0
    cset x19, ne                // 1 if defined, 0 if not

    // Skip closing ) if present
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RPAREN
    b.ne .Leval_defined_noparen
    bl _kcc_lex_next
.Leval_defined_noparen:
    mov  x0, x19
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

.Leval_defined_zero:
    bl _kcc_lex_next            // clobbers x0 with the lexer's result...
    mov  x0, #0                 // ...so zero the value AFTER advancing
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

.Leval_ident_macro:
    // Regular identifier → look up as macro, expand to 0 if not found
    // Per §6.10.1p4: remaining identifiers → 0
    adrp x0, _kcc_pp_macros@PAGE
    add  x0, x0, _kcc_pp_macros@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_start@PAGE
    add  x1, x1, _kcc_tok_start@PAGEOFF
    ldr  x1, [x1]
    adrp x2, _kcc_tok_len@PAGE
    add  x2, x2, _kcc_tok_len@PAGEOFF
    ldr  x2, [x2]
    bl _kcc_ht_get
    cbz  x0, .Leval_ident_zero

    // Found macro: fully expand and evaluate its body (handles macro chains
    // and expression bodies, per §6.10.1's full macro replacement).
    bl _kcc_pp_eval_macro
    mov  x19, x0
    bl _kcc_lex_next
    mov  x0, x19
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

.Leval_ident_zero:
    bl _kcc_lex_next            // clobbers x0 with the lexer's result...
    mov  x0, #0                 // ...so an undefined name yields 0 AFTER advancing
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
```

```
    ret
```

□

146 · defined at 141, 142, 143, 144, 145ab, 146, 147ab · used in 28f · uses _kcc_ht_get 62, _kcc_lex_next 90b, _kcc_pp_macros 116, _kcc_pp_s_defined 116, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

Recursively evaluating a macro body. A one-token object-like macro whose body is a number, such as `#define LUA_FLOAT_DOUBLE 2`, could be handled by parsing the body as an integer. But real headers chain macros—`#define LUA_FLOAT_TYPE LUA_FLOAT_DOUBLE`—and give them parenthesised expression bodies, and §6.10.1 requires the whole controlling expression to undergo full macro replacement before it is evaluated. Rather than parse the body as a flat integer, we re-point the lexer at the macro's replacement text and evaluate it with the same recursive-descent parser. A body that is itself a macro name recurses back through this routine until it bottoms out at a number or an undefined identifier (which is 0), so an arbitrarily long chain resolves correctly; a parenthesised or arithmetic body is evaluated as the expression it is. A depth counter guards against a macro that expands to itself, which §6.10.3.4 forbids from re-expanding but which we must at least not loop on.

⟨ *PP: eval* 147a ⟩ +≡

```
// -- _kcc_pp_eval_macro ---
// int64_t _kcc_pp_eval_macro(pp_macro *m)
//   x0 = macro entry.  Returns the integer value
   of its expanded body.
// Saves the lexer over the #if line, evaluates
   the body, and restores it
// so the caller resumes at the token after the
   macro identifier.
.p2align 2
_kcc_pp_eval_macro:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                    // macro entry

    // Depth guard: bail to 0 rather than loop on
   a self-referential macro.
    adrp x2, _kcc_pp_eval_depth@PAGE
    add  x2, x2, _kcc_pp_eval_depth@PAGEOFF
    ldr  x1, [x2]
    cmp  x1, #64
    b.ge .Lem_zero
    add  x1, x1, #1
    str  x1, [x2]

    // Save the current lexer position (cursor
   sits just past the identifier).
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x20, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    ldr  x21, [x0]

    // Point the lexer at the macro body and
   evaluate it as an expression.
    ldr  x0, [x19, #PP_MACRO_BODY]
    ldr  x1, [x19, #PP_MACRO_BODYLEN]
    adrp x2, _kcc_lex_cursor@PAGE
    add  x2, x2, _kcc_lex_cursor@PAGEOFF
    str  x0, [x2]
    add  x1, x0, x1
    adrp x2, _kcc_lex_end@PAGE
    add  x2, x2, _kcc_lex_end@PAGEOFF
    str  x1, [x2]

    bl _kcc_lex_next
    bl _kcc_pp_eval_expr
    mov  x19, x0                   // result

    // Restore the lexer to the #if line.
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x20, [x0]
    adrp x0, _kcc_lex_end@PAGE
    add  x0, x0, _kcc_lex_end@PAGEOFF
    str  x21, [x0]

    // Decrement the depth guard.
    adrp x2, _kcc_pp_eval_depth@PAGE
    add  x2, x2, _kcc_pp_eval_depth@PAGEOFF
    ldr  x1, [x2]
    sub  x1, x1, #1
    str  x1, [x2]

    mov  x0, x19
    ldr  x21, [sp, #32]
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #48
    ret

.Lem_zero:
    mov  x0, #0
    ldr  x21, [sp, #32]
    ldp  x19, x20, [sp, #16]
    ldp  x29, x30, [sp], #48
    ret
```

□

147a · defined at 141, 142, 143, 144, 145ab, 146, 147ab · used in 28f · uses _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_next 90b, _kcc_pp_eval_expr 141

Integer-constant parsing. The helper `_kcc_pp_atoi` converts an integer-constant token to a 64-bit value, honouring the three C bases: a `0x`/`0X` prefix selects hexadecimal, a bare leading `0` selects octal, and anything else is decimal. This matters because real headers gate on hex feature masks (`#if (FLAGS & 0x04)`); a decimal-only reader would see `0x04` as 0 and mis-evaluate every such test. Each base accumulates by its radix and stops at the first character that is not a digit of that base—which also discards any `u`/`l` suffix—so no explicit suffix handling is needed. Constants here are non-negative; unary minus is applied one precedence level up.

⟨ *PP: eval* 147b ⟩ +≡

```
// -- _kcc_pp_atoi ---
// Integer-constant to value (hex/octal/decimal):
   x0=ptr, x1=len → x0=value
.globl _kcc_pp_atoi
.p2align 2
_kcc_pp_atoi:
    mov  x2, #0                    // accumulator
    mov  x3, #0                    // index
    // Detect a 0x / 0X (hex) or bare-0 (octal)
   prefix.
    cmp  x1, #2
```

```
    b.lo .Latoi_dec             // too short for a
   prefix → decimal
    ldrb w4, [x0]
    cmp  w4, #'0'
    b.ne .Latoi_dec
    ldrb w4, [x0, #1]
    orr  w4, w4, #0x20          // fold case
    cmp  w4, #'x'
    b.eq .Latoi_hex
    b .Latoi_oct                // leading 0, not x
    → octal

.Latoi_dec:
    cmp  x3, x1
    b.hs .Latoi_done
    ldrb w4, [x0, x3]
    cmp  w4, #'0'
    b.lo .Latoi_done
    cmp  w4, #'9'
    b.hi .Latoi_done
    sub  w4, w4, #'0'
    mov  x5, #10
    mul  x2, x2, x5
    add  x2, x2, x4
    add  x3, x3, #1
    b .Latoi_dec

.Latoi_hex:
    mov  x3, #2                 // skip "0x"
.Latoi_hex_loop:
    cmp  x3, x1
    b.hs .Latoi_done
    ldrb w4, [x0, x3]
    cmp  w4, #'0'
    b.lo .Latoi_done
    cmp  w4, #'9'
    b.hi .Latoi_hex_af
    sub  w4, w4, #'0'
    b .Latoi_hex_acc
.Latoi_hex_af:
    orr  w4, w4, #0x20          // fold case
    cmp  w4, #'a'
    b.lo .Latoi_done
    cmp  w4, #'f'
    b.hi .Latoi_done
    sub  w4, w4, #'a'
    add  w4, w4, #10
.Latoi_hex_acc:
    lsl  x2, x2, #4
    add  x2, x2, x4
    add  x3, x3, #1
    b .Latoi_hex_loop

.Latoi_oct:
    mov  x3, #1                 // skip leading 0
.Latoi_oct_loop:
    cmp  x3, x1
    b.hs .Latoi_done
    ldrb w4, [x0, x3]
    cmp  w4, #'0'
    b.lo .Latoi_done
    cmp  w4, #'7'
    b.hi .Latoi_done
    sub  w4, w4, #'0'
    lsl  x2, x2, #3
    add  x2, x2, x4
    add  x3, x3, #1
    b .Latoi_oct_loop

.Latoi_done:
    mov  x0, x2
    ret
```

□

147b · defined at 141, 142, 143, 144, 145ab, 146, 147ab · used in 28f

17.16. File Inclusion (`#include`)

`#include "file"` opens and processes the named file. The current lexer and source states are pushed onto the context stack. When the included file reaches EOF, the state is restored and processing continues in the including file.

Locating the file. A quoted include is resolved (§6.10.2) relative to the directory of the file that contains the directive, then—failing that—by the name as given (which the operating system interprets against the current working directory), and finally along an ordered list of *include directories*. This is what allows a project header to sit beside the `.c` file that includes it regardless of where the compiler was invoked, while library headers are found on the search path. `_kcc_pp_resolve_include` performs the search: it splices the directory prefix of the current source path onto the include name, probes that candidate with a throwaway `open`, and—failing the including directory and the bare name—walks the include-directory list, joining each directory to the name and probing in turn, returning the first path that exists. A `_kcc_pp_file_exists` helper wraps the probe, relying on `_kcc_sys_open` returning -1 for a file that cannot be opened. Angle-bracket includes are resolved the same way; the list is populated by `-I` directories and by the compiler's own default directory (§17.16.1). A quoted include that resolves nowhere is a hard error, but a missing *system* header (`#include <…>`) is silently skipped: kcc is freestanding and supplies what such headers would declare—for instance the `va_*` variadic built-ins—directly, so a program that writes `#include <stdarg.h>` for portability still compiles.

⟨ *PP: resolve include* 148 ⟩ ≡

```
.text
// -- _kcc_pp_file_exists ---
// uint64_t _kcc_pp_file_exists(const char *path)
//   x0 = null-terminated path → 1 if it can be
   opened, else 0
.p2align 2
_kcc_pp_file_exists:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    mov x1, #0                  // O_RDONLY
    mov x2, #0
    bl _kcc_sys_open
    cmp x0, #0                  // fd < 0 → open
   failed (macOS: -1; Linux: -errno)
    b.lt .Lfe_no
    bl _kcc_sys_close           // opened: close
   the probe fd
    mov x0, #1
    ldp x29, x30, [sp], #16
    ret
.Lfe_no:
    mov x0, #0
    ldp x29, x30, [sp], #16
    ret

// -- _kcc_pp_resolve_include ---
// char *_kcc_pp_resolve_include(const char *name)
//   x0 = include name → openable path (arena), or
    0 if not found.
// Searches the including file's directory first,
   then the name as given.
```

```
.p2align 2
_kcc_pp_resolve_include:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0                     // name
    bl _kcc_strlen
    mov x20, x0                     // namelen

    adrp x0, _kcc_src_path@PAGE
    add  x0, x0, _kcc_src_path@PAGEOFF
    ldr  x21, [x0]                  // including file
    path
    cbz x21, .Lri_raw

    // dirlen = length up to and including the
   last '/'
    mov x22, #0
    mov x23, #0
.Lri_scan:
    ldrb w0, [x21, x23]
    cbz w0, .Lri_scan_done
    cmp w0, #'/'
    b.ne .Lri_scan_next
    add x22, x23, #1
.Lri_scan_next:
    add x23, x23, #1
    b .Lri_scan
.Lri_scan_done:
    cbz x22, .Lri_raw               // no directory
   prefix

    add x0, x22, x20
    add x0, x0, #1
    bl _kcc_arena_alloc
    mov x24, x0                     // candidate =
   dir + name + null
    mov x0, x24
    mov x1, x21
    mov x2, x22
    bl _kcc_memcpy
    add x0, x24, x22
    mov x1, x19
    mov x2, x20
    bl _kcc_memcpy
    add x0, x24, x22
    strb wzr, [x0, x20]

    mov x0, x24
    bl _kcc_pp_file_exists
    cbz x0, .Lri_raw
    mov x0, x24
    b .Lri_done

.Lri_raw:
    mov x0, x19
    bl _kcc_pp_file_exists
    cbz x0, .Lri_search
    mov x0, x19
    b .Lri_done

.Lri_search:
    // Walk the include-directory list: join each
   dir (stored with a trailing
    // '/') to the name and probe.  x19=name, x20=
   namelen are still live.
    mov x22, #0                     // i
.Lri_search_loop:
    adrp x0, _kcc_inc_ndirs@PAGE
    add  x0, x0, _kcc_inc_ndirs@PAGEOFF
    ldr  x0, [x0]
    cmp  x22, x0
    b.hs .Lri_none
    adrp x0, _kcc_inc_dirs@PAGE
    add  x0, x0, _kcc_inc_dirs@PAGEOFF
    ldr  x23, [x0, x22, lsl #3]  // dir[i]
    mov  x0, x23
    bl _kcc_strlen
    mov  x21, x0                    // dirlen
    add  x0, x21, x20
    add  x0, x0, #1
    bl _kcc_arena_alloc
    mov  x24, x0                    // candidate =
   dir + name + null
    mov  x0, x24
    mov  x1, x23
    mov  x2, x21
    bl _kcc_memcpy
    add  x0, x24, x21
    mov  x1, x19
    mov  x2, x20
    bl _kcc_memcpy
    add  x0, x24, x21
    strb wzr, [x0, x20]
    mov  x0, x24
    bl _kcc_pp_file_exists
    cbz  x0, .Lri_search_next
    mov  x0, x24
    b .Lri_done
.Lri_search_next:
    add  x22, x22, #1
    b .Lri_search_loop

.Lri_none:
    // Last resort: the compiler's own include/
   directory, tried only after every
    // -I so a -I shim always wins.  x19=name, x20
   =namelen are still live.
    adrp x0, _kcc_default_inc@PAGE
    add  x0, x0, _kcc_default_inc@PAGEOFF
    ldr  x23, [x0]                  // default dir (
   with trailing '/'), or 0
    cbz  x23, .Lri_really_none
    mov  x0, x23
    bl _kcc_strlen
    mov  x21, x0                    // dirlen
    add  x0, x21, x20
    add  x0, x0, #1
    bl _kcc_arena_alloc
    mov  x24, x0                    // candidate =
   dir + name + null
    mov  x0, x24
    mov  x1, x23
    mov  x2, x21
    bl _kcc_memcpy
    add  x0, x24, x21
    mov  x1, x19
    mov  x2, x20
    bl _kcc_memcpy
    add  x0, x24, x21
    strb wzr, [x0, x20]
    mov  x0, x24
    bl _kcc_pp_file_exists
    cbz  x0, .Lri_really_none
    mov  x0, x24
    b .Lri_done
.Lri_really_none:
    mov x0, #0

.Lri_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

148 · used in 28f · uses _kcc_arena_alloc 58a,

_kcc_src_path 82

17.16.1. The include search path

The list that _kcc_pp_resolve_include walks is a fixed-capacity array of directory strings, each normalised to carry a trailing slash so a candidate path is a plain concatenation. Two entry points populate it. _kcc_pp_add_include_dir appends one directory, appending the slash if the caller's string lacks it; the driver calls it for every -I argument. _kcc_pp_add_default_includes registers the compiler's *own* directory: given argv[0] it takes the prefix up to the last slash—the directory the kcc binary was run from—and appends include/, yielding the sibling include/ folder that ships beside the executable. This is where the hosted library headers (§17.16.2) live as ordinary files, so #include <stdio.h> resolves with no configuration. Crucially the default directory is kept in a *separate* slot (_kcc_default_inc) rather than pushed onto the -I array, and _kcc_pp_resolve_include consults it only *after* every -I directory has missed. It is registered once at startup—before the command line is parsed—so were it an ordinary array entry it would sit ahead of the -I dirs and shadow them; a build that points -I at a fuller shim (a real <fcntl.h> with struct flock, an <errno.h> with the platform's error numbers, as SQLite's Unix VFS needs) must instead win over the leaner shipped copy. Because both functions write to a driver-lifetime global rather than to per-file preprocessor state, the path is registered once at startup and survives every translation unit.

⟨ *PP: include path* 150 ⟩ ≡

```
.text
// -- _kcc_pp_add_include_dir ---
// void _kcc_pp_add_include_dir(const char *dir)
//   Appends dir (normalised with a trailing '/')
   to the include search list.
//   Silently ignored once the fixed-size list is
   full.
.globl _kcc_pp_add_include_dir
.p2align 2
_kcc_pp_add_include_dir:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // dir
    adrp x0, _kcc_inc_ndirs@PAGE
    add  x0, x0, _kcc_inc_ndirs@PAGEOFF
    ldr  x22, [x0]                  // count
    cmp  x22, #64
    b.hs .Ladd_done
    mov  x0, x19
    bl _kcc_strlen
    mov  x21, x0                    // dirlen
    add  x0, x21, #2                // '/' + null
    bl _kcc_arena_alloc
    mov  x20, x0                    // buf
    mov  x0, x20
    mov  x1, x19
    mov  x2, x21
    bl _kcc_memcpy
    mov  w0, #'/'
    strb w0, [x20, x21]
    add  x0, x21, #1
    strb wzr, [x20, x0]
    adrp x0, _kcc_inc_dirs@PAGE
    add  x0, x0, _kcc_inc_dirs@PAGEOFF
    str  x20, [x0, x22, lsl #3]
    add  x22, x22, #1
    adrp x0, _kcc_inc_ndirs@PAGE
    add  x0, x0, _kcc_inc_ndirs@PAGEOFF
    str  x22, [x0]
.Ladd_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- _kcc_pp_add_default_includes ---
// void _kcc_pp_add_default_includes(const char *
   argv0)
//   Derives the compiler's own include directory
   as dirname(argv0) +
//   "include" and registers it.  With no
   directory prefix in argv0 the bare
//   name "include" (relative to the working
   directory) is used.
.globl _kcc_pp_add_default_includes
.p2align 2
_kcc_pp_add_default_includes:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // argv0
    mov x20, #0                     // prefixlen (
   through last '/')
    mov x21, #0                     // i
.Ladi_scan:
    ldrb w0, [x19, x21]
    cbz  w0, .Ladi_done
    cmp  w0, #'/'
    b.ne .Ladi_next
    add  x20, x21, #1
.Ladi_next:
    add  x21, x21, #1
    b .Ladi_scan
.Ladi_done:
    add  x0, x20, #9                // prefix + "
   include"(7) + "/"(1) + null
    bl _kcc_arena_alloc
    mov  x22, x0                    // buf
    mov  x0, x22
    mov  x1, x19
    mov  x2, x20
    bl _kcc_memcpy                  // copy dir prefix
    add  x0, x22, x20
    adrp x1, _kcc_str_include@PAGE
    add  x1, x1, _kcc_str_include@PAGEOFF
    mov  x2, #7
    bl _kcc_memcpy                  // append "include"
    add  x0, x22, x20
    mov  w1, #'/'
    strb w1, [x0, #7]               // append the
   trailing '/' for a plain join
    strb wzr, [x0, #8]
    // Store as the default directory --- searched
    only AFTER every -I so a
    // hosted build's -I shim (a fuller <fcntl.h>
   with struct flock, a real
    // <errno.h>) is never shadowed by the leaner
   headers shipped in include/.
    adrp x0, _kcc_default_inc@PAGE
    add  x0, x0, _kcc_default_inc@PAGEOFF
    str  x22, [x0]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

```
.data
.p2align 3
_kcc_inc_dirs:  .space 8*64      // up to 64
   include directories
_kcc_inc_ndirs: .quad 0
_kcc_default_inc: .quad 0        // compiler's own
    include/ dir; searched LAST
.section __TEXT,__cstring
_kcc_str_include: .asciz "include"
.text
```

□

150 · used in 28f · defines _kcc_pp_add_default_includes 621, _kcc_pp_add_include_dir 606a · uses _kcc_arena_alloc 58a

⟨ PP: include 151 ⟩ ≡

```
.text
// -- _kcc_pp_do_include ---
.p2align 2
_kcc_pp_do_include:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    // Read include path token
    bl _kcc_lex_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    // Accept string literal "file.h" or scan <
   file.h>
    cmp  x0, #TOK_STRING_LIT
    b.eq .Lincl_string
    cmp  x0, #TOK_LT
    b.eq .Lincl_angle
    b .Lincl_err

.Lincl_string:
    mov  x22, #0              // x22 = 0: quoted
   include (must resolve)
    // Token text includes quotes: "file.h"
    // Strip quotes: start+1, len-2
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x19, [x0]
    add  x19, x19, #1         // skip opening "
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr  x20, [x0]
    sub  x20, x20, #2         // remove both
   quotes
    b .Lincl_open

.Lincl_angle:
    mov  x22, #1              // x22 = 1: system
   <...> include (skippable)
    // Scan from cursor to '>' to get the path
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    ldr  x19, [x0]            // start after <
    adrp x1, _kcc_lex_end@PAGE
    add  x1, x1, _kcc_lex_end@PAGEOFF
    ldr  x1, [x1]
    mov  x2, x19
.Lincl_angle_scan:
    cmp  x2, x1
    b.hs .Lincl_err
    ldrb w3, [x2]
    cmp  w3, #'>'
    b.eq .Lincl_angle_found
    cmp  w3, #0x0A
    b.eq .Lincl_err           // newline before >
    add  x2, x2, #1
    b .Lincl_angle_scan
.Lincl_angle_found:
    sub  x20, x2, x19         // path length
    add  x2, x2, #1           // skip >
    adrp x0, _kcc_lex_cursor@PAGE
    add  x0, x0, _kcc_lex_cursor@PAGEOFF
    str  x2, [x0]             // update cursor
    b .Lincl_open

.Lincl_open:
    // Create null-terminated path in arena
    add  x0, x20, #1          // +1 for null
    bl _kcc_arena_alloc
    mov  x21, x0              // x21 = path buffer

    mov  x0, x21               // dst
    mov  x1, x19               // src
    mov  x2, x20               // len
    bl _kcc_memcpy

    // Null-terminate
    strb wzr, [x21, x20]

    // Skip rest of directive line
    bl _kcc_pp_skip_line

    // Resolve on disk first (the including file's
    directory, the literal path,
    // then each -I directory); a compiler-
   resident built-in serves only as the
    // fallback when nothing is found.  Disk-
   before-built-in lets a hosted build
    // point -I at a fuller shim---the platform's
   real errno values or `struct
    // flock`---and get it, while a bare standard
   include with no -I match still
    // falls through to the resident header, so a
   freestanding program compiles
    // with no search path.  x21 (name) and x22 (
   system flag) survive the resolve
    // call for the fallback and the not-found
   decision.
    mov  x0, x21
    bl _kcc_pp_resolve_include
    cbz  x0, .Lincl_builtin
    mov  x21, x0                  // resolved path
   (name no longer needed)

    // Push current state onto context stack
    mov  x0, #PP_CTX_INCLUDE
    bl _kcc_pp_ctx_push

    // Open and process the included file
    mov  x0, x21
    bl _kcc_source_open
    bl _kcc_source_encode
    bl _kcc_source_splice

    // Initialize lexer on included file
    adrp x0, _kcc_src_base@PAGE
    add  x0, x0, _kcc_src_base@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_lex_init

    // Reset prev_line for new file
    adrp x0, _kcc_pp_prev_line@PAGE
    add  x0, x0, _kcc_pp_prev_line@PAGEOFF
```

```
    str  xzr, [x0]

    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.Lincl_builtin:
    // Not on disk: fall back to a resident built-
   in header, if any.  Use x23/x24
    // for its text/length so x22 (system flag)
   stays live for .Lincl_notfound.
    mov  x0, x21
    bl _kcc_pp_builtin_header
    cbz  x0, .Lincl_notfound
    mov  x23, x0                    // header text
    bl _kcc_strlen
    mov  x24, x0                    // length

    mov  x0, #PP_CTX_INCLUDE
    bl _kcc_pp_ctx_push

    adrp x0, _kcc_src_base@PAGE
    add  x0, x0, _kcc_src_base@PAGEOFF
    str  x23, [x0]
    adrp x0, _kcc_src_size@PAGE
    add  x0, x0, _kcc_src_size@PAGEOFF
    str  x24, [x0]
    adrp x0, _kcc_src_path@PAGE
    add  x0, x0, _kcc_src_path@PAGEOFF
    str  x21, [x0]                  // path = header
   name

    mov  x0, x23
    mov  x1, x24
    bl _kcc_lex_init
    adrp x0, _kcc_pp_prev_line@PAGE
    add  x0, x0, _kcc_pp_prev_line@PAGEOFF
    str  xzr, [x0]

    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.Lincl_notfound:
    // A missing system header (<...>) is silently
    skipped: kcc is freestanding
    // and provides what it needs (e.g. the va_*
   builtins) without headers.
    // A missing quoted header ("...") is still a
   hard error.
    cbnz x22, .Lincl_skip
    adrp x0, _kcc_pp_dir_line@PAGE
    add  x0, x0, _kcc_pp_dir_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_notfound@PAGE
    add  x2, x2, _kcc_pp_err_notfound@PAGEOFF
    bl _kcc_diag_error
.Lincl_skip:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.Lincl_err:
    adrp x0, _kcc_pp_dir_line@PAGE
    add  x0, x0, _kcc_pp_dir_line@PAGEOFF
    ldr  x0, [x0]
    mov  x1, #1
    adrp x2, _kcc_pp_err_include@PAGE
    add  x2, x2, _kcc_pp_err_include@PAGEOFF
    bl _kcc_diag_error
    bl _kcc_pp_skip_line
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

151 · used in 28f · uses _kcc_arena_alloc 58a, _kcc_diag_error 76a, _kcc_lex_cursor 104b, _kcc_lex_end 104b, _kcc_lex_init 90a, _kcc_lex_next 90b, _kcc_pp_dir_line 116, _kcc_pp_err_include 116, _kcc_pp_prev_line 116, _kcc_source_encode 85, _kcc_source_open 81b, _kcc_source_splice 87, _kcc_src_base 82, _kcc_src_path 82, _kcc_src_size 82, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

17.16.2. Built-in standard headers

A freestanding compiler that simply skips `#include <stdio.h>` forces every program to hand-declare `printf`, `malloc`, and friends. That is tolerable for the compiler's own bootstrap sources but hostile to ordinary C. kcc answers this in two ways, along the fault line the Standard itself draws. The *freestanding* headers are embedded directly in the compiler: when the include machinery finds no file on disk for a recognised name, it hands back a compiler-resident C snippet and lexes it in place of a file (the fallback at `.Lincl_builtin` above, reached only after the disk search comes up empty—so a `-I` shim carrying a fuller version of the same header wins over the built-in). The *hosted* headers are shipped as ordinary files in the `include/` directory beside the binary (§17.16.1), found on the search path like any other header. Either way the text carries real prototypes, so the front end learns each function's arity and argument classes—enough to marshal a call correctly—and the linker resolves the symbols against `libSystem`.

The prototypes are deliberately spare: parameters are named (kcc's declarator parser wants a name), qualifiers are dropped (they do not affect the calling convention), and `size_t`-shaped arguments are written as `unsigned long` so no cross-header `typedef` collides. Each header carries an `#ifndef` guard so double inclusion is a no-op. The embedded lookup is a linear scan of a name/text table—nine entries, one per freestanding header, so nothing fancier is warranted.

A distinction from the Standard shapes which of these headers kcc truly *owns*. The *freestanding* headers (§4.6)—`<float.h>`, `<iso646.h>`, `<limits.h>`, `<stdalign.h>`, `<stdarg.h>`, `<stdbool.h>`, `<stddef.h>`, `<stdint.h>`, and `<stdnoreturn.h>`—declare no library functions; they define properties of the *compiler* itself: type limits, `size_t`, `va_list`, `offsetof`, the boolean and character macros. A conforming implementation, even a freestanding one, must supply these, and it is the compiler—which alone knows `INT_MAX`, `DBL_MAX`, and the widths of the integer types on its target—that is obliged to. kcc therefore provides all nine directly: `<limits.h>` and `<float.h>` enumerate the target's integer and floating limits (an LP64 target where `long` is 64 bits and `long double` equals `double`), `<iso646.h>` the alterna-

tive operator spellings, <stdnoreturn.h> the noreturn macro, and <stdalign.h> the alignas and alignof aliases for the _Alignas and _Alignof operators, which the front end now accepts (_Alignof yields the type's alignment as an unsigned long constant; _Alignas is consumed as a declaration specifier and its argument accepted, though over-aligning storage beyond the frame's natural guarantee is not yet enforced). The remaining standard headers—<stdio.h>, <stdlib.h>, <string.h>, <math.h>, <ctype.h>, <assert.h>—are *hosted* headers that properly belong to the C library, not the compiler. kcc supplies spare prototypes for them only as a convenience bridge so the symbols resolve against libSystem at link time; rather than bake them into the binary, it ships them as the ordinary .h files below, tangled to include/. Adding another hosted bridge—<time.h>, <errno.h>—is then a matter of writing a file, not editing the compiler: a new header fragment and its @o route, no assembly.

Each of <stdio.h>, <stdlib.h>, and <string.h> opens with typedef unsigned long size_t;, because §7.21, §7.22, and §7.24 each require the header to define size_t independently of <stddef.h>. Real code leans on this constantly—malloc((size_t)n), memcpy(d, s, (size_t)len)—so without it a cast to size_t fails to parse as a type name. The typedef is intentionally repeated across the headers rather than guarded behind a shared macro: an identical typedef is a legal redefinition (§6.7), and the front end accepts it, so a translation unit that includes several of these headers sees one consistent size_t with no conflict.

For the same reason each header that the standard obliges to define NULL—<stdlib.h> (§7.22), <string.h> (§7.24), <time.h> (§7.27), <locale.h> (§7.11), and <wchar.h> (§7.29), alongside <stdio.h> and <stddef.h> which already did—opens with an #ifndef NULL guard defining it as ((void*)0). A program that includes only <stdlib.h> and writes NULL—as *doomgeneric*'s sounds.c does in { name, 0, NULL, NULL }—then compiles without having to also pull in <stddef.h>. The guard makes the repetition across headers harmless.

The standard streams stdin, stdout, and stderr are included. They are not functions but *data* objects imported from libSystem (__stdinp and friends), which the header exposes as extern declarations with the usual macro aliases. A dylib-imported datum has no fixed image address, so a reference to one must load its address through the global offset table rather than form it with a direct adrp/add pair; the back end's IR_GLOBAL_REF handler does exactly this for every external data symbol, so fprintf(stderr, …) now links and runs.

⟨ *Preprocessor: builtin headers* 153 ⟩ ≡

```
// char *_kcc_pp_builtin_header(const char *name)
//   x0 = include name → compiler-resident header
   text, or 0 if unknown.
.p2align 2
_kcc_pp_builtin_header:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                          // requested
   include name
    adrp x20, _kcc_pp_builtin_table@PAGE
    add  x20, x20, _kcc_pp_builtin_table@PAGEOFF
.Lbh_loop:
    ldr x0, [x20]                        // candidate
   name (0 terminates table)
    cbz x0, .Lbh_none
    mov x1, x19
    bl _kcc_strcmp
    cbz x0, .Lbh_found
    add x20, x20, #16
    b .Lbh_loop
.Lbh_found:
    ldr x0, [x20, #8]                    // matching
   header text
    b .Lbh_ret
.Lbh_none:
    mov x0, #0
.Lbh_ret:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

.section __TEXT,__cstring
_kcc_hdr_n_stddef:  .asciz "stddef.h"
_kcc_hdr_n_stdbool: .asciz "stdbool.h"
_kcc_hdr_n_stdarg:  .asciz "stdarg.h"
_kcc_hdr_n_stdint:  .asciz "stdint.h"
_kcc_hdr_n_limits:  .asciz "limits.h"
_kcc_hdr_n_float:   .asciz "float.h"
_kcc_hdr_n_iso646:  .asciz "iso646.h"
_kcc_hdr_n_stdnoreturn: .asciz "stdnoreturn.h"
_kcc_hdr_n_stdalign: .asciz "stdalign.h"

_kcc_hdr_stddef: .asciz "#ifndef _STDDEF_H\n#
   define _STDDEF_H\ntypedef unsigned long size_t
   ;\ntypedef long ptrdiff_t;\ntypedef long
   max_align_t;\n#define NULL ((void*)0)\n#define
   offsetof(t, m) ((size_t)&(((t*)0)->m))\n#endif\
   n"

_kcc_hdr_stdbool: .asciz "#ifndef _STDBOOL_H\n#
   define _STDBOOL_H\n#define bool int\n#define
   true 1\n#define false 0\n#endif\n"

_kcc_hdr_stdarg: .asciz "#ifndef _STDARG_H\n#
   define _STDARG_H\n#endif\n"

_kcc_hdr_stdint: .asciz "#ifndef _STDINT_H\n#
   define _STDINT_H\ntypedef signed char int8_t;\
   ntypedef unsigned char uint8_t;\ntypedef short
   int16_t;\ntypedef unsigned short uint16_t;\
   ntypedef int int32_t;\ntypedef unsigned int
   uint32_t;\ntypedef long int64_t;\ntypedef
   unsigned long uint64_t;\ntypedef long intptr_t
   ;\ntypedef unsigned long uintptr_t;\ntypedef
   long intmax_t;\ntypedef unsigned long uintmax_t
   ;\ntypedef signed char int_least8_t;\ntypedef
   unsigned char uint_least8_t;\ntypedef short
   int_least16_t;\ntypedef unsigned short
   uint_least16_t;\ntypedef int int_least32_t;\
   ntypedef unsigned int uint_least32_t;\ntypedef
   long int_least64_t;\ntypedef unsigned long
   uint_least64_t;\ntypedef signed char
   int_fast8_t;\ntypedef unsigned char
   uint_fast8_t;\ntypedef long int_fast16_t;\
   ntypedef unsigned long uint_fast16_t;\ntypedef
   long int_fast32_t;\ntypedef unsigned long
   uint_fast32_t;\ntypedef long int_fast64_t;\
   ntypedef unsigned long uint_fast64_t;\n#define
   INT8_MAX 127\n#define INT16_MAX 32767\n#define
   INT32_MAX 2147483647\n#define INT64_MAX
   9223372036854775807L\n#define UINT8_MAX 255\n#
```

```
    define UINT16_MAX 65535\n#define UINT32_MAX
    4294967295U\n#define UINT64_MAX
    18446744073709551615UL\n#define INTMAX_MAX
    9223372036854775807L\n#define UINTMAX_MAX
    18446744073709551615UL\n#define INTPTR_MAX
    9223372036854775807L\n#define UINTPTR_MAX
    18446744073709551615UL\n#define SIZE_MAX
    18446744073709551615UL\n#define INT8_MIN (-128)
    \n#define INT16_MIN (-32768)\n#define INT32_MIN
     (-2147483647-1)\n#define INT64_MIN
    (-9223372036854775807L-1)\n#define INTMAX_MIN
    (-9223372036854775807L-1)\n#define INTPTR_MIN
    (-9223372036854775807L-1)\n#endif\n"

_kcc_hdr_limits: .asciz "#ifndef _LIMITS_H\n#
    define _LIMITS_H\n#define CHAR_BIT 8\n#define
    SCHAR_MIN (-128)\n#define SCHAR_MAX 127\n#
    define UCHAR_MAX 255\n#define CHAR_MIN (-128)\n
    #define CHAR_MAX 127\n#define MB_LEN_MAX 6\n#
    define SHRT_MIN (-32768)\n#define SHRT_MAX
    32767\n#define USHRT_MAX 65535\n#define INT_MIN
     (-2147483647 - 1)\n#define INT_MAX 2147483647\
    n#define UINT_MAX 4294967295U\n#define LONG_MIN
     (-9223372036854775807L - 1)\n#define LONG_MAX
    9223372036854775807L\n#define ULONG_MAX
    18446744073709551615UL\n#define LLONG_MIN
    (-9223372036854775807LL - 1)\n#define LLONG_MAX
     9223372036854775807LL\n#define ULLONG_MAX
    18446744073709551615ULL\n#endif\n"

_kcc_hdr_float: .asciz "#ifndef _FLOAT_H\n#define
    _FLOAT_H\n#define FLT_RADIX 2\n#define
    FLT_MANT_DIG 24\n#define DBL_MANT_DIG 53\n#
    define LDBL_MANT_DIG 53\n#define FLT_DIG 6\n#
    define DBL_DIG 15\n#define LDBL_DIG 15\n#define
     FLT_MIN_EXP (-125)\n#define DBL_MIN_EXP
    (-1021)\n#define LDBL_MIN_EXP (-1021)\n#define
    FLT_MAX_EXP 128\n#define DBL_MAX_EXP 1024\n#
    define LDBL_MAX_EXP 1024\n#define
    FLT_MIN_10_EXP (-37)\n#define DBL_MIN_10_EXP
    (-307)\n#define LDBL_MIN_10_EXP (-307)\n#define
     FLT_MAX_10_EXP 38\n#define DBL_MAX_10_EXP 308\
    n#define LDBL_MAX_10_EXP 308\n#define
    FLT_ROUNDS 1\n#define FLT_EVAL_METHOD 0\n#
    define FLT_EPSILON 1.19209290e-7F\n#define
    DBL_EPSILON 2.2204460492503131e-16\n#define
    LDBL_EPSILON 2.2204460492503131e-16\n#define
    FLT_MIN 1.17549435e-38F\n#define DBL_MIN
    2.2250738585072014e-308\n#define LDBL_MIN
    2.2250738585072014e-308\n#define FLT_MAX
    3.40282347e+38F\n#define DBL_MAX
    1.7976931348623157e+308\n#define LDBL_MAX
    1.7976931348623157e+308\n#define DECIMAL_DIG
    17\n#endif\n"

_kcc_hdr_iso646: .asciz "#ifndef _ISO646_H\n#
    define _ISO646_H\n#define and &&\n#define
    and_eq &=\n#define bitand &\n#define bitor |\n#
    define compl ~\n#define not !\n#define not_eq
    !=\n#define or ||\n#define or_eq |=\n#define
    xor ^\n#define xor_eq ^=\n#endif\n"

_kcc_hdr_stdnoreturn: .asciz "#ifndef
    _STDNORETURN_H\n#define _STDNORETURN_H\n#define
     noreturn _Noreturn\n#define
    __noreturn_is_defined 1\n#endif\n"

_kcc_hdr_stdalign: .asciz "#ifndef _STDALIGN_H\n#
    define _STDALIGN_H\n#define alignas _Alignas\n#
    define alignof _Alignof\n#define
    __alignas_is_defined 1\n#define
    __alignof_is_defined 1\n#endif\n"

.data
.align 3
_kcc_pp_builtin_table:
    .quad _kcc_hdr_n_stddef,  _kcc_hdr_stddef
    .quad _kcc_hdr_n_stdbool, _kcc_hdr_stdbool
    .quad _kcc_hdr_n_stdarg,  _kcc_hdr_stdarg
    .quad _kcc_hdr_n_stdint,  _kcc_hdr_stdint
    .quad _kcc_hdr_n_limits,  _kcc_hdr_limits
    .quad _kcc_hdr_n_float,   _kcc_hdr_float
    .quad _kcc_hdr_n_iso646,  _kcc_hdr_iso646
    .quad _kcc_hdr_n_stdnoreturn,
   _kcc_hdr_stdnoreturn
    .quad _kcc_hdr_n_stdalign, _kcc_hdr_stdalign
    .quad 0, 0
.text
```

□

153 · used in 28f

17.16.3. Hosted library headers (shipped as files)

These are the convenience bridges the compiler no longer embeds. Each is a plain header fragment routed to `include/` by `assembly.weft`, so it lands on disk beside the binary and resolves through the default include directory. They read as the C they are—no escape counting, no `.asciz`—which is the whole point: a hosted header is data the library owns, and treating it as a file rather than a string baked into the preprocessor makes it trivial to correct or extend. The prototypes remain spare bridges, not faithful copies of the platform headers: enough for the front end to type each call and for the linker to bind it to `libSystem`.

`<stdio.h>` additionally exposes the standard streams `stdin`/`stdout`/`stderr` as `extern` data objects imported from `libSystem` (`__stdinp` and friends) with the usual macro aliases; the back end's `IR_GLOBAL_REF` handler loads such a dylib-imported datum through the global offset table, so `fprintf(stderr, …)` links and runs.

⟨ *Header: stdio* 154 ⟩ ≡

```
#ifndef _STDIO_H
#define _STDIO_H
#ifndef NULL
#define NULL ((void*)0)
#endif
typedef unsigned long size_t;
typedef void FILE;
#ifdef __linux__
extern FILE *stdin;
extern FILE *stdout;
extern FILE *stderr;
#else
extern FILE *__stdinp;
extern FILE *__stdoutp;
extern FILE *__stderrp;
#define stdin  __stdinp
#define stdout __stdoutp
#define stderr __stderrp
#endif
#define EOF (-1)
#define BUFSIZ 1024
#define FOPEN_MAX 20
#define FILENAME_MAX 1024
#define L_tmpnam 1024
#define SEEK_SET 0
#define SEEK_CUR 1
```

```
#define SEEK_END 2
#define _IOFBF 0
#define _IOLBF 1
#define _IONBF 2
int printf(char *fmt, ...);
int fprintf(FILE *stream, char *fmt, ...);
int sprintf(char *str, char *fmt, ...);
int snprintf(char *str, unsigned long size, char *
   fmt, ...);
int puts(char *s);
int putchar(int c);
int fputs(char *s, FILE *stream);
int fputc(int c, FILE *stream);
int getchar(void);
int scanf(char *fmt, ...);
int sscanf(char *str, char *fmt, ...);
FILE *fopen(char *path, char *mode);
int fclose(FILE *stream);
unsigned long fread(void *ptr, unsigned long size,
    unsigned long n, FILE *stream);
unsigned long fwrite(void *ptr, unsigned long size
   , unsigned long n, FILE *stream);
int fflush(FILE *stream);
int fgetc(FILE *stream);
char *fgets(char *s, int n, FILE *stream);
int getc(FILE *stream);
int ungetc(int c, FILE *stream);
int feof(FILE *stream);
int ferror(FILE *stream);
void clearerr(FILE *stream);
int fseek(FILE *stream, long offset, int whence);
long ftell(FILE *stream);
void rewind(FILE *stream);
int setvbuf(FILE *stream, char *buf, int mode,
   unsigned long size);
void setbuf(FILE *stream, char *buf);
FILE *freopen(char *path, char *mode, FILE *stream
   );
FILE *tmpfile(void);
char *tmpnam(char *s);
int fscanf(FILE *stream, char *fmt, ...);
void perror(char *s);
int remove(char *path);
int rename(char *oldp, char *newp);
#endif
```

□

154 · used in 31g

⟨ *Header: stdlib* 155a ⟩ ≡

```
#ifndef _STDLIB_H
#define _STDLIB_H
typedef unsigned long size_t;
#ifndef NULL
#define NULL ((void*)0)
#endif
#define EXIT_SUCCESS 0
#define EXIT_FAILURE 1
#define RAND_MAX 2147483647
void *malloc(unsigned long size);
void *calloc(unsigned long n, unsigned long size);
void *realloc(void *ptr, unsigned long size);
void free(void *ptr);
void exit(int status);
void abort(void);
int atoi(char *s);
long atol(char *s);
double atof(char *s);
double strtod(char *s, char **end);
float strtof(char *s, char **end);
long strtol(char *s, char **end, int base);
unsigned long strtoul(char *s, char **end, int
   base);
long long strtoll(char *s, char **end, int base);
unsigned long long strtoull(char *s, char **end,
   int base);
int abs(int x);
long labs(long x);
int rand(void);
void srand(unsigned int seed);
void qsort(void *base, unsigned long n, unsigned
   long size, int (*cmp)(void *, void *));
void *bsearch(void *key, void *base, unsigned long
    n, unsigned long size, int (*cmp)(void *, void
    *));
char *getenv(char *name);
int system(char *cmd);
#endif
```

□

155a · used in 32a

⟨ *Header: string* 155b ⟩ ≡

```
#ifndef _STRING_H
#define _STRING_H
typedef unsigned long size_t;
#ifndef NULL
#define NULL ((void*)0)
#endif
unsigned long strlen(char *s);
int strcmp(char *a, char *b);
int strncmp(char *a, char *b, unsigned long n);
char *strcpy(char *d, char *s);
char *strncpy(char *d, char *s, unsigned long n);
char *strcat(char *d, char *s);
char *strncat(char *d, char *s, unsigned long n);
char *strchr(char *s, int c);
char *strrchr(char *s, int c);
char *strstr(char *h, char *n);
char *strpbrk(char *s, char *accept);
unsigned long strspn(char *s, char *accept);
unsigned long strcspn(char *s, char *reject);
int strcoll(char *a, char *b);
char *strerror(int errnum);
void *memcpy(void *d, void *s, unsigned long n);
void *memmove(void *d, void *s, unsigned long n);
void *memset(void *d, int c, unsigned long n);
void *memchr(void *s, int c, unsigned long n);
int memcmp(void *a, void *b, unsigned long n);
#endif
```

□

155b · used in 32b

⟨ *Header: ctype* 155c ⟩ ≡

```
#ifndef _CTYPE_H
#define _CTYPE_H
int isalpha(int c);
int isdigit(int c);
int isalnum(int c);
int isspace(int c);
int isupper(int c);
int islower(int c);
int ispunct(int c);
int toupper(int c);
int tolower(int c);
#endif
```

□

155c · used in 32c

⟨ Header: math 156a ⟩ ≡

```
#ifndef _MATH_H
#define _MATH_H
#define HUGE_VAL (1e308 * 10.0)
#define INFINITY (1e308 * 10.0)
#define NAN (0.0 * (1e308 * 10.0))
double sqrt(double x);
double cbrt(double x);
double pow(double x, double y);
double hypot(double x, double y);
double sin(double x);
double cos(double x);
double tan(double x);
double asin(double x);
double acos(double x);
double atan(double x);
double atan2(double y, double x);
double sinh(double x);
double cosh(double x);
double tanh(double x);
double asinh(double x);
double acosh(double x);
double atanh(double x);
double exp(double x);
double exp2(double x);
double expm1(double x);
double log(double x);
double log10(double x);
double log2(double x);
double log1p(double x);
double frexp(double x, int *e);
double ldexp(double x, int e);
double modf(double x, double *ip);
double scalbn(double x, int n);
double fabs(double x);
double floor(double x);
double ceil(double x);
double trunc(double x);
double round(double x);
double nearbyint(double x);
double rint(double x);
double fmod(double x, double y);
double remainder(double x, double y);
double copysign(double x, double y);
double nextafter(double x, double y);
double fdim(double x, double y);
double fmax(double x, double y);
double fmin(double x, double y);
double fma(double x, double y, double z);
double nan(const char *tag);
#endif
```

□

156a · used in 32d

⟨ Header: assert 156b ⟩ ≡

```
#ifndef _ASSERT_H
#define _ASSERT_H
#define assert(x) ((void)0)
#endif
```

□

156b · used in 32e

The last two bridges show how cheap a new hosted header now is—each is a file and nothing else. `<errno.h>` is the interesting case: `errno` is not a global but a thread-local accessed through a function, and the two platforms name that function differently. When the target is Linux/x86-64 (the compiler predefines `__linux__`) the macro expands to `(*__errno_location())`, the accessor glibc provides; otherwise it expands to `(*__error())`, exactly as `libSystem` provides it. The same `__linux__`-switch picks the standard streams in `<stdio.h>`: `stderr` and friends are direct `extern FILE *` globals under glibc, but the underscored `__stderrp` accessors under `libSystem`. The `E*` constants carry the BSD values, which Linux shares for this subset. `<time.h>` gives `time_t` (a `long`) and a `struct tm` laid out to match the platform's, including the `tm_gmtoff` and `tm_zone` tail fields so a program that reads them sees the real thing.

⟨ Header: time 156c ⟩ ≡

```
#ifndef _TIME_H
#define _TIME_H
typedef long time_t;
typedef long clock_t;
#ifndef NULL
#define NULL ((void*)0)
#endif
struct tm {
    int tm_sec;
    int tm_min;
    int tm_hour;
    int tm_mday;
    int tm_mon;
    int tm_year;
    int tm_wday;
    int tm_yday;
    int tm_isdst;
    long tm_gmtoff;
    char *tm_zone;
};
time_t time(time_t *t);
double difftime(time_t end, time_t begin);
clock_t clock(void);
struct tm *localtime(time_t *t);
struct tm *gmtime(time_t *t);
time_t mktime(struct tm *tm);
char *asctime(struct tm *tm);
char *ctime(time_t *t);
unsigned long strftime(char *s, unsigned long max,
    char *fmt, struct tm *tm);
#define CLOCKS_PER_SEC 1000000
#endif
```

□

156c · used in 32f

⟨ Header: errno 156d ⟩ ≡

```
#ifndef _ERRNO_H
#define _ERRNO_H
#ifdef __linux__
int *__errno_location(void);
#define errno (*__errno_location())
#else
int *__error(void);
#define errno (*__error())
#endif
#define EPERM 1
#define ENOENT 2
#define ESRCH 3
#define EINTR 4
#define EIO 5
#define ENXIO 6
```

```
#define EBADF 9
#define ENOMEM 12
#define EACCES 13
#define EFAULT 14
#define EBUSY 16
#define EEXIST 17
#define ENOTDIR 20
#define EISDIR 21
#define EINVAL 22
#define ENOSPC 28
#define EPIPE 32
#define ERANGE 34
#endif
```

□

156d · used in 32g

The remaining hosted headers round out the C library surface. None is required of a freestanding compiler; each is a spare bridge to `libSystem`, added because it costs only a file. `<inttypes.h>` pulls in `<stdint.h>` and adds the `PRI`/`SCN` format-string macros (the `ld` family, since this is an LP64 target) with `imaxabs` and the `strtoimax` pair. `<signal.h>` and `<setjmp.h>` carry the usual handlers; the `jmp_buf` is generously sized so `setjmp` never overruns it. A caution on `<setjmp.h>`: a non-`volatile` local modified between `setjmp` and `longjmp` has an indeterminate value after the long jump (§7.13.2.1)—the program must mark such variables `volatile`, and kcc inserts no barrier of its own. `<locale.h>`, `<fenv.h>`, and the wide-character trio (`<wchar.h>`, `<wctype.h>`, `<uchar.h>`) supply the platform `struct`s and constants; `wint_t` carries a private guard so the wide headers compose without a duplicate typedef.

⟨ *Header: inttypes* 157a ⟩ ≡

```
#ifndef _INTTYPES_H
#define _INTTYPES_H
#include <stdint.h>
typedef struct { long quot; long rem; } imaxdiv_t;
long imaxabs(long n);
imaxdiv_t imaxdiv(long num, long den);
long strtoimax(char *s, char **end, int base);
unsigned long strtoumax(char *s, char **end, int
   base);
#define PRId8  "d"
#define PRId16 "d"
#define PRId32 "d"
#define PRId64 "ld"
#define PRIi64 "li"
#define PRIu8  "u"
#define PRIu16 "u"
#define PRIu32 "u"
#define PRIu64 "lu"
#define PRIx64 "lx"
#define PRIX64 "lX"
#define PRIo64 "lo"
#define PRIdPTR "ld"
#define PRIuPTR "lu"
#define PRIxPTR "lx"
#endif
```

□

157a · used in 32h

⟨ *Header: signal* 157b ⟩ ≡

```
#ifndef _SIGNAL_H
#define _SIGNAL_H
typedef int sig_atomic_t;
typedef void (*sig_handler_t)(int);
sig_handler_t signal(int sig, sig_handler_t
   handler);
int raise(int sig);
#define SIG_DFL ((sig_handler_t)0)
#define SIG_IGN ((sig_handler_t)1)
#define SIG_ERR ((sig_handler_t)-1)
#define SIGINT  2
#define SIGILL  4
#define SIGABRT 6
#define SIGFPE  8
#define SIGSEGV 11
#define SIGTERM 15
#endif
```

□

157b · used in 32i

⟨ *Header: setjmp* 157c ⟩ ≡

```
#ifndef _SETJMP_H
#define _SETJMP_H
typedef long jmp_buf[48];
int setjmp(jmp_buf env);
void longjmp(jmp_buf env, int val);
#endif
```

□

157c · used in 32j

⟨ *Header: locale* 157d ⟩ ≡

```
#ifndef _LOCALE_H
#define _LOCALE_H
#ifndef NULL
#define NULL ((void*)0)
#endif
struct lconv {
    char *decimal_point;
    char *thousands_sep;
    char *grouping;
    char *int_curr_symbol;
    char *currency_symbol;
    char *mon_decimal_point;
    char *mon_thousands_sep;
    char *mon_grouping;
    char *positive_sign;
    char *negative_sign;
    char int_frac_digits;
    char frac_digits;
    char p_cs_precedes;
    char p_sep_by_space;
    char n_cs_precedes;
    char n_sep_by_space;
    char p_sign_posn;
    char n_sign_posn;
    char int_p_cs_precedes;
    char int_n_cs_precedes;
    char int_p_sep_by_space;
    char int_n_sep_by_space;
    char int_p_sign_posn;
    char int_n_sign_posn;
};
char *setlocale(int category, char *locale);
struct lconv *localeconv(void);
#define LC_ALL      0
#define LC_COLLATE  1
#define LC_CTYPE    2
#define LC_MONETARY 3
#define LC_NUMERIC  4
#define LC_TIME     5
#endif
```

□

157d · used in 32k

⟨ *Header: fenv* 158a ⟩ ≡

```
#ifndef _FENV_H
#define _FENV_H
typedef struct { unsigned long __fpsr; unsigned
   long __fpcr; } fenv_t;
typedef unsigned long fexcept_t;
int feclearexcept(int excepts);
int fetestexcept(int excepts);
int feraiseexcept(int excepts);
int fegetround(void);
int fesetround(int round);
int fegetenv(fenv_t *envp);
int fesetenv(fenv_t *envp);
#define FE_INEXACT    0x10
#define FE_UNDERFLOW  0x08
#define FE_OVERFLOW   0x04
#define FE_DIVBYZERO  0x02
#define FE_INVALID    0x01
#define FE_ALL_EXCEPT 0x1F
#define FE_TONEAREST  0x00
#define FE_UPWARD     0x01
#define FE_DOWNWARD   0x02
#define FE_TOWARDZERO 0x03
#endif
```

□

158a · used in 32l

⟨ *Header: wchar* 158b ⟩ ≡

```
#ifndef _WCHAR_H
#define _WCHAR_H
typedef int wchar_t;
#ifndef NULL
#define NULL ((void*)0)
#endif
#ifndef _WINT_T
#define _WINT_T
typedef int wint_t;
#endif
unsigned long wcslen(wchar_t *s);
int wcscmp(wchar_t *a, wchar_t *b);
wchar_t *wcscpy(wchar_t *d, wchar_t *s);
wchar_t *wcscat(wchar_t *d, wchar_t *s);
wchar_t *wcschr(wchar_t *s, wchar_t c);
#define WEOF (-1)
#endif
```

□

158b · used in 32m

⟨ *Header: wctype* 158c ⟩ ≡

```
#ifndef _WCTYPE_H
#define _WCTYPE_H
#ifndef _WINT_T
#define _WINT_T
typedef int wint_t;
#endif
typedef unsigned long wctype_t;
int iswalpha(wint_t c);
int iswdigit(wint_t c);
int iswalnum(wint_t c);
int iswspace(wint_t c);
int iswupper(wint_t c);
int iswlower(wint_t c);
int iswpunct(wint_t c);
wint_t towupper(wint_t c);
wint_t towlower(wint_t c);
#endif
```

□

158c · used in 32n

⟨ *Header: uchar* 158d ⟩ ≡

```
#ifndef _UCHAR_H
#define _UCHAR_H
typedef unsigned short char16_t;
typedef unsigned int char32_t;
unsigned long mbrtoc16(char16_t *pc16, char *s,
   unsigned long n, void *ps);
unsigned long c16rtomb(char *s, char16_t c16, void
    *ps);
unsigned long mbrtoc32(char32_t *pc32, char *s,
   unsigned long n, void *ps);
unsigned long c32rtomb(char *s, char32_t c32, void
    *ps);
#endif
```

□

158d · used in 32o

POSIX file bridges. Two POSIX headers round out what a library's gzip file wrappers need—zlib's `gz*` layer is the motivating case. `<fcntl.h>` supplies the open-flag macros and `open`; `<unistd.h>` supplies the raw file-descriptor I/O (`read`, `write`, `close`, `lseek`) and the `SEEK_*` whence constants. Neither is an ISO C header, but the front end needs the flag values and prototypes to type each call, and the linker binds them to `libSystem`. Two details earn their keep: the open flags carry their exact Apple values, so the call reaches the platform `open` with the semantics it expects; and `lseek` returns the wide `off_t` (and `read`/`write` an `ssize_t`), so declaring them keeps a file offset or byte count from being truncated to `int` by an implicit declaration.

⟨ *Header: fcntl* 158e ⟩ ≡

```
#ifndef _FCNTL_H
#define _FCNTL_H
typedef unsigned short mode_t;
#define O_RDONLY   0x0000
#define O_WRONLY   0x0001
#define O_RDWR     0x0002
#define O_ACCMODE  0x0003
#define O_NONBLOCK 0x0004
#define O_APPEND   0x0008
#define O_CREAT    0x0200
#define O_TRUNC    0x0400
#define O_EXCL     0x0800
#define O_CLOEXEC  0x1000000
int open(const char *path, int oflag, ...);
int creat(const char *path, mode_t mode);
int fcntl(int fd, int cmd, ...);
#endif
```

□

158e · used in 32p

⟨ *Header: unistd* 158f ⟩ ≡

```
#ifndef _UNISTD_H
#define _UNISTD_H
typedef long ssize_t;
typedef long off_t;
#ifndef SEEK_SET
#define SEEK_SET 0
#define SEEK_CUR 1
#define SEEK_END 2
#endif
```

```
#define STDIN_FILENO 0
#define STDOUT_FILENO 1
#define STDERR_FILENO 2
#define F_OK 0
#define X_OK 1
#define W_OK 2
#define R_OK 4
#define _SC_PAGESIZE 30
ssize_t read(int fd, void *buf, unsigned long
   nbyte);
ssize_t write(int fd, const void *buf, unsigned
   long nbyte);
ssize_t pread(int fd, void *buf, unsigned long
   nbyte, off_t off);
ssize_t pwrite(int fd, const void *buf, unsigned
   long nbyte, off_t off);
int close(int fd);
off_t lseek(int fd, off_t offset, int whence);
int unlink(const char *path);
int rmdir(const char *path);
int isatty(int fd);
int access(const char *path, int mode);
char *getcwd(char *buf, unsigned long size);
int chdir(const char *path);
int fchdir(int fd);
int ftruncate(int fd, off_t length);
int fsync(int fd);
int fdatasync(int fd);
int fchown(int fd, unsigned uid, unsigned gid);
unsigned geteuid(void);
unsigned getuid(void);
int getpid(void);
int getpagesize(void);
ssize_t readlink(const char *path, char *buf,
   unsigned long size);
int usleep(unsigned usec);
unsigned sleep(unsigned sec);
int pipe(int *fds);
long sysconf(int name);
int gethostname(char *name, unsigned long len);
#endif
```

□

158f · used in 32q

End-to-end tests. The preprocessor already runs ahead of the code generator, so these tests compile and execute programs that lean on it: object-like and function-like macros—including both spellings of a variadic macro, the standard `.../__VA_ARGS__` form and the GNU named-rest form `args...`—conditional compilation, and—through `run_test_include`—a shared header that supplies both a macro constant and a `struct` definition to the translation unit that includes it. The last case is the one that matters for self-hosting: a project header sitting beside its `.c` file must be found and its declarations honoured on the `-c` path.

⟨ *Test: codegen preprocessor* 159a ⟩ ≡

```
echo ""
echo "--- Preprocessor ---"

run_test "pp-define-const" \
    "#define ANSWER 42
int main(void){ return ANSWER; }" 42

run_test "pp-define-arith" \
    "#define N 5
#define M 3
int main(void){ return N * M; }" 15

run_test "pp-define-funclike" \
    "#define SQ(x) ((x)*(x))
int main(void){ return SQ(6); }" 36

run_test "pp-variadic-va-args" \
    "int add3(int a,int b,int c){return a+b+c;}
#define ADD(...) add3(__VA_ARGS__)
int main(void){ return ADD(10,20,30); }" 60

run_test "pp-variadic-named" \
    "int add3(int a,int b,int c){return a+b+c;}
#define ADD(args...) add3(args)
int main(void){ return ADD(10,20,30); }" 60

run_test "pp-variadic-named-fixed" \
    "int mix(int f,int a,int b){return f*100+a*10+
   b;}
#define M(fmt, rest...) mix(fmt, rest)
int main(void){ return M(1, 2, 3); }" 123

run_test "pp-ifdef-true" \
    "#define DEBUG
int main(void){
#ifdef DEBUG
  return 7;
#else
  return 0;
#endif
}" 7

run_test "pp-ifndef-else" \
    "int main(void){
#ifdef NOPE
  return 0;
#else
  return 9;
#endif
}" 9

run_test_include "pp-include-macro" \
    "#define LIMIT 7" \
    "#include \"defs.h\"
int main(void){ return LIMIT + 3; }" 10

run_test_include "pp-include-struct" \
    "struct Pair { int a; int b; };" \
    "#include \"defs.h\"
int main(void){ struct Pair p; p.a = 4; p.b = 6;
   return p.a + p.b; }" 10
```

□

159a · used in 33b

17.17. Error Directive

`#error` emits a diagnostic with the rest of the directive line as the message.

⟨ *PP: error directive* 159b ⟩ ≡

```
.text
// -- _kcc_pp_do_error ---
.p2align 2
_kcc_pp_do_error:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
```

```
    // Get rest of line as message
    bl _kcc_pp_rest_of_line
    mov x19, x0                 // msg ptr
    mov x20, x1                 // msg len

    // Create null-terminated copy in arena
    add  x0, x20, #1
    bl _kcc_arena_alloc
    mov  x2, x0                 // dst
    mov  x0, x2
    mov  x1, x19
    mov  x3, x20
    // memcpy(dst, src, len)
    mov  x1, x19
    mov  x2, x20
    bl _kcc_memcpy

    // Null terminate
    add  x1, x0, x20
    strb wzr, [x1]

    // Emit error
    adrp x1, _kcc_pp_dir_line@PAGE
    add  x1, x1, _kcc_pp_dir_line@PAGEOFF
    ldr  x1, [x1]               // line
    mov  x2, x0                 // msg (arena copy,
  null-terminated)
    mov  x0, x1                  // line
    mov  x1, #1                 // col
    bl _kcc_diag_error

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

159b · used in 28f · uses _kcc_arena_alloc 58a, _kcc_diag_error 76a, _kcc_pp_dir_line 116

18. String Literal Processing

Phase 5 of the C17 translation pipeline processes escape sequences in character constants and string literals (§5.1.1.2, paragraph 5). At this point the preprocessor has already completed: we have a stream of preprocessing tokens. Phase 5 examines each `TOK_STRING_LIT` and `TOK_CHAR_LIT` token and replaces escape sequences with their execution-character-set values.

18.1. Escape Sequence Table

The C17 standard (§6.4.4.4) defines these simple escape sequences:

Sequence	Name	Value	C17 ref
\a	alert	0x07	§6.4.4.4/7
\b	backspace	0x08	§6.4.4.4/7
\f	form feed	0x0C	§6.4.4.4/7
\n	newline	0x0A	§6.4.4.4/7
\r	carriage return	0x0D	§6.4.4.4/7
\t	horizontal tab	0x09	§6.4.4.4/7
\v	vertical tab	0x0B	§6.4.4.4/7
\\	backslash	0x5C	§6.4.4.4/7
\'	single quote	0x27	§6.4.4.4/7
\"	double quote	0x22	§6.4.4.4/7
\?	question mark	0x3F	§6.4.4.4/7
\0	null	0x00	§6.4.4.4/6

Additionally, \x*NN* is a hexadecimal escape (§6.4.4.4/8) and *NNN* is an octal escape (§6.4.4.4/6).

18.2. Implementation

The processor walks each string/character literal byte by byte, copying characters to an arena-allocated output buffer. When a backslash is encountered, the following character determines which escape to emit.

We process tokens in-place in the token stream: after processing, the token's start pointer is updated to the new arena buffer, and the length is updated to the processed length.

⟨ *StringLit: process* 161 ⟩ ≡

```
// -- String literal escape processing ---
// kcc_strlit_process()
// Processes all pending string and char literal
   tokens.
// Called after preprocessing completes.
//
// For the initial implementation, we process a
   single token:
// x0 = pointer to token start (past the opening
   quote)
// x1 = length (excluding quotes)
// Returns: x0 = pointer to processed buffer, x1 =
    new length
.text
.globl _kcc_strlit_process
.p2align 2
_kcc_strlit_process:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0                    // x19 = src
   pointer
    mov x20, x1                    // x20 = src
   length
    add x21, x19, x20              // x21 = src end

    // Allocate output buffer (same size as input
   - escapes shrink)
    mov x0, x20
    add x0, x0, #1                 // +1 for safety
    bl _kcc_arena_alloc
    mov x22, x0                    // x22 = dst
   base
    mov x23, x0                    // x23 = dst
   cursor

.Lstrlit_loop:
    cmp x19, x21
    b.ge .Lstrlit_done

    ldrb w0, [x19]
    cmp w0, #'\\'
    b.eq .Lstrlit_escape

    // Ordinary character - copy directly
    strb w0, [x23]
    add x23, x23, #1
    add x19, x19, #1
    b .Lstrlit_loop

.Lstrlit_escape:
    add x19, x19, #1               // skip backslash
    cmp x19, x21
    b.ge .Lstrlit_done             // trailing
   backslash (error, but don't crash)

    ldrb w0, [x19]
    add x19, x19, #1               // consume escape
    char

    // Simple escape dispatch
    cmp w0, #'n'
    b.eq .Lstrlit_esc_n
    cmp w0, #'t'
    b.eq .Lstrlit_esc_t
    cmp w0, #'\\'
    b.eq .Lstrlit_esc_bs
    cmp w0, #'"'
    b.eq .Lstrlit_esc_dq
    cmp w0, #'\''
    b.eq .Lstrlit_esc_sq
    cmp w0, #'0'
    b.eq .Lstrlit_esc_0_or_oct
    cmp w0, #'a'
    b.eq .Lstrlit_esc_a
    cmp w0, #'b'
    b.eq .Lstrlit_esc_b
    cmp w0, #'f'
    b.eq .Lstrlit_esc_f
    cmp w0, #'r'
    b.eq .Lstrlit_esc_r
    cmp w0, #'v'
    b.eq .Lstrlit_esc_v
```

```
    cmp w0, #'?'
    b.eq .Lstrlit_esc_q
    cmp w0, #'x'
    b.eq .Lstrlit_esc_hex

    // Check for octal (1-7)
    cmp w0, #'1'
    b.lt .Lstrlit_esc_unknown
    cmp w0, #'7'
    b.le .Lstrlit_esc_octal

.Lstrlit_esc_unknown:
    // Unknown escape - copy the character
   literally
    // (§6.4.4.4 constraint violation, but we emit
    a diagnostic and continue)
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop
```

□

161 · defined at 161, 162ab · used in 28f · defines _kcc_strlit_process 165, 215 · uses _kcc_arena_alloc 58a

Simple escape handlers. Each single-character escape sequence maps to a fixed byte value defined by the standard. The handler stores the replacement byte into the output buffer at x23, advances the cursor, and branches back to the main loop. These thirteen cases cover every simple escape in §6.4.4.4 paragraph 7, plus the null character \0 from paragraph 6.

⟨ *StringLit: process* 162a ⟩ + ≡

```
.Lstrlit_esc_n:
    mov w0, #0x0A
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_t:
    mov w0, #0x09
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_bs:
    mov w0, #0x5C
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_dq:
    mov w0, #0x22
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_sq:
    mov w0, #0x27
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_0_or_oct:
    // Could be \0 (null) or start of octal
   sequence \0NN
    // Check if next char is an octal digit
    cmp x19, x21
    b.ge .Lstrlit_emit_null        // end of
   string, just \0
    ldrb w24, [x19]
    cmp w24, #'0'
    b.lt .Lstrlit_emit_null
    cmp w24, #'7'
    b.gt .Lstrlit_emit_null
    // It's an octal sequence starting with 0
    mov w0, #0                     // initial value
    = 0
    b .Lstrlit_oct_body

.Lstrlit_emit_null:
    mov w0, #0x00
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_a:
    mov w0, #0x07
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_b:
    mov w0, #0x08
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_f:
    mov w0, #0x0C
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_r:
    mov w0, #0x0D
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_v:
    mov w0, #0x0B
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_q:
    mov w0, #0x3F
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop
```

□

162a · defined at 161, 162ab · used in 28f

Numeric escape sequences. Octal escapes (*NNN*, §6.4.4.4/6) and hexadecimal escapes (\x*NN*, §6.4.4.4/8) require multi-digit accumulation. Octal sequences consume up to three digits and shift left by three per digit; hexadecimal sequences consume an unlimited number of digits (the standard allows it) and shift left by four. Both truncate the final value to a single byte before storing it in the output buffer. The function epilogue restores callee-saved registers and returns the processed buffer pointer and length.

⟨ *StringLit: process* 162b ⟩ + ≡

```
.Lstrlit_esc_octal:
    // w0 already has the first octal digit
   ('1'-'7')
```

```
    sub w0, w0, #'0'            // convert to
   value
.Lstrlit_oct_body:
    // w0 = accumulated value so far
    // Read up to 2 more octal digits
    mov w24, #2                  // max 2 more
   digits
.Lstrlit_oct_digit:
    cmp x19, x21
    b.ge .Lstrlit_oct_emit
    cbz w24, .Lstrlit_oct_emit
    ldrb w1, [x19]
    cmp w1, #'0'
    b.lt .Lstrlit_oct_emit
    cmp w1, #'7'
    b.gt .Lstrlit_oct_emit
    // Accumulate: val = val * 8 + digit
    lsl w0, w0, #3
    sub w1, w1, #'0'
    add w0, w0, w1
    add x19, x19, #1
    sub w24, w24, #1
    b .Lstrlit_oct_digit

.Lstrlit_oct_emit:
    and w0, w0, #0xFF           // truncate to
   byte
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_esc_hex:
    // \xNN - read hex digits (at least one
   required by standard)
    mov w0, #0                  // accumulated
   value
    mov w24, #0                 // digit count
.Lstrlit_hex_digit:
    cmp x19, x21
    b.ge .Lstrlit_hex_emit
    ldrb w1, [x19]
    // Check '0'-'9'
    cmp w1, #'0'
    b.lt .Lstrlit_hex_emit
    cmp w1, #'9'
    b.le .Lstrlit_hex_09
    // Check 'a'-'f'
    cmp w1, #'a'
    b.lt .Lstrlit_hex_check_upper
    cmp w1, #'f'
    b.le .Lstrlit_hex_af
    b .Lstrlit_hex_check_upper

.Lstrlit_hex_check_upper:
    cmp w1, #'A'
    b.lt .Lstrlit_hex_emit
    cmp w1, #'F'
    b.gt .Lstrlit_hex_emit
    // 'A'-'F'
    sub w1, w1, #'A'
    add w1, w1, #10
    b .Lstrlit_hex_accum

.Lstrlit_hex_09:
    sub w1, w1, #'0'
    b .Lstrlit_hex_accum

.Lstrlit_hex_af:
    sub w1, w1, #'a'
    add w1, w1, #10

.Lstrlit_hex_accum:
    lsl w0, w0, #4
    add w0, w0, w1
    add x19, x19, #1
    add w24, w24, #1
    b .Lstrlit_hex_digit

.Lstrlit_hex_emit:
    and w0, w0, #0xFF           // truncate to
   byte
    strb w0, [x23]
    add x23, x23, #1
    b .Lstrlit_loop

.Lstrlit_done:
    // Return processed buffer and length
    mov x0, x22                  // processed
   buffer base
    sub x1, x23, x22            // processed
   length

    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

162b · defined at 161, 162ab · used in 28f

18.3. Processing All Tokens

The `kcc_strlit_process_all` function walks the preprocessed token stream and processes each string or character literal in place. It calls `kcc_pp_next` (or `kcc_lex_next` depending on the phase) to iterate tokens, and for each `TOK_STRING_LIT` or `TOK_CHAR_LIT`, calls the escape processor above.

For Phase 3 integration, we provide a simpler approach: the preprocessor or parser calls `kcc_strlit_process` on individual token buffers as needed. Full stream processing is deferred to the parser phase.

⟨ *StringLit: data* 163 ⟩ ≡

```
// -- String literal data (reserved for future use
   ) ---
.data
.p2align 3
```

□

163 · used in 28f

19. String Literal Concatenation

Phase 6 of the C17 translation pipeline concatenates adjacent string literal tokens (§5.1.1.2, paragraph 6; §6.4.5/4). After Phase 5 has processed escape sequences, any sequence of adjacent `TOK_STRING_LIT` tokens is merged into a single string literal.

19.1. Algorithm

The gather runs where the parser reduces a primary expression to a string literal (§22). The current token is a `TOK_STRING_LIT`; `_kcc_strlit_gather` processes its escapes, copies the result into a growable arena buffer, then advances with `_kcc_pp_next` and, so long as the next token is another string literal, appends it too. Two points make this the right seam rather than a lexer-level pass. First, using `_kcc_pp_next` means an adjacent literal produced by a *macro*—`"%" PRId64 "\n"`, where `PRId64` expands to `"ld"`—is seen and concatenated like any other, which a raw lexer peek would miss. Second, the accumulation buffer doubles on demand, so a concatenation of any size is handled without a fixed ceiling. The loop leaves the first non-string token current, exactly as a plain single-literal parse would, so the caller continues unchanged.

⟨ *Concat: adjacent strings* 165 ⟩ ≡

```
// -- Adjacent string literal concatenation (
   translation phase 6) ---
// char *_kcc_strlit_gather(uint64_t *out_len)  [
   x1 returned, not by pointer]
//   On entry the current token is a
   TOK_STRING_LIT.  Processes it and every
//   adjacent string literal (through macro
   expansion, via _kcc_pp_next),
//   accumulating the escaped contents into one
   growable arena buffer.
//   Output: x0 = buffer, x1 = total length.  The
   next non-string token is
//   left current.
.text
.globl _kcc_strlit_gather
.p2align 2
_kcc_strlit_gather:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]

    mov x0, #256
    bl _kcc_arena_alloc
    mov x21, x0                    // buffer base
    mov x22, #0                    // used length
    mov x23, #256                  // capacity

.Lgather_one:
    // Process this literal's escapes into a fresh
    buffer (x24=ptr, x25=len)
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    add  x0, x0, #1                // skip opening
   quote
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    sub  x1, x1, #2                // content
   length (drop quotes)
    bl _kcc_strlit_process
    mov  x24, x0                   // processed ptr
    mov  x25, x1                   // processed len

    // Grow the buffer if used + len would
   overflow the capacity
    add  x0, x22, x25              // need
    cmp  x0, x23
    b.ls .Lgather_fits
    mov  x20, x23                  // newcap
.Lgather_grow:
    lsl  x20, x20, #1
    cmp  x20, x0
    b.lo .Lgather_grow
    mov  x0, x20
    bl _kcc_arena_alloc
    mov  x19, x0                   // new base
    mov  x0, x19
    mov  x1, x21
    mov  x2, x22
    bl _kcc_memcpy                 // copy the bytes
    so far
    mov  x21, x19                  // base = new
    mov  x23, x20                  // cap = newcap
.Lgather_fits:
    add  x0, x21, x22              // dst = base +
   used
    mov  x1, x24                   // src =
   processed content
    mov  x2, x25                   // len
    bl _kcc_memcpy
    add  x22, x22, x25             // used += len

    // Consume this literal; continue while the
   next token is a string literal
    bl _kcc_pp_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_STRING_LIT
    b.eq .Lgather_one

    mov  x0, x21                   // buffer base
    mov  x1, x22                   // total length
    ldr x25, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

165 · used in 28f · defines _kcc_strlit_gather 215, 226, 227 · uses _kcc_arena_alloc 58a, _kcc_pp_next 108, _kcc_strlit_process 161, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

19.2. Data

Reserved for future use (e.g., wide string concatenation tracking).

⟨ *Concat: data* 166 ⟩ ≡

```
// -- Concatenation data (reserved) ---
.data
.p2align 3
```

□

166 · used in 28f

20. Recursive-Descent Parser

The recursive-descent parser consumes the preprocessed token stream and builds an abstract syntax tree following the grammar of §6.9. This chapter establishes the parser's core infrastructure: a uniform 64-byte AST node layout, arena-based allocation, token consumption helpers, error recovery, and typedef disambiguation. It also provides the top-level `kcc_parse` entry point that produces an `AST_TRANS_UNIT` node, and the `kcc_ast_dump` facility for inspecting the resulting tree. The actual parsing of declarations, expressions, and statements is split across the three chapters that follow.

20.1. AST Node Layout

Every AST node is exactly 64 bytes, arena-allocated. This uniform size makes allocation trivial—a single call to `kcc_arena_alloc(64)`—and field access a constant offset from the node pointer. The five payload fields (`F0`–`F4`) carry node-specific data: child pointers, name pointers, literal values, or operator codes. `F4` is conventionally used as a `next` pointer for sibling-linked lists (parameter lists, statement lists, declaration lists).

Offset	Field	Description
0	`tag`	Node type constant (`AST_*`)
8	`line`	Source line number
16	`col`	Source column number
24	`F0`	Payload field 0 (node-specific)
32	`F1`	Payload field 1
40	`F2`	Payload field 2
48	`F3`	Payload field 3
56	`F4`	Payload field 4 (usually: next sibling)

⟨ *Parser: AST constants* 167 ⟩ ≡

```
// -- AST node layout ---
.equ AST_TAG,    0        // node type constant
.equ AST_LINE,   8        // source line
.equ AST_COL,    16       // source column
.equ AST_F0,     24       // field 0 (node-
   specific)
.equ AST_F1,     32       // field 1
.equ AST_F2,     40       // field 2
.equ AST_F3,     48       // field 3
.equ AST_F4,     56       // field 4 (usually:
   next sibling)
.equ AST_SIZE,   64       // total node size

// -- AST node type constants ---

// Translation unit
.equ AST_TRANS_UNIT,      1   // F0=first decl

// Declarations (10-19)
.equ AST_VAR_DECL,        10   // F0=type, F1=
   name_ptr, F2=init, F3=storage_class
.equ AST_FUNC_DECL,       11   // F0=type, F1=
   name_ptr, F2=params, F3=body
.equ AST_PARAM_DECL,      12   // F0=type, F1=
   name_ptr
.equ AST_TYPEDEF_DECL,    13   // F0=type, F1=
   name_ptr
.equ AST_STRUCT_DECL,     14   // F0=name_ptr, F1=
   fields
.equ AST_UNION_DECL,      15   // F0=name_ptr, F1=
   fields
.equ AST_ENUM_DECL,       16   // F0=name_ptr, F1=
   enumerators
.equ AST_ENUMERATOR,      17   // F0=name_ptr, F1=
   value_expr
.equ AST_FIELD_DECL,      18   // F0=type, F1=
   name_ptr, F2=bitwidth
.equ AST_STATIC_ASSERT,   19   // F0=expr, F1=
   msg_ptr

// Type specifiers (20-29)
.equ AST_TYPE_BUILTIN,    20   // F0=token_code
.equ AST_TYPE_NAME,       21   // F0=name_ptr
.equ AST_TYPE_PTR,        22   // F0=pointee_type,
   F1=qualifiers
.equ AST_TYPE_ARRAY,      23   // F0=elem_type, F1=
   size_expr
.equ AST_TYPE_FUNC,       24   // F0=return_type,
   F1=params, F2=variadic
.equ AST_TYPE_STRUCT,     25   // F0=struct_decl
.equ AST_TYPE_UNION,      26   // F0=union_decl
.equ AST_TYPE_ENUM,       27   // F0=enum_decl
.equ AST_TYPE_QUALS,      28   // F0=base_type, F1=
   quals
.equ AST_TYPE_TYPEOF,     29   // F0=operand (type-
   name or expr), F1=is_type (1/0)

// Statements (30-44)
.equ AST_COMPOUND,        30   // F0=first_item
.equ AST_EXPR_STMT,       31   // F0=expr
.equ AST_IF_STMT,         32   // F0=cond, F1=then,
    F2=else
.equ AST_SWITCH_STMT,     33   // F0=expr, F1=body
.equ AST_WHILE_STMT,      34   // F0=cond, F1=body
.equ AST_DO_STMT,         35   // F0=body, F1=cond
.equ AST_FOR_STMT,        36   // F0=init, F1=cond,
    F2=incr, F3=body
.equ AST_GOTO_STMT,       37   // F0=label_name_ptr
.equ AST_CONTINUE_STMT,   38
.equ AST_BREAK_STMT,      39
.equ AST_RETURN_STMT,     40   // F0=expr (or 0)
.equ AST_LABEL_STMT,      41   // F0=name_ptr, F1=
   stmt
.equ AST_CASE_STMT,       42   // F0=expr, F1=stmt
.equ AST_DEFAULT_STMT,    43   // F0=stmt
.equ AST_NULL_STMT,       44   // empty statement

// Expressions (50-74)
.equ AST_IDENT_EXPR,      50   // F0=name_ptr
.equ AST_INT_LIT,         51   // F0=value
.equ AST_STR_LIT,         52   // F0=str_ptr, F1=
   str_len
.equ AST_CHAR_LIT,        53   // F0=value
.equ AST_BINARY_EXPR,     54   // F0=op, F1=lhs, F2
   =rhs
.equ AST_UNARY_EXPR,      55   // F0=op, F1=operand
   , F2=prefix(1)/postfix(0)
.equ AST_ASSIGN_EXPR,     56   // F0=op, F1=lhs, F2
   =rhs
.equ AST_COND_EXPR,       57   // F0=cond, F1=then,
    F2=else
```

```
.equ AST_CALL_EXPR,      58   // F0=callee, F1=
   first_arg
.equ AST_SUBSCRIPT,      59   // F0=array, F1=
   index
.equ AST_MEMBER,         60   // F0=object, F1=
   name_ptr, F2=arrow(1)/dot(0)
.equ AST_CAST_EXPR,      61   // F0=type, F1=expr
.equ AST_SIZEOF_EXPR,    62   // F0=operand, F1=
   is_type
.equ AST_ALIGNOF_EXPR,   63   // F0=type
.equ AST_COMMA_EXPR,     64   // F0=lhs, F1=rhs
.equ AST_INIT_LIST,      65   // F0=first_init
.equ AST_DESIGNATOR,     66   // F0=index_or_name,
    F1=is_field, F2=next_desig
.equ AST_COMPOUND_LIT,   67   // F0=type, F1=
   init_list
.equ AST_PAREN_EXPR,     68   // F0=inner
.equ AST_GENERIC,        69   // F0=
   controlling_expr, F1=first_assoc
.equ AST_GENERIC_ASSOC,  70   // F0=type (0=
   default), F1=expr
.equ AST_FLOAT_LIT,      71   // F0=double bit
   pattern, F3=type(float/double)
.equ AST_VA_ARG,         72   // F0=va_list expr,
   F1=type node, F3=resolved type
.equ AST_STMT_EXPR,      73   // GNU ({...}): F0=
   compound, F3=resolved type
.equ AST_ASM_STMT,       74   // inline asm: F0=
   template ptr, F1=len, F2=operands, F3=noperands
.equ AST_ASM_OPERAND,    75   // F0=constraint ptr
   , F1=constraint len, F2=expr, F3=is_output, F4=
   next
.equ AST_TYPES_COMPAT,   76   //
   __builtin_types_compatible_p: F0=type1, F1=
   type2 → int const 0/1
```

□

167 · used in 28g, 29abcdef, 30ab, 31ac

20.2. AST Node Allocation

`kcc_ast_alloc` allocates a fresh 64-byte node from the arena, zeros all fields, then stamps the tag and current source location (from the lexer's token globals). Every parse function that creates a node calls this helper first.

⟨ *Parser: AST alloc* 168a ⟩ ≡

```
// -- kcc_ast_alloc ---
// node *kcc_ast_alloc(uint64_t tag)
//   x0 = AST node type constant
// Returns pointer to zeroed, tagged 64-byte node.
.text
.globl _kcc_ast_alloc
.p2align 2
_kcc_ast_alloc:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    mov x19, x0                  // x19 = tag

    // Allocate 64 bytes
    mov x0, #AST_SIZE
    bl _kcc_arena_alloc
    mov x20, x0                  // x20 = node
   pointer

    // Zero all 8 quadwords
    stp xzr, xzr, [x20]
    stp xzr, xzr, [x20, #16]
    stp xzr, xzr, [x20, #32]
    stp xzr, xzr, [x20, #48]

    // Set tag
    str x19, [x20, #AST_TAG]

    // Set source location from current token
    adrp x0, _kcc_tok_line@PAGE
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x20, #AST_LINE]

    adrp x0, _kcc_tok_col@PAGE
    add  x0, x0, _kcc_tok_col@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x20, #AST_COL]

    mov x0, x20                  // return node
   pointer

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

168a · used in 28g · defines _kcc_ast_alloc 172, 184b, 188, 189, 190, 191ab, 192, 195, 196, 199ab, 200b, 203, 205a, 207b, 208ab, 209ab, 210, 211, 212, 214, 215, 218, 225, 226, 227, 228, 230, 231ab, 232ab, 233, 234abc, 277, 325 · uses _kcc_arena_alloc 58a, _kcc_tok_col 104b, _kcc_tok_line 104b

20.3. Token Consumption Helpers

Three small functions provide the parser's interface to the token stream. `kcc_parse_peek` returns the current token type without consuming it. `kcc_parse_match` consumes the current token if it matches the given type. `kcc_parse_expect` consumes and asserts the type, emitting a diagnostic on mismatch.

⟨ *Parser: helpers* 168b ⟩ ≡

```
// -- kcc_parse_peek ---
// uint64_t kcc_parse_peek(void)
// Returns the current token type without
   consuming it.
.globl _kcc_parse_peek
.p2align 2
_kcc_parse_peek:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    ldp x29, x30, [sp], #16
    ret

// -- kcc_parse_advance ---
// void kcc_parse_advance(void)
// Consumes the current token by calling pp_next.
.globl _kcc_parse_advance
.p2align 2
_kcc_parse_advance:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
```

```
    bl _kcc_pp_next

    ldp x29, x30, [sp], #16
    ret

// -- kcc_parse_match ---
// int kcc_parse_match(uint64_t tok_type)
//   x0 = expected token type
// If current token matches, consume it and return
    1.  Otherwise return 0.
.globl _kcc_parse_match
.p2align 2
_kcc_parse_match:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                     // x19 = expected
    type

    // Check current token
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
    ldr  x1, [x1]
    cmp  x1, x19
    b.ne .Lmatch_fail

    // Match: consume and return 1
    bl _kcc_pp_next
    mov x0, #1
    b .Lmatch_done

.Lmatch_fail:
    mov x0, #0

.Lmatch_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_parse_expect ---
// void kcc_parse_expect(uint64_t tok_type)
//   x0 = expected token type
// If current token matches, consume it.
   Otherwise emit error.
.globl _kcc_parse_expect
.p2align 2
_kcc_parse_expect:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                     // x19 = expected
    type

    // Check current token
    adrp x1, _kcc_tok_type@PAGE
    add  x1, x1, _kcc_tok_type@PAGEOFF
    ldr  x1, [x1]
    cmp  x1, x19
    b.ne .Lexpect_error

    // Match: consume
    bl _kcc_pp_next
    b .Lexpect_done

.Lexpect_error:
    // Pick a specific "expected 'X'" message for
   a known punctuator; fall back
    // to the generic message otherwise.  x19 =
   expected token type.
    adrp x2, _kcc_pe_table@PAGE
    add  x2, x2, _kcc_pe_table@PAGEOFF
.Lexpect_look:
    ldr x3, [x2]                    // candidate
   token type (0 terminates)
    cbz x3, .Lexpect_generic
    cmp x3, x19
    b.eq .Lexpect_found
    add x2, x2, #16
    b .Lexpect_look
.Lexpect_found:
    ldr x0, [x2, #8]
    b .Lexpect_emit
.Lexpect_generic:
    adrp x0, _kcc_parse_msg_expected@PAGE
    add  x0, x0, _kcc_parse_msg_expected@PAGEOFF
.Lexpect_emit:
    bl _kcc_parse_error

.Lexpect_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

168b · used in 28g · defines _kcc_parse_advance never used, _kcc_parse_expect 181, 182, 184ab, 186b, 191b, 192, 196, 199ab, 200b, 203, 205a, 208b, 210, 211, 212, 218, 226, 227, 228, 230, 231ab, 232ab, 233, 234abc, _kcc_parse_match never used, _kcc_parse_peek never used · uses _kcc_parse_error 169, _kcc_pp_next 108, _kcc_tok_type 104b

20.4. Error Recovery

When the parser encounters an unexpected token, it needs to recover gracefully rather than cascading errors. kcc_parse_error emits a diagnostic and sets the global error flag. kcc_parse_sync skips tokens until it finds a synchronization point: a semicolon, closing brace, or end of file.

⟨ *Parser: error recovery* 169 ⟩ ≡

```
// -- kcc_parse_error ---
// void kcc_parse_error(const char *msg)
//   x0 = error message string
// Emits an error diagnostic at the current token
   location.
.globl _kcc_parse_error
.p2align 2
_kcc_parse_error:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                     // x19 = message

    // Set error flag
    adrp x1, _kcc_parse_error_flag@PAGE
    add  x1, x1, _kcc_parse_error_flag@PAGEOFF
    mov  x2, #1
    str  x2, [x1]

    // Emit diagnostic: error at current token
   location
    mov x0, #DIAG_ERROR
    adrp x1, _kcc_tok_line@PAGE
    add  x1, x1, _kcc_tok_line@PAGEOFF
    ldr  x1, [x1]
    adrp x2, _kcc_tok_col@PAGE
    add  x2, x2, _kcc_tok_col@PAGEOFF
    ldr  x2, [x2]
    mov  x3, x19
    bl _kcc_diag_emit
```

```
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_parse_sync ---
// void kcc_parse_sync(void)
// Skips tokens until ';', '}', or EOF is found.
// Consumes ';' if that is the sync point.
.globl _kcc_parse_sync
.p2align 2
_kcc_parse_sync:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

.Lsync_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    // EOF - stop
    cbz  x0, .Lsync_done

    // ';' - consume and stop
    cmp  x0, #TOK_SEMI
    b.eq .Lsync_consume

    // '}' - stop without consuming
    cmp  x0, #TOK_RBRACE
    b.eq .Lsync_done

    // Skip this token
    bl _kcc_pp_next
    b .Lsync_loop

.Lsync_consume:
    bl _kcc_pp_next

.Lsync_done:
    ldp x29, x30, [sp], #16
    ret
```

□

169 · used in 28g · defines _kcc_parse_error 168b, 215, _kcc_parse_sync 184a · uses _kcc_diag_emit 73c, _kcc_parse_error_flag 178a, _kcc_pp_next 108, _kcc_tok_col 104b, _kcc_tok_line 104b, _kcc_tok_type 104b

20.5. Typedef Disambiguation

The classic ambiguity in C parsing: given `T * x;`, is this a multiplication expression or a pointer declaration? The answer depends on whether `T` is a typedef name. We maintain a scope-aware stack of hash tables. Each compound statement pushes a new scope, and typedef declarations register names in the current (topmost) scope. Lookup searches from top to bottom.

⟨ *Parser: typedef table* 170a ⟩ ≡

```
// -- kcc_typedef_init ---
// void kcc_typedef_init(void)
// Initializes the typedef scope stack with a
   global scope.
.globl _kcc_typedef_init
.p2align 2
_kcc_typedef_init:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Create a vector of hash table pointers (
   elem_size = 8)
    mov x0, #8
    bl _kcc_vec_init

    // Store the scope stack
    adrp x1, _kcc_typedef_scopes@PAGE
    add  x1, x1, _kcc_typedef_scopes@PAGEOFF
    str  x0, [x1]
    mov  x19, x0                  // x19 = scope
   stack

    // Push global scope (hash table with capacity
    64)
    mov x0, #64
    bl _kcc_ht_init
    // x0 = global scope hash table

    // Push it onto the scope stack
    // vec_push expects pointer to element, so
   store on stack
    str x0, [sp, #24]            // temporary
   storage for the pointer value
    mov x0, x19                   // vec
    add x1, sp, #24               // pointer to
   element (the ht pointer)
    bl _kcc_vec_push

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

170a · defined at 170ab, 171 · used in 28g · defines _kcc_typedef_init 603, 622, 623 · uses _kcc_ht_init 61c, _kcc_typedef_scopes 178a, _kcc_vec_init 64b, _kcc_vec_push 65a

Scope Push and Pop The parser must track typedef visibility across nested scopes. Each time a compound statement (block) is entered, `kcc_typedef_push_scope` allocates a fresh hash table and pushes it onto the scope stack vector. When the block ends, `kcc_typedef_pop_scope` simply decrements the vector's count—the hash table memory is reclaimed when the arena is freed.

⟨ *Parser: typedef table* 170b ⟩ + ≡

```
// -- kcc_typedef_push_scope ---
// void kcc_typedef_push_scope(void)
// Pushes a new empty scope onto the typedef stack
   .
.globl _kcc_typedef_push_scope
.p2align 2
_kcc_typedef_push_scope:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Allocate a new hash table
    mov x0, #32
    bl _kcc_ht_init
    mov x19, x0                   // x19 = new
   scope ht

    // Push onto scope stack
    str x19, [sp, #24]
    adrp x0, _kcc_typedef_scopes@PAGE
    add  x0, x0, _kcc_typedef_scopes@PAGEOFF
    ldr  x0, [x0]
    add  x1, sp, #24
```

```
    bl _kcc_vec_push

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_typedef_pop_scope ---
// void kcc_typedef_pop_scope(void)
// Pops the topmost scope from the typedef stack.
.globl _kcc_typedef_pop_scope
.p2align 2
_kcc_typedef_pop_scope:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Decrement the count in the scope vector
    adrp x0, _kcc_typedef_scopes@PAGE
    add  x0, x0, _kcc_typedef_scopes@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [x0, #VEC_COUNT]
    sub  x1, x1, #1
    str  x1, [x0, #VEC_COUNT]

    ldp x29, x30, [sp], #16
    ret
```

□

170b · defined at 170ab, 171 · used in 28g · defines _kcc_typedef_pop_scope 230, _kcc_typedef_push_scope 230 · uses _kcc_ht_init 61c, _kcc_typedef_scopes 178a, _kcc_vec_push 65a

Name Registration and Lookup kcc_typedef_add registers a name in the topmost scope by retrieving the last hash table on the scope stack and inserting the name with a sentinel value of 1. kcc_typedef_lookup searches scopes from top to bottom, returning 1 on the first hit. This top-down search ensures that an inner scope's typedef shadows an outer variable of the same name, matching the C17 scoping rules §6.2.1. Before the scope scan it recognises two built-in type names that no user declaration introduces: va_list and its GNU spelling __builtin_va_list (the latter is what a freestanding <stdarg.h>, such as the kernel's under -nostdinc, typedefs va_list from). Both resolve to the opaque eight-byte pointer sema hands back (Section 29.9.13).

⟨ *Parser: typedef table* 171 ⟩ +≡

```
// -- kcc_typedef_add ---
// void kcc_typedef_add(const char *name, uint64_t
    len)
//   x0 = pointer to name
//   x1 = name length
// Adds a typedef name to the current (topmost)
   scope.
.globl _kcc_typedef_add
.p2align 2
_kcc_typedef_add:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                     // x19 = name
    mov x20, x1                     // x20 = len

    // Get topmost scope: scopes.data[(count-1)]
    adrp x0, _kcc_typedef_scopes@PAGE
    add  x0, x0, _kcc_typedef_scopes@PAGEOFF
    ldr  x0, [x0]                   // x0 = scope vec
    ldr  x1, [x0, #VEC_COUNT]
    sub  x1, x1, #1                 // last index
    bl _kcc_vec_get
    // x0 = pointer to element (pointer to ht
   pointer)
    ldr  x21, [x0]                  // x21 = ht
   pointer

    // ht_put(ht, name, len, 1)  - value 1 = "is a
    typedef"
    mov x0, x21
    mov x1, x19
    mov x2, x20
    mov x3, #1
    bl _kcc_ht_put

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_typedef_lookup ---
// int kcc_typedef_lookup(const char *name,
   uint64_t len)
//   x0 = pointer to name
//   x1 = name length
// Returns 1 if name is a typedef, 0 otherwise.
// Searches scopes from top to bottom.
.globl _kcc_typedef_lookup
.p2align 2
_kcc_typedef_lookup:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0                     // x19 = name
    mov x20, x1                     // x20 = len

    // Built-in typedef: va_list resolves to a
   pointer (variadic definitions).
    // Its GNU spelling __builtin_va_list (used by
    the kernel's <stdarg.h>) is
    // an alias for the same type.
    cmp x20, #7
    b.ne .Ltypedef_try_builtin_va
    adrp x0, _kcc_str_va_list@PAGE
    add  x0, x0, _kcc_str_va_list@PAGEOFF
    mov x1, x19
    mov x2, #7
    bl _kcc_memcmp
    cbz x0, .Ltypedef_found

.Ltypedef_try_builtin_va:
    cmp x20, #17
    b.ne .Ltypedef_scan
    adrp x0, _kcc_str_builtin_va_list@PAGE
    add  x0, x0, _kcc_str_builtin_va_list@PAGEOFF
    mov x1, x19
    mov x2, #17
    bl _kcc_memcmp
    cbz x0, .Ltypedef_found

.Ltypedef_scan:
    // Load scope stack
    adrp x0, _kcc_typedef_scopes@PAGE
    add  x0, x0, _kcc_typedef_scopes@PAGEOFF
    ldr  x0, [x0]
    mov  x21, x0                    // x21 = scope
   vec
```

```
    ldr  x22, [x21, #VEC_COUNT] // x22 = scope
   count (loop index, top-down)

.Ltypedef_lookup_loop:
    cbz  x22, .Ltypedef_not_found

    sub  x22, x22, #1           // decrement (
   search from top)

    // Get scope at index x22
    mov  x0, x21
    mov  x1, x22
    bl _kcc_vec_get
    // x0 = pointer to ht pointer
    ldr  x0, [x0]               // x0 = ht pointer

    // ht_get(ht, name, len)
    mov x1, x19
    mov x2, x20
    bl _kcc_ht_get
    cbnz x0, .Ltypedef_found

    b .Ltypedef_lookup_loop

.Ltypedef_not_found:
    mov x0, #0
    b .Ltypedef_done

.Ltypedef_found:
    mov x0, #1

.Ltypedef_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

171 · defined at 170ab, 171 · used in 28g · defines _kcc_typedef_add 182, 184a, _kcc_typedef_lookup 188, 202 · uses _kcc_ht_get 62, _kcc_ht_put 63, _kcc_str_builtin_va_list 179, _kcc_str_va_list 179, _kcc_typedef_scopes 178a, _kcc_vec_get 65b

20.6. Translation Unit

The top-level parsing function. `kcc_parse` consumes the first token from the preprocessor, then loops parsing external declarations until `TOK_EOF`. The result is an `AST_TRANS_UNIT` node whose `F0` points to the head of a linked list of declarations. One subtlety: a single external declaration may itself introduce several declarators (`int a, b;`), which the declaration parser returns as a short `AST_F4` chain. The loop therefore appends the *next* external declaration after the *tail* of that chain, not its head—otherwise the link from `a` to `b` would be overwritten and every declarator but the first silently dropped (they would never reach semantic analysis and their names would be "undeclared").

⟨ *Parser: translation unit* 172 ⟩ ≡

```
// -- kcc_parse ---
// node *kcc_parse(void)
// Parses a complete translation unit.
// Returns AST_TRANS_UNIT node with F0 = first
   external declaration.
.globl _kcc_parse
.p2align 2
_kcc_parse:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Consume first token
    bl _kcc_pp_next

    // Allocate translation unit node
    mov x0, #AST_TRANS_UNIT
    bl _kcc_ast_alloc
    mov x19, x0                 // x19 =
   trans_unit node

    mov x20, #0                 // x20 = first
   decl (initially null)
    mov x21, #0                 // x21 = last
   decl (tail pointer)

.Lparse_tu_loop:
    // Check for EOF
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lparse_tu_done    // TOK_EOF = 0

    // Parse external declaration
    bl _kcc_parse_decl
    mov x22, x0                 // x22 = new decl
    node

    // Skip null results from error recovery
    cbz x22, .Lparse_tu_loop

    // A single external declaration may introduce
    several declarators sharing
    // one set of specifiers---`int a, b;`---which
    the declaration parser links
    // through AST_F4.  Find the tail of that
   group so the *next* external
    // declaration is appended after all of them;
   otherwise appending to the
    // returned head would overwrite the a->b link
    and drop every declarator but
    // the first (they would never be declared).
    mov x9, x22                 // x9 = tail of
   the returned group
.Lparse_tu_tail:
    ldr x0, [x9, #AST_F4]
    cbz x0, .Lparse_tu_have_tail
    mov x9, x0
    b .Lparse_tu_tail
.Lparse_tu_have_tail:

    cbz x20, .Lparse_tu_first

    // Append: previous tail -> new group's head,
   then advance to its tail.
    str x22, [x21, #AST_F4]
    mov x21, x9
    b .Lparse_tu_loop

.Lparse_tu_first:
    // First declaration
    mov x20, x22
    mov x21, x9
    b .Lparse_tu_loop

.Lparse_tu_done:
    // Store first decl in trans_unit F0
    str x20, [x19, #AST_F0]

    mov x0, x19                 // return
   trans_unit
```

```
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

172 · used in 28g · defines _kcc_parse 603, 622, 623 · uses _kcc_ast_alloc 168a, _kcc_parse_decl 181, _kcc_pp_next 108, _kcc_tok_type 104b

20.7. AST Dump

The --dump-ast flag triggers a recursive tree walk that prints a textual representation of the AST. Each node is printed with its tag name and field values, indented by depth.

⟨ *Parser: dump AST* 173 ⟩ ≡

```
// -- kcc_ast_dump ---
// void kcc_ast_dump(node *root)
//   x0 = root node pointer
// Recursively prints the AST tree.
.globl _kcc_ast_dump
.p2align 2
_kcc_ast_dump:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    cbz x0, .Last_dump_done
    mov x19, x0                     // x19 = node

    // Print indentation
    bl _kcc_ast_print_indent

    // Print tag name
    ldr x0, [x19, #AST_TAG]
    bl _kcc_ast_tag_name
    bl _kcc_io_puts

    // Print location
    adrp x0, _kcc_ast_msg_at@PAGE
    add  x0, x0, _kcc_ast_msg_at@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #AST_LINE]
    bl _kcc_io_putu
    mov x0, #':'
    bl _kcc_io_putc
    ldr x0, [x19, #AST_COL]
    bl _kcc_io_putu

    // Dispatch on tag for specific dump behavior
    ldr x20, [x19, #AST_TAG]

    // Translation unit and compound: dump linked
   list of children
    cmp x20, #AST_TRANS_UNIT
    b.eq .Last_dump_list_node
    cmp x20, #AST_COMPOUND
    b.eq .Last_dump_list_node

    // Declarations
    cmp x20, #AST_VAR_DECL
    b.eq .Last_dump_var_decl
    cmp x20, #AST_FUNC_DECL
    b.eq .Last_dump_func_decl
    cmp x20, #AST_PARAM_DECL
    b.eq .Last_dump_param_decl
    cmp x20, #AST_TYPEDEF_DECL
    b.eq .Last_dump_typedef_decl
    cmp x20, #AST_STRUCT_DECL
    b.eq .Last_dump_struct_decl
    cmp x20, #AST_UNION_DECL
    b.eq .Last_dump_struct_decl
    cmp x20, #AST_ENUM_DECL
    b.eq .Last_dump_enum_decl
    cmp x20, #AST_ENUMERATOR
    b.eq .Last_dump_enumerator
    cmp x20, #AST_FIELD_DECL
    b.eq .Last_dump_field_decl

    // Type specifiers
    cmp x20, #AST_TYPE_BUILTIN
    b.eq .Last_dump_type_builtin
    cmp x20, #AST_TYPE_NAME
    b.eq .Last_dump_name_node
    cmp x20, #AST_TYPE_PTR
    b.eq .Last_dump_type_ptr
    cmp x20, #AST_TYPE_ARRAY
    b.eq .Last_dump_type_array
    cmp x20, #AST_TYPE_FUNC
    b.eq .Last_dump_type_func

    // Expressions
    cmp x20, #AST_IDENT_EXPR
    b.eq .Last_dump_name_node
    cmp x20, #AST_INT_LIT
    b.eq .Last_dump_int_lit
    cmp x20, #AST_FLOAT_LIT
    b.eq .Last_dump_int_lit       // leaf: prints
   the raw double bit pattern
    cmp x20, #AST_STR_LIT
    b.eq .Last_dump_str_lit
    cmp x20, #AST_CHAR_LIT
    b.eq .Last_dump_int_lit
    cmp x20, #AST_BINARY_EXPR
    b.eq .Last_dump_binary
    cmp x20, #AST_UNARY_EXPR
    b.eq .Last_dump_unary
    cmp x20, #AST_ASSIGN_EXPR
    b.eq .Last_dump_binary
    cmp x20, #AST_CALL_EXPR
    b.eq .Last_dump_call
    cmp x20, #AST_MEMBER
    b.eq .Last_dump_member
    cmp x20, #AST_SIZEOF_EXPR
    b.eq .Last_dump_sizeof
    cmp x20, #AST_CAST_EXPR
    b.eq .Last_dump_cast

    // Statements with multiple children
    cmp x20, #AST_IF_STMT
    b.eq .Last_dump_if
    cmp x20, #AST_WHILE_STMT
    b.eq .Last_dump_two_children
    cmp x20, #AST_DO_STMT
    b.eq .Last_dump_two_children
    cmp x20, #AST_SWITCH_STMT
    b.eq .Last_dump_two_children
    cmp x20, #AST_FOR_STMT
    b.eq .Last_dump_for
    cmp x20, #AST_CASE_STMT
    b.eq .Last_dump_two_children
    cmp x20, #AST_LABEL_STMT
    b.eq .Last_dump_label
    cmp x20, #AST_DEFAULT_STMT
    b.eq .Last_dump_generic

    // For all other nodes, just print newline and
    recurse into children
    b .Last_dump_generic
```

□

173 · defined at 173, 174ab, 175ab, 176 · used in 28g · defines _kcc_ast_dump 174ab, 175ab, 176, 622 · uses _kcc_ast_print_indent 177, _kcc_ast_tag_name 177,

_kcc_io_puts 70a, _kcc_io_putu 71a

Dumping Declarations Each declaration handler prints the node's name (stored as a pointer in one of the payload fields) and then recurses into the type and initializer children. Aggregate declarations—structs, unions, and enums—walk a linked list of member or enumerator children via the F4 next pointer. Type-specifier nodes follow the same pattern: pointer types recurse into their pointee, array types into their element type and size expression, and function types into their return type and parameter list.

⟨ *Parser: dump AST* 174a ⟩ + ≡

```
.Last_dump_var_decl:
    // Print name if present
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_print_name_field
    mov x0, #'\n'
    bl _kcc_io_putc
    // Recurse: type (F0), init (F2)
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    ldr x0, [x19, #AST_F2]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_func_decl:
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_print_name_field
    mov x0, #'\n'
    bl _kcc_io_putc
    // Recurse: type (F0), params (F2), body (F3)
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    // Dump param list
    ldr x22, [x19, #AST_F2]
.Last_dump_params_loop:
    cbz x22, .Last_dump_func_body
    mov x0, x22
    bl _kcc_ast_dump
    ldr x22, [x22, #AST_F4]
    b .Last_dump_params_loop
.Last_dump_func_body:
    ldr x0, [x19, #AST_F3]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_param_decl:
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_print_name_field
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next
```

□

174a · defined at 173, 174ab, 175ab, 176 · used in 28g · uses _kcc_ast_dump 173, _kcc_ast_indent_dec 177, _kcc_ast_indent_inc 177, _kcc_ast_print_name_field 177

Dumping Aggregate and Typedef Declarations Typedef declarations simply print their aliased name and recurse into the underlying type child at F0. Struct and union declarations walk the linked list of field declarations anchored at F1, while enum declarations iterate over enumerator children the same way. Each enumerator optionally carries an initializer expression in F1.

⟨ *Parser: dump AST* 174b ⟩ + ≡

```
.Last_dump_typedef_decl:
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_print_name_field
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_struct_decl:
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_print_name_field
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x22, [x19, #AST_F1]
.Last_dump_fields_loop:
    cbz x22, .Last_dump_fields_end
    mov x0, x22
    bl _kcc_ast_dump
    ldr x22, [x22, #AST_F4]
    b .Last_dump_fields_loop
.Last_dump_fields_end:
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_enum_decl:
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_print_name_field
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x22, [x19, #AST_F1]
.Last_dump_enums_loop:
    cbz x22, .Last_dump_enums_end
    mov x0, x22
    bl _kcc_ast_dump
    ldr x22, [x22, #AST_F4]
    b .Last_dump_enums_loop
.Last_dump_enums_end:
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_enumerator:
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_print_name_field
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_field_decl:
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_print_name_field
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next
```

□

174b · defined at 173, 174ab, 175ab, 176 · used in 28g · uses _kcc_ast_dump 173, _kcc_ast_indent_dec 177, _kcc_ast_indent_inc 177, _kcc_ast_print_name_field 177

Dumping Type Specifiers Type-specifier nodes describe the C type system's recursive structure. Built-in types print their token name via `kcc_lex_type_name`. Pointer types recurse into the pointee at `F0`, array types recurse into both the element type (`F0`) and the size expression (`F1`), and function types walk a parameter list linked through `F4`. Name nodes (`AST_IDENT_EXPR`, `AST_TYPE_NAME`) simply print the identifier stored in `F0`.

⟨ *Parser: dump AST* 175a ⟩ + ≡

```
.Last_dump_type_builtin:
    // Print the token name for the builtin type
    adrp x0, _kcc_ast_msg_space@PAGE
    add  x0, x0, _kcc_ast_msg_space@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #AST_F0]
    bl _kcc_lex_type_name
    bl _kcc_io_puts
    mov x0, #'\n'
    bl _kcc_io_putc
    b .Last_dump_next

.Last_dump_name_node:
    // Used for AST_IDENT_EXPR, AST_TYPE_NAME: F0=
   name_ptr
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_print_name_field
    mov x0, #'\n'
    bl _kcc_io_putc
    b .Last_dump_next

.Last_dump_type_ptr:
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_type_array:
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_type_func:
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    // Dump params
    ldr x22, [x19, #AST_F1]
.Last_dump_tfunc_params:
    cbz x22, .Last_dump_tfunc_end
    mov x0, x22
    bl _kcc_ast_dump
    ldr x22, [x22, #AST_F4]
    b .Last_dump_tfunc_params
.Last_dump_tfunc_end:
    bl _kcc_ast_indent_dec
    b .Last_dump_next
```

□

175a · defined at 173, 174ab, 175ab, 176 · used in 28g · uses _kcc_ast_dump 173, _kcc_ast_indent_dec 177, _kcc_ast_indent_inc 177, _kcc_ast_print_name_field 177, _kcc_io_puts 70a, _kcc_lex_type_name 100b

Dumping Expressions Expression handlers print their operator or value and then recurse into sub-expressions. Binary and assignment expressions print the operator token name (via `kcc_lex_type_name`) and recurse into left and right children. Unary expressions additionally distinguish prefix from postfix form. Call expressions walk their argument list, member access prints the field name and dot-vs-arrow distinction, and `sizeof` annotates whether it operates on a type or an expression.

⟨ *Parser: dump AST* 175b ⟩ + ≡

```
.Last_dump_int_lit:
    // Print ' <value>'
    adrp x0, _kcc_ast_msg_space@PAGE
    add  x0, x0, _kcc_ast_msg_space@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #AST_F0]
    bl _kcc_io_putu
    mov x0, #'\n'
    bl _kcc_io_putc
    b .Last_dump_next

.Last_dump_str_lit:
    adrp x0, _kcc_ast_msg_space@PAGE
    add  x0, x0, _kcc_ast_msg_space@PAGEOFF
    bl _kcc_io_puts
    mov x0, #'"'
    bl _kcc_io_putc
    // Print string content (F0=ptr, F1=len)
    ldr x0, [x19, #AST_F0]
    ldr x1, [x19, #AST_F1]
    bl _kcc_io_write
    mov x0, #'"'
    bl _kcc_io_putc
    mov x0, #'\n'
    bl _kcc_io_putc
    b .Last_dump_next

.Last_dump_binary:
    // Print op token name
    adrp x0, _kcc_ast_msg_op@PAGE
    add  x0, x0, _kcc_ast_msg_op@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #AST_F0]
    bl _kcc_lex_type_name
    bl _kcc_io_puts
    mov x0, #'\n'
    bl _kcc_io_putc
    // Recurse: lhs (F1), rhs (F2)
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_dump
    ldr x0, [x19, #AST_F2]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_unary:
    adrp x0, _kcc_ast_msg_op@PAGE
    add  x0, x0, _kcc_ast_msg_op@PAGEOFF
    bl _kcc_io_puts
```

```
    ldr x0, [x19, #AST_F0]
    bl _kcc_lex_type_name
    bl _kcc_io_puts
    ldr x0, [x19, #AST_F2]
    cbz x0, .Last_dump_unary_postfix
    adrp x0, _kcc_ast_msg_prefix@PAGE
    add  x0, x0, _kcc_ast_msg_prefix@PAGEOFF
    bl _kcc_io_puts
    b .Last_dump_unary_child
.Last_dump_unary_postfix:
    adrp x0, _kcc_ast_msg_postfix@PAGE
    add  x0, x0, _kcc_ast_msg_postfix@PAGEOFF
    bl _kcc_io_puts
.Last_dump_unary_child:
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_call:
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    // Dump arg list
    ldr x22, [x19, #AST_F1]
.Last_dump_args_loop:
    cbz x22, .Last_dump_args_end
    mov x0, x22
    bl _kcc_ast_dump
    ldr x22, [x22, #AST_F4]
    b .Last_dump_args_loop
.Last_dump_args_end:
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_member:
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_print_name_field
    ldr x0, [x19, #AST_F2]
    cbnz x0, .Last_dump_member_arrow
    adrp x0, _kcc_ast_msg_dot@PAGE
    add  x0, x0, _kcc_ast_msg_dot@PAGEOFF
    bl _kcc_io_puts
    b .Last_dump_member_child
.Last_dump_member_arrow:
    adrp x0, _kcc_ast_msg_arrow@PAGE
    add  x0, x0, _kcc_ast_msg_arrow@PAGEOFF
    bl _kcc_io_puts
.Last_dump_member_child:
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_sizeof:
    ldr x0, [x19, #AST_F1]
    cbnz x0, .Last_dump_sizeof_type
    adrp x0, _kcc_ast_msg_expr@PAGE
    add  x0, x0, _kcc_ast_msg_expr@PAGEOFF
    bl _kcc_io_puts
    b .Last_dump_sizeof_child
.Last_dump_sizeof_type:
    adrp x0, _kcc_ast_msg_type@PAGE
    add  x0, x0, _kcc_ast_msg_type@PAGEOFF
    bl _kcc_io_puts
.Last_dump_sizeof_child:
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_cast:
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next
```

□

175b · defined at 173, 174ab, 175ab, 176 · used in 28g · uses _kcc_ast_dump 173, _kcc_ast_indent_dec 177, _kcc_ast_indent_inc 177, _kcc_ast_print_name_field 177, _kcc_io_puts 70a, _kcc_io_putu 71a, _kcc_io_write 70b, _kcc_lex_type_name 100b

Dumping Statements and Generic Handler Statement handlers recurse into their condition, body, and optional else or increment children. The `if` handler dumps three children (condition, then-branch, else-branch), while `for` dumps four (init, condition, increment, body). Simpler two-child nodes—`while`, `do`, `switch`, and `case`—share a single handler. The generic fallback handles any unrecognized tag by dumping only the first child, and the common epilogue restores callee-saved registers and returns.

⟨ *Parser: dump AST* 176 ⟩ + ≡

```
.Last_dump_if:
    // IfStmt: F0=cond, F1=then, F2=else
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_dump
    ldr x0, [x19, #AST_F2]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_two_children:
    // Nodes with F0 and F1 as children (WhileStmt
   , DoStmt, SwitchStmt, CaseStmt)
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_for:
    // ForStmt: F0=init, F1=cond, F2=incr, F3=body
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_dump
    ldr x0, [x19, #AST_F1]
```

```
    bl _kcc_ast_dump
    ldr x0, [x19, #AST_F2]
    bl _kcc_ast_dump
    ldr x0, [x19, #AST_F3]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_label:
    // LabelStmt: F0=name_ptr, F1=stmt
    ldr x0, [x19, #AST_F0]
    bl _kcc_ast_print_name_field
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x0, [x19, #AST_F1]
    bl _kcc_ast_dump
    bl _kcc_ast_indent_dec
    b .Last_dump_next

.Last_dump_list_node:
    // TranslationUnit and CompoundStmt: dump
   linked list of children via F0/F4
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc
    ldr x22, [x19, #AST_F0]
.Last_dump_list_loop:
    cbz x22, .Last_dump_list_end
    mov x0, x22
    bl _kcc_ast_dump
    ldr x22, [x22, #AST_F4]
    b .Last_dump_list_loop
.Last_dump_list_end:
    bl _kcc_ast_indent_dec
    b .Last_dump_done

.Last_dump_generic:
    // Generic handler: print newline, then dump
   F0 children only
    // (F1/F2 might contain non-pointer data like
   operator codes)
    mov x0, #'\n'
    bl _kcc_io_putc
    bl _kcc_ast_indent_inc

    ldr x0, [x19, #AST_F0]
    cbz x0, .Last_dump_generic_end
    bl _kcc_ast_dump
.Last_dump_generic_end:
    bl _kcc_ast_indent_dec

.Last_dump_next:

.Last_dump_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

176 · defined at 173, 174ab, 175ab, 176 · used in 28g · uses _kcc_ast_dump 173, _kcc_ast_indent_dec 177, _kcc_ast_indent_inc 177, _kcc_ast_print_name_field 177

20.8. Dump Helpers

Small helper functions used by the AST dump: tag name lookup, indentation printing, and name field printing.

⟨ *Parser: dump helpers* 177 ⟩ ≡

```
// -- kcc_ast_tag_name ---
// const char *kcc_ast_tag_name(uint64_t tag)
//   x0 = AST tag constant
// Returns pointer to tag name string.
.globl _kcc_ast_tag_name
.p2align 2
_kcc_ast_tag_name:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Lookup in tag name table
    adrp x1, _kcc_ast_tag_table@PAGE
    add  x1, x1, _kcc_ast_tag_table@PAGEOFF

    // Table has entries for known tags; use a
   simple scan
    // Entry format: .quad tag, .quad name_ptr
    mov x2, x0                  // x2 = target tag
.Last_tag_scan:
    ldr x3, [x1]                // tag
    cbz x3, .Last_tag_unknown   // sentinel: tag=0
    at end
    cmp x3, x2
    b.eq .Last_tag_found
    add x1, x1, #16             // next entry
    b .Last_tag_scan

.Last_tag_found:
    ldr x0, [x1, #8]            // name pointer
    ldp x29, x30, [sp], #16
    ret

.Last_tag_unknown:
    adrp x0, _kcc_ast_str_unknown@PAGE
    add  x0, x0, _kcc_ast_str_unknown@PAGEOFF
    ldp x29, x30, [sp], #16
    ret

// -- kcc_ast_print_indent ---
// void kcc_ast_print_indent(void)
// Prints spaces for the current indentation level
    (2 spaces per level).
.globl _kcc_ast_print_indent
.p2align 2
_kcc_ast_print_indent:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    adrp x19, _kcc_ast_indent@PAGE
    add  x19, x19, _kcc_ast_indent@PAGEOFF
    ldr  x19, [x19]             // x19 = indent
   level
    lsl  x19, x19, #1           // x19 = number of
    spaces (2 per level)

.Last_indent_loop:
    cbz x19, .Last_indent_done
    mov x0, #' '
    bl _kcc_io_putc
    sub x19, x19, #1
    b .Last_indent_loop

.Last_indent_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ast_indent_inc ---
.globl _kcc_ast_indent_inc
.p2align 2
_kcc_ast_indent_inc:
    adrp x0, _kcc_ast_indent@PAGE
    add  x0, x0, _kcc_ast_indent@PAGEOFF
```

```
    ldr  x1, [x0]
    add  x1, x1, #1
    str  x1, [x0]
    ret

// -- kcc_ast_indent_dec ---
.globl _kcc_ast_indent_dec
.p2align 2
_kcc_ast_indent_dec:
    adrp x0, _kcc_ast_indent@PAGE
    add  x0, x0, _kcc_ast_indent@PAGEOFF
    ldr  x1, [x0]
    sub  x1, x1, #1
    str  x1, [x0]
    ret

// -- kcc_ast_print_name_field ---
// void kcc_ast_print_name_field(const char *
   name_ptr)
//   x0 = interned name pointer (null-terminated)
// Prints " 'name'" if name_ptr is non-null.
.globl _kcc_ast_print_name_field
.p2align 2
_kcc_ast_print_name_field:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    cbz x0, .Last_name_done
    mov x19, x0

    adrp x0, _kcc_ast_msg_space@PAGE
    add  x0, x0, _kcc_ast_msg_space@PAGEOFF
    bl _kcc_io_puts
    mov x0, #'\''
    bl _kcc_io_putc
    mov x0, x19
    bl _kcc_io_puts
    mov x0, #'\''
    bl _kcc_io_putc

.Last_name_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

177 · used in 28g · defines _kcc_ast_indent_dec 174ab, 175ab, 176, _kcc_ast_indent_inc 174ab, 175ab, 176, _kcc_ast_print_indent 173, _kcc_ast_print_name_field 174ab, 175ab, 176, _kcc_ast_tag_name 173 · uses _kcc_ast_indent 178a, _kcc_io_puts 70a

20.9. Parser Data

Global variables and string constants for the parser.

⟨ *Parser: data* 178a ⟩ ≡

```
// -- Parser global state ---
.data
.p2align 3
.globl _kcc_parse_error_flag
_kcc_parse_error_flag:
    .quad 0

.globl _kcc_typedef_scopes
_kcc_typedef_scopes:
    .quad 0                         // pointer to
   scope stack vector

.globl _kcc_ast_indent
_kcc_ast_indent:
    .quad 0                         // current dump
   indentation level
```

□

178a · defined at 178ab, 179 · used in 28g · defines _kcc_ast_indent 177, _kcc_parse_error_flag 169, _kcc_typedef_scopes 170ab, 171

Diagnostic and Dump Format Strings The parser and AST dumper share a set of short string constants placed in the __TEXT,__cstring section so the linker can merge duplicates. These include the "expected token" error message, the location separator " @ ", operator annotation prefixes, and the member-access indicators for dot versus arrow syntax.

⟨ *Parser: data* 178b ⟩ + ≡

```
// -- Parser string constants ---
.section __TEXT,__cstring,cstring_literals

_kcc_parse_msg_expected:
    .asciz "expected token"

// Specific messages for the punctuators the
   parser most often expects, so a
// mismatch reads "expected ';'" rather than the
   bare "expected token".
_kcc_pe_semi:   .asciz "expected ';'"
_kcc_pe_lparen: .asciz "expected '('"
_kcc_pe_rparen: .asciz "expected ')'"
_kcc_pe_lbrace: .asciz "expected '{'"
_kcc_pe_rbrace: .asciz "expected '}'"
_kcc_pe_lbrack: .asciz "expected '['"
_kcc_pe_rbrack: .asciz "expected ']'"
_kcc_pe_comma:  .asciz "expected ','"
_kcc_pe_colon:  .asciz "expected ':'"

.data
.p2align 3
_kcc_pe_table:
    .quad TOK_SEMI,      _kcc_pe_semi
    .quad TOK_LPAREN,    _kcc_pe_lparen
    .quad TOK_RPAREN,    _kcc_pe_rparen
    .quad TOK_LBRACE,    _kcc_pe_lbrace
    .quad TOK_RBRACE,    _kcc_pe_rbrace
    .quad TOK_LBRACKET, _kcc_pe_lbrack
    .quad TOK_RBRACKET, _kcc_pe_rbrack
    .quad TOK_COMMA,     _kcc_pe_comma
    .quad TOK_COLON,     _kcc_pe_colon
    .quad 0, 0
.section __TEXT,__cstring,cstring_literals

_kcc_ast_msg_at:
    .asciz " @ "

_kcc_ast_msg_space:
    .asciz " "

_kcc_ast_msg_op:
    .asciz " op="

_kcc_ast_msg_prefix:
    .asciz " prefix"

_kcc_ast_msg_postfix:
    .asciz " postfix"

_kcc_ast_msg_dot:
    .asciz " ."
```

```
_kcc_ast_msg_arrow:
    .asciz " ->"

_kcc_ast_msg_expr:
    .asciz " expr"

_kcc_ast_msg_type:
    .asciz " type"

_kcc_ast_str_unknown:
    .asciz "<unknown>"
```

□

178b · defined at 178ab, 179 · used in 28g

AST Tag Name Table The tag name lookup table maps each `AST_*` constant to a human-readable string for diagnostic output. The table is a flat array of (tag, string-pointer) pairs terminated by a zero sentinel. The corresponding name strings live in the `__TEXT,__cstring` section so the linker can deduplicate them.

⟨ *Parser: data* 179 ⟩ + ≡

```
// -- AST tag name table ---
// Each entry: .quad tag, .quad name_string_ptr
.p2align 3
.data
_kcc_ast_tag_table:
    .quad AST_TRANS_UNIT
    .quad _kcc_ast_n_trans_unit
    .quad AST_VAR_DECL
    .quad _kcc_ast_n_var_decl
    .quad AST_FUNC_DECL
    .quad _kcc_ast_n_func_decl
    .quad AST_PARAM_DECL
    .quad _kcc_ast_n_param_decl
    .quad AST_TYPEDEF_DECL
    .quad _kcc_ast_n_typedef_decl
    .quad AST_STRUCT_DECL
    .quad _kcc_ast_n_struct_decl
    .quad AST_UNION_DECL
    .quad _kcc_ast_n_union_decl
    .quad AST_ENUM_DECL
    .quad _kcc_ast_n_enum_decl
    .quad AST_ENUMERATOR
    .quad _kcc_ast_n_enumerator
    .quad AST_FIELD_DECL
    .quad _kcc_ast_n_field_decl
    .quad AST_STATIC_ASSERT
    .quad _kcc_ast_n_static_assert
    .quad AST_TYPE_BUILTIN
    .quad _kcc_ast_n_type_builtin
    .quad AST_TYPE_NAME
    .quad _kcc_ast_n_type_name
    .quad AST_TYPE_PTR
    .quad _kcc_ast_n_type_ptr
    .quad AST_TYPE_ARRAY
    .quad _kcc_ast_n_type_array
    .quad AST_TYPE_FUNC
    .quad _kcc_ast_n_type_func
    .quad AST_TYPE_STRUCT
    .quad _kcc_ast_n_type_struct
    .quad AST_TYPE_UNION
    .quad _kcc_ast_n_type_union
    .quad AST_TYPE_ENUM
    .quad _kcc_ast_n_type_enum
    .quad AST_TYPE_QUALS
    .quad _kcc_ast_n_type_quals
    .quad AST_COMPOUND
    .quad _kcc_ast_n_compound
    .quad AST_EXPR_STMT
    .quad _kcc_ast_n_expr_stmt
    .quad AST_IF_STMT
    .quad _kcc_ast_n_if_stmt
    .quad AST_SWITCH_STMT
    .quad _kcc_ast_n_switch_stmt
    .quad AST_WHILE_STMT
    .quad _kcc_ast_n_while_stmt
    .quad AST_DO_STMT
    .quad _kcc_ast_n_do_stmt
    .quad AST_FOR_STMT
    .quad _kcc_ast_n_for_stmt
    .quad AST_GOTO_STMT
    .quad _kcc_ast_n_goto_stmt
    .quad AST_CONTINUE_STMT
    .quad _kcc_ast_n_continue_stmt
    .quad AST_BREAK_STMT
    .quad _kcc_ast_n_break_stmt
    .quad AST_RETURN_STMT
    .quad _kcc_ast_n_return_stmt
    .quad AST_LABEL_STMT
    .quad _kcc_ast_n_label_stmt
    .quad AST_CASE_STMT
    .quad _kcc_ast_n_case_stmt
    .quad AST_DEFAULT_STMT
    .quad _kcc_ast_n_default_stmt
    .quad AST_NULL_STMT
    .quad _kcc_ast_n_null_stmt
    .quad AST_IDENT_EXPR
    .quad _kcc_ast_n_ident_expr
    .quad AST_INT_LIT
    .quad _kcc_ast_n_int_lit
    .quad AST_STR_LIT
    .quad _kcc_ast_n_str_lit
    .quad AST_CHAR_LIT
    .quad _kcc_ast_n_char_lit
    .quad AST_BINARY_EXPR
    .quad _kcc_ast_n_binary_expr
    .quad AST_UNARY_EXPR
    .quad _kcc_ast_n_unary_expr
    .quad AST_ASSIGN_EXPR
    .quad _kcc_ast_n_assign_expr
    .quad AST_COND_EXPR
    .quad _kcc_ast_n_cond_expr
    .quad AST_CALL_EXPR
    .quad _kcc_ast_n_call_expr
    .quad AST_SUBSCRIPT
    .quad _kcc_ast_n_subscript
    .quad AST_MEMBER
    .quad _kcc_ast_n_member
    .quad AST_CAST_EXPR
    .quad _kcc_ast_n_cast_expr
    .quad AST_SIZEOF_EXPR
    .quad _kcc_ast_n_sizeof_expr
    .quad AST_ALIGNOF_EXPR
    .quad _kcc_ast_n_alignof_expr
    .quad AST_COMMA_EXPR
    .quad _kcc_ast_n_comma_expr
    .quad AST_INIT_LIST
    .quad _kcc_ast_n_init_list
    .quad AST_COMPOUND_LIT
    .quad _kcc_ast_n_compound_lit
    .quad AST_PAREN_EXPR
    .quad _kcc_ast_n_paren_expr
    .quad AST_GENERIC
    .quad _kcc_ast_n_generic
    .quad AST_GENERIC_ASSOC
    .quad _kcc_ast_n_generic_assoc
    .quad AST_FLOAT_LIT
    .quad _kcc_ast_n_float_lit
    .quad AST_VA_ARG
    .quad _kcc_ast_n_va_arg
    .quad 0, 0                    // sentinel

// Tag name strings
.section __TEXT,__cstring,cstring_literals
_kcc_ast_n_trans_unit:    .asciz "TranslationUnit"
_kcc_ast_n_var_decl:      .asciz "VarDecl"
_kcc_ast_n_func_decl:     .asciz "FuncDecl"
_kcc_ast_n_param_decl:    .asciz "ParamDecl"
_kcc_ast_n_typedef_decl:  .asciz "TypedefDecl"
```

```
_kcc_ast_n_struct_decl:    .asciz "StructDecl"
_kcc_ast_n_union_decl:     .asciz "UnionDecl"
_kcc_ast_n_enum_decl:      .asciz "EnumDecl"
_kcc_ast_n_enumerator:     .asciz "Enumerator"
_kcc_ast_n_field_decl:     .asciz "FieldDecl"
_kcc_ast_n_static_assert:  .asciz "StaticAssert"
_kcc_ast_n_type_builtin:   .asciz "BuiltinType"
_kcc_ast_n_type_name:      .asciz "TypeName"
_kcc_ast_n_type_ptr:       .asciz "PointerType"
_kcc_ast_n_type_array:     .asciz "ArrayType"
_kcc_ast_n_type_func:      .asciz "FuncType"
_kcc_ast_n_type_struct:    .asciz "StructType"
_kcc_ast_n_type_union:     .asciz "UnionType"
_kcc_ast_n_type_enum:      .asciz "EnumType"
_kcc_ast_n_type_quals:     .asciz "QualType"
_kcc_ast_n_compound:       .asciz "CompoundStmt"
_kcc_ast_n_expr_stmt:      .asciz "ExprStmt"
_kcc_ast_n_if_stmt:        .asciz "IfStmt"
_kcc_ast_n_switch_stmt:    .asciz "SwitchStmt"
_kcc_ast_n_while_stmt:     .asciz "WhileStmt"
_kcc_ast_n_do_stmt:        .asciz "DoStmt"
_kcc_ast_n_for_stmt:       .asciz "ForStmt"
_kcc_ast_n_goto_stmt:      .asciz "GotoStmt"
_kcc_ast_n_continue_stmt:  .asciz "ContinueStmt"
_kcc_ast_n_break_stmt:     .asciz "BreakStmt"
_kcc_ast_n_return_stmt:    .asciz "ReturnStmt"
_kcc_ast_n_label_stmt:     .asciz "LabelStmt"
_kcc_ast_n_case_stmt:      .asciz "CaseStmt"
_kcc_ast_n_default_stmt:   .asciz "DefaultStmt"
_kcc_ast_n_null_stmt:      .asciz "NullStmt"
_kcc_ast_n_ident_expr:     .asciz "IdentExpr"
_kcc_ast_n_int_lit:        .asciz "IntLit"
_kcc_ast_n_float_lit:      .asciz "FloatLit"
_kcc_ast_n_va_arg:         .asciz "VaArg"
_kcc_str_va_list:          .asciz "va_list"
_kcc_str_builtin_va_list: .asciz "
   __builtin_va_list"
_kcc_ast_n_str_lit:        .asciz "StrLit"
_kcc_ast_n_char_lit:       .asciz "CharLit"
_kcc_ast_n_binary_expr:    .asciz "BinaryExpr"
_kcc_ast_n_unary_expr:     .asciz "UnaryExpr"
_kcc_ast_n_assign_expr:    .asciz "AssignExpr"
_kcc_ast_n_cond_expr:      .asciz "CondExpr"
_kcc_ast_n_call_expr:      .asciz "CallExpr"
_kcc_ast_n_subscript:      .asciz "Subscript"
_kcc_ast_n_member:         .asciz "MemberExpr"
_kcc_ast_n_cast_expr:      .asciz "CastExpr"
_kcc_ast_n_sizeof_expr:    .asciz "SizeofExpr"
_kcc_ast_n_alignof_expr:   .asciz "AlignofExpr"
_kcc_ast_n_comma_expr:     .asciz "CommaExpr"
_kcc_ast_n_init_list:      .asciz "InitList"
_kcc_ast_n_compound_lit:   .asciz "CompoundLit"
_kcc_ast_n_paren_expr:     .asciz "ParenExpr"
_kcc_ast_n_generic:        .asciz "GenericExpr"
_kcc_ast_n_generic_assoc:  .asciz "GenericAssoc"
```

□

179 · defined at 178ab, 179 · used in 28g ·

defines _kcc_ast_n_alignof_expr never used, _kcc_ast_n_assign_expr never used, _kcc_ast_n_binary_expr never used, _kcc_ast_n_break_stmt never used, _kcc_ast_n_call_expr never used, _kcc_ast_n_case_stmt never used, _kcc_ast_n_cast_expr never used, _kcc_ast_n_char_lit never used, _kcc_ast_n_comma_expr never used, _kcc_ast_n_compound never used, _kcc_ast_n_compound_lit never used, _kcc_ast_n_cond_expr never used, _kcc_ast_n_continue_stmt never used, _kcc_ast_n_default_stmt never used, _kcc_ast_n_do_stmt never used, _kcc_ast_n_enumerator never used, _kcc_ast_n_enum_decl never used, _kcc_ast_n_expr_stmt never used, _kcc_ast_n_field_decl never used, _kcc_ast_n_for_stmt never used, _kcc_ast_n_func_decl never used, _kcc_ast_n_generic never used, _kcc_ast_n_generic_assoc never used, _kcc_ast_n_goto_stmt never used, _kcc_ast_n_ident_expr never used, _kcc_ast_n_if_stmt never used, _kcc_ast_n_init_list never used, _kcc_ast_n_int_lit never used, _kcc_ast_n_label_stmt never used, _kcc_ast_n_member never used, _kcc_ast_n_null_stmt never used, _kcc_ast_n_param_decl never used, _kcc_ast_n_paren_expr never used, _kcc_ast_n_return_stmt never used, _kcc_ast_n_sizeof_expr never used, _kcc_ast_n_static_assert never used, _kcc_ast_n_struct_decl never used, _kcc_ast_n_str_lit never used, _kcc_ast_n_subscript never used, _kcc_ast_n_switch_stmt never used, _kcc_ast_n_trans_unit never used, _kcc_ast_n_typedef_decl never used, _kcc_ast_n_type_array never used, _kcc_ast_n_type_builtin never used, _kcc_ast_n_type_enum never used, _kcc_ast_n_type_func never used, _kcc_ast_n_type_name never used, _kcc_ast_n_type_ptr never used, _kcc_ast_n_type_quals never used, _kcc_ast_n_type_struct never used, _kcc_ast_n_type_union never used, _kcc_ast_n_unary_expr never used, _kcc_ast_n_union_decl never used, _kcc_ast_n_var_decl never used, _kcc_ast_n_va_arg never used, _kcc_ast_n_while_stmt never used, _kcc_str_builtin_va_list 171, _kcc_str_va_list 171

21. Declaration Parsing

Declarations are the most syntactically complex part of C: a declaration combines storage-class specifiers, type specifiers, qualifiers, and declarators into a form that introduces names into scope (§6.7). The central difficulty is typedef disambiguation—the parser must consult the typedef table to decide whether an identifier begins a new declaration or an expression. This chapter walks through the full declaration grammar, from top-level specifiers and nested declarators to struct/union/enum definitions, initializers, and `_Static_assert`.

21.1. Declaration

A declaration (§6.7) has the form:

declaration-specifiers init-declarator-list? ;

The specifiers include storage class (`typedef`, `static`, `extern`, etc.), type specifiers (`int`, `char`, struct/union/enum, typedef names), and qualifiers (`const`, `volatile`, `restrict`, `_Atomic`). A `typedef` whose declarator is a *function* type—`typedef int f(int);`, or the parenthesised-name form `typedef int (f)(int);` the kernel's `<linux/percpu.h>` uses—produces a function declarator node just like a prototype, so it is intercepted before the function-declaration paths and its name registered as a typedef (the node carries the name in the same field a variable declarator does).

After a declarator and its trailing attributes, GCC allows an *asm label* or *named-register binding*: `T v asm("string")`. With `register` storage it binds the variable to a hardware register (`arch/x86` declares `register unsigned long current_stack_pointer asm("%rsp")`); otherwise it renames the symbol at link time. The parser consumes an optional `asm(…)` here by paren balance—not assuming a single string, since the operand is often a macro expanding to several adjacent string literals (`_ASM_SP` becomes `" " "rsp" " "`). It is accepted but not yet honoured in code generation, which is enough for the header declarations a kernel build pulls in.

⟨ *Decl: parse declaration* 181 ⟩ ≡

```
// -- kcc_parse_decl ---
// node *kcc_parse_decl(void)
// Parses a declaration or function definition.
// Returns AST node (VarDecl, FuncDecl,
   TypedefDecl, StructDecl, etc.)
.text
.globl _kcc_parse_decl
.p2align 2
_kcc_parse_decl:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]

    // Check for _Static_assert
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK__STATIC_ASSERT
    b.eq .Ldecl_static_assert

    // Parse declaration specifiers
    // Returns: x0 = type node, x1 = storage class
    flags
    bl _kcc_parse_decl_specs
    mov x19, x0                    // x19 = type
node
    mov x20, x1                    // x20 = storage
class (0=none,1=typedef,2=extern,3=static,4=
auto,5=register)

    // Check for bare ';' (struct/union/enum
declaration without declarator)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_SEMI
    b.eq .Ldecl_bare_type

    // Parse declarator
    mov x0, x19                    // base type
    bl _kcc_parse_declarator
    // Returns: x0 = decl node (partially filled)
    mov x21, x0                    // x21 =
declarator result node
    cbz x21, .Ldecl_error_recovery

    // GNU attributes may trail a declarator: `int
    f(void) __attribute__((...))`,
    // `int x __attribute__((aligned)) = 5;`.
Consume them before deciding
    // whether a definition body, initialiser,
comma, or ';' follows.
    bl _kcc_parse_skip_attributes

    // A GCC asm label / named-register binding
may follow a declarator:
    // `T v asm("string")`.  With `register`
storage it binds v to a hardware
    // register (arch/x86 `current_stack_pointer
asm("%rsp")`); otherwise it
    // renames the symbol at link.  Parse and
accept it (not yet honoured in
    // codegen), then skip any trailing attributes
.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ASM
    b.ne .Ldecl_no_asm_label
    bl _kcc_pp_next                    // consume
'asm'
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect
    // Consume everything to the matching ')'.
The operand is usually a string
    // literal, but macros expand it to several
adjacent strings (arch/x86
    // `asm(_ASM_SP)` → `" " "rsp" " "`), so skip
by paren balance.
    mov x23, #1                        // depth
.Ldecl_asm_skip:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
```

```
    ldr  x0, [x0]
    cmp  x0, #TOK_EOF
    b.eq .Ldecl_asm_done
    cmp  x0, #TOK_LPAREN
    b.ne .Ldecl_asm_notlp
    add  x23, x23, #1
    b .Ldecl_asm_adv
.Ldecl_asm_notlp:
    cmp  x0, #TOK_RPAREN
    b.ne .Ldecl_asm_adv
    sub  x23, x23, #1
    cbz  x23, .Ldecl_asm_done
.Ldecl_asm_adv:
    bl _kcc_pp_next
    b .Ldecl_asm_skip
.Ldecl_asm_done:
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    bl _kcc_parse_skip_attributes
.Ldecl_no_asm_label:

    // Check what kind of declaration this is
    ldr x22, [x21, #AST_TAG]    // x22 = node tag

    // If it's a function declarator and next
   token is '{', it's a definition
    cmp x22, #AST_FUNC_DECL
    b.ne .Ldecl_not_func

    // A `typedef` whose declarator is a function
   type---`typedef int f(int);`,
    // or the kernel's parenthesised `typedef int
   (f)(int);`---still declares a
    // typedef NAME, not a function.  Register it
   before the function-decl paths
    // (the FuncDecl node carries the name in F1,
   exactly like a VarDecl).
    cmp x20, #1                 // storage class
   1 = typedef
    b.eq .Ldecl_typedef

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LBRACE
    b.ne .Ldecl_func_proto

    // Function definition: parse body.  Publish
   the function name so a `__func__`
    // primary expression inside the body can
   materialise it (\stdref{6.4.2.2}); clear
    // it afterwards so a file-scope `__func__`
   does not see a stale name.
    ldr x0, [x21, #AST_F1]      // function name
   pointer
    adrp x1, _kcc_cur_func_name@PAGE
    add  x1, x1, _kcc_cur_func_name@PAGEOFF
    str  x0, [x1]
    bl _kcc_parse_compound
    str x0, [x21, #AST_F3]      // body
    adrp x1, _kcc_cur_func_name@PAGE
    add  x1, x1, _kcc_cur_func_name@PAGEOFF
    str  xzr, [x1]
    // Record internal linkage.  A FuncDecl packs
   its params pointer in F2 with
    // the variadic flag in bit 0; we reuse the
   next free low bit (bit 1) for the
    // `static` storage class so the emitter can
   give the symbol local linkage.
    cmp x20, #3                 // storage class
   3 = static
    b.ne .Ldecl_func_def_done
    ldr x0, [x21, #AST_F2]
    orr x0, x0, #2              // set bit 1 =
   static
    str x0, [x21, #AST_F2]
.Ldecl_func_def_done:
    mov x0, x21
    b .Ldecl_done

.Ldecl_func_proto:
    // Function prototype (declaration without
   body).  A prototype may head a
    // comma-separated declarator list that shares
    the base type, as in
    // \texttt{int f(int), g(int), a;} --- a
   following ',' therefore continues
    // into the same multi-declarator loop the
   variable path uses, with the
    // prototype node as the first link of the
   chain.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.eq .Ldecl_more_vars
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    mov x0, x21
    b .Ldecl_done
```

□

181 · defined at 181, 182, 184a, 191a · used in 29c · defines _kcc_parse_decl 172, 230, 233 · uses _kcc_parse_compound 230, _kcc_parse_declarator 190, _kcc_parse_decl_specs 186b, _kcc_parse_expect 168b, _kcc_parse_skip_attributes 184b, _kcc_pp_next 108, _kcc_tok_type 104b

Variable and typedef declarations. When the declarator is not a function (or is a function prototype without a body), the parser falls through to variable and typedef handling. A regular variable declaration stores its storage class in AST_F3 and optionally parses an initializer after =. Multiple declarators sharing the same specifiers are supported via comma-separated lists, where each subsequent declarator is parsed against the original base type in x19 and linked through AST_F4 into a chain rooted at the first declaration node. The first declarator in such a list may itself be a function prototype (`int f(int), g(int), a;`): the prototype path joins the very same loop when it sees a comma, so a mixture of prototypes and objects under one set of specifiers parses into one chain. Each continuation declarator may carry a trailing GNU `__attribute__` before its = (the first declarator's is consumed upstream); it is skipped, matching the kernel's `DEFINE_LOCK_GUARD_0` expansion `T a = …, *b __attribute__((__unused__)) = &a;`.

⟨ *Decl: parse declaration* 182 ⟩ +≡

```
.Ldecl_not_func:
    // Variable or typedef declaration
    // Check for typedef
    cmp x20, #1
    b.eq .Ldecl_typedef

    // Regular variable declaration
    // Store storage class
    str x20, [x21, #AST_F3]     // storage_class

    // Check for initializer
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
```

```
    ldr  x0, [x0]
    cmp  x0, #TOK_ASSIGN
    b.ne .Ldecl_var_no_init

    // Consume '='
    bl _kcc_pp_next

    // Parse initializer
    bl _kcc_parse_initializer
    str x0, [x21, #AST_F2]      // init

    // __auto_type: resolve the deferred typeof
   node (x19 = base type) against
    // the initializer just parsed, so the
   variable takes the initializer's type.
    ldr x1, [x19, #AST_TAG]
    cmp x1, #AST_TYPE_TYPEOF
    b.ne .Ldecl_var_no_init
    ldr x1, [x19, #AST_F1]
    cmp x1, #2                  // deferred-auto
   marker
    b.ne .Ldecl_var_no_init
    str x0, [x19, #AST_F0]      // F0 =
   initializer expression
    str xzr, [x19, #AST_F1]     // F1 = 0:
   expression mode

.Ldecl_var_no_init:
    // Check for more declarators (comma-separated
   )
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.eq .Ldecl_more_vars

    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    mov x0, x21
    b .Ldecl_done

.Ldecl_more_vars:
    // Multiple declarators: link via F4
    mov x23, x21                // x23 = first
   decl
    mov x24, x21                // x24 = last
   decl

.Ldecl_var_loop:
    bl _kcc_pp_next             // consume ','

    // Parse next declarator with same base type
    mov x0, x19
    bl _kcc_parse_declarator
    mov x25, x0                 // x25 = new decl
    cbz x25, .Ldecl_var_loop_end

    // A GNU attribute may trail this declarator
   before its `=` or the next
    // `,` (e.g. `T a, *b __attribute__((
   __unused__)) = &a;`, as the kernel's
    // DEFINE_LOCK_GUARD_0 expansion produces).
   The first declarator is handled
    // upstream; the continuation declarators are
   handled here.
    bl _kcc_parse_skip_attributes

    str x20, [x25, #AST_F3]     // storage_class

    // A typedef continuation (`typedef ... A, *B
   ;`) registers this further name
    // and tags the node, exactly as the first
   declarator was.  x22 is free here
    // (the first node is tracked in x23) and
   callee-saved across the calls.
    cmp x20, #1
    b.ne .Ldecl_var_td_skip
    ldr x0, [x25, #AST_F1]      // name
    cbz x0, .Ldecl_var_td_tag
    mov x22, x0                 // save name
   across strlen
    bl _kcc_strlen
    mov x1, x0                  // len
    mov x0, x22                 // name
    bl _kcc_typedef_add
.Ldecl_var_td_tag:
    mov x0, x25
    bl _kcc_parse_typedef_funcfix     // typedef
   of function type keeps TY_FUNC
    mov x0, #AST_TYPEDEF_DECL
    str x0, [x25, #AST_TAG]
.Ldecl_var_td_skip:

    // Check for initializer
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ASSIGN
    b.ne .Ldecl_var_no_init2

    bl _kcc_pp_next             // consume '='
    bl _kcc_parse_initializer
    str x0, [x25, #AST_F2]      // init

.Ldecl_var_no_init2:
    // Link
    str x25, [x24, #AST_F4]
    mov x24, x25

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.eq .Ldecl_var_loop

.Ldecl_var_loop_end:
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    mov x0, x23                 // return first
   decl
    b .Ldecl_done
```

□

182 · defined at 181, 182, 184a, 191a · used in 29c · uses _kcc_parse_declarator 190, _kcc_parse_expect 168b, _kcc_parse_initializer 203, _kcc_parse_skip_attributes 184b, _kcc_parse_typedef_funcfix 191a, _kcc_pp_next 108, _kcc_tok_type 104b, _kcc_typedef_add 171

Typedef registration and special forms. A typedef declaration (`storage_class == 1`) registers the declared name in the typedef table so the lexer can disambiguate identifiers from type names in subsequent parsing. The node tag is rewritten to `AST_TYPEDEF_DECL`. A single `typedef` may introduce several names—the kernel's `typedef struct { … } Rec_Type, *Rec_Pointer;`—so when a comma follows the first name the parser joins the shared multi-declarator loop, which registers each further name as a typedef and tags its node in turn. This block also handles bare type declarations such as `struct S { … };` where no declarator follows the specifiers, as well as `_Static_assert` forwarding and error recovery with synchronisation to the next semicolon.

⟨ *Decl: parse declaration* 184a ⟩ + ≡

```
.Ldecl_typedef:
    // Typedef declaration: register the name
    ldr x0, [x21, #AST_F1]      // name_ptr
    cbz x0, .Ldecl_typedef_done
    // Get name length via strlen
    mov x23, x0                 // save name
    bl _kcc_strlen
    mov x1, x0                  // len
    mov x0, x23                 // name
    bl _kcc_typedef_add

.Ldecl_typedef_done:
    mov x0, x21
    bl _kcc_parse_typedef_funcfix     // typedef
   of function type keeps TY_FUNC
    // Change tag to TYPEDEF_DECL
    mov x0, #AST_TYPEDEF_DECL
    str x0, [x21, #AST_TAG]
    // A single typedef may name several types---`
   typedef struct {...} A, *B;`---
    // so a comma continues into the shared multi-
   declarator loop, which
    // registers each further name as a typedef
   too.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.eq .Ldecl_more_vars
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    mov x0, x21
    b .Ldecl_done

.Ldecl_bare_type:
    // Just a type followed by ';' (e.g., "struct
   S { ... };")
    bl _kcc_pp_next               // consume ';'
    // Return the type node itself (struct/union/
   enum decl)
    mov x0, x19
    b .Ldecl_done

.Ldecl_static_assert:
    bl _kcc_parse_static_assert
    b .Ldecl_done

.Ldecl_error_recovery:
    bl _kcc_parse_sync
    mov x0, #0

.Ldecl_done:
    ldr x25, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

184a · defined at 181, 182, 184a, 191a · used in 29c · uses _kcc_parse_expect 168b, _kcc_parse_static_assert 205a, _kcc_parse_sync 169, _kcc_parse_typedef_funcfix 191a, _kcc_pp_next 108, _kcc_tok_type 104b, _kcc_typedef_add 171

21.2. Declaration Specifiers

`kcc_parse_decl_specs` parses the type specifiers, qualifiers, and storage class keywords that precede a declarator. It accumulates type information and returns both the type node and the storage class.

The alignment specifier `_Alignas` (§6.7.5) may also appear here. It takes a parenthesised argument—either a constant expression or a type name—so, unlike the bare function specifiers, the loop cannot simply consume a single token. `.Ldecl_alignas` accepts the keyword, then skips its argument by counting parentheses (x24, a callee-saved register the loop leaves free, holds the nesting depth across the intervening calls) until the matching close paren. The requested alignment is thus accepted syntactically—which is what makes <stdalign.h>'s `alignas` usable—but honouring an over-alignment beyond the frame's natural guarantee would require aligning the affected stack slot and, for alignments past sixteen, dynamically realigning the stack pointer; that refinement is deferred.

⟨ *Decl: parse specifiers* 184b ⟩ ≡

```
// -- kcc_parse_skip_attributes ---
// void kcc_parse_skip_attributes(void)
// GNU attributes (\texttt{\_\_attribute\_\_((...)
   ))}) and \texttt{\_\_extension\_\_}
// may appear at many points in a declaration;
   real system headers use them
// pervasively.  Most attributes are hints that do
    not change the observable
// result (\texttt{noinline}, \texttt{format}, \
   texttt{unused}, ...), so kcc
// consumes and discards them: the point is to let
    the surrounding declaration
// parse.  Each attribute is \texttt{\_\_attribute
   \_\_} followed by a balanced
// parenthesised group (the characteristic double
   parens plus arbitrarily nested
// argument lists); several may stack, and \texttt
   {\_\_extension\_\_} is a bare
// skippable prefix.
//
// Two attributes \emph{do} change layout and must
    not be silently dropped:
// \texttt{packed} (drop inter-member padding,
   aggregate alignment~1) and
// \texttt{aligned(N)} (raise the aggregate's
   alignment).  While skipping, this
// routine records what it sees into two globals
   ---\texttt{\_kcc\_attr\_packed}
// and \texttt{\_kcc\_attr\_align}---reset at
   entry so each call reports only its
// own findings.  A caller that cares (the struct/
   union parser) reads them
// immediately after the call and stamps the
   result onto the aggregate node; all
// other callers ignore them.  Both the bare and \
   texttt{\_\_packed\_\_} /
// \texttt{\_\_aligned\_\_} spellings are
   recognised.
.data
.p2align 3
.globl _kcc_attr_packed
_kcc_attr_packed:  .quad 0      // set to 1 when
   the last attribute run named `packed`
.globl _kcc_attr_align
_kcc_attr_align:   .quad 0      // set to N when
   it named `aligned(N)` (0 = none)
.globl _kcc_attr_ht
_kcc_attr_ht:      .quad 0      // linkage-
   attribute side table (weak/section/alias); C-
   port only
.globl _kcc_cur_func_name
```

```
_kcc_cur_func_name: .quad 0      // name of the
   function whose body is being parsed (for
   __func__); 0 outside
.section __TEXT,__cstring,cstring_literals
_kcc_attr_s_packed:   .asciz "packed"
_kcc_attr_s_packed2:  .asciz "__packed__"
_kcc_attr_s_aligned:  .asciz "aligned"
_kcc_attr_s_aligned2: .asciz "__aligned__"
_kcc_str_auto_type:   .asciz "__auto_type"
_kcc_decl_msg_complex: .asciz "complex types (
   _Complex/_Imaginary) are not supported"
.text
.globl _kcc_parse_skip_attributes
.p2align 2
_kcc_parse_skip_attributes:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]       // x19 = paren
   depth, x20 = "next number is an
    str x21, [sp, #32]            // aligned()
   value" flag (both callee-saved:
                                  // _kcc_pp_next
   clobbers caller-saved across loops)
    // Reset the layout-attribute output channel
   for this run.
    adrp x0, _kcc_attr_packed@PAGE
    add  x0, x0, _kcc_attr_packed@PAGEOFF
    str  xzr, [x0]
    adrp x0, _kcc_attr_align@PAGE
    add  x0, x0, _kcc_attr_align@PAGEOFF
    str  xzr, [x0]
    mov  x20, #0
.Lpsa_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_EXTENSION        // __extension__
   : a bare skippable prefix
    b.ne .Lpsa_chk_attr
    bl _kcc_pp_next
    b .Lpsa_loop
.Lpsa_chk_attr:
    cmp x0, #TOK_ATTRIBUTE
    b.ne .Lpsa_done
    bl _kcc_pp_next               // consume
   __attribute__
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_LPAREN
    b.ne .Lpsa_done               // no '(' after
   __attribute__ → give up gracefully
    mov x19, #0                   // paren depth
.Lpsa_paren:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cbz x0, .Lpsa_done            // TOK_EOF: never
   spin past end of input
    cmp x0, #TOK_LPAREN
    b.ne .Lpsa_not_lp
    add x19, x19, #1
    b .Lpsa_padv
.Lpsa_not_lp:
    cmp x0, #TOK_RPAREN
    b.ne .Lpsa_chk_ident
    sub x19, x19, #1
    b .Lpsa_padv
.Lpsa_chk_ident:
    // Identifier inside the group: is it a layout
   attribute we honour?
    cmp x0, #TOK_IDENT
    b.eq .Lpsa_ident
    cmp x0, #TOK_NUMBER
    b.eq .Lpsa_number
    b .Lpsa_padv
.Lpsa_ident:
    ⟨ Decl: classify layout attribute 186a ⟩
    b .Lpsa_padv
.Lpsa_number:
    // A number token right after `aligned`
   supplies the alignment value.
    cbz x20, .Lpsa_padv
    mov x20, #0
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_pp_atoi               // x0 = parsed
   value
    adrp x1, _kcc_attr_align@PAGE
    add  x1, x1, _kcc_attr_align@PAGEOFF
    str  x0, [x1]
.Lpsa_padv:
    bl _kcc_pp_next
    cbnz x19, .Lpsa_paren         // still inside
   the group → keep consuming
    b .Lpsa_loop                  // balanced: this
   attribute done, check for another
.Lpsa_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_parse_typeof ---
// node *kcc_parse_typeof(void)
// GNU \texttt{typeof} (and \texttt{\_\_typeof\_\_
   }): a type-specifier whose value
// is the type of a parenthesised type-name or
   expression.  It disambiguates the
// two forms exactly as \texttt{sizeof} does---
   peeking past \texttt{(} for a
// type-start token---and wraps the operand in an
   \texttt{AST\_TYPE\_TYPEOF} node
// (\texttt{F1}=1 for a type-name, 0 for an
   expression) that the type resolver
// unwraps later, when full semantic type
   information is available.
.globl _kcc_parse_typeof
.p2align 2
_kcc_parse_typeof:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    bl _kcc_pp_next               // consume '
   typeof'
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect
    // type-name or expression?
    bl _kcc_parse_is_type_token
    cbz x0, .Lpto_expr
    // typeof(type-name)
    bl _kcc_parse_type_name
    mov x19, x0
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    mov x0, #AST_TYPE_TYPEOF
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    mov x1, #1
    str x1, [x0, #AST_F1]         // is_type = 1
    b .Lpto_done
.Lpto_expr:
    // typeof(expression)
    bl _kcc_parse_expr
    mov x19, x0
```

```
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    mov x0, #AST_TYPE_TYPEOF
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    str xzr, [x0, #AST_F1]        // is_type = 0
.Lpto_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

184b · defined at 184b, 186b, 188, 189 · used in 29c · defines _kcc_parse_skip_attributes 181, 182, 186b, 190, 195, 198, 199ab, 200b, _kcc_parse_typeof 188 · uses _kcc_ast_alloc 168a, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_is_type_token 202, _kcc_parse_type_name 201, _kcc_pp_next 108, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

Recognising the layout attributes. The identifier inside an attribute group is matched against the layout-affecting names by length and then bytes. A match on packed (or the __packed__ spelling) sets the packed flag immediately; a match on aligned instead arms x20, so the next number token inside the group—the N in aligned(N)—is read as the alignment. The token text pointer is held in the callee-saved x21 because _kcc_memcmp clobbers the argument registers.

⟨ *Decl: classify layout attribute* 186a ⟩ ≡

```
    adrp x21, _kcc_tok_start@PAGE
    add  x21, x21, _kcc_tok_start@PAGEOFF
    ldr  x21, [x21]               // token text ptr
    (survives memcmp)
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr  x0, [x0]                 // token length
    cmp x0, #6
    b.ne .Lcla_pk2
    mov x0, x21
    adrp x1, _kcc_attr_s_packed@PAGE
    add  x1, x1, _kcc_attr_s_packed@PAGEOFF
    mov x2, #6
    bl _kcc_memcmp
    cbz x0, .Lcla_set_packed
    b .Lcla_done
.Lcla_pk2:
    cmp x0, #10
    b.ne .Lcla_al
    mov x0, x21
    adrp x1, _kcc_attr_s_packed2@PAGE
    add  x1, x1, _kcc_attr_s_packed2@PAGEOFF
    mov x2, #10
    bl _kcc_memcmp
    cbz x0, .Lcla_set_packed
    b .Lcla_done
.Lcla_al:
    cmp x0, #7
    b.ne .Lcla_al2
    mov x0, x21
    adrp x1, _kcc_attr_s_aligned@PAGE
    add  x1, x1, _kcc_attr_s_aligned@PAGEOFF
    mov x2, #7
    bl _kcc_memcmp
    cbz x0, .Lcla_arm_align
    b .Lcla_done
.Lcla_al2:
    cmp x0, #11
    b.ne .Lcla_done
    mov x0, x21
    adrp x1, _kcc_attr_s_aligned2@PAGE
    add  x1, x1, _kcc_attr_s_aligned2@PAGEOFF
    mov x2, #11
    bl _kcc_memcmp
    cbnz x0, .Lcla_done
.Lcla_arm_align:
    mov x20, #1                   // next number
    token is the aligned() value
    b .Lcla_done
.Lcla_set_packed:
    adrp x0, _kcc_attr_packed@PAGE
    add  x0, x0, _kcc_attr_packed@PAGEOFF
    mov x1, #1
    str x1, [x0]
.Lcla_done:
```

□

186a · used in 184b · uses _kcc_tok_len 104b, _kcc_tok_start 104b

⟨ *Decl: parse specifiers* 186b ⟩ + ≡

```
// -- kcc_parse_decl_specs ---
// (node*, uint64_t) kcc_parse_decl_specs(void)
// Parses declaration specifiers.
// Returns: x0 = type node, x1 = storage class
//   storage class: 0=none, 1=typedef, 2=extern,
  3=static, 4=auto, 5=register
.globl _kcc_parse_decl_specs
.p2align 2
_kcc_parse_decl_specs:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, #0                   // x19 = storage
    class (0=none)
    mov x20, #0                   // x20 = type
    node (built incrementally)
    mov x21, #0                   // x21 =
    qualifier flags
    mov x22, #0                   // x22 = count of
    type specifiers seen
    mov x23, #0                   // x23 = base
    type token (for builtins)

.Ldecl_specs_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    // Storage class specifiers
    cmp x0, #TOK_TYPEDEF
    b.eq .Ldecl_sc_typedef
    cmp x0, #TOK_EXTERN
    b.eq .Ldecl_sc_extern
    cmp x0, #TOK_STATIC
    b.eq .Ldecl_sc_static
    cmp x0, #TOK_AUTO
    b.eq .Ldecl_sc_auto
    cmp x0, #TOK_REGISTER
    b.eq .Ldecl_sc_register

    // Type qualifiers
    cmp x0, #TOK_CONST
    b.eq .Ldecl_qual_const
    cmp x0, #TOK_VOLATILE
    b.eq .Ldecl_qual_volatile
    cmp x0, #TOK_RESTRICT
    b.eq .Ldecl_qual_restrict
    cmp x0, #TOK__ATOMIC
    b.eq .Ldecl_qual_atomic
```

```
    // Function specifier
    cmp x0, #TOK_INLINE
    b.eq .Ldecl_skip_spec
    cmp x0, #TOK__NORETURN
    b.eq .Ldecl_skip_spec

    // Alignment specifier: _Alignas ( constant-expr | type-name )
    cmp x0, #TOK__ALIGNAS
    b.eq .Ldecl_alignas

    // Type specifiers
    cmp x0, #TOK_VOID
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK_CHAR
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK_SHORT
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK_INT
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK_LONG
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK_FLOAT
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK_DOUBLE
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK_SIGNED
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK_UNSIGNED
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK__BOOL
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK__COMPLEX
    b.eq .Ldecl_type_builtin
    cmp x0, #TOK_INT128
    b.eq .Ldecl_type_builtin

    // struct/union/enum
    cmp x0, #TOK_STRUCT
    b.eq .Ldecl_type_struct
    cmp x0, #TOK_UNION
    b.eq .Ldecl_type_union
    cmp x0, #TOK_ENUM
    b.eq .Ldecl_type_enum

    // GNU typeof(...) - a complete type specifier
    cmp x0, #TOK_TYPEOF
    b.eq .Ldecl_type_typeof

    // Typedef name (identifier that is a typedef)
    cmp x0, #TOK_IDENT
    b.eq .Ldecl_check_typedef_name

    // _Thread_local (storage class like)
    cmp x0, #TOK__THREAD_LOCAL
    b.eq .Ldecl_skip_spec

    // GNU __attribute__((...)) / __extension__ interleaved with specifiers
    cmp x0, #TOK_ATTRIBUTE
    b.eq .Ldecl_attrs
    cmp x0, #TOK_EXTENSION
    b.eq .Ldecl_attrs

    // Not a specifier, done
    b .Ldecl_specs_done

.Ldecl_attrs:
    bl _kcc_parse_skip_attributes
    b .Ldecl_specs_loop

.Ldecl_sc_typedef:
    mov x19, #1
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_sc_extern:
    mov x19, #2
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_sc_static:
    mov x19, #3
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_sc_auto:
    mov x19, #4
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_sc_register:
    mov x19, #5
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_qual_const:
    orr x21, x21, #1
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_qual_volatile:
    orr x21, x21, #2
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_qual_restrict:
    orr x21, x21, #4
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_qual_atomic:
    orr x21, x21, #8
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_skip_spec:
    // Consume and ignore (inline, _Noreturn, _Thread_local)
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_alignas:
    // _Alignas ( constant-expr | type-name ): accept the specifier and consume
    // its parenthesised argument by skipping balanced tokens (x24 = paren
    // depth; x24 is unused by the loop and preserved across the calls below).
    // The requested alignment is accepted syntactically; enforcing over-aligned
    // storage beyond the frame's natural guarantee is a separate refinement.
    bl _kcc_pp_next                 // consume '_Alignas'
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect            // consume '('
    mov x24, #1                     // paren depth
.Ldecl_alignas_skip:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Ldecl_specs_loop      // TOK_EOF (0): malformed → bail to the loop
    cmp  x0, #TOK_LPAREN
    b.ne .Ldecl_alignas_notlp
    add  x24, x24, #1
    b    .Ldecl_alignas_adv
.Ldecl_alignas_notlp:
    cmp  x0, #TOK_RPAREN
    b.ne .Ldecl_alignas_adv
```

```
    subs x24, x24, #1
    b.eq .Ldecl_alignas_close
.Ldecl_alignas_adv:
    bl _kcc_pp_next
    b .Ldecl_alignas_skip
.Ldecl_alignas_close:
    bl _kcc_pp_next                 // consume the
   matching ')'
    b .Ldecl_specs_loop
```

□

186b · defined at 184b, 186b, 188, 189 · used in 29c · defines _kcc_parse_decl_specs 181, 195, 199a, 201 · uses _kcc_parse_expect 168b, _kcc_parse_skip_attributes 184b, _kcc_pp_next 108, _kcc_tok_type 104b

Type specifier bitmask accumulation. When the main loop dispatches to a builtin type keyword such as `int`, `char`, or `long`, the parser does not build a type node immediately. Instead, each keyword sets a corresponding bit in the accumulator register x23. This approach handles multi-keyword types like `unsigned long long int` correctly: once all specifiers have been consumed, the final bitmask encodes the full type. The special case of two successive `long` keywords is detected by checking whether bit 4 is already set before promoting to bit 5 (`long long`). The GNU `__int128` keyword sets bit 12, combining with `signed` or `unsigned` the same way the standard integer specifiers do. Struct, union, and enum specifiers build their own type nodes via dedicated sub-parsers, while typedef names are looked up in the typedef table and interned. The GNU `__auto_type` specifier—an identifier, so matched by spelling—infers its type from the initializer: it emits a deferred `typeof` node (F1=2) that `kcc_parse_decl` fills with the initializer expression once parsed, and `kcc_parse_is_type_token` likewise reports it as a type so a block-scope `__auto_type x = e;` is parsed as a declaration rather than an expression.

⟨ *Decl: parse specifiers* 188 ⟩ +≡

```
.Ldecl_type_builtin:
    // Accumulate specifier bitmask in x23
    // Each type keyword ORs its bit into x23
    cmp x0, #TOK_VOID
    b.eq .Ldecl_bit_void
    cmp x0, #TOK_CHAR
    b.eq .Ldecl_bit_char
    cmp x0, #TOK_SHORT
    b.eq .Ldecl_bit_short
    cmp x0, #TOK_INT
    b.eq .Ldecl_bit_int
    cmp x0, #TOK_LONG
    b.eq .Ldecl_bit_long
    cmp x0, #TOK_FLOAT
    b.eq .Ldecl_bit_float
    cmp x0, #TOK_DOUBLE
    b.eq .Ldecl_bit_double
    cmp x0, #TOK_SIGNED
    b.eq .Ldecl_bit_signed
    cmp x0, #TOK_UNSIGNED
    b.eq .Ldecl_bit_unsigned
    cmp x0, #TOK__BOOL
    b.eq .Ldecl_bit_bool
    cmp x0, #TOK__COMPLEX
    b.eq .Ldecl_bit_complex
    cmp x0, #TOK__IMAGINARY
    b.eq .Ldecl_bit_complex
    cmp x0, #TOK_INT128
    b.eq .Ldecl_bit_int128
    b .Ldecl_bit_done

.Ldecl_bit_void:
    orr x23, x23, #(1 << 0)
    b .Ldecl_bit_done
.Ldecl_bit_char:
    orr x23, x23, #(1 << 1)
    b .Ldecl_bit_done
.Ldecl_bit_short:
    orr x23, x23, #(1 << 2)
    b .Ldecl_bit_done
.Ldecl_bit_int:
    orr x23, x23, #(1 << 3)
    b .Ldecl_bit_done
.Ldecl_bit_long:
    // Check if long already set (bit 4) -> set
   long long (bit 5)
    tst x23, #(1 << 4)
    b.ne .Ldecl_bit_llong
    orr x23, x23, #(1 << 4)
    b .Ldecl_bit_done
.Ldecl_bit_llong:
    orr x23, x23, #(1 << 5)
    b .Ldecl_bit_done
.Ldecl_bit_float:
    orr x23, x23, #(1 << 6)
    b .Ldecl_bit_done
.Ldecl_bit_double:
    orr x23, x23, #(1 << 7)
    b .Ldecl_bit_done
.Ldecl_bit_signed:
    orr x23, x23, #(1 << 8)
    b .Ldecl_bit_done
.Ldecl_bit_unsigned:
    orr x23, x23, #(1 << 9)
    b .Ldecl_bit_done
.Ldecl_bit_bool:
    orr x23, x23, #(1 << 10)
    b .Ldecl_bit_done
.Ldecl_bit_int128:
    // __int128: the GNU 128-bit integer type
   specifier (bit 12).
    orr x23, x23, #(1 << 12)
    b .Ldecl_bit_done
.Ldecl_bit_complex:
    // _Complex / _Imaginary: kcc has no complex-
   number type.  Treating a
    // complex as its real part would silently
   miscompile complex arithmetic, so
    // reject it (see Appendix, Known Limitations)
   .  The specifier is still
    // consumed below so parsing can recover; the
   error count fails the compile.
    adrp x0, _kcc_tok_line@PAGE
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_col@PAGE
    add  x1, x1, _kcc_tok_col@PAGEOFF
    ldr  x1, [x1]
    adrp x2, _kcc_decl_msg_complex@PAGE
    add  x2, x2, _kcc_decl_msg_complex@PAGEOFF
    bl _kcc_diag_error
    orr x23, x23, #(1 << 11)

.Ldecl_bit_done:
    add x22, x22, #1            // count++
    bl _kcc_pp_next
    b .Ldecl_specs_loop

.Ldecl_type_struct:
    bl _kcc_parse_struct_or_union
    mov x20, x0
    add x22, x22, #1
```

```
    b .Ldecl_specs_loop

.Ldecl_type_union:
    bl _kcc_parse_struct_or_union
    mov x20, x0
    add x22, x22, #1
    b .Ldecl_specs_loop

.Ldecl_type_enum:
    bl _kcc_parse_enum
    mov x20, x0
    add x22, x22, #1
    b .Ldecl_specs_loop

.Ldecl_type_typeof:
    bl _kcc_parse_typeof
    mov x20, x0
    add x22, x22, #1
    b .Ldecl_specs_loop

.Ldecl_check_typedef_name:
    // Only treat as typedef name if no type
   specifier seen yet
    cbnz x22, .Ldecl_specs_done

    // GNU __auto_type: the type is inferred from
   the initializer parsed later
    // (`__auto_type x = e` means `typeof(e) x = e
   `).  Emit a deferred typeof
    // node (F1=2) that kcc_parse_decl fills with
   the initializer.
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    cmp  x1, #11
    b.ne .Ldecl_specs_not_auto
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_str_auto_type@PAGE
    add  x1, x1, _kcc_str_auto_type@PAGEOFF
    mov  x2, #11
    bl _kcc_memcmp
    cbnz x0, .Ldecl_specs_not_auto
    bl _kcc_pp_next                 // consume
   __auto_type
    mov x0, #AST_TYPE_TYPEOF
    bl _kcc_ast_alloc
    str xzr, [x0, #AST_F0]          // deferred
   operand
    mov x1, #2
    str x1, [x0, #AST_F1]           // F1=2: auto,
   fill F0 from the initializer
    mov x20, x0                     // x20 = type
   node
    add x22, x22, #1                // count++
    b .Ldecl_specs_loop

.Ldecl_specs_not_auto:
    // Check if this identifier is a typedef name
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_typedef_lookup
    cbz x0, .Ldecl_specs_done    // not a typedef
   name

    // It's a typedef name: intern it and create
   TYPE_NAME node
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x24, x0                      // x24 = interned
   name

    bl _kcc_pp_next                  // consume the
   typedef name

    mov x0, #AST_TYPE_NAME
    bl _kcc_ast_alloc
    str x24, [x0, #AST_F0]       // name_ptr
    mov x20, x0
    add x22, x22, #1

    b .Ldecl_specs_loop
```

□

188 · defined at 184b, 186b, 188, 189 · used in 29c · uses _kcc_ast_alloc 168a, _kcc_diag_error 76a, _kcc_parse_enum 200b, _kcc_parse_struct_or_union 198, _kcc_parse_typeof 184b, _kcc_pp_next 108, _kcc_strtab_intern 66b, _kcc_tok_col 104b, _kcc_tok_len 104b, _kcc_tok_line 104b, _kcc_tok_start 104b, _kcc_typedef_lookup 171

Assembling the final type. After the specifier loop terminates — because the current token no longer matches any specifier — the accumulated state is resolved into a type node. If the loop consumed a struct, union, enum, or typedef name, the type is already stored in x20. Otherwise a TYPE_BUILTIN node is allocated whose AST_F0 field holds the bitmask from x23. When no specifiers appeared at all, the default type is int (the implicit-int rule). Finally, if any qualifiers were collected in x21, the type is wrapped in a TYPE_QUALS node before returning.

⟨ *Decl: parse specifiers* 189 ⟩ +≡

```
.Ldecl_specs_done:
    // Build the final type node
    // If we have a builtin type and no struct/
   union/enum type already
    cbz x20, .Ldecl_build_builtin
    b .Ldecl_apply_quals

.Ldecl_build_builtin:
    // Create builtin type node from accumulated
   specifier bitmask
    // If no type specifiers at all, default to
   int (bit 3)
    cbz x22, .Ldecl_default_int
    mov x0, #AST_TYPE_BUILTIN
    bl _kcc_ast_alloc
    str x23, [x0, #AST_F0]       // specifier
   bitmask
    mov x20, x0
    b .Ldecl_apply_quals

.Ldecl_default_int:
    mov x0, #AST_TYPE_BUILTIN
    bl _kcc_ast_alloc
    mov x1, #(1 << 3)            // SPEC_INT
   bitmask
    str x1, [x0, #AST_F0]
    mov x20, x0

.Ldecl_apply_quals:
    // If qualifiers present, wrap in TYPE_QUALS
    cbz x21, .Ldecl_specs_return
```

```
    mov x24, x20                // save base type
    mov x0, #AST_TYPE_QUALS
    bl _kcc_ast_alloc
    str x24, [x0, #AST_F0]      // base_type
    str x21, [x0, #AST_F1]      // qualifiers
    mov x20, x0

.Ldecl_specs_return:
    mov x0, x20                 // type node
    mov x1, x19                 // storage class

    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

189 · defined at 184b, 186b, 188, 189 · used in 29c · uses _kcc_ast_alloc 168a

21.3. Declarator

The declarator (§6.7.6) introduces a name and modifies the base type with pointer, array, and function type constructors. The syntax is:

pointer? direct-declarator

where a pointer is one or more `*` optionally followed by type qualifiers. A GNU `__attribute__((…))` is also tolerated among those qualifiers (a pointer attribute) and immediately before the direct declarator's name; kcc does not model such attributes, so it consumes and discards them. The kernel headers rely on this—`extern char * __must_check skip_spaces(const char *)` places `__attribute__((__warn_unused_result__))` exactly between the `*` and the name.

⟨ *Decl: parse declarator* 190 ⟩ ≡

```
// -- kcc_parse_declarator ---
// node *kcc_parse_declarator(node *base_type)
//   x0 = base type node
// Parses a declarator and returns a declaration
   node
// (VAR_DECL or FUNC_DECL) with the type and name
   filled in.
.globl _kcc_parse_declarator
.p2align 2
_kcc_parse_declarator:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                 // x19 = current
   type

    // Parse pointer(s)
.Ldeclr_ptr_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_STAR
    b.ne .Ldeclr_direct

    // Consume '*'
    bl _kcc_pp_next

    // Parse optional qualifiers after *
    mov x20, #0                 // qualifier
   flags
.Ldeclr_ptr_quals:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_CONST
    b.eq .Ldeclr_pq_const
    cmp x0, #TOK_VOLATILE
    b.eq .Ldeclr_pq_volatile
    cmp x0, #TOK_RESTRICT
    b.eq .Ldeclr_pq_restrict
    cmp x0, #TOK__ATOMIC
    b.eq .Ldeclr_pq_atomic
    cmp x0, #TOK_ATTRIBUTE
    b.eq .Ldeclr_pq_attr
    b .Ldeclr_ptr_build

.Ldeclr_pq_attr:
    // GNU `__attribute__((...))` after the `*` (a
    pointer attribute, e.g.
    // `char * __must_check f(...)` in the kernel
   headers).  kcc does not model
    // pointer attributes, so consume and ignore
   them, then keep scanning.
    bl _kcc_parse_skip_attributes
    b .Ldeclr_ptr_quals

.Ldeclr_pq_const:
    orr x20, x20, #1
    bl _kcc_pp_next
    b .Ldeclr_ptr_quals
.Ldeclr_pq_volatile:
    orr x20, x20, #2
    bl _kcc_pp_next
    b .Ldeclr_ptr_quals
.Ldeclr_pq_restrict:
    orr x20, x20, #4
    bl _kcc_pp_next
    b .Ldeclr_ptr_quals
.Ldeclr_pq_atomic:
    orr x20, x20, #8
    bl _kcc_pp_next
    b .Ldeclr_ptr_quals

.Ldeclr_ptr_build:
    // Build pointer type node
    mov x0, #AST_TYPE_PTR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // pointee type
    str x20, [x0, #AST_F1]      // qualifiers
    mov x19, x0                 // new current
   type
    b .Ldeclr_ptr_loop

.Ldeclr_direct:
    // A GNU attribute may also sit just before
   the direct declarator's name
    // (`T __attribute__((...)) x`); skip it (a no
   -op when absent).
    bl _kcc_parse_skip_attributes
    // Parse direct declarator
    mov x0, x19
    bl _kcc_parse_direct_declarator

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

190 · used in 29c · defines _kcc_parse_declarator 181, 182, 192, 195, 199a · uses _kcc_ast_alloc 168a,

_kcc_parse_direct_declarator 191b,
_kcc_parse_skip_attributes 184b, _kcc_pp_next 108,
_kcc_tok_type 104b

A typedef (or any object declaration) whose declarator is a bare function — `typedef int filler_t(struct file *, struct folio *)` (the kernel's `filler_t`) or `int f(int)` used as a type — parses into an `AST_FUNC_DECL` node (`F0` = return type, `F2` = params). When the declaration path re-tags that node to `AST_TYPEDEF_DECL`, sema reads `F0` as the whole type and the function-ness is lost: the typedef collapses to the *return* type, so `filler_t x; x(…)` fails with "called object is not a function". This fixup rebuilds `F0` as an `AST_TYPE_FUNC` carrying the return type and the parameter list, so the typedef resolves to a genuine `TY_FUNC` — which a parameter then adjusts to pointer-to-function (§6.7.6.3) and which is directly callable as a function designator.

⟨ *Decl: parse declaration* 191a ⟩ + ≡

```
// -- kcc_parse_typedef_funcfix ---
// void kcc_parse_typedef_funcfix(node *decl)
//   If decl is an AST_FUNC_DECL, replace its F0 (currently the bare return
//   type) with an AST_TYPE_FUNC type node built from the return type and the
//   parameter list, so a typedef/object of function type keeps its function
//   type instead of decaying to the return type.
//   Any other node is untouched.
.text
.p2align 2
_kcc_parse_typedef_funcfix:
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_FUNC_DECL
    b.ne .Ltff_ret
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                     // the AST_FUNC_DECL node
    mov x0, #AST_TYPE_FUNC
    bl _kcc_ast_alloc               // x0 = new type node
    ldr x1, [x19, #AST_F0]          // return type
    str x1, [x0, #AST_F0]
    ldr x1, [x19, #AST_F2]          // params | variadic (bit 0)
    and x2, x1, #1                  // variadic flag
    bic x1, x1, #1                  // params list
    str x1, [x0, #AST_F1]           // params
    str x2, [x0, #AST_F2]           // variadic
    str x0, [x19, #AST_F0]          // decl's type is now the function type
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
.Ltff_ret:
    ret
```

□

191a · defined at 181, 182, 184a, 191a · used in 29c · defines _kcc_parse_typedef_funcfix 182, 184a · uses _kcc_ast_alloc 168a

21.4. Direct Declarator

The direct declarator is where the name lives, along with array dimensions and function parameter lists. Inside a bracketed array dimension the parser also tolerates the parameter-only ornaments of §6.7.6.3 — a leading type-qualifier list, a `static` keyword, and a `*` standing for an unspecified variable-length bound. Because kcc decays an array parameter to a pointer regardless, these are parsed and discarded rather than modelled, so declarations like `int x[static 5]` or `int x[const *]` are accepted.

⟨ *Decl: parse direct declarator* 191b ⟩ ≡

```
// -- kcc_parse_direct_declarator ---
// node *kcc_parse_direct_declarator(node *type)
//   x0 = type built so far
// Returns a VarDecl or FuncDecl node.
.globl _kcc_parse_direct_declarator
.p2align 2
_kcc_parse_direct_declarator:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0                     // x19 = type
    mov x20, #0                     // x20 = name_ptr

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    // Check for '(' - grouped declarator like (*fp)
    cmp x0, #TOK_LPAREN
    b.eq .Lddecl_grouped

    // Must be an identifier
    cmp x0, #TOK_IDENT
    b.ne .Lddecl_no_name

    // Intern the name
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x20, x0                     // x20 = interned name

    // Consume identifier
    bl _kcc_pp_next

.Lddecl_no_name:
    // Check for array/function suffixes
    mov x24, #0                     // innermost array node so far (multi-dim chain)
.Lddecl_suffix_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    cmp x0, #TOK_LBRACKET
    b.eq .Lddecl_array

    cmp x0, #TOK_LPAREN
    b.eq .Lddecl_function
```

```
    // No more suffixes - build the declaration
   node
    b .Lddecl_build_var

.Lddecl_array:
    // Array: [ qualifiers? static? size? ]
    bl _kcc_pp_next                 // consume '['

    // A parameter's outermost array may carry
   type-qualifiers, a `static`
    // keyword, or a `*` for an unspecified VLA
   bound (\stdref{6.7.6.3}).  These
    // affect only the parameter-as-pointer, which
    kcc already decays, so we
    // parse and discard them here rather than
   modelling them.
.Lddecl_array_quals:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_CONST
    b.eq .Lddecl_array_skip_qual
    cmp  x0, #TOK_VOLATILE
    b.eq .Lddecl_array_skip_qual
    cmp  x0, #TOK_RESTRICT
    b.eq .Lddecl_array_skip_qual
    cmp  x0, #TOK_STATIC
    b.eq .Lddecl_array_skip_qual
    b .Lddecl_array_after_quals
.Lddecl_array_skip_qual:
    bl _kcc_pp_next                 // consume
   qualifier / static
    b .Lddecl_array_quals

.Lddecl_array_after_quals:
    mov x21, #0                     // size expr (
   default null)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RBRACKET
    b.eq .Lddecl_array_close
    // `[*]`: unspecified VLA size in a prototype
   behaves as an empty bound.
    cmp  x0, #TOK_STAR
    b.ne .Lddecl_array_size
    bl _kcc_pp_next                 // consume '*'
    b .Lddecl_array_close

.Lddecl_array_size:
    // Parse size expression
    bl _kcc_parse_assign_expr
    mov x21, x0

.Lddecl_array_close:
    mov x0, #TOK_RBRACKET
    bl _kcc_parse_expect

    // Build array type.  For `a[N][M]` the FIRST
   bracket is the outermost
    // array (\stdref{6.7.6.2}: array N of array M
    of T).  Reading brackets
    // left-to-right, each later dimension must
   nest \emph{inside} the current
    // innermost array (as its element), not wrap
   the whole type---otherwise the
    // dimensions reverse and \texttt{a[i][j]}
   strides by the wrong extent.
    mov x0, #AST_TYPE_ARRAY
    bl _kcc_ast_alloc
    str x21, [x0, #AST_F1]      // size expr
    cbz x24, .Lddecl_array_first
    // Later dimension: element = the base element
    (held in the innermost's F0),
    // then splice this array in as that innermost
    array's element.
    ldr x1, [x24, #AST_F0]      // base element
   type
    str x1, [x0, #AST_F0]
    str x0, [x24, #AST_F0]      // innermost.
   element = new inner array
    mov x24, x0                  // new innermost
    b .Lddecl_suffix_loop
.Lddecl_array_first:
    str x19, [x0, #AST_F0]      // first bracket:
   element = the declared type
    mov x19, x0                  // top-level type
    = this array
    mov x24, x0                  // innermost =
   this array
    b .Lddecl_suffix_loop
```

□

191b · defined at 191b, 192, 195 · used in 29c · defines _kcc_parse_direct_declarator 190 · uses _kcc_ast_alloc 168a, _kcc_parse_assign_expr 208a, _kcc_parse_expect 168b, _kcc_pp_next 108, _kcc_strtab_intern 66b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

Function parameter lists and grouped declarators. When the suffix loop encounters an opening parenthesis, the parser must distinguish between a function parameter list and a grouped (parenthesised) declarator such as `int (*fp)(int)`. Grouped declarators are detected at the start of direct-declarator parsing; by the time the suffix loop runs, any remaining parenthesis introduces a parameter list. The parser builds a `TYPE_FUNC` node, iterates over comma-separated parameter declarations, and handles the variadic `...` marker. A `FUNC_DECL` or `VAR_DECL` node is then constructed to carry the name and type. The variadic flag rides in *bit 0* of the `FUNC_DECL`'s params pointer: parameter nodes are eight-byte aligned, so the low bit is free, and tagging it there avoids growing the node. Every consumer that walks the parameter list—sema and the IR builder—masks the bit off first; sema also reads it to record the fixed-parameter count that the variadic calling convention needs (§29.9.13).

The grouped declarator handler at `.Lddecl_grouped` first parses the inner declarator (e.g. `*fp`), then checks whether a parenthesised parameter list follows the closing `)`. When one is present, the handler parses it into a `TYPE_FUNC` node and walks the inner declarator's type chain to replace the base type with the function type—implementing C's inside-out declarator semantics. For instance, `int (*fp)(int, int)` produces a `VAR_DECL` whose type is `ptr(func(int,int)->int)`. An array suffix `[N]` after the group is handled the same way: the brackets build an array chain whose element is the base type, which is then spliced in place of the base, so `char (*p)[4]` becomes `ptr(array 4 of char)` — a pointer to an array rather than an array of pointers.

⟨ *Decl: parse direct declarator* 192 ⟩ +≡

```
.Lddecl_function:
    // Function: ( param-list ).  A function
   suffix ends any array chain, so a
    // subsequent bracket begins a fresh outermost
    array.
```

```
    mov x24, #0
    bl _kcc_pp_next                 // consume '('

    // Build function type
    mov x0, #AST_TYPE_FUNC
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]        // return type
    mov x21, x0                     // x21 = func
type node

    // Check for empty params or 'void'
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RPAREN
    b.eq .Lddecl_func_close

    // Check for 'void)'
    cmp  x0, #TOK_VOID
    b.ne .Lddecl_parse_params

    // Could be 'void)' or 'void *name'
    // For now, if void followed by ')', treat as
no params
    // This is a simplification; full C17 requires
lookahead
    // We'll handle it by checking: if void and
next is ')', consume both
    // Otherwise fall through to param parsing

    // Peek: we can't easily look ahead, so just
try parsing params
    b .Lddecl_parse_params

.Lddecl_parse_params:
    // Parse parameter list
    mov x22, #0                     // x22 = first
param
    mov x23, #0                     // x23 = last
param

.Lddecl_param_loop:
    // Check for '...'
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ELLIPSIS
    b.eq .Lddecl_variadic

    // Parse parameter declaration
    bl _kcc_parse_param_decl
    mov x0, x0                      // param node

    cbz x22, .Lddecl_param_first
    str x0, [x23, #AST_F4]        // last->next =
new
    mov x23, x0
    b .Lddecl_param_comma

.Lddecl_param_first:
    mov x22, x0
    mov x23, x0

.Lddecl_param_comma:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.ne .Lddecl_func_store_params

    bl _kcc_pp_next                 // consume ','

    // Check for '...' after comma
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ELLIPSIS
    b.eq .Lddecl_variadic

    b .Lddecl_param_loop

.Lddecl_variadic:
    bl _kcc_pp_next                 // consume '...'
    mov x0, #1
    str x0, [x21, #AST_F2]        // variadic = 1

.Lddecl_func_store_params:
    str x22, [x21, #AST_F1]       // params

.Lddecl_func_close:
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Build FuncDecl node
    mov x0, #AST_FUNC_DECL
    bl _kcc_ast_alloc
    ldr x1, [x21, #AST_F0]        // return type
from func type
    str x1, [x0, #AST_F0]         // type (return
type)
    str x20, [x0, #AST_F1]        // name
    ldr x1, [x21, #AST_F1]        // params
    ldr x2, [x21, #AST_F2]        // variadic (0/1)
    orr x1, x1, x2                // tag bit 0 (see
prose)
    str x1, [x0, #AST_F2]         // params |
variadic

    b .Lddecl_done

.Lddecl_grouped:
    // Grouped declarator: ( declarator )
    // Handles function pointers like: int (*fp)(
int, int)
    bl _kcc_pp_next                 // consume '('
    mov x0, x19
    bl _kcc_parse_declarator
    mov x21, x0                     // x21 = inner
result (VAR_DECL)
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // A grouped declarator may be followed by a
function suffix (params)
    // making it a function pointer, or by an
array suffix [N] making it a
    // pointer to (or array of) array --- e.g.
char (*p)[4].
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LBRACKET
    b.eq .Lddecl_grouped_array
    cmp  x0, #TOK_LPAREN
    b.ne .Lddecl_grouped_done

    // Function pointer: parse parameter list
    bl _kcc_pp_next                 // consume '('

    // Build TYPE_FUNC node
    mov x0, #AST_TYPE_FUNC
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]        // return type =
base type
    mov x24, x0                     // x24 = func
type node

    // Check for empty params ')' or void
    adrp x0, _kcc_tok_type@PAGE
```

```
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RPAREN
    b.eq .Lddecl_gfunc_close

    // Parse parameters
    mov x22, #0                  // first param
    mov x23, #0                  // last param
.Lddecl_gfunc_param_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ELLIPSIS
    b.eq .Lddecl_gfunc_variadic

    bl _kcc_parse_param_decl
    cbz x22, .Lddecl_gfunc_first
    str x0, [x23, #AST_F4]
    mov x23, x0
    b .Lddecl_gfunc_comma

.Lddecl_gfunc_first:
    mov x22, x0
    mov x23, x0

.Lddecl_gfunc_comma:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.ne .Lddecl_gfunc_store_params
    bl _kcc_pp_next
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ELLIPSIS
    b.eq .Lddecl_gfunc_variadic
    b .Lddecl_gfunc_param_loop

.Lddecl_gfunc_variadic:
    bl _kcc_pp_next               // consume '...'
    mov x0, #1
    str x0, [x24, #AST_F2]       // variadic = 1

.Lddecl_gfunc_store_params:
    str x22, [x24, #AST_F1]      // store params in
    func type

.Lddecl_gfunc_close:
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Patch inner declarator's type chain:
    // Walk from x21.F0 to find where base_type (
   x19) is stored,
    // replace it with the func type (x24).
    ldr x0, [x21, #AST_F0]       // type chain root
    cmp x0, x19                   // directly the
   base type?
    b.ne .Lddecl_gfunc_walk
    // No pointer wrapper: the group was redundant
    parentheses around a plain
    // function name, `(f)(params)` --- as Csmith'
   s FUNC_NAME macro emits.  This
    // is an ordinary function declarator, not a
   function-typed variable, so
    // retag the inner VAR_DECL as a FUNC_DECL (
   return type, name already set,
    // params|variadic) so it can head a
   definition just like `f(params)`.
    mov x0, #AST_FUNC_DECL
    str x0, [x21, #AST_TAG]
    str x19, [x21, #AST_F0]       // return type
    ldr x1, [x24, #AST_F1]        // params
    ldr x2, [x24, #AST_F2]        // variadic flag
    orr x1, x1, x2                // params |
   variadic (bit 0)
    str x1, [x21, #AST_F2]
    b .Lddecl_grouped_done

.Lddecl_gfunc_walk:
    // Walk type chain: each pointer's F0 →
   pointee
    mov x1, x0
.Lddecl_gfunc_walk_loop:
    ldr x0, [x1, #AST_F0]
    cmp x0, x19
    b.ne .Lddecl_gfunc_walk_next
    // Found: replace this node's F0 with func
   type
    str x24, [x1, #AST_F0]
    b .Lddecl_grouped_done
.Lddecl_gfunc_walk_next:
    mov x1, x0
    b .Lddecl_gfunc_walk_loop

.Lddecl_grouped_array:
    // Parse one or more [N] suffixes into an
   array chain whose element is the
    // base type, then splice that chain into the
   inner declarator's type in
    // place of the base --- inside-out declarator
    semantics (\stdref{6.7.6}).
    // For char (*p)[4] the inner declarator is
   →ptrchar; splicing array[4] for
    // the char yields →ptrarray→[4]char, i.e. p
   is a pointer to an array of 4.
    mov x24, #0                   // innermost
   array so far
    mov x22, #0                   // outermost
   array (the splice target)
.Lddecl_ga_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LBRACKET
    b.ne .Lddecl_ga_splice
    bl _kcc_pp_next               // consume '['
    mov x23, #0                   // size expr (
   default null)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RBRACKET
    b.eq .Lddecl_ga_rb
    bl _kcc_parse_assign_expr
    mov x23, x0
.Lddecl_ga_rb:
    mov x0, #TOK_RBRACKET
    bl _kcc_parse_expect
    mov x0, #AST_TYPE_ARRAY
    bl _kcc_ast_alloc
    str x23, [x0, #AST_F1]        // size
    cbz x24, .Lddecl_ga_first
    // Nest as the current innermost array's
   element (later dimension).
    ldr x1, [x24, #AST_F0]        // base element
    str x1, [x0, #AST_F0]
    str x0, [x24, #AST_F0]
    mov x24, x0
    b .Lddecl_ga_loop
.Lddecl_ga_first:
    str x19, [x0, #AST_F0]        // first
   dimension: element = base type
    mov x24, x0                   // innermost
    mov x22, x0                   // outermost
    b .Lddecl_ga_loop
.Lddecl_ga_splice:
```

```
    cbz x22, .Lddecl_grouped_done
    ldr x0, [x21, #AST_F0]          // inner
   declarator's type chain root
    cmp x0, x19
    b.ne .Lddecl_ga_walk
    str x22, [x21, #AST_F0]         // base was the
   root: replace directly
    b .Lddecl_grouped_done
.Lddecl_ga_walk:
    mov x1, x0
.Lddecl_ga_walk_loop:
    ldr x0, [x1, #AST_F0]
    cmp x0, x19
    b.ne .Lddecl_ga_walk_next
    str x22, [x1, #AST_F0]          // replace base
   within the chain
    b .Lddecl_grouped_done
.Lddecl_ga_walk_next:
    mov x1, x0
    b .Lddecl_ga_walk_loop

.Lddecl_grouped_done:
    mov x0, x21
    b .Lddecl_done

.Lddecl_build_var:
    // Build VarDecl
    mov x0, #AST_VAR_DECL
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]        // type
    str x20, [x0, #AST_F1]        // name_ptr

.Lddecl_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

192 · defined at 191b, 192, 195 · used in 29c · uses _kcc_ast_alloc 168a, _kcc_parse_assign_expr 208a, _kcc_parse_declarator 190, _kcc_parse_expect 168b, _kcc_parse_param_decl 195, _kcc_pp_next 108, _kcc_tok_type 104b

Parameter declarations. Each parameter in a function prototype is parsed as a miniature declaration: specifiers followed by a declarator. The storage class returned by `kcc_parse_decl_specs` is ignored since storage class specifiers are not meaningful for parameters. If the next token after the specifiers is a closing parenthesis, comma, or ellipsis, the parameter is unnamed — an abstract parameter consisting of a type alone.

⟨ *Decl: parse direct declarator* 195 ⟩ +≡

```
// -- kcc_parse_param_decl ---
// node *kcc_parse_param_decl(void)
// Parses a parameter declaration.
.globl _kcc_parse_param_decl
.p2align 2
_kcc_parse_param_decl:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    // Parse specifiers (ignore storage class for
   params)
    bl _kcc_parse_decl_specs
    mov x19, x0                    // x19 = type
    // x1 = storage class (ignored)

    // Check if there's a declarator or abstract
   declarator
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    // If ')' or ',' follows, it's an unnamed
   parameter
    cmp  x0, #TOK_RPAREN
    b.eq .Lparam_unnamed
    cmp  x0, #TOK_COMMA
    b.eq .Lparam_unnamed
    cmp  x0, #TOK_ELLIPSIS
    b.eq .Lparam_unnamed

    // Try to parse a declarator
    mov x0, x19
    bl _kcc_parse_declarator
    mov x20, x0                     // x20 = decl
   node

    bl _kcc_parse_skip_attributes  // GNU attrs on
    a parameter: `int x __attribute__((..))`

    // Convert to PARAM_DECL
    mov x0, #AST_PARAM_DECL
    bl _kcc_ast_alloc
    ldr x1, [x20, #AST_F0]       // type from
   declarator
    str x1, [x0, #AST_F0]
    ldr x1, [x20, #AST_F1]       // name from
   declarator
    str x1, [x0, #AST_F1]

    b .Lparam_done

.Lparam_unnamed:
    // Unnamed parameter
    mov x0, #AST_PARAM_DECL
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // type
    // F1 (name) stays null

.Lparam_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

195 · defined at 191b, 192, 195 · used in 29c · defines _kcc_parse_param_decl 192, 196 · uses _kcc_ast_alloc 168a, _kcc_parse_declarator 190, _kcc_parse_decl_specs 186b, _kcc_parse_skip_attributes 184b, _kcc_tok_type 104b

21.5. Abstract Declarator

An abstract declarator is used in casts and `sizeof(type)`. It is like a regular declarator but without a name. Besides pointers and array/function suffixes, it may *group* an inner declarator in parentheses — `void (*)(void)` is a pointer to a function — which this parser handles the same inside-out way as a named grouped declarator: it parses the inner `(*)`, then applies the trailing `(params)` or `[N]` suffix by splicing it in place of the base type (`_kcc_abst_splice`), yielding `ptr(func(void)→void)`.

⟨ Decl: parse abstract 196 ⟩ ≡

```
// -- kcc_parse_abstract_declarator ---
// node *kcc_parse_abstract_declarator(node *base_type)
//   x0 = base type
// Returns a type node (possibly wrapped in pointers/arrays).
.globl _kcc_parse_abstract_declarator
.p2align 2
_kcc_parse_abstract_declarator:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0                     // x19 = type

    // A '(' that begins the abstract declarator and is followed by '*' or
    // another '(' groups an inner declarator --- the pointer of a function or
    // array pointer such as \texttt{void (*)(void)}.  (A '(' followed by a type
    // or ')' is instead a function parameter list, handled by the suffix code.)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LPAREN
    b.ne .Labst_ptr_loop
    bl _kcc_pp_next                 // consume the grouping '('
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_STAR
    b.eq .Labst_grouped
    cmp  x0, #TOK_LPAREN
    b.eq .Labst_grouped
    // Not a grouping: the '(' opened a parameter list over the base type
    // (a bare function type-name); build the function type directly.
    mov x20, x19                    // "inner" is just the base
    b .Labst_grp_func_body

.Labst_grouped:
    // Parse the inner abstract declarator (the '*'), then its closing ')'.
    mov x0, x19
    bl _kcc_parse_abstract_declarator
    mov x20, x0                     // inner declarator (e.g. ptr(base))
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    // A function or array suffix now applies to the group and is spliced in
    // place of the base type, giving inside-out semantics.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LPAREN
    b.eq .Labst_grp_func
    cmp  x0, #TOK_LBRACKET
    b.eq .Labst_grp_array
    // No suffix: the group was just a parenthesised inner declarator.
    mov x19, x20
    b .Labst_done

.Labst_grp_func:
    bl _kcc_pp_next                 // consume the parameter-list '('
.Labst_grp_func_body:
    mov x0, #AST_TYPE_FUNC
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]          // return type = base
    mov x21, x0                     // x21 = function type node
    mov x22, #0                     // first param
    mov x23, #0                     // last param
.Labst_grp_param_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RPAREN
    b.eq .Labst_grp_params_done
    cmp  x0, #TOK_ELLIPSIS
    b.eq .Labst_grp_variadic
    bl _kcc_parse_param_decl
    cbz x22, .Labst_grp_first
    str x0, [x23, #AST_F4]
    mov x23, x0
    b .Labst_grp_comma
.Labst_grp_first:
    mov x22, x0
    mov x23, x0
.Labst_grp_comma:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.ne .Labst_grp_params_done
    bl _kcc_pp_next                 // consume ','
    b .Labst_grp_param_loop
.Labst_grp_variadic:
    bl _kcc_pp_next                 // consume '...'
    mov x0, #1
    str x0, [x21, #AST_F2]          // variadic flag
.Labst_grp_params_done:
    str x22, [x21, #AST_F1]         // params
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    // Splice the function type into the inner declarator, replacing the base.
    mov x0, x20
    mov x1, x19                     // old base
    mov x2, x21                     // new (function) type
    bl _kcc_abst_splice
    mov x19, x0                     // resulting head
    b .Labst_done

.Labst_grp_array:
    bl _kcc_pp_next                 // consume '['
    mov x22, #0                     // size expr
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RBRACKET
    b.eq .Labst_grp_arr_close
    bl _kcc_parse_assign_expr
    mov x22, x0
.Labst_grp_arr_close:
    mov x0, #TOK_RBRACKET
    bl _kcc_parse_expect
    mov x0, #AST_TYPE_ARRAY
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]          // element = base
    str x22, [x0, #AST_F1]          // size
    mov x21, x0
    mov x0, x20
```

```
    mov x1, x19
    mov x2, x21
    bl _kcc_abst_splice
    mov x19, x0                     // resulting
   head
    b .Labst_done

    // Parse pointer(s)
.Labst_ptr_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_STAR
    b.ne .Labst_done

    bl _kcc_pp_next                 // consume '*'

    // Parse qualifiers
    mov x20, #0
.Labst_quals:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_CONST
    b.ne .Labst_q2
    orr x20, x20, #1
    bl _kcc_pp_next
    b .Labst_quals
.Labst_q2:
    cmp x0, #TOK_VOLATILE
    b.ne .Labst_q3
    orr x20, x20, #2
    bl _kcc_pp_next
    b .Labst_quals
.Labst_q3:
    cmp x0, #TOK_RESTRICT
    b.ne .Labst_q4
    orr x20, x20, #4
    bl _kcc_pp_next
    b .Labst_quals
.Labst_q4:
    cmp x0, #TOK__ATOMIC
    b.ne .Labst_build_ptr
    orr x20, x20, #8
    bl _kcc_pp_next
    b .Labst_quals

.Labst_build_ptr:
    mov x0, #AST_TYPE_PTR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    str x20, [x0, #AST_F1]
    mov x19, x0
    b .Labst_ptr_loop

.Labst_done:
    // Check for array suffix [N]
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LBRACKET
    b.ne .Labst_check_func

    bl _kcc_pp_next                 // consume '['
    mov x20, #0
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RBRACKET
    b.eq .Labst_arr_close
    bl _kcc_parse_assign_expr
    mov x20, x0
.Labst_arr_close:
    mov x0, #TOK_RBRACKET
    bl _kcc_parse_expect
    mov x0, #AST_TYPE_ARRAY
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    str x20, [x0, #AST_F1]
    mov x19, x0
    b .Labst_done

.Labst_check_func:
    // A trailing '(' after pointers introduces a
   parameter list (a pointer to
    // function whose group was elided, or a bare
   function type); reuse the same
    // parameter-parsing path, with the pointer
   chain as the return-type base.
    cmp  x0, #TOK_LPAREN
    b.ne .Labst_return
    bl _kcc_pp_next                 // consume '('
    mov x20, x19                    // inner is the
   pointer chain / base
    b .Labst_grp_func_body

.Labst_return:
    mov x0, x19

    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_abst_splice ---
// node *kcc_abst_splice(node *chain, node *
   old_base, node *new_type)
// Replace old_base with new_type inside `chain`'s
    AST_F0 type links (as for a
// grouped concrete declarator), implementing
   inside-out declarator semantics
// for an abstract declarator: void (*)(void)
   becomes ptr(func(void)->void).
// Returns the resulting head: `chain` when a
   pointer wraps the base, or
// `new_type` itself when the group had no pointer
    (chain == old_base).
.text
.p2align 2
_kcc_abst_splice:
    cmp x0, x1
    b.eq .Labsp_direct              // chain IS the
   base: the new type is the head
    mov x3, x0                      // walk with x3,
    keep x0 = head
.Labsp_walk:
    ldr x4, [x3, #AST_F0]
    cmp x4, x1
    b.eq .Labsp_here
    cbz x4, .Labsp_done             // base not
   found (defensive): head unchanged
    mov x3, x4
    b .Labsp_walk
.Labsp_here:
    str x2, [x3, #AST_F0]           // relink: this
   node's pointee = new type
.Labsp_done:
    ret                              // x0 still =
   head
.Labsp_direct:
    mov x0, x2                      // head =
   new_type
    ret
```

□

196 · used in 29c · defines _kcc_abst_splice never used, _kcc_parse_abstract_declarator 201 · uses _kcc_ast_alloc 168a, _kcc_parse_assign_expr 208a,

_kcc_parse_expect 168b, _kcc_parse_param_decl 195, _kcc_pp_next 108, _kcc_tok_type 104b

21.6. Struct and Union

Struct and union type specifiers (§6.7.2.1) share the same parsing logic since their syntax is identical apart from the leading keyword. The parser consumes `struct` or `union`, records the appropriate AST tag, then looks for an optional tag name and an optional brace-enclosed member list. Each member is parsed as a declaration specifier followed by a declarator, with support for bitfield widths via the `:` syntax and anonymous struct/union members. The result is wrapped in a `TYPE_STRUCT` or `TYPE_UNION` node that the declaration specifier loop can accumulate.

⟨ *Decl: parse struct* 198 ⟩ ≡

```
// -- kcc_parse_struct_or_union ---
// node *kcc_parse_struct_or_union(void)
// Parses: struct/union [ident] [{ field-list }]
.globl _kcc_parse_struct_or_union
.p2align 2
_kcc_parse_struct_or_union:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    // Layout-attribute accumulators live in STACK
    slots, not callee-saved
    // registers: the member-list parse calls deep
    into the parser and some of
    // those hand-written routines clobber high
   registers without saving them,
    // whereas a stack slot always survives a call
   .  [sp,#64] = packed (bit),
    // [sp,#72] = strongest aligned(N).  They also
    survive a nested struct in the
    // body, whose own parse would otherwise reset
    the shared attribute globals.
    str xzr, [sp, #64]
    str xzr, [sp, #72]

    // Determine struct vs union
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_STRUCT
    b.eq .Lstruct_tag
    mov x19, #AST_UNION_DECL
    b .Lstruct_consume
.Lstruct_tag:
    mov x19, #AST_UNION_DECL
    cmp  x0, #TOK_STRUCT
    mov  x19, #AST_STRUCT_DECL

.Lstruct_consume:
    // Check actual token to set correct tag
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_STRUCT
    b.eq .Lstruct_is_struct
    mov  x19, #AST_UNION_DECL
    b .Lstruct_after_tag
.Lstruct_is_struct:
    mov  x19, #AST_STRUCT_DECL

.Lstruct_after_tag:
    bl _kcc_pp_next                // consume '
    struct' or 'union'

    // GNU attributes may sit between the keyword
    and the tag/brace:
    // `struct __attribute__((packed)) Pk { ...
    }`.
    bl _kcc_parse_skip_attributes
    ⟨ Decl: fold layout attrs into x25/x26 200a ⟩

    // Optional tag name
    mov x20, #0                    // x20 = name_ptr
    (default null)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_IDENT
    b.ne .Lstruct_check_brace

    // Intern tag name
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x20, x0

    bl _kcc_pp_next                // consume tag
    name

    // GNU attributes may also sit between the tag
    and the brace:
    // `struct Pk __attribute__((packed)) { ...
    }`.
    bl _kcc_parse_skip_attributes
    ⟨ Decl: fold layout attrs into x25/x26 200a ⟩

.Lstruct_check_brace:
    // Check for '{'
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LBRACE
    b.ne .Lstruct_no_body

    bl _kcc_pp_next                // consume '{'

    // Parse field declarations
    mov x21, #0                    // first field
    mov x22, #0                    // last field
```

□

198 · defined at 198, 199ab · used in 29c · defines _kcc_parse_struct_or_union 188 · uses _kcc_parse_skip_attributes 184b, _kcc_pp_next 108, _kcc_strtab_intern 66b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

Member list parsing. With the tag determined and the opening brace consumed, the parser enters a loop over the struct or union members. Each member consists of declaration specifiers followed by a declarator and an optional bitfield width introduced by a colon. Rather than re-implement declarator parsing, the member loop calls the full `_kcc_parse_declarator` on the base specifiers—the same routine used for ordinary declarations—so a field may carry any declarator the language allows: pointers, arrays, and crucially the grouped *function-pointer* form `int`

`(*f)(int)`. The node it returns already holds the field's full type in `AST_F0` and its name in `AST_F1`, so the loop simply retags it as an `AST_FIELD_DECL` and records the bitfield width, if any, in `AST_F2`. Parsing each comma-separated declarator against the *original* base type also fixes a latent bug where `int *a, b;` wrongly made `b` a pointer. Anonymous members (a bare specifier followed immediately by a semicolon) are still supported, allowing anonymous inner structs and unions as described in §6.7.2.1 p13. Members are linked through `AST_F4` into a singly-linked list anchored by the first field node.

⟨ *Decl: parse struct* 199a ⟩ + ≡

```
.Lstruct_field_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RBRACE
    b.eq .Lstruct_close
    cbz  x0, .Lstruct_close       // EOF

    // Parse field: specifiers + declarator
    bl _kcc_parse_decl_specs
    mov x23, x0                   // x23 = field
  type
    // Ignore storage class for fields

    // Check for anonymous struct/union field (no
  declarator)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_SEMI
    b.eq .Lstruct_anon_field

    // Parse the full field declarator (pointers,
  grouped function-pointer
    // declarators, arrays) against the base type,
    then reuse the returned
    // node as the field by retagging it.
.Lstruct_field_decl:
    mov x0, x23                   // base type
  specifiers
    bl _kcc_parse_declarator
    mov x24, x0                   // x24 = field
  node (VAR_DECL / FUNC_DECL shape)
    cbz x24, .Lstruct_field_semi
    mov x0, #AST_FIELD_DECL
    str x0, [x24, #AST_TAG]       // retag as a
  struct/union field
    str xzr, [x24, #AST_F2]       // bitwidth
  default 0
    str xzr, [x24, #AST_F4]       // no sibling yet

    bl _kcc_parse_skip_attributes  // GNU attrs on
    a member: `int x __attribute__((..))`

    // Optional bitfield ': width'
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COLON
    b.ne .Lstruct_field_append
    bl _kcc_pp_next               // consume ':'
    bl _kcc_parse_cond_expr       // bitwidth expr
    str x0, [x24, #AST_F2]

.Lstruct_field_append:
    cbz x21, .Lstruct_field_first
    str x24, [x22, #AST_F4]
    mov x22, x24
    b .Lstruct_field_more

.Lstruct_field_first:
    mov x21, x24
    mov x22, x24

.Lstruct_field_more:
    // Check for ',' (more declarators with same
  type)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.ne .Lstruct_field_semi

    bl _kcc_pp_next               // consume ','
    mov x24, #0                   // reset field
  name
    b .Lstruct_field_decl

.Lstruct_field_semi:
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    b .Lstruct_field_loop

.Lstruct_anon_field:
    // Anonymous field (e.g., anonymous struct/
  union member)
    mov x0, #AST_FIELD_DECL
    bl _kcc_ast_alloc
    str x23, [x0, #AST_F0]        // type
    cbz x21, .Lstruct_anon_first
    str x0, [x22, #AST_F4]
    mov x22, x0
    b .Lstruct_anon_semi
.Lstruct_anon_first:
    mov x21, x0
    mov x22, x0
.Lstruct_anon_semi:
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    b .Lstruct_field_loop
```

□

199a · defined at 198, 199ab · used in 29c · uses _kcc_ast_alloc 168a, _kcc_parse_cond_expr 208b, _kcc_parse_declarator 190, _kcc_parse_decl_specs 186b, _kcc_parse_expect 168b, _kcc_parse_skip_attributes 184b, _kcc_pp_next 108, _kcc_tok_type 104b

Struct/union node construction. After the closing brace terminates the member list, the parser allocates the aggregate declaration node with tag `AST_STRUCT_DECL` or `AST_UNION_DECL`, storing the tag name in `AST_F0` and the field chain in `AST_F1`. A third GNU-attribute site follows the closing brace—`struct S { ... } __attribute__((packed))`, the most common spelling of all—so we run one more attribute skip here and fold its result into the accumulator before stamping the combined layout word onto the declaration's `AST_F2`: bit 0 is packed, and the remaining bits hold the `aligned(N)` value. A zero word (the `_kcc_ast_alloc` default) means an ordinary aggregate, so nothing downstream changes for structs without these attributes. The declaration is then wrapped in a `TYPE_STRUCT` or `TYPE_UNION` node so it can be consumed by the specifier accumulator. Forward declarations (a tag without a body) follow the same wrapping logic but produce an empty node with no field list.

⟨ *Decl: parse struct* 199b ⟩ + ≡

```
.Lstruct_close:
    mov x0, #TOK_RBRACE
    bl _kcc_parse_expect

    // Third attribute site: after the closing
   brace, before the declarator.
    bl _kcc_parse_skip_attributes
    ⟨ Decl: fold layout attrs into x25/x26 200a ⟩

    // Build struct/union node
    mov x0, x19                     // tag (
   AST_STRUCT_DECL or AST_UNION_DECL)
    bl _kcc_ast_alloc
    str x20, [x0, #AST_F0]       // name_ptr
    str x21, [x0, #AST_F1]       // fields
    // Stamp the layout word: (aligned(N) << 1) |
   packed.  It goes in AST_F3, not
    // AST_F2 --- the latter is `_kcc_type_struct
   `'s per-definition descriptor
    // cache, and a non-null value there is taken
   for a cached type descriptor.
    ldr x1, [sp, #64]            // packed bit
    ldr x2, [sp, #72]            // aligned(N)
    orr x1, x1, x2, lsl #1
    str x1, [x0, #AST_F3]

    // Wrap in TYPE_STRUCT or TYPE_UNION
    mov x23, x0                     // save decl node
    cmp x19, #AST_STRUCT_DECL
    b.eq .Lstruct_wrap_struct
    mov x0, #AST_TYPE_UNION
    b .Lstruct_wrap
.Lstruct_wrap_struct:
    mov x0, #AST_TYPE_STRUCT
.Lstruct_wrap:
    bl _kcc_ast_alloc
    str x23, [x0, #AST_F0]       // struct/union
   decl

    b .Lstruct_done

.Lstruct_no_body:
    // Forward declaration or reference: struct
   tag (no body)
    mov x0, x19
    bl _kcc_ast_alloc
    str x20, [x0, #AST_F0]       // name_ptr

    // Wrap in type
    mov x23, x0
    cmp x19, #AST_STRUCT_DECL
    b.eq .Lstruct_fwd_struct
    mov x0, #AST_TYPE_UNION
    b .Lstruct_fwd_wrap
.Lstruct_fwd_struct:
    mov x0, #AST_TYPE_STRUCT
.Lstruct_fwd_wrap:
    bl _kcc_ast_alloc
    str x23, [x0, #AST_F0]

.Lstruct_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

199b · defined at 198, 199ab · used in 29c · uses _kcc_ast_alloc 168a, _kcc_parse_expect 168b, _kcc_parse_skip_attributes 184b, _kcc_type_struct 243c

Folding the attribute channel into the accumulator. Each attribute skip resets the two globals and then fills them with what that call saw. This helper OR-s a packed finding into the `[sp,#64]` slot and takes the larger of any `aligned(N)` value into `[sp,#72]`, so the three attribute sites around one aggregate (before the tag, after the tag, after the brace) combine. The accumulator lives in stack slots because the member-list parse both clobbers high registers and, for a nested struct, resets the shared attribute globals mid-parse.

⟨ *Decl: fold layout attrs into x25/x26* 200a ⟩ ≡

```
    adrp x0, _kcc_attr_packed@PAGE
    add  x0, x0, _kcc_attr_packed@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [sp, #64]
    orr  x1, x1, x0              // packed is
   sticky
    str  x1, [sp, #64]
    adrp x0, _kcc_attr_align@PAGE
    add  x0, x0, _kcc_attr_align@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [sp, #72]
    cmp  x0, x1
    csel x1, x0, x1, gt          // keep the
   strongest requested alignment
    str  x1, [sp, #72]
```

□

200a · used in 198, 199b

21.7. Enum

An enumeration type specifier (§6.7.2.2) introduces a set of named integer constants. The parser consumes the `enum` keyword, reads an optional tag name, and if an opening brace follows, iterates over the enumerator list. Each enumerator is an identifier with an optional `= constant-expression` initializer; a trailing comma before the closing brace is permitted. The resulting `AST_ENUM_DECL` node is wrapped in a `TYPE_ENUM` wrapper so it can be returned as a type specifier to the declaration specifier loop.

⟨ *Decl: parse enum* 200b ⟩ ≡

```
// -- kcc_parse_enum ---
// node *kcc_parse_enum(void)
// Parses: enum [ident] [{ enumerator-list }]
.globl _kcc_parse_enum
.p2align 2
_kcc_parse_enum:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    // Consume 'enum'
    bl _kcc_pp_next

    bl _kcc_parse_skip_attributes  // `enum
   __attribute__((..)) E { ... }`

    // Optional tag name
    mov x19, #0                     // x19 = name_ptr
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
```

```
    ldr  x0, [x0]
    cmp  x0, #TOK_IDENT
    b.ne .Lenum_check_brace

    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x19, x0

    bl _kcc_pp_next                // consume tag
   name

.Lenum_check_brace:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LBRACE
    b.ne .Lenum_no_body

    bl _kcc_pp_next                // consume '{'

    mov x20, #0                    // first
   enumerator
    mov x21, #0                    // last
   enumerator

.Lenum_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RBRACE
    b.eq .Lenum_close

    // Parse enumerator: ident [= expr]
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x22, x0                    // x22 =
   enumerator name

    bl _kcc_pp_next                // consume name

    mov x23, #0                    // value expr (
   default null)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ASSIGN
    b.ne .Lenum_build

    bl _kcc_pp_next                // consume '='
    bl _kcc_parse_cond_expr
    mov x23, x0

.Lenum_build:
    mov x0, #AST_ENUMERATOR
    bl _kcc_ast_alloc
    str x22, [x0, #AST_F0]       // name
    str x23, [x0, #AST_F1]       // value expr

    cbz x20, .Lenum_first
    str x0, [x21, #AST_F4]
    mov x21, x0
    b .Lenum_comma

.Lenum_first:
    mov x20, x0
    mov x21, x0

.Lenum_comma:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.ne .Lenum_close

    bl _kcc_pp_next                // consume ','

    // Allow trailing comma before '}'
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RBRACE
    b.ne .Lenum_loop

.Lenum_close:
    mov x0, #TOK_RBRACE
    bl _kcc_parse_expect

    mov x0, #AST_ENUM_DECL
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // name
    str x20, [x0, #AST_F1]       // enumerators

    // Wrap in TYPE_ENUM
    mov x22, x0
    mov x0, #AST_TYPE_ENUM
    bl _kcc_ast_alloc
    str x22, [x0, #AST_F0]

    b .Lenum_done

.Lenum_no_body:
    // Forward reference
    mov x0, #AST_ENUM_DECL
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]

    mov x22, x0
    mov x0, #AST_TYPE_ENUM
    bl _kcc_ast_alloc
    str x22, [x0, #AST_F0]

.Lenum_done:
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

200b · used in 29c · defines _kcc_parse_enum 188 · uses _kcc_ast_alloc 168a, _kcc_parse_cond_expr 208b, _kcc_parse_expect 168b, _kcc_parse_skip_attributes 184b, _kcc_pp_next 108, _kcc_strtab_intern 66b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

21.8. Type Name

A type name (§6.7.7) appears in casts and `sizeof`. It consists of specifier-qualifiers followed by an optional abstract declarator (which can include pointers and array brackets but no name).

⟨ *Decl: parse type name* 201 ⟩ ≡

```
// -- kcc_parse_type_name ---
// node *kcc_parse_type_name(void)
```

```
// Parses a type-name (for casts, sizeof, _Alignof).
// Returns a type node.
.globl _kcc_parse_type_name
.p2align 2
_kcc_parse_type_name:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Parse specifiers (ignore storage class)
    bl _kcc_parse_decl_specs
    mov x19, x0                    // x19 = base type

    // Check for abstract declarator: a pointer, an array suffix, or a '(' that
    // groups a function/array-pointer declarator such as void (*)(void).
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_STAR
    b.eq .Ltname_abstract
    cmp  x0, #TOK_LBRACKET
    b.eq .Ltname_abstract
    cmp  x0, #TOK_LPAREN
    b.eq .Ltname_abstract
    // No abstract declarator
    mov x0, x19
    b .Ltname_done

.Ltname_abstract:
    mov x0, x19
    bl _kcc_parse_abstract_declarator

.Ltname_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

201 · used in 29c · defines _kcc_parse_type_name 184b, 210, 211, 212, 218 · uses _kcc_parse_abstract_declarator 196, _kcc_parse_decl_specs 186b, _kcc_tok_type 104b

21.9. Is Type Token

This predicate checks whether the current token can start a type specifier. It returns 1 for type keywords (char, int, etc.), type qualifiers (const, volatile, etc.), and typedef names.

⟨ *Decl: is type token* 202 ⟩ ≡

```
// -- kcc_parse_is_type_token ---
// int kcc_parse_is_type_token(void)
// Returns 1 if the current token can start a type specifier.
.globl _kcc_parse_is_type_token
.p2align 2
_kcc_parse_is_type_token:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    // Type specifier keywords: void(41), char(13), short(32), int(27),
    // long(28), float(22), double(18), signed(33), unsigned(40),
    // _Bool(47), _Complex(48)
    cmp x0, #TOK_VOID
    b.eq .Lis_type_yes
    cmp x0, #TOK_CHAR
    b.eq .Lis_type_yes
    cmp x0, #TOK_SHORT
    b.eq .Lis_type_yes
    cmp x0, #TOK_INT
    b.eq .Lis_type_yes
    cmp x0, #TOK_LONG
    b.eq .Lis_type_yes
    cmp x0, #TOK_FLOAT
    b.eq .Lis_type_yes
    cmp x0, #TOK_DOUBLE
    b.eq .Lis_type_yes
    cmp x0, #TOK_SIGNED
    b.eq .Lis_type_yes
    cmp x0, #TOK_UNSIGNED
    b.eq .Lis_type_yes
    cmp x0, #TOK__BOOL
    b.eq .Lis_type_yes
    cmp x0, #TOK__COMPLEX
    b.eq .Lis_type_yes
    cmp x0, #TOK_INT128
    b.eq .Lis_type_yes

    // struct, union, enum
    cmp x0, #TOK_STRUCT
    b.eq .Lis_type_yes
    cmp x0, #TOK_UNION
    b.eq .Lis_type_yes
    cmp x0, #TOK_ENUM
    b.eq .Lis_type_yes

    // GNU typeof(...) starts a type just like a builtin
    cmp x0, #TOK_TYPEOF
    b.eq .Lis_type_yes

    // Type qualifiers (can start a type specifier list)
    cmp x0, #TOK_CONST
    b.eq .Lis_type_yes
    cmp x0, #TOK_VOLATILE
    b.eq .Lis_type_yes
    cmp x0, #TOK_RESTRICT
    b.eq .Lis_type_yes
    cmp x0, #TOK__ATOMIC
    b.eq .Lis_type_yes

    // _Alignas can appear in specifier-qualifier list
    cmp x0, #TOK__ALIGNAS
    b.eq .Lis_type_yes

    // Check for typedef name (identifier that is registered as typedef)
    cmp x0, #TOK_IDENT
    b.ne .Lis_type_no

    // __auto_type begins a declaration even inside a block (the statement
    // parser uses this predicate to tell a declaration from an expression).
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    cmp  x1, #11
    b.ne .Lis_type_td
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_str_auto_type@PAGE
    add  x1, x1, _kcc_str_auto_type@PAGEOFF
```

```
    mov  x2, #11
    bl _kcc_memcmp
    cbz x0, .Lis_type_yes       // spelled
   __auto_type

.Lis_type_td:
    // Lookup in typedef table
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_typedef_lookup
    // x0 = 1 if typedef, 0 otherwise
    ldp x29, x30, [sp], #16
    ret

.Lis_type_yes:
    mov x0, #1
    ldp x29, x30, [sp], #16
    ret

.Lis_type_no:
    mov x0, #0
    ldp x29, x30, [sp], #16
    ret
```

□

202 · used in 29c · defines _kcc_parse_is_type_token 184b, 210, 211, 230, 233 · uses _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b, _kcc_typedef_lookup 171

21.10. Initializer

An initializer (§6.7.9) is either an assignment expression or a brace-enclosed initializer list. Within a list, an element may be preceded by a *designator* (§6.7.9p6): `.member` to name a struct or union member, or `[constant]` to name an array element. A designator makes an initializer list sparse and order-independent—`{ [2] = 5, [0] = 1 }` or `{ .y = 2, .x = 1 }`—and it resets the "current object" so that any plain elements that follow continue from the designated position. A designation may also be a *chain* (§6.7.9p6: `designator-list '='`)—`.a.b`, `.arr[1]`, `[0].x`—naming a sub-object several aggregate levels deep. The parser collects the whole run of `.member` and `[index]` designators into a linked list of `AST_DESIGNATOR` nodes before the `=`; the IR builder (§29.11) then walks that chain to the leaf. A single designator is simply a chain of length one.

The GNU array-range form `[lo … hi]` names *every* element from `lo` to `hi` inclusive (the kernel's `atomic64.c` seeds a lock table with `[0 … (NR_LOCKS - 1)] = …`). A range is recorded on one `AST_DESIGNATOR` whose `is_field` field is set to 2 to mark it, with the low index in `F0` and the high index in `F2`; it is always a leaf, so the designator loop ends after it and the IR builder fills the run. A designator index is a constant *expression*, not just a literal—`[NR - 1]`, `[1 << 8]`, or an enum constant `[NLA_U32]`. The parser keeps the index expression node (in `F0`, and the range's high bound in `F2`) rather than folding it here: an enum constant is not in the symbol table until sema, so the index can only be evaluated when the initializer is lowered. The IR builder (§29.11) evaluates it there with the full integer constant-expression evaluator, where literals and enum constants alike resolve.

⟨ *Decl: parse init* 203 ⟩ ≡

```
// -- kcc_parse_initializer ---
// node *kcc_parse_initializer(void)
// Parses an initializer: assignment-expr or {
   init-list }
.globl _kcc_parse_initializer
.p2align 2
_kcc_parse_initializer:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LBRACE
    b.ne .Linit_expr

    // Brace-enclosed initializer list
    bl _kcc_pp_next                 // consume '{'

    mov x0, #AST_INIT_LIST
    bl _kcc_ast_alloc
    mov x19, x0                     // x19 =
   init_list node

    mov x20, #0                     // first init
    mov x21, #0                     // last init

.Linit_list_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RBRACE
    b.eq .Linit_list_close

    // Parse one initializer element, honouring a
   leading designator
    bl _kcc_parse_designated_element
    mov x0, x0                      // new init

    cbz x20, .Linit_list_first
    str x0, [x21, #AST_F4]
    mov x21, x0
    b .Linit_list_comma

.Linit_list_first:
    mov x20, x0
    mov x21, x0

.Linit_list_comma:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.ne .Linit_list_close

    bl _kcc_pp_next                 // consume ','

    // Allow trailing comma
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RBRACE
    b.ne .Linit_list_loop

.Linit_list_close:
    mov x0, #TOK_RBRACE
    bl _kcc_parse_expect
```

```
    str x20, [x19, #AST_F0]      // first init
    mov x0, x19
    b .Linit_done

.Linit_expr:
    // Simple expression initializer
    bl _kcc_parse_assign_expr

.Linit_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_parse_designated_element ---
// node *kcc_parse_designated_element(void)
// One initializer-list element.  If it starts
   with a designation---a
// *sequence* of `.member` and `[index]`
   designators (\stdref{6.7.9p6}:
// `designator-list '='`)---parse the whole chain
   followed by `=` and the
// value initializer.  Each designator is one
   AST_DESIGNATOR node
// (F0 = interned member name or integer index, F1
    = 1 for a field / 0 for
// an index); the nodes are linked head-to-tail
   through AST_F2, and the
// value initializer is stored on the *head* node'
   s AST_F3.  A single
// designator is just a chain of length one, so `.
   x = v` is unchanged.
// Anything not starting with `.`/`[` is a plain
   positional initializer.
.p2align 2
_kcc_parse_designated_element:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_DOT
    b.eq .Lde_chain
    cmp  x0, #TOK_LBRACKET
    b.eq .Lde_chain
    bl _kcc_parse_initializer
    b .Lde_ret

.Lde_chain:
    mov x19, #0                  // chain head
    mov x20, #0                  // chain tail
.Lde_desig_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_DOT
    b.eq .Lde_field
    cmp  x0, #TOK_LBRACKET
    b.eq .Lde_index
    b .Lde_desig_end

.Lde_field:
    bl _kcc_pp_next              // consume '.'
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x21, x0                  // interned
   member name (survives ast_alloc)
    bl _kcc_pp_next              // consume
   member identifier
    mov x0, #AST_DESIGNATOR
    bl _kcc_ast_alloc
    str x21, [x0, #AST_F0]       // name
    mov x2, #1
    str x2, [x0, #AST_F1]        // is_field
    b .Lde_link

.Lde_index:
    bl _kcc_pp_next              // consume '['
    bl _kcc_parse_assign_expr    // index
   expression
    // Keep the index EXPRESSION, not its folded
   value: an index may be an enum
    // constant, which is not in the symbol table
   until sema, so it can only be
    // evaluated when the initializer is lowered.
    mov x21, x0                  // low index
   expr node (survives ast_alloc)
    // GNU array-range designator `[lo ... hi]`?
   A range is always a leaf, so
    // after it the designator loop ends and the
   initializer follows.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ELLIPSIS
    b.ne .Lde_index_single
    bl _kcc_pp_next              // consume '...'
    bl _kcc_parse_assign_expr    // high index
   expression
    mov x22, x0                  // high index
   expr node (survives ast_alloc)
    mov x0, #TOK_RBRACKET
    bl _kcc_parse_expect         // consume ']'
    mov x0, #AST_DESIGNATOR
    bl _kcc_ast_alloc
    str x21, [x0, #AST_F0]       // low index
    mov x2, #2
    str x2, [x0, #AST_F1]        // range marker
   (2 = array range)
    str x22, [x0, #AST_F2]       // high index
    cbnz x19, .Lde_range_tail
    mov x19, x0                  // bare `[lo ...
   hi]`: head = this node
    mov x20, x0
    b .Lde_desig_end
.Lde_range_tail:
    str x0, [x20, #AST_F2]       // (chained
   ranges unsupported; kernel uses bare)
    b .Lde_desig_end

.Lde_index_single:
    mov x0, #TOK_RBRACKET
    bl _kcc_parse_expect         // consume ']'
    mov x0, #AST_DESIGNATOR
    bl _kcc_ast_alloc
    str x21, [x0, #AST_F0]       // index value
    str xzr, [x0, #AST_F1]       // is_field = 0

.Lde_link:
    // Append the new designator node (in x0) to
   the chain via AST_F2.
    cbnz x19, .Lde_link_tail
    mov x19, x0                  // first node →
   head and tail
    mov x20, x0
    b .Lde_desig_loop
.Lde_link_tail:
    str x0, [x20, #AST_F2]       // previous tail
   → this node
```

```
    mov x20, x0
    b .Lde_desig_loop

.Lde_desig_end:
    mov x0, #TOK_ASSIGN
    bl _kcc_parse_expect
    bl _kcc_parse_initializer
    str x0, [x19, #AST_F3]      // value on the
   head node
    mov x0, x19

.Lde_ret:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

203 · used in 29c · defines _kcc_parse_designated_element never used, _kcc_parse_initializer 182, 211 · uses _kcc_ast_alloc 168a, _kcc_parse_assign_expr 208a, _kcc_parse_expect 168b, _kcc_pp_next 108, _kcc_strtab_intern 66b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

21.11. Static Assert

The `_Static_assert` declaration (§6.7.10) provides compile-time assertion checking. It takes the form `_Static_assert(constant-expression, string-literal);` and produces an `AST_STATIC_ASSERT` node carrying the condition expression and the diagnostic message string. The assertion is evaluated during semantic analysis; at parse time we only need to extract the two operands. Interning the message string requires *both* its pointer and its length: the source buffer is not null-terminated at the token's end, so passing the pointer alone (a latent bug, harmless until the kernel's file-scope `static_asserts` first exercised this path) ran the intern hash off the mapping and faulted.

⟨ *Decl: parse static assert* 205a ⟩ ≡

```
// -- kcc_parse_static_assert ---
// node *kcc_parse_static_assert(void)
// Parses: _Static_assert ( expr , string ) ;
.globl _kcc_parse_static_assert
.p2align 2
_kcc_parse_static_assert:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // Consume '_Static_assert'
    bl _kcc_pp_next

    // Expect '('
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect

    // Parse expression
    bl _kcc_parse_cond_expr
    mov x19, x0                 // x19 = expr

    // Expect ','
    mov x0, #TOK_COMMA
    bl _kcc_parse_expect

    // Get message string.  Intern needs BOTH the
   pointer and the length; the
    // source is not null-terminated at the token'
   s end, so omitting the length
    // (a latent bug before file-scope
   _Static_assert was exercised) walked the
    // FNV hash off the mapping and faulted.
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x20, x0                 // x20 = message

    bl _kcc_pp_next             // consume
   string

    // Expect ')'
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Expect ';'
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect

    mov x0, #AST_STATIC_ASSERT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // expr
    str x20, [x0, #AST_F1]      // message

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

205a · used in 29c · defines _kcc_parse_static_assert 184a · uses _kcc_ast_alloc 168a, _kcc_parse_cond_expr 208b, _kcc_parse_expect 168b, _kcc_pp_next 108, _kcc_strtab_intern 66b, _kcc_tok_len 104b, _kcc_tok_start 104b

21.12. Declaration Data

This fragment reserves a place for any static data the declaration parser might need, such as error message strings or lookup tables. Currently no additional data is required beyond what the individual parsing functions use locally.

⟨ *Decl: data* 205b ⟩ ≡

```
// -- Declaration parser data (reserved) ---
// No additional data needed for declaration
   parsing.
```

□

205b · used in 29c

22. Expression Parsing

Expression parsing translates the operator-rich surface syntax of C into AST nodes that capture evaluation order and associativity (§6.5). Rather than writing a separate function for each of C's fifteen precedence levels, we use precedence climbing: a single `kcc_parse_binary_expr` function consults a lookup table to decide when to recurse. This chapter covers the precedence table, the entry point for comma expressions, assignment and conditional operators, the climbing loop for binary operators, unary and cast expressions, postfix chains, and primary expressions including parenthesized sub-expressions and compound literals.

22.1. Precedence Table

The precedence climbing algorithm uses a lookup table mapping each binary operator token to its precedence level. The table is indexed by `(tok_type − 60)`, covering the punctuator range. Each entry is a pair of bytes: precedence level (0 = not a binary op) and a flag indicating right-associativity (unused for binary ops, which are all left-associative).

C17 precedence levels (§6.5):

Level	Operators	Associativity
4	`||`	left
5	`&&`	left
6	`|`	left
7	`^`	left
8	`&`	left
9	`== !=`	left
10	`< > <= >=`	left
11	`<< >>`	left
12	`+ -`	left
13	`* / %`	left

⟨ *Expr: precedence table* 207a ⟩ ≡

```
// -- Operator precedence lookup ---
// Given a token type t in the punctuator range
   (60+), the precedence
// is stored at _kcc_prec_table[t - 60].  Zero
   means "not a binary op".

.text

// -- _kcc_prec_lookup ---
// uint64_t _kcc_prec_lookup(uint64_t tok_type)
//   x0 = token type
// Returns precedence level (0 = not a binary op).
.globl _kcc_prec_lookup
.p2align 2
_kcc_prec_lookup:
    // Check range: must be >= 60
    cmp x0, #60
    b.lo .Lprec_none

    sub x0, x0, #60
    // Check upper bound
    cmp x0, #48
    b.hs .Lprec_none

    adrp x1, _kcc_prec_table@PAGE
    add  x1, x1, _kcc_prec_table@PAGEOFF
    ldrb w0, [x1, x0]
    ret

.Lprec_none:
    mov x0, #0
    ret
```

□

207a · used in 29a · defines `_kcc_prec_lookup` 209a, 228

22.2. Expression Entry Point

`kcc_parse_expr` is the top-level expression parser. It parses an assignment expression, then checks for the comma operator (§6.5.17).

⟨ *Expr: parse expression* 207b ⟩ ≡

```
// -- kcc_parse_expr ---
// node *kcc_parse_expr(void)
// Parses a full expression (handles comma
   operator).
// Returns AST node.
.globl _kcc_parse_expr
.p2align 2
_kcc_parse_expr:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]     // x20 is used for
    the comma rhs below

    bl _kcc_parse_assign_expr
    mov x19, x0                  // x19 = lhs

.Lexpr_comma_loop:
    // Check for comma operator
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.ne .Lexpr_done

    // Consume comma
    bl _kcc_pp_next

    // Parse rhs
    bl _kcc_parse_assign_expr
    // x0 = rhs

    // Build comma node: x19=lhs, x20=rhs
    mov x20, x0                  // x20 = rhs (
   callee-saved)
    mov x0, #AST_COMMA_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // lhs
    str x20, [x0, #AST_F1]      // rhs
    mov x19, x0                  // new lhs =
   comma node
    b .Lexpr_comma_loop

.Lexpr_done:
    mov x0, x19

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

207b · used in 29a · defines _kcc_parse_expr 184b, 208b, 210, 211, 212, 218, 226, 228, 231ab, 232ab, 233, 234b · uses _kcc_ast_alloc 168a, _kcc_parse_assign_expr 208a, _kcc_pp_next 108, _kcc_tok_type 104b

22.3. Assignment Expression

Assignment is right-associative: parse a conditional expression, and if the next token is an assignment operator (=, +=, -=, etc.), recurse on the right-hand side.

⟨ *Expr: parse assign* 208a ⟩ ≡

```
// -- kcc_parse_assign_expr ---
// node *kcc_parse_assign_expr(void)
// Parses an assignment expression (right-
   associative).
.globl _kcc_parse_assign_expr
.p2align 2
_kcc_parse_assign_expr:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    bl _kcc_parse_cond_expr
    mov x19, x0                    // x19 = lhs

    // Check for assignment operators (94-104)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    cmp  x0, #TOK_ASSIGN
    b.lt .Lassign_done
    cmp  x0, #TOK_OR_ASSIGN
    b.gt .Lassign_done

    // It's an assignment operator
    mov x20, x0                    // x20 = operator
    token

    // Consume the operator
    bl _kcc_pp_next

    // Recurse for right-hand side (right-
   associative)
    bl _kcc_parse_assign_expr
    mov x21, x0                    // x21 = rhs

    // Build assign node
    mov x0, #AST_ASSIGN_EXPR
    bl _kcc_ast_alloc
    str x20, [x0, #AST_F0]       // op
    str x19, [x0, #AST_F1]       // lhs
    str x21, [x0, #AST_F2]       // rhs
    mov x19, x0

.Lassign_done:
    mov x0, x19

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

208a · used in 29a · defines _kcc_parse_assign_expr 191b, 192, 196, 203, 207b, 212, 218, 223a, 227, 228 · uses _kcc_ast_alloc 168a, _kcc_parse_cond_expr 208b, _kcc_pp_next 108, _kcc_tok_type 104b

22.4. Conditional Expression

The ternary operator ?: (§6.5.15): parse a binary expression, and if the next token is ?, parse the "then" expression, expect :, and parse the "else" conditional.

⟨ *Expr: parse conditional* 208b ⟩ ≡

```
// -- kcc_parse_cond_expr ---
// node *kcc_parse_cond_expr(void)
// Parses a conditional expression (ternary ? :).
.globl _kcc_parse_cond_expr
.p2align 2
_kcc_parse_cond_expr:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    // Parse binary expression with minimum
   precedence 4 (logical OR)
    mov x0, #4
    bl _kcc_parse_binary_expr
    mov x19, x0                    // x19 =
   condition

    // Check for '?'
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_QUESTION
    b.ne .Lcond_done

    // Consume '?'
    bl _kcc_pp_next

    // GNU `a ?: b': an omitted middle operand (
   next token is ':') is recorded
    // as then == 0, so the IR reuses the
   condition's value.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COLON
    b.eq .Lcond_elvis

    // Parse 'then' expression (full expression,
   not just assignment)
    bl _kcc_parse_expr
    mov x20, x0                    // x20 = then
    b .Lcond_colon
.Lcond_elvis:
    mov x20, #0                    // omitted: reuse
    the condition
.Lcond_colon:

    // Expect ':'
    mov x0, #TOK_COLON
    bl _kcc_parse_expect

    // Parse 'else' (conditional, not assignment
   - right-associative)
    bl _kcc_parse_cond_expr
    mov x21, x0                    // x21 = else

    // Build cond node
    mov x0, #AST_COND_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // condition
```

```
    str x20, [x0, #AST_F1]      // then
    str x21, [x0, #AST_F2]      // else
    mov x19, x0

.Lcond_done:
    mov x0, x19

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

208b · used in 29a · defines _kcc_parse_cond_expr 199a, 200b, 205a, 208a, 228, 234c · uses _kcc_ast_alloc 168a, _kcc_parse_binary_expr 209a, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_pp_next 108, _kcc_tok_type 104b

22.5. Binary Expression (Precedence Climbing)

The core of expression parsing. `kcc_parse_binary_expr` receives a minimum precedence level and loops: parse a unary expression as the left-hand side, then while the next token is a binary operator with precedence $\geq$ `min_prec`, consume it, recursively parse the right-hand side with `(prec + 1)` (for left-associativity), and build an `AST_BINARY_EXPR` node.

⟨ *Expr: parse binary* 209a ⟩ ≡

```
// -- kcc_parse_binary_expr ---
// node *kcc_parse_binary_expr(uint64_t min_prec)
//   x0 = minimum precedence level
// Returns AST node for the binary expression.
.globl _kcc_parse_binary_expr
.p2align 2
_kcc_parse_binary_expr:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0                 // x19 = min_prec

    // Parse left operand (unary/cast)
    bl _kcc_parse_cast_expr
    mov x20, x0                 // x20 = lhs

.Lbinary_loop:
    // Peek at current token
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    mov  x21, x0                // x21 = current
   token type

    // Look up precedence
    bl _kcc_prec_lookup
    mov x22, x0                 // x22 =
   precedence

    // If prec == 0 or prec < min_prec, stop
    cbz  x22, .Lbinary_done
    cmp  x22, x19
    b.lo .Lbinary_done

    // Consume the operator
    bl _kcc_pp_next

    // Parse right operand with prec+1 (left-
   associative)
    add x0, x22, #1
    bl _kcc_parse_binary_expr
    mov x22, x0                 // x22 = rhs (
   callee-saved)

    // Build binary node: x21=op, x20=lhs, x22=rhs
    mov x0, #AST_BINARY_EXPR
    bl _kcc_ast_alloc
    str x21, [x0, #AST_F0]      // op
    str x20, [x0, #AST_F1]      // lhs
    str x22, [x0, #AST_F2]      // rhs

    mov x20, x0                 // new lhs
    b .Lbinary_loop

.Lbinary_done:
    mov x0, x20

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

209a · used in 29a · defines _kcc_parse_binary_expr 208b, 228 · uses _kcc_ast_alloc 168a, _kcc_parse_cast_expr 211, _kcc_pp_next 108, _kcc_prec_lookup 207a, _kcc_tok_type 104b

22.6. Unary Expression

Unary expressions (§6.5.3): prefix operators `++`, `--`, `&`, `*`, `+`, `-`, `~`, `!`, plus `sizeof` and `_Alignof`.

⟨ *Expr: parse unary* 209b ⟩ ≡

```
// -- kcc_parse_unary_expr ---
// node *kcc_parse_unary_expr(void)
// Parses a unary expression.
.globl _kcc_parse_unary_expr
.p2align 2
_kcc_parse_unary_expr:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    // Check for prefix operators
    cmp x0, #TOK_INC            // ++
    b.eq .Lunary_prefix
    cmp x0, #TOK_DEC            // --
    b.eq .Lunary_prefix
    cmp x0, #TOK_AMP            // &
    b.eq .Lunary_prefix
    cmp x0, #TOK_STAR           // *
    b.eq .Lunary_prefix
    cmp x0, #TOK_PLUS           // +
    b.eq .Lunary_prefix
    cmp x0, #TOK_MINUS          // -
    b.eq .Lunary_prefix
    cmp x0, #TOK_TILDE          // ~
    b.eq .Lunary_prefix
    cmp x0, #TOK_BANG           // !
    b.eq .Lunary_prefix

    // sizeof
```

```
    cmp x0, #TOK_SIZEOF
    b.eq .Lunary_sizeof

    // _Alignof
    cmp x0, #TOK__ALIGNOF
    b.eq .Lunary_alignof

    // Otherwise, postfix expression
    bl _kcc_parse_postfix_expr
    b .Lunary_done_direct

.Lunary_prefix:
    mov x19, x0                     // x19 = operator
    token
    bl _kcc_pp_next                  // consume
   operator

    // Recurse for operand
    bl _kcc_parse_cast_expr
    mov x20, x0                     // x20 = operand

    // Build unary node
    mov x0, #AST_UNARY_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // op
    str x20, [x0, #AST_F1]       // operand
    mov x1, #1
    str  x1, [x0, #AST_F2]       // prefix = 1
    b .Lunary_done_direct
```

□

209b · defined at 209b, 210 · used in 29a · defines _kcc_parse_unary_expr 210, 211 · uses _kcc_ast_alloc 168a, _kcc_parse_cast_expr 211, _kcc_parse_postfix_expr 212, _kcc_pp_next 108, _kcc_tok_type 104b

Sizeof and _Alignof. The `sizeof` operator (§6.5.3.4) comes in two forms: `sizeof(type-name)` queries the size of a type directly, while `sizeof expr` measures the type of an expression without evaluating it. The `_Alignof` operator always takes a parenthesized type name. Both produce compile-time integer constants.

⟨ *Expr: parse unary* 210 ⟩ +≡

```
.Lunary_sizeof:
    // Consume 'sizeof'
    bl _kcc_pp_next

    // Check for '(' type-name ')'
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LPAREN
    b.ne .Lunary_sizeof_expr

    // Could be sizeof(type) or sizeof(expr) -
   check next token
    // Save state: we'll peek after '(' to see if
   it's a type
    bl _kcc_pp_next                  // consume '('

    // Check if next token starts a type
    bl _kcc_parse_is_type_token
    cbz x0, .Lunary_sizeof_paren_expr

    // sizeof(type-name)
    bl _kcc_parse_type_name
    mov x19, x0                     // x19 = type
   node
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    mov x0, #AST_SIZEOF_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // operand = type
    mov x1, #1
    str  x1, [x0, #AST_F1]       // is_type = 1
    b .Lunary_done_direct

.Lunary_sizeof_paren_expr:
    // It's sizeof(expr) - parse expression inside
    parens
    bl _kcc_parse_expr
    mov x19, x0
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Wrap in paren node
    mov x20, x19
    mov x0, #AST_PAREN_EXPR
    bl _kcc_ast_alloc
    str x20, [x0, #AST_F0]
    mov x19, x0

    mov x0, #AST_SIZEOF_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // operand = paren
   (expr)
    str xzr, [x0, #AST_F1]       // is_type = 0
    b .Lunary_done_direct

.Lunary_sizeof_expr:
    // sizeof unary-expression
    bl _kcc_parse_unary_expr
    mov x19, x0

    mov x0, #AST_SIZEOF_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    str xzr, [x0, #AST_F1]       // is_type = 0
    b .Lunary_done_direct

.Lunary_alignof:
    // _Alignof ( type-name )
    bl _kcc_pp_next                  // consume '
   _Alignof'
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect

    bl _kcc_parse_type_name
    mov x19, x0

    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    mov x0, #AST_ALIGNOF_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]

.Lunary_done_direct:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

210 · defined at 209b, 210 · used in 29a · uses _kcc_ast_alloc 168a, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_is_type_token 202, _kcc_parse_type_name 201, _kcc_parse_unary_expr 209b, _kcc_pp_next 108, _kcc_tok_type 104b

22.7. Cast Expression

A cast expression (§6.5.4) looks like `(type-name) expr`. The difficulty is distinguishing a cast from a parenthesized expression. We check: if the current token is `(` and the next token starts a type specifier, it's a cast; otherwise fall through to unary.

⟨ *Expr: parse cast* 211 ⟩ ≡

```
// -- kcc_parse_cast_expr ---
// node *kcc_parse_cast_expr(void)
// Parses a cast expression or falls through to
   unary.
.globl _kcc_parse_cast_expr
.p2align 2
_kcc_parse_cast_expr:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    // Check for '(' that might start a cast
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LPAREN
    b.ne .Lcast_unary

    // Save token state to determine if this is a
   cast or paren expr.
    // We need to look ahead: consume '(' and
   check if type follows.
    // But we can't un-consume, so we'll use
   is_type_token after '('.

    // Consume '('
    bl _kcc_pp_next

    // GNU statement expression: `({ statements })
   `.  A brace immediately
    // after the opening parenthesis is
   unambiguous -- no type-name or
    // ordinary expression can begin with `{` in
   this position.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LBRACE
    b.eq .Lcast_stmt_expr

    // Check if next token starts a type
    bl _kcc_parse_is_type_token
    cbz x0, .Lcast_paren_expr

    // It's a cast: (type-name) cast-expr
    bl _kcc_parse_type_name
    mov x19, x0                    // x19 = type

    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Check for compound literal: (type) { ... }
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_LBRACE
    b.eq .Lcast_compound_lit

    // Regular cast
    bl _kcc_parse_cast_expr
    mov x20, x0                    // x20 = inner
   expr

    mov x0, #AST_CAST_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // type
    str x20, [x0, #AST_F1]       // expr
    b .Lcast_done

.Lcast_compound_lit:
    // Compound literal: (type) { init-list }.
   Like a parenthesised primary it
    // is a postfix-expression, so member/
   subscript/call suffixes may chain onto
    // it: `(struct S){1,2}.a`, `(int[]){1,2,3}[i
   ]`.
    bl _kcc_parse_initializer
    mov x20, x0                    // x20 = init
   list

    mov x0, #AST_COMPOUND_LIT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // type
    str x20, [x0, #AST_F1]       // init_list
    bl _kcc_parse_postfix_tail    // apply any .m /
    [i] / (args) suffixes
    b .Lcast_done

.Lcast_paren_expr:
    // Not a cast - it's a parenthesized
   expression
    bl _kcc_parse_expr
    mov x19, x0

    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Wrap in paren node, then continue with
   postfix operations
    mov x0, #AST_PAREN_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]

    // Apply postfix operations (call, subscript,
   member access, etc.)
    bl _kcc_parse_postfix_tail
    b .Lcast_done

.Lcast_stmt_expr:
    // GNU statement expression: the `(` is
   consumed and a compound statement
    // follows.  `_kcc_parse_compound` consumes
   the whole `{ ... }` block; the
    // value of the construct is that of its final
    expression-statement,
    // determined later in semantic analysis.
   Suffixes may chain onto it, so
    // it too runs through the postfix tail.
    bl _kcc_parse_compound        // consumes `{
   ... }`
    mov x19, x0

    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    mov x0, #AST_STMT_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]        // F0 = compound
    statement
    bl _kcc_parse_postfix_tail
    b .Lcast_done

.Lcast_unary:
    bl _kcc_parse_unary_expr

.Lcast_done:
```

```
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

211 · used in 29a · defines _kcc_parse_cast_expr 209ab · uses _kcc_ast_alloc 168a, _kcc_parse_compound 230, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_initializer 203, _kcc_parse_is_type_token 202, _kcc_parse_postfix_tail 212, _kcc_parse_type_name 201, _kcc_parse_unary_expr 209b, _kcc_pp_next 108, _kcc_tok_type 104b

22.8. Postfix Expression

Postfix expressions (§6.5.2): a primary expression followed by zero or more postfix operations: array subscript `[i]`, function call `(args)`, member access `.field` or `->field`, and postfix `++`/`--`. The call form has one special case: `va_arg(ap, T)` (and its GNU spelling `__builtin_va_arg`, which the kernel's `<stdarg.h>` expands to) takes a *type name* as its second operand, so it is diverted to a dedicated parse that reads a type instead of an argument expression and builds an `AST_VA_ARG` node. `__builtin_types_compatible_p(T1, T2)` is handled the same way, but with *two* type-name operands; it builds an `AST_TYPES_COMPAT` node that later folds to the integer constant 1 (the types are compatible per §6.2.7) or 0. The Linux kernel leans on it through `__same_type`, above all in the file-scope `static_assert` sanity checks of `<linux/bitops.h>`—a single construct that, in a sweep, blocked roughly three-quarters of a real kernel `.c` sample. `__builtin_offsetof(T, designator)` (the kernel's `offsetof`) is handled by a small helper, `_kcc_parse_builtin_offsetof`, that *synthesises* the classic form `(size_t)&((T*)0)->designator`: a `(T*)0` cast, an arrow member for the first designator component and dot/subscript for the rest, an address-of, and a cast to `unsigned long` so the result is an integer constant. The existing constant folder then reduces it to the member's byte offset, so no new evaluation path is needed.

⟨ *Expr: parse postfix* 212 ⟩ ≡

```
// -- kcc_parse_postfix_expr ---
// node *kcc_parse_postfix_expr(void)
// Parses a primary expression followed by postfix
    operations.
.globl _kcc_parse_postfix_expr
.p2align 2
_kcc_parse_postfix_expr:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Parse primary
    bl _kcc_parse_primary_expr
    // x0 = primary node

    // Apply postfix tail
    bl _kcc_parse_postfix_tail

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_parse_postfix_tail ---
// node *kcc_parse_postfix_tail(node *expr)
//   x0 = expression to apply postfix operations
   to
// Returns the final expression node.
.globl _kcc_parse_postfix_tail
.p2align 2
_kcc_parse_postfix_tail:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                  // x19 = current
   expr

.Lpostfix_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    cmp x0, #TOK_LBRACKET
    b.eq .Lpostfix_subscript

    cmp x0, #TOK_LPAREN
    b.eq .Lpostfix_call

    cmp x0, #TOK_DOT
    b.eq .Lpostfix_member_dot

    cmp x0, #TOK_ARROW
    b.eq .Lpostfix_member_arrow

    cmp x0, #TOK_INC
    b.eq .Lpostfix_inc

    cmp x0, #TOK_DEC
    b.eq .Lpostfix_dec

    // No more postfix ops
    mov x0, x19
    b .Lpostfix_done

.Lpostfix_subscript:
    // Consume '['
    bl _kcc_pp_next
    // Parse index expression
    bl _kcc_parse_expr
    mov x20, x0                  // x20 = index
    // Expect ']'
    mov x0, #TOK_RBRACKET
    bl _kcc_parse_expect
    // Build subscript node
    mov x0, #AST_SUBSCRIPT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // array
    str x20, [x0, #AST_F1]       // index
    mov x19, x0
    b .Lpostfix_loop

.Lpostfix_call:
    // Special form va_arg(ap, type-name): the
   second operand is a type, not
    // an expression, so it cannot go through the
   normal argument list.
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_IDENT_EXPR
    b.ne .Lpostfix_call_normal
    ldr x0, [x19, #AST_F0]       // interned callee
    name
    adrp x1, _kcc_str_va_arg@PAGE
    add  x1, x1, _kcc_str_va_arg@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lpostfix_va_arg
```

```
    // Its GNU spelling __builtin_va_arg (kernel <
    stdarg.h> maps va_arg to it).
    ldr x0, [x19, #AST_F0]      // reload (strcmp
    clobbered x0; x19 is preserved)
    adrp x1, _kcc_str_builtin_va_arg@PAGE
    add  x1, x1, _kcc_str_builtin_va_arg@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lpostfix_va_arg
    // GNU __builtin_types_compatible_p(type-name,
    type-name): both operands are
    // type names; it folds to the integer
    constant 1/0 (used pervasively by the
    // kernel's __same_type sanity static_asserts)
    .
    ldr x0, [x19, #AST_F0]
    adrp x1, _kcc_str_types_compat@PAGE
    add  x1, x1, _kcc_str_types_compat@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lpostfix_types_compat
    // GNU __builtin_offsetof(type-name, member-
    designator): the kernel's
    // offsetof.  Synthesised into `&((T*)0)->
    designator` by a dedicated helper.
    ldr x0, [x19, #AST_F0]
    adrp x1, _kcc_str_builtin_offsetof@PAGE
    add  x1, x1, _kcc_str_builtin_offsetof@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lpostfix_offsetof

.Lpostfix_call_normal:
    // Consume '('
    bl _kcc_pp_next
    // Parse argument list
    bl _kcc_parse_arg_list
    mov x20, x0                 // x20 = first
    arg (or null)
    // Expect ')'
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    // Build call node
    mov x0, #AST_CALL_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // callee
    str x20, [x0, #AST_F1]      // first arg
    mov x19, x0
    b .Lpostfix_loop

.Lpostfix_va_arg:
    bl _kcc_pp_next             // consume '('
    bl _kcc_parse_assign_expr   // the va_list
    expression
    mov x20, x0
    mov x0, #TOK_COMMA
    bl _kcc_parse_expect
    bl _kcc_parse_type_name     // the target
    type
    mov x21, x0
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    mov x0, #AST_VA_ARG
    bl _kcc_ast_alloc
    str x20, [x0, #AST_F0]      // va_list expr
    str x21, [x0, #AST_F1]      // type node
    mov x19, x0
    b .Lpostfix_loop

.Lpostfix_offsetof:
    bl _kcc_parse_builtin_offsetof
    mov x19, x0
    b .Lpostfix_loop

.Lpostfix_types_compat:
    bl _kcc_pp_next             // consume '('
    bl _kcc_parse_type_name     // first type
    mov x20, x0
    mov x0, #TOK_COMMA
    bl _kcc_parse_expect
    bl _kcc_parse_type_name     // second type
    mov x21, x0
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    mov x0, #AST_TYPES_COMPAT
    bl _kcc_ast_alloc
    str x20, [x0, #AST_F0]      // type 1
    str x21, [x0, #AST_F1]      // type 2
    mov x19, x0
    b .Lpostfix_loop

// -- kcc_parse_builtin_offsetof ---
// node *kcc_parse_builtin_offsetof(void)
// On entry the current token is the '(' of
   __builtin_offsetof(T, desig).
// Synthesises the AST for `&((T*)0)->desig` (
   arrow for the first designator
// component, dot/subscript for the rest), which
   the existing constant folder
// reduces to the member's byte offset.
.p2align 2
.globl _kcc_parse_builtin_offsetof
_kcc_parse_builtin_offsetof:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    bl _kcc_pp_next             // consume '('
    bl _kcc_parse_type_name     // T
    mov x20, x0
    mov x0, #AST_TYPE_PTR       // ptrT = (T *)
    bl _kcc_ast_alloc
    str x20, [x0, #AST_F0]
    str xzr, [x0, #AST_F1]
    mov x20, x0
    mov x0, #AST_INT_LIT        // zero literal
    bl _kcc_ast_alloc
    str xzr, [x0, #AST_F0]
    mov x21, x0
    mov x0, #AST_CAST_EXPR      // (T *)0
    bl _kcc_ast_alloc
    str x20, [x0, #AST_F0]
    str x21, [x0, #AST_F1]
    mov x19, x0                 // x19 = current
    expr
    mov x0, #TOK_COMMA
    bl _kcc_parse_expect
    mov x22, #1                 // first
    designator uses '->'
.Lofs_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_IDENT
    b.eq .Lofs_member
    cmp x0, #TOK_DOT
    b.eq .Lofs_dot
    cmp x0, #TOK_LBRACKET
    b.eq .Lofs_subscript
    b .Lofs_done
.Lofs_dot:
    bl _kcc_pp_next             // consume '.';
    the next member uses '.'
    mov x22, #0
    b .Lofs_loop
.Lofs_member:
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
```

```
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x23, x0
    bl _kcc_pp_next
    mov x0, #AST_MEMBER
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    str x23, [x0, #AST_F1]
    str x22, [x0, #AST_F2]          // arrow(1)
   first, dot(0) after
    mov x19, x0
    mov x22, #0
    b .Lofs_loop
.Lofs_subscript:
    bl _kcc_pp_next                 // consume '['
    bl _kcc_parse_assign_expr
    mov x23, x0
    mov x0, #TOK_RBRACKET
    bl _kcc_parse_expect
    mov x0, #AST_SUBSCRIPT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    str x23, [x0, #AST_F1]
    mov x19, x0
    mov x22, #0
    b .Lofs_loop
.Lofs_done:
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    mov x0, #AST_UNARY_EXPR         // &(...)
    bl _kcc_ast_alloc
    mov x1, #TOK_AMP
    str x1, [x0, #AST_F0]
    str x19, [x0, #AST_F1]
    mov x1, #1
    str x1, [x0, #AST_F2]           // prefix
    mov x19, x0                     // x19 = &((T*)
   0)->desig
    // Wrap in a cast to size_t (unsigned long) so
    the offset is an INTEGER
    // constant, not a pointer---exactly the
   classic `(size_t)&...` offsetof.
    mov x0, #AST_TYPE_BUILTIN
    bl _kcc_ast_alloc
    mov x1, #528                    // SPEC_UNSIGNED
   (1<<9) | SPEC_LONG(1<<4)
    str x1, [x0, #AST_F0]
    mov x20, x0
    mov x0, #AST_CAST_EXPR
    bl _kcc_ast_alloc
    str x20, [x0, #AST_F0]          // type =
   unsigned long
    str x19, [x0, #AST_F1]          // operand = the
    address
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

212 · defined at 212, 214 · used in 29a · defines _kcc_parse_builtin_offsetof never used, _kcc_parse_postfix_expr 209b, _kcc_parse_postfix_tail 211, 226 · uses _kcc_ast_alloc 168a, _kcc_parse_arg_list 223a, _kcc_parse_assign_expr 208a, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_primary_expr 215, _kcc_parse_type_name 201, _kcc_pp_next 108, _kcc_strtab_intern 66b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

Member access and increment/decrement. The remaining postfix operators handle structure and union member access via the dot (.) and arrow (->) operators, and the postfix increment/decrement operators (++/--). The dot and arrow cases share a common path that interns the field name and builds an AST_MEMBER node.

⟨ *Expr: parse postfix* 214 ⟩ +≡

```
.Lpostfix_member_dot:
    mov x21, #0                     // x21 = 0 means
    dot
    bl _kcc_pp_next                 // consume '.'
    b .Lpostfix_member_common

.Lpostfix_member_arrow:
    mov x21, #1                     // x21 = 1 means
    arrow
    bl _kcc_pp_next                 // consume '->'

.Lpostfix_member_common:
    // Current token should be identifier (field
   name)
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x20, x0                     // x20 = interned
    name

    // Consume identifier
    bl _kcc_pp_next

    // Build member node
    mov x0, #AST_MEMBER
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]          // object
    str x20, [x0, #AST_F1]          // name_ptr
    str x21, [x0, #AST_F2]          // arrow flag (0=
   dot, 1=arrow)
    mov x19, x0
    b .Lpostfix_loop

.Lpostfix_inc:
    bl _kcc_pp_next                 // consume '++'
    mov x0, #AST_UNARY_EXPR
    bl _kcc_ast_alloc
    mov x1, #TOK_INC
    str  x1, [x0, #AST_F0]          // op = ++
    str x19, [x0, #AST_F1]          // operand
    str xzr, [x0, #AST_F2]          // postfix = 0
    mov x19, x0
    b .Lpostfix_loop

.Lpostfix_dec:
    bl _kcc_pp_next                 // consume '--'
    mov x0, #AST_UNARY_EXPR
    bl _kcc_ast_alloc
    mov x1, #TOK_DEC
    str  x1, [x0, #AST_F0]          // op = --
    str x19, [x0, #AST_F1]          // operand
    str xzr, [x0, #AST_F2]          // postfix = 0
    mov x19, x0
    b .Lpostfix_loop

.Lpostfix_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
```

```
    ret
```

□

214 · defined at 212, 214 · used in 29a · uses _kcc_ast_alloc 168a, _kcc_pp_next 108, _kcc_strtab_intern 66b, _kcc_tok_len 104b, _kcc_tok_start 104b

22.9. Primary Expression

Primary expressions (§6.5.1): identifiers, integer constants, string literals, character constants, and parenthesized expressions.

Because `kcc_ast_alloc` stamps a node with the *current* token's location, the identifier case allocates its node *before* consuming the token. Otherwise the node would record the position of the following token, and a diagnostic caret on, say, an undeclared identifier would point one token too far to the right.

One identifier is special. `__func__` (§6.4.2.2) is a predefined identifier that, inside a function body, behaves as though a `static const char __func__[]` holding the enclosing function's name were declared at the top of the body. The declarator parser publishes that name in `_kcc_cur_func_name` while it parses the body, so when the primary parser sees the identifier spelled exactly `__func__` (matched as one little-endian word after a length check) and a name is published, it emits a *string-literal* node built from that name—reusing the whole string pipeline (its `char[]` type, decay to `char*`, and data emission) rather than special-casing a new node kind. Outside any function the name is null and the token falls through to the ordinary identifier path, where it earns the usual undeclared-identifier diagnostic.

For string literals, the lexer delivers the raw token including the surrounding double quotes. We strip the quotes, then call `kcc_strlit_process` (Chapter 18) to translate C escape sequences: `\n` becomes 0x0A, `\t` becomes 0x09, `\\` becomes a single backslash, and so on. The function returns a freshly allocated buffer with the processed bytes and its new (possibly shorter) length. Character literals receive the same treatment: we strip the single quotes, run the escape processor, and read the first byte of the result. This means `'\n'` correctly evaluates to 10.

⟨ *Expr: parse primary* 215 ⟩ ≡

```
// -- kcc_parse_primary_expr ---
// node *kcc_parse_primary_expr(void)
// Parses a primary expression.
.globl _kcc_parse_primary_expr
.p2align 2
_kcc_parse_primary_expr:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    cmp x0, #TOK_IDENT
    b.eq .Lprimary_ident

    cmp x0, #TOK_NUMBER
    b.eq .Lprimary_number

    cmp x0, #TOK_STRING_LIT
    b.eq .Lprimary_string

    cmp x0, #TOK_CHAR_LIT
    b.eq .Lprimary_char

    cmp x0, #TOK_LPAREN
    b.eq .Lprimary_paren

    cmp x0, #TOK__GENERIC
    b.eq .Lprimary_generic

    // Error: unexpected token
    adrp x0, _kcc_expr_msg_expected@PAGE
    add  x0, x0, _kcc_expr_msg_expected@PAGEOFF
    bl _kcc_parse_error
    mov x0, #0                  // return null
    b .Lprimary_done

.Lprimary_ident:
    // __func__ (\stdref{6.4.2.2}): inside a
  function body it stands for a static
    // string literal of the enclosing function's
  name.  When the identifier is
    // exactly "__func__" and a function name is
  published, emit a string-literal
    // node (reusing the whole string pipeline)
  instead of an identifier.  The
    // eight-byte spelling is matched as one
  little-endian word after a length
    // check.  Outside a function it falls through
    to the ordinary identifier.
    adrp x0, _kcc_tok_len@PAGE
    add  x0, x0, _kcc_tok_len@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #8
    b.ne .Lprimary_ident_real
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [x0]               // the 8
  identifier bytes
    movz x2, #0x5f5f
    movk x2, #0x7566, lsl #16
    movk x2, #0x636e, lsl #32
    movk x2, #0x5f5f, lsl #48   // "__func__" as
    a little-endian word
    cmp  x1, x2
    b.ne .Lprimary_ident_real
    adrp x0, _kcc_cur_func_name@PAGE
    add  x0, x0, _kcc_cur_func_name@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lprimary_ident_real
    mov  x19, x0                // function name
    (callee-saved across strlen)
    bl   _kcc_strlen
    mov  x1, x0                 // len
    mov  x0, x19                // name ptr
    bl   _kcc_strlit_process    // → x0 =
  processed bytes, x1 = length
    mov  x19, x0
    mov  x20, x1
    bl   _kcc_pp_next           // consume
  __func__
    mov  x0, #AST_STR_LIT
    bl   _kcc_ast_alloc
    str  x19, [x0, #AST_F0]     // str_ptr
    str  x20, [x0, #AST_F1]     // str_len
    b .Lprimary_done

.Lprimary_ident_real:
    // Intern the identifier name
```

```
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x19, x0                     // x19 = interned
    name

    // Build ident node BEFORE consuming, so it
    records the identifier's
    // own column rather than that of the
    following token.
    mov x0, #AST_IDENT_EXPR
    bl _kcc_ast_alloc
    mov x20, x0
    str x19, [x20, #AST_F0]         // name_ptr

    // Consume identifier
    bl _kcc_pp_next
    mov x0, x20
    b .Lprimary_done

.Lprimary_number:
    // A pp-number is either an integer or a
    floating constant.
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_is_float_literal
    cbnz x0, .Lprimary_float

    // Integer literal: parse the value from token
    text.
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_parse_atoi
    mov x19, x0                     // x19 = numeric
    value

    // Scan the suffix for u/U (unsigned) and l/L
    (long rank).  Scanning the
    // whole token text is safe: `u` and `l` are
    never digits in any radix
    // (decimal, 0x hex, 0 octal, 0b binary), so
    only the suffix letters match.
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    add  x1, x0, x1                 // x1 = one past
    last char
    mov  x21, #0                    // x21 = unsigned
    flag (bit 0)
    mov  x3, #0                     // x3 = count of
    l/L
.Lnum_sfx_loop:
    cmp  x0, x1
    b.hs .Lnum_sfx_done
    ldrb w2, [x0], #1
    orr  w2, w2, #0x20              // fold to lower
    case
    cmp  w2, #'u'
    b.ne .Lnum_sfx_notu
    orr  x21, x21, #1
    b    .Lnum_sfx_loop
.Lnum_sfx_notu:
    cmp  w2, #'l'
    b.ne .Lnum_sfx_loop
    add  x3, x3, #1
    b    .Lnum_sfx_loop
.Lnum_sfx_done:
    cmp  x3, #2                      // clamp long
    rank at 2 (long long)
    b.ls .Lnum_sfx_rank
    mov  x3, #2
.Lnum_sfx_rank:
    lsl  x3, x3, #1                 // rank into bits
    1-2
    orr  x21, x21, x3               // x21 = suffix-
    derived flags

    // Value-directed widening (C17 6.4.4.1): the
    constant's type is the first
    // in a suffix-and-base-dependent list that
    can represent its value.  The
    // suffix fixed a floor (unsigned bit x4, long
    rank x5); climb from there
    // by magnitude.  LP64: int=32b, long=long
    long=64b.
    and  x4, x21, #1                // x4 = unsigned
    floor
    lsr  x5, x21, #1
    and  x5, x5, #3                 // x5 = long-rank
    floor
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    ldrb w6, [x0]                   // first char: '0'
    prefix  non-decimal
    cmp  w6, #'0'
    cset w7, ne                     // w7 = isDecimal
    mov  w8, #0xFFFF
    movk w8, #0x7FFF, lsl #16 // w8 = INT_MAX (0
    x7FFFFFFF)
    cmp  x19, x8
    cset w9, ls                     // w9 = fits signed
    int
    mov  w10, #-1                   // w10 = UINT_MAX (0
    xFFFFFFFF)
    cmp  x19, x10
    cset w11, ls                    // w11 = fits
    unsigned int
    cmp  x19, #0
    cset w12, pl                    // w12 = fits signed
    long (top bit clear)
    cbnz x4, .Lwiden_u
    cbnz w7, .Lwiden_dec
    // Non-decimal, no unsigned suffix: int,uint,
    long,ulong,llong,ullong.
    cbnz x5, .Lwiden_rank_hi
    cbnz w9, .Lwiden_r0s
    cbnz w11, .Lwiden_r0u
    cbnz w12, .Lwiden_r1s
    mov  x2, #1
    mov  x3, #1
    b .Lwiden_done
.Lwiden_dec:
    // Decimal, no unsigned suffix: int,long,long
    long (signed).
    cbnz x5, .Lwiden_dec_hi
    cbnz w9, .Lwiden_r0s
    cbnz w12, .Lwiden_r1s
    mov  x2, #1                     // > LLONG_MAX:
    unsigned long (extension)
    mov  x3, #1
    b .Lwiden_done
.Lwiden_dec_hi:
```

```
    cbnz w12, .Lwiden_keepRs
    mov  x2, x5
    mov  x3, #1
    b .Lwiden_done
.Lwiden_rank_hi:
    // Non-decimal, rank  long: long,ulong,llong,
   ullong.
    cbnz w12, .Lwiden_keepRs
    mov  x2, x5
    mov  x3, #1
    b .Lwiden_done
.Lwiden_u:
    // Unsigned suffix: uint,ulong,ullong.
    mov  x3, #1
    cbnz x5, .Lwiden_keepRu
    cbnz w11, .Lwiden_r0u_set
    mov  x2, #1               // unsigned long
    b .Lwiden_done
.Lwiden_keepRu:
    mov  x2, x5
    b .Lwiden_done
.Lwiden_r0u_set:
    mov  x2, #0
    b .Lwiden_done
.Lwiden_r0s:
    mov  x2, #0
    mov  x3, #0
    b .Lwiden_done
.Lwiden_r0u:
    mov  x2, #0
    mov  x3, #1
    b .Lwiden_done
.Lwiden_r1s:
    mov  x2, #1
    mov  x3, #0
    b .Lwiden_done
.Lwiden_keepRs:
    mov  x2, x5
    mov  x3, #0
.Lwiden_done:
    lsl  x2, x2, #1           // rank into bits
   1-2
    orr  x21, x3, x2          // x21 = final type
   flags

    // Consume number
    bl _kcc_pp_next

    mov x0, #AST_INT_LIT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // value
    str x21, [x0, #AST_F2]      // suffix flags:
   bit0=unsigned, bits1-2=long rank
    b .Lprimary_done

.Lprimary_float:
    // Floating constant: evaluate to a double bit
    pattern.  An `f`/`F` suffix
    // makes it a single-precision float: round
   the value to single precision
    // and mark the node so sema types it as float
   .
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    add  x2, x0, x1
    ldrb w2, [x2, #-1]          // last character
    = suffix
    mov  x20, #0                 // is_float flag
    cmp  w2, #'f'
    b.eq .Lpf_isfloat
    cmp  w2, #'F'
    b.ne .Lpf_atof
.Lpf_isfloat:
    mov  x20, #1
.Lpf_atof:
    bl _kcc_parse_atof
    mov x19, x0                  // x19 = double
   bit pattern
    cbz x20, .Lpf_build
    mov x0, x19
    bl _kcc_round_to_float       // round to
   single precision
    mov x19, x0
.Lpf_build:
    bl _kcc_pp_next

    mov x0, #AST_FLOAT_LIT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // double bit
   pattern (single-rounded if float)
    str x20, [x0, #AST_F2]       // is_float suffix
    flag
    b .Lprimary_done

.Lprimary_string:
    // Gather this literal and any adjacent ones (
   phase 6), with escapes
    // processed; the gather consumes every string
    token and leaves the next
    // token current.
    bl _kcc_strlit_gather
    mov x19, x0                  // concatenated
   ptr
    mov x20, x1                  // total len

    mov x0, #AST_STR_LIT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // str_ptr
    str x20, [x0, #AST_F1]       // str_len
    b .Lprimary_done

.Lprimary_char:
    // Parse character value (with escape
   processing)
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    add  x0, x0, #1             // skip opening
   quote
    sub  x1, x1, #2             // exclude quotes
    bl _kcc_strlit_process
    ldrb w19, [x0]               // first processed
    byte

    // Consume char literal
    bl _kcc_pp_next

    mov x0, #AST_CHAR_LIT
    bl _kcc_ast_alloc
    mov x1, x19
    str  x1, [x0, #AST_F0]       // value
    b .Lprimary_done
```

□

215 · defined at 215, 218, 219 · used in 29a · defines _kcc_parse_primary_expr 212 · uses _kcc_ast_alloc 168a, _kcc_is_float_literal 220, _kcc_parse_atof 220, _kcc_parse_atoi 219, _kcc_parse_error 169, _kcc_pp_next 108, _kcc_round_to_float 220, _kcc_strlit_gather 165, _kcc_strlit_process 161, _kcc_strtab_intern 66b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

Parenthesized expressions and generic selection. When the cast-expression parser has already consumed the opening parenthesis and determined that what follows is not a type name, control arrives at `.Lprimary_paren` to handle a regular parenthesized expression. The `_Generic` keyword (§6.5.1.1) introduces a generic selection: `_Generic(controlling-expr, type: expr, ...)`, resolved at compile time based on the type of the controlling expression.

⟨ *Expr: parse primary* 218 ⟩ + ≡

```
.Lprimary_paren:
    // Handled by cast_expr, but if we get here directly,
    // consume '(' and parse expression
    bl _kcc_pp_next                 // consume '('
    bl _kcc_parse_expr
    mov x19, x0

    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    mov x0, #AST_PAREN_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // inner
    b .Lprimary_done

.Lprimary_generic:
    // _Generic ( assignment-expr , generic-assoc-list )
    bl _kcc_pp_next                 // consume '_Generic'
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect

    // Parse controlling expression
    bl _kcc_parse_assign_expr
    mov x19, x0                     // x19 = controlling expr

    mov x0, #TOK_COMMA
    bl _kcc_parse_expect

    // Parse generic association list
    mov x20, #0                     // first assoc
    mov x21, #0                     // last assoc

.Lprimary_generic_loop:
    // Check for 'default' or type-name
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_DEFAULT
    b.eq .Lprimary_generic_default

    // Parse type-name
    bl _kcc_parse_type_name
    mov x9, x0                      // x9 = type (temp, will save after alloc)
    b .Lprimary_generic_assoc

.Lprimary_generic_default:
    bl _kcc_pp_next                 // consume 'default'
    mov x9, #0                      // type = null (default)

.Lprimary_generic_assoc:
    mov x0, #TOK_COLON
    bl _kcc_parse_expect
    // Save type on stack before parsing expr
    str x9, [sp, #40]
    bl _kcc_parse_assign_expr
    // x0 = expr, type at [sp, #40]
    ldr x1, [sp, #40]           // x1 = type

    // Push type and expr (volatile regs won't survive alloc)
    stp x1, x0, [sp, #-16]!
    mov x0, #AST_GENERIC_ASSOC
    bl _kcc_ast_alloc
    ldp x1, x2, [sp], #16       // x1 = type, x2 = expr
    str x1, [x0, #AST_F0]       // type (0=default)
    str x2, [x0, #AST_F1]       // expr

    // Append to list
    cbz x20, .Lprimary_generic_first
    str x0, [x21, #AST_F4]      // last->next = new
    mov x21, x0
    b .Lprimary_generic_next

.Lprimary_generic_first:
    mov x20, x0
    mov x21, x0

.Lprimary_generic_next:
    // Check for more associations
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.ne .Lprimary_generic_end
    bl _kcc_pp_next                 // consume ','
    b .Lprimary_generic_loop

.Lprimary_generic_end:
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    mov x0, #AST_GENERIC
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // controlling_expr
    str x20, [x0, #AST_F1]      // first_assoc

.Lprimary_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

218 · defined at 215, 218, 219 · used in 29a · uses _kcc_ast_alloc 168a, _kcc_parse_assign_expr 208a, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_type_name 201, _kcc_pp_next 108, _kcc_tok_type 104b

Integer literal conversion. The helper `kcc_parse_atoi` converts integer literal token text into a machine integer. It recognises four radix prefixes: bare digits for decimal, `0x`/`0X` for hexadecimal, a leading zero for octal, and `0b`/`0B` for binary. Type suffixes (`u`, `U`, `l`, `L`) are skipped when computing the *value*—they do not change the magnitude—but they are not lost: back in `.Lprimary_number` a second, whole-token scan records them as flags in `AST_F2` (bit 0 for `unsigned`, bits 1–2 for the long rank: 0 `int`, 1 `long`, 2 `long long`). Scanning

the entire token is sound because `u` and `l` are digits in no radix, so only genuine suffix letters ever match. Sema turns those flags into the constant's type, so `3L` is a `long` and `0u` an `unsigned int`—which in turn makes `_Generic` select the matching `long`/`unsigned` association.

The suffix only fixes a *floor*. §6.4.4.1 then gives the constant the first type from a list—dependent on the suffix and on whether the token is decimal or not—that can represent its value. So a bare `2147483648` is too large for `int` and becomes `long`; a hexadecimal `0xFFFFFFFF` that overflows `int` becomes `unsigned int` (the non-decimal lists admit the unsigned rungs, the decimal lists do not). We compute this in place right after the suffix scan, while both the value and the original token text are still in hand: with the unsigned floor, the long-rank floor, the decimal flag, and three representability tests (fits signed `int`, fits unsigned `int`, fits signed `long`—`long` and `long long` share a width on LP64) we climb the appropriate list and rewrite `AST_F2` to the final type's flags. Sema and the IR builder are unchanged; they simply read the upgraded flags.

⟨ *Expr: parse primary* 219 ⟩ +≡

```
// -- kcc_parse_atoi ---
// uint64_t kcc_parse_atoi(const char *str,
   uint64_t len)
//   x0 = pointer to digit string
//   x1 = length
// Returns parsed unsigned integer value.
// Handles decimal, hex (0x), octal (0), binary (0
   b).
.globl _kcc_parse_atoi
.p2align 2
_kcc_parse_atoi:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    cbz x1, .Latoi_zero

    // Check for 0x, 0b, 0 prefix
    ldrb w2, [x0]
    cmp w2, #'0'
    b.ne .Latoi_decimal

    cmp x1, #1
    b.eq .Latoi_zero                // just "0"

    ldrb w2, [x0, #1]
    cmp w2, #'x'
    b.eq .Latoi_hex
    cmp w2, #'X'
    b.eq .Latoi_hex
    cmp w2, #'b'
    b.eq .Latoi_binary
    cmp w2, #'B'
    b.eq .Latoi_binary

    // Octal
    b .Latoi_octal

.Latoi_decimal:
    mov x2, #0                      // accumulator
    mov x3, #10
.Latoi_dec_loop:
    cbz x1, .Latoi_done
    ldrb w4, [x0], #1
    // Skip type suffixes (u, U, l, L)
    cmp w4, #'u'
    b.eq .Latoi_done
    cmp w4, #'U'
    b.eq .Latoi_done
    cmp w4, #'l'
    b.eq .Latoi_done
    cmp w4, #'L'
    b.eq .Latoi_done
    sub w4, w4, #'0'
    mul x2, x2, x3
    add x2, x2, x4
    sub x1, x1, #1
    b .Latoi_dec_loop

.Latoi_hex:
    add x0, x0, #2                  // skip "0x"
    sub x1, x1, #2
    mov x2, #0
.Latoi_hex_loop:
    cbz x1, .Latoi_done
    ldrb w4, [x0], #1
    cmp w4, #'u'
    b.eq .Latoi_done
    cmp w4, #'U'
    b.eq .Latoi_done
    cmp w4, #'l'
    b.eq .Latoi_done
    cmp w4, #'L'
    b.eq .Latoi_done
    // Convert hex digit
    cmp w4, #'9'
    b.le .Latoi_hex_digit
    cmp w4, #'a'
    b.ge .Latoi_hex_lower
    // A-F
    sub w4, w4, #'A'
    add w4, w4, #10
    b .Latoi_hex_acc
.Latoi_hex_lower:
    sub w4, w4, #'a'
    add w4, w4, #10
    b .Latoi_hex_acc
.Latoi_hex_digit:
    sub w4, w4, #'0'
.Latoi_hex_acc:
    lsl x2, x2, #4
    add x2, x2, x4
    sub x1, x1, #1
    b .Latoi_hex_loop

.Latoi_octal:
    mov x2, #0
.Latoi_oct_loop:
    cbz x1, .Latoi_done
    ldrb w4, [x0], #1
    cmp w4, #'u'
    b.eq .Latoi_done
    cmp w4, #'U'
    b.eq .Latoi_done
    cmp w4, #'l'
    b.eq .Latoi_done
    cmp w4, #'L'
    b.eq .Latoi_done
    sub w4, w4, #'0'
    lsl x2, x2, #3
    add x2, x2, x4
    sub x1, x1, #1
    b .Latoi_oct_loop

.Latoi_binary:
    add x0, x0, #2                  // skip "0b"
    sub x1, x1, #2
    mov x2, #0
.Latoi_bin_loop:
    cbz x1, .Latoi_done
    ldrb w4, [x0], #1
    cmp w4, #'u'
    b.eq .Latoi_done
    cmp w4, #'U'
    b.eq .Latoi_done
```

```
    cmp w4, #'l'
    b.eq .Latoi_done
    cmp w4, #'L'
    b.eq .Latoi_done
    sub w4, w4, #'0'
    lsl x2, x2, #1
    add x2, x2, x4
    sub x1, x1, #1
    b .Latoi_bin_loop

.Latoi_zero:
    mov x2, #0

.Latoi_done:
    mov x0, x2

    ldp x29, x30, [sp], #16
    ret
```

□

219 · defined at 215, 218, 219 · used in 29a · defines _kcc_parse_atoi 215

22.10. Floating Constants

A preprocessing number is a floating constant—rather than an integer—when its spelling contains a radix point, an exponent, or a floating suffix (f/F). The classification has two regimes. A *decimal* number is a float if it contains ., e/E, or f/F. A *hexadecimal* number (0x…) needs its own rule: there f/F are ordinary hex *digits* and e is too, so only a radix point or a binary exponent p/P distinguishes a hex float (0x1.fp1) from a hex integer (0xff). kcc_is_float_literal branches on the 0x prefix and scans accordingly.

kcc_parse_atof then evaluates the constant to an IEEE-754 double *at compile time* on the host's own floating-point unit. For a decimal constant it accumulates the significant digits into a single integer mantissa M, counts the fractional digits F, reads any e-exponent, and reconstructs the value as $M \times 10^{e-F}$ with one final multiply or divide. Only the first eighteen significant digits are folded into M: a double carries about seventeen, and a 64-bit M overflows past nineteen, so an ultra-long literal—Lua's 3.14159265358979323846264338327950288 4, thirty-four digits—would wrap M around and yield garbage (an observed 8.6×10^{-18} instead of π) if every digit were multiplied in. Beyond the cap an *integer* digit is not accumulated but instead bumps the power of ten (a drop counter added back into the exponent), and a *fractional* digit is discarded without scaling; both are already below the mantissa's precision. Folding the retained digits into a single scaling step keeps the whole computation to one rounding: an exactly representable fraction such as 3.875 lands on its exact double, whereas the naive digit-by-digit $+\,d/10^k$ method rounds at every fraction digit and drifts a ULP low. The power $10^{|e-F|}$ is built by repeated multiplication, which is itself exact for $|e-F| \le 22$; only far larger magnitudes lose a bit, and those do not occur in real literals.

A hexadecimal constant 0xm.fpe takes a parallel path: the mantissa digits accumulate in base sixteen (the fraction as $d/16^k$) and the value is scaled by a *binary* exponent 2^e. Because the base is a power of two, such constants are exact whenever the mantissa fits, which is precisely why the standard provides them. Either way the 64-bit result is returned as its bit pattern (via fmov), ready to be stored verbatim in the literal's AST node and, later, materialised as an IR_CONST.

⟨ *Expr: parse float* 220 ⟩ ≡

```
// -- kcc_is_float_literal ---
// uint64_t kcc_is_float_literal(const char *str,
   uint64_t len)
//   Returns 1 if the pp-number is a floating
   constant, else 0.
.text
.globl _kcc_is_float_literal
.p2align 2
_kcc_is_float_literal:
    cbz x1, .Lisf_no
    // Hexadecimal integer? 0x / 0X → never a (
   decimal) float here.
    cmp x1, #2
    b.lo .Lisf_scan
    ldrb w2, [x0]
    cmp w2, #'0'
    b.ne .Lisf_scan
    ldrb w2, [x0, #1]
    orr w2, w2, #0x20              // tolower
    cmp w2, #'x'
    b.eq .Lisf_hex                 // 0…x → hex;
   float only if it has '.'/'p'/'P'

.Lisf_scan:
    mov x3, #0                     // index
.Lisf_loop:
    cmp x3, x1
    b.hs .Lisf_no
    ldrb w2, [x0, x3]
    cmp w2, #'.'
    b.eq .Lisf_yes
    cmp w2, #'e'
    b.eq .Lisf_yes
    cmp w2, #'E'
    b.eq .Lisf_yes
    cmp w2, #'f'
    b.eq .Lisf_yes
    cmp w2, #'F'
    b.eq .Lisf_yes
    add x3, x3, #1
    b .Lisf_loop
.Lisf_hex:
    // In a hex literal `f`/`F` are digits, not a
   float suffix; only a
    // radix point or a binary exponent `p`/`P`
   marks a hex float.
    mov x3, #2                     // skip "0x"
.Lisf_hex_loop:
    cmp x3, x1
    b.hs .Lisf_no
    ldrb w2, [x0, x3]
    cmp w2, #'.'
    b.eq .Lisf_yes
    orr w2, w2, #0x20
    cmp w2, #'p'
    b.eq .Lisf_yes
    add x3, x3, #1
    b .Lisf_hex_loop
.Lisf_yes:
    mov x0, #1
    ret
.Lisf_no:
    mov x0, #0
    ret

// -- kcc_parse_atof ---
// uint64_t kcc_parse_atof(const char *str,
```

```
   uint64_t len)
//   x0 = digit string, x1 = length
// Returns the IEEE-754 double bit pattern of the
   decimal constant.
// Leaf function: uses only caller-saved x2-x12
   and d0-d5.
.text
.globl _kcc_parse_atof
.p2align 2
_kcc_parse_atof:
    add x3, x0, x1                  // x3 = end
    mov x2, x0                      // x2 = cursor
    // Hexadecimal floating constant `0x<hex>.<hex
   >p<exp>` (\stdref{6.4.4.2}):
    // a hex mantissa scaled by a \emph{binary}
   exponent, e.g. 0x1.fp1 = 3.875.
    cmp x1, #2
    b.lo .Latof_dec
    ldrb w4, [x0]
    cmp w4, #'0'
    b.ne .Latof_dec
    ldrb w4, [x0, #1]
    orr w4, w4, #0x20
    cmp w4, #'x'
    b.eq .Latof_hex
.Latof_dec:
    // Accumulate all significant digits into a
   single integer mantissa
    // \reg{x5}, counting fractional digits in \
   reg{x6}.  Reconstructing the
    // value with one final scale by $10^{\text{
   net}}$ keeps the whole
    // computation to a single rounding step, so
   an exact decimal such as
    // 3.875 lands on its exact double.  Per-digit
    \texttt{fdiv} accumulation
    // (the naive method) rounds at every digit
   and drifts a ULP low.
    mov x5, #0                      // M = integer
   mantissa
    mov x6, #0                      // F = fractional
   -digit count
    mov x8, #0                      // e-exponent
   magnitude (0 if no 'e' part)
    mov x9, #0                      // e-exponent
   sign: 0 = +, 1 = -
    mov x13, #0                     // significant
   digits already in M (cap at 18)
    mov x14, #0                     // integer digits
    dropped past the cap
    mov x12, #10
.Latof_int:
    cmp x2, x3
    b.hs .Latof_scale
    ldrb w7, [x2]
    cmp w7, #'.'
    b.eq .Latof_frac
    cmp w7, #'e'
    b.eq .Latof_exp
    cmp w7, #'E'
    b.eq .Latof_exp
    sub x10, x7, #'0'
    cmp x10, #9
    b.hi .Latof_scale           // suffix or other
    → done
    cmp x13, #18
    b.hs .Latof_int_drop        // mantissa full:
   this digit scales the exponent
    mul x5, x5, x12
    add x5, x5, x10             // M = M*10 +
   digit
    add x13, x13, #1
    add x2, x2, #1
    b .Latof_int

.Latof_int_drop:
    add x14, x14, #1            // one more
   unaccumulated integer digit
    add x2, x2, #1
    b .Latof_int

.Latof_frac:
    add x2, x2, #1               // skip '.'
.Latof_frac_loop:
    cmp x2, x3
    b.hs .Latof_scale
    ldrb w7, [x2]
    cmp w7, #'e'
    b.eq .Latof_exp
    cmp w7, #'E'
    b.eq .Latof_exp
    sub x10, x7, #'0'
    cmp x10, #9
    b.hi .Latof_scale
    cmp x13, #18
    b.hs .Latof_frac_next       // mantissa full:
   drop this fractional digit
    mul x5, x5, x12
    add x5, x5, x10
    add x6, x6, #1              // one more
   fractional digit
    add x13, x13, #1
.Latof_frac_next:
    add x2, x2, #1
    b .Latof_frac_loop

.Latof_exp:
    add x2, x2, #1               // skip e/E
    mov x8, #0                   // exponent
   magnitude
    mov x9, #0                   // sign: 0 = +, 1
    = -
    cmp x2, x3
    b.hs .Latof_scale
    ldrb w7, [x2]
    cmp w7, #'+'
    b.eq .Latof_exp_skip
    cmp w7, #'-'
    b.ne .Latof_exp_digits
    mov x9, #1
.Latof_exp_skip:
    add x2, x2, #1
.Latof_exp_digits:
    cmp x2, x3
    b.hs .Latof_scale
    ldrb w7, [x2]
    sub x10, x7, #'0'
    cmp x10, #9
    b.hi .Latof_scale
    mov x12, #10
    mul x8, x8, x12
    add x8, x8, x10
    add x2, x2, #1
    b .Latof_exp_digits

.Latof_scale:
    // Net power of ten = (signed e-exponent) - (
   fractional-digit count).
    cbz x9, .Latof_scale_net
    neg x8, x8
.Latof_scale_net:
    sub x8, x8, x6              // x8 = net
   decimal exponent (signed)
    add x8, x8, x14             // integer digits
   past the cap raise the exponent
    scvtf d0, x5                // d0 = mantissa
   value
    mov x9, #10
    scvtf d3, x9                // d3 = 10.0
    mov x9, #1
```

```
    scvtf d1, x9                // d1 = 10^0 = 1.0
    cmp x8, #0
    b.ge .Latof_scale_mag
    neg x11, x8                 // |net| for a
   negative exponent
    b .Latof_scale_pow
.Latof_scale_mag:
    mov x11, x8
.Latof_scale_pow:
    cbz x11, .Latof_end
.Latof_scale_pow_loop:
    fmul d1, d1, d3             // build 10^|net|
   (exact for |net| <= 22)
    subs x11, x11, #1
    b.ne .Latof_scale_pow_loop
    cmp x8, #0
    b.lt .Latof_scale_div
    fmul d0, d0, d1             // one correctly-
   rounded scale up
    b .Latof_end
.Latof_scale_div:
    fdiv d0, d0, d1             // one correctly-
   rounded scale down
    b .Latof_end

.Latof_hex:
    add x2, x0, #2               // skip "0x"
    fmov d0, xzr                 // mantissa
   accumulator
    mov x5, #16
    scvtf d5, x5                 // d5 = 16.0
.Lhex_int:
    cmp x2, x3
    b.hs .Lhex_exp0
    ldrb w6, [x2]
    cmp w6, #'.'
    b.eq .Lhex_frac
    cmp w6, #'p'
    b.eq .Lhex_exp
    cmp w6, #'P'
    b.eq .Lhex_exp
    sub x7, x6, #'0'             // decode a hex
   digit
    cmp x7, #9
    b.ls .Lhex_int_add
    orr w7, w6, #0x20
    sub x7, x7, #'a'
    cmp x7, #5
    b.hi .Lhex_exp0             // not a hex digit
    → mantissa complete
    add x7, x7, #10
.Lhex_int_add:
    fmul d0, d0, d5
    scvtf d1, x7
    fadd d0, d0, d1
    add x2, x2, #1
    b .Lhex_int
.Lhex_frac:
    add x2, x2, #1               // skip '.'
    mov x5, #1
    scvtf d4, x5                 // d4 = 1.0 (
   running 16^k divisor)
.Lhex_frac_loop:
    cmp x2, x3
    b.hs .Lhex_exp0
    ldrb w6, [x2]
    cmp w6, #'p'
    b.eq .Lhex_exp
    cmp w6, #'P'
    b.eq .Lhex_exp
    sub x7, x6, #'0'
    cmp x7, #9
    b.ls .Lhex_frac_add
    orr w7, w6, #0x20
    sub x7, x7, #'a'
    cmp x7, #5
    b.hi .Lhex_exp0
    add x7, x7, #10
.Lhex_frac_add:
    fmul d4, d4, d5              // divisor *= 16
    scvtf d1, x7
    fdiv d1, d1, d4             // digit / 16^k
    fadd d0, d0, d1
    add x2, x2, #1
    b .Lhex_frac_loop
.Lhex_exp:
    add x2, x2, #1               // skip 'p'/'P'
    mov x8, #0                   // exponent
   magnitude
    mov x9, #0                   // sign: 0 = +, 1
    = -
    cmp x2, x3
    b.hs .Lhex_exp_apply
    ldrb w6, [x2]
    cmp w6, #'+'
    b.eq .Lhex_exp_sskip
    cmp w6, #'-'
    b.ne .Lhex_exp_digits
    mov x9, #1
.Lhex_exp_sskip:
    add x2, x2, #1
.Lhex_exp_digits:
    cmp x2, x3
    b.hs .Lhex_exp_apply
    ldrb w6, [x2]
    sub x7, x6, #'0'
    cmp x7, #9
    b.hi .Lhex_exp_apply         // suffix (f/l) or
    end
    mov x10, #10
    mul x8, x8, x10
    add x8, x8, x7
    add x2, x2, #1
    b .Lhex_exp_digits
.Lhex_exp0:
    mov x8, #0                   // no binary
   exponent seen → scale by 2^0
    mov x9, #0
.Lhex_exp_apply:
    // Scale the mantissa by 2^exponent (binary),
   multiplying/dividing by 2.
    mov x5, #2
    scvtf d2, x5                 // d2 = 2.0
    cbz x8, .Latof_end
    cbnz x9, .Lhex_exp_negloop
.Lhex_exp_mul:
    fmul d0, d0, d2
    subs x8, x8, #1
    b.ne .Lhex_exp_mul
    b .Latof_end
.Lhex_exp_negloop:
    fdiv d0, d0, d2
    subs x8, x8, #1
    b.ne .Lhex_exp_negloop

.Latof_end:
    fmov x0, d0                  // return the
   double's bit pattern
    ret

// -- kcc_round_to_float ---
// uint64_t kcc_round_to_float(uint64_t
   double_bits)
//   Rounds a double to single precision and
   returns it back in double format,
//   so an `f`-suffixed literal such as 16777217.0
   f holds exactly the value a
//   single-precision float would.  Uses the
   compiler host's own FPU.
.text
.globl _kcc_round_to_float
```

```
.p2align 2
_kcc_round_to_float:
    fmov d0, x0
    fcvt s0, d0                 // double →
   single (rounds to nearest)
    fcvt d0, s0                 // single →
   double (exact widening)
    fmov x0, d0
    ret
```

□

220 · used in 29a · defines _kcc_is_float_literal 215, _kcc_parse_atof 215, _kcc_round_to_float 215

22.11. Argument List

kcc_parse_arg_list parses a comma-separated list of assignment expressions for function call arguments. Returns a linked list of nodes via F4.

⟨ *Expr: parse arg list* 223a ⟩ ≡

```
// -- kcc_parse_arg_list ---
// node *kcc_parse_arg_list(void)
// Parses a comma-separated list of assignment
   expressions.
// Returns first argument node (linked via F4), or
    null if empty.
.globl _kcc_parse_arg_list
.p2align 2
_kcc_parse_arg_list:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    // Check for empty argument list (next token
   is ')')
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RPAREN
    b.eq .Largs_empty

    mov x19, #0                 // x19 = first
   arg
    mov x20, #0                 // x20 = last arg

.Largs_loop:
    // Parse one argument (assignment expression)
    bl _kcc_parse_assign_expr
    mov x21, x0                 // x21 = new arg

    // Append to list
    cbz x19, .Largs_first
    str x21, [x20, #AST_F4]     // last->next =
   new
    mov x20, x21
    b .Largs_comma

.Largs_first:
    mov x19, x21
    mov x20, x21

.Largs_comma:
    // Check for ','
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.ne .Largs_done_list
    bl _kcc_pp_next             // consume ','
    b .Largs_loop

.Largs_done_list:
    mov x0, x19
    b .Largs_done

.Largs_empty:
    mov x0, #0

.Largs_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

223a · used in 29a · defines _kcc_parse_arg_list 212 · uses _kcc_parse_assign_expr 208a, _kcc_pp_next 108, _kcc_tok_type 104b

22.12. Expression Data

This fragment collects the static data used by the expression parser: the diagnostic string emitted when an unexpected token appears where an expression is expected, and the precedence lookup table that drives the binary operator precedence climbing algorithm. The table is laid out as 48 bytes starting at token type 60, one byte per punctuator slot, with nonzero entries for the binary operators at their C17 precedence levels.

⟨ *Expr: data* 223b ⟩ ≡

```
// -- Expression parser data ---
.section __TEXT,__cstring,cstring_literals

_kcc_expr_msg_expected:
    .asciz "expected expression"

_kcc_str_va_arg:
    .asciz "va_arg"
_kcc_str_builtin_va_arg:
    .asciz "__builtin_va_arg"
_kcc_str_types_compat:
    .asciz "__builtin_types_compatible_p"
_kcc_str_builtin_offsetof:
    .asciz "__builtin_offsetof"

// -- Precedence table ---
// Indexed by (tok_type - 60).  48 entries
   covering all punctuators.
// Value = precedence level (0 = not a binary
   operator).
.data
.p2align 2
_kcc_prec_table:
    //  60  (    61  )    62  {    63  }    64  [
      65  ]    66  ;    67  ,
    .byte 0,   0,   0,   0,   0,   0,   0,   0
    //  68  .    69  ->   70  ++   71  --   72  &
      73  *    74  +    75  -
    .byte 0,   0,   0,   0,   8,  13,  12,  12
    //  76  ~    77  !    78  /    79  %    80  <<
      81  >>   82  <    83  >
    .byte 0,   0,  13,  13,  11,  11,  10,  10
    //  84  <=   85  >=   86  ==   87  !=   88  ^
      89  |    90  &&   91  ||
    .byte 10,  10,   9,   9,   7,   6,   5,   4
    //  92  ?    93  :    94  =    95  *=   96  /=
      97  %=   98  +=   99  -=
```

```
.byte 0,   0,   0,   0,   0,   0,   0,   0
// 100 <<=  101 >>=  102 &=   103 ^=   104 |=
 105 #    106 ##   107 ...
```

```
.byte 0,   0,   0,   0,   0,   0,   0,   0
```

□

223b · used in 29a

23. Statement Parsing

Statement parsing converts the token stream into AST nodes representing control flow and sequencing (§6.8). The top-level dispatcher inspects the leading token to choose among compound statements, selection (`if`, `switch`), iteration (`while`, `do`, `for`), jumps (`goto`, `continue`, `break`, `return`), and labeled statements. Each construct is handled by a dedicated parsing function that builds the appropriate AST node and links statement lists through the F4 sibling pointer.

23.1. Statement Dispatch

`kcc_parse_stmt` examines the current token and dispatches to the appropriate statement parser. A bare semicolon produces a null statement; an opening brace starts a compound statement; keywords like `if`, `while`, etc. invoke their specific handlers. An identifier at the start could be either a label (if followed by `:`) or an expression statement.

⟨ *Stmt: parse statement* 225 ⟩ ≡

```
// -- kcc_parse_stmt ---
// node *kcc_parse_stmt(void)
// Parses a single statement.
// Returns AST node for the statement.
.text
.globl _kcc_parse_stmt
.p2align 2
_kcc_parse_stmt:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]      // x20 is used by
   the labeled-statement path

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    // Compound statement
    cmp x0, #TOK_LBRACE
    b.eq .Lstmt_compound

    // Null statement (bare ;)
    cmp x0, #TOK_SEMI
    b.eq .Lstmt_null

    // Selection
    cmp x0, #TOK_IF
    b.eq .Lstmt_if
    cmp x0, #TOK_SWITCH
    b.eq .Lstmt_switch

    // Iteration
    cmp x0, #TOK_WHILE
    b.eq .Lstmt_while
    cmp x0, #TOK_DO
    b.eq .Lstmt_do
    cmp x0, #TOK_FOR
    b.eq .Lstmt_for

    // Jump
    cmp x0, #TOK_GOTO
    b.eq .Lstmt_goto
    cmp x0, #TOK_CONTINUE
    b.eq .Lstmt_continue
    cmp x0, #TOK_BREAK
    b.eq .Lstmt_break
    cmp x0, #TOK_RETURN
    b.eq .Lstmt_return

    // Labeled: case
    cmp x0, #TOK_CASE
    b.eq .Lstmt_case
    cmp x0, #TOK_DEFAULT
    b.eq .Lstmt_default

    // Inline assembly: asm / __asm__ ( "template"
    ) ;
    cmp x0, #TOK_ASM
    b.eq .Lstmt_asm

    // Could be label (ident ':') or expression
   statement
    // Check: if it's an identifier, peek ahead
   for ':'
    cmp x0, #TOK_IDENT
    b.eq .Lstmt_check_label

    // Expression statement
    b .Lstmt_expr

.Lstmt_compound:
    bl _kcc_parse_compound
    b .Lstmt_done

.Lstmt_null:
    bl _kcc_pp_next                  // consume ';'
    mov x0, #AST_NULL_STMT
    bl _kcc_ast_alloc
    b .Lstmt_done

.Lstmt_if:
    bl _kcc_parse_if
    b .Lstmt_done

.Lstmt_switch:
    bl _kcc_parse_switch
    b .Lstmt_done

.Lstmt_while:
    bl _kcc_parse_while
    b .Lstmt_done

.Lstmt_do:
    bl _kcc_parse_do
    b .Lstmt_done

.Lstmt_for:
    bl _kcc_parse_for
    b .Lstmt_done
```

□

225 · defined at 225, 226, 228 · used in 29b · defines _kcc_parse_stmt 226, 230, 231ab, 232ab, 233, 234c · uses _kcc_ast_alloc 168a, _kcc_parse_compound 230, _kcc_parse_do 232b, _kcc_parse_for 233, _kcc_parse_if 231a, _kcc_parse_switch 231b, _kcc_parse_while 232a, _kcc_pp_next 108, _kcc_tok_type 104b

Jump statements, labels, and expression statements. The second half of the statement dispatcher handles jump statements (`goto`, `continue`, `break`, `return`), labeled statements (`case`, `default`, and identifier labels), and the expression-statement fallback. The tricky case is an identifier at statement position: it could be a label (`ident:`) or

the start of an expression statement, requiring one-token lookahead.

⟨ Stmt: parse statement 226 ⟩ + ≡

```
.Lstmt_goto:
    bl _kcc_parse_goto
    b .Lstmt_done

.Lstmt_continue:
    bl _kcc_pp_next                 // consume '
   continue'
    mov x0, #AST_CONTINUE_STMT
    bl _kcc_ast_alloc
    mov x19, x0
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    mov x0, x19
    b .Lstmt_done

.Lstmt_break:
    bl _kcc_pp_next                 // consume '
   break'
    mov x0, #AST_BREAK_STMT
    bl _kcc_ast_alloc
    mov x19, x0
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    mov x0, x19
    b .Lstmt_done

.Lstmt_return:
    bl _kcc_parse_return
    b .Lstmt_done

.Lstmt_case:
    bl _kcc_parse_case
    b .Lstmt_done

.Lstmt_default:
    bl _kcc_parse_default
    b .Lstmt_done

.Lstmt_asm:
    // Inline assembly: `asm [volatile|inline|goto
   ] ( "template" [operands] ) ;`.
    // The template is captured, then `
   _kcc_parse_asm_extended` consumes any
    // GNU operand/clobber lists (the part after
   the first `:`) and the closing
    // paren, building the AST_ASM_STMT.  The back
    end splices the template into
    // the textual `.s` stream.
    bl _kcc_pp_next                 // consume 'asm'
    / '__asm__'
.Lstmt_asm_quals:
    // Skip optional volatile/inline/goto
   qualifiers up to the '('.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_LPAREN
    b.eq .Lstmt_asm_lp
    cbz x0, .Lstmt_asm_lp           // EOF safety
    bl _kcc_pp_next
    b .Lstmt_asm_quals
.Lstmt_asm_lp:
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect
    bl _kcc_strlit_gather           // x0 = template
    ptr, x1 = len
    bl _kcc_parse_asm_extended      // consumes
   operands + ')', returns AST_ASM_STMT
    mov x19, x0
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    mov x0, x19
    b .Lstmt_done

.Lstmt_check_label:
    // Save identifier info for possible label
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    // Intern it in case it's a label
    bl _kcc_strtab_intern
    mov x19, x0                     // x19 = interned
    name

    // We need lookahead: consume ident and check
   for ':'
    bl _kcc_pp_next                 // consume
   identifier

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COLON
    b.eq .Lstmt_label

    // Not a label: it's an expression starting
   with this identifier.
    // Build ident node, then parse rest as
   expression statement.
    mov x0, #AST_IDENT_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]          // name_ptr
    // Continue parsing postfix/binary/assignment
   with this as lhs
    bl _kcc_parse_postfix_tail
    // Now x0 might have postfix ops applied;
   continue with assignment
    bl _kcc_parse_expr_rest
    mov x19, x0

    // Build expression statement
    mov x0, #AST_EXPR_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    mov x19, x0
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    mov x0, x19
    b .Lstmt_done

.Lstmt_label:
    // Label: ident ':'
    bl _kcc_pp_next                 // consume ':'

    // Parse the labeled statement
    bl _kcc_parse_stmt
    mov x20, x0                     // x20 = inner
   stmt (callee-saved)

    mov x0, #AST_LABEL_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]          // name_ptr
    str x20, [x0, #AST_F1]          // stmt
    b .Lstmt_done

.Lstmt_expr:
    // Expression statement: expr ';'
    bl _kcc_parse_expr
    mov x19, x0

    mov x0, #AST_EXPR_STMT
```

```
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]

    mov x19, x0
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    mov x0, x19

.Lstmt_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

226 · defined at 225, 226, 228 · used in 29b · uses _kcc_ast_alloc 168a, _kcc_parse_asm_extended 227, _kcc_parse_case 234c, _kcc_parse_default 234c, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_expr_rest 228, _kcc_parse_goto 234a, _kcc_parse_postfix_tail 212, _kcc_parse_return 234b, _kcc_parse_stmt 225, _kcc_pp_next 108, _kcc_strlit_gather 165, _kcc_strtab_intern 66b, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b

Extended inline assembly. After the template string, GNU inline assembly may carry up to three colon-separated lists: output operands, input operands, and clobbers (a fourth, `goto` labels, is consumed but unsupported). Each operand is an optional `[name]`, a string-literal *constraint*, and a parenthesised C expression. `_kcc_parse_asm_extended` builds the `AST_ASM_STMT` with an operand chain hung off `AST_F2`; each operand is an `AST_ASM_OPERAND` recording its constraint text, its expression, and whether it is an output. Clobbers are consumed but not modelled beyond `"memory"`/`"cc"`; the back end validates the constraints it can honour and rejects the rest, so an unsupported form never miscompiles.

⟨ *Stmt: parse asm* 227 ⟩ ≡

```
// -- kcc_parse_asm_opsec ---
// void kcc_parse_asm_opsec(node *asm_stmt,
   uint64_t is_output)
// Parse one colon-section of operands, appending
   AST_ASM_OPERAND nodes to the
// statement's AST_F2 chain.  Stops at the next
   ':' or ')'.
.globl _kcc_parse_asm_opsec
.p2align 2
_kcc_parse_asm_opsec:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]
    mov x19, x0                     // asm node
    mov x20, x1                     // is_output
.Los_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_COLON
    b.eq .Los_done
    cmp x0, #TOK_RPAREN
    b.eq .Los_done
    cbz x0, .Los_done               // EOF
    // Optional [name] - parsed and ignored (
   positional %N is still supported).
    cmp x0, #TOK_LBRACKET
    b.ne .Los_constraint
    bl _kcc_pp_next
.Los_skipname:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cbz x0, .Los_done
    cmp x0, #TOK_RBRACKET
    b.eq .Los_aftername
    bl _kcc_pp_next
    b .Los_skipname
.Los_aftername:
    bl _kcc_pp_next                 // consume ']'
.Los_constraint:
    bl _kcc_strlit_gather           // x0 =
   constraint ptr, x1 = len
    mov x21, x0
    mov x22, x1
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect
    bl _kcc_parse_assign_expr
    mov x23, x0                     // operand
   expression (callee-saved)
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    mov x0, #AST_ASM_OPERAND
    bl _kcc_ast_alloc
    str x21, [x0, #AST_F0]          // constraint
   ptr
    str x22, [x0, #AST_F1]          // constraint
   len
    str x23, [x0, #AST_F2]          // expression
    str x20, [x0, #AST_F3]          // is_output
    // Append to the statement's operand chain (
   walk AST_F2 to the tail).
    mov x1, x0
    ldr x2, [x19, #AST_F2]
    cbz x2, .Los_first
.Los_walk:
    ldr x3, [x2, #AST_F4]
    cbz x3, .Los_appendtail
    mov x2, x3
    b .Los_walk
.Los_appendtail:
    str x1, [x2, #AST_F4]
    b .Los_more
.Los_first:
    str x1, [x19, #AST_F2]
.Los_more:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_COMMA
    b.ne .Los_done
    bl _kcc_pp_next
    b .Los_loop
.Los_done:
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_parse_asm_clobbers ---
// void kcc_parse_asm_clobbers(void)
// Consume the comma-separated clobber string
   literals up to ':' or ')'.
.globl _kcc_parse_asm_clobbers
.p2align 2
_kcc_parse_asm_clobbers:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
.Lac_loop:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
```

```
    ldr  x0, [x0]
    cmp x0, #TOK_COLON
    b.eq .Lac_done
    cmp x0, #TOK_RPAREN
    b.eq .Lac_done
    cbz x0, .Lac_done
    bl _kcc_strlit_gather          // consume one
   clobber string
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_COMMA
    b.ne .Lac_done
    bl _kcc_pp_next
    b .Lac_loop
.Lac_done:
    ldp x29, x30, [sp], #16
    ret

// -- kcc_parse_asm_extended ---
// node *kcc_parse_asm_extended(const char *tmpl,
   uint64_t len)
// Build the AST_ASM_STMT and consume any operand/
   clobber lists plus ')'.
.globl _kcc_parse_asm_extended
.p2align 2
_kcc_parse_asm_extended:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                    // tmpl
    mov x20, x1                    // len
    mov x0, #AST_ASM_STMT
    bl _kcc_ast_alloc
    mov x21, x0
    str x19, [x21, #AST_F0]
    str x20, [x21, #AST_F1]
    // Extended (has operand lists) iff a ':'
   follows the template.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_COLON
    b.ne .Lae_close
    bl _kcc_pp_next                // consume ':' (
   before outputs)
    mov x0, x21
    mov x1, #1                     // is_output
    bl _kcc_parse_asm_opsec
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_COLON
    b.ne .Lae_close
    bl _kcc_pp_next                // consume ':' (
   before inputs)
    mov x0, x21
    mov x1, #0
    bl _kcc_parse_asm_opsec
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_COLON
    b.ne .Lae_close
    bl _kcc_pp_next                // consume ':' (
   before clobbers)
    bl _kcc_parse_asm_clobbers
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_COLON
    b.ne .Lae_close
    bl _kcc_pp_next                // asm goto
   labels - consumed, unsupported
.Lae_goto:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_RPAREN
    b.eq .Lae_close
    cbz x0, .Lae_close
    bl _kcc_pp_next
    b .Lae_goto
.Lae_close:
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect
    mov x0, x21
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

227 · used in 29b · defines _kcc_parse_asm_clobbers never used, _kcc_parse_asm_extended 226, _kcc_parse_asm_opsec never used · uses _kcc_ast_alloc 168a, _kcc_parse_assign_expr 208a, _kcc_parse_expect 168b, _kcc_pp_next 108, _kcc_strlit_gather 165, _kcc_tok_type 104b

Expression continuation after label lookahead. When the dispatcher consumes an identifier to check for a label colon and discovers no colon follows, it must continue parsing the remainder of the expression. The helper `kcc_parse_expr_rest` takes the already-built identifier node (with postfix operations applied) and finishes parsing any binary, ternary, assignment, or comma operators that follow.

⟨ *Stmt: parse statement* 228 ⟩ +≡

```
// -- kcc_parse_expr_rest ---
// node *kcc_parse_expr_rest(node *lhs)
//   x0 = already-parsed left-hand side
// Continues parsing binary/assignment/ternary
   after an identifier+postfix.
// This is needed when we consumed the ident for
   label lookahead.
.globl _kcc_parse_expr_rest
.p2align 2
_kcc_parse_expr_rest:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                    // x19 = lhs so
   far

    // Check for binary operators
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    // Check for assignment operators (94-104)
    cmp  x0, #TOK_ASSIGN
    b.lt .Lexpr_rest_check_binary
    cmp  x0, #TOK_OR_ASSIGN
    b.gt .Lexpr_rest_check_binary

    // Assignment
    mov x20, x0                    // x20 = op
    bl _kcc_pp_next
```

```
    bl _kcc_parse_assign_expr
    mov x21, x0                     // x21 = rhs
    mov x0, #AST_ASSIGN_EXPR
    bl _kcc_ast_alloc
    str x20, [x0, #AST_F0]
    str x19, [x0, #AST_F1]
    str x21, [x0, #AST_F2]
    // The assignment becomes the new left operand
    and parsing continues into
    // the comma check---an assignment is a valid
   comma operand, so a statement
    // like `a = 1, b = 2;` (which reaches this
   path because the leading `a` was
    // consumed for label lookahead) must not stop
    after the assignment.  The
    // binary and ternary arms already fall
   through here; the assignment arm
    // used to jump straight to done, so `a = 1, b
   ` failed with `expected ';'`.
    mov x19, x0
    b .Lexpr_rest_check_comma

.Lexpr_rest_check_binary:
    // Precedence-climbing loop over binary
   operators.  We treat the running
    // result in x19 as the left operand and, for
   each operator, parse a
    // right operand at the next-higher precedence
   .  Looping back here after
    // building each node---rather than falling
   straight through to the comma
    // check---is what makes an operator *chain*
   (`a - b - c`) and, crucially,
    // a trailing ternary (`a > b ? a : b`) parse
   correctly at statement
    // position: the earlier code built a single
   binary node and skipped both
    // the remaining operators and the `?` check,
   so `a > b ? a : b;` failed
    // with `expected ';'`.  When no binary
   operator follows, x0 is 0 and we
    // fall through to the ternary check.  We
   reload the current token into x0
    // here because \texttt{\_kcc\_prec\_lookup}
   reads its argument from x0, and
    // on the loop-back edge x0 still holds the
   binary node just built --- not
    // the next token.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    bl _kcc_prec_lookup
    cbz x0, .Lexpr_rest_check_ternary

    mov x20, x0                     // x20 = prec of
   current op

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x21, [x0]                  // x21 = op token

    bl _kcc_pp_next                  // consume
   operator

    add x0, x20, #1
    bl _kcc_parse_binary_expr       // parse rhs with
    prec+1
    mov x22, x0                     // x22 = rhs

    mov x0, #AST_BINARY_EXPR
    bl _kcc_ast_alloc
    str x21, [x0, #AST_F0]          // op
    str x19, [x0, #AST_F1]          // lhs
    str x22, [x0, #AST_F2]          // rhs
    mov x19, x0

    // Continue climbing: more binary operators,
   then a ternary, then comma.
    b .Lexpr_rest_check_binary

.Lexpr_rest_check_ternary:
    // Check for '?'
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_QUESTION
    b.ne .Lexpr_rest_check_comma

    bl _kcc_pp_next                  // consume '?'
    bl _kcc_parse_expr
    mov x20, x0                     // then
    mov x0, #TOK_COLON
    bl _kcc_parse_expect
    bl _kcc_parse_cond_expr
    mov x21, x0                     // else

    mov x0, #AST_COND_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    str x20, [x0, #AST_F1]
    str x21, [x0, #AST_F2]
    mov x19, x0

.Lexpr_rest_check_comma:
    // Check for comma operator
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_COMMA
    b.ne .Lexpr_rest_no_comma

    bl _kcc_pp_next
    bl _kcc_parse_assign_expr
    mov x20, x0

    mov x0, #AST_COMMA_EXPR
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    str x20, [x0, #AST_F1]
    mov x19, x0
    b .Lexpr_rest_check_comma

.Lexpr_rest_no_comma:
    mov x0, x19

.Lexpr_rest_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

228 · defined at 225, 226, 228 · used in 29b · defines _kcc_parse_expr_rest 226 · uses _kcc_ast_alloc 168a, _kcc_parse_assign_expr 208a, _kcc_parse_binary_expr 209a, _kcc_parse_cond_expr 208b, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_pp_next 108, _kcc_prec_lookup 207a, _kcc_tok_type 104b

23.2. Compound Statement

A compound statement (§6.8.2) is a brace-enclosed sequence of declarations and statements. It introduces a new scope for typedef names. When a declaration produces multiple nodes linked via F4 (e.g. `int a=3, b=4;`), the appender walks the chain to find its tail so that the

next item is linked after the last declarator rather than overwriting the chain.

A block may open with one or more GNU *local label declarations* (`__label__ a, b;`)—the kernel's `wait_event` and RCU statement-expression macros use them so a nested `goto` can name a block-local target. kcc resolves labels per function, so the loop simply consumes a leading `__label__` run (identifiers separated by commas, terminated by `;`) and moves on; the label names still bind through the ordinary label/`goto` machinery.

⟨ *Stmt: parse compound* 230 ⟩ ≡

```
// -- kcc_parse_compound ---
// node *kcc_parse_compound(void)
// Parses a compound statement { items... }
// Returns AST_COMPOUND node with F0 = first item.
.section __TEXT,__cstring,cstring_literals
_kcc_str_label: .asciz "__label__"
.text
.globl _kcc_parse_compound
.p2align 2
_kcc_parse_compound:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Consume '{'
    mov x0, #TOK_LBRACE
    bl _kcc_parse_expect

    // Push typedef scope
    bl _kcc_typedef_push_scope

    // Allocate compound node
    mov x0, #AST_COMPOUND
    bl _kcc_ast_alloc
    mov x19, x0                     // x19 = compound
    node

    mov x20, #0                     // x20 = first
   item
    mov x21, #0                     // x21 = last
   item

.Lcompound_loop:
    // Check for '}' or EOF
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    cmp x0, #TOK_RBRACE
    b.eq .Lcompound_close
    cbz x0, .Lcompound_close        // EOF

    // GNU local label declaration `__label__ a, b
   ;` at the head of a block
    // (the kernel's wait_event/RCU statement-
   expression macros open with one).
    // kcc resolves labels per function, so just
   consume it; the names still bind
    // through the normal label/goto machinery.
    cmp x0, #TOK_IDENT
    b.ne .Lcompound_notlabel
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    cmp x1, #9
    b.ne .Lcompound_notlabel
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_str_label@PAGE
    add  x1, x1, _kcc_str_label@PAGEOFF
    mov x2, #9
    bl _kcc_memcmp
    cbnz x0, .Lcompound_notlabel
    bl _kcc_pp_next                 // consume '
   __label__'
.Lcompound_label_ids:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_IDENT
    b.ne .Lcompound_label_comma
    bl _kcc_pp_next                 // consume a label
    name
.Lcompound_label_comma:
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_COMMA
    b.ne .Lcompound_label_end
    bl _kcc_pp_next                 // consume ','
    b .Lcompound_label_ids
.Lcompound_label_end:
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect            // consume ';'
    b .Lcompound_loop
.Lcompound_notlabel:

    // Is it a declaration or a statement?
    bl _kcc_parse_is_type_token
    cbnz x0, .Lcompound_decl

    // Check for storage class specifiers that
   start declarations
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_TYPEDEF
    b.eq .Lcompound_decl
    cmp x0, #TOK_EXTERN
    b.eq .Lcompound_decl
    cmp x0, #TOK_STATIC
    b.eq .Lcompound_decl
    cmp x0, #TOK_AUTO
    b.eq .Lcompound_decl
    cmp x0, #TOK_REGISTER
    b.eq .Lcompound_decl
    cmp x0, #TOK__STATIC_ASSERT
    b.eq .Lcompound_decl

    // It's a statement
    bl _kcc_parse_stmt
    b .Lcompound_append

.Lcompound_decl:
    bl _kcc_parse_decl

.Lcompound_append:
    mov x22, x0                     // x22 = new item
    (head)
    cbz x22, .Lcompound_loop        // skip null (
   error recovery)

    // Walk to end of F4 chain (multi-declarator:
   int a,b;)
    mov x0, x22
.Lcompound_tail:
    ldr x1, [x0, #AST_F4]
    cbz x1, .Lcompound_tail_done
    mov x0, x1
    b .Lcompound_tail
.Lcompound_tail_done:
    // x22 = head, x0 = tail
```

```
    cbz x20, .Lcompound_first

    // Append: last->F4 = head
    str x22, [x21, #AST_F4]
    mov x21, x0                     // last = tail
    b .Lcompound_loop

.Lcompound_first:
    mov x20, x22
    mov x21, x0                     // last = tail
    b .Lcompound_loop

.Lcompound_close:
    // Expect '}'
    mov x0, #TOK_RBRACE
    bl _kcc_parse_expect

    // Pop typedef scope
    bl _kcc_typedef_pop_scope

    // Store first item
    str x20, [x19, #AST_F0]

    mov x0, x19                     // return
   compound node

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

230 · used in 29b · defines _kcc_parse_compound 181, 211, 225 · uses _kcc_ast_alloc 168a, _kcc_parse_decl 181, _kcc_parse_expect 168b, _kcc_parse_is_type_token 202, _kcc_parse_stmt 225, _kcc_pp_next 108, _kcc_tok_len 104b, _kcc_tok_start 104b, _kcc_tok_type 104b, _kcc_typedef_pop_scope 170b, _kcc_typedef_push_scope 170b

23.3. If Statement

The if statement (§6.8.4.1) with optional else clause. The dangling-else ambiguity is resolved by greedy binding: the else always associates with the nearest preceding if.

⟨ *Stmt: parse if* 231a ⟩ ≡

```
// -- kcc_parse_if ---
// node *kcc_parse_if(void)
// Parses: if ( expr ) stmt [else stmt]
.globl _kcc_parse_if
.p2align 2
_kcc_parse_if:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Consume 'if'
    bl _kcc_pp_next

    // Expect '('
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect

    // Parse condition
    bl _kcc_parse_expr
    mov x19, x0                     // x19 =
   condition

    // Expect ')'
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Parse 'then' statement
    bl _kcc_parse_stmt
    mov x20, x0                     // x20 = then

    // Check for 'else'
    mov x21, #0                     // x21 = else (
  default: null)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ELSE
    b.ne .Lif_build

    // Consume 'else'
    bl _kcc_pp_next

    // Parse 'else' statement
    bl _kcc_parse_stmt
    mov x21, x0                     // x21 = else

.Lif_build:
    mov x0, #AST_IF_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // condition
    str x20, [x0, #AST_F1]      // then
    str x21, [x0, #AST_F2]      // else (or null)

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

231a · used in 29b · defines _kcc_parse_if 225 · uses _kcc_ast_alloc 168a, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_stmt 225, _kcc_pp_next 108, _kcc_tok_type 104b

23.4. Switch Statement

The switch statement (§6.8.4.2) evaluates an integer expression and transfers control to the matching case label within its body. The parser consumes the `switch` keyword, expects a parenthesized expression, then parses the body as a single statement (typically a compound statement containing case and `default` labels). The resulting `AST_SWITCH_STMT` node stores the controlling expression and the body.

⟨ *Stmt: parse switch* 231b ⟩ ≡

```
// -- kcc_parse_switch ---
// node *kcc_parse_switch(void)
// Parses: switch ( expr ) stmt
.globl _kcc_parse_switch
.p2align 2
_kcc_parse_switch:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // Consume 'switch'
    bl _kcc_pp_next

    // Expect '('
    mov x0, #TOK_LPAREN
```

```
    bl _kcc_parse_expect

    // Parse expression
    bl _kcc_parse_expr
    mov x19, x0                     // x19 = expr

    // Expect ')'
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Parse body
    bl _kcc_parse_stmt
    mov x20, x0                     // x20 = body

    mov x0, #AST_SWITCH_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // expr
    str x20, [x0, #AST_F1]      // body

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

231b · used in 29b · defines _kcc_parse_switch 225 · uses _kcc_ast_alloc 168a, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_stmt 225, _kcc_pp_next 108

23.5. While Statement

The while loop (§6.8.5.1) repeatedly executes a statement as long as its controlling expression evaluates to nonzero. The parser expects the `while` keyword followed by a parenthesized condition and a loop body statement. The resulting `AST_WHILE_STMT` node stores the condition in `F0` and the body in `F1`.

⟨ *Stmt: parse while* 232a ⟩ ≡

```
// -- kcc_parse_while ---
// node *kcc_parse_while(void)
// Parses: while ( expr ) stmt
.globl _kcc_parse_while
.p2align 2
_kcc_parse_while:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // Consume 'while'
    bl _kcc_pp_next

    // Expect '('
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect

    // Parse condition
    bl _kcc_parse_expr
    mov x19, x0                     // x19 =
   condition

    // Expect ')'
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Parse body
    bl _kcc_parse_stmt
    mov x20, x0                     // x20 = body

    mov x0, #AST_WHILE_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // condition
    str x20, [x0, #AST_F1]      // body

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

232a · used in 29b · defines _kcc_parse_while 225 · uses _kcc_ast_alloc 168a, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_stmt 225, _kcc_pp_next 108

23.6. Do-While Statement

The do-while loop (§6.8.5.2) executes its body at least once before checking the loop condition. The syntax `do stmt while (expr);` requires the parser to consume the body first, then expect the `while` keyword, the parenthesized condition, and a terminating semicolon. The resulting `AST_DO_STMT` node stores the body in `F0` and the condition in `F1`, reversing the order from the while loop to reflect the evaluation sequence.

⟨ *Stmt: parse do* 232b ⟩ ≡

```
// -- kcc_parse_do ---
// node *kcc_parse_do(void)
// Parses: do stmt while ( expr ) ;
.globl _kcc_parse_do
.p2align 2
_kcc_parse_do:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // Consume 'do'
    bl _kcc_pp_next

    // Parse body
    bl _kcc_parse_stmt
    mov x19, x0                     // x19 = body

    // Expect 'while'
    mov x0, #TOK_WHILE
    bl _kcc_parse_expect

    // Expect '('
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect

    // Parse condition
    bl _kcc_parse_expr
    mov x20, x0                     // x20 =
   condition

    // Expect ')'
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Expect ';'
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect

    mov x0, #AST_DO_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // body
    str x20, [x0, #AST_F1]      // condition

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
```

```
    ret
```

□

232b · used in 29b · defines _kcc_parse_do 225 · uses _kcc_ast_alloc 168a, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_stmt 225, _kcc_pp_next 108

23.7. For Statement

The for statement (§6.8.5.3) has three optional clauses. The init clause can be a declaration (introducing a new scope) or an expression.

⟨ *Stmt: parse for* 233 ⟩ ≡

```
// -- kcc_parse_for ---
// node *kcc_parse_for(void)
// Parses: for ( [init] ; [cond] ; [incr] ) stmt
// init can be a declaration or expression.
.globl _kcc_parse_for
.p2align 2
_kcc_parse_for:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    // Consume 'for'
    bl _kcc_pp_next

    // Expect '('
    mov x0, #TOK_LPAREN
    bl _kcc_parse_expect

    // Parse init clause
    mov x19, #0                     // x19 = init (
  default null)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_SEMI
    b.eq .Lfor_skip_init

    // Check if init is a declaration
    bl _kcc_parse_is_type_token
    cbnz x0, .Lfor_init_decl

    // Check storage class keywords
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp x0, #TOK_TYPEDEF
    b.eq .Lfor_init_decl
    cmp x0, #TOK_EXTERN
    b.eq .Lfor_init_decl
    cmp x0, #TOK_STATIC
    b.eq .Lfor_init_decl
    cmp x0, #TOK_AUTO
    b.eq .Lfor_init_decl
    cmp x0, #TOK_REGISTER
    b.eq .Lfor_init_decl

    // Expression init - wrap in AST_EXPR_STMT so
  IR builder handles it
    bl _kcc_parse_expr
    mov x19, x0
    mov x0, #AST_EXPR_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]
    mov x19, x0
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect
    b .Lfor_cond

.Lfor_init_decl:
    // Declaration (includes the trailing ';')
    bl _kcc_parse_decl
    mov x19, x0
    b .Lfor_cond

.Lfor_skip_init:
    bl _kcc_pp_next                 // consume ';'

.Lfor_cond:
    // Parse condition
    mov x20, #0                     // x20 = cond (
  default null)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_SEMI
    b.eq .Lfor_skip_cond

    bl _kcc_parse_expr
    mov x20, x0

.Lfor_skip_cond:
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect

    // Parse increment
    mov x21, #0                     // x21 = incr (
  default null)
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_RPAREN
    b.eq .Lfor_skip_incr

    bl _kcc_parse_expr
    mov x21, x0

.Lfor_skip_incr:
    // Expect ')'
    mov x0, #TOK_RPAREN
    bl _kcc_parse_expect

    // Parse body
    bl _kcc_parse_stmt
    mov x22, x0                     // x22 = body

    // Build for node
    mov x0, #AST_FOR_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // init
    str x20, [x0, #AST_F1]      // cond
    str x21, [x0, #AST_F2]      // incr
    str x22, [x0, #AST_F3]      // body

    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

233 · used in 29b · defines _kcc_parse_for 225 · uses _kcc_ast_alloc 168a, _kcc_parse_decl 181, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_parse_is_type_token 202, _kcc_parse_stmt 225, _kcc_pp_next 108, _kcc_tok_type 104b

23.8. Goto Statement

The goto statement (§6.8.6.1) transfers control unconditionally to the named label within the current function. The parser consumes the goto keyword, interns the target label identifier via the string table, and expects a terminating semicolon. The resulting AST_GOTO_STMT node stores the interned label name pointer for later resolution during semantic analysis.

⟨ *Stmt: parse goto* 234a ⟩ ≡

```
// -- kcc_parse_goto ---
// node *kcc_parse_goto(void)
// Parses: goto ident ;
.globl _kcc_parse_goto
.p2align 2
_kcc_parse_goto:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Consume 'goto'
    bl _kcc_pp_next

    // Get label name
    adrp x0, _kcc_tok_start@PAGE
    add  x0, x0, _kcc_tok_start@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_tok_len@PAGE
    add  x1, x1, _kcc_tok_len@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_strtab_intern
    mov x19, x0                    // x19 = label
   name

    // Consume identifier
    bl _kcc_pp_next

    // Expect ';'
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect

    mov x0, #AST_GOTO_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // label_name_ptr

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

234a · used in 29b · defines _kcc_parse_goto 226 · uses _kcc_ast_alloc 168a, _kcc_parse_expect 168b, _kcc_pp_next 108, _kcc_strtab_intern 66b, _kcc_tok_len 104b, _kcc_tok_start 104b

23.9. Return Statement

The return statement (§6.8.6.4) exits the current function, optionally providing a value to the caller. If a semicolon immediately follows the `return` keyword, the return value is null (appropriate for `void` functions); otherwise the parser reads a full expression. The resulting AST_RETURN_STMT node stores the expression (or zero) in F0.

⟨ *Stmt: parse return* 234b ⟩ ≡

```
// -- kcc_parse_return ---
// node *kcc_parse_return(void)
// Parses: return [expr] ;
.globl _kcc_parse_return
.p2align 2
_kcc_parse_return:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Consume 'return'
    bl _kcc_pp_next

    // Check for immediate ';' (return without
   value)
    mov x19, #0                    // default: no
   return value
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_SEMI
    b.eq .Lreturn_semi

    // Parse return expression
    bl _kcc_parse_expr
    mov x19, x0

.Lreturn_semi:
    mov x0, #TOK_SEMI
    bl _kcc_parse_expect

    mov x0, #AST_RETURN_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // expr (or 0)

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

234b · used in 29b · defines _kcc_parse_return 226 · uses _kcc_ast_alloc 168a, _kcc_parse_expect 168b, _kcc_parse_expr 207b, _kcc_pp_next 108, _kcc_tok_type 104b

23.10. Labeled Statements

Case and default labels for switch statements. A case label carries its constant in F0 and its labelled statement in F1. kcc also accepts the GNU *case range* case `LOW … HIGH:`—an ellipsis between two constants—which matches every value in the inclusive interval; the high bound is parsed into F3 (null for an ordinary single-value case). The kernel uses this heavily, e.g. `case '0' … '7':`.

⟨ *Stmt: parse labeled* 234c ⟩ ≡

```
// -- kcc_parse_case ---
// node *kcc_parse_case(void)
// Parses: case expr : stmt
.globl _kcc_parse_case
.p2align 2
_kcc_parse_case:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    // Consume 'case'
    bl _kcc_pp_next

    // Parse constant expression (the value, or
   the low bound of a range)
```

```
    bl _kcc_parse_cond_expr
    mov x19, x0                 // x19 = low expr
    mov x20, xzr                // x20 = high
   expr (0 = single case)

    // GNU case range: `case LOW ... HIGH:`.  An
   ellipsis after the first
    // constant introduces the inclusive upper
   bound.
    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #TOK_ELLIPSIS
    b.ne .Lpc_colon
    bl _kcc_pp_next             // consume '...'
    bl _kcc_parse_cond_expr
    mov x20, x0                 // x20 = high
   expr

.Lpc_colon:
    // Expect ':'
    mov x0, #TOK_COLON
    bl _kcc_parse_expect

    // Parse statement
    bl _kcc_parse_stmt
    mov x21, x0                 // x21 = stmt

    mov x0, #AST_CASE_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]      // low expr
    str x21, [x0, #AST_F1]      // stmt
    str x20, [x0, #AST_F3]      // high expr (0 =
   single case)

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_parse_default ---
// node *kcc_parse_default(void)
// Parses: default : stmt
.globl _kcc_parse_default
.p2align 2
_kcc_parse_default:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Consume 'default'
    bl _kcc_pp_next

    // Expect ':'
    mov x0, #TOK_COLON
    bl _kcc_parse_expect

    // Parse statement
    bl _kcc_parse_stmt
    mov x19, x0                  // x19 = stmt

    mov x0, #AST_DEFAULT_STMT
    bl _kcc_ast_alloc
    str x19, [x0, #AST_F0]       // stmt

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

234c · used in 29b · defines _kcc_parse_case 226, _kcc_parse_default 226 · uses _kcc_ast_alloc 168a, _kcc_parse_cond_expr 208b, _kcc_parse_expect 168b, _kcc_parse_stmt 225, _kcc_pp_next 108, _kcc_tok_type 104b

23.11. Statement Data

This fragment reserves a place for any static data that the statement parser might need, such as error message strings or auxiliary tables. Currently the statement parser requires no additional data beyond what is defined inline within each parsing function.

⟨ *Stmt: data* 235 ⟩ ≡

```
// -- Statement parser data (reserved) ---
// No additional data needed for statement parsing
   .
```

□

235 · used in 29b

Part V.

Semantics

Parsing proves that a program is syntactically well-formed; semantic analysis proves that it is *meaningful*. This part implements the type system (Chapter 24), with singleton descriptors for every built-in type and constructors for pointers, arrays, functions, structs, unions, and enums; the symbol table (Chapter 25), with separate scoped name spaces for ordinary identifiers and tags as required by §6.2.3; the semantic analyser (Chapter 26), which walks the AST checking declarations, statements, and expressions against the language's type rules; and the implicit conversion module (Chapter 27), which implements the standard's array, function, and lvalue decay rules. After this pass, every AST expression node carries its resolved type, and any constraint violation has been reported as a diagnostic.

24. Type System

Every C expression has a type, and the compiler must track these types to enforce the language's constraints and to determine the size, alignment, and representation of every object. This chapter defines the type descriptor—a 48-byte arena-allocated record that encodes kind, size, alignment, and qualifiers—and builds the infrastructure around it. We begin with the descriptor layout and the pre-allocated singletons for built-in types, then provide constructors for derived types (pointers, arrays, functions, structures, unions, and enumerations), predicate functions for querying type properties, a specifier resolver that converts the parser's bitmask into a canonical type, and a diagnostic dump for debugging.

24.1. Type Descriptor Layout

A type descriptor occupies 48 bytes with the following layout:

Offset	Field	Description
0	TY_KIND	type kind constant
8	TY_SIZE	size in bytes
16	TY_ALIGN	alignment in bytes
24	TY_QUALS	const=1, volatile=2, restrict=4, _Atomic=8
32	TY_F0	kind-specific field 0
40	TY_F1	kind-specific field 1

⟨ *Types: constants* 239a ⟩ ≡

```
// -- Type descriptor layout ---
.equ TY_KIND,       0
.equ TY_SIZE,       8
.equ TY_ALIGN,     16
.equ TY_QUALS,     24
.equ TY_F0,        32
.equ TY_F1,        40
.equ TY_DESC_SIZE, 48

// -- Type kind constants ---
.equ TY_VOID,       1
.equ TY_BOOL,       2
.equ TY_CHAR,       3    // F0=signed(1)/unsigned
   (0)
.equ TY_SHORT,      4    // F0=signed(1)/unsigned
   (0)
.equ TY_INT,        5    // F0=signed(1)/unsigned
   (0)
.equ TY_LONG,       6    // F0=signed(1)/unsigned
   (0)
.equ TY_LLONG,      7    // F0=signed(1)/unsigned
   (0)
.equ TY_FLOAT,      8
.equ TY_DOUBLE,     9
.equ TY_LDOUBLE,   10
.equ TY_PTR,       11    // F0=pointee type
   descriptor
.equ TY_ARRAY,     12    // F0=element type, F1=
   length
.equ TY_FUNC,      13    // F0=return type, F1=
   param vec*
.equ TY_STRUCT,    14    // F0=struct_decl AST, F1
   =member vec*
.equ TY_UNION,     15    // F0=union_decl AST, F1=
   member vec*
.equ TY_ENUM,      16    // F0=enum_decl AST
.equ TY_INT128,    17    // __int128, F0=signed(1)
   /unsigned(0); size 16, align 16

// -- Specifier bitmask constants (from parser)
   ---
.equ SPEC_VOID,       (1 << 0)
.equ SPEC_CHAR,       (1 << 1)
.equ SPEC_SHORT,      (1 << 2)
.equ SPEC_INT,        (1 << 3)
.equ SPEC_LONG,       (1 << 4)
.equ SPEC_LLONG,      (1 << 5)
.equ SPEC_FLOAT,      (1 << 6)
.equ SPEC_DOUBLE,     (1 << 7)
.equ SPEC_SIGNED,     (1 << 8)
.equ SPEC_UNSIGNED,   (1 << 9)
.equ SPEC_BOOL,       (1 << 10)
.equ SPEC_COMPLEX,    (1 << 11)
.equ SPEC_INT128,     (1 << 12)
```

□

239a · used in 29df, 30abc, 31ac

24.2. Type Initialization

Pre-allocate singleton descriptors for all basic types. Alongside the standard integer widths, two singletons cover the GNU 128-bit integer `__int128` (kind `TY_INT128`, signed and unsigned): each carries size and alignment 16, so it lays out correctly wherever a 16-byte object is expected—notably inside the `union __u128_halves` the Linux kernel's `cmpxchg16b` code builds. Only its *arithmetic* is deferred (Appendix 3); the descriptor itself is a first-class basic type.

⟨ *Types: init* 239b ⟩ ≡

```
// -- kcc_types_init ---
// void kcc_types_init(void)
// Pre-allocates singleton type descriptors for
   all basic types.
.text
.globl _kcc_types_init
.p2align 2
_kcc_types_init:
```

```
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // void
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_VOID
    str x1, [x19, #TY_KIND]
    str xzr, [x19, #TY_SIZE]        // void has no
  size
    str xzr, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    str xzr, [x19, #TY_F0]
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_void@PAGE
    add  x1, x1, _kcc_type_void@PAGEOFF
    str x19, [x1]

    // _Bool
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_BOOL
    str x1, [x19, #TY_KIND]
    mov x1, #1
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    str xzr, [x19, #TY_F0]
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_bool@PAGE
    add  x1, x1, _kcc_type_bool@PAGEOFF
    str x19, [x1]
```

□

239b · defined at 239b, 240, 241 · used in 29d · defines _kcc_types_init 603, 623 · uses _kcc_arena_alloc 58a, _kcc_type_bool 250b, _kcc_type_void 250b

Character and Integer Type Singletons The next block allocates singletons for every standard integer type, from `char` through `long long`, each in both signed and unsigned variants. The C17 standard treats plain `char`, `signed char`, and `unsigned char` as three distinct types (§6.2.5p15); our implementation maps plain `char` to `signed char`, following the default sign convention on Apple ARM64 targets. Each descriptor stores its signedness in `TY_F0` (1 for signed, 0 for unsigned) so that later stages can select the correct conversion and promotion rules.

⟨ *Types: init* 240 ⟩ +≡

```
    // signed char
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_CHAR
    str x1, [x19, #TY_KIND]
    mov x1, #1
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    mov x1, #1                          // signed
    str x1, [x19, #TY_F0]
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_char_s@PAGE
    add  x1, x1, _kcc_type_char_s@PAGEOFF
    str x19, [x1]

    // unsigned char
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_CHAR
    str x1, [x19, #TY_KIND]
    mov x1, #1
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    str xzr, [x19, #TY_F0]          // unsigned
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_char_u@PAGE
    add  x1, x1, _kcc_type_char_u@PAGEOFF
    str x19, [x1]

    // signed short
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_SHORT
    str x1, [x19, #TY_KIND]
    mov x1, #2
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    mov x1, #1                          // signed
    str x1, [x19, #TY_F0]
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_short_s@PAGE
    add  x1, x1, _kcc_type_short_s@PAGEOFF
    str x19, [x1]

    // unsigned short
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_SHORT
    str x1, [x19, #TY_KIND]
    mov x1, #2
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    str xzr, [x19, #TY_F0]          // unsigned
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_short_u@PAGE
    add  x1, x1, _kcc_type_short_u@PAGEOFF
    str x19, [x1]

    // signed int
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_INT
    str x1, [x19, #TY_KIND]
    mov x1, #4
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    mov x1, #1                          // signed
    str x1, [x19, #TY_F0]
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    str x19, [x1]

    // unsigned int
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_INT
    str x1, [x19, #TY_KIND]
```

```
    mov x1, #4
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    str xzr, [x19, #TY_F0]          // unsigned
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_int_u@PAGE
    add  x1, x1, _kcc_type_int_u@PAGEOFF
    str x19, [x1]

    // signed long (8 bytes on ARM64)
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_LONG
    str x1, [x19, #TY_KIND]
    mov x1, #8
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    mov x1, #1                      // signed
    str x1, [x19, #TY_F0]
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_long_s@PAGE
    add  x1, x1, _kcc_type_long_s@PAGEOFF
    str x19, [x1]

    // unsigned long
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_LONG
    str x1, [x19, #TY_KIND]
    mov x1, #8
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    str xzr, [x19, #TY_F0]          // unsigned
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_long_u@PAGE
    add  x1, x1, _kcc_type_long_u@PAGEOFF
    str x19, [x1]

    // signed long long (8 bytes on ARM64)
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_LLONG
    str x1, [x19, #TY_KIND]
    mov x1, #8
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    mov x1, #1                      // signed
    str x1, [x19, #TY_F0]
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_llong_s@PAGE
    add  x1, x1, _kcc_type_llong_s@PAGEOFF
    str x19, [x1]

    // unsigned long long
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_LLONG
    str x1, [x19, #TY_KIND]
    mov x1, #8
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    str xzr, [x19, #TY_F0]          // unsigned
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_llong_u@PAGE
    add  x1, x1, _kcc_type_llong_u@PAGEOFF
    str x19, [x1]

    // signed __int128 (16 bytes, 16-byte aligned)
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_INT128
    str x1, [x19, #TY_KIND]
    mov x1, #16
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    mov x1, #1                      // signed
    str x1, [x19, #TY_F0]
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_int128_s@PAGE
    add  x1, x1, _kcc_type_int128_s@PAGEOFF
    str x19, [x1]

    // unsigned __int128
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_INT128
    str x1, [x19, #TY_KIND]
    mov x1, #16
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    str xzr, [x19, #TY_F0]          // unsigned
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_int128_u@PAGE
    add  x1, x1, _kcc_type_int128_u@PAGEOFF
    str x19, [x1]
```

□

240 · defined at 239b, 240, 241 · used in 29d · uses _kcc_arena_alloc 58a, _kcc_type_char_s 250b, _kcc_type_char_u 250b, _kcc_type_int128_s 250b, _kcc_type_int128_u 250b, _kcc_type_int_s 250b, _kcc_type_int_u 250b, _kcc_type_llong_s 250b, _kcc_type_llong_u 250b, _kcc_type_long_s 250b, _kcc_type_long_u 250b, _kcc_type_short_s 250b, _kcc_type_short_u 250b

Floating-Point Type Singletons The floating-point singletons cover `float` (4 bytes) and `double` (8 bytes). `long double` is a third floating type in the abstract machine, but the Apple ARM64 ABI defines it to have the same representation as `double` (64-bit IEEE-754)—unlike AArch64 Linux, where it is a 128-bit quad. We honour that ABI by *aliasing* the `long double` singleton to the `double` descriptor rather than allocating a distinct one, so the many places that recognise a floating type by its kind (`TY_DOUBLE`) need no change and `long double` lowers, sizes, and passes exactly as `double`. Unlike integer types, floating-point descriptors do not use `TY_F0` for signedness — all are inherently signed per §6.2.5p10. After allocating the last singleton the function restores callee-saved registers x19/x20 and returns.

⟨ *Types: init* 241 ⟩ +≡

```
    // float
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_FLOAT
```

```
    str x1, [x19, #TY_KIND]
    mov x1, #4
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    str xzr, [x19, #TY_F0]
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_float@PAGE
    add  x1, x1, _kcc_type_float@PAGEOFF
    str x19, [x1]

    // double
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0
    mov x1, #TY_DOUBLE
    str x1, [x19, #TY_KIND]
    mov x1, #8
    str x1, [x19, #TY_SIZE]
    str x1, [x19, #TY_ALIGN]
    str xzr, [x19, #TY_QUALS]
    str xzr, [x19, #TY_F0]
    str xzr, [x19, #TY_F1]
    adrp x1, _kcc_type_double@PAGE
    add  x1, x1, _kcc_type_double@PAGEOFF
    str x19, [x1]

    // long double: on the Apple ARM64 ABI, long
   double is identical to
    // double (64-bit IEEE-754), unlike AArch64
   Linux where it is a 128-bit
    // quad.  Alias the descriptor to the double
   one just built, so every
    // path---FP lowering, sizeof, printf,
   argument passing---treats a
    // long double exactly as a double.
    adrp x1, _kcc_type_double@PAGE
    add  x1, x1, _kcc_type_double@PAGEOFF
    ldr  x0, [x1]                  // the double
   descriptor
    adrp x1, _kcc_type_ldouble@PAGE
    add  x1, x1, _kcc_type_ldouble@PAGEOFF
    str  x0, [x1]

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

241 · defined at 239b, 240, 241 · used in 29d · uses _kcc_arena_alloc 58a, _kcc_type_double 250b, _kcc_type_float 250b, _kcc_type_ldouble 250b

24.3. Type Constructors

24.3.1. Pointer Type

Construct a pointer type descriptor from a given pointee type. The function arena-allocates a TY_DESC_SIZE record, sets its kind to TY_PTR, and records size and alignment as 8 bytes (the pointer width on ARM64). The pointee type descriptor is stored in TY_F0 so that later phases can dereference through the pointer chain.

⟨ *Types: make pointer* 242a ⟩ ≡

```
// -- kcc_type_ptr ---
// type_desc *kcc_type_ptr(type_desc *pointee)
//   x0 = pointee type descriptor
// Returns pointer to TY_PTR descriptor.
.text
.globl _kcc_type_ptr
.p2align 2
_kcc_type_ptr:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                        // save
   pointee

    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x1, #TY_PTR
    str x1, [x0, #TY_KIND]
    mov x1, #8                         // pointer
   size on ARM64
    str x1, [x0, #TY_SIZE]
    str x1, [x0, #TY_ALIGN]
    str xzr, [x0, #TY_QUALS]
    str x19, [x0, #TY_F0]          // pointee
    str xzr, [x0, #TY_F1]

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

242a · defined at 242ab · used in 29d · defines _kcc_type_ptr 242b, 259b, 276, 283, 286, 296ab · uses _kcc_arena_alloc 58a

Parameter type adjustment. A function parameter declared with array type is, per § 6.7.6.3, adjusted to a *pointer* to the element type; likewise a function-typed parameter becomes a pointer to the function. This matters because the callee never receives the array by value—it receives the address the array decayed to at the call site. If the parameter kept its array type, the code generator would allocate inline array storage and index *that* rather than loading the passed pointer, so `a[i]` would read the callee's own stack. `kcc_type_param_adjust` performs the adjustment; it is applied wherever a parameter's type is resolved (symbol registration, the alloca, and AAPCS64 classification).

⟨ *Types: make pointer* 242b ⟩ + ≡

```
// -- kcc_type_param_adjust ---
// type_desc *kcc_type_param_adjust(type_desc *t)
//   x0 = resolved parameter type; returns the
   adjusted type: an array or
//   function type becomes a pointer (to element /
    to function); any other
//   type is returned unchanged (\stdref{6.7.6.3})
   .
.text
.globl _kcc_type_param_adjust
.p2align 2
_kcc_type_param_adjust:
    cbz x0, .Ltpa_ret
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.eq .Ltpa_array
    cmp x1, #TY_FUNC
    b.eq _kcc_type_ptr             // function →
   pointer to function (x0 unchanged)
    ret
.Ltpa_array:
    ldr x0, [x0, #TY_F0]           // element type
```

```
    b _kcc_type_ptr              // pointer to
   element
.Ltpa_ret:
    ret
```

□

242b · defined at 242ab · used in 29d · defines _kcc_type_param_adjust 264, 320, 343 · uses _kcc_type_ptr 242a

24.3.2. Array Type

Construct an array type descriptor. The element type is saved in TY_F0 and the array length in TY_F1; a length of zero indicates an incomplete array type (§6.2.5). The total size is computed as element size multiplied by length, and alignment is inherited directly from the element type.

Aggregate element types are sized *lazily*: kcc_type_struct leaves TY_SIZE at zero until kcc_type_layout runs. If we read the element size before that layout happens—as when a local struct S a[3] is declared before S is otherwise referenced—the multiplication captures a stale zero and the whole array appears to have size zero (so sizeof a is wrong and memset(a,0,sizeof a) clears nothing). We therefore lay the element out first; layout is idempotent, so this is safe for scalar elements too and merely returns their already-known size.

⟨ *Types: make array* 243a ⟩ ≡

```
// -- kcc_type_array ---
// type_desc *kcc_type_array(type_desc *elem,
   uint64_t len)
//   x0 = element type, x1 = length (0 =
   incomplete)
// Returns TY_ARRAY descriptor.
.text
.globl _kcc_type_array
.p2align 2
_kcc_type_array:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                      // elem type
    mov x20, x1                      // length

    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x21, x0                      // array
   descriptor
    mov x1, #TY_ARRAY
    str x1, [x21, #TY_KIND]
    // Ensure the element is laid out so aggregate
    sizes are known, then
    // size = elem_size * length.
    mov x0, x19
    bl _kcc_type_layout              // x0 = element
    size
    mul x0, x0, x20
    str x0, [x21, #TY_SIZE]
    ldr x2, [x19, #TY_ALIGN]
    str x2, [x21, #TY_ALIGN]
    str xzr, [x21, #TY_QUALS]
    str x19, [x21, #TY_F0]           // element type
    str x20, [x21, #TY_F1]           // length

    mov x0, x21
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

243a · used in 29d · defines _kcc_type_array 259b, 292b · uses _kcc_arena_alloc 58a, _kcc_type_layout 267, 268

24.3.3. Function Type

Construct a function type descriptor. The return type is stored in TY_F0 and the parameter type vector in TY_F1. Function types are not object types, so both size and alignment are set to zero (§6.2.5). The parameter vector may be null for unprototyped (K&R-style) declarations.

⟨ *Types: make function* 243b ⟩ ≡

```
// -- kcc_type_func ---
// type_desc *kcc_type_func(type_desc *ret, vec *
   params)
//   x0 = return type, x1 = parameter type vec (or
    0)
// Returns TY_FUNC descriptor.
.text
.globl _kcc_type_func
.p2align 2
_kcc_type_func:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    mov x19, x0                       // return type
    mov x20, x1                       // params vec

    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x1, #TY_FUNC
    str x1, [x0, #TY_KIND]
    str xzr, [x0, #TY_SIZE]          // function
   types have no size
    str xzr, [x0, #TY_ALIGN]
    str xzr, [x0, #TY_QUALS]
    str x19, [x0, #TY_F0]            // return type
    str x20, [x0, #TY_F1]            // param vec

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

243b · used in 29d · defines _kcc_type_func 259b, 264 · uses _kcc_arena_alloc 58a

24.3.4. Struct/Union Type

Construct struct and union type descriptors. At creation time, size and alignment are left at zero because member layout has not been computed yet; the semantic analysis pass fills them in once all members are known. The AST declaration node is stored in TY_F0, and TY_F1 will later hold the member vector populated by sema. The only difference between the two constructors is the kind constant (TY_STRUCT versus TY_UNION).

⟨ *Types: make struct* 243c ⟩ ≡

```
// -- kcc_type_struct ---
// type_desc *kcc_type_struct(node *decl_node)
//   x0 = AST struct/union declaration node
// Returns TY_STRUCT descriptor.
.text
.globl _kcc_type_struct
.p2align 2
_kcc_type_struct:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                         // decl node

    // Memoise one descriptor per struct-definition AST node (cached in the
    // decl's spare AST_F2, zero-initialised by ast_alloc).  Re-resolving the
    // same definition---especially an ANONYMOUS one, which has no tag to dedup
    // through---must return the SAME descriptor.  A second descriptor would
    // share the field AST nodes, and laying it out would re-run destructive
    // bit-field packing, dereferencing an AST_F2 the first layout already
    // replaced with a packed (bit_offset<<16 | width) integer as if it were a
    // width-expression pointer.  SQLite's `struct SrcItem` has an anonymous
    // `struct {...} fg` of bit-fields accessed as `pItem->fg.isTabFunc`.
    cbz x19, .Lts_alloc
    ldr x0, [x19, #AST_F2]              // cached descriptor?
    cbnz x0, .Lts_ret
.Lts_alloc:
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x1, #TY_STRUCT
    str x1, [x0, #TY_KIND]
    str xzr, [x0, #TY_SIZE]             // computed later during layout
    str xzr, [x0, #TY_ALIGN]
    str xzr, [x0, #TY_QUALS]
    str x19, [x0, #TY_F0]               // AST decl node
    str xzr, [x0, #TY_F1]               // member vec (populated by sema)
    cbz x19, .Lts_ret
    str x0, [x19, #AST_F2]              // cache on the decl node
.Lts_ret:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_type_union ---
// type_desc *kcc_type_union(node *decl_node)
.globl _kcc_type_union
.p2align 2
_kcc_type_union:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0

    // Memoise one descriptor per union-definition node (see kcc_type_struct).
    cbz x19, .Ltu_alloc
    ldr x0, [x19, #AST_F2]
    cbnz x0, .Ltu_ret
.Ltu_alloc:
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x1, #TY_UNION
    str x1, [x0, #TY_KIND]
    str xzr, [x0, #TY_SIZE]
    str xzr, [x0, #TY_ALIGN]
    str xzr, [x0, #TY_QUALS]
    str x19, [x0, #TY_F0]
    str xzr, [x0, #TY_F1]
    cbz x19, .Ltu_ret
    str x0, [x19, #AST_F2]
.Ltu_ret:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

243c · used in 29d · defines _kcc_type_struct 199b, 259b, 269, _kcc_type_union 259b, 269 · uses _kcc_arena_alloc 58a

24.3.5. Enum Type

Construct an enum type descriptor. Per §6.7.2.2, the underlying type of an enumeration is `int`, so size and alignment are both set to 4. The AST enum declaration node is stored in TY_F0 for later reference during enumerator constant evaluation.

⟨ *Types: make enum* 244 ⟩ ≡

```
// -- kcc_type_enum ---
// type_desc *kcc_type_enum(node *decl_node)
//   x0 = AST enum declaration node
// Returns TY_ENUM descriptor (underlying type is int).
.text
.globl _kcc_type_enum
.p2align 2
_kcc_type_enum:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0

    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x1, #TY_ENUM
    str x1, [x0, #TY_KIND]
    mov x1, #4                          // enum has int size
    str x1, [x0, #TY_SIZE]
    str x1, [x0, #TY_ALIGN]
    str xzr, [x0, #TY_QUALS]
    str x19, [x0, #TY_F0]               // decl node
    str xzr, [x0, #TY_F1]

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

244 · used in 29d · defines _kcc_type_enum 259b, 269 · uses _kcc_arena_alloc 58a

24.4. Specifier Bitmask Resolution

The parser stores a bitmask of type specifiers in `AST_TYPE_BUILTIN.FO`. This function maps each valid combination to the corresponding singleton type descriptor. Besides the standard widths it accepts the GNU `__int128` specifier (bit `SPEC_INT128`): bare or with `signed` it selects the signed 128-bit singleton, with `unsigned` the unsigned one.

⟨ *Types: resolve specmask* 245a ⟩ ≡

```
// -- kcc_type_from_specmask ---
// type_desc *kcc_type_from_specmask(uint64_t mask
   )
//   x0 = specifier bitmask from parser
// Returns type descriptor pointer, or 0 on
   invalid combination.
.text
.globl _kcc_type_from_specmask
.p2align 2
_kcc_type_from_specmask:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // void
    cmp x0, #SPEC_VOID
    b.eq .Ltfm_void

    // _Bool
    mov x1, #SPEC_BOOL
    cmp x0, x1
    b.eq .Ltfm_bool

    // char (plain or signed char)
    cmp x0, #SPEC_CHAR
    b.eq .Ltfm_char_s
    mov x1, #(SPEC_SIGNED | SPEC_CHAR)
    cmp x0, x1
    b.eq .Ltfm_char_s

    // unsigned char
    mov x1, #(SPEC_UNSIGNED | SPEC_CHAR)
    cmp x0, x1
    b.eq .Ltfm_char_u

    // short / signed short / short int / signed
   short int
    cmp x0, #SPEC_SHORT
    b.eq .Ltfm_short_s
    mov x1, #(SPEC_SIGNED | SPEC_SHORT)
    cmp x0, x1
    b.eq .Ltfm_short_s
    mov x1, #(SPEC_SHORT | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_short_s
    mov x1, #(SPEC_SIGNED | SPEC_SHORT | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_short_s

    // unsigned short / unsigned short int
    mov x1, #(SPEC_UNSIGNED | SPEC_SHORT)
    cmp x0, x1
    b.eq .Ltfm_short_u
    mov x1, #(SPEC_UNSIGNED | SPEC_SHORT |
   SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_short_u

    // int / signed / signed int
    cmp x0, #SPEC_INT
    b.eq .Ltfm_int_s
    mov x1, #SPEC_SIGNED
    cmp x0, x1
    b.eq .Ltfm_int_s
    mov x1, #(SPEC_SIGNED | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_int_s

    // unsigned / unsigned int
    mov x1, #SPEC_UNSIGNED
    cmp x0, x1
    b.eq .Ltfm_int_u
    mov x1, #(SPEC_UNSIGNED | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_int_u
```

□

245a · defined at 245ab, 246 · used in 29d · defines `_kcc_type_from_specmask` 259b

Long, Long Long, and Floating-Point Dispatch The remaining dispatch covers `long` and `long long` types, which accept the most specifier combinations in C17. The parser may set `SPEC_LONG` alone or combined with `SPEC_LLONG` (from two successive `long` keywords), and any of these may appear with or without `SPEC_SIGNED`, `SPEC_UNSIGNED`, or `SPEC_INT`. After the integer specifiers, the floating-point keywords `float`, `double`, and `long double` are matched. Any bitmask that falls through every comparison returns zero to signal an invalid combination.

⟨ *Types: resolve specmask* 245b ⟩ + ≡

```
    // long / signed long / long int / signed long
    int
    mov x1, #SPEC_LONG
    cmp x0, x1
    b.eq .Ltfm_long_s
    mov x1, #(SPEC_SIGNED | SPEC_LONG)
    cmp x0, x1
    b.eq .Ltfm_long_s
    mov x1, #(SPEC_LONG | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_long_s
    mov x1, #(SPEC_SIGNED | SPEC_LONG | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_long_s

    // unsigned long / unsigned long int
    mov x1, #(SPEC_UNSIGNED | SPEC_LONG)
    cmp x0, x1
    b.eq .Ltfm_long_u
    mov x1, #(SPEC_UNSIGNED | SPEC_LONG | SPEC_INT
   )
    cmp x0, x1
    b.eq .Ltfm_long_u

    // long long / signed long long / long long
   int / signed long long int
    mov x1, #SPEC_LLONG
    cmp x0, x1
    b.eq .Ltfm_llong_s
    mov x1, #(SPEC_SIGNED | SPEC_LLONG)
    cmp x0, x1
    b.eq .Ltfm_llong_s
    mov x1, #(SPEC_LLONG | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_llong_s
    mov x1, #(SPEC_SIGNED | SPEC_LLONG | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_llong_s
    // Also accept long + long long combined
```

```
    mov x1, #(SPEC_LONG | SPEC_LLONG)
    cmp x0, x1
    b.eq .Ltfm_llong_s
    mov x1, #(SPEC_SIGNED | SPEC_LONG | SPEC_LLONG
)
    cmp x0, x1
    b.eq .Ltfm_llong_s
    mov x1, #(SPEC_LONG | SPEC_LLONG | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_llong_s
    mov x1, #(SPEC_SIGNED | SPEC_LONG | SPEC_LLONG
    | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_llong_s

    // unsigned long long / unsigned long long int
    mov x1, #(SPEC_UNSIGNED | SPEC_LLONG)
    cmp x0, x1
    b.eq .Ltfm_llong_u
    mov x1, #(SPEC_UNSIGNED | SPEC_LLONG |
SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_llong_u
    mov x1, #(SPEC_UNSIGNED | SPEC_LONG |
SPEC_LLONG)
    cmp x0, x1
    b.eq .Ltfm_llong_u
    mov x1, #(SPEC_UNSIGNED | SPEC_LONG |
SPEC_LLONG | SPEC_INT)
    cmp x0, x1
    b.eq .Ltfm_llong_u

    // float
    mov x1, #SPEC_FLOAT
    cmp x0, x1
    b.eq .Ltfm_float

    // double
    mov x1, #SPEC_DOUBLE
    cmp x0, x1
    b.eq .Ltfm_double

    // long double
    mov x1, #(SPEC_LONG | SPEC_DOUBLE)
    cmp x0, x1
    b.eq .Ltfm_ldouble

    // __int128 / signed __int128  (signed by
  default)
    mov x1, #SPEC_INT128
    cmp x0, x1
    b.eq .Ltfm_int128_s
    mov x1, #(SPEC_SIGNED | SPEC_INT128)
    cmp x0, x1
    b.eq .Ltfm_int128_s
    // unsigned __int128
    mov x1, #(SPEC_UNSIGNED | SPEC_INT128)
    cmp x0, x1
    b.eq .Ltfm_int128_u

    // Invalid combination
    mov x0, #0
    b .Ltfm_done
```

□

245b · defined at 245ab, 246 · used in 29d

Singleton Pointer Loads Each label target below loads the address of the corresponding global singleton pointer (e.g. `_kcc_type_int_s`) using an ADRP+ADD pair, dereferences it to obtain the type descriptor allocated by `kcc_types_init`, and branches to the common epilogue at `.Ltfm_done`. The pattern is uniform across all fifteen basic types — only the symbol name changes.

⟨ *Types: resolve specmask* 246 ⟩ + ≡

```
.Ltfm_void:
    adrp x0, _kcc_type_void@PAGE
    add  x0, x0, _kcc_type_void@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_bool:
    adrp x0, _kcc_type_bool@PAGE
    add  x0, x0, _kcc_type_bool@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_char_s:
    adrp x0, _kcc_type_char_s@PAGE
    add  x0, x0, _kcc_type_char_s@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_char_u:
    adrp x0, _kcc_type_char_u@PAGE
    add  x0, x0, _kcc_type_char_u@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_short_s:
    adrp x0, _kcc_type_short_s@PAGE
    add  x0, x0, _kcc_type_short_s@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_short_u:
    adrp x0, _kcc_type_short_u@PAGE
    add  x0, x0, _kcc_type_short_u@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_int_s:
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_int_u:
    adrp x0, _kcc_type_int_u@PAGE
    add  x0, x0, _kcc_type_int_u@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_long_s:
    adrp x0, _kcc_type_long_s@PAGE
    add  x0, x0, _kcc_type_long_s@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_long_u:
    adrp x0, _kcc_type_long_u@PAGE
    add  x0, x0, _kcc_type_long_u@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_llong_s:
    adrp x0, _kcc_type_llong_s@PAGE
    add  x0, x0, _kcc_type_llong_s@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_llong_u:
    adrp x0, _kcc_type_llong_u@PAGE
    add  x0, x0, _kcc_type_llong_u@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_float:
    adrp x0, _kcc_type_float@PAGE
    add  x0, x0, _kcc_type_float@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_double:
    adrp x0, _kcc_type_double@PAGE
    add  x0, x0, _kcc_type_double@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_ldouble:
```

```
    adrp x0, _kcc_type_ldouble@PAGE
    add  x0, x0, _kcc_type_ldouble@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_int128_s:
    adrp x0, _kcc_type_int128_s@PAGE
    add  x0, x0, _kcc_type_int128_s@PAGEOFF
    ldr  x0, [x0]
    b .Ltfm_done
.Ltfm_int128_u:
    adrp x0, _kcc_type_int128_u@PAGE
    add  x0, x0, _kcc_type_int128_u@PAGEOFF
    ldr  x0, [x0]

.Ltfm_done:
    ldp x29, x30, [sp], #16
    ret
```

□

246 · defined at 245ab, 246 · used in 29d · uses _kcc_type_bool 250b, _kcc_type_char_s 250b, _kcc_type_char_u 250b, _kcc_type_double 250b, _kcc_type_float 250b, _kcc_type_int128_s 250b, _kcc_type_int128_u 250b, _kcc_type_int_s 250b, _kcc_type_int_u 250b, _kcc_type_ldouble 250b, _kcc_type_llong_s 250b, _kcc_type_llong_u 250b, _kcc_type_long_s 250b, _kcc_type_long_u 250b, _kcc_type_short_s 250b, _kcc_type_short_u 250b, _kcc_type_void 250b

24.5. Type Predicates

The type predicates implement the classification rules from §6.2.5 and the compatibility check from §6.2.7. `kcc_type_compatible` first tests pointer identity (same descriptor means trivially compatible), then compares kind constants and, for basic types, the signedness field in `TY_F0`. The helper predicates `kcc_type_is_integer`, `kcc_type_is_arithmetic`, and `kcc_type_is_scalar` classify a type by checking its kind against the relevant set of constants.

⟨ *Types: compatible* 247a ⟩ ≡

```
// -- kcc_type_compatible ---
// int kcc_type_compatible(type_desc *t1,
   type_desc *t2)
//   x0 = first type, x1 = second type
// Returns 1 if types are compatible per §6.2.7,
   else 0.
.text
.globl _kcc_type_compatible
.p2align 2
_kcc_type_compatible:
    // Same pointer → trivially compatible
    cmp x0, x1
    b.eq .Ltc_yes

    // Must have same kind
    ldr x2, [x0, #TY_KIND]
    ldr x3, [x1, #TY_KIND]
    cmp x2, x3
    b.ne .Ltc_no

    // For basic types: same kind + same F0 (
   signedness) → compatible
    cmp x2, #TY_PTR
    b.ge .Ltc_derived

    // Basic type: compare signedness (F0)
    ldr x2, [x0, #TY_F0]
    ldr x3, [x1, #TY_F0]
    cmp x2, x3
    b.ne .Ltc_no

.Ltc_yes:
    mov x0, #1
    ret

.Ltc_derived:
    // For pointers: compare pointee compatibility
    (simplified)
    cmp x2, #TY_PTR
    b.ne .Ltc_no
    // TODO: recursive compatibility check for
   derived types
    mov x0, #1
    ret

.Ltc_no:
    mov x0, #0
    ret
```

□

247a · used in 29d · defines _kcc_type_compatible 276, 426

`kcc_type_is_integer` checks whether a type belongs to the integer category (§6.2.5p17): `_Bool`, `char`, `short`, `int`, `long`, `long long`, and enumerated types. The check is a linear scan of kind constants — seven comparisons at most.

⟨ *Types: is integer* 247b ⟩ ≡

```
// -- kcc_type_is_integer ---
// int kcc_type_is_integer(type_desc *t)
// Returns 1 if t is an integer type (bool, char,
   short, int, long, llong, enum).
.text
.globl _kcc_type_is_integer
.p2align 2
_kcc_type_is_integer:
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_BOOL
    b.eq .Ltii_yes
    cmp x1, #TY_CHAR
    b.eq .Ltii_yes
    cmp x1, #TY_SHORT
    b.eq .Ltii_yes
    cmp x1, #TY_INT
    b.eq .Ltii_yes
    cmp x1, #TY_LONG
    b.eq .Ltii_yes
    cmp x1, #TY_LLONG
    b.eq .Ltii_yes
    cmp x1, #TY_INT128
    b.eq .Ltii_yes
    cmp x1, #TY_ENUM
    b.eq .Ltii_yes
    mov x0, #0
    ret
.Ltii_yes:
    mov x0, #1
    ret
```

□

247b · used in 29d · defines _kcc_type_is_integer 248a, 297b

Arithmetic types are the union of integer types and floating-point types (§6.2.5p18). `kcc_type_is_arithmetic` first delegates to `kcc_type_is_integer`; if that returns false, it checks for `float`, `double`, and `long`

double.

⟨ *Types: is arithmetic* 248a ⟩ ≡

```
// -- kcc_type_is_arithmetic ---
// int kcc_type_is_arithmetic(type_desc *t)
// Returns 1 if t is an arithmetic type (integer
   or floating).
.text
.globl _kcc_type_is_arithmetic
.p2align 2
_kcc_type_is_arithmetic:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                         // save type
   in callee-saved
    bl _kcc_type_is_integer
    cbnz x0, .Ltia_yes

    ldr x1, [x19, #TY_KIND]
    cmp x1, #TY_FLOAT
    b.eq .Ltia_yes
    cmp x1, #TY_DOUBLE
    b.eq .Ltia_yes
    cmp x1, #TY_LDOUBLE
    b.eq .Ltia_yes

    mov x0, #0
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
.Ltia_yes:
    mov x0, #1
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

248a · used in 29d · defines _kcc_type_is_arithmetic 248b, 297b · uses _kcc_type_is_integer 247b

Scalar types comprise all arithmetic types plus pointer types (§6.2.5p21). This predicate is used throughout semantic analysis wherever the standard requires a "scalar expression" — for example, the controlling expression of `if`, `while`, and `for` statements.

⟨ *Types: is scalar* 248b ⟩ ≡

```
// -- kcc_type_is_scalar ---
// int kcc_type_is_scalar(type_desc *t)
// Returns 1 if t is a scalar type (arithmetic or
   pointer).
.text
.globl _kcc_type_is_scalar
.p2align 2
_kcc_type_is_scalar:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0
    bl _kcc_type_is_arithmetic
    cbnz x0, .Ltis_yes

    ldr x1, [x19, #TY_KIND]
    cmp x1, #TY_PTR
    b.eq .Ltis_yes

    mov x0, #0
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
.Ltis_yes:
    mov x0, #1
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

248b · used in 29d · defines _kcc_type_is_scalar never used · uses _kcc_type_is_arithmetic 248a

24.6. Integer Conversion Rank

Every integer type has a *conversion rank* that governs the usual arithmetic conversions (§6.3.1.1). The ranking assigns `_Bool` the lowest rank (1) and `long long` the highest (6), with `enum` sharing the rank of `int`. The function returns the rank as an unsigned integer so that callers can compare two types with a simple greater-than test.

⟨ *Types: rank* 248c ⟩ ≡

```
// -- kcc_type_rank ---
// uint64_t kcc_type_rank(type_desc *t)
// Returns integer conversion rank per §6.3.1.1.
// Higher rank = wider type for UAC purposes.
.text
.globl _kcc_type_rank
.p2align 2
_kcc_type_rank:
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_BOOL
    b.eq .Ltr_bool
    cmp x1, #TY_CHAR
    b.eq .Ltr_char
    cmp x1, #TY_SHORT
    b.eq .Ltr_short
    cmp x1, #TY_INT
    b.eq .Ltr_int
    cmp x1, #TY_LONG
    b.eq .Ltr_long
    cmp x1, #TY_LLONG
    b.eq .Ltr_llong
    cmp x1, #TY_INT128
    b.eq .Ltr_int128
    cmp x1, #TY_ENUM
    b.eq .Ltr_int               // enum has rank of
   int
    // Default: 0
    mov x0, #0
    ret
.Ltr_bool:
    mov x0, #1
    ret
.Ltr_char:
    mov x0, #2
    ret
.Ltr_short:
    mov x0, #3
    ret
.Ltr_int:
    mov x0, #4
    ret
.Ltr_long:
    mov x0, #5
    ret
.Ltr_llong:
    mov x0, #6
    ret
.Ltr_int128:
```

```
    mov x0, #7            // __int128 outranks long
    long
    ret
```

□

248c · used in 29d · defines _kcc_type_rank 295b

24.7. Type Dump

Print a human-readable representation of a type descriptor.

⟨ Types: dump 249a ⟩ ≡

```
// -- kcc_type_dump ---
// void kcc_type_dump(type_desc *t)
//   x0 = type descriptor
// Prints the type as a readable string (e.g., "
   int", "const char *").
.text
.globl _kcc_type_dump
.p2align 2
_kcc_type_dump:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    cbz x0, .Ltd_null
    mov x19, x0

    // Check qualifiers
    ldr x0, [x19, #TY_QUALS]
    cbz x0, .Ltd_kind

    tst x0, #1
    b.eq .Ltd_no_const
    adrp x0, _kcc_ty_str_const@PAGE
    add  x0, x0, _kcc_ty_str_const@PAGEOFF
    bl _kcc_io_puts
.Ltd_no_const:
    ldr x0, [x19, #TY_QUALS]
    tst x0, #2
    b.eq .Ltd_no_volatile
    adrp x0, _kcc_ty_str_volatile@PAGE
    add  x0, x0, _kcc_ty_str_volatile@PAGEOFF
    bl _kcc_io_puts
.Ltd_no_volatile:

.Ltd_kind:
    ldr x0, [x19, #TY_KIND]

    cmp x0, #TY_VOID
    b.eq .Ltd_void
    cmp x0, #TY_BOOL
    b.eq .Ltd_bool
    cmp x0, #TY_CHAR
    b.eq .Ltd_char
    cmp x0, #TY_SHORT
    b.eq .Ltd_short
    cmp x0, #TY_INT
    b.eq .Ltd_int
    cmp x0, #TY_LONG
    b.eq .Ltd_long
    cmp x0, #TY_LLONG
    b.eq .Ltd_llong
    cmp x0, #TY_FLOAT
    b.eq .Ltd_float
    cmp x0, #TY_DOUBLE
    b.eq .Ltd_double
    cmp x0, #TY_LDOUBLE
    b.eq .Ltd_ldouble
    cmp x0, #TY_PTR
    b.eq .Ltd_ptr
    cmp x0, #TY_ARRAY
    b.eq .Ltd_array
    cmp x0, #TY_FUNC
    b.eq .Ltd_func
    cmp x0, #TY_STRUCT
    b.eq .Ltd_struct
    cmp x0, #TY_UNION
    b.eq .Ltd_union
    cmp x0, #TY_ENUM
    b.eq .Ltd_enum

    // Unknown
    adrp x0, _kcc_ty_str_unknown@PAGE
    add  x0, x0, _kcc_ty_str_unknown@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
```

□

249a · defined at 249ab, 250a · used in 29d · defines _kcc_type_dump 250a, 262 · uses _kcc_io_puts 70a, _kcc_ty_str_const 250b, _kcc_ty_str_unknown 250b, _kcc_ty_str_volatile 250b

Basic Type Label Targets Each label below prints the human-readable name for one basic type kind. For integer types (`char` through `long long`), the code checks the `TY_F0` signedness flag stored in the descriptor and selects the appropriate string — for example, `"int"` versus `"unsigned int"`. Non-integer basic types (`void`, `_Bool`, `float`, `double`, `long double`) print a fixed string unconditionally since they have no signed/unsigned variants.

⟨ Types: dump 249b ⟩ + ≡

```
.Ltd_null:
    adrp x0, _kcc_ty_str_null@PAGE
    add  x0, x0, _kcc_ty_str_null@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done

.Ltd_void:
    adrp x0, _kcc_ty_str_void@PAGE
    add  x0, x0, _kcc_ty_str_void@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_bool:
    adrp x0, _kcc_ty_str_bool@PAGE
    add  x0, x0, _kcc_ty_str_bool@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_char:
    ldr x0, [x19, #TY_F0]
    cbz x0, .Ltd_char_u
    adrp x0, _kcc_ty_str_char@PAGE
    add  x0, x0, _kcc_ty_str_char@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_char_u:
    adrp x0, _kcc_ty_str_uchar@PAGE
    add  x0, x0, _kcc_ty_str_uchar@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_short:
    ldr x0, [x19, #TY_F0]
    cbz x0, .Ltd_short_u
    adrp x0, _kcc_ty_str_short@PAGE
    add  x0, x0, _kcc_ty_str_short@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_short_u:
    adrp x0, _kcc_ty_str_ushort@PAGE
    add  x0, x0, _kcc_ty_str_ushort@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_int:
```

```
    ldr x0, [x19, #TY_F0]
    cbz x0, .Ltd_int_u
    adrp x0, _kcc_ty_str_int@PAGE
    add  x0, x0, _kcc_ty_str_int@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_int_u:
    adrp x0, _kcc_ty_str_uint@PAGE
    add  x0, x0, _kcc_ty_str_uint@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_long:
    ldr x0, [x19, #TY_F0]
    cbz x0, .Ltd_long_u
    adrp x0, _kcc_ty_str_long@PAGE
    add  x0, x0, _kcc_ty_str_long@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_long_u:
    adrp x0, _kcc_ty_str_ulong@PAGE
    add  x0, x0, _kcc_ty_str_ulong@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_llong:
    ldr x0, [x19, #TY_F0]
    cbz x0, .Ltd_llong_u
    adrp x0, _kcc_ty_str_llong@PAGE
    add  x0, x0, _kcc_ty_str_llong@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_llong_u:
    adrp x0, _kcc_ty_str_ullong@PAGE
    add  x0, x0, _kcc_ty_str_ullong@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_float:
    adrp x0, _kcc_ty_str_float@PAGE
    add  x0, x0, _kcc_ty_str_float@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_double:
    adrp x0, _kcc_ty_str_double@PAGE
    add  x0, x0, _kcc_ty_str_double@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_ldouble:
    adrp x0, _kcc_ty_str_ldouble@PAGE
    add  x0, x0, _kcc_ty_str_ldouble@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
```

□

249b · defined at 249ab, 250a · used in 29d · uses _kcc_io_puts 70a, _kcc_ty_str_bool 250b, _kcc_ty_str_char 250b, _kcc_ty_str_double 250b, _kcc_ty_str_float 250b, _kcc_ty_str_int 250b, _kcc_ty_str_ldouble 250b, _kcc_ty_str_llong 250b, _kcc_ty_str_long 250b, _kcc_ty_str_null 250b, _kcc_ty_str_short 250b, _kcc_ty_str_uchar 250b, _kcc_ty_str_uint 250b, _kcc_ty_str_ullong 250b, _kcc_ty_str_ulong 250b, _kcc_ty_str_ushort 250b, _kcc_ty_str_void 250b

Derived Type Label Targets Derived types require more than a single string print. Pointer types recursively call `kcc_type_dump` on the pointee stored in `TY_F0`, then append `" *"`. Array types print the element type followed by the length in brackets via `kcc_io_putu`. Function types print the return type followed by `" ()"`. Aggregate types (`struct`, `union`, `enum`) currently print only their tag keyword; full member layout details are deferred to the semantic analysis dump routines.

⟨ *Types: dump* 250a ⟩ +≡

```
.Ltd_ptr:
    // Print pointee type then " *"
    ldr x0, [x19, #TY_F0]
    bl _kcc_type_dump
    adrp x0, _kcc_ty_str_ptr@PAGE
    add  x0, x0, _kcc_ty_str_ptr@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_array:
    ldr x0, [x19, #TY_F0]
    bl _kcc_type_dump
    adrp x0, _kcc_ty_str_arr_open@PAGE
    add  x0, x0, _kcc_ty_str_arr_open@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #TY_F1]
    bl _kcc_io_putu
    adrp x0, _kcc_ty_str_arr_close@PAGE
    add  x0, x0, _kcc_ty_str_arr_close@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_func:
    ldr x0, [x19, #TY_F0]
    bl _kcc_type_dump
    adrp x0, _kcc_ty_str_func@PAGE
    add  x0, x0, _kcc_ty_str_func@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_struct:
    adrp x0, _kcc_ty_str_struct@PAGE
    add  x0, x0, _kcc_ty_str_struct@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_union:
    adrp x0, _kcc_ty_str_union@PAGE
    add  x0, x0, _kcc_ty_str_union@PAGEOFF
    bl _kcc_io_puts
    b .Ltd_done
.Ltd_enum:
    adrp x0, _kcc_ty_str_enum@PAGE
    add  x0, x0, _kcc_ty_str_enum@PAGEOFF
    bl _kcc_io_puts

.Ltd_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

250a · defined at 249ab, 250a · used in 29d · uses _kcc_io_puts 70a, _kcc_io_putu 71a, _kcc_type_dump 249a, _kcc_ty_str_arr_open 250b, _kcc_ty_str_enum 250b, _kcc_ty_str_func 250b, _kcc_ty_str_ptr 250b, _kcc_ty_str_struct 250b, _kcc_ty_str_union 250b

24.8. Type Data

This section collects the global storage for the type subsystem. The singleton pointers (one `.quad` per basic type) are initialised to zero and filled by `kcc_types_init` at startup. The string literals in `__TEXT,__cstring` provide human-readable names used by `kcc_type_dump` for diagnostic output.

⟨ *Types: data* 250b ⟩ ≡

```
// -- Type singleton pointers ---
.data
.globl _kcc_type_void
```

```
_kcc_type_void:       .quad 0
.globl _kcc_type_bool
_kcc_type_bool:       .quad 0
.globl _kcc_type_char_s
_kcc_type_char_s:     .quad 0
.globl _kcc_type_char_u
_kcc_type_char_u:     .quad 0
.globl _kcc_type_short_s
_kcc_type_short_s:    .quad 0
.globl _kcc_type_short_u
_kcc_type_short_u:    .quad 0
.globl _kcc_type_int_s
_kcc_type_int_s:      .quad 0
.globl _kcc_type_int_u
_kcc_type_int_u:      .quad 0
.globl _kcc_type_long_s
_kcc_type_long_s:     .quad 0
.globl _kcc_type_long_u
_kcc_type_long_u:     .quad 0
.globl _kcc_type_llong_s
_kcc_type_llong_s:    .quad 0
.globl _kcc_type_llong_u
_kcc_type_llong_u:    .quad 0
.globl _kcc_type_int128_s
_kcc_type_int128_s:   .quad 0
.globl _kcc_type_int128_u
_kcc_type_int128_u:   .quad 0
.globl _kcc_type_float
_kcc_type_float:      .quad 0
.globl _kcc_type_double
_kcc_type_double:     .quad 0
.globl _kcc_type_ldouble
_kcc_type_ldouble:    .quad 0

// -- Type name strings ---
.section __TEXT,__cstring,cstring_literals
_kcc_ty_str_void:     .asciz "void"
_kcc_ty_str_bool:     .asciz "_Bool"
_kcc_ty_str_char:     .asciz "char"
_kcc_ty_str_uchar:    .asciz "unsigned char"
_kcc_ty_str_short:    .asciz "short"
_kcc_ty_str_ushort:   .asciz "unsigned short"
_kcc_ty_str_int:      .asciz "int"
_kcc_ty_str_uint:     .asciz "unsigned int"
_kcc_ty_str_long:     .asciz "long"
_kcc_ty_str_ulong:    .asciz "unsigned long"
_kcc_ty_str_llong:    .asciz "long long"
_kcc_ty_str_ullong:   .asciz "unsigned long long"
_kcc_ty_str_float:    .asciz "float"
_kcc_ty_str_double:   .asciz "double"
_kcc_ty_str_ldouble:  .asciz "long double"
_kcc_ty_str_ptr:      .asciz " *"
_kcc_ty_str_arr_open: .asciz "["
_kcc_ty_str_arr_close:.asciz "]"
_kcc_ty_str_func:     .asciz " ()"
_kcc_ty_str_struct:   .asciz "struct"
_kcc_ty_str_union:    .asciz "union"
_kcc_ty_str_enum:     .asciz "enum"
_kcc_ty_str_const:    .asciz "const "
_kcc_ty_str_volatile: .asciz "volatile "
_kcc_ty_str_unknown:  .asciz "<unknown>"
_kcc_ty_str_null:     .asciz "<null>"
```

□

250b · used in 29d · defines _kcc_type_bool 239b, 246, _kcc_type_char_s 240, 246, 283, 292b, _kcc_type_char_u 240, 246, _kcc_type_double 241, 246, 283, 295b, 343, 369, 378b, _kcc_type_float 241, 246, 283, 295b, 343, _kcc_type_int128_s 240, 246, _kcc_type_int128_u 240, 246, _kcc_type_int_s 240, 246, 259b, 269, 274, 276, 277, 283, 284, 286, 289, 290, 295a, 312, 314b, 316, 317, 343, 350, 353a, 355, 357, 367, 369, 375, 376, 378b, 383, 384, 387, 393, 398, 400bc, 404, 405, 407, 413, 417b, 426, 450, _kcc_type_int_u 240, 246, 283, 290, _kcc_type_ldouble 241, 246, 295b, _kcc_type_llong_s 240, 246, 283, _kcc_type_llong_u 240, 246, 283, _kcc_type_long_s 240, 246, 283, 284, 369, 378b, _kcc_type_long_u 240, 246, 274, 283, 292b, 347, 359, _kcc_type_short_s 240, 246, _kcc_type_short_u 240, 246, _kcc_type_void 239b, 246, 274, 326, _kcc_ty_str_arr_open 250a, _kcc_ty_str_bool 249b, _kcc_ty_str_char 249b, _kcc_ty_str_const 249a, _kcc_ty_str_double 249b, _kcc_ty_str_enum 250a, _kcc_ty_str_float 249b, _kcc_ty_str_func 250a, _kcc_ty_str_int 249b, _kcc_ty_str_ldouble 249b, _kcc_ty_str_llong 249b, _kcc_ty_str_long 249b, _kcc_ty_str_null 249b, _kcc_ty_str_ptr 250a, _kcc_ty_str_short 249b, _kcc_ty_str_struct 250a, _kcc_ty_str_uchar 249b, _kcc_ty_str_uint 249b, _kcc_ty_str_ullong 249b, _kcc_ty_str_ulong 249b, _kcc_ty_str_union 250a, _kcc_ty_str_unknown 249a, _kcc_ty_str_ushort 249b, _kcc_ty_str_void 249b, _kcc_ty_str_volatile 249a

25. Symbol Table

The C language requires that every identifier be declared before use and that declarations obey scoping rules: a name declared inside a block shadows any outer declaration, and each name space (ordinary identifiers, tags, labels) is resolved independently (§6.2.1). This chapter implements the symbol table that enforces these rules. It defines a 72-byte symbol record carrying kind, type, linkage, scope depth, and declaration line/column (the last so an unused-variable warning can point at the declaration), then builds three scope stacks—one per name space—where each level is a separate hash table. The API provides functions for initialization, scope push/pop, symbol insertion, and lookup by walking the stack from innermost to outermost scope.

25.1. Symbol Record Layout

Each symbol is a 64-byte arena-allocated record:

⟨ *Symtab: constants* 253a ⟩ ≡

```
// -- Symbol record layout ---
.equ SYM_KIND,        0    // SYM_VAR=1, SYM_FUNC
   =2, SYM_TYPEDEF=3, SYM_ENUM_CONST=4
.equ SYM_TYPE,        8    // pointer to type
   descriptor
.equ SYM_NAME,       16    // interned name pointer
.equ SYM_LINKAGE,    24    // 0=none, 1=internal,
   2=external
.equ SYM_VALUE,      32    // enum constant value (
   for SYM_ENUM_CONST)
.equ SYM_DEPTH,      40    // scope depth
.equ SYM_LINE,       48    // declaration line
.equ SYM_FLAGS,      56    // bit 0=defined, bit 1=
   used
.equ SYM_COL,        64    // declaration column (
   for -Wunused-variable)
.equ SYM_SIZE,       72

// -- Symbol kind constants ---
.equ SYM_VAR,         1
.equ SYM_FUNC,        2
.equ SYM_TYPEDEF,     3
.equ SYM_ENUM_CONST,  4
```

□

253a · used in 29e, 30ab

25.2. Initialization

The symbol table is initialised by creating three empty scope stacks—one each for ordinary identifiers, tag names, and labels—implemented as vecs of hash table pointers. After the stacks are allocated, file scope (depth 0) is pushed so that top-level declarations have a scope to land in.

⟨ *Symtab: init* 253b ⟩ ≡

```
// -- kcc_symtab_init ---
// void kcc_symtab_init(void)
// Creates three scope stacks, pushes file scope (
   depth 0).
.text
.globl _kcc_symtab_init
.p2align 2
_kcc_symtab_init:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Create ordinary scope stack (vec of ht
   pointers)
    mov x0, #8                      // element
   size = pointer
    bl _kcc_vec_init
    adrp x1, _kcc_sym_ordinary@PAGE
    add  x1, x1, _kcc_sym_ordinary@PAGEOFF
    str x0, [x1]

    // Create tags scope stack
    mov x0, #8
    bl _kcc_vec_init
    adrp x1, _kcc_sym_tags@PAGE
    add  x1, x1, _kcc_sym_tags@PAGEOFF
    str x0, [x1]

    // Create labels stack
    mov x0, #8
    bl _kcc_vec_init
    adrp x1, _kcc_sym_labels@PAGE
    add  x1, x1, _kcc_sym_labels@PAGEOFF
    str x0, [x1]

    // Initialize depth to 0
    adrp x0, _kcc_sym_depth@PAGE
    add  x0, x0, _kcc_sym_depth@PAGEOFF
    str xzr, [x0]

    // Push file scope
    bl _kcc_symtab_push_scope

    ldp x29, x30, [sp], #16
    ret
```

□

253b · used in 29e · defines _kcc_symtab_init 603, 623 · uses _kcc_symtab_push_scope 253c, _kcc_sym_depth 258, _kcc_sym_labels 258, _kcc_sym_ordinary 258, _kcc_sym_tags 258, _kcc_vec_init 64b

25.3. Scope Management

Entering a compound statement or function body pushes a new scope: a fresh hash table is allocated and appended to both the ordinary and tags scope stacks, and the depth counter is incremented. The ordinary-identifier table uses a capacity of 256 slots—large enough for file-scope programs with many functions, globals, and enum constants, since the hash table does not resize. Leaving the scope pops the top entry from each stack and decrements the depth. This mirrors the C scoping rule that each block introduces a new scope (§6.2.1).

⟨ *Symtab: push scope* 253c ⟩ ≡

```
// -- kcc_symtab_push_scope ---
// void kcc_symtab_push_scope(void)
// Pushes a new hash table onto ordinary and tags
   scope stacks.
.text
```

```
.globl _kcc_symtab_push_scope
.p2align 2
_kcc_symtab_push_scope:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    str x19, [sp, #16]
    // sp+24..31 used as temp storage for vec_push
    element

    // Create new ht for ordinary names
    mov x0, #256                    // capacity (
   must exceed largest scope)
    bl _kcc_ht_init
    // Store ht pointer on stack for vec_push (
   needs pointer-to-element)
    str x0, [sp, #24]

    // Push onto ordinary stack
    adrp x0, _kcc_sym_ordinary@PAGE
    add  x0, x0, _kcc_sym_ordinary@PAGEOFF
    ldr  x0, [x0]
    add x1, sp, #24                 // pointer to
   element on stack
    bl _kcc_vec_push

    // Create new ht for tags
    mov x0, #16
    bl _kcc_ht_init
    str x0, [sp, #24]

    // Push onto tags stack
    adrp x0, _kcc_sym_tags@PAGE
    add  x0, x0, _kcc_sym_tags@PAGEOFF
    ldr  x0, [x0]
    add x1, sp, #24
    bl _kcc_vec_push

    // Increment depth
    adrp x0, _kcc_sym_depth@PAGE
    add  x0, x0, _kcc_sym_depth@PAGEOFF
    ldr  x1, [x0]
    add  x1, x1, #1
    str  x1, [x0]

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

253c · used in 29e · defines _kcc_symtab_push_scope 253b, 264, 272 · uses _kcc_ht_init 61c, _kcc_sym_depth 258, _kcc_sym_ordinary 258, _kcc_sym_tags 258, _kcc_vec_push 65a

kcc_symtab_pop_scope reverses the effect of a push: it decrements the count on both the ordinary and tag scope stacks and decreases the depth counter. The hash tables themselves are not freed — the arena reclaims all memory at program end.

⟨ *Symtab: pop scope* 254a ⟩ ≡

```
// -- kcc_symtab_pop_scope ---
// void kcc_symtab_pop_scope(void)
// Pops from ordinary and tags scope stacks,
   decrements depth.
.text
.globl _kcc_symtab_pop_scope
.p2align 2
_kcc_symtab_pop_scope:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Pop from ordinary stack (decrement count)
    adrp x0, _kcc_sym_ordinary@PAGE
    add  x0, x0, _kcc_sym_ordinary@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [x0, #VEC_COUNT]
    sub  x1, x1, #1
    str  x1, [x0, #VEC_COUNT]

    // Pop from tags stack
    adrp x0, _kcc_sym_tags@PAGE
    add  x0, x0, _kcc_sym_tags@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [x0, #VEC_COUNT]
    sub  x1, x1, #1
    str  x1, [x0, #VEC_COUNT]

    // Decrement depth
    adrp x0, _kcc_sym_depth@PAGE
    add  x0, x0, _kcc_sym_depth@PAGEOFF
    ldr  x1, [x0]
    sub  x1, x1, #1
    str  x1, [x0]

    ldp x29, x30, [sp], #16
    ret
```

□

254a · used in 29e · defines _kcc_symtab_pop_scope 264, 272 · uses _kcc_sym_depth 258, _kcc_sym_ordinary 258, _kcc_sym_tags 258

25.4. Ordinary Name Space

The ordinary name space holds variables, functions, typedefs, and enumeration constants (§6.2.3). kcc_symtab_declare arena-allocates a 64-byte symbol record, fills in kind, type, name, depth, and line number, then inserts it into the hash table at the top of the ordinary scope stack. kcc_symtab_lookup walks the stack top-down, returning the first matching symbol—this implements C's inner-scope-shadows-outer rule.

⟨ *Symtab: declare* 254b ⟩ ≡

```
// -- kcc_symtab_declare ---
// sym *kcc_symtab_declare(char *name, uint64_t
   len, uint64_t kind, type_desc *type)
//   x0 = name, x1 = len, x2 = kind, x3 = type
   descriptor
// Creates SYM record, inserts into current scope.
// Returns sym pointer.
.text
.globl _kcc_symtab_declare
.p2align 2
_kcc_symtab_declare:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0                     // name
    mov x20, x1                     // len
    mov x21, x2                     // kind
    mov x22, x3                     // type

    // Allocate symbol record
    mov x0, #SYM_SIZE
    bl _kcc_arena_alloc
    mov x23, x0                     // x23 = sym
```

```
    // Fill symbol fields
    str x21, [x23, #SYM_KIND]
    str x22, [x23, #SYM_TYPE]
    str x19, [x23, #SYM_NAME]
    str xzr, [x23, #SYM_LINKAGE]
    str xzr, [x23, #SYM_VALUE]

    // Store depth
    adrp x0, _kcc_sym_depth@PAGE
    add  x0, x0, _kcc_sym_depth@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x23, #SYM_DEPTH]

    // Store line from current token
    adrp x0, _kcc_tok_line@PAGE
    add  x0, x0, _kcc_tok_line@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x23, #SYM_LINE]

    // flags = defined
    mov x0, #1
    str x0, [x23, #SYM_FLAGS]
    // Column is unknown at declare time (runs at
  sema, so the token globals
    // sit at EOF); the caller overwrites SYM_LINE
  /SYM_COL from the decl node.
    str xzr, [x23, #SYM_COL]

    // Get current scope ht (top of ordinary stack
  )
    adrp x0, _kcc_sym_ordinary@PAGE
    add  x0, x0, _kcc_sym_ordinary@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [x0, #VEC_COUNT]
    sub  x1, x1, #1                 // top index
    bl _kcc_vec_get
    ldr x24, [x0]                   // x24 =
  current scope ht

    // Insert into ht: key=name, len=len, val=sym
    mov x0, x24
    mov x1, x19                     // name
    mov x2, x20                     // len
    mov x3, x23                     // sym pointer
    as value
    bl _kcc_ht_put

    mov x0, x23                     // return sym
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

254b · used in 29e · defines _kcc_symtab_declare 264, 269 · uses _kcc_arena_alloc 58a, _kcc_ht_put 63, _kcc_sym_depth 258, _kcc_sym_ordinary 258, _kcc_tok_line 104b, _kcc_vec_get 65b

`kcc_symtab_lookup` searches the ordinary scope stack from top to bottom, returning the first symbol that matches the given name. This top-down walk implements C's scoping rule: an inner declaration shadows any outer declaration of the same name. If no scope contains the name, the function returns zero. Two narrower accessors round out the interface: `kcc_symtab_lookup_current` consults only the innermost scope (so a redefinition check can distinguish a same-scope duplicate from a legal shadow), and `kcc_symtab_current_ht` hands back that innermost scope's hash table whole, so semantic analysis can walk every symbol declared in a block as the block is left—the basis for the unused-variable warning.

⟨ *Symtab: lookup* 255 ⟩ ≡

```
// -- kcc_symtab_lookup ---
// sym *kcc_symtab_lookup(char *name, uint64_t len
   )
//   x0 = name, x1 = len
// Searches ordinary scopes top-down.
// Returns sym pointer or 0 if not found.
.text
.globl _kcc_symtab_lookup
.p2align 2
_kcc_symtab_lookup:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0                     // name
    mov x20, x1                     // len

    // Get ordinary stack
    adrp x0, _kcc_sym_ordinary@PAGE
    add  x0, x0, _kcc_sym_ordinary@PAGEOFF
    ldr  x0, [x0]
    mov x21, x0                     // x21 = vec
    ldr x22, [x21, #VEC_COUNT]      // x22 = count
    (search top-down)

.Lsym_lookup_loop:
    cbz x22, .Lsym_lookup_notfound
    sub x22, x22, #1

    // Get ht at index x22
    mov x0, x21
    mov x1, x22
    bl _kcc_vec_get
    ldr x0, [x0]                    // ht pointer

    // Lookup in this ht
    mov x1, x19                     // name
    mov x2, x20                     // len
    bl _kcc_ht_get
    cbnz x0, .Lsym_lookup_found

    b .Lsym_lookup_loop

.Lsym_lookup_found:
    // x0 = sym pointer (from ht_get value)
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lsym_lookup_notfound:
    mov x0, #0
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_symtab_lookup_current ---
// sym *kcc_symtab_lookup_current(char *name,
   uint64_t len)
// Searches ONLY the innermost (current) scope, so
    a redefinition check can
// tell a same-scope duplicate from a legal shadow
    of an outer declaration.
```

```
.text
.globl _kcc_symtab_lookup_current
.p2align 2
_kcc_symtab_lookup_current:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                     // name
    mov x20, x1                     // len
    adrp x0, _kcc_sym_ordinary@PAGE
    add  x0, x0, _kcc_sym_ordinary@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [x0, #VEC_COUNT]
    sub  x1, x1, #1                 // top index
    bl _kcc_vec_get
    ldr x0, [x0]                    // current
   scope ht
    mov x1, x19
    mov x2, x20
    bl _kcc_ht_get                  // sym or 0
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_symtab_current_ht ---
// void *kcc_symtab_current_ht(void)
// Returns the hash table of the innermost (
   current) ordinary scope, so a
// caller can iterate the symbols declared
   directly in that scope (used by
// the unused-variable warning at scope exit).
.text
.globl _kcc_symtab_current_ht
.p2align 2
_kcc_symtab_current_ht:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    adrp x0, _kcc_sym_ordinary@PAGE
    add  x0, x0, _kcc_sym_ordinary@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [x0, #VEC_COUNT]
    sub  x1, x1, #1                 // top index
    bl _kcc_vec_get
    ldr x0, [x0]                    // current
   scope ht
    ldp x29, x30, [sp], #16
    ret
```

□

255 · used in 29e · defines _kcc_symtab_current_ht 277, _kcc_symtab_lookup 259b, 277, 292b, 308, 350, 355, 367, 426, _kcc_symtab_lookup_current 277 · uses _kcc_ht_get 62, _kcc_sym_ordinary 258, _kcc_vec_get 65b

25.5. Tag Name Space

Tags for `struct`, `union`, and `enum` live in a separate name space from ordinary identifiers (§6.2.3). The declare and lookup functions mirror the ordinary name space interface but operate on the tags scope stack instead. This separation allows a program to use the same identifier as both a tag and an ordinary name without conflict.

⟨ *Symtab: declare tag* 256a ⟩ ≡

```
// -- kcc_symtab_declare_tag ---
// sym *kcc_symtab_declare_tag(char *name,
   uint64_t len, type_desc *type)
//   x0 = tag name, x1 = len, x2 = type descriptor
// Inserts into tags scope stack.
.text
.globl _kcc_symtab_declare_tag
.p2align 2
_kcc_symtab_declare_tag:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0                     // name
    mov x20, x1                     // len
    mov x21, x2                     // type

    // Allocate symbol record
    mov x0, #SYM_SIZE
    bl _kcc_arena_alloc
    mov x22, x0                     // x22 = sym

    // Fill
    mov x1, #SYM_VAR                // tags don't
   need a specific kind
    str x1, [x22, #SYM_KIND]
    str x21, [x22, #SYM_TYPE]
    str x19, [x22, #SYM_NAME]
    str xzr, [x22, #SYM_LINKAGE]
    str xzr, [x22, #SYM_VALUE]

    adrp x0, _kcc_sym_depth@PAGE
    add  x0, x0, _kcc_sym_depth@PAGEOFF
    ldr  x0, [x0]
    str  x0, [x22, #SYM_DEPTH]
    str xzr, [x22, #SYM_LINE]
    mov x0, #1
    str x0, [x22, #SYM_FLAGS]

    // Get current scope ht (top of tags stack)
    adrp x0, _kcc_sym_tags@PAGE
    add  x0, x0, _kcc_sym_tags@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [x0, #VEC_COUNT]
    sub  x1, x1, #1
    bl _kcc_vec_get
    ldr x0, [x0]                    // ht

    mov x1, x19
    mov x2, x20
    mov x3, x22
    bl _kcc_ht_put

    mov x0, x22
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

256a · used in 29e · defines _kcc_symtab_declare_tag 269 · uses _kcc_arena_alloc 58a, _kcc_ht_put 63, _kcc_sym_depth 258, _kcc_sym_tags 258, _kcc_vec_get 65b

`kcc_symtab_lookup_tag` mirrors the ordinary lookup but walks the tag scope stack. This is used when the parser encounters `struct foo`, `union bar`, or `enum baz` to find the corresponding type definition.

⟨ *Symtab: lookup tag* 256b ⟩ ≡

```
// -- kcc_symtab_lookup_tag ---
// sym *kcc_symtab_lookup_tag(char *name, uint64_t
   len)
//   x0 = tag name, x1 = len
// Searches tags scopes top-down.
```

```
.text
.globl _kcc_symtab_lookup_tag
.p2align 2
_kcc_symtab_lookup_tag:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0
    mov x20, x1

    adrp x0, _kcc_sym_tags@PAGE
    add  x0, x0, _kcc_sym_tags@PAGEOFF
    ldr  x0, [x0]
    mov x21, x0
    ldr x22, [x21, #VEC_COUNT]

.Lsym_tag_loop:
    cbz x22, .Lsym_tag_notfound
    sub x22, x22, #1

    mov x0, x21
    mov x1, x22
    bl _kcc_vec_get
    ldr x0, [x0]

    mov x1, x19
    mov x2, x20
    bl _kcc_ht_get
    cbnz x0, .Lsym_tag_found

    b .Lsym_tag_loop

.Lsym_tag_found:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lsym_tag_notfound:
    mov x0, #0
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

256b · used in 29e · defines _kcc_symtab_lookup_tag 259b, 269 · uses _kcc_ht_get 62, _kcc_sym_tags 258, _kcc_vec_get 65b

25.6. Label Scope

Labels are function-scoped: one hash table per function.

⟨ *Symtab: label scope* 257a ⟩ ≡

```
// -- kcc_symtab_label_begin ---
// void kcc_symtab_label_begin(void)
// Pushes a new label hash table for a function
   body.
.text
.globl _kcc_symtab_label_begin
.p2align 2
_kcc_symtab_label_begin:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    // sp+16..23 used as temp for vec_push element

    mov x0, #16
    bl _kcc_ht_init
    str x0, [sp, #16]              // store ht
   pointer for vec_push
    adrp x0, _kcc_sym_labels@PAGE
    add  x0, x0, _kcc_sym_labels@PAGEOFF
    ldr  x0, [x0]
    add x1, sp, #16                // pointer to
   element
    bl _kcc_vec_push

    ldp x29, x30, [sp], #32
    ret

// -- kcc_symtab_label_end ---
// void kcc_symtab_label_end(void)
// Pops the label hash table after function body.
.globl _kcc_symtab_label_end
.p2align 2
_kcc_symtab_label_end:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _kcc_sym_labels@PAGE
    add  x0, x0, _kcc_sym_labels@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [x0, #VEC_COUNT]
    sub  x1, x1, #1
    str  x1, [x0, #VEC_COUNT]

    ldp x29, x30, [sp], #16
    ret
```

□

257a · defined at 257ab · used in 29e · defines _kcc_symtab_label_begin 264, _kcc_symtab_label_end 264 · uses _kcc_ht_init 61c, _kcc_sym_labels 258, _kcc_vec_push 65a

Name Operations. With the scope established, the compiler needs to record each label it encounters and later verify that every goto target was actually defined. _kcc_symtab_label_add inserts a label name into the top hash table with a sentinel value of 1, while _kcc_symtab_label_lookup probes the same table and returns a boolean indicating whether the label exists (§6.8.1).

⟨ *Symtab: label scope* 257b ⟩ + ≡

```
// -- kcc_symtab_label_add ---
// void kcc_symtab_label_add(char *name, uint64_t
   len)
.globl _kcc_symtab_label_add
.p2align 2
_kcc_symtab_label_add:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    mov x19, x0
    mov x20, x1

    // Get top label ht
    adrp x0, _kcc_sym_labels@PAGE
    add  x0, x0, _kcc_sym_labels@PAGEOFF
    ldr  x0, [x0]
    ldr  x1, [x0, #VEC_COUNT]
    sub  x1, x1, #1
    bl _kcc_vec_get
    ldr x0, [x0]                   // ht

    mov x1, x19
    mov x2, x20
```

```
    mov x3, #1                          // value = 1 (
   defined)
    bl _kcc_ht_put

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_symtab_label_lookup ---
// int kcc_symtab_label_lookup(char *name,
   uint64_t len)
// Returns 1 if label is defined, 0 otherwise.
.globl _kcc_symtab_label_lookup
.p2align 2
_kcc_symtab_label_lookup:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    mov x2, x1                          // len
    mov x1, x0                          // name

    // Get top label ht
    adrp x0, _kcc_sym_labels@PAGE
    add  x0, x0, _kcc_sym_labels@PAGEOFF
    ldr  x0, [x0]
    ldr  x3, [x0, #VEC_COUNT]
    cbz x3, .Lsym_label_no
    sub  x3, x3, #1
    bl _kcc_vec_get
    ldr x0, [x0]

    // x1=name already, x2=len already
    bl _kcc_ht_get
    cbnz x0, .Lsym_label_yes

.Lsym_label_no:
    mov x0, #0
    ldp x29, x30, [sp], #16
    ret
.Lsym_label_yes:
    mov x0, #1
    ldp x29, x30, [sp], #16
    ret
```

□

257b · defined at 257ab · used in 29e · defines _kcc_symtab_label_add 273, _kcc_symtab_label_lookup never used · uses _kcc_ht_get 62, _kcc_ht_put 63, _kcc_sym_labels 258, _kcc_vec_get 65b

25.7. Symbol Table Data

The global data for the symbol table consists of three vec pointers (one per scope stack) and the current scope depth counter. All four are zero-initialised and populated by `kcc_symtab_init` at startup.

⟨ *Symtab: data* 258 ⟩ ≡

```
// -- Symbol table globals ---
.data
.globl _kcc_sym_ordinary
_kcc_sym_ordinary:    .quad 0       // vec* for
   ordinary scope stack
.globl _kcc_sym_tags
_kcc_sym_tags:        .quad 0       // vec* for
   tag scope stack
.globl _kcc_sym_labels
_kcc_sym_labels:      .quad 0       // vec* for
   label scope stack
.globl _kcc_sym_depth
_kcc_sym_depth:       .quad 0       // current
   scope depth
```

□

258 · used in 29e · defines _kcc_sym_depth 253bc, 254ab, 256a, 269, 277, _kcc_sym_labels 253b, 257ab, _kcc_sym_ordinary 253bc, 254ab, 255, _kcc_sym_tags 253bc, 254a, 256ab

26. Semantic Analysis

Parsing produces a syntactically valid AST, but many constraints in the C language are semantic rather than syntactic: an operand must have arithmetic type for +, a function call must match its prototype, and every identifier must resolve to a visible declaration. This chapter implements the semantic analysis pass that enforces these rules. It walks the AST top-down, resolves declaration types using the type system from Chapter 24, populates the symbol table from Chapter 25, and annotates every expression node with its resolved type. The chapter is organized around three domains: declaration checking, statement checking, and expression checking.

26.1. Entry Point

The top-level entry point receives the `AST_TRANS_UNIT` node and walks its linked list of declarations (chained via `AST_F4`). For each declaration it delegates to `_kcc_sema_check_decl`, which resolves types and populates the symbol table.

⟨ *Sema: check* 259a ⟩ ≡

```
// -- kcc_sema_check ---
// void kcc_sema_check(node *trans_unit)
//   x0 = AST_TRANS_UNIT node
// Walks all top-level declarations.
.text
.globl _kcc_sema_check
.p2align 2
_kcc_sema_check:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Get first declaration
    ldr x19, [x0, #AST_F0]          // first decl

.Lsc_loop:
    cbz x19, .Lsc_done

    mov x0, x19
    bl _kcc_sema_check_decl

    // Next sibling
    ldr x19, [x19, #AST_F4]
    b .Lsc_loop

.Lsc_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

259a · used in 30a · defines _kcc_sema_check 603, 623 · uses _kcc_sema_check_decl 264

26.2. Type Resolution

Type resolution converts parser-produced AST type nodes into the canonical type descriptors from Chapter 24. The dispatcher handles nine type node kinds: built-in types use the specifier bitmask to select a pre-allocated singleton; pointer, array, and function types recurse into their children and call the corresponding type constructor. Struct and union nodes with a body (field list) create a new type descriptor; those without a body are forward references and look up the tag in the symbol table to find the original definition. Typedef names are resolved via symbol table lookup, with two built-in exceptions that no declaration introduces: the name `va_list` and its GNU spelling `__builtin_va_list` both resolve to the opaque eight-byte pointer the variadic builtins use (Section 29.9.13), so a freestanding `<stdarg.h>` needs no header support. Qualified types are currently passed through to the base type. Any unrecognised or null type node falls back to `int`.

⟨ *Sema: resolve type* 259b ⟩ ≡

```
// -- kcc_sema_resolve_type ---
// type_desc *kcc_sema_resolve_type(node *
   type_node)
//   x0 = AST type node (AST_TYPE_BUILTIN,
   AST_TYPE_PTR, etc.)
// Returns type descriptor pointer.
.text
.globl _kcc_sema_resolve_type
.p2align 2
_kcc_sema_resolve_type:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    cbz x0, .Lsrt_null
    mov x19, x0

    ldr x0, [x19, #AST_TAG]

    cmp x0, #AST_TYPE_BUILTIN
    b.eq .Lsrt_builtin
    cmp x0, #AST_TYPE_PTR
    b.eq .Lsrt_ptr
    cmp x0, #AST_TYPE_ARRAY
    b.eq .Lsrt_array
    cmp x0, #AST_TYPE_FUNC
    b.eq .Lsrt_func
    cmp x0, #AST_TYPE_STRUCT
    b.eq .Lsrt_struct
    cmp x0, #AST_TYPE_UNION
    b.eq .Lsrt_union
    cmp x0, #AST_TYPE_ENUM
    b.eq .Lsrt_enum
    cmp x0, #AST_TYPE_NAME
    b.eq .Lsrt_name
    cmp x0, #AST_TYPE_QUALS
    b.eq .Lsrt_quals
    cmp x0, #AST_TYPE_TYPEOF
    b.eq .Lsrt_typeof

    // Unknown type node → return int as fallback
    b .Lsrt_fallback

.Lsrt_typeof:
    // GNU typeof: F1=1 means the operand is a
   type-name (resolve it directly);
    // F1=0 means it is an expression, whose type
   is obtained by type-checking it
```

```
    // in this unevaluated context (sema does not
    emit code, so no side effects).
    //
    // The result is memoised in AST_F3.  This
    matters when the operand is an
    // expression naming a local: the type is
    resolved twice --- once in the sema
    // pass (symbol table populated, so the name
    resolves) and again later when
    // the IR builder re-resolves the declaration'
    s type (by which point the
    // block's scope has been popped and the name
    would appear undeclared).
    // Caching the sema-pass result makes the
    second resolution a plain load, so
    // `__typeof__(local)` keeps the operand's
    true type instead of falling back
    // to int and emitting a spurious ``undeclared
    identifier'' diagnostic.
    ldr x2, [x19, #AST_F3]      // cached resolved
    type?
    cbnz x2, .Lsrt_typeof_cached
    ldr x1, [x19, #AST_F1]      // is_type
    ldr x0, [x19, #AST_F0]      // operand
    cbz x0, .Lsrt_fallback
    cbz x1, .Lsrt_typeof_expr
    bl _kcc_sema_resolve_type   // type-name →
    resolve
    cbz x0, .Lsrt_fallback
    str x0, [x19, #AST_F3]      // memoise
    b .Lsrt_done
.Lsrt_typeof_expr:
    bl _kcc_sema_expr           // expression →
    its resolved type
    cbz x0, .Lsrt_fallback
    str x0, [x19, #AST_F3]      // memoise
    b .Lsrt_done
.Lsrt_typeof_cached:
    mov x0, x2
    b .Lsrt_done

.Lsrt_builtin:
    // F0 = specifier bitmask
    ldr x0, [x19, #AST_F0]
    bl _kcc_type_from_specmask
    cbz x0, .Lsrt_fallback
    b .Lsrt_done

.Lsrt_ptr:
    // F0 = pointee type node
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    bl _kcc_type_ptr
    b .Lsrt_done

.Lsrt_array:
    // F0 = element type node, F1 = size expr (or
    0)
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    mov x20, x0                 // element
    type

    // Array length: fold the size as an integer
    constant expression
    // (\stdref{6.7.6.2}).  A bare literal, but
    equally `31 + 1` or
    // `sizeof(int) * CHAR_BIT`, must reduce to
    its value; only a genuinely
    // absent size (`a[]`) is an incomplete array.
     A size that is NOT a
    // constant expression is a variable-length
    array (`int a[n]`), which kcc
    // does not support: it is rejected with a
    diagnostic rather than folded to
    // a bogus fixed length (which would silently
    under-size the storage).  See
    // Appendix~\ref{ch:limitations}.
    ldr x1, [x19, #AST_F1]
    cbz x1, .Lsrt_array_incomplete
    mov x0, x1
    bl _kcc_expr_is_ice
    cbnz x0, .Lsrt_array_const
    // VLA: emit a diagnostic and treat the array
    as incomplete (the error count
    // fails the compile).
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    adrp x2, _kcc_sema_msg_vla@PAGE
    add  x2, x2, _kcc_sema_msg_vla@PAGEOFF
    bl _kcc_diag_error
    b .Lsrt_array_incomplete
.Lsrt_array_const:
    ldr x0, [x19, #AST_F1]
    bl _kcc_eval_const_int
    mov x1, x0                  // folded
    length
    b .Lsrt_array_make

.Lsrt_array_incomplete:
    mov x1, #0                  // incomplete
    array

.Lsrt_array_make:
    mov x0, x20
    bl _kcc_type_array
    b .Lsrt_done

.Lsrt_func:
    // AST_TYPE_FUNC: F0 = return type node, F1 =
    params (linked list),
    // F2 = variadic flag (0/1).  This node is
    produced by a grouped declarator
    // `(name)(params, ...)` and by function
    pointers---note the variadic flag
    // lives in F2 here, NOT in bit 0 of the
    params pointer (that packing is the
    // AST_FUNC_DECL convention).
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    mov x20, x0                 // return type

    // Carry the parameter declaration list as the
    function type's param vec, so
    // the call site can convert and classify each
    argument by its parameter's
    // type---e.g. an integer argument to a \
    texttt{double} parameter needs an
    // int-to-double conversion and an SSE-class
    register.
    ldr x1, [x19, #AST_F1]      // params list
    bic x1, x1, #1              // defensive:
    clear bit 0
    mov x0, x20                 // return type
    bl _kcc_type_func
    mov x20, x0                 // reuse x20 (
    return type consumed) = TY_FUNC

    // A variadic prototype records its fixed-
    parameter count in TY_QUALS (zero
    // means non-variadic); the call lowering
    reads it to place the variadic
    // tail on the stack.  Without this, EVERY
    variadic call through a grouped
    // declarator---which is how Lua declares its
    whole API, e.g.
```

```
    // `const char *(lua_pushfstring)(lua_State *,
    const char *, ...)`---was
    // treated as non-variadic and passed its
   arguments in registers, so the
    // callee read the stack and got garbage.  (
   Only x19/x20 are callee-saved
    // here, so the fixed-param walk uses caller-
   saved temporaries and makes no
    // calls.)
    ldr x2, [x19, #AST_F2]          // variadic
   flag
    cbz x2, .Lsrt_func_done
    ldr x2, [x19, #AST_F1]          // params list
    (clean)
    mov x3, #0                      // nfixed
.Lsrt_func_nfixed:
    cbz x2, .Lsrt_func_setq
    add x3, x3, #1
    ldr x2, [x2, #AST_F4]
    b .Lsrt_func_nfixed
.Lsrt_func_setq:
    str x3, [x20, #TY_QUALS]
.Lsrt_func_done:
    mov x0, x20
    b .Lsrt_done

.Lsrt_struct:
    // F0 = struct_decl AST node.  A bodyless
   reference looks up the tag; a body
    // resolves to a descriptor for that
   definition.
    ldr x20, [x19, #AST_F0]         //
   AST_STRUCT_DECL
    cbz x20, .Lsrt_fallback
    ldr x0, [x20, #AST_F1]          // fields
    cbz x0, .Lsrt_struct_fwd        // no body →
   forward-ref tag lookup
    // Body present.  If it is NAMED and the tag
   already resolves to THIS same
    // decl, reuse that descriptor rather than
   minting a second one.  A duplicate
    // descriptor shares the field AST nodes, and
   laying it out again re-runs
    // bit-field packing over an AST_F2 the first
   layout already replaced with a
    // packed (bit_offset<<16 | width) integer --
   dereferencing that small value
    // as a width-expression pointer and crashing.
     SQLite's `struct sqlite3`
    // nests a named `struct sqlite3InitInfo
   {...}` bit-field member exactly so.
    ldr x0, [x20, #AST_F0]          // tag name_ptr
    cbz x0, .Lsrt_struct_new        // anonymous
   body → must create fresh
    bl _kcc_strlen
    mov x1, x0                      // len
    ldr x0, [x20, #AST_F0]          // name
    bl _kcc_symtab_lookup_tag
    cbz x0, .Lsrt_struct_new        // tag not
   registered yet → create it
    ldr x1, [x0, #SYM_TYPE]
    cbz x1, .Lsrt_struct_new
    ldr x2, [x1, #TY_F0]            // registered
   descriptor's decl node
    cmp x2, x20                     // same
   definition as this one?
    b.ne .Lsrt_struct_new           // shadowing /
   different → fresh
    mov x0, x1                      // reuse the
   registered, already-laid-out desc
    b .Lsrt_done
.Lsrt_struct_fwd:
    // No fields: look up tag
    ldr x0, [x20, #AST_F0]          // tag name_ptr
    cbz x0, .Lsrt_struct_new
    mov x20, x0
    bl _kcc_strlen
    mov x1, x0
    mov x0, x20
    bl _kcc_symtab_lookup_tag
    cbz x0, .Lsrt_fallback
    ldr x0, [x0, #SYM_TYPE]         // sym → type
   descriptor
    b .Lsrt_done
.Lsrt_struct_new:
    mov x0, x20
    bl _kcc_type_struct
    b .Lsrt_done

.Lsrt_union:
    // Same logic as struct: a named body already
   registered as this same decl is
    // reused (avoiding a duplicate descriptor
   that would re-pack bit-fields); a
    // bodyless reference looks up the tag.
    ldr x20, [x19, #AST_F0]
    cbz x20, .Lsrt_fallback
    ldr x0, [x20, #AST_F1]
    cbz x0, .Lsrt_union_fwd         // no body →
   forward-ref tag lookup
    ldr x0, [x20, #AST_F0]          // tag name_ptr
    cbz x0, .Lsrt_union_new         // anonymous
   body → fresh
    bl _kcc_strlen
    mov x1, x0
    ldr x0, [x20, #AST_F0]
    bl _kcc_symtab_lookup_tag
    cbz x0, .Lsrt_union_new
    ldr x1, [x0, #SYM_TYPE]
    cbz x1, .Lsrt_union_new
    ldr x2, [x1, #TY_F0]            // registered
   descriptor's decl node
    cmp x2, x20
    b.ne .Lsrt_union_new            // shadowing /
   different → fresh
    mov x0, x1                      // reuse
   registered descriptor
    b .Lsrt_done
.Lsrt_union_fwd:
    ldr x0, [x20, #AST_F0]
    cbz x0, .Lsrt_union_new
    mov x20, x0
    bl _kcc_strlen
    mov x1, x0
    mov x0, x20
    bl _kcc_symtab_lookup_tag
    cbz x0, .Lsrt_fallback
    ldr x0, [x0, #SYM_TYPE]         // sym → type
   descriptor
    b .Lsrt_done
.Lsrt_union_new:
    mov x0, x20
    bl _kcc_type_union
    b .Lsrt_done

.Lsrt_enum:
    ldr x0, [x19, #AST_F0]
    bl _kcc_type_enum
    b .Lsrt_done

.Lsrt_name:
    // Typedef name: lookup in symbol table
    ldr x20, [x19, #AST_F0]         // interned
   name ptr
    cbz x20, .Lsrt_fallback
    mov x0, x20
    bl _kcc_strlen
    mov x1, x0                      // len
```

```
    mov x0, x20                     // name
    bl _kcc_symtab_lookup
    cbz x0, .Lsrt_name_builtin

    // Found symbol: return its type
    ldr x0, [x0, #SYM_TYPE]
    b .Lsrt_done

.Lsrt_name_builtin:
    // Built-in typedef va_list → an 8-byte
   pointer.  (No <stdarg.h>: the
    // variadic builtins below treat it as an
   opaque cursor into the stack.)
    // The GNU spelling __builtin_va_list (kernel
   <stdarg.h>) is an alias.
    mov x0, x20
    adrp x1, _kcc_sema_str_va_list@PAGE
    add  x1, x1, _kcc_sema_str_va_list@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lsrt_va_list_ptr
    mov x0, x20
    adrp x1, _kcc_sema_str_builtin_va_list@PAGE
    add  x1, x1,
   _kcc_sema_str_builtin_va_list@PAGEOFF
    bl _kcc_strcmp
    cbnz x0, .Lsrt_fallback
.Lsrt_va_list_ptr:
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr x0, [x0]
    bl _kcc_type_ptr
    b .Lsrt_done

.Lsrt_quals:
    // F0 = base type node, F1 = qualifier flags.
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    // Only volatile is honoured (it must not be
   optimized away); const and
    // restrict are still dropped, keeping the
   shared descriptors---and the code
    // that compares types by identity---untouched
   .  A volatile object gets a
    // fresh copy of the descriptor with the
   volatile bit set, so the qualifier
    // reaches the alloca and its loads and stores
    without disturbing the
    // canonical unqualified type.
    ldr x1, [x19, #AST_F1]
    and x1, x1, #2
    cbz x1, .Lsrt_done
    mov x20, x0                     // base
   descriptor (callee-saved)
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    ldr x1, [x20, #0]
    str x1, [x0, #0]
    ldr x1, [x20, #8]
    str x1, [x0, #8]
    ldr x1, [x20, #16]
    str x1, [x0, #16]
    ldr x1, [x20, #24]
    str x1, [x0, #24]
    ldr x1, [x20, #32]
    str x1, [x0, #32]
    ldr x1, [x20, #40]
    str x1, [x0, #40]
    ldr x1, [x0, #TY_QUALS]
    orr x1, x1, #2                  // mark volatile
    str x1, [x0, #TY_QUALS]
    b .Lsrt_done

.Lsrt_null:
.Lsrt_fallback:
    // Default to int
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]

.Lsrt_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

.section __TEXT,__cstring,cstring_literals
_kcc_sema_msg_vla:
    .asciz "variable-length arrays are not
   supported"
.text
```

□

259b · used in 30a · defines _kcc_sema_resolve_type 262, 264, 267, 269, 274, 276, 277, 290, 292ab, 308, 312, 314b, 317, 320, 340, 343, 347, 359, 393, 407, 413, 417b, 426, 465, 469, 562, 563, 564 · uses _kcc_arena_alloc 58a, _kcc_diag_error 76a, _kcc_eval_const_int 426, _kcc_expr_is_ice 426, _kcc_sema_expr 274, _kcc_sema_str_builtin_va_list 263, _kcc_sema_str_va_list 263, _kcc_symtab_lookup 255, _kcc_symtab_lookup_tag 256b, _kcc_type_array 243a, _kcc_type_enum 244, _kcc_type_from_specmask 245a, _kcc_type_func 243b, _kcc_type_int_s 250b, _kcc_type_ptr 242a, _kcc_type_struct 243c, _kcc_type_union 243c

26.3. Dump Types

The type dumper implements the `--dump-types` flag. It walks the top-level declaration list and prints each variable, function, or typedef declaration together with its resolved type descriptor. The output format is:

```
var x: int
func main: int
typedef size_t = unsigned long
```

⟨ *Sema: dump types* 262 ⟩ ≡

```
// -- kcc_sema_dump_types ---
// void kcc_sema_dump_types(node *trans_unit)
//   x0 = AST_TRANS_UNIT after semantic analysis
// Prints each declaration with resolved types.
.text
.globl _kcc_sema_dump_types
.p2align 2
_kcc_sema_dump_types:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    ldr x19, [x0, #AST_F0]          // first decl

.Lsdt_loop:
    cbz x19, .Lsdt_done

    // Print declaration info
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_VAR_DECL
    b.eq .Lsdt_var
    cmp x0, #AST_FUNC_DECL
    b.eq .Lsdt_func
    cmp x0, #AST_TYPEDEF_DECL
    b.eq .Lsdt_typedef
    b .Lsdt_next
```

```
.Lsdt_var:
    adrp x0, _kcc_sema_str_var@PAGE
    add  x0, x0, _kcc_sema_str_var@PAGEOFF
    bl _kcc_io_puts

    // Print name
    ldr x0, [x19, #AST_F1]
    cbz x0, .Lsdt_var_type
    bl _kcc_io_puts

.Lsdt_var_type:
    adrp x0, _kcc_sema_str_colon@PAGE
    add  x0, x0, _kcc_sema_str_colon@PAGEOFF
    bl _kcc_io_puts

    // Resolve and print type
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    bl _kcc_type_dump

    adrp x0, _kcc_sema_str_nl@PAGE
    add  x0, x0, _kcc_sema_str_nl@PAGEOFF
    bl _kcc_io_puts
    b .Lsdt_next

.Lsdt_func:
    adrp x0, _kcc_sema_str_func@PAGE
    add  x0, x0, _kcc_sema_str_func@PAGEOFF
    bl _kcc_io_puts

    ldr x0, [x19, #AST_F1]
    cbz x0, .Lsdt_func_type
    bl _kcc_io_puts

.Lsdt_func_type:
    adrp x0, _kcc_sema_str_colon@PAGE
    add  x0, x0, _kcc_sema_str_colon@PAGEOFF
    bl _kcc_io_puts

    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    bl _kcc_type_dump

    adrp x0, _kcc_sema_str_nl@PAGE
    add  x0, x0, _kcc_sema_str_nl@PAGEOFF
    bl _kcc_io_puts
    b .Lsdt_next

.Lsdt_typedef:
    adrp x0, _kcc_sema_str_typedef@PAGE
    add  x0, x0, _kcc_sema_str_typedef@PAGEOFF
    bl _kcc_io_puts

    ldr x0, [x19, #AST_F1]
    cbz x0, .Lsdt_td_type
    bl _kcc_io_puts

.Lsdt_td_type:
    adrp x0, _kcc_sema_str_eq@PAGE
    add  x0, x0, _kcc_sema_str_eq@PAGEOFF
    bl _kcc_io_puts

    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    bl _kcc_type_dump

    adrp x0, _kcc_sema_str_nl@PAGE
    add  x0, x0, _kcc_sema_str_nl@PAGEOFF
    bl _kcc_io_puts

.Lsdt_next:
    ldr x19, [x19, #AST_F4]
    b .Lsdt_loop

.Lsdt_done:
    bl _kcc_io_flush
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

262 · used in 30a · defines _kcc_sema_dump_types 623 · uses _kcc_io_flush 69b, _kcc_io_puts 70a, _kcc_sema_resolve_type 259b, _kcc_sema_str_colon 263, _kcc_sema_str_eq 263, _kcc_sema_str_func 263, _kcc_sema_str_nl 263, _kcc_sema_str_typedef 263, _kcc_sema_str_var 263, _kcc_type_dump 249a

26.4. Semantic Analysis Data

Module-level state for semantic analysis: a pointer to the current function's return type (used to validate `return` statements), error message strings for diagnostics, and formatting strings used by the type dumper.

⟨ *Sema: data* 263 ⟩ ≡

```
// -- Sema globals ---
.data
.globl _kcc_sema_cur_ret
_kcc_sema_cur_ret:    .quad 0       // current
   function return type
.globl _kcc_sema_callee_ctx
_kcc_sema_callee_ctx: .quad 0       // 1 while
   checking a call's direct callee
.globl _kcc_sema_callee_implicit
_kcc_sema_callee_implicit: .quad 0  // 1 if the
   callee was an implicit function

// -- Sema error messages ---
.section __TEXT,__cstring,cstring_literals
.globl _kcc_sema_undecl_pre
_kcc_sema_undecl_pre:
    .asciz "use of undeclared identifier '"
_kcc_sema_nomember_pre:
    .asciz "no member named '"
.globl _kcc_sema_redef_pre
_kcc_sema_redef_pre:
    .asciz "redefinition of '"
_kcc_sema_implicit_pre:
    .asciz "implicit declaration of function '"
_kcc_sema_unused_pre:
    .asciz "unused variable '"
_kcc_sema_err_undecl:
    .asciz "use of undeclared identifier"
_kcc_sema_err_not_func:
    .asciz "called object is not a function"
_kcc_sema_err_not_struct:
    .asciz "member access on non-struct/union type
   "
_kcc_sema_err_void_val:
    .asciz "void value not ignored as it ought to
   be"
_kcc_sema_ret_void:
    .asciz "void function should not return a
   value"
_kcc_sema_ret_nonvoid:
    .asciz "non-void function should return a
   value"
_kcc_sema_reach_end:
    .asciz "control reaches end of non-void
   function"
_kcc_sema_sign_cmp:
    .asciz "comparison of integers of different
   signedness"
_kcc_sema_int2ptr:
```

```
    .asciz "incompatible integer to pointer
   conversion"
_kcc_sema_ptr2int:
    .asciz "incompatible pointer to integer
   conversion"
_kcc_sema_main_name:
    .asciz "main"
// Known _Noreturn library functions: a trailing
   call to one of these means
// the function does not fall through (for -
   Wreturn-type).
_kcc_nr_exit:        .asciz "exit"
_kcc_nr_uexit:       .asciz "_exit"
_kcc_nr_bexit:       .asciz "_Exit"
_kcc_nr_abort:       .asciz "abort"
_kcc_nr_longjmp:     .asciz "longjmp"
_kcc_nr_unreach:     .asciz "__builtin_unreachable
   "
_kcc_nr_trap:        .asciz "__builtin_trap"
// Pointer table lives in .data (a cstring section
    holds only strings); the
// following dump strings restore the cstring
   section afterwards.
.data
.p2align 3
_kcc_sema_noreturn_tbl:
    .quad _kcc_nr_exit, _kcc_nr_uexit,
   _kcc_nr_bexit, _kcc_nr_abort
    .quad _kcc_nr_longjmp, _kcc_nr_unreach,
   _kcc_nr_trap, 0
.section __TEXT,__cstring,cstring_literals

// -- Dump type strings ---
_kcc_sema_str_var:      .asciz "var "
_kcc_sema_str_func:     .asciz "func "
_kcc_sema_str_typedef:  .asciz "typedef "
_kcc_sema_str_colon:    .asciz ": "
_kcc_sema_str_eq:       .asciz " = "
_kcc_sema_str_nl:       .asciz "\n"
_kcc_sema_str_va_list:  .asciz "va_list"
_kcc_sema_str_builtin_va_list: .asciz "
   __builtin_va_list"
```

□

263 · used in 30a · defines _kcc_sema_callee_ctx 277, 289, _kcc_sema_callee_implicit 277, 289, _kcc_sema_cur_ret 264, 273, 281, _kcc_sema_redef_pre 277, _kcc_sema_str_builtin_va_list 259b, _kcc_sema_str_colon 262, _kcc_sema_str_eq 262, _kcc_sema_str_func 262, _kcc_sema_str_nl 262, _kcc_sema_str_typedef 262, _kcc_sema_str_var 262, _kcc_sema_str_va_list 259b

26.5. Declaration Checking

The declaration checker dispatches on the AST tag to handle each declaration kind. Variable declarations resolve their type and register a SYM_VAR in the symbol table, optionally type-checking the initialiser. A variable whose type is an *incomplete* array (`int a[] = …`) first has its length inferred by `kcc_sema_infer_array_len`, which reads the initializer—the element span of a brace list, or the string length plus one for a string literal—and synthesises a size expression onto the array type node *before* it is resolved. Because every later resolve of that node (for `sizeof`, for the stack slot, for a file-scope object's storage) reads the now-present size, the completed length is seen uniformly rather than only by the one site that happened to look at the initializer. Multi-dimensional inner brace lists are not yet composed, so an inferred outer dimension over nested braces (`int m[][3] = {{…}}`) is left to the same nested-aggregate gap noted elsewhere. Function declarations resolve the return type, wrap it in a TY_FUNC descriptor via `_kcc_type_func`, and register the resulting function type as SYM_FUNC. The TY_FUNC wrapper is essential: without it, identifier lookup returns the bare return type, and function-decay produces the wrong kind of pointer—the call handler then falls back to `int`, corrupting pointer return values. After registration the checker pushes a scope for parameters and body, then checks the body statements. Typedef declarations register SYM_TYPEDEF—and, crucially, when the typedef names a struct, union, or enum *definition* (`typedef struct S {…} S;`), the checker also runs the ordinary tag-registration path on the underlying type node. Such a declaration introduces two names, the tag `struct S` and the typedef `S`, and registering only the latter left the tag undeclared: a later `struct S` would fall back to `int`—fatally so for a self-referential member like `struct S *next`, whose forward reference to the tag from inside the body would then chain-access a non-struct. Struct and union declarations register the tag in the tag namespace via `_kcc_symtab_declare_tag`. After creating the type descriptor, the checker counts the struct's fields and sets TY_SIZE to $nfields \times 8$, matching the all-spill backend's slot size. This size is essential for pointer subscript on struct pointers (`arr[i].x`), where the stride must equal the full struct width. Enum declarations register the tag and walk the enumerator list, assigning each constant an incrementing integer value. The parser wraps standalone enum declarations in an AST_TYPE_ENUM node; the dispatcher unwraps it to reach the inner AST_ENUM_DECL before processing. The same unwrapping is needed in the statement checker, since enums may be declared inside function bodies.

⟨ *Sema: check declaration* 264 ⟩ ≡

```
// -- kcc_sema_check_decl ---
// void kcc_sema_check_decl(node *decl)
//   x0 = declaration AST node
// Processes declarations, registers symbols.
.text
.globl _kcc_sema_check_decl
.p2align 2
_kcc_sema_check_decl:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    cbz x0, .Lscd_done
    mov x19, x0

    ldr x20, [x19, #AST_TAG]

    cmp x20, #AST_VAR_DECL
    b.eq .Lscd_var
    cmp x20, #AST_FUNC_DECL
    b.eq .Lscd_func
    cmp x20, #AST_TYPEDEF_DECL
    b.eq .Lscd_typedef
    cmp x20, #AST_STRUCT_DECL
    b.eq .Lscd_struct
    cmp x20, #AST_UNION_DECL
    b.eq .Lscd_struct       // same handling as
   struct
    cmp x20, #AST_TYPE_STRUCT
```

```
    b.eq .Lscd_type_struct
    cmp x20, #AST_TYPE_UNION
    b.eq .Lscd_type_struct
    cmp x20, #AST_ENUM_DECL
    b.eq .Lscd_enum
    cmp x20, #AST_TYPE_ENUM
    b.eq .Lscd_type_enum

    // Other nodes (e.g., static_assert): skip
    b .Lscd_done

.Lscd_var:
    // Register any struct/union/enum tag the declared type introduces before
    // resolving, exactly as for a typedef. Without this a block-scoped
    // `struct S { ... } x;` (or a bare `struct S { ... };`) never registers the
    // `struct S` tag, so a later `struct S` / `sizeof(struct S)` / `(struct S*)`
    // resolves to the int fallback and member access fails.  Peel ARRAY and
    // qualifier wrappers first so an inline definition behind them registers
    // too: `struct T { ... } a[2];` and `static const struct EncName {...}
    // encnames[];` (SQLite's PRAGMA encoding table) must register the tag just
    // like the scalar form.  A bodyless pointer wrapper (`struct T *p;`) is a
    // forward reference we must NOT register --- but a pointer to an inline
    // DEFINITION (`struct S { ... } *t;`, the ubiquitous `struct Node {...}
    // *head;`) still defines the tag and must register its layout, or a later
    // `struct S` resolves to a member-less descriptor and every field access and
    // aggregate initialiser silently reads/writes the wrong offsets.
    ldr x0, [x19, #AST_F0]          // type node
    cbz x0, .Lscd_var_resolve
.Lscd_var_peel:
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_TYPE_ARRAY
    b.eq .Lscd_var_peel1
    cmp x1, #AST_TYPE_QUALS
    b.eq .Lscd_var_peel1
    cmp x1, #AST_TYPE_PTR
    b.eq .Lscd_var_ptr              // pointer: peel only toward a bodied def
    b .Lscd_var_tagchk
.Lscd_var_peel1:
    ldr x0, [x0, #AST_F0]           // peel to element / base type
    cbnz x0, .Lscd_var_peel
    b .Lscd_var_resolve
.Lscd_var_ptr:
    // Peel pointer/array/qualifier wrappers toward the ultimate base type, then
    // register it ONLY if it is a struct/union carrying a body (a definition).
    // A bodyless base stays a forward reference: skip to resolve so we never
    // overwrite a complete tag with an incomplete descriptor.
    ldr x0, [x0, #AST_F0]           // pointee
    cbz x0, .Lscd_var_resolve
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_TYPE_ARRAY
    b.eq .Lscd_var_ptr
    cmp x1, #AST_TYPE_QUALS
    b.eq .Lscd_var_ptr
    cmp x1, #AST_TYPE_PTR
    b.eq .Lscd_var_ptr
    cmp x1, #AST_TYPE_STRUCT
    b.eq .Lscd_var_ptrbody
    cmp x1, #AST_TYPE_UNION
    b.ne .Lscd_var_resolve
.Lscd_var_ptrbody:
    ldr x2, [x0, #AST_F0]           // struct_decl node
    cbz x2, .Lscd_var_resolve
    ldr x2, [x2, #AST_F1]           // fields (body present iff non-zero)
    cbz x2, .Lscd_var_resolve       // bodyless → forward reference, do not register
    b .Lscd_var_pretag              // bodied → register the tag and lay it out
.Lscd_var_tagchk:
    cmp x1, #AST_TYPE_STRUCT
    b.eq .Lscd_var_pretag
    cmp x1, #AST_TYPE_UNION
    b.eq .Lscd_var_pretag
    cmp x1, #AST_TYPE_ENUM
    b.ne .Lscd_var_resolve
.Lscd_var_pretag:
    // x0 = the (peeled) struct/union/enum type node
    bl _kcc_sema_check_decl         // declares/completes the tag

.Lscd_var_resolve:
    // Complete an incomplete array's length from its initializer (so every
    // later resolve - sizeof, the alloca, BSS sizing - sees the real length).
    mov x0, x19                     // decl node
    bl _kcc_sema_infer_array_len

    // Resolve type
    ldr x0, [x19, #AST_F0]          // type node
    bl _kcc_sema_resolve_type
    mov x21, x0                      // resolved type

    // Register in symbol table
    ldr x0, [x19, #AST_F1]          // name_ptr
    cbz x0, .Lscd_var_init
    mov x22, x0                      // name

    // Reject a same-scope redefinition (block scope only; see the helper).
    mov x0, x19                     // decl node
    mov x1, x22                     // name
    bl _kcc_sema_check_redef

    mov x0, x22                     // name
    bl _kcc_strlen
    mov x1, x0                      // len
    mov x0, x22                     // name
    mov x2, #SYM_VAR
    mov x3, x21
    bl _kcc_symtab_declare
    // Record the declaration's real location on the symbol (declare stamps a
    // stale EOF line); the unused-variable warning reads it back.
    ldr x1, [x19, #AST_LINE]
    str x1, [x0, #SYM_LINE]
    ldr x1, [x19, #AST_COL]
    str x1, [x0, #SYM_COL]

.Lscd_var_init:
    // Check initializer if present
    ldr x0, [x19, #AST_F2]
    cbz x0, .Lscd_done
```

```
    bl _kcc_sema_expr               // x0 =
   initializer type

    // -Wall -Wint-conversion: declared type (x21)
    vs initializer type
    mov x2, x0                      // src type
    mov x1, x21                     // dst =
   declared type
    mov x0, x19                     // location
   node
    ldr x3, [x19, #AST_F2]          // src node
    bl _kcc_sema_check_int_ptr
    b .Lscd_done

.Lscd_func:
    // Resolve return type
    ldr x0, [x19, #AST_F0]          // type node
    bl _kcc_sema_resolve_type
    // Wrap in TY_FUNC, carrying the parameter
   declaration list (masked of the
    // variadic bit) so a call site can convert
   and classify each argument by
    // its parameter's type.
    ldr x1, [x19, #AST_F2]          // params
   pointer (bit 0=variadic, bit 1=static)
    bic x1, x1, #3
    bl _kcc_type_func
    mov x21, x0                     // TY_FUNC
   descriptor

    // Variadic prototype: record the fixed-param
   count in TY_QUALS (see prose).
    ldr x2, [x19, #AST_F2]          // params
   pointer (bit 0=variadic, bit 1=static)
    tst x2, #1
    b.eq .Lscd_func_declared
    bic x2, x2, #3
    mov x3, #0                      // nfixed
.Lscd_func_nfixed:
    cbz x2, .Lscd_func_setq
    add x3, x3, #1
    ldr x2, [x2, #AST_F4]
    b .Lscd_func_nfixed
.Lscd_func_setq:
    str x3, [x21, #TY_QUALS]
.Lscd_func_declared:

    // Register function in symbol table
    ldr x0, [x19, #AST_F1]          // name_ptr
    cbz x0, .Lscd_func_body
    mov x22, x0
    bl _kcc_strlen
    mov x1, x0
    mov x0, x22
    mov x2, #SYM_FUNC
    mov x3, x21
    bl _kcc_symtab_declare

.Lscd_func_body:
    // Check function body if present
    ldr x0, [x19, #AST_F3]          // body
    cbz x0, .Lscd_done

    // Store current function return type (extract
    from TY_FUNC)
    ldr x0, [x21, #TY_F0]           // return type
   from TY_FUNC
    adrp x1, _kcc_sema_cur_ret@PAGE
    add  x1, x1, _kcc_sema_cur_ret@PAGEOFF
    str x0, [x1]

    // Push scope for function body
    bl _kcc_symtab_push_scope
    bl _kcc_symtab_label_begin

    // Process parameters (mask the variadic tag
   in bit 0, static tag in bit 1)
    ldr x0, [x19, #AST_F2]          // first param
   (bit 0=variadic, bit 1=static)
    bic x0, x0, #3
.Lscd_param_loop:
    cbz x0, .Lscd_func_stmts
    mov x22, x0

    // Resolve param type and register.  An array/
   function parameter is
    // adjusted to a pointer (\stdref{6.7.6.3}) so
    `a[i]` indexes through the
    // passed address, not inline storage.
    ldr x0, [x22, #AST_F0]          // param type
   node
    bl _kcc_sema_resolve_type
    bl _kcc_type_param_adjust
    mov x21, x0

    ldr x0, [x22, #AST_F1]          // param name
    cbz x0, .Lscd_param_next
    mov x20, x0
    // A duplicate parameter name is a
   redefinition (params live in the body
    // scope, depth >= 2).
    mov x0, x22                     // param node
    mov x1, x20                     // name
    bl _kcc_sema_check_redef
    mov x0, x20
    bl _kcc_strlen
    mov x1, x0
    mov x0, x20
    mov x2, #SYM_VAR
    mov x3, x21
    bl _kcc_symtab_declare

.Lscd_param_next:
    ldr x0, [x22, #AST_F4]          // next param
    b .Lscd_param_loop

.Lscd_func_stmts:
    // Check function body statements
    ldr x0, [x19, #AST_F3]
    bl _kcc_sema_check_stmt

    // -Wall: warn if a non-void function can
   reach the end of its body
    mov x0, x19                     // func node
    bl _kcc_sema_check_return_path

    bl _kcc_symtab_label_end
    bl _kcc_symtab_pop_scope
    b .Lscd_done
```

□

264 · defined at 264, 269 · used in 30a · defines _kcc_sema_check_decl 259a, 269, 273 · uses _kcc_sema_check_stmt 271, _kcc_sema_cur_ret 263, _kcc_sema_expr 274, _kcc_sema_resolve_type 259b, _kcc_symtab_declare 254b, _kcc_symtab_label_begin 257a, _kcc_symtab_label_end 257a, _kcc_symtab_pop_scope 254a, _kcc_symtab_push_scope 253c, _kcc_type_func 243b, _kcc_type_param_adjust 242b

Typedefs, Aggregate Tags, and Enumerations Typedef declarations resolve the underlying type and register the name with kind `SYM_TYPEDEF`. Struct and union declarations create the corresponding type descriptor via `kcc_type_struct` or `kcc_type_union` and register the tag

in the tag name space. The struct size, alignment, and per-field byte offsets are computed by `_kcc_type_layout` (below), which lays out the aggregate following the standard C rules rather than the earlier one-slot-per-field approximation. Enum declarations register both the tag itself and each enumerator constant as an ordinary symbol of type `int`, assigning consecutive integer values starting from zero. An enumerator may carry an explicit `= expr` initializer, which §6.7.2.2 requires to be any integer constant expression and which resets the running counter; we fold it with the shared `_kcc_eval_const_int` evaluator so that arithmetic forms like `(UCHAR_MAX + 1)`—the basis of Lua's reserved-token enumeration—are honoured, not just bare literals. (Recognising only `INT_LIT` here silently renumbered such an `enum` from zero, so `TK_NAME` and its neighbours collapsed onto small single-character token codes and every keyword lexed as an "unexpected symbol".)

Aggregate Layout `_kcc_type_layout` computes the byte-accurate layout of a `struct` or `union` following §6.7.2.1: each member is placed at the next offset that satisfies its alignment, the aggregate's alignment is the maximum member alignment, and the total size is padded up to that alignment. For a `union` every member sits at offset 0 and the size is the largest member's size. The routine is idempotent and memoised: once `TY_SIZE` is non-zero the cached layout is returned, so it is safe to call from any later phase (`sizeof`, alloca sizing, member addressing) to force a lazy layout. Each member's byte offset is stored in its declaration node's `AST_F3` slot, from which the IR builder reads it when addressing a member. Because a member of aggregate type must be laid out before its own offset can be known, the routine recurses into nested aggregates first, and it recomputes array sizes from the (possibly just-laid-out) element type; C forbids a struct from containing itself by value, so the recursion always terminates. `_kcc_type_align` returns a type's alignment, reading `TY_ALIGN` directly for scalars, pointers, and already-laid-out aggregates, and recursing through the element type of an array.

⟨ *Sema: struct layout* 267 ⟩ ≡

```
// -- kcc_type_align ---
// uint64_t kcc_type_align(type_desc *ty)
//   Alignment in bytes; 8 for null/unknown.
   Arrays inherit element align.
.text
.globl _kcc_type_align
.p2align 2
_kcc_type_align:
    cbz x0, .Lta_def
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.ne .Lta_read
    ldr x0, [x0, #TY_F0]          // element type
    b _kcc_type_align             // tail recurse
.Lta_read:
    ldr x0, [x0, #TY_ALIGN]
    cbz x0, .Lta_def
    ret
.Lta_def:
    mov x0, #8
    ret

// -- kcc_type_layout ---
// uint64_t kcc_type_layout(type_desc *ty)
//   Ensures ty (and any aggregate it transitively
    contains) has a
//   byte-accurate TY_SIZE/TY_ALIGN, and stores
   each struct/union member's
//   byte offset in its field decl's AST_F3.
   Returns TY_SIZE.  Idempotent.
.text
.globl _kcc_type_layout
.p2align 2
_kcc_type_layout:
    stp x29, x30, [sp, #-112]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    cbz x0, .Ltl_zero
    mov x19, x0                   // ty
    ldr x1, [x19, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.eq .Ltl_array
    cmp x1, #TY_STRUCT
    b.eq .Ltl_aggregate
    cmp x1, #TY_UNION
    b.eq .Ltl_aggregate
    // Scalar / pointer / enum: already sized by
   its constructor.
    ldr x0, [x19, #TY_SIZE]
    b .Ltl_ret

.Ltl_array:
    ldr x20, [x19, #TY_F0]        // element type
    ldr x21, [x19, #TY_F1]        // length
    mov x0, x20
    bl _kcc_type_layout           // ensure element
   laid out → element size
    mul x0, x0, x21
    str x0, [x19, #TY_SIZE]       // size =
   element_size * length
    mov x0, x20
    bl _kcc_type_align
    str x0, [x19, #TY_ALIGN]      // align = element
    align
    ldr x0, [x19, #TY_SIZE]
    b .Ltl_ret

.Ltl_aggregate:
    ldr x0, [x19, #TY_SIZE]
    cbnz x0, .Ltl_ret             // memoised:
   already laid out
    ldr x1, [x19, #TY_KIND]
    cmp x1, #TY_UNION
    cset x20, eq                  // x20 = is_union
    ldr x21, [x19, #TY_F0]        // AST_STRUCT_DECL
    cbz x21, .Ltl_zero
    ldr x0, [x21, #AST_F3]        // layout word: (
   aligned(N) << 1) | packed
    str x0, [sp, #104]            // stash across
   the field loop
    ldr x21, [x21, #AST_F1]       // first field
    cbz x21, .Ltl_zero            // opaque (no body
   ): cannot size
    mov x22, #0                   // running offset
   (struct)
    mov x23, #1                   // max alignment
    mov x24, #0                   // max member size
    (union)
    // Bit-field container state (slots survive
   the calls in the loop body):
    // [sp,#80] container byte offset, [sp,#88]
   next free bit, [sp,#96] container
    // size in bytes (0 = no active container).
    str xzr, [sp, #96]
```

```
.Ltl_loop:
    cbz x21, .Ltl_finish
    ldr x0, [x21, #AST_F0]      // field type
   specifier
    cbz x0, .Ltl_field_default
    bl _kcc_sema_resolve_type
    cbz x0, .Ltl_field_default
    str x0, [sp, #64]           // stash ftype
   across the calls
    bl _kcc_type_layout         // ensure laid out
    → field size in x0
    str x0, [sp, #72]           // stash fsize
    ldr x0, [sp, #64]           // ftype
    bl _kcc_type_align          // falign in x0
    mov x3, x0                  // falign
    ldr x2, [sp, #72]           // fsize
    b .Ltl_field_have
.Ltl_field_default:
    mov x2, #8                  // unresolved:
   assume one 8-byte slot
    mov x3, #8
.Ltl_field_have:
    // __attribute__((packed)): every member (and
   bit-field container) aligns at
    // 1, so there is no inter-member padding and
   the aggregate alignment stays 1.
    ldr x6, [sp, #104]
    tbz x6, #0, .Ltl_fh_unpacked
    mov x3, #1
.Ltl_fh_unpacked:
    // A bit-field is marked by a non-null width
   in AST_F2 (an expression node).
    // Everything else is a whole-byte member.
    ldr x5, [x21, #AST_F2]
    cbz x5, .Ltl_not_bitfield
    ⟨ Sema: layout bit-field 268 ⟩
.Ltl_not_bitfield:
    str xzr, [sp, #96]          // a plain member
   ends any bit-field container run
    cbnz x20, .Ltl_union_field
    // struct: round running offset up to the
   field's alignment
    sub x4, x3, #1
    add x22, x22, x4
    bic x22, x22, x4            // aligned offset
   (falign is a power of two)
    str x22, [x21, #AST_F3]     // record field
   byte offset
    add x22, x22, x2            // advance by
   field size
    b .Ltl_field_align
.Ltl_union_field:
    str xzr, [x21, #AST_F3]     // union member:
   offset 0
    cmp x2, x24
    csel x24, x2, x24, gt       // track largest
   member
.Ltl_field_align:
    cmp x3, x23
    csel x23, x3, x23, gt       // track maximum
   alignment
    ldr x21, [x21, #AST_F4]     // next field
    b .Ltl_loop

.Ltl_finish:
    cbz x20, .Ltl_total
    mov x22, x24                // union total =
   largest member
.Ltl_total:
    // __attribute__((aligned(N))): raise the
   aggregate's alignment to at least N.
    ldr x6, [sp, #104]
    lsr x6, x6, #1              // requested
   alignment (0 = none)
    cmp x6, x23
    csel x23, x6, x23, gt
    // pad total size up to the aggregate
   alignment
    sub x4, x23, #1
    add x22, x22, x4
    bic x22, x22, x4
    str x22, [x19, #TY_SIZE]
    str x23, [x19, #TY_ALIGN]
    mov x0, x22
    b .Ltl_ret

.Ltl_zero:
    mov x0, #0
.Ltl_ret:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #112
    ret
```

□

267 · used in 30a · defines _kcc_type_align never used, _kcc_type_layout 243a, 269, 290, 308, 314b, 320, 326, 343, 347, 349a, 350, 355, 359, 369, 384, 387, 407, 413, 414, 417b, 426, 465, 469, 562, 563, 564 · uses _kcc_sema_resolve_type 259b

Bit-field packing A bit-field member (§6.7.2.1) declares a width in bits rather than occupying a whole storage unit. kcc lays consecutive bit-fields of the *same declared type* into one *container*—an aligned unit the size of that type—filling it from the least-significant bit upward, which is the little-endian AArch64 convention. A bit-field that would not fit in the remaining space of the current container (or whose declared type differs from it) starts a fresh, type-aligned container; its own container is reserved whole, so a lone `unsigned x : 3` still makes the struct four bytes. The layout records two numbers on the field node: `AST_F3` becomes the container's byte offset (the address the member access will load), and `AST_F2`—which held the width expression—is overwritten with the packed pair `(bit_offset ≪ 16) | width`. A zero there afterwards means "not a bit-field," so ordinary members are untouched. This is a deliberately conservative model: same-type runs, which is what real bit-field structs overwhelmingly use, match `clang` exactly; mixed-type packing and zero-width separators are not yet modelled.

⟨ Sema: layout bit-field 268 ⟩ ≡

```
// x2 = fsize (container bytes), x3 = falign, x5 =
    width expr, x20 = is_union,
// x21 = field, x22 = struct byte cursor.  Falls
   through to .Ltl_field_align.
    ldr x7, [x5, #AST_TAG]
.Ltl_bf_unwrap:
    cmp x7, #AST_PAREN_EXPR
    b.ne .Ltl_bf_folded
    ldr x5, [x5, #AST_F0]
    cbz x5, .Ltl_bf_w0
    ldr x7, [x5, #AST_TAG]
    b .Ltl_bf_unwrap
.Ltl_bf_folded:
    cmp x7, #AST_INT_LIT
    b.eq .Ltl_bf_getval
    cmp x7, #AST_CHAR_LIT
    b.eq .Ltl_bf_getval
.Ltl_bf_w0:
```

```
    mov x6, #0                  // non-constant
   width → 0 (degrades to plain)
    b .Ltl_bf_wdone
.Ltl_bf_getval:
    ldr x6, [x5, #AST_F0]       // width value
.Ltl_bf_wdone:
    cbnz x20, .Ltl_bf_union
    lsl x7, x2, #3              // container bits
    ldr x8, [sp, #96]           // active container
    size (0 = none)
    cmp x8, x2
    b.ne .Ltl_bf_newcont
    ldr x9, [sp, #88]           // next free bit
    add x10, x9, x6
    cmp x10, x7
    b.gt .Ltl_bf_newcont
    ldr x11, [sp, #80]          // reuse the active
    container
    b .Ltl_bf_store
.Ltl_bf_newcont:
    sub x4, x3, #1
    add x22, x22, x4
    bic x22, x22, x4            // align the cursor
    to the container type
    mov x11, x22
    str x11, [sp, #80]
    str x2, [sp, #96]           // container size
    mov x9, #0                  // bit offset 0 in
   the fresh unit
    add x22, x22, x2            // reserve the
   whole container
.Ltl_bf_store:
    str x11, [x21, #AST_F3]     // container byte
   offset
    lsl x12, x9, #16
    orr x12, x12, x6
    str x12, [x21, #AST_F2]     // (bit_offset <<
   16) | width
    add x9, x9, x6
    str x9, [sp, #88]           // advance the bit
    cursor
    b .Ltl_field_align
.Ltl_bf_union:
    str xzr, [x21, #AST_F3]     // union bit-field
    shares offset 0
    and x12, x6, #0xffff
    str x12, [x21, #AST_F2]     // bit_offset 0,
   width
    cmp x2, x24
    csel x24, x2, x24, gt
    b .Ltl_field_align
```

□

268 · used in 267 · defines _kcc_type_layout 243a, 267, 269, 290, 308, 314b, 320, 326, 343, 347, 349a, 350, 355, 359, 369, 384, 387, 407, 413, 414, 417b, 426, 465, 469, 562, 563, 564

⟨ Sema: check declaration 269 ⟩ + ≡

```
.Lscd_typedef:
    // Register the tag BEFORE resolving.  `
   typedef struct S {...} S;` (and the
    // forward `typedef struct S S;`) introduces
   the tag `struct S` as well as
    // the typedef name.  Declaring the tag first
   means the resolve below binds
    // the typedef to the tag's shared,
   completable descriptor rather than a
    // fallback: a forward typedef whose tag does
   not yet exist would otherwise
    // capture `int`, and a later `struct S { ...
   }` could never reach it.  It
    // also lets a self-referential member (`
   struct S *next`) resolve the tag
    // from inside the body.  The tag can sit
   under pointer/array declarators,
    // as in the opaque-handle idiom `typedef
   struct S_s *SP;` (zlib's
    // \texttt{gzFile}): unwrap those layers to
   reach the base struct/union/enum
    // node, or the later `struct S_s { ... }`
   completes a descriptor the
    // typedef never saw and `p->member` fails on
   a pointer-to-int.
    ldr x0, [x19, #AST_F0]          // type node
    cbz x0, .Lscd_done
.Lscd_td_unwrap:
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_TYPE_PTR
    b.eq .Lscd_td_unwrap_step
    cmp x1, #AST_TYPE_ARRAY
    b.ne .Lscd_td_check
.Lscd_td_unwrap_step:
    ldr x2, [x0, #AST_F0]           // pointee /
   element type
    cbz x2, .Lscd_td_resolve        // malformed:
   nothing to pre-register
    mov x0, x2
    b .Lscd_td_unwrap
.Lscd_td_check:
    cmp x1, #AST_TYPE_STRUCT
    b.eq .Lscd_td_pretag
    cmp x1, #AST_TYPE_UNION
    b.eq .Lscd_td_pretag
    cmp x1, #AST_TYPE_ENUM
    b.ne .Lscd_td_resolve
.Lscd_td_pretag:
    // x0 = the unwrapped struct/union/enum type
   node → declares the tag
    bl _kcc_sema_check_decl

.Lscd_td_resolve:
    // Resolve the (now tag-registered) type and
   bind the typedef name to it.
    ldr x0, [x19, #AST_F0]          // type node
    bl _kcc_sema_resolve_type
    mov x21, x0
    ldr x0, [x19, #AST_F1]          // name_ptr
    cbz x0, .Lscd_done              // nameless (
   invalid): the tag is already set
    mov x22, x0
    bl _kcc_strlen
    mov x1, x0
    mov x0, x22
    mov x2, #SYM_TYPEDEF
    mov x3, x21
    bl _kcc_symtab_declare
    b .Lscd_done

.Lscd_type_struct:
    // AST_TYPE_STRUCT/UNION wrapper: unwrap to
   inner struct_decl
    ldr x19, [x19, #AST_F0]         // inner
   AST_STRUCT_DECL
    cbz x19, .Lscd_done
    ldr x20, [x19, #AST_TAG]
    // Fall through to .Lscd_struct

.Lscd_struct:
    // A member may itself \emph{define} a tagged
   type --- an enum whose
    // constants leak out (\texttt{struct \{ enum
   \{ X \} x; \}}, \stdref{6.2.1}),
    // or a struct/union whose tag has enclosing (
   file/block) scope in C, not the
```

```
    // member scope of C++ (\texttt{struct Hash \{
    struct \_ht \{~\ldots~\} *ht;
    // \};} makes \texttt{struct \_ht} usable
   later).  Register each such nested
    // definition before the tag itself, peeling
   pointer/array/qualifier wrappers
    // to reach the core type.  Only a type
   carrying a \emph{body} is registered,
    // so a bare reference (including a self-
   reference like \texttt{struct Hash
    // *next}) is left alone and no incomplete tag
    is created prematurely.
    ldr x21, [x19, #AST_F1]         // first field
.Lscd_struct_efld:
    cbz x21, .Lscd_struct_efld_done
    ldr x0, [x21, #AST_F0]          // field type
   node
.Lscd_efld_peel:
    cbz x0, .Lscd_struct_efld_next
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_TYPE_PTR
    b.eq .Lscd_efld_deref
    cmp x1, #AST_TYPE_ARRAY
    b.eq .Lscd_efld_deref
    cmp x1, #AST_TYPE_QUALS
    b.eq .Lscd_efld_deref
    b .Lscd_efld_core
.Lscd_efld_deref:
    ldr x0, [x0, #AST_F0]           // peel to
   pointee / element / base
    b .Lscd_efld_peel
.Lscd_efld_core:
    // x0 = core type node, x1 = its tag.  Find
   the underlying decl (whose
    // AST_F1 holds members/enumerators) and
   register only if it has a body.
    cmp x1, #AST_TYPE_STRUCT
    b.eq .Lscd_efld_wrap
    cmp x1, #AST_TYPE_UNION
    b.eq .Lscd_efld_wrap
    cmp x1, #AST_TYPE_ENUM
    b.eq .Lscd_efld_wrap
    cmp x1, #AST_STRUCT_DECL
    b.eq .Lscd_efld_body
    cmp x1, #AST_UNION_DECL
    b.eq .Lscd_efld_body
    cmp x1, #AST_ENUM_DECL
    b.eq .Lscd_efld_body
    b .Lscd_struct_efld_next
.Lscd_efld_wrap:
    ldr x2, [x0, #AST_F0]           // wrapper →
   inner decl node
    cbz x2, .Lscd_struct_efld_next
    ldr x3, [x2, #AST_F1]           // its members
   / enumerators
    cbz x3, .Lscd_struct_efld_next // bodyless
   reference: leave it alone
    b .Lscd_struct_efld_reg
.Lscd_efld_body:
    ldr x3, [x0, #AST_F1]           // decl's
   members / enumerators
    cbz x3, .Lscd_struct_efld_next
.Lscd_struct_efld_reg:
    bl _kcc_sema_check_decl         // registers
   the nested tag / enumerators
.Lscd_struct_efld_next:
    ldr x21, [x21, #AST_F4]
    b .Lscd_struct_efld
.Lscd_struct_efld_done:

    // Register struct/union tag if named
    ldr x0, [x19, #AST_F0]          // name_ptr
    cbz x0, .Lscd_done
    mov x22, x0
    // A struct tag may already exist as an \emph{
   incomplete} descriptor---from
    // a forward declaration or a forward `typedef
    struct S S;`---and a later
    // definition must complete that \emph{same}
   object so every binding (the
    // typedef, earlier references) sees the
   members.  Only a tag declared at the
    // \emph{current} scope depth may be completed
   , though: a same-named tag from
    // an enclosing scope is a distinct type that
   this declaration shadows, so
    // \texttt{\{ struct T \{ int z; \}; \}}
   inside a function must \emph{not}
    // repoint the outer \texttt{struct T} at the
   inner body.  Four cases follow
    // from (existing at this depth?) times (this
   decl has a body?).
    mov x0, x22
    bl _kcc_strlen
    mov x1, x0
    mov x0, x22
    bl _kcc_symtab_lookup_tag
    cbz x0, .Lscd_struct_new
    ldr x21, [x0, #SYM_TYPE]        // existing
   descriptor
    cbz x21, .Lscd_struct_new
    ldr x3, [x0, #SYM_DEPTH]        // remember the
    tag's scope depth (no calls below
                                    // clobber x3
   before it is compared)
    // A bodyless occurrence (`struct S s;`, a
   reference) is a no-op and simply
    // uses the visible tag---regardless of the
   scope it was declared in.
    ldr x0, [x19, #AST_F1]          // fields?
    cbz x0, .Lscd_done
    // This declaration HAS a body, so it is a
   definition.  Complete the existing
    // tag in place only if it was declared at the
    current scope depth; a
    // same-named tag from an enclosing scope is a
    distinct type this definition
    // shadows, so fall through to create a fresh
   one.
    adrp x2, _kcc_sym_depth@PAGE
    add  x2, x2, _kcc_sym_depth@PAGEOFF
    ldr x2, [x2]
    cmp x3, x2
    b.ne .Lscd_struct_new
    str x19, [x21, #TY_F0]          // repoint at
   the completing decl
    str xzr, [x21, #TY_SIZE]        // reset so
   layout recomputes
    mov x0, x21
    bl _kcc_type_layout
    b .Lscd_done

.Lscd_struct_new:
    // Create type for struct/union
    mov x0, x19
    cmp x20, #AST_STRUCT_DECL
    b.ne .Lscd_make_union
    bl _kcc_type_struct
    b .Lscd_tag_register
.Lscd_make_union:
    bl _kcc_type_union

.Lscd_tag_register:
    mov x21, x0                      // type
   descriptor
```

```
    // Lay out the aggregate: byte-accurate size,
   alignment, field offsets.
    mov x0, x21
    bl _kcc_type_layout

    mov x0, x22                         // name
    bl _kcc_strlen
    mov x1, x0
    mov x0, x22
    mov x2, x21
    bl _kcc_symtab_declare_tag
    b .Lscd_done

.Lscd_type_enum:
    // Unwrap AST_TYPE_ENUM → get EnumDecl child
    ldr x19, [x19, #AST_F0]
    cbz x19, .Lscd_done

.Lscd_enum:
    // Register enum tag and enumerator constants
    ldr x0, [x19, #AST_F0]          // name_ptr
    cbz x0, .Lscd_enum_consts

    mov x22, x0
    mov x0, x19
    bl _kcc_type_enum
    mov x21, x0
    mov x0, x22
    bl _kcc_strlen
    mov x1, x0
    mov x0, x22
    mov x2, x21
    bl _kcc_symtab_declare_tag

.Lscd_enum_consts:
    // Register each enumerator as SYM_ENUM_CONST
   with int type
    ldr x22, [x19, #AST_F1]         // first
   enumerator
    mov x20, #0                      // counter
   value

.Lscd_enum_loop:
    cbz x22, .Lscd_done

    // Explicit value (= expr) resets the running
   counter (§6.7.2.2).  The
    // initializer is any integer constant
   expression---a literal, a
    // parenthesised or arithmetic form like (
   UCHAR_MAX + 1), a cast, or
    // sizeof---so fold it with the shared
   evaluator instead of matching a
    // few shapes by hand (which renumbered such
   enums from zero).
    ldr x0, [x22, #AST_F1]          // value expr,
   or 0
    cbz x0, .Lscd_enum_reg          // no explicit
   value → keep the counter
    bl _kcc_eval_const_int
    mov x20, x0                     // reset
   counter to the evaluated value

.Lscd_enum_reg:
    ldr x0, [x22, #AST_F0]          // name_ptr
    cbz x0, .Lscd_enum_next
    mov x21, x0
    bl _kcc_strlen
    mov x1, x0
    mov x0, x21
    mov x2, #SYM_ENUM_CONST
    adrp x3, _kcc_type_int_s@PAGE
    add  x3, x3, _kcc_type_int_s@PAGEOFF
    ldr  x3, [x3]
    bl _kcc_symtab_declare

    // Store enum value
    str x20, [x0, #SYM_VALUE]

.Lscd_enum_next:
    add x20, x20, #1
    ldr x22, [x22, #AST_F4]         // next
   enumerator
    b .Lscd_enum_loop

.Lscd_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

269 · defined at 264, 269 · used in 30a ·
uses _kcc_eval_const_int 426, _kcc_sema_check_decl 264, _kcc_sema_resolve_type 259b, _kcc_symtab_declare 254b, _kcc_symtab_declare_tag 256a, _kcc_symtab_lookup_tag 256b, _kcc_sym_depth 258, _kcc_type_enum 244, _kcc_type_int_s 250b, _kcc_type_layout 267, 268, _kcc_type_struct 243c, _kcc_type_union 243c

26.6. Statement Checking

The statement checker walks the AST recursively, dispatching on the statement tag. Compound statements push a new scope before walking their children and pop it afterward, implementing C's block scoping rules. Expression statements type-check their expression via `_kcc_sema_expr`. Control-flow statements (`if`/`while`/`do`/`for`) check their conditions and recurse into their bodies. `for` pushes its own scope to contain the loop variable declaration. Labels register themselves in the label namespace. Declarations encountered inside compound statements are forwarded to the declaration checker.

⟨ *Sema: check statement* 271 ⟩ ≡

```
// -- kcc_sema_check_stmt ---
// void kcc_sema_check_stmt(node *stmt)
//   x0 = statement AST node
.text
.globl _kcc_sema_check_stmt
.p2align 2
_kcc_sema_check_stmt:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    cbz x0, .Lscs_done
    mov x19, x0

    ldr x20, [x19, #AST_TAG]

    cmp x20, #AST_COMPOUND
    b.eq .Lscs_compound
    cmp x20, #AST_EXPR_STMT
    b.eq .Lscs_expr
    cmp x20, #AST_IF_STMT
    b.eq .Lscs_if
    cmp x20, #AST_WHILE_STMT
    b.eq .Lscs_while
    cmp x20, #AST_DO_STMT
    b.eq .Lscs_do
    cmp x20, #AST_FOR_STMT
    b.eq .Lscs_for
    cmp x20, #AST_RETURN_STMT
    b.eq .Lscs_return
```

```
    cmp x20, #AST_SWITCH_STMT
    b.eq .Lscs_switch
    cmp x20, #AST_LABEL_STMT
    b.eq .Lscs_label
    cmp x20, #AST_CASE_STMT
    b.eq .Lscs_case
    cmp x20, #AST_DEFAULT_STMT
    b.eq .Lscs_default
    cmp x20, #AST_GOTO_STMT
    b.eq .Lscs_done          // nothing to check
    cmp x20, #AST_BREAK_STMT
    b.eq .Lscs_done
    cmp x20, #AST_CONTINUE_STMT
    b.eq .Lscs_done
    cmp x20, #AST_NULL_STMT
    b.eq .Lscs_done

    // Could be a declaration within a compound
   stmt
    cmp x20, #AST_VAR_DECL
    b.eq .Lscs_decl
    cmp x20, #AST_TYPEDEF_DECL
    b.eq .Lscs_decl
    cmp x20, #AST_STRUCT_DECL
    b.eq .Lscs_decl
    cmp x20, #AST_UNION_DECL
    b.eq .Lscs_decl
    cmp x20, #AST_ENUM_DECL
    b.eq .Lscs_decl
    cmp x20, #AST_TYPE_ENUM
    b.eq .Lscs_decl
    // A bare `struct S { ... };` / `union U { ...
    };` at block scope is an
    // AST_TYPE_STRUCT/UNION statement; route it
   to the declaration checker too,
    // so its tag is registered for later `struct
   S` references in the block.
    cmp x20, #AST_TYPE_STRUCT
    b.eq .Lscs_decl
    cmp x20, #AST_TYPE_UNION
    b.eq .Lscs_decl
    cmp x20, #AST_STATIC_ASSERT
    b.eq .Lscs_done
    cmp x20, #AST_ASM_STMT
    b.eq .Lscs_asm

    b .Lscs_done

.Lscs_asm:
    // Inline asm: type-check each operand's
   expression so it carries a resolved
    // type (the IR builder reads TY_SIZE from it
   for output store widths).
    ldr x19, [x19, #AST_F2]        // first
   operand
.Lscs_asm_loop:
    cbz x19, .Lscs_done
    ldr x0, [x19, #AST_F2]         // operand
   expression
    cbz x0, .Lscs_asm_next
    bl _kcc_sema_expr
.Lscs_asm_next:
    ldr x19, [x19, #AST_F4]
    b .Lscs_asm_loop
```

□

271 · defined at 271, 272, 273 · used in 30a · defines _kcc_sema_check_stmt 264, 272, 273, 274 · uses _kcc_sema_expr 274

Compound Statements and Control Flow Compound statements push a new scope onto both the ordinary and tag stacks, walk their child list recursively, and pop the scope on exit. Just before that pop—and likewise before a `for`-scope is popped—`kcc_sema_check_unused_scope` runs, so a block-scope variable that was declared but never referenced draws an `unused variable 'metaname'` warning under `-Wall`. The check is placed at the block and `for` scopes but *not* at the function scope, whose only occupants are the parameters—an unused parameter is outside `-Wall` (it lives in `-Wextra`), and a function body's top-level locals sit in the body's own compound scope, one level in, where they are seen normally. A variable counts as referenced the moment it appears in any identifier expression—read, written, or address-taken—which the resolver marks by setting the *used* bit on the symbol; only a name that never appears at all is reported, keeping the warning free of false positives on genuinely-referenced variables. Expression statements simply type-check their contained expression. The `if`/`while`/`do`/`for` handlers each check their condition expression and recurse into their bodies; `for` additionally pushes its own scope to contain the loop variable declaration.

⟨ *Sema: check statement* 272 ⟩ + ≡

```
.Lscs_compound:
    bl _kcc_symtab_push_scope

    // Walk items
    ldr x19, [x19, #AST_F0]        // first item
.Lscs_compound_loop:
    cbz x19, .Lscs_compound_end
    mov x0, x19
    bl _kcc_sema_check_stmt
    ldr x19, [x19, #AST_F4]        // next
    b .Lscs_compound_loop

.Lscs_compound_end:
    bl _kcc_sema_check_unused_scope
    bl _kcc_symtab_pop_scope
    b .Lscs_done

.Lscs_expr:
    ldr x0, [x19, #AST_F0]
    cbz x0, .Lscs_done
    bl _kcc_sema_expr
    b .Lscs_done

.Lscs_if:
    ldr x0, [x19, #AST_F0]         // condition
    bl _kcc_sema_expr
    ldr x0, [x19, #AST_F1]         // then
    bl _kcc_sema_check_stmt
    ldr x0, [x19, #AST_F2]         // else (may be
   null)
    cbz x0, .Lscs_done
    bl _kcc_sema_check_stmt
    b .Lscs_done

.Lscs_while:
    ldr x0, [x19, #AST_F0]         // condition
    bl _kcc_sema_expr
    ldr x0, [x19, #AST_F1]         // body
    bl _kcc_sema_check_stmt
    b .Lscs_done

.Lscs_do:
    ldr x0, [x19, #AST_F0]         // body
    bl _kcc_sema_check_stmt
    ldr x0, [x19, #AST_F1]         // condition
    bl _kcc_sema_expr
    b .Lscs_done

.Lscs_for:
```

```
    bl _kcc_symtab_push_scope
    ldr x0, [x19, #AST_F0]          // init
    cbz x0, .Lscs_for_cond
    bl _kcc_sema_check_stmt
.Lscs_for_cond:
    ldr x0, [x19, #AST_F1]          // condition
    cbz x0, .Lscs_for_incr
    bl _kcc_sema_expr
.Lscs_for_incr:
    ldr x0, [x19, #AST_F2]          // increment
    cbz x0, .Lscs_for_body
    bl _kcc_sema_expr
.Lscs_for_body:
    ldr x0, [x19, #AST_F3]          // body
    cbz x0, .Lscs_for_end
    bl _kcc_sema_check_stmt
.Lscs_for_end:
    bl _kcc_sema_check_unused_scope
    bl _kcc_symtab_pop_scope
    b .Lscs_done
```

□

272 · defined at 271, 272, 273 · used in 30a · uses _kcc_sema_check_stmt 271, _kcc_sema_expr 274, _kcc_symtab_pop_scope 254a, _kcc_symtab_push_scope 253c

Return, Switch, Labels, and Declarations `return` type-checks its expression (if present) and enforces the two constraints of §6.8.6.4: a `return` with a value may not appear in a `void` function, and a bare `return` may appear *only* in a `void` function. Both are constraint violations the standard requires a diagnostic for, and no valid program triggers either, so kcc reports them as errors against the return statement's own location. The comparison reads `_kcc_sema_cur_ret`, the return-type descriptor the function-definition handler stashed before walking the body; when it is absent (a stray `return` outside any function) the check is skipped. `switch` checks the controlling expression and recurses into the body. Label, case, and default statements register or validate their labels and recurse into their associated statement. Any declaration node encountered inside a compound statement is forwarded to `kcc_sema_check_decl`.

⟨ *Sema: check statement* 273 ⟩ + ≡

```
.Lscs_return:
    ldr x0, [x19, #AST_F0]          // return
  expression
    cbz x0, .Lscs_ret_novalue

    // return <expr>; - type-check the expression,
    then enforce that a
    // value is not returned from a void function
  (\stdref{6.8.6.4}).
    bl _kcc_sema_expr               // x0 =
  returned expression type

    // -Wall -Wint-conversion: return type vs
  expression type
    adrp x1, _kcc_sema_cur_ret@PAGE
    add  x1, x1, _kcc_sema_cur_ret@PAGEOFF
    ldr  x1, [x1]                   // dst =
  current return type
    mov  x2, x0                     // src type
    mov  x0, x19                    // location (
  return node)
    ldr  x3, [x19, #AST_F0]         // src node
    bl _kcc_sema_check_int_ptr
    adrp x0, _kcc_sema_cur_ret@PAGE
    add  x0, x0, _kcc_sema_cur_ret@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lscs_done             // no current
  return type: nothing to check
    ldr  x1, [x0, #TY_KIND]
    cmp  x1, #TY_VOID
    b.ne .Lscs_done
    ldr  x0, [x19, #AST_LINE]
    ldr  x1, [x19, #AST_COL]
    adrp x2, _kcc_sema_ret_void@PAGE
    add  x2, x2, _kcc_sema_ret_void@PAGEOFF
    bl   _kcc_diag_error
    b .Lscs_done

.Lscs_ret_novalue:
    // bare return; - a value is required unless
  the function returns void.
    adrp x0, _kcc_sema_cur_ret@PAGE
    add  x0, x0, _kcc_sema_cur_ret@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lscs_done
    ldr  x1, [x0, #TY_KIND]
    cmp  x1, #TY_VOID
    b.eq .Lscs_done
    ldr  x0, [x19, #AST_LINE]
    ldr  x1, [x19, #AST_COL]
    adrp x2, _kcc_sema_ret_nonvoid@PAGE
    add  x2, x2, _kcc_sema_ret_nonvoid@PAGEOFF
    bl   _kcc_diag_error
    b .Lscs_done

.Lscs_switch:
    ldr x0, [x19, #AST_F0]          // controlling
  expression
    bl _kcc_sema_expr
    // TODO: verify integer type
    ldr x0, [x19, #AST_F1]          // body
    bl _kcc_sema_check_stmt
    b .Lscs_done

.Lscs_label:
    // Register label
    ldr x0, [x19, #AST_F0]          // label
  name_ptr
    cbz x0, .Lscs_label_stmt
    mov x20, x0
    bl _kcc_strlen
    mov x1, x0
    mov x0, x20
    bl _kcc_symtab_label_add
.Lscs_label_stmt:
    ldr x0, [x19, #AST_F1]          // labeled
  statement
    cbz x0, .Lscs_done
    bl _kcc_sema_check_stmt
    b .Lscs_done

.Lscs_case:
    ldr x0, [x19, #AST_F0]          // case
  expression (value or range low bound)
    bl _kcc_sema_expr
    ldr x0, [x19, #AST_F3]          // GNU range
  high bound (0 = single case)
    cbz x0, .Lscs_case_body
    bl _kcc_sema_expr
.Lscs_case_body:
    ldr x0, [x19, #AST_F1]          // statement
    cbz x0, .Lscs_done
    bl _kcc_sema_check_stmt
    b .Lscs_done

.Lscs_default:
    ldr x0, [x19, #AST_F0]          // statement
    cbz x0, .Lscs_done
```

```
    bl _kcc_sema_check_stmt
    b .Lscs_done

.Lscs_decl:
    mov x0, x19
    bl _kcc_sema_check_decl
    b .Lscs_done

.Lscs_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

273 · defined at 271, 272, 273 · used in 30a · uses _kcc_diag_error 76a, _kcc_sema_check_decl 264, _kcc_sema_check_stmt 271, _kcc_sema_cur_ret 263, _kcc_sema_expr 274, _kcc_symtab_label_add 257b

26.7. Expression Type-Checking

The expression type-checker is the core of semantic analysis. It dispatches on the AST tag, delegates to specialised handlers for each expression kind, and stores the resolved type in the node's AST_F3 field. Simple expressions are handled inline: parenthesised expressions unwrap to their child, comma expressions return the type of the right operand, conditional expressions return the type of the *then* branch, and subscript expressions dereference one level of pointer or array type. More complex expressions (identifiers, binary, unary, assignment, calls, member access, casts, sizeof) are delegated to dedicated subroutines described in the following subsections.

⟨ *Sema: check expression* 274 ⟩ ≡

```
// -- kcc_sema_expr ---
// type_desc *kcc_sema_expr(node *expr)
//   x0 = expression AST node
// Type-checks the expression, stores result in
   AST_F3.
// Returns the resolved type.
.text
.globl _kcc_sema_expr
.p2align 2
_kcc_sema_expr:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    cbz x0, .Lse_null
    mov x19, x0

    ldr x20, [x19, #AST_TAG]

    cmp x20, #AST_IDENT_EXPR
    b.eq .Lse_ident
    cmp x20, #AST_INT_LIT
    b.eq .Lse_int_lit
    cmp x20, #AST_FLOAT_LIT
    b.eq .Lse_int_lit              // typed by
  _kcc_sema_expr_literal →( double)
    cmp x20, #AST_STR_LIT
    b.eq .Lse_str_lit
    cmp x20, #AST_CHAR_LIT
    b.eq .Lse_char_lit
    cmp x20, #AST_BINARY_EXPR
    b.eq .Lse_binary
    cmp x20, #AST_UNARY_EXPR
    b.eq .Lse_unary
    cmp x20, #AST_ASSIGN_EXPR
    b.eq .Lse_assign
    cmp x20, #AST_COND_EXPR
    b.eq .Lse_cond
    cmp x20, #AST_CALL_EXPR
    b.eq .Lse_call
    cmp x20, #AST_SUBSCRIPT
    b.eq .Lse_subscript
    cmp x20, #AST_MEMBER
    b.eq .Lse_member
    cmp x20, #AST_CAST_EXPR
    b.eq .Lse_cast
    cmp x20, #AST_SIZEOF_EXPR
    b.eq .Lse_sizeof
    cmp x20, #AST_ALIGNOF_EXPR
    b.eq .Lse_alignof
    cmp x20, #AST_COMMA_EXPR
    b.eq .Lse_comma
    cmp x20, #AST_PAREN_EXPR
    b.eq .Lse_paren
    cmp x20, #AST_VA_ARG
    b.eq .Lse_va_arg
    cmp x20, #AST_TYPES_COMPAT
    b.eq .Lse_types_compat
    cmp x20, #AST_GENERIC
    b.eq .Lse_generic
    cmp x20, #AST_COMPOUND_LIT
    b.eq .Lse_compound_lit
    cmp x20, #AST_INIT_LIST
    b.eq .Lse_init_list
    cmp x20, #AST_STMT_EXPR
    b.eq .Lse_stmt_expr

    // Default: int
    b .Lse_default_int

.Lse_init_list:
    // Type every element of a brace initializer
   so each leaf expression carries
    // a resolved type in AST_F3.  Sema otherwise
   never descends into a list, so
    // an array-valued element stayed untyped and
   the IR builder could not tell
    // it had to decay to a pointer: a local `S s
   = { arr, ... }` then stored a
    // loaded int where the pointer member
   belonged, and the member pointed at
    // garbage.  Recursion handles nested braces;
   a designator node types
    // harmlessly (its value follows in the AST_F4
    chain and is typed in turn).
    ldr x0, [x19, #AST_F0]         // first element
.Lse_il_loop:
    cbz x0, .Lse_default_int       // list itself
   has no scalar type → int
    mov x20, x0                     // cursor (
   callee-saved across the recursion)
    ldr x1, [x20, #AST_TAG]
    cmp x1, #AST_DESIGNATOR
    b.ne .Lse_il_type
    // A designated element (`.m = v` / `[i] = v`)
    stores its value initializer
    // on the head designator's AST_F3 (the
   designator chain links via AST_F2).
    // Type that value---NOT the designator node,
   whose AST_F3 would then be
    // overwritten with the int default,
   destroying the fill's value pointer.
    ldr x0, [x20, #AST_F3]         // designated
   value initializer
    cbz x0, .Lse_il_next
    bl _kcc_sema_expr
    b .Lse_il_next
.Lse_il_type:
    bl _kcc_sema_expr
```

```
.Lse_il_next:
    ldr x0, [x20, #AST_F4]        // next sibling
    b .Lse_il_loop

.Lse_compound_lit:
    // A compound literal's type is its
   parenthesised type name; storing it in
    // AST_F3 lets a postfix `.m`/`[i]`/pass-by-
   value see a real aggregate.
    ldr x0, [x19, #AST_F0]        // the type node
    cbz x0, .Lse_default_int
    bl _kcc_sema_resolve_type
    b .Lse_store

.Lse_generic:
    mov x0, x19
    bl _kcc_sema_expr_generic
    b .Lse_store

.Lse_va_arg:
    // Type the va_list operand; the va_arg node's
    type is the requested type.
    ldr x0, [x19, #AST_F0]        // va_list
   expression
    cbz x0, .Lse_va_arg_type
    bl _kcc_sema_expr
.Lse_va_arg_type:
    ldr x0, [x19, #AST_F1]        // target type
   node
    cbz x0, .Lse_default_int
    bl _kcc_sema_resolve_type
    b .Lse_store

.Lse_ident:
    bl _kcc_sema_expr_ident
    b .Lse_store
.Lse_int_lit:
    bl _kcc_sema_expr_literal
    b .Lse_store
.Lse_str_lit:
    bl _kcc_sema_expr_literal
    b .Lse_store
.Lse_char_lit:
    bl _kcc_sema_expr_literal
    b .Lse_store
.Lse_binary:
    bl _kcc_sema_expr_binary
    b .Lse_store
.Lse_unary:
    bl _kcc_sema_expr_unary
    b .Lse_store
.Lse_assign:
    bl _kcc_sema_expr_assign
    b .Lse_store
.Lse_cond:
    // Check all three parts, result type = type
   of then-expr
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_expr
    ldr x0, [x19, #AST_F1]
    bl _kcc_sema_expr
    mov x20, x0                   // save then
   type
    ldr x0, [x19, #AST_F2]
    bl _kcc_sema_expr
    mov x0, x20
    b .Lse_store
.Lse_call:
    bl _kcc_sema_expr_call
    b .Lse_store
.Lse_subscript:
    // array[index]: check both, result = element
   type
    ldr x0, [x19, #AST_F0]        // array
    bl _kcc_sema_expr
    mov x20, x0
    ldr x0, [x19, #AST_F1]        // index
    bl _kcc_sema_expr
    // Dereference array/pointer type
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_PTR
    b.eq .Lse_sub_ptr
    cmp x1, #TY_ARRAY
    b.eq .Lse_sub_arr
    // Not a pointer or array, default to int
    b .Lse_default_int
.Lse_sub_ptr:
.Lse_sub_arr:
    ldr x0, [x20, #TY_F0]         // pointee/
   element type
    b .Lse_store
.Lse_member:
    bl _kcc_sema_expr_member
    b .Lse_store
.Lse_cast:
    bl _kcc_sema_expr_cast
    b .Lse_store
.Lse_sizeof:
    bl _kcc_sema_expr_sizeof
    b .Lse_store
.Lse_alignof:
    // _Alignof(type): validate the type; the
   result type is size_t.
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    adrp x0, _kcc_type_long_u@PAGE
    add  x0, x0, _kcc_type_long_u@PAGEOFF
    ldr  x0, [x0]
    b .Lse_store
.Lse_types_compat:
    // __builtin_types_compatible_p(type, type):
   resolve both operand types so
    // their descriptors exist for the constant
   folder; the result type is int.
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    ldr x0, [x19, #AST_F1]
    bl _kcc_sema_resolve_type
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
    b .Lse_store
.Lse_comma:
    // Evaluate LHS, result is type of RHS
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_expr
    ldr x0, [x19, #AST_F1]
    bl _kcc_sema_expr
    b .Lse_store
.Lse_paren:
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_expr
    b .Lse_store

.Lse_stmt_expr:
    // GNU statement expression `({ ... })`.  Type
   -check the compound block ---
    // this recursively types every inner
   statement, including the value-yielding
    // final expression.  The construct's type is
   that of its last item when that
    // item is an expression-statement; otherwise
   (a declaration, a control-flow
    // statement, or an empty block) it is void.
    ldr x0, [x19, #AST_F0]        // compound
   node
    cbz x0, .Lse_stmt_void
    mov x20, x0
    bl _kcc_sema_check_stmt       // types all
   inner statements
```

```
    ldr x1, [x20, #AST_F0]          // first item
   in the block
    cbz x1, .Lse_stmt_void
.Lse_se_last:
    ldr x2, [x1, #AST_F4]           // walk the
   sibling chain to the last item
    cbz x2, .Lse_se_have_last
    mov x1, x2
    b .Lse_se_last
.Lse_se_have_last:
    ldr x2, [x1, #AST_TAG]
    cmp x2, #AST_EXPR_STMT
    b.ne .Lse_stmt_void
    ldr x0, [x1, #AST_F0]           // the final
   expression
    cbz x0, .Lse_stmt_void
    ldr x0, [x0, #AST_F3]           // its resolved
    type
    cbz x0, .Lse_stmt_void
    b .Lse_store
.Lse_stmt_void:
    adrp x0, _kcc_type_void@PAGE
    add  x0, x0, _kcc_type_void@PAGEOFF
    ldr  x0, [x0]
    b .Lse_store

.Lse_default_int:
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]

.Lse_store:
    // Store resolved type in AST_F3 of the
   expression node
    str x0, [x19, #AST_F3]

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

.Lse_null:
    adrp x0, _kcc_type_void@PAGE
    add  x0, x0, _kcc_type_void@PAGEOFF
    ldr  x0, [x0]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

274 · defined at 274, 276 · used in 30a · defines _kcc_sema_expr 259b, 264, 271, 272, 273, 276, 284, 286, 287, 289, 290, 292ab, 426 · uses _kcc_sema_check_stmt 271, _kcc_sema_expr_generic 276, _kcc_sema_expr_member 290, _kcc_sema_resolve_type 259b, _kcc_type_int_s 250b, _kcc_type_long_u 250b, _kcc_type_void 250b

Generic selection. A `_Generic` expression (§6.5.1.1) is resolved entirely here, at type-check time: the controlling expression is *typed but never evaluated*, and its type chooses one association's expression, which alone is kept. The pass types the controlling expression (decaying an array to a pointer, as lvalue conversion requires), then walks the association list; the first association whose named type is compatible with the controlling type wins, and a `default` association is remembered as the fallback. Because `_kcc_type_compatible` matches on kind and signedness and ignores qualifiers, a `volatile int` controlling expression still selects the `int` association, as the Standard's lvalue conversion demands. The chosen expression is stashed in the node's spare field so the IR builder emits *only* it—never the controlling expression or the discarded arms—and its type becomes the type of the whole selection. A selection that matches nothing and has no default is a diagnosed error.

⟨ *Sema: check expression* 276 ⟩ + ≡

```
// -- kcc_sema_expr_generic ---
// uint64_t kcc_sema_expr_generic(node *generic)
// Resolve a _Generic selection: type the (
   unevaluated) controlling expression,
// pick the association whose type is compatible
   with it---or the default---and
// stash the chosen expression in AST_F2 for the
   IR builder.  Returns the chosen
// expression's type.
.text
.p2align 2
_kcc_sema_expr_generic:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // generic node

    // The controlling expression is not evaluated
   ; type it only.  An array or
    // function type decays before the match---
   handle the common array case.
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_expr
    mov x20, x0                     // controlling
   type
    cbz x20, .Lseg_walk
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.ne .Lseg_walk
    ldr x0, [x20, #TY_F0]           // element type
    bl _kcc_type_ptr
    mov x20, x0
.Lseg_walk:
    ldr x21, [x19, #AST_F1]         // current
   association
    mov x22, #0                      // remembered
   default expression (0 = none)
.Lseg_loop:
    cbz x21, .Lseg_use_default
    ldr x0, [x21, #AST_F0]          // association
   type node (0 = default)
    cbz x0, .Lseg_default
    bl _kcc_sema_resolve_type       // x0 =
   association type descriptor
    mov x1, x20
    bl _kcc_type_compatible
    cbz x0, .Lseg_next
    ldr x0, [x21, #AST_F1]          // compatible →
   this association's expression
    b .Lseg_select
.Lseg_default:
    ldr x22, [x21, #AST_F1]
.Lseg_next:
    ldr x21, [x21, #AST_F4]         // next
   association
    b .Lseg_loop
.Lseg_use_default:
    cbz x22, .Lseg_error
    mov x0, x22
.Lseg_select:
    str x0, [x19, #AST_F2]          // chosen
   expression, for the IR builder
    bl _kcc_sema_expr               // type-check it
    b .Lseg_done
.Lseg_error:
```

```
    ldr x0, [x19, #AST_LINE]
    mov x1, #0
    adrp x2, .Lseg_errmsg@PAGE
    add  x2, x2, .Lseg_errmsg@PAGEOFF
    bl _kcc_diag_error
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
.Lseg_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.section __TEXT,__cstring,cstring_literals
.Lseg_errmsg:
    .asciz "_Generic: no association compatible
   with the controlling type"
.text
```

□

276 · defined at 274, 276 · used in 30a · defines _kcc_sema_expr_generic 274 · uses _kcc_diag_error 76a, _kcc_sema_expr 274, _kcc_sema_resolve_type 259b, _kcc_type_compatible 247a, _kcc_type_int_s 250b, _kcc_type_ptr 242a

26.7.1. Identifier Expression

An identifier expression looks up the name in the symbol table. If found, the symbol's type is returned after applying function and array decay (per §6.3.2.1): a function designator decays to a pointer-to-function, and an array decays to a pointer to its first element. An undeclared identifier used as a *value* is a hard error (`use of undeclared identifier 'metaname'`); the one exception is a bare name in call position, which is taken as an implicit function declaration (legacy C) rather than an error—the call handler signals that case through `_kcc_sema_callee_ctx`. Under `-Wall` that implicit declaration is not silent: kcc emits an `implicit declaration of function 'metaname'` warning (but still compiles, assuming an `int`-returning signature). Either way the node's type falls back to `int` so the pass keeps running and reports any further errors.

The named-message helpers. Several diagnostics have the shape `metaprefix metaname'` — an undeclared identifier, a missing member, a redefinition, an implicit declaration, an unused variable. `_kcc_sema_build_named` assembles that body once in the arena (the prefix carries its own opening quote; the closing quote and NUL are appended), and two thin wrappers hand it to the diagnostic engine: `_kcc_sema_err_named` as an error, `_kcc_sema_warn_named` as a warning. Splitting error from warning at this seam is what lets the `-Wall` family reuse the exact message machinery the hard errors already use.

Enumeration constants receive special treatment. When the lookup returns a symbol whose kind is `SYM_ENUM_CONST`, the AST node is rewritten in place: its tag becomes `AST_INT_LIT` and `F0` receives the constant's integer value from `SYM_VALUE`. The IR builder then sees a plain integer literal and emits `IR_CONST`—no global reference, no symbol lookup at link time. This rewrite is safe because the node will never be examined as an identifier again.

⟨ *Sema: expr ident* 277 ⟩ ≡

```
// -- kcc_sema_expr_ident ---
// type_desc *kcc_sema_expr_ident(node *expr)
//   x19 = expression node (already loaded by
   caller)
// Looks up identifier in symbol table, returns
   its type.
.text
.p2align 2
_kcc_sema_expr_ident:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // x19 = expr node (from caller)
    // Get name pointer
    ldr x20, [x19, #AST_F0]        // x20 =
   name_ptr
    cbz x20, .Lsei_unknown

    mov x0, x20
    bl _kcc_strlen
    mov x1, x0                     // len
    mov x0, x20                    // name

    bl _kcc_symtab_lookup
    cbz x0, .Lsei_undeclared

    // Check if this is an enum constant
    ldr x1, [x0, #SYM_KIND]
    cmp x1, #SYM_ENUM_CONST
    b.ne .Lsei_normal

    // Rewrite AST node: AST_IDENT_EXPR →
   AST_INT_LIT with enum value
    ldr x1, [x0, #SYM_VALUE]
    mov x0, #AST_INT_LIT
    str x0, [x19]                   // overwrite
   tag
    str x1, [x19, #AST_F0]         // store
   constant value
    b .Lsei_int_type

.Lsei_normal:
    // Mark the symbol used (for -Wunused-variable
   ); harmless for functions.
    ldr w1, [x0, #SYM_FLAGS]
    orr w1, w1, #2
    str w1, [x0, #SYM_FLAGS]
    // Return symbol's type
    ldr x0, [x0, #SYM_TYPE]
    // Apply function/array decay
    bl _kcc_conv_func_decay
    bl _kcc_conv_array_decay

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

.Lsei_undeclared:
    // A bare identifier used as a value must be
   declared.  When it is the
    // direct target of a call we defer to
   implicit-function-declaration
    // leniency (legacy C: \texttt{printf()}
   without a prototype); otherwise it
    // is a hard error.  Either way we fall
   through to the int fallback so the
```

```
    // rest of the pass keeps running and reports
   further errors.
    adrp x0, _kcc_sema_callee_ctx@PAGE
    add  x0, x0, _kcc_sema_callee_ctx@PAGEOFF
    ldr  x0, [x0]
    cbz x0, .Lsei_emit_undecl
    // Implicit function declaration: remember it
   so the call handler does not
    // then complain that the callee is not a
   function.
    adrp x1, _kcc_sema_callee_implicit@PAGE
    add  x1, x1, _kcc_sema_callee_implicit@PAGEOFF
    mov  x2, #1
    str  x2, [x1]
    // -Wall: warn about the implicit declaration
   (still lenient - compilation
    // continues with an int-returning signature).
    adrp x0, _kcc_warn_all@PAGE
    add  x0, x0, _kcc_warn_all@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lsei_int_type
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    mov x2, x20                         // name
    adrp x3, _kcc_sema_implicit_pre@PAGE
    add  x3, x3, _kcc_sema_implicit_pre@PAGEOFF
    bl _kcc_sema_warn_named
    b .Lsei_int_type
.Lsei_emit_undecl:
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    mov x2, x20                         // name
    adrp x3, _kcc_sema_undecl_pre@PAGE
    add  x3, x3, _kcc_sema_undecl_pre@PAGEOFF
    bl _kcc_sema_err_named

.Lsei_unknown:
.Lsei_int_type:
    // Return int as fallback / enum constant type
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_sema_build_named ---
// char *kcc_sema_build_named(const char *name,
   const char *prefix)
//   x0 = name, x1 = prefix ; returns x0 = arena
   "<prefix>name'"
// Builds the quoted message body once (the prefix
    supplies its own opening
// quote); the closing quote and NUL are appended
   here.  Shared by the error
// and warning wrappers below.
.text
.p2align 2
_kcc_sema_build_named:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x21, x0                         // name
    mov x22, x1                         // prefix
    mov x0, x22
    bl _kcc_strlen
    mov x24, x0                         // prefix_len
    mov x0, x21
    bl _kcc_strlen
    mov x20, x0                         // name_len
    add x0, x24, x20
    add x0, x0, #2                      // + closing
   quote + NUL
    bl _kcc_arena_alloc
    mov x23, x0                         // buffer
    mov x0, x23
    mov x1, x22
    mov x2, x24
    bl _kcc_memcpy                      // prefix
    add x0, x23, x24
    mov x1, x21
    mov x2, x20
    bl _kcc_memcpy                      // name
    add x9, x23, x24
    add x9, x9, x20
    mov w10, #0x27                      // '\''
    strb w10, [x9]
    strb wzr, [x9, #1]
    mov x0, x23                         // return
   buffer
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_sema_err_named ---
// void kcc_sema_err_named(uint64_t line, uint64_t
    col,
//                         const char *name, const
    char *prefix)
// Emits "<prefix>name'" as an error at line:col.
    Shared by the
// undeclared-identifier, no-member, and
   redefinition diagnostics.
.globl _kcc_sema_err_named
.p2align 2
_kcc_sema_err_named:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                         // line
    mov x20, x1                         // col
    mov x0, x2                          // name
    mov x1, x3                          // prefix
    bl _kcc_sema_build_named
    mov x2, x0
    mov x0, x19
    mov x1, x20
    bl _kcc_diag_error
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_sema_warn_named ---
// void kcc_sema_warn_named(uint64_t line,
   uint64_t col,
//                          const char *name,
   const char *prefix)
// Same message body as err_named, emitted as a
   warning.  Used by the
// -Wall family (implicit function declaration,
   unused variable, ...).
.globl _kcc_sema_warn_named
.p2align 2
_kcc_sema_warn_named:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                         // line
    mov x20, x1                         // col
    mov x0, x2                          // name
    mov x1, x3                          // prefix
    bl _kcc_sema_build_named
    mov x2, x0
```

```
    mov x0, x19
    mov x1, x20
    bl _kcc_diag_warning
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_sema_check_unused_scope ---
// void kcc_sema_check_unused_scope(void)
// Under -Wall, walk the innermost ordinary scope
   and warn for each SYM_VAR
// that was declared but never referenced.  Called
    just before a block or
// for-scope is popped; the function/parameter
   scope is deliberately not
// checked (unused parameters are outside -Wall).
.globl _kcc_sema_check_unused_scope
.p2align 2
_kcc_sema_check_unused_scope:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]
    adrp x0, _kcc_warn_all@PAGE
    add  x0, x0, _kcc_warn_all@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lcus_done
    bl _kcc_symtab_current_ht
    cbz x0, .Lcus_done
    ldr x19, [x0, #HT_ENTRIES]      // entries base
    ldr x20, [x0, #HT_CAP]          // capacity
    mov x21, #0                     // i
.Lcus_loop:
    cmp x21, x20
    b.hs .Lcus_done
    mov x22, #HTE_SIZE
    mul x22, x21, x22
    add x22, x19, x22               // entry ptr
    ldr x0, [x22, #HTE_KEY]
    cbz x0, .Lcus_next              // empty slot
    ldr x23, [x22, #HTE_VALUE]      // sym
    cbz x23, .Lcus_next
    ldr x0, [x23, #SYM_KIND]
    cmp x0, #SYM_VAR
    b.ne .Lcus_next
    ldr w0, [x23, #SYM_FLAGS]
    tst w0, #2                      // used bit set
   ?
    b.ne .Lcus_next
    ldr x0, [x23, #SYM_LINE]
    ldr x1, [x23, #SYM_COL]
    ldr x2, [x23, #SYM_NAME]
    adrp x3, _kcc_sema_unused_pre@PAGE
    add  x3, x3, _kcc_sema_unused_pre@PAGEOFF
    bl _kcc_sema_warn_named
.Lcus_next:
    add x21, x21, #1
    b .Lcus_loop
.Lcus_done:
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_sema_check_redef ---
// void kcc_sema_check_redef(node *decl, const
   char *name)
// At block scope (depth >= 2; file scope is depth
    1) a name already present
// in the current scope is a redefinition, since a
    block has no tentative
// definitions.  Emits "redefinition of 'name'" at
    the decl node's location.
.text
.globl _kcc_sema_check_redef
.p2align 2
_kcc_sema_check_redef:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                     // node
    mov x20, x1                     // name
    adrp x0, _kcc_sym_depth@PAGE
    add  x0, x0, _kcc_sym_depth@PAGEOFF
    ldr  x0, [x0]
    cmp  x0, #2
    b.lt .Lscr_done
    mov x0, x20
    bl _kcc_strlen
    mov x1, x0
    mov x0, x20
    bl _kcc_symtab_lookup_current
    cbz x0, .Lscr_done
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    mov x2, x20                     // name
    adrp x3, _kcc_sema_redef_pre@PAGE
    add  x3, x3, _kcc_sema_redef_pre@PAGEOFF
    bl _kcc_sema_err_named
.Lscr_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_agg_leaf_count ---
// uint64_t kcc_agg_leaf_count(type *ty)
// Number of scalar leaves a fully brace-elided
   initializer fills for `ty`
// (\stdref{6.7.9p20}): 1 for a scalar, length*
   leaves for an array, the sum of
// its fields' leaves for a struct, the first
   member's leaves for a union.
.text
.globl _kcc_agg_leaf_count
.p2align 2
_kcc_agg_leaf_count:
    cbz x0, .Lalc_one
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.eq .Lalc_array
    cmp x1, #TY_STRUCT
    b.eq .Lalc_struct
    cmp x1, #TY_UNION
    b.eq .Lalc_union
.Lalc_one:
    mov x0, #1
    ret
.Lalc_array:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    ldr x20, [x0, #TY_F1]           // length
    ldr x0, [x0, #TY_F0]            // element type
    bl _kcc_agg_leaf_count
    mul x0, x0, x20
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
.Lalc_struct:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    ldr x19, [x0, #TY_F0]           // struct decl
    cbz x19, .Lalc_s_one
```

```
    ldr x19, [x19, #AST_F1]        // first field
    mov x20, #0                    // leaf
   accumulator
.Lalc_s_loop:
    cbz x19, .Lalc_s_ret
    ldr x0, [x19, #AST_F0]         // field type
   spec
    bl _kcc_sema_resolve_type
    bl _kcc_agg_leaf_count
    add x20, x20, x0
    ldr x19, [x19, #AST_F4]
    b .Lalc_s_loop
.Lalc_s_one:
    mov x20, #1
.Lalc_s_ret:
    mov x0, x20
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
.Lalc_union:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    ldr x1, [x0, #TY_F0]           // union decl
    cbz x1, .Lalc_u_one
    ldr x1, [x1, #AST_F1]          // first member
    (elision fills only it)
    cbz x1, .Lalc_u_one
    ldr x0, [x1, #AST_F0]          // its type
   spec
    bl _kcc_sema_resolve_type
    bl _kcc_agg_leaf_count
    ldp x29, x30, [sp], #32
    ret
.Lalc_u_one:
    mov x0, #1
    ldp x29, x30, [sp], #32
    ret

// -- kcc_infer_arr_elems ---
// uint64_t kcc_infer_arr_elems(node *head, type *
   elem)
// Count how many array elements a brace-elided
   initializer list `head..`
// forms when each element has aggregate type `
   elem`: an item with its own
// brace consumes one element, otherwise a run of
   leaf-count scalars does
// (\stdref{6.7.9p20}).  Used to size `struct P a
   [] = { 1,2,3, 4,5,6 }` at 2,
// not 6.
.text
.globl _kcc_infer_arr_elems
.p2align 2
_kcc_infer_arr_elems:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                    // item cursor
    mov x0, x1
    bl _kcc_agg_leaf_count
    mov x22, x0                    // leaves per
   element
    mov x21, #0                    // element
   count
.Liae_loop:
    cbz x19, .Liae_ret
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_INIT_LIST         // element
   wrote its own brace → one item
    b.eq .Liae_braced
    cmp x0, #AST_DESIGNATOR        // a designator
    also occupies one slot
    b.eq .Liae_braced
    // Brace elision: this element consumes up to
   `leaves` scalar items.
    mov x20, x22
.Liae_skip:
    cbz x20, .Liae_count
    cbz x19, .Liae_count
    ldr x19, [x19, #AST_F4]
    sub x20, x20, #1
    b .Liae_skip
    b .Liae_count
.Liae_braced:
    ldr x19, [x19, #AST_F4]
.Liae_count:
    add x21, x21, #1
    b .Liae_loop
.Liae_ret:
    mov x0, x21
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_sema_infer_array_len ---
// void kcc_sema_infer_array_len(node *decl)
// If the declaration's type is an incomplete
   array (an AST_TYPE_ARRAY with no
// size expression) and it has an initializer,
   synthesise an AST_INT_LIT
// length and hang it on the type node's size-expr
    slot, so that every later
// resolve of that node yields a *complete* array
   type.  A string literal
// gives strlen+1; a brace list of scalars gives
   its element span
// (designator-aware); a brace-elided list of
   aggregates is grouped by the
// element's leaf count.
.globl _kcc_sema_infer_array_len
.p2align 2
_kcc_sema_infer_array_len:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    ldr x19, [x0, #AST_F0]          // type node
    cbz x19, .Lial_done
    ldr x1, [x19, #AST_TAG]
    cmp x1, #AST_TYPE_ARRAY
    b.ne .Lial_done
    ldr x1, [x19, #AST_F1]          // existing
   size expr?
    cbnz x1, .Lial_done             // already a
   complete array
    ldr x20, [x0, #AST_F2]          // initializer
    cbz x20, .Lial_done
    ldr x1, [x20, #AST_TAG]
    cmp x1, #AST_STR_LIT
    b.eq .Lial_str
    cmp x1, #AST_INIT_LIST
    b.eq .Lial_list
    b .Lial_done
.Lial_str:
    ldr x21, [x20, #AST_F1]         // string
   length
    add x21, x21, #1               // + terminating
    NUL
    b .Lial_make
.Lial_list:
    // When the array element is itself an
   aggregate, a brace-elided list groups
    // several scalars into each element, so a
   flat count would over-size the
    // array.  Resolve the element type and, for
```

```
    an aggregate, count grouped
     // elements; a scalar element keeps the plain
    designator-aware span.
     mov x0, x19                    //
    AST_TYPE_ARRAY node
     bl _kcc_sema_resolve_type
     cbz x0, .Lial_list_flat
     ldr x0, [x0, #TY_F0]           // element type
     cbz x0, .Lial_list_flat
     ldr x1, [x0, #TY_KIND]
     cmp x1, #TY_STRUCT
     b.eq .Lial_list_agg
     cmp x1, #TY_UNION
     b.eq .Lial_list_agg
     cmp x1, #TY_ARRAY
     b.eq .Lial_list_agg
.Lial_list_flat:
     mov x0, x20
     bl _kcc_ir_initlist_count      // element span
     (max index + 1)
     mov x21, x0
     b .Lial_make
.Lial_list_agg:
     mov x1, x0                     // element type
     ldr x0, [x20, #AST_F0]         // init-list
    head
     bl _kcc_infer_arr_elems
     mov x21, x0
.Lial_make:
     mov x0, #AST_INT_LIT
     bl _kcc_ast_alloc
     str x21, [x0, #AST_F0]         // literal value
     = inferred length
     str x0, [x19, #AST_F1]         // type node's
    size expr
.Lial_done:
     ldr x21, [sp, #32]
     ldp x19, x20, [sp, #16]
     ldp x29, x30, [sp], #48
     ret
```

□

277 · used in 30a · uses _kcc_arena_alloc 58a, _kcc_ast_alloc 168a, _kcc_conv_array_decay 296a, _kcc_conv_func_decay 296b, _kcc_diag_error 76a, _kcc_diag_warning 76b, _kcc_ir_initlist_count 414, _kcc_sema_callee_ctx 263, _kcc_sema_callee_implicit 263, _kcc_sema_redef_pre 263, _kcc_sema_resolve_type 259b, _kcc_symtab_current_ht 255, _kcc_symtab_lookup 255, _kcc_symtab_lookup_current 255, _kcc_sym_depth 258, _kcc_type_int_s 250b, _kcc_warn_all 77a

26.7.2. Reaching the End of a Non-Void Function

`-Wreturn-type` warns when control can run off the bottom of a function whose return type is not `void`, because the value it then hands back is indeterminate. The whole difficulty is doing this *without* false positives: a function that ends in an exhaustive `if`/`else` where both arms return, or in an infinite loop, or in a call to `exit`, never actually falls through, and warning about it would be worse than saying nothing. So the analysis is built the sound way round: `kcc_sema_falls_through` returns true only when it can *prove* control reaches the bottom of a statement, and the warning fires only on that proof. Anything it cannot analyse it treats as "might not fall through," which costs a missed warning but never a spurious one.

The recursion mirrors the language's control flow. A `return`, `goto`, `break`, or `continue` does not fall through. A block falls through only if every one of its statements does—the first statement that diverts control ends the block. An `if` falls through if it lacks an `else` (the untaken path continues) or if either arm falls through. A `while`/`for`/`do` whose controlling expression is a constant truth and whose body contains no `break` that would leave *this* loop is an infinite loop and does not fall through; any other loop is assumed to be exitable. A `switch` is treated conservatively as possibly terminating (so an exhaustive returning `switch` is never falsely flagged), and an expression statement that is a call to a known `_Noreturn` function (`exit`, `abort`, and kin) does not fall through. `main` is exempt entirely, since falling off its end is defined to return zero (§5.1.2.2.3).

`kcc_sema_has_toplevel_break` supports the loop case: it looks for a `break` that belongs to the loop being analysed, stopping its descent at any nested loop or `switch`, whose own `break`s bind there instead.

⟨ *Sema: return path* 281 ⟩ ≡

```
// -- kcc_sema_const_true ---
// int kcc_sema_const_true(node *expr)
// 1 iff expr is (parenthesised) integer/char
   constant with a non-zero value.
.text
.p2align 2
_kcc_sema_const_true:
    cbz x0, .Lct_no
.Lct_unwrap:
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_PAREN_EXPR
    b.ne .Lct_check
    ldr x0, [x0, #AST_F0]
    cbz x0, .Lct_no
    b .Lct_unwrap
.Lct_check:
    cmp x1, #AST_INT_LIT
    b.eq .Lct_int
    cmp x1, #AST_CHAR_LIT
    b.eq .Lct_int
    mov x0, #0
    ret
.Lct_int:
    ldr x0, [x0, #AST_F0]          // value
    cmp x0, #0
    cset x0, ne
    ret
.Lct_no:
    mov x0, #0
    ret

// -- kcc_sema_has_toplevel_break ---
// int kcc_sema_has_toplevel_break(node *s)
// 1 iff s contains a break that would exit the *
   enclosing* loop, i.e. a
// break not itself inside a nested loop or switch
   .
.p2align 2
_kcc_sema_has_toplevel_break:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0
    cbz x19, .Lhb_no
    ldr x20, [x19, #AST_TAG]
    cmp x20, #AST_BREAK_STMT
    b.eq .Lhb_yes
    cmp x20, #AST_WHILE_STMT
    b.eq .Lhb_no
```

```
    cmp x20, #AST_FOR_STMT
    b.eq .Lhb_no
    cmp x20, #AST_DO_STMT
    b.eq .Lhb_no
    cmp x20, #AST_SWITCH_STMT
    b.eq .Lhb_no
    cmp x20, #AST_COMPOUND
    b.eq .Lhb_compound
    cmp x20, #AST_IF_STMT
    b.eq .Lhb_if
    cmp x20, #AST_LABEL_STMT
    b.eq .Lhb_inner_f1
    cmp x20, #AST_CASE_STMT
    b.eq .Lhb_inner_f1
    cmp x20, #AST_DEFAULT_STMT
    b.eq .Lhb_inner_f0
    b .Lhb_no
.Lhb_yes:
    mov x0, #1
    b .Lhb_ret
.Lhb_no:
    mov x0, #0
    b .Lhb_ret
.Lhb_compound:
    ldr x19, [x19, #AST_F0]
.Lhb_cl:
    cbz x19, .Lhb_no
    mov x0, x19
    bl _kcc_sema_has_toplevel_break
    cbnz x0, .Lhb_yes
    ldr x19, [x19, #AST_F4]
    b .Lhb_cl
.Lhb_if:
    ldr x0, [x19, #AST_F1]          // then
    bl _kcc_sema_has_toplevel_break
    cbnz x0, .Lhb_yes
    ldr x0, [x19, #AST_F2]          // else
    cbz x0, .Lhb_no
    bl _kcc_sema_has_toplevel_break
    b .Lhb_ret
.Lhb_inner_f1:
    ldr x0, [x19, #AST_F1]
    bl _kcc_sema_has_toplevel_break
    b .Lhb_ret
.Lhb_inner_f0:
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_has_toplevel_break
.Lhb_ret:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_sema_is_noreturn_call ---
// int kcc_sema_is_noreturn_call(node *expr)
// 1 iff expr is a direct call to a known
   _Noreturn library function.
.p2align 2
_kcc_sema_is_noreturn_call:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0
    cbz x19, .Linr_no
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_CALL_EXPR
    b.ne .Linr_no
    ldr x0, [x19, #AST_F0]          // callee
    cbz x0, .Linr_no
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_IDENT_EXPR
    b.ne .Linr_no
    ldr x19, [x0, #AST_F0]          // callee name
    cbz x19, .Linr_no
    adrp x20, _kcc_sema_noreturn_tbl@PAGE
    add  x20, x20, _kcc_sema_noreturn_tbl@PAGEOFF
.Linr_loop:
    ldr x1, [x20]
    cbz x1, .Linr_no
    mov x0, x19
    bl _kcc_strcmp
    cbz x0, .Linr_yes
    add x20, x20, #8
    b .Linr_loop
.Linr_yes:
    mov x0, #1
    b .Linr_ret
.Linr_no:
    mov x0, #0
.Linr_ret:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_sema_falls_through ---
// int kcc_sema_falls_through(node *s)
// 1 iff control can be *proven* to reach the
   bottom of statement s.
.p2align 2
_kcc_sema_falls_through:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0
    cbz x19, .Lft_yes               // null/empty
   statement falls through
    ldr x20, [x19, #AST_TAG]
    cmp x20, #AST_RETURN_STMT
    b.eq .Lft_no
    cmp x20, #AST_GOTO_STMT
    b.eq .Lft_no
    cmp x20, #AST_BREAK_STMT
    b.eq .Lft_no
    cmp x20, #AST_CONTINUE_STMT
    b.eq .Lft_no
    cmp x20, #AST_SWITCH_STMT
    b.eq .Lft_no                    // conservative
   : never falsely flag a switch
    cmp x20, #AST_COMPOUND
    b.eq .Lft_compound
    cmp x20, #AST_IF_STMT
    b.eq .Lft_if
    cmp x20, #AST_LABEL_STMT
    b.eq .Lft_label
    cmp x20, #AST_WHILE_STMT
    b.eq .Lft_while
    cmp x20, #AST_FOR_STMT
    b.eq .Lft_for
    cmp x20, #AST_DO_STMT
    b.eq .Lft_do
    cmp x20, #AST_EXPR_STMT
    b.eq .Lft_expr
    // everything else (declarations, plain labels
   , case/default wrappers) is
    // straight-line code that falls through.
    b .Lft_yes
.Lft_no:
    mov x0, #0
    b .Lft_ret
.Lft_yes:
    mov x0, #1
    b .Lft_ret
.Lft_compound:
    ldr x19, [x19, #AST_F0]         // first item
.Lft_comp_loop:
    cbz x19, .Lft_yes               // ran off the
   end → falls through
    mov x0, x19
    bl _kcc_sema_falls_through
    cbz x0, .Lft_no                 // a diverting
   statement ends the block
    ldr x19, [x19, #AST_F4]         // next item
    b .Lft_comp_loop
```

```
.Lft_if:
    ldr x0, [x19, #AST_F2]          // else
    cbz x0, .Lft_yes                // no else → the untaken path continues
    ldr x0, [x19, #AST_F1]          // then
    bl _kcc_sema_falls_through
    cbnz x0, .Lft_yes
    ldr x0, [x19, #AST_F2]          // else
    bl _kcc_sema_falls_through
    b .Lft_ret
.Lft_label:
    ldr x0, [x19, #AST_F1]          // labelled statement
    bl _kcc_sema_falls_through
    b .Lft_ret
.Lft_while:
    ldr x0, [x19, #AST_F0]          // cond
    bl _kcc_sema_const_true
    cbz x0, .Lft_yes                // exitable loop → may fall through
    ldr x0, [x19, #AST_F1]          // body
    bl _kcc_sema_has_toplevel_break
    cbnz x0, .Lft_yes
    b .Lft_no                       // infinite, no break
.Lft_for:
    ldr x0, [x19, #AST_F1]          // cond
    cbz x0, .Lft_for_inf            // for(;;) → no condition → infinite
    bl _kcc_sema_const_true
    cbz x0, .Lft_yes
.Lft_for_inf:
    ldr x0, [x19, #AST_F3]          // body
    bl _kcc_sema_has_toplevel_break
    cbnz x0, .Lft_yes
    b .Lft_no
.Lft_do:
    ldr x0, [x19, #AST_F1]          // cond
    bl _kcc_sema_const_true
    cbz x0, .Lft_yes
    ldr x0, [x19, #AST_F0]          // body
    bl _kcc_sema_has_toplevel_break
    cbnz x0, .Lft_yes
    b .Lft_no
.Lft_expr:
    ldr x0, [x19, #AST_F0]          // expression
    bl _kcc_sema_is_noreturn_call
    cbnz x0, .Lft_no
    b .Lft_yes
.Lft_ret:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_sema_check_return_path ---
// void kcc_sema_check_return_path(node *func)
// Under -Wall, warn if a non-void, non-main function can fall off its end.
// The return type is read from _kcc_sema_cur_ret, which the body handler set
// (the TY_FUNC register is reused by the parameter loop and is not reliable
// here).
.globl _kcc_sema_check_return_path
.p2align 2
_kcc_sema_check_return_path:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                     // func node
    adrp x0, _kcc_warn_all@PAGE
    add  x0, x0, _kcc_warn_all@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lcrp_done
    adrp x0, _kcc_sema_cur_ret@PAGE
    add  x0, x0, _kcc_sema_cur_ret@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lcrp_done
    ldr  x1, [x0, #TY_KIND]
    cmp  x1, #TY_VOID
    b.eq .Lcrp_done
    ldr  x0, [x19, #AST_F1]         // name
    cbz  x0, .Lcrp_done
    adrp x1, _kcc_sema_main_name@PAGE
    add  x1, x1, _kcc_sema_main_name@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lcrp_done              // main is exempt
    ldr x0, [x19, #AST_F3]          // body
    cbz x0, .Lcrp_done              // no body (prototype)
    bl _kcc_sema_falls_through
    cbz x0, .Lcrp_done              // provably returns on all paths
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    adrp x2, _kcc_sema_reach_end@PAGE
    add  x2, x2, _kcc_sema_reach_end@PAGEOFF
    bl _kcc_diag_warning
.Lcrp_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

281 · used in 30a · uses _kcc_diag_warning 76b, _kcc_sema_cur_ret 263, _kcc_warn_all 77a

26.7.3. Literal Expressions

A character literal and an unsuffixed integer literal have type `int`. A suffixed integer literal takes the type its suffix names (§6.4.4.1): the parser recorded `u`/`U` and the `l`/`L` count as flags in `AST_F2`, and here we decode them—bit 0 selects the unsigned variant, and the long rank in bits 1–2 chooses `int`, `long`, or `long long`. A char literal leaves `AST_F2` zero and so still types as `int`. This is a deliberately *suffix-only* typing: it does not yet promote an unsuffixed constant whose magnitude overflows `int` to a wider type, which is a separate value-directed rule. String literals have type `char *` (pointer to signed char), matching the C17 rule that a string literal is an array that decays to a pointer.

⟨ *Sema: expr literal* 283 ⟩ ≡

```
// -- kcc_sema_expr_literal ---
// type_desc *kcc_sema_expr_literal(node *expr)
//   x19 = expression node, x20 = tag (set by caller)
.text
.p2align 2
_kcc_sema_expr_literal:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    cmp x20, #AST_INT_LIT
    b.eq .Lsel_int
    cmp x20, #AST_CHAR_LIT
    b.eq .Lsel_int
    cmp x20, #AST_FLOAT_LIT
    b.eq .Lsel_double
    cmp x20, #AST_STR_LIT
    b.eq .Lsel_str

    // Default: int
```

```
.Lsel_int:
    // An integer literal's suffix flags live in AST_F2 (bit 0 = unsigned,
    // bits 1-2 = long rank).  A char literal and an unsuffixed integer both
    // leave AST_F2 == 0, so they fall through to plain int.
    ldr x1, [x19, #AST_F2]
    cbz x1, .Lsel_plain_int
    lsr x2, x1, #1
    and x2, x2, #3                // long rank: 0 int, 1 long, 2 long long
    and x1, x1, #1                // unsigned bit
    cbz x2, .Lsel_rank_int
    cmp x2, #1
    b.eq .Lsel_rank_long
    // long long
    cbz x1, .Lsel_llong_s
    adrp x0, _kcc_type_llong_u@PAGE
    add  x0, x0, _kcc_type_llong_u@PAGEOFF
    b .Lsel_int_ret
.Lsel_llong_s:
    adrp x0, _kcc_type_llong_s@PAGE
    add  x0, x0, _kcc_type_llong_s@PAGEOFF
    b .Lsel_int_ret
.Lsel_rank_long:
    cbz x1, .Lsel_long_s
    adrp x0, _kcc_type_long_u@PAGE
    add  x0, x0, _kcc_type_long_u@PAGEOFF
    b .Lsel_int_ret
.Lsel_long_s:
    adrp x0, _kcc_type_long_s@PAGE
    add  x0, x0, _kcc_type_long_s@PAGEOFF
    b .Lsel_int_ret
.Lsel_rank_int:
    // rank int but unsigned suffix (u/U with no l/L)
    adrp x0, _kcc_type_int_u@PAGE
    add  x0, x0, _kcc_type_int_u@PAGEOFF
    b .Lsel_int_ret
.Lsel_plain_int:
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
.Lsel_int_ret:
    ldr  x0, [x0]
    ldp x29, x30, [sp], #16
    ret

.Lsel_double:
    // A floating constant is double, or float when it carried an f/F suffix
    // (recorded in AST_F2 by the parser; the value was rounded to single there).
    ldr x0, [x19, #AST_F2]
    cbz x0, .Lsel_double_d
    adrp x0, _kcc_type_float@PAGE
    add  x0, x0, _kcc_type_float@PAGEOFF
    ldr  x0, [x0]
    ldp x29, x30, [sp], #16
    ret
.Lsel_double_d:
    adrp x0, _kcc_type_double@PAGE
    add  x0, x0, _kcc_type_double@PAGEOFF
    ldr  x0, [x0]
    ldp x29, x30, [sp], #16
    ret

.Lsel_str:
    // String literal: type is "char *" (pointer to char)
    adrp x0, _kcc_type_char_s@PAGE
    add  x0, x0, _kcc_type_char_s@PAGEOFF
    ldr  x0, [x0]
    bl _kcc_type_ptr
    ldp x29, x30, [sp], #16
    ret
```

□

283 · used in 30a · uses _kcc_type_char_s 250b, _kcc_type_double 250b, _kcc_type_float 250b, _kcc_type_int_s 250b, _kcc_type_int_u 250b, _kcc_type_llong_s 250b, _kcc_type_llong_u 250b, _kcc_type_long_s 250b, _kcc_type_long_u 250b, _kcc_type_ptr 242a

26.7.4. Binary Expression

Binary expression type-checking first resolves the types of both operands. Comparison and logical operators always produce `int` (per §6.5.8–9 and §6.5.13–14). If the left operand is a pointer and the operator is additive, the result is a pointer type (pointer arithmetic). A shift (<<, >>) is special (§6.5.7): its result type is the *promoted left operand* alone, so the right operand never enters the conversions and `1 << 1u` stays a signed `int`. Otherwise, the usual arithmetic conversions (`_kcc_conv_usual`) determine the common type.

Before any of that, the checker rejects a binary operator whose left or right operand is a `__int128`. kcc recognises the 128-bit type and lays it out, and copies and by-value passing of one work through the aggregate path, but a `__int128` *rvalue* is still materialised in a single eight-byte slot—so an add, shift, or comparison would silently drop the high 64 bits. Following the same reject-rather-than-miscompile policy as the `_Complex` and VLA limitations (Appendix 3), the checker emits a diagnostic here; the unary checker does the same for `-`, `+`, `~`, and `++`/`--`, and the assignment checker for the compound forms (plain `=` is a copy and stays allowed). Full 128-bit arithmetic—register-pair lowering with carry—is a planned follow-up.

The six relational and equality operators additionally run the `-Wsign-compare` check (`kcc_sema_check_sign_compare`). When one operand is a signed integer and the other unsigned, the usual conversions turn the signed value unsigned, which flips the meaning of a negative value—the classic `u < -1` surprise. The check warns on exactly that mismatch, with one deliberate suppression that keeps it quiet on ordinary code: if either operand is a *known non-negative constant*, there is nothing to be surprised by, so `u < 10` and `i == 0` stay silent. Identifying that constant is easy here because the AST has no negative literals—a negative value is a unary-minus expression—so an integer or character literal operand is necessarily non-negative. An enumeration constant, already folded to such a literal by the time the operands are checked, is covered by the same test. Signedness itself comes from `kcc_sema_int_signedness`, which reads `TY_F0` for the ranked integer kinds and treats an enum as signed `int`; anything that is not one of those integers (a pointer, a float) is unranked and the check is skipped.

⟨ *Sema: expr binary* 284 ⟩ ≡

```
// -- kcc_sema_expr_binary ---
// type_desc *kcc_sema_expr_binary(node *expr)
//   x19 = expression node
.text
.p2align 2
_kcc_sema_expr_binary:
```

```
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Type-check LHS
    ldr x0, [x19, #AST_F1]
    bl _kcc_sema_expr
    mov x21, x0                         // LHS type

    // Type-check RHS
    ldr x0, [x19, #AST_F2]
    bl _kcc_sema_expr
    mov x22, x0                         // RHS type

    // Reject any binary operator on a __int128
   operand.  kcc recognises the
    // type and lays it out (16 bytes), and copies
    / by-value passing / union
    // punning work through the aggregate path,
   but a __int128 *rvalue* is still
    // lowered through a single 8-byte slot, so
   arithmetic, shifts, and
    // comparisons would silently drop the high 64
    bits (carry is lost).  Reject
    // rather than miscompile, in the spirit of
   the _Complex/VLA limitations
    // (128-bit arithmetic is a planned follow-up)
   .
    cbz x21, .Lseb_i128_ok
    ldr x0, [x21, #TY_KIND]
    cmp x0, #TY_INT128
    b.eq .Lseb_i128_err
    cbz x22, .Lseb_i128_ok
    ldr x0, [x22, #TY_KIND]
    cmp x0, #TY_INT128
    b.ne .Lseb_i128_ok
.Lseb_i128_err:
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    adrp x2, .Lseb_int128_msg@PAGE
    add  x2, x2, .Lseb_int128_msg@PAGEOFF
    bl _kcc_diag_error
.Lseb_i128_ok:

    // Get operator
    ldr x20, [x19, #AST_FO]

    // Comparison operators return int (and get
   the -Wsign-compare check)
    cmp x20, #TOK_EQ
    b.eq .Lseb_cmp
    cmp x20, #TOK_NE
    b.eq .Lseb_cmp
    cmp x20, #TOK_LT
    b.eq .Lseb_cmp
    cmp x20, #TOK_GT
    b.eq .Lseb_cmp
    cmp x20, #TOK_LE
    b.eq .Lseb_cmp
    cmp x20, #TOK_GE
    b.eq .Lseb_cmp

    // Logical operators return int
    cmp x20, #TOK_AND
    b.eq .Lseb_int
    cmp x20, #TOK_OR
    b.eq .Lseb_int

    // Shift operators: the result type is the
   promoted LEFT operand alone
    // (\stdref{6.5.7}); the right operand does
   not enter the usual arithmetic
    // conversions, so `1 << 1u` stays a signed
   int, not unsigned.
    cmp x20, #TOK_SHL
    b.eq .Lseb_shift
    cmp x20, #TOK_SHR
    b.eq .Lseb_shift

    // Pointer arithmetic: ptr + int or ptr - int
    ldr x0, [x21, #TY_KIND]
    cmp x0, #TY_PTR
    b.eq .Lseb_ptr_arith

    // Arithmetic operators: usual arithmetic
   conversions
    mov x0, x21
    mov x1, x22
    bl _kcc_conv_usual
    b .Lseb_done

.Lseb_shift:
    mov x0, x21                         // left operand
    type
    bl _kcc_conv_promote                // integer-
   promote (rank < int -> int)
    b .Lseb_done

.Lseb_ptr_arith:
    // `ptr + int` and `ptr - int` yield a pointer
   , but `ptr - ptr` is a pointer
    // \emph{difference} whose type is the signed
   integer \texttt{ptrdiff_t}
    // (\stdref{6.5.6}), here \texttt{long}.
   Getting this wrong made the result
    // of `p - q` carry a pointer type, so a
   surrounding `(p - q) + n` treated it
    // as pointer arithmetic and scaled `n` by the
    element size --- `(p-buf)+buf[2]`
    // added `buf[2] * sizeof(int)` instead of `
   buf[2]`.
    cmp x20, #TOK_MINUS
    b.ne .Lseb_ptr_result
    ldr x1, [x22, #TY_KIND]          // RHS kind
    cmp x1, #TY_PTR
    b.eq .Lseb_ptrdiff
    cmp x1, #TY_ARRAY
    b.eq .Lseb_ptrdiff
.Lseb_ptr_result:
    mov x0, x21                         // ptr ± int ->
   pointer type
    b .Lseb_done
.Lseb_ptrdiff:
    adrp x0, _kcc_type_long_s@PAGE
    add  x0, x0, _kcc_type_long_s@PAGEOFF
    ldr  x0, [x0]
    b .Lseb_done

.Lseb_cmp:
    // -Wall -Wsign-compare: a signed-vs-unsigned
   comparison, then int result.
    mov x0, x19                         // expr node (
   for location + operand nodes)
    mov x1, x21                         // LHS type
    mov x2, x22                         // RHS type
    bl _kcc_sema_check_sign_compare
    // fall through to the int result

.Lseb_int:
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]

.Lseb_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
```

```
    ret

// -- kcc_sema_int_signedness ---
// int kcc_sema_int_signedness(type_desc *t)
// 1 = signed integer, 0 = unsigned integer, -1 =
   not an integer we rank.
// Integer kinds CHAR..LLONG carry signedness in
   TY_FO; an enum is signed int.
.globl _kcc_sema_int_signedness
.p2align 2
_kcc_sema_int_signedness:
    cbz x0, .Lsg_na
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ENUM
    b.eq .Lsg_signed
    cmp x1, #TY_CHAR
    b.lt .Lsg_na                    // below CHAR (
   void/bool) → skip
    cmp x1, #TY_LLONG
    b.gt .Lsg_na                    // above LLONG
   (float/ptr/...) → skip
    ldr x0, [x0, #TY_FO]            // 1=signed, 0=
   unsigned
    ret
.Lsg_signed:
    mov x0, #1
    ret
.Lsg_na:
    mov x0, #-1
    ret

// -- kcc_sema_is_nonneg_const ---
// int kcc_sema_is_nonneg_const(node *e)
// 1 iff e is a (parenthesised) integer/char
   literal.  In this AST such a
// literal is always non-negative -- a negative
   value is a unary-minus
// expression -- so this exactly identifies a
   known non-negative constant.
.p2align 2
_kcc_sema_is_nonneg_const:
    cbz x0, .Lnn_no
.Lnn_unwrap:
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_PAREN_EXPR
    b.ne .Lnn_check
    ldr x0, [x0, #AST_FO]
    cbz x0, .Lnn_no
    b .Lnn_unwrap
.Lnn_check:
    cmp x1, #AST_INT_LIT
    b.eq .Lnn_yes
    cmp x1, #AST_CHAR_LIT
    b.eq .Lnn_yes
    mov x0, #0
    ret
.Lnn_yes:
    mov x0, #1
    ret
.Lnn_no:
    mov x0, #0
    ret

// -- kcc_sema_check_sign_compare ---
// void kcc_sema_check_sign_compare(node *expr,
   type_desc *lt, type_desc *rt)
// Under -Wall, warn when a comparison mixes a
   signed and an unsigned integer
// operand, unless one operand is a known non-
   negative constant (the common,
// safe case such as `u < 10`).
.globl _kcc_sema_check_sign_compare
.p2align 2
_kcc_sema_check_sign_compare:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // expr node
    mov x21, x1                     // LHS type
    mov x22, x2                     // RHS type
    adrp x0, _kcc_warn_all@PAGE
    add  x0, x0, _kcc_warn_all@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lcsc_done
    mov x0, x21
    bl _kcc_sema_int_signedness
    cmn x0, #1                      // == -1 ?
    b.eq .Lcsc_done                 // LHS not an
   integer we rank
    mov x20, x0                     // LHS
   signedness
    mov x0, x22
    bl _kcc_sema_int_signedness
    cmn x0, #1
    b.eq .Lcsc_done                 // RHS not an
   integer
    cmp x0, x20
    b.eq .Lcsc_done                 // same
   signedness → fine
    // Signedness differs: suppress if either
   operand is a non-negative const.
    ldr x0, [x19, #AST_F1]          // LHS node
    bl _kcc_sema_is_nonneg_const
    cbnz x0, .Lcsc_done
    ldr x0, [x19, #AST_F2]          // RHS node
    bl _kcc_sema_is_nonneg_const
    cbnz x0, .Lcsc_done
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    adrp x2, _kcc_sema_sign_cmp@PAGE
    add  x2, x2, _kcc_sema_sign_cmp@PAGEOFF
    bl _kcc_diag_warning
.Lcsc_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.section __TEXT,__cstring,cstring_literals
.Lseb_int128_msg:
    .asciz "128-bit integer arithmetic (__int128)
   is not yet supported"
.text
```

□

284 · used in 30a · uses _kcc_conv_promote 295a, _kcc_conv_usual 295b, _kcc_diag_error 76a, _kcc_diag_warning 76b, _kcc_sema_expr 274, _kcc_type_int_s 250b, _kcc_type_long_s 250b, _kcc_warn_all 77a

26.7.5. Unary Expression

Unary expression types depend on the operator. Address-of (&) wraps the operand type in a pointer. Dereference (*) unwraps one pointer level; if the operand is not a pointer, the checker falls back to `int`. Logical not (!) always produces `int`. All remaining operators (bitwise not, increment, decrement, unary minus and plus) preserve the operand type.

⟨ *Sema: expr unary* 286 ⟩ ≡

```
// -- kcc_sema_expr_unary ---
// type_desc *kcc_sema_expr_unary(node *expr)
```

```
//   x19 = expression node
.text
.p2align 2
_kcc_sema_expr_unary:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // Type-check operand
    ldr x0, [x19, #AST_F1]
    bl _kcc_sema_expr
    mov x20, x0                         // operand
   type

    // Reject arithmetic unary operators on a
   __int128 operand (negate, plus,
    // bitwise-not, ++/--): like the binary case
   they would drop the high 64
    // bits.  Address-of (&) is structural and
   stays allowed.
    cbz x20, .Lseu_i128_ok
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_INT128
    b.ne .Lseu_i128_ok
    ldr x1, [x19, #AST_F0]
    cmp x1, #TOK_AMP
    b.eq .Lseu_i128_ok
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    adrp x2, .Lseb_int128_msg@PAGE
    add  x2, x2, .Lseb_int128_msg@PAGEOFF
    bl _kcc_diag_error
.Lseu_i128_ok:

    // Get operator
    ldr x0, [x19, #AST_F0]

    cmp x0, #TOK_AMP                    // address-of
    b.eq .Lseu_addr
    cmp x0, #TOK_STAR                   // dereference
    b.eq .Lseu_deref
    cmp x0, #TOK_BANG                   // logical not
   → int
    b.eq .Lseu_int
    cmp x0, #TOK_TILDE                  // bitwise not
   → promoted operand type
    b.eq .Lseu_promote
    cmp x0, #TOK_INC                    // increment →
   operand type (an lvalue)
    b.eq .Lseu_operand
    cmp x0, #TOK_DEC                    // decrement →
   operand type (an lvalue)
    b.eq .Lseu_operand
    cmp x0, #TOK_MINUS                  // unary minus
   → promoted operand type
    b.eq .Lseu_promote
    cmp x0, #TOK_PLUS                   // unary plus →
   promoted operand type
    b.eq .Lseu_promote

    // Default: operand type
    b .Lseu_operand

.Lseu_promote:
    // The arithmetic unary operators -, +, ~
   apply the integer promotions to
    // their result (\stdref{6.5.3.3}), so \texttt
   {-(unsigned short)1} is a
    // signed \texttt{int}, not an unsigned short.
    mov x0, x20
    bl _kcc_conv_promote
    b .Lseu_done

.Lseu_addr:
    // &expr → pointer to operand type
    mov x0, x20
    bl _kcc_type_ptr
    b .Lseu_done

.Lseu_deref:
    // *expr → dereference pointer
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_PTR
    b.ne .Lseu_int                 // error: not a
   pointer
    ldr x0, [x20, #TY_F0]       // pointee type
    b .Lseu_done

.Lseu_int:
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
    b .Lseu_done

.Lseu_operand:
    mov x0, x20

.Lseu_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

286 · used in 30a · uses _kcc_conv_promote 295a, _kcc_diag_error 76a, _kcc_sema_expr 274, _kcc_type_int_s 250b, _kcc_type_ptr 242a

26.7.6. Assignment Expression

Per §6.5.16, the type of an assignment expression is the type of the left operand. Both sides are type-checked, but the result type is always the LHS type. Compound assignments (+=, -=, etc.) are treated identically for type purposes.

A simple assignment additionally runs the `-Wint-conversion` check (`kcc_sema_check_int_ptr`), the same check reused for initialisation and `return`: mixing an integer and a pointer across the assignment—`int *p = 5`, or `int n = p`—without a cast is almost always a mistake, and the resulting value is meaningless on this target where pointers are wider than `int`. `kcc_sema_ptr_int_class` sorts each type into pointer, integer, or neither, and a warning is issued only for the two genuine mismatches (integer into pointer, pointer into integer). Two suppressions keep it honest: the check is limited to the simple `=` operator, since `p += n` is legitimate pointer arithmetic rather than a conversion; and the integer constant zero is recognised as the null pointer constant (`kcc_sema_is_null_const`), so `int *p = 0` stays silent while `int *p = 5` does not. Function arguments are not covered, because the function type descriptor does not yet carry its parameter types.

⟨ *Sema: expr assign* 287 ⟩ ≡

```
// -- kcc_sema_expr_assign ---
// type_desc *kcc_sema_expr_assign(node *expr)
//   x19 = expression node
// Type of assignment = type of LHS.
.text
.p2align 2
_kcc_sema_expr_assign:
    stp x29, x30, [sp, #-32]!
```

```
    mov x29, sp
    stp x19, x20, [sp, #16]

    // Type-check LHS
    ldr x0, [x19, #AST_F1]
    bl _kcc_sema_expr
    mov x20, x0                     // LHS type (=
    result type)

    // Type-check RHS
    ldr x0, [x19, #AST_F2]
    bl _kcc_sema_expr               // x0 = RHS
   type

    // Reject compound assignment (+=, -=, <<=,
   ...) on a __int128 lvalue: it is
    // arithmetic and would drop the high 64 bits.
     Plain '=' is a 16-byte copy
    // and stays allowed.
    ldr x1, [x19, #AST_FO]
    cmp x1, #TOK_ASSIGN
    b.eq .Lsea_no_i128
    cbz x20, .Lsea_no_i128
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_INT128
    b.ne .Lsea_no_i128
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    adrp x2, .Lseb_int128_msg@PAGE
    add  x2, x2, .Lseb_int128_msg@PAGEOFF
    bl _kcc_diag_error
.Lsea_no_i128:

    // -Wall -Wint-conversion: only for a simple
   '=' (compound assignment on a
    // pointer is legal arithmetic, not a
   conversion).
    ldr x1, [x19, #AST_FO]          // operator
    cmp x1, #TOK_ASSIGN
    b.ne .Lsea_result
    mov x2, x0                      // src type (
   RHS)
    mov x1, x20                     // dst type (
   LHS)
    mov x0, x19                     // location
   node
    ldr x3, [x19, #AST_F2]          // src node
    bl _kcc_sema_check_int_ptr

.Lsea_result:
    // Result type is LHS type
    mov x0, x20

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_sema_ptr_int_class ---
// int kcc_sema_ptr_int_class(type_desc *t)
// 1 = pointer, 2 = integer (bool..long long, enum
   ), 0 = anything else.
.p2align 2
_kcc_sema_ptr_int_class:
    cbz x0, .Lpic_other
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_PTR
    b.eq .Lpic_ptr
    cmp x1, #TY_ENUM
    b.eq .Lpic_int
    cmp x1, #TY_BOOL                // 2
    b.lt .Lpic_other
    cmp x1, #TY_LLONG               // 7
    b.gt .Lpic_other
    mov x0, #2
    ret
.Lpic_ptr:
    mov x0, #1
    ret
.Lpic_int:
    mov x0, #2
    ret
.Lpic_other:
    mov x0, #0
    ret

// -- kcc_sema_is_null_const ---
// int kcc_sema_is_null_const(node *e)
// 1 iff e is a (parenthesised) integer literal
   equal to zero -- the null
// pointer constant, which converts to any pointer
    without complaint.
.p2align 2
_kcc_sema_is_null_const:
    cbz x0, .Lnc_no
.Lnc_unwrap:
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_PAREN_EXPR
    b.ne .Lnc_check
    ldr x0, [x0, #AST_FO]
    cbz x0, .Lnc_no
    b .Lnc_unwrap
.Lnc_check:
    cmp x1, #AST_INT_LIT
    b.ne .Lnc_no
    ldr x0, [x0, #AST_FO]           // value
    cmp x0, #0
    cset x0, eq
    ret
.Lnc_no:
    mov x0, #0
    ret

// -- kcc_sema_check_int_ptr ---
// void kcc_sema_check_int_ptr(node *loc, type *
   dst, type *src, node *srcexpr)
// Under -Wall, warn when an integer is converted
   to a pointer or vice versa
// without a cast (assignment, initialisation,
   return).  A null pointer
// constant (integer 0) assigned to a pointer is
   exempt.
.globl _kcc_sema_check_int_ptr
.p2align 2
_kcc_sema_check_int_ptr:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // loc node
    mov x20, x1                     // dst type
    mov x21, x2                     // src type
    mov x22, x3                     // src node
    adrp x0, _kcc_warn_all@PAGE
    add  x0, x0, _kcc_warn_all@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lcip_done
    mov x0, x20
    bl _kcc_sema_ptr_int_class
    mov x20, x0                     // dst class (
   leaf call: x19/x21/x22 intact)
    mov x0, x21
    bl _kcc_sema_ptr_int_class
    mov x21, x0                     // src class
    cmp x20, #1                     // dst pointer?
    b.ne .Lcip_int_dst
    cmp x21, #2                     // src integer?
    b.ne .Lcip_done
```

```
    mov x0, x22
    bl _kcc_sema_is_null_const
    cbnz x0, .Lcip_done            // int 0 → null
    pointer constant, fine
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    adrp x2, _kcc_sema_int2ptr@PAGE
    add  x2, x2, _kcc_sema_int2ptr@PAGEOFF
    bl _kcc_diag_warning
    b .Lcip_done
.Lcip_int_dst:
    cmp x20, #2                    // dst integer?
    b.ne .Lcip_done
    cmp x21, #1                    // src pointer?
    b.ne .Lcip_done
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    adrp x2, _kcc_sema_ptr2int@PAGE
    add  x2, x2, _kcc_sema_ptr2int@PAGEOFF
    bl _kcc_diag_warning
.Lcip_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

287 · used in 30a · uses _kcc_diag_error 76a, _kcc_diag_warning 76b, _kcc_sema_expr 274, _kcc_warn_all 77a

26.7.7. Function Call

A function call first type-checks the callee expression, then walks the argument list and type-checks each argument. The result type is the callee's return type, extracted from the TY_FUNC descriptor's TY_F0 field. If the callee is a pointer-to-function, the pointer is dereferenced first. Implicit function declarations (callee not found) fall back to int return.

The "implicit" status is signalled by a global flag, _kcc_sema_callee_implicit, that kcc_sema_expr_ident raises when it takes an undeclared bare name in call position (§6.5.2.2) rather than diagnosing it. That flag is process-global, but a call's arguments may themselves be calls, and each nested call clears the flag on entry to track its own callee. The check for "called object is not a function" must therefore consult a *snapshot* of the flag taken immediately after the callee is resolved, before the argument loop runs — otherwise an argument such as f(x) inside __builtin_expect(!!(f(x) > 0), 0) would clobber the outer callee's implicit marker and the compiler would wrongly reject the enclosing call. The assembly parks the snapshot in the callee-saved x22; the C port keeps it in the local callee_implicit.

⟨ *Sema: expr call* 289 ⟩ ≡

```
// -- kcc_sema_expr_call ---
// type_desc *kcc_sema_expr_call(node *expr)
//   x19 = expression node
.text
.p2align 2
_kcc_sema_expr_call:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    str x22, [sp, #40]

    // Type-check callee.  When it is a bare
    identifier, enter "callee
    // context" so an undeclared name is taken as
    an implicit function
    // declaration rather than an undeclared-
    identifier error.
    adrp x3, _kcc_sema_callee_implicit@PAGE
    add  x3, x3, _kcc_sema_callee_implicit@PAGEOFF
    str xzr, [x3]                  // clear
    implicit-function marker
    ldr x0, [x19, #AST_F0]         // callee node
    ldr x1, [x0]                   // AST_TAG
    cmp x1, #AST_IDENT_EXPR
    cset x2, eq
    adrp x3, _kcc_sema_callee_ctx@PAGE
    add  x3, x3, _kcc_sema_callee_ctx@PAGEOFF
    str x2, [x3]
    bl _kcc_sema_expr
    mov x20, x0                    // callee type
    adrp x3, _kcc_sema_callee_ctx@PAGE
    add  x3, x3, _kcc_sema_callee_ctx@PAGEOFF
    str xzr, [x3]

    // Snapshot the implicit-function marker for
    THIS call before checking the
    // arguments.  _kcc_sema_callee_implicit is a
    global; an argument that is
    // itself a call clears it on entry, so
    reading it after the loop would lose
    // the outer callee's status and misfire "
    called object is not a function"
    // for e.g. __builtin_expect(!!(f(x) > 0), 0).
    adrp x22, _kcc_sema_callee_implicit@PAGE
    add  x22, x22,
    _kcc_sema_callee_implicit@PAGEOFF
    ldr  x22, [x22]

    // Type-check arguments
    ldr x21, [x19, #AST_F1]        // first arg
.Lsec_arg_loop:
    cbz x21, .Lsec_arg_done
    mov x0, x21
    bl _kcc_sema_expr
    ldr x21, [x21, #AST_F4]        // next arg
    b .Lsec_arg_loop

.Lsec_arg_done:
    // Get return type from callee type
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_FUNC
    b.eq .Lsec_func_ret
    cmp x1, #TY_PTR
    b.eq .Lsec_ptr_func

    // Not a function type.  Unless the callee was
    an implicit function
    // declaration (an undeclared bare name in
    call position), this is an
    // error: something that is not a function is
    being called.  Use the
    // snapshot taken before the argument loop,
    not the (possibly clobbered)
    // global.
    cbnz x22, .Lsec_default_int
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    adrp x2, _kcc_sema_err_not_func@PAGE
    add  x2, x2, _kcc_sema_err_not_func@PAGEOFF
    bl _kcc_diag_error
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
```

```
    b .Lsec_done

.Lsec_func_ret:
    ldr x0, [x20, #TY_F0]          // return type
    b .Lsec_done

.Lsec_ptr_func:
    // Pointer to function: dereference then get
   return type
    ldr x0, [x20, #TY_F0]          // pointee (
   should be TY_FUNC)
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_FUNC
    b.ne .Lsec_default_int
    ldr x0, [x0, #TY_F0]           // return type
    b .Lsec_done

.Lsec_default_int:
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]

.Lsec_done:
    ldr x22, [sp, #40]
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

289 · used in 30a · uses _kcc_diag_error 76a, _kcc_sema_callee_ctx 263, _kcc_sema_callee_implicit 263, _kcc_sema_expr 274, _kcc_type_int_s 250b

26.7.8. Member Access

Member access handles both dot (`s.m`) and arrow (`p->m`) forms. For arrow access the object type must be a pointer, which is dereferenced first. An *array* operand is equally valid: array-to-pointer decay (C §6.3.2.1) makes `(a)->m` mean `(*a).m`, so we accept `TY_ARRAY` alongside `TY_PTR` and read the element type from the same `TY_F0` slot. This is exactly how the kernel's `cpumask_bits(mask)` macro—`(mask)->bits` where `mask` is a `cpumask_var_t` array—reaches the compiler. The resulting type must be `TY_STRUCT` or `TY_UNION`; if not, the checker falls back to `int`. The lookup is factored into a shared helper, `kcc_type_find_member`, which walks the aggregate's field chain (`AST_F1` linked via `AST_F4`) for a name matching the member in `AST_MEMBER.F1`. When it meets a *nameless* field whose type is itself a struct or union — an anonymous member in the sense of C11 §6.7.2.1 — it recurses into that member and, on a hit, adds the anonymous member's own byte offset to the offset returned from within. A member of an anonymous inner aggregate is therefore reachable directly, as though its fields belonged to the enclosing type. The helper returns the matching field decl together with its accumulated byte offset; on a match the field's declared type is stored in `AST_F3` by the caller (the standard `.Lse_store` path). The IR builder calls the very same helper when lowering a member address, so type-checking and code generation agree on both the resolved field and its offset.

⟨ *Sema: expr member* 290 ⟩ ≡

```
// -- kcc_type_find_member ---
// (AST_FIELD_DECL, uint64_t offset)
   kcc_type_find_member(type *sty, char *name)
//   x0 = struct/union type, x1 = member name
// Returns x0 = matching field decl (0 if none),
   x1 = its byte offset within
// `sty`.  Recurses into anonymous (nameless)
   struct/union members and adds
// their offsets (C11 6.7.2.1), so a member of an
   anonymous inner aggregate is
// reachable directly.  Shared by member type-
   checking and the IR builder's
// member-address lowering, keeping the two in
   lock-step.
.text
.globl _kcc_type_find_member
.p2align 2
_kcc_type_find_member:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                    // struct/union
    type
    mov x20, x1                    // member name
    mov x0, x19
    bl _kcc_type_layout            // ensure per-
   field AST_F3 offsets are set
    ldr x0, [x19, #TY_F0]          //
   AST_STRUCT_DECL / AST_UNION_DECL
    cbz x0, .Lfm_notfound
    ldr x21, [x0, #AST_F1]         // first field
.Lfm_loop:
    cbz x21, .Lfm_notfound
    ldr x0, [x21, #AST_F1]         // field name
    cbz x0, .Lfm_anon              // nameless
   field: an anonymous member
    mov x1, x20
    bl _kcc_strcmp
    cbz x0, .Lfm_direct
    b .Lfm_next
.Lfm_direct:
    mov x0, x21
    ldr x1, [x21, #AST_F3]         // field byte
   offset
    b .Lfm_done
.Lfm_anon:
    ldr x0, [x21, #AST_F0]         // its type
   must be a struct/union to promote
    cbz x0, .Lfm_next
    bl _kcc_sema_resolve_type
    cbz x0, .Lfm_next
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_STRUCT
    b.eq .Lfm_recurse
    cmp x1, #TY_UNION
    b.ne .Lfm_next
.Lfm_recurse:
    mov x1, x20
    bl _kcc_type_find_member       // x0 = inner
   field, x1 = inner offset; x21 kept
    cbz x0, .Lfm_next
    ldr x2, [x21, #AST_F3]         // this
   anonymous member's own offset
    add x1, x1, x2                 // accumulate
    b .Lfm_done
.Lfm_next:
    ldr x21, [x21, #AST_F4]        // next field
    b .Lfm_loop
.Lfm_notfound:
    mov x0, #0
    mov x1, #0
.Lfm_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
```

```
    ldp x29, x30, [sp], #48
    ret

// -- kcc_sema_expr_member ---
// type_desc *kcc_sema_expr_member(node *expr)
//   x19 = expression node (AST_MEMBER)
// F0=object, F1=name_ptr, F2=arrow(1)/dot(0)
// Returns: resolved field type (or int as
   fallback)
.text
.p2align 2
_kcc_sema_expr_member:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Type-check the object expression
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_expr
    mov x20, x0                         // object type

    // If arrow: dereference pointer first
    ldr x1, [x19, #AST_F2]
    cbz x1, .Lsem_dot
    // Arrow: type should be pointer to struct/
   union.  An array operand is
    // valid too: array-to-pointer decay (C
   6.3.2.1) yields pointer-to-element,
    // and both TY_PTR and TY_ARRAY carry that
   element type in TY_F0.
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_PTR
    b.eq .Lsem_arrow_deref
    cmp x1, #TY_ARRAY
    b.ne .Lsem_fallback
.Lsem_arrow_deref:
    ldr x20, [x20, #TY_F0]          // dereference
   / decay

.Lsem_dot:
    // Type should be struct or union
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_STRUCT
    b.eq .Lsem_lookup
    cmp x1, #TY_UNION
    b.eq .Lsem_lookup

.Lsem_fallback:
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    adrp x2, _kcc_sema_err_not_struct@PAGE
    add  x2, x2, _kcc_sema_err_not_struct@PAGEOFF
    bl _kcc_diag_error
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lsem_lookup:
    // Find the member, descending through
   anonymous members (C11 6.7.2.1).
    mov x0, x20                         // struct/union
    type
    ldr x1, [x19, #AST_F1]          // member name
    bl _kcc_type_find_member
    mov x21, x0                         // matching
   AST_FIELD_DECL (0 if none)
    cbnz x21, .Lsem_found

.Lsem_not_found:
    ldr x0, [x19, #AST_LINE]
    ldr x1, [x19, #AST_COL]
    ldr x2, [x19, #AST_F1]          // member name
    adrp x3, _kcc_sema_nomember_pre@PAGE
    add  x3, x3, _kcc_sema_nomember_pre@PAGEOFF
    bl _kcc_sema_err_named
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lsem_found:
    // x21 = matching AST_FIELD_DECL
    ldr x0, [x21, #AST_F0]          // field type
   node
    cbz x0, .Lsem_found_int
    bl _kcc_sema_resolve_type
    mov x22, x0                         // declared
   field type
    // Bit-field integer promotion (\stdref
   {6.3.1.1}): a bit-field of width W is
    // promoted where int can represent all its
   values, which for a bit-field
    // narrower than int is \emph{always} --- even
    an unsigned one --- so
    // `unsigned f:3` yields a signed int and `(f
   - 2)` is a signed compare.  A
    // full-width unsigned bit-field (W == int
   width) stays unsigned int; wider
    // (long) bit-fields keep their declared type.
     W is the low 16 bits of the
    // field's packed AST_F2 (0 for a non-bit-
   field member).
    ldr x1, [x21, #AST_F2]
    and x1, x1, #0xFFFF             // bit-field
   width
    cbz x1, .Lsem_found_ret
    cbz x22, .Lsem_found_ret
    ldr x2, [x22, #TY_KIND]
    sub x3, x2, #2
    cmp x3, #5                          // integer kind
    (TY_BOOL(2)..TY_LLONG(7))?
    b.hi .Lsem_found_ret
    ldr x2, [x22, #TY_F0]           // signedness
   (1 = signed, 0 = unsigned)
    cbz x2, .Lsem_bf_unsigned
    cmp x1, #32                         // signed: W
   <=32 → int, else declared
    b.hi .Lsem_found_ret
    b .Lsem_bf_int
.Lsem_bf_unsigned:
    cmp x1, #31
    b.ls .Lsem_bf_int               // unsigned W
   <=31 → int (int holds all values)
    cmp x1, #32
    b.hi .Lsem_found_ret            // unsigned W>32
    → keep declared (unsigned long)
    adrp x0, _kcc_type_int_u@PAGE // unsigned W
   ==32 → unsigned int
    add  x0, x0, _kcc_type_int_u@PAGEOFF
    ldr  x0, [x0]
    b .Lsem_found_pret
.Lsem_bf_int:
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
    b .Lsem_found_pret
.Lsem_found_ret:
    mov x0, x22
```

```
.Lsem_found_pret:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Lsem_found_int:
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

290 · used in 30a · defines _kcc_sema_expr_member 274, _kcc_type_find_member 355, 426 · uses _kcc_diag_error 76a, _kcc_sema_expr 274, _kcc_sema_resolve_type 259b, _kcc_type_int_s 250b, _kcc_type_int_u 250b, _kcc_type_layout 267, 268

26.7.9. Cast Expression

A cast expression resolves its target type from the AST type node, type-checks the inner expression, and returns the target type as the result. No conversion code is emitted at this stage; the IR builder handles actual truncation and extension.

⟨ *Sema: expr cast* 292a ⟩ ≡

```
// -- kcc_sema_expr_cast ---
// type_desc *kcc_sema_expr_cast(node *expr)
//   x19 = expression node
// F0=type, F1=expr
.text
.p2align 2
_kcc_sema_expr_cast:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // Resolve cast target type
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    mov x20, x0                         // target type

    // Type-check the expression being cast
    ldr x0, [x19, #AST_F1]
    bl _kcc_sema_expr

    // Result is the target type
    mov x0, x20

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

292a · used in 30a · uses _kcc_sema_expr 274, _kcc_sema_resolve_type 259b

26.7.10. Sizeof Expression

The `sizeof` operator can take either a type or an expression as its operand (distinguished by the `AST_F1` flag). If the operand is an expression, it is type-checked (though its value is not evaluated at run time). In both cases the result type is `unsigned long`, which is `size_t` on ARM64.

One subtlety deserves care: §6.3.2.1 says an array is *not* converted to a pointer when it is the operand of `sizeof`, so `sizeof(arr)` must be the whole array and not a pointer. `_kcc_sema_expr` has already decayed the operand to a pointer in its `AST_F3`, so we undo that for two array-valued operand shapes by pinning the un-decayed array type back onto the operand node (which is what the IR builder and constant folder read). A named array is looked up in the symbol table and its `TY_ARRAY` type restored. A *string literal* is an array `char[N+1]` (§6.4.5) even though we type it as `char *` everywhere else for convenient decay; here we rebuild `char[len+1]` from its recorded length so that `sizeof("...")` is the array size, not 8. Without this, `sizeof(literal)/-1` length idioms—`luaS_newliteral` among them—silently truncated every string to the pointer size, corrupting memory downstream.

⟨ *Sema: expr sizeof* 292b ⟩ ≡

```
// -- kcc_sema_expr_sizeof ---
// type_desc *kcc_sema_expr_sizeof(node *expr)
//   x19 = expression node
// F0=operand (type node or expr), F1=is_type
// Result type is always unsigned long (size_t on
   ARM64).
.text
.p2align 2
_kcc_sema_expr_sizeof:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    // Check if operand is a type or expression
    ldr x1, [x19, #AST_F1]          // is_type flag
    cbnz x1, .Lses_type

    // Expression operand: type-check it
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_expr

    // sizeof does not decay arrays (§6.3.2.1): if
    the operand names an
    // array, restore its array type so the size
   is the whole array, not a
    // pointer.  sema_expr already decayed it to a
    pointer in AST_F3.  The
    // operand is the node whose AST_F3 the IR
   builder reads; the name may
    // sit under one or more parentheses, so
   descend to the inner expression.
    ldr x20, [x19, #AST_F0]         // operand node
    (IR builder reads its AST_F3)
    mov x1, x20
.Lses_unwrap:
    ldr x0, [x1, #AST_TAG]
    cmp x0, #AST_PAREN_EXPR
    b.ne .Lses_inner
    ldr x1, [x1, #AST_F0]
    cbnz x1, .Lses_unwrap
    b .Lses_result
.Lses_inner:
    cmp x0, #AST_STR_LIT
    b.eq .Lses_strlit
    cmp x0, #AST_IDENT_EXPR
    b.ne .Lses_result
    ldr x21, [x1, #AST_F0]          // name ptr
    cbz x21, .Lses_result
    mov x0, x21
    bl _kcc_strlen
```

```
    mov x1, x0
    mov x0, x21
    bl _kcc_symtab_lookup
    cbz x0, .Lses_result
    ldr x0, [x0, #SYM_TYPE]
    cbz x0, .Lses_result
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.ne .Lses_result
    str x0, [x20, #AST_F3]          // set the un-
   decayed array type on the operand
    b .Lses_result

.Lses_strlit:
    // A string literal is an array char[N+1] (§6
   .4.5); sema_expr already
    // decayed it to char* in AST_F3, but sizeof
   must see the whole array.
    // Rebuild char[len+1] from the recorded
   length and pin it on the operand.
    ldr x21, [x1, #AST_F1]          // string
   length N
    add x21, x21, #1                // + terminating
    NUL
    adrp x0, _kcc_type_char_s@PAGE
    add  x0, x0, _kcc_type_char_s@PAGEOFF
    ldr  x0, [x0]                   // element type
    char
    mov  x1, x21
    bl _kcc_type_array              // → char[N+1]
    str x0, [x20, #AST_F3]          // un-decayed
   array type on the operand
    b .Lses_result

.Lses_type:
    // Type operand: resolve it (for validation)
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type

.Lses_result:
    // sizeof always returns unsigned long (size_t
   )
    adrp x0, _kcc_type_long_u@PAGE
    add  x0, x0, _kcc_type_long_u@PAGEOFF
    ldr  x0, [x0]
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

292b · used in 30a · uses _kcc_sema_expr 274, _kcc_sema_resolve_type 259b, _kcc_symtab_lookup 255, _kcc_type_array 243a, _kcc_type_char_s 250b, _kcc_type_long_u 250b

27. Type Conversions

C specifies a set of implicit type conversions that the compiler must apply automatically whenever operands do not match the type expected by an operator or context (§ 6.3). This chapter implements the four key conversion rules: integer promotions widen types narrower than `int`; the usual arithmetic conversions harmonize two operands to a common type; array-to-pointer and function-to-pointer decay convert lvalues to their implicit pointer forms; and lvalue conversion strips qualifiers to produce an rvalue.

27.1. Integer Promotions

Per § 6.3.1.1, types with rank less than `int` promote to `int` (or `unsigned int` if the original type's values cannot be represented in `int`).

⟨ *Conv: integer promotions* 295a ⟩ ≡

```
// -- kcc_conv_promote ---
// type_desc *kcc_conv_promote(type_desc *type)
//   x0 = original type
// Returns promoted type (int or unsigned int if
   rank < int, else unchanged).
.text
.globl _kcc_conv_promote
.p2align 2
_kcc_conv_promote:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                        // save
   original

    // Check if it's an integer type
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_BOOL
    b.eq .Lcp_promote
    cmp x1, #TY_CHAR
    b.eq .Lcp_promote
    cmp x1, #TY_SHORT
    b.eq .Lcp_promote
    cmp x1, #TY_ENUM
    b.eq .Lcp_promote

    // Already int or wider, or not integer →
   return unchanged
    mov x0, x19
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

.Lcp_promote:
    // Promote to signed int (all smaller types
   fit in int on ARM64)
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr  x0, [x0]
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

295a · used in 29f · defines `_kcc_conv_promote` 284, 286, 295b, 359 · uses `_kcc_type_int_s` 250b

27.2. Usual Arithmetic Conversions

Per § 6.3.1.8: given two arithmetic operands, find the common type to which both are converted.

⟨ *Conv: usual arithmetic* 295b ⟩ ≡

```
// -- kcc_conv_usual ---
// type_desc *kcc_conv_usual(type_desc *t1,
   type_desc *t2)
//   x0 = left type, x1 = right type
// Returns the common type for binary arithmetic.
.text
.globl _kcc_conv_usual
.p2align 2
_kcc_conv_usual:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0                        // t1
    mov x20, x1                        // t2

    // If either is long double → long double
    ldr x1, [x19, #TY_KIND]
    cmp x1, #TY_LDOUBLE
    b.eq .Lcu_ldouble
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_LDOUBLE
    b.eq .Lcu_ldouble

    // If either is double → double
    ldr x1, [x19, #TY_KIND]
    cmp x1, #TY_DOUBLE
    b.eq .Lcu_double
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_DOUBLE
    b.eq .Lcu_double

    // If either is float → float
    ldr x1, [x19, #TY_KIND]
    cmp x1, #TY_FLOAT
    b.eq .Lcu_float
    ldr x1, [x20, #TY_KIND]
    cmp x1, #TY_FLOAT
    b.eq .Lcu_float

    // Both are integer types - apply integer
   promotions first
    mov x0, x19
    bl _kcc_conv_promote
    mov x19, x0
    mov x0, x20
    bl _kcc_conv_promote
    mov x20, x0

    // Same type after promotion → done
    cmp x19, x20
    b.eq .Lcu_return_t1

    // Both same signedness → higher rank wins
    ldr x21, [x19, #TY_F0]          // signedness
   of t1
    ldr x22, [x20, #TY_F0]          // signedness
   of t2
    cmp x21, x22
```

```
    b.ne .Lcu_diff_sign

    // Same signedness: compare ranks
    mov x0, x19
    bl _kcc_type_rank
    mov x21, x0
    mov x0, x20
    bl _kcc_type_rank
    cmp x21, x0
    b.ge .Lcu_return_t1
    b .Lcu_return_t2

.Lcu_diff_sign:
    // Different signedness: unsigned type with >=
    rank wins
    // Determine which is unsigned
    ldr x21, [x19, #TY_FO]
    cbz x21, .Lcu_t1_unsigned

    // t2 is unsigned
    mov x0, x20
    bl _kcc_type_rank
    mov x21, x0
    mov x0, x19
    bl _kcc_type_rank
    cmp x21, x0
    b.ge .Lcu_return_t2
    // Signed type has higher rank → use signed
   type
    b .Lcu_return_t1

.Lcu_t1_unsigned:
    // t1 is unsigned
    mov x0, x19
    bl _kcc_type_rank
    mov x21, x0
    mov x0, x20
    bl _kcc_type_rank
    cmp x21, x0
    b.ge .Lcu_return_t1
    b .Lcu_return_t2

.Lcu_ldouble:
    adrp x0, _kcc_type_ldouble@PAGE
    add  x0, x0, _kcc_type_ldouble@PAGEOFF
    ldr  x0, [x0]
    b .Lcu_done
.Lcu_double:
    adrp x0, _kcc_type_double@PAGE
    add  x0, x0, _kcc_type_double@PAGEOFF
    ldr  x0, [x0]
    b .Lcu_done
.Lcu_float:
    adrp x0, _kcc_type_float@PAGE
    add  x0, x0, _kcc_type_float@PAGEOFF
    ldr  x0, [x0]
    b .Lcu_done
.Lcu_return_t1:
    mov x0, x19
    b .Lcu_done
.Lcu_return_t2:
    mov x0, x20

.Lcu_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

295b · used in 29f · defines _kcc_conv_usual 284, 359 · uses _kcc_conv_promote 295a, _kcc_type_double 250b, _kcc_type_float 250b, _kcc_type_ldouble 250b, _kcc_type_rank 248c

27.3. Array and Function Decay

Per §6.3.2.1, an expression of array type decays to a pointer to its first element in most contexts (except `sizeof`, `_Alignof`, and the `&` operator). Similarly, a function designator decays to a pointer-to-function (§6.3.2.1p4). Both functions check the `TY_KIND` field and, if the type matches, call `kcc_type_ptr` to wrap it; otherwise they return the type unchanged.

⟨ *Conv: array decay* 296a ⟩ ≡

```
// -- kcc_conv_array_decay ---
// type_desc *kcc_conv_array_decay(type_desc *type
   )
//   x0 = type (may be TY_ARRAY or not)
// If array: returns pointer-to-element type.
// Otherwise: returns type unchanged.
.text
.globl _kcc_conv_array_decay
.p2align 2
_kcc_conv_array_decay:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.ne .Lcad_unchanged

    // Array → pointer to element type
    ldr x0, [x0, #TY_FO]           // element type
    bl _kcc_type_ptr
    ldp x29, x30, [sp], #16
    ret

.Lcad_unchanged:
    ldp x29, x30, [sp], #16
    ret
```

□

296a · used in 29f · defines _kcc_conv_array_decay 277 · uses _kcc_type_ptr 242a

Function designators undergo an analogous decay: a value of function type is implicitly converted to a pointer-to-function (§6.3.2.1p4). This allows a function name to be used directly in a call expression without the explicit `&` operator.

⟨ *Conv: func decay* 296b ⟩ ≡

```
// -- kcc_conv_func_decay ---
// type_desc *kcc_conv_func_decay(type_desc *type)
//   x0 = type (may be TY_FUNC or not)
// If function: returns pointer-to-function type.
// Otherwise: returns type unchanged.
.text
.globl _kcc_conv_func_decay
.p2align 2
_kcc_conv_func_decay:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_FUNC
    b.ne .Lcfd_unchanged

    // Function → pointer to function
    bl _kcc_type_ptr
    ldp x29, x30, [sp], #16
```

```
    ret

.Lcfd_unchanged:
    ldp x29, x30, [sp], #16
    ret
```

□

296b · used in 29f · defines _kcc_conv_func_decay 277 · uses _kcc_type_ptr 242a

27.4. Lvalue Conversion

When an lvalue appears in a context that requires an rvalue, its qualifiers are stripped (§6.3.2.1p2). The current implementation is a no-op stub that returns the descriptor unchanged, since qualifier enforcement is handled separately during semantic analysis.

⟨ *Conv: lvalue* 297a ⟩ ≡

```
// -- kcc_conv_lvalue ---
// type_desc *kcc_conv_lvalue(type_desc *type)
//   x0 = type descriptor
// Strips qualifiers for rvalue conversion.
// Returns unqualified type.
.text
.globl _kcc_conv_lvalue
.p2align 2
_kcc_conv_lvalue:
    // If type has qualifiers, return it without
   them
    // For now, qualifiers are stored directly on
   type descriptors
    // We simply clear the quals field (non-
   destructive: return same ptr)
    // Actually for correctness we return the same
    descriptor but
    // the caller should treat it as unqualified.
    // Simple implementation: just return the type
    as-is
    // (qualifier checking is separate from lvalue
    conversion)
    ret
```

□

297a · used in 29f · defines _kcc_conv_lvalue never used

27.5. Implicit Conversion Check

This predicate determines whether an implicit conversion from one type to another is valid. It returns 1 for same-type, integer-to-integer, arithmetic-to-arithmetic, pointer-to-pointer, and integer/pointer cross-conversions (the last with a diagnostic warning in later passes). The check is intentionally permissive so that sema can issue warnings rather than hard errors for narrowing or sign-changing conversions.

⟨ *Conv: implicit* 297b ⟩ ≡

```
// -- kcc_conv_implicit ---
// int kcc_conv_implicit(type_desc *from,
   type_desc *to)
//   x0 = source type, x1 = target type
// Returns 1 if implicit conversion →fromto is
   valid, 0 otherwise.
.text
.globl _kcc_conv_implicit
.p2align 2
_kcc_conv_implicit:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    mov x19, x0                     // from
    mov x20, x1                     // to

    // Same type → always valid
    cmp x19, x20
    b.eq .Lci_yes

    // Same kind → valid (widening or narrowing is
    allowed with warning)
    ldr x0, [x19, #TY_KIND]
    ldr x1, [x20, #TY_KIND]
    cmp x0, x1
    b.eq .Lci_yes

    // Integer → integer: always valid (with
   possible warning)
    mov x0, x19
    bl _kcc_type_is_integer
    cbz x0, .Lci_check_arith
    mov x0, x20
    bl _kcc_type_is_integer
    cbnz x0, .Lci_yes

.Lci_check_arith:
    // Arithmetic → arithmetic: valid
    mov x0, x19
    bl _kcc_type_is_arithmetic
    cbz x0, .Lci_check_ptr
    mov x0, x20
    bl _kcc_type_is_arithmetic
    cbnz x0, .Lci_yes

.Lci_check_ptr:
    // Pointer → pointer: valid (with warning for
   incompatible)
    ldr x0, [x19, #TY_KIND]
    cmp x0, #TY_PTR
    b.ne .Lci_check_int_ptr
    ldr x0, [x20, #TY_KIND]
    cmp x0, #TY_PTR
    b.eq .Lci_yes

.Lci_check_int_ptr:
    // Integer → pointer or pointer → integer:
   valid (with warning)
    ldr x0, [x19, #TY_KIND]
    cmp x0, #TY_PTR
    b.ne .Lci_from_not_ptr
    mov x0, x20
    bl _kcc_type_is_integer
    cbnz x0, .Lci_yes
    b .Lci_no

.Lci_from_not_ptr:
    mov x0, x19
    bl _kcc_type_is_integer
    cbz x0, .Lci_no
    ldr x0, [x20, #TY_KIND]
    cmp x0, #TY_PTR
    b.eq .Lci_yes

.Lci_no:
    mov x0, #0
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
.Lci_yes:
    mov x0, #1
```

```
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

297b · used in 29f · defines _kcc_conv_implicit never used · uses _kcc_type_is_arithmetic 248a, _kcc_type_is_integer 247b

Part VI.

Intermediate Representation

Between the tree-shaped AST and the flat machine code lies the intermediate representation: a control-flow graph of basic blocks, each containing a linear sequence of three-address instructions in static single-assignment (SSA) form. Chapter 28 defines the IR vocabulary — constants, arithmetic, comparisons, memory operations, branches, calls, and allocas — together with the data structures for functions, basic blocks, and instructions. Chapter 29 walks the typed AST and emits the corresponding IR, lowering C control flow into explicit branches and phi-less SSA. Chapter 30 provides a structural integrity check that verifies every basic block is properly terminated before the IR is handed to later phases.

28. Intermediate Representation

The front end produces an AST tied closely to C syntax; the back end needs a flat, register-based representation amenable to optimization and machine code emission. This chapter defines the intermediate representation that bridges the two. The IR follows a three-address, virtual-register design organized into basic blocks and functions—similar in spirit to LLVM IR but far simpler. We specify the 48-byte instruction layout, the basic-block and function data structures, the full opcode table covering arithmetic, comparison, memory, control flow, and function calls, and the core allocation routines that create each IR entity from the arena.

28.1. IR Instruction Layout

An IR instruction occupies 48 bytes:

Offset	Field	Description
0	IR_OP	opcode (see below)
8	IR_DEST	destination vreg number
16	IR_SRC1	source 1 (vreg or immediate)
24	IR_SRC2	source 2 (vreg, immediate, or BB id)
32	IR_TYPE	pointer to type descriptor
40	IR_NEXT	next instruction in basic block

⟨ *IR: constants* 301 ⟩ ≡

```
// -- IR instruction layout ---
.equ IR_OP,          0      // opcode
.equ IR_DEST,        8      // destination vreg
   number
.equ IR_SRC1,        16     // source 1 (vreg or
   immediate)
.equ IR_SRC2,        24     // source 2 (vreg,
   immediate, or BB id)
.equ IR_TYPE,        32     // pointer to type
   descriptor
.equ IR_NEXT,        40     // next instruction in
   BB
.equ IR_INSTR_SIZE, 48

// -- Basic block layout ---
.equ BB_ID,          0      // block label number
.equ BB_FIRST,       8      // first instruction
.equ BB_LAST,        16     // last instruction
.equ BB_NEXT,        24     // next BB in function
   (linked list)
.equ BB_FLAGS,       32     // bit 0 = terminated
.equ IR_BB_SIZE,     48

// -- IR function layout ---
.equ IRF_NAME,       0      // function name string
    pointer
.equ IRF_ENTRY,      8      // entry basic block
.equ IRF_BLOCKS,     16     // vec of BB pointers
.equ IRF_VREG_N,     24     // next vreg number
   counter
.equ IRF_RETTYPE,    32     // return type
   descriptor
.equ IRF_PARAMS,     40     // param count
.equ IRF_PARAM_CLS, 48      // arena array of per-
   param AAPCS64 classes (or 0)
.equ IRF_LINE,       56     // source declaration
   line (for DWARF; 0 if unknown)
.equ IRF_LOWPC,      64     // code offset where
   the function begins (set by codegen)
.equ IRF_STATIC,     72     // 1 if the function
   has internal linkage (static)
.equ IRF_SYMIDX,     80     // linker symbol index
   (pre-registered by codegen)
.equ IRF_VARIADIC,  88      // 1 if the function is
    variadic (affects stack-arg layout)
.equ IR_FUNC_SIZE,  96

// -- BB flags ---
.equ BB_FLAG_TERMINATED, 1

// -- IR opcodes ---
// Arithmetic: 1-8
.equ IR_ADD,       1
.equ IR_SUB,       2
.equ IR_MUL,       3
.equ IR_SDIV,      4
.equ IR_UDIV,      5
.equ IR_SMOD,      6
.equ IR_UMOD,      7
.equ IR_NEG,       8

// Bitwise: 10-16
.equ IR_AND,      10
.equ IR_OR,       11
.equ IR_XOR,      12
.equ IR_SHL,      13
.equ IR_LSHR,     14
.equ IR_ASHR,     15
.equ IR_NOT,      16

// Compare: 20-25
.equ IR_CMP_EQ,  20
.equ IR_CMP_NE,  21
.equ IR_CMP_SLT, 22
.equ IR_CMP_SLE, 23
.equ IR_CMP_SGT, 24
.equ IR_CMP_SGE, 25
.equ IR_CMP_ULT, 26
.equ IR_CMP_ULE, 27

// Memory: 30-33
.equ IR_LOAD,     30
.equ IR_STORE,    31
.equ IR_ALLOCA,   32
.equ IR_GEP,      33
```

```
// Control flow: 40-44
.equ IR_BR,       40
.equ IR_BR_COND, 41
.equ IR_RET,      42
.equ IR_CALL,     43
.equ IR_CALL_IND, 44

// Conversion: 50-53
.equ IR_SEXT,     50
.equ IR_ZEXT,     51
.equ IR_TRUNC,    52
.equ IR_BITCAST, 53
.equ IR_SITOFP,   54      // signed integer → floating point
.equ IR_FPTOSI,   55      // floating point → signed integer
.equ IR_VA_START, 56      // SRC2 = fixed-param count; DEST = &first variadic arg
.equ IR_FPROUND,  57      // round a double vreg to single precision (double format)
.equ IR_UITOFP,   58      // unsigned integer → floating point (UCVTF, not SCVTF)

// Miscellaneous: 60-63
.equ IR_CONST,    60
.equ IR_NOP,      61
.equ IR_GLOBAL_REF, 63
.equ IR_STRING_LIT, 64
.equ IR_ASM,      65          // inline asm: SRC1= template ptr, SRC2=template len
```

□

301 · used in 30bc, 31ace

28.2. IR Initialization

The IR module maintains a global state: the current function being built, the current basic block, and a vector of all IR functions.

⟨ *IR: init* 302 ⟩ ≡

```
// -- kcc_ir_init ---
// void kcc_ir_init(void)
// Initializes the IR module.  Must be called before any IR construction.
.text
.globl _kcc_ir_init
.p2align 2
_kcc_ir_init:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    // Zero out current function/block pointers
    adrp x0, _kcc_ir_cur_func@PAGE
    add  x0, x0, _kcc_ir_cur_func@PAGEOFF
    str xzr, [x0]

    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    str xzr, [x0]

    // Reset module-init state.  The driver compiles every source file in one
    // process, so these must clear per translation unit---otherwise a later
    // unit inherits an earlier one's flag and emits a stray __mod_init_func
    // pointer bound to whatever symbol happens to share that stale index.
    adrp x0, _kcc_modinit_present@PAGE
    add  x0, x0, _kcc_modinit_present@PAGEOFF
    str xzr, [x0]
    adrp x0, _kcc_modinit_sym@PAGE
    add  x0, x0, _kcc_modinit_sym@PAGEOFF
    str xzr, [x0]
    adrp x0, _kcc_modinit_fileoff@PAGE
    add  x0, x0, _kcc_modinit_fileoff@PAGEOFF
    str xzr, [x0]

    // Initialize function list vector (8 bytes per pointer, cap 16)
    mov x0, #VEC_SIZE
    bl _kcc_arena_alloc
    adrp x1, _kcc_ir_funcs@PAGE
    add  x1, x1, _kcc_ir_funcs@PAGEOFF
    str x0, [x1]
    mov x1, #8              // elem size = pointer
    str x1, [x0, #VEC_ELEMSIZE]
    mov x1, #16
    str x1, [x0, #VEC_CAP]
    str xzr, [x0, #VEC_COUNT]
    mov x19, x0             // save vec

    // Allocate initial data array
    mov x0, #128            // 16 * 8
    bl _kcc_arena_alloc
    str x0, [x19, #VEC_DATA]

    // Initialize local variable hash table
    mov x0, #64
    bl _kcc_ht_init
    adrp x1, _kcc_ir_locals@PAGE
    add  x1, x1, _kcc_ir_locals@PAGEOFF
    str x0, [x1]

    // Initialize break/continue stack (vec of pairs, 16 bytes each)
    mov x0, #VEC_SIZE
    bl _kcc_arena_alloc
    adrp x1, _kcc_ir_loop_stack@PAGE
    add  x1, x1, _kcc_ir_loop_stack@PAGEOFF
    str x0, [x1]
    mov x1, #16             // elem size = 2 pointers (continue_bb, break_bb)
    str x1, [x0, #VEC_ELEMSIZE]
    mov x1, #8
    str x1, [x0, #VEC_CAP]
    str xzr, [x0, #VEC_COUNT]
    mov x19, x0

    mov x0, #128            // 8 * 16
    bl _kcc_arena_alloc
    str x0, [x19, #VEC_DATA]

    // Zero the BB counter
    adrp x0, _kcc_ir_bb_counter@PAGE
    add  x0, x0, _kcc_ir_bb_counter@PAGEOFF
    str xzr, [x0]

    ldp x29, x30, [sp], #16
    ret
```

□

302 · used in 30b · defines _kcc_ir_init 603, 623 · uses _kcc_arena_alloc 58a, _kcc_ht_init 61c, _kcc_ir_bb_counter 305c, _kcc_ir_cur_bb 305c, _kcc_ir_cur_func 305c, _kcc_ir_funcs 305c, _kcc_ir_locals 305c, _kcc_ir_loop_stack 305c, _kcc_modinit_fileoff 305c, _kcc_modinit_present 305c, _kcc_modinit_sym 305c

28.3. Allocating IR Nodes

Three allocation functions arena-allocate and zero-initialise the core IR data structures. Each returns a pointer to the freshly allocated node.

⟨ *IR: alloc instruction* 303a ⟩ ≡

```
// -- kcc_ir_alloc_instr ---
// ir_instr *kcc_ir_alloc_instr(uint64_t opcode)
//   x0 = opcode
// Returns a zeroed IR instruction with the given
   opcode.
.text
.globl _kcc_ir_alloc_instr
.p2align 2
_kcc_ir_alloc_instr:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0              // save opcode

    mov x0, #IR_INSTR_SIZE
    bl _kcc_arena_alloc

    str x19, [x0, #IR_OP]
    str xzr, [x0, #IR_DEST]
    str xzr, [x0, #IR_SRC1]
    str xzr, [x0, #IR_SRC2]
    str xzr, [x0, #IR_TYPE]
    str xzr, [x0, #IR_NEXT]

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

303a · used in 30b · defines _kcc_ir_alloc_instr 312, 314b, 316, 317, 320, 321, 322, 326, 343, 347, 350, 353a, 355, 357, 359, 367, 369, 375, 376, 378b, 383, 384, 387, 393, 398, 400bc, 402b, 404, 405, 407, 413, 417b, 421c, 423ab, 424, 426, 435ab, 450 · uses _kcc_arena_alloc 58a

Basic blocks are allocated with a unique ID drawn from a per-function counter.

⟨ *IR: alloc basic block* 303b ⟩ ≡

```
// -- kcc_ir_alloc_bb ---
// ir_bb *kcc_ir_alloc_bb(void)
// Returns a new basic block with a unique ID.
.text
.globl _kcc_ir_alloc_bb
.p2align 2
_kcc_ir_alloc_bb:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x0, #IR_BB_SIZE
    bl _kcc_arena_alloc
    mov x19, x0

    // Assign unique BB id
    adrp x1, _kcc_ir_bb_counter@PAGE
    add  x1, x1, _kcc_ir_bb_counter@PAGEOFF
    ldr x2, [x1]
    str x2, [x19, #BB_ID]
    add x2, x2, #1
    str x2, [x1]

    str xzr, [x19, #BB_FIRST]
    str xzr, [x19, #BB_LAST]
    str xzr, [x19, #BB_NEXT]
    str xzr, [x19, #BB_FLAGS]

    mov x0, x19

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

303b · used in 30b · defines _kcc_ir_alloc_bb 316, 317, 321, 322, 376, 400c, 421c, 423ab, 424, 426 · uses _kcc_arena_alloc 58a, _kcc_ir_bb_counter 305c

A function node holds the name, parameter count, a vector of basic blocks, and the vreg counter. It also carries an `IRF_STATIC` flag—set by the builder from the declaration's storage class—that later tells the emitter to give a `static` function local Mach-O linkage.

⟨ *IR: alloc function* 303c ⟩ ≡

```
// -- kcc_ir_alloc_func ---
// ir_func *kcc_ir_alloc_func(const char *name,
   type_desc *ret_type)
//   x0 = function name pointer
//   x1 = return type descriptor
// Returns a new IR function.
.text
.globl _kcc_ir_alloc_func
.p2align 2
_kcc_ir_alloc_func:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0              // save name
    mov x20, x1              // save ret_type

    mov x0, #IR_FUNC_SIZE
    bl _kcc_arena_alloc
    mov x21, x0              // x21 = func

    str x19, [x21, #IRF_NAME]
    str x20, [x21, #IRF_RETTYPE]
    str xzr, [x21, #IRF_ENTRY]
    str xzr, [x21, #IRF_VREG_N]
    str xzr, [x21, #IRF_PARAMS]
    str xzr, [x21, #IRF_PARAM_CLS]
    str xzr, [x21, #IRF_LINE]      // set by the
   builder from the FUNC_DECL node
    str xzr, [x21, #IRF_LOWPC]     // set by
   codegen at function entry
    str xzr, [x21, #IRF_STATIC]    // extern by
   default; builder sets it for static
    str xzr, [x21, #IRF_SYMIDX]    // set by the
   codegen function pre-pass

    // Allocate blocks vec
    mov x0, #VEC_SIZE
    bl _kcc_arena_alloc
    str x0, [x21, #IRF_BLOCKS]
    mov x1, #8
    str x1, [x0, #VEC_ELEMSIZE]
    mov x1, #16
    str x1, [x0, #VEC_CAP]
    str xzr, [x0, #VEC_COUNT]
```

```
    mov x19, x0

    mov x0, #128              // 16 * 8
    bl _kcc_arena_alloc
    str x0, [x19, #VEC_DATA]

    mov x0, x21               // return func

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

303c · used in 30b · defines _kcc_ir_alloc_func 316, 317 · uses _kcc_arena_alloc 58a

28.4. Emitting Instructions

The emit helper appends an instruction to the current basic block's linked list. It also sets the instruction's BB field so validation can check that every instruction belongs to exactly one block.

A companion helper, _kcc_ir_emit_entry, instead inserts an instruction at the *head* of the current function's entry block. It exists for one purpose: the ALLOCA that reserves a local variable's stack storage must *dominate* every use of that variable's address. If the alloca were emitted at the lexical declaration point—as the natural statement walk would do—a switch case label or a goto target lying *after* the declaration but inside the same block could be entered directly, jumping past the alloca and leaving the address vreg undefined. This is exactly the shape of SQLite's sqlite3VdbeExec, where case OP_OpenRead: sits inside the brace block opened at case OP_ReopenIdx: and dispatch jumps straight to it, past the declarations of p2 and its siblings. Because an ALLOCA takes no vreg operands—its only source is a constant slot count—placing it at the very front of the entry block is always dominance-correct, and the entry block dominates the whole function. The initialising store stays at the declaration point; only the address reservation moves.

⟨ *IR: emit* 304 ⟩ ≡

```
// -- kcc_ir_emit ---
// void kcc_ir_emit(ir_instr *instr)
// Appends an instruction to the current basic
   block.
// If the BB is already terminated, the
   instruction is silently dropped.
.text
.globl _kcc_ir_emit
.p2align 2
_kcc_ir_emit:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0               // x19 = instr

    // Get current BB
    adrp x1, _kcc_ir_cur_bb@PAGE
    add  x1, x1, _kcc_ir_cur_bb@PAGEOFF
    ldr x1, [x1]
    cbz x1, .Lie_done         // no current BB

    // Check if BB is terminated
    ldr x2, [x1, #BB_FLAGS]
    tst x2, #BB_FLAG_TERMINATED
    b.ne .Lie_done            // already
   terminated, drop

    // Append to BB's instruction list
    ldr x2, [x1, #BB_LAST]
    cbz x2, .Lie_first

    // Not first: link after last
    str x19, [x2, #IR_NEXT]
    str x19, [x1, #BB_LAST]
    b .Lie_done

.Lie_first:
    str x19, [x1, #BB_FIRST]
    str x19, [x1, #BB_LAST]

.Lie_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_emit_entry ---
// void kcc_ir_emit_entry(ir_instr *instr)
// Inserts an instruction at the HEAD of the
   current function's entry block,
// ahead of everything already there.  Used for
   allocas, whose address must
// dominate every use regardless of goto/switch
   control flow.
.text
.globl _kcc_ir_emit_entry
.p2align 2
_kcc_ir_emit_entry:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0               // x19 = instr

    // Get current function, then its entry BB.
    adrp x1, _kcc_ir_cur_func@PAGE
    add  x1, x1, _kcc_ir_cur_func@PAGEOFF
    ldr x1, [x1]
    cbz x1, .Lien_done
    ldr x1, [x1, #IRF_ENTRY]  // entry basic block
    cbz x1, .Lien_done

    // Head-insert: instr->next = bb->first; bb->
   first = instr.
    ldr x2, [x1, #BB_FIRST]
    str x2, [x19, #IR_NEXT]
    str x19, [x1, #BB_FIRST]

    // If the block was empty, instr is also the
   last.
    ldr x2, [x1, #BB_LAST]
    cbnz x2, .Lien_done
    str x19, [x1, #BB_LAST]

.Lien_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

304 · used in 30b · defines _kcc_ir_emit 312, 314b, 316, 317, 320, 321, 322, 326, 343, 347, 350, 353a, 355, 357, 359, 367, 369, 375, 376, 378b, 383, 384, 387, 393, 398, 400bc, 402b, 404, 405, 407, 413, 417b, 421c, 423ab, 424, 426, 435ab, _kcc_ir_emit_entry 417b · uses _kcc_ir_cur_bb 305c,

_kcc_ir_cur_func 305c

28.5. Virtual Register Allocation

Each function maintains a monotonically increasing vreg counter.

⟨ *IR: next vreg* 305a ⟩ ≡

```
// -- kcc_ir_next_vreg ---
// uint64_t kcc_ir_next_vreg(void)
// Returns the next virtual register number for
   the current function.
.text
.globl _kcc_ir_next_vreg
.p2align 2
_kcc_ir_next_vreg:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _kcc_ir_cur_func@PAGE
    add  x0, x0, _kcc_ir_cur_func@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lnv_zero

    ldr x1, [x0, #IRF_VREG_N]
    add x2, x1, #1
    str x2, [x0, #IRF_VREG_N]
    mov x0, x1

    ldp x29, x30, [sp], #16
    ret

.Lnv_zero:
    mov x0, #0
    ldp x29, x30, [sp], #16
    ret
```

□

305a · used in 30b · defines _kcc_ir_next_vreg 312, 317, 320, 343, 347, 350, 353a, 355, 357, 359, 367, 369, 375, 376, 378b, 383, 384, 387, 393, 398, 400bc, 402b, 404, 405, 407, 413, 417b, 426 · uses _kcc_ir_cur_func 305c

28.6. Switching Basic Blocks

When the IR builder starts a new basic block, it calls `kcc_ir_set_bb` to redirect subsequent `emit` calls to the new block. The old current-block pointer is not saved—the builder is responsible for managing block references in callee-saved registers.

⟨ *IR: set current bb* 305b ⟩ ≡

```
// -- kcc_ir_set_bb ---
// void kcc_ir_set_bb(ir_bb *bb)
// Sets bb as the current insertion point and
   appends it to the
// current function's block list.
.text
.globl _kcc_ir_set_bb
.p2align 2
_kcc_ir_set_bb:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0

    // Set as current BB
    adrp x1, _kcc_ir_cur_bb@PAGE
    add  x1, x1, _kcc_ir_cur_bb@PAGEOFF
    str x19, [x1]

    // Add to current function's block vec
    adrp x0, _kcc_ir_cur_func@PAGE
    add  x0, x0, _kcc_ir_cur_func@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lisb_done

    ldr x0, [x0, #IRF_BLOCKS]   // vec
    // vec_push takes (x0=vec, x1=
   pointer_to_element)
    // Store x19 on stack so we can pass its
   address
    str x19, [sp, #24]          // use stack slot
    for element
    add x1, sp, #24
    bl _kcc_vec_push

.Lisb_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

305b · used in 30b · defines _kcc_ir_set_bb 316, 317, 322, 342, 376, 400c, 421c, 423ab, 424, 426 · uses _kcc_ir_cur_bb 305c, _kcc_ir_cur_func 305c, _kcc_vec_push 65a

28.7. IR Module Data

Module-level state: the current function being built, the current basic block receiving emitted instructions, and the global function vector that `codegen_all` iterates.

Alongside the `locals` table sits a second per-function map, `statics`, keyed the same way (source name) but holding the *mangled* symbol name of a `static` local rather than an alloca vreg. A static local has no stack storage—it is hoisted to a uniquely-named global (§29.11.1)—so the identifier and assignment builders consult this table, before falling back to a file-scope global, to redirect a reference onto the hoisted symbol. Like `locals` it is re-initialised for each function so that a static's name is visible only within its defining function.

⟨ *IR: data* 305c ⟩ ≡

```
// -- IR module data ---
.data
.p2align 3
.globl _kcc_ir_cur_func
_kcc_ir_cur_func:    .quad 0     // current IR
   function being built
.globl _kcc_ir_cur_bb
_kcc_ir_cur_bb:      .quad 0     // current basic
   block for emission
.globl _kcc_ir_funcs
_kcc_ir_funcs:       .quad 0     // vec of all IR
   functions
.globl _kcc_ir_locals
_kcc_ir_locals:      .quad 0     // hash table:
   name → alloca vreg
.globl _kcc_ir_statics
_kcc_ir_statics:     .quad 0     // hash table:
   name → mangled static-local symbol
.globl _kcc_ir_static_types
_kcc_ir_static_types: .quad 0    // hash table:
   name → static local's declared type
```

```
.globl _kcc_ir_labels
_kcc_ir_labels:      .quad 0     // hash table:
   label name → target basic block
.globl _kcc_ir_loop_stack
_kcc_ir_loop_stack:  .quad 0     // vec of (
   continue_bb, break_bb) pairs
.globl _kcc_ir_bb_counter
_kcc_ir_bb_counter:  .quad 0     // global BB id
   counter
.globl _kcc_ir_scope_jdepth
_kcc_ir_scope_jdepth: .quad 0    // live entries
   in the scope-shadowing journal
.globl _kcc_ir_scope_sp
_kcc_ir_scope_sp:    .quad 0     // open block
   scopes on the scope stack
.globl _kcc_modinit_present
_kcc_modinit_present: .quad 0    // 1 if this TU
   synthesised a module-init function
.globl _kcc_modinit_sym
_kcc_modinit_sym:    .quad 0     // module-init
   symbol index + 1 (0 = none), set by codegen
.globl _kcc_modinit_fileoff
_kcc_modinit_fileoff: .quad 0    // file offset of
    the __mod_init_func pointer, set by the writer

// Block-scope shadowing journal (zero-filled, so
   .bss keeps it out of the
// object image).  Each open block records, per
   name it declares or shadows, a
// {name, keylen, previous locals-map value}
   triple; leaving the block replays
// them in reverse to restore the enclosing
   bindings.  16384 live entries and
// 1024 nested scopes comfortably exceed any real
   function.
.bss
.p2align 3
.globl _kcc_ir_scope_journal
_kcc_ir_scope_journal: .space 16384*24
.globl _kcc_ir_scope_stack
_kcc_ir_scope_stack:  .space 1024*8
.data
```

□

305c · used in 30b · defines _kcc_ir_bb_counter 302, 303b, _kcc_ir_cur_bb 302, 304, 305b, 316, 317, 321, 326, 376, 400c, 421c, 423ab, 424, 426, 435ab, _kcc_ir_cur_func 302, 304, 305ab, 316, 317, 326, 398, _kcc_ir_funcs 302, 316, 317, 334, 441, 445a, 472, 473, 560b, 596, _kcc_ir_labels 316, 317, 321, _kcc_ir_locals 302, 316, 317, 320, 322, 350, 355, 367, 378b, 382, 387, 393, 398, 405, _kcc_ir_loop_stack 302, _kcc_ir_scope_jdepth 316, 322, _kcc_ir_scope_journal 322, _kcc_ir_scope_sp 316, 322, _kcc_ir_scope_stack 322, _kcc_ir_statics 312, 316, 317, 350, 355, 367, 378b, 383, 387, 417b, _kcc_ir_static_types 312, 316, 317, 350, 367, 417b, _kcc_modinit_fileoff 302, 580, 582, 583, _kcc_modinit_present 302, 316, 567, 580, 582, 583, 586, 587, 588, _kcc_modinit_sym 302, 472, 543, 587

29. IR Builder

With the IR data structures defined in Chapter 28, we now need machinery to translate the typed AST into linear IR. This chapter implements the IR builder, which walks the AST recursively and emits IR instructions for each expression, statement, and declaration. Control-flow constructs (`if`/`else`, `while`, `for`, `do`/`while`, `break`/`continue`) are lowered into conditional and unconditional branches between basic blocks. The chapter also provides the textual dump used by the `--dump-ir` flag for debugging.

29.1. IR Builder Entry Point

The builder walks the translation unit, processing each function declaration. Non-function declarations (globals, typedefs) are skipped for now.

Module initialisation across translation units. Global initializers are lowered to runtime stores rather than a pre-initialised data image (§29.1.1). For the translation unit that defines `main` those stores are injected at the top of `main`, which runs before any user code. But a translation unit that does *not* define `main`—a library object compiled separately and linked in—has no such entry point, so its file-scope initializers would never execute and its globals would keep their zero (BSS) image. A separately compiled `int g = 7;` would read as 0; a global function pointer would be `NULL` and calling it would fault.

The fix is a per-unit *module-init function*. When this unit has no `main` but does carry file-scope initializers, we synthesise a function holding exactly the same initializer stores and register its address in a `(__DATA,__mod_init_func)` section (§40.3.2); `dyld` runs every module initializer before `main`, for every unit in the image. The unit that owns `main` is left untouched—its initializers still run at `main`'s entry—so single-unit programs are unaffected.

⟨ *IR: build* 307 ⟩ ≡

```
// -- kcc_ir_build ---
// void kcc_ir_build(node *trans_unit)
//   x0 = AST_TRANS_UNIT node
// Walks all top-level declarations and builds IR
   for functions.
.text
.globl _kcc_ir_build
.p2align 2
_kcc_ir_build:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    // Save translation unit for global init
   emission in main
    adrp x1, _kcc_ir_trans_unit@PAGE
    add  x1, x1, _kcc_ir_trans_unit@PAGEOFF
    str x0, [x1]
    ldr x19, [x0, #AST_F0]      // first
   declaration

.Lirb_loop:
    cbz x19, .Lirb_scan

    // Check if it's a function declaration with a
    body
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_FUNC_DECL
    b.ne .Lirb_next

    // F3 = body (compound stmt); skip prototypes
    ldr x0, [x19, #AST_F3]
    cbz x0, .Lirb_next

    mov x0, x19
    bl _kcc_ir_build_func

.Lirb_next:
    ldr x19, [x19, #AST_F4]     // next sibling
    b .Lirb_loop

.Lirb_scan:
    // Decide whether this unit needs a module-
   init function: it must lack a
    // main definition yet carry at least one
   initializer to run at startup ---
    // a file-scope one, or a `static` local
   initializer inside a function body.
    adrp x0, _kcc_ir_trans_unit@PAGE
    add  x0, x0, _kcc_ir_trans_unit@PAGEOFF
    ldr x0, [x0]
    ldr x19, [x0, #AST_F0]      // first
   declaration
    mov x20, #0                  // has_main
    mov x21, #0                  // has_ginit
.Lirb_scan_loop:
    cbz x19, .Lirb_scan_done
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_FUNC_DECL
    b.ne .Lirb_scan_var
    ldr x0, [x19, #AST_F3]      // body → a
   definition, not a prototype
    cbz x0, .Lirb_scan_next

    // Scan the body for a `static` local with an
   initializer: it too must run
    // once at startup, so in a non-main unit it
   is what obliges a module-init
    // function (no calls here, so x9/x10 survive
   to the main check below).
    ldr x9, [x0, #AST_F0]       // first body
   statement
.Lirb_scan_body:
    cbz x9, .Lirb_scan_main
    ldr x10, [x9, #AST_TAG]
    cmp x10, #AST_VAR_DECL
    b.ne .Lirb_scan_body_next
    ldr x10, [x9, #AST_F3]      // storage class
   (3 = static)
    cmp x10, #3
    b.ne .Lirb_scan_body_next
    ldr x10, [x9, #AST_F2]      // initializer
   present?
    cbz x10, .Lirb_scan_body_next
    mov x21, #1                  // static-local
   initializer → needs module init
    b .Lirb_scan_main
.Lirb_scan_body_next:
    ldr x9, [x9, #AST_F4]
    b .Lirb_scan_body
```

```
.Lirb_scan_main:
    ldr x0, [x19, #AST_F1]       // name
    cbz x0, .Lirb_scan_next
    adrp x1, _kcc_str_main@PAGE
    add  x1, x1, _kcc_str_main@PAGEOFF
    bl _kcc_strcmp
    cbnz x0, .Lirb_scan_next
    mov x20, #1                   // defines main
    b .Lirb_scan_next
.Lirb_scan_var:
    cmp x0, #AST_VAR_DECL
    b.ne .Lirb_scan_next
    ldr x0, [x19, #AST_F2]       // initializer
   present?
    cbz x0, .Lirb_scan_next
    mov x21, #1                   // has a file-
   scope initializer
.Lirb_scan_next:
    ldr x19, [x19, #AST_F4]
    b .Lirb_scan_loop
.Lirb_scan_done:
    // Synthesise module-init only for a non-main
   unit that has initializers.
    cbnz x20, .Lirb_done
    cbz x21, .Lirb_done
    bl _kcc_ir_emit_modinit

.Lirb_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

307 · defined at 307, 308, 312, 314ab, 316 · used in 30b · defines _kcc_ir_build 603, 623 · uses _kcc_ir_build_func 317, _kcc_ir_emit_modinit 316

29.1.1. Global Variable Initializers

File-scope variables with initializers (`int x = 42;`) are not placed in a pre-initialized data section—the all-spill model allocates all globals in BSS. Instead, the builder emits store instructions at the top of `main()`'s entry block: an `IR_GLOBAL_REF` materialises the address, the initializer expression produces a value vreg, and an `IR_STORE` writes it. This runs before any user statements in `main`.

A brace-enclosed initializer for a file-scope array (`int t[3] = {7, 8, 9}`) is lowered the same way, one `_kcc_ir_emit_indexed_store` per element against the single `IR_GLOBAL_REF` base. Because BSS is already zero, a short list needs no explicit tail fill: the unmentioned elements retain their zero image. These global tables are exactly the lookup tables a self-hosted lexer relies on.

A second pass then seeds the `static` locals (§29.11.1). Since these are hoisted to globals but their declarations live inside function bodies, this pass walks each function's top-level statement list for `static` declarations. Before emitting any store it first re-registers that function's static-locals in the mangled-name table: the module-init function reset that table, so an initialiser that takes the address of a sibling static (`static T *p[] = { other_static, …}`) would otherwise resolve to the source name and link against an undefined symbol. It then materialises each static's own mangled symbol with an `IR_GLOBAL_REF`, and hands it to the shared `_kcc_ir_emit_init_stores` routine—so a scalar, an aggregate list, or a string static is initialised by exactly the same code path a file-scope global uses. Running it here—at the top of `main`—is what gives a static its *initialise once* semantics: the store executes at program start rather than on every entry to the declaring function. A static without an initialiser, or one initialised to zero, needs nothing here; BSS already supplies the zero. Static locals declared inside nested blocks are not yet swept by this pass, so a non-zero initialiser on one is not applied (its storage still persists, starting at zero).

Read-only const globals (M5). Most globals are zero-initialised storage seeded at load time by the stores this routine emits. A `const` global whose initialiser is a pure integer byte image is different: the back end places its bytes in the read-only `__TEXT` tail (§B.7), where a stray write faults rather than silently corrupting a neighbour. Such an object has *no* writable storage, so this routine must *not* emit a runtime store for it—a store into a read-only page would fault at startup. `_kcc_global_const_rodata` is the predicate both this routine (to suppress the store) and the code generator (to lay the bytes) consult; being a pure function of the declaration AST, they always agree. It reads `const` straight from the declarator—peeling `AST_TYPE_ARRAY` to the innermost `AST_TYPE_QUALS` and testing bit 0—because semantics drops the qualifier from resolved descriptors, and it accepts an integer *or floating* scalar or array whose initialiser is a matching scalar or a flat list of matching leaves (`_kcc_rodata_is_lit`): for an integer element, any constant expression built from integer/character literals, enum-constant identifiers, and the operators `_kcc_eval_const_int` folds (`_kcc_rodata_const_intexpr`, so `{1<<0, 1<<1}`, a masked literal, or `{STATE_A, STATE_B}` qualifies); for a `float`/`double` element, a floating literal or a unary minus of one. A `struct` of scalar fields with a positional literal initialiser—and a 1-D array of such structs (`_kcc_rodata_struct_ok` per element, via `_kcc_rodata_aos_ok`)—is also accepted, laid field-by-field at each member's offset. Anything else (`sizeof`, non-enum identifiers, designators, pointers, bit-fields, brace elision, multi-dimensional arrays of structs, structs with aggregate fields) falls back to the ordinary writable path, correct but unprotected.

⟨ *IR: build* 308 ⟩ +≡

```
// -- kcc_rodata_is_lit ---
// uint64_t kcc_rodata_binop_ok(uint64_t op)
// 1 iff op is a binary operator
   _kcc_eval_const_int folds (arithmetic, shift,
// bitwise).  Comparison and logical ops are
   excluded so an accepted expression
// tree never reaches eval's return-0 default (
   which would silently misfold).
.text
.globl _kcc_rodata_binop_ok
.p2align 2
_kcc_rodata_binop_ok:
    cmp x0, #TOK_PLUS
    b.eq .Lrbo_yes
    cmp x0, #TOK_MINUS
    b.eq .Lrbo_yes
    cmp x0, #TOK_STAR
    b.eq .Lrbo_yes
    cmp x0, #TOK_SLASH
    b.eq .Lrbo_yes
    cmp x0, #TOK_PERCENT
```

```
    b.eq .Lrbo_yes
    cmp x0, #TOK_SHL
    b.eq .Lrbo_yes
    cmp x0, #TOK_SHR
    b.eq .Lrbo_yes
    cmp x0, #TOK_AMP
    b.eq .Lrbo_yes
    cmp x0, #TOK_PIPE
    b.eq .Lrbo_yes
    cmp x0, #TOK_CARET
    b.eq .Lrbo_yes
    mov x0, #0
    ret
.Lrbo_yes:
    mov x0, #1
    ret

// uint64_t kcc_rodata_const_intexpr(node *n)
// 1 iff n is an integer constant expression built
    from integer/character
// literals, enum-constant identifiers, and the
   operators _kcc_eval_const_int
// folds.  An identifier is accepted only when it
   names an enum constant, looked
// up in the scope stack that both IR-build and
   codegen see identically (neither
// modifies the frozen post-sema stack), so the
   predicate agrees in both phases;
// sizeof and non-enum identifiers are rejected so
    eval never hits its return-0
// default.  This lets bit-flag tables (`{1<<0,
   1<<1}`), masked literals, negated
// constants, and enum tables (`{STATE_A, STATE_B
   }`) reach rodata.
.text
.globl _kcc_rodata_const_intexpr
.p2align 2
_kcc_rodata_const_intexpr:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0
    cbz x19, .Lrci_no
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_INT_LIT
    b.eq .Lrci_yes
    cmp x0, #AST_CHAR_LIT
    b.eq .Lrci_yes
    cmp x0, #AST_PAREN_EXPR
    b.eq .Lrci_paren
    cmp x0, #AST_CAST_EXPR
    b.eq .Lrci_cast
    cmp x0, #AST_UNARY_EXPR
    b.eq .Lrci_unary
    cmp x0, #AST_BINARY_EXPR
    b.eq .Lrci_binary
    cmp x0, #AST_IDENT_EXPR
    b.eq .Lrci_ident
    cmp x0, #AST_TYPES_COMPAT
    b.eq .Lrci_yes                  //
   __builtin_types_compatible_p folds to 0/1
    b .Lrci_no
.Lrci_ident:
    // An identifier is a constant only when it
   names an enum constant; the
    // symbol table is consulted identically at IR
   -build and codegen (both after
    // sema), so the predicate agrees in both
   phases and _kcc_eval_const_int
    // yields the enumerator's value.
    ldr x0, [x19, #AST_F0]          // name ptr
    cbz x0, .Lrci_no
    bl _kcc_strlen
    mov x1, x0
    ldr x0, [x19, #AST_F0]
    bl _kcc_symtab_lookup
    cbz x0, .Lrci_no
    ldr x1, [x0, #SYM_KIND]
    cmp x1, #SYM_ENUM_CONST
    b.ne .Lrci_no
    b .Lrci_yes
.Lrci_paren:
    ldr x0, [x19, #AST_F0]
    bl _kcc_rodata_const_intexpr
    b .Lrci_done
.Lrci_cast:
    ldr x0, [x19, #AST_F1]          // cast value
   (type ignored, as eval does)
    bl _kcc_rodata_const_intexpr
    b .Lrci_done
.Lrci_unary:
    ldr x1, [x19, #AST_F0]          // op: minus/
   tilde/bang are folded by eval
    cmp x1, #TOK_MINUS
    b.eq .Lrci_un_ok
    cmp x1, #TOK_TILDE
    b.eq .Lrci_un_ok
    cmp x1, #TOK_BANG
    b.ne .Lrci_no
.Lrci_un_ok:
    ldr x0, [x19, #AST_F1]          // operand
    bl _kcc_rodata_const_intexpr
    b .Lrci_done
.Lrci_binary:
    ldr x0, [x19, #AST_F0]          // op must be
   in the folded set
    bl _kcc_rodata_binop_ok
    cbz x0, .Lrci_no
    ldr x0, [x19, #AST_F1]          // lhs
    bl _kcc_rodata_const_intexpr
    cbz x0, .Lrci_no
    ldr x0, [x19, #AST_F2]          // rhs
    bl _kcc_rodata_const_intexpr
    b .Lrci_done
.Lrci_yes:
    mov x0, #1
    b .Lrci_done
.Lrci_no:
    mov x0, #0
.Lrci_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// uint64_t kcc_rodata_is_lit(node *n, uint64_t
   isfloat)
// For an integer element, n must be an integer
   constant expression
// (kcc_rodata_const_intexpr).  For a floating
   element, a floating literal or a
// unary minus of one -- _kcc_eval_const_int is
   integer-only, so a float leaf
// stays a literal whose bits come straight from
   AST_F0.
.text
.globl _kcc_rodata_is_lit
.p2align 2
_kcc_rodata_is_lit:
    cbnz x1, .Lril_float
    b _kcc_rodata_const_intexpr     // integer:
   tail-call the validator
.Lril_float:
    ldr x2, [x0, #AST_TAG]
    cmp x2, #AST_UNARY_EXPR
    b.ne .Lril_fdirect
    ldr x3, [x0, #AST_F0]           // unary op
    cmp x3, #TOK_MINUS
    b.ne .Lril_fno
    ldr x0, [x0, #AST_F1]           // operand
   must be a floating literal
```

```
    cbz x0, .Lril_fno
    ldr x2, [x0, #AST_TAG]
.Lril_fdirect:
    cmp x2, #AST_FLOAT_LIT
    b.eq .Lril_fyes
.Lril_fno:
    mov x0, #0
    ret
.Lril_fyes:
    mov x0, #1
    ret

// uint64_t kcc_rodata_struct_ok(type *sty, node *init)
// 1 iff sty is a struct whose members are all scalar integer/floating fields
// (no bit-fields, no nested aggregates) and init is a positional list of
// matching literals -- so the struct's byte image can be laid field-by-field.
.text
.globl _kcc_rodata_struct_ok
.p2align 2
_kcc_rodata_struct_ok:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                     // struct type
    ldr x0, [x1, #AST_TAG]          // init must be a brace list
    cmp x0, #AST_INIT_LIST
    b.ne .Lrso_no
    ldr x20, [x1, #AST_F0]          // x20 = init element cursor
    mov x0, x19                     // force layout so offsets/sizes are valid
    bl _kcc_type_layout
    ldr x0, [x19, #TY_F0]           // AST_STRUCT_DECL
    cbz x0, .Lrso_no
    ldr x21, [x0, #AST_F1]          // x21 = field cursor
.Lrso_loop:
    cbz x20, .Lrso_yes              // init exhausted: remaining fields stay zero
    cbz x21, .Lrso_no               // more initializers than fields
    ldr x0, [x21, #AST_F2]          // reject bit-fields (width expr present)
    cbnz x0, .Lrso_no
    ldr x0, [x20, #AST_TAG]         // reject a designated element (positional only)
    cmp x0, #AST_DESIGNATOR
    b.eq .Lrso_no
    ldr x0, [x21, #AST_F0]          // field type must be scalar int/float
    cbz x0, .Lrso_no
    bl _kcc_sema_resolve_type
    cbz x0, .Lrso_no
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_BOOL
    b.lt .Lrso_no
    cmp x1, #TY_DOUBLE
    b.gt .Lrso_no
    cmp x1, #TY_FLOAT               // matching literal (float leaf if kind>=FLOAT)
    cset x1, ge
    mov x0, x20
    bl _kcc_rodata_is_lit
    cbz x0, .Lrso_no
    ldr x21, [x21, #AST_F4]         // next field
    ldr x20, [x20, #AST_F4]         // next element
    b .Lrso_loop
.Lrso_yes:
    mov x0, #1
    b .Lrso_done
.Lrso_no:
    mov x0, #0
.Lrso_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// uint64_t kcc_rodata_aos_ok(type *sty, node *outer)
// 1 iff outer is a brace list whose every element is itself a brace list that
// passes kcc_rodata_struct_ok for the element struct type -- i.e. a 1-D array of
// scalar-field structs, each initialised positionally.  A short list leaves its
// trailing structs zero; a flat (brace-elided) element is rejected.
.text
.globl _kcc_rodata_aos_ok
.p2align 2
_kcc_rodata_aos_ok:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                     // element struct type
    ldr x0, [x1, #AST_TAG]
    cmp x0, #AST_INIT_LIST
    b.ne .Laos_no
    ldr x21, [x1, #AST_F0]          // x21 = outer element cursor
.Laos_loop:
    cbz x21, .Laos_yes              // exhausted: remaining structs stay zero
    ldr x0, [x21, #AST_TAG]         // each element must be its own brace
    cmp x0, #AST_INIT_LIST
    b.ne .Laos_no
    mov x0, x19
    mov x1, x21
    bl _kcc_rodata_struct_ok
    cbz x0, .Laos_no
    ldr x21, [x21, #AST_F4]
    b .Laos_loop
.Laos_yes:
    mov x0, #1
    b .Laos_done
.Laos_no:
    mov x0, #0
.Laos_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_global_const_rodata ---
// uint64_t kcc_global_const_rodata(node *decl)
// 1 iff decl is a const-qualified global whose storage can be a fixed byte
// image: an integer/floating scalar or array with matching literal leaves, a
// (non-array) struct of scalar fields, or a 1-D array of such structs, each with
// a positional literal initializer.
.text
.globl _kcc_global_const_rodata
.p2align 2
_kcc_global_const_rodata:
```

```
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0
    mov x22, #0                     // x22 = count
    of array wrappers peeled
    ldr x0, [x19, #AST_F2]          // must have an
    initializer
    cbz x0, .Lgcr_no
    ldr x1, [x19, #AST_F0]          // declared
   type spec
.Lgcr_peel:
    cbz x1, .Lgcr_no
    ldr x2, [x1, #AST_TAG]
    cmp x2, #AST_TYPE_ARRAY
    b.ne .Lgcr_qual
    add x22, x22, #1                // count the
   array wrapper
    ldr x1, [x1, #AST_F0]           // element type
    b .Lgcr_peel
.Lgcr_qual:
    cmp x2, #AST_TYPE_QUALS         // innermost
   must be a qualifier node
    b.ne .Lgcr_no
    ldr x2, [x1, #AST_F1]           // qualifier
   flags
    and x2, x2, #1                  // const bit
    cbz x2, .Lgcr_no
    ldr x0, [x1, #AST_F0]           // base type
   under the qualifier
    cbz x0, .Lgcr_no
    bl _kcc_sema_resolve_type
    cbz x0, .Lgcr_no
    ldr x2, [x0, #TY_KIND]
    // A struct base (no array wrapper) validates
   field-by-field; the integer
    // kinds TY_BOOL(2)..TY_LLONG(7) take integer
   literals; the floating kinds
    // TY_FLOAT(8)/TY_DOUBLE(9) take floating
   literals.  x21 = float mode.
    cmp x2, #TY_STRUCT
    b.eq .Lgcr_struct               // x0 = struct
    type
    cmp x2, #TY_BOOL
    b.lt .Lgcr_no
    cmp x2, #TY_LLONG
    b.le .Lgcr_intmode
    cmp x2, #TY_DOUBLE
    b.gt .Lgcr_no
    mov x21, #1                     // float/
   double element
    b .Lgcr_init
.Lgcr_struct:
    // x0 = struct type.  No array wrapper ->
   scalar struct; exactly one -> a
    // 1-D array of structs; more -> not handled.
    cbz x22, .Lgcr_scalar_struct
    cmp x22, #1
    b.ne .Lgcr_no
    ldr x1, [x19, #AST_F2]          // outer init
   list
    bl _kcc_rodata_aos_ok
    b .Lgcr_done
.Lgcr_scalar_struct:
    ldr x1, [x19, #AST_F2]          // init list (
   x0 = struct type)
    bl _kcc_rodata_struct_ok
    b .Lgcr_done
.Lgcr_intmode:
    mov x21, #0
.Lgcr_init:
    ldr x0, [x19, #AST_F2]          // initializer
    shape
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_INIT_LIST
    b.eq .Lgcr_list
    mov x1, x21                     // scalar: the
    init node itself
    bl _kcc_rodata_is_lit
    b .Lgcr_done
.Lgcr_list:
    ldr x20, [x0, #AST_F0]          // first
   element
.Lgcr_list_loop:
    cbz x20, .Lgcr_yes              // every
   element was a matching literal
    mov x0, x20
    mov x1, x21
    bl _kcc_rodata_is_lit
    cbz x0, .Lgcr_no
    ldr x20, [x20, #AST_F4]         // next
   element
    b .Lgcr_list_loop
.Lgcr_yes:
    mov x0, #1
    b .Lgcr_done
.Lgcr_no:
    mov x0, #0
.Lgcr_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// uint64_t kcc_global_const_ptrarr(node *decl)
// 1 iff decl is a 1-D array of const pointers (`
   const char *const opts[]`)
// initialised with string literals and/or integer
    (NULL) constants.  Such a
// table cannot live in immutable __TEXT---its
   pointers are relocated at load---so
// it goes to __DATA_CONST,__const (a separate
   emit path, kcc_emit_rodata_ptrarr).
.text
.globl _kcc_global_const_ptrarr
.p2align 2
_kcc_global_const_ptrarr:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0
    ldr x0, [x19, #AST_F2]          // must have
   an initializer
    cbz x0, .Lgcp_no
    ldr x1, [x19, #AST_F0]          // declared
   type
    cbz x1, .Lgcp_no
    ldr x2, [x1, #AST_TAG]
    cmp x2, #AST_TYPE_ARRAY         // a 1-D array
    b.ne .Lgcp_no
    ldr x1, [x1, #AST_F0]           // element
   type: must be a pointer
    cbz x1, .Lgcp_no
    ldr x2, [x1, #AST_TAG]
    cmp x2, #AST_TYPE_PTR
    b.ne .Lgcp_no
    ldr x2, [x1, #AST_F1]           // the pointer
   's own qualifiers (the *const)
    and x2, x2, #1                  // const bit:
   the element pointers are const,
    cbz x2, .Lgcp_no                // so the
   array is read-only
    ldr x0, [x19, #AST_F2]          // init: flat
   list of string/int-constant leaves
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_INIT_LIST
    b.ne .Lgcp_no
    ldr x20, [x0, #AST_F0]
```

```
.Lgcp_loop:
    cbz x20, .Lgcp_yes
    ldr x0, [x20, #AST_TAG]
    cmp x0, #AST_STR_LIT
    b.eq .Lgcp_elem_ok
    cmp x0, #AST_INT_LIT            // an integer
   constant (NULL) leaves a zero slot
    b.ne .Lgcp_no
.Lgcp_elem_ok:
    ldr x20, [x20, #AST_F4]
    b .Lgcp_loop
.Lgcp_yes:
    mov x0, #1
    b .Lgcp_pdone
.Lgcp_no:
    mov x0, #0
.Lgcp_pdone:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

308 · defined at 307, 308, 312, 314ab, 316 · used in 30b · defines _kcc_global_const_ptrarr 312, 465, 562, _kcc_global_const_rodata 312, 465, 562, 563, _kcc_rodata_aos_ok never used, _kcc_rodata_binop_ok never used, _kcc_rodata_const_intexpr never used, _kcc_rodata_is_lit never used, _kcc_rodata_struct_ok never used · uses _kcc_eval_const_int 426, _kcc_sema_resolve_type 259b, _kcc_symtab_lookup 255, _kcc_type_layout 267, 268

⟨ *IR: build* 312 ⟩ +≡

```
// -- kcc_ir_emit_global_inits ---
// void kcc_ir_emit_global_inits(void)
// Walks the saved translation unit for global
   var_decls with initializers
// and emits IR_GLOBAL_REF + build_expr(init) +
   IR_STORE into the current BB.
.text
.globl _kcc_ir_emit_global_inits
.p2align 2
_kcc_ir_emit_global_inits:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    adrp x0, _kcc_ir_trans_unit@PAGE
    add  x0, x0, _kcc_ir_trans_unit@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Legi_done
    ldr x19, [x0, #AST_F0]       // first
   declaration

.Legi_loop:
    cbz x19, .Legi_statics
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_VAR_DECL
    b.ne .Legi_next

    // Check for initializer (AST_F2)
    ldr x0, [x19, #AST_F2]
    cbz x0, .Legi_next
    // Check for name (AST_F1)
    ldr x20, [x19, #AST_F1]
    cbz x20, .Legi_next

    // A const scalar/aggregate global emitted as
   read-only __TEXT bytes, or a
    // const pointer array emitted to __DATA_CONST
   , has no writable storage; a
    // runtime store would fault.  Suppress it.
    mov x0, x19
    bl _kcc_global_const_rodata
    cbnz x0, .Legi_next
    mov x0, x19
    bl _kcc_global_const_ptrarr
    cbnz x0, .Legi_next

    // Emit IR_GLOBAL_REF for this variable
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    mov x21, x0
    str x20, [x21, #IR_SRC1]     // name
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x21, #IR_TYPE]
    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x22, x0                  // x22 = address
   vreg
    mov x0, x21
    bl _kcc_ir_emit

    // Emit the initializer (scalar, aggregate
   list, or string) against the
    // just-materialised base address.  The same
   routine serves the static
    // locals below, keyed only on the base
   address it is handed.
    mov x0, x19                  // decl node
    mov x1, x22                  // base address
   vreg
    bl _kcc_ir_emit_init_stores

.Legi_next:
    ldr x19, [x19, #AST_F4]      // next sibling
    b .Legi_loop

.Legi_statics:
    // Second pass: one-time initialisers for
   static locals, hoisted globals
    adrp x0, _kcc_ir_trans_unit@PAGE
    add  x0, x0, _kcc_ir_trans_unit@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Legi_done
    ldr x19, [x0, #AST_F0]       // first top-level
    decl
.Legi_fn_loop:
    cbz x19, .Legi_reset
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_FUNC_DECL
    b.ne .Legi_fn_next
    ldr x0, [x19, #AST_F3]       // function body (
   compound stmt)
    cbz x0, .Legi_fn_next
    ldr x23, [x0, #AST_F0]       // body head (
   preserved across the reg sweep)

    // Registration sweep: seed this function's
   static-locals into kcc_ir_statics
    // (and static_types) so an initializer taking
    the address of a sibling
    // static (`static T *p[] = { other_static, ..
    }`) resolves to the hoisted
    // mangled symbol.  emit_modinit reset the
   table, so without this the
    // reference would link against the source
   name and be undefined.
    mov x24, x23                 // cursor
.Legi_reg_loop:
    cbz x24, .Legi_reg_done
```

```
    ldr x0, [x24, #AST_TAG]
    cmp x0, #AST_VAR_DECL
    b.ne .Legi_reg_next
    ldr x0, [x24, #AST_F3]      // storage class
    cmp x0, #3
    b.ne .Legi_reg_next
    ldr x0, [x24, #AST_F1]      // name
    cbz x0, .Legi_reg_next
    // name -> static_name(decl)
    mov x0, x24
    bl _kcc_ir_static_name
    mov x21, x0                 // mangled (callee
-saved, survives strlen)
    ldr x0, [x24, #AST_F1]      // name
    bl _kcc_strlen
    mov x2, x0                  // keylen
    ldr x1, [x24, #AST_F1]      // key = name
    mov x3, x21                 // value = mangled
    adrp x0, _kcc_ir_statics@PAGE
    add  x0, x0, _kcc_ir_statics@PAGEOFF
    ldr x0, [x0]
    bl _kcc_ht_put
    // name -> resolve_type(decl->F0) into
static_types
    ldr x0, [x24, #AST_F0]      // type node
    cbz x0, .Legi_reg_next
    bl _kcc_sema_resolve_type
    cbz x0, .Legi_reg_next
    mov x21, x0                 // rt (survives
strlen)
    ldr x0, [x24, #AST_F1]      // name
    bl _kcc_strlen
    mov x2, x0                  // keylen
    ldr x1, [x24, #AST_F1]      // key = name
    mov x3, x21                 // value = rt
    adrp x0, _kcc_ir_static_types@PAGE
    add  x0, x0, _kcc_ir_static_types@PAGEOFF
    ldr x0, [x0]
    bl _kcc_ht_put
.Legi_reg_next:
    ldr x24, [x24, #AST_F4]
    b .Legi_reg_loop
.Legi_reg_done:
    mov x20, x23                // reset to body
head for the emit loop
.Legi_stmt_loop:
    cbz x20, .Legi_fn_next
    ldr x0, [x20, #AST_TAG]
    cmp x0, #AST_VAR_DECL
    b.ne .Legi_stmt_next
    ldr x0, [x20, #AST_F3]      // storage class
(3 = static)
    cmp x0, #3
    b.ne .Legi_stmt_next
    ldr x0, [x20, #AST_F2]      // initializer
    cbz x0, .Legi_stmt_next

    // Same read-only const rule as the file-scope
    pass: no runtime store.
    mov x0, x20
    bl _kcc_global_const_rodata
    cbnz x0, .Legi_stmt_next
    mov x0, x20
    bl _kcc_global_const_ptrarr
    cbnz x0, .Legi_stmt_next

    // GLOBAL_REF(mangled) + shared initializer
emission.  Because the static
    // shares one routine with file-scope globals,
    an aggregate static
    // (`static const struct P p = {..}` or `
static const int a[] = {..}`) is
    // laid out element-by-element into its
storage, not truncated to a single
    // scalar store.
    mov x0, x20
    bl _kcc_ir_static_name
    mov x21, x0                 // mangled name

    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_SRC1]
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x22, x0
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                 // x21 = address
vreg
    mov x0, x22
    bl _kcc_ir_emit

    mov x0, x20                 // static decl
node
    mov x1, x21                 // base address
vreg
    bl _kcc_ir_emit_init_stores

.Legi_stmt_next:
    ldr x20, [x20, #AST_F4]
    b .Legi_stmt_loop
.Legi_fn_next:
    ldr x19, [x19, #AST_F4]
    b .Legi_fn_loop

.Legi_reset:
    // Discard the static-local names seeded above
    : this runs at the top of main
    // (or a module-init), and leaving other
    functions' statics in the table
    // would let them shadow a like-named global
    in the body that follows.
    mov x0, #16
    bl _kcc_ht_init
    adrp x1, _kcc_ir_statics@PAGE
    add  x1, x1, _kcc_ir_statics@PAGEOFF
    str x0, [x1]
    mov x0, #16
    bl _kcc_ht_init
    adrp x1, _kcc_ir_static_types@PAGE
    add  x1, x1, _kcc_ir_static_types@PAGEOFF
    str x0, [x1]

.Legi_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

312 · defined at 307, 308, 312, 314ab, 316 · used in 30b · defines _kcc_ir_emit_global_inits 316, 317 · uses _kcc_global_const_ptrarr 308, _kcc_global_const_rodata 308, _kcc_ht_init 61c, _kcc_ht_put 63, _kcc_ir_alloc_instr 303a, _kcc_ir_emit 304, _kcc_ir_emit_init_stores 314b, _kcc_ir_next_vreg 305a, _kcc_ir_statics 305c, _kcc_ir_static_name 417a, _kcc_ir_static_types 305c, _kcc_sema_resolve_type 259b, _kcc_type_int_s 250b

Shared initializer emission. Both file-scope globals and hoisted static locals reduce to the same problem: given a base address and a declaration, emit the store instruc-

tions that realise its initialiser. Factoring that into one routine, `_kcc_ir_emit_init_stores`, is what lets a `static const` aggregate be laid out correctly—the earlier static pass truncated `{7, 9}` to a single scalar store, leaving the second field at its BSS zero, so `lcode.c`'s `static const expdesc ef = {VKINT, {0}, NO_JUMP, NO_JUMP}` came back mis-initialised. The routine takes the declaration in `x0` and the base-address vreg in `x1`, dispatches on the shape of the initialiser, and emits the corresponding stores: a scalar (with the `_Bool` 0/1 normalisation), a brace-enclosed list (one `_kcc_ir_store_elem` per element, honouring designators and byte-accurate offsets), or a string literal copied into an array. It writes nothing for the unmentioned tail, since BSS already supplies the zero image.

Nested aggregate initialisers. A brace-enclosed initialiser can nest: an array of structs (`static const luaL_Reg lib[] = {{"a",f},…}`) or a struct with an aggregate member, and it may use designator chains (`.p.a = 1`) or brace elision (`int m[2][2] = {1,2,3,4}`) just as a block-scope initialiser can. Rather than duplicate that logic, `_kcc_ir_emit_aggregate_init` is now a thin adapter onto the shared recursive initializer (§29.11): it hands the list, the aggregate type, and the global's base *address* to `_kcc_ir_fill_list`, which descends the object emitting one `IR_GEP`-plus-store per leaf. The base being a materialised address rather than an alloca vreg makes no difference—`IR_GEP` strides either by a byte offset all the same—so file-scope and block-scope aggregates share a single, tested code path. A flat single-store-per-element scheme (the original behaviour) once placed a function-pointer field's brace initialiser at the wrong slot, so Lua's `loadedlibs` table called into its own data (an `EXC_BAD_ACCESS` during `luaL_openlibs`); routing both scopes through the same recursion keeps that class of bug fixed in one place.

⟨ *IR: build* 314a ⟩ + ≡

```
// -- kcc_ir_emit_aggregate_init ---
// void kcc_ir_emit_aggregate_init(node *init_list
   , type *agg, uint64_t base)
//   x0 = AST_INIT_LIST, x1 = aggregate type (
   array/struct/union, or a scalar
//   type for the untyped int fallback), x2 = base
    ADDRESS vreg.
// Thin adapter onto the shared recursive
   initializer: the file-scope path
// emits its stores against a materialised base
   address exactly as the local
// path does against an alloca, so both go through
    _kcc_ir_fill_list and get
// designator chains and brace elision for free.
   No zero-fill here---a
// file-scope aggregate starts life in zeroed BSS,
    so a sparse list's gaps
// are already zero.
.text
.globl _kcc_ir_emit_aggregate_init
.p2align 2
_kcc_ir_emit_aggregate_init:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    ldr x3, [x0, #AST_FO]          // list head
   element
    mov x0, x1                     // aggregate type
    mov x1, x2                     // base address
   vreg
    mov x2, #0                     // base offset
    bl _kcc_ir_fill_list
    ldp x29, x30, [sp], #16
    ret
```

□

314a · defined at 307, 308, 312, 314ab, 316 · used in 30b · defines `_kcc_ir_emit_aggregate_init` 314b · uses `_kcc_ir_fill_list` 414

⟨ *IR: build* 314b ⟩ + ≡

```
// -- kcc_ir_emit_init_stores ---
// void kcc_ir_emit_init_stores(node *decl,
   uint64_t base_vreg)
//   x0 = AST_VAR_DECL (AST_FO = declared type,
   AST_F2 = initializer)
//   x1 = base address vreg (storage the
   initializer writes into)
// Emits the store(s) for a scalar, aggregate init
   -list, or string init.
.text
.globl _kcc_ir_emit_init_stores
.p2align 2
_kcc_ir_emit_init_stores:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0                    // decl node
    mov x22, x1                    // base address
   vreg

    ldr x0, [x19, #AST_F2]        // initializer
    cbz x0, .Leis_done
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_INIT_LIST
    b.eq .Leis_initlist
    cmp x1, #AST_STR_LIT
    b.eq .Leis_string

.Leis_scalar:
    // An initializer that is neither a brace list
    nor a string literal, but
    // whose target is a struct/union (`expdesc
   temp = *e1;`, `S a = b;`), is a
    // whole-object COPY, not a scalar store---
   otherwise only the first word is
    // copied and the rest of the object is left
   as stack garbage.  (This is a
    // different path from a plain assignment `a =
    b`: a declaration with an
    // initializer.)  It broke Lua's `lcode.c`,
   whose `swapexps` does exactly
    // `expdesc temp = *e1;` to compile the
   `>`/`>=` operators, so every `a > b`
    // crashed the parser.
    ldr x0, [x19, #AST_FO]        // declared type
   spec
    bl _kcc_sema_resolve_type
    cbz x0, .Leis_scalar_val
    mov x23, x0                    // x23 = resolved
    type
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_STRUCT
    b.eq .Leis_struct_copy
    cmp x1, #TY_UNION
    b.eq .Leis_struct_copy
    b .Leis_scalar_val
```

```
.Leis_struct_copy:
    ldr x0, [x19, #AST_F2]       // initializer
   expression
    bl _kcc_ir_build_struct_addr // x0 = source
   struct base address vreg
    mov x24, x0                  // x24 = src
    mov x0, x23
    bl _kcc_type_layout          // x0 = aggregate
    byte size
    mov x2, x0                    // byte size (
   exact)
    mov x0, x24                  // src
    mov x1, x22                  // dst = base
   address
    bl _kcc_ir_emit_struct_copy
    b .Leis_done

.Leis_scalar_val:
    // Build the initializer expression,
   remembering its sema type first so the
    // value can be converted to the declared type
   ---exactly as the block-scope
    // scalar path does.  Without this a global `
   double d = 100;` stored the raw
    // integer bits of 100 instead of the IEEE-754
    double, and a `float f = 5;`
    // stored 8 integer bytes; both read back as
   ~0.
    ldr x0, [x19, #AST_F2]       // init expr node
    ldr x24, [x0, #AST_F3]       // from =
   initializer's sema type (may be 0)
    bl _kcc_ir_build_expr
    mov x21, x0                   // value vreg

    ldr x0, [x19, #AST_F0]       // declared type
   spec
    bl _kcc_sema_resolve_type
    mov x23, x0                  // to = declared
   type (may be 0)

    // Convert across the int/float boundary if
   both types are known.
    cbz x23, .Leis_sv_bool
    cbz x24, .Leis_sv_bool
    mov x0, x21
    mov x1, x24                  // from
    mov x2, x23                  // to
    bl _kcc_ir_convert
    mov x21, x0

.Leis_sv_bool:
    // A _Bool target normalises its initializer
   to 0/1.
    cbz x23, .Leis_scalar_store
    ldr x0, [x23, #TY_KIND]
    cmp x0, #TY_BOOL
    b.ne .Leis_scalar_store
    mov x0, x21
    bl _kcc_ir_emit_bool_norm
    mov x21, x0

.Leis_scalar_store:
    // Compute the access-size from the declared
   type (x24 is free after the
    // conversion) so a narrow scalar---and a `
   float`, whose 4-byte single form
    // differs from the double working format---is
    written at its true width.
    mov x0, x23
    bl _kcc_ir_store_width
    mov x24, x0                  // access-size
   code
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    cbz x23, .Leis_sv_inttype
    str x23, [x0, #IR_TYPE]      // declared type
    b .Leis_sv_havetype
.Leis_sv_inttype:
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
.Leis_sv_havetype:
    str x21, [x0, #IR_SRC1]      // value
    str x22, [x0, #IR_SRC2]      // address
    str x24, [x0, #IR_DEST]      // access-size
   override
    bl _kcc_ir_emit
    b .Leis_done

.Leis_initlist:
    // Lay out the brace initialiser against the
   declared aggregate type,
    // recursing for nested aggregates.  When the
   type does not resolve, fall
    // back to an int-strided array (the
   historical untyped behaviour).
    ldr x0, [x19, #AST_F0]       // declared type
   spec
    bl _kcc_sema_resolve_type
    cbnz x0, .Leis_il_have_type
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr x0, [x0]
.Leis_il_have_type:
    mov x1, x0                    // aggregate type
    ldr x0, [x19, #AST_F2]       // init-list node
    mov x2, x22                   // base address
   vreg
    bl _kcc_ir_emit_aggregate_init
    b .Leis_done

.Leis_string:
    // char s[] = "abc": copy bytes into the array
    storage (a char *g stores
    // the address, so only an array target takes
   this path).
    ldr x0, [x19, #AST_F0]       // declared type
    bl _kcc_sema_resolve_type
    cbz x0, .Leis_scalar
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.ne .Leis_scalar
    ldr x21, [x0, #TY_F1]        // array length (
   char → bytes)
    ldr x0, [x19, #AST_F2]
    ldr x23, [x0, #AST_F0]       // str_ptr
    ldr x24, [x0, #AST_F1]       // str_len
    cbnz x21, .Leis_str_call
    add x21, x24, #1
.Leis_str_call:
    mov x0, x22                   // base = address
    vreg
    mov x1, x23                   // str_ptr
    mov x2, x24                   // str_len
    mov x3, x21                   // array_len
    mov x4, #0                    // BSS already
   zero
    bl _kcc_ir_emit_string_init

.Leis_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
```

```
    ret
```

□

314b · defined at 307, 308, 312, 314ab, 316 · used in 30b · defines _kcc_ir_emit_init_stores 312 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_build_struct_addr 405, _kcc_ir_convert 343, _kcc_ir_emit 304, _kcc_ir_emit_aggregate_init 314a, _kcc_ir_emit_struct_copy 404, _kcc_ir_store_width 349b, _kcc_sema_resolve_type 259b, _kcc_type_int_s 250b, _kcc_type_layout 267, 268

The synthesised module-init function is an ordinary IR function built by hand: it reuses `_kcc_ir_emit_global_inits` for its body, so its stores are byte-for-byte the ones `main` would have run, then closes with an implicit return. It is named `kcc.module_init`—a name no C identifier can spell, so it never collides with a user symbol—and the code generator gives it *local* linkage, so several separately compiled units may each define one without a duplicate-symbol error at link time. Setting `_kcc_modinit_present` tells the code generator and the Mach-O writer to emit the `__mod_init_func` pointer that makes `dyld` run it.

⟨ *IR: build* 316 ⟩ + ≡

```
// -- kcc_ir_emit_modinit ---
// void kcc_ir_emit_modinit(void)
// Synthesises a local void(void) function whose
   body is the file-scope
// initializer stores, so a non-main translation
   unit initialises its
// globals via a (__DATA,__mod_init_func) entry
   run by dyld before main.
.text
.globl _kcc_ir_emit_modinit
.p2align 2
_kcc_ir_emit_modinit:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // Create the IR function (name "kcc.
   module_init", return type int)
    adrp x0, _kcc_str_modinit@PAGE
    add  x0, x0, _kcc_str_modinit@PAGEOFF
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    bl _kcc_ir_alloc_func
    mov x19, x0                    // ir_func

    adrp x1, _kcc_ir_cur_func@PAGE
    add  x1, x1, _kcc_ir_cur_func@PAGEOFF
    str x19, [x1]

    // Fresh per-function tables:
   emit_global_inits' build_expr may consult
    // the locals/statics/label maps even though
   this body defines no locals.
    mov x0, #64
    bl _kcc_ht_init
    adrp x1, _kcc_ir_locals@PAGE
    add  x1, x1, _kcc_ir_locals@PAGEOFF
    str x0, [x1]
    mov x0, #16
    bl _kcc_ht_init
    adrp x1, _kcc_ir_statics@PAGE
    add  x1, x1, _kcc_ir_statics@PAGEOFF
    str x0, [x1]
    mov x0, #16
    bl _kcc_ht_init
    adrp x1, _kcc_ir_static_types@PAGE
    add  x1, x1, _kcc_ir_static_types@PAGEOFF
    str x0, [x1]
    mov x0, #16
    bl _kcc_ht_init
    adrp x1, _kcc_ir_labels@PAGE
    add  x1, x1, _kcc_ir_labels@PAGEOFF
    str x0, [x1]

    // Fresh block-scope shadowing journal for
   this function.
    adrp x0, _kcc_ir_scope_jdepth@PAGE
    add  x0, x0, _kcc_ir_scope_jdepth@PAGEOFF
    str xzr, [x0]
    adrp x0, _kcc_ir_scope_sp@PAGE
    add  x0, x0, _kcc_ir_scope_sp@PAGEOFF
    str xzr, [x0]

    // Entry basic block
    bl _kcc_ir_alloc_bb
    mov x20, x0
    str x20, [x19, #IRF_ENTRY]
    mov x0, x20
    bl _kcc_ir_set_bb

    // Body: exactly the stores main would inject
    bl _kcc_ir_emit_global_inits

    // Implicit return
    mov x0, #IR_RET
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    bl _kcc_ir_emit
    adrp x1, _kcc_ir_cur_bb@PAGE
    add  x1, x1, _kcc_ir_cur_bb@PAGEOFF
    ldr x1, [x1]
    ldr x2, [x1, #BB_FLAGS]
    orr x2, x2, #BB_FLAG_TERMINATED
    str x2, [x1, #BB_FLAGS]

    // Append to the function list so codegen
   lowers it
    adrp x0, _kcc_ir_funcs@PAGE
    add  x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x0, [x0]
    str x19, [sp, #32]
    add x1, sp, #32
    bl _kcc_vec_push

    // Signal the backend to emit the
   __mod_init_func pointer
    adrp x0, _kcc_modinit_present@PAGE
    add  x0, x0, _kcc_modinit_present@PAGEOFF
    mov x1, #1
    str x1, [x0]

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

316 · defined at 307, 308, 312, 314ab, 316 · used in 30b · defines _kcc_ir_emit_modinit 307 · uses _kcc_ht_init 61c, _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_func 303c, _kcc_ir_alloc_instr 303a, _kcc_ir_cur_bb 305c, _kcc_ir_cur_func 305c, _kcc_ir_emit 304, _kcc_ir_emit_global_inits 312, _kcc_ir_funcs 305c, _kcc_ir_labels 305c, _kcc_ir_locals 305c,

_kcc_ir_scope_jdepth 305c, _kcc_ir_scope_sp 305c, _kcc_ir_set_bb 305b, _kcc_ir_statics 305c, _kcc_ir_static_types 305c, _kcc_modinit_present 305c, _kcc_type_int_s 250b, _kcc_vec_push 65a

29.2. Building a Function

For each function, we create an IR function, allocate an entry basic block, build alloca instructions for parameters and local variables, then lower the body.

A function whose return type is a `struct` follows the *hidden result pointer* convention (Section 29.6): the caller allocates storage for the returned value and passes its address as an extra trailing argument. The callee therefore has one more incoming parameter than the source declares. We account for this by bumping the parameter count when `IRF_RETTYPE` is a struct, which reserves an additional incoming vreg; the codegen prologue spills it from register $\mathtt{x}k$ (where k is the declared parameter count), and `build_return` reads it back as the copy destination.

⟨ *IR: build function* 317 ⟩ ≡

```
// -- kcc_ir_build_func ---
// void kcc_ir_build_func(node *func_decl)
//   x0 = AST_FUNC_DECL node (F0=type, F1=name, F2=params, F3=body)
// Builds IR for one function.
.text
.globl _kcc_ir_build_func
.p2align 2
_kcc_ir_build_func:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]

    mov x19, x0                     // x19 = func_decl AST

    // Get return type from the resolved type (F3 from sema = type desc)
    // The function type is in F0 (type AST), resolved type should be
    // obtained from the symbol table entry or from AST_F3 (sema type)
    // For now, use int as default; check if sema stored type
    ldr x20, [x19, #AST_F1]      // x20 = name_ptr
    ldr x21, [x19, #AST_F0]      // x21 = type AST

    // Resolve return type from the declarator's type node. A FuncDecl
    // stores its return type directly in F0 (e.g. a StructType); an
    // AST_TYPE_FUNC wrapper, when present, holds it one level down in F0.
    mov x22, #0                     // default ret type = NULL
    cbz x21, .Libf_default_ret
    ldr x0, [x21, #AST_TAG]
    cmp x0, #AST_TYPE_FUNC
    b.ne .Libf_resolve_ret
    ldr x21, [x21, #AST_F0]      // unwrap func type → return type AST
    cbz x21, .Libf_default_ret

.Libf_resolve_ret:
    mov x0, x21
    bl _kcc_sema_resolve_type
    mov x22, x0
    cbz x22, .Libf_default_ret
    b .Libf_create

.Libf_default_ret:
    // Default: signed int
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr x22, [x0]

.Libf_create:
    // Create IR function
    mov x0, x20                     // name
    mov x1, x22                     // ret type
    bl _kcc_ir_alloc_func
    mov x23, x0                     // x23 = ir_func

    // Record the declaration line for DWARF debug info.
    ldr x0, [x19, #AST_LINE]
    str x0, [x23, #IRF_LINE]

    // Propagate internal linkage: bit 1 of the packed params pointer marks a
    // static function, which the emitter turns into a local Mach-O symbol.
    ldr x0, [x19, #AST_F2]
    and x0, x0, #2
    lsr x0, x0, #1                  // 1 if static, else 0
    str x0, [x23, #IRF_STATIC]

    // Set as current function
    adrp x1, _kcc_ir_cur_func@PAGE
    add  x1, x1, _kcc_ir_cur_func@PAGEOFF
    str x23, [x1]

    // Reset locals hash table
    mov x0, #64
    bl _kcc_ht_init
    adrp x1, _kcc_ir_locals@PAGE
    add  x1, x1, _kcc_ir_locals@PAGEOFF
    str x0, [x1]

    // Reset statics hash table (static locals visible only in this function)
    mov x0, #16
    bl _kcc_ht_init
    adrp x1, _kcc_ir_statics@PAGE
    add  x1, x1, _kcc_ir_statics@PAGEOFF
    str x0, [x1]

    // Parallel table: static local name → its declared type, so the subscript
    // and identifier-decay paths can tell an array from a pointer once the
    // symbol table's block scope has been popped.
    mov x0, #16
    bl _kcc_ht_init
    adrp x1, _kcc_ir_static_types@PAGE
    add  x1, x1, _kcc_ir_static_types@PAGEOFF
    str x0, [x1]

    // Reset label table (label names visible only in this function)
    mov x0, #16
    bl _kcc_ht_init
    adrp x1, _kcc_ir_labels@PAGE
    add  x1, x1, _kcc_ir_labels@PAGEOFF
    str x0, [x1]
```

```
    // Create entry basic block
    bl _kcc_ir_alloc_bb
    mov x24, x0                     // x24 = entry BB
    str x24, [x23, #IRF_ENTRY]
    mov x0, x24
    bl _kcc_ir_set_bb

    // Count parameters and set IRF_PARAMS (mask
   the variadic tag in bit 0 and
    // the static tag in bit 1 before
   dereferencing the param pointer)
    ldr x25, [x19, #AST_F2]      // x25 = first
   param decl (bit 0=variadic, bit 1=static)
    bic x25, x25, #3
    mov x20, #0                     // param count
.Libf_count_params:
    cbz x25, .Libf_count_done
    ldr x0, [x25, #AST_TAG]
    cmp x0, #AST_PARAM_DECL
    b.ne .Libf_count_next
    ldr x0, [x25, #AST_F1]          // an unnamed
   param is the `(void)` marker
    cbz x0, .Libf_count_next
    add x20, x20, #1
.Libf_count_next:
    ldr x25, [x25, #AST_F4]
    b .Libf_count_params
.Libf_count_done:
    // AAPCS64 struct return: a struct larger than
    16 bytes (class REF) is
    // returned through a hidden result pointer
   the caller supplies in x8, so
    // reserve one extra incoming vreg for it.  A
   struct of 16 bytes or less
    // (ST1/ST2) is returned in x0(-x1) and needs
   no hidden parameter.
    ldr x0, [x23, #IRF_RETTYPE]
    bl _kcc_ir_classify_type
    cmp x0, #4                      // REF (>16B
   aggregate)
    b.ne .Libf_params_set
    add x20, x20, #1
.Libf_params_set:
    str x20, [x23, #IRF_PARAMS]  // set param
   count (incl. hidden sret)
    // Record whether the function is variadic (
   bit 0 of the param-list pointer);
    // codegen needs it to place a register-
   overflow fixed parameter at the
    // stack offset the variadic caller used.
    ldr x0, [x19, #AST_F2]
    and x0, x0, #1
    str x0, [x23, #IRF_VARIADIC]

    // Record each incoming parameter's AAPCS64
   class for the prologue.
    mov x0, x23                     // ir_func
    mov x1, x19                     // func_decl
    bl _kcc_ir_fill_param_classes

    // Reserve incoming-parameter vregs.  A 9..16-
   byte struct parameter
    // (class ST2) arrives in two registers and
   occupies two vregs; every
    // other parameter occupies one.  Sum the
   widths, then reserve that many.
    mov x24, #0                     // total vreg
   width
    mov x25, #0                     // param index
.Libf_reserve_count:
    cmp x25, x20
    b.ge .Libf_reserve_do
    ldr x0, [x23, #IRF_PARAM_CLS]
    cbz x0, .Libf_rc_one
    ldr x0, [x0, x25, lsl #3]      // class
    cmp x0, #3                      // ST2 → width 2
    b.eq .Libf_rc_two
.Libf_rc_one:
    add x24, x24, #1
    b .Libf_rc_next
.Libf_rc_two:
    add x24, x24, #2
.Libf_rc_next:
    add x25, x25, #1
    b .Libf_reserve_count
.Libf_reserve_do:
    mov x25, #0
.Libf_reserve_vregs:
    cmp x25, x24
    b.ge .Libf_build_params
    bl _kcc_ir_next_vreg
    add x25, x25, #1
    b .Libf_reserve_vregs
.Libf_build_params:

    // Build alloca + store for each parameter,
   passing its vreg BASE (which
    // accounts for two-vreg ST2 parameters), not
   its ordinal index.
    ldr x25, [x19, #AST_F2]      // first param
   decl (bit 0=variadic, bit 1=static)
    bic x25, x25, #3
    mov x20, #0                     // param index
    mov x24, #0                     // vreg base (
   cumulative)

.Libf_param_loop:
    cbz x25, .Libf_body

    ldr x0, [x25, #AST_TAG]
    cmp x0, #AST_PARAM_DECL
    b.ne .Libf_param_next
    ldr x0, [x25, #AST_F1]          // skip the
   unnamed `(void)` marker
    cbz x0, .Libf_param_next

    mov x0, x25
    mov x1, x24                     // vreg base for
    this parameter
    bl _kcc_ir_build_param
    ldr x1, [x23, #IRF_PARAM_CLS]
    ldr x1, [x1, x20, lsl #3]      // class
    cmp x1, #3
    b.ne .Libf_pl_one
    add x24, x24, #2                // ST2 consumed
   two vregs
    b .Libf_pl_adv
.Libf_pl_one:
    add x24, x24, #1
.Libf_pl_adv:
    add x20, x20, #1

.Libf_param_next:
    ldr x25, [x25, #AST_F4]
    b .Libf_param_loop

.Libf_body:
    // If this is main(), emit global variable
   initializers first
    ldr x0, [x19, #AST_F1]          // function name
    adrp x1, _kcc_str_main@PAGE
    add  x1, x1, _kcc_str_main@PAGEOFF
    bl _kcc_strcmp
    cbnz x0, .Libf_body_stmts
    bl _kcc_ir_emit_global_inits

.Libf_body_stmts:
    // Pre-allocate a BB for every label so
```

```
    forward gotos resolve
    ldr x0, [x19, #AST_F3]
    bl _kcc_ir_prescan_labels

    // Lower the function body (compound statement
    )
    ldr x0, [x19, #AST_F3]
    bl _kcc_ir_build_stmt

    // Ensure the last BB has a terminator (
    implicit return)
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Libf_register

    ldr x1, [x0, #BB_FLAGS]
    tst x1, #BB_FLAG_TERMINATED
    b.ne .Libf_register

    // Emit implicit return.  Reaching the closing
    brace of main returns 0
    // (\stdref{5.1.2.2.3}); every other non-void
    function that falls off the
    // end is undefined behaviour, so it returns
    no value (unchanged).
    ldr x0, [x23, #IRF_NAME]
    cbz x0, .Libf_impret_void
    adrp x1, .Libf_main_name@PAGE
    add  x1, x1, .Libf_main_name@PAGEOFF
    bl _kcc_strcmp
    cbnz x0, .Libf_impret_void
    mov x0, #0
    bl _kcc_ir_emit_const        // vreg holding
    the constant 0
    str x0, [sp, #72]            // stash across
    alloc_instr
    mov x0, #IR_RET
    bl _kcc_ir_alloc_instr
    str x22, [x0, #IR_TYPE]
    ldr x1, [sp, #72]
    str x1, [x0, #IR_SRC1]       // main's
    implicit return value = 0
    bl _kcc_ir_emit
    b .Libf_impret_done
.Libf_impret_void:
    // A function that falls off its end returns
    no value.  For a void function
    // that is correct; for a value-returning one
    it is undefined behaviour the
    // caller must not rely on --- but the
    implicit return must still carry NO
    // type, not the declared one.  Tagging it
    with a struct/union return type
    // (while leaving SRC1 = 0) made the backend
    marshal a return value that was
    // never produced: it read vreg 0 (which
    happens to be the first parameter)
    // as the aggregate's address and faulted.  A
    null IR_TYPE makes the backend
    // emit a bare RET, returning whatever is in
    the result registers.
    mov x0, #IR_RET
    bl _kcc_ir_alloc_instr
    str xzr, [x0, #IR_TYPE]      // no value: a
    bare return
    bl _kcc_ir_emit
.Libf_impret_done:

    // Mark BB terminated
    adrp x1, _kcc_ir_cur_bb@PAGE
    add  x1, x1, _kcc_ir_cur_bb@PAGEOFF
    ldr x1, [x1]
    ldr x2, [x1, #BB_FLAGS]
    orr x2, x2, #BB_FLAG_TERMINATED
    str x2, [x1, #BB_FLAGS]

.Libf_register:
    // Add function to global list
    adrp x0, _kcc_ir_funcs@PAGE
    add  x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x0, [x0]
    str x23, [sp, #72]           // store func ptr
    as element
    add x1, sp, #72
    bl _kcc_vec_push

    ldr x25, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret

.section __TEXT,__cstring,cstring_literals
.Libf_main_name:
    .asciz "main"
.text
```

□

317 · used in 30b · defines _kcc_ir_build_func 307 · uses _kcc_ht_init 61c, _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_func 303c, _kcc_ir_alloc_instr 303a, _kcc_ir_build_param 320, _kcc_ir_build_stmt 322, _kcc_ir_classify_type 343, _kcc_ir_cur_bb 305c, _kcc_ir_cur_func 305c, _kcc_ir_emit 304, _kcc_ir_emit_global_inits 312, _kcc_ir_fill_param_classes 343, _kcc_ir_funcs 305c, _kcc_ir_labels 305c, _kcc_ir_locals 305c, _kcc_ir_next_vreg 305a, _kcc_ir_prescan_labels 321, _kcc_ir_set_bb 305b, _kcc_ir_statics 305c, _kcc_ir_static_types 305c, _kcc_sema_resolve_type 259b, _kcc_type_int_s 250b, _kcc_vec_push 65a

29.3. Building Parameters

Each function parameter gets an ALLOCA for its stack storage and a STORE that saves the incoming register value. The codegen prologue copies x0–x7 into vreg slots 0–7, so the parameter's "incoming vreg" is simply its index. The alloca vreg is recorded in the `locals` hash table so that later identifier references find the right address.

Passing a struct by value. A `struct` does not fit a single register, so it is passed the same way it is returned (§29.6): the caller places the *address* of the object in the argument register— `build_call` uses `_kcc_ir_build_struct_addr` instead of loading a value—and the callee makes its own copy. Here in `build_param`, a struct parameter therefore allocates its full N-slot storage and, rather than storing the 8-byte register, copies N slots from the incoming pointer into that storage with `_kcc_ir_emit_struct_copy`. The copy is what gives C's by-value semantics: mutations to the parameter touch the callee's copy, never the caller's object. Because each struct argument still consumes exactly one register (a pointer), this composes cleanly with the hidden result pointer of struct return and with ordinary scalar arguments. Passing more than eight registers' worth of arguments on the stack remains a separate, still-open ABI item.

⟨ IR: build param 320 ⟩ ≡

```
// -- kcc_ir_build_param ---
// void kcc_ir_build_param(node *param_decl,
   uint64_t param_index)
//   x0 = AST_PARAM_DECL (F0=type, F1=name)
//   x1 = param index (0-based, corresponds to
   reserved vreg)
// Emits alloca + store from the incoming param
   vreg, records in locals.
// The codegen prologue stores x<i> into vreg slot
    i. We emit:
//   %N = alloca type
//   store %i, ptr %N        (i = param_index =
   incoming value vreg)
.text
.globl _kcc_ir_build_param
.p2align 2
_kcc_ir_build_param:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    mov x19, x0                    // param_decl
    mov x22, x1                    // x22 = param
   index (= incoming vreg)

    // Resolve type.  An array/function parameter
   decays to a pointer
    // (\stdref{6.7.6.3}): the callee receives the
    passed address, so the
    // storage is a single pointer slot, not
   inline array space.
    ldr x0, [x19, #AST_F0]
    cbz x0, .Libp_done
    bl _kcc_sema_resolve_type
    bl _kcc_type_param_adjust
    mov x20, x0                    // x20 = type
   desc
    cbz x20, .Libp_done

    // Get name
    ldr x21, [x19, #AST_F1]      // x21 = name ptr
    cbz x21, .Libp_done

    // Slot count: a struct parameter occupies N
   eight-byte slots and is
    // received as a pointer to the caller's
   object; a scalar occupies one.
    ldr x0, [x20, #TY_KIND]
    cmp x0, #TY_STRUCT
    b.ne .Libp_scalar_slots
    mov x0, x20
    bl _kcc_ir_type_slots        // x0 = N
    str x0, [sp, #56]
    b .Libp_alloca
.Libp_scalar_slots:
    mov x0, #1
    str x0, [sp, #56]

.Libp_alloca:
    // Emit alloca for param storage
    mov x0, #IR_ALLOCA
    bl _kcc_ir_alloc_instr
    str x20, [x0, #IR_TYPE]
    ldr x1, [sp, #56]
    str x1, [x0, #IR_SRC1]       // slot count (N
   for struct, 1 for scalar)
    mov x19, x0                    // save instr
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x23, x0                    // x23 = alloca
   vreg number
    mov x0, x19
    bl _kcc_ir_emit

    // Initialise parameter storage from the
   incoming register(s).
    ldr x0, [x20, #TY_KIND]
    cmp x0, #TY_STRUCT
    b.eq .Libp_agg
    cmp x0, #TY_UNION
    b.eq .Libp_agg
    b .Libp_scalar_store

.Libp_agg:
    // <=8B (ST1): arrives as a value in one
   register → store like a scalar.
    // 9..16B (ST2): arrives in two registers (
   vregs base, base+1) → store both
    //   words into the alloca.
    // >16B (REF): arrives as a pointer → copy
   slot by slot.
    mov x0, x20
    bl _kcc_type_layout
    cmp x0, #8
    b.le .Libp_scalar_store
    cmp x0, #16
    b.gt .Libp_struct_copy

    // ST2: reconstruct the 16-byte aggregate from
    the two incoming vregs.
    mov x0, x23                    // base = alloca
   vreg
    mov x1, #0                     // byte offset 0
    mov x2, x22                    // value =
   incoming word 0 (vreg base)
    mov x3, #0
    bl _kcc_ir_emit_indexed_store
    mov x0, x23
    mov x1, #8                     // byte offset 8
    add x2, x22, #1                // value =
   incoming word 1 (vreg base + 1)
    mov x3, #0
    bl _kcc_ir_emit_indexed_store
    b .Libp_record

.Libp_scalar_store:
    // Scalar / ST1: store the incoming value into
    the alloca.
    // IR_STORE: SRC1 = value vreg (param index),
   SRC2 = pointer vreg (alloca)
    // The access-size code (store_width of the
   parameter type) must be set so a
    // `float` parameter is written to its stack
   home as a 4-byte single, exactly
    // as every read of it loads a single---
   otherwise the copy stored 8 double
    // bytes while each use read 4 single bytes,
   and the value came back as ~0.
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    str x0, [sp, #56]              // stash store
   instr
    str x22, [x0, #IR_SRC1]        // value =
   incoming param vreg
    str x23, [x0, #IR_SRC2]        // ptr = alloca
   vreg
    str x20, [x0, #IR_TYPE]
    mov x0, x20                    // parameter type
   → access-size code
    bl _kcc_ir_store_width
    ldr x1, [sp, #56]
    str x0, [x1, #IR_DEST]
    ldr x0, [sp, #56]
```

```
    bl _kcc_ir_emit
    b .Libp_record

.Libp_struct_copy:
    // The incoming register holds a pointer to
   the caller's struct;
    // copy exactly sizeof(param) bytes into this
   parameter's own storage
    // (pass by value).  x20 still holds the
   parameter's type descriptor.
    mov x0, x20
    bl _kcc_type_layout
    mov x2, x0                      // byte size (
   exact)
    mov x0, x22                     // src base =
   incoming pointer vreg
    mov x1, x23                     // dst base =
   local param storage
    bl _kcc_ir_emit_struct_copy

.Libp_record:
    // Record in locals: name → alloca vreg
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x4, x0                      // save ht
    mov x0, x21
    bl _kcc_strlen
    mov x2, x0                      // keylen
    mov x0, x4                      // ht
    mov x1, x21                     // key
    add x3, x23, #1                 // value = alloca
    vreg + 1
    ldr x4, [x20, #TY_KIND]
    cmp x4, #TY_PTR
    b.ne .Libp_put
    orr x3, x3, #0x8000000000000000
.Libp_put:
    bl _kcc_ht_put

.Libp_done:
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

320 · used in 30b · defines _kcc_ir_build_param 317 · uses _kcc_ht_put 63, _kcc_ir_alloc_instr 303a, _kcc_ir_emit 304, _kcc_ir_emit_indexed_store 414, _kcc_ir_emit_struct_copy 404, _kcc_ir_locals 305c, _kcc_ir_next_vreg 305a, _kcc_ir_store_width 349b, _kcc_ir_type_slots 349a, _kcc_sema_resolve_type 259b, _kcc_type_layout 267, 268, _kcc_type_param_adjust 242b

29.4. Labels and Goto

goto may jump *forward* to a label not yet seen, so the target basic block must exist before any goto is lowered. Before building a function body, _kcc_ir_prescan_labels walks it recursively—through compounds, both arms of an if, loop and switch bodies, and nested labels—allocating a basic block for every AST_LABEL_STMT and recording name → block in a per-function table. Because a block joins the function's ordered block list only when set_bb makes it current, these pre-allocated blocks stay out of the emission order until their label is actually reached, so code order is preserved.

Lowering then uses two small helpers. _kcc_ir_branch_to ends the current block with an IR_BR to a target block and marks it terminated—unless it already has a terminator, in which case it is a no-op. _kcc_ir_label_bb resolves a name to its pre-allocated block. A *label* branches the current block into its own block (a fall-through) and continues there; a *goto* branches to the target and opens a fresh block for any unreachable statements that follow.

⟨ *IR: build goto* 321 ⟩ ≡

```
// -- kcc_ir_branch_to ---
// void kcc_ir_branch_to(bb *target)
//   Ends the current BB with IR_BR to target (and
    marks it terminated),
//   unless the current BB already has a
   terminator.
.text
.globl _kcc_ir_branch_to
.p2align 2
_kcc_ir_branch_to:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                     // target BB

    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lbrt_done
    ldr x1, [x0, #BB_FLAGS]
    tst x1, #BB_FLAG_TERMINATED
    b.ne .Lbrt_done

    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x19, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit

    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]
.Lbrt_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_label_bb ---
// bb *kcc_ir_label_bb(const char *name)   → pre-
   allocated BB, or 0
.text
.globl _kcc_ir_label_bb
.p2align 2
_kcc_ir_label_bb:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0
    bl _kcc_strlen
    mov x2, x0                      // keylen
    adrp x0, _kcc_ir_labels@PAGE
    add  x0, x0, _kcc_ir_labels@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Llbb_none
    mov x1, x19
    bl _kcc_ht_get
    b .Llbb_done
```

```
.Llbb_none:
    mov x0, #0
.Llbb_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_prescan_labels ---
// void kcc_ir_prescan_labels(node *stmt)
//   Recursively allocate a BB for every label and
    register name -> BB.
.text
.globl _kcc_ir_prescan_labels
.p2align 2
_kcc_ir_prescan_labels:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0
    cbz x19, .Lps_done
    ldr x20, [x19, #AST_TAG]

    cmp x20, #AST_COMPOUND
    b.eq .Lps_compound
    cmp x20, #AST_LABEL_STMT
    b.eq .Lps_label
    cmp x20, #AST_IF_STMT
    b.eq .Lps_if
    cmp x20, #AST_WHILE_STMT
    b.eq .Lps_body_f1
    cmp x20, #AST_SWITCH_STMT
    b.eq .Lps_body_f1
    cmp x20, #AST_CASE_STMT
    b.eq .Lps_body_f1
    cmp x20, #AST_DO_STMT
    b.eq .Lps_body_f0
    cmp x20, #AST_DEFAULT_STMT
    b.eq .Lps_body_f0
    cmp x20, #AST_FOR_STMT
    b.eq .Lps_body_f3
    b .Lps_done

.Lps_compound:
    ldr x19, [x19, #AST_F0]
.Lps_comp_loop:
    cbz x19, .Lps_done
    mov x0, x19
    bl _kcc_ir_prescan_labels
    ldr x19, [x19, #AST_F4]
    b .Lps_comp_loop

.Lps_label:
    ldr x21, [x19, #AST_F0]       // label name
    bl _kcc_ir_alloc_bb
    mov x20, x0                   // new BB
    mov x0, x21
    bl _kcc_strlen
    mov x2, x0                    // keylen
    adrp x0, _kcc_ir_labels@PAGE
    add  x0, x0, _kcc_ir_labels@PAGEOFF
    ldr x0, [x0]
    mov x1, x21
    mov x3, x20
    bl _kcc_ht_put
    ldr x0, [x19, #AST_F1]        // labelled
   statement
    bl _kcc_ir_prescan_labels
    b .Lps_done

.Lps_if:
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_prescan_labels
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_prescan_labels
    b .Lps_done
.Lps_body_f0:
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_prescan_labels
    b .Lps_done
.Lps_body_f1:
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_prescan_labels
    b .Lps_done
.Lps_body_f3:
    ldr x0, [x19, #AST_F3]
    bl _kcc_ir_prescan_labels
.Lps_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

321 · used in 30b · defines _kcc_ir_branch_to 322, 342, 426, _kcc_ir_label_bb 322, 342, _kcc_ir_prescan_labels 317, 342 · uses _kcc_ht_get 62, _kcc_ht_put 63, _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_instr 303a, _kcc_ir_cur_bb 305c, _kcc_ir_emit 304, _kcc_ir_labels 305c

29.5. Building Statements

The statement dispatcher matches the AST tag and calls the appropriate lowering function. Expression statements call `ir_build_expr` and discard the result vreg. Compound statements recurse over their children. All control-flow statements (if, while, do, for) and break/continue have dedicated builders in §29.12; labels and `goto` use the helpers of §29.4.

Block scope. A compound statement opens a name scope. The locals map, though, is one flat name → vreg hash table for the whole function, so a nested `{ int v; }` that redeclares an outer `v` would overwrite the outer binding and leave it overwritten after the block—the outer read would then see the inner value. Rather than give the table a scope chain, we *journal* it: `kcc_ir_locals_put` records, before each write, the name's previous value; `kcc_ir_scope_enter` remembers the journal's depth on block entry, and `kcc_ir_scope_leave` replays every entry logged since, restoring the enclosing bindings exactly. Because the compound handler brackets its body between an enter and a leave, arbitrary nesting and shadowing come out right with no per-scope allocation. The same mechanism gives a block-scope `extern` its meaning: it binds the name to the "resolve as a global" marker (value 0) for the block, shadowing any enclosing local so a reference lowers to a `GLOBAL_REF`, then the outer binding returns on exit. The journal lives in `.bss` and is reset per function.

⟨ *IR: build statement* 322 ⟩ ≡

```
// -- kcc_ir_locals_put ---
// void kcc_ir_locals_put(char *name, size_t
   keylen, uint64_t value)
//   Records name → value in the locals map, first
    journalling the name's
//   PREVIOUS value so the enclosing block scope
   can be restored on exit.  A
//   value of 0 means "resolve as a global" (a
   block-scope `extern`), which
```

```
//   shadows any enclosing local of the same name
   for the block's duration.
.text
.globl _kcc_ir_locals_put
.p2align 2
_kcc_ir_locals_put:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // name
    mov x20, x1                     // keylen
    mov x21, x2                     // value

    // old = ht_get(locals, name, keylen)  --- 0
   if the name is unbound
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x19
    mov x2, x20
    bl _kcc_ht_get
    mov x22, x0                     // previous
   value

    // journal[jdepth] = {name, keylen, old};
   jdepth++
    adrp x0, _kcc_ir_scope_jdepth@PAGE
    add  x0, x0, _kcc_ir_scope_jdepth@PAGEOFF
    ldr x1, [x0]
    adrp x2, _kcc_ir_scope_journal@PAGE
    add  x2, x2, _kcc_ir_scope_journal@PAGEOFF
    mov x3, #24
    madd x2, x1, x3, x2             // entry =
   journal + jdepth*24
    str x19, [x2]
    str x20, [x2, #8]
    str x22, [x2, #16]
    add x1, x1, #1
    str x1, [x0]

    // ht_put(locals, name, keylen, value)
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x19
    mov x2, x20
    mov x3, x21
    bl _kcc_ht_put

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_ir_scope_enter ---
// void kcc_ir_scope_enter(void)
//   Marks the start of a block scope by pushing
   the journal's current depth.
.globl _kcc_ir_scope_enter
.p2align 2
_kcc_ir_scope_enter:
    adrp x0, _kcc_ir_scope_jdepth@PAGE
    add  x0, x0, _kcc_ir_scope_jdepth@PAGEOFF
    ldr x1, [x0]                    // current
   jdepth
    adrp x2, _kcc_ir_scope_sp@PAGE
    add  x2, x2, _kcc_ir_scope_sp@PAGEOFF
    ldr x3, [x2]                    // scope sp
    adrp x4, _kcc_ir_scope_stack@PAGE
    add  x4, x4, _kcc_ir_scope_stack@PAGEOFF
    str x1, [x4, x3, lsl #3]
    add x3, x3, #1
    str x3, [x2]
    ret

// -- kcc_ir_scope_leave ---
// void kcc_ir_scope_leave(void)
//   Ends the current block scope: replays every
   journal entry recorded since
//   the matching enter, restoring each name's
   previous locals-map binding.
.globl _kcc_ir_scope_leave
.p2align 2
_kcc_ir_scope_leave:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    adrp x0, _kcc_ir_scope_sp@PAGE
    add  x0, x0, _kcc_ir_scope_sp@PAGEOFF
    ldr x1, [x0]
    sub x1, x1, #1
    str x1, [x0]                    // pop scope
   stack
    adrp x2, _kcc_ir_scope_stack@PAGE
    add  x2, x2, _kcc_ir_scope_stack@PAGEOFF
    ldr x19, [x2, x1, lsl #3]       // x19 = saved
   jdepth (scope floor)

    adrp x20, _kcc_ir_scope_jdepth@PAGE
    add  x20, x20, _kcc_ir_scope_jdepth@PAGEOFF
.Lscl_loop:
    ldr x1, [x20]                   // jdepth
    cmp x1, x19
    b.ls .Lscl_done
    sub x1, x1, #1
    str x1, [x20]                   // pop journal
   entry
    adrp x2, _kcc_ir_scope_journal@PAGE
    add  x2, x2, _kcc_ir_scope_journal@PAGEOFF
    mov x3, #24
    madd x21, x1, x3, x2            // entry =
   journal + jdepth*24
    // ht_put(locals, key, keylen, old_value) ---
   restores the enclosing binding
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x21]                   // key
    ldr x2, [x21, #8]               // keylen
    ldr x3, [x21, #16]              // previous
   value
    bl _kcc_ht_put
    b .Lscl_loop
.Lscl_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_build_stmt ---
// void kcc_ir_build_stmt(node *stmt)
//   x0 = AST statement node
// Dispatches to the appropriate lowering based on
    AST_TAG.
.globl _kcc_ir_build_stmt
.p2align 2
_kcc_ir_build_stmt:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    cbz x0, .Libs_done
    mov x19, x0

    ldr x0, [x19, #AST_TAG]
```

```
    cmp x0, #AST_COMPOUND
    b.eq .Libs_compound
    cmp x0, #AST_EXPR_STMT
    b.eq .Libs_expr
    cmp x0, #AST_RETURN_STMT
    b.eq .Libs_return
    cmp x0, #AST_IF_STMT
    b.eq .Libs_if
    cmp x0, #AST_WHILE_STMT
    b.eq .Libs_while
    cmp x0, #AST_DO_STMT
    b.eq .Libs_do
    cmp x0, #AST_FOR_STMT
    b.eq .Libs_for
    cmp x0, #AST_VAR_DECL
    b.eq .Libs_var_decl
    cmp x0, #AST_BREAK_STMT
    b.eq .Libs_break
    cmp x0, #AST_CONTINUE_STMT
    b.eq .Libs_continue
    cmp x0, #AST_SWITCH_STMT
    b.eq .Libs_switch
    cmp x0, #AST_LABEL_STMT
    b.eq .Libs_label
    cmp x0, #AST_GOTO_STMT
    b.eq .Libs_goto
    cmp x0, #AST_CASE_STMT
    b.eq .Libs_case
    cmp x0, #AST_DEFAULT_STMT
    b.eq .Libs_default
    cmp x0, #AST_NULL_STMT
    b.eq .Libs_done
    cmp x0, #AST_ASM_STMT
    b.eq .Libs_asm

    // Unhandled statement type - skip
    b .Libs_done

.Libs_asm:
    // Inline assembly.  For extended asm, lower
   each operand's expression first
    // (inputs → a value vreg, outputs → the
   lvalue's address vreg), recording
    // vreg+1 in the operand node (0 = none).
   Then emit IR_ASM carrying the
    // template (SRC1/SRC2) and a pointer to the
   AST node (IR_TYPE) so the back
    // end can walk the operands.  Basic asm has
   an empty operand chain.
    mov x0, x19
    bl _kcc_ir_asm_lower_operands
    mov x0, #IR_ASM
    bl _kcc_ir_alloc_instr
    ldr x1, [x19, #AST_F0]      // template ptr
    str x1, [x0, #IR_SRC1]
    ldr x1, [x19, #AST_F1]      // template len
    str x1, [x0, #IR_SRC2]
    str x19, [x0, #IR_TYPE]     // AST_ASM_STMT (
   operands live here)
    bl _kcc_ir_emit
    b .Libs_done

.Libs_label:
    // Fall the current block into the label's
   block, then build the body
    ldr x0, [x19, #AST_F0]      // label name
    bl _kcc_ir_label_bb
    cbz x0, .Libs_label_inner
    mov x20, x0                  // x20 = label BB
    bl _kcc_ir_branch_to
    mov x0, x20
    bl _kcc_ir_set_bb
.Libs_label_inner:
    ldr x0, [x19, #AST_F1]      // labelled
   statement
    bl _kcc_ir_build_stmt
    b .Libs_done

.Libs_goto:
    ldr x0, [x19, #AST_F0]      // target label
   name
    bl _kcc_ir_label_bb
    cbz x0, .Libs_done
    bl _kcc_ir_branch_to        // branch to
   target, terminate current block
    bl _kcc_ir_alloc_bb         // fresh block for
    unreachable trailing code
    bl _kcc_ir_set_bb
    b .Libs_done

.Libs_case:
    // A `case` label placed by the enclosing
   switch's scan (its BB is stashed
    // in AST_F2).  Like a goto label, fall the
   current block into the case
    // block and continue there --- giving C fall-
   through --- then build the
    // labelled statement.  Labels can nest
   anywhere in the body, so this path
    // in build_stmt is what makes Duff's device
   work.
    ldr x20, [x19, #AST_F2]     // case BB (0 if
   not inside a switch)
    cbz x20, .Libs_case_body
    mov x0, x20
    bl _kcc_ir_branch_to
    mov x0, x20
    bl _kcc_ir_set_bb
.Libs_case_body:
    ldr x0, [x19, #AST_F1]      // labelled
   statement
    bl _kcc_ir_build_stmt
    b .Libs_done

.Libs_default:
    ldr x20, [x19, #AST_F1]     // default BB (
   stashed in F1)
    cbz x20, .Libs_default_body
    mov x0, x20
    bl _kcc_ir_branch_to
    mov x0, x20
    bl _kcc_ir_set_bb
.Libs_default_body:
    ldr x0, [x19, #AST_F0]      // labelled
   statement
    bl _kcc_ir_build_stmt
    b .Libs_done

.Libs_compound:
    // A block introduces a name scope: names it
   declares (or `extern`-shadows)
    // are undone on exit so an enclosing local of
    the same name is restored.
    bl _kcc_ir_scope_enter
    // Walk all items in compound statement
    ldr x19, [x19, #AST_F0]     // first item
.Libs_compound_loop:
    cbz x19, .Libs_compound_end
    mov x0, x19
    bl _kcc_ir_build_stmt
    ldr x19, [x19, #AST_F4]
    b .Libs_compound_loop
.Libs_compound_end:
    bl _kcc_ir_scope_leave
    b .Libs_done

.Libs_expr:
    ldr x0, [x19, #AST_F0]      // expression
```

```
    bl _kcc_ir_build_expr
    b .Libs_done

.Libs_return:
    mov x0, x19
    bl _kcc_ir_build_return
    b .Libs_done

.Libs_if:
    mov x0, x19
    bl _kcc_ir_build_if
    b .Libs_done

.Libs_while:
    mov x0, x19
    bl _kcc_ir_build_while
    b .Libs_done

.Libs_do:
    mov x0, x19
    bl _kcc_ir_build_do
    b .Libs_done

.Libs_for:
    mov x0, x19
    bl _kcc_ir_build_for
    b .Libs_done

.Libs_var_decl:
    mov x0, x19
    bl _kcc_ir_build_var_decl
    b .Libs_done

.Libs_break:
    mov x0, x19
    bl _kcc_ir_build_break
    b .Libs_done

.Libs_continue:
    mov x0, x19
    bl _kcc_ir_build_continue
    b .Libs_done

.Libs_switch:
    mov x0, x19
    bl _kcc_ir_build_switch
    b .Libs_done

.Libs_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

322 · used in 30b · defines _kcc_ir_build_stmt 317, 342, 421c, 423ab, 424, 426 · uses _kcc_ht_get 62, _kcc_ht_put 63, _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_instr 303a, _kcc_ir_asm_lower_operands 325, _kcc_ir_branch_to 321, _kcc_ir_build_break 435a, _kcc_ir_build_continue 435b, _kcc_ir_build_do 423b, _kcc_ir_build_expr 340, _kcc_ir_build_for 424, _kcc_ir_build_if 421c, _kcc_ir_build_return 326, _kcc_ir_build_switch 426, _kcc_ir_build_var_decl 417b, _kcc_ir_build_while 423a, _kcc_ir_emit 304, _kcc_ir_label_bb 321, _kcc_ir_locals 305c, _kcc_ir_scope_jdepth 305c, _kcc_ir_scope_journal 305c, _kcc_ir_scope_sp 305c, _kcc_ir_scope_stack 305c, _kcc_ir_set_bb 305b

Lowering extended-asm operands. Each operand's constraint begins with = (output), + (read-write), or neither (input). An input needs a *value* vreg, obtained by building its expression; an output needs the lvalue's *address* vreg, obtained by building a synthetic `&lvalue` (which reuses the ordinary address-of lowering). The two vregs are stored as vreg+1 on the operand node (`AST_F2` = value, `AST_F3` = address; 0 means absent, so the valid vreg 0 is not mistaken for "none").

⟨ *IR: build asm operands* 325 ⟩ ≡

```
// -- kcc_ir_asm_addr_of ---
// uint64_t kcc_ir_asm_addr_of(node *lvalue)
// Build the address of an lvalue by lowering a
   synthetic `&lvalue`.
.globl _kcc_ir_asm_addr_of
.p2align 2
_kcc_ir_asm_addr_of:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                     // lvalue
   expression
    mov x0, #AST_UNARY_EXPR
    bl _kcc_ast_alloc
    mov x1, #TOK_AMP
    str x1, [x0, #AST_F0]           // op = &
    str x19, [x0, #AST_F1]          // operand
    mov x1, #1
    str x1, [x0, #AST_F2]           // prefix
    bl _kcc_ir_build_expr           // → address
   vreg
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_asm_lower_operands ---
// void kcc_ir_asm_lower_operands(node *asm_stmt)
// Lower every operand's expression.  AST_F2
   receives (size<<32)|(value_vreg+1)
// for an input, AST_F3 receives (size<<32)|(
   addr_vreg+1) for an output; a zero
// low half means that half is absent.  The size
   is the operand type's byte
// size (from the sema-resolved type), used by the
    back end to pick the store
// width and the x/w register form.
.globl _kcc_ir_asm_lower_operands
.p2align 2
_kcc_ir_asm_lower_operands:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    ldr x19, [x0, #AST_F2]          // first operand
.Lalo_loop:
    cbz x19, .Lalo_done
    // size from the operand expression's resolved
    type (default 8).
    ldr x0, [x19, #AST_F2]          // expression
    mov x20, #8
    ldr x1, [x0, #AST_F3]           // type
    cbz x1, .Lalo_have_size
    ldr x1, [x1, #TY_SIZE]
    cbz x1, .Lalo_have_size
    mov x20, x1                     // size
.Lalo_have_size:
    ldr x0, [x19, #AST_F0]          // constraint
   ptr
    ldrb w21, [x0]                  // first char
    cmp w21, #'='
    b.eq .Lalo_output
    cmp w21, #'+'
    b.eq .Lalo_inout
    // input: value only
    ldr x0, [x19, #AST_F2]
```

```
    bl _kcc_ir_build_expr
    add x0, x0, #1
    orr x0, x0, x20, lsl #32
    str x0, [x19, #AST_F2]
    str xzr, [x19, #AST_F3]
    b .Lalo_next
.Lalo_inout:
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_build_expr
    add x22, x0, #1
    orr x22, x22, x20, lsl #32     // value half (
   stashed)
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_asm_addr_of
    add x0, x0, #1
    orr x0, x0, x20, lsl #32
    str x0, [x19, #AST_F3]         // address half
    str x22, [x19, #AST_F2]        // value half (
   overwrite expr)
    b .Lalo_next
.Lalo_output:
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_asm_addr_of
    add x0, x0, #1
    orr x0, x0, x20, lsl #32
    str x0, [x19, #AST_F3]
    str xzr, [x19, #AST_F2]
.Lalo_next:
    ldr x19, [x19, #AST_F4]
    b .Lalo_loop
.Lalo_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

325 · used in 30b · defines _kcc_ir_asm_addr_of never used, _kcc_ir_asm_lower_operands 322 · uses _kcc_ast_alloc 168a, _kcc_ir_build_expr 340

29.6. Building Return

A return statement evaluates its expression (if any), *converts* the value to the function's declared return type, emits IR_RET with the result vreg as SRC1, and marks the current basic block as terminated. The conversion is the same §6.8.6.4 coercion an assignment applies: return 504 from a function typed double must yield 504.0, not the integer's bit pattern reinterpreted in a floating register—the defect that made Lua's lua_version (return LUA_VERSION_NUM) hand back zero and trip luaL_checkversion's "version mismatch". A void return emits IR_RET with a void type and no source operand. The termination flag prevents the builder from appending further instructions to this block.

Returning a struct by value. When the current function returns a struct, we do not squeeze the aggregate into result registers. Instead the ABI reserves a *hidden result pointer*: the caller has allocated storage for the value and handed us its address in the last incoming parameter vreg (IRF_PARAMS−1). The return therefore reduces to a field-by-field copy from the returned object into that destination, followed by a void IR_RET. We obtain the source *address* of the returned expression rather than a loaded scalar: an identifier resolves to its alloca vreg, a nested call yields the address of its own result buffer, and .f/[i] lvalues reuse the member and subscript address builders. _kcc_type_layout gives the exact byte size to copy through the hidden sret pointer; copying the exact size (rather than rounding up to whole slots) avoids overrunning the caller's result buffer. A union return takes this same aggregate path—it too is passed by its address, not squeezed into a register—so the kind test admits TY_UNION alongside TY_STRUCT; routing a union down the scalar path instead loaded four bytes of member 0 and returned that in place of the object. The caller side mirrors this: the call builder reserves the result buffer for a union return just as for a struct, or the backend unpacks the returned registers into a never-materialised address.

One ABI wrinkle differs between the two targets on a *memory-class* (over-16-byte) aggregate return. AArch64's caller reads the result from the x8 buffer it handed in, so the callee need only copy through that pointer and ret void. x86-64 SysV instead requires the callee to *return* the hidden sret pointer in %rax, because the caller reads the aggregate through that register. So under --target=x86_64 (_kcc_target) the memory-class return emits a *non-void* IR_RET whose SRC1 is the hidden pointer's parameter vreg—which the code generator loads into the return register—while the ARM64 path keeps its void ret. Without this the x86 caller read %rax garbage; when a *self-hosted* kcc then returned a struct by value (e.g. kcc_ir_resolve_designator's four-word result) it dereferenced that garbage and crashed.

⟨ *IR: build return* 326 ⟩ ≡

```
// -- kcc_ir_build_return ---
// void kcc_ir_build_return(node *ret_stmt)
//   x0 = AST_RETURN_STMT (F0=expr or 0)
.text
.globl _kcc_ir_build_return
.p2align 2
_kcc_ir_build_return:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]             // x21 holds the
   sret vreg across struct_copy (x86 ret)

    mov x19, x0
    ldr x0, [x19, #AST_F0]       // return
   expression
    cbz x0, .Libret_void

    // Struct return-by-value: copy the aggregate
   to the hidden result
    // pointer instead of returning a scalar in x0
   .
    adrp x1, _kcc_ir_cur_func@PAGE
    add  x1, x1, _kcc_ir_cur_func@PAGEOFF
    ldr x1, [x1]
    ldr x1, [x1, #IRF_RETTYPE]
    cbz x1, .Libret_scalar
    ldr x1, [x1, #TY_KIND]
    cmp x1, #TY_STRUCT
    b.eq .Libret_struct
    cmp x1, #TY_UNION              // a union
   returns by value exactly like a struct:
    b.eq .Libret_struct          // pass its
   ADDRESS, not a scalar load of member 0

.Libret_scalar:
```

```
    // Build the return expression
    bl _kcc_ir_build_expr
    mov x20, x0                     // x20 = vreg of
   return value

    // Convert the value to the function's return
   type (int<->double, widths):
    // `return 504` from a double function must
   produce 504.0, not int bits.
    adrp x0, _kcc_ir_cur_func@PAGE
    add  x0, x0, _kcc_ir_cur_func@PAGEOFF
    ldr x0, [x0]
    ldr x2, [x0, #IRF_RETTYPE]    // to = return
   type
    cbz x2, .Libret_scalar_emit
    ldr x1, [x19, #AST_F0]       // return expr
   node
    cbz x1, .Libret_scalar_emit
    ldr x1, [x1, #AST_F3]        // from = expr
   type
    cbz x1, .Libret_scalar_emit
    mov x0, x20                   // value vreg
    bl _kcc_ir_convert            // (value x0,
   from x1, to x2) → converted
    mov x20, x0

.Libret_scalar_emit:
    // Emit ret with value
    mov x0, #IR_RET
    bl _kcc_ir_alloc_instr

    // Get return type from current function
    adrp x1, _kcc_ir_cur_func@PAGE
    add  x1, x1, _kcc_ir_cur_func@PAGEOFF
    ldr x1, [x1]
    ldr x1, [x1, #IRF_RETTYPE]
    str x1, [x0, #IR_TYPE]
    str x20, [x0, #IR_SRC1]
    bl _kcc_ir_emit

    // Mark BB as terminated
    b .Libret_terminate

.Libret_void:
    mov x0, #IR_RET
    bl _kcc_ir_alloc_instr
    // void ret
    adrp x1, _kcc_type_void@PAGE
    add  x1, x1, _kcc_type_void@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    bl _kcc_ir_emit
    b .Libret_terminate

.Libret_struct:
    // x0 = return expression node; resolve its
   struct address (x20).
    bl _kcc_ir_build_struct_addr
    mov x20, x0
    adrp x0, _kcc_ir_cur_func@PAGE
    add  x0, x0, _kcc_ir_cur_func@PAGEOFF
    ldr x0, [x0]
    ldr x19, [x0, #IRF_RETTYPE]    // x19 = return
   type
    mov x0, x19
    bl _kcc_ir_classify_type
    cmp x0, #4                     // REF (>16B) →
   hidden result pointer
    b.eq .Libret_struct_ref

    // ST1/ST2: return the aggregate in x0(-x1).
   IR_RET carries the struct
    // address in SRC1 and the struct type;
   codegen loads the words.
    mov x0, #IR_RET
    bl _kcc_ir_alloc_instr
    str x19, [x0, #IR_TYPE]
    str x20, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    b .Libret_terminate

.Libret_struct_ref:
    // >16B: copy through the hidden result
   pointer (received in x8), void ret.
    // Copy exactly sizeof(struct) bytes; the
   caller's sret buffer is sized to
    // the struct, so rounding up could overrun it
   .
    mov x0, x19
    bl _kcc_type_layout           // byte size (
   kept in x2, untouched below)
    mov x2, x0
    // The hidden sret pointer is the last
   reserved parameter vreg, i.e. the
    // sum of every parameter's vreg width minus
   one (an ST2 param takes two).
    adrp x0, _kcc_ir_cur_func@PAGE
    add  x0, x0, _kcc_ir_cur_func@PAGEOFF
    ldr x0, [x0]
    ldr x3, [x0, #IRF_PARAMS]
    ldr x4, [x0, #IRF_PARAM_CLS]
    mov x1, #0                     // width sum
    mov x5, #0                     // index
.Libret_sret_loop:
    cmp x5, x3
    b.ge .Libret_sret_done
    add x1, x1, #1
    cbz x4, .Libret_sret_next
    ldr x6, [x4, x5, lsl #3]
    cmp x6, #3
    b.ne .Libret_sret_next
    add x1, x1, #1
.Libret_sret_next:
    add x5, x5, #1
    b .Libret_sret_loop
.Libret_sret_done:
    sub x1, x1, #1                // dst = hidden
   sret vreg
    mov x21, x1                   // preserve it
   across the struct_copy call
    mov x0, x20                   // src base
    bl _kcc_ir_emit_struct_copy

    mov x0, #IR_RET
    bl _kcc_ir_alloc_instr
    // x86-64 SysV hands a memory-class aggregate'
   s address back in %rax, and the
    // caller reads the result through it; so on
   x86 the callee must RETURN the
    // hidden sret pointer (the last param vreg),
   not merely write through it.  A
    // non-void RET carrying that vreg makes
   codegen load it into %rax.  AArch64's
    // caller reads its own x8 buffer, so its RET
   stays void (unchanged).
    adrp x1, _kcc_target@PAGE
    add  x1, x1, _kcc_target@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Libret_ref_void
    str  x19, [x0, #IR_TYPE]      // struct return
    type (non-void)
    str  x21, [x0, #IR_SRC1]      // the sret
   pointer vreg
    b    .Libret_ref_emit
.Libret_ref_void:
    adrp x1, _kcc_type_void@PAGE
    add  x1, x1, _kcc_type_void@PAGEOFF
    ldr  x1, [x1]
```

```
    str  x1, [x0, #IR_TYPE]
.Libret_ref_emit:
    bl _kcc_ir_emit

.Libret_terminate:
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Libret_done
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

.Libret_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

326 · used in 30b · defines _kcc_ir_build_return 322 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_build_struct_addr 405, _kcc_ir_classify_type 343, _kcc_ir_convert 343, _kcc_ir_cur_bb 305c, _kcc_ir_cur_func 305c, _kcc_ir_emit 304, _kcc_ir_emit_struct_copy 404, _kcc_target 568b, _kcc_type_layout 267, 268, _kcc_type_void 250b

29.7. String Constants

The IR dump and builder share a set of string constants for opcode mnemonics, punctuation, and formatting. These live in the __TEXT,__cstring section so the linker can deduplicate them. The loop stack data (depth counter and 16-entry array) is placed in .data since it is mutable at run time.

⟨ *IR: builder data* 328 ⟩ ≡

```
// -- IR builder string constants ---
.section __TEXT,__cstring,cstring_literals

_kcc_str_main:          .asciz "main"
.globl _kcc_str_modinit
_kcc_str_modinit:       .asciz "kcc.module_init"
_kcc_ir_str_func:       .asciz "func @"
_kcc_ir_str_arrow:      .asciz "() -> "
_kcc_ir_str_colon_nl:   .asciz ":\n"
_kcc_ir_str_bb:         .asciz "  bb"
_kcc_ir_str_indent:     .asciz "    "
_kcc_ir_str_eq:         .asciz " = "
_kcc_ir_str_comma:      .asciz ", %"
_kcc_ir_str_ptr_pct:    .asciz ", ptr %"
_kcc_ir_str_comma_bb:   .asciz ", bb"
_kcc_ir_str_unknown:    .asciz "???"

// Opcode names
_kcc_ir_op_add:         .asciz "add"
_kcc_ir_op_sub:         .asciz "sub"
_kcc_ir_op_mul:         .asciz "mul"
_kcc_ir_op_sdiv:        .asciz "sdiv"
_kcc_ir_op_udiv:        .asciz "udiv"
_kcc_ir_op_smod:        .asciz "smod"
_kcc_ir_op_umod:        .asciz "umod"
_kcc_ir_op_neg:         .asciz "neg"
_kcc_ir_op_and:         .asciz "and"
_kcc_ir_op_or:          .asciz "or"
_kcc_ir_op_xor:         .asciz "xor"
_kcc_ir_op_shl:         .asciz "shl"
_kcc_ir_op_lshr:        .asciz "lshr"
_kcc_ir_op_ashr:        .asciz "ashr"
_kcc_ir_op_not:         .asciz "not"
_kcc_ir_op_cmp_eq:      .asciz "cmp_eq"
_kcc_ir_op_cmp_ne:      .asciz "cmp_ne"
_kcc_ir_op_cmp_slt:     .asciz "cmp_slt"
_kcc_ir_op_cmp_sle:     .asciz "cmp_sle"
_kcc_ir_op_cmp_sgt:     .asciz "cmp_sgt"
_kcc_ir_op_cmp_sge:     .asciz "cmp_sge"
_kcc_ir_op_cmp_ult:     .asciz "cmp_ult"
_kcc_ir_op_cmp_ule:     .asciz "cmp_ule"
_kcc_ir_op_load:        .asciz "load "
_kcc_ir_op_store:       .asciz "store "
_kcc_ir_op_alloca:      .asciz "alloca "
_kcc_ir_op_gep:         .asciz "gep "
_kcc_ir_op_br:          .asciz "br bb"
_kcc_ir_op_br_cond:     .asciz "br_cond %"
_kcc_ir_op_ret:         .asciz "ret "
_kcc_ir_op_call:        .asciz "call "
_kcc_ir_op_call_ind:    .asciz "call_ind "
_kcc_ir_op_sext:        .asciz "sext"
_kcc_ir_op_zext:        .asciz "zext"
_kcc_ir_op_trunc:       .asciz "trunc"
_kcc_ir_op_bitcast:     .asciz "bitcast"
_kcc_ir_op_sitofp:      .asciz "sitofp"
_kcc_ir_op_uitofp:      .asciz "uitofp"
_kcc_ir_op_fptosi:      .asciz "fptosi"
_kcc_ir_op_const:       .asciz "const "
_kcc_ir_op_nop:         .asciz "nop"
_kcc_ir_op_global_ref:  .asciz "global_ref "
_kcc_ir_op_string_lit:  .asciz "string_lit "

// -- Loop stack (break/continue targets) --
.data
.p2align 3
.globl _kcc_loop_depth
_kcc_loop_depth: .quad 0
.globl _kcc_loop_stack
_kcc_loop_stack: .space 256   // 16 levels * 16
   bytes (break_bb, continue_bb)
.globl _kcc_ir_trans_unit
_kcc_ir_trans_unit: .quad 0
// Field decl matched by the most recent
   _kcc_ir_build_member_addr, so the
// member load/store can see its bit-field
   metadata (0 if none).
.globl _kcc_ir_member_field
_kcc_ir_member_field: .quad 0
// Lazily-cloned `volatile unsigned long`
   descriptor: a volatile bit-field's
// container load/store carries this so the
   optimizer sees the volatile bit.
.globl _kcc_ir_vol_longu
_kcc_ir_vol_longu: .quad 0

// Short type names
.section __TEXT,__cstring,cstring_literals
_kcc_ir_ty_void:        .asciz "void"
_kcc_ir_ty_i1:          .asciz "i1"
_kcc_ir_ty_i8:          .asciz "i8"
_kcc_ir_ty_i16:         .asciz "i16"
_kcc_ir_ty_i32:         .asciz "i32"
_kcc_ir_ty_i64:         .asciz "i64"
_kcc_ir_ty_f32:         .asciz "f32"
_kcc_ir_ty_f64:         .asciz "f64"
_kcc_ir_ty_ptr:         .asciz "ptr"
_kcc_ir_ty_struct:      .asciz "struct"
_kcc_ir_ty_unknown:     .asciz "?type"
.Lidts_volstr:          .asciz "volatile "
```

□

328 · used in 30b · defines _kcc_ir_op_add 336, _kcc_ir_op_alloca 338, _kcc_ir_op_and 336, _kcc_ir_op_ashr 336, _kcc_ir_op_bitcast 337, _kcc_ir_op_br 338, _kcc_ir_op_br_cond 338, _kcc_ir_op_call 338, _kcc_ir_op_call_ind 338, _kcc_ir_op_cmp_eq 336, _kcc_ir_op_cmp_ne 336,

_kcc_ir_op_cmp_sge 336, _kcc_ir_op_cmp_sgt 336,
_kcc_ir_op_cmp_sle 336, _kcc_ir_op_cmp_slt 336,
_kcc_ir_op_cmp_ule 336, _kcc_ir_op_cmp_ult 336,
_kcc_ir_op_const 338, _kcc_ir_op_gep 338,
_kcc_ir_op_global_ref 338, _kcc_ir_op_load 338,
_kcc_ir_op_lshr 336, _kcc_ir_op_mul 336,
_kcc_ir_op_neg 337, _kcc_ir_op_nop 338, _kcc_ir_op_not 337,
_kcc_ir_op_or 336, _kcc_ir_op_ret 338, _kcc_ir_op_sdiv 336,
_kcc_ir_op_sext 337, _kcc_ir_op_shl 336,
_kcc_ir_op_smod 336, _kcc_ir_op_store 338,
_kcc_ir_op_string_lit 338, _kcc_ir_op_sub 336,
_kcc_ir_op_trunc 337, _kcc_ir_op_udiv 336,
_kcc_ir_op_umod 336, _kcc_ir_op_xor 336,
_kcc_ir_op_zext 337, _kcc_ir_str_arrow 334,
_kcc_ir_str_bb 334, _kcc_ir_str_colon_nl 334,
_kcc_ir_str_comma 336, 338, _kcc_ir_str_comma_bb 338,
_kcc_ir_str_eq 335, _kcc_ir_str_func 334,
_kcc_ir_str_indent 335, _kcc_ir_str_ptr_pct 338,
_kcc_ir_str_unknown 335, _kcc_ir_ty_f32 339,
_kcc_ir_ty_f64 339, _kcc_ir_ty_i1 339, _kcc_ir_ty_i16 339,
_kcc_ir_ty_i32 339, _kcc_ir_ty_i64 339, _kcc_ir_ty_i8 339,
_kcc_ir_ty_ptr 339, _kcc_ir_ty_struct 339,
_kcc_ir_ty_unknown 339, _kcc_ir_ty_void 339,
_kcc_loop_depth 426, 434b, 435ab, _kcc_loop_stack 426, 434b,
435ab · uses _kcc_ir_build_member_addr 355

29.8. IR Dump

We implement the dump first so that we can test the builder incrementally. The format mirrors LLVM IR loosely:

```
func @main() -> i32:
  bb0:
    %0 = const i32 42
    ret i32 %0
```

⟨ *IR: dump* 334 ⟩ ≡

```
// -- kcc_ir_dump ---
// void kcc_ir_dump(void)
// Prints all IR functions in textual form.
.text
.globl _kcc_ir_dump
.p2align 2
_kcc_ir_dump:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]

    // Load function list vec
    adrp x0, _kcc_ir_funcs@PAGE
    add  x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x19, [x0]                // x19 = vec ptr
    cbz x19, .Lird_done

    ldr x20, [x19, #VEC_COUNT]   // x20 = func
  count
    mov x21, #0                  // x21 = index

.Lird_func_loop:
    cmp x21, x20
    b.ge .Lird_done

    // Get func pointer: vec_get(vec, index)
    mov x0, x19
    mov x1, x21
    bl _kcc_vec_get
    ldr x22, [x0]                // x22 = IR func
  ptr (deref pointer elem)

    // Print "func @"
    adrp x0, _kcc_ir_str_func@PAGE
    add  x0, x0, _kcc_ir_str_func@PAGEOFF
    bl _kcc_io_puts

    // Print function name
    ldr x0, [x22, #IRF_NAME]
    bl _kcc_io_puts

    // Print "() -> "
    adrp x0, _kcc_ir_str_arrow@PAGE
    add  x0, x0, _kcc_ir_str_arrow@PAGEOFF
    bl _kcc_io_puts

    // Print return type
    ldr x0, [x22, #IRF_RETTYPE]
    bl _kcc_ir_dump_type_short

    // Print ":"
    adrp x0, _kcc_ir_str_colon_nl@PAGE
    add  x0, x0, _kcc_ir_str_colon_nl@PAGEOFF
    bl _kcc_io_puts

    // Walk basic blocks via the blocks vec
    ldr x0, [x22, #IRF_BLOCKS]
    cbz x0, .Lird_next_func
    // Save blocks vec info in callee-saved regs
    // We reuse x22 temporarily - save func ptr on
    stack
    str x22, [sp, #80]           // save func ptr
  in free stack slot
    mov x22, x0                  // x22 = blocks
  vec
    ldr x23, [x22, #VEC_COUNT]
    mov x24, #0

.Lird_bb_loop:
    cmp x24, x23
    b.ge .Lird_bb_done

    mov x0, x22
    mov x1, x24
    bl _kcc_vec_get
    ldr x25, [x0]                // x25 = BB ptr

    // Print "  bb"
    adrp x0, _kcc_ir_str_bb@PAGE
    add  x0, x0, _kcc_ir_str_bb@PAGEOFF
    bl _kcc_io_puts

    // Print BB id
    ldr x0, [x25, #BB_ID]
    bl _kcc_io_putu

    // Print ":"
    adrp x0, _kcc_ir_str_colon_nl@PAGE
    add  x0, x0, _kcc_ir_str_colon_nl@PAGEOFF
    bl _kcc_io_puts

    // Walk instructions in this BB
    ldr x26, [x25, #BB_FIRST]

.Lird_instr_loop:
    cbz x26, .Lird_next_bb

    mov x0, x26
    bl _kcc_ir_dump_instr
```

```
    ldr x26, [x26, #IR_NEXT]
    b .Lird_instr_loop

.Lird_next_bb:
    add x24, x24, #1
    b .Lird_bb_loop

.Lird_bb_done:
    ldr x22, [sp, #80]          // restore func ptr
    // Print blank line between functions
    mov x0, #'\n'
    bl _kcc_io_putc

.Lird_next_func:
    add x21, x21, #1
    b .Lird_func_loop

.Lird_done:
    bl _kcc_io_flush
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

334 · used in 30b · defines _kcc_ir_dump 623 · uses _kcc_io_flush 69b, _kcc_io_puts 70a, _kcc_io_putu 71a, _kcc_ir_dump_instr 335, _kcc_ir_dump_type_short 339, _kcc_ir_funcs 305c, _kcc_ir_str_arrow 328, _kcc_ir_str_bb 328, _kcc_ir_str_colon_nl 328, _kcc_ir_str_func 328, _kcc_vec_get 65b

29.8.1. Dumping a Single Instruction

Each instruction is printed on one line with four-space indentation. The general format is *%dest = op type src1, src2*, though the exact layout varies by opcode: branches print bb*N* targets, calls print the callee name and argument list, and side-effect-only instructions (STORE, RET, BR) omit the destination.

⟨ *IR: dump instruction* 335 ⟩ ≡

```
// -- kcc_ir_dump_instr ---
// void kcc_ir_dump_instr(ir_instr *instr)
//   x0 = instruction pointer
// Prints one instruction line with leading "    "
    indent.
.text
.globl _kcc_ir_dump_instr
.p2align 2
_kcc_ir_dump_instr:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0                 // x19 = instr
    ldr x20, [x19, #IR_OP]      // x20 = opcode

    // Print indent
    adrp x0, _kcc_ir_str_indent@PAGE
    add  x0, x0, _kcc_ir_str_indent@PAGEOFF
    bl _kcc_io_puts

    // Opcodes that don't produce a dest: store,
   br, br_cond, ret, nop
    cmp x20, #IR_STORE
    b.eq .Lidi_no_dest
    cmp x20, #IR_BR
    b.eq .Lidi_no_dest
    cmp x20, #IR_BR_COND
    b.eq .Lidi_no_dest
    cmp x20, #IR_RET
    b.eq .Lidi_no_dest
    cmp x20, #IR_NOP
    b.eq .Lidi_no_dest

    // Print "%N = "
    mov x0, #'%'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_DEST]
    bl _kcc_io_putu
    adrp x0, _kcc_ir_str_eq@PAGE
    add  x0, x0, _kcc_ir_str_eq@PAGEOFF
    bl _kcc_io_puts

.Lidi_no_dest:
    // Dispatch on opcode
    cmp x20, #IR_ADD
    b.eq .Lidi_add
    cmp x20, #IR_SUB
    b.eq .Lidi_sub
    cmp x20, #IR_MUL
    b.eq .Lidi_mul
    cmp x20, #IR_SDIV
    b.eq .Lidi_sdiv
    cmp x20, #IR_UDIV
    b.eq .Lidi_udiv
    cmp x20, #IR_SMOD
    b.eq .Lidi_smod
    cmp x20, #IR_UMOD
    b.eq .Lidi_umod
    cmp x20, #IR_NEG
    b.eq .Lidi_neg
    cmp x20, #IR_AND
    b.eq .Lidi_and
    cmp x20, #IR_OR
    b.eq .Lidi_or
    cmp x20, #IR_XOR
    b.eq .Lidi_xor
    cmp x20, #IR_SHL
    b.eq .Lidi_shl
    cmp x20, #IR_LSHR
    b.eq .Lidi_lshr
    cmp x20, #IR_ASHR
    b.eq .Lidi_ashr
    cmp x20, #IR_NOT
    b.eq .Lidi_not
    cmp x20, #IR_CMP_EQ
    b.eq .Lidi_cmp_eq
    cmp x20, #IR_CMP_NE
    b.eq .Lidi_cmp_ne
    cmp x20, #IR_CMP_SLT
    b.eq .Lidi_cmp_slt
    cmp x20, #IR_CMP_SLE
    b.eq .Lidi_cmp_sle
    cmp x20, #IR_CMP_SGT
    b.eq .Lidi_cmp_sgt
    cmp x20, #IR_CMP_SGE
    b.eq .Lidi_cmp_sge
    cmp x20, #IR_CMP_ULT
    b.eq .Lidi_cmp_ult
    cmp x20, #IR_CMP_ULE
    b.eq .Lidi_cmp_ule
    cmp x20, #IR_LOAD
    b.eq .Lidi_load
    cmp x20, #IR_STORE
    b.eq .Lidi_store
    cmp x20, #IR_ALLOCA
    b.eq .Lidi_alloca
    cmp x20, #IR_GEP
    b.eq .Lidi_gep
    cmp x20, #IR_BR
    b.eq .Lidi_br
    cmp x20, #IR_BR_COND
```

```
    b.eq .Lidi_br_cond
    cmp x20, #IR_RET
    b.eq .Lidi_ret
    cmp x20, #IR_CALL
    b.eq .Lidi_call
    cmp x20, #IR_CALL_IND
    b.eq .Lidi_call_ind
    cmp x20, #IR_SEXT
    b.eq .Lidi_sext
    cmp x20, #IR_ZEXT
    b.eq .Lidi_zext
    cmp x20, #IR_TRUNC
    b.eq .Lidi_trunc
    cmp x20, #IR_BITCAST
    b.eq .Lidi_bitcast
    cmp x20, #IR_SITOFP
    b.eq .Lidi_sitofp
    cmp x20, #IR_UITOFP
    b.eq .Lidi_uitofp
    cmp x20, #IR_FPTOSI
    b.eq .Lidi_fptosi
    cmp x20, #IR_CONST
    b.eq .Lidi_const
    cmp x20, #IR_NOP
    b.eq .Lidi_nop
    cmp x20, #IR_GLOBAL_REF
    b.eq .Lidi_global_ref
    cmp x20, #IR_STRING_LIT
    b.eq .Lidi_string_lit

    // Unknown opcode
    adrp x0, _kcc_ir_str_unknown@PAGE
    add  x0, x0, _kcc_ir_str_unknown@PAGEOFF
    bl _kcc_io_puts
    b .Lidi_nl
```

□

335 · defined at 335, 336, 337, 338 · used in 30b · defines _kcc_ir_dump_instr 334 · uses _kcc_io_puts 70a, _kcc_io_putu 71a, _kcc_ir_str_eq 328, _kcc_ir_str_indent 328, _kcc_ir_str_unknown 328

Binary and Comparison Operations Every arithmetic, bitwise, and comparison opcode follows the same printing pattern: load the opcode's string label into x0 and branch to the shared .Lidi_binop handler. That handler emits the format op type %src1, %src2, reading the type descriptor from IR_TYPE and both source virtual registers from IR_SRC1 and IR_SRC2. Arithmetic opcodes (ADD, SUB, MUL, SDIV/UDIV, SMOD/UMOD) come first, followed by bitwise (AND, OR, XOR, SHL, LSHR, ASHR) and the six signed comparison predicates (CMP_EQ through CMP_SGE).

⟨ *IR: dump instruction* 336 ⟩ +≡

```
// -- Binary ops: "op type %src1, %src2" ---
.Lidi_add:
    adrp x0, _kcc_ir_op_add@PAGE
    add  x0, x0, _kcc_ir_op_add@PAGEOFF
    b .Lidi_binop
.Lidi_sub:
    adrp x0, _kcc_ir_op_sub@PAGE
    add  x0, x0, _kcc_ir_op_sub@PAGEOFF
    b .Lidi_binop
.Lidi_mul:
    adrp x0, _kcc_ir_op_mul@PAGE
    add  x0, x0, _kcc_ir_op_mul@PAGEOFF
    b .Lidi_binop
.Lidi_sdiv:
    adrp x0, _kcc_ir_op_sdiv@PAGE
    add  x0, x0, _kcc_ir_op_sdiv@PAGEOFF
    b .Lidi_binop
.Lidi_udiv:
    adrp x0, _kcc_ir_op_udiv@PAGE
    add  x0, x0, _kcc_ir_op_udiv@PAGEOFF
    b .Lidi_binop
.Lidi_smod:
    adrp x0, _kcc_ir_op_smod@PAGE
    add  x0, x0, _kcc_ir_op_smod@PAGEOFF
    b .Lidi_binop
.Lidi_umod:
    adrp x0, _kcc_ir_op_umod@PAGE
    add  x0, x0, _kcc_ir_op_umod@PAGEOFF
    b .Lidi_binop
.Lidi_and:
    adrp x0, _kcc_ir_op_and@PAGE
    add  x0, x0, _kcc_ir_op_and@PAGEOFF
    b .Lidi_binop
.Lidi_or:
    adrp x0, _kcc_ir_op_or@PAGE
    add  x0, x0, _kcc_ir_op_or@PAGEOFF
    b .Lidi_binop
.Lidi_xor:
    adrp x0, _kcc_ir_op_xor@PAGE
    add  x0, x0, _kcc_ir_op_xor@PAGEOFF
    b .Lidi_binop
.Lidi_shl:
    adrp x0, _kcc_ir_op_shl@PAGE
    add  x0, x0, _kcc_ir_op_shl@PAGEOFF
    b .Lidi_binop
.Lidi_lshr:
    adrp x0, _kcc_ir_op_lshr@PAGE
    add  x0, x0, _kcc_ir_op_lshr@PAGEOFF
    b .Lidi_binop
.Lidi_ashr:
    adrp x0, _kcc_ir_op_ashr@PAGE
    add  x0, x0, _kcc_ir_op_ashr@PAGEOFF
    b .Lidi_binop
.Lidi_cmp_eq:
    adrp x0, _kcc_ir_op_cmp_eq@PAGE
    add  x0, x0, _kcc_ir_op_cmp_eq@PAGEOFF
    b .Lidi_binop
.Lidi_cmp_ne:
    adrp x0, _kcc_ir_op_cmp_ne@PAGE
    add  x0, x0, _kcc_ir_op_cmp_ne@PAGEOFF
    b .Lidi_binop
.Lidi_cmp_slt:
    adrp x0, _kcc_ir_op_cmp_slt@PAGE
    add  x0, x0, _kcc_ir_op_cmp_slt@PAGEOFF
    b .Lidi_binop
.Lidi_cmp_sle:
    adrp x0, _kcc_ir_op_cmp_sle@PAGE
    add  x0, x0, _kcc_ir_op_cmp_sle@PAGEOFF
    b .Lidi_binop
.Lidi_cmp_sgt:
    adrp x0, _kcc_ir_op_cmp_sgt@PAGE
    add  x0, x0, _kcc_ir_op_cmp_sgt@PAGEOFF
    b .Lidi_binop
.Lidi_cmp_sge:
    adrp x0, _kcc_ir_op_cmp_sge@PAGE
    add  x0, x0, _kcc_ir_op_cmp_sge@PAGEOFF
    b .Lidi_binop
.Lidi_cmp_ult:
    adrp x0, _kcc_ir_op_cmp_ult@PAGE
    add  x0, x0, _kcc_ir_op_cmp_ult@PAGEOFF
    b .Lidi_binop
.Lidi_cmp_ule:
    adrp x0, _kcc_ir_op_cmp_ule@PAGE
    add  x0, x0, _kcc_ir_op_cmp_ule@PAGEOFF
    b .Lidi_binop

.Lidi_binop:
    bl _kcc_io_puts
    // Print " type %src1, %src2"
    mov x0, #' '
    bl _kcc_io_putc
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_dump_type_short
    mov x0, #' '
```

```
    bl _kcc_io_putc
    mov x0, #'%'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_SRC1]
    bl _kcc_io_putu
    adrp x0, _kcc_ir_str_comma@PAGE
    add  x0, x0, _kcc_ir_str_comma@PAGEOFF
    bl _kcc_io_puts
    mov x0, #'%'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_SRC2]
    bl _kcc_io_putu
    b .Lidi_nl
```

□

336 · defined at 335, 336, 337, 338 · used in 30b · uses _kcc_io_puts 70a, _kcc_io_putu 71a, _kcc_ir_dump_type_short 339, _kcc_ir_op_add 328, _kcc_ir_op_and 328, _kcc_ir_op_ashr 328, _kcc_ir_op_cmp_eq 328, _kcc_ir_op_cmp_ne 328, _kcc_ir_op_cmp_sge 328, _kcc_ir_op_cmp_sgt 328, _kcc_ir_op_cmp_sle 328, _kcc_ir_op_cmp_slt 328, _kcc_ir_op_cmp_ule 328, _kcc_ir_op_cmp_ult 328, _kcc_ir_op_lshr 328, _kcc_ir_op_mul 328, _kcc_ir_op_or 328, _kcc_ir_op_sdiv 328, _kcc_ir_op_shl 328, _kcc_ir_op_smod 328, _kcc_ir_op_sub 328, _kcc_ir_op_udiv 328, _kcc_ir_op_umod 328, _kcc_ir_op_xor 328, _kcc_ir_str_comma 328

Unary and Type-Conversion Operations Unary instructions—NEG, NOT, and the three width-changing casts SEXT, ZEXT, TRUNC—use the shared .Lidi_unop handler. The output format is op type %src1, identical to the binary pattern but with only one source operand read from IR_SRC1. Sign- and zero-extension are semantically distinct at the IR level so that the backend can select the correct ARM64 widening instruction (SXTW versus UXTW).

⟨ *IR: dump instruction* 337 ⟩ + ≡

```
// -- Unary ops: "op type %src1" ---
.Lidi_neg:
    adrp x0, _kcc_ir_op_neg@PAGE
    add  x0, x0, _kcc_ir_op_neg@PAGEOFF
    b .Lidi_unop
.Lidi_not:
    adrp x0, _kcc_ir_op_not@PAGE
    add  x0, x0, _kcc_ir_op_not@PAGEOFF
    b .Lidi_unop
.Lidi_sext:
    adrp x0, _kcc_ir_op_sext@PAGE
    add  x0, x0, _kcc_ir_op_sext@PAGEOFF
    b .Lidi_unop
.Lidi_zext:
    adrp x0, _kcc_ir_op_zext@PAGE
    add  x0, x0, _kcc_ir_op_zext@PAGEOFF
    b .Lidi_unop
.Lidi_trunc:
    adrp x0, _kcc_ir_op_trunc@PAGE
    add  x0, x0, _kcc_ir_op_trunc@PAGEOFF
    b .Lidi_unop
.Lidi_bitcast:
    adrp x0, _kcc_ir_op_bitcast@PAGE
    add  x0, x0, _kcc_ir_op_bitcast@PAGEOFF
    b .Lidi_unop
.Lidi_sitofp:
    adrp x0, _kcc_ir_op_sitofp@PAGE
    add  x0, x0, _kcc_ir_op_sitofp@PAGEOFF
    b .Lidi_unop
.Lidi_uitofp:
    adrp x0, _kcc_ir_op_uitofp@PAGE
    add  x0, x0, _kcc_ir_op_uitofp@PAGEOFF
    b .Lidi_unop
.Lidi_fptosi:
    adrp x0, _kcc_ir_op_fptosi@PAGE
    add  x0, x0, _kcc_ir_op_fptosi@PAGEOFF
    b .Lidi_unop

.Lidi_unop:
    bl _kcc_io_puts
    mov x0, #' '
    bl _kcc_io_putc
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_dump_type_short
    mov x0, #' '
    bl _kcc_io_putc
    mov x0, #'%'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_SRC1]
    bl _kcc_io_putu
    b .Lidi_nl
```

□

337 · defined at 335, 336, 337, 338 · used in 30b · uses _kcc_io_puts 70a, _kcc_io_putu 71a, _kcc_ir_dump_type_short 339, _kcc_ir_op_bitcast 328, _kcc_ir_op_neg 328, _kcc_ir_op_not 328, _kcc_ir_op_sext 328, _kcc_ir_op_trunc 328, _kcc_ir_op_zext 328

Memory, Control Flow, and Miscellaneous Operations The remaining opcodes each have their own bespoke printing logic because their operand layouts differ from one another. Memory operations LOAD and STORE include pointer annotations; ALLOCA prints only its type. The branch family uses basic-block identifiers (bb$N$) rather than virtual registers: BR takes one target while BR_COND takes a condition vreg plus true/false targets. RET checks whether the return type is void to decide if a value operand follows. CALL prints the callee name prefixed with @. Finally, CONST, NOP, GLOBAL_REF, and STRING_LIT handle immediates, no-ops, global symbol references, and inline string data respectively. The shared epilogue at .Lidi_nl appends a newline and restores callee-saved registers before returning.

⟨ *IR: dump instruction* 338 ⟩ + ≡

```
// -- load: "load type, ptr %src1" ---
.Lidi_load:
    adrp x0, _kcc_ir_op_load@PAGE
    add  x0, x0, _kcc_ir_op_load@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_dump_type_short
    adrp x0, _kcc_ir_str_ptr_pct@PAGE
    add  x0, x0, _kcc_ir_str_ptr_pct@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_SRC1]
    bl _kcc_io_putu
    b .Lidi_nl

// -- store: "store type %src1, ptr %src2" ---
.Lidi_store:
    adrp x0, _kcc_ir_op_store@PAGE
    add  x0, x0, _kcc_ir_op_store@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_dump_type_short
    mov x0, #' '
    bl _kcc_io_putc
```

```
    mov x0, #'%'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_SRC1]
    bl _kcc_io_putu
    adrp x0, _kcc_ir_str_ptr_pct@PAGE
    add  x0, x0, _kcc_ir_str_ptr_pct@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_SRC2]
    bl _kcc_io_putu
    b .Lidi_nl

// -- alloca: "alloca type" ---
.Lidi_alloca:
    adrp x0, _kcc_ir_op_alloca@PAGE
    add  x0, x0, _kcc_ir_op_alloca@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_dump_type_short
    b .Lidi_nl

// -- gep: "gep %base, %index" ---
.Lidi_gep:
    adrp x0, _kcc_ir_op_gep@PAGE
    add  x0, x0, _kcc_ir_op_gep@PAGEOFF
    bl _kcc_io_puts
    mov x0, #'%'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_SRC1]
    bl _kcc_io_putu
    adrp x0, _kcc_ir_str_comma@PAGE
    add  x0, x0, _kcc_ir_str_comma@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_SRC2]
    bl _kcc_io_putu
    b .Lidi_nl

// -- br: "br bb<N>" ---
.Lidi_br:
    adrp x0, _kcc_ir_op_br@PAGE
    add  x0, x0, _kcc_ir_op_br@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_SRC1]
    bl _kcc_io_putu
    b .Lidi_nl

// -- br_cond: "br_cond %cond, bb<T>, bb<F>" ---
.Lidi_br_cond:
    adrp x0, _kcc_ir_op_br_cond@PAGE
    add  x0, x0, _kcc_ir_op_br_cond@PAGEOFF
    bl _kcc_io_puts
    mov x0, #'%'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_DEST]   // cond vreg is in
   DEST for br_cond
    bl _kcc_io_putu
    adrp x0, _kcc_ir_str_comma_bb@PAGE
    add  x0, x0, _kcc_ir_str_comma_bb@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_SRC1]   // true BB id
    bl _kcc_io_putu
    adrp x0, _kcc_ir_str_comma_bb@PAGE
    add  x0, x0, _kcc_ir_str_comma_bb@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_SRC2]   // false BB id
    bl _kcc_io_putu
    b .Lidi_nl

// -- ret: "ret type %src1" or "ret void" ---
.Lidi_ret:
    adrp x0, _kcc_ir_op_ret@PAGE
    add  x0, x0, _kcc_ir_op_ret@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_dump_type_short
    // Check if void return
    ldr x0, [x19, #IR_TYPE]
    cbz x0, .Lidi_nl
    ldr x0, [x0, #TY_KIND]
    cmp x0, #TY_VOID
    b.eq .Lidi_nl
    // Non-void: print " %src1"
    mov x0, #' '
    bl _kcc_io_putc
    mov x0, #'%'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_SRC1]
    bl _kcc_io_putu
    b .Lidi_nl

// -- call: "call type @name(%src1, ...)" ---
.Lidi_call:
    adrp x0, _kcc_ir_op_call@PAGE
    add  x0, x0, _kcc_ir_op_call@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_dump_type_short
    // Print " @<name>"
    mov x0, #' '
    bl _kcc_io_putc
    mov x0, #'@'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_SRC1]   // SRC1 = name
   pointer
    cbz x0, .Lidi_nl
    bl _kcc_io_puts
    b .Lidi_nl

// -- call_ind: "call_ind type %<vreg>(...)" ---
.Lidi_call_ind:
    adrp x0, _kcc_ir_op_call_ind@PAGE
    add  x0, x0, _kcc_ir_op_call_ind@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_dump_type_short
    // Print " %<vreg>"
    mov x0, #' '
    bl _kcc_io_putc
    mov x0, #'%'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_SRC1]       // SRC1 = fn-ptr
   vreg
    bl _kcc_io_putu
    b .Lidi_nl

// -- const: "const type <imm>" ---
.Lidi_const:
    adrp x0, _kcc_ir_op_const@PAGE
    add  x0, x0, _kcc_ir_op_const@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_dump_type_short
    mov x0, #' '
    bl _kcc_io_putc
    ldr x0, [x19, #IR_SRC1]   // immediate value
    bl _kcc_io_putu
    b .Lidi_nl

// -- nop ---
.Lidi_nop:
    adrp x0, _kcc_ir_op_nop@PAGE
    add  x0, x0, _kcc_ir_op_nop@PAGEOFF
    bl _kcc_io_puts
    b .Lidi_nl

// -- global_ref: "global_ref type @name" ---
.Lidi_global_ref:
    adrp x0, _kcc_ir_op_global_ref@PAGE
    add  x0, x0, _kcc_ir_op_global_ref@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x19, #IR_TYPE]
```

```
    bl _kcc_ir_dump_type_short
    mov x0, #' '
    bl _kcc_io_putc
    mov x0, #'@'
    bl _kcc_io_putc
    ldr x0, [x19, #IR_SRC1]    // name pointer
    cbz x0, .Lidi_nl
    bl _kcc_io_puts
    b .Lidi_nl

// -- string_lit: "string_lit \"<data>\"" ---
.Lidi_string_lit:
    adrp x0, _kcc_ir_op_string_lit@PAGE
    add  x0, x0, _kcc_ir_op_string_lit@PAGEOFF
    bl _kcc_io_puts
    mov x0, #'"'
    bl _kcc_io_putc
    ldr x21, [x19, #IR_SRC1]
    ldr x22, [x19, #IR_SRC2]
    mov x20, #0
.Lidi_string_ch:
    cmp x20, x22
    b.ge .Lidi_string_end
    ldrb w0, [x21, x20]
    bl _kcc_io_putc
    add x20, x20, #1
    b .Lidi_string_ch
.Lidi_string_end:
    mov x0, #'"'
    bl _kcc_io_putc
    b .Lidi_nl

.Lidi_nl:
    mov x0, #'\n'
    bl _kcc_io_putc

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

338 · defined at 335, 336, 337, 338 · used in 30b · uses _kcc_io_puts 70a, _kcc_io_putu 71a, _kcc_ir_dump_type_short 339, _kcc_ir_op_alloca 328, _kcc_ir_op_br 328, _kcc_ir_op_br_cond 328, _kcc_ir_op_call 328, _kcc_ir_op_call_ind 328, _kcc_ir_op_const 328, _kcc_ir_op_gep 328, _kcc_ir_op_global_ref 328, _kcc_ir_op_load 328, _kcc_ir_op_nop 328, _kcc_ir_op_ret 328, _kcc_ir_op_store 328, _kcc_ir_op_string_lit 328, _kcc_ir_str_comma 328, _kcc_ir_str_comma_bb 328, _kcc_ir_str_ptr_pct 328

29.8.2. Short Type Name

For IR output we print compact type names: i32, i64, void, ptr, etc.

⟨ *IR: dump type short* 339 ⟩ ≡

```
// -- kcc_ir_dump_type_short ---
// void kcc_ir_dump_type_short(type_desc *t)
//   x0 = type descriptor (may be NULL)
// Prints a short type label.
.text
.globl _kcc_ir_dump_type_short
.p2align 2
_kcc_ir_dump_type_short:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    cbz x0, .Lidts_unknown
    // A real type descriptor is arena-allocated (a large address).  Guard the
    // dereference so a malformed IR_TYPE (a small non-pointer sentinel) prints
    // as unknown instead of faulting---this keeps --dump-ir usable for
    // inspecting broken IR, which is exactly when a dump is most wanted.
    cmp x0, #0x1000
    b.lo .Lidts_unknown
    mov x19, x0

    // Prefix a volatile-qualified type with "volatile " so a dump makes the
    // qualifier visible (and confirms it reached the IR).
    ldr x0, [x19, #TY_QUALS]
    tst x0, #2
    b.eq .Lidts_kind
    adrp x0, .Lidts_volstr@PAGE
    add  x0, x0, .Lidts_volstr@PAGEOFF
    bl _kcc_io_puts
.Lidts_kind:
    ldr x0, [x19, #TY_KIND]

    cmp x0, #TY_VOID
    b.eq .Lidts_void
    cmp x0, #TY_BOOL
    b.eq .Lidts_i1
    cmp x0, #TY_CHAR
    b.eq .Lidts_i8
    cmp x0, #TY_SHORT
    b.eq .Lidts_i16
    cmp x0, #TY_INT
    b.eq .Lidts_i32
    cmp x0, #TY_LONG
    b.eq .Lidts_i64
    cmp x0, #TY_LLONG
    b.eq .Lidts_i64
    cmp x0, #TY_FLOAT
    b.eq .Lidts_f32
    cmp x0, #TY_DOUBLE
    b.eq .Lidts_f64
    cmp x0, #TY_PTR
    b.eq .Lidts_ptr
    cmp x0, #TY_ARRAY
    b.eq .Lidts_ptr
    cmp x0, #TY_FUNC
    b.eq .Lidts_ptr
    cmp x0, #TY_STRUCT
    b.eq .Lidts_struct
    cmp x0, #TY_UNION
    b.eq .Lidts_struct
    cmp x0, #TY_ENUM
    b.eq .Lidts_i32
    b .Lidts_unknown

.Lidts_void:
    adrp x0, _kcc_ir_ty_void@PAGE
    add  x0, x0, _kcc_ir_ty_void@PAGEOFF
    b .Lidts_print
.Lidts_i1:
    adrp x0, _kcc_ir_ty_i1@PAGE
    add  x0, x0, _kcc_ir_ty_i1@PAGEOFF
    b .Lidts_print
.Lidts_i8:
    adrp x0, _kcc_ir_ty_i8@PAGE
    add  x0, x0, _kcc_ir_ty_i8@PAGEOFF
    b .Lidts_print
.Lidts_i16:
    adrp x0, _kcc_ir_ty_i16@PAGE
    add  x0, x0, _kcc_ir_ty_i16@PAGEOFF
    b .Lidts_print
.Lidts_i32:
```

```
    adrp x0, _kcc_ir_ty_i32@PAGE
    add  x0, x0, _kcc_ir_ty_i32@PAGEOFF
    b .Lidts_print
.Lidts_i64:
    adrp x0, _kcc_ir_ty_i64@PAGE
    add  x0, x0, _kcc_ir_ty_i64@PAGEOFF
    b .Lidts_print
.Lidts_f32:
    adrp x0, _kcc_ir_ty_f32@PAGE
    add  x0, x0, _kcc_ir_ty_f32@PAGEOFF
    b .Lidts_print
.Lidts_f64:
    adrp x0, _kcc_ir_ty_f64@PAGE
    add  x0, x0, _kcc_ir_ty_f64@PAGEOFF
    b .Lidts_print
.Lidts_ptr:
    adrp x0, _kcc_ir_ty_ptr@PAGE
    add  x0, x0, _kcc_ir_ty_ptr@PAGEOFF
    b .Lidts_print
.Lidts_struct:
    adrp x0, _kcc_ir_ty_struct@PAGE
    add  x0, x0, _kcc_ir_ty_struct@PAGEOFF
    b .Lidts_print
.Lidts_unknown:
    adrp x0, _kcc_ir_ty_unknown@PAGE
    add  x0, x0, _kcc_ir_ty_unknown@PAGEOFF

.Lidts_print:
    bl _kcc_io_puts

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

339 · used in 30b · defines _kcc_ir_dump_type_short 334, 336, 337, 338 · uses _kcc_io_puts 70a, _kcc_ir_ty_f32 328, _kcc_ir_ty_f64 328, _kcc_ir_ty_i1 328, _kcc_ir_ty_i16 328, _kcc_ir_ty_i32 328, _kcc_ir_ty_i64 328, _kcc_ir_ty_i8 328, _kcc_ir_ty_ptr 328, _kcc_ir_ty_struct 328, _kcc_ir_ty_unknown 328, _kcc_ir_ty_void 328

29.9. Building Expressions

The expression dispatcher examines the AST tag of its input node and delegates to a specialised builder for each expression kind. Every builder returns the vreg number holding the result; this convention lets callers compose expressions by feeding one builder's result into another's operand slots. A cast expression delegates to `_kcc_ir_build_cast`, which emits `IR_TRUNC` when the target type is narrower than the source. Parenthesised expressions unwrap to their child. The comma operator evaluates both sides and returns the right-hand vreg.

⟨ *IR: build expression* 340 ⟩ ≡

```
// -- kcc_ir_build_expr ---
// uint64_t kcc_ir_build_expr(node *expr)
//   x0 = AST expression node
// Returns vreg number containing the result.
.text
.globl _kcc_ir_build_expr
.p2align 2
_kcc_ir_build_expr:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    cbz x0, .Libe_zero
    mov x19, x0

    ldr x0, [x19, #AST_TAG]

    cmp x0, #AST_INT_LIT
    b.eq .Libe_int_lit
    cmp x0, #AST_FLOAT_LIT
    b.eq .Libe_float_lit
    cmp x0, #AST_CHAR_LIT
    b.eq .Libe_char_lit
    cmp x0, #AST_IDENT_EXPR
    b.eq .Libe_ident
    cmp x0, #AST_BINARY_EXPR
    b.eq .Libe_binary
    cmp x0, #AST_UNARY_EXPR
    b.eq .Libe_unary
    cmp x0, #AST_ASSIGN_EXPR
    b.eq .Libe_assign
    cmp x0, #AST_CALL_EXPR
    b.eq .Libe_call
    cmp x0, #AST_CAST_EXPR
    b.eq .Libe_cast
    cmp x0, #AST_COMMA_EXPR
    b.eq .Libe_comma
    cmp x0, #AST_COND_EXPR
    b.eq .Libe_cond
    cmp x0, #AST_PAREN_EXPR
    b.eq .Libe_paren
    cmp x0, #AST_STR_LIT
    b.eq .Libe_string_lit
    cmp x0, #AST_SIZEOF_EXPR
    b.eq .Libe_sizeof
    cmp x0, #AST_ALIGNOF_EXPR
    b.eq .Libe_alignof
    cmp x0, #AST_SUBSCRIPT
    b.eq .Libe_subscript
    cmp x0, #AST_MEMBER
    b.eq .Libe_member
    cmp x0, #AST_VA_ARG
    b.eq .Libe_va_arg
    cmp x0, #AST_TYPES_COMPAT
    b.eq .Libe_types_compat
    cmp x0, #AST_GENERIC
    b.eq .Libe_generic
    cmp x0, #AST_COMPOUND_LIT
    b.eq .Libe_compound_lit
    cmp x0, #AST_STMT_EXPR
    b.eq .Libe_stmt_expr

    // Unhandled expression - return 0
    b .Libe_zero

.Libe_stmt_expr:
    mov x0, x19
    bl _kcc_ir_build_stmt_expr
    b .Libe_done

.Libe_compound_lit:
    // A compound literal materialises an
   anonymous object; its address is the
    // aggregate ``value'' (as for an aggregate
   identifier), while a scalar
    // literal loads through that address.  Only
   x19 is callee-saved here, so
    // the object address rides the free frame
   slot [sp,#24] across the calls.
    mov x0, x19
    bl _kcc_ir_build_compound_lit
    str x0, [sp, #24]            // stash object
   address
    ldr x0, [x19, #AST_FO]       // literal type
    bl _kcc_sema_resolve_type
    cbz x0, .Libe_cl_addr
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_STRUCT
    b.eq .Libe_cl_addr
    cmp x1, #TY_UNION
```

```
    b.eq .Libe_cl_addr
    cmp x1, #TY_ARRAY
    b.eq .Libe_cl_addr
    // Scalar compound literal: load its value at
   the object's width.
    bl _kcc_ir_store_width        // x0 = size code
    (x0 was the scalar type)
    mov x1, x0                    // size code
    ldr x0, [sp, #24]             // address
    bl _kcc_ir_emit_load
    b .Libe_done
.Libe_cl_addr:
    ldr x0, [sp, #24]             // aggregate: the
    address is the value
    b .Libe_done

.Libe_generic:
    // Sema already chose the association; build
   its expression from AST_F2.
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_build_expr
    b .Libe_done

.Libe_va_arg:
    mov x0, x19
    bl _kcc_ir_build_va_arg
    b .Libe_done

.Libe_int_lit:
    mov x0, x19
    bl _kcc_ir_build_int_lit
    b .Libe_done

.Libe_float_lit:
    mov x0, x19
    bl _kcc_ir_build_float_lit
    b .Libe_done

.Libe_char_lit:
    mov x0, x19
    bl _kcc_ir_build_int_lit    // same treatment:
   F0=value
    b .Libe_done

.Libe_ident:
    mov x0, x19
    bl _kcc_ir_build_ident
    b .Libe_done

.Libe_binary:
    mov x0, x19
    bl _kcc_ir_build_binary
    b .Libe_done

.Libe_unary:
    mov x0, x19
    bl _kcc_ir_build_unary
    b .Libe_done

.Libe_assign:
    mov x0, x19
    bl _kcc_ir_build_assign
    b .Libe_done

.Libe_call:
    mov x0, x19
    bl _kcc_ir_build_call
    b .Libe_done

.Libe_cast:
    mov x0, x19
    bl _kcc_ir_build_cast
    b .Libe_done

.Libe_comma:
    // Evaluate LHS (discard), return RHS
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_build_expr
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_expr
    b .Libe_done

.Libe_cond:
    mov x0, x19
    bl _kcc_ir_build_cond_expr
    b .Libe_done

.Libe_paren:
    // Unwrap parenthesized expression - just
   build inner
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_build_expr
    b .Libe_done

.Libe_string_lit:
    mov x0, x19
    bl _kcc_ir_build_string_lit
    b .Libe_done

.Libe_sizeof:
    mov x0, x19
    bl _kcc_ir_build_sizeof
    b .Libe_done

.Libe_alignof:
    mov x0, x19
    bl _kcc_ir_build_alignof
    b .Libe_done

.Libe_types_compat:
    // __builtin_types_compatible_p folds to an
   integer constant (1/0).
    mov x0, x19
    bl _kcc_eval_const_int
    bl _kcc_ir_emit_const
    b .Libe_done

.Libe_subscript:
    mov x0, x19
    bl _kcc_ir_build_subscript
    b .Libe_done

.Libe_member:
    mov x0, x19
    bl _kcc_ir_build_member
    b .Libe_done

.Libe_zero:
    mov x0, #0

.Libe_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

340 · defined at 340, 342 · used in 30b · defines _kcc_ir_build_expr 314b, 322, 325, 326, 342, 350, 355, 369, 376, 378b, 382, 387, 393, 400c, 402b, 405, 407, 417b, 421c, 423ab, 424, 426 · uses _kcc_eval_const_int 426, _kcc_ir_build_alignof 347, _kcc_ir_build_assign 387, _kcc_ir_build_binary 369, _kcc_ir_build_call 393, _kcc_ir_build_cast 402b, _kcc_ir_build_compound_lit 413, _kcc_ir_build_cond_expr 400c, _kcc_ir_build_float_lit 343, _kcc_ir_build_ident 367, _kcc_ir_build_int_lit 343, _kcc_ir_build_member 357, _kcc_ir_build_sizeof 347, _kcc_ir_build_stmt_expr 342, _kcc_ir_build_string_lit 400b, _kcc_ir_build_subscript 353a, _kcc_ir_build_unary 378b, _kcc_ir_build_va_arg 398, _kcc_ir_store_width 349b,

_kcc_sema_resolve_type 259b

29.9.1. Statement Expression

The GNU statement expression `({ stmt …})` evaluates a compound block and takes as its value that of the block's final expression-statement. We build every item in order — the same walk that `.Libs_compound` performs, bracketed by a scope enter/leave so inner declarations do not leak — but treat the *last* item specially: when it is an expression-statement its expression is built directly with `_kcc_ir_build_expr` so its result vreg becomes the value of the whole construct. A label on that tail is peeled first—its block emitted so a `goto` can reach it—so a `label: expr;` tail still yields `expr`; this is exactly the shape the kernel's `FIND_*_BIT` macros take, ending `out: sz;` after an early `goto out`. For the same reason the block's labels are pre-registered on entry, since the function-level label prescan does not descend into a statement expression. If the block is empty, or ends in a declaration or a control-flow statement, the construct is `void` and we return the null vreg. Because SSA vregs are function-global, the returned vreg remains valid after the block's names go out of scope.

⟨ *IR: build expression* 342 ⟩ + ≡

```
// -- kcc_ir_build_stmt_expr ---
// uint64_t kcc_ir_build_stmt_expr(node *se)
//   x0 = AST_STMT_EXPR (F0 = compound statement)
// Returns the value vreg of the final expression-
   statement (0 if void).
.text
.globl _kcc_ir_build_stmt_expr
.p2align 2
_kcc_ir_build_stmt_expr:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x22, #0                 // default value:
    void → null vreg
    ldr x20, [x0, #AST_F0]      // compound node
    cbz x20, .Libse_done

    bl _kcc_ir_scope_enter
    // Register the block's labels: the function-
   level prescan does not descend
    // into a statement expression, so a goto
   inside it (the kernel FIND_*_BIT
    // macros jump to `out:') would otherwise not
   resolve to the label's block.
    mov x0, x20
    bl _kcc_ir_prescan_labels
    ldr x21, [x20, #AST_F0]     // first item in
   the block
    cbz x21, .Libse_leave       // empty block →
   void

.Libse_loop:
    ldr x19, [x21, #AST_F4]     // next sibling (
   callee-saved across the call)
    cbz x19, .Libse_last        // x21 is the
   final item
    mov x0, x21
    bl _kcc_ir_build_stmt       // ordinary non-
   value statement
    mov x21, x19
    b .Libse_loop

.Libse_last:
    // Peel off any labels on the tail---emitting
   each label's block so a goto
    // can still reach it---so a `label: expr;'
   tail yields its expression.
.Libse_peel:
    ldr x0, [x21, #AST_TAG]
    cmp x0, #AST_LABEL_STMT
    b.ne .Libse_last_check
    ldr x0, [x21, #AST_F0]      // label name
    bl _kcc_ir_label_bb
    mov x19, x0                 // the label's
   block
    cbz x19, .Libse_peel_next
    mov x0, x19
    bl _kcc_ir_branch_to
    mov x0, x19
    bl _kcc_ir_set_bb
.Libse_peel_next:
    ldr x21, [x21, #AST_F1]     // descend to the
    labeled statement
    cbz x21, .Libse_leave       // empty labeled
   body → void
    b .Libse_peel
.Libse_last_check:
    ldr x0, [x21, #AST_TAG]
    cmp x0, #AST_EXPR_STMT
    b.ne .Libse_last_stmt
    // Value-producing tail: build the expression
   itself, keep its vreg.
    ldr x0, [x21, #AST_F0]      // the expression
    bl _kcc_ir_build_expr
    mov x22, x0
    b .Libse_leave
.Libse_last_stmt:
    mov x0, x21
    bl _kcc_ir_build_stmt       // non-expression
    tail → void

.Libse_leave:
    bl _kcc_ir_scope_leave
.Libse_done:
    mov x0, x22
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

342 · defined at 340, 342 · used in 30b · defines _kcc_ir_build_stmt_expr 340 · uses _kcc_ir_branch_to 321, _kcc_ir_build_expr 340, _kcc_ir_build_stmt 322, _kcc_ir_label_bb 321, _kcc_ir_prescan_labels 321, _kcc_ir_set_bb 305b

29.9.2. Integer Literal

An integer or character literal becomes an `IR_CONST` instruction whose `SRC1` field stores the immediate value. The instruction's type is the one sema assigned to the node (`AST_F3`), so a suffixed constant such as `3L` is materialised as a `long` rather than truncated to `i32`; synthetic literal nodes that sema never typed fall back to `int`. The builder allocates a fresh vreg for the result. Character literals receive the same treatment because the parser stores the character's numeric value in `AST_F0`.

⟨ *IR: build int literal* 343 ⟩ ≡

```
// -- kcc_ir_build_int_lit ---
// uint64_t kcc_ir_build_int_lit(node *lit)
//   x0 = AST_INT_LIT or AST_CHAR_LIT (F0=value)
// Returns vreg.
.text
.globl _kcc_ir_build_int_lit
.p2align 2
_kcc_ir_build_int_lit:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                    // x19 = AST node
    (KEEP)
    ldr x20, [x19, #AST_F0]      // x20 = value

    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x21, x0                    // x21 = instr

    // Type from sema (AST_F3): honours the
   literal's suffix (long/unsigned)
    // so the constant is materialised at its true
    width.  Fall back to int
    // for synthetic literal nodes sema never
   typed (enum folds, array lengths).
    ldr x0, [x19, #AST_F3]
    cmp x0, #0x1000
    b.hs .Libil_havety
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr x0, [x0]
.Libil_havety:
    str x0, [x21, #IR_TYPE]

    str x20, [x21, #IR_SRC1]     // immediate value

    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x20, x0                    // save vreg

    mov x0, x21
    bl _kcc_ir_emit

    mov x0, x20                    // return vreg

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_build_float_lit ---
// uint64_t kcc_ir_build_float_lit(node *float_lit
   )
//   x0 = AST_FLOAT_LIT (F0 = IEEE-754 double bit
   pattern)
// A floating constant lowers to the same IR_CONST
    as an integer: the
// 64-bit pattern is materialised into the vreg's
   slot. Only the type
// differs (double), which is what makes the
   backend spill and reload it
// through the floating-point unit.
.text
.globl _kcc_ir_build_float_lit
.p2align 2
_kcc_ir_build_float_lit:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0
    ldr x20, [x19, #AST_F0]      // x20 = double
   bit pattern

    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x21, x0

    adrp x0, _kcc_type_double@PAGE
    add  x0, x0, _kcc_type_double@PAGEOFF
    ldr x0, [x0]
    str x0, [x21, #IR_TYPE]      // double

    str x20, [x21, #IR_SRC1]     // bit pattern as
   the immediate

    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x20, x0

    mov x0, x21
    bl _kcc_ir_emit

    mov x0, x20

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_type_is_fp ---
// uint64_t kcc_ir_type_is_fp(type_desc *ty)  -> 1
   if float/double, else 0.
.text
.globl _kcc_ir_type_is_fp
.p2align 2
_kcc_ir_type_is_fp:
    cbz x0, .Litif_no
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_FLOAT
    b.eq .Litif_yes
    cmp x1, #TY_DOUBLE
    b.eq .Litif_yes
.Litif_no:
    mov x0, #0
    ret
.Litif_yes:
    mov x0, #1
    ret

// -- kcc_ir_classify_type ---
// uint64_t kcc_ir_classify_type(type_desc *ty)
//   AAPCS64 argument class:
//     0 = GP    (integer/pointer, one general
   register)
//     1 = FP    (float/double, one SIMD register)
//     2 = ST1   (aggregate <=8 bytes, one general
    register, by value)
//     3 = ST2   (aggregate 9..16 bytes, two
   general registers, by value)
//     4 = REF   (aggregate >16 bytes, passed by
   reference in one register)
.text
.globl _kcc_ir_classify_type
.p2align 2
_kcc_ir_classify_type:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    cbz x0, .Lct_gp
    mov x19, x0
    bl _kcc_ir_type_is_fp
    cbnz x0, .Lct_fp
    ldr x0, [x19, #TY_KIND]
```

```
    cmp x0, #TY_STRUCT
    b.eq .Lct_struct
    cmp x0, #TY_UNION
    b.eq .Lct_struct
    b .Lct_gp
.Lct_struct:
    mov x0, x19
    bl _kcc_type_layout          // x0 = byte size
    cmp x0, #8
    b.le .Lct_st1
    cmp x0, #16
    b.le .Lct_st2
    mov x0, #4
    b .Lct_ret
.Lct_st1:
    mov x0, #2
    b .Lct_ret
.Lct_st2:
    mov x0, #3
    b .Lct_ret
.Lct_fp:
    mov x0, #1
    b .Lct_ret
.Lct_gp:
    mov x0, #0
.Lct_ret:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_emit_sitofp ---
// uint64_t kcc_ir_emit_sitofp(uint64_t src_vreg)
// Emits IR_SITOFP: converts a signed integer vreg
    to a double vreg.
.text
.globl _kcc_ir_emit_sitofp
.p2align 2
_kcc_ir_emit_sitofp:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                  // src vreg
    mov x0, #IR_SITOFP
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_double@PAGE
    add  x1, x1, _kcc_type_double@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    str x19, [x0, #IR_SRC1]
    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    str x0, [sp, #24]            // save dest vreg
    across the emit call
    mov x0, x19
    bl _kcc_ir_emit
    ldr x0, [sp, #24]
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_emit_uitofp ---
// uint64_t kcc_ir_emit_uitofp(uint64_t src_vreg)
// Emits IR_UITOFP: converts an UNSIGNED integer
   vreg to a double vreg.  The
// only difference from sitofp is the opcode;
   codegen picks UCVTF over SCVTF so
// a 64-bit value with its top bit set (e.g. 0
   xFFFF...FF) becomes ~1.8e19, not
// -1.
.text
.globl _kcc_ir_emit_uitofp
.p2align 2
_kcc_ir_emit_uitofp:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                  // src vreg
    mov x0, #IR_UITOFP
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_double@PAGE
    add  x1, x1, _kcc_type_double@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    str x19, [x0, #IR_SRC1]
    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    str x0, [sp, #24]
    mov x0, x19
    bl _kcc_ir_emit
    ldr x0, [sp, #24]
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_emit_int_to_fp ---
// uint64_t kcc_ir_emit_int_to_fp(uint64_t
   src_vreg, type *int_type)
// Converts an integer vreg to double, choosing
   signed (SCVTF) or unsigned
// (UCVTF) widening by the SOURCE integer's
   signedness.  A null type falls back
// to signed, preserving the historical behaviour
   for callers without a type.
.text
.globl _kcc_ir_emit_int_to_fp
.p2align 2
_kcc_ir_emit_int_to_fp:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                  // src vreg
    mov x20, x1                  // int type
    cbz x20, .Litf_signed        // no type →
   signed (default)
    mov x0, x20
    bl _kcc_sema_int_signedness  // 1 = signed, 0
   = unsigned
    cbnz x0, .Litf_signed
    mov x0, x19
    bl _kcc_ir_emit_uitofp
    b .Litf_ret
.Litf_signed:
    mov x0, x19
    bl _kcc_ir_emit_sitofp
.Litf_ret:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_emit_fpround ---
// uint64_t kcc_ir_emit_fpround(uint64_t src_vreg)
// Emits IR_FPROUND: rounds a double vreg to
   single precision (float type).
.text
.globl _kcc_ir_emit_fpround
.p2align 2
_kcc_ir_emit_fpround:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                  // src vreg
    mov x0, #IR_FPROUND
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_float@PAGE
    add  x1, x1, _kcc_type_float@PAGEOFF
    ldr x1, [x1]
```

```
    str x1, [x0, #IR_TYPE]
    str x19, [x0, #IR_SRC1]
    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    str x0, [sp, #24]
    mov x0, x19
    bl _kcc_ir_emit
    ldr x0, [sp, #24]
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_emit_fptosi ---
// uint64_t kcc_ir_emit_fptosi(uint64_t src_vreg)
// Emits IR_FPTOSI: converts a double vreg to a
   signed integer vreg.
.text
.globl _kcc_ir_emit_fptosi
.p2align 2
_kcc_ir_emit_fptosi:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0
    mov x0, #IR_FPTOSI
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    str x19, [x0, #IR_SRC1]
    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    str x0, [sp, #24]              // save dest vreg
    across the emit call
    mov x0, x19
    bl _kcc_ir_emit
    ldr x0, [sp, #24]
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_convert ---
// uint64_t kcc_ir_convert(uint64_t vreg, type *
   from, type *to)
// Inserts an  intdouble conversion when the value
   crosses the boundary
// between the integer and floating domains;
   otherwise returns the vreg
// unchanged. ( Integerinteger resizing is handled
   by cast lowering.)
.text
.globl _kcc_ir_convert
.p2align 2
_kcc_ir_convert:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                    // vreg (result
   accumulator)
    str x1, [sp, #32]              // from type
    str x2, [sp, #40]              // to type
    mov x0, x2
    bl _kcc_ir_type_is_fp
    mov x20, x0                    // to_is_fp
    ldr x0, [sp, #32]
    bl _kcc_ir_type_is_fp          // from_is_fp
    cmp x0, x20
    b.eq .Licv_maybe_round         // same domain →
   maybe just round
    cbnz x20, .Licv_to_fp
    mov x0, x19
    bl _kcc_ir_emit_fptosi         // fp → int
    mov x19, x0
    b .Licv_maybe_round
.Licv_to_fp:
    mov x0, x19
    ldr x1, [sp, #32]              // source integer
    type (for signed vs unsigned)
    bl _kcc_ir_emit_int_to_fp      // int → double (
   SCVTF or UCVTF by signedness)
    mov x19, x0
.Licv_maybe_round:
    // Converting to float truncates the value to
   single precision.
    ldr x0, [sp, #40]
    cbz x0, .Licv_done
    ldr x0, [x0, #TY_KIND]
    cmp x0, #TY_FLOAT
    b.ne .Licv_maybe_bool
    mov x0, x19
    bl _kcc_ir_emit_fpround
    mov x19, x0
.Licv_maybe_bool:
    // Converting to _Bool normalises any non-zero
    value to 1 (\stdref{6.3.1.2}).
    ldr x0, [sp, #40]
    cbz x0, .Licv_done
    ldr x0, [x0, #TY_KIND]
    cmp x0, #TY_BOOL
    b.ne .Licv_int_resize
    mov x0, x19
    bl _kcc_ir_emit_bool_norm
    mov x19, x0
    b .Licv_done

.Licv_int_resize:
    // An integer-to-integer implicit conversion
   must re-canonicalise the value
    // to the target width and signedness, exactly
    as an explicit cast does.
    // Convert used to no-op here and rely on the
   register invariant, but the
    // \emph{value} an assignment yields is this
   converted result: a chained
    // `ul = us = l2 = -1` has to narrow -1 to `
   unsigned short` (0xFFFF) before
    // widening it to `ul`, not store the raw -1
   into all three lvalues.  The
    // narrowing is the load-bearing step (ZEXT/
   SEXT to the target width masks
    // the high bytes); a widening from an already
   -canonical source is a no-op.
    ldr x1, [sp, #40]              // to type
    cbz x1, .Licv_done
    ldr x0, [x1, #TY_KIND]
    cmp x0, #TY_BOOL               // 2
    b.lo .Licv_done                // void/invalid
   target → leave as-is
    cmp x0, #TY_LLONG              // 7
    b.hi .Licv_done                // float/pointer/
   aggregate target → not this path
    ldr x2, [sp, #32]              // from type
    cbz x2, .Licv_done
    ldr x0, [x2, #TY_KIND]
    cmp x0, #TY_BOOL
    b.lo .Licv_done
    cmp x0, #TY_LLONG
    b.hi .Licv_done                // from not an
   integer → leave as-is
    ldr x3, [x1, #TY_SIZE]         // to size
    ldr x4, [x2, #TY_SIZE]         // from size
    cmp x3, x4
    b.ne .Licv_ir_emit             // different width
    → resize
```

```
    cmp x3, #8
    b.hs .Licv_done             // same width,
   full 8 bytes → nothing to do
    ldr x5, [x1, #TY_FO]        // to signedness
    ldr x6, [x2, #TY_FO]        // from signedness
    cmp x5, x6
    b.eq .Licv_done             // same width and
   signedness → nothing to do
.Licv_ir_emit:
    ldr x0, [x1, #TY_FO]        // target
   signedness (1 = signed → SEXT)
    mov x20, #IR_SEXT
    cbnz x0, .Licv_ir_have_op
    mov x20, #IR_ZEXT
.Licv_ir_have_op:
    mov x0, x20
    bl _kcc_ir_alloc_instr
    mov x20, x0
    ldr x1, [sp, #40]           // to type
    str x1, [x20, #IR_TYPE]
    str x19, [x20, #IR_SRC1]    // source vreg
    ldr x0, [x1, #TY_SIZE]      // target width in
    bytes
    str x0, [x20, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x19, x0
    mov x0, x20
    bl _kcc_ir_emit

.Licv_done:
    mov x0, x19
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_fill_param_classes ---
// void kcc_ir_fill_param_classes(ir_func *fn,
   node *func_decl)
//   Allocates IRF_PARAM_CLS: one AAPCS64 class
   per incoming parameter, in
//   order, so the codegen prologue can spill each
    from the right register.
//   Named source parameters are classified from
   their type; a trailing
//   hidden sret pointer (when the return type is
   a struct) stays GP.
.text
.globl _kcc_ir_fill_param_classes
.p2align 2
_kcc_ir_fill_param_classes:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                 // ir_func
    mov x23, x1                 // func_decl
    ldr x20, [x19, #IRF_PARAMS]
    cbz x20, .Lfpc_done
    lsl x0, x20, #3
    bl _kcc_arena_alloc
    str x0, [x19, #IRF_PARAM_CLS]
    mov x22, x0                 // class array
    // Default every slot to GP (arena is not zero
   -initialised).
    mov x21, #0
.Lfpc_zero:
    cmp x21, x20
    b.ge .Lfpc_walk
    str xzr, [x22, x21, lsl #3]
    add x21, x21, #1
    b .Lfpc_zero
.Lfpc_walk:
    ldr x24, [x23, #AST_F2]     // first param
   decl (bit 0=variadic, bit 1=static)
    bic x24, x24, #3
    mov x21, #0                 // named-param
   index
.Lfpc_loop:
    cbz x24, .Lfpc_sret
    ldr x0, [x24, #AST_TAG]
    cmp x0, #AST_PARAM_DECL
    b.ne .Lfpc_next
    ldr x0, [x24, #AST_F1]      // name (skip the
    (void) marker)
    cbz x0, .Lfpc_next
    ldr x0, [x24, #AST_F0]      // param type
   spec
    bl _kcc_sema_resolve_type
    bl _kcc_type_param_adjust    // array/
   function param → pointer (\stdref{6.7.6.3})
    bl _kcc_ir_classify_type
    str x0, [x22, x21, lsl #3]
    add x21, x21, #1
.Lfpc_next:
    ldr x24, [x24, #AST_F4]
    b .Lfpc_loop
.Lfpc_sret:
    // A >16B struct return adds a trailing hidden
    parameter received in x8
    // (class 5), not in a general argument
   register.
    ldr x0, [x19, #IRF_RETTYPE]
    bl _kcc_ir_classify_type
    cmp x0, #4
    b.ne .Lfpc_done
    sub x0, x20, #1             // last param
   index
    mov x1, #5                  // class X8
    str x1, [x22, x0, lsl #3]
.Lfpc_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

343 · used in 30b · defines _kcc_ir_build_float_lit 340, _kcc_ir_build_int_lit 340, _kcc_ir_classify_type 317, 326, 393, _kcc_ir_convert 314b, 326, 359, 384, 387, 393, 402b, 407, 417b, _kcc_ir_emit_fpround 369, _kcc_ir_emit_fptosi never used, _kcc_ir_emit_int_to_fp 369, _kcc_ir_emit_sitofp never used, _kcc_ir_emit_uitofp never used, _kcc_ir_fill_param_classes 317, _kcc_ir_type_is_fp 359, 369, 402b, 554 · uses _kcc_arena_alloc 58a, _kcc_ir_alloc_instr 303a, _kcc_ir_emit 304, _kcc_ir_next_vreg 305a, _kcc_sema_resolve_type 259b, _kcc_type_double 250b, _kcc_type_float 250b, _kcc_type_int_s 250b, _kcc_type_layout 267, 268, _kcc_type_param_adjust 242b

29.9.3. Sizeof Expression

The `sizeof` operator (§6.5.3.4) is a compile-time constant: it measures the size of a type without generating any run-time code. The IR builder lowers it to a single `IR_CONST` whose immediate value is the type's size in bytes.

Two forms are possible. When the operand is a type name (`AST_F1 = 1`), the builder calls `_kcc_sema_resolve_type` on the type AST node to obtain a type descriptor and reads its `TY_SIZE` field. When the operand

is an expression (AST_F1 = 0), the semantic pass has already type-checked the operand and stored its resolved type in the operand node's AST_F3 field; the builder reads the size from that descriptor. In neither case is the operand expression evaluated at run time.

⟨ *IR: build sizeof* 347 ⟩ ≡

```
// -- kcc_ir_build_sizeof ---
// uint64_t kcc_ir_build_sizeof(node *expr)
//   x0 = AST_SIZEOF_EXPR (F0=operand, F1=is_type)
// Returns vreg holding the size constant.
.text
.globl _kcc_ir_build_sizeof
.p2align 2
_kcc_ir_build_sizeof:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                  // x19 = sizeof node

    // Determine operand type
    ldr x0, [x19, #AST_F1]       // is_type flag
    cbnz x0, .Libs_sizeof_type

    // sizeof(expr): operand's resolved type is in F0->AST_F3
    ldr x0, [x19, #AST_F0]       // operand expression
    ldr x0, [x0, #AST_F3]        // resolved type descriptor
    b .Libs_sizeof_emit

.Libs_sizeof_type:
    // sizeof(type-name): resolve the type AST node
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type

.Libs_sizeof_emit:
    // x0 = type descriptor.  Lay it out first: an aggregate whose size is
    // computed lazily reports zero until kcc_type_layout runs, and nothing
    // else triggers that when the operand is a bare type name with no object
    // of it declared (e.g. sizeof(TValue) in a header).  Layout is idempotent.
    bl _kcc_type_layout          // returns TY_SIZE, populating it on the way
    mov x20, x0                  // x20 = size in bytes

    // Emit IR_CONST with the size value
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x21, x0                  // x21 = instr

    adrp x0, _kcc_type_long_u@PAGE
    add  x0, x0, _kcc_type_long_u@PAGEOFF
    ldr x0, [x0]
    str x0, [x21, #IR_TYPE]      // sizeof yields unsigned long

    str x20, [x21, #IR_SRC1]     // immediate = size

    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x20, x0                  // save vreg

    mov x0, x21
    bl _kcc_ir_emit

    mov x0, x20                  // return vreg

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_build_alignof ---
// uint64_t kcc_ir_build_alignof(node *expr)
//   x0 = AST_ALIGNOF_EXPR (F0 = type node)
// The _Alignof operator (\stdref{6.5.3.4}) mirrors sizeof but yields the
// type's alignment: it resolves the type and emits an IR_CONST of its
// TY_ALIGN, typed unsigned long (size_t).
.text
.globl _kcc_ir_build_alignof
.p2align 2
_kcc_ir_build_alignof:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                  // x19 = alignof node
    ldr x0, [x19, #AST_F0]       // type node
    bl _kcc_sema_resolve_type
    mov x20, x0                  // save the descriptor
    bl _kcc_type_layout          // lay it out so TY_ALIGN is populated (idempotent)
    ldr x20, [x20, #TY_ALIGN]    // x20 = alignment in bytes

    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x21, x0

    adrp x0, _kcc_type_long_u@PAGE
    add  x0, x0, _kcc_type_long_u@PAGEOFF
    ldr x0, [x0]
    str x0, [x21, #IR_TYPE]      // _Alignof yields unsigned long

    str x20, [x21, #IR_SRC1]     // immediate = alignment

    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x20, x0

    mov x0, x21
    bl _kcc_ir_emit

    mov x0, x20
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

347 · used in 30b · defines _kcc_ir_build_alignof 340, _kcc_ir_build_sizeof 340 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_emit 304, _kcc_ir_next_vreg 305a, _kcc_sema_resolve_type 259b, _kcc_type_layout 267, 268, _kcc_type_long_u 250b

End-to-end tests. The sizeof operator must produce the correct size for each fundamental type: `char` is 1 byte, `short` is 2, `int` is 4, and `long` is 8 on ARM64. The expression form `sizeof expr` must also return the type's size without evaluating the expression.

⟨ *Test: codegen sizeof* 348 ⟩ ≡

```
echo ""
echo "--- Sizeof ---"

run_test "sizeof-int" \
    "int main(void) { return sizeof(int); }" 4

run_test "sizeof-char" \
    "int main(void) { return sizeof(char); }" 1

run_test "sizeof-long" \
    "int main(void) { return sizeof(long); }" 8

run_test "sizeof-expr" \
    "int main(void) { int x = 42; return sizeof(x)
   ; }" 4

run_test "sizeof-ptr" \
    "int main(void) { int *p; return sizeof(p); }"
    8

run_test "sizeof-arith" \
    "int main(void) { return sizeof(int) + sizeof(
   char); }" 5
```

□

348 · used in 33b

29.9.4. Array Subscript

The subscript operator `e[i]` has two distinct lowerings depending on the declared type of the base operand.

Array subscript. When the base is a local array (`int a[N]`), the alloca vreg *is* the array base address. The builder emits an `IR_GEP` that computes `base + index` × 8 (the all-spill slot stride), and an `IR_LOAD` reads the value at that address.

Pointer subscript. When the base is a pointer variable (`int *p`), the alloca holds the pointer value, which must be loaded first. The builder emits `IR_LOAD` to dereference the alloca, scales the index by the pointee's `TY_SIZE` (via `IR_CONST` + `IR_MUL`), and emits `IR_ADD` to compute the element address. The resulting `LOAD`/`STORE` carries an access-size override so the code generator emits the correct width instruction (LDRB/STRB for `char`, LDR/STR `w` for `int`).

Global pointer variables (and any base expression that is not a simple local identifier) also take the pointer subscript path. The fallback branch redirects to `.Libsa_ptr`, ensuring that the index is scaled by `TY_SIZE` rather than the fixed slot stride of 8. Without this, `vars[i]` on a global `struct Var *vars` would compute `base + i×8` instead of `base + i×16` for a two-field struct.

The local array/pointer distinction relies on a tag bit in the locals hash table. Sema applies array-to-pointer decay (§6.3.2.1p3), so both arrays and pointer variables have `TY_PTR` in `AST_F3`. Furthermore, the symbol table's local scope is popped after sema completes. To communicate the original declared type to the IR builder, `_kcc_ir_build_var_decl` sets bit 63 of the hash value when the resolved type is `TY_PTR`. The subscript address builder tests this bit with TBNZ; all other hash-value consumers clear it with BIC.

Two helpers split the work. `_kcc_ir_build_subscript_addr` resolves the address—it is shared by both read (`e[i]`) and write (`e[i]=v`) paths. `_kcc_ir_build_subscript` calls the address helper and then emits an `IR_LOAD` to produce a value vreg. For assignment, the assign builder calls the address helper directly and emits an `IR_STORE` to write the right-hand side.

For arrays of structs, the element size exceeds a single eight-byte slot. In that case the builder pre-scales the index by the ratio `TY_SIZE / 8` before the `IR_GEP`, so that the fixed stride 8 of the GEP instruction still produces the correct byte offset.

Global arrays require special treatment: the address produced by `IR_GLOBAL_REF` already points at element 0, so subscripting must *not* load from it (that would misinterpret element data as a pointer). The builder detects global arrays by looking up the symbol table for the pre-decay type (`TY_ARRAY`), emits a `GLOBAL_REF`, and feeds the resulting address vreg into the GEP index path. A `static` local array is handled the same way, but its block scope is gone by code-generation time, so the base is recognised through the `_kcc_ir_statics`/`_kcc_ir_static_types` tables instead of the symbol table: if the subscript base names a static of array type, the builder emits a `GLOBAL_REF` for its *mangled* symbol and feeds that address into the same GEP path.

Slots per type. Aggregates are laid out byte-accurately (see `_kcc_type_layout`), but the all-spill frame still allocates storage in whole eight-byte slots. `_kcc_ir_type_slots` bridges the two: it forces a layout and then rounds the real byte size up to a whole number of slots, $\max(1, \lceil \text{TY_SIZE}/8 \rceil)$. For `int a[4]` that is two slots (sixteen bytes) rather than the four the old one-slot-per-element model used; the packed sixteen-byte object is addressed with byte-accurate strides by the subscript and member code below. This figure sizes an alloca; the subscript's element step now comes from the element's real size, not from a slot count.

⟨ *IR: build subscript* 349a ⟩ ≡

```
// -- kcc_ir_type_slots ---
// uint64_t kcc_ir_type_slots(type_desc *ty)
//   Number of 8-byte all-spill slots a value of
   this type occupies.
.text
.globl _kcc_ir_type_slots
.p2align 2
_kcc_ir_type_slots:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    cbz x0, .Lts_one
    // Force a byte-accurate layout, then round
   the real size up to whole
    // 8-byte all-spill slots.  Arrays and structs
    are sized uniformly.
```

```
    bl _kcc_type_layout          // x0 = TY_SIZE
   in bytes
    cmp x0, #8
    b.le .Lts_one
    add x0, x0, #7
    lsr x0, x0, #3               // ceil(size / 8)
    b .Lts_done
.Lts_one:
    mov x0, #1
.Lts_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

349a · defined at 349a, 350, 353a · used in 30b · defines _kcc_ir_type_slots 320, 393, 417b · uses _kcc_type_layout 267, 268

Access widths. Once fields and array elements are packed, a memory access must name its exact width and signedness or it will read or clobber neighbouring bytes. An eight-byte slot load of a two-byte `short` field would drag in the next field's low bytes; an eight-byte store would overwrite them. Two helpers translate a scalar type into the access-size override the code generator understands. `_kcc_ir_store_width` returns the byte width 1, 2, or 4 for a narrow integer (and 0—meaning the natural eight-byte slot—for anything pointer-width or aggregate). Stores need no signedness because they simply truncate. `_kcc_ir_load_width` additionally encodes signedness, because a narrow load must extend into the full 64-bit slot: it returns $1/2/4$ for unsigned `LDRB`/`LDRH`/`LDR w` and $5/6/7$ for signed `LDRSB`/`LDRSH`/`LDRSW`. A signed `int` field holding -1 thus round-trips as -1 rather than $2^{32}-1$.

⟨ *IR: access width* 349b ⟩ ≡

```
// -- kcc_ir_store_width ---
// uint64_t kcc_ir_store_width(type_desc *ty)
//   Store access-size override: 1/2/4 for a
   narrow scalar, 0 for the
//   natural 8-byte slot (pointers, long,
   aggregates).
.text
.globl _kcc_ir_store_width
.p2align 2
_kcc_ir_store_width:
    cbz x0, .Lsw_default
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_FLOAT            // single
   precision: 4-byte FP store (code 8)
    b.eq .Lsw_float
    cmp x1, #TY_BOOL
    b.eq .Lsw_one
    cmp x1, #TY_ENUM
    b.eq .Lsw_int_size
    cmp x1, #TY_CHAR
    b.lt .Lsw_default            // void
    cmp x1, #TY_LLONG
    b.gt .Lsw_default            // float/double/ptr
   /array/struct
.Lsw_int_size:
    ldr x2, [x0, #TY_SIZE]
    cmp x2, #4
    b.eq .Lsw_from_size
    cmp x2, #2
    b.eq .Lsw_from_size
    cmp x2, #1
    b.eq .Lsw_from_size
    b .Lsw_default               // 8-byte long →
   natural slot
.Lsw_from_size:
    mov x0, x2
    ret
.Lsw_one:
    mov x0, #1
    ret
.Lsw_float:
    mov x0, #8
    ret
.Lsw_default:
    mov x0, #0
    ret

// -- kcc_ir_load_width ---
// uint64_t kcc_ir_load_width(type_desc *ty)
//   Load access-size override: 1/2/4 unsigned,
   5/6/7 signed, 0 = 8-byte.
.text
.globl _kcc_ir_load_width
.p2align 2
_kcc_ir_load_width:
    cbz x0, .Llw_default
    ldr x1, [x0, #TY_KIND]
    ldr x2, [x0, #TY_SIZE]
    cmp x1, #TY_FLOAT            // single
   precision: 4-byte FP load (code 8)
    b.eq .Llw_float
    cmp x1, #TY_BOOL
    b.eq .Llw_unsigned
    cmp x1, #TY_ENUM
    b.eq .Llw_signed
    cmp x1, #TY_CHAR
    b.lt .Llw_default            // void
    cmp x1, #TY_LLONG
    b.gt .Llw_default            // float/double/ptr
   /array/struct
    ldr x3, [x0, #TY_FO]         // integer
   signedness (1 = signed)
    cbz x3, .Llw_unsigned
.Llw_signed:
    cmp x2, #1
    b.eq .Llw_s8
    cmp x2, #2
    b.eq .Llw_s16
    cmp x2, #4
    b.eq .Llw_s32
    b .Llw_default               // 8-byte → natural
    slot
.Llw_s8:
    mov x0, #5
    ret
.Llw_s16:
    mov x0, #6
    ret
.Llw_s32:
    mov x0, #7
    ret
.Llw_unsigned:
    cmp x2, #1
    b.eq .Llw_u8
    cmp x2, #2
    b.eq .Llw_u16
    cmp x2, #4
    b.eq .Llw_u32
    b .Llw_default
.Llw_u8:
    mov x0, #1
    ret
.Llw_u16:
    mov x0, #2
    ret
.Llw_u32:
    mov x0, #4
    ret
```

```
.Llw_float:
    mov x0, #8
    ret
.Llw_default:
    mov x0, #0
    ret
```

□

349b · used in 30b · defines _kcc_ir_load_width 353a, 357, 359, 367, 378b, 384, 387, 518, _kcc_ir_store_width 314b, 320, 340, 359, 384, 387, 407, 417b

⟨ IR: build subscript 350 ⟩ + ≡

```
// -- kcc_ir_build_subscript_addr ---
// uint64_t kcc_ir_build_subscript_addr(node *
   subscript)
//   x0 = AST_SUBSCRIPT (F0=array ident, F1=index
   expr)
// Returns vreg holding the element address (GEP
   result).
.text
.globl _kcc_ir_build_subscript_addr
.p2align 2
_kcc_ir_build_subscript_addr:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    mov x19, x0                    // x19 =
   subscript node

    // Get base expression - check its shape
    ldr x0, [x19, #AST_F0]       // array/pointer
   expression
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_SUBSCRIPT
    b.eq .Libsa_nested_base
    cmp x1, #AST_MEMBER
    b.eq .Libsa_member_base
    cmp x1, #AST_IDENT_EXPR
    b.ne .Libsa_expr

    // Ident: look up in locals
    ldr x20, [x0, #AST_F0]       // name ptr
    mov x0, x20
    bl _kcc_strlen
    mov x2, x0
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x20
    bl _kcc_ht_get
    cbz x0, .Libsa_expr

    // Bit 63 of the hash value tags pointer
   variables (set by
    // _kcc_ir_build_var_decl).  Pointer
   subscripts need load +
    // scaled ADD; arrays use GEP with the alloca
   directly.
    tbnz x0, #63, .Libsa_ptr_local
    sub x20, x0, #1                // x20 = alloca
   vreg (array/scalar)
    b .Libsa_index

.Libsa_ptr_local:
    b .Libsa_ptr

.Libsa_nested_base:
    // The base is itself a subscript, `X[i
   ][...]`.  Two very different shapes
    // share this syntax: a multi-dimensional
   array, where `X[i]` is a
    // contiguous sub-array whose ADDRESS is the
   base; and an array/pointer of
    // pointers, where `X[i]` is a pointer VALUE
   that must be loaded before the
    // second index (`char **argv; argv[1][0]`).
   Distinguish on the inner
    // subscript's resolved type: an array decays
   to its address, a pointer is
    // loaded.  Getting this wrong made `argv[i][j
   ]` read a char through the
    // sub-array address instead of through the
   stored pointer.
    ldr x0, [x19, #AST_F0]       // inner subscript
    node
    ldr x1, [x0, #AST_F3]        // its resolved
   type
    cbz x1, .Libsa_nested_addr
    ldr x2, [x1, #TY_KIND]
    cmp x2, #TY_ARRAY
    b.ne .Libsa_nested_load
.Libsa_nested_addr:
    bl _kcc_ir_build_subscript_addr   // sub-array
   : its address is the base
    mov x20, x0
    b .Libsa_index
.Libsa_nested_load:
    bl _kcc_ir_build_expr              // pointer
   element: load the pointer value
    mov x20, x0
    b .Libsa_index

.Libsa_member_base:
    // base is s.arr / s->arr.  When the member is
    itself an array, its
    // ADDRESS is the array base; a pointer member
    is loaded like any pointer.
    ldr x1, [x0, #AST_F3]        // member's
   resolved type
    cbz x1, .Libsa_ptr
    ldr x2, [x1, #TY_KIND]
    cmp x2, #TY_ARRAY
    b.ne .Libsa_ptr
    bl _kcc_ir_build_member_addr // x0 = base
   member node → member address
    mov x20, x0
    b .Libsa_index

.Libsa_expr:
    // Check if base is a global array (needs
   global_ref + GEP,
    // not global_ref + load + ADD like pointer
   globals).
    ldr x0, [x19, #AST_F0]       // base expression
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_IDENT_EXPR
    b.ne .Libsa_ptr

    ldr x22, [x0, #AST_F0]       // name ptr (
   source name)

    // A static local array behaves like a file-
   scope global array here, but the
    // block scope is popped so the symbol table
   cannot see it.  Consult the
    // static tables: if the name is a static
   local of array type, use its
    // mangled global address as the GEP base (no
   load).
    adrp x0, _kcc_ir_statics@PAGE
    add  x0, x0, _kcc_ir_statics@PAGEOFF
```

```
    ldr x0, [x0]
    cbz x0, .Libsa_expr_symtab
    mov x0, x22
    bl _kcc_strlen
    mov x23, x0                     // namelen
    adrp x0, _kcc_ir_statics@PAGE
    add  x0, x0, _kcc_ir_statics@PAGEOFF
    ldr x0, [x0]
    mov x1, x22
    mov x2, x23
    bl _kcc_ht_get
    cbz x0, .Libsa_expr_symtab    // not a static
→ symtab-based detection
    mov x20, x0                     // x20 = mangled
name
    adrp x0, _kcc_ir_static_types@PAGE
    add  x0, x0, _kcc_ir_static_types@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Libsa_ptr
    mov x1, x22
    mov x2, x23
    bl _kcc_ht_get
    cbz x0, .Libsa_ptr
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.ne .Libsa_ptr              // static pointer/
scalar subscript → ptr path
    // Static array: GLOBAL_REF(mangled) IS the
element-0 address; feed the GEP.
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x20, [x0, #IR_SRC1]      // mangled name
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x23, x0
    bl _kcc_ir_next_vreg
    str x0, [x23, #IR_DEST]
    mov x20, x0                     // x20 = base
address vreg
    mov x0, x23
    bl _kcc_ir_emit
    b .Libsa_index

.Libsa_expr_symtab:
    // Look up in symbol table for pre-decay type
    mov x0, x22
    bl _kcc_strlen
    mov x1, x0
    mov x0, x22
    bl _kcc_symtab_lookup
    cbz x0, .Libsa_ptr
    ldr x1, [x0, #SYM_TYPE]
    cbz x1, .Libsa_ptr
    ldr x2, [x1, #TY_KIND]
    cmp x2, #TY_ARRAY
    b.ne .Libsa_ptr

    // Global array: emit IR_GLOBAL_REF for the
base address
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x22, [x0, #IR_SRC1]      // name
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x23, x0
    bl _kcc_ir_next_vreg
    str x0, [x23, #IR_DEST]
    mov x20, x0                     // x20 = global
address vreg
    mov x0, x23
    bl _kcc_ir_emit
    b .Libsa_index

.Libsa_index:
    // Build index expression
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_expr
    mov x21, x0                      // index vreg

    // Scale the index by the element's real byte
size so the stride-1 GEP
    // lands on the right byte (a[i] steps by
sizeof(element), including
    // packed structs and sub-arrays).
    ldr x0, [x19, #AST_F3]       // subscript
result type (element type)
    cbz x0, .Libsa_gep
    bl _kcc_type_layout          // x0 = element
size in bytes
    mov x22, x0
    cmp x22, #1
    b.le .Libsa_gep              // size 1: index
is already the byte offset

    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x23, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x23, #IR_TYPE]
    str x22, [x23, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x23, #IR_DEST]
    mov x22, x0
    mov x0, x23
    bl _kcc_ir_emit

    mov x0, #IR_MUL
    bl _kcc_ir_alloc_instr
    mov x23, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x23, #IR_TYPE]
    str x21, [x23, #IR_SRC1]
    str x22, [x23, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x23, #IR_DEST]
    mov x21, x0
    mov x0, x23
    bl _kcc_ir_emit

.Libsa_gep:
    // Emit IR_GEP: base + byte offset (index
already scaled by element size)
    mov x0, #IR_GEP
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x20, [x22, #IR_SRC1]     // base pointer
    str x21, [x22, #IR_SRC2]     // byte offset
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x23, x0                      // result vreg
    mov x0, x22
    bl _kcc_ir_emit

    mov x0, x23                      // return GEP
vreg
    b .Libsa_done
```

```
.Libsa_ptr:
    // Pointer subscript: load pointer, scale
   index, emit ADD
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_build_expr
    mov x20, x0                    // x20 = loaded
   pointer vreg

    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_expr
    mov x21, x0                    // x21 = index
   vreg

    // Get pointee size (real, byte-accurate - the
    array it points into is
    // now packed at the same stride, so no clamp
   to 8 is needed).
    ldr x0, [x19, #AST_F0]
    ldr x0, [x0, #AST_F3]          // TY_PTR
    ldr x0, [x0, #TY_F0]           // pointee type
    cbz x0, .Libsa_ptr_add
    bl _kcc_type_layout            // x22 = sizeof(
   pointee)
    mov x22, x0
    cmp x22, #1
    b.le .Libsa_ptr_add            // size 1: index
   is already the byte offset

    // Scale index by pointee size
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x23, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x23, #IR_TYPE]
    str x22, [x23, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x23, #IR_DEST]
    mov x22, x0
    mov x0, x23
    bl _kcc_ir_emit

    mov x0, #IR_MUL
    bl _kcc_ir_alloc_instr
    mov x23, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x23, #IR_TYPE]
    str x21, [x23, #IR_SRC1]
    str x22, [x23, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x23, #IR_DEST]
    mov x21, x0
    mov x0, x23
    bl _kcc_ir_emit

.Libsa_ptr_add:
    mov x0, #IR_ADD
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x20, [x22, #IR_SRC1]
    str x21, [x22, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x23, x0
    mov x0, x22
    bl _kcc_ir_emit
    mov x0, x23

.Libsa_done:
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

350 · defined at 349a, 350, 353a · used in 30b · defines _kcc_ir_build_subscript_addr 353a, 355, 378b, 382, 387, 398, 405 · uses _kcc_ht_get 62, _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_build_member_addr 355, _kcc_ir_build_var_decl 417b, _kcc_ir_emit 304, _kcc_ir_locals 305c, _kcc_ir_next_vreg 305a, _kcc_ir_statics 305c, _kcc_ir_static_types 305c, _kcc_symtab_lookup 255, _kcc_type_int_s 250b, _kcc_type_layout 267, 268

Reading a subscript is the element address followed by a load—except when the element is itself an *array* (or function). In a multi-dimensional array a partial subscript, such as `g->cache[i]` where `cache` has type `T[N][M]`, denotes the i-th *row*: an array `T[M]` that decays to a pointer to its first element (§6.3.2.1) rather than being loaded. The full subscript `g->cache[i][j]` never trips on this—it lowers through `_kcc_ir_build_subscript_addr`, which chains the two index computations without an intervening load—but a partial subscript used as a value must return the row *address*. Loading it instead reads the first element as if it were a pointer and dereferences that: Lua's `luaS_init` fills `g->strcache[i][j]` by first taking the row `g->strcache[i]`, so a load there scribbled the cache with garbage and `luaS_new` later dereferenced a null slot.

⟨ *IR: build subscript* 353a ⟩ +≡

```
// -- kcc_ir_build_subscript ---
// uint64_t kcc_ir_build_subscript(node *subscript
   )
//   x0 = AST_SUBSCRIPT (F0=array, F1=index)
// Returns vreg holding the loaded value (or the
   element address for an
// array/function element, which decays rather
   than loading).
.text
.globl _kcc_ir_build_subscript
.p2align 2
_kcc_ir_build_subscript:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x20, x0                    // x20 =
   subscript node

    // Get element address
    bl _kcc_ir_build_subscript_addr
    mov x19, x0                    // address vreg

    // Array/function element: decays to its
   address, no load.  A partial
    // subscript of a multi-dim array (`g->cache[i
   ]` → row `T[M]`) is this case.
    ldr x1, [x20, #AST_F3]         // element type
```

```
    cmp x1, #0x1000
    b.lo .Libs_scalar
    ldr x2, [x1, #TY_KIND]
    cmp x2, #TY_ARRAY
    b.eq .Libs_decay
    cmp x2, #TY_FUNC
    b.ne .Libs_scalar
.Libs_decay:
    mov x0, x19                    // element
   address (array/function decay)
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Libs_scalar:
    // Element load width+sign from the subscript
   result (element) type -
    // covers both array and pointer element sizes
    now that they are packed.
    ldr x0, [x20, #AST_F3]
    bl _kcc_ir_load_width
    mov x21, x0
    // Load type: the element's (volatile-
   qualified) descriptor when volatile,
    // else the canonical int.  Held in x22 across
    the alloc.
    ldr x22, [x20, #AST_F3]      // element type
    cmp x22, #0x1000
    b.lo .Libs_intty
    ldr x0, [x22, #TY_QUALS]
    tst x0, #2
    b.ne .Libs_haveity
.Libs_intty:
    adrp x22, _kcc_type_int_s@PAGE
    add  x22, x22, _kcc_type_int_s@PAGEOFF
    ldr x22, [x22]
.Libs_haveity:

.Libs_load:
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    mov x20, x0
    str x22, [x20, #IR_TYPE]     // volatile
   element type when volatile, else int
    str x19, [x20, #IR_SRC1]     // pointer
    str x21, [x20, #IR_SRC2]     // access size
   override
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x19, x0                   // result vreg
    mov x0, x20
    bl _kcc_ir_emit

    mov x0, x19                   // return value
   vreg

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

353a · defined at 349a, 350, 353a · used in 30b · defines _kcc_ir_build_subscript 340 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_build_subscript_addr 350, _kcc_ir_emit 304, _kcc_ir_load_width 349b, _kcc_ir_next_vreg 305a, _kcc_type_int_s 250b

Array tests verify declaration, element write, element read, multiple elements, loop-based filling, and variable indexing.

⟨ *Test: codegen arrays* 353b ⟩ ≡

```
echo ""
echo "--- Arrays ---"

run_test "array-write-read" \
    "int main(void) { int a[3]; a[0] = 10; return
   a[0]; }" 10

run_test "array-multi-elem" \
    "int main(void) { int a[3]; a[0] = 1; a[1] =
   2; a[2] = 3; return a[0] + a[1] + a[2]; }" 6

run_test "array-overwrite" \
    "int main(void) { int a[2]; a[0] = 5; a[0] =
   9; return a[0]; }" 9

run_test "array-index-var" \
    "int main(void) { int a[4]; int i = 2; a[i] =
   7; return a[2]; }" 7

run_test "array-loop-fill" \
    "int main(void) { int a[5]; int i = 0; while (
   i < 5) { a[i] = i; i = i + 1; } return a[3]; }"
    3

run_test "array-sum-loop" \
    "int main(void) { int a[4]; a[0]=1; a[1]=2; a
   [2]=3; a[3]=4; int s=0; int i=0; while(i<4){s=s
   +a[i];i=i+1;} return s; }" 10

run_test "array-2d" \
    "int main(void) { int a[3][3]; int i; int j;
   for (i=0;i<3;i=i+1) for (j=0;j<3;j=j+1) a[i][j
   ]=i*3+j; return a[2][2]; }" 8

run_test "array-2d-rows" \
    "int main(void) { int a[4][2]; int i; for (i
   =0;i<4;i=i+1) { a[i][0]=i; a[i][1]=i*i; }
   return a[3][0]+a[3][1]; }" 12

run_test "array-3d" \
    "int main(void) { int a[2][2][2]; a
   [1][1][1]=42; a[0][1][0]=5; return a[1][1][1]+a
   [0][1][0]; }" 47

echo ""
echo "--- Pointer Subscript ---"

run_test "ptr-sub-write-read" \
    "void *mmap(void*,long,int,int,int,long); int
   main(void) { int *p = (int*)mmap
   (0,4096,3,4098,-1,0); p[0] = 42; return p[0]; }
   " 42

run_test "ptr-sub-multi" \
    "void *mmap(void*,long,int,int,int,long); int
   main(void) { int *p = (int*)mmap
   (0,4096,3,4098,-1,0); p[0]=1; p[1]=2; p[2]=3;
   return p[0]+p[1]+p[2]; }" 6

run_test "ptr-sub-var-index" \
    "void *mmap(void*,long,int,int,int,long); int
   main(void) { int *p = (int*)mmap
   (0,4096,3,4098,-1,0); int i = 2; p[i] = 99;
   return p[2]; }" 99

run_test "ptr-sub-char" \
    "void *mmap(void*,long,int,int,int,long); int
   main(void) { char *s = (char*)mmap
   (0,4096,3,4098,-1,0); s[0]=65; s[1]=66; return
```

```
    s[0]+s[1]; }" 131

run_test "ptr-sub-expr-index" \
    "void *mmap(void*,long,int,int,int,long); int
    main(void) { int *p = (int*)mmap
    (0,4096,3,4098,-1,0); p[0]=10; p[1]=20; p
    [2]=30; int j=0; return p[j+1]; }" 20

run_test "ptr-sub-swap" \
    "void *mmap(void*,long,int,int,int,long); int
    main(void) { int *p = (int*)mmap
    (0,4096,3,4098,-1,0); p[0]=50; p[1]=10; int t=p
    [0]; p[0]=p[1]; p[1]=t; return p[0]; }" 10

run_test "ptr-sub-bubble" \
    "void *mmap(void*,long,int,int,int,long); int
    main(void) { int *a = (int*)mmap
    (0,4096,3,4098,-1,0); a[0]=5; a[1]=3; a[2]=4; a
    [3]=1; a[4]=2; int i=0; while(i<4){int j=0;
    while(j<4-i){if(a[j]>a[j+1]){int t=a[j]; a[j]=a
    [j+1]; a[j+1]=t;} j=j+1;} i=i+1;} return a[0]+a
    [4]; }" 6
```

□

353b · used in 33b

29.9.5. Struct Member Access

The struct member access `s.x` follows the same pattern as array subscript: an `IR_GEP` computes the field address, and an `IR_LOAD` reads the value. The key difference is that the field index comes from semantic analysis (stored in `AST_MEMBER.F3` during `kcc_sema_expr_member`) rather than from a runtime expression.

The address helper `_kcc_ir_build_member_addr` looks up the struct variable's alloca vreg, emits `IR_CONST` with the field index, and emits `IR_GEP`. It is shared by both the read path (`s.x` in expressions) and the write path (`s.x = v` in assignments). When the object is a subscript expression rather than a simple identifier—as in `arr[i].x`—the handler calls `_kcc_ir_build_subscript_addr` to obtain the element's *address* rather than `_kcc_ir_build_expr` which would dereference it. Nested struct access (`o.a.x`) is handled similarly: when the object is itself an `AST_MEMBER` with dot access, the handler calls itself recursively to obtain the inner field's *address*. Loading the inner field's value would produce raw struct bytes rather than a valid pointer, causing a crash on the subsequent GEP.

Arrow access (`p->x`) requires two adjustments. First, the identifier path must emit an `IR_LOAD` to read the pointer value stored in the local variable's alloca—the alloca holds the address of `p`, not of the struct. Second, the type resolution must dereference through the pointer type: the object's `AST_F3` is `TY_PTR`; we follow `TY_F0` to reach the underlying `TY_STRUCT` so that the field-name search produces the correct offset. That search is the same shared `kcc_type_find_member` the type-checker uses, so an access that reaches through an anonymous (nameless) struct or union member resolves to the correct field and its offset is accumulated across the enclosing anonymous members.

Global structs require a separate path from local structs. When the identifier is not found in the locals hash table, the builder emits `IR_GLOBAL_REF` to materialise the struct's address. For dot access, this address is the base for `IR_GEP` directly. For arrow access on a global *pointer*, an additional `IR_LOAD` fetches the pointer value before the GEP—but arrow access on a global *array* (`T a[]; a->m`, i.e. `a[0].m`) must *not* load: the array name already denotes its element-0 address. Sema decays the array to `TY_PTR` in `AST_F3`, so the pre-decay array-ness is recovered from the symbol table, exactly as the subscript address builder recovers it for `a[i]`. Without this, the constant initializer `&((Upgrade_items)->uaattrid)` loaded element 0's `int` and dereferenced it as an address.

Parentheses around the object are transparent: `(p->m).f` is `p->m.f`. Because the object is inspected by tag to choose the address path (identifier, nested member, subscript) from the value path, a stray `AST_PAREN_EXPR` wrapper would slip past the nested-member case and fall to `build_expr`, which *loads* the inner aggregate and dereferences that value as if it were a pointer. That is exactly how `lobject.h`'s `val_(io).i = x`—which expands to `((io)->value_).i = x`—corrupted a `TValue`: the assignment wrote through the union's first eight bytes instead of into them. So we peel any parenthesis wrappers off the object up front, rewriting the member node's object field to the bare inner expression (the parens carry no runtime meaning) before dispatching.

⟨ *IR: build member* 355 ⟩ ≡

```
// -- kcc_ir_build_member_addr ---
// uint64_t kcc_ir_build_member_addr(node *member)
//   x0 = AST_MEMBER (F0=object, F1=name_ptr, F3=
   resolved type)
// Returns vreg holding the field address (GEP
   result).
// The field byte offset is computed by
   kcc_type_find_member, which
// matches the member name and descends into
   anonymous members.
.text
.globl _kcc_ir_build_member_addr
.p2align 2
_kcc_ir_build_member_addr:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]

    mov x19, x0                    // x19 = member
   node

    // Parentheses are transparent to a member
   address: `(p->m).f` is `p->m.f`.
    // Peel any AST_PAREN_EXPR off the object and
   rewrite the member node's
    // object field in place, so the tag-dispatch
   below sees the real object
    // (identifier / nested member / subscript)
   rather than a wrapper that
    // would wrongly take the value path and load
   the aggregate.
    ldr x0, [x19, #AST_F0]       // object
   expression
.Libma_unparen:
    cbz x0, .Libma_obj_ready
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_PAREN_EXPR
    b.ne .Libma_obj_ready
    ldr x0, [x0, #AST_F0]        // inner
```

```
    expression
    str x0, [x19, #AST_F0]      // rewrite object
    → inner (parens dropped)
    b .Libma_unparen
.Libma_obj_ready:

    // Get object (struct variable) - must be an
    identifier
    ldr x0, [x19, #AST_F0]      // object
    expression
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_IDENT_EXPR
    b.ne .Libma_expr

    // Local struct: look up alloca vreg
    ldr x20, [x0, #AST_F0]      // name ptr
    mov x0, x20
    bl _kcc_strlen
    mov x2, x0
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x20
    bl _kcc_ht_get
    cbz x0, .Libma_global

    // Arrow access loads the pointer value from
    the alloca---but only for a
    // pointer variable (bit 63 set).  An array
    base (`T a[1]; a->m`) decays to
    // its own address, so its alloca already IS
    the pointer; loading from it
    // would read the array's first bytes as a
    bogus pointer.
    ldr x1, [x19, #AST_F2]
    cbz x1, .Libma_local_noload  // dot access →
    alloca IS struct base
    tbz x0, #63, .Libma_local_noload  // arrow on
    a non-pointer (array) → no load
    bic x0, x0, #0x8000000000000000
    sub x20, x0, #1              // pointer alloca
    vreg
    // alloca holds a pointer - emit load to get
    the actual pointer
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x20, [x22, #IR_SRC1]    // load from
    alloca
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0                  // now x20 =
    loaded pointer value
    mov x0, x22
    bl _kcc_ir_emit
    b .Libma_find_index

.Libma_local_noload:
    // Dot access, or arrow on an array: the
    alloca vreg is already the base
    // address of the struct.
    bic x0, x0, #0x8000000000000000
    sub x20, x0, #1
    b .Libma_find_index

.Libma_global:
    // Global struct: emit global_ref for the
    address.  First redirect a static
    // local onto its mangled hoisted-global
    symbol---it is not in kcc_ir_locals,
    // so it reaches this global path; without the
    redirect `static struct S s;
    // s.a` would reference an undefined external
    `_s' while the reservation sits
    // under the mangled name (the read/compound-
    assign/inc paths already do this,
    // but the member-address base did not, so
    struct-typed static locals failed
    // to link).  The type-table lookup below
    still keys on the source name.
    ldr x0, [x19, #AST_F0]      // object ident
    node
    ldr x24, [x0, #AST_F0]      // x24 = effective
    symbol name (source name default)
    adrp x0, _kcc_ir_statics@PAGE
    add  x0, x0, _kcc_ir_statics@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Libma_global_emit
    mov x0, x24
    bl _kcc_strlen
    mov x2, x0                   // keylen
    adrp x0, _kcc_ir_statics@PAGE
    add  x0, x0, _kcc_ir_statics@PAGEOFF
    ldr x0, [x0]
    mov x1, x24                  // key = source
    name
    bl _kcc_ht_get
    cbz x0, .Libma_global_emit
    mov x24, x0                  // x24 = mangled
    hoisted-global name
.Libma_global_emit:
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    mov x22, x0
    str x24, [x22, #IR_SRC1]     // symbol name (
    redirected for a static local)
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0                  // global address
    vreg
    mov x0, x22
    bl _kcc_ir_emit
    // Arrow on global: also load the pointer
    value---UNLESS the global is an
    // array.  An array name decays to its own
    address (there is no stored
    // pointer to follow), so `T a[]; a->m` must
    use the global_ref address
    // directly, exactly as the local path does
    via the alloca tag.  Loading
    // here read the array's first bytes as a
    bogus pointer: the static
    // initializer `&((Upgrade_items)->uaattrid)`
    dereferenced element 0's int
    // as an address and faulted.  Sema decays
    arrays to TY_PTR in AST_F3, so
    // the pre-decay type must come from the
    symbol table (as the subscript
    // address builder does for `a[i]` on a global
    array).
    ldr x1, [x19, #AST_F2]
    cbz x1, .Libma_find_index    // dot → address
    is the base
    ldr x0, [x19, #AST_F0]       // object ident
    node
    ldr x21, [x0, #AST_F0]       // name ptr (x21
    is re-derived by find_index)
    mov x0, x21
    bl _kcc_strlen
```

```
    mov x1, x0
    mov x0, x21
    bl _kcc_symtab_lookup
    cbz x0, .Libma_glob_load      // no symbol info
    → keep the load
    ldr x0, [x0, #SYM_TYPE]
    cbz x0, .Libma_glob_load
    ldr x0, [x0, #TY_KIND]
    cmp x0, #TY_ARRAY
    b.eq .Libma_find_index        // array base →
   global_ref IS the address
.Libma_glob_load:
    // arrow → load pointer from global address
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x20, [x22, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0
    mov x0, x22
    bl _kcc_ir_emit
    b .Libma_find_index

.Libma_expr:
    // Non-identifier object: arrow uses value (
   pointer), dot+subscript uses address
    ldr x0, [x19, #AST_F0]
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_SUBSCRIPT
    b.ne .Libma_check_nested
    ldr x1, [x19, #AST_F2]
    cbnz x1, .Libma_expr_val      // arrow → value
   path
    // dot + subscript: get element address, not
   value
    bl _kcc_ir_build_subscript_addr
    mov x20, x0
    b .Libma_find_index
.Libma_check_nested:
    cmp x1, #AST_MEMBER
    b.ne .Libma_expr_val
    ldr x1, [x19, #AST_F2]
    cbnz x1, .Libma_expr_val      // arrow → value
   path
    // dot + nested member: get field address
   recursively
    bl _kcc_ir_build_member_addr
    mov x20, x0
    b .Libma_find_index
.Libma_expr_val:
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_build_expr
    mov x20, x0                   // base vreg

.Libma_find_index:
    // Find the field's byte offset by walking the
    struct type's field list.
    // Get struct type from sema (AST_F3 of object
    expression).
    ldr x0, [x19, #AST_F0]        // object node
    ldr x0, [x0, #AST_F3]         // resolved type
    cbz x0, .Libma_index_zero
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_PTR
    b.ne .Libma_check_struct
    ldr x0, [x0, #TY_F0]          // arrow: follow
   pointer to struct type
    cbz x0, .Libma_index_zero
    ldr x1, [x0, #TY_KIND]
.Libma_check_struct:
    cmp x1, #TY_STRUCT
    b.eq .Libma_have_struct
    cmp x1, #TY_UNION
    b.ne .Libma_index_zero
.Libma_have_struct:
    // Ensure the aggregate is laid out (per-field
    byte offsets in AST_F3).
    mov x25, x0                   // preserve struct
    type across the call
    bl _kcc_type_layout

    // Find the member.  A member of an anonymous
   (nameless) struct/union
    // member is reachable directly, so we
   delegate to the shared recursive
    // search that accumulates the enclosing
   anonymous members' offsets --
    // the same routine sema uses, keeping type-
   check and lowering in step.
    mov x0, x25                   // struct/union
   type
    ldr x1, [x19, #AST_F1]        // member name to
   find
    bl _kcc_type_find_member      // x0 = field decl
    (0 if none), x1 = byte offset
    cbz x0, .Libma_index_zero
    mov x23, x0                   // field decl
    mov x21, x1                   // accumulated
   byte offset

.Libma_found:
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    str x23, [x0]                  // expose the
   field decl for bit-field access
    b .Libma_emit_const

.Libma_index_zero:
    mov x21, #0
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    str xzr, [x0]                  // not a resolved
    field → no bit-field metadata

.Libma_emit_const:
    // Emit IR_CONST for the field's byte offset
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x22, x0
    str x21, [x22, #IR_SRC1]      // constant value
   = field byte offset
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                    // offset vreg
    mov x0, x22
    bl _kcc_ir_emit

    // Emit IR_GEP: base + byte offset (stride 1)
    mov x0, #IR_GEP
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x20, [x22, #IR_SRC1]      // base pointer
    str x21, [x22, #IR_SRC2]      // index
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
```

```
    mov x23, x0                     // result vreg
    mov x0, x22
    bl _kcc_ir_emit

    mov x0, x23                     // return GEP
   vreg

    ldr x25, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

355 · defined at 355, 357 · used in 30b · defines _kcc_ir_build_member_addr 328, 350, 357, 378b, 382, 387, 398, 405 · uses _kcc_ht_get 62, _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_build_subscript_addr 350, _kcc_ir_emit 304, _kcc_ir_locals 305c, _kcc_ir_next_vreg 305a, _kcc_ir_statics 305c, _kcc_symtab_lookup 255, _kcc_type_find_member 290, _kcc_type_int_s 250b, _kcc_type_layout 267, 268

Reading a member value is the address computation followed by a load: `_kcc_ir_build_member` calls `_kcc_ir_build_member_addr` for the field's address, then emits an `IR_LOAD` (or the shift-and-mask sequence for a bit-field). One case must *not* load, though: an *array* or *function* member decays to a pointer to its first element rather than being read (§6.3.2.1). So `s.a`, or `p->a` with `a` an array, yields the field address itself—the same value `&s.a[0]` would. Missing this turned `setjmp((c)->b)` in `luaD_rawrunprotected`—where `b` is a `jmp_buf` array member—into a load of the buffer's first eight bytes, which `setjmp` then used as the save area, faulting on a truncated address.

⟨ *IR: build member* 357 ⟩ + ≡

```
// -- kcc_ir_build_member ---
// uint64_t kcc_ir_build_member(node *member)
//   x0 = AST_MEMBER (F0=object, F1=name_ptr, F3=
   field_index)
// Returns vreg holding the loaded value (or the
   field address for an
// array/function member, which decays rather than
    loading).
.text
.globl _kcc_ir_build_member
.p2align 2
_kcc_ir_build_member:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                     // save member
   node (for field type)

    // Get field address
    bl _kcc_ir_build_member_addr
    mov x20, x0                     // GEP vreg (
   field address)

    // Array/function member: decays to its
   address, no load (`c->b` with b a
    // jmp_buf[] yields &c->b[0]).  Return the GEP
    result directly.
    ldr x1, [x19, #AST_F3]      // field type
    cmp x1, #0x1000
    b.lo .Libm_bitfield_check
    ldr x2, [x1, #TY_KIND]
    cmp x2, #TY_ARRAY
    b.eq .Libm_decay
    cmp x2, #TY_FUNC
    b.ne .Libm_bitfield_check
.Libm_decay:
    mov x0, x20                     // field address
   (array/function decay)
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Libm_bitfield_check:
    // Bit-field?  _kcc_ir_build_member_addr
   published the matched field decl;
    // a non-zero packed AST_F2 means a bit-field,
    read by shift-and-mask.
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    ldr x1, [x0]
    cbz x1, .Libm_scalar
    ldr x2, [x1, #AST_F2]
    cbz x2, .Libm_scalar
    mov x0, x20                     // container
   address
    bl _kcc_ir_bitfield_load     // x1 = field
   decl
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.Libm_scalar:
    // Load type: the field's own descriptor when
   it is volatile or floating
    // (so the back end emits an FP load for a `
   float`/`double` member rather
    // than an integer one), else the canonical
   int with a width override.
    // Held in x21 across the width call and the
   alloc.
    ldr x21, [x19, #AST_F3]      // field type
    cmp x21, #0x1000
    b.lo .Libm_intty
    ldr x0, [x21, #TY_QUALS]
    tst x0, #2
    b.ne .Libm_haveity
    ldr x0, [x21, #TY_KIND]
    cmp x0, #TY_FLOAT
    b.eq .Libm_haveity
    cmp x0, #TY_DOUBLE
    b.eq .Libm_haveity
.Libm_intty:
    adrp x21, _kcc_type_int_s@PAGE
    add  x21, x21, _kcc_type_int_s@PAGEOFF
    ldr x21, [x21]
.Libm_haveity:

    // Field access width + signedness from the
   member's resolved type.
    ldr x0, [x19, #AST_F3]       // field type
    bl _kcc_ir_load_width
    mov x19, x0                     // load-size
   override

    // Emit IR_LOAD from field address
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
```

```
    mov x1, x0
    str x21, [x1, #IR_TYPE]      // volatile field
   type when volatile, else int
    str x20, [x1, #IR_SRC1]      // pointer = GEP
   result
    str x19, [x1, #IR_SRC2]      // access-size
   override
    mov x20, x1                   // load instr
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x19, x0                   // result vreg
    mov x0, x20
    bl _kcc_ir_emit

    mov x0, x19                   // return value
   vreg

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

357 · defined at 355, 357 · used in 30b · defines _kcc_ir_build_member 340 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_build_member_addr 355, _kcc_ir_emit 304, _kcc_ir_load_width 349b, _kcc_ir_next_vreg 305a, _kcc_type_int_s 250b

29.9.6. Bit-field Access

A bit-field is stored inside a wider container, so it cannot be read with a plain load or written with a plain store. Rather than teach the backend new instructions, kcc *synthesises* the access out of IR operations it already lowers: shifts, masks, and—for the write—an or. A read loads the whole container, shifts the field down to bit zero, masks it to its width, and, for a signed bit-field, sign-extends by shifting left then arithmetic-right by $64 - \texttt{width}$. A write is a read-modify-write: load the container, clear the field's bits with AND texttilde $(\texttt{mask} \ll \texttt{offset})$, splice in $(\texttt{value} \,\&\, \texttt{mask}) \ll \texttt{offset}$ with an OR, and store the container back. All the intermediate values are 64-bit so masks up to a full long container materialise correctly; only the final store narrows to the container's width.

Two tiny emitters keep the synthesis readable: kcc_ir_emit_const materialises an integer, and kcc_ir_emit_binop emits one two-operand instruction and returns its result vreg. The field to act on is the one _kcc_ir_build_member_addr just matched, published through _kcc_ir_member_field; its packed AST_F2 gives the bit offset and width, and its declared type gives the container size and signedness.

A volatile bit-field (§ 6.7.3) needs one more thing: the whole read-modify-write is expressed as ordinary loads and stores of the container, and the optimizer keys volatility off an instruction's IR_TYPE, so those container accesses must carry the volatile bit or CSE could merge two reads and dead-store elimination could drop a write—silently defeating the qualifier. _kcc_ir_bitfield_load and _kcc_ir_bitfield_store therefore ask _kcc_ir_bf_container_type for the container's IR type: the canonical unsigned long, or—when the field's type is volatile-qualified—a once-cloned copy of it with the volatile bit set, which they pass to the type-taking _kcc_ir_emit_load_t / _kcc_ir_emit_store_t instead of the default wrappers. This brings a volatile bit-field to parity with a volatile scalar member; a bit-field made volatile only through a volatile *object* (volatile struct S s) is not yet distinguished, the same broader gap that non-bit-field members share.

One further subtlety concerns the *value* of a bit-field assignment expression. For l.m = j the standard (§ 6.5.16.1) says the result is the value of the left operand *after* the store—and a store into an n-bit field keeps only its low n bits. So (signed int:11) = 1081 does not yield 1081; it yields $1081 - 2048 = -967$, the eleven-bit two's-complement truncation. _kcc_ir_bf_trunc_value recomputes this from the stored value in registers—masking to the width and sign-extending a signed field—rather than reloading the container, so a volatile bit-field is not read twice. Both the simple and the compound bit-field store paths of _kcc_ir_assign_through route their result through it.

⟨ *IR: bitfield access* 359 ⟩ ≡

```
// -- kcc_ir_emit_const ---
// uint64_t kcc_ir_emit_const(uint64_t value) →
   result vreg
.text
.p2align 2
_kcc_ir_emit_const:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x20, x0
    str x19, [x20, #IR_SRC1]
    adrp x1, _kcc_type_long_u@PAGE
    add  x1, x1, _kcc_type_long_u@PAGEOFF
    ldr x1, [x1]
    str x1, [x20, #IR_TYPE]
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x19, x0
    mov x0, x20
    bl _kcc_ir_emit
    mov x0, x19
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_emit_binop ---
// uint64_t kcc_ir_emit_binop(int op, uint64_t
   a_vreg, uint64_t b_vreg) → vreg
.p2align 2
_kcc_ir_emit_binop:
    // Backwards-compatible entry: the result type
    is long_u, right for the
    // integer and pointer/bit-field address
   arithmetic all existing callers do.
    adrp x3, _kcc_type_long_u@PAGE
    add  x3, x3, _kcc_type_long_u@PAGEOFF
    ldr x3, [x3]
    // fall through with x3 = result type
_kcc_ir_emit_binop_typed:
    // uint64_t kcc_ir_emit_binop_typed(op x0, a
   x1, b x2, type x3)
    // Emit `a op b` carrying the given result
   type.  Typing it floating is what
```

```
    // makes the back end select FADD/FSUB/FMUL/
   FDIV for a floating operand.
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x1                     // a
    mov x20, x2                     // b
    mov x22, x3                     // result type
    bl _kcc_ir_alloc_instr          // x0 = op passed
    through
    mov x21, x0
    str x22, [x21, #IR_TYPE]
    str x19, [x21, #IR_SRC1]
    str x20, [x21, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x22, x0
    mov x0, x21
    bl _kcc_ir_emit
    mov x0, x22
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_compound_apply ---
// uint64_t kcc_ir_compound_apply(op x0, old x1, L
    x2, rhs x3, R x4)
// Compute `old OP rhs` for a compound assignment
   E1 op= E2 under the usual
// arithmetic conversions.  When either operand is
    floating the operation is
// done in the floating (common) type and its
   result converted back to the
// lvalue type L; a purely integer operation keeps
    the previous behaviour (a
// long_u binop followed by width normalisation).
    Returns the result vreg,
// typed L, ready to store.  This is what makes `d
    += n` (n an int) add in
// double and `i += x` (x a double) add in double
   then truncate to int, exactly
// as the equivalent `E1 = E1 OP E2` would.  When
   the lvalue is a pointer and
// the operator is `+`/`-`, the right operand is
   first scaled by
// \texttt{sizeof(*p)}, so `p += n` advances by
   whole elements exactly as
// `p = p + n` does.
.text
.globl _kcc_ir_compound_apply
.p2align 2
_kcc_ir_compound_apply:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                     // op (IR opcode)
    mov x20, x1                     // old vreg
    mov x21, x2                     // L (lvalue type
   )
    mov x22, x3                     // rhs vreg
    mov x23, x4                     // R (rhs type)

    // Pointer `+=`/`-=`: scale the right operand
   by sizeof(pointee) so the
    // step counts elements, not bytes (`p += n`
   == `p = p + n`).
    cmp x19, #IR_ADD
    b.eq .Lca_ptr
    cmp x19, #IR_SUB
    b.ne .Lca_notptr
.Lca_ptr:
    cmp x21, #0x1000
    b.lo .Lca_notptr
    ldr x0, [x21, #TY_KIND]
    cmp x0, #TY_PTR
    b.ne .Lca_notptr
    ldr x0, [x21, #TY_F0]           // pointee type
    cbz x0, .Lca_notptr
    bl _kcc_type_layout             // x0 = sizeof(
   pointee)
    cmp x0, #1
    b.le .Lca_notptr                // stride 1 →
   nothing to scale
    bl _kcc_ir_emit_const           // const stride
   vreg (value in x0)
    mov x2, x0                      // stride vreg
    mov x0, #IR_MUL
    mov x1, x22                     // rhs
    bl _kcc_ir_emit_binop           // rhs * stride (
   long_u)
    mov x22, x0                     // scaled rhs
.Lca_notptr:
    // The compound-op table hands us the SIGNED
   opcode (SDIV/SMOD/ASHR).  The
    // unsigned form is required when the
   OPERATION is unsigned---and the
    // operation follows `E1 op E2`'s rules, NOT
   the lvalue's own type.  Division
    // and remainder run in the common type of the
    promoted operands (usual
    // arithmetic conversions); a shift runs in
   the promoted left operand.  A
    // narrow unsigned lvalue promotes to signed
   int, so `unsigned char uc /=
    // short` divides SIGNED (50 / -5 == -10);
   reading `uc`'s own unsigned-ness
    // made it an unsigned divide that folded 50 /
    (unsigned)-5 to 0.  Lua's
    // `count /= (unsigned)step` stays unsigned
   because there the lvalue is
    // already an unsigned int, whose promotion
   and common type are unsigned.
    cmp x19, #IR_ASHR
    b.eq .Lca_sign_shift
    cmp x19, #IR_SDIV
    b.eq .Lca_sign_arith
    cmp x19, #IR_SMOD
    b.eq .Lca_sign_arith
    b .Lca_signdone                 // add/sub/mul/and
   /or/xor/shl: no signed variant
.Lca_sign_arith:
    mov x0, x21                     // L
    mov x1, x23                     // R
    bl _kcc_conv_usual              // common type of
    the promoted operands
    b .Lca_sign_check
.Lca_sign_shift:
    mov x0, x21                     // L
    bl _kcc_conv_promote            // promoted left
   operand
.Lca_sign_check:
    cbz x0, .Lca_signdone
    ldr x9, [x0, #TY_KIND]
    cmp x9, #TY_BOOL                // 2
    b.lo .Lca_signdone
    cmp x9, #TY_LLONG               // 7
    b.hi .Lca_signdone              // float/aggregate
    → keep default (fp path remaps)
    ldr x9, [x0, #TY_F0]            // 1 = signed, 0 =
    unsigned
    cbnz x9, .Lca_signdone          // signed
```

```
   operation → keep the signed opcode
    cmp x19, #IR_SDIV
    b.ne .Lca_chk_smod
    mov x19, #IR_UDIV
    b .Lca_signdone
.Lca_chk_smod:
    cmp x19, #IR_SMOD
    b.ne .Lca_chk_ashr
    mov x19, #IR_UMOD
    b .Lca_signdone
.Lca_chk_ashr:
    mov x19, #IR_LSHR           // only ASHR
   reaches here
.Lca_signdone:

    mov x0, x21
    bl _kcc_ir_type_is_fp
    mov x24, x0                 // L_is_fp
    mov x0, x23
    bl _kcc_ir_type_is_fp
    orr x1, x0, x24
    cbnz x1, .Lca_fp

    // Integer path (unchanged): long_u binop then
    normalise to L's width.
    mov x0, x19
    mov x1, x20
    mov x2, x22
    adrp x3, _kcc_type_long_u@PAGE
    add  x3, x3, _kcc_type_long_u@PAGEOFF
    ldr x3, [x3]
    bl _kcc_ir_emit_binop_typed
    mov x1, x19                 // op
    mov x2, x21                 // L
    mov x3, x21
    bl _kcc_ir_norm32
    b .Lca_ret

.Lca_fp:
    // Common type C = L when it is floating, else
    the (floating) rhs type R.
    cbz x24, .Lca_C_from_R
    mov x24, x21                // C = L
    b .Lca_conv
.Lca_C_from_R:
    mov x24, x23                // C = R
.Lca_conv:
    mov x0, x20                 // old
    mov x1, x21                 // from L
    mov x2, x24                 // to C
    bl _kcc_ir_convert
    mov x20, x0                 // old_c
    mov x0, x22                 // rhs
    mov x1, x23                 // from R
    mov x2, x24                 // to C
    bl _kcc_ir_convert
    mov x22, x0                 // rhs_c
    mov x0, x19                 // op
    mov x1, x20
    mov x2, x22
    mov x3, x24                 // C
    bl _kcc_ir_emit_binop_typed
    mov x1, x24                 // from C
    mov x2, x21                 // to L
    bl _kcc_ir_convert          // result back to
    lvalue type
.Lca_ret:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_ir_bf_code ---
// int kcc_ir_bf_code(uint64_t container_bytes) →
   access-size override
// (unsigned): →80 (full 64-bit), →44, →22, →11.
.p2align 2
_kcc_ir_bf_code:
    cmp x0, #4
    b.eq .Lbfc_4
    cmp x0, #2
    b.eq .Lbfc_2
    cmp x0, #1
    b.eq .Lbfc_1
    mov x0, #0
    ret
.Lbfc_4:
    mov x0, #4
    ret
.Lbfc_2:
    mov x0, #2
    ret
.Lbfc_1:
    mov x0, #1
    ret

// -- kcc_ir_bf_mask ---
// uint64_t kcc_ir_bf_mask(uint64_t width) → (1<<
   width)-1, saturating at ~0.
.p2align 2
_kcc_ir_bf_mask:
    cmp x0, #64
    b.hs .Lbfm_all
    mov x1, #1
    lsl x1, x1, x0
    sub x0, x1, #1
    ret
.Lbfm_all:
    mov x0, #-1
    ret

// -- kcc_ir_bitfield_load ---
// uint64_t kcc_ir_bitfield_load(uint64_t
   addr_vreg, node *field) → value vreg
//   addr_vreg = container address, field = the
   bit-field decl node.
.globl _kcc_ir_bitfield_load
.p2align 2
_kcc_ir_bitfield_load:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                 // addr
    ldr x0, [x1, #AST_F2]       // packed (
   bit_offset<<16)|width
    and x22, x0, #0xffff        // width
    lsr x23, x0, #16            // bit offset (
   lsb)
    ldr x0, [x1, #AST_F0]       // field type
   spec
    bl _kcc_sema_resolve_type
    mov x24, x0                 // field type
    bl _kcc_ir_type_layout_size // x0 = container
    bytes
    bl _kcc_ir_bf_code
    mov x20, x0                 // unsigned load
   code
    // container LOAD --- carry a volatile-
   qualified container type for a
    // volatile bit-field so its read is never
   merged or eliminated.
    mov x0, x24                 // field type
    bl _kcc_ir_bf_container_type
    mov x2, x0                  // container IR
   type
```

```
    mov x0, x19                 // addr
    mov x1, x20                 // access-size
   code
    bl _kcc_ir_emit_load_t
    mov x21, x0                 // loaded value
   vreg
    // shift right by lsb
    cbz x23, .Lbfl_mask
    mov x0, x23
    bl _kcc_ir_emit_const
    mov x1, x21
    mov x2, x0
    mov x0, #IR_LSHR
    bl _kcc_ir_emit_binop
    mov x21, x0
.Lbfl_mask:
    mov x0, x22
    bl _kcc_ir_bf_mask
    bl _kcc_ir_emit_const
    mov x1, x21
    mov x2, x0
    mov x0, #IR_AND
    bl _kcc_ir_emit_binop
    mov x21, x0
    // An enum bit-field is treated as unsigned (
   implementation-defined,
    // \stdref{6.7.2.1}): its enumerators are non-
   negative and clang
    // zero-extends, so a value whose top bit is
   set (e.g. an enum constant
    // \(\ge 128\) in an 8-bit field) reads back
   positive, not sign-extended.
    ldr x0, [x24, #TY_KIND]
    cmp x0, #TY_ENUM
    b.eq .Lbfl_done              // already
   masked → zero-extended
    // signed? sign-extend from `width` bits
    mov x0, x24
    bl _kcc_sema_int_signedness
    cmp x0, #1
    b.ne .Lbfl_done
    mov x0, #64
    sub x0, x0, x22             // s = 64 - width
    mov x24, x0                 // reuse x24 =
   shift amount
    bl _kcc_ir_emit_const
    mov x1, x21
    mov x2, x0
    mov x0, #IR_SHL
    bl _kcc_ir_emit_binop
    mov x21, x0
    mov x0, x24
    bl _kcc_ir_emit_const
    mov x1, x21
    mov x2, x0
    mov x0, #IR_ASHR
    bl _kcc_ir_emit_binop
    mov x21, x0
.Lbfl_done:
    mov x0, x21
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_ir_bitfield_store ---
// uint64_t kcc_ir_bitfield_store(uint64_t addr,
   uint64_t value, node *field)
//   → the stored value vreg.  Read-modify-write
   of the container.
.globl _kcc_ir_bitfield_store
.p2align 2
_kcc_ir_bitfield_store:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    mov x19, x0                 // addr
    mov x20, x1                 // value
    ldr x0, [x2, #AST_F2]       // packed
    and x22, x0, #0xffff        // width
    lsr x23, x0, #16            // lsb
    ldr x0, [x2, #AST_F0]       // field type
    bl _kcc_sema_resolve_type
    mov x25, x0                 // field type (
   kept for the volatile check)
    bl _kcc_ir_type_layout_size  // container
   bytes
    bl _kcc_ir_bf_code
    mov x24, x0                 // access-size
   code
    // old = LOAD(addr)  (intermediates live in
   x21/x22/x26; the emitters only
    // clobber x0-x2 and their own x19-x22, so x23
   -x26 survive across calls).
    // A volatile bit-field's read-modify-write
   reads and writes its container
    // through a volatile-qualified type so
   neither half is optimized away.
    mov x0, x25                 // field type
    bl _kcc_ir_bf_container_type
    mov x2, x0                  // container IR
   type
    mov x0, x19                 // addr
    mov x1, x24                 // access-size
   code
    bl _kcc_ir_emit_load_t
    mov x26, x0                 // old container
   vreg
    mov x0, x22
    bl _kcc_ir_bf_mask
    mov x22, x0                 // mask (integer)
    // vbits = value & mask
    bl _kcc_ir_emit_const       // x0 already =
   mask
    mov x1, x20
    mov x2, x0
    mov x0, #IR_AND
    bl _kcc_ir_emit_binop
    mov x21, x0                 // vbits vreg
    cbz x23, .Lbfs_cleared
    mov x0, x23
    bl _kcc_ir_emit_const       // lsb const
    mov x1, x21
    mov x2, x0
    mov x0, #IR_SHL
    bl _kcc_ir_emit_binop       // vbits <<= lsb
    mov x21, x0
.Lbfs_cleared:
    // cleared = old & ~(mask << lsb)
    lsl x0, x22, x23
    mvn x0, x0
    bl _kcc_ir_emit_const
    mov x1, x26
    mov x2, x0
    mov x0, #IR_AND
    bl _kcc_ir_emit_binop
    mov x1, x0                  // cleared
    mov x2, x21                 // vbits
    mov x0, #IR_OR
    bl _kcc_ir_emit_binop
    mov x21, x0                 // newval
    // STORE newval → addr (container width),
   volatile-qualified when the
    // field is volatile so the write is a
```

```
    mandatory side effect.
    mov x0, x25                     // field type
    bl _kcc_ir_bf_container_type
    mov x3, x0                      // container IR
    type
    mov x0, x21                     // value
    mov x1, x19                     // addr
    mov x2, x24                     // access-size
    code
    bl _kcc_ir_emit_store_t
    mov x0, x20                     // assignment
    result = the value
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret

// -- kcc_ir_emit_load / kcc_ir_emit_store ---
// vreg kcc_ir_emit_load(addr_vreg, size_code);
    void kcc_ir_emit_store(value,
// addr, size_code).  Thin wrappers around IR_LOAD
    / IR_STORE with an
// access-size override, used by the bit-field
    synthesis.  Each is a stub that
// supplies the canonical unsigned-long type and
    tail-calls the `_t` variant;
// the bit-field code calls the `_t` variant
    directly to pass a
// volatile-qualified container type when the
    field is volatile, so the
// optimizer sees the required side effect.
.p2align 2
_kcc_ir_emit_load:
    adrp x2, _kcc_type_long_u@PAGE
    add  x2, x2, _kcc_type_long_u@PAGEOFF
    ldr x2, [x2]
    b _kcc_ir_emit_load_t

// vreg kcc_ir_emit_load_t(addr_vreg, size_code,
    type)
.globl _kcc_ir_emit_load_t
.p2align 2
_kcc_ir_emit_load_t:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x1                     // size code
    mov x20, x0                     // addr
    mov x21, x2                     // IR type (
    volatile-qualified when volatile)
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    mov x1, x0
    str x21, [x1, #IR_TYPE]
    str x20, [x1, #IR_SRC1]
    str x19, [x1, #IR_SRC2]
    mov x20, x1
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x19, x0
    mov x0, x20
    bl _kcc_ir_emit
    mov x0, x19
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

.p2align 2
_kcc_ir_emit_store:
    adrp x3, _kcc_type_long_u@PAGE
    add  x3, x3, _kcc_type_long_u@PAGEOFF
    ldr x3, [x3]
    b _kcc_ir_emit_store_t

// void kcc_ir_emit_store_t(value, addr, size_code
    , type)
.globl _kcc_ir_emit_store_t
.p2align 2
_kcc_ir_emit_store_t:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // value
    mov x20, x1                     // addr
    mov x21, x2                     // size code
    mov x22, x3                     // IR type (
    volatile-qualified when volatile)
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    str x22, [x0, #IR_TYPE]
    str x19, [x0, #IR_SRC1]         // value
    str x20, [x0, #IR_SRC2]         // addr
    str x21, [x0, #IR_DEST]         // access-size
    override
    bl _kcc_ir_emit
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_bf_container_type ---
// type *kcc_ir_bf_container_type(type *field_type
    )
//   The IR type to stamp on a bit-field's
    container load/store: the canonical
//   unsigned long, or---when the field (or its
    object) is volatile---a cached
//   clone of it with the volatile bit set, so the
    optimizer's CSE and DCE
//   passes (which key volatility off the
    instruction's IR_TYPE) treat every
//   container access as a mandatory, un-mergeable
    side effect (\stdref{6.7.3}).
.globl _kcc_ir_bf_container_type
.p2align 2
_kcc_ir_bf_container_type:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    cbz x0, .Lbct_plain
    ldr x1, [x0, #TY_QUALS]
    tst x1, #2
    b.eq .Lbct_plain
    adrp x0, _kcc_ir_vol_longu@PAGE
    add  x0, x0, _kcc_ir_vol_longu@PAGEOFF
    ldr x1, [x0]
    cbnz x1, .Lbct_cached
    // First volatile bit-field: clone the long_u
    descriptor and set volatile.
    mov x0, #TY_DESC_SIZE
    bl _kcc_arena_alloc
    mov x19, x0                     // new descriptor
    adrp x0, _kcc_type_long_u@PAGE
    add  x0, x0, _kcc_type_long_u@PAGEOFF
    ldr x0, [x0]                    // source
    descriptor
    ldr x1, [x0, #0]
    str x1, [x19, #0]
    ldr x1, [x0, #8]
    str x1, [x19, #8]
    ldr x1, [x0, #16]
```

```
    str x1, [x19, #16]
    ldr x1, [x0, #24]
    str x1, [x19, #24]
    ldr x1, [x0, #32]
    str x1, [x19, #32]
    ldr x1, [x0, #40]
    str x1, [x19, #40]
    ldr x1, [x19, #TY_QUALS]
    orr x1, x1, #2                // mark volatile
    str x1, [x19, #TY_QUALS]
    adrp x0, _kcc_ir_vol_longu@PAGE
    add  x0, x0, _kcc_ir_vol_longu@PAGEOFF
    str x19, [x0]                 // cache for
   subsequent volatile bit-fields
    mov x0, x19
    b .Lbct_ret
.Lbct_cached:
    mov x0, x1
    b .Lbct_ret
.Lbct_plain:
    adrp x0, _kcc_type_long_u@PAGE
    add  x0, x0, _kcc_type_long_u@PAGEOFF
    ldr x0, [x0]
.Lbct_ret:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_type_layout_size ---
// uint64_t kcc_ir_type_layout_size(type *t) →
   size in bytes (forces layout).
.p2align 2
_kcc_ir_type_layout_size:
    cbz x0, .Ltls_zero
    b _kcc_type_layout            // tail call:
   returns TY_SIZE in x0
.Ltls_zero:
    mov x0, #0
    ret

// -- kcc_ir_emit_gep_const ---
// uint64_t kcc_ir_emit_gep_const(base_vreg,
   byte_offset) → address vreg
.p2align 2
_kcc_ir_emit_gep_const:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                   // base
    mov x0, x1                    // offset
    bl _kcc_ir_emit_const
    mov x20, x0                   // offset vreg
    mov x0, #IR_GEP
    bl _kcc_ir_alloc_instr
    mov x21, x0
    adrp x1, _kcc_type_long_u@PAGE
    add  x1, x1, _kcc_type_long_u@PAGEOFF
    ldr x1, [x1]
    str x1, [x21, #IR_TYPE]
    str x19, [x21, #IR_SRC1]
    str x20, [x21, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x19, x0
    mov x0, x21
    bl _kcc_ir_emit
    mov x0, x19
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_store_elem ---
// void kcc_ir_store_elem(base_vreg, byte_offset,
   value_vreg, width)
// Writes one initializer element.  If the field
   just resolved by
// _kcc_ir_elem_off_width (published in
   _kcc_ir_member_field) is a bit-field,
// read-modify-write its container; otherwise a
   plain sized store.
.globl _kcc_ir_store_elem
.p2align 2
_kcc_ir_store_elem:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                   // base
    mov x20, x1                   // offset
    mov x21, x2                   // value
    mov x22, x3                   // width
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lse_plain
    ldr x1, [x0, #AST_F2]
    cbz x1, .Lse_plain
    mov x22, x0                   // field decl (
   survives emit_gep_const)
    mov x0, x19
    mov x1, x20
    bl _kcc_ir_emit_gep_const     // x0 = container
    address
    mov x1, x21                   // value
    mov x2, x22                   // field
    bl _kcc_ir_bitfield_store
    b .Lse_done
.Lse_plain:
    mov x0, x19
    mov x1, x20
    mov x2, x21
    mov x3, x22
    bl _kcc_ir_emit_indexed_store
.Lse_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_emit_string_init ---
// void kcc_ir_emit_string_init(base, str_ptr,
   str_len, array_len, zero_first)
// Copies min(str_len, array_len) bytes of a
   compile-time string into an array
// as per-byte constant stores; the NUL and tail
   come from the zero-fill/BSS.
.globl _kcc_ir_emit_string_init
.p2align 2
_kcc_ir_emit_string_init:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                   // base
    mov x20, x1                   // str_ptr
    mov x21, x2                   // str_len
    mov x22, x3                   // array_len
    cbz x4, .Lesi_copy
    add x0, x22, #7
    lsr x23, x0, #3               // slot count
    mov x24, #0
.Lesi_zloop:
    cmp x24, x23
```

```
    b.ge .Lesi_copy
    mov x0, #0
    bl _kcc_ir_emit_const
    mov x2, x0
    mov x0, x19
    lsl x1, x24, #3
    mov x3, #0
    bl _kcc_ir_emit_indexed_store
    add x24, x24, #1
    b .Lesi_zloop
.Lesi_copy:
    mov x23, x21
    cmp x22, x23
    csel x23, x22, x23, lo        // min(array_len,
    str_len)
    mov x24, #0
.Lesi_cloop:
    cmp x24, x23
    b.ge .Lesi_done
    ldrb w0, [x20, x24]
    bl _kcc_ir_emit_const
    mov x2, x0
    mov x0, x19
    mov x1, x24
    mov x3, #1
    bl _kcc_ir_emit_indexed_store
    add x24, x24, #1
    b .Lesi_cloop
.Lesi_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_ir_emit_bool_norm ---
// uint64_t kcc_ir_emit_bool_norm(uint64_t vreg)
   → (vreg != 0) as 0/1
.globl _kcc_ir_emit_bool_norm
.p2align 2
_kcc_ir_emit_bool_norm:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0
    mov x0, #0
    bl _kcc_ir_emit_const
    mov x1, x19
    mov x2, x0
    mov x0, #IR_CMP_NE
    bl _kcc_ir_emit_binop
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_tok_to_binop ---
// int kcc_ir_tok_to_binop(int compound_assign_tok
   ) → IR opcode
.p2align 2
_kcc_ir_tok_to_binop:
    cmp x0, #TOK_SUB_ASSIGN
    b.eq .Ltb_sub
    cmp x0, #TOK_MUL_ASSIGN
    b.eq .Ltb_mul
    cmp x0, #TOK_DIV_ASSIGN
    b.eq .Ltb_div
    cmp x0, #TOK_MOD_ASSIGN
    b.eq .Ltb_mod
    cmp x0, #TOK_AND_ASSIGN
    b.eq .Ltb_and
    cmp x0, #TOK_OR_ASSIGN
    b.eq .Ltb_or
    cmp x0, #TOK_XOR_ASSIGN
    b.eq .Ltb_xor
    cmp x0, #TOK_SHL_ASSIGN
    b.eq .Ltb_shl
    cmp x0, #TOK_SHR_ASSIGN
    b.eq .Ltb_shr
    mov x0, #IR_ADD              // TOK_ADD_ASSIGN
    (and default)
    ret
.Ltb_sub: mov x0, #IR_SUB
    ret
.Ltb_mul: mov x0, #IR_MUL
    ret
.Ltb_div: mov x0, #IR_SDIV
    ret
.Ltb_mod: mov x0, #IR_SMOD
    ret
.Ltb_and: mov x0, #IR_AND
    ret
.Ltb_or:  mov x0, #IR_OR
    ret
.Ltb_xor: mov x0, #IR_XOR
    ret
.Ltb_shl: mov x0, #IR_SHL
    ret
.Ltb_shr: mov x0, #IR_ASHR
    ret

// -- kcc_ir_bf_trunc_value ---
// uint64_t kcc_ir_bf_trunc_value(uint64_t
   value_vreg, node *field) → vreg
// The value of a bit-field assignment expression
   is what was *stored* --- the
// low `width` bits of the assigned value, read
   back at the field's type
// (\stdref{6.5.16.1}, \stdref{6.7.2.1}).
   Truncate `value` to the field width
// in-register (AND with the width mask; sign-
   extend with SHL/ASHR when the
// field is signed), avoiding a memory reload that
    a volatile bit-field would
// forbid.  An enum bit-field is treated as
   unsigned.
.globl _kcc_ir_bf_trunc_value
.p2align 2
_kcc_ir_bf_trunc_value:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]
    mov x21, x0                  // value vreg
    mov x19, x1                  // field decl
    ldr x0, [x19, #AST_F2]       // packed (
   bit_offset<<16)|width
    and x22, x0, #0xffff         // width
    ldr x0, [x19, #AST_F0]       // field type
   spec
    bl _kcc_sema_resolve_type
    mov x23, x0                  // field type
    // masked = value AND ((1<<width)-1)
    mov x0, x22
    bl _kcc_ir_bf_mask
    bl _kcc_ir_emit_const
    mov x1, x21
    mov x2, x0
    mov x0, #IR_AND
    bl _kcc_ir_emit_binop
    mov x21, x0
    ldr x0, [x23, #TY_KIND]
    cmp x0, #TY_ENUM
    b.eq .Lbtv_done              // enum bit-field
   : unsigned (already zero-ext)
    mov x0, x23
    bl _kcc_sema_int_signedness
    cmp x0, #1
    b.ne .Lbtv_done              // unsigned:
   masked value is the result
```

```
    // signed: sign-extend from `width` bits via
   SHL then ASHR by 64-width.
    mov x0, #64
    sub x0, x0, x22
    mov x20, x0                    // shift amount =
    64 - width
    bl _kcc_ir_emit_const
    mov x1, x21
    mov x2, x0
    mov x0, #IR_SHL
    bl _kcc_ir_emit_binop
    mov x21, x0
    mov x0, x20
    bl _kcc_ir_emit_const
    mov x1, x21
    mov x2, x0
    mov x0, #IR_ASHR
    bl _kcc_ir_emit_binop
    mov x21, x0
.Lbtv_done:
    mov x0, x21
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_ir_assign_through ---
// uint64_t kcc_ir_assign_through(addr, rhs,
   op_tok, field, type) → result vreg
// Stores `rhs` (simple `=`) or `old op rhs` (
   compound) through the address
// vreg `addr`.  `field` non-null with a packed
   AST_F2 selects a bit-field
// read-modify-write; otherwise `type` gives the
   scalar access width.  This is
// the load-modify-store any address-based lvalue
   (member, subscript, deref)
// needs for compound assignment.
.globl _kcc_ir_assign_through
.p2align 2
_kcc_ir_assign_through:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                    // addr
    mov x20, x1                    // rhs (later
   reused for the value to store)
    mov x21, x2                    // op token
    mov x22, x3                    // field decl (0
   if none)
    mov x23, x4                    // scalar type (
   lvalue type L)
    str x5, [sp, #72]              // rhs type R (
   for compound-assign conversions)
    mov x24, #0                    // is-bit-field?
    cbz x22, .Lat_op
    ldr x0, [x22, #AST_F2]
    cbz x0, .Lat_op
    mov x24, #1
.Lat_op:
    cmp x21, #TOK_ASSIGN
    b.ne .Lat_compound
    // simple assignment: store rhs directly
    cbnz x24, .Lat_simple_bf
    mov x0, x23
    bl _kcc_ir_store_width
    mov x2, x0
    mov x0, x20                    // value
    mov x1, x19                    // addr
    bl _kcc_ir_emit_store
    b .Lat_ret
.Lat_simple_bf:
    mov x0, x19
    mov x1, x20
    mov x2, x22
    bl _kcc_ir_bitfield_store
    b .Lat_bf_result
.Lat_compound:
    // load the current value
    cbnz x24, .Lat_load_bf
    mov x0, x23
    bl _kcc_ir_load_width
    mov x1, x0
    mov x0, x19
    bl _kcc_ir_emit_load
    b .Lat_haveold
.Lat_load_bf:
    mov x0, x19
    mov x1, x22
    bl _kcc_ir_bitfield_load
.Lat_haveold:
    str x0, [sp, #64]              // stash old
   value vreg
    mov x0, x21                    // op token
    bl _kcc_ir_tok_to_binop      // x0 = IR opcode
    ldr x1, [sp, #64]            // old
    mov x2, x23                    // L (lvalue type
   )
    mov x3, x20                    // rhs
    ldr x4, [sp, #72]            // R (rhs type)
    bl _kcc_ir_compound_apply     // new = old OP
   rhs, under usual conversions
    mov x20, x0                    // value to store
    = new
    cbnz x24, .Lat_store_bf
    mov x0, x23
    bl _kcc_ir_store_width
    mov x2, x0
    mov x0, x20
    mov x1, x19
    bl _kcc_ir_emit_store
    b .Lat_ret
.Lat_store_bf:
    mov x0, x19
    mov x1, x20
    mov x2, x22
    bl _kcc_ir_bitfield_store
.Lat_bf_result:
    // The assignment expression's value is the
   truncated stored value, not the
    // raw operand: `(signed:11) = 1081` yields
   -967, not 1081.
    mov x0, x20
    mov x1, x22                    // field
    bl _kcc_ir_bf_trunc_value
    mov x20, x0
.Lat_ret:
    mov x0, x20                    // result of the
   assignment expression
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

359 · used in 30b · uses _kcc_arena_alloc 58a, _kcc_conv_promote 295a, _kcc_conv_usual 295b, _kcc_ir_alloc_instr 303a, _kcc_ir_convert 343, _kcc_ir_elem_off_width 414, _kcc_ir_emit 304, _kcc_ir_emit_indexed_store 414, _kcc_ir_load_width 349b, _kcc_ir_next_vreg 305a, _kcc_ir_norm32 375, _kcc_ir_store_width 349b, _kcc_ir_type_is_fp 343, _kcc_sema_resolve_type 259b, _kcc_type_layout 267, 268,

_kcc_type_long_u 250b

Struct tests verify field write, field read, multiple fields, and structs with initializers.

⟨ *Test: codegen structs* 366 ⟩ ≡

```
echo ""
echo "--- Structs ---"

run_test "struct-write-read" \
    "struct S { int x; int y; }; int main(void) {
   struct S s; s.x = 10; return s.x; }" 10

run_test "struct-two-fields" \
    "struct S { int x; int y; }; int main(void) {
   struct S s; s.x = 3; s.y = 7; return s.x + s.y;
    }" 10

run_test "struct-three-fields" \
    "struct S { int a; int b; int c; }; int main(
   void) { struct S s; s.a = 1; s.b = 2; s.c = 3;
   return s.a + s.b + s.c; }" 6

run_test "struct-overwrite" \
    "struct S { int x; int y; }; int main(void) {
   struct S s; s.x = 5; s.x = 9; return s.x; }" 9

run_test "struct-independent" \
    "struct S { int x; int y; }; int main(void) {
   struct S s; s.x = 10; s.y = 20; return s.y; }"
   20

run_test "struct-arithmetic" \
    "struct P { int x; int y; }; int main(void) {
   struct P p; p.x = 3; p.y = 4; return p.x * p.x
   + p.y * p.y; }" 25

run_test "struct-ptr-subscript" \
    "void *mmap(void*,long,int,int,int,long);
   struct S { int a; int b; }; int main(void) {
   struct S *p = (struct S*)mmap
   (0,4096,3,4098,-1,0); p[0].a=10; p[0].b=20; p
   [1].a=30; p[1].b=40; return p[0].a + p[1].b; }"
    50

run_test "struct-ptr-sub-loop" \
    "void *mmap(void*,long,int,int,int,long);
   struct V { int x; int y; }; int main(void) {
   struct V *v = (struct V*)mmap
   (0,4096,3,4098,-1,0); v[0].x=1; v[1].x=2; v[2].
   x=3; int s=0; int i=0; while(i<3){s=s+v[i].x; i
   =i+1;} return s; }" 6

run_test "struct-arrow-write-read" \
    "struct S { int x; int y; }; void set(struct S
    *p) { p->x=10; p->y=20; } int main(void) {
   struct S s; set(&s); return s.x + s.y; }" 30

run_test "struct-arrow-second-field" \
    "struct S { int a; int b; int c; }; int get(
   struct S *p) { return p->c; } int main(void) {
   struct S s; s.a=1; s.b=2; s.c=3; return get(&s)
   ; }" 3

run_test "struct-arrow-overwrite" \
    "struct S { int x; int y; }; void f(struct S *
   p) { p->x=5; p->x=9; } int main(void) { struct
   S s; f(&s); return s.x; }" 9
```

□

366 · used in 33b

29.9.7. Identifier (Variable Load)

Loading a variable is a two-step lookup. First the builder searches the `_kcc_ir_locals` hash table, which maps variable names to alloca vreg numbers (stored as `vreg+1` so that zero can serve as "not found"). If the name is found, a single `IR_LOAD` from the alloca vreg suffices.

Local *arrays* receive the same decay treatment. The locals hash table tags pointer variables with bit 63; non-pointer locals that have a `TY_PTR` resolved type (stored in `AST_F3` by sema's array decay) must be arrays whose alloca vreg *is* the decayed pointer. The builder returns it directly instead of loading from it.

If the name is absent from locals, the variable must be a global. For scalar and pointer globals, the builder emits `IR_GLOBAL_REF` followed by `IR_LOAD`—two instructions. Global *arrays* are different: the address produced by `GLOBAL_REF` already represents the decayed pointer to element 0, so no load is needed; the builder checks the symbol table for `TY_ARRAY` and returns the address vreg directly.

⟨ *IR: build ident* 367 ⟩ ≡

```
// -- kcc_ir_build_ident ---
// uint64_t kcc_ir_build_ident(node *ident)
//   x0 = AST_IDENT_EXPR (F0=name_ptr)
// Looks up the variable in locals and emits a
   load.
.text
.globl _kcc_ir_build_ident
.p2align 2
_kcc_ir_build_ident:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    mov x19, x0
    ldr x20, [x19, #AST_F0]      // name ptr
    cbz x20, .Libi_zero

    // Look up in locals hash table
    mov x0, x20
    bl _kcc_strlen
    mov x21, x0                  // keylen

    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x20                  // key
    mov x2, x21                  // keylen
    bl _kcc_ht_get
    // x0 = value (vreg+1) or 0 if not found
    cbz x0, .Libi_global
    // Bit 63 marks pointer variables; arrays have
    no tag
    tbnz x0, #63, .Libi_local_ptr
    // Non-pointer local (scalar or array)
    sub x22, x0, #1              // x22 = alloca
vreg
    // Detect array decay.  Sema usually rewrites
a decayed array's node type to
    // TY_PTR, but not in every context: as an
aggregate-initializer element
```

```
    // (`S s = {a, 0}`) the node keeps its
    TY_ARRAY type.  An array identifier in
    // a value position ALWAYS decays to its
    element-0 address (its alloca vreg)
    // and is never loaded, so accept both TY_PTR
    and TY_ARRAY here.  Reading
    // AST_F3 alone once loaded `a[0]` as the
    initializer value and stored a
    // 4-byte int where a pointer field belonged,
    so the struct member pointed
    // at garbage.
    ldr x0, [x19, #AST_F3]
    cbz x0, .Libi_local_load
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_PTR
    b.eq .Libi_local_array
    cmp x1, #TY_ARRAY
    b.eq .Libi_local_array
    b .Libi_local_load

.Libi_local_ptr:
    bic x0, x0, #0x8000000000000000
    sub x22, x0, #1                 // x22 = alloca
    vreg

.Libi_local_load:
    // Load from the alloca with a width+sign
    override matching the variable's
    // declared type: a narrow variable was stored
    with its own width (e.g. a
    // char as one byte), so it must be reloaded
    and sign/zero-extended the same
    // way, not read as a raw 8-byte slot (which
    would pick up stale bytes).
    ldr x0, [x19, #AST_F3]        // variable type
    bl _kcc_ir_load_width
    mov x21, x0                   // width+sign
    override
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    // The load's type is normally the canonical
    int; but a volatile variable
    // must be tagged so the optimizer never
    merges or drops the access, so use
    // its (volatile-qualified) descriptor in that
    case.
    ldr x1, [x19, #AST_F3]        // variable type
    cmp x1, #0x1000
    b.lo .Libi_ll_intty
    ldr x2, [x1, #TY_QUALS]
    tst x2, #2
    b.ne .Libi_ll_havety
.Libi_ll_intty:
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
.Libi_ll_havety:
    str x1, [x0, #IR_TYPE]
    str x22, [x0, #IR_SRC1]       // src = alloca
    vreg
    str x21, [x0, #IR_SRC2]       // access-size
    override

    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x20, x0                    // save result
    vreg

    mov x0, x19
    bl _kcc_ir_emit

    mov x0, x20
    b .Libi_done

.Libi_local_array:
    // Local array: alloca vreg IS the decayed
    pointer - no load
    mov x0, x22
    b .Libi_done

.Libi_global:
    // static local hoisted to a global? redirect
    onto its mangled symbol
    adrp x0, _kcc_ir_statics@PAGE
    add  x0, x0, _kcc_ir_statics@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Libi_global_notstatic
    mov x1, x20                   // key = source
    name
    mov x2, x21                   // keylen
    bl _kcc_ht_get
    cbz x0, .Libi_global_notstatic
    mov x22, x0                   // x22 = mangled
    name (source name stays in x20)

    // A static local array (or function) decays
    to its address, not a load.
    // Consult the parallel type table (the block
    scope is gone, so AST_F3's
    // decayed type is not reliable here) to route
    arrays through the array path.
    adrp x0, _kcc_ir_static_types@PAGE
    add  x0, x0, _kcc_ir_static_types@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Libi_gstat_scalar
    mov x1, x20                   // source name
    mov x2, x21                   // keylen
    bl _kcc_ht_get
    cbz x0, .Libi_gstat_scalar
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.eq .Libi_gstat_array
    cmp x1, #TY_FUNC
    b.eq .Libi_gstat_array
.Libi_gstat_scalar:
    mov x20, x22                  // mangled name
    → x20
    b .Libi_global_scalar
.Libi_gstat_array:
    mov x20, x22                  // mangled name
    → x20
    b .Libi_global_array

.Libi_global_notstatic:
    // Not a local - check if it's an array (
    address = decayed pointer)
    mov x0, x20                   // name
    mov x1, x21                   // len
    bl _kcc_symtab_lookup
    cbz x0, .Libi_global_scalar
    // An enumeration constant reaches here as a
    bare identifier (sema folds
    // enumerators in scalar contexts but not
    inside an aggregate initializer's
    // element list); emit its value as an
    IR_CONST, never a symbol reference.
    ldr x2, [x0, #SYM_KIND]
    cmp x2, #SYM_ENUM_CONST
    b.eq .Libi_enum_const
    ldr x1, [x0, #SYM_TYPE]
    cbz x1, .Libi_global_scalar
    ldr x2, [x1, #TY_KIND]
    cmp x2, #TY_ARRAY
    b.eq .Libi_global_array
    cmp x2, #TY_FUNC
    b.eq .Libi_global_array
    b .Libi_global_scalar
```

```
.Libi_enum_const:
    ldr x22, [x0, #SYM_VALUE]     // enumerator
   value
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x19, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x19, #IR_TYPE]
    str x22, [x19, #IR_SRC1]      // value
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x20, x0
    mov x0, x19
    bl _kcc_ir_emit
    mov x0, x20
    b .Libi_done

.Libi_global_scalar:
    // Scalar/pointer global: emit global_ref +
   load.  Capture the variable's
    // load width+sign first (x19 still points at
   the ident node here).
    ldr x0, [x19, #AST_F3]
    bl _kcc_ir_load_width
    mov x21, x0                   // width+sign
   override
    // The load's type: the variable's (volatile-
   qualified) descriptor when it
    // is volatile, else the canonical int.  Held
   in x23 across the global_ref.
    ldr x23, [x19, #AST_F3]
    cmp x23, #0x1000
    b.lo .Libi_gs_intty
    ldr x0, [x23, #TY_QUALS]
    tst x0, #2
    b.ne .Libi_gs_haveity         // volatile: keep
    the qualified type
    // Floating: keep the real type so the back
   end emits an FP load rather
    // than an integer one---otherwise a global `
   double`/`float` read loaded
    // its bytes into a general register and every
    use saw garbage.
    ldr x0, [x23, #TY_KIND]
    cmp x0, #TY_FLOAT
    b.eq .Libi_gs_haveity
    cmp x0, #TY_DOUBLE
    b.eq .Libi_gs_haveity
.Libi_gs_intty:
    adrp x23, _kcc_type_int_s@PAGE
    add  x23, x23, _kcc_type_int_s@PAGEOFF
    ldr x23, [x23]
.Libi_gs_haveity:

    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x20, [x0, #IR_SRC1]       // name ptr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]

    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x22, x0                   // x22 = address
   vreg

    mov x0, x19
    bl _kcc_ir_emit

    // Emit load from the global address (width+
   sign-aware, like locals)
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    str x23, [x0, #IR_TYPE]       // volatile var
   type when volatile, else int
    str x22, [x0, #IR_SRC1]       // src = address
   vreg
    str x21, [x0, #IR_SRC2]       // access-size
   override

    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x20, x0                    // x20 = loaded
   value vreg

    mov x0, x19
    bl _kcc_ir_emit

    mov x0, x20
    b .Libi_done

.Libi_global_array:
    // Array global: address IS the decayed
   pointer - no load needed
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x20, [x0, #IR_SRC1]
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]

    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x20, x0

    mov x0, x19
    bl _kcc_ir_emit

    mov x0, x20
    b .Libi_done

.Libi_zero:
    mov x0, #0

.Libi_done:
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

367 · used in 30b · defines _kcc_ir_build_ident 340 · uses _kcc_ht_get 62, _kcc_ir_alloc_instr 303a, _kcc_ir_emit 304, _kcc_ir_load_width 349b, _kcc_ir_locals 305c, _kcc_ir_next_vreg 305a, _kcc_ir_statics 305c, _kcc_ir_static_types 305c, _kcc_symtab_lookup 255, _kcc_type_int_s 250b

29.9.8. Binary Expression

Binary expressions follow one of two paths. For ordinary operators (+, −, *, etc.) the builder evaluates both sides, maps the AST token to an IR opcode, and emits a single instruction. For the short-circuit logical operators && and ||, it must generate control flow: evaluating the right-hand side only when the left-hand side does not already determine the result.

Ordinary operators. The builder evaluates LHS and RHS first (in that order), then dispatches through a token-to-opcode table covering arithmetic (`+ - * / %`), bitwise (`& | ^ << >>`), and relational (`== != < <= > >=`) operators. Each case sets x23 to the corresponding IR opcode and falls through to the common emitter. Before allocating the instruction, the emitter checks the left operand's type: if unsigned, it remaps signed opcodes to their unsigned variants (`SDIV`→`UDIV`, `SMOD`→`UMOD`, `CMP_SLT`→`CMP_ULT`, `CMP_SLE`→`CMP_ULE`).

Pointer arithmetic scaling. When `+` or `-` involves a pointer operand, the C standard requires the integer operand to be scaled by `sizeof(*ptr)` (§6.5.6). The builder inspects each operand's sema type; if one is `TY_PTR`, it emits `IR_CONST` + `IR_MUL` to scale the non-pointer operand by the pointee's `TY_SIZE`. If the pointee size is ≤ 1 (e.g. `char *`), no scaling is needed.

Floating-point operations and single-precision rounding. If either operand or the result is floating, the emitter takes the FP path: any integer operand is widened to `double` with `IR_SITOFP`, and the operation is emitted carrying the `double` type, which is the code generator's signal to use the floating-point unit. Values therefore *live* as doubles in SIMD registers regardless of their C type—a uniform model that keeps the backend simple. The catch is `float`: Apple ARM64 uses `FLT_EVAL_METHOD` 0, so every `float` operation must yield a result rounded to single precision, not a wider double carried on into the next operation. So when the expression's *own* result type is `float`, the emitter appends an `IR_FPROUND` (a `double`→`single`→`double` round-trip) to the result, rounding each `float` sub-expression exactly once as it is produced. A chain like `a/b*c` thus rounds `a/b` to single before multiplying, matching `clang` to the last bit; a comparison (result type `int`) and a mixed `float`/`double` expression (result type `double`, full precision kept) are both correctly left un-rounded.

Result-rounding is not enough for a *comparison* whose operands are `float`: its result is `int`, so nothing rounds the operands, yet the usual arithmetic conversions (§6.3.1.8) put the comparison in the *float* domain when neither operand is `double`. An integer promoted only to `double` would keep more precision than a `float` holds—`16777217L != (float)16777217e0` would read *true* because the long stayed the exact `16777217.0` while the float rounded to `16777216.0f`. So when the *other* operand is `float`, the emitter rounds the just-promoted integer to single before the operation; when the other operand is `double`, the common type is `double` and the exact value is right, so it is left alone.

⟨ *IR: build binary* 369 ⟩ ≡

```
// -- kcc_ir_build_binary ---
// uint64_t kcc_ir_build_binary(node *expr)
//   x0 = AST_BINARY_EXPR (F0=op token, F1=lhs, F2=rhs)
// Returns vreg.
.text
.globl _kcc_ir_build_binary
.p2align 2
_kcc_ir_build_binary:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]

    mov x19, x0                     // AST node
    ldr x20, [x19, #AST_F0]         // op token code

    // Check for short-circuit operators
    cmp x20, #TOK_AND
    b.eq .Libb_short_and
    cmp x20, #TOK_OR
    b.eq .Libb_short_or

    // Build LHS
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_expr
    mov x21, x0                     // x21 = lhs vreg

    // Build RHS
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_build_expr
    mov x22, x0                     // x22 = rhs vreg

    // Map token to IR opcode
    cmp x20, #TOK_PLUS
    b.eq .Libb_op_add
    cmp x20, #TOK_MINUS
    b.eq .Libb_op_sub
    cmp x20, #TOK_STAR
    b.eq .Libb_op_mul
    cmp x20, #TOK_SLASH
    b.eq .Libb_op_sdiv
    cmp x20, #TOK_PERCENT
    b.eq .Libb_op_smod
    cmp x20, #TOK_AMP
    b.eq .Libb_op_and
    cmp x20, #TOK_PIPE
    b.eq .Libb_op_or
    cmp x20, #TOK_CARET
    b.eq .Libb_op_xor
    cmp x20, #TOK_SHL
    b.eq .Libb_op_shl
    cmp x20, #TOK_SHR
    b.eq .Libb_op_ashr
    cmp x20, #TOK_EQ
    b.eq .Libb_op_eq
    cmp x20, #TOK_NE
    b.eq .Libb_op_ne
    cmp x20, #TOK_LT
    b.eq .Libb_op_slt
    cmp x20, #TOK_LE
    b.eq .Libb_op_sle
    cmp x20, #TOK_GT
    b.eq .Libb_op_sgt
    cmp x20, #TOK_GE
    b.eq .Libb_op_sge

    // Default: add
    b .Libb_op_add

.Libb_op_add:  mov x23, #IR_ADD
    b .Libb_ptr_scale
.Libb_op_sub:  mov x23, #IR_SUB
    b .Libb_ptr_scale

// Pointer arithmetic: ptr+int or int+ptr must scale the integer
// operand by the pointed-to type's real byte size.  x23 = opcode
// (ADD or SUB), x21 = LHS vreg, x22 = RHS vreg.
```

```
    Since arrays are now
// packed at their natural stride, the scale is
    the exact pointee size
// (no clamp to 8).  A TY_ARRAY operand (an
    undecayed struct-member array)
// is treated the same as a pointer, with its
    element type in TY_F0.
// After scaling, falls through to the emitter.
.Libb_ptr_scale:
    // Check if LHS is a pointer (or a still-
    undecayed array) type.  A struct
    // member of array type, e.g. `state->lens + n
    `, keeps its TY_ARRAY type
    // because the member-access sema path does
    not apply array-to-pointer
    // decay (an ident array does); its IR value
    is nevertheless the element-0
    // address, so it must be scaled exactly like
    a pointer.  For both TY_PTR
    // and TY_ARRAY the element/pointee type is
    TY_F0, so the scaling below is
    // identical.  Without this, `member_array + n
    ` added an unscaled index and
    // read at the wrong stride (the bug that
    corrupted inflate's distance
    // lengths so zlib decode failed).
    ldr x0, [x19, #AST_F1]
    cbz x0, .Libb_emit
    ldr x0, [x0, #AST_F3]
    cbz x0, .Libb_ps_check_rhs
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_PTR
    b.eq .Libb_ps_lhs_ptr
    cmp x1, #TY_ARRAY
    b.ne .Libb_ps_check_rhs
.Libb_ps_lhs_ptr:

    // LHS is a pointer.  ptr - ptr is a pointer
    difference: fall through to
    // the ptrdiff handler; ptr + int / ptr - int
    scales the integer.
    cmp x23, #IR_SUB
    b.ne .Libb_ps_scale_rhs
    ldr x2, [x19, #AST_F2]
    cbz x2, .Libb_ps_scale_rhs
    ldr x2, [x2, #AST_F3]
    cbz x2, .Libb_ps_scale_rhs
    ldr x2, [x2, #TY_KIND]
    cmp x2, #TY_PTR
    b.eq .Libb_ptr_diff
    cmp x2, #TY_ARRAY
    b.eq .Libb_ptr_diff
    b .Libb_ps_scale_rhs

.Libb_ps_check_rhs:
    // Check if RHS is a pointer (or undecayed
    array) type (int + ptr)
    ldr x0, [x19, #AST_F2]
    cbz x0, .Libb_emit
    ldr x0, [x0, #AST_F3]
    cbz x0, .Libb_emit
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_PTR
    b.eq .Libb_ps_rhs_ptr
    cmp x1, #TY_ARRAY
    b.ne .Libb_emit
.Libb_ps_rhs_ptr:

    // RHS is pointer - scale LHS (x21) by the
    real pointee size
    ldr x0, [x0, #TY_F0]
    cbz x0, .Libb_emit
    bl _kcc_type_layout
    mov x24, x0
    cmp x24, #1
    b.le .Libb_emit

    // Emit const + mul to scale x21
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x20, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x20, #IR_TYPE]
    str x24, [x20, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x24, x0
    mov x0, x20
    bl _kcc_ir_emit

    mov x0, #IR_MUL
    bl _kcc_ir_alloc_instr
    mov x20, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x20, #IR_TYPE]
    str x21, [x20, #IR_SRC1]
    str x24, [x20, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x21, x0
    mov x0, x20
    bl _kcc_ir_emit
    b .Libb_emit

.Libb_ps_scale_rhs:
    // LHS is pointer - scale RHS (x22) by the
    real pointee size
    ldr x0, [x0, #TY_F0]
    cbz x0, .Libb_emit
    bl _kcc_type_layout
    mov x24, x0
    cmp x24, #1
    b.le .Libb_emit

    // Emit const + mul to scale x22
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x20, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x20, #IR_TYPE]
    str x24, [x20, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x24, x0
    mov x0, x20
    bl _kcc_ir_emit

    mov x0, #IR_MUL
    bl _kcc_ir_alloc_instr
    mov x20, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x20, #IR_TYPE]
    str x22, [x20, #IR_SRC1]
    str x24, [x20, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x22, x0
    mov x0, x20
    bl _kcc_ir_emit
    b .Libb_emit
```

```
.Libb_ptr_diff:
    // Pointer difference: (lhs - rhs) / stride.
   The stride is the real
    // pointee size, mirroring the ptr+int scaling
    so the two are consistent.
    ldr x0, [x19, #AST_F1]
    ldr x0, [x0, #AST_F3]
    ldr x0, [x0, #TY_F0]            // pointee type
    mov x24, #1
    cbz x0, .Libb_pd_have
    bl _kcc_type_layout             // real pointee
    size (matches ptr+int stride)
    mov x24, x0
    cbnz x24, .Libb_pd_have
    mov x24, #1
.Libb_pd_have:
    // SUB: byte difference of the two addresses
    mov x0, #IR_SUB
    bl _kcc_ir_alloc_instr
    mov x20, x0
    adrp x1, _kcc_type_long_s@PAGE
    add  x1, x1, _kcc_type_long_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x20, #IR_TYPE]
    str x21, [x20, #IR_SRC1]
    str x22, [x20, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x21, x0                     // x21 = byte
   difference
    mov x0, x20
    bl _kcc_ir_emit
    // CONST stride
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x20, x0
    adrp x1, _kcc_type_long_s@PAGE
    add  x1, x1, _kcc_type_long_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x20, #IR_TYPE]
    str x24, [x20, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x22, x0                     // x22 = stride
    mov x0, x20
    bl _kcc_ir_emit
    // SDIV: element count = byte difference /
   stride
    mov x0, #IR_SDIV
    bl _kcc_ir_alloc_instr
    mov x20, x0
    adrp x1, _kcc_type_long_s@PAGE
    add  x1, x1, _kcc_type_long_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x20, #IR_TYPE]
    str x21, [x20, #IR_SRC1]
    str x22, [x20, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x23, x0
    mov x0, x20
    bl _kcc_ir_emit
    mov x0, x23
    b .Libb_done

.Libb_op_mul:  mov x23, #IR_MUL
    b .Libb_emit
.Libb_op_sdiv: mov x23, #IR_SDIV
    b .Libb_emit
.Libb_op_smod: mov x23, #IR_SMOD
    b .Libb_emit
.Libb_op_and:  mov x23, #IR_AND
    b .Libb_emit
.Libb_op_or:   mov x23, #IR_OR
    b .Libb_emit
.Libb_op_xor:  mov x23, #IR_XOR
    b .Libb_emit
.Libb_op_shl:  mov x23, #IR_SHL
    b .Libb_emit
.Libb_op_ashr: mov x23, #IR_ASHR
    b .Libb_emit
.Libb_op_eq:   mov x23, #IR_CMP_EQ
    b .Libb_emit
.Libb_op_ne:   mov x23, #IR_CMP_NE
    b .Libb_emit
.Libb_op_slt:  mov x23, #IR_CMP_SLT
    b .Libb_emit
.Libb_op_sle:  mov x23, #IR_CMP_SLE
    b .Libb_emit
.Libb_op_sgt:  mov x23, #IR_CMP_SGT
    b .Libb_emit
.Libb_op_sge:  mov x23, #IR_CMP_SGE
    b .Libb_emit

.Libb_emit:
    // Floating-point operation? If the result or
   either operand is
    // float/double, take the FP path (promote int
    operands, emit with
    // double type so codegen uses the floating-
   point unit).
    ldr x0, [x19, #AST_F3]
    bl _kcc_ir_type_is_fp
    cbnz x0, .Libb_fp_emit
    ldr x0, [x19, #AST_F1]
    cbz x0, .Libb_chk_rhs_fp
    ldr x0, [x0, #AST_F3]
    bl _kcc_ir_type_is_fp
    cbnz x0, .Libb_fp_emit
.Libb_chk_rhs_fp:
    ldr x0, [x19, #AST_F2]
    cbz x0, .Libb_emit_int
    ldr x0, [x0, #AST_F3]
    bl _kcc_ir_type_is_fp
    cbnz x0, .Libb_fp_emit

.Libb_emit_int:
    // A relational comparison is unsigned when
   its COMMON type is unsigned
    // (the usual arithmetic conversions), which
   depends on BOTH operands ---
    // `int < unsigned` is an unsigned compare
   even though the left side is
    // signed.  Division, modulo and the
   arithmetic-vs-logical shift instead
    // key off the left operand alone, handled in
   the block below.
    cmp x23, #IR_CMP_SLT
    b.lo .Libb_int_left
    cmp x23, #IR_CMP_SGE
    b.hi .Libb_int_left
    ldr x0, [x19, #AST_F1]
    ldr x1, [x19, #AST_F2]
    bl _kcc_ir_cmp_unsigned
    cbz x0, .Libb_emit_alloc      // signed common
   type → keep signed opcode
    cmp x23, #IR_CMP_SLT
    b.eq .Libb_u_lt
    cmp x23, #IR_CMP_SLE
    b.eq .Libb_u_le
    cmp x23, #IR_CMP_SGT
    b.eq .Libb_u_gt
    b .Libb_u_ge                  // IR_CMP_SGE

.Libb_int_left:
    // Remap signed div/mod/shift to unsigned by
   the RESULT type's signedness,
    // not the raw left operand's.  Division and
   remainder use the operands'
```

```
    // common type after the usual arithmetic
   conversions, and a shift uses the
    // promoted left operand (\stdref{6.5.5}, \
   stdref{6.5.7}); both promote a
    // narrow operand to (signed) int first.  So `
   unsigned char / short` divides
    // in signed int -- reading the raw `unsigned
   char` here made it an unsigned
    // divide, and 50 / (unsigned)-5 folded to 0
   instead of -10.
    ldr x0, [x19, #AST_F3]      // result type
   sema assigned to this expression
    cbz x0, .Libb_emit_alloc
    ldr x0, [x0, #TY_FO]
    cbnz x0, .Libb_emit_alloc

    cmp x23, #IR_SDIV
    b.eq .Libb_u_div
    cmp x23, #IR_SMOD
    b.eq .Libb_u_mod
    cmp x23, #IR_ASHR
    b.eq .Libb_u_shr
    b .Libb_emit_alloc

.Libb_u_div:
    mov x23, #IR_UDIV
    b .Libb_emit_alloc
.Libb_u_mod:
    mov x23, #IR_UMOD
    b .Libb_emit_alloc
.Libb_u_lt:
    mov x23, #IR_CMP_ULT
    b .Libb_emit_alloc
.Libb_u_le:
    mov x23, #IR_CMP_ULE
    b .Libb_emit_alloc
.Libb_u_gt:
    // No unsigned-greater opcode: `a > b` (
   unsigned) is `b < a`, so swap the
    // operands and emit CMP_ULT.  Without this, a
    size_t compare with the high
    // bit set (e.g. near SIZE_MAX) used a signed
   condition and read backwards.
    mov x0, x21
    mov x21, x22
    mov x22, x0
    mov x23, #IR_CMP_ULT
    b .Libb_emit_alloc
.Libb_u_ge:
    // `a >= b` (unsigned) is `b <= a`: swap
   operands, emit CMP_ULE.
    mov x0, x21
    mov x21, x22
    mov x22, x0
    mov x23, #IR_CMP_ULE
    b .Libb_emit_alloc
.Libb_u_shr:
    // Unsigned `>>` is a LOGICAL shift (zero-fill
   ), not arithmetic (sign-fill).
    // A 32-bit unsigned value works either way (
   zero-extended, bit 63 clear),
    // but a 64-bit unsigned with the high bit set
    (e.g. a hash or size_t)
    // sign-extends wrongly under ASHR.
    mov x23, #IR_LSHR
    b .Libb_emit_alloc

.Libb_emit_alloc:
    mov x25, x23                // preserve
   opcode across the alloc
    mov x0, x23
    bl _kcc_ir_alloc_instr
    mov x24, x0

    // Type: default i32
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr x0, [x0]
    str x0, [x24, #IR_TYPE]
    str x21, [x24, #IR_SRC1]
    str x22, [x24, #IR_SRC2]

    // For an integer comparison OR a divide/
   remainder, record the operands'
    // common width so codegen runs at 32 or 64
   bits.  Two <=32-bit operands use
    // 32 bits, reconciling a sign-extended signed
    operand with a zero-extended
    // unsigned one (e.g. `int` against `0
   xffffffffu`); any wider operand keeps
    // the 64-bit width.  Width matters for a
   divide because a 64-bit unsigned
    // divide sees a signed operand's sign-
   extension (`i % u`, i<0) --- so an
    // `unsigned int` remainder MUST run in 32
   bits, and an `unsigned long` one
    // in 64.  An unknown operand width defaults
   to the wider type.
    sub x0, x25, #IR_CMP_EQ
    cmp x0, #(IR_CMP_ULE - IR_CMP_EQ)
    b.ls .Libb_want_width       // comparison
    sub x0, x25, #IR_SDIV
    cmp x0, #(IR_UMOD - IR_SDIV)
    b.hi .Libb_cmp_type_done    // neither
   compare nor div/mod → keep i32
.Libb_want_width:
    mov x2, #8                  // lhs width (
   default wide)
    ldr x0, [x19, #AST_F1]
    cbz x0, .Libb_cmp_w_rhs
    ldr x0, [x0, #AST_F3]
    cbz x0, .Libb_cmp_w_rhs
    ldr x2, [x0, #TY_SIZE]
.Libb_cmp_w_rhs:
    mov x3, #8                  // rhs width (
   default wide)
    ldr x0, [x19, #AST_F2]
    cbz x0, .Libb_cmp_w_max
    ldr x0, [x0, #AST_F3]
    cbz x0, .Libb_cmp_w_max
    ldr x3, [x0, #TY_SIZE]
.Libb_cmp_w_max:
    cmp x3, x2
    csel x2, x3, x2, hi         // x2 = max(lhs,
   rhs) width
    cmp x2, #4
    b.ls .Libb_cmp_type_done    // both <=32-bit
   → keep i32 (32-bit compare)
    adrp x0, _kcc_type_long_s@PAGE
    add  x0, x0, _kcc_type_long_s@PAGEOFF
    ldr x0, [x0]
    str x0, [x24, #IR_TYPE]     // 8-byte type →
   64-bit compare
.Libb_cmp_type_done:

    bl _kcc_ir_next_vreg
    str x0, [x24, #IR_DEST]
    mov x23, x0                 // result vreg

    mov x0, x24
    bl _kcc_ir_emit

    // Truncate 32-bit integer results of width-
   growing ops to their width
    mov x2, #0
    ldr x0, [x19, #AST_F1]
    cbz x0, .Libb_norm_lhs
    ldr x2, [x0, #AST_F3]       // lhs sema type
```

```
.Libb_norm_lhs:
    mov x3, #0
    ldr x0, [x19, #AST_F2]
    cbz x0, .Libb_norm_rhs
    ldr x3, [x0, #AST_F3]          // rhs sema type
.Libb_norm_rhs:
    mov x0, x23                    // result vreg
    mov x1, x25                    // opcode
    bl _kcc_ir_norm32
    mov x23, x0

    mov x0, x23
    b .Libb_done

.Libb_fp_emit:
    // Promote any integer operand to double, then
    emit the operation with
    // double type - the codegen signal to use
   floating-point instructions.
    // No 32-bit normalisation applies.
   Comparisons still yield an int 0/1
    // but are computed with fcmp.
    ldr x0, [x19, #AST_F1]
    ldr x0, [x0, #AST_F3]
    bl _kcc_ir_type_is_fp
    cbnz x0, .Libb_fp_rhs
    ldr x0, [x19, #AST_F1]
    ldr x1, [x0, #AST_F3]          // lhs integer
   type (signedness)
    mov x0, x21
    bl _kcc_ir_emit_int_to_fp
    mov x21, x0
    // If the OTHER operand is `float` (single),
   the operands' common type is
    // float, so this integer must round to single
    precision before the compare
    // --- `16777217L != (float)16777217e0` is
   false because the long rounds to
    // 16777216.0f, exactly as the float does.  A
   comparison's own result type is
    // int, so the result-rounding below never
   covers it; the operand must round
    // here.  When the other operand is `double`,
   the common type is double and
    // the exact integer-to-double value is
   correct, so we leave it.
    ldr x0, [x19, #AST_F2]
    cbz x0, .Libb_fp_rhs
    ldr x0, [x0, #AST_F3]
    cbz x0, .Libb_fp_rhs
    ldr x0, [x0, #TY_KIND]
    cmp x0, #8                     // TY_FLOAT
    b.ne .Libb_fp_rhs
    mov x0, x21
    bl _kcc_ir_emit_fpround
    mov x21, x0
.Libb_fp_rhs:
    ldr x0, [x19, #AST_F2]
    ldr x0, [x0, #AST_F3]
    bl _kcc_ir_type_is_fp
    cbnz x0, .Libb_fp_do
    ldr x0, [x19, #AST_F2]
    ldr x1, [x0, #AST_F3]          // rhs integer
   type (signedness)
    mov x0, x22
    bl _kcc_ir_emit_int_to_fp
    mov x22, x0
    // Symmetric single-rounding when the LHS
   operand is `float`.
    ldr x0, [x19, #AST_F1]
    cbz x0, .Libb_fp_do
    ldr x0, [x0, #AST_F3]
    cbz x0, .Libb_fp_do
    ldr x0, [x0, #TY_KIND]
    cmp x0, #8                     // TY_FLOAT
    b.ne .Libb_fp_do
    mov x0, x22
    bl _kcc_ir_emit_fpround
    mov x22, x0
.Libb_fp_do:
    mov x0, x23                    // opcode (ADD/
   SUB/MUL/SDIV or CMP_*)
    bl _kcc_ir_alloc_instr
    mov x24, x0
    adrp x0, _kcc_type_double@PAGE
    add  x0, x0, _kcc_type_double@PAGEOFF
    ldr x0, [x0]
    str x0, [x24, #IR_TYPE]        // double →
   floating-point codegen
    str x21, [x24, #IR_SRC1]
    str x22, [x24, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x24, #IR_DEST]
    mov x23, x0
    mov x0, x24
    bl _kcc_ir_emit
    // If this expression's own type is `float` (
   single precision), round the
    // just-computed double result back to single,
    so each float sub-expression
    // is rounded once as it is produced.  Without
    this the operation carries a
    // wider double intermediate into the next one
   ---FLT_EVAL_METHOD 1/2 rather
    // than the 0 that Apple ARM64 (and clang) use
   ---so a chain like `a/b*c`
    // differed from clang by an ULP.  A
   comparison's result type is int, so it
    // is naturally excluded; a mixed float/double
    expression is typed double
    // and correctly keeps full precision.
    ldr x1, [x19, #AST_F3]         // this
   expression's result type
    cbz x1, .Libb_fp_ret
    ldr x1, [x1, #TY_KIND]
    cmp x1, #TY_FLOAT
    b.ne .Libb_fp_ret
    mov x0, x23                    // double result
   vreg
    bl _kcc_ir_emit_fpround        // → single-
   rounded result
    mov x23, x0
.Libb_fp_ret:
    mov x0, x23
    b .Libb_done
```

□

369 · defined at 369, 374, 375, 376 · used in 30b · defines _kcc_ir_build_binary 340 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_cmp_unsigned 374, _kcc_ir_emit 304, _kcc_ir_emit_fpround 343, _kcc_ir_emit_int_to_fp 343, _kcc_ir_next_vreg 305a, _kcc_ir_norm32 375, _kcc_ir_type_is_fp 343, _kcc_type_double 250b, _kcc_type_int_s 250b, _kcc_type_layout 267, 268, _kcc_type_long_s 250b

Signedness of a comparison. Whether a relational comparison uses signed or unsigned condition codes follows the *common* type of its two operands, not the left one alone. `_kcc_ir_cmp_unsigned` implements the sign half of the usual arithmetic conversions (§6.3.1.8) for two integer operands. It first applies the integer promotions — an operand narrower than `int` becomes signed `int` — then,

at equal rank the result is unsigned if either operand is unsigned; at unequal rank the wider operand's signedness decides, since on this LP64 target a wider signed type represents every value of a narrower unsigned one. Promotion is what makes `(unsigned char)200 > (signed char)-1` a *signed* compare (both promote to `int`), answering true. It answers `signed` (0) for any non-integer operand — notably pointers, whose relational comparisons kcc keeps signed — so only genuine integer comparisons are remapped. Without this, `-1 > 1u` compared the signed -1 against 1 and read false, where C converts -1 to a large unsigned value and the answer is true.

⟨ *IR: build binary* 374 ⟩ + ≡

```
// -- kcc_ir_cmp_unsigned ---
// int kcc_ir_cmp_unsigned(node *lhs, node *rhs)
//   x0 = lhs AST node, x1 = rhs AST node
// Returns x0 = 1 when the comparison's common
   type is unsigned, else 0.
// Leaf routine: reads operand sema types (AST_F3)
   ; integer types carry a
// signed flag in TY_F0 (1 = signed, 0 = unsigned)
    and a byte width in TY_SIZE.
.text
.p2align 2
_kcc_ir_cmp_unsigned:
    cbz x0, .Lcmpu_signed
    ldr x2, [x0, #AST_F3]          // lhs type
    cbz x2, .Lcmpu_signed
    cbz x1, .Lcmpu_signed
    ldr x3, [x1, #AST_F3]          // rhs type
    cbz x3, .Lcmpu_signed
    // Both operands must be integer kinds (
   TY_BOOL(2)..TY_LLONG(7)).
    ldr x4, [x2, #TY_KIND]
    sub x4, x4, #2
    cmp x4, #5
    b.hi .Lcmpu_signed
    ldr x4, [x3, #TY_KIND]
    sub x4, x4, #2
    cmp x4, #5
    b.hi .Lcmpu_signed
    // Apply the integer promotions first: an
   operand narrower than int
    // (char, short, _Bool) promotes to signed int
   , because int holds all its
    // values.  Only types of int width or wider
   keep their own signedness.
    ldr x4, [x2, #TY_SIZE]         // size_l
    ldr x6, [x2, #TY_F0]           // sign_l (1=
   signed, 0=unsigned)
    cmp x4, #4
    b.hs .Lcmpu_l_promoted
    mov x4, #4
    mov x6, #1                     // promoted int
   is signed
.Lcmpu_l_promoted:
    ldr x5, [x3, #TY_SIZE]         // size_r
    ldr x7, [x3, #TY_F0]           // sign_r
    cmp x5, #4
    b.hs .Lcmpu_r_promoted
    mov x5, #4
    mov x7, #1
.Lcmpu_r_promoted:
    cmp x4, x5
    b.hi .Lcmpu_l_wider
    b.lo .Lcmpu_r_wider
    // equal rank: unsigned if either operand is
   unsigned
    cbz x6, .Lcmpu_unsigned
    cbz x7, .Lcmpu_unsigned
    b .Lcmpu_signed
.Lcmpu_l_wider:
    cbz x6, .Lcmpu_unsigned        // wider operand
   unsigned → unsigned
    b .Lcmpu_signed
.Lcmpu_r_wider:
    cbz x7, .Lcmpu_unsigned
    b .Lcmpu_signed
.Lcmpu_unsigned:
    mov x0, #1
    ret
.Lcmpu_signed:
    mov x0, #0
    ret
```

□

374 · defined at 369, 374, 375, 376 · used in 30b · defines _kcc_ir_cmp_unsigned 369

Truncating to the declared width. The all-spill backend computes in 64-bit registers, so an `int` or `unsigned int` sum that overflows 32 bits keeps its extra high bits—an unsigned wraparound past 2^{32} then differs from Standard C. `_kcc_ir_norm32` closes that gap for the operations that can *grow* a value past its width (`+`, `-`, `*`, `<<`): when both operands are integers no wider than `int`, it appends an `IR_SEXT` or `IR_ZEXT` (source width 4) that re-canonicalises the result to 32 bits—sign-extending a signed result, zero-extending an unsigned one. Operands wider than `int` (a `long`) or of pointer type fall outside the kind window and are left in 64 bits, so pointer arithmetic and `long` computations are untouched. For values that never leave 32-bit range the extension is a no-op, so nothing else changes.

⟨ *IR: build binary* 375 ⟩ + ≡

```
// -- kcc_ir_norm32 ---
// uint64_t kcc_ir_norm32(vreg, opcode, lhs_type,
   rhs_type)
//   x0 = op result vreg, x1 = IR opcode, x2 = lhs
    sema type, x3 = rhs sema type
// For width-growing ops (ADD/SUB/MUL/SHL/NEG/NOT)
    whose operands are both
// <=32-bit integers, emit IR_SEXT/IR_ZEXT to
   truncate the result to 32-bit
// width; returns the (possibly new) vreg.  Unary
   callers pass the operand
// type as both lhs_type and rhs_type.
.text
.globl _kcc_ir_norm32
.p2align 2
_kcc_ir_norm32:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                    // vreg
    mov x20, x1                    // opcode
    mov x21, x2                    // lhs type

    cmp x20, #IR_ADD
    b.eq .Lnorm_op_ok
    cmp x20, #IR_SUB
    b.eq .Lnorm_op_ok
    cmp x20, #IR_MUL
    b.eq .Lnorm_op_ok
    cmp x20, #IR_SHL
    b.eq .Lnorm_op_ok
```

```
    cmp x20, #IR_NEG
    b.eq .Lnorm_op_ok
    cmp x20, #IR_NOT
    b.eq .Lnorm_op_ok
    // Bitwise AND/OR/XOR also need re-
   canonicalising: they pass their operands'
    // upper bits straight through, so mixing a
   sign-extended operand into an
    // unsigned result leaves dirty upper bits.
   `0u ^ (short)-1` is unsigned int
    // 0xFFFFFFFF, but the short's sign extension
   made the register
    // 0xFFFF...FFFF; without a ZEXT here, a later
    widen or 64-bit compare read
    // it as -1 (the pr48197 / 960801 mixed-width
   bug).
    cmp x20, #IR_AND
    b.eq .Lnorm_op_ok
    cmp x20, #IR_OR
    b.eq .Lnorm_op_ok
    cmp x20, #IR_XOR
    b.eq .Lnorm_op_ok
    b .Lnorm_skip
.Lnorm_op_ok:

    // lhs: a <=32-bit integer? (TY_KIND in [
   TY_BOOL(2)..TY_INT(5)])
    cbz x21, .Lnorm_skip
    ldr x0, [x21, #TY_KIND]
    sub x0, x0, #2
    cmp x0, #3
    b.hi .Lnorm_skip

    // rhs: also a <=32-bit integer?
    cbz x3, .Lnorm_skip
    ldr x0, [x3, #TY_KIND]
    sub x0, x0, #2
    cmp x0, #3
    b.hi .Lnorm_skip

    // Signedness of the re-canonicalising
   extension.  An operand forces an
    // unsigned (zero-extended) result only if it
   is `unsigned int` or wider;
    // a narrower unsigned type (unsigned char/
   short) promotes to signed int
    // (§6.3.1.1) and so keeps the result signed.
    The test is therefore
    // `unsigned flag == 0 AND size >= 4`, not the
    flag alone.  A shift is a
    // further exception: it takes the signedness
   of its LEFT operand only
    // (§6.5.7), so `-1 << 1u` stays signed.  x20
   still holds the opcode, so
    // test it before reusing the register.
    ldr x0, [x21, #TY_FO]          // lhs signed
   flag (1 = signed, 0 = unsigned)
    cbnz x0, .Lnorm_lhs_ok         // signed → does
   not force an unsigned result
    ldr x0, [x21, #TY_SIZE]
    cmp x0, #4
    b.hs .Lnorm_unsigned           // unsigned int
   or wider → ZEXT
.Lnorm_lhs_ok:
    cmp x20, #IR_SHL
    b.eq .Lnorm_signed             // shift: the
   left operand alone decides → SEXT
    ldr x0, [x3, #TY_FO]           // rhs signed
   flag
    cbnz x0, .Lnorm_signed         // signed rhs →
   SEXT
    ldr x0, [x3, #TY_SIZE]
    cmp x0, #4
    b.hs .Lnorm_unsigned           // unsigned int
   or wider rhs → ZEXT
.Lnorm_signed:
    mov x20, #IR_SEXT
    b .Lnorm_emit
.Lnorm_unsigned:
    mov x20, #IR_ZEXT

.Lnorm_emit:
    mov x0, x20
    bl _kcc_ir_alloc_instr
    mov x21, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x21, #IR_TYPE]
    str x19, [x21, #IR_SRC1]       // source = op
   result vreg
    mov x1, #4
    str x1, [x21, #IR_SRC2]        // source width =
    4 bytes (32-bit)
    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x19, x0                    // normalised
   vreg
    mov x0, x21
    bl _kcc_ir_emit
    mov x0, x19
    b .Lnorm_done

.Lnorm_skip:
    mov x0, x19
.Lnorm_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

375 · defined at 369, 374, 375, 376 · used in 30b · defines _kcc_ir_norm32 359, 369, 378b · uses _kcc_ir_alloc_instr 303a, _kcc_ir_emit 304, _kcc_ir_next_vreg 305a, _kcc_type_int_s 250b

Short-circuit logical AND. The C standard requires that `a && b` evaluate `b` only when `a` is nonzero. The builder creates three fresh basic blocks: *rhs_bb*, *short_bb*, and *merge_bb*. It allocates a temporary variable and emits a conditional branch on the LHS vreg: true falls through to *rhs_bb* (which evaluates the RHS, stores the result, and branches to *merge_bb*), false jumps to *short_bb* (which stores zero and branches to *merge_bb*). The returned vreg is a load from the temporary, ensuring a defined value on both paths. Crucially the value of `a && b` is the `int` 0 or 1 (§6.5.13), *not* the raw right operand: *rhs_bb* normalises the RHS with `_kcc_ir_emit_bool_norm` (an `IR_CMP_NE` against zero) before storing it, so `4 && 2` yields 1, not 2.

The short-circuit `||` mirrors this logic with the branch targets swapped and the short-circuit value set to one; its *rhs_bb* likewise normalises the RHS to 0/1 (§6.5.14).

⟨ *IR: build binary* 376 ⟩ +≡

```
// -- Short-circuit && ---
.Libb_short_and:
    // Allocate temporary for merge result
    mov x0, #IR_ALLOCA
    bl _kcc_ir_alloc_instr
    mov x25, x0
```

```
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x25, #IR_TYPE]
    bl _kcc_ir_next_vreg
    str x0, [x25, #IR_DEST]
    mov x26, x0                    // x26 = tmp
alloca vreg
    mov x0, x25
    bl _kcc_ir_emit

    // Evaluate LHS
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_expr
    mov x21, x0                    // lhs vreg

    // Create BBs: rhs_bb, short_bb, merge_bb
    bl _kcc_ir_alloc_bb
    mov x22, x0                    // rhs_bb
    bl _kcc_ir_alloc_bb
    mov x23, x0                    // short_bb
    bl _kcc_ir_alloc_bb
    mov x24, x0                    // merge_bb

    // br_cond: if lhs, goto rhs_bb, else goto
short_bb
    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_DEST]
    ldr x1, [x22, #BB_ID]
    str x1, [x0, #IR_SRC1]       // true → rhs_bb
    ldr x1, [x23, #BB_ID]
    str x1, [x0, #IR_SRC2]       // false →
short_bb
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // -- short_bb: store 0 to tmp, branch to
merge --
    mov x0, x23
    bl _kcc_ir_set_bb
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x25, x0
    str xzr, [x25, #IR_SRC1]     // constant 0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x25, #IR_TYPE]
    bl _kcc_ir_next_vreg
    str x0, [x25, #IR_DEST]
    mov x21, x0                    // zero vreg
    mov x0, x25
    bl _kcc_ir_emit
    // store zero to tmp alloca
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_SRC1]      // value = 0
    str x26, [x0, #IR_SRC2]      // ptr = tmp
alloca
    bl _kcc_ir_emit
    // branch to merge
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x24, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // -- rhs_bb: evaluate RHS, store to tmp,
branch to merge --
    mov x0, x22
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_build_expr
    // The result of `a && b` is the int 0 or 1 (\
stdref{6.5.13}), never the
    // raw operand: normalise the RHS to (rhs !=
0) before storing.
    bl _kcc_ir_emit_bool_norm
    mov x21, x0                    // rhs vreg,
normalised to 0/1
    // store rhs to tmp alloca
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_SRC1]      // value = rhs
    str x26, [x0, #IR_SRC2]      // ptr = tmp
alloca
    bl _kcc_ir_emit
    // branch to merge
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x24, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // -- merge_bb: load result from tmp --
    mov x0, x24
    bl _kcc_ir_set_bb
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    mov x25, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x25, #IR_TYPE]
    str x26, [x25, #IR_SRC1]     // load from tmp
alloca
    bl _kcc_ir_next_vreg
    str x0, [x25, #IR_DEST]
    mov x21, x0                    // result vreg
    mov x0, x25
    bl _kcc_ir_emit

    mov x0, x21
    b .Libb_done

// -- Short-circuit || ---
.Libb_short_or:
    // Allocate temporary for merge result
    mov x0, #IR_ALLOCA
    bl _kcc_ir_alloc_instr
    mov x25, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x25, #IR_TYPE]
    bl _kcc_ir_next_vreg
    str x0, [x25, #IR_DEST]
    mov x26, x0                    // x26 = tmp
alloca vreg
    mov x0, x25
```

```
    bl _kcc_ir_emit

    // Evaluate LHS
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_expr
    mov x21, x0

    // Create BBs: short_bb, rhs_bb, merge_bb
    bl _kcc_ir_alloc_bb
    mov x22, x0                    // short_bb (lhs
true → skip rhs)
    bl _kcc_ir_alloc_bb
    mov x23, x0                    // rhs_bb
    bl _kcc_ir_alloc_bb
    mov x24, x0                    // merge_bb

    // br_cond: if lhs, goto short_bb, else goto
rhs_bb
    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_DEST]
    ldr x1, [x22, #BB_ID]
    str x1, [x0, #IR_SRC1]       // true → short_bb
    ldr x1, [x23, #BB_ID]
    str x1, [x0, #IR_SRC2]       // false → rhs_bb
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // -- short_bb: store 1 to tmp, branch to
merge --
    mov x0, x22
    bl _kcc_ir_set_bb
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x25, x0
    mov x1, #1
    str x1, [x25, #IR_SRC1]      // constant 1
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x25, #IR_TYPE]
    bl _kcc_ir_next_vreg
    str x0, [x25, #IR_DEST]
    mov x21, x0                    // one vreg
    mov x0, x25
    bl _kcc_ir_emit
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_SRC1]      // value = 1
    str x26, [x0, #IR_SRC2]      // ptr = tmp
alloca
    bl _kcc_ir_emit
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x24, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // -- rhs_bb: evaluate RHS, store to tmp,
branch to merge --
    mov x0, x23
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_build_expr
    // The result of `a || b` is the int 0 or 1 (\
stdref{6.5.14}), never the
    // raw operand: normalise the RHS to (rhs !=
0) before storing.
    bl _kcc_ir_emit_bool_norm
    mov x21, x0
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_SRC1]      // value = rhs
    str x26, [x0, #IR_SRC2]      // ptr = tmp
alloca
    bl _kcc_ir_emit
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x24, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // -- merge_bb: load result from tmp --
    mov x0, x24
    bl _kcc_ir_set_bb
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    mov x25, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x25, #IR_TYPE]
    str x26, [x25, #IR_SRC1]     // load from tmp
alloca
    bl _kcc_ir_next_vreg
    str x0, [x25, #IR_DEST]
    mov x21, x0                     // result vreg
    mov x0, x25
    bl _kcc_ir_emit

    mov x0, x21
    b .Libb_done

.Libb_done:
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

376 · defined at 369, 374, 375, 376 · used in 30b · uses _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_cur_bb 305c, _kcc_ir_emit 304, _kcc_ir_next_vreg 305a, _kcc_ir_set_bb 305b, _kcc_type_int_s 250b

End-to-end tests. Comparison operators (==, !=, <, >, <=, >=) lower to an IR_CMP_* instruction that the code generator implements as CMP followed by CSET with the appropriate condition code. The logical connectives && and || use short-circuit evaluation: the IR builder creates conditional branches so that the right-hand operand is only evaluated when the left-hand side does not already determine the result. Logical negation ! is lowered to IR_CMP_EQ against a synthesised zero constant.

⟨ *Test: codegen logical* 378a ⟩ ≡

```
echo ""
echo "--- Logical and Comparison ---"

run_test "logical and true" \
    "int main(void) { return 1 && 1; }" 1

run_test "logical and false" \
    "int main(void) { return 1 && 0; }" 0

run_test "logical or" \
    "int main(void) { return 0 || 1; }" 1

run_test "logical not" \
    "int main(void) { return !0; }" 1

run_test "logical not true" \
    "int main(void) { return !42; }" 0

run_test "chained comparison" \
    "int main(void) { return (3 > 2) + (2 > 1); }"
    2

run_test "equality" \
    "int main(void) { return (5 == 5) + (5 != 3);
   }" 2

run_test "less/greater" \
    "int main(void) { return (1 < 2) + (2 > 1) +
   (2 <= 2) + (3 >= 3); }" 4

run_test "complex logical" \
    "int main(void) { return (1 && 0) || (1 && 1);
    }" 1
```

□

378a · used in 33b

29.9.9. Unary Expression

Unary operators dispatch on the token stored in `AST_F0`. Increment and decrement (`++`/`--`) are intercepted first and delegated to a dedicated builder (see below), because they require address-based read-modify-write lowering rather than a simple value transformation. Arithmetic negation (`-`) emits `IR_NEG`, bitwise complement (`~`) emits `IR_NOT`, and logical negation (`!`) is lowered to `IR_CMP_EQ` against a synthesised zero constant—producing 1 when the operand is zero and 0 otherwise.

Address-of (`&e`) dispatches on the shape of its operand rather than assuming a bare variable. For an identifier it returns the storage address directly—the vreg produced by `IR_ALLOCA` already holds the stack address, or a file-scope name yields an `IR_GLOBAL_REF`—so no load is needed. When the name is not a local it may be a *static* local hoisted to a mangled global (a `static const` whose address escapes, as in `static const T x=…; return &x;`); the global path consults `_kcc_ir_statics` and, on a hit, emits the reference against the mangled symbol so the taken address resolves to the hoisted storage rather than a phantom unmangled `_x`. For a subscript `&a[i]` or a member `&s.f` it reuses the very address computation the assignment and load paths use (`build_subscript_addr`, `build_member_addr`), returning the element or field address. Without this dispatch, `&a[i]` would mis-read the subscript node as if it were an identifier and emit a nonsensical global reference—the exact defect a tokenizer's `lex_next(&tokens[n])` call surfaces. Dereference (`*p`) evaluates the pointer operand and emits an `IR_LOAD` from the resulting address.

⟨ *IR: build unary* 378b ⟩ ≡

```
// -- kcc_ir_build_unary ---
// uint64_t kcc_ir_build_unary(node *expr)
//   x0 = AST_UNARY_EXPR (F0=op, F1=operand, F2=
   prefix/postfix)
.text
.globl _kcc_ir_build_unary
.p2align 2
_kcc_ir_build_unary:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0
    ldr x20, [x19, #AST_F0]     // op token

    // Operators that need special operand
   handling (no _kcc_ir_build_expr)
    cmp x20, #TOK_INC
    b.eq .Libu_inc_dec
    cmp x20, #TOK_DEC
    b.eq .Libu_inc_dec
    cmp x20, #TOK_AMP
    b.eq .Libu_addr

    // Build operand (value)
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_expr
    mov x21, x0                 // operand vreg

    cmp x20, #TOK_MINUS
    b.eq .Libu_neg
    cmp x20, #TOK_TILDE
    b.eq .Libu_not
    cmp x20, #TOK_BANG
    b.eq .Libu_lnot
    cmp x20, #TOK_STAR
    b.eq .Libu_deref

    // Default: return operand as-is
    mov x0, x21
    b .Libu_done

.Libu_neg:
    // A floating operand negates in the FP unit (
   FNEG), typed floating so the
    // back end and the optimizer both treat it as
    such; an integer operand keeps
    // the integer path (and its width
   normalisation).  Float-ness is decided two
    // ways: a floating *literal* is recognised by
    its tag---it always builds as a
    // double constant (\_kcc\_ir\_build\_float\
   _lit), even in an aggregate
    // initialiser, whose element expressions sema
    does not type---and any other
    // operand by its sema type (AST_F3).  Without
    the tag check, `-4.0` inside a
    // brace list fell to the integer path, and
   two's-complement-negating the
    // double bit pattern turned it into a
   different (wrong) double: `-4.0`
    // stored back as `-1.0`.
    ldr x2, [x19, #AST_F1]      // operand node
    cbz x2, .Libu_neg_int
    ldr x1, [x2, #AST_TAG]
```

```
    cmp x1, #AST_FLOAT_LIT
    b.eq .Libu_neg_fp_dbl        // literal:
   negate as double (the const's type)
    ldr x0, [x2, #AST_F3]        // else consult
   the operand's sema type
    cbz x0, .Libu_neg_int
    cmp x0, #0x1000
    b.lo .Libu_neg_int
    ldr x1, [x0]                 // TY_KIND
    cmp x1, #8                   // FLOAT
    b.lo .Libu_neg_int
    cmp x1, #10                  // ..LDOUBLE
    b.hi .Libu_neg_int
    // non-literal floating operand: type the FNEG
    with its sema float type.
    mov x0, #IR_NEG
    bl _kcc_ir_alloc_instr
    mov x22, x0
    str x21, [x22, #IR_SRC1]
    ldr x2, [x19, #AST_F1]       // reload operand
    (x19 preserved across alloc)
    ldr x0, [x2, #AST_F3]
    str x0, [x22, #IR_TYPE]
    b .Libu_neg_fp_fin
.Libu_neg_fp_dbl:
    // Floating literal: the operand vreg is a
   double constant, so negate as
    // double regardless of any (possibly single)
   declared literal type.
    mov x0, #IR_NEG
    bl _kcc_ir_alloc_instr
    mov x22, x0
    str x21, [x22, #IR_SRC1]
    adrp x0, _kcc_type_double@PAGE
    add  x0, x0, _kcc_type_double@PAGEOFF
    ldr x0, [x0]
    str x0, [x22, #IR_TYPE]
.Libu_neg_fp_fin:
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0
    mov x0, x22
    bl _kcc_ir_emit
    mov x0, x20                  // no integer
   width normalization for a float
    b .Libu_done
.Libu_neg_int:
    mov x0, #IR_NEG
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    str x21, [x0, #IR_SRC1]
    mov x22, x0
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0
    mov x0, x22
    bl _kcc_ir_emit
    mov x1, #IR_NEG
    b .Libu_norm

.Libu_not:
    mov x0, #IR_NOT
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    str x21, [x0, #IR_SRC1]
    mov x22, x0
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0
    mov x0, x22
    bl _kcc_ir_emit
    mov x1, #IR_NOT

.Libu_norm:
    // Truncate an unsigned -a / ~a result to 32-
   bit width (x1 = opcode)
    mov x2, #0
    ldr x0, [x19, #AST_F1]
    cbz x0, .Libu_norm_emit
    ldr x2, [x0, #AST_F3]        // operand sema
   type
.Libu_norm_emit:
    mov x3, x2
    mov x0, x20                  // result vreg
    bl _kcc_ir_norm32
    mov x20, x0
    mov x0, x20
    b .Libu_done

.Libu_lnot:
    // Logical not: the result is (operand == 0).
    A floating operand needs a
    // FLOATING compare against 0.0 so the back
   end selects FCMP; typing the
    // compare as int compared the float's raw bit
    pattern as an integer, so
    // `!1.0f` tested 0x3F800000 in the wrong
   domain and came out 1.  The
    // compare and its zero constant are typed `
   double` (the FP working format,
    // matching how a binary `c < x` compare is
   lowered); the operand is
    // fcvt'd to double by the FCMP path.
    // Decide the compare/const type: `double` for
    a floating operand, else int.
    // Reuse x19 (saved by the prologue) to hold
   that type -- the operand node is
    // no longer needed once we have read its type
    -- rather than a scratch the
    // prologue does not preserve, which would
   corrupt the caller.
    ldr x9, [x19, #AST_F1]       // operand node
    cbz x9, .Libu_lnot_int_ty
    ldr x9, [x9, #AST_F3]        // operand type
    cbz x9, .Libu_lnot_int_ty
    ldr x10, [x9, #TY_KIND]
    cmp x10, #TY_FLOAT           // 8
    b.lo .Libu_lnot_int_width
    cmp x10, #TY_LDOUBLE         // 10
    b.hi .Libu_lnot_int_width    // not floating
   → integer compare (pick width)
    adrp x19, _kcc_type_double@PAGE
    add  x19, x19, _kcc_type_double@PAGEOFF
    ldr x19, [x19]               // floating
   operand → compare in double
    b .Libu_lnot_emit
.Libu_lnot_int_width:
    // Compare against zero at the operand's own
   width: a 64-bit value (long or
    // pointer) whose low 32 bits are zero must
   still test true only when the
    // whole word is zero.  x9 is the operand type
    here.
    ldr x10, [x9, #TY_SIZE]
    cmp x10, #4
    b.ls .Libu_lnot_int_ty       // <=4 bytes →
   int_s
    adrp x19, _kcc_type_long_s@PAGE
    add  x19, x19, _kcc_type_long_s@PAGEOFF
    ldr x19, [x19]               // >4 bytes →
   long_s (64-bit compare)
    b .Libu_lnot_emit
```

```
.Libu_lnot_int_ty:
    adrp x19, _kcc_type_int_s@PAGE
    add  x19, x19, _kcc_type_int_s@PAGEOFF
    ldr x19, [x19]                  // int_s
.Libu_lnot_emit:
    // Emit const 0 (int 0, or double 0.0 -- both
   are all-zero bits)
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    str x19, [x0, #IR_TYPE]
    str xzr, [x0, #IR_SRC1]
    mov x22, x0
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0                     // zero vreg
    mov x0, x22
    bl _kcc_ir_emit

    // Now cmp_eq, typed like the constant so a
   floating operand uses FCMP.
    mov x0, #IR_CMP_EQ
    bl _kcc_ir_alloc_instr
    str x19, [x0, #IR_TYPE]
    str x21, [x0, #IR_SRC1]
    str x20, [x0, #IR_SRC2]
    mov x22, x0
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0
    mov x0, x22
    bl _kcc_ir_emit
    mov x0, x20
    b .Libu_done

.Libu_deref:
    // Dereference: operand vreg is a pointer,
   emit load through it
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    mov x22, x0
    // Use pointee type from sema (AST_F3) for
   IR_TYPE
    ldr x1, [x19, #AST_F3]
    cbz x1, .Libu_deref_default
    str x1, [x22, #IR_TYPE]
    // Access width+sign override from the pointee
    type.
    mov x0, x1
    bl _kcc_ir_load_width
    str x0, [x22, #IR_SRC2]
    b .Libu_deref_type
.Libu_deref_default:
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str xzr, [x22, #IR_SRC2]
.Libu_deref_type:
    str x21, [x22, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0
    mov x0, x22
    bl _kcc_ir_emit
    mov x0, x20
    b .Libu_done

.Libu_addr:
    // Address-of: dispatch on the operand's shape
    .
    ldr x21, [x19, #AST_F1]       // operand node
    cbz x21, .Libu_addr_zero
.Libu_addr_unwrap:
    // Parentheses are transparent to `&`: `&(p->m
    )` is `&p->m`.  Unwrap any
    // AST_PAREN_EXPR wrappers before dispatching,
    otherwise the operand looked
    // "unsupported" and address-of yielded 0 (
   dropping the base pointer).
    ldr x0, [x21, #AST_TAG]
    cmp x0, #AST_PAREN_EXPR
    b.ne .Libu_addr_dispatch
    ldr x21, [x21, #AST_F0]       // inner
   expression
    cbz x21, .Libu_addr_zero
    b .Libu_addr_unwrap
.Libu_addr_dispatch:
    cmp x0, #AST_SUBSCRIPT
    b.eq .Libu_addr_subscript
    cmp x0, #AST_MEMBER
    b.eq .Libu_addr_member
    cmp x0, #AST_COMPOUND_LIT
    b.eq .Libu_addr_compound
    cmp x0, #AST_UNARY_EXPR
    b.eq .Libu_addr_deref
    cmp x0, #AST_IDENT_EXPR
    b.ne .Libu_addr_zero          // unsupported
   operand → 0

    // &ident: the variable's storage address (
   alloca vreg, or global_ref)
    ldr x20, [x21, #AST_F0]       // name ptr
    cbz x20, .Libu_addr_zero
    mov x0, x20
    bl _kcc_strlen
    mov x21, x0
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x20
    mov x2, x21
    bl _kcc_ht_get
    cbz x0, .Libu_addr_global
    bic x0, x0, #0x8000000000000000
    sub x0, x0, #1                // alloca vreg (
   stored as vreg+1)
    b .Libu_done

.Libu_addr_subscript:
    // &a[i]: the element address is exactly the
   subscript's GEP result
    mov x0, x21
    bl _kcc_ir_build_subscript_addr
    b .Libu_done

.Libu_addr_member:
    // &s.f / &s->f: the member address
    mov x0, x21
    bl _kcc_ir_build_member_addr
    b .Libu_done

.Libu_addr_deref:
    // &(*e) cancels to e (the pointer value);
   needed by memory-constraint asm
    // operands (`"m"(*addr)`), whose address is
   the pointer itself.
    ldr x1, [x21, #AST_F0]        // unary operator
    cmp x1, #TOK_STAR
    b.ne .Libu_addr_zero          // any other
   unary → unsupported
    ldr x0, [x21, #AST_F1]        // inner
   expression (the pointer)
    cbz x0, .Libu_addr_zero
    bl _kcc_ir_build_expr
    b .Libu_done

.Libu_addr_compound:
    // &(T){...}: the address of the materialised
   compound-literal object.
```

```
    mov x0, x21
    bl _kcc_ir_build_compound_lit
    b .Libu_done

.Libu_addr_global:
    // Not a local: it may be a static local
   hoisted to a mangled global.
    // Redirect the reference name onto that
   mangled symbol so `&x` resolves
    // to the hoisted storage instead of a phantom
    unmangled `_x`.
    adrp x0, _kcc_ir_statics@PAGE
    add  x0, x0, _kcc_ir_statics@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Libu_addr_global_emit
    mov x1, x20                    // key = source
   name
    mov x2, x21                    // keylen
    bl _kcc_ht_get
    cbz x0, .Libu_addr_global_emit
    mov x20, x0                    // x20 = mangled
   name

.Libu_addr_global_emit:
    // Global: emit global_ref, return address
   vreg
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x20, [x0, #IR_SRC1]
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x22, x0
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0
    mov x0, x22
    bl _kcc_ir_emit
    mov x0, x20
    b .Libu_done

.Libu_addr_zero:
    mov x0, xzr
    b .Libu_done

.Libu_inc_dec:
    mov x0, x19                    // pass AST node
    bl _kcc_ir_build_inc_dec
    b .Libu_done

.Libu_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

378b · used in 30b · defines _kcc_ir_build_unary 340 · uses _kcc_ht_get 62, _kcc_ir_alloc_instr 303a, _kcc_ir_build_compound_lit 413, _kcc_ir_build_expr 340, _kcc_ir_build_inc_dec 382, _kcc_ir_build_member_addr 355, _kcc_ir_build_subscript_addr 350, _kcc_ir_emit 304, _kcc_ir_load_width 349b, _kcc_ir_locals 305c, _kcc_ir_next_vreg 305a, _kcc_ir_norm32 375, _kcc_ir_statics 305c, _kcc_type_double 250b, _kcc_type_int_s 250b, _kcc_type_long_s 250b

End-to-end tests: pointers. The address-of operator `&x` returns the alloca vreg directly, which already holds the stack address of the variable. Dereferencing a pointer `*p` emits `IR_LOAD` with the pointer vreg as source, producing the value stored at that address. Cast expressions currently pass through the inner expression unchanged, which is correct for same-width integer casts on a 64-bit architecture.

⟨ *Test: codegen pointers* 381 ⟩ ≡

```
echo ""
echo "--- Pointers ---"

run_test "address and deref" \
    "int main(void) { int x = 42; int *p = &x;
   return *p; }" 42

run_test "cast expression" \
    "int main(void) { long x = 42; return (int)x;
   }" 42

run_test "trunc int-to-char" \
    "int main(void) { int x = 321; return (char)x;
    }" 65

run_test "trunc mask byte" \
    "int main(void) { int x = 256; return (char)x;
    }" 0

run_test "trunc preserves low" \
    "int main(void) { int x = 42; return (char)x;
   }" 42

run_test "trunc short" \
    "int main(void) { int x = 65537; return (short
   )x; }" 1

run_test "cast same-width" \
    "int main(void) { int x = 99; return (int)x; }
   " 99

echo ""
echo "--- Pointer Arithmetic ---"

run_test "ptr-arith-add" \
    "void *mmap(void*,long,int,int,int,long); int
   main(void) { int *p = (int*)mmap
   (0,4096,3,4098,-1,0); *p = 10; *(p + 1) = 20;
   return *(p + 1); }" 20

run_test "ptr-arith-stride" \
    "void *mmap(void*,long,int,int,int,long); int
   main(void) { int *p = (int*)mmap
   (0,4096,3,4098,-1,0); *(p+0)=1; *(p+1)=2; *(p
   +2)=3; return *(p+0)+*(p+1)+*(p+2); }" 6

run_test "ptr-arith-char" \
    "void *mmap(void*,long,int,int,int,long); int
   main(void) { char *s = (char*)mmap
   (0,4096,3,4098,-1,0); *(s+0)=65; *(s+1)=66;
   return *(s+0)+*(s+1); }" 131
```

□

381 · used in 33b

29.9.10. Increment and Decrement

The increment and decrement operators (++ and --) are the only unary operators that *modify* their operand: they read the current value, add or subtract one, and write the result back. The C17 standard (§6.5.2.4, §6.5.3.1) distinguishes *postfix* (`x++`) from *prefix* (`++x`): the postfix form evaluates to the value *before* the modification, while the prefix form evaluates to the value *after*.

IR lowering pattern. Both forms produce the same four-instruction sequence—only the returned vreg differs:

```
%old  = load  i32, ptr %x_alloca
%one  = const i32, 1
%new  = add   i32, %old, %one     ; or sub for --
store i32, %new, ptr %x_alloca
; result = %old (postfix) or %new (prefix)
```

The builder looks up the operand in the locals table to obtain the alloca vreg. If the identifier is not a local, it falls back to `IR_GLOBAL_REF` to obtain the address. The `AST_F2` field of the `AST_UNARY_EXPR` node carries the prefix/postfix flag (1 for prefix, 0 for postfix), set by the parser.

The `i32` in that sketch is only the *default*. The load-modify-store must run in the operand's own type, for two separate reasons. A *pointer* lvalue steps by `sizeof(pointee)` rather than one, and must compute in the 64-bit pointer type or the address is truncated. A narrow *integer* lvalue—a `char`, `short`, or a `lu_byte` bit-count field—must load and store at its real width: reading a one-byte field as a four-byte `int` returns the value byte glued to whatever three bytes follow it (so `n++` evaluates to garbage) and writing four bytes back scribbles the neighbours. That is precisely what stalled Lua: its `FuncState` packs `nups`, `nactvar`, and `freereg` as adjacent `lu_byte` counters, and `allocupvalue`'s `&f->upvalues[fs->nups++]` indexed with a garbage subscript, so the `_ENV` upvalue landed a megabyte away and every global read as `nil`. So the builder reads the operand's resolved type and works in it: a pointer (stepping by the pointee size), or an integer/enum (stepping by one, at the true width). A *floating* operand works in its own `float`/`double` type, and its step is a genuine `1.0`: we build an integer one and convert it (`SITOFP`), so the add or subtract lowers to `fadd`/`fsub` through the floating-point unit. Keeping the `int` default here—as an earlier version did—made `x++` add one to the *raw bits* of the double (`0.0`'s all-zero pattern became the smallest denormal), so a loop as ordinary as `for (double x = 0; x < 20; x++)` never advanced. Only an aggregate or otherwise unranked operand keeps the `int` default.

⟨ *IR: build increment/decrement* 382 ⟩ ≡

```
// -- kcc_ir_build_inc_dec ---
// uint64_t kcc_ir_build_inc_dec(node *expr)
//   x0 = AST_UNARY_EXPR (F0=op, F1=operand, F2=
   prefix(1)/postfix(0))
// Lowers i++, ++i, i--, --i into a load-add/sub-
   store sequence.
// Returns old value (postfix) or new value (
   prefix).
.text
.globl _kcc_ir_build_inc_dec
.p2align 2
_kcc_ir_build_inc_dec:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str xzr, [sp, #80]          // bit-field decl
    (0 unless a bit-field member)
    str xzr, [sp, #88]          // floating-
   operand flag (0 unless float/double)
    mov x19, x0                  // AST node

    // Parentheses are transparent to `++`/`--`:
    `((b)->n)++` is `(b->n)++`.
    // This matters because macros parenthesise
   lvalues---Lua's
    // `luaZ_bufflen(b)` expands to `((b)->n)`, so
    `luaZ_bufflen(b)++` is
    // `((b)->n)++`.  Peel any AST_PAREN_EXPR off
   the operand and rewrite it in
    // place; otherwise the operand matches no
   lvalue shape below and the whole
    // operator collapses to 0 (returning vreg
   ~0---the first parameter---which,
    // used as `b->buffer[...]`'s index, scribbled
    the token buffer).
    ldr x0, [x19, #AST_F1]
.Libid_unparen:
    cbz x0, .Libid_unp_done
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_PAREN_EXPR
    b.ne .Libid_unp_done
    ldr x0, [x0, #AST_F0]        // inner
   expression
    str x0, [x19, #AST_F1]       // rewrite operand
    → inner (parens dropped)
    b .Libid_unparen
.Libid_unp_done:

    ldr x20, [x19, #AST_F0]      // op (TOK_INC or
   TOK_DEC)
    ldr x21, [x19, #AST_F2]      // prefix flag (1=
   prefix, 0=postfix)

    // Operand is an lvalue: a plain identifier, a
    member (`p->m++`, `s.m++`),
    // a subscript (`a[i]++`), a dereference (`(*p
   )++`), or any of these wrapped
    // in parentheses (unwrapped just above).  For
    member/subscript/deref the
    // load/store go through the computed address;
    without them `p->m++` was a
    // no-op (it returned 0 and never advanced the
    object)---which, for
    // `input_buffer->offset++` in a recursive-
   descent parser, spun forever, and
    // for `*((*fmt)++)` in Lua's `string.pack`
   option scanner never advanced the
    // format pointer.
    ldr x0, [x19, #AST_F1]
    cbz x0, .Libid_zero
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_MEMBER
    b.eq .Libid_member
    cmp x1, #AST_SUBSCRIPT
    b.eq .Libid_subscript
    cmp x1, #AST_UNARY_EXPR
    b.eq .Libid_deref
    cmp x1, #AST_IDENT_EXPR
    b.ne .Libid_zero

    ldr x22, [x0, #AST_F0]       // name ptr
    cbz x22, .Libid_zero

    // Look up in locals hash table
    mov x0, x22
    bl _kcc_strlen
    mov x2, x0                    // keylen

    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
```

```
    mov x1, x22
    bl _kcc_ht_get
    cbz x0, .Libid_global
    bic x0, x0, #0x8000000000000000
    sub x22, x0, #1              // x22 = alloca
   vreg
    b .Libid_load

.Libid_member:
    // Member lvalue (`p->m++` / `s.m++`): its GEP
    address is what we modify.
    // build_member_addr publishes the matched
   field; if it is a bit-field
    // (packed AST_F2 non-zero) the RMW must go
   through the bit-field
    // load/store, not a plain container load---
   otherwise `--b.j` subtracts 1
    // from the whole container's low bit (the
   field at offset 0), not from j.
    bl _kcc_ir_build_member_addr
    mov x22, x0
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Libid_load
    ldr x1, [x0, #AST_F2]        // packed (
   bit_offset<<16)|width; 0 if not a bit-field
    cbz x1, .Libid_load
    str x0, [sp, #80]            // remember the
   bit-field decl for .Libid_load
    b .Libid_load

.Libid_subscript:
    // Subscript lvalue (`a[i]++`): its element
   address is what we modify.
    bl _kcc_ir_build_subscript_addr
    mov x22, x0
    b .Libid_load

.Libid_deref:
    // Dereference lvalue (`(*p)++`): the object
   modified is `*p`, whose address
    // is simply the VALUE of `p`.  Build the
   pointer operand and use it as the
    // load/store address---exactly as the member/
   subscript paths use their
    // computed address.  Only `*` is an lvalue
   here; any other unary operator
    // (`-x`, `!x`, `&x`) is not, so it falls
   through to the no-op.  Without this
    // case `(*p)++` matched no shape and
   collapsed to 0, never writing the
    // incremented pointer back---so a `char **fmt
   ` walk (Lua's `string.pack`
    // option scanner, `*((*fmt)++)`) never
   advanced and spun on the first byte.
    ldr x1, [x0, #AST_F0]        // unary op token
    cmp x1, #TOK_STAR
    b.ne .Libid_zero
    ldr x0, [x0, #AST_F1]        // pointer operand
    bl _kcc_ir_build_expr
    mov x22, x0                   // x22 = address
   vreg (the pointer's value)
    b .Libid_load
```

□

382 · defined at 382, 383, 384 · used in 30b · defines _kcc_ir_build_inc_dec 378b · uses _kcc_ht_get 62, _kcc_ir_build_expr 340, _kcc_ir_build_member_addr 355, _kcc_ir_build_subscript_addr 350, _kcc_ir_locals 305c

For global variables, the builder first emits an IR_GLOBAL_REF to materialise the address into a vreg, then continues with the same load-modify-store sequence. When the name is a `static` local, it is first redirected onto its mangled hoisted-global symbol—exactly as the read and compound-assignment paths do—so that `static int x; x++;` targets the hoisted storage rather than emitting a reference to an undefined external _x.

⟨ *IR: build increment/decrement* 383 ⟩ +≡

```
.Libid_global:
    // A static local is hoisted to a mangled
   global; redirect the name onto
    // that symbol so ++/-- target the hoisted
   storage rather than an undefined
    // external `_x`.  Read and compound-assign
   already do this---increment did
    // not, so `static int x; x++;` failed to link
   .
    adrp x0, _kcc_ir_statics@PAGE
    add  x0, x0, _kcc_ir_statics@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Libid_global_ref
    mov x0, x22
    bl _kcc_strlen
    mov x2, x0                        // keylen
    adrp x0, _kcc_ir_statics@PAGE
    add  x0, x0, _kcc_ir_statics@PAGEOFF
    ldr x0, [x0]
    mov x1, x22                       // key = source
   name
    bl _kcc_ht_get
    cbz x0, .Libid_global_ref
    mov x22, x0                       // x22 = mangled
   name

.Libid_global_ref:
    // Emit GLOBAL_REF to get address vreg.  Hold
   the instruction in x23, NOT
    // x19: the shared `.Libid_load` tail still
   expects x19 to be the AST node
    // so it can read the operand's type (and step
    a pointer by sizeof(pointee)
    // in 64-bit).  The member/subscript/deref
   paths preserve x19; this one once
    // clobbered it, so `int *b; *b++ = v;` loaded
    the pointer as a 32-bit int,
    // stepped it by 1 instead of 4, and stored
   through the truncated address.
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x22, [x0, #IR_SRC1]      // name ptr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x23, x0
    bl _kcc_ir_next_vreg
    str x0, [x23, #IR_DEST]
    mov x22, x0                       // x22 = address
   vreg
    mov x0, x23
    bl _kcc_ir_emit
```

□

383 · defined at 382, 383, 384 · used in 30b · uses _kcc_ht_get 62, _kcc_ir_alloc_instr 303a, _kcc_ir_emit 304, _kcc_ir_next_vreg 305a, _kcc_ir_statics 305c, _kcc_type_int_s 250b

The core read-modify-write sequence is shared by both local and global operands. The four steps—load, constant step, add or subtract, and store—follow directly. The step

is *one* for a scalar but `sizeof(*p)` for a pointer: `p++` advances by a whole element, exactly as `p+1` does (§6.5.6). For a pointer lvalue the whole sequence also runs in the pointer's 64-bit type, since a 32-bit add would lop the top half off the address—the defect that made `L->top.p++` (Lua's per-push `api_incr_top`) crawl one byte at a time and scribble the value stack.

A *bit-field* member is the one lvalue this plain sequence cannot touch: its value lives in a sub-word slice of a container, reachable only through the shift-and-mask load and the read–modify–write store that the compound operators already use. A container-wide +1 lands on the container's *lowest* bit—the field at offset zero—so `b.i++` happened to work while `--b.j` silently decremented `i` instead of `j`, leaving `j` untouched. When `build_member_addr` reports the operand is a bit-field, the increment therefore takes a dedicated path: `_kcc_ir_bitfield_load` extracts the old value, an ordinary `int` ±1 forms the new one, and `_kcc_ir_bitfield_store` masks it back into the container. Postfix still returns the old value, prefix the new.

⟨ *IR: build increment/decrement* 384 ⟩ +≡

```
.Libid_load:
    // A bit-field member reaches its value only
   through the shift-and-mask
    // load/store, never a plain container access:
    `--b.j` must subtract 1 from
    // j, not from the container's low bit.  old =
    bitfield_load; new = old ± 1
    // typed as an ordinary int; bitfield_store
   masks it back into place.
    ldr x9, [sp, #80]           // bit-field decl
   (0 for a non-bit-field lvalue)
    cbz x9, .Libid_load_scalar
    mov x0, x22                 // container
   address
    mov x1, x9
    bl _kcc_ir_bitfield_load    // x0 = old value
    (extracted, sign/zero-extended)
    mov x23, x0                 // old value
    mov x0, #1
    bl _kcc_ir_emit_const
    mov x1, x23                 // old
    mov x2, x0                  // step 1
    cmp x20, #TOK_INC
    mov x0, #IR_ADD
    b.eq .Libid_bf_op
    mov x0, #IR_SUB
.Libid_bf_op:
    bl _kcc_ir_emit_binop       // x0 = new value
    mov x24, x0
    ldr x9, [sp, #80]           // reload field
   decl (emit calls clobber x9)
    mov x0, x22                 // container
   address
    mov x1, x24                 // new value
    mov x2, x9                  // field decl
    bl _kcc_ir_bitfield_store
    cbnz x21, .Libid_bf_prefix
    mov x0, x23                 // postfix → old
   value
    b .Libid_done
.Libid_bf_prefix:
    mov x0, x24                 // prefix → new
   value
    b .Libid_done
.Libid_load_scalar:
    // For a pointer lvalue, `++`/`--` steps by
   sizeof(pointee), not 1, and the
    // load/add/store must run in the pointer's
   64-bit type or a 32-bit add would
    // truncate the address.  x26 = working type (
   the pointer type, else int);
    // x25 = step amount (sizeof(pointee), else 1)
   .  x19 is still the AST node
    // here (Step 1 clobbers it), so read the
   operand type now.
    adrp x26, _kcc_type_int_s@PAGE
    add  x26, x26, _kcc_type_int_s@PAGEOFF
    ldr  x26, [x26]             // default
   working type = int
    mov  x25, #1                // default step =
    1
    ldr  x9, [x19, #AST_F1]     // operand node
    cbz  x9, .Libid_typed
    ldr  x9, [x9, #AST_F3]      // operand
   resolved type
    cmp  x9, #0x1000
    b.lo .Libid_typed
    ldr  x10, [x9, #TY_KIND]
    cmp  x10, #TY_PTR
    b.eq .Libid_ptr_step
    // Integer or enum lvalue: work in the operand
   's OWN type so the load and
    // store use its real width.  Hardcoding int
   here read four bytes from a
    // `char`/`short` (or eight from a `long`), so
    a byte field's `n++`
    // returned garbage (the value byte plus its
   neighbours) and clobbered the
    // adjacent bytes on store---which broke Lua's
    parser, whose `lu_byte nups`,
    // `nactvar`, and `freereg` counters all
   advance with `++`.
    cmp  x10, #TY_BOOL          // 2
    b.lo .Libid_typed           // void/invalid →
   keep int default
    cmp  x10, #TY_LLONG         // 7
    b.ls .Libid_int_width       // 2..7 = bool/char
   /short/int/long/llong
    cmp  x10, #TY_ENUM          // 16
    b.eq .Libid_int_width
    cmp  x10, #TY_FLOAT         // 8
    b.eq .Libid_fp_width
    cmp  x10, #TY_DOUBLE        // 9
    b.eq .Libid_fp_width
    b .Libid_typed              // aggregate →
   keep int default
.Libid_int_width:
    mov  x26, x9                // work in the
   operand's integer type; step = 1
    b .Libid_typed
.Libid_fp_width:
    // Floating lvalue: work in the operand's own
   FP type so the load, add/sub,
    // and store all run through the FP unit.  The
    step (below) becomes 1.0, not
    // integer 1 --- otherwise `x++` did an
   INTEGER +1 on the raw double bits
    // (0.0's bits + 1 = a denormal), so `for (
   double x=0; x<20; x++)` never
    // advanced and spun forever (gcc.c-torture
   20000731-1).
    mov  x26, x9                // work in the
   operand's floating type
    mov  x0, #1
    str  x0, [sp, #88]          // mark the step
   as floating
    b .Libid_typed
.Libid_ptr_step:
    mov  x26, x9                // pointer lvalue
```

```
    → work in the pointer type
    ldr  x0, [x9, #TY_F0]        // pointee type
    cbz  x0, .Libid_typed
    bl   _kcc_type_layout        // x0 = sizeof(
   pointee)
    cbz  x0, .Libid_typed
    mov  x25, x0                 // step = sizeof(
   pointee)
.Libid_typed:

    // Step 1: load current value, at the operand'
   s own width.  IR_SRC2 carries
    // the access-size code (1/2/4 unsigned, 5/6/7
    signed, 0 = 8-byte); the load
    // codegen reads it, not IR_TYPE.  Without it
   a `char`/`short` field loaded
    // as a full word and the old value dragged in
    the neighbouring bytes.
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    mov x19, x0                  // x19 = load
   instr (survives the calls below)
    str x26, [x19, #IR_TYPE]
    str x22, [x19, #IR_SRC1]     // address vreg
    mov x0, x26
    bl _kcc_ir_load_width        // x0 = access-
   size code for the working type
    str x0, [x19, #IR_SRC2]      // access-size
   override
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x23, x0                  // x23 = old
   value vreg
    mov x0, x19
    bl _kcc_ir_emit

    // Step 2: constant step.  A floating lvalue
   steps by 1.0: build integer 1
    // and convert it (SITOFP) to the operand's FP
    type, so Step 3's add/sub runs
    // as fadd/fsub.  Otherwise the step is 1 (or
   sizeof(pointee) for a pointer).
    ldr x9, [sp, #88]            // floating-
   operand flag
    cbz x9, .Libid_int_step
    mov x0, #1
    bl _kcc_ir_emit_const        // x0 = integer 1
    vreg
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]                 // from = int
    mov x2, x26                  // to   = the
   operand's FP type
    bl _kcc_ir_convert           // SITOFP → 1.0
    mov x24, x0                  // x24 = FP step
   vreg
    b .Libid_step_done
.Libid_int_step:
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    str x26, [x0, #IR_TYPE]
    str x25, [x0, #IR_SRC1]      // step amount
    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x24, x0                  // x24 = step
   vreg
    mov x0, x19
    bl _kcc_ir_emit
.Libid_step_done:

    // Step 3: add (++) or sub (--)
    cmp x20, #TOK_INC
    mov x0, #IR_ADD
    b.eq .Libid_binop
    mov x0, #IR_SUB
.Libid_binop:
    bl _kcc_ir_alloc_instr
    str x26, [x0, #IR_TYPE]
    str x23, [x0, #IR_SRC1]      // old value
    str x24, [x0, #IR_SRC2]      // step
    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x24, x0                  // x24 = new
   value vreg
    mov x0, x19
    bl _kcc_ir_emit

    // Step 3.5: truncate the new value to the
   operand's own width, so ++/-- of a
    // narrow type wraps like the stored value
   does --- `--(unsigned char)0` is
    // 255, not the 64-bit -1 the subtraction
   produced.  The store already
    // truncates, but a prefix `++`/`--` \emph{
   returns} this value, so it must be
    // re-canonicalised: sign- or zero-extend from
    the operand's byte width.
    ldr x0, [x26, #TY_KIND]
    sub x1, x0, #2
    cmp x1, #5                   // integer kind
   (TY_BOOL(2)..TY_LLONG(7))?
    b.hi .Libid_no_trunc
    ldr x25, [x26, #TY_SIZE]     // width (callee-
   saved: survives the alloc below)
    cmp x25, #8
    b.hs .Libid_no_trunc         // already full
   width: nothing to truncate
    ldr x1, [x26, #TY_F0]        // signedness (1
    = signed → SEXT, 0 → ZEXT)
    mov x0, #IR_SEXT
    cbnz x1, .Libid_tr_emit
    mov x0, #IR_ZEXT
.Libid_tr_emit:
    bl _kcc_ir_alloc_instr
    mov x19, x0
    str x26, [x19, #IR_TYPE]
    str x24, [x19, #IR_SRC1]     // source =
   untruncated new value
    str x25, [x19, #IR_SRC2]     // source width (
   bytes)
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x24, x0                  // x24 =
   truncated new value
    mov x0, x19
    bl _kcc_ir_emit
.Libid_no_trunc:

    // Step 4: store new value back, at the
   operand's own width.  For a store
    // the access-size code lives in IR_DEST (SRC1
   /SRC2 are value/address); a
    // wide store would scribble the fields packed
    next to a narrow one.
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    mov x19, x0                  // x19 = store
   instr
    str x26, [x19, #IR_TYPE]
    str x24, [x19, #IR_SRC1]     // new value
    str x22, [x19, #IR_SRC2]     // address
    mov x0, x26
    bl _kcc_ir_store_width       // x0 = access-
   size code
```

```
    str x0, [x19, #IR_DEST]       // access-size
   override
    mov x0, x19
    bl _kcc_ir_emit

    // Return old (postfix) or new (prefix)
    cbnz x21, .Libid_prefix
    mov x0, x23                   // postfix:
   return old value
    b .Libid_done
.Libid_prefix:
    mov x0, x24                   // prefix: return
    new value
    b .Libid_done

.Libid_zero:
    mov x0, #0

.Libid_done:
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

384 · defined at 382, 383, 384 · used in 30b · uses _kcc_ir_alloc_instr 303a, _kcc_ir_convert 343, _kcc_ir_emit 304, _kcc_ir_load_width 349b, _kcc_ir_next_vreg 305a, _kcc_ir_store_width 349b, _kcc_type_int_s 250b, _kcc_type_layout 267, 268

End-to-end tests. The following tests verify the full compilation pipeline for all four operator forms. The for-loop test with `i++` is the most critical: before this feature was implemented, the increment emitted `IR_NOP` and the loop ran forever.

⟨ *Test: codegen increment* 386 ⟩ ≡

```
echo ""
echo "--- Increment / Decrement ---"

run_test "postfix increment stmt" \
    "int main(void) { int x = 5; x++; return x; }"
    6

run_test "prefix increment" \
    "int main(void) { int x = 5; return ++x; }" 6

run_test "postfix returns old" \
    "int main(void) { int x = 5; int y = x++;
   return y; }" 5

run_test "prefix returns new" \
    "int main(void) { int x = 5; int y = ++x;
   return y; }" 6

run_test "postfix decrement stmt" \
    "int main(void) { int x = 5; x--; return x; }"
    4

run_test "prefix decrement" \
    "int main(void) { int x = 5; return --x; }" 4

run_test "for-loop with i++" \
    "int main(void) { int s = 0; for (int i = 0; i
    < 5; i++) s = s + 2; return s; }" 10

run_test "for-loop sum with i++" \
    "int main(void) { int s = 0; for (int i = 0; i
    < 5; i++) s = s + i; return s; }" 10

run_test "do-while with i++" \
    "int main(void) { int i = 0; int s = 0; do { s
    = s + i; i++; } while (i <= 5); return s; }"
   15

run_test "while with decrement" \
    "int main(void) { int x = 5; while (x > 0) x
   --; return x; }" 0

run_test "triple increment" \
    "int main(void) { int x = 0; x++; x++; x++;
   return x; }" 3
```

□

386 · used in 33b

29.9.11. Assignment

Assignment handles three cases. A simple = evaluates the RHS, looks up the LHS identifier in the locals table, and emits `IR_STORE` to write the value into the variable's alloca. Compound assignments (+=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=) first load the current value, emit the corresponding binary operation, then store the result back—exactly the read-modify-write pattern described for increment/decrement above. Finally, if the LHS identifier is not in locals, it must be a global: the builder emits `IR_GLOBAL_REF` to materialise the address. For simple assignment the value is stored directly; for compound operators the same load-binop-store sequence is applied, loading from the global address vreg before emitting the binary operation and storing the result back. That reload must carry the lvalue type's access-size code (`IR_SRC2`), exactly as the local-alloca path and the matching store do: a narrow scalar occupies only the low bytes of its natural width, so a hand-built `IR_LOAD` that left the size code zero would read a raw 64-bit word and splice adjacent memory (or, for a signed global, fail to sign-extend) into the value. A subsequent arithmetic >>= then shifted that un-extended pattern, turning an arithmetic right shift into a logical one—so `int g; g >>= 4` with `g < 0` produced a large positive. Routing the reload through `_kcc_ir_emit_load_t` sets both the size code and the type, keeping the global path identical to the local one.

The three *address-based* lvalues—a member `s.f`, a subscript `a[i]`, and a dereference `*p`—share a single helper, `kcc_ir_assign_through`. Each computes its target address once, then hands the address, the right-hand value, the operator token, an optional bit-field descriptor, and the access type to the helper, which performs a plain store for = or a load-operate-store for a compound operator, sized to the lvalue's type (so a narrow field is not clobbered) and routed through the bit-field read-modify-write when the target is a bit-field. Sharing this path is what makes `s.f += v`, `a[i] *= v`, and `*p -= v` work uniformly—and, because the helper reuses the bit-field primitives, a compound assignment to a bit-field member folds out for free.

When the LHS has an aggregate type—`TY_STRUCT` *or* `TY_UNION`—a single store would only copy the first eight bytes. A union is easy to overlook here: syntactically `*dst = *src` looks like a scalar assignment, and if the union is not recognised the store copies one word and leaves the

rest of the object stale. This is exactly how Lua's hash table corrupted itself: `luaH_newkey` moves a colliding `Node` (a union) with `*f = *mp`, and a one-word copy left the moved node's `next` link garbage, so the collision chain no longer reached its own main position and `luaH_newkey` spun forever walking it. So both aggregate kinds take the copy path.

The builder resolves both the LHS and RHS base addresses (alloca vreg or global ref) and calls `_kcc_ir_emit_struct_copy`, which copies exactly `TY_SIZE` bytes—*not* a number of fields, which for a union (a handful of overlapping members) or a struct with sub-word fields bears no relation to its size, and copying the wrong amount either truncates the object or runs off its end into the neighbour. Both sides accept the same lvalue shapes: a plain variable, an aggregate-typed member, an array element `a[i]`, or a dereferenced pointer `*p`, whose pointer *value* is the aggregate's address. The destination must recognise each of these just as the source does—otherwise the copy is silently dropped, leaving the target uninitialised. The *source* accepts three more shapes that carry an aggregate value without naming an lvalue directly: a parenthesised expression; a comma operator whose right operand is the aggregate (`g = (setup(), s)`); and a nested assignment (`a = b = c`), whose value is the object it just assigned. The builder peels the parentheses, evaluates a comma's discarded left side for its side effects, and *builds* a nested assignment—performing the inner copy—then takes the address it returns as the source. Matching no case for these is what let `g = (1, other)` and `a = b = c` typecheck yet copy nothing.

The compound path uses callee-saved registers x23 (current value vreg) and x24 (IR opcode), so the frame must be 64 bytes to save x19–x24. In all cases the returned vreg is the stored value, matching the C semantics where assignment is an expression (§6.5.16).

The current-value load that opens the compound path must carry the same narrow access-size code (from `_kcc_ir_load_width`) that its paired store uses—not the natural 8-byte default. A narrow scalar occupies only the low bytes of its 8-byte stack slot, and the store writes just those bytes, so an 8-byte reload would splice the slot's stale upper bytes into the value. This was invisible while the leaked bits happened to be zero, but surfaced in zlib's `inflate_table`: an `unsigned incr` written 4 bytes wide and then updated with `incr >>= 1` was reloaded 8 bytes wide, so the 64-bit shift dragged the garbage upper word down into `incr` the first time the loop body ran, corrupting the Huffman sub-table walk into an out-of-bounds write.

Parentheses around the target are transparent: `(x) = v` is `x = v`. The lvalue dispatch keys on the target's node tag (identifier, member, subscript, dereference); a stray `AST_PAREN_EXPR` matches none of them and the store is silently dropped. This is not a rare corner: the macro `#define G(L) (L->l_G)` makes `G(L) = g` expand to `(L->l_G) = g`, so `preinit_thread`'s wiring of the global-state pointer vanished, leaving `L->l_G` null and crashing the first allocation through it. So we peel any parenthesis wrappers off the target first, rewriting the assignment node's LHS field to the bare lvalue before dispatching.

⟨ *IR: build assign* 387 ⟩ ≡

```
// -- kcc_ir_build_assign ---
// uint64_t kcc_ir_build_assign(node *expr)
//   x0 = AST_ASSIGN_EXPR (F0=op, F1=lhs, F2=rhs)
// For simple assignment (=), evaluates RHS and
   stores to LHS alloca.
.text
.globl _kcc_ir_build_assign
.p2align 2
_kcc_ir_build_assign:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0

    // Parentheses are transparent to an
   assignment target: `(x) = v` is
    // `x = v`.  Peel any AST_PAREN_EXPR off the
   LHS and rewrite it in place,
    // so the lvalue dispatch below sees the real
   target instead of dropping
    // the store (as `G(L) = g`, i.e. `(L->l_G) =
   g`, otherwise would).
    ldr x0, [x19, #AST_F1]       // LHS node
.Liba_unparen:
    cbz x0, .Liba_unparen_done
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_PAREN_EXPR
    b.ne .Liba_unparen_done
    ldr x0, [x0, #AST_F0]        // inner
   expression
    str x0, [x19, #AST_F1]       // rewrite LHS →
   inner (parens dropped)
    b .Liba_unparen
.Liba_unparen_done:

    ldr x0, [x19, #AST_F3]       // assignment
   result type (= LHS type)
    str x0, [sp, #64]            // stashed for
   compound-assign width normalisation
    ldr x20, [x19, #AST_F0]      // op token

    // Check for struct assignment (simple assign
   only)
    cmp x20, #TOK_ASSIGN
    b.ne .Liba_normal_rhs
    ldr x0, [x19, #AST_F1]       // LHS node
    cbz x0, .Liba_normal_rhs
    ldr x1, [x0, #AST_F3]        // LHS sema type
    cbz x1, .Liba_normal_rhs
    ldr x2, [x1, #TY_KIND]
    cmp x2, #TY_STRUCT
    b.eq .Liba_struct_copy
    cmp x2, #TY_UNION
    b.eq .Liba_struct_copy

.Liba_normal_rhs:
    // Build RHS first
    ldr x0, [x19, #AST_F2]
    ldr x2, [x0, #AST_F3]        // rhs sema type
    str x2, [sp, #72]
    bl _kcc_ir_build_expr
    mov x21, x0                   // rhs vreg

    // Simple assignment converts the RHS to the
   LHS type across the
    // int/float boundary ([sp,#64] holds the
   assignment's (LHS) type).
    cmp x20, #TOK_ASSIGN
```

```
    b.ne .Liba_get_lhs
    mov x0, x21
    ldr x1, [sp, #72]           // from = rhs type
    ldr x2, [sp, #64]           // to   = LHS type
    bl _kcc_ir_convert
    mov x21, x0

.Liba_get_lhs:
    // Get LHS - identifier, subscript, member, or
    dereference
    ldr x0, [x19, #AST_F1]
    cbz x0, .Liba_done
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_SUBSCRIPT
    b.eq .Liba_subscript
    cmp x1, #AST_MEMBER
    b.eq .Liba_member
    cmp x1, #AST_UNARY_EXPR
    b.eq .Liba_deref
    cmp x1, #AST_IDENT_EXPR
    b.ne .Liba_done

    // Look up variable alloca
    ldr x22, [x0, #AST_F0]      // name ptr
    cbz x22, .Liba_done

    mov x0, x22
    bl _kcc_strlen
    mov x2, x0

    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x22
    bl _kcc_ht_get
    cbz x0, .Liba_global_store
    bic x0, x0, #0x8000000000000000
    sub x22, x0, #1              // alloca vreg (
  stored as vreg+1)

    // For compound assignment (+=, etc.), load
  current value first
    cmp x20, #TOK_ASSIGN
    b.eq .Liba_store

    // Load current value.  Use the lvalue's own
  type (stashed at [sp,#64]) so a
    // floating local's compound update stays
  floating: the binop below inherits
    // this type and the back end selects FADD/
  FSUB/FMUL/FDIV instead of the
    // integer opcodes.  The load also carries an
  access-size code (IR_SRC2)
    // derived from that type, exactly as the
  matching store does below: a
    // narrow scalar occupies only the low bytes
  of its 8-byte slot, so reading
    // it back at the natural 8-byte width would
  splice the slot's stale upper
    // bytes into the value.  For an `unsigned
  incr` written with a 4-byte store,
    // an 8-byte reload followed by `incr >>= 1`
  would shift that garbage down.
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    ldr x1, [sp, #64]           // lvalue (LHS)
  type
    str x1, [x0, #IR_TYPE]
    str x22, [x0, #IR_SRC1]
    mov x19, x0
    ldr x0, [sp, #64]           // lvalue type →
  load access-size code (IR_SRC2)
    bl _kcc_ir_load_width
    str x0, [x19, #IR_SRC2]     // narrow read
    must zero/sign-extend, not slurp 8 bytes
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x23, x0                 // current value
  vreg
    mov x0, x19
    bl _kcc_ir_emit

    // Map compound op to IR opcode
    mov x24, #IR_ADD            // default
    cmp x20, #TOK_ADD_ASSIGN
    b.eq .Liba_compound
    mov x24, #IR_SUB
    cmp x20, #TOK_SUB_ASSIGN
    b.eq .Liba_compound
    mov x24, #IR_MUL
    cmp x20, #TOK_MUL_ASSIGN
    b.eq .Liba_compound
    mov x24, #IR_SDIV
    cmp x20, #TOK_DIV_ASSIGN
    b.eq .Liba_compound
    mov x24, #IR_SMOD
    cmp x20, #TOK_MOD_ASSIGN
    b.eq .Liba_compound
    mov x24, #IR_AND
    cmp x20, #TOK_AND_ASSIGN
    b.eq .Liba_compound
    mov x24, #IR_OR
    cmp x20, #TOK_OR_ASSIGN
    b.eq .Liba_compound
    mov x24, #IR_XOR
    cmp x20, #TOK_XOR_ASSIGN
    b.eq .Liba_compound
    mov x24, #IR_SHL
    cmp x20, #TOK_SHL_ASSIGN
    b.eq .Liba_compound
    mov x24, #IR_ASHR
    cmp x20, #TOK_SHR_ASSIGN
    b.eq .Liba_compound
    // Fallthrough: treat as simple assign
    b .Liba_store

.Liba_compound:
    // Apply the compound operation under the
  usual arithmetic conversions: the
    // shared helper computes in the common type (
  floating when either operand
    // is) and converts the result back to the
  lvalue type.
    mov x0, x24                 // op (IR opcode)
    mov x1, x23                 // old value
    ldr x2, [sp, #64]           // L (lvalue type)
    mov x3, x21                 // rhs value
    ldr x4, [sp, #72]           // R (rhs type)
    bl _kcc_ir_compound_apply
    mov x21, x0                 // value to store

.Liba_store:
    // Emit store.  The access-size code comes
  from the lvalue type (stashed at
    // [sp,#64]) so a narrow scalar---especially a
     `float`, whose 4-byte single
    // representation differs from the double
  working format---is written at its
    // true width rather than as a 64-bit default.
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    mov x23, x0                 // store instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x23, #IR_TYPE]
    str x21, [x23, #IR_SRC1]    // value
    str x22, [x23, #IR_SRC2]    // alloca
    ldr x0, [sp, #64]           // lvalue type →
```

```
    access-size code
    bl _kcc_ir_store_width
    str x0, [x23, #IR_DEST]
    mov x0, x23
    bl _kcc_ir_emit

    mov x0, x21                     // return the
    stored value's vreg
    b .Liba_done

.Liba_global_store:
    // x22 = name ptr, x21 = rhs vreg, x20 = op
    token
    // static local? redirect the store onto its
    mangled global
    adrp x0, _kcc_ir_statics@PAGE
    add  x0, x0, _kcc_ir_statics@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Liba_global_ref_emit
    mov x23, x0                     // ht
    mov x0, x22
    bl _kcc_strlen
    mov x2, x0                      // keylen
    mov x0, x23
    mov x1, x22                     // key = source
    name
    bl _kcc_ht_get
    cbz x0, .Liba_global_ref_emit
    mov x22, x0                     // x22 = mangled
    name

.Liba_global_ref_emit:
    // Emit global_ref to get address vreg
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x22, [x0, #IR_SRC1]      // name ptr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x22, x0                     // x22 = global
    addr vreg
    mov x0, x19
    bl _kcc_ir_emit

    // For compound assignment on globals, load-
    modify-store
    cmp x20, #TOK_ASSIGN
    b.eq .Liba_global_simple

    // Load current value from global at the
    lvalue type's access width, so a
    // narrow signed global is sign-extended (
    ldrsw), not read as a raw 64-bit
    // word.  A hand-built IR_LOAD left IR_SRC2 (
    the access-size/sign code)
    // zero, which codegen lowers to a 64-bit `ldr
    `; feeding that
    // un-sign-extended value to a compound `>>=`
    made the arithmetic right
    // shift behave like a logical one (`int g >>=
     4` with g<0 yielded a huge
    // positive).  Route through the shared load
    helper so the size code and
    // lvalue type (floating globals stay floating
    ) are both set.
    ldr x0, [sp, #64]           // lvalue (LHS)
    type
    bl _kcc_ir_load_width       // x0 = access-
    size/sign code
    mov x1, x0                  // size code
    mov x0, x22                 // global addr
    vreg
    ldr x2, [sp, #64]           // lvalue type
    bl _kcc_ir_emit_load_t
    mov x23, x0                  // x23 = current
    value vreg

    // Map compound op to IR opcode (same table as
     local path)
    mov x24, #IR_ADD
    cmp x20, #TOK_ADD_ASSIGN
    b.eq .Liba_global_compound
    mov x24, #IR_SUB
    cmp x20, #TOK_SUB_ASSIGN
    b.eq .Liba_global_compound
    mov x24, #IR_MUL
    cmp x20, #TOK_MUL_ASSIGN
    b.eq .Liba_global_compound
    mov x24, #IR_SDIV
    cmp x20, #TOK_DIV_ASSIGN
    b.eq .Liba_global_compound
    mov x24, #IR_SMOD
    cmp x20, #TOK_MOD_ASSIGN
    b.eq .Liba_global_compound
    mov x24, #IR_AND
    cmp x20, #TOK_AND_ASSIGN
    b.eq .Liba_global_compound
    mov x24, #IR_OR
    cmp x20, #TOK_OR_ASSIGN
    b.eq .Liba_global_compound
    mov x24, #IR_XOR
    cmp x20, #TOK_XOR_ASSIGN
    b.eq .Liba_global_compound
    mov x24, #IR_SHL
    cmp x20, #TOK_SHL_ASSIGN
    b.eq .Liba_global_compound
    mov x24, #IR_ASHR
    cmp x20, #TOK_SHR_ASSIGN
    b.eq .Liba_global_compound
    b .Liba_global_simple

.Liba_global_compound:
    // Same shared helper as the local path:
    compute under the usual arithmetic
    // conversions and convert the result back to
    the lvalue type.
    mov x0, x24                  // op (IR opcode)
    mov x1, x23                  // current value
    ldr x2, [sp, #64]           // L (lvalue type)
    mov x3, x21                  // rhs
    ldr x4, [sp, #72]           // R (rhs type)
    bl _kcc_ir_compound_apply
    mov x21, x0                  // value to store

.Liba_global_simple:
    // Store value to global address, at the
    lvalue type's access width (so a
    // global `float` is written as a 4-byte
    single, not a 64-bit double).
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    mov x23, x0                  // store instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x23, #IR_TYPE]
    str x21, [x23, #IR_SRC1]     // value (rhs or
    compound result)
    str x22, [x23, #IR_SRC2]     // global addr
    ldr x0, [sp, #64]           // lvalue type →
    access-size code
    bl _kcc_ir_store_width
    str x0, [x23, #IR_DEST]
    mov x0, x23
    bl _kcc_ir_emit
```

```
    mov x0, x21
    b .Liba_done

.Liba_subscript:
    // LHS is a[i]: x0 = AST_SUBSCRIPT node, x21 =
    rhs vreg, x20 = op token.
    // Same load-op-store path as a member, so a[i
  ] += v works too.
    mov x23, x0                    // subscript node
    bl _kcc_ir_build_subscript_addr
    mov x22, x0                    // element
  address
    mov x0, x22                    // addr
    mov x1, x21                    // rhs
    mov x2, x20                    // op token
    mov x3, #0                     // not a bit-
  field
    ldr x4, [x23, #AST_F3]         // element type →
  access width (lvalue type L)
    ldr x5, [sp, #72]              // rhs type R (for
    compound conversions)
    bl _kcc_ir_assign_through
    b .Liba_done

.Liba_member:
    // LHS is s.x: x0 = AST_MEMBER node, x21 = rhs
    vreg, x20 = op token.
    // The unified helper does simple or compound
  (load-op-store), and a
    // bit-field read-modify-write when the field
  carries bit metadata.
    mov x23, x0                    // member node
    bl _kcc_ir_build_member_addr
    mov x22, x0                    // field address
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    ldr x24, [x0]                  // matched field
  decl (0 if none)
    mov x0, x22                    // addr
    mov x1, x21                    // rhs
    mov x2, x20                    // op token
    mov x3, x24                    // field (bit-
  field metadata)
    ldr x4, [x23, #AST_F3]         // member's
  resolved type → access width (L)
    ldr x5, [sp, #72]              // rhs type R (for
    compound conversions)
    bl _kcc_ir_assign_through
    b .Liba_done

.Liba_deref:
    // LHS is *expr - evaluate the pointer, store
  RHS through it (load-op-store
    // for compound), reusing the shared address-
  lvalue helper.
    ldr x0, [x0, #AST_F0]          // check op is
  TOK_STAR
    cmp x0, #TOK_STAR
    b.ne .Liba_done
    ldr x23, [x19, #AST_F1]        // unary *expr
    ldr x23, [x23, #AST_F3]        // sema type of *
  expr (may be 0 → 64-bit)
    ldr x0, [x19, #AST_F1]         // unary expr
    ldr x0, [x0, #AST_F1]          // operand of * (e
  .g., IdentExpr 'p')
    bl _kcc_ir_build_expr
    mov x22, x0                    // pointer vreg
    mov x0, x22                    // addr
    mov x1, x21                    // rhs
    mov x2, x20                    // op token
    mov x3, #0                     // not a bit-
  field
    mov x4, x23                    // pointed-to
  type → access width (lvalue type L)
    ldr x5, [sp, #72]              // rhs type R (for
    compound conversions)
    bl _kcc_ir_assign_through
    b .Liba_done

.Liba_struct_copy:
    // x19 = assign node, LHS type is TY_STRUCT
    // x0 = LHS node (from check above), x1 =
  struct type
    mov x21, x1                    // x21 = struct
  type

    // Get LHS base address: a plain identifier's
  alloca, or---when the
    // destination is a struct-typed member (`s.m
  = rhs`)---the member's own
    // address.  Without the member case a struct
  assignment to a member emitted
    // no copy at all, leaving the member
  uninitialised.
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_MEMBER
    b.eq .Liba_sc_dst_member
    cmp x1, #AST_SUBSCRIPT
    b.eq .Liba_sc_dst_subscript
    cmp x1, #AST_UNARY_EXPR
    b.eq .Liba_sc_dst_deref
    cmp x1, #AST_IDENT_EXPR
    b.ne .Liba_done

    ldr x22, [x0, #AST_F0]         // LHS name
    mov x0, x22
    bl _kcc_strlen
    mov x2, x0
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x22
    bl _kcc_ht_get
    cbz x0, .Liba_sc_dst_global

    bic x0, x0, #0x8000000000000000
    sub x23, x0, #1                // x23 = dst
  alloca vreg
    b .Liba_sc_get_src

.Liba_sc_dst_global:
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x22, [x0, #IR_SRC1]
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x22, x0
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x23, x0                    // x23 = dst
  global ref vreg
    mov x0, x22
    bl _kcc_ir_emit
    b .Liba_sc_get_src

.Liba_sc_dst_member:
    // LHS is a struct-typed member: its GEP
  address is the copy destination.
    bl _kcc_ir_build_member_addr
    mov x23, x0                    // x23 = dst
  member address vreg
    b .Liba_sc_get_src

.Liba_sc_dst_subscript:
    // LHS is an array element `a[i]`: its
```

```
    computed address is the destination.
    bl _kcc_ir_build_subscript_addr
    mov x23, x0                    // x23 = dst
    element address vreg
    b .Liba_sc_get_src

.Liba_sc_dst_deref:
    // LHS is `*p` (pointer to a struct): the
    pointer's value IS the destination
    // struct's address, so building the pointer
    operand yields the dst base.
    ldr x1, [x0, #AST_F0]          // unary op
    cmp x1, #TOK_STAR
    b.ne .Liba_done
    ldr x0, [x0, #AST_F1]          // pointer operand
    bl _kcc_ir_build_expr
    mov x23, x0                    // x23 = dst base
    address vreg

.Liba_sc_get_src:
    // Get RHS base address: an identifier's
    alloca, or a struct-returning
    // call's result buffer.
    ldr x0, [x19, #AST_F2]
    cbz x0, .Liba_done
.Liba_sc_unwrap:
    // Peel parentheses, and evaluate the
    discarded left side of any comma
    // operator, before reading the RHS's real
    shape.  A struct assignment whose
    // right side is `(x , s)` still copies the
    struct `s`; without this the comma
    // (and paren) node matched no case below and
    the copy was silently skipped
    // --- `g = (1, other)` left `g` unchanged.
    x20 holds the node across the
    // side-effect build (it is callee-saved, so
    build_expr preserves it).
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_PAREN_EXPR
    b.ne .Liba_sc_chkcomma
    ldr x0, [x0, #AST_F0]          // unwrap
    parentheses
    cbz x0, .Liba_done
    b .Liba_sc_unwrap
.Liba_sc_chkcomma:
    cmp x1, #AST_COMMA_EXPR
    b.ne .Liba_sc_shape
    mov x20, x0                    // save the comma
    node
    ldr x0, [x20, #AST_F0]         // comma LHS:
    evaluate for side effects, discard
    bl _kcc_ir_build_expr
    ldr x0, [x20, #AST_F1]         // comma RHS is
    the struct source
    cbz x0, .Liba_done
    b .Liba_sc_unwrap
.Liba_sc_shape:
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_ASSIGN_EXPR
    b.eq .Liba_sc_src_assign
    cmp x1, #AST_CALL_EXPR
    b.eq .Liba_sc_src_call
    cmp x1, #AST_MEMBER
    b.eq .Liba_sc_src_member
    cmp x1, #AST_SUBSCRIPT
    b.eq .Liba_sc_src_subscript
    cmp x1, #AST_UNARY_EXPR
    b.eq .Liba_sc_src_deref
    cmp x1, #AST_VA_ARG
    b.eq .Liba_sc_src_va_arg
    cmp x1, #AST_IDENT_EXPR
    b.ne .Liba_done

    ldr x22, [x0, #AST_F0]         // RHS name
    mov x0, x22
    bl _kcc_strlen
    mov x2, x0
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x22
    bl _kcc_ht_get
    cbz x0, .Liba_sc_src_global

    bic x0, x0, #0x8000000000000000
    sub x24, x0, #1                // x24 = src
    alloca vreg
    b .Liba_sc_do_copy

.Liba_sc_src_global:
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x22, [x0, #IR_SRC1]
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x22, x0
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x24, x0                    // x24 = src
    global ref vreg
    mov x0, x22
    bl _kcc_ir_emit
    b .Liba_sc_do_copy

.Liba_sc_src_member:
    // RHS is a struct-typed member (`dst = a.m`):
    its GEP address is the source.
    bl _kcc_ir_build_member_addr
    mov x24, x0
    b .Liba_sc_do_copy

.Liba_sc_src_subscript:
    // RHS is an array element (`dst = a[i]`): its
    computed address is the source.
    bl _kcc_ir_build_subscript_addr
    mov x24, x0
    b .Liba_sc_do_copy

.Liba_sc_src_deref:
    // RHS is `*p` (a dereferenced pointer to a
    struct): the pointer's value IS
    // the source struct's address, so building
    the pointer operand suffices.
    ldr x1, [x0, #AST_F0]          // unary op
    cmp x1, #TOK_STAR
    b.ne .Liba_done
    ldr x0, [x0, #AST_F1]          // pointer operand
    bl _kcc_ir_build_expr
    mov x24, x0
    b .Liba_sc_do_copy

.Liba_sc_src_assign:
    // RHS is a nested assignment `dst2 = src2` (a
    chain `a = b = c`): build it,
    // performing the inner aggregate copy, then
    use its result --- the address
    // of the just-assigned object --- as this
    copy's source.  A struct
    // assignment returns that address, so `a = (b
    = c)` copies c into b and then
    // b into a; without this the chained copy
    matched no case and did nothing.
    bl _kcc_ir_build_assign
    mov x24, x0                    // x24 =
    assigned object's address vreg
    b .Liba_sc_do_copy
```

```
.Liba_sc_src_call:
    // RHS is a struct-returning call: its value
   is the result buffer address.
    bl _kcc_ir_build_call
    mov x24, x0                    // x24 = src
   base vreg
    b .Liba_sc_do_copy

.Liba_sc_src_va_arg:
    // RHS is `va_arg(ap, struct T)`: build_va_arg
    yields the argument object's
    // address (and advances the cursor), which is
    the copy source.
    bl _kcc_ir_build_va_arg
    mov x24, x0                    // x24 = src
   base vreg
    b .Liba_sc_do_copy

.Liba_sc_do_copy:
    // x24 = src, x23 = dst, x21 = aggregate type.
     Copy exactly TY_SIZE bytes;
    // a field count would be wrong for a union or
    a struct with sub-word fields.
    // _kcc_type_layout is idempotent, so it just
   returns the (already-computed)
    // byte size.
    mov x0, x21
    bl _kcc_type_layout          // x0 = aggregate
   byte size
    mov x2, x0                     // byte size (
   exact)
    mov x0, x24                    // src
    mov x1, x23                    // dst
    bl _kcc_ir_emit_struct_copy
    mov x0, x23
    b .Liba_done

.Liba_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

387 · used in 30b · defines _kcc_ir_build_assign 340, 405 · uses _kcc_ht_get 62, _kcc_ir_alloc_instr 303a, _kcc_ir_build_call 393, _kcc_ir_build_expr 340, _kcc_ir_build_member_addr 355, _kcc_ir_build_subscript_addr 350, _kcc_ir_build_va_arg 398, _kcc_ir_convert 343, _kcc_ir_emit 304, _kcc_ir_emit_struct_copy 404, _kcc_ir_load_width 349b, _kcc_ir_locals 305c, _kcc_ir_next_vreg 305a, _kcc_ir_statics 305c, _kcc_ir_store_width 349b, _kcc_type_int_s 250b, _kcc_type_layout 267, 268

End-to-end tests. Compound assignment operators follow the read-modify-write pattern for both local and global variables. For locals, the builder loads via the alloca vreg; for globals, it emits `IR_GLOBAL_REF` to materialise the address, then the same load-binop-store sequence. The final three tests specifically exercise the global path, which was initially missing and caused the RHS to overwrite the variable without reading its current value.

⟨ *Test: codegen compound* 392 ⟩ ≡

```
echo ""
echo "--- Compound Assignment ---"

run_test "add-assign" \
    "int main(void) { int x = 10; x += 5; return x
   ; }" 15

run_test "sub-assign" \
    "int main(void) { int x = 10; x -= 5; return x
   ; }" 5

run_test "mul-assign" \
    "int main(void) { int x = 10; x *= 5; return x
   ; }" 50

run_test "div-assign" \
    "int main(void) { int x = 100; x /= 4; return
   x; }" 25

run_test "mod-assign" \
    "int main(void) { int x = 17; x %= 5; return x
   ; }" 2

run_test "and-assign" \
    "int main(void) { int x = 0xFF; x &= 0x0F;
   return x; }" 15

run_test "or-assign" \
    "int main(void) { int x = 0xF0; x |= 0x0F;
   return x; }" 255

run_test "xor-assign" \
    "int main(void) { int x = 0xFF; x ^= 0x0F;
   return x; }" 240

run_test "shl-assign" \
    "int main(void) { int x = 1; x <<= 4; return x
   ; }" 16

run_test "shr-assign" \
    "int main(void) { int x = 32; x >>= 2; return
   x; }" 8

run_test "compound in loop" \
    "int main(void) { int s = 0; for (int i = 1; i
    <= 5; i++) s += i; return s; }" 15

run_test "chained compound" \
    "int main(void) { int a = 1; int b = 2; a +=
   10; b *= 3; return a + b; }" 17

run_test "global add-assign" \
    "int g; int main(void) { g = 10; g += 5;
   return g; }" 15

run_test "global mul-assign" \
    "int g; int main(void) { g = 6; g *= 7; return
    g; }" 42

run_test "global sub-assign" \
    "int g; int main(void) { g = 10; g -= 3;
   return g; }" 7

run_test "narrow uchar shr-assign" \
    "int main(void) { unsigned char c = 200; c >>=
    1; return c; }" 100

run_test "narrow ushort shr-assign" \
    "int main(void) { unsigned short s = 40000; s
   >>= 8; return s; }" 156

run_test "unsigned shr-assign loop" \
```

```
    "int main(void) { unsigned incr = 1u << 14;
   unsigned huff = 0x7FFF; while (huff & incr)
   incr >>= 1; return incr == 0; }" 1
```

□

392 · used in 33b

29.9.12. Function Call

A function call walks the argument list (linked via AST_F4), evaluates each argument expression, and stores the resulting vregs into an arena-allocated array. The array layout is [count, vreg0, vreg1, …] so the code generator can marshal arguments into the ABI-correct registers or stack slots. The IR_CALL instruction stores the callee name in SRC1 and the argument array pointer in SRC2; the destination vreg receives the return value.

When the callee has a prototype, each argument is converted to its *parameter's* type before being stored (§6.5.2.2). We walk a parameter cursor (x25) alongside the argument list, reading the parameter declaration's resolved type into x26; the function type carries this list because a plain TY_FUNC would otherwise forget it. An integer argument passed to a double parameter is what makes this load-bearing: without the conversion the value would be marshalled in an integer register and reinterpreted as raw bits, so f(42) against double d would pass garbage. The conversion also drives register classification—once we know the parameter is double, the argument is placed in an SSE register d0–d7 rather than x0–x7. The cursor stops once we run past the fixed parameters (an unprototyped or variadic tail), where each argument keeps its own promoted type.

Two GNU builtins are intercepted by name before dispatch, alongside the variadic operators. __builtin_va_start and __builtin_va_end lower to their variadic handlers (Section 29.9.13), and __builtin_expect(x, c) — a branch-prediction hint — lowers to just its first argument x, since with no profile feedback the hint has no effect on the value. The kernel reaches this builtin constantly: likely(e) and unlikely(e) expand to __builtin_expect(!!(e), 1) and __builtin_expect(!!(e), 0), wrapping arbitrary sub-expressions, calls among them. Were it not intercepted it would be emitted as a direct call to an undefined external __builtin_expect and fail to link.

The callee decides direct versus indirect dispatch. A bare name is a direct IR_CALL by symbol only when it truly names a *function*: it is neither a local variable nor a global object whose resolved type is a pointer. Every other callee is evaluated for its function-pointer value and dispatched through IR_CALL_IND: a local function-pointer variable (found in the locals table), a *global* function-pointer object (a bare name typed TY_PTR), or any other expression such as a struct field o.f, a dereference (*pp), or a subscript table[i]. The global case is easy to miss: the name is not in the locals table, so a local-only test would fall through to a direct call and branch into the data section (a SIGBUS), when the intent is to *load* the pointer from that symbol and call through it. A name with no resolved type—an implicit declaration—is assumed to be a function and called directly. Routing every indirect callee to one path is what lets a function pointer stored in a struct field, or in a file-scope variable, be called.

One subtlety on the indirect path: a *dereferenced* function pointer (*f)(…) must not lower to a load. In C a function designator—which *f is, when f points to a function—decays straight back to the function's address (§6.3.2.1), so (*f), (**f), and f all call the same code. Were we to build (*f) as an ordinary expression it would emit IR_LOAD of the pointer's target—reading the first eight bytes of the callee's *machine code* as if they were an address and branching there (an EXC_BAD_ACCESS, exactly what lua_newstate's (*f)(ud,…) allocator call triggered). So before evaluating the indirect callee we peel any parentheses and any * whose result type is TY_FUNC, leaving the bare pointer expression to build into the function-pointer vreg directly.

⟨ *IR: build call* 393 ⟩ ≡

```
// -- kcc_ir_build_call ---
// uint64_t kcc_ir_build_call(node *expr)
//   x0 = AST_CALL_EXPR (F0=callee, F1=first_arg
   linked via F4)
// Evaluates arguments, stores arg vregs in
   IR_SRC2 array, emits IR_CALL.
// IR_SRC2 points to arena-allocated: [count(8),
   vreg0(8), vreg1(8), ...]
.text
.globl _kcc_ir_build_call
.p2align 2
_kcc_ir_build_call:
    stp x29, x30, [sp, #-128]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #112]      // x25 = param
   cursor, x26 = param type

    str xzr, [sp, #80]             // sret result-
   buffer vreg (0 = none)
    mov x19, x0                    // x19 =
   AST_CALL_EXPR node

    // Get callee expression
    ldr x0, [x19, #AST_F0]        // callee expr
    cbz x0, .Libc_done_zero
    str x0, [sp, #72]             // save callee
   expr node (indirect path rebuilds it)
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_IDENT_EXPR
    b.ne .Libc_callee_indirect    // non-ident
   callee (o.f, (*pp), a[i]) → indirect

    ldr x20, [x0, #AST_F0]        // x20 = callee
   name ptr

    // Variadic builtins: va_start(ap, last) and
   va_end(ap) are lowered
    // specially rather than emitted as calls.
   Their GNU spellings
    // __builtin_va_start / __builtin_va_end (the
   kernel's <stdarg.h> maps the
    // va_* macros to them) are accepted as
   aliases.
    mov x0, x20
    adrp x1, _kcc_ir_str_va_start@PAGE
    add  x1, x1, _kcc_ir_str_va_start@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Libc_va_start
    mov x0, x20
```

```
    adrp x1, _kcc_ir_str_builtin_va_start@PAGE
    add  x1, x1,
_kcc_ir_str_builtin_va_start@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Libc_va_start
    mov x0, x20
    adrp x1, _kcc_ir_str_va_end@PAGE
    add  x1, x1, _kcc_ir_str_va_end@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Libc_va_end
    mov x0, x20
    adrp x1, _kcc_ir_str_builtin_va_end@PAGE
    add  x1, x1,
_kcc_ir_str_builtin_va_end@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Libc_va_end

    // __builtin_expect(x, c) is a branch-
prediction hint; with no profile
    // feedback its value is exactly its first
argument, so lower it to that
    // argument and drop the hint.  The kernel's
likely()/unlikely() expand to
    // it around ordinary sub-expressions (
including calls), so intercepting it
    // by name here keeps those uses from becoming
an undefined external.
    mov x0, x20
    adrp x1, _kcc_ir_str_builtin_expect@PAGE
    add  x1, x1,
_kcc_ir_str_builtin_expect@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Libc_builtin_expect

    // __builtin_constant_p(x): a compile-time
predicate, 1 if x is an integer
    // constant expression and 0 otherwise.  The
kernel gates whole code paths on
    // it; folding it to a constant here lets a
constant `? :` drop its dead arm.
    mov x0, x20
    adrp x1, _kcc_ir_str_builtin_constant_p@PAGE
    add  x1, x1,
_kcc_ir_str_builtin_constant_p@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Libc_builtin_constant_p

    // Direct name, a local function-pointer
variable, or a global one?
    mov x0, x20
    bl _kcc_strlen
    mov x24, x0                     // x24 = name
len
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x20
    mov x2, x24
    bl _kcc_ht_get
    // x0 = vreg+1 (with possible bit-63 tag) or 0
    if not found
    cbnz x0, .Libc_disp_set       // a local
variable → indirect through its vreg
    // Not a local.  A bare name whose resolved
type is a pointer names a
    // global function-pointer *object*: its value
    lives in memory at that
    // symbol, so we must load it and dispatch
indirectly, exactly as for a
    // local fn-ptr.  Emitting a direct IR_CALL to
    the data symbol instead
    // would branch into the data section (SIGBUS)
.  A name typed TY_FUNC (an
    // actual function) or with no resolved type (
an implicit declaration)
    // stays a direct call by name.
    ldr x1, [sp, #72]             // callee ident
node
    ldr x1, [x1, #AST_F3]         // resolved type
    cbz x1, .Libc_disp_set
    ldr x1, [x1, #TY_KIND]
    cmp x1, #TY_PTR
    b.ne .Libc_disp_set
    mov x0, #1                    // global fn-ptr
object → force the indirect path
.Libc_disp_set:
    str x0, [sp, #64]             // locals_result:
    0 → direct call by name
    b .Libc_count_start

.Libc_callee_indirect:
    // A callee that is any expression other than
a bare name: evaluate it
    // for its function-pointer value and dispatch
    indirectly.
    mov x0, #1
    str x0, [sp, #64]             // force the
indirect path

.Libc_count_start:
    // Count arguments (walk F4 chain)
    ldr x21, [x19, #AST_F1]       // x21 = first
arg
    mov x22, #0                     // x22 = arg
count
    mov x0, x21
.Libc_count_args:
    cbz x0, .Libc_count_done
    add x22, x22, #1
    ldr x0, [x0, #AST_F4]
    b .Libc_count_args
.Libc_count_done:

    // Aggregate return-by-value: if the callee
returns a struct OR union,
    // allocate a result buffer whose address
becomes a hidden trailing argument
    // and the call's own result value.  A union
return needs this just as much
    // as a struct: without the buffer the backend
    still unpacks x0(-x1) into
    // DEST, storing through a never-materialised
(garbage) address and faulting.
    ldr x0, [x19, #AST_F3]        // call result
type (sema-resolved)
    cbz x0, .Libc_alloc_args
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_STRUCT
    b.eq .Libc_ret_agg
    cmp x1, #TY_UNION
    b.ne .Libc_alloc_args
.Libc_ret_agg:
    str x0, [sp, #88]             // result
aggregate type
    bl _kcc_ir_type_slots         // x0 = slot
count
    str x0, [sp, #96]             // slot count
    mov x0, #IR_ALLOCA
    bl _kcc_ir_alloc_instr
    ldr x1, [sp, #88]
    str x1, [x0, #IR_TYPE]
    ldr x1, [sp, #96]
    str x1, [x0, #IR_SRC1]        // slot count
    str x0, [sp, #96]             // save instr ptr
    bl _kcc_ir_next_vreg
    ldr x1, [sp, #96]
    str x0, [x1, #IR_DEST]
    add x2, x0, #1                // bias by 1:
vreg 0 is a valid buffer vreg
```

```
    str x2, [sp, #80]           // sret buffer
   vreg + 1 (0 = none)
    mov x0, x1
    bl _kcc_ir_emit

.Libc_alloc_args:
    // Allocate args array.  Layout: args[0] =
   total arg count, then two words
    // per argument: args[1+i*2] = vreg, args[2+i
   *2] = AAPCS64 class.  Size is
    // therefore (1 + total*2) * 8 bytes.
    ldr x0, [sp, #80]           // biased sret
   buffer vreg (0 = none)
    cmp x0, #0
    cset x1, ne                 // has_sret
   {0,1}
    add x1, x22, x1             // total argument
    count
    str x1, [sp, #104]          // total count (
   args[0])
    lsl x0, x1, #1              // total*2
    add x0, x0, #1              // 1 + total*2
    lsl x0, x0, #3              // in bytes
    bl _kcc_arena_alloc
    mov x23, x0                   // x23 = args
   array

    // first_stack_idx: where the caller's stack
   image begins.  For a variadic
    // callee it is min(nfixed, 8): with nfixed<=8
    the variadic tail starts at
    // [sp,#0] (the Apple convention the system
   libc also uses, so interop with
    // sprintf/printf holds); with nfixed>8 it is
   8, so a fixed parameter that
    // overflowed the eight argument registers
   lands on the stack BEFORE the
    // variadic tail rather than colliding with it
   .  A non-variadic call uses 1.
    ldr x1, [sp, #72]           // callee expr
   node
    mov x2, #1
    cbz x1, .Libc_fsi_done
    ldr x1, [x1, #AST_F3]       // callee
   resolved type
    cbz x1, .Libc_fsi_done
    ldr x3, [x1, #TY_KIND]
    cmp x3, #TY_PTR
    b.ne .Libc_fsi_chk
    ldr x1, [x1, #TY_F0]        // pointer-to-
   function → the function type
    cbz x1, .Libc_fsi_done
    ldr x3, [x1, #TY_KIND]
.Libc_fsi_chk:
    cmp x3, #TY_FUNC
    b.ne .Libc_fsi_done
    ldr x3, [x1, #TY_QUALS]     // nfixed if
   variadic, else 0
    cbz x3, .Libc_fsi_done
    mov x2, x3                  // fsi = nfixed
    cmp x2, #8
    b.ls .Libc_fsi_done
    mov x2, #8                  // clamp: nfixed
   >8 → fsi = 8
.Libc_fsi_done:
    ldr x0, [sp, #104]          // total count
    orr x0, x0, x2, lsl #32
    str x0, [x23]               // args[0] =
   count | (first_stack_idx << 32)

    // Parameter cursor for prototyped conversion/
   classification.  The callee's
    // resolved type carries the param declaration
    list in TY_F1 (0 when
    // unprototyped or once variadic overflow
   begins); a function-pointer callee
    // is dereferenced one level to reach the
   function type.
    mov x25, #0
    ldr x0, [sp, #72]           // callee expr
   node
    cbz x0, .Libc_params_ready
    ldr x0, [x0, #AST_F3]       // callee
   resolved type
    cbz x0, .Libc_params_ready
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_FUNC
    b.eq .Libc_params_func
    cmp x1, #TY_PTR
    b.ne .Libc_params_ready
    ldr x0, [x0, #TY_F0]        // pointee
    cbz x0, .Libc_params_ready
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_FUNC
    b.ne .Libc_params_ready
.Libc_params_func:
    ldr x25, [x0, #TY_F1]       // param list
.Libc_params_ready:

    // Evaluate each argument expression: store
   its vreg and AAPCS64 class.
    mov x24, x21                  // x24 =
   current arg node
    mov x21, #0                   // x21 = arg
   index
.Libc_eval_args:
    cmp x21, x22
    b.ge .Libc_eval_done
    // Resolve this position's parameter type (x26
    = 0 for variadic overflow or
    // an unprototyped callee), advancing the
   param cursor.
    mov x26, #0
    cbz x25, .Libc_ptype_done
    ldr x0, [x25, #AST_F0]      // param type node
    cbz x0, .Libc_ptype_adv
    bl _kcc_sema_resolve_type
    mov x26, x0                 // x26 = parameter
    type
.Libc_ptype_adv:
    ldr x25, [x25, #AST_F4]     // advance param
   cursor
.Libc_ptype_done:
    // Classify by the parameter type when known (
   so an int passed to a double
    // parameter lands in an SSE register), else
   by the argument's own type.
    cbnz x26, .Libc_classify_param
    ldr x0, [x24, #AST_F3]      // argument's
   resolved type
    b .Libc_classify_go
.Libc_classify_param:
    mov x0, x26
.Libc_classify_go:
    bl _kcc_ir_classify_type
    str x0, [sp, #104]           // stash class
    cmp x0, #2
    b.lt .Libc_eval_scalar
    mov x0, x24
    bl _kcc_ir_build_struct_addr  // vreg holding
   the struct's address
    b .Libc_eval_store
.Libc_eval_scalar:
    mov x0, x24
    bl _kcc_ir_build_expr          // returns vreg
    in x0
    // Convert the argument to the parameter type
   (int<->double) when prototyped.
```

```
    cbz x26, .Libc_eval_store
    ldr x1, [x24, #AST_F3]      // from = argument
    type
    mov x2, x26                 // to = parameter
   type
    bl _kcc_ir_convert
.Libc_eval_store:
    lsl x1, x21, #1              // index*2
    add x1, x1, #1              // 1 + index*2
    str x0, [x23, x1, lsl #3]    // vreg
    add x1, x1, #1
    ldr x2, [sp, #104]           // class
    str x2, [x23, x1, lsl #3]    // class
    ldr x24, [x24, #AST_F4]      // next arg
    add x21, x21, #1
    b .Libc_eval_args
.Libc_eval_done:

    // Append the hidden result pointer with class
    X8 (5): the marshal loads
    // it into x8 (AAPCS64 indirect-result
   register).  A >16B callee reads it;
    // a <=16B callee ignores x8 and returns the
   aggregate in x0(-x1) instead.
    ldr x0, [sp, #80]            // biased sret
   buffer vreg
    cbz x0, .Libc_dispatch
    sub x0, x0, #1               // recover real
   buffer vreg
    lsl x1, x22, #1              // arg_count*2
    add x1, x1, #1              // 1 + arg_count*2
    str x0, [x23, x1, lsl #3]    // vreg = buffer
   address
    add x1, x1, #1
    mov x2, #5
    str x2, [x23, x1, lsl #3]    // class = X8

.Libc_dispatch:
    // Check if indirect call (callee is a local
   variable)
    ldr x0, [sp, #64]            // locals_result
    cbnz x0, .Libc_indirect

    // Direct call: emit IR_CALL.  IR_TYPE = the
   call's result type, so the
    // backend knows whether the result comes back
    in x0 or d0.
    mov x0, #IR_CALL
    bl _kcc_ir_alloc_instr
    str x20, [x0, #IR_SRC1]      // callee name
    str x23, [x0, #IR_SRC2]      // args array
   ptr
    ldr x1, [x19, #AST_F3]       // call result
   type (x19 = call node)
    cbnz x1, .Libc_dtype
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
.Libc_dtype:
    str x1, [x0, #IR_TYPE]
    mov x19, x0
    // DEST is the sret buffer vreg for a struct
   return (so the backend can
    // unpack x0/x1 into it), otherwise a fresh
   result vreg.
    ldr x1, [sp, #80]
    cbz x1, .Libc_dfresh
    sub x1, x1, #1
    str x1, [x19, #IR_DEST]
    b .Libc_demit
.Libc_dfresh:
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
.Libc_demit:
    mov x0, x19
    bl _kcc_ir_emit
    ldr x0, [x19, #IR_DEST]        // result vreg
   (.Libc_done overrides for struct)
    b .Libc_done

.Libc_indirect:
    // Indirect call: build callee expression to
   get function pointer vreg.
    // First peel parentheses and any dereference
   of a function pointer:
    // `(*f)` is a function designator identical
   to `f`, so building it as a
    // plain expression would wrongly LOAD through
    the pointer.
    ldr x1, [sp, #72]            // callee expr
   node
.Libc_ind_unwrap:
    ldr x2, [x1, #AST_TAG]
    cmp x2, #AST_PAREN_EXPR
    b.ne .Libc_ind_chkderef
    ldr x3, [x1, #AST_F0]        // inner
   expression
    cbz x3, .Libc_ind_build
    mov x1, x3
    b .Libc_ind_unwrap
.Libc_ind_chkderef:
    cmp x2, #AST_UNARY_EXPR
    b.ne .Libc_ind_build
    ldr x3, [x1, #AST_F0]        // unary op
    cmp x3, #TOK_STAR
    b.ne .Libc_ind_build
    ldr x3, [x1, #AST_F3]        // deref result
   type
    cbz x3, .Libc_ind_build
    ldr x3, [x3, #TY_KIND]
    cmp x3, #TY_FUNC             // *f where f ->
   function: designator, no load
    b.ne .Libc_ind_build
    ldr x3, [x1, #AST_F1]        // operand (the
   function pointer expression)
    cbz x3, .Libc_ind_build
    mov x1, x3
    b .Libc_ind_unwrap
.Libc_ind_build:
    mov x0, x1
    bl _kcc_ir_build_expr          // returns fn-
   ptr vreg in x0
    mov x24, x0                    // x24 = fn-ptr
    vreg

    // Emit IR_CALL_IND (result type in IR_TYPE,
   as for the direct call)
    mov x0, #IR_CALL_IND
    bl _kcc_ir_alloc_instr
    str x24, [x0, #IR_SRC1]        // fn-ptr vreg
    str x23, [x0, #IR_SRC2]        // args array
   ptr
    ldr x1, [x19, #AST_F3]        // call result
   type (x19 = call node)
    cbnz x1, .Libc_itype
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
.Libc_itype:
    str x1, [x0, #IR_TYPE]
    mov x19, x0
    ldr x1, [sp, #80]
    cbz x1, .Libc_ifresh
    sub x1, x1, #1
    str x1, [x19, #IR_DEST]
    b .Libc_iemit
.Libc_ifresh:
```

```
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
.Libc_iemit:
    mov x0, x19
    bl _kcc_ir_emit
    ldr x0, [x19, #IR_DEST]
    b .Libc_done

.Libc_va_start:
    mov x0, x19                     // call node
    bl _kcc_ir_build_va_start
    b .Libc_epilogue

.Libc_va_end:
    mov x0, #0                      // va_end is a no-
   op in this ABI
    b .Libc_epilogue

.Libc_builtin_expect:
    ldr x0, [x19, #AST_F1]          // first argument
   (the predicted expression)
    bl _kcc_ir_build_expr           // its vreg is the
    whole builtin's value
    b .Libc_epilogue

.Libc_builtin_constant_p:
    ldr x0, [x19, #AST_F1]          // first argument
    bl _kcc_expr_is_ice             // 1 if a constant
    expression, else 0
    bl _kcc_ir_emit_const           // materialise
   that 0/1
    b .Libc_epilogue

.Libc_done_zero:
    mov x0, #0
.Libc_done:
    // A struct-returning call yields the address
   of its result buffer,
    // not the (unused) scalar result vreg.
    ldr x1, [sp, #80]               // biased sret
   buffer vreg
    cbz x1, .Libc_epilogue
    sub x0, x1, #1                  // recover real
   buffer vreg
.Libc_epilogue:
    ldp x25, x26, [sp, #112]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #128
    ret
```

□

393 · used in 30b · defines _kcc_ir_build_call 340, 387, 405 · uses _kcc_arena_alloc 58a, _kcc_expr_is_ice 426, _kcc_ht_get 62, _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_build_struct_addr 405, _kcc_ir_build_va_start 398, _kcc_ir_classify_type 343, _kcc_ir_convert 343, _kcc_ir_emit 304, _kcc_ir_locals 305c, _kcc_ir_next_vreg 305a, _kcc_ir_str_builtin_constant_p 398, _kcc_ir_str_builtin_expect 398, _kcc_ir_str_builtin_va_end 398, _kcc_ir_str_builtin_va_start 398, _kcc_ir_str_va_end 398, _kcc_ir_str_va_start 398, _kcc_ir_type_slots 349a, _kcc_sema_resolve_type 259b, _kcc_type_int_s 250b

29.9.13. Variadic Function Definitions

A function *defined* with a trailing `...` reads its unnamed arguments through the <stdarg.h> interface—`va_list`, `va_start`, `va_arg`, `va_end`. kcc provides these as built-ins rather than a header. The call lowering pushes the variadic tail onto the stack (the Apple variadic ABI push), so a variadic callee needs only a cursor into that stack area. Accordingly `va_list` is an opaque eight-byte pointer (resolved by sema); `va_start(ap, last)` points the cursor at the first variadic argument; `va_arg(ap, T)` reads the eight-byte slot at the cursor and advances it; and `va_end` is a no-op.

kcc also answers to the GNU `__builtin_` spellings of these operators—`__builtin_va_start`, `__builtin_va_arg`, `__builtin_va_end`, and the type `__builtin_va_list`—as exact aliases for the short names. A freestanding <stdarg.h> (the kernel's, reached under `-nostdinc`) is written entirely in terms of these builtins: `typedef __builtin_va_list va_list;` and `#define va_arg(v,T) __builtin_va_arg(v,T)`, so once the macros expand it is the `__builtin_` form that reaches the parser and the IR builder. Recognising both spellings lets kcc consume such a header without a built-in <stdarg.h> of its own. (`va_copy`/`__builtin_va_copy` is not yet provided.)

Both sides depend on the prototype's fixed-parameter count. Sema records it when it checks a function declaration, storing the count in the otherwise unused `TY_QUALS` slot of the `TY_FUNC` descriptor (zero there means non-variadic); `kcc_ir_build_call` reads it back from the callee's type to choose the stack layout, and the caller places the variadic tail at `[sp,#0]` of the pushed argument area (`kcc_cg_marshal_args` indexes the stack image from that count), so in the callee the first variadic argument lives at `[x29 + frame_size]`—independent of how many fixed parameters preceded it, and with `frame_size` known only in the backend. `va_start` therefore lowers to a dedicated `IR_VA_START` instruction; the code generator materialises that address and `va_start` stores it into ap. (The instruction still records the fixed count F in `SRC2` for diagnostics, though the offset no longer depends on it.) `va_arg` needs no new opcode: it loads the cursor, dereferences it, adds eight, and writes the cursor back, all with ordinary IR. Both builtins locate ap through `_kcc_ir_ident_alloca`, which resolves a local identifier to its alloca vreg.

⟨ *IR: variadic builtins* 398 ⟩ ≡

```
// -- kcc_ir_ident_alloca ---
// uint64_t kcc_ir_ident_alloca(node *ident)
//   x0 = AST_IDENT_EXPR.  Returns the identifier'
   s alloca vreg, or ~0 when
//   it is not a known local.
.text
.globl _kcc_ir_ident_alloca
.p2align 2
_kcc_ir_ident_alloca:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    cbz x0, .Lia_err
    // The va_list expression is usually a plain
   identifier, but the standard
    // allows any lvalue --- an array element (`
   aps[4]`) or a struct member
    // (`a.g`).  Route those to the subscript/
   member address builders, whose
```

```
    // result is exactly the va_list storage
   address the callers want.
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_IDENT_EXPR
    b.eq .Lia_ident
    cmp x1, #AST_SUBSCRIPT
    b.eq .Lia_subscript_lv
    cmp x1, #AST_MEMBER
    b.eq .Lia_member_lv
    b .Lia_err
.Lia_subscript_lv:
    bl _kcc_ir_build_subscript_addr
    b .Lia_done
.Lia_member_lv:
    bl _kcc_ir_build_member_addr
    b .Lia_done
.Lia_ident:
    ldr x19, [x0, #AST_F0]        // name ptr
    cbz x19, .Lia_err
    mov x0, x19
    bl _kcc_strlen
    mov x2, x0
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x19
    bl _kcc_ht_get
    cbz x0, .Lia_none
    bic x0, x0, #0x8000000000000000
    sub x0, x0, #1
    b .Lia_done
.Lia_err:
    mov x0, #-1
    b .Lia_done
.Lia_none:
    // Not a local: a file-scope `va_list` (e.g. a
    global `va_list gap`).  Its
    // storage address is a global reference,
   which serves exactly as the local
    // alloca vreg would --- va_start/va_arg both
   want the address of the
    // va_list object.  x19 still holds the
   interned name.
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x19, [x0, #IR_SRC1]       // symbol name
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x20, x0
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x19, x0                   // global-ref
   vreg
    mov x0, x20
    bl _kcc_ir_emit
    mov x0, x19
.Lia_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_build_va_start ---
// void kcc_ir_build_va_start(node *call)   - call
    = va_start(ap, last)
.text
.globl _kcc_ir_build_va_start
.p2align 2
_kcc_ir_build_va_start:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                   // call node
    ldr x0, [x19, #AST_F1]        // first arg = ap
    ident
    bl _kcc_ir_ident_alloca
    mov x20, x0                   // ap alloca vreg
    // F = current function's fixed-parameter
   count
    adrp x0, _kcc_ir_cur_func@PAGE
    add  x0, x0, _kcc_ir_cur_func@PAGEOFF
    ldr x0, [x0]
    ldr x21, [x0, #IRF_PARAMS]
    // IR_VA_START: DEST = base-address vreg, SRC2
    = F
    mov x0, #IR_VA_START
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x21, [x22, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                   // base-address
   vreg
    mov x0, x22
    bl _kcc_ir_emit
    // *ap = base
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x21, [x22, #IR_SRC1]      // value = base
   address
    str x20, [x22, #IR_SRC2]      // pointer = ap's
    storage
    mov x0, x22
    bl _kcc_ir_emit
    mov x0, x21
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_build_va_arg ---
// uint64_t kcc_ir_build_va_arg(node *va_arg)
//   Reads one argument from the va_list cursor
   and advances it by one 8-byte
//   slot.  For a scalar the loaded 8 bytes are
   the value.  A struct/union is
//   passed through `...` by \emph{address} in kcc
   's ABI (the caller's
//   marshal-args stores the object's address in
   the variadic stack slot, see
//   \S\ref{sec:aapcs64-marshal}), so the same
   loaded 8 bytes ARE the object's
//   address --- exactly what an aggregate rvalue
   is elsewhere --- and the
//   caller (a struct copy) reads the object
   through it.
.text
.globl _kcc_ir_build_va_arg
.p2align 2
_kcc_ir_build_va_arg:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                   // va_arg node
    ldr x0, [x19, #AST_F0]        // ap ident
```

```
    bl _kcc_ir_ident_alloca
    mov x20, x0                     // ap alloca vreg
    // p = *ap  (current cursor)
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x20, [x22, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                     // p
    mov x0, x22
    bl _kcc_ir_emit
    // val = *p  (one 8-byte argument slot: a
  scalar value, or a struct address)
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x21, [x22, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x23, x0                     // val
    mov x0, x22
    bl _kcc_ir_emit
    // c8 = 8
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    mov x1, #8
    str x1, [x22, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x24, x0                     // c8
    mov x0, x22
    bl _kcc_ir_emit
    // p2 = p + 8
    mov x0, #IR_ADD
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x21, [x22, #IR_SRC1]
    str x24, [x22, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                     // p2
    mov x0, x22
    bl _kcc_ir_emit
    // *ap = p2
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x21, [x22, #IR_SRC1]
    str x20, [x22, #IR_SRC2]
    mov x0, x22
    bl _kcc_ir_emit
    mov x0, x23                     // return val (
  scalar value, or struct address)
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.section __TEXT,__cstring,cstring_literals
_kcc_ir_str_va_start:    .asciz "va_start"
_kcc_ir_str_va_end:      .asciz "va_end"
_kcc_ir_str_builtin_va_start: .asciz "
  __builtin_va_start"
_kcc_ir_str_builtin_va_end:   .asciz "
  __builtin_va_end"
_kcc_ir_str_builtin_expect:   .asciz "
  __builtin_expect"
_kcc_ir_str_builtin_constant_p: .asciz "
  __builtin_constant_p"
.text
```

□

398 · used in 30b · defines _kcc_ir_build_va_arg 340, 387, _kcc_ir_build_va_start 393, _kcc_ir_ident_alloca never used, _kcc_ir_str_builtin_constant_p 393, _kcc_ir_str_builtin_expect 393, _kcc_ir_str_builtin_va_end 393, _kcc_ir_str_builtin_va_start 393, _kcc_ir_str_va_end 393, _kcc_ir_str_va_start 393 · uses _kcc_ht_get 62, _kcc_ir_alloc_instr 303a, _kcc_ir_build_member_addr 355, _kcc_ir_build_subscript_addr 350, _kcc_ir_cur_func 305c, _kcc_ir_emit 304, _kcc_ir_locals 305c, _kcc_ir_next_vreg 305a, _kcc_type_int_s 250b

End-to-end tests. The `IR_CALL` instruction carries a callee name and an arena-allocated argument array. The code generator loads each argument vreg from its stack slot into x0–x7 (following AAPCS64), emits a `BL` with an `ARM64_RELOC_BRANCH26` relocation, and stores the return value from x0 into the destination vreg's slot. Locally defined functions become `N_SECT|N_EXT` symbols; external calls become `N_UNDF|N_EXT` symbols resolved by the linker. The recursive factorial test is particularly demanding: it verifies that the prologue and epilogue correctly save and restore the frame pointer and link register across a deep call chain.

⟨ *Test: codegen functions* 400a ⟩ ≡

```
echo ""
echo "--- Functions ---"

run_test "simple call" \
    "int foo(int x) { return x + 1; } int main(
   void) { return foo(41); }" 42

run_test "nested calls" \
    "int foo(int x) { return x + 1; } int main(
   void) { return foo(foo(1)); }" 3

run_test "multi-arg" \
    "int add(int a, int b) { return a + b; } int
   main(void) { return add(20, 22); }" 42

run_test "three args" \
    "int sum3(int a, int b, int c) { return a + b
   + c; } int main(void) { return sum3(10, 20, 12)
   ; }" 42

run_test "recursive factorial" \
```

```
    "int fact(int n) { if (n <= 1) return 1;
   return n * fact(n - 1); } int main(void) {
   return fact(5); }" 120
```

□

400a · used in 33b

29.9.14. String Literal

A string literal is represented by IR_STRING_LIT, which carries the data pointer in SRC1 and the byte length in SRC2. The code generator (Chapter 35) later appends this data to the __text section and emits a PC-relative reference so the vreg holds the string's run-time address.

⟨ *IR: build string literal* 400b ⟩ ≡

```
// -- kcc_ir_build_string_lit ---
// uint64_t kcc_ir_build_string_lit(node *lit)
//   x0 = AST_STR_LIT (F0=str_ptr, F1=str_len)
// Returns vreg holding the string's address.
.text
.globl _kcc_ir_build_string_lit
.p2align 2
_kcc_ir_build_string_lit:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0
    ldr x20, [x19, #AST_F0]      // str_ptr
    ldr x21, [x19, #AST_F1]      // str_len

    mov x0, #IR_STRING_LIT
    bl _kcc_ir_alloc_instr
    mov x19, x0                   // save instr

    str x20, [x19, #IR_SRC1]     // str data
   pointer
    str x21, [x19, #IR_SRC2]     // str length

    // Type: use int for now (value is a 64-bit
   pointer)
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr x0, [x0]
    str x0, [x19, #IR_TYPE]

    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x20, x0                   // save vreg

    mov x0, x19
    bl _kcc_ir_emit

    mov x0, x20                   // return vreg

    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

400b · used in 30b · defines _kcc_ir_build_string_lit 340 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_emit 304, _kcc_ir_next_vreg 305a, _kcc_type_int_s 250b

29.9.15. Conditional Expression

The ternary operator cond ? then : else is lowered to three fresh basic blocks plus a temporary alloca that serves as a rendezvous point. The condition vreg feeds an IR_BR_COND that routes control to the *then_bb* or *else_bb*. Each arm evaluates its expression and *stores* the result into the shared alloca, then branches unconditionally to the *merge_bb*. The merge block loads the result back from the alloca, producing a single vreg that the caller can reference regardless of which branch executed.

This store/load pattern replaces the earlier approach of "returning the last vreg," which failed because the then-branch vreg was never referenced in the merge block and DCE eliminated it. The alloca approach works naturally with the all-spill code generator and avoids phi nodes entirely.

⟨ *IR: build conditional* 400c ⟩ ≡

```
// -- kcc_ir_build_cond_expr ---
// uint64_t kcc_ir_build_cond_expr(node *expr)
//   x0 = AST_COND_EXPR (F0=cond, F1=then, F2=else
   )
// Lowers to: alloca tmp → br_cond → then: store
   → br merge
//                                  → else: store
   → br merge
//            merge: load tmp → return vreg
.text
.globl _kcc_ir_build_cond_expr
.p2align 2
_kcc_ir_build_cond_expr:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]

    mov x19, x0

    // Allocate a temporary slot for the result
    mov x0, #IR_ALLOCA
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x20, x0                   // x20 = alloca
   instr
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x25, x0                   // x25 = alloca
   vreg (result slot)
    mov x0, x20
    bl _kcc_ir_emit

    // Build condition
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_build_expr
    mov x20, x0                   // cond vreg

    // Create then/else/merge BBs
    bl _kcc_ir_alloc_bb
    mov x21, x0                   // then_bb
    bl _kcc_ir_alloc_bb
    mov x22, x0                   // else_bb
    bl _kcc_ir_alloc_bb
    mov x23, x0                   // merge_bb
```

```
    // Branch on condition
    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
    str x20, [x0, #IR_DEST]
    ldr x1, [x21, #BB_ID]
    str x1, [x0, #IR_SRC1]
    ldr x1, [x22, #BB_ID]
    str x1, [x0, #IR_SRC2]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // Then branch: evaluate, store to alloca,
   branch to merge
    mov x0, x21
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F1]
    cbz x0, .Lcond_elvis_then     // GNU `a ?: b':
   reuse the condition value
    bl _kcc_ir_build_expr
    mov x24, x0                   // then_vreg
    b .Lcond_then_stored
.Lcond_elvis_then:
    mov x24, x20                  // omitted middle
   : the condition (x20) is the value
.Lcond_then_stored:

    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    str x24, [x0, #IR_SRC1]      // value
    str x25, [x0, #IR_SRC2]      // alloca
    bl _kcc_ir_emit

    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x23, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // Else branch: evaluate, store to alloca,
   branch to merge
    mov x0, x22
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_build_expr
    mov x24, x0                   // else_vreg

    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    str x24, [x0, #IR_SRC1]      // value
    str x25, [x0, #IR_SRC2]      // alloca
    bl _kcc_ir_emit

    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x23, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // Merge: load from alloca → result vreg
    mov x0, x23
    bl _kcc_ir_set_bb

    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    str x25, [x0, #IR_SRC1]      // alloca vreg
    mov x20, x0
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x24, x0                   // result vreg
    mov x0, x20
    bl _kcc_ir_emit

    mov x0, x24                   // return result
   vreg

    ldr x25, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

400c · used in 30b · defines _kcc_ir_build_cond_expr 340 · uses _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_cur_bb 305c, _kcc_ir_emit 304, _kcc_ir_next_vreg 305a, _kcc_ir_set_bb 305b, _kcc_type_int_s 250b

End-to-end tests. The alloca rendezvous pattern ensures that whichever branch executes, the result is stored to a known location and loaded in the merge block. These tests cover both truth values, variable conditions, initialiser context, nesting, use as a function argument, and sub-expression evaluation within the arms.

⟨ *Test: codegen ternary* 402a ⟩ ≡

```
echo ""
echo "--- Ternary Operator ---"

run_test "ternary-true" \
    "int main(void) { return 1 ? 42 : 99; }" 42

run_test "ternary-false" \
    "int main(void) { return 0 ? 42 : 99; }" 99

run_test "ternary-var-cond" \
    "int main(void) { int x = 1; return x ? 10 :
   20; }" 10

run_test "ternary-var-init" \
    "int main(void) { int x = 1 ? 5 : 9; return x;
    }" 5
```

```
run_test "ternary-nested" \
    "int main(void) { return 1 ? (0 ? 1 : 2) : 3;
   }" 2

run_test "ternary-expr-arms" \
    "int main(void) { int a=10; int b=3; return 1
   ? a+b : a-b; }" 13

run_test "ternary-false-path" \
    "int main(void) { int a=10; int b=3; return 0
   ? a+b : a-b; }" 7
```

□

402a · used in 33b

29.10. Building Cast Expressions

An explicit cast `(T)e` compares the target type `T` with the source type of the inner expression. Any cast that touches a floating type is delegated to `_kcc_ir_convert`; the two `_kcc_ir_type_is_fp` probes are branched on *immediately* rather than OR-ed together, because `is_fp` uses `x1` as scratch and a stale target result would poison the decision — an early version held `target_is_fp` in `x1` across the second call, so every integer cast saw a non-zero type kind there and was misrouted into the FP path (which no-ops integer resizes), silently dropping the conversion.

For integer targets, when the target is strictly narrower (by `TY_SIZE`) the builder emits `IR_TRUNC` to mask the result to the target width; when it is wider, it emits `IR_SEXT` for signed source types and `IR_ZEXT` for unsigned ones, with the source width in `IR_SRC2`. Equal-width casts are *not* always no-ops: a sub-word value (`char`/`short`) is physically held in a register extended per its own signedness, so a same-size cast that only flips signedness must re-extend the low `size` bytes to the target's convention (`IR_ZEXT` towards unsigned, `IR_SEXT` towards signed). Without this, `(unsigned char)(signed char)0xFF` stays -1 instead of 255 — precisely the misread that made Lua's `zgetc` treat every `0xFF` byte as `EOZ` and reject valid bytecode as a truncated chunk. Word-width targets (≥ 4 bytes) share a bit pattern under AArch64's zero-extending 32-bit operations and are left unchanged.

⟨ *IR: build cast* 402b ⟩ ≡

```
// -- kcc_ir_build_cast ---
// uint64_t kcc_ir_build_cast(node *cast)
//   x0 = AST_CAST_EXPR (F0=type, F1=expr, F3=
   resolved target type)
// Returns vreg.
.text
.globl _kcc_ir_build_cast
.p2align 2
_kcc_ir_build_cast:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    mov x19, x0                    // cast node

    // Save inner expr pointer before building
    ldr x20, [x19, #AST_F1]      // inner
   expression node

    // Build inner expression
    mov x0, x20
    bl _kcc_ir_build_expr
    mov x21, x0                    // src_vreg

    // Get target type from cast node's F3 (set by
    sema)
    ldr x22, [x19, #AST_F3]
    cbz x22, .Lbc_done

    // Get source type from inner expr's F3
    ldr x23, [x20, #AST_F3]
    cbz x23, .Lbc_done

    // Any conversion touching a floating type is
   delegated to
    // _kcc_ir_convert, which inserts sitofp /
   fptosi as needed and rounds to
    // single precision when the target is float.
    Integer-only casts fall
    // through to the width-resize path below.  We
    test each operand
    // separately and branch immediately: holding
   one is_fp result across the
    // second call is unsafe, because
   _kcc_ir_type_is_fp clobbers x1 as scratch
    // (it reads TY_KIND into x1).  A stale x1 OR-
   ed into the decision would be
    // a non-zero type kind, mis-routing every
   integer cast into _kcc_ir_convert
    // (which no-ops integer width changes),
   silently dropping the conversion.
    mov x0, x22
    bl _kcc_ir_type_is_fp
    cbnz x0, .Lbc_fp_convert
    mov x0, x23
    bl _kcc_ir_type_is_fp
    cbz x0, .Lbc_int_resize
.Lbc_fp_convert:
    mov x0, x21                    // src vreg
    mov x1, x23                    // from type
    mov x2, x22                    // to type
    bl _kcc_ir_convert
    mov x21, x0
    b .Lbc_done

.Lbc_int_resize:
    // A cast to _Bool normalises to 0/1, not a
   low-byte truncation.
    ldr x0, [x22, #TY_KIND]
    cmp x0, #TY_BOOL
    b.ne .Lbc_resize_sizes
    mov x0, x21
    bl _kcc_ir_emit_bool_norm
    mov x21, x0
    b .Lbc_done
.Lbc_resize_sizes:
    // Compare sizes
    ldr x1, [x22, #TY_SIZE]      // target size
    ldr x2, [x23, #TY_SIZE]      // source size

    cmp x1, x2
    b.eq .Lbc_same_size          // same width:
   maybe re-extend for signedness
    // Both narrowing and widening re-canonicalise
    to the target: extend the low
    // target-width bytes by the target's
   signedness (SEXT/ZEXT below).  A single
    // SEXT/ZEXT with source-width = target width
   does the narrow (it ignores the
    // upper source bytes) and the widen (the low
   bytes are already the value)
    // alike, and always leaves the register in
```

```
    the target type's canonical form.
    b .Lbc_extend

.Lbc_same_size:
    // Equal byte width.  A value narrower than 8
    bytes is physically held in a
    // 64-bit register sign- or zero-extended
    according to its own type, so a
    // cast that only flips signedness must re-
    extend the low `size` bytes to
    // match the TARGET type.  Two cases motivate
    this.  At sub-word width,
    // (unsigned char)(signed char)0xFF must
    become 255, not -1 -- exactly how
    // Lua's zgetc misread every 0xFF byte as EOZ.
     At word width (4 bytes) the
    // same applies whenever the source's upper 32
    bits are dirty: a negative
    // constant like -1 is held as 0
    xFFFFFFFFFFFFFFFF, so (unsigned)-1 must be
    // masked to 0xFFFFFFFF -- otherwise `((
    unsigned)-1 >> 2) + 1` folds to 0
    // instead of 0x40000000 (zlib's gz_comp used
    exactly that for its write
    // chunk size, and a 0 made it loop forever
    writing nothing).  Only a full
    // 8-byte value uses every bit already and
    needs no re-extension.
    cmp x1, #8
    b.hs .Lbc_done
    ldr x3, [x22, #TY_F0]          // target
    signedness (1=signed, 0=unsigned)
    ldr x4, [x23, #TY_F0]          // source
    signedness
    cmp x3, x4
    b.eq .Lbc_done                 // same
    signedness: nothing to materialise
    cbz x3, .Lbc_ss_zext
    mov x0, #IR_SEXT
    b .Lbc_ss_emit
.Lbc_ss_zext:
    mov x0, #IR_ZEXT
.Lbc_ss_emit:
    bl _kcc_ir_alloc_instr
    mov x20, x0
    str x22, [x20, #IR_TYPE]     // target type
    str x21, [x20, #IR_SRC1]     // source vreg
    ldr x0, [x22, #TY_SIZE]      // re-extend from
    `size` bytes
    str x0, [x20, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x21, x0
    mov x0, x20
    bl _kcc_ir_emit
    b .Lbc_done

.Lbc_trunc:
    mov x0, #IR_TRUNC
    bl _kcc_ir_alloc_instr
    mov x20, x0

    str x22, [x20, #IR_TYPE]     // target type
    str x21, [x20, #IR_SRC1]     // source vreg

    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x21, x0

    mov x0, x20
    bl _kcc_ir_emit
    b .Lbc_done

.Lbc_extend:
    // Re-canonicalise to the TARGET's
    representation: extend the value per the
    // TARGET's signedness and width.  The result'
    s low target-width bits are
    // already correct in the source register, so
    extending from the target
    // width by the target's sign gives the right
    64-bit value.  Using the
    // SOURCE's signedness here sign-extended a
    signed-to-wider-unsigned cast
    // like (unsigned short)(signed char)-1 to 0
    xFFFF...FF instead of 65535, so
    // a following bitwise operation saw dirty
    upper bits and computed the wrong
    // result (a char/short xor promoted to int).
    ldr x0, [x22, #TY_F0]        // TARGET
    signedness (1=signed, 0=unsigned)
    cbz x0, .Lbc_zext
    mov x0, #IR_SEXT
    b .Lbc_ext_emit
.Lbc_zext:
    mov x0, #IR_ZEXT

.Lbc_ext_emit:
    bl _kcc_ir_alloc_instr
    mov x20, x0

    str x22, [x20, #IR_TYPE]     // target type
    str x21, [x20, #IR_SRC1]     // source vreg
    ldr x0, [x22, #TY_SIZE]      // TARGET width in
    bytes
    str x0, [x20, #IR_SRC2]      // pass to codegen

    bl _kcc_ir_next_vreg
    str x0, [x20, #IR_DEST]
    mov x21, x0

    mov x0, x20
    bl _kcc_ir_emit

.Lbc_done:
    mov x0, x21                   // return vreg

    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

402b · used in 30b · defines _kcc_ir_build_cast 340 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_convert 343, _kcc_ir_emit 304, _kcc_ir_next_vreg 305a, _kcc_ir_type_is_fp 343

29.11. Building Variable Declarations

A variable declaration emits `IR_ALLOCA` to reserve a stack slot, then records the mapping `name` → `vreg+1` in the locals hash table (the +1 bias lets zero represent "not found"). If the declaration carries an initialiser, the builder evaluates the expression and emits `IR_STORE` to write the initial value into the alloca. The type is resolved via `_kcc_sema_resolve_type`; if resolution fails, the builder falls back to `i32`.

Pointer tagging. The locals entry carries a one-bit annotation in bit 63 of its value: set for pointer *variables*, clear for arrays and scalars. The subscript builder reads this bit to choose its addressing mode—a pointer variable is loaded

and then offset, whereas an array is indexed directly from its alloca base (§29.9.4). The distinction must be drawn from the *declared* type, which is why the flag is captured at `.Libvd_alloca` before the array branch overwrites the working type register with the element type. Were the bit derived after that point, an array of pointers such as `int *a[N]` would inherit its element's `TY_PTR` kind and be mistaken for a pointer variable; subscripting would then load `a[0]`'s contents as a base address and compute a wild pointer. An array is an array regardless of what it holds, so the flag reflects only the top-level declared kind.

Struct copy. When the declared type is `TY_STRUCT`, a single load/store would copy only the first 8-byte slot. Instead, the builder resolves the source struct's base address—an alloca vreg for locals or a `global_ref` for globals—and calls `_kcc_ir_emit_struct_copy`, which emits a `GEP+LOAD+GEP+STORE` sequence walking the object in descending power-of-two chunks: as many 8-byte transfers as fit, then a 4-byte, 2-byte, and 1-byte tail as the size requires, for *exactly* `TY_SIZE` bytes. An earlier version copied `ceil(TY_SIZE/8)` whole 8-byte slots on the theory that an aggregate's storage is always rounded up to a slot, so the trailing padding belonged to the object. That assumption fails for a struct stored into a *packed* location—an element of an array of sub-8-byte structs, such as zlib's Huffman decode table (`code table[]` with `sizeof(code)==4`): an 8-byte store to `table[i]` silently overwrote the first four bytes of `table[i+1]`, corrupting the decode tables so inflation failed. Copying the exact size also avoids reading past a caller's struct on a by-value or `sret` copy. The tail transfers use the narrow load/store access-size codes (word/half/byte), so the copy stays independent of the packed field layout. The same helper is reused by the assignment builder for statements like `cur = saved;`.

⟨ *IR: struct copy helper* 404 ⟩ ≡

```
// -- kcc_ir_emit_struct_copy ---
// void kcc_ir_emit_struct_copy(src_base_vreg,
   dst_base_vreg, byte_size)
//   x0 = source base vreg
//   x1 = destination base vreg
//   x2 = number of bytes to copy (TY_SIZE, exact)
// Emits GEP+LOAD+GEP+STORE per chunk, choosing
   the largest power-of-two
// transfer (8/4/2/1 bytes) that still fits in the
    remaining bytes so the
// copy touches exactly byte_size bytes and never
   spills into a neighbour.
.text
.globl _kcc_ir_emit_struct_copy
.p2align 2
_kcc_ir_emit_struct_copy:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]

    mov x19, x0                  // src base vreg
    mov x20, x1                  // dst base vreg
    str x2, [sp, #80]            // total byte
   size
    mov x22, #0                  // current byte
   offset

.Liesc_loop:
    ldr x0, [sp, #80]
    sub x0, x0, x22              // remaining
   bytes
    cmp x0, #0
    b.le .Liesc_done

    // Pick the largest chunk (x25) that fits, and
    its access-size code (x26):
    // →80 (default 64-bit), →44 (word), →22 (half
   ), →11 (byte).
    cmp x0, #8
    b.ge .Liesc_c8
    cmp x0, #4
    b.ge .Liesc_c4
    cmp x0, #2
    b.ge .Liesc_c2
    mov x25, #1
    mov x26, #1
    b .Liesc_have
.Liesc_c8:
    mov x25, #8
    mov x26, #0
    b .Liesc_have
.Liesc_c4:
    mov x25, #4
    mov x26, #4
    b .Liesc_have
.Liesc_c2:
    mov x25, #2
    mov x26, #2
.Liesc_have:

    // IR_CONST byte offset = current offset (
   stride-1 GEP adds it directly)
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x21, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x21, #IR_TYPE]
    str x22, [x21, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x24, x0                  // x24 = idx_vreg
    mov x0, x21
    bl _kcc_ir_emit

    // IR_GEP src_base, idx → src element address
    mov x0, #IR_GEP
    bl _kcc_ir_alloc_instr
    mov x21, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x21, #IR_TYPE]
    str x19, [x21, #IR_SRC1]
    str x24, [x21, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x23, x0                  // x23 = src_elem
   vreg
    mov x0, x21
    bl _kcc_ir_emit

    // IR_LOAD src_elem → value, with the chunk's
   access-size code
    mov x0, #IR_LOAD
    bl _kcc_ir_alloc_instr
```

```
    mov x21, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x21, #IR_TYPE]
    str x23, [x21, #IR_SRC1]
    str x26, [x21, #IR_SRC2]      // load access-
  size code
    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x23, x0                   // x23 = val vreg
    mov x0, x21
    bl _kcc_ir_emit

    // IR_GEP dst_base, idx → dst element address
    mov x0, #IR_GEP
    bl _kcc_ir_alloc_instr
    mov x21, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x21, #IR_TYPE]
    str x20, [x21, #IR_SRC1]
    str x24, [x21, #IR_SRC2]
    bl _kcc_ir_next_vreg
    str x0, [x21, #IR_DEST]
    mov x24, x0                   // x24 = dst_elem
    vreg
    mov x0, x21
    bl _kcc_ir_emit

    // IR_STORE value → dst element, with the
  chunk's access-size code
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    mov x21, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x21, #IR_TYPE]
    str x23, [x21, #IR_SRC1]
    str x24, [x21, #IR_SRC2]
    str x26, [x21, #IR_DEST]      // store access-
  size code
    mov x0, x21
    bl _kcc_ir_emit

    add x22, x22, x25             // advance by the
    chunk size
    b .Liesc_loop

.Liesc_done:
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

404 · used in 30b · defines _kcc_ir_emit_struct_copy 314b, 320, 326, 387, 414, 417b · uses _kcc_ir_alloc_instr 303a, _kcc_ir_emit 304, _kcc_ir_next_vreg 305a, _kcc_type_int_s 250b

The address of a struct value. Both struct return (§29.6) and struct pass-by-value (§29.3) need the *address* of a struct-valued expression rather than a loaded scalar, because an aggregate never fits a single register. _kcc_ir_build_struct_addr centralises that resolution: an identifier yields its alloca vreg (or a global_ref for a file-scope struct), a call yields the address of its own result buffer (the struct-return path already returns exactly that), and .f/[i] lvalues reuse the member and subscript address builders. A parenthesised expression is unwrapped and re-dispatched. A comma operator evaluates its discarded left side and takes the address of its aggregate right; a nested assignment is built (performing its copy) and yields the assigned object's address—so f((setup(), s)) and f((a = s)) pass s by value rather than reading it loaded as a scalar and faulting on it as an address. Anything else falls back to build_expr, which for a pointer operand already produces an address.

⟨ IR: struct address helper 405 ⟩ ≡

```
// -- kcc_ir_build_struct_addr ---
// uint64_t kcc_ir_build_struct_addr(node *expr)
//   x0 = expression node of struct type
//   returns x0 = vreg holding the struct's base
  address (0 if unknown)
.text
.globl _kcc_ir_build_struct_addr
.p2align 2
_kcc_ir_build_struct_addr:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    mov x19, x0
    cbz x19, .Libsad_zero
    ldr x1, [x19, #AST_TAG]
    cmp x1, #AST_PAREN_EXPR
    b.ne .Libsad_dispatch
    ldr x19, [x19, #AST_F0]       // unwrap
  parentheses
    cbz x19, .Libsad_zero
    ldr x1, [x19, #AST_TAG]

.Libsad_dispatch:
    cmp x1, #AST_IDENT_EXPR
    b.eq .Libsad_ident
    cmp x1, #AST_CALL_EXPR
    b.eq .Libsad_call
    cmp x1, #AST_MEMBER
    b.eq .Libsad_member
    cmp x1, #AST_SUBSCRIPT
    b.eq .Libsad_subscript
    cmp x1, #AST_UNARY_EXPR
    b.eq .Libsad_deref
    cmp x1, #AST_COMPOUND_LIT
    b.eq .Libsad_compound
    cmp x1, #AST_COMMA_EXPR
    b.eq .Libsad_comma
    cmp x1, #AST_ASSIGN_EXPR
    b.eq .Libsad_assign
    mov x0, x19
    bl _kcc_ir_build_expr         // fallback:
  operand already yields an address
    b .Libsad_done

.Libsad_comma:
    // `f((setup(), s))`: a comma operator whose
  value is the aggregate `s`.
    // Evaluate the discarded left side for its
  effects, then take the address of
    // the right (recursively, in case it is
  itself an lvalue shape).  Without
    // this the comma matched no case and
  build_expr returned the union LOADED as
    // a scalar; the caller then read that value
  as an address and faulted.
    ldr x0, [x19, #AST_F0]        // comma LHS
```

```
    bl _kcc_ir_build_expr        // side effects,
   discarded
    ldr x0, [x19, #AST_F1]       // comma RHS =
   the aggregate
    bl _kcc_ir_build_struct_addr
    b .Libsad_done

.Libsad_assign:
    // `f((a = s))`: the value is the object just
   assigned; build the assignment
    // (which performs the copy and returns that
   object's address).
    mov x0, x19
    bl _kcc_ir_build_assign
    b .Libsad_done

.Libsad_compound:
    // A compound literal is an lvalue; its
   materialised object's address is
    // exactly what a struct address wants (used
   by `&(T){...}`, member access
    // on a literal, and struct-copy initialisers)
   .
    mov x0, x19
    bl _kcc_ir_build_compound_lit
    b .Libsad_done

.Libsad_deref:
    // `*p` (a dereferenced pointer to a struct):
   the pointer's VALUE is the
    // struct's address, so build the pointer
   operand---not the deref, which
    // would emit a struct load.  Any other unary
   operator falls back.
    ldr x1, [x19, #AST_F0]       // unary op token
    cmp x1, #TOK_STAR
    b.ne .Libsad_fallback
    ldr x0, [x19, #AST_F1]       // pointer operand
    bl _kcc_ir_build_expr
    b .Libsad_done
.Libsad_fallback:
    mov x0, x19
    bl _kcc_ir_build_expr
    b .Libsad_done

.Libsad_ident:
    ldr x20, [x19, #AST_F0]      // name ptr
    mov x0, x20
    bl _kcc_strlen
    mov x2, x0
    adrp x0, _kcc_ir_locals@PAGE
    add  x0, x0, _kcc_ir_locals@PAGEOFF
    ldr x0, [x0]
    mov x1, x20
    bl _kcc_ht_get
    cbz x0, .Libsad_global
    bic x0, x0, #0x8000000000000000
    sub x0, x0, #1               // local: alloca
   vreg is the struct base
    b .Libsad_done

.Libsad_global:
    // File-scope struct: its address is a global
   reference.
    mov x0, #IR_GLOBAL_REF
    bl _kcc_ir_alloc_instr
    str x20, [x0, #IR_SRC1]      // symbol name
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    mov x19, x0
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x20, x0
    mov x0, x19
    bl _kcc_ir_emit
    mov x0, x20
    b .Libsad_done

.Libsad_call:
    mov x0, x19
    bl _kcc_ir_build_call        // struct-
   returning call → result buffer address
    b .Libsad_done

.Libsad_member:
    mov x0, x19
    bl _kcc_ir_build_member_addr
    b .Libsad_done

.Libsad_subscript:
    mov x0, x19
    bl _kcc_ir_build_subscript_addr
    b .Libsad_done

.Libsad_zero:
    mov x0, #0

.Libsad_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

405 · used in 30b · defines _kcc_ir_build_struct_addr 314b, 326, 393, 414, 417b · uses _kcc_ht_get 62, _kcc_ir_alloc_instr 303a, _kcc_ir_build_assign 387, _kcc_ir_build_call 393, _kcc_ir_build_compound_lit 413, _kcc_ir_build_expr 340, _kcc_ir_build_member_addr 355, _kcc_ir_build_subscript_addr 350, _kcc_ir_emit 304, _kcc_ir_locals 305c, _kcc_ir_next_vreg 305a, _kcc_type_int_s 250b

Aggregate initializer lists. A brace-enclosed initializer such as `int a[3] = {10, 20, 30}` or `struct P p = {3, 4}` is parsed into an `AST_INIT_LIST` whose elements hang off `F0` and chain through `F4`. In the all-spill model an array element and a struct field alike occupy one eight-byte slot at their ordinal position, so a single loop lowers both aggregate kinds: element i of the list is written to slot i of the destination. Two small helpers support this. `_kcc_ir_initlist_count` counts the top-level elements—used to size an implicitly-bounded array (`int a[] = {…}`), whose length the declarator leaves unspecified. `_kcc_ir_emit_indexed_store` emits the `IR_CONST`+`IR_GEP`+`IR_STORE` triple that places one value at a given index, factoring the register juggling out of the declaration builder's loop. The stride is the eight-byte slot, so these writes are read back correctly by the subscript (§29.9.4) and member-access lowerings.

When a list is shorter than the aggregate (`int a[4] = {5, 6}`), §6.7.9 requires the unmentioned elements to be initialised as if they had static storage duration—that is, to zero. The builder therefore records the total slot count when it sizes the alloca and, after the provided values are placed, stores a single reused zero constant into every remaining slot.

Designated and nested initializers (§6.7.9) are emitted by a small recursive family, so an arbitrarily deep aggregate is initialised by descending it, not by a single flat loop. After zero-filling the whole region, `_kcc_ir_fill_list`

walks a brace-delimited list against the sub-objects of the target type, keeping a running "current object" position (§6.7.9p17–18). A plain element with its own brace, or a scalar landing on a scalar sub-object, is written by `_kcc_ir_fill_one`; a designation—which may now be a *chain* such as `.p.a`, `.arr[1]`, or `[0].x`—is resolved by `_kcc_ir_resolve_designator`, which follows the `AST_F2` links one sub-object at a time (`_kcc_ir_subobj` descending both the byte offset and the type) to the leaf, and retargets the cursor to the first designator's position so plain elements that follow resume from there. A `.name` that is not a direct member is looked up through the type's *anonymous* members (§6.7.2.1p13): `_kcc_ir_field_index` skips an unnamed member (its name pointer is `NULL`, so it never reaches `_kcc_strcmp`), and the resolver, guided by `_kcc_ir_type_has_member` and `_kcc_ir_anon_containing`, steps into the unnamed member that promotes the name—adding its offset and continuing from its type—until the name is a direct field; the first member so entered becomes the recorded cursor position. A GNU array-range designator (`is_field` = 2) is handled before that chain walk: the same initializer value is filled into every element from the low index (`F0`) to the high index (`F2`) inclusive, and the cursor resumes just past the range. Both bounds—like every designator index—are constant *expression* nodes evaluated here at lowering time (so an enum constant, absent from the symbol table during parsing, resolves), not integers folded during the parse. The remaining subtlety is *brace elision* (§6.7.9p20): when a plain scalar meets an aggregate sub-object that has no brace of its own, the scalar and those after it flow into that aggregate's leaves in order. `_kcc_ir_fill` models exactly this—it treats the list as a flat stream and returns the next unconsumed element—so `int m[2][2] = {1,2,3,4}` fills the four leaves left to right and `struct {int a; int r[2]; int b;} = {1,2,3,4}` spills the middle two into the array. Because the region was zero-filled first, a sparse list such as `int a[6] = { [5] = 9 }` still leaves the gaps at zero, and a bit-field leaf keeps its read–modify–write because `_kcc_ir_subobj` publishes the resolved field for `_kcc_ir_store_elem`. An *unnamed* bit-field (`int :4`) is a layout-only member that reserves bits but takes no initializer (§6.7.9): the positional walk steps over its cursor position without consuming an element, so `{2,3,4}` against `struct {int a:4; int :4; int b:4; int c:4;}` fills `a`, `b`, `c`—not the padding member—and the anonymous slot stays zero. A *zero-size* aggregate element—an empty `struct {}` member initialised from a call—is the mirror image: it copies no bytes and has no sub-objects, so the elision loop would drop the node altogether. But the element expression may carry a side effect (`.raw = one_raw_spinlock()` increments a counter), so `fill` still *evaluates* it (emitting the call) before consuming it, rather than discarding it with the empty copy. A string literal landing on a *char-array sub-object*—a member `char p[3] = "cq"`, a row of `char m[][N]`, or a trailing *flexible* `char p[] = "wx"`—is a third special case: `fill` copies the string's bytes into the array (truncated to a fixed field's capacity, or the whole string for a flexible member, whose layout size is zero; the NUL and tail come from the surrounding zero-fill), because a scalar store would instead deposit the string's *address*, as it correctly does for a `char *` member. This string test is made *before* the zero-size test so a flexible member is copied rather than mistaken for an empty aggregate. A top-level `char s[] = "abc"` declaration takes its own byte-copy path in the variable-declaration builder above; this handles the same initialiser one level down, inside an aggregate. For a file-scope object the flexible member also grows the reserved storage (`_kcc_cg_fam_extra`, §29.1.1) so the copied tail has somewhere to live. File-scope aggregates share this exact machinery: their emitter (`_kcc_ir_emit_aggregate_init`, §29.1.1) is a thin adapter that calls `_kcc_ir_fill_list` against the global's materialised base address instead of a stack alloca, relying on zeroed BSS in place of the explicit zero-fill, so designator chains and brace elision work identically at file scope.

⟨ *IR: aggregate init helpers* 407 ⟩ ≡

```
// -- kcc_ir_initlist_count ---
// uint64_t kcc_ir_initlist_count(node *init_list)
//   x0 = AST_INIT_LIST node → number of array
   slots the list spans.  Plain
//   elements advance a running index; an `[i] =`
   designator resets it, so a
//   sparse list like { [9] = 1 } reports a length
    of 10.  The result is the
//   highest index reached plus one (used only for
    `int a[]` sizing).
.text
.globl _kcc_ir_initlist_count
.p2align 2
_kcc_ir_initlist_count:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    ldr x19, [x0, #AST_F0]      // first element (
   cursor)
    mov x20, #0                 // running index
    mov x21, #0                 // max slots
.Lilc_loop:
    cbz x19, .Lilc_done
    ldr x3, [x19, #AST_TAG]
    cmp x3, #AST_DESIGNATOR
    b.ne .Lilc_step
    ldr x3, [x19, #AST_F1]      // is_field?
    cbnz x3, .Lilc_step         // a member
   designator does not move the index
    ldr x0, [x19, #AST_F0]      // [i] = :
   evaluate the index expression
    bl _kcc_eval_const_int
    mov x20, x0                 // running index
   becomes it
.Lilc_step:
    add x20, x20, #1            // this element
   consumes slot (running-1)
    cmp x20, x21
    b.le .Lilc_nomax
    mov x21, x20
.Lilc_nomax:
    ldr x19, [x19, #AST_F4]
    b .Lilc_loop
.Lilc_done:
    mov x0, x21
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_emit_indexed_store ---
```

```
// void kcc_ir_emit_indexed_store(base_vreg,
   byte_offset, value_vreg, width)
//   x0 = base vreg, x1 = byte offset, x2 = value
   vreg, x3 = store width
//        (0 = 8-byte slot, else 1/2/4)
// Emits IR_CONST(byte_offset) + IR_GEP(base,
   offset) + IR_STORE(value,addr).
.text
.globl _kcc_ir_emit_indexed_store
.p2align 2
_kcc_ir_emit_indexed_store:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    mov x19, x0                  // base vreg
    mov x20, x1                  // byte offset
    mov x21, x2                  // value vreg
    mov x22, x3                  // store width

    // IR_CONST byte offset
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x23, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x23, #IR_TYPE]
    str x20, [x23, #IR_SRC1]     // const value =
   byte offset
    bl _kcc_ir_next_vreg
    str x0, [x23, #IR_DEST]
    mov x20, x0                  // x20 = offset
   vreg
    mov x0, x23
    bl _kcc_ir_emit

    // IR_GEP base + byte offset
    mov x0, #IR_GEP
    bl _kcc_ir_alloc_instr
    mov x23, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x23, #IR_TYPE]
    str x19, [x23, #IR_SRC1]     // base pointer
    str x20, [x23, #IR_SRC2]     // byte offset
   vreg
    bl _kcc_ir_next_vreg
    str x0, [x23, #IR_DEST]
    mov x20, x0                  // x20 = element
   address vreg
    mov x0, x23
    bl _kcc_ir_emit

    // IR_STORE value -> element address (IR_DEST
   = access-size override)
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    str x21, [x0, #IR_SRC1]      // value
    str x20, [x0, #IR_SRC2]      // address
    str x22, [x0, #IR_DEST]      // width override
    bl _kcc_ir_emit

    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_ir_elem_off_width ---
// (byte_offset, width) kcc_ir_elem_off_width(type
    *ty, uint64_t index)
//   x0 = element type (array) or aggregate type (
   struct/union), x1 = index
// Returns x0 = byte offset of the index-th
   initializer element,
//         x1 = its store-width override.  For an
   array-element type the
//   stride is uniform (index * size); for a
   struct/union the offset comes
//   from the index-th field's laid-out AST_F3 and
    the width from its type.
.text
.globl _kcc_ir_elem_off_width
.p2align 2
_kcc_ir_elem_off_width:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                  // type
    mov x20, x1                  // index
    // Clear any previously published field; an
   array element has none.
    adrp x2, _kcc_ir_member_field@PAGE
    add  x2, x2, _kcc_ir_member_field@PAGEOFF
    str xzr, [x2]
    cbz x19, .Leow_scalar
    ldr x2, [x19, #TY_KIND]
    cmp x2, #TY_STRUCT
    b.eq .Leow_struct
    cmp x2, #TY_UNION
    b.eq .Leow_struct

    // Array element: uniform stride = index *
   sizeof(element)
    mov x0, x19
    bl _kcc_type_layout          // element size
    mul x21, x20, x0             // byte offset
    mov x0, x19
    bl _kcc_ir_store_width
    mov x1, x0                   // width
    mov x0, x21                  // offset
    b .Leow_ret

.Leow_scalar:
    lsl x0, x20, #3              // fallback: 8-
   byte slots
    mov x1, #0
    b .Leow_ret

.Leow_struct:
    mov x0, x19
    bl _kcc_type_layout          // ensure per-
   field offsets
    ldr x21, [x19, #TY_F0]       // struct decl
    cbz x21, .Leow_struct_zero
    ldr x21, [x21, #AST_F1]      // first field
.Leow_walk:
    cbz x21, .Leow_struct_zero
    cbz x20, .Leow_found
    ldr x21, [x21, #AST_F4]      // next field
    sub x20, x20, #1
    b .Leow_walk
.Leow_found:
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    str x21, [x0]                // publish the
   field for a possible bit-field store
    ldr x20, [x21, #AST_F3]      // field byte
```

```
    offset
    ldr x1, [x21, #AST_F0]        // field type
    spec
    cbz x1, .Leow_found_w0
    mov x0, x1
    bl _kcc_sema_resolve_type
    bl _kcc_ir_store_width
    mov x1, x0                    // width
    mov x0, x20                   // offset
    b .Leow_ret
.Leow_found_w0:
    mov x0, x20
    mov x1, #0
    b .Leow_ret
.Leow_struct_zero:
    mov x0, #0
    mov x1, #0
.Leow_ret:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_field_index ---
// int64_t kcc_ir_field_index(type *aggregate,
    char *name)
//   x0 = struct/union type, x1 = interned member
    name
// Returns the zero-based position of the named
    member (so the positional
// offset machinery can be reused), or -1 if there
    is no such member.
.globl _kcc_ir_field_index
.p2align 2
_kcc_ir_field_index:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x1                   // name
    ldr x2, [x0, #TY_F0]          // struct decl
    cbz x2, .Lfi_notfound
    ldr x20, [x2, #AST_F1]        // first field
    mov x21, #0                   // index (callee-
    saved across strcmp)
.Lfi_loop:
    cbz x20, .Lfi_notfound
    ldr x0, [x20, #AST_F1]        // field name
    cbz x0, .Lfi_skip             // anonymous
    member: no name to match, skip
    mov x1, x19
    bl _kcc_strcmp
    cbz x0, .Lfi_found
.Lfi_skip:
    ldr x20, [x20, #AST_F4]       // next field
    add x21, x21, #1
    b .Lfi_loop
.Lfi_found:
    mov x0, x21
    b .Lfi_ret
.Lfi_notfound:
    mov x0, #-1
.Lfi_ret:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_type_has_member ---
// uint64_t kcc_ir_type_has_member(type *ty, char
    *name)
//   Whether `name` is a member of struct/union `
    ty`, directly or promoted
//   through one of its anonymous (unnamed) struct
    /union members (C17
//   6.7.2.1p13).  Recurses into each anonymous
    member's type.
.p2align 2
_kcc_ir_type_has_member:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x1                   // name
    cbz x0, .Lthm_no
    ldr x2, [x0, #TY_KIND]
    cmp x2, #TY_STRUCT
    b.eq .Lthm_ok
    cmp x2, #TY_UNION
    b.ne .Lthm_no
.Lthm_ok:
    ldr x2, [x0, #TY_F0]          // struct decl
    cbz x2, .Lthm_no
    ldr x20, [x2, #AST_F1]        // first field
.Lthm_loop:
    cbz x20, .Lthm_no
    ldr x0, [x20, #AST_F1]        // field name
    cbz x0, .Lthm_anon
    mov x1, x19
    bl _kcc_strcmp
    cbz x0, .Lthm_yes
    b .Lthm_cont
.Lthm_anon:
    ldr x0, [x20, #AST_F0]        // anonymous
    member's type node
    cbz x0, .Lthm_cont
    bl _kcc_sema_resolve_type     // x0 = resolved
    type
    mov x1, x19
    bl _kcc_ir_type_has_member    // recurse into
    the anonymous member
    cbnz x0, .Lthm_yes
.Lthm_cont:
    ldr x20, [x20, #AST_F4]       // next field
    b .Lthm_loop
.Lthm_yes:
    mov x0, #1
    b .Lthm_ret
.Lthm_no:
    mov x0, #0
.Lthm_ret:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_anon_containing ---
// int64_t kcc_ir_anon_containing(type *agg, char
    *name)
//   Position of the anonymous struct/union member
    of `agg` through which
//   `name` is reachable, or -1.  Used by the
    designator resolver to step
//   into an unnamed member on the way to a
    promoted name.
.p2align 2
_kcc_ir_anon_containing:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x1                   // name
    ldr x2, [x0, #TY_F0]          // struct decl (
    x0 = agg, still live)
    cbz x2, .Lac2_no
    ldr x20, [x2, #AST_F1]        // first field
    mov x21, #0                   // index
.Lac2_loop:
    cbz x20, .Lac2_no
    ldr x0, [x20, #AST_F1]        // field name
```

```
    cbnz x0, .Lac2_cont          // named member:
   not a descent candidate
    ldr x0, [x20, #AST_F0]       // anonymous
   member's type node
    cbz x0, .Lac2_cont
    bl _kcc_sema_resolve_type
    mov x1, x19
    bl _kcc_ir_type_has_member
    cbnz x0, .Lac2_found
.Lac2_cont:
    ldr x20, [x20, #AST_F4]      // next field
    add x21, x21, #1
    b .Lac2_loop
.Lac2_found:
    mov x0, x21
    b .Lac2_ret
.Lac2_no:
    mov x0, #-1
.Lac2_ret:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_subobj ---
// (offset, width, sub_type) kcc_ir_subobj(type *
   agg, uint64_t index)
//   x0 = the *actual* aggregate type (array,
   struct, or union); x1 = the
//        position of a sub-object within it.
//   Returns x0 = the sub-object's byte offset
   inside `agg`, x1 = its store
//   access-size code, x2 = its type descriptor.
   A struct/union position is
//   the index-th field (offset from its laid-out
   AST_F3); an array position
//   is index * sizeof(element).  Also publishes
   the resolved field in
//   _kcc_ir_member_field (cleared for an array
   element) so a following
//   _kcc_ir_store_elem can detect a bit-field
   target.  This is the typed
//   companion to _kcc_ir_elem_off_width, which
   the recursive initializer
//   needs so it can descend into a nested
   aggregate sub-object.
.text
.globl _kcc_ir_subobj
.p2align 2
_kcc_ir_subobj:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                  // aggregate type
    mov x20, x1                  // index
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    str xzr, [x0]                // default: no
   bit-field field published
    cbz x19, .Lso_void
    ldr x2, [x19, #TY_KIND]
    cmp x2, #TY_ARRAY
    b.eq .Lso_array
    cmp x2, #TY_STRUCT
    b.eq .Lso_struct
    cmp x2, #TY_UNION
    b.eq .Lso_struct
    // Scalar (or a braced scalar `int x = {5}`):
   one element of `agg` itself.
    mov x0, x19
    bl _kcc_type_layout          // size
    mul x20, x20, x0             // offset = index
    * size
    mov x0, x19
    bl _kcc_ir_store_width       // width
    mov x1, x0
    mov x0, x20                  // offset
    mov x2, x19                  // sub_type = the
    scalar itself
    b .Lso_ret
.Lso_void:
    lsl x0, x20, #3              // no type: fall
   back to 8-byte slots
    mov x1, #0
    mov x2, #0
    b .Lso_ret
.Lso_array:
    ldr x21, [x19, #TY_F0]       // element type =
    sub_type
    mov x0, x21
    bl _kcc_type_layout          // element size
    mul x20, x20, x0             // offset = index
    * element size
    mov x0, x21
    bl _kcc_ir_store_width       // width
    mov x1, x0
    mov x0, x20                  // offset
    mov x2, x21                  // sub_type
    b .Lso_ret
.Lso_struct:
    mov x0, x19
    bl _kcc_type_layout          // ensure per-
   field offsets
    ldr x21, [x19, #TY_F0]       // struct decl
    cbz x21, .Lso_zero
    ldr x21, [x21, #AST_F1]      // first field
.Lso_walk:
    cbz x21, .Lso_zero
    cbz x20, .Lso_found
    ldr x21, [x21, #AST_F4]      // next field
    sub x20, x20, #1
    b .Lso_walk
.Lso_found:
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    str x21, [x0]                // publish for a
   possible bit-field store
    ldr x1, [x21, #AST_F0]       // field type
   spec
    cbz x1, .Lso_found_w0
    mov x0, x1
    bl _kcc_sema_resolve_type    // x0 = sub_type
    mov x20, x0                  // hold sub_type
   across store_width
    bl _kcc_ir_store_width       // x0 = width (
   reads x0 = sub_type)
    mov x1, x0                   // width
    ldr x0, [x21, #AST_F3]       // field byte
   offset
    mov x2, x20                  // sub_type
    b .Lso_ret
.Lso_found_w0:
    ldr x0, [x21, #AST_F3]       // offset
    mov x1, #0                   // width
    mov x2, #0                   // sub_type
    b .Lso_ret
.Lso_zero:
    mov x0, #0
    mov x1, #0
    mov x2, #0
.Lso_ret:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_resolve_designator ---
```

```
// (offset, sub_type, first_pos, field)
//     kcc_ir_resolve_designator(type *agg,
   uint64_t base_off, node *desig)
//   x0 = the aggregate type the designation
   applies to; x1 = its base byte
//        offset; x2 = the head AST_DESIGNATOR of
   a designator chain (linked
//        through AST_F2, e.g. `.p.a`, `.arr[1]`,
   `[0].x`).
//   Walks the chain, descending one sub-object
   per designator, and returns
//   x0 = the absolute byte offset of the
   designated leaf, x1 = its type,
//   x2 = the *first* designator's position (so
   plain elements that follow in
//   the enclosing list resume from there), x3 =
   the leaf field descriptor
//   (0 for an array leaf) for bit-field-aware
   storage.
.text
.globl _kcc_ir_resolve_designator
.p2align 2
_kcc_ir_resolve_designator:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    mov x19, x0                     // cur_type
    mov x20, x1                     // cur_off (
   absolute)
    mov x21, x2                     // current
   designator
    mov x22, #-1                    // first_pos
   sentinel
    mov x23, #0                     // leaf field
.Lrd_loop:
    cbz x21, .Lrd_done
    ldr x0, [x21, #AST_F1]          // is_field?
    cbz x0, .Lrd_index
    mov x0, x19                     // cur_type
    ldr x1, [x21, #AST_F0]          // member name
    bl _kcc_ir_field_index          // x0 = position
   (or -1)
    mov x24, x0                     // position
    // Descend through anonymous members while the
    name is not a direct field
    // of cur_type: step into the unnamed member
   that promotes it, accumulate
    // its offset, and re-resolve there (C17
   6.7.2.1p13).
.Lrd_desc:
    cmn x24, #1                     // position == -1
    ?
    b.ne .Lrd_sub                   // resolved to a
   direct field: proceed
    mov x0, x19
    ldr x1, [x21, #AST_F0]          // name
    bl _kcc_ir_anon_containing      // x0 = anon
   member index, or -1
    cmn x0, #1
    b.eq .Lrd_sub                   // genuinely
   absent: fall through with pos=-1
    mov x25, x0                     // outer
   anonymous member index
    mov x0, x19
    mov x1, x25
    bl _kcc_ir_subobj               // x0=off, x2=
   sub_type
    add x20, x20, x0                // cur_off +=
   anonymous member offset
    mov x19, x2                     // cur_type =
   anonymous member type
    cmn x22, #1                     // first_pos
   still -1?
    b.ne .Lrd_desc2
    mov x22, x25                    // record the
   outer position for cursor resume
.Lrd_desc2:
    mov x0, x19
    ldr x1, [x21, #AST_F0]          // name
    bl _kcc_ir_field_index
    mov x24, x0
    b .Lrd_desc
.Lrd_index:
    ldr x0, [x21, #AST_F0]          // [i] : evaluate
    the index expression
    bl _kcc_eval_const_int
    mov x24, x0                     // position = i
.Lrd_sub:
    mov x0, x19
    mov x1, x24
    bl _kcc_ir_subobj               // x0=off, x1=
   width, x2=sub_type; publishes field
    add x20, x20, x0                // cur_off += off
    mov x19, x2                     // descend:
   cur_type = sub_type
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    ldr x23, [x0]                   // remember leaf
   field (last one wins)
    cmn x22, #1                     // first_pos
   still -1?
    b.ne .Lrd_next
    mov x22, x24                    // record the
   first position
.Lrd_next:
    ldr x21, [x21, #AST_F2]         // next
   designator in the chain
    b .Lrd_loop
.Lrd_done:
    mov x0, x20                     // absolute
   offset
    mov x1, x19                     // leaf type
    mov x2, x22                     // first position
    mov x3, x23                     // leaf field
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret

// -- kcc_ir_agg_count ---
// uint64_t kcc_ir_agg_count(type *ty)
//   Number of sub-objects the brace-elision
   consumer must fill for an
//   aggregate: an array's element count, a struct
   's field count, 1 for a
//   union (elision fills only the first member)
   or a scalar.
.text
.globl _kcc_ir_agg_count
.p2align 2
_kcc_ir_agg_count:
    cbz x0, .Lac_one
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.eq .Lac_array
    cmp x1, #TY_STRUCT
    b.eq .Lac_struct
.Lac_one:
    mov x0, #1
    ret
.Lac_array:
    ldr x0, [x0, #TY_F1]            // element count
```

```
    ret
.Lac_struct:
    ldr x1, [x0, #TY_F0]          // struct decl
    cbz x1, .Lac_one
    ldr x1, [x1, #AST_F1]         // first field
    mov x0, #0
.Lac_sc:
    cbz x1, .Lac_scdone
    add x0, x0, #1
    ldr x1, [x1, #AST_F4]         // next field
    b .Lac_sc
.Lac_scdone:
    ret

// -- kcc_ir_fill_one ---
// void kcc_ir_fill_one(type *ty, base_alloca,
   uint64_t off, node, field)
//   Initialize the single object of type `ty` at
   `off` from ONE initializer
//   `node` (no stream elision).  A braced `node`
   fills `ty` from its own
//   sub-list; a scalar `node` whose target is an
   aggregate (a designator
//   naming an aggregate, `.m = 1`) elides from
   that point via _kcc_ir_fill;
//   an ordinary scalar leaf is stored at `ty`'s
   width, honouring `field`
//   for a bit-field.  This is the one place a
   value is actually emitted.
.text
.globl _kcc_ir_fill_one
.p2align 2
_kcc_ir_fill_one:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                   // ty
    mov x20, x1                   // base_alloca
    mov x21, x2                   // off
    mov x22, x3                   // node
    mov x23, x4                   // field
    cbz x22, .Lfo_done
    ldr x0, [x22, #AST_TAG]
    cmp x0, #AST_INIT_LIST
    b.eq .Lfo_list
    // Scalar node.  If its target type is an
   aggregate, elide into it.
    cbz x19, .Lfo_scalar
    ldr x0, [x19, #TY_KIND]
    cmp x0, #TY_STRUCT
    b.eq .Lfo_elide
    cmp x0, #TY_UNION
    b.eq .Lfo_elide
    cmp x0, #TY_ARRAY
    b.eq .Lfo_elide
.Lfo_scalar:
    mov x0, x22
    bl _kcc_ir_build_expr
    mov x24, x0                   // value vreg
    // Convert the initializer value to the target
    element type before storing.
    // A brace element carries its own type (\
   stdref{6.7.9}: each is assigned to
    // the corresponding object), so `double a[] =
    {1, 2}` must deposit 1.0/2.0,
    // not the raw integer bits of 1 and 2 --- an
   integer store into a double
    // slot.  build_expr yields the element in its
    own type; _kcc_ir_convert
    // bridges int<->float and integer widths, and
    is a no-op when they match.
    ldr x1, [x22, #AST_F3]        // from = element
    expression's resolved type
    cbz x1, .Lfo_scalar_store
    cbz x19, .Lfo_scalar_store    // no target type
    (bare designator): store as-is
    mov x0, x24                   // value vreg
    mov x2, x19                   // to = target
   element type
    bl _kcc_ir_convert
    mov x24, x0                   // converted
   value
.Lfo_scalar_store:
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    str x23, [x0]                 // republish
   field (build_expr may clear it)
    mov x0, x19
    bl _kcc_ir_store_width
    mov x3, x0                    // width
    mov x0, x20                   // base
    mov x1, x21                   // offset
    mov x2, x24                   // value
    bl _kcc_ir_store_elem
    b .Lfo_done
.Lfo_list:
    ldr x3, [x22, #AST_F0]        // sub-list head
    mov x0, x19
    mov x1, x20
    mov x2, x21
    bl _kcc_ir_fill_list
    b .Lfo_done
.Lfo_elide:
    mov x0, x19
    mov x1, x20
    mov x2, x21
    mov x3, x22
    bl _kcc_ir_fill               // consume this
   one node into the aggregate
.Lfo_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

407 · defined at 407, 413, 414 · used in 30b · uses _kcc_eval_const_int 426, _kcc_ir_agg_count 414, _kcc_ir_alloc_instr 303a, _kcc_ir_anon_containing 414, _kcc_ir_build_expr 340, _kcc_ir_convert 343, _kcc_ir_elem_off_width 414, _kcc_ir_emit 304, _kcc_ir_emit_indexed_store 414, _kcc_ir_field_index 414, _kcc_ir_fill 414, _kcc_ir_fill_list 414, _kcc_ir_fill_one 414, _kcc_ir_initlist_count 414, _kcc_ir_next_vreg 305a, _kcc_ir_resolve_designator 414, _kcc_ir_store_width 349b, _kcc_ir_subobj 414, _kcc_ir_type_has_member 414, _kcc_sema_resolve_type 259b, _kcc_type_int_s 250b, _kcc_type_layout 267, 268

Compound literals. A compound literal `(T){…}` (§6.5.2.5) denotes an unnamed object of type `T` initialised by the brace list. Inside a function it has automatic storage, so `_kcc_ir_build_compound_lit` materialises exactly what a local aggregate declaration would: an `alloca` of the right size, a slot-by-slot zero-fill, then `_kcc_ir_fill_list` to place the initializer (designators and brace elision included). A compound literal is an *lvalue*, so the builder returns the vreg holding the object's *address*; a caller wanting the value (a scalar literal, or a member ac-

cess on it) loads through that address, while aggregate uses — the common case, `&(struct S){1,2}` or passing one by value — take the address directly. A file-scope compound literal is a different animal (static storage), handled in the global-initializer path.

⟨ *IR: aggregate init helpers* 413 ⟩ + ≡

```
// -- kcc_ir_build_compound_lit ---
// uint64_t kcc_ir_build_compound_lit(node *cl)
//   x0 = AST_COMPOUND_LIT (F0 = type, F1 = brace
   init list)
// Returns the vreg holding the address of the
   materialised automatic object.
.text
.globl _kcc_ir_build_compound_lit
.p2align 2
_kcc_ir_build_compound_lit:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                     // compound-
   literal node
    ldr x0, [x19, #AST_F0]          // type node
    bl _kcc_sema_resolve_type
    mov x21, x0                     // object type
    cbz x21, .Lbcl_zero

    // Slot count = ceil(TY_SIZE / 8), at least
   one.
    mov x0, x21
    bl _kcc_type_layout             // byte size
    add x0, x0, #7
    lsr x24, x0, #3
    cbnz x24, .Lbcl_alloca
    mov x24, #1
.Lbcl_alloca:
    mov x0, #IR_ALLOCA
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_TYPE]
    str x24, [x0, #IR_SRC1]         // slot count
    mov x22, x0
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x23, x0                     // x23 = alloca
   vreg (the object's address)
    mov x0, x22
    bl _kcc_ir_emit

    // Zero the whole object, so a sparse
   initializer leaves zeros (§6.7.9).
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str xzr, [x22, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x20, x0                     // zero vreg
    mov x0, x22
    bl _kcc_ir_emit
    mov x22, #0                     // slot index
.Lbcl_zf:
    cmp x22, x24
    b.ge .Lbcl_fill
    mov x0, x23                     // base
    lsl x1, x22, #3                 // byte offset =
   slot * 8
    mov x2, x20                     // zero vreg
    mov x3, #0                      // 8-byte slot
    bl _kcc_ir_emit_indexed_store
    add x22, x22, #1
    b .Lbcl_zf

.Lbcl_fill:
    ldr x0, [x19, #AST_F1]          // brace init
   list
    cbz x0, .Lbcl_ret               // no list → the
   zeroed object stands
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_INIT_LIST
    b.ne .Lbcl_scalar
    ldr x3, [x0, #AST_F0]           // list head
    mov x0, x21                     // object type
    mov x1, x23                     // base alloca
    mov x2, #0                      // base offset
    bl _kcc_ir_fill_list
    b .Lbcl_ret
.Lbcl_scalar:
    // A non-brace initializer (a scalar compound
   literal whose value the
    // parser did not wrap in a list): store it
   into the single object.
    mov x3, x0                      // init node
    mov x0, x21                     // type
    mov x1, x23                     // base alloca
    mov x2, #0                      // offset
    mov x4, #0                      // no bit-field
    bl _kcc_ir_fill_one
.Lbcl_ret:
    mov x0, x23                     // address of the
    object
    b .Lbcl_done
.Lbcl_zero:
    mov x0, #0
.Lbcl_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

413 · defined at 407, 413, 414 · used in 30b · defines _kcc_ir_build_compound_lit 340, 378b, 405 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_emit 304, _kcc_ir_emit_indexed_store 414, _kcc_ir_fill_list 414, _kcc_ir_fill_one 414, _kcc_ir_next_vreg 305a, _kcc_sema_resolve_type 259b, _kcc_type_int_s 250b, _kcc_type_layout 267, 268

⟨ *IR: aggregate init helpers* 414 ⟩ + ≡

```
// -- kcc_ir_fill ---
// node *kcc_ir_fill(type *ty, base_alloca,
   uint64_t off, node *elem)
//   Brace-elision stream consumer.  Fills the
   object of type `ty` at `off`
//   by drawing elements from the flat list stream
    starting at `elem`, and
//   returns the next unconsumed element.  A
   scalar `ty` consumes exactly
//   one element; an aggregate with no brace of
   its own recurses across its
//   sub-objects, so `int m[2][2] = {1,2,3,4}`
   flows the four scalars into
//   the four leaves in order (\stdref{6.7.9p20}).
.text
```

```
.globl _kcc_ir_fill
.p2align 2
_kcc_ir_fill:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]
    mov x19, x0                 // ty
    mov x20, x1                 // base_alloca
    mov x21, x2                 // off
    mov x22, x3                 // elem
    cbz x22, .Lfl_ret
    cbz x19, .Lfl_scalar
    ldr x0, [x19, #TY_KIND]
    cmp x0, #TY_STRUCT
    b.eq .Lfl_agg
    cmp x0, #TY_UNION
    b.eq .Lfl_agg
    cmp x0, #TY_ARRAY
    b.eq .Lfl_agg
.Lfl_scalar:
    mov x0, x19
    mov x1, x20
    mov x2, x21
    mov x3, x22
    mov x4, #0                  // elision has no
    bit-field context
    bl _kcc_ir_fill_one
    ldr x22, [x22, #AST_F4]     // consumed one
   element
    b .Lfl_ret
.Lfl_agg:
    // A struct/union VALUE element initializes
   this aggregate sub-object by a
    // whole-object COPY, not by brace elision: `
   struct A a = {2, b}` copies all
    // of the struct `b` into `a.b`.  Elision
   would instead splice b's fields one
    // scalar at a time and lose everything past
   the first.  Sema types init-list
    // elements, so the element's AST_F3 is its
   struct/union type; a matching
    // aggregate sub-object gets the copy.
    ldr x0, [x22, #AST_TAG]
    cmp x0, #AST_INIT_LIST
    b.eq .Lfl_agg_count         // its own braces
    → recurse/elide as before
    // A string literal initialising a char-array
   sub-object --- `char p[3] =
    // "cq"` as a struct member, a row of `char m
   [][N]`, or a trailing FLEXIBLE
    // `char p[] = "wx"` --- copies the string's
   BYTES into the array.  A plain
    // scalar store would leave the string's
   address (right for `char *`, wrong
    // for an array).  This precedes the zero-size
    test below so a flexible (or
    // zero-length) member, whose layout size is
   0, still copies the whole
    // string instead of being mistaken for an
   empty aggregate.
    cmp x0, #AST_STR_LIT
    b.ne .Lfl_agg_nonstr
    ldr x1, [x19, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.ne .Lfl_agg_nonstr
    mov x0, x19
    bl _kcc_type_layout         // capacity (0
   for a flexible member)
    mov x25, x0
    ldr x23, [x22, #AST_F0]     // string bytes
    ldr x24, [x22, #AST_F1]     // string length
   (no NUL)
    cbz x25, .Lfl_str_go        // flexible
   member: copy the whole string
    cmp x24, x25
    csel x24, x24, x25, ls      // len = min(
   length, capacity)
.Lfl_str_go:
    mov x19, #0                 // byte index (
   sub-object type no longer needed)
.Lfl_str_loop:
    cmp x19, x24
    b.ge .Lfl_str_done
    ldrb w0, [x23, x19]
    bl _kcc_ir_emit_const
    mov x2, x0                  // value
    mov x0, x20                 // base
    add x1, x21, x19            // byte offset =
   sub-object offset + i
    mov x3, #1                  // one byte
    bl _kcc_ir_emit_indexed_store
    add x19, x19, #1
    b .Lfl_str_loop
.Lfl_str_done:
    ldr x22, [x22, #AST_F4]     // consumed one
   element
    b .Lfl_ret
.Lfl_agg_nonstr:
    // A zero-size aggregate (an empty struct/
   union) copies no bytes and has no
    // sub-objects, so the ordinary elision loop
   would drop the element node
    // entirely --- along with any side effect its
    expression carries, such as
    // `.raw = one_raw_spinlock()`.  Evaluate the
   element for its effects, then
    // consume it.
    mov x0, x19
    bl _kcc_type_layout         // sub-object
   byte size
    cbnz x0, .Lfl_agg_typed
    mov x0, x22
    bl _kcc_ir_build_struct_addr // evaluate (
   emits a struct-returning call)
    ldr x22, [x22, #AST_F4]     // consumed one
   element
    b .Lfl_ret
.Lfl_agg_typed:
    ldr x0, [x22, #AST_F3]      // element's
   resolved type
    cbz x0, .Lfl_agg_count
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_STRUCT
    b.eq .Lfl_agg_copy
    cmp x1, #TY_UNION
    b.ne .Lfl_agg_count
.Lfl_agg_copy:
    mov x0, x20                 // base alloca
    mov x1, x21                 // sub-object
   byte offset
    bl _kcc_ir_emit_gep_const   // x0 =
   destination address vreg
    mov x23, x0                 // dst
    mov x0, x22                 // element
   expression (ident/member/call…/)
    bl _kcc_ir_build_struct_addr // x0 = source
   struct base address vreg
    mov x24, x0                 // src
    mov x0, x19                 // sub-object
   type
    bl _kcc_type_layout         // x0 = exact
   byte size
    mov x2, x0
```

```
    mov x0, x24                 // src
    mov x1, x23                 // dst
    bl _kcc_ir_emit_struct_copy
    ldr x22, [x22, #AST_F4]     // consumed one
   element
    b .Lfl_ret
.Lfl_agg_count:
    mov x0, x19
    bl _kcc_ir_agg_count
    mov x23, x0                 // number of sub-
   objects
    mov x24, #0                 // cursor
.Lfl_agg_loop:
    cbz x22, .Lfl_ret           // stream
   exhausted
    cmp x24, x23
    b.ge .Lfl_ret               // all positions
   filled
    mov x0, x19
    mov x1, x24
    bl _kcc_ir_subobj           // x0=off, x2=
   sub_type
    mov x25, x2                 // sub_type
    add x2, x21, x0             // absolute
   offset
    mov x0, x25                 // sub_type
    mov x1, x20                 // base
    mov x3, x22                 // elem
    bl _kcc_ir_fill             // consume;
   returns next element
    mov x22, x0
    add x24, x24, #1
    b .Lfl_agg_loop
.Lfl_ret:
    mov x0, x22                 // next
   unconsumed element (may be 0)
    ldr x25, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret

// -- kcc_ir_fill_list ---
// void kcc_ir_fill_list(type *ty, base_alloca,
   uint64_t base_off, node *head)
//   Fill the aggregate `ty` at `base_off` from a
   brace-delimited element
//   list `head..`.  Walks a running cursor over `
   ty`'s sub-objects; a
//   designator chain retargets the cursor (and
   the value goes to the named
//   leaf), a plain element with an explicit brace
    or scalar-into-scalar
//   fills one sub-object, and a plain scalar
   whose sub-object is itself an
//   aggregate spills into it by brace elision (
   via _kcc_ir_fill).  Gaps left
//   by a sparse list keep the caller's zero-fill.
.text
.globl _kcc_ir_fill_list
.p2align 2
_kcc_ir_fill_list:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    mov x19, x0                 // ty
    mov x20, x1                 // base_alloca
    mov x21, x2                 // base_off
    mov x22, x3                 // current
   element
    mov x23, #0                 // cursor
   position
.Lfll_loop:
    cbz x22, .Lfll_done
    ldr x0, [x22, #AST_TAG]
    cmp x0, #AST_DESIGNATOR
    b.ne .Lfll_plain
    ldr x0, [x22, #AST_F1]      // is_field? 2 =
   GNU array range
    cmp x0, #2
    b.eq .Lfll_range
    // Designator chain: retarget the cursor and
   fill the named leaf.
    mov x0, x19
    mov x1, x21
    mov x2, x22
    bl _kcc_ir_resolve_designator  // x0=abs, x1=
   sub_type, x2=first_pos, x3=field
    mov x23, x2                 // cursor = first
    position
    mov x24, x0                 // absolute
   offset
    mov x25, x1                 // sub_type
    mov x26, x3                 // leaf field
    ldr x3, [x22, #AST_F3]      // value node (on
    the head)
    mov x0, x25
    mov x1, x20
    mov x2, x24
    mov x4, x26
    bl _kcc_ir_fill_one
    add x23, x23, #1
    ldr x22, [x22, #AST_F4]
    b .Lfll_loop
.Lfll_range:
    // GNU array range `[lo ... hi] = value`: fill
    every element in [lo, hi]
    // with the same initializer.  The bounds are
   now index EXPRESSIONS, so
    // evaluate them once; the high bound is
   stashed on the frame at [sp,#80]
    // (call-clobbered registers cannot hold it)
   and reloaded each iteration.
    // x24 = running pos (preserved by fill_one
   and subobj).  Evaluate hi first
    // so the last call is lo, leaving x24
   undisturbed by any bl.
    ldr x0, [x22, #AST_F2]      // hi expression
    bl _kcc_eval_const_int
    str x0, [sp, #80]           // stash
   evaluated hi
    ldr x0, [x22, #AST_F0]      // lo expression
    bl _kcc_eval_const_int
    mov x24, x0                 // pos = lo
.Lfll_range_loop:
    ldr x25, [sp, #80]          // hi (reload:
   not call-preserved)
    cmp x24, x25
    b.hi .Lfll_range_next
    mov x0, x19                 // ty
    mov x1, x24                 // pos
    bl _kcc_ir_subobj           // x0=off, x2=
   sub_type; publishes _kcc_ir_member_field
    add x5, x21, x0             // x5 = absolute
   offset
    mov x6, x2                  // x6 = sub_type
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    ldr x4, [x0]                // x4 = leaf
   field
    ldr x3, [x22, #AST_F3]      // x3 = value
   node
```

```
    mov x0, x6                  // x0 = sub_type
    mov x1, x20                 // x1 = base
    mov x2, x5                  // x2 = absolute
   offset
    bl _kcc_ir_fill_one
    add x24, x24, #1
    b .Lfll_range_loop
.Lfll_range_next:
    ldr x25, [sp, #80]          // hi
    add x25, x25, #1
    mov x23, x25                // cursor resumes
    after the range
    ldr x22, [x22, #AST_F4]
    b .Lfll_loop
.Lfll_plain:
    mov x0, x19
    mov x1, x23                 // sub-object at
   the cursor
    bl _kcc_ir_subobj           // x0=off, x2=
   sub_type; publishes field
    mov x25, x2                 // sub_type
    add x24, x21, x0            // absolute
   offset
    adrp x0, _kcc_ir_member_field@PAGE
    add  x0, x0, _kcc_ir_member_field@PAGEOFF
    ldr x26, [x0]               // leaf field for
    this position
    // An unnamed bit-field (no name, non-zero
   width) reserves layout bits but
    // takes NO initializer (C17 6.7.9): step over
    its position without
    // consuming the current element, so `{2,3,4}`
    fills a, b, c -- not the
    // anonymous padding member in between.
    cbz x26, .Lfll_notanon
    ldr x0, [x26, #AST_F1]      // field name
    cbnz x0, .Lfll_notanon
    ldr x0, [x26, #AST_F2]      // packed bit-
   field width (0 = not a bit-field)
    cbz x0, .Lfll_notanon
    add x23, x23, #1            // advance the
   cursor, keep the element
    b .Lfll_loop
.Lfll_notanon:
    cbz x25, .Lfll_skip         // ran past the
   aggregate: ignore the element
    // Elision only when the sub-object is an
   aggregate AND the element is
    // not its own brace; otherwise one element
   fills this sub-object.
    ldr x0, [x22, #AST_TAG]
    cmp x0, #AST_INIT_LIST
    b.eq .Lfll_one
    ldr x0, [x25, #TY_KIND]
    cmp x0, #TY_STRUCT
    b.eq .Lfll_elide
    cmp x0, #TY_UNION
    b.eq .Lfll_elide
    cmp x0, #TY_ARRAY
    b.eq .Lfll_elide
.Lfll_one:
    mov x0, x25
    mov x1, x20
    mov x2, x24
    mov x3, x22
    mov x4, x26
    bl _kcc_ir_fill_one
    ldr x22, [x22, #AST_F4]
    b .Lfll_adv
.Lfll_elide:
    mov x0, x25
    mov x1, x20
    mov x2, x24
    mov x3, x22
    bl _kcc_ir_fill             // spill the
   stream into this aggregate
    mov x22, x0                 // next
   unconsumed element
    b .Lfll_adv
.Lfll_skip:
    ldr x22, [x22, #AST_F4]
.Lfll_adv:
    add x23, x23, #1
    b .Lfll_loop
.Lfll_done:
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

414 · defined at 407, 413, 414 · used in 30b · defines _kcc_ir_agg_count 407, _kcc_ir_anon_containing 407, _kcc_ir_elem_off_width 359, 407, _kcc_ir_emit_indexed_store 320, 359, 407, 413, 417b, _kcc_ir_field_index 407, _kcc_ir_fill 407, _kcc_ir_fill_list 314a, 407, 413, 417b, _kcc_ir_fill_one 407, 413, _kcc_ir_initlist_count 277, 407, 417b, _kcc_ir_resolve_designator 407, _kcc_ir_subobj 407, _kcc_ir_type_has_member 407 · uses _kcc_eval_const_int 426, _kcc_ir_build_struct_addr 405, _kcc_ir_emit_struct_copy 404, _kcc_type_layout 267, 268

29.11.1. Static Locals

A `static` local has block scope but static storage duration: it is initialised once, before the program starts, and retains its value across calls. The all-spill model already has a natural home for such storage—the BSS globals every ordinary file-scope variable uses—so the builder *hoists* each static local to a synthetic global rather than giving it a stack alloca.

Two functions may each declare their own `static int i`, so the hoisted global needs a name that is unique yet reproducible from the declaration alone. `_kcc_ir_static_name` supplies it by formatting the declaration node's own address as `sl_` meta16 hex digits. Because the name is a pure function of the node pointer, the two passes that must agree on it—the declaration builder, which records the mapping and suppresses the alloca, and `_kcc_ir_emit_global_inits`, which emits the one-time initialiser at program start—derive the identical string independently, with no shared table. The symbol itself is created lazily by the code generator when it lowers the first `IR_GLOBAL_REF` that names it.

The declaration builder records *two* things about a static local: the name-to-mangled mapping in `_kcc_ir_statics`, and, in the parallel `_kcc_ir_static_types`, the declared type. The type is needed because an *array* static must decay to its address (like a file-scope global array) rather than be loaded as a scalar, and the block scope that would tell the symbol table so is popped once semantic analysis finishes. The identifier-decay and subscript-address builders consult this table to route a static array through the address path. The code generator's static pre-

registration then reserves the array's real byte size, not the scalar eight bytes the lazy `IR_GLOBAL_REF` fallback would give, so a static array larger than a slot has correct storage. (A brace initialiser on a static *aggregate* is still stored as a scalar and remains a gap.)

⟨ *IR: static local helper* 417a ⟩ ≡

```
// -- kcc_ir_static_name ---
// char *kcc_ir_static_name(node *decl)
//   x0 = declaration node → arena string "sl_" +
   16 hex digits of the node
//        pointer, the unique symbol name for this
    static local's global.
.text
.globl _kcc_ir_static_name
.p2align 2
_kcc_ir_static_name:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    mov x19, x0                    // node pointer =
    value to encode

    mov x0, #24
    bl _kcc_arena_alloc
    mov x20, x0

    mov w1, #'s'
    strb w1, [x20]
    mov w1, #'l'
    strb w1, [x20, #1]
    mov w1, #'_'
    strb w1, [x20, #2]

    add x2, x20, #3                // write cursor
    mov x3, #60                    // shift, most-
   significant nibble first
.Lisn_loop:
    lsr x4, x19, x3
    and x4, x4, #0xf
    cmp x4, #10
    b.lt .Lisn_dec
    add x4, x4, #87              // 'a' - 10
    b .Lisn_put
.Lisn_dec:
    add x4, x4, #48              // '0'
.Lisn_put:
    strb w4, [x2], #1
    subs x3, x3, #4
    b.ge .Lisn_loop

    strb wzr, [x2]                 // null
   terminator

    mov x0, x20
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

417a · used in 30b · defines _kcc_ir_static_name 312, 417b, 465, 563 · uses _kcc_arena_alloc 58a

The declaration builder keys on the storage class. A `static` local hoists to a global as just described; a block-scope `extern` declares no storage at all—it re-exposes a file-scope object—so it emits no alloca and instead binds its name to the "resolve as a global" marker in the (journalled) locals map, shadowing any enclosing local of the same name for the block. Everything else is an ordinary automatic with a stack alloca.

⟨ *IR: build var decl* 417b ⟩ ≡

```
// -- kcc_ir_build_var_decl ---
// void kcc_ir_build_var_decl(node *decl)
//   x0 = AST_VAR_DECL (F0=type, F1=name, F2=init)
.text
.globl _kcc_ir_build_var_decl
.p2align 2
_kcc_ir_build_var_decl:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0                     // x19 = decl
   node (KEEP)
    ldr x20, [x19, #AST_F1]      // x20 = name
    cbz x20, .Libvd_done

    // static local? hoist to a global instead of
   an alloca
    ldr x0, [x19, #AST_F3]       // storage class
   (3 = static)
    cmp x0, #3
    b.eq .Libvd_static
    cmp x0, #2                      // extern: a
   block-scope reference to a
    b.eq .Libvd_extern              // file-scope
   object, not a new local

    // Resolve type
    ldr x0, [x19, #AST_F0]
    cbz x0, .Libvd_default_type
    bl _kcc_sema_resolve_type
    mov x21, x0
    cbnz x21, .Libvd_alloca
.Libvd_default_type:
    adrp x0, _kcc_type_int_s@PAGE
    add  x0, x0, _kcc_type_int_s@PAGEOFF
    ldr x21, [x0]

.Libvd_alloca:
    // Pointer-variable flag from the declared
   type (before x21 is reassigned)
    mov x2, #0
    ldr x0, [x21, #TY_KIND]
    cmp x0, #TY_PTR
    b.ne .Libvd_ptrflag_done
    mov x2, #1
.Libvd_ptrflag_done:
    str x2, [sp, #64]

    // Determine alloca slot count: arrays need N
   slots, structs need field count, scalars need 1
    mov x24, #1                     // default: 1
   slot
    ldr x0, [x21, #TY_KIND]
    cmp x0, #TY_ARRAY
    b.eq .Libvd_array
    cmp x0, #TY_STRUCT
    b.eq .Libvd_struct
    cmp x0, #TY_UNION
    b.eq .Libvd_struct              // a union is
   sized like a struct: ceil(TY_SIZE/8)
    b .Libvd_emit_alloca

.Libvd_array:
    ldr x24, [x21, #TY_F1]       // array length (0
    if unspecified: int a[])
    ldr x22, [x21, #TY_F0]       // element type
```

```
    cbnz x24, .Libvd_array_scale
    // Implicit size - infer length from the
   initializer list element count
    ldr x0, [x19, #AST_F2]
    cbz x0, .Libvd_array_scale
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_INIT_LIST
    b.ne .Libvd_array_scale
    bl _kcc_ir_initlist_count
    mov x24, x0
.Libvd_array_scale:
    mov x21, x22                // x21 = element
   type (for the alloca's IR_TYPE)
    cbz x24, .Libvd_emit_alloca
    // Byte-accurate: total bytes = length *
   element size, rounded up to
    // whole 8-byte slots.  int a[4] is 16 bytes
   → two slots (not four).
    mov x0, x22                 // element type
    bl _kcc_type_layout         // element size in
    bytes
    mul x24, x24, x0            // total bytes
    add x24, x24, #7
    lsr x24, x24, #3            // ceil(bytes / 8)
    b .Libvd_emit_alloca

.Libvd_struct:
    // Slot count = ceil(TY_SIZE / 8) over the
   packed struct/union size.  A union
    // is as large as its widest member, so a 16-
   byte `union{int i,j[4];}` needs
    // two slots; without this it fell to the 1-
   slot default and left the tail
    // past the first member unallocated and un-
   zeroed (garbage `t.j[2..3]`).
    mov x0, x21
    bl _kcc_ir_type_slots
    mov x24, x0

.Libvd_emit_alloca:
    str x24, [sp, #72]          // total slot
   count (for init-list zero-fill)

    // Emit alloca - SRC1 = slot count
    mov x0, #IR_ALLOCA
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_TYPE]
    str x24, [x0, #IR_SRC1]     // slot count (1
   for scalar, N for array)
    mov x22, x0                  // x22 = alloca
   instr
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x23, x0                  // x23 = alloca
   vreg number
    mov x0, x22
    // Hoist the storage reservation to the entry
   block: a local's address must
    // dominate every use, but a switch case or
   goto label after this
    // declaration (yet inside the same block) can
    be entered directly, past
    // this point.  The initialising stores below
   stay where they are.
    bl _kcc_ir_emit_entry

    // Record in locals: name -> alloca vreg (bit
   63 tags pointer variables)
    mov x0, x20
    bl _kcc_strlen
    mov x2, x0                   // keylen
    mov x1, x2                   // (locals_put
   takes keylen in x1)
    add x3, x23, #1              // value = alloca
    vreg + 1 (0 reserved for not-found)
    ldr x4, [sp, #64]            // pointer-
   variable flag
    cbz x4, .Libvd_put
    orr x3, x3, #0x8000000000000000
.Libvd_put:
    // Journalled put so a later block exit
   restores any shadowed outer binding.
    mov x2, x3                   // value
    mov x0, x20                  // name
    bl _kcc_ir_locals_put

    // Check for initializer (x19 = decl node,
   still valid)
    ldr x0, [x19, #AST_F2]       // init expr
    cbz x0, .Libvd_done

    // Brace-enclosed aggregate initializer (
   arrays and flat structs)
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_INIT_LIST
    b.eq .Libvd_initlist
    cmp x1, #AST_STR_LIT
    b.eq .Libvd_string_init

    // Struct/union types initialised from another
    aggregate value copy the
    // whole object; a scalar initializer (or an
   aggregate whose type failed to
    // resolve) takes the single-store path.
    ldr x1, [x21, #TY_KIND]
    cmp x1, #TY_STRUCT
    b.eq .Libvd_struct_init
    cmp x1, #TY_UNION
    b.eq .Libvd_struct_init
    b .Libvd_scalar_init

.Libvd_string_init:
    // char s[] = "abc" copies the bytes; char *s
   = "abc" (pointer) instead
    // holds the string's address, so only an
   array target takes the copy path.
    ldr x22, [x0, #AST_F0]       // str_ptr
    ldr x24, [x0, #AST_F1]       // str_len
    ldr x0, [x19, #AST_F0]       // declared type
   spec
    bl _kcc_sema_resolve_type
    cbz x0, .Libvd_str_scalar
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.ne .Libvd_str_scalar
    ldr x21, [x0, #TY_F1]        // array length (
   char → bytes)
    cbnz x21, .Libvd_str_call
    add x21, x24, #1
.Libvd_str_call:
    mov x0, x23                  // base alloca
    mov x1, x22                  // str_ptr
    mov x2, x24                  // str_len
    mov x3, x21                  // array_len
    mov x4, #1                   // zero-fill (
   local)
    bl _kcc_ir_emit_string_init
    b .Libvd_done
.Libvd_str_scalar:
    ldr x0, [x19, #AST_F2]       // reload init
   node (clobbered by resolve)

.Libvd_scalar_init:
    // Build initializer expression (scalar/
   pointer)
    ldr x24, [x0, #AST_F3]       // init expr's
```

```
    sema type (stash before building)
    str x24, [sp, #72]
    bl _kcc_ir_build_expr
    mov x24, x0                     // x24 = init
value vreg

    // Convert across the int/float boundary if
the declared type differs.
    ldr x1, [sp, #72]               // from = init
type
    mov x2, x21                     // to = declared
type
    bl _kcc_ir_convert
    mov x24, x0

    // Emit store: value -> alloca.  Set the
access-size from the declared type
    // so the value is written at its true width;
in particular a `float`, whose
    // 4-byte single representation differs from
the double working format, is
    // narrowed and stored as 4 bytes (store_width
→ code 8) rather than as an
    // 8-byte double---otherwise `&f`/`sizeof`-4
copies of the local saw double
    // bytes (Lua's `string.pack("<f", …)` wrote
zeros).
    mov x0, #IR_STORE
    bl _kcc_ir_alloc_instr
    mov x22, x0                     // x22 = store
instr
    str x21, [x22, #IR_TYPE]        // type
    str x24, [x22, #IR_SRC1]        // value vreg
    str x23, [x22, #IR_SRC2]        // alloca vreg
    mov x0, x21                     // declared type
→ access-size code
    bl _kcc_ir_store_width
    str x0, [x22, #IR_DEST]         // access-size
override
    mov x0, x22
    bl _kcc_ir_emit
    b .Libvd_done

.Libvd_struct_init:
    // x0 = init expr, x21 = struct/union type,
x23 = dst alloca vreg.
    // A struct/union declared with an initializer
that is itself a
    // struct/union value---`S a = b;`, `S a = *p
;`, `S a = q.m;`, `S a = f();`,
    // `expdesc temp = *e1;`---is a whole-object
COPY.  `_kcc_ir_build_struct_addr`
    // resolves the source's base address for
every such shape (ident, call,
    // member, subscript, dereferenced pointer,
parenthesised); copy exactly
    // TY_SIZE bytes.  Earlier this recognised
only an identifier
    // and a call, and used the field count
instead of the byte size---so a
    // union or a `= *p` initializer copied only
the first word.  lcode's
    // `swapexps` (`expdesc temp = *e1;`) hit
exactly that and crashed on `>`.
    bl _kcc_ir_build_struct_addr // x0 = source
struct base address vreg
    mov x24, x0                     // src
    mov x0, x21
    bl _kcc_type_layout             // x0 = aggregate
byte size
    mov x2, x0                       // byte size (
exact)
    mov x0, x24                     // src
    mov x1, x23                     // dst = this
variable's alloca
    bl _kcc_ir_emit_struct_copy
    b .Libvd_done

.Libvd_initlist:
    // x0 = AST_INIT_LIST; x23 = base alloca vreg;
[sp,#72] = total slot count.
    // Zero the whole region first (slot by slot),
then hand the list to the
    // recursive initializer, which descends into
nested braces and follows
    // designator chains.  The declared aggregate
type is resolved afresh
    // because x21 has been narrowed to the *
element* type for an array; the
    // recursion needs the real array/struct type
to size its sub-objects.
    mov x22, x0                     // init-list node
(survives the zero-fill)
    ldr x0, [x19, #AST_F0]          // declared type
spec
    bl _kcc_sema_resolve_type
    mov x21, x0                     // aggregate type
(array/struct/union)

    ldr x24, [sp, #72]              // total slot
count
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x19, x0                     // zero-const
instr (decl node no longer needed)
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x19, #IR_TYPE]
    str xzr, [x19, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x19, #IR_DEST]
    mov x20, x0                     // zero vreg
    mov x0, x19
    bl _kcc_ir_emit

    mov x19, #0                     // slot index
.Libvd_il_zf_loop:
    cmp x19, x24
    b.ge .Libvd_il_emit
    mov x0, x23                     // base
    lsl x1, x19, #3                 // byte offset =
slot * 8
    mov x2, x20                     // zero vreg
    mov x3, #0                      // width = 8-byte
slot
    bl _kcc_ir_emit_indexed_store
    add x19, x19, #1
    b .Libvd_il_zf_loop

.Libvd_il_emit:
    ldr x3, [x22, #AST_F0]          // list head
element
    mov x0, x21                     // aggregate type
    mov x1, x23                     // base alloca
    mov x2, #0                      // base offset
    bl _kcc_ir_fill_list
    b .Libvd_done

.Libvd_extern:
    // `extern int v;` at block scope declares no
storage: it re-exposes the
    // file-scope object.  Bind the name to the "
resolve as global" marker
    // (value 0), journalled so the block's exit
restores any enclosing local of
    // the same name.  A reference then lowers to
```

```
    a GLOBAL_REF, not a stack slot.
     mov x0, x20                    // name
     bl _kcc_strlen
     mov x1, x0                     // keylen
     mov x0, x20                    // name
     mov x2, #0                     // 0 → global
     bl _kcc_ir_locals_put
     b .Libvd_done

.Libvd_static:
     // Hoist: record source-name -> mangled global
    ; skip alloca and init
     mov x0, x19
     bl _kcc_ir_static_name
     mov x21, x0                    // mangled name
    ptr

     mov x0, x20
     bl _kcc_strlen
     mov x2, x0                     // keylen (kept
    in x24 for the type table too)
     mov x24, x0
     adrp x0, _kcc_ir_statics@PAGE
     add  x0, x0, _kcc_ir_statics@PAGEOFF
     ldr x0, [x0]
     mov x1, x20                    // key = source
    name
     mov x3, x21                    // value =
    mangled name ptr
     bl _kcc_ht_put

     // Also record the declared type so the array-
    vs-pointer decision survives
     // the popped block scope (used by identifier
    decay and subscript).
     ldr x0, [x19, #AST_F0]       // declared type
    node
     cbz x0, .Libvd_done
     bl _kcc_sema_resolve_type
     cbz x0, .Libvd_done
     mov x3, x0                     // value = type
    descriptor
     adrp x0, _kcc_ir_static_types@PAGE
     add  x0, x0, _kcc_ir_static_types@PAGEOFF
     ldr x0, [x0]
     mov x1, x20                    // key = source
    name
     mov x2, x24                    // keylen
     bl _kcc_ht_put

.Libvd_done:
     ldp x23, x24, [sp, #48]
     ldp x21, x22, [sp, #32]
     ldp x19, x20, [sp, #16]
     ldp x29, x30, [sp], #80
     ret
```

□

417b · used in 30b · defines _kcc_ir_build_var_decl 322, 350 · uses _kcc_ht_put 63, _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_build_struct_addr 405, _kcc_ir_convert 343, _kcc_ir_emit 304, _kcc_ir_emit_entry 304, _kcc_ir_emit_indexed_store 414, _kcc_ir_emit_struct_copy 404, _kcc_ir_fill_list 414, _kcc_ir_initlist_count 414, _kcc_ir_next_vreg 305a, _kcc_ir_statics 305c, _kcc_ir_static_name 417a, _kcc_ir_static_types 305c, _kcc_ir_store_width 349b, _kcc_ir_type_slots 349a, _kcc_sema_resolve_type 259b, _kcc_type_int_s 250b, _kcc_type_layout 267, 268

End-to-end tests. Each local variable declaration emits an IR_ALLOCA instruction, which the code generator maps to a unique 8-byte slot in the function's stack frame at offset $-(16 + \text{vreg} \times 8)$ from the frame pointer. An initialiser compiles to an IR_STORE that writes the value into the slot, and every subsequent use compiles to an IR_LOAD that reads it back. These tests verify that distinct locals occupy distinct slots and that values survive across intervening statements.

⟨ *Test: codegen locals* 420 ⟩ ≡

```
echo ""
echo "--- Local Variables ---"

run_test "local variable" \
    "int main(void) { int x = 10; return x; }" 10

run_test "two locals" \
    "int main(void) { int a = 3; int b = 4; return
    a + b; }" 7

run_test "local reassignment" \
    "int main(void) { int x = 1; x = 42; return x;
    }" 42

run_test "char literal" \
    "int main(void) { char c = 'A'; return c; }"
   65
```

□

420 · used in 33b

Multi-variable declarations. C allows several variables to share one set of declaration specifiers, separated by commas: `int a=3, b=4;`. The parser links the declarators through F4; the compound-statement builder walks the chain to its tail so that subsequent items are appended correctly.

⟨ *Test: codegen multi-var* 421a ⟩ ≡

```
echo ""
echo "--- Multi-Variable Declarations ---"

run_test "multi-var basic" \
    "int main(void) { int a = 3, b = 4; return a +
    b; }" 7

run_test "multi-var three" \
    "int main(void) { int a = 1, b = 2, c = 3;
   return a + b + c; }" 6

run_test "multi-var after stmt" \
    "int main(void) { int x = 10; int a = 1, b =
   2; return x + a + b; }" 13

run_test "multi-var before stmt" \
    "int main(void) { int a = 5, b = 6; int c = a
   + b; return c; }" 11

run_test "multi-var no init" \
    "int main(void) { int a = 7, b; b = 3; return
   a + b; }" 10

run_test "multi-var in loop" \
    "int main(void) { int s = 0; int i; for (i =
   0; i < 3; i++) { int a = i, b = i + 1; s = s +
   a + b; } return s; }" 9
```

□

421a · used in 33b

⟨ *Test: codegen unsigned* 421b ⟩ ≡

```
echo ""
echo "--- Unsigned and Extensions ---"

run_test "unsigned modulo" \
    "int main(void) { unsigned int a = 17;
   unsigned int b = 5; return a % b; }" 2

run_test "unsigned divide" \
    "int main(void) { unsigned int a = 100;
   unsigned int b = 7; return a / b; }" 14

run_test "unsigned less-than true" \
    "int main(void) { unsigned int a = 3; unsigned
    int b = 5; if (a < b) return 1; return 0; }" 1

run_test "unsigned less-than false" \
    "int main(void) { unsigned int a = 5; unsigned
    int b = 3; if (a < b) return 1; return 0; }" 0

run_test "unsigned less-equal true" \
    "int main(void) { unsigned int a = 5; unsigned
    int b = 5; if (a <= b) return 1; return 0; }"
   1

run_test "unsigned less-equal false" \
    "int main(void) { unsigned int a = 6; unsigned
    int b = 5; if (a <= b) return 1; return 0; }"
   0

run_test "char to int sext" \
    "int main(void) { char c = (char)200; int x =
   (int)c; if (x < 0) return 1; return 0; }" 1

run_test "unsigned char to int zext" \
    "int main(void) { unsigned char c = (unsigned
   char)200; int x = (int)c; return x - 200; }" 0
```

□

421b · used in 33b

29.12. Building Control Flow

Control-flow statements create fresh basic blocks and wire them together with `IR_BR` and `IR_BR_COND`. Each builder follows the same discipline: after emitting a terminator (`IR_BR`, `IR_BR_COND`, or `IR_RET`), it sets `BB_FLAG_TERMINATED` on the current block so that later code does not accidentally append instructions after a branch. Only after switching to a new block with `_kcc_ir_set_bb` can the builder resume emitting.

29.12.1. If Statement

An `if` with an else clause produces three blocks: *then_bb*, *else_bb*, and *merge_bb*. The condition vreg feeds a conditional branch; each arm builds its body, then jumps to *merge_bb* (unless it was already terminated by a `return` or `break`). An `if` without an else skips the *else_bb* and branches either to *then_bb* or directly to *merge_bb*.

⟨ *IR: build if* 421c ⟩ ≡

```
// -- kcc_ir_build_if ---
// void kcc_ir_build_if(node *stmt)
//   x0 = AST_IF_STMT (F0=cond, F1=then, F2=else)
.text
.globl _kcc_ir_build_if
.p2align 2
_kcc_ir_build_if:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    mov x19, x0                  // if_stmt

    // Build condition
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_build_expr
    mov x20, x0                  // cond vreg

    // Create BBs
    bl _kcc_ir_alloc_bb
    mov x21, x0                  // then_bb
    ldr x0, [x19, #AST_F2]      // else branch
    cbz x0, .Libif_no_else

    // Has else
    bl _kcc_ir_alloc_bb
    mov x22, x0                  // else_bb
    bl _kcc_ir_alloc_bb
    mov x23, x0                  // merge_bb

    // Emit br_cond
    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
    str x20, [x0, #IR_DEST]
    ldr x1, [x21, #BB_ID]
    str x1, [x0, #IR_SRC1]
    ldr x1, [x22, #BB_ID]
    str x1, [x0, #IR_SRC2]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // Then block
    mov x0, x21
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_stmt
    // Branch to merge (if not already terminated)
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    tst x1, #BB_FLAG_TERMINATED
    b.ne .Libif_else_block
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x23, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]
```

```
.Libif_else_block:
    mov x0, x22
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_build_stmt
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    tst x1, #BB_FLAG_TERMINATED
    b.ne .Libif_merge
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x23, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

.Libif_merge:
    mov x0, x23
    bl _kcc_ir_set_bb
    b .Libif_done

.Libif_no_else:
    // No else: branch to then_bb or merge_bb
    bl _kcc_ir_alloc_bb
    mov x23, x0                     // merge_bb

    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
    str x20, [x0, #IR_DEST]
    ldr x1, [x21, #BB_ID]
    str x1, [x0, #IR_SRC1]
    ldr x1, [x23, #BB_ID]
    str x1, [x0, #IR_SRC2]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    mov x0, x21
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_stmt
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    tst x1, #BB_FLAG_TERMINATED
    b.ne .Libif_merge2
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x23, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

.Libif_merge2:
    mov x0, x23
    bl _kcc_ir_set_bb

.Libif_done:
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

421c · used in 30b · defines _kcc_ir_build_if 322 · uses _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_build_stmt 322, _kcc_ir_cur_bb 305c, _kcc_ir_emit 304, _kcc_ir_set_bb 305b

29.12.2. While Statement

A `while` loop creates three blocks: *cond_bb* (the loop header), *body_bb*, and *exit_bb*. The entry block branches unconditionally to *cond_bb*, which evaluates the condition and branches to *body_bb* (true) or *exit_bb* (false). After executing the body, a back-edge branches to *cond_bb*. The loop stack receives (`break=exit_bb, continue=cond_bb`) so that `break` and `continue` inside the body emit the correct branch target.

⟨ *IR: build while* 423a ⟩ ≡

```
// -- kcc_ir_build_while ---
// void kcc_ir_build_while(node *stmt)
//   x0 = AST_WHILE_STMT (F0=cond, F1=body)
.text
.globl _kcc_ir_build_while
.p2align 2
_kcc_ir_build_while:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    mov x19, x0

    // Create: cond_bb, body_bb, exit_bb
    bl _kcc_ir_alloc_bb
    mov x20, x0                     // cond_bb
    bl _kcc_ir_alloc_bb
    mov x21, x0                     // body_bb
    bl _kcc_ir_alloc_bb
    mov x22, x0                     // exit_bb

    // Branch to cond_bb
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x20, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // Cond block
    mov x0, x20
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_build_expr
    mov x23, x0                     // cond vreg

    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
```

```
    str x23, [x0, #IR_DEST]
    ldr x1, [x21, #BB_ID]
    str x1, [x0, #IR_SRC1]      // true → body
    ldr x1, [x22, #BB_ID]
    str x1, [x0, #IR_SRC2]      // false → exit
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // Body block - push loop stack (break=exit_bb
    , continue=cond_bb)
    ldr x0, [x22, #BB_ID]       // break target =
    exit_bb id
    ldr x1, [x20, #BB_ID]       // continue target
    = cond_bb id
    bl _kcc_loop_push

    mov x0, x21
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_stmt

    bl _kcc_loop_pop

    // Loop back to cond
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    tst x1, #BB_FLAG_TERMINATED
    b.ne .Libw_exit
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x20, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

.Libw_exit:
    mov x0, x22
    bl _kcc_ir_set_bb

    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

423a · used in 30b · defines _kcc_ir_build_while 322 · uses _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_build_stmt 322, _kcc_ir_cur_bb 305c, _kcc_ir_emit 304, _kcc_ir_set_bb 305b, _kcc_loop_pop 434b, _kcc_loop_push 434b

29.12.3. Do-While Statement

A `do-while` loop guarantees at least one body execution. The CFG is the mirror image of `while`: the entry block branches directly to *body_bb*, which falls through to *cond_bb*. The condition evaluates its expression and branches back to *body_bb* (true) or out to *exit_bb* (false). The loop stack entries are `(break=exit_bb, continue=cond_bb)`, just as with `while`.

⟨ *IR: build do* 423b ⟩ ≡

```
// -- kcc_ir_build_do ---
// void kcc_ir_build_do(node *stmt)
//   x0 = AST_DO_STMT (F0=body, F1=cond)
.text
.globl _kcc_ir_build_do
.p2align 2
_kcc_ir_build_do:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    mov x19, x0

    bl _kcc_ir_alloc_bb
    mov x20, x0                 // body_bb
    bl _kcc_ir_alloc_bb
    mov x21, x0                 // cond_bb
    bl _kcc_ir_alloc_bb
    mov x22, x0                 // exit_bb

    // Branch to body
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x20, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // Body - push loop stack (break=exit_bb,
    continue=cond_bb)
    ldr x0, [x22, #BB_ID]       // break target =
    exit_bb
    ldr x1, [x21, #BB_ID]       // continue target
    = cond_bb
    bl _kcc_loop_push

    mov x0, x20
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_build_stmt

    bl _kcc_loop_pop

    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    tst x1, #BB_FLAG_TERMINATED
    b.ne .Libd_cond
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x21, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

.Libd_cond:
    mov x0, x21
```

```
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_expr
    mov x23, x0

    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
    str x23, [x0, #IR_DEST]
    ldr x1, [x20, #BB_ID]
    str x1, [x0, #IR_SRC1]      // true → body (
    loop)
    ldr x1, [x22, #BB_ID]
    str x1, [x0, #IR_SRC2]      // false → exit
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    mov x0, x22
    bl _kcc_ir_set_bb

    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

423b · used in 30b · defines _kcc_ir_build_do 322 · uses _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_build_stmt 322, _kcc_ir_cur_bb 305c, _kcc_ir_emit 304, _kcc_ir_set_bb 305b, _kcc_loop_pop 434b, _kcc_loop_push 434b

29.12.4. For Statement

A `for` loop is the most complex control-flow construct, producing four blocks: *cond_bb*, *body_bb*, *incr_bb*, and *exit_bb*. The builder first emits the initialiser (which may be a declaration or expression) into the current block, then branches to *cond_bb*. The condition block evaluates its expression (or branches unconditionally to the body for `for(;;)`), the body executes, control flows to *incr_bb* for the increment expression, and a back-edge returns to *cond_bb*. Note that `continue` targets *incr_bb*, not *cond_bb*—ensuring the increment runs before the next condition test.

⟨ *IR: build for* 424 ⟩ ≡

```
// -- kcc_ir_build_for ---
// void kcc_ir_build_for(node *stmt)
//   x0 = AST_FOR_STMT (F0=init, F1=cond, F2=incr,
    F3=body)
.text
.globl _kcc_ir_build_for
.p2align 2
_kcc_ir_build_for:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    mov x19, x0

    // Build init (may be decl or expr)
    ldr x0, [x19, #AST_F0]
    cbz x0, .Libfor_bbs
    bl _kcc_ir_build_stmt

.Libfor_bbs:
    bl _kcc_ir_alloc_bb
    mov x20, x0                 // cond_bb
    bl _kcc_ir_alloc_bb
    mov x21, x0                 // body_bb
    bl _kcc_ir_alloc_bb
    mov x22, x0                 // incr_bb
    bl _kcc_ir_alloc_bb
    mov x23, x0                 // exit_bb

    // Branch to cond
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x20, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    // Cond block
    mov x0, x20
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F1]
    cbz x0, .Libfor_no_cond
    bl _kcc_ir_build_expr
    mov x24, x0

    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
    str x24, [x0, #IR_DEST]
    ldr x1, [x21, #BB_ID]
    str x1, [x0, #IR_SRC1]
    ldr x1, [x23, #BB_ID]
    str x1, [x0, #IR_SRC2]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]
    b .Libfor_body

.Libfor_no_cond:
    // Infinite loop: branch directly to body
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x21, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

.Libfor_body:
    // Push loop stack (break=exit_bb, continue=
    incr_bb)
    ldr x0, [x23, #BB_ID]       // break target =
    exit_bb
    ldr x1, [x22, #BB_ID]       // continue target
    = incr_bb
    bl _kcc_loop_push
```

```
    mov x0, x21
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F3]
    cbz x0, .Libfor_pop_incr
    bl _kcc_ir_build_stmt

.Libfor_pop_incr:
    bl _kcc_loop_pop

.Libfor_incr:
    // Branch to incr (if not terminated)
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    tst x1, #BB_FLAG_TERMINATED
    b.ne .Libfor_incr_bb
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x22, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

.Libfor_incr_bb:
    mov x0, x22
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F2]
    cbz x0, .Libfor_loop_back
    bl _kcc_ir_build_expr        // discard result

.Libfor_loop_back:
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    ldr x1, [x20, #BB_ID]
    str x1, [x0, #IR_SRC1]
    bl _kcc_ir_emit
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    mov x0, x23
    bl _kcc_ir_set_bb

    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

424 · used in 30b · defines _kcc_ir_build_for 322 · uses _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_instr 303a, _kcc_ir_build_expr 340, _kcc_ir_build_stmt 322, _kcc_ir_cur_bb 305c, _kcc_ir_emit 304, _kcc_ir_set_bb 305b, _kcc_loop_pop 434b, _kcc_loop_push 434b

End-to-end tests. The IR builder lowers each control-flow construct into a graph of basic blocks connected by `IR_BR` (unconditional) and `IR_BR_COND` (conditional) terminators. An `if/else` produces a diamond of four blocks (entry, true, false, merge); a `while` loop produces a condition block with a back-edge; a `for` loop adds a separate increment block that `continue` targets. The code generator records each basic block's starting offset in a hash table and patches forward branches after all blocks have been emitted. These tests use the explicit `i = i + 1` form to isolate loop lowering from the increment operators tested elsewhere.

⟨ *Test: codegen control flow* 425 ⟩ ≡

```
echo ""
echo "--- Control Flow ---"

run_test "if true" \
    "int main(void) { if (1) return 10; return 20;
    }" 10

run_test "if false" \
    "int main(void) { if (0) return 10; return 20;
    }" 20

run_test "if-else" \
    "int main(void) { int x = 5; if (x > 3) return
    1; else return 0; }" 1

run_test "nested if" \
    "int main(void) { int x = 5; if (x > 0) { if (
   x > 3) return 1; return 2; } return 3; }" 1

run_test "while loop" \
    "int main(void) { int x = 0; while (x < 10) x
   = x + 1; return x; }" 10

run_test "do-while loop" \
    "int main(void) { int i = 0; int s = 0; do { s
    = s + i; i = i + 1; } while (i <= 5); return s
   ; }" 15

run_test "for loop" \
    "int main(void) { int s = 0; for (int i = 0; i
    < 5; i = i + 1) s = s + 2; return s; }" 10

run_test "break" \
    "int main(void) { int i = 0; while (1) { if (i
    == 5) break; i = i + 1; } return i; }" 5

run_test "continue" \
    "int main(void) { int s = 0; for (int i = 0; i
    < 10; i = i + 1) { if (i % 2 == 0) continue; s
    = s + 1; } return s; }" 5

run_test "goto backward loop" \
    "int main(void) { int i = 0; int s = 0; loop:
   if (i >= 5) goto end; s = s + i; i = i + 1;
   goto loop; end: return s; }" 10

run_test "goto forward" \
    "int main(void) { int x = 5; if (x > 0) goto
   pos; return 1; pos: return 42; }" 42

run_test "goto cleanup" \
    "int f(int n) { int r = 0; if (n < 0) goto
   done; r = n * 2; done: return r; } int main(
   void) { return f(21); }" 42

run_test "multiple returns" \
    "int f(int x) { if (x > 0) return 1; if (x <
   0) return 2; return 0; } int main(void) {
   return f(5) + f(-3) + f(0); }" 3
```

□

425 · used in 33b

29.12.5. Switch Statement

A `switch` statement (§6.8.4.2) evaluates a controlling expression and transfers control to the matching `case` label. The IR builder lowers it to a compare-and-branch dispatch chain: for each `case`, a fresh basic block tests `cond_vreg == case_val` and branches to the case body on success or to the next comparison on failure. After all cases, the chain falls through to the `default` body if present, or to the exit block otherwise.

A GNU *case range* (`case LOW…HIGH:`, with the high bound in the label's `F3`) lowers to two comparisons instead of one: the dispatch block tests `cond_vreg >= LOW` (signed) and branches to a *middle* block on success, which then tests `cond_vreg <= HIGH` and branches to the case body; either test failing falls through to the next comparison. Signed comparisons match the promoted controlling type for the small integer and character ranges this feature is used for.

Case bodies observe C fall-through semantics: unless a `break` terminates the current basic block, execution flows into the next case's body. The builder achieves this by emitting an unconditional branch from a non-terminated body block to the next case's block. The exit block is pushed onto the loop stack as a break target so that `break` statements inside the switch branch to it.

A `case` or `default` label is not confined to the top level of the switch's body: the standard (§6.8.1) lets it appear anywhere inside, nested within blocks, `if`s and loops — the basis of Duff's device, where cases interleave a `do`/`while`. The builder therefore treats a label like a `goto` target. Two recursive scans, `_kcc_ir_sw_scan` and `_kcc_ir_sw_disp`, walk the whole body sub-tree — descending through compounds, `if`s, loops and labels but stopping at a nested `switch`, whose labels are its own. The first allocates a basic block for every label and stashes it in an unused AST field (`F2` for a case, `F1` for a default), recording the default's block in `_kcc_sw_default_bb`. The second, driven by the controlling value in `_kcc_sw_cond_vreg`, emits the dispatch chain in source order, each comparison in its own block terminated by `IR_BR_COND`. The body is then built by a single ordinary `_kcc_ir_build_stmt` call: a `case`/`default` node encountered during that walk branches the current block into its pre-allocated block and continues there, giving C fall-through for free and placing interior labels exactly where they textually occur.

The constant folder `_kcc_eval_const_int` that backs case values and array bounds also evaluates `__builtin_types_compatible_p`: it resolves both operand types and returns `_kcc_type_compatible`'s 1/0 verdict, so the builtin is a genuine integer constant expression usable in a `static_assert`, an array bound, or a `_Generic`-style compile-time choice. The rodata predicate accepts it too, so a file-scope initializer folds rather than being mistaken for a run-time value.

The evaluator also folds `offsetof`-style null-based address constants, the one addition that lets kcc compile the SQLite amalgamation on this target as it already does on x86-64. `offsetof(T, m)` expands to `((size_t)&(((T*)0)->m))`, and a bound such as `char b[sizeof(P) - offsetof(P, tail)]` — SQLite's saved-parser buffer — is a perfectly good integer constant expression that clang accepts. Two small mutually-recursive helpers add it: `_kcc_addr_is_iconst` decides whether the address of an lvalue is a constant (its ultimate base must be a constant pointer, so it walks `&`, member access, and `*` down to a cast-of-zero), and `_kcc_eval_addr` computes the value — a member adds its byte offset (found through the same `_kcc_type_find_member` the IR builder uses, so type-check and fold agree) to the base pointer's value. `_kcc_expr_is_ice` routes a `&` through the former and `_kcc_eval_const_int` routes it through the latter; without them the member offset folds to 0 and the array is wrongly rejected as a variable-length array. During array-bound folding the operand's type may not yet be resolved, so `_kcc_eval_addr` type-checks it on demand, exactly as the `sizeof` case does.

⟨ *IR: build switch* 426 ⟩ ≡

```
// -- kcc_eval_const_int ---
// int64_t kcc_eval_const_int(node *n)
//   Evaluate an integer constant expression (\
   stdref{6.6}): literals,
//   parentheses, casts, sizeof, an enum-constant
   identifier, and the common
//   unary and binary integer operators---
   including the relational and logical
//   operators (evaluated signed), so a constant
   condition such as `[(1 < 2) + 3]'
//   folds rather than reporting 0.  Used both for
    a switch case value
//   (`1 << 5`, `cJSON_True`) and for an array
   declarator's size
//   (`a[MAXABITS + 1]`, or `tmname[TM_N]` where
   TM_N is an enumerator); a size
//   that is not a bare AST_INT_LIT must still be
   folded, or the array is sized
//   zero and its storage collides with whatever
   the frame---or the enclosing
//   struct---packs next.  An identifier is
   resolved through the symbol table
//   and yields its value only when it names an
   enum constant; every other
//   unsupported shape evaluates to 0.
.text
.globl _kcc_eval_const_int
.p2align 2
_kcc_eval_const_int:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    cbz x0, .Lec_zero
    mov x19, x0
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_INT_LIT
    b.eq .Lec_lit
    cmp x0, #AST_CHAR_LIT
    b.eq .Lec_lit
    cmp x0, #AST_PAREN_EXPR
    b.eq .Lec_paren
    cmp x0, #AST_CAST_EXPR
    b.eq .Lec_cast
    cmp x0, #AST_UNARY_EXPR
    b.eq .Lec_unary
    cmp x0, #AST_BINARY_EXPR
    b.eq .Lec_binary
    cmp x0, #AST_SIZEOF_EXPR
    b.eq .Lec_sizeof
    cmp x0, #AST_IDENT_EXPR
    b.eq .Lec_ident
    cmp x0, #AST_TYPES_COMPAT
```

```
    b.eq .Lec_types_compat
.Lec_zero:
    mov x0, #0
    b .Lec_ret
.Lec_lit:
    ldr x0, [x19, #AST_F0]
    b .Lec_ret

.Lec_ident:
    // An identifier in an integer constant
   expression must name an enum
    // constant (§6.6): look it up and yield its
   value.  Without this, an
    // array size or enum initializer written in
   terms of an enumerator---
    // e.g. `TString *tmname[TM_N]`---folded to 0,
    giving the array zero
    // length and silently shifting every later
   struct field.
    ldr x21, [x19, #AST_F0]      // name ptr
    cbz x21, .Lec_zero
    mov x0, x21
    bl _kcc_strlen
    mov x1, x0
    mov x0, x21
    bl _kcc_symtab_lookup
    cbz x0, .Lec_zero
    ldr x1, [x0, #SYM_KIND]
    cmp x1, #SYM_ENUM_CONST
    b.ne .Lec_zero
    ldr x0, [x0, #SYM_VALUE]
    b .Lec_ret

.Lec_sizeof:
    // sizeof(T) → the type's byte size; sizeof(
   expr) → the operand's type size.
    // F1 flags a type operand; F0 is the type
   node or the expression.
    ldr x1, [x19, #AST_F1]       // is_type?
    ldr x0, [x19, #AST_F0]       // operand
    cbz x0, .Lec_zero
    cbz x1, .Lec_sizeof_expr
    bl _kcc_sema_resolve_type    // sizeof(type)
    b .Lec_sizeof_size
.Lec_sizeof_expr:
    // sizeof(expr): the operand's resolved type
   in AST_F3.  Folding an array
    // bound (\stdref{6.7.6.2}) runs during
   declaration resolution, before sema
    // has visited the operand, so its AST_F3 may
   still be null; type-check it
    // now.  Without this `unsigned int buf[sizeof
   (x)/sizeof(int)]` folded to a
    // zero-length array whose under-sized alloca
   overran the frame's
    // callee-save area---exactly the corruption
   in Lua's `l_randomizePivot`.
    ldr x1, [x0, #AST_F3]
    cbnz x1, .Lec_sizeof_have
    bl _kcc_sema_expr            // resolve operand
    type; returns it in x0
    b .Lec_sizeof_size
.Lec_sizeof_have:
    mov x0, x1
.Lec_sizeof_size:
    cbz x0, .Lec_zero
    bl _kcc_type_layout          // x0 = byte size
    b .Lec_ret
.Lec_types_compat:
    // __builtin_types_compatible_p(T1, T2) → 1 if
    compatible, else 0.  x20/x21
    // are free here (only x19 holds the node,
   restored at the epilogue's ldp).
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    mov x20, x0
    ldr x0, [x19, #AST_F1]
    bl _kcc_sema_resolve_type
    mov x1, x0
    mov x0, x20
    bl _kcc_type_compatible
    b .Lec_ret
.Lec_paren:
    ldr x0, [x19, #AST_F0]
    bl _kcc_eval_const_int
    b .Lec_ret
.Lec_cast:
    ldr x0, [x19, #AST_F1]       // value of the
   cast operand (type ignored)
    bl _kcc_eval_const_int
    b .Lec_ret
.Lec_unary:
    ldr x20, [x19, #AST_F0]      // op
    cmp x20, #TOK_AMP
    b.eq .Lec_un_addr            // &lvalue: a null
   -based address constant (offsetof)
    ldr x0, [x19, #AST_F1]
    bl _kcc_eval_const_int
    cmp x20, #TOK_MINUS
    b.ne .Lec_un_tilde
    neg x0, x0
    b .Lec_ret
.Lec_un_tilde:
    cmp x20, #TOK_TILDE
    b.ne .Lec_un_bang
    mvn x0, x0
    b .Lec_ret
.Lec_un_bang:
    cmp x20, #TOK_BANG
    b.ne .Lec_ret                // +x, *x → value
   unchanged
    cmp x0, #0
    cset x0, eq
    b .Lec_ret
.Lec_un_addr:
    // &(((T*)0)->m) --- the expansion of offsetof
    --- folds to the member's byte
    // offset via the address evaluator, so a
   bound like
    // `char b[sizeof(P) - offsetof(P, tail)]`
   lays out at its true size instead
    // of folding to 0 (which under-sizes the
   array).
    ldr x0, [x19, #AST_F1]
    bl _kcc_eval_addr
    b .Lec_ret
.Lec_binary:
    ldr x20, [x19, #AST_F0]      // op
    ldr x0, [x19, #AST_F1]
    bl _kcc_eval_const_int
    mov x21, x0                  // lhs value
    ldr x0, [x19, #AST_F2]
    bl _kcc_eval_const_int
    mov x1, x0                   // rhs value
    mov x0, x21                  // lhs value
    cmp x20, #TOK_PLUS
    b.eq .Lec_add
    cmp x20, #TOK_MINUS
    b.eq .Lec_sub
    cmp x20, #TOK_STAR
    b.eq .Lec_mul
    cmp x20, #TOK_SLASH
    b.eq .Lec_div
    cmp x20, #TOK_PERCENT
    b.eq .Lec_mod
    cmp x20, #TOK_SHL
    b.eq .Lec_shl
    cmp x20, #TOK_SHR
    b.eq .Lec_shr
```

```
    cmp x20, #TOK_AMP
    b.eq .Lec_and
    cmp x20, #TOK_PIPE
    b.eq .Lec_or
    cmp x20, #TOK_CARET
    b.eq .Lec_xor
    cmp x20, #TOK_LT
    b.eq .Lec_lt
    cmp x20, #TOK_GT
    b.eq .Lec_gt
    cmp x20, #TOK_LE
    b.eq .Lec_le
    cmp x20, #TOK_GE
    b.eq .Lec_ge
    cmp x20, #TOK_EQ
    b.eq .Lec_eq
    cmp x20, #TOK_NE
    b.eq .Lec_ne
    cmp x20, #TOK_AND
    b.eq .Lec_land
    cmp x20, #TOK_OR
    b.eq .Lec_lor
    b .Lec_zero
.Lec_add:  add x0, x0, x1
    b .Lec_ret
.Lec_sub:  sub x0, x0, x1
    b .Lec_ret
.Lec_mul:  mul x0, x0, x1
    b .Lec_ret
.Lec_div:  cbz x1, .Lec_zero
    sdiv x0, x0, x1
    b .Lec_ret
.Lec_mod:  cbz x1, .Lec_zero
    sdiv x2, x0, x1
    msub x0, x2, x1, x0
    b .Lec_ret
.Lec_shl:  lsl x0, x0, x1
    b .Lec_ret
.Lec_shr:  asr x0, x0, x1
    b .Lec_ret
.Lec_and:  and x0, x0, x1
    b .Lec_ret
.Lec_or:   orr x0, x0, x1
    b .Lec_ret
.Lec_xor:  eor x0, x0, x1
    b .Lec_ret
    // Relational and logical operators yield 0/1;
    comparisons are signed.
.Lec_lt:   cmp x0, x1
    cset x0, lt
    b .Lec_ret
.Lec_gt:   cmp x0, x1
    cset x0, gt
    b .Lec_ret
.Lec_le:   cmp x0, x1
    cset x0, le
    b .Lec_ret
.Lec_ge:   cmp x0, x1
    cset x0, ge
    b .Lec_ret
.Lec_eq:   cmp x0, x1
    cset x0, eq
    b .Lec_ret
.Lec_ne:   cmp x0, x1
    cset x0, ne
    b .Lec_ret
.Lec_land: cmp x0, #0                  // (x0 && x1
  )
    ccmp x1, #0, #4, ne
    cset x0, ne
    b .Lec_ret
.Lec_lor:  orr x2, x0, x1              // (x0 || x1
  )
    cmp x2, #0
    cset x0, ne
.Lec_ret:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_eval_addr ---
// uint64_t kcc_eval_addr(node *n)
//   The integer value of a null-based address
  constant --- the address of an
//   lvalue whose ultimate base is a constant
  pointer, as produced by
//   `offsetof(T, m)` = `((size_t)&(((T*)0)->m))`.
    A member access adds its
//   byte offset within the aggregate to the base;
    `p->m` takes the pointer's
//   value as the base, `s.m` the address of `s`;
  a dereference `*p` yields the
//   pointer's value.  Anything else contributes
  0.  Paired with
//   kcc_addr_is_iconst, which decides when such
  an address is a constant.
.p2align 2
_kcc_eval_addr:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    cbz x0, .Lea_zero
    mov x19, x0
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_PAREN_EXPR
    b.eq .Lea_paren
    cmp x0, #AST_MEMBER
    b.eq .Lea_member
    cmp x0, #AST_UNARY_EXPR
    b.eq .Lea_unary
.Lea_zero:
    mov x0, #0
    b .Lea_ret
.Lea_paren:
    ldr x0, [x19, #AST_F0]
    bl _kcc_eval_addr
    b .Lea_ret
.Lea_unary:
    ldr x1, [x19, #AST_F0]       // op
    cmp x1, #TOK_STAR
    b.ne .Lea_zero               // only *p
  contributes an address value
    ldr x0, [x19, #AST_F1]
    bl _kcc_eval_const_int
    b .Lea_ret
.Lea_member:
    ldr x22, [x19, #AST_F0]      // object
  expression
    ldr x0, [x19, #AST_F2]       // arrow (p->m) vs
    dot (s.m)
    cbz x0, .Lea_dot
    mov x0, x22                  // p->m: base =
  the pointer's value
    bl _kcc_eval_const_int
    b .Lea_base_done
.Lea_dot:
    mov x0, x22                  // s.m: base = the
    address of s
    bl _kcc_eval_addr
.Lea_base_done:
    mov x21, x0                  // base
    // Resolve the object's type.  During array-
  bound folding sema has not yet
    // visited the operand, so its AST_F3 may be
  null; type-check it now.
    ldr x20, [x22, #AST_F3]
    cbnz x20, .Lea_have_type
    mov x0, x22
    bl _kcc_sema_expr
```

```
    mov x20, x0
.Lea_have_type:
    cbz x20, .Lea_base_ret
    ldr x0, [x20, #TY_KIND]
    cmp x0, #TY_PTR
    b.ne .Lea_check_agg
    ldr x20, [x20, #TY_F0]      // arrow: follow
   the pointer to the aggregate
    cbz x20, .Lea_base_ret
.Lea_check_agg:
    ldr x0, [x20, #TY_KIND]
    cmp x0, #TY_STRUCT
    b.eq .Lea_find
    cmp x0, #TY_UNION
    b.ne .Lea_base_ret
.Lea_find:
    mov x0, x20                 // struct/union
   type
    ldr x1, [x19, #AST_F1]      // member name
    bl _kcc_type_find_member    // x0 = field decl
    (0 if none), x1 = byte offset
    cbz x0, .Lea_base_ret       // member not
   found → add nothing
    add x0, x21, x1
    b .Lea_ret
.Lea_base_ret:
    mov x0, x21
.Lea_ret:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_expr_is_ice ---
// uint64_t kcc_expr_is_ice(node *n)
//   1 if n is an integer constant expression (\
   stdref{6.6}) --- literals,
//   sizeof/_Alignof, an enum-constant identifier,
    and const-ops thereof ---
//   else 0.  Used to tell a real array bound from
    a variable-length one: an
//   array declarator whose size is not an ICE is
   a VLA.  A null bound (`a[]`)
//   is reported as constant so an incomplete
   array is not mistaken for a VLA.
.globl _kcc_expr_is_ice
.p2align 2
_kcc_expr_is_ice:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    cbz x0, .Lice_yes
    mov x19, x0
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_INT_LIT
    b.eq .Lice_yes
    cmp x0, #AST_CHAR_LIT
    b.eq .Lice_yes
    cmp x0, #AST_SIZEOF_EXPR
    b.eq .Lice_yes
    cmp x0, #AST_ALIGNOF_EXPR
    b.eq .Lice_yes
    cmp x0, #AST_PAREN_EXPR
    b.eq .Lice_f0
    cmp x0, #AST_CAST_EXPR
    b.eq .Lice_f1
    cmp x0, #AST_UNARY_EXPR
    b.eq .Lice_unary
    cmp x0, #AST_BINARY_EXPR
    b.eq .Lice_bin
    cmp x0, #AST_COND_EXPR
    b.eq .Lice_cond
    cmp x0, #AST_IDENT_EXPR
    b.eq .Lice_ident
    b .Lice_no
.Lice_f0:
    ldr x0, [x19, #AST_F0]
    bl _kcc_expr_is_ice
    b .Lice_ret
.Lice_f1:
    ldr x0, [x19, #AST_F1]
    bl _kcc_expr_is_ice
    b .Lice_ret
.Lice_unary:
    ldr x1, [x19, #AST_F0]      // op
    cmp x1, #TOK_AMP
    b.eq .Lice_addr             // &lvalue → ICE
   iff its address is (offsetof)
    cmp x1, #TOK_STAR
    b.eq .Lice_no               // *p is not a
   constant expression
    ldr x0, [x19, #AST_F1]      // +x, -x, ~x, !x
   → operand must be an ICE
    bl _kcc_expr_is_ice
    b .Lice_ret
.Lice_addr:
    ldr x0, [x19, #AST_F1]
    bl _kcc_addr_is_iconst
    b .Lice_ret
.Lice_bin:
    ldr x0, [x19, #AST_F1]
    bl _kcc_expr_is_ice
    cbz x0, .Lice_no
    ldr x0, [x19, #AST_F2]
    bl _kcc_expr_is_ice
    b .Lice_ret
.Lice_cond:
    ldr x0, [x19, #AST_F0]
    bl _kcc_expr_is_ice
    cbz x0, .Lice_no
    ldr x0, [x19, #AST_F1]
    bl _kcc_expr_is_ice
    cbz x0, .Lice_no
    ldr x0, [x19, #AST_F2]
    bl _kcc_expr_is_ice
    b .Lice_ret
.Lice_ident:
    ldr x20, [x19, #AST_F0]      // name
    cbz x20, .Lice_no
    mov x0, x20
    bl _kcc_strlen
    mov x1, x0
    mov x0, x20
    bl _kcc_symtab_lookup
    cbz x0, .Lice_no
    ldr x1, [x0, #SYM_KIND]
    cmp x1, #SYM_ENUM_CONST
    b.eq .Lice_yes
.Lice_no:
    mov x0, #0
    b .Lice_ret
.Lice_yes:
    mov x0, #1
.Lice_ret:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_addr_is_iconst ---
// uint64_t kcc_addr_is_iconst(node *n)
//   Whether the address of the lvalue `n` reduces
    to an integer constant
//   expression: its ultimate base must be a
   constant pointer, as in the
//   `&((T*)0)->m)` that offsetof expands to.  A
   member keeps walking (`p->m`
//   requires the pointer `p` to be an ICE, `s.m`
   requires `s`'s address to be a
//   constant); a dereference `*p` requires `p` to
    be an ICE; anything else is 0.
```

```
.p2align 2
_kcc_addr_is_iconst:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    cbz x0, .Laic_no
    mov x19, x0
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_PAREN_EXPR
    b.eq .Laic_paren
    cmp x0, #AST_MEMBER
    b.eq .Laic_member
    cmp x0, #AST_UNARY_EXPR
    b.eq .Laic_unary
.Laic_no:
    mov x0, #0
    b .Laic_ret
.Laic_paren:
    ldr x0, [x19, #AST_F0]
    bl _kcc_addr_is_iconst
    b .Laic_ret
.Laic_member:
    ldr x1, [x19, #AST_F2]      // arrow (p->m) vs
    dot (s.m)
    ldr x0, [x19, #AST_F0]      // base object
    cbz x1, .Laic_member_dot
    bl _kcc_expr_is_ice         // p->m: the
   pointer must be an ICE
    b .Laic_ret
.Laic_member_dot:
    bl _kcc_addr_is_iconst      // s.m: the base
   address must be constant
    b .Laic_ret
.Laic_unary:
    ldr x1, [x19, #AST_F0]      // op
    cmp x1, #TOK_STAR
    b.ne .Laic_no
    ldr x0, [x19, #AST_F1]
    bl _kcc_expr_is_ice
    b .Laic_ret
.Laic_ret:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- Switch scan state (shared by the two
   recursive walkers below) ---
.data
.p2align 3
_kcc_sw_default_bb: .quad 0
_kcc_sw_cond_vreg:  .quad 0
.text

// -- kcc_ir_sw_scan ---
// void kcc_ir_sw_scan(node *stmt)
// Allocate a basic block for every case/default
   label reachable as a
// sub-statement of a switch body, stashing it in
   the label (F2 for a case,
// F1 for a default) and recording the default's
   block in _kcc_sw_default_bb.
// Descends through compounds, if, loops and
   labels; a nested switch owns its
// own labels, so it is not entered.
.p2align 2
_kcc_ir_sw_scan:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    cbz x0, .Lsws_done
    mov x19, x0
    ldr x20, [x19, #AST_TAG]
    cmp x20, #AST_CASE_STMT
    b.eq .Lsws_case
    cmp x20, #AST_DEFAULT_STMT
    b.eq .Lsws_default
    cmp x20, #AST_COMPOUND
    b.eq .Lsws_compound
    cmp x20, #AST_IF_STMT
    b.eq .Lsws_if
    cmp x20, #AST_WHILE_STMT
    b.eq .Lsws_body_f1
    cmp x20, #AST_DO_STMT
    b.eq .Lsws_body_f0
    cmp x20, #AST_FOR_STMT
    b.eq .Lsws_body_f3
    cmp x20, #AST_LABEL_STMT
    b.eq .Lsws_body_f1
    b .Lsws_done                // nested switch
   / leaf: do not descend
.Lsws_case:
    bl _kcc_ir_alloc_bb
    str x0, [x19, #AST_F2]      // stash case BB
    ldr x0, [x19, #AST_F1]      // case body may
   nest further labels
    bl _kcc_ir_sw_scan
    b .Lsws_done
.Lsws_default:
    bl _kcc_ir_alloc_bb
    str x0, [x19, #AST_F1]      // stash default
   BB
    adrp x1, _kcc_sw_default_bb@PAGE
    add  x1, x1, _kcc_sw_default_bb@PAGEOFF
    str x0, [x1]
    ldr x0, [x19, #AST_F0]      // default body
   may nest further labels
    bl _kcc_ir_sw_scan
    b .Lsws_done
.Lsws_compound:
    ldr x19, [x19, #AST_F0]     // first item
.Lsws_comp_loop:
    cbz x19, .Lsws_done
    mov x0, x19
    bl _kcc_ir_sw_scan
    ldr x19, [x19, #AST_F4]
    b .Lsws_comp_loop
.Lsws_if:
    ldr x0, [x19, #AST_F1]      // then
    bl _kcc_ir_sw_scan
    ldr x0, [x19, #AST_F2]      // else
    bl _kcc_ir_sw_scan
    b .Lsws_done
.Lsws_body_f0:
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_sw_scan
    b .Lsws_done
.Lsws_body_f1:
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_sw_scan
    b .Lsws_done
.Lsws_body_f3:
    ldr x0, [x19, #AST_F3]
    bl _kcc_ir_sw_scan
.Lsws_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_ir_sw_disp ---
// void kcc_ir_sw_disp(node *stmt)
// Emit the dispatch comparison for every case
   label, in source order, testing
// the controlling value in _kcc_sw_cond_vreg.
   Same traversal as the scan; a
// default label emits no comparison (it is the
   fall-through target) but its
// body is still descended for nested cases.
.p2align 2
_kcc_ir_sw_disp:
```

```
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    cbz x0, .Lswd_done
    mov x19, x0
    ldr x20, [x19, #AST_TAG]
    cmp x20, #AST_CASE_STMT
    b.eq .Lswd_case
    cmp x20, #AST_DEFAULT_STMT
    b.eq .Lswd_default
    cmp x20, #AST_COMPOUND
    b.eq .Lswd_compound
    cmp x20, #AST_IF_STMT
    b.eq .Lswd_if
    cmp x20, #AST_WHILE_STMT
    b.eq .Lswd_body_f1
    cmp x20, #AST_DO_STMT
    b.eq .Lswd_body_f0
    cmp x20, #AST_FOR_STMT
    b.eq .Lswd_body_f3
    cmp x20, #AST_LABEL_STMT
    b.eq .Lswd_body_f1
    b .Lswd_done
.Lswd_case:
    // GNU case range `case LOW ... HIGH:` carries
    the high bound in F3.
    ldr x0, [x19, #AST_F3]
    cbnz x0, .Lswd_case_range
    // Evaluate the case constant → IR_CONST
    ldr x0, [x19, #AST_F0]
    bl _kcc_eval_const_int
    mov x21, x0
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x21, [x22, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                    //
  case_const_vreg
    mov x0, x22
    bl _kcc_ir_emit
    // IR_CMP_EQ(cond_vreg, case_const_vreg)
    mov x0, #IR_CMP_EQ
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    adrp x0, _kcc_sw_cond_vreg@PAGE
    add  x0, x0, _kcc_sw_cond_vreg@PAGEOFF
    ldr x0, [x0]
    str x0, [x22, #IR_SRC1]        // cond_vreg
    str x21, [x22, #IR_SRC2]       //
  case_const_vreg
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                    // cmp result
  vreg
    mov x0, x22
    bl _kcc_ir_emit
    // Fresh BB for the next comparison (false
  target)
    bl _kcc_ir_alloc_bb
    mov x22, x0
    // IR_BR_COND: true → case_bb, false → next
  dispatch BB
    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_DEST]
    ldr x1, [x19, #AST_F2]         // case_bb
    ldr x1, [x1, #BB_ID]
    str x1, [x0, #IR_SRC1]
    ldr x1, [x22, #BB_ID]
    str x1, [x0, #IR_SRC2]
    bl _kcc_ir_emit
    // Terminate current dispatch BB, switch to
  the next one
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]
    mov x0, x22
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F1]         // descend case
  body for nested cases
    bl _kcc_ir_sw_disp
    b .Lswd_done

.Lswd_case_range:
    // GNU case range: take the case body iff the
  switch value is in [LOW, HIGH]
    // (signed).  Dispatch as `cond >= LOW` then,
  in a middle block, `cond <= HIGH`.
    // IR_CONST(LOW)
    ldr x0, [x19, #AST_F0]
    bl _kcc_eval_const_int
    mov x21, x0
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x21, [x22, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                    // low_const_vreg
    mov x0, x22
    bl _kcc_ir_emit
    // IR_CMP_SGE(cond_vreg, low_const_vreg)
    mov x0, #IR_CMP_SGE
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    adrp x0, _kcc_sw_cond_vreg@PAGE
    add  x0, x0, _kcc_sw_cond_vreg@PAGEOFF
    ldr x0, [x0]
    str x0, [x22, #IR_SRC1]
    str x21, [x22, #IR_SRC2]       // low_const_vreg
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                    // ge result vreg
    mov x0, x22
    bl _kcc_ir_emit
    // Middle BB (high check) and next dispatch BB
    (false target for both tests)
    bl _kcc_ir_alloc_bb
    mov x22, x0                    // mid_bb
    bl _kcc_ir_alloc_bb
    mov x20, x0                    // next_bb
    // IR_BR_COND: cond>=LOW → mid_bb, else
  next_bb
    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_DEST]
```

```
    ldr x1, [x22, #BB_ID]
    str x1, [x0, #IR_SRC1]
    ldr x1, [x20, #BB_ID]
    str x1, [x0, #IR_SRC2]
    bl _kcc_ir_emit
    // Terminate current dispatch BB, switch to
mid_bb
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]
    mov x0, x22
    bl _kcc_ir_set_bb
    // IR_CONST(HIGH)
    ldr x0, [x19, #AST_F3]
    bl _kcc_eval_const_int
    mov x21, x0
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    str x21, [x22, #IR_SRC1]
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                    //
high_const_vreg
    mov x0, x22
    bl _kcc_ir_emit
    // IR_CMP_SLE(cond_vreg, high_const_vreg)
    mov x0, #IR_CMP_SLE
    bl _kcc_ir_alloc_instr
    mov x22, x0
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x22, #IR_TYPE]
    adrp x0, _kcc_sw_cond_vreg@PAGE
    add  x0, x0, _kcc_sw_cond_vreg@PAGEOFF
    ldr x0, [x0]
    str x0, [x22, #IR_SRC1]
    str x21, [x22, #IR_SRC2]      //
high_const_vreg
    bl _kcc_ir_next_vreg
    str x0, [x22, #IR_DEST]
    mov x21, x0                    // le result vreg
    mov x0, x22
    bl _kcc_ir_emit
    // IR_BR_COND: cond<=HIGH → case_bb, else
next_bb
    mov x0, #IR_BR_COND
    bl _kcc_ir_alloc_instr
    str x21, [x0, #IR_DEST]
    ldr x1, [x19, #AST_F2]         // case_bb
    ldr x1, [x1, #BB_ID]
    str x1, [x0, #IR_SRC1]
    ldr x1, [x20, #BB_ID]          // next_bb
    str x1, [x0, #IR_SRC2]
    bl _kcc_ir_emit
    // Terminate mid_bb, switch to next_bb
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]
    mov x0, x20
    bl _kcc_ir_set_bb
    ldr x0, [x19, #AST_F1]         // descend case
body for nested cases
    bl _kcc_ir_sw_disp
    b .Lswd_done
.Lswd_default:
    ldr x0, [x19, #AST_F0]         // descend
  default body for nested cases
    bl _kcc_ir_sw_disp
    b .Lswd_done
.Lswd_compound:
    ldr x19, [x19, #AST_F0]
.Lswd_comp_loop:
    cbz x19, .Lswd_done
    mov x0, x19
    bl _kcc_ir_sw_disp
    ldr x19, [x19, #AST_F4]
    b .Lswd_comp_loop
.Lswd_if:
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_sw_disp
    ldr x0, [x19, #AST_F2]
    bl _kcc_ir_sw_disp
    b .Lswd_done
.Lswd_body_f0:
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_sw_disp
    b .Lswd_done
.Lswd_body_f1:
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_sw_disp
    b .Lswd_done
.Lswd_body_f3:
    ldr x0, [x19, #AST_F3]
    bl _kcc_ir_sw_disp
.Lswd_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_ir_build_switch ---
// void kcc_ir_build_switch(node *stmt)
//   x0 = AST_SWITCH_STMT (F0=expr, F1=body)
// Lowers to: dispatch chain → body (labels placed
    by build_stmt) → exit_bb
.text
.globl _kcc_ir_build_switch
.p2align 2
_kcc_ir_build_switch:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str x27, [sp, #80]

    mov x19, x0                    // x19 = switch
  node

    // 1. Controlling expression → global cond
  vreg (read by the dispatch walk)
    ldr x0, [x19, #AST_F0]
    bl _kcc_ir_build_expr
    adrp x1, _kcc_sw_cond_vreg@PAGE
    add  x1, x1, _kcc_sw_cond_vreg@PAGEOFF
    str x0, [x1]

    // 2. Allocate exit BB
    bl _kcc_ir_alloc_bb
    mov x21, x0                    // x21 = exit_bb

    // 3. Pass 1: recursively allocate a BB for
  every label; reset default first
    adrp x0, _kcc_sw_default_bb@PAGE
    add  x0, x0, _kcc_sw_default_bb@PAGEOFF
```

```
    str xzr, [x0]
    ldr x0, [x19, #AST_F1]        // switch body
    bl _kcc_ir_sw_scan
    adrp x0, _kcc_sw_default_bb@PAGE
    add  x0, x0, _kcc_sw_default_bb@PAGEOFF
    ldr x22, [x0]                 // x22 =
   default_bb (0 if none)

    // 4. Pass 2: recursively emit the dispatch
   chain
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_sw_disp

    // 5. End of dispatch: fall into the default
   block, or the exit if none
    cbz x22, .Lsw_tail_exit
    mov x0, x22
    b .Lsw_tail_go
.Lsw_tail_exit:
    mov x0, x21
.Lsw_tail_go:
    bl _kcc_ir_branch_to

    // 6. Pass 3: build the whole body as one
   statement; case/default labels
    //    place themselves (see kcc_ir_build_stmt)
   .  Exit is the break target.
    //    A switch is transparent to `continue` (\
   stdref{6.8.6.2}): a continue
    //    in a switch body belongs to the
   enclosing loop, not the switch, so we
    //    inherit the current top-of-stack
   continue target rather than clearing
    //    it (0 only when the switch is not inside
    any loop).
    ldr x0, [x21, #BB_ID]         // break =
   exit_bb
    adrp x1, _kcc_loop_depth@PAGE
    add  x1, x1, _kcc_loop_depth@PAGEOFF
    ldr  x2, [x1]
    cbz  x2, .Lsw_cont_none       // no enclosing
   loop → continue target 0
    sub  x2, x2, #1               // enclosing
   entry index
    lsl  x2, x2, #4               // * 16
    add  x2, x2, #8               // + 8 =
   continue_bb_id field
    adrp x3, _kcc_loop_stack@PAGE
    add  x3, x3, _kcc_loop_stack@PAGEOFF
    ldr  x1, [x3, x2]             // x1 = inherited
    continue target
    b    .Lsw_cont_have
.Lsw_cont_none:
    mov  x1, #0
.Lsw_cont_have:
    bl _kcc_loop_push
    ldr x0, [x19, #AST_F1]
    bl _kcc_ir_build_stmt
    bl _kcc_loop_pop

    // 7. Fall the final body block into exit,
   then continue there
    mov x0, x21
    bl _kcc_ir_branch_to
    mov x0, x21
    bl _kcc_ir_set_bb

    ldr x27, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

426 · used in 30b · defines _kcc_addr_is_iconst never used, _kcc_eval_addr never used, _kcc_eval_const_int 259b, 269, 308, 340, 407, 414, 468a, _kcc_expr_is_ice 259b, 393, _kcc_ir_build_switch 322, _kcc_ir_sw_disp never used, _kcc_ir_sw_scan never used · uses _kcc_ir_alloc_bb 303b, _kcc_ir_alloc_instr 303a, _kcc_ir_branch_to 321, _kcc_ir_build_expr 340, _kcc_ir_build_stmt 322, _kcc_ir_cur_bb 305c, _kcc_ir_emit 304, _kcc_ir_next_vreg 305a, _kcc_ir_set_bb 305b, _kcc_loop_depth 328, _kcc_loop_pop 434b, _kcc_loop_push 434b, _kcc_loop_stack 328, _kcc_sema_expr 274, _kcc_sema_resolve_type 259b, _kcc_symtab_lookup 255, _kcc_type_compatible 247a, _kcc_type_find_member 290, _kcc_type_int_s 250b, _kcc_type_layout 267, 268

End-to-end tests. The dispatch chain compares the controlling expression against each `case` constant in order and branches to the first match. Fall-through between cases follows C semantics: without a `break`, control flows into the next case body. The `default` label catches any value not matched by the cases.

⟨ *Test: codegen switch* 434a ⟩ ≡

```
echo ""
echo "--- Switch ---"

run_test "switch-case1" \
    "int main(void) { switch (1) { case 1: return
   10; case 2: return 20; } return 0; }" 10

run_test "switch-case2" \
    "int main(void) { switch (2) { case 1: return
   10; case 2: return 20; } return 0; }" 20

run_test "switch-default" \
    "int main(void) { switch (5) { case 1: return
   10; default: return 99; } return 0; }" 99

run_test "switch-no-match" \
    "int main(void) { switch (5) { case 1: return
   10; case 2: return 20; } return 0; }" 0

run_test "switch-break" \
    "int main(void) { int r = 0; switch (2) { case
    1: r = 10; break; case 2: r = 20; break; case
   3: r = 30; break; } return r; }" 20

run_test "switch-fallthrough" \
    "int main(void) { int r = 0; switch (1) { case
    1: r = r + 10; case 2: r = r + 20; case 3: r =
    r + 30; break; } return r; }" 60

run_test "switch-default-fallthrough" \
    "int main(void) { int r = 0; switch (9) { case
    1: r = 1; break; default: r = 50; break; }
   return r; }" 50

run_test "switch-var-expr" \
    "int main(void) { int x = 3; int r = 0; switch
    (x) { case 1: r = 10; break; case 3: r = 30;
   break; } return r; }" 30

run_test "switch-bare-labeled-body" \
```

```
    "int main(void) { int x = 0; switch (x) case
   0: return 7; return 1; }" 7

run_test "switch-nested-case-loop" \
    "int main(void) { int n = 3, i = 0; switch (n
   % 2) { case 0: do { i = i + 1; case 1: i = i +
   1; } while (--n > 0); } return i; }" 5

run_test "switch-case-in-block" \
    "int main(void) { int x = 1, r = 0; switch (x)
    { case 0: { r = 5; case 1: r = r + 8; } break;
    } return r; }" 8

run_test "switch-continue-in-loop" \
    "int main(void) { int s = 0, i = 0; while (i <
    5) { switch (i) { case 2: i = i + 1; continue;
    default: break; } s = s + i; i = i + 1; }
   return s; }" 8
```

□

434a · used in 33b

29.12.6. Break and Continue

The loop stack is a fixed-size array of 16 entries (supporting up to 16 levels of nesting), where each entry holds a (break_bb_id, continue_bb_id) pair. Every loop builder pushes its targets onto the stack before building the body, and pops after. _kcc_ir_build_break reads the break target from the stack top, emits IR_BR to that block, and marks the current block terminated. _kcc_ir_build_continue does the same with the continue target. A switch also pushes an entry—so that a break inside it exits the switch—but a switch is transparent to continue (§6.8.6.2): its entry therefore carries the *enclosing loop's* continue target, inherited from the entry beneath it, so a continue in a switch body correctly restarts the surrounding loop rather than branching to a non-existent target.

⟨ *IR: build break* 434b ⟩ ≡

```
// -- Loop Stack ---
// Tracks break/continue BB targets for nested
   loops.
// Max 16 nesting levels. Each entry: {break_bb_id
   , continue_bb_id} (8 bytes each).
.text

// void _kcc_loop_push(uint64_t break_bb_id,
   uint64_t continue_bb_id)
.globl _kcc_loop_push
.p2align 2
_kcc_loop_push:
    adrp x2, _kcc_loop_depth@PAGE
    add  x2, x2, _kcc_loop_depth@PAGEOFF
    ldr  x3, [x2]
    // Store break_bb at stack[depth*2]
    adrp x4, _kcc_loop_stack@PAGE
    add  x4, x4, _kcc_loop_stack@PAGEOFF
    lsl  x5, x3, #4              // depth * 16
    str  x0, [x4, x5]            // break_bb_id
    add  x5, x5, #8
    str  x1, [x4, x5]            // continue_bb_id
    add  x3, x3, #1
    str  x3, [x2]
    ret

// void _kcc_loop_pop(void)
.globl _kcc_loop_pop
.p2align 2
_kcc_loop_pop:
    adrp x0, _kcc_loop_depth@PAGE
    add  x0, x0, _kcc_loop_depth@PAGEOFF
    ldr  x1, [x0]
    sub  x1, x1, #1
    str  x1, [x0]
    ret
```

□

434b · defined at 434b, 435a · used in 30b · defines _kcc_loop_pop 423ab, 424, 426, _kcc_loop_push 423ab, 424, 426 · uses _kcc_loop_depth 328, _kcc_loop_stack 328

Break Statement. When the parser encounters a break statement inside a loop, the IR builder must emit an unconditional branch to the loop's exit basic block. _kcc_ir_build_break reads the break target from the top of the loop stack, allocates an IR_BR instruction pointing at that block, and marks the current basic block as terminated so that no further instructions are appended to it (§6.8.6.3).

⟨ *IR: build break* 435a ⟩ + ≡

```
// -- kcc_ir_build_break ---
// void kcc_ir_build_break(node *stmt)
// Emits unconditional branch to current loop's
   break target (exit BB).
.globl _kcc_ir_build_break
.p2align 2
_kcc_ir_build_break:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Get break target from top of loop stack
    adrp x0, _kcc_loop_depth@PAGE
    add  x0, x0, _kcc_loop_depth@PAGEOFF
    ldr  x0, [x0]
    sub  x0, x0, #1              // top index
    lsl  x0, x0, #4              // *16
    adrp x1, _kcc_loop_stack@PAGE
    add  x1, x1, _kcc_loop_stack@PAGEOFF
    ldr  x19, [x1, x0]           // x19 =
   break_bb_id (callee-saved)

    // Emit IR_BR to break target
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    str x19, [x0, #IR_SRC1]
    bl _kcc_ir_emit

    // Mark current BB as terminated
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

435a · defined at 434b, 435a · used in 30b · defines _kcc_ir_build_break 322 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_cur_bb 305c, _kcc_ir_emit 304, _kcc_loop_depth 328, _kcc_loop_stack 328

A continue statement jumps to the loop's condition-test basic block rather than the exit. The logic mirrors break except that it reads the continue target (offset +8) from the loop stack entry instead of the break target (offset +0).

⟨ *IR: build continue* 435b ⟩ ≡

```
// -- kcc_ir_build_continue ---
// void kcc_ir_build_continue(node *stmt)
// Emits unconditional branch to current loop's
   continue target (cond BB).
.text
.globl _kcc_ir_build_continue
.p2align 2
_kcc_ir_build_continue:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Get continue target from top of loop stack
    adrp x0, _kcc_loop_depth@PAGE
    add  x0, x0, _kcc_loop_depth@PAGEOFF
    ldr  x0, [x0]
    sub  x0, x0, #1              // top index
    lsl  x0, x0, #4              // *16
    add  x0, x0, #8              // offset to
   continue_bb_id
    adrp x1, _kcc_loop_stack@PAGE
    add  x1, x1, _kcc_loop_stack@PAGEOFF
    ldr  x19, [x1, x0]          // x19 =
   continue_bb_id (callee-saved)

    // Emit IR_BR to continue target
    mov x0, #IR_BR
    bl _kcc_ir_alloc_instr
    str x19, [x0, #IR_SRC1]
    bl _kcc_ir_emit

    // Mark current BB as terminated
    adrp x0, _kcc_ir_cur_bb@PAGE
    add  x0, x0, _kcc_ir_cur_bb@PAGEOFF
    ldr x0, [x0]
    ldr x1, [x0, #BB_FLAGS]
    orr x1, x1, #BB_FLAG_TERMINATED
    str x1, [x0, #BB_FLAGS]

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

435b · used in 30b · defines _kcc_ir_build_continue 322 · uses _kcc_ir_alloc_instr 303a, _kcc_ir_cur_bb 305c, _kcc_ir_emit 304, _kcc_loop_depth 328, _kcc_loop_stack 328

End-to-end tests: miscellaneous. The comma operator evaluates the left operand for its side effects, discards the result, and returns the right operand—the IR builder simply calls kcc_ir_build_expr on each side and returns the second vreg. The remaining tests guard boundary conditions: returning zero (ensures the epilogue does not clobber x0) and returning 255 (the maximum value representable in an 8-bit exit status on Unix).

⟨ *Test: codegen misc* 435c ⟩ ≡

```
echo ""
echo "--- Miscellaneous ---"

run_test "comma operator" \
    "int main(void) { return (1, 2, 3); }" 3

run_test "return zero" \
    "int main(void) { return 0; }" 0

run_test "large constant" \
    "int main(void) { return 255; }" 255
```

□

435c · used in 33b

Enumeration constants are rewritten to integer literals during semantic analysis, so the IR builder sees plain IR_CONST instructions. These tests verify the rewrite for file-scope enums, function-local enums, arithmetic on enumerators, and the implicit counter (RED=0, GREEN=1, etc.).

⟨ *Test: codegen enums* 435d ⟩ ≡

```
echo ""
echo "--- Enums ---"

run_test "enum file-scope" \
    "enum E { A, B, C }; int main(void) { return C
   ; }" 2

run_test "enum local" \
    "int main(void) { enum { X, Y, Z }; return X +
    Y + Z; }" 3

run_test "enum arithmetic" \
    "enum D { N, E, S, W }; int main(void) {
   return W * 10 + S; }" 32

run_test "enum in if" \
    "enum B { OFF, ON }; int main(void) { int x =
   ON; if (x == ON) return 1; return 0; }" 1
```

□

435d · used in 33b

The following test groups exercise features added during Phase 8 work on self-hosting readiness. Each group targets a specific bug that was discovered and fixed: nested struct member access, struct arrays with multi-slot elements, local array-to-pointer decay, global variable initializers, and pointer arithmetic scaling in the all-spill model.

⟨ *Test: codegen nested structs* 436a ⟩ ≡

```
echo ""
echo "--- Nested Structs ---"

run_test "nested-struct-basic" \
    "struct Inner { int x; int y; }; struct Outer
   { struct Inner a; int b; }; int main(void) {
   struct Outer o; o.a.x = 10; o.a.y = 20; o.b =
   30; return o.a.x + o.a.y + o.b; }" 60

run_test "nested-struct-second-inner" \
    "struct V { int x; int y; }; struct Box {
   struct V lo; struct V hi; }; int main(void) {
   struct Box b; b.lo.x = 1; b.lo.y = 2; b.hi.x =
   3; b.hi.y = 4; return b.hi.x + b.hi.y; }" 7

run_test "nested-struct-overwrite" \
    "struct Inner { int x; int y; }; struct Outer
   { struct Inner a; int b; }; int main(void) {
```

```
    struct Outer o; o.a.x = 5; o.a.x = 9; return o.
    a.x; }" 9

run_test "nested-struct-field-isolation" \
     "struct Inner { int x; int y; }; struct Outer
    { struct Inner a; int b; }; int main(void) {
    struct Outer o; o.a.x = 10; o.a.y = 20; o.b =
    30; return o.b; }" 30
```

□

436a · used in 33b

⟨ *Test: codegen struct arrays* 436b ⟩ ≡

```
echo ""
echo "--- Struct Arrays ---"

run_test "struct-array-basic" \
     "struct Pair { int a; int b; }; int main(void)
     { struct Pair arr[3]; arr[0].a=1; arr[0].b=2;
    arr[1].a=3; arr[1].b=4; arr[2].a=5; arr[2].b=6;
     return arr[1].a + arr[2].b; }" 9

run_test "struct-array-first-last" \
     "struct Pair { int a; int b; }; int main(void)
     { struct Pair arr[2]; arr[0].a=10; arr[0].b
    =20; arr[1].a=30; arr[1].b=40; return arr[0].a
    + arr[1].b; }" 50

run_test "struct-array-decay" \
     "struct Pair { int a; int b; }; int main(void)
     { struct Pair arr[3]; arr[0].a=1; arr[0].b=2;
    arr[1].a=3; arr[1].b=4; arr[2].a=5; arr[2].b=6;
     struct Pair *p = arr; return p[1].a + p[2].b;
    }" 9

run_test "struct-array-loop" \
     "struct V { int x; int y; }; int main(void) {
    struct V a[3]; a[0].x=1; a[1].x=2; a[2].x=3; a
    [0].y=10; a[1].y=20; a[2].y=30; int s=0; int i
    =0; while(i<3){s=s+a[i].x+a[i].y; i=i+1;}
    return s; }" 66
```

□

436b · used in 33b

⟨ *Test: codegen global init* 436c ⟩ ≡

```
echo ""
echo "--- Global Initializers ---"

run_test "global-init-simple" \
     "int x = 42; int main(void) { return x; }" 42

run_test "global-init-two" \
     "int x = 10; int y = 20; int main(void) {
    return x + y; }" 30

run_test "global-init-expr" \
     "int x = 10; int y = 20; int z; int main(void)
     { z = x + y; return z; }" 30

run_test "global-init-zero" \
     "int x; int y = 5; int main(void) { return x +
     y; }" 5

run_test "global-init-overwrite" \
     "int x = 10; int main(void) { x = 20; return x
    ; }" 20

run_test "global-init-negative" \
     "int x = 99; int main(void) { return x - 100 +
     1; }" 0

run_test "global-desig-nested" \
     "struct A{int a,b;}; struct O{struct A p; int
    z;}; struct O s={.p.a=1,.p.b=2,.z=3}; int main(
    void){ return s.p.a*100+s.p.b*10+s.z; }" 123

run_test "global-brace-elision" \
     "int m[2][2]={1,2,3,4}; int main(void){ return
     m[1][0]*10+m[1][1]; }" 34

run_test "global-array-of-struct" \
     "struct P{int x,y;}; struct P a
    [2]={{1,2},{3,4}}; int main(void){ return a[1].
    x*10+a[1].y; }" 34
```

□

436c · used in 33b

⟨ *Test: codegen ptr arithmetic* 436d ⟩ ≡

```
echo ""
echo "--- Pointer Arithmetic ---"

run_test "ptr-arith-int-scale" \
     "void *mmap(void*,long,int,int,int,long); int
    main(void) { int *p = (int *)mmap
    (0,4096,3,4098,-1,0); *p = 10; *(p + 1) = 20;
    *(p + 2) = 30; return *(p + 0) + *(p + 1) + *(p
     + 2); }" 60

run_test "ptr-arith-char-scale" \
     "void *mmap(void*,long,int,int,int,long); int
    main(void) { char *p = (char *)mmap
    (0,4096,3,4098,-1,0); *p = 1; *(p + 1) = 2; *(p
     + 2) = 3; return *p + *(p+1) + *(p+2); }" 6

run_test "ptr-arith-reverse" \
     "void *mmap(void*,long,int,int,int,long); int
    main(void) { int *p = (int *)mmap
    (0,4096,3,4098,-1,0); *(p+0)=5; *(p+1)=10; *(p
    +2)=15; return *(p+2); }" 15

run_test "ptr-arith-loop" \
     "void *mmap(void*,long,int,int,int,long); int
    main(void) { int *p = (int *)mmap
    (0,4096,3,4098,-1,0); int i=0; while(i<5){*(p+i
    )=i*i; i=i+1;} return *(p+3); }" 9
```

□

436d · used in 33b

29.12.7. Pointer Arrays

An array whose elements are themselves pointers—`int *a[N]`, and the `char **argv` idiom—exercises the boundary between the array and pointer subscript lowerings described in §29.9.4. The array variable is indexed from its alloca base to yield a slot holding a pointer, which a second dereference then follows; the top-level declared kind (`TY_ARRAY`) rather than the element kind (`TY_PTR`) determines the addressing mode. These tests cover element writes and reads, indirection through a decayed `int **` view, variable indices, and passing a pointer array to a function.

⟨ *Test: codegen pointer arrays* 437a ⟩ ≡

```
echo ""
```

```
echo "--- Pointer Arrays ---"

run_test "ptrarr-idx0" \
    "int main(void){ int a=20; int *arr[2]; arr
   [0]=&a; return *arr[0]; }" 20

run_test "ptrarr-idx1" \
    "int main(void){ int b=20; int *arr[2]; arr
   [1]=&b; return *arr[1]; }" 20

run_test "ptrarr-var-index" \
    "int main(void){ int a=7,b=8; int *arr[2]; arr
   [0]=&a; arr[1]=&b; int i=1; return *arr[i]; }"
   8

run_test "ptrarr-decayed-view" \
    "int main(void){ int a=4,b=9; int *arr[2]; arr
   [0]=&a; arr[1]=&b; int **pp=arr; return *pp[1];
    }" 9

run_test "ptrarr-param" \
    "int sum(int **pp,int n){ int i,s=0; for(i=0;i
   <n;i=i+1){ s=s+*pp[i]; } return s; } int main(
   void){ int a=1,b=2,c=3; int *arr[3]; arr[0]=&a;
   arr[1]=&b;arr[2]=&c; return sum(arr,3); }" 6

run_test "charpp-argv-like" \
    "int slen(char*s){int n=0; while(s[n]) n=n+1;
   return n;} int main(void){ char *words[2];
   words[0]=\"hi\"; words[1]=\"abc\"; char **w=
   words; return slen(w[1]); }" 3
```

□

437a · used in 33b

29.12.8. Initializer Lists

Brace-enclosed initializers seed an aggregate positionally, one value per eight-byte slot, or—with a designator—at a named position. The tests below cover explicit and implicit array bounds, a short list whose tail must zero-fill (§29.9.4 reads the same slots back), flat struct initialization, and a file-scope lookup table—the shape a self-hosted lexer's character-class tables take. Designated forms (`[i] =`, `.member =`) are exercised against `clang` in the differential suite.

⟨ *Test: codegen init lists* 437b ⟩ ≡

```
echo ""
echo "--- Initializer Lists ---"

run_test "initlist-array-explicit" \
    "int main(void){ int a[3]={10,20,30}; return a
   [0]+a[1]+a[2]; }" 60

run_test "initlist-array-implicit" \
    "int main(void){ int a[]={4,5,6,7}; return a
   [3]; }" 7

run_test "initlist-array-partial-zero" \
    "int main(void){ int a[4]={5,6}; return a[0]+a
   [1]+a[2]+a[3]; }" 11

run_test "initlist-array-index-loop" \
    "int main(void){ int a[5]={2,4,6,8,10}; int s
   =0; int i=0; while(i<5){s=s+a[i]; i=i+1;}
   return s; }" 30

run_test "initlist-struct-flat" \
    "struct P{int x;int y;}; int main(void){
   struct P p={3,4}; return p.x+p.y; }" 7

run_test "initlist-struct-partial-zero" \
    "struct P{int x;int y;int z;}; int main(void){
    struct P p={3,4}; return p.x+p.y+p.z; }" 7

run_test "initlist-global-table" \
    "int t[4]={7,8,9,10}; int main(void){ return t
   [1]+t[3]; }" 18

run_test "initlist-global-lut" \
    "int digit[10]={0,1,2,3,4,5,6,7,8,9}; int main
   (void){ int s=0; int i=0; while(i<10){s=s+digit
   [i]; i=i+1;} return s; }" 45

run_test "desig-nested-member" \
    "struct A{int a,b;}; struct O{struct A p; int
   z;}; int main(void){ struct O s={.p.a=1,.p.b
   =2,.z=3}; return s.p.a*100+s.p.b*10+s.z; }" 123

run_test "desig-array-in-struct" \
    "struct S{int arr[3]; int t;}; int main(void){
    struct S s={.arr[1]=5,.t=9}; return s.arr[0]+s
   .arr[1]*10+s.t; }" 59

run_test "desig-array-of-struct" \
    "struct P{int x,y;}; int main(void){ struct P
   a[2]={[0].x=1,[1].y=2}; return a[0].x*10+a[1].y
   ; }" 12

run_test "nested-braces-array-struct" \
    "struct P{int x,y;}; int main(void){ struct P
   a[2]={{1,2},{3,4}}; return a[1].x*10+a[1].y; }"
    34

run_test "brace-elision-2d" \
    "int main(void){ int m[2][2]={1,2,3,4}; return
    m[1][0]*10+m[1][1]; }" 34

run_test "brace-elision-struct-array" \
    "struct S{int a; int arr[2]; int b;}; int main
   (void){ struct S s={1,2,3,4}; return s.a+s.arr
   [0]+s.arr[1]+s.b; }" 10

run_test "mixed-brace-elision" \
    "int main(void){ int m[2][2]={{1,2},3,4};
   return m[1][0]*10+m[1][1]; }" 34
```

□

437b · used in 33b

29.12.9. Static Locals

A `static` local keeps its value across calls and is initialised just once. The tests exercise persistence across repeated calls, the initialise-once rule (a non-zero seed applied at program start, not on each entry), the implicit zero of an uninitialised static, a static that shadows a like-named file-scope global, and two functions whose identically-named statics must stay distinct.

⟨ *Test: codegen static locals* 438a ⟩ ≡

```
echo ""
echo "--- Static Locals ---"

run_test "static-persist" \
    "int next(void){ static int c=0; c=c+1; return
    c; } int main(void){ next(); next(); return
```

```
    next(); }" 3

run_test "static-init-once" \
    "int f(void){ static int x=10; x=x+5; return x
    ; } int main(void){ f(); return f(); }" 20

run_test "static-zero-default" \
    "int g(void){ static int n; n=n+2; return n; }
    int main(void){ g(); g(); return g(); }" 6

run_test "static-compound-assign" \
    "int acc(int v){ static int t=0; t+=v; return
    t; } int main(void){ acc(10); acc(20); return
    acc(5); }" 35

run_test "static-shadows-global" \
    "int c=100; int inc(void){ static int c=0; c=c
    +1; return c; } int main(void){ inc(); inc();
    return inc()+c; }" 103

run_test "static-two-distinct" \
    "int a(void){ static int c=0; c=c+1; return c;
    } int b(void){ static int c=50; c=c+1; return
    c; } int main(void){ b(); b(); return a()*10 +
    (b()-50); }" 13
```

□

438a · used in 33b

The final test group assembles several patterns from the self-hosting compiler into one program: arena allocation, linked list traversal, recursive tree evaluation, and multi-field struct access through pointers. Passing these tests gives high confidence that the backend can handle the patterns found in kcc itself.

⟨ *Test: codegen self-hosting* 438b ⟩ ≡

```
echo ""
echo "--- Self-hosting Patterns ---"

run_test "arena-alloc-pattern" \
    "void *mmap(void*,long,int,int,int,long); char
    *base; long pos; void init(void) { base=(char
    *)mmap(0,65536,3,4098,-1,0); pos=0; } char *
    alloc(long n) { char *p = base + pos; pos = pos
    + n; return p; } int main(void) { init(); char
    *a=alloc(8); char *b=alloc(8); return (b > a)
    ? 1 : 0; }" 1

run_test "linked-list" \
    "void *mmap(void*,long,int,int,int,long); char
    *base; long pos; void init(void) { base=(char
    *)mmap(0,65536,3,4098,-1,0); pos=0; } char *
    alloc(long n) { char *p = base + pos; pos = pos
    + n; return p; } struct Node { int val; struct
    Node *next; }; struct Node *make(int v, struct
    Node *nx) { struct Node *n = (struct Node *)
    alloc(16); n->val = v; n->next = nx; return n;
    } int sum(struct Node *n) { int s=0; while(n) {
    s=s+n->val; n=n->next; } return s; } int main(
    void) { init(); struct Node *list = make(1,
    make(2, make(3, 0))); return sum(list); }" 6

run_test "recursive-tree-eval" \
    "void *mmap(void*,long,int,int,int,long); char
    *base; long pos; void init(void) { base=(char
    *)mmap(0,65536,3,4098,-1,0); pos=0; } char *
    alloc(long n) { char *p = base + pos; pos = pos
    + n; return p; } struct N { int tag; long val;
    struct N *l; struct N *r; }; struct N *leaf(
    long v) { struct N *n=(struct N*)alloc(32); n->
    tag=1; n->val=v; n->l=0; n->r=0; return n; }
    struct N *bin(int t, struct N *a, struct N *b)
    { struct N *n=(struct N*)alloc(32); n->tag=t; n
    ->l=a; n->r=b; n->val=0; return n; } long ev(
    struct N *n) { if(n->tag==1) return n->val;
    long a=ev(n->l); long b=ev(n->r); if(n->tag==2)
    return a+b; if(n->tag==3) return a-b; if(n->
    tag==4) return a*b; return 0; } int main(void)
    { init(); struct N *r=bin(4,bin(2,leaf(2),leaf
    (3)),leaf(4)); return (int)ev(r); }" 20

run_test "string-compare" \
    "int streq(char *a, char *b, int n) { int i=0;
    while(i<n){ if(*(a+i)!=*(b+i)) return 0; i=i
    +1; } return 1; } int main(void) { char *x = \"
    hello\"; char *y = \"hello\"; char *z = \"world
    \"; return streq(x,y,5) + streq(x,z,5); }" 1

run_test "classify-char" \
    "int cls(char c) { if(c>='a' && c<='z') return
    1; if(c>='0' && c<='9') return 2; return 0; }
    int main(void) { return cls('x') + cls('3') +
    cls(' '); }" 3

run_test "func-name-builtin" \
    "int se(const char *a, const char *b){ int i
    =0; while(a[i] && a[i]==b[i]) i++; return a[i
    ]==b[i]; } int f(void){ return se(__func__, \"f
    \"); } int main(void){ return f()*10 + (se(
    __func__, \"main\") ? 2 : 0); }" 12

run_test "int128-layout" \
    "typedef unsigned __int128 u128; union h {
    u128 full; struct { unsigned long lo, hi; }; };
    int main(void){ return (int)(sizeof(u128) +
    _Alignof(u128) + sizeof(union h)); }" 48

run_test "int128-copy-abi" \
    "typedef unsigned __int128 u128; union h {
    u128 full; struct { unsigned long lo, hi; }; };
    u128 id(u128 v){ union h o; o.full = v; return
    o.full; } int main(void){ union h a; a.lo=42;
    a.hi=0; union h r; r.full = id(a.full); return
    (int)r.lo; }" 42
```

□

438b · used in 33b

Function pointers require three pieces to work together: the parser must handle grouped declarators with function suffixes (`int (*fp)(int, int)`), the IR builder must distinguish indirect calls from direct calls, and the code generator must emit `BLR` instead of `BL`. The tests below exercise local function pointer variables, function pointers as parameters, and higher-order patterns like `apply`.

⟨ *Test: codegen fnptr* 439 ⟩ ≡

```
echo ""
echo "--- Function Pointers ---"

run_test "fnptr-basic" \
    "int add(int a, int b) { return a + b; } int
    main(void) { int (*fp)(int, int) = add; return
    fp(3, 4); }" 7

run_test "fnptr-swap" \
```

```
    "int add(int a, int b) { return a + b; } int
   sub(int a, int b) { return a - b; } int main(
   void) { int (*fp)(int, int) = add; int r1 = fp
   (10, 3); fp = sub; int r2 = fp(10, 3); return
   r1 + r2; }" 20

run_test "fnptr-param" \
    "int add(int a, int b) { return a + b; } int
   apply(int (*fn)(int, int), int x, int y) {
   return fn(x, y); } int main(void) { return
   apply(add, 5, 6); }" 11

run_test "fnptr-multi-ops" \
    "int add(int a, int b) { return a + b; } int
   sub(int a, int b) { return a - b; } int mul(int
    a, int b) { return a * b; } int apply(int (*fn
   )(int, int), int x, int y) { return fn(x, y); }
    int main(void) { return apply(add, 3, 4) +
   apply(sub, 10, 3) + apply(mul, 2, 5); }" 24

run_test "fnptr-higher-order" \
    "int dbl(int x) { return x * 2; } int sq(int x
   ) { return x * x; } int mapsum(int (*f)(int),
   int n) { int s=0; int i=1; while(i<=n){s=s+f(i)
   ; i=i+1;} return s; } int main(void) { return
   mapsum(dbl, 3) + mapsum(sq, 3); }" 26

run_test "fnptr-struct-field" \
    "struct Ops { int (*f)(int, int); }; int add(
   int a, int b) { return a + b; } int main(void)
   { struct Ops o; o.f = add; return o.f(20, 22);
   }" 42

run_test "fnptr-struct-arrow" \
    "struct C { int (*op)(int); }; int dbl(int x)
   { return x * 2; } int apply(struct C *c, int v)
    { return c->op(v); } int main(void) { struct C
    c; c.op = dbl; return apply(&c, 21); }" 42
```

□

439 · used in 33b

30. IR Validation

Before the IR is handed to the code generator, a validation pass checks that structural invariants hold. With a single code-generation target this is a useful safety net; with a *second* target on the horizon (Appendix B) it becomes essential, because a bug in the shared front end and a bug in a new back end present identically—a crash deep in code generation. A verifier that rejects the malformed IR *at the seam* keeps a front-end defect from masquerading as a back-end one, and it runs between the optimizer and the back end so it also catches an optimization pass that corrupts the IR (exactly how the stale-`NOP` bug once fed a garbage opcode into register allocation and crashed it).

30.1. What the Verifier Checks

The verifier walks every function, every basic block (down the `BB_NEXT` list from the entry block), and every instruction (down the `IR_NEXT` list from `BB_FIRST`). It enforces three invariants that hold for all well-formed kcc IR and whose violation would otherwise surface as undefined back-end behaviour:

- **Valid opcodes.** Every instruction's `IR_OP` lies in the defined opcode range $[1, \texttt{IR_STRING_LIT}]$. A zero or out-of-range opcode means an uninitialised or corrupted instruction—the shape of the stale-`NOP` corruption—which the instruction selector would then dispatch on as if it were real.
- **Terminated blocks.** Every non-empty block carries the `BB_FLAG_TERMINATED` flag, i.e. it was closed with a branch or a return. An unterminated block would let the code generator fall through into the next block's body, producing incorrect machine code.
- **Finite instruction lists.** The walk down each block is bounded; a self-referential or cyclic `IR_NEXT` link (which would hang both the verifier and every later pass) is reported rather than spun on.

On the first violation the verifier prints an internal-error diagnostic naming the offending function and returns non-zero; the driver treats that as a compilation failure. Because every program in the test corpus, plus Lua and zlib, passes cleanly, the check is a pure safety net: it never rejects valid IR, only genuinely malformed IR.

Deliberately *not* checked here are dataflow invariants that need dominance analysis (strict def-before-use across control flow) or that do not hold in kcc's pre-register-allocation IR (single static assignment); those would risk false rejections and belong to a later, dataflow-aware pass.

30.2. The Verifier

⟨ *IR: validate* 441 ⟩ ≡

```
// -- kcc_ir_validate ---
// uint64_t kcc_ir_validate(void)
// Structural check of every IR function: valid
   opcodes, terminated blocks,
// and finite (acyclic) instruction lists.
   Returns 0 on success, 1 on the
// first violation (after printing an internal-
   error diagnostic).
.text
.globl _kcc_ir_validate
.p2align 2
_kcc_ir_validate:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]

    adrp x0, _kcc_ir_funcs@PAGE
    add  x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x19, [x0]
    cbz x19, .Lirv_ok
    ldr x20, [x19, #VEC_COUNT]
    mov x21, #0

.Lirv_func_loop:
    cmp x21, x20
    b.ge .Lirv_ok
    mov x0, x19
    mov x1, x21
    bl _kcc_vec_get
    ldr x22, [x0]                 // function
   record

    ldr x23, [x22, #IRF_ENTRY]    // first basic
   block
.Lirv_blk_loop:
    cbz x23, .Lirv_next_func

    ldr x24, [x23, #BB_FIRST]     // first
   instruction
    mov x25, #0                   // instruction
   counter (cycle bound)
.Lirv_ins_loop:
    cbz x24, .Lirv_ins_done
    add x25, x25, #1
    mov x26, #0x100000            // 1M-instruction
    ceiling per block
    cmp x25, x26
    b.hi .Lirv_fail_loop
    ldr x0, [x24, #IR_OP]
    cbz x0, .Lirv_fail_op         // opcode 0 =
   uninitialised
    cmp x0, #IR_ASM               // highest
   defined opcode
    b.hi .Lirv_fail_op
    ldr x24, [x24, #IR_NEXT]
    b .Lirv_ins_loop
.Lirv_ins_done:
    // A non-empty block must have been terminated
    (branch or return).
    ldr x0, [x23, #BB_FIRST]
    cbz x0, .Lirv_blk_next
    ldr x0, [x23, #BB_FLAGS]
    tst x0, #BB_FLAG_TERMINATED
    b.eq .Lirv_fail_term
.Lirv_blk_next:
    ldr x23, [x23, #BB_NEXT]
    b .Lirv_blk_loop

.Lirv_next_func:
    add x21, x21, #1
```

```
    b .Lirv_func_loop

.Lirv_ok:
    mov x0, #0
    b .Lirv_ret

.Lirv_fail_op:
    adrp x0, .Lirv_msg_op@PAGE
    add  x0, x0, .Lirv_msg_op@PAGEOFF
    bl _kcc_io_puts
    b .Lirv_fail_name
.Lirv_fail_term:
    adrp x0, .Lirv_msg_term@PAGE
    add  x0, x0, .Lirv_msg_term@PAGEOFF
    bl _kcc_io_puts
    b .Lirv_fail_name
.Lirv_fail_loop:
    adrp x0, .Lirv_msg_loop@PAGE
    add  x0, x0, .Lirv_msg_loop@PAGEOFF
    bl _kcc_io_puts
.Lirv_fail_name:
    ldr x0, [x22, #IRF_NAME]
    cbz x0, .Lirv_fail_nl
    bl _kcc_io_puts
.Lirv_fail_nl:
    adrp x0, .Lirv_msg_nl@PAGE
    add  x0, x0, .Lirv_msg_nl@PAGEOFF
    bl _kcc_io_puts
    mov x0, #1

.Lirv_ret:
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret

.section __TEXT,__cstring,cstring_literals
.Lirv_msg_op:
    .asciz "kcc: internal error: invalid IR opcode
    in function "
.Lirv_msg_term:
    .asciz "kcc: internal error: unterminated
   basic block in function "
.Lirv_msg_loop:
    .asciz "kcc: internal error: cyclic
   instruction list in function "
.Lirv_msg_nl:
    .asciz "\n"
.text
```

□

441 · used in 30b · defines _kcc_ir_validate 603 · uses _kcc_io_puts 70a, _kcc_ir_funcs 305c, _kcc_vec_get 65b

Part VII.

Optimization

The IR produced by the builder is correct but naïve: it loads and stores every temporary through a stack slot, and it faithfully translates constant sub-expressions that could be folded at compile time. This part applies two classical optimisation passes over the IR before code generation. The pass manager (Chapter 31) iterates over all functions and drives the individual passes. Constant folding and peephole simplification (Chapter 32) evaluate arithmetic on constants at compile time and replace redundant instruction patterns. Dead code elimination (Chapter 33) uses a mark-and-sweep algorithm over the use–def chains to remove instructions whose results are never consumed. Together these passes eliminate the most obvious waste without introducing the complexity of a full register allocator or loop optimiser.

31. Optimization Passes

Generating correct IR is necessary, but the initial translation from AST to IR often leaves redundancy—constant expressions that could be evaluated at compile time, subexpressions and loads computed more than once, and instructions whose results are never consumed. This chapter provides the pass manager that drives optimization. It iterates over every IR function and runs the available passes—constant folding, strength reduction, common subexpression elimination, copy/constant propagation, and dead code elimination—in a loop until no further changes occur, achieving a fixed-point result. Running them together matters: folding a cast can expose a common subexpression, strength-reducing a multiply into a shift can expose a fresh constant, forwarding a stored value to a load can leave that load dead, eliminating a subexpression can leave another load dead, and each pass feeds the next until the IR settles. Every pass is a pure IR-to-IR transform, so it lives above the code generator and a second target inherits all of them unchanged.

31.1. Pass Manager

The pass manager runs optimization passes over each IR function until a fixed point is reached—that is, until no pass reports any changes. Each pass is a function that takes an IR function pointer and returns non-zero if it modified something. The outer loop iterates all functions in the module; the inner loop repeats the pass sequence until stable.

⟨ *Opt: pass manager* 445a ⟩ ≡

```
// -- kcc_opt_run ---
// void kcc_opt_run(void)
// Runs optimization passes on all IR functions
   until stable.
.text
.globl _kcc_opt_run
.p2align 2
_kcc_opt_run:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Load function list
    adrp x0, _kcc_ir_funcs@PAGE
    add  x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x19, [x0]
    cbz x19, .Lpr_done

    ldr x20, [x19, #VEC_COUNT]
    mov x21, #0

.Lpr_func_loop:
    cmp x21, x20
    b.ge .Lpr_done

    mov x0, x19
    mov x1, x21
    bl _kcc_vec_get
    ldr x22, [x0]                // func ptr

    // Run passes on this function
    mov x0, x22
    bl _kcc_opt_run_func

    add x21, x21, #1
    b .Lpr_func_loop

.Lpr_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

445a · defined at 445ab · used in 30c · defines _kcc_opt_run 603, 623 · uses _kcc_ir_funcs 305c, _kcc_opt_run_func 445b, _kcc_vec_get 65b

31.2. Per-Function Iteration

The per-function driver calls each registered pass and tracks whether any modification occurred. When a pass fires, the changed flag resets the fixed-point loop so all passes run again on the modified IR.

⟨ *Opt: pass manager* 445b ⟩ + ≡

```
// -- kcc_opt_run_func ---
// void kcc_opt_run_func(ir_func *func)
// Iterates constfold + cse + dce until no changes
   .
.text
.globl _kcc_opt_run_func
.p2align 2
_kcc_opt_run_func:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    mov x19, x0                  // func
    mov x20, #0                  // iteration
   counter (safety limit)

.Lprf_loop:
    cmp x20, #10                 // max 10
   iterations
```

```
    b.ge .Lprf_done

    // Run constant folding
    mov x0, x19
    bl _kcc_opt_constfold
    mov x1, x0                  // fold count

    // Strength reduction / algebraic
simplification (mul/div/mod by a power
    // of two -> shift/mask).  Runs after folding
so a constant operand it
    // relies on is already materialised, and
before CSE/DCE so the shift it
    // introduces is available for their
bookkeeping.
    mov x0, x19
    bl _kcc_opt_strength
    add x1, x1, x0              // + strength
count

    // Run common subexpression elimination
    mov x0, x19
    bl _kcc_opt_cse
    add x1, x1, x0              // + cse count

    // Copy / constant propagation (store-to-load
forwarding); DCE below then
    // sweeps the loads it makes dead.
    mov x0, x19
    bl _kcc_opt_copyprop
    add x1, x1, x0              // + copyprop count

    // Run DCE
    mov x0, x19
    bl _kcc_opt_dce
    add x0, x0, x1              // total changes

    // If no changes, we're done
    cbz x0, .Lprf_done

    add x20, x20, #1
    b .Lprf_loop

.Lprf_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

445b · defined at 445ab · used in 30c · defines _kcc_opt_run_func 445a · uses _kcc_opt_constfold 447, _kcc_opt_copyprop 457, _kcc_opt_cse 457, _kcc_opt_dce 453a

32. Peephole Optimization

Peephole optimizations examine a small window of instructions and replace patterns that can be simplified without global analysis. This chapter implements constant folding on the IR: an instruction whose inputs are all compile-time constants is evaluated at compile time and replaced with a single `IR_CONST`. Three shapes fold. A *binary* operation with two constant operands (`2 * 3`) evaluates directly. A *unary* `NEG` or `NOT` of a constant folds likewise—and although a bare 64-bit complement of `~0u` is not the 32-bit `0xFFFFFFFF` the source means, the front end appends a width cast after every such operation, and that cast folds in turn, narrowing the value to the correct width. That is the third and load-bearing shape: a *width cast*—the `SEXT`/`ZEXT`/`TRUNC` inserted on nearly every integer operation—folds when its operand is constant. Without it a constant chain stalls at the first cast (folding `3*4` but not the surrounding `2 + …− 1`, because the intervening sign-extend hid the intermediate constant), so folding through casts is what lets a whole constant expression collapse to one value. All three feed the fixed-point loop of Chapter 31, so each fold that produces a fresh constant can unblock the next.

32.1. Constant Folding

Constant folding is the classic peephole optimization: when both operands of a binary instruction are compile-time constants, we evaluate the operation at compile time and replace the instruction with a single `IR_CONST`. This eliminates work at run time and, equally important, feeds additional constants into later iterations—an `IR_ADD` whose result becomes constant may enable a downstream `IR_MUL` to fold as well.

The implementation works in two linear passes over the function's basic blocks. Pass 1 scans every instruction and records which vregs were produced by `IR_CONST`; pass 2 revisits every binary operation and, when both source vregs appear in the constant map, computes the result and rewrites the instruction in place. A dense array indexed by vreg number stores the known constant values, with a parallel valid array so we can distinguish "not a constant" from "constant zero."

32.1.1. Building the Constant Map

Before folding can begin we need a fast lookup from vreg number to constant value. We allocate two arena arrays sized to the function's vreg count: `values[]` holds the immediate and `valid[]` records whether a mapping exists. Pass 1 then walks every instruction; for each `IR_CONST` it finds, it stores the immediate in the corresponding slot and marks it valid.

⟨ *Opt: constant fold* 447 ⟩ ≡

```
// -- kcc_opt_constfold ---
// uint64_t kcc_opt_constfold(ir_func *func)
// Folds constant expressions.  Returns number of
   folds performed.
.text
.globl _kcc_opt_constfold
.p2align 2
_kcc_opt_constfold:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str x27, [sp, #80]

    mov x19, x0                 // func
    mov x27, #0                 // fold count

    // Allocate const value map: array of (valid,
   value) pairs
    // Max vregs = func.vreg_n
    ldr x20, [x19, #IRF_VREG_N]
    cbz x20, .Lcf_done
    add x20, x20, #1            // +1 for safety

    // Allocate arrays: values[vreg_n] and valid[
   vreg_n]
    lsl x0, x20, #3             // values: 8 bytes
    each
    bl _kcc_arena_alloc
    mov x21, x0                 // x21 = values
   array

    mov x0, x20                 // valid: 1 byte
   each (we'll use 8)
    lsl x0, x0, #3
    bl _kcc_arena_alloc
    mov x22, x0                 // x22 = valid
   array

    // Zero valid array
    mov x0, x22
    mov x1, #0
    lsl x2, x20, #3
    bl _kcc_memset

    // Pass 1: collect all IR_CONST → record in
   map
    ldr x0, [x19, #IRF_BLOCKS]
    cbz x0, .Lcf_done
    mov x23, x0                 // blocks vec
    ldr x24, [x23, #VEC_COUNT]
    mov x25, #0

.Lcf_collect_bb:
    cmp x25, x24
    b.ge .Lcf_pass2

    mov x0, x23
    mov x1, x25
    bl _kcc_vec_get
    ldr x26, [x0]               // BB ptr

    ldr x0, [x26, #BB_FIRST]
.Lcf_collect_instr:
    cbz x0, .Lcf_collect_next_bb
    mov x26, x0                 // current instr

    ldr x1, [x26, #IR_OP]
    cmp x1, #IR_CONST
    b.ne .Lcf_collect_skip
```

```
    // Record: valid[dest] = 1, values[dest] =
   src1
    ldr x1, [x26, #IR_DEST]
    cmp x1, x20
    b.ge .Lcf_collect_skip
    mov x2, #1
    str x2, [x22, x1, lsl #3]
    ldr x2, [x26, #IR_SRC1]
    str x2, [x21, x1, lsl #3]

.Lcf_collect_skip:
    ldr x0, [x26, #IR_NEXT]
    b .Lcf_collect_instr

.Lcf_collect_next_bb:
    add x25, x25, #1
    b .Lcf_collect_bb
```

□

447 · defined at 447, 448, 449 · used in 30c · defines _kcc_opt_constfold 445b · uses _kcc_arena_alloc 58a, _kcc_vec_get 65b

32.1.2. Folding Binary Operations

With the constant map in hand, pass 2 walks every instruction a second time. For each binary operation (arithmetic, bitwise, shift, or comparison) it checks whether both source vregs have entries in the map. When they do, the two constant values are loaded and the appropriate operation is dispatched. Non-binary opcodes and operations with at least one non-constant source are skipped untouched.

⟨ *Opt: constant fold* 448 ⟩ + ≡

```
    // Pass 2: fold binary ops with two const
   sources
.Lcf_pass2:
    mov x25, #0

.Lcf_fold_bb:
    cmp x25, x24
    b.ge .Lcf_done

    mov x0, x23
    mov x1, x25
    bl _kcc_vec_get
    ldr x26, [x0]

    ldr x0, [x26, #BB_FIRST]
.Lcf_fold_instr:
    cbz x0, .Lcf_fold_next_bb
    mov x26, x0

    // A floating operation must never be folded
   with integer arithmetic: the
    // fold below negates/adds the raw 64-bit
   pattern, which is meaningless for a
    // double.  Skip any FP-typed instruction and
   let the back end materialise
    // the constant and compute with FADD/FNEG at
   run time.
    ldr x1, [x26, #IR_TYPE]
    cmp x1, #0x1000
    b.lo .Lcf_check_op
    ldr x2, [x1, #TY_KIND]
    cmp x2, #8                  // FLOAT
    b.lo .Lcf_check_op
    cmp x2, #10                 // ..LDOUBLE
    b.ls .Lcf_fold_skip
.Lcf_check_op:
    ldr x1, [x26, #IR_OP]
    // Width casts of a constant fold too: the
   front end inserts SEXT/ZEXT/TRUNC
    // on nearly every integer operation, and
   without folding through them a
    // constant chain stalls at the first cast (e.
   g. 2+3*4 folds 3*4 but the
    // sign-extend then blocks the add).
    cmp x1, #IR_SEXT
    b.eq .Lcf_ext
    cmp x1, #IR_ZEXT
    b.eq .Lcf_ext
    cmp x1, #IR_TRUNC
    b.eq .Lcf_ext
    // Check if it's a foldable binary op (1-7,
   10-15, 20-25)
    cmp x1, #IR_ADD
    b.lt .Lcf_fold_skip
    cmp x1, #IR_CMP_SGE
    b.gt .Lcf_fold_skip
    // Unary NEG/NOT of a constant fold to a
   constant too; the width
    // normalization the front end appends is a
   cast that folds in turn, so the
    // 64-bit result is narrowed to the right
   width by the next iteration.
    cmp x1, #IR_NEG
    b.eq .Lcf_unary
    cmp x1, #IR_NOT
    b.eq .Lcf_unary

    // Check both sources are const
    ldr x2, [x26, #IR_SRC1]
    cmp x2, x20
    b.ge .Lcf_fold_skip
    ldr x3, [x22, x2, lsl #3]   // valid[src1]?
    cbz x3, .Lcf_fold_skip

    ldr x3, [x26, #IR_SRC2]
    cmp x3, x20
    b.ge .Lcf_fold_skip
    ldr x4, [x22, x3, lsl #3]   // valid[src2]?
    cbz x4, .Lcf_fold_skip

    // Both const! Get values
    ldr x2, [x26, #IR_SRC1]
    ldr x2, [x21, x2, lsl #3]   // val1
    ldr x3, [x26, #IR_SRC2]
    ldr x3, [x21, x3, lsl #3]   // val2

    // Compute result based on opcode
    ldr x1, [x26, #IR_OP]
    cmp x1, #IR_ADD
    b.eq .Lcf_do_add
    cmp x1, #IR_SUB
    b.eq .Lcf_do_sub
    cmp x1, #IR_MUL
    b.eq .Lcf_do_mul
    cmp x1, #IR_SDIV
    b.eq .Lcf_do_sdiv
    cmp x1, #IR_AND
    b.eq .Lcf_do_and
    cmp x1, #IR_OR
    b.eq .Lcf_do_or
    cmp x1, #IR_XOR
    b.eq .Lcf_do_xor
    cmp x1, #IR_SHL
    b.eq .Lcf_do_shl
    cmp x1, #IR_ASHR
    b.eq .Lcf_do_ashr
    cmp x1, #IR_CMP_EQ
    b.eq .Lcf_do_eq
    cmp x1, #IR_CMP_NE
    b.eq .Lcf_do_ne
    cmp x1, #IR_CMP_SLT
    b.eq .Lcf_do_slt
```

```
    cmp x1, #IR_CMP_SGT
    b.eq .Lcf_do_sgt
    b .Lcf_fold_skip
```

□

448 · defined at 447, 448, 449 · used in 30c · uses _kcc_vec_get 65b

32.1.3. Arithmetic Dispatch

Each foldable opcode maps to a short straight-line sequence that computes the result in register x4 from the two constant operands in x2 and x3. After the computation, control falls through to the replacement logic, which rewrites the original instruction as an IR_CONST carrying the folded value, updates the constant map (so cascading folds work within the same pass), and increments the fold counter.

⟨ *Opt: constant fold* 449 ⟩ + ≡

```
.Lcf_do_add:
    add x4, x2, x3
    b .Lcf_replace
.Lcf_do_sub:
    sub x4, x2, x3
    b .Lcf_replace
.Lcf_do_mul:
    mul x4, x2, x3
    b .Lcf_replace
.Lcf_do_sdiv:
    cbz x3, .Lcf_fold_skip      // div by zero: skip
    sdiv x4, x2, x3
    b .Lcf_replace
.Lcf_do_and:
    and x4, x2, x3
    b .Lcf_replace
.Lcf_do_or:
    orr x4, x2, x3
    b .Lcf_replace
.Lcf_do_xor:
    eor x4, x2, x3
    b .Lcf_replace
.Lcf_do_shl:
    lsl x4, x2, x3
    b .Lcf_replace
.Lcf_do_ashr:
    asr x4, x2, x3
    b .Lcf_replace
.Lcf_do_eq:
    cmp x2, x3
    cset x4, eq
    b .Lcf_replace
.Lcf_do_ne:
    cmp x2, x3
    cset x4, ne
    b .Lcf_replace
.Lcf_do_slt:
    cmp x2, x3
    cset x4, lt
    b .Lcf_replace
.Lcf_do_sgt:
    cmp x2, x3
    cset x4, gt
    b .Lcf_replace

.Lcf_unary:
    ldr x2, [x26, #IR_SRC1]
    cmp x2, x20
    b.ge .Lcf_fold_skip
    ldr x3, [x22, x2, lsl #3]
    cbz x3, .Lcf_fold_skip
    ldr x2, [x21, x2, lsl #3]
    ldr x1, [x26, #IR_OP]
    cmp x1, #IR_NEG
    b.eq .Lcf_un_neg
    mvn x4, x2
    b .Lcf_replace
.Lcf_un_neg:
    neg x4, x2
    b .Lcf_replace

.Lcf_ext:
    // Fold a width cast of a constant.  SEXT/ZEXT read the source width from
    // SRC2; TRUNC reads the target width from its result type.  The folded
    // value is the constant sign- or zero-adjusted to that width.
    ldr x2, [x26, #IR_SRC1]
    cmp x2, x20
    b.ge .Lcf_fold_skip
    ldr x3, [x22, x2, lsl #3]    // valid[src1]?
    cbz x3, .Lcf_fold_skip
    ldr x2, [x21, x2, lsl #3]    // val
    ldr x1, [x26, #IR_OP]
    cmp x1, #IR_TRUNC
    b.eq .Lcf_ext_trunc
    ldr x5, [x26, #IR_SRC2]      // source width (bytes)
    cbz x5, .Lcf_fold_skip
    cmp x5, #8
    b.hs .Lcf_ext_id
    cmp x1, #IR_ZEXT
    b.eq .Lcf_ext_mask
    // SEXT: sign-extend from width*8 bits
    lsl x6, x5, #3
    mov x7, #64
    sub x7, x7, x6
    lsl x4, x2, x7
    asr x4, x4, x7
    b .Lcf_replace
.Lcf_ext_trunc:
    ldr x5, [x26, #IR_TYPE]
    cbz x5, .Lcf_fold_skip
    ldr x5, [x5, #TY_SIZE]       // target width
    cbz x5, .Lcf_fold_skip
    cmp x5, #8
    b.hs .Lcf_ext_id
.Lcf_ext_mask:
    // ZEXT / TRUNC: keep the low width*8 bits, zero the rest
    lsl x6, x5, #3
    mov x7, #1
    lsl x7, x7, x6
    sub x7, x7, #1
    and x4, x2, x7
    b .Lcf_replace
.Lcf_ext_id:
    mov x4, x2                   // width >= 8: value unchanged

.Lcf_replace:
    // Replace instruction with IR_CONST
    mov x1, #IR_CONST
    str x1, [x26, #IR_OP]
    str x4, [x26, #IR_SRC1]      // folded value
    str xzr, [x26, #IR_SRC2]

    // Update const map
    ldr x1, [x26, #IR_DEST]
    cmp x1, x20
    b.ge .Lcf_fold_count
    mov x2, #1
    str x2, [x22, x1, lsl #3]
    str x4, [x21, x1, lsl #3]

.Lcf_fold_count:
```

```
    add x27, x27, #1

.Lcf_fold_skip:
    ldr x0, [x26, #IR_NEXT]
    b .Lcf_fold_instr

.Lcf_fold_next_bb:
    add x25, x25, #1
    b .Lcf_fold_bb

.Lcf_done:
    mov x0, x27                    // return fold
   count

    ldr x27, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

449 · defined at 447, 448, 449 · used in 30c

32.2. Strength Reduction and Algebraic Simplification

Constant folding collapses expressions whose operands are *all* constant. A second, complementary peephole handles the common case where only *one* operand is constant: a multiply, unsigned divide, or unsigned remainder by a power of two, which the target lowers far more cheaply as a shift or a mask. Because these rewrites are pure IR-to-IR transforms they belong above the code generator, so every back end inherits them. `_kcc_opt_strength` reuses the constant map built exactly as in constant folding (a values array and a parallel valid array indexed by vreg), then walks each block a second time and rewrites:

- `x * 2^k` → `x << k`. Correct for both signed and unsigned operands: two's-complement multiplication and a left shift agree bit-for-bit, overflow included.
- `x /u 2^k` → `x >>u k` (logical), for *unsigned* divide only. A signed `SDIV` rounds toward zero while an arithmetic shift rounds toward $-\infty$, so `SDIV`/`SMOD` are left untouched.
- `x %u 2^k` → `x & (2^k-1)`, unsigned only.
- `x * 0` and `x & 0` → the constant 0, rewritten in place like a fold.

A shift or mask needs a fresh constant for its amount, so the pass allocates an `IR_CONST` (drawing a new vreg straight from the function's `IRF_VREG_N` counter) and splices it into the block's instruction list immediately before the rewritten instruction—which is why the walk tracks the previous instruction. Floating-point instructions are skipped outright: a `double` multiply carries an `IR_MUL` whose constant operand is an IEEE-754 bit pattern that can *look* like a power of two (`2.0` is 0x4000…0), and turning that into a shift would be catastrophic. The pass feeds the same fixed-point loop as the other peepholes, and its rewrites are behaviour-preserving, so the differential corpus, Lua, and zlib are the oracle rather than byte-identical output.

⟨ *Opt: strength reduction* 450 ⟩ ≡

```
// -- kcc_opt_strength ---
// uint64_t kcc_opt_strength(ir_func *func) ->
   number of rewrites.
// Strength reduction + algebraic simplification,
   all behaviour-preserving:
//   x * 2^k   -> x << k          (signed &
   unsigned)
//   x /u 2^k  -> x >>u k         (UDIV only; SDIV
    rounds differently)
//   x %u 2^k  -> x & (2^k - 1)   (UMOD only)
//   x * 0, x & 0 -> const 0
.text
.globl _kcc_opt_strength
.p2align 2
_kcc_opt_strength:
    stp x29, x30, [sp, #-112]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    stp x27, x28, [sp, #80]

    mov x19, x0                  // func
    mov x23, #0                  // rewrite count

    ldr x20, [x19, #IRF_VREG_N]
    cbz x20, .Lsr_done
    add x20, x20, #1             // map size (const
    map covers old vregs only)

    lsl x0, x20, #3
    bl _kcc_arena_alloc
    mov x21, x0                  // values[]
    mov x0, x20
    lsl x0, x0, #3
    bl _kcc_arena_alloc
    mov x22, x0                  // valid[]
    mov x0, x22
    mov x1, #0
    lsl x2, x20, #3
    bl _kcc_memset

    // Pass 1: record every IR_CONST's (dest ->
   value).
    ldr x24, [x19, #IRF_ENTRY]
.Lsr_col_bb:
    cbz x24, .Lsr_pass2
    ldr x0, [x24, #BB_FIRST]
.Lsr_col_i:
    cbz x0, .Lsr_col_nb
    ldr x1, [x0, #IR_OP]
    cmp x1, #IR_CONST
    b.ne .Lsr_col_sk
    ldr x1, [x0, #IR_DEST]
    cmp x1, x20
    b.ge .Lsr_col_sk
    mov x2, #1
    str x2, [x22, x1, lsl #3]
    ldr x2, [x0, #IR_SRC1]
    str x2, [x21, x1, lsl #3]
.Lsr_col_sk:
    ldr x0, [x0, #IR_NEXT]
    b .Lsr_col_i
.Lsr_col_nb:
    ldr x24, [x24, #BB_NEXT]
```

```
    b .Lsr_col_bb

.Lsr_pass2:
    ldr x24, [x19, #IRF_ENTRY]
.Lsr_bb:
    cbz x24, .Lsr_done
    mov x26, #0                 // prev = 0
    ldr x25, [x24, #BB_FIRST]   // cur
.Lsr_i:
    cbz x25, .Lsr_nb

    // Skip floating-point instructions: a
   constant FP operand is a raw
    // IEEE-754 pattern, not an integer power of
   two.
    ldr x1, [x25, #IR_TYPE]
    cmp x1, #0x1000
    b.lo .Lsr_intop
    ldr x2, [x1, #TY_KIND]
    cmp x2, #8
    b.lo .Lsr_intop
    cmp x2, #10
    b.ls .Lsr_next
.Lsr_intop:
    ldr x9, [x25, #IR_OP]
    cmp x9, #IR_MUL
    b.eq .Lsr_mul
    cmp x9, #IR_UDIV
    b.eq .Lsr_udiv
    cmp x9, #IR_UMOD
    b.eq .Lsr_umod
    b .Lsr_next

.Lsr_mul:
    // Multiplication is commutative: try SRC2
   first, then SRC1.
    ldr x10, [x25, #IR_SRC2]
    bl _kcc_opt_sr_cval         // x0 = const
   value if SRC2 const, else marker
    cbz x0, .Lsr_mul_s1
    mov x12, x1                 // C
    ldr x27, [x25, #IR_SRC1]    // x operand
    b .Lsr_mul_have
.Lsr_mul_s1:
    ldr x10, [x25, #IR_SRC1]
    bl _kcc_opt_sr_cval
    cbz x0, .Lsr_next
    mov x12, x1                 // C
    ldr x27, [x25, #IR_SRC2]    // x operand
.Lsr_mul_have:
    cbz x12, .Lsr_to_zero       // x * 0 -> 0
    cmp x12, #2
    b.lt .Lsr_next              // C == 1 (
   identity) or negative: leave alone
    sub x14, x12, #1
    tst x12, x14
    b.ne .Lsr_next              // not a power of
   two
    clz x15, x12
    mov x0, #63
    sub x28, x0, x15            // amount = log2(C
   )
    mov x9, #IR_SHL
    b .Lsr_emit

.Lsr_udiv:
    ldr x10, [x25, #IR_SRC2]    // divisor (not
   commutative)
    bl _kcc_opt_sr_cval
    cbz x0, .Lsr_next
    mov x12, x1
    cmp x12, #2
    b.lt .Lsr_next              // /1 identity or
   /0 undefined
    sub x14, x12, #1
    tst x12, x14
    b.ne .Lsr_next
    clz x15, x12
    mov x0, #63
    sub x28, x0, x15            // amount = log2(C
   )
    ldr x27, [x25, #IR_SRC1]
    mov x9, #IR_LSHR
    b .Lsr_emit

.Lsr_umod:
    ldr x10, [x25, #IR_SRC2]
    bl _kcc_opt_sr_cval
    cbz x0, .Lsr_next
    mov x12, x1
    cmp x12, #1
    b.lt .Lsr_next              // %0 undefined
    sub x14, x12, #1
    tst x12, x14
    b.ne .Lsr_next              // C=1 passes (mask
   0 -> x%1 == 0)
    sub x28, x12, #1            // mask = C - 1
    ldr x27, [x25, #IR_SRC1]
    mov x9, #IR_AND
    b .Lsr_emit

.Lsr_to_zero:
    mov x0, #IR_CONST
    str x0, [x25, #IR_OP]
    str xzr, [x25, #IR_SRC1]
    str xzr, [x25, #IR_SRC2]
    add x23, x23, #1
    b .Lsr_next

.Lsr_emit:
    // x27 = value operand vreg, x28 = shift/mask
   amount, x9 = new opcode.
    // Splice a fresh IR_CONST(amount) before cur,
    then rewrite cur.
    str x9, [sp, #96]           // stash new
   opcode across calls
    mov x0, #IR_CONST
    bl _kcc_ir_alloc_instr
    str x0, [sp, #104]          // stash const
   instr
    adrp x1, _kcc_type_int_s@PAGE
    add  x1, x1, _kcc_type_int_s@PAGEOFF
    ldr x1, [x1]
    str x1, [x0, #IR_TYPE]
    str x28, [x0, #IR_SRC1]     // the amount
    str xzr, [x0, #IR_SRC2]
    // Fresh vreg straight from this function's
   counter.
    ldr x1, [x19, #IRF_VREG_N]
    add x2, x1, #1
    str x2, [x19, #IRF_VREG_N]
    ldr x0, [sp, #104]
    str x1, [x0, #IR_DEST]      // const.dest =
   new vreg
    mov x11, x1                 // new vreg number
    str x25, [x0, #IR_NEXT]     // const.next =
   cur
    cbz x26, .Lsr_emit_head
    str x0, [x26, #IR_NEXT]     // prev.next =
   const
    b .Lsr_emit_rw
.Lsr_emit_head:
    str x0, [x24, #BB_FIRST]    // block head =
   const
.Lsr_emit_rw:
    ldr x9, [sp, #96]
    str x9, [x25, #IR_OP]       // cur becomes SHL
   /LSHR/AND
```

```
    str x27, [x25, #IR_SRC1]     // value operand
    str x11, [x25, #IR_SRC2]     // amount vreg
    add x23, x23, #1

.Lsr_next:
    mov x26, x25                 // prev = cur
    ldr x25, [x25, #IR_NEXT]     // cur = cur.next
    b .Lsr_i
.Lsr_nb:
    ldr x24, [x24, #BB_NEXT]
    b .Lsr_bb

.Lsr_done:
    mov x0, x23
    ldp x27, x28, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #112
    ret
```

```
// -- kcc_opt_sr_cval ---
// Helper: given a vreg in x10 and the const map (
   x20 size, x21 values,
// x22 valid), return x0 = 1 and x1 = value if the
    vreg holds a known
// constant, else x0 = 0.  Leaf; clobbers x0-x3
   only.
.p2align 2
_kcc_opt_sr_cval:
    cmp x10, x20
    b.ge .Lsrc_no
    ldr x2, [x22, x10, lsl #3]
    cbz x2, .Lsrc_no
    ldr x1, [x21, x10, lsl #3]
    mov x0, #1
    ret
.Lsrc_no:
    mov x0, #0
    ret
```

□

450 · used in 30c · uses _kcc_arena_alloc 58a, _kcc_ir_alloc_instr 303a, _kcc_type_int_s 250b

33. Dead Code Elimination

After constant folding and identity elimination have simplified the IR, some instructions may no longer contribute to the program's output—their destination vregs are never referenced by any subsequent instruction. This chapter removes such dead code through a mark-and-sweep algorithm. The mark pass records every vreg that appears as a source operand; the sweep pass deletes any instruction whose destination vreg was not marked, provided the instruction has no side effects (stores, calls, branches, and returns are always preserved).

33.1. Dead Code Elimination

DCE removes instructions whose results are never used. It works in two passes: a *mark* pass walks every instruction and records which vregs are live (referenced as sources), and a *sweep* pass deletes any instruction whose destination vreg was not marked. Instructions with side effects—stores, calls, branches, returns—are never deleted regardless of whether their result is used.

33.1.1. Setup and Bitmap Allocation

The function begins by allocating a dense used-vreg bitmap sized to the function's vreg count. Each slot is eight bytes wide (one machine word) for alignment convenience; a nonzero value means the vreg is referenced somewhere in the function. The bitmap is zeroed before the mark pass begins.

⟨ *Opt: dead code elimination* 453a ⟩ ≡

```
// -- kcc_opt_dce ---
// uint64_t kcc_opt_dce(ir_func *func)
// Eliminates dead instructions.  Returns number
   of eliminations.
.text
.globl _kcc_opt_dce
.p2align 2
_kcc_opt_dce:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str x27, [sp, #80]

    mov x19, x0                 // func
    mov x27, #0                 // elimination
   count

    ldr x20, [x19, #IRF_VREG_N]
    cbz x20, .Ldce_done
    add x20, x20, #1

    // Allocate used bitmap (8 bytes per vreg for
   simplicity)
    lsl x0, x20, #3
    bl _kcc_arena_alloc
    mov x21, x0                 // used array

    // Zero it
    mov x0, x21
    mov x1, #0
    lsl x2, x20, #3
    bl _kcc_memset
```

□

453a · defined at 453ab, 454 · used in 30c · defines _kcc_opt_dce 445b · uses _kcc_arena_alloc 58a

33.1.2. Mark Pass

The mark pass iterates every instruction in every basic block and records which vregs are referenced as source operands. Special care is needed for instructions whose fields carry non-vreg data: `IR_CONST` encodes an immediate in `src1`, `IR_CALL` stores a function name pointer, and branch instructions carry basic-block identifiers. A subtler case is the `IR_NOP` left behind by an earlier sweep: its source fields still hold whatever they carried before deletion (a folded constant leaves `src2` at zero), so reading them would spuriously mark vreg 0—or another—as used and keep a genuinely dead instruction alive across the fixed point. Because the pass and the sweep run in a loop, a `NOP` must therefore be skipped outright. The pass also handles `IR_CALL` argument arrays by walking each element and marking the argument vregs individually.

⟨ *Opt: dead code elimination* 453b ⟩ + ≡

```
    // Pass 1: mark used vregs
    ldr x0, [x19, #IRF_BLOCKS]
    cbz x0, .Ldce_done
    mov x22, x0                 // blocks vec
    ldr x23, [x22, #VEC_COUNT]
    mov x24, #0

.Ldce_mark_bb:
    cmp x24, x23
    b.ge .Ldce_sweep

    mov x0, x22
    mov x1, x24
    bl _kcc_vec_get
    ldr x25, [x0]               // BB ptr

    ldr x0, [x25, #BB_FIRST]
.Ldce_mark_instr:
    cbz x0, .Ldce_mark_next_bb
    mov x26, x0

    // A NOP (an instruction a previous sweep
   already deleted) references
    // nothing.  Its src fields still hold
   whatever they did before deletion---a
    // folded constant leaves src2 = 0---so
   reading them here would spuriously
    // mark vreg 0 (or another) as used, keeping a
    genuinely dead instruction
    // alive across the fixed point.  Skip NOPs
   entirely.
    ldr x2, [x26, #IR_OP]
    cmp x2, #IR_NOP
    b.eq .Ldce_mark_next

    // Mark SRC1 as used (if it looks like a vreg
   - <vreg_n)
```

```
    ldr x1, [x26, #IR_SRC1]
    ldr x2, [x26, #IR_OP]

    // Skip marking SRC1 for IR_CONST (src1=
   immediate), IR_CALL (src1=name),
    // IR_BR (src1=BB id), IR_GLOBAL_REF (src1=
   name)
    // IR_CALL_IND SRC1 is a vreg - DO mark it
    cmp x2, #IR_CONST
    b.eq .Ldce_mark_src2
    cmp x2, #IR_CALL
    b.eq .Ldce_mark_src2
    cmp x2, #IR_BR
    b.eq .Ldce_mark_src2
    cmp x2, #IR_GLOBAL_REF
    b.eq .Ldce_mark_src2

    cmp x1, x20
    b.ge .Ldce_mark_src2
    mov x3, #1
    str x3, [x21, x1, lsl #3]

.Ldce_mark_src2:
    ldr x1, [x26, #IR_SRC2]
    ldr x2, [x26, #IR_OP]

    // Skip SRC2 for IR_BR_COND (src2=BB id),
   IR_BR (no src2)
    cmp x2, #IR_BR_COND
    b.eq .Ldce_mark_dest
    cmp x2, #IR_BR
    b.eq .Ldce_mark_dest
    cmp x2, #IR_CONST
    b.eq .Ldce_mark_dest

    // IR_CALL/IR_CALL_IND: SRC2 is a pointer to
   args array, mark each arg vreg
    cmp x2, #IR_CALL
    b.eq .Ldce_mark_call_args
    cmp x2, #IR_CALL_IND
    b.eq .Ldce_mark_call_args

    cmp x1, x20
    b.ge .Ldce_mark_dest
    mov x3, #1
    str x3, [x21, x1, lsl #3]
    b .Ldce_mark_dest

.Ldce_mark_call_args:
    // x1 = SRC2 = pointer to [count, vreg0,
   class0, vreg1, class1, ...]
    // Two words per argument; the vreg is at
   offset 1 + index*2.
    cbz x1, .Ldce_mark_dest
    ldr x4, [x1]                    // packed: count
   | (first_stack_idx << 32)
    and x4, x4, #0xffffffff         // count (low
   half; high half is the ABI hint)
    mov x5, #0                       // index
.Ldce_mark_call_arg_loop:
    cmp x5, x4
    b.ge .Ldce_mark_dest
    lsl x6, x5, #1
    add x6, x6, #1                  // offset = 1 +
   index*2
    ldr x7, [x1, x6, lsl #3]     // vreg = args[1 +
    index*2]
    cmp x7, x20
    b.ge .Ldce_mark_call_arg_next
    mov x3, #1
    str x3, [x21, x7, lsl #3]    // mark vreg as
   used
.Ldce_mark_call_arg_next:
    add x5, x5, #1
    b .Ldce_mark_call_arg_loop

.Ldce_mark_dest:
    // Also mark DEST for br_cond (dest=condition
   vreg)
    ldr x2, [x26, #IR_OP]
    cmp x2, #IR_BR_COND
    b.ne .Ldce_mark_ret
    ldr x1, [x26, #IR_DEST]
    cmp x1, x20
    b.ge .Ldce_mark_ret
    mov x3, #1
    str x3, [x21, x1, lsl #3]

.Ldce_mark_ret:
    // Mark SRC1 for ret (src1=return value vreg)
    ldr x2, [x26, #IR_OP]
    cmp x2, #IR_RET
    b.ne .Ldce_mark_store
    ldr x1, [x26, #IR_SRC1]
    cmp x1, x20
    b.ge .Ldce_mark_next
    mov x3, #1
    str x3, [x21, x1, lsl #3]
    b .Ldce_mark_next

.Ldce_mark_store:
    // Mark both src1 and src2 for store
    ldr x2, [x26, #IR_OP]
    cmp x2, #IR_STORE
    b.ne .Ldce_mark_next
    ldr x1, [x26, #IR_SRC1]
    cmp x1, x20
    b.ge .Ldce_mark_store2
    mov x3, #1
    str x3, [x21, x1, lsl #3]
.Ldce_mark_store2:
    ldr x1, [x26, #IR_SRC2]
    cmp x1, x20
    b.ge .Ldce_mark_next
    mov x3, #1
    str x3, [x21, x1, lsl #3]

.Ldce_mark_next:
    ldr x0, [x26, #IR_NEXT]
    b .Ldce_mark_instr

.Ldce_mark_next_bb:
    add x24, x24, #1
    b .Ldce_mark_bb
```

□

453b · defined at 453ab, 454 · used in 30c · uses _kcc_vec_get 65b

33.1.3. Sweep Pass

Armed with the completed used-vreg bitmap, the sweep pass makes a second traversal over every instruction. Any instruction whose destination vreg is not marked as used—and that has no side effects—is replaced with IR_NOP. Side-effecting instructions (IR_STORE, IR_BR, IR_BR_COND, IR_RET, IR_CALL) and stack allocations (IR_ALLOCA) are unconditionally preserved. So are IR_VA_START, which writes the va_list structure to memory (its destination vreg may go unused on a target that keeps the register-save pointer in the structure rather than in the result), and IR_ASM, whose inline assembly may have arbitrary effects. The function returns the total number of eliminations so the pass manager can detect whether another round of optimization is warranted.

⟨ *Opt: dead code elimination* 454 ⟩ + ≡

```
    // Pass 2: sweep - replace dead non-side-
   effect instructions with NOP
.Ldce_sweep:
    mov x24, #0

.Ldce_sweep_bb:
    cmp x24, x23
    b.ge .Ldce_done

    mov x0, x22
    mov x1, x24
    bl _kcc_vec_get
    ldr x25, [x0]

    ldr x0, [x25, #BB_FIRST]
.Ldce_sweep_instr:
    cbz x0, .Ldce_sweep_next_bb
    mov x26, x0

    ldr x1, [x26, #IR_OP]
    // Skip side-effecting instructions
    cmp x1, #IR_STORE
    b.eq .Ldce_sweep_skip
    cmp x1, #IR_BR
    b.eq .Ldce_sweep_skip
    cmp x1, #IR_BR_COND
    b.eq .Ldce_sweep_skip
    cmp x1, #IR_RET
    b.eq .Ldce_sweep_skip
    cmp x1, #IR_CALL
    b.eq .Ldce_sweep_skip
    cmp x1, #IR_CALL_IND
    b.eq .Ldce_sweep_skip
    cmp x1, #IR_NOP
    b.eq .Ldce_sweep_skip
    cmp x1, #IR_ALLOCA
    b.eq .Ldce_sweep_skip       // keep allocas
    cmp x1, #IR_VA_START
    b.eq .Ldce_sweep_skip       // va_start writes
    the va_list struct in memory
    cmp x1, #IR_ASM
    b.eq .Ldce_sweep_skip       // inline asm has
   arbitrary side effects
    // A volatile load is a side effect: the read
   must occur even if its value
    // is unused, so it is never dead.
    cmp x1, #IR_LOAD
    b.ne .Ldce_notvol
    ldr x2, [x26, #IR_TYPE]
    cmp x2, #0x1000
    b.lo .Ldce_notvol
    ldr x2, [x2, #TY_QUALS]
    tst x2, #2
    b.ne .Ldce_sweep_skip
.Ldce_notvol:

    // Check if dest vreg is used
    ldr x2, [x26, #IR_DEST]
    cmp x2, x20
    b.ge .Ldce_sweep_skip
    ldr x3, [x21, x2, lsl #3]
    cbnz x3, .Ldce_sweep_skip

    // Dead! Replace with NOP
    mov x1, #IR_NOP
    str x1, [x26, #IR_OP]
    add x27, x27, #1

.Ldce_sweep_skip:
    ldr x0, [x26, #IR_NEXT]
    b .Ldce_sweep_instr

.Ldce_sweep_next_bb:
    add x24, x24, #1
    b .Ldce_sweep_bb

.Ldce_done:
    mov x0, x27

    ldr x27, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

454 · defined at 453ab, 454 · used in 30c · uses _kcc_vec_get 65b

34. Common Subexpression Elimination

Constant folding removes work whose *operands* are known; common subexpression elimination removes work that is simply *repeated*. When a basic block computes the same pure operation on the same operands twice—or loads the same address twice with nothing written in between—the second occurrence is redundant and its result can be taken from the first.

The pass is deliberately *local*: it value-numbers within a single basic block and starts fresh at each block boundary. Inside one block, textual order is execution order and the first occurrence dominates the second, so no dominance analysis is needed. The IR is quasi-SSA—every instruction defines a fresh virtual register that is never rewritten—so two pure instructions with the same opcode and the same source registers are guaranteed to compute the same value. A function-wide *replacement map* then records, for each redundant register, the earlier register that supersedes it; as the walk proceeds it rewrites every operand through this map, so a later use of a redundant value reads the surviving one instead. The redundant instruction is thereby left with no users and the dead-code pass deletes it.

Loads are the real prize. Because kcc keeps every C local in memory, an expression like `(x+y)*(x+y)` reloads `x` and `y` four times; the two `x+y` additions look different only because each reads a fresh load. Eliminating the redundant *loads* is therefore what unlocks the arithmetic CSE above it. A load is redundant when the same address was already loaded in this block and nothing has been stored since. Without alias analysis the pass is conservative: *any* store or call invalidates every cached load, since either might write the address in question. That is safe and still catches the common reload-a-local pattern, where the stores sit at the top of the block and the reloads follow uninterrupted.

Which operands are registers. Rewriting operands through the replacement map requires knowing, per opcode, which fields hold a virtual register and which hold something else—an immediate, a byte width, a basic-block id, a symbol. Writing a canonical register into a width field would corrupt it, so the canonicaliser skips those fields precisely. It rewrites `src1` for every value-producing op and a load/store/return operand, `src2` for binary arithmetic, `GEP`, and a store's address, and the condition of a `BR_COND`; it leaves constants, allocas, branches, and cast widths untouched. Call arguments are left alone—rewriting them is possible but fiddly, and skipping them only forgoes an optimization, never correctness.

⟨ *Opt: common subexpression elimination* 457 ⟩ ≡

```
// -- kcc_cse_canon ---
// uint64_t kcc_cse_canon(uint64_t vreg) -> the
   register that supersedes it.
// Reads x20 = vreg_n bound, x21 = replacement map
   .  Leaf; touches x0 only.
.text
.p2align 2
_kcc_cse_canon:
    cmp x0, x20
    b.hs .Lcsc_ret                // not a vreg
   number → unchanged
    ldr x0, [x21, x0, lsl #3]
.Lcsc_ret:
    ret

// -- kcc_cse_canon_ops ---
// void kcc_cse_canon_ops(ir_instr *instr)
// Rewrite the instruction's virtual-register
   operands through the map, in
// place.  x20 = vreg_n, x21 = map (canon reads
   them).
.text
.p2align 2
_kcc_cse_canon_ops:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    mov x1, x0                     // x1 = instr (
   survives the leaf canon calls)
    ldr x9, [x1, #IR_OP]

    // --- src1: a vreg for every op except these
   ---
    cmp x9, #IR_CONST
    b.eq .Lcco_src1_done
    cmp x9, #IR_CALL
    b.eq .Lcco_src1_done
    cmp x9, #IR_BR
    b.eq .Lcco_src1_done
    cmp x9, #IR_GLOBAL_REF
    b.eq .Lcco_src1_done
    cmp x9, #IR_ALLOCA
    b.eq .Lcco_src1_done
    cmp x9, #IR_NOP
    b.eq .Lcco_src1_done
    cmp x9, #IR_BR_COND
    b.eq .Lcco_src1_done
    cmp x9, #IR_STRING_LIT
    b.eq .Lcco_src1_done
    cmp x9, #IR_VA_START
    b.eq .Lcco_src1_done
    ldr x0, [x1, #IR_SRC1]
    bl _kcc_cse_canon
    str x0, [x1, #IR_SRC1]
.Lcco_src1_done:

    // --- src2: a vreg only for binary arith/
   logic/cmp, GEP, and store ---
    ldr x9, [x1, #IR_OP]
    cmp x9, #IR_GEP
    b.eq .Lcco_do_src2
    cmp x9, #IR_STORE
    b.eq .Lcco_do_src2
    cmp x9, #1
    b.lt .Lcco_src2_done
    cmp x9, #7
    b.le .Lcco_do_src2
    cmp x9, #10
    b.lt .Lcco_src2_done
    cmp x9, #15
    b.le .Lcco_do_src2
    cmp x9, #20
    b.lt .Lcco_src2_done
    cmp x9, #27
    b.le .Lcco_do_src2
    b .Lcco_src2_done
.Lcco_do_src2:
```

```
    ldr x0, [x1, #IR_SRC2]
    bl _kcc_cse_canon
    str x0, [x1, #IR_SRC2]
.Lcco_src2_done:

    // --- br_cond condition lives in dest; ret value in src1 ---
    ldr x9, [x1, #IR_OP]
    cmp x9, #IR_BR_COND
    b.ne .Lcco_done
    ldr x0, [x1, #IR_DEST]
    bl _kcc_cse_canon
    str x0, [x1, #IR_DEST]
.Lcco_done:
    ldp x29, x30, [sp], #16
    ret

// -- kcc_opt_cse ---
// uint64_t kcc_opt_cse(ir_func *func) -> number of redundancies removed.
.text
.globl _kcc_opt_cse
.p2align 2
_kcc_opt_cse:
    stp x29, x30, [sp, #-160]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    stp x27, x28, [sp, #80]

    mov x19, x0                     // func
    mov x27, #0                     // change count
    ldr x20, [x19, #IRF_VREG_N]
    cbz x20, .Lcse_done
    add x20, x20, #1                // bound (safety)

    // Allocate: replacement map (8B/vreg), value table (32B/vreg),
    // load table (16B/vreg).
    lsl x0, x20, #3
    bl _kcc_arena_alloc
    mov x21, x0                     // replacement map
    lsl x0, x20, #5                 // 32 * vreg_n
    bl _kcc_arena_alloc
    mov x22, x0                     // value table {op,s1,s2,dest}
    lsl x0, x20, #4                 // 16 * vreg_n
    bl _kcc_arena_alloc
    mov x23, x0                     // load table {addr,dest}

    // Initialise the replacement map to the identity (map[i] = i).
    mov x9, #0
.Lcse_init:
    cmp x9, x20
    b.hs .Lcse_init_done
    str x9, [x21, x9, lsl #3]
    add x9, x9, #1
    b .Lcse_init
.Lcse_init_done:

    // --- Walk basic blocks ---
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x25, [x0, #VEC_COUNT]       // bb count
    mov x24, #0                     // bb index
.Lcse_bb:
    cmp x24, x25
    b.hs .Lcse_done

    // reset both tables for this block
    str xzr, [sp, #96]              // value_count
    str xzr, [sp, #104]             // load_count
    ldr x0, [x19, #IRF_BLOCKS]
    mov x1, x24
    bl _kcc_vec_get
    ldr x26, [x0]
    ldr x26, [x26, #BB_FIRST]
.Lcse_instr:
    cbz x26, .Lcse_next_bb

    // 1. Canonicalise this instruction's register operands.
    mov x0, x26
    bl _kcc_cse_canon_ops

    ldr x9, [x26, #IR_OP]

    // 2a. A store, call, or inline-asm invalidates every cached load (the asm
    // may write memory through its output operands or a "memory" clobber).
    cmp x9, #IR_STORE
    b.eq .Lcse_kill
    cmp x9, #IR_CALL
    b.eq .Lcse_kill
    cmp x9, #IR_CALL_IND
    b.eq .Lcse_kill
    cmp x9, #IR_ASM
    b.eq .Lcse_kill

    // 2b. Load: reuse an earlier load of the same address.
    cmp x9, #IR_LOAD
    b.eq .Lcse_load

    // 2c. Pure op: value-number by (op, src1, src2).
    // pure = 1..16, 20..27, GEP(33), 50..55, 57
    cmp x9, #1
    b.lt .Lcse_next
    cmp x9, #16
    b.le .Lcse_pure
    cmp x9, #20
    b.lt .Lcse_chk_cast
    cmp x9, #27
    b.le .Lcse_pure
.Lcse_chk_cast:
    cmp x9, #IR_GEP
    b.eq .Lcse_pure
    cmp x9, #50
    b.lt .Lcse_next
    cmp x9, #55
    b.le .Lcse_pure
    cmp x9, #57
    b.eq .Lcse_pure
    b .Lcse_next

.Lcse_kill:
    str xzr, [sp, #104]             // load_count = 0
    b .Lcse_next

.Lcse_load:
    // A volatile load must actually happen every time and must not be reordered,
    // so it acts as a barrier: invalidate the cache and neither reuse nor record
    // it.
    ldr x10, [x26, #IR_TYPE]
    cmp x10, #0x1000
    b.lo .Lcse_load_ok
    ldr x10, [x10, #TY_QUALS]
    tst x10, #2
    b.eq .Lcse_load_ok
    str xzr, [sp, #104]             // kill cache
```

```
    b .Lcse_next
.Lcse_load_ok:
    ldr x10, [x26, #IR_SRC1]      // canonical
   address
    ldr x11, [sp, #104]           // load_count
    mov x12, #0                     // scan index
.Lcse_load_scan:
    cmp x12, x11
    b.hs .Lcse_load_add
    add x13, x23, x12, lsl #4      // &load_table[i
   ]
    ldr x14, [x13]                 // addr
    cmp x14, x10
    b.ne .Lcse_load_next
    // hit: map this load's dest to the earlier
   load's dest
    ldr x14, [x13, #8]            // earlier dest
    ldr x15, [x26, #IR_DEST]
    cmp x15, x20
    b.hs .Lcse_next
    str x14, [x21, x15, lsl #3]   // map[dest] =
   earlier
    add x27, x27, #1
    b .Lcse_next
.Lcse_load_next:
    add x12, x12, #1
    b .Lcse_load_scan
.Lcse_load_add:
    add x13, x23, x11, lsl #4
    str x10, [x13]                // addr
    ldr x15, [x26, #IR_DEST]
    str x15, [x13, #8]            // dest
    add x11, x11, #1
    str x11, [sp, #104]
    b .Lcse_next

.Lcse_pure:
    ldr x9, [x26, #IR_OP]
    ldr x10, [x26, #IR_SRC1]
    ldr x11, [x26, #IR_SRC2]
    ldr x28, [sp, #96]            // value_count
    mov x12, #0
.Lcse_pure_scan:
    cmp x12, x28
    b.hs .Lcse_pure_add
    add x13, x22, x12, lsl #5      // &value_table[
   i]
    ldr x14, [x13]                // op
    cmp x14, x9
    b.ne .Lcse_pure_next
    ldr x14, [x13, #8]            // s1
    cmp x14, x10
    b.ne .Lcse_pure_next
    ldr x14, [x13, #16]           // s2
    cmp x14, x11
    b.ne .Lcse_pure_next
    // hit
    ldr x14, [x13, #24]           // earlier dest
    ldr x15, [x26, #IR_DEST]
    cmp x15, x20
    b.hs .Lcse_next
    str x14, [x21, x15, lsl #3]
    add x27, x27, #1
    b .Lcse_next
.Lcse_pure_next:
    add x12, x12, #1
    b .Lcse_pure_scan
.Lcse_pure_add:
    add x13, x22, x28, lsl #5
    str x9, [x13]
    str x10, [x13, #8]
    str x11, [x13, #16]
    ldr x15, [x26, #IR_DEST]
    str x15, [x13, #24]
    add x28, x28, #1
    str x28, [sp, #96]

.Lcse_next:
    ldr x26, [x26, #IR_NEXT]
    b .Lcse_instr
.Lcse_next_bb:
    add x24, x24, #1
    b .Lcse_bb

.Lcse_done:
    mov x0, x27
    ldp x27, x28, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #160
    ret

// -- kcc_opt_copyprop ---
// uint64_t kcc_opt_copyprop(ir_func *func) ->
   number of forwards.
// Store-to-load forwarding = copy propagation +
   constant propagation for the
// memory-backed locals this IR uses.  It is
   deliberately airtight: within one
// straight-line block it tracks the single most
   recent store (address, value,
// and access-size code); a following load of that
    exact address forwards the
// stored value ONLY when (a) no store, call, or
   va_start intervened---so memory
// is provably unchanged---and (b) both sides are
   the natural 8-byte access, so
// no truncation or re-extension can make the
   loaded register differ from the
// stored vreg.  Substitution is intra-block (the
   map is reset at each block
// boundary), so no cross-block dominance question
    arises.  DCE then sweeps the
// loads this makes dead.  These three conditions
   together mean it never needs
// alias, escape, or dominance analysis to be
   correct.
.p2align 2
.globl _kcc_opt_copyprop
_kcc_opt_copyprop:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    stp x27, x28, [sp, #80]

    mov x19, x0                   // func
    mov x22, #0                   // forward count
    ldr x20, [x19, #IRF_VREG_N]
    cbz x20, .Lcp_done
    add x20, x20, #1              // vreg bound (
   canon reads x20)
    lsl x0, x20, #3
    bl _kcc_arena_alloc
    mov x21, x0                   // replacement
   map (canon reads x21)

    ldr x24, [x19, #IRF_ENTRY]
.Lcp_bb:
    cbz x24, .Lcp_done
    mov x9, #0                    // reset map to
   identity (forwards are block-local)
.Lcp_reset:
```

```
    cmp x9, x20
    b.ge .Lcp_bb_start
    str x9, [x21, x9, lsl #3]
    add x9, x9, #1
    b .Lcp_reset
.Lcp_bb_start:
    mov x25, #0                     // last-store
   valid = no
    ldr x23, [x24, #BB_FIRST]
.Lcp_i:
    cbz x23, .Lcp_nb
    mov x0, x23
    bl _kcc_cse_canon_ops           // rewrite
   operands via forwards seen so far

    ldr x9, [x23, #IR_OP]
    cmp x9, #IR_STORE
    b.eq .Lcp_store
    cmp x9, #IR_LOAD
    b.eq .Lcp_load
    cmp x9, #IR_CALL
    b.eq .Lcp_kill
    cmp x9, #IR_CALL_IND
    b.eq .Lcp_kill
    cmp x9, #IR_ASM
    b.eq .Lcp_kill
    cmp x9, #IR_VA_START
    b.eq .Lcp_kill
    b .Lcp_next
.Lcp_store:
    ldr x26, [x23, #IR_SRC2]        // address
    ldr x27, [x23, #IR_SRC1]        // value (
   already canonical)
    ldr x28, [x23, #IR_DEST]        // store access-
   size code
    mov x25, #1
    b .Lcp_next
.Lcp_kill:
    mov x25, #0                     // memory may
   have changed
    b .Lcp_next
.Lcp_load:
    cbz x25, .Lcp_next
    ldr x9, [x23, #IR_SRC1]         // load address
    cmp x9, x26
    b.ne .Lcp_next
    cbnz x28, .Lcp_next             // store not the
    natural 8-byte
    ldr x9, [x23, #IR_SRC2]         // load access-
   size code
    cbnz x9, .Lcp_next              // load not 8-
   byte
    ldr x10, [x23, #IR_DEST]
    cmp x10, x20
    b.ge .Lcp_next
    str x27, [x21, x10, lsl #3]     // map[load_dest
   ] = stored value
    add x22, x22, #1
.Lcp_next:
    ldr x23, [x23, #IR_NEXT]
    b .Lcp_i
.Lcp_nb:
    ldr x24, [x24, #BB_NEXT]
    b .Lcp_bb

.Lcp_done:
    mov x0, x22
    ldp x27, x28, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

457 · used in 30c · defines _kcc_cse_canon never used, _kcc_cse_canon_ops never used, _kcc_opt_copyprop 445b, _kcc_opt_cse 445b · uses _kcc_arena_alloc 58a, _kcc_vec_get 65b

Part VIII.

The Backend

The backend closes the loop: it lowers the optimised IR into ARM64 machine code and packages the result as a Mach-O object file that Apple's linker can consume. The code generator (Chapter 35) uses a simple all-spill strategy — every virtual register lives in a stack slot, and three scratch registers (x9, x10, x11) shuttle values between slots and ALU — trading runtime performance for implementation simplicity. The ARM64 encoder (Chapter 36) provides functions that pack each instruction class into its 32-bit binary encoding. The Mach-O writer (Chapter 40) assembles the object file: header, load commands, code section, relocations, symbol table, and string table. The linker-info module (Chapter 41) manages relocations and symbols during code generation, and the compilation driver (Chapter 43) orchestrates the full pipeline from source file to `.o`.

35. Code Generation

This chapter is where the compiler's abstract IR meets the target machine. The code generator walks each IR function and emits ARM64 machine code using a simple "all-spill" strategy: every virtual register is assigned a stack slot, and each instruction loads its sources from the stack into scratch registers (x9, x10), computes the result in x11, and stores it back. This avoids the complexity of register allocation while producing correct code. The chapter covers the stack-frame layout, initialization of the cstring buffer for string literals, the main instruction-lowering loop, and the post-codegen fixup pass that patches string-literal symbol values.

35.1. Strategy

The stack frame for a generated function has three regions:

```
[x29+0]              saved x29/x30 (16 bytes)
[x29+16]             vreg value slots (vreg_count * 8)
[x29+16+vc*8]        alloca storage  (total_alloca_slots * 8)
```

The total frame size is $align16(16 + \text{vreg_count} \times 8 + \text{alloca_slots} \times 8)$. A pre-scan pass walks all `IR_ALLOCA` instructions before code generation, sums their slot counts, and assigns each alloca a running offset (stored in `SRC2`). This lets `IR_ALLOCA` compute its storage address at emit time and ensures alloca storage does not overlap vreg slots. We use x9 for source 1, x10 for source 2, x11 as the destination scratch register, and x12 as an additional scratch for `IR_GEP` address arithmetic.

35.2. Constants

The code generator's fixed parameters—slot size, scratch register assignments, and patch-entry layout—are defined here as assembler equates so they can be referenced symbolically throughout the backend.

⟨ *CodeGen: constants* 463a ⟩ ≡

```
// -- CodeGen constants ---
.equ CG_SLOT_SIZE,    8           // each vreg
   occupies 8 bytes on stack
.equ CG_REG_SRC1,     9           // x9 = first
   source scratch
.equ CG_REG_SRC2,    10           // x10 = second
   source scratch
.equ CG_REG_DST,     11           // x11 =
   destination scratch

// Forward-branch patch entry: {code_offset(8),
   target_bb_id(8)} = 16 bytes
.equ PATCH_OFFSET,    0
.equ PATCH_TARGET,    8
.equ PATCH_SIZE,     16
```

□

463a · used in 31c

35.3. Initialization

Initialization zeroes the frame-size state and prepares the infrastructure for string literals. We allocate a 4 KB *cstring buffer* where literal bytes will accumulate during code generation, a counter for naming symbols (`l_.str.0`, `l_.str.1`, …), and a *fixup vector* whose elements are pairs $(sym_index, cstr_offset)$. After all instructions have been lowered, the fixup pass (§35.7) patches each symbol's value and appends the raw string data to the code section.

⟨ *CodeGen: init* 463b ⟩ ≡

```
// -- kcc_codegen_init ---
.text
.globl _kcc_codegen_init
.p2align 2
_kcc_codegen_init:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Zero state
    adrp x0, _kcc_cg_frame_size@PAGE
    add  x0, x0, _kcc_cg_frame_size@PAGEOFF
    str xzr, [x0]

    // Allocate cstring buffer (4KB initial) and
   record its capacity so it can
    // grow: the buffer holds not only string
   literals but (M5) the byte images
    // of read-only const globals, which can
   exceed the old fixed 4KB.
    mov x0, #4096
    bl _kcc_arena_alloc
    adrp x1, _kcc_cstr_buf@PAGE
    add  x1, x1, _kcc_cstr_buf@PAGEOFF
    str x0, [x1]
    mov x0, #4096
    adrp x1, _kcc_cstr_cap@PAGE
    add  x1, x1, _kcc_cstr_cap@PAGEOFF
    str x0, [x1]

    adrp x1, _kcc_cstr_pos@PAGE
    add  x1, x1, _kcc_cstr_pos@PAGEOFF
    str xzr, [x1]

    adrp x1, _kcc_cstr_count@PAGE
```

```
    add  x1, x1, _kcc_cstr_count@PAGEOFF
    str xzr, [x1]

    // Allocate fixup vec (elem=16: {sym_index,
   cstr_offset})
    mov x0, #VEC_SIZE
    bl _kcc_arena_alloc
    adrp x1, _kcc_cstr_fixups@PAGE
    add  x1, x1, _kcc_cstr_fixups@PAGEOFF
    str x0, [x1]
    mov x1, #16
    str x1, [x0, #VEC_ELEMSIZE]
    mov x1, #32
    str x1, [x0, #VEC_CAP]
    str xzr, [x0, #VEC_COUNT]
    mov x19, x0

    mov x0, #512                    // 32 * 16
    bl _kcc_arena_alloc
    str x0, [x19, #VEC_DATA]

    // __DATA_CONST,__const state: absent until a
   const pointer array needs it.
    adrp x1, _kcc_rodata_const_present@PAGE
    add  x1, x1, _kcc_rodata_const_present@PAGEOFF
    str xzr, [x1]
    adrp x1, _kcc_rodata_const_size@PAGE
    add  x1, x1, _kcc_rodata_const_size@PAGEOFF
    str xzr, [x1]
    mov x0, #VEC_SIZE               // relocs vec:
   {slot_offset, sym_index}
    bl _kcc_arena_alloc
    adrp x1, _kcc_rodata_const_relocs@PAGE
    add  x1, x1, _kcc_rodata_const_relocs@PAGEOFF
    str x0, [x1]
    mov x1, #16
    str x1, [x0, #VEC_ELEMSIZE]
    mov x1, #32
    str x1, [x0, #VEC_CAP]
    str xzr, [x0, #VEC_COUNT]
    mov x19, x0
    mov x0, #512                    // 32 * 16
    bl _kcc_arena_alloc
    str x0, [x19, #VEC_DATA]

    // Fresh set of globals defined in this
   translation unit
    mov x0, #32
    bl _kcc_ht_init
    adrp x1, _kcc_cg_defined_globals@PAGE
    add  x1, x1, _kcc_cg_defined_globals@PAGEOFF
    str x0, [x1]

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

463b · used in 31c · defines _kcc_codegen_init 603 · uses _kcc_arena_alloc 58a, _kcc_cg_defined_globals 479, _kcc_cg_frame_size 479, _kcc_cstr_buf 479, _kcc_cstr_count 479, _kcc_cstr_fixups 479, _kcc_cstr_pos 479, _kcc_ht_init 61c, _kcc_rodata_const_present 479, _kcc_rodata_const_relocs 479, _kcc_rodata_const_size 479

35.4. Pre-register Global Sizes

Before code generation begins, the driver calls `kcc_preregister_globals` to walk the AST translation unit and pre-register every global variable in the linker with its correct allocation size. Scalar and pointer globals need eight bytes, while struct globals need $n \times 8$ bytes (one eight-byte slot per field). Because `kcc_linker_global_sym` deduplicates—returning immediately when a symbol is already defined—subsequent references during code generation are no-ops with respect to allocation.

A global declared `extern` is a *declaration*, not a definition: its storage lives in another translation unit, so this pass skips it and reserves no bytes. The names it does define are recorded in a per-unit `defined_globals` set. This set is what lets the `IR_GLOBAL_REF` handler decide, for each reference, between a locally defined symbol and an undefined external one: a name absent from the set is emitted through `kcc_linker_extern` so the linker resolves it against whichever object supplies the definition. Without the set, every reference would define its own copy and two objects naming the same global would collide.

A second pass reserves storage for `static` locals, which are hoisted to globals under a mangled `sl_` node name (§29.11.1). Two subtleties matter here. Its size must be the declared type's real byte count—an aggregate `static` (`static const char *evs[] = {…}`) needs $n \times 8$ bytes, not the eight-byte scalar fallback, or it under-allocates and overlaps whatever symbol the linker packs next. And the reserved symbol must carry the Mach-O leading underscore, since an `IR_GLOBAL_REF` to the static is emitted as an underscore-prefixed reference; a bare reservation would split into two symbols, the reference binding to an unsized stand-in. Both bite hardest across translation units: a file-scope aggregate in a non-`main` unit is initialised from `kcc.module_init`, and a mis-sized or mis-named static there scribbles over its neighbours—the failure that corrupted Lua's `luaT_eventname` metamethod-name table.

The two cases also differ in *how* the address is formed. A locally defined global has an address inside the image, reachable with a direct ADRP+ADD page/offset pair. An external one may be defined in another object—or imported from a dynamic library, like the standard streams `stdin`/`stdout`/`stderr` (which are the `libSystem` data symbols `__stdinp` and friends). A dylib datum has no fixed image address; it is reached only through the global offset table, so the handler emits ADRP `GOTPAGE` followed by an LDR `GOTPAGEOFF` that loads the symbol's resolved address from a GOT slot the linker binds. The same GOT form works for an ordinary cross-object extern (the linker simply points the slot at the local definition), so all external references take it uniformly; only the load, not the code generator, pays for the extra indirection.

One wrinkle in sizing a global's storage is the *flexible array member* (§6.7.2.1): `struct { char c; char p[]; } a = {'o', "wx"}` occupies more than the struct's nominal size, because the trailing `p[]` is laid out with length zero but its initializer supplies real bytes. Before reserving storage, `_kcc_cg_fam_extra` inspects the declaration: if the type is a struct whose last member is an unsized `[]` array and a matching initializer element supplies its contents, it returns those bytes (a string's length plus its NUL, or a brace list's element count times the member size), which the pre-registration adds to the reserved size. The initializer itself is copied by the shared aggregate-fill machinery (§**??**), which now recognises a string landing on the flexible member.

⟨ CodeGen: preregister globals 465 ⟩ ≡

```
// -- kcc_cg_fam_extra ---
// uint64_t kcc_cg_fam_extra(node *decl, type *ty)
// The extra storage bytes an initialized trailing
    flexible array member needs
// beyond the struct's nominal size.  Returns 0
   unless `ty` is a struct, `decl`
// has a brace initializer, the struct's last
   member is an unsized `[]` array,
// and a matching initializer element supplies its
    contents.  A string element
// contributes its length + 1 (the NUL); a brace
   list contributes its element
// count times the member's element size.
.text
.p2align 2
_kcc_cg_fam_extra:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]
    mov x19, x0                     // decl
    mov x21, #0                     // result
    ldr x0, [x1, #TY_KIND]
    cmp x0, #TY_STRUCT
    b.ne .Lfam_ret
    ldr x22, [x19, #AST_F2]         // initializer
    cbz x22, .Lfam_ret
    ldr x0, [x22, #AST_TAG]
    cmp x0, #AST_INIT_LIST
    b.ne .Lfam_ret
    ldr x0, [x1, #TY_F0]            // struct decl
   AST
    cbz x0, .Lfam_ret
    ldr x23, [x0, #AST_F1]          // first field
    cbz x23, .Lfam_ret
    mov x20, #1                     // field count
.Lfam_lastf:
    ldr x0, [x23, #AST_F4]
    cbz x0, .Lfam_havef
    mov x23, x0
    add x20, x20, #1
    b .Lfam_lastf
.Lfam_havef:
    // Last field must be an unsized array (a
   flexible array member).
    ldr x0, [x23, #AST_F0]
    bl _kcc_sema_resolve_type
    cbz x0, .Lfam_ret
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_ARRAY
    b.ne .Lfam_ret
    ldr x1, [x0, #TY_F1]            // declared
   length
    cbnz x1, .Lfam_ret              // sized array:
    not flexible
    ldr x0, [x0, #TY_F0]            // element type
    bl _kcc_type_layout             // element byte
    size
    mov x23, x0
    // Seek the initializer element at the FAM's
   position (field count - 1).
    ldr x0, [x22, #AST_F0]          // first
   element
    sub x1, x20, #1
.Lfam_seek:
    cbz x0, .Lfam_ret               // fewer
   elements than fields: FAM uninitialized
    cbz x1, .Lfam_gote
    ldr x0, [x0, #AST_F4]
    sub x1, x1, #1
    b .Lfam_seek
.Lfam_gote:
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_STR_LIT
    b.ne .Lfam_list
    ldr x21, [x0, #AST_F1]          // string
   length
    add x21, x21, #1                // + NUL
    b .Lfam_ret
.Lfam_list:
    cmp x1, #AST_INIT_LIST
    b.ne .Lfam_ret                  // other FAM
   init forms: conservatively 0
    ldr x0, [x0, #AST_F0]           // first sub-
   element
    mov x1, #0
.Lfam_cnt:
    cbz x0, .Lfam_cntd
    add x1, x1, #1
    ldr x0, [x0, #AST_F4]
    b .Lfam_cnt
.Lfam_cntd:
    mul x21, x1, x23                // element
   count * element size
.Lfam_ret:
    mov x0, x21
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_preregister_globals ---
// void kcc_preregister_globals(node *trans_unit)
//   x0 = AST_TRANS_UNIT node
// Walks top-level variable declarations, resolves
    their types, and
// pre-registers each global with the linker at
   the correct byte size.
.text
.globl _kcc_preregister_globals
.p2align 2
_kcc_preregister_globals:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    ldr x23, [x0, #AST_F0]      // first
   declaration (kept for the static pass)
    mov x19, x23

.Lprg_loop:
    cbz x19, .Lprg_statics

    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_VAR_DECL
    b.ne .Lprg_next

    ldr x20, [x19, #AST_F1]     // name
    cbz x20, .Lprg_next

    // extern globals are declarations, not
   definitions: emit no storage
    ldr x0, [x19, #AST_F3]      // storage class
   (2 = extern)
    cmp x0, #2
    b.eq .Lprg_next

    // Resolve the declared type
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    cbz x0, .Lprg_next
```

```
    // Reserve the type's real byte size, floored
   at 8 and rounded up to a
    // whole 8-byte slot.  Rounding keeps slot-
   granular struct copies (which
    // move ceil(size/8) eight-byte chunks) inside
    the reserved storage.  An
    // initialized trailing flexible array member
   extends the object past its
    // nominal size, so its initializer's bytes
   are added here.
    mov x22, x0                 // resolved type (
   survives type_layout)
    bl _kcc_type_layout         // x0 = real
   TY_SIZE
    mov x21, x0
    mov x0, x19                 // decl
    mov x1, x22                 // type
    bl _kcc_cg_fam_extra        // x0 = flexible-
   array-member init bytes (0 if none)
    add x21, x21, x0
    cmp x21, #8
    b.ge .Lprg_round
    mov x21, #8
.Lprg_round:
    add x21, x21, #7
    bic x21, x21, #7            // round up to a
   multiple of 8
.Lprg_have_size:

    // Build "_" + name
    mov x0, x20
    bl _kcc_strlen
    add x0, x0, #2
    bl _kcc_arena_alloc
    mov x22, x0
    mov w1, #'_'
    strb w1, [x22]
    mov x0, x20
    bl _kcc_strlen
    add x2, x0, #1
    add x0, x22, #1
    mov x1, x20
    bl _kcc_memcpy

    // A const scalar/aggregate becomes read-only
   __TEXT bytes; a const pointer
    // array goes to __DATA_CONST,__const;
   otherwise, __common storage.
    mov x0, x19
    bl _kcc_global_const_rodata
    cbz x0, .Lprg_try_ptrarr
    mov x0, x19                 // decl
    mov x1, x22                 // "_"+name
    bl _kcc_emit_rodata_global
    b .Lprg_defined
.Lprg_try_ptrarr:
    mov x0, x19
    bl _kcc_global_const_ptrarr
    cbz x0, .Lprg_common
    mov x0, x19
    mov x1, x22
    bl _kcc_emit_rodata_ptrarr
    b .Lprg_defined
.Lprg_common:
    // A file-scope `static' object is externally
   invisible: emit it as a local
    // symbol (N_EXT cleared) so two TUs that each
    define a same-named static do
    // not collide.  A non-static object stays
   external (a tentative definition
    // other TUs may reference).
    mov x0, x22
    mov x1, x21
    mov x2, #N_EXT
    ldr x4, [x19, #AST_F3]      // storage class
    (3 = static)
    cmp x4, #3
    csel x2, xzr, x2, eq        // static → 0 (
   local); else N_EXT
    bl _kcc_linker_global_sym
.Lprg_defined:

    // Remember the source name as defined-in-this
   -TU
    mov x0, x20
    bl _kcc_strlen
    mov x2, x0                  // keylen
    adrp x0, _kcc_cg_defined_globals@PAGE
    add  x0, x0, _kcc_cg_defined_globals@PAGEOFF
    ldr x0, [x0]
    mov x1, x20                 // key = source
   name
    mov x3, #1                  // present
    bl _kcc_ht_put

.Lprg_next:
    ldr x19, [x19, #AST_F4]     // next sibling
    b .Lprg_loop

.Lprg_statics:
    // Second pass: static locals are defined here
    too, under their mangled
    // names.  Record each so IR_GLOBAL_REF treats
    it as locally defined.
    mov x19, x23
.Lprg_fn_loop:
    cbz x19, .Lprg_done
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_FUNC_DECL
    b.ne .Lprg_fn_next
    ldr x0, [x19, #AST_F3]      // function body (
   compound statement)
    cbz x0, .Lprg_fn_next
    bl _kcc_cg_prereg_stmt      // recurse the
   whole body, at any block depth
    b .Lprg_fn_next

// -- kcc_cg_prereg_static ---
// void kcc_cg_prereg_static(node *decl)
//   Reserve backing storage for one `static`
   local (block-scoped, static
//   storage duration) and record it as defined in
    this TU.
.globl _kcc_cg_prereg_static
.p2align 2
_kcc_cg_prereg_static:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]
    mov x20, x0                 // decl (the
   reservation body below uses x20)

    mov x0, x20
    bl _kcc_ir_static_name
    mov x21, x0                 // mangled name

    // Reserve the static's backing storage at its
    real type size (floored at 8,
    // rounded to a whole 8-byte slot), as the
   file-scope pass does; otherwise
    // the IR_GLOBAL_REF handler falls back to a
   scalar 8 bytes and a static
    // array or struct larger than that is under-
   allocated.
    ldr x0, [x20, #AST_F0]      // declared type
   node
```

```
    cbz x0, .Lprg_st_size8
    bl _kcc_sema_resolve_type
    cbz x0, .Lprg_st_size8
    bl _kcc_type_layout          // x0 = real
   TY_SIZE (arrays/structs included)
    mov x22, x0
    cmp x22, #8
    b.lt .Lprg_st_size8
    add x22, x22, #7
    bic x22, x22, #7             // round up to a
   whole 8-byte slot
    b .Lprg_st_reserve
.Lprg_st_size8:
    mov x22, #8
.Lprg_st_reserve:
    // The linker symbol must carry the Mach-O
   leading underscore, exactly as
    // the file-scope pass builds "_"+name: an
   IR_GLOBAL_REF to this static is
    // emitted as an underscore-prefixed reference
   , so a bare reservation and
    // the prefixed reference would split into two
    symbols---the reference then
    // binding to an unsized, overlapping stand-in
   .  Build "_"+mangled for the
    // reservation; the bare mangled name still
   keys defined_globals below,
    // matching how the reference handler looks
   the name up.
    mov x0, x21
    bl _kcc_strlen
    add x0, x0, #2               // '_' + name +
   NUL
    bl _kcc_arena_alloc
    mov x23, x0                  // x23 = "_"+
   mangled
    mov w1, #'_'
    strb w1, [x23]
    mov x0, x21
    bl _kcc_strlen
    add x2, x0, #1               // copy name + NUL
    add x0, x23, #1
    mov x1, x21
    bl _kcc_memcpy

    // Read-only const static locals go to __TEXT
   (scalars/aggregates) or
    // __DATA_CONST,__const (pointer arrays), like
    file-scope const globals.
    mov x0, x20                  // static decl
    bl _kcc_global_const_rodata
    cbz x0, .Lprg_st_try_ptrarr
    mov x0, x20
    mov x1, x23                  // "_"+mangled
   name
    bl _kcc_emit_rodata_global
    b .Lprg_st_defined
.Lprg_st_try_ptrarr:
    mov x0, x20
    bl _kcc_global_const_ptrarr
    cbz x0, .Lprg_st_common
    mov x0, x20
    mov x1, x23                  // "_"+mangled
   name
    bl _kcc_emit_rodata_ptrarr
    b .Lprg_st_defined
.Lprg_st_common:
    mov x0, x23                  // "_"+mangled
   name
    mov x1, x22                  // byte size
    mov x2, #N_EXT               // mangled name
   is already unique per TU
    bl _kcc_linker_global_sym
.Lprg_st_defined:
    mov x0, x21
    bl _kcc_strlen
    mov x2, x0                   // keylen
    adrp x0, _kcc_cg_defined_globals@PAGE
    add  x0, x0, _kcc_cg_defined_globals@PAGEOFF
    ldr x0, [x0]
    mov x1, x21                  // key = mangled
   name
    mov x3, #1
    bl _kcc_ht_put

    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

.Lprg_fn_next:
    ldr x19, [x19, #AST_F4]
    b .Lprg_fn_loop

.Lprg_done:
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_cg_prereg_stmt ---
// void kcc_cg_prereg_stmt(node *stmt)
//   Walk a statement (and its AST_F4 sibling
   chain) and reserve storage for
//   every `static` local declaration at any block
    depth, descending into
//   compound blocks, if/switch/while/do/for
   bodies, and case/default/label
//   statements.  Without descending, a static
   declared inside a nested block
//   -- e.g. SQLite's `static const struct EncName
    encnames[]` inside a switch
//   case -- got an IR_GLOBAL_REF but no storage,
   leaving the symbol undefined.
.globl _kcc_cg_prereg_stmt
.p2align 2
_kcc_cg_prereg_stmt:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                  // stmt cursor
.Lprs_loop:
    cbz x19, .Lprs_ret
    ldr x20, [x19, #AST_TAG]
    cmp x20, #AST_VAR_DECL
    b.ne .Lprs_notvar
    ldr x0, [x19, #AST_F3]       // storage
   class (3 = static)
    cmp x0, #3
    b.ne .Lprs_next
    mov x0, x19
    bl _kcc_cg_prereg_static
    b .Lprs_next
.Lprs_notvar:
    cmp x20, #AST_COMPOUND
    b.eq .Lprs_body_f0
    cmp x20, #AST_IF_STMT
    b.eq .Lprs_if
    cmp x20, #AST_SWITCH_STMT
    b.eq .Lprs_body_f1
    cmp x20, #AST_WHILE_STMT
    b.eq .Lprs_body_f1
    cmp x20, #AST_DO_STMT
    b.eq .Lprs_body_f0
    cmp x20, #AST_FOR_STMT
    b.eq .Lprs_for
```

```
    cmp x20, #AST_CASE_STMT
    b.eq .Lprs_body_f1
    cmp x20, #AST_LABEL_STMT
    b.eq .Lprs_body_f1
    cmp x20, #AST_DEFAULT_STMT
    b.eq .Lprs_body_f0
    b .Lprs_next
.Lprs_if:
    ldr x0, [x19, #AST_F1]          // then branch
    bl _kcc_cg_prereg_stmt
    ldr x0, [x19, #AST_F2]          // else branch
    bl _kcc_cg_prereg_stmt
    b .Lprs_next
.Lprs_for:
    ldr x0, [x19, #AST_F0]          // init clause
    bl _kcc_cg_prereg_stmt
    ldr x0, [x19, #AST_F3]          // body
    bl _kcc_cg_prereg_stmt
    b .Lprs_next
.Lprs_body_f1:
    ldr x0, [x19, #AST_F1]
    bl _kcc_cg_prereg_stmt
    b .Lprs_next
.Lprs_body_f0:
    ldr x0, [x19, #AST_F0]
    bl _kcc_cg_prereg_stmt
.Lprs_next:
    ldr x19, [x19, #AST_F4]         // next sibling
    in the statement list
    b .Lprs_loop
.Lprs_ret:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

465 · defined at 465, 468a, 469 · used in 31c · defines _kcc_cg_prereg_static never used, _kcc_cg_prereg_stmt never used, _kcc_preregister_globals 603 · uses _kcc_arena_alloc 58a, _kcc_cg_defined_globals 479, _kcc_emit_rodata_global 469, _kcc_emit_rodata_ptrarr 469, _kcc_global_const_ptrarr 308, _kcc_global_const_rodata 308, _kcc_ht_put 63, _kcc_ir_static_name 417a, _kcc_linker_global_sym 593, _kcc_sema_resolve_type 259b, _kcc_type_layout 267, 268

35.4.1. Read-Only Const Globals

When `_kcc_global_const_rodata` (§29.1.1) accepts a declaration, pre-registration routes it here instead of reserving `__common` storage. `_kcc_emit_rodata_global` evaluates the initialiser to a byte image directly—an integer or character literal's value lives in its `AST_F0`, and a floating literal's is the IEEE-754 double bit pattern there (narrowed to single precision for a `float` element by a compile-time `fcvt`), so no expression lowering is needed—and appends it to the same growable buffer that holds string literals, which `_kcc_codegen_emit_strings` concatenates onto the read-only `__text` tail. Three bookkeeping steps make the object linkable and referenceable: a *section-1* symbol at the object's buffer offset (external unless the object is a `static` local); a `_kcc_cstr_fixups` entry so the symbol's value is patched to $code_size + offset$ alongside the string literals; and a `_kcc_linker_extern_ht` entry so that every `IR_GLOBAL_REF`—which dedups through `_kcc_linker_global_sym`—resolves to this read-only symbol rather than allocating a fresh writable slot. For a scalar or array the reservation spans $\max(layout,\ count \times elem)$ so a type whose inferred array length lags its initialiser cannot under-allocate, and the tail is zero-filled to match C's default initialisation of the unmentioned elements. A `struct` takes a parallel branch: it zeroes its whole `layout` span first (so inter-field padding and any unmentioned trailing fields are zero) and then writes each initialiser element at its member's recorded byte offset (`AST_F3`), with that member's own kind driving the width and the `float` narrowing—so a mixed `{int, double, char}` record is laid out byte-accurately.

⟨ *CodeGen: preregister globals* 468a ⟩ +≡

```
// -- kcc_rodata_leaf_bits ---
// uint64_t kcc_rodata_leaf_bits(node *n, uint64_t
    kind)
// Raw initializer bits for a leaf.  An integer
   element is a symtab-free constant
// expression (kcc_rodata_const_intexpr accepted
   it), so it folds through
// _kcc_eval_const_int -- literals, negation, and
   arithmetic alike.  A floating
// element is a literal or a unary minus of one;
   its double bit pattern comes
// straight from AST_F0, negation being a sign-bit
    flip.
.text
.globl _kcc_rodata_leaf_bits
.p2align 2
_kcc_rodata_leaf_bits:
    cmp x1, #TY_FLOAT               // integer
   kinds are < TY_FLOAT
    b.ge .Lrlb_fp
    b _kcc_eval_const_int           // integer:
   tail-call the constant folder
.Lrlb_fp:
    ldr x2, [x0, #AST_TAG]
    cmp x2, #AST_UNARY_EXPR
    b.ne .Lrlb_flit
    ldr x2, [x0, #AST_F1]           // operand
   literal
    ldr x0, [x2, #AST_F0]           // its double
   bit pattern
    mov x3, #1
    lsl x3, x3, #63
    eor x0, x0, x3                  // flip the
   sign bit
    ret
.Lrlb_flit:
    ldr x0, [x0, #AST_F0]
    ret
```

□

468a · defined at 465, 468a, 469 · used in 31c · defines _kcc_rodata_leaf_bits 469, 564 · uses _kcc_eval_const_int 426

⟨ *CodeGen: rodata write LE* 468b ⟩ ≡

```
    // A floating literal always carries the IEEE
   -754 *double* bit pattern in
    // AST_F0.  For a float element, narrow it to
   single precision (the value
    // model is double, but its 4-byte memory
   image is single) with a hardware
    // fcvt on the literal itself; a double
   element keeps all 8 bytes, an integer
    // element writes its value verbatim.
    ldr x11, [x23, #TY_KIND]
    cmp x11, #TY_FLOAT
    b.ne 3f
    fmov d0, x6
```

```
    fcvt s0, d0
    fmov w6, s0                     // x6 = single-
   precision bits (upper 32 = 0)
3:
    // Write x24 little-endian bytes of the value
   in x6 at x25[x26].
    mov x8, x6                      // value
    mov x9, #0                      // byte index
1:  cmp x9, x24
    b.ge 2f
    add x10, x26, x9
    strb w8, [x25, x10]
    lsr x8, x8, #8
    add x9, x9, #1
    b 1b
2:
```

□

468b · used in 469

⟨ CodeGen: preregister globals 469 ⟩ + ≡

```
// -- kcc_emit_struct_fields ---
// void kcc_emit_struct_fields(type *sty, node *
   init, char *buf, uint64_t base)
// Lay one struct's fields into buf at byte offset
    `base`, positionally from the
// brace list `init`.  The region is assumed
   already zeroed, so a short list
// leaves its trailing fields zero.  Shared by the
    scalar-struct and
// array-of-struct emit paths.  Preserves x19-x24/
   x27/x28; uses x25/x26 (the
// write fragment's buffer and cursor) as scratch.
.text
.globl _kcc_emit_struct_fields
.p2align 2
_kcc_emit_struct_fields:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x25, x2                     // buffer (for
   the write fragment)
    mov x22, x3                     // base byte
   offset
    ldr x0, [x0, #TY_F0]            //
   AST_STRUCT_DECL
    ldr x19, [x0, #AST_F1]          // x19 = field
   cursor
    ldr x20, [x1, #AST_F0]          // x20 = init
   element cursor
.Lesf_loop:
    cbz x19, .Lesf_done             // no more
   fields
    cbz x20, .Lesf_done             // initializer
   exhausted: rest stays zero
    ldr x0, [x19, #AST_F0]          // field type
    bl _kcc_sema_resolve_type
    mov x23, x0                     // field type (
   for the write fragment's fcvt)
    ldr x24, [x23, #TY_SIZE]        // field size
    ldr x1, [x23, #TY_KIND]         // field kind
    mov x0, x20                     // element node
    bl _kcc_rodata_leaf_bits
    mov x6, x0                      // field bits
    ldr x9, [x19, #AST_F3]          // field byte
   offset
    add x26, x22, x9                // write cursor
   = base + field offset
    ⟨ CodeGen: rodata write LE 468b ⟩
    ldr x19, [x19, #AST_F4]         // next field
    ldr x20, [x20, #AST_F4]         // next element
    b .Lesf_loop
.Lesf_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_emit_rodata_global ---
// void kcc_emit_rodata_global(node *decl, char *
   uname)
//   decl  = AST_VAR_DECL accepted by
   kcc_global_const_rodata
//   uname = linker symbol name ("_"+source or "_
   "+mangled), arena-owned
// Lays the initializer's byte image into the read
   -only cstring buffer and mints
// a section-1 symbol at that offset, with a fixup
    (value patched to
// code_size+offset) and an extern-table dedup
   entry so IR_GLOBAL_REF resolves
// here instead of allocating __common storage.
.text
.globl _kcc_emit_rodata_global
.p2align 2
_kcc_emit_rodata_global:
    stp x29, x30, [sp, #-112]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    stp x27, x28, [sp, #80]
    mov x19, x0                     // decl
    mov x20, x1                     // uname

    // Resolve type; total byte size; element type
    and size.
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    mov x21, x0                     // full type
    bl _kcc_type_layout             // x0 = total
   size
    mov x22, x0                     // x22 = total
   (becomes span)
    ldr x0, [x21, #TY_KIND]
    cmp x0, #TY_STRUCT              // a struct is
   laid out field-by-field
    b.eq .Lerg_struct
    cmp x0, #TY_ARRAY
    b.ne .Lerg_scalar_ty
    ldr x23, [x21, #TY_F0]          // element type
    ldr x0, [x23, #TY_KIND]
    cmp x0, #TY_STRUCT              // array of
   structs: lay each element out
    b.eq .Lerg_aos
    b .Lerg_have_ty
.Lerg_scalar_ty:
    mov x23, x21                    // scalar:
   element is the type itself
.Lerg_have_ty:
    ldr x24, [x23, #TY_SIZE]        // element size

    // Count elements; span = max(total, count*
   elem) guards a type whose
    // inferred array length lags the initializer.
    mov x9, #0
    ldr x0, [x19, #AST_F2]
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_INIT_LIST
    b.ne .Lerg_scalar_cnt
```

```
    ldr x2, [x0, #AST_F0]
.Lerg_cnt_loop:
    cbz x2, .Lerg_cnt_done
    add x9, x9, #1
    ldr x2, [x2, #AST_F4]
    b .Lerg_cnt_loop
.Lerg_scalar_cnt:
    mov x9, #1
.Lerg_cnt_done:
    mul x10, x9, x24
    cmp x10, x22
    csel x22, x10, x22, gt          // x22 = span

    // Align the buffer position up to the element
    alignment (remember pos).
    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    ldr x26, [x0]                   // x26 = pos
    ldr x9, [x23, #TY_ALIGN]
    sub x11, x9, #1
    add x8, x26, x11
    bic x27, x8, x11                // x27 = O (
   aligned start)

    // Reserve O+span; the buffer may relocate, so
    re-read it afterwards.
    add x0, x27, x22
    bl _kcc_cstr_reserve
    adrp x0, _kcc_cstr_buf@PAGE
    add  x0, x0, _kcc_cstr_buf@PAGEOFF
    ldr x25, [x0]                   // x25 = buffer
    (stable after reserve)

    // Zero the alignment padding [pos, O).
    mov x1, x26
.Lerg_pad:
    cmp x1, x27
    b.ge .Lerg_write
    strb wzr, [x25, x1]
    add x1, x1, #1
    b .Lerg_pad

.Lerg_write:
    mov x26, x27                    // x26 = w (
   write cursor), from O
    ldr x0, [x19, #AST_F2]
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_INIT_LIST
    b.ne .Lerg_scalar_wr
    ldr x28, [x0, #AST_F0]          // first
   element
.Lerg_wr_loop:
    cbz x28, .Lerg_zero
    mov x0, x28
    ldr x1, [x23, #TY_KIND]         // element kind
    (for negation / narrowing)
    bl _kcc_rodata_leaf_bits
    mov x6, x0                      // literal bits
    (negated if unary minus)
    ⟨ CodeGen: rodata write LE 468b ⟩
    add x26, x26, x24
    ldr x28, [x28, #AST_F4]
    b .Lerg_wr_loop
.Lerg_scalar_wr:
    ldr x1, [x23, #TY_KIND]         // x0 already =
    init node
    bl _kcc_rodata_leaf_bits
    mov x6, x0                      // scalar
   literal bits
    ⟨ CodeGen: rodata write LE 468b ⟩
    add x26, x26, x24

.Lerg_zero:
    // Zero-fill the tail [w, O+span).
    add x7, x27, x22
.Lerg_zf:
    cmp x26, x7
    b.ge .Lerg_sym
    strb wzr, [x25, x26]
    add x26, x26, #1
    b .Lerg_zf

.Lerg_struct:
    // Scalar struct: x21 = type, x22 = total size
    .  Align, reserve, zero the
    // whole region (so padding gaps and
    unmentioned fields are zero), then lay
    // out the fields with the shared helper.
    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    ldr x26, [x0]                   // pos
    ldr x9, [x21, #TY_ALIGN]
    sub x11, x9, #1
    add x8, x26, x11
    bic x27, x8, x11                // O (aligned
    start)
    add x0, x27, x22
    bl _kcc_cstr_reserve
    adrp x0, _kcc_cstr_buf@PAGE
    add  x0, x0, _kcc_cstr_buf@PAGEOFF
    ldr x25, [x0]                   // buffer
    add x7, x27, x22                // zero [pos, O+
    total)
    mov x1, x26
.Lerg_st_zero:
    cmp x1, x7
    b.ge .Lerg_st_lay
    strb wzr, [x25, x1]
    add x1, x1, #1
    b .Lerg_st_zero
.Lerg_st_lay:
    mov x0, x21                     // struct type
    ldr x1, [x19, #AST_F2]          // init list
    mov x2, x25                     // buffer
    mov x3, x27                     // base = O
    bl _kcc_emit_struct_fields
    b .Lerg_sym

.Lerg_aos:
    // Array of structs: x21 = array type, x23 =
    element struct type, x22 = total
    // size.  Reserve and zero, then lay each
    element out at O + i*stride, where
    // the stride is the element struct's (already
    padded) size.
    ldr x24, [x23, #TY_SIZE]        // stride
    // span = max(layout, count*stride) guards a
    lagging inferred array length.
    mov x9, #0
    ldr x0, [x19, #AST_F2]
    ldr x0, [x0, #AST_F0]           // first outer
    element
.Lerg_aos_cnt:
    cbz x0, .Lerg_aos_cnt_done
    add x9, x9, #1
    ldr x0, [x0, #AST_F4]
    b .Lerg_aos_cnt
.Lerg_aos_cnt_done:
    mul x10, x9, x24
    cmp x10, x22
    csel x22, x10, x22, gt          // x22 = span
    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    ldr x26, [x0]                   // pos
    ldr x9, [x23, #TY_ALIGN]
    sub x11, x9, #1
    add x8, x26, x11
    bic x27, x8, x11                // O
```

```
    add x0, x27, x22
    bl _kcc_cstr_reserve
    adrp x0, _kcc_cstr_buf@PAGE
    add  x0, x0, _kcc_cstr_buf@PAGEOFF
    ldr x28, [x0]                   // x28 = buffer
    (survives the helper)
    add x7, x27, x22              // zero [pos, 0+
   total)
    mov x1, x26
.Lerg_aos_zero:
    cmp x1, x7
    b.ge .Lerg_aos_lay
    strb wzr, [x28, x1]
    add x1, x1, #1
    b .Lerg_aos_zero
.Lerg_aos_lay:
    ldr x0, [x19, #AST_F2]          // outer init
   list
    ldr x0, [x0, #AST_F0]           // first element
    (a struct brace)
    str x0, [sp, #96]             // element cursor
    str x27, [sp, #104]           // running base
   offset = 0
.Lerg_aos_loop:
    ldr x0, [sp, #96]
    cbz x0, .Lerg_sym               // initializer
   exhausted: remaining structs zero
    mov x0, x23                     // element
   struct type
    ldr x1, [sp, #96]               // element init
   (brace list)
    mov x2, x28                     // buffer
    ldr x3, [sp, #104]            // base for this
   element
    bl _kcc_emit_struct_fields
    ldr x0, [sp, #104]
    add x0, x0, x24               // base += stride
    str x0, [sp, #104]
    ldr x0, [sp, #96]
    ldr x0, [x0, #AST_F4]           // next element
    str x0, [sp, #96]
    b .Lerg_aos_loop

.Lerg_sym:
    // Advance cstr_pos past this object.
    add x0, x27, x22
    adrp x1, _kcc_cstr_pos@PAGE
    add  x1, x1, _kcc_cstr_pos@PAGEOFF
    str x0, [x1]

    // Mint symbol: section 1 (__text), value 0 (
   patched later).  External
    // unless the object is a static local (
   storage class 3).
    ldr x0, [x19, #AST_F3]
    mov x3, #N_SECT
    cmp x0, #3
    b.eq .Lerg_flags
    orr x3, x3, #N_EXT
.Lerg_flags:
    mov x0, x20                     // name
    mov x1, #1                      // section 1
    mov x2, #0                      // value
   placeholder
    bl _kcc_linker_add_sym
    mov x21, x0                     // sym_index

    // Fixup {sym_index, 0}: emit_strings patches
   value to code_size + 0.
    str x21, [sp, #96]
    str x27, [sp, #104]
    adrp x0, _kcc_cstr_fixups@PAGE
    add  x0, x0, _kcc_cstr_fixups@PAGEOFF
    ldr x0, [x0]
    add x1, sp, #96
    bl _kcc_vec_push

    // Dedup entry: IR_GLOBAL_REF via
   kcc_linker_global_sym resolves here.
    mov x0, x20
    bl _kcc_strlen
    mov x2, x0                      // keylen
    adrp x0, _kcc_linker_extern_ht@PAGE
    add  x0, x0, _kcc_linker_extern_ht@PAGEOFF
    ldr x0, [x0]
    mov x1, x20
    add x3, x21, #1               // value =
   sym_index + 1
    bl _kcc_ht_put

    ldp x27, x28, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #112
    ret

// -- kcc_emit_rodata_ptrarr ---
// void kcc_emit_rodata_ptrarr(node *decl, char *
   uname)
//   decl accepted by kcc_global_const_ptrarr (a
   const array of const pointers
//        initialised with string literals / NULL)
//   uname = linker symbol name
// Reserves the array's pointer slots in the
   __DATA_CONST,__const section, emits
// each string element into the read-only cstring
   buffer and records an UNSIGNED
// relocation binding its slot to the string
   symbol (NULL slots stay zero), and
// mints a section-3 symbol at the array's offset.
     Marks the __const section
// present so the emitter lays it out.
.text
.globl _kcc_emit_rodata_ptrarr
.p2align 2
_kcc_emit_rodata_ptrarr:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]
    mov x19, x0                     // decl
    mov x20, x1                     // uname

    // The object now needs a __DATA_CONST,__const
   section.
    adrp x0, _kcc_rodata_const_present@PAGE
    add  x0, x0, _kcc_rodata_const_present@PAGEOFF
    mov x1, #1
    str x1, [x0]

    // Total array bytes (declared length * 8).
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    bl _kcc_type_layout             // x0 = total
   size
    mov x25, x0                     // x25 = total

    // 0 = current const size, aligned to 8 (
   pointer alignment).
    adrp x0, _kcc_rodata_const_size@PAGE
    add  x0, x0, _kcc_rodata_const_size@PAGEOFF
    ldr x21, [x0]
```

```
    add x21, x21, #7
    bic x21, x21, #7                // x21 = 0 (
   array's offset in __const)

    ldr x0, [x19, #AST_F2]          // init list
    ldr x22, [x0, #AST_F0]          // x22 =
   element cursor
    mov x23, #0                     // x23 = slot
   index
.Lerp_loop:
    cbz x22, .Lerp_done
    ldr x0, [x22, #AST_TAG]
    cmp x0, #AST_STR_LIT
    b.ne .Lerp_next                 // NULL / non-
   string leaves the slot zero
    ldr x0, [x22, #AST_F0]          // string data
    ldr x1, [x22, #AST_F1]          // string
   length
    bl _kcc_emit_cstring            // x0 = string
   symbol index
    mov x24, x0
    lsl x1, x23, #3                 // slot offset
   = 0 + i*8
    add x1, x21, x1
    str x1, [sp, #72]              // reloc {
   slot_offset, sym_index}
    str x24, [sp, #80]
    adrp x0, _kcc_rodata_const_relocs@PAGE
    add  x0, x0, _kcc_rodata_const_relocs@PAGEOFF
    ldr x0, [x0]
    add x1, sp, #72
    bl _kcc_vec_push
.Lerp_next:
    add x23, x23, #1
    ldr x22, [x22, #AST_F4]
    b .Lerp_loop
.Lerp_done:
    // span = max(total, slot_count*8) guards a
   lagging inferred array length.
    lsl x0, x23, #3
    cmp x0, x25
    csel x25, x0, x25, gt
    add x0, x21, x25                // new const
   size = 0 + span
    adrp x1, _kcc_rodata_const_size@PAGE
    add  x1, x1, _kcc_rodata_const_size@PAGEOFF
    str x0, [x1]

    // Mint the section-3 symbol at offset 0;
   external unless static local.
    ldr x0, [x19, #AST_F3]
    mov x3, #N_SECT
    cmp x0, #3
    b.eq .Lerp_flags
    orr x3, x3, #N_EXT
.Lerp_flags:
    mov x0, x20                     // name
    mov x1, #3                      // section 3 (
   __DATA_CONST,__const)
    mov x2, x21                     // value =
   offset in section
    bl _kcc_linker_add_sym
    mov x24, x0                     // sym index

    // Dedup entry so IR_GLOBAL_REF resolves to
   this symbol.
    mov x0, x20
    bl _kcc_strlen
    mov x2, x0
    adrp x0, _kcc_linker_extern_ht@PAGE
    add  x0, x0, _kcc_linker_extern_ht@PAGEOFF
    ldr x0, [x0]
    mov x1, x20
    add x3, x24, #1
    bl _kcc_ht_put

    ldr x25, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

469 · defined at 465, 468a, 469 · used in 31c · defines _kcc_emit_rodata_global 465, _kcc_emit_rodata_ptrarr 465, _kcc_emit_struct_fields never used · uses _kcc_cstr_buf 479, _kcc_cstr_fixups 479, _kcc_cstr_pos 479, _kcc_cstr_reserve 475, _kcc_emit_cstring 476, _kcc_ht_put 63, _kcc_linker_add_sym 592a, _kcc_linker_extern_ht 594, _kcc_rodata_const_present 479, _kcc_rodata_const_relocs 479, _kcc_rodata_const_size 479, _kcc_rodata_leaf_bits 468a, _kcc_sema_resolve_type 259b, _kcc_type_layout 267, 268, _kcc_vec_push 65a

35.5. Compile All Functions

The top-level entry point iterates the IR function vector and calls `kcc_codegen_func` for each one. After this loop completes, the code buffer contains the machine code for every function but string literal data has not yet been appended—that is a separate finalization step (§35.7).

Pre-registering function symbols. Before compiling any body, a first pass registers a defined linker symbol for every function, with a placeholder value of zero, and records its index in `IRF_SYMIDX`. Non-`static` functions are external (`N_SECT | N_EXT`); `static` functions and the synthesised module-init are local (`N_SECT`). This ordering is what makes a forward reference resolve correctly: when `deflate` calls the `static` helper `deflate_stored` that is defined *later* in the file, the call site's `kcc_linker_extern` lookup finds the already-registered *defined* symbol rather than minting a fresh undefined external. Without the pre-pass, that forward reference produced an `N_UNDF | N_EXT` symbol which the linker could coalesce with a global definition but never with a local one—so a `static` callee referenced before its definition failed to link. The subsequent `kcc_codegen_func` then only patches the pre-registered symbol's `SYM_VALUE` to the real code offset.

⟨ *CodeGen: compile all* 472 ⟩ ≡

```
// -- kcc_cg_preregister_funcs ---
// void kcc_cg_preregister_funcs(void)
// Registers a defined (value=0) linker symbol for
    every IR function, records
// the index in IRF_SYMIDX, and enters non-modinit
    functions into the extern
// hash table so forward call sites resolve to the
    definition.
.text
.globl _kcc_cg_preregister_funcs
.p2align 2
_kcc_cg_preregister_funcs:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
```

```
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    adrp x0, _kcc_ir_funcs@PAGE
    add  x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lcpf_done
    ldr x19, [x0, #VEC_COUNT]       // function
   count
    mov x20, #0                     // index

.Lcpf_loop:
    cmp x20, x19
    b.ge .Lcpf_done
    adrp x0, _kcc_ir_funcs@PAGE
    add  x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x0, [x0]
    mov x1, x20
    bl _kcc_vec_get
    ldr x21, [x0]                   // x21 =
   ir_func
    ldr x22, [x21, #IRF_NAME]       // x22 = name

    // is_modinit = (name == "kcc.module_init")
    mov x0, x22
    adrp x1, _kcc_str_modinit@PAGE
    add  x1, x1, _kcc_str_modinit@PAGEOFF
    bl _kcc_strcmp
    cmp x0, #0
    cset x23, eq                    // x23 =
   is_modinit

    // Build "_" + name
    mov x0, x22
    bl _kcc_strlen
    add x0, x0, #2
    bl _kcc_arena_alloc
    mov x24, x0                     // x24 =
   mangled name buffer
    mov w1, #'_'
    strb w1, [x24]
    mov x0, x22
    bl _kcc_strlen
    add x2, x0, #1                  // len incl.
   null
    add x0, x24, #1
    mov x1, x22
    bl _kcc_memcpy

    // Linkage flags: external unless modinit or
   static
    mov x3, #(N_SECT | N_EXT)
    cbnz x23, .Lcpf_local
    ldr x4, [x21, #IRF_STATIC]
    cbz x4, .Lcpf_flags
.Lcpf_local:
    mov x3, #N_SECT
.Lcpf_flags:
    mov x0, x24                     // name
    mov x1, #1                      // section 1 (
   __text)
    mov x2, #0                      // value
   placeholder (patched by codegen_func)
    bl _kcc_linker_add_sym
    str x0, [x21, #IRF_SYMIDX]      // record index
    on the IR function

    // modinit: record its symbol index and stay
   out of the extern table.
    cbz x23, .Lcpf_extern
    adrp x1, _kcc_modinit_sym@PAGE
    add  x1, x1, _kcc_modinit_sym@PAGEOFF
    add x2, x0, #1                  // index + 1 (0
    = none)
    str x2, [x1]
    b .Lcpf_next

.Lcpf_extern:
    // Enter into the extern hash table so calls
   by name find this definition.
    mov x22, x0                     // save sym
   index
    mov x0, x24
    bl _kcc_strlen
    mov x2, x0                      // keylen
    adrp x0, _kcc_linker_extern_ht@PAGE
    add  x0, x0, _kcc_linker_extern_ht@PAGEOFF
    ldr x0, [x0]
    mov x1, x24                     // key =
   mangled name
    add x3, x22, #1                 // value = index
    + 1
    bl _kcc_ht_put

.Lcpf_next:
    add x20, x20, #1
    b .Lcpf_loop

.Lcpf_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

472 · defined at 472, 473 · used in 31c · defines _kcc_cg_preregister_funcs 473, 498b · uses _kcc_arena_alloc 58a, _kcc_ht_put 63, _kcc_ir_funcs 305c, _kcc_linker_add_sym 592a, _kcc_linker_extern_ht 594, _kcc_modinit_sym 305c, _kcc_vec_get 65b

⟨ CodeGen: compile all 473 ⟩ + ≡

```
// -- kcc_codegen_all ---
// void kcc_codegen_all(void)
// Iterates all IR functions and generates code
   for each.
.text
.globl _kcc_codegen_all
.p2align 2
_kcc_codegen_all:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    // Pre-register every function symbol so
   forward references resolve.
    bl _kcc_cg_preregister_funcs

    // Get IR functions vec
    adrp x0, _kcc_ir_funcs@PAGE
    add  x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lcga_done

    ldr x19, [x0, #VEC_COUNT]       // number of
   functions
    mov x20, #0                     // index

.Lcga_loop:
    cmp x20, x19
    b.ge .Lcga_done
```

```
    // Get function pointer: vec_get(funcs_vec,
   index)
    adrp x0, _kcc_ir_funcs@PAGE
    add  x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x0, [x0]
    mov x1, x20
    bl _kcc_vec_get
    ldr x0, [x0]                    // dereference:
   vec stores pointers

    bl _kcc_codegen_func

    add x20, x20, #1
    b .Lcga_loop

.Lcga_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

473 · defined at 472, 473 · used in 31c · defines _kcc_codegen_all 603 · uses _kcc_cg_preregister_funcs 472, _kcc_codegen_func 493, _kcc_ir_funcs 305c, _kcc_vec_get 65b

35.6. Branch Patching

After all basic blocks have been emitted, we resolve forward branch references. Each entry in the patch vector contains a code offset and a target BB ID. We look up the target in the label hash table, compute the signed byte offset, and call code_patch32 to overwrite the placeholder. Two instruction formats need handling: B (where the offset occupies the low 26 bits) and CBNZ (where the 19-bit offset sits at bits [23:5]). We distinguish the two by reading the existing instruction and checking bit 24.

⟨ CodeGen: patch branches 474 ⟩ ≡

```
// -- kcc_codegen_patch_branches ---
// void kcc_codegen_patch_branches(ht *bb_labels,
   vec *patches)
//   x0 = bb_labels hash table
//   x1 = patch vec
.text
.globl _kcc_codegen_patch_branches
.p2align 2
_kcc_codegen_patch_branches:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    str x23, [sp, #48]

    mov x19, x0                     // bb_labels ht
    mov x20, x1                     // patch vec
    ldr x21, [x20, #VEC_COUNT]      // patch count
    mov x22, #0                     // index

.Lcpb_loop:
    cmp x22, x21
    b.ge .Lcpb_done

    // Get patch entry
    mov x0, x20
    mov x1, x22
    bl _kcc_vec_get
    mov x23, x0                     // pointer to
   patch entry

    ldr x0, [x23, #PATCH_OFFSET]   // code offset
   of branch instr
    ldr x1, [x23, #PATCH_TARGET]   // target BB_ID

    // Look up target BB offset
    str x1, [sp, #56]               // store bb_id
   as key
    mov x0, x19                     // ht
    add x1, sp, #56                 // key
    mov x2, #8                      // keylen
    bl _kcc_ht_get
    cbz x0, .Lcpb_next              // target not
   found (shouldn't happen)
    sub x0, x0, #1                  // undo +1
   encoding → target_offset

    // Read original instruction to determine type
    ldr x1, [x23, #PATCH_OFFSET]   // branch_offset
    adrp x2, _kcc_code_buf@PAGE
    add  x2, x2, _kcc_code_buf@PAGEOFF
    ldr x2, [x2]
    ldr w3, [x2, x1]                // original
   instruction

    // Compute offset in bytes: target -
   branch_pos
    sub x4, x0, x1                  // offset_bytes
   = target_offset - branch_offset

    // Check instruction type: B (0x14) vs CBNZ (0
   xB5) vs CBZ (0xB4)
    lsr w5, w3, #24
    cmp w5, #0x14                   // B?
    b.eq .Lcpb_patch_b
    cmp w5, #0xB5                   // CBNZ 64-bit?
    b.eq .Lcpb_patch_cbnz
    cmp w5, #0x35                   // CBNZ 32-bit?
    b.eq .Lcpb_patch_cbnz
    // Default: treat as B
    b .Lcpb_patch_b

.Lcpb_patch_b:
    // Re-encode B with correct offset
    mov x0, x4                      // offset_bytes
    bl _kcc_enc_b
    // Patch
    ldr x1, [x23, #PATCH_OFFSET]
    mov x1, x1
    mov w1, w0                      // new
   instruction in w1... wait
    // kcc_code_patch32(offset, word): x0=offset,
   x1=word
    mov w2, w0                      // save encoded
    instruction
    ldr x0, [x23, #PATCH_OFFSET]
    mov w1, w2
    bl _kcc_code_patch32
    b .Lcpb_next

.Lcpb_patch_cbnz:
    // Re-encode CBNZ: keep rt and sf from
   original, update imm19
    and w5, w3, #0x1F               // extract Rt
    asr x6, x4, #2                  // offset in
   instructions
    and w6, w6, #0x7FFFF            // imm19
    mov w7, #0xB5000000              // CBNZ 64-bit
   base
    orr w7, w7, w6, lsl #5          // imm19
    orr w7, w7, w5                  // Rt
```

```
    // Patch
    ldr x0, [x23, #PATCH_OFFSET]
    mov w1, w7
    bl _kcc_code_patch32
    b .Lcpb_next

.Lcpb_next:
    add x22, x22, #1
    b .Lcpb_loop

.Lcpb_done:
    ldr x23, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

474 · used in 31c · defines _kcc_codegen_patch_branches 498b · uses _kcc_code_buf 533c, _kcc_code_patch32 526a, _kcc_enc_b 532a, _kcc_ht_get 62, _kcc_vec_get 65b

35.7. String Literal Codegen

String literals require two things that no other IR instruction needs: persistent data that outlives the instruction, and a PC-relative pointer to that data. Our approach avoids adding a separate Mach-O section (which would change section counts, segment sizes, and symbol numbering): we append the raw string bytes to the *end* of the `__text` section, after all machine code.

The scheme works in two phases:

Phase 1: per-instruction (`kcc_codegen_string_lit`). Each `IR_STRING_LIT` instruction triggers the following steps:

1. Copy the string bytes (already escape-processed by the parser) into the *cstring buffer*, a 4 KB arena allocation, and append a null terminator.
2. Mint a local symbol name `l_.str.`N using a monotonically increasing counter. Because the final offset is not yet known, the symbol is created with `value = 0`.
3. Record a fixup entry $(sym_index, cstr_offset)$ so the value can be patched later.
4. Emit an ADRP + ADD pair with `PAGE21` and `PAGEOFF12` relocations. The linker will resolve these to the symbol's final address.
5. Store the resulting pointer (in x11) to the destination vreg's stack slot.

Phase 2: finalization (`kcc_codegen_emit_strings`). Called from the driver after `kcc_codegen_all` and before `kcc_macho_init`:

1. Record the current code position as the boundary between machine code and string data.
2. Walk the fixup vector and set each symbol's value to $code_size + cstr_offset$.
3. Copy the entire cstring buffer to the end of the code buffer (growing it if necessary).

The result is a single contiguous `__text` section where the first part is executable instructions and the tail is read-only string data—exactly the layout that Apple's linker expects.

Because that read-only tail now carries more than string literals—under M5 it also holds the byte images of `const` globals (§B.7)—the buffer can no longer be a fixed 4 KB block. `_kcc_cstr_reserve` ensures the buffer can hold a requested end offset, doubling (allocate-and-copy, mirroring the code buffer's growth) when it would overflow. Every writer calls it before appending and re-reads `_kcc_cstr_buf` afterwards, so a relocation of the buffer is transparent. This also closes a latent overflow: a translation unit with more than 4 KB of string data previously wrote past the block.

⟨ *CodeGen: emit strings* 475 ⟩ ≡

```
// -- kcc_cstr_reserve ---
// void kcc_cstr_reserve(uint64_t need)
// Ensure the cstring buffer holds at least `need`
    bytes; grow (allocate 2x or
// `need`, whichever is larger, and copy the live
   prefix) otherwise.  Preserves
// x19-x28 so callers holding state across it are
   safe.
.text
.globl _kcc_cstr_reserve
.p2align 2
_kcc_cstr_reserve:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                     // need (
   required end offset)
    adrp x0, _kcc_cstr_cap@PAGE
    add  x0, x0, _kcc_cstr_cap@PAGEOFF
    ldr x0, [x0]                    // current
   capacity
    cmp x19, x0
    b.le .Lcrsv_done                // already fits

    lsl x20, x0, #1                 // candidate
   new cap = cap*2
    cmp x20, x19
    b.ge .Lcrsv_alloc
    mov x20, x19                    // ...but at
   least `need`
.Lcrsv_alloc:
    mov x0, x20
    bl _kcc_arena_alloc             // x0 = new
   buffer
    mov x21, x0                     // x21 = new
   buffer

    // Copy the live prefix (cstr_pos bytes) from
   old buffer to new.
    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    ldr x2, [x0]                    // n = live
   bytes
    adrp x0, _kcc_cstr_buf@PAGE
    add  x0, x0, _kcc_cstr_buf@PAGEOFF
    ldr x1, [x0]                    // src = old
   buffer
```

```
    mov x0, x21                     // dst = new
   buffer
    bl _kcc_memcpy

    adrp x0, _kcc_cstr_buf@PAGE
    add  x0, x0, _kcc_cstr_buf@PAGEOFF
    str x21, [x0]                   // publish new
   buffer
    adrp x0, _kcc_cstr_cap@PAGE
    add  x0, x0, _kcc_cstr_cap@PAGEOFF
    str x20, [x0]                   // publish new
   capacity

.Lcrsv_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

475 · defined at 475, 476, 477ab, 478 · used in 31c · defines _kcc_cstr_reserve 469, 476, 477a · uses _kcc_arena_alloc 58a, _kcc_cstr_buf 479, _kcc_cstr_pos 479

⟨ CodeGen: emit strings 476 ⟩ + ≡

```
// -- kcc_emit_cstring ---
// uint64_t kcc_emit_cstring(const char *data,
   uint64_t len)
// Copy a string into the read-only cstring buffer
    (NUL-terminated), mint a
// local l_.str.N symbol at its offset with a
   fixup (value patched to
// code_size+offset alongside the other string
   literals), and return the symbol
// index.  This is the string-materialisation half
    of kcc_codegen_string_lit,
// used to build the string elements of a const
   pointer table (M5 Stage 2).
.text
.globl _kcc_emit_cstring
.p2align 2
_kcc_emit_cstring:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    mov x20, x0                     // data
    mov x21, x1                     // len

    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    ldr x23, [x0]                   // x23 =
   cstr_offset
    add x0, x23, x21
    add x0, x0, #1
    bl _kcc_cstr_reserve
    adrp x0, _kcc_cstr_buf@PAGE
    add  x0, x0, _kcc_cstr_buf@PAGEOFF
    ldr x0, [x0]
    add x0, x0, x23
    mov x1, x20
    mov x2, x21
    cbz x2, .Lecs_nullterm
    bl _kcc_memcpy
.Lecs_nullterm:
    adrp x0, _kcc_cstr_buf@PAGE
    add  x0, x0, _kcc_cstr_buf@PAGEOFF
    ldr x0, [x0]
    add x1, x23, x21
    strb wzr, [x0, x1]
    add x24, x23, x21
    add x24, x24, #1
    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    str x24, [x0]

    // Mint a local l_.str.N symbol.
    adrp x0, _kcc_cstr_count@PAGE
    add  x0, x0, _kcc_cstr_count@PAGEOFF
    ldr x24, [x0]
    add x1, x24, #1
    str x1, [x0]
    mov x0, #20
    bl _kcc_arena_alloc
    mov x25, x0
    mov w1, #'l'
    strb w1, [x25, #0]
    mov w1, #'_'
    strb w1, [x25, #1]
    mov w1, #'.'
    strb w1, [x25, #2]
    mov w1, #'s'
    strb w1, [x25, #3]
    mov w1, #'t'
    strb w1, [x25, #4]
    mov w1, #'r'
    strb w1, [x25, #5]
    mov w1, #'.'
    strb w1, [x25, #6]
    add x0, x25, #7
    mov x1, x24
    cbz x1, .Lecs_zero
    mov x2, x0
.Lecs_cvt:
    cbz x1, .Lecs_rev
    mov x3, #10
    udiv x4, x1, x3
    msub x5, x4, x3, x1
    add w5, w5, #'0'
    strb w5, [x0], #1
    mov x1, x4
    b .Lecs_cvt
.Lecs_rev:
    strb wzr, [x0]
    sub x3, x0, #1
.Lecs_rvlp:
    cmp x2, x3
    b.ge .Lecs_named
    ldrb w4, [x2]
    ldrb w5, [x3]
    strb w5, [x2], #1
    strb w4, [x3], #-1
    b .Lecs_rvlp
.Lecs_zero:
    mov w1, #'0'
    strb w1, [x0]
    strb wzr, [x0, #1]
.Lecs_named:
    mov x0, x25
    mov x1, #1                      // section 1 (
   __text)
    mov x2, #0                      // value
   placeholder
    mov x3, #N_SECT
    bl _kcc_linker_add_sym
    mov x26, x0                     // sym index
    str x26, [sp, #80]
    str x23, [sp, #88]
    adrp x0, _kcc_cstr_fixups@PAGE
    add  x0, x0, _kcc_cstr_fixups@PAGEOFF
    ldr x0, [x0]
    add x1, sp, #80
    bl _kcc_vec_push
```

```
    mov x0, x26                     // return the
   symbol index

    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

476 · defined at 475, 476, 477ab, 478 · used in 31c · defines _kcc_emit_cstring 469 · uses _kcc_arena_alloc 58a, _kcc_cstr_buf 479, _kcc_cstr_count 479, _kcc_cstr_fixups 479, _kcc_cstr_pos 479, _kcc_cstr_reserve 475, _kcc_linker_add_sym 592a, _kcc_vec_push 65a

⟨ CodeGen: emit strings 477a ⟩ + ≡

```
// -- kcc_codegen_string_lit ---
// void kcc_codegen_string_lit(ir_instr *instr)
// Handles IR_STRING_LIT: copies string to cstring
    buffer, creates a
// local symbol with ADRP+ADD relocations, stores
   pointer to dest vreg.
.text
.globl _kcc_codegen_string_lit
.p2align 2
_kcc_codegen_string_lit:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]

    mov x19, x0                     // instr
    ldr x20, [x19, #IR_SRC1]       // string data
   ptr
    ldr x21, [x19, #IR_SRC2]       // string length
    ldr x22, [x19, #IR_DEST]       // dest vreg

    // 1. Get current cstring buffer position
    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    ldr x23, [x0]                   // x23 =
   cstr_offset

    // 1a. Ensure room for len+1 bytes (data + NUL
   ) at cstr_offset; the buffer
    // may relocate, so every later access re-
   reads _kcc_cstr_buf.
    add x0, x23, x21
    add x0, x0, #1
    bl _kcc_cstr_reserve

    // 2. Copy string data to cstring buffer
    adrp x0, _kcc_cstr_buf@PAGE
    add  x0, x0, _kcc_cstr_buf@PAGEOFF
    ldr x0, [x0]
    add x0, x0, x23                 // dest = buf +
    offset
    mov x1, x20                     // src = string
    data
    mov x2, x21                     // len
    cbz x2, .Lcgsl_nullterm
    bl _kcc_memcpy

.Lcgsl_nullterm:
    // 3. Null-terminate
    adrp x0, _kcc_cstr_buf@PAGE
    add  x0, x0, _kcc_cstr_buf@PAGEOFF
    ldr x0, [x0]
    add x1, x23, x21
    strb wzr, [x0, x1]

    // 4. Advance cstr_pos
    add x24, x23, x21
    add x24, x24, #1
    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    str x24, [x0]
```

□

477a · defined at 475, 476, 477ab, 478 · used in 31c · defines _kcc_codegen_string_lit 521 · uses _kcc_cstr_buf 479, _kcc_cstr_pos 479, _kcc_cstr_reserve 475

Symbol Name Construction and Relocation Emission With the string bytes safely buffered, the next step is to give the literal a linker-visible identity. The code mints a local symbol name of the form `l_.str.`*N* by incrementing a monotonic counter and converting it to decimal with a digit-reversal loop. It then registers the symbol as *local* (`N_SECT` without `N_EXT`: a string literal belongs to its own translation unit, so it must not be exported—else two objects that each mint `l_.str.0` collide at link time), in section 1 with value 0 as a placeholder, records a fixup pair $(sym_index, cstr_offset)$, and emits the ADRP + ADD relocation sequence that will let the linker resolve the pointer at link time.

⟨ CodeGen: emit strings 477b ⟩ + ≡

```
    // 5. Build symbol name "l_.str.N"
    adrp x0, _kcc_cstr_count@PAGE
    add  x0, x0, _kcc_cstr_count@PAGEOFF
    ldr x24, [x0]                   // x24 = N
    add x1, x24, #1
    str x1, [x0]                    // count++

    mov x0, #20
    bl _kcc_arena_alloc
    mov x25, x0                     // x25 = name
   buffer

    // Write "l_.str." prefix (7 bytes)
    mov w1, #'l'
    strb w1, [x25, #0]
    mov w1, #'_'
    strb w1, [x25, #1]
    mov w1, #'.'
    strb w1, [x25, #2]
    mov w1, #'s'
    strb w1, [x25, #3]
    mov w1, #'t'
    strb w1, [x25, #4]
    mov w1, #'r'
    strb w1, [x25, #5]
    mov w1, #'.'
    strb w1, [x25, #6]

    // Convert N to decimal at offset 7
    add x0, x25, #7
    mov x1, x24
    cbz x1, .Lcgsl_zero

    mov x2, x0
.Lcgsl_cvt:
    cbz x1, .Lcgsl_rev
```

```
    mov x3, #10
    udiv x4, x1, x3
    msub x5, x4, x3, x1
    add w5, w5, #'0'
    strb w5, [x0], #1
    mov x1, x4
    b .Lcgsl_cvt

.Lcgsl_rev:
    strb wzr, [x0]
    sub x3, x0, #1
.Lcgsl_rvlp:
    cmp x2, x3
    b.ge .Lcgsl_named
    ldrb w4, [x2]
    ldrb w5, [x3]
    strb w5, [x2], #1
    strb w4, [x3], #-1
    b .Lcgsl_rvlp

.Lcgsl_zero:
    mov w1, #'0'
    strb w1, [x0]
    strb wzr, [x0, #1]

.Lcgsl_named:
    // 6. Add symbol: section 1, value 0 (fixed up
    later).  A string literal is
    // private to its translation unit, so its
   symbol is LOCAL (N_SECT, no
    // N_EXT); otherwise two objects that both
   mint `l_.str.0` collide at link.
    mov x0, x25
    mov x1, #1                      // section = 1 (
   __text)
    mov x2, #0                      // value = 0 (
   placeholder)
    mov x3, #N_SECT
    bl _kcc_linker_add_sym
    mov x26, x0                      // x26 =
   sym_index

    // 7. Record fixup {sym_index, cstr_offset}
    str x26, [sp, #80]
    str x23, [sp, #88]
    adrp x0, _kcc_cstr_fixups@PAGE
    add  x0, x0, _kcc_cstr_fixups@PAGEOFF
    ldr x0, [x0]
    add x1, sp, #80
    bl _kcc_vec_push

    // 8. Emit ADRP x11, sym@PAGE
    bl _kcc_code_current_pos
    mov x1, #ARM64_RELOC_PAGE21
    mov x2, x26
    bl _kcc_linker_add_reloc

    mov x0, #CG_REG_DST
    mov x1, #0
    bl _kcc_enc_adrp
    bl _kcc_code_emit32

    // 9. Emit ADD x11, x11, sym@PAGEOFF
    bl _kcc_code_current_pos
    mov x1, #ARM64_RELOC_PAGEOFF12
    mov x2, x26
    bl _kcc_linker_add_reloc

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_DST
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_add_imm
    bl _kcc_code_emit32

    // 10. Store x11 to dest vreg
    mov x0, x22
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg

    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

477b · defined at 475, 476, 477ab, 478 · used in 31c · uses _kcc_arena_alloc 58a, _kcc_cg_store_vreg 499b, _kcc_code_current_pos 526a, _kcc_code_emit32 525b, _kcc_cstr_count 479, _kcc_cstr_fixups 479, _kcc_enc_add_imm 527a, _kcc_enc_adrp 533b, _kcc_linker_add_reloc 592b, _kcc_linker_add_sym 592a, _kcc_vec_push 65a

String Data Finalization After all functions have been compiled, the string data must be appended to the code buffer so that the ADRP + ADD relocations resolve to valid addresses within the `__text` section. The finalization pass first records the current code position as the boundary between executable instructions and read-only data. It then walks the fixup vector and patches each string symbol's value to $code_size + cstr_offset$. Finally, it copies the accumulated cstring buffer to the tail of the code buffer, growing the buffer if the combined size exceeds its capacity.

⟨ CodeGen: emit strings 478 ⟩ +≡

```
// -- kcc_codegen_emit_strings ---
// void kcc_codegen_emit_strings(void)
// Fixes up string symbol values and appends
   cstring data to code buffer.
// Must be called after kcc_codegen_all() and
   before kcc_macho_init().
.text
.globl _kcc_codegen_emit_strings
.p2align 2
_kcc_codegen_emit_strings:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]

    // Check if any strings were emitted
    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lces_done

    // 1. Boundary between code and cstring data
    bl _kcc_code_current_pos
    mov x19, x0                      // x19 =
   code_size

    // 2. Patch each string symbol's value
    adrp x0, _kcc_cstr_fixups@PAGE
    add  x0, x0, _kcc_cstr_fixups@PAGEOFF
    ldr x20, [x0]                    // fixup vec
    ldr x21, [x20, #VEC_COUNT]
    mov x22, #0

.Lces_fixup_loop:
    cmp x22, x21
```

```
    b.ge .Lces_append

    mov x0, x20
    mov x1, x22
    bl _kcc_vec_get
    ldr x23, [x0]                    // sym_index
    ldr x24, [x0, #8]                // cstr_offset

    // value = code_size + cstr_offset
    add x24, x19, x24

    // Update symbol entry
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    ldr x0, [x0, #VEC_DATA]
    mov x1, #SYM_ENTRY_SIZE
    mul x1, x23, x1
    add x0, x0, x1
    str x24, [x0, #SYM_VALUE]

    add x22, x22, #1
    b .Lces_fixup_loop

.Lces_append:
    // 3. Append cstring data to code buffer
    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    ldr x23, [x0]                    // cstr total
   size

    adrp x0, _kcc_code_pos@PAGE
    add  x0, x0, _kcc_code_pos@PAGEOFF
    ldr x20, [x0]                    // code_pos

    adrp x0, _kcc_code_cap@PAGE
    add  x0, x0, _kcc_code_cap@PAGEOFF
    ldr x21, [x0]                    // code_cap

    // Grow if needed
    add x24, x20, x23
    cmp x24, x21
    b.le .Lces_copy

    lsl x0, x24, #1
    adrp x1, _kcc_code_cap@PAGE
    add  x1, x1, _kcc_code_cap@PAGEOFF
    str x0, [x1]
    bl _kcc_arena_alloc
    mov x21, x0

    adrp x0, _kcc_code_buf@PAGE
    add  x0, x0, _kcc_code_buf@PAGEOFF
    ldr x1, [x0]
    str x21, [x0]

    mov x0, x21
    mov x2, x20
    bl _kcc_memcpy

.Lces_copy:
    adrp x0, _kcc_code_buf@PAGE
    add  x0, x0, _kcc_code_buf@PAGEOFF
    ldr x0, [x0]
    add x0, x0, x20

    adrp x1, _kcc_cstr_buf@PAGE
    add  x1, x1, _kcc_cstr_buf@PAGEOFF
    ldr x1, [x1]

    mov x2, x23
    bl _kcc_memcpy

    // Update code_pos
    add x20, x20, x23

    adrp x0, _kcc_code_pos@PAGE
    add  x0, x0, _kcc_code_pos@PAGEOFF
    str x20, [x0]

.Lces_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

478 · defined at 475, 476, 477ab, 478 · used in 31c · defines _kcc_codegen_emit_strings 603 · uses _kcc_arena_alloc 58a, _kcc_code_buf 533c, _kcc_code_cap 533c, _kcc_code_current_pos 526a, _kcc_code_pos 533c, _kcc_cstr_buf 479, _kcc_cstr_fixups 479, _kcc_cstr_pos 479, _kcc_linker_syms 594, _kcc_vec_get 65b

35.8. CodeGen Data

Module-level mutable state for the code generator: the current function's frame size (set per-function, read by the instruction dispatcher for alloca and epilogue emission) and four words for the cstring subsystem—the data buffer, write position, literal counter, and fixup vector pointer.

⟨ CodeGen: data 479 ⟩ ≡

```
// -- CodeGen data ---
.data
.p2align 3
.globl _kcc_cg_frame_size
_kcc_cg_frame_size:  .quad 0     // current
   function's frame size
.globl _kcc_cg_vreg_count
_kcc_cg_vreg_count:  .quad 0     // current
   function's vreg count
.globl _kcc_cg_slotmap
_kcc_cg_slotmap:     .quad 0     // per-function
   →vregcompacted spill-slot map
                                 // (0 = identity:
    slot index == vreg number)
.globl _kcc_cg_max_slot
_kcc_cg_max_slot:    .quad 0     // number of vreg
    value slots reserved in the frame
.globl _kcc_cstr_buf
_kcc_cstr_buf:       .quad 0     // cstring data
   buffer
.globl _kcc_cstr_cap
_kcc_cstr_cap:       .quad 0     // capacity (
   bytes) of the cstring buffer
.globl _kcc_cstr_pos
_kcc_cstr_pos:       .quad 0     // write position
    in cstring buffer
.globl _kcc_cstr_count
_kcc_cstr_count:     .quad 0     // string literal
    counter (for naming)
.globl _kcc_cstr_fixups
_kcc_cstr_fixups:    .quad 0     // vec of {
   sym_index, cstr_offset} fixups
.globl _kcc_rodata_const_present
_kcc_rodata_const_present: .quad 0 // 1 if a
   __DATA_CONST,__const section is needed
.globl _kcc_rodata_const_size
_kcc_rodata_const_size: .quad 0  // running byte
   size of the __const section
.globl _kcc_rodata_const_relocs
```

```
_kcc_rodata_const_relocs: .quad 0 // vec of {
   slot_offset, sym_index} for __const
.globl _kcc_rodata_const_fileoff
_kcc_rodata_const_fileoff: .quad 0 // file offset
   of __const data (set in finalize)
.globl _kcc_cg_defined_globals
_kcc_cg_defined_globals: .quad 0 // hash set:
   globals defined in this TU
.globl _kcc_cg_regmap
_kcc_cg_regmap:       .quad 0      // per-function
   →vregphysreg map (0 = all-spill)
.globl _kcc_cg_promoted
_kcc_cg_promoted:     .quad 0      // per-function
   map: alloca vreg → physreg holding
                                  // its CONTENTS (
   mem2reg), or 0 if not promoted
.globl _kcc_cg_callee_mask
_kcc_cg_callee_mask: .quad 0      // bitmask of
   callee-saved regs used (bit r-19)
.globl _kcc_cg_save_count
_kcc_cg_save_count:   .quad 0      // number of
   callee-saved regs to save
.globl _kcc_cg_save_base
_kcc_cg_save_base:    .quad 0      // byte offset of
    the callee-save area from x29
.globl _kcc_cg_fpregmap
_kcc_cg_fpregmap:     .quad 0      // per-function
   FP →vregd-register map (0 = spill)
.globl _kcc_cg_fp_callee_mask
_kcc_cg_fp_callee_mask: .quad 0   // bitmask of
   callee-saved d-regs used (bit d-8)
.globl _kcc_cg_fp_save_count
_kcc_cg_fp_save_count:   .quad 0  // number of
   callee-saved d-regs to save
.globl _kcc_cg_fp_save_base
_kcc_cg_fp_save_base:    .quad 0  // byte offset of
    the FP callee-save area from x29
```

□

479 · used in 31c · defines _kcc_cg_callee_mask 481, 499b, _kcc_cg_defined_globals 463b, 465, 521, _kcc_cg_fpregmap 481, 490, 499b, _kcc_cg_fp_callee_mask 481, 499b, _kcc_cg_fp_save_base 493, 499b, _kcc_cg_fp_save_count 481, 493, _kcc_cg_frame_size 463b, 493, 513, 517, _kcc_cg_max_slot 481, 490, 493, 513, _kcc_cg_promoted 481, 490, 493, 499b, _kcc_cg_regmap 481, 490, 499b, _kcc_cg_save_base 493, 499b, _kcc_cg_save_count 481, 493, _kcc_cg_slotmap 481, 490, 499a, _kcc_cg_vreg_count 493, _kcc_cstr_buf 463b, 469, 475, 476, 477a, 478, _kcc_cstr_count 463b, 476, 477b, _kcc_cstr_fixups 463b, 469, 476, 477b, 478, _kcc_cstr_pos 463b, 469, 475, 476, 477a, 478, 537, _kcc_rodata_const_fileoff 580, 582, 583, _kcc_rodata_const_present 463b, 469, 580, 582, 583, 586, 587, _kcc_rodata_const_relocs 463b, 469, 580, 583, 587, _kcc_rodata_const_size 463b, 469, 580, 582, 583, 586

35.9. Register Allocation

The baseline back end is an *all-spill* model: every virtual register owns an eight-byte stack slot, and every instruction loads its operands from those slots and stores its result back. That is simple and always correct, but it pays a memory round-trip for every value. Register allocation removes the traffic for the values that can safely live in a physical register.

The allocator here is a linear scan over two register banks: the four caller-saved scratch registers `x12`–`x15` and the nine callee-saved registers `x19`–`x27`. The generated code otherwise touches only `x0`–`x11` and `x29`, so all thirteen are free for allocation. A virtual register is a candidate only when it is safe beyond any doubt, so the pass allocates a vreg to a physical register precisely when *both* of these hold:

- Its defining instruction produces a scalar—integer, pointer, or floating. Integer and pointer values take a general register; floating values take a *floating* register from a dedicated bank (a separate scan described below), keeping the two kinds out of each other's way. Aggregate values are never allocated—they are addressed by pointer.
- Its live range can be reasoned about with the linear instruction numbering. This is always true for a value confined to a *single basic block* (straight-line code, where textual order is execution order). It is *also* true for a value that escapes its block, provided the function has no back-edge: with only forward control flow the block layout is a topological order, so a definition still precedes every use and the interval `[def, last-use]` is an exact bound. A back-edge breaks this—a register freed at a value's textual last use could be reused before a loop re-executes an earlier use—so when any back-edge is present the pass falls back to the single-block rule and leaves cross-block values spilled. This is the *global* step: it lifts the old single-block-only restriction for the common loop-free shapes (`if`/`else`, `switch`, the short-circuit and ternary operators), keeping a value defined before a branch in a register through the branch instead of reloading it from the frame in each arm.

A back-edge is detected structurally: block ids are handed out in creation order, which is the layout order the back end emits, so a branch is a back-edge exactly when its target id does not exceed the branching block's id. A single pass over the blocks sets a per-function *has-loop* flag from this test.

When the flag is set, cross-block values are still allocated, but by a different route that needs no liveness at all. Every vreg has a single definition, so a value that escapes its block can be given a callee-saved register *for the whole function*: dedicated end to end, the register is never reused, which is trivially correct across a back-edge, and being callee-saved it also survives any call in the loop. The pass hands out the nine callee-saved registers to the escaping candidates in order, pins each one's scan slot busy so the in-order scan leaves it alone, and records it in the save mask. The trade is register *pressure*, not correctness: a value reserved whole-function occupies its register even outside the loop, so the supply is finite and the linear scan allocates the single-block values around what remains. A tighter scheme—reserving only across the live loop, which needs real liveness—is a later refinement.

Whether the value's live range *crosses a function call* then decides the bank. A call clobbers the caller-saved registers but preserves the callee-saved ones, so a call-free value may take any register (preferring `x12`–`x15`, which cost no frame save), whereas a call-crossing value is restricted to `x19`–`x27`. Each callee-saved register the function actually claims is recorded in a mask; the prologue

spills exactly those to a save area placed *above* the vreg slots and alloca storage (so their offsets, and the frame-relative `va_arg`/stack-argument reads, are unchanged—only `frame_size` grows), and each `IR_RET` restores them before tearing the frame down.

The pass makes three walks over the function. The first computes, per vreg, its defining block, its last-use position, whether any use escapes the defining block, and whether a call falls in its live range—the last via a monotonically increasing call counter, comparing its value at the definition with its value at each use. Determining which operand fields are vreg *uses* mirrors the dead-code eliminator's classification (§**??**); over-counting a use is harmless here (it only lengthens a live range), so mirroring that liberal rule stays safe. The second walk is the loop check above, setting the has-loop flag. The third walk is the linear scan itself: at each candidate's definition it frees any register whose occupant's last use has passed, then claims a free one from the permitted bank (or leaves the vreg spilled). The result is a map that the `load_vreg`/`store_vreg` family consults—an unallocated vreg (map entry 0) behaves exactly as in the all-spill model, so the change is transparent wherever the allocator declines to act.

One instruction writes a candidate vreg without *defining* it: a struct- or union-returning call. Its `DEST` is the pre-existing sret-buffer alloca (the call writes the aggregate result *through* that buffer address), so the vreg was already given a register at the alloca. The scan must therefore skip a struct-returning call's `DEST`, exactly as pass 1 does—a struct result never reaches the def-recording arm. Were the scan to treat the call as a def, it would re-run for a vreg already live, handing it a register freed since the alloca; the alloca's address, meanwhile, was already emitted into the *original* register, so the two mappings diverge and the earlier register's current occupant is silently clobbered. This is precisely the failure `gcc.c-torture`'s `20030613-1` provoked: a struct returned in a loop, copied through a temporary whose buffer shared a register with a still-live local. A scalar-returning call, by contrast, produces a fresh result vreg and is allocated normally.

The mirror concern is keeping that same buffer alive *across* the call. A ≤16-byte struct result comes back in `x0`(–`x1`) and is unpacked into the buffer *after* the call returns—a use of the buffer's address vreg that no operand field records (`DEST` is not an operand). When the result is assigned, the following copy-out reads the buffer and keeps it live; but when the result is *discarded* nothing does, so the buffer's address could take a caller-saved register the call clobbers, and the unpack would store through a stale (often null) pointer. Pass 1 therefore records the `DEST` of a struct/union-returning call as a use at the just-incremented call count, marking it call-crossing so it lands in a callee-saved register that survives the call. `gcc.c-torture`'s `920908-1`—a variadic function returning a struct, invoked purely for effect—faulted here until then.

The use-marking helper reads the pass's array bases from the stable callee-saved registers the scan keeps live (`x20`–`x24`) and takes the per-instruction varying values—the used vreg, current block, position, and call count—as arguments.

Register promotion of scalar locals (mem2reg). A C local is lowered as an `IR_ALLOCA` with stack storage; every read is a `LOAD` and every write a `STORE` against that slot. The allocator already treats the alloca's result vreg as an ordinary candidate—so today it may hold the *address* of the slot in a register while the loads and stores still go to memory. Promotion observes that if the local is a single-slot integer or pointer whose address never escapes, that same register can hold the local's *contents* instead, and the memory traffic disappears. The escape test is exact in this IR: the alloca's vreg is safe iff it appears only as a load's address (`SRC1`) or a store's address (`SRC2`); any other appearance is the value produced by `&x`. A scan after pass 1 marks each eligible alloca (bit `0x100`) and clears the mark on the first disqualifying use, and `volatile` locals are never marked. Crucially this reuses the allocation the scan already makes—no new register is reserved, so promotion adds *zero* register pressure, unlike a scheme that grabs a register per local up front. The finalisation step publishes `_kcc_cg_promoted[v]`, the physical register the load, store, and alloca handlers read to route a promoted local's accesses through a register (and to emit nothing for its now-storage-less `ALLOCA`).

The floating register bank. Floating values get their own linear scan over a dedicated bank so they never compete with integer values for the general registers and so an FP chain no longer round-trips through FMOV on every access. The bank is `d16`–`d31` (caller-saved, no prologue cost) for a value whose range crosses no call, and `d8`–`d15` (callee-saved) for a call-crossing or loop-carried one; `d0`–`d7` stay reserved for argument passing and the FP scratch. The scan is an exact mirror of the integer one—same liveness, same single-block/loop-free/loop-carried rules, same call-decides-the-bank logic—running over its own `reg_end` array and writing `_kcc_cg_fpregmap` and a floating callee-saved mask. A floating candidate is tagged in pass 1 (flag bit `0x8`) so the integer scan skips it. The `load_vreg`/`store_vreg` family consults the FP map first: a value homed in a `d` register is moved with FMOV (d-to-d for the FP paths, or bridged to a general register for the integer paths that materialise a constant or spill a double to memory), and the prologue saves exactly the `d8`–`d15` the scan claimed just above the general callee-saved area.

⟨ *CodeGen: register allocation* 481 ⟩ ≡

```
// -- kcc_ra_use ---
// Record one vreg use during pass 1.  x0=vreg, x1
   =cur_bb, x2=pos, x3=calls.
// Reads bases x20=vreg_n, x21=lastuse, x22=defbb,
    x23=calldef, x24=flags.
// flags bits: 1=candidate def, 2=escapes its
   block, 4=crosses a call.
.text
.p2align 2
_kcc_ra_use:
    cmp x0, x20
    b.hs .Lrau_ret                  // not a vreg
   number
    str x2, [x21, x0, lsl #3]       // lastuse[u] =
   pos
    ldr x4, [x22, x0, lsl #3]       // defbb[u]
    cmp x4, x1
    b.eq .Lrau_cross
```

```
    ldr x5, [x24, x0, lsl #3]
    orr x5, x5, #2                // escapes
   defining block
    str x5, [x24, x0, lsl #3]
.Lrau_cross:
    ldr x4, [x23, x0, lsl #3]     // calldef[u]
    cmp x3, x4
    b.ls .Lrau_ret
    ldr x5, [x24, x0, lsl #3]
    orr x5, x5, #4                // a call lies in
    [def, use]
    str x5, [x24, x0, lsl #3]
.Lrau_ret:
    ret

// -- kcc_ra_disq ---
// Disqualify a vreg from scalar-local promotion:
   if x0 is a valid vreg whose
// flags (x1 = flags base, x2 = vreg_n) carry the
   promotable bit (0x100), clear
// it.  Called for every vreg operand that appears
    in a position OTHER than
// LOAD.SRC1 / STORE.SRC2 - i.e. any use of a
   local's address as a value, which
// is exactly the `&x` escape that makes the local
    non-promotable.
.text
.p2align 2
_kcc_ra_disq:
    cmp x0, x2
    b.hs .Lrd_ret
    ldr x3, [x1, x0, lsl #3]
    tst x3, #0x100
    b.eq .Lrd_ret
    bic x3, x3, #0x100
    str x3, [x1, x0, lsl #3]
.Lrd_ret:
    ret

// -- kcc_cg_regalloc ---
// void kcc_cg_regalloc(ir_func *func)
// Computes the per-function →vregphysreg map and
   publishes it in
// _kcc_cg_regmap (0 for an unallocated vreg).
//   Stack frame (512 bytes): x29/x30 + x19..x28
   saved; locals from #96;
//   integer reg_end[13] at #152, FP reg_end_fp
   [24] at #256.
.text
.globl _kcc_cg_regalloc
.p2align 2
_kcc_cg_regalloc:
    stp x29, x30, [sp, #-496]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    stp x27, x28, [sp, #80]

    mov x19, x0                   // func
    ldr x20, [x19, #IRF_VREG_N]   // vreg_n

    // Default to the all-spill map (no allocation
   ) until we build a real one.
    adrp x0, _kcc_cg_regmap@PAGE
    add  x0, x0, _kcc_cg_regmap@PAGEOFF
    str xzr, [x0]
    adrp x0, _kcc_cg_promoted@PAGE
    add  x0, x0, _kcc_cg_promoted@PAGEOFF
    str xzr, [x0]
    adrp x0, _kcc_cg_callee_mask@PAGE
    add  x0, x0, _kcc_cg_callee_mask@PAGEOFF
    str xzr, [x0]
    adrp x0, _kcc_cg_save_count@PAGE
    add  x0, x0, _kcc_cg_save_count@PAGEOFF
    str xzr, [x0]
    adrp x0, _kcc_cg_fpregmap@PAGE
    add  x0, x0, _kcc_cg_fpregmap@PAGEOFF
    str xzr, [x0]
    adrp x0, _kcc_cg_fp_callee_mask@PAGE
    add  x0, x0, _kcc_cg_fp_callee_mask@PAGEOFF
    str xzr, [x0]
    adrp x0, _kcc_cg_fp_save_count@PAGE
    add  x0, x0, _kcc_cg_fp_save_count@PAGEOFF
    str xzr, [x0]
    // Default the slot colouring to the identity
   all-spill layout: no map, and
    // one slot per vreg.  The colouring pass at
   the end overwrites both once the
    // allocation maps are known.
    adrp x0, _kcc_cg_slotmap@PAGE
    add  x0, x0, _kcc_cg_slotmap@PAGEOFF
    str xzr, [x0]
    adrp x0, _kcc_cg_max_slot@PAGE
    add  x0, x0, _kcc_cg_max_slot@PAGEOFF
    str x20, [x0]                  // max_slot =
   vreg_n (identity default)
    cbz x20, .Lra_done             // no vregs

    // Allocate one zeroed block of seven vreg_n-
   entry arrays (8 bytes each):
    // lastuse, defbb, calldef, flags, map,
   promoted, fpmap.  promoted is the
    // alloca-vreg → contents-physreg map for
   mem2reg; fpmap is the floating
    // vreg → d-register map.
    lsl x9, x20, #3               // vreg_n * 8  (
   one array's byte size)
    mov x11, #7
    mul x0, x9, x11               // vreg_n*8 * 7
   = total bytes
    bl _kcc_arena_alloc
    mov x21, x0                    // lastuse base
    (also block base)
    lsl x9, x20, #3
    add x22, x21, x9               // defbb    =
   block + vreg_n*8
    add x23, x22, x9               // calldef  =
   block + vreg_n*16
    add x24, x23, x9               // flags    =
   block + vreg_n*24
    add x25, x24, x9               // map      =
   block + vreg_n*32
    //                                promoted =
   block + vreg_n*40 (= map + n*8)
    //                                fpmap    =
   block + vreg_n*48 (= map + n*16)

    // Zero the whole block (7*vreg_n entries).
    mov x11, #7
    mul x9, x20, x11              // 7 * vreg_n
   entries
    mov x10, x21
.Lra_zero:
    cbz x9, .Lra_zeroed
    str xzr, [x10], #8
    sub x9, x9, #1
    b .Lra_zero
.Lra_zeroed:

    // --- Pass 1: liveness / candidate analysis
   ---
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [x0, #VEC_COUNT]
    str x1, [sp, #112]             // bb_count
```

```
    str xzr, [sp, #104]          // bb_index
    mov x27, #0                   // pos
    mov x28, #0                   // calls_so_far
.Lp1_bb:
    ldr x0, [sp, #104]
    ldr x1, [sp, #112]
    cmp x0, x1
    b.hs .Lp1_done
    str x0, [sp, #96]            // cur_bb =
   bb_index
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [sp, #104]
    bl _kcc_vec_get
    ldr x26, [x0]                 // BB ptr
    ldr x26, [x26, #BB_FIRST]    // first
   instruction
.Lp1_instr:
    cbz x26, .Lp1_next_bb

    // --- SRC1 use? (skip CONST, CALL, BR,
   GLOBAL_REF) ---
    ldr x9, [x26, #IR_OP]
    cmp x9, #IR_ASM
    b.eq .Lp1_asmops
    cmp x9, #IR_CONST
    b.eq .Lp1_src2
    cmp x9, #IR_CALL
    b.eq .Lp1_src2
    cmp x9, #IR_BR
    b.eq .Lp1_src2
    cmp x9, #IR_GLOBAL_REF
    b.eq .Lp1_src2
    ldr x0, [x26, #IR_SRC1]
    ldr x1, [sp, #96]
    mov x2, x27
    mov x3, x28
    bl _kcc_ra_use
.Lp1_src2:
    ldr x9, [x26, #IR_OP]
    cmp x9, #IR_CALL
    b.eq .Lp1_callargs
    cmp x9, #IR_CALL_IND
    b.eq .Lp1_callargs
    cmp x9, #IR_BR_COND
    b.eq .Lp1_bccond
    cmp x9, #IR_BR
    b.eq .Lp1_defck
    cmp x9, #IR_CONST
    b.eq .Lp1_defck
    ldr x0, [x26, #IR_SRC2]
    ldr x1, [sp, #96]
    mov x2, x27
    mov x3, x28
    bl _kcc_ra_use
    b .Lp1_defck

.Lp1_bccond:
    ldr x0, [x26, #IR_DEST]      // condition vreg
   is a use
    ldr x1, [sp, #96]
    mov x2, x27
    mov x3, x28
    bl _kcc_ra_use
    b .Lp1_next                   // BR_COND
   defines nothing

.Lp1_asmops:
    // Inline asm operands are carried on the AST
   node (IR_TYPE), not in the
    // SRC fields.  Record each operand's value
   and address vreg as a use so an
    // input value stays live up to the asm, and
   an output lvalue's address is
    // seen to escape (which keeps the local in
   memory, since the asm stores its
    // result through that address rather than
   promoting the local to a
    // register).  x9-x28 survive the leaf
   _kcc_ra_use; the cursor rides x10.
    ldr x10, [x26, #IR_TYPE]     // asm node
    cbz x10, .Lp1_next
    ldr x10, [x10, #AST_F2]      // first operand
.Lp1_asmop_loop:
    cbz x10, .Lp1_next
    ldr x0, [x10, #AST_F2]       // value half
    and x0, x0, #0xffffffff
    cbz x0, .Lp1_asmop_addr
    sub x0, x0, #1
    ldr x1, [sp, #96]
    mov x2, x27
    mov x3, x28
    bl _kcc_ra_use
.Lp1_asmop_addr:
    ldr x0, [x10, #AST_F3]       // address half
    and x0, x0, #0xffffffff
    cbz x0, .Lp1_asmop_next
    sub x0, x0, #1
    ldr x1, [sp, #96]
    mov x2, x27
    mov x3, x28
    bl _kcc_ra_use
.Lp1_asmop_next:
    ldr x10, [x10, #AST_F4]
    b .Lp1_asmop_loop

.Lp1_callargs:
    ldr x0, [x26, #IR_SRC2]      // args array
    cbz x0, .Lp1_call_inc
    str x0, [sp, #128]           // args ptr
    ldr x1, [x0]
    and x1, x1, #0xffffffff       // arg count
    str x1, [sp, #136]
    str xzr, [sp, #144]          // i = 0
.Lp1_arg:
    ldr x5, [sp, #144]
    ldr x6, [sp, #136]
    cmp x5, x6
    b.hs .Lp1_call_inc
    ldr x7, [sp, #128]
    lsl x8, x5, #1
    add x8, x8, #1               // 1 + i*2
    ldr x0, [x7, x8, lsl #3]     // arg vreg
    ldr x1, [sp, #96]
    mov x2, x27
    mov x3, x28
    bl _kcc_ra_use
    ldr x5, [sp, #144]
    add x5, x5, #1
    str x5, [sp, #144]
    b .Lp1_arg
.Lp1_call_inc:
    add x28, x28, #1             // a call
   clobbers the caller-saved pool
    // A struct/union-returning call whose result
   is <=16B is unpacked from
    // x0(-x1) into the DEST buffer AFTER the call
    returns.  That post-call write
    // is a use of DEST the operand scan never
   sees (DEST is not an IR operand
    // field), so without help the buffer's
   address vreg looks dead at the call
    // and may take a caller-saved register the
   call then clobbers --- the unpack
    // stores through a stale (often 0) pointer
   and faults.  This bit only when
    // the result is discarded: an assigned result
    already reads the buffer after
    // the call (`r = f()` copies it out), which
   keeps it live.  Record DEST as a
```

```
    // use at the just-incremented call count so
   it is marked call-crossing and
    // lands in a callee-saved register that
   survives the call.
    ldr x9, [x26, #IR_OP]
    cmp x9, #IR_CALL
    b.eq .Lp1_ci_structret
    cmp x9, #IR_CALL_IND
    b.ne .Lp1_defck
.Lp1_ci_structret:
    ldr x9, [x26, #IR_TYPE]
    cbz x9, .Lp1_defck
    ldr x9, [x9]                    // TY_KIND
    cmp x9, #14                     // STRUCT
    b.eq .Lp1_ci_use
    cmp x9, #15                     // UNION
    b.ne .Lp1_defck
.Lp1_ci_use:
    ldr x0, [x26, #IR_DEST]         // sret buffer
   vreg
    ldr x1, [sp, #96]               // cur_bb
    mov x2, x27                     // pos
    mov x3, x28                     // call count (
   already incremented → crossing)
    bl _kcc_ra_use
    // Also give the buffer a DEDICATED spill slot
   .  Marking it call-crossing
    // steers the register allocator, but a buffer
    that still spills (a nested
    // aggregate-returning call inside a busy
   expression) is coloured by the slot
    // pass, which recycles a single-block value's
    slot once its recorded last
    // use passes.  The post-call unpack reads the
    buffer's address from that
    // slot AFTER the call, a use the operand scan
    cannot see, so a constant or
    // GEP evaluated for a later argument could
   seize the slot and the unpack
    // would store through it (the address became
   a small integer and faulted).
    // The escape bit forces the colourer to
   dedicate the slot and never reuse it.
    ldr x0, [x26, #IR_DEST]
    cmp x0, x20
    b.hs .Lp1_defck
    ldr x1, [x24, x0, lsl #3]
    orr x1, x1, #2                  // escapes →
   dedicated slot
    str x1, [x24, x0, lsl #3]
    b .Lp1_defck

.Lp1_defck:
    ldr x9, [x26, #IR_OP]
    cmp x9, #IR_BR
    b.eq .Lp1_next
    cmp x9, #IR_STORE
    b.eq .Lp1_next
    cmp x9, #IR_RET
    b.eq .Lp1_next
    cmp x9, #IR_ALLOCA
    b.eq .Lp1_defok                 // an alloca's
   result is an address (integer bank),
                                    // regardless of
   the element type it points at
    ldr x10, [x26, #IR_TYPE]
    cbz x10, .Lp1_next
    ldr x11, [x10]                  // TY_KIND
    cmp x11, #2
    b.lo .Lp1_next
    cmp x11, #7                      // BOOL..LLONG
    b.ls .Lp1_defok
    cmp x11, #10                    // FLOAT..
   LDOUBLE → allocated to the FP bank
    b.ls .Lp1_defok_fp
    cmp x11, #11                    // PTR
    b.eq .Lp1_defok
    cmp x11, #16                    // ENUM
    b.eq .Lp1_defok
    b .Lp1_next
.Lp1_defok_fp:
    // A floating candidate is tagged with an
   extra bit (0x8) so the integer
    // linear scan skips it and the dedicated FP
   scan claims a d-register.
    ldr x0, [x26, #IR_DEST]
    cmp x0, x20
    b.hs .Lp1_next
    ldr x2, [x24, x0, lsl #3]
    orr x2, x2, #0x8                // FP candidate
    str x2, [x24, x0, lsl #3]
    // fall through to record defbb/calldef and
   the candidate bit
.Lp1_defok:
    ldr x0, [x26, #IR_DEST]
    cmp x0, x20
    b.hs .Lp1_next
    ldr x1, [sp, #96]
    str x1, [x22, x0, lsl #3]       // defbb[dest]
    str x28, [x23, x0, lsl #3]      // calldef[dest]
    ldr x2, [x24, x0, lsl #3]
    orr x2, x2, #1                  // candidate def
    str x2, [x24, x0, lsl #3]
.Lp1_next:
    add x27, x27, #1
    ldr x26, [x26, #IR_NEXT]
    b .Lp1_instr
.Lp1_next_bb:
    ldr x0, [sp, #104]
    add x0, x0, #1
    str x0, [sp, #104]
    b .Lp1_bb
.Lp1_done:

    // --- Promotability analysis (mem2reg) ---
    // A scalar local (an IR_ALLOCA of an integer/
   pointer type occupying a
    // single slot) can live in a physical
   register instead of memory, provided
    // its address never escapes.  In this IR the
   address escapes exactly when
    // the alloca's vreg appears anywhere other
   than as the address operand of a
    // load (LOAD.SRC1) or a store (STORE.SRC2)
   --- i.e. as a plain value, which
    // is what `&x` produces.  We mark each
   eligible alloca with flags bit 0x100
    // at its definition, then clear it for every
   disqualifying use.  Because the
    // register allocator already treats an alloca
   's vreg as an ordinary
    // candidate (today it holds the slot ADDRESS)
   , promotion needs no new
    // allocation: after the scan we simply
   reinterpret the register it was given
    // as holding the local's CONTENTS.  This is
   why promotion adds no register
    // pressure --- the same vregs are allocated
   either way.  Volatile locals are
    // excluded so every access still touches
   memory.
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [x0, #VEC_COUNT]
    str x1, [sp, #112]
    str xzr, [sp, #104]
```

```
.Lpr_bb:
    ldr x0, [sp, #104]
    ldr x1, [sp, #112]
    cmp x0, x1
    b.hs .Lpr_done
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [sp, #104]
    bl _kcc_vec_get
    ldr x26, [x0]
    ldr x26, [x26, #BB_FIRST]
.Lpr_instr:
    cbz x26, .Lpr_next_bb
    ldr x9, [x26, #IR_OP]
    cmp x9, #IR_ASM
    b.eq .Lpr_asm
    cmp x9, #IR_ALLOCA
    b.eq .Lpr_alloca
    cmp x9, #IR_STORE
    b.eq .Lpr_store
    cmp x9, #IR_LOAD
    b.eq .Lpr_next_i              // LOAD.SRC1 is
   an allowed address use
    cmp x9, #IR_BR_COND
    b.eq .Lpr_brcond
    cmp x9, #IR_RET
    b.eq .Lpr_ret
    cmp x9, #IR_CALL
    b.eq .Lpr_call
    cmp x9, #IR_CALL_IND
    b.eq .Lpr_callind
    cmp x9, #IR_VA_START
    b.eq .Lpr_next_i              // SRC2 = fixed
   count (immediate), no vreg src
    cmp x9, #IR_CONST
    b.eq .Lpr_next_i
    cmp x9, #IR_NOP
    b.eq .Lpr_next_i
    cmp x9, #IR_GLOBAL_REF
    b.eq .Lpr_next_i
    cmp x9, #IR_STRING_LIT
    b.eq .Lpr_next_i
    cmp x9, #IR_BR
    b.eq .Lpr_next_i
    cmp x9, #IR_NEG
    b.eq .Lpr_unary
    cmp x9, #IR_NOT
    b.eq .Lpr_unary
    cmp x9, #50                   // SEXT..FPROUND
    (50-57): SRC1 value, SRC2 width
    b.lo .Lpr_binary              // everything
   else <50 (arith/cmp/GEP): SRC1+SRC2
    b .Lpr_unary

.Lpr_alloca:
    ldr x10, [x26, #IR_TYPE]      // element type
    cbz x10, .Lpr_next_i
    ldr x11, [x10, #TY_QUALS]
    tst x11, #2                    // volatile →
   never promote
    b.ne .Lpr_next_i
    ldr x11, [x10]                // TY_KIND
    cmp x11, #2                    // < BOOL → not
   a scalar (VOID)
    b.lo .Lpr_next_i
    cmp x11, #7                    // BOOL..LLONG
    b.ls .Lpr_alloca_ok
    cmp x11, #11                   // PTR
    b.eq .Lpr_alloca_ok
    cmp x11, #16                   // ENUM
    b.eq .Lpr_alloca_ok
    b .Lpr_next_i                  // FLOAT/DOUBLE/
   aggregate → not promoted here
.Lpr_alloca_ok:
    ldr x11, [x26, #IR_SRC1]      // slot count
    cmp x11, #1
    b.hi .Lpr_next_i              // multi-slot →
   not a single-register scalar
    ldr x0, [x26, #IR_DEST]       // alloca vreg
    cmp x0, x20
    b.hs .Lpr_next_i
    ldr x3, [x24, x0, lsl #3]
    orr x3, x3, #0x100
    str x3, [x24, x0, lsl #3]
    b .Lpr_next_i

.Lpr_store:
    ldr x0, [x26, #IR_SRC1]       // stored VALUE (
   SRC2 address is allowed)
    mov x1, x24
    mov x2, x20
    bl _kcc_ra_disq
    b .Lpr_next_i
.Lpr_brcond:
    ldr x0, [x26, #IR_DEST]       // condition vreg
    mov x1, x24
    mov x2, x20
    bl _kcc_ra_disq
    b .Lpr_next_i
.Lpr_ret:
    ldr x10, [x26, #IR_TYPE]
    cbz x10, .Lpr_next_i          // void return:
   SRC1 carries no value vreg
    ldr x11, [x10]
    cmp x11, #1                    // VOID
    b.eq .Lpr_next_i
    ldr x0, [x26, #IR_SRC1]       // returned value
    vreg
    mov x1, x24
    mov x2, x20
    bl _kcc_ra_disq
    b .Lpr_next_i
.Lpr_callind:
    ldr x0, [x26, #IR_SRC1]       // indirect
   callee target vreg
    mov x1, x24
    mov x2, x20
    bl _kcc_ra_disq
    // fall through to argument disqualification
.Lpr_call:
    ldr x7, [x26, #IR_SRC2]       // args array (
   count + vreg pairs)
    cbz x7, .Lpr_next_i
    ldr x8, [x7]
    and x8, x8, #0xffffffff        // arg count
    mov x5, #0
.Lpr_carg:
    cmp x5, x8
    b.hs .Lpr_next_i
    lsl x6, x5, #1
    add x6, x6, #1                // 1 + i*2
    ldr x0, [x7, x6, lsl #3]      // arg vreg
    mov x1, x24
    mov x2, x20
    str x5, [sp, #128]
    str x7, [sp, #136]
    str x8, [sp, #144]
    bl _kcc_ra_disq
    ldr x5, [sp, #128]
    ldr x7, [sp, #136]
    ldr x8, [sp, #144]
    add x5, x5, #1
    b .Lpr_carg
.Lpr_binary:
    ldr x0, [x26, #IR_SRC1]
    mov x1, x24
    mov x2, x20
    bl _kcc_ra_disq
    ldr x0, [x26, #IR_SRC2]
    mov x1, x24
```

```
    mov x2, x20
    bl _kcc_ra_disq
    b .Lpr_next_i
.Lpr_asm:
    // An extended-asm output takes the address of
    its lvalue (&x), which is a
    // disqualifying use: the local must stay in
   memory so the asm can store its
    // result through that address.  Disqualify
   each operand's address vreg.
    ldr x10, [x26, #IR_TYPE]      // asm node
    cbz x10, .Lpr_next_i
    ldr x10, [x10, #AST_F2]       // first operand
.Lpr_asm_loop:
    cbz x10, .Lpr_next_i
    ldr x0, [x10, #AST_F3]        // address half
    and x0, x0, #0xffffffff
    cbz x0, .Lpr_asm_next
    sub x0, x0, #1
    mov x1, x24
    mov x2, x20
    bl _kcc_ra_disq
.Lpr_asm_next:
    ldr x10, [x10, #AST_F4]
    b .Lpr_asm_loop

.Lpr_unary:
    ldr x0, [x26, #IR_SRC1]
    mov x1, x24
    mov x2, x20
    bl _kcc_ra_disq
.Lpr_next_i:
    ldr x26, [x26, #IR_NEXT]
    b .Lpr_instr
.Lpr_next_bb:
    ldr x0, [sp, #104]
    add x0, x0, #1
    str x0, [sp, #104]
    b .Lpr_bb
.Lpr_done:

    // --- Loop detection ---
    // A value whose live range escapes its
   defining block can be kept in a
    // register across blocks only when the linear
    instruction numbering is a
    // valid execution order---that is, when the
   function has no back-edge.  A
    // back-edge is a branch to an earlier-or-
   equal block; since block ids are
    // handed out in creation (= layout) order,
   that is simply a branch whose
    // target id does not exceed the branching
   block's id.  has_loop at
    // [sp,#120] records whether any exists; when
   it is set, escaping values
    // stay spilled (the conservative local
   behaviour) and only single-block
    // values are allocated.
    str xzr, [sp, #120]           // has_loop = 0
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [x0, #VEC_COUNT]
    str x1, [sp, #112]
    str xzr, [sp, #104]
.Lhl_bb:
    ldr x0, [sp, #104]
    ldr x1, [sp, #112]
    cmp x0, x1
    b.hs .Lhl_done
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [sp, #104]
    bl _kcc_vec_get
    ldr x2, [x0]                  // BB ptr
    ldr x3, [x2, #BB_ID]          // this block's
   id
    ldr x4, [x2, #BB_LAST]        // terminator
    cbz x4, .Lhl_next
    ldr x5, [x4, #IR_OP]
    cmp x5, #IR_BR
    b.eq .Lhl_br
    cmp x5, #IR_BR_COND
    b.eq .Lhl_brcond
    b .Lhl_next
.Lhl_br:
    ldr x6, [x4, #IR_SRC1]        // target id
    cmp x6, x3
    b.ls .Lhl_set                 // target <=
   current → back-edge
    b .Lhl_next
.Lhl_brcond:
    ldr x6, [x4, #IR_SRC1]        // true target
    cmp x6, x3
    b.ls .Lhl_set
    ldr x6, [x4, #IR_SRC2]        // false target
    cmp x6, x3
    b.ls .Lhl_set
    b .Lhl_next
.Lhl_set:
    mov x6, #1
    str x6, [sp, #120]
.Lhl_next:
    ldr x0, [sp, #104]
    add x0, x0, #1
    str x0, [sp, #104]
    b .Lhl_bb
.Lhl_done:

    // --- Pass 2: linear scan over x12..x15 (
   caller-saved) and x19..x27
    //      (callee-saved).  reg_end[i] lives at [
   sp,#152 + i*8]; -1 = free.
    add x1, sp, #152
    mov x9, #-1
    mov x10, #0
.Lp2_rinit:
    cmp x10, #13
    b.ge .Lp2_rinit_done
    str x9, [x1, x10, lsl #3]
    add x10, x10, #1
    b .Lp2_rinit
.Lp2_rinit_done:

    // --- Loop-carried reservation ---
    // In a function with a back-edge the in-order
    scan below still declines
    // cross-block values (they need liveness to
   bound the reservation across
    // the loop).  But every vreg has a single
   definition, so a cross-block
    // value can be given a callee-saved register
   for the \emph{whole} function:
    // dedicated for the entire body, it is never
   reused, so it is correct
    // across the back-edge without any liveness,
   and being callee-saved it also
    // survives any call in the loop.  We hand out
    the nine callee-saved
    // registers to the escaping candidates in
   order, pin each one's scan slot
    // busy so the linear scan leaves it alone,
   and record it in the save mask.
    ldr x9, [sp, #120]            // has_loop?
    cbz x9, .Lp2_scan_start
    mov x10, #0                    // vreg index
    mov x11, #4                    // next callee-
   saved slot (index 4 = x19)
.Lp2_wf:
```

```
    cmp x10, x20                    // vreg_n
    b.hs .Lp2_scan_start
    cmp x11, #13                    // callee-saved
   pool exhausted?
    b.hs .Lp2_scan_start
    ldr x12, [x24, x10, lsl #3]     // flags[v]
    tst x12, #0x8                    // FP candidate
    → handled by the FP scan
    b.ne .Lp2_wf_next
    and x13, x12, #3
    cmp x13, #3                     // candidate AND
    escapes its block?
    b.ne .Lp2_wf_next
    ldr x13, [x21, x10, lsl #3]     // lastuse[v]
    cbz x13, .Lp2_wf_next           // dead → skip
    add x14, x11, #15               // physreg = 15
   + slot (slot 4 → x19)
    str x14, [x25, x10, lsl #3]     // map[v] =
   physreg
    add x6, sp, #152
    mov x7, #0x40000000             // pin reg_end[
   slot] busy for the whole scan
    str x7, [x6, x11, lsl #3]
    sub x7, x14, #19                // callee-saved
   bit index
    mov x8, #1
    lsl x8, x8, x7
    adrp x6, _kcc_cg_callee_mask@PAGE
    add  x6, x6, _kcc_cg_callee_mask@PAGEOFF
    ldr x9, [x6]
    orr x9, x9, x8
    str x9, [x6]
    add x11, x11, #1                // consume this
   callee-saved register
.Lp2_wf_next:
    add x10, x10, #1
    b .Lp2_wf

.Lp2_scan_start:
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [x0, #VEC_COUNT]
    str x1, [sp, #112]
    str xzr, [sp, #104]
    mov x27, #0                      // pos
.Lp2_bb:
    ldr x0, [sp, #104]
    ldr x1, [sp, #112]
    cmp x0, x1
    b.hs .Lp2_done
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [sp, #104]
    bl _kcc_vec_get
    ldr x26, [x0]
    ldr x26, [x26, #BB_FIRST]
.Lp2_instr:
    cbz x26, .Lp2_next_bb
    ldr x9, [x26, #IR_OP]
    cmp x9, #IR_BR
    b.eq .Lp2_next
    cmp x9, #IR_BR_COND
    b.eq .Lp2_next
    cmp x9, #IR_STORE
    b.eq .Lp2_next
    cmp x9, #IR_RET
    b.eq .Lp2_next
    // A struct/union-returning call does not
   DEFINE its DEST vreg: the vreg is
    // the pre-existing sret-buffer alloca,
   already assigned a register at the
    // alloca.  The call merely writes THROUGH
   that buffer address.  Treating the
    // call as a def here would re-run the linear
   scan for the same vreg at the
    // call site and hand it a register freed
   since the alloca --- clobbering
    // whatever now lives there while the alloca's
    address was already emitted
    // into the ORIGINAL register.  Skip it,
   exactly as pass 1 does (a struct
    // result never reaches .Lp1_defok).  Scalar-
   returning calls define a fresh
    // vreg and are allocated normally.
    cmp x9, #IR_CALL
    b.eq .Lp2_chkstructret
    cmp x9, #IR_CALL_IND
    b.eq .Lp2_chkstructret
    b .Lp2_defcand
.Lp2_chkstructret:
    ldr x1, [x26, #IR_TYPE]
    cbz x1, .Lp2_defcand
    ldr x1, [x1]                    // TY_KIND
    cmp x1, #14                     // STRUCT
    b.eq .Lp2_next
    cmp x1, #15                     // UNION
    b.eq .Lp2_next
.Lp2_defcand:
    ldr x0, [x26, #IR_DEST]
    cmp x0, x20
    b.hs .Lp2_next
    ldr x1, [x24, x0, lsl #3]       // flags[dest]
    tst x1, #1                      // candidate def
   ?
    b.eq .Lp2_next
    tst x1, #0x8                    // FP candidate
    → the FP scan handles it
    b.ne .Lp2_next
    tst x1, #2                      // escapes its
   defining block?
    b.eq .Lp2_gate_ok               // no: a single-
   block value is always eligible
    ldr x9, [sp, #120]             // yes: allocate
   across blocks only when loop-free
    cbnz x9, .Lp2_next
.Lp2_gate_ok:
    and x4, x1, #4                 // x4 = crossed-a
   -call bit
    ldr x3, [x21, x0, lsl #3]       // lastuse[dest]
    cmp x3, x27
    b.ls .Lp2_next                  // no later use
    // Scan for a free register (reg_end < pos,
   signed).  A call-crossing
    // value may take only a callee-saved register
    (index >= 4); a call-free
    // value prefers the caller-saved registers (
   index 0..3) to spare a frame
    // save, falling through to callee-saved as
   overflow.
    mov x5, #0
    cbz x4, .Lp2_scan
    mov x5, #4
.Lp2_scan:
    cmp x5, #13
    b.ge .Lp2_next                  // no free
   register → spill
    add x6, sp, #152
    ldr x7, [x6, x5, lsl #3]        // reg_end[i]
    cmp x7, x27
    b.ge .Lp2_scan_next
    // free: physical register = i<4 ? 12+i : 15+i
    cmp x5, #4
    b.lo .Lp2_caller
    add x8, x5, #15
    b .Lp2_assign
.Lp2_caller:
    add x8, x5, #12
.Lp2_assign:
```

```
    str x8, [x25, x0, lsl #3]      // map[dest] =
   physreg
    add x6, sp, #152
    str x3, [x6, x5, lsl #3]       // reg_end[i] =
   lastuse[dest]
    cmp x5, #4
    b.lo .Lp2_next                 // caller-saved:
    no save needed
    sub x9, x8, #19               // callee-saved:
   record it in the save mask
    mov x10, #1
    lsl x10, x10, x9
    adrp x6, _kcc_cg_callee_mask@PAGE
    add  x6, x6, _kcc_cg_callee_mask@PAGEOFF
    ldr x7, [x6]
    orr x7, x7, x10
    str x7, [x6]
    b .Lp2_next
.Lp2_scan_next:
    add x5, x5, #1
    b .Lp2_scan
.Lp2_next:
    add x27, x27, #1
    ldr x26, [x26, #IR_NEXT]
    b .Lp2_instr
.Lp2_next_bb:
    ldr x0, [sp, #104]
    add x0, x0, #1
    str x0, [sp, #104]
    b .Lp2_bb
.Lp2_done:

    // --- FP linear scan (dedicated d-register
   bank) ---
    // Floating candidates (flag bit 0x8) are
   allocated to a floating register
    // instead of riding in a general register:
   this frees the general bank for
    // integer values and drops the fmov round-
   trip on FP-only chains.  The bank
    // is d16..d31 (caller-saved, no prologue cost
   ) for call-free values and
    // d8..d15 (callee-saved, spilled by the
   prologue) for call-crossing and
    // loop-carried ones.  The scan mirrors the
   integer one exactly, over its own
    // reg_end_fp[24] at [sp,#256] (slots 0..15 →
   d16..d31, 16..23 → d8..d15).
    add x1, sp, #256
    mov x9, #-1
    mov x10, #0
.Lpf_rinit:
    cmp x10, #24
    b.ge .Lpf_rinit_done
    str x9, [x1, x10, lsl #3]
    add x10, x10, #1
    b .Lpf_rinit
.Lpf_rinit_done:

    // Loop-carried FP reservation: an escaping
   floating value in a loopy
    // function gets a callee-saved d-register (d8
   ..d15) for the whole function,
    // correct across the back-edge because every
   vreg is single-def.
    ldr x9, [sp, #120]             // has_loop?
    cbz x9, .Lpf_scan_start
    mov x10, #0
    mov x11, #16                    // next callee
   FP slot (16 → d8)
.Lpf_wf:
    cmp x10, x20
    b.hs .Lpf_scan_start
    cmp x11, #24                    // d8..d15 pool
    exhausted?
    b.hs .Lpf_scan_start
    ldr x12, [x24, x10, lsl #3]    // flags[v]
    tst x12, #0x8                   // FP candidate
   ?
    b.eq .Lpf_wf_next
    and x13, x12, #3
    cmp x13, #3                    // candidate AND
    escapes its block?
    b.ne .Lpf_wf_next
    ldr x13, [x21, x10, lsl #3]    // lastuse[v]
    cbz x13, .Lpf_wf_next
    sub x14, x11, #8               // d-reg = slot
   -8 (slot 16 → d8)
    add x6, x25, x20, lsl #4       // fpmap base =
   map + vreg_n*16
    str x14, [x6, x10, lsl #3]      // fpmap[v] = d
   -reg
    add x6, sp, #256
    mov x7, #0x40000000            // pin
   reg_end_fp[slot] for the whole scan
    str x7, [x6, x11, lsl #3]
    sub x7, x14, #8               // fp_callee_mask
    bit index (d8 → 0)
    mov x8, #1
    lsl x8, x8, x7
    adrp x6, _kcc_cg_fp_callee_mask@PAGE
    add  x6, x6, _kcc_cg_fp_callee_mask@PAGEOFF
    ldr x9, [x6]
    orr x9, x9, x8
    str x9, [x6]
    add x11, x11, #1
.Lpf_wf_next:
    add x10, x10, #1
    b .Lpf_wf

.Lpf_scan_start:
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [x0, #VEC_COUNT]
    str x1, [sp, #112]
    str xzr, [sp, #104]
    mov x27, #0                     // pos
.Lpf_bb:
    ldr x0, [sp, #104]
    ldr x1, [sp, #112]
    cmp x0, x1
    b.hs .Lpf_done
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [sp, #104]
    bl _kcc_vec_get
    ldr x26, [x0]
    ldr x26, [x26, #BB_FIRST]
.Lpf_instr:
    cbz x26, .Lpf_next_bb
    ldr x9, [x26, #IR_OP]
    cmp x9, #IR_BR
    b.eq .Lpf_next
    cmp x9, #IR_BR_COND
    b.eq .Lpf_next
    cmp x9, #IR_STORE
    b.eq .Lpf_next
    cmp x9, #IR_RET
    b.eq .Lpf_next
    ldr x0, [x26, #IR_DEST]
    cmp x0, x20
    b.hs .Lpf_next
    ldr x1, [x24, x0, lsl #3]      // flags[dest]
    tst x1, #0x8                   // FP candidate?
    b.eq .Lpf_next
    tst x1, #2                     // escapes its
    defining block?
    b.eq .Lpf_gate_ok
```

```
    ldr x9, [sp, #120]           // escaping →
   allocate only when loop-free
    cbnz x9, .Lpf_next
.Lpf_gate_ok:
    and x4, x1, #4               // crossed-a-call
    bit
    ldr x3, [x21, x0, lsl #3]     // lastuse[dest]
    cmp x3, x27
    b.ls .Lpf_next                // no later use
    // Call-crossing → callee d8..d15 only (slot
   >=16); call-free prefers the
    // caller-saved d16..d31 (slots 0..15) to
   spare a prologue save.
    mov x5, #0
    cbz x4, .Lpf_findreg
    mov x5, #16
.Lpf_findreg:
    cmp x5, #24
    b.ge .Lpf_next                // no free d-
   register → spill
    add x6, sp, #256
    ldr x7, [x6, x5, lsl #3]      // reg_end_fp[
   slot]
    cmp x7, x27
    b.ge .Lpf_findreg_next
    cmp x5, #16
    b.lo .Lpf_caller
    sub x8, x5, #8               // callee: slot
   16 → d8
    b .Lpf_assign
.Lpf_caller:
    add x8, x5, #16              // caller: slot 0
    → d16
.Lpf_assign:
    add x6, x25, x20, lsl #4      // fpmap base
    str x8, [x6, x0, lsl #3]     // fpmap[dest] =
   d-reg
    add x6, sp, #256
    str x3, [x6, x5, lsl #3]      // reg_end_fp[
   slot] = lastuse[dest]
    cmp x5, #16
    b.lo .Lpf_next                // caller-saved
   d: no prologue save
    sub x9, x8, #8               // callee-saved d8
   ..d15: record in the FP mask
    mov x10, #1
    lsl x10, x10, x9
    adrp x6, _kcc_cg_fp_callee_mask@PAGE
    add  x6, x6, _kcc_cg_fp_callee_mask@PAGEOFF
    ldr x7, [x6]
    orr x7, x7, x10
    str x7, [x6]
    b .Lpf_next
.Lpf_findreg_next:
    add x5, x5, #1
    b .Lpf_findreg
.Lpf_next:
    add x27, x27, #1
    ldr x26, [x26, #IR_NEXT]
    b .Lpf_instr
.Lpf_next_bb:
    ldr x0, [sp, #104]
    add x0, x0, #1
    str x0, [sp, #104]
    b .Lpf_bb
.Lpf_done:
    // Publish the FP map and its callee-saved
   save count.
    add x6, x25, x20, lsl #4      // fpmap base
    adrp x0, _kcc_cg_fpregmap@PAGE
    add  x0, x0, _kcc_cg_fpregmap@PAGEOFF
    str x6, [x0]
    adrp x0, _kcc_cg_fp_callee_mask@PAGE
    add  x0, x0, _kcc_cg_fp_callee_mask@PAGEOFF
    ldr x1, [x0]
    mov x2, #0
.Lpf_pop:
    cbz x1, .Lpf_pop_done
    and x3, x1, #1
    add x2, x2, x3
    lsr x1, x1, #1
    b .Lpf_pop
.Lpf_pop_done:
    adrp x0, _kcc_cg_fp_save_count@PAGE
    add  x0, x0, _kcc_cg_fp_save_count@PAGEOFF
    str x2, [x0]

    // Build the promotion map: promoted[v] = map[
   v] for every alloca vreg the
    // scan actually placed in a register (map[v]
   != 0) that survived the
    // escape scan (flags bit 0x100).  The
   register the allocator handed the
    // alloca---which would otherwise hold the
   slot address---is reinterpreted
    // by the load/store/alloca codegen as the
   local's live-in-register value.
    // The block was zeroed, so we only write the
   promoted entries.
    lsl x13, x20, #3
    add x13, x25, x13             // promoted base
    = map + vreg_n*8
    mov x9, #0
.Lpr_fill:
    cmp x9, x20
    b.hs .Lpr_fill_done
    ldr x10, [x24, x9, lsl #3]    // flags[v]
    tst x10, #0x100
    b.eq .Lpr_fill_next
    ldr x11, [x25, x9, lsl #3]    // map[v]
    cbz x11, .Lpr_fill_next
    str x11, [x13, x9, lsl #3]    // promoted[v] =
    physreg
.Lpr_fill_next:
    add x9, x9, #1
    b .Lpr_fill
.Lpr_fill_done:
    adrp x0, _kcc_cg_promoted@PAGE
    add  x0, x0, _kcc_cg_promoted@PAGEOFF
    str x13, [x0]

    // Publish the finished map.
    adrp x0, _kcc_cg_regmap@PAGE
    add  x0, x0, _kcc_cg_regmap@PAGEOFF
    str x25, [x0]

    // save_count = popcount(callee_mask) - how
   many regs the prologue spills.
    adrp x0, _kcc_cg_callee_mask@PAGE
    add  x0, x0, _kcc_cg_callee_mask@PAGEOFF
    ldr x1, [x0]
    mov x2, #0
.Lra_popcnt:
    cbz x1, .Lra_popcnt_done
    and x3, x1, #1
    add x2, x2, x3
    lsr x1, x1, #1
    b .Lra_popcnt
.Lra_popcnt_done:
    adrp x0, _kcc_cg_save_count@PAGE
    add  x0, x0, _kcc_cg_save_count@PAGEOFF
    str x2, [x0]

    // Colour the spill slots now that the
   register/FP/promotion maps are final.
    // x19=func, x20=vreg_n, x21=lastuse, x24=
   flags are still the pass's bases.
    mov x0, x19
    mov x1, x20
```

```
    mov x2, x21
    mov x3, x24
    bl _kcc_cg_color_slots

.Lra_done:
    ldp x27, x28, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #496
    ret
```

□

481 · used in 31c · defines _kcc_cg_regalloc 493, _kcc_ra_disq never used, _kcc_ra_use never used · uses _kcc_arena_alloc 58a, _kcc_cg_callee_mask 479, _kcc_cg_color_slots 490, _kcc_cg_fpregmap 479, _kcc_cg_fp_callee_mask 479, _kcc_cg_fp_save_count 479, _kcc_cg_max_slot 479, _kcc_cg_promoted 479, _kcc_cg_regmap 479, _kcc_cg_save_count 479, _kcc_cg_slotmap 479, _kcc_vec_get 65b

35.10. Spill-Slot Colouring

The all-spill frame gives every vreg its own eight-byte slot, so a function's frame grows with its *total* vreg count. For a small function this is invisible, but a very large one—Lua's `luaV_execute`, a single function of some fifteen thousand IR instructions—then demands a $15000 \times 8 \approx 120\,\mathrm{KB}$ frame. Two hundred nested C calls through such a frame overrun the default eight-megabyte stack, so a program that recurses in C (Lua catching its own stack-overflow, say) faults the hardware stack before the interpreter's own recursion guard can fire. A production compiler must not let a frame scale with the instruction count when most of those values are short-lived temporaries.

The remedy is to let non-overlapping vregs *share* a slot, exactly as the register allocator lets them share a register. The colouring runs right after the register/FP/promotion maps are final and reuses the liveness the allocator already computed: the per-vreg last-use position and the "escapes its block" flag. A subtlety governs *which* vregs get a slot. It is tempting to colour only the values that stay in memory and skip anything the allocator put in a register, but that is unsafe: a value the integer allocator kept in a general register can still have its stack slot addressed—by the incoming-argument spill, and by the general-register paths that stage a value through its slot—so leaving it slot-less lets that address land on another value's slot and corrupt it. The colouring therefore assigns a slot to *every* integer value, excluding only the two kinds that provably never touch an integer slot: a value homed in a d-register (its accesses go through the FP map) and a promoted scalar local (its accesses become register moves). Reuse then shrinks the frame; the register-resident values simply occupy a slot across their live range like any other.

A single linear scan over the instructions, in the same order and with the same position numbering pass 1 used, does the work. At each position it first *expires* every active single-block vreg whose last use lies strictly before the current position, returning its slot to a free list; then, if the instruction defines a slot-needing vreg, it hands that vreg a slot. "Defines a vreg" is the load-bearing qualifier: only value-producing opcodes carry a vreg in `IR_DEST`. A `STORE` keeps its *access-size code* there, a `BR_COND` its condition (a use, not a def); reading either as a fresh definition would slot a bogus number. This was not hypothetical—a 16-bit store's access code 2 posed as "vreg 2" and pulled a slot off the free list mid-block, so under enough pressure a local's address and the constant stored into it aliased one slot and `l = 1` became a store of 1 *through* the value 1, faulting at address 1. So the scan skips the non-defining opcodes (`STORE`, `BR`, `BR_COND`, `RET`) exactly as pass 1 does before it looks at `IR_DEST`. A value confined to one basic block (the escape flag clear) is a *reuse* candidate: it takes a slot off the free list when one is available, and joins the active set so its slot is recycled once it dies. This is safe irrespective of loops—within a single block textual order is execution order, and across a back-edge every single-block value from the previous iteration is already dead when the block re-executes, so no live value ever shares a slot. A value that escapes its block is given a *dedicated* slot that is never freed; that is trivially correct and keeps the pass from needing any cross-block liveness.

Incoming parameters need a subtler rule. The prologue spills all of them to their slots at function *entry*, where they are simultaneously live, so two parameters must never share a slot even if the reuse test would otherwise permit it—were `db` to expire (in the single-block sense) before `mx` is defined, the free-list would hand `mx` the slot the prologue is about to reuse for `db`, and the later spill would clobber the earlier one before the body ever reads it. This is exactly how SQLite's `sqlite3StrAccumInit` lost its `db` argument to `mx`. The incoming parameters are the first `IRF_PARAMS` vregs, so the reuse test short-circuits for them straight to a dedicated slot. Parameters that never reach a defining instruction at all are still caught by a final straggler sweep, which gives every such vreg its own fresh dedicated slot. The high-water mark of the slot counter becomes `_kcc_cg_max_slot`, the size of the frame's vreg-value area, and the vreg→slot array is published in `_kcc_cg_slotmap` for `slot_offset` to consult. Because the map defaults to absent and `max_slot` to the raw vreg count, a function the pass declines to touch keeps its exact all-spill layout.

⟨ *CodeGen: slot colouring* 490 ⟩ ≡

```
// -- kcc_cg_color_slots ---
// void kcc_cg_color_slots(ir_func *func, uint64_t
    vreg_n,
//                         uint64_t *lastuse,
   uint64_t *flags)
// Builds _kcc_cg_slotmap (vreg → reused stack-
   slot index) and _kcc_cg_max_slot
// (slot count) by a linear-scan colouring of the
   SPILLED vregs, reusing a slot
// among single-block vregs whose live ranges do
   not overlap.
//   Frame (192): x29/x30 + x19..x28; locals from
   #96.
.text
.globl _kcc_cg_color_slots
.p2align 2
_kcc_cg_color_slots:
    stp x29, x30, [sp, #-192]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
```

```
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    stp x27, x28, [sp, #80]
    mov x19, x0                     // func
    mov x20, x1                     // vreg_n
    mov x21, x2                     // lastuse base
    mov x22, x3                     // flags base

    // Four vreg_n-entry arrays: slotmap,
   active_vreg, active_slot, free_slot.
    lsl x9, x20, #3                 // vreg_n * 8
    lsl x0, x9, #2                 // * 4
    bl _kcc_arena_alloc
    mov x23, x0                     // slotmap
    lsl x9, x20, #3
    add x24, x23, x9                // active_vreg
    add x25, x24, x9                // active_slot
    add x26, x25, x9                // free_slot

    // slotmap[v] = -1 (SENTINEL: unassigned) for
   every v.
    mov x9, #0
    mov x10, #-1
.Lcs_init:
    cmp x9, x20
    b.hs .Lcs_init_done
    str x10, [x23, x9, lsl #3]
    add x9, x9, #1
    b .Lcs_init
.Lcs_init_done:

    // Cache the allocation maps (published by the
    caller before this runs).
    adrp x0, _kcc_cg_regmap@PAGE
    add  x0, x0, _kcc_cg_regmap@PAGEOFF
    ldr x0, [x0]
    str x0, [sp, #144]
    adrp x0, _kcc_cg_fpregmap@PAGE
    add  x0, x0, _kcc_cg_fpregmap@PAGEOFF
    ldr x0, [x0]
    str x0, [sp, #152]
    adrp x0, _kcc_cg_promoted@PAGE
    add  x0, x0, _kcc_cg_promoted@PAGEOFF
    ldr x0, [x0]
    str x0, [sp, #160]

    str xzr, [sp, #96]              // next_slot
    str xzr, [sp, #104]             // active_count
    str xzr, [sp, #112]             // free_count
    str xzr, [sp, #120]             // pos

    ldr x0, [x19, #IRF_BLOCKS]
    ldr x0, [x0, #VEC_COUNT]
    str x0, [sp, #136]              // bb_count
    str xzr, [sp, #128]             // bb_index
.Lcs_bb:
    ldr x0, [sp, #128]
    ldr x1, [sp, #136]
    cmp x0, x1
    b.hs .Lcs_final
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x1, [sp, #128]
    bl _kcc_vec_get
    ldr x27, [x0]
    ldr x27, [x27, #BB_FIRST]
.Lcs_instr:
    cbz x27, .Lcs_next_bb

    // --- expire active single-block vregs whose
   lastuse < pos ---
    ldr x28, [sp, #120]             // pos
    ldr x11, [sp, #104]             // active_count
    mov x9, #0                      // read index
    mov x10, #0                     // write index (
   compaction)
.Lcs_exp:
    cmp x9, x11
    b.hs .Lcs_exp_done
    ldr x12, [x24, x9, lsl #3]      // v
    ldr x13, [x25, x9, lsl #3]      // slot
    ldr x14, [x21, x12, lsl #3]     // lastuse[v]
    cmp x14, x28
    b.hs .Lcs_exp_keep              // still live (
   lastuse >= pos)
    ldr x15, [sp, #112]            // free_count
    str x13, [x26, x15, lsl #3]     // free_slot[
   free_count++] = slot
    add x15, x15, #1
    str x15, [sp, #112]
    b .Lcs_exp_adv
.Lcs_exp_keep:
    str x12, [x24, x10, lsl #3]
    str x13, [x25, x10, lsl #3]
    add x10, x10, #1
.Lcs_exp_adv:
    add x9, x9, #1
    b .Lcs_exp
.Lcs_exp_done:
    str x10, [sp, #104]            // active_count =
    kept

    // --- assign a slot to a spilled DEST ---
    // Only instructions that DEFINE a vreg have a
    vreg in IR_DEST.  A STORE
    // keeps its access-size code there, a BR_COND
    its condition vreg (a use, not
    // a def), and BR/RET carry unrelated fields
   --- reading any of these as a
    // freshly-defined vreg would assign a slot to
    a bogus (or already-slotted)
    // number and derail the colouring.  A store's
    access-size code 2 masquerading
    // as "vreg 2", for one, stole a slot mid-
   block: the address `&l` and the
    // constant it stored aliased one slot, so `l
   = 1` wrote 1 through the value 1
    // (a store to address 1) and faulted.  Skip
   the non-defining opcodes exactly
    // as pass 1 does.
    ldr x9, [x27, #IR_OP]
    cmp x9, #IR_STORE
    b.eq .Lcs_adv
    cmp x9, #IR_BR
    b.eq .Lcs_adv
    cmp x9, #IR_BR_COND
    b.eq .Lcs_adv
    cmp x9, #IR_RET
    b.eq .Lcs_adv
    ldr x0, [x27, #IR_DEST]
    cbz x0, .Lcs_adv               // no dest
    cmp x0, x20
    b.hs .Lcs_adv                  // out of range
    mov x28, x0                     // d
    // A value homed in a d-register or a promoted
    local never touches an
    // integer stack slot, so it needs none.  A
   value the integer allocator put
    // in a general register is NOT excluded:
   several codegen paths still address
    // its slot (the incoming-argument spill
   writes each parameter to its slot
    // whether or not it stayed in a register; a
   general-register value is also
    // materialised through its slot on some paths
   ), so it must own a valid,
    // non-colliding slot for its whole live range
```

```
    ---hence it joins the scan.
    ldr x1, [sp, #152]            // fpmap
    cbz x1, .Lcs_chk_pr
    ldr x2, [x1, x28, lsl #3]
    cbnz x2, .Lcs_adv             // in a d-
    register
.Lcs_chk_pr:
    ldr x1, [sp, #160]            // promoted
    cbz x1, .Lcs_spilled
    ldr x2, [x1, x28, lsl #3]
    cbnz x2, .Lcs_adv             // promoted local
    (register)
.Lcs_spilled:
    // Parameters are spilled to their slots by
    the prologue at function ENTRY,
    // where all of them are simultaneously live,
    so two parameters must never
    // share a slot -- otherwise the later spill
    clobbers the earlier one before
    // the body runs (SQLite's sqlite3StrAccumInit
    lost its `db` argument to the
    // `mx` argument this way).  The incoming
    parameters are the first IRF_PARAMS
    // vregs; give each a dedicated, never-reused
    slot.
    ldr x1, [x19, #IRF_PARAMS]
    cmp x28, x1
    b.lo .Lcs_dedicated
    ldr x1, [x22, x28, lsl #3]    // flags[d]
    tst x1, #2                    // escapes its
    block?
    b.ne .Lcs_dedicated
    // single-block: reuse a freed slot if any,
    else a fresh one; track active.
    ldr x2, [sp, #112]            // free_count
    cbz x2, .Lcs_fresh_sb
    sub x2, x2, #1
    str x2, [sp, #112]
    ldr x3, [x26, x2, lsl #3]     // slot =
    free_slot[--free_count]
    b .Lcs_sb_have
.Lcs_fresh_sb:
    ldr x3, [sp, #96]
    add x4, x3, #1
    str x4, [sp, #96]             // next_slot++
.Lcs_sb_have:
    str x3, [x23, x28, lsl #3]    // slotmap[d] =
    slot
    ldr x2, [sp, #104]
    str x28, [x24, x2, lsl #3]    // active_vreg[
    active_count] = d
    str x3, [x25, x2, lsl #3]     // active_slot[
    active_count] = slot
    add x2, x2, #1
    str x2, [sp, #104]
    b .Lcs_adv
.Lcs_dedicated:
    ldr x3, [sp, #96]
    add x4, x3, #1
    str x4, [sp, #96]             // next_slot++ (
    never freed)
    str x3, [x23, x28, lsl #3]    // slotmap[d] =
    slot
.Lcs_adv:
    ldr x0, [sp, #120]
    add x0, x0, #1
    str x0, [sp, #120]            // pos++
    ldr x27, [x27, #IR_NEXT]
    b .Lcs_instr
.Lcs_next_bb:
    ldr x0, [sp, #128]
    add x0, x0, #1
    str x0, [sp, #128]
    b .Lcs_bb

.Lcs_final:
    // Straggler sweep: parameters and any other
    spilled vreg with no defining
    // instruction get a fresh dedicated slot.
    mov x9, #0
.Lcs_str:
    cmp x9, x20
    b.hs .Lcs_publish
    ldr x10, [x23, x9, lsl #3]
    adds xzr, x10, #1             // slotmap[v] ==
    -1 (unassigned)?
    b.ne .Lcs_str_next
    ldr x1, [sp, #152]            // fp-homed → no
    integer slot
    cbz x1, .Lcs_str_pr
    ldr x2, [x1, x9, lsl #3]
    cbnz x2, .Lcs_str_next
.Lcs_str_pr:
    ldr x1, [sp, #160]
    cbz x1, .Lcs_str_assign
    ldr x2, [x1, x9, lsl #3]
    cbnz x2, .Lcs_str_next
.Lcs_str_assign:
    ldr x3, [sp, #96]
    add x4, x3, #1
    str x4, [sp, #96]
    str x3, [x23, x9, lsl #3]
.Lcs_str_next:
    add x9, x9, #1
    b .Lcs_str

.Lcs_publish:
    adrp x0, _kcc_cg_slotmap@PAGE
    add  x0, x0, _kcc_cg_slotmap@PAGEOFF
    str x23, [x0]
    ldr x0, [sp, #96]             // next_slot high-
    water = slot count
    adrp x1, _kcc_cg_max_slot@PAGE
    add  x1, x1, _kcc_cg_max_slot@PAGEOFF
    str x0, [x1]

    ldp x27, x28, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #192
    ret
```

□

490 · used in 31c · defines _kcc_cg_color_slots 481 · uses _kcc_arena_alloc 58a, _kcc_cg_fpregmap 479, _kcc_cg_max_slot 479, _kcc_cg_promoted 479, _kcc_cg_regmap 479, _kcc_cg_slotmap 479, _kcc_vec_get 65b

35.11. Compile Function

Compiling one IR function proceeds in nine steps:

1. **Frame size.** Pre-scan all `IR_ALLOCA` instructions, sum their slot counts, then compute $\text{align16}(16 + \mathit{vreg_count} \times 8 + \mathit{alloca_slots} \times 8)$.

2. **Function start.** Record the current code-buffer offset so the linker symbol gets the correct value.

3. **BB label table.** Allocate a hash table that maps BB IDs to code offsets, used by branch instructions. The table is open-addressed and never resizes, so it

must be sized to the function up front: its capacity is the next power of two at or above four times the block count (never below 64). A fixed capacity would spin forever in `ht_put` once a large function—a real tokenizer's `lex_next`, say—exceeded it; sizing to the block count keeps the load factor low and the insertion loop bounded.

4. **Patch vector.** A vector of $(code_offset, target_bb_id)$ pairs for forward branches that need back-patching.

5. **Prologue.** For small frames (≤ 504 bytes), emit `STP x29, x30, [sp, #-frame_size]!`; for larger frames, emit `SUB sp, sp, #frame_size` followed by `STP x29, x30, [sp]` using the signed-offset form, because the signed 7-bit STP/LDP immediate (scaled by 8) only covers -512 to $+504$; the epilogue's positive post-index hits the limit first. Both paths finish with `MOV x29, sp`.

6. **Store arguments.** Copy x0–x7 to their corresponding vreg stack slots.

7. **Walk BBs.** For each basic block, record its label offset in the hash table, then generate code for each instruction.

8. **Patch.** Resolve all forward branch placeholders.

9. **Symbol.** Register the function as a defined `N_SECT|N_EXT` symbol with the Mach-O underscore prefix.

⟨ *CodeGen: compile function* 493 ⟩ ≡

```
// -- kcc_codegen_func ---
// void kcc_codegen_func(ir_func *func)
//   x0 = pointer to IR function
.text
.globl _kcc_codegen_func
.p2align 2
_kcc_codegen_func:
    stp x29, x30, [sp, #-112]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    stp x27, x28, [sp, #80]

    mov x19, x0                     // x19 =
  ir_func

    // 1. Compute frame size via alloca pre-scan.
    //    Value slots: vreg_count * 8 (one 8-byte
  slot per vreg).
    //    Alloca storage: sum of all IR_ALLOCA.
  SRC1 values * 8.
    //    Each alloca records its running offset
  in SRC2.
    ldr x20, [x19, #IRF_VREG_N]   // vreg_count (
  temporary)

    // Store vreg_count (the alloca address
  computation reads it) and run
    // register allocation FIRST.  Allocation
  publishes _kcc_cg_promoted, so the
    // pre-scan below can leave every promoted
  scalar local out of the frame:
    // a local that lives in a register needs no
  stack slot.
    adrp x1, _kcc_cg_vreg_count@PAGE
    add  x1, x1, _kcc_cg_vreg_count@PAGEOFF
    str x20, [x1]
    mov x0, x19
    bl _kcc_cg_regalloc

    // Pre-scan: walk all BBs/instructions, find
  allocas, assign offsets
    mov x0, #0                      // running
  alloca slot offset
    ldr x1, [x19, #IRF_BLOCKS]    // blocks vec
    ldr x2, [x1, #VEC_COUNT]      // num BBs
    mov x3, #0                      // BB index
.Lcgf_alloca_scan_bb:
    cmp x3, x2
    b.ge .Lcgf_alloca_scan_done
    // save loop state to stack temp area [sp,
  #96..104]
    str x0, [sp, #96]
    str x2, [sp, #104]
    stp x1, x3, [sp, #88]         // x1=blocks_vec
  , x3=bb_idx overlapping - use 88,96 is taken
    // Actually use dedicated temp save
    stp x3, x2, [sp, #88]         // save bb_idx,
  bb_count
    str x0, [sp, #104]            // save running
  offset
    mov x0, x1                      // blocks vec
    mov x1, x3                      // index
    bl _kcc_vec_get
    ldr x4, [x0]                    // x4 = BB ptr
    ldr x4, [x4, #BB_FIRST]       // first instr
.Lcgf_alloca_scan_instr:
    cbz x4, .Lcgf_alloca_scan_next_bb
    ldr x5, [x4, #IR_OP]
    cmp x5, #IR_ALLOCA
    b.ne .Lcgf_alloca_scan_skip
    // A promoted scalar local lives in a register
  - give it no frame storage.
    adrp x5, _kcc_cg_promoted@PAGE
    add  x5, x5, _kcc_cg_promoted@PAGEOFF
    ldr x5, [x5]
    cbz x5, .Lcgf_alloca_store
    ldr x6, [x4, #IR_DEST]
    ldr x5, [x5, x6, lsl #3]
    cbnz x5, .Lcgf_alloca_scan_skip
.Lcgf_alloca_store:
    // Found alloca: store running offset in SRC2,
    advance by SRC1
    ldr x0, [sp, #104]            // running
  offset
    str x0, [x4, #IR_SRC2]        // record offset
    for this alloca
    ldr x5, [x4, #IR_SRC1]        // slot count
    cbz x5, .Lcgf_alloca_one
    add x0, x0, x5                // advance by
  slot count
    b .Lcgf_alloca_save
.Lcgf_alloca_one:
    add x0, x0, #1                // default: 1
  slot
.Lcgf_alloca_save:
    str x0, [sp, #104]
.Lcgf_alloca_scan_skip:
    ldr x4, [x4, #IR_NEXT]
    b .Lcgf_alloca_scan_instr
.Lcgf_alloca_scan_next_bb:
    ldp x3, x2, [sp, #88]         // restore
  bb_idx, bb_count
    ldr x0, [sp, #104]            // running
  offset
    ldr x1, [x19, #IRF_BLOCKS]    // blocks vec (
```

```
    reload)
    add x3, x3, #1
    b .Lcgf_alloca_scan_bb
.Lcgf_alloca_scan_done:
    // x0 = total alloca slots (promoted locals
    excluded)
    mov x24, x0                         //
    total_alloca_slots
    // The vreg value area is sized by the
    colouring pass's slot count
    // (_kcc_cg_max_slot), which reuses one slot
    for many non-overlapping
    // spilled vregs; it equals the raw vreg count
    when colouring is inactive.
    adrp x1, _kcc_cg_max_slot@PAGE
    add  x1, x1, _kcc_cg_max_slot@PAGEOFF
    ldr x25, [x1]                       // max_slot (
    vreg value-slot count)

    // save_base = 16 + max_slot*8 + total_alloca
    *8 - the callee-save area
    // sits above the vreg slots and alloca
    storage, so those offsets (and the
    // frame-relative va_arg/stack-argument reads)
    are unaffected.
    lsl x1, x25, #3                     // max_slot * 8
    lsl x0, x24, #3                     //
    total_alloca_slots * 8
    add x0, x0, x1
    add x0, x0, #16                     // + fp/lr
    adrp x1, _kcc_cg_save_base@PAGE
    add  x1, x1, _kcc_cg_save_base@PAGEOFF
    str x0, [x1]
    mov x2, x0                          // save_base

    // fp_save_base = save_base + gp_save_count*8
    - the callee-saved d-registers
    // are spilled just above the general callee-
    saved area.
    adrp x1, _kcc_cg_save_count@PAGE
    add  x1, x1, _kcc_cg_save_count@PAGEOFF
    ldr x1, [x1]
    add x2, x2, x1, lsl #3              // x2 =
    fp_save_base = save_base + gp_saves*8
    adrp x1, _kcc_cg_fp_save_base@PAGE
    add  x1, x1, _kcc_cg_fp_save_base@PAGEOFF
    str x2, [x1]

    // frame_size = align16(fp_save_base +
    fp_save_count*8)
    adrp x1, _kcc_cg_fp_save_count@PAGE
    add  x1, x1, _kcc_cg_fp_save_count@PAGEOFF
    ldr x1, [x1]
    add x0, x2, x1, lsl #3
    add x0, x0, #15
    and x0, x0, #~15                    // align to 16
    mov x20, x0                         // x20 =
    frame_size

    adrp x1, _kcc_cg_frame_size@PAGE
    add  x1, x1, _kcc_cg_frame_size@PAGEOFF
    str x20, [x1]

    // 2. Record function start offset
    bl _kcc_code_current_pos
    mov x21, x0                         // x21 =
    func_start_offset
    str x21, [x19, #IRF_LOWPC]          // low_pc for
    DWARF debug info

    // 3. Initialize BB label hash table (bb_id →
    code_offset), sized to
    //    the function's BB count: next power of
    two >= 4*bb_count, min 64.
    ldr x0, [x19, #IRF_BLOCKS]
    ldr x0, [x0, #VEC_COUNT]            // bb_count
    lsl x0, x0, #2                      // 4 * bb_count
    (keeps load factor low)
    cmp x0, #64
    b.ls .Lcgf_bbcap_min
    sub x0, x0, #1
    clz x1, x0
    mov x2, #64
    sub x1, x2, x1                      // 64 - clz(x-1)
    mov x2, #1
    lsl x0, x2, x1                      // next power of
    two
    b .Lcgf_bbcap_done
.Lcgf_bbcap_min:
    mov x0, #64
.Lcgf_bbcap_done:
    bl _kcc_ht_init
    mov x22, x0                         // x22 =
    bb_labels ht

    // 4. Initialize forward-branch patch vec
    mov x0, #VEC_SIZE
    bl _kcc_arena_alloc
    mov x23, x0                         // x23 = patch
    vec
    mov x1, #PATCH_SIZE
    str x1, [x23, #VEC_ELEMSIZE]
    mov x1, #32
    str x1, [x23, #VEC_CAP]
    str xzr, [x23, #VEC_COUNT]

    mov x0, #512                        // 32 * 16
    bl _kcc_arena_alloc
    str x0, [x23, #VEC_DATA]

    // 5. Emit prologue
    cmp x20, #504
    b.hi .Lcgf_large_prologue

    // Small frame: stp x29, x30, [sp, #-
    frame_size]!
    mov x0, #29
    mov x1, #30
    mov x2, #31
    neg x3, x20
    asr x3, x3, #3
    bl _kcc_enc_stp_pre
    bl _kcc_code_emit32
    b .Lcgf_prologue_mov

.Lcgf_large_prologue:
    // Large frame: sub sp, sp, #frame_size (split
    when >4095) then stp x29,x30,[sp]
    mov x0, #31                         // rd = sp
    mov x1, #31                         // rn = sp
    mov x2, x20                         // frame_size
    bl _kcc_cg_emit_sub_imm
    // stp x29, x30, [sp, #0] - signed-offset form
    : 0xA9007BFD
    movz w0, #0x7BFD
    movk w0, #0xA900, lsl #16
    bl _kcc_code_emit32

.Lcgf_prologue_mov:
    // mov x29, sp → ADD x29, sp, #0
    mov x0, #29
    mov x1, #31
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_add_imm
    bl _kcc_code_emit32

    // Save the callee-saved registers the
    allocator claimed.
```

```
    mov x0, #0
    bl _kcc_cg_callee_transfer
```

□

493 · defined at 493, 496, 498ab · used in 31c · defines _kcc_codegen_func 473 · uses _kcc_arena_alloc 58a, _kcc_cg_callee_transfer 499b, _kcc_cg_emit_sub_imm 495, _kcc_cg_fp_save_base 479, _kcc_cg_fp_save_count 479, _kcc_cg_frame_size 479, _kcc_cg_max_slot 479, _kcc_cg_promoted 479, _kcc_cg_regalloc 481, _kcc_cg_save_base 479, _kcc_cg_save_count 479, _kcc_cg_vreg_count 479, _kcc_code_current_pos 526a, _kcc_code_emit32 525b, _kcc_enc_add_imm 527a, _kcc_enc_stp_pre 529b, _kcc_ht_init 61c, _kcc_vec_get 65b

Adjusting a register by a large immediate A subtlety that surfaces on large functions is that add/sub with an immediate encode it as a *12-bit* unsigned field—a maximum of 4095. Three sites can exceed that: the large-frame prologue (`sub sp, sp, #frame_size`), the matching epilogue (`add sp, sp, #frame_size`), and the address of a stack slot (`add rd, x29, #offset` for an `alloca` or a taken address). A function with hundreds of locals or temporaries—a large `switch`, a deeply nested block, machine-generated code—pushes any of these past 4095, and a naive encoder silently overflows the immediate into the adjacent `shift` bit, producing a tagged ADDG/SUBG instruction that faults with an illegal-instruction trap at run time.

The ARM64 add/subtract-immediate format anticipates this: a `shift` bit multiplies the 12-bit immediate by 4096, so any value up to $2^{24}-1$ splits into a *high* part (bits 12–23, shifted) plus a *low* part (bits 0–11). The first two helpers below emit exactly that—one instruction when the value fits in 12 bits, otherwise the shifted high part followed by the low part—for an arbitrary destination and source register. All three add/sub sites route through them, so frames and slot addresses of any realistic size are computed correctly. When the second instruction runs it adds into the destination register (not the original source), so the two parts accumulate.

The same 12-bit ceiling limits the *other* frame instruction, the LDR/STR that reads or writes a stack slot: its immediate is scaled by the access size, so an 8-byte load or store reaches only $4095 \times 8 = 32760$ bytes from x29. A function whose frame exceeds that—thousands of spilled temporaries, a callee-saved area high in a large frame, an incoming stack argument above a big local region—cannot address its upper slots with the scaled form. The third helper, `_kcc_cg_emit_frame_ldst`, closes this: for an in-range offset it emits the plain scaled instruction (so ordinary functions are unchanged), and for a larger one it materialises the slot address in the scratch register x17 (through the add helper above, itself split when necessary) and transfers through `[x17, #0]`. Every spill path—the integer and floating vreg transfers, the callee-saved save/restore, and the incoming stack-parameter spill—routes its load and store through it, so a frame of any size is addressed correctly.

⟨ *CodeGen: large immediate adjust* 495 ⟩ ≡

```
.text
// -- _kcc_cg_emit_add_imm ---
// void _kcc_cg_emit_add_imm(uint64_t rd, uint64_t
    rn, uint64_t imm)
// Emit `add rd, rn, #imm`, splitting into shifted
   -high + low when imm
// exceeds the 12-bit immediate (4095).  imm==0
   emits a single `add rd,rn,#0`.
.p2align 2
_kcc_cg_emit_add_imm:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                     // rd
    mov x20, x1                     // rn (running
   source)
    mov x21, x2                     // imm
    lsr x9, x21, #12                // high = imm
   >> 12
    cbz x9, .Leai_low
    mov x0, x19
    mov x1, x20
    mov x2, x9
    mov x3, #1
    bl _kcc_enc_add_imm
    orr w0, w0, #0x400000           // set sh=1 →
   immediate shifted left by 12
    bl _kcc_code_emit32
    mov x20, x19                    // low part
   accumulates into rd
.Leai_low:
    and x2, x21, #0xFFF             // low = imm &
   0xFFF
    lsr x9, x21, #12                // recompute
   high (x9 clobbered by emit)
    cbnz x2, .Leai_emit_low         // low nonzero
   → emit it
    cbnz x9, .Leai_done             // low zero but
    high already emitted → done
.Leai_emit_low:
    mov x0, x19
    mov x1, x20
    mov x3, #1
    bl _kcc_enc_add_imm
    bl _kcc_code_emit32
.Leai_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- _kcc_cg_emit_sub_imm ---
// void _kcc_cg_emit_sub_imm(uint64_t rd, uint64_t
    rn, uint64_t imm)
// Emit `sub rd, rn, #imm`, splitting the same way
    as the add helper.
.p2align 2
_kcc_cg_emit_sub_imm:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                     // rd
    mov x20, x1                     // rn
    mov x21, x2                     // imm
    lsr x9, x21, #12
    cbz x9, .Lesi_low
    mov x0, x19
    mov x1, x20
    mov x2, x9
    mov x3, #1
    bl _kcc_enc_sub_imm
    orr w0, w0, #0x400000           // sh=1
    bl _kcc_code_emit32
```

```
    mov x20, x19
.Lesi_low:
    and x2, x21, #0xFFF
    lsr x9, x21, #12
    cbnz x2, .Lesi_emit_low
    cbnz x9, .Lesi_done
.Lesi_emit_low:
    mov x0, x19
    mov x1, x20
    mov x3, #1
    bl _kcc_enc_sub_imm
    bl _kcc_code_emit32
.Lesi_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- _kcc_cg_emit_frame_ldst ---
// void _kcc_cg_emit_frame_ldst(uint64_t rt,
   uint64_t byte_off, uint64_t kind)
//   kind: 0 = LDR int, 1 = STR int, 2 = LDR fp, 3
   = STR fp
// Emit a transfer between register rt and [x29 +
   byte_off].  A frame-relative
// unsigned-offset ldr/str scales its 12-bit
   immediate by the access size, so
// an 8-byte access can only reach 32760 bytes;
   beyond that the immediate would
// overflow.  For a larger offset the helper
   materialises the address in the
// intra-procedure scratch register x17 (add x17,
   x29, #byte_off, itself split
// when needed) and uses [x17, #0].  For an in-
   range offset it emits the plain
// scaled form, so small frames stay byte-for-byte
    identical to before.
.p2align 2
_kcc_cg_emit_frame_ldst:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                     // rt
    mov x20, x1                     // byte_off
    mov x21, x2                     // kind
    mov x0, x19                     // rt (for
   encoder)
    mov x9, #0x7FF8                 // 32760 = 4095
    * 8 (max scaled 8-byte reach)
    cmp x20, x9
    b.hi .Lefl_large
    // In range: rn = x29, imm12 = byte_off >> 3
    mov x1, #29
    lsr x2, x20, #3
    b .Lefl_encode
.Lefl_large:
    // Out of range: x17 = x29 + byte_off, then [
   x17, #0]
    mov x0, #17
    mov x1, #29
    mov x2, x20
    bl _kcc_cg_emit_add_imm
    mov x0, x19                     // rt
    mov x1, #17                     // rn = x17
    mov x2, #0                      // imm12 = 0
.Lefl_encode:
    // x0 = rt, x1 = rn, x2 = imm12; dispatch by
   kind
    cbz x21, .Lefl_ldr_int
    cmp x21, #1
    b.eq .Lefl_str_int
    cmp x21, #2
    b.eq .Lefl_ldr_fp
    bl _kcc_enc_str_fp              // kind == 3
    b .Lefl_emit
.Lefl_ldr_int:
    mov x3, #1
    bl _kcc_enc_ldr_imm
    b .Lefl_emit
.Lefl_str_int:
    mov x3, #1
    bl _kcc_enc_str_imm
    b .Lefl_emit
.Lefl_ldr_fp:
    bl _kcc_enc_ldr_fp
.Lefl_emit:
    bl _kcc_code_emit32
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

495 · used in 31c · defines _kcc_cg_emit_add_imm 513, 517, _kcc_cg_emit_frame_ldst 496, 499b, _kcc_cg_emit_sub_imm 493 · uses _kcc_code_emit32 525b, _kcc_enc_add_imm 527a, _kcc_enc_ldr_fp 530b, _kcc_enc_ldr_imm 529a, _kcc_enc_str_fp 530b, _kcc_enc_str_imm 529a, _kcc_enc_sub_imm 527a

Storing Register Arguments After the prologue, the compiler copies each register argument (x0–x7) into its corresponding vreg stack slot. The ARM64 calling convention passes the first eight integer arguments in registers, so parameter k lands in register k and must be spilled to `[x29, #16 + k*8]`. The scaled immediate for the unsigned-offset STR is simply $k+2$, reusing the same slot-offset formula as the rest of the all-spill backend.

Parameters beyond the eighth do not arrive in registers—the caller places them on the stack (see the call lowering below). Because the caller pushes argument k at offset $(k-1)\times 8$ below its pre-call stack pointer, and this callee's prologue subtracts `frame_size` from that same pointer, incoming parameter k ($k \geq 8$) is found at `[x29, #frame_size + (k-1)×8]`. The spill loop reads each such parameter through scratch x9 and stores it into its own vreg slot, so from that point on a stack parameter is indistinguishable from a register one. This is kcc's own uniform slot convention rather than the exact AAPCS64 stack-slot ordering; it is self-consistent for kcc-to-kcc calls, which is what the corpus needs.

⟨ *CodeGen: compile function* 496 ⟩ +≡

```
    // 6. Spill incoming parameters (AAPCS64) to
   their vreg stack slots.
    // Walk the parameters keeping NGRN (x26),
   NSRN (x27), and a running vreg
    // base ([sp,#96]) that advances by two for a
   9..16-byte struct (which
    // occupies two vregs) and by one otherwise.
   Each parameter is spilled
    // from x<ngrn>, d<nsrn>, or (a 9..16B struct)
    two general registers, per
    // its class in IRF_PARAM_CLS; a parameter
   that exhausts its register file
    // is read from the caller's stack image at [
   x29 + frame_size + (base-1)*8].
    ldr x24, [x19, #IRF_PARAMS]    // param count
    ldr x28, [x19, #IRF_PARAM_CLS] // per-param
   class array (may be 0)
```

```
    mov x25, #0                     // param index
    mov x26, #0                     // NGRN
    mov x27, #0                     // NSRN
    str xzr, [sp, #96]              // vreg base

.Lcgf_store_args:
    cmp x25, x24
    b.ge .Lcgf_walk_bbs
    mov x0, #0                      // class (
   default GP)
    cbz x28, .Lcgf_sa_have_cls
    ldr x0, [x28, x25, lsl #3]
.Lcgf_sa_have_cls:
    cmp x0, #1                      // FP?
    b.eq .Lcgf_sa_fp
    cmp x0, #5                      // hidden sret
   pointer (x8)?
    b.eq .Lcgf_sa_x8
    cmp x0, #3                      // ST2 (9..16B
   struct)?
    b.eq .Lcgf_sa_st2

    // GP / ST1 / REF: spill x<ngrn> into slot(
   base); base += 1
    cmp x26, #8
    b.ge .Lcgf_sa_stack
    ldr x0, [sp, #96]               // base vreg →
   colored slot immediate
    bl _kcc_cg_slot_offset
    mov x2, x0                      // imm12_scaled =
    slot(base) + 2
    mov x0, x26                     // rt = x<ngrn>
    mov x1, #29
    mov x3, #1
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
    add x26, x26, #1
    ldr x2, [sp, #96]
    add x2, x2, #1
    str x2, [sp, #96]
    b .Lcgf_sa_next

.Lcgf_sa_fp:
    cmp x27, #8
    b.ge .Lcgf_sa_stack
    ldr x0, [sp, #96]               // vreg = base
    mov x1, x27                     // fp reg =
   nsrn
    bl _kcc_cg_store_vreg_fp
    add x27, x27, #1
    ldr x2, [sp, #96]
    add x2, x2, #1
    str x2, [sp, #96]
    b .Lcgf_sa_next

.Lcgf_sa_x8:
    // Hidden result pointer: spill x8 into the
   sret vreg slot (no NGRN/NSRN).
    ldr x0, [sp, #96]               // base vreg →
   colored slot immediate
    bl _kcc_cg_slot_offset
    mov x2, x0
    mov x0, #8
    mov x1, #29
    mov x3, #1
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
    ldr x2, [sp, #96]
    add x2, x2, #1
    str x2, [sp, #96]
    b .Lcgf_sa_next

.Lcgf_sa_st2:
    // 9..16B struct in two registers: spill x<
    ngrn→>slot(base),
     // x<ngrn→+1>slot(base+1); base += 2.  The two
     vregs may map to
     // non-adjacent colored slots, so each is
    resolved separately.
     cmp x26, #6
     b.gt .Lcgf_sa_stack
     ldr x0, [sp, #96]              // base → slot(
    base)+2
     bl _kcc_cg_slot_offset
     mov x2, x0
     mov x0, x26
     mov x1, #29
     mov x3, #1
     bl _kcc_enc_str_imm
     bl _kcc_code_emit32
     ldr x0, [sp, #96]
     add x0, x0, #1                 // base+1 → slot(
    base+1)+2
     bl _kcc_cg_slot_offset
     mov x2, x0
     add x0, x26, #1
     mov x1, #29
     mov x3, #1
     bl _kcc_enc_str_imm
     bl _kcc_code_emit32
     add x26, x26, #2
     ldr x2, [sp, #96]
     add x2, x2, #2
     str x2, [sp, #96]
     b .Lcgf_sa_next

.Lcgf_sa_stack:
     // Overflow: read from [x29 + frame_size + (
    base-S)*8] into x9, store to
     // slot(base); base += 1.  S is the caller's
    first-stack index: 1 for a
     // normal call, but 8 for a variadic callee,
    whose caller placed the
     // register-overflow fixed parameters at [sp
    ,#(base-8)*8] (before the
     // variadic tail).  Both offsets can exceed
    the scaled immediate on a large
     // frame, so they route through the frame-ldst
     helper.
     ldr x0, [x19, #IRF_VARIADIC]
     cbz x0, .Lcgf_sa_s1
     ldr x1, [sp, #96]              // base
     sub x1, x1, #8                  // variadic: (
    base-8)*8
     b .Lcgf_sa_soff
.Lcgf_sa_s1:
     // Normal call: the caller keyed its stack
    image by ARGUMENT index (arg k at
     // slot k-1), so the overflow read must use
    the parameter index, not the
     // running register-slot base.  They differ
    only when a by-value struct
     // precedes this parameter: an ST2 struct
    consumes two registers (base += 2)
     // but occupies one argument slot, so a base-
    relative offset read the arg one
     // slot too high (a 16B struct + enough ints
    to spill the last read the wrong
     // stack word).
     sub x1, x25, #1                // normal: (
    arg_index - 1)*8
.Lcgf_sa_soff:
     lsl x1, x1, #3
     add x1, x1, x20                // + frame_size
    (x20 = frame_size) = byte offset
     mov x0, #9                     // rt = x9
     mov x2, #0                     // kind = LDR int
```

```
    bl _kcc_cg_emit_frame_ldst
    ldr x0, [sp, #96]           // base vreg → colored slot immediate
    bl _kcc_cg_slot_offset
    lsl x1, x0, #3              // byte offset from x29
    mov x0, #9                  // rt = x9
    mov x2, #1                  // kind = STR int
    bl _kcc_cg_emit_frame_ldst
    ldr x2, [sp, #96]
    add x2, x2, #1
    str x2, [sp, #96]

.Lcgf_sa_next:
    add x25, x25, #1
    b .Lcgf_store_args
```

□

496 · defined at 493, 496, 498ab · used in 31c · uses _kcc_cg_emit_frame_ldst 495, _kcc_cg_slot_offset 499a, _kcc_cg_store_vreg_fp 499b, _kcc_code_emit32 525b, _kcc_enc_str_imm 529a

Walking Basic Blocks With the prologue in place and arguments spilled, the compiler enters the main loop over basic blocks. For each BB it records the current code-buffer offset in the label hash table (keyed by BB ID) so that backward branches can compute their displacement immediately. It then walks the linked list of IR instructions and delegates each one to `kcc_codegen_instr`, passing the label table and patch vector as context.

⟨ *CodeGen: compile function* 498a ⟩ +≡

```
.Lcgf_walk_bbs:
    // 7. Walk all BBs: get blocks vec from function
    ldr x24, [x19, #IRF_BLOCKS]    // blocks vec
    ldr x25, [x24, #VEC_COUNT]     // number of BBs
    mov x26, #0                    // BB index

.Lcgf_bb_loop:
    cmp x26, x25
    b.ge .Lcgf_patch

    // Get BB pointer
    mov x0, x24
    mov x1, x26
    bl _kcc_vec_get
    ldr x27, [x0]                  // x27 = current BB

    // Record BB label: bb_id → current code offset
    // Allocate persistent key buffer (8 bytes) so ht stores a unique pointer
    // (All entries must have distinct key pointers for memcmp during lookup)
    mov x0, #8
    bl _kcc_arena_alloc
    mov x28, x0                    // x28 = persistent key buffer
    ldr x1, [x27, #BB_ID]
    str x1, [x28]                  // store bb_id in arena-allocated memory

    // ht_put(bb_labels, key_ptr, 8, code_offset+1)
    bl _kcc_code_current_pos
    add x3, x0, #1                 // value = offset + 1 (distinguish from 0)
    mov x0, x22                    // ht
    mov x1, x28                    // key = arena-allocated pointer to bb_id
    mov x2, #8                     // keylen = 8 bytes
    bl _kcc_ht_put

    // Walk instructions in this BB
    ldr x27, [x27, #BB_FIRST]      // first instruction

.Lcgf_instr_loop:
    cbz x27, .Lcgf_next_bb

    // Generate code for this instruction
    mov x0, x27                    // instruction
    mov x1, x22                    // bb_labels ht
    mov x2, x23                    // patch vec
    bl _kcc_codegen_instr

    // Next instruction
    ldr x27, [x27, #IR_NEXT]
    b .Lcgf_instr_loop

.Lcgf_next_bb:
    add x26, x26, #1
    b .Lcgf_bb_loop
```

□

498a · defined at 493, 496, 498ab · used in 31c · uses _kcc_arena_alloc 58a, _kcc_codegen_instr 504, _kcc_code_current_pos 526a, _kcc_ht_put 63, _kcc_vec_get 65b

Patching Forward Branches and the Symbol Value Once every basic block has been lowered, two bookkeeping tasks remain. First, the forward-branch patch pass resolves all placeholder B and CBNZ instructions whose targets were not yet known at emission time. Second, the function's linker symbol—already created by the pre-pass (§35.4) with the correct underscore-prefixed name, linkage, and extern-table entry, but a placeholder value of zero—has its `SYM_VALUE` patched to the code offset where the function now begins. Registering the symbol earlier (rather than here) is what lets a call to a not-yet-compiled callee resolve to the definition instead of a fresh undefined external; that mattered especially for `static` callees, whose local symbols the linker cannot coalesce a stray undefined reference against. The linkage itself is decided in the pre-pass: external (`N_SECT | N_EXT`) by default, local (`N_SECT` alone) for a `static` function or the synthesised `kcc.module_init`, so two units that each define a private helper of the same name (for instance zlib's per-file `fixedtables`) do not collide.

⟨ *CodeGen: compile function* 498b ⟩ +≡

```
.Lcgf_patch:
    // 8. Patch forward branches
    mov x0, x22                    // bb_labels ht
    mov x1, x23                    // patch vec
    bl _kcc_codegen_patch_branches

    // 9. Patch the pre-registered function symbol's value to the code offset.
    // The symbol itself (with correct linkage and extern-table entry) was
    // created by _kcc_cg_preregister_funcs before any body was compiled, so
```

```
    // forward references have already resolved to
    it; here we only fill in the
    // value that was unknown until the function
   was laid out.
    ldr x26, [x19, #IRF_SYMIDX]    // pre-
   registered symbol index
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    ldr x0, [x0, #VEC_DATA]        // symbol entry
   array
    mov x1, #SYM_ENTRY_SIZE
    mul x1, x26, x1
    add x0, x0, x1                 // &syms[idx]
    str x21, [x0, #SYM_VALUE]      // value =
   func_start_offset

.Lcgf_sym_done:
    ldp x27, x28, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #112
    ret
```

□

498b · defined at 493, 496, 498ab · used in 31c · uses _kcc_cg_preregister_funcs 472, _kcc_codegen_patch_branches 474, _kcc_linker_syms 594

35.12. Vreg Slot Offset

The "all-spill" strategy means every virtual register lives at a fixed stack offset. The frame layout (with x29 pointing to its base) is:

```
[x29 + 0]   : saved x29
[x29 + 8]   : saved x30
[x29 + 16]  : vreg 0
[x29 + 24]  : vreg 1
...
[x29 + 16 + N*8] : vreg N
```

ARM64 unsigned-offset loads and stores encode the immediate as `imm12_scaled` = byte_offset / access_size. For 64-bit accesses the divisor is 8, so the slot at frame index S has $imm12_scaled = (16 + S \times 8)/8 = S + 2$.

The frame index S is *not* the vreg number in general: the slot colouring pass (§35.10) reuses one slot for many spilled vregs whose live ranges do not overlap, and publishes the mapping in `_kcc_cg_slotmap`. `slot_offset` is the single point that turns a vreg into its scaled slot immediate: it applies the map (falling back to the identity $S = vreg$ when no map is published, so the all-spill model is unchanged), then adds the constant 2. Every spill site—the `load_vreg`/`store_vreg` family and the incoming-argument spill—routes through it, so no caller ever assumes slot = vreg.

⟨ *CodeGen: vreg slot offset* 499a ⟩ ≡

```
// -- kcc_cg_slot_offset ---
// uint64_t kcc_cg_slot_offset(uint64_t vreg)
// Returns the scaled imm12 value for a 64-bit LDR
   /STR to the vreg's spill slot:
// imm12_scaled = slotmap[vreg] + 2 (or vreg + 2
   when no colouring map exists).
// Clobbers x0 (return) and x1 (scratch) only.
.text
.globl _kcc_cg_slot_offset
.p2align 2
_kcc_cg_slot_offset:
    adrp x1, _kcc_cg_slotmap@PAGE
    add  x1, x1, _kcc_cg_slotmap@PAGEOFF
    ldr x1, [x1]
    cbz x1, .Lso_ident
    ldr x0, [x1, x0, lsl #3]       // x0 = slotmap[
   vreg]
.Lso_ident:
    add x0, x0, #2                 // imm12_scaled
   = slot + 2
    ret
```

□

499a · defined at 499ab · used in 31c · defines _kcc_cg_slot_offset 496, 499b · uses _kcc_cg_slotmap 479

Register Transfers. With the offset calculation factored out, loading and storing virtual registers becomes a thin wrapper around the ARM64 LDR/STR encoding helpers. `_kcc_cg_load_vreg` emits a 64-bit LDR from the vreg's stack slot into a scratch register (x9–x11), while `_kcc_cg_store_vreg` emits the corresponding STR to spill a scratch register back to the slot. Both use x29 as the base register and the scaled immediate computed by `_kcc_cg_slot_offset`.

⟨ *CodeGen: vreg slot offset* 499b ⟩ + ≡

```
// -- kcc_cg_load_vreg ---
// void kcc_cg_load_vreg(uint64_t vreg, uint64_t
   reg)
//   x0 = vreg number, x1 = target register number
    (9, 10, or 11)
// If the vreg is register-allocated (see the
   allocation map), emits a
// register move; otherwise a LDR from the vreg's
   stack slot.
.text
.globl _kcc_cg_load_vreg
.p2align 2
_kcc_cg_load_vreg:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                    // vreg
    mov x20, x1                    // target reg

    // A floating vreg lives in a d-register; move
    its 64-bit pattern into the
    // requested general register with FMOV so the
    integer paths (a double
    // stored to memory, say) see the right bits.
    adrp x0, _kcc_cg_fpregmap@PAGE
    add  x0, x0, _kcc_cg_fpregmap@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Llv_gp
    ldr x0, [x0, x19, lsl #3]      // home d-reg
   (0 = not FP-allocated)
    cbz x0, .Llv_gp
    mov x1, x0                     // Rn = home d-
   reg
    mov x0, x20                    // Rd = target
   general reg
```

```
    bl _kcc_enc_fmov_x_d            // FMOV x<
   target>, d<home>
    bl _kcc_code_emit32
    b .Llv_done
.Llv_gp:
    adrp x0, _kcc_cg_regmap@PAGE
    add  x0, x0, _kcc_cg_regmap@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Llv_spill
    ldr x0, [x0, x19, lsl #3]       // preg = map[
   vreg] (0 = spilled)
    cbz x0, .Llv_spill
    cmp x20, x0
    b.eq .Llv_done                  // already in
   the target register
    mov x1, x0                      // rm = preg
    mov x0, x20                     // rd = target
   reg
    mov x2, #1                      // sf = 64-bit
    bl _kcc_enc_mov_reg
    bl _kcc_code_emit32
    b .Llv_done
.Llv_spill:
    mov x0, x19                     // vreg →
   colored slot immediate
    bl _kcc_cg_slot_offset
    lsl x1, x0, #3                  // byte offset
   from x29
    mov x0, x20                     // rt = target
   reg
    mov x2, #0                      // kind = LDR
   int
    bl _kcc_cg_emit_frame_ldst
.Llv_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_cg_store_vreg ---
// void kcc_cg_store_vreg(uint64_t vreg, uint64_t
   reg)
//   x0 = vreg number, x1 = source register number
// If the vreg is register-allocated, emits a
   register move into its physical
// register; otherwise a STR to the vreg's stack
   slot.
.text
.globl _kcc_cg_store_vreg
.p2align 2
_kcc_cg_store_vreg:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                     // vreg
    mov x20, x1                     // source reg

    // A floating vreg lives in a d-register;
   write the general-register value
    // (e.g. a materialised double constant, or a
   double loaded from memory)
    // into its home d-register with FMOV.
    adrp x0, _kcc_cg_fpregmap@PAGE
    add  x0, x0, _kcc_cg_fpregmap@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lsv_gp
    ldr x0, [x0, x19, lsl #3]       // home d-reg
   (0 = not FP-allocated)
    cbz x0, .Lsv_gp
    mov x1, x20                     // Rn = source
   general reg
    // x0 = home d-reg = Rd
    bl _kcc_enc_fmov_d_x            // FMOV d<home
   >, x<source>
    bl _kcc_code_emit32
    b .Lsv_done
.Lsv_gp:
    adrp x0, _kcc_cg_regmap@PAGE
    add  x0, x0, _kcc_cg_regmap@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lsv_spill
    ldr x0, [x0, x19, lsl #3]       // preg = map[
   vreg] (0 = spilled)
    cbz x0, .Lsv_spill
    cmp x20, x0
    b.eq .Lsv_done                  // value
   already in its physical register
    mov x1, x20                     // rm = source
   reg
    // x0 already = preg = rd
    mov x2, #1                      // sf = 64-bit
    bl _kcc_enc_mov_reg
    bl _kcc_code_emit32
    b .Lsv_done
.Lsv_spill:
    mov x0, x19                     // vreg →
   colored slot immediate
    bl _kcc_cg_slot_offset
    lsl x1, x0, #3                  // byte offset
   from x29
    mov x0, x20                     // rt = source
   reg
    mov x2, #1                      // kind = STR
   int
    bl _kcc_cg_emit_frame_ldst
.Lsv_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_cg_promoted_reg ---
// uint64_t kcc_cg_promoted_reg(uint64_t vreg)
//   x0 = vreg number
// Returns the physical register holding a
   promoted scalar local's contents
// (mem2reg), or 0 if the vreg is not a promoted
   alloca.  The load/store/alloca
// handlers consult this to route a local's
   accesses to its register instead of
// memory.
.text
.globl _kcc_cg_promoted_reg
.p2align 2
_kcc_cg_promoted_reg:
    adrp x1, _kcc_cg_promoted@PAGE
    add  x1, x1, _kcc_cg_promoted@PAGEOFF
    ldr x1, [x1]
    cbz x1, .Lcpr_zero
    ldr x0, [x1, x0, lsl #3]
    ret
.Lcpr_zero:
    mov x0, #0
    ret

// -- kcc_cg_load_vreg_fp / kcc_cg_store_vreg_fp
   ---
// Move a double between a vreg and a floating-
   point register.
//   x0 = vreg number, x1 = FP register number (0,
    1, or 2)
// A register-allocated vreg lives in a general
   register even when its value
// is a double (its slot is never written), so the
    fp paths bridge with FMOV;
// otherwise a double round-trips through the vreg
   's 8-byte stack slot.
.text
.globl _kcc_cg_load_vreg_fp
.p2align 2
```

```
_kcc_cg_load_vreg_fp:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                     // vreg
    mov x20, x1                     // fp reg

    // Allocated to a d-register?  The value
   already lives in the FP bank; move
    // it d-to-d (or nothing when it is already
   the target register).
    adrp x0, _kcc_cg_fpregmap@PAGE
    add  x0, x0, _kcc_cg_fpregmap@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Llvf_gp
    ldr x0, [x0, x19, lsl #3]       // home d-reg
   (0 = not FP-allocated)
    cbz x0, .Llvf_gp
    cmp x20, x0
    b.eq .Llvf_done                 // already in
   the target d-register
    mov x1, x0                      // Rn = home d-
   reg
    mov x0, x20                     // Rd = target
   d-reg
    bl _kcc_enc_fmov_d_d
    bl _kcc_code_emit32
    b .Llvf_done
.Llvf_gp:
    adrp x0, _kcc_cg_regmap@PAGE
    add  x0, x0, _kcc_cg_regmap@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Llvf_spill
    ldr x0, [x0, x19, lsl #3]       // preg = map[
   vreg]
    cbz x0, .Llvf_spill
    mov x1, x0                      // Rn = preg (
   general)
    mov x0, x20                     // Rd = fp reg
    bl _kcc_enc_fmov_d_x            // FMOV d<reg>,
    x<preg>
    bl _kcc_code_emit32
    b .Llvf_done
.Llvf_spill:
    mov x0, x19                     // vreg →
   colored slot immediate
    bl _kcc_cg_slot_offset
    lsl x1, x0, #3                  // byte offset
   from x29
    mov x0, x20                     // Rt = fp reg
    mov x2, #2                      // kind = LDR
   fp
    bl _kcc_cg_emit_frame_ldst
.Llvf_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

.text
.globl _kcc_cg_store_vreg_fp
.p2align 2
_kcc_cg_store_vreg_fp:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                     // vreg
    mov x20, x1                     // fp reg

    // Allocated to a d-register?  Write the value
    into its home d-register.
    adrp x0, _kcc_cg_fpregmap@PAGE
    add  x0, x0, _kcc_cg_fpregmap@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lsvf_gp
    ldr x0, [x0, x19, lsl #3]       // home d-reg
   (0 = not FP-allocated)
    cbz x0, .Lsvf_gp
    cmp x20, x0
    b.eq .Lsvf_done                 // source
   already is the home d-register
    mov x1, x20                     // Rn = source
   d-reg
    // x0 = home d-reg = Rd
    bl _kcc_enc_fmov_d_d
    bl _kcc_code_emit32
    b .Lsvf_done
.Lsvf_gp:
    adrp x0, _kcc_cg_regmap@PAGE
    add  x0, x0, _kcc_cg_regmap@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lsvf_spill
    ldr x0, [x0, x19, lsl #3]       // preg = map[
   vreg]
    cbz x0, .Lsvf_spill
    mov x1, x20                     // Rn = fp reg
    // x0 already = preg = Rd (general)
    bl _kcc_enc_fmov_x_d            // FMOV x<preg
   >, d<reg>
    bl _kcc_code_emit32
    b .Lsvf_done
.Lsvf_spill:
    mov x0, x19                     // vreg →
   colored slot immediate
    bl _kcc_cg_slot_offset
    lsl x1, x0, #3                  // byte offset
   from x29
    mov x0, x20                     // Rt = fp reg
    mov x2, #3                      // kind = STR
   fp
    bl _kcc_cg_emit_frame_ldst
.Lsvf_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_cg_callee_transfer ---
// void kcc_cg_callee_transfer(uint64_t restore)
//   x0 = 0 → save (STR), non-zero → restore (LDR)
   .  Walks _kcc_cg_callee_mask
//   and emits one transfer per used callee-saved
   register (x19..x28) at
//   [x29, #save_base + k*8]; k counts the
   registers actually saved.
.text
.globl _kcc_cg_callee_transfer
.p2align 2
_kcc_cg_callee_transfer:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                     // restore flag

    adrp x0, _kcc_cg_callee_mask@PAGE
    add  x0, x0, _kcc_cg_callee_mask@PAGEOFF
    ldr x20, [x0]                   // mask
    cbz x20, .Lct_fp
    adrp x0, _kcc_cg_save_base@PAGE
    add  x0, x0, _kcc_cg_save_base@PAGEOFF
    ldr x0, [x0]
    lsr x21, x0, #3                 // save_base as
   an imm12 (scaled by 8)
    mov x22, #19                    // current
   register number
    mov x23, #0                     // k = save-
   slot index
.Lct_loop:
```

```
    cmp x22, #29
    b.ge .Lct_fp
    sub x0, x22, #19
    lsr x1, x20, x0
    tst x1, #1
    b.eq .Lct_next
    add x1, x21, x23              // slot index =
   save_base_scaled + k
    lsl x1, x1, #3                // byte offset
   from x29
    mov x0, x22                   // rt = register
    mov x2, #0                    // kind = LDR
   int (restore)
    cbnz x19, .Lct_kind_ready     // restore → LDR
    mov x2, #1                    // save → STR
   int
.Lct_kind_ready:
    bl _kcc_cg_emit_frame_ldst
    add x23, x23, #1
.Lct_next:
    add x22, x22, #1
    b .Lct_loop
.Lct_fp:
    // Save/restore the callee-saved d-registers (
   d8..d15) the FP scan claimed,
    // at [x29, #fp_save_base + k*8], with the FP
   transfer kinds (LDR/STR fp).
    adrp x0, _kcc_cg_fp_callee_mask@PAGE
    add  x0, x0, _kcc_cg_fp_callee_mask@PAGEOFF
    ldr x20, [x0]                  // fp mask
    cbz x20, .Lct_done
    adrp x0, _kcc_cg_fp_save_base@PAGE
    add  x0, x0, _kcc_cg_fp_save_base@PAGEOFF
    ldr x0, [x0]
    lsr x21, x0, #3               // fp_save_base
   scaled by 8
    mov x22, #8                    // current d-
   register number
    mov x23, #0                    // k = fp-save-
   slot index
.Lct_fp_loop:
    cmp x22, #16
    b.ge .Lct_done
    sub x0, x22, #8
    lsr x1, x20, x0
    tst x1, #1
    b.eq .Lct_fp_next
    add x1, x21, x23
    lsl x1, x1, #3
    mov x0, x22                   // rt = d-
   register
    mov x2, #2                    // kind = LDR fp
    (restore)
    cbnz x19, .Lct_fp_kind
    mov x2, #3                    // kind = STR fp
    (save)
.Lct_fp_kind:
    bl _kcc_cg_emit_frame_ldst
    add x23, x23, #1
.Lct_fp_next:
    add x22, x22, #1
    b .Lct_fp_loop
.Lct_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

499b · defined at 499ab · used in 31c · defines _kcc_cg_callee_transfer 493, 517, _kcc_cg_load_vreg 502, 506, 508, 509, 510, 511, 513, 516, 517, 518, 522b, _kcc_cg_load_vreg_fp 502, 506, 511, 513, 517, _kcc_cg_store_vreg 477b, 505, 506, 508, 509, 510, 511, 513, 518, 521, _kcc_cg_store_vreg_fp 496, 506, 511, 513, 518 · uses _kcc_cg_callee_mask 479, _kcc_cg_emit_frame_ldst 495, _kcc_cg_fpregmap 479, _kcc_cg_fp_callee_mask 479, _kcc_cg_fp_save_base 479, _kcc_cg_promoted 479, _kcc_cg_regmap 479, _kcc_cg_save_base 479, _kcc_cg_slot_offset 499a, _kcc_code_emit32 525b, _kcc_enc_mov_reg 530a

35.13. AAPCS64 Argument Marshaling

Both the direct and indirect call paths share `_kcc_cg_marshal_args`, which realises the AAPCS64 argument-passing rules. The argument array carries, per argument, a vreg and a class (Section 29.9.13 lists the class codes: GP, FP, ST1, ST2, REF). The routine walks the arguments keeping two counters—NGRN, the next general register, and NSRN, the next SIMD/floating register—and loads each argument into `x[ngrn]` or `d[nsrn]` accordingly; an integer, pointer, or (for now) by-reference/by-address aggregate takes one general register, a `double` takes one floating register. After the register phase it also writes a stack image—the arguments from the first-stack index `fsi` onward, argument k at `[sp,#(k-fsi)*8]`—so that a variadic Apple callee, which reads its variadic tail from `[sp,#0]` regardless of registers, finds it in place; the caller restores `sp` with the returned `stack_adj` once the call returns. `fsi` is the callee's fixed-parameter count for a variadic prototype and one otherwise (which also covers the >eight-argument overflow). Because the stack image loads each value from its vreg slot (not from a register), it carries the correct bits for floating arguments even though those now travel in `d` registers.

⟨ *CodeGen: marshal args* 502 ⟩ ≡

```
// -- kcc_cg_marshal_args ---
// uint64_t kcc_cg_marshal_args(uint64_t *args,
   uint64_t count, uint64_t fsi)
//   args = [count, vreg0, class0, ...]; returns
   the stack_adj subtracted from sp.
.text
.globl _kcc_cg_marshal_args
.p2align 2
_kcc_cg_marshal_args:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    mov x19, x0                   // args array
    mov x20, x1                   // arg count
    mov x25, x2                   //
   first_stack_idx
    cbnz x25, .Lma_fsi_ok
    mov x25, #1
.Lma_fsi_ok:
    mov x24, #0                   // stack_adj (
   return value)
    cbz x20, .Lma_done

    // --- register phase: NGRN (x22) / NSRN (x23)
    ---
    mov x21, #0                   // arg index
    mov x22, #0                   // NGRN
```

```
    mov x23, #0                     // NSRN
.Lma_reg_loop:
    cmp x21, x20
    b.ge .Lma_stack
    lsl x0, x21, #1
    add x1, x0, #2
    ldr x1, [x19, x1, lsl #3]     // class
    add x0, x0, #1
    ldr x0, [x19, x0, lsl #3]     // vreg
    cmp x1, #1                      // FP?
    b.eq .Lma_fp
    cmp x1, #5                      // X8 (indirect
   result pointer)?
    b.eq .Lma_x8
    cmp x1, #2                      // ST1: struct
   <=8B, one register
    b.eq .Lma_st1
    cmp x1, #3                      // ST2: struct
   9..16B, two registers
    b.eq .Lma_st2
    // GP / REF: load vreg (value or address) into
    x[ngrn]
    cmp x22, #8
    b.ge .Lma_reg_next
    mov x1, x22
    bl _kcc_cg_load_vreg
    add x22, x22, #1
    b .Lma_reg_next
.Lma_st1:
    cmp x22, #8
    b.ge .Lma_reg_next
    // x0 = struct address vreg → x9 = address,
   then x[ngrn] = [x9]
    mov x1, #9
    bl _kcc_cg_load_vreg
    mov x0, x22
    mov x1, #9
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    add x22, x22, #1
    b .Lma_reg_next
.Lma_st2:
    // Two consecutive general registers; if fewer
    than two remain, overflow.
    cmp x22, #6
    b.gt .Lma_reg_next
    mov x1, #9
    bl _kcc_cg_load_vreg          // x9 = struct
   address
    mov x0, x22                    // x[ngrn] = [
   x9]
    mov x1, #9
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    add x0, x22, #1                // x[ngrn+1] = [
   x9, #8]
    mov x1, #9
    mov x2, #1
    mov x3, #1
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    add x22, x22, #2
    b .Lma_reg_next
.Lma_fp:
    cmp x23, #8
    b.ge .Lma_reg_next
    mov x1, x23
    bl _kcc_cg_load_vreg_fp
    add x23, x23, #1
    b .Lma_reg_next
.Lma_x8:
    // The indirect result pointer travels in x8
   and consumes no NGRN/NSRN.
    mov x1, #8
    bl _kcc_cg_load_vreg
.Lma_reg_next:
    add x21, x21, #1
    b .Lma_reg_loop

.Lma_stack:
    // --- stack image: args[fsi..count-1] at [sp
   ,#(k-fsi)*8] ---
    cmp x20, x25
    b.le .Lma_done
    sub x0, x20, x25
    lsl x0, x0, #3
    add x0, x0, #15
    bic x24, x0, #15              // stack_adj =
   align16((count-fsi)*8)
    mov x0, #31
    mov x1, #31
    mov x2, x24
    mov x3, #1
    bl _kcc_enc_sub_imm
    bl _kcc_code_emit32
    mov x21, x25                   // k = fsi
.Lma_stack_loop:
    cmp x21, x20
    b.ge .Lma_done
    lsl x0, x21, #1
    add x0, x0, #1
    ldr x0, [x19, x0, lsl #3]     // vreg
    mov x1, #9
    bl _kcc_cg_load_vreg
    mov x0, #9
    mov x1, #31
    sub x2, x21, x25              // slot = k - fsi
    mov x3, #1
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
    add x21, x21, #1
    b .Lma_stack_loop

.Lma_done:
    mov x0, x24                    // return
   stack_adj
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

502 · used in 31c · defines _kcc_cg_marshal_args 518 · uses _kcc_cg_load_vreg 499b, _kcc_cg_load_vreg_fp 499b, _kcc_code_emit32 525b, _kcc_enc_ldr_imm 529a, _kcc_enc_str_imm 529a, _kcc_enc_sub_imm 527a

35.14. Instruction Dispatch

The instruction dispatcher is a large switch over IR opcodes. Most cases follow the same pattern: load source vregs into scratch registers, compute, store the result. Two cases deserve special attention.

Function calls and the Apple variadic ABI. On Apple ARM64, variadic functions read their non-fixed arguments from the *stack*, not from registers. The standard AAPCS64 passes arguments in x0–x7, but Apple diverges: the callee's `va_list` walks the stack frame, and—crucially—the variadic tail begins at `[sp,#0]`, immediately past the fixed

parameters, wherever those landed. A correct caller must therefore know how many fixed parameters precede the ellipsis. kcc recovers this count from the prototype: sema records it in the callee function type (§29.9.13), and `kcc_ir_build_call` threads it to the code generator as a *first-stack-index* F packed into the high half of the argument-array header. The marshaller then loads the first eight arguments into x0–x7 as usual *and* writes the variadic tail—`args[F..N-1]`—to the stack starting at `[sp, #0]`. For an ordinary (non-variadic) call F defaults to one, reproducing the legacy layout; the same stack image also feeds a callee with more than eight parameters, whose ninth-and-later arguments are read from this area (§"Storing Register Arguments"). Arguments in the tail are loaded from their vreg slots into scratch x9 before being stored. The emitted sequence is:

1. `SUB sp, sp, #aligned_size`
2. `STR xk, [sp, #(k-F)*8]` for each $k \in [F, N-1]$
3. `BL _callee`
4. `ADD sp, sp, #aligned_size`

String literals. `IR_STRING_LIT` delegates to `kcc_codegen_string_lit` (§35.7), which copies the string bytes into the cstring buffer, creates a local symbol, and emits an ADRP+ADD pair so the generated code gets a pointer to the data.

⟨ *CodeGen: instruction dispatch* 504 ⟩ ≡

```
// -- kcc_codegen_instr ---
// void kcc_codegen_instr(ir_instr *instr, ht *
   bb_labels, vec *patches)
//   x0 = instruction, x1 = bb label ht, x2 =
   patch vec
.text
.globl _kcc_codegen_instr
.p2align 2
_kcc_codegen_instr:
    stp x29, x30, [sp, #-112]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str x27, [sp, #80]

    mov x19, x0                     // instr
    mov x20, x1                     // bb_labels
    mov x21, x2                     // patches

    ldr x22, [x19, #IR_OP]         // opcode

    // Floating-point arithmetic and comparison
   route to a parallel set of
    // handlers.  Only these opcodes use the FP
   unit; IR_CONST/LOAD/STORE
    // also carry a double type but move the raw
   bit pattern, so they stay
    // on the ordinary path.
    ldr x0, [x19, #IR_TYPE]
    cbz x0, .Lcgi_dispatch
    ldr x0, [x0, #TY_KIND]
    cmp x0, #TY_FLOAT
    b.eq .Lcgi_fp_dispatch
    cmp x0, #TY_DOUBLE
    b.ne .Lcgi_dispatch
.Lcgi_fp_dispatch:
    cmp x22, #IR_ADD
    b.eq .Lcgi_fadd
    cmp x22, #IR_SUB
    b.eq .Lcgi_fsub
    cmp x22, #IR_MUL
    b.eq .Lcgi_fmul
    cmp x22, #IR_SDIV
    b.eq .Lcgi_fdiv
    cmp x22, #IR_CMP_EQ
    b.eq .Lcgi_fcmp
    cmp x22, #IR_CMP_NE
    b.eq .Lcgi_fcmp
    cmp x22, #IR_CMP_SLT
    b.eq .Lcgi_fcmp
    cmp x22, #IR_CMP_SLE
    b.eq .Lcgi_fcmp
    cmp x22, #IR_CMP_SGT
    b.eq .Lcgi_fcmp
    cmp x22, #IR_CMP_SGE
    b.eq .Lcgi_fcmp
    // Other fp-typed opcodes (const/load/store):
   ordinary path.

.Lcgi_dispatch:
    // Dispatch on opcode
    cmp x22, #IR_NOP
    b.eq .Lcgi_done

    cmp x22, #IR_CONST
    b.eq .Lcgi_const

    cmp x22, #IR_ADD
    b.eq .Lcgi_add
    cmp x22, #IR_SUB
    b.eq .Lcgi_sub
    cmp x22, #IR_MUL
    b.eq .Lcgi_mul
    cmp x22, #IR_SDIV
    b.eq .Lcgi_sdiv
    cmp x22, #IR_UDIV
    b.eq .Lcgi_udiv
    cmp x22, #IR_SMOD
    b.eq .Lcgi_smod
    cmp x22, #IR_UMOD
    b.eq .Lcgi_umod
    cmp x22, #IR_NEG
    b.eq .Lcgi_neg

    cmp x22, #IR_AND
    b.eq .Lcgi_and
    cmp x22, #IR_OR
    b.eq .Lcgi_or
    cmp x22, #IR_XOR
    b.eq .Lcgi_xor
    cmp x22, #IR_SHL
    b.eq .Lcgi_shl
    cmp x22, #IR_LSHR
    b.eq .Lcgi_lshr
    cmp x22, #IR_ASHR
    b.eq .Lcgi_ashr
    cmp x22, #IR_NOT
    b.eq .Lcgi_not

    cmp x22, #IR_CMP_EQ
    b.eq .Lcgi_cmp_eq
    cmp x22, #IR_CMP_NE
    b.eq .Lcgi_cmp_ne
    cmp x22, #IR_CMP_SLT
    b.eq .Lcgi_cmp_slt
    cmp x22, #IR_CMP_SLE
    b.eq .Lcgi_cmp_sle
    cmp x22, #IR_CMP_SGT
    b.eq .Lcgi_cmp_sgt
    cmp x22, #IR_CMP_SGE
```

```
    b.eq .Lcgi_cmp_sge
    cmp x22, #IR_CMP_ULT
    b.eq .Lcgi_cmp_ult
    cmp x22, #IR_CMP_ULE
    b.eq .Lcgi_cmp_ule

    cmp x22, #IR_LOAD
    b.eq .Lcgi_load
    cmp x22, #IR_STORE
    b.eq .Lcgi_store
    cmp x22, #IR_ALLOCA
    b.eq .Lcgi_alloca
    cmp x22, #IR_GEP
    b.eq .Lcgi_gep

    cmp x22, #IR_BR
    b.eq .Lcgi_br
    cmp x22, #IR_BR_COND
    b.eq .Lcgi_br_cond
    cmp x22, #IR_RET
    b.eq .Lcgi_ret
    cmp x22, #IR_CALL
    b.eq .Lcgi_call
    cmp x22, #IR_CALL_IND
    b.eq .Lcgi_call_ind
    cmp x22, #IR_GLOBAL_REF
    b.eq .Lcgi_global_ref
    cmp x22, #IR_STRING_LIT
    b.eq .Lcgi_string_lit

    cmp x22, #IR_TRUNC
    b.eq .Lcgi_trunc
    cmp x22, #IR_BITCAST
    b.eq .Lcgi_bitcast
    cmp x22, #IR_SEXT
    b.eq .Lcgi_sext
    cmp x22, #IR_ZEXT
    b.eq .Lcgi_zext
    cmp x22, #IR_SITOFP
    b.eq .Lcgi_sitofp
    cmp x22, #IR_UITOFP
    b.eq .Lcgi_uitofp
    cmp x22, #IR_FPTOSI
    b.eq .Lcgi_fptosi
    cmp x22, #IR_VA_START
    b.eq .Lcgi_va_start
    cmp x22, #IR_FPROUND
    b.eq .Lcgi_fpround
    cmp x22, #IR_ASM
    b.eq .Lcgi_asm

    // Unknown opcode: skip
    b .Lcgi_done

.Lcgi_asm:
    // Inline assembly.  The direct encoder cannot
    assemble an arbitrary
    // template, so it records the snippet (
   template pointer/length keyed by the
    // current code offset) for the textual `.s`
   renderer and emits a single NOP
    // placeholder to occupy the slot.  Extended
   asm (with an operand chain on
    // the AST node) additionally loads inputs
   into operand registers, substitutes
    // %N in the template, and stores outputs back
    after the placeholder.
    ldr x0, [x19, #IR_TYPE]        // AST_ASM_STMT
   node
    cbz x0, .Lcgi_asm_basic
    ldr x1, [x0, #AST_F2]          // operand chain
    cbz x1, .Lcgi_asm_basic
    bl _kcc_cg_asm_extended
    b .Lcgi_done
.Lcgi_asm_basic:
    bl _kcc_code_current_pos       // x0 = offset
    ldr x1, [x19, #IR_SRC1]        // template ptr
    ldr x2, [x19, #IR_SRC2]        // template len
    bl _kcc_as_add_snip
    bl _kcc_enc_nop
    bl _kcc_code_emit32
    b .Lcgi_done
```

□

504 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · defines _kcc_codegen_instr 498a · uses _kcc_cg_asm_extended 522b, _kcc_code_current_pos 526a, _kcc_code_emit32 525b, _kcc_enc_nop 533a

35.14.1. Loading Constants

An IR_CONST materialises an arbitrary 64-bit immediate into x11 using a MOVZ + MOVK sequence. MOVZ loads the lowest 16 bits and zeroes the rest; each subsequent MOVK patches one 16-bit half-word lane if it is non-zero. This produces 1–4 instructions depending on the value.

⟨ *CodeGen: instruction dispatch* 505 ⟩ +≡

```
// --- IR_CONST: load immediate into x11, store to
    dest slot ---
.Lcgi_const:
    ldr x23, [x19, #IR_SRC1]       // immediate
   value
    ldr x24, [x19, #IR_DEST]       // dest vreg

    // Emit MOVZ for bits [15:0]
    mov x0, #CG_REG_DST            // rd = x11
    and x1, x23, #0xFFFF           // imm16 = low
   16 bits
    mov x2, #0                      // hw = 0
    mov x3, #1                      // sf = 64-bit
    bl _kcc_enc_movz
    bl _kcc_code_emit32

    // Check if we need MOVK for bits [31:16]
    lsr x0, x23, #16
    and x0, x0, #0xFFFF
    cbz x0, .Lcgi_const_hw2

    mov x0, #CG_REG_DST
    lsr x1, x23, #16
    and x1, x1, #0xFFFF
    mov x2, #1                      // hw = 1
    mov x3, #1
    bl _kcc_enc_movk
    bl _kcc_code_emit32

.Lcgi_const_hw2:
    lsr x0, x23, #32
    and x0, x0, #0xFFFF
    cbz x0, .Lcgi_const_hw3

    mov x0, #CG_REG_DST
    lsr x1, x23, #32
    and x1, x1, #0xFFFF
    mov x2, #2                      // hw = 2
    mov x3, #1
    bl _kcc_enc_movk
    bl _kcc_code_emit32

.Lcgi_const_hw3:
    lsr x0, x23, #48
    and x0, x0, #0xFFFF
    cbz x0, .Lcgi_const_store

    mov x0, #CG_REG_DST
    lsr x1, x23, #48
    and x1, x1, #0xFFFF
```

```
    mov x2, #3                      // hw = 3
    mov x3, #1
    bl _kcc_enc_movk
    bl _kcc_code_emit32

.Lcgi_const_store:
    // Store x11 to dest vreg slot
    mov x0, x24                     // vreg
    mov x1, #CG_REG_DST             // reg = x11
    bl _kcc_cg_store_vreg
    b .Lcgi_done
```

□

505 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · uses _kcc_cg_store_vreg 499b, _kcc_code_emit32 525b, _kcc_enc_movk 527a, _kcc_enc_movz 527a

35.14.2. Arithmetic and Bitwise Operations

All binary arithmetic and bitwise instructions follow the same pattern: load the two source vregs into x9 and x10, perform the operation into x11, and store x11 to the destination slot. The ADD, SUB, MUL, SDIV, and UDIV cases differ only in which encoder they call.

Signed modulo requires two instructions: `sdiv x11, x9, x10` followed by `msub x11, x11, x10, x9`, which computes $x_9 - (x_9/x_{10}) \times x_{10}$. Negation is `sub x11, xzr, x9`.

The *unsigned* divide and remainder are the one place operation width must be honoured, because the high half of a register is not don't-care: an unsigned divide reads all 64 bits, so a sign-extended signed operand (`i % u` with `i` negative, converted to `unsigned int`) would poison a 64-bit UDIV. The handlers therefore pick the `sf` bit from the result type's size—0 (32-bit) for an `unsigned int` remainder, 1 (64-bit) for `unsigned long`—so a 32-bit UDIV ignores the sign-extension exactly as C's conversion to `unsigned int` demands. (Signed SDIV/SMOD are width-agnostic: sign-extended operands give the right low word at either width, so they stay 64-bit.) The IR builder cooperates by recording the operands' common width on the divide's type, the same width it already computes for a comparison.

The bitwise operations (AND, ORR, EOR, shifts, and NOT) use the same load–compute–store pattern with logical-class encoders.

⟨ *CodeGen: instruction dispatch* 506 ⟩ + ≡

```
// --- Binary arithmetic: load →src1x9, →src2x10,
   →opx11, store ---
.Lcgi_add:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_add_reg
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_sub:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_sub_reg
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_mul:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_mul
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_sdiv:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_sdiv
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_udiv:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    // An unsigned divide must run at the
   operation's own width.  A 32-bit
    // `unsigned int` divide done in 64 bits sees
   the sign-extended high half of
    // a signed operand (`i % u` with i<0) and
```

```
    yields a wildly wrong quotient; a
     // 32-bit UDIV ignores the high half, matching
     C's conversion to unsigned
     // int.  sf = (result type is 8 bytes) ? 1 :
    0.
     ldr x0, [x19, #IR_TYPE]
     mov x23, #1
     cmp x0, #0x1000
     b.lo .Lcgi_udiv_sf
     ldr x0, [x0, #TY_SIZE]
     cmp x0, #8
     cset x23, hs
.Lcgi_udiv_sf:
     mov x0, #CG_REG_DST
     mov x1, #CG_REG_SRC1
     mov x2, #CG_REG_SRC2
     mov x3, x23
     bl _kcc_enc_udiv
     bl _kcc_code_emit32

     ldr x0, [x19, #IR_DEST]
     mov x1, #CG_REG_DST
     bl _kcc_cg_store_vreg
     b .Lcgi_done

.Lcgi_smod:
     // SMOD: x11 = x9 - (x9/x10)*x10  → sdiv x11,
    x9,x10; msub x11,x11,x10,x9
     ldr x0, [x19, #IR_SRC1]
     mov x1, #CG_REG_SRC1
     bl _kcc_cg_load_vreg
     ldr x0, [x19, #IR_SRC2]
     mov x1, #CG_REG_SRC2
     bl _kcc_cg_load_vreg

     // sdiv x11, x9, x10
     mov x0, #CG_REG_DST
     mov x1, #CG_REG_SRC1
     mov x2, #CG_REG_SRC2
     mov x3, #1
     bl _kcc_enc_sdiv
     bl _kcc_code_emit32

     // msub x11, x11, x10, x9  (x11 = x9 - x11*x10
    )
     mov x0, #CG_REG_DST            // rd
     mov x1, #CG_REG_DST            // rn (quotient)
     mov x2, #CG_REG_SRC2           // rm (divisor)
     mov x3, #CG_REG_SRC1           // ra (dividend)
     mov x4, #1                      // sf
     bl _kcc_enc_msub
     bl _kcc_code_emit32

     ldr x0, [x19, #IR_DEST]
     mov x1, #CG_REG_DST
     bl _kcc_cg_store_vreg
     b .Lcgi_done

.Lcgi_umod:
     // UMOD: x11 = x9 - (x9/x10)*x10  → udiv x11,
    x9,x10; msub x11,x11,x10,x9
     ldr x0, [x19, #IR_SRC1]
     mov x1, #CG_REG_SRC1
     bl _kcc_cg_load_vreg
     ldr x0, [x19, #IR_SRC2]
     mov x1, #CG_REG_SRC2
     bl _kcc_cg_load_vreg

     // Like UDIV, a 32-bit unsigned modulo must
    run in 32 bits so the sign
     // extension of a signed dividend (`i % u`, i
    <0) does not corrupt it.
     ldr x0, [x19, #IR_TYPE]
     mov x23, #1
     cmp x0, #0x1000
     b.lo .Lcgi_umod_sf
     ldr x0, [x0, #TY_SIZE]
     cmp x0, #8
     cset x23, hs
.Lcgi_umod_sf:
     // udiv x11, x9, x10
     mov x0, #CG_REG_DST
     mov x1, #CG_REG_SRC1
     mov x2, #CG_REG_SRC2
     mov x3, x23
     bl _kcc_enc_udiv
     bl _kcc_code_emit32

     // msub x11, x11, x10, x9  (x11 = x9 - x11*x10
    )
     mov x0, #CG_REG_DST
     mov x1, #CG_REG_DST
     mov x2, #CG_REG_SRC2
     mov x3, #CG_REG_SRC1
     mov x4, x23
     bl _kcc_enc_msub
     bl _kcc_code_emit32

     ldr x0, [x19, #IR_DEST]
     mov x1, #CG_REG_DST
     bl _kcc_cg_store_vreg
     b .Lcgi_done

.Lcgi_neg:
     // A floating NEG negates in the FP unit (FNEG
     d2, d0); an integer NEG is
     // SUB x11, xzr, x9.
     ldr x0, [x19, #IR_TYPE]
     cmp x0, #0x1000
     b.lo .Lcgi_neg_int
     ldr x0, [x0, #TY_KIND]
     cmp x0, #8                      // FLOAT
     b.lo .Lcgi_neg_int
     cmp x0, #10                     // ..LDOUBLE
     b.hi .Lcgi_neg_int
     ldr x0, [x19, #IR_SRC1]
     mov x1, #0                      // d0
     bl _kcc_cg_load_vreg_fp
     mov x0, #2                      // Rd = d2
     mov x1, #0                      // Rn = d0
     bl _kcc_enc_fneg_d
     bl _kcc_code_emit32
     ldr x0, [x19, #IR_DEST]
     mov x1, #2                      // d2
     bl _kcc_cg_store_vreg_fp
     b .Lcgi_done
.Lcgi_neg_int:
     ldr x0, [x19, #IR_SRC1]
     mov x1, #CG_REG_SRC1
     bl _kcc_cg_load_vreg

     mov x0, #CG_REG_DST            // rd
     mov x1, #31                     // rn = xzr
     mov x2, #CG_REG_SRC1           // rm
     mov x3, #1                      // sf
     bl _kcc_enc_sub_reg
     bl _kcc_code_emit32

     ldr x0, [x19, #IR_DEST]
     mov x1, #CG_REG_DST
     bl _kcc_cg_store_vreg
     b .Lcgi_done
```

□

506 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · uses _kcc_cg_load_vreg 499b, _kcc_cg_load_vreg_fp 499b, _kcc_cg_store_vreg 499b, _kcc_cg_store_vreg_fp 499b, _kcc_code_emit32 525b, _kcc_enc_add_reg 526b, _kcc_enc_msub 526b,

_kcc_enc_mul 526b, _kcc_enc_sdiv 526b, _kcc_enc_sub_reg 526b, _kcc_enc_udiv 526b

Bitwise Operations The bitwise group—AND, ORR, EOR, shifts, and MVN—follows the same load–compute–store pattern as arithmetic. Each binary bitwise operation loads the two source vregs into x9 and x10, calls the appropriate logical-class encoder, and stores the result from x11 to the destination slot. Shifts use register-variable forms (LSL, LSR, ASR) rather than immediate encodings, since the shift amount comes from a vreg. The unary NOT (IR_NOT) uses MVN with a single source operand.

⟨ *CodeGen: instruction dispatch* 508 ⟩ + ≡

```
// --- Bitwise operations ---
.Lcgi_and:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_and_reg
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_or:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_orr_reg
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_xor:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_eor_reg
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_shl:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_lsl_reg
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_lshr:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_lsr_reg
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_ashr:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_asr_reg
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_not:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #1                          // sf
    bl _kcc_enc_mvn
    bl _kcc_code_emit32
```

```
    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done
```

□

508 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · uses _kcc_cg_load_vreg 499b, _kcc_cg_store_vreg 499b, _kcc_code_emit32 525b, _kcc_enc_and_reg 527b, _kcc_enc_asr_reg 528a, _kcc_enc_eor_reg 527b, _kcc_enc_lsl_reg 528a, _kcc_enc_lsr_reg 528a, _kcc_enc_mvn 527b, _kcc_enc_orr_reg 527b

35.14.3. Comparison Operations

Integer comparisons emit `CMP x9, x10` (which is really `SUBS xzr, x9, x10`—subtract and set flags, discard result) followed by `CSET x11, cond`. The compare runs at the operands' common width, taken from the instruction's type: a 32-bit type selects a `w`-register compare (`sf=0`), a 64-bit type an `x`-register one. This matters because kcc keeps a signed value sign-extended and an unsigned value zero-extended in its 64-bit register; comparing `int` against a `0xffffffffu` constant at 64 bits would see `0xffff_ffff_ffff_ffff` against `0x0000_0000_ffff_ffff` and wrongly report inequality, whereas the 32-bit compare examines only the low word the standard's usual arithmetic conversions produce. The CSET pseudo-instruction encodes as `CSINC x11, xzr, xzr, inv(cond)`: if the *inverted* condition is false, the result is 1; otherwise 0. All six comparisons share a common tail; each case simply loads the inverted condition code into x23 and falls through:

IR opcode	ARM64 cond	Inverted
CMP_EQ	EQ (0)	NE (1)
CMP_NE	NE (1)	EQ (0)
CMP_SLT	LT (11)	GE (10)
CMP_SLE	LE (13)	GT (12)
CMP_SGT	GT (12)	LE (13)
CMP_SGE	GE (10)	LT (11)
CMP_ULT	LO/CC (3)	HS/CS (2)
CMP_ULE	LS (9)	HI (8)

⟨ *CodeGen: instruction dispatch* 509 ⟩ + ≡

```
// --- Compare operations ---
// Pattern: load →src1x9, →src2x10, CMP x9,x10,
   CSET x11,<inv_cond>

.Lcgi_cmp_eq:
    mov x23, #1                    // inv_cond for
    EQ = NE (1)
    b .Lcgi_cmp_common
.Lcgi_cmp_ne:
    mov x23, #0                    // inv_cond for
    NE = EQ (0)
    b .Lcgi_cmp_common
.Lcgi_cmp_slt:
    mov x23, #10                   // inv_cond for
    LT = GE (10)
    b .Lcgi_cmp_common
.Lcgi_cmp_sle:
    mov x23, #12                   // inv_cond for
    LE = GT (12)
    b .Lcgi_cmp_common
.Lcgi_cmp_sgt:
    mov x23, #13                   // inv_cond for
    GT = LE (13)
    b .Lcgi_cmp_common
.Lcgi_cmp_sge:
    mov x23, #11                   // inv_cond for
    GE = LT (11)
    b .Lcgi_cmp_common
.Lcgi_cmp_ult:
    mov x23, #2                    // inv_cond for
    LO/CC = HS/CS (2)
    b .Lcgi_cmp_common
.Lcgi_cmp_ule:
    mov x23, #8                    // inv_cond for
    LS = HI (8)
    b .Lcgi_cmp_common

.Lcgi_cmp_common:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    // CMP x9, x10 at the operands' common width.
    The IR builder records that
    // width in the comparison's type: a <=32-bit
   common type compares at 32
    // bits (sf=0), so a signed operand (sign-
   extended in its register) and an
    // unsigned one (zero-extended) agree on the
   low word, as C's usual
    // arithmetic conversions require; anything
   wider (long, pointer) keeps the
    // 64-bit compare.  A missing type defaults to
    64-bit.
    ldr x0, [x19, #IR_TYPE]
    cbz x0, .Lcgi_cmp_sf64
    ldr x0, [x0, #TY_SIZE]
    cmp x0, #8
    b.eq .Lcgi_cmp_sf64
    mov x2, #0                     // 32-bit
   compare
    b .Lcgi_cmp_doenc
.Lcgi_cmp_sf64:
    mov x2, #1                     // 64-bit
   compare
.Lcgi_cmp_doenc:
    mov x0, #CG_REG_SRC1
    mov x1, #CG_REG_SRC2
    bl _kcc_enc_cmp_reg
    bl _kcc_code_emit32

    // CSET x11, inv_cond
    mov x0, #CG_REG_DST
    mov x1, x23                    // inv_cond
    mov x2, #1                     // sf
    bl _kcc_enc_cset
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done
```

□

509 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · uses _kcc_cg_load_vreg 499b, _kcc_cg_store_vreg 499b, _kcc_code_emit32 525b, _kcc_enc_cmp_reg 528b, _kcc_enc_cset 528b

35.14.4. Type Conversions

IR_TRUNC masks a value to the width of the target type. The target's TY_SIZE (1, 2, or 4 bytes) selects the mask; the handler loads the mask into x10 and uses AND x11, x9, x10. If no type is attached or the size is 8, the instruction is a plain copy.

IR_SEXT sign-extends a narrow value to 64 bits. The source width is stored in IR_SRC2 (1, 2, or 4 bytes); the handler emits SBFM with the appropriate imms field (7, 15, or 31). IR_ZEXT zero-extends analogously using UBFM.

IR_BITCAST reinterprets the bit pattern as a different type without changing any bits—a no-op in the all-spill strategy.

⟨ *CodeGen: instruction dispatch* 510 ⟩ +≡

```
// --- Type conversions ---

.Lcgi_trunc:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1          // x9 = source
   value
    bl _kcc_cg_load_vreg

    // Get target type size
    ldr x0, [x19, #IR_TYPE]
    cbz x0, .Lcgi_trunc_copy
    ldr x23, [x0, #TY_SIZE]

    cmp x23, #1
    b.eq .Lcgi_trunc_1
    cmp x23, #2
    b.eq .Lcgi_trunc_2
    cmp x23, #4
    b.eq .Lcgi_trunc_4
    b .Lcgi_trunc_copy

.Lcgi_trunc_1:
    // MOVZ x10, #0xFF
    mov x0, #CG_REG_SRC2
    mov x1, #0xFF
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_movz
    bl _kcc_code_emit32
    b .Lcgi_trunc_mask

.Lcgi_trunc_2:
    // MOVZ x10, #0xFFFF
    mov x0, #CG_REG_SRC2
    mov x1, #0xFFFF
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_movz
    bl _kcc_code_emit32
    b .Lcgi_trunc_mask

.Lcgi_trunc_4:
    // MOVZ x10, #0xFFFF; MOVK x10, #0xFFFF, hw=1
    mov x0, #CG_REG_SRC2
    mov x1, #0xFFFF
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_movz
    bl _kcc_code_emit32

    mov x0, #CG_REG_SRC2
    mov x1, #0xFFFF
    mov x2, #1
    mov x3, #1
    bl _kcc_enc_movk
    bl _kcc_code_emit32
    b .Lcgi_trunc_mask

.Lcgi_trunc_mask:
    // AND x11, x9, x10
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #CG_REG_SRC2
    mov x3, #1
    bl _kcc_enc_and_reg
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_trunc_copy:
    // No masking - copy src to dest via ADD x11,
   x9, #0
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_add_imm
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_bitcast:
    // Pure type reinterpretation - copy src to
   dest
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg

    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_add_imm
    bl _kcc_code_emit32

    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_sext:
    // Sign-extend: SRC2 = source width in bytes
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg

    ldr x23, [x19, #IR_SRC2]      // source width
    cmp x23, #1
    b.eq .Lcgi_sext_1
    cmp x23, #2
    b.eq .Lcgi_sext_2
    cmp x23, #4
    b.eq .Lcgi_sext_4
    b .Lcgi_sext_copy

.Lcgi_sext_1:
    // SXTB x11, x9 → SBFM x11, x9, #0, #7 = 0
   x93401D2B
    movz w0, #0x1D2B
    movk w0, #0x9340, lsl #16
    bl _kcc_code_emit32
    b .Lcgi_sext_store

.Lcgi_sext_2:
    // SXTH x11, x9 → SBFM x11, x9, #0, #15 = 0
   x93403D2B
```

```
    movz w0, #0x3D2B
    movk w0, #0x9340, lsl #16
    bl _kcc_code_emit32
    b .Lcgi_sext_store

.Lcgi_sext_4:
    // SXTW x11, x9 → SBFM x11, x9, #0, #31 = 0
   x93407D2B
    movz w0, #0x7D2B
    movk w0, #0x9340, lsl #16
    bl _kcc_code_emit32
    b .Lcgi_sext_store

.Lcgi_sext_copy:
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_add_imm
    bl _kcc_code_emit32

.Lcgi_sext_store:
    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_zext:
    // Zero-extend: SRC2 = source width in bytes
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg

    ldr x23, [x19, #IR_SRC2]
    cmp x23, #1
    b.eq .Lcgi_zext_1
    cmp x23, #2
    b.eq .Lcgi_zext_2
    cmp x23, #4
    b.eq .Lcgi_zext_4
    b .Lcgi_sext_copy              // fallback:
   plain copy

.Lcgi_zext_1:
    // UXTB x11, x9 → UBFM x11, x9, #0, #7 = 0
   xD3401D2B
    movz w0, #0x1D2B
    movk w0, #0xD340, lsl #16
    bl _kcc_code_emit32
    b .Lcgi_sext_store

.Lcgi_zext_2:
    // UXTH x11, x9 → UBFM x11, x9, #0, #15 = 0
   xD3403D2B
    movz w0, #0x3D2B
    movk w0, #0xD340, lsl #16
    bl _kcc_code_emit32
    b .Lcgi_sext_store

.Lcgi_zext_4:
    // →3264: UBFM x11, x9, #0, #31 = 0xD3407D2B
    movz w0, #0x7D2B
    movk w0, #0xD340, lsl #16
    bl _kcc_code_emit32
    b .Lcgi_sext_store
```

□

510 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · uses _kcc_cg_load_vreg 499b, _kcc_cg_store_vreg 499b, _kcc_code_emit32 525b, _kcc_enc_add_imm 527a, _kcc_enc_and_reg 527b, _kcc_enc_movk 527a, _kcc_enc_movz 527a

35.14.5. Floating-Point Operations

The floating-point handlers mirror the integer ones but route their operands through the FP register file. A double lives in the same eight-byte spill slot as any other value, so _kcc_cg_load_vreg_fp reloads it into d0/d1, the two-source op computes into d2, and _kcc_cg_store_vreg_fp spills it back. Comparisons share the FCMP + CSET tail with the integer path, but *not* the same condition codes. A quiet NaN makes FCMP *unordered* (C=1, V=1), and every IEEE comparison against it must yield false. The signed-integer conditions do not honour that: GE/GT (used for </<=) both fire on the unordered flags, so NaN < x and NaN <= x wrongly came out true. The floating relations therefore use the *unordered-false* conditions—< tests MI (via inverted PL) and <= tests LS (via inverted HI); > and >= already map to GT/GE, which reject the unordered case, so they were correct. The conversions bridge the two register files—SCVTF reads an integer slot and writes a double slot, FCVTZS does the reverse with round-toward-zero, matching C's truncating cast. A *signed* source uses SCVTF; an *unsigned* one (IR_UITOFP) uses UCVTF, which differs only in the opcode field. The distinction is invisible for values that fit in 63 bits but decisive at the top: (double)0xFFFFFFFFFFFFFFFF is $\approx 1.8 \times 10^{19}$ under UCVTF, but -1 under SCVTF. The producer (_kcc_ir_emit_int_to_fp) picks the opcode from the source integer's signedness.

⟨ *CodeGen: floating point* 511 ⟩ ≡

```
// --- Floating-point arithmetic: →src1d0, →src2d1
   , →fOPd2, store ---
.Lcgi_fadd:
    mov x23, #2                    // fp dp2
   opcode: fadd
    b .Lcgi_fp_arith
.Lcgi_fsub:
    mov x23, #3                    // fsub
    b .Lcgi_fp_arith
.Lcgi_fmul:
    mov x23, #0                    // fmul
    b .Lcgi_fp_arith
.Lcgi_fdiv:
    mov x23, #1                    // fdiv

.Lcgi_fp_arith:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #0                     // d0
    bl _kcc_cg_load_vreg_fp
    ldr x0, [x19, #IR_SRC2]
    mov x1, #1                     // d1
    bl _kcc_cg_load_vreg_fp
    mov x0, #2                     // Rd = d2
    mov x1, #0                     // Rn = d0
    mov x2, #1                     // Rm = d1
    mov x3, x23                    // opcode
    bl _kcc_enc_fp_dp2
    bl _kcc_code_emit32
    ldr x0, [x19, #IR_DEST]
    mov x1, #2                     // d2
    bl _kcc_cg_store_vreg_fp
    b .Lcgi_done

// --- Floating-point comparison: fcmp d0,d1 then
   cset x11,<inv_cond> ---
.Lcgi_fcmp:
```

```
    mov x23, #1                     // EQ → inv NE
    (1)
    cmp x22, #IR_CMP_EQ
    b.eq .Lcgi_fcmp_do
    mov x23, #0                     // NE → inv EQ
    (0)
    cmp x22, #IR_CMP_NE
    b.eq .Lcgi_fcmp_do
    mov x23, #5                     // SLT → inv PL
    (5): 1 iff MI (ordered a<b)
    cmp x22, #IR_CMP_SLT
    b.eq .Lcgi_fcmp_do
    mov x23, #8                     // SLE → inv HI
    (8): 1 iff LS (ordered a<=b)
    cmp x22, #IR_CMP_SLE
    b.eq .Lcgi_fcmp_do
    mov x23, #13                    // SGT → inv LE
    (13): 1 iff GT (ordered a>b)
    cmp x22, #IR_CMP_SGT
    b.eq .Lcgi_fcmp_do
    mov x23, #11                    // SGE → inv LT
    (11): 1 iff GE (ordered a>=b)

.Lcgi_fcmp_do:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #0
    bl _kcc_cg_load_vreg_fp
    ldr x0, [x19, #IR_SRC2]
    mov x1, #1
    bl _kcc_cg_load_vreg_fp
    mov x0, #0                      // Rn = d0
    mov x1, #1                      // Rm = d1
    bl _kcc_enc_fcmp
    bl _kcc_code_emit32
    mov x0, #CG_REG_DST
    mov x1, x23                     // inverted
   condition
    mov x2, #1
    bl _kcc_enc_cset
    bl _kcc_code_emit32
    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg           // result is an
    integer 0/1
    b .Lcgi_done

// --- Conversions between the integer and
   floating register files ---
.Lcgi_sitofp:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1            // x9
    bl _kcc_cg_load_vreg
    mov x0, #0                      // Rd = d0
    mov x1, #CG_REG_SRC1            // Rn = x9
    bl _kcc_enc_scvtf
    bl _kcc_code_emit32
    ldr x0, [x19, #IR_DEST]
    mov x1, #0                      // d0
    bl _kcc_cg_store_vreg_fp
    b .Lcgi_done

.Lcgi_uitofp:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1            // x9
    bl _kcc_cg_load_vreg
    mov x0, #0                      // Rd = d0
    mov x1, #CG_REG_SRC1            // Rn = x9
    bl _kcc_enc_ucvtf               // unsigned 64-
   bit → double
    bl _kcc_code_emit32
    ldr x0, [x19, #IR_DEST]
    mov x1, #0                      // d0
    bl _kcc_cg_store_vreg_fp
    b .Lcgi_done

.Lcgi_fptosi:
    ldr x0, [x19, #IR_SRC1]
    mov x1, #0                      // d0
    bl _kcc_cg_load_vreg_fp
    mov x0, #CG_REG_SRC1            // Rd = x9 (
   integer)
    mov x1, #0                      // Rn = d0
    bl _kcc_enc_fcvtzs
    bl _kcc_code_emit32
    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_fpround:
    // Round a double to single precision: FCVT s0
   ,d0 then FCVT d0,s0.
    ldr x0, [x19, #IR_SRC1]
    mov x1, #0                      // d0
    bl _kcc_cg_load_vreg_fp
    mov x0, #0                      // Rd = s0
    mov x1, #0                      // Rn = d0
    bl _kcc_enc_fcvt_s_d
    bl _kcc_code_emit32
    mov x0, #0                      // Rd = d0
    mov x1, #0                      // Rn = s0
    bl _kcc_enc_fcvt_d_s
    bl _kcc_code_emit32
    ldr x0, [x19, #IR_DEST]
    mov x1, #0                      // d0
    bl _kcc_cg_store_vreg_fp
    b .Lcgi_done
```

□

511 · used in 31c · uses _kcc_cg_load_vreg 499b, _kcc_cg_load_vreg_fp 499b, _kcc_cg_store_vreg 499b, _kcc_cg_store_vreg_fp 499b, _kcc_code_emit32 525b, _kcc_enc_cset 528b, _kcc_enc_fcmp 530b, _kcc_enc_fcvtzs 530b, _kcc_enc_fcvt_d_s 530b, _kcc_enc_fcvt_s_d 530b, _kcc_enc_fp_dp2 530b, _kcc_enc_scvtf 530b, _kcc_enc_ucvtf 530b

35.14.6. Memory Operations

`IR_LOAD` dereferences a pointer: load the address from the source vreg into x9, then `LDR x11, [x9]` to fetch the value, and store x11 to the destination slot.

`IR_STORE` is the inverse: load the value into x9 and the pointer into x10, then `STR x9, [x10]`. When the `IR_DEST` field is nonzero it acts as an access-size override: 1 → STRB, 2 → STRH, 4 → 32-bit STR. This parallels the `IR_LOAD` override in `IR_SRC2` and is used by the assignment builder for pointer-dereference writes (`*p = v`), where a 64-bit store would corrupt adjacent bytes.

`IR_ALLOCA` computes the *address* of the variable's storage. To avoid the slot-aliasing bug (see Debugging Lessons), alloca storage lives past the vreg value slots. The pre-scan in `kcc_codegen_func` assigns each alloca a running offset (stored in SRC2) and sizes the frame accordingly. The address is `x29 + 16 + vreg_count * 8 + offset * 8`, which is distinct from the vreg's own value slot at `x29 + 16 + dest * 8`.

`IR_GEP` (Get Element Pointer) computes an element or member address as `base + offset`, where the second operand is already a *byte* offset. The IR builder is responsible for byte-scaling: it pre-multiplies an array subscript by the element's real size and passes a struct member's

offset as a constant. Codegen therefore only loads the base pointer and the byte offset from their vreg slots and emits a single `ADD x11, x9, x10`, storing the address in the destination vreg. Collapsing the old fixed stride 8 into a plain add is what lets packed aggregates—whose fields and elements are no longer eight bytes apart—be addressed correctly.

⟨ *CodeGen: instruction dispatch* 513 ⟩ +≡

```
// --- Memory operations ---
.Lcgi_load:
    // If the address is a promoted scalar local (mem2reg), its value lives in a
    // register: read it directly, extending per the width/sign code, instead of
    // touching memory.
    ldr x0, [x19, #IR_SRC1]
    bl _kcc_cg_promoted_reg
    cbnz x0, .Lcgi_load_promoted

    // Load: x9 = pointer from src1 slot, x11 = *x9, store x11 to dest
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg

    // Check for a narrow access (IR_SRC2 = access size override).
    // 1/2/4 = unsigned byte/half/word; 5/6/7 = signed byte/half/word;
    // 8 = single-precision float (4-byte FP load widened to double).
    ldr x23, [x19, #IR_SRC2]
    cmp x23, #8
    b.eq .Lcgi_load_float
    cmp x23, #1
    b.eq .Lcgi_load_8
    cmp x23, #2
    b.eq .Lcgi_load_16
    cmp x23, #4
    b.eq .Lcgi_load_32
    cmp x23, #5
    b.eq .Lcgi_load_s8
    cmp x23, #6
    b.eq .Lcgi_load_s16
    cmp x23, #7
    b.eq .Lcgi_load_s32

    // Default: 64-bit load (works for alloca slots and pointer-width values)
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    b .Lcgi_load_done
.Lcgi_load_8:
    // LDRB x11, [x9] - byte load, zero-extends to 64-bit
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #0
    mov x3, #2
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    b .Lcgi_load_done
.Lcgi_load_16:
    // LDRH x11, [x9] - halfword load, zero-extends to 64-bit
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #0
    mov x3, #3
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    b .Lcgi_load_done
.Lcgi_load_32:
    // LDR w11, [x9] - word load, zero-extends to 64-bit
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #0
    mov x3, #0
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    b .Lcgi_load_done
.Lcgi_load_s8:
    // LDRSB x11, [x9] - byte load, sign-extends to 64-bit
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #0
    mov x3, #5
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    b .Lcgi_load_done
.Lcgi_load_s16:
    // LDRSH x11, [x9] - halfword load, sign-extends to 64-bit
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #0
    mov x3, #6
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    b .Lcgi_load_done
.Lcgi_load_s32:
    // LDRSW x11, [x9] - word load, sign-extends to 64-bit
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #0
    mov x3, #4
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    b .Lcgi_load_done

.Lcgi_load_float:
    // Loading a `float` lvalue: read the 4-byte single into s0 and widen it to
    // the double working format with FCVT, then home it in the destination FP
    // vreg.  The pointer is in CG_REG_SRC1.  This is the exact inverse of the
    // float store, so a `float` round-trips through memory at single precision.
    mov x0, #0                      // Rt = s0
    mov x1, #CG_REG_SRC1            // Rn = pointer
    mov x2, #0                      // imm12 = 0
    bl _kcc_enc_ldr_fp_s            // LDR s0, [ptr]
    bl _kcc_code_emit32
    mov x0, #0                      // Dd = d0
    mov x1, #0                      // Sn = s0
    bl _kcc_enc_fcvt_d_s            // FCVT d0, s0
    bl _kcc_code_emit32
    ldr x0, [x19, #IR_DEST]         // destination vreg
    mov x1, #0                      // from fp reg d0
    bl _kcc_cg_store_vreg_fp
    b .Lcgi_done

.Lcgi_load_done:
    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
```

```
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_load_promoted:
    // x0 = physreg holding the local's contents.
    Produce the loaded value in
    // x11, sign/zero-extended per the SRC2 width
   code, then store to DEST.
    // 0/other = 64-bit (plain move); 1/2/4 = u8/
   u16/u32; 5/6/7 = s8/s16/s32.
    mov x23, x0                    // x23 =
   contents reg
    ldr x24, [x19, #IR_SRC2]       // width/sign
   code
    cbnz x24, .Lclp_narrow
    mov x0, #CG_REG_DST            // rd = x11
    mov x1, x23                    // rm =
   contents reg
    mov x2, #1
    bl _kcc_enc_mov_reg
    bl _kcc_code_emit32
    b .Lclp_store
.Lclp_narrow:
    // Move the contents into x9 so the fixed-form
    extend words (x9 → x11) apply.
    mov x0, #CG_REG_SRC1
    mov x1, x23
    mov x2, #1
    bl _kcc_enc_mov_reg
    bl _kcc_code_emit32
    cmp x24, #1
    b.eq .Lclp_u8
    cmp x24, #2
    b.eq .Lclp_u16
    cmp x24, #4
    b.eq .Lclp_u32
    cmp x24, #5
    b.eq .Lclp_s8
    cmp x24, #6
    b.eq .Lclp_s16
    cmp x24, #7
    b.eq .Lclp_s32
    // Unknown code: plain 64-bit copy x11 ← x9.
    mov x0, #CG_REG_DST
    mov x1, #CG_REG_SRC1
    mov x2, #1
    bl _kcc_enc_mov_reg
    bl _kcc_code_emit32
    b .Lclp_store
.Lclp_u8:                          // UXTB x11, x9
    movz w0, #0x1D2B
    movk w0, #0xD340, lsl #16
    bl _kcc_code_emit32
    b .Lclp_store
.Lclp_u16:                         // UXTH x11, x9
    movz w0, #0x3D2B
    movk w0, #0xD340, lsl #16
    bl _kcc_code_emit32
    b .Lclp_store
.Lclp_u32:                         // UBFM x11, x9
   , #0, #31 →(3264 zero-extend)
    movz w0, #0x7D2B
    movk w0, #0xD340, lsl #16
    bl _kcc_code_emit32
    b .Lclp_store
.Lclp_s8:                          // SXTB x11, x9
    movz w0, #0x1D2B
    movk w0, #0x9340, lsl #16
    bl _kcc_code_emit32
    b .Lclp_store
.Lclp_s16:                         // SXTH x11, x9
    movz w0, #0x3D2B
    movk w0, #0x9340, lsl #16
    bl _kcc_code_emit32
    b .Lclp_store
.Lclp_s32:                         // SXTW x11, x9
    movz w0, #0x7D2B
    movk w0, #0x9340, lsl #16
    bl _kcc_code_emit32
.Lclp_store:
    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_store:
    // If the destination is a promoted scalar
   local (mem2reg), write its value
    // straight into the register that homes it,
   with no memory access.  The
    // width code is irrelevant here: the matching
    load re-normalises on read.
    ldr x0, [x19, #IR_SRC2]
    bl _kcc_cg_promoted_reg
    cbnz x0, .Lcgi_store_promoted

    // Store: x9 = value from src1, x10 = pointer
   from src2, *x10 = x9
    // IR_STORE: SRC1 = value vreg, SRC2 = pointer
    vreg
    // IR_DEST = access size override (0 → 64-bit
   default)
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2
    bl _kcc_cg_load_vreg

    ldr x23, [x19, #IR_DEST]
    cmp x23, #8
    b.eq .Lcgi_store_float
    cmp x23, #1
    b.eq .Lcgi_store_8
    cmp x23, #2
    b.eq .Lcgi_store_16
    cmp x23, #4
    b.eq .Lcgi_store_32

    // Default: 64-bit store (alloca slots and
   pointer-sized values)
    mov x0, #CG_REG_SRC1
    mov x1, #CG_REG_SRC2
    mov x2, #0
    mov x3, #1                     // size = 64-
   bit
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
    b .Lcgi_done

.Lcgi_store_8:
    mov x0, #CG_REG_SRC1
    mov x1, #CG_REG_SRC2
    mov x2, #0
    mov x3, #2                     // size = byte
   (STRB)
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
    b .Lcgi_done

.Lcgi_store_16:
    mov x0, #CG_REG_SRC1
    mov x1, #CG_REG_SRC2
    mov x2, #0
    mov x3, #3                     // size =
   halfword (STRH)
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
    b .Lcgi_done
```

```
.Lcgi_store_32:
    mov x0, #CG_REG_SRC1
    mov x1, #CG_REG_SRC2
    mov x2, #0
    mov x3, #0                          // size = word
   (STR W)
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
    b .Lcgi_done

.Lcgi_store_float:
    // Storing to a `float` lvalue.  The value
   vreg carries a double-format FP
    // number (arithmetic runs in double); narrow
   it to single with FCVT and
    // store the 4-byte single so the memory image
    is a true IEEE-754 float.
    // The pointer is already in CG_REG_SRC2;
   reload the value into d0.
    ldr x0, [x19, #IR_SRC1]         // value vreg
    mov x1, #0                          // fp scratch
   d0
    bl _kcc_cg_load_vreg_fp         // d0 = value (
   double format)
    mov x0, #0                          // Sd = s0
    mov x1, #0                          // Dn = d0
    bl _kcc_enc_fcvt_s_d            // FCVT s0, d0
    bl _kcc_code_emit32
    mov x0, #0                          // Rt = s0
    mov x1, #CG_REG_SRC2            // Rn = pointer
    mov x2, #0                          // imm12 = 0
    bl _kcc_enc_str_fp_s            // STR s0, [ptr
   ]
    bl _kcc_code_emit32
    b .Lcgi_done

.Lcgi_store_promoted:
    // x0 = the local's home register.  Load the
   value into x9 and move it there.
    mov x23, x0                         // x23 =
   contents reg
    ldr x0, [x19, #IR_SRC1]         // value vreg
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg
    mov x0, x23                         // rd =
   contents reg
    mov x1, #CG_REG_SRC1            // rm = x9
    mov x2, #1
    bl _kcc_enc_mov_reg
    bl _kcc_code_emit32
    b .Lcgi_done

.Lcgi_alloca:
    // A promoted scalar local has no stack
   storage: its ALLOCA emits nothing,
    // because the register holding its contents
   is written by the stores above.
    ldr x0, [x19, #IR_DEST]
    bl _kcc_cg_promoted_reg
    cbnz x0, .Lcgi_done

    // Alloca: compute address of storage past the
    vreg value slots.
    // Pre-scan stored the running offset in SRC2.
    // Address = x29 + 16 + max_slot*8 + SRC2*8
    ldr x24, [x19, #IR_DEST]        // dest vreg
    ldr x25, [x19, #IR_SRC2]        // alloca offset
    (from pre-scan)

    ldr x0, [x19, #IR_SRC1]         // slot count (
   for frame calc verification)

    // The alloca storage begins just past the
   vreg value slots, whose count is
    // the colouring pass's max_slot (equals
   vreg_count when colouring is off).
    adrp x0, _kcc_cg_max_slot@PAGE
    add  x0, x0, _kcc_cg_max_slot@PAGEOFF
    ldr x0, [x0]

    // Storage address = x29 + (16 + max_slot*8 +
   offset*8).  The offset can
    // exceed the 12-bit immediate on a large
   frame, so route through the
    // splitting helper rather than the raw
   encoder.
    lsl x1, x0, #3                  // max_slot * 8
    lsl x2, x25, #3                 // offset * 8
    add x2, x2, x1
    add x2, x2, #16                 // imm = 16 +
   vreg_count*8 + offset*8

    mov x0, #CG_REG_DST             // rd = x11
    mov x1, #29                      // rn = x29
    // x2 already set (imm)
    bl _kcc_cg_emit_add_imm

    // Store x11 (the address) to dest's VALUE
   slot
    mov x0, x24
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_gep:
    // GEP: base + index, where index is already a
    byte offset.
    // SRC1 = base ptr vreg, SRC2 = byte-offset
   vreg.  The IR builder
    // pre-scales array indices by the element
   size and passes member
    // offsets as constants, so codegen only adds.
    ldr x0, [x19, #IR_SRC1]
    mov x1, #CG_REG_SRC1            // x9 = base
    bl _kcc_cg_load_vreg
    ldr x0, [x19, #IR_SRC2]
    mov x1, #CG_REG_SRC2            // x10 = byte
   offset
    bl _kcc_cg_load_vreg

    // ADD x11, x9, x10  →  base + byte_offset
    mov x0, #CG_REG_DST             // rd = x11
    mov x1, #CG_REG_SRC1            // rn = x9 (base
   )
    mov x2, #CG_REG_SRC2            // rm = x10 (
   byte offset)
    mov x3, #1                          // sf = 64-bit
    bl _kcc_enc_add_reg
    bl _kcc_code_emit32

    // Store result to dest
    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done

.Lcgi_va_start:
    // va_start: DEST = x29 + frame_size + max(0,
   nfixed-8)*8.  The variadic tail
    // begins at [sp,#0] when nfixed<=8 (the Apple
   /system layout), and after the
    // (nfixed-8) register-overflow fixed
   parameters when nfixed>8.  nfixed rides
    // in the instruction's SRC2.
    adrp x1, _kcc_cg_frame_size@PAGE
    add  x1, x1, _kcc_cg_frame_size@PAGEOFF
```

```
    ldr x25, [x1]                   // x25 =
   frame_size
    ldr x0, [x19, #IR_SRC2]         // nfixed
    subs x0, x0, #8                 // nfixed - 8
    b.le .Lcgi_vs_off_ready         // nfixed <= 8:
   variadic tail at [sp,#0]
    lsl x0, x0, #3                 // (nfixed-8) * 8
    add x25, x25, x0
.Lcgi_vs_off_ready:

    // MOVZ x11, #(offset & 0xffff)
    mov x0, #CG_REG_DST
    and x1, x25, #0xffff
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_movz
    bl _kcc_code_emit32
    // MOVK x11, #((offset>>16)&0xffff), lsl 16
   (only if non-zero)
    lsr x1, x25, #16
    and x1, x1, #0xffff
    cbz x1, .Lcgi_vs_add
    mov x0, #CG_REG_DST
    mov x2, #1                       // hw = 1 →
   shift 16
    mov x3, #1
    bl _kcc_enc_movk
    bl _kcc_code_emit32
.Lcgi_vs_add:
    // ADD x11, x29, x11
    mov x0, #CG_REG_DST
    mov x1, #29
    mov x2, #CG_REG_DST
    mov x3, #1
    bl _kcc_enc_add_reg
    bl _kcc_code_emit32
    // Store the address into the DEST vreg slot
    ldr x0, [x19, #IR_DEST]
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
    b .Lcgi_done
```

□

513 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · uses _kcc_cg_emit_add_imm 495, _kcc_cg_frame_size 479, _kcc_cg_load_vreg 499b, _kcc_cg_load_vreg_fp 499b, _kcc_cg_max_slot 479, _kcc_cg_store_vreg 499b, _kcc_cg_store_vreg_fp 499b, _kcc_code_emit32 525b, _kcc_enc_add_reg 526b, _kcc_enc_fcvt_d_s 530b, _kcc_enc_fcvt_s_d 530b, _kcc_enc_ldr_fp_s 530b, _kcc_enc_ldr_imm 529a, _kcc_enc_movk 527a, _kcc_enc_movz 527a, _kcc_enc_mov_reg 530a, _kcc_enc_str_fp_s 530b, _kcc_enc_str_imm 529a

35.14.7. Control Flow

Branch instructions must handle both backward and forward targets. For IR_BR (unconditional) and IR_BR_COND (conditional), the code generator looks up the target BB in the label hash table. If found, the offset is known and the branch is emitted directly. If not found (a forward reference), a placeholder instruction is emitted and the location is recorded in the patch vector; after all BBs have been processed, the patching pass resolves these.

Conditional branches emit CBNZ x9, true_bb (branch if the condition vreg is non-zero) followed by B false_bb. Either or both targets may be forward references.

IR_RET loads the return value (if non-void) into x0 and emits the function epilogue: LDP x29, x30, [sp], #frame_size followed by RET.

⟨ CodeGen: instruction dispatch 516 ⟩ +≡

```
// --- Control flow ---
.Lcgi_br:
    // Unconditional branch to BB (SRC1 = target
   BB_ID)
    ldr x23, [x19, #IR_SRC1]       // target BB_ID

    // Look up target in bb_labels
    str x23, [sp, #88]             // store bb_id
   for key
    mov x0, x20                     // ht
    add x1, sp, #88                // key = &bb_id
    mov x2, #8                      // keylen
    bl _kcc_ht_get
    cbz x0, .Lcgi_br_forward

    // Backward branch: compute offset
    sub x0, x0, #1                 // undo +1
   encoding
    bl _kcc_code_current_pos
    mov x24, x0                     // current pos

    // Look up again (we lost the value)
    mov x0, x20
    add x1, sp, #88
    mov x2, #8
    bl _kcc_ht_get
    sub x23, x0, #1                // target offset

    // offset = target - current
    sub x0, x23, x24
    bl _kcc_enc_b
    bl _kcc_code_emit32
    b .Lcgi_done

.Lcgi_br_forward:
    // Forward branch: emit placeholder, record
   for patching
    bl _kcc_code_current_pos
    mov x24, x0                     // patch offset

    // Emit NOP as placeholder
    mov x0, #0x14000000             // B with
   offset=0
    bl _kcc_code_emit32

    // Add to patch list: {code_offset,
   target_bb_id}
    str x24, [sp, #88]             // PATCH_OFFSET
    ldr x0, [x19, #IR_SRC1]
    str x0, [sp, #96]             // PATCH_TARGET =
   bb_id

    mov x0, x21                     // patch vec
    add x1, sp, #88                // entry pointer
    bl _kcc_vec_push
    b .Lcgi_done

.Lcgi_br_cond:
    // Conditional branch: DEST=cond vreg, SRC1=
   true BB_ID, SRC2=false BB_ID
    ldr x0, [x19, #IR_DEST]        // condition
   vreg
    mov x1, #CG_REG_SRC1
    bl _kcc_cg_load_vreg

    // CBNZ x9, <true_bb> (may be forward)
    ldr x23, [x19, #IR_SRC1]       // true BB_ID

    // Check if true target is known
```

```
    str x23, [sp, #88]
    mov x0, x20
    add x1, sp, #88
    mov x2, #8
    bl _kcc_ht_get
    cbz x0, .Lcgi_brcond_fwd_true

    // True target is backward
    sub x23, x0, #1                // target offset
    bl _kcc_code_current_pos
    sub x0, x23, x0                // offset =
  target - current
    mov x1, x0                      // offset in
  bytes
    mov x0, #CG_REG_SRC1           // rt = x9
    mov x2, #1                      // sf
    bl _kcc_enc_cbnz
    bl _kcc_code_emit32
    b .Lcgi_brcond_false

.Lcgi_brcond_fwd_true:
    // Forward: emit CBNZ placeholder
    bl _kcc_code_current_pos
    mov x24, x0

    // CBNZ x9, +0 (placeholder)
    mov w0, #0xB5000000             // CBNZ 64-bit
  with imm19=0
    add w0, w0, #CG_REG_SRC1       // rt = x9 (Rt
  field at bits [4:0])
    bl _kcc_code_emit32

    // Record patch: type CBNZ
    str x24, [sp, #88]
    ldr x0, [x19, #IR_SRC1]
    str x0, [sp, #96]
    mov x0, x21
    add x1, sp, #88
    bl _kcc_vec_push

.Lcgi_brcond_false:
    // Now emit B to false BB_ID
    ldr x23, [x19, #IR_SRC2]       // false BB_ID
    str x23, [sp, #88]
    mov x0, x20
    add x1, sp, #88
    mov x2, #8
    bl _kcc_ht_get
    cbz x0, .Lcgi_brcond_fwd_false

    // Backward
    sub x23, x0, #1
    bl _kcc_code_current_pos
    sub x0, x23, x0
    bl _kcc_enc_b
    bl _kcc_code_emit32
    b .Lcgi_done

.Lcgi_brcond_fwd_false:
    bl _kcc_code_current_pos
    mov x24, x0
    mov x0, #0x14000000
    bl _kcc_code_emit32
    str x24, [sp, #88]
    ldr x0, [x19, #IR_SRC2]
    str x0, [sp, #96]
    mov x0, x21
    add x1, sp, #88
    bl _kcc_vec_push
    b .Lcgi_done
```

□

516 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · uses _kcc_cg_load_vreg 499b, _kcc_code_current_pos 526a, _kcc_code_emit32 525b, _kcc_enc_b 532a, _kcc_enc_cbnz 532b, _kcc_ht_get 62, _kcc_vec_push 65a

Function Return The IR_RET handler closes the current function. If the return type is not void, the source vreg is loaded into x0 (the ABI return register). For small frames ($\leq$ 504 bytes) the epilogue uses a post-indexed LDP that restores x29/x30 and deallocates the frame in one instruction; for larger frames it uses a signed-offset LDP followed by ADD sp, sp, #frame_size through the large-immediate helper, mirroring the large-frame prologue so frames past the 12-bit immediate tear down correctly. Both paths finish with RET. The 504-byte limit ensures the LDP post-index immediate ($frame/8$) fits in a signed 7-bit field whose positive range is 0–63.

A struct or union return of *zero* size—an empty struct {} or union {}, a GNU extension kcc accepts—carries no value. The callee must load nothing (its object occupies no bytes, so reading the first eight would dereference past it) and the caller must unpack nothing; both the return path here and the two call-return unpackers guard on a zero TY_SIZE and skip straight to the epilogue / done. Without the guard, returning a union {} faulted—the very crash gcc.c-torture's pr23324 hit.

⟨ *CodeGen: instruction dispatch* 517 ⟩ +≡

```
.Lcgi_ret:
    // Return: load the value to x0 (scalar), d0 (
  floating), or x0(-x1) for a
    // small struct, then emit the epilogue.
    ldr x23, [x19, #IR_SRC1]       // return value
  vreg (0 if void)

    ldr x25, [x19, #IR_TYPE]       // return type (
  kept in x25)
    cbz x25, .Lcgi_ret_epilogue
    ldr x24, [x25]                  // type kind
    cmp x24, #1                     // TY_VOID
    b.eq .Lcgi_ret_epilogue
    cmp x24, #8                     // TY_FLOAT
    b.eq .Lcgi_ret_fp
    cmp x24, #9                     // TY_DOUBLE
    b.eq .Lcgi_ret_fp
    cmp x24, #14                    // TY_STRUCT
    b.eq .Lcgi_ret_struct
    cmp x24, #15                    // TY_UNION
    b.eq .Lcgi_ret_struct

    // Load return value to x0
    mov x0, x23
    mov x1, #0                      // target reg =
  x0
    bl _kcc_cg_load_vreg
    b .Lcgi_ret_epilogue

.Lcgi_ret_fp:
    // A floating return goes in d0 (AAPCS64)
    mov x0, x23
    mov x1, #0                      // d0
    bl _kcc_cg_load_vreg_fp
    b .Lcgi_ret_epilogue

.Lcgi_ret_struct:
    // Small struct (<=16 bytes): x23 = struct
  address vreg, x25 = struct type.
    // Load [addr] into x0, and [addr+8] into x1
  when the struct exceeds 8 bytes.
```

```
    // (x23 may legitimately be vreg 0 -- the
   first alloca -- so it is NOT a
    // "no value" sentinel; a valueless aggregate
   return is caught by the size-0
    // guard below or simply loads an unused
   register, never faulting here.)
    ldr x0, [x25, #TY_SIZE]
    cbz x0, .Lcgi_ret_epilogue      // empty struct
   /union (GNU ext): return nothing
    mov x0, x23
    mov x1, #9                      // x9 = struct
   address
    bl _kcc_cg_load_vreg
    mov x0, #0                      // x0 = [x9]
    mov x1, #9
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32
    ldr x0, [x25, #TY_SIZE]
    cmp x0, #8
    b.le .Lcgi_ret_epilogue
    mov x0, #1                      // x1 = [x9,
   #8]
    mov x1, #9
    mov x2, #1                      // imm12_scaled
    = 1 → byte offset 8
    mov x3, #1
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32

.Lcgi_ret_epilogue:
    // Restore the callee-saved registers (the
   return value already sits in
    // x0/d0), then tear the frame down.
    mov x0, #1
    bl _kcc_cg_callee_transfer

    // Emit epilogue
    adrp x0, _kcc_cg_frame_size@PAGE
    add  x0, x0, _kcc_cg_frame_size@PAGEOFF
    ldr x24, [x0]

    cmp x24, #504
    b.hi .Lcgi_ret_large

    // Small frame: ldp x29, x30, [sp], #
   frame_size
    mov x0, #29
    mov x1, #30
    mov x2, #31
    lsr x3, x24, #3
    bl _kcc_enc_ldp_post
    bl _kcc_code_emit32
    b .Lcgi_ret_emit_ret

.Lcgi_ret_large:
    // Large frame: ldp x29, x30, [sp]  then  add
   sp, sp, #frame_size (split if >4095)
    // ldp x29, x30, [sp, #0] - signed-offset form
   : 0xA9407BFD
    movz w0, #0x7BFD
    movk w0, #0xA940, lsl #16
    bl _kcc_code_emit32
    mov x0, #31                     // rd = sp
    mov x1, #31                     // rn = sp
    mov x2, x24                     // frame_size
    bl _kcc_cg_emit_add_imm

.Lcgi_ret_emit_ret:
    // RET
    bl _kcc_enc_ret
    bl _kcc_code_emit32
    b .Lcgi_done
```

□

517 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · uses _kcc_cg_callee_transfer 499b, _kcc_cg_emit_add_imm 495, _kcc_cg_frame_size 479, _kcc_cg_load_vreg 499b, _kcc_cg_load_vreg_fp 499b, _kcc_code_emit32 525b, _kcc_enc_ldp_post 529b, _kcc_enc_ldr_imm 529a, _kcc_enc_ret 532b

35.14.8. Function Calls and External References

The call sequence is the most complex case. For each IR_CALL:

1. Extract the argument count and load each argument vreg into x0–x7 using the vreg-load helper.
2. If two or more arguments exist, push them onto the stack for variadic ABI compatibility (see the discussion at the start of this section).
3. Prefix the callee name with an underscore (Mach-O convention), register it as an external symbol, record a BRANCH26 relocation, and emit BL with a zero offset (the linker fills in the real target).
4. Store x0 (the return value) to the destination vreg slot.
5. If we adjusted the stack, emit ADD sp, sp, #adj to restore it.

IR_GLOBAL_REF emits an ADRP + ADD pair with PAGE21 and PAGEOFF12 relocations, storing the resulting address in x11. The symbol is registered via kcc_linker_global_sym, which allocates space in the __DATA,__common zerofill section.

IR_STRING_LIT delegates to the string literal handler described in §35.7.

⟨ *CodeGen: instruction dispatch* 518 ⟩ +≡

```
.Lcgi_call:
    // Call: SRC1 = callee name, SRC2 = args array
   , DEST = result vreg.
    // The args array is [count, vreg0, class0,
   vreg1, class1, ...]; the class
    // drives AAPCS64 register selection (see
   kcc_cg_marshal_args).
    ldr x23, [x19, #IR_SRC1]        // callee name
    ldr x24, [x19, #IR_DEST]        // result vreg
    ldr x26, [x19, #IR_SRC2]        // args array (
   or 0 if no args)

    mov x22, #0
    mov x2, #1                      //
   first_stack_idx default
    cbz x26, .Lcgi_call_load
    ldr x22, [x26]                  // count | (
   first_stack_idx << 32)
    lsr x2, x22, #32               // first_stack_idx
    and x22, x22, #0xffffffff       // count
.Lcgi_call_load:
    mov x0, x26
    mov x1, x22
    bl _kcc_cg_marshal_args         // x0 =
   stack_adj
    str x0, [sp, #88]
```

```
    // Build Mach-O name with underscore prefix: "
_" + name
    mov x0, x23
    bl _kcc_strlen
    add x0, x0, #2                  // +1 prefix, +1
    null
    bl _kcc_arena_alloc
    mov x25, x0
    mov w1, #'_'
    strb w1, [x25]
    mov x0, x23
    bl _kcc_strlen
    add x2, x0, #1                  // len including
    null
    add x0, x25, #1                 // dest = after '
_'
    mov x1, x23                     // src =
    original name
    bl _kcc_memcpy

    // Add external symbol
    mov x0, x25
    bl _kcc_linker_extern
    mov x25, x0                     // sym_index

    // Record relocation at current position
    bl _kcc_code_current_pos
    mov x1, #ARM64_RELOC_BRANCH26
    mov x2, x25                     // sym_index
    bl _kcc_linker_add_reloc

    // Emit BL with offset=0 (linker will patch)
    mov x0, #0
    bl _kcc_enc_bl
    bl _kcc_code_emit32

    // Store the return value: d0 (floating), x0(-
    x1) unpacked into the sret
    // buffer (small struct), or x0 (scalar).
    ldr x0, [x19, #IR_TYPE]
    cbz x0, .Lcgi_call_ret_gp
    ldr x0, [x0]
    cmp x0, #8                      // TY_FLOAT
    b.eq .Lcgi_call_ret_fp
    cmp x0, #9                      // TY_DOUBLE
    b.eq .Lcgi_call_ret_fp
    cmp x0, #14                     // TY_STRUCT
    b.eq .Lcgi_call_ret_struct
    cmp x0, #15                     // TY_UNION
    b.eq .Lcgi_call_ret_struct
.Lcgi_call_ret_gp:
    // Normalise the return to its declared width
    before homing it.  The AArch64
    // procedure call standard defines only the
    low bits of a sub-word integer
    // return: a function returning `int` sets w0
    and leaves the top half of x0
    // unspecified.  kcc's own callees happen to
    return a full sign-extended
    // register, but an external libc call does
    not---so a return used directly
    // as a value (`(c = getc(f)) != EOF`, where c
    's assignment yields the call
    // result itself) compared 64 bits of which
    the top half was garbage, and
    // the EOF test never fired---an infinite line
    -reading loop.
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_cg_normalize_ret
    mov x0, x24
    mov x1, #0                      // x0
    bl _kcc_cg_store_vreg
    b .Lcgi_call_ret_done
.Lcgi_call_ret_fp:
    mov x0, x24
    mov x1, #0                      // d0
    bl _kcc_cg_store_vreg_fp
    b .Lcgi_call_ret_done
.Lcgi_call_ret_struct:
    // >16B (REF): the callee already wrote the
    buffer through x8. <=16B:
    // unpack x0(-x1) into the buffer whose
    address is in DEST (x24).
    ldr x1, [x19, #IR_TYPE]
    ldr x1, [x1, #TY_SIZE]
    cbz x1, .Lcgi_call_ret_done    // empty struct/
    union (GNU ext): nothing to unpack
    cmp x1, #16
    b.gt .Lcgi_call_ret_done
    mov x0, x24                     // buffer vreg
    → x9 = its address
    mov x1, #9
    bl _kcc_cg_load_vreg
    mov x0, #0                      // STR x0, [x9]
    mov x1, #9
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
    ldr x0, [x19, #IR_TYPE]
    ldr x0, [x0, #TY_SIZE]
    cmp x0, #8
    b.le .Lcgi_call_ret_done
    mov x0, #1                      // STR x1, [x9,
    #8]
    mov x1, #9
    mov x2, #1
    mov x3, #1
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
.Lcgi_call_ret_done:

    // Restore stack if the marshal pushed a stack
    image: ADD sp, sp, #stack_adj
    ldr x2, [sp, #88]
    cbz x2, .Lcgi_done
    mov x0, #31
    mov x1, #31
    mov x3, #1
    bl _kcc_enc_add_imm
    bl _kcc_code_emit32
    b .Lcgi_done

.Lcgi_call_ind:
    // Indirect call: SRC1 = fn-ptr vreg, SRC2 =
    args array, DEST = result vreg
    ldr x23, [x19, #IR_SRC1]       // fn-ptr vreg
    ldr x24, [x19, #IR_DEST]
    ldr x26, [x19, #IR_SRC2]

    mov x22, #0
    mov x2, #1                      //
    first_stack_idx default
    cbz x26, .Lcgi_cind_load
    ldr x22, [x26]                  // count | (
    first_stack_idx << 32)
    lsr x2, x22, #32              // first_stack_idx
    and x22, x22, #0xffffffff     // count
.Lcgi_cind_load:
    mov x0, x26
    mov x1, x22
    bl _kcc_cg_marshal_args         // x0 =
    stack_adj
    str x0, [sp, #88]

    // Load fn-ptr vreg into x9 (x29-relative,
    unaffected by the stack image)
```

```
    mov x0, x23
    mov x1, #9
    bl _kcc_cg_load_vreg

    // Emit BLR x9
    mov x0, #9
    bl _kcc_enc_blr
    bl _kcc_code_emit32

    // Store the return value: d0 (floating), x0(-
   x1) into the sret buffer
    // (small struct), or x0 (scalar).
    ldr x0, [x19, #IR_TYPE]
    cbz x0, .Lcgi_cind_ret_gp
    ldr x0, [x0]
    cmp x0, #8                          // TY_FLOAT
    b.eq .Lcgi_cind_ret_fp
    cmp x0, #9                          // TY_DOUBLE
    b.eq .Lcgi_cind_ret_fp
    cmp x0, #14                         // TY_STRUCT
    b.eq .Lcgi_cind_ret_struct
    cmp x0, #15                         // TY_UNION
    b.eq .Lcgi_cind_ret_struct
.Lcgi_cind_ret_gp:
    // Same width normalisation as the direct path
   .  kcc lowers every
    // global-function call to global_ref +
   call_ind, so this indirect path
    // carries all of libc---getc, fgetc, and
   friends---and needs the fix too.
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_cg_normalize_ret
    mov x0, x24
    mov x1, #0
    bl _kcc_cg_store_vreg
    b .Lcgi_cind_ret_done
.Lcgi_cind_ret_fp:
    mov x0, x24
    mov x1, #0                          // d0
    bl _kcc_cg_store_vreg_fp
    b .Lcgi_cind_ret_done
.Lcgi_cind_ret_struct:
    ldr x1, [x19, #IR_TYPE]
    ldr x1, [x1, #TY_SIZE]
    cbz x1, .Lcgi_cind_ret_done    // empty struct/
   union (GNU ext): nothing to unpack
    cmp x1, #16
    b.gt .Lcgi_cind_ret_done
    mov x0, x24
    mov x1, #9
    bl _kcc_cg_load_vreg
    mov x0, #0
    mov x1, #9
    mov x2, #0
    mov x3, #1
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
    ldr x0, [x19, #IR_TYPE]
    ldr x0, [x0, #TY_SIZE]
    cmp x0, #8
    b.le .Lcgi_cind_ret_done
    mov x0, #1
    mov x1, #9
    mov x2, #1
    mov x3, #1
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
.Lcgi_cind_ret_done:

    // Restore stack if a stack image was pushed
    ldr x2, [sp, #88]
    cbz x2, .Lcgi_done
    mov x0, #31
    mov x1, #31
    mov x3, #1
    bl _kcc_enc_add_imm
    bl _kcc_code_emit32

    b .Lcgi_done

// -- kcc_cg_normalize_ret ---
// void kcc_cg_normalize_ret(type_desc *ret_type)
// Emits a sign/zero extension of the return
   register x0 to the declared width,
// so a sub-word integer return (from an external
   function, which sets only w0)
// is a valid full-register value.  Nothing is
   emitted for a void, 8-byte, or
// pointer return.  Shared by the direct and
   indirect call return paths.
.p2align 2
_kcc_cg_normalize_ret:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    cbz x0, .Lnr_done
    bl _kcc_ir_load_width           // 5/6/7 signed
   , 1/2/4 unsigned, else none
    cmp x0, #5
    b.eq .Lnr_sxtb
    cmp x0, #6
    b.eq .Lnr_sxth
    cmp x0, #7
    b.eq .Lnr_sxtw
    cmp x0, #1
    b.eq .Lnr_uxtb
    cmp x0, #2
    b.eq .Lnr_uxth
    cmp x0, #4
    b.eq .Lnr_uxtw
    b .Lnr_done
.Lnr_sxtb:
    movz w0, #0x1C00
    movk w0, #0x9340, lsl #16       // SXTB x0, w0
    bl _kcc_code_emit32
    b .Lnr_done
.Lnr_sxth:
    movz w0, #0x3C00
    movk w0, #0x9340, lsl #16       // SXTH x0, w0
    bl _kcc_code_emit32
    b .Lnr_done
.Lnr_sxtw:
    movz w0, #0x7C00
    movk w0, #0x9340, lsl #16       // SXTW x0, w0
    bl _kcc_code_emit32
    b .Lnr_done
.Lnr_uxtb:
    movz w0, #0x1C00
    movk w0, #0x5300, lsl #16       // UXTB w0, w0
    bl _kcc_code_emit32
    b .Lnr_done
.Lnr_uxth:
    movz w0, #0x3C00
    movk w0, #0x5300, lsl #16       // UXTH w0, w0
    bl _kcc_code_emit32
    b .Lnr_done
.Lnr_uxtw:
    movz w0, #0x03E0
    movk w0, #0x2A00, lsl #16       // MOV w0, w0 (
   zero-extend to x0)
    bl _kcc_code_emit32
.Lnr_done:
    ldp x29, x30, [sp], #16
    ret
```

□

518 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · defines _kcc_cg_normalize_ret never used · uses _kcc_arena_alloc 58a, _kcc_cg_load_vreg 499b, _kcc_cg_marshal_args 502, _kcc_cg_store_vreg 499b, _kcc_cg_store_vreg_fp 499b, _kcc_code_current_pos 526a,

_kcc_code_emit32 525b, _kcc_enc_add_imm 527a, _kcc_enc_bl 532a, _kcc_enc_blr 532b, _kcc_enc_str_imm 529a, _kcc_ir_load_width 349b, _kcc_linker_add_reloc 592b, _kcc_linker_extern 592c

Indirect Calls. IR_CALL_IND handles calls through function pointers. The code generator loads the function pointer value from the stack slot identified by SRC1 into scratch register x9, marshals arguments into x0–x7 (with variadic copies on the stack per Apple ABI), and emits BLR x9 instead of BL <symbol>.

Global Variable References and String Literals IR_GLOBAL_REF materialises the address of a global variable using the two-instruction ADRP + ADD idiom. The symbol is registered via kcc_linker_global_sym, which allocates space in the __DATA,__common zerofill section, and two relocations (PAGE21 and PAGEOFF12) are recorded so the linker can patch the instruction pair at link time. IR_STRING_LIT delegates to a dedicated handler described in §35.7. The .Lcgi_done label restores callee-saved registers and returns.

⟨ *CodeGen: instruction dispatch* 521 ⟩ +≡

```
.Lcgi_global_ref:
    // GLOBAL_REF: SRC1 = symbol name (char*),
   DEST = vreg to store addr
    ldr x23, [x19, #IR_SRC1]       // symbol name
    ldr x24, [x19, #IR_DEST]       // dest vreg

    // Build Mach-O name: "_" + name
    mov x0, x23
    bl _kcc_strlen
    add x0, x0, #2                 // +1 for '_',
   +1 for null
    bl _kcc_arena_alloc
    mov x25, x0                     // x25 =
   prefixed name buffer
    mov w1, #'_'
    strb w1, [x25]
    mov x0, x23
    bl _kcc_strlen
    add x2, x0, #1                // len + null
    add x0, x25, #1               // dest = after '
   _'
    mov x1, x23                     // src =
   original name
    bl _kcc_memcpy

    // Defined in this TU, or an external
   reference (extern / another TU)?
    mov x0, x23
    bl _kcc_strlen
    mov x2, x0                      // keylen
    adrp x0, _kcc_cg_defined_globals@PAGE
    add  x0, x0, _kcc_cg_defined_globals@PAGEOFF
    ldr x0, [x0]
    mov x1, x23                     // key = source
    name
    bl _kcc_ht_get
    cbz x0, .Lcgi_gref_extern

    // Defined here: reserve BSS storage (section
   2) and address it directly
    // with an ADRP/ADD pair, since a local symbol
    has an image address.  The
    // preregister pass already minted this symbol
    with its correct linkage
    // (local for a `static'), so this reference
   dedups to that entry and the
    // N_EXT passed here is only used if the
   symbol is somehow not yet defined.
    mov x0, x25
    mov x1, #8                      // default 8
   bytes per scalar global
    mov x2, #N_EXT
    bl _kcc_linker_global_sym
    mov x25, x0                     // sym_index

    bl _kcc_code_current_pos
    mov x1, #ARM64_RELOC_PAGE21
    mov x2, x25
    bl _kcc_linker_add_reloc
    mov x0, #CG_REG_DST            // rd = x11
    mov x1, #0                      // imm21 = 0 (
   linker patches)
    bl _kcc_enc_adrp
    bl _kcc_code_emit32

    bl _kcc_code_current_pos
    mov x1, #ARM64_RELOC_PAGEOFF12
    mov x2, x25
    bl _kcc_linker_add_reloc
    mov x0, #CG_REG_DST            // rd = x11
    mov x1, #CG_REG_DST            // rn = x11
    mov x2, #0                      // imm12 = 0 (
   linker patches)
    mov x3, #1                      // sf = 64-bit
    bl _kcc_enc_add_imm
    bl _kcc_code_emit32
    b .Lcgi_gref_store

.Lcgi_gref_extern:
    // Defined elsewhere (another TU or a dylib
   import): reach the symbol's
    // address through the GOT with ADRP GOTPAGE +
    LDR GOTPAGEOFF, so that
    // dylib-imported data (stdin/stdout/stderr)
   resolves via a bound GOT slot.
    mov x0, x25
    bl _kcc_linker_extern
    mov x25, x0                     // sym_index

    bl _kcc_code_current_pos
    mov x1, #ARM64_RELOC_GOT_LOAD_PAGE21
    mov x2, x25
    bl _kcc_linker_add_reloc
    mov x0, #CG_REG_DST            // rd = x11
    mov x1, #0                      // imm21 = 0 (
   linker patches)
    bl _kcc_enc_adrp
    bl _kcc_code_emit32

    bl _kcc_code_current_pos
    mov x1, #ARM64_RELOC_GOT_LOAD_PAGEOFF12
    mov x2, x25
    bl _kcc_linker_add_reloc
    mov x0, #CG_REG_DST            // rt = x11
    mov x1, #CG_REG_DST            // rn = x11
    mov x2, #0                      // imm12 = 0 (
   linker patches)
    mov x3, #1                      // size = 64-
   bit
    bl _kcc_enc_ldr_imm
    bl _kcc_code_emit32

.Lcgi_gref_store:
    // x11 now holds the global's address (loaded
   from the GOT for externs).
    mov x0, x24
    mov x1, #CG_REG_DST
    bl _kcc_cg_store_vreg
```

```
    b .Lcgi_done

.Lcgi_string_lit:
    mov x0, x19                         // instruction
    bl _kcc_codegen_string_lit
    b .Lcgi_done

.Lcgi_done:
    ldr x27, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #112
    ret
```

□

521 · defined at 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 · used in 31c · uses _kcc_arena_alloc 58a, _kcc_cg_defined_globals 479, _kcc_cg_store_vreg 499b, _kcc_codegen_string_lit 477a, _kcc_code_current_pos 526a, _kcc_code_emit32 525b, _kcc_enc_add_imm 527a, _kcc_enc_adrp 533b, _kcc_enc_ldr_imm 529a, _kcc_ht_get 62, _kcc_linker_add_reloc 592b, _kcc_linker_extern 592c, _kcc_linker_global_sym 593

End-to-end tests: constants and arithmetic. The code generator lowers each arithmetic IR_ADD/IR_SUB/… instruction by loading the two source vregs from their stack slots into scratch registers x9 and x10, executing the ARM64 operation, and storing the result from x11 back to the destination slot. The simplest case—returning a constant—verifies that IR_CONST correctly encodes a MOVZ/MOVK sequence and that IR_RET loads the value into x0 before the epilogue. Subsequent tests exercise each binary operator and confirm that operator precedence (handled at the parser level) is preserved through IR lowering.

⟨ *Test: codegen constants* 522a ⟩ ≡

```
echo ""
echo "--- Constants and Arithmetic ---"

run_test "return constant" \
    "int main(void) { return 42; }" 42

run_test "addition" \
    "int main(void) { return 2 + 3; }" 5

run_test "subtraction" \
    "int main(void) { return 10 - 3; }" 7

run_test "multiplication" \
    "int main(void) { return 6 * 7; }" 42

run_test "division" \
    "int main(void) { return 100 / 10; }" 10

run_test "modulo" \
    "int main(void) { return 17 % 5; }" 2

run_test "complex arithmetic" \
    "int main(void) { return 2 + 3 * 4; }" 14

run_test "negation" \
    "int main(void) { return -(-42); }" 42

run_test "bitwise and" \
    "int main(void) { return 0xFF & 0x0F; }" 15

run_test "bitwise or" \
    "int main(void) { return 0xF0 | 0x0F; }" 255

run_test "bitwise xor" \
    "int main(void) { return 0xFF ^ 0x0F; }" 240

run_test "left shift" \
    "int main(void) { return 1 << 4; }" 16

run_test "right shift" \
    "int main(void) { return 32 >> 2; }" 8
```

□

522a · used in 33b

35.15. Extended Inline Assembly

Extended asm binds C operands to template placeholders. kcc supports the common register and immediate constraints (r, =r, +r, i) with up to two operands, which it places in the always-free scratch registers x9–x10. Inputs are loaded into their operand registers before the template; %N in the template is rewritten to the operand's register; outputs are stored back afterwards, at the operand's width. Anything outside this subset (memory operands, specific registers, three or more operands) is rejected with a diagnostic rather than miscompiled. The two-operand ceiling covers the single-input/output and read-modify-write (+r) forms; a third live operand currently collides in the register file, so it is refused.

⟨ *CodeGen: asm* 522b ⟩ ≡

```
.section __TEXT,__cstring,cstring_literals
_kcc_cg_asm_err:
    .asciz "kcc: error: unsupported extended asm (
   only r/=r/+r/i constraints, up to 2 operands)\n
   "
.text
// -- kcc_cg_asm_validate ---
// uint64_t kcc_cg_asm_validate(node *asm_stmt) -
   0 if the operands are within
// the supported subset, else emit a diagnostic
   and return 1.
.globl _kcc_cg_asm_validate
.p2align 2
_kcc_cg_asm_validate:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    ldr x1, [x0, #AST_F2]          // first operand
    mov x2, #0                     // count
.Lcav_loop:
    cbz x1, .Lcav_count
    add x2, x2, #1
    ldr x3, [x1, #AST_F0]          // constraint ptr
    ldrb w4, [x3]
    cmp w4, #'='
    b.eq .Lcav_skip
    cmp w4, #'+'
    b.eq .Lcav_skip
    b .Lcav_check
.Lcav_skip:
    ldrb w4, [x3, #1]
.Lcav_check:
    cmp w4, #'r'
    b.eq .Lcav_ok
    cmp w4, #'i'
    b.eq .Lcav_ok
    b .Lcav_reject
```

```
.Lcav_ok:
    ldr x1, [x1, #AST_F4]
    b .Lcav_loop
.Lcav_count:
    cmp x2, #2
    b.hi .Lcav_reject
    mov x0, #0
    b .Lcav_done
.Lcav_reject:
    adrp x0, _kcc_cg_asm_err@PAGE
    add  x0, x0, _kcc_cg_asm_err@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
.Lcav_done:
    ldp x29, x30, [sp], #16
    ret

// -- kcc_cg_asm_subst ---
// (char*, uint64_t) kcc_cg_asm_subst(node *
   asm_stmt)
// Build a copy of the template with %N / %wN / %
   xN replaced by the operand's
// register name (operand N -> x/w (9+N)) and %%
   -> %.  Returns ptr in x0, len
// in x1.
.globl _kcc_cg_asm_subst
.p2align 2
_kcc_cg_asm_subst:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    ldr x20, [x0, #AST_F0]        // template ptr
    ldr x21, [x0, #AST_F1]        // template len
    // out buffer: len*4 + 64
    lsl x0, x21, #2
    add x0, x0, #64
    bl _kcc_arena_alloc
    mov x22, x0                   // out buf
    mov x23, #0                   // out pos
    mov x24, #0                   // in pos
.Lcas_loop:
    cmp x24, x21
    b.hs .Lcas_done
    ldrb w0, [x20, x24]
    add x24, x24, #1
    cmp w0, #'%'
    b.ne .Lcas_emit
    // '%': look at the next char
    cmp x24, x21
    b.hs .Lcas_emit_pct           // trailing '%'
    ldrb w0, [x20, x24]
    cmp w0, #'%'
    b.eq .Lcas_dblpct
    mov w25, #'x'                 // default width
    cmp w0, #'w'
    b.eq .Lcas_setw
    cmp w0, #'x'
    b.eq .Lcas_setx
    b .Lcas_num
.Lcas_setw:
    mov w25, #'w'
    add x24, x24, #1
    b .Lcas_num
.Lcas_setx:
    mov w25, #'x'
    add x24, x24, #1
    b .Lcas_num
.Lcas_num:
    // parse operand number into x26
    mov x26, #0
.Lcas_numloop:
    cmp x24, x21
    b.hs .Lcas_numdone
    ldrb w0, [x20, x24]
    cmp w0, #'0'
    b.lt .Lcas_numdone
    cmp w0, #'9'
    b.gt .Lcas_numdone
    sub w0, w0, #'0'
    mov w1, #10
    mul x26, x26, x1
    add x26, x26, x0
    add x24, x24, #1
    b .Lcas_numloop
.Lcas_numdone:
    // emit width char, then decimal of (9 +
   operand number)
    strb w25, [x22, x23]
    add x23, x23, #1
    add x26, x26, #9              // physical
   register number
    cmp x26, #10
    b.lo .Lcas_onedigit
    mov w0, #'1'
    strb w0, [x22, x23]
    add x23, x23, #1
    sub x26, x26, #10
.Lcas_onedigit:
    add w0, w26, #'0'
    strb w0, [x22, x23]
    add x23, x23, #1
    b .Lcas_loop
.Lcas_dblpct:
    add x24, x24, #1              // consume the
   second '%'
    mov w0, #'%'
    b .Lcas_emit
.Lcas_emit_pct:
    mov w0, #'%'
.Lcas_emit:
    strb w0, [x22, x23]
    add x23, x23, #1
    b .Lcas_loop
.Lcas_done:
    mov x0, x22
    mov x1, x23
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret

// -- kcc_cg_asm_extended ---
// void kcc_cg_asm_extended(node *asm_stmt)
// Load inputs into x9..x11, splice the
   substituted template, store outputs.
.globl _kcc_cg_asm_extended
.p2align 2
_kcc_cg_asm_extended:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                    // asm node
    bl _kcc_cg_asm_validate
    cbnz x0, .Lcae_done           // unsupported ->
    emit nothing
    // Load inputs: operand N -> x(9+N).
    ldr x20, [x19, #AST_F2]
    mov x21, #0
.Lcae_inloop:
    cbz x20, .Lcae_inputs_done
    ldr x0, [x20, #AST_F2]        // value half
```

```
    and x0, x0, #0xffffffff
    cbz x0, .Lcae_in_next
    sub x0, x0, #1                // value vreg
    add x1, x21, #9
    bl _kcc_cg_load_vreg
.Lcae_in_next:
    add x21, x21, #1
    ldr x20, [x20, #AST_F4]
    b .Lcae_inloop
.Lcae_inputs_done:
    // Record the substituted template at the
   placeholder offset, emit the NOP.
    bl _kcc_code_current_pos
    mov x22, x0                    // offset
    mov x0, x19
    bl _kcc_cg_asm_subst           // x0 = ptr, x1
   = len
    mov x2, x1
    mov x1, x0
    mov x0, x22
    bl _kcc_as_add_snip
    bl _kcc_enc_nop
    bl _kcc_code_emit32
    // Store outputs: operand N register -> its
   lvalue address (in x8).
    ldr x20, [x19, #AST_F2]
    mov x21, #0
.Lcae_outloop:
    cbz x20, .Lcae_done
    ldr x23, [x20, #AST_F3]       // address half (
   size<<32)|(addr+1)
    and x0, x23, #0xffffffff
    cbz x0, .Lcae_out_next
    sub x0, x0, #1                // address vreg
    mov x1, #8                    // x8 = address
   scratch
    bl _kcc_cg_load_vreg
    lsr x24, x23, #32             // size
    add x0, x21, #9              // Rt = operand
   register
    mov x1, #8                   // Rn = x8
    mov x2, #0                   // imm12
    cmp x24, #1
    b.eq .Lcae_sz8
    cmp x24, #2
    b.eq .Lcae_sz16
    cmp x24, #4
    b.eq .Lcae_sz32
    mov x3, #1                    // 8-byte store (
   str x)
    b .Lcae_str
.Lcae_sz8:
    mov x3, #2
    b .Lcae_str
.Lcae_sz16:
    mov x3, #3
    b .Lcae_str
.Lcae_sz32:
    mov x3, #0
.Lcae_str:
    bl _kcc_enc_str_imm
    bl _kcc_code_emit32
.Lcae_out_next:
    add x21, x21, #1
    ldr x20, [x20, #AST_F4]
    b .Lcae_outloop
.Lcae_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret
```

□

522b · used in 31c · defines _kcc_cg_asm_extended 504, _kcc_cg_asm_subst never used, _kcc_cg_asm_validate never used · uses _kcc_arena_alloc 58a, _kcc_cg_load_vreg 499b, _kcc_code_current_pos 526a, _kcc_code_emit32 525b, _kcc_enc_nop 533a, _kcc_enc_str_imm 529a, _kcc_io_err 71b

36. ARM64 Encoding

The code generator from Chapter 35 decides which ARM64 instructions to emit; this chapter provides the low-level machinery that encodes those instructions into their 32-bit binary representations. It manages a growable code buffer, implements the encoding functions for each instruction format (arithmetic, logical, memory, compare-and-branch, conditional branch, and unconditional branch), and provides the `patch32` facility used to resolve forward branches after their targets are known.

36.1. Code Buffer

The code buffer is a simple growable byte array. Since ARM64 instructions are fixed-width (4 bytes), we always append in 4-byte increments.

⟨ *Backend: constants* 525a ⟩ ≡

```
// -- Backend constants ---
.equ CODEBUF_INIT_CAP, 4096      // initial code
   buffer capacity (bytes)
```

□

525a · used in 30d

The buffer provides four operations: `init` allocates a 4 KB arena block; `emit32` appends one instruction (doubling the buffer when full); `current_pos` returns the write offset (used to compute branch distances and record relocation sites); and `patch32` overwrites an already-emitted instruction (used to resolve forward branches).

⟨ *Backend: code buffer* 525b ⟩ ≡

```
// -- kcc_code_init ---
// void kcc_code_init(void)
// Allocates the initial code buffer.
.text
.globl _kcc_code_init
.p2align 2
_kcc_code_init:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    mov x0, #CODEBUF_INIT_CAP
    bl _kcc_arena_alloc

    adrp x1, _kcc_code_buf@PAGE
    add  x1, x1, _kcc_code_buf@PAGEOFF
    str x0, [x1]

    adrp x1, _kcc_code_pos@PAGE
    add  x1, x1, _kcc_code_pos@PAGEOFF
    str xzr, [x1]

    mov x2, #CODEBUF_INIT_CAP
    adrp x1, _kcc_code_cap@PAGE
    add  x1, x1, _kcc_code_cap@PAGEOFF
    str x2, [x1]

    ldp x29, x30, [sp], #16
    ret

// -- kcc_code_emit32 ---
// void kcc_code_emit32(uint32_t word)
//   x0 = 32-bit instruction word
// Appends a 4-byte instruction to the code buffer
   .
.text
.globl _kcc_code_emit32
.p2align 2
_kcc_code_emit32:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                    // save word

    // Check if we need to grow
    adrp x1, _kcc_code_pos@PAGE
    add  x1, x1, _kcc_code_pos@PAGEOFF
    ldr x2, [x1]                   // pos
    add x3, x2, #4
    adrp x4, _kcc_code_cap@PAGE
    add  x4, x4, _kcc_code_cap@PAGEOFF
    ldr x5, [x4]                   // cap
    cmp x3, x5
    b.le .Lce32_write

    // Grow: allocate new buffer (double capacity)
    lsl x0, x5, #1                 // new cap = old
   * 2
    str x0, [x4]                    // update cap
    mov x5, x0                     // save new cap
    bl _kcc_arena_alloc
    mov x6, x0                     // new buf

    // Copy old data
    adrp x1, _kcc_code_buf@PAGE
    add  x1, x1, _kcc_code_buf@PAGEOFF
    ldr x7, [x1]                    // old buf
    str x6, [x1]                    // update buf
   ptr

    // memcpy(new, old, pos)
    mov x0, x6
    mov x1, x7
    adrp x2, _kcc_code_pos@PAGE
    add  x2, x2, _kcc_code_pos@PAGEOFF
    ldr x2, [x2]
    bl _kcc_memcpy

.Lce32_write:
    adrp x1, _kcc_code_buf@PAGE
    add  x1, x1, _kcc_code_buf@PAGEOFF
    ldr x1, [x1]                   // buf
    adrp x2, _kcc_code_pos@PAGE
    add  x2, x2, _kcc_code_pos@PAGEOFF
    ldr x3, [x2]                   // pos
    str w19, [x1, x3]              // write word at
    buf+pos
    add x3, x3, #4
    str x3, [x2]                   // pos += 4

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

525b · defined at 525b, 526a · used in 30d · defines `_kcc_code_emit32` 477b, 493, 495, 496, 499b, 502, 504, 505, 506, 508, 509, 510, 511, 513, 516, 517, 518, 521, 522b, `_kcc_code_init` 603 · uses `_kcc_arena_alloc` 58a, `_kcc_code_buf` 533c, `_kcc_code_cap` 533c, `_kcc_code_pos` 533c

Inspection and Patching The remaining two operations read or modify the buffer rather than appending to it. `kcc_code_current_pos` returns the byte offset where the next instruction will be written—the codegen uses this to compute branch distances and to record relocation sites for the linker. `kcc_code_patch32` overwrites an already-emitted 32-bit word at a given offset, which is how forward branches are resolved once the target address becomes known.

⟨ *Backend: code buffer* 526a ⟩ + ≡

```
// -- kcc_code_current_pos ---
// uint64_t kcc_code_current_pos(void)
// Returns the current offset in the code buffer.
.text
.globl _kcc_code_current_pos
.p2align 2
_kcc_code_current_pos:
    adrp x0, _kcc_code_pos@PAGE
    add  x0, x0, _kcc_code_pos@PAGEOFF
    ldr x0, [x0]
    ret

// -- kcc_code_patch32 ---
// void kcc_code_patch32(uint64_t offset, uint32_t
    word)
//   x0 = offset in code buffer
//   x1 = new 32-bit word
// Overwrites the instruction at the given offset.
.text
.globl _kcc_code_patch32
.p2align 2
_kcc_code_patch32:
    adrp x2, _kcc_code_buf@PAGE
    add  x2, x2, _kcc_code_buf@PAGEOFF
    ldr x2, [x2]
    str w1, [x2, x0]
    ret
```

□

526a · defined at 525b, 526a · used in 30d · defines _kcc_code_current_pos 477b, 478, 493, 498a, 504, 516, 518, 521, 522b, _kcc_code_patch32 474 · uses _kcc_code_buf 533c, _kcc_code_pos 533c

36.2. Arithmetic Encoding (Register)

All register-form arithmetic instructions share a common encoding: the `sf` bit (bit 31) selects 64-bit vs. 32-bit operation, `Rm` occupies bits [20:16], `Rn` bits [9:5], and `Rd` bits [4:0]. The opcode prefix distinguishes the operation (e.g., 0x0B for ADD, 0x4B for SUB). Multiply uses the three-source MADD form with `Ra=xzr`, and the modulo helper MSUB computes $R_d = R_a - R_n \times R_m$.

⟨ *Backend: encode arithmetic reg* 526b ⟩ ≡

```
// -- kcc_enc_add_reg ---
// uint32_t kcc_enc_add_reg(uint32_t rd, uint32_t
   rn, uint32_t rm, uint32_t sf)
//   x0=rd, x1=rn, x2=rm, x3=sf (1=64-bit, 0=32-
   bit)
.text
.globl _kcc_enc_add_reg
.p2align 2
_kcc_enc_add_reg:
    // ADD: sf=0 → 0x0B000000, sf=1 → 0x8B000000
    mov w4, #0x0B000000
    orr w4, w4, w3, lsl #31       // set sf bit
    orr w4, w4, w2, lsl #16       // Rm
    orr w4, w4, w1, lsl #5        // Rn
    orr w4, w4, w0                // Rd
    mov w0, w4
    ret

// -- kcc_enc_sub_reg ---
.text
.globl _kcc_enc_sub_reg
.p2align 2
_kcc_enc_sub_reg:
    // SUB: sf=0 → 0x4B000000, sf=1 → 0xCB000000
    mov w4, #0x4B000000
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #16
    orr w4, w4, w1, lsl #5
    orr w4, w4, w0
    mov w0, w4
    ret

// -- kcc_enc_mul ---
.text
.globl _kcc_enc_mul
.p2align 2
_kcc_enc_mul:
    // MUL (MADD Rd,Rn,Rm,XZR): sf=0 → 0x1B007C00,
    sf=1 → 0x9B007C00
    mov w4, #0x1B000000
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #16       // Rm
    orr w4, w4, #(31 << 10)       // Ra = xzr (31)
    orr w4, w4, w1, lsl #5        // Rn
    orr w4, w4, w0                // Rd
    mov w0, w4
    ret

// -- kcc_enc_sdiv ---
.text
.globl _kcc_enc_sdiv
.p2align 2
_kcc_enc_sdiv:
    // SDIV: sf=0 → 0x1AC00C00, sf=1 → 0x9AC00C00
    movz w4, #0x1AC0, lsl #16
    movk w4, #0x0C00
    orr w4, w4, w3, lsl #31       // sf
    orr w4, w4, w2, lsl #16       // Rm
    orr w4, w4, w1, lsl #5        // Rn
    orr w4, w4, w0                // Rd
    mov w0, w4
    ret

// -- kcc_enc_udiv ---
.text
.globl _kcc_enc_udiv
.p2align 2
_kcc_enc_udiv:
    // UDIV: sf=0 → 0x1AC00800, sf=1 → 0x9AC00800
    movz w4, #0x1AC0, lsl #16
    movk w4, #0x0800
    orr w4, w4, w3, lsl #31       // sf
    orr w4, w4, w2, lsl #16       // Rm
    orr w4, w4, w1, lsl #5        // Rn
    orr w4, w4, w0                // Rd
    mov w0, w4
    ret

// -- kcc_enc_msub ---
// MSUB Rd, Rn, Rm, Ra: Rd = Ra - Rn*Rm
// x0=rd, x1=rn, x2=rm, x3=ra, x4=sf
.text
.globl _kcc_enc_msub
.p2align 2
```

```
_kcc_enc_msub:
    // MSUB: sf=0 → 0x1B008000, sf=1 → 0x9B008000
    mov w5, #0x1B000000
    orr w5, w5, w4, lsl #31        // sf
    orr w5, w5, #0x8000            // o0 bit (
   distinguishes MSUB from MADD)
    orr w5, w5, w2, lsl #16        // Rm
    orr w5, w5, w3, lsl #10        // Ra
    orr w5, w5, w1, lsl #5         // Rn
    orr w5, w5, w0                 // Rd
    mov w0, w5
    ret
```

□

526b · used in 30d · defines _kcc_enc_add_reg 506, 513, _kcc_enc_msub 506, _kcc_enc_mul 506, _kcc_enc_sdiv 506, _kcc_enc_sub_reg 506, _kcc_enc_udiv 506

36.3. Arithmetic Encoding (Immediate)

Immediate-form arithmetic encodes a 12-bit unsigned immediate in bits [21:10]. MOVZ and MOVK together materialise arbitrary 64-bit constants: MOVZ zeroes the register and inserts a 16-bit chunk at the half-word position selected by `hw`, while MOVK overwrites one 16-bit lane without disturbing the rest.

⟨ *Backend: encode arithmetic imm* 527a ⟩ ≡

```
// -- kcc_enc_add_imm ---
// uint32_t kcc_enc_add_imm(uint32_t rd, uint32_t
   rn, uint32_t imm12, uint32_t sf)
.text
.globl _kcc_enc_add_imm
.p2align 2
_kcc_enc_add_imm:
    // ADD imm: sf=0 → 0x11000000, sf=1 → 0
   x91000000
    mov w4, #0x11000000
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #10        // imm12
    orr w4, w4, w1, lsl #5         // Rn
    orr w4, w4, w0                 // Rd
    mov w0, w4
    ret

// -- kcc_enc_sub_imm ---
.text
.globl _kcc_enc_sub_imm
.p2align 2
_kcc_enc_sub_imm:
    // SUB imm: sf=0 → 0x51000000, sf=1 → 0
   xD1000000
    mov w4, #0x51000000
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #10        // imm12
    orr w4, w4, w1, lsl #5         // Rn
    orr w4, w4, w0                 // Rd
    mov w0, w4
    ret

// -- kcc_enc_movz ---
// uint32_t kcc_enc_movz(uint32_t rd, uint32_t
   imm16, uint32_t hw, uint32_t sf)
//   hw = shift amount / 16 (0,1,2,3)
.text
.globl _kcc_enc_movz
.p2align 2
_kcc_enc_movz:
    // MOVZ: sf=0 → 0x52800000, sf=1 → 0xD2800000
    mov w4, #0x52800000
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #21        // hw
    orr w4, w4, w1, lsl #5         // imm16
    orr w4, w4, w0                 // Rd
    mov w0, w4
    ret

// -- kcc_enc_movk ---
.text
.globl _kcc_enc_movk
.p2align 2
_kcc_enc_movk:
    // MOVK: sf=0 → 0x72800000, sf=1 → 0xF2800000
    mov w4, #0x72800000
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #21        // hw
    orr w4, w4, w1, lsl #5         // imm16
    orr w4, w4, w0                 // Rd
    mov w0, w4
    ret
```

□

527a · used in 30d · defines _kcc_enc_add_imm 477b, 493, 495, 510, 518, 521, _kcc_enc_movk 505, 510, 513, _kcc_enc_movz 505, 510, 513, _kcc_enc_sub_imm 495, 502

36.4. Logical Encoding

Logical register operations use the same (sf, R_m, R_n, R_d) layout as arithmetic, with opcode prefixes 0x0A (AND), 0x2A (ORR), and 0x4A (EOR). Bitwise NOT is encoded as ORN R_d, xzr, R_m (the `N` bit at position 21 selects the inverted second operand). The variable-distance shifts (LSLV, LSRV, ASRV) live in the Data Processing (2-source) space with distinct sub-opcodes in bits [11:10].

⟨ *Backend: encode logical* 527b ⟩ ≡

```
// -- kcc_enc_and_reg ---
.text
.globl _kcc_enc_and_reg
.p2align 2
_kcc_enc_and_reg:
    // AND: sf=0 → 0x0A000000, sf=1 → 0x8A000000
    mov w4, #0x0A000000
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #16
    orr w4, w4, w1, lsl #5
    orr w4, w4, w0
    mov w0, w4
    ret

// -- kcc_enc_orr_reg ---
.text
.globl _kcc_enc_orr_reg
.p2align 2
_kcc_enc_orr_reg:
    // ORR: sf=0 → 0x2A000000, sf=1 → 0xAA000000
    mov w4, #0x2A000000
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #16
    orr w4, w4, w1, lsl #5
    orr w4, w4, w0
    mov w0, w4
    ret

// -- kcc_enc_eor_reg ---
.text
.globl _kcc_enc_eor_reg
.p2align 2
_kcc_enc_eor_reg:
```

```
    // EOR: sf=0 → 0x4A000000, sf=1 → 0xCA000000
    mov w4, #0x4A000000
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #16
    orr w4, w4, w1, lsl #5
    orr w4, w4, w0
    mov w0, w4
    ret

// -- kcc_enc_mvn ---
// MVN Rd, Rm → ORN Rd, XZR, Rm
// x0=rd, x1=rm, x2=sf
.text
.globl _kcc_enc_mvn
.p2align 2
_kcc_enc_mvn:
    // ORN: sf=0 → 0x2A200000, sf=1 → 0xAA200000
    mov w4, #0x2A200000
    orr w4, w4, w2, lsl #31        // sf
    orr w4, w4, w1, lsl #16        // Rm
    mov w5, #31
    orr w4, w4, w5, lsl #5         // Rn = XZR (31)
    orr w4, w4, w0                 // Rd
    mov w0, w4
    ret
```

□

527b · defined at 527b, 528a · used in 30d · defines _kcc_enc_and_reg 508, 510, _kcc_enc_eor_reg 508, _kcc_enc_mvn 508, _kcc_enc_orr_reg 508

Variable-Distance Shifts The variable-distance shift instructions LSLV, LSRV, and ASRV belong to a different ARM64 instruction class—Data Processing (2-source)—rather than the Logical Shifted Register group above. They share the same opcode prefix 0x1AC0 (with sf at bit 31) and differ only in the sub-opcode at bits [11:10]: 0x2000 for left shift, 0x2400 for logical right shift, and 0x2800 for arithmetic right shift. The shift amount comes from register Rm rather than an immediate.

⟨ *Backend: encode logical* 528a ⟩ + ≡

```
// -- kcc_enc_lsl_reg ---
// LSLV: x0=rd, x1=rn, x2=rm, x3=sf
.text
.globl _kcc_enc_lsl_reg
.p2align 2
_kcc_enc_lsl_reg:
    // LSLV: sf=0 → 0x1AC02000, sf=1 → 0x9AC02000
    movz w4, #0x1AC0, lsl #16
    movk w4, #0x2000
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #16
    orr w4, w4, w1, lsl #5
    orr w4, w4, w0
    mov w0, w4
    ret

// -- kcc_enc_lsr_reg ---
.text
.globl _kcc_enc_lsr_reg
.p2align 2
_kcc_enc_lsr_reg:
    // LSRV: sf=0 → 0x1AC02400, sf=1 → 0x9AC02400
    movz w4, #0x1AC0, lsl #16
    movk w4, #0x2400
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #16
    orr w4, w4, w1, lsl #5
    orr w4, w4, w0
    mov w0, w4
    ret

// -- kcc_enc_asr_reg ---
.text
.globl _kcc_enc_asr_reg
.p2align 2
_kcc_enc_asr_reg:
    // ASRV: sf=0 → 0x1AC02800, sf=1 → 0x9AC02800
    movz w4, #0x1AC0, lsl #16
    movk w4, #0x2800
    orr w4, w4, w3, lsl #31
    orr w4, w4, w2, lsl #16
    orr w4, w4, w1, lsl #5
    orr w4, w4, w0
    mov w0, w4
    ret
```

□

528a · defined at 527b, 528a · used in 30d · defines _kcc_enc_asr_reg 508, _kcc_enc_lsl_reg 508, _kcc_enc_lsr_reg 508

36.5. Compare and Conditional Set

Comparison is encoded as SUBS xzr, Rn, Rm—a subtract whose result is discarded but whose flags are set. CSET is a pseudo-instruction for CSINC Rd, xzr, xzr, inv(cond): if the *inverted* condition is false (meaning the original condition is true), the result increments zero to one; otherwise it stays zero. Bit 10 distinguishes CSINC from CSEL.

⟨ *Backend: encode compare* 528b ⟩ ≡

```
// -- kcc_enc_cmp_reg ---
// CMP Rn, Rm → SUBS XZR, Rn, Rm
// x0=rn, x1=rm, x2=sf
.text
.globl _kcc_enc_cmp_reg
.p2align 2
_kcc_enc_cmp_reg:
    // SUBS: sf=0 → 0x6B000000, sf=1 → 0xEB000000
    mov w3, #0x6B000000
    orr w3, w3, w2, lsl #31
    orr w3, w3, w1, lsl #16        // Rm
    orr w3, w3, w0, lsl #5         // Rn
    mov w4, #31
    orr w3, w3, w4                 // Rd = XZR (31)
    mov w0, w3
    ret

// -- kcc_enc_cset ---
// CSET Rd, cond → CSINC Rd, XZR, XZR, inv(cond)
// x0=rd, x1=cond (already inverted!), x2=sf
.text
.globl _kcc_enc_cset
.p2align 2
_kcc_enc_cset:
    // CSINC: sf=0 → 0x1A9F07E0, sf=1 → 0x9A9F07E0
    // Encoding: sf|0|0|11010100|Rm(=11111)|cond
   |01|Rn(=11111)|Rd
    // = sf | 0x1A800000 | 31<<16 | cond<<12 |
   1<<10 | 31<<5 | rd
    mov w3, #0x1A800000
    orr w3, w3, #0x400             // bit 10 = 1 →
   CSINC (not CSEL)
    orr w3, w3, w2, lsl #31        // sf
    mov w4, #31
    orr w3, w3, w4, lsl #16        // Rm = XZR
    orr w3, w3, w1, lsl #12        // cond (
   inverted)
    orr w3, w3, w4, lsl #5         // Rn = XZR
```

```
    orr w3, w3, w0                  // Rd
    mov w0, w3
    ret
```

□

528b · used in 30d · defines _kcc_enc_cmp_reg 509, _kcc_enc_cset 509, 511

36.6. Memory Encoding

Load and store use the unsigned-offset form: the 12-bit immediate is pre-scaled by the access size (divided by 8 for 64-bit, by 4 for 32-bit). The load encoder also provides the three sign-extending narrow forms LDRSB, LDRSH, and LDRSW (size codes 5, 6, 4), which set opc = 10 to extend a byte, halfword, or word into the full 64-bit register—needed now that packed struct fields and array elements are addressed at their natural width and a signed narrow value must keep its sign in the eight-byte spill slot. STP (pre-index) and LDP (post-index) use a 7-bit signed immediate scaled by 8, encoded in bits [21:15]. MOV R_d, R_m is encoded as `ORR Rd, xzr, Rm`.

⟨ *Backend: encode memory* 529a ⟩ ≡

```
// -- kcc_enc_str_imm ---
// STR (unsigned offset): x0=rt, x1=rn, x2=
   imm12_scaled,
//   x3=size(0=32,1=64,2=byte,3=half)
.text
.globl _kcc_enc_str_imm
.p2align 2
_kcc_enc_str_imm:
    cmp w3, #2
    b.eq .Lstr_8
    cmp w3, #3
    b.eq .Lstr_16
    cbz w3, .Lstr_32
    mov w4, #0xF9000000             // 64-bit
    b .Lstr_common
.Lstr_32:
    mov w4, #0xB9000000             // 32-bit
    b .Lstr_common
.Lstr_8:
    mov w4, #0x39000000             // 8-bit (STRB)
    b .Lstr_common
.Lstr_16:
    mov w4, #0x79000000             // 16-bit (STRH
   )
.Lstr_common:
    orr w4, w4, w2, lsl #10         // imm12 (pre-
   scaled)
    orr w4, w4, w1, lsl #5          // Rn
    orr w4, w4, w0                  // Rt
    mov w0, w4
    ret

// -- kcc_enc_ldr_imm ---
// LDR (unsigned offset): x0=rt, x1=rn, x2=
   imm12_scaled,
//   x3=size(0=32,1=64,2=byte,3=half,4=LDRSW,5=
   LDRSB,6=LDRSH)
// Codes 4--6 sign-extend a narrow load into the
   full 64-bit register
// (opc=10 in bits [23:22]); the unsigned forms
   0/2/3 zero-extend (opc=01).
.text
.globl _kcc_enc_ldr_imm
.p2align 2
_kcc_enc_ldr_imm:
    cmp w3, #2
    b.eq .Lldr_8
    cmp w3, #3
    b.eq .Lldr_16
    cmp w3, #4
    b.eq .Lldr_sw
    cmp w3, #5
    b.eq .Lldr_sb
    cmp w3, #6
    b.eq .Lldr_sh
    cbz w3, .Lldr_32
    mov w4, #0xF9400000             // 64-bit
    b .Lldr_common
.Lldr_32:
    mov w4, #0xB9400000             // 32-bit
    b .Lldr_common
.Lldr_8:
    mov w4, #0x39400000             // 8-bit (LDRB)
    b .Lldr_common
.Lldr_16:
    mov w4, #0x79400000             // 16-bit (LDRH
   )
    b .Lldr_common
.Lldr_sw:
    mov w4, #0xB9800000             // LDRSW →(3264
    signed)
    b .Lldr_common
.Lldr_sb:
    mov w4, #0x39800000             // LDRSB →(864
   signed)
    b .Lldr_common
.Lldr_sh:
    mov w4, #0x79800000             // LDRSH →(1664
    signed)
.Lldr_common:
    orr w4, w4, w2, lsl #10         // imm12 (pre-
   scaled)
    orr w4, w4, w1, lsl #5          // Rn
    orr w4, w4, w0                  // Rt
    mov w0, w4
    ret
```

□

529a · defined at 529ab, 530a · used in 30d · defines _kcc_enc_ldr_imm 495, 502, 513, 517, 521, _kcc_enc_str_imm 495, 496, 502, 513, 518, 522b

Paired Load/Store The STP (store pair, pre-index) and LDP (load pair, post-index) instructions transfer two 64-bit registers in a single operation. They are used almost exclusively in function prologues and epilogues to save and restore x29/x30. The 7-bit signed immediate in bits [21:15] is scaled by 8, giving a ±512-byte range from the base register.

⟨ *Backend: encode memory* 529b ⟩ + ≡

```
// -- kcc_enc_stp_pre ---
// STP (pre-index, 64-bit): x0=rt1, x1=rt2, x2=rn,
    x3=imm7 (signed, scaled by 8)
// Encoding: 0xA9800000 | (imm7&0x7F)<<15 | rt2
   <<10 | rn<<5 | rt1
.text
.globl _kcc_enc_stp_pre
.p2align 2
_kcc_enc_stp_pre:
    mov w4, #0xA9800000
    and w5, w3, #0x7F
    orr w4, w4, w5, lsl #15         // imm7
    orr w4, w4, w1, lsl #10         // Rt2
    orr w4, w4, w2, lsl #5          // Rn
```

```
    orr w4, w4, w0                  // Rt1
    mov w0, w4
    ret

// -- kcc_enc_ldp_post ---
// LDP (post-index, 64-bit): x0=rt1, x1=rt2, x2=rn
   , x3=imm7 (signed, scaled by 8)
// Encoding: 0xA8C00000 | (imm7&0x7F)<<15 | rt2
   <<10 | rn<<5 | rt1
.text
.globl _kcc_enc_ldp_post
.p2align 2
_kcc_enc_ldp_post:
    mov w4, #0xA8C00000
    and w5, w3, #0x7F
    orr w4, w4, w5, lsl #15         // imm7
    orr w4, w4, w1, lsl #10         // Rt2
    orr w4, w4, w2, lsl #5          // Rn
    orr w4, w4, w0                  // Rt1
    mov w0, w4
    ret
```

□

529b · defined at 529ab, 530a · used in 30d · defines _kcc_enc_ldp_post 517, _kcc_enc_stp_pre 493

Register Move MOV R_d, R_m is not a first-class ARM64 instruction but an alias for ORR Rd, XZR, Rm—a bitwise OR with the zero register that simply copies Rm into Rd. We encode it as a logical operation with Rn = 31 (the zero register), opcode prefix 0x2A000000, and sf at bit 31.

⟨ *Backend: encode memory* 530a ⟩ + ≡

```
// -- kcc_enc_mov_reg ---
// MOV Rd, Rm → ORR Rd, XZR, Rm
// x0=rd, x1=rm, x2=sf
.text
.globl _kcc_enc_mov_reg
.p2align 2
_kcc_enc_mov_reg:
    mov w3, #0x2A000000
    orr w3, w3, w2, lsl #31         // sf
    orr w3, w3, w1, lsl #16         // Rm
    mov w4, #31
    orr w3, w3, w4, lsl #5          // Rn = XZR
    orr w3, w3, w0                  // Rd
    mov w0, w3
    ret
```

□

530a · defined at 529ab, 530a · used in 30d · defines _kcc_enc_mov_reg 499b, 513

36.7. Floating-Point Encoding

Floating-point work uses a small, fixed slice of the SIMD&FP encoding space, all in double precision (ptype = 01). The two-source data-processing form (FADD, FSUB, FMUL, FDIV) shares a base word and selects the operation with a four-bit opcode in bits [15:12]. Conversions between the integer and floating files are single-source: SCVTF signed-int → double, FCVTZS double → signed-int (round toward zero). FCMP sets the NZCV flags from a double comparison, after which an ordinary CSET materialises the boolean. Finally, doubles move to and from stack slots with the SIMD&FP unsigned-offset LDR/STR (bit 26, the V bit, distinguishes them from the general-register forms).

⟨ *Backend: encode floating point* 530b ⟩ ≡

```
// -- kcc_enc_fp_dp2 ---
// Double-precision 2-source op: FADD/FSUB/FMUL/
   FDIV.
//   x0=Rd, x1=Rn, x2=Rm, x3=opcode (fmul=0,fdiv
   =1,fadd=2,fsub=3)
.text
.globl _kcc_enc_fp_dp2
.p2align 2
_kcc_enc_fp_dp2:
    movz w4, #0x0800              // base: fp dp2,
   double (bits[11:10]=10)
    movk w4, #0x1E60, lsl #16
    orr w4, w4, w3, lsl #12        // opcode
    orr w4, w4, w2, lsl #16        // Rm
    orr w4, w4, w1, lsl #5         // Rn
    orr w4, w4, w0                 // Rd
    mov w0, w4
    ret

// -- kcc_enc_scvtf ---
// SCVTF Dd, Xn - signed 64-bit integer → double.
    x0=Rd(fp), x1=Rn(int)
.text
.globl _kcc_enc_scvtf
.p2align 2
_kcc_enc_scvtf:
    movz w2, #0x9E62, lsl #16
    orr w2, w2, w1, lsl #5         // Rn
    orr w2, w2, w0                 // Rd
    mov w0, w2
    ret

// -- kcc_enc_ucvtf ---
// UCVTF Dd, Xn - UNSIGNED 64-bit integer → double
   .  x0=Rd(fp), x1=Rn(int)
// Identical to SCVTF but for the opcode field
   (011 vs 010): base 0x9E63.
.text
.globl _kcc_enc_ucvtf
.p2align 2
_kcc_enc_ucvtf:
    movz w2, #0x9E63, lsl #16
    orr w2, w2, w1, lsl #5         // Rn
    orr w2, w2, w0                 // Rd
    mov w0, w2
    ret

// -- kcc_enc_fcvtzs ---
// FCVTZS Xd, Dn - double → signed 64-bit integer
   (round toward zero).
//   x0=Rd(int), x1=Rn(fp)
.text
.globl _kcc_enc_fcvtzs
.p2align 2
_kcc_enc_fcvtzs:
    movz w2, #0x9E78, lsl #16
    orr w2, w2, w1, lsl #5         // Rn
    orr w2, w2, w0                 // Rd
    mov w0, w2
    ret

// -- kcc_enc_fcvt_s_d / kcc_enc_fcvt_d_s ---
// FCVT Sd, Dn (double → single, rounds) and FCVT
   Dd, Sn (single → double).
// Rounding a double through single and back is
   how a value is truncated to
// single precision at a conversion to float.  x0=
   Rd, x1=Rn.
```

```
.text
.globl _kcc_enc_fcvt_s_d
.p2align 2
_kcc_enc_fcvt_s_d:
    movz w2, #0x4000
    movk w2, #0x1E62, lsl #16      // FCVT Sd, Dn =
    0x1E624000
    orr w2, w2, w1, lsl #5         // Rn
    orr w2, w2, w0                 // Rd
    mov w0, w2
    ret

.text
.globl _kcc_enc_fcvt_d_s
.p2align 2
_kcc_enc_fcvt_d_s:
    movz w2, #0xC000
    movk w2, #0x1E22, lsl #16      // FCVT Dd, Sn =
    0x1E22C000
    orr w2, w2, w1, lsl #5         // Rn
    orr w2, w2, w0                 // Rd
    mov w0, w2
    ret

// -- kcc_enc_fmov_d_x / kcc_enc_fmov_x_d ---
// Move the raw 64 bits between a general register
    and a double register, so a
// register-allocated value (which lives in a GP
   register) can be read or
// written by the floating-point paths.  x0=Rd, x1
   =Rn.
.text
.globl _kcc_enc_fmov_d_x
.p2align 2
_kcc_enc_fmov_d_x:
    movz w2, #0x0000
    movk w2, #0x9E67, lsl #16      // FMOV Dd, Xn =
    0x9E670000
    orr w2, w2, w1, lsl #5         // Rn (general)
    orr w2, w2, w0                 // Rd (double)
    mov w0, w2
    ret

.text
.globl _kcc_enc_fmov_x_d
.p2align 2
_kcc_enc_fmov_x_d:
    movz w2, #0x0000
    movk w2, #0x9E66, lsl #16      // FMOV Xd, Dn =
    0x9E660000
    orr w2, w2, w1, lsl #5         // Rn (double)
    orr w2, w2, w0                 // Rd (general)
    mov w0, w2
    ret

// -- kcc_enc_fmov_d_d ---
// FMOV Dd, Dn - copy one double register to
   another (register-to-register), so
// a value already allocated to a d-register can
   be moved into a scratch d-reg.
// x0=Rd, x1=Rn.
.text
.globl _kcc_enc_fmov_d_d
.p2align 2
_kcc_enc_fmov_d_d:
    movz w2, #0x4000
    movk w2, #0x1E60, lsl #16      // FMOV Dd, Dn =
    0x1E604000
    orr w2, w2, w1, lsl #5         // Rn (double)
    orr w2, w2, w0                 // Rd (double)
    mov w0, w2
    ret

// -- kcc_enc_fneg_d ---
// FNEG Dd, Dn - negate a double (flips the sign
   bit in the FP unit).  x0=Rd, x1=Rn.
.text
.globl _kcc_enc_fneg_d
.p2align 2
_kcc_enc_fneg_d:
    movz w2, #0x4000
    movk w2, #0x1E61, lsl #16      // FNEG Dd, Dn =
    0x1E614000
    orr w2, w2, w1, lsl #5         // Rn (double)
    orr w2, w2, w0                 // Rd (double)
    mov w0, w2
    ret

// -- kcc_enc_fcmp ---
// FCMP Dn, Dm - set NZCV from a double comparison
   .  x0=Rn, x1=Rm
.text
.globl _kcc_enc_fcmp
.p2align 2
_kcc_enc_fcmp:
    movz w2, #0x2000
    movk w2, #0x1E60, lsl #16
    orr w2, w2, w1, lsl #16        // Rm
    orr w2, w2, w0, lsl #5         // Rn
    mov w0, w2
    ret

// -- kcc_enc_ldr_fp / kcc_enc_str_fp ---
// LDR/STR Dt, [Xn, #imm12*8] - SIMD&FP unsigned
   offset (64-bit double).
//   x0=Rt(fp), x1=Rn, x2=imm12 (already scaled by
    8)
.text
.globl _kcc_enc_ldr_fp
.p2align 2
_kcc_enc_ldr_fp:
    movz w3, #0xFD40, lsl #16
    orr w3, w3, w2, lsl #10        // imm12
    orr w3, w3, w1, lsl #5         // Rn
    orr w3, w3, w0                 // Rt
    mov w0, w3
    ret

.text
.globl _kcc_enc_str_fp
.p2align 2
_kcc_enc_str_fp:
    movz w3, #0xFD00, lsl #16
    orr w3, w3, w2, lsl #10        // imm12
    orr w3, w3, w1, lsl #5         // Rn
    orr w3, w3, w0                 // Rt
    mov w0, w3
    ret

// -- kcc_enc_ldr_fp_s / kcc_enc_str_fp_s ---
// LDR/STR St, [Xn, #imm12*4] - SIMD&FP unsigned
   offset, single precision (32
// bits).  A `float` in memory is a 4-byte IEEE
   -754 single; these move it to and
// from an `s`-register (which aliases the low
   half of the matching `d`), so the
// float paths pair them with FCVT to reach the
   double working format.
//   x0=Rt(fp), x1=Rn, x2=imm12 (already scaled by
    4)
.text
.globl _kcc_enc_ldr_fp_s
.p2align 2
_kcc_enc_ldr_fp_s:
    movz w3, #0xBD40, lsl #16      // LDR St = 0
   xBD400000
    orr w3, w3, w2, lsl #10        // imm12
    orr w3, w3, w1, lsl #5         // Rn
```

```
    orr w3, w3, w0                  // Rt
    mov w0, w3
    ret

.text
.globl _kcc_enc_str_fp_s
.p2align 2
_kcc_enc_str_fp_s:
    movz w3, #0xBD00, lsl #16       // STR St = 0
   xBD000000
    orr w3, w3, w2, lsl #10         // imm12
    orr w3, w3, w1, lsl #5          // Rn
    orr w3, w3, w0                  // Rt
    mov w0, w3
    ret
```

□

530b · used in 30d · defines _kcc_enc_fcmp 511, _kcc_enc_fcvtzs 511, _kcc_enc_fcvt_d_s 511, 513, _kcc_enc_fcvt_s_d 511, 513, _kcc_enc_fp_dp2 511, _kcc_enc_ldr_fp 495, _kcc_enc_ldr_fp_s 513, _kcc_enc_scvtf 511, _kcc_enc_str_fp 495, _kcc_enc_str_fp_s 513, _kcc_enc_ucvtf 511

36.8. Branch Encoding

Unconditional branches (B, BL) encode a 26-bit signed offset in instruction units (divide byte offset by 4). Conditional B.COND uses a 19-bit offset with the condition code in bits [3:0]. CBZ and CBNZ combine a register test with a 19-bit branch offset. RET is hard-coded to branch to the address in x30.

⟨ *Backend: encode branch* 532a ⟩ ≡

```
// -- kcc_enc_b ---
// B imm26: x0 = offset in bytes (will be divided
   by 4)
.text
.globl _kcc_enc_b
.p2align 2
_kcc_enc_b:
    asr x0, x0, #2                  // offset in
   instructions
    mov w1, #0x14000000
    and w0, w0, #0x03FFFFFF
    orr w0, w0, w1
    ret

// -- kcc_enc_bl ---
// BL imm26: x0 = offset in bytes
.text
.globl _kcc_enc_bl
.p2align 2
_kcc_enc_bl:
    asr x0, x0, #2
    mov w1, #0x94000000
    and w0, w0, #0x03FFFFFF
    orr w0, w0, w1
    ret
```

□

532a · defined at 532ab · used in 30d · defines _kcc_enc_b 474, 516, _kcc_enc_bl 518

Conditional Branches and Return Conditional branches use a narrower 19-bit signed offset (still in instruction units). B.COND tests the NZCV flags against a 4-bit condition code in bits [3:0]; the codegen emits a CMP immediately before it. CBZ and CBNZ combine a zero/non-zero register test with the branch, avoiding a separate compare—useful for loop exits and null checks. RET is a fixed encoding (0xD65F03C0) that branches to the address in x30.

⟨ *Backend: encode branch* 532b ⟩ + ≡

```
// -- kcc_enc_b_cond ---
// B.cond imm19: x0 = offset in bytes, x1 =
   condition code
.text
.globl _kcc_enc_b_cond
.p2align 2
_kcc_enc_b_cond:
    asr x0, x0, #2                  // offset in
   instructions
    mov w2, #0x54000000
    and w3, w0, #0x7FFFF            // imm19
    orr w2, w2, w3, lsl #5          // imm19 << 5
    orr w2, w2, w1                  // cond
    mov w0, w2
    ret

// -- kcc_enc_cbz ---
// CBZ Rt, imm19: x0 = rt, x1 = offset in bytes,
   x2 = sf
.text
.globl _kcc_enc_cbz
.p2align 2
_kcc_enc_cbz:
    asr x1, x1, #2                  // offset in
   instructions
    mov w3, #0x34000000
    orr w3, w3, w2, lsl #31         // sf
    and w4, w1, #0x7FFFF
    orr w3, w3, w4, lsl #5          // imm19
    orr w3, w3, w0                  // Rt
    mov w0, w3
    ret

// -- kcc_enc_cbnz ---
// CBNZ Rt, imm19: x0 = rt, x1 = offset in bytes,
   x2 = sf
.text
.globl _kcc_enc_cbnz
.p2align 2
_kcc_enc_cbnz:
    asr x1, x1, #2
    mov w3, #0x35000000
    orr w3, w3, w2, lsl #31         // sf
    and w4, w1, #0x7FFFF
    orr w3, w3, w4, lsl #5          // imm19
    orr w3, w3, w0                  // Rt
    mov w0, w3
    ret

// -- kcc_enc_ret ---
.text
.globl _kcc_enc_ret
.p2align 2
_kcc_enc_ret:
    movz w0, #0xD65F, lsl #16
    movk w0, #0x03C0
    ret
// -- kcc_enc_blr ---
// uint32_t kcc_enc_blr(uint32_t rn)
//   Encodes BLR Xn: 0xD63F0000 | (rn << 5)
.text
.globl _kcc_enc_blr
.p2align 2
_kcc_enc_blr:
    movz w1, #0xD63F, lsl #16
```

```
    movk w1, #0x0000
    lsl w0, w0, #5
    orr w0, w1, w0
    ret
```

□

532b · defined at 532ab · used in 30d ·
defines _kcc_enc_blr 518, _kcc_enc_b_cond never used,
_kcc_enc_cbnz 516, _kcc_enc_cbz never used, _kcc_enc_ret 517

36.9. System Instructions

Two fixed-encoding system instructions: NOP (0xD503201F) for alignment or placeholder purposes, and BRK for software breakpoints (the 16-bit immediate identifies the breakpoint to the debugger).

⟨ *Backend: encode system* 533a ⟩ ≡

```
// -- kcc_enc_nop ---
.text
.globl _kcc_enc_nop
.p2align 2
_kcc_enc_nop:
    movz w0, #0xD503, lsl #16
    movk w0, #0x201F
    ret

// -- kcc_enc_brk ---
// BRK #imm16: x0 = imm16
.text
.globl _kcc_enc_brk
.p2align 2
_kcc_enc_brk:
    mov w1, #0xD4200000
    orr w0, w1, w0, lsl #5
    ret
```

□

533a · used in 30d · defines _kcc_enc_brk never used,
_kcc_enc_nop 504, 522b

36.10. ADRP and Page-Offset Encoding

The ADRP instruction loads a PC-relative page address into a register. The 21-bit immediate is split: bits [1:0] go to immlo at instruction bits [30:29], and bits [20:2] to immhi at bits [23:5]. The linker fills in the actual page offset via an ARM64_RELOC_PAGE21 relocation; we emit zero as a placeholder.

⟨ *Backend: encode adrp* 533b ⟩ ≡

```
// -- kcc_enc_adrp ---
// ADRP Rd, imm21: x0=rd, x1=imm21 (page offset in
    4KB pages)
// Encoding: 1|immlo(2)|10000|immhi(19)|Rd(5)
// immlo = imm21[1:0], immhi = imm21[20:2]
.text
.globl _kcc_enc_adrp
.p2align 2
_kcc_enc_adrp:
    mov w2, #0x90000000
    and w3, w1, #0x3                // immlo =
   imm21[1:0]
    orr w2, w2, w3, lsl #29         // immlo at
   bits [30:29]
    lsr w4, w1, #2                  // immhi =
   imm21[20:2]
    and w4, w4, #0x7FFFF            // mask to 19
   bits
    orr w2, w2, w4, lsl #5          // immhi at
   bits [23:5]
    orr w2, w2, w0                  // Rd
    mov w0, w2
    ret
```

□

533b · used in 30d · defines _kcc_enc_adrp 477b, 521

36.11. Code Buffer Data

Three words of mutable state track the code buffer: a pointer to the arena-allocated memory, the current write position, and the capacity.

⟨ *Backend: emit data* 533c ⟩ ≡

```
// -- Code buffer state ---
.data
.p2align 3
.globl _kcc_code_buf
_kcc_code_buf:      .quad 0     // pointer to
   code buffer
.globl _kcc_code_pos
_kcc_code_pos:      .quad 0     // current write
   position (bytes)
.globl _kcc_code_cap
_kcc_code_cap:      .quad 0     // buffer
   capacity (bytes)
```

□

533c · used in 30d · defines _kcc_code_buf 474, 478, 525b,
526a, 537, 586, _kcc_code_cap 478, 525b, _kcc_code_pos 478,
525b, 526a, 537, 577b, 580, 588, 596

37. Textual Assembly Emission

The direct encoder of Chapter 36 is fast and self-contained, but it can only emit the fixed instruction set kcc itself selects: it is not a general assembler, so it cannot turn an arbitrary `asm("...")` template into machine code. The retargeting playbook (Chapter 2) describes the alternative—emit textual `.s` and invoke the platform assembler—and this chapter implements it for .

The renderer works *from the already-generated code buffer*, so the entire front end, optimiser, register allocator, and instruction selector are reused unchanged; only the final packaging differs. For each 32-bit word the code generator produced, the renderer emits one of two things. If a relocation covers that word—a call or a data-address reference whose target address the linker chooses—it emits the *symbolic* form (`bl _f`, `adrp x0, _g@PAGE`, …) so the assembler re-creates the relocation. Otherwise it emits `.long 0x`*word*: the assembler reproduces the exact bytes, and because internal branches are PC-relative and the instruction sequence is unchanged, their distances are preserved. This makes the renderer correct by construction even where it has no pretty-printer for an opcode.

37.1. The Output Buffer

Assembly text is accumulated in a growable arena buffer and written out in one `write` at the end. The helpers below append a byte, a C string, a decimal number, and an eight-digit hexadecimal word.

⟨ *Emit-asm: data* 535a ⟩ ≡

```
.data
.p2align 3
_kcc_as_buf:    .quad 0         // growable text
   buffer
_kcc_as_pos:    .quad 0         // bytes used
_kcc_as_cap:    .quad 0         // capacity
_kcc_as_fd:     .quad 0         // open .s file
   descriptor
// Inline-asm snippets: an array of {code_offset,
   template_ptr, template_len}
// (24 bytes each) recorded by codegen at each
   IR_ASM, spliced by the renderer.
.globl _kcc_asm_snips
_kcc_asm_snips:       .quad 0
.globl _kcc_asm_snip_count
_kcc_asm_snip_count: .quad 0
.globl _kcc_asm_snip_cap
_kcc_asm_snip_cap:    .quad 0
// Per-word byte flags: 1 = a branch target that
   needs a synthetic label.
.globl _kcc_as_targets
_kcc_as_targets:      .quad 0
.section __TEXT,__cstring,cstring_literals
_kcc_as_s_sect:    .asciz "\t.section __TEXT,__text
   ,regular\n"
_kcc_as_s_p2a:     .asciz "\t.p2align 2\n"
_kcc_as_s_globl:   .asciz "\t.globl "
_kcc_as_s_long:    .asciz "\t.long 0x"
_kcc_as_s_byte:    .asciz "\t.byte 0x"
_kcc_as_s_bl:      .asciz "\tbl\t"
_kcc_as_s_adrp:    .asciz "\tadrp\t"
_kcc_as_s_add:     .asciz "\tadd\t"
_kcc_as_s_ldr:     .asciz "\tldr\t"
_kcc_as_s_page:    .asciz "@PAGE"
_kcc_as_s_pageoff:.asciz "@PAGEOFF"
_kcc_as_s_gotpage:.asciz "@GOTPAGE"
_kcc_as_s_gotoff: .asciz "@GOTPAGEOFF"
_kcc_as_s_hexdig: .asciz "0123456789abcdef"
_kcc_as_s_bdot:    .asciz "\tb."
_kcc_as_s_b:       .asciz "\tb\t"
_kcc_as_s_cbz:     .asciz "\tcbz\t"
_kcc_as_s_cbnz:    .asciz "\tcbnz\t"
_kcc_as_s_lc:      .asciz "Lc"
_kcc_as_s_zerofill:.asciz "\t.zerofill __DATA,
   __common,"
_kcc_as_s_a3nl:    .asciz ",3\n"
_kcc_as_s_modsect:.asciz "\t.section __DATA,
   __mod_init_func,mod_init_funcs\n\t.p2align 3\n\
   t.quad "
// Condition names indexed by cond*3 (two chars +
   NUL each).
_kcc_as_s_conds:   .ascii "eq\0ne\0hs\0lo\0mi\0pl\0
   vs\0vc\0hi\0ls\0ge\0lt\0gt\0le\0al\0nv\0"
.globl _kcc_as_s_env
_kcc_as_s_env:     .asciz "KCC_EMIT_ASM"
.text
```

□

535a · used in 30d

⟨ *Emit-asm: buffer* 535b ⟩ ≡

```
// -- kcc_as_init ---
// Allocate the text buffer (64 KiB, grows on
   demand).
.globl _kcc_as_init
.p2align 2
_kcc_as_init:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    mov x0, #65536
    bl _kcc_arena_alloc
    adrp x1, _kcc_as_buf@PAGE
    add  x1, x1, _kcc_as_buf@PAGEOFF
    str x0, [x1]
    adrp x1, _kcc_as_pos@PAGE
    add  x1, x1, _kcc_as_pos@PAGEOFF
    str xzr, [x1]
    adrp x1, _kcc_as_cap@PAGE
    add  x1, x1, _kcc_as_cap@PAGEOFF
    mov x2, #65536
    str x2, [x1]
    ldp x29, x30, [sp], #16
    ret

// -- kcc_as_ch ---
// void kcc_as_ch(uint32_t byte) - append one byte
   , growing the buffer if full.
.globl _kcc_as_ch
.p2align 2
_kcc_as_ch:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov w19, w0                     // byte
    adrp x0, _kcc_as_pos@PAGE
    add  x0, x0, _kcc_as_pos@PAGEOFF
    ldr x1, [x0]                    // pos
    adrp x2, _kcc_as_cap@PAGE
```

```
    add  x2, x2, _kcc_as_cap@PAGEOFF
    ldr x3, [x2]                    // cap
    cmp x1, x3
    b.lo .Lasch_write
    // Grow: double, copy.
    lsl x4, x3, #1
    str x4, [x2]
    mov x0, x4
    bl _kcc_arena_alloc             // new buffer
    mov x5, x0
    adrp x6, _kcc_as_buf@PAGE
    add  x6, x6, _kcc_as_buf@PAGEOFF
    ldr x7, [x6]                    // old buffer
    str x5, [x6]
    mov x0, x5
    mov x1, x7
    adrp x2, _kcc_as_pos@PAGE
    add  x2, x2, _kcc_as_pos@PAGEOFF
    ldr x2, [x2]
    bl _kcc_memcpy
.Lasch_write:
    adrp x0, _kcc_as_buf@PAGE
    add  x0, x0, _kcc_as_buf@PAGEOFF
    ldr x0, [x0]                    // buffer
    adrp x2, _kcc_as_pos@PAGE
    add  x2, x2, _kcc_as_pos@PAGEOFF
    ldr x3, [x2]                    // pos
    strb w19, [x0, x3]
    add x3, x3, #1
    str x3, [x2]
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_as_str ---
// void kcc_as_str(const char *s) - append a NUL-
   terminated string.
.globl _kcc_as_str
.p2align 2
_kcc_as_str:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    str x19, [sp, #-16]!            // (scratch save
    below main frame not needed)
    mov x19, x0
.Lasstr_loop:
    ldrb w0, [x19], #1
    cbz w0, .Lasstr_done
    bl _kcc_as_ch
    b .Lasstr_loop
.Lasstr_done:
    ldr x19, [sp], #16
    ldp x29, x30, [sp], #16
    ret

// -- kcc_as_putn ---
// void kcc_as_putn(const char *p, uint64_t n) -
   append n raw bytes.
.globl _kcc_as_putn
.p2align 2
_kcc_as_putn:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0
    mov x20, x1
.Lputn_loop:
    cbz x20, .Lputn_done
    ldrb w0, [x19], #1
    bl _kcc_as_ch
    sub x20, x20, #1
    b .Lputn_loop
.Lputn_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_as_dec ---
// void kcc_as_dec(uint64_t n) - append decimal
   digits.
.globl _kcc_as_dec
.p2align 2
_kcc_as_dec:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0
    add x20, sp, #72                // one past the
   scratch digit area [sp,#48..71]
    add x21, sp, #72                // end marker (
   fixed)
    cbnz x19, .Lasdec_loop
    mov w0, #'0'
    bl _kcc_as_ch
    b .Lasdec_done
.Lasdec_loop:
    cbz x19, .Lasdec_emit
    mov x2, #10
    udiv x3, x19, x2
    msub x4, x3, x2, x19            // x4 = n % 10
    add w4, w4, #'0'
    strb w4, [x20, #-1]!
    mov x19, x3
    b .Lasdec_loop
.Lasdec_emit:
    // digits from x20 (start) up to x21 (end)
.Lasdec_e2:
    cmp x20, x21
    b.hs .Lasdec_done
    ldrb w0, [x20], #1
    bl _kcc_as_ch
    b .Lasdec_e2
.Lasdec_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret

// -- kcc_as_hex8 ---
// void kcc_as_hex8(uint32_t w) - append exactly 8
    hex digits (no 0x prefix).
.globl _kcc_as_hex8
.p2align 2
_kcc_as_hex8:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov w19, w0
    mov w20, #28                    // shift for the
    top nibble
.Lashex_loop:
    lsr w0, w19, w20
    and w0, w0, #0xf
    adrp x1, _kcc_as_s_hexdig@PAGE
    add  x1, x1, _kcc_as_s_hexdig@PAGEOFF
    ldrb w0, [x1, x0]
    bl _kcc_as_ch
    subs w20, w20, #4
    b.ge .Lashex_loop
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_as_hex2 ---
// void kcc_as_hex2(uint32_t b) - append exactly 2
    hex digits (low byte of b).
.globl _kcc_as_hex2
```

```
.p2align 2
_kcc_as_hex2:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    and w19, w0, #0xff
    lsr w0, w19, #4
    adrp x1, _kcc_as_s_hexdig@PAGE
    add  x1, x1, _kcc_as_s_hexdig@PAGEOFF
    ldrb w0, [x1, x0]
    bl _kcc_as_ch
    and w0, w19, #0xf
    adrp x1, _kcc_as_s_hexdig@PAGE
    add  x1, x1, _kcc_as_s_hexdig@PAGEOFF
    ldrb w0, [x1, x0]
    bl _kcc_as_ch
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_as_xreg ---
// void kcc_as_xreg(uint32_t n) - append an x-
   register name (31 → xzr).
.globl _kcc_as_xreg
.p2align 2
_kcc_as_xreg:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    str x19, [sp, #-16]!
    mov w19, w0
    cmp w19, #31
    b.eq .Lasxr_zr
    mov w0, #'x'
    bl _kcc_as_ch
    mov x0, x19
    bl _kcc_as_dec
    b .Lasxr_done
.Lasxr_zr:
    mov w0, #'x'
    bl _kcc_as_ch
    mov w0, #'z'
    bl _kcc_as_ch
    mov w0, #'r'
    bl _kcc_as_ch
.Lasxr_done:
    ldr x19, [sp], #16
    ldp x29, x30, [sp], #16
    ret
```

□

535b · used in 30d · uses _kcc_arena_alloc 58a

37.2. Rendering the Text Section

The renderer walks the code buffer one instruction at a time. Before each word it emits any symbol *label* defined at that offset; then, if a relocation covers the word, the symbolic form, else the raw `.long`. A small helper appends a symbol's name given its index in the linker's symbol vector.

⟨ *Emit-asm: render* 537 ⟩ ≡

```
// -- kcc_as_symname ---
// void kcc_as_symname(uint64_t idx) - append
   symbol[idx].name.
.globl _kcc_as_symname
.p2align 2
_kcc_as_symname:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    adrp x1, _kcc_linker_syms@PAGE
    add  x1, x1, _kcc_linker_syms@PAGEOFF
    ldr x1, [x1]                    // syms vec
    ldr x1, [x1, #VEC_DATA]         // entry array
    mov x2, #SYM_ENTRY_SIZE
    madd x1, x0, x2, x1             // &syms[idx]
    ldr x0, [x1, #SYM_NAME]         // name ptr
    bl _kcc_as_str
    ldp x29, x30, [sp], #16
    ret

// -- kcc_as_reloc_at ---
// uint64_t kcc_as_reloc_at(uint64_t off)
//   Returns pointer to the reloc entry whose
   RELOC_OFFSET == off, or 0.
.globl _kcc_as_reloc_at
.p2align 2
_kcc_as_reloc_at:
    adrp x1, _kcc_linker_relocs@PAGE
    add  x1, x1, _kcc_linker_relocs@PAGEOFF
    ldr x1, [x1]
    ldr x2, [x1, #VEC_COUNT]
    ldr x3, [x1, #VEC_DATA]
    mov x4, #0
.Lasra_loop:
    cmp x4, x2
    b.hs .Lasra_none
    mov x5, #RELOC_SIZE
    madd x6, x4, x5, x3             // &reloc[x4]
    ldr x7, [x6, #RELOC_OFFSET]
    cmp x7, x0
    b.eq .Lasra_hit
    add x4, x4, #1
    b .Lasra_loop
.Lasra_hit:
    mov x0, x6
    ret
.Lasra_none:
    mov x0, #0
    ret

// -- kcc_as_add_snip ---
// void kcc_as_add_snip(uint64_t off, const char *
   tmpl, uint64_t len)
//   Record an inline-asm snippet at code offset `
   off`.  Called by codegen.
.globl _kcc_as_add_snip
.p2align 2
_kcc_as_add_snip:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // off
    mov x20, x1                     // tmpl
    mov x21, x2                     // len
    // Ensure capacity (lazily allocate 16 entries
   , grow ×2).
    adrp x0, _kcc_asm_snip_cap@PAGE
    add  x0, x0, _kcc_asm_snip_cap@PAGEOFF
    ldr x22, [x0]                   // cap
    adrp x0, _kcc_asm_snip_count@PAGE
    add  x0, x0, _kcc_asm_snip_count@PAGEOFF
    ldr x2, [x0]                    // count
    cmp x2, x22
    b.lo .Lsnip_store
    // grow
    mov x0, #16
    cmp x22, #0
    csel x22, x0, x22, eq
    lsl x22, x22, #1                // new cap
    mov x0, x22
    mov x1, #24
    mul x0, x0, x1
    bl _kcc_arena_alloc
    mov x3, x0                      // new array
```

```
    adrp x0, _kcc_asm_snips@PAGE
    add  x0, x0, _kcc_asm_snips@PAGEOFF
    ldr x4, [x0]                    // old array
    str x3, [x0]
    adrp x0, _kcc_asm_snip_cap@PAGE
    add  x0, x0, _kcc_asm_snip_cap@PAGEOFF
    str x22, [x0]
    // copy old entries (count*24)
    cbz x4, .Lsnip_store
    adrp x0, _kcc_asm_snip_count@PAGE
    add  x0, x0, _kcc_asm_snip_count@PAGEOFF
    ldr x2, [x0]
    mov x1, #24
    mul x2, x2, x1
    mov x0, x3
    mov x1, x4
    bl _kcc_memcpy
.Lsnip_store:
    adrp x0, _kcc_asm_snips@PAGE
    add  x0, x0, _kcc_asm_snips@PAGEOFF
    ldr x0, [x0]                    // array
    adrp x1, _kcc_asm_snip_count@PAGE
    add  x1, x1, _kcc_asm_snip_count@PAGEOFF
    ldr x2, [x1]                    // count
    mov x3, #24
    madd x0, x2, x3, x0             // &array[count]
    str x19, [x0, #0]               // offset
    str x20, [x0, #8]               // tmpl
    str x21, [x0, #16]              // len
    add x2, x2, #1
    str x2, [x1]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_as_snip_at ---
// uint64_t kcc_as_snip_at(uint64_t off) - pointer
    to the snippet at `off`, or 0.
.globl _kcc_as_snip_at
.p2align 2
_kcc_as_snip_at:
    adrp x1, _kcc_asm_snips@PAGE
    add  x1, x1, _kcc_asm_snips@PAGEOFF
    ldr x1, [x1]
    cbz x1, .Lsnat_none
    adrp x2, _kcc_asm_snip_count@PAGE
    add  x2, x2, _kcc_asm_snip_count@PAGEOFF
    ldr x2, [x2]
    mov x3, #0
.Lsnat_loop:
    cmp x3, x2
    b.hs .Lsnat_none
    mov x4, #24
    madd x5, x3, x4, x1
    ldr x6, [x5, #0]
    cmp x6, x0
    b.eq .Lsnat_hit
    add x3, x3, #1
    b .Lsnat_loop
.Lsnat_hit:
    mov x0, x5
    ret
.Lsnat_none:
    mov x0, #0
    ret

// -- kcc_as_labels_at ---
// void kcc_as_labels_at(uint64_t off) - emit `
   name:` for every text symbol
// defined at this offset (and `.globl name` first
    if it is external).
.globl _kcc_as_labels_at
.p2align 2
_kcc_as_labels_at:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // target offset
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    ldr x20, [x0, #VEC_COUNT]
    ldr x21, [x0, #VEC_DATA]
    mov x22, #0
.Lasla_loop:
    cmp x22, x20
    b.hs .Lasla_done
    mov x0, #SYM_ENTRY_SIZE
    madd x0, x22, x0, x21           // &sym[x22]
    ldr x1, [x0, #SYM_SECTION]
    cmp x1, #1                      // text section?
    b.ne .Lasla_next
    ldr x1, [x0, #SYM_VALUE]
    cmp x1, x19
    b.ne .Lasla_next
    // Emit `.globl name` if external.  Registers
   x21 (entry array) and x22
    // (index) are callee-saved and survive the
   calls, so the symbol pointer is
    // recomputed by madd rather than stashed on
   the stack.
    ldr x2, [x0, #SYM_FLAGS]
    tst x2, #N_EXT
    b.eq .Lasla_label
    adrp x0, _kcc_as_s_globl@PAGE
    add  x0, x0, _kcc_as_s_globl@PAGEOFF
    bl _kcc_as_str
    mov x0, #SYM_ENTRY_SIZE
    madd x0, x22, x0, x21
    ldr x0, [x0, #SYM_NAME]
    bl _kcc_as_str
    mov w0, #'\n'
    bl _kcc_as_ch
.Lasla_label:
    mov x0, #SYM_ENTRY_SIZE
    madd x0, x22, x0, x21
    ldr x0, [x0, #SYM_NAME]
    bl _kcc_as_str
    mov w0, #':'
    bl _kcc_as_ch
    mov w0, #'\n'
    bl _kcc_as_ch
.Lasla_next:
    add x22, x22, #1
    b .Lasla_loop
.Lasla_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_as_reloc_line ---
// void kcc_as_reloc_line(uint64_t reloc, uint32_t
    word)
//   Emit the symbolic assembly for a relocated
   instruction.
.globl _kcc_as_reloc_line
.p2align 2
_kcc_as_reloc_line:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                     // reloc entry
    mov w20, w1                     // word
    ldr x21, [x19, #RELOC_TYPE]
```

```
    and w0, w20, #0x1f              // Rd/Rt = bits
   [4:0]
    lsr w1, w20, #5
    and w1, w1, #0x1f               // Rn = bits
   [9:5]
    // Dispatch on reloc type.
    cmp x21, #ARM64_RELOC_BRANCH26
    b.eq .Lasrl_branch
    cmp x21, #ARM64_RELOC_PAGE21
    b.eq .Lasrl_adrp
    cmp x21, #ARM64_RELOC_GOT_LOAD_PAGE21
    b.eq .Lasrl_gotadrp
    cmp x21, #ARM64_RELOC_PAGEOFF12
    b.eq .Lasrl_pageoff
    cmp x21, #ARM64_RELOC_GOT_LOAD_PAGEOFF12
    b.eq .Lasrl_gotoff
    // Unknown reloc type: fall back to a raw word
    (loses the reloc, but at
    // least assembles); should not happen for the
    types kcc emits.
    b .Lasrl_done

.Lasrl_branch:
    adrp x0, _kcc_as_s_bl@PAGE
    add  x0, x0, _kcc_as_s_bl@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #RELOC_SYM]
    bl _kcc_as_symname
    b .Lasrl_nl

.Lasrl_adrp:
    adrp x0, _kcc_as_s_adrp@PAGE
    add  x0, x0, _kcc_as_s_adrp@PAGEOFF
    bl _kcc_as_str
    and w0, w20, #0x1f
    bl _kcc_as_xreg
    mov w0, #','
    bl _kcc_as_ch
    mov w0, #' '
    bl _kcc_as_ch
    ldr x0, [x19, #RELOC_SYM]
    bl _kcc_as_symname
    adrp x0, _kcc_as_s_page@PAGE
    add  x0, x0, _kcc_as_s_page@PAGEOFF
    bl _kcc_as_str
    b .Lasrl_nl

.Lasrl_gotadrp:
    adrp x0, _kcc_as_s_adrp@PAGE
    add  x0, x0, _kcc_as_s_adrp@PAGEOFF
    bl _kcc_as_str
    and w0, w20, #0x1f
    bl _kcc_as_xreg
    mov w0, #','
    bl _kcc_as_ch
    mov w0, #' '
    bl _kcc_as_ch
    ldr x0, [x19, #RELOC_SYM]
    bl _kcc_as_symname
    adrp x0, _kcc_as_s_gotpage@PAGE
    add  x0, x0, _kcc_as_s_gotpage@PAGEOFF
    bl _kcc_as_str
    b .Lasrl_nl

.Lasrl_pageoff:
    // add xD, xN, sym@PAGEOFF   (kcc materialises
    an address with add)
    adrp x0, _kcc_as_s_add@PAGE
    add  x0, x0, _kcc_as_s_add@PAGEOFF
    bl _kcc_as_str
    and w0, w20, #0x1f
    bl _kcc_as_xreg
    mov w0, #','
    bl _kcc_as_ch
    mov w0, #' '
    bl _kcc_as_ch
    lsr w0, w20, #5
    and w0, w0, #0x1f
    bl _kcc_as_xreg
    mov w0, #','
    bl _kcc_as_ch
    mov w0, #' '
    bl _kcc_as_ch
    ldr x0, [x19, #RELOC_SYM]
    bl _kcc_as_symname
    adrp x0, _kcc_as_s_pageoff@PAGE
    add  x0, x0, _kcc_as_s_pageoff@PAGEOFF
    bl _kcc_as_str
    b .Lasrl_nl

.Lasrl_gotoff:
    // ldr xD, [xN, sym@GOTPAGEOFF]
    adrp x0, _kcc_as_s_ldr@PAGE
    add  x0, x0, _kcc_as_s_ldr@PAGEOFF
    bl _kcc_as_str
    and w0, w20, #0x1f
    bl _kcc_as_xreg
    mov w0, #','
    bl _kcc_as_ch
    mov w0, #' '
    bl _kcc_as_ch
    mov w0, #'['
    bl _kcc_as_ch
    lsr w0, w20, #5
    and w0, w0, #0x1f
    bl _kcc_as_xreg
    mov w0, #','
    bl _kcc_as_ch
    mov w0, #' '
    bl _kcc_as_ch
    ldr x0, [x19, #RELOC_SYM]
    bl _kcc_as_symname
    adrp x0, _kcc_as_s_gotoff@PAGE
    add  x0, x0, _kcc_as_s_gotoff@PAGEOFF
    bl _kcc_as_str
    mov w0, #']'
    bl _kcc_as_ch
.Lasrl_nl:
    mov w0, #'\n'
    bl _kcc_as_ch
.Lasrl_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_as_wreg ---
// void kcc_as_wreg(uint32_t n) - append a w-
   register name (31 → wzr).
.globl _kcc_as_wreg
.p2align 2
_kcc_as_wreg:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov w19, w0
    mov w0, #'w'
    bl _kcc_as_ch
    cmp w19, #31
    b.ne .Laswr_num
    mov w0, #'z'
    bl _kcc_as_ch
    mov w0, #'r'
    bl _kcc_as_ch
    b .Laswr_done
.Laswr_num:
    mov x0, x19
    bl _kcc_as_dec
.Laswr_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

```
// -- kcc_as_sym_at ---
// uint64_t kcc_as_sym_at(uint64_t off) - name
   pointer of a text symbol defined
// at `off`, or 0.  Used to reference a branch/
   call target by its real name.
.globl _kcc_as_sym_at
.p2align 2
_kcc_as_sym_at:
    adrp x1, _kcc_linker_syms@PAGE
    add  x1, x1, _kcc_linker_syms@PAGEOFF
    ldr x1, [x1]
    ldr x2, [x1, #VEC_COUNT]
    ldr x3, [x1, #VEC_DATA]
    mov x4, #0
.Lsyat_loop:
    cmp x4, x2
    b.hs .Lsyat_none
    mov x5, #SYM_ENTRY_SIZE
    madd x6, x4, x5, x3
    ldr x7, [x6, #SYM_SECTION]
    cmp x7, #1
    b.ne .Lsyat_next
    ldr x7, [x6, #SYM_VALUE]
    cmp x7, x0
    b.ne .Lsyat_next
    ldr x0, [x6, #SYM_NAME]
    ret
.Lsyat_next:
    add x4, x4, #1
    b .Lsyat_loop
.Lsyat_none:
    mov x0, #0
    ret

// -- kcc_as_branch_kind ---
// uint64_t kcc_as_branch_kind(uint32_t word)
//   0 = not a branch, 1 = b, 2 = bl, 3 = b.cond,
   4 = cbz, 5 = cbnz.
.globl _kcc_as_branch_kind
.p2align 2
_kcc_as_branch_kind:
    lsr w1, w0, #26
    cmp w1, #0x05                   // b
    b.eq .Lbk_b
    cmp w1, #0x25                   // bl
    b.eq .Lbk_bl
    lsr w1, w0, #24
    and w2, w1, #0xff
    cmp w2, #0x54                   // b.cond (bits
   [31:24]=0x54, bit4=0)
    b.ne .Lbk_cb
    tst w0, #0x10
    b.ne .Lbk_none
    mov x0, #3
    ret
.Lbk_cb:
    and w2, w1, #0x7f
    cmp w2, #0x34                   // cbz
    b.eq .Lbk_cbz
    cmp w2, #0x35                   // cbnz
    b.eq .Lbk_cbnz
.Lbk_none:
    mov x0, #0
    ret
.Lbk_b:
    mov x0, #1
    ret
.Lbk_bl:
    mov x0, #2
    ret
.Lbk_cbz:
    mov x0, #4
    ret
.Lbk_cbnz:
    mov x0, #5
    ret

// -- kcc_as_branch_target ---
// uint64_t kcc_as_branch_target(uint32_t word,
   uint64_t off)
//   Byte offset the branch targets, or (uint64_t)
   -1 if not a branch.
.globl _kcc_as_branch_target
.p2align 2
_kcc_as_branch_target:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov w19, w0                     // word
    mov x20, x1                     // off
    bl _kcc_as_branch_kind          // (x0=word
   already) → kind
    cbz x0, .Lbt_none
    cmp x0, #2
    b.gt .Lbt_imm19                 // kinds 3,4,5
   use imm19; 1,2 use imm26
    and w1, w19, #0x03FFFFFF        // imm26
    lsl w1, w1, #6                 // sign-extend
   from bit 25
    asr w1, w1, #6
    b .Lbt_compute
.Lbt_imm19:
    lsr w1, w19, #5
    and w1, w1, #0x7FFFF            // imm19
    lsl w1, w1, #13                // sign-extend
   from bit 18
    asr w1, w1, #13
.Lbt_compute:
    sxtw x1, w1
    lsl x1, x1, #2                 // instructions
   → bytes
    add x0, x20, x1
    b .Lbt_done
.Lbt_none:
    mov x0, #-1
.Lbt_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_as_label_ref ---
// void kcc_as_label_ref(uint64_t target) - append
   the target's symbol name if
// one is defined there, else a synthetic `Lc<
   target>` label.
.globl _kcc_as_label_ref
.p2align 2
_kcc_as_label_ref:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0
    bl _kcc_as_sym_at               // (x0=target)
   → name or 0
    cbz x0, .Llr_synth
    bl _kcc_as_str
    b .Llr_done
.Llr_synth:
    adrp x0, _kcc_as_s_lc@PAGE
    add  x0, x0, _kcc_as_s_lc@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_as_dec
.Llr_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

```
// -- kcc_as_render_branch ---
// void kcc_as_render_branch(uint32_t word,
   uint64_t off) - emit the symbolic
// form of an internal (non-relocated) branch, so
   `as` recomputes its distance.
.globl _kcc_as_render_branch
.p2align 2
_kcc_as_render_branch:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov w19, w0                     // word
    mov x20, x1                     // off
    bl _kcc_as_branch_kind          // (x0=word) →
   kind
    mov x21, x0
    mov w0, w19
    mov x1, x20
    bl _kcc_as_branch_target
    mov x22, x0                     // target
    cmp x21, #1
    b.eq .Lrb_b
    cmp x21, #2
    b.eq .Lrb_bl
    cmp x21, #3
    b.eq .Lrb_bcond
    cmp x21, #4
    b.eq .Lrb_cbz
    b .Lrb_cbnz
.Lrb_b:
    adrp x0, _kcc_as_s_b@PAGE
    add  x0, x0, _kcc_as_s_b@PAGEOFF
    bl _kcc_as_str
    mov x0, x22
    bl _kcc_as_label_ref
    b .Lrb_nl
.Lrb_bl:
    adrp x0, _kcc_as_s_bl@PAGE
    add  x0, x0, _kcc_as_s_bl@PAGEOFF
    bl _kcc_as_str
    mov x0, x22
    bl _kcc_as_label_ref
    b .Lrb_nl
.Lrb_bcond:
    adrp x0, _kcc_as_s_bdot@PAGE
    add  x0, x0, _kcc_as_s_bdot@PAGEOFF
    bl _kcc_as_str
    and w0, w19, #0xf               // cond
    mov w1, #3
    mul w0, w0, w1
    adrp x1, _kcc_as_s_conds@PAGE
    add  x1, x1, _kcc_as_s_conds@PAGEOFF
    add x0, x1, x0
    bl _kcc_as_str
    mov w0, #'\t'
    bl _kcc_as_ch
    mov x0, x22
    bl _kcc_as_label_ref
    b .Lrb_nl
.Lrb_cbz:
    adrp x0, _kcc_as_s_cbz@PAGE
    add  x0, x0, _kcc_as_s_cbz@PAGEOFF
    bl _kcc_as_str
    b .Lrb_cbcommon
.Lrb_cbnz:
    adrp x0, _kcc_as_s_cbnz@PAGE
    add  x0, x0, _kcc_as_s_cbnz@PAGEOFF
    bl _kcc_as_str
.Lrb_cbcommon:
    // register: x if sf(bit31)=1 else w
    and w0, w19, #0x1f
    tst w19, #0x80000000
    b.eq .Lrb_cbw
    bl _kcc_as_xreg
    b .Lrb_cbsep
.Lrb_cbw:
    bl _kcc_as_wreg
.Lrb_cbsep:
    mov w0, #','
    bl _kcc_as_ch
    mov w0, #' '
    bl _kcc_as_ch
    mov x0, x22
    bl _kcc_as_label_ref
.Lrb_nl:
    mov w0, #'\n'
    bl _kcc_as_ch
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_as_mark_targets ---
// void kcc_as_mark_targets(uint64_t code_size) -
   allocate the per-word target
// flag array and mark the target of every
   internal (non-relocated) branch that
// lands where no symbol is defined, so the
   renderer can emit a synthetic label.
.globl _kcc_as_mark_targets
.p2align 2
_kcc_as_mark_targets:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                     // code_size
    lsr x20, x19, #2                // number of
   words
    add x0, x20, #1
    bl _kcc_arena_alloc
    mov x21, x0                     // flag array
    adrp x1, _kcc_as_targets@PAGE
    add  x1, x1, _kcc_as_targets@PAGEOFF
    str x21, [x1]
    // zero it
    mov x2, #0
.Lmt_zero:
    cmp x2, x20
    b.hs .Lmt_scan
    strb wzr, [x21, x2]
    add x2, x2, #1
    b .Lmt_zero
.Lmt_scan:
    mov x22, #0                     // offset
.Lmt_loop:
    cmp x22, x19
    b.hs .Lmt_done
    // Skip relocated words (calls/data refs go
   through the reloc path).
    mov x0, x22
    bl _kcc_as_reloc_at
    cbnz x0, .Lmt_next
    adrp x0, _kcc_code_buf@PAGE
    add  x0, x0, _kcc_code_buf@PAGEOFF
    ldr x0, [x0]
    ldr w0, [x0, x22]               // word
    mov w23, w0
    bl _kcc_as_branch_kind
    cbz x0, .Lmt_next
    mov w0, w23
    mov x1, x22
    bl _kcc_as_branch_target        // x0 = target
    cmn x0, #1
    b.eq .Lmt_next                  // -1
    mov x24, x0                     // save target
    bl _kcc_as_sym_at               // (x0=target)
   → name or 0
```

```
    cbnz x0, .Lmt_next          // has a real
   symbol → no synthetic label
    // mark flag[target/4] = 1
    adrp x0, _kcc_as_targets@PAGE
    add  x0, x0, _kcc_as_targets@PAGEOFF
    ldr x0, [x0]
    lsr x1, x24, #2
    mov w2, #1
    strb w2, [x0, x1]
.Lmt_next:
    add x22, x22, #4
    b .Lmt_loop
.Lmt_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_as_render_text ---
// void kcc_as_render_text(void) - emit the __text
    section.  The code buffer is
// [ instructions | string-literal bytes ]:
   emit_strings appended the cstring
// region and advanced code_pos, so the
   instructions occupy the first
// (code_pos - cstr_pos) bytes and the string data
    the rest.  Instructions are
// rendered four bytes at a time (symbolic if
   relocated, else `.long`); the
// string region is rendered byte at a time as `.
   byte`, so a literal whose length
// is not a multiple of four, or a label that
   lands mid-word, is still exact.
.globl _kcc_as_render_text
.p2align 2
_kcc_as_render_text:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    // Section header + alignment.
    adrp x0, _kcc_as_s_sect@PAGE
    add  x0, x0, _kcc_as_s_sect@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_as_s_p2a@PAGE
    add  x0, x0, _kcc_as_s_p2a@PAGEOFF
    bl _kcc_as_str
    // x22 = total buffer size (code + strings);
   x20 = code/string boundary.
    adrp x0, _kcc_code_pos@PAGE
    add  x0, x0, _kcc_code_pos@PAGEOFF
    ldr x22, [x0]
    adrp x0, _kcc_cstr_pos@PAGE
    add  x0, x0, _kcc_cstr_pos@PAGEOFF
    ldr x0, [x0]                // cstring size
    sub x20, x22, x0            // code_size =
   code_pos - cstr_pos
    // Mark internal branch targets so they get
   synthetic labels.
    mov x0, x20
    bl _kcc_as_mark_targets
    mov x19, #0                 // offset
.Lasrt_loop:
    cmp x19, x20
    b.hs .Lasrt_strings
    // Emit any labels at this offset.
    mov x0, x19
    bl _kcc_as_labels_at
    // Synthetic branch-target label (only where
   no real symbol is defined).
    lsr x0, x19, #2
    adrp x1, _kcc_as_targets@PAGE
    add  x1, x1, _kcc_as_targets@PAGEOFF
    ldr x1, [x1]
    cbz x1, .Lasrt_nolabel
    ldrb w2, [x1, x0]
    cbz w2, .Lasrt_nolabel
    adrp x0, _kcc_as_s_lc@PAGE
    add  x0, x0, _kcc_as_s_lc@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_as_dec
    mov w0, #':'
    bl _kcc_as_ch
    mov w0, #'\n'
    bl _kcc_as_ch
.Lasrt_nolabel:
    // Inline-asm snippet at this offset?  Splice
   the template verbatim in place
    // of the NOP placeholder the codegen emitted.
    mov x0, x19
    bl _kcc_as_snip_at
    cbz x0, .Lasrt_notsnip
    mov w1, #'\t'
    mov x21, x0                 // snippet entry
    (callee-saved)
    mov w0, w1
    bl _kcc_as_ch
    ldr x0, [x21, #8]           // template ptr
    ldr x1, [x21, #16]          // template len
    bl _kcc_as_putn
    mov w0, #'\n'
    bl _kcc_as_ch
    b .Lasrt_advance
.Lasrt_notsnip:
    // Load the instruction word.
    adrp x0, _kcc_code_buf@PAGE
    add  x0, x0, _kcc_code_buf@PAGEOFF
    ldr x0, [x0]
    ldr w21, [x0, x19]          // word
    // Relocated?
    mov x0, x19
    bl _kcc_as_reloc_at
    cbz x0, .Lasrt_raw
    mov x1, x21
    bl _kcc_as_reloc_line
    b .Lasrt_advance
.Lasrt_raw:
    // Internal (non-relocated) branch → symbolic
   form so `as` recomputes the
    // distance (robust to inline-asm templates
   that change the word count).
    mov w0, w21
    bl _kcc_as_branch_kind
    cbz x0, .Lasrt_rawlong
    mov w0, w21
    mov x1, x19
    bl _kcc_as_render_branch
    b .Lasrt_advance
.Lasrt_rawlong:
    adrp x0, _kcc_as_s_long@PAGE
    add  x0, x0, _kcc_as_s_long@PAGEOFF
    bl _kcc_as_str
    mov w0, w21
    bl _kcc_as_hex8
    mov w0, #'\n'
    bl _kcc_as_ch
.Lasrt_advance:
    add x19, x19, #4
    b .Lasrt_loop
.Lasrt_strings:
    // String-literal region [code_size, code_pos)
   : one `.byte` per byte, with
    // any string-literal label emitted at its
   exact offset.
    cmp x19, x22
    b.hs .Lasrt_done
    mov x0, x19
```

```
    bl _kcc_as_labels_at
    adrp x0, _kcc_as_s_byte@PAGE
    add  x0, x0, _kcc_as_s_byte@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_code_buf@PAGE
    add  x0, x0, _kcc_code_buf@PAGEOFF
    ldr x0, [x0]
    ldrb w0, [x0, x19]            // the byte
    bl _kcc_as_hex2
    mov w0, #'\n'
    bl _kcc_as_ch
    add x19, x19, #1
    b .Lasrt_strings
.Lasrt_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

537 · defined at 537, 543 · used in 30d · uses _kcc_arena_alloc 58a, _kcc_code_buf 533c, _kcc_code_pos 533c, _kcc_cstr_pos 479, _kcc_linker_relocs 594, _kcc_linker_syms 594

37.3. Writing the File

The top-level entry initialises the buffer, renders the text section, and writes the accumulated bytes to the given path in a single `write`.

⟨ *Emit-asm: render* 543 ⟩ + ≡

```
// -- kcc_as_common_size ---
// uint64_t kcc_as_common_size(uint64_t off) -
   size of the __common global at
// `off`: the gap to the next section-2 symbol
   offset, or to the end of the
// __common region.  (kcc stores only offsets, not
    per-symbol sizes.)
.globl _kcc_as_common_size
.p2align 2
_kcc_as_common_size:
    mov x8, x0                    // this offset
    adrp x1, _kcc_globals_size@PAGE
    add  x1, x1, _kcc_globals_size@PAGEOFF
    ldr x9, [x1]                  // best = total
   common size
    adrp x1, _kcc_linker_syms@PAGE
    add  x1, x1, _kcc_linker_syms@PAGEOFF
    ldr x1, [x1]
    ldr x2, [x1, #VEC_COUNT]
    ldr x3, [x1, #VEC_DATA]
    mov x4, #0
.Lcs_loop:
    cmp x4, x2
    b.hs .Lcs_done
    mov x5, #SYM_ENTRY_SIZE
    madd x6, x4, x5, x3
    ldr x7, [x6, #SYM_SECTION]
    cmp x7, #2
    b.ne .Lcs_next
    ldr x7, [x6, #SYM_VALUE]
    cmp x7, x8
    b.ls .Lcs_next                // <= this: not a
    successor
    cmp x7, x9
    b.hs .Lcs_next                // not smaller
   than best
    mov x9, x7
.Lcs_next:
    add x4, x4, #1
    b .Lcs_loop
.Lcs_done:
    sub x0, x9, x8
    ret

// -- kcc_as_render_common ---
// void kcc_as_render_common(void) - emit a `.
   zerofill __DATA,__common` entry for
// every zero-storage global (section 2).  Their
   initialisers, if any, run as
// ordinary code in the module-init function.
.globl _kcc_as_render_common
.p2align 2
_kcc_as_render_common:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    ldr x19, [x0, #VEC_COUNT]
    ldr x20, [x0, #VEC_DATA]
    mov x21, #0
.Lrc_loop:
    cmp x21, x19
    b.hs .Lrc_done
    mov x0, #SYM_ENTRY_SIZE
    madd x22, x21, x0, x20        // &sym[i]
    ldr x0, [x22, #SYM_SECTION]
    cmp x0, #2                    // __common?
    b.ne .Lrc_next
    // .globl if external
    ldr x0, [x22, #SYM_FLAGS]
    tst x0, #N_EXT
    b.eq .Lrc_zf
    adrp x0, _kcc_as_s_globl@PAGE
    add  x0, x0, _kcc_as_s_globl@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x22, #SYM_NAME]
    bl _kcc_as_str
    mov w0, #'\n'
    bl _kcc_as_ch
.Lrc_zf:
    adrp x0, _kcc_as_s_zerofill@PAGE
    add  x0, x0, _kcc_as_s_zerofill@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x22, #SYM_NAME]
    bl _kcc_as_str
    mov w0, #','
    bl _kcc_as_ch
    ldr x0, [x22, #SYM_VALUE]
    bl _kcc_as_common_size
    bl _kcc_as_dec
    adrp x0, _kcc_as_s_a3nl@PAGE
    add  x0, x0, _kcc_as_s_a3nl@PAGEOFF
    bl _kcc_as_str
.Lrc_next:
    add x21, x21, #1
    b .Lrc_loop
.Lrc_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_as_render_modinit ---
// void kcc_as_render_modinit(void) - when this TU
    synthesised a module-init
// function, emit the __mod_init_func pointer so
   dyld runs it before main (this
// is what initialises non-const globals).
.globl _kcc_as_render_modinit
.p2align 2
```

```
_kcc_as_render_modinit:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    adrp x0, _kcc_modinit_sym@PAGE
    add  x0, x0, _kcc_modinit_sym@PAGEOFF
    ldr x19, [x0]                   // symbol index
   + 1 (0 = none)
    cbz x19, .Lrmi_done
    adrp x0, _kcc_as_s_modsect@PAGE
    add  x0, x0, _kcc_as_s_modsect@PAGEOFF
    bl _kcc_as_str
    sub x0, x19, #1
    bl _kcc_as_symname
    mov w0, #'\n'
    bl _kcc_as_ch
.Lrmi_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_as_render ---
// int kcc_as_render(const char *path) - render
   the current translation unit's
// generated code as `.s` text and write it to `
   path`.  Returns 0 on success.
.globl _kcc_as_render
.p2align 2
_kcc_as_render:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                     // path
    bl _kcc_as_init
    bl _kcc_as_render_text
    bl _kcc_as_render_common        // __common zero
   -storage globals
    bl _kcc_as_render_modinit       //
   __mod_init_func (global initialisers)
    mov x0, x19
    bl _kcc_as_write_file
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_as_write_file ---
// int kcc_as_write_file(const char *path) - write
    the rendered text buffer
// (_kcc_as_buf, _kcc_as_pos) to `path` and close
   it.  Shared by every
// assembly-emission back end (ARM64 `.s` and x86
   -64), so the buffer globals
// stay file-local to this module.  Returns 0 on
   success, 1 on error.
.globl _kcc_as_write_file
.p2align 2
_kcc_as_write_file:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x20, [sp, #16]
    // Open the output file (O_WRONLY|O_CREAT|
   O_TRUNC = 0x601, mode 0644).
    mov x1, #0x601
    mov x2, #420
    bl _kcc_sys_open
    cmp x0, #0
    b.lt .Lawf_err
    mov x20, x0                     // fd
    // write(fd, buf, pos)
    adrp x0, _kcc_as_buf@PAGE
    add  x0, x0, _kcc_as_buf@PAGEOFF
    ldr x1, [x0]                    // buffer
    adrp x0, _kcc_as_pos@PAGE
    add  x0, x0, _kcc_as_pos@PAGEOFF
    ldr x2, [x0]                    // length
    mov x0, x20                     // fd
    bl _kcc_sys_write
    mov x0, x20
    bl _kcc_sys_close
    mov x0, #0
    b .Lawf_done
.Lawf_err:
    mov x0, #1
.Lawf_done:
    ldr x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

543 · defined at 537, 543 · used in 30d · uses _kcc_globals_size 594, _kcc_linker_syms 594, _kcc_modinit_sym 305c

38. A Second Target: x86-64 by Assembly Emission

The flagship ARM64 back end (Chapters 36–40) proves kcc can go all the way to the metal: it encodes every instruction into its own bytes and lays out the Mach-O container by hand. The second target takes the other road the porting playbook (Chapter 2) sanctions—*assembly emission*. Instead of encoding instructions and writing an object file, this back end walks the same target-neutral IR the ARM64 path consumes and *prints x86-64 assembly text*, then hands that text to the platform assembler (`as -arch x86_64`). The assembler does the encoding and produces the Mach-O object; the linker (`ld -arch x86_64`) produces an executable that runs on Apple silicon under the Rosetta translator. Nothing above the emission step—the front end, the IR, the optimizer—knows or cares that a different machine is now the destination; that is the whole point of the seam.

This chapter is built one rung of the playbook ladder at a time. Rung one (§38.15) was the smallest program that exercises the entire pipeline end to end, `int main(void) { return N; }`, whose optimized IR is a single `IR_CONST` feeding an `IR_RET`. With the scaffolding proven live, this chapter is *rung two*: the integer arithmetic–logic unit plus the memory operations that give locals an address. That is exactly the set of opcodes a function like

```
int f(void) {
    volatile int x = 40;
    volatile int y = 2;
    return x + y;
}
```

lowers to—`IR_ALLOCA` for each local, `IR_STORE` to initialise it, `IR_LOAD` to read it back (`volatile` stops `mem2reg` from promoting the locals into registers, so the memory traffic survives to the back end), `IR_ADD` to combine them, and an `IR_SEXT` feeding the return. Rung two selects for the whole integer ALU (add, subtract, multiply, divide, remainder, the bitwise and shift operators, negate and complement), the width-changing conversions (`sext`/`zext`/`trunc`/`bitcast`), and the memory opcodes (`alloca`/`load`/`store`/`gep`).

Rung three completes the scalar language: control flow and function calls. It adds the comparisons (`cmp_eq` through `cmp_ule`), the branches (`br`/`br_cond`), and the call machinery (`call`/`call_ind`/`global_ref`) marshalled to the System V AMD64 ABI, along with the callee-side spill of incoming parameters. With this rung a whole program of mutually recursive integer functions, conditionals, and loops compiles and runs exit-identically to clang, provided its data lives in *functions*: parameters, locals, and calls between defined functions all work, and an `IR_GLOBAL_REF` to a defined *function* resolves (that is how an indirect call names its target). What this rung deliberately leaves out is *data emission*: it emits no storage for file-scope variables, so a reference to a global object is an unresolved symbol at link time rather than a miscompile. That, together with string-literal data, floating point, and by-value aggregates, is the work of a later rung; an unrecognised opcode is still silently skipped.

38.1. Strategy: an All-Spill Frame

The x86-64 back end borrows the ARM64 code generator's "all-spill" strategy (§**??**) wholesale, because it is the simplest thing that is correct and it sidesteps register allocation entirely. *Every virtual register owns one eight-byte slot in the stack frame.* An instruction loads its operands from their slots into scratch registers, computes, and stores the result back to the destination's slot. No value is assumed to survive in a register from one IR instruction to the next, so there is no allocation, no liveness, and no spilling logic—just a fixed, mechanical lowering.

The frame is the conventional `%rbp`-anchored one and has two regions below the saved base pointer:

```
0(%rbp)                  saved %rbp (pushq'd on en
-8(%rbp) .. -vc*8(%rbp)  vreg value slots   (vreg
below that               alloca storage     (allo
```

Virtual register V lives at `-(V+1)*8(%rbp)`. Alloca storage begins past the value slots so an `alloca`'s object can never overlap a vreg slot (the same anti-aliasing discipline the ARM64 frame uses). A pre-scan assigns each `IR_ALLOCA` a running slot offset and sums the total; the frame is `align16((vreg_count + alloca_slots) * 8)` bytes, carved out with one `subq` in the prologue and released by `leave` at each return.

Arithmetic runs at full 64-bit width, exactly as the ARM64 path does: operands arrive in their slots already sign- or zero-extended (the loads and the explicit conversion opcodes see to that), so a 64-bit `addq` computes the correct low bits and the IR's own `sext`/`trunc` nodes renormalise a narrower result where the language requires it. The scratch registers are `%r10` (first source and result), `%r11` (second source), and—only for the shift and divide instructions, which pin specific registers—`%rcx`, `%rax`, and `%rdx`.

38.2. Target State and Assembly Vocabulary

A single global, `_kcc_target`, records which back end the driver selected: `0` (the default) is ARM64/Mach-O and leaves the flagship path completely untouched; `1` is x86-64. A second file-local global, `_kcc_x86_vreg_n`, holds the current function's virtual-register count so instruction selection can size the frame and compute alloca addresses. The remaining data are the fixed fragments of AT&T assembly this back end pours into the text buffer—section and symbol directives, the frame prologue/epilogue, every instruction mnemonic, and the register operands. AT&T

syntax names registers with a % sigil, immediates with $, orders operands source-then-destination, and writes a memory operand as `disp(base)`.

⟨ *X86: data* 546 ⟩ ≡

```
.data
.globl _kcc_target
.globl _kcc_x86_tu
.globl _kcc_x86_os
_kcc_target:      .quad 0         // 0 = arm64/Mach-O (default), 1 = x86-64
_kcc_x86_vreg_n: .quad 0          // current function's vreg count (frame sizing)
_kcc_x86_tu:      .quad 0         // the AST translation unit (for data emission)
_kcc_x86_strs:    .quad 0         // vec of {data_ptr, len} string-literal entries
_kcc_x86_os:      .quad 0         // 0 = Darwin/Mach-O (default), 1 = Linux/ELF

.section __TEXT,__cstring,cstring_literals
// Directives and symbol scaffolding.
_kcc_x86_s_text:      .asciz ".text\n"
_kcc_x86_s_globl:     .asciz ".globl "
_kcc_x86_s_us:        .asciz "_"

// Linux/ELF-flavour directives (selected when _kcc_x86_os = 1).
_kcc_x86_s_rodata:    .asciz ".section .rodata\n"
_kcc_x86_s_comm:      .asciz "\t.comm "
_kcc_x86_s_comm_a8:   .asciz ",8\n"           // ",<size>,8\n" .comm alignment tail
_kcc_x86_s_local:     .asciz "\t.local "      // marks a static common as ELF-local

// Module-init pointer: the section header, then ".quad <name>\n" registers the
// synthesised kcc.module_init so it runs before main.  Darwin: __mod_init_func;
// Linux/ELF: .init_array.
_kcc_x86_s_modinit_d:.asciz ".section __DATA,__mod_init_func,mod_init_funcs\n\t.p2align 3\n\t.quad "
_kcc_x86_s_modinit_l:.asciz ".section .init_array,\"aw\"\n\t.p2align 3\n\t.quad "
_kcc_x86_s_modinitnm:.asciz "kcc.module_init"

// Read-only initialised data (const globals): section, alignment, and the
// value directives keyed by element width.  Darwin uses __TEXT,__const.
_kcc_x86_s_constsect:.asciz ".section __TEXT,__const\n"
_kcc_x86_s_p2align3: .asciz "\t.p2align 3\n"
_kcc_x86_s_d_byte:    .asciz "\t.byte "
_kcc_x86_s_d_short:   .asciz "\t.short "
_kcc_x86_s_d_long:    .asciz "\t.long "
_kcc_x86_s_d_quad:    .asciz "\t.quad "
_kcc_x86_s_zerodir:   .asciz "\t.zero "
_kcc_x86_s_colon_nl: .asciz ":\n"
_kcc_x86_s_minus:     .asciz "-"
_kcc_x86_s_rbp:       .asciz "(%rbp)"
_kcc_x86_s_comma:     .asciz ", "

// Frame prologue/epilogue.
_kcc_x86_s_push_rbp: .asciz "\tpushq\t%rbp\n"
_kcc_x86_s_mov_sp:    .asciz "\tmovq\t%rsp, %rbp\n"
_kcc_x86_s_sub_sp:    .asciz "\tsubq\t$"
_kcc_x86_s_sp_nl:     .asciz ", %rsp\n"
_kcc_x86_s_leave:     .asciz "\tleave\n"
_kcc_x86_s_retq:      .asciz "\tretq\n"

// Move family (with a trailing tab where an operand follows).
_kcc_x86_s_movq:      .asciz "\tmovq\t"
_kcc_x86_s_movabsq:   .asciz "\tmovabsq\t$"
_kcc_x86_s_movl:      .asciz "\tmovl\t"
_kcc_x86_s_movb:      .asciz "\tmovb\t"
_kcc_x86_s_movw:      .asciz "\tmovw\t"
_kcc_x86_s_movslq:    .asciz "\tmovslq\t"
_kcc_x86_s_movsbq:    .asciz "\tmovsbq\t"
_kcc_x86_s_movswq:    .asciz "\tmovswq\t"
_kcc_x86_s_movzbq:    .asciz "\tmovzbq\t"
_kcc_x86_s_movzwq:    .asciz "\tmovzwq\t"
_kcc_x86_s_leaq:      .asciz "\tleaq\t"

// Integer ALU mnemonics.
_kcc_x86_s_addq:      .asciz "\taddq\t"
_kcc_x86_s_subq:      .asciz "\tsubq\t"
_kcc_x86_s_imulq:     .asciz "\timulq\t"
_kcc_x86_s_andq:      .asciz "\tandq\t"
_kcc_x86_s_orq:       .asciz "\torq\t"
_kcc_x86_s_xorq:      .asciz "\txorq\t"
_kcc_x86_s_negq:      .asciz "\tnegq\t"
_kcc_x86_s_notq:      .asciz "\tnotq\t"
_kcc_x86_s_shlq:      .asciz "\tshlq\t"
_kcc_x86_s_sarq:      .asciz "\tsarq\t"
_kcc_x86_s_shrq:      .asciz "\tshrq\t"
_kcc_x86_s_cqto:      .asciz "\tcqto\n"
_kcc_x86_s_xor_rdx:   .asciz "\txorq\t%rdx, %rdx\n"
_kcc_x86_s_idiv_rcx: .asciz "\tidivq\t%rcx\n"
_kcc_x86_s_div_rcx:   .asciz "\tdivq\t%rcx\n"

// Register operands and common instruction tails.
_kcc_x86_s_r10:       .asciz "%r10"
_kcc_x86_s_r11:       .asciz "%r11"
_kcc_x86_s_rax:       .asciz "%rax"
_kcc_x86_s_rcx:       .asciz "%rcx"
_kcc_x86_s_rdx:       .asciz "%rdx"
_kcc_x86_s_r11_r10:   .asciz "%r11, %r10\n"   // binary op tail: dst = dst OP r11
_kcc_x86_s_r10_nl:    .asciz "%r10\n"          // unary op tail
_kcc_x86_s_cl_r10:    .asciz "%cl, %r10\n"     // shift-by-%cl tail
_kcc_x86_s_c_r10:     .asciz ", %r10\n"        // ", %r10\n"
_kcc_x86_s_c_r10d:    .asciz ", %r10d\n"       // ", %r10d\n"
// Memory operand tails for load (source (%r11)) and store (dest (%r11)).
_kcc_x86_s_pr11_r10: .asciz "(%r11), %r10\n"
_kcc_x86_s_pr11_r10d:.asciz "(%r11), %r10d\n"
_kcc_x86_s_r10_pr11: .asciz "%r10, (%r11)\n"
_kcc_x86_s_r10b_pr11:.asciz "%r10b, (%r11)\n"
_kcc_x86_s_r10w_pr11:.asciz "%r10w, (%r11)\n"
_kcc_x86_s_r10d_pr11:.asciz "%r10d, (%r11)\n"
```

□

546 · defined at 546, 547ab · used in 31a · uses _kcc_target 568b, _kcc_x86_tu 568b

38.2.1. Vocabulary for Control Flow and Calls (Rung 3)

Rung three adds the comparison, branch, and call vocabulary. Comparisons emit a `cmp` at the operands' common width followed by a `set`*cc* into `%al` and a `movzbl` that widens the boolean into `%r10`. Branches are the great

simplification of assembly emission: a basic block becomes a named label `.LkccbbN` (block ids are globally unique), so an `IR_BR` is a `jmp` to that label and the assembler resolves forward and backward references itself—no patch vector. Calls follow the System V AMD64 ABI: the integer argument registers, in order, are `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9` (collected in `_kcc_x86_argregs` for index-based lookup), the return value comes back in `%rax`, and a direct call names the callee with the Mach-O leading underscore.

⟨ *X86: data* 547a ⟩ + ≡

```
// Comparison: cmp at width, set<cc> into %al,
   widen with movzbl.
_kcc_x86_s_cmpl:     .asciz "\tcmpl\t%r11d, %r10d\
   n"
_kcc_x86_s_cmpq:     .asciz "\tcmpq\t%r11, %r10\n"
_kcc_x86_s_movzbl_al:.asciz "\tmovzbl\t%al, %r10d\
   n"
_kcc_x86_s_sete:     .asciz "\tsete\t%al\n"
_kcc_x86_s_setne:    .asciz "\tsetne\t%al\n"
_kcc_x86_s_setl:     .asciz "\tsetl\t%al\n"
_kcc_x86_s_setle:    .asciz "\tsetle\t%al\n"
_kcc_x86_s_setg:     .asciz "\tsetg\t%al\n"
_kcc_x86_s_setge:    .asciz "\tsetge\t%al\n"
_kcc_x86_s_setb:     .asciz "\tsetb\t%al\n"
_kcc_x86_s_setbe:    .asciz "\tsetbe\t%al\n"

// Branches: named basic-block labels resolved by
   the assembler.
_kcc_x86_s_bb:       .asciz ".Lkccbb"        //
   label stem; a decimal id follows
_kcc_x86_s_label_nl: .asciz ":\n"
_kcc_x86_s_jmp:      .asciz "\tjmp\t"
_kcc_x86_s_jne:      .asciz "\tjne\t"
_kcc_x86_s_test_r10: .asciz "\ttestq\t%r10, %r10\n
   "

// Calls: System V AMD64 argument registers, call
   forms, and stack adjust.
_kcc_x86_s_rdi:      .asciz "%rdi"
_kcc_x86_s_rsi:      .asciz "%rsi"
_kcc_x86_s_r8:       .asciz "%r8"
_kcc_x86_s_r9:       .asciz "%r9"
_kcc_x86_s_call:     .asciz "\tcall\t"
_kcc_x86_s_call_r11: .asciz "\tcall\t*%r11\n"
_kcc_x86_s_zero_al:  .asciz "\tmovb\t$0, %al\n"
_kcc_x86_s_sub_rsp:  .asciz "\tsubq\t$"
_kcc_x86_s_add_rsp:  .asciz "\taddq\t$"
_kcc_x86_s_rsp:      .asciz ", %rsp\n"
_kcc_x86_s_paren_rsp:.asciz "(%rsp)"
_kcc_x86_s_rip_r10:  .asciz "(%rip), %r10\n"
_kcc_x86_s_got_r10:  .asciz "@GOTPCREL(%rip), %r10
   \n"

// Data emission: zeroed common storage for a file
   -scope global.
_kcc_x86_s_zerofill: .asciz "\t.zerofill __DATA,
   __common,_"
_kcc_x86_s_zf_a3:    .asciz ",3\n"      // ",<size
   >,3\n" alignment tail (8-byte)
_kcc_x86_s_comma1:   .asciz ","

// Refusal diagnostic for an opcode the back end
   does not yet implement.
_kcc_x86_s_unsup:    .asciz "kcc: --target=x86_64:
    unsupported operation (floating point, inline
   asm, or by-value aggregate not yet implemented
   for the x86-64 back end)\n"

// String literals: a private label per literal,
   its address via leaq, and the
// cstring section that holds the bytes.
_kcc_x86_s_strlbl:   .asciz ".Lx86str"      //
   label stem; a decimal id follows
_kcc_x86_s_cstrsect: .asciz ".section __TEXT,
   __cstring,cstring_literals\n"
_kcc_x86_s_byte:     .asciz "\t.byte "
_kcc_x86_s_zero_nl:  .asciz "0\n"

// The argument-register lookup table holds
   pointers, so it lives in a
// relocatable data section, not the cstring pool
   (which admits no relocations).
.data
.p2align 3
_kcc_x86_argregs:
    .quad _kcc_x86_s_rdi
    .quad _kcc_x86_s_rsi
    .quad _kcc_x86_s_rdx
    .quad _kcc_x86_s_rcx
    .quad _kcc_x86_s_r8
    .quad _kcc_x86_s_r9
```

□

547a · defined at 546, 547ab · used in 31a

38.2.2. Vocabulary for Floating Point (Rung 4)

Floating point is the fourth rung, and the one kcc needs to compile itself (its float-literal parser and its `const`-float narrowing both do real FP). The all-spill model carries a floating value in the same 8-byte slot an integer uses—holding the IEEE-754 *double* bit pattern, exactly as the ARM64 back end does—so the only new machinery is the SSE instructions that move a slot into an `%xmm` register, operate, and write it back. Two scratch SSE registers, `%xmm0` and `%xmm1`, mirror the integer `%r10`/`%r11` pair. A `float` in memory is the single-precision narrowing of that double, so a width-code-8 load widens with `cvtss2sd` and a width-code-8 store narrows with `cvtsd2ss`; every value the selection carries between slots stays a double.

⟨ *X86: data* 547b ⟩ + ≡

```
.section __TEXT,__cstring,cstring_literals
// SSE moves and the scratch xmm operands.
_kcc_x86_s_movsd:    .asciz "\tmovsd\t"
_kcc_x86_s_movss:    .asciz "\tmovss\t"
_kcc_x86_s_cx0_nl:   .asciz ", %xmm0\n"
_kcc_x86_s_cx1_nl:   .asciz ", %xmm1\n"
_kcc_x86_s_x0_c:     .asciz "%xmm0, "
_kcc_x86_s_nl:       .asciz "\n"
// FP binary ops: dst(%xmm0) = %xmm0 OP %xmm1.
_kcc_x86_s_addsd:    .asciz "\taddsd\t"
_kcc_x86_s_subsd:    .asciz "\tsubsd\t"
_kcc_x86_s_mulsd:    .asciz "\tmulsd\t"
_kcc_x86_s_divsd:    .asciz "\tdivsd\t"
_kcc_x86_s_x1_x0:    .asciz "%xmm1, %xmm0\n"
// Conversions between the integer file, xmm, and
   single/double precision.
_kcc_x86_s_cvtsi2sd: .asciz "\tcvtsi2sdq\t%r10, %
   xmm0\n"
_kcc_x86_s_cvttsd2si:.asciz "\tcvttsd2si\t%xmm0, %
   r10\n"
_kcc_x86_s_cvtsd2ss: .asciz "\tcvtsd2ss\t%xmm0, %
   xmm0\n"
_kcc_x86_s_cvtss2sd: .asciz "\tcvtss2sd\t%xmm0, %
   xmm0\n"
```

```
// Single-precision memory operands (float load/
   store through %r11).
_kcc_x86_s_px11_x0:  .asciz "(%r11), %xmm0\n"
_kcc_x86_s_x0_px11:  .asciz "%xmm0, (%r11)\n"
```

□

547b · defined at 546, 547ab · used in 31a

38.3. Two Object-Format Flavours: Darwin and Linux

The instruction *selection* above is pure x86-64 and does not care which operating system will host the program—but the surrounding assembly *directives* do. Mach-O (Darwin) and ELF (Linux) disagree on three things this back end emits: Mach-O prefixes every C symbol with an underscore while ELF does not; their section directives are spelled differently (`.section __TEXT,__cstring` versus `.section .rodata`, `.zerofill __DATA,__common` versus `.comm`); and a reference to an external symbol goes through the GOT on Darwin (a dylib import has no image address) but resolves directly in a statically linked ELF executable. A single global, `_kcc_x86_os`, selects the flavour: 0 is Darwin (the default, so the existing Mach-O path is untouched) and 1 is Linux.

The one pervasive difference—the underscore—is funnelled through a helper. `_kcc_x86_sym` emits a symbol name preceded by an underscore only under Darwin, so every label, `.globl`, and call target is written the same way and the flavour is decided in one place.

⟨ *X86: render* 548a ⟩ ≡

```
.text
// -- kcc_x86_sym(const char *name) - append a
   symbol: "_"+name on Darwin,
//    bare name on Linux/ELF.
.p2align 2
_kcc_x86_sym:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                          // name
    adrp x0, _kcc_x86_os@PAGE
    add  x0, x0, _kcc_x86_os@PAGEOFF
    ldr x0, [x0]
    cbnz x0, .Lx86sym_bare               // Linux: no
   underscore
    adrp x0, _kcc_x86_s_us@PAGE
    add  x0, x0, _kcc_x86_s_us@PAGEOFF
    bl _kcc_as_str
.Lx86sym_bare:
    mov x0, x19
    bl _kcc_as_str
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

548a · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a

38.4. Text-Buffer Emission Helpers

Selection is written in terms of a handful of primitives over the growable text buffer (§??): `_kcc_as_str` appends a NUL-terminated string, `_kcc_as_ch` one byte, and `_kcc_as_dec` an unsigned decimal. On top of those, `_kcc_x86_disp` prints a frame displacement `-D(%rbp)`, and `_kcc_x86_slot` prints the operand for virtual register V, whose displacement is $(V+1)\times 8$. `_kcc_x86_load` emits `movq slot(V), reg` (a slot always holds a canonical 64-bit value, so a plain `movq` reads any vreg), and `_kcc_x86_store` the inverse `movq reg, slot(V)`. A register operand is passed by pointer to its name string.

⟨ *X86: render* 548b ⟩ + ≡

```
.text
// -- kcc_x86_disp(uint64_t d) - append "-d(%rbp)"
    (a frame displacement).
.p2align 2
_kcc_x86_disp:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                          // d
    adrp x0, _kcc_x86_s_minus@PAGE
    add  x0, x0, _kcc_x86_s_minus@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_rbp@PAGE
    add  x0, x0, _kcc_x86_s_rbp@PAGEOFF
    bl _kcc_as_str
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_x86_slot(uint64_t v) - append the
   operand for vreg v: "-(v+1)*8(%rbp)".
.p2align 2
_kcc_x86_slot:
    add x0, x0, #1
    lsl x0, x0, #3
    b _kcc_x86_disp

// -- kcc_x86_load(uint64_t v, const char *reg) -
   emit "movq slot(v), reg".
.p2align 2
_kcc_x86_load:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                          // v
    mov x20, x1                          // reg name
    adrp x0, _kcc_x86_s_movq@PAGE
    add  x0, x0, _kcc_x86_s_movq@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_x86_slot
    adrp x0, _kcc_x86_s_comma@PAGE
    add  x0, x0, _kcc_x86_s_comma@PAGEOFF
    bl _kcc_as_str
    mov x0, x20
    bl _kcc_as_str
    mov x0, #10                          // '\n'
    bl _kcc_as_ch
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_x86_store(uint64_t v, const char *reg)
   - emit "movq reg, slot(v)".
.p2align 2
_kcc_x86_store:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
```

```
    stp x19, x20, [sp, #16]
    mov x19, x0                         // v
    mov x20, x1                         // reg name
    adrp x0, _kcc_x86_s_movq@PAGE
    add  x0, x0, _kcc_x86_s_movq@PAGEOFF
    bl _kcc_as_str
    mov x0, x20
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_comma@PAGE
    add  x0, x0, _kcc_x86_s_comma@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_x86_slot
    mov x0, #10                         // '\n'
    bl _kcc_as_ch
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

548b · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a

38.5. Instruction Selection

Selection is a per-instruction dispatch: given one IR instruction, emit the equivalent x86-64 and leave the result in the destination's slot. The opcode routes to a handler; anything rung two does not yet implement (control flow, floating point) falls through and emits nothing. We build the handlers bottom-up as a set of small helpers—one per instruction *shape*—then the dispatch that calls them.

38.5.1. Binary, Unary, and Shift Shapes

The commutative and two-operand ALU instructions all share one shape: load SRC1 into %r10, load SRC2 into %r11, emit the mnemonic followed by %r11, %r10 (so %r10 becomes %r10 OP %r11), and store %r10 to DEST. Subtraction is non-commutative but the operand order is already correct (subq %r11, %r10 is src1 − src2). The unary shape (negate, complement) omits the second load. The shift shape loads the count into %rcx because x86 variable shifts take their amount only in %cl.

⟨ *X86: render* 549 ⟩ +≡

```
// -- kcc_x86_binop(ir_instr *i, const char *mnem)
    - dst = src1 OP src2.
.p2align 2
_kcc_x86_binop:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                         // instr
    mov x20, x1                         // mnemonic
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_load
    ldr x0, [x19, #IR_SRC2]
    adrp x1, _kcc_x86_s_r11@PAGE
    add  x1, x1, _kcc_x86_s_r11@PAGEOFF
    bl _kcc_x86_load
    mov x0, x20
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_r11_r10@PAGE
    add  x0, x0, _kcc_x86_s_r11_r10@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_store
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_x86_unop(ir_instr *i, const char *mnem)
    - dst = OP src1.
.p2align 2
_kcc_x86_unop:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0
    mov x20, x1
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_load
    mov x0, x20
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_r10_n1@PAGE
    add  x0, x0, _kcc_x86_s_r10_n1@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_store
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_x86_shift(ir_instr *i, const char *mnem)
    - dst = src1 SHIFT src2 (%cl).
.p2align 2
_kcc_x86_shift:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0
    mov x20, x1
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_load
    ldr x0, [x19, #IR_SRC2]
    adrp x1, _kcc_x86_s_rcx@PAGE
    add  x1, x1, _kcc_x86_s_rcx@PAGEOFF
    bl _kcc_x86_load
    mov x0, x20
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_cl_r10@PAGE
    add  x0, x0, _kcc_x86_s_cl_r10@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_store
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_x86_fpload(uint64_t v, const char *
   xmm_tail) - "movsd slot(v)<tail>".
//    xmm_tail is ", %xmm0\n" or ", %xmm1\n".
.p2align 2
_kcc_x86_fpload:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                         // v
    mov x20, x1                         // tail
    adrp x0, _kcc_x86_s_movsd@PAGE
```

```
    add  x0, x0, _kcc_x86_s_movsd@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_x86_slot
    mov x0, x20
    bl _kcc_as_str
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_x86_fpstore(uint64_t v) - "movsd %xmm0,
   slot(v)".
.p2align 2
_kcc_x86_fpstore:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                          // v
    adrp x0, _kcc_x86_s_movsd@PAGE
    add  x0, x0, _kcc_x86_s_movsd@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_x0_c@PAGE
    add  x0, x0, _kcc_x86_s_x0_c@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_x86_slot
    adrp x0, _kcc_x86_s_nl@PAGE
    add  x0, x0, _kcc_x86_s_nl@PAGEOFF
    bl _kcc_as_str
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_x86_fpbinop(ir_instr *i, const char *
   mnem) - %xmm0 = src1 OP src2.
//    Loads both operands into %xmm0/%xmm1,
   applies the SSE op, stores %xmm0.
.p2align 2
_kcc_x86_fpbinop:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                          // instr
    mov x20, x1                          // mnemonic
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_cx0_nl@PAGE
    add  x1, x1, _kcc_x86_s_cx0_nl@PAGEOFF
    bl _kcc_x86_fpload
    ldr x0, [x19, #IR_SRC2]
    adrp x1, _kcc_x86_s_cx1_nl@PAGE
    add  x1, x1, _kcc_x86_s_cx1_nl@PAGEOFF
    bl _kcc_x86_fpload
    mov x0, x20
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_x1_x0@PAGE
    add  x0, x0, _kcc_x86_s_x1_x0@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    bl _kcc_x86_fpstore
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

549 · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a

38.5.2. Division and Remainder

x86 division is the one integer operation with fixed register operands. The dividend goes in %rdx:%rax; idivq/divq leaves the quotient in %rax and the remainder in %rdx. The helper loads SRC1 into %rax, widens it into %rdx (cqto sign-extends for the signed forms, xorq %rdx, %rdx zeroes it for the unsigned ones), loads the divisor into %rcx, divides, and stores either %rax (quotient, for sdiv/udiv) or %rdx (remainder, for smod/umod) to DEST. Because the operands were sign- or zero-extended when they were loaded from memory, the full-width division agrees with C's trunc-toward-zero semantics on the narrow type.

⟨ X86: render 550 ⟩ +≡

```
// -- kcc_x86_divop(ir_instr *i, uint64_t
   is_signed, uint64_t want_rem)
.p2align 2
_kcc_x86_divop:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                          // instr
    mov x20, x1                          // is_signed
    mov x21, x2                          // want_rem
    // %rax = src1
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_rax@PAGE
    add  x1, x1, _kcc_x86_s_rax@PAGEOFF
    bl _kcc_x86_load
    // Widen the dividend into %rdx.
    cbz x20, .Lx86div_unsigned
    adrp x0, _kcc_x86_s_cqto@PAGE
    add  x0, x0, _kcc_x86_s_cqto@PAGEOFF
    bl _kcc_as_str
    b .Lx86div_divisor
.Lx86div_unsigned:
    adrp x0, _kcc_x86_s_xor_rdx@PAGE
    add  x0, x0, _kcc_x86_s_xor_rdx@PAGEOFF
    bl _kcc_as_str
.Lx86div_divisor:
    // %rcx = src2
    ldr x0, [x19, #IR_SRC2]
    adrp x1, _kcc_x86_s_rcx@PAGE
    add  x1, x1, _kcc_x86_s_rcx@PAGEOFF
    bl _kcc_x86_load
    // idivq / divq %rcx
    cbz x20, .Lx86div_unsigned_op
    adrp x0, _kcc_x86_s_idiv_rcx@PAGE
    add  x0, x0, _kcc_x86_s_idiv_rcx@PAGEOFF
    bl _kcc_as_str
    b .Lx86div_result
.Lx86div_unsigned_op:
    adrp x0, _kcc_x86_s_div_rcx@PAGE
    add  x0, x0, _kcc_x86_s_div_rcx@PAGEOFF
    bl _kcc_as_str
.Lx86div_result:
    // Store %rax (quotient) or %rdx (remainder)
   to DEST.
    ldr x0, [x19, #IR_DEST]
    cbnz x21, .Lx86div_rem
    adrp x1, _kcc_x86_s_rax@PAGE
    add  x1, x1, _kcc_x86_s_rax@PAGEOFF
    b .Lx86div_store
.Lx86div_rem:
    adrp x1, _kcc_x86_s_rdx@PAGE
    add  x1, x1, _kcc_x86_s_rdx@PAGEOFF
.Lx86div_store:
    bl _kcc_x86_store
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

550 · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a

38.5.3. Width-Changing Conversions

The four conversions all read SRC1 straight from its slot with a single extending move—the low bytes of the canonical slot value are exactly the narrow source—and store the widened result. _kcc_x86_extend takes the mnemonic and the destination-register tail (, %r10 for the 64-bit forms, , %r10d for movl, which zero-extends into %r10) and emits mnem slot(SRC1), reg. IR_SEXT and IR_ZEXT choose the mnemonic from their source width in SRC2 (movsbq/movswq/movslq signed, movzbq/movzwq/movl unsigned); IR_TRUNC chooses from its *target* width in IR_TYPE and zero-extends the surviving low bytes, matching the ARM64 mask-and-copy; IR_BITCAST is a plain movq copy.

⟨ *X86: render* 551 ⟩ + ≡

```
// -- kcc_x86_extend(ir_instr *i, const char *mnem
   , const char *tail)
//    Emit "mnem slot(SRC1)tail" then store %r10
   to DEST.  `tail` supplies the
//    ", %r10\n" (or ", %r10d\n") destination-
   register text.
.p2align 2
_kcc_x86_extend:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                         // instr
    mov x20, x1                         // mnemonic
    mov x21, x2                         // tail
    mov x0, x20
    bl _kcc_as_str
    ldr x0, [x19, #IR_SRC1]
    bl _kcc_x86_slot
    mov x0, x21
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_store
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_x86_conv(ir_instr *i, uint64_t width,
   uint64_t is_signed)
//    Pick the extending mnemonic/tail for a `
   width`-byte source and emit it.
//    is_signed selects movs* vs movz*/movl; width
    8 or 0 is a plain movq copy.
.p2align 2
_kcc_x86_conv:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x1                         // width
    mov x20, x2                         // is_signed
    // x0 already holds instr; stash it so the
   dispatch below can reload.
    str x0, [sp, #-16]!                 // save instr
   (16-byte aligned push)
    cmp x19, #1
    b.eq .Lx86conv_b
    cmp x19, #2
    b.eq .Lx86conv_w
    cmp x19, #4
    b.eq .Lx86conv_l
    // Default (8 or unknown): movq copy, no
   extension.
    ldr x0, [sp], #16
    adrp x1, _kcc_x86_s_movq@PAGE
    add  x1, x1, _kcc_x86_s_movq@PAGEOFF
    adrp x2, _kcc_x86_s_c_r10@PAGE
    add  x2, x2, _kcc_x86_s_c_r10@PAGEOFF
    bl _kcc_x86_extend
    b .Lx86conv_done
.Lx86conv_b:
    ldr x0, [sp], #16
    cbz x20, .Lx86conv_bz
    adrp x1, _kcc_x86_s_movsbq@PAGE
    add  x1, x1, _kcc_x86_s_movsbq@PAGEOFF
    b .Lx86conv_b_go
.Lx86conv_bz:
    adrp x1, _kcc_x86_s_movzbq@PAGE
    add  x1, x1, _kcc_x86_s_movzbq@PAGEOFF
.Lx86conv_b_go:
    adrp x2, _kcc_x86_s_c_r10@PAGE
    add  x2, x2, _kcc_x86_s_c_r10@PAGEOFF
    bl _kcc_x86_extend
    b .Lx86conv_done
.Lx86conv_w:
    ldr x0, [sp], #16
    cbz x20, .Lx86conv_wz
    adrp x1, _kcc_x86_s_movswq@PAGE
    add  x1, x1, _kcc_x86_s_movswq@PAGEOFF
    b .Lx86conv_w_go
.Lx86conv_wz:
    adrp x1, _kcc_x86_s_movzwq@PAGE
    add  x1, x1, _kcc_x86_s_movzwq@PAGEOFF
.Lx86conv_w_go:
    adrp x2, _kcc_x86_s_c_r10@PAGE
    add  x2, x2, _kcc_x86_s_c_r10@PAGEOFF
    bl _kcc_x86_extend
    b .Lx86conv_done
.Lx86conv_l:
    ldr x0, [sp], #16
    cbz x20, .Lx86conv_lz
    // Signed →3264: movslq slot, %r10.
    adrp x1, _kcc_x86_s_movslq@PAGE
    add  x1, x1, _kcc_x86_s_movslq@PAGEOFF
    adrp x2, _kcc_x86_s_c_r10@PAGE
    add  x2, x2, _kcc_x86_s_c_r10@PAGEOFF
    bl _kcc_x86_extend
    b .Lx86conv_done
.Lx86conv_lz:
    // Unsigned →3264: movl slot, %r10d (
   implicitly zero-extends into %r10).
    adrp x1, _kcc_x86_s_movl@PAGE
    add  x1, x1, _kcc_x86_s_movl@PAGEOFF
    adrp x2, _kcc_x86_s_c_r10d@PAGE
    add  x2, x2, _kcc_x86_s_c_r10d@PAGEOFF
    bl _kcc_x86_extend
.Lx86conv_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

551 · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a

38.5.4. Memory Operations

IR_LOAD dereferences a pointer with an access-size override in SRC2 (the code $1/2/4$ unsigned, $5/6/7$ signed, 0 eight-byte; 8 is a single-precision float, deferred to a later rung). The helper loads the pointer into %r11, emits the width-appropriate move from (%r11) into %r10 (or %r10d for the zero-extending 32-bit movl), and stores %r10 to DEST. IR_STORE is the inverse with its size override in IR_DEST ($0/1/2/4$): load the value into %r10 and the pointer

into %r11, then mov %r10x, (%r11) at the right width. IR_GEP (get element pointer) is the address arithmetic that array subscripts and struct member accesses lower to: its second operand is already a *byte* offset (the IR builder pre-scales a subscript by the element size), so the address is just SRC1 + SRC2—the very shape of a two-operand ALU add, and so it reuses _kcc_x86_binop with addq.

⟨ *X86: render* 552a ⟩ + ≡

```
// -- kcc_x86_memload(ir_instr *i, const char *
   mnem, const char *tail)
//    Emit "mov* (%r11), reg" for a pointer
   already implied by SRC1, storing to
//    DEST.  Loads the pointer into %r11 first.
.p2align 2
_kcc_x86_memload:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                         // instr
    mov x20, x1                         // mnemonic
    mov x21, x2                         // tail  "(%
   r11), %r10\n" etc.
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_r11@PAGE
    add  x1, x1, _kcc_x86_s_r11@PAGEOFF
    bl _kcc_x86_load
    mov x0, x20
    bl _kcc_as_str
    mov x0, x21
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_store
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_x86_memstore(ir_instr *i, const char *
   mnem, const char *valtail)
//    Emit "mov* reg, (%r11)".  Loads value into %
   r10, pointer into %r11.
.p2align 2
_kcc_x86_memstore:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0
    mov x20, x1                         // mnemonic
    mov x21, x2                         // valtail  "%
   r10, (%r11)\n" etc.
    ldr x0, [x19, #IR_SRC1]             // value
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_load
    ldr x0, [x19, #IR_SRC2]             // pointer
    adrp x1, _kcc_x86_s_r11@PAGE
    add  x1, x1, _kcc_x86_s_r11@PAGEOFF
    bl _kcc_x86_load
    mov x0, x20
    bl _kcc_as_str
    mov x0, x21
    bl _kcc_as_str
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

552a · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a

38.5.5. Comparison, Label, and Call Helpers (Rung 3)

Three more shape helpers cover control flow. _kcc_x86_cmp emits a comparison: it loads both operands, compares them at the width recorded in the instruction's type (32-bit cmpl for an int-or-narrower common type, else 64-bit cmpq), materialises the boolean with the caller's chosen set*cc* into %al, widens it into %r10 with movzbl, and stores it. _kcc_x86_bblabel appends a basic-block label reference .LkccbbN (used both to define a block's label and to name a branch target). _kcc_x86_marshal realises the System V argument-passing rules for the integer class: it reads the argument array ([count | fsi<<32, vreg0, class0, …]), places arguments seven and beyond into a freshly subtracted stack region at (k-6)*8(%rsp), loads the first six into %rdi–%r9, and returns the stack_adj the caller adds back after the call. Floating-point and by-value aggregate arguments are a later rung; rung three treats every argument as integer/pointer.

⟨ *X86: render* 552b ⟩ + ≡

```
// -- kcc_x86_cmp(ir_instr *i, const char *setcc)
   - dst = (src1 CMP src2) ? 1:0.
.p2align 2
_kcc_x86_cmp:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                         // instr
    mov x20, x1                         // set<cc>
   string
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_load
    ldr x0, [x19, #IR_SRC2]
    adrp x1, _kcc_x86_s_r11@PAGE
    add  x1, x1, _kcc_x86_s_r11@PAGEOFF
    bl _kcc_x86_load
    // Compare width: <=4-byte common type → 32-
   bit cmpl, else 64-bit cmpq.
    ldr x0, [x19, #IR_TYPE]
    cbz x0, .Lx86cmp_q
    ldr x0, [x0, #TY_SIZE]
    cmp x0, #8
    b.eq .Lx86cmp_q
    adrp x0, _kcc_x86_s_cmpl@PAGE
    add  x0, x0, _kcc_x86_s_cmpl@PAGEOFF
    b .Lx86cmp_emit
.Lx86cmp_q:
    adrp x0, _kcc_x86_s_cmpq@PAGE
    add  x0, x0, _kcc_x86_s_cmpq@PAGEOFF
.Lx86cmp_emit:
    bl _kcc_as_str
    mov x0, x20                         // set<cc> %al
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_movzbl_al@PAGE
    add  x0, x0, _kcc_x86_s_movzbl_al@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_store
```

```
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_x86_bblabel(uint64_t bb_id) - append ".
   Lkccbb<id>".
.p2align 2
_kcc_x86_bblabel:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0
    adrp x0, _kcc_x86_s_bb@PAGE
    add  x0, x0, _kcc_x86_s_bb@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_as_dec
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_x86_marshal(uint64_t *args, uint64_t
   count) → stack_adj.
//    Loads integer/pointer args into %rdi..%r9
   and args 7+ onto the stack.
.p2align 2
_kcc_x86_marshal:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                         // args array
    mov x20, x1                         // count
    mov x22, #0                         // stack_adj (
   return value)
    cbz x20, .Lx86mar_done

    // Stack phase: if count > 6, reserve align16
   ((count-6)*8) and store the
    // overflow args (index 6..count-1) at (k-6)
   *8(%rsp).
    cmp x20, #6
    b.le .Lx86mar_regs
    sub x0, x20, #6
    lsl x0, x0, #3
    add x0, x0, #15
    and x22, x0, #~15                   // stack_adj
    adrp x0, _kcc_x86_s_sub_rsp@PAGE
    add  x0, x0, _kcc_x86_s_sub_rsp@PAGEOFF
    bl _kcc_as_str
    mov x0, x22
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_rsp@PAGE
    add  x0, x0, _kcc_x86_s_rsp@PAGEOFF
    bl _kcc_as_str
    mov x21, #6                         // k = 6
.Lx86mar_stack_loop:
    cmp x21, x20
    b.ge .Lx86mar_regs
    // movq slot(vreg_k), %r10
    lsl x0, x21, #1
    add x0, x0, #1                      // 1 + 2k →
   vreg field
    ldr x0, [x19, x0, lsl #3]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_load
    // movq %r10, (k-6)*8(%rsp)
    adrp x0, _kcc_x86_s_movq@PAGE
    add  x0, x0, _kcc_x86_s_movq@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_r10@PAGE
    add  x0, x0, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_comma@PAGE
    add  x0, x0, _kcc_x86_s_comma@PAGEOFF
    bl _kcc_as_str
    sub x0, x21, #6
    lsl x0, x0, #3                      // (k-6)*8
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_paren_rsp@PAGE
    add  x0, x0, _kcc_x86_s_paren_rsp@PAGEOFF
    bl _kcc_as_str
    mov x0, #10
    bl _kcc_as_ch
    add x21, x21, #1
    b .Lx86mar_stack_loop

.Lx86mar_regs:
    // Register phase: args 0..min(count,6)-1 into
    %rdi..%r9.
    mov x21, #0
.Lx86mar_reg_loop:
    cmp x21, x20
    b.ge .Lx86mar_done
    cmp x21, #6
    b.ge .Lx86mar_done
    lsl x0, x21, #1
    add x0, x0, #1                      // 1 + 2k →
   vreg field
    ldr x0, [x19, x0, lsl #3]           // vreg
    adrp x1, _kcc_x86_argregs@PAGE
    add  x1, x1, _kcc_x86_argregs@PAGEOFF
    ldr x1, [x1, x21, lsl #3]           // argreg[k]
   string ptr
    bl _kcc_x86_load
    add x21, x21, #1
    b .Lx86mar_reg_loop

.Lx86mar_done:
    mov x0, x22                         // stack_adj
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_x86_call_args(uint64_t *args) →
   stack_adj.  Reads the count from the
//    argument array header and marshals; a null
   array is a zero-argument call.
.p2align 2
_kcc_x86_call_args:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    cbz x0, .Lx86ca_none
    ldr x1, [x0]                        // word0 =
   count | (fsi << 32)
    and x1, x1, #0xffffffff             // count
    bl _kcc_x86_marshal                 // marshal(
   args=x0, count=x1) → x0 = stack_adj
    b .Lx86ca_ret
.Lx86ca_none:
    mov x0, #0
.Lx86ca_ret:
    ldp x29, x30, [sp], #16
    ret
```

□

552b · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a

38.5.6. The Dispatch

With the shape helpers in hand, `_kcc_x86_sel` is a flat opcode switch. `IR_CONST` materialises its immediate with `movabsq` (the one x86 move that takes a full 64-bit immediate) into `%r10` and stores it.

IR_ALLOCA computes the *address* of its storage—-(vreg_n + offset + size)*8(%rbp), where the prescan left the running slot offset in SRC2 and SRC1 is the object's slot count—with a leaq and stores that pointer. IR_RET loads the returned vreg into %rax (the low 32 bits are the int result the process exit status reads) and falls into the leave/retq epilogue; a void return skips straight to it. Everything else routes to a shape helper.

An opcode the back end has *not* yet learned—a floating-point operation, inline assembly, a by-value aggregate—does not fall through silently. A genuine IR_NOP emits nothing and is skipped, but any other unrecognised opcode makes selection print a diagnostic and exit non-zero. This is a deliberate choice: emitting nothing for, say, an IR_FMUL would leave its destination slot holding garbage and *miscompile* the program with no warning—the worst possible failure. Refusing cleanly instead keeps a program that uses an unimplemented feature (a double-heavy interpreter like Lua) an honest compile-time error rather than a wrong answer at run time, matching the front end's treatment of VLAs and _Complex. As each later rung teaches the back end a new opcode, the set that trips the refusal shrinks.

⟨ *X86: render* 554 ⟩ + ≡

```
// -- kcc_x86_sel(ir_instr *i) - select x86-64 for
    one IR instruction.
.p2align 2
_kcc_x86_sel:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                          // instr
    ldr x22, [x19, #IR_OP]

    cmp x22, #IR_CONST
    b.eq .Lx86sel_const
    cmp x22, #IR_RET
    b.eq .Lx86sel_ret
    cmp x22, #IR_ALLOCA
    b.eq .Lx86sel_alloca
    cmp x22, #IR_LOAD
    b.eq .Lx86sel_load
    cmp x22, #IR_STORE
    b.eq .Lx86sel_store

    // Two-operand ALU: route by opcode to a
   mnemonic, then kcc_x86_binop.
    // IR_GEP is address arithmetic (base + byte
   offset), i.e. a plain addq.
    cmp x22, #IR_ADD
    b.eq .Lx86sel_add
    cmp x22, #IR_GEP
    b.eq .Lx86sel_add
    cmp x22, #IR_SUB
    b.eq .Lx86sel_sub
    cmp x22, #IR_MUL
    b.eq .Lx86sel_mul
    cmp x22, #IR_AND
    b.eq .Lx86sel_and
    cmp x22, #IR_OR
    b.eq .Lx86sel_or
    cmp x22, #IR_XOR
    b.eq .Lx86sel_xor
    cmp x22, #IR_NEG
    b.eq .Lx86sel_neg
    cmp x22, #IR_NOT
    b.eq .Lx86sel_not
    cmp x22, #IR_SHL
    b.eq .Lx86sel_shl
    cmp x22, #IR_LSHR
    b.eq .Lx86sel_lshr
    cmp x22, #IR_ASHR
    b.eq .Lx86sel_ashr
    cmp x22, #IR_SDIV
    b.eq .Lx86sel_sdiv
    cmp x22, #IR_UDIV
    b.eq .Lx86sel_udiv
    cmp x22, #IR_SMOD
    b.eq .Lx86sel_smod
    cmp x22, #IR_UMOD
    b.eq .Lx86sel_umod
    cmp x22, #IR_SEXT
    b.eq .Lx86sel_sext
    cmp x22, #IR_ZEXT
    b.eq .Lx86sel_zext
    cmp x22, #IR_TRUNC
    b.eq .Lx86sel_trunc
    cmp x22, #IR_BITCAST
    b.eq .Lx86sel_bitcast

    // Floating-point conversions (rung 4).
   UITOFP shares the signed cvtsi2sd
    // path: kcc's own source only ever converts
   signed integers to floating.
    cmp x22, #IR_SITOFP
    b.eq .Lx86sel_sitofp
    cmp x22, #IR_UITOFP
    b.eq .Lx86sel_sitofp
    cmp x22, #IR_FPTOSI
    b.eq .Lx86sel_fptosi
    cmp x22, #IR_FPROUND
    b.eq .Lx86sel_fpround

    // Comparisons (rung 3): cmp + set<cc> +
   movzbl.
    cmp x22, #IR_CMP_EQ
    b.eq .Lx86sel_cmpeq
    cmp x22, #IR_CMP_NE
    b.eq .Lx86sel_cmpne
    cmp x22, #IR_CMP_SLT
    b.eq .Lx86sel_cmpslt
    cmp x22, #IR_CMP_SLE
    b.eq .Lx86sel_cmpsle
    cmp x22, #IR_CMP_SGT
    b.eq .Lx86sel_cmpsgt
    cmp x22, #IR_CMP_SGE
    b.eq .Lx86sel_cmpsge
    cmp x22, #IR_CMP_ULT
    b.eq .Lx86sel_cmpult
    cmp x22, #IR_CMP_ULE
    b.eq .Lx86sel_cmpule

    // Control flow and calls (rung 3).
    cmp x22, #IR_BR
    b.eq .Lx86sel_br
    cmp x22, #IR_BR_COND
    b.eq .Lx86sel_brcond
    cmp x22, #IR_CALL
    b.eq .Lx86sel_call
    cmp x22, #IR_CALL_IND
    b.eq .Lx86sel_callind
    cmp x22, #IR_GLOBAL_REF
    b.eq .Lx86sel_globalref
    cmp x22, #IR_STRING_LIT
    b.eq .Lx86sel_strlit

    // A genuine no-op emits nothing and is safe
   to skip.
    cmp x22, #IR_NOP
    b.eq .Lx86sel_done

    // Any other opcode is a feature this back end
    does not yet implement
    // (floating point, inline asm, by-value
```

```
    aggregates, ...).  Silently
     // skipping it would emit a value-less slot
    and miscompile the program, so
     // refuse cleanly instead---the same
    discipline the front end applies to
     // VLAs and _Complex.  This is what keeps a
    double-using program (Lua) an
     // honest compile-time error rather than a
    wrong answer at run time.
     adrp x0, _kcc_x86_s_unsup@PAGE
     add  x0, x0, _kcc_x86_s_unsup@PAGEOFF
     bl _kcc_io_err
     bl _kcc_io_flush
     mov x0, #1
     bl _kcc_sys_exit

.Lx86sel_const:
     adrp x0, _kcc_x86_s_movabsq@PAGE
     add  x0, x0, _kcc_x86_s_movabsq@PAGEOFF
     bl _kcc_as_str
     ldr x0, [x19, #IR_SRC1]
     bl _kcc_as_dec
     adrp x0, _kcc_x86_s_c_r10@PAGE
     add  x0, x0, _kcc_x86_s_c_r10@PAGEOFF
     bl _kcc_as_str
     ldr x0, [x19, #IR_DEST]
     adrp x1, _kcc_x86_s_r10@PAGE
     add  x1, x1, _kcc_x86_s_r10@PAGEOFF
     bl _kcc_x86_store
     b .Lx86sel_done

.Lx86sel_alloca:
     // D = (vreg_n + offset + size) * 8.
     ldr x20, [x19, #IR_SRC2]          // offset (
    slots, from pre-scan)
     ldr x21, [x19, #IR_SRC1]          // size (slots
    )
     cbnz x21, .Lx86sel_alloca_sz
     mov x21, #1
.Lx86sel_alloca_sz:
     adrp x0, _kcc_x86_vreg_n@PAGE
     add  x0, x0, _kcc_x86_vreg_n@PAGEOFF
     ldr x0, [x0]
     add x0, x0, x20
     add x0, x0, x21
     lsl x20, x0, #3                   // D (bytes)
     adrp x0, _kcc_x86_s_leaq@PAGE
     add  x0, x0, _kcc_x86_s_leaq@PAGEOFF
     bl _kcc_as_str
     mov x0, x20
     bl _kcc_x86_disp
     adrp x0, _kcc_x86_s_c_r10@PAGE
     add  x0, x0, _kcc_x86_s_c_r10@PAGEOFF
     bl _kcc_as_str
     ldr x0, [x19, #IR_DEST]
     adrp x1, _kcc_x86_s_r10@PAGE
     add  x1, x1, _kcc_x86_s_r10@PAGEOFF
     bl _kcc_x86_store
     b .Lx86sel_done

.Lx86sel_ret:
     ldr x0, [x19, #IR_TYPE]
     cbz x0, .Lx86sel_ret_epi          // null type
    → void
     ldr x1, [x0, #TY_KIND]
     cmp x1, #TY_VOID
     b.eq .Lx86sel_ret_epi
     ldr x0, [x19, #IR_SRC1]
     adrp x1, _kcc_x86_s_rax@PAGE
     add  x1, x1, _kcc_x86_s_rax@PAGEOFF
     bl _kcc_x86_load
.Lx86sel_ret_epi:
     adrp x0, _kcc_x86_s_leave@PAGE
     add  x0, x0, _kcc_x86_s_leave@PAGEOFF
     bl _kcc_as_str
     adrp x0, _kcc_x86_s_retq@PAGE
     add  x0, x0, _kcc_x86_s_retq@PAGEOFF
     bl _kcc_as_str
     b .Lx86sel_done

.Lx86sel_load:
     // Access-size override in SRC2 selects the
    extending move.
     ldr x20, [x19, #IR_SRC2]
     cmp x20, #1
     b.eq .Lx86sel_load_u8
     cmp x20, #2
     b.eq .Lx86sel_load_u16
     cmp x20, #4
     b.eq .Lx86sel_load_u32
     cmp x20, #5
     b.eq .Lx86sel_load_s8
     cmp x20, #6
     b.eq .Lx86sel_load_s16
     cmp x20, #7
     b.eq .Lx86sel_load_s32
     cmp x20, #8
     b.eq .Lx86sel_load_f32            // single-
    precision float: movss + widen
     // Default: 64-bit movq.
     mov x0, x19
     adrp x1, _kcc_x86_s_movq@PAGE
     add  x1, x1, _kcc_x86_s_movq@PAGEOFF
     adrp x2, _kcc_x86_s_pr11_r10@PAGE
     add  x2, x2, _kcc_x86_s_pr11_r10@PAGEOFF
     bl _kcc_x86_memload
     b .Lx86sel_done
.Lx86sel_load_u8:
     mov x0, x19
     adrp x1, _kcc_x86_s_movzbq@PAGE
     add  x1, x1, _kcc_x86_s_movzbq@PAGEOFF
     adrp x2, _kcc_x86_s_pr11_r10@PAGE
     add  x2, x2, _kcc_x86_s_pr11_r10@PAGEOFF
     bl _kcc_x86_memload
     b .Lx86sel_done
.Lx86sel_load_u16:
     mov x0, x19
     adrp x1, _kcc_x86_s_movzwq@PAGE
     add  x1, x1, _kcc_x86_s_movzwq@PAGEOFF
     adrp x2, _kcc_x86_s_pr11_r10@PAGE
     add  x2, x2, _kcc_x86_s_pr11_r10@PAGEOFF
     bl _kcc_x86_memload
     b .Lx86sel_done
.Lx86sel_load_u32:
     mov x0, x19
     adrp x1, _kcc_x86_s_movl@PAGE
     add  x1, x1, _kcc_x86_s_movl@PAGEOFF
     adrp x2, _kcc_x86_s_pr11_r10d@PAGE
     add  x2, x2, _kcc_x86_s_pr11_r10d@PAGEOFF
     bl _kcc_x86_memload
     b .Lx86sel_done
.Lx86sel_load_s8:
     mov x0, x19
     adrp x1, _kcc_x86_s_movsbq@PAGE
     add  x1, x1, _kcc_x86_s_movsbq@PAGEOFF
     adrp x2, _kcc_x86_s_pr11_r10@PAGE
     add  x2, x2, _kcc_x86_s_pr11_r10@PAGEOFF
     bl _kcc_x86_memload
     b .Lx86sel_done
.Lx86sel_load_s16:
     mov x0, x19
     adrp x1, _kcc_x86_s_movswq@PAGE
     add  x1, x1, _kcc_x86_s_movswq@PAGEOFF
     adrp x2, _kcc_x86_s_pr11_r10@PAGE
     add  x2, x2, _kcc_x86_s_pr11_r10@PAGEOFF
     bl _kcc_x86_memload
     b .Lx86sel_done
.Lx86sel_load_s32:
     mov x0, x19
     adrp x1, _kcc_x86_s_movslq@PAGE
     add  x1, x1, _kcc_x86_s_movslq@PAGEOFF
     adrp x2, _kcc_x86_s_pr11_r10@PAGE
```

```
    add  x2, x2, _kcc_x86_s_pr11_r10@PAGEOFF
    bl _kcc_x86_memload
    b .Lx86sel_done

.Lx86sel_store:
    // Access-size override in DEST selects the
   store width.
    ldr x20, [x19, #IR_DEST]
    cmp x20, #1
    b.eq .Lx86sel_store_8
    cmp x20, #2
    b.eq .Lx86sel_store_16
    cmp x20, #4
    b.eq .Lx86sel_store_32
    cmp x20, #8
    b.eq .Lx86sel_store_f32          // single-
   precision float: narrow + movss
    // Default: 64-bit movq.
    mov x0, x19
    adrp x1, _kcc_x86_s_movq@PAGE
    add  x1, x1, _kcc_x86_s_movq@PAGEOFF
    adrp x2, _kcc_x86_s_r10_pr11@PAGE
    add  x2, x2, _kcc_x86_s_r10_pr11@PAGEOFF
    bl _kcc_x86_memstore
    b .Lx86sel_done
.Lx86sel_store_8:
    mov x0, x19
    adrp x1, _kcc_x86_s_movb@PAGE
    add  x1, x1, _kcc_x86_s_movb@PAGEOFF
    adrp x2, _kcc_x86_s_r10b_pr11@PAGE
    add  x2, x2, _kcc_x86_s_r10b_pr11@PAGEOFF
    bl _kcc_x86_memstore
    b .Lx86sel_done
.Lx86sel_store_16:
    mov x0, x19
    adrp x1, _kcc_x86_s_movw@PAGE
    add  x1, x1, _kcc_x86_s_movw@PAGEOFF
    adrp x2, _kcc_x86_s_r10w_pr11@PAGE
    add  x2, x2, _kcc_x86_s_r10w_pr11@PAGEOFF
    bl _kcc_x86_memstore
    b .Lx86sel_done
.Lx86sel_store_32:
    mov x0, x19
    adrp x1, _kcc_x86_s_movl@PAGE
    add  x1, x1, _kcc_x86_s_movl@PAGEOFF
    adrp x2, _kcc_x86_s_r10d_pr11@PAGE
    add  x2, x2, _kcc_x86_s_r10d_pr11@PAGEOFF
    bl _kcc_x86_memstore
    b .Lx86sel_done
.Lx86sel_load_f32:
    // float in memory → double in the slot: load
   ptr into %r11, movss the
    // single, widen with cvtss2sd, store the
   double.
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_r11@PAGE
    add  x1, x1, _kcc_x86_s_r11@PAGEOFF
    bl _kcc_x86_load
    adrp x0, _kcc_x86_s_movss@PAGE
    add  x0, x0, _kcc_x86_s_movss@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_px11_x0@PAGE
    add  x0, x0, _kcc_x86_s_px11_x0@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_cvtss2sd@PAGE
    add  x0, x0, _kcc_x86_s_cvtss2sd@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    bl _kcc_x86_fpstore
    b .Lx86sel_done
.Lx86sel_store_f32:
    // double in the slot → float in memory: load
   the double into %xmm0, narrow
    // with cvtsd2ss, load the address into %r11,
   movss the single.
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_cx0_nl@PAGE
    add  x1, x1, _kcc_x86_s_cx0_nl@PAGEOFF
    bl _kcc_x86_fpload
    adrp x0, _kcc_x86_s_cvtsd2ss@PAGE
    add  x0, x0, _kcc_x86_s_cvtsd2ss@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_SRC2]
    adrp x1, _kcc_x86_s_r11@PAGE
    add  x1, x1, _kcc_x86_s_r11@PAGEOFF
    bl _kcc_x86_load
    adrp x0, _kcc_x86_s_movss@PAGE
    add  x0, x0, _kcc_x86_s_movss@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_x0_px11@PAGE
    add  x0, x0, _kcc_x86_s_x0_px11@PAGEOFF
    bl _kcc_as_str
    b .Lx86sel_done

// --- Two-operand ALU tails: load mnemonic, call
   the binop shape. ---
.Lx86sel_add:
    // A floating ADD/SUB/MUL/DIV (typed float/
   double) uses the SSE unit; the
    // shared IR_GEP that also lands here is
   integer address arithmetic, so its
    // int type keeps it on the integer path.
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_type_is_fp
    cbz x0, .Lx86sel_add_i
    mov x0, x19
    adrp x1, _kcc_x86_s_addsd@PAGE
    add  x1, x1, _kcc_x86_s_addsd@PAGEOFF
    bl _kcc_x86_fpbinop
    b .Lx86sel_done
.Lx86sel_add_i:
    mov x0, x19
    adrp x1, _kcc_x86_s_addq@PAGE
    add  x1, x1, _kcc_x86_s_addq@PAGEOFF
    bl _kcc_x86_binop
    b .Lx86sel_done
.Lx86sel_sub:
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_type_is_fp
    cbz x0, .Lx86sel_sub_i
    mov x0, x19
    adrp x1, _kcc_x86_s_subsd@PAGE
    add  x1, x1, _kcc_x86_s_subsd@PAGEOFF
    bl _kcc_x86_fpbinop
    b .Lx86sel_done
.Lx86sel_sub_i:
    mov x0, x19
    adrp x1, _kcc_x86_s_subq@PAGE
    add  x1, x1, _kcc_x86_s_subq@PAGEOFF
    bl _kcc_x86_binop
    b .Lx86sel_done
.Lx86sel_mul:
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_type_is_fp
    cbz x0, .Lx86sel_mul_i
    mov x0, x19
    adrp x1, _kcc_x86_s_mulsd@PAGE
    add  x1, x1, _kcc_x86_s_mulsd@PAGEOFF
    bl _kcc_x86_fpbinop
    b .Lx86sel_done
.Lx86sel_mul_i:
    mov x0, x19
    adrp x1, _kcc_x86_s_imulq@PAGE
    add  x1, x1, _kcc_x86_s_imulq@PAGEOFF
    bl _kcc_x86_binop
    b .Lx86sel_done
.Lx86sel_and:
    mov x0, x19
    adrp x1, _kcc_x86_s_andq@PAGE
    add  x1, x1, _kcc_x86_s_andq@PAGEOFF
    bl _kcc_x86_binop
    b .Lx86sel_done
```

```
.Lx86sel_or:
    mov x0, x19
    adrp x1, _kcc_x86_s_orq@PAGE
    add  x1, x1, _kcc_x86_s_orq@PAGEOFF
    bl _kcc_x86_binop
    b .Lx86sel_done
.Lx86sel_xor:
    mov x0, x19
    adrp x1, _kcc_x86_s_xorq@PAGE
    add  x1, x1, _kcc_x86_s_xorq@PAGEOFF
    bl _kcc_x86_binop
    b .Lx86sel_done
.Lx86sel_neg:
    mov x0, x19
    adrp x1, _kcc_x86_s_negq@PAGE
    add  x1, x1, _kcc_x86_s_negq@PAGEOFF
    bl _kcc_x86_unop
    b .Lx86sel_done
.Lx86sel_not:
    mov x0, x19
    adrp x1, _kcc_x86_s_notq@PAGE
    add  x1, x1, _kcc_x86_s_notq@PAGEOFF
    bl _kcc_x86_unop
    b .Lx86sel_done
.Lx86sel_shl:
    mov x0, x19
    adrp x1, _kcc_x86_s_shlq@PAGE
    add  x1, x1, _kcc_x86_s_shlq@PAGEOFF
    bl _kcc_x86_shift
    b .Lx86sel_done
.Lx86sel_lshr:
    mov x0, x19
    adrp x1, _kcc_x86_s_shrq@PAGE
    add  x1, x1, _kcc_x86_s_shrq@PAGEOFF
    bl _kcc_x86_shift
    b .Lx86sel_done
.Lx86sel_ashr:
    mov x0, x19
    adrp x1, _kcc_x86_s_sarq@PAGE
    add  x1, x1, _kcc_x86_s_sarq@PAGEOFF
    bl _kcc_x86_shift
    b .Lx86sel_done
.Lx86sel_sdiv:
    ldr x0, [x19, #IR_TYPE]
    bl _kcc_ir_type_is_fp
    cbz x0, .Lx86sel_sdiv_i
    mov x0, x19
    adrp x1, _kcc_x86_s_divsd@PAGE
    add  x1, x1, _kcc_x86_s_divsd@PAGEOFF
    bl _kcc_x86_fpbinop
    b .Lx86sel_done
.Lx86sel_sdiv_i:
    mov x0, x19
    mov x1, #1                          // signed
    mov x2, #0                          // quotient
    bl _kcc_x86_divop
    b .Lx86sel_done

// --- Floating-point conversions (rung 4). ---
.Lx86sel_sitofp:
    // int slot → double: cvtsi2sdq %r10, %xmm0 ;
   movsd %xmm0, slot(dest).
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_load
    adrp x0, _kcc_x86_s_cvtsi2sd@PAGE
    add  x0, x0, _kcc_x86_s_cvtsi2sd@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    bl _kcc_x86_fpstore
    b .Lx86sel_done
.Lx86sel_fptosi:
    // double slot → int: movsd slot(src),%xmm0 ;
   cvttsd2si %xmm0,%r10 ; store.
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_cx0_nl@PAGE
    add  x1, x1, _kcc_x86_s_cx0_nl@PAGEOFF
    bl _kcc_x86_fpload
    adrp x0, _kcc_x86_s_cvttsd2si@PAGE
    add  x0, x0, _kcc_x86_s_cvttsd2si@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_store
    b .Lx86sel_done
.Lx86sel_fpround:
    // round a double to single precision, keeping
    it in double format:
    // movsd slot(src),%xmm0 ; cvtsd2ss ; cvtss2sd
    ; movsd %xmm0, slot(dest).
    ldr x0, [x19, #IR_SRC1]
    adrp x1, _kcc_x86_s_cx0_nl@PAGE
    add  x1, x1, _kcc_x86_s_cx0_nl@PAGEOFF
    bl _kcc_x86_fpload
    adrp x0, _kcc_x86_s_cvtsd2ss@PAGE
    add  x0, x0, _kcc_x86_s_cvtsd2ss@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_cvtss2sd@PAGE
    add  x0, x0, _kcc_x86_s_cvtss2sd@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    bl _kcc_x86_fpstore
    b .Lx86sel_done
.Lx86sel_udiv:
    mov x0, x19
    mov x1, #0
    mov x2, #0
    bl _kcc_x86_divop
    b .Lx86sel_done
.Lx86sel_smod:
    mov x0, x19
    mov x1, #1
    mov x2, #1                          // remainder
    bl _kcc_x86_divop
    b .Lx86sel_done
.Lx86sel_umod:
    mov x0, x19
    mov x1, #0
    mov x2, #1
    bl _kcc_x86_divop
    b .Lx86sel_done
.Lx86sel_sext:
    mov x0, x19
    ldr x1, [x19, #IR_SRC2]             // source
   width
    mov x2, #1                          // signed
    bl _kcc_x86_conv
    b .Lx86sel_done
.Lx86sel_zext:
    mov x0, x19
    ldr x1, [x19, #IR_SRC2]             // source
   width
    mov x2, #0                          // unsigned
    bl _kcc_x86_conv
    b .Lx86sel_done
.Lx86sel_trunc:
    mov x0, x19
    ldr x1, [x19, #IR_TYPE]             // target type
    → width
    cbz x1, .Lx86sel_trunc_copy
    ldr x1, [x1, #TY_SIZE]
    mov x2, #0                          // truncation
   keeps low bytes, zero-extended
    bl _kcc_x86_conv
    b .Lx86sel_done
.Lx86sel_trunc_copy:
    mov x1, #8                          // unknown
   width → plain copy
    mov x2, #0
    bl _kcc_x86_conv
```

```
    b .Lx86sel_done
.Lx86sel_bitcast:
    mov x0, x19
    mov x1, #8                          // plain movq
   copy
    mov x2, #0
    bl _kcc_x86_conv
    b .Lx86sel_done

// --- Comparisons: load the set<cc> mnemonic,
   call the cmp shape. ---
.Lx86sel_cmpeq:
    adrp x1, _kcc_x86_s_sete@PAGE
    add  x1, x1, _kcc_x86_s_sete@PAGEOFF
    b .Lx86sel_cmp_go
.Lx86sel_cmpne:
    adrp x1, _kcc_x86_s_setne@PAGE
    add  x1, x1, _kcc_x86_s_setne@PAGEOFF
    b .Lx86sel_cmp_go
.Lx86sel_cmpslt:
    adrp x1, _kcc_x86_s_setl@PAGE
    add  x1, x1, _kcc_x86_s_setl@PAGEOFF
    b .Lx86sel_cmp_go
.Lx86sel_cmpsle:
    adrp x1, _kcc_x86_s_setle@PAGE
    add  x1, x1, _kcc_x86_s_setle@PAGEOFF
    b .Lx86sel_cmp_go
.Lx86sel_cmpsgt:
    adrp x1, _kcc_x86_s_setg@PAGE
    add  x1, x1, _kcc_x86_s_setg@PAGEOFF
    b .Lx86sel_cmp_go
.Lx86sel_cmpsge:
    adrp x1, _kcc_x86_s_setge@PAGE
    add  x1, x1, _kcc_x86_s_setge@PAGEOFF
    b .Lx86sel_cmp_go
.Lx86sel_cmpult:
    adrp x1, _kcc_x86_s_setb@PAGE
    add  x1, x1, _kcc_x86_s_setb@PAGEOFF
    b .Lx86sel_cmp_go
.Lx86sel_cmpule:
    adrp x1, _kcc_x86_s_setbe@PAGE
    add  x1, x1, _kcc_x86_s_setbe@PAGEOFF
.Lx86sel_cmp_go:
    mov x0, x19
    bl _kcc_x86_cmp
    b .Lx86sel_done

// --- Unconditional branch: jmp .Lkccbb<target>.
   ---
.Lx86sel_br:
    adrp x0, _kcc_x86_s_jmp@PAGE
    add  x0, x0, _kcc_x86_s_jmp@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_SRC1]             // target BB
   id
    bl _kcc_x86_bblabel
    mov x0, #10
    bl _kcc_as_ch
    b .Lx86sel_done

// --- Conditional branch: test cond, jne true,
   jmp false. ---
.Lx86sel_brcond:
    ldr x0, [x19, #IR_DEST]             // condition
   vreg
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_load
    adrp x0, _kcc_x86_s_test_r10@PAGE
    add  x0, x0, _kcc_x86_s_test_r10@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_jne@PAGE
    add  x0, x0, _kcc_x86_s_jne@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_SRC1]             // true BB id
    bl _kcc_x86_bblabel
    mov x0, #10
    bl _kcc_as_ch
    adrp x0, _kcc_x86_s_jmp@PAGE
    add  x0, x0, _kcc_x86_s_jmp@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_SRC2]             // false BB id
    bl _kcc_x86_bblabel
    mov x0, #10
    bl _kcc_as_ch
    b .Lx86sel_done

// --- Global reference: the symbol's address into
    %r10, stored to dest. ---
.Lx86sel_globalref:
    // Darwin: movq _<name>@GOTPCREL(%rip), %r10
   --- the GOT form resolves
    // uniformly for a locally defined symbol and
   for a dylib import (like puts),
    // which has no fixed image address, exactly
   the ARM64 GOTPAGE discipline.
    // Linux/ELF (static): leaq <name>(%rip), %r10
    --- a statically linked
    // executable resolves every symbol to a real
   address at link time, so the
    // direct RIP form suffices and no GOT slot is
    needed.
    adrp x0, _kcc_x86_os@PAGE
    add  x0, x0, _kcc_x86_os@PAGEOFF
    ldr x0, [x0]
    cbnz x0, .Lx86gref_elf
    // Darwin GOT form.
    adrp x0, _kcc_x86_s_movq@PAGE
    add  x0, x0, _kcc_x86_s_movq@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_SRC1]             // symbol name
    bl _kcc_x86_sym                     // "_"+name on
    Darwin
    adrp x0, _kcc_x86_s_got_r10@PAGE
    add  x0, x0, _kcc_x86_s_got_r10@PAGEOFF
    bl _kcc_as_str
    b .Lx86gref_store
.Lx86gref_elf:
    // Linux direct form: leaq <name>(%rip), %r10.
    adrp x0, _kcc_x86_s_leaq@PAGE
    add  x0, x0, _kcc_x86_s_leaq@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_SRC1]
    bl _kcc_x86_sym                     // bare name
   on Linux
    adrp x0, _kcc_x86_s_rip_r10@PAGE
    add  x0, x0, _kcc_x86_s_rip_r10@PAGEOFF
    bl _kcc_as_str
.Lx86gref_store:
    ldr x0, [x19, #IR_DEST]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_store
    b .Lx86sel_done

// --- String literal: record (bytes,len), leaq .
   Lx86str<id>(%rip), %r10. ---
.Lx86sel_strlit:
    // id = current count of the string table;
   record {SRC1, SRC2}.
    adrp x0, _kcc_x86_strs@PAGE
    add  x0, x0, _kcc_x86_strs@PAGEOFF
    ldr x0, [x0]
    ldr x20, [x0, #VEC_COUNT]           // id =
   current entry count
    ldr x1, [x19, #IR_SRC1]             // data ptr
    ldr x2, [x19, #IR_SRC2]             // length
    stp x1, x2, [sp, #-16]!             // build the {
   ptr, len} entry
    mov x1, sp
    bl _kcc_vec_push                    // push(strs,
```

```
    &entry)
    add sp, sp, #16
    // leaq .Lx86str<id>(%rip), %r10
    adrp x0, _kcc_x86_s_leaq@PAGE
    add  x0, x0, _kcc_x86_s_leaq@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_strlbl@PAGE
    add  x0, x0, _kcc_x86_s_strlbl@PAGEOFF
    bl _kcc_as_str
    mov x0, x20                         // id
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_rip_r10@PAGE
    add  x0, x0, _kcc_x86_s_rip_r10@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_DEST]
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_store
    b .Lx86sel_done

// --- Direct call: marshal args, call _<name>,
   store %rax. ---
.Lx86sel_call:
    ldr x0, [x19, #IR_SRC2]             // args array
   (0 if none)
    bl _kcc_x86_call_args               // loads args
   into ABI regs/stack
    mov x20, x0                         // x20 =
   stack_adj (survives the call sequence)
    // movb $0, %al   (variadic safety: no FP args
    in rung 3)
    adrp x0, _kcc_x86_s_zero_al@PAGE
    add  x0, x0, _kcc_x86_s_zero_al@PAGEOFF
    bl _kcc_as_str
    // call <name>  (underscore-prefixed on Darwin
    via kcc_x86_sym)
    adrp x0, _kcc_x86_s_call@PAGE
    add  x0, x0, _kcc_x86_s_call@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x19, #IR_SRC1]             // callee name
    bl _kcc_x86_sym
    mov x0, #10
    bl _kcc_as_ch
    b .Lx86sel_call_tail

// --- Indirect call: marshal args, call *%r11,
   store %rax. ---
.Lx86sel_callind:
    ldr x0, [x19, #IR_SRC2]             // args array
    bl _kcc_x86_call_args               // loads args
   into ABI regs/stack
    mov x20, x0                         // x20 =
   stack_adj
    // Load the function pointer into %r11 (arg
   regs already set; r11 is free).
    ldr x0, [x19, #IR_SRC1]             // fn-ptr vreg
    adrp x1, _kcc_x86_s_r11@PAGE
    add  x1, x1, _kcc_x86_s_r11@PAGEOFF
    bl _kcc_x86_load
    adrp x0, _kcc_x86_s_zero_al@PAGE
    add  x0, x0, _kcc_x86_s_zero_al@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_call_r11@PAGE
    add  x0, x0, _kcc_x86_s_call_r11@PAGEOFF
    bl _kcc_as_str
    // fall through to the shared return/stack-
   restore tail

.Lx86sel_call_tail:
    // Restore the stack if arguments overflowed
   onto it (x20 = stack_adj).
    cbz x20, .Lx86sel_call_ret
    adrp x0, _kcc_x86_s_add_rsp@PAGE
    add  x0, x0, _kcc_x86_s_add_rsp@PAGEOFF
    bl _kcc_as_str
    mov x0, x20
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_rsp@PAGE
    add  x0, x0, _kcc_x86_s_rsp@PAGEOFF
    bl _kcc_as_str
.Lx86sel_call_ret:
    // Store %rax into the result vreg unless the
   call is void.
    ldr x0, [x19, #IR_TYPE]
    cbz x0, .Lx86sel_done               // no type →
   nothing to home
    ldr x1, [x0, #TY_KIND]
    cmp x1, #TY_VOID
    b.eq .Lx86sel_done
    ldr x0, [x19, #IR_DEST]
    adrp x1, _kcc_x86_s_rax@PAGE
    add  x1, x1, _kcc_x86_s_rax@PAGEOFF
    bl _kcc_x86_store
    b .Lx86sel_done

.Lx86sel_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

554 · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a · uses _kcc_io_err 71b, _kcc_io_flush 69b, _kcc_ir_type_is_fp 343, _kcc_vec_push 65a

38.6. The Frame: Pre-scan, Prologue, Epilogue

Before a function's instructions can be selected, its frame must be sized. `_kcc_x86_prescan` walks every basic block and, for each `IR_ALLOCA`, records the running slot offset in SRC2 and advances by the alloca's slot count (SRC1, defaulting to one). It returns the total number of alloca slots. The vreg value slots and the alloca storage together fix the frame size; the prologue emits the standard `pushq %rbp` / `movq %rsp, %rbp` and, when the frame is non-empty, one `subq $frame, %rsp` (rounded up to a 16-byte multiple so the stack stays aligned). The `leave` in every `IR_RET` epilogue undoes all of it in one instruction, so no per-function epilogue state is needed.

⟨ X86: render 559 ⟩ +≡

```
// -- kcc_x86_prescan(ir_func *f) → total alloca
   slots; sets each alloca's SRC2.
.p2align 2
_kcc_x86_prescan:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                         // func
    mov x20, #0                         // running
   offset (slots)
    ldr x21, [x19, #IRF_BLOCKS]
    cbz x21, .Lx86ps_done
    ldr x22, [x21, #VEC_COUNT]
    mov x23, #0                         // bb index
.Lx86ps_bb:
    cmp x23, x22
```

```
    b.ge .Lx86ps_done
    mov x0, x21
    mov x1, x23
    bl _kcc_vec_get
    ldr x24, [x0]                       // BB ptr
    ldr x24, [x24, #BB_FIRST]
.Lx86ps_instr:
    cbz x24, .Lx86ps_next_bb
    ldr x0, [x24, #IR_OP]
    cmp x0, #IR_ALLOCA
    b.ne .Lx86ps_skip
    str x20, [x24, #IR_SRC2]            // record this
    alloca's offset
    ldr x0, [x24, #IR_SRC1]             // slot count
    cbnz x0, .Lx86ps_add
    mov x0, #1
.Lx86ps_add:
    add x20, x20, x0
.Lx86ps_skip:
    ldr x24, [x24, #IR_NEXT]
    b .Lx86ps_instr
.Lx86ps_next_bb:
    add x23, x23, #1
    b .Lx86ps_bb
.Lx86ps_done:
    mov x0, x20
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

559 · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a · uses _kcc_vec_get 65b

38.7. Spilling Incoming Parameters

On the callee side, the System V ABI delivers the first six integer parameters in %rdi–%r9 and the rest on the caller's stack. Because the IR numbers a function's parameters as its first vregs (%0, %1, …), the prologue copies each incoming register into its parameter's vreg slot, after which a parameter is indistinguishable from any other value in the all-spill model. A parameter past the sixth is read from the caller's stack image at 16 + (k−6)*8(%rbp) (the saved %rbp and return address occupy the first sixteen bytes above %rbp). As with the argument marshalling, rung three assumes every parameter is of the integer class; floating-point and by-value aggregate parameters are a later rung.

⟨ X86: render 560a ⟩ + ≡

```
// -- kcc_x86_spill_params(ir_func *f) - copy
   incoming args into vreg slots.
.p2align 2
_kcc_x86_spill_params:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    ldr x19, [x0, #IRF_PARAMS]          // param count
    mov x20, #0                         // k
.Lx86sp_loop:
    cmp x20, x19
    b.ge .Lx86sp_done
    cmp x20, #6
    b.ge .Lx86sp_stack
    // movq %argreg[k], slot(k)
    mov x0, x20                         // vreg = k
    adrp x1, _kcc_x86_argregs@PAGE
    add  x1, x1, _kcc_x86_argregs@PAGEOFF
    ldr x1, [x1, x20, lsl #3]           // argreg[k]
   string ptr
    bl _kcc_x86_store
    b .Lx86sp_next
.Lx86sp_stack:
    // movq 16+(k-6)*8(%rbp), %r10   (positive
   displacement: caller stack image)
    adrp x0, _kcc_x86_s_movq@PAGE
    add  x0, x0, _kcc_x86_s_movq@PAGEOFF
    bl _kcc_as_str
    sub x0, x20, #6
    lsl x0, x0, #3
    add x0, x0, #16                     // 16 + (k-6)
   *8
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_rbp@PAGE
    add  x0, x0, _kcc_x86_s_rbp@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_comma@PAGE
    add  x0, x0, _kcc_x86_s_comma@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_r10@PAGE
    add  x0, x0, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_as_str
    mov x0, #10
    bl _kcc_as_ch
    // store %r10 → slot(k)
    mov x0, x20
    adrp x1, _kcc_x86_s_r10@PAGE
    add  x1, x1, _kcc_x86_s_r10@PAGEOFF
    bl _kcc_x86_store
.Lx86sp_next:
    add x20, x20, #1
    b .Lx86sp_loop
.Lx86sp_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

560a · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a

38.8. Walking the Module

The top-level walk mirrors the IR dumper (§??): iterate the function list _kcc_ir_funcs, and for each function emit its symbol declaration and label, publish its vreg count, run the pre-scan, emit the prologue, then stream its instructions through selection. A non-static function gets a .globl; the label and declaration route the name through _kcc_x86_sym, so it carries the Mach-O leading underscore under Darwin and none under Linux/ELF.

⟨ X86: render 560b ⟩ + ≡

```
// -- kcc_x86_render_text() - walk the IR, emit
   x86-64 assembly to the buffer.
.p2align 2
_kcc_x86_render_text:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str x27, [sp, #80]

    // Section directive.
```

```
    adrp x0, _kcc_x86_s_text@PAGE
    add  x0, x0, _kcc_x86_s_text@PAGEOFF
    bl _kcc_as_str

    adrp x0, _kcc_ir_funcs@PAGE
    add  x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x19, [x0]
    cbz x19, .Lx86rt_done
    ldr x20, [x19, #VEC_COUNT]       // func count
    mov x21, #0                      // func index

.Lx86rt_func:
    cmp x21, x20
    b.ge .Lx86rt_done
    mov x0, x19
    mov x1, x21
    bl _kcc_vec_get
    ldr x22, [x0]                    // func ptr

    // .globl <name>   (unless static, or the
   synthesised module-init function).
    // "kcc.module_init" must have internal
   linkage: every translation unit with
    // a file-scope initialiser emits one, and
   exporting them would collide at
    // link time.  The ARM64 code generator
   special-cases the same name.
    ldr x0, [x22, #IRF_STATIC]
    cbnz x0, .Lx86rt_label
    ldr x0, [x22, #IRF_NAME]
    adrp x1, _kcc_str_modinit@PAGE
    add  x1, x1, _kcc_str_modinit@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lx86rt_label          // name == "kcc
   .module_init" → no .globl
    adrp x0, _kcc_x86_s_globl@PAGE
    add  x0, x0, _kcc_x86_s_globl@PAGEOFF
    bl _kcc_as_str
    ldr x0, [x22, #IRF_NAME]
    bl _kcc_x86_sym
    mov x0, #10                      // '\n'
    bl _kcc_as_ch

.Lx86rt_label:
    // <name>:  (underscore-prefixed on Darwin)
    ldr x0, [x22, #IRF_NAME]
    bl _kcc_x86_sym
    adrp x0, _kcc_x86_s_colon_nl@PAGE
    add  x0, x0, _kcc_x86_s_colon_nl@PAGEOFF
    bl _kcc_as_str

    // Publish the vreg count (alloca addressing
   reads it) and pre-scan allocas.
    ldr x26, [x22, #IRF_VREG_N]      // vreg count
    adrp x0, _kcc_x86_vreg_n@PAGE
    add  x0, x0, _kcc_x86_vreg_n@PAGEOFF
    str x26, [x0]
    mov x0, x22
    bl _kcc_x86_prescan              // x0 = total
   alloca slots

    // frame_bytes = align16((vreg_count +
   alloca_slots) * 8).
    add x0, x0, x26
    lsl x0, x0, #3                   // bytes
    add x0, x0, #15
    and x0, x0, #~15
    mov x27, x0                      // frame_bytes

    // Prologue: pushq %rbp ; movq %rsp, %rbp ;
   subq $frame, %rsp.
    adrp x0, _kcc_x86_s_push_rbp@PAGE
    add  x0, x0, _kcc_x86_s_push_rbp@PAGEOFF
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_mov_sp@PAGE
    add  x0, x0, _kcc_x86_s_mov_sp@PAGEOFF
    bl _kcc_as_str
    cbz x27, .Lx86rt_body
    adrp x0, _kcc_x86_s_sub_sp@PAGE
    add  x0, x0, _kcc_x86_s_sub_sp@PAGEOFF
    bl _kcc_as_str
    mov x0, x27
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_sp_nl@PAGE
    add  x0, x0, _kcc_x86_s_sp_nl@PAGEOFF
    bl _kcc_as_str

.Lx86rt_body:
    // Spill incoming parameters into their vreg
   slots (SysV integer class).
    mov x0, x22
    bl _kcc_x86_spill_params

    // Walk basic blocks.
    ldr x23, [x22, #IRF_BLOCKS]
    cbz x23, .Lx86rt_next_func
    ldr x24, [x23, #VEC_COUNT]       // block count
    mov x25, #0                      // block index

.Lx86rt_bb:
    cmp x25, x24
    b.ge .Lx86rt_next_func
    mov x0, x23
    mov x1, x25
    bl _kcc_vec_get
    ldr x27, [x0]                    // BB ptr
    // Emit the block's label ".Lkccbb<id>:" so
   branches can target it.
    ldr x0, [x27, #BB_ID]
    bl _kcc_x86_bblabel
    adrp x0, _kcc_x86_s_label_nl@PAGE
    add  x0, x0, _kcc_x86_s_label_nl@PAGEOFF
    bl _kcc_as_str
    ldr x27, [x27, #BB_FIRST]        // first
   instruction

.Lx86rt_instr:
    cbz x27, .Lx86rt_next_bb
    mov x0, x27
    bl _kcc_x86_sel
    ldr x27, [x27, #IR_NEXT]
    b .Lx86rt_instr

.Lx86rt_next_bb:
    add x25, x25, #1
    b .Lx86rt_bb

.Lx86rt_next_func:
    add x21, x21, #1
    b .Lx86rt_func

.Lx86rt_done:
    ldr x27, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret
```

□

560b · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a · uses _kcc_ir_funcs 305c, _kcc_vec_get 65b

38.9. Emitting Global Variable Storage

Functions are only half of a translation unit; the other half is its file-scope data. kcc emits a *mutable* global's initialiser as runtime stores (they were streamed into `main` above, exactly like any other assignment), so the storage itself only has to be reserved and zeroed—an initialised store then overwrites the zero with the real value. `_kcc_x86_emit_data` walks the AST translation unit (`_kcc_x86_tu`, published by the driver) and, for each defining variable declaration, emits a `.zerofill __DATA,__common` reservation at the declared type's size (floored at eight bytes and rounded to a whole eight-byte slot, matching the ARM64 pre-registration). An external global (`extern`) is a declaration, not a definition, and is skipped; a `static` file-scope global gets no `.globl` (internal linkage); everything else is exported so other units can link against it. On ELF a bare `.comm` is an *external* common that the linker coalesces with same-named commons from other objects, which would silently fuse two units' distinct statics into one cell; so a static common is first marked `.local` to keep it private to its translation unit.

A `const` global whose initialiser is a fixed byte image takes a different path. kcc realises those as read-only data with their actual bytes and emits *no* runtime store, so zeroed storage would read as zero rather than the initialiser. `_kcc_x86_emit_rodata_global` (§38.11) handles the *scalar and array-of-scalar* cases by emitting the initialiser as `.byte`/`.short`/`.long`/`.quad` value directives; the aggregate cases it does not yet lay out (a `const` struct, a 1-D array of structs, or a `const` pointer array) decline and stay undefined—a clean link error, not a wrong value—as does a block-scoped `static` local, whose mangled `sl_` node storage would need a walk into function bodies.

⟨ *X86: render* 562 ⟩ +≡

```
// -- kcc_x86_emit_zerofill(const char *name,
   uint64_t size, uint64_t is_static)
//    Emit zeroed common storage.  Darwin: .
   zerofill __DATA,__common,_<name>,size,3.
//    Linux/ELF: .comm <name>,size,8.  A non-
   static symbol is exported (.globl).
.p2align 2
_kcc_x86_emit_zerofill:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                     // name
    mov x20, x1                     // size
    mov x21, x2                     // is_static
    // .globl <name>  unless static (internal
   linkage)
    cbnz x21, .Lx86zf_storage
    adrp x0, _kcc_x86_s_globl@PAGE
    add  x0, x0, _kcc_x86_s_globl@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_x86_sym
    mov x0, #10
    bl _kcc_as_ch
.Lx86zf_storage:
    adrp x0, _kcc_x86_os@PAGE
    add  x0, x0, _kcc_x86_os@PAGEOFF
    ldr x0, [x0]
    cbnz x0, .Lx86zf_comm
    // Darwin .zerofill: the directive string
   already carries the "_" prefix.
    adrp x0, _kcc_x86_s_zerofill@PAGE
    add  x0, x0, _kcc_x86_s_zerofill@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_comma1@PAGE
    add  x0, x0, _kcc_x86_s_comma1@PAGEOFF
    bl _kcc_as_str
    mov x0, x20
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_zf_a3@PAGE
    add  x0, x0, _kcc_x86_s_zf_a3@PAGEOFF
    bl _kcc_as_str
    b .Lx86zf_done
.Lx86zf_comm:
    // static → .local <name>  (internal linkage;
   keeps the linker from
    // coalescing same-named commons from
   different objects into one cell)
    cbz x21, .Lx86zf_comm_dir
    adrp x0, _kcc_x86_s_local@PAGE
    add  x0, x0, _kcc_x86_s_local@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_as_str
    mov x0, #10
    bl _kcc_as_ch
.Lx86zf_comm_dir:
    // Linux .comm <name>,<size>,8
    adrp x0, _kcc_x86_s_comm@PAGE
    add  x0, x0, _kcc_x86_s_comm@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_as_str                  // bare name (
   ELF: no underscore)
    adrp x0, _kcc_x86_s_comma1@PAGE
    add  x0, x0, _kcc_x86_s_comma1@PAGEOFF
    bl _kcc_as_str
    mov x0, x20
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_comm_a8@PAGE
    add  x0, x0, _kcc_x86_s_comm_a8@PAGEOFF
    bl _kcc_as_str
.Lx86zf_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_x86_emit_data() - emit .zerofill storage
    for file-scope globals.
.p2align 2
_kcc_x86_emit_data:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    adrp x0, _kcc_x86_tu@PAGE
    add  x0, x0, _kcc_x86_tu@PAGEOFF
    ldr x19, [x0]
    cbz x19, .Lx86ed_done
    ldr x19, [x19, #AST_F0]         // first top-
   level declaration
.Lx86ed_loop:
    cbz x19, .Lx86ed_done
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_VAR_DECL
    b.ne .Lx86ed_next
    ldr x20, [x19, #AST_F1]         // name
    cbz x20, .Lx86ed_next
    ldr x0, [x19, #AST_F3]          // storage
   class
    cmp x0, #2                      // extern →
```

```
    declaration, no storage
    b.eq .Lx86ed_next
    cmp x0, #1                          // typedef → no storage
    b.eq .Lx86ed_next
    mov x21, x0                         // x21 = storage class (3 = static)
    // A const global that becomes a read-only byte image: emit its bytes.
    // The scalar / array-of-scalar cases are laid out here; the aggregate
    // cases decline (return 0) and fall through to stay undefined (deferred).
    mov x0, x19
    bl _kcc_global_const_rodata
    cbz x0, .Lx86ed_chk_ptrarr
    mov x0, x19
    mov x1, x20                         // source name
    bl _kcc_x86_emit_rodata_global      // 1 = emitted, 0 = declined (aggregate)
    b .Lx86ed_next                      // either way, done with this decl
.Lx86ed_chk_ptrarr:
    mov x0, x19
    bl _kcc_global_const_ptrarr
    cbnz x0, .Lx86ed_next               // const pointer array: deferred
    // Size = round_up_8(max(8, type_layout(resolve(AST_F0)))).
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    cbz x0, .Lx86ed_next
    bl _kcc_type_layout
    mov x22, x0
    cmp x22, #8
    b.ge .Lx86ed_round
    mov x22, #8
.Lx86ed_round:
    add x22, x22, #7
    and x22, x22, #~7                   // round up to a whole 8-byte slot

    // Emit zeroed common storage: name = x20, size = x22, static? = (x21 == 3).
    mov x0, x20
    mov x1, x22
    cmp x21, #3
    cset x2, eq
    bl _kcc_x86_emit_zerofill
.Lx86ed_next:
    ldr x19, [x19, #AST_F4]             // next sibling
    b .Lx86ed_loop
.Lx86ed_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

562 · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a · uses _kcc_global_const_ptrarr 308, _kcc_global_const_rodata 308, _kcc_sema_resolve_type 259b, _kcc_type_layout 267, 268, _kcc_x86_tu 568b

38.10. Static Locals

A block-scoped `static` local has static storage duration but is declared inside a function body, so the top-level walk of §38.9 never sees it. kcc hoists each such local to a file-level global under a mangled `sl_node` name, and every reference lowers to an `IR_GLOBAL_REF` to that name—so the storage must be emitted or the symbol is undefined at link. `_kcc_x86_emit_statics` recovers them by walking every function body exactly as the ARM64 pre-registration's second pass does (§35.4): `_kcc_x86_prereg_stmt` descends compound blocks, the arms of `if`/`switch`/loops, and `case`/`label`/`default` bodies, and for each `static` declaration it finds it emits storage under the same mangled name `_kcc_ir_static_name` produced during IR construction. The split is identical to the file-scope case: a `const` static with a scalar or array-of-scalar initialiser becomes read-only data, and a mutable static becomes zeroed common storage that its runtime initialiser fills—both with internal linkage (no `.globl`).

⟨ X86: render 563 ⟩ +≡

```
// -- kcc_x86_emit_one_static(node *decl) - emit storage for one static local.
.p2align 2
_kcc_x86_emit_one_static:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                         // decl
    // Mangled name (matches the sl_<node> the IR_GLOBAL_REF uses).
    bl _kcc_ir_static_name
    mov x20, x0                         // name
    // const scalar/array-of-scalar → read-only data; else zeroed storage.
    mov x0, x19
    bl _kcc_global_const_rodata
    cbz x0, .Lx86os_zero
    mov x0, x19
    mov x1, x20
    bl _kcc_x86_emit_rodata_global  // emits, or declines (aggregate → undefined)
    b .Lx86os_done
.Lx86os_zero:
    // Mutable static: size = round_up_8(max(8, layout)); zeroed, internal.
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    cbz x0, .Lx86os_done
    bl _kcc_type_layout
    mov x21, x0
    cmp x21, #8
    b.ge .Lx86os_round
    mov x21, #8
.Lx86os_round:
    add x21, x21, #7
    and x21, x21, #~7
    mov x0, x20                         // name
    mov x1, x21                         // size
    mov x2, #1                          // is_static (internal linkage)
    bl _kcc_x86_emit_zerofill
.Lx86os_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_x86_prereg_stmt(node *stmt) - walk a statement (and its sibling chain),
//    emitting storage for every static local at any block depth.  Mirrors the
//    ARM64 pre-registration walker field for field.
```

```
.p2align 2
_kcc_x86_prereg_stmt:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                     // stmt cursor
.Lx86ps2_loop:
    cbz x19, .Lx86ps2_ret
    ldr x20, [x19, #AST_TAG]
    cmp x20, #AST_VAR_DECL
    b.ne .Lx86ps2_notvar
    ldr x0, [x19, #AST_F3]          // storage
   class (3 = static)
    cmp x0, #3
    b.ne .Lx86ps2_next
    mov x0, x19
    bl _kcc_x86_emit_one_static
    b .Lx86ps2_next
.Lx86ps2_notvar:
    cmp x20, #AST_COMPOUND
    b.eq .Lx86ps2_f0
    cmp x20, #AST_IF_STMT
    b.eq .Lx86ps2_if
    cmp x20, #AST_SWITCH_STMT
    b.eq .Lx86ps2_f1
    cmp x20, #AST_WHILE_STMT
    b.eq .Lx86ps2_f1
    cmp x20, #AST_DO_STMT
    b.eq .Lx86ps2_f0
    cmp x20, #AST_FOR_STMT
    b.eq .Lx86ps2_for
    cmp x20, #AST_CASE_STMT
    b.eq .Lx86ps2_f1
    cmp x20, #AST_LABEL_STMT
    b.eq .Lx86ps2_f1
    cmp x20, #AST_DEFAULT_STMT
    b.eq .Lx86ps2_f0
    b .Lx86ps2_next
.Lx86ps2_if:
    ldr x0, [x19, #AST_F1]          // then branch
    bl _kcc_x86_prereg_stmt
    ldr x0, [x19, #AST_F2]          // else branch
    bl _kcc_x86_prereg_stmt
    b .Lx86ps2_next
.Lx86ps2_for:
    ldr x0, [x19, #AST_F0]          // init clause
    bl _kcc_x86_prereg_stmt
    ldr x0, [x19, #AST_F3]          // body
    bl _kcc_x86_prereg_stmt
    b .Lx86ps2_next
.Lx86ps2_f1:
    ldr x0, [x19, #AST_F1]
    bl _kcc_x86_prereg_stmt
    b .Lx86ps2_next
.Lx86ps2_f0:
    ldr x0, [x19, #AST_F0]
    bl _kcc_x86_prereg_stmt
.Lx86ps2_next:
    ldr x19, [x19, #AST_F4]         // next
   sibling
    b .Lx86ps2_loop
.Lx86ps2_ret:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_x86_emit_statics() - walk every function
    body for static locals.
.p2align 2
_kcc_x86_emit_statics:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    adrp x0, _kcc_x86_tu@PAGE
    add  x0, x0, _kcc_x86_tu@PAGEOFF
    ldr x19, [x0]
    cbz x19, .Lx86es2_done
    ldr x19, [x19, #AST_F0]         // first top-
   level declaration
.Lx86es2_loop:
    cbz x19, .Lx86es2_done
    ldr x0, [x19, #AST_TAG]
    cmp x0, #AST_FUNC_DECL
    b.ne .Lx86es2_next
    ldr x0, [x19, #AST_F3]          // function
   body
    cbz x0, .Lx86es2_next
    bl _kcc_x86_prereg_stmt
.Lx86es2_next:
    ldr x19, [x19, #AST_F4]
    b .Lx86es2_loop
.Lx86es2_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

563 · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a · uses _kcc_global_const_rodata 308, _kcc_ir_static_name 417a, _kcc_sema_resolve_type 259b, _kcc_type_layout 267, 268, _kcc_x86_tu 568b

38.11. Read-Only Initialised Data: const Globals

A `const` global with a fixed initialiser is realised as bytes in a read-only section rather than as zeroed storage plus a runtime store—the IR carries no store for it, so the value must come from the object image. For a scalar or a one-dimensional array of scalars, the byte image is just the initialiser's elements laid out end to end, which the textual back end emits directly as value directives: `.byte` for a one-byte element, `.short` for two, `.long` for four, `.quad` for eight. Each element's bits come from the shared `_kcc_rodata_leaf_bits`, the same evaluator the ARM64 back end uses, so a constant expression, a negation, or a folded arithmetic initialiser all reduce to a number here. An array longer than its initialiser has its tail zero-filled with a single `.zero` directive.

A `const` struct of scalar fields, and a one-dimensional array of such structs, are laid out field by field. A struct's initialiser matches its fields positionally, but a field does not always begin where the previous one ended—the ABI inserts padding so each member sits at its natural alignment—so `_kcc_x86_emit_struct_bytes` tracks a write cursor and, before each field, emits a `.zero` run to bridge any gap from the cursor to that field's recorded byte offset. Each field is then a scalar value directive at its own width, and any bytes past the last initialised field (trailing padding or unmentioned fields) are zero-filled to the struct's total size. An array of structs repeats this per element and zero-fills any element the initialiser omits. The only `const` aggregate still declined is a pointer array—its elements are relocations to other symbols, not plain bytes—which stays undefined, an honest link error. The routine returns 1 when it emitted the data.

⟨ *X86: render* 564 ⟩ + ≡

```
// -- kcc_x86_emit_struct_bytes(type *sty, node *
   init, uint64_t total) - lay out
//    one struct's scalar fields as value
   directives, bridging alignment padding
//    with .zero and zero-filling the tail to `
   total` bytes.
.p2align 2
_kcc_x86_emit_struct_bytes:
    stp x29, x30, [sp, #-80]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]
    mov x22, x2                     // total
   struct size
    ldr x0, [x0, #TY_F0]            //
   AST_STRUCT_DECL
    ldr x19, [x0, #AST_F1]          // field
   cursor
    mov x20, #0                     // init
   element cursor
    cbz x1, .Lx86sb_fields
    ldr x20, [x1, #AST_F0]          // first init
   element
.Lx86sb_fields:
    mov x23, #0                     // write
   cursor (byte position)
.Lx86sb_loop:
    cbz x19, .Lx86sb_tail
    // Field type → size + kind; field byte offset
   .
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    cbz x0, .Lx86sb_tail
    ldr x24, [x0, #TY_SIZE]         // field size
    ldr x25, [x0, #TY_KIND]         // field kind
    ldr x0, [x19, #AST_F3]          // field byte
   offset
    // Bridge any alignment padding [cursor,
   offset) with .zero.
    cmp x0, x23
    b.ls .Lx86sb_atfield
    sub x1, x0, x23                 // gap bytes
    mov x23, x0                     // cursor =
   offset
    mov x0, x1
    bl _kcc_x86_emit_zero
.Lx86sb_atfield:
    // Value bits: from the init element if
   present, else zero.
    cbz x20, .Lx86sb_zeroval
    mov x0, x20
    mov x1, x25
    bl _kcc_rodata_leaf_bits
    ldr x20, [x20, #AST_F4]         // advance init
   element
    b .Lx86sb_emit
.Lx86sb_zeroval:
    mov x0, #0
.Lx86sb_emit:
    mov x1, x24                     // field size
    bl _kcc_x86_emit_value
    add x23, x23, x24               // cursor +=
   field size
    ldr x19, [x19, #AST_F4]         // next field
    b .Lx86sb_loop
.Lx86sb_tail:
    // Zero-fill from the cursor to the struct's
   total size.
    cmp x23, x22
    b.ge .Lx86sb_done
    sub x0, x22, x23
    bl _kcc_x86_emit_zero
.Lx86sb_done:
    ldr x25, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #80
    ret

// -- kcc_x86_emit_zero(uint64_t n) - emit "\t.
   zero n\n" (nothing if n == 0).
.p2align 2
_kcc_x86_emit_zero:
    cbz x0, .Lx86ez_ret
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0
    adrp x0, _kcc_x86_s_zerodir@PAGE
    add  x0, x0, _kcc_x86_s_zerodir@PAGEOFF
    bl _kcc_as_str
    mov x0, x19
    bl _kcc_as_dec
    mov x0, #10
    bl _kcc_as_ch
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
.Lx86ez_ret:
    ret

// -- kcc_x86_emit_rodata_global(node *decl, const
    char *name) → 1 emitted, 0 declined.
//    Emits a const global's read-only byte image:
    scalar, array-of-scalar,
//    struct-of-scalars, or array of such structs.
     Declines a pointer array.
//    `name` is the linker symbol: the source name
    for a file-scope global, the
//    mangled sl_<node> for a static local.
.p2align 2
_kcc_x86_emit_rodata_global:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str x27, [sp, #80]
    mov x19, x0                             // decl
    mov x20, x1                             // symbol name
    (source name, or mangled sl_ for a static
   local)
    cbz x20, .Lx86rg_decline

    // Resolve type and classify the shape into
   x22:
    //   0 = scalar, 1 = array of scalar, 2 =
   struct, 3 = array of struct.
    // x23 = the element/struct type; x25 =
   element size; x26 = element kind;
    // x24 = declared element count (for the array
    shapes).
    ldr x0, [x19, #AST_F0]
    bl _kcc_sema_resolve_type
    cbz x0, .Lx86rg_decline
    mov x21, x0                             // full type
    ldr x0, [x21, #TY_KIND]
    cmp x0, #TY_STRUCT
    b.eq .Lx86rg_shape_struct
    cmp x0, #TY_ARRAY
    b.eq .Lx86rg_shape_array
    // Scalar.
    mov x22, #0
```

```
    mov x23, x21
    mov x24, #1
    b .Lx86rg_classified
.Lx86rg_shape_struct:
    mov x22, #2
    mov x23, x21                    // struct type
    b .Lx86rg_classified
.Lx86rg_shape_array:
    ldr x23, [x21, #TY_F0]          // element
   type
    ldr x0, [x23, #TY_KIND]
    cmp x0, #TY_STRUCT
    b.eq .Lx86rg_shape_aos
    mov x22, #1                     // array of
   scalar
    b .Lx86rg_arr_count
.Lx86rg_shape_aos:
    mov x22, #3                     // array of
   struct
.Lx86rg_arr_count:
    ldr x25, [x23, #TY_SIZE]        // element
   size
    cbz x25, .Lx86rg_decline
    mov x0, x21
    bl _kcc_type_layout             // total bytes
    udiv x24, x0, x25               // declared
   element count = total / elem_size
.Lx86rg_classified:
    ldr x25, [x23, #TY_SIZE]        // element
   size (bytes)
    ldr x26, [x23, #TY_KIND]        // element
   kind (for leaf_bits)

    // Read-only section header + alignment, then
   .globl / label (once).
    adrp x0, _kcc_x86_os@PAGE
    add  x0, x0, _kcc_x86_os@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lx86rg_dsect
    adrp x0, _kcc_x86_s_rodata@PAGE
    add  x0, x0, _kcc_x86_s_rodata@PAGEOFF
    b .Lx86rg_sect
.Lx86rg_dsect:
    adrp x0, _kcc_x86_s_constsect@PAGE
    add  x0, x0, _kcc_x86_s_constsect@PAGEOFF
.Lx86rg_sect:
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_p2align3@PAGE
    add  x0, x0, _kcc_x86_s_p2align3@PAGEOFF
    bl _kcc_as_str
    // .globl <name>  (unless static)
    ldr x0, [x19, #AST_F3]          // storage
   class
    cmp x0, #3                      // static →
   internal linkage
    b.eq .Lx86rg_label
    adrp x0, _kcc_x86_s_globl@PAGE
    add  x0, x0, _kcc_x86_s_globl@PAGEOFF
    bl _kcc_as_str
    mov x0, x20
    bl _kcc_x86_sym
    mov x0, #10
    bl _kcc_as_ch
.Lx86rg_label:
    mov x0, x20
    bl _kcc_x86_sym
    adrp x0, _kcc_x86_s_colon_nl@PAGE
    add  x0, x0, _kcc_x86_s_colon_nl@PAGEOFF
    bl _kcc_as_str

    // Dispatch by shape.
    cmp x22, #2
    b.eq .Lx86rg_struct
    cmp x22, #3
    b.eq .Lx86rg_aos

    // Scalar / array-of-scalar: one value
   directive per element.  x22 = emitted.
    mov x22, #0
    ldr x0, [x19, #AST_F2]          // initialiser
    cbz x0, .Lx86rg_tail            // no init →
   all zero (tail fill covers it)
    ldr x1, [x0, #AST_TAG]
    cmp x1, #AST_INIT_LIST
    b.ne .Lx86rg_scalar_init
    ldr x27, [x0, #AST_F0]          // first
   element
.Lx86rg_elem_loop:
    cbz x27, .Lx86rg_tail
    mov x0, x27
    mov x1, x26                     // element
   kind
    bl _kcc_rodata_leaf_bits        // x0 =
   element bits
    mov x1, x25                     // element
   size
    bl _kcc_x86_emit_value          // emit ".<
   width> <masked value>\n"
    add x22, x22, #1
    ldr x27, [x27, #AST_F4]
    b .Lx86rg_elem_loop
.Lx86rg_scalar_init:
    ldr x0, [x19, #AST_F2]          // the scalar
   init node
    mov x1, x26
    bl _kcc_rodata_leaf_bits
    mov x1, x25
    bl _kcc_x86_emit_value
    mov x22, #1
.Lx86rg_tail:
    // Zero-fill any declared elements the
   initialiser did not cover.
    cmp x22, x24
    b.ge .Lx86rg_ok
    sub x0, x24, x22
    mul x0, x0, x25                 // (count -
   emitted) * elem_size bytes
    cbz x0, .Lx86rg_ok
    mov x27, x0                     // byte count
    adrp x0, _kcc_x86_s_zerodir@PAGE
    add  x0, x0, _kcc_x86_s_zerodir@PAGEOFF
    bl _kcc_as_str
    mov x0, x27
    bl _kcc_as_dec
    mov x0, #10
    bl _kcc_as_ch
    b .Lx86rg_ok

.Lx86rg_struct:
    // One struct: lay out its fields with padding
   .  total size = layout.
    mov x0, x21
    bl _kcc_type_layout
    mov x24, x0                     // total
   struct size
    mov x0, x23                     // struct type
    ldr x1, [x19, #AST_F2]          // init list
    mov x2, x24                     // total size
    bl _kcc_x86_emit_struct_bytes
    b .Lx86rg_ok
.Lx86rg_aos:
    // Array of structs: each initialiser element
   is a struct brace list.
    // x23 = struct element type, x25 = struct
   size, x24 = declared count.
    mov x22, #0                     // emitted
   struct count
    ldr x27, [x19, #AST_F2]         // array init
   list
```

```
    cbz x27, .Lx86rg_aos_tail
    ldr x27, [x27, #AST_FO]          // first
   struct element
.Lx86rg_aos_loop:
    cbz x27, .Lx86rg_aos_tail
    cmp x22, x24
    b.ge .Lx86rg_aos_tail
    mov x0, x23                      // struct type
    mov x1, x27                      // this struct
   's brace list
    mov x2, x25                      // struct size
    bl _kcc_x86_emit_struct_bytes
    add x22, x22, #1
    ldr x27, [x27, #AST_F4]          // next struct
    element
    b .Lx86rg_aos_loop
.Lx86rg_aos_tail:
    // Zero-fill missing array elements.
    cmp x22, x24
    b.ge .Lx86rg_ok
    sub x0, x24, x22
    mul x0, x0, x25
    cbz x0, .Lx86rg_ok
    mov x27, x0
    adrp x0, _kcc_x86_s_zerodir@PAGE
    add  x0, x0, _kcc_x86_s_zerodir@PAGEOFF
    bl _kcc_as_str
    mov x0, x27
    bl _kcc_as_dec
    mov x0, #10
    bl _kcc_as_ch

.Lx86rg_ok:
    mov x0, #1
    b .Lx86rg_ret
.Lx86rg_decline:
    mov x0, #0
.Lx86rg_ret:
    ldr x27, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #96
    ret

// -- kcc_x86_emit_value(uint64_t bits, uint64_t
   size) - emit one value
//    directive (.byte/.short/.long/.quad) with `
   bits` masked to `size` bytes.
.p2align 2
_kcc_x86_emit_value:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    mov x19, x0                      // bits
    mov x20, x1                      // size
    cmp x20, #1
    b.eq .Lx86ev_b
    cmp x20, #2
    b.eq .Lx86ev_s
    cmp x20, #8
    b.eq .Lx86ev_q
    // default: 4-byte .long
    adrp x0, _kcc_x86_s_d_long@PAGE
    add  x0, x0, _kcc_x86_s_d_long@PAGEOFF
    bl _kcc_as_str
    and x19, x19, #0xffffffff
    b .Lx86ev_num
.Lx86ev_b:
    adrp x0, _kcc_x86_s_d_byte@PAGE
    add  x0, x0, _kcc_x86_s_d_byte@PAGEOFF
    bl _kcc_as_str
    and x19, x19, #0xff
    b .Lx86ev_num
.Lx86ev_s:
    adrp x0, _kcc_x86_s_d_short@PAGE
    add  x0, x0, _kcc_x86_s_d_short@PAGEOFF
    bl _kcc_as_str
    and x19, x19, #0xffff
    b .Lx86ev_num
.Lx86ev_q:
    adrp x0, _kcc_x86_s_d_quad@PAGE
    add  x0, x0, _kcc_x86_s_d_quad@PAGEOFF
    bl _kcc_as_str
.Lx86ev_num:
    mov x0, x19
    bl _kcc_as_dec
    mov x0, #10
    bl _kcc_as_ch
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

564 · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a · uses _kcc_rodata_leaf_bits 468a, _kcc_sema_resolve_type 259b, _kcc_type_layout 267, 268

38.12. Registering the Module Initialiser

A file-scope initialiser that is not a compile-time constant—and, as it happens, a `static` array initialised from a string literal, which kcc fills with runtime stores rather than read-only bytes—is emitted as code inside a synthesised `kcc.module_init` function. That function only runs if something *calls* it, and in a `main`-bearing unit its work is inlined into `main`; but in every other translation unit it must be registered so the loader runs it before `main`. When the IR builder recorded a module initialiser (`_kcc_modinit_present`), `_kcc_x86_emit_modinit` emits a pointer to it in the platform's init-array section—`__DATA,__mod_init_func` on Darwin, `.init_array` on Linux—so a global or static's initialiser in a non-`main` unit actually executes. Without this the storage stays zero, which is exactly why a freshly linked zlib reported a bogus version: `deflateInit`'s `static const char my_version[] = ZLIB_VERSION` was never filled.

⟨ X86: render 567 ⟩ +≡

```
// -- kcc_x86_emit_modinit() - register kcc.
   module_init in the init-array.
.p2align 2
_kcc_x86_emit_modinit:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    adrp x0, _kcc_modinit_present@PAGE
    add  x0, x0, _kcc_modinit_present@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lx86mi_done
    // Section header (Darwin __mod_init_func vs
   Linux .init_array), then
    // ".quad <kcc.module_init>".
    adrp x0, _kcc_x86_os@PAGE
    add  x0, x0, _kcc_x86_os@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lx86mi_darwin
    adrp x0, _kcc_x86_s_modinit_l@PAGE
    add  x0, x0, _kcc_x86_s_modinit_l@PAGEOFF
    b .Lx86mi_emit
.Lx86mi_darwin:
```

```
    adrp x0, _kcc_x86_s_modinit_d@PAGE
    add  x0, x0, _kcc_x86_s_modinit_d@PAGEOFF
.Lx86mi_emit:
    bl _kcc_as_str
    adrp x0, _kcc_x86_s_modinitnm@PAGE
    add  x0, x0, _kcc_x86_s_modinitnm@PAGEOFF
    bl _kcc_x86_sym                  // "_kcc.
   module_init" (Darwin) / bare (Linux)
    mov x0, #10
    bl _kcc_as_ch
.Lx86mi_done:
    ldp x29, x30, [sp], #16
    ret
```

□

567 · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a · uses _kcc_modinit_present 305c

38.13. Emitting String-Literal Data

A string literal reaches the back end as an IR_STRING_LIT whose SRC1 points at the raw bytes and SRC2 is their length. Selection cannot emit the bytes inline—they belong in the read-only __TEXT,__cstring section, not in the instruction stream—so it does the same two-phase trick the ARM64 back end does: at the use site it assigns the literal the next private label .Lx86str$N$, records the (bytes, length) pair in a table, and loads the label's address with a leaq. After all functions and globals are emitted, _kcc_x86_emit_strings walks the table once and emits the cstring section: each entry's label followed by a .byte line carrying its bytes and a terminating NUL. Emitting raw .byte values (rather than a quoted .asciz) sidesteps any escaping question—an embedded quote, backslash, or NUL is just another number. The table is per-render state, initialised in _kcc_x86_render; the label is private (.L-prefixed), so two translation units never collide.

⟨ *X86: render* 568a ⟩ + ≡

```
// -- kcc_x86_emit_strings() - emit the cstring
   section for recorded literals.
.p2align 2
_kcc_x86_emit_strings:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    adrp x0, _kcc_x86_strs@PAGE
    add  x0, x0, _kcc_x86_strs@PAGEOFF
    ldr x19, [x0]
    cbz x19, .Lx86es_done
    ldr x20, [x19, #VEC_COUNT]       // number of
   literals
    cbz x20, .Lx86es_done

    // Read-only section header: Darwin __TEXT,
   __cstring vs Linux .rodata.
    adrp x0, _kcc_x86_os@PAGE
    add  x0, x0, _kcc_x86_os@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lx86es_darwin_sect
    adrp x0, _kcc_x86_s_rodata@PAGE
    add  x0, x0, _kcc_x86_s_rodata@PAGEOFF
    b .Lx86es_sect_emit
.Lx86es_darwin_sect:
    adrp x0, _kcc_x86_s_cstrsect@PAGE
    add  x0, x0, _kcc_x86_s_cstrsect@PAGEOFF
.Lx86es_sect_emit:
    bl _kcc_as_str

    mov x21, #0                      // literal
   index
.Lx86es_lit:
    cmp x21, x20
    b.ge .Lx86es_done
    // .Lx86str<id>:
    adrp x0, _kcc_x86_s_strlbl@PAGE
    add  x0, x0, _kcc_x86_s_strlbl@PAGEOFF
    bl _kcc_as_str
    mov x0, x21
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_label_nl@PAGE
    add  x0, x0, _kcc_x86_s_label_nl@PAGEOFF
    bl _kcc_as_str
    // Fetch entry {ptr, len}.
    mov x0, x19
    mov x1, x21
    bl _kcc_vec_get
    ldr x23, [x0]                    // data ptr
    ldr x24, [x0, #8]                // length
    // \t.byte
    adrp x0, _kcc_x86_s_byte@PAGE
    add  x0, x0, _kcc_x86_s_byte@PAGEOFF
    bl _kcc_as_str
    mov x22, #0                      // byte index
.Lx86es_byte:
    cmp x22, x24
    b.ge .Lx86es_nul
    ldrb w0, [x23, x22]
    bl _kcc_as_dec
    adrp x0, _kcc_x86_s_comma1@PAGE
    add  x0, x0, _kcc_x86_s_comma1@PAGEOFF
    bl _kcc_as_str
    add x22, x22, #1
    b .Lx86es_byte
.Lx86es_nul:
    // trailing NUL terminator, then newline
    adrp x0, _kcc_x86_s_zero_nl@PAGE
    add  x0, x0, _kcc_x86_s_zero_nl@PAGEOFF
    bl _kcc_as_str
    add x21, x21, #1
    b .Lx86es_lit
.Lx86es_done:
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

568a · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a · uses _kcc_vec_get 65b

38.14. Rendering to a File

_kcc_x86_render(path) is the entry point the driver calls. It reuses the growable text buffer of the .s back end (§??)—_kcc_as_init to allocate it and _kcc_as_str/_kcc_as_ch/_kcc_as_dec to fill it—renders the functions and then the global data, then writes the buffer to the output path through the shared _kcc_as_write_file helper, exactly as the ARM64 assembly path does. The buffer machinery is target-neutral; only the text poured into it is x86-64.

⟨ *X86: render* 568b ⟩ + ≡

```
// -- kcc_x86_render(const char *path) - render
   x86-64 asm to <path>. 0 on ok.
.globl _kcc_x86_render
.p2align 2
_kcc_x86_render:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x0, [sp, #16]              // save path
   across the render calls
    bl _kcc_as_init
    // Fresh string-literal table for this render
   (16-byte {ptr,len} entries).
    mov x0, #16
    bl _kcc_vec_init
    adrp x1, _kcc_x86_strs@PAGE
    add  x1, x1, _kcc_x86_strs@PAGEOFF
    str x0, [x1]
    bl _kcc_x86_render_text        // emits
   functions (records string literals)
    bl _kcc_x86_emit_data          // file-scope
   global storage
    bl _kcc_x86_emit_statics       // static-local
    storage (mangled sl_ names)
    bl _kcc_x86_emit_strings       // cstring
   section for recorded literals
    bl _kcc_x86_emit_modinit       // register kcc
   .module_init (non-const inits)
    ldr x0, [sp, #16]             // path
    bl _kcc_as_write_file
    ldp x29, x30, [sp], #32
    ret
```

□

568b · defined at 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab · used in 31a · defines `_kcc_target` 326, 546, 603, 609, 611, `_kcc_x86_render` 603, `_kcc_x86_tu` 546, 562, 563, 603 · uses `_kcc_vec_init` 64b

38.15. Rung One in Retrospect

Rung one selected only `IR_CONST` and `IR_RET` and carried the returned constant in a small per-function value map, emitting the whole function as `movl $N, %eax` / `retq` with no stack frame at all. Rung two subsumes that path: a constant now materialises into its vreg slot and the return loads from it, so `int main(void){return N;}` still exits with `N`—now through the general all-spill frame rather than a special case. The map is gone; every value lives in memory between instructions, which is exactly what let the arithmetic and memory opcodes join the target without disturbing the ones already working.

39. The Linux Runtime

Chapter 7 gave the compiler its ARM64/macOS floor: thin wrappers over `svc #0x80` and a Mach-O entry point. Everything above that floor is target-neutral by construction — the ported subsystems bottom out on the same `kcc_sys_*` names — so bringing kcc to a second operating system is a matter of laying a *second* floor with the same shape. This chapter is that floor for Linux on x86-64: the ELF entry point and the syscall wrappers, hand-written in the assembly the platform's as consumes, so that a translation unit kcc emits with `--target=x86_64-linux` links into a running executable with no C library at all.

It is a distinct artefact from the compiler's own object files: it is never part of the ARM64 kcc binary, and it is assembled on a Linux host (or by a cross assembler). It is the runtime that the compiler's Linux output — and, once every subsystem is C, the Linux kcc itself — links against.

39.1. Calling Convention and Syscall ABI

The Linux/x86-64 syscall ABI differs from macOS/ARM64 in three ways that the wrappers must respect. The syscall *number* goes in `rax` (not a dedicated register), the trap is the `syscall` instruction (not `svc`), and the fourth argument travels in `r10` — because the System V C convention puts it in `rcx`, but `syscall` clobbers `rcx` (and `r11`) as part of its operation. So a wrapper that forwards four or more C arguments must copy `rcx` into `r10` first. The result comes back in `rax`, with errors as small negative values.

⟨ *Runtime-linux: constants* 571a ⟩ ≡

```
# Linux/x86-64 syscall numbers (arch/x86/entry/
   syscalls/syscall_64.tbl)
.set SYS_read,     0
.set SYS_write,    1
.set SYS_open,     2
.set SYS_close,    3
.set SYS_lseek,    8
.set SYS_mmap,     9
.set SYS_munmap,  11
.set SYS_fork,    57
.set SYS_execve,  59
.set SYS_exit,    60
.set SYS_wait4,   61
.set SYS_unlink,  87
```

□

571a · used in 573a

39.2. The ELF Entry Point

The kernel transfers control to `_start` with the stack, not registers, carrying the program arguments: `rsp` points at `argc`, immediately followed by the `argv` pointers, a null terminator, and then the `envp` pointers. There is no return address and no C runtime to unwind into. The entry code reconstructs the System V argument registers a C `main` expects — `argc` in `rdi`, `argv` in `rsi`, `envp` in `rdx` — calls `main`, and turns its return value into an `exit` syscall. Since `rsp` is sixteen-byte aligned at process entry and the `call` pushes the eight-byte return address, `main` sees exactly the alignment the ABI promises.

⟨ *Runtime-linux: start* 571b ⟩ ≡

```
        .text
        .globl _start
_start:
        # Run the module initializers before main.
     kcc lowers a non-main
        # translation unit's file-scope
   initializers into a synthesised
        # `kcc.module_init` and registers its
   address in .init_array (the ELF form
        # of a Mach-O __mod_init_func); a normal C
    runtime runs those before main,
        # but a -nostdlib static executable has
   none, so the entry does it here.
        # Without this, every non-main TU's
   initialised global keeps its zero image
        # --- e.g. io's `kcc_io_fd = 1` stays 0,
   so buffered output goes to fd 0.
        # The linker provides __init_array_start/
   _end around the section; %rbx and
        # %rbp are callee-saved, so they survive
   each initializer call, and the
        # balanced call/ret keeps %rsp (hence argc
    at (%rsp)) unchanged.
        leaq    __init_array_start(%rip), %rbx
        leaq    __init_array_end(%rip), %rbp
.Linit_loop:
        cmpq    %rbp, %rbx
        jae     .Linit_done
        call    *(%rbx)
        addq    $8, %rbx
        jmp     .Linit_loop
.Linit_done:
        movq    $1, kcc_host_linux(%rip)   # tell
   the driver its as/ld are ELF tools
        movq    (%rsp), %rdi            # argc
        leaq    8(%rsp), %rsi           # argv  = &
   stack[1]
        leaq    16(%rsp,%rdi,8), %rdx   # envp  = &
   argv[argc+1]
        call    main
        movq    %rax, %rdi              # exit
   status = main()'s return value
        movq    $SYS_exit, %rax
        syscall
        hlt                             #
   unreachable
```

□

571b · used in 573a

39.3. Process Control

`exit` terminates the process and never returns; `fork`, `execve`, and `wait4` are the three the driver uses to run the assembler and linker as child processes. Each is a direct forward of its C arguments to the syscall, with the `rcx`→`r10` fix-up where a fourth argument is present.

⟨ *Runtime-linux: process* 571c ⟩ ≡

```
# void kcc_sys_exit(int status)          - %edi =
   status; does not return.
        .globl kcc_sys_exit
kcc_sys_exit:
        movq    $SYS_exit, %rax
        syscall
        hlt

# long kcc_sys_fork(void)
        .globl kcc_sys_fork
kcc_sys_fork:
        movq    $SYS_fork, %rax
        syscall
        ret

# long kcc_sys_execve(const char *path, char *
   const argv[], char *const envp[])
        .globl kcc_sys_execve
kcc_sys_execve:
        movq    $SYS_execve, %rax
        syscall
        ret

# long kcc_sys_wait4(long pid, int *status, int
   options, void *rusage)
        .globl kcc_sys_wait4
kcc_sys_wait4:
        movq    %rcx, %r10              # 4th arg:
   SysV %rcx -> syscall %r10
        movq    $SYS_wait4, %rax
        syscall
        ret
```

□

571c · used in 573a

39.4. File I/O

The file primitives — `read`, `write`, `open`, `close`, `lseek`, `unlink` — each take at most three arguments, so the System V registers `rdi`, `rsi`, `rdx` already sit where `syscall` wants them; the wrappers only set the number and trap.

⟨ *Runtime-linux: fileio* 572a ⟩ ≡

```
# long kcc_sys_read(long fd, void *buf, long len)
        .globl kcc_sys_read
kcc_sys_read:
        movq    $SYS_read, %rax
        syscall
        ret

# long kcc_sys_write(long fd, const void *buf,
   long len)
        .globl kcc_sys_write
kcc_sys_write:
        movq    $SYS_write, %rax
        syscall
        ret

# long kcc_sys_open(const char *path, long flags,
   long mode)
# kcc hard-codes the macOS open() flag values (
   O_CREAT=0x200, O_TRUNC=0x400),
# so when it runs on Linux those bits mean
   something else and an output file is
# never created.  Translate the two flag words kcc
    actually emits --- 0 (read)
# and 0x601 (write|create|trunc) --- into the
   Linux convention (O_CREAT=0x40,
# O_TRUNC=0x200) before the syscall; the access
   mode (low two bits) is identical.
        .globl kcc_sys_open
kcc_sys_open:
        movq    %rsi, %r11              # macOS-
   convention flags
        andq    $3, %rsi                # keep
   access mode (RDONLY/WRONLY/RDWR)
        testq   $0x200, %r11            # macOS
   O_CREAT
        jz      1f
        orq     $0x40, %rsi             # Linux
   O_CREAT
1:      testq   $0x400, %r11            # macOS
   O_TRUNC
        jz      2f
        orq     $0x200, %rsi            # Linux
   O_TRUNC
2:      movq    $SYS_open, %rax
        syscall
        ret

# long kcc_sys_close(long fd)
        .globl kcc_sys_close
kcc_sys_close:
        movq    $SYS_close, %rax
        syscall
        ret

# long kcc_sys_lseek(long fd, long off, long
   whence)
        .globl kcc_sys_lseek
kcc_sys_lseek:
        movq    $SYS_lseek, %rax
        syscall
        ret

# long kcc_sys_unlink(const char *path)
        .globl kcc_sys_unlink
kcc_sys_unlink:
        movq    $SYS_unlink, %rax
        syscall
        ret
```

□

572a · used in 573a

39.5. Memory Mapping

`mmap` is where the two operating systems disagree on more than registers. The arena (Chapter 48) requests its chunks with `flags = MAP_PRIVATE | MAP_ANON`, but that constant is `0x1002` on macOS and `0x22` on Linux — the anonymous bit is `0x1000` there and `0x20` here. Rather than fork the arena for each target, the OS layer owns the difference: when the incoming flags carry the Mach anonymous bit, the wrapper clears it and sets the Linux one, so the arena's unmodified `kcc_sys_mmap(0, size, 3, 0x1002, -1, 0)` maps anonymous private memory here too. (A future refactor may replace the raw `mmap` call with a target-neutral "map anonymous" primitive; until then this translation is the seam.) Beyond the flags, `mmap` is a six-argument syscall, so the fourth argument still takes the `rcx`→`r10` route.

⟨ *Runtime-linux: mmap* 572b ⟩ ≡

```
# void *kcc_sys_mmap(void *addr, long len, int
   prot, int flags, int fd, long off)
        .globl kcc_sys_mmap
kcc_sys_mmap:
```

```
        testq   $0x1000, %rcx          # Mach
   MAP_ANON present?
        jz      .Lmmap_go
        btrq    $12, %rcx              # clear
   Mach MAP_ANON (0x1000)
        orq     $0x20, %rcx            # set Linux
    MAP_ANONYMOUS
.Lmmap_go:
        movq    %rcx, %r10             # 4th arg:
   SysV %rcx -> syscall %r10
        movq    $SYS_mmap, %rax
        syscall
        ret

# long kcc_sys_munmap(void *addr, long len)
        .globl kcc_sys_munmap
kcc_sys_munmap:
        movq    $SYS_munmap, %rax
        syscall
        ret
```

□

572b · used in 573a

39.6. Assembling the Runtime

The runtime is a single `.s` file. Marking the stack non-executable keeps the linker from emitting an executable-stack warning or an `RWX` segment.

⟨ *Runtime-linux: file* 573a ⟩ ≡

```
⟨ Runtime-linux: constants 571a ⟩
⟨ Runtime-linux: start 571b ⟩
⟨ Runtime-linux: process 571c ⟩
⟨ Runtime-linux: fileio 572a ⟩
⟨ Runtime-linux: mmap 572b ⟩
        .section .note.GNU-stack,"",@progbits
```

□

573a · used in 37f

39.7. Verification: A Cross-Compiled, Native Linux Run

`make meta-a` closes the loop the retargeting playbook set out: it takes a small *multi-file* C program, compiles each translation unit with this compiler under `--target=x86_-64-linux`, links the results against the runtime above — statically, with no libc — and runs the executable on Linux. The program exercises the three seams the runtime provides: it maps arena memory through `kcc_sys_mmap`, calls across a translation-unit boundary, writes its result through `kcc_sys_write`, and returns a status the harness checks. A green run means a program kcc compiled runs natively on a different operating system and architecture than the one kcc runs on.

The program under test. Two translation units, to prove cross-object linking: `demo_util.c` provides a leaf routine, and `demo_main.c` drives the arena and I/O.

⟨ *Runtime-linux: demo util* 573b ⟩ ≡

```
/* demo_util.c - a leaf called across a TU
   boundary. */
long demo_sum(const long *a, long n) {
    long s = 0, i = 0;
    while (i < n) { s = s + a[i]; i = i + 1; }
    return s;
}
```

□

573b · used in 37g

⟨ *Runtime-linux: demo main* 573c ⟩ ≡

```
/* demo_main.c - arena via mmap, cross-TU call,
   output, distinctive exit. */
extern long kcc_sys_write(long fd, const void *buf
   , long len);
extern void *kcc_sys_mmap(void *addr, long len,
   int prot, int flags,
                          int fd, long off);
extern long demo_sum(const long *a, long n);

static unsigned long slen(const char *s) {
    unsigned long n = 0;
    while (s[n]) n++;
    return n;
}

int main(void) {
    /* 0x1002 = the arena's macOS MAP_PRIVATE|
   MAP_ANON; the runtime translates it. */
    long *p = (long *)kcc_sys_mmap(0, 1 << 16, 3,
   0x1002, -1, 0);
    if ((long)p == -1 || p == 0) return 1;

    long i = 0;
    while (i < 6) { p[i] = i + 1; i = i + 1; }
   /* 1+2+3+4+5+6 = 21 */

    const char *msg = "kcc cross-compiled to linux
   /x86-64, running native\n";
    kcc_sys_write(1, msg, slen(msg));

    long got = demo_sum(p, 6) + demo_sum(p, 6);
   /* 21 + 21 = 42 */
    return (int)got;
}
```

□

573c · used in 37h

⟨ *Runtime-linux: meta-a verify* 573d ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make meta-a - cross-compile a multi-TU C program
    to Linux/x86-64 with kcc,
# link it against the Linux runtime (no libc), and
    run it natively.
#
# kcc runs on the (ARM64/macOS) build host and
   emits ELF-flavour .s; the
# assemble/link/run happens on Linux, here inside
   a linux/amd64 container.
# Apple Silicon's Docker uses Rosetta, whose ELF
   loader rejects a valid
# non-page-aligned RW segment ("bss_size overflow"
   ); we run under qemu-x86_64,
# a conforming loader, which is what a real Linux
   kernel behaves like.
set -euo pipefail
```

```
R=$BASE/runtime
KCC=$BASE/kcc

echo "[1/3] kcc compiles the demo TUs -> Linux/x86
   -64 .s"
"$KCC" -c "$R/demo_util.c" -o "$R/demo_util.s" --
   target=x86_64-linux >/dev/null
"$KCC" -c "$R/demo_main.c" -o "$R/demo_main.s" --
   target=x86_64-linux >/dev/null

echo "[2/3] assemble + link + run inside linux/
   amd64 (qemu loader)"
docker run --rm -v "$PWD/$R":/w -w /w --platform
   linux/amd64 gcc:latest bash -c '
  set -e
  apt-get update -qq >/dev/null 2>&1 && apt-get
   install -y -qq qemu-user >/dev/null 2>&1
  as --64 os_linux.s   -o os_linux.o
  as --64 demo_util.s  -o demo_util.o
  as --64 demo_main.s  -o demo_main.o
  ld -static -nostdlib -e _start -o demo os_linux.
   o demo_util.o demo_main.o
  file demo
  set +e; qemu-x86_64 ./demo; code=$?; set -e
  echo "exit=$code (expect 42)"
  [ "$code" -eq 42 ] && echo "META_A_PASS" || {
   echo "META_A_FAIL"; exit 1; }
'

echo "[3/3] done"
```

□

573d · used in 37i

39.8. kcc Self-Hosts on Linux

The runtime above lets a program kcc compiled run on Linux; the natural question is whether *kcc itself* is such a program. It is. The milestone ladder has three rungs, each strictly stronger than the last. *Meta-A* (§39.7) shows kcc's *output* runs on Linux. *Meta-B* shows kcc *runs* on Linux: cross-compile every one of kcc's now-all-C subsystems under `--target=x86_64-linux`, assemble and statically link them with the runtime and no libc, and the resulting ELF lexes, parses, checks and compiles C under `qemu-x86_64`. *Meta-C*, this section, shows kcc *self-hosts* on Linux: that Linux binary, call it `kcc_linux`, compiles kcc's own source into a second-generation compiler that is itself correct.

The bootstrap is deliberately simple. `kcc_linux` compiles each of its twenty-nine C modules (the lexer slot is the compiler's real lexer, not the standalone tokenizer) under `--target=x86_64-linux`; the host `as` and `ld` assemble and link them, again static and libc-free, into `kcc_linux2`. The second generation compiles `hello.c` to exit 42, a floating-point program to its expected value — exercising the SSE rung the `x86-64` back end now carries in C — and a character constant to its true code. Crucially, `kcc_linux` and `kcc_linux2` emit *byte-identical* assembly for every module: the compiler compiled by itself reproduces itself. That is the fixed point that distinguishes a self-hosting compiler from one that merely runs.

Reaching the fixed point surfaced two bugs, both invisible until the whole compiler was kcc-compiled *for* `x86-64` rather than `arm64`. The first was a small-struct return-ABI mismatch in the character-literal path (§on primary expressions): a two-word `struct` received through a `void *` declaration, which reads the first return register. On `arm64` that register holds the first field; the `x86-64` back end returns such a struct as a pointer to it, so the character's value became the low byte of an address. Receiving the real struct fixes it under either convention.

The second was subtler and only bit once *two* kcc-compiled objects were linked together: the self-compiled compiler wrote `.globl` with two spaces before each name. The cause was a linkage error, not a codegen one. A file-scope `static` array was reserved with a bare `.comm`, which on ELF is an *external* common: the linker coalesces same-named commons from different objects into a single cell. Both back ends happen to name a directive string `s_globl` (one `".globl "`, the other `"
t.globl "`), so the two distinct statics fused into one, and their initialisers—run as code from `.init_array`—overwrote each other, leaving the survivor without its terminating `NUL`. The fix belongs to the data emitter (§on global data): a `static` common is marked `.local` first, restoring the internal linkage the language requires. With both fixed, the self-compiled compiler's output is *byte-identical* to the cross-compiler's for every module—the strongest form of the fixed point.

Because the *initial* bootstrap needs an `x86-64` Linux toolchain and a conforming ELF loader — a `linux/amd64` container plus `qemu-x86_64` on an Apple-Silicon host — it runs as a manual procedure rather than a `make` target, in the spirit of the Meta-A verifier above but one rung higher.

Once kcc exists on Linux, though, it needs no scaffolding of its own. The driver detects its host (`os_linux`'s `_start` sets `kcc_host_linux`) and, on a Linux host, drives the assembler and linker itself: it assembles each `.s` with `as --64`, emits and assembles the embedded runtime (§on the driver's link phase), and links statically with no libc. So `kcc foo.c -o foo` turns source into a running executable in a single command, with no external runtime files — and the runtime it embeds carries the same `kcc_host_linux` hook, so a kcc that kcc builds is itself a one-command Linux self-host. The whole ladder therefore closes on itself: after the one-time cross-compile, the compiler reproduces and re-hosts itself on Linux unaided.

39.9. Verification: The Automated Self-Host

`make meta-c` performs the whole loop end to end. It cross-compiles every one of kcc's own C modules to Linux assembly on the build host, then, inside a `linux/amd64` container under `qemu-x86_64`, assembles and links them into a native `kcc_linux`. It then exercises the one-command driver: `kcc_linux` turns a source file into a running executable in a single step (checked by its exit status), builds a *second* compiler from all twenty-nine modules in one command, and finally has *that* compiler build and run a program — proving the self-host is recursive. A green run prints `META_C_PASS`. Like the Meta-A verifier it needs Docker and `qemu-user`, so it is a milestone check rather than part of the default suite.

⟨ *Runtime-linux: meta-c verify* 574 ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make meta-c - cross-compile kcc's own source to
   Linux, link it, and prove the
# Linux-native compiler self-hosts (and re-hosts
   itself) in a single command.
set -euo pipefail

KCC=$BASE/kcc
S=$BASE/selfhost
M=$BASE/metac
mkdir -p "$M"

mods="arena strings io hashtab vec strtab source
   lexer_tables diag preprocessor \
parser expressions statements declarations types
   symtab conversions sema ir opt \
emit_arm64 emit_x86 linker codegen emit_macho
   dwarf driver main"

echo "[1/3] cross-compile kcc's own source ->
   Linux/x86-64 .s (+ stage .c)"
for m in $mods; do
  "$KCC" -c "$S/$m.c" --target=x86_64-linux -o "$M
   /$m.s" >/dev/null
  cp "$S/$m.c" "$M/$m.c"
done
"$KCC" -c "$S/lexer_core.c" --target=x86_64-linux
   -o "$M/lexer.s" >/dev/null
cp "$S/lexer_core.c" "$M/lexer.c"
cp $BASE/runtime/os_linux.s "$M/os_linux.s"
printf 'int main(void){return 42;}\n' > "$M/hello.
   c"

echo "[2/3] link kcc_linux and self-host inside
   linux/amd64 (qemu loader)"
docker run --rm -v "$PWD/$M":/w -w /w --platform
   linux/amd64 gcc:latest bash -c '
  set -e
  for i in 1 2 3; do
    apt-get update -qq >/dev/null 2>&1 && apt-get
   install -y -qq qemu-user >/dev/null 2>&1 &&
   break
    sleep 2
  done
  ALL="os_linux arena strings io hashtab vec
   strtab source lexer lexer_tables diag \
preprocessor parser expressions statements
   declarations types symtab conversions \
sema ir opt emit_arm64 emit_x86 linker codegen
   emit_macho dwarf driver main"
  for m in $ALL; do as --64 -o "$m.o" "$m.s"; done
  OBJ="main.o arena.o strings.o io.o hashtab.o vec
   .o strtab.o source.o lexer.o \
lexer_tables.o diag.o preprocessor.o parser.o
   expressions.o statements.o \
declarations.o types.o symtab.o conversions.o sema
   .o ir.o opt.o emit_arm64.o \
emit_x86.o linker.o codegen.o emit_macho.o dwarf.o
    driver.o"
  ld -static -nostdlib -e _start -o kcc_linux
   os_linux.o $OBJ
  echo "  built $(file kcc_linux | cut -d, -f1-2)"

  echo "  one command: kcc_linux hello.c -o hello"
  qemu-x86_64 ./kcc_linux hello.c --target=x86_64-
   linux -o hello
  set +e; qemu-x86_64 ./hello; c=$?; set -e
  [ "$c" -eq 42 ] || { echo "META_C_FAIL: hello=$c
    (want 42)"; exit 1; }

  echo "  capstone: kcc_linux builds kcc_linux3
   (29 modules) in one command"
  SRC="main.c arena.c strings.c io.c hashtab.c vec
   .c strtab.c source.c lexer.c \
lexer_tables.c diag.c preprocessor.c parser.c
   expressions.c statements.c \
declarations.c types.c symtab.c conversions.c sema
   .c ir.c opt.c emit_arm64.c \
emit_x86.c linker.c codegen.c emit_macho.c dwarf.c
    driver.c"
  qemu-x86_64 ./kcc_linux $SRC --target=x86_64-
   linux -o kcc_linux3

  echo "  recursive: kcc_linux3 builds and runs
   hello in one command"
  qemu-x86_64 ./kcc_linux3 hello.c --target=x86_64
   -linux -o hello3
  set +e; qemu-x86_64 ./hello3; c=$?; set -e
  [ "$c" -eq 42 ] || { echo "META_C_FAIL:
   recursive hello3=$c (want 42)"; exit 1; }

  echo "META_C_PASS"
'
echo "[3/3] done"
```

□

574 · used in 37j

577a · used in 31d

40. Mach-O Emission

Once all machine code has been emitted and relocations recorded, the compiler must package the result into a file format that the system linker can consume. This chapter writes a minimal Mach-O 64-bit object file (`MH_OBJECT`) following Apple's conventions. The output contains a `__TEXT,__text` section holding the generated code, a symbol table of defined and external symbols, relocation entries for branch and data-reference instructions, a string table for symbol names, and an `LC_BUILD_VERSION` load command to suppress linker warnings.

40.1. Constants

The constants define Mach-O magic numbers, CPU type codes, load command identifiers, and structure sizes according to the `mach-o/loader.h` and `mach-o/nlist.h` headers. All values are for the 64-bit ARM64 object file format (`MH_OBJECT`).

⟨ *MachO: constants* 577a ⟩ ≡

```
// -- Mach-O constants ---
.equ MH_MAGIC_64,                0xFEEDFACF
.equ CPU_TYPE_ARM64,             0x0100000C   //
   CPU_TYPE_ARM64
.equ CPU_SUBTYPE_ALL,            0x00000000
.equ MH_OBJECT,                  0x01
.equ MH_SUBSECTIONS_VIA_SYMBOLS, 0x2000

// Load commands
.equ LC_SEGMENT_64,              0x19
.equ LC_SYMTAB,                  0x02
.equ LC_DYSYMTAB,                0x0B
.equ LC_BUILD_VERSION,           0x32

// Sizes
.equ MACH_HEADER_SIZE,           32
.equ SEGMENT_CMD_SIZE,           72
.equ SECTION_64_SIZE,            80
.equ SYMTAB_CMD_SIZE,            24
.equ DYSYMTAB_CMD_SIZE,          80
.equ BUILD_VERSION_CMD_SIZE,     24
.equ NLIST_64_SIZE,              16
.equ RELOC_INFO_SIZE,            8

// Section types/attributes
.equ S_REGULAR,                  0x00
.equ S_MOD_INIT_FUNC_POINTERS,   0x09
.equ S_ATTR_PURE_INSTRUCTIONS,   0x80000000
.equ S_ATTR_SOME_INSTRUCTIONS,   0x00000400
.equ ARM64_RELOC_UNSIGNED,       0x00

// Relocation info bit packing
// r_info = r_symbolnum(24) | r_pcrel(1) |
   r_length(2) | r_extern(1) | r_type(4)
// For ARM64_RELOC_BRANCH26: pcrel=1, length=2 (4
   bytes), extern=1, type=2

// Output buffer
.equ MACHO_BUF_SIZE,             65536
```

□

40.2. Output Buffer Management

Rather than seeking and writing pieces of the file in random order, we build the entire object file in one arena-allocated buffer and flush it in a single `write` system call. A *fixed* 64 KB buffer was the original design, but that silently overflowed on large inputs—a deeply nested or very long function easily emits more than 64 KB of code, and the writers would then `memcpy` past the buffer and crash. Because `kcc_macho_init` runs *after* code generation and string-table emission, every unbounded contributor is already known, so it sizes the buffer to the content: the code size, the DWARF blob, `nsyms` entries at a generous 288-byte budget each (the 16-byte `nlist_64` plus a 256-byte name allowance), the relocations, and a fixed 64 KB of slack for the header, load commands, and alignment. Header, symbol table, and relocations therefore always fit; the sole residual risk—a symbol name longer than the per-symbol budget, copied through `write_bytes`—is caught by an explicit capacity guard (`_kcc_macho_cap`) that turns an oversized object into a clean diagnostic instead of memory corruption. The helpers `write8/16/32/64` append little-endian values; `write_bytes` copies raw data (and holds the guard); `pad_to` inserts zero bytes until the write position is aligned to the given power of two.

⟨ *MachO: output buffer* 577b ⟩ ≡

```
// -- kcc_macho_init ---
// void kcc_macho_init(const char *filename)
//   x0 = output file path
// Opens the file for writing and allocates the
   output buffer.
.text
.globl _kcc_macho_init
.p2align 2
_kcc_macho_init:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                    // save
   filename

    // Open file: O_WRONLY|O_CREAT|O_TRUNC = 0x601
    mov x0, x19
    mov x1, #0x601
    mov x2, #0644
    movk x2, #0, lsl #16           // mode 0644
   octal = 420 decimal
    mov x2, #420
    bl _kcc_sys_open
    // Check for error
    cmp x0, #0
    b.lt .Lmi_err

    adrp x1, _kcc_macho_fd@PAGE
    add  x1, x1, _kcc_macho_fd@PAGEOFF
    str x0, [x1]

    // Size the buffer to the content (see prose):
    code + dwarf
    // + nsyms*288 + nrelocs*8 + 64 KiB slack.
    bl   _kcc_dwarf_total_len      // x0 = DWARF
   bytes (0 unless -g); clobbers x0-x1
```

```
    mov  x19, x0                    // reuse x19 (
   filename no longer needed)
    adrp x0, _kcc_code_pos@PAGE
    add  x0, x0, _kcc_code_pos@PAGEOFF
    ldr  x0, [x0]                   // code size
    add  x19, x19, x0               // x19 = code +
    dwarf
    adrp x1, _kcc_linker_syms@PAGE
    add  x1, x1, _kcc_linker_syms@PAGEOFF
    ldr  x1, [x1]
    ldr  x1, [x1, #VEC_COUNT]       // nsyms
    mov  x2, #288
    madd x19, x1, x2, x19           // + nsyms*288
    adrp x1, _kcc_linker_relocs@PAGE
    add  x1, x1, _kcc_linker_relocs@PAGEOFF
    ldr  x1, [x1]
    ldr  x1, [x1, #VEC_COUNT]       // nrelocs
    add  x19, x19, x1, lsl #3       // + nrelocs*8
    add  x19, x19, #MACHO_BUF_SIZE  // + 64 KiB
   fixed slack (floors tiny objects)
    mov  x0, x19
    bl _kcc_arena_alloc
    adrp x2, _kcc_macho_buf@PAGE
    add  x2, x2, _kcc_macho_buf@PAGEOFF
    str x0, [x2]
    adrp x2, _kcc_macho_cap@PAGE
    add  x2, x2, _kcc_macho_cap@PAGEOFF
    str x19, [x2]                   // record
   capacity for the overflow guard

    // Reset position
    adrp x1, _kcc_macho_pos@PAGE
    add  x1, x1, _kcc_macho_pos@PAGEOFF
    str xzr, [x1]

    b .Lmi_done

.Lmi_err:
    adrp x0, _kcc_macho_err_open@PAGE
    add  x0, x0, _kcc_macho_err_open@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lmi_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

577b · defined at 577b, 578, 579 · used in 31d · defines _kcc_macho_init 603 · uses _kcc_arena_alloc 58a, _kcc_code_pos 533c, _kcc_io_err 71b, _kcc_linker_relocs 594, _kcc_linker_syms 594, _kcc_macho_buf 590, _kcc_macho_cap 590, _kcc_macho_fd 590, _kcc_macho_pos 590

Primitive Integer Writers The next four functions—_kcc_macho_write8, _kcc_macho_write16, _kcc_macho_write32, and _kcc_macho_write64—each store a single little-endian value at the current buffer position and advance the position counter by the appropriate number of bytes. They share an identical pattern: load the buffer base from _kcc_macho_buf, load the current offset from _kcc_macho_pos, perform a sized store (strb/strh/str w/str x), and bump the offset. Because they never call another function, they are leaf routines with no stack frame.

⟨ *MachO: output buffer* 578 ⟩ +≡

```
// -- kcc_macho_write8 ---
.text
.globl _kcc_macho_write8
.p2align 2
_kcc_macho_write8:
    adrp x1, _kcc_macho_buf@PAGE
    add  x1, x1, _kcc_macho_buf@PAGEOFF
    ldr x1, [x1]
    adrp x2, _kcc_macho_pos@PAGE
    add  x2, x2, _kcc_macho_pos@PAGEOFF
    ldr x3, [x2]
    strb w0, [x1, x3]
    add x3, x3, #1
    str x3, [x2]
    ret

// -- kcc_macho_write16 ---
.text
.globl _kcc_macho_write16
.p2align 2
_kcc_macho_write16:
    adrp x1, _kcc_macho_buf@PAGE
    add  x1, x1, _kcc_macho_buf@PAGEOFF
    ldr x1, [x1]
    adrp x2, _kcc_macho_pos@PAGE
    add  x2, x2, _kcc_macho_pos@PAGEOFF
    ldr x3, [x2]
    strh w0, [x1, x3]
    add x3, x3, #2
    str x3, [x2]
    ret

// -- kcc_macho_write32 ---
.text
.globl _kcc_macho_write32
.p2align 2
_kcc_macho_write32:
    adrp x1, _kcc_macho_buf@PAGE
    add  x1, x1, _kcc_macho_buf@PAGEOFF
    ldr x1, [x1]
    adrp x2, _kcc_macho_pos@PAGE
    add  x2, x2, _kcc_macho_pos@PAGEOFF
    ldr x3, [x2]
    str w0, [x1, x3]
    add x3, x3, #4
    str x3, [x2]
    ret

// -- kcc_macho_write64 ---
.text
.globl _kcc_macho_write64
.p2align 2
_kcc_macho_write64:
    adrp x1, _kcc_macho_buf@PAGE
    add  x1, x1, _kcc_macho_buf@PAGEOFF
    ldr x1, [x1]
    adrp x2, _kcc_macho_pos@PAGE
    add  x2, x2, _kcc_macho_pos@PAGEOFF
    ldr x3, [x2]
    str x0, [x1, x3]
    add x3, x3, #8
    str x3, [x2]
    ret
```

□

578 · defined at 577b, 578, 579 · used in 31d · defines _kcc_macho_write16 588, 589a, _kcc_macho_write32 582, 583, 585, 587, 588, 589a, 600, _kcc_macho_write64 582, 583, 586, 588, 589a, 600, _kcc_macho_write8 588, 589ab · uses _kcc_macho_buf 590, _kcc_macho_pos 590

Bulk Data Copy and Alignment Padding Two more routines complete the buffer API. _kcc_macho_write_bytes copies an arbitrary byte range into the output buffer using _kcc_memcpy and advances the position by the source length; it is used to emit raw code bytes and symbol-name strings. _kcc_macho_pad_to inserts zero bytes until the write position reaches the next multiple of a given alignment—needed because Mach-O relocations must be 4-byte aligned and the code section size is not guaranteed to be a multiple of 4.

⟨ *MachO: output buffer* 579 ⟩ + ≡

```
// -- kcc_macho_write_bytes ---
// void kcc_macho_write_bytes(const void *src,
   uint64_t len)
.text
.globl _kcc_macho_write_bytes
.p2align 2
_kcc_macho_write_bytes:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x1                     // len

    adrp x2, _kcc_macho_buf@PAGE
    add  x2, x2, _kcc_macho_buf@PAGEOFF
    ldr x2, [x2]
    adrp x3, _kcc_macho_pos@PAGE
    add  x3, x3, _kcc_macho_pos@PAGEOFF
    ldr x4, [x3]

    // Overflow guard (see prose): pos + len must
   fit within capacity.
    adrp x5, _kcc_macho_cap@PAGE
    add  x5, x5, _kcc_macho_cap@PAGEOFF
    ldr x5, [x5]
    add x6, x4, x19
    cmp x6, x5
    b.hi .Lmacho_overflow

    // memcpy(buf+pos, src, len)
    add x5, x2, x4                  // dest = buf +
   pos
    // x0 = src, x19 = len
    mov x2, x19
    mov x1, x0
    mov x0, x5
    bl _kcc_memcpy

    // Update pos
    adrp x3, _kcc_macho_pos@PAGE
    add  x3, x3, _kcc_macho_pos@PAGEOFF
    ldr x4, [x3]
    add x4, x4, x19
    str x4, [x3]

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// Shared overflow bailout: the object file did
   not fit the sized buffer.
// Reached only from the write_bytes guard; prints
    a diagnostic and exits so a
// pathological input (e.g. an identifier longer
   than the per-symbol budget)
// fails cleanly rather than corrupting memory.
.Lmacho_overflow:
    adrp x0, _kcc_macho_err_overflow@PAGE
    add  x0, x0, _kcc_macho_err_overflow@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

// -- kcc_macho_pad_to ---
// void kcc_macho_pad_to(uint64_t alignment)
// Pads with zeros until position is aligned to
   given boundary.
.text
.globl _kcc_macho_pad_to
.p2align 2
_kcc_macho_pad_to:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x1, _kcc_macho_pos@PAGE
    add  x1, x1, _kcc_macho_pos@PAGEOFF
    ldr x2, [x1]

    // Compute padding needed
    sub x3, x0, #1                  // alignment - 1
    add x4, x2, x3
    bic x4, x4, x3                  // aligned pos
    sub x5, x4, x2                  // bytes to pad
    cbz x5, .Lmpt_done

    // Write zeros
    adrp x1, _kcc_macho_buf@PAGE
    add  x1, x1, _kcc_macho_buf@PAGEOFF
    ldr x1, [x1]
    add x1, x1, x2                  // buf +
   current_pos

.Lmpt_loop:
    cbz x5, .Lmpt_update
    strb wzr, [x1], #1
    sub x5, x5, #1
    b .Lmpt_loop

.Lmpt_update:
    adrp x1, _kcc_macho_pos@PAGE
    add  x1, x1, _kcc_macho_pos@PAGEOFF
    str x4, [x1]

.Lmpt_done:
    ldp x29, x30, [sp], #16
    ret
```

□

579 · defined at 577b, 578, 579 · used in 31d · defines _kcc_macho_pad_to 586, _kcc_macho_write_bytes 586, 589b, 600 · uses _kcc_io_err 71b, _kcc_macho_buf 590, _kcc_macho_cap 590, _kcc_macho_pos 590

40.3. Finalize and Write

The finalize function is the heart of the Mach-O writer. It computes the file layout, writes every structure in order, and flushes the result to disk.

The file layout for our object files is:

```
+0     : mach_header_64          (32 bytes)
+32    : LC_SEGMENT_64           (72 + 80*nsects)
         LC_BUILD_VERSION        (24 bytes)
         LC_SYMTAB               (24 bytes)
         LC_DYSYMTAB             (80 bytes)
+hdrsz: __text section data      (code_size bytes)
         padding to 4-byte align
         relocation entries       (8 bytes each)
         nlist_64 symbol table   (16 bytes each)
         string table            (variable)
```

We emit two sections: `__TEXT,__text` (executable code plus trailing string data) and `__DATA,__common` (a `S_ZEROFILL` section for global variables, which occupies no file space).

40.3.1. Layout Computation

The first phase reads the linker tables to determine symbol counts, string table size, and the byte offsets of every structure in the file. These offsets are stored in callee-saved registers and stack slots for use by subsequent phases.

⟨ *MachO: finalize* 580 ⟩ ≡

```
// -- kcc_macho_finalize ---
// void kcc_macho_finalize(void)
// Assembles and writes the complete Mach-O object
    file.
.text
.globl _kcc_macho_finalize
.p2align 2
_kcc_macho_finalize:
    stp x29, x30, [sp, #-128]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    stp x27, x28, [sp, #80]

    // --- Compute layout sizes ---
    // Get code size
    adrp x0, _kcc_code_pos@PAGE
    add  x0, x0, _kcc_code_pos@PAGEOFF
    ldr x19, [x0]                   // x19 =
   code_size

    // Get symbol count
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    ldr x20, [x0, #VEC_COUNT]      // x20 = nsyms

    // Get reloc count
    adrp x0, _kcc_linker_relocs@PAGE
    add  x0, x0, _kcc_linker_relocs@PAGEOFF
    ldr x0, [x0]
    ldr x21, [x0, #VEC_COUNT]      // x21 = nrelocs

    // Get globals size
    adrp x0, _kcc_globals_size@PAGE
    add  x0, x0, _kcc_globals_size@PAGEOFF
    ldr x0, [x0]
    str x0, [sp, #112]              // sp+112 =
   globals_size

    // Compute load commands size:
    // LC_SEGMENT_64 (72 + 80*2 = 232) +
   LC_BUILD_VERSION (24) + LC_SYMTAB (24) +
   LC_DYSYMTAB (80) = 360
    // Base layout: 2 sections (__text + __DATA,
   __common zerofill).
    mov x22, #(232 + 24 + 24 + 80)  // = 360
    // A module-init unit adds a third section (
   __DATA,__mod_init_func), i.e.
    // one more 80-byte section_64 inside the same
    segment (no extra command).
    adrp x0, _kcc_modinit_present@PAGE
    add  x0, x0, _kcc_modinit_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_modinit_lc
    add  x22, x22, #80
.Lmf_no_modinit_lc:
    // A const-pointer table adds a __DATA_CONST,
   __const section (one more
    // 80-byte section_64 inside the same segment)
   .
    adrp x0, _kcc_rodata_const_present@PAGE
    add  x0, x0, _kcc_rodata_const_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_const_lc
    add  x22, x22, #80
.Lmf_no_const_lc:
    // Under -g, reserve a __DWARF LC_SEGMENT_64
   (72 + 80*3 sections = 312).
    // Adding it to x22 here auto-shifts every
   downstream file offset.
    adrp x0, _kcc_dwarf_enabled@PAGE
    add  x0, x0, _kcc_dwarf_enabled@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_dwarf_seg
    add  x22, x22, #(72 + 80*3)
.Lmf_no_dwarf_seg:

    // Section data offset = header + load cmds
    add x23, x22, #MACH_HEADER_SIZE  // x23 =
   text_offset (header + LCs)

    // Relocation offset = text_offset + code_size
    (aligned to 4).  Under -g
    // the DWARF blob is laid down right after the
    code, so its span is
    // reserved here - this keeps LINKEDIT (relocs
   /symtab/strtab) last, as ld
    // requires, and auto-shifts every downstream
   file offset past the blob.
    add x24, x23, x19
    adrp x0, _kcc_dwarf_enabled@PAGE
    add  x0, x0, _kcc_dwarf_enabled@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_dw_reserve
    bl   _kcc_dwarf_total_len
    add  x24, x24, x0
.Lmf_no_dw_reserve:
    // Reserve the 8-byte __mod_init_func pointer
   (8-aligned) between the code
    // (and any DWARF blob) and the LINKEDIT
   tables, and remember its offset.
    adrp x0, _kcc_modinit_present@PAGE
    add  x0, x0, _kcc_modinit_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_modinit_off
    add x24, x24, #7
    bic x24, x24, #7                // 8-align →
   pointer file offset
    adrp x0, _kcc_modinit_fileoff@PAGE
    add  x0, x0, _kcc_modinit_fileoff@PAGEOFF
    str x24, [x0]
    add x24, x24, #8                // reserve the
   pointer's 8 bytes
.Lmf_no_modinit_off:
    // Reserve the __const section's data (
   const_size bytes, 8-aligned) after the
    // modinit pointer; remember its file offset.
    adrp x0, _kcc_rodata_const_present@PAGE
    add  x0, x0, _kcc_rodata_const_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_const_off
    add x24, x24, #7
    bic x24, x24, #7
    adrp x0, _kcc_rodata_const_fileoff@PAGE
    add  x0, x0, _kcc_rodata_const_fileoff@PAGEOFF
    str x24, [x0]
    adrp x0, _kcc_rodata_const_size@PAGE
    add  x0, x0, _kcc_rodata_const_size@PAGEOFF
```

```
    ldr x0, [x0]
    add x24, x24, x0              // reserve the
   const data
.Lmf_no_const_off:
    add x24, x24, #3
    bic x24, x24, #3              // x24 =
   reloc_offset (4-byte aligned)

    // Symbol table offset = reloc_offset +
   nrelocs * 8
    mov x25, #RELOC_INFO_SIZE
    mul x25, x21, x25
    add x25, x24, x25             // x25 =
   symtab_offset
    // A module-init unit appends one extra
   relocation (the pointer's UNSIGNED).
    adrp x0, _kcc_modinit_present@PAGE
    add  x0, x0, _kcc_modinit_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_modinit_reloc_sz
    add x25, x25, #RELOC_INFO_SIZE
.Lmf_no_modinit_reloc_sz:
    // The __const section appends one UNSIGNED
   reloc per non-NULL slot.
    adrp x0, _kcc_rodata_const_present@PAGE
    add  x0, x0, _kcc_rodata_const_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_const_reloc_sz
    adrp x0, _kcc_rodata_const_relocs@PAGE
    add  x0, x0, _kcc_rodata_const_relocs@PAGEOFF
    ldr x0, [x0]
    ldr x0, [x0, #VEC_COUNT]      // number of
   const relocs
    mov x1, #RELOC_INFO_SIZE
    mul x0, x0, x1
    add x25, x25, x0
.Lmf_no_const_reloc_sz:

    // String table offset = symtab_offset + nsyms
    * 16
    mov x26, #NLIST_64_SIZE
    mul x26, x20, x26
    add x26, x25, x26             // x26 =
   strtab_offset

    // Compute string table size (sum of all
   symbol name lengths + 1 for each + 1 for
   leading NUL)
    // We'll need to build the strtab as we write
   symbols
    // For now, compute it
    mov x27, #1                    // start at 1 (
   leading NUL byte)
    mov x28, #0                    // symbol index
.Lmf_strtab_size:
    cmp x28, x20
    b.ge .Lmf_strtab_done
    // Get symbol entry
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    mov x1, x28
    bl _kcc_vec_get
    ldr x0, [x0, #SYM_NAME]       // name ptr
    bl _kcc_strlen
    add x27, x27, x0
    add x27, x27, #1              // +1 for null
   terminator
    add x28, x28, #1
    b .Lmf_strtab_size
.Lmf_strtab_done:
    // x27 = strtab_size
```

□

580 · defined at 580, 581, 582, 583, 585, 586, 587, 588, 589ab · used in 31d · defines _kcc_macho_finalize 603 · uses _kcc_code_pos 533c, _kcc_globals_size 594, _kcc_linker_relocs 594, _kcc_linker_syms 594, _kcc_modinit_fileoff 305c, _kcc_modinit_present 305c, _kcc_rodata_const_fileoff 479, _kcc_rodata_const_present 479, _kcc_rodata_const_relocs 479, _kcc_rodata_const_size 479, _kcc_vec_get 65b

Symbol Classification for LC_DYSYMTAB The dynamic symbol table command requires the counts of defined external and undefined external symbols. Mach-O mandates that defined symbols precede undefined ones in the symbol table, so we walk the linker's symbol vector and count how many have a non-zero SYM_SECTION (defined) versus zero (undefined). The defined count is stored in x28; the undefined count is derived later by subtraction from the total x20.

⟨ *MachO: finalize* 581 ⟩ + ≡

```
    // Count defined vs undefined symbols for
   dysymtab
    mov x28, #0                    // index
    str xzr, [sp, #96]            // nlocal = 0
    str xzr, [sp, #104]           // nextdef = 0 (
   we count defined externals)
.Lmf_count_syms:
    cmp x28, x20
    b.ge .Lmf_count_done
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    mov x1, x28
    bl _kcc_vec_get
    ldr x1, [x0, #SYM_FLAGS]
    // If section != 0, it's defined
    ldr x2, [x0, #SYM_SECTION]
    cbz x2, .Lmf_sym_undef
    // Defined external
    ldr x3, [sp, #104]
    add x3, x3, #1
    str x3, [sp, #104]
    b .Lmf_sym_next
.Lmf_sym_undef:
    // Undefined - counted by subtraction later
.Lmf_sym_next:
    add x28, x28, #1
    b .Lmf_count_syms
.Lmf_count_done:
    ldr x28, [sp, #104]           // nextdef (
   defined external count)
```

□

581 · defined at 580, 581, 582, 583, 585, 586, 587, 588, 589ab · used in 31d · uses _kcc_linker_syms 594, _kcc_vec_get 65b

40.3.2. Header and Load Commands

With all offsets computed, we write the Mach-O header (magic, CPU type, file type, load command count and size) followed by the four load commands: LC_SEGMENT_64 (containing the __text and __common section headers), LC_BUILD_VERSION (suppresses Apple linker warnings), LC_SYMTAB, and LC_DYSYMTAB.

Section names (__text, __TEXT, etc.) are written as inline immediate bytes rather than referencing string

constants—this avoids the linker dead-stripping trailing null bytes from local atoms, which caused subtle bugs in earlier versions.

⟨ *MachO: finalize* 582 ⟩ + ≡

```
    // --- Write Mach-O header ---
    // magic
    mov w0, #0xFACF
    movk w0, #0xFEED, lsl #16
    bl _kcc_macho_write32
    // cputype
    mov w0, #0x000C
    movk w0, #0x0100, lsl #16
    bl _kcc_macho_write32
    // cpusubtype
    mov w0, #0
    bl _kcc_macho_write32
    // filetype = MH_OBJECT
    mov w0, #MH_OBJECT
    bl _kcc_macho_write32
    // ncmds = 4 (segment + build_version + symtab
    + dysymtab), +1 __DWARF under -g
    mov w0, #4
    adrp x1, _kcc_dwarf_enabled@PAGE
    add  x1, x1, _kcc_dwarf_enabled@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_ncmds_done
    add  w0, w0, #1
.Lmf_ncmds_done:
    bl _kcc_macho_write32
    // sizeofcmds
    mov w0, w22
    bl _kcc_macho_write32
    // flags = MH_SUBSECTIONS_VIA_SYMBOLS
    mov w0, #MH_SUBSECTIONS_VIA_SYMBOLS
    bl _kcc_macho_write32
    // reserved (64-bit padding)
    mov w0, #0
    bl _kcc_macho_write32

    // --- LC_SEGMENT_64 ---
    // cmd
    mov w0, #LC_SEGMENT_64
    bl _kcc_macho_write32
    // cmdsize = 72 + 80*nsects.  Base 232 (2
   sections); +80 for
    // __mod_init_func, +80 for __DATA_CONST,
   __const.
    mov w0, #232
    adrp x1, _kcc_modinit_present@PAGE
    add  x1, x1, _kcc_modinit_present@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_cmdsz_nomod
    add  w0, w0, #80
.Lmf_cmdsz_nomod:
    adrp x1, _kcc_rodata_const_present@PAGE
    add  x1, x1, _kcc_rodata_const_present@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_seg_cmdsize
    add  w0, w0, #80
.Lmf_seg_cmdsize:
    bl _kcc_macho_write32
    // segname (16 bytes, empty for object files)
    mov x0, #0
    bl _kcc_macho_write64
    mov x0, #0
    bl _kcc_macho_write64
    // vmaddr
    mov x0, #0
    bl _kcc_macho_write64
    // vmsize = code_size + globals_size (+ the
   aligned 8-byte modinit pointer)
    ldr x0, [sp, #112]              // globals_size
    add x0, x0, x19                 // + code_size
    adrp x1, _kcc_modinit_present@PAGE
    add  x1, x1, _kcc_modinit_present@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_seg_vmsize
    add x0, x0, #7
    bic x0, x0, #7                  // 8-align the
   pointer's VM address
    add x0, x0, #8                  // + 8 bytes for
   the pointer
.Lmf_seg_vmsize:
    adrp x1, _kcc_rodata_const_present@PAGE
    add  x1, x1, _kcc_rodata_const_present@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_seg_vmsize_done
    add x0, x0, #7
    bic x0, x0, #7                  // 8-align the
   __const section's VM address
    adrp x1, _kcc_rodata_const_size@PAGE
    add  x1, x1, _kcc_rodata_const_size@PAGEOFF
    ldr x1, [x1]
    add x0, x0, x1                  // + const data
.Lmf_seg_vmsize_done:
    bl _kcc_macho_write64
    // fileoff = text_offset
    mov x0, x23
    bl _kcc_macho_write64
    // filesize = code_size (zerofill doesn't
   occupy file space); when a
    // __mod_init_func pointer follows the code,
   extend to cover its 8 bytes.
    mov x0, x19
    adrp x1, _kcc_modinit_present@PAGE
    add  x1, x1, _kcc_modinit_present@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_seg_filesize
    adrp x0, _kcc_modinit_fileoff@PAGE
    add  x0, x0, _kcc_modinit_fileoff@PAGEOFF
    ldr x0, [x0]
    add x0, x0, #8                  // end of the
   pointer
    sub x0, x0, x23                 // relative to
   the segment's fileoff
.Lmf_seg_filesize:
    adrp x1, _kcc_rodata_const_present@PAGE
    add  x1, x1, _kcc_rodata_const_present@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_seg_filesize_done
    adrp x1, _kcc_rodata_const_fileoff@PAGE
    add  x1, x1, _kcc_rodata_const_fileoff@PAGEOFF
    ldr x1, [x1]
    adrp x2, _kcc_rodata_const_size@PAGE
    add  x2, x2, _kcc_rodata_const_size@PAGEOFF
    ldr x2, [x2]
    add x1, x1, x2                  // end of the
   const data (furthest file byte)
    sub x0, x1, x23                 // relative to
   the segment's fileoff
.Lmf_seg_filesize_done:
    bl _kcc_macho_write64
    // maxprot = 7 (rwx)
    mov w0, #7
    bl _kcc_macho_write32
    // initprot = 7
    mov w0, #7
    bl _kcc_macho_write32
    // nsects = 2 + (__mod_init_func) + (
   __DATA_CONST,__const)
    mov w0, #2
    adrp x1, _kcc_modinit_present@PAGE
    add  x1, x1, _kcc_modinit_present@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_nsects_nomod
    add  w0, w0, #1
```

```
.Lmf_nsects_nomod:
    adrp x1, _kcc_rodata_const_present@PAGE
    add  x1, x1, _kcc_rodata_const_present@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_seg_nsects
    add  w0, w0, #1
.Lmf_seg_nsects:
    bl _kcc_macho_write32
    // flags = 0
    mov w0, #0
    bl _kcc_macho_write32
```

□

582 · defined at 580, 581, 582, 583, 585, 586, 587, 588, 589ab · used in 31d · uses _kcc_macho_write32 578, _kcc_macho_write64 578, _kcc_modinit_fileoff 305c, _kcc_modinit_present 305c, _kcc_rodata_const_fileoff 479, _kcc_rodata_const_present 479, _kcc_rodata_const_size 479

Section Headers Each section within the segment gets an 80-byte `section_64` descriptor. The `__TEXT,__text` section carries the executable machine code: its `offset` points to the raw bytes in the file, its `reloff`/`nreloc` reference the relocation entries, and its flags mark it as containing pure instructions. The `__DATA,__common` section is a `S_ZEROFILL` region for uninitialized global variables; it occupies virtual address space immediately after the text section but requires no file storage—the `offset` and `reloff` fields are both zero. Section and segment names are written as inline immediate constants to avoid linker dead-stripping issues.

A module-init unit (§29.1.1) contributes a third section, `__DATA,__mod_init_func`, whose 8-byte body is a single pointer to the local `kcc.module_init` function. Its `S_MOD_INIT_FUNC_POINTERS` flag tells `dyld` to call every pointer in the section before `main`. Unlike `__common` it carries real file data, so it has a non-zero `offset`; the pointer's value is supplied by a single `ARM64_RELOC_UNSIGNED` relocation against the function symbol, so the eight bytes on disk are zero. This whole section header is emitted only when `_kcc_modinit_present` is set, leaving the common two-section object byte-for-byte unchanged.

⟨ *MachO: finalize* 583 ⟩ + ≡

```
    // --- Section: __text ---
    // sectname (16 bytes): "__text" + 10 null
  bytes
    // Little-endian: "__text\0\0" = 0
  x0000747865745F5F
    mov x0, #0x5F5F
    movk x0, #0x6574, lsl #16
    movk x0, #0x7478, lsl #32
    bl _kcc_macho_write64
    mov x0, #0
    bl _kcc_macho_write64
    // segname (16 bytes): "__TEXT" + 10 null
  bytes
    // Little-endian: "__TEXT\0\0" = 0
  x0000545845545F5F
    mov x0, #0x5F5F
    movk x0, #0x4554, lsl #16
    movk x0, #0x5458, lsl #32
    bl _kcc_macho_write64
    mov x0, #0
    bl _kcc_macho_write64
    // addr = 0
    mov x0, #0
    bl _kcc_macho_write64
    // size = code_size
    mov x0, x19
    bl _kcc_macho_write64
    // offset = text_offset
    mov w0, w23
    bl _kcc_macho_write32
    // align = 2 (4-byte aligned, log2(4)=2)
    mov w0, #2
    bl _kcc_macho_write32
    // reloff
    mov w0, w24
    bl _kcc_macho_write32
    // nreloc
    mov w0, w21
    bl _kcc_macho_write32
    // flags = S_REGULAR |
  S_ATTR_PURE_INSTRUCTIONS |
  S_ATTR_SOME_INSTRUCTIONS
    mov w0, #S_ATTR_SOME_INSTRUCTIONS
    movk w0, #0x8000, lsl #16     //
  S_ATTR_PURE_INSTRUCTIONS
    bl _kcc_macho_write32
    // reserved1, reserved2, reserved3 (padding to
    80 bytes)
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32

    // --- Section: __DATA,__common (S_ZEROFILL)
  ---
    // sectname (16 bytes): "__common" + 8 null
  bytes
    // Little-endian: "__common" = 0
  x6E6F6D6D6F635F5F
    mov x0, #0x5F5F
    movk x0, #0x6F63, lsl #16
    movk x0, #0x6D6D, lsl #32
    movk x0, #0x6E6F, lsl #48
    bl _kcc_macho_write64
    mov x0, #0
    bl _kcc_macho_write64
    // segname (16 bytes): "__DATA" + 10 null
  bytes
    // Little-endian halfwords: "__"=5F5F "DA
  "=4144 "TA"=4154
    mov x0, #0x5F5F
    movk x0, #0x4144, lsl #16
    movk x0, #0x4154, lsl #32
    bl _kcc_macho_write64
    mov x0, #0
    bl _kcc_macho_write64
    // addr = code_size (follows text in virtual
  memory)
    mov x0, x19
    bl _kcc_macho_write64
    // size = globals_size
    ldr x0, [sp, #112]
    bl _kcc_macho_write64
    // offset = 0 (S_ZEROFILL: no file data)
    mov w0, #0
    bl _kcc_macho_write32
    // align = 3 (8-byte aligned, log2(8)=3)
    mov w0, #3
    bl _kcc_macho_write32
    // reloff = 0
    mov w0, #0
    bl _kcc_macho_write32
    // nreloc = 0
    mov w0, #0
    bl _kcc_macho_write32
    // flags = S_ZEROFILL (type 1)
```

```
    mov w0, #1
    bl _kcc_macho_write32
    // reserved1, reserved2, reserved3
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32

    // --- Section: __DATA_CONST,__const (const
   pointer tables, section 3) ---
    adrp x0, _kcc_rodata_const_present@PAGE
    add  x0, x0, _kcc_rodata_const_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_const_sect
    // sectname "__const\0..."
    mov x0, #0x5F5F
    movk x0, #0x6F63, lsl #16
    movk x0, #0x736E, lsl #32
    movk x0, #0x0074, lsl #48
    bl _kcc_macho_write64
    mov x0, #0
    bl _kcc_macho_write64
    // segname "__DATA_CONST"
    mov x0, #0x5F5F
    movk x0, #0x4144, lsl #16
    movk x0, #0x4154, lsl #32
    movk x0, #0x435F, lsl #48
    bl _kcc_macho_write64
    mov x0, #0x4E4F
    movk x0, #0x5453, lsl #16
    bl _kcc_macho_write64
    // addr = VM address after __common (and past
   the modinit pointer if any)
    ldr x0, [sp, #112]            // globals_size
    add x0, x0, x19               // + code_size
    adrp x1, _kcc_modinit_present@PAGE
    add  x1, x1, _kcc_modinit_present@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_const_addr
    add x0, x0, #7
    bic x0, x0, #7
    add x0, x0, #8                // past the
   modinit pointer's VM region
.Lmf_const_addr:
    add x0, x0, #7
    bic x0, x0, #7              // 8-align the
   const section's VM address
    bl _kcc_macho_write64
    // size = const_size
    adrp x0, _kcc_rodata_const_size@PAGE
    add  x0, x0, _kcc_rodata_const_size@PAGEOFF
    ldr x0, [x0]
    bl _kcc_macho_write64
    // offset = const file offset
    adrp x0, _kcc_rodata_const_fileoff@PAGE
    add  x0, x0, _kcc_rodata_const_fileoff@PAGEOFF
    ldr w0, [x0]
    bl _kcc_macho_write32
    // align = 3 (8-byte aligned)
    mov w0, #3
    bl _kcc_macho_write32
    // reloff = reloc_offset + (nrelocs + modinit
   ?1:0) * 8  (const relocs trail
    // the text relocs and the modinit reloc)
    mov x0, #RELOC_INFO_SIZE
    mul x0, x21, x0
    add x0, x24, x0
    adrp x1, _kcc_modinit_present@PAGE
    add  x1, x1, _kcc_modinit_present@PAGEOFF
    ldr  x1, [x1]
    cbz  x1, .Lmf_const_reloff
    add x0, x0, #RELOC_INFO_SIZE
.Lmf_const_reloff:
    bl _kcc_macho_write32
    // nreloc = const reloc count
    adrp x0, _kcc_rodata_const_relocs@PAGE
    add  x0, x0, _kcc_rodata_const_relocs@PAGEOFF
    ldr x0, [x0]
    ldr w0, [x0, #VEC_COUNT]
    bl _kcc_macho_write32
    // flags = S_REGULAR
    mov w0, #S_REGULAR
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
.Lmf_no_const_sect:

    // --- Section: __DATA,__mod_init_func (only
   for a module-init unit) ---
    adrp x0, _kcc_modinit_present@PAGE
    add  x0, x0, _kcc_modinit_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_modinit_sect
    // sectname (16 bytes): "__mod_init_func" (15
   chars) + 1 NUL, fills all 16
    mov x0, #0x5F5F
    movk x0, #0x6F6D, lsl #16
    movk x0, #0x5F64, lsl #32
    movk x0, #0x6E69, lsl #48     // "__mod_in"
    bl _kcc_macho_write64
    mov x0, #0x7469
    movk x0, #0x665F, lsl #16
    movk x0, #0x6E75, lsl #32
    movk x0, #0x0063, lsl #48     // "it_func\0"
    bl _kcc_macho_write64
    // segname (16 bytes): "__DATA" + 10 null
   bytes
    mov x0, #0x5F5F
    movk x0, #0x4144, lsl #16
    movk x0, #0x4154, lsl #32
    bl _kcc_macho_write64
    mov x0, #0
    bl _kcc_macho_write64
    // addr = 8-aligned VM address after __common
    ldr x0, [sp, #112]            // globals_size
    add x0, x0, x19               // + code_size
    add x0, x0, #7
    bic x0, x0, #7
    bl _kcc_macho_write64
    // size = 8 (one pointer)
    mov x0, #8
    bl _kcc_macho_write64
    // offset = file offset of the pointer
    adrp x0, _kcc_modinit_fileoff@PAGE
    add  x0, x0, _kcc_modinit_fileoff@PAGEOFF
    ldr w0, [x0]
    bl _kcc_macho_write32
    // align = 3 (8-byte aligned)
    mov w0, #3
    bl _kcc_macho_write32
    // reloff = reloc_offset + nrelocs*8 (the
   modinit reloc trails the text relocs)
    mov x0, #RELOC_INFO_SIZE
    mul x0, x21, x0
    add x0, x24, x0
    bl _kcc_macho_write32
    // nreloc = 1
    mov w0, #1
    bl _kcc_macho_write32
    // flags = S_MOD_INIT_FUNC_POINTERS
    mov w0, #S_MOD_INIT_FUNC_POINTERS
    bl _kcc_macho_write32
    // reserved1, reserved2, reserved3
    mov w0, #0
```

```
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
.Lmf_no_modinit_sect:
```

□

583 · defined at 580, 581, 582, 583, 585, 586, 587, 588, 589ab · used in 31d · uses _kcc_macho_write32 578, _kcc_macho_write64 578, _kcc_modinit_fileoff 305c, _kcc_modinit_present 305c, _kcc_rodata_const_fileoff 479, _kcc_rodata_const_present 479, _kcc_rodata_const_relocs 479, _kcc_rodata_const_size 479

Auxiliary Load Commands Three more load commands follow the segment. LC_BUILD_VERSION records the target platform (macOS) and minimum OS version; without it Apple's linker emits a warning on every link. LC_SYMTAB gives the file offsets and counts of the symbol table (nlist_64 array at x25) and string table (at x26, size x27). LC_DYSYMTAB partitions symbols into local, defined-external, and undefined-external ranges so the dynamic linker can resolve imports efficiently—most of its 80 bytes are zero because our object file has no indirect or module tables.

⟨ *MachO: finalize* 585 ⟩ + ≡

```
    // --- LC_BUILD_VERSION ---
    // cmd = 0x32
    mov w0, #LC_BUILD_VERSION
    bl _kcc_macho_write32
    // cmdsize = 24
    mov w0, #24
    bl _kcc_macho_write32
    // platform = 1 (PLATFORM_MACOS)
    mov w0, #1
    bl _kcc_macho_write32
    // minos = 14.0.0 → (14<<16) = 0x000E0000
    mov w0, #0
    movk w0, #14, lsl #16
    bl _kcc_macho_write32
    // sdk = 14.0.0
    mov w0, #0
    movk w0, #14, lsl #16
    bl _kcc_macho_write32
    // ntools = 0
    mov w0, #0
    bl _kcc_macho_write32

    // --- LC_SYMTAB ---
    // cmd
    mov w0, #LC_SYMTAB
    bl _kcc_macho_write32
    // cmdsize
    mov w0, #SYMTAB_CMD_SIZE
    bl _kcc_macho_write32
    // symoff
    mov w0, w25
    bl _kcc_macho_write32
    // nsyms
    mov w0, w20
    bl _kcc_macho_write32
    // stroff
    mov w0, w26
    bl _kcc_macho_write32
    // strsize
    mov w0, w27
    bl _kcc_macho_write32

    // --- LC_DYSYMTAB ---
    // cmd
    mov w0, #LC_DYSYMTAB
    bl _kcc_macho_write32
    // cmdsize
    mov w0, #DYSYMTAB_CMD_SIZE
    bl _kcc_macho_write32
    // ilocalsym = 0
    mov w0, #0
    bl _kcc_macho_write32
    // nlocalsym = 0
    mov w0, #0
    bl _kcc_macho_write32
    // iextdefsym = 0
    mov w0, #0
    bl _kcc_macho_write32
    // nextdefsym = number of defined symbols
    mov w0, w28
    bl _kcc_macho_write32
    // iundefsym = nextdef
    mov w0, w28
    bl _kcc_macho_write32
    // nundefsym = total - defined
    sub w0, w20, w28
    bl _kcc_macho_write32
    // Rest of dysymtab fields are 0 (tocoff, ntoc
  , etc - 12 more uint32_t)
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
    mov w0, #0
    bl _kcc_macho_write32
```

□

585 · defined at 580, 581, 582, 583, 585, 586, 587, 588, 589ab · used in 31d · uses _kcc_macho_write32 578

40.3.3. Code, Relocations, and Symbol Remapping

After the load commands, we write the raw code bytes (the __text section content), pad to 4-byte alignment, and then emit the relocation entries.

Before writing relocations, we must build a *remap array* that translates original symbol indices to their output positions. The Mach-O convention requires defined symbols to appear before undefined ones in the symbol table; since our linker adds symbols in encounter order (which interleaves defined and undefined), we sort them in two passes: first defined, then undefined. Each relocation's r_symbolnum is translated through this remap so the linker sees the correct index.

Each relocation packs into 8 bytes: a 32-bit r_address

(section-relative), and a 32-bit info word containing the symbol number (24 bits), pcrel flag, length code, extern flag, and relocation type.

⟨ *MachO: finalize* 586 ⟩ + ≡

```
    // --- __DWARF LC_SEGMENT_64 (under -g) ---
    // Still part of the load commands (fills x22's reserved 312 bytes so the
    // file position reaches text_offset).  The debug blob itself is laid down
    // right after the code bytes; its file base = text_offset + code_size.
    adrp x0, _kcc_dwarf_enabled@PAGE
    add  x0, x0, _kcc_dwarf_enabled@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_dw_seg_done
    add  x0, x23, x19              // dwarf file base = text_offset + code_size
    ldr  x1, [sp, #112]            // globals_size
    add  x1, x1, x19               // vmaddr = code_size + globals (past __TEXT)
    bl _kcc_dwarf_emit_macho
.Lmf_dw_seg_done:

    // --- Write code bytes ---
    // Pad to text_offset if needed (should already be there)
    adrp x0, _kcc_code_buf@PAGE
    add  x0, x0, _kcc_code_buf@PAGEOFF
    ldr x0, [x0]
    mov x1, x19                    // code_size
    bl _kcc_macho_write_bytes

    // --- Append the DWARF debug blob (under -g) ---
    // It sits between the code and the LINKEDIT region so relocs/symtab/strtab
    // stay last in the file, matching the offsets reserved in the layout above.
    adrp x0, _kcc_dwarf_enabled@PAGE
    add  x0, x0, _kcc_dwarf_enabled@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_dw_blob_done
    bl   _kcc_dwarf_append_blob
.Lmf_dw_blob_done:

    // --- Write the __mod_init_func pointer (8 zero bytes; value from reloc) ---
    // Aligning to 8 lands the file position exactly on the reserved slot.
    adrp x0, _kcc_modinit_present@PAGE
    add  x0, x0, _kcc_modinit_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_modinit_data
    mov x0, #8
    bl _kcc_macho_pad_to
    mov x0, #0
    bl _kcc_macho_write64
.Lmf_no_modinit_data:

    // --- Write the __const section data (const_size zero bytes; pointer values
    // come from the relocations) ---
    adrp x0, _kcc_rodata_const_present@PAGE
    add  x0, x0, _kcc_rodata_const_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_const_data
    mov x0, #8
    bl _kcc_macho_pad_to           // lands on the reserved 8-aligned const offset
    adrp x0, _kcc_rodata_const_size@PAGE
    add  x0, x0, _kcc_rodata_const_size@PAGEOFF
    ldr x28, [x0]                  // remaining bytes (a multiple of 8)
.Lmf_const_data_loop:
    cbz x28, .Lmf_no_const_data
    mov x0, #0
    bl _kcc_macho_write64
    sub x28, x28, #8
    b .Lmf_const_data_loop
.Lmf_no_const_data:

    // Pad to reloc_offset
    mov x0, #4
    bl _kcc_macho_pad_to

    // --- Build symbol index remapping (original → sorted output) ---
    // Output order: defined first, then undefined. Need remap[old] = new.
    lsl x0, x20, #3                // nsyms * 8 bytes
    bl _kcc_arena_alloc
    str x0, [sp, #120]             // remap array ptr

    // Pass 1: assign output indices to defined symbols
    mov x28, #0                    // original index
    str xzr, [sp, #96]             // output index counter (on stack, survives calls)
.Lmf_remap_def:
    cmp x28, x20
    b.ge .Lmf_remap_undef_start
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    mov x1, x28
    bl _kcc_vec_get
    ldr x4, [x0, #SYM_SECTION]
    cbz x4, .Lmf_remap_def_skip    // skip undefined
    // Defined: remap[x28] = output_idx
    ldr x5, [sp, #120]
    ldr x3, [sp, #96]              // load output counter
    str x3, [x5, x28, lsl #3]
    add x3, x3, #1
    str x3, [sp, #96]              // store back
.Lmf_remap_def_skip:
    add x28, x28, #1
    b .Lmf_remap_def
.Lmf_remap_undef_start:

    // Pass 2: assign output indices to undefined symbols
    // output counter continues from where pass 1 left off
    mov x28, #0
.Lmf_remap_undef:
    cmp x28, x20
    b.ge .Lmf_remap_done
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    mov x1, x28
    bl _kcc_vec_get
    ldr x4, [x0, #SYM_SECTION]
    cbnz x4, .Lmf_remap_undef_skip // skip defined
    // Undefined: remap[x28] = output_idx
    ldr x5, [sp, #120]
    ldr x3, [sp, #96]              // load output counter
    str x3, [x5, x28, lsl #3]
    add x3, x3, #1
```

```
    str x3, [sp, #96]              // store back
.Lmf_remap_undef_skip:
    add x28, x28, #1
    b .Lmf_remap_undef
.Lmf_remap_done:
```

□

586 · defined at 580, 581, 582, 583, 585, 586, 587, 588, 589ab · used in 31d · uses _kcc_arena_alloc 58a, _kcc_code_buf 533c, _kcc_linker_syms 594, _kcc_macho_pad_to 579, _kcc_macho_write64 578, _kcc_macho_write_bytes 579, _kcc_modinit_present 305c, _kcc_rodata_const_present 479, _kcc_rodata_const_size 479, _kcc_vec_get 65b

Emitting Relocation Entries With the remap array in place, we iterate the linker's relocation vector and write one 8-byte `relocation_info` record per entry. The first word is the section-relative `r_address`; the second packs the (remapped) symbol index into bits 0–23, followed by the `r_pcrel` flag (bit 24), `r_length` (bits 25–26, 2 for the 4-byte instruction fixups in this loop), `r_extern` (bit 27, always 1), and the relocation type (bits 28–31). `ARM64_RELOC_BRANCH26` and `ARM64_RELOC_PAGE21` are PC-relative; `PAGEOFF12` is not, so the `pcrel` bit is cleared for that case. The lone exception, emitted just after this loop, is the `__mod_init_func` pointer's `ARM64_RELOC_UNSIGNED`: an 8-byte, non-PC-relative fixup (`r_length` = 3) against the module-init function symbol.

⟨ *MachO: finalize* 587 ⟩ + ≡

```
    // --- Write relocations ---
    // For each reloc: pack into relocation_info format
    // r_address (32 bits) | r_symbolnum(24)|r_pcrel(1)|r_length(2)|r_extern(1)|r_type(4)
    mov x28, #0
.Lmf_reloc_loop:
    cmp x28, x21
    b.ge .Lmf_reloc_done

    adrp x0, _kcc_linker_relocs@PAGE
    add  x0, x0, _kcc_linker_relocs@PAGEOFF
    ldr x0, [x0]
    mov x1, x28
    bl _kcc_vec_get
    str x0, [sp, #112]             // save reloc entry ptr

    // r_address = reloc.offset
    ldr w0, [x0, #RELOC_OFFSET]
    bl _kcc_macho_write32

    // r_info: sym_idx(24) | pcrel(1) | length(2) | extern(1) | type(4)
    ldr x2, [sp, #112]             // reload reloc entry ptr
    ldr x3, [x2, #RELOC_SYM]       // original sym_index
    ldr x4, [x2, #RELOC_TYPE]      // reloc type

    // Remap sym_index to sorted output index
    ldr x5, [sp, #120]             // remap array
    ldr x3, [x5, x3, lsl #3]       // x3 = remap[original_index]

    // Pack: (sym_idx & 0xFFFFFF) | (pcrel<<24) | (length<<25) | (extern<<27) | (type<<28)
    // pcrel: BRANCH26/PAGE21/GOT_LOAD_PAGE21=1, PAGEOFF12/GOT_LOAD_PAGEOFF12=0
    and w0, w3, #0xFFFFFF          // r_symbolnum (remapped)
    // Determine pcrel based on type: the two PAGEOFF12 forms are absolute
    cmp w4, #ARM64_RELOC_PAGEOFF12
    b.eq .Lmf_reloc_no_pcrel
    cmp w4, #ARM64_RELOC_GOT_LOAD_PAGEOFF12
    b.eq .Lmf_reloc_no_pcrel
    orr w0, w0, #(1 << 24)         // r_pcrel = 1
.Lmf_reloc_no_pcrel:
    orr w0, w0, #(2 << 25)         // r_length = 2 (4 bytes)
    orr w0, w0, #(1 << 27)         // r_extern = 1
    orr w0, w0, w4, lsl #28        // r_type
    bl _kcc_macho_write32

    add x28, x28, #1
    b .Lmf_reloc_loop
.Lmf_reloc_done:

    // --- Module-init pointer relocation ---
    // One ARM64_RELOC_UNSIGNED (length 3 = 8 bytes, not pc-relative) binding the
    // __mod_init_func pointer to the local kcc.module_init symbol.  It trails the
    // text relocations, matching the reloff recorded in that section header.
    adrp x0, _kcc_modinit_present@PAGE
    add  x0, x0, _kcc_modinit_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_modinit_reloc_emit
    // r_address = 0 (pointer sits at the start of __mod_init_func)
    mov w0, #0
    bl _kcc_macho_write32
    // r_symbolnum = remap[modinit sym index]
    adrp x0, _kcc_modinit_sym@PAGE
    add  x0, x0, _kcc_modinit_sym@PAGEOFF
    ldr x0, [x0]
    sub x0, x0, #1                 // stored index + 1 → original index
    ldr x5, [sp, #120]             // remap array
    ldr x3, [x5, x0, lsl #3]       // remapped output index
    and w0, w3, #0xFFFFFF          // r_symbolnum
    orr w0, w0, #(3 << 25)         // r_length = 3 (8-byte pointer)
    orr w0, w0, #(1 << 27)         // r_extern = 1
    // r_pcrel = 0, r_type = ARM64_RELOC_UNSIGNED (0)
    bl _kcc_macho_write32
.Lmf_no_modinit_reloc_emit:

    // --- __const section relocations ---
    // One ARM64_RELOC_UNSIGNED (length 3, not pc-relative) per pointer slot,
    // binding it to its string-literal symbol.  They trail the modinit reloc,
    // matching the reloff recorded in the __const section header.
    adrp x0, _kcc_rodata_const_present@PAGE
    add  x0, x0, _kcc_rodata_const_present@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Lmf_no_const_reloc_emit
    mov x28, #0                    // reloc index
.Lmf_const_reloc_loop:
    adrp x0, _kcc_rodata_const_relocs@PAGE
    add  x0, x0, _kcc_rodata_const_relocs@PAGEOFF
    ldr x0, [x0]                   // relocs vec
    ldr x2, [x0, #VEC_COUNT]
    cmp x28, x2
```

```
    b.ge .Lmf_no_const_reloc_emit
    mov x1, x28
    bl _kcc_vec_get                 // x0 = entry {
   slot_offset, sym_index}
    str x0, [sp, #96]               // stash entry
   ptr across write32
    ldr w0, [x0]                    // r_address =
   slot_offset
    bl _kcc_macho_write32
    ldr x0, [sp, #96]
    ldr x2, [x0, #8]               // original
   string-lit sym index
    ldr x5, [sp, #120]             // remap array
    ldr x3, [x5, x2, lsl #3]       // remapped
   output index
    and w0, w3, #0xFFFFFF           // r_symbolnum
    orr w0, w0, #(3 << 25)         // r_length = 3
   (8-byte pointer)
    orr w0, w0, #(1 << 27)         // r_extern = 1
    bl _kcc_macho_write32
    add x28, x28, #1
    b .Lmf_const_reloc_loop
.Lmf_no_const_reloc_emit:
```

□

587 · defined at 580, 581, 582, 583, 585, 586, 587, 588, 589ab · used in 31d · uses _kcc_linker_relocs 594, _kcc_macho_write32 578, _kcc_modinit_present 305c, _kcc_modinit_sym 305c, _kcc_rodata_const_present 479, _kcc_rodata_const_relocs 479, _kcc_vec_get 65b

40.3.4. Symbol Table and String Table

The symbol table consists of `nlist_64` entries (16 bytes each): a 32-bit string-table index, an 8-bit type, an 8-bit section number, a 16-bit description, and a 64-bit value. A defined symbol's type is `N_SECT` (`0x0E`) with the `N_EXT` bit added only when the symbol carries it—so an external function or global is `0x0F` while a translation-unit-private symbol such as a string literal stays `0x0E` and is not exported. The bit is read from the symbol's own flags rather than assumed, which is what keeps a local `l_.str.`*N* from colliding with the same name in another object. A defined symbol's value is the offset within the section. Section 2 symbols (global variables in `__DATA,__common`) have their value adjusted by adding `code_size`—the `S_ZEROFILL` section's virtual start follows the text section.

The string table starts with a null byte at index 0 (the "empty string" sentinel). Symbol names are written in the same defined-then-undefined order, and a running offset tracks each symbol's `n_strx` value.

Finally, the entire output buffer is flushed to the file descriptor with a single `write` system call, and the file is closed.

⟨ *MachO: finalize* 588 ⟩ + ≡

```
    // --- Write symbol table (nlist_64 entries)
   ---
    // First: defined symbols, then undefined
    // We need to write in order: defined first,
   then undefined
    // Pass 1: defined symbols
    mov x28, #0
    mov x3, #1                      // strtab
   offset starts at 1 (skip leading NUL)
    str x3, [sp, #96]               // current
   strtab_pos
.Lmf_sym_def_loop:
    cmp x28, x20
    b.ge .Lmf_sym_def_done

    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    mov x1, x28
    bl _kcc_vec_get
    str x0, [sp, #112]              // save sym
   entry ptr

    ldr x3, [x0, #SYM_SECTION]
    cbz x3, .Lmf_sym_def_skip       // skip
   undefined

    // nlist_64: n_strx(4), n_type(1), n_sect(1),
   n_desc(2), n_value(8)
    // n_strx = current strtab position
    ldr w0, [sp, #96]
    bl _kcc_macho_write32

    // n_type = N_SECT, plus N_EXT only for an
   external (non-local) symbol.
    // Local defined symbols (string literals) get
    0x0E so they are not exported
    // and cannot collide across objects.
    ldr x2, [sp, #112]              // reload sym
   entry ptr
    ldr x3, [x2, #SYM_FLAGS]
    and w3, w3, #N_EXT
    orr w0, w3, #N_SECT
    bl _kcc_macho_write8

    // n_sect = section number from sym entry
    ldr x2, [sp, #112]              // reload sym
   entry ptr
    ldr w0, [x2, #SYM_SECTION]
    bl _kcc_macho_write8

    // n_desc = 0
    mov w0, #0
    bl _kcc_macho_write16

    // n_value = symbol value (offset in section)
   rebased to the section's VM
    // start.  Section 2 (__common) starts at
   code_size; section 3
    // (__DATA_CONST,__const) starts 8-aligned
   after __common (past the modinit
    // pointer when present), matching the addr
   written in its section header.
    ldr x2, [sp, #112]              // reload sym
   entry ptr
    ldr x0, [x2, #SYM_VALUE]
    ldr x3, [x2, #SYM_SECTION]
    cmp x3, #2
    b.eq .Lmf_sym_val_sec2
    cmp x3, #3
    b.eq .Lmf_sym_val_sec3
    b .Lmf_sym_val_ok
.Lmf_sym_val_sec2:
    adrp x3, _kcc_code_pos@PAGE
    add  x3, x3, _kcc_code_pos@PAGEOFF
    ldr x3, [x3]                    // code_size
    add x0, x0, x3
    b .Lmf_sym_val_ok
.Lmf_sym_val_sec3:
    adrp x3, _kcc_code_pos@PAGE
    add  x3, x3, _kcc_code_pos@PAGEOFF
    ldr x3, [x3]                    // code_size
    adrp x4, _kcc_globals_size@PAGE
    add  x4, x4, _kcc_globals_size@PAGEOFF
    ldr x4, [x4]
```

```
    add x3, x3, x4                  // code_size +
   globals_size
    adrp x4, _kcc_modinit_present@PAGE
    add  x4, x4, _kcc_modinit_present@PAGEOFF
    ldr x4, [x4]
    cbz x4, .Lmf_sym_val_sec3_al
    add x3, x3, #7
    bic x3, x3, #7
    add x3, x3, #8                  // past the
   modinit pointer
.Lmf_sym_val_sec3_al:
    add x3, x3, #7
    bic x3, x3, #7                  // 8-align the
   const section start
    add x0, x0, x3
.Lmf_sym_val_ok:
    bl _kcc_macho_write64

    // Advance strtab_pos by strlen(name) + 1
    ldr x2, [sp, #112]              // reload sym
   entry ptr
    ldr x0, [x2, #SYM_NAME]
    bl _kcc_strlen
    add x0, x0, #1
    ldr x3, [sp, #96]
    add x3, x3, x0
    str x3, [sp, #96]

.Lmf_sym_def_skip:
    add x28, x28, #1
    b .Lmf_sym_def_loop
.Lmf_sym_def_done:
```

□

588 · defined at 580, 581, 582, 583, 585, 586, 587, 588, 589ab · used in 31d · uses _kcc_code_pos 533c, _kcc_globals_size 594, _kcc_linker_syms 594, _kcc_macho_write16 578, _kcc_macho_write32 578, _kcc_macho_write64 578, _kcc_macho_write8 578, _kcc_modinit_present 305c, _kcc_vec_get 65b

Undefined Symbol Entries The second pass emits nlist_64 entries for symbols whose SYM_SECTION is zero—these are external references (typically libc functions like _printf or _exit). Their n_type is set to N_UNDF | N_EXT (0x01), n_sect is NO_SECT (0), and n_value is zero since the actual address will be resolved by the linker. The string-table offset counter sp+96 continues from where the defined-symbol pass left off, maintaining a consistent mapping between the symbol table and the string table.

⟨ *MachO: finalize* 589a ⟩ +≡

```
    // Pass 2: undefined symbols
    mov x28, #0
.Lmf_sym_undef_loop:
    cmp x28, x20
    b.ge .Lmf_sym_undef_done

    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    mov x1, x28
    bl _kcc_vec_get
    str x0, [sp, #112]              // save sym
   entry ptr

    ldr x3, [x0, #SYM_SECTION]
    cbnz x3, .Lmf_sym_undef_skip  // skip defined
    // n_strx
    ldr w0, [sp, #96]
    bl _kcc_macho_write32

    // n_type = N_UNDF | N_EXT = 0x01
    mov w0, #0x01
    bl _kcc_macho_write8

    // n_sect = 0 (NO_SECT)
    mov w0, #0
    bl _kcc_macho_write8

    // n_desc = 0
    mov w0, #0
    bl _kcc_macho_write16

    // n_value = 0
    mov x0, #0
    bl _kcc_macho_write64

    // Advance strtab_pos
    ldr x2, [sp, #112]              // reload sym
   entry ptr
    ldr x0, [x2, #SYM_NAME]
    bl _kcc_strlen
    add x0, x0, #1
    ldr x3, [sp, #96]
    add x3, x3, x0
    str x3, [sp, #96]

.Lmf_sym_undef_skip:
    add x28, x28, #1
    b .Lmf_sym_undef_loop
.Lmf_sym_undef_done:
```

□

589a · defined at 580, 581, 582, 583, 585, 586, 587, 588, 589ab · used in 31d · uses _kcc_linker_syms 594, _kcc_macho_write16 578, _kcc_macho_write32 578, _kcc_macho_write64 578, _kcc_macho_write8 578, _kcc_vec_get 65b

String Table and File Flush The string table is a flat byte array referenced by the n_strx offsets in each nlist_64 entry. It begins with a single NUL byte at index 0 (the Mach-O "empty string" sentinel), followed by each symbol's name including its null terminator, written in the same defined-then-undefined order as the symbol table. After the string table is complete, the entire output buffer—from the Mach-O header through the last string byte—is flushed to disk in one write system call, and the file descriptor is closed. The callee-saved registers are restored and the function returns.

⟨ *MachO: finalize* 589b ⟩ +≡

```
    // --- Write string table ---
    // Leading NUL byte
    mov w0, #0
    bl _kcc_macho_write8

    // Write symbol names in same order (defined
   first, then undefined)
    // Pass 1: defined
    mov x28, #0
.Lmf_str_def_loop:
    cmp x28, x20
    b.ge .Lmf_str_def_done

    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
```

```
    ldr x0, [x0]
    mov x1, x28
    bl _kcc_vec_get
    str x0, [sp, #112]              // save sym
   entry ptr

    ldr x3, [x0, #SYM_SECTION]
    cbz x3, .Lmf_str_def_skip

    ldr x0, [x0, #SYM_NAME]
    str x0, [sp, #120]              // save name ptr
    bl _kcc_strlen
    add x1, x0, #1                  // include null
   terminator
    ldr x0, [sp, #120]              // reload name
   ptr
    bl _kcc_macho_write_bytes

.Lmf_str_def_skip:
    add x28, x28, #1
    b .Lmf_str_def_loop
.Lmf_str_def_done:

    // Pass 2: undefined
    mov x28, #0
.Lmf_str_undef_loop:
    cmp x28, x20
    b.ge .Lmf_str_undef_done

    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    mov x1, x28
    bl _kcc_vec_get
    str x0, [sp, #112]              // save sym
   entry ptr

    ldr x3, [x0, #SYM_SECTION]
    cbnz x3, .Lmf_str_undef_skip

    ldr x0, [x0, #SYM_NAME]
    str x0, [sp, #120]              // save name ptr
    bl _kcc_strlen
    add x1, x0, #1
    ldr x0, [sp, #120]              // reload name
   ptr
    bl _kcc_macho_write_bytes

.Lmf_str_undef_skip:
    add x28, x28, #1
    b .Lmf_str_undef_loop
.Lmf_str_undef_done:

    // --- Flush to file ---
    adrp x0, _kcc_macho_fd@PAGE
    add  x0, x0, _kcc_macho_fd@PAGEOFF
    ldr x0, [x0]                    // fd
    adrp x1, _kcc_macho_buf@PAGE
    add  x1, x1, _kcc_macho_buf@PAGEOFF
    ldr x1, [x1]                    // buf
    adrp x2, _kcc_macho_pos@PAGE
    add  x2, x2, _kcc_macho_pos@PAGEOFF
    ldr x2, [x2]                    // len
    bl _kcc_sys_write

    // Close file
    adrp x0, _kcc_macho_fd@PAGE
    add  x0, x0, _kcc_macho_fd@PAGEOFF
    ldr x0, [x0]
    bl _kcc_sys_close

    ldp x27, x28, [sp, #80]
    ldp x25, x26, [sp, #64]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #128
    ret
```

□

589b · defined at 580, 581, 582, 583, 585, 586, 587, 588, 589ab · used in 31d · uses _kcc_linker_syms 594, _kcc_macho_buf 590, _kcc_macho_fd 590, _kcc_macho_pos 590, _kcc_macho_write8 578, _kcc_macho_write_bytes 579, _kcc_vec_get 65b

40.4. Mach-O Data

Module state: the output file descriptor, the write buffer pointer, and the current buffer position. The error string lives in the __cstring section so it can be passed to kcc_io_err.

⟨ *MachO: data* 590 ⟩ ≡

```
// -- Mach-O module data ---
.data
.p2align 3
.globl _kcc_macho_fd
_kcc_macho_fd:        .quad 0      // output file
   descriptor
.globl _kcc_macho_buf
_kcc_macho_buf:       .quad 0      // output buffer
   pointer
.globl _kcc_macho_pos
_kcc_macho_pos:       .quad 0      // current write
   position
.globl _kcc_macho_cap
_kcc_macho_cap:       .quad 0      // allocated
   output buffer size (overflow guard)

.section __TEXT,__cstring,cstring_literals
_kcc_macho_err_open:
    .asciz "kcc: error: cannot open output file\n"
_kcc_macho_err_overflow:
    .asciz "kcc: error: object file exceeds output
    buffer\n"

// Section/segment names are now written as inline
    immediates in the emitter
// (avoids linker dead-stripping trailing null
   bytes from local atoms)
```

□

590 · used in 31d · defines _kcc_macho_buf 577b, 578, 579, 589b, _kcc_macho_cap 577b, 579, _kcc_macho_fd 577b, 589b, _kcc_macho_pos 577b, 578, 579, 589b

41. Linker Information

The code generator emits branch and data-reference instructions whose final addresses are not known until link time. This chapter manages the metadata that the linker needs to resolve those references. It defines two data structures—relocation entries and symbol entries—and provides functions to add relocations, register defined and external symbols, look up symbols by name (using a hash table to deduplicate externs), and query the accumulated tables when the Mach-O writer serializes the object file.

41.1. Data Structures

A relocation entry is 24 bytes:

Offset	Field	Description
0	offset	Byte offset in code section
8	type	Relocation type
16	sym_idx	Symbol table index

A symbol entry is 32 bytes:

Offset	Field	Description
0	name	Pointer to symbol name string
8	section	Section number (1-based, 0=undefined)
16	value	Symbol value (offset within section)
24	flags	N_EXT, N_SECT, etc.

⟨ *Linker: constants* 591a ⟩ ≡

```
// -- Linker data structure layout ---
.equ RELOC_OFFSET,      0
.equ RELOC_TYPE,        8
.equ RELOC_SYM,        16
.equ RELOC_SIZE,       24

.equ SYM_NAME,          0
.equ SYM_SECTION,       8
.equ SYM_VALUE,        16
.equ SYM_FLAGS,        24
.equ SYM_ENTRY_SIZE,   32

// -- Mach-O relocation types ---
.equ ARM64_RELOC_BRANCH26,           2
.equ ARM64_RELOC_PAGE21,             3
.equ ARM64_RELOC_PAGEOFF12,          4
.equ ARM64_RELOC_GOT_LOAD_PAGE21,    5
.equ ARM64_RELOC_GOT_LOAD_PAGEOFF12, 6

// -- Mach-O symbol flags ---
.equ N_UNDF,    0x00
.equ N_EXT,     0x01
.equ N_SECT,    0x0E
```

□

591a · used in 30d, 31bcd

41.2. Initialization

The linker module maintains three data structures: a *relocation vector* (24-byte entries), a *symbol vector* (32-byte entries), and a *hash table* for deduplicating external symbol references. Each vector is arena-allocated with an initial capacity of 32 entries.

⟨ *Linker: init* 591b ⟩ ≡

```
// -- kcc_linker_init ---
// void kcc_linker_init(void)
// Allocates relocation vec, symbol vec, and
   string table buffer.
.text
.globl _kcc_linker_init
.p2align 2
_kcc_linker_init:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    // Allocate reloc vec (elem=24 bytes, initial
   cap=32)
    mov x0, #VEC_SIZE
    bl _kcc_arena_alloc
    adrp x1, _kcc_linker_relocs@PAGE
    add  x1, x1, _kcc_linker_relocs@PAGEOFF
    str x0, [x1]
    mov x1, #RELOC_SIZE
    str x1, [x0, #VEC_ELEMSIZE]
    mov x1, #32
    str x1, [x0, #VEC_CAP]
    str xzr, [x0, #VEC_COUNT]
    mov x19, x0

    mov x0, #768                    // 32 * 24
    bl _kcc_arena_alloc
    str x0, [x19, #VEC_DATA]

    // Allocate symbol vec (elem=32 bytes, initial
    cap=32)
    mov x0, #VEC_SIZE
    bl _kcc_arena_alloc
    adrp x1, _kcc_linker_syms@PAGE
    add  x1, x1, _kcc_linker_syms@PAGEOFF
    str x0, [x1]
    mov x1, #SYM_ENTRY_SIZE
    str x1, [x0, #VEC_ELEMSIZE]
    mov x1, #32
    str x1, [x0, #VEC_CAP]
    str xzr, [x0, #VEC_COUNT]
    mov x19, x0

    mov x0, #1024                   // 32 * 32
    bl _kcc_arena_alloc
    str x0, [x19, #VEC_DATA]

    // Allocate extern lookup hash table
    mov x0, #64
    bl _kcc_ht_init
    adrp x1, _kcc_linker_extern_ht@PAGE
    add  x1, x1, _kcc_linker_extern_ht@PAGEOFF
    str x0, [x1]

    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

591b · used in 31b · defines _kcc_linker_init 603 · uses _kcc_arena_alloc 58a, _kcc_ht_init 61c,

_kcc_linker_extern_ht 594, _kcc_linker_relocs 594, _kcc_linker_syms 594

41.3. Adding Symbols

Each call appends a 32-byte entry to the symbol vector and returns the new symbol's zero-based index. The caller fills in the name pointer, section number, value, and Mach-O flags. Index 0 is the first symbol added, not a reserved sentinel.

⟨ *Linker: add symbol* 592a ⟩ ≡

```
// -- kcc_linker_add_sym ---
// uint64_t kcc_linker_add_sym(const char *name,
   uint64_t section,
//                              uint64_t value,
   uint64_t flags)
//   x0=name, x1=section, x2=value, x3=flags
// Returns the symbol index (0-based).
.text
.globl _kcc_linker_add_sym
.p2align 2
_kcc_linker_add_sym:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    // Build entry on stack
    str x0, [sp, #32]              // SYM_NAME
    str x1, [sp, #40]              // SYM_SECTION
    str x2, [sp, #48]              // SYM_VALUE
    str x3, [sp, #56]              // SYM_FLAGS

    // Get symbol vec
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]

    // Save current count as return value
    ldr x19, [x0, #VEC_COUNT]

    // Push entry
    add x1, sp, #32                // pointer to
   entry on stack
    bl _kcc_vec_push

    mov x0, x19                    // return index

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

592a · used in 31b · defines _kcc_linker_add_sym 469, 472, 476, 477b, 592c, 593 · uses _kcc_linker_syms 594, _kcc_vec_push 65a

41.4. Adding Relocations

A relocation records three pieces of information: the byte offset within the code section where the instruction lives, the relocation type (one of BRANCH26, PAGE21, or PAGEOFF12), and the index of the target symbol. The Mach-O writer later packs these into the 8-byte relocation_info format the linker expects.

⟨ *Linker: add reloc* 592b ⟩ ≡

```
// -- kcc_linker_add_reloc ---
// void kcc_linker_add_reloc(uint64_t offset,
   uint64_t type, uint64_t sym_index)
//   x0=offset, x1=type, x2=sym_index
.text
.globl _kcc_linker_add_reloc
.p2align 2
_kcc_linker_add_reloc:
    stp x29, x30, [sp, #-48]!
    mov x29, sp

    // Build entry on stack
    str x0, [sp, #16]              // RELOC_OFFSET
    str x1, [sp, #24]              // RELOC_TYPE
    str x2, [sp, #32]              // RELOC_SYM

    // Get reloc vec
    adrp x0, _kcc_linker_relocs@PAGE
    add  x0, x0, _kcc_linker_relocs@PAGEOFF
    ldr x0, [x0]

    add x1, sp, #16                // pointer to
   entry
    bl _kcc_vec_push

    ldp x29, x30, [sp], #48
    ret
```

□

592b · used in 31b · defines _kcc_linker_add_reloc 477b, 518, 521 · uses _kcc_linker_relocs 594, _kcc_vec_push 65a

41.5. External Symbol Lookup

When code references an external function (e.g., a call to a C library function), we need to add it to the symbol table as undefined and return its index. We deduplicate using a hash table.

⟨ *Linker: lookup or add extern* 592c ⟩ ≡

```
// -- kcc_linker_extern ---
// uint64_t kcc_linker_extern(const char *name)
//   x0 = null-terminated symbol name
// Returns the symbol index, adding a new N_UNDF|
   N_EXT entry if needed.
.text
.globl _kcc_linker_extern
.p2align 2
_kcc_linker_extern:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    mov x19, x0                    // save name

    // Check hash table for existing entry
    adrp x0, _kcc_linker_extern_ht@PAGE
    add  x0, x0, _kcc_linker_extern_ht@PAGEOFF
    ldr x0, [x0]
    mov x1, x19                    // key = name
    mov x20, x19
    // Need key length
    mov x0, x19
    bl _kcc_strlen
    mov x20, x0                    // name len

    adrp x0, _kcc_linker_extern_ht@PAGE
    add  x0, x0, _kcc_linker_extern_ht@PAGEOFF
```

```
    ldr x0, [x0]
    mov x1, x19                     // key
    mov x2, x20                     // keylen
    bl _kcc_ht_get
    cbz x0, .Lle_new

    // Found: value is sym_index+1 (so 0 means "
   not found")
    sub x0, x0, #1
    b .Lle_done

.Lle_new:
    // Add new undefined external symbol
    mov x0, x19                     // name
    mov x1, #0                      // section = 0 (
   N_UNDF)
    mov x2, #0                      // value = 0
    mov x3, #(N_EXT)                // flags = N_EXT
   (external, undefined)
    bl _kcc_linker_add_sym
    mov x19, x0                     // save
   sym_index (now reuse x19)

    // Store in hash table (value = sym_index + 1)
    str x19, [sp, #32]              // temp save
    adrp x0, _kcc_linker_extern_ht@PAGE
    add  x0, x0, _kcc_linker_extern_ht@PAGEOFF
    ldr x0, [x0]
    ldr x1, [sp, #32]               // restore saved
    name? No, x19 is now sym_index

    // We need name and len again
    // x19 = sym_index, but we saved original name
   ...
    // Actually let's restructure: save name in sp
   +32
    // The issue is we've overwritten x19. Let me
   fix:
    // Get name from symbol table entry
    adrp x1, _kcc_linker_syms@PAGE
    add  x1, x1, _kcc_linker_syms@PAGEOFF
    ldr x1, [x1]
    ldr x1, [x1, #VEC_DATA]
    mov x2, #SYM_ENTRY_SIZE
    mul x2, x19, x2
    add x1, x1, x2
    ldr x1, [x1, #SYM_NAME]         // get name back
   from entry

    // hash table put: ht, key, keylen, value
    str x1, [sp, #32]               // save name ptr
    mov x0, x1
    bl _kcc_strlen
    mov x20, x0                     // keylen

    adrp x0, _kcc_linker_extern_ht@PAGE
    add  x0, x0, _kcc_linker_extern_ht@PAGEOFF
    ldr x0, [x0]
    ldr x1, [sp, #32]               // name
    mov x2, x20                     // keylen
    add x3, x19, #1                 // value =
   sym_index + 1
    bl _kcc_ht_put

    mov x0, x19                     // return
   sym_index

.Lle_done:
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

592c · used in 31b · defines _kcc_linker_extern 518, 521 · uses _kcc_ht_get 62, _kcc_ht_put 63, _kcc_linker_add_sym 592a, _kcc_linker_extern_ht 594, _kcc_linker_syms 594

41.6. Global Symbol Registration

When the code generator encounters an IR_GLOBAL_REF, the referenced global variable needs a *defined* symbol in section 2 (the __DATA,__common zerofill section). The caller passes the allocation size in x1: eight bytes for scalar globals, $n \times 8$ for struct globals with n fields. We deduplicate via the same extern hash table. If the symbol is already *defined* (section already set to 2), we return its index without re-allocating space—this makes repeated references and pre-registration safe. If it was previously registered as an extern, we upgrade it to defined and allocate the requested number of bytes.

⟨ *Linker: global sym* 593 ⟩ ≡

```
// -- kcc_linker_global_sym ---
// uint64_t kcc_linker_global_sym(const char *name
   , uint64_t size, uint64_t extflag)
//   x0 = null-terminated symbol name (already
   prefixed with '_')
//   x1 = allocation size in bytes (typically 8
   for scalars, N*8 for structs)
//   x2 = N_EXT for an externally-visible
   definition, 0 for a file-local
//        (`static') one.  A static file-scope
   object keeps its source name but
//        must be local, or two translation units
   that each define a `static'
//        with the same spelling (doom's `static
   player_t *plr;') collide at link.
// Returns the symbol index.  If the symbol
   already exists as extern,
// upgrades it to defined in section 2.  Otherwise
    adds new with N_SECT|extflag.
.text
.globl _kcc_linker_global_sym
.p2align 2
_kcc_linker_global_sym:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                     // save name
    str x1, [sp, #40]               // save size (x1
   )
    str x2, [sp, #48]               // save external
   -linkage flag (N_EXT or 0)

    // Check if symbol already in extern hash
   table
    mov x0, x19
    bl _kcc_strlen
    mov x20, x0                     // name len

    adrp x0, _kcc_linker_extern_ht@PAGE
    add  x0, x0, _kcc_linker_extern_ht@PAGEOFF
    ldr x0, [x0]
    mov x1, x19
    mov x2, x20
    bl _kcc_ht_get
    cbz x0, .Llgs_new

    // Already exists - get sym_index
```

```
    sub x21, x0, #1                 // sym_index

    // Get symbol entry
    adrp x0, _kcc_linker_syms@PAGE
    add  x0, x0, _kcc_linker_syms@PAGEOFF
    ldr x0, [x0]
    ldr x0, [x0, #VEC_DATA]
    mov x1, #SYM_ENTRY_SIZE
    mul x1, x21, x1
    add x0, x0, x1                  // sym entry ptr

    // If already defined (section == 1 or 2),
   return immediately
    ldr x1, [x0, #SYM_SECTION]
    cmp x1, #0
    b.ne .Llgs_dedup

    // Upgrade from extern to defined
    mov x1, #2
    str x1, [x0, #SYM_SECTION]

    // Set value = current data section offset
    adrp x1, _kcc_globals_size@PAGE
    add  x1, x1, _kcc_globals_size@PAGEOFF
    ldr x2, [x1]
    str x2, [x0, #SYM_VALUE]

    // Advance data section size by requested
   amount
    ldr x3, [sp, #40]               // saved size
    add x2, x2, x3
    str x2, [x1]

.Llgs_dedup:
    mov x0, x21
    b .Llgs_done

.Llgs_new:
    // Add new defined symbol
    adrp x0, _kcc_globals_size@PAGE
    add  x0, x0, _kcc_globals_size@PAGEOFF
    ldr x21, [x0]                   // current
   offset = value for new sym

    mov x0, x19                     // name
    mov x1, #2                      // section = 2 (
   DATA)
    mov x2, x21                     // value =
   offset in data section
    ldr x3, [sp, #48]               // caller's
   linkage flag (N_EXT or 0)
    orr x3, x3, #N_SECT             // flags =
   defined [+ external]
    bl _kcc_linker_add_sym
    mov x21, x0                     // sym_index

    // Advance globals size by requested amount
    adrp x0, _kcc_globals_size@PAGE
    add  x0, x0, _kcc_globals_size@PAGEOFF
    ldr x1, [x0]
    ldr x2, [sp, #40]               // saved size
    add x1, x1, x2
    str x1, [x0]

    // Store in extern hash table for dedup
    adrp x0, _kcc_linker_extern_ht@PAGE
    add  x0, x0, _kcc_linker_extern_ht@PAGEOFF
    ldr x0, [x0]
    mov x1, x19                     // key = name
    mov x2, x20                     // keylen
    add x3, x21, #1                // value =
   sym_index + 1
    bl _kcc_ht_put
    mov x0, x21

.Llgs_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

593 · used in 31b · defines _kcc_linker_global_sym 465, 521 · uses _kcc_globals_size 594, _kcc_ht_get 62, _kcc_ht_put 63, _kcc_linker_add_sym 592a, _kcc_linker_extern_ht 594, _kcc_linker_syms 594

41.7. Linker Data

Module-level state: pointers to the relocation and symbol vectors, the extern deduplication hash table, and the running total of bytes allocated in the global data section (used as the n_value for new __DATA,__common symbols).

⟨ *Linker: data* 594 ⟩ ≡

```
// -- Linker module data ---
.data
.p2align 3
.globl _kcc_linker_relocs
_kcc_linker_relocs:      .quad 0     // vec of
   relocation entries
.globl _kcc_linker_syms
_kcc_linker_syms:        .quad 0     // vec of
   symbol entries
.globl _kcc_linker_extern_ht
_kcc_linker_extern_ht:   .quad 0     // hash table
    for extern dedup
.globl _kcc_globals_size
_kcc_globals_size:       .quad 0     // total size
    of global data section
```

□

594 · used in 31b · defines _kcc_globals_size 543, 580, 588, 593, _kcc_linker_extern_ht 469, 472, 591b, 592c, 593, _kcc_linker_relocs 537, 577b, 580, 587, 591b, 592b, _kcc_linker_syms 478, 498b, 537, 543, 577b, 580, 581, 586, 588, 589ab, 591b, 592ac, 593

42. DWARF Debug Information

Debuggers such as `lldb` read *DWARF* debug information (§, DWARF 4) to map machine addresses back to source functions and lines. kcc emits three sections: `.debug_abbrev` (a table of DIE shapes), `.debug_info` (a compile-unit DIE plus one `subprogram` DIE per function, carrying its name, address range, and declaration line), and `.debug_line` (a line-number program mapping each function's entry address to its source line). This first increment is at *function* granularity—enough for `lldb` to name functions in a backtrace and break on a function; statement-level rows are a later refinement that needs a source line on each IR instruction.

Everything here is additive and switched on only by -g: with the flag off, `kcc_dwarf_build` returns immediately and the Mach-O writer emits exactly the object file it did before.

42.1. Constants

The DWARF tag, attribute, and form codes we use, plus the standard line-number opcodes.

⟨ *DWARF: constants* 595a ⟩ ≡

```
// -- DWARF v4 constants ---
.equ DW_TAG_compile_unit, 0x11
.equ DW_TAG_subprogram,   0x2e
.equ DW_CHILDREN_no,      0
.equ DW_CHILDREN_yes,     1
.equ DW_AT_name,          0x03
.equ DW_AT_stmt_list,     0x10
.equ DW_AT_low_pc,        0x11
.equ DW_AT_high_pc,       0x12
.equ DW_AT_language,      0x13
.equ DW_AT_comp_dir,      0x1b
.equ DW_AT_producer,      0x25
.equ DW_AT_decl_line,     0x3b
.equ DW_FORM_addr,        0x01
.equ DW_FORM_data2,       0x05
.equ DW_FORM_string,      0x08
.equ DW_FORM_udata,       0x0f
.equ DW_FORM_sec_offset,  0x17
.equ DW_LANG_C99,         0x0c
.equ DW_LNS_copy,         1
.equ DW_LNS_advance_line, 3
.equ DW_LNE_end_sequence, 1
.equ DW_LNE_set_address,  2
.equ DWARF_BUF_CAP,       262144    // 256 KiB
   scratch for the debug blob
.equ LC_SEGMENT_64,       0x19      // (also
   defined in the Mach-O module)
```

□

595a · used in 31e

42.2. Byte Emitters

The debug sections are built into one contiguous buffer (`_kcc_dwarf_buf`); the Mach-O writer records each section's offset and length and copies the blob out. The emitters append little-endian integers, LEB128 values (DWARF's variable-length encoding), raw strings, and—for the two length-prefixed sections—a back-patch of a previously reserved u32.

⟨ *DWARF: emit* 595b ⟩ ≡

```
// -- dwarf byte emitters ---
// All append to _kcc_dwarf_buf at _kcc_dwarf_pos
   (bounds-guarded).
.text
.p2align 2
_kcc_dw_u8:
    adrp x1, _kcc_dwarf_pos@PAGE
    add  x1, x1, _kcc_dwarf_pos@PAGEOFF
    ldr  x2, [x1]
    adrp x3, _kcc_dwarf_buf@PAGE
    add  x3, x3, _kcc_dwarf_buf@PAGEOFF
    ldr  x3, [x3]
    strb w0, [x3, x2]
    add  x2, x2, #1
    str  x2, [x1]
    ret

.p2align 2
_kcc_dw_u16:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                    // value (callee-
   saved; _kcc_dw_u8 clobbers x1-x3)
    and x0, x19, #0xff
    bl _kcc_dw_u8
    lsr x0, x19, #8
    and x0, x0, #0xff
    bl _kcc_dw_u8
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

.p2align 2
_kcc_dw_u32:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0
    and x0, x19, #0xff
    bl _kcc_dw_u8
    lsr x0, x19, #8
    and x0, x0, #0xff
    bl _kcc_dw_u8
    lsr x0, x19, #16
    and x0, x0, #0xff
    bl _kcc_dw_u8
    lsr x0, x19, #24
    and x0, x0, #0xff
    bl _kcc_dw_u8
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

.p2align 2
_kcc_dw_u64:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0
    and x0, x19, #0xffffffff
```

```
    bl _kcc_dw_u32
    lsr x0, x19, #32
    bl _kcc_dw_u32
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// unsigned LEB128
.p2align 2
_kcc_dw_uleb:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                     // value
.Luleb_loop:
    and x0, x19, #0x7f              // low 7 bits
    lsr x19, x19, #7
    cbz x19, .Luleb_last
    orr x0, x0, #0x80               // more bytes
   follow
    bl _kcc_dw_u8
    b .Luleb_loop
.Luleb_last:
    bl _kcc_dw_u8
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// signed LEB128
.p2align 2
_kcc_dw_sleb:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                     // value (sign-
   extended in x19)
.Lsleb_loop:
    and x0, x19, #0x7f              // low 7 bits (
   byte candidate)
    asr x19, x19, #7                // arithmetic
   shift keeps the sign
    // done when x19 is 0 and sign bit of byte
   clear, or x19 is -1 and set
    tst x0, #0x40                   // sign bit of
   the 7-bit group
    b.eq .Lsleb_check_pos
    // negative group: done if x19 == -1
    cmn x19, #1
    b.eq .Lsleb_last
    b .Lsleb_more
.Lsleb_check_pos:
    cbz x19, .Lsleb_last
.Lsleb_more:
    orr x0, x0, #0x80
    bl _kcc_dw_u8
    b .Lsleb_loop
.Lsleb_last:
    bl _kcc_dw_u8
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// Append a NUL-terminated string (including the
   NUL).
.p2align 2
_kcc_dw_str:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                     // string ptr
    cbz x19, .Ldwstr_nul
.Ldwstr_loop:
    ldrb w0, [x19]
    cbz w0, .Ldwstr_end
    bl _kcc_dw_u8
    add x19, x19, #1
    b .Ldwstr_loop
.Ldwstr_end:
    mov w0, #0
    bl _kcc_dw_u8
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
.Ldwstr_nul:
    mov w0, #0
    bl _kcc_dw_u8
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// Return the current write position (for a later
   back-patch).
.p2align 2
_kcc_dw_pos:
    adrp x0, _kcc_dwarf_pos@PAGE
    add  x0, x0, _kcc_dwarf_pos@PAGEOFF
    ldr  x0, [x0]
    ret

// Overwrite a u32 at a saved position (little-
   endian).  x0 = pos, x1 = value.
.p2align 2
_kcc_dw_patch32:
    adrp x2, _kcc_dwarf_buf@PAGE
    add  x2, x2, _kcc_dwarf_buf@PAGEOFF
    ldr  x2, [x2]
    add  x2, x2, x0                 // &buf[pos]
    strb w1, [x2]
    lsr  w3, w1, #8
    strb w3, [x2, #1]
    lsr  w3, w1, #16
    strb w3, [x2, #2]
    lsr  w3, w1, #24
    strb w3, [x2, #3]
    ret
```

□

595b · used in 31e

42.3. Building the Sections

`kcc_dwarf_build` is the entry point the driver calls. It returns immediately unless `-g` set `_kcc_dwarf_enabled`. Otherwise it allocates the scratch buffer and lays down the three sections back to back, recording each one's `(offset, length)` so the Mach-O writer can emit a `section_64` for it.

The abbreviation table declares two DIE shapes: a `compile_unit` (with children) carrying the producer, language, file name, working directory, whole-unit address range, and a pointer to the line program; and a `subprogram` (leaf) carrying a function's name, address range, and declaration line. The info section then instantiates one compile unit and one subprogram per function, reading each function's name (`IRF_NAME`), entry offset (`IRF_LOWPC`, recorded by codegen), and declaration line (`IRF_LINE`, recorded by the IR builder). A function's `high_pc` is the next function's entry (functions are laid out in order), or the total code size for the last.

⟨ *DWARF: build* 596 ⟩ ≡

```
// -- kcc_dwarf_build ---
```

```
// void kcc_dwarf_build(void)
// No-op unless -g.  Builds .debug_abbrev/.debug_info/.debug_line into
// _kcc_dwarf_buf and records per-section offset/length globals.
.text
.globl _kcc_dwarf_build
.p2align 2
_kcc_dwarf_build:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    adrp x0, _kcc_dwarf_enabled@PAGE
    add  x0, x0, _kcc_dwarf_enabled@PAGEOFF
    ldr  x0, [x0]
    cbz  x0, .Ldwb_done
    // Allocate the scratch buffer.
    mov x0, #DWARF_BUF_CAP
    bl _kcc_arena_alloc
    adrp x1, _kcc_dwarf_buf@PAGE
    add  x1, x1, _kcc_dwarf_buf@PAGEOFF
    str x0, [x1]
    adrp x1, _kcc_dwarf_pos@PAGE
    add  x1, x1, _kcc_dwarf_pos@PAGEOFF
    str xzr, [x1]

    bl _kcc_dwarf_build_abbrev
    bl _kcc_dwarf_build_info
    bl _kcc_dwarf_build_line
.Ldwb_done:
    ldp x29, x30, [sp], #16
    ret

// -- .debug_abbrev ---
.p2align 2
_kcc_dwarf_build_abbrev:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    bl _kcc_dw_pos
    adrp x1, _kcc_dw_abbrev_off@PAGE
    add  x1, x1, _kcc_dw_abbrev_off@PAGEOFF
    str x0, [x1]
    // abbrev 1: compile_unit, has children
    mov x0, #1
    bl _kcc_dw_uleb
    mov x0, #DW_TAG_compile_unit
    bl _kcc_dw_uleb
    mov w0, #DW_CHILDREN_yes
    bl _kcc_dw_u8
    mov x0, #DW_AT_producer
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_string
    bl _kcc_dw_uleb
    mov x0, #DW_AT_language
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_data2
    bl _kcc_dw_uleb
    mov x0, #DW_AT_name
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_string
    bl _kcc_dw_uleb
    mov x0, #DW_AT_comp_dir
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_string
    bl _kcc_dw_uleb
    mov x0, #DW_AT_low_pc
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_addr
    bl _kcc_dw_uleb
    mov x0, #DW_AT_high_pc
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_addr
    bl _kcc_dw_uleb
    mov x0, #DW_AT_stmt_list
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_sec_offset
    bl _kcc_dw_uleb
    mov w0, #0
    bl _kcc_dw_u8
    mov w0, #0
    bl _kcc_dw_u8    // end of attrs
    // abbrev 2: subprogram, leaf
    mov x0, #2
    bl _kcc_dw_uleb
    mov x0, #DW_TAG_subprogram
    bl _kcc_dw_uleb
    mov w0, #DW_CHILDREN_no
    bl _kcc_dw_u8
    mov x0, #DW_AT_name
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_string
    bl _kcc_dw_uleb
    mov x0, #DW_AT_low_pc
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_addr
    bl _kcc_dw_uleb
    mov x0, #DW_AT_high_pc
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_addr
    bl _kcc_dw_uleb
    mov x0, #DW_AT_decl_line
    bl _kcc_dw_uleb
    mov x0, #DW_FORM_udata
    bl _kcc_dw_uleb
    mov w0, #0
    bl _kcc_dw_u8
    mov w0, #0
    bl _kcc_dw_u8
    // terminator abbrev (code 0)
    mov w0, #0
    bl _kcc_dw_u8
    // record length
    bl _kcc_dw_pos
    adrp x1, _kcc_dw_abbrev_off@PAGE
    add x1, x1, _kcc_dw_abbrev_off@PAGEOFF
    ldr x1, [x1]
    sub x0, x0, x1
    adrp x1, _kcc_dw_abbrev_len@PAGE
    add x1, x1, _kcc_dw_abbrev_len@PAGEOFF
    str x0, [x1]
    ldp x29, x30, [sp], #16
    ret

// -- .debug_info ---
// x19 = length-field patch position, x20 = func index, x21 = func count,
// x22 = funcs vec, x23 = current func, x24 = code size.
.p2align 2
_kcc_dwarf_build_info:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    bl _kcc_dw_pos
    adrp x1, _kcc_dw_info_off@PAGE
    add x1, x1, _kcc_dw_info_off@PAGEOFF
    str x0, [x1]
    // unit_length (patched later)
    bl _kcc_dw_pos
    mov x19, x0
    mov x0, #0
    bl _kcc_dw_u32
    mov x0, #4
    bl _kcc_dw_u16              // version
    mov x0, #0
    bl _kcc_dw_u32              // debug_abbrev_offset
    mov w0, #8
    bl _kcc_dw_u8               // address_size
    // CU DIE (abbrev 1)
```

```
    mov x0, #1
    bl _kcc_dw_uleb
    adrp x0, _kcc_dw_producer@PAGE
    add x0, x0, _kcc_dw_producer@PAGEOFF
    bl _kcc_dw_str
    mov x0, #DW_LANG_C99
    bl _kcc_dw_u16
    adrp x0, _kcc_src_path@PAGE
    add x0, x0, _kcc_src_path@PAGEOFF
    ldr x0, [x0]
    bl _kcc_dw_str
    adrp x0, _kcc_dw_compdir@PAGE
    add x0, x0, _kcc_dw_compdir@PAGEOFF
    bl _kcc_dw_str
    mov x0, #0
    bl _kcc_dw_u64              // low_pc = text
  base
    // high_pc = code size
    adrp x0, _kcc_code_pos@PAGE
    add x0, x0, _kcc_code_pos@PAGEOFF
    ldr x24, [x0]
    mov x0, x24
    bl _kcc_dw_u64
    mov x0, #0
    bl _kcc_dw_u32              // stmt_list
  offset into .debug_line

    // subprogram DIEs, one per function
    adrp x0, _kcc_ir_funcs@PAGE
    add x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x22, [x0]
    cbz x22, .Ldwinfo_end
    ldr x21, [x22, #VEC_COUNT]
    mov x20, #0
.Ldwinfo_loop:
    cmp x20, x21
    b.ge .Ldwinfo_end
    mov x0, x22
    mov x1, x20
    bl _kcc_vec_get
    ldr x23, [x0]   // func ptr
    cbz x23, .Ldwinfo_next
    mov x0, #2
    bl _kcc_dw_uleb              // abbrev
  subprogram
    ldr x0, [x23, #IRF_NAME]
    bl _kcc_dw_str
    ldr x0, [x23, #IRF_LOWPC]
    bl _kcc_dw_u64   // low_pc
    // high_pc = next func low_pc, or code size
  for the last
    add x1, x20, #1
    cmp x1, x21
    b.ge .Ldwinfo_last_hi
    mov x0, x22
    bl _kcc_vec_get
    ldr x0, [x0]
    ldr x0, [x0, #IRF_LOWPC]
    b .Ldwinfo_emit_hi
.Ldwinfo_last_hi:
    mov x0, x24                                //
  code size
.Ldwinfo_emit_hi:
    bl _kcc_dw_u64
    ldr x0, [x23, #IRF_LINE]
    bl _kcc_dw_uleb   // decl_line
.Ldwinfo_next:
    add x20, x20, #1
    b .Ldwinfo_loop
.Ldwinfo_end:
    mov w0, #0
    bl _kcc_dw_u8               // null DIE ends
  CU children
    // patch unit_length = (end - (patchpos+4))
    bl _kcc_dw_pos
    add x1, x19, #4
    sub x1, x0, x1
    mov x0, x19
    bl _kcc_dw_patch32
    // record section length
    bl _kcc_dw_pos
    adrp x1, _kcc_dw_info_off@PAGE
    add x1, x1, _kcc_dw_info_off@PAGEOFF
    ldr x1, [x1]
    sub x0, x0, x1
    adrp x1, _kcc_dw_info_len@PAGE
    add x1, x1, _kcc_dw_info_len@PAGEOFF
    str x0, [x1]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- .debug_line ---
// x19 = unit_length patch pos, x20 =
  header_length patch pos,
// x21 = func index, x22 = funcs vec, x23 = func
  count, x24 = cur line reg.
.p2align 2
_kcc_dwarf_build_line:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    bl _kcc_dw_pos
    adrp x1, _kcc_dw_line_off@PAGE
    add x1, x1, _kcc_dw_line_off@PAGEOFF
    str x0, [x1]
    // unit_length (patched)
    bl _kcc_dw_pos
    mov x19, x0
    mov x0, #0
    bl _kcc_dw_u32
    mov x0, #4
    bl _kcc_dw_u16              // version
    // header_length (patched)
    bl _kcc_dw_pos
    mov x20, x0
    mov x0, #0
    bl _kcc_dw_u32
    mov w0, #4
    bl _kcc_dw_u8               //
  minimum_instruction_length (ARM64)
    mov w0, #1
    bl _kcc_dw_u8               //
  maximum_operations_per_instruction
    mov w0, #1
    bl _kcc_dw_u8               // default_is_stmt
    mov w0, #-5
    bl _kcc_dw_u8               // line_base (
  signed)
    mov w0, #14
    bl _kcc_dw_u8               // line_range
    mov w0, #13
    bl _kcc_dw_u8               // opcode_base
    // standard_opcode_lengths for opcodes 1..12
    mov w0, #0
    bl _kcc_dw_u8               // 1 copy
    mov w0, #1
    bl _kcc_dw_u8               // 2 advance_pc
    mov w0, #1
    bl _kcc_dw_u8               // 3 advance_line
    mov w0, #1
    bl _kcc_dw_u8               // 4 set_file
    mov w0, #1
    bl _kcc_dw_u8               // 5 set_column
    mov w0, #0
```

```
    bl _kcc_dw_u8                // 6 negate_stmt
    mov w0, #0
    bl _kcc_dw_u8                // 7
  set_basic_block
    mov w0, #0
    bl _kcc_dw_u8                // 8 const_add_pc
    mov w0, #1
    bl _kcc_dw_u8                // 9
  fixed_advance_pc
    mov w0, #0
    bl _kcc_dw_u8                // 10
  set_prologue_end
    mov w0, #0
    bl _kcc_dw_u8                // 11
  set_epilogue_begin
    mov w0, #1
    bl _kcc_dw_u8                // 12 set_isa
    // include_directories: empty → single
  terminating 0
    mov w0, #0
    bl _kcc_dw_u8
    // file_names: one entry for the source file
    adrp x0, _kcc_src_path@PAGE
    add x0, x0, _kcc_src_path@PAGEOFF
    ldr x0, [x0]
    bl _kcc_dw_str
    mov x0, #0
    bl _kcc_dw_uleb              // dir index
    mov x0, #0
    bl _kcc_dw_uleb              // mtime
    mov x0, #0
    bl _kcc_dw_uleb              // length
    mov w0, #0
    bl _kcc_dw_u8                // end of file
  table
    // patch header_length = (here - (patchpos+4))
    bl _kcc_dw_pos
    add x1, x20, #4
    sub x1, x0, x1
    mov x0, x20
    bl _kcc_dw_patch32
    // --- line-number program ---
    mov x24, #1                                //
  line register starts at 1
    adrp x0, _kcc_ir_funcs@PAGE
    add x0, x0, _kcc_ir_funcs@PAGEOFF
    ldr x22, [x0]
    cbz x22, .Ldwline_endseq
    ldr x23, [x22, #VEC_COUNT]
    mov x21, #0
.Ldwline_loop:
    cmp x21, x23
    b.ge .Ldwline_endseq
    mov x0, x22
    mov x1, x21
    bl _kcc_vec_get
    ldr x2, [x0]    // func
    cbz x2, .Ldwline_next
    // DW_LNE_set_address low_pc
    mov w0, #0
    bl _kcc_dw_u8                // extended opcode
    intro
    mov x0, #9
    bl _kcc_dw_uleb              // length = 1 + 8
    mov x0, #DW_LNE_set_address
    bl _kcc_dw_u8
    mov x0, x22
    mov x1, x21
    bl _kcc_vec_get
    ldr x0, [x0]
    ldr x0, [x0, #IRF_LOWPC]
    bl _kcc_dw_u64
    // DW_LNS_advance_line (target_line - current)
  : opcode first, then the
    // delta computed and sleb'd back-to-back so
  no temp crosses a call.
    mov x0, #DW_LNS_advance_line
    bl _kcc_dw_u8
    mov x0, x22
    mov x1, x21
    bl _kcc_vec_get
    ldr x0, [x0]
    ldr x0, [x0, #IRF_LINE]
    sub x1, x0, x24                            //
  delta
    mov x24, x0                                //
  current = target line
    mov x0, x1
    bl _kcc_dw_sleb
    // DW_LNS_copy → emit a row
    mov x0, #DW_LNS_copy
    bl _kcc_dw_u8
.Ldwline_next:
    add x21, x21, #1
    b .Ldwline_loop
.Ldwline_endseq:
    // final set_address to code size, then
  end_sequence
    mov w0, #0
    bl _kcc_dw_u8
    mov x0, #9
    bl _kcc_dw_uleb
    mov x0, #DW_LNE_set_address
    bl _kcc_dw_u8
    adrp x0, _kcc_code_pos@PAGE
    add x0, x0, _kcc_code_pos@PAGEOFF
    ldr x0, [x0]
    bl _kcc_dw_u64
    mov w0, #0
    bl _kcc_dw_u8
    mov x0, #1
    bl _kcc_dw_uleb
    mov x0, #DW_LNE_end_sequence
    bl _kcc_dw_u8
    // patch unit_length
    bl _kcc_dw_pos
    add x1, x19, #4
    sub x1, x0, x1
    mov x0, x19
    bl _kcc_dw_patch32
    // record section length
    bl _kcc_dw_pos
    adrp x1, _kcc_dw_line_off@PAGE
    add x1, x1, _kcc_dw_line_off@PAGEOFF
    ldr x1, [x1]
    sub x0, x0, x1
    adrp x1, _kcc_dw_line_len@PAGE
    add x1, x1, _kcc_dw_line_len@PAGEOFF
    str x0, [x1]
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

596 · used in 31e · uses _kcc_arena_alloc 58a, _kcc_code_pos 533c, _kcc_ir_funcs 305c, _kcc_src_path 82, _kcc_vec_get 65b

42.4. Mach-O Emission

The Mach-O writer keeps its own layout math; to stay decoupled it hands DWARF two small entry points. `kcc_dwarf_emit_macho` writes the extra `__DWARF` `LC_SEGMENT_64` load command and its three `section_64` descriptors (given the file offset where the blob will land), and

`kcc_dwarf_append_blob` copies the built bytes out immediately after the code section. Placing the blob there—*before* the LINKEDIT region (relocations, symbol table, string table)—is what the linker requires: section data first, LINKEDIT last. A third helper, `kcc_dwarf_total_len`, lets the writer reserve the blob's file span when it computes the relocation offset, so every downstream offset shifts past it. All three are called only under `-g`; the Mach-O writer gates on `_kcc_dwarf_enabled` and reserves the load-command space accordingly, so a non-`-g` object is unchanged.

The `__DWARF` segment is not memory-mapped, but the linker still validates that no two segments' VM ranges overlap. The writer therefore passes `kcc_dwarf_emit_macho` a *vmaddr* placed past the end of `__TEXT` (the code size plus the common globals); the three debug sections take addresses running upward from there while their *file* offsets stay in the blob's own span.

⟨ *DWARF: macho* 600 ⟩ ≡

```
// -- kcc_dwarf_sect64 ---
// Write one 80-byte section_64 in the __DWARF
   segment.
//   x0 = 16-byte section name ptr, x1 = addr, x2
   = size, x3 = file offset
.text
.p2align 2
_kcc_dwarf_sect64:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0
    mov x20, x1
    mov x21, x2
    mov x22, x3
    mov x1, #16
    bl _kcc_macho_write_bytes          // sectname
    (x0=name)
    adrp x0, _kcc_dw_segname@PAGE
    add x0, x0, _kcc_dw_segname@PAGEOFF
    mov x1, #16
    bl _kcc_macho_write_bytes          // segname
  __DWARF
    mov x0, x20
    bl _kcc_macho_write64              // addr
    mov x0, x21
    bl _kcc_macho_write64              // size
    mov x0, x22
    bl _kcc_macho_write32              // offset
    mov w0, #0
    bl _kcc_macho_write32               // align
    mov w0, #0
    bl _kcc_macho_write32               // reloff
    mov w0, #0
    bl _kcc_macho_write32               // nreloc
    mov w0, #0
    bl _kcc_macho_write32               // flags
    mov w0, #0
    bl _kcc_macho_write32               //
  reserved1
    mov w0, #0
    bl _kcc_macho_write32               //
  reserved2
    mov w0, #0
    bl _kcc_macho_write32               //
  reserved3
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_dwarf_emit_macho ---
// void kcc_dwarf_emit_macho(uint64_t
   dwarf_file_base, uint64_t vmaddr)
// Emit the __DWARF LC_SEGMENT_64 + 3 section_64
   records.  vmaddr is placed
// past the __TEXT segment so the segments' VM
   ranges do not overlap (the
// linker rejects overlapping segments).
.globl _kcc_dwarf_emit_macho
.p2align 2
_kcc_dwarf_emit_macho:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    mov x19, x0                                //
  dwarf_base (file offset)
    mov x24, x1                                //
  vmaddr (past __TEXT)
    adrp x0, _kcc_dw_abbrev_len@PAGE
    add x0, x0, _kcc_dw_abbrev_len@PAGEOFF
    ldr x20, [x0]
    adrp x0, _kcc_dw_info_len@PAGE
    add x0, x0, _kcc_dw_info_len@PAGEOFF
    ldr x21, [x0]
    adrp x0, _kcc_dw_line_len@PAGE
    add x0, x0, _kcc_dw_line_len@PAGEOFF
    ldr x22, [x0]
    add x23, x20, x21
    add x23, x23, x22    // total blob size
    // --- LC_SEGMENT_64 (__DWARF) ---
    mov w0, #LC_SEGMENT_64
    bl _kcc_macho_write32
    mov w0, #(72 + 80*3)
    bl _kcc_macho_write32   // cmdsize = 312
    adrp x0, _kcc_dw_segname@PAGE
    add x0, x0, _kcc_dw_segname@PAGEOFF
    mov x1, #16
    bl _kcc_macho_write_bytes  // segname __DWARF
    mov x0, x24
    bl _kcc_macho_write64        // vmaddr (past
  __TEXT)
    mov x0, x23
    bl _kcc_macho_write64       // vmsize = total
    mov x0, x19
    bl _kcc_macho_write64       // fileoff
    mov x0, x23
    bl _kcc_macho_write64       // filesize
    mov w0, #0
    bl _kcc_macho_write32        // maxprot (not
  mapped)
    mov w0, #0
    bl _kcc_macho_write32        // initprot
    mov w0, #3
    bl _kcc_macho_write32        // nsects
    mov w0, #0
    bl _kcc_macho_write32        // flags
    // __debug_abbrev: addr vmaddr, offset
  dwarf_base
    adrp x0, _kcc_dw_sn_abbrev@PAGE
    add x0, x0, _kcc_dw_sn_abbrev@PAGEOFF
    mov x1, x24
    mov x2, x20
    mov x3, x19
    bl _kcc_dwarf_sect64
    // __debug_info: addr vmaddr+abbrev_len,
  offset dwarf_base+abbrev_len
    adrp x0, _kcc_dw_sn_info@PAGE
    add x0, x0, _kcc_dw_sn_info@PAGEOFF
```

```
    add x1, x24, x20
    mov x2, x21
    add x3, x19, x20
    bl _kcc_dwarf_sect64
    // __debug_line: addr vmaddr+abbrev_len+
   info_len, offset dwarf_base+that
    adrp x0, _kcc_dw_sn_line@PAGE
    add x0, x0, _kcc_dw_sn_line@PAGEOFF
    add x2, x20, x21
    add x1, x24, x2
    add x3, x19, x2
    mov x2, x22
    bl _kcc_dwarf_sect64
    ldp x23, x24, [sp, #48]
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_dwarf_total_len ---
// uint64_t kcc_dwarf_total_len(void) ->
   abbrev_len + info_len + line_len.
// The Mach-O layout uses this to reserve the
   debug blob's file span between
// the code bytes and the LINKEDIT region (relocs/
   symtab/strtab).
.globl _kcc_dwarf_total_len
.p2align 2
_kcc_dwarf_total_len:
    adrp x0, _kcc_dw_abbrev_len@PAGE
    add x0, x0, _kcc_dw_abbrev_len@PAGEOFF
    ldr x0, [x0]
    adrp x1, _kcc_dw_info_len@PAGE
    add x1, x1, _kcc_dw_info_len@PAGEOFF
    ldr x1, [x1]
    add x0, x0, x1
    adrp x1, _kcc_dw_line_len@PAGE
    add x1, x1, _kcc_dw_line_len@PAGEOFF
    ldr x1, [x1]
    add x0, x0, x1
    ret

// -- kcc_dwarf_append_blob ---
// void kcc_dwarf_append_blob(void)
// Copy the built debug sections into the Mach-O
   output.  Called right after
// the code bytes so the blob precedes the
   LINKEDIT region.
.globl _kcc_dwarf_append_blob
.p2align 2
_kcc_dwarf_append_blob:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    adrp x0, _kcc_dwarf_buf@PAGE
    add x0, x0, _kcc_dwarf_buf@PAGEOFF
    ldr x0, [x0]
    adrp x1, _kcc_dw_abbrev_len@PAGE
    add x1, x1, _kcc_dw_abbrev_len@PAGEOFF
    ldr x1, [x1]
    adrp x2, _kcc_dw_info_len@PAGE
    add x2, x2, _kcc_dw_info_len@PAGEOFF
    ldr x2, [x2]
    add x1, x1, x2
    adrp x2, _kcc_dw_line_len@PAGE
    add x2, x2, _kcc_dw_line_len@PAGEOFF
    ldr x2, [x2]
    add x1, x1, x2                                  //
   total length
    bl _kcc_macho_write_bytes
    ldp x29, x30, [sp], #16
    ret
```

□

600 · used in 31e · uses _kcc_macho_write32 578, _kcc_macho_write64 578, _kcc_macho_write_bytes 579

42.5. Global State

⟨ DWARF: data 601 ⟩ ≡

```
// -- DWARF global state ---
.data
.p2align 3
.globl _kcc_dwarf_enabled
_kcc_dwarf_enabled: .quad 0     // set by -g
_kcc_dwarf_buf:     .quad 0     // scratch blob
_kcc_dwarf_pos:     .quad 0     // write cursor
.globl _kcc_dw_abbrev_off
_kcc_dw_abbrev_off: .quad 0
.globl _kcc_dw_abbrev_len
_kcc_dw_abbrev_len: .quad 0
.globl _kcc_dw_info_off
_kcc_dw_info_off:   .quad 0
.globl _kcc_dw_info_len
_kcc_dw_info_len:   .quad 0
.globl _kcc_dw_line_off
_kcc_dw_line_off:   .quad 0
.globl _kcc_dw_line_len
_kcc_dw_line_len:   .quad 0
.globl _kcc_dwarf_buf
.globl _kcc_dwarf_pos

// Fixed 16-byte section/segment names for the
   Mach-O section_64 records.
.p2align 3
_kcc_dw_segname:    .ascii "__DWARF"
    .space 9
_kcc_dw_sn_abbrev:  .ascii "__debug_abbrev"
    .space 2
_kcc_dw_sn_info:    .ascii "__debug_info"
    .space 4
_kcc_dw_sn_line:    .ascii "__debug_line"
    .space 4

.section __TEXT,__cstring,cstring_literals
_kcc_dw_producer:   .asciz "kcc"
_kcc_dw_compdir:    .asciz "/"
```

□

601 · used in 31e

43. Compilation Driver

The preceding chapters implement each compilation phase in isolation; this chapter wires them together into a single end-to-end pipeline. When the user invokes `kcc -c`, the driver opens the source file, runs translation phases, parses, type-checks, lowers to IR, optimizes, generates machine code, and writes the resulting Mach-O object file. The pipeline is deliberately sequential and straight-line—each phase completes before the next begins—so that errors are caught early and the overall flow is easy to trace.

43.1. Full Compilation Pipeline

The driver orchestrates every phase of compilation in a single straight-line function. Given a source path and an output path it:

1. Opens and reads the source file.
2. Runs translation phases 1–2 (trigraph replacement and line splicing).
3. Initializes the lexer, preprocessor, string table, and typedef table.
4. Parses the translation unit into an AST.
5. Initializes the type system and symbol table, then runs semantic analysis.
6. Builds the IR and runs optimization passes.
7. Initializes the code buffer, linker, and code generator (including the cstring buffer for string literals).
8. Pre-registers global variable sizes so that struct globals receive the correct allocation (one 8-byte slot per field).
9. Lowers all IR to ARM64 machine code.
10. Appends string literal data to the code section and patches symbol values (`kcc_codegen_emit_strings`).
11. Writes the Mach-O object file.

⟨ *Driver: compile* 603 ⟩ ≡

```
// -- kcc_driver_compile ---
// void kcc_driver_compile(const char *src_path,
   const char *out_path)
//   x0 = source file path
//   x1 = output file path
.text
.globl _kcc_driver_compile
.p2align 2
_kcc_driver_compile:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                     // src_path
    mov x20, x1                     // out_path

    // 1. Open and read source file
    mov x0, x19
    bl _kcc_source_open

    // 2. Translation Phase 1: trigraph
replacement
    bl _kcc_source_encode

    // 3. Translation Phase 2: line splicing
    bl _kcc_source_splice

    // 4. Initialize lexer
    adrp x0, _kcc_src_base@PAGE
    add  x0, x0, _kcc_src_base@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_lex_init

    // 5. Initialize preprocessor
    bl _kcc_pp_init

    // 6. Initialize string table and typedef
table
    bl _kcc_strtab_init
    bl _kcc_typedef_init

    // 7. Parse
    bl _kcc_parse
    mov x21, x0                     // trans_unit

    // 8. Initialize type system and symbol table
    bl _kcc_types_init
    bl _kcc_symtab_init

    // 9. Semantic analysis
    mov x0, x21
    bl _kcc_sema_check

    // 10. Check for errors
    bl _kcc_diag_had_errors
    cbnz x0, .Ldc_error

    // 11. Build IR
    bl _kcc_ir_init
    mov x0, x21
    bl _kcc_ir_build

    // 12. Optimize
    bl _kcc_opt_run

    // 12b. Verify the optimized IR before handing
    it to the back end.  A
    // structural violation (bad opcode,
unterminated block, cyclic list) is an
    // internal error, not a user error, so it
aborts the compile rather than
    // letting the code generator produce garbage.
    bl _kcc_ir_validate
    cbnz x0, .Ldc_error

    // 12c. Target dispatch.  Under --target=
x86_64 the IR is handed to the
    // x86-64 assembly-emission back end (Chapter
~\ref{ch:emit-x86}), which
    // walks the same optimized IR and assembles
with `as -arch x86_64`,
```

```
    // bypassing the ARM64 code generator and Mach-O writer entirely.  The
    // default (0) leaves the flagship ARM64 path below completely untouched.
    adrp x0, _kcc_target@PAGE
    add  x0, x0, _kcc_target@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Ldc_arm64
    // Publish the AST translation unit so the x86 back end can emit storage for
    // file-scope globals (its data pass walks the same tree the ARM64
    // pre-registration does).
    adrp x1, _kcc_x86_tu@PAGE
    add  x1, x1, _kcc_x86_tu@PAGEOFF
    str x21, [x1]                   // x21 = trans_unit
    mov x0, x20                     // out_path
    bl _kcc_driver_x86_object
    b .Ldc_return
.Ldc_arm64:

    // 13. Initialize backend
    bl _kcc_code_init
    bl _kcc_linker_init
    bl _kcc_codegen_init

    // 13b. Pre-register global variable sizes
    mov x0, x21                     // trans_unit
    bl _kcc_preregister_globals

    // 14. Generate code
    bl _kcc_codegen_all

    // 14b. Append string literal data to code section
    bl _kcc_codegen_emit_strings

    // 14z. Emission strategy.  Under --emit-asm the object is produced through
    // the textual `.s` + platform `as` back end; otherwise the direct Mach-O
    // writer runs (with an optional KCC_EMIT_ASM=<path> side-render for
    // bring-up, which leaves the Mach-O path unaffected).
    adrp x0, _kcc_emit_asm@PAGE
    add  x0, x0, _kcc_emit_asm@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Ldc_macho
    mov x0, x20                     // out_path
    bl _kcc_driver_emit_asm_object
    b .Ldc_return

.Ldc_macho:
    adrp x0, _kcc_as_s_env@PAGE
    add  x0, x0, _kcc_as_s_env@PAGEOFF
    mov x1, #12
    bl _kcc_getenv
    cbz x0, .Ldc_noasm
    bl _kcc_as_render
.Ldc_noasm:

    // 14c. Build DWARF debug sections (only under -g); code size and all
    // function addresses are final by now.
    bl _kcc_dwarf_build

    // 15. Write Mach-O object file
    mov x0, x20                     // out_path
    bl _kcc_macho_init
    bl _kcc_macho_finalize

    // 16. Success
    mov x0, #0
    b .Ldc_return

.Ldc_error:
    bl _kcc_io_flush
    mov x0, #1

.Ldc_return:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_driver_emit_asm_object ---
// int kcc_driver_emit_asm_object(const char *out_path)
// Renders the generated code to "<out_path>.s", then runs the platform
// assembler (`as -o <out_path> <out_path>.s`) to produce the object file.
// Returns 0 on success, 1 on error.
.globl _kcc_driver_emit_asm_object
.p2align 2
_kcc_driver_emit_asm_object:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // out_path
    // Build "<out_path>.s" (out_len + 3 bytes for ".s\0").
    bl _kcc_strlen
    mov x20, x0                     // out_len
    add x0, x20, #3
    bl _kcc_arena_alloc
    mov x21, x0                     // out_s buffer
    mov x0, x21
    mov x1, x19
    mov x2, x20
    bl _kcc_memcpy                  // copy out_path
    add x0, x21, x20                // &out_s[out_len]
    adrp x1, _kcc_str_dot_s@PAGE
    add  x1, x1, _kcc_str_dot_s@PAGEOFF
    mov x2, #3                      // ".s\0"
    bl _kcc_memcpy
    // Render the assembly to out_s.
    mov x0, x21
    bl _kcc_as_render
    cbnz x0, .Leao_err
    // Build argv = { "as", "-o", out_path, out_s, 0 } and run /usr/bin/as.
    mov x0, #48                     // 6 pointers
    bl _kcc_arena_alloc
    mov x22, x0
    adrp x0, _kcc_str_as_argv0@PAGE
    add  x0, x0, _kcc_str_as_argv0@PAGEOFF
    str x0, [x22, #0]
    adrp x0, _kcc_opt_o@PAGE
    add  x0, x0, _kcc_opt_o@PAGEOFF
    str x0, [x22, #8]
    str x19, [x22, #16]             // out_path
    str x21, [x22, #24]             // out_s
    str xzr, [x22, #32]             // NULL terminator
    adrp x0, _kcc_str_as_path@PAGE
    add  x0, x0, _kcc_str_as_path@PAGEOFF
    mov x1, x22
    bl _kcc_driver_spawn            // returns child exit status
    cbnz x0, .Leao_err
    mov x0, #0
    b .Leao_done
.Leao_err:
```

```
    mov x0, #1
.Leao_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret

// -- kcc_driver_x86_object ---
// int kcc_driver_x86_object(const char *out_path)
// Renders the x86-64 assembly to "<out_path>.s",
   then runs the platform
// assembler for x86-64 (`as -arch x86_64 -o <
   out_path> <out_path>.s`) to
// produce the Mach-O object.  Returns 0 on
   success, 1 on error.  Mirrors
// kcc_driver_emit_asm_object, but renders x86-64
   and passes `-arch x86_64`.
.globl _kcc_driver_x86_object
.p2align 2
_kcc_driver_x86_object:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                         // out_path

    // Linux/ELF target: this compiler runs on
   Darwin, whose `as` emits Mach-O,
    // not ELF.  So under --target=x86_64-linux
   kcc renders the ELF-flavour
    // assembly straight to the output path and
   stops---the object is produced
    // by assembling that `.s` on a Linux host (or
    with a cross-assembler).
    adrp x0, _kcc_x86_os@PAGE
    add  x0, x0, _kcc_x86_os@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lxo_darwin
    mov x0, x19                         // out_path
   receives the assembly text
    bl _kcc_x86_render
    b .Lxo_done
.Lxo_darwin:
    // Build "<out_path>.s".  (Restore x0 =
   out_path; the os check above clobbered it.)
    mov x0, x19
    bl _kcc_strlen
    mov x20, x0                         // out_len
    add x0, x20, #3
    bl _kcc_arena_alloc
    mov x21, x0                         // out_s buffer
    mov x0, x21
    mov x1, x19
    mov x2, x20
    bl _kcc_memcpy                      // copy
   out_path
    add x0, x21, x20                   // &out_s[
   out_len]
    adrp x1, _kcc_str_dot_s@PAGE
    add  x1, x1, _kcc_str_dot_s@PAGEOFF
    mov x2, #3                          // ".s\0"
    bl _kcc_memcpy
    // Render the x86-64 assembly to out_s.
    mov x0, x21
    bl _kcc_x86_render
    cbnz x0, .Lxo_err
    // Build argv = { "as", "-arch", "x86_64", "-o
   ", out_path, out_s, 0 }.
    mov x0, #56                         // 7 pointers
    bl _kcc_arena_alloc
    mov x22, x0
    adrp x0, _kcc_str_as_argv0@PAGE
    add  x0, x0, _kcc_str_as_argv0@PAGEOFF
    str x0, [x22, #0]
    adrp x0, _kcc_str_arch@PAGE
    add  x0, x0, _kcc_str_arch@PAGEOFF
    str x0, [x22, #8]
    adrp x0, _kcc_str_x86_64@PAGE
    add  x0, x0, _kcc_str_x86_64@PAGEOFF
    str x0, [x22, #16]
    adrp x0, _kcc_opt_o@PAGE
    add  x0, x0, _kcc_opt_o@PAGEOFF
    str x0, [x22, #24]
    str x19, [x22, #32]             // out_path
    str x21, [x22, #40]             // out_s
    str xzr, [x22, #48]             // NULL
   terminator
    adrp x0, _kcc_str_as_path@PAGE
    add  x0, x0, _kcc_str_as_path@PAGEOFF
    mov x1, x22
    bl _kcc_driver_spawn
    cbnz x0, .Lxo_err
    mov x0, #0
    b .Lxo_done
.Lxo_err:
    mov x0, #1
.Lxo_done:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #64
    ret
```

□

603 · defined at 603, 606ab, 607, 608ab, 609 · used in 31f · defines _kcc_driver_compile 609, 611, _kcc_driver_emit_asm_object never used, _kcc_driver_x86_object never used · uses _kcc_arena_alloc 58a, _kcc_codegen_all 473, _kcc_codegen_emit_strings 478, _kcc_codegen_init 463b, _kcc_code_init 525b, _kcc_diag_had_errors 76d, _kcc_driver_spawn 611, _kcc_getenv 611, _kcc_io_flush 69b, _kcc_ir_build 307, _kcc_ir_init 302, _kcc_ir_validate 441, _kcc_lex_init 90a, _kcc_linker_init 591b, _kcc_macho_finalize 580, _kcc_macho_init 577b, _kcc_opt_run 445a, _kcc_parse 172, _kcc_pp_init 107b, _kcc_preregister_globals 465, _kcc_sema_check 259a, _kcc_source_encode 85, _kcc_source_open 81b, _kcc_source_splice 87, _kcc_src_base 82, _kcc_src_size 82, _kcc_strtab_init 66a, _kcc_symtab_init 253b, _kcc_target 568b, _kcc_typedef_init 170a, _kcc_types_init 239b, _kcc_x86_render 568b, _kcc_x86_tu 568b

43.2. Compiling Several Translation Units

The pipeline above compiles one file; a real project has many. So that a single `kcc -c` invocation can turn a whole subsystem into object files—the shape the self-hosting build needs—the driver loops `_kcc_driver_compile` over every source argument. This is why that function now *returns* a status (0 success, 1 failure) instead of exiting: the loop must survive an individual file's outcome, accumulate the failures, and exit once at the end with the combined result.

Both scanning passes begin at `argv[1]` and skip `-c` wherever it sits. A hand-written invocation puts `-c` first, but a gcc-driven build (Kbuild) leads with the flags and ends with `-c -o out.o in.c`; treating `-c` as the compile-only selector regardless of position—rather than assum-

ing `argv[1]`—lets kcc stand in as a drop-in `CC=` for such command lines. `_kcc_opt_compile` (the `"-c"` string) is therefore defined here in the driver, so the scan resolves it whether or not the assembly `main`—which the self-host build can replace with the C port—is linked in.

Because every per-translation-unit subsystem (`ir_init`, `code_init`, `linker_init`, the string, typedef, type, and symbol tables) reinitialises its state at the top of `_kcc_driver_compile`, each file starts from a clean slate; only the arena grows monotonically, which is harmless.

Cross-unit sharing works through ordinary external linkage. A function called without a definition in the current file already becomes an undefined symbol the linker resolves against the object that defines it. Data is symmetric once `extern` is honoured: a plain file-scope `int total;` reserves storage, while `extern int total;` in another unit references that same storage without redefining it (§35.4).

Naming the object file. When the output is not given explicitly, each object file takes the name of its source with a `.c` suffix rewritten to `.o` (or `.o` appended if there is no such suffix).

The `-Wall` flag. Both passes recognise `-Wall` the same way they recognise `-o`: as an argument that is *not* a source file. The first pass, on seeing it, sets `_kcc_warn_all` so the warning checks in semantic analysis switch on; the second pass simply skips it. Because the flag is matched before the source-file fall-through, it may appear anywhere in the argument list without being mistaken for an input.

The `-I` flag. An include directory arrives in either of two shapes: attached, `-Idir`, or separated, `-I dir`. `_kcc_driver_I` recognises both from a two-byte `-I` prefix and reports how many argument slots the option occupies—one for the attached form, two for the separated form—so each parsing loop can register or skip it uniformly. The `do_register` argument distinguishes the roles: the counting pre-pass calls it with registration *on* (adding the directory to the search path via `_kcc_pp_add_include_dir`), while the compile pass calls it with registration *off*, using only the returned slot count to step over the option. This keeps a directory from being registered twice and localises the attached-vs-separated logic to one place.

The `-D` flag. `_kcc_driver_D` is the exact analogue for command-line macro definitions, in the same two shapes: attached `-DNAME` or `-DNAME=VALUE`, and separated `-D NAME`. A bare name takes the value `1` (the convention `cc` follows), while an `=`-suffix supplies an explicit body. Because the macro table is rebuilt for every translation unit, the scan pass merely *records* each definition's payload (via `_kcc_pp_add_cmdline_define`); the actual macros are (re)installed from that list at the end of `_kcc_pp_predefine`, which runs on every preprocessor initialisation. So each source sees the same command-line macros on top of the built-in predefines, and configuration switches such as `-DNDEBUG` or a library's `-DLUA_USE_POSIX` behave as they do under any C compiler.

⟨ *Driver: compile* 606a ⟩ + ≡

```
// -- kcc_driver_I ---
// uint64_t kcc_driver_I(const char *arg, const
   char *next, uint64_t do_register)
//   Detect a -I option: attached "-Idir" or
   separated "-I dir".
//   Returns slots consumed: 0 = not -I, 1 =
   attached, 2 = separated.
//   When do_register != 0, registers the
   directory with the preprocessor.
.text
.p2align 2
_kcc_driver_I:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                     // arg
    mov x20, x1                     // next
    mov x21, x2                     // do_register
    mov x0, x19
    adrp x1, _kcc_opt_incdir@PAGE
    add  x1, x1, _kcc_opt_incdir@PAGEOFF
    mov  x2, #2
    bl _kcc_memcmp
    cbnz x0, .LdI_no                // arg does not
    start with "-I"
    ldrb w0, [x19, #2]
    cbz  w0, .LdI_sep               // exactly "-I
  ": separated form
    // Attached "-Idir": directory is arg+2
    cbz  x21, .LdI_att_ret
    add  x0, x19, #2
    bl _kcc_pp_add_include_dir
.LdI_att_ret:
    mov  x0, #1
    b .LdI_ret
.LdI_sep:
    cbz  x21, .LdI_sep_ret
    cbz  x20, .LdI_sep_ret          // no following
    argument to register
    mov  x0, x20
    bl _kcc_pp_add_include_dir
.LdI_sep_ret:
    mov  x0, #2
    b .LdI_ret
.LdI_no:
    mov  x0, #0
.LdI_ret:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

606a · defined at 603, 606ab, 607, 608ab, 609 · used in 31f · defines `_kcc_driver_I` 609, 611 · uses `_kcc_pp_add_include_dir` 150

⟨ *Driver: compile* 606b ⟩ + ≡

```
// -- kcc_driver_D ---
// uint64_t kcc_driver_D(const char *arg, const
   char *next, uint64_t do_register)
//   Detect a -D option: attached "-DNAME"/"-DNAME
   =VALUE" or separated "-D NAME".
//   Returns slots consumed: 0 = not -D, 1 =
   attached, 2 = separated.  With
//   do_register != 0 the definition's payload is
   recorded for the preprocessor.
//   Mirrors kcc_driver_I so every argument-
   parsing pass handles -D uniformly.
```

```
.text
.p2align 2
_kcc_driver_D:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                     // arg
    mov x20, x1                     // next
    mov x21, x2                     // do_register
    mov x0, x19
    adrp x1, _kcc_opt_defmac@PAGE
    add  x1, x1, _kcc_opt_defmac@PAGEOFF
    mov  x2, #2
    bl _kcc_memcmp
    cbnz x0, .LdD_no                // arg does not
    start with "-D"
    ldrb w0, [x19, #2]
    cbz  w0, .LdD_sep               // exactly "-D
   ": separated form
    // Attached "-DNAME[=VALUE]": payload is arg+2
    cbz  x21, .LdD_att_ret
    add  x0, x19, #2
    bl _kcc_pp_add_cmdline_define
.LdD_att_ret:
    mov  x0, #1
    b .LdD_ret
.LdD_sep:
    cbz  x21, .LdD_sep_ret
    cbz  x20, .LdD_sep_ret          // no following
    argument to record
    mov  x0, x20
    bl _kcc_pp_add_cmdline_define
.LdD_sep_ret:
    mov  x0, #2
    b .LdD_ret
.LdD_no:
    mov  x0, #0
.LdD_ret:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
```

□

606b · defined at 603, 606ab, 607, 608ab, 609 · used in 31f · defines _kcc_driver_D 609, 611 · uses _kcc_pp_add_cmdline_define 132

The kernel is compiled with two code-generation flags that every argument pass must recognise: `-mno-red-zone` and `-mcmodel=kernel`. Both are already satisfied by kcc's x86-64 output. The red zone—the 128 bytes below %rsp that a leaf function may use without adjusting the stack—is never touched, because every function allocates its whole frame with an explicit subq; and the code model is immaterial because symbol references are RIP-relative (`leaq sym(%rip)`, direct `call sym`) and large immediates are `movabsq`, all of which resolve identically whether the image sits in the low or the top two gigabytes. So `kcc_driver_mflag` accepts these flags as no-ops—`-mcmodel=small` and `kernel` and either red-zone spelling—and refuses `-mcmodel=large/medium` cleanly, since kcc's direct calls and RIP-relative data cannot span the four-gigabyte reach those models allow. It returns 1 for a flag to skip and 0 for anything else, leaving unrelated arguments to the caller.

⟨ *Driver: compile* 607 ⟩ + ≡

```
// -- kcc_driver_mflag ---
// uint64_t kcc_driver_mflag(const char *arg)
//   Recognise the x86 code-model / red-zone flags
   .  Returns 1 if arg is one
//   kcc honours (skip it), 0 if it is not an -m
   flag we handle.  An -mcmodel=
//   with an unsupported model is refused here (
   message + exit).
.text
.p2align 2
.globl _kcc_driver_mflag
_kcc_driver_mflag:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                         // arg
    mov x0, x19
    adrp x1, _kcc_opt_mnoredzone@PAGE
    add  x1, x1, _kcc_opt_mnoredzone@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lmf_yes
    mov x0, x19
    adrp x1, _kcc_opt_mredzone@PAGE
    add  x1, x1, _kcc_opt_mredzone@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lmf_yes
    mov x0, x19
    adrp x1, _kcc_opt_mcmodel@PAGE
    add  x1, x1, _kcc_opt_mcmodel@PAGEOFF
    mov  x2, #9                         // strlen("-
   mcmodel=")
    bl _kcc_memcmp
    cbnz x0, .Lmf_no                    // not a -
   mcmodel= flag
    add x19, x19, #9                    // suffix after
    "-mcmodel="
    mov x0, x19
    adrp x1, _kcc_str_cm_small@PAGE
    add  x1, x1, _kcc_str_cm_small@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lmf_yes
    mov x0, x19
    adrp x1, _kcc_str_cm_kernel@PAGE
    add  x1, x1, _kcc_str_cm_kernel@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lmf_yes
    // unsupported model: refuse cleanly
    adrp x0, _kcc_driver_err_cmodel@PAGE
    add  x0, x0, _kcc_driver_err_cmodel@PAGEOFF
    bl _kcc_io_err
    bl _kcc_io_flush
    mov x0, #1
    bl _kcc_sys_exit
.Lmf_yes:
    mov x0, #1
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
.Lmf_no:
    mov x0, #0
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

607 · defined at 603, 606ab, 607, 608ab, 609 · used in 31f · defines _kcc_driver_mflag 608b · uses _kcc_io_err 71b, _kcc_io_flush 69b

A real Kbuild command line carries dozens of code-generation, tuning, warning, and optimisation flags—`-O2`, `-fno-strict-aliasing`, `-fno-stack-protector`, `-mno-sse`, `-mtune=generic`, `-Wno-sign-compare`, `-std=gnu11`, `-nostdinc`, and many more. None of them change the code kcc generates for the freestanding in-

teger subset the kernel is written in: kcc has a single fixed optimisation pipeline, never emits vector registers except for actual floating point (which kernel C avoids), and has no strict-aliasing or stack-protector passes to turn off. So `kcc_driver_xflag` accepts the whole family as no-ops by prefix—anything beginning `-O`, `-f`, `-W` (past the `-Wall` handled earlier), `-m` (past the code-model flags), `-std=`, `-pipe`, or `-nostdinc`. This is a deliberate simplification, not full support: the flags are ignored, not implemented. `kcc_driver_skipopt` chains the code-model recogniser and this one, so a single call at each argument scan consumes any tuning flag wherever it appears. It also recognises `-w` directly: unlike the prefix no-ops, `-w` carries meaning—inhibit every warning (§gcc -w)—so `skipopt` forces `_kcc_warn_all` back to 0 as it consumes the flag. Because both the compile and link argument scanners route unrecognised options through `skipopt`, this one check makes `-w` work in every mode, so a build that passes `-w` (doomgeneric's `build_ref.sh`, most autotools projects) compiles with kcc directly, with no wrapper to strip the flag.

⟨ *Driver: compile* 608a ⟩ + ≡

```
// -- kcc_driver_xflag ---
// uint64_t kcc_driver_xflag(const char *arg)
//   1 if arg is a single-slot tuning/warning/
   optimisation flag kcc tolerates as
//   a no-op, matched by prefix; 0 otherwise.
.text
.p2align 2
_kcc_driver_xflag:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0                         // arg
    mov x0, x19
    adrp x1, _kcc_xf_dO@PAGE
    add  x1, x1, _kcc_xf_dO@PAGEOFF
    mov  x2, #2
    bl _kcc_memcmp
    cbz x0, .Lxf_yes
    mov x0, x19
    adrp x1, _kcc_xf_df@PAGE
    add  x1, x1, _kcc_xf_df@PAGEOFF
    mov  x2, #2
    bl _kcc_memcmp
    cbz x0, .Lxf_yes
    mov x0, x19
    adrp x1, _kcc_xf_dW@PAGE
    add  x1, x1, _kcc_xf_dW@PAGEOFF
    mov  x2, #2
    bl _kcc_memcmp
    cbz x0, .Lxf_yes
    mov x0, x19
    adrp x1, _kcc_xf_dm@PAGE
    add  x1, x1, _kcc_xf_dm@PAGEOFF
    mov  x2, #2
    bl _kcc_memcmp
    cbz x0, .Lxf_yes
    mov x0, x19
    adrp x1, _kcc_xf_std@PAGE
    add  x1, x1, _kcc_xf_std@PAGEOFF
    mov  x2, #5                         // "-std="
    bl _kcc_memcmp
    cbz x0, .Lxf_yes
    mov x0, x19
    adrp x1, _kcc_xf_pipe@PAGE
    add  x1, x1, _kcc_xf_pipe@PAGEOFF
    mov  x2, #5                         // "-pipe"
    bl _kcc_memcmp
    cbz x0, .Lxf_yes
    mov x0, x19
    adrp x1, _kcc_xf_nostdinc@PAGE
    add  x1, x1, _kcc_xf_nostdinc@PAGEOFF
    mov  x2, #9                         // "-nostdinc"
    bl _kcc_memcmp
    cbz x0, .Lxf_yes
    mov x0, #0
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
.Lxf_yes:
    mov x0, #1
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

608a · defined at 603, 606ab, 607, 608ab, 609 · used in 31f · defines `_kcc_driver_xflag` 608b

⟨ *Driver: compile* 608b ⟩ + ≡

```
// -- kcc_driver_skipopt ---
// uint64_t kcc_driver_skipopt(const char *arg)
//   1 if arg is a code-model / red-zone / tuning
   flag to skip as a non-source
//   option (mflag may exit for an unsupported
   code model); 0 otherwise.
.text
.p2align 2
.globl _kcc_driver_skipopt
_kcc_driver_skipopt:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    mov x19, x0
    // "-w": inhibit all warnings (\stdref{gcc -w
   }).  kcc keeps the -Wall family
    // off by default, so this forces
   _kcc_warn_all back to 0 and consumes the
    // flag; both the compile and link flag
   scanners route unrecognised options
    // here, so one check covers every path (no
   wrapper needed to strip -w).
    adrp x1, _kcc_opt_wsuppress@PAGE
    add  x1, x1, _kcc_opt_wsuppress@PAGEOFF
    bl _kcc_strcmp
    cbnz x0, .Lso_chk_m
    adrp x0, _kcc_warn_all@PAGE
    add  x0, x0, _kcc_warn_all@PAGEOFF
    str  xzr, [x0]
    b .Lso_yes
.Lso_chk_m:
    mov x0, x19
    bl _kcc_driver_mflag
    cbnz x0, .Lso_yes
    mov x0, x19
    bl _kcc_driver_xflag
    cbnz x0, .Lso_yes
    mov x0, #0
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
.Lso_yes:
    mov x0, #1
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

608b · defined at 603, 606ab, 607, 608ab, 609 · used in 31f · defines `_kcc_driver_skipopt` 609, 611 · uses `_kcc_driver_mflag` 607, `_kcc_driver_xflag` 608a,

_kcc_warn_all 77a

⟨ Driver: compile 609 ⟩ + ≡

```
// -- kcc_driver_obj_name ---
// char *kcc_driver_obj_name(const char *src)
//   x0 = source path → arena path with ".c"
   rewritten to ".o"
.text
.globl _kcc_driver_obj_name
.p2align 2
_kcc_driver_obj_name:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]

    mov x19, x0                     // src
    bl _kcc_strlen
    mov x20, x0                     // len

    add x0, x20, #3                 // len + ".o" +
    null
    bl _kcc_arena_alloc
    mov x21, x0                     // dst

    mov x0, x21
    mov x1, x19
    mov x2, x20
    bl _kcc_memcpy

    // ".c" suffix? (len >= 2, src[len-2]=='.',
   src[len-1]=='c')
    cmp x20, #2
    b.lt .Lobj_append
    sub x2, x20, #2
    ldrb w3, [x19, x2]
    cmp w3, #'.'
    b.ne .Lobj_append
    sub x2, x20, #1
    ldrb w3, [x19, x2]
    cmp w3, #'c'
    b.ne .Lobj_append

    // Rewrite the trailing 'c' to 'o'
    sub x2, x20, #1
    mov w3, #'o'
    strb w3, [x21, x2]              // dst[len-1] =
    'o'
    strb wzr, [x21, x20]            // null at len
    mov x0, x21
    b .Lobj_done

.Lobj_append:
    mov w3, #'.'
    strb w3, [x21, x20]
    add x2, x20, #1
    mov w3, #'o'
    strb w3, [x21, x2]
    add x2, x20, #2
    strb wzr, [x21, x2]
    mov x0, x21

.Lobj_done:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_driver_compile_all ---
// void kcc_driver_compile_all(int argc, char **
   argv)
//   x0 = argc, x1 = argv.  Handles "-c": compiles
    every source argument
//   (argv[2..]) to an object file, honouring "-o
   <file>" only when there is
//   exactly one source.  Exits 0 if all compiles
   succeed, else 1.
.text
.globl _kcc_driver_compile_all
.p2align 2
_kcc_driver_compile_all:
    stp x29, x30, [sp, #-96]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    str x25, [sp, #64]

    mov x19, x0                     // argc
    mov x20, x1                     // argv

    // Pass 1: locate an explicit "-o <out>" and
   count source files.  Scan from
    // argv[1]: `-c` may sit anywhere (gcc/Kbuild
   ordering), so it is skipped as
    // an option rather than assumed to be argv
   [1].
    mov x21, #0                     // explicit out
    (0 = none)
    mov x22, #0                     // source count
    mov x23, #1                     // i = 1
.Lca_scan:
    cmp x23, x19
    b.ge .Lca_scan_done
    ldr x0, [x20, x23, lsl #3]      // argv[i]
    adrp x1, _kcc_opt_compile@PAGE
    add  x1, x1, _kcc_opt_compile@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_scan_skipc
    ldr x0, [x20, x23, lsl #3]      // argv[i]
    adrp x1, _kcc_opt_wall@PAGE
    add  x1, x1, _kcc_opt_wall@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_scan_wall
    ldr x0, [x20, x23, lsl #3]      // argv[i]
    adrp x1, _kcc_opt_emitasm@PAGE
    add  x1, x1, _kcc_opt_emitasm@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_scan_emitasm
    ldr x0, [x20, x23, lsl #3]      // argv[i]
    adrp x1, _kcc_opt_target_x86@PAGE
    add  x1, x1, _kcc_opt_target_x86@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_scan_target
    ldr x0, [x20, x23, lsl #3]      // argv[i]
    adrp x1, _kcc_opt_target_linux@PAGE
    add  x1, x1, _kcc_opt_target_linux@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_scan_target_linux
    ldr x0, [x20, x23, lsl #3]      // argv[i]
    adrp x1, _kcc_opt_g@PAGE
    add  x1, x1, _kcc_opt_g@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_scan_g
    ldr x0, [x20, x23, lsl #3]      // argv[i]
    adrp x1, _kcc_opt_o@PAGE
    add  x1, x1, _kcc_opt_o@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_scan_o
    // -Idir / -I dir: register the include
   directory; not a source
    ldr x0, [x20, x23, lsl #3]      // argv[i]
    add x1, x23, #1
    cmp x1, x19
    b.ge .Lca_scan_I_nonext
    ldr x1, [x20, x1, lsl #3]       // argv[i+1]
    b .Lca_scan_I_call
```

```
.Lca_scan_I_nonext:
    mov x1, #0
.Lca_scan_I_call:
    mov x2, #1                      // register the
    directory
    bl _kcc_driver_I
    cbz x0, .Lca_scan_D
    add x23, x23, x0                // consumed 1
   or 2 slots
    b .Lca_scan
.Lca_scan_D:
    // -DNAME[=VALUE] / -D NAME: record a command-
   line macro; not a source
    ldr x0, [x20, x23, lsl #3]      // argv[i]
    add x1, x23, #1
    cmp x1, x19
    b.ge .Lca_scan_D_nonext
    ldr x1, [x20, x1, lsl #3]       // argv[i+1]
    b .Lca_scan_D_call
.Lca_scan_D_nonext:
    mov x1, #0
.Lca_scan_D_call:
    mov x2, #1                      // register the
    macro
    bl _kcc_driver_D
    cbz x0, .Lca_scan_m
    add x23, x23, x0
    b .Lca_scan
.Lca_scan_m:
    // code-model / red-zone / tuning flag (-
   mcmodel=, -O2, -f*, ...): not a source
    ldr x0, [x20, x23, lsl #3]
    bl _kcc_driver_skipopt
    cbz x0, .Lca_scan_src
    add x23, x23, #1
    b .Lca_scan
.Lca_scan_src:
    add x22, x22, #1                // a source
    add x23, x23, #1
    b .Lca_scan
.Lca_scan_skipc:
    // "-c": the compile-only selector; consumed
   here so it is not mistaken for a
    // source file when it appears among the flags
    (gcc/Kbuild ordering).
    add x23, x23, #1
    b .Lca_scan
.Lca_scan_wall:
    // "-Wall": enable the warning family; not a
   source
    adrp x0, _kcc_warn_all@PAGE
    add  x0, x0, _kcc_warn_all@PAGEOFF
    mov  x1, #1
    str  x1, [x0]
    add x23, x23, #1
    b .Lca_scan
.Lca_scan_g:
    // "-g": emit DWARF debug info; not a source
    adrp x0, _kcc_dwarf_enabled@PAGE
    add  x0, x0, _kcc_dwarf_enabled@PAGEOFF
    mov  x1, #1
    str  x1, [x0]
    add x23, x23, #1
    b .Lca_scan
.Lca_scan_emitasm:
    // "--emit-asm": produce the object through
   the `.s` + `as` back end
    adrp x0, _kcc_emit_asm@PAGE
    add  x0, x0, _kcc_emit_asm@PAGEOFF
    mov  x1, #1
    str  x1, [x0]
    add x23, x23, #1
    b .Lca_scan
.Lca_scan_target:
    // "--target=x86_64": select the x86-64 back
    end; not a source
    adrp x0, _kcc_target@PAGE
    add  x0, x0, _kcc_target@PAGEOFF
    mov  x1, #1
    str  x1, [x0]
    add x23, x23, #1
    b .Lca_scan
.Lca_scan_target_linux:
    // "--target=x86_64-linux": x86-64 back end in
    its Linux/ELF flavour.
    adrp x0, _kcc_target@PAGE
    add  x0, x0, _kcc_target@PAGEOFF
    mov  x1, #1
    str  x1, [x0]
    adrp x0, _kcc_x86_os@PAGE
    add  x0, x0, _kcc_x86_os@PAGEOFF
    mov  x1, #1
    str  x1, [x0]
    add x23, x23, #1
    b .Lca_scan
.Lca_scan_o:
    add x23, x23, #1                // step to the
   -o value
    cmp x23, x19
    b.ge .Lca_scan_done
    ldr x21, [x20, x23, lsl #3]     // explicit out
    add x23, x23, #1
    b .Lca_scan
.Lca_scan_done:
    cbz x22, .Lca_no_input

    // Pass 2: compile each source.  Scan from
   argv[1] and skip `-c` wherever it
    // sits, mirroring pass 1.
    mov x24, #0                     // error
   accumulator
    mov x23, #1
.Lca_loop:
    cmp x23, x19
    b.ge .Lca_finish
    ldr x25, [x20, x23, lsl #3]     // argv[i]
    mov x0, x25
    adrp x1, _kcc_opt_compile@PAGE
    add  x1, x1, _kcc_opt_compile@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_skip_wall          // "-c": the
   compile-only selector, skip
    mov x0, x25
    adrp x1, _kcc_opt_wall@PAGE
    add  x1, x1, _kcc_opt_wall@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_skip_wall
    mov x0, x25
    adrp x1, _kcc_opt_g@PAGE
    add  x1, x1, _kcc_opt_g@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_skip_wall          // -g: same as
   -Wall, skip (already handled)
    mov x0, x25
    adrp x1, _kcc_opt_emitasm@PAGE
    add  x1, x1, _kcc_opt_emitasm@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_skip_wall          // --emit-asm:
   handled in pass 1
    mov x0, x25
    adrp x1, _kcc_opt_target_x86@PAGE
    add  x1, x1, _kcc_opt_target_x86@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_skip_wall          // --target=
   x86_64: handled in pass 1
    mov x0, x25
    adrp x1, _kcc_opt_target_linux@PAGE
    add  x1, x1, _kcc_opt_target_linux@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_skip_wall          // --target=
```

```
    x86_64-linux: handled in pass 1
    mov x0, x25
    adrp x1, _kcc_opt_o@PAGE
    add  x1, x1, _kcc_opt_o@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lca_skip_o
    // -Idir / -I dir: skip (registered in pass 1)
    mov x0, x25
    mov x1, #0
    mov x2, #0                          // do not
    register again
    bl _kcc_driver_I
    cbz x0, .Lca_skip_D
    add x23, x23, x0
    b .Lca_loop
.Lca_skip_D:
    // -DNAME / -D NAME: skip (recorded in pass 1)
    mov x0, x25
    mov x1, #0
    mov x2, #0
    bl _kcc_driver_D
    cbz x0, .Lca_loop_m
    add x23, x23, x0
    b .Lca_loop
.Lca_loop_m:
    // code-model / red-zone / tuning flag:
    honoured in pass 1, skip here
    mov x0, x25
    bl _kcc_driver_skipopt
    cbz x0, .Lca_compile_src
    add x23, x23, #1
    b .Lca_loop
.Lca_compile_src:

    // Output name: explicit only when there is a
    single source
    cmp x22, #1
    b.ne .Lca_derive
    cbz x21, .Lca_derive
    mov x0, x21
    b .Lca_have_out
.Lca_derive:
    mov x0, x25
    bl _kcc_driver_obj_name
.Lca_have_out:
    mov x1, x0                          // out
    mov x0, x25                         // src
    bl _kcc_driver_compile
    orr x24, x24, x0                    // remember any
    failure
    add x23, x23, #1
    b .Lca_loop
.Lca_skip_o:
    add x23, x23, #2                    // skip "-o"
    and its value
    b .Lca_loop
.Lca_skip_wall:
    add x23, x23, #1                    // skip "-Wall"
    (already handled in pass 1)
    b .Lca_loop

.Lca_finish:
    bl _kcc_io_flush
    mov x0, x24
    bl _kcc_sys_exit

.Lca_no_input:
    adrp x0, _kcc_driver_err_noinput@PAGE
    add  x0, x0, _kcc_driver_err_noinput@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit
```

□

609 · defined at 603, 606ab, 607, 608ab, 609 · used in 31f · defines _kcc_driver_compile_all 624, _kcc_driver_obj_name 611 · uses _kcc_arena_alloc 58a, _kcc_driver_compile 603, _kcc_driver_D 606b, _kcc_driver_I 606a, _kcc_driver_skipopt 608b, _kcc_io_err 71b, _kcc_io_flush 69b, _kcc_target 568b, _kcc_warn_all 77a

43.3. Automatic Linking

Invoked without `-c`, kcc behaves like a normal compiler driver: it compiles every source argument to a temporary object file and then links those objects into an executable, so `kcc hello.c -o hello` produces a runnable program in one step. Linking is delegated to the system linker `/usr/bin/ld`, run as a child process (Chapter 7 supplies `fork`/`execve`/`wait4`). Because kcc-compiled programs are freestanding—their entry point is `_main`, with no C runtime start-up—the link line is `ld -o hello hello.o -e _main`. The architecture follows the selected target: `arm64` by default, or `x86_64` under `--target=x86_64` (link mode's pre-scan sets `_kcc_target` exactly as the compile-mode scan does, and each source is then compiled through the same target dispatch, so `kcc --target=x86_64 hello.c -o hello` both compiles and links for x86-64 in one step). The SDK root needed by `-syslibroot` is found from `$SDKROOT` or, failing that, the two standard SDK locations. After a successful link the intermediate object files are removed.

Not every non-option argument is a source. As the driver scans the command line it classifies each one: a name ending in `.c` is compiled to a temporary object; a leading `-` is an unrecognised linker flag (`-lm`, `-Lpath`, `-framework`) forwarded verbatim so `ld` sees it in order; and anything else—most importantly a pre-compiled `.o` or a `.a` archive—is handed straight to the linker. This is what lets `kcc main.c libfoo.o -o app` mix sources and objects in one invocation, rather than trying to *compile* the object and choking on its binary contents. Only the objects kcc compiled itself are tracked in a temp-object list and removed afterwards; a `.o` supplied by the user is left untouched.

Because this path compiles each source inline as it scans the arguments, it first makes a preliminary pass to honour `-Wall` and `-g` wherever they appear, so a warning or debug-info request applies to every source regardless of its position on the command line; the compile pass then skips those flags rather than mistaking them for source files.

⟨ *Driver: link* 611 ⟩ ≡

```
// -- kcc_getenv ---
// char *kcc_getenv(const char *name, uint64_t
   namelen)
//   Returns the value (text after '=') of
   environment variable name, or 0.
.text
.globl _kcc_getenv
.p2align 2
_kcc_getenv:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
```

```
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    mov x19, x0                     // name
    mov x20, x1                     // namelen
    adrp x0, _kcc_envp@PAGE
    add  x0, x0, _kcc_envp@PAGEOFF
    ldr x21, [x0]                   // envp
    cbz x21, .Lge_none
.Lge_loop:
    ldr x22, [x21]                  // entry
    cbz x22, .Lge_none
    mov x0, x22
    mov x1, x19
    mov x2, x20
    bl _kcc_memcmp
    cbnz x0, .Lge_next
    ldrb w0, [x22, x20]             // entry[namelen]
    == '=' ?
    cmp w0, #'='
    b.ne .Lge_next
    add x0, x22, x20
    add x0, x0, #1                  // value = entry
   + namelen + 1
    b .Lge_ret
.Lge_next:
    add x21, x21, #8
    b .Lge_loop
.Lge_none:
    mov x0, #0
.Lge_ret:
    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_dir_exists ---
// int kcc_dir_exists(const char *path) - 1 if it
   can be opened, else 0.
.text
.globl _kcc_dir_exists
.p2align 2
_kcc_dir_exists:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    mov x1, #0                      // O_RDONLY
    mov x2, #0
    bl _kcc_sys_open
    cmp x0, #0
    b.lt .Lde_no
    bl _kcc_sys_close
    mov x0, #1
    b .Lde_ret
.Lde_no:
    mov x0, #0
.Lde_ret:
    ldp x29, x30, [sp], #16
    ret

// -- kcc_driver_sdkroot ---
// char *kcc_driver_sdkroot(void) - SDK path for `
   ld -syslibroot`.
.text
.globl _kcc_driver_sdkroot
.p2align 2
_kcc_driver_sdkroot:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]
    // 1. $SDKROOT if set and openable
    adrp x0, _kcc_str_sdkroot_env@PAGE
    add  x0, x0, _kcc_str_sdkroot_env@PAGEOFF
    mov x1, #7
    bl _kcc_getenv
    cbz x0, .Lsr_clt
    mov x19, x0
    bl _kcc_dir_exists
    cbnz x0, .Lsr_use
    // 2. Command Line Tools SDK
.Lsr_clt:
    adrp x19, _kcc_str_sdk_clt@PAGE
    add  x19, x19, _kcc_str_sdk_clt@PAGEOFF
    mov x0, x19
    bl _kcc_dir_exists
    cbnz x0, .Lsr_use
    // 3. Xcode SDK
    adrp x19, _kcc_str_sdk_xcode@PAGE
    add  x19, x19, _kcc_str_sdk_xcode@PAGEOFF
    mov x0, x19
    bl _kcc_dir_exists
    cbnz x0, .Lsr_use
    // 4. fall back to the CLT path
   unconditionally
    adrp x19, _kcc_str_sdk_clt@PAGE
    add  x19, x19, _kcc_str_sdk_clt@PAGEOFF
.Lsr_use:
    mov x0, x19
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_driver_spawn ---
// int kcc_driver_spawn(const char *path, char *
   const argv[])
//   fork+execve+wait4; returns the child's exit
   status (or -1 on fork error).
.text
.globl _kcc_driver_spawn
.p2align 2
_kcc_driver_spawn:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    str x21, [sp, #32]
    mov x19, x0                      // path
    mov x20, x1                      // argv
    bl _kcc_sys_fork
    cbz x0, .Lsp_child
    cmp x0, #0
    b.lt .Lsp_err
    mov x21, x0                      // child pid
    add x1, sp, #40                 // &status
    mov x0, x21
    mov x2, #0
    mov x3, #0
    bl _kcc_sys_wait4
    ldr w0, [sp, #40]               // status
    lsr w0, w0, #8                  // WEXITSTATUS
    and w0, w0, #0xff
    b .Lsp_ret
.Lsp_child:
    mov x0, x19                      // path
    mov x1, x20                      // argv
    adrp x2, _kcc_envp@PAGE
    add  x2, x2, _kcc_envp@PAGEOFF
    ldr x2, [x2]                     // envp
    bl _kcc_sys_execve
    mov x0, #127                     // exec failed
    bl _kcc_sys_exit
.Lsp_err:
    mov x0, #-1
.Lsp_ret:
    ldr x21, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret

// -- kcc_driver_link_all ---
// void kcc_driver_link_all(int argc, char **argv)
//   Compile every source argument to a .o, link
```

```
   them into an executable
//   (output from "-o" or "a.out"), remove the
   objects, and exit with the
//   linker's status.
.text
.globl _kcc_driver_link_all
.p2align 2
_kcc_driver_link_all:
    stp x29, x30, [sp, #-112]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]
    stp x23, x24, [sp, #48]
    stp x25, x26, [sp, #64]
    str x27, [sp, #80]
    mov x19, x0                  // argc
    mov x20, x1                  // argv
    // [sp,#88] = temp-object list (compiled .c →
   .o names to unlink after the
    // link), [sp,#96] = its count.  Only these
   are removed; a .o handed to us
    // on the command line is the user's and must
   survive.

    // Pass 1: honour -Wall and -g wherever they
   appear on the command line,
    // so every source is compiled with them
   active regardless of position
    // (the compile-and-link loop below processes
   sources inline, in one pass).
    mov x21, #1
.Lla_pre:
    cmp x21, x19
    b.ge .Lla_pre_done
    ldr x0, [x20, x21, lsl #3]
    adrp x1, _kcc_opt_wall@PAGE
    add  x1, x1, _kcc_opt_wall@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lla_pre_wall
    ldr x0, [x20, x21, lsl #3]
    adrp x1, _kcc_opt_emitasm@PAGE
    add  x1, x1, _kcc_opt_emitasm@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lla_pre_emitasm
    ldr x0, [x20, x21, lsl #3]
    adrp x1, _kcc_opt_target_x86@PAGE
    add  x1, x1, _kcc_opt_target_x86@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lla_pre_target
    ldr x0, [x20, x21, lsl #3]
    adrp x1, _kcc_opt_g@PAGE
    add  x1, x1, _kcc_opt_g@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lla_pre_g
    // -Idir / -I dir: register the include
   directory
    ldr x0, [x20, x21, lsl #3]
    add x1, x21, #1
    cmp x1, x19
    b.ge .Lla_pre_I_nonext
    ldr x1, [x20, x1, lsl #3]
    b .Lla_pre_I_call
.Lla_pre_I_nonext:
    mov x1, #0
.Lla_pre_I_call:
    mov x2, #1                   // register the
   directory
    bl _kcc_driver_I
    cbz x0, .Lla_pre_D
    add x21, x21, x0
    b .Lla_pre
.Lla_pre_D:
    // -DNAME[=VALUE] / -D NAME: record a command-
   line macro
    ldr x0, [x20, x21, lsl #3]
    add x1, x21, #1
    cmp x1, x19
    b.ge .Lla_pre_D_nonext
    ldr x1, [x20, x1, lsl #3]
    b .Lla_pre_D_call
.Lla_pre_D_nonext:
    mov x1, #0
.Lla_pre_D_call:
    mov x2, #1
    bl _kcc_driver_D
    cbz x0, .Lla_pre_m
    add x21, x21, x0
    b .Lla_pre
.Lla_pre_m:
    // code-model / red-zone / tuning flag:
   honoured as a no-op
    ldr x0, [x20, x21, lsl #3]
    bl _kcc_driver_skipopt
    cbz x0, .Lla_pre_next
    add x21, x21, #1
    b .Lla_pre
.Lla_pre_next:
    add x21, x21, #1
    b .Lla_pre
.Lla_pre_wall:
    adrp x0, _kcc_warn_all@PAGE
    add  x0, x0, _kcc_warn_all@PAGEOFF
    mov  x1, #1
    str  x1, [x0]
    add x21, x21, #1
    b .Lla_pre
.Lla_pre_g:
    adrp x0, _kcc_dwarf_enabled@PAGE
    add  x0, x0, _kcc_dwarf_enabled@PAGEOFF
    mov  x1, #1
    str  x1, [x0]
    add x21, x21, #1
    b .Lla_pre
.Lla_pre_emitasm:
    adrp x0, _kcc_emit_asm@PAGE
    add  x0, x0, _kcc_emit_asm@PAGEOFF
    mov  x1, #1
    str  x1, [x0]
    add x21, x21, #1
    b .Lla_pre
.Lla_pre_target:
    // "--target=x86_64": select the x86-64 back
   end for both compilation and
    // the final link (which then passes `-arch
   x86_64` to ld).
    adrp x0, _kcc_target@PAGE
    add  x0, x0, _kcc_target@PAGEOFF
    mov  x1, #1
    str  x1, [x0]
    add x21, x21, #1
    b .Lla_pre
.Lla_pre_done:

    // Linker argv: at most argc + 12 pointers.
    add x0, x19, #12
    lsl x0, x0, #3
    bl _kcc_arena_alloc
    mov x21, x0                  // larg

    // Temp-object list: at most one per argument.
    add x0, x19, #1
    lsl x0, x0, #3
    bl _kcc_arena_alloc
    str x0, [sp, #88]            // temp-object
   array
    str xzr, [sp, #96]           // temp-object
   count

    adrp x0, _kcc_str_ld_argv0@PAGE
    add  x0, x0, _kcc_str_ld_argv0@PAGEOFF
```

```
    str x0, [x21]                   // larg[0] = "ld"
    mov x22, #1                     // k (next larg
   slot)
    mov x23, #0                     // output (0 =
   default)
    mov x24, #1                     // scan index (
   from argv[1])
    mov x25, #0                     // compile-error
   accumulator
    mov x26, #0                     // source count

.Lla_loop:
    cmp x24, x19
    b.ge .Lla_link
    ldr x27, [x20, x24, lsl #3]  // argv[i]
    mov x0, x27
    adrp x1, _kcc_opt_o@PAGE
    add  x1, x1, _kcc_opt_o@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lla_o
    // -Wall / -g were consumed in pass 1; skip
   them here (not source files).
    mov x0, x27
    adrp x1, _kcc_opt_wall@PAGE
    add  x1, x1, _kcc_opt_wall@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lla_skipopt
    mov x0, x27
    adrp x1, _kcc_opt_g@PAGE
    add  x1, x1, _kcc_opt_g@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lla_skipopt
    mov x0, x27
    adrp x1, _kcc_opt_emitasm@PAGE
    add  x1, x1, _kcc_opt_emitasm@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lla_skipopt            // --emit-asm:
   handled in pass 1
    mov x0, x27
    adrp x1, _kcc_opt_target_x86@PAGE
    add  x1, x1, _kcc_opt_target_x86@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lla_skipopt            // --target=
   x86_64: handled in pass 1
    // -Idir / -I dir: skip (registered in pass 1)
    mov x0, x27
    mov x1, #0
    mov x2, #0                      // do not
   register again
    bl _kcc_driver_I
    cbz x0, .Lla_skip_D
    add x24, x24, x0
    b .Lla_loop
.Lla_skip_D:
    // -DNAME / -D NAME: skip (recorded in pass 1)
    mov x0, x27
    mov x1, #0
    mov x2, #0
    bl _kcc_driver_D
    cbz x0, .Lla_m
    add x24, x24, x0
    b .Lla_loop
.Lla_m:
    // code-model / red-zone / tuning flag:
   consumed so it is not forwarded to ld
    mov x0, x27
    bl _kcc_driver_skipopt
    cbz x0, .Lla_src
    add x24, x24, #1
    b .Lla_loop
.Lla_src:
    // Classify a non-option argument.  A leading
   '-' is an unrecognised linker
    // flag (e.g. -lm, -Lpath, -framework):
   forward it verbatim so ld sees it in
    // order.  A ".c" file is compiled to a
   temporary object.  Anything else
    // (".o", ".a", or another file) is handed
   straight to the linker.
    ldrb w0, [x27]
    cmp w0, #'-'
    b.eq .Lla_passthru
    mov x0, x27
    bl _kcc_strlen
    cmp x0, #2
    b.lt .Lla_object                // too short for
   "x.c" → treat as a linker input
    add x1, x27, x0
    ldrh w2, [x1, #-2]              // last two bytes
   , little-endian: 'c'<<8 | '.'
    mov w3, #0x632E                 // ".c"
    cmp w2, w3
    b.ne .Lla_object

    // A ".c" source: compile it to a temporary
   object and record that object
    // both for the linker (larg) and for later
   removal (temp-object list).
    mov x0, x27
    bl _kcc_driver_obj_name
    str x0, [x21, x22, lsl #3]    // larg[k] = obj
   name
    ldr x1, [sp, #88]             // temp-object
   array
    ldr x2, [sp, #96]             // temp-object
   count
    str x0, [x1, x2, lsl #3]
    add x2, x2, #1
    str x2, [sp, #96]
    mov x1, x0                     // out
    mov x0, x27                    // src
    bl _kcc_driver_compile
    orr x25, x25, x0
    add x22, x22, #1
    add x26, x26, #1               // a file input
    add x24, x24, #1
    b .Lla_loop
.Lla_object:
    // A pre-compiled object / archive / other
   file: pass to the linker as-is.
    str x27, [x21, x22, lsl #3]
    add x22, x22, #1
    add x26, x26, #1               // a file input
    add x24, x24, #1
    b .Lla_loop
.Lla_passthru:
    // An unrecognised linker flag: forward it
   without counting it as an input.
    str x27, [x21, x22, lsl #3]
    add x22, x22, #1
    add x24, x24, #1
    b .Lla_loop
.Lla_skipopt:
    add x24, x24, #1
    b .Lla_loop
.Lla_o:
    add x24, x24, #1
    cmp x24, x19
    b.ge .Lla_link
    ldr x23, [x20, x24, lsl #3]    // output name
    add x24, x24, #1
    b .Lla_loop

.Lla_link:
    cbnz x25, .Lla_fail            // a compile
   failed → do not link
    cbz x26, .Lla_noinput

    cbnz x23, .Lla_out_ok
    adrp x23, _kcc_str_aout@PAGE
```

```
    add  x23, x23, _kcc_str_aout@PAGEOFF
.Lla_out_ok:
    // Append: -o <out> -lSystem -syslibroot <sdk>
    -arch arm64 -e _main NULL
    adrp x0, _kcc_opt_o@PAGE
    add  x0, x0, _kcc_opt_o@PAGEOFF
    str x0, [x21, x22, lsl #3]
    add x22, x22, #1
    str x23, [x21, x22, lsl #3]
    add x22, x22, #1
    adrp x0, _kcc_str_lsystem@PAGE
    add  x0, x0, _kcc_str_lsystem@PAGEOFF
    str x0, [x21, x22, lsl #3]
    add x22, x22, #1
    adrp x0, _kcc_str_syslibroot@PAGE
    add  x0, x0, _kcc_str_syslibroot@PAGEOFF
    str x0, [x21, x22, lsl #3]
    add x22, x22, #1
    bl _kcc_driver_sdkroot
    str x0, [x21, x22, lsl #3]
    add x22, x22, #1
    adrp x0, _kcc_str_arch@PAGE
    add  x0, x0, _kcc_str_arch@PAGEOFF
    str x0, [x21, x22, lsl #3]
    add x22, x22, #1
    // Architecture: "x86_64" under --target=
    x86_64, else the native "arm64".
    adrp x0, _kcc_target@PAGE
    add  x0, x0, _kcc_target@PAGEOFF
    ldr x0, [x0]
    cbz x0, .Lla_arch_arm64
    adrp x0, _kcc_str_x86_64@PAGE
    add  x0, x0, _kcc_str_x86_64@PAGEOFF
    b .Lla_arch_set
.Lla_arch_arm64:
    adrp x0, _kcc_str_arm64@PAGE
    add  x0, x0, _kcc_str_arm64@PAGEOFF
.Lla_arch_set:
    str x0, [x21, x22, lsl #3]
    add x22, x22, #1
    adrp x0, _kcc_str_dash_e@PAGE
    add  x0, x0, _kcc_str_dash_e@PAGEOFF
    str x0, [x21, x22, lsl #3]
    add x22, x22, #1
    adrp x0, _kcc_str_umain@PAGE
    add  x0, x0, _kcc_str_umain@PAGEOFF
    str x0, [x21, x22, lsl #3]
    add x22, x22, #1
    str xzr, [x21, x22, lsl #3]    // NULL
    terminator

    bl _kcc_io_flush               // flush before
    fork

    adrp x0, _kcc_str_ld_path@PAGE
    add  x0, x0, _kcc_str_ld_path@PAGEOFF
    mov x1, x21
    bl _kcc_driver_spawn
    mov x25, x0                     // link status

    // Remove only the objects we compiled
    ourselves (the temp-object list);
    // a .o passed in on the command line belongs
    to the user and is left alone.
    ldr x21, [sp, #88]           // temp-object
    array
    ldr x26, [sp, #96]           // temp-object
    count
    mov x24, #0
.Lla_unlink:
    cmp x24, x26
    b.ge .Lla_done
    ldr x0, [x21, x24, lsl #3]
    bl _kcc_sys_unlink
    add x24, x24, #1
    b .Lla_unlink
.Lla_done:
    bl _kcc_io_flush
    mov x0, x25
    bl _kcc_sys_exit

.Lla_fail:
    bl _kcc_io_flush
    mov x0, #1
    bl _kcc_sys_exit

.Lla_noinput:
    adrp x0, _kcc_driver_err_noinput@PAGE
    add  x0, x0, _kcc_driver_err_noinput@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit
```

□

611 · used in 31f · defines _kcc_dir_exists never used, _kcc_driver_link_all 621, _kcc_driver_sdkroot never used, _kcc_driver_spawn 603, _kcc_getenv 603 · uses _kcc_arena_alloc 58a, _kcc_driver_compile 603, _kcc_driver_D 606b, _kcc_driver_I 606a, _kcc_driver_obj_name 609, _kcc_driver_skipopt 608b, _kcc_envp 615, _kcc_io_err 71b, _kcc_io_flush 69b, _kcc_target 568b, _kcc_warn_all 77a, _main 621, 629

⟨ *Driver: data* 615 ⟩ ≡

```
// -- Driver string constants ---
.section __TEXT,__cstring,cstring_literals
.globl _kcc_opt_compile
_kcc_opt_compile:
    .asciz "-c"
_kcc_opt_o:
    .asciz "-o"
_kcc_opt_wall:
    .asciz "-Wall"
_kcc_opt_wsuppress:
    .asciz "-w"
_kcc_opt_g:
    .asciz "-g"
_kcc_opt_incdir:
    .asciz "-I"
_kcc_opt_defmac:
    .asciz "-D"
_kcc_opt_emitasm:
    .asciz "--emit-asm"
_kcc_opt_target_x86:
    .asciz "--target=x86_64"
_kcc_opt_target_linux:
    .asciz "--target=x86_64-linux"
_kcc_opt_mcmodel:
    .asciz "-mcmodel="
_kcc_opt_mnoredzone:
    .asciz "-mno-red-zone"
_kcc_opt_mredzone:
    .asciz "-mred-zone"
_kcc_str_cm_small:
    .asciz "small"
_kcc_str_cm_kernel:
    .asciz "kernel"
_kcc_driver_err_cmodel:
    .asciz "kcc: error: unsupported -mcmodel (only
    'small' and 'kernel')\n"
_kcc_xf_dO:
    .asciz "-O"
_kcc_xf_df:
    .asciz "-f"
_kcc_xf_dW:
    .asciz "-W"
_kcc_xf_dm:
    .asciz "-m"
_kcc_xf_std:
```

```
    .asciz "-std="
_kcc_xf_pipe:
    .asciz "-pipe"
_kcc_xf_nostdinc:
    .asciz "-nostdinc"
_kcc_str_x86_64:
    .asciz "x86_64"
_kcc_str_as_argv0:
    .asciz "as"
_kcc_str_as_path:
    .asciz "/usr/bin/as"
_kcc_str_dot_s:
    .asciz ".s"
_kcc_driver_err_noinput:
    .asciz "kcc: error: no input files\n"
_kcc_str_ld_argv0:
    .asciz "ld"
_kcc_str_ld_path:
    .asciz "/usr/bin/ld"
_kcc_str_lsystem:
    .asciz "-lSystem"
_kcc_str_syslibroot:
    .asciz "-syslibroot"
_kcc_str_arch:
    .asciz "-arch"
_kcc_str_arm64:
    .asciz "arm64"
_kcc_str_dash_e:
    .asciz "-e"
_kcc_str_umain:
    .asciz "_main"
_kcc_str_aout:
    .asciz "a.out"
_kcc_str_sdkroot_env:
    .asciz "SDKROOT"
_kcc_str_sdk_clt:
    .asciz "/Library/Developer/CommandLineTools/
   SDKs/MacOSX.sdk"
_kcc_str_sdk_xcode:
    .asciz "/Applications/Xcode.app/Contents/
   Developer/Platforms/MacOSX.platform/Developer/
   SDKs/MacOSX.sdk"

.data
.p2align 3
.globl _kcc_envp
_kcc_envp:
    .quad 0
.globl _kcc_emit_asm
_kcc_emit_asm:                  // 1 = produce the
   object via `.s` + `as` (--emit-asm)
    .quad 0
```

□

615 · used in 31f · defines _kcc_envp 611, 621 · uses _main 621, 629

43.4. Multi-file Tests

These tests split a program across two translation units that share declarations through a prototype: one file defines the functions, the other calls them. A single `kcc -c a.c b.c` compiles both, and the system linker resolves the cross-object references—the end-to-end shape a self-hosting build takes.

⟨ *Test: codegen multifile* 616a ⟩ ≡

```
echo ""
echo "--- Multi-file Compilation ---"

run_test_multi "multi-two-units" \
    "int helper(int x){ return x * 10; }" \
    "int helper(int x); int main(void){ return
   helper(4) + 2; }" 42

run_test_multi "multi-extern-global" \
    "int total; void add(int n){ total = total + n
   ; }" \
    "extern int total; void add(int n); int main(
   void){ add(5); add(7); return total; }" 12

run_test_multi "multi-mutual-call" \
    "int dbl(int x); int quad(int x){ return dbl(
   dbl(x)); }" \
    "int quad(int x); int dbl(int x){ return x + x
   ; } int main(void){ return quad(6); }" 24
```

□

616a · used in 33b

43.5. End-to-End Driver Tests

Automatic linking (§43.3) lets kcc produce runnable executables directly. These tests drive the whole toolchain the way a user would—`kcc src.c -o out` then run `out`—covering a single file, the default `a.out` name, a multi-file link, a call into `libSystem` (`printf`), and the propagation of a linker error.

⟨ *E2E: runner* 616b ⟩ ≡

```
#!/bin/bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# kcc end-to-end driver tests (compile + auto-link
    + run) - generated by weft
set -u
PASS=0
FAIL=0
KCC="$BASE/kcc"
TMP="/$BASE/kcc_e2e_$$"
mkdir -p "$TMP"
trap 'rm -rf "$TMP"' EXIT

# e2e name expected src...   (last-but-one is
   expected exit, rest are sources)
run_e2e() {
    local name="$1"; shift
    local expected="$1"; shift
    local srcs=() i=0
    for s in "$@"; do printf '%s\n' "$s" > "$TMP/
   f$i.c"; srcs+=("$TMP/f$i.c"); i=$((i+1)); done
    "$KCC" "${srcs[@]}" -o "$TMP/prog" >/dev/null
   2>&1
    if [ ! -x "$TMP/prog" ]; then
        printf "  %-24s FAIL (no executable)\n" "
   $name"; FAIL=$((FAIL+1)); return
    fi
    "$TMP/prog" >/dev/null 2>&1
    local rc=$?
    rm -f "$TMP/prog"
    if [ "$rc" = "$expected" ]; then
        printf "  %-24s ok (%s)\n" "$name" "$rc";
   PASS=$((PASS+1))
    else
        printf "  %-24s FAIL got=%s exp=%s\n" "
   $name" "$rc" "$expected"; FAIL=$((FAIL+1))
    fi
}
```

```
echo "=== end-to-end driver (compile + link + run)
    ==="
run_e2e "single-file" 42 "int main(void){ return
   42; }"
run_e2e "arithmetic"  30 "int sq(int x){ return x*
   x; } int main(void){ return sq(5)+sq(2)+1; }"
run_e2e "two-units"   42 "int add(int a,int b){
   return a+b; }" \
                         "int add(int,int); int
   main(void){ return add(20,22); }"
run_e2e "libsystem-printf" 7 "int printf(const
   char*,...); int main(void){ printf(\"kcc e2e ok
   \n\"); return 7; }"
run_e2e "header-stdlib-string" 5 "#include <stdlib
   .h>
#include <string.h>
int main(void){ char *p = malloc(16); strcpy(p, \"
   hello\"); int n = strlen(p); free(p); return n;
    }"
run_e2e "header-math" 7 "#include <math.h>
int main(void){ return (int) sqrt(49.0); }"
run_e2e "header-vararg-sprintf" 23 "#include <
   stdio.h>
#include <stdlib.h>
int main(void){ char buf[16]; sprintf(buf, \"%d\",
    123); return atoi(buf) - 100; }"
run_e2e "header-std-streams" 9 "#include <stdio.h>
int main(void){ fprintf(stdout, \"out %d\n\", 1);
   fprintf(stderr, \"err %d\n\", 2); return 9; }"

# A missing symbol must make the driver fail (no
   executable, nonzero status).
printf 'int gone(void); int main(void){ return
   gone(); }\n' > "$TMP/bad.c"
$KCC "$TMP/bad.c" -o "$TMP/bad" >/dev/null 2>&1
if [ $? -ne 0 ] && [ ! -x "$TMP/bad" ]; then
    printf "  %-24s ok (rejected)\n" "link-error";
    PASS=$((PASS+1))
else
    printf "  %-24s FAIL (should reject)\n" "link-
   error"; FAIL=$((FAIL+1))
fi

echo ""
echo "=== e2e: $PASS ok, $FAIL fail ==="
[ "$FAIL" -eq 0 ]
```

□

616b · used in 37m

Part IX.

The Driver

The driver is the user-facing entry point of the compiler. It parses the command-line arguments, selects the requested mode of operation — `--dump-tokens` for lexer output, `-E` for preprocessor output, `--dump-ast` for the parse tree, `--dump-types` for type-annotated output, `--check` for semantic-only validation, `--dump-ir` for the intermediate representation, or `-c` for full compilation to an object file — and dispatches to the appropriate pipeline stage. Because the compiler has no C runtime, `_main` is the true process entry point: it receives `argc` and `argv` directly from the kernel's stack frame.

44. The Driver

This chapter is where execution begins. The `_main` function receives `argc` and `argv` from the macOS kernel, initializes the compiler's core subsystems (arena allocator, keyword table, I/O), and dispatches to the appropriate pipeline based on the command-line flags. Supported modes include `--dump-tokens` for lexer debugging, `--dump-ast` for parser output, `--dump-ir` for IR inspection, and `-c` for full compilation to a Mach-O object file. The diagnostic modes are selected by `argv[1]`, but `-c` is recognised *anywhere* in the argument list: a gcc-driven build (Kbuild) leads with its flags and ends with `-c -o out.o in.c`, so a fixed `argv[1]` test would miss the `-c` and fall through to link mode.

44.1. Entry Point

On macOS, the kernel passes control to `_main` with:

- `x0` = `argc` (argument count)
- `x1` = `argv` (pointer to argument vector)

We save these immediately in callee-saved registers, then initialize the arena and I/O subsystems before doing any real work.

⟨ *Main: entry point* 621 ⟩ ≡

```
// -- _main ---
// Entry point for kcc.
// x0 = argc, x1 = argv
.text
.globl _main
.p2align 2
_main:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Save argc and argv
    mov x19, x0                     // x19 = argc
    mov x20, x1                     // x20 = argv

    // Capture envp (= argv + (argc+1)*8, past the
    NULL that ends argv) so the
    // driver can pass the environment on to the
   linker via execve.
    add x2, x19, #1
    add x2, x20, x2, lsl #3
    adrp x3, _kcc_envp@PAGE
    add  x3, x3, _kcc_envp@PAGEOFF
    str x2, [x3]

    // Initialize subsystems
    bl _kcc_arena_init
    bl _kcc_lex_kw_init

    // Register the compiler's default include
   directory (dirname(argv[0]) +
    // "include") once, before any file is
   processed and outliving the per-file
    // preprocessor state, so the shipped hosted
   headers resolve with no -I.
    // Uses the arena, so it must follow arena
   initialisation.
    ldr x0, [x20]                   // argv[0] =
   program path
    bl _kcc_pp_add_default_includes

    // Print banner
    adrp x0, _kcc_banner@PAGE
    add  x0, x0, _kcc_banner@PAGEOFF
    bl _kcc_io_puts

    // Check for arguments
    cmp x19, #2
    b.lt .Lmain_no_args

    // Check if argv[1] is "--dump-tokens"
    ldr x0, [x20, #8]               // argv[1]
    adrp x1, _kcc_opt_dump_tokens@PAGE
    add  x1, x1, _kcc_opt_dump_tokens@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lmain_dump_tokens

    // Check if argv[1] is "-E" (preprocess only)
    ldr x0, [x20, #8]               // argv[1]
    adrp x1, _kcc_opt_preprocess@PAGE
    add  x1, x1, _kcc_opt_preprocess@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lmain_preprocess

    // Check if argv[1] is "--dump-ast"
    ldr x0, [x20, #8]               // argv[1]
    adrp x1, _kcc_opt_dump_ast@PAGE
    add  x1, x1, _kcc_opt_dump_ast@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lmain_dump_ast

    // Check if argv[1] is "--dump-types"
    ldr x0, [x20, #8]               // argv[1]
    adrp x1, _kcc_opt_dump_types@PAGE
    add  x1, x1, _kcc_opt_dump_types@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lmain_dump_types

    // Check if argv[1] is "--check"
    ldr x0, [x20, #8]               // argv[1]
    adrp x1, _kcc_opt_check@PAGE
    add  x1, x1, _kcc_opt_check@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lmain_check

    // Check if argv[1] is "--dump-ir"
    ldr x0, [x20, #8]               // argv[1]
    adrp x1, _kcc_opt_dump_ir@PAGE
    add  x1, x1, _kcc_opt_dump_ir@PAGEOFF
```

```
    bl _kcc_strcmp
    cbz x0, .Lmain_dump_ir

    // Compile-only mode: "-c" anywhere in the
   argument list, not just as the
    // first argument.  gcc-driven builds (Kbuild)
    place the compiler flags
    // first and end with `-c -o out.o in.c`, so a
    fixed argv[1] test would miss
    // the `-c` and fall through to link mode---
   feeding the object flags to ld.
    mov x21, #1                     // i = 1
.Lmain_scan_c:
    cmp x21, x19
    b.ge .Lmain_scan_c_done
    ldr x0, [x20, x21, lsl #3]    // argv[i]
    adrp x1, _kcc_opt_compile@PAGE
    add  x1, x1, _kcc_opt_compile@PAGEOFF
    bl _kcc_strcmp
    cbz x0, .Lmain_compile
    add x21, x21, #1
    b .Lmain_scan_c
.Lmain_scan_c_done:

    // Not a flag: the argument is a source file.
    Compile every source and
    // link the resulting objects into an
   executable (the default mode).
    mov x0, x19
    mov x1, x20
    bl _kcc_driver_link_all
    b .Lmain_exit
```

□

621 · defined at 621, 622, 623, 624 · used in 27a · defines _main 38k, 611, 615, 629, 637, 640b, 644, 646b, 648b, 651, 653b, 656c, 659b, 662c, 665, 668c, 673, 681c, 684b, 686c, 694c, 702b, 710b, 717b, 719, 725b, 738b, 742b, 751, 768b, 792b, 818b, 836b, 902b, 909, 914, 917 · uses _kcc_arena_init 57b, _kcc_driver_link_all 611, _kcc_envp 615, _kcc_io_puts 70a, _kcc_lex_kw_init 104a, _kcc_pp_add_default_includes 150

Front-end diagnostic modes. The first group of flags run progressively deeper prefixes of the compilation pipeline, stopping early to dump intermediate results. `--dump-tokens` opens the source, runs translation phases 1–2, initializes the lexer, and prints every token. `-E` adds the preprocessor and dumps its output. `--dump-ast` further adds the parser and prints the AST tree.

⟨ *Main: entry point* 622 ⟩ +≡

```
.Lmain_dump_tokens:
    // --dump-tokens requires a source file
   argument
    cmp x19, #3
    b.lt .Lmain_dump_usage

    // Open and mmap the source file
    ldr x0, [x20, #16]            // argv[2] =
   source file path
    bl _kcc_source_open

    // Translation Phase 1: trigraph replacement (
   §5.1.1.2 para 1)
    bl _kcc_source_encode

    // Translation Phase 2: line splicing (§5
   .1.1.2 para 2)
    bl _kcc_source_splice

    // Initialize lexer with the translated source
    adrp x0, _kcc_src_base@PAGE
    add  x0, x0, _kcc_src_base@PAGEOFF
    ldr  x0, [x0]

    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    ldr  x1, [x1]

    bl _kcc_lex_init

    // Dump all tokens
    bl _kcc_lex_dump_all

    // Check if any errors occurred during lexing
    bl _kcc_diag_had_errors
    cbnz x0, .Lmain_error_exit
    b .Lmain_exit

.Lmain_preprocess:
    // -E requires a source file argument
    cmp x19, #3
    b.lt .Lmain_preprocess_usage

    // Open and mmap the source file
    ldr x0, [x20, #16]            // argv[2] =
   source file path
    bl _kcc_source_open

    // Translation Phase 1: trigraph replacement
    bl _kcc_source_encode

    // Translation Phase 2: line splicing
    bl _kcc_source_splice

    // Initialize lexer with the translated source
    adrp x0, _kcc_src_base@PAGE
    add  x0, x0, _kcc_src_base@PAGEOFF
    ldr  x0, [x0]

    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    ldr  x1, [x1]

    bl _kcc_lex_init

    // Initialize preprocessor
    bl _kcc_pp_init

    // Dump preprocessed tokens
    bl _kcc_pp_dump_all

    // Check for errors
    bl _kcc_diag_had_errors
    cbnz x0, .Lmain_error_exit
    b .Lmain_exit

.Lmain_dump_ast:
    // --dump-ast requires a source file argument
    cmp x19, #3
    b.lt .Lmain_ast_usage

    // Open and mmap the source file
    ldr x0, [x20, #16]            // argv[2] =
   source file path
    bl _kcc_source_open

    // Translation Phase 1: trigraph replacement
    bl _kcc_source_encode

    // Translation Phase 2: line splicing
    bl _kcc_source_splice

    // Initialize lexer with the translated source
    adrp x0, _kcc_src_base@PAGE
```

```
    add  x0, x0, _kcc_src_base@PAGEOFF
    ldr  x0, [x0]

    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    ldr  x1, [x1]

    bl _kcc_lex_init

    // Initialize preprocessor
    bl _kcc_pp_init

    // Initialize string table
    bl _kcc_strtab_init

    // Initialize typedef table
    bl _kcc_typedef_init

    // Parse translation unit
    bl _kcc_parse

    // Dump AST
    bl _kcc_ast_dump

    // Flush output
    bl _kcc_io_flush

    // Check for errors
    bl _kcc_diag_had_errors
    cbnz x0, .Lmain_error_exit
    b .Lmain_exit
```

□

622 · defined at 621, 622, 623, 624 · used in 27a · uses _kcc_ast_dump 173, _kcc_diag_had_errors 76d, _kcc_io_flush 69b, _kcc_lex_dump_all 100a, _kcc_lex_init 90a, _kcc_parse 172, _kcc_pp_dump_all 115, _kcc_pp_init 107b, _kcc_source_encode 85, _kcc_source_open 81b, _kcc_source_splice 87, _kcc_src_base 82, _kcc_src_size 82, _kcc_strtab_init 66a, _kcc_typedef_init 170a

Semantic analysis and IR modes. `--dump-types` runs the full front-end plus semantic analysis, then prints each declaration with its resolved type. `--check` performs the same pipeline but only reports errors—no output on success except a confirmation message. `--dump-ir` extends the pipeline through IR construction and optimization, then prints the textual IR.

⟨ *Main: entry point* 623 ⟩ + ≡

```
.Lmain_dump_types:
    // --dump-types: parse + sema + print typed
   declarations
    cmp x19, #3
    b.lt .Lmain_types_usage

    ldr x0, [x20, #16]          // argv[2]
    bl _kcc_source_open
    bl _kcc_source_encode
    bl _kcc_source_splice

    adrp x0, _kcc_src_base@PAGE
    add  x0, x0, _kcc_src_base@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_lex_init
    bl _kcc_pp_init
    bl _kcc_strtab_init
    bl _kcc_typedef_init
    bl _kcc_parse
    mov x21, x0                 // save
   trans_unit

    // Initialize type system and symbol table
    bl _kcc_types_init
    bl _kcc_symtab_init

    // Run semantic analysis
    mov x0, x21
    bl _kcc_sema_check

    // Dump types
    mov x0, x21
    bl _kcc_sema_dump_types

    bl _kcc_io_flush
    bl _kcc_diag_had_errors
    cbnz x0, .Lmain_error_exit
    b .Lmain_exit

.Lmain_check:
    // --check: parse + sema, report errors only
    cmp x19, #3
    b.lt .Lmain_check_usage

    ldr x0, [x20, #16]          // argv[2]
    bl _kcc_source_open
    bl _kcc_source_encode
    bl _kcc_source_splice

    adrp x0, _kcc_src_base@PAGE
    add  x0, x0, _kcc_src_base@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_lex_init
    bl _kcc_pp_init
    bl _kcc_strtab_init
    bl _kcc_typedef_init
    bl _kcc_parse
    mov x21, x0                 // save
   trans_unit

    // Initialize type system and symbol table
    bl _kcc_types_init
    bl _kcc_symtab_init

    // Run semantic analysis
    mov x0, x21
    bl _kcc_sema_check

    bl _kcc_io_flush
    bl _kcc_diag_had_errors
    cbnz x0, .Lmain_error_exit

    // Print success message
    adrp x0, _kcc_msg_check_ok@PAGE
    add  x0, x0, _kcc_msg_check_ok@PAGEOFF
    bl _kcc_io_puts
    b .Lmain_exit

.Lmain_dump_ir:
    // --dump-ir: parse + sema + IR build +
   optimize + dump
    cmp x19, #3
    b.lt .Lmain_ir_usage

    ldr x0, [x20, #16]          // argv[2]
    bl _kcc_source_open
    bl _kcc_source_encode
    bl _kcc_source_splice
```

```
    adrp x0, _kcc_src_base@PAGE
    add  x0, x0, _kcc_src_base@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_lex_init
    bl _kcc_pp_init
    bl _kcc_strtab_init
    bl _kcc_typedef_init
    bl _kcc_parse
    mov x21, x0                     // save
   trans_unit

    // Initialize type system and symbol table
    bl _kcc_types_init
    bl _kcc_symtab_init

    // Run semantic analysis
    mov x0, x21
    bl _kcc_sema_check

    // Check for errors before IR
    bl _kcc_diag_had_errors
    cbnz x0, .Lmain_error_exit

    // Initialize IR module
    bl _kcc_ir_init

    // Build IR from AST
    mov x0, x21
    bl _kcc_ir_build

    // Run optimization passes
    bl _kcc_opt_run

    // Dump IR
    bl _kcc_ir_dump

    bl _kcc_io_flush
    bl _kcc_diag_had_errors
    cbnz x0, .Lmain_error_exit
    b .Lmain_exit
```

□

623 · defined at 621, 622, 623, 624 · used in 27a · uses _kcc_diag_had_errors 76d, _kcc_io_flush 69b, _kcc_io_puts 70a, _kcc_ir_build 307, _kcc_ir_dump 334, _kcc_ir_init 302, _kcc_lex_init 90a, _kcc_opt_run 445a, _kcc_parse 172, _kcc_pp_init 107b, _kcc_sema_check 259a, _kcc_sema_dump_types 262, _kcc_source_encode 85, _kcc_source_open 81b, _kcc_source_splice 87, _kcc_src_base 82, _kcc_src_size 82, _kcc_strtab_init 66a, _kcc_symtab_init 253b, _kcc_typedef_init 170a, _kcc_types_init 239b

Compilation and error handling. The -c flag hands the whole argument vector to _kcc_driver_compile_all (Chapter 43), which compiles *each* source argument through the full pipeline and writes one Mach-O object file per input—so a multi-file subsystem builds in a single invocation. An explicit -o names the output only when there is exactly one source; otherwise each object takes its source's name with .c rewritten to .o. The remainder of the function handles usage errors for each flag, the no-arguments case, and the two exit paths (success with code 0, error with code 1).

⟨ *Main: entry point* 624 ⟩ + ≡

```
.Lmain_compile:
    // -c <source...> [-o <output>]
    // At minimum one source: kcc -c source.c  (
   argc >= 3)
    cmp x19, #3
    b.lt .Lmain_compile_usage

    mov x0, x19                     // argc
    mov x1, x20                     // argv
    bl _kcc_driver_compile_all      // compiles each
    source, then exits
    b .Lmain_exit

.Lmain_compile_usage:
    // Minimal: kcc -c source.c -o out.o (but
   allow kcc -c src -o out)
    // Also allow: kcc -c source (default output:
   a.o? No, require -o)
    adrp x0, _kcc_err_compile_usage@PAGE
    add  x0, x0, _kcc_err_compile_usage@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lmain_ir_usage:
    adrp x0, _kcc_err_ir_usage@PAGE
    add  x0, x0, _kcc_err_ir_usage@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lmain_types_usage:
    adrp x0, _kcc_err_types_usage@PAGE
    add  x0, x0, _kcc_err_types_usage@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lmain_check_usage:
    adrp x0, _kcc_err_check_usage@PAGE
    add  x0, x0, _kcc_err_check_usage@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lmain_ast_usage:
    adrp x0, _kcc_err_ast_usage@PAGE
    add  x0, x0, _kcc_err_ast_usage@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lmain_preprocess_usage:
    adrp x0, _kcc_err_preprocess_usage@PAGE
    add  x0, x0, _kcc_err_preprocess_usage@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lmain_dump_usage:
    adrp x0, _kcc_err_dump_usage@PAGE
    add  x0, x0, _kcc_err_dump_usage@PAGEOFF
    bl _kcc_io_err
    mov x0, #1
    bl _kcc_sys_exit

.Lmain_no_args:
    adrp x0, _kcc_msg_usage@PAGE
    add  x0, x0, _kcc_msg_usage@PAGEOFF
    bl _kcc_io_puts
    b .Lmain_exit

.Lmain_error_exit:
    bl _kcc_io_flush
    mov x0, #1
    bl _kcc_sys_exit
```

```
.Lmain_exit:
    // Flush buffered output
    bl _kcc_io_flush

    // Exit cleanly
    mov x0, #0
    bl _kcc_sys_exit

    // Unreachable
    brk #1
```

□

624 · defined at 621, 622, 623, 624 · used in 27a · uses _kcc_driver_compile_all 609, _kcc_io_err 71b, _kcc_io_flush 69b, _kcc_io_puts 70a

44.2. String Constants

The driver's string literals. These live in the read-only data section. Each is null-terminated for use with kcc_io_puts.

⟨ Main: data 625 ⟩ ≡

```
// -- Driver string constants ---
.section __TEXT,__cstring,cstring_literals
_kcc_banner:
    .asciz "kcc v⟨ version ? ⟩ - a formal C17 compiler
    for ARM64\n"
_kcc_msg_source:
    .asciz "source: "
_kcc_msg_usage:
    .asciz "usage: kcc [--dump-tokens|-E|--dump-
   ast|--dump-types|--check|--dump-ir|-c <src> -o
   <out>] <source.c>\n"
_kcc_msg_newline:
    .asciz "\n"
_kcc_opt_dump_tokens:
    .asciz "--dump-tokens"
_kcc_opt_preprocess:
    .asciz "-E"
_kcc_err_dump_usage:
    .asciz "kcc: error: --dump-tokens requires a
   source file\n"
_kcc_err_preprocess_usage:
    .asciz "kcc: error: -E requires a source file\
   n"
_kcc_opt_dump_ast:
    .asciz "--dump-ast"
_kcc_opt_dump_types:
    .asciz "--dump-types"
_kcc_opt_check:
    .asciz "--check"
_kcc_err_ast_usage:
    .asciz "kcc: error: --dump-ast requires a
   source file\n"
_kcc_err_types_usage:
    .asciz "kcc: error: --dump-types requires a
   source file\n"
_kcc_err_check_usage:
    .asciz "kcc: error: --check requires a source
   file\n"
_kcc_msg_check_ok:
    .asciz "semantic check passed\n"
_kcc_opt_dump_ir:
    .asciz "--dump-ir"
_kcc_err_ir_usage:
    .asciz "kcc: error: --dump-ir requires a
   source file\n"
_kcc_err_compile_usage:
    .asciz "kcc: error: usage: kcc -c <source.c
   >... [-o <output.o>]\n"
```

□

625 · used in 27a

Part X.

Verification

Trust but verify. This part contains the test infrastructure that guards against regressions as the compiler evolves. The test harness (Chapter 45) provides a minimal xUnit-style framework — written in the same bare-metal ARM64 assembly as the compiler itself — with setup helpers, assertion macros, and a runner that reports pass/fail counts to stdout. The conformance tests (Chapter 46) exercise the lexer against the token categories defined by the C17 grammar: keywords, numeric constants, string and character literals, punctuators, comments, trigraphs, and line splicing. Each test feeds a known input string to the lexer and asserts the expected token type and spelling. The code generation tests (Chapter 47) take a different approach: they compile complete C programs with kcc, link the resulting object files, execute the binaries, and verify that the exit code matches the expected value.

45. Test Harness

A compiler that cannot verify its own output is a liability, so we maintain a separate test binary alongside the compiler itself. This chapter implements an xUnit-style test runner in pure ARM64 assembly: each test is a function that returns zero on success and non-zero on failure, registered in a global table alongside its name. The runner iterates through the table, calls each test, tallies pass and fail counts, and exits with a non-zero status if any test failed.

45.1. Runner Entry Point

The test runner's `_main` initializes the arena and keyword table, then iterates through the test registration table. Each entry contains a test name string and a function pointer. The table is terminated by a null name sentinel.

For each test, the runner prints the test name, calls the function, and prints `PASS` or `FAIL` based on the return value. After all tests, a summary line reports the pass/fail counts and the process exits with status 0 (all passed) or 1 (any failed).

⟨ *Test: runner entry* 629 ⟩ ≡

```
// -- test_kcc _main ---
// Entry point for the test binary.
.text
.globl _main
.p2align 2
_main:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    // Initialize subsystems
    bl _kcc_arena_init
    bl _kcc_lex_kw_init

    // Print header
    adrp x0, _test_header@PAGE
    add  x0, x0, _test_header@PAGEOFF
    bl _kcc_io_puts
    bl _kcc_io_flush

    mov x19, #0                    // x19 = pass
   count
    mov x20, #0                    // x20 = fail
   count

    // Load test table pointer
    adrp x21, _kcc_test_table@PAGE
    add  x21, x21, _kcc_test_table@PAGEOFF

.Ltest_loop:
    ldr x0, [x21]                  // test name
    cbz x0, .Ltest_summary         // null sentinel
  → done
    ldr x22, [x21, #8]             // test function
   pointer

    // Print "  name... "
    adrp x0, _test_prefix@PAGE
    add  x0, x0, _test_prefix@PAGEOFF
    bl _kcc_io_puts
    ldr x0, [x21]                  // test name
    bl _kcc_io_puts
    adrp x0, _test_dots@PAGE
    add  x0, x0, _test_dots@PAGEOFF
    bl _kcc_io_puts
    bl _kcc_io_flush

    // Reset diagnostic counters before each test
    bl _kcc_diag_reset

    // Call test function
    blr x22

    // Check result: 0 = pass, non-zero = fail
    cbnz x0, .Ltest_fail

    // PASS
    adrp x0, _test_pass@PAGE
    add  x0, x0, _test_pass@PAGEOFF
    bl _kcc_io_puts
    bl _kcc_io_flush
    add x19, x19, #1
    b .Ltest_next

.Ltest_fail:
    adrp x0, _test_fail@PAGE
    add  x0, x0, _test_fail@PAGEOFF
    bl _kcc_io_puts
    bl _kcc_io_flush
    add x20, x20, #1

.Ltest_next:
    add x21, x21, #16              // advance to
   next table entry
    b .Ltest_loop

.Ltest_summary:
    // Print summary: "\nResults: N passed, M
   failed\n"
    adrp x0, _test_results@PAGE
    add  x0, x0, _test_results@PAGEOFF
    bl _kcc_io_puts
    mov x0, x19
    bl _kcc_io_putu
    adrp x0, _test_passed_str@PAGE
    add  x0, x0, _test_passed_str@PAGEOFF
    bl _kcc_io_puts
    mov x0, x20
    bl _kcc_io_putu
    adrp x0, _test_failed_str@PAGE
```

```
    add  x0, x0, _test_failed_str@PAGEOFF
    bl _kcc_io_puts

    bl _kcc_io_flush

    // Exit: 0 if all passed, 1 if any failed
    cmp x20, #0
    b.ne .Ltest_exit_fail
    mov x0, #0
    bl _kcc_sys_exit
.Ltest_exit_fail:
    mov x0, #1
    bl _kcc_sys_exit
```

□

629 · used in 32r · defines _main 38k, 611, 615, 621, 637, 640b, 644, 646b, 648b, 651, 653b, 656c, 659b, 662c, 665, 668c, 673, 681c, 684b, 686c, 694c, 702b, 710b, 717b, 719, 725b, 738b, 742b, 751, 768b, 792b, 818b, 836b, 902b, 909, 914, 917 · uses _kcc_arena_init 57b, _kcc_diag_reset 76e, _kcc_io_flush 69b, _kcc_io_puts 70a, _kcc_io_putu 71a, _kcc_lex_kw_init 104a, _kcc_test_table 636a

45.2. Test Assertion Helpers

Two helper functions simplify test writing:

`kcc_test_lex_setup` initializes the lexer with a test source string, applying translation phases 1 and 2. It also sets `_kcc_src_path` to `<test>` for diagnostic output.

`kcc_test_expect_tok` calls `lex_next` and checks whether the resulting token type matches the expected value. Returns 0 on match, 1 on mismatch.

⟨ *Test: assert helpers* 630a ⟩ ≡

```
// -- kcc_test_lex_setup ---
// void kcc_test_lex_setup(const char *src, size_t
    len)
//   x0 = pointer to test source string
//   x1 = length in bytes (excluding null
   terminator)
// Sets up the lexer with translation phases 1 and
    2.
.globl _kcc_test_lex_setup
.p2align 2
_kcc_test_lex_setup:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    stp x19, x20, [sp, #16]

    mov x19, x0                    // src pointer
    mov x20, x1                    // src length

    // Set source path to "<test>"
    adrp x2, _kcc_test_path@PAGE
    add  x2, x2, _kcc_test_path@PAGEOFF
    adrp x3, _kcc_src_path@PAGE
    add  x3, x3, _kcc_src_path@PAGEOFF
    str  x2, [x3]

    // Set src_base and src_size for translation
   phases
    adrp x3, _kcc_src_base@PAGE
    add  x3, x3, _kcc_src_base@PAGEOFF
    str  x19, [x3]

    adrp x3, _kcc_src_size@PAGE
    add  x3, x3, _kcc_src_size@PAGEOFF
    str  x20, [x3]

    // Apply translation phase 1 (trigraphs →
   arena copy)
    bl _kcc_source_encode

    // Apply translation phase 2 (line splicing,
   in-place)
    bl _kcc_source_splice

    // Initialize lexer with the translated buffer
    adrp x0, _kcc_src_base@PAGE
    add  x0, x0, _kcc_src_base@PAGEOFF
    ldr  x0, [x0]
    adrp x1, _kcc_src_size@PAGE
    add  x1, x1, _kcc_src_size@PAGEOFF
    ldr  x1, [x1]
    bl _kcc_lex_init

    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #32
    ret

// -- kcc_test_expect_tok ---
// int kcc_test_expect_tok(int expected_type)
//   x0 = expected token type code
// Calls lex_next, checks tok_type against
   expected.
// Returns 0 if match, 1 if mismatch.
.globl _kcc_test_expect_tok
.p2align 2
_kcc_test_expect_tok:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x19, [sp, #16]

    mov x19, x0                     // save expected
    type

    bl _kcc_lex_next

    adrp x0, _kcc_tok_type@PAGE
    add  x0, x0, _kcc_tok_type@PAGEOFF
    ldr  x0, [x0]

    cmp x0, x19
    b.eq .Lexpect_match
    mov x0, #1                     // FAIL
    b .Lexpect_done
.Lexpect_match:
    mov x0, #0                     // PASS
.Lexpect_done:
    ldr x19, [sp, #16]
    ldp x29, x30, [sp], #32
    ret
```

□

630a · used in 32r · defines _kcc_test_expect_tok 633abc, 634ab, 635ab, _kcc_test_lex_setup 633abc, 634ab, 635ab · uses _kcc_lex_init 90a, _kcc_lex_next 90b, _kcc_source_encode 85, _kcc_source_splice 87, _kcc_src_base 82, _kcc_src_path 82, _kcc_src_size 82, _kcc_test_path 630b, _kcc_tok_type 104b

45.3. Runner Data

String constants for the test runner's output formatting.

⟨ *Test: runner data* 630b ⟩ ≡

```
// -- Test runner string constants ---
.section __TEXT,__cstring,cstring_literals
```

```
_test_header:     .asciz "kcc test suite\n
   ==============\n"
_test_prefix:     .asciz "  "
_test_dots:       .asciz "... "
_test_pass:       .asciz "PASS\n"
_test_fail:       .asciz "FAIL\n"
_test_results:    .asciz "\nResults: "
_test_passed_str: .asciz " passed, "
_test_failed_str: .asciz " failed\n"
_kcc_test_path:   .asciz "<test>"
```

□

630b · used in 32r · defines _kcc_test_path 630a

46. Conformance Tests

With the test harness from Chapter 45 in place, this chapter populates it with concrete test cases for the lexer. Each test function constructs a small source string, feeds it to the lexer, and verifies that the resulting token stream matches the expected sequence of token kinds. The suite covers keyword recognition, operator and punctuator scanning, numeric and string literal tokenization, and edge cases such as maximal-munch disambiguation.

46.1. Keyword Recognition

Tests that representative keywords from all categories are correctly identified, and that identifiers resembling keyword prefixes are *not* misidentified.

⟨ *Test: lex keywords* 633a ⟩ ≡

```
// -- test_lex_keywords ---
// Source: "auto int return _Static_assert foo"
// Expected: AUTO, INT, RETURN, _STATIC_ASSERT,
   IDENT, EOF
.text
.globl _test_lex_keywords
.p2align 2
_test_lex_keywords:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _test_src_kw@PAGE
    add  x0, x0, _test_src_kw@PAGEOFF
    mov  x1, #TEST_SRC_KW_LEN
    bl _kcc_test_lex_setup

    mov x0, #TOK_AUTO
    bl _kcc_test_expect_tok
    cbnz x0, .Ltkw_fail

    mov x0, #TOK_INT
    bl _kcc_test_expect_tok
    cbnz x0, .Ltkw_fail

    mov x0, #TOK_RETURN
    bl _kcc_test_expect_tok
    cbnz x0, .Ltkw_fail

    mov x0, #TOK__STATIC_ASSERT
    bl _kcc_test_expect_tok
    cbnz x0, .Ltkw_fail

    mov x0, #TOK_IDENT
    bl _kcc_test_expect_tok
    cbnz x0, .Ltkw_fail

    mov x0, #TOK_EOF
    bl _kcc_test_expect_tok
    cbnz x0, .Ltkw_fail

    mov x0, #0
    ldp x29, x30, [sp], #16
    ret
.Ltkw_fail:
    mov x0, #1
    ldp x29, x30, [sp], #16
    ret
```

□

633a · used in 33a · defines _test_lex_keywords 636a · uses _kcc_test_expect_tok 630a, _kcc_test_lex_setup 630a

46.2. Numeric Literals

Tests that decimal, hexadecimal, floating-point, and scientific notation numbers are all classified as TOK_NUMBER (pp-number).

⟨ *Test: lex numbers* 633b ⟩ ≡

```
// -- test_lex_numbers ---
// Source: "42 0xFF 3.14 1e10 0x1p2"
// Expected: NUMBER, NUMBER, NUMBER, NUMBER,
   NUMBER, EOF
.text
.globl _test_lex_numbers
.p2align 2
_test_lex_numbers:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _test_src_num@PAGE
    add  x0, x0, _test_src_num@PAGEOFF
    mov  x1, #TEST_SRC_NUM_LEN
    bl _kcc_test_lex_setup

    mov x0, #TOK_NUMBER
    bl _kcc_test_expect_tok
    cbnz x0, .Ltnum_fail

    mov x0, #TOK_NUMBER
    bl _kcc_test_expect_tok
    cbnz x0, .Ltnum_fail

    mov x0, #TOK_NUMBER
    bl _kcc_test_expect_tok
    cbnz x0, .Ltnum_fail

    mov x0, #TOK_NUMBER
    bl _kcc_test_expect_tok
    cbnz x0, .Ltnum_fail

    mov x0, #TOK_NUMBER
    bl _kcc_test_expect_tok
    cbnz x0, .Ltnum_fail

    mov x0, #TOK_EOF
    bl _kcc_test_expect_tok
    cbnz x0, .Ltnum_fail

    mov x0, #0
    ldp x29, x30, [sp], #16
    ret
.Ltnum_fail:
    mov x0, #1
    ldp x29, x30, [sp], #16
    ret
```

□

633b · used in 33a · defines _test_lex_numbers 636a · uses _kcc_test_expect_tok 630a, _kcc_test_lex_setup 630a

46.3. String and Character Literals

Tests that string literals and character constants are correctly classified.

⟨ *Test: lex strings* 633c ⟩ ≡

```
// -- test_lex_strings ---
// Source: "hello" 'x'
// (The outer quotes in the .asciz are assembler
   string delimiters;
//  the inner \" produce the actual " character in
    the test source.)
// Expected: STRING_LIT, CHAR_LIT, EOF
.text
.globl _test_lex_strings
.p2align 2
_test_lex_strings:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _test_src_str@PAGE
    add  x0, x0, _test_src_str@PAGEOFF
    mov  x1, #TEST_SRC_STR_LEN
    bl _kcc_test_lex_setup

    mov x0, #TOK_STRING_LIT
    bl _kcc_test_expect_tok
    cbnz x0, .Ltstr_fail

    mov x0, #TOK_CHAR_LIT
    bl _kcc_test_expect_tok
    cbnz x0, .Ltstr_fail

    mov x0, #TOK_EOF
    bl _kcc_test_expect_tok
    cbnz x0, .Ltstr_fail

    mov x0, #0
    ldp x29, x30, [sp], #16
    ret
.Ltstr_fail:
    mov x0, #1
    ldp x29, x30, [sp], #16
    ret
```

□

633c · used in 33a · defines _test_lex_strings 636a · uses _kcc_test_expect_tok 630a, _kcc_test_lex_setup 630a

46.4. Punctuators

Tests that single-character and multi-character punctuators are correctly recognized.

⟨ *Test: lex punctuators* 634a ⟩ ≡

```
// -- test_lex_punctuators ---
// Source: "(){};,+-*/"
// Expected: LPAREN RPAREN LBRACE RBRACE SEMI
   COMMA PLUS MINUS STAR SLASH EOF
.text
.globl _test_lex_punctuators
.p2align 2
_test_lex_punctuators:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _test_src_punct@PAGE
    add  x0, x0, _test_src_punct@PAGEOFF
    mov  x1, #TEST_SRC_PUNCT_LEN
    bl _kcc_test_lex_setup

    mov x0, #TOK_LPAREN
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #TOK_RPAREN
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #TOK_LBRACE
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #TOK_RBRACE
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #TOK_SEMI
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #TOK_COMMA
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #TOK_PLUS
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #TOK_MINUS
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #TOK_STAR
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #TOK_SLASH
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #TOK_EOF
    bl _kcc_test_expect_tok
    cbnz x0, .Ltpunct_fail

    mov x0, #0
    ldp x29, x30, [sp], #16
    ret
.Ltpunct_fail:
    mov x0, #1
    ldp x29, x30, [sp], #16
    ret
```

□

634a · used in 33a · defines _test_lex_punctuators 636a · uses _kcc_test_expect_tok 630a, _kcc_test_lex_setup 630a

46.5. Comments

Tests that both line comments (//) and block comments (/* */) are correctly skipped by the lexer.

⟨ *Test: lex comments* 634b ⟩ ≡

```
// -- test_lex_comments ---
// Source: "x /* comment */ y // line\nz"
// Expected: IDENT(x) IDENT(y) IDENT(z) EOF
.text
.globl _test_lex_comments
.p2align 2
_test_lex_comments:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _test_src_comments@PAGE
    add  x0, x0, _test_src_comments@PAGEOFF
    mov  x1, #TEST_SRC_COMMENTS_LEN
    bl _kcc_test_lex_setup

    mov x0, #TOK_IDENT
    bl _kcc_test_expect_tok
```

```
    cbnz x0, .Ltcmt_fail

    mov x0, #TOK_IDENT
    bl _kcc_test_expect_tok
    cbnz x0, .Ltcmt_fail

    mov x0, #TOK_IDENT
    bl _kcc_test_expect_tok
    cbnz x0, .Ltcmt_fail

    mov x0, #TOK_EOF
    bl _kcc_test_expect_tok
    cbnz x0, .Ltcmt_fail

    mov x0, #0
    ldp x29, x30, [sp], #16
    ret
.Ltcmt_fail:
    mov x0, #1
    ldp x29, x30, [sp], #16
    ret
```

□

634b · used in 33a · defines _test_lex_comments 636a · uses _kcc_test_expect_tok 630a, _kcc_test_lex_setup 630a

46.6. Trigraph Replacement

Tests that trigraph sequences are correctly replaced before tokenization. The source ??= becomes #, which the lexer should tokenize as TOK_HASH.

⟨ *Test: lex trigraphs* 635a ⟩ ≡

```
// -- test_lex_trigraphs ---
// Source: "??= x ??( ??) ??< ??>"
// After trigraph replacement: "# x [ ] { }"
// Expected: HASH IDENT LBRACKET RBRACKET LBRACE
   RBRACE EOF
.text
.globl _test_lex_trigraphs
.p2align 2
_test_lex_trigraphs:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _test_src_trigraph@PAGE
    add  x0, x0, _test_src_trigraph@PAGEOFF
    mov  x1, #TEST_SRC_TRIGRAPH_LEN
    bl _kcc_test_lex_setup

    mov x0, #TOK_HASH
    bl _kcc_test_expect_tok
    cbnz x0, .Lttrig_fail

    mov x0, #TOK_IDENT
    bl _kcc_test_expect_tok
    cbnz x0, .Lttrig_fail

    mov x0, #TOK_LBRACKET
    bl _kcc_test_expect_tok
    cbnz x0, .Lttrig_fail

    mov x0, #TOK_RBRACKET
    bl _kcc_test_expect_tok
    cbnz x0, .Lttrig_fail

    mov x0, #TOK_LBRACE
    bl _kcc_test_expect_tok
    cbnz x0, .Lttrig_fail

    mov x0, #TOK_RBRACE
    bl _kcc_test_expect_tok
    cbnz x0, .Lttrig_fail

    mov x0, #TOK_EOF
    bl _kcc_test_expect_tok
    cbnz x0, .Lttrig_fail

    mov x0, #0
    ldp x29, x30, [sp], #16
    ret
.Lttrig_fail:
    mov x0, #1
    ldp x29, x30, [sp], #16
    ret
```

□

635a · used in 33a · defines _test_lex_trigraphs 636a · uses _kcc_test_expect_tok 630a, _kcc_test_lex_setup 630a

46.7. Line Splicing

Tests that backslash-newline sequences are removed before tokenization. The source `in\\n_t x` (with a physical newline after the backslash) becomes `int x` after splicing, so `int` is recognized as a keyword.

⟨ *Test: lex line splice* 635b ⟩ ≡

```
// -- test_lex_line_splice ---
// Source: "in\<newline>t x" (stored as bytes: i n
    \ 0x0A t space x)
// After line splicing: "int x"
// Expected: INT IDENT(x) EOF
.text
.globl _test_lex_line_splice
.p2align 2
_test_lex_line_splice:
    stp x29, x30, [sp, #-16]!
    mov x29, sp

    adrp x0, _test_src_splice@PAGE
    add  x0, x0, _test_src_splice@PAGEOFF
    mov  x1, #TEST_SRC_SPLICE_LEN
    bl _kcc_test_lex_setup

    mov x0, #TOK_INT
    bl _kcc_test_expect_tok
    cbnz x0, .Ltspl_fail

    mov x0, #TOK_IDENT
    bl _kcc_test_expect_tok
    cbnz x0, .Ltspl_fail

    mov x0, #TOK_EOF
    bl _kcc_test_expect_tok
    cbnz x0, .Ltspl_fail

    mov x0, #0
    ldp x29, x30, [sp], #16
    ret
.Ltspl_fail:
    mov x0, #1
    ldp x29, x30, [sp], #16
    ret
```

□

635b · used in 33a · defines _test_lex_line_splice 636a · uses _kcc_test_expect_tok 630a, _kcc_test_lex_setup 630a

46.8. Test Registration Table

The test table is a flat array of (name, function-pointer) pairs, terminated by a null sentinel. The test runner in

Chapter 45 iterates through this table.

⟨ *Test: registration* 636a ⟩ ≡

```
// -- Test registration table ---
.data
.p2align 3
.globl _kcc_test_table
_kcc_test_table:
    .quad _test_name_keywords
    .quad _test_lex_keywords
    .quad _test_name_numbers
    .quad _test_lex_numbers
    .quad _test_name_strings
    .quad _test_lex_strings
    .quad _test_name_punctuators
    .quad _test_lex_punctuators
    .quad _test_name_comments
    .quad _test_lex_comments
    .quad _test_name_trigraphs
    .quad _test_lex_trigraphs
    .quad _test_name_line_splice
    .quad _test_lex_line_splice
    .quad 0                         // sentinel:
   null name
    .quad 0
```

□

636a · used in 33a · defines _kcc_test_table 629 · uses _test_lex_comments 634b, _test_lex_keywords 633a, _test_lex_line_splice 635b, _test_lex_numbers 633b, _test_lex_punctuators 634a, _test_lex_strings 633c, _test_lex_trigraphs 635a

46.9. Test Source Data

Source strings and test names used by the conformance tests above. Each string has a companion .equ that computes its length (excluding the null terminator) for passing to kcc_lex_init.

⟨ *Test: lex data* 636b ⟩ ≡

```
// -- Test source strings ---
.section __TEXT,__cstring,cstring_literals

_test_src_kw:       .asciz "auto int return
   _Static_assert foo"
_test_src_kw_end:
.equ TEST_SRC_KW_LEN, _test_src_kw_end -
   _test_src_kw - 1

_test_src_num:      .asciz "42 0xFF 3.14 1e10 0
   x1p2"
_test_src_num_end:
.equ TEST_SRC_NUM_LEN, _test_src_num_end -
   _test_src_num - 1

_test_src_str:
    .byte '"'
    .ascii "hello"
    .byte '"'
    .ascii " "
    .byte '\''
    .ascii "x"
    .byte '\''
    .byte 0
_test_src_str_end:
.equ TEST_SRC_STR_LEN, _test_src_str_end -
   _test_src_str - 1

_test_src_punct:    .asciz "(){};,+-*/"
_test_src_punct_end:
.equ TEST_SRC_PUNCT_LEN, _test_src_punct_end -
   _test_src_punct - 1

_test_src_comments:
    .ascii "x /* comment */ y // line"
    .byte 0x0A
    .ascii "z"
    .byte 0
_test_src_comments_end:
.equ TEST_SRC_COMMENTS_LEN, _test_src_comments_end
    - _test_src_comments - 1

_test_src_trigraph: .asciz "??= x ??( ??) ??< ??>"
_test_src_trigraph_end:
.equ TEST_SRC_TRIGRAPH_LEN, _test_src_trigraph_end
    - _test_src_trigraph - 1

_test_src_splice:
    .ascii "in"
    .byte '\\'
    .byte 0x0A
    .ascii "t x"
    .byte 0
_test_src_splice_end:
.equ TEST_SRC_SPLICE_LEN, _test_src_splice_end -
   _test_src_splice - 1

// Test name strings
_test_name_keywords:     .asciz "lex_keywords"
_test_name_numbers:      .asciz "lex_numbers"
_test_name_strings:      .asciz "lex_strings"
_test_name_punctuators:  .asciz "lex_punctuators"
_test_name_comments:     .asciz "lex_comments"
_test_name_trigraphs:    .asciz "lex_trigraphs"
_test_name_line_splice:  .asciz "lex_line_splice"
```

□

636b · used in 33a

47. Code Generation Tests

The lexer conformance tests of Chapter 46 exercise the compiler's frontend by calling internal functions from ARM64 assembly. Code generation tests require a different strategy: they compile complete C programs with kcc, link the resulting object files into executables, and verify that the process exits with the expected status code. A shell script orchestrates the pipeline—write source, compile, link, run, compare—and reports results in the same pass/fail format as the unit test harness.

Individual test groups are defined alongside the features they exercise: constants and arithmetic in Chapter 35, local variables, control flow, switch statements, ternary expressions, sizeof, arrays, structs, multi-variable declarations, unsigned operations and type extensions, increment/decrement, compound assignment, logical operators, function calls, pointers, and miscellaneous cases in Chapter 29. This chapter provides the runner infrastructure that ties them together.

47.1. Test Runner

The script defines a helper function `run_test` that receives a test name, a C source string, and the expected exit code. It writes the source to a temporary file, invokes `gen/kcc -c` to produce an object file, links it with the system linker, runs the resulting binary, and compares the actual exit code against the expectation. Failures print the expected and actual values for diagnosis.

A sibling `run_test_include` additionally writes a companion header `defs.h` next to the source before compiling, so that tests can exercise `#include "defs.h"` and confirm the preprocessor resolves a header sitting beside the translation unit. A second sibling `run_test_multi` writes two source files and compiles them with a single `kcc -c a.c b.c` invocation, then links both objects, exercising multi-translation-unit builds and cross-object symbol resolution.

⟨ *Test: codegen runner* 637 ⟩ ≡

```
#!/bin/bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# kcc code generation test suite
# Generated by weft - do not edit
set -u

PASS=0
FAIL=0
TOTAL=0
KCC="$BASE/kcc"
TMPDIR="/$BASE/kcc_test_$$"
SDK=$(xcrun --show-sdk-path 2>/dev/null)

mkdir -p "$TMPDIR"
trap 'rm -rf "$TMPDIR"' EXIT

run_test() {
    local name="$1"
    local src="$2"
    local expected="$3"
    TOTAL=$((TOTAL + 1))

    echo "$src" > "$TMPDIR/t.c"

    # Compile
    if ! $KCC -c "$TMPDIR/t.c" -o "$TMPDIR/t.o"
   2>/dev/null; then
        FAIL=$((FAIL + 1))
        printf "  %-40s FAIL (compile error)\n" "
   $name..."
        return
    fi

    # Link
    if ! ld "$TMPDIR/t.o" -o "$TMPDIR/t" \
            -lSystem -syslibroot "$SDK" \
            -e _main -arch arm64 2>/dev/null; then
        FAIL=$((FAIL + 1))
        printf "  %-40s FAIL (link error)\n" "
   $name..."
        return
    fi

    # Run
    "$TMPDIR/t"
    local actual=$?

    if [ "$actual" -eq "$expected" ]; then
        PASS=$((PASS + 1))
        printf "  %-40s PASS\n" "$name..."
    else
        FAIL=$((FAIL + 1))
        printf "  %-40s FAIL (expected %d, got %d)
   \n" "$name..." "$expected" "$actual"
    fi
}

# Variant for #include: writes a companion header
   "defs.h" beside the source
# so that the preprocessor's source-relative
   resolution locates it.
run_test_include() {
    local name="$1"
    local header="$2"
    local src="$3"
    local expected="$4"
    TOTAL=$((TOTAL + 1))

    echo "$header" > "$TMPDIR/defs.h"
    echo "$src" > "$TMPDIR/t.c"

    if ! $KCC -c "$TMPDIR/t.c" -o "$TMPDIR/t.o"
   2>/dev/null; then
        FAIL=$((FAIL + 1))
        printf "  %-40s FAIL (compile error)\n" "
   $name..."
        return
    fi
    if ! ld "$TMPDIR/t.o" -o "$TMPDIR/t" \
            -lSystem -syslibroot "$SDK" \
            -e _main -arch arm64 2>/dev/null; then
        FAIL=$((FAIL + 1))
        printf "  %-40s FAIL (link error)\n" "
   $name..."
        return
    fi

    "$TMPDIR/t"
    local actual=$?
```

```
    if [ "$actual" -eq "$expected" ]; then
        PASS=$((PASS + 1))
        printf "  %-40s PASS\n" "$name..."
    else
        FAIL=$((FAIL + 1))
        printf "  %-40s FAIL (expected %d, got %d)
   \n" "$name..." "$expected" "$actual"
    fi
}

# Variant for multi-file compilation: compiles two
    translation units in a
# single "kcc -c a.c b.c" invocation, then links
   both objects together.
run_test_multi() {
    local name="$1"
    local file_a="$2"
    local file_b="$3"
    local expected="$4"
    TOTAL=$((TOTAL + 1))

    echo "$file_a" > "$TMPDIR/a.c"
    echo "$file_b" > "$TMPDIR/b.c"

    if ! $KCC -c "$TMPDIR/a.c" "$TMPDIR/b.c" >/dev
   /null 2>&1; then
        FAIL=$((FAIL + 1))
        printf "  %-40s FAIL (compile error)\n" "
   $name..."
        return
    fi
    if ! ld "$TMPDIR/a.o" "$TMPDIR/b.o" -o "
   $TMPDIR/t" \
            -lSystem -syslibroot "$SDK" \
            -e _main -arch arm64 2>/dev/null; then
        FAIL=$((FAIL + 1))
        printf "  %-40s FAIL (link error)\n" "
   $name..."
        return
    fi

    "$TMPDIR/t"
    local actual=$?

    if [ "$actual" -eq "$expected" ]; then
        PASS=$((PASS + 1))
        printf "  %-40s PASS\n" "$name..."
    else
        FAIL=$((FAIL + 1))
        printf "  %-40s FAIL (expected %d, got %d)
   \n" "$name..." "$expected" "$actual"
    fi
}

echo "kcc codegen test suite"
echo "======================"
```

□

637 · used in 33b · uses _main 621, 629

47.2. Results

After all test groups have run, the script prints a summary line and exits with status 0 only if every test passed.

⟨ *Test: codegen results* 638 ⟩ ≡

```
echo ""
echo "======================"
echo "Results: $PASS passed, $FAIL failed (of
   $TOTAL)"
[ "$FAIL" -eq 0 ]
```

□

638 · used in 33b

48. Self-Hosting: The Arena in C

Every preceding chapter builds kcc out of hand-written ARM64 assembly. This chapter closes a loop: it re-implements one of the compiler's own subsystems—the arena allocator of Chapter 8—in C, compiles that C with kcc, and links the resulting object back into the compiler in place of the assembly version. If the twice-removed compiler (assembly kcc compiling C that becomes part of kcc) still passes the entire code-generation suite, then kcc has demonstrably compiled a non-trivial, correctness-critical piece of itself. This is the first concrete step of Stage 8b.

The arena is the ideal candidate. It is small, self-contained, and exercised by *every* other subsystem: tokens, AST nodes, types, symbols, IR, and machine code all draw their storage from it. A single mistake in the C port—a mis-computed offset, a botched pointer comparison, an off-by-one in the alignment—would corrupt allocations and cascade into failures across the whole suite. Passing all 201 tests is therefore a strong signal that the features this port relies on (`struct` layout, `->` access, pointer casts and arithmetic, external calls, globals, the ternary and bitwise operators) are correct together, not merely in isolation.

48.1. Interface and Chunk Layout

The C port must be a drop-in replacement, so it exports exactly the symbols the assembly arena did—`kcc_arena_init`, `kcc_arena_alloc`, and the global `kcc_arena_current`—and it depends only on the two syscall wrappers the assembly arena already used. Those wrappers stay in assembly; the port calls them as ordinary `extern` functions, and the linker resolves the references across objects exactly as §35.4 describes.

The `Chunk` structure mirrors the 24-byte header of Chapter 8 field for field: `next` at offset 0, `limit` at 8, `avail` at 16. In the all-spill model each struct field occupies one eight-byte slot, so this C layout and the assembly layout coincide byte for byte—which is what lets the port read and write the same header a future chunk allocation will expect. `limit` and `avail` are `char *` so that pointer arithmetic on them counts bytes rather than being scaled.

⟨ *Selfhost: arena preamble* 639a ⟩ ≡

```
struct Chunk {
    struct Chunk *next;
    char *limit;
    char *avail;
};

void *kcc_sys_mmap(void *addr, long len, int prot,
    int flags, int fd, long off);
void  kcc_sys_exit(int status);

struct Chunk *kcc_arena_current;
```

□

639a · used in 640a

48.2. Initialization

`kcc_arena_init` requests a 64 KiB private, anonymous mapping from the kernel (`prot` = `PROT_READ|PROT_WRITE` = 3, `flags` = `MAP_PRIVATE|MAP_ANON` = 0x1002 = 4098), bails out with status 1 on failure, and writes the three header fields: an empty `next`, a `limit` one past the mapping, and an `avail` pointing just past the 24-byte header. The finished chunk becomes the current one. The failure test casts the returned pointer to `long` and compares against -1 (`MAP_FAILED`), mirroring the assembly's `cmn` check.

⟨ *Selfhost: arena init* 639b ⟩ ≡

```
void kcc_arena_init(void) {
    struct Chunk *c = (struct Chunk *)kcc_sys_mmap
   (0, 65536, 3, 4098, -1, 0);
    if ((long)c == -1) kcc_sys_exit(1);
    c->next = 0;
    c->limit = (char *)c + 65536;
    c->avail = (char *)c + 24;
    kcc_arena_current = c;
}
```

□

639b · used in 640a

48.3. Allocation

`kcc_arena_alloc` first rounds the request up to the next multiple of eight with the classic `(size + 7) & ~7`. The fast path is a single bounds check: if the bumped `avail` would still fit under `limit`, it advances the pointer and returns the old value. Both operands of the comparison are `char *`, so `p + size` is a true byte offset.

When the current chunk cannot satisfy the request, the slow path maps a fresh chunk sized to the larger of the default 64 KiB and the request plus a header, links it ahead of the current chunk, and serves the allocation from it. Unlike the assembly version—which conservatively set every chunk's `limit` to base + 64 KiB—this port sets `limit` to base + *actual size*, so an oversized chunk is fully usable; for every allocation the test suite makes, the two behave identically.

⟨ *Selfhost: arena alloc* 639c ⟩ ≡

```
void *kcc_arena_alloc(long size) {
    size = (size + 7) & ~7;
    struct Chunk *c = kcc_arena_current;
    char *p = c->avail;
    if (p + size <= c->limit) {
        c->avail = p + size;
        return p;
    }
    long need = size + 24;
    long chunksz = need > 65536 ? need : 65536;
```

```
    struct Chunk *n = (struct Chunk *)kcc_sys_mmap
   (0, chunksz, 3, 4098, -1, 0);
    if ((long)n == -1) kcc_sys_exit(2);
    n->next = c;
    n->limit = (char *)n + chunksz;
    char *base = (char *)n + 24;
    n->avail = base + size;
    kcc_arena_current = n;
    return base;
}
```

□

639c · used in 640a

48.4. Assembling the Translation Unit

The three fragments compose into one C translation unit, routed by the assembly file to `gen/selfhost/arena.c`. This is the file `make selfhost` feeds to kcc.

⟨ *Selfhost: arena* 640a ⟩ ≡

```
⟨ Selfhost: arena preamble 639a ⟩
⟨ Selfhost: arena init 639b ⟩
⟨ Selfhost: arena alloc 639c ⟩
```

□

640a · used in 33c

48.5. Verification

The `make selfhost` recipe performs the whole loop. It compiles `gen/selfhost/arena.c` with the assembly-built kcc (`kcc -c arena.c -o arena_c.o`), then relinks a second compiler binary, `gen/kcc_selfhost`, from all the usual objects but with the hand-written `arena.o` swapped out for the freshly compiled `arena_c.o`. Finally it points the code-generation suite at that binary.

The passing criterion is unforgiving and complete: `kcc_selfhost` must reproduce all 201 results of the assembly-built compiler. Because the arena underlies every allocation, and because the 201 programs between them allocate enough to spill past a single 64 KiB chunk many times over, a green run means the C arena—compiled by kcc—is byte-for-byte behaviourally equivalent to the assembly original, slow path and all.

⟨ *Selfhost: verify* 640b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost - compile kcc's arena in C with
   kcc, relink, run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/arena.c with
   kcc"
$BASE/kcc -c "$SH/arena.c" -o "$SH/arena_c.o" >/
   dev/null

echo "[2/3] linking $BASE/kcc_selfhost (arena.o ->
    kcc-compiled arena_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "arena" ]; then OBJ="$OBJ $SH/
   arena_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
   fi
done
ld -o $BASE/kcc_selfhost $OBJ -lSystem -syslibroot
    "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted arena"
cp $BASE/kcc "$SH/kcc_asm_backup"
cp $BASE/kcc_selfhost $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup" $BASE/kcc
exit $status
```

□

640b · used in 33d · uses `_main` 621, 629

49. Self-Hosting: A Tokenizer in C

The arena of Chapter 48 was small enough to drop straight back into the compiler. A tokenizer is a larger test: it branches heavily, keeps mutable cursor state, indexes tables, and builds tokens through pointers. Rather than swap it into kcc's own front-end—which would entangle it with the parser and preprocessor—this chapter takes the tokenizer as a self-contained *program*: a faithful C re-implementation of the scanning logic of Chapter 16, using the same token codes, that tokenizes embedded source buffers and asserts the result. kcc compiles it; its own assertions and a differential comparison against `clang` judge it.

Writing it flushed out two real defects in kcc, both triggered only by a function of this size and shape, and both now fixed:

- The code generator's basic-block label table was a fixed 64-slot, open-addressed hash with no resize. A function with more than that many blocks—`lex_next` has well over sixty—overflowed it, and `ht_put` spun forever. The table is now sized to the block count (§35.4 neighbourhood, in `codegen_func`).
- The address-of operator assumed its operand was always a bare identifier. `&tokens[n]`—a subscript—was mis-read as a name and lowered to a bogus global reference. `&` now dispatches on operand shape.

That a 250-line program surfaced two codegen bugs is the point of Stage 8b: real code exercises feature *combinations* the unit tests never reach.

49.1. Token Codes

The codes match Chapter 16 exactly, so a token stream from this C lexer is directly comparable to kcc's own. Only the subset the tests reference is named; the rest share the same numbering.

⟨ *Selfhost: lexer tokens* 641a ⟩ ≡

```
#define TOK_EOF         0
#define TOK_IDENT       1
#define TOK_NUMBER      2
#define TOK_CHAR_LIT    3
#define TOK_STRING_LIT  4
#define TOK_UNKNOWN     5

#define TOK_INT        27
#define TOK_RETURN     31
#define TOK_IF         25
#define TOK_ELSE       19
#define TOK_WHILE      43
#define TOK_FOR        23
#define TOK_VOID       41
#define TOK_CHAR       13
#define TOK_STRUCT     36

#define TOK_LPAREN     60
#define TOK_RPAREN     61
#define TOK_LBRACE     62
#define TOK_RBRACE     63
#define TOK_SEMI       66
#define TOK_STAR       73
#define TOK_PLUS       74
#define TOK_ARROW      69
#define TOK_INC        70
#define TOK_EQ         86
#define TOK_ASSIGN     94
#define TOK_LE         84
#define TOK_ELLIPSIS  107

struct Token {
    int type;
    int start;
    int len;
    int line;
    int col;
};
□
```

641a · used in 643c

49.2. State and the Keyword Table

Like the assembly lexer, cursor state lives in globals. The keyword table is two parallel arrays—names and codes—initialised with brace-enclosed lists. The name array is an array of `char *` seeded with string literals, which is precisely the pointer-array initializer form Stage 8a added; getting it wrong would surface as a mis-classified keyword.

⟨ *Selfhost: lexer state* 641b ⟩ ≡

```
const char *g_src;
int g_pos;
int g_line;
int g_line_start;

#define NKW 9
char *kw_names[NKW] = { "int", "return", "if", "
   else", "while",
                        "for", "void", "char", "
   struct" };
int   kw_codes[NKW] = { TOK_INT, TOK_RETURN,
   TOK_IF, TOK_ELSE, TOK_WHILE,
                        TOK_FOR, TOK_VOID,
   TOK_CHAR, TOK_STRUCT };
□
```

641b · used in 643c

49.3. Character Classes and Keyword Lookup

Character classification is expressed as range tests rather than a lookup table—simpler, and equally within kcc's reach. A keyword is recognised by scanning the parallel arrays: the candidate must match the identifier text and end exactly at its length, so `intx` is an identifier, not `int` followed by `x`.

⟨ *Selfhost: lexer classify* 641c ⟩ ≡

```
int is_alpha(int c) { return (c >= 'a' && c <= 'z'
   ) || (c >= 'A' && c <= 'Z') || c == '_'; }
int is_digit(int c) { return c >= '0' && c <= '9';
    }
int is_alnum(int c) { return is_alpha(c) ||
   is_digit(c); }
int is_space(int c) { return c == ' ' || c == '\t'
    || c == '\n' || c == '\r'; }

int kw_match(const char *kw, const char *s, int
   len) {
    int i = 0;
    while (i < len) {
        if (kw[i] == 0) return 0;
        if (kw[i] != s[i]) return 0;
        i = i + 1;
    }
    return kw[len] == 0;
}

int lookup_kw(const char *s, int len) {
    int i = 0;
    while (i < NKW) {
        if (kw_match(kw_names[i], s, len)) return
   kw_codes[i];
        i = i + 1;
    }
    return TOK_IDENT;
}
```

□

641c · used in 643c

49.4. The Cursor

peek reads the current byte; advance steps forward, tracking line and column by remembering where each line began. skip_trivia consumes whitespace and both comment forms before each token.

⟨ *Selfhost: lexer cursor* 642a ⟩ ≡

```
int peek(void) { return g_src[g_pos]; }

void advance(void) {
    if (g_src[g_pos] == '\n') {
        g_line = g_line + 1;
        g_line_start = g_pos + 1;
    }
    g_pos = g_pos + 1;
}

void skip_trivia(void) {
    while (peek() != 0) {
        int c = peek();
        if (is_space(c)) { advance(); }
        else if (c == '/' && g_src[g_pos + 1] == '
   /') {
            while (peek() != 0 && peek() != '\n')
   advance();
        }
        else if (c == '/' && g_src[g_pos + 1] == '
   *') {
            advance(); advance();
            while (peek() != 0) {
                if (peek() == '*' && g_src[g_pos +
    1] == '/') { advance(); advance(); return; }
                advance();
            }
        }
        else { return; }
    }
}
```

□

642a · used in 643c

49.5. Scanning One Token

lex_next records the start position, line, and column, then classifies by first character: identifier or keyword, pp-number, string, character constant, or a punctuator scanned with maximal munch—three-character ... first, then the two-character operators, then the single-character remainder. The result is written through the caller's Token pointer.

⟨ *Selfhost: lexer next* 642b ⟩ ≡

```
int lex_next(struct Token *t) {
    skip_trivia();
    int start = g_pos;
    int col = start - g_line_start + 1;
    int line = g_line;
    int c = peek();

    t->start = start;
    t->line = line;
    t->col = col;

    if (c == 0) { t->type = TOK_EOF; t->len = 0;
   return TOK_EOF; }

    if (is_alpha(c)) {
        while (is_alnum(peek())) advance();
        int len = g_pos - start;
        t->len = len;
        t->type = lookup_kw(g_src + start, len);
        return t->type;
    }
    if (is_digit(c)) {
        while (is_alnum(peek()) || peek() == '.')
   advance();
        t->len = g_pos - start;
        t->type = TOK_NUMBER;
        return TOK_NUMBER;
    }
    if (c == '"') {
        advance();
        while (peek() != 0 && peek() != '"') {
            if (peek() == '\\') advance();
            advance();
        }
        if (peek() == '"') advance();
        t->len = g_pos - start;
        t->type = TOK_STRING_LIT;
        return TOK_STRING_LIT;
    }
    if (c == '\'') {
        advance();
        while (peek() != 0 && peek() != '\'') {
            if (peek() == '\\') advance();
            advance();
        }
        if (peek() == '\'') advance();
        t->len = g_pos - start;
```

```
        t->type = TOK_CHAR_LIT;
        return TOK_CHAR_LIT;
    }

    int c1 = g_src[g_pos + 1];
    if (c == '.' && c1 == '.' && g_src[g_pos + 2]
== '.') {
        advance(); advance(); advance();
        t->len = 3; t->type = TOK_ELLIPSIS; return
 TOK_ELLIPSIS;
    }
    if (c == '-' && c1 == '>') { advance();
advance(); t->len = 2; t->type = TOK_ARROW;
return TOK_ARROW; }
    if (c == '+' && c1 == '+') { advance();
advance(); t->len = 2; t->type = TOK_INC;
return TOK_INC; }
    if (c == '=' && c1 == '=') { advance();
advance(); t->len = 2; t->type = TOK_EQ; return
 TOK_EQ; }
    if (c == '<' && c1 == '=') { advance();
advance(); t->len = 2; t->type = TOK_LE; return
 TOK_LE; }

    advance();
    t->len = 1;
    if (c == '(') t->type = TOK_LPAREN;
    else if (c == ')') t->type = TOK_RPAREN;
    else if (c == '{') t->type = TOK_LBRACE;
    else if (c == '}') t->type = TOK_RBRACE;
    else if (c == ';') t->type = TOK_SEMI;
    else if (c == '*') t->type = TOK_STAR;
    else if (c == '+') t->type = TOK_PLUS;
    else if (c == '=') t->type = TOK_ASSIGN;
    else t->type = TOK_UNKNOWN;
    return t->type;
}
```

□

642b · used in 643c

49.6. Tokenizing a Buffer

`lex_all` resets the cursor and fills an array of tokens until end of input or the caller's capacity. Its `lex_next(&out[n])` call is the one that exposed the address-of defect: `&out[n]` is the address of a subscript on a pointer parameter.

⟨ *Selfhost: lexer all* 643a ⟩ ≡

```
int lex_all(const char *src, struct Token *out,
   int max) {
    g_src = src; g_pos = 0; g_line = 1;
   g_line_start = 0;
    int n = 0;
    while (n < max) {
        int ty = lex_next(&out[n]);
        n = n + 1;
        if (ty == TOK_EOF) return n;
    }
    return n;
}
```

□

643a · used in 643c

49.7. The Self-Test

`main` tokenizes four representative inputs and checks token counts, types, lengths, and—for the character literal after a newline—line and column tracking. It returns 0 only when every check holds, so the exit status alone certifies the run; a distinct non-zero code per check pinpoints any failure.

⟨ *Selfhost: lexer main* 643b ⟩ ≡

```
int main(void) {
    struct Token toks[64];

    int n = lex_all("int main(void){ return 42; }"
   , toks, 64);
    if (n != 11) return 1;
    if (toks[0].type != TOK_INT) return 2;
    if (toks[1].type != TOK_IDENT) return 3;
    if (toks[2].type != TOK_LPAREN) return 4;
    if (toks[3].type != TOK_VOID) return 5;
    if (toks[4].type != TOK_RPAREN) return 6;
    if (toks[5].type != TOK_LBRACE) return 7;
    if (toks[6].type != TOK_RETURN) return 8;
    if (toks[7].type != TOK_NUMBER) return 9;
    if (toks[7].len != 2) return 10;
    if (toks[8].type != TOK_SEMI) return 11;
    if (toks[10].type != TOK_EOF) return 12;

    n = lex_all("a->b ++ x == y // trailing\n",
   toks, 64);
    if (n != 8) return 13;
    if (toks[1].type != TOK_ARROW) return 14;
    if (toks[3].type != TOK_INC) return 15;
    if (toks[5].type != TOK_EQ) return 16;

    n = lex_all("char *s = \"hi\";\n'x'", toks,
   64);
    if (toks[4].type != TOK_STRING_LIT) return 17;
    if (toks[4].len != 4) return 18;
    int i = 0;
    int found = 0;
    while (i < n) {
        if (toks[i].type == TOK_CHAR_LIT) {
            if (toks[i].line != 2) return 19;
            if (toks[i].col != 1) return 20;
            found = 1;
        }
        i = i + 1;
    }
    if (found == 0) return 21;

    n = lex_all("f(int a, ...) /* skip */ ;", toks
   , 64);
    i = 0; found = 0;
    while (i < n) { if (toks[i].type ==
   TOK_ELLIPSIS) found = 1; i = i + 1; }
    if (found == 0) return 22;

    return 0;
}
```

□

643b · used in 643c

49.8. Assembling the Translation Unit

⟨ *Selfhost: lexer* 643c ⟩ ≡

⟨ *Selfhost: lexer tokens* 641a ⟩
⟨ *Selfhost: lexer state* 641b ⟩
⟨ *Selfhost: lexer classify* 641c ⟩
⟨ *Selfhost: lexer cursor* 642a ⟩
⟨ *Selfhost: lexer next* 642b ⟩
⟨ *Selfhost: lexer all* 643a ⟩
⟨ *Selfhost: lexer main* 643b ⟩

□

643c · used in 33e

49.9. Verification

`make selfhost-lexer` compiles `gen/selfhost/lexer.c` with kcc, links it freestanding (entry `_main`, no libc), and runs it: exit 0 means every assertion held. It then compiles the identical source with `clang` and confirms the two binaries agree—differential testing in miniature, and a preview of Stage 8c. A match means kcc generated correct code for a real, branch-heavy, table-driven program.

⟨ *Selfhost: lexer verify* 644 ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-lexer - compile the C tokenizer
   with kcc, self-test, diff clang.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost

echo "[1/2] compiling + running the tokenizer
   built by kcc"
$BASE/kcc -c "$SH/lexer.c" -o "$SH/lexer_kcc.o" >/
   dev/null
ld "$SH/lexer_kcc.o" -o "$SH/lexer_kcc" -lSystem -
   syslibroot "$SDK" -e _main -arch arm64
set +e
"$SH/lexer_kcc"; kcc_rc=$?
set -e
echo "      kcc-compiled tokenizer self-test exit:
    $kcc_rc (expect 0)"

echo "[2/2] differential check against clang"
clang -c "$SH/lexer.c" -o "$SH/lexer_clang.o"
ld "$SH/lexer_clang.o" -o "$SH/lexer_clang" -
   lSystem -syslibroot "$SDK" -e _main -arch arm64
set +e
"$SH/lexer_clang"; clang_rc=$?
set -e
echo "      clang-compiled tokenizer self-test
   exit: $clang_rc (expect 0)"

if [ "$kcc_rc" -eq 0 ] && [ "$kcc_rc" -eq "
   $clang_rc" ]; then
    echo "PASS: kcc and clang agree, tokenizer
   self-test green"
    exit 0
fi
echo "FAIL: kcc=$kcc_rc clang=$clang_rc"
exit 1
```

□

644 · used in 33f · uses `_main` 621, 629

50. Self-Hosting: The Vector in C

The arena (Chapter 48) proved kcc can compile a piece of its own *storage* layer; this chapter climbs one level up and ports a *data structure*—the growable vector of Chapter 9. The vector is the workhorse container of the whole compiler: the list of IR functions, each function's basic blocks, the symbol-table scopes, the relocation and symbol vectors of the linker, and the back end's patch lists are all `kcc_vecs`. Re-implementing it in C, compiling that C with kcc, and linking the result back in place of the hand-written assembly is therefore a second, independent self-hosting step: where the arena stressed one allocation path, the vector stresses element copying, the geometric-growth reallocation, and pointer arithmetic on a caller-supplied element size—exercised millions of times across the suite.

50.1. Interface and Layout

The C port must be a drop-in replacement, so it exports exactly the three symbols the assembly vector did—`kcc_vec_init`, `kcc_vec_push`, and `kcc_vec_get`—and it reads and writes a header whose fields sit at the very offsets the rest of the compiler already hard-codes: `data` at 0, `count` at 8, `cap` at 16, `elem_size` at 24. Because kcc's all-spill layout gives every scalar struct field its own eight-byte slot, the natural C declaration—a pointer followed by three `unsigned long`s—occupies exactly those offsets, so the C header and the assembly header are the same 32 bytes byte for byte. The port depends only on the two primitives the assembly version used, the arena allocator and `memcpy`, declared here as ordinary `extern` functions the linker resolves across objects.

⟨ *Selfhost: vec preamble* 645a ⟩ ≡

```
struct Vec {
    void *data;
    unsigned long count;
    unsigned long cap;
    unsigned long elem_size;
};

void *kcc_arena_alloc(long size);
void  kcc_memcpy(void *dst, const void *src,
   unsigned long n);
```

□

645a · used in 646a

50.2. Initialization

`kcc_vec_init` arena-allocates the 32-byte header and an initial backing array for sixteen elements, mirroring the assembly routine exactly: the count starts at zero and the caller never specifies a capacity.

⟨ *Selfhost: vec init* 645b ⟩ ≡

```
void *kcc_vec_init(unsigned long elem_size) {
    struct Vec *v = (struct Vec *)kcc_arena_alloc(
   sizeof(struct Vec));
    v->data = kcc_arena_alloc((long)(16 *
   elem_size));
    v->count = 0;
    v->cap = 16;
    v->elem_size = elem_size;
    return v;
}
```

□

645b · used in 646a

50.3. Push

`kcc_vec_push` appends a copy of the caller's element. When the vector is full it doubles the capacity, allocates a fresh array, and copies the live elements across before storing the new one—the geometric growth that keeps a sequence of n pushes amortised linear. Both copies go through the same `memcpy` the assembly used, and all sizes are computed from the header's `elem_size`, so a vector of any element width behaves identically.

⟨ *Selfhost: vec push* 645c ⟩ ≡

```
void kcc_vec_push(void *vec, const void *elem) {
    struct Vec *v = (struct Vec *)vec;
    if (v->count >= v->cap) {
        unsigned long newcap = v->cap * 2;
        void *nd = kcc_arena_alloc((long)(newcap *
    v->elem_size));
        kcc_memcpy(nd, v->data, v->count * v->
   elem_size);
        v->data = nd;
        v->cap = newcap;
    }
    kcc_memcpy((char *)v->data + v->count * v->
   elem_size, elem, v->elem_size);
    v->count = v->count + 1;
}
```

□

645c · used in 646a

50.4. Element Access

`kcc_vec_get` returns a pointer to the element at an index, computed as `data + index * elem_size`, with no bounds checking—the caller guarantees the index is in range, exactly as the assembly contract required.

⟨ *Selfhost: vec get* 645d ⟩ ≡

```
void *kcc_vec_get(void *vec, unsigned long index)
   {
    struct Vec *v = (struct Vec *)vec;
    return (char *)v->data + index * v->elem_size;
}
```

□

645d · used in 646a

50.5. Assembling the Translation Unit

The three fragments compose into one C file, `gen/selfhost/vec.c`, routed by `assembly.weft`.

⟨ *Selfhost: vec* 646a ⟩ ≡

⟨ *Selfhost: vec preamble* 645a ⟩
⟨ *Selfhost: vec init* 645b ⟩
⟨ *Selfhost: vec push* 645c ⟩
⟨ *Selfhost: vec get* 645d ⟩

□

646a · used in 33g

50.6. Verification

`make selfhost-vec` runs the same drop-in loop the arena uses. It compiles `gen/selfhost/vec.c` with the assembly-built kcc, relinks a second compiler binary with the hand-written `vec.o` swapped for the freshly compiled `vec_c.o`, and points the code-generation suite at it.

The passing criterion is again complete: the self-hosted compiler must reproduce every result of the assembly-built one. Because vectors carry the IR function list, every function's basic blocks, and the symbol-table scopes, a green run means the C vector—compiled by kcc—is behaviourally byte-for-byte equivalent to the assembly original, growth path and all.

⟨ *Selfhost: vec verify* 646b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-vec - compile kcc's vector in C
   with kcc, relink, run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/vec.c with
   kcc"
$BASE/kcc -c "$SH/vec.c" -o "$SH/vec_c.o" >/dev/
   null

echo "[2/3] linking $BASE/kcc_selfhost_vec (vec.o
   -> kcc-compiled vec_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "vec" ]; then OBJ="$OBJ $SH/vec_c.
   o"; else OBJ="$OBJ $BASE/src/${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_vec $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted vector"
cp $BASE/kcc "$SH/kcc_asm_backup_vec"
cp $BASE/kcc_selfhost_vec $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_vec" $BASE/kcc
exit $status
```

□

646b · used in 33h · uses `_main` 621, 629

51. Self-Hosting: The String Primitives in C

The arena and the vector proved kcc can compile its own storage and its own workhorse container; this chapter ports the layer beneath both—the five string and memory primitives of Chapter **??**: `kcc_strlen`, `kcc_memcpy`, `kcc_memcmp`, `kcc_memset`, and `kcc_strcmp`. These are the most *pervasive* functions in the compiler. `memcpy` moves every struct copy and every vector reallocation; `memset` clears every zeroed buffer; `strcmp` and `strlen` drive keyword recognition, symbol lookup, and option parsing; `memcmp` compares interned strings. Swapping the hand-written assembly for a C re-implementation compiled by kcc therefore routes essentially every byte the compiler touches through code kcc produced. Because they are leaf routines with no dependencies, the port is self-contained—no `extern` declarations at all.

51.1. The Byte-Exact Contract

A drop-in replacement must be behaviourally identical to the last bit, and the one place that bites is the comparison result. The assembly loads each byte with LDRB, which zero-extends, and returns the difference of the two *unsigned* byte values (`s1[i] - s2[i]`, each in 0–255). A naive C port using plain `char`—which is *signed* on this target—would compare a high-bit byte as negative and return the wrong sign. The port therefore reads bytes through `unsigned char` throughout, so `kcc_strcmp` and `kcc_memcmp` reproduce the assembly's exact signed-`int` result, high bytes and all.

51.2. Length

`kcc_strlen` scans to the terminating NUL and returns the count of bytes before it.

⟨ *Selfhost: strings strlen* 647a ⟩ ≡

```
unsigned long kcc_strlen(const char *s) {
    const char *p = s;
    while (*p) p = p + 1;
    return (unsigned long)(p - s);
}
```

□

647a · used in 648a

51.3. Copy and Fill

`kcc_memcpy` copies `n` bytes front to back (no overlap handling, matching `memcpy` rather than `memmove`) and returns the destination; `kcc_memset` writes the low eight bits of `c` into `n` bytes and returns its buffer.

⟨ *Selfhost: strings memcpy* 647b ⟩ ≡

```
void *kcc_memcpy(void *dst, const void *src,
   unsigned long n) {
    unsigned char *d = (unsigned char *)dst;
    const unsigned char *s = (const unsigned char
   *)src;
    unsigned long i = 0;
    while (i < n) { d[i] = s[i]; i = i + 1; }
    return dst;
}
```

□

647b · used in 648a

⟨ *Selfhost: strings memset* 647c ⟩ ≡

```
void *kcc_memset(void *s, int c, unsigned long n)
   {
    unsigned char *p = (unsigned char *)s;
    unsigned char b = (unsigned char)c;
    unsigned long i = 0;
    while (i < n) { p[i] = b; i = i + 1; }
    return s;
}
```

□

647c · used in 648a

51.4. Comparison

Both comparisons read bytes as `unsigned char` and return the signed difference of the first differing pair, zero when the ranges (or strings) match. `kcc_memcmp` runs to a fixed length; `kcc_strcmp` stops at the first NUL common to both.

⟨ *Selfhost: strings memcmp* 647d ⟩ ≡

```
int kcc_memcmp(const void *s1, const void *s2,
   unsigned long n) {
    const unsigned char *a = (const unsigned char
   *)s1;
    const unsigned char *b = (const unsigned char
   *)s2;
    unsigned long i = 0;
    while (i < n) {
        if (a[i] != b[i]) return (int)a[i] - (int)
   b[i];
        i = i + 1;
    }
    return 0;
}
```

□

647d · used in 648a

⟨ *Selfhost: strings strcmp* 647e ⟩ ≡

```
int kcc_strcmp(const char *s1, const char *s2) {
    const unsigned char *a = (const unsigned char
   *)s1;
    const unsigned char *b = (const unsigned char
   *)s2;
    while (*a == *b) {
        if (*a == 0) return 0;
        a = a + 1;
        b = b + 1;
    }
```

```
    return (int)*a - (int)*b;
}
```

□

647e · used in 648a

51.5. Assembling the Translation Unit

The five fragments compose into `gen/selfhost/strings.c`, routed by `assembly.weft`.

⟨ *Selfhost: strings* 648a ⟩ ≡

⟨ *Selfhost: strings strlen* 647a ⟩
⟨ *Selfhost: strings memcpy* 647b ⟩
⟨ *Selfhost: strings memset* 647c ⟩
⟨ *Selfhost: strings memcmp* 647d ⟩
⟨ *Selfhost: strings strcmp* 647e ⟩

□

648a · used in 33i

51.6. Verification

`make selfhost-strings` runs the drop-in loop: compile `gen/selfhost/strings.c` with the assembly-built kcc, relink a second compiler with the hand-written `strings.o` replaced by the freshly compiled `strings_c.o`, and re-run the code-generation suite through it. Because these primitives underlie every allocation, copy, and name comparison in the compiler, a green run is the strongest of the self-hosting checks: it means kcc compiled the very memory operations it then used to compile the whole suite, byte for byte.

⟨ *Selfhost: strings verify* 648b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-strings - compile kcc's string
   primitives in C with kcc,
# relink, run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/strings.c
   with kcc"
$BASE/kcc -c "$SH/strings.c" -o "$SH/strings_c.o"
   >/dev/null

echo "[2/3] linking $BASE/kcc_selfhost_strings (
   strings.o -> kcc-compiled strings_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "strings" ]; then OBJ="$OBJ $SH/
   strings_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
    fi
done
ld -o $BASE/kcc_selfhost_strings $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted string primitives"
cp $BASE/kcc "$SH/kcc_asm_backup_strings"
cp $BASE/kcc_selfhost_strings $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_strings" $BASE/kcc
exit $status
```

□

648b · used in 33j · uses _main 621, 629

52. Self-Hosting: The Hash Table in C

The arena, the vector, and the string primitives ported kcc's foundation; this chapter ports the associative structure built on top of it—the open-addressing hash table of Chapter 9, together with its FNV-1a hash. The table is how kcc answers "have I seen this name?": the keyword recogniser, the string interner, every lexical scope's symbol map, and the per-function local and label tables are all `kcc_hts`. It is the first ported subsystem with real *algorithmic* content—a hash, a probe sequence, and a rehashing growth step—so getting it byte-compatible exercises the compiler on loops, struct arrays, and 64-bit unsigned arithmetic, all at once.

52.1. The Byte-Exact Contract

A hash table is only a drop-in replacement if it places every key in the same slot the assembly did; otherwise a later pass that walks the table in slot order would see a different sequence and the output would drift. The port therefore reproduces the algorithm exactly: the same FNV-1a constants, the same `hash & (cap-1)` starting index, the same linear probe with power-of-two wrap, and the same growth trigger—double the capacity once inserting would push the load factor past three-quarters, tested as `(count+1)*4 > cap*3`. The two 32-byte and 24-byte layouts match the assembly field for field, so the header's `cap`/`count` and each entry's `hash`/`key`/`keylen`/`value` sit where the rest of the compiler expects them. The port depends on the arena, `memset`, and `memcmp`, declared as `extern`.

⟨ *Selfhost: hashtab preamble* 649a ⟩ ≡

```
struct HTEntry {
    unsigned long hash;
    const void *key;
    unsigned long keylen;
    unsigned long value;
};

struct HT {
    struct HTEntry *entries;
    unsigned long cap;
    unsigned long count;
};

void *kcc_arena_alloc(long size);
void *kcc_memset(void *s, int c, unsigned long n);
int   kcc_memcmp(const void *s1, const void *s2,
   unsigned long n);
```

□

649a · used in 650c

52.2. FNV-1a

The 64-bit FNV-1a hash folds each byte into a running accumulator: exclusive-or the byte, then multiply by the prime, starting from the offset basis. The constants are the canonical ones, and the multiply wraps modulo 2^{64} as unsigned arithmetic—exactly what the assembly's `mul` does.

⟨ *Selfhost: hashtab fnv1a* 649b ⟩ ≡

```
unsigned long kcc_fnv1a(const void *data, unsigned
    long len) {
    const unsigned char *p = (const unsigned char
   *)data;
    unsigned long h = 0xCBF29CE484222325UL;
    unsigned long i = 0;
    while (i < len) {
        h = h ^ (unsigned long)p[i];
        h = h * 0x00000100000001B3UL;
        i = i + 1;
    }
    return h;
}
```

□

649b · used in 650c

52.3. Initialization

`kcc_ht_init` arena-allocates the 24-byte header and a capacity-slot entry array, zero-filling the latter so every key pointer starts `NULL` (an empty slot). The capacity must be a power of two, so the mask `cap-1` selects the low bits of a hash.

⟨ *Selfhost: hashtab init* 649c ⟩ ≡

```
void *kcc_ht_init(unsigned long capacity) {
    struct HT *h = (struct HT *)kcc_arena_alloc((
   long)sizeof(struct HT));
    unsigned long bytes = capacity * (unsigned
   long)sizeof(struct HTEntry);
    h->entries = (struct HTEntry *)kcc_arena_alloc
   ((long)bytes);
    kcc_memset(h->entries, 0, bytes);
    h->cap = capacity;
    h->count = 0;
    return h;
}
```

□

649c · used in 650c

52.4. Lookup

`kcc_ht_get` hashes the key, starts at `hash & mask`, and probes forward. An empty slot means the key is absent, so it returns 0 (a valid "not found" sentinel, since the table's stored values—token types, symbol pointers—are never zero where a caller distinguishes them). A slot matches only when the cached hash, the length, and the bytes all agree, the cheap checks gating the `memcmp`.

⟨ *Selfhost: hashtab get* 649d ⟩ ≡

```
unsigned long kcc_ht_get(void *ht, const void *key
   , unsigned long keylen) {
    struct HT *h = (struct HT *)ht;
    unsigned long hash = kcc_fnv1a(key, keylen);
    unsigned long mask = h->cap - 1;
    unsigned long i = hash & mask;
    while (1) {
        struct HTEntry *e = &h->entries[i];
        if (e->key == 0) return 0;
        if (e->hash == hash && e->keylen == keylen
    &&
            kcc_memcmp(e->key, key, keylen) == 0)
            return e->value;
        i = (i + 1) & mask;
    }
}
```

□

649d · used in 650c

52.5. Growth

kcc_ht_grow doubles the capacity and rehashes. Because the existing keys are already unique, each is reinserted by probing the new array for its first empty slot from hash & new_mask—no equality check is needed. The entry is copied field by field, matching the assembly and sidestepping any question of struct-assignment lowering. Growth updates the header's pointer and capacity in place, so every outstanding handle stays valid.

⟨ *Selfhost: hashtab grow* 650a ⟩ ≡

```
void kcc_ht_grow(void *ht) {
    struct HT *h = (struct HT *)ht;
    struct HTEntry *old = h->entries;
    unsigned long oldcap = h->cap;
    unsigned long newcap = oldcap * 2;
    unsigned long bytes = newcap * (unsigned long)
   sizeof(struct HTEntry);
    struct HTEntry *ne = (struct HTEntry *)
   kcc_arena_alloc((long)bytes);
    kcc_memset(ne, 0, bytes);
    unsigned long mask = newcap - 1;
    unsigned long j = 0;
    while (j < oldcap) {
        if (old[j].key != 0) {
            unsigned long i = old[j].hash & mask;
            while (ne[i].key != 0) i = (i + 1) &
   mask;
            ne[i].hash = old[j].hash;
            ne[i].key = old[j].key;
            ne[i].keylen = old[j].keylen;
            ne[i].value = old[j].value;
        }
        j = j + 1;
    }
    h->entries = ne;
    h->cap = newcap;
}
```

□

650a · used in 650c

52.6. Insertion

kcc_ht_put grows first if this insertion would tip the load factor past three-quarters, then probes: an empty slot receives the entry (and bumps the count), while a slot whose key already matches has its value overwritten. The grow-first order keeps the pointer and capacity it reads afterwards current.

⟨ *Selfhost: hashtab put* 650b ⟩ ≡

```
void kcc_ht_put(void *ht, const void *key,
   unsigned long keylen,
                unsigned long value) {
    struct HT *h = (struct HT *)ht;
    if ((h->count + 1) * 4 > h->cap * 3)
   kcc_ht_grow(h);
    unsigned long hash = kcc_fnv1a(key, keylen);
    unsigned long mask = h->cap - 1;
    unsigned long i = hash & mask;
    while (1) {
        struct HTEntry *e = &h->entries[i];
        if (e->key == 0) {
            e->hash = hash;
            e->key = key;
            e->keylen = keylen;
            e->value = value;
            h->count = h->count + 1;
            return;
        }
        if (e->hash == hash && e->keylen == keylen
    &&
            kcc_memcmp(e->key, key, keylen) == 0)
   {
            e->value = value;
            return;
        }
        i = (i + 1) & mask;
    }
}
```

□

650b · used in 650c

52.7. Assembling the Translation Unit

The fragments compose into gen/selfhost/hashtab.c, routed by assembly.weft. It supplies the whole hashtab.s object—the hash and all four table routines—so the drop-in swap is complete.

⟨ *Selfhost: hashtab* 650c ⟩ ≡

```
⟨ Selfhost: hashtab preamble 649a ⟩
⟨ Selfhost: hashtab fnv1a 649b ⟩
⟨ Selfhost: hashtab init 649c ⟩
⟨ Selfhost: hashtab get 649d ⟩
⟨ Selfhost: hashtab grow 650a ⟩
⟨ Selfhost: hashtab put 650b ⟩
```

□

650c · used in 34a

52.8. Verification

make selfhost-hashtab runs the drop-in loop: compile gen/selfhost/hashtab.c with the assembly-built kcc,

relink a second compiler with the hand-written `hashtab.o` replaced by the kcc-compiled `hashtab_c.o`, and re-run the code-generation suite. Because every name the compiler resolves—keywords, identifiers, symbols, labels—passes through this table, a green run means kcc compiled its own associative core correctly, hash and rehash and all.

⟨ *Selfhost: hashtab verify* 651 ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-hashtab - compile kcc's hash table
    in C with kcc, relink,
# run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/hashtab.c
   with kcc"
$BASE/kcc -c "$SH/hashtab.c" -o "$SH/hashtab_c.o"
   >/dev/null

echo "[2/3] linking $BASE/kcc_selfhost_hashtab (
   hashtab.o -> kcc-compiled hashtab_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "hashtab" ]; then OBJ="$OBJ $SH/
   hashtab_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
    fi
done
ld -o $BASE/kcc_selfhost_hashtab $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted hash table"
cp $BASE/kcc "$SH/kcc_asm_backup_hashtab"
cp $BASE/kcc_selfhost_hashtab $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_hashtab" $BASE/kcc
exit $status
```

□

651 · used in 34b · uses _main 621, 629

53. Self-Hosting: The String Interner in C

With the arena, the vector, the string primitives, and the hash table all ported, the natural next piece is the one that sits directly on top of the last two: the string interner of Chapter 9. Interning is how kcc guarantees that each distinct identifier or keyword text is stored exactly once, so two names can be compared by pointer instead of byte by byte—the property the symbol table relies on. The interner is a thin layer, just two routines and a global handle, but it is a satisfying rung because *every* function it calls is now itself C kcc compiled: the hash table (`kcc_ht_init/get/put`), the arena (`kcc_arena_alloc`), and `kcc_memcpy`. Porting it means a name lookup runs entirely through code the compiler produced.

53.1. State and Contract

The interner keeps one global, `kcc_strtab`, a pointer to its backing hash table, zero until `kcc_strtab_init` creates a 256-slot table. As a drop-in that global keeps the exact name and zero-initialised layout the assembly used. `kcc_strtab_intern` takes a string and length (the text need not be NUL-terminated): if the hash table already holds it, the stored pointer—which is the canonical copy—comes straight back; otherwise the routine allocates `len+1` bytes from the arena, copies the text, NUL-terminates it, records it in the table keyed by its own bytes with itself as the value, and returns it. Storing the copy as both key and value is what lets a later lookup return the canonical pointer.

⟨ *Selfhost: strtab* 653a ⟩ ≡

```
void *kcc_ht_init(unsigned long capacity);
unsigned long kcc_ht_get(void *ht, const void *key
   , unsigned long keylen);
void kcc_ht_put(void *ht, const void *key,
   unsigned long keylen,
                unsigned long value);
void *kcc_arena_alloc(long size);
void *kcc_memcpy(void *dst, const void *src,
   unsigned long n);

void *kcc_strtab;

void kcc_strtab_init(void) {
    kcc_strtab = kcc_ht_init(256);
}

const char *kcc_strtab_intern(const char *s,
   unsigned long len) {
    unsigned long found = kcc_ht_get(kcc_strtab, s
   , len);
    if (found) return (const char *)found;
    char *copy = (char *)kcc_arena_alloc((long)(
   len + 1));
    kcc_memcpy(copy, s, len);
    copy[len] = 0;
    kcc_ht_put(kcc_strtab, copy, len, (unsigned
   long)copy);
    return copy;
}
```

□

653a · used in 34c

53.2. Verification

`make selfhost-strtab` runs the drop-in loop: compile `gen/selfhost/strtab.c` with the assembly-built kcc, relink a second compiler with the hand-written `strtab.o` replaced by the kcc-compiled `strtab_c.o`, and re-run the code-generation suite. Because every identifier and keyword the lexer sees is interned through this layer, a green run means kcc compiled its own name-canonicalisation correctly.

⟨ *Selfhost: strtab verify* 653b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-strtab - compile kcc's string
   interner in C with kcc, relink,
# run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/strtab.c with
    kcc"
$BASE/kcc -c "$SH/strtab.c" -o "$SH/strtab_c.o" >/
   dev/null

echo "[2/3] linking $BASE/kcc_selfhost_strtab (
   strtab.o -> kcc-compiled strtab_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "strtab" ]; then OBJ="$OBJ $SH/
   strtab_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
   fi
done
ld -o $BASE/kcc_selfhost_strtab $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted interner"
cp $BASE/kcc "$SH/kcc_asm_backup_strtab"
cp $BASE/kcc_selfhost_strtab $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_strtab" $BASE/kcc
exit $status
```

□

653b · used in 34d · uses _main 621, 629

54. Self-Hosting: Buffered Output in C

The last of the foundational subsystems to port is the one every other layer ultimately writes through: the buffered output of Chapter 11. Every diagnostic, every `--dump` listing, and the assembly the `.s` back end emits pass through `kcc_io_puts`, `kcc_io_write`, and their siblings. Unlike the earlier ports this one owns real *mutable global state*—a 4 KiB buffer, a write cursor, and a file descriptor—so it is also a test that kcc initialises a non-`main` unit's file-scope globals correctly: `kcc_io_fd` starts at 1 (stdout) and `kcc_io_cap` at 4096, values a module initialiser must set before the first byte is written.

54.1. State and the Flush Contract

The module keeps four globals under the exact names and initial values the assembly used: the destination `kcc_io_fd` (1, stdout), the current `kcc_io_pos` (0), the capacity `kcc_io_cap` (4096), and the 4 KiB `kcc_io_buf` itself. `kcc_io_flush` writes the buffered prefix through the `kcc_sys_write` syscall wrapper and resets the cursor; the two syscall-facing routines (`flush` and the unbuffered `kcc_io_err`) are the module's only dependencies, declared `extern` along with `kcc_strlen`.

The one behavioural subtlety a faithful port must preserve is *when* the flush happens: `kcc_io_putc` flushes *before* storing when the buffer is already full, whereas `kcc_io_puts` and `kcc_io_write` store first and flush once the cursor reaches the end. Both avoid overrun, but the resulting flush boundaries differ, so the C reproduces each shape exactly.

⟨ *Selfhost: io preamble* 655a ⟩ ≡

```
long kcc_sys_write(long fd, const void *buf,
   unsigned long n);
unsigned long kcc_strlen(const char *s);

long kcc_io_fd = 1;
unsigned long kcc_io_pos = 0;
unsigned long kcc_io_cap = 4096;
char kcc_io_buf[4096];
```

□

655a · used in 656b

54.2. Flush and Put

⟨ *Selfhost: io flush* 655b ⟩ ≡

```
void kcc_io_flush(void) {
    if (kcc_io_pos == 0) return;
    kcc_sys_write(kcc_io_fd, kcc_io_buf,
   kcc_io_pos);
    kcc_io_pos = 0;
}
```

□

655b · used in 656b

⟨ *Selfhost: io putc* 655c ⟩ ≡

```
void kcc_io_putc(int c) {
    if (kcc_io_pos >= kcc_io_cap) kcc_io_flush();
    kcc_io_buf[kcc_io_pos] = (char)c;
    kcc_io_pos = kcc_io_pos + 1;
}
```

□

655c · used in 656b

⟨ *Selfhost: io puts* 655d ⟩ ≡

```
void kcc_io_puts(const char *s) {
    while (*s) {
        kcc_io_buf[kcc_io_pos] = *s;
        kcc_io_pos = kcc_io_pos + 1;
        if (kcc_io_pos >= 4096) kcc_io_flush();
        s = s + 1;
    }
}
```

□

655d · used in 656b

54.3. Write, Format, and Error

`kcc_io_write` emits an explicit byte count (for token text, which is a pointer/length pair rather than a NUL-terminated string). `kcc_io_putu` renders an unsigned integer by taking decimal digits least-significant first into a small stack buffer and writing them in order. `kcc_io_err` sends a string straight to stderr through the syscall, unbuffered, so a diagnostic survives an immediate crash.

⟨ *Selfhost: io write* 655e ⟩ ≡

```
void kcc_io_write(const void *ptr, unsigned long
   len) {
    const char *p = (const char *)ptr;
    unsigned long i = 0;
    while (i < len) {
        kcc_io_buf[kcc_io_pos] = p[i];
        kcc_io_pos = kcc_io_pos + 1;
        if (kcc_io_pos >= 4096) kcc_io_flush();
        i = i + 1;
    }
}
```

□

655e · used in 656b

⟨ *Selfhost: io putu* 655f ⟩ ≡

```
void kcc_io_putu(unsigned long n) {
    char buf[24];
    int i = 24;
    if (n == 0) {
        i = i - 1;
        buf[i] = '0';
    } else {
        while (n) {
            i = i - 1;
            buf[i] = (char)('0' + (n % 10));
            n = n / 10;
```

```
        }
    }
    kcc_io_write(&buf[i], (unsigned long)(24 - i))
   ;
}
```

□

655f · used in 656b

⟨ *Selfhost: io err* 656a ⟩ ≡

```
void kcc_io_err(const char *s) {
    unsigned long len = kcc_strlen(s);
    kcc_sys_write(2, s, len);
}
```

□

656a · used in 656b

54.4. Assembling the Translation Unit

⟨ *Selfhost: io* 656b ⟩ ≡

```
⟨ Selfhost: io preamble 655a ⟩
⟨ Selfhost: io flush 655b ⟩
⟨ Selfhost: io putc 655c ⟩
⟨ Selfhost: io puts 655d ⟩
⟨ Selfhost: io write 655e ⟩
⟨ Selfhost: io putu 655f ⟩
⟨ Selfhost: io err 656a ⟩
```

□

656b · used in 34e

54.5. Verification

`make selfhost-io` runs the drop-in loop: compile `gen/selfhost/io.c` with the assembly-built kcc, relink a second compiler with the hand-written `io.o` replaced by the kcc-compiled `io_c.o`, and re-run the code-generation suite. Because the suite's harness reads the exit status rather than the text, and because the compiler itself writes nothing on a clean run, this port is also implicitly checked by every other verification that now flows through the C buffer.

⟨ *Selfhost: io verify* 656c ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-io - compile kcc's buffered output
    in C with kcc, relink,
# run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/io.c with kcc
   "
$BASE/kcc -c "$SH/io.c" -o "$SH/io_c.o" >/dev/null

echo "[2/3] linking $BASE/kcc_selfhost_io (io.o ->
    kcc-compiled io_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "io" ]; then OBJ="$OBJ $SH/io_c.o"
   ; else OBJ="$OBJ $BASE/src/${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_io $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted output layer"
cp $BASE/kcc "$SH/kcc_asm_backup_io"
cp $BASE/kcc_selfhost_io $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_io" $BASE/kcc
exit $status
```

□

656c · used in 34f · uses _main 621, 629

55. Self-Hosting: The Source Manager in C

With the foundation ported, the next subsystem up is the one that produces the bytes everything else consumes: the source manager of Chapter **??**, together with translation phases 1 and 2 (Chapters 14 and 15). Its three routines map a file into memory, replace trigraphs, and splice backslash–newline line continuations—the first bytes of every compilation. It is the first port to touch the operating system directly (through the `kcc_sys_*` wrappers, which stay in assembly because they are raw `svc`s) and the first whose global state—the mapped buffer and its size—is read by *other* modules, so it keeps those symbols exactly.

55.1. Opening the File

`kcc_source_open` records the path, opens it read-only, measures it with `lseek(SEEK_END)`, rejects the empty file, and maps it `PROT_READ/MAP_PRIVATE`; a `-1` from any syscall prints the corresponding message to stderr and exits. The mapped base and byte size land in the exported globals the lexer and the diagnostic engine read. The syscall flag values are the numeric constants the assembly used (O_RDONLY 0, SEEK_END 2, PROT_READ 1, MAP_PRIVATE 2).

The path `-` is a special case for standard input: a pipe can be neither `lseek`'d nor `mmap`'d, so the whole of `fd 0` is read into a growing arena buffer instead. gcc-driven build systems pipe a snippet to the compiler this way for their capability probes.

⟨ *Selfhost: source preamble* 657a ⟩ ≡

```
long kcc_sys_open(const char *path, int flags, int
    mode);
long kcc_sys_read(int fd, void *buf, unsigned long
    len);
long kcc_sys_close(int fd);
void *kcc_sys_mmap(void *addr, unsigned long len,
   int prot, int flags,
                   int fd, long off);
long kcc_sys_lseek(int fd, long offset, int whence
   );
void kcc_sys_exit(int status);
void kcc_io_err(const char *s);
void *kcc_arena_alloc(long size);

void *kcc_src_base = 0;
unsigned long kcc_src_size = 0;
const char *kcc_src_path = 0;
extern unsigned long kcc_lex_asm;     /* .S/.s
   input: skip phase-3 comment removal */
```

□

657a · used in 659a

⟨ *Selfhost: source open* 657b ⟩ ≡

```
void kcc_source_open(const char *path) {
    kcc_src_path = path;
    if (path[0] == '-' && path[1] == 0) {
        /* "-" means standard input.  A pipe
   cannot be lseek'd or mmap'd, so read
           it in full into a growing arena buffer.
     gcc-driven build systems pipe
           a snippet to the compiler this way (
   Kbuild's scripts/cc-version.sh). */
        unsigned long cap = 65536, len = 0;
        char *buf = (char *)kcc_arena_alloc((long)
   cap);
        for (;;) {
            if (len == cap) {
                unsigned long ncap = cap * 2;
                char *nb = (char *)kcc_arena_alloc
   ((long)ncap);
                for (unsigned long i = 0; i < len;
    i++) nb[i] = buf[i];
                buf = nb; cap = ncap;
            }
            long n = kcc_sys_read(0, buf + len,
   cap - len);
            if (n <= 0) break;
            len += (unsigned long)n;
        }
        kcc_src_base = buf;
        kcc_src_size = len;
        return;
    }
    long fd = kcc_sys_open(path, 0, 0);
    if (fd == -1) {
        kcc_io_err("kcc: error: cannot open file:
   ");
        kcc_io_err(path);
        kcc_io_err("\n");
        kcc_sys_exit(1);
    }
    long size = kcc_sys_lseek((int)fd, 0, 2);
    if (size == -1) {
        kcc_sys_close((int)fd);
        kcc_io_err("kcc: error: cannot read file\n
   ");
        kcc_sys_exit(1);
    }
    if (size == 0) {
        kcc_sys_close((int)fd);
        kcc_io_err("kcc: error: source file is
   empty\n");
        kcc_sys_exit(1);
    }
    void *base = kcc_sys_mmap(0, (unsigned long)
   size, 1, 2, (int)fd, 0);
    if ((long)base == -1) {
        kcc_sys_close((int)fd);
        kcc_io_err("kcc: error: cannot map file\n"
   );
        kcc_sys_exit(1);
    }
    kcc_src_base = base;
    kcc_src_size = (unsigned long)size;
    kcc_sys_close((int)fd);
}
```

□

657b · used in 659a

55.2. Phase 1: Trigraphs

Because the mapping is read-only, `kcc_source_encode` copies the source into a writable arena buffer, replacing each of the nine `??X` trigraphs with its single character as it goes (advancing the reader by three) and copying any other byte verbatim (advancing by one). A lone or unmatched `?` is just copied—the same fall-through the assembly takes. The new base and its shorter length replace the globals, readying the buffer for in-place splicing.

⟨ *Selfhost: source encode* 658a ⟩ ≡

```
void kcc_source_encode(void) {
    const unsigned char *r = (const unsigned char
   *)kcc_src_base;
    const unsigned char *end = r + kcc_src_size;
    unsigned char *w0 = (unsigned char *)
   kcc_arena_alloc((long)kcc_src_size);
    unsigned char *w = w0;
    while (r < end) {
        if (r[0] == '?' && (r + 2) < end && r[1]
   == '?') {
            unsigned char c = r[2];
            unsigned char rep = 0;
            if (c == '=') rep = '#';
            else if (c == '(') rep = '[';
            else if (c == '/') rep = '\\';
            else if (c == ')') rep = ']';
            else if (c == '\'') rep = '^';
            else if (c == '<') rep = '{';
            else if (c == '!') rep = '|';
            else if (c == '>') rep = '}';
            else if (c == '-') rep = '~';
            if (rep) {
                *w = rep;
                w = w + 1;
                r = r + 3;
                continue;
            }
        }
        *w = *r;
        w = w + 1;
        r = r + 1;
    }
    kcc_src_base = w0;
    kcc_src_size = (unsigned long)(w - w0);
}
```

□

658a · used in 659a

55.3. Phases 2 and 3: Line Splicing and Comment Removal

`kcc_source_splice` rewrites the (now writable) buffer in place. It performs §5.1.1.2 phase 2—dropping every backslash immediately followed by a newline—and phase 3, replacing each comment with a single space. Doing comment removal *here*, up front, rather than in the lexer is what keeps a comment inside a macro argument from surviving into the expansion: a macro argument is recorded as a raw span of this buffer, so a comment left in the text would resurface when the argument is stringised (#), pasted (##), or forwarded through `__VA_ARGS__`—the kernel's `dev_err(dev, fmt, (int) /* note */ 0)` would otherwise carry the `/* note */` into the call and parse its words as code. A block comment keeps the newlines it spans so downstream line numbers stay accurate; a string or character literal is copied verbatim so a `/*` or `//` inside it is not mistaken for a comment. The surviving length updates `kcc_src_size`.

The lexer still strips comments as a fallback (the built-in headers are lexed without passing through here), and its `kcc_lex_no_comments` guard still matters: a `//` *formed* by token pasting during expansion never reaches this phase-3 pass, so only that guard keeps it from being read as a comment.

⟨ *Selfhost: source splice* 658b ⟩ ≡

```
void kcc_source_splice(void) {
    unsigned char *base = (unsigned char *)
   kcc_src_base;
    unsigned char *end = base + kcc_src_size;
    unsigned char *w = base;
    unsigned char *r = base;
    /* Phase 3 (comment removal) runs for C only.
    A `.S'/`.s' input treats `#',
       `;', and bare `'' very differently---
   head_64.S's `# don''t loop' carries an
       apostrophe in an assembler comment---so
   tracking string/char literals here
       would corrupt it; there the lexer's asm-
   aware pass owns comments. */
    int strip = !kcc_lex_asm;
    while (r < end) {
        unsigned char c = *r;
        if (c == '\\' && (r + 1) < end && r[1] ==
   0x0A) {   /* phase 2 */
            r = r + 2;
            continue;
        }
        if (strip && (c == '"' || c == '\'')) {
            /* literal: verbatim */
            unsigned char q = c;
            *w++ = *r++;
            while (r < end) {
                unsigned char d = *r;
                if (d == '\\' && (r + 1) < end &&
   r[1] == 0x0A) { r += 2; continue; }
                if (d == '\\' && (r + 1) < end) {
   *w++ = *r++; *w++ = *r++; continue; }
                *w++ = *r++;
                if (d == q) break;
            }
            continue;
        }
        if (strip && c == '/' && (r + 1) < end &&
   r[1] == '/') {   /* line comment */
            *w++ = ' ';
            r += 2;
            while (r < end) {
                if (*r == '\\' && (r + 1) < end &&
    r[1] == 0x0A) { r += 2; continue; }
                if (*r == 0x0A) break;
                r++;
            }
            continue;
        }
        if (strip && c == '/' && (r + 1) < end &&
   r[1] == '*') {   /* block comment */
            *w++ = ' ';
            r += 2;
            while (r < end) {
                /* A backslash-newline is a phase
   -2 splice even inside the comment:
```

```
                    a multi-line block comment in a
    macro body joins its lines with
                    `\', and keeping the raw
    newline would end the #define early. */
                if (*r == '\\' && (r + 1) < end &&
     r[1] == 0x0A) { r += 2; continue; }
                if (*r == '*' && (r + 1) < end &&
    r[1] == '/') { r += 2; break; }
                if (*r == 0x0A) *w++ = 0x0A;
             /* keep line count */
                r++;
            }
            continue;
        }
        *w = *r;
        w = w + 1;
        r = r + 1;
    }
    kcc_src_size = (unsigned long)(w - base);
}
```

□

658b · used in 659a

55.4. Assembling the Translation Unit

⟨ *Selfhost: source* 659a ⟩ ≡

⟨ *Selfhost: source preamble* 657a ⟩
⟨ *Selfhost: source open* 657b ⟩
⟨ *Selfhost: source encode* 658a ⟩
⟨ *Selfhost: source splice* 658b ⟩

□

659a · used in 34g

55.5. Verification

`make selfhost-source` runs the drop-in loop. Because every byte the compiler parses first passes through this buffer—mapped, trigraph-translated, and spliced—a green code-generation suite means kcc compiled its own front door correctly.

⟨ *Selfhost: source verify* 659b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-source - compile kcc's source
   manager in C with kcc, relink,
# run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/source.c with
    kcc"
$BASE/kcc -c "$SH/source.c" -o "$SH/source_c.o" >/
   dev/null
echo "[2/3] linking $BASE/kcc_selfhost_source (
   source.o -> kcc-compiled source_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "source" ]; then OBJ="$OBJ $SH/
   source_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
   fi
done
ld -o $BASE/kcc_selfhost_source $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted source manager"
cp $BASE/kcc "$SH/kcc_asm_backup_source"
cp $BASE/kcc_selfhost_source $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_source" $BASE/kcc
exit $status
```

□

659b · used in 34h · uses _main 621, 629

56. Self-Hosting: The Diagnostic Engine in C

The diagnostic engine of Chapter 12 is the first ported subsystem whose *output* is itself under test. It formats every error, warning, and note the compiler emits—the `file:line:col: severity: message` line, the offending source line, and a caret under the column—and the `diagtest` and `warntest` suites check those bytes exactly. A byte-compatible C port therefore has a second, independent verifier beyond the code-generation suite: it must reproduce not just correct behaviour on valid programs but the precise presentation of every rejection.

56.1. State and Dependencies

The engine owns two counters and one policy flag under their exact names: `kcc_diag_error_count`, `kcc_diag_warn_count`, and the exported `kcc_warn_all` (the `-Wall` switch the driver sets and semantic analysis reads). It writes through `kcc_io_err` and the raw `kcc_sys_write`, and it reads the live source buffer (`kcc_src_base`/`kcc_src_size`/`kcc_src_path`) that the now-C source manager exports—so a diagnostic renders the true offending line. Two file-local helpers stay internal: `diag_putu` prints a decimal to stderr, and `diag_caret` draws the snippet.

⟨ *Selfhost: diag preamble* 661a ⟩ ≡

```
long kcc_sys_write(long fd, const void *buf,
   unsigned long n);
void kcc_sys_exit(int status);
void kcc_io_err(const char *s);

extern void *kcc_src_base;
extern unsigned long kcc_src_size;
extern const char *kcc_src_path;

unsigned long kcc_diag_error_count = 0;
unsigned long kcc_diag_warn_count = 0;
unsigned long kcc_warn_all = 0;

static void diag_putu(unsigned long n) {
    char buf[24];
    int i = 24;
    if (n == 0) {
        i = i - 1;
        buf[i] = '0';
    } else {
        while (n) {
            i = i - 1;
            buf[i] = (char)('0' + (n % 10));
            n = n / 10;
        }
    }
    kcc_sys_write(2, &buf[i], (unsigned long)(24 -
    i));
}
```

□

661a · used in 662b

56.2. The Caret

`diag_caret` walks the source to the target line, writes it, then assembles the caret line: `col−1` indent characters (a tab copied from the source where the source has one, so the terminal's tab stops keep the caret aligned; a space otherwise), clamped to 500, then the caret and a newline. It is unfailing—a null buffer, a zero line, or a line past the end simply suppresses the snippet.

⟨ *Selfhost: diag caret* 661b ⟩ ≡

```
static void diag_caret(unsigned long line,
   unsigned long col) {
    unsigned char *base = (unsigned char *)
   kcc_src_base;
    if (base == 0) return;
    if (line == 0) return;
    unsigned char *end = base + kcc_src_size;
    unsigned char *p = base;
    unsigned long cur = 1;
    while (cur < line) {
        if (p >= end) return;
        unsigned char c = *p;
        p = p + 1;
        if (c == 10) cur = cur + 1;
    }
    unsigned char *lstart = p;
    unsigned char *lend = p;
    while (lend < end && *lend != 10) lend = lend
   + 1;
    kcc_sys_write(2, lstart, (unsigned long)(lend
   - lstart));
    kcc_io_err("\n");

    char buf[512];
    unsigned long k = 0;
    unsigned long indent = (col > 1) ? (col - 1) :
    0;
    unsigned char *sc = lstart;
    while (indent > 0 && k < 500) {
        char ch = 32;
        if (sc < lend && *sc == 9) ch = 9;
        buf[k] = ch;
        k = k + 1;
        sc = sc + 1;
        indent = indent - 1;
    }
    buf[k] = '^';
    k = k + 1;
    buf[k] = 10;
    k = k + 1;
    kcc_sys_write(2, buf, k);
}
```

□

661b · used in 662b

56.3. Emit and Wrappers

`kcc_diag_emit` prints the location and message, draws the caret, and updates the counters: an error past the twentieth is fatal (the same limit the assembly enforced), and a warning bumps its own count. The three public wrappers fix the severity; `kcc_diag_had_errors` and `kcc_diag_reset` query and clear. The severity constants are the assembly's (error 0, warning 1, note 2), and the format strings—`<input>` for a missing path, the colons, and the severity words—match byte for byte.

⟨ *Selfhost: diag emit* 662a ⟩ ≡

```
void kcc_diag_emit(int severity, unsigned long
   line, unsigned long col,
                   const char *msg) {
    if (kcc_src_path) kcc_io_err(kcc_src_path);
    else kcc_io_err("<input>");
    kcc_io_err(":");
    diag_putu(line);
    kcc_io_err(":");
    diag_putu(col);
    kcc_io_err(": ");
    if (severity == 1) kcc_io_err("warning");
    else if (severity == 2) kcc_io_err("note");
    else kcc_io_err("error");
    kcc_io_err(": ");
    kcc_io_err(msg);
    kcc_io_err("\n");
    diag_caret(line, col);
    if (severity == 0) {
        kcc_diag_error_count =
   kcc_diag_error_count + 1;
        if (kcc_diag_error_count >= 20) {
            kcc_io_err("kcc: fatal: too many
   errors emitted, stopping\n");
            kcc_sys_exit(1);
        }
    } else if (severity == 1) {
        kcc_diag_warn_count = kcc_diag_warn_count
   + 1;
    }
}

void kcc_diag_error(unsigned long line, unsigned
   long col, const char *msg) {
    kcc_diag_emit(0, line, col, msg);
}
void kcc_diag_warning(unsigned long line, unsigned
    long col, const char *msg) {
    kcc_diag_emit(1, line, col, msg);
}
void kcc_diag_note(unsigned long line, unsigned
   long col, const char *msg) {
    kcc_diag_emit(2, line, col, msg);
}

unsigned long kcc_diag_had_errors(void) {
    return kcc_diag_error_count;
}
void kcc_diag_reset(void) {
    kcc_diag_error_count = 0;
    kcc_diag_warn_count = 0;
}
```

□

662a · used in 662b

56.4. Assembling the Translation Unit

⟨ *Selfhost: diag* 662b ⟩ ≡

```
⟨ Selfhost: diag preamble 661a ⟩
⟨ Selfhost: diag caret 661b ⟩
⟨ Selfhost: diag emit 662a ⟩
```

□

662b · used in 34i

56.5. Verification

`make selfhost-diag` runs the drop-in loop through the code-generation suite, and because this subsystem's product is error text, the swapped-in compiler is *also* exercised by `make diagtest` and `make warntest`, whose 23 and 31 checks compare the exact stderr of a rejected or warned program. A clean sweep of all three means kcc compiled its own diagnostics—format, caret, and counters—byte for byte.

⟨ *Selfhost: diag verify* 662c ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-diag - compile kcc's diagnostic
   engine in C with kcc, relink,
# run the codegen, diagnostic, and warning suites.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/diag.c with
   kcc"
$BASE/kcc -c "$SH/diag.c" -o "$SH/diag_c.o" >/dev/
   null

echo "[2/3] linking $BASE/kcc_selfhost_diag (diag.
   o -> kcc-compiled diag_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "diag" ]; then OBJ="$OBJ $SH/
   diag_c.o"; else OBJ="$OBJ $BASE/src/${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_diag $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running codegen + diagtest + warntest
   with the self-hosted diagnostics"
cp $BASE/kcc "$SH/kcc_asm_backup_diag"
cp $BASE/kcc_selfhost_diag $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh; s1=$?
bash $BASE/tests/diagtest.sh;      s2=$?
bash $BASE/tests/warntest.sh;      s3=$?
cp "$SH/kcc_asm_backup_diag" $BASE/kcc
[ $s1 -eq 0 ] && [ $s2 -eq 0 ] && [ $s3 -eq 0 ]
```

□

662c · used in 34j · uses _main 621, 629

57. Self-Hosting: The Symbol Table in C

The symbol table of Chapter 25 is the first *semantic* subsystem to port, and the first built entirely on containers that are now themselves C: it is a stack of scopes, each a hash table, layered over the vector. It answers "is this name declared, and as what?"—the question underneath every use of an identifier, tag, or label. Because both the vector and the hash table it stands on are already kcc-compiled C, porting the symbol table means name resolution runs top to bottom through code kcc produced.

57.1. Layout and State

Three scope stacks—ordinary identifiers, tags (`struct`/`union`/`enum`), and labels—are vectors of hash table pointers, kept in the exported globals `kcc_sym_ordinary`, `kcc_sym_tags`, `kcc_sym_labels`, alongside the current `kcc_sym_depth`. A symbol record is the 72-byte layout the assembly used, its fields—kind, type, name, linkage, value, depth, line, flags, column—at the same offsets semantic analysis reads. The port declares the vector's own header so it can pop a scope by decrementing the count directly, as the assembly does, and reads the lexer's `kcc_tok_line` for a declaration's provisional line.

⟨ *Selfhost: symtab preamble* 663a ⟩ ≡

```
struct Vec {
    void *data;
    unsigned long count;
    unsigned long cap;
    unsigned long elem_size;
};

struct Sym {
    unsigned long kind;
    void *type;
    const char *name;
    unsigned long linkage;
    unsigned long value;
    unsigned long depth;
    unsigned long line;
    unsigned long flags;
    unsigned long col;
};

void *kcc_vec_init(unsigned long elem_size);
void kcc_vec_push(void *vec, const void *elem);
void *kcc_vec_get(void *vec, unsigned long index);
void *kcc_ht_init(unsigned long capacity);
unsigned long kcc_ht_get(void *ht, const void *key
   , unsigned long keylen);
void kcc_ht_put(void *ht, const void *key,
   unsigned long keylen,
                unsigned long value);
void *kcc_arena_alloc(long size);

extern unsigned long kcc_tok_line;

void *kcc_sym_ordinary = 0;
void *kcc_sym_tags = 0;
void *kcc_sym_labels = 0;
unsigned long kcc_sym_depth = 0;

static void *sym_top_ht(void *stack) {
    struct Vec *v = (struct Vec *)stack;
    void **slot = (void **)kcc_vec_get(stack, v->
   count - 1);
    return *slot;
}
```

□

663a · used in 664c

57.2. Scopes

Pushing a scope allocates a fresh 256-slot ordinary table and a 16-slot tag table, appends each to its stack, and deepens the counter; popping decrements both counts and the depth (the arena reclaims the tables at exit). Because `kcc_symtab_init` pushes file scope, it is defined after the push it calls.

⟨ *Selfhost: symtab scopes* 663b ⟩ ≡

```
void kcc_symtab_push_scope(void) {
    void *ord = kcc_ht_init(256);
    kcc_vec_push(kcc_sym_ordinary, &ord);
    void *tag = kcc_ht_init(16);
    kcc_vec_push(kcc_sym_tags, &tag);
    kcc_sym_depth = kcc_sym_depth + 1;
}

void kcc_symtab_pop_scope(void) {
    ((struct Vec *)kcc_sym_ordinary)->count =
        ((struct Vec *)kcc_sym_ordinary)->count -
   1;
    ((struct Vec *)kcc_sym_tags)->count =
        ((struct Vec *)kcc_sym_tags)->count - 1;
    kcc_sym_depth = kcc_sym_depth - 1;
}

void kcc_symtab_init(void) {
    kcc_sym_ordinary = kcc_vec_init(8);
    kcc_sym_tags = kcc_vec_init(8);
    kcc_sym_labels = kcc_vec_init(8);
    kcc_sym_depth = 0;
    kcc_symtab_push_scope();
}
```

□

663b · used in 664c

57.3. Ordinary Names

`kcc_symtab_declare` builds a symbol record—kind, type, name, depth, provisional line, defined flag—and inserts it in the innermost ordinary scope. `kcc_symtab_lookup` walks the stack top-down so an inner declaration shadows an outer one; `lookup_current` consults only the innermost scope (to tell a same-scope redefinition from a legal shadow); and `current_ht` hands back that scope's table so a caller can iterate the block's symbols at exit.

⟨ *Selfhost: symtab ordinary* 663c ⟩ ≡

```
void *kcc_symtab_declare(const char *name,
   unsigned long len,
                            unsigned long kind, void
   *type) {
    struct Sym *s = (struct Sym *)kcc_arena_alloc
   (72);
    s->kind = kind;
    s->type = type;
    s->name = name;
    s->linkage = 0;
    s->value = 0;
    s->depth = kcc_sym_depth;
    s->line = kcc_tok_line;
    s->flags = 1;
    s->col = 0;
    kcc_ht_put(sym_top_ht(kcc_sym_ordinary), name,
    len, (unsigned long)s);
    return s;
}

void *kcc_symtab_lookup(const char *name, unsigned
    long len) {
    struct Vec *v = (struct Vec *)kcc_sym_ordinary
   ;
    unsigned long i = v->count;
    while (i > 0) {
        i = i - 1;
        void **slot = (void **)kcc_vec_get(
   kcc_sym_ordinary, i);
        unsigned long r = kcc_ht_get(*slot, name,
   len);
        if (r) return (void *)r;
    }
    return 0;
}

void *kcc_symtab_lookup_current(const char *name,
   unsigned long len) {
    return (void *)kcc_ht_get(sym_top_ht(
   kcc_sym_ordinary), name, len);
}

void *kcc_symtab_current_ht(void) {
    return sym_top_ht(kcc_sym_ordinary);
}
```

□

663c · used in 664c

57.4. Tags

Tags live in their own name space, so the same identifier can be a tag and an ordinary name at once. Declaration and lookup mirror the ordinary case on the tag stack.

⟨ *Selfhost: symtab tags* 664a ⟩ ≡

```
void *kcc_symtab_declare_tag(const char *name,
   unsigned long len, void *type) {
    struct Sym *s = (struct Sym *)kcc_arena_alloc
   (72);
    s->kind = 1;
    s->type = type;
    s->name = name;
    s->linkage = 0;
    s->value = 0;
    s->depth = kcc_sym_depth;
    s->line = 0;
    s->flags = 1;
    s->col = 0;
    kcc_ht_put(sym_top_ht(kcc_sym_tags), name, len
   , (unsigned long)s);
    return s;
}

void *kcc_symtab_lookup_tag(const char *name,
   unsigned long len) {
    struct Vec *v = (struct Vec *)kcc_sym_tags;
    unsigned long i = v->count;
    while (i > 0) {
        i = i - 1;
        void **slot = (void **)kcc_vec_get(
   kcc_sym_tags, i);
        unsigned long r = kcc_ht_get(*slot, name,
   len);
        if (r) return (void *)r;
    }
    return 0;
}
```

□

664a · used in 664c

57.5. Labels

Labels are function-scoped: one table per body. `label_begin` and `label_end` bracket the function, `label_add` records a label with a sentinel value, and `label_lookup` reports whether a `goto` target exists—guarding against an empty label stack.

⟨ *Selfhost: symtab labels* 664b ⟩ ≡

```
void kcc_symtab_label_begin(void) {
    void *h = kcc_ht_init(16);
    kcc_vec_push(kcc_sym_labels, &h);
}

void kcc_symtab_label_end(void) {
    ((struct Vec *)kcc_sym_labels)->count =
        ((struct Vec *)kcc_sym_labels)->count - 1;
}

void kcc_symtab_label_add(const char *name,
   unsigned long len) {
    kcc_ht_put(sym_top_ht(kcc_sym_labels), name,
   len, 1);
}

int kcc_symtab_label_lookup(const char *name,
   unsigned long len) {
    struct Vec *v = (struct Vec *)kcc_sym_labels;
    if (v->count == 0) return 0;
    unsigned long r = kcc_ht_get(sym_top_ht(
   kcc_sym_labels), name, len);
    return r ? 1 : 0;
}
```

□

664b · used in 664c

57.6. Assembling the Translation Unit

⟨ *Selfhost: symtab* 664c ⟩ ≡

⟨ *Selfhost: symtab preamble* 663a ⟩
⟨ *Selfhost: symtab scopes* 663b ⟩
⟨ *Selfhost: symtab ordinary* 663c ⟩

⟨ *Selfhost: symtab tags* 664a ⟩
⟨ *Selfhost: symtab labels* 664b ⟩
□

664c · used in 34k

57.7. Verification

`make selfhost-symtab` runs the drop-in loop. Because every declared name—variable, function, typedef, tag, or label—is entered and resolved through this table, a green code-generation suite means kcc compiled its own scoping and name resolution correctly, shadowing rules and all.

⟨ *Selfhost: symtab verify* 665 ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-symtab - compile kcc's symbol
   table in C with kcc, relink,
# run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/symtab.c with
    kcc"
$BASE/kcc -c "$SH/symtab.c" -o "$SH/symtab_c.o" >/
   dev/null

echo "[2/3] linking $BASE/kcc_selfhost_symtab (
   symtab.o -> kcc-compiled symtab_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "symtab" ]; then OBJ="$OBJ $SH/
   symtab_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
   fi
done
ld -o $BASE/kcc_selfhost_symtab $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted symbol table"
cp $BASE/kcc "$SH/kcc_asm_backup_symtab"
cp $BASE/kcc_selfhost_symtab $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_symtab" $BASE/kcc
exit $status
```

□

665 · used in 34l · uses `_main` 621, 629

58. Self-Hosting: The Standard Conversions in C

The conversion rules of Chapter 27 are the purest logic ported so far: six side-effect-free functions that, given type descriptors, answer the questions §6.3 poses—what does a small integer promote to, what common type do two arithmetic operands share, does an array or function decay, and is an implicit conversion allowed. They own no state and touch no I/O; they read a type's kind and signedness and call the type constructors and predicates that remain in the (still-assembly) type module. Porting them puts the compiler's type arithmetic through kcc-compiled C.

58.1. Reading a Type

Only two fields of a type descriptor are needed here—the kind at offset 0 and the field at offset 32, which for an integer holds its signedness (1 signed, 0 unsigned) and for an array or pointer holds the element or pointee type. The port declares a struct exposing just those, and the four canonical type descriptors (`int`, `float`, `double`, `long double`) and four type helpers as `extern`s. The kind constants are the type module's (bool 2, char 3, short 4, int 5, float 8, double 9, long double 10, pointer 11, array 12, function 13, enum 16).

⟨ *Selfhost: conversions preamble* 667a ⟩ ≡

```
struct Type {
    unsigned long kind;   /* TY_KIND, offset 0 */
    unsigned long f_size; /* offset 8 */
    unsigned long f_2;    /* offset 16 */
    unsigned long f_3;    /* offset 24 */
    unsigned long f0;     /* TY_F0, offset 32:
   signedness, or element/pointee type */
};

void *kcc_type_ptr(void *pointee);
int kcc_type_is_integer(void *t);
int kcc_type_is_arithmetic(void *t);
unsigned long kcc_type_rank(void *t);

extern void *kcc_type_int_s;
extern void *kcc_type_float;
extern void *kcc_type_double;
extern void *kcc_type_ldouble;
```

□

667a · used in 668b

58.2. Integer Promotions and Usual Arithmetic

`kcc_conv_promote` maps a below-`int`-rank integer (bool, char, short, or an enum) to `int` and leaves anything else alone. `kcc_conv_usual` finds the common type of two arithmetic operands: any floating operand pulls the result up to the widest float; otherwise both are promoted and the C rules pick by rank and signedness—equal rank returns the first, equal signedness gives the higher rank, and mixed signedness lets the unsigned type win only when its rank is at least the signed one's.

⟨ *Selfhost: conversions arith* 667b ⟩ ≡

```
void *kcc_conv_promote(void *type) {
    struct Type *t = (struct Type *)type;
    unsigned long k = t->kind;
    if (k == 2 || k == 3 || k == 4 || k == 16)
   return kcc_type_int_s;
    return type;
}

void *kcc_conv_usual(void *a, void *b) {
    struct Type *t1 = (struct Type *)a;
    struct Type *t2 = (struct Type *)b;
    if (t1->kind == 10 || t2->kind == 10) return
   kcc_type_ldouble;
    if (t1->kind == 9 || t2->kind == 9) return
   kcc_type_double;
    if (t1->kind == 8 || t2->kind == 8) return
   kcc_type_float;
    void *p1 = kcc_conv_promote(a);
    void *p2 = kcc_conv_promote(b);
    if (p1 == p2) return p1;
    struct Type *u1 = (struct Type *)p1;
    struct Type *u2 = (struct Type *)p2;
    if (u1->f0 == u2->f0) {
        if (kcc_type_rank(p1) >= kcc_type_rank(p2)
   ) return p1;
        return p2;
    }
    if (u1->f0 == 0) {
        if (kcc_type_rank(p1) >= kcc_type_rank(p2)
   ) return p1;
        return p2;
    }
    if (kcc_type_rank(p2) >= kcc_type_rank(p1))
   return p2;
    return p1;
}
```

□

667b · used in 668b

58.3. Decay and Lvalue

An array decays to a pointer to its element type, a function designator to a pointer to the function, and anything else is returned unchanged. The lvalue conversion is the no-op stub the assembly carried—qualifier stripping is handled elsewhere in semantic analysis—so it returns its argument.

⟨ *Selfhost: conversions decay* 667c ⟩ ≡

```
void *kcc_conv_array_decay(void *type) {
    struct Type *t = (struct Type *)type;
    if (t->kind == 12) return kcc_type_ptr((void
   *)t->f0);
    return type;
}

void *kcc_conv_func_decay(void *type) {
```

```
    struct Type *t = (struct Type *)type;
    if (t->kind == 13) return kcc_type_ptr(type);
    return type;
}

void *kcc_conv_lvalue(void *type) {
    return type;
}
```

□

667c · used in 668b

58.4. Implicit Conversion Check

`kcc_conv_implicit` is intentionally permissive: identical types, the same kind, integer-to-integer, arithmetic-to-arithmetic, and pointer-to-pointer all pass, as do the integer/pointer cross-conversions—leaving sema to warn rather than reject narrowing or sign changes. The clauses run in the same order as the assembly, so a pointer source only reaches the integer-target test after pointer-to-pointer has been ruled out.

⟨ *Selfhost: conversions implicit* 668a ⟩ ≡

```
int kcc_conv_implicit(void *from, void *to) {
    struct Type *f = (struct Type *)from;
    struct Type *t = (struct Type *)to;
    if (from == to) return 1;
    if (f->kind == t->kind) return 1;
    if (kcc_type_is_integer(from) &&
   kcc_type_is_integer(to)) return 1;
    if (kcc_type_is_arithmetic(from) &&
   kcc_type_is_arithmetic(to)) return 1;
    if (f->kind == 11 && t->kind == 11) return 1;
    if (f->kind == 11) return kcc_type_is_integer(
   to) ? 1 : 0;
    if (kcc_type_is_integer(from) && t->kind ==
   11) return 1;
    return 0;
}
```

□

668a · used in 668b

58.5. Assembling the Translation Unit

⟨ *Selfhost: conversions* 668b ⟩ ≡

```
⟨ Selfhost: conversions preamble 667a ⟩
⟨ Selfhost: conversions arith 667b ⟩
⟨ Selfhost: conversions decay 667c ⟩
⟨ Selfhost: conversions implicit 668a ⟩
```

□

668b · used in 34m

58.6. Verification

`make selfhost-conversions` runs the drop-in loop. Because the usual arithmetic conversions and the decays fire on nearly every expression the compiler type-checks, a green code-generation suite means kcc compiled its own type arithmetic correctly.

⟨ *Selfhost: conversions verify* 668c ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-conversions - compile kcc's
   conversions in C with kcc, relink,
# run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/conversions.c
    with kcc"
$BASE/kcc -c "$SH/conversions.c" -o "$SH/
   conversions_c.o" >/dev/null

echo "[2/3] linking $BASE/kcc_selfhost_conversions
    (conversions.o -> kcc-compiled)"
OBJ=""
for f in $sources; do
    if [ "$f" = "conversions" ]; then OBJ="$OBJ
   $SH/conversions_c.o"; else OBJ="$OBJ $BASE/src/
   ${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_conversions $OBJ -lSystem
    -syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted conversions"
cp $BASE/kcc "$SH/kcc_asm_backup_conversions"
cp $BASE/kcc_selfhost_conversions $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_conversions" $BASE/kcc
exit $status
```

□

668c · used in 34n · uses _main 621, 629

59. Self-Hosting: The Type System in C

The type system of Chapter 24 is the semantic core the whole front end stands on: every declaration resolves to one of its descriptors, every expression is assigned one, and the code generator reads a descriptor's size, alignment, and kind to lay out storage and select instructions. Porting it is the largest single step so far—fifteen exported functions and fifteen canonical descriptor globals—but the logic is uniform: allocate a forty-eight byte record, fill six fields, and read those same six fields back. Because the conversions ported in Chapter 58 already call into this module, a green suite with *both* in C means the compiler's type arithmetic runs top to bottom through code kcc produced.

59.1. The Descriptor

A type descriptor is the same forty-eight byte record the assembly used: a `kind` tag at offset 0, the object `size` and `align` at 8 and 16, the qualifier bits (`const` 1, `volatile` 2) at 24, and two general-purpose slots `f0` and `f1` at 32 and 40. The two slots carry whatever a kind needs—for a basic integer type `f0` holds its signedness (1 signed, 0 unsigned); for a pointer, array, function, or aggregate it holds the pointee, element, return, or declaration; and `f1` holds an array's length or a function's parameter vector. Every field is an `unsigned long` so the port mirrors the assembly's untyped machine words exactly, casting to a pointer only where a slot is dereferenced.

The kind constants are the type module's: void 1, _Bool 2, char 3, short 4, int 5, long 6, long long 7, float 8, double 9, long double 10, pointer 11, array 12, function 13, struct 14, union 15, enum 16, and the GNU 128-bit integer `__int128` 17. Three externs remain in the (still-assembly) rest of the compiler: the arena allocator, the layout pass that sizes an aggregate on demand (`kcc_type_layout`, which lives in declarations), and the two buffered-output helpers the dump routine prints through.

⟨ *Selfhost: types preamble* 669a ⟩ ≡

```
struct Type {
    unsigned long kind;   /* TY_KIND,  offset 0
   */
    unsigned long size;   /* TY_SIZE,  offset 8
   */
    unsigned long align;  /* TY_ALIGN, offset 16
   */
    unsigned long quals;  /* TY_QUALS, offset 24
   */
    unsigned long f0;     /* TY_F0,    offset 32
   */
    unsigned long f1;     /* TY_F1,    offset 40
   */
};
void *kcc_arena_alloc(long size);
unsigned long kcc_type_layout(void *t);
void kcc_io_puts(const char *s);
void kcc_io_putu(unsigned long n);

void *kcc_type_void = 0;
void *kcc_type_bool = 0;
void *kcc_type_char_s = 0;
void *kcc_type_char_u = 0;
void *kcc_type_short_s = 0;
void *kcc_type_short_u = 0;
void *kcc_type_int_s = 0;
void *kcc_type_int_u = 0;
void *kcc_type_long_s = 0;
void *kcc_type_long_u = 0;
void *kcc_type_llong_s = 0;
void *kcc_type_llong_u = 0;
void *kcc_type_int128_s = 0;
void *kcc_type_int128_u = 0;
void *kcc_type_float = 0;
void *kcc_type_double = 0;
void *kcc_type_ldouble = 0;
```

□

669a · used in 672b

59.2. Initialising the Canonical Descriptors

`kcc_types_init` allocates one singleton per basic type at startup and records it in the matching global. A file-local helper `mk` captures the shared shape—allocate, set kind/size/align, zero the qualifiers, store the signedness in `f0` and zero `f1`—so the allocations read as a table. The pair of `__int128` singletons carries size and alignment 16 (only its arithmetic is deferred). Plain `char` maps to *signed* char, following the Apple ARM64 sign convention; the integer types carry their width (1, 2, 4, 8) and signedness; and `long double` is not allocated at all but *aliased* to the `double` descriptor, because the Apple ARM64 ABI defines it as a 64-bit IEEE double—so every path that recognises a floating type by its kind treats a long double exactly as a double.

⟨ *Selfhost: types init* 669b ⟩ ≡

```
static void *mk(unsigned long kind, unsigned long
   size,
                unsigned long align, unsigned long
    f0) {
    struct Type *t = (struct Type *)
   kcc_arena_alloc(48);
    t->kind = kind;
    t->size = size;
    t->align = align;
    t->quals = 0;
    t->f0 = f0;
    t->f1 = 0;
    return t;
}

void kcc_types_init(void) {
    kcc_type_void    = mk(1, 0, 0, 0);
    kcc_type_bool    = mk(2, 1, 1, 0);
    kcc_type_char_s  = mk(3, 1, 1, 1);
    kcc_type_char_u  = mk(3, 1, 1, 0);
    kcc_type_short_s = mk(4, 2, 2, 1);
    kcc_type_short_u = mk(4, 2, 2, 0);
    kcc_type_int_s   = mk(5, 4, 4, 1);
```

```
    kcc_type_int_u   = mk(5, 4, 4, 0);
    kcc_type_long_s  = mk(6, 8, 8, 1);
    kcc_type_long_u  = mk(6, 8, 8, 0);
    kcc_type_llong_s = mk(7, 8, 8, 1);
    kcc_type_llong_u = mk(7, 8, 8, 0);
    kcc_type_int128_s = mk(17, 16, 16, 1);
    kcc_type_int128_u = mk(17, 16, 16, 0);
    kcc_type_float   = mk(8, 4, 4, 0);
    kcc_type_double  = mk(9, 8, 8, 0);
    kcc_type_ldouble = kcc_type_double;
}
```

□

669b · used in 672b

59.3. Constructors

The derived-type constructors each allocate a fresh descriptor and wire the component types into `f0`/`f1`. A pointer is eight bytes wide and holds its pointee in `f0`. `kcc_type_param_adjust` applies the parameter rule of §6.7.6.3: an array parameter decays to a pointer to its element, a function parameter to a pointer to the function, and anything else passes through—so the callee indexes the passed address rather than inline storage. An array lays its element out *first* (`kcc_type_layout` is idempotent) so that an aggregate element declared before it is otherwise referenced still contributes a correct size; the total is element-size times length, alignment is inherited, and length lives in `f1` (zero meaning incomplete). A function type has no size; its return type sits in `f0` and its parameter vector in `f1`.

⟨ *Selfhost: types constructors* 670a ⟩ ≡

```
void *kcc_type_ptr(void *pointee) {
    struct Type *t = (struct Type *)
   kcc_arena_alloc(48);
    t->kind = 11;
    t->size = 8;
    t->align = 8;
    t->quals = 0;
    t->f0 = (unsigned long)pointee;
    t->f1 = 0;
    return t;
}

void *kcc_type_param_adjust(void *type) {
    if (type == 0) return type;
    struct Type *t = (struct Type *)type;
    if (t->kind == 12) return kcc_type_ptr((void
   *)t->f0);
    if (t->kind == 13) return kcc_type_ptr(type);
    return type;
}

void *kcc_type_array(void *elem, unsigned long len
   ) {
    struct Type *a = (struct Type *)
   kcc_arena_alloc(48);
    a->kind = 12;
    a->size = kcc_type_layout(elem) * len;
    a->align = ((struct Type *)elem)->align;
    a->quals = 0;
    a->f0 = (unsigned long)elem;
    a->f1 = len;
    return a;
}

void *kcc_type_func(void *ret, void *params) {
    struct Type *f = (struct Type *)
   kcc_arena_alloc(48);
    f->kind = 13;
    f->size = 0;
    f->align = 0;
    f->quals = 0;
    f->f0 = (unsigned long)ret;
    f->f1 = (unsigned long)params;
    return f;
}
```

□

670a · used in 672b

59.4. Aggregate Constructors

Struct and union descriptors are *memoised* on the defining AST node. The node's spare field at offset 40 (`AST_F2`, zero-initialised by `ast_alloc`) caches the descriptor, and re-resolving the same definition—particularly an anonymous one, which has no tag to deduplicate through—returns the same record. A second descriptor would share the field AST nodes and, on layout, re-run the destructive bit-field packing that rewrites those nodes, corrupting a definition the first layout already consumed. Struct and union differ only in kind (14 versus 15); both leave size and alignment at zero for the layout pass to fill. An enum needs no memoisation: its underlying type is `int`, so it is four bytes, four-aligned, with the declaration in `f0` (§6.7.2.2).

⟨ *Selfhost: types aggregates* 670b ⟩ ≡

```
void *kcc_type_struct(void *decl) {
    if (decl) {
        unsigned long cached = *(unsigned long *)
   ((char *)decl + 40);
        if (cached) return (void *)cached;
    }
    struct Type *t = (struct Type *)
   kcc_arena_alloc(48);
    t->kind = 14;
    t->size = 0;
    t->align = 0;
    t->quals = 0;
    t->f0 = (unsigned long)decl;
    t->f1 = 0;
    if (decl) *(unsigned long *)((char *)decl +
   40) = (unsigned long)t;
    return t;
}

void *kcc_type_union(void *decl) {
    if (decl) {
        unsigned long cached = *(unsigned long *)
   ((char *)decl + 40);
        if (cached) return (void *)cached;
    }
    struct Type *t = (struct Type *)
   kcc_arena_alloc(48);
    t->kind = 15;
    t->size = 0;
    t->align = 0;
    t->quals = 0;
    t->f0 = (unsigned long)decl;
    t->f1 = 0;
    if (decl) *(unsigned long *)((char *)decl +
   40) = (unsigned long)t;
    return t;
```

```
}

void *kcc_type_enum(void *decl) {
    struct Type *t = (struct Type *)
   kcc_arena_alloc(48);
    t->kind = 16;
    t->size = 4;
    t->align = 4;
    t->quals = 0;
    t->f0 = (unsigned long)decl;
    t->f1 = 0;
    return t;
}
```

□

670b · used in 672b

59.5. Specifier Bitmask Resolution

The parser accumulates type keywords into a bitmask, and `kcc_type_from_specmask` maps each valid combination to a canonical descriptor. The bit values are the specifier constants (void 1, char 2, short 4, int 8, long 16, long long 32, float 64, double 128, signed 256, unsigned 512, _Bool 1024), and the accepted combinations are exactly those §6.7.2 permits: `signed`/`unsigned` may qualify any integer, a redundant `int` may accompany `short`/`long`, and two `long`s (or `long` plus the internal long-long bit) name `long long`. Plain and `signed` forms select the signed singleton; an `unsigned` bit selects the unsigned one. Any bitmask that matches nothing—an invalid soup like `float unsigned`—returns zero, and semantic analysis reports the error.

⟨ *Selfhost: types specmask* 671a ⟩ ≡

```
#define SPEC_VOID     1
#define SPEC_CHAR     2
#define SPEC_SHORT    4
#define SPEC_INT      8
#define SPEC_LONG     16
#define SPEC_LLONG    32
#define SPEC_FLOAT    64
#define SPEC_DOUBLE   128
#define SPEC_SIGNED   256
#define SPEC_UNSIGNED 512
#define SPEC_BOOL     1024
#define SPEC_INT128   4096

void *kcc_type_from_specmask(unsigned long m) {
    if (m == SPEC_VOID) return kcc_type_void;
    if (m == SPEC_BOOL) return kcc_type_bool;

    if (m == SPEC_CHAR || m == (SPEC_SIGNED |
   SPEC_CHAR)) return kcc_type_char_s;
    if (m == (SPEC_UNSIGNED | SPEC_CHAR)) return
   kcc_type_char_u;

    if (m == SPEC_SHORT || m == (SPEC_SIGNED |
   SPEC_SHORT) ||
        m == (SPEC_SHORT | SPEC_INT) ||
        m == (SPEC_SIGNED | SPEC_SHORT | SPEC_INT)
   ) return kcc_type_short_s;
    if (m == (SPEC_UNSIGNED | SPEC_SHORT) ||
        m == (SPEC_UNSIGNED | SPEC_SHORT |
   SPEC_INT)) return kcc_type_short_u;

    if (m == SPEC_INT || m == SPEC_SIGNED ||
        m == (SPEC_SIGNED | SPEC_INT)) return
   kcc_type_int_s;
    if (m == SPEC_UNSIGNED ||
        m == (SPEC_UNSIGNED | SPEC_INT)) return
   kcc_type_int_u;

    if (m == SPEC_LONG || m == (SPEC_SIGNED |
   SPEC_LONG) ||
        m == (SPEC_LONG | SPEC_INT) ||
        m == (SPEC_SIGNED | SPEC_LONG | SPEC_INT))
    return kcc_type_long_s;
    if (m == (SPEC_UNSIGNED | SPEC_LONG) ||
        m == (SPEC_UNSIGNED | SPEC_LONG | SPEC_INT
   )) return kcc_type_long_u;

    if (m == SPEC_LLONG || m == (SPEC_SIGNED |
   SPEC_LLONG) ||
        m == (SPEC_LLONG | SPEC_INT) ||
        m == (SPEC_SIGNED | SPEC_LLONG | SPEC_INT)
    ||
        m == (SPEC_LONG | SPEC_LLONG) ||
        m == (SPEC_SIGNED | SPEC_LONG | SPEC_LLONG
   ) ||
        m == (SPEC_LONG | SPEC_LLONG | SPEC_INT)
   ||
        m == (SPEC_SIGNED | SPEC_LONG | SPEC_LLONG
    | SPEC_INT))
        return kcc_type_llong_s;
    if (m == (SPEC_UNSIGNED | SPEC_LLONG) ||
        m == (SPEC_UNSIGNED | SPEC_LLONG |
   SPEC_INT) ||
        m == (SPEC_UNSIGNED | SPEC_LONG |
   SPEC_LLONG) ||
        m == (SPEC_UNSIGNED | SPEC_LONG |
   SPEC_LLONG | SPEC_INT))
        return kcc_type_llong_u;

    if (m == SPEC_INT128 || m == (SPEC_SIGNED |
   SPEC_INT128))
        return kcc_type_int128_s;
    if (m == (SPEC_UNSIGNED | SPEC_INT128)) return
    kcc_type_int128_u;

    if (m == SPEC_FLOAT) return kcc_type_float;
    if (m == SPEC_DOUBLE) return kcc_type_double;
    if (m == (SPEC_LONG | SPEC_DOUBLE)) return
   kcc_type_ldouble;

    return 0;
}
```

□

671a · used in 672b

59.6. Predicates and Rank

The predicates classify a type by kind. `kcc_type_compatible` treats pointer identity as trivially compatible, then requires the same kind; for a basic type it additionally compares signedness (`f0`), and for a derived type only pointers are (permissively) accepted, deferring a deeper structural check. `kcc_type_is_integer` recognises the integer kinds including `enum`; `kcc_type_is_arithmetic` adds the three floating kinds; and `kcc_type_is_scalar` adds pointers. `kcc_type_rank` assigns the integer conversion rank of §6.3.1.1—_Bool 1 through long long 6, with `enum` ranking as `int`—so callers pick the wider operand with a single comparison.

⟨ *Selfhost: types predicates* 671b ⟩ ≡

```
int kcc_type_compatible(void *a, void *b) {
    if (a == b) return 1;
    struct Type *t1 = (struct Type *)a;
    struct Type *t2 = (struct Type *)b;
    if (t1->kind != t2->kind) return 0;
    if (t1->kind >= 11) {
        if (t1->kind == 11) return 1;
        return 0;
    }
    if (t1->f0 != t2->f0) return 0;
    return 1;
}

int kcc_type_is_integer(void *type) {
    unsigned long k = ((struct Type *)type)->kind;
    if (k == 2 || k == 3 || k == 4 || k == 5 || k
   == 6 || k == 7 || k == 16 ||
        k == 17)
        return 1;
    return 0;
}

int kcc_type_is_arithmetic(void *type) {
    if (kcc_type_is_integer(type)) return 1;
    unsigned long k = ((struct Type *)type)->kind;
    if (k == 8 || k == 9 || k == 10) return 1;
    return 0;
}

int kcc_type_is_scalar(void *type) {
    if (kcc_type_is_arithmetic(type)) return 1;
    if (((struct Type *)type)->kind == 11) return
   1;
    return 0;
}

unsigned long kcc_type_rank(void *type) {
    unsigned long k = ((struct Type *)type)->kind;
    if (k == 2) return 1;
    if (k == 3) return 2;
    if (k == 4) return 3;
    if (k == 5) return 4;
    if (k == 16) return 4;
    if (k == 6) return 5;
    if (k == 7) return 6;
    if (k == 17) return 7;      /* __int128
   outranks long long */
    return 0;
}
```

□

671b · used in 672b

59.7. Dump

`kcc_type_dump` prints a type the way a diagnostic shows it, and its bytes must match the assembly's exactly. It leads with any qualifier words (`const `, `volatile `), then the kind: the integer kinds choose between the plain and `unsigned` spelling on the signedness flag, the floating kinds print a fixed word, a pointer recurses on its pointee and appends `*`, an array recurses and appends the bracketed length, a function recurses on its return type and appends `()`, and the aggregates print their keyword. A null descriptor prints `<null>` and an unrecognised kind `<unknown>`.

⟨ *Selfhost: types dump* 672a ⟩ ≡

```
void kcc_type_dump(void *type) {
    if (type == 0) { kcc_io_puts("<null>"); return
   ; }
    struct Type *t = (struct Type *)type;
    if (t->quals & 1) kcc_io_puts("const ");
    if (t->quals & 2) kcc_io_puts("volatile ");
    unsigned long k = t->kind;
    if (k == 1) { kcc_io_puts("void"); return; }
    if (k == 2) { kcc_io_puts("_Bool"); return; }
    if (k == 3) { kcc_io_puts(t->f0 ? "char" : "
   unsigned char"); return; }
    if (k == 4) { kcc_io_puts(t->f0 ? "short" : "
   unsigned short"); return; }
    if (k == 5) { kcc_io_puts(t->f0 ? "int" : "
   unsigned int"); return; }
    if (k == 6) { kcc_io_puts(t->f0 ? "long" : "
   unsigned long"); return; }
    if (k == 7) { kcc_io_puts(t->f0 ? "long long"
   : "unsigned long long"); return; }
    if (k == 8) { kcc_io_puts("float"); return; }
    if (k == 9) { kcc_io_puts("double"); return; }
    if (k == 10) { kcc_io_puts("long double");
   return; }
    if (k == 11) { kcc_type_dump((void *)t->f0);
   kcc_io_puts(" *"); return; }
    if (k == 12) {
        kcc_type_dump((void *)t->f0);
        kcc_io_puts("[");
        kcc_io_putu(t->f1);
        kcc_io_puts("]");
        return;
    }
    if (k == 13) { kcc_type_dump((void *)t->f0);
   kcc_io_puts(" ()"); return; }
    if (k == 14) { kcc_io_puts("struct"); return;
   }
    if (k == 15) { kcc_io_puts("union"); return; }
    if (k == 16) { kcc_io_puts("enum"); return; }
    kcc_io_puts("<unknown>");
}
```

□

672a · used in 672b

59.8. Assembling the Translation Unit

⟨ *Selfhost: types* 672b ⟩ ≡

⟨ *Selfhost: types preamble* 669a ⟩
⟨ *Selfhost: types init* 669b ⟩
⟨ *Selfhost: types constructors* 670a ⟩
⟨ *Selfhost: types aggregates* 670b ⟩
⟨ *Selfhost: types specmask* 671a ⟩
⟨ *Selfhost: types predicates* 671b ⟩
⟨ *Selfhost: types dump* 672a ⟩

□

672b · used in 34o

59.9. Verification

`make selfhost-types` runs the drop-in loop. Because every declaration resolves through these constructors and every expression is typed through these predicates, a green code-generation suite means kcc compiled its own type

system—descriptors, layout hooks, promotions, and all—correctly.

⟨ *Selfhost: types verify* 673 ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-types - compile kcc's type system
   in C with kcc, relink,
# run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/types.c with
   kcc"
$BASE/kcc -c "$SH/types.c" -o "$SH/types_c.o" >/
   dev/null

echo "[2/3] linking $BASE/kcc_selfhost_types (
   types.o -> kcc-compiled types_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "types" ]; then OBJ="$OBJ $SH/
   types_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
   fi
done
ld -o $BASE/kcc_selfhost_types $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted type system"
cp $BASE/kcc "$SH/kcc_asm_backup_types"
cp $BASE/kcc_selfhost_types $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_types" $BASE/kcc
exit $status
```

□

673 · used in 35a · uses _main 621, 629

60. Self-Hosting: The Optimizer in C

The optimizer of Part 31 is the first *middle-end* subsystem to port, and the most thoroughly exercised: every one of the code-generation tests runs its program through the whole fixed-point loop—constant folding, strength reduction, common-subexpression elimination, store-to-load forwarding, and dead-code elimination—so a byte-identical code-generation suite is a strong witness that the C port reproduces each pass exactly. The passes are pure IR-to-IR transforms with no I/O and no target knowledge, so they read only as walks over the IR data structures; the whole subsystem is arithmetic over three record layouts.

60.1. Reading the IR

The port declares the three IR records at the offsets Chapter 28 fixed: an instruction (opcode, destination vreg, two sources, a type pointer, and the next-instruction link), a basic block (its first instruction and its next-block link), and a function (its entry block, its vector of blocks, and its running vreg counter). It also declares the vector header, so a pass can read a block-vector's count directly, and a minimal view of a type descriptor—its kind and qualifier bits—so the passes can refuse to fold or strength-reduce a floating-point instruction (whose constant operand is an IEEE-754 bit pattern, not an integer) and can honour `volatile`. The externs are the arena, the vector accessor, the instruction allocator (used by strength reduction to splice in a fresh constant), and the global list of IR functions and the canonical `int` type.

⟨ *Selfhost: opt preamble* 675 ⟩ ≡

```
struct Instr {
    unsigned long op;     /* IR_OP,    0  */
    unsigned long dest;   /* IR_DEST,  8  */
    unsigned long src1;   /* IR_SRC1,  16 */
    unsigned long src2;   /* IR_SRC2,  24 */
    void *type;           /* IR_TYPE,  32 */
    struct Instr *next;   /* IR_NEXT,  40 */
};

struct BB {
    unsigned long id;     /* BB_ID,     0  */
    struct Instr *first;  /* BB_FIRST,  8  */
    struct Instr *last;   /* BB_LAST,   16 */
    struct BB *next;      /* BB_NEXT,   24 */
    unsigned long flags;  /* BB_FLAGS,  32 */
    unsigned long pad;    /* to 48 bytes   */
};

struct Func {
    void *name;               /* IRF_NAME,    0  */
    struct BB *entry;         /* IRF_ENTRY,   8  */
    void *blocks;             /* IRF_BLOCKS,  16 (
   vec of BB*) */
    unsigned long vreg_n;     /* IRF_VREG_N, 24 */
    /* remaining fields (rettype, params, ...) are
    not read by the optimizer */
};

struct Vec {
    void *data;
    unsigned long count;      /* VEC_COUNT, 8 */
    unsigned long cap;
    unsigned long elem_size;
};

struct Type {
    unsigned long kind;       /* TY_KIND,   0  */
    unsigned long size;       /* TY_SIZE,   8  */
    unsigned long align;      /* TY_ALIGN,  16 */
    unsigned long quals;      /* TY_QUALS,  24 */
};

void *kcc_arena_alloc(long size);
void *kcc_vec_get(void *vec, unsigned long index);
void *kcc_ir_alloc_instr(long op);
extern void *kcc_ir_funcs;
extern void *kcc_type_int_s;

/* An IR_ASM instruction carries its operand list
   off in the AST node reached
   through the type field; each operand encodes a
   register in the low 32 bits
   (register+1, or 0 for none) and a byte width in
    the high 32.  The optimiser
   reaches those encoded registers with a byte
   accessor so it can renumber and
   mark them alongside the ordinary source fields.
    */
#define OPT_AST_F2 40
#define OPT_AST_F3 48
#define OPT_AST_F4 56
#define OPTL(p,off) (*(unsigned long *)((char *)(p
   ) + (off)))

/* IR opcodes (Chapter: IR design). */
enum {
    IR_ADD = 1, IR_SUB = 2, IR_MUL = 3, IR_SDIV =
   4, IR_UDIV = 5,
    IR_SMOD = 6, IR_UMOD = 7, IR_NEG = 8, IR_AND =
    10, IR_OR = 11,
    IR_XOR = 12, IR_SHL = 13, IR_LSHR = 14,
   IR_ASHR = 15, IR_NOT = 16,
    IR_CMP_EQ = 20, IR_CMP_NE = 21, IR_CMP_SLT =
   22, IR_CMP_SLE = 23,
    IR_CMP_SGT = 24, IR_CMP_SGE = 25, IR_CMP_ULT =
    26, IR_CMP_ULE = 27,
    IR_LOAD = 30, IR_STORE = 31, IR_ALLOCA = 32,
   IR_GEP = 33,
    IR_BR = 40, IR_BR_COND = 41, IR_RET = 42,
   IR_CALL = 43, IR_CALL_IND = 44,
    IR_SEXT = 50, IR_ZEXT = 51, IR_TRUNC = 52,
   IR_VA_START = 56, IR_FPROUND = 57,
    IR_CONST = 60, IR_NOP = 61, IR_GLOBAL_REF =
   63, IR_STRING_LIT = 64, IR_ASM = 65
};

/* A floating type: kind FLOAT(8), DOUBLE(9), or
   LDOUBLE(10).  The pointer
 * guard mirrors the assembly, which treats a
   small non-pointer type slot as
 * non-floating. */
static int opt_is_fp(void *type) {
    if ((unsigned long)type < 0x1000) return 0;
    unsigned long k = ((struct Type *)type)->kind;
    return k >= 8 && k <= 10;
}
```

```
/* A volatile-qualified type: quals bit 2 (0x2)
   set. */
static int opt_is_vol(void *type) {
    if ((unsigned long)type < 0x1000) return 0;
    return (((struct Type *)type)->quals & 2) !=
   0;
}
```

□

675 · used in 681b

60.2. Constant Folding

Constant folding runs two linear passes over the blocks. The first records, in a dense per-vreg map, every value produced by an IR_CONST; the second rewrites in place any instruction all of whose register inputs are known constants. Three shapes fold: a binary arithmetic, bitwise, shift, or comparison instruction with two constant sources; a unary NEG or NOT of a constant; and—the load-bearing case—a width cast (SEXT/ZEXT/TRUNC) of a constant, without which a constant chain stalls at the first cast the front end inserts. A folded instruction becomes an IR_CONST, and its value is entered in the map so a later instruction in the same pass can fold on top of it. Floating-point instructions are skipped so their bit patterns are never mangled by integer arithmetic. Shift amounts are masked to six bits to match the machine's register-shift semantics.

⟨ *Selfhost: opt constfold* 676 ⟩ ≡

```
static void opt_cf_replace(struct Instr *in,
   unsigned long result,
                           unsigned long *values,
   unsigned long *valid,
                           unsigned long n,
   unsigned long *count) {
    in->op = IR_CONST;
    in->src1 = result;
    in->src2 = 0;
    if (in->dest < n) { valid[in->dest] = 1;
   values[in->dest] = result; }
    *count += 1;
}

unsigned long kcc_opt_constfold(void *fn) {
    struct Func *func = (struct Func *)fn;
    unsigned long count = 0;
    unsigned long n = func->vreg_n;
    if (n == 0) return 0;
    n = n + 1;

    unsigned long *values = (unsigned long *)
   kcc_arena_alloc((long)(n * 8));
    unsigned long *valid = (unsigned long *)
   kcc_arena_alloc((long)(n * 8));
    for (unsigned long i = 0; i < n; i++) valid[i]
    = 0;

    struct Vec *blocks = (struct Vec *)func->
   blocks;
    if (blocks == 0) return 0;
    unsigned long nb = blocks->count;

    /* Pass 1: collect constants. */
    for (unsigned long b = 0; b < nb; b++) {
        struct BB *bb = *(struct BB **)kcc_vec_get
   (blocks, b);
        for (struct Instr *in = bb->first; in; in
   = in->next) {
            if (in->op == IR_CONST && in->dest < n
   ) {
                valid[in->dest] = 1;
                values[in->dest] = in->src1;
            }
        }
    }

    /* Pass 2: fold. */
    for (unsigned long b = 0; b < nb; b++) {
        struct BB *bb = *(struct BB **)kcc_vec_get
   (blocks, b);
        for (struct Instr *in = bb->first; in; in
   = in->next) {
            if (opt_is_fp(in->type)) continue;
            unsigned long op = in->op;

            if (op == IR_SEXT || op == IR_ZEXT ||
   op == IR_TRUNC) {
                unsigned long s1 = in->src1;
                if (s1 >= n || !valid[s1])
   continue;
                unsigned long val = values[s1];
                unsigned long result, width;
                if (op == IR_TRUNC) {
                    struct Type *t = (struct Type
   *)in->type;
                    if (t == 0 || t->size == 0)
   continue;
                    width = t->size;
                    if (width >= 8) result = val;
                    else result = val & ((1UL << (
   width * 8)) - 1);
                } else {
                    width = in->src2;
  /* source width in bytes */
                    if (width == 0) continue;
                    if (width >= 8) result = val;
                    else if (op == IR_ZEXT) result
    = val & ((1UL << (width * 8)) - 1);
                    else {
    /* SEXT */
                        unsigned long sh = 64 -
   width * 8;
                        result = (unsigned long)
   (((long)(val << sh)) >> sh);
                    }
                }
                opt_cf_replace(in, result, values,
    valid, n, &count);
                continue;
            }

            if (op < IR_ADD || op > IR_CMP_SGE)
   continue;

            if (op == IR_NEG || op == IR_NOT) {
                unsigned long s1 = in->src1;
                if (s1 >= n || !valid[s1])
   continue;
                unsigned long val = values[s1];
                unsigned long result = (op ==
   IR_NEG) ? (unsigned long)(-(long)val)

    : ~val;
                opt_cf_replace(in, result, values,
    valid, n, &count);
                continue;
            }
```

```
            /* Binary: both sources must be
    constant. */
            unsigned long s1 = in->src1, s2 = in->
    src2;
            if (s1 >= n || !valid[s1]) continue;
            if (s2 >= n || !valid[s2]) continue;
            unsigned long a = values[s1], bb2 =
    values[s2];
            unsigned long r;
            /* An if-else chain (not a switch): it
     mirrors the assembly's
             * compare-and-branch dispatch, and
    keeps `continue` directly in
             * the enclosing for-loop. */
            if (op == IR_ADD) r = a + bb2;
            else if (op == IR_SUB) r = a - bb2;
            else if (op == IR_MUL) r = a * bb2;
            else if (op == IR_SDIV) {
                if (bb2 == 0) continue;
                r = (unsigned long)((long)a / (
    long)bb2);
            }
            else if (op == IR_AND) r = a & bb2;
            else if (op == IR_OR)  r = a | bb2;
            else if (op == IR_XOR) r = a ^ bb2;
            else if (op == IR_SHL) r = a << (bb2 &
     63);
            else if (op == IR_ASHR) r = (unsigned
    long)((long)a >> (bb2 & 63));
            else if (op == IR_CMP_EQ)  r = (a ==
    bb2) ? 1 : 0;
            else if (op == IR_CMP_NE)  r = (a !=
    bb2) ? 1 : 0;
            else if (op == IR_CMP_SLT) r = ((long)
    a < (long)bb2) ? 1 : 0;
            else if (op == IR_CMP_SGT) r = ((long)
    a > (long)bb2) ? 1 : 0;
            else continue;                  /* not
    a foldable binary op */
            opt_cf_replace(in, r, values, valid, n
    , &count);
        }
    }
    return count;
}
```

□

676 · used in 681b

60.3. Strength Reduction

Where folding needs *both* operands constant, strength reduction handles a single constant operand that is a power of two: a multiply becomes a left shift (for signed and unsigned alike), an unsigned divide a logical right shift, and an unsigned remainder a mask; a multiply by zero collapses to the constant zero. Signed divide and remainder are left alone, since an arithmetic shift rounds the wrong way. A shift or mask needs a fresh constant for its amount, so the pass allocates an `IR_CONST`, draws a new vreg from the function's counter, and splices it in just before the rewritten instruction—which is why the walk tracks the previous instruction to relink the list. Floating-point instructions are skipped so a `2.0` bit pattern is never mistaken for a power of two.

⟨ *Selfhost: opt strength* 677 ⟩ ≡

```
static int opt_sr_cval(unsigned long vreg,
    unsigned long n,
                       unsigned long *values,
    unsigned long *valid,
                       unsigned long *out) {
    if (vreg >= n) return 0;
    if (!valid[vreg]) return 0;
    *out = values[vreg];
    return 1;
}

/* log2 of a power of two, matching 63 - clz. */
static unsigned long opt_log2(unsigned long c) {
    unsigned long k = 0;
    while ((c >> k) != 1) k++;
    return k;
}

unsigned long kcc_opt_strength(void *fn) {
    struct Func *func = (struct Func *)fn;
    unsigned long count = 0;
    unsigned long n = func->vreg_n;
    if (n == 0) return 0;
    n = n + 1;

    unsigned long *values = (unsigned long *)
    kcc_arena_alloc((long)(n * 8));
    unsigned long *valid = (unsigned long *)
    kcc_arena_alloc((long)(n * 8));
    for (unsigned long i = 0; i < n; i++) valid[i]
    = 0;

    /* Pass 1: collect constants (via the entry
    chain). */
    for (struct BB *bb = func->entry; bb; bb = bb
    ->next)
        for (struct Instr *in = bb->first; in; in
    = in->next)
            if (in->op == IR_CONST && in->dest < n
    ) {
                valid[in->dest] = 1;
                values[in->dest] = in->src1;
            }

    /* Pass 2: rewrite. */
    for (struct BB *bb = func->entry; bb; bb = bb
    ->next) {
        struct Instr *prev = 0;
        for (struct Instr *cur = bb->first; cur; )
    {
            if (opt_is_fp(cur->type)) { prev = cur
    ; cur = cur->next; continue; }
            unsigned long op = cur->op;
            unsigned long C = 0, x = 0, amount =
    0, newop = 0;
            int to_zero = 0, rewrite = 0;

            if (op == IR_MUL) {
                unsigned long v;
                if (opt_sr_cval(cur->src2, n,
    values, valid, &v)) {
                    C = v; x = cur->src1;
                } else if (opt_sr_cval(cur->src1,
    n, values, valid, &v)) {
                    C = v; x = cur->src2;
                } else { prev = cur; cur = cur->
    next; continue; }
                if (C == 0) { to_zero = 1; }
                else if ((long)C < 2) { prev = cur
    ; cur = cur->next; continue; }
```

```
                else if (C & (C - 1)) { prev = cur
    ; cur = cur->next; continue; }
                else { amount = opt_log2(C); newop
     = IR_SHL; rewrite = 1; }
            } else if (op == IR_UDIV) {
                unsigned long v;
                if (!opt_sr_cval(cur->src2, n,
    values, valid, &v)) { prev = cur; cur = cur->
    next; continue; }
                C = v;
                if ((long)C < 2) { prev = cur; cur
     = cur->next; continue; }
                if (C & (C - 1)) { prev = cur; cur
     = cur->next; continue; }
                amount = opt_log2(C); x = cur->
    src1; newop = IR_LSHR; rewrite = 1;
            } else if (op == IR_UMOD) {
                unsigned long v;
                if (!opt_sr_cval(cur->src2, n,
    values, valid, &v)) { prev = cur; cur = cur->
    next; continue; }
                C = v;
                if ((long)C < 1) { prev = cur; cur
     = cur->next; continue; }
                if (C & (C - 1)) { prev = cur; cur
     = cur->next; continue; }
                amount = C - 1; x = cur->src1;
    newop = IR_AND; rewrite = 1;
            } else { prev = cur; cur = cur->next;
    continue; }

            if (to_zero) {
                cur->op = IR_CONST;
                cur->src1 = 0;
                cur->src2 = 0;
                count++;
                prev = cur; cur = cur->next;
    continue;
            }

            if (rewrite) {
                struct Instr *k = (struct Instr *)
    kcc_ir_alloc_instr(IR_CONST);
                k->type = kcc_type_int_s;
                k->src1 = amount;
                k->src2 = 0;
                unsigned long v = func->vreg_n;
                func->vreg_n = v + 1;
                k->dest = v;
                k->next = cur;
                if (prev) prev->next = k;
                else bb->first = k;
                cur->op = newop;
                cur->src1 = x;
                cur->src2 = v;
                count++;
                prev = cur; cur = cur->next;
    continue;
            }
        }
    }
    return count;
}
```

□

677 · used in 681b

60.4. Common Subexpression Elimination

CSE value-numbers within a single basic block: two pure instructions with the same opcode and the same (already-canonicalised) source registers compute the same value, so the second's destination is mapped to the first's, and a redundant load reuses an earlier load of the same address when no store, call, or inline asm has intervened. A function-wide replacement map records these equivalences, and every instruction's operands are first rewritten through it, so a later use reads the surviving register; dead-code elimination then removes the instruction left with no users. The canonicaliser rewrites exactly the fields that hold a register—skipping immediates, widths, block ids, and symbols—and a volatile load acts as a barrier that is neither reused nor recorded. An `IR_ASM` is a special case: its input values and output addresses are not in the source fields at all but encoded (register+1 in the low half, byte width in the high) in the operand list hung off the AST node, so the canonicaliser walks that list and renumbers each encoded register in place. Without it, store-to-load forwarding would map an asm input's load to the stored value yet leave the asm still naming the swept load—reading an uninitialised slot at run time. It must renumber *only* the encoded registers, though: an operand flagged immediate, symbolic, or string (a marker bit in F2) carries a constant, a byte offset, or a length in F3 rather than a register, and renumbering that value silently corrupts it when it happens to equal a coalesced vreg number.

⟨ *Selfhost: opt cse* 678 ⟩ ≡

```
static unsigned long opt_canon(unsigned long vreg,
    unsigned long n,
                               unsigned long *map)
    {
    if (vreg >= n) return vreg;
    return map[vreg];
}

static void opt_asm_canon(void *node, unsigned
   long n, unsigned long *map) {
    for (void *op = (void *)OPTL(node, OPT_AST_F2)
   ; op;
         op = (void *)OPTL(op, OPT_AST_F4)) {
        unsigned long v = OPTL(op, OPT_AST_F2);
        if (v & 0xffffffffUL)
            OPTL(op, OPT_AST_F2) =
                ((opt_canon((v & 0xffffffffUL) -
   1, n, map) + 1) |
                 (v & 0xffffffff00000000UL));
        /* An immediate (F2 bit 63), symbolic (bit
    62), or string (bit 61)
           operand keeps a constant value, byte
   offset, or length in F3---not a
           register+1---so it must never be
   renumbered.  Only a plain input's
           register (F2) or a memory/output/read-
   write address (F3, register+1 in
           the low half) is an encoded vreg.
   Without this guard CSE remaps such a
           constant through the vreg map whenever
   it collides with a live vreg
           number: the kernel's `.org 2b+%c3' with
    `"i"(sizeof(struct bug_entry))'
```

```
            had its 12 rewritten to 6 in functions
   where vreg 11 was coalesced. */
        if (v >> 61) continue;
        unsigned long a = OPTL(op, OPT_AST_F3);
        if (a & 0xffffffffUL)
            OPTL(op, OPT_AST_F3) =
                ((opt_canon((a & 0xffffffffUL) -
   1, n, map) + 1) |
                 (a & 0xffffffff00000000UL));
    }
}

static void opt_canon_ops(struct Instr *in,
   unsigned long n, unsigned long *map) {
    unsigned long op = in->op;

    if (!(op == IR_CONST || op == IR_CALL || op ==
    IR_BR ||
          op == IR_GLOBAL_REF || op == IR_ALLOCA
   || op == IR_NOP ||
          op == IR_BR_COND || op == IR_STRING_LIT
   || op == IR_VA_START))
        in->src1 = opt_canon(in->src1, n, map);

    if (op == IR_GEP || op == IR_STORE ||
        (op >= 1 && op <= 7) || (op >= 10 && op <=
    15) ||
        (op >= 20 && op <= 27))
        in->src2 = opt_canon(in->src2, n, map);

    if (op == IR_BR_COND)
        in->dest = opt_canon(in->dest, n, map);

    if (op == IR_ASM)
        opt_asm_canon(in->type, n, map);
}

unsigned long kcc_opt_cse(void *fn) {
    struct Func *func = (struct Func *)fn;
    unsigned long count = 0;
    unsigned long n = func->vreg_n;
    if (n == 0) return 0;
    n = n + 1;

    unsigned long *map = (unsigned long *)
   kcc_arena_alloc((long)(n * 8));
    unsigned long *vtab = (unsigned long *)
   kcc_arena_alloc((long)(n * 32)); /* op,s1,s2,
   dest */
    unsigned long *ltab = (unsigned long *)
   kcc_arena_alloc((long)(n * 16)); /* addr,dest
   */
    for (unsigned long i = 0; i < n; i++) map[i] =
    i;

    struct Vec *blocks = (struct Vec *)func->
   blocks;
    unsigned long nb = blocks->count;

    for (unsigned long b = 0; b < nb; b++) {
        unsigned long vcount = 0, lcount = 0;
        struct BB *bb = *(struct BB **)kcc_vec_get
   (blocks, b);
        for (struct Instr *in = bb->first; in; in
   = in->next) {
            opt_canon_ops(in, n, map);
            unsigned long op = in->op;

            if (op == IR_STORE || op == IR_CALL ||
    op == IR_CALL_IND || op == IR_ASM) {
                lcount = 0;
                continue;
            }

            if (op == IR_LOAD) {
                if (opt_is_vol(in->type)) { lcount
     = 0; continue; }
                unsigned long addr = in->src1;
                unsigned long i;
                for (i = 0; i < lcount; i++) {
                    if (ltab[i * 2] == addr) {
                        unsigned long earlier =
   ltab[i * 2 + 1];
                        if (in->dest >= n) break;
                        map[in->dest] = earlier;
                        count++;
                        break;
                    }
                }
                if (i >= lcount) {          /* no
   hit: record this load */
                    ltab[lcount * 2] = addr;
                    ltab[lcount * 2 + 1] = in->
   dest;
                    lcount++;
                }
                continue;
            }

            int pure = (op >= 1 && op <= 16) || (
   op >= 20 && op <= 27) ||
                       op == IR_GEP || (op >= 50
   && op <= 55) || op == IR_FPROUND;
            if (!pure) continue;

            unsigned long s1 = in->src1, s2 = in->
   src2;
            unsigned long i;
            for (i = 0; i < vcount; i++) {
                if (vtab[i * 4] == op && vtab[i *
   4 + 1] == s1 && vtab[i * 4 + 2] == s2) {
                    unsigned long earlier = vtab[i
    * 4 + 3];
                    if (in->dest >= n) break;
                    map[in->dest] = earlier;
                    count++;
                    break;
                }
            }
            if (i >= vcount) {
                vtab[vcount * 4] = op;
                vtab[vcount * 4 + 1] = s1;
                vtab[vcount * 4 + 2] = s2;
                vtab[vcount * 4 + 3] = in->dest;
                vcount++;
            }
        }
    }
    return count;
}
```

□

678 · used in 681b

60.5. Store-to-Load Forwarding

Copy propagation forwards a stored value to a following load of the same address, but only when it is provably safe: within one block it tracks the single most recent store, and forwards to a load only if no store, call, or `va_start` intervened and both the store and the load are the natural eight-byte access (so no truncation can make the values

differ). The forward is recorded in the same replacement map the canonicaliser reads, so subsequent uses pick up the stored register directly; dead-code elimination sweeps the load left behind.

⟨ *Selfhost: opt copyprop* 680a ⟩ ≡

```
unsigned long kcc_opt_copyprop(void *fn) {
    struct Func *func = (struct Func *)fn;
    unsigned long count = 0;
    unsigned long n = func->vreg_n;
    if (n == 0) return 0;
    n = n + 1;

    unsigned long *map = (unsigned long *)
   kcc_arena_alloc((long)(n * 8));

    for (struct BB *bb = func->entry; bb; bb = bb
   ->next) {
        for (unsigned long i = 0; i < n; i++) map[
   i] = i;
        int have = 0;
        unsigned long st_addr = 0, st_val = 0,
   st_size = 0;
        for (struct Instr *in = bb->first; in; in
   = in->next) {
            opt_canon_ops(in, n, map);
            unsigned long op = in->op;
            if (op == IR_STORE) {
                st_addr = in->src2;
                st_val = in->src1;
                st_size = in->dest;
                have = 1;
            } else if (op == IR_LOAD) {
                if (!have) continue;
                if (in->src1 != st_addr) continue;
                if (st_size != 0) continue;
                if (in->src2 != 0) continue;
                if (in->dest >= n) continue;
                map[in->dest] = st_val;
                count++;
            } else if (op == IR_CALL || op ==
   IR_CALL_IND ||
                       op == IR_ASM || op ==
   IR_VA_START) {
                have = 0;
            }
        }
    }
    return count;
}
```

□

680a · used in 681b

60.6. Dead-Code Elimination

Dead-code elimination is mark-and-sweep. The mark pass records every vreg that appears as a source operand—taking care that a folded `IR_CONST`'s immediate, a call's name and argument array, and a branch's block ids are not mistaken for registers, and that an `IR_NOP` left by an earlier sweep references nothing. An `IR_ASM` likewise marks the registers encoded in its operand list, so a load feeding an asm input—whose only consumer is the asm and thus invisible to the field walk—survives the sweep. The sweep pass then turns into a `NOP` any instruction whose destination was never marked and that has no side effect; stores, branches, returns, calls, allocas, inline asm, and volatile loads are always kept.

⟨ *Selfhost: opt dce* 680b ⟩ ≡

```
unsigned long kcc_opt_dce(void *fn) {
    struct Func *func = (struct Func *)fn;
    unsigned long count = 0;
    unsigned long n = func->vreg_n;
    if (n == 0) return 0;
    n = n + 1;

    unsigned long *used = (unsigned long *)
   kcc_arena_alloc((long)(n * 8));
    for (unsigned long i = 0; i < n; i++) used[i]
   = 0;

    struct Vec *blocks = (struct Vec *)func->
   blocks;
    if (blocks == 0) return 0;
    unsigned long nb = blocks->count;

    /* Mark. */
    for (unsigned long b = 0; b < nb; b++) {
        struct BB *bb = *(struct BB **)kcc_vec_get
   (blocks, b);
        for (struct Instr *in = bb->first; in; in
   = in->next) {
            unsigned long op = in->op;
            if (op == IR_NOP) continue;

            /* src1, unless it is not a register
   for this opcode */
            if (!(op == IR_CONST || op == IR_CALL
   || op == IR_BR ||
                  op == IR_GLOBAL_REF)) {
                unsigned long s1 = in->src1;
                if (s1 < n) used[s1] = 1;
            }

            /* src2 */
            if (op == IR_BR_COND || op == IR_BR ||
    op == IR_CONST) {
                /* no src2 register */
            } else if (op == IR_CALL || op ==
   IR_CALL_IND) {
                unsigned long *args = (unsigned
   long *)in->src2;
                if (args) {
                    unsigned long cnt = args[0] &
   0xffffffffUL;
                    for (unsigned long i = 0; i <
   cnt; i++) {
                        unsigned long v = args[1 +
    i * 2];
                        if (v < n) used[v] = 1;
                    }
                }
            } else {
                unsigned long s2 = in->src2;
                if (s2 < n) used[s2] = 1;
            }

            /* br_cond condition is in dest */
            if (op == IR_BR_COND) {
                unsigned long d = in->dest;
                if (d < n) used[d] = 1;
            }

            /* ret value in src1; store marks both
    (redundant but faithful) */
```

```
            if (op == IR_RET) {
                unsigned long s1 = in->src1;
                if (s1 < n) used[s1] = 1;
            } else if (op == IR_STORE) {
                unsigned long s1 = in->src1, s2 =
    in->src2;
                if (s1 < n) used[s1] = 1;
                if (s2 < n) used[s2] = 1;
            } else if (op == IR_ASM) {
                /* the input values and output
    addresses live in the operand
                   list off the AST node,
    invisible to the ordinary field
                   walk---mark them so an asm-only
     load survives the sweep */
                for (void *o = (void *)OPTL(in->
    type, OPT_AST_F2); o;
                     o = (void *)OPTL(o,
    OPT_AST_F4)) {
                    unsigned long v = OPTL(o,
    OPT_AST_F2) & 0xffffffffUL;
                    if (v && v - 1 < n) used[v -
    1] = 1;
                    unsigned long a = OPTL(o,
    OPT_AST_F3) & 0xffffffffUL;
                    if (a && a - 1 < n) used[a -
    1] = 1;
                }
            }
        }
    }

    /* Sweep. */
    for (unsigned long b = 0; b < nb; b++) {
        struct BB *bb = *(struct BB **)kcc_vec_get
    (blocks, b);
        for (struct Instr *in = bb->first; in; in
    = in->next) {
            unsigned long op = in->op;
            if (op == IR_STORE || op == IR_BR ||
    op == IR_BR_COND ||
                op == IR_RET || op == IR_CALL ||
    op == IR_CALL_IND ||
                op == IR_NOP || op == IR_ALLOCA ||
                op == IR_VA_START || op == IR_ASM)
                continue;
            if (op == IR_LOAD) {
                struct Type *t = (struct Type *)in
    ->type;
                if ((unsigned long)t >= 0x1000 &&
    (t->quals & 2)) continue;
            }
            if (in->dest >= n) continue;
            if (used[in->dest]) continue;
            in->op = IR_NOP;
            count++;
        }
    }
    return count;
}
```

□

680b · used in 681b

60.7. The Pass Manager

The manager runs the passes over every function to a fixed point: for each function it repeats folding, strength reduction, CSE, copy propagation, and DCE until a round makes no change, capped at ten rounds for safety. Running them together is what lets each unblock the next—a fold exposes a subexpression, a strength-reduced shift exposes a constant, a forwarded store leaves a load dead.

⟨ *Selfhost: opt run* 681a ⟩ ≡

```
void kcc_opt_run_func(void *fn) {
    for (int iter = 0; iter < 10; iter++) {
        unsigned long changed = 0;
        changed += kcc_opt_constfold(fn);
        changed += kcc_opt_strength(fn);
        changed += kcc_opt_cse(fn);
        changed += kcc_opt_copyprop(fn);
        changed += kcc_opt_dce(fn);
        if (changed == 0) break;
    }
}

void kcc_opt_run(void) {
    struct Vec *funcs = (struct Vec *)kcc_ir_funcs
    ;
    if (funcs == 0) return;
    unsigned long cnt = funcs->count;
    for (unsigned long i = 0; i < cnt; i++) {
        void *fn = *(void **)kcc_vec_get(funcs, i)
    ;
        kcc_opt_run_func(fn);
    }
}
```

□

681a · used in 681b

60.8. Assembling the Translation Unit

⟨ *Selfhost: opt* 681b ⟩ ≡

```
⟨ Selfhost: opt preamble 675 ⟩
⟨ Selfhost: opt constfold 676 ⟩
⟨ Selfhost: opt strength 677 ⟩
⟨ Selfhost: opt cse 678 ⟩
⟨ Selfhost: opt copyprop 680a ⟩
⟨ Selfhost: opt dce 680b ⟩
⟨ Selfhost: opt run 681a ⟩
```

□

681b · used in 35b

60.9. Verification

`make selfhost-opt` runs the drop-in loop. Because every optimized program passes through this fixed-point loop, a green code-generation suite means kcc compiled its own middle end—folding, strength reduction, CSE, forwarding, and DCE—so faithfully that the generated code is byte-for-byte unchanged.

⟨ *Selfhost: opt verify* 681c ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-opt - compile kcc's optimizer in C
    with kcc, relink, run suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
```

```
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/opt.c with
   kcc"
$BASE/kcc -c "$SH/opt.c" -o "$SH/opt_c.o" >/dev/
   null

echo "[2/3] linking $BASE/kcc_selfhost_opt (opt.o
   -> kcc-compiled opt_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "opt" ]; then OBJ="$OBJ $SH/opt_c.
   o"; else OBJ="$OBJ $BASE/src/${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_opt $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted optimizer"
cp $BASE/kcc "$SH/kcc_asm_backup_opt"
cp $BASE/kcc_selfhost_opt $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_opt" $BASE/kcc
exit $status
```

□

681c · used in 35c · uses _main 621, 629

61. Self-Hosting: The Lexer Tables in C

Chapter 16 split the tokenizer's static tables into their own object so they could be ported independently of the scanner's hot loop. This is that port: the keyword registration table and its initializer, and the token-name table and its lookup. It is almost pure data—two tables and two short functions over them—so it is the safest kind of drop-in, yet it is exercised constantly: `kcc_lex_kw_init` builds the hash table every identifier is checked against, and `kcc_lex_type_name` supplies the token spellings that diagnostics and `--dump-tokens` print.

61.1. Keyword Registration

`kcc_lex_kw_init` creates a 128-slot hash table, stores it in the lexer's global `kcc_kw_table`, and inserts every keyword—its text, its byte length, and its token code. The 66 entries are the 44 C17 keywords plus the GNU spellings the compiler accepts: the `__x__`/`__x` forms alias the standard token codes (`__const`→14, `__inline`→26, `__alignof__`→45, …), while `__attribute__`, `typeof`, `asm`, and `__extension__` take their own codes 54–57 and the GNU 128-bit integer keyword `__int128` takes code 58. The lengths are carried explicitly, exactly as the assembly table did, so the hash keys match byte-for-byte.

⟨ *Selfhost: lexer-tables kw* 683a ⟩ ≡

```
struct KwEntry { const char *name; unsigned long
   len; unsigned long token; };

static struct KwEntry kw_reg[66] = {
    {"auto", 4, 10}, {"break", 5, 11}, {"case", 4,
    12}, {"char", 4, 13},
    {"const", 5, 14}, {"continue", 8, 15}, {"
   default", 7, 16}, {"do", 2, 17},
    {"double", 6, 18}, {"else", 4, 19}, {"enum",
   4, 20}, {"extern", 6, 21},
    {"float", 5, 22}, {"for", 3, 23}, {"goto", 4,
   24}, {"if", 2, 25},
    {"inline", 6, 26}, {"int", 3, 27}, {"long", 4,
    28}, {"register", 8, 29},
    {"restrict", 8, 30}, {"return", 6, 31}, {"
   short", 5, 32}, {"signed", 6, 33},
    {"sizeof", 6, 34}, {"static", 6, 35}, {"struct
   ", 6, 36}, {"switch", 6, 37},
    {"typedef", 7, 38}, {"union", 5, 39}, {"
   unsigned", 8, 40}, {"void", 4, 41},
    {"volatile", 8, 42}, {"while", 5, 43}, {"
   _Alignas", 8, 44},
    {"_Alignof", 8, 45}, {"_Atomic", 7, 46}, {"
   _Bool", 5, 47},
    {"_Complex", 8, 48}, {"_Generic", 8, 49}, {"
   _Imaginary", 10, 50},
    {"_Noreturn", 9, 51}, {"_Static_assert", 14,
   52}, {"_Thread_local", 13, 53},
    {"__attribute__", 13, 54}, {"__attribute", 11,
    54},
    {"typeof", 6, 55}, {"__typeof__", 10, 55}, {"
   __typeof", 8, 55},
    {"asm", 3, 56}, {"__asm__", 7, 56}, {"__asm",
   5, 56},
    {"__extension__", 13, 57},
    {"__inline", 8, 26}, {"__inline__", 10, 26},
    {"__const", 7, 14}, {"__const__", 9, 14},
    {"__signed", 8, 33}, {"__signed__", 10, 33},
    {"__volatile", 10, 42}, {"__volatile__", 12,
   42},
    {"__restrict", 10, 30}, {"__restrict__", 12,
   30},
    {"__int128", 8, 58},
    {"__alignof__", 11, 45}, {"__alignof", 9, 45}
};

void *kcc_ht_init(unsigned long capacity);
void kcc_ht_put(void *ht, const void *key,
   unsigned long keylen,
                unsigned long value);
extern void *kcc_kw_table;

void kcc_lex_kw_init(void) {
    void *ht = kcc_ht_init(128);
    kcc_kw_table = ht;
    for (unsigned long i = 0; i < 66; i++)
        kcc_ht_put(ht, kw_reg[i].name, kw_reg[i].
   len, kw_reg[i].token);
}
```

□

683a · used in 684a

61.2. Token Names

`kcc_lex_type_name` maps a token code to a readable name through a flat 108-entry table indexed by the code. The gaps in the code space—6–9, and 54–57/59 between the keywords and the punctuators (code 58 is `__int128`)—are null, and both an out-of-range code and a null slot return `"unknown"`. The punctuator names follow Clang's dump convention (`l_paren`, `semi`, `plusplus`, …); the entries are listed positionally so the index is the token code.

⟨ *Selfhost: lexer-tables names* 683b ⟩ ≡

```
static const char *tok_names[108] = {
    "eof", "identifier", "number", "char_constant"
   , "string_literal",
    "unknown",
    0, 0, 0, 0,                                 /*
   6-9 reserved */
    "auto", "break", "case", "char", "const", "
   continue", "default", "do",
    "double", "else", "enum", "extern", "float", "
   for", "goto", "if",
    "inline", "int", "long", "register", "restrict
   ", "return", "short",
    "signed", "sizeof", "static", "struct", "
   switch", "typedef", "union",
    "unsigned", "void", "volatile", "while", "
   _Alignas", "_Alignof",
    "_Atomic", "_Bool", "_Complex", "_Generic", "
   _Imaginary", "_Noreturn",
    "_Static_assert", "_Thread_local",
    0, 0, 0, 0, "__int128", 0,                  /*
   54-57,59 reserved; 58 __int128 */
```

```
    "l_paren", "r_paren", "l_brace", "r_brace", "
   l_bracket", "r_bracket",
    "semi", "comma", "dot", "arrow", "plusplus", "
   minusminus", "amp", "star",
    "plus", "minus", "tilde", "exclaim", "slash",
   "percent", "lessless",
    "greatergreater", "less", "greater", "
   lessequal", "greaterequal",
    "equalequal", "exclaimequal", "caret", "pipe",
    "ampamp", "pipepipe",
    "question", "colon", "equal", "starequal", "
   slashequal", "percentequal",
    "plusequal", "minusequal", "lesslessequal", "
   greatergreaterequal",
    "ampequal", "caretequal", "pipeequal", "hash",
    "hashhash", "ellipsis"
};

const char *kcc_lex_type_name(unsigned long type)
   {
    if (type >= 108) return "unknown";
    const char *n = tok_names[type];
    return n ? n : "unknown";
}
```

□

683b · used in 684a

61.3. Assembling the Translation Unit

⟨ *Selfhost: lexer-tables* 684a ⟩ ≡

⟨ *Selfhost: lexer-tables kw* 683a ⟩
⟨ *Selfhost: lexer-tables names* 683b ⟩

□

684a · used in 35d

61.4. Verification

`make selfhost-lexer-tables` runs the drop-in loop. Because every identifier the compiler reads is checked against the keyword table this file builds, a green code-generation suite means kcc compiled its own keyword recognition correctly; the diagnostic suite additionally exercises `kcc_lex_type_name` through the token spellings it prints.

⟨ *Selfhost: lexer-tables verify* 684b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-lexer-tables - compile kcc's lexer
    tables in C with kcc,
# relink, run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/lexer_tables.
   c with kcc"
$BASE/kcc -c "$SH/lexer_tables.c" -o "$SH/
   lexer_tables_c.o" >/dev/null

echo "[2/3] linking $BASE/
   kcc_selfhost_lexer_tables (lexer_tables.o -> C)
   "
OBJ=""
for f in $sources; do
    if [ "$f" = "lexer_tables" ]; then OBJ="$OBJ
   $SH/lexer_tables_c.o"; else OBJ="$OBJ $BASE/src
   /${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_lexer_tables $OBJ -
   lSystem -syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running codegen + diagtest with the
   self-hosted lexer tables"
cp $BASE/kcc "$SH/kcc_asm_backup_lexer_tables"
cp $BASE/kcc_selfhost_lexer_tables $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh; s1=$?
bash $BASE/tests/diagtest.sh;      s2=$?
cp "$SH/kcc_asm_backup_lexer_tables" $BASE/kcc
[ $s1 -eq 0 ] && [ $s2 -eq 0 ]
```

□

684b · used in 35e · uses _main 621, 629

62. Self-Hosting: The Linker Information in C

The linker-information module of Chapter 41 is the seam between code generation and object emission: the code generator records, as it emits, every symbol it defines or references and every place that needs relocating, and the Mach-O writer serialises those tables into the object file. Porting it puts that bookkeeping—the symbol vector, the relocation vector, and the extern-deduplication hash table—through kcc-compiled C, exercised on every single compilation the suite runs.

62.1. Tables and State

The module owns four globals: the two vectors, the extern hash table, and the running size of the global data section. A relocation entry is 24 bytes (a code offset, a type, a symbol index); a symbol entry is 32 bytes (a name pointer, a section number, a value, and Mach-O flags). The port declares the vector header so it can build the vectors exactly as the assembly did—allocating the header and its backing store directly rather than through `vec_init`, so the initial capacity of 32 matches—and a symbol view for the in-place upgrade of an extern to a defined global.

⟨ *Selfhost: linker preamble* 685a ⟩ ≡

```
struct Vec {
    void *data;              /* VEC_DATA,     0
   */
    unsigned long count;     /* VEC_COUNT,    8
   */
    unsigned long cap;       /* VEC_CAP,      16
   */
    unsigned long elem_size; /* VEC_ELEMSIZE, 24
   */
};

struct LinkSym {
    const char *name;        /* SYM_NAME,     0  */
    unsigned long section;   /* SYM_SECTION,  8  */
    unsigned long value;     /* SYM_VALUE,    16 */
    unsigned long flags;     /* SYM_FLAGS,    24 */
};

void *kcc_arena_alloc(long size);
void kcc_vec_push(void *vec, const void *elem);
void *kcc_ht_init(unsigned long capacity);
unsigned long kcc_ht_get(void *ht, const void *key
   , unsigned long keylen);
void kcc_ht_put(void *ht, const void *key,
   unsigned long keylen,
                unsigned long value);
unsigned long kcc_strlen(const char *s);

void *kcc_linker_relocs = 0;
void *kcc_linker_syms = 0;
void *kcc_linker_extern_ht = 0;
unsigned long kcc_globals_size = 0;

/* Mach-O symbol flags. */
#define N_EXT  0x01
#define N_SECT 0x0E
```

□

685a · used in 686b

62.2. Initialization

`kcc_linker_init` builds the relocation vector (24-byte entries), the symbol vector (32-byte entries), each with an initial capacity of 32 and its own arena-backed store, and a 64-slot extern hash table. `vec_push` grows either vector when it fills, so the initial capacity affects only when the first growth happens, not the tables' contents.

⟨ *Selfhost: linker init* 685b ⟩ ≡

```
void kcc_linker_init(void) {
    struct Vec *rv = (struct Vec *)kcc_arena_alloc
   (32);   /* VEC_SIZE */
    kcc_linker_relocs = rv;
    rv->elem_size = 24;
    rv->cap = 32;
    rv->count = 0;
    rv->data = kcc_arena_alloc(768);
            /* 32 * 24 */

    struct Vec *sv = (struct Vec *)kcc_arena_alloc
   (32);
    kcc_linker_syms = sv;
    sv->elem_size = 32;
    sv->cap = 32;
    sv->count = 0;
    sv->data = kcc_arena_alloc(1024);
            /* 32 * 32 */

    kcc_linker_extern_ht = kcc_ht_init(64);
}
```

□

685b · used in 686b

62.3. Adding Symbols and Relocations

Each adder builds the entry as a small stack array—in the exact field order of the record—and pushes it onto its vector. `add_sym` returns the new symbol's zero-based index (the vector's count before the push).

⟨ *Selfhost: linker add* 685c ⟩ ≡

```
unsigned long kcc_linker_add_sym(const char *name,
    unsigned long section,
                                 unsigned long
   value, unsigned long flags) {
    unsigned long entry[4];
    entry[0] = (unsigned long)name;   /* SYM_NAME
   */
    entry[1] = section;               /*
   SYM_SECTION */
    entry[2] = value;                 /* SYM_VALUE
    */
    entry[3] = flags;                 /* SYM_FLAGS
    */
    struct Vec *v = (struct Vec *)kcc_linker_syms;
    unsigned long idx = v->count;
```

```
    kcc_vec_push(v, entry);
    return idx;
}

void kcc_linker_add_reloc(unsigned long offset,
   unsigned long type,
                          unsigned long sym_index)
    {
    unsigned long entry[3];
    entry[0] = offset;                /*
   RELOC_OFFSET */
    entry[1] = type;                  /*
   RELOC_TYPE */
    entry[2] = sym_index;             /* RELOC_SYM
    */
    kcc_vec_push(kcc_linker_relocs, entry);
}
```

□

685c · used in 686b

62.4. External and Global Symbols

`kcc_linker_extern` returns the index of an undefined external symbol, adding an `N_EXT` entry the first time a name is seen and deduplicating through the hash table (whose stored value is the index plus one, so zero can mean "absent"). `kcc_linker_global_sym` registers a *defined* global in section 2: if the name is already defined it returns the index untouched; if it was previously an extern it upgrades the entry in place, assigning it the current data-section offset and advancing that offset by the requested size; otherwise it adds a fresh defined symbol. Both dedup through the same hash table, so repeated references and pre-registration are safe.

⟨ *Selfhost: linker externs* 686a ⟩ ≡

```
unsigned long kcc_linker_extern(const char *name)
   {
    unsigned long len = kcc_strlen(name);
    unsigned long r = kcc_ht_get(
   kcc_linker_extern_ht, name, len);
    if (r) return r - 1;
    unsigned long idx = kcc_linker_add_sym(name,
   0, 0, N_EXT);
    kcc_ht_put(kcc_linker_extern_ht, name, len,
   idx + 1);
    return idx;
}

unsigned long kcc_linker_global_sym(const char *
   name, unsigned long size) {
    unsigned long len = kcc_strlen(name);
    unsigned long r = kcc_ht_get(
   kcc_linker_extern_ht, name, len);
    if (r) {
        unsigned long idx = r - 1;
        struct Vec *sv = (struct Vec *)
   kcc_linker_syms;
        struct LinkSym *s = (struct LinkSym *)((
   char *)sv->data + idx * 32);
        if (s->section != 0) return idx;    /*
   already defined */
        s->section = 2;
        s->value = kcc_globals_size;
        kcc_globals_size += size;
        return idx;
    }
    unsigned long value = kcc_globals_size;
    unsigned long idx = kcc_linker_add_sym(name,
   2, value, N_SECT | N_EXT);
    kcc_globals_size += size;
    kcc_ht_put(kcc_linker_extern_ht, name, len,
   idx + 1);
    return idx;
}
```

□

686a · used in 686b

62.5. Assembling the Translation Unit

⟨ *Selfhost: linker* 686b ⟩ ≡

```
⟨ Selfhost: linker preamble 685a ⟩
⟨ Selfhost: linker init 685b ⟩
⟨ Selfhost: linker add 685c ⟩
⟨ Selfhost: linker externs 686a ⟩
```

□

686b · used in 35f

62.6. Verification

`make selfhost-linker` runs the drop-in loop. Because every compilation registers its function and its referenced externs and globals through these tables and the Mach-O writer serialises them, a green code-generation suite—each test of which compiles, links, and runs a program—means kcc compiled its own symbol and relocation bookkeeping correctly.

⟨ *Selfhost: linker verify* 686c ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-linker - compile kcc's linker info
    in C with kcc, relink,
# run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/linker.c with
    kcc"
$BASE/kcc -c "$SH/linker.c" -o "$SH/linker_c.o" >/
   dev/null

echo "[2/3] linking $BASE/kcc_selfhost_linker (
   linker.o -> kcc-compiled linker_c.o)"
OBJ=""
for f in $sources; do
    if [ "$f" = "linker" ]; then OBJ="$OBJ $SH/
   linker_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
   fi
done
ld -o $BASE/kcc_selfhost_linker $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64
```

```
echo "[3/3] running the codegen suite with the
   self-hosted linker info"
cp $BASE/kcc "$SH/kcc_asm_backup_linker"
cp $BASE/kcc_selfhost_linker $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_linker" $BASE/kcc
exit $status
```

□

686c · used in 35g · uses _main 621, 629

63. Self-Hosting: The Statement Parser in C

The statement parser of Chapter 23 is the first slice of the recursive-descent *parser* to port. It is the layer that turns a token stream into the control-flow skeleton of a function—compound blocks, the selection and iteration statements, jumps, labels, and inline assembly—so every function the code-generation suite compiles is shaped by it. It stands on the parser's token API (the current-token globals, the token advance, and the "expect" helper) and calls out to the expression and declaration parsers, which stay in assembly; the port is the statement grammar itself, node-building walk for node-building walk.

63.1. Nodes, Tokens, and the Parser API

An AST node is the 64-byte record Chapter 20 fixed: a tag, a source line and column, and five general-purpose fields `f0`–`f4` (`f4` is the sibling link that threads statement and declarator lists). The port declares that struct, the token codes and node tags it dispatches on, and the parser primitives as externs: the current token's type, text pointer, and length; `pp_next` to advance; `parse_expect` to consume an expected token or diagnose; `ast_alloc` to make a node; and the expression/declaration entry points. `strlit_gather` returns two values (the gathered string pointer and its length), so it is declared as returning a 16-byte struct, which the ABI hands back in the first two registers exactly as the assembly leaves them.

⟨ *Selfhost: statements preamble* 689 ⟩ ≡

```
struct Node {
    unsigned long tag;   /* AST_TAG,  0  */
    unsigned long line;  /* AST_LINE, 8  */
    unsigned long col;   /* AST_COL,  16 */
    unsigned long f0;    /* AST_F0,   24 */
    unsigned long f1;    /* AST_F1,   32 */
    unsigned long f2;    /* AST_F2,   40 */
    unsigned long f3;    /* AST_F3,   48 */
    unsigned long f4;    /* AST_F4,   56 */
};

struct StrLit { void *ptr; unsigned long len; };
    /* returned in x0:x1 */

extern unsigned long kcc_tok_type;
extern const char *kcc_tok_start;
extern unsigned long kcc_tok_len;
int kcc_memcmp(const void *a, const void *b,
   unsigned long n);

void kcc_pp_next(void);
void kcc_parse_expect(unsigned long tok_type);
void *kcc_ast_alloc(unsigned long tag);
void *kcc_parse_expr(void);
void *kcc_parse_assign_expr(void);
void *kcc_parse_cond_expr(void);
void *kcc_parse_binary_expr(unsigned long min_prec
   );
void *kcc_parse_postfix_tail(void *lhs);
unsigned long kcc_prec_lookup(unsigned long op);
void *kcc_parse_decl(void);
unsigned long kcc_parse_is_type_token(void);
static int starts_decl(unsigned long t);
struct StrLit kcc_strlit_gather(void);
void *kcc_strtab_intern(const void *ptr, unsigned
   long len);
void kcc_typedef_push_scope(void);
void kcc_typedef_pop_scope(void);

/* Token codes (Chapter: lexer). */
enum {
    TOK_EOF = 0, TOK_IDENT = 1, TOK_BREAK = 11,
   TOK_CASE = 12,
    TOK_CONTINUE = 15, TOK_DEFAULT = 16, TOK_DO =
   17, TOK_ELSE = 19,
    TOK_EXTERN = 21, TOK_FOR = 23, TOK_GOTO = 24,
   TOK_IF = 25,
    TOK_REGISTER = 29, TOK_RETURN = 31, TOK_STATIC
    = 35, TOK_SWITCH = 37,
    TOK_TYPEDEF = 38, TOK_WHILE = 43,
   TOK_STATIC_ASSERT = 52, TOK_ASM = 56,
    TOK_AUTO = 10, TOK_ATTRIBUTE = 54,
    TOK_LPAREN = 60, TOK_RPAREN = 61, TOK_LBRACE =
    62, TOK_RBRACE = 63,
    TOK_LBRACKET = 64, TOK_RBRACKET = 65, TOK_SEMI
    = 66, TOK_COMMA = 67,
    TOK_STAR = 73, TOK_QUESTION = 92, TOK_COLON =
   93,
    TOK_ASSIGN = 94, TOK_OR_ASSIGN = 104,
   TOK_ELLIPSIS = 107
};

/* AST node tags (Chapter: parser). */
enum {
    AST_COMPOUND = 30, AST_EXPR_STMT = 31,
   AST_IF_STMT = 32,
    AST_SWITCH_STMT = 33, AST_WHILE_STMT = 34,
   AST_DO_STMT = 35,
    AST_FOR_STMT = 36, AST_GOTO_STMT = 37,
   AST_CONTINUE_STMT = 38,
    AST_BREAK_STMT = 39, AST_RETURN_STMT = 40,
   AST_LABEL_STMT = 41,
    AST_CASE_STMT = 42, AST_DEFAULT_STMT = 43,
   AST_NULL_STMT = 44,
    AST_IDENT_EXPR = 50, AST_BINARY_EXPR = 54,
   AST_ASSIGN_EXPR = 56,
    AST_COND_EXPR = 57, AST_COMMA_EXPR = 64,
    AST_ASM_STMT = 74, AST_ASM_OPERAND = 75
};

/* Forward declarations for the module's own
   functions. */
void kcc_parse_skip_attributes(void);
void *kcc_parse_stmt(void);
void *kcc_parse_compound(void);
void *kcc_parse_if(void);
void *kcc_parse_switch(void);
void *kcc_parse_while(void);
void *kcc_parse_do(void);
void *kcc_parse_for(void);
void *kcc_parse_goto(void);
void *kcc_parse_return(void);
void *kcc_parse_case(void);
void *kcc_parse_default(void);
void *kcc_parse_expr_rest(void *lhs);
void *kcc_parse_asm_extended(const char *tmpl,
   unsigned long len);
```

```
void kcc_parse_asm_opsec(void *asm_stmt, unsigned
   long is_output);
void kcc_parse_asm_clobbers(void);
```

□

689 · used in 694b

63.2. The Dispatcher

`kcc_parse_stmt` chooses a handler by the leading token: a brace opens a compound; a bare semicolon is the null statement; the keywords route to their handlers; `continue` and `break` are built inline (each followed by its semicolon); `asm` captures a template and hands off to the extended form. An identifier needs one-token lookahead—interned in case it is a label, then consumed to see whether a colon follows. If it does, the statement is a label; otherwise the identifier is the start of an expression statement, and parsing resumes from the already-built identifier node through `postfix_tail` and `expr_rest`. A label may carry a GNU attribute between its colon and its statement—`done: __attribute__((unused));`, which the kernel's cleanup/guard machinery emits on exit labels that a given path never reaches—so any `__attribute__` run is skipped before the labelled statement is parsed.

A GNU attribute may also *begin* a statement. The dominant case is `__attribute__((__fallthrough__));`—the kernel's `fallthrough;` placed between `switch` cases (it appears in the byte-size-dispatched `__memcpy`/page-clearing helpers pulled in through `efi.h`). It is advisory only, so once the attribute run is consumed a trailing semicolon makes it a null statement. The same leading position can prefix a declaration (`__attribute__((unused)) int x;`); the compound loop cannot distinguish the two without first consuming the run—`starts_decl` does not fire on `__attribute__`—so it routes both here, and after skipping the attributes we hand a declaration off to the declaration parser and anything else back to the statement dispatcher.

⟨ *Selfhost: statements dispatch* 690 ⟩ ≡

```
void *kcc_parse_stmt(void) {
    unsigned long t = kcc_tok_type;
    if (t == TOK_LBRACE) return kcc_parse_compound
   ();
    if (t == TOK_SEMI) { kcc_pp_next(); return
   kcc_ast_alloc(AST_NULL_STMT); }
    if (t == TOK_ATTRIBUTE) {
        /* A GNU attribute at statement position.
    `__attribute__((__fallthrough__));'
           is the kernel's `fallthrough;' between
   switch cases---advisory only (kcc
           does not warn on implicit fallthrough),
    so it becomes a null statement.
           An attribute that instead prefixes a
   declaration
           (`__attribute__((unused)) int x;')
   hands off to the declaration parser
           once the attributes are consumed---the
   compound loop routes a leading
           attribute here, not to the declaration
   path, because `starts_decl'
           cannot tell the two apart without
   consuming the run first. */
        while (kcc_tok_type == TOK_ATTRIBUTE)
kcc_parse_skip_attributes();
        if (kcc_tok_type == TOK_SEMI) {
kcc_pp_next(); return kcc_ast_alloc(
AST_NULL_STMT); }
        if (kcc_parse_is_type_token() ||
starts_decl(kcc_tok_type)) return
kcc_parse_decl();
        return kcc_parse_stmt();
    }
    if (t == TOK_IF) return kcc_parse_if();
    if (t == TOK_SWITCH) return kcc_parse_switch()
;
    if (t == TOK_WHILE) return kcc_parse_while();
    if (t == TOK_DO) return kcc_parse_do();
    if (t == TOK_FOR) return kcc_parse_for();
    if (t == TOK_GOTO) return kcc_parse_goto();
    if (t == TOK_CONTINUE) {
        kcc_pp_next();
        void *n = kcc_ast_alloc(AST_CONTINUE_STMT)
;
        kcc_parse_expect(TOK_SEMI);
        return n;
    }
    if (t == TOK_BREAK) {
        kcc_pp_next();
        void *n = kcc_ast_alloc(AST_BREAK_STMT);
        kcc_parse_expect(TOK_SEMI);
        return n;
    }
    if (t == TOK_RETURN) return kcc_parse_return()
;
    if (t == TOK_CASE) return kcc_parse_case();
    if (t == TOK_DEFAULT) return kcc_parse_default
();
    if (t == TOK_ASM) {
        kcc_pp_next();                          /*
consume 'asm' */
        while (kcc_tok_type != TOK_LPAREN &&
kcc_tok_type != TOK_EOF)
            kcc_pp_next();                      /*
skip volatile/inline/goto */
        kcc_parse_expect(TOK_LPAREN);
        struct StrLit tmpl = kcc_strlit_gather();
        void *n = kcc_parse_asm_extended((const
char *)tmpl.ptr, tmpl.len);
        kcc_parse_expect(TOK_SEMI);
        return n;
    }
    if (t == TOK_IDENT) {
        void *name = kcc_strtab_intern(
kcc_tok_start, kcc_tok_len);
        kcc_pp_next();                          /*
consume identifier */
        if (kcc_tok_type == TOK_COLON) {
            kcc_pp_next();                      /*
consume ':' */
            /* A label may carry a GNU attribute
before its statement, e.g.
               `done: __attribute__((unused));' (
the kernel's cleanup/guard
               macros emit exactly this on an
unreachable exit label). */
            while (kcc_tok_type == TOK_ATTRIBUTE)
kcc_parse_skip_attributes();
            void *inner = kcc_parse_stmt();
            struct Node *n = (struct Node *)
kcc_ast_alloc(AST_LABEL_STMT);
            n->f0 = (unsigned long)name;
            n->f1 = (unsigned long)inner;
```

```
            return n;
        }
        struct Node *id = (struct Node *)
    kcc_ast_alloc(AST_IDENT_EXPR);
        id->f0 = (unsigned long)name;
        void *e = kcc_parse_postfix_tail(id);
        e = kcc_parse_expr_rest(e);
        struct Node *n = (struct Node *)
    kcc_ast_alloc(AST_EXPR_STMT);
        n->f0 = (unsigned long)e;
        kcc_parse_expect(TOK_SEMI);
        return n;
    }
    /* Expression statement. */
    void *e = kcc_parse_expr();
    struct Node *n = (struct Node *)kcc_ast_alloc(
    AST_EXPR_STMT);
    n->f0 = (unsigned long)e;
    kcc_parse_expect(TOK_SEMI);
    return n;
}
```

□

690 · used in 694b

63.3. Expression Continuation

When the dispatcher consumed an identifier for label lookahead and found no colon, `kcc_parse_expr_rest` finishes the expression from that left-hand side. An assignment operator takes the whole thing as its left operand and then falls straight to the comma check (an assignment is a valid comma operand); otherwise it climbs binary operators by precedence, then a possible ternary, then the comma operator—the same shape the expression parser uses, replayed here because the leading identifier was already consumed.

⟨ *Selfhost: statements exprrest* 691a ⟩ ≡

```
void *kcc_parse_expr_rest(void *lhs) {
    struct Node *cur = (struct Node *)lhs;
    unsigned long t = kcc_tok_type;
    if (t >= TOK_ASSIGN && t <= TOK_OR_ASSIGN) {
        unsigned long op = t;
        kcc_pp_next();
        void *rhs = kcc_parse_assign_expr();
        struct Node *n = (struct Node *)
    kcc_ast_alloc(AST_ASSIGN_EXPR);
        n->f0 = op;
        n->f1 = (unsigned long)cur;
        n->f2 = (unsigned long)rhs;
        cur = n;
    } else {
        for (;;) {
            unsigned long prec = kcc_prec_lookup(
    kcc_tok_type);
            if (prec == 0) break;
            unsigned long op = kcc_tok_type;
            kcc_pp_next();
            void *rhs = kcc_parse_binary_expr(prec
     + 1);
            struct Node *n = (struct Node *)
    kcc_ast_alloc(AST_BINARY_EXPR);
            n->f0 = op;
            n->f1 = (unsigned long)cur;
            n->f2 = (unsigned long)rhs;
            cur = n;
        }
        if (kcc_tok_type == TOK_QUESTION) {
            kcc_pp_next();
            /* GNU `a ?: b': the middle operand is
     omitted and the condition's
               value is reused.  Record it as f1
    == 0 so the IR evaluates the
               condition once. */
            void *then = 0;
            if (kcc_tok_type != TOK_COLON)
                then = kcc_parse_expr();
            kcc_parse_expect(TOK_COLON);
            void *els = kcc_parse_cond_expr();
            struct Node *n = (struct Node *)
    kcc_ast_alloc(AST_COND_EXPR);
            n->f0 = (unsigned long)cur;
            n->f1 = (unsigned long)then;
            n->f2 = (unsigned long)els;
            cur = n;
        }
    }
    while (kcc_tok_type == TOK_COMMA) {
        kcc_pp_next();
        void *r = kcc_parse_assign_expr();
        struct Node *n = (struct Node *)
    kcc_ast_alloc(AST_COMMA_EXPR);
        n->f0 = (unsigned long)cur;
        n->f1 = (unsigned long)r;
        cur = n;
    }
    return cur;
}
```

□

691a · used in 694b

63.4. Inline Assembly

The extended inline-asm grammar carries, after the template, up to four colon-separated lists: outputs, inputs, clobbers, and—for `asm goto`—the target labels. Each label name is interned and hung off the statement's `f3` chain, where the IR resolves it to a block and the back end expands `%lN`/`%l[name]` to that block's label. `parse_asm_opsec` reads one list, building an `AST_ASM_OPERAND` per operand—an optional `[name]` skipped, a string constraint, and a parenthesised expression—and appending it to the statement's operand chain through `f4`. `parse_asm_clobbers` consumes the clobber strings, and `parse_asm_extended` assembles the `AST_ASM_STMT` and drives the colon sections, closing on the right paren.

⟨ *Selfhost: statements asm* 691b ⟩ ≡

```
void kcc_parse_asm_opsec(void *asm_stmt, unsigned
    long is_output) {
    struct Node *as = (struct Node *)asm_stmt;
    for (;;) {
        unsigned long t = kcc_tok_type;
        if (t == TOK_COLON || t == TOK_RPAREN || t
    == TOK_EOF) return;
        void *symname = 0;
    /* optional [name] for %[name] */
        if (t == TOK_LBRACKET) {
            kcc_pp_next();
            if (kcc_tok_type == TOK_IDENT)
                symname = kcc_strtab_intern(
    kcc_tok_start, kcc_tok_len);
```

```
            while (kcc_tok_type != TOK_EOF &&
    kcc_tok_type != TOK_RBRACKET)
                kcc_pp_next();
            if (kcc_tok_type == TOK_EOF) return;
            kcc_pp_next();                          /*
     consume ']' */
        }
        struct StrLit c = kcc_strlit_gather();
        kcc_parse_expect(TOK_LPAREN);
        void *expr = kcc_parse_assign_expr();
        kcc_parse_expect(TOK_RPAREN);
        struct Node *op = (struct Node *)
    kcc_ast_alloc(AST_ASM_OPERAND);
        op->f0 = (unsigned long)c.ptr;
        op->f1 = (unsigned long)symname;
    /* %[name] target; 0 if unnamed */
        op->f2 = (unsigned long)expr;
        op->f3 = is_output;
        if (as->f2 == 0) {
            as->f2 = (unsigned long)op;
        } else {
            struct Node *p = (struct Node *)as->f2
    ;
            while (p->f4) p = (struct Node *)p->f4
    ;
            p->f4 = (unsigned long)op;
        }
        if (kcc_tok_type != TOK_COMMA) return;
        kcc_pp_next();
     }
}

void kcc_parse_asm_clobbers(void) {
     for (;;) {
        unsigned long t = kcc_tok_type;
        if (t == TOK_COLON || t == TOK_RPAREN || t
     == TOK_EOF) return;
        kcc_strlit_gather();                        /*
     consume one clobber string */
        if (kcc_tok_type != TOK_COMMA) return;
        kcc_pp_next();
     }
}

void *kcc_parse_asm_extended(const char *tmpl,
    unsigned long len) {
     struct Node *a = (struct Node *)kcc_ast_alloc(
    AST_ASM_STMT);
     a->f0 = (unsigned long)tmpl;
     a->f1 = len;
     if (kcc_tok_type != TOK_COLON) {
    kcc_parse_expect(TOK_RPAREN); return a; }
     kcc_pp_next();                                 /*
     ':' before outputs */
     kcc_parse_asm_opsec(a, 1);
     if (kcc_tok_type != TOK_COLON) {
    kcc_parse_expect(TOK_RPAREN); return a; }
     kcc_pp_next();                                 /*
     ':' before inputs */
     kcc_parse_asm_opsec(a, 0);
     if (kcc_tok_type != TOK_COLON) {
    kcc_parse_expect(TOK_RPAREN); return a; }
     kcc_pp_next();                                 /*
     ':' before clobbers */
     kcc_parse_asm_clobbers();
     if (kcc_tok_type != TOK_COLON) {
    kcc_parse_expect(TOK_RPAREN); return a; }
     kcc_pp_next();                                 /*
     ':' before goto labels */
      /* Record each `asm goto' target label (a
    comma-separated identifier list)
         as its own node chained off f3; the IR
    resolves the names to blocks and
         the back end expands %lN to the Nth one. */
      while (kcc_tok_type != TOK_RPAREN &&
    kcc_tok_type != TOK_EOF) {
          if (kcc_tok_type == TOK_IDENT) {
              void *name = kcc_strtab_intern(
    kcc_tok_start, kcc_tok_len);
              struct Node *l = (struct Node *)
    kcc_ast_alloc(AST_ASM_OPERAND);
              l->f0 = (unsigned long)name;
              if (a->f3 == 0) a->f3 = (unsigned long
    )l;
              else { struct Node *p = (struct Node
    *)a->f3;
                     while (p->f4) p = (struct Node
    *)p->f4; p->f4 = (unsigned long)l; }
          }
          kcc_pp_next();
      }
      kcc_parse_expect(TOK_RPAREN);
      return a;
}
```

□

691b · used in 694b

63.5. Compound Statements and the Control Forms

A compound statement opens a typedef scope, then reads items until the closing brace: an item is a declaration when the leading token is a type or a storage class, else a statement, and each item—which may itself be a multi-declarator chain—is appended by walking to its `f4` tail. The selection and iteration forms are the direct grammar: each consumes its keyword and parenthesised parts and builds its node. `for` accepts a declaration or an expression initializer and allows any of its three clauses to be empty. A leading GNU local label declaration (`__label__ a, b;`), which the kernel's `wait_event`/RCU statement-expression macros use, is consumed and discarded—kcc scopes labels per function, so the names bind through the ordinary `goto` machinery.

⟨ *Selfhost: statements compound* 692 ⟩ ≡

```
static int starts_decl(unsigned long t) {
     return t == TOK_TYPEDEF || t == TOK_EXTERN ||
    t == TOK_STATIC ||
            t == TOK_AUTO || t == TOK_REGISTER || t
     == TOK_STATIC_ASSERT;
}

void *kcc_parse_compound(void) {
     kcc_parse_expect(TOK_LBRACE);
     kcc_typedef_push_scope();
     struct Node *comp = (struct Node *)
    kcc_ast_alloc(AST_COMPOUND);
     struct Node *first = 0, *last = 0;
     for (;;) {
         unsigned long t = kcc_tok_type;
         if (t == TOK_RBRACE || t == TOK_EOF) break
    ;
```

```
        /* GNU local label declaration: `__label__
    a, b;` at the head of a block
           (the kernel's wait_event/RCU statement-
    expression macros open with one
           so a nested `goto` can name a block-
    local target).  kcc resolves labels
           per function, so the declaration only
    has to be consumed; the names
           still bind through the normal label/
    goto machinery. */
        if (t == TOK_IDENT && kcc_tok_len == 9 &&
            kcc_memcmp(kcc_tok_start, "__label__",
     9) == 0) {
            kcc_pp_next();
            for (;;) {
                if (kcc_tok_type == TOK_IDENT)
    kcc_pp_next();
                if (kcc_tok_type != TOK_COMMA)
    break;
                kcc_pp_next();
            }
            kcc_parse_expect(TOK_SEMI);
            continue;
        }
        void *item;
        if (kcc_parse_is_type_token() ||
    starts_decl(kcc_tok_type))
            item = kcc_parse_decl();
        else
            item = kcc_parse_stmt();
        struct Node *head = (struct Node *)item;
        if (head == 0) continue;                /*
    error recovery */
        struct Node *tail = head;
        while (tail->f4) tail = (struct Node *)
    tail->f4;
        if (first == 0) { first = head; last =
    tail; }
        else { last->f4 = (unsigned long)head;
    last = tail; }
    }
    kcc_parse_expect(TOK_RBRACE);
    kcc_typedef_pop_scope();
    comp->f0 = (unsigned long)first;
    return comp;
}

void *kcc_parse_if(void) {
    kcc_pp_next();
    kcc_parse_expect(TOK_LPAREN);
    void *cond = kcc_parse_expr();
    kcc_parse_expect(TOK_RPAREN);
    void *then = kcc_parse_stmt();
    void *els = 0;
    if (kcc_tok_type == TOK_ELSE) { kcc_pp_next();
    els = kcc_parse_stmt(); }
    struct Node *n = (struct Node *)kcc_ast_alloc(
    AST_IF_STMT);
    n->f0 = (unsigned long)cond;
    n->f1 = (unsigned long)then;
    n->f2 = (unsigned long)els;
    return n;
}

void *kcc_parse_switch(void) {
    kcc_pp_next();
    kcc_parse_expect(TOK_LPAREN);
    void *e = kcc_parse_expr();
    kcc_parse_expect(TOK_RPAREN);
    void *body = kcc_parse_stmt();
    struct Node *n = (struct Node *)kcc_ast_alloc(
    AST_SWITCH_STMT);
    n->f0 = (unsigned long)e;
    n->f1 = (unsigned long)body;
    return n;
}

void *kcc_parse_while(void) {
    kcc_pp_next();
    kcc_parse_expect(TOK_LPAREN);
    void *cond = kcc_parse_expr();
    kcc_parse_expect(TOK_RPAREN);
    void *body = kcc_parse_stmt();
    struct Node *n = (struct Node *)kcc_ast_alloc(
    AST_WHILE_STMT);
    n->f0 = (unsigned long)cond;
    n->f1 = (unsigned long)body;
    return n;
}

void *kcc_parse_do(void) {
    kcc_pp_next();
    void *body = kcc_parse_stmt();
    kcc_parse_expect(TOK_WHILE);
    kcc_parse_expect(TOK_LPAREN);
    void *cond = kcc_parse_expr();
    kcc_parse_expect(TOK_RPAREN);
    kcc_parse_expect(TOK_SEMI);
    struct Node *n = (struct Node *)kcc_ast_alloc(
    AST_DO_STMT);
    n->f0 = (unsigned long)body;
    n->f1 = (unsigned long)cond;
    return n;
}

void *kcc_parse_for(void) {
    kcc_pp_next();
    kcc_parse_expect(TOK_LPAREN);
    void *init = 0;
    if (kcc_tok_type == TOK_SEMI) {
        kcc_pp_next();
    } else if (kcc_parse_is_type_token() ||
    starts_decl(kcc_tok_type)) {
        init = kcc_parse_decl();                /*
    consumes trailing ';' */
    } else {
        void *e = kcc_parse_expr();
        struct Node *es = (struct Node *)
    kcc_ast_alloc(AST_EXPR_STMT);
        es->f0 = (unsigned long)e;
        init = es;
        kcc_parse_expect(TOK_SEMI);
    }
    void *cond = 0;
    if (kcc_tok_type != TOK_SEMI) cond =
    kcc_parse_expr();
    kcc_parse_expect(TOK_SEMI);
    void *incr = 0;
    if (kcc_tok_type != TOK_RPAREN) incr =
    kcc_parse_expr();
    kcc_parse_expect(TOK_RPAREN);
    void *body = kcc_parse_stmt();
    struct Node *n = (struct Node *)kcc_ast_alloc(
    AST_FOR_STMT);
    n->f0 = (unsigned long)init;
    n->f1 = (unsigned long)cond;
    n->f2 = (unsigned long)incr;
    n->f3 = (unsigned long)body;
    return n;
}
```

□

692 · used in 694b

63.6. Jumps and Labels

goto interns its target name for later resolution; return carries an optional value; case parses a constant expression before its colon and default none, each labeling the statement that follows. A GNU case range case LO…HI: is recognised when an ellipsis follows the first constant; the high bound is stored in the label's f3.

goto also accepts the GNU *computed* form goto *expr, which jumps to a label address produced by &&label (see the unary parser). A leading * switches it: f1 flags the indirect goto and f0 then holds the target expression instead of an interned name. The Linux bpf interpreter dispatches its opcode loop this way, through a static jumptable of &&label addresses.

⟨ *Selfhost: statements jumps* 694a ⟩ ≡

```
void *kcc_parse_goto(void) {
    kcc_pp_next();
    /* GNU computed goto: `goto *expr' jumps to a
   label address (an `&&label'
       value).  f1 marks the indirect form; f0
   then holds the target expression
       instead of a label name.  The bpf
   interpreter dispatches this way. */
    if (kcc_tok_type == TOK_STAR) {
        kcc_pp_next();
        void *e = kcc_parse_expr();
        kcc_parse_expect(TOK_SEMI);
        struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_GOTO_STMT);
        n->f0 = (unsigned long)e;
        n->f1 = 1;                              /*
   indirect */
        return n;
    }
    void *name = kcc_strtab_intern(kcc_tok_start,
   kcc_tok_len);
    kcc_pp_next();                              /*
   consume identifier */
    kcc_parse_expect(TOK_SEMI);
    struct Node *n = (struct Node *)kcc_ast_alloc(
   AST_GOTO_STMT);
    n->f0 = (unsigned long)name;
    return n;
}

void *kcc_parse_return(void) {
    kcc_pp_next();
    void *expr = 0;
    if (kcc_tok_type != TOK_SEMI) expr =
   kcc_parse_expr();
    kcc_parse_expect(TOK_SEMI);
    struct Node *n = (struct Node *)kcc_ast_alloc(
   AST_RETURN_STMT);
    n->f0 = (unsigned long)expr;
    return n;
}

void *kcc_parse_case(void) {
    kcc_pp_next();
    void *e = kcc_parse_cond_expr();
    void *hi = 0;
    if (kcc_tok_type == TOK_ELLIPSIS) {   /* GNU
   case range: case LO ... HI: */
        kcc_pp_next();
        hi = kcc_parse_cond_expr();
    }
    kcc_parse_expect(TOK_COLON);
    void *stmt = kcc_parse_stmt();
    struct Node *n = (struct Node *)kcc_ast_alloc(
   AST_CASE_STMT);
    n->f0 = (unsigned long)e;
    n->f1 = (unsigned long)stmt;
    n->f3 = (unsigned long)hi;              /* high
   bound of a range, else 0 */
    return n;
}

void *kcc_parse_default(void) {
    kcc_pp_next();
    kcc_parse_expect(TOK_COLON);
    void *stmt = kcc_parse_stmt();
    struct Node *n = (struct Node *)kcc_ast_alloc(
   AST_DEFAULT_STMT);
    n->f0 = (unsigned long)stmt;
    return n;
}
```

□

694a · used in 694b

63.7. Assembling the Translation Unit

⟨ *Selfhost: statements* 694b ⟩ ≡

```
⟨ Selfhost: statements preamble 689 ⟩
⟨ Selfhost: statements dispatch 690 ⟩
⟨ Selfhost: statements exprrest 691a ⟩
⟨ Selfhost: statements asm 691b ⟩
⟨ Selfhost: statements compound 692 ⟩
⟨ Selfhost: statements jumps 694a ⟩
```

□

694b · used in 35h

63.8. Verification

make selfhost-statements runs the drop-in loop. Because every function body the suite compiles is parsed into statements by this file, a green code-generation suite means kcc compiled its own statement grammar correctly; difftest additionally confirms the resulting programs behave identically to clang.

⟨ *Selfhost: statements verify* 694c ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-statements - compile kcc's
   statement parser in C with kcc,
# relink, run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
```

```
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/statements.c
   with kcc"
$BASE/kcc -c "$SH/statements.c" -o "$SH/
   statements_c.o" >/dev/null

echo "[2/3] linking $BASE/kcc_selfhost_statements
   (statements.o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "statements" ]; then OBJ="$OBJ $SH
   /statements_c.o"; else OBJ="$OBJ $BASE/src/${f
   }.o"; fi
done
ld -o $BASE/kcc_selfhost_statements $OBJ -lSystem
   -syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted statement parser"
cp $BASE/kcc "$SH/kcc_asm_backup_statements"
cp $BASE/kcc_selfhost_statements $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_statements" $BASE/kcc
exit $status
```

□

694c · used in 35i · uses _main 621, 629

64. Self-Hosting: The Mach-O Writer in C

The Mach-O writer of Chapter 40 is the compiler's last step: it packages the generated code, the symbol and relocation tables the linker module (Chapter 62) accumulated, and the string table into the bytes an object file is made of. Porting it completes the back-end pair—the tables built in C are now also *serialised* in C—and because every test in the code-generation suite links and runs the object this file writes, and `difftest` compares those objects to `clang`'s behaviour, a green sweep means kcc reproduced its own object layout byte for byte.

It is the largest single function ported so far: `finalize` computes the file layout and writes every structure in order, with conditional sections—a module-init pointer, a `__const` pointer table, and a DWARF segment under `-g`—each shifting the downstream offsets. The port keeps the assembly's exact arithmetic so the two agree to the byte.

64.1. Buffer, State, and the Primitive Writers

The writer builds the whole object in one arena buffer and flushes it with a single `write`. The port owns the four module globals (the file descriptor, buffer pointer, write position, and capacity) and the little-endian append primitives—`write8/16/32/64`, `write_bytes` (with the capacity guard), and `pad_to`. These are exported because the DWARF emitter (still assembly) writes through them. It declares the Mach-O constants, the linker-table record offsets, and—as externs—the code buffer and size, the linker tables, the module-init and const-data state, and the DWARF hooks.

⟨ *Selfhost: emit-macho preamble* 697a ⟩ ≡

```
struct Vec {
    void *data;              /* VEC_DATA,     0
   */
    unsigned long count;     /* VEC_COUNT,    8
   */
    unsigned long cap;
    unsigned long elem_size;
};

struct MSym {
    const char *name;        /* SYM_NAME,     0  */
    unsigned long section;   /* SYM_SECTION,  8  */
    unsigned long value;     /* SYM_VALUE,    16 */
    unsigned long flags;     /* SYM_FLAGS,    24 */
};

void *kcc_arena_alloc(long size);
void *kcc_memcpy(void *dst, const void *src,
   unsigned long n);
void *kcc_vec_get(void *vec, unsigned long index);
unsigned long kcc_strlen(const char *s);
void kcc_io_err(const char *s);
long kcc_sys_open(const char *path, long flags,
   long mode);
long kcc_sys_write(long fd, const void *buf,
   unsigned long n);
long kcc_sys_close(long fd);
void kcc_sys_exit(int status);

/* Contributors and target state, all defined
   elsewhere (codegen, linker, ir). */
extern unsigned long kcc_code_pos;          /*
   code size */
extern void *kcc_code_buf;
extern void *kcc_linker_syms;
extern void *kcc_linker_relocs;
extern unsigned long kcc_globals_size;
extern unsigned long kcc_modinit_present;
extern unsigned long kcc_modinit_fileoff;
extern unsigned long kcc_modinit_sym;
extern unsigned long kcc_rodata_const_present;
extern unsigned long kcc_rodata_const_fileoff;
extern unsigned long kcc_rodata_const_size;
extern void *kcc_rodata_const_relocs;
extern unsigned long kcc_dwarf_enabled;
unsigned long kcc_dwarf_total_len(void);
void kcc_dwarf_emit_macho(unsigned long file_base,
    unsigned long vmaddr);
void kcc_dwarf_append_blob(void);

/* Module globals (defined here; the DWARF emitter
    reads buf/pos). */
long kcc_macho_fd = 0;
void *kcc_macho_buf = 0;
unsigned long kcc_macho_pos = 0;
unsigned long kcc_macho_cap = 0;

static const char macho_err_open[] = "kcc: error:
   cannot open output file\n";
static const char macho_err_overflow[] =
    "kcc: error: object file exceeds output buffer
   \n";

/* Mach-O relocation types read by the packer. */
#define ARM64_RELOC_PAGEOFF12          4
#define ARM64_RELOC_GOT_LOAD_PAGEOFF12 6
#define N_EXT  0x01
#define N_SECT 0x0E
```

□

697a · used in 702a

⟨ *Selfhost: emit-macho writers* 697b ⟩ ≡

```
void kcc_macho_write8(unsigned long v) {
    unsigned char *b = (unsigned char *)
   kcc_macho_buf;
    unsigned long p = kcc_macho_pos;
    b[p] = (unsigned char)(v & 0xff);
    kcc_macho_pos = p + 1;
}

void kcc_macho_write16(unsigned long v) {
    unsigned char *b = (unsigned char *)
   kcc_macho_buf;
    unsigned long p = kcc_macho_pos;
    b[p] = (unsigned char)(v & 0xff);
    b[p + 1] = (unsigned char)((v >> 8) & 0xff);
    kcc_macho_pos = p + 2;
}

void kcc_macho_write32(unsigned long v) {
```

```
    unsigned char *b = (unsigned char *)
   kcc_macho_buf;
    unsigned long p = kcc_macho_pos;
    b[p] = (unsigned char)(v & 0xff);
    b[p + 1] = (unsigned char)((v >> 8) & 0xff);
    b[p + 2] = (unsigned char)((v >> 16) & 0xff);
    b[p + 3] = (unsigned char)((v >> 24) & 0xff);
    kcc_macho_pos = p + 4;
}

void kcc_macho_write64(unsigned long v) {
    unsigned char *b = (unsigned char *)
   kcc_macho_buf;
    unsigned long p = kcc_macho_pos;
    for (int i = 0; i < 8; i++) b[p + i] = (
   unsigned char)((v >> (i * 8)) & 0xff);
    kcc_macho_pos = p + 8;
}

void kcc_macho_write_bytes(const void *src,
   unsigned long len) {
    if (kcc_macho_pos + len > kcc_macho_cap) {
        kcc_io_err(macho_err_overflow);
        kcc_sys_exit(1);
    }
    kcc_memcpy((unsigned char *)kcc_macho_buf +
   kcc_macho_pos, src, len);
    kcc_macho_pos += len;
}

void kcc_macho_pad_to(unsigned long align) {
    unsigned char *b = (unsigned char *)
   kcc_macho_buf;
    unsigned long p = kcc_macho_pos;
    unsigned long a = (p + align - 1) & ~(align -
   1);
    while (p < a) { b[p] = 0; p++; }
    kcc_macho_pos = a;
}

void kcc_macho_init(const char *filename) {
    long fd = kcc_sys_open(filename, 0x601, 420);
    if (fd < 0) { kcc_io_err(macho_err_open);
   kcc_sys_exit(1); }
    kcc_macho_fd = fd;
    unsigned long sz = kcc_dwarf_total_len();
    sz += kcc_code_pos;
    sz += ((struct Vec *)kcc_linker_syms)->count *
    288;
    sz += ((struct Vec *)kcc_linker_relocs)->count
    * 8;
    sz += 65536;
    kcc_macho_buf = kcc_arena_alloc((long)sz);
    kcc_macho_cap = sz;
    kcc_macho_pos = 0;
}
```

□

697b · used in 702a

64.2. Layout Computation

Finalize first reads the code size and the symbol, relocation, and globals counts, then computes each structure's file offset. The load-command size grows by 80 bytes for a `__mod_init_func` section, 80 for a `__const` section, and a whole 72+80×3 for a DWARF segment; the section-data offset follows the header and commands; the relocation offset follows the code (and the reserved DWARF blob, the 8-aligned module-init pointer, and the 8-aligned const data); and the symbol and string tables follow in turn. The string-table size is one (its leading NUL) plus each name and its terminator, and the defined-symbol count drives `LC_DYSYMTAB`.

⟨ *Selfhost: emit-macho layout* 698 ⟩ ≡

```
void kcc_macho_finalize(void) {
    unsigned long code_size = kcc_code_pos;
    unsigned long nsyms = ((struct Vec *)
   kcc_linker_syms)->count;
    unsigned long nrelocs = ((struct Vec *)
   kcc_linker_relocs)->count;
    unsigned long globals_size = kcc_globals_size;

    /* Load-commands size. */
    unsigned long cmdsz = 232 + 24 + 24 + 80;
     /* segment + build + symtab + dysymtab */
    if (kcc_modinit_present) cmdsz += 80;
    if (kcc_rodata_const_present) cmdsz += 80;
    if (kcc_dwarf_enabled) cmdsz += 72 + 80 * 3;

    unsigned long text_offset = cmdsz + 32;
     /* MACH_HEADER_SIZE */

    /* Relocation offset: after code (+ reserved
   DWARF blob, modinit ptr, const data). */
    unsigned long roff = text_offset + code_size;
    if (kcc_dwarf_enabled) roff +=
   kcc_dwarf_total_len();
    if (kcc_modinit_present) {
        roff = (roff + 7) & ~7UL;
        kcc_modinit_fileoff = roff;
        roff += 8;
    }
    if (kcc_rodata_const_present) {
        roff = (roff + 7) & ~7UL;
        kcc_rodata_const_fileoff = roff;
        roff += kcc_rodata_const_size;
    }
    roff = (roff + 3) & ~3UL;
      /* 4-byte aligned */

    /* Symbol table offset: after the text relocs,
    the modinit reloc, the const relocs. */
    unsigned long symoff = roff + nrelocs * 8;
    if (kcc_modinit_present) symoff += 8;
    if (kcc_rodata_const_present)
        symoff += ((struct Vec *)
   kcc_rodata_const_relocs)->count * 8;

    unsigned long stroff = symoff + nsyms * 16;

    /* String-table size and the defined-symbol
   count. */
    unsigned long strsize = 1;
    unsigned long nextdef = 0;
    for (unsigned long i = 0; i < nsyms; i++) {
        struct MSym *s = (struct MSym *)
   kcc_vec_get(kcc_linker_syms, i);
        strsize += kcc_strlen(s->name) + 1;
        if (s->section != 0) nextdef++;
    }
```

□

698 · used in 702a

64.3. Header, Segment, and Sections

The header records the magic, CPU type, MH_OBJECT, the command count (four, plus one for a DWARF segment) and size, and the subsections-via-symbols flag. The single LC_SEGMENT_64 carries the __text section (the code, its relocations, pure-instructions flags), a __common zero-fill section for globals, and—conditionally—the __const pointer table and the __mod_init_func pointer. Section and segment names are emitted as the exact little-endian words the assembly used, so the bytes match including their embedded NULs.

⟨ *Selfhost: emit-macho header* 699 ⟩ ≡

```
/* --- Mach-O header --- */
kcc_macho_write32(0xFEEDFACF);
 /* magic */
kcc_macho_write32(0x0100000C);
 /* cputype ARM64 */
kcc_macho_write32(0);
 /* cpusubtype */
kcc_macho_write32(0x01);
  /* MH_OBJECT */
kcc_macho_write32(kcc_dwarf_enabled ? 5 : 4);
 /* ncmds */
kcc_macho_write32(cmdsz);
  /* sizeofcmds */
kcc_macho_write32(0x2000);
  /* MH_SUBSECTIONS_VIA_SYMBOLS */
kcc_macho_write32(0);
  /* reserved */

/* --- LC_SEGMENT_64 --- */
kcc_macho_write32(0x19);
  /* cmd */
{
    unsigned long segcmdsz = 232;
    if (kcc_modinit_present) segcmdsz += 80;
    if (kcc_rodata_const_present) segcmdsz +=
80;
    kcc_macho_write32(segcmdsz);
}
kcc_macho_write64(0);
  /* segname (16 bytes, empty) */
kcc_macho_write64(0);
kcc_macho_write64(0);
  /* vmaddr */
{
    unsigned long vmsize = globals_size +
code_size;
    if (kcc_modinit_present) { vmsize = (
vmsize + 7) & ~7UL; vmsize += 8; }
    if (kcc_rodata_const_present) {
        vmsize = (vmsize + 7) & ~7UL;
        vmsize += kcc_rodata_const_size;
    }
    kcc_macho_write64(vmsize);
}
kcc_macho_write64(text_offset);
 /* fileoff */
{
    unsigned long filesize = code_size;
    if (kcc_modinit_present)
        filesize = (kcc_modinit_fileoff + 8) -
 text_offset;
    if (kcc_rodata_const_present)
        filesize = (kcc_rodata_const_fileoff +
 kcc_rodata_const_size) - text_offset;
    kcc_macho_write64(filesize);
}
kcc_macho_write32(7);
  /* maxprot */
kcc_macho_write32(7);
  /* initprot */
{
    unsigned long nsects = 2;
    if (kcc_modinit_present) nsects++;
    if (kcc_rodata_const_present) nsects++;
    kcc_macho_write32(nsects);
}
kcc_macho_write32(0);
  /* flags */

/* --- Section: __text --- */
kcc_macho_write64(0x0000747865745F5FUL);
 /* "__text\0\0" */
kcc_macho_write64(0);
kcc_macho_write64(0x0000545845545F5FUL);
 /* "__TEXT\0\0" */
kcc_macho_write64(0);
kcc_macho_write64(0);
  /* addr */
kcc_macho_write64(code_size);
 /* size */
kcc_macho_write32(text_offset);
 /* offset */
kcc_macho_write32(2);
  /* align = 4 */
kcc_macho_write32(roff);
  /* reloff */
kcc_macho_write32(nrelocs);
  /* nreloc */
kcc_macho_write32(0x80000400);
 /* PURE|SOME instructions */
kcc_macho_write32(0);
kcc_macho_write32(0);
kcc_macho_write32(0);

/* --- Section: __DATA,__common (S_ZEROFILL)
--- */
kcc_macho_write64(0x6E6F6D6D6F635F5FUL);
 /* "__common" */
kcc_macho_write64(0);
kcc_macho_write64(0x0000415441445F5FUL);
 /* "__DATA\0\0" */
kcc_macho_write64(0);
kcc_macho_write64(code_size);
 /* addr */
kcc_macho_write64(globals_size);
 /* size */
kcc_macho_write32(0);
 /* offset */
kcc_macho_write32(3);
 /* align = 8 */
kcc_macho_write32(0);
 /* reloff */
kcc_macho_write32(0);
 /* nreloc */
kcc_macho_write32(1);
 /* S_ZEROFILL */
kcc_macho_write32(0);
kcc_macho_write32(0);
kcc_macho_write32(0);
```

□

699 · used in 702a

The two conditional sections mirror the assembly ex-

actly. The __const pointer table follows __common (and the module-init pointer, when present) in memory, so its address is 8-aligned past them; its relocations trail the text and module-init relocations. The __mod_init_func section is a single dyld-called pointer whose one ARM64_RELOC_UNSIGNED relocation supplies its value.

⟨ *Selfhost: emit-macho sections* 700a ⟩ ≡

```
    /* --- Section: __DATA_CONST,__const --- */
    if (kcc_rodata_const_present) {
        kcc_macho_write64(0x0074736E6F635F5FUL);
     /* "__const\0" */
        kcc_macho_write64(0);
        kcc_macho_write64(0x435F415441445F5FUL);
     /* "__DATA_C" */
        kcc_macho_write64(0x0000000054534E4FUL);
     /* "ONST\0\0\0\0" */
        {
            unsigned long addr = globals_size +
code_size;
            if (kcc_modinit_present) { addr = (
addr + 7) & ~7UL; addr += 8; }
            addr = (addr + 7) & ~7UL;
            kcc_macho_write64(addr);
     /* addr */
        }
        kcc_macho_write64(kcc_rodata_const_size);
     /* size */
        kcc_macho_write32(kcc_rodata_const_fileoff
);/* offset */
        kcc_macho_write32(3);
     /* align = 8 */
        {
            unsigned long r = roff + nrelocs * 8;
            if (kcc_modinit_present) r += 8;
            kcc_macho_write32(r);
     /* reloff */
        }
        kcc_macho_write32(((struct Vec *)
kcc_rodata_const_relocs)->count); /* nreloc */
        kcc_macho_write32(0);
     /* S_REGULAR */
        kcc_macho_write32(0);
        kcc_macho_write32(0);
        kcc_macho_write32(0);
    }

    /* --- Section: __DATA,__mod_init_func --- */
    if (kcc_modinit_present) {
        kcc_macho_write64(0x6E695F646F6D5F5FUL);
     /* "__mod_in" */
        kcc_macho_write64(0x0063756E665F7469UL);
     /* "it_func\0" */
        kcc_macho_write64(0x0000415441445F5FUL);
     /* "__DATA\0\0" */
        kcc_macho_write64(0);
        {
            unsigned long addr = globals_size +
code_size;
            addr = (addr + 7) & ~7UL;
            kcc_macho_write64(addr);
     /* addr */
        }
        kcc_macho_write64(8);
     /* size = 8 (one pointer) */
        kcc_macho_write32(kcc_modinit_fileoff);
     /* offset */
        kcc_macho_write32(3);
     /* align = 8 */
        kcc_macho_write32(roff + nrelocs * 8);
     /* reloff */
        kcc_macho_write32(1);
     /* nreloc = 1 */
        kcc_macho_write32(0x09);
     /* S_MOD_INIT_FUNC_POINTERS */
        kcc_macho_write32(0);
        kcc_macho_write32(0);
        kcc_macho_write32(0);
    }
```

□

700a · used in 702a

64.4. Auxiliary Load Commands

LC_BUILD_VERSION names the platform (macOS 14) to quiet the linker; LC_SYMTAB gives the symbol- and string-table offsets and sizes; and LC_DYSYMTAB partitions the symbols into the defined-external range (from zero) and the undefined-external range (from the defined count), the rest of its fields zero.

⟨ *Selfhost: emit-macho auxcmds* 700b ⟩ ≡

```
    /* --- LC_BUILD_VERSION --- */
    kcc_macho_write32(0x32);
     /* cmd */
    kcc_macho_write32(24);
     /* cmdsize */
    kcc_macho_write32(1);
     /* PLATFORM_MACOS */
    kcc_macho_write32(14 << 16);
     /* minos 14.0.0 */
    kcc_macho_write32(14 << 16);
     /* sdk 14.0.0 */
    kcc_macho_write32(0);
     /* ntools */

    /* --- LC_SYMTAB --- */
    kcc_macho_write32(0x02);
     /* cmd */
    kcc_macho_write32(24);
     /* cmdsize */
    kcc_macho_write32(symoff);
    kcc_macho_write32(nsyms);
    kcc_macho_write32(stroff);
    kcc_macho_write32(strsize);

    /* --- LC_DYSYMTAB --- */
    kcc_macho_write32(0x0B);
     /* cmd */
    kcc_macho_write32(80);
     /* cmdsize */
    kcc_macho_write32(0);
     /* ilocalsym */
    kcc_macho_write32(0);
     /* nlocalsym */
    kcc_macho_write32(0);
     /* iextdefsym */
    kcc_macho_write32(nextdef);
     /* nextdefsym */
    kcc_macho_write32(nextdef);
     /* iundefsym */
    kcc_macho_write32(nsyms - nextdef);
     /* nundefsym */
```

```
    for (int i = 0; i < 12; i++) kcc_macho_write32
(0);
```

□

700b · used in 702a

64.5. Code, Relocations, Symbols, and Flush

After the commands come the DWARF segment (its blob laid after the code), the raw code bytes, the module-init and const bodies (zeros filled by relocations), and the LINKEDIT region. The relocations are written against a *remap* array that reorders symbol indices into the defined-then-undefined order Mach-O requires; each 8-byte record packs the remapped symbol number, the PC-relative flag (cleared for the two `PAGEOFF12` forms), a length code, the extern bit, and the type. The symbol table writes the same two passes of `nlist_64` entries—a defined symbol's value rebased to its section's VM start, a local string literal kept unexported—and the string table their names in the same order. One `write` flushes it all.

⟨ *Selfhost: emit-macho body* 701 ⟩ ≡

```
    /* --- DWARF segment (under -g) --- */
    if (kcc_dwarf_enabled)
        kcc_dwarf_emit_macho(text_offset +
code_size, code_size + globals_size);

    /* --- Code bytes --- */
    kcc_macho_write_bytes(kcc_code_buf, code_size)
;

    if (kcc_dwarf_enabled) kcc_dwarf_append_blob()
;

    /* --- Module-init pointer (8 zero bytes) ---
*/
    if (kcc_modinit_present) {
        kcc_macho_pad_to(8);
        kcc_macho_write64(0);
    }

    /* --- __const section data (zeros; values
come from relocations) --- */
    if (kcc_rodata_const_present) {
        kcc_macho_pad_to(8);
        unsigned long rem = kcc_rodata_const_size;
        while (rem) { kcc_macho_write64(0); rem -=
8; }
    }

    kcc_macho_pad_to(4);

    /* --- Symbol index remap: defined first, then
    undefined --- */
    unsigned long *remap = (unsigned long *)
kcc_arena_alloc((long)(nsyms * 8));
    unsigned long out = 0;
    for (unsigned long i = 0; i < nsyms; i++) {
        struct MSym *s = (struct MSym *)
kcc_vec_get(kcc_linker_syms, i);
        if (s->section != 0) remap[i] = out++;
    }
    for (unsigned long i = 0; i < nsyms; i++) {
        struct MSym *s = (struct MSym *)
kcc_vec_get(kcc_linker_syms, i);
        if (s->section == 0) remap[i] = out++;
    }

    /* --- Text relocations --- */
    for (unsigned long i = 0; i < nrelocs; i++) {
        unsigned long *r = (unsigned long *)
kcc_vec_get(kcc_linker_relocs, i);
        kcc_macho_write32(r[0]);
     /* r_address = offset */
        unsigned long type = r[1];
        unsigned long rsym = remap[r[2]];
        unsigned long info = rsym & 0xFFFFFF;
        if (type != ARM64_RELOC_PAGEOFF12 && type
!= ARM64_RELOC_GOT_LOAD_PAGEOFF12)
            info |= (1u << 24);
     /* r_pcrel */
        info |= (2u << 25);
     /* r_length = 2 */
        info |= (1u << 27);
     /* r_extern */
        info |= (type << 28);
     /* r_type */
        kcc_macho_write32(info);
    }

    /* --- Module-init pointer relocation --- */
    if (kcc_modinit_present) {
        kcc_macho_write32(0);
     /* r_address */
        unsigned long rsym = remap[kcc_modinit_sym
- 1];
        unsigned long info = (rsym & 0xFFFFFF) |
(3u << 25) | (1u << 27);
        kcc_macho_write32(info);
     /* UNSIGNED, length 3, extern */
    }

    /* --- __const relocations --- */
    if (kcc_rodata_const_present) {
        struct Vec *cr = (struct Vec *)
kcc_rodata_const_relocs;
        for (unsigned long i = 0; i < cr->count; i
++) {
            unsigned long *e = (unsigned long *)
kcc_vec_get(cr, i);
            kcc_macho_write32(e[0]);
     /* r_address = slot offset */
            unsigned long rsym = remap[e[1]];
            unsigned long info = (rsym & 0xFFFFFF)
 | (3u << 25) | (1u << 27);
            kcc_macho_write32(info);
        }
    }

    /* --- Symbol table: nlist_64, defined then
undefined --- */
    unsigned long strpos = 1;
    for (unsigned long i = 0; i < nsyms; i++) {
        struct MSym *s = (struct MSym *)
kcc_vec_get(kcc_linker_syms, i);
        if (s->section == 0) continue;
        kcc_macho_write32(strpos);
     /* n_strx */
        kcc_macho_write8((s->flags & N_EXT) |
N_SECT); /* n_type */
        kcc_macho_write8(s->section);
     /* n_sect */
        kcc_macho_write16(0);
     /* n_desc */
        unsigned long val = s->value;
```

```
        if (s->section == 2) {
            val += code_size;
        } else if (s->section == 3) {
            unsigned long base = code_size +
    globals_size;
            if (kcc_modinit_present) { base = (
    base + 7) & ~7UL; base += 8; }
            base = (base + 7) & ~7UL;
            val += base;
        }
        kcc_macho_write64(val);
      /* n_value */
        strpos += kcc_strlen(s->name) + 1;
     }
     for (unsigned long i = 0; i < nsyms; i++) {
        struct MSym *s = (struct MSym *)
    kcc_vec_get(kcc_linker_syms, i);
        if (s->section != 0) continue;
        kcc_macho_write32(strpos);
      /* n_strx */
        kcc_macho_write8(0x01);
      /* N_UNDF | N_EXT */
        kcc_macho_write8(0);
      /* n_sect */
        kcc_macho_write16(0);
      /* n_desc */
        kcc_macho_write64(0);
      /* n_value */
        strpos += kcc_strlen(s->name) + 1;
     }

     /* --- String table --- */
     kcc_macho_write8(0);
      /* leading NUL */
     for (unsigned long i = 0; i < nsyms; i++) {
        struct MSym *s = (struct MSym *)
    kcc_vec_get(kcc_linker_syms, i);
        if (s->section == 0) continue;
        kcc_macho_write_bytes(s->name, kcc_strlen(
    s->name) + 1);
     }
     for (unsigned long i = 0; i < nsyms; i++) {
        struct MSym *s = (struct MSym *)
    kcc_vec_get(kcc_linker_syms, i);
        if (s->section != 0) continue;
        kcc_macho_write_bytes(s->name, kcc_strlen(
    s->name) + 1);
     }

     /* --- Flush --- */
     kcc_sys_write(kcc_macho_fd, kcc_macho_buf,
    kcc_macho_pos);
     kcc_sys_close(kcc_macho_fd);
}
```

□

701 · used in 702a

64.6. Assembling the Translation Unit

⟨ *Selfhost: emit-macho* 702a ⟩ ≡

⟨ *Selfhost: emit-macho preamble* 697a ⟩
⟨ *Selfhost: emit-macho writers* 697b ⟩
⟨ *Selfhost: emit-macho layout* 698 ⟩
⟨ *Selfhost: emit-macho header* 699 ⟩
⟨ *Selfhost: emit-macho sections* 700a ⟩
⟨ *Selfhost: emit-macho auxcmds* 700b ⟩
⟨ *Selfhost: emit-macho body* 701 ⟩

□

702a · used in 35j

64.7. Verification

`make selfhost-emit-macho` runs the drop-in loop. Because every test in the code-generation suite links and runs the object this file writes, and `difftest` compares behaviour against `clang`, a green sweep means kcc reproduced its own Mach-O layout byte for byte.

⟨ *Selfhost: emit-macho verify* 702b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-emit-macho - compile kcc's Mach-O
   writer in C with kcc, relink,
# run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/emit_macho.c
   with kcc"
$BASE/kcc -c "$SH/emit_macho.c" -o "$SH/
   emit_macho_c.o" >/dev/null

echo "[2/3] linking $BASE/kcc_selfhost_emit_macho
   (emit_macho.o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "emit_macho" ]; then OBJ="$OBJ $SH
   /emit_macho_c.o"; else OBJ="$OBJ $BASE/src/${f
   }.o"; fi
done
ld -o $BASE/kcc_selfhost_emit_macho $OBJ -lSystem
   -syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted Mach-O writer"
cp $BASE/kcc "$SH/kcc_asm_backup_emit_macho"
cp $BASE/kcc_selfhost_emit_macho $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_emit_macho" $BASE/kcc
exit $status
```

□

702b · used in 35k · uses _main 621, 629

65. Self-Hosting: The Expression Parser in C

The expression parser of Chapter 22 is the companion to the statement parser already ported (Chapter 63): the precedence-climbing core, the unary/cast/postfix chain, primary expressions, and the literal converters that turn a token's text into an integer or a floating value. Porting it puts the operator-rich heart of the front end through kcc-compiled C.

It is the first port that computes with *floating point*: the compiler evaluates a floating constant on the host FPU at translation time, so `parse_atof` and `round_to_float` use `double` and `float` arithmetic. The port replays the assembly's exact sequence of floating operations, so the 64-bit result is bit-identical. (Because kcc's x86-64 back end does not yet emit floating point, this is the first subsystem whose C does not also cross-compile to x86—its ARM64 self-host is what the drop-in verifies.)

65.1. Nodes, Tokens, and the Parser API

The port declares the 64-byte AST node, the token codes and node tags the grammar dispatches on, the precedence table, and the parser primitives as externs. Two helpers reinterpret a `double` as its 64-bit pattern and back—the literal node stores the pattern, and the FPU produces and consumes it.

⟨ *Selfhost: expressions preamble* 703 ⟩ ≡

```
struct Node {
    unsigned long tag;    /* AST_TAG,   0  */
    unsigned long line;
    unsigned long col;
    unsigned long f0;     /* AST_F0,    24 */
    unsigned long f1;
    unsigned long f2;
    unsigned long f3;
    unsigned long f4;     /* AST_F4,    56 */
};

struct StrLit { void *ptr; unsigned long len; };

extern unsigned long kcc_tok_type;
extern const char *kcc_tok_start;
extern unsigned long kcc_tok_len;
extern const char *kcc_cur_func_name;   /* set by
   the declarator parser for __func__ */
unsigned long kcc_strlen(const char *s);

void kcc_pp_next(void);
void kcc_parse_expect(unsigned long tok_type);
void kcc_parse_error(const char *msg);
void *kcc_ast_alloc(unsigned long tag);
unsigned long kcc_parse_is_type_token(void);
void *kcc_parse_type_name(void);
void *kcc_parse_compound(void);
void *kcc_parse_initializer(void);
int kcc_strcmp(const char *a, const char *b);
struct StrLit kcc_strlit_gather(void);
struct StrLit kcc_strlit_process(const char *ptr,
   unsigned long len);
void *kcc_strtab_intern(const void *ptr, unsigned
   long len);

enum {
    TOK_NUMBER = 2, TOK_CHAR_LIT = 3,
   TOK_STRING_LIT = 4, TOK_IDENT = 1,
    TOK_SIZEOF = 34, TOK__ALIGNOF = 45,
   TOK__GENERIC = 49, TOK_DEFAULT = 16,
    TOK_LPAREN = 60, TOK_RPAREN = 61, TOK_LBRACE =
    62,
    TOK_LBRACKET = 64, TOK_RBRACKET = 65,
   TOK_COMMA = 67, TOK_DOT = 68,
    TOK_ARROW = 69, TOK_INC = 70, TOK_DEC = 71,
   TOK_AMP = 72, TOK_STAR = 73,
    TOK_PLUS = 74, TOK_MINUS = 75, TOK_TILDE = 76,
    TOK_BANG = 77, TOK_AND = 90,
    TOK_QUESTION = 92, TOK_COLON = 93, TOK_ASSIGN
   = 94, TOK_OR_ASSIGN = 104
};

enum {
    AST_TAG = 0, AST_BINARY_EXPR = 54,
   AST_UNARY_EXPR = 55, AST_ASSIGN_EXPR = 56,
    AST_COND_EXPR = 57, AST_CALL_EXPR = 58,
   AST_SUBSCRIPT = 59, AST_MEMBER = 60,
    AST_CAST_EXPR = 61, AST_SIZEOF_EXPR = 62,
   AST_ALIGNOF_EXPR = 63,
    AST_COMMA_EXPR = 64, AST_COMPOUND_LIT = 67,
   AST_PAREN_EXPR = 68,
    AST_GENERIC = 69, AST_GENERIC_ASSOC = 70,
   AST_IDENT_EXPR = 50,
    AST_INT_LIT = 51, AST_STR_LIT = 52,
   AST_CHAR_LIT = 53, AST_FLOAT_LIT = 71,
    AST_VA_ARG = 72, AST_STMT_EXPR = 73,
   AST_TYPES_COMPAT = 76, AST_LABEL_ADDR = 77,
    AST_TYPE_BUILTIN = 20, AST_TYPE_PTR = 22
};

static unsigned char prec_table[48] = {
    0,  0,  0,  0,  0,  0,  0,  0,
    0,  0,  0,  0,  8, 13, 12, 12,
    0,  0, 13, 13, 11, 11, 10, 10,
   10, 10,  9,  9,  7,  6,  5,  4,
    0,  0,  0,  0,  0,  0,  0,  0,
    0,  0,  0,  0,  0,  0,  0,  0
};

static double bits_to_double(unsigned long b) {
    union { unsigned long u; double d; } x;
    x.u = b;
    return x.d;
}
static unsigned long double_to_bits(double d) {
    union { double d; unsigned long u; } x;
    x.d = d;
    return x.u;
}

/* Forward declarations of the module's own
   functions. */
unsigned long kcc_prec_lookup(unsigned long tok);
void *kcc_parse_expr(void);
void *kcc_parse_assign_expr(void);
void *kcc_parse_cond_expr(void);
void *kcc_parse_binary_expr(unsigned long min_prec
   );
void *kcc_parse_unary_expr(void);
void *kcc_parse_cast_expr(void);
void *kcc_parse_postfix_expr(void);
```

```
void *kcc_parse_postfix_tail(void *expr);
void *kcc_parse_primary_expr(void);
unsigned long kcc_parse_atoi(const char *str,
   unsigned long len);
unsigned long kcc_is_float_literal(const char *str
   , unsigned long len);
unsigned long kcc_parse_atof(const char *str,
   unsigned long len);
unsigned long kcc_round_to_float(unsigned long
   bits);
void *kcc_parse_arg_list(void);
```

□

703 · used in 710a

65.2. Precedence, Comma, Assignment, Conditional

prec_lookup maps a punctuator to its precedence through the table; parse_expr folds comma operators; parse_assign_expr is right-associative on the assignment operators; and parse_cond_expr parses a binary expression at precedence four and, on a ?, the two arms of the ternary.

⟨ *Selfhost: expressions core* 704 ⟩ ≡

```
unsigned long kcc_prec_lookup(unsigned long tok) {
    if (tok < 60) return 0;
    unsigned long t = tok - 60;
    if (t >= 48) return 0;
    return prec_table[t];
}

void *kcc_parse_expr(void) {
    struct Node *lhs = (struct Node *)
   kcc_parse_assign_expr();
    while (kcc_tok_type == TOK_COMMA) {
        kcc_pp_next();
        void *rhs = kcc_parse_assign_expr();
        struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_COMMA_EXPR);
        n->f0 = (unsigned long)lhs;
        n->f1 = (unsigned long)rhs;
        lhs = n;
    }
    return lhs;
}

void *kcc_parse_assign_expr(void) {
    struct Node *lhs = (struct Node *)
   kcc_parse_cond_expr();
    unsigned long t = kcc_tok_type;
    if (t >= TOK_ASSIGN && t <= TOK_OR_ASSIGN) {
        unsigned long op = t;
        kcc_pp_next();
        void *rhs = kcc_parse_assign_expr();
        struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_ASSIGN_EXPR);
        n->f0 = op;
        n->f1 = (unsigned long)lhs;
        n->f2 = (unsigned long)rhs;
        lhs = n;
    }
    return lhs;
}

void *kcc_parse_cond_expr(void) {
    struct Node *cond = (struct Node *)
   kcc_parse_binary_expr(4);
    if (kcc_tok_type != TOK_QUESTION) return cond;
    kcc_pp_next();
    /* GNU `a ?: b': omitted middle operand,
   recorded as f1 == 0, so the IR
       reuses the condition's value (evaluating `a
   ' once). */
    void *then = 0;
    if (kcc_tok_type != TOK_COLON)
        then = kcc_parse_expr();
    kcc_parse_expect(TOK_COLON);
    void *els = kcc_parse_cond_expr();
    struct Node *n = (struct Node *)kcc_ast_alloc(
   AST_COND_EXPR);
    n->f0 = (unsigned long)cond;
    n->f1 = (unsigned long)then;
    n->f2 = (unsigned long)els;
    return n;
}

void *kcc_parse_binary_expr(unsigned long min_prec
   ) {
    struct Node *lhs = (struct Node *)
   kcc_parse_cast_expr();
    for (;;) {
        unsigned long op = kcc_tok_type;
        unsigned long prec = kcc_prec_lookup(op);
        if (prec == 0 || prec < min_prec) break;
        kcc_pp_next();
        void *rhs = kcc_parse_binary_expr(prec +
   1);
        struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_BINARY_EXPR);
        n->f0 = op;
        n->f1 = (unsigned long)lhs;
        n->f2 = (unsigned long)rhs;
        lhs = n;
    }
    return lhs;
}
```

□

704 · used in 710a

65.3. Unary, Cast, Postfix

Unary first recognises the GNU labels-as-values form &&label: an && where a unary expression is expected cannot be the logical-and operator (which needs a left operand), so it is the address of a label, parsed into an AST_LABEL_ADDR node carrying the label name. It then parses the prefix operators, sizeof (of a parenthesised type or of an expression), and _Alignof. The alignment operator takes the same two shapes as sizeof: strict C11 _Alignof(type-name), and— because it shares a token with the GNU __alignof__—an *expression* operand, either parenthesised (__alignof__(*p)) or bare, which the kernel's kstrtox/rseq code uses; the operand's type is recovered in the same way sizeof(expr) recovers it (from the operand node's resolved-type field), and an is_type flag in f1 tells the lowering which shape it was. Cast distinguishes a parenthesised type (a cast, or a compound literal on {) from a parenthesised expression and a GNU statement expression (on ({); each non-cast form runs through the postfix tail so suffixes can chain. Postfix applies subscript, call (with the va_arg special form—also matched under its GNU __builtin_va_arg spelling—and the type-name builtins __builtin_types_compatible_-

p and __builtin_offsetof, the latter synthesised into (unsigned long)&((T*)0)->designator), member access, and postfix ++/--.

⟨ *Selfhost: expressions unary* 705 ⟩ ≡

```
void *kcc_parse_unary_expr(void) {
    unsigned long t = kcc_tok_type;
    /* GNU labels-as-values: `&&label' in operand
   position is the address of a
       label (a `void *'), not a logical-and.
   Since `&&' as a binary operator
       needs a left operand, seeing it where a
   unary expression is expected is
       unambiguously the address-of-label form (
   the bpf interpreter's jumptable). */
    if (t == TOK_AND) {
        kcc_pp_next();
        void *name = kcc_strtab_intern(
   kcc_tok_start, kcc_tok_len);
        kcc_pp_next();                          /*
    consume the label identifier */
        struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_LABEL_ADDR);
        n->f0 = (unsigned long)name;
        return n;
    }
    if (t == TOK_INC || t == TOK_DEC || t ==
   TOK_AMP || t == TOK_STAR ||
        t == TOK_PLUS || t == TOK_MINUS || t ==
   TOK_TILDE || t == TOK_BANG) {
        unsigned long op = t;
        kcc_pp_next();
        void *operand = kcc_parse_cast_expr();
        struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_UNARY_EXPR);
        n->f0 = op;
        n->f1 = (unsigned long)operand;
        n->f2 = 1;                              /*
   prefix */
        return n;
    }
    if (t == TOK_SIZEOF) {
        kcc_pp_next();
        if (kcc_tok_type == TOK_LPAREN) {
            kcc_pp_next();
            if (kcc_parse_is_type_token()) {
                void *ty = kcc_parse_type_name();
                kcc_parse_expect(TOK_RPAREN);
                struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_SIZEOF_EXPR);
                n->f0 = (unsigned long)ty;
                n->f1 = 1;                      /*
   is_type */
                return n;
            }
            void *e = kcc_parse_expr();
            kcc_parse_expect(TOK_RPAREN);
            struct Node *par = (struct Node *)
   kcc_ast_alloc(AST_PAREN_EXPR);
            par->f0 = (unsigned long)e;
            struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_SIZEOF_EXPR);
            n->f0 = (unsigned long)par;
            n->f1 = 0;
            return n;
        }
        void *u = kcc_parse_unary_expr();
        struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_SIZEOF_EXPR);
        n->f0 = (unsigned long)u;
        n->f1 = 0;
        return n;
    }
    if (t == TOK__ALIGNOF) {
        kcc_pp_next();
        if (kcc_tok_type == TOK_LPAREN) {
            kcc_pp_next();
            if (kcc_parse_is_type_token()) {
                void *ty = kcc_parse_type_name();
                kcc_parse_expect(TOK_RPAREN);
                struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_ALIGNOF_EXPR);
                n->f0 = (unsigned long)ty;
                n->f1 = 1;                      /*
   is_type */
                return n;
            }
            /* GNU `__alignof__(expr)': the
   operand is a parenthesised
               expression, not a type name (C11 `
   _Alignof' forbids this, but the
               kernel's kstrtox/rseq code relies
   on it, e.g.
               `__alignof__(*rseq)').  Parse it
   like `sizeof(expr)'. */
            void *e = kcc_parse_expr();
            kcc_parse_expect(TOK_RPAREN);
            struct Node *par = (struct Node *)
   kcc_ast_alloc(AST_PAREN_EXPR);
            par->f0 = (unsigned long)e;
            struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_ALIGNOF_EXPR);
            n->f0 = (unsigned long)par;
            n->f1 = 0;
            return n;
        }
        /* `__alignof__ expr' without parentheses
   (also a GNU form). */
        void *u = kcc_parse_unary_expr();
        struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_ALIGNOF_EXPR);
        n->f0 = (unsigned long)u;
        n->f1 = 0;
        return n;
    }
    return kcc_parse_postfix_expr();
}

void *kcc_parse_cast_expr(void) {
    if (kcc_tok_type != TOK_LPAREN) return
   kcc_parse_unary_expr();
    kcc_pp_next();                              /*
   consume '(' */
    if (kcc_tok_type == TOK_LBRACE) {
        void *body = kcc_parse_compound();
        kcc_parse_expect(TOK_RPAREN);
        struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_STMT_EXPR);
        n->f0 = (unsigned long)body;
        return kcc_parse_postfix_tail(n);
    }
    if (kcc_parse_is_type_token()) {
        void *ty = kcc_parse_type_name();
        kcc_parse_expect(TOK_RPAREN);
        if (kcc_tok_type == TOK_LBRACE) {
            void *init = kcc_parse_initializer();
            struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_COMPOUND_LIT);
            n->f0 = (unsigned long)ty;
            n->f1 = (unsigned long)init;
```

```
            return kcc_parse_postfix_tail(n);
        }
        void *inner = kcc_parse_cast_expr();
        struct Node *n = (struct Node *)
    kcc_ast_alloc(AST_CAST_EXPR);
        n->f0 = (unsigned long)ty;
        n->f1 = (unsigned long)inner;
        return n;
    }
    void *e = kcc_parse_expr();
    kcc_parse_expect(TOK_RPAREN);
    struct Node *n = (struct Node *)kcc_ast_alloc(
    AST_PAREN_EXPR);
    n->f0 = (unsigned long)e;
    return kcc_parse_postfix_tail(n);
}
```

□

705 · used in 710a

⟨ *Selfhost: expressions postfix* 706 ⟩ ≡

```
void *kcc_parse_postfix_expr(void) {
    void *prim = kcc_parse_primary_expr();
    return kcc_parse_postfix_tail(prim);
}

void *kcc_parse_postfix_tail(void *expr) {
    struct Node *cur = (struct Node *)expr;
    for (;;) {
        unsigned long t = kcc_tok_type;
        if (t == TOK_LBRACKET) {
            kcc_pp_next();
            void *idx = kcc_parse_expr();
            kcc_parse_expect(TOK_RBRACKET);
            struct Node *n = (struct Node *)
    kcc_ast_alloc(AST_SUBSCRIPT);
            n->f0 = (unsigned long)cur;
            n->f1 = (unsigned long)idx;
            cur = n;
        } else if (t == TOK_LPAREN) {
            if (cur->tag == AST_IDENT_EXPR &&
                (kcc_strcmp((const char *)cur->f0,
    "va_arg") == 0 ||
                 kcc_strcmp((const char *)cur->f0,
    "__builtin_va_arg") == 0)) {
                kcc_pp_next();
                void *ap = kcc_parse_assign_expr()
    ;
                kcc_parse_expect(TOK_COMMA);
                void *ty = kcc_parse_type_name();
                kcc_parse_expect(TOK_RPAREN);
                struct Node *n = (struct Node *)
    kcc_ast_alloc(AST_VA_ARG);
                n->f0 = (unsigned long)ap;
                n->f1 = (unsigned long)ty;
                cur = n;
            } else if (cur->tag == AST_IDENT_EXPR
    &&
                kcc_strcmp((const char *)cur->f0,
    "__builtin_types_compatible_p") == 0) {
                /* GNU
    __builtin_types_compatible_p(type, type): both
    operands are
                   type names; folds to the
    integer constant 1/0. */
                kcc_pp_next();
                void *t1 = kcc_parse_type_name();
                kcc_parse_expect(TOK_COMMA);
                void *t2 = kcc_parse_type_name();
                kcc_parse_expect(TOK_RPAREN);
                struct Node *n = (struct Node *)
    kcc_ast_alloc(AST_TYPES_COMPAT);
                n->f0 = (unsigned long)t1;
                n->f1 = (unsigned long)t2;
                cur = n;
            } else if (cur->tag == AST_IDENT_EXPR
    &&
                kcc_strcmp((const char *)cur->f0,
    "__builtin_offsetof") == 0) {
                /* __builtin_offsetof(T, desig) ->
     (unsigned long)&((T*)0)->desig
                   (arrow for the first designator
     component, dot/subscript after);
                   the constant folder reduces it
    to the member's byte offset. */
                kcc_pp_next();
                void *T = kcc_parse_type_name();
                struct Node *ptrT = (struct Node
    *)kcc_ast_alloc(AST_TYPE_PTR);
                ptrT->f0 = (unsigned long)T; ptrT
    ->f1 = 0;
                struct Node *zero = (struct Node
    *)kcc_ast_alloc(AST_INT_LIT);
                zero->f0 = 0;
                struct Node *e = (struct Node *)
    kcc_ast_alloc(AST_CAST_EXPR);
                e->f0 = (unsigned long)ptrT; e->f1
     = (unsigned long)zero;
                kcc_parse_expect(TOK_COMMA);
                unsigned long arrow = 1;
                for (;;) {
                    if (kcc_tok_type == TOK_IDENT)
     {
                        void *nm =
    kcc_strtab_intern(kcc_tok_start, kcc_tok_len);
                        kcc_pp_next();
                        struct Node *m = (struct
    Node *)kcc_ast_alloc(AST_MEMBER);
                        m->f0 = (unsigned long)e;
    m->f1 = (unsigned long)nm; m->f2 = arrow;
                        e = m; arrow = 0;
                    } else if (kcc_tok_type ==
    TOK_DOT) {
                        kcc_pp_next(); arrow = 0;
                    } else if (kcc_tok_type ==
    TOK_LBRACKET) {
                        kcc_pp_next();
                        void *idx =
    kcc_parse_assign_expr();
                        kcc_parse_expect(
    TOK_RBRACKET);
                        struct Node *s = (struct
    Node *)kcc_ast_alloc(AST_SUBSCRIPT);
                        s->f0 = (unsigned long)e;
    s->f1 = (unsigned long)idx;
                        e = s; arrow = 0;
                    } else break;
                }
                kcc_parse_expect(TOK_RPAREN);
                struct Node *addr = (struct Node
    *)kcc_ast_alloc(AST_UNARY_EXPR);
                addr->f0 = TOK_AMP; addr->f1 = (
    unsigned long)e; addr->f2 = 1;
                struct Node *ty = (struct Node *)
    kcc_ast_alloc(AST_TYPE_BUILTIN);
                ty->f0 = 528;   /* unsigned long:
    SPEC_UNSIGNED(1<<9)|SPEC_LONG(1<<4) */
                struct Node *cst = (struct Node *)
    kcc_ast_alloc(AST_CAST_EXPR);
                cst->f0 = (unsigned long)ty; cst->
    f1 = (unsigned long)addr;
```

```
            cur = cst;
        } else {
            kcc_pp_next();
            void *args = kcc_parse_arg_list();
            kcc_parse_expect(TOK_RPAREN);
            struct Node *n = (struct Node *)
kcc_ast_alloc(AST_CALL_EXPR);
            n->f0 = (unsigned long)cur;
            n->f1 = (unsigned long)args;
            cur = n;
        }
    } else if (t == TOK_DOT || t == TOK_ARROW)
 {
        unsigned long arrow = (t == TOK_ARROW)
 ? 1 : 0;
        kcc_pp_next();
        void *name = kcc_strtab_intern(
kcc_tok_start, kcc_tok_len);
        kcc_pp_next();
        struct Node *n = (struct Node *)
kcc_ast_alloc(AST_MEMBER);
        n->f0 = (unsigned long)cur;
        n->f1 = (unsigned long)name;
        n->f2 = arrow;
        cur = n;
    } else if (t == TOK_INC || t == TOK_DEC) {
        kcc_pp_next();
        struct Node *n = (struct Node *)
kcc_ast_alloc(AST_UNARY_EXPR);
        n->f0 = t;
        n->f1 = (unsigned long)cur;
        n->f2 = 0;                          /*
postfix */
        cur = n;
    } else {
        break;
    }
 }
 return cur;
}
```

□

706 · used in 710a

65.4. Primary Expressions

Primary handles identifiers (whose node is allocated *before* the token is consumed, so its caret points at the identifier), integer and floating constants (classified and converted below), string and character literals (escape-processed), parenthesised expressions, and `_Generic` selection. An integer constant's suffix and magnitude set its type flags per §6.4.4.1: the suffix fixes a floor, then the value climbs the suffix- and base-dependent list to the first type that can hold it.

A character constant's value is the first byte of the escape-processed body, which `kcc_strlit_process` returns as a `struct StrLit` (pointer and length). We must receive that *as the struct* and read `s.ptr[0]` — exactly as the string-literal case receives `kcc_strlit_gather`'s result. An earlier shortcut declared the callee as `void *` and read the returned register as the pointer directly; that relied on the ARM64 convention of returning a two-word struct in `x0:x1` (buffer in `x0`). The x86-64 back end returns such a struct as a pointer to it, so grabbing the first return register yields the *address* of the pair, not the buffer, and the character's value would be the low byte of that address. Consuming the real struct is correct under either return convention.

⟨ *Selfhost: expressions primary* 707 ⟩ ≡

```
/* Value-directed integer type (§6.4.4.1): flags
   bit0=unsigned, bits1-2=rank. */
static unsigned long widen_int(unsigned long value
   , unsigned long sfx,
                               int is_decimal) {
    unsigned long ufloor = sfx & 1;
    unsigned long rfloor = (sfx >> 1) & 3;
    int fits_sint = value <= 0x7FFFFFFFUL;
    int fits_uint = value <= 0xFFFFFFFFUL;
    int fits_slong = (long)value >= 0;
    unsigned long rank, uns;
    if (ufloor) {
        uns = 1;
        if (rfloor) rank = rfloor;
        else if (fits_uint) rank = 0;
        else rank = 1;
    } else if (is_decimal) {
        if (rfloor) { rank = rfloor; uns =
fits_slong ? 0 : 1; }
        else if (fits_sint) { rank = 0; uns = 0; }
        else if (fits_slong) { rank = 1; uns = 0;
}
        else { rank = 1; uns = 1; }
    } else {
        if (rfloor) { rank = rfloor; uns =
fits_slong ? 0 : 1; }
        else if (fits_sint) { rank = 0; uns = 0; }
        else if (fits_uint) { rank = 0; uns = 1; }
        else if (fits_slong) { rank = 1; uns = 0;
}
        else { rank = 1; uns = 1; }
    }
    return uns | (rank << 1);
}

void *kcc_parse_primary_expr(void) {
    unsigned long t = kcc_tok_type;
    const char *start = kcc_tok_start;
    unsigned long len = kcc_tok_len;

    if (t == TOK_IDENT) {
        /* __func__ (\stdref{6.4.2.2}): inside a
function body it stands for a
           static string literal of the enclosing
function's name.  Materialise it
           as an ordinary string-literal node so
the rest of the pipeline (its
           char[] type, decay to char*, and data
emission) needs no special case.
           Outside any function it falls through
to the undeclared-identifier
           diagnostic. */
        if (len == 8 && start[0] == '_' && start
[1] == '_' &&
            start[2] == 'f' && start[3] == 'u' &&
start[4] == 'n' &&
            start[5] == 'c' && start[6] == '_' &&
start[7] == '_' &&
            kcc_cur_func_name) {
            unsigned long fl = kcc_strlen(
kcc_cur_func_name);
            struct StrLit s = kcc_strlit_process(
kcc_cur_func_name, fl);
            struct Node *n = (struct Node *)
kcc_ast_alloc(AST_STR_LIT);
            n->f0 = (unsigned long)s.ptr;
```

```
        n->f1 = s.len;
        kcc_pp_next();
        return n;
    }
    void *name = kcc_strtab_intern(start, len)
;
    struct Node *n = (struct Node *)
kcc_ast_alloc(AST_IDENT_EXPR);
    n->f0 = (unsigned long)name;
    kcc_pp_next();
    return n;
 }
 if (t == TOK_NUMBER) {
    if (kcc_is_float_literal(start, len)) {
        char suffix = start[len - 1];
        int is_float = (suffix == 'f' ||
suffix == 'F');
        unsigned long bits = kcc_parse_atof(
start, len);
        if (is_float) bits =
kcc_round_to_float(bits);
        kcc_pp_next();
        struct Node *n = (struct Node *)
kcc_ast_alloc(AST_FLOAT_LIT);
        n->f0 = bits;
        n->f2 = is_float;
        return n;
    }
    unsigned long value = kcc_parse_atoi(start
, len);
    unsigned long sfx = 0, lcount = 0;
    for (unsigned long i = 0; i < len; i++) {
        char c = start[i] | 0x20;
        if (c == 'u') sfx |= 1;
        else if (c == 'l') lcount++;
    }
    if (lcount > 2) lcount = 2;
    sfx |= (lcount << 1);
    int is_decimal = start[0] != '0';
    unsigned long flags = widen_int(value, sfx
, is_decimal);
    kcc_pp_next();
    struct Node *n = (struct Node *)
kcc_ast_alloc(AST_INT_LIT);
    n->f0 = value;
    n->f2 = flags;
    return n;
 }
 if (t == TOK_STRING_LIT) {
    struct StrLit s = kcc_strlit_gather();
    struct Node *n = (struct Node *)
kcc_ast_alloc(AST_STR_LIT);
    n->f0 = (unsigned long)s.ptr;
    n->f1 = s.len;
    return n;
 }
 if (t == TOK_CHAR_LIT) {
    struct StrLit s = kcc_strlit_process(start
 + 1, len - 2);
    unsigned long value = (unsigned char)((
char *)s.ptr)[0];
    kcc_pp_next();
    struct Node *n = (struct Node *)
kcc_ast_alloc(AST_CHAR_LIT);
    n->f0 = value;
    return n;
 }
 if (t == TOK_LPAREN) {
    kcc_pp_next();
    void *e = kcc_parse_expr();
    kcc_parse_expect(TOK_RPAREN);
    struct Node *n = (struct Node *)
kcc_ast_alloc(AST_PAREN_EXPR);
    n->f0 = (unsigned long)e;
    return n;
 }
 if (t == TOK__GENERIC) {
    kcc_pp_next();
    kcc_parse_expect(TOK_LPAREN);
    void *ctrl = kcc_parse_assign_expr();
    kcc_parse_expect(TOK_COMMA);
    struct Node *first = 0, *last = 0;
    for (;;) {
        void *ty;
        if (kcc_tok_type == TOK_DEFAULT) {
kcc_pp_next(); ty = 0; }
        else ty = kcc_parse_type_name();
        kcc_parse_expect(TOK_COLON);
        void *e = kcc_parse_assign_expr();
        struct Node *a = (struct Node *)
kcc_ast_alloc(AST_GENERIC_ASSOC);
        a->f0 = (unsigned long)ty;
        a->f1 = (unsigned long)e;
        if (first == 0) { first = a; last = a;
 }
        else { last->f4 = (unsigned long)a;
last = a; }
        if (kcc_tok_type != TOK_COMMA) break;
        kcc_pp_next();
    }
    kcc_parse_expect(TOK_RPAREN);
    struct Node *n = (struct Node *)
kcc_ast_alloc(AST_GENERIC);
    n->f0 = (unsigned long)ctrl;
    n->f1 = (unsigned long)first;
    return n;
 }
 kcc_parse_error("expected expression");
 return 0;
}
```

□

707 · used in 710a

65.5. Literal Conversion

parse_atoi reads decimal, hex (0x), octal (leading 0), or binary (0b) digits, stopping at a type suffix. is_float_literal classifies a pp-number (a hex literal is a float only on . or p/P, since f there is a digit). parse_atof evaluates the constant on the host FPU into a double bit pattern, folding all significant digits into one mantissa and applying a single decimal (or binary, for hex floats) scale so the result rounds once; the exact sequence of floating operations matches the assembly so the bits agree. round_to_float narrows a double to single precision and widens it back.

⟨ *Selfhost: expressions literals* 708 ⟩ ≡

```
static long hexdig(unsigned char c) {
    unsigned long t = (unsigned long)(c - '0');
    if (t <= 9) return (long)t;
    unsigned long h = (unsigned long)((c | 0x20) -
    'a');
    if (h <= 5) return (long)(h + 10);
    return -1;
}
```

```
unsigned long kcc_parse_atoi(const char *str,
   unsigned long len) {
    if (len == 0) return 0;
    unsigned long acc = 0;
    if (str[0] == '0') {
        if (len == 1) return 0;
        char c1 = str[1];
        if (c1 == 'x' || c1 == 'X') {
            for (unsigned long i = 2; i < len; i
   ++) {
                unsigned char c = (unsigned char)
   str[i];
                if (c=='u'||c=='U'||c=='l'||c=='L'
   ) break;
                acc = (acc << 4) + (unsigned long)
   hexdig(c);
            }
            return acc;
        }
        if (c1 == 'b' || c1 == 'B') {
            for (unsigned long i = 2; i < len; i
   ++) {
                unsigned char c = (unsigned char)
   str[i];
                if (c=='u'||c=='U'||c=='l'||c=='L'
   ) break;
                acc = (acc << 1) + (unsigned long)
   (c - '0');
            }
            return acc;
        }
        /* octal, including the leading '0' (
   contributes 0). */
        for (unsigned long i = 0; i < len; i++) {
            unsigned char c = (unsigned char)str[i
   ];
            if (c=='u'||c=='U'||c=='l'||c=='L')
   break;
            acc = (acc << 3) + (unsigned long)(c -
    '0');
        }
        return acc;
    }
    for (unsigned long i = 0; i < len; i++) {
        unsigned char c = (unsigned char)str[i];
        if (c=='u'||c=='U'||c=='l'||c=='L') break;
        acc = acc * 10 + (unsigned long)(c - '0');
    }
    return acc;
}

unsigned long kcc_is_float_literal(const char *str
   , unsigned long len) {
    if (len == 0) return 0;
    if (len >= 2 && str[0] == '0' && (((unsigned
   char)str[1] | 0x20) == 'x')) {
        for (unsigned long i = 2; i < len; i++) {
            unsigned char c = (unsigned char)str[i
   ];
            if (c == '.') return 1;
            if ((c | 0x20) == 'p') return 1;
        }
        return 0;
    }
    for (unsigned long i = 0; i < len; i++) {
        char c = str[i];
        if (c=='.'||c=='e'||c=='E'||c=='f'||c=='F'
   ) return 1;
    }
    return 0;
}

unsigned long kcc_parse_atof(const char *str,
   unsigned long len) {
    const char *end = str + len;
    if (len >= 2 && str[0] == '0' && (((unsigned
   char)str[1] | 0x20) == 'x')) {
        const char *p = str + 2;
        double d = 0.0;
        while (p < end) {
            unsigned char c = (unsigned char)*p;
            if (c == '.' || (c | 0x20) == 'p')
   break;
            long dig = hexdig(c);
            if (dig < 0) break;
            d = d * 16.0 + (double)dig;
            p++;
        }
        if (p < end && *p == '.') {
            p++;
            double div = 1.0;
            while (p < end) {
                unsigned char c = (unsigned char)*
   p;
                if ((c | 0x20) == 'p') break;
                long dig = hexdig(c);
                if (dig < 0) break;
                div = div * 16.0;
                d = d + (double)dig / div;
                p++;
            }
        }
        long emag = 0; int esign = 0;
        if (p < end && ((unsigned char)*p | 0x20)
   == 'p') {
            p++;
            if (p < end) {
                char c = *p;
                if (c == '+') p++;
                else if (c == '-') { esign = 1; p
   ++; }
            }
            while (p < end) {
                unsigned long dv = (unsigned long)
   ((unsigned char)*p - '0');
                if (dv > 9) break;
                emag = emag * 10 + (long)dv;
                p++;
            }
        }
        if (emag != 0) {
            if (!esign) for (long i = 0; i < emag;
    i++) d = d * 2.0;
            else for (long i = 0; i < emag; i++) d
    = d / 2.0;
        }
        return double_to_bits(d);
    }
    /* decimal --- accumulate at most 18
   significant digits into the 64-bit
       mantissa M; a double keeps ~17, and feeding
    every digit of an ultra-long
       literal (Lua's `\_VERSION' aside, its pi is
       3.14159265358979323846264338327950288)
   overflowed M and produced garbage
       -- 8.6e-18 instead of pi.  Integer digits
   past the cap raise the exponent
       (`drop'); fractional digits past it are
   dropped without scaling. */
    const char *p = str;
    unsigned long M = 0, F = 0;
```

```
    long emag = 0, drop = 0; int esign = 0, digits
    = 0;
    while (p < end) {
        char c = *p;
        if (c == '.' || c == 'e' || c == 'E')
   break;
        unsigned long dv = (unsigned long)((
   unsigned char)c - '0');
        if (dv > 9) break;
        if (digits < 18) { M = M * 10 + dv; digits
   ++; } else drop++;
        p++;
    }
    if (p < end && *p == '.') {
        p++;
        while (p < end) {
            char c = *p;
            if (c == 'e' || c == 'E') break;
            unsigned long dv = (unsigned long)((
   unsigned char)c - '0');
            if (dv > 9) break;
            if (digits < 18) { M = M * 10 + dv; F
   ++; digits++; }
            p++;
        }
    }
    if (p < end && (*p == 'e' || *p == 'E')) {
        p++;
        if (p < end) {
            char c = *p;
            if (c == '+') p++;
            else if (c == '-') { esign = 1; p++; }
        }
        while (p < end) {
            unsigned long dv = (unsigned long)((
   unsigned char)*p - '0');
            if (dv > 9) break;
            emag = emag * 10 + (long)dv;
            p++;
        }
    }
    long net = (esign ? -emag : emag) - (long)F +
   drop;
    double m = (double)(long)M;
    long k = net < 0 ? -net : net;
    double pw = 1.0;
    for (long i = 0; i < k; i++) pw = pw * 10.0;
    double r = (net >= 0) ? (m * pw) : (m / pw);
    return double_to_bits(r);
}

unsigned long kcc_round_to_float(unsigned long
   bits) {
    double d = bits_to_double(bits);
    float f = (float)d;
    double d2 = (double)f;
    return double_to_bits(d2);
}

void *kcc_parse_arg_list(void) {
    if (kcc_tok_type == TOK_RPAREN) return 0;
    struct Node *first = 0, *last = 0;
    for (;;) {
        struct Node *arg = (struct Node *)
   kcc_parse_assign_expr();
        if (first == 0) { first = arg; last = arg;
    }
        else { last->f4 = (unsigned long)arg; last
    = arg; }
        if (kcc_tok_type != TOK_COMMA) break;
        kcc_pp_next();
    }
    return first;
}
```

□

708 · used in 710a

65.6. Assembling the Translation Unit

⟨ *Selfhost: expressions* 710a ⟩ ≡

```
⟨ Selfhost: expressions preamble 703 ⟩
⟨ Selfhost: expressions core 704 ⟩
⟨ Selfhost: expressions unary 705 ⟩
⟨ Selfhost: expressions postfix 706 ⟩
⟨ Selfhost: expressions primary 707 ⟩
⟨ Selfhost: expressions literals 708 ⟩
```

□

710a · used in 351

65.7. Verification

`make selfhost-expressions` runs the drop-in loop. Because every expression in every function the suite compiles is parsed here, a green code-generation suite—and `difftest`, which checks the values programs compute, floating constants included—means kcc compiled its own expression grammar and literal arithmetic correctly.

⟨ *Selfhost: expressions verify* 710b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-expressions - compile kcc's
   expression parser in C with kcc,
# relink, run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/expressions.c
    with kcc"
$BASE/kcc -c "$SH/expressions.c" -o "$SH/
   expressions_c.o" >/dev/null

echo "[2/3] linking $BASE/kcc_selfhost_expressions
    (expressions.o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "expressions" ]; then OBJ="$OBJ
   $SH/expressions_c.o"; else OBJ="$OBJ $BASE/src/
   ${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_expressions $OBJ -lSystem
    -syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted expression parser"
cp $BASE/kcc "$SH/kcc_asm_backup_expressions"
```

```
cp $BASE/kcc_selfhost_expressions $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_expressions" $BASE/kcc
exit $status
```

□

710b · used in 35m · uses _main 621, 629

66. Self-Hosting: The Parser Spine in C

With the statement and expression parsers already in C (Chapters 63 and 65), this port completes the parser: the spine they stand on. It is the node allocator that stamps every AST node with its source location, the token-consumption helpers, the "expected" diagnostic, the typedef-name scope stack that resolves C's declaration/expression ambiguity, the translation-unit driver, and the `--dump-ast` tree printer. The code-generation suite exercises the whole spine on every compile; the tree printer is checked separately by comparing `--dump-ast` output against the assembly's.

66.1. Nodes, State, and the Tag Table

The port declares the 64-byte node and the vector header (so it can pop a typedef scope by decrementing the count), owns the parser's three globals (the error flag, the typedef scope stack, and the dump indentation), and carries the tag-name table the dumper looks names up in. Externs are the arena, the containers, the token stream, the diagnostic engine, the token-name helper, buffered output, and the declaration parser.

⟨ *Selfhost: parser preamble* 713 ⟩ ≡

```
struct Node {
    unsigned long tag;
    unsigned long line;
    unsigned long col;
    unsigned long f0;
    unsigned long f1;
    unsigned long f2;
    unsigned long f3;
    unsigned long f4;
};

struct Vec {
    void *data;
    unsigned long count;      /* VEC_COUNT, 8 */
    unsigned long cap;
    unsigned long elem_size;
};

void *kcc_arena_alloc(long size);
void *kcc_vec_init(unsigned long elem_size);
void kcc_vec_push(void *vec, const void *elem);
void *kcc_vec_get(void *vec, unsigned long index);
void *kcc_ht_init(unsigned long capacity);
unsigned long kcc_ht_get(void *ht, const void *key
   , unsigned long keylen);
void kcc_ht_put(void *ht, const void *key,
   unsigned long keylen,
                unsigned long value);
int kcc_memcmp(const void *a, const void *b,
   unsigned long n);
void kcc_pp_next(void);
void kcc_diag_emit(int severity, unsigned long
   line, unsigned long col,
                   const char *msg);
const char *kcc_lex_type_name(unsigned long type);
void kcc_io_puts(const char *s);
void kcc_io_putc(unsigned long c);
void kcc_io_putu(unsigned long n);
void kcc_io_write(const void *buf, unsigned long
   len);
void *kcc_parse_decl(void);
struct StrLit { void *ptr; unsigned long len; };
struct StrLit kcc_strlit_gather(void);
void *kcc_parse_asm_extended(const char *tmpl,
   unsigned long len);

extern unsigned long kcc_tok_type;
extern unsigned long kcc_tok_line;
extern unsigned long kcc_tok_col;

unsigned long kcc_parse_error_flag = 0;
void *kcc_typedef_scopes = 0;
unsigned long kcc_ast_indent = 0;

enum {
    TOK_EOF = 0, TOK_LPAREN = 60, TOK_RPAREN = 61,
    TOK_LBRACE = 62,
    TOK_RBRACE = 63, TOK_LBRACKET = 64,
   TOK_RBRACKET = 65, TOK_SEMI = 66,
    TOK_COMMA = 67, TOK_COLON = 93, TOK_ASM = 56
};

enum {
    AST_TRANS_UNIT = 1, AST_VAR_DECL = 10,
   AST_FUNC_DECL = 11,
    AST_PARAM_DECL = 12, AST_TYPEDEF_DECL = 13,
   AST_STRUCT_DECL = 14,
    AST_UNION_DECL = 15, AST_ENUM_DECL = 16,
   AST_ENUMERATOR = 17,
    AST_FIELD_DECL = 18, AST_TYPE_BUILTIN = 20,
   AST_TYPE_NAME = 21,
    AST_TYPE_PTR = 22, AST_TYPE_ARRAY = 23,
   AST_TYPE_FUNC = 24,
    AST_COMPOUND = 30, AST_IF_STMT = 32,
   AST_SWITCH_STMT = 33,
    AST_WHILE_STMT = 34, AST_DO_STMT = 35,
   AST_FOR_STMT = 36,
    AST_CASE_STMT = 42, AST_LABEL_STMT = 41,
   AST_DEFAULT_STMT = 43,
    AST_IDENT_EXPR = 50, AST_INT_LIT = 51,
   AST_STR_LIT = 52, AST_CHAR_LIT = 53,
    AST_BINARY_EXPR = 54, AST_UNARY_EXPR = 55,
   AST_ASSIGN_EXPR = 56,
    AST_CALL_EXPR = 58, AST_MEMBER = 60,
   AST_SIZEOF_EXPR = 62,
    AST_CAST_EXPR = 61, AST_FLOAT_LIT = 71
};

struct TagName { unsigned long tag; const char *
   name; };
static struct TagName tag_table[] = {
    {1,"TranslationUnit"}, {10,"VarDecl"}, {11,"
   FuncDecl"}, {12,"ParamDecl"},
    {13,"TypedefDecl"}, {14,"StructDecl"}, {15,"
   UnionDecl"}, {16,"EnumDecl"},
    {17,"Enumerator"}, {18,"FieldDecl"}, {19,"
   StaticAssert"}, {20,"BuiltinType"},
    {21,"TypeName"}, {22,"PointerType"}, {23,"
   ArrayType"}, {24,"FuncType"},
    {25,"StructType"}, {26,"UnionType"}, {27,"
   EnumType"}, {28,"QualType"},
    {30,"CompoundStmt"}, {31,"ExprStmt"}, {32,"
   IfStmt"}, {33,"SwitchStmt"},
    {34,"WhileStmt"}, {35,"DoStmt"}, {36,"ForStmt"
   }, {37,"GotoStmt"},
    {38,"ContinueStmt"}, {39,"BreakStmt"}, {40,"
   ReturnStmt"}, {41,"LabelStmt"},
```

```
    {42,"CaseStmt"}, {43,"DefaultStmt"}, {44,"
   NullStmt"}, {50,"IdentExpr"},
    {51,"IntLit"}, {71,"FloatLit"}, {72,"VaArg"},
   {52,"StrLit"}, {53,"CharLit"},
    {54,"BinaryExpr"}, {55,"UnaryExpr"}, {56,"
   AssignExpr"}, {57,"CondExpr"},
    {58,"CallExpr"}, {59,"Subscript"}, {60,"
   MemberExpr"}, {61,"CastExpr"},
    {62,"SizeofExpr"}, {63,"AlignofExpr"}, {64,"
   CommaExpr"}, {65,"InitList"},
    {67,"CompoundLit"}, {68,"ParenExpr"}, {69,"
   GenericExpr"},
    {70,"GenericAssoc"}, {0,0}
};

void *kcc_ast_alloc(unsigned long tag);
void kcc_ast_dump(void *node);
void kcc_ast_indent_inc(void);
void kcc_ast_indent_dec(void);
void kcc_ast_print_indent(void);
void kcc_ast_print_name_field(const char *name);
const char *kcc_ast_tag_name(unsigned long tag);
```

□

713 · used in 717a

66.2. Allocation, Token Helpers, Errors

ast_alloc zeroes a 64-byte node, tags it, and stamps the current token's line and column so a diagnostic caret lands on the right token. peek/advance/match/expect are the token interface; expect reads a specific "expected 'X'" message for the common punctuators. error sets the error flag and emits at the current location; sync skips to the next ; or }.

⟨ *Selfhost: parser core* 714a ⟩ ≡

```
void *kcc_ast_alloc(unsigned long tag) {
    struct Node *n = (struct Node *)
   kcc_arena_alloc(64);
    n->tag = tag; n->line = kcc_tok_line; n->col =
    kcc_tok_col;
    n->f0 = 0; n->f1 = 0; n->f2 = 0; n->f3 = 0; n
   ->f4 = 0;
    return n;
}

unsigned long kcc_parse_peek(void) { return
   kcc_tok_type; }
void kcc_parse_advance(void) { kcc_pp_next(); }

int kcc_parse_match(unsigned long tok_type) {
    if (kcc_tok_type == tok_type) { kcc_pp_next();
    return 1; }
    return 0;
}

void kcc_parse_error(const char *msg) {
    kcc_parse_error_flag = 1;
    kcc_diag_emit(0, kcc_tok_line, kcc_tok_col,
   msg);  /* DIAG_ERROR */
}

void kcc_parse_expect(unsigned long tok_type) {
    if (kcc_tok_type == tok_type) { kcc_pp_next();
    return; }
    const char *msg;
    if (tok_type == TOK_SEMI) msg = "expected ';'"
   ;
    else if (tok_type == TOK_LPAREN) msg = "
   expected '('";
    else if (tok_type == TOK_RPAREN) msg = "
   expected ')'";
    else if (tok_type == TOK_LBRACE) msg = "
   expected '{'";
    else if (tok_type == TOK_RBRACE) msg = "
   expected '}'";
    else if (tok_type == TOK_LBRACKET) msg = "
   expected '['";
    else if (tok_type == TOK_RBRACKET) msg = "
   expected ']'";
    else if (tok_type == TOK_COMMA) msg = "
   expected ','";
    else if (tok_type == TOK_COLON) msg = "
   expected ':'";
    else msg = "expected token";
    kcc_parse_error(msg);
}

void kcc_parse_sync(void) {
    for (;;) {
        unsigned long t = kcc_tok_type;
        if (t == TOK_EOF) return;
        if (t == TOK_SEMI) { kcc_pp_next(); return
   ; }
        if (t == TOK_RBRACE) return;
        kcc_pp_next();
    }
}
```

□

714a · used in 717a

66.3. Typedef Scopes and the Translation Unit

The typedef stack is a vector of hash tables, one per scope. init makes the vector and a file-scope table; push_scope appends a fresh table and pop_scope drops the top by decrementing the count (the arena reclaims it). add inserts a name in the innermost scope as a typedef (value 1); shadow inserts it as an ordinary identifier (value 2) when a variable or function reuses a typedef's name, since §6.2.1p4 hides the type-name for the rest of that scope. lookup searches top-down (so the innermost binding wins): a value 2 answers "not a type" immediately, a value 1 answers "type", and only a miss falls through to the next scope out; va_list and its GNU spelling __builtin_va_list are built-in type-names, checked last so a local of that name can shadow them too. parse reads the first token and appends external declarations—following each declarator group's f4 tail—into a translation-unit node.

⟨ *Selfhost: parser typedef* 714b ⟩ ≡

```
void kcc_typedef_init(void) {
    kcc_typedef_scopes = kcc_vec_init(8);
    void *ht = kcc_ht_init(64);
    kcc_vec_push(kcc_typedef_scopes, &ht);
}

void kcc_typedef_push_scope(void) {
    void *ht = kcc_ht_init(32);
```

```
    kcc_vec_push(kcc_typedef_scopes, &ht);
}

void kcc_typedef_pop_scope(void) {
    ((struct Vec *)kcc_typedef_scopes)->count -=
   1;
}

void kcc_typedef_add(const char *name, unsigned
   long len) {
    struct Vec *v = (struct Vec *)
   kcc_typedef_scopes;
    void *ht = *(void **)kcc_vec_get(v, v->count -
    1);
    kcc_ht_put(ht, name, len, 1);
}

/* Record that an ordinary identifier (a variable
   or function) whose name matches
   a typedef has been declared in the innermost
   scope: within that scope the name
   is no longer a type-name (\stdref{6.2.1}p4).
   The kernel's efi libstub vsprintf
   declares a `u8 *s8' local that shadows the `s8'
    typedef and then writes
   `for (s8 += clen; ...)'---without this the
   parser reads `s8' as a type and the
   for-init as a declaration, failing at the `+='.
    */
void kcc_typedef_shadow(const char *name, unsigned
    long len) {
    struct Vec *v = (struct Vec *)
   kcc_typedef_scopes;
    void *ht = *(void **)kcc_vec_get(v, v->count -
    1);
    kcc_ht_put(ht, name, len, 2);   /* 2 =
   shadowed non-type */
}

int kcc_typedef_lookup(const char *name, unsigned
   long len) {
    struct Vec *v = (struct Vec *)
   kcc_typedef_scopes;
    unsigned long i = v->count;
    while (i > 0) {
        i = i - 1;
        void *ht = *(void **)kcc_vec_get(v, i);
        unsigned long val = (unsigned long)
   kcc_ht_get(ht, name, len);
        if (val == 2) return 0;     /* shadowed by
    a variable here: not a type */
        if (val) return 1;          /* a typedef
   in this scope */
    }
    if (len == 7 && kcc_memcmp(name, "va_list", 7)
    == 0) return 1;
    if (len == 17 && kcc_memcmp(name, "
   __builtin_va_list", 17) == 0) return 1;
    return 0;
}

/* A file-scope \texttt{asm("...")} declaration (a
    GNU top-level asm, which is
   what \texttt{EXPORT_SYMBOL} expands to): parse
   it like the statement form and
   hand back the \texttt{AST_ASM_STMT} so the
   translation unit carries it to the
   emitter, which pastes the template verbatim
   into the output. */
void *kcc_parse_toplevel_asm(void) {
    kcc_pp_next();                          /*
   consume 'asm' */
    while (kcc_tok_type != TOK_LPAREN &&
   kcc_tok_type != TOK_EOF)
        kcc_pp_next();                      /*
   skip volatile/goto */
    kcc_parse_expect(TOK_LPAREN);
    struct StrLit tmpl = kcc_strlit_gather();
    void *n = kcc_parse_asm_extended((const char
   *)tmpl.ptr, tmpl.len);
    kcc_parse_expect(TOK_SEMI);
    return n;
}
void *kcc_parse(void) {
    kcc_pp_next();
    struct Node *tu = (struct Node *)kcc_ast_alloc
   (AST_TRANS_UNIT);
    struct Node *first = 0, *last = 0;
    while (kcc_tok_type != TOK_EOF) {
        if (kcc_tok_type == TOK_SEMI) {
   kcc_pp_next(); continue; }  /* stray ';' */
        struct Node *d;
        if (kcc_tok_type == TOK_ASM) d = (struct
   Node *)kcc_parse_toplevel_asm();
        else d = (struct Node *)kcc_parse_decl();
        if (d == 0) continue;
        struct Node *tail = d;
        while (tail->f4) tail = (struct Node *)
   tail->f4;
        if (first == 0) { first = d; last = tail;
   }
        else { last->f4 = (unsigned long)d; last =
    tail; }
    }
    tu->f0 = (unsigned long)first;
    return tu;
}
```

□

714b · used in 717a

66.4. The AST Dumper

ast_dump prints a node's tag and location, then dispatches on the tag to print its fields and recurse into its children, indenting by depth. The handlers mirror the assembly one for one—which fields are children, which are names, operators, or flags—so the --dump-ast output matches byte for byte.

⟨ *Selfhost: parser dump* 715 ⟩ ≡

```
const char *kcc_ast_tag_name(unsigned long tag) {
    for (struct TagName *e = tag_table; e->name; e
   ++)
        if (e->tag == tag) return e->name;
    return "<unknown>";
}

void kcc_ast_indent_inc(void) { kcc_ast_indent +=
   1; }
void kcc_ast_indent_dec(void) { kcc_ast_indent -=
   1; }

void kcc_ast_print_indent(void) {
    unsigned long n = kcc_ast_indent * 2;
    while (n) { kcc_io_putc(' '); n--; }
}

void kcc_ast_print_name_field(const char *name) {
    if (name == 0) return;
```

```
    kcc_io_puts(" ");
    kcc_io_putc('\'');
    kcc_io_puts(name);
    kcc_io_putc('\'');
}

static void dump_list(struct Node *head) {
    for (struct Node *c = head; c; c = (struct
   Node *)c->f4)
        kcc_ast_dump(c);
}

void kcc_ast_dump(void *nodep) {
    struct Node *n = (struct Node *)nodep;
    if (n == 0) return;
    kcc_ast_print_indent();
    kcc_io_puts(kcc_ast_tag_name(n->tag));
    kcc_io_puts(" @ ");
    kcc_io_putu(n->line);
    kcc_io_putc(':');
    kcc_io_putu(n->col);

    unsigned long t = n->tag;

    if (t == AST_TRANS_UNIT || t == AST_COMPOUND)
   {
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        dump_list((struct Node *)n->f0);
        kcc_ast_indent_dec();
        return;
    }
    if (t == AST_VAR_DECL) {
        kcc_ast_print_name_field((const char *)n->
   f1); kcc_io_putc('\n');
        kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0); kcc_ast_dump
   ((void *)n->f2);
        kcc_ast_indent_dec(); return;
    }
    if (t == AST_FUNC_DECL) {
        kcc_ast_print_name_field((const char *)n->
   f1); kcc_io_putc('\n');
        kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0);
        dump_list((struct Node *)n->f2);
        kcc_ast_dump((void *)n->f3);
        kcc_ast_indent_dec(); return;
    }
    if (t == AST_PARAM_DECL || t ==
   AST_TYPEDEF_DECL || t == AST_FIELD_DECL) {
        kcc_ast_print_name_field((const char *)n->
   f1); kcc_io_putc('\n');
        kcc_ast_indent_inc(); kcc_ast_dump((void
   *)n->f0); kcc_ast_indent_dec();
        return;
    }
    if (t == AST_STRUCT_DECL || t ==
   AST_UNION_DECL || t == AST_ENUM_DECL) {
        kcc_ast_print_name_field((const char *)n->
   f0); kcc_io_putc('\n');
        kcc_ast_indent_inc(); dump_list((struct
   Node *)n->f1); kcc_ast_indent_dec();
        return;
    }
    if (t == AST_ENUMERATOR) {
        kcc_ast_print_name_field((const char *)n->
   f0); kcc_io_putc('\n');
        kcc_ast_indent_inc(); kcc_ast_dump((void
   *)n->f1); kcc_ast_indent_dec();
        return;
    }
    if (t == AST_TYPE_BUILTIN) {
        kcc_io_puts(" "); kcc_io_puts(
kcc_lex_type_name(n->f0)); kcc_io_putc('\n');
        return;
    }
    if (t == AST_TYPE_NAME || t == AST_IDENT_EXPR)
    {
        kcc_ast_print_name_field((const char *)n->
f0); kcc_io_putc('\n');
        return;
    }
    if (t == AST_TYPE_PTR) {
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0);
kcc_ast_indent_dec(); return;
    }
    if (t == AST_TYPE_ARRAY) {
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0); kcc_ast_dump
((void *)n->f1);
        kcc_ast_indent_dec(); return;
    }
    if (t == AST_TYPE_FUNC) {
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0);
        dump_list((struct Node *)n->f1);
        kcc_ast_indent_dec(); return;
    }
    if (t == AST_INT_LIT || t == AST_FLOAT_LIT ||
t == AST_CHAR_LIT) {
        kcc_io_puts(" "); kcc_io_putu(n->f0);
kcc_io_putc('\n'); return;
    }
    if (t == AST_STR_LIT) {
        kcc_io_puts(" "); kcc_io_putc('"');
        kcc_io_write((const void *)n->f0, n->f1);
        kcc_io_putc('"'); kcc_io_putc('\n');
return;
    }
    if (t == AST_BINARY_EXPR || t ==
AST_ASSIGN_EXPR) {
        kcc_io_puts(" op="); kcc_io_puts(
kcc_lex_type_name(n->f0)); kcc_io_putc('\n');
        kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f1); kcc_ast_dump
((void *)n->f2);
        kcc_ast_indent_dec(); return;
    }
    if (t == AST_UNARY_EXPR) {
        kcc_io_puts(" op="); kcc_io_puts(
kcc_lex_type_name(n->f0));
        kcc_io_puts(n->f2 ? " prefix" : " postfix"
);
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f1);
kcc_ast_indent_dec(); return;
    }
    if (t == AST_CALL_EXPR) {
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0);
        dump_list((struct Node *)n->f1);
        kcc_ast_indent_dec(); return;
    }
    if (t == AST_MEMBER) {
        kcc_ast_print_name_field((const char *)n->
f1);
        kcc_io_puts(n->f2 ? " ->" : " .");
        kcc_io_putc('\n'); kcc_ast_indent_inc();
```

```
        kcc_ast_dump((void *)n->f0);
    kcc_ast_indent_dec(); return;
     }
     if (t == AST_SIZEOF_EXPR) {
        kcc_io_puts(n->f1 ? " type" : " expr");
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0);
    kcc_ast_indent_dec(); return;
     }
     if (t == AST_CAST_EXPR) {
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0); kcc_ast_dump
    ((void *)n->f1);
        kcc_ast_indent_dec(); return;
     }
     if (t == AST_IF_STMT) {
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0); kcc_ast_dump
    ((void *)n->f1);
        kcc_ast_dump((void *)n->f2);
    kcc_ast_indent_dec(); return;
     }
     if (t == AST_WHILE_STMT || t == AST_DO_STMT ||
     t == AST_SWITCH_STMT ||
        t == AST_CASE_STMT) {
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0); kcc_ast_dump
    ((void *)n->f1);
        kcc_ast_indent_dec(); return;
     }
     if (t == AST_FOR_STMT) {
        kcc_io_putc('\n'); kcc_ast_indent_inc();
        kcc_ast_dump((void *)n->f0); kcc_ast_dump
    ((void *)n->f1);
        kcc_ast_dump((void *)n->f2); kcc_ast_dump
    ((void *)n->f3);
        kcc_ast_indent_dec(); return;
     }
     if (t == AST_LABEL_STMT) {
        kcc_ast_print_name_field((const char *)n->
    f0); kcc_io_putc('\n');
        kcc_ast_indent_inc(); kcc_ast_dump((void
    *)n->f1); kcc_ast_indent_dec();
        return;
     }
     /* Generic: newline, then the first child only
    . */
     kcc_io_putc('\n'); kcc_ast_indent_inc();
     if (n->f0) kcc_ast_dump((void *)n->f0);
     kcc_ast_indent_dec();
}
```

□

715 · used in 717a

66.5. Assembling the Translation Unit

⟨ *Selfhost: parser* 717a ⟩ ≡

```
⟨ Selfhost: parser preamble 713 ⟩
⟨ Selfhost: parser core 714a ⟩
⟨ Selfhost: parser typedef 714b ⟩
⟨ Selfhost: parser dump 715 ⟩
```

□

717a · used in 35n

66.6. Verification

`make selfhost-parser` runs the drop-in loop through the code-generation suite (which drives `kcc_parse`, `ast_alloc`, the token helpers, and the typedef stack on every program) and additionally diffs `--dump-ast` output against the assembly writer's, so the tree printer is verified too.

⟨ *Selfhost: parser verify* 717b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-parser - compile kcc's parser
   spine in C with kcc, relink, run
# the codegen suite and check --dump-ast byte-
   identical.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/4] compiling $BASE/selfhost/parser.c with
    kcc"
$BASE/kcc -c "$SH/parser.c" -o "$SH/parser_c.o" >/
   dev/null

echo "[2/4] linking $BASE/kcc_selfhost_parser (
   parser.o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "parser" ]; then OBJ="$OBJ $SH/
   parser_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
   fi
done
ld -o $BASE/kcc_selfhost_parser $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/4] --dump-ast byte-identical check"
cat > "$SH/dump_probe.c" <<'PROBE'
typedef int myint;
struct P { int x; int y; };
int add(int a, int b) { return a + b; }
int main(void) {
    myint n = 3;
    int arr[2] = {1, 2};
    for (int i = 0; i < n; i++) { arr[0] += add(i,
    arr[1]) * (i ? 2 : 3); }
    return arr[0] > 5 ? sizeof(int) : arr[0];
}
PROBE
$BASE/kcc --dump-ast "$SH/dump_probe.c" > "$SH/
   dump_asm.txt" 2>/dev/null || true
$BASE/kcc_selfhost_parser --dump-ast "$SH/
   dump_probe.c" > "$SH/dump_c.txt" 2>/dev/null ||
    true
if ! diff -q "$SH/dump_asm.txt" "$SH/dump_c.txt"
   >/dev/null; then
    echo "  --dump-ast MISMATCH:"; diff "$SH/
   dump_asm.txt" "$SH/dump_c.txt" | head -20; exit
    1
fi
echo "  --dump-ast identical"
```

```
echo "[4/4] running the codegen suite with the
   self-hosted parser spine"
cp $BASE/kcc "$SH/kcc_asm_backup_parser"
cp $BASE/kcc_selfhost_parser $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_parser" $BASE/kcc
exit $status
```

□

717b · used in 35o · uses _main 621, 629

67. Self-Hosting: The Compilation Driver in C

The driver of Chapter 43 is the compiler's outer shell: it runs every phase in order for one file, loops that over many under `-c`, and—invoked without `-c`—compiles and then links an executable by running the system assembler and linker as child processes. Porting it puts the orchestration itself through kcc-compiled C: the argument parsing, the straight-line pipeline, the fork/exec spawning, and the linker command line. Every compilation the code-generation suite runs and every executable the end-to-end suite links passes through this file, so a green sweep of both is the witness.

67.1. State, Strings, and the Pipeline

The port owns the two driver globals (the captured environment and the `--emit-asm` flag) and declares as externs the target/debug flags other modules own, the whole phase pipeline, and the syscalls used to spawn processes. The command-line option spellings and the assembler/linker paths are string constants—compared against arguments and passed as `argv` elements. `kcc_driver_compile` is the sequential pipeline: read the source, run the phases to optimized IR, then either dispatch to the x86 back end or generate ARM64 code and write the Mach-O object.

⟨ *Selfhost: driver preamble* 719 ⟩ ≡

```
void *kcc_arena_alloc(long size);
void *kcc_memcpy(void *dst, const void *src,
   unsigned long n);
int kcc_memcmp(const void *a, const void *b,
   unsigned long n);
unsigned long kcc_strlen(const char *s);
int kcc_strcmp(const char *a, const char *b);
void kcc_io_flush(void);
void kcc_io_err(const char *s);

void kcc_source_open(const char *path);
void kcc_source_encode(void);
void kcc_source_splice(void);
void kcc_lex_init(void *base, unsigned long size);
void kcc_pp_init(void);
void kcc_pp_force_include(const char *path);
void kcc_strtab_init(void);
void kcc_typedef_init(void);
void *kcc_parse(void);
void kcc_types_init(void);
void kcc_symtab_init(void);
void kcc_sema_check(void *tu);
unsigned long kcc_diag_had_errors(void);
void kcc_ir_init(void);
void kcc_ir_build(void *tu);
void kcc_opt_run(void);
unsigned long kcc_ir_validate(void);
void kcc_code_init(void);
void kcc_linker_init(void);
void kcc_codegen_init(void);
void kcc_preregister_globals(void *tu);
void kcc_codegen_all(void);
void kcc_codegen_emit_strings(void);
int kcc_as_render(const char *path);
int kcc_x86_render(const char *path);
void kcc_dwarf_build(void);
void kcc_macho_init(const char *out);
void kcc_macho_finalize(void);
void kcc_pp_add_include_dir(const char *dir);
void kcc_pp_add_cmdline_define(const char *def);

long kcc_sys_fork(void);
long kcc_sys_execve(const char *path, char *const
   argv[], char *const envp[]);
long kcc_sys_wait4(long pid, int *status, long
   options, void *rusage);
long kcc_sys_open(const char *path, long flags,
   long mode);
long kcc_sys_close(long fd);
long kcc_sys_write(long fd, const void *buf, long
   len);
long kcc_sys_unlink(const char *path);
void kcc_sys_exit(int status);

extern void *kcc_src_base;
extern unsigned long kcc_src_size;
extern unsigned long kcc_target;
extern unsigned long kcc_x86_os;
extern const char *kcc_dep_out;
   /* .d output path for -MD/-Wp,-MMD */
void kcc_dep_reset(void);
void kcc_dep_write(const char *obj, const char *
   src);
extern void *kcc_x86_tu;
extern unsigned long kcc_warn_all;
extern unsigned long kcc_tu_salt;
   /* per-file static-name salt (ir.c) */
extern unsigned long kcc_dwarf_enabled;

void *kcc_envp = 0;
unsigned long kcc_emit_asm = 0;
unsigned long kcc_host_linux = 0;    /* set to 1
   by the Linux _start (os_linux) */

static const char OPT_O[] = "-o";
static const char OPT_COMPILE[] = "-c";
static const char OPT_WALL[] = "-Wall";
static const char OPT_G[] = "-g";
static const char OPT_EMITASM[] = "--emit-asm";
static const char OPT_TARGET_X86[] = "--target=
   x86_64";
static const char OPT_TARGET_LINUX[] = "--target=
   x86_64-linux";
static const char STR_X86_64[] = "x86_64";
static const char STR_AS_ARGV0[] = "as";
static const char STR_AS_PATH[] = "/usr/bin/as";
static const char STR_DOT_S[] = ".s";
static const char STR_ARCH[] = "-arch";
static const char STR_LD_ARGV0[] = "ld";
static const char STR_LD_PATH[] = "/usr/bin/ld";
static const char STR_LSYSTEM[] = "-lSystem";
static const char STR_SYSLIBROOT[] = "-syslibroot"
   ;
static const char STR_ARM64[] = "arm64";
static const char STR_DASH_E[] = "-e";
static const char STR_UMAIN[] = "_main";
static const char STR_AS_64[] = "--64";
   /* ELF: assemble 64-bit */
static const char STR_STATIC[] = "-static";
   /* ELF: static, no dynamic loader */
```

```
static const char STR_NOSTDLIB[] = "-nostdlib";
   /* ELF: no libc; kcc supplies the runtime */
static const char STR_USTART[] = "_start";
   /* ELF entry point */
static const char STR_DOT_O[] = ".o";
static const char STR_AOUT[] = "a.out";
static const char STR_SDKROOT_ENV[] = "SDKROOT";
static const char STR_SDK_CLT[] =
    "/Library/Developer/CommandLineTools/SDKs/
   MacOSX.sdk";
static const char STR_SDK_XCODE[] =
    "/Applications/Xcode.app/Contents/Developer/
   Platforms/MacOSX.platform/Developer/SDKs/MacOSX
   .sdk";
static const char ERR_NOINPUT[] = "kcc: error: no
   input files\n";

/* The Linux runtime kcc emits and assembles at
   link time so that a -nostdlib
   static executable has an entry point and
   syscall stubs.  It mirrors the
   hand-written os_linux.s that kcc itself is
   built from --- the same syscall
   numbers and the same macOS-to-Linux flag/
   argument translations --- minus the
   kcc-internal kcc_host_linux hook, so it links
   against any freestanding TU. */
static const char STR_RT_S[] = "kcc_rt.s";
static const char STR_RT_O[] = "kcc_rt.o";
static const char STR_RUNTIME[] =
    ".set SYS_read,0\n.set SYS_write,1\n.set
   SYS_open,2\n.set SYS_close,3\n"
    ".set SYS_lseek,8\n.set SYS_mmap,9\n.set
   SYS_munmap,11\n.set SYS_fork,57\n"
    ".set SYS_execve,59\n.set SYS_exit,60\n.set
   SYS_wait4,61\n.set SYS_unlink,87\n"
    "\t.comm kcc_host_linux,8,8\n"    /* merges
   with the kcc that links us, if any */
    "\t.text\n\t.globl _start\n_start:\n"
    "\tleaq __init_array_start(%rip), %rbx\n"
    "\tleaq __init_array_end(%rip), %rbp\n"
    ".Linit_loop:\n\tcmpq %rbp, %rbx\n\tjae .
   Linit_done\n"
    "\tcall *(%rbx)\n\taddq $8, %rbx\n\tjmp .
   Linit_loop\n.Linit_done:\n"
    "\tmovq $1, kcc_host_linux(%rip)\n"   /* a kcc
    we build can itself drive as/ld */
    "\tmovq (%rsp), %rdi\n\tleaq 8(%rsp), %rsi\n\
   tleaq 16(%rsp,%rdi,8), %rdx\n"
    "\tcall main\n\tmovq %rax, %rdi\n\tmovq
   $SYS_exit, %rax\n\tsyscall\n\thlt\n"
    "\t.globl kcc_sys_exit\nkcc_sys_exit:\n\tmovq
   $SYS_exit, %rax\n\tsyscall\n\thlt\n"
    "\t.globl kcc_sys_fork\nkcc_sys_fork:\n\tmovq
   $SYS_fork, %rax\n\tsyscall\n\tret\n"
    "\t.globl kcc_sys_execve\nkcc_sys_execve:\n\
   tmovq $SYS_execve, %rax\n\tsyscall\n\tret\n"
    "\t.globl kcc_sys_wait4\nkcc_sys_wait4:\n\
   tmovq %rcx, %r10\n\tmovq $SYS_wait4, %rax\n\
   tsyscall\n\tret\n"
    "\t.globl kcc_sys_read\nkcc_sys_read:\n\tmovq
   $SYS_read, %rax\n\tsyscall\n\tret\n"
    "\t.globl kcc_sys_write\nkcc_sys_write:\n\
   tmovq $SYS_write, %rax\n\tsyscall\n\tret\n"
    "\t.globl kcc_sys_open\nkcc_sys_open:\n\tmovq
   %rsi, %r11\n\tandq $3, %rsi\n"
    "\ttestq $0x200, %r11\n\tjz 1f\n\torq $0x40, %
   rsi\n"
    "1:\ttestq $0x400, %r11\n\tjz 2f\n\torq $0x200
   , %rsi\n"
    "2:\tmovq $SYS_open, %rax\n\tsyscall\n\tret\n"
    "\t.globl kcc_sys_close\nkcc_sys_close:\n\
   tmovq $SYS_close, %rax\n\tsyscall\n\tret\n"
    "\t.globl kcc_sys_lseek\nkcc_sys_lseek:\n\
   tmovq $SYS_lseek, %rax\n\tsyscall\n\tret\n"
    "\t.globl kcc_sys_unlink\nkcc_sys_unlink:\n\
   tmovq $SYS_unlink, %rax\n\tsyscall\n\tret\n"
    "\t.globl kcc_sys_mmap\nkcc_sys_mmap:\n\ttestq
    $0x1000, %rcx\n\tjz .Lmmap_go\n"
    "\tbtrq $12, %rcx\n\torq $0x20, %rcx\n.
   Lmmap_go:\n\tmovq %rcx, %r10\n"
    "\tmovq $SYS_mmap, %rax\n\tsyscall\n\tret\n"
    "\t.globl kcc_sys_munmap\nkcc_sys_munmap:\n\
   tmovq $SYS_munmap, %rax\n\tsyscall\n\tret\n"
    "\t.section .note.GNU-stack,\"\",@progbits\n";

char *kcc_getenv(const char *name, unsigned long
   namelen);
int kcc_driver_x86_object(const char *out);
int kcc_driver_emit_asm_object(const char *out);
int kcc_driver_spawn(const char *path, char *const
    argv[]);
int kcc_assemble_source(const char *src, const
   char *out);  /* .S/.s → as (defined in main) */
char *kcc_driver_obj_name(const char *src);
unsigned long kcc_driver_I(const char *arg, const
   char *next, unsigned long reg);
unsigned long kcc_driver_D(const char *arg, const
   char *next, unsigned long reg);
```

□

719 · used in 725a · uses _main 621, 629

⟨ *Selfhost: driver compile* 720 ⟩ ≡

```
/* gcc `-include FILE` force-includes, collected
   from the command line (pass 1)
   and applied to every translation unit before
   parsing. */
#define KCC_MAX_FORCE_INC 32
static const char *kcc_force_inc[KCC_MAX_FORCE_INC
   ];
static long kcc_force_inc_n = 0;

/* Force-includes are pushed in reverse so the
   preprocessor reads them back in
   command-line order, ahead of the primary source
   .  Shared by the compile path
   and by preprocess-only (`-E`), which must
   honour `-include` identically. */
void kcc_driver_apply_force_includes(void) {
    for (long k = kcc_force_inc_n - 1; k >= 0; k
   --)
        kcc_pp_force_include(kcc_force_inc[k]);
}

int kcc_driver_compile(const char *src, const char
    *out) {
    {
   /* per-file static-name salt */
        unsigned long h = 5381;
        const char *p = src;
        while (*p) { h = h * 33 + (unsigned char)
   (*p); p++; }
        kcc_tu_salt = h;
    }
    kcc_dep_reset();
    kcc_source_open(src);
    kcc_source_encode();
    kcc_source_splice();
    kcc_lex_init(kcc_src_base, kcc_src_size);
    kcc_pp_init();
```

```
    kcc_driver_apply_force_includes();
    kcc_strtab_init();
    kcc_typedef_init();
    void *tu = kcc_parse();
    kcc_types_init();
    kcc_symtab_init();
    kcc_sema_check(tu);
    if (kcc_diag_had_errors()) { kcc_io_flush();
   return 1; }
    kcc_ir_init();
    kcc_ir_build(tu);
    kcc_opt_run();
    if (kcc_ir_validate()) { kcc_io_flush();
   return 1; }
    if (kcc_target) {
        kcc_x86_tu = tu;
        int r = kcc_driver_x86_object(out);
        if (r == 0) kcc_dep_write(out, src);
   /* -MD: emit the dependency file */
        return r;
    }
    kcc_code_init();
    kcc_linker_init();
    kcc_codegen_init();
    kcc_preregister_globals(tu);
    kcc_codegen_all();
    kcc_codegen_emit_strings();
    if (kcc_emit_asm) return
   kcc_driver_emit_asm_object(out);
    char *side = kcc_getenv("KCC_EMIT_ASM", 12);
    if (side) kcc_as_render(side);
    kcc_dwarf_build();
    kcc_macho_init(out);
    kcc_macho_finalize();
    return 0;
}

/* Build "<out>.s", render to it, then assemble to
    <out>. */
int kcc_driver_emit_asm_object(const char *out) {
    unsigned long len = kcc_strlen(out);
    char *outs = (char *)kcc_arena_alloc((long)(
   len + 3));
    kcc_memcpy(outs, out, len);
    kcc_memcpy(outs + len, STR_DOT_S, 3);
    if (kcc_as_render(outs)) return 1;
    char *av[5];
    av[0] = (char *)STR_AS_ARGV0;
    av[1] = (char *)OPT_O;
    av[2] = (char *)out;
    av[3] = outs;
    av[4] = 0;
    if (kcc_driver_spawn(STR_AS_PATH, av)) return
   1;
    return 0;
}

int kcc_driver_x86_object(const char *out) {
    if (kcc_x86_os && !kcc_host_linux) {  /* macOS
    host, Linux target: can only emit .s */
        kcc_x86_render(out);
        return 0;
    }
    unsigned long len = kcc_strlen(out);
    char *outs = (char *)kcc_arena_alloc((long)(
   len + 3));
    kcc_memcpy(outs, out, len);
    kcc_memcpy(outs + len, STR_DOT_S, 3);
    if (kcc_x86_render(outs)) return 1;
    char *av[7];
    long k = 0;
    av[k++] = (char *)STR_AS_ARGV0;
    if (kcc_host_linux) {                 /* ELF
   assembler: as --64 */
        av[k++] = (char *)STR_AS_64;
    } else {                              /*
   Darwin assembler: as -arch x86_64 */
        av[k++] = (char *)STR_ARCH;
        av[k++] = (char *)STR_X86_64;
    }
    av[k++] = (char *)OPT_O;
    av[k++] = (char *)out;
    av[k++] = outs;
    av[k] = 0;
    if (kcc_driver_spawn(STR_AS_PATH, av)) return
   1;
    return 0;
}
```

□

720 · used in 725a

67.2. Argument Helpers and Object Naming

`driver_I` and `driver_D` each recognise their option in the attached (`-Idir`) or separated (`-I dir`) shape and return how many argument slots it occupies, optionally registering the directory or macro; this lets every argument loop step over them uniformly. `obj_name` rewrites a `.c` suffix to `.o`, or appends `.o`.

⟨ *Selfhost: driver args* 721 ⟩ ≡

```
unsigned long kcc_driver_I(const char *arg, const
   char *next, unsigned long reg) {
    if (kcc_memcmp(arg, "-I", 2) != 0) return 0;
    if (arg[2] == 0) {                    /*
   separated "-I dir" */
        if (reg && next) kcc_pp_add_include_dir(
   next);
        return 2;
    }
    if (reg) kcc_pp_add_include_dir(arg + 2);  /*
   attached "-Idir" */
    return 1;
}

unsigned long kcc_driver_D(const char *arg, const
   char *next, unsigned long reg) {
    if (kcc_memcmp(arg, "-D", 2) != 0) return 0;
    if (arg[2] == 0) {                    /*
   separated "-D NAME" */
        if (reg && next) kcc_pp_add_cmdline_define
   (next);
        return 2;
    }
    if (reg) kcc_pp_add_cmdline_define(arg + 2);
    return 1;
}

char *kcc_driver_obj_name(const char *src) {
    unsigned long len = kcc_strlen(src);
    char *dst = (char *)kcc_arena_alloc((long)(len
    + 3));
    kcc_memcpy(dst, src, len);
    if (len >= 2 && src[len - 2] == '.' && src[len
    - 1] == 'c') {
```

```
        dst[len - 1] = 'o';
        dst[len] = 0;
    } else {
        dst[len] = '.';
        dst[len + 1] = 'o';
        dst[len + 2] = 0;
    }
    return dst;
}
```

□

721 · used in 725a

67.3. Compiling Several Files

`compile_all` handles `-c`: a first pass sets the flags (`-Wall`, `-g`, `--emit-asm`, `--target`), registers include dirs and macros, and finds an explicit `-o` and the source count; a second pass compiles each source, honouring `-o` only for a single input. It exits with the combined status. A source whose name ends in `.S` or `.s` is not a C file: it is routed to `kcc_assemble_source` instead of the front end—a `.S` is preprocessed (the faithful `-E` text) into `<out>.s` and a `.s` is taken as-is, then the system assembler produces the object. This is what lets kcc stand in as `CC=` for the kernel's hand-written entry and library assembly (`head_64.S`, `getuser.S`, …), not just its C.

Both passes scan from `argv[1]` and are *gcc-compatible enough to be a drop-in `CC=`* for a real Kbuild command line, which leads with dozens of `-f`/`-W`/`-m`/`-O` flags and ends with `-c -o out.o in.c`: `-c` is skipped wherever it sits, the two-word options a kernel build uses (`-isystem dir`, `-include file`, `-MF`, `-x`, …) consume their second word, and every other dash-led token is an option kcc ignores. `-include FILE` is not merely skipped: pass 1 collects each such `FILE`, and every translation unit is then force-included with them (through `kcc_pp_force_include`) before its own text—the mechanism Kbuild uses to inject `kconfig.h` and `compiler_types.h`, and thus the `CONFIG_*` macros, ahead of each kernel object. Dependency generation is honoured too: on `-MD`/`-MMD` or the comma-joined `-Wp,-M[M]D,file` the driver records the `.d` path and, after a successful compile, writes the Makefile fragment (`obj: src headers…`) that Kbuild's `fixdep` consumes; the preprocessor supplies the include list. `kcc_dep_out` stays null otherwise, so an ordinary compile writes no `.d`.

⟨ *Selfhost: driver compile-all* 722 ⟩ ≡

```
/* Some gcc options take a separate second word (a
    path or name) that must be
   skipped along with the option so it is not
   mistaken for a source file.  These
   are the ones a Kbuild command line actually
   uses. */
static int kcc_driver_two_arg_opt(const char *a) {
    return kcc_strcmp(a, "-isystem") == 0 ||
   kcc_strcmp(a, "-include") == 0 ||
           kcc_strcmp(a, "-idirafter") == 0 ||
   kcc_strcmp(a, "-imacros") == 0 ||
           kcc_strcmp(a, "-iquote") == 0 ||
   kcc_strcmp(a, "-x") == 0 ||
           kcc_strcmp(a, "-MT") == 0 || kcc_strcmp
   (a, "-MF") == 0 ||
           kcc_strcmp(a, "-MQ") == 0 || kcc_strcmp
   (a, "-aux-info") == 0 ||
           kcc_strcmp(a, "-MD") == 0 || kcc_strcmp
   (a, "-MMD") == 0;
}

/* Recognise the dependency-file options and
   record the .d path.  Kbuild uses the
   comma-joined `-Wp,-MMD,<file>`; the standalone
   `-MD <file>`/`-MMD <file>` take
   the next word.  Returns the number of argv
   slots consumed (0 if not a dep opt). */
static int kcc_str_startswith(const char *s, const
    char *pre) {
    while (*pre) { if (*s++ != *pre++) return 0; }
    return 1;
}
static unsigned long kcc_dep_from_arg(const char *
   a, const char *next) {
    if (kcc_str_startswith(a, "-Wp,-MMD,")) {
   kcc_dep_out = a + 9; return 1; }
    if (kcc_str_startswith(a, "-Wp,-MD,"))  {
   kcc_dep_out = a + 8; return 1; }
    if (kcc_strcmp(a, "-MD") == 0 || kcc_strcmp(a,
    "-MMD") == 0) {
        if (next) { kcc_dep_out = next; return 2;
   }
        return 1;
    }
    return 0;
}

/* Register the preprocessing-relevant flags of a
   gcc-style command line---the
   target, every `-I` directory, every `-D` macro,
    and every `-include`
   force-include---and return the source operand (
   the last non-flag word, or "-"
   for standard input).  Preprocess-only mode (`-E
   `) uses this so that a bare
   `kcc -E` sees exactly the include search path
   and macros the compile path
   would, which a Kbuild `-nostdinc -I... -include
   ...` command line depends on.
   The compile path keeps its own scan because it
   additionally counts sources
   and captures `-o`. */
const char *kcc_driver_register_flags(long argc,
   char **argv) {
    const char *src = 0;
    for (long i = 1; i < argc; ) {
        char *a = argv[i];
        if (kcc_strcmp(a, "-E") == 0) { i++;
   continue; }
        if (kcc_strcmp(a, OPT_TARGET_X86) == 0) {
   kcc_target = 1; i++; continue; }
        if (kcc_strcmp(a, OPT_TARGET_LINUX) == 0)
   { kcc_target = 1; kcc_x86_os = 1; i++; continue
   ; }
        if (kcc_strcmp(a, OPT_O) == 0) { i += 2;
   continue; }   /* skip `-o out` */
        const char *next = (i + 1 < argc) ? argv[i
    + 1] : 0;
        unsigned long n = kcc_driver_I(a, next, 1)
   ;
        if (n) { i += (long)n; continue; }
        n = kcc_driver_D(a, next, 1);
        if (n) { i += (long)n; continue; }
        if (kcc_strcmp(a, "-include") == 0) {
            if (next && kcc_force_inc_n <
   KCC_MAX_FORCE_INC)
                kcc_force_inc[kcc_force_inc_n++] =
```

```
    next;
            i += 2; continue;
        }
        if (kcc_driver_two_arg_opt(a)) { i += 2;
   continue; }
        if (a[0] == '-' && a[1] != 0) { i++;
   continue; }  /* other flag; "-" alone is stdin
   */
        src = a; i++;
           /* last operand wins */
    }
    return src;
}

void kcc_driver_compile_all(long argc, char **argv
   ) {
    const char *explicit_out = 0;
    long src_count = 0;
    /* Pass 1: register -I/-D, capture -o, count
   sources.  Scan from argv[1] and
       treat every other dash-led token as an
   option kcc ignores---the driver is
       gcc-compatible enough for Kbuild, which
   leads with dozens of -f/-W/-m/-O
       flags and ends with `-c -o out.o in.c`. */
    for (long i = 1; i < argc; ) {
        char *a = argv[i];
        if (kcc_strcmp(a, OPT_COMPILE) == 0) { i
   ++; continue; }
        if (kcc_strcmp(a, OPT_WALL) == 0) {
   kcc_warn_all = 1; i++; continue; }
        if (kcc_strcmp(a, OPT_EMITASM) == 0) {
   kcc_emit_asm = 1; i++; continue; }
        if (kcc_strcmp(a, OPT_TARGET_X86) == 0) {
   kcc_target = 1; i++; continue; }
        if (kcc_strcmp(a, OPT_TARGET_LINUX) == 0)
   {
            kcc_target = 1; kcc_x86_os = 1; i++;
   continue;
        }
        if (kcc_strcmp(a, OPT_G) == 0) {
   kcc_dwarf_enabled = 1; i++; continue; }
        if (kcc_strcmp(a, OPT_O) == 0) {
            i++;
            if (i >= argc) break;
            explicit_out = argv[i]; i++; continue;
        }
        const char *next = (i + 1 < argc) ? argv[i
    + 1] : 0;
        unsigned long n = kcc_driver_I(a, next, 1)
   ;
        if (n) { i += (long)n; continue; }
        n = kcc_driver_D(a, next, 1);
        if (n) { i += (long)n; continue; }
        { unsigned long nd = kcc_dep_from_arg(a,
   next); if (nd) { i += (long)nd; continue; } }
        if (kcc_strcmp(a, "-include") == 0) {
    /* gcc -include FILE: force-include it */
            if (next && kcc_force_inc_n <
   KCC_MAX_FORCE_INC)
                kcc_force_inc[kcc_force_inc_n++] =
    next;
            i += 2; continue;
        }
        if (kcc_driver_two_arg_opt(a)) { i += 2;
   continue; }
        if (a[0] == '-') { i++; continue; }
   /* any other flag: ignore */
        src_count++; i++;
    }
    if (src_count == 0) { kcc_io_err(ERR_NOINPUT);
    kcc_sys_exit(1); }

    long err = 0;
    for (long i = 1; i < argc; ) {
        char *a = argv[i];
        if (kcc_strcmp(a, OPT_O) == 0) { i += 2;
   continue; }
        unsigned long n = kcc_driver_I(a, 0, 0);
        if (n) { i += (long)n; continue; }
        n = kcc_driver_D(a, 0, 0);
        if (n) { i += (long)n; continue; }
        if (kcc_driver_two_arg_opt(a)) { i += 2;
   continue; }
        if (a[0] == '-') { i++; continue; }
   /* every option: handled above */
        const char *out;
        if (src_count == 1 && explicit_out) out =
   explicit_out;
        else out = kcc_driver_obj_name(a);
        /* An assembly input (`.S'/`.s') is
   preprocessed-and-assembled, not run
            through the C front end---this is what
   lets kcc build the kernel's
            hand-written entry/lib assembly (
   head_64.S, getuser.S, ...). */
        unsigned long la = kcc_strlen(a);
        if (la >= 2 && a[la - 2] == '.' && (a[la -
    1] == 'S' || a[la - 1] == 's'))
            err |= kcc_assemble_source(a, out);
        else
            err |= kcc_driver_compile(a, out);
        i++;
    }
    kcc_io_flush();
    kcc_sys_exit((int)err);
}
```

□

722 · used in 725a

67.4. Environment, SDK, and Spawning

getenv scans the captured environment; dir_exists probes a path by opening it; sdkroot finds the macOS SDK (from $SDKROOT or the two standard locations) for ld -syslibroot; and spawn forks, execs the child, and returns its exit status.

⟨ *Selfhost: driver spawn* 723 ⟩ ≡

```
char *kcc_getenv(const char *name, unsigned long
   namelen) {
    char **envp = (char **)kcc_envp;
    if (envp == 0) return 0;
    for (; *envp; envp++) {
        char *e = *envp;
        if (kcc_memcmp(e, name, namelen) == 0 && e
   [namelen] == '=')
            return e + namelen + 1;
    }
    return 0;
}

int kcc_dir_exists(const char *path) {
    long fd = kcc_sys_open(path, 0, 0);
    if (fd < 0) return 0;
    kcc_sys_close(fd);
    return 1;
}
```

```
char *kcc_driver_sdkroot(void) {
    char *p = kcc_getenv(STR_SDKROOT_ENV, 7);
    if (p && kcc_dir_exists(p)) return p;
    if (kcc_dir_exists(STR_SDK_CLT)) return (char
   *)STR_SDK_CLT;
    if (kcc_dir_exists(STR_SDK_XCODE)) return (
   char *)STR_SDK_XCODE;
    return (char *)STR_SDK_CLT;
}

int kcc_driver_spawn(const char *path, char *const
    argv[]) {
    long pid = kcc_sys_fork();
    if (pid == 0) {
        kcc_sys_execve(path, argv, (char *const *)
   kcc_envp);
        kcc_sys_exit(127);
    }
    if (pid < 0) return -1;
    int status = 0;
    kcc_sys_wait4(pid, &status, 0, 0);
    return (status >> 8) & 0xff;
}
```

□

723 · used in 725a

67.5. Automatic Linking

`link_all` is the no-`-c` path: a first pass sets the flags—and, like `compile_all`, collects the `-include` force-includes and consumes the two-word options (`-isystem`, `-MF`, `-x`, …) and the `-Wp,-M[M]D` dependency flag so they reach the compile step rather than leaking to the linker. It then scans the arguments building the linker command line—compiling each `.c` to a temporary object (tracked for removal), forwarding an unknown `-flag` verbatim, and handing any other file (a `.o`, an archive) straight to `ld`. Collecting `-include` here matters because Kbuild compiles its generated `.s` prepare-stage files with `-S` (no `-c`), which routes through this path. On a Darwin host it appends the system link line (`-o out -lSystem -syslibroot <sdk> -arch <arch> -e _main`). On a Linux host (`kcc_host_linux`, set by `os_linux`'s `_start`) it instead emits and assembles the embedded runtime to a temporary object and links statically with no libc (`-static -nostdlib -o out <objs> kcc_rt.o -e _start`); this is what lets kcc produce a running executable from source in a single command on Linux. It then runs `ld`, removes the temporary objects, and exits with the linker's status.

⟨ *Selfhost: driver link* 724 ⟩ ≡

```
static int kcc_driver_write_runtime(const char *
   path) {
    long fd = kcc_sys_open(path, 0x601, 0x1a4);
   /* O_WRONLY|O_CREAT|O_TRUNC, 0644 */
    if (fd < 0) return 1;
    long n = (long)kcc_strlen(STR_RUNTIME);
    int ok = (kcc_sys_write(fd, STR_RUNTIME, n) ==
    n);
    kcc_sys_close(fd);
    return ok ? 0 : 1;
}
void kcc_driver_link_all(long argc, char **argv) {
    /* Pass 1: honour flags wherever they appear.
*/
    for (long i = 1; i < argc; ) {
        char *a = argv[i];
        if (kcc_strcmp(a, OPT_WALL) == 0) {
kcc_warn_all = 1; i++; continue; }
        if (kcc_strcmp(a, OPT_EMITASM) == 0) {
kcc_emit_asm = 1; i++; continue; }
        if (kcc_strcmp(a, OPT_TARGET_X86) == 0) {
kcc_target = 1; i++; continue; }
        if (kcc_strcmp(a, OPT_TARGET_LINUX) == 0)
{ kcc_target = 1; kcc_x86_os = 1; i++; continue
; }
        if (kcc_strcmp(a, OPT_G) == 0) {
kcc_dwarf_enabled = 1; i++; continue; }
        const char *next = (i + 1 < argc) ? argv[i
    + 1] : 0;
        unsigned long n = kcc_driver_I(a, next, 1)
;
        if (n) { i += (long)n; continue; }
        n = kcc_driver_D(a, next, 1);
        if (n) { i += (long)n; continue; }
        { unsigned long nd = kcc_dep_from_arg(a,
next); if (nd) { i += (long)nd; continue; } }
        if (kcc_strcmp(a, "-include") == 0) {
    /* gcc -include FILE: force-include it */
            if (next && kcc_force_inc_n <
KCC_MAX_FORCE_INC)
                kcc_force_inc[kcc_force_inc_n++] =
    next;
            i += 2; continue;
        }
        if (kcc_driver_two_arg_opt(a)) { i += 2;
continue; }
        i++;
    }

    char **larg = (char **)kcc_arena_alloc((long)
((argc + 12) * 8));
    char **tmpobj = (char **)kcc_arena_alloc((long
)((argc + 1) * 8));
    long tmpcount = 0;

    larg[0] = (char *)STR_LD_ARGV0;
    long k = 1;                                 /* next
    larg slot */
    const char *out = 0;
    long err = 0;
    long inputs = 0;

    for (long i = 1; i < argc; ) {
        char *a = argv[i];
        if (kcc_strcmp(a, OPT_O) == 0) {
            i++;
            if (i >= argc) break;
            out = argv[i]; i++; continue;
        }
        if (kcc_strcmp(a, OPT_WALL) == 0 ||
kcc_strcmp(a, OPT_G) == 0 ||
            kcc_strcmp(a, OPT_EMITASM) == 0 ||
            kcc_strcmp(a, OPT_TARGET_X86) == 0 ||
            kcc_strcmp(a, OPT_TARGET_LINUX) == 0)
{ i++; continue; }
        unsigned long n = kcc_driver_I(a, 0, 0);
        if (n) { i += (long)n; continue; }
        n = kcc_driver_D(a, 0, 0);
        if (n) { i += (long)n; continue; }
        { unsigned long nd = kcc_dep_from_arg(a,
0); if (nd) { i += (long)nd; continue; } }
        if (kcc_strcmp(a, "-include") == 0) { i +=
    2; continue; }  /* consumed in pass 1 */
```

```
    if (kcc_driver_two_arg_opt(a)) { i += 2;
continue; }
    /* Classify a non-option argument. */
    if (a[0] == '-') {                  /*
unknown linker flag: forward */
        larg[k++] = a; i++; continue;
    }
    unsigned long al = kcc_strlen(a);
    if (al >= 2 && a[al - 2] == '.' && a[al -
1] == 'c') {
        char *obj = kcc_driver_obj_name(a);
        larg[k++] = obj;
        tmpobj[tmpcount++] = obj;
        err |= kcc_driver_compile(a, obj);
        inputs++; i++;
    } else {                            /* .o /
 .a / other: linker input */
        larg[k++] = a;
        inputs++; i++;
    }
 }

 if (err) { kcc_io_flush(); kcc_sys_exit(1); }
 if (inputs == 0) { kcc_io_err(ERR_NOINPUT);
kcc_sys_exit(1); }

 if (out == 0) out = STR_AOUT;
 if (kcc_x86_os && kcc_host_linux) {   /* Linux
 host: static, no libc, embedded runtime */
    if (kcc_driver_write_runtime(STR_RT_S)) {
kcc_io_flush(); kcc_sys_exit(1); }
    char *rav[6];
    rav[0] = (char *)STR_AS_ARGV0; rav[1] = (
char *)STR_AS_64;
    rav[2] = (char *)OPT_O; rav[3] = (char *)
STR_RT_O; rav[4] = (char *)STR_RT_S; rav[5] =
0;
    if (kcc_driver_spawn(STR_AS_PATH, rav)) {
kcc_io_flush(); kcc_sys_exit(1); }
    larg[k++] = (char *)STR_RT_O;     /*
runtime object as a link input */
    larg[k++] = (char *)OPT_O;
    larg[k++] = (char *)out;
    larg[k++] = (char *)STR_STATIC;
    larg[k++] = (char *)STR_NOSTDLIB;
    larg[k++] = (char *)STR_DASH_E;
    larg[k++] = (char *)STR_USTART;
 } else {                               /*
Darwin host: link against libSystem */
    larg[k++] = (char *)OPT_O;
    larg[k++] = (char *)out;
    larg[k++] = (char *)STR_LSYSTEM;
    larg[k++] = (char *)STR_SYSLIBROOT;
    larg[k++] = kcc_driver_sdkroot();
    larg[k++] = (char *)STR_ARCH;
    larg[k++] = kcc_target ? (char *)
STR_X86_64 : (char *)STR_ARM64;
    larg[k++] = (char *)STR_DASH_E;
    larg[k++] = (char *)STR_UMAIN;
 }
 larg[k] = 0;

 kcc_io_flush();
 int status = kcc_driver_spawn(STR_LD_PATH,
larg);

 if (kcc_x86_os && kcc_host_linux) {
kcc_sys_unlink(STR_RT_S); kcc_sys_unlink(
STR_RT_O); }
  for (long i = 0; i < tmpcount; i++)
   kcc_sys_unlink(tmpobj[i]);

  kcc_io_flush();
  kcc_sys_exit(status);
}
```

□

724 · used in 725a

67.6. Assembling the Translation Unit

⟨ *Selfhost: driver* 725a ⟩ ≡

```
⟨ Selfhost: driver preamble 719 ⟩
⟨ Selfhost: driver compile 720 ⟩
⟨ Selfhost: driver args 721 ⟩
⟨ Selfhost: driver compile-all 722 ⟩
⟨ Selfhost: driver spawn 723 ⟩
⟨ Selfhost: driver link 724 ⟩
```

□

725a · used in 35p

67.7. Verification

`make selfhost-driver` runs the drop-in loop through the code-generation suite (whose harness drives `kcc_driver_compile_all`) and the end-to-end suite (which drives `kcc_driver_link_all` to build and run real executables), so both the compile and the link orchestration are verified.

⟨ *Selfhost: driver verify* 725b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-driver - compile kcc's driver in C
    with kcc, relink, run the
# codegen and end-to-end suites.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/driver.c with
    kcc"
$BASE/kcc -c "$SH/driver.c" -o "$SH/driver_c.o" >/
   dev/null

echo "[2/3] linking $BASE/kcc_selfhost_driver (
   driver.o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "driver" ]; then OBJ="$OBJ $SH/
   driver_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
   fi
done
ld -o $BASE/kcc_selfhost_driver $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64
```

```
echo "[3/3] running codegen + e2e with the self-
   hosted driver"
cp $BASE/kcc "$SH/kcc_asm_backup_driver"
cp $BASE/kcc_selfhost_driver $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh; s1=$?
bash $BASE/tests/e2e.sh;          s2=$?
cp "$SH/kcc_asm_backup_driver" $BASE/kcc
[ $s1 -eq 0 ] && [ $s2 -eq 0 ]
```

□

725b · used in 36a · uses _main 621, 629

68. Self-Hosting: The Declaration Parser in C

The declaration parser of Chapter 21 is the last and most intricate slice of the front end to port, and completing it puts the *entire* parse—spine, expressions, statements, and now declarations—through kcc-compiled C. It is where C's syntax is hardest: the specifier bitmask that folds `unsigned long long int` into one type, the inside-out declarator that turns `int (*fp)(int)` into a pointer to a function, struct/union member lists with bit-fields and anonymous members, enums, abstract declarators for casts and `sizeof`, designated initializers, and the GNU attribute and `typeof` extensions real headers depend on. The port follows the assembly node-build for node-build so every AST it produces is identical.

68.1. Nodes, Tokens, and Specifier Bits

The port declares the 64-byte node, owns the two layout-attribute globals the struct parser reads (`packed` and `aligned`), and declares the token and node-tag constants along with the specifier bits (`void` 1, `char` 2, … `unsigned` 512, …) and qualifier bits. `decl_specs` returns two values—the type node and the storage class—so it is declared as a 16-byte struct the ABI hands back in the first two registers.

⟨ *Selfhost: declarations preamble* 727 ⟩ ≡

```
struct Node {
    unsigned long tag;
    unsigned long line;
    unsigned long col;
    unsigned long f0;
    unsigned long f1;
    unsigned long f2;
    unsigned long f3;
    unsigned long f4;
};

struct DeclSpecs { void *type; unsigned long
   storage; };

extern unsigned long kcc_tok_type;
extern const char *kcc_tok_start;
extern unsigned long kcc_tok_len;
extern unsigned long kcc_tok_line;
extern unsigned long kcc_tok_col;

void *kcc_ast_alloc(unsigned long tag);
void kcc_parse_expect(unsigned long tok_type);
void kcc_pp_next(void);
void *kcc_parse_expr(void);
void *kcc_parse_assign_expr(void);
void *kcc_parse_cond_expr(void);
void *kcc_parse_compound(void);
void kcc_parse_sync(void);
void kcc_diag_error(unsigned long line, unsigned
   long col, const char *msg);
int kcc_memcmp(const void *a, const void *b,
   unsigned long n);
unsigned long kcc_strlen(const char *s);
void *kcc_strtab_intern(const void *ptr, unsigned
   long len);
unsigned long kcc_pp_atoi(const char *s, unsigned
   long len);
void kcc_typedef_add(const char *name, unsigned
   long len);
void kcc_typedef_shadow(const char *name, unsigned
    long len);
int kcc_typedef_lookup(const char *name, unsigned
   long len);
void *kcc_ht_init(unsigned long capacity);
unsigned long kcc_ht_get(void *ht, const void *key
   , unsigned long keylen);
void kcc_ht_put(void *ht, const void *key,
   unsigned long keylen, unsigned long value);
void *kcc_arena_alloc(unsigned long n);

unsigned long kcc_attr_packed = 0;
unsigned long kcc_attr_align = 0;
/* Side table of the linkage attributes that
   cannot ride on the fixed-field AST
   node: `__attribute__((weak))`, `((section("name
   ")))`, `((alias("target")))`.
   Keyed by symbol name; consumed by the code
   emitter. */
struct AttrRec { unsigned long weak; const char *
   section; const char *alias; };
unsigned long kcc_attr_weak = 0;
const char *kcc_attr_section = 0;
const char *kcc_attr_alias = 0;
void *kcc_attr_ht = 0;
/* Set when an `__attribute__((always_inline))` (
   kernel `__always_inline`) is
   seen; the function-definition path hands the
   flagged function to the inliner
   registry, then clears it.  Accumulated like the
    linkage attributes, not reset
   per skip_attributes call. */
unsigned long kcc_attr_always_inline = 0;
void kcc_register_always_inline(const char *name,
   void *fndecl);
/* Name of the function whose body is currently
   being parsed, so that a primary
   expression `__func__` (\stdref{6.4.2.2}) can
   materialise it as a string
   literal.  Null outside any function body. */
const char *kcc_cur_func_name = 0;
void kcc_attr_record(const char *name) {
    if (!kcc_attr_ht) kcc_attr_ht = kcc_ht_init
   (64);
    struct AttrRec *r = (struct AttrRec *)
   kcc_arena_alloc(sizeof(struct AttrRec));
    r->weak = kcc_attr_weak; r->section =
   kcc_attr_section; r->alias = kcc_attr_alias;
    kcc_ht_put(kcc_attr_ht, name, kcc_strlen(name)
   , (unsigned long)r);
}

enum {
    TOK_EOF = 0, TOK_IDENT = 1, TOK_NUMBER = 2,
   TOK_STRING_LIT = 4, TOK_AUTO = 10, TOK_CHAR =
   13,
    TOK_CONST = 14, TOK_DOUBLE = 18, TOK_ENUM =
   20, TOK_EXTERN = 21,
    TOK_FLOAT = 22, TOK_INLINE = 26, TOK_INT = 27,
    TOK_LONG = 28,
    TOK_REGISTER = 29, TOK_RESTRICT = 30,
   TOK_SHORT = 32, TOK_SIGNED = 33,
    TOK_STATIC = 35, TOK_STRUCT = 36, TOK_TYPEDEF
   = 38, TOK_UNION = 39,
```

```
    TOK_UNSIGNED = 40, TOK_VOID = 41, TOK_VOLATILE
    = 42,
    TOK__ALIGNAS = 44, TOK__ATOMIC = 46, TOK__BOOL
    = 47, TOK__COMPLEX = 48,
    TOK__IMAGINARY = 50, TOK__NORETURN = 51,
   TOK__STATIC_ASSERT = 52,
    TOK__THREAD_LOCAL = 53, TOK_ATTRIBUTE = 54,
   TOK_TYPEOF = 55, TOK_ASM = 56,
    TOK_EXTENSION = 57, TOK_INT128 = 58,
    TOK_LPAREN = 60, TOK_RPAREN = 61, TOK_LBRACE =
    62, TOK_RBRACE = 63,
    TOK_LBRACKET = 64, TOK_RBRACKET = 65, TOK_SEMI
    = 66, TOK_COMMA = 67,
    TOK_DOT = 68, TOK_STAR = 73, TOK_PLUS = 74,
   TOK_MINUS = 75, TOK_SHL = 80,
    TOK_QUESTION = 92, TOK_COLON = 93,
    TOK_ASSIGN = 94, TOK_ELLIPSIS = 107
};

enum {
    AST_TAG = 0, AST_VAR_DECL = 10, AST_FUNC_DECL
   = 11, AST_PARAM_DECL = 12,
    AST_TYPEDEF_DECL = 13, AST_STRUCT_DECL = 14,
   AST_UNION_DECL = 15,
    AST_ENUM_DECL = 16, AST_ENUMERATOR = 17,
   AST_FIELD_DECL = 18,
    AST_STATIC_ASSERT = 19, AST_TYPE_BUILTIN = 20,
    AST_TYPE_NAME = 21,
    AST_TYPE_PTR = 22, AST_TYPE_ARRAY = 23,
   AST_TYPE_FUNC = 24,
    AST_TYPE_STRUCT = 25, AST_TYPE_UNION = 26,
   AST_TYPE_ENUM = 27,
    AST_TYPE_QUALS = 28, AST_TYPE_TYPEOF = 29,
   AST_INT_LIT = 51,
    AST_CHAR_LIT = 53, AST_INIT_LIST = 65,
   AST_DESIGNATOR = 66,
    AST_PAREN_EXPR = 68
};

/* Forward declarations. */
void *kcc_parse_decl(void);
void kcc_parse_skip_attributes(void);
void *kcc_parse_typeof(void);
struct DeclSpecs kcc_parse_decl_specs(void);
void *kcc_parse_declarator(void *base);
void *kcc_parse_direct_declarator(void *type);
void *kcc_parse_param_decl(void);
void *kcc_parse_abstract_declarator(void *base);
void *kcc_parse_struct_or_union(void);
void *kcc_parse_enum(void);
void *kcc_parse_type_name(void);
unsigned long kcc_parse_is_type_token(void);
void *kcc_parse_initializer(void);
void *kcc_parse_static_assert(void);
static void *abst_splice(void *chain, void *
   old_base, void *new_type);
static void *parse_designated_element(void);

#define F0(n) (((struct Node *)(n))->f0)
#define F1(n) (((struct Node *)(n))->f1)
#define F2(n) (((struct Node *)(n))->f2)
#define F3(n) (((struct Node *)(n))->f3)
#define F4(n) (((struct Node *)(n))->f4)
#define TAG(n) (((struct Node *)(n))->tag)
```

□

727 · used in 738a

68.2. Attributes and `typeof`

`skip_attributes` consumes GNU `__attribute__((...))` and `__extension__` runs, discarding all but the ones that change layout or linkage: it records `packed` and `aligned(N)` into the per-declarator globals (reset at entry), and `weak`, `section("name")`, and `alias("target")` into globals that *accumulate* across a declaration's leading and trailing attribute lists. When the top-level parser finishes a declaration and knows its name, `kcc_attr_record` copies those three into the `kcc_attr_ht` side table (keyed by symbol name), then clears them—the AST node has no free field to carry linkage on, and the code emitter reads the table back when it lays each symbol down. The name inside a `section`/`alias` may be written as several adjacent string literals, which §6.4.5 concatenates into one; the parser folds every consecutive literal rather than keeping only the first, so the kernel's `DEFINE_PER_CPU_FIRST`—whose section is `".data..percpu" "..first"`—places `fixed_percpu_data` in `.data..percpu..first` and not the base `.data..percpu`. `typeof` wraps its parenthesised type-name or expression in a `TYPE_TYPEOF` node.

⟨ *Selfhost: declarations attrs* 728 ⟩ ≡

```
void kcc_parse_skip_attributes(void) {
    /* packed/align are per-declarator (reset each
    call); weak/section/alias are
       linkage properties that accumulate across a
    declaration's leading and
       trailing attribute lists and are cleared
   once kcc_attr_record consumes
       them. */
    kcc_attr_packed = 0;
    kcc_attr_align = 0;
    int want_align = 0, want_section = 0,
   want_alias = 0;
    /* The `aligned(...)' argument is a constant
   expression, not just a literal:
       the kernel's ubiquitous `
   ____cacheline_aligned' expands to
       `__aligned__((1 << (6)))'.  Reading only
   the first number token gave 1 (or
       0), so every cacheline-aligned member lost
   its padding --- e.g. the empty
       `struct sched_statistics' aligned(64)
   inside `task_struct' collapsed to
       align 1, shifting every field after it by
   32 bytes and crashing code that
       reads `current->...' at the wrong offset.
   Evaluate the argument left to
       right over `<< + - *' and nested parens (
   kernel forms are that simple; any
       macro operands are already numbers by this
   phase). */
    unsigned long al_val = 0, al_have = 0, al_op =
    0; long al_depth = -1;
    for (;;) {
        unsigned long t = kcc_tok_type;
        if (t == TOK_EXTENSION) { kcc_pp_next();
   continue; }
        if (t != TOK_ATTRIBUTE) return;
        kcc_pp_next();                          /*
   consume __attribute__ */
        if (kcc_tok_type != TOK_LPAREN) return;
        long depth = 0;
        for (;;) {
            unsigned long tt = kcc_tok_type;
            if (tt == TOK_EOF) return;
            if (tt == TOK_LPAREN) depth++;
            else if (tt == TOK_RPAREN) {
                depth--;
```

```
            if (want_align && depth ==
al_depth) {   /* aligned(expr) closed */
                want_align = 0; kcc_attr_align
 = al_val;
            }
        }
        else if (tt == TOK_IDENT) {
            const char *txt = kcc_tok_start;
            unsigned long l = kcc_tok_len;
            if (l == 6 && kcc_memcmp(txt, "
packed", 6) == 0) kcc_attr_packed = 1;
            else if (l == 10 && kcc_memcmp(txt
, "__packed__", 10) == 0) kcc_attr_packed = 1;
            else if ((l == 7 && kcc_memcmp(txt
, "aligned", 7) == 0) ||
                     (l == 11 && kcc_memcmp(
txt, "__aligned__", 11) == 0)) {
                want_align = 1; al_have = 0;
al_val = 0; al_op = 0; al_depth = depth;
            }
            else if ((l == 4 && kcc_memcmp(txt
, "weak", 4) == 0) ||
                     (l == 8 && kcc_memcmp(txt
, "__weak__", 8) == 0)) kcc_attr_weak = 1;
            else if ((l == 7 && kcc_memcmp(txt
, "section", 7) == 0) ||
                     (l == 11 && kcc_memcmp(
txt, "__section__", 11) == 0)) want_section =
1;
            else if ((l == 5 && kcc_memcmp(txt
, "alias", 5) == 0) ||
                     (l == 9 && kcc_memcmp(txt
, "__alias__", 9) == 0)) want_alias = 1;
            else if ((l == 13 && kcc_memcmp(
txt, "always_inline", 13) == 0) ||
                     (l == 17 && kcc_memcmp(
txt, "__always_inline__", 17) == 0))
                kcc_attr_always_inline = 1;
        } else if (tt == TOK_NUMBER &&
want_align) {
            unsigned long n = kcc_pp_atoi(
kcc_tok_start, kcc_tok_len);
            if (!al_have) { al_val = n;
al_have = 1; }
            else if (al_op == TOK_SHL) al_val
<<= n;
            else if (al_op == TOK_PLUS) al_val
 += n;
            else if (al_op == TOK_MINUS)
al_val -= n;
            else if (al_op == TOK_STAR) al_val
 *= n;
            al_op = 0;
        } else if (want_align && (tt ==
TOK_SHL || tt == TOK_PLUS ||
                                  tt ==
TOK_MINUS || tt == TOK_STAR)) {
            al_op = tt;
        } else if (tt == TOK_STRING_LIT && (
want_section || want_alias)) {
            /* A section or alias name may be
several adjacent string literals
               that translation phase~6
concatenates (\stdref{6.4.5}): the
               kernel spells
DEFINE_PER_CPU_FIRST's section as
               `".data..percpu" "..first"'.
Fold every consecutive literal
               (each stripped of its quotes)
into one interned name; taking
               only the first would drop `..
first' and mis-place
                  fixed_percpu_data out of `.data
..percpu..first', which
                  vmlinux.lds requires at offset
~0 of the per-cpu area. */
            unsigned long cap = 64, len = 0;
            char *buf = (char *)
kcc_arena_alloc(cap);
            while (kcc_tok_type ==
TOK_STRING_LIT) {
                unsigned long inner =
kcc_tok_len >= 2 ? kcc_tok_len - 2 : 0;
                if (len + inner >= cap) {
                    unsigned long nc = cap *
2;
                    while (nc <= len + inner)
nc *= 2;
                    char *nb = (char *)
kcc_arena_alloc(nc);
                    kcc_memcpy(nb, buf, len);
                    buf = nb; cap = nc;
                }
                kcc_memcpy(buf + len,
kcc_tok_start + 1, inner);
                len += inner;
                kcc_pp_next();
            }
            const char *s = (const char *)
kcc_strtab_intern(buf, len);
            if (want_section) {
kcc_attr_section = s; want_section = 0; }
            else { kcc_attr_alias = s;
want_alias = 0; }
            continue;
        /* already advanced past the literals */
        }
        kcc_pp_next();
        if (depth == 0) break;
    }
  }
}

void *kcc_parse_typeof(void) {
    kcc_pp_next();                              /*
    consume typeof */
    kcc_parse_expect(TOK_LPAREN);
    void *operand;
    unsigned long is_type;
    if (kcc_parse_is_type_token()) { operand =
    kcc_parse_type_name(); is_type = 1; }
    else { operand = kcc_parse_expr(); is_type =
    0; }
    kcc_parse_expect(TOK_RPAREN);
    struct Node *n = (struct Node *)kcc_ast_alloc(
    AST_TYPE_TYPEOF);
    n->f0 = (unsigned long)operand;
    n->f1 = is_type;
    return n;
}
□
```

728 · used in 738a

68.3. Declaration Specifiers

`decl_specs` loops over storage classes, qualifiers, function specifiers, `_Alignas` (whose parenthesised argument is skipped), builtin type keywords (each ORing its specifier bit, with two `long`s promoting to long long and `__int128` setting bit 12), struct/union/enum/typeof subparsers, typedef names, and interleaved attributes. When

the loop ends it materialises the type: an aggregate or typedef already built, else a `TYPE_BUILTIN` from the bitmask (defaulting to `int`), then wraps it in `TYPE_QUALS` if any qualifiers appeared.

⟨ *Selfhost: declarations specs* 730 ⟩ ≡

```
struct DeclSpecs kcc_parse_decl_specs(void) {
    unsigned long storage = 0, quals = 0, mask =
   0, count = 0;
    void *type = 0;
    for (;;) {
        unsigned long t = kcc_tok_type;
        if (t == TOK_TYPEDEF) { storage = 1;
   kcc_pp_next(); continue; }
        if (t == TOK_EXTERN) { storage = 2;
   kcc_pp_next(); continue; }
        if (t == TOK_STATIC) { storage = 3;
   kcc_pp_next(); continue; }
        if (t == TOK_AUTO) { storage = 4;
   kcc_pp_next(); continue; }
        if (t == TOK_REGISTER) { storage = 5;
   kcc_pp_next(); continue; }
        if (t == TOK_CONST) { quals |= 1;
   kcc_pp_next(); continue; }
        if (t == TOK_VOLATILE) { quals |= 2;
   kcc_pp_next(); continue; }
        if (t == TOK_RESTRICT) { quals |= 4;
   kcc_pp_next(); continue; }
        if (t == TOK__ATOMIC) { quals |= 8;
   kcc_pp_next(); continue; }
        if (t == TOK_INLINE || t == TOK__NORETURN
   || t == TOK__THREAD_LOCAL) {
            kcc_pp_next(); continue;
        }
        if (t == TOK__ALIGNAS) {
            kcc_pp_next();
            kcc_parse_expect(TOK_LPAREN);
            long depth = 1;
            for (;;) {
                unsigned long tt = kcc_tok_type;
                if (tt == TOK_EOF) break;
                if (tt == TOK_LPAREN) depth++;
                else if (tt == TOK_RPAREN) { depth
   --; if (depth == 0) { kcc_pp_next(); break; } }
                kcc_pp_next();
            }
            continue;
        }
        if (t == TOK_VOID) { mask |= 1; count++;
   kcc_pp_next(); continue; }
        if (t == TOK_CHAR) { mask |= 2; count++;
   kcc_pp_next(); continue; }
        if (t == TOK_SHORT) { mask |= 4; count++;
   kcc_pp_next(); continue; }
        if (t == TOK_INT) { mask |= 8; count++;
   kcc_pp_next(); continue; }
        if (t == TOK_LONG) {
            if (mask & 16) mask |= 32; else mask
   |= 16;
            count++; kcc_pp_next(); continue;
        }
        if (t == TOK_FLOAT) { mask |= 64; count++;
    kcc_pp_next(); continue; }
        if (t == TOK_DOUBLE) { mask |= 128; count
   ++; kcc_pp_next(); continue; }
        if (t == TOK_SIGNED) { mask |= 256; count
   ++; kcc_pp_next(); continue; }
        if (t == TOK_UNSIGNED) { mask |= 512;
   count++; kcc_pp_next(); continue; }
        if (t == TOK__BOOL) { mask |= 1024; count
   ++; kcc_pp_next(); continue; }
        if (t == TOK_INT128) { mask |= 4096; count
   ++; kcc_pp_next(); continue; }
        if (t == TOK__COMPLEX || t ==
   TOK__IMAGINARY) {
            kcc_diag_error(kcc_tok_line,
   kcc_tok_col,
                "complex types (_Complex/
   _Imaginary) are not supported");
            mask |= 2048; count++; kcc_pp_next();
   continue;
        }
        if (t == TOK_STRUCT || t == TOK_UNION) {
   type = kcc_parse_struct_or_union(); count++;
   continue; }
        if (t == TOK_ENUM) { type = kcc_parse_enum
   (); count++; continue; }
        if (t == TOK_TYPEOF) { type =
   kcc_parse_typeof(); count++; continue; }
        if (t == TOK_ATTRIBUTE || t ==
   TOK_EXTENSION) { kcc_parse_skip_attributes();
   continue; }
        if (t == TOK_IDENT) {
            if (count) break;
            /* GNU __auto_type: the declared type
   is inferred from the
               initializer (parsed later), i.e. `
   __auto_type x = e` means
               `typeof(e) x = e`.  Leave a
   deferred typeof node (f1==2) that the
               declaration fills with the
   initializer once it is parsed. */
            if (kcc_tok_len == 11 && kcc_memcmp(
   kcc_tok_start, "__auto_type", 11) == 0) {
                kcc_pp_next();
                struct Node *n = (struct Node *)
   kcc_ast_alloc(AST_TYPE_TYPEOF);
                n->f0 = 0; n->f1 = 2;
                type = n; count++;
                continue;
            }
            if (!kcc_typedef_lookup(kcc_tok_start,
    kcc_tok_len)) break;
            void *name = kcc_strtab_intern(
   kcc_tok_start, kcc_tok_len);
            kcc_pp_next();
            struct Node *tn = (struct Node *)
   kcc_ast_alloc(AST_TYPE_NAME);
            tn->f0 = (unsigned long)name;
            type = tn; count++;
            continue;
        }
        break;
    }
    if (type == 0) {
        struct Node *bt = (struct Node *)
   kcc_ast_alloc(AST_TYPE_BUILTIN);
        bt->f0 = count ? mask : 8;              /*
   default int */
        type = bt;
    }
    if (quals) {
        struct Node *q = (struct Node *)
   kcc_ast_alloc(AST_TYPE_QUALS);
        q->f0 = (unsigned long)type;
        q->f1 = quals;
        type = q;
    }
```

```
    struct DeclSpecs r;
    r.type = type;
    r.storage = storage;
    return r;
}
```

□

730 · used in 738a

68.4. Declarators

A declarator wraps the base type in pointer nodes (each with its qualifiers), then hands off to the direct declarator, which reads the name (or a grouped inner declarator) and the array and function suffixes. A GNU `__attribute__`—among the pointer qualifiers (`char * __must_check f`) or just before the name is consumed and discarded, the way the assembly declarator does. Array dimensions nest left-to-right (the first bracket is outermost); a function suffix builds a `TYPE_FUNC` and a `FUNC_DECL`, tagging the variadic flag into bit 0 of the params pointer. A grouped declarator such as `int (*fp)(int)` parses the inner `*fp`, then splices the trailing function or array type in place of the base within the inner declarator's type chain—C's inside-out rule. A parameter is parsed the same way, with one adjustment (§6.7.6.3p8): a parameter whose declarator is itself a function—`int getfrag(void *, int)`, as ip_output's `__ip_append_data` spells its callback—has function type, which decays to a pointer to function. The declarator returns a `FUNC_DECL` whose `f0` is only the *return* type, so `kcc_parse_param_decl` rebuilds the `TYPE_FUNC` from it (return type plus the params-and-variadic word) and lets `kcc_type_param_adjust` turn it into a pointer; without this the parameter would keep just its return type and a later call through it would be rejected as a non-function.

⟨ *Selfhost: declarations declarator* 731 ⟩ ≡

```
void *kcc_parse_declarator(void *base) {
    void *type = base;
    /* A GNU attribute may LEAD the declarator,
   ahead of the pointer stars---a
       calling-convention attribute bound to a
   function pointer, as in
       `(__attribute__((ms_abi)) *handler)(...)' (
   the kernel's
       efi_call_acpi_prm_handler).  The post-star
   skip below only catches an
       attribute between the last `*' and the name
   , so a leading one would strand
       the `*' unconsumed and derail the enclosing
    `)'. */
    while (kcc_tok_type == TOK_ATTRIBUTE)
   kcc_parse_skip_attributes();
    while (kcc_tok_type == TOK_STAR) {
        kcc_pp_next();
        unsigned long q = 0;
        for (;;) {
            unsigned long t = kcc_tok_type;
            if (t == TOK_CONST) { q |= 1;
   kcc_pp_next(); }
            else if (t == TOK_VOLATILE) { q |= 2;
   kcc_pp_next(); }
            else if (t == TOK_RESTRICT) { q |= 4;
   kcc_pp_next(); }
            else if (t == TOK__ATOMIC) { q |= 8;
   kcc_pp_next(); }
            else if (t == TOK_ATTRIBUTE)
   kcc_parse_skip_attributes(); /* pointer attr, e
   .g. `* __must_check` */
            else break;
        }
        struct Node *p = (struct Node *)
   kcc_ast_alloc(AST_TYPE_PTR);
        p->f0 = (unsigned long)type;
        p->f1 = q;
        type = p;
    }
    /* A GNU attribute may precede the direct
   declarator's name (a no-op here
       when absent): `T __attribute__((...)) x`.
   */
    if (kcc_tok_type == TOK_ATTRIBUTE)
   kcc_parse_skip_attributes();
    void *d = kcc_parse_direct_declarator(type);
    /* ...and one may TRAIL the whole declarator,
   before the `='/`,'/`;' or a
       function body: `bool x __read_mostly;' (a
   kernel section attribute) or
       `int f(void) __weak'.  Route it through the
    same side table so linkage
       attributes still bind to this declaration.
   */
    while (kcc_tok_type == TOK_ATTRIBUTE)
   kcc_parse_skip_attributes();
    return d;
}

/* Parse array [ ... ] suffixes into a chain whose
    element is `base`; returns the
 * outermost array (or 0 if none), advancing past
   the brackets. */
static void *parse_array_suffix(void *base) {
    void *innermost = 0, *outermost = 0;
    while (kcc_tok_type == TOK_LBRACKET) {
        kcc_pp_next();
        while (kcc_tok_type == TOK_CONST ||
   kcc_tok_type == TOK_VOLATILE ||
               kcc_tok_type == TOK_RESTRICT ||
   kcc_tok_type == TOK_STATIC)
            kcc_pp_next();
        void *size = 0;
        if (kcc_tok_type == TOK_RBRACKET) {
            /* empty */
        } else if (kcc_tok_type == TOK_STAR) {
            kcc_pp_next();
        } else {
            size = kcc_parse_assign_expr();
        }
        kcc_parse_expect(TOK_RBRACKET);
        struct Node *a = (struct Node *)
   kcc_ast_alloc(AST_TYPE_ARRAY);
        a->f1 = (unsigned long)size;
        if (innermost == 0) { a->f0 = (unsigned
   long)base; innermost = a; outermost = a; }
        else { a->f0 = F0(innermost); F0(innermost
   ) = (unsigned long)a; innermost = a; }
    }
    return outermost;
}

/* Parse a parameter list (after the '(' is
   consumed) into fn->f1, setting the
 * variadic flag in fn->f2. */
static void parse_param_list(struct Node *fn) {
    void *first = 0, *last = 0;
    for (;;) {
        unsigned long t = kcc_tok_type;
```

```
        if (t == TOK_RPAREN) break;
        if (t == TOK_ELLIPSIS) { kcc_pp_next(); fn
    ->f2 = 1; break; }
        void *p = kcc_parse_param_decl();
        if (first == 0) { first = p; last = p; }
        else { F4(last) = (unsigned long)p; last =
     p; }
        if (kcc_tok_type != TOK_COMMA) break;
        kcc_pp_next();
        if (kcc_tok_type == TOK_ELLIPSIS) {
    kcc_pp_next(); fn->f2 = 1; break; }
     }
     fn->f1 = (unsigned long)first;
}
```

□

731 · used in 738a

⟨ *Selfhost: declarations direct* 732 ⟩ ≡

```
void *kcc_parse_direct_declarator(void *type) {
     void *name = 0;
     if (kcc_tok_type == TOK_LPAREN) {
         /* grouped declarator: ( declarator )
    suffix? */
         kcc_pp_next();
         struct Node *inner = (struct Node *)
    kcc_parse_declarator(type);
         kcc_parse_expect(TOK_RPAREN);
         if (kcc_tok_type == TOK_LPAREN) {
             kcc_pp_next();
             struct Node *ft = (struct Node *)
    kcc_ast_alloc(AST_TYPE_FUNC);
             ft->f0 = (unsigned long)type;
             parse_param_list(ft);
             kcc_parse_expect(TOK_RPAREN);
             if (F0(inner) == (unsigned long)type)
    {
                 /* redundant parens around a
    function name: retag as FuncDecl */
                 inner->tag = AST_FUNC_DECL;
                 inner->f0 = (unsigned long)type;
                 inner->f2 = ft->f1 | ft->f2;
             } else {
                 struct Node *w = inner;
                 while (F0(w) != (unsigned long)
    type) w = (struct Node *)F0(w);
                 F0(w) = (unsigned long)ft;
             }
             return inner;
         }
         if (kcc_tok_type == TOK_LBRACKET) {
             void *arr = parse_array_suffix(type);
             if (arr) {
                 if (F0(inner) == (unsigned long)
    type) F0(inner) = (unsigned long)arr;
                 else {
                     struct Node *w = inner;
                     while (F0(w) != (unsigned long
    )type) w = (struct Node *)F0(w);
                     F0(w) = (unsigned long)arr;
                 }
             }
             return inner;
         }
         return inner;
     }
     if (kcc_tok_type == TOK_IDENT) {
         name = kcc_strtab_intern(kcc_tok_start,
    kcc_tok_len);
         kcc_pp_next();
     }
     /* suffixes */
     void *innermost = 0;                        /*
    innermost array (multi-dim) */
     for (;;) {
         if (kcc_tok_type == TOK_LBRACKET) {
             kcc_pp_next();
             while (kcc_tok_type == TOK_CONST ||
    kcc_tok_type == TOK_VOLATILE ||
                    kcc_tok_type == TOK_RESTRICT ||
     kcc_tok_type == TOK_STATIC)
                 kcc_pp_next();
             void *size = 0;
             if (kcc_tok_type == TOK_RBRACKET) { /*
     empty */ }
             else if (kcc_tok_type == TOK_STAR) {
    kcc_pp_next(); }
             else size = kcc_parse_assign_expr();
             kcc_parse_expect(TOK_RBRACKET);
             struct Node *a = (struct Node *)
    kcc_ast_alloc(AST_TYPE_ARRAY);
             a->f1 = (unsigned long)size;
             if (innermost == 0) { a->f0 = (
    unsigned long)type; type = a; innermost = a; }
             else { a->f0 = F0(innermost); F0(
    innermost) = (unsigned long)a; innermost = a; }
             continue;
         }
         if (kcc_tok_type == TOK_LPAREN) {
             innermost = 0;                      /*
    a function suffix ends an array chain */
             kcc_pp_next();
             struct Node *ft = (struct Node *)
    kcc_ast_alloc(AST_TYPE_FUNC);
             ft->f0 = (unsigned long)type;
             if (kcc_tok_type != TOK_RPAREN)
    parse_param_list(ft);
             kcc_parse_expect(TOK_RPAREN);
             struct Node *fd = (struct Node *)
    kcc_ast_alloc(AST_FUNC_DECL);
             fd->f0 = ft->f0;                    /*
    return type */
             fd->f1 = (unsigned long)name;
             fd->f2 = ft->f1 | ft->f2;           /*
    params | variadic */
             return fd;
         }
         break;
     }
     struct Node *vd = (struct Node *)kcc_ast_alloc
    (AST_VAR_DECL);
     vd->f0 = (unsigned long)type;
     vd->f1 = (unsigned long)name;
     return vd;
}

void *kcc_parse_param_decl(void) {
     struct DeclSpecs ds = kcc_parse_decl_specs();
     void *type = ds.type;
     unsigned long t = kcc_tok_type;
     if (t == TOK_RPAREN || t == TOK_COMMA || t ==
    TOK_ELLIPSIS) {
         struct Node *p = (struct Node *)
    kcc_ast_alloc(AST_PARAM_DECL);
         p->f0 = (unsigned long)type;
         return p;
     }
     struct Node *d = (struct Node *)
    kcc_parse_declarator(type);
```

```
    kcc_parse_skip_attributes();
    struct Node *p = (struct Node *)kcc_ast_alloc(
AST_PARAM_DECL);
    if (d->tag == AST_FUNC_DECL) {
        /* A parameter written with function-
declarator syntax---the kernel's
           `int getfrag(void *from, ...)' in
ip_output's __ip_append_data---has
           function type, which \stdref{6.7.6.3}p8
 adjusts to a pointer to
           function.  kcc_parse_declarator hands
back a FuncDecl whose f0 is the
           \emph{return} type and f2 is `params |
variadic-bit'; taking f0 as the
           parameter type would silently keep only
 the return type (so a later
           call `getfrag(...)' fails "called
object is not a function").  Rebuild
           the function type so sema sees TY_FUNC
and kcc_type_param_adjust turns
           it into a function pointer. */
        struct Node *ft = (struct Node *)
kcc_ast_alloc(AST_TYPE_FUNC);
        ft->f0 = F0(d);
    /* return type */
        ft->f1 = F2(d);
    /* params (variadic bit masked on resolve) */
        ft->f2 = F2(d) & 1UL;
    /* variadic flag */
        p->f0 = (unsigned long)ft;
        p->f1 = F1(d);
        return p;
    }
    p->f0 = F0(d);
    p->f1 = F1(d);
    return p;
}
```

□

732 · used in 738a

68.5. Abstract Declarators

An abstract declarator (used in casts and `sizeof(type)`) is a declarator without a name. Besides pointers and array/function suffixes it may group an inner `(*)`, which is handled inside-out with `abst_splice`: the trailing function or array type replaces the base within the inner chain, so `void (*)(void)` becomes a pointer to a function.

⟨ *Selfhost: declarations abstract* 733 ⟩ ≡

```
static void *abst_splice(void *chain, void *
old_base, void *new_type) {
    if (chain == old_base) return new_type;
    struct Node *w = (struct Node *)chain;
    for (;;) {
        void *nx = (void *)w->f0;
        if (nx == old_base) { w->f0 = (unsigned
long)new_type; return chain; }
        if (nx == 0) return chain;
        w = (struct Node *)nx;
    }
}

/* Build a TYPE_FUNC over `base` from a parameter
list (the '(' already consumed
 * for the grouped case; here we parse params then
 ')'). */
static void *build_func_type(void *base) {
    struct Node *ft = (struct Node *)kcc_ast_alloc
(AST_TYPE_FUNC);
    ft->f0 = (unsigned long)base;
    void *first = 0, *last = 0;
    for (;;) {
        unsigned long t = kcc_tok_type;
        if (t == TOK_RPAREN) break;
        if (t == TOK_ELLIPSIS) { kcc_pp_next(); ft
->f2 = 1; break; }
        void *p = kcc_parse_param_decl();
        if (first == 0) { first = p; last = p; }
        else { F4(last) = (unsigned long)p; last =
 p; }
        if (kcc_tok_type != TOK_COMMA) break;
        kcc_pp_next();
    }
    ft->f1 = (unsigned long)first;
    kcc_parse_expect(TOK_RPAREN);
    return ft;
}

void *kcc_parse_abstract_declarator(void *base) {
    void *type = base;
    if (kcc_tok_type == TOK_LPAREN) {
        kcc_pp_next();
        if (kcc_tok_type == TOK_STAR ||
kcc_tok_type == TOK_LPAREN) {
            void *inner =
kcc_parse_abstract_declarator(type);
            kcc_parse_expect(TOK_RPAREN);
            if (kcc_tok_type == TOK_LPAREN) {
                kcc_pp_next();
                void *ft = build_func_type(type);
                return abst_splice(inner, type, ft
);
            }
            if (kcc_tok_type == TOK_LBRACKET) {
                kcc_pp_next();
                void *size = 0;
                if (kcc_tok_type != TOK_RBRACKET)
size = kcc_parse_assign_expr();
                kcc_parse_expect(TOK_RBRACKET);
                struct Node *a = (struct Node *)
kcc_ast_alloc(AST_TYPE_ARRAY);
                a->f0 = (unsigned long)type;
                a->f1 = (unsigned long)size;
                return abst_splice(inner, type, a)
;
            }
            return inner;
        }
        /* '(' opened a parameter list over the
base (bare function type). */
        return build_func_type(type);
    }
    while (kcc_tok_type == TOK_STAR) {
        kcc_pp_next();
        unsigned long q = 0;
        for (;;) {
            unsigned long t = kcc_tok_type;
            if (t == TOK_CONST) { q |= 1;
kcc_pp_next(); }
            else if (t == TOK_VOLATILE) { q |= 2;
kcc_pp_next(); }
            else if (t == TOK_RESTRICT) { q |= 4;
kcc_pp_next(); }
            else if (t == TOK__ATOMIC) { q |= 8;
kcc_pp_next(); }
            else break;
        }
```

```
        struct Node *p = (struct Node *)
    kcc_ast_alloc(AST_TYPE_PTR);
        p->f0 = (unsigned long)type;
        p->f1 = q;
        type = p;
     }
     /* trailing array / function suffixes */
     for (;;) {
         if (kcc_tok_type == TOK_LBRACKET) {
             kcc_pp_next();
             void *size = 0;
             if (kcc_tok_type != TOK_RBRACKET) size
     = kcc_parse_assign_expr();
             kcc_parse_expect(TOK_RBRACKET);
             struct Node *a = (struct Node *)
    kcc_ast_alloc(AST_TYPE_ARRAY);
             a->f0 = (unsigned long)type;
             a->f1 = (unsigned long)size;
             type = a;
             continue;
         }
         if (kcc_tok_type == TOK_LPAREN) {
             kcc_pp_next();
             type = build_func_type(type);
             continue;
         }
         break;
     }
     return type;
}
```

□

733 · used in 738a

68.6. Struct, Union, and Enum

Struct and union parsing consume the keyword, read an optional tag and an optional brace-enclosed member list (each member a specifier-plus-declarator, with optional bit-field width and anonymous members), then build the aggregate declaration and wrap it in a TYPE_STRUCT/TYPE_UNION, folding the layout attributes (packed, aligned) from the three sites around it into f3. Enum parsing reads its enumerators with optional values.

⟨ *Selfhost: declarations aggregate* 734 ⟩ ≡

```
void *kcc_parse_struct_or_union(void) {
    unsigned long packed = 0, align = 0;
    unsigned long decl_tag = (kcc_tok_type ==
    TOK_STRUCT) ? AST_STRUCT_DECL

            : AST_UNION_DECL;
     kcc_pp_next();                                /*
    consume struct/union */
     kcc_parse_skip_attributes();
     if (kcc_attr_packed) packed = 1;
     if (kcc_attr_align > align) align =
    kcc_attr_align;

     void *name = 0;
     if (kcc_tok_type == TOK_IDENT) {
         name = kcc_strtab_intern(kcc_tok_start,
    kcc_tok_len);
         kcc_pp_next();
         kcc_parse_skip_attributes();
         if (kcc_attr_packed) packed = 1;
         if (kcc_attr_align > align) align =
    kcc_attr_align;
     }

     if (kcc_tok_type != TOK_LBRACE) {
         struct Node *d = (struct Node *)
    kcc_ast_alloc(decl_tag);
         d->f0 = (unsigned long)name;
         struct Node *w = (struct Node *)
    kcc_ast_alloc(
             decl_tag == AST_STRUCT_DECL ?
    AST_TYPE_STRUCT : AST_TYPE_UNION);
         w->f0 = (unsigned long)d;
         return w;
     }

     kcc_pp_next();                                /*
    consume '{' */
     void *first = 0, *last = 0;
     while (kcc_tok_type != TOK_RBRACE &&
    kcc_tok_type != TOK_EOF) {
         struct DeclSpecs ds = kcc_parse_decl_specs
    ();
         void *ftype = ds.type;
         if (kcc_tok_type == TOK_SEMI) {           /*
    anonymous member */
             struct Node *fd = (struct Node *)
    kcc_ast_alloc(AST_FIELD_DECL);
             fd->f0 = (unsigned long)ftype;
             if (first == 0) { first = fd; last =
    fd; }
             else { F4(last) = (unsigned long)fd;
    last = fd; }
             kcc_parse_expect(TOK_SEMI);
             continue;
         }
         for (;;) {
             struct Node *fd = (struct Node *)
    kcc_parse_declarator(ftype);
             if (fd == 0) break;
             fd->tag = AST_FIELD_DECL;
             fd->f2 = 0;
             fd->f4 = 0;
             kcc_parse_skip_attributes();
             if (kcc_tok_type == TOK_COLON) {
                 kcc_pp_next();
                 fd->f2 = (unsigned long)
    kcc_parse_cond_expr();
             }
             if (first == 0) { first = fd; last =
    fd; }
             else { F4(last) = (unsigned long)fd;
    last = fd; }
             if (kcc_tok_type != TOK_COMMA) break;
             kcc_pp_next();
         }
         kcc_parse_expect(TOK_SEMI);
     }
     kcc_parse_expect(TOK_RBRACE);
     kcc_parse_skip_attributes();
     if (kcc_attr_packed) packed = 1;
     if (kcc_attr_align > align) align =
    kcc_attr_align;

     struct Node *d = (struct Node *)kcc_ast_alloc(
    decl_tag);
     d->f0 = (unsigned long)name;
     d->f1 = (unsigned long)first;
     d->f3 = packed | (align << 1);
     struct Node *w = (struct Node *)kcc_ast_alloc(
         decl_tag == AST_STRUCT_DECL ?
    AST_TYPE_STRUCT : AST_TYPE_UNION);
     w->f0 = (unsigned long)d;
```

```
    return w;
}

void *kcc_parse_enum(void) {
    kcc_pp_next();                                  /*
   consume enum */
    kcc_parse_skip_attributes();
    void *name = 0;
    if (kcc_tok_type == TOK_IDENT) {
        name = kcc_strtab_intern(kcc_tok_start,
   kcc_tok_len);
        kcc_pp_next();
    }
    if (kcc_tok_type != TOK_LBRACE) {
        struct Node *d = (struct Node *)
   kcc_ast_alloc(AST_ENUM_DECL);
        d->f0 = (unsigned long)name;
        struct Node *w = (struct Node *)
   kcc_ast_alloc(AST_TYPE_ENUM);
        w->f0 = (unsigned long)d;
        return w;
    }
    kcc_pp_next();                                  /*
   consume '{' */
    void *first = 0, *last = 0;
    while (kcc_tok_type != TOK_RBRACE) {
        void *ename = kcc_strtab_intern(
   kcc_tok_start, kcc_tok_len);
        kcc_pp_next();
        void *value = 0;
        if (kcc_tok_type == TOK_ASSIGN) {
   kcc_pp_next(); value = kcc_parse_cond_expr(); }
        struct Node *e = (struct Node *)
   kcc_ast_alloc(AST_ENUMERATOR);
        e->f0 = (unsigned long)ename;
        e->f1 = (unsigned long)value;
        if (first == 0) { first = e; last = e; }
        else { F4(last) = (unsigned long)e; last =
    e; }
        if (kcc_tok_type != TOK_COMMA) break;
        kcc_pp_next();
        if (kcc_tok_type == TOK_RBRACE) break;
    }
    kcc_parse_expect(TOK_RBRACE);
    struct Node *d = (struct Node *)kcc_ast_alloc(
   AST_ENUM_DECL);
    d->f0 = (unsigned long)name;
    d->f1 = (unsigned long)first;
    struct Node *w = (struct Node *)kcc_ast_alloc(
   AST_TYPE_ENUM);
    w->f0 = (unsigned long)d;
    return w;
}
```

□

734 · used in 738a

68.7. Type Name, Predicate, Initializer, and Dispatch

type_name is specifiers plus an optional abstract declarator; is_type_token tests whether the current token can begin a type; initializer reads an expression or a brace list of possibly-designated elements; and parse_decl ties it together—function definition versus prototype, variable and typedef declarations (with multi-declarator chains), bare aggregate types, and _Static_assert. A designator accepts the GNU array-range form [lo … hi] (marked by is_field = 2, high index in F2) and folds its indices as full constant expressions, matching the assembly parser.

Between the declarator (with its trailing attributes) and the initializer or terminator, GCC permits an *asm label* or *named-register binding*: T v asm("string"). With register storage it ties the variable to a hardware register—arch/x86 declares register unsigned long current_stack_pointer asm("%rsp")—and otherwise it renames the symbol at link time. parse_decl consumes an optional asm(…) here by paren balance rather than assuming a lone string, because the operand is routinely a macro that expands to several adjacent string literals (x86's _ASM_SP becomes " " "rsp" " "). The binding is accepted but not yet honoured in code generation—a plain object is emitted—which suffices for the header declarations a kernel build pulls in; aliasing a register variable's reads and writes to the named register is left for when a compiled function actually uses one.

⟨ *Selfhost: declarations rest* 735 ⟩ ≡

```
void *kcc_parse_type_name(void) {
    struct DeclSpecs ds = kcc_parse_decl_specs();
    void *type = ds.type;
    unsigned long t = kcc_tok_type;
    if (t == TOK_STAR || t == TOK_LBRACKET || t ==
    TOK_LPAREN)
        return kcc_parse_abstract_declarator(type)
   ;
    return type;
}

unsigned long kcc_parse_is_type_token(void) {
    unsigned long t = kcc_tok_type;
    if (t == TOK_VOID || t == TOK_CHAR || t ==
   TOK_SHORT || t == TOK_INT ||
        t == TOK_LONG || t == TOK_FLOAT || t ==
   TOK_DOUBLE || t == TOK_SIGNED ||
        t == TOK_UNSIGNED || t == TOK__BOOL || t
   == TOK__COMPLEX || t == TOK_INT128 ||
        t == TOK_STRUCT || t == TOK_UNION || t ==
   TOK_ENUM || t == TOK_TYPEOF ||
        t == TOK_CONST || t == TOK_VOLATILE || t
   == TOK_RESTRICT ||
        t == TOK__ATOMIC || t == TOK__ALIGNAS)
        return 1;
    if (t != TOK_IDENT) return 0;
    /* __auto_type begins a declaration (`
   __auto_type x = e;`) even inside a
       block, where the statement parser uses this
    predicate to tell a
       declaration from an expression statement.
   */
    if (kcc_tok_len == 11 && kcc_memcmp(
   kcc_tok_start, "__auto_type", 11) == 0) return
   1;
    return kcc_typedef_lookup(kcc_tok_start,
   kcc_tok_len) ? 1 : 0;
}

static void *parse_designated_element(void) {
    unsigned long t = kcc_tok_type;
    if (t != TOK_DOT && t != TOK_LBRACKET) return
   kcc_parse_initializer();
    struct Node *head = 0, *tail = 0;
    for (;;) {
        if (kcc_tok_type == TOK_DOT) {
            kcc_pp_next();
```

```
            void *nm = kcc_strtab_intern(
    kcc_tok_start, kcc_tok_len);
            kcc_pp_next();
            struct Node *d = (struct Node *)
    kcc_ast_alloc(AST_DESIGNATOR);
            d->f0 = (unsigned long)nm;
            d->f1 = 1;
            if (head == 0) { head = d; tail = d; }
            else { tail->f2 = (unsigned long)d;
    tail = d; }
        } else if (kcc_tok_type == TOK_LBRACKET) {
            kcc_pp_next();
            void *ie = kcc_parse_assign_expr();
            struct Node *d = (struct Node *)
    kcc_ast_alloc(AST_DESIGNATOR);
            /* Keep the index EXPRESSION rather
    than folding it now: an index may
               name an enum constant (`[NLA_U32] =
     ...`), which is not in the
               symbol table until sema runs, so it
     can only be evaluated later,
               when the initializer is lowered.
    Literals and macro-expanded
               constants fold there just as well.
    */
            d->f0 = (unsigned long)ie;
            d->f1 = 0;
            if (kcc_tok_type == TOK_ELLIPSIS) {
    /* GNU array range [lo ... hi] */
                kcc_pp_next();
                void *he = kcc_parse_assign_expr()
    ;
                d->f2 = (unsigned long)he;
     /* high index (leaf: no chain) */
                d->f1 = 2;
            }
            kcc_parse_expect(TOK_RBRACKET);
            if (head == 0) { head = d; tail = d; }
            else { tail->f2 = (unsigned long)d;
    tail = d; }
            if (d->f1 == 2) break;
     /* a range is always the leaf */
        } else break;
     }
     kcc_parse_expect(TOK_ASSIGN);
     head->f3 = (unsigned long)
    kcc_parse_initializer();
     return head;
}

void *kcc_parse_initializer(void) {
     if (kcc_tok_type != TOK_LBRACE) return
    kcc_parse_assign_expr();
     kcc_pp_next();                                /*
    consume '{' */
     struct Node *lst = (struct Node *)
    kcc_ast_alloc(AST_INIT_LIST);
     void *first = 0, *last = 0;
     while (kcc_tok_type != TOK_RBRACE) {
         void *el = parse_designated_element();
         if (first == 0) { first = el; last = el; }
         else { F4(last) = (unsigned long)el; last
    = el; }
         if (kcc_tok_type != TOK_COMMA) break;
         kcc_pp_next();
         if (kcc_tok_type == TOK_RBRACE) break;
     }
     kcc_parse_expect(TOK_RBRACE);
     lst->f0 = (unsigned long)first;
     return lst;
}

void *kcc_parse_static_assert(void) {
     kcc_pp_next();
     kcc_parse_expect(TOK_LPAREN);
     void *expr = kcc_parse_cond_expr();
     kcc_parse_expect(TOK_COMMA);
     void *msg = kcc_strtab_intern(kcc_tok_start,
    kcc_tok_len);
     kcc_pp_next();
     kcc_parse_expect(TOK_RPAREN);
     kcc_parse_expect(TOK_SEMI);
     struct Node *n = (struct Node *)kcc_ast_alloc(
    AST_STATIC_ASSERT);
     n->f0 = (unsigned long)expr;
     n->f1 = (unsigned long)msg;
     return n;
}

void *kcc_parse_decl(void) {
     if (kcc_tok_type == TOK__STATIC_ASSERT) return
     kcc_parse_static_assert();

     struct DeclSpecs ds = kcc_parse_decl_specs();
     void *base = ds.type;
     unsigned long storage = ds.storage;

     if (kcc_tok_type == TOK_SEMI) { kcc_pp_next();
     return base; }

     struct Node *d = (struct Node *)
    kcc_parse_declarator(base);
     if (d == 0) { kcc_parse_sync(); return 0; }
     kcc_parse_skip_attributes();
     /* A GCC asm label / named-register binding
    may follow a declarator:
         `T v asm("string")`.  With `register`
    storage it binds v to a hardware
         register (arch/x86 `current_stack_pointer
    asm("%rsp")`); otherwise it
         renames the symbol at link.  Parse and
    accept it so the declaration is not
         rejected; trailing attributes are then
    skipped as usual.  (The binding is
         not yet honoured in codegen---a plain
    object is emitted---which suffices
         for the header declarations the kernel
    build pulls in.) */
     if (kcc_tok_type == TOK_ASM) {
         kcc_pp_next();
         kcc_parse_expect(TOK_LPAREN);
         /* Consume everything to the matching ')'.
      The operand is usually a
            string literal, but macros expand it to
     several adjacent strings
            (arch/x86 `asm(_ASM_SP)` → `" " "rsp" "
     "`), so skip tokens by paren
            balance rather than assuming a single
    string. */
         long depth = 1;
         while (depth > 0 && kcc_tok_type !=
    TOK_EOF) {
             if (kcc_tok_type == TOK_LPAREN) depth
    ++;
             else if (kcc_tok_type == TOK_RPAREN) {
     depth--; if (depth == 0) break; }
             kcc_pp_next();
         }
         kcc_parse_expect(TOK_RPAREN);
         kcc_parse_skip_attributes();
     }
     /* Record any linkage attributes gathered from
     the leading (decl-specs) and
```

```
    trailing lists, then clear them so they do
not leak into the next decl. */
 if (d->f1 && (kcc_attr_weak ||
kcc_attr_section || kcc_attr_alias))
     kcc_attr_record((const char *)d->f1);
 kcc_attr_weak = 0; kcc_attr_section = 0;
kcc_attr_alias = 0;
 unsigned long always_inl =
kcc_attr_always_inline; kcc_attr_always_inline
= 0;

 if (d->tag == AST_FUNC_DECL) {
     if (storage == 1) {
         /* typedef of a function type (`
typedef int f(int);`, or the kernel's
            parenthesised `typedef int (f)(int)
;`): register the name, not a
            function.  The FuncDecl carries the
 name in f1 like a VarDecl. */
         if (d->f1) kcc_typedef_add((const char
 *)d->f1, kcc_strlen((const char *)d->f1));
         /* Rebuild f0 as an AST_TYPE_FUNC so
the typedef keeps its function
            type (return + params) instead of
collapsing to the return type,
            which made `filler_t x; x(...)'
fail as "not a function". */
         { struct Node *ft = (struct Node *)
kcc_ast_alloc(AST_TYPE_FUNC);
           ft->f0 = d->f0;                 /*
return type */
           ft->f1 = d->f2 & ~1UL;          /*
params (mask variadic bit) */
           ft->f2 = d->f2 & 1UL;           /*
variadic */
           d->f0 = (unsigned long)ft; }
         d->tag = AST_TYPEDEF_DECL;
         kcc_parse_expect(TOK_SEMI);
         return d;
     }
     if (kcc_tok_type == TOK_LBRACE) {
         kcc_cur_func_name = (const char *)d->
f1;   /* for __func__ in the body */
         d->f3 = (unsigned long)
kcc_parse_compound();
         kcc_cur_func_name = 0;
         if (storage == 3) d->f2 |= 2;      /*
static function → local linkage */
         /* An `always_inline` definition joins
 the inliner registry so its
            callers can substitute the (
constant) arguments into the body---the
            kernel's static_cpu_has and friends
 need this to fold the constant
            feature bit into an `"i"` inline-
asm operand. */
         if (always_inl && d->f1)
             kcc_register_always_inline((const
char *)d->f1, d);
         return d;
     }
     if (kcc_tok_type != TOK_COMMA) {
         kcc_parse_expect(TOK_SEMI);
         return d;
     }
     /* fall into the multi-declarator loop
below */
 } else if (storage == 1) {
     /* typedef: register the name */
     if (d->f1) {
         const char *nm = (const char *)d->f1;
         kcc_typedef_add(nm, kcc_strlen(nm));
     }
     d->tag = AST_TYPEDEF_DECL;
     /* A single typedef may name several types
---the kernel's
        `typedef struct {...} Rec_Type, *
Rec_Pointer;`---so a comma continues
        into the shared multi-declarator loop,
which registers each further
        name as a typedef too. */
     if (kcc_tok_type != TOK_COMMA) {
         kcc_parse_expect(TOK_SEMI);
         return d;
     }
 } else {
     d->f3 = storage;
     /* An ordinary identifier that reuses a
typedef name shadows it in this
        scope, so a later statement reading the
 name is an expression, not a
        declaration (efi libstub vsprintf's `u8
 *s8' over the `s8' typedef). */
     if (d->f1 && kcc_typedef_lookup((const
char *)d->f1, kcc_strlen((const char *)d->f1)))
         kcc_typedef_shadow((const char *)d->f1
, kcc_strlen((const char *)d->f1));
     /* GNU attribute trailing the declarator,
before '='/','/';' (the kernel's
        DEFINE_LOCK_GUARD_0 emits `T a, *b
__attribute__((__unused__)) = &a;`). */
     if (kcc_tok_type == TOK_ATTRIBUTE)
kcc_parse_skip_attributes();
     if (kcc_tok_type == TOK_ASSIGN) {
         kcc_pp_next();
         d->f2 = (unsigned long)
kcc_parse_initializer();
     }
     /* __auto_type: resolve the deferred
typeof node against the initializer
        just parsed, so the variable takes the
initializer's type. */
     if (base) {
         struct Node *bn = (struct Node *)base;
         if (bn->tag == AST_TYPE_TYPEOF && bn->
f1 == 2 && d->f2) {
             bn->f0 = d->f2; bn->f1 = 0;
         }
     }
     if (kcc_tok_type != TOK_COMMA) {
         kcc_parse_expect(TOK_SEMI);
         return d;
     }
 }

 /* multi-declarator list sharing `base`. */
 struct Node *first = d, *lastn = d;
 while (kcc_tok_type == TOK_COMMA) {
     kcc_pp_next();
     struct Node *nd = (struct Node *)
kcc_parse_declarator(base);
     if (nd == 0) break;
     nd->f3 = storage;
     if (storage == 1) {
/* typedef: register each further name */
         if (nd->f1) kcc_typedef_add((const
char *)nd->f1, kcc_strlen((const char *)nd->f1)
);
         nd->tag = AST_TYPEDEF_DECL;
     } else if (nd->f1 &&
                kcc_typedef_lookup((const char
*)nd->f1, kcc_strlen((const char *)nd->f1))) {
```

```
            kcc_typedef_shadow((const char *)nd->
    f1, kcc_strlen((const char *)nd->f1));
        }
        if (kcc_tok_type == TOK_ATTRIBUTE)
    kcc_parse_skip_attributes();
        if (kcc_tok_type == TOK_ASSIGN) {
            kcc_pp_next();
            nd->f2 = (unsigned long)
    kcc_parse_initializer();
        }
        lastn->f4 = (unsigned long)nd;
        lastn = nd;
     }
     kcc_parse_expect(TOK_SEMI);
     return first;
}
```

□

735 · used in 738a

68.8. Assembling the Translation Unit

⟨ *Selfhost: declarations* 738a ⟩ ≡

```
⟨ Selfhost: declarations preamble 727 ⟩
⟨ Selfhost: declarations attrs 728 ⟩
⟨ Selfhost: declarations specs 730 ⟩
⟨ Selfhost: declarations declarator 731 ⟩
⟨ Selfhost: declarations direct 732 ⟩
⟨ Selfhost: declarations abstract 733 ⟩
⟨ Selfhost: declarations aggregate 734 ⟩
⟨ Selfhost: declarations rest 735 ⟩
```

□

738a · used in 36b

68.9. Verification

`make selfhost-declarations` runs the drop-in loop through the code-generation suite (which declares and defines every function, type, and variable the tests use) and `difftest`; a clean sweep means kcc compiled its own declaration grammar—specifiers, declarators, aggregates, and initializers—so faithfully that the generated code is unchanged.

⟨ *Selfhost: declarations verify* 738b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-declarations - compile kcc's
   declaration parser in C with kcc,
# relink, run the suite.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/3] compiling $BASE/selfhost/declarations.
   c with kcc"
$BASE/kcc -c "$SH/declarations.c" -o "$SH/
   declarations_c.o" >/dev/null

echo "[2/3] linking $BASE/
   kcc_selfhost_declarations (declarations.o -> C)
   "
OBJ=""
for f in $sources; do
    if [ "$f" = "declarations" ]; then OBJ="$OBJ
   $SH/declarations_c.o"; else OBJ="$OBJ $BASE/src
   /${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_declarations $OBJ -
   lSystem -syslibroot "$SDK" -e _main -arch arm64

echo "[3/3] running the codegen suite with the
   self-hosted declaration parser"
cp $BASE/kcc "$SH/kcc_asm_backup_declarations"
cp $BASE/kcc_selfhost_declarations $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_declarations" $BASE/kcc
exit $status
```

□

738b · used in 36c · uses _main 621, 629

69. Self-Hosting: The DWARF Emitter in C

The DWARF emitter of Chapter 42 is a clean, self-contained slice of the back end: it consumes the IR function list and the code-buffer size, and produces the three DWARF v4 sections (`.debug_abbrev`, `.debug_info`, `.debug_line`) into a scratch blob that the Mach-O writer places in a non-loaded `__DWARF` segment. Because it is gated behind `-g` and touches no other object except through four exported entry points and one flag, it ports to C almost mechanically—the same little-endian and LEB128 byte streams, the same abbreviation table, the same per-function DIEs and line-number program.

The verification is stronger than the code-generation suite alone can show: that suite never passes `-g`, so beyond confirming no regression we compile a translation unit *with* `-g` under both the assembly and the C-compiled emitter and byte-compare the resulting objects—identical debug bytes mean the port is faithful.

69.1. Buffer State, Constants, and Byte Emitters

The port owns the scratch buffer and the per-section offset/length words as file-internal state, exports only `kcc_dwarf_enabled` (the driver and the Mach-O writer read it), and declares the DWARF tag/attribute/form codes and the IR-function field offsets it reads. The byte emitters append little-endian integers, unsigned and signed LEB128 values, and NUL-terminated strings, exactly as the assembly does—every multi-byte value decomposes through `dw_u8` so the byte stream is identical.

⟨ *Selfhost: dwarf preamble* 739 ⟩ ≡

```
void *kcc_arena_alloc(unsigned long n);
void *kcc_vec_get(void *vec, unsigned long index);
void kcc_macho_write_bytes(const void *ptr,
   unsigned long len);
void kcc_macho_write64(unsigned long v);
void kcc_macho_write32(unsigned long v);

extern char *kcc_src_path;          /* pointer to
   the source file path */
extern unsigned long kcc_code_pos;  /* code-buffer
    write position (= code size) */
extern void *kcc_ir_funcs;          /* vec of IR
   function pointers */

/* DWARF v4 codes. */
enum {
    DW_TAG_compile_unit = 0x11, DW_TAG_subprogram
   = 0x2e,
    DW_CHILDREN_no = 0, DW_CHILDREN_yes = 1,
    DW_AT_name = 0x03, DW_AT_stmt_list = 0x10,
   DW_AT_low_pc = 0x11,
    DW_AT_high_pc = 0x12, DW_AT_language = 0x13,
   DW_AT_comp_dir = 0x1b,
    DW_AT_producer = 0x25, DW_AT_decl_line = 0x3b,
    DW_FORM_addr = 0x01, DW_FORM_data2 = 0x05,
   DW_FORM_string = 0x08,
    DW_FORM_udata = 0x0f, DW_FORM_sec_offset = 0
   x17,
    DW_LANG_C99 = 0x0c,
    DW_LNS_copy = 1, DW_LNS_advance_line = 3,
    DW_LNE_end_sequence = 1, DW_LNE_set_address =
   2
};

#define DWARF_BUF_CAP 262144
#define LC_SEGMENT_64 0x19
#define VEC_COUNT     8
#define IRF_NAME      0
#define IRF_LINE      56
#define IRF_LOWPC     64

unsigned long kcc_dwarf_enabled = 0;     /* set by
    -g (read by driver + macho) */

static void *dwarf_buf = 0;
static unsigned long dwarf_pos = 0;
static unsigned long dw_abbrev_off = 0,
   dw_abbrev_len = 0;
static unsigned long dw_info_off = 0, dw_info_len
   = 0;
static unsigned long dw_line_off = 0, dw_line_len
   = 0;

static const char dw_producer[] = "kcc";
static const char dw_compdir[]  = "/";
static const char dw_segname[16]   = "__DWARF";
static const char dw_sn_abbrev[16] = "
   __debug_abbrev";
static const char dw_sn_info[16]   = "__debug_info
   ";
static const char dw_sn_line[16]   = "__debug_line
   ";

static unsigned long irf(void *f, unsigned long
   off) {
    return *(unsigned long *)((char *)f + off);
}

static void dw_u8(unsigned long v) {
    unsigned long p = dwarf_pos;
    ((unsigned char *)dwarf_buf)[p] = (unsigned
   char)v;
    dwarf_pos = p + 1;
}
static void dw_u16(unsigned long v) { dw_u8(v & 0
   xff); dw_u8((v >> 8) & 0xff); }
static void dw_u32(unsigned long v) {
    dw_u8(v & 0xff); dw_u8((v >> 8) & 0xff);
    dw_u8((v >> 16) & 0xff); dw_u8((v >> 24) & 0
   xff);
}
static void dw_u64(unsigned long v) { dw_u32(v & 0
   xffffffff); dw_u32(v >> 32); }

static void dw_uleb(unsigned long v) {
    for (;;) {
        unsigned long byte = v & 0x7f;
        v >>= 7;
        if (v == 0) { dw_u8(byte); return; }
        dw_u8(byte | 0x80);
    }
}
static void dw_sleb(long v) {
    for (;;) {
        unsigned long byte = (unsigned long)v & 0
   x7f;
```

```
        v >>= 7;                                /*
   arithmetic shift keeps the sign */
        int sign = byte & 0x40;
        if ((v == 0 && !sign) || (v == -1 && sign)
   ) { dw_u8(byte); return; }
        dw_u8(byte | 0x80);
    }
}
static void dw_str(const char *s) {
    if (s) { while (*s) { dw_u8((unsigned char)*s)
   ; s++; } }
    dw_u8(0);
}
static unsigned long dw_pos(void) { return
   dwarf_pos; }
static void dw_patch32(unsigned long pos, unsigned
    long v) {
    unsigned char *p = (unsigned char *)dwarf_buf
   + pos;
    p[0] = v & 0xff; p[1] = (v >> 8) & 0xff;
    p[2] = (v >> 16) & 0xff; p[3] = (v >> 24) & 0
   xff;
}
```

□

739 · used in 742a

69.2. The Abbreviation Table

Two DIE shapes: a compile_unit with children (producer, language, file name, working directory, whole-unit address range, and a pointer to the line program) and a leaf subprogram (name, address range, declaration line).

⟨ *Selfhost: dwarf abbrev* 740a ⟩ ≡

```
static void dwarf_build_abbrev(void) {
    dw_abbrev_off = dw_pos();
    /* abbrev 1: compile_unit, has children */
    dw_uleb(1);
    dw_uleb(DW_TAG_compile_unit);
    dw_u8(DW_CHILDREN_yes);
    dw_uleb(DW_AT_producer);  dw_uleb(
   DW_FORM_string);
    dw_uleb(DW_AT_language);  dw_uleb(
   DW_FORM_data2);
    dw_uleb(DW_AT_name);      dw_uleb(
   DW_FORM_string);
    dw_uleb(DW_AT_comp_dir);  dw_uleb(
   DW_FORM_string);
    dw_uleb(DW_AT_low_pc);    dw_uleb(DW_FORM_addr
   );
    dw_uleb(DW_AT_high_pc);   dw_uleb(DW_FORM_addr
   );
    dw_uleb(DW_AT_stmt_list); dw_uleb(
   DW_FORM_sec_offset);
    dw_u8(0); dw_u8(0);
    /* abbrev 2: subprogram, leaf */
    dw_uleb(2);
    dw_uleb(DW_TAG_subprogram);
    dw_u8(DW_CHILDREN_no);
    dw_uleb(DW_AT_name);      dw_uleb(
   DW_FORM_string);
    dw_uleb(DW_AT_low_pc);    dw_uleb(DW_FORM_addr
   );
    dw_uleb(DW_AT_high_pc);   dw_uleb(DW_FORM_addr
   );
    dw_uleb(DW_AT_decl_line); dw_uleb(
   DW_FORM_udata);
    dw_u8(0); dw_u8(0);
    /* terminator abbrev (code 0) */
    dw_u8(0);
    dw_abbrev_len = dw_pos() - dw_abbrev_off;
}
```

□

740a · used in 742a

69.3. The Info Section

One compile-unit DIE followed by one subprogram DIE per function. Each function's high_pc is the next function's entry (functions are laid out in order) or the total code size for the last. The unit-length field is reserved and back-patched once the section is complete.

⟨ *Selfhost: dwarf info* 740b ⟩ ≡

```
static void dwarf_build_info(void) {
    dw_info_off = dw_pos();
    unsigned long len_pos = dw_pos();
    dw_u32(0);                                  /*
   unit_length (patched) */
    dw_u16(4);                                  /*
   version */
    dw_u32(0);                                  /*
   debug_abbrev_offset */
    dw_u8(8);                                   /*
   address_size */
    /* CU DIE (abbrev 1) */
    dw_uleb(1);
    dw_str(dw_producer);
    dw_u16(DW_LANG_C99);
    dw_str(kcc_src_path);
    dw_str(dw_compdir);
    dw_u64(0);                                  /*
   low_pc = text base */
    unsigned long code_size = kcc_code_pos;
    dw_u64(code_size);                          /*
   high_pc = code size */
    dw_u32(0);                                  /*
   stmt_list offset */

    void *funcs = kcc_ir_funcs;
    if (funcs) {
        unsigned long count = *(unsigned long *)((
   char *)funcs + VEC_COUNT);
        for (unsigned long i = 0; i < count; i++)
   {
            void *f = *(void **)kcc_vec_get(funcs,
    i);
            if (!f) continue;
            dw_uleb(2);                         /*
   abbrev subprogram */
            dw_str((const char *)irf(f, IRF_NAME))
   ;
            dw_u64(irf(f, IRF_LOWPC));          /*
   low_pc */
            unsigned long hi;
            if (i + 1 < count) {
                void *nf = *(void **)kcc_vec_get(
   funcs, i + 1);
                hi = irf(nf, IRF_LOWPC);
            } else {
                hi = code_size;
            }
```

```
            dw_u64(hi);                                 /*
    high_pc */
            dw_uleb(irf(f, IRF_LINE));                  /*
    decl_line */
        }
    }
    dw_u8(0);                                           /*
    null DIE ends CU children */
    dw_patch32(len_pos, dw_pos() - (len_pos + 4));
    dw_info_len = dw_pos() - dw_info_off;
}
```

□

740b · used in 742a

69.4. The Line-Number Program

A standard v4 line-program header (opcode table, empty include-directory list, one file entry) followed by, for each function, a `set_address` to its entry, an `advance_line` by the delta from the running line register, and a copy to emit a row. A closing `set_address` to the code size and an `end_sequence` terminate the program.

⟨ *Selfhost: dwarf line* 741a ⟩ ≡

```
static void dwarf_build_line(void) {
    dw_line_off = dw_pos();
    unsigned long unit_len_pos = dw_pos();
    dw_u32(0);                                          /*
    unit_length (patched) */
    dw_u16(4);                                          /*
    version */
    unsigned long hdr_len_pos = dw_pos();
    dw_u32(0);                                          /*
    header_length (patched) */
    dw_u8(4);                                           /*
    minimum_instruction_length */
    dw_u8(1);                                           /*
    maximum_operations_per_instruction */
    dw_u8(1);                                           /*
    default_is_stmt */
    dw_u8((unsigned long)(-5));                         /*
    line_base (signed) */
    dw_u8(14);                                          /*
    line_range */
    dw_u8(13);                                          /*
    opcode_base */
    /* standard_opcode_lengths for opcodes 1..12
    */
    dw_u8(0); dw_u8(1); dw_u8(1); dw_u8(1); dw_u8
    (1); dw_u8(0);
    dw_u8(0); dw_u8(0); dw_u8(1); dw_u8(0); dw_u8
    (0); dw_u8(1);
    /* include_directories: empty → single
    terminating 0 */
    dw_u8(0);
    /* file_names: one entry for the source file
    */
    dw_str(kcc_src_path);
    dw_uleb(0);                                         /*
    dir index */
    dw_uleb(0);                                         /*
    mtime */
    dw_uleb(0);                                         /*
    length */
    dw_u8(0);                                           /*
    end of file table */
    dw_patch32(hdr_len_pos, dw_pos() - (
    hdr_len_pos + 4));

    /* line-number program */
    long cur_line = 1;                                  /*
    line register starts at 1 */
    void *funcs = kcc_ir_funcs;
    if (funcs) {
        unsigned long count = *(unsigned long *)((
    char *)funcs + VEC_COUNT);
        for (unsigned long i = 0; i < count; i++)
    {
            void *f = *(void **)kcc_vec_get(funcs,
    i);
            if (!f) continue;
            /* DW_LNE_set_address low_pc */
            dw_u8(0); dw_uleb(9); dw_u8(
    DW_LNE_set_address);
            dw_u64(irf(f, IRF_LOWPC));
            /* DW_LNS_advance_line by (target -
    current) */
            dw_u8(DW_LNS_advance_line);
            long target = (long)irf(f, IRF_LINE);
            dw_sleb(target - cur_line);
            cur_line = target;
            /* DW_LNS_copy → emit a row */
            dw_u8(DW_LNS_copy);
        }
    }
    /* final set_address to code size, then
    end_sequence */
    dw_u8(0); dw_uleb(9); dw_u8(DW_LNE_set_address
    );
    dw_u64(kcc_code_pos);
    dw_u8(0); dw_uleb(1); dw_u8(
    DW_LNE_end_sequence);
    dw_patch32(unit_len_pos, dw_pos() - (
    unit_len_pos + 4));
    dw_line_len = dw_pos() - dw_line_off;
}
```

□

741a · used in 742a

69.5. The Entry Point and Mach-O Emission

`kcc_dwarf_build` is a no-op unless `-g`; otherwise it allocates the scratch buffer and lays down the three sections. The three Mach-O helpers give the writer a decoupled interface: the section-64 descriptors, the `__DWARF` segment load command, the total blob length (so the writer can reserve the file span), and the blob copy itself.

⟨ *Selfhost: dwarf build* 741b ⟩ ≡

```
void kcc_dwarf_build(void) {
    if (!kcc_dwarf_enabled) return;
    dwarf_buf = kcc_arena_alloc(DWARF_BUF_CAP);
    dwarf_pos = 0;
    dwarf_build_abbrev();
    dwarf_build_info();
    dwarf_build_line();
}

/* Write one 80-byte section_64 in the __DWARF
   segment. */
```

```
static void dwarf_sect64(const char *name,
   unsigned long addr,
                         unsigned long size,
   unsigned long fileoff) {
    kcc_macho_write_bytes(name, 16);          /*
   sectname */
    kcc_macho_write_bytes(dw_segname, 16);    /*
   segname __DWARF */
    kcc_macho_write64(addr);
    kcc_macho_write64(size);
    kcc_macho_write32(fileoff);
    kcc_macho_write32(0);                      /*
    align */
    kcc_macho_write32(0);                      /*
    reloff */
    kcc_macho_write32(0);                      /*
    nreloc */
    kcc_macho_write32(0);                      /*
    flags */
    kcc_macho_write32(0);                      /*
    reserved1 */
    kcc_macho_write32(0);                      /*
    reserved2 */
    kcc_macho_write32(0);                      /*
    reserved3 */
}

void kcc_dwarf_emit_macho(unsigned long
   dwarf_file_base, unsigned long vmaddr) {
    unsigned long a = dw_abbrev_len, in =
   dw_info_len, ln = dw_line_len;
    unsigned long total = a + in + ln;
    /* LC_SEGMENT_64 (__DWARF) */
    kcc_macho_write32(LC_SEGMENT_64);
    kcc_macho_write32(72 + 80 * 3);           /*
   cmdsize = 312 */
    kcc_macho_write_bytes(dw_segname, 16);
    kcc_macho_write64(vmaddr);                /*
   vmaddr (past __TEXT) */
    kcc_macho_write64(total);                 /*
   vmsize */
    kcc_macho_write64(dwarf_file_base);       /*
   fileoff */
    kcc_macho_write64(total);                 /*
   filesize */
    kcc_macho_write32(0);                      /*
    maxprot (not mapped) */
    kcc_macho_write32(0);                      /*
    initprot */
    kcc_macho_write32(3);                      /*
    nsects */
    kcc_macho_write32(0);                      /*
    flags */
    dwarf_sect64(dw_sn_abbrev, vmaddr, a,
   dwarf_file_base);
    dwarf_sect64(dw_sn_info, vmaddr + a, in,
   dwarf_file_base + a);
    dwarf_sect64(dw_sn_line, vmaddr + a + in, ln,
   dwarf_file_base + a + in);
}

unsigned long kcc_dwarf_total_len(void) {
    return dw_abbrev_len + dw_info_len +
   dw_line_len;
}

void kcc_dwarf_append_blob(void) {
    kcc_macho_write_bytes(dwarf_buf, dw_abbrev_len
    + dw_info_len + dw_line_len);
}
```

□

741b · used in 742a

69.6. Assembling the Translation Unit

⟨ *Selfhost: dwarf* 742a ⟩ ≡

⟨ *Selfhost: dwarf preamble* 739 ⟩
⟨ *Selfhost: dwarf abbrev* 740a ⟩
⟨ *Selfhost: dwarf info* 740b ⟩
⟨ *Selfhost: dwarf line* 741a ⟩
⟨ *Selfhost: dwarf build* 741b ⟩

□

742a · used in 36d

69.7. Verification

`make selfhost-dwarf` runs the code-generation suite with the C emitter linked in (proving no symbol or link regression), then compiles a small translation unit *with* `-g` under both the assembly and the C emitter and byte-compares the two objects. Byte-identical `__DWARF` segments confirm the port reproduces the abbreviation table, the DIEs, and the line-number program exactly.

⟨ *Selfhost: dwarf verify* 742b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-dwarf - compile kcc's DWARF
   emitter in C with kcc, relink,
# run the codegen suite, and byte-compare a -g
   object against the asm build.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/4] compiling $BASE/selfhost/dwarf.c with
   kcc"
$BASE/kcc -c "$SH/dwarf.c" -o "$SH/dwarf_c.o" >/
   dev/null

echo "[2/4] linking $BASE/kcc_selfhost_dwarf (
   dwarf.o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "dwarf" ]; then OBJ="$OBJ $SH/
   dwarf_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
   fi
done
ld -o $BASE/kcc_selfhost_dwarf $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

echo "[3/4] byte-comparing a -g object: asm
   emitter vs C emitter"
printf 'int add(int a,int b){return a+b;}\nint
   main(void){return add(2,3);}\n' > "$SH/
   dwarf_probe.c"
```

```
$BASE/kcc -c -g "$SH/dwarf_probe.c" -o "$SH/
   dwarf_probe_asm.o" >/dev/null
cp $BASE/kcc "$SH/kcc_asm_backup_dwarf"
cp $BASE/kcc_selfhost_dwarf $BASE/kcc
$BASE/kcc -c -g "$SH/dwarf_probe.c" -o "$SH/
   dwarf_probe_c.o" >/dev/null
cp "$SH/kcc_asm_backup_dwarf" $BASE/kcc
if ! cmp "$SH/dwarf_probe_asm.o" "$SH/
   dwarf_probe_c.o"; then
    echo "FAIL: -g object differs between asm and
   C DWARF emitter"; exit 1
fi
echo "  -g objects byte-identical"

echo "[4/4] running the codegen suite with the
   self-hosted DWARF emitter"
cp $BASE/kcc_selfhost_dwarf $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_dwarf" $BASE/kcc
exit $status
```

□

742b · used in 36e · uses _main 621, 629

70. Self-Hosting: The ARM64 Encoder in C

The `emit_arm64` object carries two cooperating back ends: the direct binary encoder of Chapter 36—a growable code buffer plus one pure bit-packing function per instruction format—and the textual assembler of Chapter 37, which walks that same code buffer and renders it as `.s` for the platform assembler. Both are mechanical: the encoders are arithmetic on register and immediate fields, and the renderer is linear scans that append bytes to a text buffer. Porting the pair to C is therefore low-risk, and the verification is unusually strong—`difftest` byte-compares the direct-encoder object against clang on every case, and an `--emit-asm` byte-compare covers the renderer.

The second target's x86-64 back end reuses this module's text-buffer helpers (`kcc_as_init`, `kcc_as_ch`, `kcc_as_str`, `kcc_as_dec`, `kcc_as_write_file`), so the C port exports them under the same names; the encoders and code-buffer entry points stay exported for the code generator, and this module remains the definition site of the code buffer's three state words.

70.1. Externs, Constants, and Module State

⟨ *Selfhost: emit_arm64 preamble* 745a ⟩ ≡

```
void *kcc_arena_alloc(unsigned long n);
void *kcc_memcpy(void *dst, const void *src,
   unsigned long n);
long kcc_sys_open(const char *path, unsigned long
   flags, unsigned long mode);
long kcc_sys_write(long fd, const void *buf,
   unsigned long n);
long kcc_sys_close(long fd);

/* Linker tables (vec pointers) and codegen state
   read by the renderer. */
extern void *kcc_linker_syms;
extern void *kcc_linker_relocs;
extern unsigned long kcc_cstr_pos;
extern unsigned long kcc_globals_size;
extern unsigned long kcc_modinit_sym;

/* Vec / linker-symbol / relocation layout. */
#define VEC_DATA        0
#define VEC_COUNT       8
#define SYM_NAME        0
#define SYM_SECTION     8
#define SYM_VALUE       16
#define SYM_FLAGS       24
#define SYM_ENTRY_SIZE  32
#define N_EXT           0x01
#define RELOC_OFFSET    0
#define RELOC_TYPE      8
#define RELOC_SYM       16
#define RELOC_SIZE      24
#define ARM64_RELOC_BRANCH26            2
#define ARM64_RELOC_PAGE21              3
#define ARM64_RELOC_PAGEOFF12           4
#define ARM64_RELOC_GOT_LOAD_PAGE21     5
#define ARM64_RELOC_GOT_LOAD_PAGEOFF12  6

#define CODEBUF_INIT_CAP 4096

/* Code-buffer state - this module is its
   definition site (read by codegen and
 * by the DWARF emitter). */
void *kcc_code_buf = 0;
unsigned long kcc_code_pos = 0;
unsigned long kcc_code_cap = 0;

/* Textual-.s buffer + inline-asm snippet table +
   branch-target flag array. */
static void *as_buf = 0;
static unsigned long as_pos = 0;
static unsigned long as_cap = 0;
void *kcc_asm_snips = 0;
unsigned long kcc_asm_snip_count = 0;
unsigned long kcc_asm_snip_cap = 0;
void *kcc_as_targets = 0;
char kcc_as_s_env[] = "KCC_EMIT_ASM";

static const char s_sect[]    = "\t.section __TEXT
   ,__text,regular\n";
static const char s_p2a[]     = "\t.p2align 2\n";
static const char s_globl[]   = "\t.globl ";
static const char s_long[]    = "\t.long 0x";
static const char s_byte[]    = "\t.byte 0x";
static const char s_bl[]      = "\tbl\t";
static const char s_adrp[]    = "\tadrp\t";
static const char s_add[]     = "\tadd\t";
static const char s_ldr[]     = "\tldr\t";
static const char s_page[]    = "@PAGE";
static const char s_pageoff[] = "@PAGEOFF";
static const char s_gotpage[] = "@GOTPAGE";
static const char s_gotoff[]  = "@GOTPAGEOFF";
static const char s_hexdig[]  = "0123456789abcdef"
   ;
static const char s_bdot[]    = "\tb.";
static const char s_b[]       = "\tb\t";
static const char s_cbz[]     = "\tcbz\t";
static const char s_cbnz[]    = "\tcbnz\t";
static const char s_lc[]      = "Lc";
static const char s_zerofill[]= "\t.zerofill
   __DATA,__common,";
static const char s_a3nl[]    = ",3\n";
static const char s_modsect[] =
    "\t.section __DATA,__mod_init_func,
   mod_init_funcs\n\t.p2align 3\n\t.quad ";
static const char s_conds[] =
    "eq\0ne\0hs\0lo\0mi\0pl\0vs\0vc\0hi\0ls\0ge\0
   lt\0gt\0le\0al\0nv";

static unsigned long u64at(void *p, unsigned long
   off) {
    return *(unsigned long *)((char *)p + off);
}
```

□

745a · used in 750b

70.2. Instruction Encoders

Each encoder packs its operand fields into the fixed base word for its format; the `sf` bit selects 64- vs. 32-bit, and register/immediate fields land at their documented positions. These are the exact bit patterns of Chapter 36.

⟨ *Selfhost: emit_arm64 encoders* 745b ⟩ ≡

```
unsigned kcc_enc_add_reg(unsigned rd, unsigned rn,
    unsigned rm, unsigned sf) {
    return 0x0B000000u | (sf << 31) | (rm << 16) |
    (rn << 5) | rd;
}
unsigned kcc_enc_sub_reg(unsigned rd, unsigned rn,
    unsigned rm, unsigned sf) {
    return 0x4B000000u | (sf << 31) | (rm << 16) |
    (rn << 5) | rd;
}
unsigned kcc_enc_mul(unsigned rd, unsigned rn,
   unsigned rm, unsigned sf) {
    return 0x1B000000u | (sf << 31) | (rm << 16) |
    (31u << 10) | (rn << 5) | rd;
}
unsigned kcc_enc_sdiv(unsigned rd, unsigned rn,
   unsigned rm, unsigned sf) {
    return 0x1AC00C00u | (sf << 31) | (rm << 16) |
    (rn << 5) | rd;
}
unsigned kcc_enc_udiv(unsigned rd, unsigned rn,
   unsigned rm, unsigned sf) {
    return 0x1AC00800u | (sf << 31) | (rm << 16) |
    (rn << 5) | rd;
}
unsigned kcc_enc_msub(unsigned rd, unsigned rn,
   unsigned rm, unsigned ra, unsigned sf) {
    return 0x1B008000u | (sf << 31) | (rm << 16) |
    (ra << 10) | (rn << 5) | rd;
}
unsigned kcc_enc_add_imm(unsigned rd, unsigned rn,
    unsigned imm12, unsigned sf) {
    return 0x11000000u | (sf << 31) | (imm12 <<
   10) | (rn << 5) | rd;
}
unsigned kcc_enc_sub_imm(unsigned rd, unsigned rn,
    unsigned imm12, unsigned sf) {
    return 0x51000000u | (sf << 31) | (imm12 <<
   10) | (rn << 5) | rd;
}
unsigned kcc_enc_movz(unsigned rd, unsigned imm16,
    unsigned hw, unsigned sf) {
    return 0x52800000u | (sf << 31) | (hw << 21) |
    (imm16 << 5) | rd;
}
unsigned kcc_enc_movk(unsigned rd, unsigned imm16,
    unsigned hw, unsigned sf) {
    return 0x72800000u | (sf << 31) | (hw << 21) |
    (imm16 << 5) | rd;
}
unsigned kcc_enc_and_reg(unsigned rd, unsigned rn,
    unsigned rm, unsigned sf) {
    return 0x0A000000u | (sf << 31) | (rm << 16) |
    (rn << 5) | rd;
}
unsigned kcc_enc_orr_reg(unsigned rd, unsigned rn,
    unsigned rm, unsigned sf) {
    return 0x2A000000u | (sf << 31) | (rm << 16) |
    (rn << 5) | rd;
}
unsigned kcc_enc_eor_reg(unsigned rd, unsigned rn,
    unsigned rm, unsigned sf) {
    return 0x4A000000u | (sf << 31) | (rm << 16) |
    (rn << 5) | rd;
}
unsigned kcc_enc_mvn(unsigned rd, unsigned rm,
   unsigned sf) {
    return 0x2A200000u | (sf << 31) | (rm << 16) |
    (31u << 5) | rd;
}
unsigned kcc_enc_lsl_reg(unsigned rd, unsigned rn,
    unsigned rm, unsigned sf) {
    return 0x1AC02000u | (sf << 31) | (rm << 16) |
    (rn << 5) | rd;
}
unsigned kcc_enc_lsr_reg(unsigned rd, unsigned rn,
    unsigned rm, unsigned sf) {
    return 0x1AC02400u | (sf << 31) | (rm << 16) |
    (rn << 5) | rd;
}
unsigned kcc_enc_asr_reg(unsigned rd, unsigned rn,
    unsigned rm, unsigned sf) {
    return 0x1AC02800u | (sf << 31) | (rm << 16) |
    (rn << 5) | rd;
}
unsigned kcc_enc_cmp_reg(unsigned rn, unsigned rm,
    unsigned sf) {
    return 0x6B000000u | (sf << 31) | (rm << 16) |
    (rn << 5) | 31u;
}
unsigned kcc_enc_cset(unsigned rd, unsigned cond,
   unsigned sf) {
    return 0x1A800000u | 0x400u | (sf << 31) | (31
   u << 16) | (cond << 12) | (31u << 5) | rd;
}
unsigned kcc_enc_str_imm(unsigned rt, unsigned rn,
    unsigned imm12, unsigned size) {
    unsigned base;
    if (size == 2) base = 0x39000000u;          /*
   STRB */
    else if (size == 3) base = 0x79000000u;     /*
    STRH */
    else if (size == 0) base = 0xB9000000u;     /*
    32-bit */
    else base = 0xF9000000u;                    /*
    64-bit */
    return base | (imm12 << 10) | (rn << 5) | rt;
}
unsigned kcc_enc_ldr_imm(unsigned rt, unsigned rn,
    unsigned imm12, unsigned size) {
    unsigned base;
    if (size == 2) base = 0x39400000u;          /*
    LDRB */
    else if (size == 3) base = 0x79400000u;     /*
    LDRH */
    else if (size == 4) base = 0xB9800000u;     /*
    LDRSW */
    else if (size == 5) base = 0x39800000u;     /*
    LDRSB */
    else if (size == 6) base = 0x79800000u;     /*
    LDRSH */
    else if (size == 0) base = 0xB9400000u;     /*
    32-bit */
    else base = 0xF9400000u;                    /*
    64-bit */
    return base | (imm12 << 10) | (rn << 5) | rt;
}
unsigned kcc_enc_stp_pre(unsigned rt1, unsigned
   rt2, unsigned rn, unsigned imm7) {
    return 0xA9800000u | ((imm7 & 0x7F) << 15) | (
   rt2 << 10) | (rn << 5) | rt1;
}
unsigned kcc_enc_ldp_post(unsigned rt1, unsigned
   rt2, unsigned rn, unsigned imm7) {
    return 0xA8C00000u | ((imm7 & 0x7F) << 15) | (
   rt2 << 10) | (rn << 5) | rt1;
}
```

```
unsigned kcc_enc_mov_reg(unsigned rd, unsigned rm,
    unsigned sf) {
    return 0x2A000000u | (sf << 31) | (rm << 16) |
    (31u << 5) | rd;
}
unsigned kcc_enc_fp_dp2(unsigned rd, unsigned rn,
   unsigned rm, unsigned opcode) {
    return 0x1E600800u | (opcode << 12) | (rm <<
   16) | (rn << 5) | rd;
}
unsigned kcc_enc_scvtf(unsigned rd, unsigned rn)
    { return 0x9E620000u | (rn << 5) | rd; }
unsigned kcc_enc_ucvtf(unsigned rd, unsigned rn)
    { return 0x9E630000u | (rn << 5) | rd; }
unsigned kcc_enc_fcvtzs(unsigned rd, unsigned rn)
    { return 0x9E780000u | (rn << 5) | rd; }
unsigned kcc_enc_fcvt_s_d(unsigned rd, unsigned rn
   ){ return 0x1E624000u | (rn << 5) | rd; }
unsigned kcc_enc_fcvt_d_s(unsigned rd, unsigned rn
   ){ return 0x1E22C000u | (rn << 5) | rd; }
unsigned kcc_enc_fmov_d_x(unsigned rd, unsigned rn
   ){ return 0x9E670000u | (rn << 5) | rd; }
unsigned kcc_enc_fmov_x_d(unsigned rd, unsigned rn
   ){ return 0x9E660000u | (rn << 5) | rd; }
unsigned kcc_enc_fmov_d_d(unsigned rd, unsigned rn
   ){ return 0x1E604000u | (rn << 5) | rd; }
unsigned kcc_enc_fneg_d(unsigned rd, unsigned rn)
    { return 0x1E614000u | (rn << 5) | rd; }
unsigned kcc_enc_fcmp(unsigned rn, unsigned rm)
    { return 0x1E602000u | (rm << 16) | (rn << 5);
    }
unsigned kcc_enc_ldr_fp(unsigned rt, unsigned rn,
   unsigned imm12) {
    return 0xFD400000u | (imm12 << 10) | (rn << 5)
    | rt;
}
unsigned kcc_enc_str_fp(unsigned rt, unsigned rn,
   unsigned imm12) {
    return 0xFD000000u | (imm12 << 10) | (rn << 5)
    | rt;
}
unsigned kcc_enc_ldr_fp_s(unsigned rt, unsigned rn
   , unsigned imm12) {
    return 0xBD400000u | (imm12 << 10) | (rn << 5)
    | rt;
}
unsigned kcc_enc_str_fp_s(unsigned rt, unsigned rn
   , unsigned imm12) {
    return 0xBD000000u | (imm12 << 10) | (rn << 5)
    | rt;
}
unsigned kcc_enc_b(long off) {
    return (unsigned)(((unsigned long)(off >> 2))
   & 0x03FFFFFF) | 0x14000000u;
}
unsigned kcc_enc_bl(long off) {
    return (unsigned)(((unsigned long)(off >> 2))
   & 0x03FFFFFF) | 0x94000000u;
}
unsigned kcc_enc_b_cond(long off, unsigned cond) {
    unsigned imm19 = (unsigned)(((unsigned long)(
   off >> 2)) & 0x7FFFF);
    return 0x54000000u | (imm19 << 5) | cond;
}
unsigned kcc_enc_cbz(unsigned rt, long off,
   unsigned sf) {
    unsigned imm19 = (unsigned)(((unsigned long)(
   off >> 2)) & 0x7FFFF);
    return 0x34000000u | (sf << 31) | (imm19 << 5)
    | rt;
}
unsigned kcc_enc_cbnz(unsigned rt, long off,
   unsigned sf) {
    unsigned imm19 = (unsigned)(((unsigned long)(
   off >> 2)) & 0x7FFFF);
    return 0x35000000u | (sf << 31) | (imm19 << 5)
    | rt;
}
unsigned kcc_enc_ret(void) { return 0xD65F03C0u; }
unsigned kcc_enc_blr(unsigned rn) { return 0
   xD63F0000u | (rn << 5); }
unsigned kcc_enc_nop(void) { return 0xD503201Fu; }
unsigned kcc_enc_brk(unsigned imm16) { return 0
   xD4200000u | (imm16 << 5); }
unsigned kcc_enc_adrp(unsigned rd, unsigned imm21)
    {
    unsigned immlo = imm21 & 0x3;
    unsigned immhi = (imm21 >> 2) & 0x7FFFF;
    return 0x90000000u | (immlo << 29) | (immhi <<
    5) | rd;
}
```

□

745b · used in 750b

70.3. The Code Buffer

The direct encoder appends four bytes per instruction, doubling the arena buffer when full; `patch32` back-patches a resolved forward branch.

⟨ *Selfhost: emit_arm64 codebuf* 747 ⟩ ≡

```
void kcc_code_init(void) {
    kcc_code_buf = kcc_arena_alloc(
   CODEBUF_INIT_CAP);
    kcc_code_pos = 0;
    kcc_code_cap = CODEBUF_INIT_CAP;
}
void kcc_code_emit32(unsigned word) {
    if (kcc_code_pos + 4 > kcc_code_cap) {
        unsigned long ncap = kcc_code_cap << 1;
        kcc_code_cap = ncap;
        void *nb = kcc_arena_alloc(ncap);
        kcc_memcpy(nb, kcc_code_buf, kcc_code_pos)
   ;
        kcc_code_buf = nb;
    }
    *(unsigned *)((char *)kcc_code_buf +
   kcc_code_pos) = word;
    kcc_code_pos += 4;
}
unsigned long kcc_code_current_pos(void) { return
   kcc_code_pos; }
void kcc_code_patch32(unsigned long offset,
   unsigned word) {
    *(unsigned *)((char *)kcc_code_buf + offset) =
    word;
}
```

□

747 · used in 750b

70.4. The Text Buffer

Assembly text accumulates in a growable arena buffer, appended a byte, string, decimal, or hex digit at a time; the register-name helpers spell `xzr` and `wzr` for register 31.

⟨ *Selfhost: emit_arm64 as-buffer* 748a ⟩ ≡

```
void kcc_as_init(void) {
    as_buf = kcc_arena_alloc(65536);
    as_pos = 0;
    as_cap = 65536;
}
void kcc_as_ch(unsigned byte) {
    if (as_pos >= as_cap) {
        unsigned long ncap = as_cap << 1;
        as_cap = ncap;
        void *nb = kcc_arena_alloc(ncap);
        kcc_memcpy(nb, as_buf, as_pos);
        as_buf = nb;
    }
    ((unsigned char *)as_buf)[as_pos] = (unsigned
   char)byte;
    as_pos += 1;
}
void kcc_as_str(const char *s) {
    while (*s) { kcc_as_ch((unsigned char)*s); s
   ++; }
}
void kcc_as_putn(const char *p, unsigned long n) {
    while (n) { kcc_as_ch((unsigned char)*p); p++;
    n--; }
}
void kcc_as_dec(unsigned long n) {
    if (n == 0) { kcc_as_ch('0'); return; }
    char tmp[24];
    int i = 0;
    while (n) { tmp[i++] = '0' + (n % 10); n /=
   10; }
    while (i) kcc_as_ch((unsigned char)tmp[--i]);
}
void kcc_as_hex8(unsigned w) {
    int sh = 28;
    while (sh >= 0) { kcc_as_ch((unsigned char)
   s_hexdig[(w >> sh) & 0xf]); sh -= 4; }
}
void kcc_as_hex2(unsigned b) {
    b &= 0xff;
    kcc_as_ch((unsigned char)s_hexdig[(b >> 4) & 0
   xf]);
    kcc_as_ch((unsigned char)s_hexdig[b & 0xf]);
}
void kcc_as_xreg(unsigned n) {
    if (n == 31) { kcc_as_ch('x'); kcc_as_ch('z');
    kcc_as_ch('r'); return; }
    kcc_as_ch('x'); kcc_as_dec(n);
}
void kcc_as_wreg(unsigned n) {
    kcc_as_ch('w');
    if (n == 31) { kcc_as_ch('z'); kcc_as_ch('r');
    return; }
    kcc_as_dec(n);
}
```

□

748a · used in 750b

70.5. Rendering the Text Section

The renderer walks the code buffer word by word. It emits any symbol labels at each offset, splices inline-asm templates, renders relocated words symbolically (so the assembler re-creates the relocation) and internal branches by name (so distances are recomputed), and otherwise drops a raw `.long`. The trailing string-literal region is rendered one `.byte` at a time.

⟨ *Selfhost: emit_arm64 as-render* 748b ⟩ ≡

```
void kcc_as_symname(unsigned long idx) {
    char *data = *(char **)((char *)
   kcc_linker_syms + VEC_DATA);
    char *sym = data + idx * SYM_ENTRY_SIZE;
    kcc_as_str(*(char **)(sym + SYM_NAME));
}
void *kcc_as_reloc_at(unsigned long off) {
    void *vec = kcc_linker_relocs;
    unsigned long count = u64at(vec, VEC_COUNT);
    char *data = *(char **)((char *)vec + VEC_DATA
   );
    for (unsigned long i = 0; i < count; i++) {
        char *r = data + i * RELOC_SIZE;
        if (u64at(r, RELOC_OFFSET) == off) return
   r;
    }
    return 0;
}
void kcc_as_add_snip(unsigned long off, const char
    *tmpl, unsigned long len) {
    if (kcc_asm_snip_count >= kcc_asm_snip_cap) {
        unsigned long nc = (kcc_asm_snip_cap == 0)
    ? 16 : kcc_asm_snip_cap;
        nc <<= 1;
        char *na = kcc_arena_alloc(nc * 24);
        if (kcc_asm_snips) kcc_memcpy(na,
   kcc_asm_snips, kcc_asm_snip_count * 24);
        kcc_asm_snips = na;
        kcc_asm_snip_cap = nc;
    }
    char *e = (char *)kcc_asm_snips +
   kcc_asm_snip_count * 24;
    *(unsigned long *)(e + 0) = off;
    *(unsigned long *)(e + 8) = (unsigned long)
   tmpl;
    *(unsigned long *)(e + 16) = len;
    kcc_asm_snip_count += 1;
}
void *kcc_as_snip_at(unsigned long off) {
    if (!kcc_asm_snips) return 0;
    for (unsigned long i = 0; i <
   kcc_asm_snip_count; i++) {
        char *e = (char *)kcc_asm_snips + i * 24;
        if (u64at(e, 0) == off) return e;
    }
    return 0;
}
void kcc_as_labels_at(unsigned long off) {
    void *vec = kcc_linker_syms;
    unsigned long count = u64at(vec, VEC_COUNT);
    char *data = *(char **)((char *)vec + VEC_DATA
   );
    for (unsigned long i = 0; i < count; i++) {
        char *sym = data + i * SYM_ENTRY_SIZE;
        if (u64at(sym, SYM_SECTION) != 1) continue
   ;
        if (u64at(sym, SYM_VALUE) != off) continue
   ;
```

```
        if (u64at(sym, SYM_FLAGS) & N_EXT) {
            kcc_as_str(s_globl);
            kcc_as_str(*(char **)(sym + SYM_NAME))
    ;
            kcc_as_ch('\n');
        }
        kcc_as_str(*(char **)(sym + SYM_NAME));
        kcc_as_ch(':');
        kcc_as_ch('\n');
    }
}
void kcc_as_reloc_line(void *reloc, unsigned word)
    {
    unsigned long type = u64at(reloc, RELOC_TYPE);
    unsigned rd = word & 0x1f;
    unsigned rn = (word >> 5) & 0x1f;
    if (type == ARM64_RELOC_BRANCH26) {
        kcc_as_str(s_bl);
        kcc_as_symname(u64at(reloc, RELOC_SYM));
        kcc_as_ch('\n');
    } else if (type == ARM64_RELOC_PAGE21) {
        kcc_as_str(s_adrp); kcc_as_xreg(rd);
    kcc_as_ch(','); kcc_as_ch(' ');
        kcc_as_symname(u64at(reloc, RELOC_SYM));
    kcc_as_str(s_page); kcc_as_ch('\n');
    } else if (type == ARM64_RELOC_GOT_LOAD_PAGE21
    ) {
        kcc_as_str(s_adrp); kcc_as_xreg(rd);
    kcc_as_ch(','); kcc_as_ch(' ');
        kcc_as_symname(u64at(reloc, RELOC_SYM));
    kcc_as_str(s_gotpage); kcc_as_ch('\n');
    } else if (type == ARM64_RELOC_PAGEOFF12) {
        kcc_as_str(s_add); kcc_as_xreg(rd);
    kcc_as_ch(','); kcc_as_ch(' ');
        kcc_as_xreg(rn); kcc_as_ch(','); kcc_as_ch
    (' ');
        kcc_as_symname(u64at(reloc, RELOC_SYM));
    kcc_as_str(s_pageoff); kcc_as_ch('\n');
    } else if (type ==
    ARM64_RELOC_GOT_LOAD_PAGEOFF12) {
        kcc_as_str(s_ldr); kcc_as_xreg(rd);
    kcc_as_ch(','); kcc_as_ch(' ');
        kcc_as_ch('['); kcc_as_xreg(rn); kcc_as_ch
    (','); kcc_as_ch(' ');
        kcc_as_symname(u64at(reloc, RELOC_SYM));
    kcc_as_str(s_gotoff); kcc_as_ch(']');
        kcc_as_ch('\n');
    }
}
void *kcc_as_sym_at(unsigned long off) {
    void *vec = kcc_linker_syms;
    unsigned long count = u64at(vec, VEC_COUNT);
    char *data = *(char **)((char *)vec + VEC_DATA
    );
    for (unsigned long i = 0; i < count; i++) {
        char *sym = data + i * SYM_ENTRY_SIZE;
        if (u64at(sym, SYM_SECTION) != 1) continue
    ;
        if (u64at(sym, SYM_VALUE) != off) continue
    ;
        return *(void **)(sym + SYM_NAME);
    }
    return 0;
}
unsigned long kcc_as_branch_kind(unsigned word) {
    unsigned top6 = word >> 26;
    if (top6 == 0x05) return 1;                 /*
    b */
    if (top6 == 0x25) return 2;                 /*
    bl */
    unsigned top8 = (word >> 24) & 0xff;
    if (top8 == 0x54) { return (word & 0x10) ? 0 :
    3; }  /* b.cond */
    unsigned top7 = (word >> 24) & 0x7f;
    if (top7 == 0x34) return 4;                 /*
    cbz */
    if (top7 == 0x35) return 5;                 /*
    cbnz */
    return 0;
}
long kcc_as_branch_target(unsigned word, unsigned
    long off) {
    unsigned long kind = kcc_as_branch_kind(word);
    if (kind == 0) return -1;
    int imm;
    if (kind <= 2) { imm = (int)((word & 0
    x03FFFFFFu) << 6); imm >>= 6; }
    else { imm = (int)((((word >> 5) & 0x7FFFFu))
    << 13); imm >>= 13; }
    return (long)off + ((long)imm << 2);
}
void kcc_as_label_ref(unsigned long target) {
    void *name = kcc_as_sym_at(target);
    if (name) { kcc_as_str((char *)name); return;
    }
    kcc_as_str(s_lc);
    kcc_as_dec(target);
}
void kcc_as_render_branch(unsigned word, unsigned
    long off) {
    unsigned long kind = kcc_as_branch_kind(word);
    long target = kcc_as_branch_target(word, off);
    if (kind == 1) {
        kcc_as_str(s_b); kcc_as_label_ref(target);
    } else if (kind == 2) {
        kcc_as_str(s_bl); kcc_as_label_ref(target)
    ;
    } else if (kind == 3) {
        kcc_as_str(s_bdot);
        kcc_as_str(s_conds + (word & 0xf) * 3);
        kcc_as_ch('\t');
        kcc_as_label_ref(target);
    } else {                                    /*
    cbz (4) / cbnz (5) */
        kcc_as_str(kind == 4 ? s_cbz : s_cbnz);
        unsigned r = word & 0x1f;
        if (word & 0x80000000u) kcc_as_xreg(r);
    else kcc_as_wreg(r);
        kcc_as_ch(','); kcc_as_ch(' ');
        kcc_as_label_ref(target);
    }
    kcc_as_ch('\n');
}
void kcc_as_mark_targets(unsigned long code_size)
    {
    unsigned long words = code_size >> 2;
    unsigned char *flags = kcc_arena_alloc(words +
    1);
    kcc_as_targets = flags;
    for (unsigned long i = 0; i < words; i++)
    flags[i] = 0;
    for (unsigned long off = 0; off < code_size;
    off += 4) {
        if (kcc_as_reloc_at(off)) continue;
        unsigned word = *(unsigned *)((char *)
    kcc_code_buf + off);
        if (kcc_as_branch_kind(word) == 0)
    continue;
        long target = kcc_as_branch_target(word,
    off);
```

```
        if (target == -1) continue;
        if (kcc_as_sym_at((unsigned long)target))
    continue;
        ((unsigned char *)kcc_as_targets)[(
    unsigned long)target >> 2] = 1;
     }
}
void kcc_as_render_text(void) {
     kcc_as_str(s_sect);
     kcc_as_str(s_p2a);
     unsigned long total = kcc_code_pos;
     unsigned long code_size = total - kcc_cstr_pos
    ;
     kcc_as_mark_targets(code_size);
     unsigned long off = 0;
     for (; off < code_size; off += 4) {
         kcc_as_labels_at(off);
         if (kcc_as_targets && ((unsigned char *)
    kcc_as_targets)[off >> 2]) {
             kcc_as_str(s_lc); kcc_as_dec(off);
    kcc_as_ch(':'); kcc_as_ch('\n');
         }
         void *snip = kcc_as_snip_at(off);
         if (snip) {
             kcc_as_ch('\t');
             kcc_as_putn(*(char **)((char *)snip +
    8), u64at(snip, 16));
             kcc_as_ch('\n');
             continue;
         }
         unsigned word = *(unsigned *)((char *)
    kcc_code_buf + off);
         void *reloc = kcc_as_reloc_at(off);
         if (reloc) { kcc_as_reloc_line(reloc, word
    ); continue; }
         if (kcc_as_branch_kind(word) != 0) {
    kcc_as_render_branch(word, off); continue; }
         kcc_as_str(s_long); kcc_as_hex8(word);
    kcc_as_ch('\n');
     }
     for (; off < total; off += 1) {
         kcc_as_labels_at(off);
         kcc_as_str(s_byte);
         kcc_as_hex2(*(unsigned char *)((char *)
    kcc_code_buf + off));
         kcc_as_ch('\n');
     }
}
```

□

748b · defined at 748b, 750a · used in 750b

70.6. Globals, Module Init, and File Output

Zero-storage globals become `.zerofill` entries; a synthesised module-init function is registered in `__mod_init_func`; and the accumulated text is written to the output path in a single `write`.

⟨ *Selfhost: emit_arm64 as-render* 750a ⟩ + ≡

```
unsigned long kcc_as_common_size(unsigned long off
    ) {
     unsigned long best = kcc_globals_size;
     void *vec = kcc_linker_syms;
     unsigned long count = u64at(vec, VEC_COUNT);
     char *data = *(char **)((char *)vec + VEC_DATA
    );
     for (unsigned long i = 0; i < count; i++) {
         char *sym = data + i * SYM_ENTRY_SIZE;
         if (u64at(sym, SYM_SECTION) != 2) continue
    ;
         unsigned long v = u64at(sym, SYM_VALUE);
         if (v <= off) continue;                 /*
     not a successor */
         if (v >= best) continue;                /*
     not smaller than best */
         best = v;
     }
     return best - off;
}
void kcc_as_render_common(void) {
     void *vec = kcc_linker_syms;
     unsigned long count = u64at(vec, VEC_COUNT);
     char *data = *(char **)((char *)vec + VEC_DATA
    );
     for (unsigned long i = 0; i < count; i++) {
         char *sym = data + i * SYM_ENTRY_SIZE;
         if (u64at(sym, SYM_SECTION) != 2) continue
    ;
         if (u64at(sym, SYM_FLAGS) & N_EXT) {
             kcc_as_str(s_globl);
             kcc_as_str(*(char **)(sym + SYM_NAME))
    ;
             kcc_as_ch('\n');
         }
         kcc_as_str(s_zerofill);
         kcc_as_str(*(char **)(sym + SYM_NAME));
         kcc_as_ch(',');
         kcc_as_dec(kcc_as_common_size(u64at(sym,
    SYM_VALUE)));
         kcc_as_str(s_a3nl);
     }
}
void kcc_as_render_modinit(void) {
     unsigned long m = kcc_modinit_sym;          /*
     symbol index + 1 (0 = none) */
     if (m == 0) return;
     kcc_as_str(s_modsect);
     kcc_as_symname(m - 1);
     kcc_as_ch('\n');
}
int kcc_as_write_file(const char *path) {
     long fd = kcc_sys_open(path, 0x601, 420);   /*
     O_WRONLY|O_CREAT|O_TRUNC, 0644 */
     if (fd < 0) return 1;
     kcc_sys_write(fd, as_buf, as_pos);
     kcc_sys_close(fd);
     return 0;
}
int kcc_as_render(const char *path) {
     kcc_as_init();
     kcc_as_render_text();
     kcc_as_render_common();
     kcc_as_render_modinit();
     return kcc_as_write_file(path);
}
```

□

750a · defined at 748b, 750a · used in 750b

70.7. Assembling the Translation Unit

⟨ *Selfhost: emit_arm64* 750b ⟩ ≡

⟨ *Selfhost: emit_arm64 preamble* 745a ⟩

⟨ *Selfhost: emit_arm64 encoders* 745b ⟩
⟨ *Selfhost: emit_arm64 codebuf* 747 ⟩
⟨ *Selfhost: emit_arm64 as-buffer* 748a ⟩
⟨ *Selfhost: emit_arm64 as-render* 748b, … ⟩
□

750b · used in 36f

70.8. Verification

`make selfhost-emit-arm64` exercises both halves. `difftest` byte-compares the direct-encoder object against clang across the whole corpus, covering every instruction encoder and the code buffer. For the textual renderer, a probe rich in branches, calls, and global references is built *through* `--emit-asm` (so the C renderer emits `.s` that the platform assembler turns into an object) and run: its exit status must match the direct-encoder reference—the assembled program behaves identically, which is exactly what the renderer must guarantee. (A byte-compare of the object is not usable: the platform assembler embeds nondeterministic bytes, so even two runs of the same compiler differ.) The code-generation suite runs last as a smoke test.

⟨ *Selfhost: emit_arm64 verify* 751 ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-emit-arm64 - compile kcc's ARM64
   encoder + .s renderer in C with
# kcc, relink, and verify both halves (difftest +
   --emit-asm byte-compare).
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/5] compiling $BASE/selfhost/emit_arm64.c
   with kcc"
$BASE/kcc -c "$SH/emit_arm64.c" -o "$SH/
   emit_arm64_c.o" >/dev/null

echo "[2/5] linking $BASE/kcc_selfhost_emit_arm64
   (emit_arm64.o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "emit_arm64" ]; then OBJ="$OBJ $SH
   /emit_arm64_c.o"; else OBJ="$OBJ $BASE/src/${f
   }.o"; fi
done
ld -o $BASE/kcc_selfhost_emit_arm64 $OBJ -lSystem
   -syslibroot "$SDK" -e _main -arch arm64

cp $BASE/kcc "$SH/kcc_asm_backup_emit_arm64"

echo "[3/5] --emit-asm functional check: C
   renderer -> as -> runnable program"
cat > "$SH/emit_probe.c" <<'PROBE'
int g;
static int add(int a, int b) { return a + b; }
int fib(int n) { if (n < 2) return n; return fib(n
    - 1) + fib(n - 2); }
int main(void) {
    g = 7; int s = 0;
    for (int i = 0; i < 5; i++) s += add(i, g);
    return (s + fib(6)) % 100;
}
PROBE
# Ground truth: the direct-encoder object, built
   by the assembly compiler.
$BASE/kcc "$SH/emit_probe.c" -o "$SH/
   emit_probe_ref" >/dev/null
set +e; "$SH/emit_probe_ref"; ref=$?; set -e
# The C renderer, driving --emit-asm through the
   platform assembler.
cp $BASE/kcc_selfhost_emit_arm64 $BASE/kcc
$BASE/kcc --emit-asm "$SH/emit_probe.c" -o "$SH/
   emit_probe_c" >/dev/null
set +e; "$SH/emit_probe_c"; got=$?; set -e
if [ "$ref" != "$got" ]; then
    cp "$SH/kcc_asm_backup_emit_arm64" $BASE/kcc
    echo "FAIL: --emit-asm program (C renderer)
   exit $got != direct-encoder $ref"; exit 1
fi
echo "  --emit-asm program runs identically (exit
   $got)"

echo "[4/5] difftest (byte-compares every encoder
   against clang)"
set +e
bash $BASE/tests/difftest.sh >/$BASE/
   emit_arm64_difftest.log 2>&1
dstatus=$?
tail -1 /$BASE/emit_arm64_difftest.log
set -e
if [ $dstatus -ne 0 ]; then cp "$SH/
   kcc_asm_backup_emit_arm64" $BASE/kcc; exit
   $dstatus; fi

echo "[5/5] running the codegen suite with the
   self-hosted encoder"
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_emit_arm64" $BASE/kcc
exit $status
```

□

751 · used in 36g · uses _main 621, 629

71. Self-Hosting: The Semantic Analyzer in C

The semantic analyzer of Chapters 26–26 (the spine plus its declaration, statement, and expression chapters) is the compiler's semantic core: it walks the AST, resolves every parser type node into a canonical type descriptor, populates the symbol table, lays out aggregates byte-accurately, annotates each expression with its resolved type, and raises the `-Wall` family of diagnostics. Porting it completes the front end and the entire semantic layer in kcc-compiled C.

The port is verified by the strongest oracle available: `difftest` byte-compares kcc's object output against clang across the whole corpus, and every one of those cases runs through semantic analysis—any deviation in type resolution, aggregate layout, or the enum-constant rewrite changes the generated code and fails the comparison. The diagnostic and warning suites (`diagtest`, `warntest`) additionally pin the exact message text.

71.1. Nodes, Types, Symbols, and Constants

The port declares the 64-byte AST node, the 48-byte type descriptor, and the symbol/hash-table field offsets, together with the AST-tag, type-kind, token, and symbol-kind enumerations the analyzer dispatches on. The type singletons (`kcc_type_int_s` and kin) are globals holding descriptor pointers; `eval_const_int`, `expr_is_ice`, and `ir_initlist_count` live in the IR builder and are declared extern.

⟨ *Selfhost: sema preamble* 753 ⟩ ≡

```
struct Node { unsigned long tag, line, col, f0, f1
   , f2, f3, f4; };
struct Type { unsigned long kind, size, align,
   quals, f0, f1; };

#define TY_DESC_SIZE 48
/* type descriptor field offsets are the struct
   members above */
/* symbol entry (semantic symtab) */
#define SYM_KIND   0
#define SYM_TYPE   8
#define SYM_NAME   16
#define SYM_VALUE  32
#define SYM_DEPTH  40
#define SYM_LINE   48
#define SYM_FLAGS  56
#define SYM_COL    64
/* hash table */
#define HT_ENTRIES 0
#define HT_CAP     8
#define HTE_KEY    8
#define HTE_VALUE  24
#define HTE_SIZE   32

/* type kinds */
enum { TY_VOID=1, TY_BOOL=2, TY_CHAR=3, TY_LLONG
   =7, TY_PTR=11, TY_ARRAY=12,
       TY_FUNC=13, TY_STRUCT=14, TY_UNION=15,
   TY_ENUM=16, TY_INT128=17 };
/* symbol kinds */
enum { SYM_VAR=1, SYM_FUNC=2, SYM_TYPEDEF=3,
   SYM_ENUM_CONST=4 };

/* AST tags */
enum {
    AST_TRANS_UNIT=1, AST_VAR_DECL=10,
   AST_FUNC_DECL=11, AST_PARAM_DECL=12,
    AST_TYPEDEF_DECL=13, AST_STRUCT_DECL=14,
   AST_UNION_DECL=15, AST_ENUM_DECL=16,
    AST_ENUMERATOR=17, AST_FIELD_DECL=18,
   AST_STATIC_ASSERT=19,
    AST_TYPE_BUILTIN=20, AST_TYPE_NAME=21,
   AST_TYPE_PTR=22, AST_TYPE_ARRAY=23,
    AST_TYPE_FUNC=24, AST_TYPE_STRUCT=25,
   AST_TYPE_UNION=26, AST_TYPE_ENUM=27,
    AST_TYPE_QUALS=28, AST_TYPE_TYPEOF=29,
    AST_COMPOUND=30, AST_EXPR_STMT=31, AST_IF_STMT
   =32, AST_SWITCH_STMT=33,
    AST_WHILE_STMT=34, AST_DO_STMT=35,
   AST_FOR_STMT=36, AST_GOTO_STMT=37,
    AST_CONTINUE_STMT=38, AST_BREAK_STMT=39,
   AST_RETURN_STMT=40,
    AST_LABEL_STMT=41, AST_CASE_STMT=42,
   AST_DEFAULT_STMT=43, AST_NULL_STMT=44,
    AST_IDENT_EXPR=50, AST_INT_LIT=51, AST_STR_LIT
   =52, AST_CHAR_LIT=53,
    AST_BINARY_EXPR=54, AST_UNARY_EXPR=55,
   AST_ASSIGN_EXPR=56, AST_COND_EXPR=57,
    AST_CALL_EXPR=58, AST_SUBSCRIPT=59, AST_MEMBER
   =60, AST_CAST_EXPR=61,
    AST_SIZEOF_EXPR=62, AST_ALIGNOF_EXPR=63,
   AST_COMMA_EXPR=64, AST_INIT_LIST=65,
    AST_DESIGNATOR=66, AST_COMPOUND_LIT=67,
   AST_PAREN_EXPR=68, AST_GENERIC=69,
    AST_FLOAT_LIT=71, AST_VA_ARG=72, AST_STMT_EXPR
   =73, AST_ASM_STMT=74,
    AST_TYPES_COMPAT=76, AST_LABEL_ADDR=77
};

/* tokens (operators) */
enum { TOK_INC=70, TOK_DEC=71, TOK_AMP=72,
   TOK_STAR=73, TOK_PLUS=74,
       TOK_MINUS=75, TOK_TILDE=76, TOK_BANG=77,
   TOK_SHL=80, TOK_SHR=81,
       TOK_LT=82, TOK_GT=83, TOK_LE=84, TOK_GE=85,
    TOK_EQ=86, TOK_NE=87,
       TOK_AND=90, TOK_OR=91, TOK_ASSIGN=94 };

/* externs - type system */
void *kcc_type_ptr(void *pointee);
void *kcc_type_array(void *elem, unsigned long len
   );
void *kcc_type_func(void *ret, void *params);
void *kcc_type_struct(void *decl);
void *kcc_type_union(void *decl);
void *kcc_type_enum(void *decl);
void *kcc_type_from_specmask(unsigned long mask);
void *kcc_type_param_adjust(void *ty);
int kcc_type_compatible(void *a, void *b);
void kcc_type_dump(void *ty);
/* conversions */
void *kcc_conv_func_decay(void *ty);
void *kcc_conv_array_decay(void *ty);
void *kcc_conv_usual(void *a, void *b);
void *kcc_conv_promote(void *ty);
/* symbol table */
void *kcc_symtab_lookup(const char *name, unsigned
    long len);
```

```
void *kcc_symtab_lookup_tag(const char *name,
    unsigned long len);
void *kcc_symtab_lookup_current(const char *name,
    unsigned long len);
void *kcc_symtab_declare(const char *name,
    unsigned long len, unsigned long kind, void *
    type);
void kcc_symtab_declare_tag(const char *name,
    unsigned long len, void *type);
void kcc_symtab_push_scope(void);
void kcc_symtab_pop_scope(void);
void *kcc_symtab_current_ht(void);
void kcc_symtab_label_add(const char *name,
    unsigned long len);
void kcc_symtab_label_begin(void);
void kcc_symtab_label_end(void);
/* diagnostics + misc */
void kcc_diag_error(unsigned long line, unsigned
    long col, const char *msg);
void kcc_diag_warning(unsigned long line, unsigned
     long col, const char *msg);
void *kcc_arena_alloc(unsigned long n);
void *kcc_memcpy(void *d, const void *s, unsigned
    long n);
unsigned long kcc_strlen(const char *s);
int kcc_strcmp(const char *a, const char *b);
void kcc_io_puts(const char *s);
void kcc_io_flush(void);
void *kcc_ast_alloc(unsigned long tag);
unsigned long kcc_eval_const_int(void *node);
int kcc_expr_is_ice(void *node);
unsigned long kcc_ir_initlist_count(void *node);
/* extern globals */
extern void *kcc_type_int_s, *kcc_type_int_u, *
    kcc_type_long_s, *kcc_type_long_u;
extern void *kcc_type_llong_s, *kcc_type_llong_u,
    *kcc_type_char_s;
extern void *kcc_type_void, *kcc_type_double, *
    kcc_type_float;
extern unsigned long kcc_warn_all;
extern unsigned long kcc_sym_depth;

#define SL(s,off) (*(unsigned long *)((char *)(s)
    + (off)))

/* module state */
static void *sema_cur_ret = 0;
static unsigned long sema_callee_ctx = 0;
static unsigned long sema_callee_implicit = 0;

/* diagnostic message strings */
static const char m_undecl[]   = "use of
    undeclared identifier '";
static const char m_nomember[] = "no member named
    '";
static const char m_redef[]    = "redefinition of
    '";
static const char m_implicit[] = "implicit
    declaration of function '";
static const char m_unused[]   = "unused variable
    '";
static const char m_not_func[] = "called object is
     not a function";
static const char m_not_struct[] = "member access
    on non-struct/union type";
static const char m_ret_void[]   = "void function
    should not return a value";
static const char m_ret_nonvoid[]= "non-void
    function should return a value";
static const char m_reach_end[]  = "control
    reaches end of non-void function";
static const char m_sign_cmp[]   = "comparison of
    integers of different signedness";
static const char m_int2ptr[]    = "incompatible
    integer to pointer conversion";
static const char m_ptr2int[]    = "incompatible
    pointer to integer conversion";
static const char m_vla[]        = "variable-
    length arrays are not supported";
static const char m_int128[]     = "128-bit
    integer arithmetic (__int128) is not yet
    supported";
static const char m_generic[]    = "_Generic: no
    association compatible with the controlling
    type";
static const char m_main[]       = "main";
static const char m_va_list[]    = "va_list";
static const char m_builtin_va_list[] = "
    __builtin_va_list";
static const char *sema_noreturn_tbl[] = {
    "exit", "_exit", "_Exit", "abort", "longjmp",
    "__builtin_unreachable", "__builtin_trap", 0
};

/* forward declarations */
void *kcc_sema_resolve_type(void *node);
void *kcc_sema_expr(void *node);
void kcc_sema_check_stmt(void *node);
void kcc_sema_check_decl(void *node);
unsigned long kcc_type_align(void *ty);
unsigned long kcc_type_layout(void *ty);
void kcc_sema_err_named(unsigned long line,
    unsigned long col, const char *name, const char
     *prefix);
void kcc_sema_warn_named(unsigned long line,
    unsigned long col, const char *name, const char
     *prefix);
void kcc_sema_check_unused_scope(void);
void kcc_sema_check_redef(void *decl, const char *
    name);
unsigned long kcc_agg_leaf_count(void *ty);
unsigned long kcc_infer_arr_elems(void *head, void
     *elem);
void kcc_sema_infer_array_len(void *decl);
void kcc_sema_check_return_path(void *func);
long kcc_sema_int_signedness(void *t);
void kcc_sema_check_sign_compare(void *expr, void
    *lt, void *rt);
void kcc_sema_check_int_ptr(void *loc, void *dst,
    void *src, void *srcexpr);
struct FindResult { void *field; unsigned long off
    ; };
struct FindResult kcc_type_find_member(void *sty,
    const char *name);
static void *sema_expr_generic(struct Node *n);
static void *sema_expr_ident(struct Node *n);
static void *sema_expr_literal(struct Node *n,
    unsigned long tag);
static void *sema_expr_binary(struct Node *n);
static void *sema_expr_unary(struct Node *n);
static void *sema_expr_assign(struct Node *n);
static void *sema_expr_call(struct Node *n);
static void *sema_expr_member(struct Node *n);
static void *sema_expr_cast(struct Node *n);
static void *sema_expr_sizeof(struct Node *n);
```

□

753 · used in 768a

71.2. Entry Point and Type Resolution

kcc_sema_check walks the translation unit's declaration list. kcc_sema_resolve_type is the dispatcher that turns a parser type node into a canonical descriptor: builtins from the specifier bitmask; pointer, array, and function types by recursion and construction (an array folds its length as an integer constant expression, rejecting VLAs; a variadic function records its fixed count in TY_QUALS); struct/union bodies reused when the tag already names this same definition; typedef names by symbol lookup (with va_list and its __builtin_va_list spelling recognised as built-in aliases for the opaque pointer); and a volatile qualifier by cloning the descriptor with the volatile bit set.

⟨ *Selfhost: sema resolve* 755 ⟩ ≡

```
void kcc_sema_check(void *tu) {
    struct Node *d = (struct Node *)((struct Node
  *)tu)->f0;
    while (d) { kcc_sema_check_decl(d); d = (
  struct Node *)d->f4; }
}

void *kcc_sema_resolve_type(void *node) {
    if (!node) return kcc_type_int_s;
    struct Node *n = (struct Node *)node;
    unsigned long tag = n->tag;

    if (tag == AST_TYPE_TYPEOF) {
        if (n->f3) return (void *)n->f3;        /*
  memoised */
        void *operand = (void *)n->f0;
        if (!operand) return kcc_type_int_s;
        void *r;
        if (n->f1) r = kcc_sema_resolve_type(
  operand);  /* type-name */
        else r = kcc_sema_expr(operand);
         /* expression */
        if (!r) return kcc_type_int_s;
        n->f3 = (unsigned long)r;
        return r;
    }
    if (tag == AST_TYPE_BUILTIN) {
        void *r = kcc_type_from_specmask(n->f0);
        return r ? r : kcc_type_int_s;
    }
    if (tag == AST_TYPE_PTR) {
        return kcc_type_ptr(kcc_sema_resolve_type
  ((void *)n->f0));
    }
    if (tag == AST_TYPE_ARRAY) {
        void *elem = kcc_sema_resolve_type((void
  *)n->f0);
        unsigned long len;
        void *size = (void *)n->f1;
        if (!size) len = 0;                     /*
  incomplete */
        else if (kcc_expr_is_ice(size)) len =
  kcc_eval_const_int(size);
        else {
            kcc_diag_error(n->line, n->col, m_vla)
  ;
            len = 0;
        }
        return kcc_type_array(elem, len);
    }
    if (tag == AST_TYPE_FUNC) {
        void *ret = kcc_sema_resolve_type((void *)
  n->f0);
        void *params = (void *)(n->f1 & ~1UL);
        struct Type *ft = (struct Type *)
  kcc_type_func(ret, params);
        if (n->f2) {                            /*
  variadic: record nfixed */
            unsigned long nfixed = 0;
            struct Node *p = (struct Node *)(n->f1
   & ~1UL);
            while (p) { nfixed++; p = (struct Node
   *)p->f4; }
            ft->quals = nfixed;
        }
        return ft;
    }
    if (tag == AST_TYPE_STRUCT || tag ==
  AST_TYPE_UNION) {
        struct Node *decl = (struct Node *)n->f0;
        if (!decl) return kcc_type_int_s;
        if (!decl->f1) {                        /*
  no body → forward-ref lookup */
            void *name = (void *)decl->f0;
            if (!name) return (tag ==
  AST_TYPE_STRUCT) ? kcc_type_struct(decl)

        : kcc_type_union(decl);
            void *sym = kcc_symtab_lookup_tag(name
  , kcc_strlen(name));
            if (sym) return (void *)SL(sym,
  SYM_TYPE);
            /* An as-yet-undeclared struct/union
  tag names an INCOMPLETE type
               (\stdref{6.7.2.3}), not int:
  register it now so a pointer to it is a
               real struct pointer and a later `
  struct T { ... };' completes this
               same descriptor in place.  The
  kernel needs this for a forward
               reference through a function's
  return type---`skb_dst()' returns
               `struct dst_entry *' whose
  definition follows the inline, so
               `skb_dst(skb)->input' must still
  see a struct. */
            void *desc = (tag == AST_TYPE_STRUCT)
  ? kcc_type_struct(decl)

  : kcc_type_union(decl);
            kcc_symtab_declare_tag(name,
  kcc_strlen(name), desc);
            return desc;
        }
        /* body present: reuse the registered
  descriptor iff same definition */
        void *name = (void *)decl->f0;
        if (name) {
            void *sym = kcc_symtab_lookup_tag(name
  , kcc_strlen(name));
            if (sym) {
                struct Type *desc = (struct Type
  *)SL(sym, SYM_TYPE);
                if (desc && (void *)desc->f0 ==
  decl) return desc;
            }
        }
        return (tag == AST_TYPE_STRUCT) ?
  kcc_type_struct(decl)
                                        :
  kcc_type_union(decl);
    }
```

```
    if (tag == AST_TYPE_ENUM) {
        return kcc_type_enum((void *)n->f0);
    }
    if (tag == AST_TYPE_NAME) {
        void *name = (void *)n->f0;
        if (!name) return kcc_type_int_s;
        void *sym = kcc_symtab_lookup(name,
  kcc_strlen(name));
        if (sym) return (void *)SL(sym, SYM_TYPE);
        /* built-in typedef va_list (and its
  __builtin_va_list spelling) → pointer */
        if (kcc_strcmp(name, m_va_list) == 0)
  return kcc_type_ptr(kcc_type_int_s);
        if (kcc_strcmp(name, m_builtin_va_list) ==
   0) return kcc_type_ptr(kcc_type_int_s);
        return kcc_type_int_s;
    }
    if (tag == AST_TYPE_QUALS) {
        struct Type *base = (struct Type *)
  kcc_sema_resolve_type((void *)n->f0);
        if (!(n->f1 & 2)) return base;          /*
  only volatile is honoured */
        struct Type *nt = (struct Type *)
  kcc_arena_alloc(TY_DESC_SIZE);
        *nt = *base;
        nt->quals |= 2;
        return nt;
    }
    return kcc_type_int_s;                      /*
    unknown / fallback */
}
```

□

755 · used in 768a

71.3. Statement Checking

The statement checker dispatches on tag, pushing a scope for compound and `for` bodies (and running the unused-variable check before each pop), type-checking conditions and recursing into sub-statements, enforcing the `return`-value constraints of §6.8.6.4, and forwarding declaration nodes to the declaration checker.

⟨ *Selfhost: sema statements* 756 ⟩ ≡

```
void kcc_sema_check_stmt(void *node) {
    if (!node) return;
    struct Node *s = (struct Node *)node;
    unsigned long tag = s->tag;
    switch (tag) {
    case AST_COMPOUND: {
        kcc_symtab_push_scope();
        struct Node *it = (struct Node *)s->f0;
        while (it) { kcc_sema_check_stmt(it); it =
    (struct Node *)it->f4; }
        kcc_sema_check_unused_scope();
        kcc_symtab_pop_scope();
        return;
    }
    case AST_EXPR_STMT:
        if (s->f0) kcc_sema_expr((void *)s->f0);
        return;
    case AST_IF_STMT:
        kcc_sema_expr((void *)s->f0);
        kcc_sema_check_stmt((void *)s->f1);
        if (s->f2) kcc_sema_check_stmt((void *)s->
  f2);
        return;
    case AST_WHILE_STMT:
        kcc_sema_expr((void *)s->f0);
        kcc_sema_check_stmt((void *)s->f1);
        return;
    case AST_DO_STMT:
        kcc_sema_check_stmt((void *)s->f0);
        kcc_sema_expr((void *)s->f1);
        return;
    case AST_FOR_STMT:
        kcc_symtab_push_scope();
        /* A for-init declaration may declare
several objects sharing one type
           (`for (T a, *b = ...; ...)`, as the
kernel's scoped_guard plants via
           `CLASS(...)(args), *done = NULL');
parse_decl chains the extra
           declarators on f4 exactly as a block
does, so walk that chain --- the
           head node alone would leave the
siblings undeclared. */
        for (struct Node *it = (struct Node *)s->
f0; it; it = (struct Node *)it->f4)
            kcc_sema_check_stmt(it);
        if (s->f1) kcc_sema_expr((void *)s->f1);
        if (s->f2) kcc_sema_expr((void *)s->f2);
        if (s->f3) kcc_sema_check_stmt((void *)s->
f3);
        kcc_sema_check_unused_scope();
        kcc_symtab_pop_scope();
        return;
    case AST_RETURN_STMT:
        if (s->f0) {
            void *st = kcc_sema_expr((void *)s->f0
);
            kcc_sema_check_int_ptr(s, sema_cur_ret
, st, (void *)s->f0);
            /* `return <void-expr>;` from a void
function is legal C
               (\S6.8.6.4): only a value-typed
expression is the error. */
            if (sema_cur_ret && SL(sema_cur_ret,
0) == TY_VOID &&
                !(st && SL(st, 0) == TY_VOID))
                kcc_diag_error(s->line, s->col,
m_ret_void);
        } else {
            if (sema_cur_ret && SL(sema_cur_ret,
0) != TY_VOID)
                kcc_diag_error(s->line, s->col,
m_ret_nonvoid);
        }
        return;
    case AST_SWITCH_STMT:
        kcc_sema_expr((void *)s->f0);
        kcc_sema_check_stmt((void *)s->f1);
        return;
    case AST_LABEL_STMT:
        if (s->f0) kcc_symtab_label_add((void *)s
->f0, kcc_strlen((void *)s->f0));
        if (s->f1) kcc_sema_check_stmt((void *)s->
f1);
        return;
    case AST_CASE_STMT:
        kcc_sema_expr((void *)s->f0);
        if (s->f3) kcc_sema_expr((void *)s->f3);
    /* GNU range high bound */
        if (s->f1) kcc_sema_check_stmt((void *)s->
f1);
        return;
    case AST_DEFAULT_STMT:
        if (s->f0) kcc_sema_check_stmt((void *)s->
f0);
```

```
        return;
    case AST_ASM_STMT: {
        struct Node *op = (struct Node *)s->f2;
        while (op) {
            if (op->f2) kcc_sema_expr((void *)op->
f2);
            op = (struct Node *)op->f4;
        }
        return;
    }
    case AST_GOTO_STMT:
        /* A direct goto carries a label name in
f0; a computed `goto *expr'
           (f1 set) carries a target expression
that must be type-checked. */
        if (s->f1 && s->f0) kcc_sema_expr((void *)
s->f0);
        return;
    case AST_VAR_DECL: case AST_TYPEDEF_DECL: case
 AST_STRUCT_DECL:
    case AST_UNION_DECL: case AST_ENUM_DECL: case
AST_TYPE_ENUM:
    case AST_TYPE_STRUCT: case AST_TYPE_UNION:
        kcc_sema_check_decl(s);
        return;
    default:
        return;                                 /*
break/continue/null/static_assert */
    }
}
```

□

756 · used in 768a

71.4. Declaration Checking and Aggregate Layout

The declaration checker registers variables, functions, typedefs, and tags, pre-registering any struct/union/enum a declared type introduces (peeling pointer, array, and qualifier wrappers to reach a bodied definition), inferring incomplete-array lengths, and completing a forward-declared tag in place when a definition arrives at the same scope depth. `kcc_type_layout` computes byte-accurate size, alignment, and per-field offsets, packing consecutive same-type bit-fields into a container from the least-significant bit up.

Laying out a struct is *destructive*: each field's `f2` slot, which the parser fills with the bit-field width *expression*, is overwritten with the packed `(bit-offset<<16)|width` descriptor the IR later reads — and a zero-width bit-field's `f2` is cleared to nothing. Layout must therefore run exactly once per tag. The obvious memo guard, `if (ty->size) return`, fails for a *genuinely zero-sized* struct: an empty struct, or one whose only members are zero-width bit-fields, has size 0, indistinguishable from "not yet laid out". A second layout then sees the already-cleared `f2`, misreads the `:0` as a plain member, and reserves a whole word — so `sizeof(struct { int : 0; })` comes out 4 instead of 0. That single miscount breaks the kernel's `BUILD_BUG_ON_ZERO` (hence `ARRAY_SIZE`): `ARRAY_SIZE(initcall_levels)` inflates from 9 to 13, `do_initcalls` walks four levels past the table into the adjacent `initcall_level_names[]`, and calls a name string as an initcall — a wild jump early in boot. We memoise on `ty->align` instead: a completed layout always stamps a nonzero alignment (at least the alignment of some member) even when the size is 0, so the guard fires on the second call and the field descriptors are never re-derived.

⟨ *Selfhost: sema layout* 757 ⟩ ≡

```
unsigned long kcc_type_align(void *ty_) {
    struct Type *ty = (struct Type *)ty_;
    if (!ty) return 8;
    if (ty->kind == TY_ARRAY) return
   kcc_type_align((void *)ty->f0);
    return ty->align ? ty->align : 8;
}

unsigned long kcc_type_layout(void *ty_) {
    struct Type *ty = (struct Type *)ty_;
    if (!ty) return 0;
    unsigned long kind = ty->kind;
    if (kind == TY_ARRAY) {
        struct Type *elem = (struct Type *)ty->f0;
        unsigned long len = ty->f1;
        unsigned long esz = kcc_type_layout(elem);
        ty->size = esz * len;
        ty->align = kcc_type_align(elem);
        return ty->size;
    }
    if (kind != TY_STRUCT && kind != TY_UNION)
   return ty->size;
    if (ty->align) return ty->size;             /*
   memoised: align is stamped

   after a completed layout even

   when size==0 (empty struct /

   only zero-width bit-fields);

   size==0 alone can't say

   "laid out" and re-running

   would re-derive the now

   destructively-packed field

   descriptors */
    int is_union = (kind == TY_UNION);
    struct Node *decl = (struct Node *)ty->f0;
    if (!decl) return 0;
    unsigned long layout_word = decl->f3;       /*
   (aligned<<1)|packed */
    struct Node *field = (struct Node *)decl->f1;
    if (!field) {
        /* No members: an opaque (forward-declared
   ) tag, or a genuinely empty
           struct.  An empty struct is size 0, but
    an `__attribute__((aligned(N)))'
           on it still sets its alignment --- and
   that alignment must propagate to
           where the struct is used as a member,
   or the member gets no padding.
           The kernel's `struct sched_statistics'
   is empty under !CONFIG_SCHEDSTATS
           yet `aligned(64)'; without honouring it
    here, `task_struct.stats' lost
           its 32 bytes of leading pad, shifting
   every later field's offset and
           faulting code that reads e.g. `current
   ->signal' at the wrong offset. */
        unsigned long req = decl->f3 >> 1;      /*
    aligned(N) */
        if (req > ty->align) ty->align = req;
```

```
    return ty->size;                        /*
 stays 0 */
 }
 unsigned long cursor = 0, maxalign = 1,
maxsize = 0;
 unsigned long cont_off = 0, cont_bit = 0,
cont_size = 0;
 while (field) {
    struct Type *ftype = 0;
    unsigned long fsize, falign;
    void *fts = (void *)field->f0;
    if (fts) ftype = (struct Type *)
kcc_sema_resolve_type(fts);
    if (ftype) { fsize = kcc_type_layout(ftype
); falign = kcc_type_align(ftype); }
    else { fsize = 8; falign = 8; }
    if (layout_word & 1) falign = 1;        /*
 packed */
    struct Node *width_expr = (struct Node *)
field->f2;
    if (width_expr) {
        /* bit-field: unwrap parens, fold
width */
        struct Node *we = width_expr;
        unsigned long wtag = we->tag;
        while (wtag == AST_PAREN_EXPR) {
            we = (struct Node *)we->f0;
            if (!we) break;
            wtag = we->tag;
        }
        unsigned long width;
        if (we && (wtag == AST_INT_LIT || wtag
 == AST_CHAR_LIT)) width = we->f0;
        else width = 0;
        if (is_union) {
            field->f3 = 0;
            field->f2 = width & 0xffff;
            if (fsize > maxsize) maxsize =
fsize;
        } else if (width == 0) {
            /* A zero-width bit-field (`
unsigned :0;') packs no bits but closes
               the current unit, so the next
bit-field starts in a fresh
               container (\stdref{6.7.2.1}).
The kernel's task_struct scheduler
               flags use it to split two bit-
field groups; without honouring it
               kcc coalesced them into one
word, shrinking the struct by a word
               and skewing every later field's
 offset. */
            cont_size = 0;
            field->f3 = cursor;
            field->f2 = 0;
        } else {
            unsigned long cont_bits = fsize <<
 3;
            if (cont_size == fsize && (
cont_bit + width) <= cont_bits) {
                /* reuse the active container
*/
            } else {
                cursor = (cursor + falign - 1)
 & ~(falign - 1);
                cont_off = cursor;
                cont_size = fsize;
                cont_bit = 0;
                cursor += fsize;
            }
            field->f3 = cont_off;
            field->f2 = (cont_bit << 16) |
width;
            cont_bit += width;
        }
    } else {
        cont_size = 0;                      /*
 plain member ends the run */
        if (is_union) {
            field->f3 = 0;
            if (fsize > maxsize) maxsize =
fsize;
        } else {
            cursor = (cursor + falign - 1) &
~(falign - 1);
            field->f3 = cursor;
            cursor += fsize;
        }
    }
    if (falign > maxalign) maxalign = falign;
    field = (struct Node *)field->f4;
 }
 if (is_union) cursor = maxsize;
 unsigned long req = layout_word >> 1;      /*
 aligned(N) */
 if (req > maxalign) maxalign = req;
 cursor = (cursor + maxalign - 1) & ~(maxalign
- 1);
 ty->size = cursor;
 ty->align = maxalign;
 return cursor;
}
```

□

757 · used in 768a

The declaration dispatcher. Both a variable declaration and a function definition first run `decl_pretag` over their leading type, so that a struct, union, or enum *defined* in a declaration-specifier position enters the tag namespace. Resolving a type only manufactures its descriptor; the tag is entered into the symbol table by the struct/union/enum declaration handler, which `decl_pretag` reaches by peeling the type's array, qualifier, and (toward a bodied definition) pointer wrappers. A function's return type needs this as much as a variable's type: the kernel's i915 helper is written `static __always_inline struct sgt_iter { …} __sgt_iter(…)`, defining `struct sgt_iter` where the return type goes. Without pre-tagging, that definition builds a descriptor but registers no tag, so every later bodyless `struct sgt_iter`—the field in `i915_mm.c`'s `struct remap_pfn`, the locals in the iterator macros—resolves to an opaque type and each `.sgp`/`.curr` access is rejected as "no member named".

⟨ *Selfhost: sema declarations* 758 ⟩ ≡

```
/* Peel array/qual (and pointer only toward a
   bodied struct/union def) wrappers,
 * registering any struct/union/enum tag a
   declared type introduces. */
static void decl_pretag(struct Node *tn) {
    struct Node *t = tn;
    for (;;) {
        if (!t) return;
        unsigned long tag = t->tag;
        if (tag == AST_TYPE_ARRAY || tag ==
AST_TYPE_QUALS) {
            t = (struct Node *)t->f0;
            continue;
        }
```

```
        if (tag == AST_TYPE_PTR) {
            /* peel toward a bodied struct/union
   definition only */
            struct Node *p = (struct Node *)t->f0;
            while (p) {
                unsigned long pt = p->tag;
                if (pt == AST_TYPE_ARRAY || pt ==
   AST_TYPE_QUALS || pt == AST_TYPE_PTR) {
                    p = (struct Node *)p->f0;
   continue;
                }
                if (pt == AST_TYPE_STRUCT || pt ==
    AST_TYPE_UNION) {
                    struct Node *sd = (struct Node
    *)p->f0;
                    if (sd && sd->f1)
   kcc_sema_check_decl(p);  /* bodied → register
   */
                }
                return;
            }
            return;
        }
        if (tag == AST_TYPE_STRUCT || tag ==
   AST_TYPE_UNION || tag == AST_TYPE_ENUM)
            kcc_sema_check_decl(t);
        return;
    }
}

void kcc_sema_check_decl(void *node) {
    if (!node) return;
    struct Node *d = (struct Node *)node;
    unsigned long tag = d->tag;

    if (tag == AST_VAR_DECL) {
        if (d->f0) decl_pretag((struct Node *)d->
   f0);
        kcc_sema_infer_array_len(d);
        void *ty = kcc_sema_resolve_type((void *)d
   ->f0);
        void *name = (void *)d->f1;
        if (name) {
            kcc_sema_check_redef(d, name);
            void *sym = kcc_symtab_declare(name,
   kcc_strlen(name), SYM_VAR, ty);
            SL(sym, SYM_LINE) = d->line;
            SL(sym, SYM_COL) = d->col;
        }
        if (d->f2) {
            void *it = kcc_sema_expr((void *)d->f2
   );
            kcc_sema_check_int_ptr(d, ty, it, (
   void *)d->f2);
        }
        return;
    }
    if (tag == AST_FUNC_DECL) {
        /* Register any struct/union/enum tag the
   return type DEFINES, exactly as
           a variable declaration does---
   resolve_type alone builds the aggregate
           descriptor but never enters the tag in
   the symtab, so a later bodyless
           `struct T' reference would find no
   members.  The kernel's i915
           `static __always_inline struct sgt_iter
    { ... } __sgt_iter(...)' defines
           its struct in the return-type position;
    without this, `struct sgt_iter'
           used elsewhere resolves to an opaque
   type and every `.sgp'/`.curr'
           member access is rejected as "no member
    named". */
        if (d->f0) decl_pretag((struct Node *)d->
   f0);
        void *ret = kcc_sema_resolve_type((void *)
   d->f0);
        void *params = (void *)(d->f2 & ~3UL);
        struct Type *ft = (struct Type *)
   kcc_type_func(ret, params);
        if (d->f2 & 1) {                          /*
    variadic: nfixed → TY_QUALS */
            unsigned long nfixed = 0;
            struct Node *p = (struct Node *)(d->f2
    & ~3UL);
            while (p) { nfixed++; p = (struct Node
    *)p->f4; }
            ft->quals = nfixed;
        }
        void *name = (void *)d->f1;
        if (name) kcc_symtab_declare(name,
   kcc_strlen(name), SYM_FUNC, ft);
        if (!d->f3) return;
   /* prototype */
        sema_cur_ret = (void *)ft->f0;
   /* return type from TY_FUNC */
        kcc_symtab_push_scope();
        kcc_symtab_label_begin();
        struct Node *p = (struct Node *)(d->f2 &
   ~3UL);
        while (p) {
            void *pty = kcc_type_param_adjust(
   kcc_sema_resolve_type((void *)p->f0));
            void *pname = (void *)p->f1;
            if (pname) {
                kcc_sema_check_redef(p, pname);
                kcc_symtab_declare(pname,
   kcc_strlen(pname), SYM_VAR, pty);
            }
            p = (struct Node *)p->f4;
        }
        kcc_sema_check_stmt((void *)d->f3);
        kcc_sema_check_return_path(d);
        kcc_symtab_label_end();
        kcc_symtab_pop_scope();
        return;
    }
    if (tag == AST_TYPEDEF_DECL) {
        struct Node *t = (struct Node *)d->f0;
        if (!t) return;
        while (t && (t->tag == AST_TYPE_PTR || t->
   tag == AST_TYPE_ARRAY)) {
            struct Node *inner = (struct Node *)t
   ->f0;
            if (!inner) break;
            t = inner;
        }
        if (t && (t->tag == AST_TYPE_STRUCT || t->
   tag == AST_TYPE_UNION ||
                  t->tag == AST_TYPE_ENUM))
            kcc_sema_check_decl(t);
        void *ty = kcc_sema_resolve_type((void *)d
   ->f0);
        void *name = (void *)d->f1;
        if (!name) return;
        kcc_symtab_declare(name, kcc_strlen(name),
    SYM_TYPEDEF, ty);
        return;
    }
    if (tag == AST_TYPE_STRUCT || tag ==
   AST_TYPE_UNION) {
```

```
    d = (struct Node *)d->f0;                /*
 unwrap to inner decl */
    if (!d) return;
    tag = d->tag;
    /* fall through to struct/union decl
handling below */
 }
 if (tag == AST_STRUCT_DECL || tag ==
AST_UNION_DECL) {
    /* register any tagged type a member
defines */
    struct Node *fld = (struct Node *)d->f1;
    while (fld) {
        struct Node *ct = (struct Node *)fld->
f0;
        while (ct && (ct->tag == AST_TYPE_PTR
|| ct->tag == AST_TYPE_ARRAY ||
                      ct->tag ==
AST_TYPE_QUALS))
            ct = (struct Node *)ct->f0;
        if (ct) {
            unsigned long ctag = ct->tag;
            struct Node *inner = 0;
            if (ctag == AST_TYPE_STRUCT ||
ctag == AST_TYPE_UNION ||
                ctag == AST_TYPE_ENUM) {
                inner = (struct Node *)ct->f0;
                if (inner && inner->f1)
kcc_sema_check_decl(ct);
            } else if (ctag == AST_STRUCT_DECL
 || ctag == AST_UNION_DECL ||
                       ctag == AST_ENUM_DECL)
{
                if (ct->f1)
kcc_sema_check_decl(ct);
            }
        }
        fld = (struct Node *)fld->f4;
    }
    void *name = (void *)d->f0;
    if (!name) return;
    void *sym = kcc_symtab_lookup_tag(name,
kcc_strlen(name));
    if (sym) {
        struct Type *desc = (struct Type *)SL(
sym, SYM_TYPE);
        if (desc) {
            unsigned long tdepth = SL(sym,
SYM_DEPTH);
            if (!d->f1) return;
/* bodyless reference: no-op */
            if (tdepth == kcc_sym_depth) {
/* complete in place */
                desc->f0 = (unsigned long)d;
                desc->size = 0;
                kcc_type_layout(desc);
                return;
            }
            /* enclosing-scope tag: this
definition shadows → fresh */
        }
    }
    void *desc = (tag == AST_STRUCT_DECL) ?
kcc_type_struct(d)
                                          :
kcc_type_union(d);
    kcc_type_layout(desc);
    kcc_symtab_declare_tag(name, kcc_strlen(
name), desc);
    return;
 }
 if (tag == AST_TYPE_ENUM) {
     d = (struct Node *)d->f0;
     if (!d) return;
     tag = AST_ENUM_DECL;
 }
 if (tag == AST_ENUM_DECL) {
     void *name = (void *)d->f0;
     if (name) {
         void *desc = kcc_type_enum(d);
         kcc_symtab_declare_tag(name,
kcc_strlen(name), desc);
     }
     struct Node *en = (struct Node *)d->f1;
     long counter = 0;
     while (en) {
         if (en->f1) counter = (long)
kcc_eval_const_int((void *)en->f1);
         void *ename = (void *)en->f0;
         if (ename) {
             void *sym = kcc_symtab_declare(
ename, kcc_strlen(ename),

SYM_ENUM_CONST, kcc_type_int_s);
             SL(sym, SYM_VALUE) = (unsigned
long)counter;
         }
         counter++;
         en = (struct Node *)en->f4;
     }
     return;
 }
}
```

□

758 · used in 768a

71.5. Expression Type-Checking

`kcc_sema_expr` dispatches on tag, storing the resolved type in the node's `AST_F3`. Simple forms are inline; identifiers, literals, binary, unary, assignment, calls, member access, casts, and `sizeof` are delegated. `_Generic` is resolved here—the controlling expression is typed but not evaluated, and the first compatible association (or the default) is chosen and stashed for the IR builder. A compound literal `(T){…}` resolves `T` *and* type-checks its brace initializer eagerly: the initializer must be walked while the enclosing function's parameters are still in scope, because a `typeof(expr)` nested in it (the shape the kernel's `min()` macro plants through `__cmp_once`) would otherwise be resolved only later by the IR builder, after those parameters have gone out of scope, and every identifier the `typeof` names would read as undeclared.

⟨ *Selfhost: sema expr* 760 ⟩ ≡

```
void *kcc_sema_expr(void *node) {
    if (!node) return kcc_type_void;
    struct Node *n = (struct Node *)node;
    unsigned long tag = n->tag;
    void *t;
    switch (tag) {
    case AST_IDENT_EXPR: t = sema_expr_ident(n);
   break;
    case AST_INT_LIT: case AST_FLOAT_LIT: case
   AST_STR_LIT: case AST_CHAR_LIT:
        t = sema_expr_literal(n, tag); break;
```

```
 case AST_BINARY_EXPR: t = sema_expr_binary(n);
 break;
 case AST_UNARY_EXPR: t = sema_expr_unary(n);
break;
 case AST_ASSIGN_EXPR: t = sema_expr_assign(n);
 break;
 case AST_COND_EXPR: {
     struct Type *ct = (struct Type *)
kcc_sema_expr((void *)n->f0);
     /* GNU `a ?: b': the middle operand is
omitted (f1 == 0) and the result's
        then-value is the condition itself, so
the then-type is the condition's
        type---the kernel's `(nalg ?: alg)->
cra_flags' in crypto/algapi.c needs
        this or the `->' lands on a null type
("member access on non-struct"). */
     struct Type *tt = n->f1 ? (struct Type *)
kcc_sema_expr((void *)n->f1) : ct;
     struct Type *et = (struct Type *)
kcc_sema_expr((void *)n->f2);   /* else */
     t = tt;
/* result = then type, usually */
     /* \stdref{6.5.15}p6: when one operand is
a null pointer constant, the
        result has the OTHER operand's type.
The kernel's __is_constexpr(x)
        leans on this: `8 ? (void *)((long)(x)
* 0l) : (int *)8' is `int *'
        exactly when x is an integer constant
expression (its void* arm is
        then a null pointer constant), so `
sizeof(*that)' is 4 and the min/max
        macros pick their constant branch---kcc
 previously always took the then
        (void*) type, gave sizeof 0, and
rejected the fixed array as a VLA. */
     if (n->f1 && tt && et && tt->kind ==
TY_PTR && et->kind == TY_PTR) {
         if (kcc_expr_is_ice((void *)n->f1) &&
kcc_eval_const_int((void *)n->f1) == 0) t = et;
         else if (kcc_expr_is_ice((void *)n->f2
) && kcc_eval_const_int((void *)n->f2) == 0) t
= tt;
     }
     break;
 }
 case AST_CALL_EXPR: t = sema_expr_call(n);
break;
 case AST_SUBSCRIPT: {
     struct Type *at = (struct Type *)
kcc_sema_expr((void *)n->f0);
     kcc_sema_expr((void *)n->f1);
     if (at->kind == TY_PTR || at->kind ==
TY_ARRAY) t = (void *)at->f0;
     else t = kcc_type_int_s;
     break;
 }
 case AST_MEMBER: t = sema_expr_member(n);
break;
 case AST_CAST_EXPR: t = sema_expr_cast(n);
break;
 case AST_SIZEOF_EXPR: t = sema_expr_sizeof(n);
 break;
 case AST_ALIGNOF_EXPR:
     if (n->f1) kcc_sema_resolve_type((void *)n
->f0);  /* _Alignof(type) */
     else kcc_sema_expr((void *)n->f0);
        /* __alignof__(expr): type in F3 */
     t = kcc_type_long_u;
     break;
 case AST_COMMA_EXPR:
     kcc_sema_expr((void *)n->f0);
     t = kcc_sema_expr((void *)n->f1);
     break;
 case AST_PAREN_EXPR:
     t = kcc_sema_expr((void *)n->f0);
     break;
 case AST_GENERIC: t = sema_expr_generic(n);
break;
 case AST_COMPOUND_LIT:
     if (!n->f0) t = kcc_type_int_s;
     else t = kcc_sema_resolve_type((void *)n->
f0);
     /* Type-check the brace initializer now,
while the enclosing function's
        scope is still live.  If we skip it,
any `typeof(expr)` buried in the
        initializer (as the kernel's min()
macro plants via __cmp_once) is
        left for the IR builder to resolve
later, by which point the params
        it names are out of scope and read as
undeclared identifiers. */
     if (n->f1) kcc_sema_expr((void *)n->f1);
     break;
 case AST_VA_ARG:
     if (n->f0) kcc_sema_expr((void *)n->f0);
     if (!n->f1) t = kcc_type_int_s;
     else t = kcc_sema_resolve_type((void *)n->
f1);
     break;
 case AST_TYPES_COMPAT:
     /* __builtin_types_compatible_p: resolve
both operand types; result int */
     if (n->f0) kcc_sema_resolve_type((void *)n
->f0);
     if (n->f1) kcc_sema_resolve_type((void *)n
->f1);
     t = kcc_type_int_s;
     break;
 case AST_LABEL_ADDR:
     /* GNU `&&label': the address of a label,
typed `void *' (\stdref{J.5.8}). */
     t = kcc_type_ptr(kcc_type_void);
     break;
 case AST_INIT_LIST: {
     struct Node *el = (struct Node *)n->f0;
     while (el) {
         if (el->tag == AST_DESIGNATOR) {
             if (el->f3) kcc_sema_expr((void *)
el->f3);
         } else {
             kcc_sema_expr(el);
         }
         el = (struct Node *)el->f4;
     }
     t = kcc_type_int_s;
     break;
 }
 case AST_STMT_EXPR: {
     struct Node *cmp = (struct Node *)n->f0;
     if (!cmp) { t = kcc_type_void; break; }
     kcc_sema_check_stmt(cmp);
     struct Node *it = (struct Node *)cmp->f0;
     if (!it) { t = kcc_type_void; break; }
     while (it->f4) it = (struct Node *)it->f4;
   /* last item */
     if (it->tag != AST_EXPR_STMT || !it->f0) {
 t = kcc_type_void; break; }
     void *ft = (void *)((struct Node *)it->f0)
->f3;
     t = ft ? ft : kcc_type_void;
     break;
```

```
    }
    default: t = kcc_type_int_s; break;
    }
    n->f3 = (unsigned long)t;
    return t;
}

static void *sema_expr_generic(struct Node *n) {
    struct Type *ct = (struct Type *)kcc_sema_expr
   ((void *)n->f0);
    if (ct && ct->kind == TY_ARRAY) ct = (struct
   Type *)kcc_type_ptr((void *)ct->f0);
    struct Node *assoc = (struct Node *)n->f1;
    void *dflt = 0;
    void *chosen = 0;
    while (assoc) {
        if (!assoc->f0) {
   /* default association */
            dflt = (void *)assoc->f1;
        } else {
            void *aty = kcc_sema_resolve_type((
   void *)assoc->f0);
            if (kcc_type_compatible(aty, ct)) {
   chosen = (void *)assoc->f1; break; }
        }
        assoc = (struct Node *)assoc->f4;
    }
    if (!chosen) {
        if (!dflt) {
            kcc_diag_error(n->line, 0, m_generic);
            return kcc_type_int_s;
        }
        chosen = dflt;
    }
    n->f2 = (unsigned long)chosen;
   /* for the IR builder */
    return kcc_sema_expr(chosen);
}
```

□

760 · used in 768a

Identifiers, literals, and the named-message helpers. ⟨ *Selfhost: sema expridentifier* 762a ⟩ ≡

```
static char *sema_build_named(const char *name,
   const char *prefix) {
    unsigned long plen = kcc_strlen(prefix);
    unsigned long nlen = kcc_strlen(name);
    char *buf = (char *)kcc_arena_alloc(plen +
   nlen + 2);
    kcc_memcpy(buf, prefix, plen);
    kcc_memcpy(buf + plen, name, nlen);
    buf[plen + nlen] = '\'';
    buf[plen + nlen + 1] = 0;
    return buf;
}
void kcc_sema_err_named(unsigned long line,
   unsigned long col, const char *name, const char
    *prefix) {
    kcc_diag_error(line, col, sema_build_named(
   name, prefix));
}
void kcc_sema_warn_named(unsigned long line,
   unsigned long col, const char *name, const char
    *prefix) {
    kcc_diag_warning(line, col, sema_build_named(
   name, prefix));
}
```

```
static void *sema_expr_ident(struct Node *n) {
    void *name = (void *)n->f0;
    if (!name) return kcc_type_int_s;
    void *sym = kcc_symtab_lookup(name, kcc_strlen
   (name));
    if (!sym) {
        if (sema_callee_ctx) {
            sema_callee_implicit = 1;
            if (kcc_warn_all)
                kcc_sema_warn_named(n->line, n->
   col, name, m_implicit);
            return kcc_type_int_s;
        }
        kcc_sema_err_named(n->line, n->col, name,
   m_undecl);
        return kcc_type_int_s;
    }
    if (SL(sym, SYM_KIND) == SYM_ENUM_CONST) {
        n->tag = AST_INT_LIT;
   /* rewrite in place */
        n->f0 = SL(sym, SYM_VALUE);
        return kcc_type_int_s;
    }
    unsigned *flags = (unsigned *)((char *)sym +
   SYM_FLAGS);
    *flags |= 2;
   /* mark used */
    void *ty = (void *)SL(sym, SYM_TYPE);
    ty = kcc_conv_func_decay(ty);
    ty = kcc_conv_array_decay(ty);
    return ty;
}

static void *sema_expr_literal(struct Node *n,
   unsigned long tag) {
    if (tag == AST_FLOAT_LIT) return n->f2 ?
   kcc_type_float : kcc_type_double;
    if (tag == AST_STR_LIT) return kcc_type_ptr(
   kcc_type_char_s);
    /* int / char literal */
    unsigned long f2 = n->f2;
    if (!f2) return kcc_type_int_s;
    unsigned long rank = (f2 >> 1) & 3;
    unsigned long uns = f2 & 1;
    if (rank == 0) return kcc_type_int_u;
   /* u/U, no l */
    if (rank == 1) return uns ? kcc_type_long_u :
   kcc_type_long_s;
    return uns ? kcc_type_llong_u :
   kcc_type_llong_s;
}
```

□

762a · used in 768a

The -Wall semantic checks: unused, redefinition, sign-compare, int/pointer. ⟨ *Selfhost: sema checks* 762b ⟩ ≡

```
void kcc_sema_check_unused_scope(void) {
    if (!kcc_warn_all) return;
    void *ht = kcc_symtab_current_ht();
    if (!ht) return;
    char *entries = *(char **)((char *)ht +
   HT_ENTRIES);
    unsigned long cap = SL(ht, HT_CAP);
    for (unsigned long i = 0; i < cap; i++) {
        char *e = entries + i * HTE_SIZE;
        if (!*(void **)(e + HTE_KEY)) continue;
        void *sym = *(void **)(e + HTE_VALUE);
```

```
        if (!sym) continue;
        if (SL(sym, SYM_KIND) != SYM_VAR) continue
    ;
        if (*(unsigned *)((char *)sym + SYM_FLAGS)
     & 2) continue;
        kcc_sema_warn_named(SL(sym, SYM_LINE), SL(
    sym, SYM_COL),
                            (const char *)SL(sym,
    SYM_NAME), m_unused);
     }
}

void kcc_sema_check_redef(void *decl, const char *
    name) {
     struct Node *d = (struct Node *)decl;
     if (kcc_sym_depth < 2) return;
    /* block scope only */
     void *sym = kcc_symtab_lookup_current(name,
    kcc_strlen(name));
     if (!sym) return;
     kcc_sema_err_named(d->line, d->col, name,
    m_redef);
}

long kcc_sema_int_signedness(void *t_) {
     struct Type *t = (struct Type *)t_;
     if (!t) return -1;
     unsigned long k = t->kind;
     if (k == TY_ENUM) return 1;
     if (k < TY_CHAR || k > TY_LLONG) return -1;
     return (long)t->f0;
    /* 1 signed, 0 unsigned */
}

static int sema_is_nonneg_const(void *e_) {
     struct Node *e = (struct Node *)e_;
     if (!e) return 0;
     while (e->tag == AST_PAREN_EXPR) { e = (struct
     Node *)e->f0; if (!e) return 0; }
     return (e->tag == AST_INT_LIT || e->tag ==
    AST_CHAR_LIT);
}

void kcc_sema_check_sign_compare(void *expr, void
    *lt, void *rt) {
     struct Node *n = (struct Node *)expr;
     if (!kcc_warn_all) return;
     long ls = kcc_sema_int_signedness(lt);
     if (ls == -1) return;
     long rs = kcc_sema_int_signedness(rt);
     if (rs == -1) return;
     if (ls == rs) return;
     if (sema_is_nonneg_const((void *)n->f1))
    return;
     if (sema_is_nonneg_const((void *)n->f2))
    return;
     kcc_diag_warning(n->line, n->col, m_sign_cmp);
}

static int sema_ptr_int_class(void *t_) {
     struct Type *t = (struct Type *)t_;
     if (!t) return 0;
     unsigned long k = t->kind;
     if (k == TY_PTR) return 1;
     if (k == TY_ENUM) return 2;
     if (k < TY_BOOL || k > TY_LLONG) return 0;
     return 2;
}

static int sema_is_null_const(void *e_) {
     struct Node *e = (struct Node *)e_;
     if (!e) return 0;
     while (e->tag == AST_PAREN_EXPR) { e = (struct
     Node *)e->f0; if (!e) return 0; }
     if (e->tag != AST_INT_LIT) return 0;
     return e->f0 == 0;
}

void kcc_sema_check_int_ptr(void *loc, void *dst,
    void *src, void *srcexpr) {
     struct Node *n = (struct Node *)loc;
     if (!kcc_warn_all) return;
     int dc = sema_ptr_int_class(dst);
     int sc = sema_ptr_int_class(src);
     if (dc == 1) {
         if (sc != 2) return;
         if (sema_is_null_const(srcexpr)) return;
         kcc_diag_warning(n->line, n->col,
    m_int2ptr);
     } else if (dc == 2) {
         if (sc != 1) return;
         kcc_diag_warning(n->line, n->col,
    m_ptr2int);
     }
}
□
```

762b · used in 768a

Binary, unary, and assignment expressions. Each of the three checkers first rejects an operation that would perform 128-bit *arithmetic*: a binary operator with a `__int128` operand, an arithmetic unary operator (`-`/`+`/`~`/`++`/`--`, but not address-of) on one, or a compound assignment to one. kcc lowers a `__int128` rvalue through a single eight-byte slot, so these would silently drop the high 64 bits; rejecting matches the assembly semantics and the `_Complex`/VLA policy. Plain `=`, copies, and by-value passing stay valid.

⟨ *Selfhost: sema binunasgn* 763 ⟩ ≡

```
static void *sema_expr_binary(struct Node *n) {
     struct Type *lt = (struct Type *)kcc_sema_expr
    ((void *)n->f1);
     struct Type *rt = (struct Type *)kcc_sema_expr
    ((void *)n->f2);
     unsigned long op = n->f0;
     /* A __int128 rvalue is lowered as the address
     of a sixteen-byte stack slot
        (Section~\ref{sec:ir-int128}), and a
    handful of operators are implemented
        on the two halves: an unsigned 128-bit
    multiply --- whose low 128 bits are
        the same for signed and unsigned operands,
    so it is accepted either way ---
        a logical right shift of an unsigned
    __int128 (together exactly the
        kernel's math64.h mul_u64_u32_shr /
    mul_u64_u64_shr helpers), and equality
        (\texttt{==}/\texttt{!=}), which is a per-
    half compare the kernel's
        \texttt{try\_cmpxchg128} needs to test the
    result of a 128-bit swap.  Every
        other operator would still need carry-aware
     two-word arithmetic that is not
        yet emitted, so it stays rejected. */
     if ((lt && lt->kind == TY_INT128) || (rt && rt
    ->kind == TY_INT128)) {
         unsigned long i128_ok = (op == TOK_STAR)
```

```
            || (op == TOK_SHR && lt && lt->kind ==
    TY_INT128 && lt->f0 == 0)
            || op == TOK_EQ || op == TOK_NE;
        if (!i128_ok)
            kcc_diag_error(n->line, n->col,
   m_int128);
    }
    switch (op) {
    case TOK_EQ: case TOK_NE: case TOK_LT: case
   TOK_GT: case TOK_LE: case TOK_GE:
        kcc_sema_check_sign_compare(n, lt, rt);
        return kcc_type_int_s;
    case TOK_AND: case TOK_OR:
        return kcc_type_int_s;
    case TOK_SHL: case TOK_SHR:
        return kcc_conv_promote(lt);
   /* result = promoted left */
    default: break;
    }
    if (lt->kind == TY_PTR) {
   /* pointer arithmetic */
        if (op == TOK_MINUS) {
            unsigned long rk = rt->kind;
            if (rk == TY_PTR || rk == TY_ARRAY)
   return kcc_type_long_s;  /* ptrdiff */
        }
        return lt;
    }
    return kcc_conv_usual(lt, rt);
}

static void *sema_expr_unary(struct Node *n) {
    struct Type *ot = (struct Type *)kcc_sema_expr
   ((void *)n->f1);
    unsigned long op = n->f0;
    /* Reject arithmetic unary operators on a
   __int128 operand (-, +, ~, ++/--);
       address-of (&) is structural and stays
   allowed. */
    if (ot && ot->kind == TY_INT128 && op !=
   TOK_AMP)
        kcc_diag_error(n->line, n->col, m_int128);
    switch (op) {
    case TOK_AMP: return kcc_type_ptr(ot);
    case TOK_STAR:
        if (ot->kind != TY_PTR) return
   kcc_type_int_s;
        return (void *)ot->f0;
    case TOK_BANG: return kcc_type_int_s;
    case TOK_TILDE: case TOK_MINUS: case TOK_PLUS:
        return kcc_conv_promote(ot);
    default: return ot;
   /* ++/-- and fallback */
    }
}

static void *sema_expr_assign(struct Node *n) {
    void *lt = kcc_sema_expr((void *)n->f1);
    void *rt = kcc_sema_expr((void *)n->f2);
    /* Compound assignment (+=, -=, ...) on a
   __int128 lvalue is arithmetic and
       would drop the high 64 bits; reject it.
   Plain '=' is a 16-byte copy. */
    if (n->f0 != TOK_ASSIGN && lt && ((struct Type
    *)lt)->kind == TY_INT128)
        kcc_diag_error(n->line, n->col, m_int128);
    if (n->f0 == TOK_ASSIGN)
        kcc_sema_check_int_ptr(n, lt, rt, (void *)
   n->f2);
    return lt;
}
```

□

763 · used in 768a

Calls, member access, casts, and sizeof. The call handler intercepts `__builtin_choose_expr(cond, a, b)` before anything else: it is not a real call but a compile-time selection, so the handler folds `cond` to a constant and rewrites the node into the chosen argument, keeping this node's sibling link so it stays correctly placed in an enclosing argument list. The whole compiler then sees the selected expression—crucially its *type*, which the kernel's `__inttype` builds a get_user operand's width from; treating the builtin as an ordinary (implicit-int) call would size that operand at four bytes and emit `movq …,%r8d`.

Member access dereferences the object for an arrow (`p->m`) and then requires a struct or union. An *array* object is accepted here too: array-to-pointer decay (C §6.3.2.1) makes `(a)->m` equivalent to `(*a).m`, and since `TY_ARRAY` stores its element type in the same `f0` slot as a pointer's pointee, one extra kind test lets the element type flow straight through. This is what the kernel's `cpumask_bits(mask)` macro needs—it expands to `(mask)->bits` where `mask` is a `cpumask_var_t` array—so `smp.c`'s `for_each_cpu(cpu, cfd->cpumask)` loop now type-checks.

`sizeof` of a *type* lays that type out immediately rather than merely resolving its descriptor. Resolving an aggregate only builds the shell; its members—and therefore any `typeof(expr)` that spells a member's type—are walked only by `kcc_type_layout`. The kernel's `__is_kfifo_ptr` plants that shape inside a `sizeof`: `sizeof(struct{ union{ …; typeof(*(__tmp)->type) *type; …}; …})`, where `__tmp` is a block local of the enclosing statement expression. Were the layout left to the IR builder, those `typeof` operands would be resolved after `__tmp`'s scope had been popped and every name in them would read as undeclared—the same late-resolution hazard the compound-literal handler guards against. Forcing the layout here, while the scope is live, resolves them correctly; the descriptor and its size are memoised, so the later size query is a no-op.

⟨ *Selfhost: sema callmembercast* 764 ⟩ ≡

```
static void *sema_expr_call(struct Node *n) {
    struct Node *callee = (struct Node *)n->f0;
    /* __builtin_choose_expr(cond, a, b): a
   compile-time selection whose type AND
       value are the chosen argument's; the
   unchosen arg is not evaluated
       (GCC extension).  The kernel forms `
   __inttype(x)' ---and hence the width of
       a get_user output operand--- from it, so
   its type must be the selected
       operand's, not a default int.  Rewrite the
   call node in place to become the
       chosen argument, preserving this node's own
    position in any containing list
       (its f4 sibling link), so sizeof,
   __typeof__, constant folding, and code
       generation all see straight through it. */
    if (callee && callee->tag == AST_IDENT_EXPR &&
    callee->f0 &&
        kcc_strcmp((const char *)callee->f0, "
   __builtin_choose_expr") == 0) {
        struct Node *a0 = (struct Node *)n->f1;
```

```
        struct Node *a1 = a0 ? (struct Node *)a0->
    f4 : 0;
        struct Node *a2 = a1 ? (struct Node *)a1->
    f4 : 0;
        if (a0) kcc_sema_expr(a0);
        struct Node *pick = (a0 &&
    kcc_eval_const_int(a0)) ? a1 : a2;
        if (pick) {
            unsigned long sib = n->f4;      /*
    keep our slot in the arg/sibling list */
            *n = *pick;
            n->f4 = sib;
            return kcc_sema_expr(n);
        }
     }
     sema_callee_implicit = 0;
     sema_callee_ctx = (callee->tag ==
    AST_IDENT_EXPR);
     struct Type *ct = (struct Type *)kcc_sema_expr
    (callee);
     sema_callee_ctx = 0;
     /* Capture whether the callee was an implicit
    (undeclared) function BEFORE
        checking the arguments:
    sema_callee_implicit is a global, and
    evaluating
        an argument that is itself a call resets it
     to 0 at that nested call's
        entry.  Reading it after the arg loop would
     then misfire "called object
        is not a function" for e.g.
    __builtin_expect(!!(f(x) > 0), 0). */
     unsigned long callee_implicit =
    sema_callee_implicit;
     struct Node *arg = (struct Node *)n->f1;
     while (arg) { kcc_sema_expr(arg); arg = (
    struct Node *)arg->f4; }
     if (ct->kind == TY_FUNC) return (void *)ct->f0
    ;
     if (ct->kind == TY_PTR) {
         struct Type *fn = (struct Type *)ct->f0;
         if (fn->kind != TY_FUNC) return
    kcc_type_int_s;
         return (void *)fn->f0;
     }
     if (callee_implicit) return kcc_type_int_s;
     kcc_diag_error(n->line, n->col, m_not_func);
     return kcc_type_int_s;
}

struct FindResult kcc_type_find_member(void *sty,
    const char *name) {
     struct FindResult r;
     r.field = 0; r.off = 0;
     struct Type *ty = (struct Type *)sty;
     kcc_type_layout(ty);
     struct Node *decl = (struct Node *)ty->f0;
     if (!decl) return r;
     struct Node *field = (struct Node *)decl->f1;
     while (field) {
         void *fname = (void *)field->f1;
         if (!fname) {
             void *fty = (void *)field->f0;
             if (fty) {
                 struct Type *ift = (struct Type *)
    kcc_sema_resolve_type(fty);
                 if (ift && (ift->kind == TY_STRUCT
     || ift->kind == TY_UNION)) {
                     struct FindResult inner =
    kcc_type_find_member(ift, name);
                     if (inner.field) {
                         r.field = inner.field;
                         r.off = inner.off + field
    ->f3;
                         return r;
                     }
                 }
             }
         } else if (kcc_strcmp(fname, name) == 0) {
             r.field = field;
             r.off = field->f3;
             return r;
         }
         field = (struct Node *)field->f4;
     }
     return r;
}

static void *sema_expr_member(struct Node *n) {
     struct Type *ot = (struct Type *)kcc_sema_expr
    ((void *)n->f0);
     if (n->f2) {
    /* arrow: deref pointer (or decay array) */
         if (ot->kind != TY_PTR && ot->kind !=
    TY_ARRAY) goto fallback;
         ot = (struct Type *)ot->f0;
     }
     if (ot->kind != TY_STRUCT && ot->kind !=
    TY_UNION) goto fallback;
     {
         struct Node *fd = (struct Node *)
    kcc_type_find_member(ot, (void *)n->f1).field;
         if (!fd) {
             kcc_sema_err_named(n->line, n->col, (
    const char *)n->f1, m_nomember);
             return kcc_type_int_s;
         }
         void *fts = (void *)fd->f0;
         if (!fts) return kcc_type_int_s;
         struct Type *fty = (struct Type *)
    kcc_sema_resolve_type(fts);
         /* bit-field integer promotion (§6.3.1.1)
    */
         unsigned long width = fd->f2 & 0xffff;
         if (!width || !fty) return fty;
         unsigned long k = fty->kind;
         if (k - 2 > 5) return fty;
    /* not TY_BOOL..TY_LLONG */
         if (fty->f0) {
    /* signed */
             if (width > 32) return fty;
             return kcc_type_int_s;
         }
         /* unsigned */
         if (width <= 31) return kcc_type_int_s;
         if (width > 32) return fty;
         return kcc_type_int_u;
    /* width == 32 */
     }
fallback:
     kcc_diag_error(n->line, n->col, m_not_struct);
     return kcc_type_int_s;
}

static void *sema_expr_cast(struct Node *n) {
     void *target = kcc_sema_resolve_type((void *)n
    ->f0);
     kcc_sema_expr((void *)n->f1);
     return target;
}

static void *sema_expr_sizeof(struct Node *n) {
     if (n->f1) {
```

```
    /* type operand */
         void *ty = kcc_sema_resolve_type((void *)n
    ->f0);
         /* Lay the type out NOW, while the
    enclosing scope is still live, so a
            `typeof(expr)' buried in an anonymous
    struct/union member resolves
            against the locals it names.
    resolve_type only builds the aggregate
            descriptor; its members---and thus
    their typeof operands---are walked
            only by kcc_type_layout.  The kernel's
    __is_kfifo_ptr plants exactly
            this shape: `sizeof(struct{ union{ ...;
     typeof(*(__tmp)->type) *type;
            ... }; ... })'.  Deferring the layout
    to the IR builder would resolve
            `__tmp' after its block scope has been
    popped, so it would read as an
            undeclared identifier. */
         kcc_type_layout(ty);
         return kcc_type_long_u;
     }
     kcc_sema_expr((void *)n->f0);
     /* sizeof does not decay arrays (§6.3.2.1):
    restore the array type */
     struct Node *operand = (struct Node *)n->f0;
     struct Node *inner = operand;
     while (inner && inner->tag == AST_PAREN_EXPR)
    inner = (struct Node *)inner->f0;
     if (inner) {
         if (inner->tag == AST_STR_LIT) {
             unsigned long len = inner->f1 + 1;
             void *arr = kcc_type_array(
    kcc_type_char_s, len);
             operand->f3 = (unsigned long)arr;
         } else if (inner->tag == AST_IDENT_EXPR &&
     inner->f0) {
             void *name = (void *)inner->f0;
             void *sym = kcc_symtab_lookup(name,
    kcc_strlen(name));
             if (sym) {
                 struct Type *st = (struct Type *)
    SL(sym, SYM_TYPE);
                 if (st && st->kind == TY_ARRAY)
    operand->f3 = (unsigned long)st;
             }
         }
     }
     return kcc_type_long_u;
}
```

□

764 · used in 768a

Array-length inference and the reaching-end (-Wreturn-type) analysis. ⟨ *Selfhost: sema returnpath* 766 ⟩ ≡

```
unsigned long kcc_agg_leaf_count(void *ty_) {
     struct Type *ty = (struct Type *)ty_;
     if (!ty) return 1;
     if (ty->kind == TY_ARRAY)
         return kcc_agg_leaf_count((void *)ty->f0)
    * ty->f1;
     if (ty->kind == TY_STRUCT) {
         struct Node *decl = (struct Node *)ty->f0;
         if (!decl) return 1;
         struct Node *f = (struct Node *)decl->f1;
         if (!f) return 1;
         unsigned long acc = 0;
         while (f) {
             acc += kcc_agg_leaf_count(
    kcc_sema_resolve_type((void *)f->f0));
             f = (struct Node *)f->f4;
         }
         return acc;
     }
     if (ty->kind == TY_UNION) {
         struct Node *decl = (struct Node *)ty->f0;
         if (!decl) return 1;
         struct Node *f = (struct Node *)decl->f1;
         if (!f) return 1;
         return kcc_agg_leaf_count(
    kcc_sema_resolve_type((void *)f->f0));
     }
     return 1;
}

unsigned long kcc_infer_arr_elems(void *head, void
     *elem) {
     struct Node *it = (struct Node *)head;
     unsigned long leaves = kcc_agg_leaf_count(elem
    );
     unsigned long count = 0;
     while (it) {
         unsigned long tg = it->tag;
         if (tg == AST_INIT_LIST || tg ==
    AST_DESIGNATOR) {
             it = (struct Node *)it->f4;
         } else {
             unsigned long skip = leaves;
             while (skip && it) { it = (struct Node
     *)it->f4; skip--; }
         }
         count++;
     }
     return count;
}

void kcc_sema_infer_array_len(void *decl) {
     struct Node *d = (struct Node *)decl;
     struct Node *tn = (struct Node *)d->f0;
     if (!tn || tn->tag != AST_TYPE_ARRAY) return;
     if (tn->f1) return;
    /* already complete */
     struct Node *init = (struct Node *)d->f2;
     if (!init) return;
     unsigned long len;
     if (init->tag == AST_STR_LIT) {
         len = init->f1 + 1;
     } else if (init->tag == AST_INIT_LIST) {
         struct Type *at = (struct Type *)
    kcc_sema_resolve_type(tn);
         struct Type *elem = at ? (struct Type *)at
    ->f0 : 0;
         if (elem && (elem->kind == TY_STRUCT ||
    elem->kind == TY_UNION ||
                      elem->kind == TY_ARRAY)) {
             len = kcc_infer_arr_elems((void *)init
    ->f0, elem);
         } else {
             len = kcc_ir_initlist_count(init);
         }
     } else {
         return;
     }
     struct Node *lit = (struct Node *)
    kcc_ast_alloc(AST_INT_LIT);
     lit->f0 = len;
     tn->f1 = (unsigned long)lit;
}
```

```
static int sema_const_true(struct Node *e) {
    if (!e) return 0;
    while (e->tag == AST_PAREN_EXPR) { e = (struct
    Node *)e->f0; if (!e) return 0; }
    if (e->tag != AST_INT_LIT && e->tag !=
   AST_CHAR_LIT) return 0;
    return e->f0 != 0;
}

static int sema_has_toplevel_break(struct Node *s)
    {
    if (!s) return 0;
    switch (s->tag) {
    case AST_BREAK_STMT: return 1;
    case AST_WHILE_STMT: case AST_FOR_STMT: case
   AST_DO_STMT:
    case AST_SWITCH_STMT: return 0;
    case AST_COMPOUND: {
        struct Node *it = (struct Node *)s->f0;
        while (it) {
            if (sema_has_toplevel_break(it))
   return 1;
            it = (struct Node *)it->f4;
        }
        return 0;
    }
    case AST_IF_STMT:
        if (sema_has_toplevel_break((struct Node
   *)s->f1)) return 1;
        if (!s->f2) return 0;
        return sema_has_toplevel_break((struct
   Node *)s->f2);
    case AST_LABEL_STMT: case AST_CASE_STMT:
        return sema_has_toplevel_break((struct
   Node *)s->f1);
    case AST_DEFAULT_STMT:
        return sema_has_toplevel_break((struct
   Node *)s->f0);
    default: return 0;
    }
}

static int sema_is_noreturn_call(struct Node *e) {
    if (!e || e->tag != AST_CALL_EXPR) return 0;
    struct Node *callee = (struct Node *)e->f0;
    if (!callee || callee->tag != AST_IDENT_EXPR)
   return 0;
    const char *name = (const char *)callee->f0;
    if (!name) return 0;
    for (int i = 0; sema_noreturn_tbl[i]; i++)
        if (kcc_strcmp(name, sema_noreturn_tbl[i])
    == 0) return 1;
    return 0;
}

static int sema_falls_through(struct Node *s) {
    if (!s) return 1;
    switch (s->tag) {
    case AST_RETURN_STMT: case AST_GOTO_STMT: case
    AST_BREAK_STMT:
    case AST_CONTINUE_STMT: case AST_SWITCH_STMT:
        return 0;
    case AST_COMPOUND: {
        struct Node *it = (struct Node *)s->f0;
        while (it) {
            if (!sema_falls_through(it)) return 0;
            it = (struct Node *)it->f4;
        }
        return 1;
    }
    case AST_IF_STMT:
        if (!s->f2) return 1;
    /* no else → untaken path */
        if (sema_falls_through((struct Node *)s->
   f1)) return 1;
        return sema_falls_through((struct Node *)s
   ->f2);
    case AST_LABEL_STMT:
        return sema_falls_through((struct Node *)s
   ->f1);
    case AST_WHILE_STMT:
        if (!sema_const_true((struct Node *)s->f0)
   ) return 1;
        return sema_has_toplevel_break((struct
   Node *)s->f1);
    case AST_FOR_STMT:
        if (s->f1 && !sema_const_true((struct Node
    *)s->f1)) return 1;
        return sema_has_toplevel_break((struct
   Node *)s->f3);
    case AST_DO_STMT:
        if (!sema_const_true((struct Node *)s->f1)
   ) return 1;
        return sema_has_toplevel_break((struct
   Node *)s->f0);
    case AST_EXPR_STMT:
        return !sema_is_noreturn_call((struct Node
    *)s->f0);
    default: return 1;
    }
}

void kcc_sema_check_return_path(void *func) {
    struct Node *f = (struct Node *)func;
    if (!kcc_warn_all) return;
    if (!sema_cur_ret || SL(sema_cur_ret, 0) ==
   TY_VOID) return;
    void *name = (void *)f->f1;
    if (!name) return;
    if (kcc_strcmp(name, m_main) == 0) return;
   /* main is exempt */
    if (!f->f3) return;
    /* prototype */
    if (!sema_falls_through((struct Node *)f->f3))
    return;
    kcc_diag_warning(f->line, f->col, m_reach_end)
   ;
}
```

□

766 · used in 768a

71.6. Type Dumping and Assembly

kcc_sema_dump_types implements --dump-types, printing each top-level variable, function, and typedef with its resolved type.

⟨ *Selfhost: sema dump* 767 ⟩ ≡

```
void kcc_sema_dump_types(void *tu) {
    struct Node *d = (struct Node *)((struct Node
   *)tu)->f0;
    while (d) {
        unsigned long tag = d->tag;
        if (tag == AST_VAR_DECL || tag ==
   AST_FUNC_DECL || tag == AST_TYPEDEF_DECL) {
            const char *kw = (tag == AST_VAR_DECL)
    ? "var "
```

```
                            : (tag == AST_FUNC_DECL
    ) ? "func " : "typedef ";
              kcc_io_puts(kw);
              if (d->f1) kcc_io_puts((const char *)d
    ->f1);
              kcc_io_puts(tag == AST_TYPEDEF_DECL ?
    " = " : ": ");
              kcc_type_dump(kcc_sema_resolve_type((
    void *)d->f0));
              kcc_io_puts("\n");
          }
          d = (struct Node *)d->f4;
      }
      kcc_io_flush();
}
```

□

767 · used in 768a

⟨ *Selfhost: sema* 768a ⟩ ≡

⟨ *Selfhost: sema preamble* 753 ⟩
⟨ *Selfhost: sema resolve* 755 ⟩
⟨ *Selfhost: sema statements* 756 ⟩
⟨ *Selfhost: sema layout* 757 ⟩
⟨ *Selfhost: sema declarations* 758 ⟩
⟨ *Selfhost: sema expr* 760 ⟩
⟨ *Selfhost: sema exprident* 762a ⟩
⟨ *Selfhost: sema checks* 762b ⟩
⟨ *Selfhost: sema binunasgn* 763 ⟩
⟨ *Selfhost: sema callmembercast* 764 ⟩
⟨ *Selfhost: sema returnpath* 766 ⟩
⟨ *Selfhost: sema dump* 767 ⟩

□

768a · used in 36h

71.7. Verification

`make selfhost-sema` runs the strongest available oracle: the code-generation suite plus `difftest` (byte-comparing every case against clang), and the diagnostic/warning suites that pin the exact `-Wall` message text. A clean sweep means kcc compiled its own semantic core—type resolution, aggregate layout, the enum-constant rewrite, and every diagnostic path—faithfully enough that not one byte of generated code, nor one character of a diagnostic, changed.

⟨ *Selfhost: sema verify* 768b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-sema - compile kcc's semantic
   analyzer in C with kcc, relink,
# run codegen + difftest + diagtest + warntest.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/5] compiling $BASE/selfhost/sema.c with
   kcc"
$BASE/kcc -c "$SH/sema.c" -o "$SH/sema_c.o" >/dev/
   null

echo "[2/5] linking $BASE/kcc_selfhost_sema (sema.
   o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "sema" ]; then OBJ="$OBJ $SH/
   sema_c.o"; else OBJ="$OBJ $BASE/src/${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_sema $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

cp $BASE/kcc "$SH/kcc_asm_backup_sema"
cp $BASE/kcc_selfhost_sema $BASE/kcc

echo "[3/5] difftest (byte-compare generated
   objects against clang)"
set +e
bash $BASE/tests/difftest.sh >/$BASE/sema_difftest
   .log 2>&1
dstatus=$?
tail -1 /$BASE/sema_difftest.log
set -e
if [ $dstatus -ne 0 ]; then cp "$SH/
   kcc_asm_backup_sema" $BASE/kcc; exit $dstatus;
   fi

echo "[4/5] diagtest + warntest (pin the -Wall
   message text)"
set +e
bash $BASE/tests/diagtest.sh >/$BASE/sema_diag.log
    2>&1; diagst=$?
bash $BASE/tests/warntest.sh >/$BASE/sema_warn.log
    2>&1; warnst=$?
tail -1 /$BASE/sema_diag.log; tail -1 /$BASE/
   sema_warn.log
set -e
if [ $diagst -ne 0 ] || [ $warnst -ne 0 ]; then cp
    "$SH/kcc_asm_backup_sema" $BASE/kcc; exit 1;
   fi

echo "[5/5] codegen suite with the self-hosted
   semantic analyzer"
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_sema" $BASE/kcc
exit $status
```

□

768b · used in 36i · uses _main 621, 629

72. Self-Hosting: The x86-64 Back End in C

The second target's back end (Chapter 38) walks the same neutral IR the ARM64 path consumes and prints x86-64 assembly text for the system assembler. Porting it to C completes *both* back ends in kcc-compiled C—the ARM64 encoder (Chapter 70) and now the x86-64 emitter. The module is long but entirely mechanical: a family of small helpers that append AT&T text to the shared growable buffer (`kcc_as_str`/`kcc_as_ch`/`kcc_as_dec`), a flat opcode dispatch that routes each IR instruction to the right shape, an all-spill frame, System V argument marshalling, and the data-emission paths for globals, static locals, read-only constants, and string literals. A single flag, `kcc_x86_os`, chooses Mach-O (Darwin) or ELF (Linux) directives.

Unlike the ARM64 suites, the code-generation and difftest corpora do not exercise x86 selection (they target ARM64), so the port is verified *functionally*: a broad, opcode-exercising probe suite is compiled with `--target=x86_64` by the C emitter and run under Rosetta, and its exit status must match the ARM64 reference.

72.1. State, Constants, and the AT&T Vocabulary

⟨ *Selfhost: emit_x86 preamble* 769 ⟩ ≡

```
void kcc_as_init(void);
void kcc_as_str(const char *s);
void kcc_as_ch(unsigned c);
void kcc_as_dec(unsigned long n);
int kcc_as_write_file(const char *path);
void *kcc_vec_init(unsigned long elemsize);
void kcc_vec_push(void *vec, const void *elem);
void *kcc_vec_get(void *vec, unsigned long i);
void *kcc_sema_resolve_type(void *node);
unsigned long kcc_type_layout(void *ty);
int kcc_strcmp(const char *a, const char *b);
void kcc_io_err(const char *s);
void kcc_io_flush(void);
void kcc_sys_exit(long code);
int kcc_global_const_rodata(void *decl);
int kcc_global_const_ptrarr(void *decl);
int kcc_global_sym_data(void *decl);
int kcc_global_writable(void *decl);
int kcc_global_static_ok(void *decl);
const char *kcc_ir_const_symaddr(void *n, long *
   off);
unsigned long kcc_eval_const_int(void *n);
unsigned long kcc_expr_is_ice(void *n);
void *kcc_arena_alloc(long size);
struct kcc_member_res { void *field; unsigned long
    off; };
struct kcc_member_res kcc_type_find_member(void *
   sty, const char *name);
extern void *kcc_ir_static_decls;      /* every
   static-local decl (ir.c) */
unsigned long kcc_rodata_leaf_bits(void *node,
   unsigned long kind);
const char *kcc_ir_static_name(void *decl);
void *kcc_ht_init(unsigned long capacity);
unsigned long kcc_ht_get(void *ht, const void *key
   , unsigned long keylen);
void kcc_ht_put(void *ht, const void *key,
   unsigned long keylen, unsigned long value);
unsigned long kcc_strlen(const char *s);
/* Linkage-attribute side table populated by the
   parser (48t). */
struct AttrRec { unsigned long weak; const char *
   section; const char *alias; };
extern void *kcc_attr_ht;
static struct AttrRec *x86_attr(const char *name)
   {
    if (!kcc_attr_ht || !name) return 0;
    return (struct AttrRec *)kcc_ht_get(
   kcc_attr_ht, name, kcc_strlen(name));
}

extern void *kcc_ir_funcs;           /* vec of IR
   function pointers */
extern char kcc_str_modinit[];       /* "kcc.
   module_init" */
extern unsigned long kcc_modinit_present;

/* IR instruction / block / function / type / vec
   / AST field offsets */
#define IR_OP 0
#define IR_DEST 8
#define IR_SRC1 16
#define IR_SRC2 24
#define IR_TYPE 32
#define IR_NEXT 40
#define BB_ID 0
#define BB_FIRST 8
#define IRF_NAME 0
#define IRF_BLOCKS 16
#define IRF_VREG_N 24
#define IRF_PARAMS 40
#define IRF_PARAM_CLS 48
#define IRF_STATIC 72
#define IRF_VARIADIC 88
#define VEC_COUNT 8
#define TY_KIND 0
#define TY_SIZE 8
#define TY_F0 32
#define TY_VOID 1
#define TY_INT 5
#define TY_STRUCT 14
#define TY_ARRAY 12
#define TY_FUNC 13
#define TY_PTR 11
#define TY_FLOAT 8
#define TY_DOUBLE 9
#define TY_UNION 15
#define AST_COMPOUND_LIT 67
#define AST_PAREN_EXPR 68
#define AST_TAG 0
#define AST_COL 16
#define AST_F0 24
#define AST_F1 32
#define AST_F2 40
#define AST_F3 48
#define AST_F4 56

/* IR opcodes */
enum {
    IR_ADD=1, IR_SUB=2, IR_MUL=3, IR_SDIV=4,
   IR_UDIV=5, IR_SMOD=6, IR_UMOD=7,
    IR_NEG=8, IR_AND=10, IR_OR=11, IR_XOR=12,
```

```
    IR_SHL=13, IR_LSHR=14, IR_ASHR=15,
     IR_NOT=16, IR_CMP_EQ=20, IR_CMP_NE=21,
    IR_CMP_SLT=22, IR_CMP_SLE=23,
     IR_CMP_SGT=24, IR_CMP_SGE=25, IR_CMP_ULT=26,
    IR_CMP_ULE=27,
     IR_LOAD=30, IR_STORE=31, IR_ALLOCA=32, IR_GEP
    =33, IR_BR=40, IR_BR_COND=41,
     IR_RET=42, IR_CALL=43, IR_CALL_IND=44, IR_SEXT
    =50, IR_ZEXT=51, IR_TRUNC=52,
     IR_BITCAST=53, IR_SITOFP=54, IR_FPTOSI=55,
    IR_VA_START=56, IR_FPROUND=57, IR_UITOFP=58,
     IR_CONST=60, IR_NOP=61, IR_GLOBAL_REF=63,
    IR_STRING_LIT=64, IR_ASM=65,
     IR_LADDR=66, IR_BR_IND=67, IR_FRAME_ADDR=68,
    IR_RET_ADDR=69
};
/* AST tags used by the data/static walks */
enum {
     AST_VAR_DECL=10, AST_FUNC_DECL=11, AST_STR_LIT
    =52, AST_ASM_STMT=74, AST_COMPOUND=30,
    AST_IF_STMT=32,
     AST_SWITCH_STMT=33, AST_WHILE_STMT=34,
    AST_DO_STMT=35, AST_FOR_STMT=36,
     AST_CASE_STMT=42, AST_LABEL_STMT=41,
    AST_DEFAULT_STMT=43, AST_INIT_LIST=65,
     AST_IDENT_EXPR=50, AST_UNARY_EXPR=55,
    AST_CAST_EXPR=61, AST_DESIGNATOR=66
};

/* exported target-selection state (the driver
    sets these) */
unsigned long kcc_target = 0;         /* 0 = arm64/
    Mach-O, 1 = x86-64 */
void *kcc_x86_tu = 0;                 /* AST
    translation unit (data emission) */
unsigned long kcc_x86_os = 0;         /* 0 = Darwin
    /Mach-O, 1 = Linux/ELF */
static unsigned long kcc_x86_vreg_n = 0;
static void *kcc_x86_strs = 0;
static unsigned long kcc_x86_rsa_top = 0;   /* rbp
    -relative top of the variadic register-save
    area */

#define S(name, str) static const char name[] =
    str
S(s_text, ".text\n"); S(s_globl, ".globl "); S(
    s_us, "_");
S(s_weak, ".weak "); S(s_section, ".section "); S(
    s_set, "\t.set "); S(s_prev, "\t.previous\n");
S(s_bss, ".bss\n"); S(s_ax, ",\"ax\",@progbits\n")
    ; S(s_aw, ",\"aw\"\n");
S(s_data, ".data\n"); S(s_plus, "+");
S(s_rodata, ".section .rodata\n"); S(s_comm, "\t.
    comm "); S(s_comm_a8, ",8\n");
S(s_local, "\t.local ");
S(s_modinit_d, ".section __DATA,__mod_init_func,
    mod_init_funcs\n\t.p2align 3\n\t.quad ");
S(s_modinit_l, ".section .init_array,\"aw\"\n\t.
    p2align 3\n\t.quad ");
S(s_modinitnm, "kcc.module_init");
S(s_constsect, ".section __TEXT,__const\n"); S(
    s_p2align3, "\t.p2align 3\n");
S(s_d_byte, "\t.byte "); S(s_d_short, "\t.short ")
    ; S(s_d_long, "\t.long ");
S(s_d_quad, "\t.quad "); S(s_zerodir, "\t.zero ");
     S(s_colon_nl, ":\n");
S(s_minus, "-"); S(s_rbp, "(%rbp)"); S(s_comma, ",
    ");
S(s_push_rbp, "\tpushq\t%rbp\n"); S(s_mov_sp, "\
    tmovq\t%rsp, %rbp\n");
S(s_sub_sp, "\tsubq\t$"); S(s_sp_nl, ", %rsp\n");
    S(s_leave, "\tleave\n");
S(s_retq, "\tretq\n"); S(s_movq, "\tmovq\t"); S(
    s_movabsq, "\tmovabsq\t$");
S(s_movl, "\tmovl\t"); S(s_movb, "\tmovb\t"); S(
    s_movw, "\tmovw\t");
S(s_movslq, "\tmovslq\t"); S(s_movsbq, "\tmovsbq\t
    "); S(s_movswq, "\tmovswq\t");
S(s_movzbq, "\tmovzbq\t"); S(s_movzwq, "\tmovzwq\t
    "); S(s_leaq, "\tleaq\t");
S(s_addq, "\taddq\t"); S(s_subq, "\tsubq\t"); S(
    s_imulq, "\timulq\t");
S(s_andq, "\tandq\t"); S(s_orq, "\torq\t"); S(
    s_xorq, "\txorq\t");
S(s_negq, "\tnegq\t"); S(s_notq, "\tnotq\t"); S(
    s_shlq, "\tshlq\t");
S(s_sarq, "\tsarq\t"); S(s_shrq, "\tshrq\t"); S(
    s_cqto, "\tcqto\n");
S(s_xor_rdx, "\txorq\t%rdx, %rdx\n"); S(s_idiv_rcx
    , "\tidivq\t%rcx\n");
S(s_div_rcx, "\tdivq\t%rcx\n");
S(s_r10, "%r10"); S(s_r11, "%r11"); S(s_rax, "%rax
    "); S(s_rcx, "%rcx");
S(s_rdx, "%rdx"); S(s_r11_r10, "%r11, %r10\n"); S(
    s_r10_nl, "%r10\n");
S(s_rbx, "%rbx"); S(s_push_rbx, "\tpushq\t%rbx\n")
    ; S(s_pop_rbx, "\tpopq\t%rbx\n");
S(s_setcc, "\tset"); S(s_tab_al, "\t%al\n"); S(
    s_movzx_al, "\tmovzbl\t%al, %eax\n");
S(s_r12, "%r12"); S(s_r13, "%r13"); S(s_r14, "%r14
    ");
S(s_push_r12, "\tpushq\t%r12\n"); S(s_push_r13, "\
    tpushq\t%r13\n");
S(s_push_r14, "\tpushq\t%r14\n"); S(s_pop_r12, "\
    tpopq\t%r12\n");
S(s_pop_r13, "\tpopq\t%r13\n"); S(s_pop_r14, "\
    tpopq\t%r14\n");
S(s_cl_r10, "%cl, %r10\n"); S(s_c_r10, ", %r10\n")
    ; S(s_c_r10d, ", %r10d\n");
S(s_pr11_r10, "(%r11), %r10\n"); S(s_pr11_r10d, "
    (%r11), %r10d\n");
S(s_pr11, "(%r11)");
S(s_r10_pr11, "%r10, (%r11)\n"); S(s_r10b_pr11, "%
    r10b, (%r11)\n");
S(s_r10w_pr11, "%r10w, (%r11)\n"); S(s_r10d_pr11,
    "%r10d, (%r11)\n");
S(s_cmpl, "\tcmpl\t%r11d, %r10d\n"); S(s_cmpq, "\
    tcmpq\t%r11, %r10\n");
S(s_movzbl_al, "\tmovzbl\t%al, %r10d\n"); S(s_sete
    , "\tsete\t%al\n");
S(s_setne, "\tsetne\t%al\n"); S(s_setl, "\tsetl\t%
    al\n"); S(s_setle, "\tsetle\t%al\n");
S(s_setg, "\tsetg\t%al\n"); S(s_setge, "\tsetge\t%
    al\n"); S(s_setb, "\tsetb\t%al\n");
S(s_setbe, "\tsetbe\t%al\n"); S(s_bb, ".Lkccbb");
    S(s_label_nl, ":\n");
S(s_jmp, "\tjmp\t"); S(s_jne, "\tjne\t"); S(
    s_test_r10, "\ttestq\t%r10, %r10\n");
S(s_jmp_ind_r10, "\tjmp\t*%r10\n");
S(s_rdi, "%rdi"); S(s_rsi, "%rsi"); S(s_r8, "%r8")
    ; S(s_r9, "%r9");
S(s_call, "\tcall\t"); S(s_call_r11, "\tcall\t*%
    r11\n"); S(s_zero_al, "\tmovb\t$0, %al\n");
S(s_movb_d, "\tmovb\t$"); S(s_al_nl, ", %al\n");
S(s_add_rsp, "\taddq\t$"); S(s_paren_rsp, "(%rsp)"
    ); S(s_rip_r10, "(%rip), %r10\n");
S(s_rip, "(%rip)");
S(s_mov_rbp_r10, "\tmovq\t%rbp, %r10\n");
     /* frame_address(0) seed */
S(s_mov_r10ind_r10, "\tmovq\t(%r10), %r10\n");
     /* walk to the caller's frame */
S(s_mov_8r10_r10, "\tmovq\t8(%r10), %r10\n");
```

```
    /* return address in that frame */
S(s_got_r10, "@GOTPCREL(%rip), %r10\n");
S(s_zerofill, "\t.zerofill __DATA,__common,_"); S(
   s_zf_a3, ",3\n"); S(s_comma1, ",");
S(s_unsup, "kcc: --target=x86_64: unsupported
   operation (floating point, inline asm, or by-
   value aggregate not yet implemented for the x86
   -64 back end)\n");
S(s_strlbl, ".Lx86str"); S(s_cstrsect, ".section
   __TEXT,__cstring,cstring_literals\n");
S(s_zero_nl, "0\n");
/* Floating point (rung 4): SSE moves, arithmetic,
    and conversions.  A value
   lives in the same 8-byte slot as an integer (
   the IEEE double bit pattern);
   %xmm0/%xmm1 are the scratch pair, single
   precision only at the memory edge. */
S(s_movsd, "\tmovsd\t"); S(s_movss, "\tmovss\t");
S(s_cx0_nl, ", %xmm0\n"); S(s_cx1_nl, ", %xmm1\n")
   ; S(s_x0_c, "%xmm0, "); S(s_nl, "\n");
S(s_addsd, "\taddsd\t"); S(s_subsd, "\tsubsd\t");
   S(s_mulsd, "\tmulsd\t");
S(s_divsd, "\tdivsd\t"); S(s_x1_x0, "%xmm1, %xmm0\
   n");
S(s_cvtsi2sd, "\tcvtsi2sdq\t%r10, %xmm0\n"); S(
   s_cvttsd2si, "\tcvttsd2si\t%xmm0, %r10\n");
S(s_cvtsd2ss, "\tcvtsd2ss\t%xmm0, %xmm0\n"); S(
   s_cvtss2sd, "\tcvtss2sd\t%xmm0, %xmm0\n");
S(s_px11_x0, "(%r11), %xmm0\n"); S(s_x0_px11, "%
   xmm0, (%r11)\n");
static const char *const argregs[6] = { s_rdi,
   s_rsi, s_rdx, s_rcx, s_r8, s_r9 };
S(s_xmm0, "%xmm0"); S(s_xmm1, "%xmm1"); S(s_xmm2,
   "%xmm2"); S(s_xmm3, "%xmm3");
S(s_xmm4, "%xmm4"); S(s_xmm5, "%xmm5"); S(s_xmm6,
   "%xmm6"); S(s_xmm7, "%xmm7");
static const char *const xmmregs[8] = { s_xmm0,
   s_xmm1, s_xmm2, s_xmm3, s_xmm4, s_xmm5, s_xmm6,
    s_xmm7 };

#define IL(i,off) (*(unsigned long *)((char *)(i)
   + (off)))
struct Type { unsigned long kind, size, align,
   quals, f0, f1; };
struct Node { unsigned long tag, line, col, f0, f1
   , f2, f3, f4; };

/* forward decls */
static void x86_emit_rodata_global(void *decl,
   const char *name);
static void x86_emit_rodata_ptrarr(void *decl,
   const char *name);
static void x86_emit_sym_data(void *decl, const
   char *name);
static void x86_emit_general(void *decl, const
   char *name);
static void x86_emit_zero(unsigned long n);
static void x86_emit_value(unsigned long bits,
   unsigned long size);
static void x86_prereg_stmt(void *stmt);
```

□

769 · used in 792a

72.2. Symbol and Operand Helpers

x86_sym prefixes a symbol with an underscore on Darwin only. x86_disp/x86_slot print a frame displacement and a vreg's slot operand; x86_load/x86_store move a slot to or from a named register.

⟨ *Selfhost: emit_x86 helpers* 771 ⟩ ≡

```
static void x86_sym(const char *name) {
    if (!kcc_x86_os) kcc_as_str(s_us);          /*
    Darwin underscore */
    kcc_as_str(name);
}
static void x86_disp(unsigned long d) {
    kcc_as_str(s_minus); kcc_as_dec(d); kcc_as_str
   (s_rbp);
}
static void x86_slot(unsigned long v) { x86_disp((
   v + 1) * 8); }
/* All-spill leaves every value in a stack slot,
   so a dependent chain emits
    `movq %r10, SLOT' immediately followed by `movq
    SLOT, %r10' --- a store then a
   reload of a value that is already in the
   register.  x86_r10_slot names the vreg
   whose current value provably sits in %r10 (or
   -1 for "unknown"), so a reload of
   that same slot into %r10 can be skipped.  Only
   %r10 is tracked; %r11 is the
   address scratch and turns over too fast to
   cache.  The invariant is that at
   every x86_load entry %r10 holds x86_r10_slot's
   value: it is set whenever %r10 is
   loaded from or stored to a slot, cleared when a
    slot it names is overwritten
   from another register, and reset at every block
    entry and across every %r10
   clobber (calls, inline asm, va_start). */
static long x86_r10_slot = -1;
static void x86_load(unsigned long v, const char *
   reg) {
    if (reg == s_r10) {
        if (x86_r10_slot == (long)v) return;
      /* already in %r10 */
        x86_r10_slot = (long)v;
    }
    kcc_as_str(s_movq); x86_slot(v); kcc_as_str(
   s_comma); kcc_as_str(reg); kcc_as_ch('\n');
}
static void x86_store(unsigned long v, const char
   *reg) {
    kcc_as_str(s_movq); kcc_as_str(reg);
   kcc_as_str(s_comma); x86_slot(v); kcc_as_ch('\n
   ');
    if (reg == s_r10) x86_r10_slot = (long)v;
      /* %r10 and slot v now equal */
    else if (x86_r10_slot == (long)v) x86_r10_slot
    = -1;  /* slot v changed; %r10 stale */
}
```

□

771 · used in 792a

72.3. Instruction Shapes

The ALU, shift, division, conversion, memory, comparison, and call shapes each lower one IR instruction to its x86-64 form, leaving the result in DEST's slot.

Because the frame is all-spill, a dependent chain such as a = b + c; d = a + e; lowers to a store of a to its slot immediately followed by a reload of that same slot into %r10—the value is already in the register. x86_load/x86_store therefore carry a one-entry cache, x86_-

r10_slot, naming the vreg whose value provably occupies %r10; a reload of that slot into %r10 is elided. The invariant holds because every shape ends either by storing %r10 to DEST (which re-establishes the cache) or without touching %r10 at all (the divide and the floating shapes), so the intermediate scratch writes inside a shape are always resealed before the next shape's first load. It is reset at every block entry—a block is reachable from several predecessors, so %r10's contents are unknown there—and across every genuine %r10 clobber: a call, an inline-asm body, or the va_start sequence. Only %r10 is tracked; %r11, the address scratch, turns over on nearly every memory shape and is not worth caching. This removes about a fifth of all stores' matching reloads—the single densest redundancy in −O0-class all-spill code.

⟨ *Selfhost: emit_x86 shapes* 772 ⟩ ≡

```
static void x86_binop(void *i, const char *mnem) {
    x86_load(IL(i, IR_SRC1), s_r10);
    x86_load(IL(i, IR_SRC2), s_r11);
    kcc_as_str(mnem); kcc_as_str(s_r11_r10);
    x86_store(IL(i, IR_DEST), s_r10);
}
static void x86_unop(void *i, const char *mnem) {
    x86_load(IL(i, IR_SRC1), s_r10);
    kcc_as_str(mnem); kcc_as_str(s_r10_nl);
    x86_store(IL(i, IR_DEST), s_r10);
}
static void x86_shift(void *i, const char *mnem) {
    x86_load(IL(i, IR_SRC1), s_r10);
    x86_load(IL(i, IR_SRC2), s_rcx);
    kcc_as_str(mnem); kcc_as_str(s_cl_r10);
    x86_store(IL(i, IR_DEST), s_r10);
}
static void x86_divop(void *i, int is_signed, int
   want_rem) {
    x86_load(IL(i, IR_SRC1), s_rax);
    kcc_as_str(is_signed ? s_cqto : s_xor_rdx);
    x86_load(IL(i, IR_SRC2), s_rcx);
    kcc_as_str(is_signed ? s_idiv_rcx : s_div_rcx)
   ;
    x86_store(IL(i, IR_DEST), want_rem ? s_rdx :
   s_rax);
}
static void x86_extend(void *i, const char *mnem,
   const char *tail) {
    kcc_as_str(mnem); x86_slot(IL(i, IR_SRC1));
   kcc_as_str(tail);
    x86_store(IL(i, IR_DEST), s_r10);
}
static void x86_conv(void *i, unsigned long width,
    int is_signed) {
    if (width == 1) x86_extend(i, is_signed ?
   s_movsbq : s_movzbq, s_c_r10);
    else if (width == 2) x86_extend(i, is_signed ?
    s_movswq : s_movzwq, s_c_r10);
    else if (width == 4) {
        if (is_signed) x86_extend(i, s_movslq,
   s_c_r10);
        else x86_extend(i, s_movl, s_c_r10d);
    } else x86_extend(i, s_movq, s_c_r10);
   /* 8 / unknown: plain copy */
}
static void x86_memload(void *i, const char *mnem,
    const char *tail) {
    x86_load(IL(i, IR_SRC1), s_r11);
    kcc_as_str(mnem); kcc_as_str(tail);
    x86_store(IL(i, IR_DEST), s_r10);
}
static void x86_memstore(void *i, const char *mnem
   , const char *valtail) {
    x86_load(IL(i, IR_SRC1), s_r10);
    x86_load(IL(i, IR_SRC2), s_r11);
    kcc_as_str(mnem); kcc_as_str(valtail);
}
static void x86_cmp(void *i, const char *setcc) {
    x86_load(IL(i, IR_SRC1), s_r10);
    x86_load(IL(i, IR_SRC2), s_r11);
    struct Type *ty = (struct Type *)IL(i, IR_TYPE
   );
    if (!ty || ty->size == 8) kcc_as_str(s_cmpq);
    else kcc_as_str(s_cmpl);
    kcc_as_str(setcc); kcc_as_str(s_movzbl_al);
    x86_store(IL(i, IR_DEST), s_r10);
}
static int is_fp_type(void *ty) {
    if (!ty) return 0;
    unsigned long k = IL(ty, TY_KIND);
    return k == TY_FLOAT || k == TY_DOUBLE;
}
static void x86_fpload(unsigned long v, const char
    *tail) {
    kcc_as_str(s_movsd); x86_slot(v); kcc_as_str(
   tail);
}
static void x86_fpstore(unsigned long v) {
    kcc_as_str(s_movsd); kcc_as_str(s_x0_c);
   x86_slot(v); kcc_as_str(s_nl);
}
static void x86_fpbinop(void *i, const char *mnem)
    {
    x86_fpload(IL(i, IR_SRC1), s_cx0_nl);
    x86_fpload(IL(i, IR_SRC2), s_cx1_nl);
    kcc_as_str(mnem); kcc_as_str(s_x1_x0);
    x86_fpstore(IL(i, IR_DEST));
}
static void x86_bblabel(unsigned long id) {
   kcc_as_str(s_bb); kcc_as_dec(id); }

static unsigned long kcc_x86_call_nfp = 0;
/* `movq %r10, off(%rsp)` - copy a scratch word to
    the outgoing stack area. */
static void x86_push_r10(unsigned long off) {
    kcc_as_str(s_movq); kcc_as_str(s_r10);
   kcc_as_str(s_comma);
    kcc_as_dec(off); kcc_as_str(s_paren_rsp);
   kcc_as_ch('\n');
}
/* `movq disp(%r11), dst` - load one word of a by-
   value aggregate (r11 = its
   address) into a GP argument register or scratch
   . */
static void x86_load_word(unsigned long disp,
   const char *dst) {
    kcc_as_str(s_movq); kcc_as_dec(disp);
   kcc_as_str(s_pr11);
    kcc_as_str(s_comma); kcc_as_str(dst);
   kcc_as_ch('\n');
}
static unsigned long x86_marshal(unsigned long *
   args, unsigned long count) {
    kcc_x86_call_nfp = 0;
    if (!count) return 0;
    /* System-V argument classification: class-1 (
   float/double) → %xmm0-7;
        class-2/3 aggregates travel by VALUE in one
```

```
    /two GP registers loaded from
        their address vreg; everything else in %rdi
    ..%r9.  Whatever overflows a
        bank is copied to the outgoing stack in
    argument order. */
     unsigned long ngp = 0, nfp = 0, nstk = 0;
     for (unsigned long k = 0; k < count; k++) {
         unsigned long c = args[2 + 2 * k];
         if (c == 1)      { if (nfp < 8) nfp++;
    else nstk++; }
         else if (c == 3) { if (ngp + 1 < 6) ngp +=
     2; else nstk += 2; }
         else             { if (ngp < 6) ngp++;
    else nstk++; }
     }
     unsigned long stack_adj = 0;
     if (nstk) {
         stack_adj = ((nstk * 8) + 15) & ~15UL;
         kcc_as_str(s_sub_sp); kcc_as_dec(stack_adj
    ); kcc_as_str(s_sp_nl);
     }
     kcc_x86_call_nfp = nfp;
     ngp = 0; nfp = 0;
     unsigned long soff = 0;
     for (unsigned long k = 0; k < count; k++) {
         unsigned long v = args[1 + 2 * k];
         unsigned long c = args[2 + 2 * k];
         if (c == 1) {
             if (nfp < 8) {
                 kcc_as_str(s_movsd); x86_slot(v);
    kcc_as_str(s_comma);
                 kcc_as_str(xmmregs[nfp]);
    kcc_as_ch('\n'); nfp++;
             } else { x86_load(v, s_r10);
    x86_push_r10(soff); soff += 8; }
         } else if (c == 3) {
             if (ngp + 1 < 6) {
                 x86_load(v, s_r11);
                 x86_load_word(0, argregs[ngp]);
    x86_load_word(8, argregs[ngp + 1]);
                 ngp += 2;
             } else {
                 x86_load(v, s_r11);
                 x86_load_word(0, s_r10);
    x86_push_r10(soff); soff += 8;
                 x86_load_word(8, s_r10);
    x86_push_r10(soff); soff += 8;
             }
         } else if (c == 2) {
             if (ngp < 6) { x86_load(v, s_r11);
    x86_load_word(0, argregs[ngp]); ngp++; }
             else { x86_load(v, s_r11);
    x86_load_word(0, s_r10); x86_push_r10(soff);
    soff += 8; }
         } else {
             if (ngp < 6) { x86_load(v, argregs[ngp
    ]); ngp++; }
             else { x86_load(v, s_r10);
    x86_push_r10(soff); soff += 8; }
         }
     }
     return stack_adj;
}
/* System~V requires %al to hold the number of
    vector registers used when
    calling a variadic (or unprototyped) function;
    benign for the rest. */
static void x86_set_al(void) {
     kcc_as_str(s_movb_d); kcc_as_dec(
    kcc_x86_call_nfp); kcc_as_str(s_al_nl);
}
static unsigned long x86_call_args(unsigned long *
    args) {
     if (!args) return 0;
     return x86_marshal(args, args[0] & 0xffffffff)
    ;
}
```

□

772 · used in 792a

72.4. The Dispatch

x86_sel routes one IR instruction to its shape. IR_CONST, IR_ALLOCA, IR_RET, loads, stores, branches, calls, global references, string literals, and extended inline asm are lowered inline; the ALU/shift/div/conv and floating-point opcodes go through the shape helpers. A genuine IR_NOP emits nothing; an unrecognised opcode (a by-value aggregate the marshaller cannot classify) is refused cleanly rather than miscompiled.

Extended inline asm (IR_ASM) is where the kernel's word-at-a-time and atomic idioms live, so x86_emit_asm renders it directly. Register constraints ("r", "=r", "+r", an early-clobber & ignored)—together with "g" (general: reg/mem/imm), "q" (byte-addressable register, which on x86-64 is any register), and "p" (address), all of which kcc materialises into a register just like "r", and which dominate the kernel's inline asm—and matching constraints ("0", "1", …) are honoured. An operand may also be named, and %[name] in the template resolves to it the same way %N does. The six physical registers %r8, %r9, %r10, %r12, %r13, %r14—the first three caller-saved, so the surrounding all-spill code survives the asm untouched, and %r12–%r14, callee-saved, bracketed with pushq/popq when the operand count spills into them—are handed out from a *compact* pool (nphys) drawn only by operands that actually need one: an r/g/q/p register operand or an m memory operand. An immediate, a symbolic %P operand, a string literal, and a condition-code =@cc output all render inline and consume no register, so a template may name well more than six operands as long as at most six of them require a register. (Earlier the sixteen-entry operand arrays were indexed one-to-one with the physical registers, capping the *total* operand count at six; the kernel's const_ilog2/test_bit sites routinely name eight or more, most of them immediates, and were refused on the operand count alone.) Only when the register-needing operands themselves exceed six—or the array holds more than sixteen operands total—is the asm refused rather than silently mis-substituted. %r11 stays reserved as the scratch that carries an output operand's address. Beyond the positional r registers, an operand may be pinned to a specific register—a/b/c/d for %rax/%rbx/%rcx/%rdx and S/D for %rsi/%rdi—as mul/div (a/d), cpuid, rdtsc, and shift-by-cl (c) require; its value is moved in and out of that register and the template renders it at the operand's width (%eax, %cl, …). Because %rbx is callee-saved, an asm that pins an operand to it is bracketed with pushq/popq %rbx so the caller's value survives; the others are caller-saved and need no save. A specific register can also be the register *alternative* of a multi-alternative constraint whose other alternative is an immediate: the in/out port constraint

"Nd" means an immediate port or %dx, so a non-constant port must land in %dx, not a positional register. When the constraint that reached the register path carries no general-register letter (r/q/g/p) but does name a specific one, that specific register is used. A condition-code output ("=@cccond", as test_and_set_bit's "=@ccc") takes no register and is not named in the template: after the body a setcond captures the still-live flag into %al, which is zero-extended and stored—the condition suffix maps straight onto the setCC mnemonic (c, z, l, …). Register outputs are stored first, with plain movs that leave the flags intact for these. A matching constraint does not take a register of its own: it *aliases* the tied operand's register (specific one included), which is exactly the register-tie GCC guarantees so a value seeded through one operand can be read or written as another—as in arch/x86's variable_ffs, bsfl %1,%0 : "=r"(r) : "rm"(x), "0"(-1), where -1 is loaded into the output's register so a zero input leaves it unchanged. Each input value is loaded into its assigned register first; then the template is pasted verbatim except that %% collapses to a literal % and %N (optionally size-prefixed %bN/%wN/%kN/%qN) expands to operand N's register at the operand's own byte width, so an int operand emits %r8d while a long emits %r8; finally each output register is stored back through %r11 to the operand's address at that width. Because those operand registers live in the AST node's operand list rather than in the IR instruction's source fields, the optimiser must renumber and mark them explicitly (see the optimiser's CSE and dead-code passes)—otherwise store-to-load forwarding would leave an input reading a swept load's slot.

A related modifier is %V (and its lowercase %v): it prints a *register* operand's name without the leading %, so that a register can be spliced into a symbol name. The kernel's retpoline CALL_NOSPEC uses call __x86_indirect_thunk_%V[thunk_target] with [thunk_target] "rm"(addr): kcc holds addr in a general register rN, and %V renders it as the bare rN, forming __x86_indirect_thunk_r11—a real per-register thunk symbol—rather than the un-substituted %V[thunk_target] the assembler rejects. A single bare flag, set when the modifier is seen, tells the register printer to drop the %.

The %p, %P and %c operand modifiers print an operand *bare*: a symbol or constant without the $/register decoration, so call %P[func] becomes a direct call func, movq %P[var], %r8 loads the object's address by name, and .4byte %c[feature] emits the bare feature number. The lowercase %p is GCC's spelling for the same raw-operand print and is what the kernel's RIP-relative accessors use—_ASM_RIP expands to %p[sym](%%rip), which must render as sym(%rip), not a register—so it is treated identically here. The kernel leans on these in ALTERNATIVE (call %P[old] / call %P[new]) and percpu templates, where the operand is a "i"-constrained function or a memory object, not a plain value. kcc materialises every asm operand into a register or immediate and so would otherwise have lost the name; to support %P the operand lowering records the emit-name of a symbolic operand (a function or file-scope object, or the address of one) alongside the materialised register—F1 holds the name and a marker bit in F2 flags its presence. Rendering a %P reference then emits that bare symbol; for a %P operand with no symbol it falls back to the bare value (a constant), a memory reference (%reg) (a memory operand, as in the percpu read movq %%gs:%P[var], %0 → movq %gs:(%r9), ...), or the raw address register (anything else). This makes the ALTERNATIVE/percpu translation units assemble; note that kcc does not emit the .altinstructions metadata that would patch the alternative at run time, so only the default (oldinstr) path is exercised.

A *plain* (non-modifier) %N on a symbolic memory operand needs the same by-name treatment. The commonest percpu read spells its access %%gs:%1 with "m"(var)—no %p modifier and no literal (%%rip) in the template—so %1 must itself expand to the object RIP-relative, var(%rip), giving %gs:var(%rip) exactly as GCC renders an "m" operand. The lowering already recorded the operand's symbol (its "m" address register is left unmaterialised), so a symbolic "m" operand keeps its ismem flag and, ahead of the generic (%reg) memory case, a plain reference prints sym+off(%rip) rather than an empty-register (%r8). This is what lets the many lib/, kernel/, and mm/ translation units whose percpu and this_cpu idioms name a file-scope object through "m" assemble once the operand count no longer caps them.

Floating point is a first-class citizen of both the call boundary and the return path. x86_marshal classifies each outgoing argument the System V way—class-1 (float/double) values into %xmm0–7; a small aggregate travelling by value into one (class 2, $\leq$8 bytes) or two (class 3, 9–16 bytes) integer registers, its words loaded from the address vreg; everything else into %rdi..%r9; and the overflow of a bank onto the outgoing stack in argument order—and records the vector-register count so the call site can load %al (mandatory when the callee is variadic). Because a class-3 aggregate occupies two parameter vregs (matching the IR builder's width count), x86_spill_params advances a separate base vreg cursor and spills two registers for it—getting this wrong dropped the sixth register of SQLite's yy_reduce (its trailing Parse * after a 16-byte Token argument), leaving a garbage pointer. Both IR_RET and the call-result store route a floating-point value through %xmm0 rather than %rax.

⟨ *Selfhost: emit_x86 dispatch* 774 ⟩ ≡

```
/* Extended inline asm: operand k is materialised
   in %r8+k (all caller-saved);
   %r11 stays free as the scratch that carries an
   output operand's address. The
   supported subset is `r`/`=r`/`+r` operands, up
   to three of them. */
/* Up to six positional register operands: r8-r10
   are caller-saved; r12-r14 are
   callee-saved and get a push/pop bracket when
   used.  r11 stays scratch. */
static const char *const asmregs[6] = { s_r8, s_r9
   , s_r10, s_r12, s_r13, s_r14 };
static const unsigned long asmregnum[6] = { 8, 9,
   10, 12, 13, 14 };
/* Emit operand k's register at the requested byte
    width: %r8 / %r8d / %r8w /
   %r8b for 8 / 4 / 2 / 1. */
static void x86_asm_wreg_b(unsigned long k,
   unsigned long size, int bare) {
```

```
    if (!bare) kcc_as_ch('%');
   /* %V modifier prints the bare name */
    kcc_as_ch('r'); kcc_as_dec(k < 6 ? asmregnum[k
   ] : 8 + k);
    if (size == 4) kcc_as_ch('d');
    else if (size == 2) kcc_as_ch('w');
    else if (size == 1) kcc_as_ch('b');
}
static void x86_asm_wreg(unsigned long k, unsigned
    long size) {
    x86_asm_wreg_b(k, size, 0);
}
static unsigned long x86_asm_opsize(void *node,
   unsigned long num) {
    void *op = (void *)IL(node, AST_F2);
    for (unsigned long j = 0; op && j < num; j++)
   op = (void *)IL(op, AST_F4);
    if (!op) return 8;
    /* mask off the top-two marker bits (63 =
   immediate, 62 = symbolic) so they
       neither flip the F2-vs-F3 selection nor
   bleed into the size in bits 32+ */
    unsigned long f2 = IL(op, AST_F2) & ~(7UL <<
   61);
    unsigned long e = f2 ? f2 : (IL(op, AST_F3) &
   ~(7UL << 61));
    unsigned long sz = e >> 32;
    return sz ? sz : 8;
}
/* The 64-bit name of a specific-register
   constraint (a/b/c/d/S/D), used to
   move an operand's value in or out of that
   register. */
static const char *x86_spec64(char rc) {
    if (rc == 'a') return s_rax;
    if (rc == 'b') return s_rbx;
    if (rc == 'c') return s_rcx;
    if (rc == 'd') return s_rdx;
    if (rc == 'S') return s_rsi;
    return s_rdi;
   /* 'D' */
}
/* Emit a specific register at the requested byte
   width, e.g. 'a'/4 -> %eax,
   'c'/1 -> %cl, 'S'/2 -> %si. */
static void x86_spec_reg_b(char rc, unsigned long
   size, int bare) {
    if (!bare) kcc_as_ch('%');
   /* %V: bare name (rax, not %rax) */
    if (rc == 'a' || rc == 'b' || rc == 'c' || rc
   == 'd') {
        if (size == 1) { kcc_as_ch(rc); kcc_as_ch(
   'l'); return; }
        if (size == 8) kcc_as_ch('r');
        else if (size == 4) kcc_as_ch('e');
        kcc_as_ch(rc); kcc_as_ch('x');
    } else {
   /* S -> si, D -> di */
        if (size == 8) kcc_as_ch('r');
        else if (size == 4) kcc_as_ch('e');
        kcc_as_ch(rc == 'S' ? 's' : 'd');
   kcc_as_ch('i');
        if (size == 1) kcc_as_ch('l');
    }
}
static void x86_spec_reg(char rc, unsigned long
   size) {
    x86_spec_reg_b(rc, size, 0);
}
/* An `asm goto' label reference (%lN or %l[name])
    expands to the target
   block's assembly label.  The goto labels hang
   off the AST node's F3 chain,
   each carrying its resolved block in F2. */
static void x86_asm_gotobb(void *bb) {
    if (bb) { kcc_as_str(s_bb); kcc_as_dec(IL(bb,
   BB_ID)); }
}
static void x86_asm_label_idx(void *node, unsigned
    long idx) {
    void *l = (void *)IL(node, AST_F3);
    for (unsigned long j = 0; l && j < idx; j++) l
   = (void *)IL(l, AST_F4);
    if (l) x86_asm_gotobb((void *)IL(l, AST_F2));
}
static void x86_asm_label_name(void *node, const
   char *nm, unsigned long nlen) {
    for (void *l = (void *)IL(node, AST_F3); l; l
   = (void *)IL(l, AST_F4)) {
        const char *ln = (const char *)IL(l,
   AST_F0);
        unsigned long j = 0;
        while (j < nlen && ln[j] && ln[j] == nm[j
   ]) j++;
        if (j == nlen && ln[j] == 0) {
   x86_asm_gotobb((void *)IL(l, AST_F2)); return;
   }
    }
}
/* Resolve a symbolic operand reference %[name] to
    its operand index.  The
   parser stores each operand's `[name]' in the
   operand node's F1 (a field the
   constraint length no longer needs, since the
   string is NUL-terminated).  An
   immediate keeps its name in F1 too (its value
   lives in F3), so a named
   immediate like the kernel's `%[bitnum]`
   resolves here just like any other. */
static unsigned long x86_asm_opnum(void *node,
   const char *nm, unsigned long nlen) {
    unsigned long k = 0;
    for (void *op = (void *)IL(node, AST_F2); op;
   op = (void *)IL(op, AST_F4), k++) {
        const char *on = (const char *)IL(op,
   AST_F1);
        if (!on) continue;
        unsigned long j = 0;
        while (j < nlen && on[j] && on[j] == nm[j
   ]) j++;
        if (j == nlen && on[j] == 0) return k;
    }
    return k;
        /* not found: out of range */
}
static void x86_emit_asm(void *i) {
    void *node = (void *)IL(i, IR_TYPE);
   /* the AST_ASM_STMT */
    /* regof[k] is the compact physical-register
   slot (0..5, indexing asmregnum)
       that operand k occupies: the next free slot
    for an ordinary register or
       memory operand, or the tied operand's slot
   for a matching constraint
       ("0", "1", ...), which shares that operand'
   s register (the register-alias
       GCC guarantees so arch code can seed a
   register through one operand and
       read/write it as another, e.g. `bsfl %1,%0
   : "=r"(r) : "rm"(x), "0"(-1)`).
       Operands that render inline---immediates, %
```

```
P symbols, string literals,
    condition-code outputs---draw no slot, so
nphys (not the operand index)
    measures how many of the six physical
registers are in use. */
 unsigned long regof[16];
 unsigned long ismem[16];
 /* operand k is a memory ref */
 unsigned long isimm[16];
  /* operand k is an immediate */
 unsigned long immval[16];
   /* its constant value */
 unsigned long issym[16];
   /* operand k names a symbol (for %P) */
 const char *symname[16];
   /* its emit-name */
 long symoff[16];
   /* +byte offset for name+off */
 unsigned long isstr[16];
   /* operand k is a string literal */
 const char *strbytes[16];
   /* its bytes */
 unsigned long strlen_[16];
    /* its length */
 unsigned long isaddr[16];
    /* operand k is an address ("p"): %P -> (%
reg) */
 char spec[16];
  /* specific-register letter or 0 */
 char isflag[16];
  /* condition-code output (=@cc) */
 const char *flagcond[16];
   /* its setCC suffix */
 unsigned long uses_rbx = 0;
 /* any operand pinned to %rbx */
 unsigned long max_areg = 0;
 /* highest compact register slot used */
 unsigned long nphys = 0;
  /* next free physical asm register (0..5) */
 unsigned long nop = 0;
 for (void *op = (void *)IL(node, AST_F2); op;
op = (void *)IL(op, AST_F4)) {
     if (nop >= 16) { kcc_io_err(s_unsup);
kcc_io_flush(); kcc_sys_exit(1); }
     const char *c = (const char *)IL(op,
AST_F0);
     char ch = c[0]; if (ch == '=' || ch == '+'
) ch = c[1];
     if (ch == '&') ch = c[2];
 /* early-clobber: ignored (each

    operand already has its own reg) */
     ismem[nop] = 0;
     isaddr[nop] = (ch == 'p');
 /* "p": an address held in a register */
     spec[nop] = 0;
     isflag[nop] = 0;
     /* `=@cc<cond>`: a condition-code output.
 It takes no register and is
        not named in the template; after the
asm a setCC captures the flag. */
     if (c[0] == '=' && c[1] == '@' && c[2] ==
'c' && c[3] == 'c') {
         isflag[nop] = 1;
         isimm[nop] = 0;
         flagcond[nop] = c + 4;
         regof[nop] = nop;
         nop++;
         continue;
     }
     /* the IR marks a resolved immediate with
the top bit of AST_F2 and puts
        the value in AST_F1; that decision (
constant vs register) already
        accounts for `Ir`-style alternatives.
*/
     isimm[nop] = (IL(op, AST_F2) >> 63) & 1;
     immval[nop] = IL(op, AST_F3);          /*
the constant value (F1 stays the name) */
     issym[nop] = (IL(op, AST_F2) >> 62) & 1;
  /* symbolic operand: name in COL (for %P) */
     symname[nop] = issym[nop] ? (const char *)
IL(op, AST_COL) : 0;
     symoff[nop] = issym[nop] ? (long)IL(op,
AST_F3) : 0;
     isstr[nop] = (IL(op, AST_F2) >> 61) & 1;
  /* string-literal operand: bytes in COL */
     strbytes[nop] = isstr[nop] ? (const char
*)IL(op, AST_COL) : 0;
     strlen_[nop] = isstr[nop] ? IL(op, AST_F3)
 : 0;
     regof[nop] = 0;
     /* Only operands that occupy one of the
six physical asm registers
        (asmregnum) draw from the compact pool;
 an immediate/symbolic/string
        operand renders as `$value'/`name'/`.
Lstr' and takes none, so a
        template may name far more than six
operands as long as at most six
        need a register. */
     unsigned long wants_reg = 0;
     if (isimm[nop] || issym[nop] || isstr[nop
]) {
         /* rendered inline; no register of its
 own.  A symbolic *memory*
            operand ("m"(sym)) still records
ismem so a plain `%N' reference
            prints the object RIP-relative (`
sym(%rip)') the way GCC does---
            the percpu read `%%gs:%1' with `"m
"(var)' becomes `%gs:var(%rip)'
            rather than dereferencing a
register kcc never loaded. */
         if (issym[nop] && ch == 'm') ismem[nop
] = 1;
     } else if (ch >= '0' && ch <= '9') {
         unsigned long tied = (unsigned long)(
ch - '0');
         if (tied >= nop) { kcc_io_err(s_unsup)
; kcc_io_flush(); kcc_sys_exit(1); }
         regof[nop] = regof[tied];
/* alias the tied operand's reg */
         spec[nop] = spec[tied];
/* ...including a specific one */
     } else if (ch == 'r' || ch == 'i' || ch ==
 'I' || ch == 'n' ||
                ch == 'g' || ch == 'q' || ch ==
 'p' || ch == 'e' ||
                ch == 'J' || ch == 'K' || ch ==
 'L' || ch == 'M' ||
                ch == 'N' || ch == 'O' || ch ==
 'Z') {
         /* register.  An immediate-class
letter that reached here (the IR did
            not mark the operand a constant)
means a multi-alternative
            constraint like `"er"` or `"Ir"`
whose register alternative is
            taken for a non-constant value.  `g
` (general: reg/mem/imm) and
            `p` (address) admit a register,
```

```
which is what kcc materialises the
            operand into; `q` (byte-addressable
 register) is any register on
            x86-64.  All are handled exactly
like `r`.

            But a constraint whose only
register alternative is a *specific*
            register---the x86 in/out port `"Nd
"` (immediate or %dx)---must use
            that register for a non-constant
value, not a positional one. */
          unsigned long has_gen = 0; char sp =
0;
          for (const char *cc = c; *cc; cc++) {
              if (*cc == 'r' || *cc == 'q' || *
cc == 'g' || *cc == 'p') has_gen = 1;
              else if (!sp && (*cc == 'a' || *cc
 == 'b' || *cc == 'c' ||
                               *cc == 'd' || *cc
 == 'S' || *cc == 'D')) sp = *cc;
          }
          if (!has_gen && sp) { spec[nop] = sp;
if (sp == 'b') uses_rbx = 1; }
          else wants_reg = 1;
/* a general register from the pool */
      } else if (ch == 'a' || ch == 'b' || ch ==
 'c' || ch == 'd' ||
                 ch == 'S' || ch == 'D') {
          /* pinned to a specific register (mul/
div use a/d, cpuid a/b/c/d,
            shift-by-cl uses c, string ops S/D)
. */
          spec[nop] = ch;
          if (ch == 'b') uses_rbx = 1;
/* callee-saved: save around */
      } else if (ch == 'm') {
          /* memory operand: its register holds
the object's *address*, and
            the template references it as %r8+k
 in parentheses. */
          ismem[nop] = 1;
          wants_reg = 1;
      } else { kcc_io_err(s_unsup); kcc_io_flush
(); kcc_sys_exit(1); }
      if (wants_reg) {
/* draw the next physical register */
          if (nphys >= 6) { kcc_io_err(s_unsup);
 kcc_io_flush(); kcc_sys_exit(1); }
          regof[nop] = nphys++;
      }
      /* track the highest register slot used (
for callee-save push/pop) */
      if (!spec[nop] && !isimm[nop] && !isflag[
nop] && regof[nop] > max_areg)
          max_areg = regof[nop];
      nop++;
  }
  /* Callee-saved registers used as asm operands
  (rbx, and r12-r14 once the
     positional operands spill past r10) are
preserved around the body. */
  if (uses_rbx) kcc_as_str(s_push_rbx);
  if (max_areg >= 3) kcc_as_str(s_push_r12);
  if (max_areg >= 4) kcc_as_str(s_push_r13);
  if (max_areg >= 5) kcc_as_str(s_push_r14);
  /* load each operand's register: an input's
value, or a memory operand's
     address (held in AST_F3 exactly like an
output's address).  Immediates
     need no register. */
  unsigned long k = 0;
  for (void *op = (void *)IL(node, AST_F2); op;
op = (void *)IL(op, AST_F4), k++) {
      if (isimm[k] || isflag[k] || issym[k] ||
isstr[k]) continue;
      const char *dst = spec[k] ? x86_spec64(
spec[k]) : asmregs[regof[k]];
      if (ismem[k]) {
          unsigned long a = IL(op, AST_F3) & 0
xffffffff;
          if (a) x86_load(a - 1, dst);
      } else {
          unsigned long v = IL(op, AST_F2) & 0
xffffffff;
          if (v) x86_load(v - 1, dst);
      }
  }
  /* paste the template, rewriting %% -> % and
%[bwkq]N -> operand register */
  const char *t = (const char *)IL(i, IR_SRC1);
  unsigned long len = IL(i, IR_SRC2);
  kcc_as_ch('\t');
  for (unsigned long p = 0; p < len; p++) {
      char c = t[p];
      if (c != '%') { kcc_as_ch(c); continue; }
      if (p + 1 >= len) { kcc_as_ch('%'); break;
 }
      char nx = t[p + 1];
      if (nx == '%') { kcc_as_ch('%'); p++;
continue; }
      if (nx == 'l') {
/* asm-goto label: %lN or %l[name] */
          p++;
/* consume 'l' */
          if (p + 1 < len && t[p + 1] == '[') {
              p++;
/* consume '[' */
              unsigned long ns = p + 1;
              while (p + 1 < len && t[p + 1] !=
']') p++;
              x86_asm_label_name(node, t + ns, (
p + 1) - ns);
              if (p + 1 < len) p++;
/* consume ']' */
          } else {
              unsigned long lnum = 0;
              while (p + 1 < len && t[p + 1] >=
'0' && t[p + 1] <= '9') {
                  lnum = lnum * 10 + (t[p + 1] -
 '0'); p++;
              }
              /* label numbers continue from the
 input/output operands */
              x86_asm_label_idx(node, lnum >=
nop ? lnum - nop : lnum);
          }
          continue;
      }
      unsigned long sz = 0;
/* 0 = the operand's own size */
      unsigned long raw = 0;
/* %p/%P/%c: render the operand bare (symbol/
const) */
      unsigned long bare = 0;
/* %V/%v: a register's name without the `%' */
      if (nx == 'p' || nx == 'P' || nx == 'c') {
 raw = 1; p++; nx = t[p + 1]; }
      else if (nx == 'V' || nx == 'v') { bare =
1; p++; nx = t[p + 1]; }
      else if (nx == 'b') { sz = 1; p++; nx = t[
p + 1]; }
```

```
    else if (nx == 'w') { sz = 2; p++; nx = t[
p + 1]; }
    else if (nx == 'k') { sz = 4; p++; nx = t[
p + 1]; }
    else if (nx == 'q') { sz = 8; p++; nx = t[
p + 1]; }
    /* An operand reference: a number (%0) or
a symbolic name (%[name]). */
    long num = -1;
    if (nx >= '0' && nx <= '9') {
        unsigned long n = 0;
        while (p + 1 < len && t[p + 1] >= '0'
&& t[p + 1] <= '9') { n = n * 10 + (t[p + 1] -
'0'); p++; }
        num = (long)n;
    } else if (nx == '[') {
        p++;
/* consume '[' */
        unsigned long ns = p + 1;
        while (p + 1 < len && t[p + 1] != ']')
 p++;
        num = (long)x86_asm_opnum(node, t + ns
, (p + 1) - ns);
        if (p + 1 < len) p++;
/* consume ']' */
    }
    if (num < 0) { kcc_as_ch('%'); continue; }
    unsigned long unum = (unsigned long)num;
    unsigned long reg = (unum < nop) ? regof[
unum] : unum;
    unsigned long owidth = sz ? sz :
x86_asm_opsize(node, unum);
    if (unum < nop && isstr[unum]) {
        /* string literal: emit it to .rodata
and reference the label, both for
           a bare `%c[f]' (a __bug_table `.
long %c0 - .') and a plain `%0'. */
        unsigned long sid = IL(kcc_x86_strs,
VEC_COUNT);
        unsigned long entry[2];
        entry[0] = (unsigned long)strbytes[
unum];
        entry[1] = strlen_[unum];
        kcc_vec_push(kcc_x86_strs, entry);
        kcc_as_str(s_strlbl); kcc_as_dec(sid);
    } else if (raw && unum < nop && issym[unum
]) {
        x86_sym(symname[unum]);
     /* %P[f] -> bare symbol: call f */
        if (symoff[unum] > 0) {
     /* &arr[k] / &s.field -> name+off */
            kcc_as_ch('+'); kcc_as_dec((
unsigned long)symoff[unum]);
        } else if (symoff[unum] < 0) {
            kcc_as_ch('-'); kcc_as_dec((
unsigned long)(-symoff[unum]));
        }
    } else if (raw && unum < nop && isimm[unum
]) {
        kcc_as_dec(immval[unum]);
     /* %P on a constant -> bare value, no $
*/
    } else if (raw && unum < nop && (ismem[
unum] || isaddr[unum])) {
        /* %P on a memory ("m") or address ("p
") operand---the kernel's percpu
           read `%%gs:%P[var]` with `[var] "p
"(&var)`: kcc holds the object's
           address in a register, so a memory
reference `(%reg)` is the
           bare-pointer form.  `%gs:(%r9)`
assembles and, like GCC's
           `%gs:var`, dereferences the object.
 */
        kcc_as_ch('('); x86_asm_wreg(reg, 8);
kcc_as_ch(')');
    } else if (raw && unum < nop) {
        x86_asm_wreg(reg, 8);
     /* %P without a symbol -> bare address
register */
    } else if (unum < nop && isimm[unum]) {
        kcc_as_ch('$'); kcc_as_dec(immval[unum
]);   /* $value */
    } else if (unum < nop && issym[unum]) {
        /* plain `%N' on a symbolic memory
operand ("m"(var)): the object
           itself, RIP-relative, exactly as
GCC renders an "m" operand.  With
           the template's literal `%%gs:' this
 yields the kernel percpu form
           `%gs:var(%rip)'.  No register was
loaded for it (it renders inline),
           so this must precede the generic
memory case, which would emit an
           empty-register `(%r8)'. */
        x86_sym(symname[unum]);
        if (symoff[unum] > 0) { kcc_as_ch('+')
; kcc_as_dec((unsigned long)symoff[unum]); }
        else if (symoff[unum] < 0) { kcc_as_ch
('-'); kcc_as_dec((unsigned long)(-symoff[unum
])); }
        kcc_as_str(s_rip);
    } else if (unum < nop && ismem[unum]) {
        kcc_as_ch('('); x86_asm_wreg(reg, 8);
kcc_as_ch(')');  /* (%reg) */
    } else if (unum < nop && spec[unum]) {
        x86_spec_reg_b(spec[unum], owidth, (
int)bare);
    } else {
        x86_asm_wreg_b(reg, owidth, (int)bare)
;  /* %V -> bare name, for
            `__x86_indirect_thunk_%V[target]'
(retpoline CALL_NOSPEC) */
    }
 }
 kcc_as_ch('\n');
 /* Store the register outputs first; these are
 all mov instructions, which
    leave the flags the asm set intact for the
condition-code outputs below
    (and store any %rax result before setCC
would overwrite %al). */
 k = 0;
 for (void *op = (void *)IL(node, AST_F2); op;
op = (void *)IL(op, AST_F4), k++) {
    if (ismem[k] || isimm[k] || isflag[k] ||
issym[k] || isstr[k]) continue;
    unsigned long ah = IL(op, AST_F3);
    unsigned long a = ah & 0xffffffff;
    if (!a) continue;
    x86_load(a - 1, s_r11);
/* r11 = &output */
    unsigned long sz = ah >> 32; if (!sz) sz =
 8;
    kcc_as_str(sz == 1 ? s_movb : sz == 2 ?
s_movw : sz == 4 ? s_movl : s_movq);
    if (spec[k]) x86_spec_reg(spec[k], sz);
    else x86_asm_wreg(regof[k], sz);
    kcc_as_str(s_comma); kcc_as_str(s_pr11);
kcc_as_ch('\n');
 }
```

```
    /* Condition-code outputs: setCC captures the
   still-live flag into %al,
       zero-extend, and store it at the
   destination's width. */
    k = 0;
    for (void *op = (void *)IL(node, AST_F2); op;
   op = (void *)IL(op, AST_F4), k++) {
        if (!isflag[k]) continue;
        unsigned long ah = IL(op, AST_F3);
        unsigned long a = ah & 0xffffffff;
        if (!a) continue;
        kcc_as_str(s_setcc); kcc_as_str(flagcond[k
   ]); kcc_as_str(s_tab_al);
        kcc_as_str(s_movzx_al);
        unsigned long sz = ah >> 32; if (!sz) sz =
    4;
        x86_load(a - 1, s_r11);
        kcc_as_str(sz == 1 ? s_movb : sz == 2 ?
   s_movw : sz == 4 ? s_movl : s_movq);
        x86_spec_reg('a', sz); kcc_as_str(s_comma)
   ; kcc_as_str(s_pr11); kcc_as_ch('\n');
    }
    if (max_areg >= 5) kcc_as_str(s_pop_r14);
    if (max_areg >= 4) kcc_as_str(s_pop_r13);
    if (max_areg >= 3) kcc_as_str(s_pop_r12);
    if (uses_rbx) kcc_as_str(s_pop_rbx);
}
void kcc_x86_sel(void *i) {
    unsigned long op = IL(i, IR_OP);
    switch (op) {
    case IR_CONST:
        kcc_as_str(s_movabsq); kcc_as_dec(IL(i,
   IR_SRC1)); kcc_as_str(s_c_r10);
        x86_store(IL(i, IR_DEST), s_r10);
        return;
    case IR_ALLOCA: {
        unsigned long off = IL(i, IR_SRC2);
        unsigned long size = IL(i, IR_SRC1);
        if (!size) size = 1;
        unsigned long d = (kcc_x86_vreg_n + off +
   size) * 8;
        kcc_as_str(s_leaq); x86_disp(d);
   kcc_as_str(s_c_r10);
        x86_store(IL(i, IR_DEST), s_r10);
        return;
    }
    case IR_RET: {
        struct Type *ty = (struct Type *)IL(i,
   IR_TYPE);
        if (ty && is_fp_type(ty)) x86_fpload(IL(i,
    IR_SRC1), s_cx0_nl);
        else if (ty && ty->kind != TY_VOID)
   x86_load(IL(i, IR_SRC1), s_rax);
        kcc_as_str(s_leave); kcc_as_str(s_retq);
        return;
    }
    case IR_LOAD: {
        unsigned long sz = IL(i, IR_SRC2);
        if (sz == 1) x86_memload(i, s_movzbq,
   s_pr11_r10);
        else if (sz == 2) x86_memload(i, s_movzwq,
    s_pr11_r10);
        else if (sz == 4) x86_memload(i, s_movl,
   s_pr11_r10d);
        else if (sz == 5) x86_memload(i, s_movsbq,
    s_pr11_r10);
        else if (sz == 6) x86_memload(i, s_movswq,
    s_pr11_r10);
        else if (sz == 7) x86_memload(i, s_movslq,
    s_pr11_r10);
        else if (sz == 8) {
   /* single float: movss + widen */
            x86_load(IL(i, IR_SRC1), s_r11);
            kcc_as_str(s_movss); kcc_as_str(
   s_px11_x0);
            kcc_as_str(s_cvtss2sd);
            x86_fpstore(IL(i, IR_DEST));
        }
        else x86_memload(i, s_movq, s_pr11_r10);
        return;
    }
    case IR_STORE: {
        unsigned long sz = IL(i, IR_DEST);
        if (sz == 1) x86_memstore(i, s_movb,
   s_r10b_pr11);
        else if (sz == 2) x86_memstore(i, s_movw,
   s_r10w_pr11);
        else if (sz == 4) x86_memstore(i, s_movl,
   s_r10d_pr11);
        else if (sz == 8) {
   /* single float: narrow + movss */
            x86_fpload(IL(i, IR_SRC1), s_cx0_nl);
            kcc_as_str(s_cvtsd2ss);
            x86_load(IL(i, IR_SRC2), s_r11);
            kcc_as_str(s_movss); kcc_as_str(
   s_x0_px11);
        }
        else x86_memstore(i, s_movq, s_r10_pr11);
        return;
    }
    /* A floating ADD/SUB/MUL/DIV (typed float/
   double) uses SSE; the shared
       IR_GEP that also lands on IR_ADD is integer
    address arithmetic. */
    case IR_ADD:
        if (is_fp_type((void *)IL(i, IR_TYPE)))
   x86_fpbinop(i, s_addsd);
        else x86_binop(i, s_addq);
        return;
    case IR_GEP: x86_binop(i, s_addq); return;
    case IR_SUB:
        if (is_fp_type((void *)IL(i, IR_TYPE)))
   x86_fpbinop(i, s_subsd);
        else x86_binop(i, s_subq);
        return;
    case IR_MUL:
        if (is_fp_type((void *)IL(i, IR_TYPE)))
   x86_fpbinop(i, s_mulsd);
        else x86_binop(i, s_imulq);
        return;
    case IR_AND: x86_binop(i, s_andq); return;
    case IR_OR: x86_binop(i, s_orq); return;
    case IR_XOR: x86_binop(i, s_xorq); return;
    case IR_NEG: x86_unop(i, s_negq); return;
    case IR_NOT: x86_unop(i, s_notq); return;
    case IR_SHL: x86_shift(i, s_shlq); return;
    case IR_LSHR: x86_shift(i, s_shrq); return;
    case IR_ASHR: x86_shift(i, s_sarq); return;
    case IR_SDIV:
        if (is_fp_type((void *)IL(i, IR_TYPE)))
   x86_fpbinop(i, s_divsd);
        else x86_divop(i, 1, 0);
        return;
    /* Floating-point conversions (rung 4); UITOFP
    shares the signed cvtsi2sd,
       as kcc's own source only converts signed
   integers to floating. */
    case IR_SITOFP: case IR_UITOFP:
        x86_load(IL(i, IR_SRC1), s_r10);
        kcc_as_str(s_cvtsi2sd);
        x86_fpstore(IL(i, IR_DEST));
```

```
        return;
    case IR_FPTOSI:
        x86_fpload(IL(i, IR_SRC1), s_cx0_nl);
        kcc_as_str(s_cvttsd2si);
        x86_store(IL(i, IR_DEST), s_r10);
        return;
    case IR_FPROUND:
        x86_fpload(IL(i, IR_SRC1), s_cx0_nl);
        kcc_as_str(s_cvtsd2ss);
        kcc_as_str(s_cvtss2sd);
        x86_fpstore(IL(i, IR_DEST));
        return;
    case IR_UDIV: x86_divop(i, 0, 0); return;
    case IR_SMOD: x86_divop(i, 1, 1); return;
    case IR_UMOD: x86_divop(i, 0, 1); return;
    case IR_SEXT: x86_conv(i, IL(i, IR_SRC2), 1);
return;
    case IR_ZEXT: x86_conv(i, IL(i, IR_SRC2), 0);
return;
    case IR_TRUNC: {
        struct Type *ty = (struct Type *)IL(i,
IR_TYPE);
        x86_conv(i, ty ? ty->size : 8, 0);
        return;
    }
    case IR_BITCAST: x86_conv(i, 8, 0); return;
    case IR_CMP_EQ: x86_cmp(i, s_sete); return;
    case IR_CMP_NE: x86_cmp(i, s_setne); return;
    case IR_CMP_SLT: x86_cmp(i, s_setl); return;
    case IR_CMP_SLE: x86_cmp(i, s_setle); return;
    case IR_CMP_SGT: x86_cmp(i, s_setg); return;
    case IR_CMP_SGE: x86_cmp(i, s_setge); return;
    case IR_CMP_ULT: x86_cmp(i, s_setb); return;
    case IR_CMP_ULE: x86_cmp(i, s_setbe); return;
    case IR_BR:
        kcc_as_str(s_jmp); x86_bblabel(IL(i,
IR_SRC1)); kcc_as_ch('\n');
        return;
    case IR_LADDR:
/* &&label → leaq .LkccbbN(%rip) */
        kcc_as_str(s_leaq); x86_bblabel(IL(i,
IR_SRC1)); kcc_as_str(s_rip_r10);
        x86_store(IL(i, IR_DEST), s_r10);
        return;
    case IR_BR_IND:
/* goto *reg → jmp *%r10 */
        x86_load(IL(i, IR_SRC1), s_r10);
        kcc_as_str(s_jmp_ind_r10);
        return;
    case IR_FRAME_ADDR: {
/* __builtin_frame_address(n) */
        kcc_as_str(s_mov_rbp_r10);
        for (unsigned long l = IL(i, IR_SRC1); l;
l--) kcc_as_str(s_mov_r10ind_r10);
        x86_store(IL(i, IR_DEST), s_r10);
        return;
    }
    case IR_RET_ADDR: {
/* __builtin_return_address(n) */
        kcc_as_str(s_mov_rbp_r10);
/* frame n, then its saved return */
        for (unsigned long l = IL(i, IR_SRC1); l;
l--) kcc_as_str(s_mov_r10ind_r10);
        kcc_as_str(s_mov_8r10_r10);
        x86_store(IL(i, IR_DEST), s_r10);
        return;
    }
    case IR_BR_COND:
        x86_load(IL(i, IR_DEST), s_r10);
        kcc_as_str(s_test_r10);
        kcc_as_str(s_jne); x86_bblabel(IL(i,
IR_SRC1)); kcc_as_ch('\n');
        kcc_as_str(s_jmp); x86_bblabel(IL(i,
IR_SRC2)); kcc_as_ch('\n');
        return;
    case IR_GLOBAL_REF:
        if (kcc_x86_os) {
/* Linux: leaq name(%rip) */
            kcc_as_str(s_leaq); x86_sym((const
char *)IL(i, IR_SRC1)); kcc_as_str(s_rip_r10);
        } else {
/* Darwin: movq name@GOTPCREL(%rip) */
            kcc_as_str(s_movq); x86_sym((const
char *)IL(i, IR_SRC1)); kcc_as_str(s_got_r10);
        }
        x86_store(IL(i, IR_DEST), s_r10);
        return;
    case IR_STRING_LIT: {
        unsigned long id = IL(kcc_x86_strs,
VEC_COUNT);
        unsigned long entry[2];
        entry[0] = IL(i, IR_SRC1);
        entry[1] = IL(i, IR_SRC2);
        kcc_vec_push(kcc_x86_strs, entry);
        kcc_as_str(s_leaq); kcc_as_str(s_strlbl);
kcc_as_dec(id); kcc_as_str(s_rip_r10);
        x86_store(IL(i, IR_DEST), s_r10);
        return;
    }
    case IR_CALL: {
        x86_r10_slot = -1;
      /* argument marshalling and the call clobber
    %r10 */
        unsigned long adj = x86_call_args((
unsigned long *)IL(i, IR_SRC2));
        x86_set_al();
        kcc_as_str(s_call); x86_sym((const char *)
IL(i, IR_SRC1)); kcc_as_ch('\n');
        x86_r10_slot = -1;
        if (adj) { kcc_as_str(s_add_rsp);
kcc_as_dec(adj); kcc_as_str(s_sp_nl); }
        {
            struct Type *ty = (struct Type *)IL(i,
    IR_TYPE);
            if (ty && is_fp_type(ty)) x86_fpstore(
IL(i, IR_DEST));
            else if (ty && ty->kind != TY_VOID)
x86_store(IL(i, IR_DEST), s_rax);
        }
        return;
    }
    case IR_CALL_IND: {
        x86_r10_slot = -1;
      /* argument marshalling and the call clobber
    %r10 */
        unsigned long adj = x86_call_args((
unsigned long *)IL(i, IR_SRC2));
        x86_load(IL(i, IR_SRC1), s_r11);
        x86_set_al();
        kcc_as_str(s_call_r11);
        x86_r10_slot = -1;
        if (adj) { kcc_as_str(s_add_rsp);
kcc_as_dec(adj); kcc_as_str(s_sp_nl); }
        {
            struct Type *ty = (struct Type *)IL(i,
    IR_TYPE);
            if (ty && is_fp_type(ty)) x86_fpstore(
IL(i, IR_DEST));
            else if (ty && ty->kind != TY_VOID)
x86_store(IL(i, IR_DEST), s_rax);
        }
        return;
```

```
    }
    case IR_VA_START: {
        /* Build the System V va_list at SRC1 (its
    address).  SRC2 packs the
           named-argument counts: (named_fp << 8)
   | named_gp.  Layout (kcc-
           internal, 8-byte fields): [0] gp_offset
   , [8] fp_offset, [16]
           overflow_arg_area, [24] reg_save_area.
   */
        unsigned long named = IL(i, IR_SRC2);
        unsigned long ngp = named & 0xff, nfp = (
   named >> 8) & 0xff;
        x86_load(IL(i, IR_SRC1), s_r11);
           /* r11 = &va_list */
        kcc_as_str(s_movabsq); kcc_as_dec(ngp * 8)
   ; kcc_as_str(s_c_r10);
        kcc_as_str(s_movq); kcc_as_str(s_r10);
   kcc_as_str(s_comma); kcc_as_dec(0); kcc_as_str(
   s_pr11); kcc_as_ch('\n');
        kcc_as_str(s_movabsq); kcc_as_dec(nfp *
   16); kcc_as_str(s_c_r10);
        kcc_as_str(s_movq); kcc_as_str(s_r10);
   kcc_as_str(s_comma); kcc_as_dec(8); kcc_as_str(
   s_pr11); kcc_as_ch('\n');
        kcc_as_str(s_leaq); kcc_as_dec(16);
   kcc_as_str(s_rbp); kcc_as_str(s_c_r10);   /*
   overflow = rbp+16 */
        kcc_as_str(s_movq); kcc_as_str(s_r10);
   kcc_as_str(s_comma); kcc_as_dec(16); kcc_as_str
   (s_pr11); kcc_as_ch('\n');
        kcc_as_str(s_leaq); x86_disp(
   kcc_x86_rsa_top); kcc_as_str(s_c_r10);
     /* reg_save = rbp-rsa_top */
        kcc_as_str(s_movq); kcc_as_str(s_r10);
   kcc_as_str(s_comma); kcc_as_dec(24); kcc_as_str
   (s_pr11); kcc_as_ch('\n');
        x86_r10_slot = -1;
      /* built the struct through raw %r10 writes
   */
        return;
    }
    case IR_ASM:
        x86_emit_asm(i);
        x86_r10_slot = -1;
      /* the asm body may clobber %r10 */
        return;
    case IR_NOP:
        return;
    default:
        kcc_io_err(s_unsup);
        kcc_io_flush();
        kcc_sys_exit(1);
    }
}
```

□

774 · used in 792a

72.5. Frame, Parameters, and the Function Walk

`x86_prescan` assigns each `IR_ALLOCA` its slot offset and returns the total; `x86_spill_params` copies the incoming System V registers into their parameter vreg slots; `x86_render_text` walks the function list, emitting the label, prologue, and each block's instructions.

The GNU linkage attributes the parser stashed in `kcc_attr_ht` (Chapter 68) are honoured here. `x86_attr` looks a symbol up by name; a function so annotated is wrapped in `.section name,"ax",@progbits … .text` for `__attribute__((section))` and headed by `.weak` instead of `.globl` for `__weak`. A `__weak` global takes a real `.bss` definition rather than a `.comm` (a common symbol may not also be weak), and an `alias` target—whose declaration is bodiless and so never reaches the IR—is emitted as a `.set` by a separate walk of the translation unit.

72.5.1. Dead static-function elimination

The kernel's bit and `cmpxchg` helpers are `static __always_inline` functions that dispatch on a compile-time-constant operand and emit a byte-sized inline-asm instruction only on the constant path—for example `arch_test_bit` evaluates `__builtin_constant_p(nr) ? constant_test_bit(...) :` where `constant_test_bit` contains a `testb %2,%1` valid only when its operand folds to an immediate. GCC always inlines these into a caller that supplies a literal, so `nr` is constant, the ternary collapses to the constant arm, and the byte instruction is emitted with a genuine immediate.

kcc does not inline, so it emits `arch_test_bit` as a stand-alone body in which `nr` is a runtime parameter. There the constant-condition fold (Chapter 75) is what saves us: `__builtin_constant_p(nr)` is an integer constant expression whose value is 0 because `nr` is not itself constant, so the ternary folds to the *variable* arm and no call to `constant_test_bit` is emitted. `constant_test_bit` thereby becomes unreferenced—and, being `static`, GCC would never emit it either. We match that: before the function walk we compute the set of *referenced* names and skip any `static` function whose name is absent from it, so its offending `testb` is never rendered.

The reference set is deliberately over-approximated for safety, because a dropped-but-needed function would surface only at final link, which the per-object sweep never reaches. A name is referenced if it appears as the callee of an `IR_CALL` or the target of an `IR_GLOBAL_REF` anywhere in *any* function's IR (this captures both direct calls and address-of-function), or as an identifier inside any file-scope initializer, as the target of an `alias` attribute, or as a token in a file-scope `asm` template. The initializer scan looks *through* casts and parentheses, not just bare identifiers and `&`: SQLite's syscall table is an array of structs whose function-pointer members are written `(sqlite3_syscall_ptr)posixOpen`, so without descending into the cast the scanner would miss `posixOpen` and its siblings and the VFS would fail to link. The asm-template case is how an `__initcall` or `DECLARE_PCI_FIXUP` names its `static` hook on x86-64, through a `.long fn - .` the IR never sees. A function is additionally kept whenever it carries any linkage attribute (`section`, `weak`, `alias`), since such a function is placed or exported deliberately. The compiler's own self-host is the correctness gate: it uses many static helpers and function-pointer tables, so the three-way byte-identical fixpoint would break instantly if the analysis dropped a live function.

⟨ *Selfhost: emit_x86 frame* 781 ⟩ ≡

```
static unsigned long x86_prescan(void *f) {
    unsigned long off = 0;
    void *blocks = (void *)IL(f, IRF_BLOCKS);
    if (!blocks) return 0;
    unsigned long nbb = IL(blocks, VEC_COUNT);
    for (unsigned long b = 0; b < nbb; b++) {
        void *bb = *(void **)kcc_vec_get(blocks, b
   );
        void *ins = (void *)IL(bb, BB_FIRST);
        while (ins) {
            if (IL(ins, IR_OP) == IR_ALLOCA) {
                IL(ins, IR_SRC2) = off;
                unsigned long sz = IL(ins, IR_SRC1
   );
                off += sz ? sz : 1;
            }
            ins = (void *)IL(ins, IR_NEXT);
        }
    }
    return off;
}

static void x86_spill_params(void *f) {
    unsigned long np = IL(f, IRF_PARAMS);
    void *cls = (void *)IL(f, IRF_PARAM_CLS);
    /* Mirror the caller's System-V classification
    into the parameter vreg slots.
       `base` is the vreg index (a class-3
   aggregate occupies TWO consecutive
       vregs, matching the IR builder's width
   count); `ngp`/`nfp` track the
       register banks; overflow sits above the
   return address at 16(%rbp),
       24(%rbp), ... in parameter order. */
    unsigned long ngp = 0, nfp = 0, base = 0, soff
    = 16;
    for (unsigned long k = 0; k < np; k++) {
        unsigned long cl = cls ? ((unsigned long
   *)cls)[k] : 0;
        if (cl == 1) {
            /* float/double → %xmm */
            if (nfp < 8) {
                kcc_as_str(s_movsd); kcc_as_str(
   xmmregs[nfp]); kcc_as_str(s_comma);
                x86_slot(base); kcc_as_ch('\n');
   nfp++; base++; continue;
            }
        } else if (cl == 3) {
            /* 9-16B aggregate → 2 GP */
            if (ngp + 1 < 6) {
                x86_store(base, argregs[ngp]);
   x86_store(base + 1, argregs[ngp + 1]);
                ngp += 2; base += 2; continue;
            }
        } else if (ngp < 6) {
            /* GP / ptr / 8B aggregate */
            x86_store(base, argregs[ngp]); ngp++;
   base++; continue;
        }
        /* overflow: read from the caller's stack
   image */
        kcc_as_str(s_movq); kcc_as_dec(soff);
   kcc_as_str(s_rbp);
        kcc_as_str(s_comma); kcc_as_str(s_r10);
   kcc_as_ch('\n');
        x86_store(base, s_r10); soff += 8; base++;
        if (cl == 3) {
            kcc_as_str(s_movq); kcc_as_dec(soff);
   kcc_as_str(s_rbp);
            kcc_as_str(s_comma); kcc_as_str(s_r10)
   ; kcc_as_ch('\n');
            x86_store(base, s_r10); soff += 8;
   base++;
        }
    }
}

/* Reference set for dead static-function
   elimination: names that must not be
   dropped even when static.  Marked liberally (
   over-approximate) so a live
   function is never elided; see the prose above.
   */
static void *kcc_x86_refset = 0;
static void x86_ref_mark(const char *name) {
    if (name && *name)
        kcc_ht_put(kcc_x86_refset, name,
   kcc_strlen(name), 1);
}
static int x86_is_referenced(const char *name) {
    return name && kcc_ht_get(kcc_x86_refset, name
   , kcc_strlen(name)) != 0;
}
/* Walk an initializer AST subtree, marking every
   identifier it names (a
   function whose address a file-scope initializer
    takes must be kept).  The
   walk is tag-directed rather than blindly
   recursive: literal nodes store raw
   values (not pointers) in their fields, so it
   only descends into fields known
   to hold child nodes---an init-list's element
   chain and a unary operand. */
static void x86_ref_scan_init(struct Node *n) {
    if (!n) return;
    if (n->tag == AST_IDENT_EXPR) {
        x86_ref_mark((const char *)n->f0);
    } else if (n->tag == AST_UNARY_EXPR) {
     /* e.g. &func */
        x86_ref_scan_init((struct Node *)n->f1);
    } else if (n->tag == AST_CAST_EXPR) {
     /* (T)func --- SQLite's aSyscall[] */
        x86_ref_scan_init((struct Node *)n->f1);
    } else if (n->tag == AST_PAREN_EXPR) {
        x86_ref_scan_init((struct Node *)n->f0);
    } else if (n->tag == AST_INIT_LIST) {
        for (struct Node *e = (struct Node *)n->f0
   ; e; e = (struct Node *)e->f4)
            x86_ref_scan_init(e);
    }
}
/* Mark every identifier-like token in a file-
   scope asm template as referenced.
   On x86-64 (PREL32 relocations) an __initcall
   and a DECLARE_PCI_FIXUP place a
   function's address into a `.initcall'/`.
   pci_fixup' table through a
   `.long fn - .' emitted by a raw file-scope `asm
   (...)', so `fn' is a static
   function referenced only from inside that
   template string---invisible to the
   IR and initializer walks.  Marking every token
   is a safe over-approximation:
   the extra words (`section', `long', `previous',
    ...) name nothing the drop
   would ever consider, while the real hook/stub/
   initcall names are kept.  The
   token is marked by its (pointer, length) into
   the stable template, so no copy
   is needed---the refset compares keys by content
   . */
static void x86_ref_scan_asm(const char *t,
```

```
    unsigned long n) {
     if (!t || !kcc_x86_refset) return;
     unsigned long i = 0;
     while (i < n) {
         char c = t[i];
         if ((c >= 'A' && c <= 'Z') || (c >= 'a' &&
     c <= 'z') || c == '_') {
             unsigned long s = i;
             while (i < n) {
                 char d = t[i];
                 if ((d >= 'A' && d <= 'Z') || (d
    >= 'a' && d <= 'z') ||
                     (d >= '0' && d <= '9') || d ==
     '_') i++;
                 else break;
             }
             kcc_ht_put(kcc_x86_refset, t + s, i -
    s, 1);
         } else i++;
     }
}
static void x86_build_refset(void) {
     kcc_x86_refset = kcc_ht_init(256);
     void *funcs = kcc_ir_funcs;
     if (funcs) {
         unsigned long nf = IL(funcs, VEC_COUNT);
         for (unsigned long fi = 0; fi < nf; fi++)
    {
             void *f = *(void **)kcc_vec_get(funcs,
     fi);
             void *blocks = (void *)IL(f,
    IRF_BLOCKS);
             if (!blocks) continue;
             unsigned long nbb = IL(blocks,
    VEC_COUNT);
             for (unsigned long b = 0; b < nbb; b
    ++) {
                 void *bb = *(void **)kcc_vec_get(
    blocks, b);
                 for (void *ins = (void *)IL(bb,
    BB_FIRST); ins;
                      ins = (void *)IL(ins, IR_NEXT
    )) {
                     unsigned long op = IL(ins,
    IR_OP);
                     if (op == IR_CALL || op ==
    IR_GLOBAL_REF)
                         x86_ref_mark((const char
    *)IL(ins, IR_SRC1));
                 }
             }
         }
     }
     struct Node *tu = (struct Node *)kcc_x86_tu;
     if (tu) {
         for (struct Node *d = (struct Node *)tu->
    f0; d; d = (struct Node *)d->f4) {
             /* A file-scope asm statement's f0/f1
    are its template and length, not
                a name; scan the template for the
    symbols its `.long fn - .'
                directives reference (initcall/pci-
    fixup tables). */
             if (d->tag == AST_ASM_STMT) {
                 x86_ref_scan_asm((const char *)d->
    f0, d->f1);
                 continue;
             }
             /* Only VAR_DECL/FUNC_DECL carry a
    name pointer in f1; reading it as a
                string on any other decl would
    fault. */
             if (d->tag != AST_VAR_DECL && d->tag
    != AST_FUNC_DECL) continue;
             if (d->tag == AST_VAR_DECL && d->f2)
     /* f2 = initializer */
                 x86_ref_scan_init((struct Node *)d
    ->f2);
             struct AttrRec *ar = x86_attr((const
    char *)d->f1);
             if (ar && ar->alias) x86_ref_mark(ar->
    alias);  /* alias .set target */
         }
     }
     /* Static-local initializers are emitted as
    their own objects (below) and one
        may take a static function's address---Lua'
    s `createsearcherstable' holds a
        block-local `static const lua_CFunction
    searchers[] = { searcher_preload,
        ... }'.  Those decls are not in the file-
    scope chain, so scan them here too,
        or a function referenced only from such a
    table would be dropped as unused
        and the link would fail with an undefined
    reference. */
     if (kcc_ir_static_decls) {
         unsigned long ns = IL(kcc_ir_static_decls,
     VEC_COUNT);
         for (unsigned long i = 0; i < ns; i++) {
             struct Node *sd = *(struct Node **)
    kcc_vec_get(kcc_ir_static_decls, i);
             if (sd && sd->f2) x86_ref_scan_init((
    struct Node *)sd->f2);
         }
     }
}

static void x86_render_text(void) {
     kcc_as_str(s_text);
     void *funcs = kcc_ir_funcs;
     if (!funcs) return;
     x86_build_refset();
     unsigned long nf = IL(funcs, VEC_COUNT);
     for (unsigned long fi = 0; fi < nf; fi++) {
         void *f = *(void **)kcc_vec_get(funcs, fi)
    ;
         const char *name = (const char *)IL(f,
    IRF_NAME);
         struct AttrRec *ar = x86_attr(name);
         /* Drop a static function unreferenced
    across all IR and initializers,
            unless it carries a placement/export
    attribute. */
         if (IL(f, IRF_STATIC) && !
    x86_is_referenced(name)
             && kcc_strcmp(name, kcc_str_modinit)
    != 0
             && !(ar && (ar->section || ar->weak ||
     ar->alias)))
             continue;
         if (ar && ar->section) {
    /* __attribute__((section("..."))): code
    section */
             kcc_as_str(s_section); kcc_as_str(ar->
    section); kcc_as_str(s_ax);
         }
         if (!IL(f, IRF_STATIC) && kcc_strcmp(name,
     kcc_str_modinit) != 0) {
             kcc_as_str(ar && ar->weak ? s_weak :
    s_globl);   /* __weak → .weak */
             x86_sym(name); kcc_as_ch('\n');
```

```
        }
        x86_sym(name); kcc_as_str(s_colon_nl);
        unsigned long vn = IL(f, IRF_VREG_N);
        kcc_x86_vreg_n = vn;
        unsigned long allocas = x86_prescan(f);
        unsigned long save_base = (vn + allocas) *
 8;
        unsigned long variadic = IL(f,
IRF_VARIADIC);
        kcc_x86_rsa_top = 0;
        if (variadic) {                 /* reserve a
System V register-save area: 6 GP + 8 XMM */
            save_base += 176;
            kcc_x86_rsa_top = save_base;
        }
        unsigned long frame = ((save_base) + 15) &
 ~15UL;
        kcc_as_str(s_push_rbp); kcc_as_str(
s_mov_sp);
        if (frame) { kcc_as_str(s_sub_sp);
kcc_as_dec(frame); kcc_as_str(s_sp_nl); }
        x86_spill_params(f);
        if (variadic) {                 /* spill the
incoming argument registers for va_arg */
            for (unsigned long k = 0; k < 6; k++)
{
                kcc_as_str(s_movq); kcc_as_str(
argregs[k]); kcc_as_str(s_comma);
                x86_disp(kcc_x86_rsa_top - k * 8);
 kcc_as_ch('\n');
            }
            for (unsigned long k = 0; k < 8; k++)
{
                kcc_as_str(s_movsd); kcc_as_str(
xmmregs[k]); kcc_as_str(s_comma);
                x86_disp(kcc_x86_rsa_top - 48 - k
* 16); kcc_as_ch('\n');
            }
        }
        void *blocks = (void *)IL(f, IRF_BLOCKS);
        if (blocks) {
            unsigned long nbb = IL(blocks,
VEC_COUNT);
            for (unsigned long b = 0; b < nbb; b
++) {
                void *bb = *(void **)kcc_vec_get(
blocks, b);
                x86_bblabel(IL(bb, BB_ID));
kcc_as_str(s_label_nl);
                x86_r10_slot = -1;
   /* a block is entered from many preds: %r10
unknown */
                void *ins = (void *)IL(bb,
BB_FIRST);
                while (ins) { kcc_x86_sel(ins);
ins = (void *)IL(ins, IR_NEXT); }
            }
        }
        if (ar && ar->section) kcc_as_str(s_text);
    /* leave the custom section */
    }
}
```

□

781 · used in 792a

72.6. Global, Static, and Read-Only Data

Mutable file-scope globals become zeroed common storage (their initialisers run as code); `const` scalars, arrays of scalars, structs of scalars, and arrays of such structs become read-only value directives; static locals are recovered by walking every function body—and, for those a spine walk cannot reach, by a decl list the IR builder keeps (below). A declarator whose type resolves to a *function* type is not an object at all—it declares a function and gets no storage—so the data pass skips it: the kernel's `INDIRECT_CALLABLE_DECLARE` spells a prototype as `inet_ehashfn_t udp_ehashfn;` through a function-type typedef, and a `.comm` for that name would collide with the real definition.

A common symbol emitted with `.comm` has *external* linkage: the linker coalesces same-named commons from different objects into one. A file-scope `static` has *internal* linkage and must stay private to its translation unit, so for a static we first mark the name `.local`. Without it two units that each name a static the same—say both back ends' `s_globl`—would share one storage cell and their initialisers would overwrite each other.

A `__attribute__((section("...")))` on a defining declaration is the one case where a zero-storage global must *not* become a common symbol. A common lives in `.bss`; the section attribute demands the cell appear in a *specific* named section instead. The kernel's per-cpu machinery depends on this: `DEFINE_PER_CPU_FIRST` lands `fixed_percpu_data` in `.data..percpu..first`, which `vmlinux.lds` orders first in the per-cpu output and then asserts sits at offset 0. Emitted as a `.comm` the symbol would drift into `.bss` and the link would fail the assert, so a section-attributed definition is written out longhand—`.section name,"aw"`, the label, and a `.zero` of its size—exactly as gcc does. (The attribute record carries no alignment, so the cell takes the default 8-byte `.p2align`; being the sole and first symbol of its section it still resolves to offset 0.)

A subtler duplicate arises when a tentative declaration is followed by a *defining* one that carries an initialiser—`u32 sqlite3WhereTrace;` near the top of the SQLite amalgamation and `u32 sqlite3WhereTrace = 0;` tens of thousands of lines later. These are one object with one definition (§6.9.2), and the defining declaration is the one that must reach the assembler; the tentative would otherwise be emitted as a zero-storage cell *and* the definition as a real data label, and as refuses the pair ("symbol already defined")—the `.comm`-coalescing that hides a double *tentative* does not save a common paired with a genuine label. So before the data pass writes a tentative's zero cell it scans the unit for a full definition of the same name (`x86_superseded_tentative`) and, finding one, drops the tentative; the defining declaration then emits the object once, with its initialiser. The ARM back end already handled this; the x86 pass now matches it.

⟨ *Selfhost: emit_x86 data* 784 ⟩ ≡

```
/* A single file-scope object may be spelled by
   both a tentative declaration
   (\verb|u32 x;|) and a defining one (\verb|u32 x
    = 0;|); C makes that one
   object, but each reaches the emitter separately
   .  Emitting the zero-storage
   cell twice is harmless as a Mach-O common (the
   linker coalesces them) yet a
   hard error for a GNU-assembler \verb|.local .
   comm| pair, so the set of names
   already given storage is remembered and re-
   emission is skipped. */
static void *kcc_x86_emitted = 0;
static void x86_emit_zerofill(const char *name,
   unsigned long size, int is_static) {
    if (kcc_x86_emitted && name) {
        unsigned long kl = kcc_strlen(name);
        if (kcc_ht_get(kcc_x86_emitted, name, kl))
    return;
        kcc_ht_put(kcc_x86_emitted, name, kl, 1);
    }
    struct AttrRec *ar = x86_attr(name);
    int weak = !is_static && ar && ar->weak;
   /* __weak global */
    const char *sec = ar ? ar->section : 0;
   /* __attribute__((section("..."))) */
    if (!is_static) { kcc_as_str(weak ? s_weak :
   s_globl); x86_sym(name); kcc_as_ch('\n'); }
    if (sec && kcc_x86_os) {
        /* A section attribute on a definition
   forces a real, zero-filled cell
           in the named section --- never a common
    symbol. This matters for the
           kernel per-cpu area:
   DEFINE_PER_CPU_FIRST places fixed_percpu_data
   in
           .data..percpu..first, and vmlinux.lds
   asserts it sits at offset 0 of
           the per-cpu output. Emitted as a `.comm
   `, it would land in .bss and
           never reach the per-cpu section,
   tripping that assert. */
        kcc_as_str(s_section); kcc_as_str(sec);
   kcc_as_str(s_aw);
        kcc_as_str(s_p2align3);
        x86_sym(name); kcc_as_str(s_colon_nl);
        kcc_as_str(s_zerodir); kcc_as_dec(size);
   kcc_as_ch('\n');
        kcc_as_str(s_text);
    } else if (weak && kcc_x86_os) {
        /* a common symbol cannot also be weak;
   give it a real weak definition
           in .bss instead of a `.comm`. */
        kcc_as_str(s_bss); kcc_as_str(s_p2align3);
        x86_sym(name); kcc_as_str(s_colon_nl);
        kcc_as_str(s_zerodir); kcc_as_dec(size);
   kcc_as_ch('\n');
        kcc_as_str(s_text);
    } else if (kcc_x86_os) {
   /* Linux .comm name,size,8 */
        if (is_static) { kcc_as_str(s_local);
   kcc_as_str(name); kcc_as_ch('\n'); }
        kcc_as_str(s_comm); kcc_as_str(name);
   kcc_as_str(s_comma1);
        kcc_as_dec(size); kcc_as_str(s_comm_a8);
    } else {
   /* Darwin .zerofill (prefix carries _) */
        kcc_as_str(s_zerofill); kcc_as_str(name);
   kcc_as_str(s_comma1);
        kcc_as_dec(size); kcc_as_str(s_zf_a3);
    }
}

static unsigned long round8_min8(unsigned long n)
   {
    if (n < 8) n = 8;
    return (n + 7) & ~7UL;
}

/* A tentative definition (`static int x;', no
   initializer) is superseded by a
   full definition of the same name elsewhere in
   the translation unit
   (\stdref{6.9.2}): SQLite writes `u32
   sqlite3WhereTrace;' and, thousands of
   lines later, `u32 sqlite3WhereTrace = 0;'.
   Only the full definition may reach
   the assembler---emitting both gives it a `.comm
   ' (or zerofill label) and a
   real data label for one symbol, which `as'
   rejects as "already defined".  The
   full definition carries the object's storage,
   so the tentative is dropped. */
static int x86_superseded_tentative(struct Node *
   tu, const char *name) {
    for (struct Node *d = (struct Node *)tu->f0; d
   ; d = (struct Node *)d->f4) {
        if (d->tag != AST_VAR_DECL || !d->f2)
   continue;   /* only full definitions */
        const char *n2 = (const char *)d->f1;
        if (n2 && kcc_strcmp(n2, name) == 0)
   return 1;
    }
    return 0;
}

void kcc_x86_emit_data(void) {
    struct Node *tu = (struct Node *)kcc_x86_tu;
    if (!tu) return;
    kcc_x86_emitted = kcc_ht_init(256);    /*
   fresh per translation unit */
    struct Node *d = (struct Node *)tu->f0;
    for (; d; d = (struct Node *)d->f4) {
        if (d->tag != AST_VAR_DECL) continue;
        void *name = (void *)d->f1;
        if (!name) continue;
        unsigned long sc = d->f3;
   /* storage class */
        if (sc == 2 || sc == 1) continue;
   /* extern / typedef: no storage */
        if (!d->f2 && x86_superseded_tentative(tu,
    (const char *)name))
            continue;
   /* tentative def; a full one follows */
        if (kcc_global_const_rodata(d)) {
            x86_emit_rodata_global(d, name);
            continue;
        }
        if (kcc_global_const_ptrarr(d)) {
   x86_emit_rodata_ptrarr(d, name); continue; }
        if (kcc_global_sym_data(d)) {
   x86_emit_sym_data(d, name); continue; }
        if (kcc_global_static_ok(d)) {
   x86_emit_general(d, name); continue; }
        void *ty = kcc_sema_resolve_type((void *)d
   ->f0);
        if (!ty) continue;
        /* A declarator whose type is a function
   type declares a function, not an
           object---`fntype name;' through a
   function-type typedef is a prototype
           with no storage, exactly like `u32 name
   (args);'.  The kernel writes
           these through INDIRECT_CALLABLE_DECLARE
    (under CONFIG_RETPOLINE it is a
```

```
            bare `inet_ehashfn_t udp_ehashfn;');
    emitting a `.comm' for it would
            collide with the real function
    definition---"symbol already defined". */
        if (IL(ty, TY_KIND) == TY_FUNC) continue;
        unsigned long size = round8_min8(
    kcc_type_layout(ty));
        x86_emit_zerofill(name, size, sc == 3);
    }
}

static void x86_emit_one_static(void *decl) {
    struct Node *d = (struct Node *)decl;
    const char *name = kcc_ir_static_name(d);
    /* Emit each static's storage once.  Check
    only here---do NOT claim the name,
        or x86_emit_zerofill's own guard would see
    it already taken and emit
        nothing.  The rodata paths have no such
    guard, so they are marked below. */
    unsigned long kl = name ? kcc_strlen(name) :
    0;
    if (kcc_x86_emitted && name && kcc_ht_get(
    kcc_x86_emitted, name, kl)) return;
    if (kcc_global_const_rodata(d)) {
        x86_emit_rodata_global(d, name);
        if (kcc_x86_emitted && name) kcc_ht_put(
    kcc_x86_emitted, name, kl, 1);
        return;
    }
    if (kcc_global_const_ptrarr(d)) {
        x86_emit_rodata_ptrarr(d, name);
        if (kcc_x86_emitted && name) kcc_ht_put(
    kcc_x86_emitted, name, kl, 1);
        return;
    }
    if (kcc_global_sym_data(d)) {
        x86_emit_sym_data(d, name);
        if (kcc_x86_emitted && name) kcc_ht_put(
    kcc_x86_emitted, name, kl, 1);
        return;
    }
    if (kcc_global_static_ok(d)) {
        x86_emit_general(d, name);
        if (kcc_x86_emitted && name) kcc_ht_put(
    kcc_x86_emitted, name, kl, 1);
        return;
    }
    void *ty = kcc_sema_resolve_type((void *)d->f0
    );
    if (!ty) return;
    unsigned long size = round8_min8(
    kcc_type_layout(ty));
    x86_emit_zerofill(name, size, 1);
}

static void x86_prereg_stmt(void *stmt) {
    struct Node *s = (struct Node *)stmt;
    for (; s; s = (struct Node *)s->f4) {
        unsigned long tag = s->tag;
        if (tag == AST_VAR_DECL) {
            if (s->f3 == 3) x86_emit_one_static(s)
    ;
            continue;
        }
        switch (tag) {
        case AST_COMPOUND: case AST_DO_STMT: case
    AST_DEFAULT_STMT:
            x86_prereg_stmt((void *)s->f0); break;
        case AST_SWITCH_STMT: case AST_WHILE_STMT:
    case AST_CASE_STMT:
        case AST_LABEL_STMT:
            x86_prereg_stmt((void *)s->f1); break;
        case AST_IF_STMT:
            x86_prereg_stmt((void *)s->f1);
            x86_prereg_stmt((void *)s->f2);
            break;
        case AST_FOR_STMT:
            x86_prereg_stmt((void *)s->f0);
            x86_prereg_stmt((void *)s->f3);
            break;
        default: break;
        }
    }
}

void kcc_x86_emit_statics(void) {
    struct Node *tu = (struct Node *)kcc_x86_tu;
    if (!tu) return;
    struct Node *d = (struct Node *)tu->f0;
    for (; d; d = (struct Node *)d->f4)
        if (d->tag == AST_FUNC_DECL && d->f3)
            x86_prereg_stmt((void *)d->f3);
    ⟨ Selfhost: emit_x86 stmt-expr static storage 787a ⟩
}
□
```

784 · used in 792a

The function-body walk finds every static local that sits on a statement spine, but a static declared inside a GNU statement-expression (`({ ... })`) hides from it: the `printk_once`/`WARN_ONCE`/`DO_ONCE` family each expands to `({ static bool __c; ... })`, and the once-flag lives in an expression the spine walk never descends into. The IR builder does reach it—it registers the static and its references resolve to an `sl_` name—but the name's storage was never emitted, so a whole-kernel link left thousands of `sl_` symbols undefined. The remedy is a belt-and-braces list: the builder records *every* static-local declaration it registers in `kcc_ir_static_decls`, and after the spine walk we emit storage for any entry not already given a cell. The shared `kcc_x86_emitted` guard (now consulted at the top of `x86_emit_one_static` as well) means a static found by both routes is emitted once; a stmt-expr static, found only by the list, still gets its definition. This gives every stmt-expr static a *storage* cell, which is what the whole-kernel link needs. Its *initialiser* is a separate matter: the once-flag families initialise to zero, and a common lands in zeroed `.bss`, so no init code is required and the value is already correct. A stmt-expr static with a *non-zero* initialiser would need that initialiser hoisted to module-init code, which the spine-only `kcc_ir_emit_global_inits` does not do for statics nested in expressions—a documented limitation the kernel's zero-valued once-flags never exercise.

One hazard remains. `kcc_ir_static_name` mangles a static to `sl_` plus the hex of its declaration node's address—an arena offset that *resets* every compilation, so two unrelated files' statics can mangle to the same name. While those names were undefined this was invisible; once we define them, the linker sees two objects each defining `sl_0000...` and rejects the second. The driver therefore seeds a per-file salt (`kcc_tu_salt`, a hash of the source path) that `kcc_ir_static_name` exclusive-ors into the address before hex, making the name unique across the tree yet stable within one file—so the reference and its definition still agree.

⟨ *Selfhost: emit_x86 stmt-expr static storage* 787a ⟩ ≡

```
if (kcc_ir_static_decls) {
    unsigned long n = IL(kcc_ir_static_decls,
   VEC_COUNT);
    for (unsigned long i = 0; i < n; i++)
        x86_emit_one_static(*(void **)kcc_vec_get(
   kcc_ir_static_decls, i));
}
```

□

787a · used in 784

72.7. Read-Only Byte Images

⟨ *Selfhost: emit_x86 rodata* 787b ⟩ ≡

```
static void x86_emit_zero(unsigned long n) {
    if (!n) return;
    kcc_as_str(s_zerodir); kcc_as_dec(n);
   kcc_as_ch('\n');
}
static void x86_emit_value(unsigned long bits,
   unsigned long size) {
    if (size == 1) { kcc_as_str(s_d_byte); bits &=
    0xff; }
    else if (size == 2) { kcc_as_str(s_d_short);
   bits &= 0xffff; }
    else if (size == 8) { kcc_as_str(s_d_quad); }
    else { kcc_as_str(s_d_long); bits &= 0
   xffffffff; }
    kcc_as_dec(bits); kcc_as_ch('\n');
}
/* Emit a struct's bytes by first painting the
   whole object into a byte image at
   each field's offset --- little-endian for a
   plain field, bit-packed for a
   bit-field (F2 = (bitoff<<16)|width, F3 = the
   container's byte offset) --- then
   laying it down as `.byte's.  Building the image
    (rather than emitting field by
   field) is what lets adjacent bit-fields share
   their container, so a
   `const desc_struct[]' GDT lands with its real
   descriptor bits instead of the
   zero it used to get for containing bit-fields
   at all. */
/* Paint one field's value into the object image:
   bit-packed for a bit-field
   (F2 = (bitoff<<16)|width, F3 = container byte
   offset), little-endian bytes for
   a plain field (at its byte offset F3). */
static void x86_paint_field(unsigned char *buf,
   unsigned long total,
                            struct Node *field,
   unsigned long bits) {
    unsigned long bf = field->f2, foff = field->f3
   ;
    if (bf) {
        unsigned long bitoff = (bf >> 16) & 0xffff
   , width = bf & 0xffff;
        for (unsigned long b = 0; b < width; b++)
            if ((bits >> b) & 1UL) {
                unsigned long bit = bitoff + b, by
    = foff + (bit >> 3);
                if (by < total) buf[by] |= (
   unsigned char)(1u << (bit & 7));
            }
    } else {
        struct Type *ft = (struct Type *)
   kcc_sema_resolve_type((void *)field->f0);
        unsigned long fsz = ft ? ft->size : 8;
        for (unsigned long b = 0; b < fsz; b++)
            if (foff + b < total) buf[foff + b] =
   (unsigned char)((bits >> (8 * b)) & 0xff);
    }
}
static struct Node *x86_find_field(void *sty,
   const char *name) {
    struct Node *decl = (struct Node *)((struct
   Type *)sty)->f0;
    for (struct Node *f = (struct Node *)decl->f1;
    f; f = (struct Node *)f->f4) {
        const char *fn = (const char *)f->f1;
        if (fn && name && (fn == name ||
   kcc_strcmp(fn, name) == 0)) return f;
    }
    return 0;
}
static struct Type *x86_field_type(struct Node *
   field) {
    return (struct Type *)kcc_sema_resolve_type((
   void *)field->f0);
}
static void x86_emit_struct_bytes(void *sty, void
   *init, unsigned long total) {
    struct Node *sdecl = (struct Node *)((struct
   Type *)sty)->f0;   /* AST_STRUCT_DECL */
    struct Node *field = (struct Node *)sdecl->f1;
                       /* first field (positional
   cursor) */
    struct Node *elem = init ? (struct Node *)((
   struct Node *)init)->f0 : 0;
    unsigned char buf[1024];
    if (total > 1024) total = 1024;
    for (unsigned long i = 0; i < total; i++) buf[
   i] = 0;
    while (elem) {
        struct Node *fld; void *val;
        if (elem->tag == AST_DESIGNATOR && elem->
   f1 == 1) {   /* `.member = value' */
            fld = x86_find_field(sty, (const char
   *)elem->f0);
            val = (void *)elem->f3;
        } else {
            /* positional */
            fld = field; val = elem;
            if (field) field = (struct Node *)
   field->f4;
        }
        if (fld && val) {
            struct Type *ft = x86_field_type(fld);
            x86_paint_field(buf, total, fld,
   kcc_rodata_leaf_bits(val, ft ? ft->kind :
   TY_INT));
        }
        elem = (struct Node *)elem->f4;
    }
    for (unsigned long i = 0; i < total; i++) {
   kcc_as_str(s_d_byte); kcc_as_dec(buf[i]);
   kcc_as_ch('\n'); }
}

/* Emit one initializer leaf as data: a symbol
   address becomes a `.quad sym+off'
   relocation, a string literal a `.quad .Lx86str<
   id>' (registered for the render
   pass), anything else its integer/float bits at
   the field's width. */
```

```
static void x86_emit_data_leaf(struct Node *e,
   unsigned long fkind, unsigned long fsize) {
    long off = 0;
    const char *s = e ? kcc_ir_const_symaddr(e, &
   off) : 0;
    if (s) {
        kcc_as_str(s_d_quad); x86_sym(s);
        if (off > 0) { kcc_as_str(s_plus);
   kcc_as_dec((unsigned long)off); }
        else if (off < 0) { kcc_as_str(s_minus);
   kcc_as_dec((unsigned long)(-off)); }
        kcc_as_ch('\n');
    } else if (e && e->tag == AST_STR_LIT) {
        unsigned long id = IL(kcc_x86_strs,
   VEC_COUNT);
        unsigned long entry[2]; entry[0] = e->f0;
   entry[1] = e->f1;
        kcc_vec_push(kcc_x86_strs, entry);
        kcc_as_str(s_d_quad); kcc_as_str(s_strlbl)
   ; kcc_as_dec(id); kcc_as_ch('\n');
    } else {
        x86_emit_value(e ? kcc_rodata_leaf_bits((
   void *)e, fkind) : 0, fsize);
    }
}
/* Struct data with symbol-aware leaves (mirrors
   x86_emit_struct_bytes, but a
   pointer field initialised to `&sym'/`fn' emits
   a relocation). */
static void x86_emit_struct_syms(void *sty, void *
   init, unsigned long total) {
    struct Node *decl = (struct Node *)((struct
   Type *)sty)->f0;
    struct Node *field = (struct Node *)decl->f1;
    struct Node *elem = init ? (struct Node *)((
   struct Node *)init)->f0 : 0;
    unsigned long cursor = 0;
    while (field) {
        struct Type *ft = (struct Type *)
   kcc_sema_resolve_type((void *)field->f0);
        if (!ft) break;
        unsigned long fsize = ft->size, fkind = ft
   ->kind, foff = field->f3;
        if (foff > cursor) { x86_emit_zero(foff -
   cursor); cursor = foff; }
        x86_emit_data_leaf(elem, fkind, fsize);
        if (elem) elem = (struct Node *)elem->f4;
        cursor += fsize;
        field = (struct Node *)field->f4;
    }
    if (cursor < total) x86_emit_zero(total -
   cursor);
}
/* A global whose initializer names a symbol
   address (a scalar pointer, or a
   struct mixing literals and function/object
   pointers): emit it as static DATA
   with relocations, in its attributed section (or
    `.data'), the way gcc does ---
   instead of a run-time store from kcc.
   module_init that a kernel never runs.
   This is what puts the exitcall / __setup / PCI-
   fixup tables' function pointers
   in place at link time. */
static void x86_emit_sym_data(void *decl, const
   char *name) {
    struct Node *d = (struct Node *)decl;
    if (!name) return;
    struct Type *ty = (struct Type *)
   kcc_sema_resolve_type((void *)d->f0);
    if (!ty) return;
    struct AttrRec *ar = x86_attr(name);
    const char *sec = ar ? ar->section : 0;
    unsigned long is_static = (d->f3 == 3);
    if (sec) { kcc_as_str(s_section); kcc_as_str(
   sec); kcc_as_str(s_aw); }
    else kcc_as_str(s_data);
    kcc_as_str(s_p2align3);
    if (!is_static) { kcc_as_str(s_globl); x86_sym
   (name); kcc_as_ch('\n'); }
    else if (kcc_x86_os) { kcc_as_str(s_local);
   kcc_as_str(name); kcc_as_ch('\n'); }
    x86_sym(name); kcc_as_str(s_colon_nl);
    if (ty->kind == TY_STRUCT)
        x86_emit_struct_syms(ty, (void *)d->f2,
   kcc_type_layout(ty));
    else
        x86_emit_data_leaf((struct Node *)d->f2,
   ty->kind, 8);
    if (sec) kcc_as_str(s_text);
}
static void x86_emit_rodata_global(void *decl,
   const char *name) {
    struct Node *d = (struct Node *)decl;
    if (!name) return;
    struct Type *full = (struct Type *)
   kcc_sema_resolve_type((void *)d->f0);
    if (!full) return;
    unsigned long shape, count = 0, elemsize,
   elemkind;
    struct Type *elemtype;
    unsigned long fk = full->kind;
    if (fk == TY_STRUCT) { shape = 2; elemtype =
   full; }
    else if (fk == TY_ARRAY) {
        elemtype = (struct Type *)full->f0;
        shape = (elemtype->kind == TY_STRUCT) ? 3
   : 1;
        elemsize = elemtype->size;
        if (!elemsize) return;
        count = kcc_type_layout(full) / elemsize;
    } else { shape = 0; elemtype = full; count =
   1; }
    elemsize = elemtype->size;
    elemkind = elemtype->kind;

    struct AttrRec *ar = x86_attr(name);
    const char *sec = ar ? ar->section : 0;
   /* honour __attribute__((section)) */
    if (sec) { kcc_as_str(s_section); kcc_as_str(
   sec); kcc_as_str(s_aw); }
    else if (kcc_global_writable(d)) kcc_as_str(
   s_data);          /* non-const -> writable .
   data */
    else kcc_as_str(kcc_x86_os ? s_rodata :
   s_constsect);         /* const -> .rodata */
    kcc_as_str(s_p2align3);
    if (d->f3 != 3) { kcc_as_str(s_globl); x86_sym
   (name); kcc_as_ch('\n'); }
    else if (kcc_x86_os) { kcc_as_str(s_local);
   kcc_as_str(name); kcc_as_ch('\n'); }
    x86_sym(name); kcc_as_str(s_colon_nl);

    if (shape == 2) {
        unsigned long total = kcc_type_layout(full
   );
        x86_emit_struct_bytes(full, (void *)d->f2,
    total);
        return;
    }
    if (shape == 3) {
```

```
        struct Node *arr = (struct Node *)d->f2;
        struct Node *e = arr ? (struct Node *)arr
    ->f0 : 0;
        unsigned long cur = 0;
        /* next element index to emit */
        while (e && cur < count) {
            struct Node *sinit = e; unsigned long
    idx = cur;
            if (e->tag == AST_DESIGNATOR && e->f1
    == 0) {   /* `[index] = { ... }' */
                idx = kcc_eval_const_int((void *)e
    ->f0);
                sinit = (struct Node *)e->f3;
            }
            if (idx >= cur && idx < count) {
                if (idx > cur) { x86_emit_zero((
    idx - cur) * elemsize); cur = idx; }
                x86_emit_struct_bytes(elemtype,
    sinit, elemsize);
                cur++;
            }
            e = (struct Node *)e->f4;
        }
        if (cur < count) x86_emit_zero((count -
    cur) * elemsize);
        return;
     }
     /* scalar / array-of-scalar */
     unsigned long emitted = 0;
     struct Node *init = (struct Node *)d->f2;
     if (init) {
        if (init->tag == AST_INIT_LIST) {
            struct Node *e = (struct Node *)init->
    f0;
            while (e) {
                x86_emit_value(
    kcc_rodata_leaf_bits(e, elemkind), elemsize);
                emitted++;
                e = (struct Node *)e->f4;
            }
        } else {
            x86_emit_value(kcc_rodata_leaf_bits(
    init, elemkind), elemsize);
            emitted = 1;
        }
     }
     if (emitted < count) x86_emit_zero((count -
    emitted) * elemsize);
}

/* A const array of pointers whose initialisers
    are string literals: emit one
    `.quad .Lx86str<id>` per element, registering
    each string so the render
    entry lays it down with a matching label.  (Not
     emitting `.globl` for a
    static leaves the symbol local to the
    translation unit.) */
static void x86_emit_rodata_ptrarr(void *decl,
    const char *name) {
     struct Node *d = (struct Node *)decl;
     if (!name) return;
     struct Type *full = (struct Type *)
    kcc_sema_resolve_type((void *)d->f0);
     if (!full) return;
     unsigned long total = kcc_type_layout(full);
     kcc_as_str(kcc_x86_os ? s_rodata : s_constsect
    );
     kcc_as_str(s_p2align3);
     if (d->f3 != 3) { kcc_as_str(s_globl); x86_sym
    (name); kcc_as_ch('\n'); }
     x86_sym(name); kcc_as_str(s_colon_nl);
     struct Node *init = (struct Node *)d->f2;
     struct Node *e = init ? (struct Node *)init->
    f0 : 0;
     unsigned long emitted = 0;
     for (; e; e = (struct Node *)e->f4) {
         if (e->tag == AST_STR_LIT) {
             unsigned long id = IL(kcc_x86_strs,
    VEC_COUNT);
             unsigned long entry[2]; entry[0] = e->
    f0; entry[1] = e->f1;
             kcc_vec_push(kcc_x86_strs, entry);
             kcc_as_str(s_d_quad); kcc_as_str(
    s_strlbl); kcc_as_dec(id); kcc_as_ch('\n');
         } else {
             kcc_as_str(s_d_quad); kcc_as_dec(
    kcc_rodata_leaf_bits(e, TY_PTR)); kcc_as_ch('\n
    ');
         }
         emitted++;
     }
     if (emitted * 8 < total) x86_emit_zero(total -
     emitted * 8);
}
```

□

787b · used in 792a

72.8. General Static Initializers

The three predicates above each materialise one *flat* shape—a const array of scalars, an array of string pointers, a struct whose leaves are all literals or a single level of symbol addresses. A global like the kernel's `init_task`, however, is an arbitrarily nested tree: designated initializers inside designated initializers, unions, arrays, compound literals `(cpumask_t){...}`, a `char comm[]="swapper"`, and—crucially—pointers to *its own members* (`LIST_HEAD_INIT(init_task.tasks)` is `{ &init_task.tasks, &init_task.tasks }`, i.e. `init_task` plus a member offset). None of the flat predicates accept it, so it fell to a zero `.comm`; the kernel derives its very first stack pointer from `init_task` (`initial_stack = init_thread_union + THREAD_SIZE`), so the zero left `%rsp` garbage and `startup_64` triple-faulted before a single boot message.

The general path handles any constant initializer the way gcc does: it *paints* the object into a byte image sized by `kcc_arena_alloc` (no fixed buffer—`init_task` is nearly 6 KB), recursing through every nested aggregate and writing each scalar leaf little-endian at its byte offset, bit-packing bit-fields into their container. A leaf that is a symbol address or a string literal cannot live in the image—it is a link-time relocation—so it is recorded in a side list and the eight pointer bytes are left zero. After painting, the relocations are sorted by offset and the object is emitted as `.byte` runs interrupted by a `.quad sym+off` (or `.quad` string-label) at each relocation, in the attributed section (or `.data` if writable, `.rodata` if const). Because this path is tried only *after* the three flat predicates, it changes nothing they already emit; it only replaces the zero `.comm` that everything else used to get.

⟨ *Selfhost: emit_x86 general-init* 789 ⟩ ≡

```
/* A relocation found while painting a static
   object: at byte OFFSET place an
   8-byte pointer to SYM+ADDEND, or --- when STRID
    >= 0 --- to string number STRID. */
struct X86Reloc { unsigned long off; const char *
   sym; long addend; long strid; };

/* Peel `(type){...}' compound literals and
   parentheses to the underlying init. */
static struct Node *x86_init_peel(struct Node *e)
   {
    while (e) {
        if (e->tag == AST_PAREN_EXPR)   { e = (
   struct Node *)e->f0; continue; }
        if (e->tag == AST_COMPOUND_LIT) { e = (
   struct Node *)e->f1; continue; }
        break;
    }
    return e;
}

/* Paint one scalar/pointer leaf into buf[base..
   base+size): a symbol address or a
   pointer-typed string literal becomes a
   relocation (the 8 bytes stay zero in the
   image, the linker fills them); anything else
   its little-endian bits.  Float
   bytes are written straight from the double bit
   pattern, matching the existing
   x86 field painter (no single-precision
   narrowing). */
static void x86_paint_leaf(unsigned char *buf,
   unsigned long total,
                           unsigned long base,
   unsigned long size, unsigned long kind,
                           struct Node *v, struct
   X86Reloc *rl, unsigned long *nrel,
                           unsigned long rmax) {
    if (!v) return;
    long off = 0;
    const char *s = kcc_ir_const_symaddr(v, &off);
    if (s) {
        if (*nrel < rmax) { rl[*nrel].off = base;
   rl[*nrel].sym = s;
                            rl[*nrel].addend = off
   ; rl[*nrel].strid = -1; (*nrel)++; }
        return;
    }
    if (v->tag == AST_STR_LIT && kind == TY_PTR) {
        unsigned long id = IL(kcc_x86_strs,
   VEC_COUNT);
        unsigned long entry[2]; entry[0] = v->f0;
   entry[1] = v->f1;
        kcc_vec_push(kcc_x86_strs, entry);
        if (*nrel < rmax) { rl[*nrel].off = base;
   rl[*nrel].sym = 0;
                            rl[*nrel].addend = 0;
   rl[*nrel].strid = (long)id; (*nrel)++; }
        return;
    }
    unsigned long bits = kcc_rodata_leaf_bits(v,
   kind);
    for (unsigned long b = 0; b < size; b++)
        if (base + b < total) { buf[base + b] = (
   unsigned char)(bits & 0xff); bits >>= 8; }
}

/* Recursively paint `init' of type `ty' into buf
   at byte offset `base', collecting
   pointer/string relocations.  Handles nested
   structs/unions/arrays,
    `.member'/`[i]' designators, bit-fields,
   compound literals and `char[]="..."'. */
static void x86_paint(unsigned char *buf, unsigned
    long total, struct Type *ty,
                      struct Node *init, unsigned
   long base,
                      struct X86Reloc *rl,
   unsigned long *nrel, unsigned long rmax) {
    if (!ty) return;
    init = x86_init_peel(init);
    unsigned long k = ty->kind;
    if (k == TY_STRUCT || k == TY_UNION) {
        if (!init || init->tag != AST_INIT_LIST)
   return;
        kcc_type_layout(ty);
        struct Node *decl = (struct Node *)ty->f0;
        struct Node *field = decl ? (struct Node
   *)decl->f1 : 0;   /* positional cursor */
        for (struct Node *e = (struct Node *)init
   ->f0; e; e = (struct Node *)e->f4) {
            struct Node *fld; struct Node *val;
   unsigned long foff;
            if (e->tag == AST_DESIGNATOR) {
                struct kcc_member_res mr =
   kcc_type_find_member(ty, (const char *)e->f0);
                fld = (struct Node *)mr.field; if
   (!fld) continue;
                foff = mr.off; val = (struct Node
   *)e->f3;
            } else {
                if (!field) break;
                fld = field; foff = field->f3; val
    = e;
                field = (struct Node *)field->f4;
            }
            unsigned long bf = fld->f2;
            if (bf) {
                /* bit-field: OR bits into container */
                unsigned long bitoff = (bf >> 16)
   & 0xffff, width = bf & 0xffff;
                unsigned long bits =
   kcc_rodata_leaf_bits(val, TY_INT);
                for (unsigned long b = 0; b <
   width; b++)
                    if ((bits >> b) & 1UL) {
                        unsigned long bit = bitoff
    + b, by = base + fld->f3 + (bit >> 3);
                        if (by < total) buf[by] |=
    (unsigned char)(1u << (bit & 7));
                    }
                continue;
            }
            struct Type *fty = (struct Type *)
   kcc_sema_resolve_type((void *)fld->f0);
            if (!fty) continue;
            unsigned long fk = fty->kind;
            if (fk == TY_STRUCT || fk == TY_UNION
   || fk == TY_ARRAY)
                x86_paint(buf, total, fty, val,
   base + foff, rl, nrel, rmax);
            else
                x86_paint_leaf(buf, total, base +
   foff, fty->size, fk,
                               x86_init_peel(val),
    rl, nrel, rmax);
        }
        return;
    }
    if (k == TY_ARRAY) {
        struct Type *et = (struct Type *)ty->f0;
        unsigned long es = et ? et->size : 1;
        if (init && init->tag == AST_STR_LIT) {
```

```
        /* char[] = "..." */
            unsigned char *sd = (unsigned char *)
    init->f0; unsigned long sl = init->f1;
            for (unsigned long i = 0; i < sl; i++)
                if (base + i < total) buf[base + i
    ] = sd[i];
            return;
        }
        if (!init || init->tag != AST_INIT_LIST ||
     !et) return;
        unsigned long ek = et->kind, idx = 0;
        for (struct Node *e = (struct Node *)init
    ->f0; e; e = (struct Node *)e->f4) {
            struct Node *v = e;
            if (e->tag == AST_DESIGNATOR) { idx =
    kcc_eval_const_int((void *)e->f0);
                                              v = (
    struct Node *)e->f3; }
            if (ek == TY_STRUCT || ek == TY_UNION
    || ek == TY_ARRAY)
                x86_paint(buf, total, et, v, base
    + idx * es, rl, nrel, rmax);
            else
                x86_paint_leaf(buf, total, base +
    idx * es, es, ek,
                               x86_init_peel(v),
    rl, nrel, rmax);
            idx++;
        }
        return;
     }
     /* scalar / pointer at top level (`T x = expr
    ;' or brace-elided `{expr}') */
     if (init && init->tag == AST_INIT_LIST) init =
     (struct Node *)init->f0;
     x86_paint_leaf(buf, total, base, ty->size, k,
    x86_init_peel(init), rl, nrel, rmax);
}

static void x86_emit_general(void *decl, const
    char *name) {
     struct Node *d = (struct Node *)decl;
     if (!name) return;
     struct Type *ty = (struct Type *)
    kcc_sema_resolve_type((void *)d->f0);
     if (!ty) return;
     unsigned long total = kcc_type_layout(ty);
     if (!total) return;
     unsigned char *buf = (unsigned char *)
    kcc_arena_alloc((long)total);
     for (unsigned long i = 0; i < total; i++) buf[
    i] = 0;
     unsigned long rmax = 1024, nrel = 0;
     struct X86Reloc *rl =
         (struct X86Reloc *)kcc_arena_alloc((long)(
    rmax * sizeof(struct X86Reloc)));
     x86_paint(buf, total, ty, (struct Node *)d->f2
    , 0, rl, &nrel, rmax);
     for (unsigned long i = 1; i < nrel; i++) {
         /* sort relocations by offset */
         struct X86Reloc t = rl[i]; unsigned long j
     = i;
         while (j > 0 && rl[j - 1].off > t.off) {
    rl[j] = rl[j - 1]; j--; }
         rl[j] = t;
     }
     struct AttrRec *ar = x86_attr(name);
     const char *sec = ar ? ar->section : 0;
     if (sec) { kcc_as_str(s_section); kcc_as_str(
    sec); kcc_as_str(s_aw); }
     else if (kcc_global_writable(d)) kcc_as_str(
    s_data);
     else kcc_as_str(kcc_x86_os ? s_rodata :
    s_constsect);
     kcc_as_str(s_p2align3);
     if (d->f3 != 3) { kcc_as_str(s_globl); x86_sym
    (name); kcc_as_ch('\n'); }
     else if (kcc_x86_os) { kcc_as_str(s_local);
    kcc_as_str(name); kcc_as_ch('\n'); }
     x86_sym(name); kcc_as_str(s_colon_nl);
     unsigned long cur = 0, ri = 0;
     while (cur < total) {
         if (ri < nrel && rl[ri].off == cur) {
             if (rl[ri].strid >= 0) {
                 kcc_as_str(s_d_quad); kcc_as_str(
    s_strlbl);
                 kcc_as_dec((unsigned long)rl[ri].
    strid); kcc_as_ch('\n');
             } else {
                 kcc_as_str(s_d_quad); x86_sym(rl[
    ri].sym);
                 if (rl[ri].addend > 0) {
    kcc_as_str(s_plus); kcc_as_dec((unsigned long)
    rl[ri].addend); }
                 else if (rl[ri].addend < 0) {
    kcc_as_str(s_minus); kcc_as_dec((unsigned long)
    (-rl[ri].addend)); }
                 kcc_as_ch('\n');
             }
             cur += 8; ri++;
         } else {
             unsigned long next = (ri < nrel) ? rl[
    ri].off : total;
             if (next > total) next = total;
             while (cur < next) { kcc_as_str(
    s_d_byte); kcc_as_dec(buf[cur]); kcc_as_ch('\n'
    ); cur++; }
         }
     }
     if (sec) kcc_as_str(s_text);
}
□
```

789 · used in 792a

72.9. Module Init, String Literals, and the Render Entry

⟨ *Selfhost: emit_x86 render* 791 ⟩ ≡

```
void kcc_x86_emit_modinit(void) {
    if (!kcc_modinit_present) return;
    kcc_as_str(kcc_x86_os ? s_modinit_l :
    s_modinit_d);
    x86_sym(s_modinitnm);
    kcc_as_ch('\n');
}

void kcc_x86_emit_strings(void) {
    void *strs = kcc_x86_strs;
    if (!strs) return;
    unsigned long n = IL(strs, VEC_COUNT);
    if (!n) return;
    kcc_as_str(kcc_x86_os ? s_rodata : s_cstrsect)
    ;
    for (unsigned long i = 0; i < n; i++) {
        kcc_as_str(s_strlbl); kcc_as_dec(i);
    kcc_as_str(s_label_nl);
        unsigned long *e = (unsigned long *)
    kcc_vec_get(strs, i);
```

```
        unsigned char *data = (unsigned char *)e
    [0];
        unsigned long len = e[1];
        kcc_as_str(s_d_byte);
        for (unsigned long b = 0; b < len; b++) {
    kcc_as_dec(data[b]); kcc_as_str(s_comma1); }
        kcc_as_str(s_zero_nl);
     }
}

/* A file-scope \verb|asm("...")| (e.g. every \
    texttt{EXPORT_SYMBOL}) carries a
    raw assembler template with its own \verb|.
    section|/\verb|.previous|
    directives; paste each one verbatim into the
    text stream. */
static void kcc_x86_emit_toplevel_asm(void) {
     struct Node *tu = (struct Node *)kcc_x86_tu;
     if (!tu) return;
     for (struct Node *d = (struct Node *)tu->f0; d
    ; d = (struct Node *)d->f4) {
        if (d->tag != AST_ASM_STMT) continue;
        const char *t = (const char *)d->f0;
        unsigned long n = d->f1;
        for (unsigned long i = 0; i < n; i++)
    kcc_as_ch((unsigned char)t[i]);
        kcc_as_ch('\n');
     }
}
/* `int foo(void) __attribute__((alias("bar")));`
    is a bodiless declaration, so
    it never reaches the IR function list; walk the
     translation unit and emit a
    `.set` for each name the parser recorded an
    alias target for. */
static void kcc_x86_emit_aliases(void) {
     struct Node *tu = (struct Node *)kcc_x86_tu;
     if (!tu) return;
     void *seen = kcc_ht_init(16);          /*
    alias names already emitted */
     for (struct Node *d = (struct Node *)tu->f0; d
    ; d = (struct Node *)d->f4) {
        if (d->tag != AST_FUNC_DECL && d->tag !=
    AST_VAR_DECL) continue;
        const char *name = (const char *)d->f1;
        struct AttrRec *ar = x86_attr(name);
        if (!ar || !ar->alias) continue;
        /* Attributes are keyed by name, so a
    prototype and the defining
           declaration of the same alias (mm/
    hugetlb.c both prototypes and
           defines the weak alias
    alloc_bootmem_huge_page) each reach here and
           would emit the `.set' twice---a
    redefinition gas rejects the moment
           the symbol is reloc-referenced.  Emit
    each alias name once. */
        unsigned long nl = kcc_strlen(name);
        if (kcc_ht_get(seen, name, nl)) continue;
        kcc_ht_put(seen, name, nl, 1);
        /* A `weak, alias(...)' pair---the kernel'
    s CPU_SHOW_VULN_FALLBACK
           defaults are `__weak' aliases of
    cpu_show_not_affected---must head the
           alias with `.weak', not `.globl'.
    Emitted global, the fallback would
           be a strong second definition and
    collide at the vmlinux link with the
           arch's real cpu_show_* override. */
        kcc_as_str(ar->weak ? s_weak : s_globl);
    x86_sym(name); kcc_as_ch('\n');
        kcc_as_str(s_set); x86_sym(name);
    kcc_as_str(s_comma);
        x86_sym(ar->alias); kcc_as_ch('\n');
     }
}
int kcc_x86_render(const char *path) {
     kcc_as_init();
     kcc_x86_strs = kcc_vec_init(16);
     x86_render_text();
     kcc_x86_emit_toplevel_asm();
     kcc_x86_emit_aliases();
     kcc_x86_emit_data();
     kcc_x86_emit_statics();
     kcc_x86_emit_strings();
     kcc_x86_emit_modinit();
     return kcc_as_write_file(path);
}
□
```

791 · used in 792a

⟨ *Selfhost: emit_x86* 792a ⟩ ≡

```
⟨ Selfhost: emit_x86 preamble 769 ⟩
⟨ Selfhost: emit_x86 helpers 771 ⟩
⟨ Selfhost: emit_x86 shapes 772 ⟩
⟨ Selfhost: emit_x86 dispatch 774 ⟩
⟨ Selfhost: emit_x86 frame 781 ⟩
⟨ Selfhost: emit_x86 data 784 ⟩
⟨ Selfhost: emit_x86 rodata 787b ⟩
⟨ Selfhost: emit_x86 general-init 789 ⟩
⟨ Selfhost: emit_x86 render 791 ⟩
□
```

792a · used in 36j

72.10. Verification

`make selfhost-emit-x86` first confirms the drop-in links and the ARM64 path is untouched (codegen 226), then verifies the x86 emitter functionally: a probe suite spanning the integer ALU, division, shifts, comparisons, control flow, recursion, many-argument calls, pointers, arrays, structs, globals, and string handling is compiled with `--target=x86_64` by the C emitter, run under Rosetta, and its exit status compared against the ARM64 reference built by the assembly compiler. Matching exits across the suite mean the C emitter lowers the whole implemented opcode set correctly.

⟨ *Selfhost: emit_x86 verify* 792b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-emit-x86 - compile kcc's x86-64
    back end in C with kcc, relink,
# and verify functionally (arm64 codegen unbroken
    + x86 programs run correctly).
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
    vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
    statements declarations \
types symtab conversions sema ir opt emit_arm64
    emit_x86 linker codegen emit_macho dwarf driver
    "
```

```
echo "[1/4] compiling $BASE/selfhost/emit_x86.c
   with kcc"
$BASE/kcc -c "$SH/emit_x86.c" -o "$SH/emit_x86_c.o
   " >/dev/null

echo "[2/4] linking $BASE/kcc_selfhost_emit_x86 (
   emit_x86.o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "emit_x86" ]; then OBJ="$OBJ $SH/
   emit_x86_c.o"; else OBJ="$OBJ $BASE/src/${f}.o"
   ; fi
done
ld -o $BASE/kcc_selfhost_emit_x86 $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64

cp $BASE/kcc "$SH/kcc_asm_backup_emit_x86"

echo "[3/4] x86-64 functional check: C emitter ->
   as -> Rosetta run vs reference"
mkdir -p "$SH/x86probes"
# A probe suite spanning the implemented opcode
   set.
cat > "$SH/x86probes/p1.c" <<'P1'
int g; static int add(int a,int b){return a+b;}
int fib(int n){ if(n<2) return n; return fib(n-1)+
   fib(n-2); }
int main(void){ g=7; int s=0; for(int i=0;i<5;i++)
    s+=add(i,g); return (s+fib(6))%100; }
P1
cat > "$SH/x86probes/p2.c" <<'P2'
unsigned uf(unsigned x){ return (x*3u + 7u) / 2u;
   }
int main(void){ int a=100,b=7; long r=0; r+=a/b; r
   +=a%b; r+=(a<<2); r+=(a>>1);
  r+=(a&0x3c); r+=(a|1); r+=(a^5); r+=(-a); r+=(~b
   ); r+=uf(41); return (int)(r%97); }
P2
cat > "$SH/x86probes/p3.c" <<'P3'
struct P{ int x; char c; long z; }; int arr[5];
int sum6(int a,int b,int c,int d,int e,int f){
   return a+b+c+d+e+f; }
int main(void){ struct P p; p.x=10; p.c=3; p.z
   =100; for(int i=0;i<5;i++) arr[i]=i*i;
  int t=0; for(int i=0;i<5;i++) t+=arr[i];
  return (p.x+p.c+(int)p.z+t+sum6(1,2,3,4,5,6))
   %100; }
P3
cat > "$SH/x86probes/p4.c" <<'P4'
const char *msg="hello"; const int tbl
   [4]={11,22,33,44};
int slen(const char*s){int n=0;while(*s){n++;s++;}
   return n;}
int cnt(void){ static int k; k++; return k; }
int main(void){ int r=slen(msg); for(int i=0;i<4;i
   ++) r+=tbl[i];
  r+=cnt(); r+=cnt(); r+=cnt(); return r%100; }
P4

status=0
for p in p1 p2 p3 p4; do
    src="$SH/x86probes/$p.c"
    # ARM64 reference (assembly compiler, native
   default target).
    cp "$SH/kcc_asm_backup_emit_x86" $BASE/kcc
    $BASE/kcc "$src" -o "$SH/x86probes/${p}_ref"
   >/dev/null 2>&1
    set +e; "$SH/x86probes/${p}_ref"; ref=$?; set
   -e
    # x86-64 via the C emitter, run under Rosetta.
    cp $BASE/kcc_selfhost_emit_x86 $BASE/kcc
    $BASE/kcc --target=x86_64 "$src" -o "$SH/
   x86probes/${p}_x86" >/dev/null 2>&1
    set +e; "$SH/x86probes/${p}_x86"; got=$?; set
   -e
    if [ "$ref" != "$got" ]; then
        echo "  FAIL $p: x86(C emitter) exit $got
   != arm64 reference $ref"; status=1
    else
        echo "  ok   $p: exit $got (x86 C emitter
   == arm64 reference)"
    fi
done
cp "$SH/kcc_asm_backup_emit_x86" $BASE/kcc
if [ $status -ne 0 ]; then exit 1; fi

echo "[4/4] codegen suite with the self-hosted x86
    back end linked in (arm64 path)"
cp $BASE/kcc_selfhost_emit_x86 $BASE/kcc
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_emit_x86" $BASE/kcc
exit $status
```

□

792b · used in 36k · uses _main 621, 629

73. Self-Hosting: The Preprocessor in C

The preprocessor of Chapters 17–17 (the spine plus its macro, conditional, and include chapters) is the last and most stateful slice of the front end, and porting it completes the *entire* front end—lexer, preprocessor, parser, and semantics—in kcc-compiled C. It wraps the Phase 3 lexer, executing directives and expanding macros so the parser sees a fully preprocessed token stream: `#define`/`#undef` with object- and function-like macros, argument prescan (§6.10.3.1), the # and ## operators, the `#if` family with a recursive-descent constant-expression evaluator, `#include` with a search path and a set of compiler-resident freestanding headers, and the `push_macro`/`pop_macro` pragmas.

The port is verified by the strongest oracle available—`difftest` byte-compares generated objects against clang, and every case runs through the preprocessor—plus the self-host and lexer suites and the diagnostic tests. The preprocessor-heavy real-world corpora (Lua, zlib, bzip2) exercise the macro machinery far past what a synthetic test would.

73.1. State, Constants, and Externs

⟨ *Selfhost: pp preamble* 795 ⟩ ≡

```
/* macro definition (56 bytes) */
struct PPMacro {
    const char *name; unsigned long namelen;
   unsigned long flags;
    unsigned long nparams; const char *body;
   unsigned long bodylen; void *params;
};
struct PPParam { const char *name; unsigned long
   len; };   /* 16 bytes */
struct PPArg { const char *start; unsigned long
   len; };     /* 16 bytes */
struct StrResult { char *buf; unsigned long len;
   };         /* returned in x0:x1 */
/* context stack entry (80 bytes): saved lexer/
   source state */
struct PPCtx {
    unsigned long type; const char *cursor; const
   char *end; unsigned long line;
    const char *lstart; const char *src_base;
   unsigned long src_size;
    const char *src_path; struct PPMacro *macro;
   unsigned long prevline;
    unsigned long nocomments;   /* saved
   kcc_lex_no_comments of the buffer we left */
};
/* push_macro save-stack node (32 bytes) */
struct PPSave { struct PPSave *next; const char *
   name; unsigned long len; void *saved; };

/* macro flags */
#define PP_FLAG_FUNCLIKE  1
#define PP_FLAG_EXPANDING 2
#define PP_FLAG_BUILTIN   4
#define PP_FLAG_VARIADIC  8
/* context types */
#define PP_CTX_INCLUDE 0
#define PP_CTX_EXPAND  1
/* conditional states */
#define PP_COND_ACTIVE 0
#define PP_COND_SKIP   1
#define PP_COND_DONE   2
/* limits */
#define PP_MAX_CTX_DEPTH 256
#define PP_MAX_COND 64
#define PP_MAX_ARGS 64

/* tokens */
enum {
    TOK_EOF=0, TOK_IDENT=1, TOK_NUMBER=2,
   TOK_STRING_LIT=4,
    TOK_LPAREN=60, TOK_RPAREN=61, TOK_COMMA=67,
   TOK_STAR=73, TOK_PLUS=74,
    TOK_MINUS=75, TOK_TILDE=76, TOK_BANG=77,
   TOK_SLASH=78, TOK_PERCENT=79,
    TOK_SHL=80, TOK_SHR=81, TOK_LT=82, TOK_GT=83,
   TOK_LE=84, TOK_GE=85,
    TOK_EQ=86, TOK_NE=87, TOK_CARET=88, TOK_PIPE
   =89, TOK_AND=90, TOK_OR=91,
    TOK_QUESTION=92, TOK_COLON=93, TOK_AMP=72,
   TOK_HASH=105, TOK_ELLIPSIS=107
};

/* externs - lexer / source globals */
extern unsigned long kcc_x86_os;     /* 1 when the
    target is Linux/x86-64 */
extern unsigned long kcc_tok_type, kcc_tok_len,
   kcc_tok_line;
extern const char *kcc_tok_start;
extern const char *kcc_lex_cursor, *kcc_lex_end, *
   kcc_lex_line_start;
extern unsigned long kcc_lex_line;
extern unsigned long kcc_lex_no_comments;
extern const char *kcc_src_base, *kcc_src_path;
extern unsigned long kcc_src_size;
void kcc_lex_next(void);
void kcc_lex_init(const char *base, unsigned long
   size);
void kcc_lex_dump_token(void);
/* externs - hashtable / arena / string / diag /
   os / source */
void *kcc_ht_init(unsigned long cap);
void kcc_ht_put(void *ht, const void *key,
   unsigned long keylen, void *val);
void *kcc_ht_get(void *ht, const void *key,
   unsigned long keylen);
void *kcc_arena_alloc(unsigned long n);
void *kcc_memcpy(void *d, const void *s, unsigned
   long n);
int kcc_memcmp(const void *a, const void *b,
   unsigned long n);
unsigned long kcc_strlen(const char *s);
void kcc_diag_error(unsigned long line, unsigned
   long col, const char *msg);
void kcc_diag_warning(unsigned long line, unsigned
    long col, const char *msg);
long kcc_sys_open(const char *path, unsigned long
   flags, unsigned long mode);
long kcc_sys_write(long fd, const void *buf,
   unsigned long len);
long kcc_sys_close(long fd);
void kcc_sys_exit(long code);
void kcc_source_open(const char *path);
void kcc_source_encode(void);
void kcc_source_splice(void);

/* exported state (driver/other modules reference
   these) */
```

```
void *kcc_pp_macros = 0;
long kcc_line_delta = 0;
unsigned long kcc_pp_counter = 0;
   /* __COUNTER__: next value to emit */
unsigned long kcc_pp_ctx_depth = 0;
/* internal state */
static struct PPSave *pp_macro_stack = 0;
static unsigned long pp_eval_depth = 0;
static unsigned long pp_drain_floor = 0;
static unsigned long pp_cond_depth = 0;
static unsigned long pp_prev_line = 0;
static unsigned long pp_has_pending = 0;
static unsigned long pp_dir_line = 0;
/* Whether the token kcc_pp_next just produced was
    preceded by whitespace in the
   effective source---1/0---for the -E text
   emitter to reproduce spacing across a
   macro-expansion boundary, where the byte before
    the token lives in a macro
   buffer and cannot be inspected directly.  `
   pp_pending_lead_ws' carries the
   leading whitespace of a macro invocation
   forward to the FIRST token of its
   expansion (and, unset until consumed, through
   nested expansions so the
   outermost invocation's spacing wins), mirroring
    gcc's PREV_WHITE flag. */
unsigned long kcc_pp_lead_ws = 0;
static long pp_pending_lead_ws = -1;
static struct PPCtx pp_ctx_stack[PP_MAX_CTX_DEPTH
   ];
static unsigned char pp_cond_stack[PP_MAX_COND];
static const char *pp_inc_dirs[64];
static unsigned long pp_inc_ndirs = 0;
static const char *pp_default_inc = 0;   /*
   compiler's own include/; searched LAST */
static const char *pp_cmdline_defs[64];
static unsigned long pp_cmdline_ndefs = 0;

/* error messages */
static const char m_unknown_dir[] = "unknown
   preprocessing directive";
static const char m_def_name[]   = "expected
   identifier after #define";
static const char m_def_param[] = "expected
   parameter name or ')' in macro parameter list";
static const char m_ifdef_name[]= "expected
   identifier after #ifdef/#ifndef";
static const char m_stray_elif[]= "#elif without
   matching #if";
static const char m_stray_else[]= "#else without
   matching #if";
static const char m_stray_endif[]="#endif without
   matching #if";
static const char m_include[]    = "#include
   expects \"file\" or <file>";
static const char m_notfound[]   = "#include file
   not found";
static const char m_unterm_cond[]="unterminated
   conditional directive";
static const char m_unterm_args[]="unterminated
   macro argument list";
static const char m_depth[]      = "maximum include
   /expansion depth exceeded";

/* forward decls */
void kcc_pp_next(void);
int kcc_pp_try_expand(void);
unsigned long kcc_pp_atoi(const char *p, unsigned
   long len);
long kcc_pp_eval_expr(void);
void kcc_pp_do_line(void);
void kcc_pp_add_include_dir(const char *dir);
void kcc_pp_apply_cmdline_defines(void);
static void pp_dispatch(void);
static void pp_skip_directive(void);
static void pp_ctx_pop(void);
static void pp_skip_line(void);
static const char *pp_rest_of_line(unsigned long *
   len_out);
static void pp_predefine(void);
static void pp_do_endif(void);
static void pp_do_elif(void);
static void pp_do_else(void);
static void pp_def_simple(const char *name,
   unsigned long nl, const char *body, unsigned
   long bl);
static const char *pp_prepare_cond(const char *src
   , unsigned long len, unsigned long *lo);
static const char *pp_builtin_header(const char *
   name);
static int kcc_strcmp2(const char *a, const char *
   b);
void kcc_pp_init(void);
void kcc_pp_dump_all(void);
void kcc_pp_do_define(void);
void kcc_pp_do_undef(void);
void kcc_pp_do_ifdef(int negate);
void kcc_pp_do_if(void);
void kcc_pp_do_include(void);
void kcc_pp_do_error(void);
void kcc_pp_do_pragma(void);
void kcc_pp_add_cmdline_define(const char *payload
   );
void kcc_pp_add_default_includes(const char *argv0
   );
const char *pp_substitute(const char *body,
   unsigned long blen, struct PPParam *pp,
                             unsigned long np, void *
   aa, void *raa, unsigned long ac, unsigned long
   *lo);
```

□

795 · used in 818a

73.2. Initialization, the Main Loop, and Contexts

kcc_pp_next returns one preprocessed token: it handles a false conditional branch by skipping to the next directive, a # at beginning-of-line as a directive, and a macro name by pushing its expansion. The context stack saves and restores lexer state across #include and macro expansion.

⟨ *Selfhost: pp core* 796 ⟩ ≡

```
void kcc_pp_init(void) {
    kcc_pp_macros = kcc_ht_init(256);
    kcc_pp_ctx_depth = 0;
    pp_cond_depth = 0;
    pp_prev_line = 0;
    pp_has_pending = 0;
    pp_predefine();
}

static void pp_ctx_push(unsigned long type) {
    if (kcc_pp_ctx_depth >= PP_MAX_CTX_DEPTH) {
        kcc_diag_error(kcc_tok_line, 1, m_depth);
        kcc_sys_exit(1);
    }
```

```
    struct PPCtx *e = &pp_ctx_stack[
   kcc_pp_ctx_depth];
    e->type = type;
    e->cursor = kcc_lex_cursor;
    e->end = kcc_lex_end;
    e->line = kcc_lex_line;
    e->lstart = kcc_lex_line_start;
    e->src_base = kcc_src_base;
    e->src_size = kcc_src_size;
    e->src_path = kcc_src_path;
    e->prevline = pp_prev_line;
    e->macro = 0;
    e->nocomments = kcc_lex_no_comments;
    kcc_pp_ctx_depth++;
}

static void pp_ctx_push_expand(struct PPMacro *m,
   const char *buf, unsigned long buflen) {
    pp_ctx_push(PP_CTX_EXPAND);
    pp_ctx_stack[kcc_pp_ctx_depth - 1].macro = m;
    kcc_lex_cursor = buf;
    kcc_lex_end = buf + buflen;
    kcc_lex_line_start = buf;
    /* A macro's expanded body/argument text (`m'
   non-null) is already past phase 3,
       so a `//' or `/*' it grows by pasting
   tokens---the kernel's tracepoint path
       `__stringify(TRACE_INCLUDE_PATH/system.h)'
   concatenates a `.../' directory
       onto `/system.h'---must lex as two `/'
   operators, never a comment that would
       eat the rest of the argument (and its
   closing `)') and report an
       "unterminated macro argument list".  A raw
   source span replayed for its own
       sake (`m' null: a computed #include line,
   an #if expression) keeps ordinary
       comment stripping. */
    kcc_lex_no_comments = m ? 1 : 0;
}

static void pp_ctx_pop(void) {
    if (kcc_pp_ctx_depth == 0) return;
    kcc_pp_ctx_depth--;
    struct PPCtx *e = &pp_ctx_stack[
   kcc_pp_ctx_depth];
    if (e->type == PP_CTX_INCLUDE) {
        kcc_src_base = e->src_base;
        kcc_src_size = e->src_size;
        kcc_src_path = e->src_path;
    } else if (e->macro) {
        e->macro->flags &= ~(unsigned long)
   PP_FLAG_EXPANDING;
    }
    kcc_lex_cursor = e->cursor;
    kcc_lex_end = e->end;
    kcc_lex_line = e->line;
    kcc_lex_line_start = e->lstart;
    pp_prev_line = e->prevline;
    kcc_lex_no_comments = e->nocomments;
}

void kcc_pp_next(void) {
    for (;;) {
        if (pp_has_pending) pp_has_pending = 0;
        else kcc_lex_next();
        unsigned long tt = kcc_tok_type;
        unsigned long tl = kcc_tok_line;
        if (tt == TOK_EOF) {
            if (kcc_pp_ctx_depth) {
                pp_ctx_pop();
                if (pp_drain_floor &&
   kcc_pp_ctx_depth < pp_drain_floor) return;
                continue;
            }
            if (pp_cond_depth) kcc_diag_error(
   kcc_tok_line, 1, m_unterm_cond);
            return;
        }
        /* conditional skip */
        if (pp_cond_depth && pp_cond_stack[
   pp_cond_depth - 1] != PP_COND_ACTIVE) {
            if (tt == TOK_HASH && tl >
   pp_prev_line) {
                pp_prev_line = tl;
                pp_skip_directive();
                continue;
            }
            pp_prev_line = tl;
            continue;
        }
        /* directive at BOL */
        if (tt == TOK_HASH && tl > pp_prev_line) {
            pp_prev_line = tl;
            pp_dispatch();
            continue;
        }
        /* macro expansion: IDENT or keyword-class
    [10,60) */
        if (tt == TOK_IDENT || (tt - 10) < 50) {
            /* Capture the macro name's own
   leading whitespace before expanding;
               if the expansion fires and no outer
    invocation's spacing is already
               pending, it becomes the first
   expanded token's leading whitespace. */
            long namews = (kcc_tok_start[-1] == '
   ' || kcc_tok_start[-1] == 0x09) ? 1 : 0;
            if (kcc_pp_try_expand()) {
                if (pp_pending_lead_ws < 0)
   pp_pending_lead_ws = namews;
                continue;
            }
        }
        pp_prev_line = tl;
        /* Publish the leading-whitespace flag for
    the -E emitter: a pending value
           from a macro invocation overrides (and
   is consumed), otherwise it is the
           byte just before this token in its own
   buffer. */
        if (pp_pending_lead_ws >= 0) {
            kcc_pp_lead_ws = (unsigned long)
   pp_pending_lead_ws;
            pp_pending_lead_ws = -1;
        } else {
            kcc_pp_lead_ws = (kcc_tok_start[-1] ==
    ' ' || kcc_tok_start[-1] == 0x09) ? 1 : 0;
        }
        return;
    }
}

void kcc_pp_dump_all(void) {
    for (;;) {
        kcc_pp_next();
        if (kcc_tok_type == TOK_EOF) return;
        kcc_lex_dump_token();
    }
}
```

□

796 · used in 818a

73.3. Helpers: Skip-Line and Rest-of-Line

pp_skip_line advances the cursor past the line's newline, skipping a block comment that spans the newline so a skipped directive's logical line ends where the comment does; pp_rest_of_line returns the directive's remaining text, trimmed, scanning past block comments and string/character literals so a newline inside one does not end the directive.

⟨ *Selfhost: pp helpers* 798 ⟩ ≡

```
static void pp_skip_line(void) {
    const char *p = kcc_lex_cursor;
    const char *end = kcc_lex_end;
    while (p < end) {
        char c = *p;
        /* A block comment is whitespace that may span newlines (phase 3), so a
           directive's trailing `/ * ... * /` extends its logical line past those
           newlines.  Skip the whole comment, or a skipped `#define X 0x20 / *
           ...multi-line... * /` would leave the cursor mid-comment and the
           apostrophe in a word like "that's" would then be lexed as a char
           constant (kernel uapi/linux/openat2.h). */
        if (c == '/' && p + 1 < end && p[1] == '*') {
            p += 2;
            while (p < end && !(*p == '*' && p + 1 < end && p[1] == '/')) {
                if (*p == 0x0A) { kcc_lex_line++; kcc_lex_line_start = p + 1; }
                p++;
            }
            if (p < end) p += 2;
        /* consume the closing star-slash */
            continue;
        }
        p++;
        if (c == 0x0A) {
            kcc_lex_line++;
            kcc_lex_line_start = p;
            break;
        }
    }
    kcc_lex_cursor = p;
}

static const char *pp_rest_of_line(unsigned long *len_out) {
    const char *p = kcc_lex_cursor;
    const char *end = kcc_lex_end;
    while (p < end && (*p == ' ' || *p == 0x09)) p++;
    const char *start = p;
    /* Return a comment-stripped copy of the body (translation phase 3, which
       precedes macro definition): a macro body must not retain comments, or an
       apostrophe or quote inside one---as in the kernel's doc comment
       "a node's interval"---would later flip pp_substitute into char/string mode
       and copy the rest of the body verbatim, leaving every parameter and `##`
       past that point unexpanded.  Each comment becomes one space so adjacent
       tokens stay separated; string and character literals are copied intact. */
    unsigned long cap = (unsigned long)(end - start) + 2;
    char *out = (char *)kcc_arena_alloc(cap);
    unsigned long op = 0;
    int state = 0;                                  /* 0 normal,1 block,2 str,3 char */
    const char *nl = 0;
    while (p < end) {
        char c = *p;
        if (state == 1) {                           /* block comment: skip, emit a space at its end */
            if (c == '*' && p + 1 < end && p[1] == '/') { state = 0; p += 2; out[op++] = ' '; continue; }
            if (c == 0x0A) kcc_lex_line++;
            p++;
            continue;
        }
        if (state == 2) {                           /* string literal: copy verbatim */
            if (c == 0x5C) { out[op++] = c; if (p + 1 < end) out[op++] = p[1]; p += 2; continue; }
            if (c == '"') { out[op++] = c; state = 0; p++; continue; }
            if (c == 0x0A) { nl = p; break; }
            out[op++] = c; p++;
            continue;
        }
        if (state == 3) {                           /* character constant: copy verbatim */
            if (c == 0x5C) { out[op++] = c; if (p + 1 < end) out[op++] = p[1]; p += 2; continue; }
            if (c == 0x27) { out[op++] = c; state = 0; p++; continue; }
            if (c == 0x0A) { nl = p; break; }
            out[op++] = c; p++;
            continue;
        }
        if (c == 0x0A) { nl = p; break; }
        if (c == '"') { out[op++] = c; state = 2; p++; continue; }
        if (c == 0x27) { out[op++] = c; state = 3; p++; continue; }
        if (c == '/' && p + 1 < end && p[1] == '*') { state = 1; p += 2; continue; }
        if (c == '/' && p + 1 < end && p[1] == '/') {   /* line comment: to logical end-of-line */
            p += 2;
            while (p < end && *p != 0x0A) p++;
            continue;
        }
        out[op++] = c; p++;
    }
    if (nl) kcc_lex_cursor = nl;
    else kcc_lex_cursor = end;
    /* trim trailing whitespace */
    while (op && (out[op - 1] == ' ' || out[op - 1] == 0x09)) op--;
    *len_out = op;
    return out;
}
```

□

798 · used in 818a

73.4. Directive Dispatch

⟨ *Selfhost: pp dispatch* 799a ⟩ ≡

```
static int dname(const char *n, unsigned long l,
   const char *s, unsigned long sl) {
    return l == sl && kcc_memcmp(n, s, sl) == 0;
}

static void pp_dispatch(void) {
    pp_dir_line = kcc_tok_line;
    kcc_lex_next();
    unsigned long tt = kcc_tok_type;
    if (tt != TOK_IDENT && (tt < 10 || tt > 53)) {
    pp_skip_line(); return; }
    const char *n = kcc_tok_start;
    unsigned long l = kcc_tok_len;
    if (dname(n, l, "define", 6)) kcc_pp_do_define
   ();
    else if (dname(n, l, "undef", 5))
   kcc_pp_do_undef();
    else if (dname(n, l, "ifdef", 5))
   kcc_pp_do_ifdef(0);
    else if (dname(n, l, "ifndef", 6))
   kcc_pp_do_ifdef(1);
    else if (dname(n, l, "if", 2)) kcc_pp_do_if();
    else if (dname(n, l, "elif", 4)) pp_do_elif();
    else if (dname(n, l, "else", 4)) pp_do_else();
    else if (dname(n, l, "endif", 5)) pp_do_endif
   ();
    else if (dname(n, l, "include", 7))
   kcc_pp_do_include();
    else if (dname(n, l, "error", 5))
   kcc_pp_do_error();
    else if (dname(n, l, "line", 4))
   kcc_pp_do_line();
    else if (dname(n, l, "pragma", 6))
   kcc_pp_do_pragma();
    else {
        kcc_diag_warning(pp_dir_line, 1,
   m_unknown_dir);
        pp_skip_line();
    }
}
```

□

799a · used in 818a

73.5. `#define`, `#undef`, and Pragmas

`kcc_pp_do_define` reads the macro name and, if a `(` follows it with no intervening space, the parameter list. Each parameter is an identifier; a bare `...` in final position makes the macro variadic, binding the trailing arguments to `__VA_ARGS__` (§6.10.3). kcc also accepts the GNU *named* variadic form `name...`: an identifier immediately followed by `...` names the rest parameter, so the macro body refers to the trailing arguments as `name` rather than `__VA_ARGS__`. This spelling is pervasive in the Linux kernel headers (for example `ASM_NO_INPUT_CLOBBER(clbr...)`). In both cases the binding name is written into the variadic slot `params[nparams]` here, at definition time, and the fixed-parameter count `nparams` excludes it; the expander later collapses the trailing arguments into that already-named slot.

⟨ *Selfhost: pp define* 799b ⟩ ≡

```
void kcc_pp_do_define(void) {
    kcc_lex_next();
    unsigned long tt = kcc_tok_type;
    if (tt != TOK_IDENT && (tt - 10) >= 50) {
        kcc_diag_error(pp_dir_line, 1, m_def_name)
;
        pp_skip_line();
        return;
    }
    const char *name = kcc_tok_start;
    unsigned long namelen = kcc_tok_len;
    unsigned long flags = 0, nparams = 0;
    struct PPParam *params = 0;
    /* function-like iff '(' immediately follows
the name (no space) */
    int funclike = (kcc_lex_cursor < kcc_lex_end
&& *kcc_lex_cursor == '(');
    if (funclike) {
        flags = PP_FLAG_FUNCLIKE;
        kcc_lex_next();
/* '(' */
        kcc_lex_next();
/* first param or ')' */
        params = (struct PPParam *)kcc_arena_alloc
(PP_MAX_ARGS * sizeof(struct PPParam));
        if (kcc_tok_type != TOK_RPAREN) {
            for (;;) {
                if (kcc_tok_type == TOK_ELLIPSIS)
{
                    flags |= PP_FLAG_VARIADIC;
                    params[nparams].name = "
__VA_ARGS__";
                    params[nparams].len = 11;
                    kcc_lex_next();
                    if (kcc_tok_type != TOK_RPAREN
) { kcc_diag_error(pp_dir_line, 1, m_def_param)
; pp_skip_line(); return; }
                    break;
                }
                /* A parameter name, like the
macro name, may be a keyword-class
                   token [10,60): phase 4 has no
keywords (6.4.1), and the
                   kernel's TRACE_EVENT names a
parameter `struct`. */
                if (kcc_tok_type != TOK_IDENT && (
kcc_tok_type - 10) >= 50) { kcc_diag_error(
pp_dir_line, 1, m_def_param); pp_skip_line();
return; }
                params[nparams].name =
kcc_tok_start;
                params[nparams].len = kcc_tok_len;
                kcc_lex_next();
                if (kcc_tok_type == TOK_ELLIPSIS)
{
                    /* GNU named variadic
parameter "name...": bind the trailing
                       arguments to `name` rather
than to __VA_ARGS__.  Leave the
                       name in the variadic slot
params[nparams] without counting
                       it among the fixed
parameters. */
                    flags |= PP_FLAG_VARIADIC;
                    kcc_lex_next();
                    if (kcc_tok_type != TOK_RPAREN
) { kcc_diag_error(pp_dir_line, 1, m_def_param)
; pp_skip_line(); return; }
                    break;
                }
                nparams++;
```

```
            if (kcc_tok_type == TOK_COMMA) {
   kcc_lex_next(); continue; }
            if (kcc_tok_type != TOK_RPAREN) {
   kcc_diag_error(pp_dir_line, 1, m_def_param);
   pp_skip_line(); return; }
            break;
          }
        }
    }
    unsigned long bodylen;
    const char *body = pp_rest_of_line(&bodylen);
    struct PPMacro *m = (struct PPMacro *)
   kcc_arena_alloc(sizeof(struct PPMacro));
    m->name = name; m->namelen = namelen; m->flags
    = flags; m->nparams = nparams;
    m->body = body; m->bodylen = bodylen; m->
   params = params;
    kcc_ht_put(kcc_pp_macros, name, namelen, m);
    pp_skip_line();
}

void kcc_pp_do_undef(void) {
    kcc_lex_next();
    if (kcc_tok_type == TOK_IDENT)
        kcc_ht_put(kcc_pp_macros, kcc_tok_start,
   kcc_tok_len, 0);
    pp_skip_line();
}

/* Parse ( "NAME" ) after push_macro/pop_macro;
   returns name/len (len 0 = fail). */
static const char *pp_pragma_name(unsigned long *
   lo) {
    kcc_lex_next();
    if (kcc_tok_type != TOK_LPAREN) { *lo = 0;
   return 0; }
    kcc_lex_next();
    if (kcc_tok_type != TOK_STRING_LIT) { *lo = 0;
    return 0; }
    *lo = kcc_tok_len - 2;
    return kcc_tok_start + 1;
}

void kcc_pp_do_pragma(void) {
    kcc_lex_next();
    if (kcc_tok_type == TOK_IDENT) {
        const char *v = kcc_tok_start;
        unsigned long vl = kcc_tok_len;
        if (vl == 10 && kcc_memcmp(v, "push_macro"
   , 10) == 0) {
            unsigned long nl;
            const char *nm = pp_pragma_name(&nl);
            if (nl) {
                void *saved = kcc_ht_get(
   kcc_pp_macros, nm, nl);
                struct PPSave *node = (struct
   PPSave *)kcc_arena_alloc(sizeof(struct PPSave))
   ;
                node->name = nm; node->len = nl;
   node->saved = saved;
                node->next = pp_macro_stack;
                pp_macro_stack = node;
            }
        } else if (vl == 9 && kcc_memcmp(v, "
   pop_macro", 9) == 0) {
            unsigned long nl;
            const char *nm = pp_pragma_name(&nl);
            if (nl) {
                struct PPSave **link = &
   pp_macro_stack;
                while (*link) {
                    struct PPSave *node = *link;
                    if (node->len == nl &&
   kcc_memcmp(nm, node->name, nl) == 0) {
                        kcc_ht_put(kcc_pp_macros,
   nm, nl, node->saved);
                        *link = node->next;
                        break;
                    }
                    link = &node->next;
                }
            }
        }
    }
    pp_skip_line();
}
□
```

799b · used in 818a

73.6. Macro Expansion

kcc_pp_try_expand looks the identifier up, honours the anti-recursion flag, substitutes __LINE__ and __FILE__ dynamically, and for a function-like macro collects arguments (across context boundaries, never past the argument prescan floor), prescan-expands them, substitutes, and pushes the result. __FILE__ cannot be a static body because its value is the current source path (kcc_src_path, which the include machinery saves and restores per file); it is emitted as a freshly-quoted string-literal token at each use.

Argument collection normally records each argument as a contiguous source span, but the kernel drives a conditional *through* a macro call—malloc(flush ? a() : #ifdef X ... #else sizeof(...) #endif)—so a # directive can interrupt an argument. A bare span would swallow the directive keywords and both branches; instead, when a directive is seen at line start during collection it is evaluated with the ordinary conditional machinery (pp_dispatch when the enclosing branch is live, pp_skip_directive otherwise), tokens inside a dead branch are dropped, and—because the surviving tokens are no longer contiguous—the argument is rebuilt token-by-token in an arena buffer via pp_arg_append. Arguments with no directive keep the fast span path.

try_expand also intercepts the _Pragma("…") operator here, before the macro lookup: it is not a macro but a destringized #pragma, which the kernel plants through __diag_ignore_all to toggle GCC diagnostics kcc does not act on. Recognised by name, it consumes its parenthesised argument—balancing the nested parens of the __diag_str that constructs the string—and expands to nothing, locating its (exactly as a function-like macro does.

⟨ *Selfhost: pp expand* 800 ⟩ ≡

```
static const char *pp_line_digits(unsigned long *
   lo) {
    unsigned long line = kcc_lex_line + (unsigned
   long)kcc_line_delta;
    char *buf = (char *)kcc_arena_alloc(24);
    char *e = buf + 24;
    if (line == 0) { *--e = '0'; }
    else while (line) { *--e = '0' + (line % 10);
   line /= 10; }
    *lo = (unsigned long)((buf + 24) - e);
```

```
    return e;
}

static const char *pp_expand_arg(const char *text,
    unsigned long len, unsigned long *lo);

/* Append s[0..n) to an arena-backed,
   geometrically-grown buffer.  Used to
   rebuild a macro argument whose token run a
   conditional directive interrupts. */
static void pp_arg_append(char **buf, unsigned
   long *cap, unsigned long *len,
                          const char *s, unsigned
   long n) {
    if (*len + n > *cap) {
        unsigned long nc = *cap ? *cap : 256;
        while (nc < *len + n) nc = nc * 2;
        char *nb = (char *)kcc_arena_alloc(nc);
        kcc_memcpy(nb, *buf, *len);
        *buf = nb; *cap = nc;
    }
    kcc_memcpy(*buf + *len, s, n);
    *len = *len + n;
}

int kcc_pp_try_expand(void) {
    const char *name = kcc_tok_start;
    unsigned long namelen = kcc_tok_len;
    struct PPMacro *m = (struct PPMacro *)
   kcc_ht_get(kcc_pp_macros, name, namelen);
    /* The `_Pragma("...")' operator is a
   destringized #pragma, not a macro.  The
       kernel plants it through __diag_ignore_all
   (compiler-gcc.h) to toggle GCC
       diagnostics; kcc honours none of those, so
   consume the parenthesised
       string-literal argument (balancing nested
   parens, e.g. the __diag_str(...)
       that builds it) and expand to nothing.  The
    `(' is located exactly as a
       function-like macro's is, dropping only
   exhausted expansion contexts. */
    if (namelen == 7 && kcc_memcmp(name, "_Pragma"
   , 7) == 0) {
        for (;;) {
            if (kcc_lex_cursor < kcc_lex_end)
   break;
            if (kcc_pp_ctx_depth == 0) break;
            if (kcc_pp_ctx_depth <= pp_drain_floor
   ) break;
            pp_ctx_pop();
        }
        const char *p = kcc_lex_cursor;
        while (p < kcc_lex_end && (*p == ' ' || *p
    == 0x09 || *p == 0x0A || *p == 0x0D)) p++;
        if (p >= kcc_lex_end || *p != '(') return
   0;   /* bare `_Pragma' → leave as ident */
        kcc_lex_next();
         /* '(' */
        long pdepth = 1;
        for (;;) {
            kcc_lex_next();
            unsigned long tt = kcc_tok_type;
            if (tt == TOK_EOF) return 1;
            if (tt == TOK_LPAREN) pdepth++;
            else if (tt == TOK_RPAREN) { pdepth--;
    if (pdepth == 0) break; }
        }
        return 1;
         /* expands to nothing */
    }
    if (!m) return 0;
    if (m->flags & PP_FLAG_EXPANDING) return 0;
    /* dynamic __LINE__ */
    if ((m->flags & PP_FLAG_BUILTIN) && namelen ==
    8 && kcc_memcmp(name, "__LINE__", 8) == 0) {
        unsigned long dl;
        const char *d = pp_line_digits(&dl);
        m->flags |= PP_FLAG_EXPANDING;
        pp_ctx_push_expand(m, d, dl);
        return 1;
    }
    /* dynamic __FILE__ → the current source path
   as a string literal.  The
      * include machinery saves/restores
   kcc_src_path, so this is the header's
      * own name inside a header (C17 6.10.8.1).
   GCC's __BASE_FILE__ is the
      * primary source rather than the current
   header; kcc approximates it with
      * the same current path---exact for the usual
    case (it is expanded in the
      * .c itself, e.g. sound/core/seq's ALSA
   logging) and harmless otherwise. */
    if ((m->flags & PP_FLAG_BUILTIN) &&
        ((namelen == 8 && kcc_memcmp(name, "
   __FILE__", 8) == 0) ||
         (namelen == 13 && kcc_memcmp(name, "
   __BASE_FILE__", 13) == 0))) {
        const char *p = kcc_src_path ?
   kcc_src_path : "<stdin>";
        unsigned long pl = kcc_strlen(p);
        char *buf = (char *)kcc_arena_alloc(pl +
   3);
        buf[0] = '"';
        kcc_memcpy(buf + 1, p, pl);
        buf[pl + 1] = '"';
        m->flags |= PP_FLAG_EXPANDING;
        pp_ctx_push_expand(m, buf, pl + 2);
        return 1;
    }
    /* dynamic __COUNTER__ → a fresh incrementing
   integer at each use. */
    if ((m->flags & PP_FLAG_BUILTIN) && namelen ==
    11 && kcc_memcmp(name, "__COUNTER__", 11) ==
   0) {
        unsigned long v = kcc_pp_counter++;
        char *buf = (char *)kcc_arena_alloc(24);
        char *e = buf + 24;
        if (v == 0) { *--e = '0'; }
        else while (v) { *--e = '0' + (v % 10); v
   /= 10; }
        m->flags |= PP_FLAG_EXPANDING;
        pp_ctx_push_expand(m, e, (unsigned long)((
   buf + 24) - e));
        return 1;
    }
    if (!(m->flags & PP_FLAG_FUNCLIKE)) {
        /* object-like */
        unsigned long bl = m->bodylen;
        if (!bl) return 1;
   /* empty */
        char *buf = (char *)kcc_arena_alloc(bl);
        kcc_memcpy(buf, m->body, bl);
        m->flags |= PP_FLAG_EXPANDING;
        pp_ctx_push_expand(m, buf, bl);
        return 1;
    }
    /* function-like: drop exhausted contexts to
   find '(' in enclosing text,
      * but never cross the argument-prescan drain
   floor. */
    for (;;) {
```

```
    if (kcc_lex_cursor < kcc_lex_end) break;
    if (kcc_pp_ctx_depth == 0) break;
    if (kcc_pp_ctx_depth <= pp_drain_floor)
break;
    pp_ctx_pop();
 }
 const char *p = kcc_lex_cursor;
 while (p < kcc_lex_end && (*p == ' ' || *p ==
0x09 || *p == 0x0A || *p == 0x0D)) p++;
 if (p >= kcc_lex_end || *p != '(') return 0;
 kcc_lex_next();
/* '(' */
 unsigned long nparams = m->nparams;
 struct PPParam *params = (struct PPParam *)m->
params;
 struct PPArg *args = (struct PPArg *)
kcc_arena_alloc(PP_MAX_ARGS * 16);
 unsigned long argc = 0;
 long depth = 1;
 args[0].start = kcc_lex_cursor;
 args[0].len = 0;
 /* Buffer state for an argument a conditional
directive interrupts: `abuf' is
    non-null once this argument switched from
span to rebuilt-buffer mode, and
    `aprev' is the source end of the last token
 folded into it. */
 char *abuf = 0; unsigned long acap = 0, alen =
 0;
 const char *aprev = args[0].start;
 int any_buf = 0;                  /* any
argument rebuilt in a buffer? */
 for (;;) {
    kcc_lex_next();
    unsigned long tt = kcc_tok_type;
    unsigned long tl = kcc_tok_line;
    if (tt == TOK_EOF) { kcc_diag_error(
pp_dir_line, 1, m_unterm_args); return 0; }
    /* A directive at line start interrupts
the argument (GNU extension): fold
       the live text collected so far into a
buffer, evaluate the directive so
       the conditional stack tracks which
branch is taken, and continue. */
    if (tt == TOK_HASH && tl > pp_prev_line) {
        if (!abuf) { acap = 256; abuf = (char
*)kcc_arena_alloc(acap); alen = 0; any_buf = 1;
 }
        pp_arg_append(&abuf, &acap, &alen,
aprev, (unsigned long)(kcc_tok_start - aprev));
        pp_prev_line = tl;
        if (pp_cond_depth && pp_cond_stack[
pp_cond_depth - 1] != PP_COND_ACTIVE)
            pp_skip_directive();
        else
            pp_dispatch();
        aprev = kcc_lex_cursor;
        continue;
    }
    /* Tokens inside a dead conditional branch
 are dropped from the argument. */
    if (pp_cond_depth && pp_cond_stack[
pp_cond_depth - 1] != PP_COND_ACTIVE) {
        pp_prev_line = tl;
        aprev = kcc_lex_cursor;
        continue;
    }
    pp_prev_line = tl;
    if (tt == TOK_LPAREN) { depth++; continue;
 }
    if (tt == TOK_RPAREN) {
        depth--;
        if (depth == 0) {
            if (abuf) {
                pp_arg_append(&abuf, &acap, &
alen, aprev, (unsigned long)(kcc_tok_start -
aprev));
                args[argc].start = abuf; args[
argc].len = alen;
            } else {
                args[argc].len = (unsigned
long)(kcc_tok_start - args[argc].start);
            }
            argc++;
            break;
        }
        continue;
    }
    if (tt == TOK_COMMA && depth == 1) {
        if (abuf) {
            pp_arg_append(&abuf, &acap, &alen,
 aprev, (unsigned long)(kcc_tok_start - aprev))
;
            args[argc].start = abuf; args[argc
].len = alen;
        } else {
            args[argc].len = (unsigned long)(
kcc_tok_start - args[argc].start);
        }
        argc++;
        args[argc].start = kcc_lex_cursor;
        args[argc].len = 0;
        abuf = 0; acap = 0; alen = 0;
        aprev = kcc_lex_cursor;
    }
 }
 /* variadic: collapse trailing args into the
variadic slot params[nparams],
    whose name (__VA_ARGS__, or the GNU named-
rest identifier) was bound when
    the macro was defined. */
 if (m->flags & PP_FLAG_VARIADIC) {
    if (argc > nparams) {
        if (any_buf) {
            /* Some trailing argument was
rebuilt in its own buffer, so the
               arguments are no longer one
contiguous source run and the
               pointer span below would be
meaningless; join them---with the
               separating commas---into a
fresh buffer instead. */
            char *vb = 0; unsigned long vc =
0, vl = 0;
            for (unsigned long i = nparams; i
< argc; i++) {
                if (i > nparams) pp_arg_append
(&vb, &vc, &vl, ",", 1);
                pp_arg_append(&vb, &vc, &vl,
args[i].start, args[i].len);
            }
            args[nparams].start = vb; args[
nparams].len = vl;
        } else {
            const char *first = args[nparams].
start;
            const char *last_end = args[argc -
 1].start + args[argc - 1].len;
            args[nparams].len = (unsigned long
)(last_end - first);
        }
    } else {
        args[nparams].start = args[0].start;
```

```
            args[nparams].len = 0;
        }
        nparams++;
        argc = nparams;
    }
    /* Preserve each argument's raw (pre-prescan)
   span: a parameter that is an
       operand of `#' (stringise, 6.10.3.2) or
   `##' (paste, 6.10.3.1) uses its
       argument UNexpanded, whereas an ordinary
   parameter uses the prescan
       expansion.  The kernel's ftrace_eval_map
   generator relies on this ---
       `__##TRACE_SYSTEM##_##a' must paste the
   enum's spelling, not its value. */
    struct PPArg *rawargs = (struct PPArg *)
   kcc_arena_alloc(PP_MAX_ARGS * 16);
    for (unsigned long i = 0; i < argc; i++)
   rawargs[i] = args[i];
    /* argument prescan (6.10.3.1) */
    for (unsigned long i = 0; i < argc; i++) {
        unsigned long el;
        const char *eb = pp_expand_arg(args[i].
   start, args[i].len, &el);
        args[i].start = eb;
        args[i].len = el;
    }
    /* substitute */
    unsigned long rl;
    const char *rbuf = pp_substitute(m->body, m->
   bodylen, params, nparams, args, rawargs, argc,
   &rl);
    if (!rl) return 1;
    m->flags |= PP_FLAG_EXPANDING;
    pp_ctx_push_expand(m, rbuf, rl);
    return 1;
}
```

□

800 · used in 818a

73.7. Argument Prescan and Substitution

pp_expand_arg drives kcc_pp_next over an argument's text behind a *drain floor*, so expansion stops at the argument boundary rather than reading into the enclosing text. pp_substitute walks the body, replacing parameter names with argument text and honouring the # (stringise) and ## (paste) operators. The choice of *which* argument text is the crux of §6.10.3.1: an ordinary parameter takes the prescan expansion, but a parameter that is an operand of # or ## takes its *raw*, unexpanded argument. Both spans are kept (the prescan overwrites the argument array, so the raw spans are copied aside first); prev_paste flags that the previous body token was ## so the following parameter is its right operand, and a one-token look-ahead catches a parameter that is a *left* ## operand. Without this the kernel's ftrace eval-map generator misfires: __##TRACE_SYSTEM##_##a would paste an enum's *value* ((1 << …)) instead of its spelling, because the argument would have been expanded before the paste. It also implements the GNU comma-elision special case , ## __VA_ARGS__: when the right operand of ## is a parameter whose argument is empty, the paste deletes both the operand and a preceding comma—the mechanism behind the kernel's static_assert(expr, ...) wrapper, and thus a hard prerequisite for the <linux/bitops.h> sanity assertions.

Because substitution splices an argument's text directly into the body, a trailing pp-number in the argument can silently fuse with an adjacent body +/-/. into a single, wrong number—F(1e) through #define F(x) x+1 would become 1e+1, and G(3) through x.5 would become 3.5. This is a real token merge, not a spacing nicety: it changes what the re-lexer reads, so pp_tail_is_number gates a separating space wherever the two abut (outside a ## paste, whose intent is exactly to join).

Stringisation itself, in pp_emit_string, must honour §6.10.3.2p2 precisely: a \ or " is escaped *only* when it belongs to a character constant or string literal in the argument. A backslash that stands outside any literal passes through untouched. This is what lets the entry code assemble: <asm/nospec-branch.h>'s FILL_RETURN_BUFFER stringises a GAS .macro body through __stringify, and its \reg/\nr parameter references are bare backslashes, not literal contents; escaping them to \\reg would make the assembler reject the operand. The scan therefore tracks string- and char-literal state and only doubles a backslash inside one.

⟨ *Selfhost: pp substitute* 803 ⟩ ≡

```
static int is_id(int c) {
    return c == '_' || (c >= 'a' && c <= 'z') || (
   c >= 'A' && c <= 'Z');
}
static int is_idn(int c) { return is_id(c) || (c
   >= '0' && c <= '9'); }

/* Does the text end in a pp-number token?  Scan
   back over the trailing run of
   number/identifier/`.' bytes and ask whether it
   begins like a number (a digit,
   or `.' then a digit).  Used when splicing a
   macro argument into a body: a
   trailing pp-number abutting a body `+'/`-'/`.'
   would fuse into one number
   (`1e'+`+1', `3'+`.5'), so a separating space
   must go in. */
static int pp_tail_is_number(const char *s,
   unsigned long n) {
    if (!n) return 0;
    unsigned long j = n;
    while (j > 0) {
        unsigned char ch = (unsigned char)s[j -
   1];
        if (is_idn(ch) || ch == '.') j--; else
   break;
    }
    if (j >= n) return 0;
    unsigned char f = (unsigned char)s[j];
    if (f >= '0' && f <= '9') return 1;
    return (f == '.' && j + 1 < n && s[j + 1] >= '
   0' && s[j + 1] <= '9');
}

/* Would a token ending in `a' and one starting
   with `b', written adjacent, lex
   as a single longer pp-token?  Exactly the two-
   character C punctuators (plus a
   `.'`.' that could open `...'): `--', `<<',
   `->', `+=' and friends must not
   fuse across a macro-expansion boundary.
   Crucially `/'`/' is NOT here---after
   phase 3 it is two `/' tokens, never a comment
```

```
   ---so a computed include path
   like `asm/trace//x86_fpu.h' keeps its `//'
   rather than gaining a stray space. */
static int pp_pastes(char a, char b) {
    switch (a) {
    case '+': return b == '+' || b == '=';
    case '-': return b == '-' || b == '>' || b ==
   '=';
    case '<': return b == '<' || b == '=' || b ==
   ':' || b == '%';
    case '>': return b == '>' || b == '=';
    case '=': return b == '=';
    case '!': return b == '=';
    case '&': return b == '&' || b == '=';
    case '|': return b == '|' || b == '=';
    case '*': case '/': case '%': case '^': return
    b == '=';
    case '#': return b == '#';
    case ':': return b == '>';
    case '.': return b == '.';
    default:  return 0;
    }
}

static const char *pp_expand_arg(const char *text,
    unsigned long len, unsigned long *lo) {
    if (!len) { *lo = 0; return text; }
    unsigned long base_depth = kcc_pp_ctx_depth;
    unsigned long saved_floor = pp_drain_floor;
    pp_drain_floor = base_depth + 1;
    unsigned long saved_pending = pp_has_pending;
    pp_has_pending = 0;
    unsigned long cap = len * 4 + 256;
    char *out = (char *)kcc_arena_alloc(cap);
    unsigned long outlen = 0;
    const char *prev_end = 0;
    pp_ctx_push_expand(0, text, len);
    for (;;) {
        kcc_pp_next();
        if (kcc_pp_ctx_depth <= base_depth) break;
        if (kcc_tok_type == TOK_EOF) break;
        const char *ts = kcc_tok_start;
        unsigned long tl = kcc_tok_len;
        if (outlen + tl + 1 > cap) {
            unsigned long ncap = (outlen + tl + 1)
    * 2;
            char *nb = (char *)kcc_arena_alloc(
   ncap);
            kcc_memcpy(nb, out, outlen);
            out = nb; cap = ncap;
        }
        /* Separate tokens with a single space
   only where the source had
           whitespace (the byte before the token
   is whitespace, so `#'-stringify
           reproduces `a > b' and `.section' stays
    joined), or---across a bare
           macro-expansion boundary, where that
   byte lives in another buffer---
           only where the two tokens would
   otherwise fuse into one: two word
           tokens (`FOO'`bar') or a real two-
   character punctuator (`-'`-'->`--',
           `<'`<'->`<<') per pp_pastes.  A non-
   pasting punctuation boundary stays
           joined: `TRACE_INCLUDE_PATH' (->`asm/
   trace/') abutting `/system.h'
           keeps `asm/trace//x86_fpu.h', a valid
   computed #include path, not
           `asm/trace/ /x86_fpu.h' which no header
    search resolves. */
        if (outlen && ts != prev_end) {
            char before = ts[-1];
            char a = out[outlen - 1], b = ts[0];
            int fuse = (is_idn((unsigned char)a)
   && is_idn((unsigned char)b)) ||
                       pp_pastes(a, b);
            if (before == ' ' || before == 0x09 ||
    before == 0x0A ||
                before == 0x0D || fuse)
                out[outlen++] = ' ';
        }
        kcc_memcpy(out + outlen, ts, tl);
        outlen += tl;
        prev_end = ts + tl;
    }
    pp_drain_floor = saved_floor;
    pp_has_pending = saved_pending;
    *lo = outlen;
    return out;
}

/* Emit a "..." literal of src with whitespace
   collapsed and the " / \ of any
   embedded string or character literal escaped.
   */
static unsigned long pp_emit_string(const char *
   src, unsigned long len, char *dst) {
    char *d = dst;
    *d++ = '"';
    unsigned long i = 0;
    /* Each whitespace SEQUENCE (spaces, tabs, and
    the newlines of a multi-line
       argument such as a BUILD_BUG_ON condition)
   collapses to one space, and
       leading/trailing whitespace is dropped (\
   stdref{6.10.3.2}); an embedded
       newline copied verbatim would close the
   string prematurely. */
    while (i < len && (src[i] == ' ' || src[i] ==
   0x09 || src[i] == 0x0A || src[i] == 0x0D)) i++;
    int pending = 0;
    /* \stdref{6.10.3.2}p2 escapes a `\' or `"'
   only when it belongs to a
       character constant or string literal in the
    argument; a bare backslash
       outside any literal --- e.g. the `\reg'/`\
   nr' parameter references in a
       GAS `.macro' body stringised through `
   __stringify' (nospec-branch.h's
       FILL_RETURN_BUFFER) --- passes through
   verbatim.  Escaping it too would
       emit `\\reg', which the assembler rejects
   as an invalid operand. */
    int in_str = 0, in_chr = 0;
    for (; i < len; i++) {
        char c = src[i];
        if (!in_str && !in_chr && (c == ' ' || c
   == 0x09 || c == 0x0A || c == 0x0D)) { pending =
    1; continue; }
        if (pending) { *d++ = ' '; pending = 0; }
        if (in_str || in_chr) {
            if (c == 0x5C) {
   /* escape sequence within a literal */
                *d++ = 0x5C; *d++ = 0x5C;
   /* the backslash itself is escaped */
                if (i + 1 < len) {
                    char n = src[++i];
                    if (n == '"' || n == 0x5C) *d
   ++ = 0x5C;
                    *d++ = n;
                }
                continue;
            }
            if (c == '"') { *d++ = 0x5C; *d++ = '"
```

```
    '; if (in_str) in_str = 0; continue; }
             if (c == '\'') { *d++ = c; if (in_chr)
     in_chr = 0; continue; }
             *d++ = c;
             continue;
         }
         if (c == '"') { *d++ = 0x5C; *d++ = '"';
    in_str = 1; continue; }   /* opening delimiter
    */
         if (c == '\'') { *d++ = c; in_chr = 1;
    continue; }
         *d++ = c;
    /* bare backslash / other: verbatim */
     }
     *d++ = '"';
     return (unsigned long)(d - dst);
}

const char *pp_substitute(const char *body,
    unsigned long blen, struct PPParam *params,
                              unsigned long np, void *
    aa, void *raa, unsigned long ac, unsigned long
    *lo) {
     struct PPArg *args = (struct PPArg *)aa;
     struct PPArg *rawargs = (struct PPArg *)raa;
     (void)ac;
     /* Set once a `##' operator was just emitted,
    so the next parameter is its
         right operand and must substitute its RAW
    argument (6.10.3.1). */
     int after_paste = 0;
     /* The output buffer must grow: a macro like
    the kernel's `pr_alert(fmt,
         data_race(x), data_race(y), ...)`
    substitutes several arguments that are
         themselves large _Generic expansions,
    easily exceeding any fixed size (a
         former 4 KB buffer truncated mid-string ->
    "unterminated string literal").
         `guard` bounds the most any single
    iteration can emit---a string literal
         spanning the rest of the body, or a
    stringised/pasted argument (up to
         twice its length)---so one check per
    iteration keeps `out` big enough. */
     unsigned long maxarg = 0;
     for (unsigned long k = 0; k < np; k++)
         if (args[k].len > maxarg) maxarg = args[k
    ].len;
     unsigned long guard = blen + 2 * maxarg + 16;
     unsigned long cap = guard + 256;
     char *out = (char *)kcc_arena_alloc(cap);
     unsigned long op = 0;
     unsigned long i = 0;
     while (i < blen) {
         int prev_paste = after_paste; after_paste
    = 0;
         if (op + guard > cap) {
             unsigned long nc = cap * 2;
             while (nc < op + guard) nc *= 2;
             char *nb = (char *)kcc_arena_alloc(nc)
    ;
             kcc_memcpy(nb, out, op);
             out = nb; cap = nc;
         }
         char c = body[i];
         if (c == '"' || c == '\'') {
             /* A string literal or character
    constant is a single token: its
                contents are never subject to
    parameter replacement (\stdref{6.10.3.1}).
                Copy it through verbatim, honouring
     backslash escapes, so that a
                parameter name that happens to
    appear inside quotes---e.g. `x` in
                `"value x here"`, or `%[ptr]` in an
     inline-asm template---survives. */
             char q = c;
             out[op++] = body[i++];
             while (i < blen) {
                 char d = body[i];
                 out[op++] = d; i++;
                 if (d == '\\' && i < blen) { out[
    op++] = body[i++]; continue; }
                 if (d == q) break;
             }
             continue;
         }
         if (c == '#') {
             if (i + 1 < blen && body[i + 1] == '#'
    ) {
                 /* GNU comma elision `, ##
    __VA_ARGS__`: when the right operand of
                    ## is a parameter whose
    argument is empty, the ## deletes it and
                    a preceding comma (used
    pervasively via the kernel's
                    `static_assert(expr, ...)`
    macro). */
                 unsigned long pj = i + 2;
                 while (pj < blen && (body[pj] == '
     ' || body[pj] == 0x09)) pj++;
                 unsigned long ps = pj;
                 while (pj < blen && is_idn((
    unsigned char)body[pj])) pj++;
                 unsigned long plen = pj - ps;
                 long ppi = -1;
                 for (unsigned long k = 0; k < np;
    k++)
                     if (params[k].len == plen &&
    kcc_memcmp(body + ps, params[k].name, plen) ==
    0) { ppi = (long)k; break; }
                 if (ppi >= 0 && rawargs[ppi].len
    == 0) {
                     while (op && (out[op - 1] == '
     ' || out[op - 1] == 0x09)) op--;
                     if (op && out[op - 1] == ',')
    op--;     /* drop the dangling comma */
                     i = pj;
                  /* consume ## and the empty operand
    */
                     continue;
                 }
                 /* ## paste: trim emitted
    whitespace, skip operator + whitespace.
                    Mark that a paste just happened
     so the next parameter, its
                    right operand, substitutes its
    raw (unexpanded) argument. */
                 while (op && (out[op - 1] == ' '
    || out[op - 1] == 0x09)) op--;
                 i += 2;
                 while (i < blen && (body[i] == ' '
     || body[i] == 0x09)) i++;
                 after_paste = 1;
                 continue;
             }
             /* # stringise */
             unsigned long j = i + 1;
             while (j < blen && (body[j] == ' ' ||
    body[j] == 0x09)) j++;
             unsigned long s = j;
             while (j < blen && is_idn((unsigned
    char)body[j])) j++;
             unsigned long idlen = j - s;
```

```
            long pi = -1;
            for (unsigned long k = 0; k < np; k++)
                if (params[k].len == idlen &&
kcc_memcmp(body + s, params[k].name, idlen) ==
0) { pi = (long)k; break; }
            if (pi >= 0) {
                /* # stringises the RAW argument,
before prescan (6.10.3.2). */
                op += pp_emit_string(rawargs[pi].
start, rawargs[pi].len, out + op);
                i = j;
            } else {
                out[op++] = '#';
                for (unsigned long k = s; k < j; k
++) out[op++] = body[k];
                i = j;
            }
            continue;
        }
        if (is_id((unsigned char)c)) {
            unsigned long s = i;
            i++;
            while (i < blen && is_idn((unsigned
char)body[i])) i++;
            unsigned long idlen = i - s;
            long pi = -1;
            for (unsigned long k = 0; k < np; k++)
                if (params[k].len == idlen &&
kcc_memcmp(body + s, params[k].name, idlen) ==
0) { pi = (long)k; break; }
            if (pi >= 0) {
                /* A parameter that is an operand
of `##' --- preceded by a paste
                   (prev_paste) or immediately
followed by one --- substitutes its
                   raw argument; otherwise the
prescan-expanded one (6.10.3.1). */
                int use_raw = prev_paste;
                if (!use_raw) {
                    unsigned long pk = i;
                    while (pk < blen && (body[pk]
== ' ' || body[pk] == 0x09)) pk++;
                    if (pk + 1 < blen && body[pk]
== '#' && body[pk + 1] == '#') use_raw = 1;
                }
                const char *as = args[pi].start;
                unsigned long al = args[pi].len;
                if (use_raw) {
                    /* The raw span keeps the call
's surrounding whitespace (`, 3'
                       -> " 3"); a paste
juxtaposes tokens, so trim it --- the
                       prescan-expanded span is
already trimmed. */
                    as = rawargs[pi].start; al =
rawargs[pi].len;
                    while (al && (*as == ' ' || *
as == 0x09 || *as == 0x0A || *as == 0x0D)) { as
++; al--; }
                    while (al && (as[al - 1] == '
' || as[al - 1] == 0x09 ||
                                  as[al - 1] == 0
x0A || as[al - 1] == 0x0D)) al--;
                }
                kcc_memcpy(out + op, as, al);
                op += al;
                /* Keep the argument's trailing pp
-number from pasting with an
                   adjacent body `+'/`-'/`.' into
one number---`F(1e)' through
                   `x+1' must stay `1e +1', not
   lex as `1e+1'.  Only when they
                   abut (no body separator) and
   this is not a `##' operand, whose
                   whole intent is to paste. */
                if (!use_raw && al && i < blen &&
                    (body[i] == '+' || body[i] ==
   '-' || body[i] == '.') &&
                    pp_tail_is_number(as, al))
                    out[op++] = ' ';
            } else {
                kcc_memcpy(out + op, body + s,
   idlen);
                op += idlen;
            }
            continue;
        }
        out[op++] = c;
        i++;
    }
    *lo = op;
    return out;
}
```

□

803 · used in 818a

73.8. Predefined Macros and Command-Line Defines

pp_predefine seeds the macro table, for every translation unit, with the object-like built-ins a hosted C compiler is expected to define—the __STDC__ family, __LP64__, and the dynamically substituted __LINE__. It also advertises __SIZEOF_INT128__ as 16, the way gcc and clang do on 64-bit targets, because kcc now supports the __int128 type—the Linux kernel gates its u128/__u128 typedefs on that macro, so without it <asm/cmpxchg_64.h> would not even see the type. It publishes the __BYTE_ORDER__/__ORDER_*_ENDIAN__ family (both targets little-endian) so endian-detecting code compiles instead of tripping its #error fallback—all three order names defined together, since an undefined __BYTE_ORDER__ is 0 in a #if and would match a likewise-undefined __ORDER_*. Under --target=x86_64-linux it adds __linux__ and __x86_64__ so headers pick their glibc bindings, and—so that gcc-oriented build systems (Kbuild's scripts/cc-version.sh) recognise kcc as a supported compiler, exactly as clang and icc do—the GCC identity macros __GNUC__ / __GNUC_MINOR__ / __GNUC_PATCHLEVEL__. For an assembly input—a .S/.s file, flagged by kcc_lex_asm—it also predefines __ASSEMBLER__, the language macro every C compiler sets when preprocessing assembler (distinct from the kernel's hand-passed -D__ASSEMBLY__); headers such as <linux/elfnote.h> include their C-only companions (here <uapi/linux/elf.h> with its Elf32_Addr typedefs) only when it is *absent*, so predefining it keeps those typedefs out of the assembler stream. Finally it replays any -D command-line macros on top.

⟨ *Selfhost: pp predefined* 806 ⟩ ≡

```
extern unsigned long kcc_lex_asm;           /*
   set for a .S/.s input (assembler-with-cpp) */
```

```
static void pp_def_simple(const char *name,
   unsigned long nl, const char *body, unsigned
   long bl) {
    struct PPMacro *m = (struct PPMacro *)
   kcc_arena_alloc(sizeof(struct PPMacro));
    m->name = name; m->namelen = nl; m->flags =
   PP_FLAG_BUILTIN; m->nparams = 0;
    m->body = body; m->bodylen = bl; m->params =
   0;
    kcc_ht_put(kcc_pp_macros, name, nl, m);
}

static void pp_predefine(void) {
    pp_def_simple("__STDC__", 8, "1", 1);
    pp_def_simple("__STDC_VERSION__", 16, "201710L
   ", 7);
    pp_def_simple("__STDC_HOSTED__", 15, "1", 1);
    pp_def_simple("__STDC_NO_ATOMICS__", 19, "1",
   1);
    pp_def_simple("__STDC_NO_THREADS__", 19, "1",
   1);
    pp_def_simple("__STDC_NO_COMPLEX__", 19, "1",
   1);
    pp_def_simple("__STDC_NO_VLA__", 15, "1", 1);
    pp_def_simple("__LP64__", 8, "1", 1);
    pp_def_simple("_LP64", 5, "1", 1);
    pp_def_simple("__LINE__", 8, "1", 1);
    pp_def_simple("__FILE__", 8, "1", 1);   /*
   substituted dynamically in kcc_pp_try_expand */
    pp_def_simple("__BASE_FILE__", 13, "1", 1); /*
    approximated as __FILE__ (current path) */
    pp_def_simple("__COUNTER__", 11, "1", 1);
    /* kcc supports the GNU 128-bit integer type
   (16 bytes), so advertise it the
       way gcc and clang do on 64-bit targets.
   The Linux kernel gates its
       u128/__u128 typedefs on this macro (include
   /uapi/linux/types.h). */
    pp_def_simple("__SIZEOF_INT128__", 17, "16",
   2);
    /* Byte-order macros, exactly as gcc/clang
   publish them.  Real code (here
       doomgeneric's i_swap.h) selects its swap
   routines with
       `#if __BYTE_ORDER__ ==
   __ORDER_LITTLE_ENDIAN__' and `#error's if none
   of
       the three names resolve; an undefined `
   __BYTE_ORDER__' is 0 in a `#if',
       which would compare equal to a likewise-
   undefined `__ORDER_*' and pick the
       wrong branch, so all four must be defined.
    Both kcc targets---Apple
       ARM64 and Linux x86-64---are little-endian.
    */
    pp_def_simple("__ORDER_LITTLE_ENDIAN__", 23, "
   1234", 4);
    pp_def_simple("__ORDER_BIG_ENDIAN__", 20, "
   4321", 4);
    pp_def_simple("__ORDER_PDP_ENDIAN__", 20, "
   3412", 4);
    pp_def_simple("__BYTE_ORDER__", 14, "1234", 4)
   ;
    pp_def_simple("__LITTLE_ENDIAN__", 17, "1", 1)
   ;
    if (kcc_x86_os) {                    /* Linux
   /x86-64 target: let headers pick glibc bindings
    */
        pp_def_simple("__linux__", 9, "1", 1);
        pp_def_simple("__x86_64__", 10, "1", 1);
        /* Claim GCC compatibility so gcc-oriented
    build systems (Kbuild's
           scripts/cc-version.sh) recognise kcc,
   exactly as clang and icc do. */
        pp_def_simple("__GNUC__", 8, "11", 2);
        pp_def_simple("__GNUC_MINOR__", 14, "2",
   1);
        pp_def_simple("__GNUC_PATCHLEVEL__", 19, "
   0", 1);
    }
    /* The compiler predefines __ASSEMBLER__ when
   the input language is assembly
       (a .S/.s file), exactly as gcc and clang do
   ; it is distinct from the
       kernel's own -D__ASSEMBLY__.  Headers gate
   their C-only halves on it:
       <linux/elfnote.h> pulls in <uapi/linux/elf.
   h> --- and its Elf32_Addr and
       kin typedefs --- only in the `#else' of `#
   ifdef __ASSEMBLER__', so without
       this a stringised .S (vdso-note.S) would
   drag C typedefs into the
       assembler and fail with `no such
   instruction: typedef ...'. */
    if (kcc_lex_asm) pp_def_simple("__ASSEMBLER__"
   , 13, "1", 1);
    kcc_pp_apply_cmdline_defines();
}

void kcc_pp_add_cmdline_define(const char *payload
   ) {
    if (pp_cmdline_ndefs < 64) pp_cmdline_defs[
   pp_cmdline_ndefs++] = payload;
}

void kcc_pp_apply_cmdline_defines(void) {
    for (unsigned long i = 0; i < pp_cmdline_ndefs
   ; i++) {
        const char *p = pp_cmdline_defs[i];
        unsigned long nl = 0;
        while (p[nl] && p[nl] != '=') nl++;
        if (p[nl] == '=') {
            const char *val = p + nl + 1;
            pp_def_simple(p, nl, val, kcc_strlen(
   val));
        } else {
            pp_def_simple(p, nl, "1", 1);
        }
    }
}
```

□

806 · used in 818a

73.9. Conditionals and the #if Evaluator

⟨ *Selfhost: pp conditional* 807 ⟩ ≡

```
static void pp_cond_push(unsigned long state) {
    if (pp_cond_depth >= PP_MAX_COND) return;
    pp_cond_stack[pp_cond_depth++] = (unsigned
   char)state;
}

void kcc_pp_do_ifdef(int negate) {
    kcc_lex_next();
    /* Accept a keyword-class token as the name
   too (phase 4 has no keywords):
       `#ifndef __inline__` in SDL's begin_code.h
   names a would-be macro whose
```

```
       spelling the lexer tagged as a keyword; the
    lookup keys on the text. */
    unsigned long tt = kcc_tok_type;
    if (tt != TOK_IDENT && (tt - 10) >= 50) {
        kcc_diag_error(pp_dir_line, 1,
   m_ifdef_name);
        pp_cond_push(PP_COND_SKIP);
        pp_skip_line();
        return;
    }
    void *m = kcc_ht_get(kcc_pp_macros,
   kcc_tok_start, kcc_tok_len);
    int cond = (m != 0);
    if (negate) cond = !cond;
    pp_cond_push(cond ? PP_COND_ACTIVE :
   PP_COND_SKIP);
    pp_skip_line();
}

/* Rewrite `defined X`/`defined(X)` → 1/0 (operand
    looked up, not expanded),
 * copying every other token verbatim. */
static const char *pp_protect_defined(const char *
   src, unsigned long len, unsigned long *lo) {
    const char *saved_cur = kcc_lex_cursor, *
   saved_end = kcc_lex_end;
    kcc_lex_cursor = src;
    kcc_lex_end = src + len;
    char *out = (char *)kcc_arena_alloc(2 * len +
   64);
    unsigned long outlen = 0;
    kcc_lex_next();
    for (;;) {
        if (kcc_tok_type == TOK_EOF) break;
        if (kcc_tok_type == TOK_IDENT &&
   kcc_tok_len == 7 &&
            kcc_memcmp(kcc_tok_start, "defined",
   7) == 0) {
            kcc_lex_next();
            int paren = (kcc_tok_type ==
   TOK_LPAREN);
            if (paren) kcc_lex_next();
            int bit = (kcc_ht_get(kcc_pp_macros,
   kcc_tok_start, kcc_tok_len) != 0);
            kcc_lex_next();
   /* past name */
            if (paren) kcc_lex_next();
   /* past ')' */
            if (outlen) out[outlen++] = ' ';
            out[outlen++] = '0' + bit;
            continue;
        }
        if (outlen) out[outlen++] = ' ';
        kcc_memcpy(out + outlen, kcc_tok_start,
   kcc_tok_len);
        outlen += kcc_tok_len;
        kcc_lex_next();
    }
    kcc_lex_cursor = saved_cur;
    kcc_lex_end = saved_end;
    *lo = outlen;
    return out;
}

static const char *pp_prepare_cond(const char *src
   , unsigned long len, unsigned long *lo) {
    unsigned long la;
    const char *a = pp_protect_defined(src, len, &
   la);
    if (!la) { *lo = la; return a; }
    unsigned long saved = pp_cond_depth;
    pp_cond_depth = 0;
    unsigned long lb;
    const char *b = pp_expand_arg(a, la, &lb);
    pp_cond_depth = saved;
    *lo = lb;
    return b;
}

/* Evaluate the expanded controlling text; returns
    the integer result. */
static long pp_eval_line(const char *src, unsigned
    long len) {
    unsigned long el;
    const char *e = pp_prepare_cond(src, len, &el)
   ;
    const char *saved_cur = kcc_lex_cursor, *
   saved_end = kcc_lex_end;
    kcc_lex_cursor = e;
    kcc_lex_end = e + el;
    kcc_lex_next();
    long r = kcc_pp_eval_expr();
    kcc_lex_cursor = saved_cur;
    kcc_lex_end = saved_end;
    return r;
}

void kcc_pp_do_if(void) {
    unsigned long len;
    const char *src = pp_rest_of_line(&len);
    long r = pp_eval_line(src, len);
    pp_cond_push(r ? PP_COND_ACTIVE : PP_COND_SKIP
   );
}

void kcc_pp_do_line(void) {
    unsigned long phys = kcc_lex_line;
    unsigned long len;
    const char *src = pp_rest_of_line(&len);
    long n = pp_eval_line(src, len);
    kcc_line_delta = n - (long)phys - 1;
}

static void pp_do_elif(void) {
    if (pp_cond_depth == 0) {
        kcc_diag_error(pp_dir_line, 1,
   m_stray_elif);
        pp_skip_line();
        return;
    }
    unsigned char st = pp_cond_stack[pp_cond_depth
    - 1];
    if (st == PP_COND_ACTIVE) {
        pp_cond_stack[pp_cond_depth - 1] =
   PP_COND_DONE;
        pp_skip_line();
        return;
    }
    if (st == PP_COND_DONE) { pp_skip_line();
   return; }
    unsigned long len;
    const char *src = pp_rest_of_line(&len);
    long r = pp_eval_line(src, len);
    pp_cond_stack[pp_cond_depth - 1] = r ?
   PP_COND_ACTIVE : PP_COND_SKIP;
}

static void pp_do_else(void) {
    if (pp_cond_depth == 0) {
        kcc_diag_error(pp_dir_line, 1,
   m_stray_else);
        pp_skip_line();
        return;
```

```
    }
    unsigned char st = pp_cond_stack[pp_cond_depth
    - 1];
    if (st == PP_COND_ACTIVE) pp_cond_stack[
   pp_cond_depth - 1] = PP_COND_DONE;
    else if (st == PP_COND_SKIP) pp_cond_stack[
   pp_cond_depth - 1] = PP_COND_ACTIVE;
    pp_skip_line();
}

static void pp_do_endif(void) {
    if (pp_cond_depth == 0) {
        kcc_diag_error(pp_dir_line, 1,
   m_stray_endif);
        pp_skip_line();
        return;
    }
    pp_cond_depth--;
    pp_skip_line();
}

static void pp_skip_directive(void) {
    pp_dir_line = kcc_tok_line;
    kcc_lex_next();
    unsigned long tt = kcc_tok_type;
    if (tt != TOK_IDENT && (tt < 10 || tt > 53)) {
    pp_skip_line(); return; }
    const char *n = kcc_tok_start;
    unsigned long l = kcc_tok_len;
    if (dname(n, l, "if", 2) || dname(n, l, "ifdef
   ", 5) || dname(n, l, "ifndef", 6)) {
        pp_cond_push(PP_COND_DONE);
        pp_skip_line();
    } else if (dname(n, l, "endif", 5)) {
        pp_do_endif();
    } else if (dname(n, l, "elif", 4)) {
        pp_do_elif();
    } else if (dname(n, l, "else", 4)) {
        pp_do_else();
    } else {
        pp_skip_line();
    }
}
```

□

807 · used in 818a

⟨ *Selfhost: pp eval* 809 ⟩ ≡

```
static long pp_eval_ternary(void);
static long pp_eval_primary(void);

long kcc_pp_eval_expr(void) { return
   pp_eval_ternary(); }

#define BINLEVEL(name, sub, body) \
static long name(void) { long a = sub(); for (;;)
   { unsigned long op = kcc_tok_type; body } }

static long pp_eval_unary(void);
static long pp_eval_mul(void) {
    long a = pp_eval_unary();
    for (;;) {
        unsigned long op = kcc_tok_type;
        if (op != TOK_STAR && op != TOK_SLASH &&
   op != TOK_PERCENT) return a;
        kcc_lex_next();
        long b = pp_eval_unary();
        if (op == TOK_STAR) a = a * b;
        else if (op == TOK_SLASH) a = b ? a / b :
   0;
        else a = b ? a % b : 0;
    }
}
static long pp_eval_add(void) {
    long a = pp_eval_mul();
    for (;;) {
        unsigned long op = kcc_tok_type;
        if (op != TOK_PLUS && op != TOK_MINUS)
   return a;
        kcc_lex_next();
        long b = pp_eval_mul();
        a = (op == TOK_PLUS) ? a + b : a - b;
    }
}
static long pp_eval_shift(void) {
    long a = pp_eval_add();
    for (;;) {
        unsigned long op = kcc_tok_type;
        if (op != TOK_SHL && op != TOK_SHR) return
    a;
        kcc_lex_next();
        long b = pp_eval_add();
        a = (op == TOK_SHL) ? (long)((unsigned
   long)a << b) : (long)((unsigned long)a >> b);
    }
}
static long pp_eval_rel(void) {
    long a = pp_eval_shift();
    for (;;) {
        unsigned long op = kcc_tok_type;
        if (op != TOK_LT && op != TOK_GT && op !=
   TOK_LE && op != TOK_GE) return a;
        kcc_lex_next();
        long b = pp_eval_shift();
        if (op == TOK_LT) a = a < b;
        else if (op == TOK_GT) a = a > b;
        else if (op == TOK_LE) a = a <= b;
        else a = a >= b;
    }
}
static long pp_eval_eq(void) {
    long a = pp_eval_rel();
    for (;;) {
        unsigned long op = kcc_tok_type;
        if (op != TOK_EQ && op != TOK_NE) return a
   ;
        kcc_lex_next();
        long b = pp_eval_rel();
        a = (op == TOK_EQ) ? (a == b) : (a != b);
    }
}
static long pp_eval_band(void) {
    long a = pp_eval_eq();
    while (kcc_tok_type == TOK_AMP) { kcc_lex_next
   (); a = a & pp_eval_eq(); }
    return a;
}
static long pp_eval_bxor(void) {
    long a = pp_eval_band();
    while (kcc_tok_type == TOK_CARET) {
   kcc_lex_next(); a = a ^ pp_eval_band(); }
    return a;
}
static long pp_eval_bor(void) {
    long a = pp_eval_bxor();
    while (kcc_tok_type == TOK_PIPE) {
   kcc_lex_next(); a = a | pp_eval_bxor(); }
    return a;
}
static long pp_eval_land(void) {
    long a = pp_eval_bor();
```

```
    while (kcc_tok_type == TOK_AND) { kcc_lex_next
   (); long b = pp_eval_bor(); a = (a && b); }
    return a;
}
static long pp_eval_lor(void) {
    long a = pp_eval_land();
    while (kcc_tok_type == TOK_OR) { kcc_lex_next
   (); long b = pp_eval_land(); a = (a || b); }
    return a;
}
static long pp_eval_ternary(void) {
    long c = pp_eval_lor();
    if (kcc_tok_type != TOK_QUESTION) return c;
    kcc_lex_next();
    long t = kcc_pp_eval_expr();
    if (kcc_tok_type == TOK_COLON) kcc_lex_next();
    long e = pp_eval_ternary();
    return c ? t : e;
}

static long pp_eval_unary(void) {
    unsigned long op = kcc_tok_type;
    if (op == TOK_BANG) { kcc_lex_next(); return !
   pp_eval_unary(); }
    if (op == TOK_TILDE) { kcc_lex_next(); return
   ~pp_eval_unary(); }
    if (op == TOK_MINUS) { kcc_lex_next(); return
   -pp_eval_unary(); }
    if (op == TOK_PLUS) { kcc_lex_next(); return
   pp_eval_unary(); }
    return pp_eval_primary();
}

/* evaluate a macro's body as an integer (macro
   chains + expression bodies) */
static long pp_eval_macro(struct PPMacro *m) {
    if (pp_eval_depth >= 64) return 0;
    pp_eval_depth++;
    const char *sc = kcc_lex_cursor, *se =
   kcc_lex_end;
    kcc_lex_cursor = m->body;
    kcc_lex_end = m->body + m->bodylen;
    kcc_lex_next();
    long r = kcc_pp_eval_expr();
    kcc_lex_cursor = sc;
    kcc_lex_end = se;
    pp_eval_depth--;
    return r;
}

static long pp_eval_primary(void) {
    unsigned long tt = kcc_tok_type;
    if (tt == TOK_NUMBER) {
        long v = (long)kcc_pp_atoi(kcc_tok_start,
   kcc_tok_len);
        kcc_lex_next();
        return v;
    }
    if (tt == TOK_LPAREN) {
        kcc_lex_next();
        long v = kcc_pp_eval_expr();
        if (kcc_tok_type == TOK_RPAREN)
   kcc_lex_next();
        return v;
    }
    if (tt == TOK_IDENT) {
        if (kcc_tok_len == 7 && kcc_memcmp(
   kcc_tok_start, "defined", 7) == 0) {
            kcc_lex_next();
            int paren = (kcc_tok_type ==
   TOK_LPAREN);
            if (paren) kcc_lex_next();
            if (kcc_tok_type != TOK_IDENT) {
   kcc_lex_next(); return 0; }
            int bit = (kcc_ht_get(kcc_pp_macros,
   kcc_tok_start, kcc_tok_len) != 0);
            kcc_lex_next();
            if (paren && kcc_tok_type ==
   TOK_RPAREN) kcc_lex_next();
            return bit;
        }
        struct PPMacro *m = (struct PPMacro *)
   kcc_ht_get(kcc_pp_macros, kcc_tok_start,
   kcc_tok_len);
        if (!m) { kcc_lex_next(); return 0; }
        long v = pp_eval_macro(m);
        kcc_lex_next();
        return v;
    }
    return 0;
}

unsigned long kcc_pp_atoi(const char *p, unsigned
   long len) {
    unsigned long acc = 0;
    if (len >= 2 && p[0] == '0' && ((p[1] | 0x20)
   == 'x')) {
        for (unsigned long i = 2; i < len; i++) {
            char c = p[i];
            unsigned long d;
            if (c >= '0' && c <= '9') d = c - '0';
            else { char lc = c | 0x20; if (lc < 'a
   ' || lc > 'f') break; d = lc - 'a' + 10; }
            acc = (acc << 4) + d;
        }
        return acc;
    }
    if (len >= 1 && p[0] == '0') {
        for (unsigned long i = 1; i < len; i++) {
            char c = p[i];
            if (c < '0' || c > '7') break;
            acc = (acc << 3) + (c - '0');
        }
        return acc;
    }
    for (unsigned long i = 0; i < len; i++) {
        char c = p[i];
        if (c < '0' || c > '9') break;
        acc = acc * 10 + (c - '0');
    }
    return acc;
}
□
```

809 · used in 818a

73.10. File Inclusion and Built-in Headers

#include resolves a header by trying, in order, the including file's own directory, the literal path, each -I directory, and finally the compiler's own include/ directory; only if no file is found there is the header served from memory as a built-in. Two orderings are deliberate. Disk before built-in: the compiler-resident headers are a freestanding fallback so a program can compile with no search path at all, but a hosted build that supplies a fuller shim through -I—the platform's real errno values or struct flock, say—must get that shim rather than the leaner built-in. And the de-

fault `include/` directory is kept in a separate slot searched *after* every `-I`, not pushed onto the `-I` array where—being registered at startup—it would sit ahead of the user's `-I` and shadow it. Alongside resolution, the preprocessor keeps the *dependency list* for `-MD`: every real include file it opens is recorded (built-ins have no path and are skipped), and `kcc_dep_write` later emits the Makefile fragment `obj: src headers…` plus trailing phony `header:` targets—the form Kbuild's `fixdep` reads, and what `make` needs so a deleted header does not wedge the build. The list is written only when the driver has set `kcc_dep_out` from a `-MD`/`-Wp,-MMD` option.

`kcc_pp_force_include` implements gcc's `-include FILE`: it resolves and pushes `FILE` exactly as `#include "FILE"` would, but with the path handed in by the driver rather than read from a directive. The driver calls it once per `-include` file after `kcc_pp_init` and before parsing (in reverse order, so the pushed contexts read back in command-line order), which places each file's macros and typedefs in scope ahead of the primary source. Kbuild depends on this: every kernel object is compiled with `-include ./include/linux/kconfig.h` (which pulls `autoconf.h` and thus every `CONFIG_*` macro) and `-include ./include/linux/compiler_types.h`, so without honouring `-include` a `CONFIG_64BIT`-gated type such as `atomic64_t` would never be declared.

A `#include` whose first token after the directive is neither a string literal nor `<` is a *computed* include (C17 §6.10.2p4): the remaining tokens are macro-expanded and the result must then read as one of the two header-name forms. The kernel's tracepoint generator is the canonical user: `include/trace/define_trace.h` ends with `#include TRACE_INCLUDE(TRACE_INCLUDE_FILE)`, and with no `TRACE_INCLUDE_PATH` set `TRACE_INCLUDE(system)` expands to `<trace/events/system.h>` (otherwise to a `__stringify`'d `"..."`). `kcc_pp_do_include` handles this by expanding the whole rest of the directive line through `pp_expand_arg`—the same engine `#if` uses—and then reading a `"…"` or `<…>` form out of the expansion. For the angle form the header name is the concatenation of the token spellings between `<` and `>` with intervening whitespace stripped, because expansion inserts spaces at every buffer boundary (the path `trace / events / system . h` arrives as separate tokens) yet none of that spacing belongs to the file name. Without this, every translation unit that defines `CREATE_TRACE_POINTS` stalls on the first `define_trace.h` computed include.

⟨ *Selfhost: pp include* 811 ⟩ ≡

```
static int pp_file_exists(const char *path) {
    long fd = kcc_sys_open(path, 0, 0);
    if (fd < 0) return 0;    /* open failed (macOS
    returns -1; Linux returns -errno) */
    kcc_sys_close(fd);
    return 1;
}

static const char *pp_resolve_include(const char *
   name, int system) {
    unsigned long namelen = kcc_strlen(name);
     /* A quote include (`"..."') is looked up
    first in the directory of the file
        that holds the directive, then relative to
    the working directory, before
        the -I path (\stdref{6.10.2p3}).  An angle
    include (`<...>') searches ONLY
        the -I / system directories (\stdref{6.10.2
    p2})---never the working
        directory: a kernel tree has a top-level `
    crypto/hash.h' (a small internal
        shim) that must not shadow the public `
    include/crypto/hash.h' that
        `#include <crypto/hash.h>' means, or the
    wrong header quietly omits
        `struct crypto_istat_hash' and every
    reference to it fails to type-check. */
    if (!system) {
        const char *sp = kcc_src_path;
        if (sp) {
            unsigned long dirlen = 0;
            for (unsigned long i = 0; sp[i]; i++)
    if (sp[i] == '/') dirlen = i + 1;
            if (dirlen) {
                char *cand = (char *)
    kcc_arena_alloc(dirlen + namelen + 1);
                kcc_memcpy(cand, sp, dirlen);
                kcc_memcpy(cand + dirlen, name,
    namelen);
                cand[dirlen + namelen] = 0;
                if (pp_file_exists(cand)) return
    cand;
            }
        }
        if (pp_file_exists(name)) return name;
    }
    for (unsigned long i = 0; i < pp_inc_ndirs; i
    ++) {
        const char *dir = pp_inc_dirs[i];
        unsigned long dl = kcc_strlen(dir);
        char *cand = (char *)kcc_arena_alloc(dl +
    namelen + 1);
        kcc_memcpy(cand, dir, dl);
        kcc_memcpy(cand + dl, name, namelen);
        cand[dl + namelen] = 0;
        if (pp_file_exists(cand)) return cand;
    }
     /* The compiler's own include/ directory is
    consulted only AFTER every -I, so
        a hosted build's -I shim (a fuller <fcntl.h
    >/<errno.h>) is never shadowed
        by the leaner headers shipped beside the
    binary. */
    if (pp_default_inc) {
        unsigned long dl = kcc_strlen(
    pp_default_inc);
        char *cand = (char *)kcc_arena_alloc(dl +
    namelen + 1);
        kcc_memcpy(cand, pp_default_inc, dl);
        kcc_memcpy(cand + dl, name, namelen);
        cand[dl + namelen] = 0;
        if (pp_file_exists(cand)) return cand;
    }
    return 0;
}

void kcc_pp_add_include_dir(const char *dir) {
    if (pp_inc_ndirs >= 64) return;
    unsigned long dl = kcc_strlen(dir);
    char *buf = (char *)kcc_arena_alloc(dl + 2);
    kcc_memcpy(buf, dir, dl);
    buf[dl] = '/';
```

```
    buf[dl + 1] = 0;
    pp_inc_dirs[pp_inc_ndirs++] = buf;
}

void kcc_pp_add_default_includes(const char *argv0
   ) {
    unsigned long prefixlen = 0;
    for (unsigned long i = 0; argv0[i]; i++) if (
   argv0[i] == '/') prefixlen = i + 1;
    char *buf = (char *)kcc_arena_alloc(prefixlen
   + 9);
    kcc_memcpy(buf, argv0, prefixlen);
    kcc_memcpy(buf + prefixlen, "include", 7);
    buf[prefixlen + 7] = '/';    /* trailing slash
    for a plain join */
    buf[prefixlen + 8] = 0;
    /* Kept out of the -I array so it is searched
   only after every -I (see
       pp_resolve_include), never ahead of a user'
   s -I shim. */
    pp_default_inc = buf;
}

void kcc_pp_do_include(void) {
    kcc_lex_next();
    unsigned long tt = kcc_tok_type;
    const char *nstart;
    unsigned long nlen;
    int system;
    if (tt == TOK_STRING_LIT) {
        system = 0;
        nstart = kcc_tok_start + 1;
        nlen = kcc_tok_len - 2;
    } else if (tt == TOK_LT) {
        system = 1;
        const char *s = kcc_lex_cursor;
        const char *e = kcc_lex_end;
        const char *p = s;
        while (p < e && *p != '>' && *p != 0x0A) p
   ++;
        if (p >= e || *p == 0x0A) { kcc_diag_error
   (pp_dir_line, 1, m_include); pp_skip_line();
   return; }
        nstart = s;
        nlen = (unsigned long)(p - s);
        kcc_lex_cursor = p + 1;
    } else {
        /* Computed include (C17 6.10.2p4): the
   directive's tokens form neither a
           `"q-char"' nor a `<h-char>' header name
   , so macro-expand the rest of
           the line and re-interpret the result.
   The kernel's tracepoint
           machinery leans on this---`#include
   TRACE_INCLUDE(TRACE_INCLUDE_FILE)'
           in `include/trace/define_trace.h'
   expands to
           `<trace/events/SYSTEM.h>' (or a `
   __stringify'd `"..."'').  We expand the
           whole remaining line (the first token
   was already lexed, so start from
           its beginning) and then parse a string
   or angle form out of the
           expansion; for the angle form the
   header name is the token spellings
           between `<' and `>' with intervening
   whitespace removed, since
           expansion separates the path's tokens
   (`trace' `/' `events' `/' ...)
           with spaces that are not part of the
   file name. */
        const char *ls = kcc_tok_start;
        const char *le = kcc_lex_end;
        const char *pe = ls;
        while (pe < le && *pe != 0x0A) pe++;
        unsigned long xlen;
        const char *x = pp_expand_arg(ls, (
unsigned long)(pe - ls), &xlen);
        unsigned long i = 0;
        while (i < xlen && (x[i] == ' ' || x[i] ==
 0x09)) i++;
        if (i < xlen && x[i] == '"') {
            i++;
            unsigned long j = i;
            while (j < xlen && x[j] != '"') j++;
            system = 0; nstart = x + i; nlen = j -
 i;
        } else if (i < xlen && x[i] == '<') {
            i++;
            char *hb = (char *)kcc_arena_alloc(
xlen + 1);
            unsigned long bl = 0;
            while (i < xlen && x[i] != '>') {
                if (x[i] != ' ' && x[i] != 0x09)
hb[bl++] = x[i];
                i++;
            }
            system = 1; nstart = hb; nlen = bl;
        } else {
            kcc_diag_error(pp_dir_line, 1,
m_include);
            pp_skip_line();
            return;
        }
    }
    char *path = (char *)kcc_arena_alloc(nlen + 1)
;
    kcc_memcpy(path, nstart, nlen);
    path[nlen] = 0;
    pp_skip_line();
    /* A header found on disk (via the same
directory, a literal path, or a -I
       directory) takes precedence over a compiler
-resident one: the built-in set
       is a freestanding FALLBACK, so a hosted
build that points -I at a fuller
       system shim gets that shim.  Without this
order, adding a built-in
       <errno.h>/<fcntl.h> would shadow a -I shim
carrying the platform's real
       errno values and `struct flock`, breaking a
 program (SQLite) that needs
       them.  A bare <name> with no -I match still
 falls through to the built-in,
       so a program that includes nothing but
standard headers compiles with no
       search path at all. */
    const char *rp = pp_resolve_include(path,
system);
    if (rp) {
        kcc_dep_add(rp);
/* record for -MD dependency output */
        pp_ctx_push(PP_CTX_INCLUDE);
        kcc_source_open(rp);
        kcc_source_encode();
        kcc_source_splice();
        kcc_lex_init(kcc_src_base, kcc_src_size);
        pp_prev_line = 0;
        return;
    }
    const char *hdr = pp_builtin_header(path);
    if (hdr) {
        unsigned long hlen = kcc_strlen(hdr);
        pp_ctx_push(PP_CTX_INCLUDE);
        kcc_src_base = hdr;
```

```
        kcc_src_size = hlen;
        kcc_src_path = path;
        kcc_lex_init(hdr, hlen);
        pp_prev_line = 0;
        return;
    }
    if (!system) kcc_diag_error(pp_dir_line, 1,
   m_notfound);
}

/* Force-include: process FILE as if `#include "
   FILE"` stood at the very top of
   the primary translation unit, for gcc's `-
   include FILE`.  The driver calls
   this once per -include file (in reverse command
   -line order) after
   kcc_pp_init and before parsing, so the pushed
   contexts are read back in
   forward order and their macros/typedefs are
   already in scope when the primary
   source is reached.  Kbuild leans on this to
   inject
   include/linux/kconfig.h (all the CONFIG_*
   macros via autoconf.h) and
   compiler_types.h before every kernel object. */
void kcc_pp_force_include(const char *path) {
    const char *hdr = pp_builtin_header(path);
    if (hdr) {
        unsigned long hlen = kcc_strlen(hdr);
        pp_ctx_push(PP_CTX_INCLUDE);
        kcc_src_base = hdr;
        kcc_src_size = hlen;
        kcc_src_path = path;
        kcc_lex_init(hdr, hlen);
        pp_prev_line = 0;
        return;
    }
    const char *rp = pp_resolve_include(path, 0);
    if (!rp) { kcc_diag_error(1, 1, m_notfound);
   return; }
    kcc_dep_add(rp);
    pp_ctx_push(PP_CTX_INCLUDE);
    kcc_source_open(rp);
    kcc_source_encode();
    kcc_source_splice();
    kcc_lex_init(kcc_src_base, kcc_src_size);
    pp_prev_line = 0;
}

/* Dependency tracking for `-MD`/`-Wp,-MMD,file`:
   record every real include file
   opened while preprocessing, then write a
   Makefile fragment `obj: src hdrs...`
   that Kbuild's fixdep consumes.  Built-in
   headers have no path and are skipped. */
const char *kcc_dep_out = 0;                    /*
   the .d output path, set by the driver */
static const char *kcc_dep_list[1024];
static unsigned long kcc_dep_n = 0;
void kcc_dep_reset(void) { kcc_dep_n = 0; }
void kcc_dep_add(const char *path) { if (kcc_dep_n
    < 1024) kcc_dep_list[kcc_dep_n++] = path; }
static void dep_puts(long fd, const char *s) {
   kcc_sys_write(fd, s, kcc_strlen(s)); }
void kcc_dep_write(const char *obj, const char *
   src) {
    if (!kcc_dep_out) return;
    /* O_WRONLY|O_CREAT|O_TRUNC = 0x241 on Linux,
   mode 0644; -MD is a Kbuild
       (Linux) feature so the Linux flag values
   apply. */
    long fd = kcc_sys_open(kcc_dep_out, 0x241,
   0644);
    if (fd < 0) return;
    dep_puts(fd, obj); dep_puts(fd, ": ");
   dep_puts(fd, src);
    for (unsigned long i = 0; i < kcc_dep_n; i++)
   { dep_puts(fd, " \\\n  "); dep_puts(fd,
   kcc_dep_list[i]); }
    dep_puts(fd, "\n\n");
    for (unsigned long i = 0; i < kcc_dep_n; i++)
   { dep_puts(fd, kcc_dep_list[i]); dep_puts(fd, "
   :\n"); }
    kcc_sys_close(fd);
}

void kcc_pp_do_error(void) {
    unsigned long len;
    const char *msg = pp_rest_of_line(&len);
    char *buf = (char *)kcc_arena_alloc(len + 1);
    kcc_memcpy(buf, msg, len);
    buf[len] = 0;
    kcc_diag_error(pp_dir_line, 1, buf);
}
□
```

811 · used in 818a

73.11. Phase 5 and 6: String-Literal Escapes and Concatenation

The `preprocessor.o` object also carries translation phases 5 and 6 (Chapters 18–19), which the parser calls: `kcc_strlit_process` converts escape sequences to their execution-character-set bytes, and `kcc_strlit_gather` concatenates the current string literal with every adjacent one (through macro expansion, via `kcc_pp_next`). Both hand back a `{buffer, length}` pair in the first two registers.

⟨ *Selfhost: pp strlit* 813 ⟩ ≡

```
struct StrResult kcc_strlit_process(const char *
   src, unsigned long len) {
    const char *p = src, *end = src + len;
    char *dst = (char *)kcc_arena_alloc(len + 1);
    char *d = dst;
    while (p < end) {
        char c = *p;
        if (c != 0x5C) { *d++ = c; p++; continue;
   }
        p++;
   /* skip backslash */
        if (p >= end) break;
   /* trailing backslash */
        char e = *p++;
        if (e == 'n') *d++ = 0x0A;
        else if (e == 't') *d++ = 0x09;
        else if (e == 0x5C) *d++ = 0x5C;
        else if (e == '"') *d++ = 0x22;
        else if (e == 0x27) *d++ = 0x27;
        else if (e == 'a') *d++ = 0x07;
        else if (e == 'b') *d++ = 0x08;
        else if (e == 'f') *d++ = 0x0C;
        else if (e == 'r') *d++ = 0x0D;
        else if (e == 'v') *d++ = 0x0B;
        else if (e == '?') *d++ = 0x3F;
        else if (e == 'x') {
            unsigned v = 0;
            while (p < end) {
```

```
                char h = *p;
                unsigned dg;
                if (h >= '0' && h <= '9') dg = h -
    '0';
                else { char lc = h | 0x20; if (lc
   < 'a' || lc > 'f') break; dg = lc - 'a' + 10; }
                v = (v << 4) + dg;
                p++;
            }
            *d++ = (char)(v & 0xFF);
        } else if (e == '0') {
            if (p < end && *p >= '0' && *p <= '7')
    {
                unsigned v = 0; int n = 2;
                while (p < end && n && *p >= '0'
   && *p <= '7') { v = (v << 3) + (*p - '0'); p++;
    n--; }
                *d++ = (char)(v & 0xFF);
            } else *d++ = 0x00;
        } else if (e >= '1' && e <= '7') {
            unsigned v = e - '0'; int n = 2;
            while (p < end && n && *p >= '0' && *p
    <= '7') { v = (v << 3) + (*p - '0'); p++; n--;
    }
            *d++ = (char)(v & 0xFF);
        } else {
            *d++ = e;
   /* unknown escape → literal */
        }
    }
    struct StrResult r;
    r.buf = dst;
    r.len = (unsigned long)(d - dst);
    return r;
}

struct StrResult kcc_strlit_gather(void) {
    unsigned long cap = 256, used = 0;
    char *buf = (char *)kcc_arena_alloc(256);
    for (;;) {
        struct StrResult pr = kcc_strlit_process(
   kcc_tok_start + 1, kcc_tok_len - 2);
        if (used + pr.len > cap) {
            unsigned long nc = cap;
            while (nc < used + pr.len) nc <<= 1;
            char *nb = (char *)kcc_arena_alloc(nc)
   ;
            kcc_memcpy(nb, buf, used);
            buf = nb; cap = nc;
        }
        kcc_memcpy(buf + used, pr.buf, pr.len);
        used += pr.len;
        kcc_pp_next();
        if (kcc_tok_type != TOK_STRING_LIT) break;
    }
    struct StrResult r;
    r.buf = buf;
    r.len = used;
    return r;
}
```

□

813 · used in 818a

73.12. The Compiler-Resident Built-in Headers

Like the assembly front end (§17.16.2), the self-hosting preprocessor carries a set of headers *inside the binary*, so that a program `#include`-ing `<stddef.h>` or `<stdio.h>` compiles with no header search path at all. This began as the six freestanding headers the C standard guarantees to a hosted-independent implementation (`stddef`, `stdint`, `stdarg`, `stdbool`, `limits`, `float`), plus the empty-but-required `iso646`, `stdnoreturn`, and `stdalign`. Compiling a real interpreter—Lua 5.4.7—pulls in far more: `<math.h>` for its numeric library, `<stdio.h>` and `<stdlib.h>` and `<string.h>` throughout, `<ctype.h>`, `<errno.h>`, `<locale.h>`, `<time.h>`, `<setjmp.h>` for its error mechanism, and a handful more. Rather than force every downloader to point `-I` at a system tree, the built-in set is extended to the twenty-eight headers below—*declarations only*, matching the assembly compiler's built-ins (§17.16.2) byte for byte, so a program compiles identically whether kcc is the assembly binary or its self-hosted self. The bodies are prototypes and macros; the actual definitions come from the system C library at link time, exactly as with a real freestanding toolchain. Where a declaration differs by platform—`errno` resolves through `__errno_location` on Linux and `__error` on macOS, and the `stdio` stream globals are named differently—an inner `#ifdef __linux__` selects the right one, which is why these headers are correct on both targets.

⟨ *Selfhost: pp builtin* 814 ⟩ ≡

```
static const char hdr_stddef[] = "#ifndef
   _STDDEF_H\n#define _STDDEF_H\ntypedef unsigned
   long size_t;\ntypedef long ptrdiff_t;\ntypedef
   long max_align_t;\n#define NULL ((void*)0)\n#
   define offsetof(t, m) ((size_t)&(((t*)0)->m))\n
   #endif\n";
static const char hdr_stdbool[] = "#ifndef
   _STDBOOL_H\n#define _STDBOOL_H\n#define bool
   int\n#define true 1\n#define false 0\n#endif\n"
   ;
static const char hdr_stdarg[] = "#ifndef
   _STDARG_H\n#define _STDARG_H\n#endif\n";
static const char hdr_stdint[] = "#ifndef
   _STDINT_H\n#define _STDINT_H\ntypedef signed
   char int8_t;\ntypedef unsigned char uint8_t;\
   ntypedef short int16_t;\ntypedef unsigned short
    uint16_t;\ntypedef int int32_t;\ntypedef
   unsigned int uint32_t;\ntypedef long int64_t;\
   ntypedef unsigned long uint64_t;\ntypedef long
   intptr_t;\ntypedef unsigned long uintptr_t;\
   ntypedef long intmax_t;\ntypedef unsigned long
   uintmax_t;\ntypedef signed char int_least8_t;\
   ntypedef unsigned char uint_least8_t;\ntypedef
   short int_least16_t;\ntypedef unsigned short
   uint_least16_t;\ntypedef int int_least32_t;\
   ntypedef unsigned int uint_least32_t;\ntypedef
   long int_least64_t;\ntypedef unsigned long
   uint_least64_t;\ntypedef signed char
   int_fast8_t;\ntypedef unsigned char
   uint_fast8_t;\ntypedef long int_fast16_t;\
   ntypedef unsigned long uint_fast16_t;\ntypedef
   long int_fast32_t;\ntypedef unsigned long
   uint_fast32_t;\ntypedef long int_fast64_t;\
   ntypedef unsigned long uint_fast64_t;\n#define
   INT8_MAX 127\n#define INT16_MAX 32767\n#define
   INT32_MAX 2147483647\n#define INT64_MAX
   9223372036854775807L\n#define UINT8_MAX 255\n#
   define UINT16_MAX 65535\n#define UINT32_MAX
   4294967295U\n#define UINT64_MAX
   18446744073709551615UL\n#define INTMAX_MAX
```

```
    9223372036854775807L\n#define UINTMAX_MAX
    18446744073709551615UL\n#define INTPTR_MAX
    9223372036854775807L\n#define UINTPTR_MAX
    18446744073709551615UL\n#define SIZE_MAX
    18446744073709551615UL\n#define INT8_MIN (-128)
    \n#define INT16_MIN (-32768)\n#define INT32_MIN
     (-2147483647-1)\n#define INT64_MIN
    (-9223372036854775807L-1)\n#define INTMAX_MIN
    (-9223372036854775807L-1)\n#define INTPTR_MIN
    (-9223372036854775807L-1)\n#endif\n";
static const char hdr_limits[] = "#ifndef
    _LIMITS_H\n#define _LIMITS_H\n#define CHAR_BIT
    8\n#define SCHAR_MIN (-128)\n#define SCHAR_MAX
    127\n#define UCHAR_MAX 255\n#define CHAR_MIN
    (-128)\n#define CHAR_MAX 127\n#define
    MB_LEN_MAX 6\n#define SHRT_MIN (-32768)\n#
    define SHRT_MAX 32767\n#define USHRT_MAX 65535\
    n#define INT_MIN (-2147483647 - 1)\n#define
    INT_MAX 2147483647\n#define UINT_MAX 4294967295
    U\n#define LONG_MIN (-9223372036854775807L - 1)
    \n#define LONG_MAX 9223372036854775807L\n#
    define ULONG_MAX 18446744073709551615UL\n#
    define LLONG_MIN (-9223372036854775807LL - 1)\n
    #define LLONG_MAX 9223372036854775807LL\n#
    define ULLONG_MAX 18446744073709551615ULL\n#
    endif\n";
static const char hdr_float[] = "#ifndef _FLOAT_H\
    n#define _FLOAT_H\n#define FLT_RADIX 2\n#define
     FLT_MANT_DIG 24\n#define DBL_MANT_DIG 53\n#
    define LDBL_MANT_DIG 53\n#define FLT_DIG 6\n#
    define DBL_DIG 15\n#define LDBL_DIG 15\n#define
     FLT_MIN_EXP (-125)\n#define DBL_MIN_EXP
    (-1021)\n#define LDBL_MIN_EXP (-1021)\n#define
    FLT_MAX_EXP 128\n#define DBL_MAX_EXP 1024\n#
    define LDBL_MAX_EXP 1024\n#define
    FLT_MIN_10_EXP (-37)\n#define DBL_MIN_10_EXP
    (-307)\n#define LDBL_MIN_10_EXP (-307)\n#define
     FLT_MAX_10_EXP 38\n#define DBL_MAX_10_EXP 308\
    n#define LDBL_MAX_10_EXP 308\n#define
    FLT_ROUNDS 1\n#define FLT_EVAL_METHOD 0\n#
    define FLT_EPSILON 1.19209290e-7F\n#define
    DBL_EPSILON 2.2204460492503131e-16\n#define
    LDBL_EPSILON 2.2204460492503131e-16\n#define
    FLT_MIN 1.17549435e-38F\n#define DBL_MIN
    2.2250738585072014e-308\n#define LDBL_MIN
    2.2250738585072014e-308\n#define FLT_MAX
    3.40282347e+38F\n#define DBL_MAX
    1.7976931348623157e+308\n#define LDBL_MAX
    1.7976931348623157e+308\n#define DECIMAL_DIG
    17\n#endif\n";
static const char hdr_iso646[] = "#ifndef
    _ISO646_H\n#define _ISO646_H\n#define and &&\n#
    define and_eq &=\n#define bitand &\n#define
    bitor |\n#define compl ~\n#define not !\n#
    define not_eq !=\n#define or ||\n#define or_eq
    |=\n#define xor ^\n#define xor_eq ^=\n#endif\n"
    ;
static const char hdr_stdnoreturn[] = "#ifndef
    _STDNORETURN_H\n#define _STDNORETURN_H\n#define
     noreturn _Noreturn\n#define
    __noreturn_is_defined 1\n#endif\n";
static const char hdr_stdalign[] = "#ifndef
    _STDALIGN_H\n#define _STDALIGN_H\n#define
    alignas _Alignas\n#define alignof _Alignof\n#
    define __alignas_is_defined 1\n#define
    __alignof_is_defined 1\n#endif\n";
static const char hdr_assert[] = "#ifndef
    _ASSERT_H\n#define _ASSERT_H\n#define assert(x)
     ((void)0)\n#endif\n\n";
static const char hdr_ctype[] = "#ifndef _CTYPE_H\
    n#define _CTYPE_H\nint isalpha(int c);\nint
    isdigit(int c);\nint isalnum(int c);\nint
    isspace(int c);\nint isupper(int c);\nint
    islower(int c);\nint ispunct(int c);\nint
    toupper(int c);\nint tolower(int c);\n#endif\n\
    n";
static const char hdr_errno[] = "#ifndef _ERRNO_H\
    n#define _ERRNO_H\n#ifdef __linux__\nint *
    __errno_location(void);\n#define errno (*
    __errno_location())\n#else\nint *__error(void)
    ;\n#define errno (*__error())\n#endif\n#define
    EPERM 1\n#define ENOENT 2\n#define ESRCH 3\n#
    define EINTR 4\n#define EIO 5\n#define ENXIO 6\
    n#define EBADF 9\n#define ENOMEM 12\n#define
    EACCES 13\n#define EFAULT 14\n#define EBUSY 16\
    n#define EEXIST 17\n#define ENOTDIR 20\n#define
     EISDIR 21\n#define EINVAL 22\n#define ENOSPC
    28\n#define EPIPE 32\n#define ERANGE 34\n#endif
    \n\n";
static const char hdr_fcntl[] = "#ifndef _FCNTL_H\
    n#define _FCNTL_H\ntypedef unsigned short
    mode_t;\n#define O_RDONLY   0x0000\n#define
    O_WRONLY   0x0001\n#define O_RDWR     0x0002\n#
    define O_ACCMODE  0x0003\n#define O_NONBLOCK 0
    x0004\n#define O_APPEND   0x0008\n#define
    O_CREAT    0x0200\n#define O_TRUNC    0x0400\n#
    define O_EXCL     0x0800\n#define O_CLOEXEC  0
    x1000000\nint open(const char *path, int oflag,
     ...);\nint creat(const char *path, mode_t mode
    );\nint fcntl(int fd, int cmd, ...);\n#endif\n\
    n";
static const char hdr_fenv[] = "#ifndef _FENV_H\n#
    define _FENV_H\ntypedef struct { unsigned long
    __fpsr; unsigned long __fpcr; } fenv_t;\
    ntypedef unsigned long fexcept_t;\nint
    feclearexcept(int excepts);\nint fetestexcept(
    int excepts);\nint feraiseexcept(int excepts);\
    nint fegetround(void);\nint fesetround(int
    round);\nint fegetenv(fenv_t *envp);\nint
    fesetenv(fenv_t *envp);\n#define FE_INEXACT
    0x10\n#define FE_UNDERFLOW  0x08\n#define
    FE_OVERFLOW   0x04\n#define FE_DIVBYZERO  0x02\
    n#define FE_INVALID    0x01\n#define
    FE_ALL_EXCEPT 0x1F\n#define FE_TONEAREST  0x00\
    n#define FE_UPWARD     0x01\n#define
    FE_DOWNWARD   0x02\n#define FE_TOWARDZERO 0x03\
    n#endif\n\n";
static const char hdr_inttypes[] = "#ifndef
    _INTTYPES_H\n#define _INTTYPES_H\n#include <
    stdint.h>\ntypedef struct { long quot; long rem
    ; } imaxdiv_t;\nlong imaxabs(long n);\
    nimaxdiv_t imaxdiv(long num, long den);\nlong
    strtoimax(char *s, char **end, int base);\
    nunsigned long strtoumax(char *s, char **end,
    int base);\n#define PRId8  \"d\"\n#define
    PRId16 \"d\"\n#define PRId32 \"d\"\n#define
    PRId64 \"ld\"\n#define PRIi64 \"li\"\n#define
    PRIu8  \"u\"\n#define PRIu16 \"u\"\n#define
    PRIu32 \"u\"\n#define PRIu64 \"lu\"\n#define
    PRIx64 \"lx\"\n#define PRIX64 \"lX\"\n#define
    PRIo64 \"lo\"\n#define PRIdPTR \"ld\"\n#define
    PRIuPTR \"lu\"\n#define PRIxPTR \"lx\"\n#endif\
    n\n";
static const char hdr_locale[] = "#ifndef
    _LOCALE_H\n#define _LOCALE_H\nstruct lconv {\n
       char *decimal_point;\n    char *
    thousands_sep;\n    char *grouping;\n    char *
    int_curr_symbol;\n    char *currency_symbol;\n
       char *mon_decimal_point;\n    char *
```

```
    mon_thousands_sep;\n    char *mon_grouping;\n
      char *positive_sign;\n    char *negative_sign
    ;\n    char int_frac_digits;\n    char
    frac_digits;\n    char p_cs_precedes;\n    char
     p_sep_by_space;\n    char n_cs_precedes;\n
    char n_sep_by_space;\n    char p_sign_posn;\n
      char n_sign_posn;\n    char int_p_cs_precedes
    ;\n    char int_n_cs_precedes;\n    char
    int_p_sep_by_space;\n    char
    int_n_sep_by_space;\n    char int_p_sign_posn;\
    n    char int_n_sign_posn;\n};\nchar *setlocale
    (int category, char *locale);\nstruct lconv *
    localeconv(void);\n#define LC_ALL      0\n#
    define LC_COLLATE  1\n#define LC_CTYPE    2\n#
    define LC_MONETARY 3\n#define LC_NUMERIC  4\n#
    define LC_TIME     5\n#endif\n\n";
static const char hdr_math[] = "#ifndef _MATH_H\n#
    define _MATH_H\n#define HUGE_VAL (1e308 * 10.0)
    \n#define INFINITY (1e308 * 10.0)\n#define NAN
    (0.0 * (1e308 * 10.0))\ndouble sqrt(double x);\
    ndouble cbrt(double x);\ndouble pow(double x,
    double y);\ndouble hypot(double x, double y);\
    ndouble sin(double x);\ndouble cos(double x);\
    ndouble tan(double x);\ndouble asin(double x);\
    ndouble acos(double x);\ndouble atan(double x)
    ;\ndouble atan2(double y, double x);\ndouble
    sinh(double x);\ndouble cosh(double x);\ndouble
     tanh(double x);\ndouble asinh(double x);\
    ndouble acosh(double x);\ndouble atanh(double x
    );\ndouble exp(double x);\ndouble exp2(double x
    );\ndouble expm1(double x);\ndouble log(double
    x);\ndouble log10(double x);\ndouble log2(
    double x);\ndouble log1p(double x);\ndouble
    frexp(double x, int *e);\ndouble ldexp(double x
    , int e);\ndouble modf(double x, double *ip);\
    ndouble scalbn(double x, int n);\ndouble fabs(
    double x);\ndouble floor(double x);\ndouble
    ceil(double x);\ndouble trunc(double x);\
    ndouble round(double x);\ndouble nearbyint(
    double x);\ndouble rint(double x);\ndouble fmod
    (double x, double y);\ndouble remainder(double
    x, double y);\ndouble copysign(double x, double
     y);\ndouble nextafter(double x, double y);\
    ndouble fdim(double x, double y);\ndouble fmax(
    double x, double y);\ndouble fmin(double x,
    double y);\ndouble fma(double x, double y,
    double z);\ndouble nan(const char *tag);\n#
    endif\n\n";
static const char hdr_setjmp[] = "#ifndef
    _SETJMP_H\n#define _SETJMP_H\ntypedef long
    jmp_buf[48];\nint setjmp(jmp_buf env);\nvoid
    longjmp(jmp_buf env, int val);\n#endif\n\n";
static const char hdr_signal[] = "#ifndef
    _SIGNAL_H\n#define _SIGNAL_H\ntypedef int
    sig_atomic_t;\ntypedef void (*sig_handler_t)(
    int);\nsig_handler_t signal(int sig,
    sig_handler_t handler);\nint raise(int sig);\n#
    define SIG_DFL ((sig_handler_t)0)\n#define
    SIG_IGN ((sig_handler_t)1)\n#define SIG_ERR ((
    sig_handler_t)-1)\n#define SIGINT  2\n#define
    SIGILL  4\n#define SIGABRT 6\n#define SIGFPE
    8\n#define SIGSEGV 11\n#define SIGTERM 15\n#
    endif\n\n";
static const char hdr_stdio[] = "#ifndef _STDIO_H\
    n#define _STDIO_H\n#ifndef NULL\n#define NULL
    ((void*)0)\n#endif\ntypedef unsigned long
    size_t;\ntypedef void FILE;\n#ifdef __linux__\
    nextern FILE *stdin;\nextern FILE *stdout;\
    nextern FILE *stderr;\n#else\nextern FILE *
    __stdinp;\nextern FILE *__stdoutp;\nextern FILE
     *__stderrp;\n#define stdin  __stdinp\n#define
    stdout __stdoutp\n#define stderr __stderrp\n#
    endif\n#define EOF (-1)\n#define BUFSIZ 1024\n#
    define FOPEN_MAX 20\n#define FILENAME_MAX 1024\
    n#define L_tmpnam 1024\n#define SEEK_SET 0\n#
    define SEEK_CUR 1\n#define SEEK_END 2\n#define
    _IOFBF 0\n#define _IOLBF 1\n#define _IONBF 2\
    nint printf(char *fmt, ...);\nint fprintf(FILE
    *stream, char *fmt, ...);\nint sprintf(char *
    str, char *fmt, ...);\nint snprintf(char *str,
    unsigned long size, char *fmt, ...);\nint puts(
    char *s);\nint putchar(int c);\nint fputs(char
    *s, FILE *stream);\nint fputc(int c, FILE *
    stream);\nint getchar(void);\nint scanf(char *
    fmt, ...);\nint sscanf(char *str, char *fmt,
    ...);\nFILE *fopen(char *path, char *mode);\
    nint fclose(FILE *stream);\nunsigned long fread
    (void *ptr, unsigned long size, unsigned long n
    , FILE *stream);\nunsigned long fwrite(void *
    ptr, unsigned long size, unsigned long n, FILE
    *stream);\nint fflush(FILE *stream);\nint fgetc
    (FILE *stream);\nchar *fgets(char *s, int n,
    FILE *stream);\nint getc(FILE *stream);\nint
    ungetc(int c, FILE *stream);\nint feof(FILE *
    stream);\nint ferror(FILE *stream);\nvoid
    clearerr(FILE *stream);\nint fseek(FILE *stream
    , long offset, int whence);\nlong ftell(FILE *
    stream);\nvoid rewind(FILE *stream);\nint
    setvbuf(FILE *stream, char *buf, int mode,
    unsigned long size);\nvoid setbuf(FILE *stream,
     char *buf);\nFILE *freopen(char *path, char *
    mode, FILE *stream);\nFILE *tmpfile(void);\
    nchar *tmpnam(char *s);\nint fscanf(FILE *
    stream, char *fmt, ...);\nvoid perror(char *s)
    ;\nint remove(char *path);\nint rename(char *
    oldp, char *newp);\n#endif\n\n";
static const char hdr_stdlib[] = "#ifndef
    _STDLIB_H\n#define _STDLIB_H\ntypedef unsigned
    long size_t;\n#define EXIT_SUCCESS 0\n#define
    EXIT_FAILURE 1\n#define RAND_MAX 2147483647\
    nvoid *malloc(unsigned long size);\nvoid *
    calloc(unsigned long n, unsigned long size);\
    nvoid *realloc(void *ptr, unsigned long size);\
    nvoid free(void *ptr);\nvoid exit(int status);\
    nvoid abort(void);\nint atoi(char *s);\nlong
    atol(char *s);\ndouble atof(char *s);\ndouble
    strtod(char *s, char **end);\nfloat strtof(char
     *s, char **end);\nlong strtol(char *s, char **
    end, int base);\nunsigned long strtoul(char *s,
     char **end, int base);\nlong long strtoll(char
     *s, char **end, int base);\nunsigned long long
     strtoull(char *s, char **end, int base);\nint
    abs(int x);\nlong labs(long x);\nint rand(void)
    ;\nvoid srand(unsigned int seed);\nvoid qsort(
    void *base, unsigned long n, unsigned long size
    , int (*cmp)(void *, void *));\nvoid *bsearch(
    void *key, void *base, unsigned long n,
    unsigned long size, int (*cmp)(void *, void *))
    ;\nchar *getenv(char *name);\nint system(char *
    cmd);\n#endif\n\n";
static const char hdr_string[] = "#ifndef
    _STRING_H\n#define _STRING_H\ntypedef unsigned
    long size_t;\nunsigned long strlen(char *s);\
    nint strcmp(char *a, char *b);\nint strncmp(
    char *a, char *b, unsigned long n);\nchar *
    strcpy(char *d, char *s);\nchar *strncpy(char *
    d, char *s, unsigned long n);\nchar *strcat(
    char *d, char *s);\nchar *strncat(char *d, char
     *s, unsigned long n);\nchar *strchr(char *s,
```

```
    int c);\nchar *strrchr(char *s, int c);\nchar *
    strstr(char *h, char *n);\nchar *strpbrk(char *
    s, char *accept);\nunsigned long strspn(char *s
    , char *accept);\nunsigned long strcspn(char *s
    , char *reject);\nint strcoll(char *a, char *b)
    ;\nchar *strerror(int errnum);\nvoid *memcpy(
    void *d, void *s, unsigned long n);\nvoid *
    memmove(void *d, void *s, unsigned long n);\
    nvoid *memset(void *d, int c, unsigned long n)
    ;\nvoid *memchr(void *s, int c, unsigned long n
    );\nint memcmp(void *a, void *b, unsigned long
    n);\n#endif\n\n";
static const char hdr_time[] = "#ifndef _TIME_H\n#
    define _TIME_H\ntypedef long time_t;\ntypedef
    long clock_t;\nstruct tm {\n    int tm_sec;\n
      int tm_min;\n    int tm_hour;\n    int
    tm_mday;\n    int tm_mon;\n    int tm_year;\n
      int tm_wday;\n    int tm_yday;\n    int
    tm_isdst;\n    long tm_gmtoff;\n    char *
    tm_zone;\n};\ntime_t time(time_t *t);\ndouble
    difftime(time_t end, time_t begin);\nclock_t
    clock(void);\nstruct tm *localtime(time_t *t);\
    nstruct tm *gmtime(time_t *t);\ntime_t mktime(
    struct tm *tm);\nchar *asctime(struct tm *tm);\
    nchar *ctime(time_t *t);\nunsigned long
    strftime(char *s, unsigned long max, char *fmt,
     struct tm *tm);\n#define CLOCKS_PER_SEC
    1000000\n#endif\n\n";
static const char hdr_uchar[] = "#ifndef _UCHAR_H\
    n#define _UCHAR_H\ntypedef unsigned short
    char16_t;\ntypedef unsigned int char32_t;\
    nunsigned long mbrtoc16(char16_t *pc16, char *s
    , unsigned long n, void *ps);\nunsigned long
    c16rtomb(char *s, char16_t c16, void *ps);\
    nunsigned long mbrtoc32(char32_t *pc32, char *s
    , unsigned long n, void *ps);\nunsigned long
    c32rtomb(char *s, char32_t c32, void *ps);\n#
    endif\n\n";
static const char hdr_unistd[] = "#ifndef
    _UNISTD_H\n#define _UNISTD_H\ntypedef long
    ssize_t;\ntypedef long off_t;\n#ifndef SEEK_SET
    \n#define SEEK_SET 0\n#define SEEK_CUR 1\n#
    define SEEK_END 2\n#endif\n#define STDIN_FILENO
     0\n#define STDOUT_FILENO 1\n#define
    STDERR_FILENO 2\n#define F_OK 0\n#define X_OK
    1\n#define W_OK 2\n#define R_OK 4\n#define
    _SC_PAGESIZE 30\nssize_t read(int fd, void *buf
    , unsigned long nbyte);\nssize_t write(int fd,
    const void *buf, unsigned long nbyte);\nssize_t
     pread(int fd, void *buf, unsigned long nbyte,
    off_t off);\nssize_t pwrite(int fd, const void
    *buf, unsigned long nbyte, off_t off);\nint
    close(int fd);\noff_t lseek(int fd, off_t
    offset, int whence);\nint unlink(const char *
    path);\nint rmdir(const char *path);\nint
    isatty(int fd);\nint access(const char *path,
    int mode);\nchar *getcwd(char *buf, unsigned
    long size);\nint chdir(const char *path);\nint
    fchdir(int fd);\nint ftruncate(int fd, off_t
    length);\nint fsync(int fd);\nint fdatasync(int
     fd);\nint fchown(int fd, unsigned uid,
    unsigned gid);\nunsigned geteuid(void);\
    nunsigned getuid(void);\nint getpid(void);\nint
     getpagesize(void);\nssize_t readlink(const
    char *path, char *buf, unsigned long size);\
    nint usleep(unsigned usec);\nunsigned sleep(
    unsigned sec);\nint pipe(int *fds);\nlong
    sysconf(int name);\nint gethostname(char *name,
     unsigned long len);\n#endif\n\n";
static const char hdr_wchar[] = "#ifndef _WCHAR_H\
    n#define _WCHAR_H\ntypedef int wchar_t;\n#
    ifndef _WINT_T\n#define _WINT_T\ntypedef int
    wint_t;\n#endif\nunsigned long wcslen(wchar_t *
    s);\nint wcscmp(wchar_t *a, wchar_t *b);\
    nwchar_t *wcscpy(wchar_t *d, wchar_t *s);\
    nwchar_t *wcscat(wchar_t *d, wchar_t *s);\
    nwchar_t *wcschr(wchar_t *s, wchar_t c);\n#
    define WEOF (-1)\n#endif\n\n";
static const char hdr_wctype[] = "#ifndef
    _WCTYPE_H\n#define _WCTYPE_H\n#ifndef _WINT_T\n
    #define _WINT_T\ntypedef int wint_t;\n#endif\
    ntypedef unsigned long wctype_t;\nint iswalpha(
    wint_t c);\nint iswdigit(wint_t c);\nint
    iswalnum(wint_t c);\nint iswspace(wint_t c);\
    nint iswupper(wint_t c);\nint iswlower(wint_t c
    );\nint iswpunct(wint_t c);\nwint_t towupper(
    wint_t c);\nwint_t towlower(wint_t c);\n#endif\
    n\n";

static const char *pp_builtin_header(const char *
    name) {
      if (kcc_strcmp2(name, "stddef.h")) return
    hdr_stddef;
      if (kcc_strcmp2(name, "stdbool.h")) return
    hdr_stdbool;
      if (kcc_strcmp2(name, "stdarg.h")) return
    hdr_stdarg;
      if (kcc_strcmp2(name, "stdint.h")) return
    hdr_stdint;
      if (kcc_strcmp2(name, "limits.h")) return
    hdr_limits;
      if (kcc_strcmp2(name, "float.h")) return
    hdr_float;
      if (kcc_strcmp2(name, "iso646.h")) return
    hdr_iso646;
      if (kcc_strcmp2(name, "stdnoreturn.h")) return
     hdr_stdnoreturn;
      if (kcc_strcmp2(name, "stdalign.h")) return
    hdr_stdalign;
      if (kcc_strcmp2(name, "assert.h")) return
    hdr_assert;
      if (kcc_strcmp2(name, "ctype.h")) return
    hdr_ctype;
      if (kcc_strcmp2(name, "errno.h")) return
    hdr_errno;
      if (kcc_strcmp2(name, "fcntl.h")) return
    hdr_fcntl;
      if (kcc_strcmp2(name, "fenv.h")) return
    hdr_fenv;
      if (kcc_strcmp2(name, "inttypes.h")) return
    hdr_inttypes;
      if (kcc_strcmp2(name, "locale.h")) return
    hdr_locale;
      if (kcc_strcmp2(name, "math.h")) return
    hdr_math;
      if (kcc_strcmp2(name, "setjmp.h")) return
    hdr_setjmp;
      if (kcc_strcmp2(name, "signal.h")) return
    hdr_signal;
      if (kcc_strcmp2(name, "stdio.h")) return
    hdr_stdio;
      if (kcc_strcmp2(name, "stdlib.h")) return
    hdr_stdlib;
      if (kcc_strcmp2(name, "string.h")) return
    hdr_string;
      if (kcc_strcmp2(name, "time.h")) return
    hdr_time;
      if (kcc_strcmp2(name, "uchar.h")) return
    hdr_uchar;
      if (kcc_strcmp2(name, "unistd.h")) return
    hdr_unistd;
```

```
    if (kcc_strcmp2(name, "wchar.h")) return
   hdr_wchar;
    if (kcc_strcmp2(name, "wctype.h")) return
   hdr_wctype;
    return 0;
}
```

□

814 · used in 818a

73.13. Assembling the Translation Unit

⟨ *Selfhost: preprocessor* 818a ⟩ ≡

```
⟨ Selfhost: pp preamble 795 ⟩
static int kcc_strcmp2(const char *a, const char *
   b) {
    unsigned long i = 0;
    while (a[i] && a[i] == b[i]) i++;
    return a[i] == 0 && b[i] == 0;
}
⟨ Selfhost: pp core 796 ⟩
⟨ Selfhost: pp helpers 798 ⟩
⟨ Selfhost: pp dispatch 799a ⟩
⟨ Selfhost: pp define 799b ⟩
⟨ Selfhost: pp expand 800 ⟩
⟨ Selfhost: pp substitute 803 ⟩
⟨ Selfhost: pp predefined 806 ⟩
⟨ Selfhost: pp conditional 807 ⟩
⟨ Selfhost: pp eval 809 ⟩
⟨ Selfhost: pp include 811 ⟩
⟨ Selfhost: pp builtin 814 ⟩
⟨ Selfhost: pp strlit 813 ⟩
```

□

818a · used in 36l

73.14. Verification

`make selfhost-preprocessor` runs the strongest oracle—`difftest` byte-compares generated objects against clang, and every case is preprocessed by the C engine—together with the self-host and lexer suites, the diagnostic suite, and the code-generation suite. A clean sweep means kcc preprocessed its own directives, macro expansions, conditionals, and includes faithfully enough that not one byte of generated code changed.

⟨ *Selfhost: preprocessor verify* 818b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-preprocessor - compile kcc's
   preprocessor in C with kcc, relink,
# run difftest + selfhost + diagtest + codegen.
set -euo pipefail

SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "

echo "[1/4] compiling $BASE/selfhost/preprocessor.
   c with kcc"
$BASE/kcc -c "$SH/preprocessor.c" -o "$SH/
   preprocessor_c.o" >/dev/null

echo "[2/4] linking $BASE/
   kcc_selfhost_preprocessor (preprocessor.o -> C)
   "
OBJ=""
for f in $sources; do
    if [ "$f" = "preprocessor" ]; then OBJ="$OBJ
   $SH/preprocessor_c.o"; else OBJ="$OBJ $BASE/src
   /${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_preprocessor $OBJ -
   lSystem -syslibroot "$SDK" -e _main -arch arm64

cp $BASE/kcc "$SH/kcc_asm_backup_preprocessor"
cp $BASE/kcc_selfhost_preprocessor $BASE/kcc

echo "[3/4] difftest (byte-compare) + diagtest"
set +e
bash $BASE/tests/difftest.sh >/$BASE/pp_difftest.
   log 2>&1; dstatus=$?
tail -1 /$BASE/pp_difftest.log
bash $BASE/tests/diagtest.sh >/$BASE/pp_diag.log
   2>&1; diagst=$?
tail -1 /$BASE/pp_diag.log
set -e
if [ $dstatus -ne 0 ] || [ $diagst -ne 0 ]; then
   cp "$SH/kcc_asm_backup_preprocessor" $BASE/kcc;
    exit 1; fi

echo "[4/4] codegen suite with the self-hosted
   preprocessor"
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_preprocessor" $BASE/kcc
exit $status
```

□

818b · used in 36m · uses _main 621, 629

74. Self-Hosting: The ARM64 Code Generator in C

The code generator of Chapters 35–35 is the compiler's back-end spine: it walks the neutral IR and emits ARM64 machine code, byte for byte the object clang would produce. Porting it—with the linear-scan register allocator, spill-slot colouring, the AAPCS64 argument marshaller, the FP d-register bank, and the per-opcode instruction dispatch—leaves only the IR builder in assembly. Because every generated instruction is checked against clang by `difftest`, this is the most byte-sensitive port in the whole self-hosting effort: a single different register assignment or slot colour would change the machine code and fail the comparison.

The C form is far more compact than the assembly it replaces, because each mov/bl/bl triple that computed an instruction word and appended it collapses to one nested call: `kcc_code_emit32(kcc_enc_add_reg(...))`.

74.1. State, Constants, Externs

⟨ *Selfhost: cg preamble* 819 ⟩ ≡

```
/* IR instruction / block / function field offsets
    */
#define IR_OP 0
#define IR_DEST 8
#define IR_SRC1 16
#define IR_SRC2 24
#define IR_TYPE 32
#define IR_NEXT 40
#define BB_ID 0
#define BB_FIRST 8
#define BB_LAST 16
#define IRF_NAME 0
#define IRF_BLOCKS 16
#define IRF_VREG_N 24
#define IRF_PARAMS 40
#define IRF_PARAM_CLS 48
#define IRF_LOWPC 64
#define IRF_STATIC 72
#define IRF_SYMIDX 80
#define IRF_VARIADIC 88
/* type / AST / vec / sym offsets */
#define TY_KIND 0
#define TY_SIZE 8
#define TY_ALIGN 16
#define TY_QUALS 24
#define TY_F0 32
#define TY_F1 40
#define AST_TAG 0
#define AST_F0 24
#define AST_F1 32
#define AST_F2 40
#define AST_F3 48
#define AST_F4 56
#define VEC_DATA 0
#define VEC_COUNT 8
#define VEC_CAP 16
#define VEC_ELEMSIZE 24
#define VEC_SIZE 32
#define SYM_ENTRY_SIZE 32
#define SYM_VALUE 16
#define N_SECT 0x0E
#define N_EXT 0x01
#define PATCH_SIZE 16
/* scratch registers */
#define CG_REG_SRC1 9
#define CG_REG_SRC2 10
#define CG_REG_DST 11
/* type kinds */
enum { TY_VOID=1, TY_BOOL=2, TY_LLONG=7, TY_FLOAT
   =8, TY_DOUBLE=9, TY_LDOUBLE=10,
       TY_PTR=11, TY_ARRAY=12, TY_STRUCT=14,
   TY_UNION=15, TY_ENUM=16 };
/* IR opcodes */
enum {
    IR_ADD=1, IR_SUB=2, IR_MUL=3, IR_SDIV=4,
   IR_UDIV=5, IR_SMOD=6, IR_UMOD=7,
    IR_NEG=8, IR_AND=10, IR_OR=11, IR_XOR=12,
   IR_SHL=13, IR_LSHR=14, IR_ASHR=15,
    IR_NOT=16, IR_CMP_EQ=20, IR_CMP_NE=21,
   IR_CMP_SLT=22, IR_CMP_SLE=23,
    IR_CMP_SGT=24, IR_CMP_SGE=25, IR_CMP_ULT=26,
   IR_CMP_ULE=27,
    IR_LOAD=30, IR_STORE=31, IR_ALLOCA=32, IR_GEP
   =33, IR_BR=40, IR_BR_COND=41,
    IR_RET=42, IR_CALL=43, IR_CALL_IND=44, IR_SEXT
   =50, IR_ZEXT=51, IR_TRUNC=52,
    IR_BITCAST=53, IR_SITOFP=54, IR_FPTOSI=55,
   IR_VA_START=56, IR_FPROUND=57,
    IR_UITOFP=58, IR_CONST=60, IR_NOP=61,
   IR_GLOBAL_REF=63, IR_STRING_LIT=64,
    IR_ASM=65
};
/* AST tags used by the prereg walk and rodata
   layout */
enum {
    AST_VAR_DECL=10, AST_FUNC_DECL=11, AST_STR_LIT
   =52, AST_UNARY_EXPR=55,
    AST_COMPOUND=30, AST_IF_STMT=32,
   AST_SWITCH_STMT=33, AST_WHILE_STMT=34,
    AST_DO_STMT=35, AST_FOR_STMT=36, AST_CASE_STMT
   =42, AST_LABEL_STMT=41,
    AST_DEFAULT_STMT=43, AST_INIT_LIST=65
};
/* ARM64 reloc types */
enum { ARM64_RELOC_BRANCH26=2, ARM64_RELOC_PAGE21
   =3, ARM64_RELOC_PAGEOFF12=4,
       ARM64_RELOC_GOT_LOAD_PAGE21=5,
   ARM64_RELOC_GOT_LOAD_PAGEOFF12=6 };

/* ARM64 instruction encoders (from emit_arm64) */
unsigned kcc_enc_add_reg(unsigned,unsigned,
   unsigned,unsigned);
unsigned kcc_enc_sub_reg(unsigned,unsigned,
   unsigned,unsigned);
unsigned kcc_enc_mul(unsigned,unsigned,unsigned,
   unsigned);
unsigned kcc_enc_sdiv(unsigned,unsigned,unsigned,
   unsigned);
unsigned kcc_enc_udiv(unsigned,unsigned,unsigned,
   unsigned);
unsigned kcc_enc_msub(unsigned,unsigned,unsigned,
   unsigned,unsigned);
unsigned kcc_enc_add_imm(unsigned,unsigned,
   unsigned,unsigned);
unsigned kcc_enc_sub_imm(unsigned,unsigned,
   unsigned,unsigned);
unsigned kcc_enc_movz(unsigned,unsigned,unsigned,
   unsigned);
unsigned kcc_enc_movk(unsigned,unsigned,unsigned,
   unsigned);
```

```
unsigned kcc_enc_and_reg(unsigned,unsigned,
    unsigned,unsigned);
unsigned kcc_enc_orr_reg(unsigned,unsigned,
    unsigned,unsigned);
unsigned kcc_enc_eor_reg(unsigned,unsigned,
    unsigned,unsigned);
unsigned kcc_enc_mvn(unsigned,unsigned,unsigned);
unsigned kcc_enc_lsl_reg(unsigned,unsigned,
    unsigned,unsigned);
unsigned kcc_enc_lsr_reg(unsigned,unsigned,
    unsigned,unsigned);
unsigned kcc_enc_asr_reg(unsigned,unsigned,
    unsigned,unsigned);
unsigned kcc_enc_cmp_reg(unsigned,unsigned,
    unsigned);
unsigned kcc_enc_cset(unsigned,unsigned,unsigned);
unsigned kcc_enc_str_imm(unsigned,unsigned,
    unsigned,unsigned);
unsigned kcc_enc_ldr_imm(unsigned,unsigned,
    unsigned,unsigned);
unsigned kcc_enc_stp_pre(unsigned,unsigned,
    unsigned,unsigned);
unsigned kcc_enc_ldp_post(unsigned,unsigned,
    unsigned,unsigned);
unsigned kcc_enc_mov_reg(unsigned,unsigned,
    unsigned);
unsigned kcc_enc_fp_dp2(unsigned,unsigned,unsigned
    ,unsigned);
unsigned kcc_enc_scvtf(unsigned,unsigned);
unsigned kcc_enc_ucvtf(unsigned,unsigned);
unsigned kcc_enc_fcvtzs(unsigned,unsigned);
unsigned kcc_enc_fcvt_s_d(unsigned,unsigned);
unsigned kcc_enc_fcvt_d_s(unsigned,unsigned);
unsigned kcc_enc_fmov_d_x(unsigned,unsigned);
unsigned kcc_enc_fmov_x_d(unsigned,unsigned);
unsigned kcc_enc_fmov_d_d(unsigned,unsigned);
unsigned kcc_enc_fneg_d(unsigned,unsigned);
unsigned kcc_enc_fcmp(unsigned,unsigned);
unsigned kcc_enc_ldr_fp(unsigned,unsigned,unsigned
    );
unsigned kcc_enc_str_fp(unsigned,unsigned,unsigned
    );
unsigned kcc_enc_ldr_fp_s(unsigned,unsigned,
    unsigned);
unsigned kcc_enc_str_fp_s(unsigned,unsigned,
    unsigned);
unsigned kcc_enc_b(long);
unsigned kcc_enc_bl(long);
unsigned kcc_enc_cbnz(unsigned,long,unsigned);
unsigned kcc_enc_ret(void);
unsigned kcc_enc_blr(unsigned);
unsigned kcc_enc_nop(void);
unsigned kcc_enc_adrp(unsigned,unsigned);
/* code buffer (emit_arm64) */
void kcc_code_init(void);
void kcc_code_emit32(unsigned word);
unsigned long kcc_code_current_pos(void);
void kcc_code_patch32(unsigned long off, unsigned
    word);
extern void *kcc_code_buf; extern unsigned long
    kcc_code_pos, kcc_code_cap;
/* linker / vec / ht / arena / string / sema */
void *kcc_arena_alloc(unsigned long);
void *kcc_memcpy(void*,const void*,unsigned long);
unsigned long kcc_strlen(const char*);
int kcc_strcmp(const char*,const char*);
void *kcc_ht_init(unsigned long);
void kcc_ht_put(void*,const void*,unsigned long,
    void*);
void *kcc_ht_get(void*,const void*,unsigned long);
void *kcc_vec_get(void*,unsigned long);
void kcc_vec_push(void*,const void*);
void *kcc_sema_resolve_type(void*);
unsigned long kcc_type_layout(void*);
unsigned long kcc_eval_const_int(void*);
const char *kcc_ir_static_name(void*);
unsigned long kcc_ir_load_width(void*);
int kcc_global_const_rodata(void*);
int kcc_global_const_ptrarr(void*);
void kcc_as_add_snip(unsigned long,const char*,
    unsigned long);
void kcc_io_err(const char*);
unsigned long kcc_linker_add_sym(const char*,
    unsigned long,unsigned long,unsigned long);
void kcc_linker_add_reloc(unsigned long,unsigned
    long,unsigned long);
unsigned long kcc_linker_extern(const char*);
unsigned long kcc_linker_global_sym(const char*,
    unsigned long);
extern void *kcc_ir_funcs, *kcc_linker_syms, *
    kcc_linker_extern_ht;
extern char kcc_str_modinit[];
extern unsigned long kcc_modinit_sym;

/* codegen state - this module is its definition
    site */
unsigned long kcc_cg_frame_size = 0;
unsigned long kcc_cg_vreg_count = 0;
void *kcc_cg_slotmap = 0;
unsigned long kcc_cg_max_slot = 0;
void *kcc_cstr_buf = 0;
unsigned long kcc_cstr_cap = 0;
unsigned long kcc_cstr_pos = 0;
unsigned long kcc_cstr_count = 0;
void *kcc_cstr_fixups = 0;
unsigned long kcc_rodata_const_present = 0;
unsigned long kcc_rodata_const_size = 0;
void *kcc_rodata_const_relocs = 0;
unsigned long kcc_rodata_const_fileoff = 0;
void *kcc_cg_defined_globals = 0;
void *kcc_cg_regmap = 0;
void *kcc_cg_promoted = 0;
unsigned long kcc_cg_callee_mask = 0;
unsigned long kcc_cg_save_count = 0;
unsigned long kcc_cg_save_base = 0;
void *kcc_cg_fpregmap = 0;
unsigned long kcc_cg_fp_callee_mask = 0;
unsigned long kcc_cg_fp_save_count = 0;
unsigned long kcc_cg_fp_save_base = 0;

#define IL(i,off) (*(unsigned long *)((char *)(i)
    + (off)))
#define EMIT(w) kcc_code_emit32((w))

/* forward decls */
void kcc_codegen_func(void *func);
void kcc_cg_regalloc(void *func);
void kcc_cg_color_slots(void *func, unsigned long
    vreg_n, unsigned long *lastuse, unsigned long *
    flags);
unsigned long kcc_cg_slot_offset(unsigned long
    vreg);
void kcc_cg_load_vreg(unsigned long vreg, unsigned
     long reg);
void kcc_cg_store_vreg(unsigned long vreg,
    unsigned long reg);
void kcc_cg_load_vreg_fp(unsigned long vreg,
    unsigned long reg);
void kcc_cg_store_vreg_fp(unsigned long vreg,
    unsigned long reg);
unsigned long kcc_cg_promoted_reg(unsigned long
    vreg);
void kcc_cg_callee_transfer(unsigned long restore)
```

```
    ;
unsigned long kcc_cg_marshal_args(unsigned long *
    args, unsigned long count, unsigned long fsi);
void kcc_codegen_instr(void *instr, void *
    bb_labels, void *patches);
void kcc_codegen_patch_branches(void *bb_labels,
    void *patches);
void kcc_codegen_string_lit(void *instr);
void kcc_cg_emit_add_imm(unsigned long rd,
    unsigned long rn, unsigned long imm);
void kcc_cg_emit_sub_imm(unsigned long rd,
    unsigned long rn, unsigned long imm);
void kcc_cg_emit_frame_ldst(unsigned long rt,
    unsigned long byte_off, unsigned long kind);
unsigned long kcc_emit_cstring(const char *data,
    unsigned long len);
void kcc_cstr_reserve(unsigned long need);
unsigned long kcc_rodata_leaf_bits(void *n,
    unsigned long kind);
void kcc_emit_rodata_global(void *decl, const char
     *uname);
void kcc_emit_rodata_ptrarr(void *decl, const char
     *uname);
void kcc_emit_struct_fields(void *sty, void *init,
     char *buf, unsigned long base);
void kcc_cg_prereg_static(void *decl);
void kcc_cg_prereg_stmt(void *stmt);
void kcc_cg_asm_extended(void *node);
static void cg_normalize_ret(void *ret_type);

/* Build the Mach-O "_"+name in the arena. */
static char *underscore(const char *name) {
    unsigned long l = kcc_strlen(name);
    char *b = (char *)kcc_arena_alloc(l + 2);
    b[0] = '_';
    kcc_memcpy(b + 1, name, l + 1);
    return b;
}
```

□

819 · used in 836a

74.2. Initialization and the Frame Constants

⟨ *Selfhost: cg init* 821a ⟩ ≡

```
void kcc_codegen_init(void) {
    kcc_cg_frame_size = 0;
    kcc_cstr_buf = kcc_arena_alloc(4096);
    kcc_cstr_cap = 4096;
    kcc_cstr_pos = 0;
    kcc_cstr_count = 0;
    void *fx = kcc_arena_alloc(VEC_SIZE);
    kcc_cstr_fixups = fx;
    IL(fx, VEC_ELEMSIZE) = 16;
    IL(fx, VEC_CAP) = 32;
    IL(fx, VEC_COUNT) = 0;
    IL(fx, VEC_DATA) = (unsigned long)
   kcc_arena_alloc(512);
    kcc_rodata_const_present = 0;
    kcc_rodata_const_size = 0;
    void *rc = kcc_arena_alloc(VEC_SIZE);
    kcc_rodata_const_relocs = rc;
    IL(rc, VEC_ELEMSIZE) = 16;
    IL(rc, VEC_CAP) = 32;
    IL(rc, VEC_COUNT) = 0;
    IL(rc, VEC_DATA) = (unsigned long)
   kcc_arena_alloc(512);
    kcc_cg_defined_globals = kcc_ht_init(32);
}
```

□

821a · used in 836a

74.3. Read-Only Constant Globals

`rodata_leaf_bits` folds a leaf initialiser to its raw bits; the rodata emitters lay a const scalar, array, struct, or array-of-struct into the read-only cstring buffer with a section-1 symbol and a fixup, and a const pointer array into `__DATA_CONST,__const` with per-slot relocations.

⟨ *Selfhost: cg rodata* 821b ⟩ ≡

```
unsigned long kcc_rodata_leaf_bits(void *n,
   unsigned long kind) {
    /* Only the three floating kinds take the
   float-bits path; a pointer or enum
       leaf (TY_PTR, TY_ENUM --- both numerically
   ABOVE TY_LDOUBLE) is an integer
       constant expression.  Treating a pointer as
    float returned the AST node's
       f0 word (a garbage address) instead of the
   value: a `.mm = (void *)0' in
       `init_task' became a wild pointer, so
   cpu_init's `BUG_ON(cur->mm)' fired
       and the kernel panicked killing the idle
   task. */
    if (kind < TY_FLOAT || kind > TY_LDOUBLE)
   return kcc_eval_const_int(n);
    if (IL(n, AST_TAG) == AST_UNARY_EXPR) {
          /* -literal: flip sign bit */
        void *lit = (void *)IL(n, AST_F1);
        return IL(lit, AST_F0) ^ (1UL << 63);
    }
    return IL(n, AST_F0);
          /* float literal bits */
}

/* Write `size` little-endian bytes of `bits` at
   buf[off], narrowing to single
 * precision when the element kind is float. */
static void write_le(char *buf, unsigned long off,
    unsigned long bits,
                         unsigned long size, unsigned
   long kind) {
    if (kind == TY_FLOAT) {
        /* narrow the double bit pattern to a
   single via the FP unit */
        double d;
        kcc_memcpy(&d, &bits, 8);
        float f = (float)d;
        unsigned w;
        kcc_memcpy(&w, &f, 4);
        bits = w;
    }
    for (unsigned long j = 0; j < size; j++) {
        buf[off + j] = (char)(bits & 0xff);
        bits >>= 8;
    }
}

void kcc_emit_struct_fields(void *sty, void *init,
    char *buf, unsigned long base) {
    void *decl = (void *)IL(sty, TY_F0);
    void *field = (void *)IL(decl, AST_F1);
    void *elem = (void *)IL(init, AST_F0);
```

```
    while (field && elem) {
        void *ft = kcc_sema_resolve_type((void *)
    IL(field, AST_F0));
        unsigned long fsize = IL(ft, TY_SIZE);
        unsigned long fkind = IL(ft, TY_KIND);
        unsigned long bits = kcc_rodata_leaf_bits(
    elem, fkind);
        unsigned long foff = IL(field, AST_F3);
        write_le(buf, base + foff, bits, fsize,
    fkind);
        field = (void *)IL(field, AST_F4);
        elem = (void *)IL(elem, AST_F4);
    }
}

void kcc_emit_rodata_global(void *decl, const char
    *uname) {
    void *full = kcc_sema_resolve_type((void *)IL(
    decl, AST_F0));
    unsigned long total = kcc_type_layout(full);
    unsigned long span = total;
    unsigned long kind = IL(full, TY_KIND);
    void *elemtype;
    if (kind == TY_STRUCT) {
        /* scalar struct */
        unsigned long pos = kcc_cstr_pos;
        unsigned long al = IL(full, TY_ALIGN);
        unsigned long o = (pos + al - 1) & ~(al -
    1);
        kcc_cstr_reserve(o + total);
        char *buf = (char *)kcc_cstr_buf;
        for (unsigned long i = pos; i < o + total;
     i++) buf[i] = 0;
        kcc_emit_struct_fields(full, (void *)IL(
    decl, AST_F2), buf, o);
        kcc_cstr_pos = o + total;
        unsigned long flags = N_SECT;
        if (IL(decl, AST_F3) != 3) flags |= N_EXT;
        unsigned long sym = kcc_linker_add_sym(
    uname, 1, 0, flags);
        unsigned long fix[2] = { sym, o };
        kcc_vec_push(kcc_cstr_fixups, fix);
        kcc_ht_put(kcc_linker_extern_ht, uname,
    kcc_strlen(uname), (void *)(sym + 1));
        return;
    }
    if (kind == TY_ARRAY && IL((void *)IL(full,
    TY_F0), TY_KIND) == TY_STRUCT) {
        /* array of structs */
        elemtype = (void *)IL(full, TY_F0);
        unsigned long stride = IL(elemtype,
    TY_SIZE);
        unsigned long cnt = 0;
        for (void *e = (void *)IL((void *)IL(decl,
     AST_F2), AST_F0); e; e = (void *)IL(e, AST_F4)
    ) cnt++;
        if (cnt * stride > span) span = cnt *
    stride;
        unsigned long pos = kcc_cstr_pos;
        unsigned long al = IL(elemtype, TY_ALIGN);
        unsigned long o = (pos + al - 1) & ~(al -
    1);
        kcc_cstr_reserve(o + span);
        char *buf = (char *)kcc_cstr_buf;
        for (unsigned long i = pos; i < o + span;
    i++) buf[i] = 0;
        unsigned long b = o;
        for (void *e = (void *)IL((void *)IL(decl,
     AST_F2), AST_F0); e; e = (void *)IL(e, AST_F4)
    ) {
            kcc_emit_struct_fields(elemtype, e,
    buf, b);
            b += stride;
        }
        kcc_cstr_pos = o + span;
        {
            unsigned long flags = N_SECT;
            if (IL(decl, AST_F3) != 3) flags |=
    N_EXT;
            unsigned long sym = kcc_linker_add_sym
    (uname, 1, 0, flags);
            unsigned long fix[2] = { sym, o };
            kcc_vec_push(kcc_cstr_fixups, fix);
            kcc_ht_put(kcc_linker_extern_ht, uname
    , kcc_strlen(uname), (void *)(sym + 1));
        }
        return;
    }
    /* scalar / array-of-scalar */
    if (kind == TY_ARRAY) elemtype = (void *)IL(
    full, TY_F0);
    else elemtype = full;
    unsigned long esize = IL(elemtype, TY_SIZE);
    unsigned long ekind = IL(elemtype, TY_KIND);
    unsigned long cnt = 1;
    void *init = (void *)IL(decl, AST_F2);
    if (IL(init, AST_TAG) == AST_INIT_LIST) {
        cnt = 0;
        for (void *e = (void *)IL(init, AST_F0); e
    ; e = (void *)IL(e, AST_F4)) cnt++;
    }
    if (cnt * esize > span) span = cnt * esize;
    unsigned long pos = kcc_cstr_pos;
    unsigned long al = IL(elemtype, TY_ALIGN);
    unsigned long o = (pos + al - 1) & ~(al - 1);
    kcc_cstr_reserve(o + span);
    char *buf = (char *)kcc_cstr_buf;
    for (unsigned long i = pos; i < o; i++) buf[i]
    = 0;
    unsigned long w = o;
    if (IL(init, AST_TAG) == AST_INIT_LIST) {
        for (void *e = (void *)IL(init, AST_F0); e
    ; e = (void *)IL(e, AST_F4)) {
            write_le(buf, w, kcc_rodata_leaf_bits(
    e, ekind), esize, ekind);
            w += esize;
        }
    } else {
        write_le(buf, w, kcc_rodata_leaf_bits(init
    , ekind), esize, ekind);
        w += esize;
    }
    for (; w < o + span; w++) buf[w] = 0;
    kcc_cstr_pos = o + span;
    {
        unsigned long flags = N_SECT;
        if (IL(decl, AST_F3) != 3) flags |= N_EXT;
        unsigned long sym = kcc_linker_add_sym(
    uname, 1, 0, flags);
        unsigned long fix[2] = { sym, o };
        kcc_vec_push(kcc_cstr_fixups, fix);
        kcc_ht_put(kcc_linker_extern_ht, uname,
    kcc_strlen(uname), (void *)(sym + 1));
    }
}

void kcc_emit_rodata_ptrarr(void *decl, const char
    *uname) {
    kcc_rodata_const_present = 1;
    unsigned long total = kcc_type_layout(
    kcc_sema_resolve_type((void *)IL(decl, AST_F0))
```

```
    );
    unsigned long o = (kcc_rodata_const_size + 7)
    & ~7UL;
    void *e = (void *)IL((void *)IL(decl, AST_F2),
    AST_F0);
    unsigned long i = 0;
    for (; e; e = (void *)IL(e, AST_F4), i++) {
        if (IL(e, AST_TAG) != AST_STR_LIT)
    continue;
        unsigned long sym = kcc_emit_cstring((
    const char *)IL(e, AST_F0), IL(e, AST_F1));
        unsigned long rel[2] = { o + i * 8, sym };
        kcc_vec_push(kcc_rodata_const_relocs, rel)
    ;
    }
    unsigned long span = i * 8;
    if (span < total) span = total;
    kcc_rodata_const_size = o + span;
    unsigned long flags = N_SECT;
    if (IL(decl, AST_F3) != 3) flags |= N_EXT;
    unsigned long sym = kcc_linker_add_sym(uname,
    3, o, flags);
    kcc_ht_put(kcc_linker_extern_ht, uname,
    kcc_strlen(uname), (void *)(sym + 1));
}
```

□

821b · used in 836a

74.4. Pre-registering Globals and Functions

⟨ *Selfhost: cg prereg* 823 ⟩ ≡

```
static unsigned long fam_extra(void *decl, void *
    ty) {
    if (IL(ty, TY_KIND) != TY_STRUCT) return 0;
    void *init = (void *)IL(decl, AST_F2);
    if (!init || IL(init, AST_TAG) !=
    AST_INIT_LIST) return 0;
    void *sdecl = (void *)IL(ty, TY_F0);
    if (!sdecl) return 0;
    void *field = (void *)IL(sdecl, AST_F1);
    if (!field) return 0;
    unsigned long nf = 1;
    while (IL(field, AST_F4)) { field = (void *)IL
    (field, AST_F4); nf++; }
    void *ft = kcc_sema_resolve_type((void *)IL(
    field, AST_F0));
    if (!ft || IL(ft, TY_KIND) != TY_ARRAY) return
    0;
    if (IL(ft, TY_F1)) return 0;
        /* sized: not flexible */
    unsigned long esz = kcc_type_layout((void *)IL
    (ft, TY_F0));
    void *e = (void *)IL(init, AST_F0);
    unsigned long skip = nf - 1;
    while (e && skip) { e = (void *)IL(e, AST_F4);
    skip--; }
    if (!e) return 0;
    if (IL(e, AST_TAG) == AST_STR_LIT) return IL(e
    , AST_F1) + 1;
    if (IL(e, AST_TAG) != AST_INIT_LIST) return 0;
    unsigned long c = 0;
    for (void *s = (void *)IL(e, AST_F0); s; s = (
    void *)IL(s, AST_F4)) c++;
    return c * esz;
}

void kcc_preregister_globals(void *tu) {
    void *first = (void *)IL(tu, AST_F0);
    for (void *d = first; d; d = (void *)IL(d,
    AST_F4)) {
        if (IL(d, AST_TAG) != AST_VAR_DECL)
    continue;
        void *name = (void *)IL(d, AST_F1);
        if (!name) continue;
        if (IL(d, AST_F3) == 2) continue;
          /* extern */
        void *ty = kcc_sema_resolve_type((void *)
    IL(d, AST_F0));
        if (!ty) continue;
        unsigned long size = kcc_type_layout(ty) +
     fam_extra(d, ty);
        if (size < 8) size = 8;
        size = (size + 7) & ~7UL;
        char *uname = underscore(name);
        if (kcc_global_const_rodata(d))
    kcc_emit_rodata_global(d, uname);
        else if (kcc_global_const_ptrarr(d))
    kcc_emit_rodata_ptrarr(d, uname);
        else kcc_linker_global_sym(uname, size);
        kcc_ht_put(kcc_cg_defined_globals, name,
    kcc_strlen(name), (void *)1);
    }
    for (void *d = first; d; d = (void *)IL(d,
    AST_F4)) {
        if (IL(d, AST_TAG) != AST_FUNC_DECL)
    continue;
        void *body = (void *)IL(d, AST_F3);
        if (body) kcc_cg_prereg_stmt(body);
    }
}

void kcc_cg_prereg_static(void *decl) {
    const char *mangled = kcc_ir_static_name(decl)
    ;
    void *ty0 = (void *)IL(decl, AST_F0);
    unsigned long size = 8;
    if (ty0) {
        void *ty = kcc_sema_resolve_type(ty0);
        if (ty) {
            unsigned long s = kcc_type_layout(ty);
            if (s >= 8) size = (s + 7) & ~7UL;
        }
    }
    char *uname = underscore(mangled);
    if (kcc_global_const_rodata(decl))
    kcc_emit_rodata_global(decl, uname);
    else if (kcc_global_const_ptrarr(decl))
    kcc_emit_rodata_ptrarr(decl, uname);
    else kcc_linker_global_sym(uname, size);
    kcc_ht_put(kcc_cg_defined_globals, mangled,
    kcc_strlen(mangled), (void *)1);
}

void kcc_cg_prereg_stmt(void *stmt) {
    for (void *s = stmt; s; s = (void *)IL(s,
    AST_F4)) {
        unsigned long tag = IL(s, AST_TAG);
        if (tag == AST_VAR_DECL) { if (IL(s,
    AST_F3) == 3) kcc_cg_prereg_static(s); continue
    ; }
        switch (tag) {
        case AST_COMPOUND: case AST_DO_STMT: case
    AST_DEFAULT_STMT:
            kcc_cg_prereg_stmt((void *)IL(s,
    AST_F0)); break;
        case AST_SWITCH_STMT: case AST_WHILE_STMT:
     case AST_CASE_STMT: case AST_LABEL_STMT:
```

```
            kcc_cg_prereg_stmt((void *)IL(s,
   AST_F1)); break;
        case AST_IF_STMT:
            kcc_cg_prereg_stmt((void *)IL(s,
   AST_F1));
            kcc_cg_prereg_stmt((void *)IL(s,
   AST_F2));
            break;
        case AST_FOR_STMT:
            kcc_cg_prereg_stmt((void *)IL(s,
   AST_F0));
            kcc_cg_prereg_stmt((void *)IL(s,
   AST_F3));
            break;
        default: break;
        }
    }
}

void kcc_cg_preregister_funcs(void) {
    void *funcs = kcc_ir_funcs;
    if (!funcs) return;
    unsigned long n = IL(funcs, VEC_COUNT);
    for (unsigned long i = 0; i < n; i++) {
        void *f = *(void **)kcc_vec_get(funcs, i);
        const char *name = (const char *)IL(f,
   IRF_NAME);
        int is_modinit = (kcc_strcmp(name,
   kcc_str_modinit) == 0);
        char *uname = underscore(name);
        unsigned long flags = N_SECT | N_EXT;
        if (is_modinit || IL(f, IRF_STATIC)) flags
    = N_SECT;
        unsigned long sym = kcc_linker_add_sym(
   uname, 1, 0, flags);
        IL(f, IRF_SYMIDX) = sym;
        if (is_modinit) { kcc_modinit_sym = sym +
   1; continue; }
        kcc_ht_put(kcc_linker_extern_ht, uname,
   kcc_strlen(uname), (void *)(sym + 1));
    }
}

void kcc_codegen_all(void) {
    kcc_cg_preregister_funcs();
    void *funcs = kcc_ir_funcs;
    if (!funcs) return;
    unsigned long n = IL(funcs, VEC_COUNT);
    for (unsigned long i = 0; i < n; i++)
        kcc_codegen_func(*(void **)kcc_vec_get(
   funcs, i));
}
```

□

823 · used in 836a

74.5. Register Allocation

The linear-scan allocator: pass 1 records per-vreg liveness (last use, defining block, escape, call-crossing) and candidate flags; a promotability scan marks scalar locals whose address never escapes; loop detection sets has_loop; then the integer and FP scans hand out registers; finally the promotion map is built and the spill slots are coloured.

⟨ *Selfhost: cg regalloc* 824 ⟩ ≡

```
static void ra_use(unsigned long u, unsigned long
   cur_bb, unsigned long pos, unsigned long calls,
                   unsigned long vreg_n, unsigned
   long *lastuse, unsigned long *defbb,
                   unsigned long *calldef,
   unsigned long *flags) {
    if (u >= vreg_n) return;
    lastuse[u] = pos;
    if (defbb[u] != cur_bb) flags[u] |= 2;
        /* escapes */
    if (calls > calldef[u]) flags[u] |= 4;
        /* crosses a call */
}
static void ra_disq(unsigned long u, unsigned long
    *flags, unsigned long vreg_n) {
    if (u >= vreg_n) return;
    if (flags[u] & 0x100) flags[u] &= ~(unsigned
   long)0x100;
}

void kcc_cg_regalloc(void *func) {
    unsigned long vreg_n = IL(func, IRF_VREG_N);
    kcc_cg_regmap = 0; kcc_cg_promoted = 0;
   kcc_cg_callee_mask = 0; kcc_cg_save_count = 0;
    kcc_cg_fpregmap = 0; kcc_cg_fp_callee_mask =
   0; kcc_cg_fp_save_count = 0;
    kcc_cg_slotmap = 0; kcc_cg_max_slot = vreg_n;
    if (!vreg_n) return;
    unsigned long *block = (unsigned long *)
   kcc_arena_alloc(vreg_n * 8 * 7);
    for (unsigned long i = 0; i < vreg_n * 7; i++)
    block[i] = 0;
    unsigned long *lastuse = block, *defbb = block
    + vreg_n, *calldef = block + 2 * vreg_n;
    unsigned long *flags = block + 3 * vreg_n, *
   map = block + 4 * vreg_n;
    unsigned long *promoted = block + 5 * vreg_n,
   *fpmap = block + 6 * vreg_n;
    void *blocks = (void *)IL(func, IRF_BLOCKS);
    unsigned long nbb = IL(blocks, VEC_COUNT);

    /* --- pass 1: liveness / candidates --- */
    unsigned long pos = 0, calls = 0;
    for (unsigned long b = 0; b < nbb; b++) {
        void *bb = *(void **)kcc_vec_get(blocks, b
   );
        for (void *i = (void *)IL(bb, BB_FIRST); i
   ; i = (void *)IL(i, IR_NEXT)) {
            unsigned long op = IL(i, IR_OP);
            if (op == IR_ASM) {
                void *node = (void *)IL(i, IR_TYPE
   );
                if (node) for (void *o = (void *)
   IL(node, AST_F2); o; o = (void *)IL(o, AST_F4))
    {
                    unsigned long v = IL(o, AST_F2
   ) & 0xffffffff;
                    if (v) ra_use(v - 1, b, pos,
   calls, vreg_n, lastuse, defbb, calldef, flags);
                    unsigned long a = IL(o, AST_F3
   ) & 0xffffffff;
                    if (a) ra_use(a - 1, b, pos,
   calls, vreg_n, lastuse, defbb, calldef, flags);
                }
                pos++;
                continue;
            }
            /* SRC1 use (skip const/call/br/
   global_ref) */
            if (op != IR_CONST && op != IR_CALL &&
    op != IR_BR && op != IR_GLOBAL_REF)
                ra_use(IL(i, IR_SRC1), b, pos,
   calls, vreg_n, lastuse, defbb, calldef, flags);
```

```
        /* SRC2 use / call args / brcond */
        if (op == IR_CALL || op == IR_CALL_IND
) {
            unsigned long *args = (unsigned
long *)IL(i, IR_SRC2);
            if (args) {
                unsigned long ac = args[0] & 0
xffffffff;
                for (unsigned long k = 0; k <
ac; k++)
                    ra_use(args[1 + 2 * k], b,
 pos, calls, vreg_n, lastuse, defbb, calldef,
flags);
            }
            calls++;
            /* struct/union-returning call:
DEST is a use, escapes */
            void *ty = (void *)IL(i, IR_TYPE);
            if (ty && (IL(ty, 0) == 14 || IL(
ty, 0) == 15)) {
                ra_use(IL(i, IR_DEST), b, pos,
 calls, vreg_n, lastuse, defbb, calldef, flags)
;
                unsigned long dd = IL(i,
IR_DEST);
                if (dd < vreg_n) flags[dd] |=
2;
            }
            pos++;
            continue;
        }
        if (op == IR_BR_COND) {
            ra_use(IL(i, IR_DEST), b, pos,
calls, vreg_n, lastuse, defbb, calldef, flags);
            pos++;
            continue;
        }
        if (op != IR_BR && op != IR_CONST)
            ra_use(IL(i, IR_SRC2), b, pos,
calls, vreg_n, lastuse, defbb, calldef, flags);
        /* def check */
        if (op != IR_BR && op != IR_STORE &&
op != IR_RET) {
            int is_fp = 0, is_cand = 0;
            if (op == IR_ALLOCA) is_cand = 1;
            else {
                void *ty = (void *)IL(i,
IR_TYPE);
                if (ty) {
                    unsigned long k = IL(ty,
TY_KIND);
                    if (k >= 2 && k <= 7)
is_cand = 1;
                    else if (k >= 8 && k <=
10) { is_cand = 1; is_fp = 1; }
                    else if (k == 11 || k ==
16) is_cand = 1;
                }
            }
            if (is_cand) {
                unsigned long d = IL(i,
IR_DEST);
                if (d < vreg_n) {
                    if (is_fp) flags[d] |= 0x8
;
                    defbb[d] = b;
                    calldef[d] = calls;
                    flags[d] |= 1;
                }
            }
        }
        pos++;
    }
}

/* --- promotability (mem2reg) --- */
for (unsigned long b = 0; b < nbb; b++) {
    void *bb = *(void **)kcc_vec_get(blocks, b
);
    for (void *i = (void *)IL(bb, BB_FIRST); i
; i = (void *)IL(i, IR_NEXT)) {
        unsigned long op = IL(i, IR_OP);
        if (op == IR_ASM) {
            void *node = (void *)IL(i, IR_TYPE
);
            if (node) for (void *o = (void *)
IL(node, AST_F2); o; o = (void *)IL(o, AST_F4))
 {
                unsigned long a = IL(o, AST_F3
) & 0xffffffff;
                if (a) ra_disq(a - 1, flags,
vreg_n);
            }
            continue;
        }
        if (op == IR_ALLOCA) {
            void *et = (void *)IL(i, IR_TYPE);
            if (!et) continue;
            if (IL(et, TY_QUALS) & 2) continue
;
            unsigned long k = IL(et, TY_KIND);
            int scalar = (k >= 2 && k <= 7) ||
 k == 11 || k == 16;
            if (!scalar) continue;
            if (IL(i, IR_SRC1) > 1) continue;
            unsigned long d = IL(i, IR_DEST);
            if (d < vreg_n) flags[d] |= 0x100;
            continue;
        }
        if (op == IR_STORE) { ra_disq(IL(i,
IR_SRC1), flags, vreg_n); continue; }
        if (op == IR_LOAD || op == IR_VA_START
 || op == IR_CONST || op == IR_NOP ||
            op == IR_GLOBAL_REF || op ==
IR_STRING_LIT || op == IR_BR) continue;
        if (op == IR_BR_COND) { ra_disq(IL(i,
IR_DEST), flags, vreg_n); continue; }
        if (op == IR_RET) {
            void *ty = (void *)IL(i, IR_TYPE);
            if (ty && IL(ty, 0) != TY_VOID)
ra_disq(IL(i, IR_SRC1), flags, vreg_n);
            continue;
        }
        if (op == IR_CALL_IND) ra_disq(IL(i,
IR_SRC1), flags, vreg_n);
        if (op == IR_CALL || op == IR_CALL_IND
) {
            unsigned long *args = (unsigned
long *)IL(i, IR_SRC2);
            if (args) {
                unsigned long ac = args[0] & 0
xffffffff;
                for (unsigned long k = 0; k <
ac; k++) ra_disq(args[1 + 2 * k], flags, vreg_n
);
            }
            continue;
        }
        if (op == IR_NEG || op == IR_NOT || op
 >= 50) {
```

```
            ra_disq(IL(i, IR_SRC1), flags,
vreg_n);
        } else {
            ra_disq(IL(i, IR_SRC1), flags,
vreg_n);
            ra_disq(IL(i, IR_SRC2), flags,
vreg_n);
        }
    }
}

/* --- loop detection --- */
int has_loop = 0;
for (unsigned long b = 0; b < nbb; b++) {
    void *bb = *(void **)kcc_vec_get(blocks, b
);
    unsigned long id = IL(bb, BB_ID);
    void *term = (void *)IL(bb, BB_LAST);
    if (!term) continue;
    unsigned long top = IL(term, IR_OP);
    if (top == IR_BR) { if (IL(term, IR_SRC1)
<= id) has_loop = 1; }
    else if (top == IR_BR_COND) {
        if (IL(term, IR_SRC1) <= id || IL(term
, IR_SRC2) <= id) has_loop = 1;
    }
}

/* --- integer linear scan --- */
long reg_end[13];
for (int i = 0; i < 13; i++) reg_end[i] = -1;
if (has_loop) {
    unsigned long slot = 4;
      /* index 4 = x19 */
    for (unsigned long v = 0; v < vreg_n &&
slot < 13; v++) {
        unsigned long fl = flags[v];
        if (fl & 0x8) continue;
        if ((fl & 3) != 3) continue;
        if (!lastuse[v]) continue;
        unsigned long preg = 15 + slot;
        map[v] = preg;
        reg_end[slot] = 0x40000000;
        kcc_cg_callee_mask |= 1UL << (preg -
19);
        slot++;
    }
}
pos = 0;
for (unsigned long b = 0; b < nbb; b++) {
    void *bb = *(void **)kcc_vec_get(blocks, b
);
    for (void *i = (void *)IL(bb, BB_FIRST); i
; i = (void *)IL(i, IR_NEXT), pos++) {
        unsigned long op = IL(i, IR_OP);
        if (op == IR_BR || op == IR_BR_COND ||
 op == IR_STORE || op == IR_RET) continue;
        if (op == IR_CALL || op == IR_CALL_IND
) {
            void *ty = (void *)IL(i, IR_TYPE);
            if (ty && (IL(ty, 0) == 14 || IL(
ty, 0) == 15)) continue;
        }
        unsigned long d = IL(i, IR_DEST);
        if (d >= vreg_n) continue;
        unsigned long fl = flags[d];
        if (!(fl & 1)) continue;
        if (fl & 0x8) continue;
        if (fl & 2) { if (has_loop) continue;
}
        unsigned long lu = lastuse[d];
        if (lu <= pos) continue;
        unsigned long start = (fl & 4) ? 4 :
0;
        for (unsigned long s = start; s < 13;
s++) {
            if (reg_end[s] >= (long)pos)
continue;
            unsigned long preg = (s < 4) ? (12
 + s) : (15 + s);
            map[d] = preg;
            reg_end[s] = (long)lu;
            if (s >= 4) kcc_cg_callee_mask |=
1UL << (preg - 19);
            break;
        }
    }
}

/* --- FP linear scan --- */
long reg_end_fp[24];
for (int i = 0; i < 24; i++) reg_end_fp[i] =
-1;
if (has_loop) {
    unsigned long slot = 16;
      /* 16 → d8 */
    for (unsigned long v = 0; v < vreg_n &&
slot < 24; v++) {
        unsigned long fl = flags[v];
        if (!(fl & 0x8)) continue;
        if ((fl & 3) != 3) continue;
        if (!lastuse[v]) continue;
        unsigned long dreg = slot - 8;
        fpmap[v] = dreg;
        reg_end_fp[slot] = 0x40000000;
        kcc_cg_fp_callee_mask |= 1UL << (dreg
- 8);
        slot++;
    }
}
pos = 0;
for (unsigned long b = 0; b < nbb; b++) {
    void *bb = *(void **)kcc_vec_get(blocks, b
);
    for (void *i = (void *)IL(bb, BB_FIRST); i
; i = (void *)IL(i, IR_NEXT), pos++) {
        unsigned long op = IL(i, IR_OP);
        if (op == IR_BR || op == IR_BR_COND ||
 op == IR_STORE || op == IR_RET) continue;
        unsigned long d = IL(i, IR_DEST);
        if (d >= vreg_n) continue;
        unsigned long fl = flags[d];
        if (!(fl & 0x8)) continue;
        if (fl & 2) { if (has_loop) continue;
}
        unsigned long lu = lastuse[d];
        if (lu <= pos) continue;
        unsigned long start = (fl & 4) ? 16 :
0;
        for (unsigned long s = start; s < 24;
s++) {
            if (reg_end_fp[s] >= (long)pos)
continue;
            unsigned long dreg = (s < 16) ?
(16 + s) : (s - 8);
            fpmap[d] = dreg;
            reg_end_fp[s] = (long)lu;
            if (s >= 16) kcc_cg_fp_callee_mask
 |= 1UL << (dreg - 8);
            break;
        }
```

```
        }
    }
    kcc_cg_fpregmap = fpmap;
    { unsigned long m = kcc_cg_fp_callee_mask, c =
    0; while (m) { c += m & 1; m >>= 1; }
   kcc_cg_fp_save_count = c; }

    /* --- promotion map --- */
    for (unsigned long v = 0; v < vreg_n; v++)
        if ((flags[v] & 0x100) && map[v]) promoted
   [v] = map[v];
    kcc_cg_promoted = promoted;
    kcc_cg_regmap = map;
    { unsigned long m = kcc_cg_callee_mask, c = 0;
    while (m) { c += m & 1; m >>= 1; }
   kcc_cg_save_count = c; }

    kcc_cg_color_slots(func, vreg_n, lastuse,
   flags);
}
```

□

824 · used in 836a

74.6. Spill-Slot Colouring

⟨ *Selfhost: cg color* 827a ⟩ ≡

```
void kcc_cg_color_slots(void *func, unsigned long
   vreg_n, unsigned long *lastuse, unsigned long *
   flags) {
    long *slotmap = (long *)kcc_arena_alloc(vreg_n
    * 8 * 4);
    unsigned long *active_vreg = (unsigned long *)
   slotmap + vreg_n;
    unsigned long *active_slot = active_vreg +
   vreg_n;
    unsigned long *free_slot = active_slot +
   vreg_n;
    for (unsigned long v = 0; v < vreg_n; v++)
   slotmap[v] = -1;
    unsigned long *regmap = (unsigned long *)
   kcc_cg_regmap;
    unsigned long *fpmap = (unsigned long *)
   kcc_cg_fpregmap;
    unsigned long *promoted = (unsigned long *)
   kcc_cg_promoted;
    (void)regmap;
    unsigned long next_slot = 0, active_count = 0,
    free_count = 0, pos = 0;
    void *blocks = (void *)IL(func, IRF_BLOCKS);
    unsigned long nbb = IL(blocks, VEC_COUNT);
    unsigned long params = IL(func, IRF_PARAMS);
    for (unsigned long b = 0; b < nbb; b++) {
        void *bb = *(void **)kcc_vec_get(blocks, b
   );
        for (void *i = (void *)IL(bb, BB_FIRST); i
   ; i = (void *)IL(i, IR_NEXT), pos++) {
            /* expire */
            unsigned long w = 0;
            for (unsigned long r = 0; r <
   active_count; r++) {
                unsigned long v = active_vreg[r],
   sl = active_slot[r];
                if (lastuse[v] >= pos) {
   active_vreg[w] = v; active_slot[w] = sl; w++; }
                else free_slot[free_count++] = sl;
            }
            active_count = w;
            /* assign */
            unsigned long op = IL(i, IR_OP);
            if (op == IR_STORE || op == IR_BR ||
   op == IR_BR_COND || op == IR_RET) continue;
            unsigned long d = IL(i, IR_DEST);
            if (d == 0 || d >= vreg_n) continue;
           /* matches asm cbz/hs skip */
            if (fpmap && fpmap[d]) continue;
            if (promoted && promoted[d]) continue;
            if (d < params) { slotmap[d] =
   next_slot++; continue; }
            if (flags[d] & 2) { slotmap[d] =
   next_slot++; continue; }
            unsigned long slot;
            if (free_count) slot = free_slot[--
   free_count];
            else slot = next_slot++;
            slotmap[d] = slot;
            active_vreg[active_count] = d;
            active_slot[active_count] = slot;
            active_count++;
        }
    }
    for (unsigned long v = 0; v < vreg_n; v++) {
        if (slotmap[v] != -1) continue;
        if (fpmap && fpmap[v]) continue;
        if (promoted && promoted[v]) continue;
        slotmap[v] = next_slot++;
    }
    kcc_cg_slotmap = slotmap;
    kcc_cg_max_slot = next_slot;
}
```

□

827a · used in 836a

74.7. Frame, Slots, Transfers, and Marshalling

⟨ *Selfhost: cg frame* 827b ⟩ ≡

```
void kcc_cg_emit_add_imm(unsigned long rd,
   unsigned long rn, unsigned long imm) {
    unsigned long high = imm >> 12;
    unsigned long run = rn;
    if (high) {
        EMIT(kcc_enc_add_imm(rd, run, high, 1) | 0
   x400000u);
        run = rd;
    }
    unsigned long low = imm & 0xFFF;
    if (low || !high) EMIT(kcc_enc_add_imm(rd, run
   , low, 1));
}
void kcc_cg_emit_sub_imm(unsigned long rd,
   unsigned long rn, unsigned long imm) {
    unsigned long high = imm >> 12;
    unsigned long run = rn;
    if (high) {
        EMIT(kcc_enc_sub_imm(rd, run, high, 1) | 0
   x400000u);
        run = rd;
    }
    unsigned long low = imm & 0xFFF;
    if (low || !high) EMIT(kcc_enc_sub_imm(rd, run
   , low, 1));
}
void kcc_cg_emit_frame_ldst(unsigned long rt,
   unsigned long byte_off, unsigned long kind) {
    unsigned long rn, imm12;
    if (byte_off <= 0x7FF8) { rn = 29; imm12 =
   byte_off >> 3; }
```

```
    else { kcc_cg_emit_add_imm(17, 29, byte_off);
   rn = 17; imm12 = 0; }
    if (kind == 0) EMIT(kcc_enc_ldr_imm(rt, rn,
   imm12, 1));
    else if (kind == 1) EMIT(kcc_enc_str_imm(rt,
   rn, imm12, 1));
    else if (kind == 2) EMIT(kcc_enc_ldr_fp(rt, rn
   , imm12));
    else EMIT(kcc_enc_str_fp(rt, rn, imm12));
}

unsigned long kcc_cg_slot_offset(unsigned long
   vreg) {
    long *sm = (long *)kcc_cg_slotmap;
    unsigned long s = sm ? (unsigned long)sm[vreg]
    : vreg;
    return s + 2;
}

void kcc_cg_load_vreg(unsigned long vreg, unsigned
    long reg) {
    unsigned long *fp = (unsigned long *)
   kcc_cg_fpregmap;
    if (fp && fp[vreg]) { EMIT(kcc_enc_fmov_x_d(
   reg, fp[vreg])); return; }
    unsigned long *rm = (unsigned long *)
   kcc_cg_regmap;
    if (rm && rm[vreg]) {
        if (reg == rm[vreg]) return;
        EMIT(kcc_enc_mov_reg(reg, rm[vreg], 1));
        return;
    }
    kcc_cg_emit_frame_ldst(reg, kcc_cg_slot_offset
   (vreg) << 3, 0);
}
void kcc_cg_store_vreg(unsigned long vreg,
   unsigned long reg) {
    unsigned long *fp = (unsigned long *)
   kcc_cg_fpregmap;
    if (fp && fp[vreg]) { EMIT(kcc_enc_fmov_d_x(fp
   [vreg], reg)); return; }
    unsigned long *rm = (unsigned long *)
   kcc_cg_regmap;
    if (rm && rm[vreg]) {
        if (reg == rm[vreg]) return;
        EMIT(kcc_enc_mov_reg(rm[vreg], reg, 1));
        return;
    }
    kcc_cg_emit_frame_ldst(reg, kcc_cg_slot_offset
   (vreg) << 3, 1);
}
unsigned long kcc_cg_promoted_reg(unsigned long
   vreg) {
    unsigned long *p = (unsigned long *)
   kcc_cg_promoted;
    return p ? p[vreg] : 0;
}
void kcc_cg_load_vreg_fp(unsigned long vreg,
   unsigned long reg) {
    unsigned long *fp = (unsigned long *)
   kcc_cg_fpregmap;
    if (fp && fp[vreg]) {
        if (reg == fp[vreg]) return;
        EMIT(kcc_enc_fmov_d_d(reg, fp[vreg]));
        return;
    }
    unsigned long *rm = (unsigned long *)
   kcc_cg_regmap;
    if (rm && rm[vreg]) { EMIT(kcc_enc_fmov_d_x(
   reg, rm[vreg])); return; }
    kcc_cg_emit_frame_ldst(reg, kcc_cg_slot_offset
   (vreg) << 3, 2);
}
void kcc_cg_store_vreg_fp(unsigned long vreg,
   unsigned long reg) {
    unsigned long *fp = (unsigned long *)
   kcc_cg_fpregmap;
    if (fp && fp[vreg]) {
        if (reg == fp[vreg]) return;
        EMIT(kcc_enc_fmov_d_d(fp[vreg], reg));
        return;
    }
    unsigned long *rm = (unsigned long *)
   kcc_cg_regmap;
    if (rm && rm[vreg]) { EMIT(kcc_enc_fmov_x_d(rm
   [vreg], reg)); return; }
    kcc_cg_emit_frame_ldst(reg, kcc_cg_slot_offset
   (vreg) << 3, 3);
}

void kcc_cg_callee_transfer(unsigned long restore)
    {
    unsigned long mask = kcc_cg_callee_mask;
    if (mask) {
        unsigned long base = kcc_cg_save_base >>
   3;
        unsigned long k = 0;
        for (unsigned long r = 19; r < 29; r++) {
            if (!((mask >> (r - 19)) & 1))
   continue;
            unsigned long off = (base + k) << 3;
            kcc_cg_emit_frame_ldst(r, off, restore
    ? 0 : 1);
            k++;
        }
    }
    unsigned long fmask = kcc_cg_fp_callee_mask;
    if (!fmask) return;
    unsigned long base = kcc_cg_fp_save_base >> 3;
    unsigned long k = 0;
    for (unsigned long r = 8; r < 16; r++) {
        if (!((fmask >> (r - 8)) & 1)) continue;
        unsigned long off = (base + k) << 3;
        kcc_cg_emit_frame_ldst(r, off, restore ? 2
    : 3);
        k++;
    }
}

unsigned long kcc_cg_marshal_args(unsigned long *
   args, unsigned long count, unsigned long fsi) {
    if (!fsi) fsi = 1;
    unsigned long stack_adj = 0;
    if (!count) return 0;
    unsigned long ngrn = 0, nsrn = 0;
    for (unsigned long a = 0; a < count; a++) {
        unsigned long cls = args[2 + 2 * a];
        unsigned long vreg = args[1 + 2 * a];
        if (cls == 1) {
          /* FP */
            if (nsrn < 8) { kcc_cg_load_vreg_fp(
   vreg, nsrn); nsrn++; }
        } else if (cls == 5) {
          /* x8 sret ptr */
            kcc_cg_load_vreg(vreg, 8);
        } else if (cls == 2) {
          /* ST1 */
            if (ngrn < 8) {
                kcc_cg_load_vreg(vreg, 9);
                EMIT(kcc_enc_ldr_imm(ngrn, 9, 0,
   1));
```

```
                ngrn++;
            }
        } else if (cls == 3) {
          /* ST2 */
            if (ngrn <= 6) {
                kcc_cg_load_vreg(vreg, 9);
                EMIT(kcc_enc_ldr_imm(ngrn, 9, 0,
   1));
                EMIT(kcc_enc_ldr_imm(ngrn + 1, 9,
   1, 1));
                ngrn += 2;
            }
        } else {
          /* GP / REF */
            if (ngrn < 8) { kcc_cg_load_vreg(vreg,
    ngrn); ngrn++; }
        }
    }
    if (count > fsi) {
        stack_adj = (((count - fsi) * 8) + 15) &
   ~15UL;
        EMIT(kcc_enc_sub_imm(31, 31, stack_adj, 1)
   );
        for (unsigned long k = fsi; k < count; k
   ++) {
            kcc_cg_load_vreg(args[1 + 2 * k], 9);
            EMIT(kcc_enc_str_imm(9, 31, k - fsi,
   1));
        }
    }
    return stack_adj;
}
```

□

827b · used in 836a

74.8. The Function Driver

⟨ *Selfhost: cg func* 829 ⟩ ≡

```
void kcc_codegen_func(void *func) {
    unsigned long vreg_n = IL(func, IRF_VREG_N);
    kcc_cg_vreg_count = vreg_n;
    kcc_cg_regalloc(func);
    void *blocks = (void *)IL(func, IRF_BLOCKS);
    unsigned long nbb = IL(blocks, VEC_COUNT);
    /* alloca pre-scan */
    unsigned long off = 0;
    unsigned long *promoted = (unsigned long *)
   kcc_cg_promoted;
    for (unsigned long b = 0; b < nbb; b++) {
        void *bb = *(void **)kcc_vec_get(blocks, b
   );
        for (void *i = (void *)IL(bb, BB_FIRST); i
   ; i = (void *)IL(i, IR_NEXT)) {
            if (IL(i, IR_OP) != IR_ALLOCA)
   continue;
            if (promoted && promoted[IL(i, IR_DEST
   )]) continue;
            IL(i, IR_SRC2) = off;
            unsigned long sc = IL(i, IR_SRC1);
            off += sc ? sc : 1;
        }
    }
    unsigned long total_alloca = off;
    unsigned long max_slot = kcc_cg_max_slot;
    unsigned long save_base = 16 + max_slot * 8 +
   total_alloca * 8;
    kcc_cg_save_base = save_base;
    unsigned long fp_save_base = save_base +
kcc_cg_save_count * 8;
    kcc_cg_fp_save_base = fp_save_base;
    unsigned long frame = fp_save_base +
kcc_cg_fp_save_count * 8;
    frame = (frame + 15) & ~15UL;
    kcc_cg_frame_size = frame;

    unsigned long func_start =
kcc_code_current_pos();
    IL(func, IRF_LOWPC) = func_start;

    /* bb label ht: next pow2 >= 4*bb_count, min
64 */
    unsigned long cap = nbb * 4;
    if (cap <= 64) cap = 64;
    else { unsigned long p = 64; while (p < cap) p
    <<= 1; cap = p; }
    void *bb_labels = kcc_ht_init(cap);
    void *patch = kcc_arena_alloc(VEC_SIZE);
    IL(patch, VEC_ELEMSIZE) = PATCH_SIZE;
    IL(patch, VEC_CAP) = 32;
    IL(patch, VEC_COUNT) = 0;
    IL(patch, VEC_DATA) = (unsigned long)
kcc_arena_alloc(512);

    /* prologue */
    if (frame <= 504) {
        EMIT(kcc_enc_stp_pre(29, 30, 31, (unsigned
)(-(long)frame >> 3)));
    } else {
        kcc_cg_emit_sub_imm(31, 31, frame);
        EMIT(0xA9007BFDu);
          /* stp x29,x30,[sp] */
    }
    EMIT(kcc_enc_add_imm(29, 31, 0, 1));
          /* mov x29, sp */
    kcc_cg_callee_transfer(0);

    /* spill incoming params */
    unsigned long np = IL(func, IRF_PARAMS);
    void *clsarr = (void *)IL(func, IRF_PARAM_CLS)
;
    unsigned long ngrn = 0, nsrn = 0, base = 0;
    for (unsigned long p = 0; p < np; p++) {
        unsigned long cls = 0;
        if (clsarr) cls = ((unsigned long *)clsarr
)[p];
        if (cls == 1) {
          /* FP */
            if (nsrn < 8) { kcc_cg_store_vreg_fp(
base, nsrn); nsrn++; base++; continue; }
            goto stack_param;
        }
        if (cls == 5) {
          /* x8 sret */
            EMIT(kcc_enc_str_imm(8, 29,
kcc_cg_slot_offset(base), 1));
            base++;
            continue;
        }
        if (cls == 3) {
          /* ST2 */
            if (ngrn <= 6) {
                EMIT(kcc_enc_str_imm(ngrn, 29,
kcc_cg_slot_offset(base), 1));
                EMIT(kcc_enc_str_imm(ngrn + 1, 29,
 kcc_cg_slot_offset(base + 1), 1));
                ngrn += 2; base += 2;
                continue;
            }
            goto stack_param;
```

```
        }
        /* GP / ST1 / REF */
        if (ngrn < 8) {
            EMIT(kcc_enc_str_imm(ngrn, 29,
    kcc_cg_slot_offset(base), 1));
            ngrn++; base++;
            continue;
        }
       stack_param:;
        unsigned long idx;
        if (IL(func, IRF_VARIADIC)) idx = base -
    8;
        else idx = p - 1;
        unsigned long boff = idx * 8 + frame;
        kcc_cg_emit_frame_ldst(9, boff, 0);
        kcc_cg_emit_frame_ldst(9,
    kcc_cg_slot_offset(base) << 3, 1);
        base++;
     }

     /* walk BBs */
     for (unsigned long b = 0; b < nbb; b++) {
        void *bb = *(void **)kcc_vec_get(blocks, b
    );
        unsigned long *key = (unsigned long *)
    kcc_arena_alloc(8);
        *key = IL(bb, BB_ID);
        kcc_ht_put(bb_labels, key, 8, (void *)(
    kcc_code_current_pos() + 1));
        for (void *i = (void *)IL(bb, BB_FIRST); i
    ; i = (void *)IL(i, IR_NEXT))
            kcc_codegen_instr(i, bb_labels, patch)
    ;
     }
     kcc_codegen_patch_branches(bb_labels, patch);

     /* patch the function symbol's value */
     unsigned long symidx = IL(func, IRF_SYMIDX);
     char *syms = *(char **)((char *)
    kcc_linker_syms + VEC_DATA);
     IL(syms + symidx * SYM_ENTRY_SIZE, SYM_VALUE)
    = func_start;
}
```

□

829 · used in 836a

74.9. Branch Patching and String Literals

⟨ *Selfhost: cg strings* 830 ⟩ ≡

```
void kcc_codegen_patch_branches(void *bb_labels,
    void *patches) {
     unsigned long n = IL(patches, VEC_COUNT);
     for (unsigned long i = 0; i < n; i++) {
        void *e = kcc_vec_get(patches, i);
        unsigned long branch_off = IL(e, 0);
        unsigned long target_id = IL(e, 8);
        void *v = kcc_ht_get(bb_labels, &target_id
    , 8);
        if (!v) continue;
        unsigned long target = (unsigned long)v -
    1;
        unsigned instr = *(unsigned *)((char *)
    kcc_code_buf + branch_off);
        long delta = (long)target - (long)
    branch_off;
        unsigned top = instr >> 24;
        if (top == 0x14) {
            kcc_code_patch32(branch_off, kcc_enc_b
    (delta));
        } else {
          /* CBNZ */
            unsigned rt = instr & 0x1F;
            unsigned imm19 = (unsigned)((delta >>
    2) & 0x7FFFF);
            unsigned w = 0xB5000000u | (imm19 <<
    5) | rt;
            kcc_code_patch32(branch_off, w);
        }
     }
}

void kcc_cstr_reserve(unsigned long need) {
    if (need <= kcc_cstr_cap) return;
    unsigned long ncap = kcc_cstr_cap << 1;
    if (ncap < need) ncap = need;
    void *nb = kcc_arena_alloc(ncap);
    kcc_memcpy(nb, kcc_cstr_buf, kcc_cstr_pos);
    kcc_cstr_buf = nb;
    kcc_cstr_cap = ncap;
}

/* build "l_.str.N" into the 20-byte buffer */
static void strlit_name(char *b, unsigned long n)
    {
    kcc_memcpy(b, "l_.str.", 7);
    char tmp[20];
    int t = 0;
    if (n == 0) tmp[t++] = '0';
    else while (n) { tmp[t++] = '0' + (n % 10); n
    /= 10; }
    int j = 7;
    while (t) b[j++] = tmp[--t];
    b[j] = 0;
}

unsigned long kcc_emit_cstring(const char *data,
    unsigned long len) {
    unsigned long off = kcc_cstr_pos;
    kcc_cstr_reserve(off + len + 1);
    char *buf = (char *)kcc_cstr_buf;
    if (len) kcc_memcpy(buf + off, data, len);
    buf[off + len] = 0;
    kcc_cstr_pos = off + len + 1;
    unsigned long n = kcc_cstr_count++;
    char *name = (char *)kcc_arena_alloc(20);
    strlit_name(name, n);
    unsigned long sym = kcc_linker_add_sym(name,
    1, 0, N_SECT);
    unsigned long fix[2] = { sym, off };
    kcc_vec_push(kcc_cstr_fixups, fix);
    return sym;
}

void kcc_codegen_string_lit(void *instr) {
    unsigned long data = IL(instr, IR_SRC1);
    unsigned long len = IL(instr, IR_SRC2);
    unsigned long dest = IL(instr, IR_DEST);
    unsigned long off = kcc_cstr_pos;
    kcc_cstr_reserve(off + len + 1);
    char *buf = (char *)kcc_cstr_buf;
    if (len) kcc_memcpy(buf + off, (const void *)
    data, len);
    buf[off + len] = 0;
    kcc_cstr_pos = off + len + 1;
    unsigned long n = kcc_cstr_count++;
    char *name = (char *)kcc_arena_alloc(20);
    strlit_name(name, n);
```

```
    unsigned long sym = kcc_linker_add_sym(name,
   1, 0, N_SECT);
    unsigned long fix[2] = { sym, off };
    kcc_vec_push(kcc_cstr_fixups, fix);
    kcc_linker_add_reloc(kcc_code_current_pos(),
   ARM64_RELOC_PAGE21, sym);
    EMIT(kcc_enc_adrp(CG_REG_DST, 0));
    kcc_linker_add_reloc(kcc_code_current_pos(),
   ARM64_RELOC_PAGEOFF12, sym);
    EMIT(kcc_enc_add_imm(CG_REG_DST, CG_REG_DST,
   0, 1));
    kcc_cg_store_vreg(dest, CG_REG_DST);
}

void kcc_codegen_emit_strings(void) {
    if (!kcc_cstr_pos) return;
    unsigned long code_size = kcc_code_current_pos
   ();
    void *fx = kcc_cstr_fixups;
    unsigned long n = IL(fx, VEC_COUNT);
    char *syms = *(char **)((char *)
   kcc_linker_syms + VEC_DATA);
    for (unsigned long i = 0; i < n; i++) {
        void *e = kcc_vec_get(fx, i);
        unsigned long sym = IL(e, 0), coff = IL(e,
    8);
        IL(syms + sym * SYM_ENTRY_SIZE, SYM_VALUE)
    = code_size + coff;
    }
    unsigned long csz = kcc_cstr_pos;
    unsigned long cp = kcc_code_pos;
    if (cp + csz > kcc_code_cap) {
        unsigned long ncap = (cp + csz) << 1;
        void *nb = kcc_arena_alloc(ncap);
        kcc_memcpy(nb, kcc_code_buf, cp);
        kcc_code_buf = nb;
        kcc_code_cap = ncap;
    }
    kcc_memcpy((char *)kcc_code_buf + cp,
   kcc_cstr_buf, csz);
    kcc_code_pos = cp + csz;
}
```

□

830 · used in 836a

74.10. Per-Opcode Instruction Selection

⟨ *Selfhost: cg dispatch* 831 ⟩ ≡

```
/* helper for the two-source integer ALU shape */
static void binop(void *i, unsigned (*enc)(
   unsigned,unsigned,unsigned,unsigned)) {
    kcc_cg_load_vreg(IL(i, IR_SRC1), CG_REG_SRC1);
    kcc_cg_load_vreg(IL(i, IR_SRC2), CG_REG_SRC2);
    EMIT(enc(CG_REG_DST, CG_REG_SRC1, CG_REG_SRC2,
    1));
    kcc_cg_store_vreg(IL(i, IR_DEST), CG_REG_DST);
}
static unsigned long udiv_sf(void *i) {
    void *ty = (void *)IL(i, IR_TYPE);
    if ((unsigned long)ty < 0x1000) return 1;
    return (IL(ty, TY_SIZE) >= 8) ? 1 : 0;
}

static void cg_normalize_ret(void *ret_type) {
    if (!ret_type) return;
    unsigned long w = kcc_ir_load_width(ret_type);
    if (w == 5) EMIT(0x93401C00u);       /* SXTB
   x0,w0 */
    else if (w == 6) EMIT(0x93403C00u);  /* SXTH
   */
    else if (w == 7) EMIT(0x93407C00u);  /* SXTW
   */
    else if (w == 1) EMIT(0x53001C00u);  /* UXTB
   w0,w0 */
    else if (w == 2) EMIT(0x53003C00u);  /* UXTH
   */
    else if (w == 4) EMIT(0x2A0003E0u);  /* MOV w0
   ,w0 */
}

void kcc_codegen_instr(void *i, void *bb_labels,
   void *patches) {
    unsigned long op = IL(i, IR_OP);
    /* FP arithmetic/comparison dispatch */
    void *ty0 = (void *)IL(i, IR_TYPE);
    if (ty0) {
        unsigned long k = IL(ty0, TY_KIND);
        if (k == TY_FLOAT || k == TY_DOUBLE) {
            if (op == IR_ADD || op == IR_SUB || op
    == IR_MUL || op == IR_SDIV) {
                unsigned long fop = (op == IR_ADD)
    ? 2 : (op == IR_SUB) ? 3 : (op == IR_MUL) ? 0
   : 1;
                kcc_cg_load_vreg_fp(IL(i, IR_SRC1)
   , 0);
                kcc_cg_load_vreg_fp(IL(i, IR_SRC2)
   , 1);
                EMIT(kcc_enc_fp_dp2(2, 0, 1, fop))
   ;
                kcc_cg_store_vreg_fp(IL(i, IR_DEST
   ), 2);
                return;
            }
            if (op >= IR_CMP_EQ && op <=
   IR_CMP_SGE) {
                unsigned long inv;
                if (op == IR_CMP_EQ) inv = 1;
                else if (op == IR_CMP_NE) inv = 0;
                else if (op == IR_CMP_SLT) inv =
   5;
                else if (op == IR_CMP_SLE) inv =
   8;
                else if (op == IR_CMP_SGT) inv =
   13;
                else inv = 11;
                kcc_cg_load_vreg_fp(IL(i, IR_SRC1)
   , 0);
                kcc_cg_load_vreg_fp(IL(i, IR_SRC2)
   , 1);
                EMIT(kcc_enc_fcmp(0, 1));
                EMIT(kcc_enc_cset(CG_REG_DST, inv,
    1));
                kcc_cg_store_vreg(IL(i, IR_DEST),
   CG_REG_DST);
                return;
            }
        }
    }
    switch (op) {
    case IR_NOP: return;
    case IR_CONST: {
        unsigned long v = IL(i, IR_SRC1);
        EMIT(kcc_enc_movz(CG_REG_DST, v & 0xFFFF,
   0, 1));
        if ((v >> 16) & 0xFFFF) EMIT(kcc_enc_movk(
   CG_REG_DST, (v >> 16) & 0xFFFF, 1, 1));
```

```
     if ((v >> 32) & 0xFFFF) EMIT(kcc_enc_movk(
CG_REG_DST, (v >> 32) & 0xFFFF, 2, 1));
     if ((v >> 48) & 0xFFFF) EMIT(kcc_enc_movk(
CG_REG_DST, (v >> 48) & 0xFFFF, 3, 1));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_ADD: binop(i, kcc_enc_add_reg); return
;
 case IR_SUB: binop(i, kcc_enc_sub_reg); return
;
 case IR_MUL: binop(i, kcc_enc_mul); return;
 case IR_SDIV: binop(i, kcc_enc_sdiv); return;
 case IR_AND: binop(i, kcc_enc_and_reg); return
;
 case IR_OR: binop(i, kcc_enc_orr_reg); return;
 case IR_XOR: binop(i, kcc_enc_eor_reg); return
;
 case IR_SHL: binop(i, kcc_enc_lsl_reg); return
;
 case IR_LSHR: binop(i, kcc_enc_lsr_reg);
return;
 case IR_ASHR: binop(i, kcc_enc_asr_reg);
return;
 case IR_GEP: binop(i, kcc_enc_add_reg); return
;
 case IR_UDIV: {
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     kcc_cg_load_vreg(IL(i, IR_SRC2),
CG_REG_SRC2);
     EMIT(kcc_enc_udiv(CG_REG_DST, CG_REG_SRC1,
 CG_REG_SRC2, udiv_sf(i)));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_SMOD: {
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     kcc_cg_load_vreg(IL(i, IR_SRC2),
CG_REG_SRC2);
     EMIT(kcc_enc_sdiv(CG_REG_DST, CG_REG_SRC1,
 CG_REG_SRC2, 1));
     EMIT(kcc_enc_msub(CG_REG_DST, CG_REG_DST,
CG_REG_SRC2, CG_REG_SRC1, 1));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_UMOD: {
     unsigned long sf = udiv_sf(i);
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     kcc_cg_load_vreg(IL(i, IR_SRC2),
CG_REG_SRC2);
     EMIT(kcc_enc_udiv(CG_REG_DST, CG_REG_SRC1,
 CG_REG_SRC2, sf));
     EMIT(kcc_enc_msub(CG_REG_DST, CG_REG_DST,
CG_REG_SRC2, CG_REG_SRC1, sf));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_NEG: {
     void *ty = (void *)IL(i, IR_TYPE);
     if ((unsigned long)ty >= 0x1000) {
         unsigned long k = IL(ty, TY_KIND);
         if (k >= 8 && k <= 10) {
             kcc_cg_load_vreg_fp(IL(i, IR_SRC1)
, 0);
             EMIT(kcc_enc_fneg_d(2, 0));
             kcc_cg_store_vreg_fp(IL(i, IR_DEST
), 2);
             return;
         }
     }
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     EMIT(kcc_enc_sub_reg(CG_REG_DST, 31,
CG_REG_SRC1, 1));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_NOT: {
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     EMIT(kcc_enc_mvn(CG_REG_DST, CG_REG_SRC1,
1));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_CMP_EQ: case IR_CMP_NE: case
IR_CMP_SLT: case IR_CMP_SLE:
 case IR_CMP_SGT: case IR_CMP_SGE: case
IR_CMP_ULT: case IR_CMP_ULE: {
     unsigned long inv;
     switch (op) {
     case IR_CMP_EQ: inv = 1; break; case
IR_CMP_NE: inv = 0; break;
     case IR_CMP_SLT: inv = 10; break; case
IR_CMP_SLE: inv = 12; break;
     case IR_CMP_SGT: inv = 13; break; case
IR_CMP_SGE: inv = 11; break;
     case IR_CMP_ULT: inv = 2; break; default:
inv = 8; break;
     }
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     kcc_cg_load_vreg(IL(i, IR_SRC2),
CG_REG_SRC2);
     void *ty = (void *)IL(i, IR_TYPE);
     unsigned long sf = (!ty || IL(ty, TY_SIZE)
 == 8) ? 1 : 0;
     EMIT(kcc_enc_cmp_reg(CG_REG_SRC1,
CG_REG_SRC2, sf));
     EMIT(kcc_enc_cset(CG_REG_DST, inv, 1));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_TRUNC: {
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     void *ty = (void *)IL(i, IR_TYPE);
     unsigned long sz = ty ? IL(ty, TY_SIZE) :
8;
     if (sz == 1) EMIT(kcc_enc_movz(CG_REG_SRC2
, 0xFF, 0, 1));
     else if (sz == 2) EMIT(kcc_enc_movz(
CG_REG_SRC2, 0xFFFF, 0, 1));
     else if (sz == 4) { EMIT(kcc_enc_movz(
CG_REG_SRC2, 0xFFFF, 0, 1)); EMIT(kcc_enc_movk(
CG_REG_SRC2, 0xFFFF, 1, 1)); }
     else { EMIT(kcc_enc_add_imm(CG_REG_DST,
CG_REG_SRC1, 0, 1)); kcc_cg_store_vreg(IL(i,
IR_DEST), CG_REG_DST); return; }
```

```
     EMIT(kcc_enc_and_reg(CG_REG_DST,
CG_REG_SRC1, CG_REG_SRC2, 1));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_BITCAST: {
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     EMIT(kcc_enc_add_imm(CG_REG_DST,
CG_REG_SRC1, 0, 1));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_SEXT: {
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     unsigned long w = IL(i, IR_SRC2);
     if (w == 1) EMIT(0x93401D2Bu);
     else if (w == 2) EMIT(0x93403D2Bu);
     else if (w == 4) EMIT(0x93407D2Bu);
     else EMIT(kcc_enc_add_imm(CG_REG_DST,
CG_REG_SRC1, 0, 1));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_ZEXT: {
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     unsigned long w = IL(i, IR_SRC2);
     if (w == 1) EMIT(0xD3401D2Bu);
     else if (w == 2) EMIT(0xD3403D2Bu);
     else if (w == 4) EMIT(0xD3407D2Bu);
     else EMIT(kcc_enc_add_imm(CG_REG_DST,
CG_REG_SRC1, 0, 1));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_SITOFP: {
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     EMIT(kcc_enc_scvtf(0, CG_REG_SRC1));
     kcc_cg_store_vreg_fp(IL(i, IR_DEST), 0);
     return;
 }
 case IR_UITOFP: {
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     EMIT(kcc_enc_ucvtf(0, CG_REG_SRC1));
     kcc_cg_store_vreg_fp(IL(i, IR_DEST), 0);
     return;
 }
 case IR_FPTOSI: {
     kcc_cg_load_vreg_fp(IL(i, IR_SRC1), 0);
     EMIT(kcc_enc_fcvtzs(CG_REG_SRC1, 0));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_SRC1);
     return;
 }
 case IR_FPROUND: {
     kcc_cg_load_vreg_fp(IL(i, IR_SRC1), 0);
     EMIT(kcc_enc_fcvt_s_d(0, 0));
     EMIT(kcc_enc_fcvt_d_s(0, 0));
     kcc_cg_store_vreg_fp(IL(i, IR_DEST), 0);
     return;
 }
 case IR_LOAD: {
     unsigned long pr = kcc_cg_promoted_reg(IL(
i, IR_SRC1));
     if (pr) {
         unsigned long w = IL(i, IR_SRC2);
         if (!w) { EMIT(kcc_enc_mov_reg(
CG_REG_DST, pr, 1)); }
         else {
             EMIT(kcc_enc_mov_reg(CG_REG_SRC1,
pr, 1));
             if (w == 1) EMIT(0xD3401D2Bu);
else if (w == 2) EMIT(0xD3403D2Bu);
             else if (w == 4) EMIT(0xD3407D2Bu)
; else if (w == 5) EMIT(0x93401D2Bu);
             else if (w == 6) EMIT(0x93403D2Bu)
; else if (w == 7) EMIT(0x93407D2Bu);
             else EMIT(kcc_enc_mov_reg(
CG_REG_DST, CG_REG_SRC1, 1));
         }
         kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
         return;
     }
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     unsigned long sz = IL(i, IR_SRC2);
     if (sz == 8) {
         EMIT(kcc_enc_ldr_fp_s(0, CG_REG_SRC1,
0));
         EMIT(kcc_enc_fcvt_d_s(0, 0));
         kcc_cg_store_vreg_fp(IL(i, IR_DEST),
0);
         return;
     }
     unsigned long enc_sz;
     if (sz == 1) enc_sz = 2; else if (sz == 2)
 enc_sz = 3; else if (sz == 4) enc_sz = 0;
     else if (sz == 5) enc_sz = 5; else if (sz
== 6) enc_sz = 6; else if (sz == 7) enc_sz = 4;
     else enc_sz = 1;
     EMIT(kcc_enc_ldr_imm(CG_REG_DST,
CG_REG_SRC1, 0, enc_sz));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_STORE: {
     unsigned long pr = kcc_cg_promoted_reg(IL(
i, IR_SRC2));
     if (pr) {
         kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
         EMIT(kcc_enc_mov_reg(pr, CG_REG_SRC1,
1));
         return;
     }
     kcc_cg_load_vreg(IL(i, IR_SRC1),
CG_REG_SRC1);
     kcc_cg_load_vreg(IL(i, IR_SRC2),
CG_REG_SRC2);
     unsigned long sz = IL(i, IR_DEST);
     if (sz == 8) {
         kcc_cg_load_vreg_fp(IL(i, IR_SRC1), 0)
;
         EMIT(kcc_enc_fcvt_s_d(0, 0));
         EMIT(kcc_enc_str_fp_s(0, CG_REG_SRC2,
0));
         return;
     }
     unsigned long enc_sz = (sz == 1) ? 2 : (sz
 == 2) ? 3 : (sz == 4) ? 0 : 1;
```

```
     EMIT(kcc_enc_str_imm(CG_REG_SRC1,
CG_REG_SRC2, 0, enc_sz));
     return;
 }
 case IR_ALLOCA: {
     if (kcc_cg_promoted_reg(IL(i, IR_DEST)))
return;
     unsigned long offv = IL(i, IR_SRC2);
     unsigned long imm = 16 + kcc_cg_max_slot *
 8 + offv * 8;
     kcc_cg_emit_add_imm(CG_REG_DST, 29, imm);
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_VA_START: {
     unsigned long frame = kcc_cg_frame_size;
     long nfx = (long)IL(i, IR_SRC2) - 8;
     if (nfx > 0) frame += (unsigned long)nfx *
 8;
     EMIT(kcc_enc_movz(CG_REG_DST, frame & 0
xffff, 0, 1));
     if ((frame >> 16) & 0xffff) EMIT(
kcc_enc_movk(CG_REG_DST, (frame >> 16) & 0xffff
, 1, 1));
     EMIT(kcc_enc_add_reg(CG_REG_DST, 29,
CG_REG_DST, 1));
     kcc_cg_store_vreg(IL(i, IR_DEST),
CG_REG_DST);
     return;
 }
 case IR_BR: {
     unsigned long tid = IL(i, IR_SRC1);
     void *v = kcc_ht_get(bb_labels, &tid, 8);
     if (v) {
         unsigned long target = (unsigned long)
v - 1;
         EMIT(kcc_enc_b((long)target - (long)
kcc_code_current_pos()));
     } else {
         unsigned long poff =
kcc_code_current_pos();
         EMIT(0x14000000u);
         unsigned long ent[2] = { poff, IL(i,
IR_SRC1) };
         kcc_vec_push(patches, ent);
     }
     return;
 }
 case IR_BR_COND: {
     kcc_cg_load_vreg(IL(i, IR_DEST),
CG_REG_SRC1);
     unsigned long tt = IL(i, IR_SRC1);
     void *v = kcc_ht_get(bb_labels, &tt, 8);
     if (v) {
         unsigned long target = (unsigned long)
v - 1;
         EMIT(kcc_enc_cbnz(CG_REG_SRC1, (long)
target - (long)kcc_code_current_pos(), 1));
     } else {
         unsigned long poff =
kcc_code_current_pos();
         EMIT(0xB5000000u + CG_REG_SRC1);
         unsigned long ent[2] = { poff, IL(i,
IR_SRC1) };
         kcc_vec_push(patches, ent);
     }
     unsigned long tf = IL(i, IR_SRC2);
     void *vf = kcc_ht_get(bb_labels, &tf, 8);
     if (vf) {
         unsigned long target = (unsigned long)
vf - 1;
         EMIT(kcc_enc_b((long)target - (long)
kcc_code_current_pos()));
     } else {
         unsigned long poff =
kcc_code_current_pos();
         EMIT(0x14000000u);
         unsigned long ent[2] = { poff, IL(i,
IR_SRC2) };
         kcc_vec_push(patches, ent);
     }
     return;
 }
 case IR_RET: {
     unsigned long rv = IL(i, IR_SRC1);
     void *ty = (void *)IL(i, IR_TYPE);
     if (ty) {
         unsigned long k = IL(ty, TY_KIND);
         if (k != TY_VOID) {
             if (k == TY_FLOAT || k ==
TY_DOUBLE) kcc_cg_load_vreg_fp(rv, 0);
             else if (k == TY_STRUCT || k ==
TY_UNION) {
                 unsigned long sz = IL(ty,
TY_SIZE);
                 if (sz) {
                     kcc_cg_load_vreg(rv, 9);
                     EMIT(kcc_enc_ldr_imm(0, 9,
 0, 1));
                     if (sz > 8) EMIT(
kcc_enc_ldr_imm(1, 9, 1, 1));
                 }
             } else kcc_cg_load_vreg(rv, 0);
         }
     }
     kcc_cg_callee_transfer(1);
     unsigned long frame = kcc_cg_frame_size;
     if (frame <= 504) EMIT(kcc_enc_ldp_post
(29, 30, 31, frame >> 3));
     else { EMIT(0xA9407BFDu);
kcc_cg_emit_add_imm(31, 31, frame); }
     EMIT(kcc_enc_ret());
     return;
 }
 case IR_CALL: case IR_CALL_IND: {
     int ind = (op == IR_CALL_IND);
     unsigned long callee = IL(i, IR_SRC1);
     unsigned long dest = IL(i, IR_DEST);
     unsigned long *aa = (unsigned long *)IL(i,
 IR_SRC2);
     unsigned long count = 0, fsi = 1;
     if (aa) { count = aa[0] & 0xffffffff; fsi
= aa[0] >> 32; }
     unsigned long adj = kcc_cg_marshal_args(aa
, count, fsi);
     if (ind) {
         kcc_cg_load_vreg(callee, 9);
         EMIT(kcc_enc_blr(9));
     } else {
         unsigned long sym = kcc_linker_extern(
underscore((const char *)callee));
         kcc_linker_add_reloc(
kcc_code_current_pos(), ARM64_RELOC_BRANCH26,
sym);
         EMIT(kcc_enc_bl(0));
     }
     void *ty = (void *)IL(i, IR_TYPE);
     if (!ty) { cg_normalize_ret(0);
kcc_cg_store_vreg(dest, 0); }
```

```
    else {
        unsigned long k = IL(ty, TY_KIND);
        if (k == TY_FLOAT || k == TY_DOUBLE)
kcc_cg_store_vreg_fp(dest, 0);
        else if (k == TY_STRUCT || k ==
TY_UNION) {
            unsigned long sz = IL(ty, TY_SIZE)
;
            if (sz && sz <= 16) {
                kcc_cg_load_vreg(dest, 9);
                EMIT(kcc_enc_str_imm(0, 9, 0,
1));
                if (sz > 8) EMIT(
kcc_enc_str_imm(1, 9, 1, 1));
            }
        } else { cg_normalize_ret(ty);
kcc_cg_store_vreg(dest, 0); }
    }
    if (adj) EMIT(kcc_enc_add_imm(31, 31, adj,
 1));
    return;
 }
 case IR_GLOBAL_REF: {
    const char *name = (const char *)IL(i,
IR_SRC1);
    unsigned long dest = IL(i, IR_DEST);
    char *uname = underscore(name);
    void *defined = kcc_ht_get(
kcc_cg_defined_globals, name, kcc_strlen(name))
;
    if (defined) {
        unsigned long sym =
kcc_linker_global_sym(uname, 8);
        kcc_linker_add_reloc(
kcc_code_current_pos(), ARM64_RELOC_PAGE21, sym
);
        EMIT(kcc_enc_adrp(CG_REG_DST, 0));
        kcc_linker_add_reloc(
kcc_code_current_pos(), ARM64_RELOC_PAGEOFF12,
sym);
        EMIT(kcc_enc_add_imm(CG_REG_DST,
CG_REG_DST, 0, 1));
    } else {
        unsigned long sym = kcc_linker_extern(
uname);
        kcc_linker_add_reloc(
kcc_code_current_pos(),
ARM64_RELOC_GOT_LOAD_PAGE21, sym);
        EMIT(kcc_enc_adrp(CG_REG_DST, 0));
        kcc_linker_add_reloc(
kcc_code_current_pos(),
ARM64_RELOC_GOT_LOAD_PAGEOFF12, sym);
        EMIT(kcc_enc_ldr_imm(CG_REG_DST,
CG_REG_DST, 0, 1));
    }
    kcc_cg_store_vreg(dest, CG_REG_DST);
    return;
 }
 case IR_STRING_LIT: kcc_codegen_string_lit(i);
 return;
 case IR_ASM: {
    void *node = (void *)IL(i, IR_TYPE);
    if (node && IL(node, AST_F2)) {
kcc_cg_asm_extended(node); return; }
    kcc_as_add_snip(kcc_code_current_pos(), (
const char *)IL(i, IR_SRC1), IL(i, IR_SRC2));
    EMIT(kcc_enc_nop());
    return;
 }
 default: return;
    }
}
```

□

831 · used in 836a

74.11. Global Reference to a Defined Symbol, and Extended Asm

The IR_GLOBAL_REF of a locally defined global reserves BSS storage and addresses it directly; the assembly did this by calling kcc_linker_global_sym then emitting a direct ADRP/ADD to a PAGE21/PAGEOFF12 relocation against the freshly minted symbol.

⟨ *Selfhost: cg asm* 835 ⟩ ≡

```
static const char *asm_err = "kcc: error:
   unsupported extended asm (only r/=r/+r/i
   constraints, up to 2 operands)\n";

static int cg_asm_validate(void *node) {
    void *op = (void *)IL(node, AST_F2);
    unsigned long count = 0;
    while (op) {
        count++;
        const char *c = (const char *)IL(op,
   AST_F0);
        char ch = c[0];
        if (ch == '=' || ch == '+') ch = c[1];
        if (ch != 'r' && ch != 'i') { kcc_io_err(
   asm_err); return 1; }
        op = (void *)IL(op, AST_F4);
    }
    if (count > 2) { kcc_io_err(asm_err); return
   1; }
    return 0;
}

/* Build the template with %N / %wN / %xN →
   operand register, %% → %. */
static char *cg_asm_subst(void *node, unsigned
   long *lo) {
    const char *tmpl = (const char *)IL(node,
   AST_F0);
    unsigned long len = IL(node, AST_F1);
    char *out = (char *)kcc_arena_alloc(len * 4 +
   64);
    unsigned long o = 0, in = 0;
    while (in < len) {
        char c = tmpl[in++];
        if (c != '%') { out[o++] = c; continue; }
        if (in >= len) { out[o++] = '%'; break; }
        char n = tmpl[in];
        if (n == '%') { in++; out[o++] = '%';
   continue; }
        char width = 'x';
        if (n == 'w') { width = 'w'; in++; }
        else if (n == 'x') { width = 'x'; in++; }
        unsigned long num = 0;
        while (in < len && tmpl[in] >= '0' && tmpl
   [in] <= '9') { num = num * 10 + (tmpl[in] - '0'
   ); in++; }
        out[o++] = width;
        num += 9;
        if (num >= 10) { out[o++] = '1'; num -=
   10; }
        out[o++] = '0' + num;
    }
```

```
    *lo = o;
    return out;
}

void kcc_cg_asm_extended(void *node) {
    if (cg_asm_validate(node)) return;
    unsigned long k = 0;
    for (void *op = (void *)IL(node, AST_F2); op;
   op = (void *)IL(op, AST_F4), k++) {
        unsigned long v = IL(op, AST_F2) & 0
   xffffffff;
        if (v) kcc_cg_load_vreg(v - 1, k + 9);
    }
    unsigned long off = kcc_code_current_pos();
    unsigned long slen;
    char *sub = cg_asm_subst(node, &slen);
    kcc_as_add_snip(off, sub, slen);
    EMIT(kcc_enc_nop());
    k = 0;
    for (void *op = (void *)IL(node, AST_F2); op;
   op = (void *)IL(op, AST_F4), k++) {
        unsigned long ah = IL(op, AST_F3);
        unsigned long a = ah & 0xffffffff;
        if (!a) continue;
        kcc_cg_load_vreg(a - 1, 8);
        unsigned long size = ah >> 32;
        unsigned long enc_sz = (size == 1) ? 2 : (
   size == 2) ? 3 : (size == 4) ? 0 : 1;
        EMIT(kcc_enc_str_imm(k + 9, 8, 0, enc_sz))
   ;
    }
}
```

□

835 · used in 836a

⟨ *Selfhost: codegen* 836a ⟩ ≡

⟨ *Selfhost: cg preamble* 819 ⟩
⟨ *Selfhost: cg init* 821a ⟩
⟨ *Selfhost: cg rodata* 821b ⟩
⟨ *Selfhost: cg prereg* 823 ⟩
⟨ *Selfhost: cg regalloc* 824 ⟩
⟨ *Selfhost: cg color* 827a ⟩
⟨ *Selfhost: cg frame* 827b ⟩
⟨ *Selfhost: cg func* 829 ⟩
⟨ *Selfhost: cg strings* 830 ⟩
⟨ *Selfhost: cg asm* 835 ⟩
⟨ *Selfhost: cg dispatch* 831 ⟩

□

836a · used in 36n

74.12. Verification

`make selfhost-codegen` runs the strongest oracle: `difftest` byte-compares every generated object against clang, exercising the register allocator, slot colouring, marshalling, and every instruction handler, plus the code-generation and diagnostic suites. A clean sweep means kcc generated its own machine code—down to the last register and slot—identically to the assembly it replaces.

⟨ *Selfhost: codegen verify* 836b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-codegen - compile kcc's ARM64 code
    generator in C with kcc,
# relink, run difftest + diagtest + codegen.
set -euo pipefail
SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "
echo "[1/4] compiling $BASE/selfhost/codegen.c
   with kcc"
$BASE/kcc -c "$SH/codegen.c" -o "$SH/codegen_c.o"
   >/dev/null
echo "[2/4] linking $BASE/kcc_selfhost_codegen (
   codegen.o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "codegen" ]; then OBJ="$OBJ $SH/
   codegen_c.o"; else OBJ="$OBJ $BASE/src/${f}.o";
    fi
done
ld -o $BASE/kcc_selfhost_codegen $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64
cp $BASE/kcc "$SH/kcc_asm_backup_codegen"
cp $BASE/kcc_selfhost_codegen $BASE/kcc
echo "[3/4] difftest (byte-compare) + diagtest"
set +e
bash $BASE/tests/difftest.sh >/$BASE/cg_difftest.
   log 2>&1; dstatus=$?
tail -1 /$BASE/cg_difftest.log
bash $BASE/tests/diagtest.sh >/$BASE/cg_diag.log
   2>&1; diagst=$?
tail -1 /$BASE/cg_diag.log
set -e
if [ $dstatus -ne 0 ] || [ $diagst -ne 0 ]; then
   cp "$SH/kcc_asm_backup_codegen" $BASE/kcc; exit
    1; fi
echo "[4/4] codegen suite with the self-hosted
   code generator"
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_codegen" $BASE/kcc
exit $status
```

□

836b · used in 36o · uses _main 621, 629

75. Self-Hosting: The IR Builder in C

The IR builder of Chapters 28–29 is the compiler's middle-end spine: it walks the typed AST and produces the neutral, three-address IR that both back ends—the ARM64 code generator and the x86-64 emitter—consume. It is by a wide margin the largest module in kcc: the expression lowering alone (pointer scaling, the integer promotions, struct copies, bit-field packing, the AAPCS64 argument classification) fills thousands of lines, and every shape it emits is checked against clang by `difftest`. Porting it leaves *only* the raw-syscall entry stubs (`main`, `syscalls`) in assembly, so with this chapter the compiler that compiles kcc is, but for its bootstrap floor, written in the language it accepts.

Because the IR determines every downstream byte—the register a value lands in, the slot it spills to, the instruction word emitted—this port is verified exactly as the code generator was: `difftest` byte-compares every object kcc produces against clang. A single different virtual-register number or a reordered `emit` would change the machine code and fail the comparison. The C form is dramatically more compact than the assembly it replaces, because each `alloc_instr`/field-store/`emit` sequence that built one IR node collapses to a handful of `IL()` assignments and one `kcc_ir_emit()` call.

75.1. State, Constants, Externs

Every IR node is reached through byte offsets—`IL(p,off)` is the lvalue at `p+off`, the C spelling of `ldr/str [reg,#off]`—so the C port reads and writes exactly the fields the assembly did, at the same displacements. This module is the definition site of the IR builder's global state (the current function and block, the function vector, the locals and label maps, the block-scope journal) and of the module-init flags the code generator later consults.

⟨ *Selfhost: ir preamble* 837 ⟩ ≡

```
/* IR instruction / basic-block / function field
   offsets */
#define IR_OP 0
#define IR_DEST 8
#define IR_SRC1 16
#define IR_SRC2 24
#define IR_TYPE 32
#define IR_NEXT 40
#define IR_INSTR_SIZE 48
#define BB_ID 0
#define BB_FIRST 8
#define BB_LAST 16
#define BB_NEXT 24
#define BB_FLAGS 32
#define IR_BB_SIZE 48
#define BB_FLAG_TERMINATED 1
#define IRF_NAME 0
#define IRF_ENTRY 8
#define IRF_BLOCKS 16
#define IRF_VREG_N 24
#define IRF_RETTYPE 32
#define IRF_PARAMS 40
#define IRF_PARAM_CLS 48
#define IRF_LINE 56
#define IRF_LOWPC 64
#define IRF_STATIC 72
#define IRF_SYMIDX 80
#define IRF_VARIADIC 88
#define IR_FUNC_SIZE 96
/* AST node fields (64-byte node) */
#define AST_TAG 0
#define AST_LINE 8
#define AST_COL 16
#define AST_F0 24
#define AST_F1 32
#define AST_F2 40
#define AST_F3 48
#define AST_F4 56
/* type descriptor fields */
#define TY_KIND 0
#define TY_SIZE 8
#define TY_ALIGN 16
#define TY_QUALS 24
#define TY_F0 32
#define TY_F1 40
#define TY_DESC_SIZE 48
/* symbol-table entry fields */
#define SYM_KIND 0
#define SYM_TYPE 8
#define SYM_NAME 16
#define SYM_VALUE 32
#define SYM_ENUM_CONST 4
#define SYM_FUNC 2
/* vector fields */
#define VEC_DATA 0
#define VEC_COUNT 8
#define VEC_CAP 16
#define VEC_ELEMSIZE 24
#define VEC_SIZE 32

/* type kinds */
enum { TY_VOID=1, TY_BOOL=2, TY_CHAR=3, TY_SHORT
   =4, TY_INT=5, TY_LONG=6,
       TY_LLONG=7, TY_FLOAT=8, TY_DOUBLE=9,
   TY_LDOUBLE=10, TY_PTR=11,
       TY_ARRAY=12, TY_FUNC=13, TY_STRUCT=14,
   TY_UNION=15, TY_ENUM=16,
       TY_INT128=17 };
/* IR opcodes */
enum {
    IR_ADD=1, IR_SUB=2, IR_MUL=3, IR_SDIV=4,
   IR_UDIV=5, IR_SMOD=6, IR_UMOD=7,
    IR_NEG=8, IR_AND=10, IR_OR=11, IR_XOR=12,
   IR_SHL=13, IR_LSHR=14, IR_ASHR=15,
    IR_NOT=16, IR_CMP_EQ=20, IR_CMP_NE=21,
   IR_CMP_SLT=22, IR_CMP_SLE=23,
    IR_CMP_SGT=24, IR_CMP_SGE=25, IR_CMP_ULT=26,
   IR_CMP_ULE=27,
    IR_LOAD=30, IR_STORE=31, IR_ALLOCA=32, IR_GEP
   =33, IR_BR=40, IR_BR_COND=41,
    IR_RET=42, IR_CALL=43, IR_CALL_IND=44, IR_SEXT
   =50, IR_ZEXT=51, IR_TRUNC=52,
    IR_BITCAST=53, IR_SITOFP=54, IR_FPTOSI=55,
   IR_VA_START=56, IR_FPROUND=57,
    IR_UITOFP=58, IR_CONST=60, IR_NOP=61,
   IR_GLOBAL_REF=63, IR_STRING_LIT=64,
    IR_ASM=65, IR_LADDR=66, IR_BR_IND=67,
   IR_FRAME_ADDR=68, IR_RET_ADDR=69
};
/* AST tags */
enum {
    AST_TRANS_UNIT=1, AST_VAR_DECL=10,
   AST_FUNC_DECL=11, AST_PARAM_DECL=12,
    AST_STRUCT_DECL=14, AST_COMPOUND=30,
   AST_EXPR_STMT=31, AST_IF_STMT=32,
```

```
    AST_SWITCH_STMT=33, AST_WHILE_STMT=34,
   AST_DO_STMT=35, AST_FOR_STMT=36,
    AST_GOTO_STMT=37, AST_CONTINUE_STMT=38,
   AST_BREAK_STMT=39, AST_RETURN_STMT=40,
    AST_LABEL_STMT=41, AST_CASE_STMT=42,
   AST_DEFAULT_STMT=43, AST_NULL_STMT=44,
    AST_IDENT_EXPR=50, AST_INT_LIT=51, AST_STR_LIT
   =52, AST_CHAR_LIT=53,
    AST_BINARY_EXPR=54, AST_UNARY_EXPR=55,
   AST_ASSIGN_EXPR=56, AST_COND_EXPR=57,
    AST_CALL_EXPR=58, AST_SUBSCRIPT=59, AST_MEMBER
   =60, AST_CAST_EXPR=61,
    AST_SIZEOF_EXPR=62, AST_ALIGNOF_EXPR=63,
   AST_COMMA_EXPR=64, AST_INIT_LIST=65,
    AST_DESIGNATOR=66, AST_COMPOUND_LIT=67,
   AST_PAREN_EXPR=68, AST_GENERIC=69,
    AST_FLOAT_LIT=71, AST_VA_ARG=72, AST_STMT_EXPR
   =73, AST_ASM_STMT=74,
    AST_TYPES_COMPAT=76, AST_LABEL_ADDR=77,
    AST_TYPE_PTR=22, AST_TYPE_ARRAY=23,
   AST_TYPE_FUNC=24, AST_TYPE_QUALS=28
};
/* token kinds (operators) */
enum {
    TOK_INC=70, TOK_DEC=71, TOK_AMP=72, TOK_STAR
   =73, TOK_PLUS=74, TOK_MINUS=75,
    TOK_TILDE=76, TOK_BANG=77, TOK_SLASH=78,
   TOK_PERCENT=79, TOK_SHL=80,
    TOK_SHR=81, TOK_LT=82, TOK_GT=83, TOK_LE=84,
   TOK_GE=85, TOK_EQ=86, TOK_NE=87,
    TOK_CARET=88, TOK_PIPE=89, TOK_AND=90, TOK_OR
   =91, TOK_ASSIGN=94,
    TOK_MUL_ASSIGN=95, TOK_DIV_ASSIGN=96,
   TOK_MOD_ASSIGN=97, TOK_ADD_ASSIGN=98,
    TOK_SUB_ASSIGN=99, TOK_SHL_ASSIGN=100,
   TOK_SHR_ASSIGN=101, TOK_AND_ASSIGN=102,
    TOK_XOR_ASSIGN=103, TOK_OR_ASSIGN=104
};

/* arena / string / io / hash / vec / sema / types
    --- other modules */
void *kcc_arena_alloc(unsigned long);
void *kcc_strtab_intern(const char *, unsigned
   long);
static void *kcc_always_inline_fns = 0;   /*
   registry of always_inline definitions */
static void *kcc_inl_need_out = 0;        /*
   always_inline fns a call could NOT inline */
unsigned long kcc_strlen(const char *);
int kcc_strcmp(const char *, const char *);
void *kcc_ht_init(unsigned long);
void kcc_ht_put(void *, const void *, unsigned
   long, void *);
void *kcc_ht_get(void *, const void *, unsigned
   long);
void *kcc_vec_get(void *, unsigned long);
void kcc_vec_push(void *, const void *);
void *kcc_ast_alloc(unsigned long tag);
void *kcc_sema_resolve_type(void *);
void *kcc_sema_expr(void *);
unsigned long kcc_type_compatible(void *a, void *b
   );
unsigned long kcc_sema_int_signedness(void *);
unsigned long kcc_type_layout(void *);
void *kcc_type_param_adjust(void *);
unsigned long kcc_type_layout_size(void *);
void *kcc_conv_promote(void *);
void *kcc_conv_usual(void *, void *);
void *kcc_symtab_lookup(const char *, unsigned
   long);
void kcc_io_putc(unsigned long);
void kcc_io_puts(const char *);
void kcc_io_putu(unsigned long);
void kcc_io_flush(void);
/* kcc_type_find_member returns {field, offset} in
    x0:x1 (<=16B struct) */
struct kcc_member_res { void *field; unsigned long
    off; };
struct kcc_member_res kcc_type_find_member(void *
   sty, const char *name);
/* type singletons (pointers to descriptors)
   defined by the types module */
extern void *kcc_type_int_s, *kcc_type_long_s, *
   kcc_type_long_u,
            *kcc_type_double, *kcc_type_float, *
   kcc_type_void;
extern void *kcc_type_int128_u;      /* unsigned
   __int128 (16-byte) singleton */
extern unsigned long kcc_target;     /* 1 when
   compiling for x86-64 (System V va_list) */

/* -- IR builder global state (this module is its
   definition site) -- */
void *kcc_ir_cur_func = 0;
void *kcc_ir_cur_bb = 0;
void *kcc_ir_funcs = 0;
void *kcc_ir_locals = 0;
void *kcc_ir_local_consts = 0;
void *kcc_ir_statics = 0;
void *kcc_ir_static_types = 0;
void *kcc_ir_static_decls = 0;             /*
   every static-local decl, for storage */
unsigned long kcc_tu_salt = 0;             /* per-
   file salt, mixed into static names */
void *kcc_ir_labels = 0;
void *kcc_ir_loop_stack = 0;
unsigned long kcc_ir_bb_counter = 0;
unsigned long kcc_ir_scope_jdepth = 0;
unsigned long kcc_ir_scope_sp = 0;
unsigned long kcc_modinit_present = 0;
unsigned long kcc_modinit_sym = 0;
unsigned long kcc_modinit_fileoff = 0;
void *kcc_ir_trans_unit = 0;
void *kcc_ir_member_field = 0;
void *kcc_ir_vol_longu = 0;
unsigned long kcc_loop_depth = 0;
unsigned long kcc_loop_stack[32];        /* 16
   levels * 2 longs */
char kcc_str_modinit[] = "kcc.module_init";
/* block-scope shadowing journal (name,keylen,prev
    triples) and scope stack */
unsigned long kcc_ir_scope_journal[16384*3];
unsigned long kcc_ir_scope_stack[1024];
/* switch lowering scratch (defined in the control
    chapter's scan) */
unsigned long kcc_sw_cond_vreg = 0;
void *kcc_sw_default_bb = 0;

/* byte-offset field accessor: the C spelling of
   ldr/str [p, #off] */
#define IL(p,off) (*(unsigned long *)((char *)(p)
   + (off)))

/* -- forward declarations (functions are mutually
    recursive) -- */
void *kcc_ir_alloc_instr(unsigned long op);
void *kcc_ir_alloc_bb(void);
void *kcc_ir_alloc_func(const char *name, void *
   ret_type);
void kcc_ir_emit(void *instr);
void kcc_ir_emit_entry(void *instr);
unsigned long kcc_ir_next_vreg(void);
void kcc_ir_set_bb(void *bb);
void kcc_ir_init(void);
void kcc_ir_build(void *tu);
```

```
void kcc_ir_build_func(void *fd);
void kcc_ir_build_param(void *pd, unsigned long
    vreg_base);
void kcc_ir_build_stmt(void *stmt);
void kcc_ir_build_return(void *rs);
unsigned long kcc_ir_build_expr(void *e);
unsigned long kcc_ir_build_stmt_expr(void *se);
unsigned long kcc_ir_build_int_lit(void *lit);
unsigned long kcc_ir_build_float_lit(void *lit);
unsigned long kcc_ir_build_string_lit(void *lit);
unsigned long kcc_ir_build_sizeof(void *e);
unsigned long kcc_ir_build_alignof(void *e);
unsigned long kcc_ir_build_ident(void *e);
unsigned long kcc_ir_build_binary(void *e);
unsigned long kcc_ir_build_unary(void *e);
unsigned long kcc_ir_build_inc_dec(void *e);
unsigned long kcc_ir_build_assign(void *e);
unsigned long kcc_ir_build_call(void *e);
unsigned long kcc_ir_build_cast(void *e);
unsigned long kcc_ir_build_cond_expr(void *e);
unsigned long kcc_ir_build_subscript(void *e);
unsigned long kcc_ir_build_subscript_addr(void *e)
    ;
unsigned long kcc_ir_build_member(void *e);
unsigned long kcc_ir_build_member_addr(void *e);
unsigned long kcc_ir_build_struct_addr(void *e);
unsigned long kcc_ir_build_va_arg(void *e);
unsigned long kcc_ir_build_va_start(void *e);
unsigned long kcc_ir_build_va_copy(void *e);
unsigned long kcc_ir_build_compound_lit(void *e);
void kcc_ir_build_if(void *s);
void kcc_ir_build_while(void *s);
void kcc_ir_build_do(void *s);
void kcc_ir_build_for(void *s);
void kcc_ir_build_switch(void *s);
void kcc_ir_build_break(void *s);
void kcc_ir_build_continue(void *s);
void kcc_ir_build_var_decl(void *s);
void kcc_ir_branch_to(void *target);
void *kcc_ir_label_bb(const char *name);
void kcc_ir_prescan_labels(void *stmt);
void kcc_ir_locals_put(const char *name, unsigned
    long keylen, unsigned long value);
void kcc_ir_local_const_set(const char *name,
    unsigned long keylen, unsigned long lv,
    unsigned long value);
void kcc_ir_local_const_kill(const char *name);
void kcc_ir_scope_enter(void);
void kcc_ir_scope_leave(void);
void kcc_ir_asm_addr_of(void *lvalue);
void kcc_ir_asm_lower_operands(void *as);
void kcc_ir_emit_global_inits(void);
static int kcc_ir_section_discarded(const char *
    name);
void kcc_ir_emit_modinit(void);
void kcc_ir_emit_aggregate_init(void *init_list,
    void *agg, unsigned long base);
void kcc_ir_emit_init_stores(void *decl, unsigned
    long base_vreg);
unsigned long kcc_ir_type_slots(void *ty);
unsigned long kcc_ir_type_is_fp(void *ty);
unsigned long kcc_ir_classify_type(void *ty);
unsigned long kcc_ir_store_width(void *ty);
unsigned long kcc_ir_load_width(void *ty);
unsigned long kcc_ir_emit_sitofp(unsigned long v);
unsigned long kcc_ir_emit_uitofp(unsigned long v);
unsigned long kcc_ir_emit_int_to_fp(unsigned long
    v, void *it);
unsigned long kcc_ir_emit_fpround(unsigned long v)
    ;
unsigned long kcc_ir_emit_fptosi(unsigned long v);
unsigned long kcc_ir_emit_bool_norm(unsigned long
    v);
unsigned long kcc_ir_convert(unsigned long v, void
     *from, void *to);
void kcc_ir_fill_param_classes(void *fn, void *fd)
    ;
unsigned long kcc_ir_emit_const(unsigned long
    value);
unsigned long kcc_ir_emit_load(unsigned long addr,
     unsigned long size);
unsigned long kcc_ir_emit_load_t(unsigned long
    addr, unsigned long size, void *ty);
void kcc_ir_emit_store(unsigned long value,
    unsigned long addr, unsigned long size);
void kcc_ir_emit_store_t(unsigned long value,
    unsigned long addr, unsigned long size, void *
    ty);
void kcc_ir_emit_indexed_store(unsigned long base,
     unsigned long off, unsigned long value,
    unsigned long size);
unsigned long kcc_ir_emit_gep_const(unsigned long
    base, unsigned long off);
void kcc_ir_emit_struct_copy(unsigned long src,
    unsigned long dst, unsigned long size);
void kcc_ir_emit_string_init(unsigned long base,
    const char *sp, unsigned long slen, unsigned
    long alen, unsigned long zerofill);
unsigned long kcc_ir_emit_binop(unsigned long op,
    unsigned long a, unsigned long b);
unsigned long kcc_ir_emit_binop_typed(unsigned
    long op, unsigned long a, unsigned long b, void
     *ty);
unsigned long kcc_ir_tok_to_binop(unsigned long
    tok);
unsigned long kcc_ir_cmp_unsigned(void *lhs_node,
    void *rhs_node);
unsigned long kcc_ir_ident_alloca(void *ident);
const char *kcc_ir_static_name(void *decl);
unsigned long kcc_ir_assign_through(unsigned long
    addr, unsigned long rhs, unsigned long op_tok,
    void *field, void *ltype, void *rtype);
unsigned long kcc_ir_bitfield_load(unsigned long
    addr, void *field);
unsigned long kcc_ir_bitfield_store(unsigned long
    addr, unsigned long value, void *field);
void *kcc_ir_bf_container_type(void *field_type);
unsigned long kcc_ir_bf_trunc_value(unsigned long
    value, void *field);
unsigned long kcc_ir_compound_apply(unsigned long
    op, unsigned long old, void *L, unsigned long
    rhs, void *R);
unsigned long kcc_ir_bf_code(unsigned long
    container_bytes);
unsigned long kcc_ir_bf_mask(unsigned long width);
unsigned long kcc_ir_norm32(unsigned long value,
    unsigned long op, void *ltype, void *rtype);
void kcc_ir_fill_list(void *ty, unsigned long base
    , unsigned long base_off, void *head);
void *kcc_ir_fill(void *ty, unsigned long base,
    unsigned long off, void *elem);
void kcc_ir_fill_one(void *ty, unsigned long base,
     unsigned long off, void *node, void *field);
void kcc_ir_store_elem(unsigned long base,
    unsigned long off, unsigned long value,
    unsigned long width);
long kcc_ir_field_index(void *agg, const char *
    name);
unsigned long kcc_ir_type_has_member(void *ty,
    const char *name);
```

```
long kcc_ir_anon_containing(void *agg, const char
   *name);
struct desig_res { unsigned long off; void *
   sub_type; unsigned long first_pos; void *field;
    };
struct desig_res kcc_ir_resolve_designator(void *
   agg, unsigned long base_off, void *desig);
void *kcc_ir_subobj(void *agg, unsigned long idx,
   unsigned long *out_off);
void kcc_ir_elem_off_width(void *ty, unsigned long
    idx, unsigned long *out_off, unsigned long *
   out_width);
unsigned long kcc_ir_agg_count(void *ty);
unsigned long kcc_ir_initlist_count(void *list);
void kcc_loop_push(unsigned long break_id,
   unsigned long continue_id);
void kcc_loop_pop(void);
void kcc_ir_sw_scan(void *stmt);
void kcc_ir_sw_disp(void *stmt);
unsigned long kcc_eval_const_int(void *n);
unsigned long kcc_expr_is_ice(void *n);
unsigned long kcc_call_is_bcp(void *n);
unsigned long kcc_call_is_choose_expr(void *n);
unsigned long kcc_eval_addr(void *n);
unsigned long kcc_addr_is_iconst(void *n);
unsigned long kcc_rodata_binop_ok(unsigned long op
   );
unsigned long kcc_rodata_const_intexpr(void *n);
unsigned long kcc_rodata_is_lit(void *n, unsigned
   long isfloat);
unsigned long kcc_rodata_struct_ok(void *sty, void
    *init);
unsigned long kcc_rodata_aos_ok(void *sty, void *
   outer);
unsigned long kcc_global_const_rodata(void *decl);
unsigned long kcc_global_const_ptrarr(void *decl);
unsigned long kcc_global_writable(void *decl);
unsigned long kcc_global_sym_data(void *decl);
unsigned long kcc_static_leaf_ok(void *e);
unsigned long kcc_static_init_ok(void *ty, void *
   init);
unsigned long kcc_global_static_ok(void *decl);
const char *kcc_ir_const_symaddr(void *n, long *
   off);
void kcc_ir_dump(void);
void kcc_ir_dump_instr(void *instr);
void kcc_ir_dump_type_short(void *ty);
```

□

837 · used in 902a

75.2. Initialisation and Node Allocation

The IR module's global state is cleared per translation unit, and the three node kinds—instruction, basic block, function—are arena-allocated and zero-filled. These mirror §28 field for field.

⟨ *Selfhost: ir design* 840 ⟩ ≡

```
void kcc_ir_init(void) {
    kcc_ir_cur_func = 0;
    kcc_ir_cur_bb = 0;
    /* clear per-TU module-init state so a later
   unit does not inherit a
       stale flag and emit a stray __mod_init_func
    pointer */
    kcc_modinit_present = 0;
    kcc_modinit_sym = 0;
    kcc_modinit_fileoff = 0;
    /* function list vector: 8-byte pointer
   elements, cap 16 */
    void *v = kcc_arena_alloc(VEC_SIZE);
    kcc_ir_funcs = v;
    IL(v, VEC_ELEMSIZE) = 8;
    IL(v, VEC_CAP) = 16;
    IL(v, VEC_COUNT) = 0;
    IL(v, VEC_DATA) = (unsigned long)
   kcc_arena_alloc(128);
    kcc_ir_locals = kcc_ht_init(64);
    kcc_ir_local_consts = kcc_ht_init(64);
    /* break/continue stack: pairs of pointers, 16
    bytes each */
    v = kcc_arena_alloc(VEC_SIZE);
    kcc_ir_loop_stack = v;
    IL(v, VEC_ELEMSIZE) = 16;
    IL(v, VEC_CAP) = 8;
    IL(v, VEC_COUNT) = 0;
    IL(v, VEC_DATA) = (unsigned long)
   kcc_arena_alloc(128);
    /* static-local decl list: 8-byte pointer
   elements.  Every static local the
       IR builder registers is recorded here so
   the back end can emit storage
       for it, including statics hidden inside
   statement-expressions that no
       statement-spine walk would reach. */
    v = kcc_arena_alloc(VEC_SIZE);
    kcc_ir_static_decls = v;
    IL(v, VEC_ELEMSIZE) = 8;
    IL(v, VEC_CAP) = 16;
    IL(v, VEC_COUNT) = 0;
    IL(v, VEC_DATA) = (unsigned long)
   kcc_arena_alloc(128);
    kcc_ir_bb_counter = 0;
}

void *kcc_ir_alloc_instr(unsigned long op) {
    void *i = kcc_arena_alloc(IR_INSTR_SIZE);
    IL(i, IR_OP) = op;
    IL(i, IR_DEST) = 0;
    IL(i, IR_SRC1) = 0;
    IL(i, IR_SRC2) = 0;
    IL(i, IR_TYPE) = 0;
    IL(i, IR_NEXT) = 0;
    return i;
}

void *kcc_ir_alloc_bb(void) {
    void *b = kcc_arena_alloc(IR_BB_SIZE);
    IL(b, BB_ID) = kcc_ir_bb_counter;
    kcc_ir_bb_counter = kcc_ir_bb_counter + 1;
    IL(b, BB_FIRST) = 0;
    IL(b, BB_LAST) = 0;
    IL(b, BB_NEXT) = 0;
    IL(b, BB_FLAGS) = 0;
    return b;
}

void *kcc_ir_alloc_func(const char *name, void *
   ret_type) {
    void *f = kcc_arena_alloc(IR_FUNC_SIZE);
    IL(f, IRF_NAME) = (unsigned long)name;
    IL(f, IRF_RETTYPE) = (unsigned long)ret_type;
    IL(f, IRF_ENTRY) = 0;
    IL(f, IRF_VREG_N) = 0;
    IL(f, IRF_PARAMS) = 0;
    IL(f, IRF_PARAM_CLS) = 0;
    IL(f, IRF_LINE) = 0;
    IL(f, IRF_LOWPC) = 0;
```

```
    IL(f, IRF_STATIC) = 0;
    IL(f, IRF_SYMIDX) = 0;
    void *v = kcc_arena_alloc(VEC_SIZE);
    IL(f, IRF_BLOCKS) = (unsigned long)v;
    IL(v, VEC_ELEMSIZE) = 8;
    IL(v, VEC_CAP) = 16;
    IL(v, VEC_COUNT) = 0;
    IL(v, VEC_DATA) = (unsigned long)
   kcc_arena_alloc(128);
    return f;
}

void kcc_ir_emit(void *instr) {
    void *bb = kcc_ir_cur_bb;
    if (!bb) return;
    if (IL(bb, BB_FLAGS) & BB_FLAG_TERMINATED)
   return;
    void *last = (void *)IL(bb, BB_LAST);
    if (last) {
        IL(last, IR_NEXT) = (unsigned long)instr;
        IL(bb, BB_LAST) = (unsigned long)instr;
    } else {
        IL(bb, BB_FIRST) = (unsigned long)instr;
        IL(bb, BB_LAST) = (unsigned long)instr;
    }
}

void kcc_ir_emit_entry(void *instr) {
    void *fn = kcc_ir_cur_func;
    if (!fn) return;
    void *bb = (void *)IL(fn, IRF_ENTRY);
    if (!bb) return;
    IL(instr, IR_NEXT) = IL(bb, BB_FIRST);
    IL(bb, BB_FIRST) = (unsigned long)instr;
    if (!IL(bb, BB_LAST)) IL(bb, BB_LAST) = (
   unsigned long)instr;
}

unsigned long kcc_ir_next_vreg(void) {
    void *fn = kcc_ir_cur_func;
    if (!fn) return 0;
    unsigned long n = IL(fn, IRF_VREG_N);
    IL(fn, IRF_VREG_N) = n + 1;
    return n;
}

void kcc_ir_set_bb(void *bb) {
    kcc_ir_cur_bb = bb;
    void *fn = kcc_ir_cur_func;
    if (!fn) return;
    void *v = (void *)IL(fn, IRF_BLOCKS);
    kcc_vec_push(v, &bb);
}
```

□

840 · used in 902a

75.3. The Builder Spine

The spine holds the read-only-globals predicates (which decide whether a `const` global can live as a fixed byte image), the global-initializer emission that seeds file-scope and `static`-local storage at startup, the per-function driver, parameter lowering, the label/goto helpers, the block-scope shadowing journal, the statement dispatcher, and `return`.

One object never gets a run-time initializer: a global whose `section` attribute names a *discarded* section (prefix `.discard`). The kernel throws those away in its linker script, so an initializer lowered into the module-init function would leave a live relocation from the kept `.text` into a section that vanishes. This is the `__ADDRESSABLE(sym)` idiom—a `.discard.addressable` pointer set to `&sym`, used to keep a symbol referenced; gcc emits it as a data relocation in the discarded section with no code, and `kcc_ir_section_discarded` makes the builder skip the initializer (and the module-init trigger) so no such dangling `.text` reference is forged.

Alongside the shadowing journal the spine keeps a second, x86-only table of *const-initialised locals*. A kernel `WARN`/`BUG` entry passes its flags word to an `"i"` (immediate) inline-asm operand, but through a macro-local variable: `__auto_type __flags = BUGFLAG_WARNING(flags);` then `"i"(__flags)`. GCC constant-propagates `__flags` into the operand under optimisation; kcc does not run general constant propagation, so without help it materialises the variable in a register and emits a nonsensical `.word %r8` (and a backwards `.org`). The fix records, at each scalar integer declaration whose initialiser is an integer constant expression, a box `{lv, value}` keyed by the variable's name—where `lv` is its `kcc_ir_locals` handle (`base+1`, unique per variable instance and reset per function). An `"i"` operand naming such a local then folds to its recorded value. Keying the match on `lv` rather than the name alone keeps it sound: a stale box left by a departed inner scope, or one from a prior function, carries a different `lv` and never matches a live local of the same name. Any later assignment, address-of, or `++`/`--` of the variable disables its box (the value is no longer a manifest constant at a subsequent asm use)—and a well-formed `"i"` operand can never name a variable modified before that use anyway, so the textual-order kill is enough.

A constant that reaches an inline-asm operand is emitted immediately only when it fits: an x86 ALU instruction encodes at most a sign-extended 32-bit immediate, so when the folded value overflows that field and the constraint also admits a register (the `"re"` of the percpu add, whose `PERCPU_COUNT_BIAS` is 2^{63}), the operand is handed to the register path instead and loaded with `movabsq`, matching GCC's choice of the register alternative.

⟨ *Selfhost: ir spine* 841 ⟩ ≡

```
/* -- entry point: walk the translation unit,
   build every function body -- */
/* True if an AST subtree (statements AND
   expressions) contains inline asm.  The
   recursion touches only the child-POINTER fields
    of each node---the exact set
   kcc_inl_clone walks---so it never dereferences
   an operator token or literal
   value.  Reaching into expressions matters
   because the kernel hides asm inside
   statement-expressions: `arch_atomic_add_return'
    is `return i + xadd(...)' and
   `xadd' expands to `({ ...; asm volatile(...);
   ...; })'.  Such a function must
   NOT be emitted out-of-line---its asm needs the
   caller's operands---so a
   statement-spine-only check would miss the asm
   and crash codegen on it. */
static int kcc_ast_has_asm(void *n);
static int kcc_ast_has_asm_seq(void *s) {
```

```
    for (; s; s = (void *)IL(s, AST_F4))
        if (kcc_ast_has_asm(s)) return 1;
    return 0;
}
static int kcc_ast_has_asm(void *n) {
    if (!n) return 0;
    unsigned long tag = IL(n, AST_TAG);
    switch (tag) {
    case AST_ASM_STMT: return 1;
    case AST_COMPOUND: return kcc_ast_has_asm_seq
   ((void *)IL(n, AST_F0));
    case AST_CALL_EXPR:
        return kcc_ast_has_asm((void *)IL(n,
   AST_F0)) ||
               kcc_ast_has_asm_seq((void *)IL(n,
   AST_F1));
    case AST_PAREN_EXPR: case AST_EXPR_STMT: case
   AST_STMT_EXPR:
    case AST_MEMBER: case AST_RETURN_STMT:
    case AST_DEFAULT_STMT:
   /* body in F0 (kcc_parse_default) */
        return kcc_ast_has_asm((void *)IL(n,
   AST_F0));
    case AST_UNARY_EXPR: case AST_CAST_EXPR:
    case AST_LABEL_STMT:
        return kcc_ast_has_asm((void *)IL(n,
   AST_F1));
    case AST_VAR_DECL:
        return kcc_ast_has_asm((void *)IL(n,
   AST_F2));
    case AST_BINARY_EXPR: case AST_ASSIGN_EXPR:
   case AST_COMMA_EXPR:
        return kcc_ast_has_asm((void *)IL(n,
   AST_F1)) ||
               kcc_ast_has_asm((void *)IL(n,
   AST_F2));
    case AST_SUBSCRIPT: case AST_WHILE_STMT: case
   AST_DO_STMT:
    case AST_SWITCH_STMT: case AST_CASE_STMT:
        return kcc_ast_has_asm((void *)IL(n,
   AST_F0)) ||
               kcc_ast_has_asm((void *)IL(n,
   AST_F1));
    case AST_COND_EXPR: case AST_IF_STMT:
        return kcc_ast_has_asm((void *)IL(n,
   AST_F0)) ||
               kcc_ast_has_asm((void *)IL(n,
   AST_F1)) ||
               kcc_ast_has_asm((void *)IL(n,
   AST_F2));
    case AST_FOR_STMT:
        return kcc_ast_has_asm((void *)IL(n,
   AST_F0)) ||
               kcc_ast_has_asm((void *)IL(n,
   AST_F1)) ||
               kcc_ast_has_asm((void *)IL(n,
   AST_F2)) ||
               kcc_ast_has_asm((void *)IL(n,
   AST_F3));
    case AST_SIZEOF_EXPR: case AST_ALIGNOF_EXPR:
        return IL(n, AST_F1) ? 0 : kcc_ast_has_asm
   ((void *)IL(n, AST_F0));
    default: return 0;              /* leaves:
   identifiers, literals, goto, ... */
    }
}

void kcc_ir_build(void *tu) {
    kcc_ir_trans_unit = tu;
    kcc_inl_need_out = 0;
    void *d = (void *)IL(tu, AST_F0);
    while (d) {
        if (IL(d, AST_TAG) == AST_FUNC_DECL && IL(
d, AST_F3)) {
            /* An always_inline function is
expanded at each call, so it is not
            emitted standalone---which also
keeps its body AST pristine, as a
            standalone build lowers the asm
operands in place and inlining
            needs the original operand
expressions to fold constants. */
            const char *nm = (const char *)IL(d,
AST_F1);
            if (!(kcc_always_inline_fns && nm &&
                  kcc_ht_get(kcc_always_inline_fns
, nm, kcc_strlen(nm))))
                kcc_ir_build_func(d);
        }
        d = (void *)IL(d, AST_F4);
    }
    /* decide whether this TU needs a module-init
function: no main, but at
       least one file-scope or static-local
initializer to run at startup */
    unsigned long has_main = 0, has_ginit = 0;
    d = (void *)IL(tu, AST_F0);
    while (d) {
        unsigned long tag = IL(d, AST_TAG);
        if (tag == AST_FUNC_DECL) {
            void *body = (void *)IL(d, AST_F3);
            if (body) {
                void *s = (void *)IL(body, AST_F0)
;
                while (s) {
                    if (IL(s, AST_TAG) ==
AST_VAR_DECL && IL(s, AST_F3) == 3 && IL(s,
AST_F2)) {
                        has_ginit = 1;
                        break;
                    }
                    s = (void *)IL(s, AST_F4);
                }
                void *name = (void *)IL(d, AST_F1)
;
                if (name && kcc_strcmp((const char
 *)name, "main") == 0) has_main = 1;
            }
        } else if (tag == AST_VAR_DECL) {
            if (IL(d, AST_F2) && !
kcc_ir_section_discarded((const char *)IL(d,
AST_F1)))
                has_ginit = 1;
        }
        d = (void *)IL(d, AST_F4);
    }
    if (!has_main && has_ginit)
kcc_ir_emit_modinit();
    /* Out-of-line pass: emit a standalone body
for any always_inline function a
       call could not inline, or whose address was
    taken as a value (asm-bearing
       ones excluded---their `"i"' operands need
the caller's constant). Runs last
       so in-place lowering can't disturb an
inline expansion.  Emission is
       TRANSITIVE---building one out-of-line body
may itself call another
       always_inline that could not inline (the
kernel reaches folio_test_anon
       only through such a chain)---so iterate to
a fixpoint, tracking the ones
       already emitted, until a full sweep adds
nothing new. */
```

```
    if (kcc_inl_need_out) {
        void *ool_done = kcc_ht_init(32);
        unsigned long progress = 1;
        while (progress) {
            progress = 0;
            d = (void *)IL(tu, AST_F0);
            while (d) {
                if (IL(d, AST_TAG) ==
   AST_FUNC_DECL && IL(d, AST_F3)) {
                    const char *nm = (const char
   *)IL(d, AST_F1);
                    unsigned long nl = nm ?
   kcc_strlen(nm) : 0;
                    /* Only a STATIC (internal-
   linkage) always_inline is safe to
                       emit out-of-line: its copy
   is local so multiple TUs each
                       carry their own without
   colliding.  An EXTERNAL gnu-inline
                       already has its one copy in
    a designated unit; emitting
                       kcc's own would be a `
   multiple definition' at the link. */
                    if (nm && (IL(d, AST_F2) & 2)
   &&
                        kcc_ht_get(
   kcc_inl_need_out, nm, nl) &&
                        !kcc_ht_get(ool_done, nm,
   nl) &&
                        !kcc_ast_has_asm((void *)
   IL((void *)IL(d, AST_F3), AST_F0))) {
                        kcc_ht_put(ool_done, nm,
   nl, (void *)1);
                        kcc_ir_build_func(d);   /*
    may mark further callees */
                        progress = 1;
                    }
                }
                d = (void *)IL(d, AST_F4);
            }
        }
    }
}

/* -- read-only const-global predicates (\S\ref{
   sec:global-inits}) -- */
unsigned long kcc_rodata_binop_ok(unsigned long op
   ) {
    if (op == TOK_PLUS || op == TOK_MINUS || op ==
    TOK_STAR || op == TOK_SLASH ||
        op == TOK_PERCENT || op == TOK_SHL || op
   == TOK_SHR || op == TOK_AMP ||
        op == TOK_PIPE || op == TOK_CARET) return
   1;
    return 0;
}

unsigned long kcc_rodata_const_intexpr(void *n) {
    if (!n) return 0;
    unsigned long tag = IL(n, AST_TAG);
    if (tag == AST_INT_LIT || tag == AST_CHAR_LIT)
    return 1;
    if (tag == AST_TYPES_COMPAT) return 1;   /*
   folds to 0/1 */
    /* sizeof/alignof are integer constant
   expressions; kcc_rodata_leaf_bits
       evaluates them through kcc_eval_const_int.
    `startup_gdt_descr.len =
       sizeof(startup_gdt) - 1' needs this, or the
    descriptor is left zero and
       lgdt loads a zero-length GDT. */
    if (tag == AST_SIZEOF_EXPR || tag ==
   AST_ALIGNOF_EXPR) return 1;
    if (tag == AST_PAREN_EXPR) return
   kcc_rodata_const_intexpr((void *)IL(n, AST_F0))
   ;
    if (tag == AST_CAST_EXPR) return
   kcc_rodata_const_intexpr((void *)IL(n, AST_F1))
   ;
    if (tag == AST_UNARY_EXPR) {
        unsigned long op = IL(n, AST_F0);
        if (op != TOK_MINUS && op != TOK_TILDE &&
   op != TOK_BANG) return 0;
        return kcc_rodata_const_intexpr((void *)IL
   (n, AST_F1));
    }
    if (tag == AST_BINARY_EXPR) {
        if (!kcc_rodata_binop_ok(IL(n, AST_F0)))
   return 0;
        if (!kcc_rodata_const_intexpr((void *)IL(n
   , AST_F1))) return 0;
        return kcc_rodata_const_intexpr((void *)IL
   (n, AST_F2));
    }
    if (tag == AST_IDENT_EXPR) {
        char *name = (char *)IL(n, AST_F0);
        if (!name) return 0;
        void *sym = kcc_symtab_lookup(name,
   kcc_strlen(name));
        if (!sym) return 0;
        if (IL(sym, SYM_KIND) != SYM_ENUM_CONST)
   return 0;
        return 1;
    }
    return 0;
}

unsigned long kcc_rodata_is_lit(void *n, unsigned
   long isfloat) {
    if (!isfloat) return kcc_rodata_const_intexpr(
   n);
    unsigned long tag = IL(n, AST_TAG);
    if (tag == AST_UNARY_EXPR) {
        if (IL(n, AST_F0) != TOK_MINUS) return 0;
        n = (void *)IL(n, AST_F1);
        if (!n) return 0;
        tag = IL(n, AST_TAG);
    }
    return tag == AST_FLOAT_LIT ? 1 : 0;
}

unsigned long kcc_rodata_struct_ok(void *sty, void
    *init) {
    if (IL(init, AST_TAG) != AST_INIT_LIST) return
    0;
    void *elem = (void *)IL(init, AST_F0);
    kcc_type_layout(sty);
    void *sd = (void *)IL(sty, TY_F0);
   /* AST_STRUCT_DECL */
    if (!sd) return 0;
    void *field = (void *)IL(sd, AST_F1);
    while (elem) {
        /* bit-fields and member designators (`.
   name = v') are accepted:
           x86_emit_struct_bytes paints each field
    into a byte image by offset,
           bit-packing bit-fields.  The early GDT
    `startup_gdt' is a const
           desc_struct[] of all-bit-field entries
   written with `.limit0='-style
           designators; rejecting either left it
   zero in .bss, so the GDT was
           zero at boot and the first `mov %ds' #
```

```
   GP'd. */
        void *val = elem, *ft;
        if (IL(elem, AST_TAG) == AST_DESIGNATOR) {
            if (IL(elem, AST_F1) != 1) return 0;
    /* only `.member' inside a struct */
            val = (void *)IL(elem, AST_F3);
            if (!val) return 0;
            /* the field is looked up by name in
   the emitter; here just require the
               value be a literal of some width (
   int or float both accepted). */
            if (!kcc_rodata_is_lit(val, 0) && !
   kcc_rodata_is_lit(val, 1)) return 0;
            elem = (void *)IL(elem, AST_F4);
            continue;
        }
        if (!field) return 0;
   /* more positional inits than fields */
        ft = (void *)IL(field, AST_F0);
        if (!ft) return 0;
        ft = kcc_sema_resolve_type(ft);
        if (!ft) return 0;
        unsigned long k = IL(ft, TY_KIND);
        if (k < TY_BOOL || k > TY_DOUBLE) return
   0;
        unsigned long isf = (k >= TY_FLOAT) ? 1 :
   0;
        if (!kcc_rodata_is_lit(val, isf)) return
   0;
        field = (void *)IL(field, AST_F4);
        elem = (void *)IL(elem, AST_F4);
    }
    return 1;
}

unsigned long kcc_rodata_aos_ok(void *sty, void *
   outer) {
    if (IL(outer, AST_TAG) != AST_INIT_LIST)
   return 0;
    void *elem = (void *)IL(outer, AST_F0);
    while (elem) {
        void *v = elem;
        if (IL(elem, AST_TAG) == AST_DESIGNATOR) {
      /* `[index] = { ... }' */
            if (IL(elem, AST_F1) != 0) return 0;
      /* only a plain index, no range/member */
            v = (void *)IL(elem, AST_F3);
        }
        if (!v || IL(v, AST_TAG) != AST_INIT_LIST)
    return 0;
        if (!kcc_rodata_struct_ok(sty, v)) return
   0;
        elem = (void *)IL(elem, AST_F4);
    }
    return 1;
}

unsigned long kcc_global_const_rodata(void *decl)
   {
    if (!IL(decl, AST_F2)) return 0;
   /* needs an initializer */
    /* Any global with a fully-constant
   initializer is emittable as static data,
        const or not (a non-const one lands in .
   data, a const one in .rodata ---
        the emitter chooses).  Dropping the old
   const-only restriction is what
        lets a non-const table like the early GDT
   (`struct desc_struct
        startup_gdt[]', writable, so .data in gcc)
   be materialised at link time
        instead of left zero in .bss with a never-
   run kcc.module_init store. */
    void *t = (void *)IL(decl, AST_F0);
   /* declared type spec */
    unsigned long wrappers = 0;
    while (t) {
        unsigned long tg = IL(t, AST_TAG);
        if (tg == AST_TYPE_ARRAY) { wrappers++; t
   = (void *)IL(t, AST_F0); continue; }
        if (tg == AST_TYPE_QUALS) { t = (void *)IL
   (t, AST_F0); continue; }  /* const or not */
        break;
    }
    {
        if (!t) return 0;
        void *base = kcc_sema_resolve_type(t);
        if (!base) return 0;
        unsigned long k = IL(base, TY_KIND);
        if (k == TY_STRUCT) {
            if (wrappers == 0) return
   kcc_rodata_struct_ok(base, (void *)IL(decl,
   AST_F2));
            if (wrappers != 1) return 0;
            return kcc_rodata_aos_ok(base, (void
   *)IL(decl, AST_F2));
        }
        if (k < TY_BOOL) return 0;
        unsigned long isf;
        if (k <= TY_LLONG) isf = 0;
        else if (k <= TY_DOUBLE) isf = 1;
        else return 0;
        void *init = (void *)IL(decl, AST_F2);
        if (IL(init, AST_TAG) != AST_INIT_LIST)
   return kcc_rodata_is_lit(init, isf);
        void *e = (void *)IL(init, AST_F0);
        while (e) {
            if (!kcc_rodata_is_lit(e, isf)) return
    0;
            e = (void *)IL(e, AST_F4);
        }
        return 1;
    }
}

/* Whether a static-data global is writable (no `
   const' qualifier anywhere in its
   type) --- it then belongs in .data rather than
   .rodata. */
unsigned long kcc_global_writable(void *decl) {
    void *t = (void *)IL(decl, AST_F0);
    while (t) {
        unsigned long tg = IL(t, AST_TAG);
        if (tg == AST_TYPE_ARRAY) { t = (void *)IL
   (t, AST_F0); continue; }
        if (tg == AST_TYPE_QUALS) { if (IL(t,
   AST_F1) & 1) return 0; t = (void *)IL(t, AST_F0
   ); continue; }
        break;
    }
    return 1;
}

unsigned long kcc_global_const_ptrarr(void *decl)
   {
    if (!IL(decl, AST_F2)) return 0;
    void *t = (void *)IL(decl, AST_F0);
    if (!t) return 0;
    if (IL(t, AST_TAG) != AST_TYPE_ARRAY) return
   0;
    t = (void *)IL(t, AST_F0);
   /* element type: a pointer */
    if (!t) return 0;
```

```
    if (IL(t, AST_TAG) != AST_TYPE_PTR) return 0;
    if (!(IL(t, AST_F1) & 1)) return 0;
   /* *const element */
    void *init = (void *)IL(decl, AST_F2);
    if (IL(init, AST_TAG) != AST_INIT_LIST) return
    0;
    void *e = (void *)IL(init, AST_F0);
    while (e) {
        unsigned long tg = IL(e, AST_TAG);
        if (tg != AST_STR_LIT && tg != AST_INT_LIT
   ) return 0;
        e = (void *)IL(e, AST_F4);
    }
    return 1;
}

/* If `n' is a compile-time symbol-address
   constant --- a function or file-scope
   object named directly (its name is its address)
   , `&lvalue', through casts,
   parens and a trailing `+/- integer', or the
   address of a \emph{member} or
   \emph{element} of such a constant (`&s.m', `p->
   m', `&a[i]') --- return the
   symbol name and add any integer displacement
   into *off; else 0.  This is
   exactly the `&sym'/`fn' initializer gcc lowers
   to a `.quad sym+off' data
   relocation, where kcc instead stores at run
   time from `kcc.module_init'
   (never called in a kernel, so the initcall/
   exitcall/pci-fixup/__setup tables
   end up zero at boot).  The member/element cases
    are what let a self-referential
   `LIST_HEAD_INIT(init_task.tasks)' --- `{ &
   init_task.tasks, &init_task.tasks }',
   i.e. `init_task + offsetof(tasks)' --- be
   materialised as data; without them
   `init_task' fell to a zero `.comm' and the
   kernel's initial stack pointer,
   derived from it, was garbage (a triple fault in
    `startup_64'). */
const char *kcc_ir_const_symaddr(void *n, long *
   off) {
    if (!n) return 0;
    unsigned long tag = IL(n, AST_TAG);
    if (tag == AST_PAREN_EXPR) return
   kcc_ir_const_symaddr((void *)IL(n, AST_F0), off
   );
    if (tag == AST_CAST_EXPR)  return
   kcc_ir_const_symaddr((void *)IL(n, AST_F1), off
   );
    if (tag == AST_UNARY_EXPR && IL(n, AST_F0) ==
   TOK_AMP)
        return kcc_ir_const_symaddr((void *)IL(n,
   AST_F1), off);      /* &X */
    if (tag == AST_BINARY_EXPR) {
        unsigned long op = IL(n, AST_F0);
        if (op == TOK_PLUS || op == TOK_MINUS) {
            void *l = (void *)IL(n, AST_F1), *r =
   (void *)IL(n, AST_F2);
            if (kcc_expr_is_ice(r)) {
                const char *s =
   kcc_ir_const_symaddr(l, off);
                if (s) { long c = (long)
   kcc_eval_const_int(r);
                    *off += (op == TOK_PLUS)
   ? c : -c; return s; }
            }
            if (op == TOK_PLUS && kcc_expr_is_ice(
   l)) {
                const char *s =
   kcc_ir_const_symaddr(r, off);
                if (s) { *off += (long)
   kcc_eval_const_int(l); return s; }
            }
        }
        return 0;
    }
    if (tag == AST_IDENT_EXPR) {
        char *name = (char *)IL(n, AST_F0);
        if (!name) return 0;
        void *sym = kcc_symtab_lookup(name,
   kcc_strlen(name));
        if (!sym || IL(sym, SYM_KIND) ==
   SYM_ENUM_CONST) return 0;   /* enum = a value
   */
        return name;                        /* a
   function/array/object name IS its address */
    }
    if (tag == AST_MEMBER) {
        /* `s.m' = address-of(s) + offsetof(m); `p
   ->m' = value-of(p) + offsetof(m).
           Recursing on the base gives the base
   symbol either way (the base of an
           arrow is itself an address constant
   like `&init_task'); the member's
           byte offset --- resolved through the
   base's cached type, descending
           anonymous struct/union members --- is
   added on. */
        void *obj = (void *)IL(n, AST_F0);
        const char *s = kcc_ir_const_symaddr(obj,
   off);
        if (!s) return 0;
        void *st = (void *)IL(obj, AST_F3);
      /* the base's type, cached by sema */
        if (!st) st = kcc_sema_expr(obj);
        if (st && IL(st, TY_KIND) == TY_PTR) st =
   (void *)IL(st, TY_F0);   /* through `->' */
        if (!st || (IL(st, TY_KIND) != TY_STRUCT
   && IL(st, TY_KIND) != TY_UNION)) return 0;
        struct kcc_member_res mr =
   kcc_type_find_member(st, (const char *)IL(n,
   AST_F1));
        if (!mr.field) return 0;
        *off += (long)mr.off;
        return s;
    }
    if (tag == AST_SUBSCRIPT) {
        /* `a[i]' with a constant index i: address
   -of(a) + i*sizeof(*a). */
        void *base = (void *)IL(n, AST_F0), *idx =
    (void *)IL(n, AST_F1);
        if (!kcc_expr_is_ice(idx)) return 0;
        const char *s = kcc_ir_const_symaddr(base,
    off);
        if (!s) return 0;
        void *bt = (void *)IL(base, AST_F3);
        if (!bt) bt = kcc_sema_expr(base);
        unsigned long es = 1;
        if (bt) {
            unsigned long bk = IL(bt, TY_KIND);
            if (bk == TY_ARRAY || bk == TY_PTR) {
                void *et = (void *)IL(bt, TY_F0);
                if (et) es = IL(et, TY_SIZE);
            }
        }
        *off += (long)kcc_eval_const_int(idx) * (
   long)es;
        return s;
```

```
    }
    return 0;
}

/* True when a global's initializer must be
   emitted as static DATA carrying symbol
   relocations rather than a run-time store: a
   scalar pointer set to a symbol
   address or a string literal, or a (non-array)
   struct whose leaves are all
   integer/float/string literals or symbol
   addresses, with at least one symbol (a
   pure-literal struct stays on the existing const
   -rodata path).

   A file-scope \texttt{char *p = "..."} is the
   string-literal case: its
   initializer is the address of a string literal,
    which the x86 emitter already
   lays down as \texttt{.quad .Lx86str<id>} (
   registering the string for the render
   pass).  Without this it fell to a run-time
   store in \texttt{kcc.module\_init} ---
   fine on Mach-O, where the loader runs module-
   init, but fatal in a kernel, which
   never runs \texttt{.init\_array}: the kernel's
   \texttt{ramdisk\_execute\_command
   = "/init"} stayed a zero \texttt{.comm}, so \
   texttt{init\_eaccess(NULL)} ->
   \texttt{strlen(NULL)} faulted PID~1 just after
   the device initcalls.  The case is
   gated on \texttt{kcc\_target}: only the x86
   back end has the string-literal data
   emitter, and the run-time-store skip (below) is
    likewise ungated, so on ARM ---
   where module-init does run --- the literal must
    keep its run-time store. */
unsigned long kcc_global_sym_data(void *decl) {
    void *init = (void *)IL(decl, AST_F2);
    if (!init) return 0;
    void *ty = kcc_sema_resolve_type((void *)IL(
   decl, AST_F0));
    if (!ty) return 0;
    unsigned long k = IL(ty, TY_KIND);
    long off;
    if (k == TY_PTR) return kcc_ir_const_symaddr(
   init, &off) != 0
                            || (kcc_target && IL(
   init, AST_TAG) == AST_STR_LIT);
    if (k == TY_STRUCT) {
        if (IL(init, AST_TAG) != AST_INIT_LIST)
   return 0;
        unsigned long anysym = 0;
        for (void *e = (void *)IL(init, AST_F0); e
   ; e = (void *)IL(e, AST_F4)) {
            unsigned long et = IL(e, AST_TAG);
            if (et == AST_DESIGNATOR || et ==
   AST_INIT_LIST) return 0;  /* keep it flat */
            if (kcc_ir_const_symaddr(e, &off)) {
   anysym = 1; continue; }
            if (et == AST_STR_LIT) continue;
            if (kcc_rodata_is_lit(e, 0) ||
   kcc_rodata_is_lit(e, 1)) continue;
            return 0;
        }
        return anysym;
    }
    return 0;
}

/* A scalar initializer leaf the static emitter
   can lay down as bytes: a string,
   a symbol address (`&x'/`fn'/`&s.m'), an integer
    constant expression, or a
   float literal.  (Integer constants are
   evaluated by kcc_eval_const_int and
   float leaves by kcc_rodata_leaf_bits, so the
   acceptance set matches exactly
   what the emitter can render.) */
unsigned long kcc_static_leaf_ok(void *e) {
    long off;
    if (!e) return 1;
       /* a hole --- zero-filled */
    if (IL(e, AST_TAG) == AST_STR_LIT) return 1;
    if (kcc_ir_const_symaddr(e, &off)) return 1;
    if (kcc_expr_is_ice(e)) return 1;
    if (kcc_rodata_is_lit(e, 1)) return 1;
       /* float literal, maybe negated */
    return 0;
}

/* Whether the whole (aggregate) initializer of `
   ty' is a compile-time constant
   the x86 back end can materialise as a byte
   image with relocations: every
   nested struct/union/array must be a brace (or
   compound-literal) initializer of
   the same, and every scalar leaf must satisfy
   kcc_static_leaf_ok.  This is the
   general form the three earlier predicates (
   const-rodata / const-ptrarr /
   sym-data) only handle in flat special cases; it
    is what lets `init_task' ---
   deeply nested designated initializers, self-
   referential `&init_task.member'
   list heads, compound literals, a `char comm[] =
    "swapper"' --- be emitted as
   real `.data' instead of a zero `.comm'.
   Anything not fully understood is
   rejected so the global stays on the existing
   path (never worse than before). */
unsigned long kcc_static_init_ok(void *ty, void *
   init) {
    if (!ty) return 0;
    while (init) {
       /* peel compound-literal / paren */
        unsigned long t = IL(init, AST_TAG);
        if (t == AST_PAREN_EXPR) { init = (void *)
   IL(init, AST_F0); continue; }
        if (t == AST_COMPOUND_LIT) { init = (void
   *)IL(init, AST_F1); continue; }
        break;
    }
    unsigned long k = IL(ty, TY_KIND);
    if (k == TY_STRUCT || k == TY_UNION) {
        if (!init || IL(init, AST_TAG) !=
   AST_INIT_LIST) return 0;
        kcc_type_layout(ty);
        void *decl = (void *)IL(ty, TY_F0);
        void *field = decl ? (void *)IL(decl,
   AST_F1) : 0;   /* positional cursor */
        for (void *e = (void *)IL(init, AST_F0); e
   ; e = (void *)IL(e, AST_F4)) {
            void *val, *fty;
            if (IL(e, AST_TAG) == AST_DESIGNATOR)
   {
                if (IL(e, AST_F1) != 1 || IL(e,
   AST_F2)) return 0;   /* only single `.member'
   */
                struct kcc_member_res mr =
   kcc_type_find_member(ty, (const char *)IL(e,
   AST_F0));
                if (!mr.field) return 0;
                fty = kcc_sema_resolve_type((void
```

```
    *)IL(mr.field, AST_F0));
                val = (void *)IL(e, AST_F3);
            } else {
                if (!field) return 0;
                fty = kcc_sema_resolve_type((void
    *)IL(field, AST_F0));
                val = e;
                field = (void *)IL(field, AST_F4);
            }
            if (!fty) return 0;
            unsigned long fk = IL(fty, TY_KIND);
            if (fk == TY_STRUCT || fk == TY_UNION
    || fk == TY_ARRAY) {
                if (!kcc_static_init_ok(fty, val))
     return 0;
            } else if (!kcc_static_leaf_ok(val))
    return 0;
        }
        return 1;
     }
     if (k == TY_ARRAY) {
        void *et = (void *)IL(ty, TY_F0);
        if (init && IL(init, AST_TAG) ==
    AST_STR_LIT) return 1;      /* char[] = "..."
    */
        if (!init || IL(init, AST_TAG) !=
    AST_INIT_LIST) return 0;
        unsigned long ek = et ? IL(et, TY_KIND) :
    0;
        for (void *e = (void *)IL(init, AST_F0); e
    ; e = (void *)IL(e, AST_F4)) {
            void *v = e;
            if (IL(e, AST_TAG) == AST_DESIGNATOR)
    {
                if (IL(e, AST_F1) != 0 || IL(e,
    AST_F2)) return 0;   /* only plain `[i]' */
                if (!kcc_expr_is_ice((void *)IL(e,
     AST_F0))) return 0;
                v = (void *)IL(e, AST_F3);
            }
            if (ek == TY_STRUCT || ek == TY_UNION
    || ek == TY_ARRAY) {
                if (!kcc_static_init_ok(et, v))
    return 0;
            } else if (!kcc_static_leaf_ok(v))
    return 0;
        }
        return 1;
     }
     if (init && IL(init, AST_TAG) == AST_INIT_LIST
    ) init = (void *)IL(init, AST_F0);
     return kcc_static_leaf_ok(init);
}

/* Route an aggregate global with a fully-constant
     initializer (the three flat
    predicates missed) through the general byte-
    image emitter rather than a zero
    `.comm'.  Scalars keep their existing paths. */
unsigned long kcc_global_static_ok(void *decl) {
     void *init = (void *)IL(decl, AST_F2);
     if (!init) return 0;
     void *ty = kcc_sema_resolve_type((void *)IL(
    decl, AST_F0));
     if (!ty) return 0;
     unsigned long k = IL(ty, TY_KIND);
     if (k != TY_STRUCT && k != TY_UNION && k !=
    TY_ARRAY) return 0;
     return kcc_static_init_ok(ty, init);
}

/* A file-scope object placed in a discarded
    section --- one the kernel's
    linker script throws away --- has no run-time
    existence, so running its
    initializer as module-init code would forge a
    relocation from the kept
    `.text' into a section that vanishes.  Two
    families are discarded: the
    `__ADDRESSABLE(sym)' pattern's `.discard.
    addressable' (`/DISCARD/ :
    { *(.discard) *(.discard.*) }'), and the module
     teardown sections a
    built-in kernel never runs --- `.exit.text', `.
    exit.data', `.exitcall.exit'
    (`__exitcall(fn)' = `static exitcall_t = fn' in
     `.exitcall.exit').  kcc
    lowers such an initializer's store into `kcc.
    module_init' and so fails the
    vmlinux link with ".exitcall.exit referenced in
     .text ... defined in
    discarded section".  gcc emits the pointer as a
     data relocation in the
    discarded section with no code; kcc simply
    skips the initializer, since the
    storage is discarded either way.  A prefix
    match on `.discard' or `.exit'
    catches every one (no kept section begins with
    `.exit'). */
struct AttrRec { unsigned long weak; const char *
    section; const char *alias; };
extern void *kcc_attr_ht;
static int kcc_ir_prefix(const char *s, const char
     *p) {
     for (unsigned long i = 0; p[i]; i++) if (s[i]
    != p[i]) return 0;
     return 1;
}
static int kcc_ir_section_discarded(const char *
    name) {
     if (!name || !kcc_attr_ht) return 0;
     struct AttrRec *ar = (struct AttrRec *)
    kcc_ht_get(kcc_attr_ht, name, kcc_strlen(name))
    ;
     if (!ar || !ar->section) return 0;
     return kcc_ir_prefix(ar->section, ".discard")
    || kcc_ir_prefix(ar->section, ".exit");
}

/* -- global-initializer emission at the top of
    main / a module-init fn -- */
void kcc_ir_emit_global_inits(void) {
     void *tu = kcc_ir_trans_unit;
     if (!tu) return;
     void *d = (void *)IL(tu, AST_F0);
     while (d) {
         if (IL(d, AST_TAG) == AST_VAR_DECL && IL(d
    , AST_F2) && IL(d, AST_F1)) {
             void *name = (void *)IL(d, AST_F1);
             /* On x86 an aggregate accepted by
    kcc_global_static_ok is materialised
                as static `.data' by
    x86_emit_general, so it must NOT also get a
                run-time store here (that would
    double-initialise it, and a store
                into a const `.rodata' object
    faults).  The guard is gated on
                kcc_target because only the x86
    back end has the general emitter;
                the ARM back end still fills these
    at run time from module-init. */
             if (!kcc_global_const_rodata(d) && !
    kcc_global_const_ptrarr(d) &&
                 !kcc_global_sym_data(d) &&
```

```
            !(kcc_target &&
kcc_global_static_ok(d)) &&
            !kcc_ir_section_discarded((const
char *)name)) {
            void *g = kcc_ir_alloc_instr(
IR_GLOBAL_REF);
            IL(g, IR_SRC1) = (unsigned long)
name;
            IL(g, IR_TYPE) = (unsigned long)
kcc_type_int_s;
            unsigned long addr =
kcc_ir_next_vreg();
            IL(g, IR_DEST) = addr;
            kcc_ir_emit(g);
            kcc_ir_emit_init_stores(d, addr);
        }
    }
    d = (void *)IL(d, AST_F4);
 }
 /* second pass: one-time static-local
initialisers, hoisted to globals */
 d = (void *)IL(tu, AST_F0);
 while (d) {
    if (IL(d, AST_TAG) == AST_FUNC_DECL) {
        void *body = (void *)IL(d, AST_F3);
        if (body) {
            /* Re-register this function's
static-locals: emit_modinit reset
               the mangled-name table, so an
initializer that takes the
               address of a sibling static (`
static T *p[] = { other, ... }')
               would otherwise resolve to the
source name and link against an
               undefined symbol.  Register
before emitting any init store. */
            void *rs = (void *)IL(body, AST_F0
);
            while (rs) {
                if (IL(rs, AST_TAG) ==
AST_VAR_DECL && IL(rs, AST_F3) == 3 &&
                    IL(rs, AST_F1)) {
                    void *nm = (void *)IL(rs,
AST_F1);
                    unsigned long kl =
kcc_strlen((const char *)nm);
                    kcc_ht_put(kcc_ir_statics,
 nm, kl,
                               (void *)
kcc_ir_static_name(rs));
                    void *tn = (void *)IL(rs,
AST_F0);
                    if (tn) {
                        void *rt =
kcc_sema_resolve_type(tn);
                        if (rt) kcc_ht_put(
kcc_ir_static_types, nm, kl, rt);
                    }
                }
                rs = (void *)IL(rs, AST_F4);
            }
            void *s = (void *)IL(body, AST_F0)
;
            while (s) {
                if (IL(s, AST_TAG) ==
AST_VAR_DECL && IL(s, AST_F3) == 3 &&
                    IL(s, AST_F2) &&
                    !kcc_global_const_rodata(s
) && !kcc_global_const_ptrarr(s) &&
                    !(kcc_target &&
kcc_global_static_ok(s)) &&
                    !kcc_ir_section_discarded
((const char *)IL(s, AST_F1))) {
                    const char *mangled =
kcc_ir_static_name(s);
                    void *g =
kcc_ir_alloc_instr(IR_GLOBAL_REF);
                    IL(g, IR_SRC1) = (unsigned
 long)mangled;
                    IL(g, IR_TYPE) = (unsigned
 long)kcc_type_int_s;
                    unsigned long addr =
kcc_ir_next_vreg();
                    IL(g, IR_DEST) = addr;
                    kcc_ir_emit(g);
                    kcc_ir_emit_init_stores(s,
 addr);
                }
                s = (void *)IL(s, AST_F4);
            }
        }
        d = (void *)IL(d, AST_F4);
    }
    /* Discard the static-local names seeded above
: this runs at the top of
       main (or a module-init), and leaving other
functions' statics in the
       table would let them shadow a like-named
global in the body that
       follows (a static `c` masking the global `c
`). */
    kcc_ir_statics = kcc_ht_init(16);
    kcc_ir_static_types = kcc_ht_init(16);
}

void kcc_ir_emit_aggregate_init(void *init_list,
   void *agg, unsigned long base) {
    void *head = (void *)IL(init_list, AST_F0);
    kcc_ir_fill_list(agg, base, 0, head);
}

void kcc_ir_emit_init_stores(void *decl, unsigned
   long base_vreg) {
    void *init = (void *)IL(decl, AST_F2);
    if (!init) return;
    unsigned long itag = IL(init, AST_TAG);
    if (itag == AST_INIT_LIST) {
        void *ty = kcc_sema_resolve_type((void *)
IL(decl, AST_F0));
        if (!ty) ty = kcc_type_int_s;
        kcc_ir_emit_aggregate_init(init, ty,
base_vreg);
        return;
    }
    if (itag == AST_STR_LIT) {
        void *ty = kcc_sema_resolve_type((void *)
IL(decl, AST_F0));
        if (ty && IL(ty, TY_KIND) == TY_ARRAY) {
            unsigned long alen = IL(ty, TY_F1);
            const char *sp = (const char *)IL(init
, AST_F0);
            unsigned long slen = IL(init, AST_F1);
            if (!alen) alen = slen + 1;
            kcc_ir_emit_string_init(base_vreg, sp,
 slen, alen, 0);
            return;
        }
        /* a char *g = "..." target is a scalar
store of the address */
    }
    /* scalar (or whole-object struct copy) */
```

```
    void *rty = kcc_sema_resolve_type((void *)IL(
decl, AST_F0));
    if (rty) {
        unsigned long k = IL(rty, TY_KIND);
        if (k == TY_STRUCT || k == TY_UNION) {
            unsigned long src =
kcc_ir_build_struct_addr(init);
            unsigned long sz = kcc_type_layout(rty
);
            kcc_ir_emit_struct_copy(src, base_vreg
, sz);
            return;
        }
    }
    void *from = (void *)IL(init, AST_F3);
    /* initializer's sema type */
    unsigned long val = kcc_ir_build_expr(init);
    void *to = kcc_sema_resolve_type((void *)IL(
decl, AST_F0));
    if (to && from) val = kcc_ir_convert(val, from
, to);
    if (to && IL(to, TY_KIND) == TY_BOOL) val =
kcc_ir_emit_bool_norm(val);
    unsigned long size = kcc_ir_store_width(to);
    void *st = kcc_ir_alloc_instr(IR_STORE);
    IL(st, IR_TYPE) = to ? (unsigned long)to : (
unsigned long)kcc_type_int_s;
    IL(st, IR_SRC1) = val;
    IL(st, IR_SRC2) = base_vreg;
    IL(st, IR_DEST) = size;
    kcc_ir_emit(st);
}

void kcc_ir_emit_modinit(void) {
    void *f = kcc_ir_alloc_func(kcc_str_modinit,
kcc_type_int_s);
    kcc_ir_cur_func = f;
    kcc_ir_locals = kcc_ht_init(64);
    kcc_ir_local_consts = kcc_ht_init(64);
    kcc_ir_statics = kcc_ht_init(16);
    kcc_ir_static_types = kcc_ht_init(16);
    kcc_ir_labels = kcc_ht_init(16);
    kcc_ir_scope_jdepth = 0;
    kcc_ir_scope_sp = 0;
    void *bb = kcc_ir_alloc_bb();
    IL(f, IRF_ENTRY) = (unsigned long)bb;
    kcc_ir_set_bb(bb);
    kcc_ir_emit_global_inits();
    void *ret = kcc_ir_alloc_instr(IR_RET);
    IL(ret, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    kcc_ir_emit(ret);
    void *cur = kcc_ir_cur_bb;
    IL(cur, BB_FLAGS) = IL(cur, BB_FLAGS) |
BB_FLAG_TERMINATED;
    kcc_vec_push(kcc_ir_funcs, &f);
    kcc_modinit_present = 1;
}

/* -- per-function driver -- */
void kcc_ir_build_func(void *fd) {
    void *name = (void *)IL(fd, AST_F1);
    void *ta = (void *)IL(fd, AST_F0);
    /* type AST */
    void *ret = 0;
    if (ta) {
        if (IL(ta, AST_TAG) == AST_TYPE_FUNC) ta =
(void *)IL(ta, AST_F0);
        if (ta) ret = kcc_sema_resolve_type(ta);
    }
    if (!ret) ret = kcc_type_int_s;
    void *f = kcc_ir_alloc_func(name, ret);
    IL(f, IRF_LINE) = IL(fd, AST_LINE);
    IL(f, IRF_STATIC) = (IL(fd, AST_F2) & 2) >> 1;
    kcc_ir_cur_func = f;
    kcc_ir_locals = kcc_ht_init(64);
    kcc_ir_local_consts = kcc_ht_init(64);
    kcc_ir_statics = kcc_ht_init(16);
    kcc_ir_static_types = kcc_ht_init(16);
    kcc_ir_labels = kcc_ht_init(16);

    void *entry = kcc_ir_alloc_bb();
    IL(f, IRF_ENTRY) = (unsigned long)entry;
    kcc_ir_set_bb(entry);

    /* count named parameters */
    void *p = (void *)(IL(fd, AST_F2) & ~3UL);
    unsigned long nparams = 0;
    while (p) {
        if (IL(p, AST_TAG) == AST_PARAM_DECL && IL
(p, AST_F1)) nparams++;
        p = (void *)IL(p, AST_F4);
    }
    /* >16B struct return adds a hidden result-
pointer parameter */
    if (kcc_ir_classify_type((void *)IL(f,
IRF_RETTYPE)) == 4) nparams++;
    IL(f, IRF_PARAMS) = nparams;
    IL(f, IRF_VARIADIC) = IL(fd, AST_F2) & 1;

    kcc_ir_fill_param_classes(f, fd);

    /* reserve incoming-parameter vregs (an ST2
struct param takes two) */
    unsigned long width = 0;
    unsigned long i = 0;
    while (i < nparams) {
        void *cls = (void *)IL(f, IRF_PARAM_CLS);
        unsigned long c = cls ? IL(cls, i * 8) :
0;
        width += (c == 3) ? 2 : 1;
        i++;
    }
    i = 0;
    while (i < width) { kcc_ir_next_vreg(); i++; }

    /* build alloca + store for each parameter,
tracking its vreg base */
    p = (void *)(IL(fd, AST_F2) & ~3UL);
    unsigned long pidx = 0;
    unsigned long vbase = 0;
    while (p) {
        if (IL(p, AST_TAG) == AST_PARAM_DECL && IL
(p, AST_F1)) {
            kcc_ir_build_param(p, vbase);
            void *cls = (void *)IL(f,
IRF_PARAM_CLS);
            unsigned long c = IL(cls, pidx * 8);
            vbase += (c == 3) ? 2 : 1;
            pidx++;
        }
        p = (void *)IL(p, AST_F4);
    }

    if (name && kcc_strcmp((const char *)name, "
main") == 0)
        kcc_ir_emit_global_inits();

    kcc_ir_prescan_labels((void *)IL(fd, AST_F3));
    kcc_ir_build_stmt((void *)IL(fd, AST_F3));
```

```
    /* implicit return if the last block has no
   terminator */
    void *cur = kcc_ir_cur_bb;
    if (cur && !(IL(cur, BB_FLAGS) &
   BB_FLAG_TERMINATED)) {
        void *fname = (void *)IL(f, IRF_NAME);
        if (fname && kcc_strcmp((const char *)
   fname, "main") == 0) {
            unsigned long z = kcc_ir_emit_const(0)
   ;
            void *r = kcc_ir_alloc_instr(IR_RET);
            IL(r, IR_TYPE) = (unsigned long)ret;
            IL(r, IR_SRC1) = z;
            kcc_ir_emit(r);
        } else {
            void *r = kcc_ir_alloc_instr(IR_RET);
            IL(r, IR_TYPE) = 0;
   /* bare return: no value */
            kcc_ir_emit(r);
        }
        cur = kcc_ir_cur_bb;
        IL(cur, BB_FLAGS) = IL(cur, BB_FLAGS) |
   BB_FLAG_TERMINATED;
    }
    kcc_vec_push(kcc_ir_funcs, &f);
}

/* -- parameter lowering -- */
void kcc_ir_build_param(void *pd, unsigned long
   vreg_base) {
    void *t = (void *)IL(pd, AST_F0);
    if (!t) return;
    t = kcc_sema_resolve_type(t);
    t = kcc_type_param_adjust(t);
    if (!t) return;
    void *name = (void *)IL(pd, AST_F1);
    if (!name) return;

    unsigned long slots;
    if (IL(t, TY_KIND) == TY_STRUCT) slots =
   kcc_ir_type_slots(t);
    else slots = 1;

    void *a = kcc_ir_alloc_instr(IR_ALLOCA);
    IL(a, IR_TYPE) = (unsigned long)t;
    IL(a, IR_SRC1) = slots;
    unsigned long avreg = kcc_ir_next_vreg();
    IL(a, IR_DEST) = avreg;
    kcc_ir_emit(a);

    unsigned long k = IL(t, TY_KIND);
    if (k == TY_STRUCT || k == TY_UNION) {
        unsigned long sz = kcc_type_layout(t);
        if (sz <= 8) {
            goto scalar_store;
        } else if (sz <= 16) {
            kcc_ir_emit_indexed_store(avreg, 0,
   vreg_base, 0);
            kcc_ir_emit_indexed_store(avreg, 8,
   vreg_base + 1, 0);
        } else {
            kcc_ir_emit_struct_copy(vreg_base,
   avreg, sz);
        }
    } else {
    scalar_store: {
        void *st = kcc_ir_alloc_instr(IR_STORE);
        IL(st, IR_SRC1) = vreg_base;
        IL(st, IR_SRC2) = avreg;
        IL(st, IR_TYPE) = (unsigned long)t;
        IL(st, IR_DEST) = kcc_ir_store_width(t);
            kcc_ir_emit(st);
    }
    }
    /* record name -> alloca vreg+1 (bit 63 tags a
    pointer variable) */
    unsigned long v = avreg + 1;
    if (IL(t, TY_KIND) == TY_PTR) v = v | 0
   x8000000000000000UL;
    kcc_ht_put(kcc_ir_locals, name, kcc_strlen((
   const char *)name), (void *)v);
}

/* -- labels and goto -- */
void kcc_ir_branch_to(void *target) {
    void *cur = kcc_ir_cur_bb;
    if (!cur) return;
    if (IL(cur, BB_FLAGS) & BB_FLAG_TERMINATED)
   return;
    void *br = kcc_ir_alloc_instr(IR_BR);
    IL(br, IR_SRC1) = IL(target, BB_ID);
    kcc_ir_emit(br);
    cur = kcc_ir_cur_bb;
    IL(cur, BB_FLAGS) = IL(cur, BB_FLAGS) |
   BB_FLAG_TERMINATED;
}

void *kcc_ir_label_bb(const char *name) {
    if (!kcc_ir_labels) return 0;
    return kcc_ht_get(kcc_ir_labels, name,
   kcc_strlen(name));
}

void kcc_ir_prescan_labels(void *stmt) {
    if (!stmt) return;
    unsigned long tag = IL(stmt, AST_TAG);
    if (tag == AST_COMPOUND) {
        void *s = (void *)IL(stmt, AST_F0);
        while (s) { kcc_ir_prescan_labels(s); s =
   (void *)IL(s, AST_F4); }
    } else if (tag == AST_LABEL_STMT) {
        void *name = (void *)IL(stmt, AST_F0);
        void *bb = kcc_ir_alloc_bb();
        kcc_ht_put(kcc_ir_labels, name, kcc_strlen
   ((const char *)name), bb);
        kcc_ir_prescan_labels((void *)IL(stmt,
   AST_F1));
    } else if (tag == AST_IF_STMT) {
        kcc_ir_prescan_labels((void *)IL(stmt,
   AST_F1));
        kcc_ir_prescan_labels((void *)IL(stmt,
   AST_F2));
    } else if (tag == AST_WHILE_STMT || tag ==
   AST_SWITCH_STMT || tag == AST_CASE_STMT) {
        kcc_ir_prescan_labels((void *)IL(stmt,
   AST_F1));
    } else if (tag == AST_DO_STMT || tag ==
   AST_DEFAULT_STMT) {
        kcc_ir_prescan_labels((void *)IL(stmt,
   AST_F0));
    } else if (tag == AST_FOR_STMT) {
        kcc_ir_prescan_labels((void *)IL(stmt,
   AST_F3));
    }
}

/* -- block-scope shadowing journal -- */
void kcc_ir_locals_put(const char *name, unsigned
   long keylen, unsigned long value) {
    unsigned long old = (unsigned long)kcc_ht_get(
   kcc_ir_locals, name, keylen);
    unsigned long j = kcc_ir_scope_jdepth;
```

```
    kcc_ir_scope_journal[j*3 + 0] = (unsigned long
   )name;
    kcc_ir_scope_journal[j*3 + 1] = keylen;
    kcc_ir_scope_journal[j*3 + 2] = old;
    kcc_ir_scope_jdepth = j + 1;
    kcc_ht_put(kcc_ir_locals, name, keylen, (void
   *)value);
}

void kcc_ir_scope_enter(void) {
    kcc_ir_scope_stack[kcc_ir_scope_sp] =
   kcc_ir_scope_jdepth;
    kcc_ir_scope_sp = kcc_ir_scope_sp + 1;
}

void kcc_ir_scope_leave(void) {
    kcc_ir_scope_sp = kcc_ir_scope_sp - 1;
    unsigned long floor = kcc_ir_scope_stack[
   kcc_ir_scope_sp];
    while (kcc_ir_scope_jdepth > floor) {
        unsigned long j = kcc_ir_scope_jdepth - 1;
        kcc_ir_scope_jdepth = j;
        kcc_ht_put(kcc_ir_locals,
                   (void *)kcc_ir_scope_journal[j
   *3 + 0],
                   kcc_ir_scope_journal[j*3 + 1],
                   (void *)kcc_ir_scope_journal[j
   *3 + 2]);
    }
}

/* -- const-initialised locals (x86 inline-asm "i
   "-operand folding) -- */
void kcc_ir_local_const_set(const char *name,
   unsigned long keylen,
                            unsigned long lv,
   unsigned long value) {
    unsigned long *box = (unsigned long *)
   kcc_arena_alloc(16);
    box[0] = lv;
    box[1] = value;
    kcc_ht_put(kcc_ir_local_consts, name, keylen,
   (void *)box);
}
void kcc_ir_local_const_kill(const char *name) {
    if (!kcc_target || !kcc_ir_local_consts || !
   name) return;
    unsigned long keylen = kcc_strlen(name);
    unsigned long lv = (unsigned long)kcc_ht_get(
   kcc_ir_locals, name, keylen);
    unsigned long *box = (unsigned long *)
   kcc_ht_get(kcc_ir_local_consts, name, keylen);
    if (box && box[0] == lv) box[0] = 0;   /* no
   longer a manifest constant */
}

/* -- statement dispatcher -- */
void kcc_ir_build_stmt(void *s) {
    if (!s) return;
    unsigned long tag = IL(s, AST_TAG);
    if (tag == AST_COMPOUND) {
        kcc_ir_scope_enter();
        void *it = (void *)IL(s, AST_F0);
        while (it) { kcc_ir_build_stmt(it); it = (
   void *)IL(it, AST_F4); }
        kcc_ir_scope_leave();
    } else if (tag == AST_EXPR_STMT) {
        kcc_ir_build_expr((void *)IL(s, AST_F0));
    } else if (tag == AST_RETURN_STMT) {
        kcc_ir_build_return(s);
    } else if (tag == AST_IF_STMT) {
        kcc_ir_build_if(s);
    } else if (tag == AST_WHILE_STMT) {
        kcc_ir_build_while(s);
    } else if (tag == AST_DO_STMT) {
        kcc_ir_build_do(s);
    } else if (tag == AST_FOR_STMT) {
        kcc_ir_build_for(s);
    } else if (tag == AST_VAR_DECL) {
        kcc_ir_build_var_decl(s);
    } else if (tag == AST_BREAK_STMT) {
        kcc_ir_build_break(s);
    } else if (tag == AST_CONTINUE_STMT) {
        kcc_ir_build_continue(s);
    } else if (tag == AST_SWITCH_STMT) {
        kcc_ir_build_switch(s);
    } else if (tag == AST_LABEL_STMT) {
        void *bb = kcc_ir_label_bb((const char *)
IL(s, AST_F0));
        if (bb) { kcc_ir_branch_to(bb);
kcc_ir_set_bb(bb); }
        kcc_ir_build_stmt((void *)IL(s, AST_F1));
    } else if (tag == AST_GOTO_STMT) {
        if (IL(s, AST_F1)) {
            /* computed `goto *expr': jump to a
label address in a register. */
            unsigned long v = kcc_ir_build_expr((
void *)IL(s, AST_F0));
            void *cur = kcc_ir_cur_bb;
            if (cur && !(IL(cur, BB_FLAGS) &
BB_FLAG_TERMINATED)) {
                void *br = kcc_ir_alloc_instr(
IR_BR_IND);
                IL(br, IR_SRC1) = v;
                kcc_ir_emit(br);
                cur = kcc_ir_cur_bb;
                IL(cur, BB_FLAGS) = IL(cur,
BB_FLAGS) | BB_FLAG_TERMINATED;
            }
            kcc_ir_set_bb(kcc_ir_alloc_bb());
        } else {
            void *bb = kcc_ir_label_bb((const char
 *)IL(s, AST_F0));
            if (bb) { kcc_ir_branch_to(bb);
kcc_ir_set_bb(kcc_ir_alloc_bb()); }
        }
    } else if (tag == AST_CASE_STMT) {
        void *bb = (void *)IL(s, AST_F2);
        if (bb) { kcc_ir_branch_to(bb);
kcc_ir_set_bb(bb); }
        kcc_ir_build_stmt((void *)IL(s, AST_F1));
    } else if (tag == AST_DEFAULT_STMT) {
        void *bb = (void *)IL(s, AST_F1);
        if (bb) { kcc_ir_branch_to(bb);
kcc_ir_set_bb(bb); }
        kcc_ir_build_stmt((void *)IL(s, AST_F0));
    } else if (tag == AST_ASM_STMT) {
        kcc_ir_asm_lower_operands(s);
        /* `asm goto': resolve each target label (
chained off AST_F3) to its
           block and stash the block in the label
node's F2, so the back end
           can expand %lN to that block's assembly
 label.  F0 stays the name the
           template's %l[name] matches; an inlined
 label carries the renamed
           resolve-name in F1, so resolution
follows F1 when present. */
        for (void *l = (void *)IL(s, AST_F3); l; l
 = (void *)IL(l, AST_F4)) {
            const char *rn = (const char *)IL(l,
```

```
    AST_F1);
            if (!rn) rn = (const char *)IL(l,
    AST_F0);
            void *bb = kcc_ir_label_bb(rn);
            IL(l, AST_F2) = (unsigned long)bb;
        }
        void *ai = kcc_ir_alloc_instr(IR_ASM);
        IL(ai, IR_SRC1) = IL(s, AST_F0);
        IL(ai, IR_SRC2) = IL(s, AST_F1);
        IL(ai, IR_TYPE) = (unsigned long)s;
        kcc_ir_emit(ai);
    }
    /* AST_NULL_STMT and anything unhandled:
    nothing to emit */
}

/* -- extended-asm operand lowering -- */
void kcc_ir_asm_addr_of(void *lvalue) {
    void *u = kcc_ast_alloc(AST_UNARY_EXPR);
    IL(u, AST_F0) = TOK_AMP;
    IL(u, AST_F1) = (unsigned long)lvalue;
    IL(u, AST_F2) = 1;
    /* prefix */
    kcc_ir_build_expr(u);
    /* returns address vreg */
}

/* Does the constraint permit an immediate operand
    ?  Besides the generic 'i'
    (any constant), 'I'/'n' (small/known integer),
    x86 has the letter classes
    'e' (32-bit signed const), 'J'/'K'/'L'/'M'/'N
    '/'O'/'Z' (specific ranges), and
    'g' (general, which also admits a constant).  A
    constant operand under any of
    these is rendered as an immediate; a non-
    constant 'g' falls through to the
    register path. */
static unsigned long kcc_asm_cons_imm(const char *
    c) {
    for (const char *s = c; *s; s++)
        if (*s == 'i' || *s == 'I' || *s == 'n' ||
    *s == 'e' || *s == 'g' ||
            *s == 'J' || *s == 'K' || *s == 'L' ||
    *s == 'M' || *s == 'N' ||
            *s == 'O' || *s == 'Z') return 1;
    return 0;
}

/* Whether a constant VALUE fits one of the
    constraint's immediate classes.  The
    x86 letters bound the range: a plain `i'/`n'/`g
    '/`e' is the ALU's
    sign-extended 32-bit field; `I'-`O' name the
    narrow ranges the shift/bit and
    in/out forms need (`I' 0-31 for `bts', `N'
    0-255 for `in $imm').  When a
    constant falls outside every immediate class
    the constraint allows but the
    constraint also admits a register, it must go
    to the register alternative---
    emitting `btsq $330' (an `"Ir"' bit number the
    compiler would otherwise put in
    a register) is rejected by the assembler as an
    operand-type mismatch. */
static int kcc_asm_imm_fits(const char *c, long v)
    {
    for (const char *s = c; *s; s++) {
        switch (*s) {
        case 'i': case 'n': case 'g': case 'e':
            if (v >= (long)0xFFFFFFFF80000000UL &&
     v <= (long)0x7FFFFFFFL) return 1; break;
        case 'Z': if (v >= 0 && v <= (long)0
    xFFFFFFFFL) return 1; break;
        case 'I': if (v >= 0 && v <= 31) return 1;
     break;
        case 'J': if (v >= 0 && v <= 63) return 1;
     break;
        case 'K': if (v >= -128 && v <= 127)
    return 1; break;
        case 'L': if (v == 0xFF || v == 0xFFFF ||
    v == (long)0xFFFFFFFFL) return 1; break;
        case 'M': if (v >= 0 && v <= 3) return 1;
    break;
        case 'N': if (v >= 0 && v <= 255) return
    1; break;
        case 'O': if (v >= 0 && v <= 127) return
    1; break;
        default: break;
        }
    }
    return 0;
}

/* Whether the constraint also admits a general or
     specific integer register, so
    a constant that will not fit an x86 ALU
    instruction's 32-bit immediate field
    can be materialised there instead---the
    register alternative of `"re"'/`"ri"'. */
static unsigned long kcc_asm_cons_reg(const char *
    c) {
    for (const char *s = c; *s; s++)
        if (*s == 'r' || *s == 'q' || *s == 'Q' ||
    *s == 'a' || *s == 'b' ||
            *s == 'c' || *s == 'd' || *s == 'S' ||
    *s == 'D' || *s == 'R' ||
            *s == 'l' || *s == 'A' || *s == 'U')
    return 1;
    return 0;
}

/* If an asm operand's expression names a function
     or file-scope object (or the
    address of one), return the symbol's emit-name.
      The back end renders a `%P`
    reference to such an operand as the bare symbol
     (so `call %P[f]` becomes
    `call f`), which is what the kernel's
    ALTERNATIVE and percpu templates need.
    Returns 0 for a local or non-symbolic
    expression, which have no linker name;
    `%P` then falls back to the operand's
    materialised register. */
static const char *kcc_asm_operand_lval(void *lv);

/* A string-literal inline-asm operand (through
    casts/parens), or 0. */
static void *kcc_asm_operand_strlit(void *expr) {
    if (!expr) return 0;
    unsigned long tag = IL(expr, AST_TAG);
    while (tag == AST_PAREN_EXPR) { expr = (void
    *)IL(expr, AST_F0); if (!expr) return 0; tag =
    IL(expr, AST_TAG); }
    if (tag == AST_CAST_EXPR) return
    kcc_asm_operand_strlit((void *)IL(expr, AST_F1)
    );
    if (tag == AST_STR_LIT) return expr;
    return 0;
}

/* Constant byte offset accumulated while
    resolving an inline-asm symbol operand
```

```
   like `&arr[3]' or `&s.field': the operand
   renders as `name+offset'. */
static long kcc_asm_sym_off = 0;

static const char *kcc_asm_operand_symname(void *
   expr) {
    if (!expr) return 0;
    unsigned long tag = IL(expr, AST_TAG);
    while (tag == AST_PAREN_EXPR) { expr = (void
   *)IL(expr, AST_F0); if (!expr) return 0; tag =
   IL(expr, AST_TAG); }
    if (tag == AST_CAST_EXPR) return
   kcc_asm_operand_symname((void *)IL(expr, AST_F1
   ));
    if (tag == AST_UNARY_EXPR && IL(expr, AST_F0)
   == TOK_AMP)
        return kcc_asm_operand_lval((void *)IL(
   expr, AST_F1));      /* &lvalue */
    if (tag == AST_BINARY_EXPR) {
                     /* sym +/- const */
        unsigned long op = IL(expr, AST_F0);
        void *L = (void *)IL(expr, AST_F1), *R = (
   void *)IL(expr, AST_F2);
        if (op == TOK_PLUS || op == TOK_MINUS) {
            const char *s =
   kcc_asm_operand_symname(L);
            if (s && kcc_expr_is_ice(R)) {
                long o = (long)kcc_eval_const_int(
   R);
                kcc_asm_sym_off += (op == TOK_PLUS
   ) ? o : -o;
                return s;
            }
            if (op == TOK_PLUS) {
                s = kcc_asm_operand_symname(R);
                if (s && kcc_expr_is_ice(L)) {
   kcc_asm_sym_off += (long)kcc_eval_const_int(L);
    return s; }
            }
        }
        return 0;
    }
    if (tag != AST_IDENT_EXPR) return 0;
    char *name = (char *)IL(expr, AST_F0);
    if (!name) return 0;
    unsigned long keylen = kcc_strlen(name);
    if (kcc_ir_locals && kcc_ht_get(kcc_ir_locals,
    name, keylen)) return 0;  /* a local */
    if (kcc_ir_statics) {
        void *mangled = kcc_ht_get(kcc_ir_statics,
    name, keylen);
        if (mangled) return (const char *)mangled;
                    /* static: mangled name */
    }
    void *sym = kcc_symtab_lookup(name, keylen);
    if (sym && IL(sym, SYM_KIND) == SYM_ENUM_CONST
   ) return 0;       /* an enum const is a value
   */
    return name;
                    /* file-scope function/object
    */
}

/* Resolve the base symbol (and add the constant
   byte offset) of an lvalue whose
   address is taken: an object `x', an array
   element `a[k]' (k an ICE), or a
   struct member `s.field'.  Nested forms compose
   ---the kernel's cpu-feature byte
   is `&((char *)boot_cpu_data.x86_capability)[bit
    >> 3]'. */
static const char *kcc_asm_operand_lval(void *lv)
   {
    if (!lv) return 0;
    unsigned long tag = IL(lv, AST_TAG);
    while (tag == AST_PAREN_EXPR) { lv = (void *)
   IL(lv, AST_F0); if (!lv) return 0; tag = IL(lv,
    AST_TAG); }
    if (tag == AST_CAST_EXPR) return
   kcc_asm_operand_lval((void *)IL(lv, AST_F1));
    if (tag == AST_IDENT_EXPR) return
   kcc_asm_operand_symname(lv);
    if (tag == AST_SUBSCRIPT) {
        void *idx = (void *)IL(lv, AST_F1);
        if (!kcc_expr_is_ice(idx)) return 0;
        void *et = (void *)IL(lv, AST_F3);
       /* the element's (result) type */
        unsigned long esz = et ? kcc_type_layout(
   et) : 1;
        kcc_asm_sym_off += (long)
   kcc_eval_const_int(idx) * (long)esz;
        return kcc_asm_operand_lval((void *)IL(lv,
    AST_F0));
    }
    if (tag == AST_MEMBER) {
        void *obj = (void *)IL(lv, AST_F0);
        void *st = (void *)IL(obj, AST_F3);
        if (st && IL(st, TY_KIND) == TY_PTR) st =
   (void *)IL(st, TY_F0);
        if (st && (IL(st, TY_KIND) == TY_STRUCT ||
    IL(st, TY_KIND) == TY_UNION)) {
            kcc_type_layout(st);
            struct kcc_member_res mr =
   kcc_type_find_member(st, (const char *)IL(lv,
   AST_F1));
            if (mr.field) kcc_asm_sym_off += (long
   )mr.off;
        }
        /* `s.m': the address is &s + offset.  `p
   ->m': the address is p (a pointer
           value---a constant only when p is
   itself `&sym') + offset; the kernel's
           static_branch takes `&(&key)->key'. */
        if (IL(lv, AST_F2)) return
   kcc_asm_operand_symname(obj);
        return kcc_asm_operand_lval(obj);
    }
    return 0;
}

/* If an inline-asm operand expression (through
   casts/parens) names a scalar
   local that was recorded as a compile-time
   constant, store its value in *out
   and return 1.  Matched on the live `lv' so a
   stale box left by a departed
   inner scope or a prior function never folds.
   x86 only. */
static int kcc_asm_operand_local_const(void *expr,
    unsigned long *out) {
    if (!kcc_target || !kcc_ir_local_consts || !
   expr) return 0;
    unsigned long tag = IL(expr, AST_TAG);
    if (tag == AST_PAREN_EXPR) return
   kcc_asm_operand_local_const((void *)IL(expr,
   AST_F0), out);
    if (tag == AST_CAST_EXPR) return
   kcc_asm_operand_local_const((void *)IL(expr,
   AST_F1), out);
    if (tag != AST_IDENT_EXPR) return 0;
    char *name = (char *)IL(expr, AST_F0);
```

```
    if (!name) return 0;
    unsigned long keylen = kcc_strlen(name);
    unsigned long lv = (unsigned long)kcc_ht_get(
   kcc_ir_locals, name, keylen);
    unsigned long *box = (unsigned long *)
   kcc_ht_get(kcc_ir_local_consts, name, keylen);
    if (box && lv && box[0] == lv) { *out = box
   [1]; return 1; }
    return 0;
}

/* Whether an asm operand's expression is ADDRESS-
   like---its symbol/address is
   the thing the asm wants, so recording a name
   for `%P'/plain-`%N' rendering is
   correct: a function designator (`call %P[f]'),
   an `&lvalue' (`"i"(&sym)'), or a
   symbol +/- constant.  A bare DATA object read
   as a value (`"Nd"(pmtmr_ioport)'
   for `inl') is NOT address-like: its value must
   sit in a register (`%dx'), and
   marking it symbolic would wrongly emit `inl
   pmtmr_ioport(%rip),%eax'. */
static int kcc_asm_operand_is_addr_like(void *e) {
    if (!e) return 0;
    unsigned long tag = IL(e, AST_TAG);
    while (tag == AST_PAREN_EXPR) { e = (void *)IL
   (e, AST_F0); if (!e) return 0; tag = IL(e,
   AST_TAG); }
    if (tag == AST_CAST_EXPR) return
   kcc_asm_operand_is_addr_like((void *)IL(e,
   AST_F1));
    if (tag == AST_UNARY_EXPR && IL(e, AST_F0) ==
   TOK_AMP) return 1;   /* &lvalue */
    if (tag == AST_BINARY_EXPR) {
        unsigned long op = IL(e, AST_F0);
        if (op == TOK_PLUS || op == TOK_MINUS)
            return kcc_asm_operand_is_addr_like((
   void *)IL(e, AST_F1)) ||
                   kcc_asm_operand_is_addr_like((
   void *)IL(e, AST_F2));
        return 0;
    }
    if (tag == AST_IDENT_EXPR) {
        char *name = (char *)IL(e, AST_F0);
        if (!name) return 0;
        void *sym = kcc_symtab_lookup(name,
   kcc_strlen(name));
        return sym && IL(sym, SYM_KIND) ==
   SYM_FUNC;  /* function designator = its address
    */
    }
    return 0;
}

void kcc_ir_asm_lower_operands(void *as) {
    void *op = (void *)IL(as, AST_F2);
    while (op) {
        void *expr = (void *)IL(op, AST_F2);
        unsigned long size = 8;
        void *ety = (void *)IL(expr, AST_F3);
        if (ety && IL(ety, TY_SIZE)) size = IL(ety
   , TY_SIZE);
        char *cons = (char *)IL(op, AST_F0);
        char c = cons[0];
        char cls = (c == '=' || c == '+') ? cons
   [1] : c;
        kcc_asm_sym_off = 0;
        const char *sym = kcc_target ?
   kcc_asm_operand_symname(expr) : 0;
        long symoff = kcc_asm_sym_off;
        void *strlit = kcc_target ?
kcc_asm_operand_strlit(expr) : 0;
        /* An `"i"'-class immediate value: a
genuine integer constant expression,
           or a scalar local recorded as const-
initialised (the kernel's
           `__auto_type __flags = ...' behind WARN
's `"i"(flags)'). */
        unsigned long asm_imm = 0;
        int asm_is_imm = 0;
        if (kcc_target && c != '=' && c != '+' &&
kcc_asm_cons_imm(cons)) {
            if (kcc_expr_is_ice(expr)) { asm_imm =
 kcc_eval_const_int(expr); asm_is_imm = 1; }
            else if (kcc_asm_operand_local_const(
expr, &asm_imm)) asm_is_imm = 1;
        }
        /* A constant that fits none of the
constraint's immediate classes but
           could sit in a register (the percpu
refcount's PERCPU_COUNT_BIAS =
           1UL<<63 fed to `addq $val,mem'
overflowing the 32-bit ALU field; an
           `"Ir"' bit number > 31 fed to `bts') is
 handed to the register path so
           it is loaded first, exactly as GCC
picks the register alternative. */
        if (asm_is_imm && kcc_asm_cons_reg(cons)
&&
            !kcc_asm_imm_fits(cons, (long)asm_imm)
)
            asm_is_imm = 0;
        if (strlit && c != '=' && c != '+' &&
kcc_asm_cons_imm(cons)) {
            /* string-literal `"i"'/`"s"' operand:
 the back end emits the string
               to .rodata and references its label
---the kernel's WARN/BUG entries
               store `"i"(__FILE__)' as `.long %c0
 - .' in __bug_table.  Stash the
               bytes/length (COL/F3) under a
string marker (F2 bit 61); no
               register is materialised. */
            IL(op, AST_COL) = IL(strlit, AST_F0);
            IL(op, AST_F3) = IL(strlit, AST_F1);
            IL(op, AST_F2) = (1UL << 61);
        } else if (asm_is_imm) {
            /* x86 immediate operand: a constant
input the back end renders as
               $value.  Only on x86 (ARM64's asm
path materialises it in a
               register instead); a matching
register alternative ("Ir") on a
               non-constant value falls through to
 the register path below.  The
               value goes in F3, NOT F1: F1 must
stay the operand's `[name]` so a
               `%[name]` reference (the kernel's
`%[bitnum]`) still resolves---the
               marker bit alone tells the back end
 this operand is a constant. */
            IL(op, AST_F2) = (1UL << 63);
/* immediate marker */
            IL(op, AST_F3) = asm_imm;
        } else if (cls == 'm') {
            /* memory operand ("m"/"=m"/"+m"): the
 asm accesses the object
               through its address, so we only
need that address---no register
               value is loaded and nothing is
stored back.  The address goes in
               AST_F3, as for an output, and the
```

```
back end tells the two apart by
           the constraint spelling. */
        void *u = kcc_ast_alloc(AST_UNARY_EXPR
);
        IL(u, AST_F0) = TOK_AMP;
        IL(u, AST_F1) = IL(op, AST_F2);
        IL(u, AST_F2) = 1;
        unsigned long addr = kcc_ir_build_expr
(u);
        IL(op, AST_F3) = ((addr + 1) | (size
<< 32));
        IL(op, AST_F2) = 0;
    } else if (c == '=') {
        /* output: address only. asm_addr_of
returns the vreg via build_expr */
        void *u = kcc_ast_alloc(AST_UNARY_EXPR
);
        IL(u, AST_F0) = TOK_AMP;
        IL(u, AST_F1) = IL(op, AST_F2);
        IL(u, AST_F2) = 1;
        unsigned long addr = kcc_ir_build_expr
(u);
        IL(op, AST_F3) = ((addr + 1) | (size
<< 32));
        IL(op, AST_F2) = 0;
    } else if (c == '+') {
        unsigned long val = kcc_ir_build_expr
((void *)IL(op, AST_F2));
        unsigned long vhalf = (val + 1) | (
size << 32);
        void *u = kcc_ast_alloc(AST_UNARY_EXPR
);
        IL(u, AST_F0) = TOK_AMP;
        IL(u, AST_F1) = IL(op, AST_F2);
        IL(u, AST_F2) = 1;
        unsigned long addr = kcc_ir_build_expr
(u);
        IL(op, AST_F3) = ((addr + 1) | (size
<< 32));
        IL(op, AST_F2) = vhalf;
    } else {
        unsigned long val = kcc_ir_build_expr
((void *)IL(op, AST_F2));
        IL(op, AST_F2) = ((val + 1) | (size <<
 32));
        IL(op, AST_F3) = 0;
    }
    /* Record a symbolic operand's emit-name
for `%P'/plain-`%N' rendering,
       unless it took the immediate path (F2
bit 63).  The name goes in the
       unused COL field (the emitter never
reads an operand's source column)
       with F2 bit 62 flagging its presence---
F1 must stay the operand's
       `[name]', which the back end matches to
 resolve a `%[name]' reference.
       We do this only when the operand is
genuinely used as an ADDRESS: a
       memory operand (`cls=='m''), or an
address-like expression (a function
       designator or `&lvalue').  A bare data
object read as a VALUE
       (`"Nd"(pmtmr_ioport)' for `inl') keeps
its materialised register
       instead---marking it symbolic would
emit `inl pmtmr_ioport(%rip),%eax'
       rather than loading the port into `%dx
'. */
    if (sym && !((IL(op, AST_F2) >> 63) & 1)
&&
        (cls == 'm' ||
 kcc_asm_operand_is_addr_like(expr))) {
        IL(op, AST_COL) = (unsigned long)sym;
        IL(op, AST_F2) = IL(op, AST_F2) | (1UL
 << 62);
        IL(op, AST_F3) = (unsigned long)symoff
;   /* byte offset for name+off */
     }
     op = (void *)IL(op, AST_F4);
  }
}

/* -- return -- */
void kcc_ir_build_return(void *rs) {
   void *e = (void *)IL(rs, AST_F0);
   if (!e) {
      void *r = kcc_ir_alloc_instr(IR_RET);
      IL(r, IR_TYPE) = (unsigned long)
 kcc_type_void;
      kcc_ir_emit(r);
      goto terminate;
   }
   void *rettype = (void *)IL(kcc_ir_cur_func,
 IRF_RETTYPE);
   if (rettype) {
      unsigned long rk = IL(rettype, TY_KIND);
      if (rk == TY_STRUCT || rk == TY_UNION) {
         unsigned long src =
 kcc_ir_build_struct_addr(e);
         void *rty = (void *)IL(kcc_ir_cur_func
, IRF_RETTYPE);
         if (kcc_ir_classify_type(rty) == 4) {
            unsigned long sz = kcc_type_layout
(rty);
            /* hidden sret pointer = last
reserved parameter vreg */
            void *fn = kcc_ir_cur_func;
            unsigned long np = IL(fn,
IRF_PARAMS);
            void *cls = (void *)IL(fn,
IRF_PARAM_CLS);
            unsigned long w = 0, ix = 0;
            while (ix < np) {
               w += 1;
               if (cls && IL(cls, ix * 8) ==
3) w += 1;
               ix++;
            }
            unsigned long dst = w - 1;
            kcc_ir_emit_struct_copy(src, dst,
sz);
            void *r = kcc_ir_alloc_instr(
IR_RET);
            if (kcc_target) {
               /* x86-64 SysV returns a
memory-class aggregate by placing
                  its address back in %rax;
the caller reads the result
                  through that register, so
the callee must hand the hidden
                  sret pointer (the last
parameter vreg) back---not merely
                  write through it.  A non-
void RET carrying that vreg makes
                  the emitter load it into %
rax.  (AArch64's caller reads its
                  own x8 buffer, so its RET
stays void: unchanged.) */
               IL(r, IR_TYPE) = (unsigned
long)rty;
               IL(r, IR_SRC1) = dst;
```

```
                } else {
                    IL(r, IR_TYPE) = (unsigned
    long)kcc_type_void;
                }
                kcc_ir_emit(r);
            } else {
                void *r = kcc_ir_alloc_instr(
    IR_RET);
                IL(r, IR_TYPE) = (unsigned long)
    rty;
                IL(r, IR_SRC1) = src;
                kcc_ir_emit(r);
            }
            goto terminate;
        }
     }
     /* scalar return */
     {
         unsigned long v = kcc_ir_build_expr(e);
         void *to = (void *)IL(kcc_ir_cur_func,
    IRF_RETTYPE);
         void *from = (void *)IL(e, AST_F3);
         if (to && from) v = kcc_ir_convert(v, from
    , to);
         void *r = kcc_ir_alloc_instr(IR_RET);
         IL(r, IR_TYPE) = (unsigned long)IL(
    kcc_ir_cur_func, IRF_RETTYPE);
         IL(r, IR_SRC1) = v;
         kcc_ir_emit(r);
     }
terminate: {
     void *cur = kcc_ir_cur_bb;
     if (cur) IL(cur, BB_FLAGS) = IL(cur, BB_FLAGS)
     | BB_FLAG_TERMINATED;
     }
}
```

□

841 · used in 902a

75.4. Expressions

The expression dispatcher returns the vreg holding each subexpression's value, so builders compose by feeding one result into another's operand slot. This first group covers the dispatcher itself, the literals, the int/float conversions the assignment and return paths lean on, AAPCS64 parameter classification, `sizeof`/`_Alignof`, the access-width helpers, and array/pointer subscripting.

⟨ *Selfhost: ir exprs* 856 ⟩ ≡

```
unsigned long kcc_ir_build_expr(void *e) {
    if (!e) return 0;
    unsigned long tag = IL(e, AST_TAG);
    if (tag == AST_INT_LIT) return
    kcc_ir_build_int_lit(e);
     if (tag == AST_FLOAT_LIT) return
    kcc_ir_build_float_lit(e);
     if (tag == AST_CHAR_LIT) return
    kcc_ir_build_int_lit(e);
     if (tag == AST_IDENT_EXPR) return
    kcc_ir_build_ident(e);
     if (tag == AST_BINARY_EXPR) return
    kcc_ir_build_binary(e);
     if (tag == AST_UNARY_EXPR) return
    kcc_ir_build_unary(e);
     if (tag == AST_ASSIGN_EXPR) return
kcc_ir_build_assign(e);
     if (tag == AST_CALL_EXPR) return
kcc_ir_build_call(e);
     if (tag == AST_CAST_EXPR) return
kcc_ir_build_cast(e);
     if (tag == AST_COMMA_EXPR) {
         kcc_ir_build_expr((void *)IL(e, AST_F0));
         return kcc_ir_build_expr((void *)IL(e,
AST_F1));
     }
     if (tag == AST_COND_EXPR) return
kcc_ir_build_cond_expr(e);
     if (tag == AST_PAREN_EXPR) return
kcc_ir_build_expr((void *)IL(e, AST_F0));
     if (tag == AST_STR_LIT) return
kcc_ir_build_string_lit(e);
     if (tag == AST_SIZEOF_EXPR) return
kcc_ir_build_sizeof(e);
     if (tag == AST_ALIGNOF_EXPR) return
kcc_ir_build_alignof(e);
     if (tag == AST_SUBSCRIPT) return
kcc_ir_build_subscript(e);
     if (tag == AST_MEMBER) return
kcc_ir_build_member(e);
     if (tag == AST_VA_ARG) return
kcc_ir_build_va_arg(e);
     if (tag == AST_TYPES_COMPAT) return
kcc_ir_emit_const(kcc_eval_const_int(e));
     if (tag == AST_GENERIC) return
kcc_ir_build_expr((void *)IL(e, AST_F2));
     if (tag == AST_COMPOUND_LIT) {
         unsigned long addr =
kcc_ir_build_compound_lit(e);
         void *ty = kcc_sema_resolve_type((void *)
IL(e, AST_F0));
         if (ty) {
             unsigned long k = IL(ty, TY_KIND);
             if (k == TY_STRUCT || k == TY_UNION ||
 k == TY_ARRAY) return addr;
             unsigned long sz = kcc_ir_store_width(
ty);
             return kcc_ir_emit_load(addr, sz);
         }
         return addr;
     }
     if (tag == AST_LABEL_ADDR) {
         /* GNU `&&label': materialise the address
of the label's basic block.
            f2 set means the target BB was pre-
resolved and stashed in f1 while
            its function was built (a static
jumptable initialised elsewhere
            references labels this function owns);
otherwise the label is in the
            function being built now, so resolve it
 through the live label map. */
         void *bb = IL(e, AST_F2) ? (void *)IL(e,
AST_F1)
                                  : kcc_ir_label_bb
((const char *)IL(e, AST_F0));
         void *g = kcc_ir_alloc_instr(IR_LADDR);
         IL(g, IR_SRC1) = bb ? IL(bb, BB_ID) : 0;
         IL(g, IR_TYPE) = (unsigned long)
kcc_type_long_u;
         unsigned long rv = kcc_ir_next_vreg();
         IL(g, IR_DEST) = rv;
         kcc_ir_emit(g);
         return rv;
     }
```

```
    if (tag == AST_STMT_EXPR) return
   kcc_ir_build_stmt_expr(e);
    return 0;
}

unsigned long kcc_ir_build_stmt_expr(void *se) {
    void *comp = (void *)IL(se, AST_F0);
    if (!comp) return 0;
    kcc_ir_scope_enter();
    /* Register the block's labels: the function-
   level prescan does not descend
       into a statement expression, so a goto
   inside it (the kernel FIND_*_BIT
       macros jump to `out:') would otherwise not
   resolve to the label's block. */
    kcc_ir_prescan_labels(comp);
    void *it = (void *)IL(comp, AST_F0);
    unsigned long val = 0;
    if (!it) { kcc_ir_scope_leave(); return 0; }
    while (it) {
        void *next = (void *)IL(it, AST_F4);
        if (!next) {
            /* The final item carries the
   statement-expression's value.  Peel
               off any labels first---emitting
   each label's block so a goto can
               still reach it---so a `label: expr
   ;' tail (as the `out:' target
               inside the kernel's FIND_*_BIT
   macros) yields its expression's
               value rather than being discarded.
   */
            while (it && IL(it, AST_TAG) ==
   AST_LABEL_STMT) {
                void *bb = kcc_ir_label_bb((const
   char *)IL(it, AST_F0));
                if (bb) { kcc_ir_branch_to(bb);
   kcc_ir_set_bb(bb); }
                it = (void *)IL(it, AST_F1);
            }
            if (it && IL(it, AST_TAG) ==
   AST_EXPR_STMT)
                val = kcc_ir_build_expr((void *)IL
   (it, AST_F0));
            else if (it)
                kcc_ir_build_stmt(it);
            break;
        }
        kcc_ir_build_stmt(it);
        it = next;
    }
    kcc_ir_scope_leave();
    return val;
}

unsigned long kcc_ir_build_int_lit(void *lit) {
    unsigned long value = IL(lit, AST_F0);
    void *inst = kcc_ir_alloc_instr(IR_CONST);
    void *ty = (void *)IL(lit, AST_F3);
    if ((unsigned long)ty < 0x1000) ty =
   kcc_type_int_s;
    IL(inst, IR_TYPE) = (unsigned long)ty;
    IL(inst, IR_SRC1) = value;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_build_float_lit(void *lit) {
    unsigned long bits = IL(lit, AST_F0);
    void *inst = kcc_ir_alloc_instr(IR_CONST);
    IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_double;
    IL(inst, IR_SRC1) = bits;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_type_is_fp(void *ty) {
    if (!ty) return 0;
    unsigned long k = IL(ty, TY_KIND);
    if (k == TY_FLOAT || k == TY_DOUBLE) return 1;
    return 0;
}

unsigned long kcc_ir_classify_type(void *ty) {
    if (!ty) return 0;
    if (kcc_ir_type_is_fp(ty)) return 1;
    unsigned long k = IL(ty, TY_KIND);
    if (k == TY_STRUCT || k == TY_UNION) {
        unsigned long sz = kcc_type_layout(ty);
        if (sz <= 8) return 2;
        if (sz <= 16) return 3;
        return 4;
    }
    return 0;
}

unsigned long kcc_ir_emit_sitofp(unsigned long src
   ) {
    void *inst = kcc_ir_alloc_instr(IR_SITOFP);
    IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_double;
    IL(inst, IR_SRC1) = src;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_emit_uitofp(unsigned long src
   ) {
    void *inst = kcc_ir_alloc_instr(IR_UITOFP);
    IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_double;
    IL(inst, IR_SRC1) = src;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_emit_int_to_fp(unsigned long
   src, void *it) {
    if (it && kcc_sema_int_signedness(it) == 0)
   return kcc_ir_emit_uitofp(src);
    return kcc_ir_emit_sitofp(src);
}

unsigned long kcc_ir_emit_fpround(unsigned long
   src) {
    void *inst = kcc_ir_alloc_instr(IR_FPROUND);
    IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_float;
    IL(inst, IR_SRC1) = src;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
```

```
}

unsigned long kcc_ir_emit_fptosi(unsigned long src
   ) {
    void *inst = kcc_ir_alloc_instr(IR_FPTOSI);
    IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    IL(inst, IR_SRC1) = src;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_convert(unsigned long vreg,
   void *from, void *to) {
    unsigned long to_fp = kcc_ir_type_is_fp(to);
    unsigned long from_fp = kcc_ir_type_is_fp(from
   );
    if (from_fp != to_fp) {
        if (to_fp) vreg = kcc_ir_emit_int_to_fp(
   vreg, from);
        else vreg = kcc_ir_emit_fptosi(vreg);
    }
    /* to float: round to single precision */
    if (to && IL(to, TY_KIND) == TY_FLOAT) vreg =
   kcc_ir_emit_fpround(vreg);
    /* to _Bool: normalise non-zero to 1 */
    if (to && IL(to, TY_KIND) == TY_BOOL) return
   kcc_ir_emit_bool_norm(vreg);
    /* integer-to-integer implicit conversion: re-
   canonicalise width/signedness */
    if (to) {
        unsigned long tk = IL(to, TY_KIND);
        if (tk >= TY_BOOL && tk <= TY_LLONG &&
   from) {
            unsigned long fk = IL(from, TY_KIND);
            if (fk >= TY_BOOL && fk <= TY_LLONG) {
                unsigned long ts = IL(to, TY_SIZE)
   ;
                unsigned long fs = IL(from,
   TY_SIZE);
                unsigned long resize = 0;
                if (ts != fs) resize = 1;
                else if (ts < 8 && IL(to, TY_F0)
   != IL(from, TY_F0)) resize = 1;
                if (resize) {
                    unsigned long op = IL(to,
   TY_F0) ? IR_SEXT : IR_ZEXT;
                    void *inst =
   kcc_ir_alloc_instr(op);
                    IL(inst, IR_TYPE) = (unsigned
   long)to;
                    IL(inst, IR_SRC1) = vreg;
                    IL(inst, IR_SRC2) = IL(to,
   TY_SIZE);
                    vreg = kcc_ir_next_vreg();
                    IL(inst, IR_DEST) = vreg;
                    kcc_ir_emit(inst);
                }
            }
        }
    }
    return vreg;
}

void kcc_ir_fill_param_classes(void *fn, void *fd)
    {
    unsigned long np = IL(fn, IRF_PARAMS);
    if (!np) return;
    void *arr = kcc_arena_alloc(np * 8);
    IL(fn, IRF_PARAM_CLS) = (unsigned long)arr;
    unsigned long i = 0;
    while (i < np) { IL(arr, i * 8) = 0; i++; }
    void *p = (void *)(IL(fd, AST_F2) & ~3UL);
    i = 0;
    while (p) {
        if (IL(p, AST_TAG) == AST_PARAM_DECL && IL
   (p, AST_F1)) {
            void *t = kcc_sema_resolve_type((void
   *)IL(p, AST_F0));
            t = kcc_type_param_adjust(t);
            IL(arr, i * 8) = kcc_ir_classify_type(
   t);
            i++;
        }
        p = (void *)IL(p, AST_F4);
    }
    /* >16B struct return: trailing hidden x8
   pointer (class 5) */
    if (kcc_ir_classify_type((void *)IL(fn,
   IRF_RETTYPE)) == 4)
        IL(arr, (np - 1) * 8) = 5;
}

unsigned long kcc_ir_build_sizeof(void *e) {
    void *ty;
    if (IL(e, AST_F1))
   /* sizeof(type-name) */
        ty = kcc_sema_resolve_type((void *)IL(e,
   AST_F0));
    else
   /* sizeof expr: operand's type */
        ty = (void *)IL((void *)IL(e, AST_F0),
   AST_F3);
    unsigned long sz = kcc_type_layout(ty);
    void *inst = kcc_ir_alloc_instr(IR_CONST);
    IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_long_u;
    IL(inst, IR_SRC1) = sz;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_build_alignof(void *e) {
    void *ty;
    if (IL(e, AST_F1))
   /* _Alignof(type-name) */
        ty = kcc_sema_resolve_type((void *)IL(e,
   AST_F0));
    else
   /* __alignof__ expr: operand's type */
        ty = (void *)IL((void *)IL(e, AST_F0),
   AST_F3);
    kcc_type_layout(ty);
    unsigned long al = IL(ty, TY_ALIGN);
    void *inst = kcc_ir_alloc_instr(IR_CONST);
    IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_long_u;
    IL(inst, IR_SRC1) = al;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_type_slots(void *ty) {
    if (!ty) return 1;
    unsigned long sz = kcc_type_layout(ty);
```

```
    if (sz <= 8) return 1;
    return (sz + 7) >> 3;
}

unsigned long kcc_ir_store_width(void *ty) {
    if (!ty) return 0;
    unsigned long k = IL(ty, TY_KIND);
    if (k == TY_FLOAT) return 8;
    if (k == TY_BOOL) return 1;
    if (k != TY_ENUM) {
        if (k < TY_CHAR) return 0;
        if (k > TY_LLONG) return 0;
    }
    unsigned long sz = IL(ty, TY_SIZE);
    if (sz == 4 || sz == 2 || sz == 1) return sz;
    return 0;
}

unsigned long kcc_ir_load_width(void *ty) {
    if (!ty) return 0;
    unsigned long k = IL(ty, TY_KIND);
    unsigned long sz = IL(ty, TY_SIZE);
    if (k == TY_FLOAT) return 8;
    unsigned long signd;
    if (k == TY_BOOL) signd = 0;
    else if (k == TY_ENUM) signd = 1;
    else {
        if (k < TY_CHAR) return 0;
        if (k > TY_LLONG) return 0;
        signd = IL(ty, TY_F0) ? 1 : 0;
    }
    if (signd) {
        if (sz == 1) return 5;
        if (sz == 2) return 6;
        if (sz == 4) return 7;
        return 0;
    }
    if (sz == 1) return 1;
    if (sz == 2) return 2;
    if (sz == 4) return 4;
    return 0;
}
```

□

856 · defined at 856, 859, 860, 862, 865, 872 · used in 902a

⟨ *Selfhost: ir exprs* 859 ⟩ + ≡

```
/* Element address for e[i], shared by the read
   and write paths.  The gotos
   mirror the assembly's labels: `index` computes
   base+scaled-index via GEP,
   `ptr` loads a pointer then adds a scaled index.
    */
unsigned long kcc_ir_build_subscript_addr(void *
   sub) {
    unsigned long base_addr = 0;
    void *base = (void *)IL(sub, AST_F0);
    unsigned long btag = IL(base, AST_TAG);
    if (btag == AST_SUBSCRIPT) {
        void *it = (void *)IL(base, AST_F3);
   /* inner subscript's type */
        if (it && IL(it, TY_KIND) != TY_ARRAY)
            base_addr = kcc_ir_build_expr(base);
          /* pointer element: load */
        else
            base_addr =
   kcc_ir_build_subscript_addr(base); /* sub-array
    address */
        goto index;
    }
    if (btag == AST_MEMBER) {
        void *mt = (void *)IL(base, AST_F3);
        if (mt && IL(mt, TY_KIND) == TY_ARRAY) {
            base_addr = kcc_ir_build_member_addr(
   base);
            goto index;
        }
        goto ptr;
    }
    if (btag == AST_IDENT_EXPR) {
        char *name = (char *)IL(base, AST_F0);
        unsigned long v = (unsigned long)
   kcc_ht_get(kcc_ir_locals, name, kcc_strlen(name
   ));
        if (v) {
            if (v & 0x8000000000000000UL) goto ptr
   ;       /* pointer local */
            base_addr = v - 1;
           /* array/scalar alloca */
            goto index;
        }
    }
    /* expr fallback: an ident not in locals (
   static/global array), else ptr */
    base = (void *)IL(sub, AST_F0);
    if (IL(base, AST_TAG) != AST_IDENT_EXPR) goto
   ptr;
    {
        char *name = (char *)IL(base, AST_F0);
        unsigned long namelen = kcc_strlen(name);
        if (kcc_ir_statics) {
            void *mangled = kcc_ht_get(
   kcc_ir_statics, name, namelen);
            if (mangled) {
                void *stt = kcc_ir_static_types
                            ? kcc_ht_get(
   kcc_ir_static_types, name, namelen) : 0;
                if (!stt || IL(stt, TY_KIND) !=
   TY_ARRAY) goto ptr;
                void *g = kcc_ir_alloc_instr(
   IR_GLOBAL_REF);
                IL(g, IR_SRC1) = (unsigned long)
   mangled;
                IL(g, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
                base_addr = kcc_ir_next_vreg();
                IL(g, IR_DEST) = base_addr;
                kcc_ir_emit(g);
                goto index;
            }
        }
        void *sym = kcc_symtab_lookup(name,
   namelen);
        if (!sym) goto ptr;
        void *st = (void *)IL(sym, SYM_TYPE);
        if (!st || IL(st, TY_KIND) != TY_ARRAY)
   goto ptr;
        void *g = kcc_ir_alloc_instr(IR_GLOBAL_REF
   );
        IL(g, IR_SRC1) = (unsigned long)name;
        IL(g, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        base_addr = kcc_ir_next_vreg();
        IL(g, IR_DEST) = base_addr;
        kcc_ir_emit(g);
        goto index;
    }
index: {
    unsigned long idx = kcc_ir_build_expr((void *)
   IL(sub, AST_F1));
```

```
    void *et = (void *)IL(sub, AST_F3);
   /* element type */
    if (et) {
        unsigned long esz = kcc_type_layout(et);
        if (esz > 1) {
            void *c = kcc_ir_alloc_instr(IR_CONST)
   ;
            IL(c, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
            IL(c, IR_SRC1) = esz;
            unsigned long cv = kcc_ir_next_vreg();
            IL(c, IR_DEST) = cv;
            kcc_ir_emit(c);
            void *m = kcc_ir_alloc_instr(IR_MUL);
            IL(m, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
            IL(m, IR_SRC1) = idx;
            IL(m, IR_SRC2) = cv;
            idx = kcc_ir_next_vreg();
            IL(m, IR_DEST) = idx;
            kcc_ir_emit(m);
        }
    }
    void *gep = kcc_ir_alloc_instr(IR_GEP);
    IL(gep, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    IL(gep, IR_SRC1) = base_addr;
    IL(gep, IR_SRC2) = idx;
    unsigned long gv = kcc_ir_next_vreg();
    IL(gep, IR_DEST) = gv;
    kcc_ir_emit(gep);
    return gv;
}
ptr: {
    unsigned long pv = kcc_ir_build_expr((void *)
   IL(sub, AST_F0));
    unsigned long idx = kcc_ir_build_expr((void *)
   IL(sub, AST_F1));
    void *pt = (void *)IL((void *)IL(sub, AST_F0),
    AST_F3);  /* base's TY_PTR */
    void *pointee = pt ? (void *)IL(pt, TY_F0) :
   0;
    if (pointee) {
        unsigned long psz = kcc_type_layout(
   pointee);
        if (psz > 1) {
            void *c = kcc_ir_alloc_instr(IR_CONST)
   ;
            IL(c, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
            IL(c, IR_SRC1) = psz;
            unsigned long cv = kcc_ir_next_vreg();
            IL(c, IR_DEST) = cv;
            kcc_ir_emit(c);
            void *m = kcc_ir_alloc_instr(IR_MUL);
            IL(m, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
            IL(m, IR_SRC1) = idx;
            IL(m, IR_SRC2) = cv;
            idx = kcc_ir_next_vreg();
            IL(m, IR_DEST) = idx;
            kcc_ir_emit(m);
        }
    }
    void *add = kcc_ir_alloc_instr(IR_ADD);
    IL(add, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    IL(add, IR_SRC1) = pv;
    IL(add, IR_SRC2) = idx;
    unsigned long av = kcc_ir_next_vreg();
    IL(add, IR_DEST) = av;
    kcc_ir_emit(add);
    return av;
}
}

unsigned long kcc_ir_build_subscript(void *sub) {
    unsigned long addr =
   kcc_ir_build_subscript_addr(sub);
    void *et = (void *)IL(sub, AST_F3);
   /* element type */
    if ((unsigned long)et >= 0x1000) {
        unsigned long k = IL(et, TY_KIND);
        if (k == TY_ARRAY || k == TY_FUNC) return
   addr;  /* array/func decay */
    }
    unsigned long width = kcc_ir_load_width(et);
    void *lty;
    if ((unsigned long)et >= 0x1000 && (IL(et,
   TY_QUALS) & 2)) lty = et;
    else lty = kcc_type_int_s;
    void *ld = kcc_ir_alloc_instr(IR_LOAD);
    IL(ld, IR_TYPE) = (unsigned long)lty;
    IL(ld, IR_SRC1) = addr;
    IL(ld, IR_SRC2) = width;
    unsigned long v = kcc_ir_next_vreg();
    IL(ld, IR_DEST) = v;
    kcc_ir_emit(ld);
    return v;
}
```

□

859 · defined at 856, 859, 860, 862, 865, 872 · used in 902a

75.5. Member Access and Bit-fields

Member address resolution (shared by reads and writes) peels parentheses, finds the object's base address (loading through a pointer for arrow access), then adds the field's byte offset via `kcc_type_find_member`. A bit-field is synthesised from shifts and masks; a `volatile` bit-field carries a volatile-qualified container type so the optimizer preserves every access.

⟨ *Selfhost: ir exprs* 860 ⟩ + ≡

```
unsigned long kcc_ir_build_member_addr(void *m) {
    unsigned long base_addr = 0;
    /* parentheses are transparent: peel them,
   rewriting the object in place */
    void *obj = (void *)IL(m, AST_F0);
    while (obj && IL(obj, AST_TAG) ==
   AST_PAREN_EXPR) {
        obj = (void *)IL(obj, AST_F0);
        IL(m, AST_F0) = (unsigned long)obj;
    }
    obj = (void *)IL(m, AST_F0);
    if (IL(obj, AST_TAG) == AST_IDENT_EXPR) {
        char *name = (char *)IL(obj, AST_F0);
        unsigned long v = (unsigned long)
   kcc_ht_get(kcc_ir_locals, name, kcc_strlen(name
   ));
        if (v) {
            unsigned long arrow = IL(m, AST_F2);
            if (arrow && (v & 0x8000000000000000UL
   )) {
                unsigned long alloca = (v & ~0
   x8000000000000000UL) - 1;
```

```
            void *ld = kcc_ir_alloc_instr(
IR_LOAD);
            IL(ld, IR_TYPE) = (unsigned long)
kcc_type_int_s;
            IL(ld, IR_SRC1) = alloca;
            base_addr = kcc_ir_next_vreg();
            IL(ld, IR_DEST) = base_addr;
            kcc_ir_emit(ld);
        } else {
            base_addr = (v & ~0
x8000000000000000UL) - 1;
        }
        goto find_index;
    }
    /* global or STATIC-LOCAL struct: a static
 local's storage is emitted
       under its mangled `sl_' name, so the
address reference must use that
       name too---otherwise `sp.x' on a `
static struct P sp' emits a
       relocation against the raw `sp' that
nothing defines (the kernel's
       `static const struct printf_spec
str_spec', `static struct tk_read_base
       tkr_dummy', ...). */
    char *symname = name;
    if (kcc_ir_statics) {
        void *mangled = kcc_ht_get(
kcc_ir_statics, name, kcc_strlen(name));
        if (mangled) symname = (char *)mangled
;
    }
    void *g = kcc_ir_alloc_instr(IR_GLOBAL_REF
);
    IL(g, IR_SRC1) = (unsigned long)symname;
    IL(g, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    base_addr = kcc_ir_next_vreg();
    IL(g, IR_DEST) = base_addr;
    kcc_ir_emit(g);
    if (!IL(m, AST_F2)) goto find_index;
/* dot */
    void *sym = kcc_symtab_lookup(name,
kcc_strlen(name));
    if (sym) {
        void *st = (void *)IL(sym, SYM_TYPE);
        if (st && IL(st, TY_KIND) == TY_ARRAY)
 goto find_index;
    }
    void *ld = kcc_ir_alloc_instr(IR_LOAD);
/* arrow on global pointer */
    IL(ld, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    IL(ld, IR_SRC1) = base_addr;
    base_addr = kcc_ir_next_vreg();
    IL(ld, IR_DEST) = base_addr;
    kcc_ir_emit(ld);
    goto find_index;
 }
 {
    unsigned long otag = IL(obj, AST_TAG);
    unsigned long arrow = IL(m, AST_F2);
    if (otag == AST_SUBSCRIPT && !arrow) {
        base_addr =
kcc_ir_build_subscript_addr(obj);
        goto find_index;
    }
    if (otag == AST_MEMBER && !arrow) {
        base_addr = kcc_ir_build_member_addr(
obj);
        goto find_index;
        }
        base_addr = kcc_ir_build_expr(obj);
    /* arrow / other → value */
    }
find_index: {
    unsigned long off = 0;
    void *st = (void *)IL((void *)IL(m, AST_F0),
AST_F3);
    unsigned long resolved = 0;
    if (st) {
        unsigned long k = IL(st, TY_KIND);
        if (k == TY_PTR) { st = (void *)IL(st,
TY_F0); k = st ? IL(st, TY_KIND) : 0; }
        if (st && (k == TY_STRUCT || k == TY_UNION
)) {
            kcc_type_layout(st);
            struct kcc_member_res mr =
kcc_type_find_member(st, (const char *)IL(m,
AST_F1));
            if (mr.field) {
                kcc_ir_member_field = mr.field;
                off = mr.off;
                resolved = 1;
            }
        }
    }
    if (!resolved) kcc_ir_member_field = 0;
    void *c = kcc_ir_alloc_instr(IR_CONST);
    IL(c, IR_SRC1) = off;
    IL(c, IR_TYPE) = (unsigned long)kcc_type_int_s
;
    unsigned long ov = kcc_ir_next_vreg();
    IL(c, IR_DEST) = ov;
    kcc_ir_emit(c);
    void *gep = kcc_ir_alloc_instr(IR_GEP);
    IL(gep, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    IL(gep, IR_SRC1) = base_addr;
    IL(gep, IR_SRC2) = ov;
    unsigned long rv = kcc_ir_next_vreg();
    IL(gep, IR_DEST) = rv;
    kcc_ir_emit(gep);
    return rv;
}
}

unsigned long kcc_ir_build_member(void *m) {
    unsigned long addr = kcc_ir_build_member_addr(
m);
    void *ft = (void *)IL(m, AST_F3);
    if ((unsigned long)ft >= 0x1000) {
        unsigned long k = IL(ft, TY_KIND);
        if (k == TY_ARRAY || k == TY_FUNC) return
addr;   /* decay */
    }
    void *field = kcc_ir_member_field;
    if (field && IL(field, AST_F2)) return
kcc_ir_bitfield_load(addr, field);
    /* scalar: load-type is the field's own
descriptor when volatile or FP */
    void *lty;
    if ((unsigned long)ft >= 0x1000 &&
        ((IL(ft, TY_QUALS) & 2) || IL(ft, TY_KIND)
 == TY_FLOAT || IL(ft, TY_KIND) == TY_DOUBLE))
        lty = ft;
    else
        lty = kcc_type_int_s;
    unsigned long width = kcc_ir_load_width(ft);
    void *ld = kcc_ir_alloc_instr(IR_LOAD);
    IL(ld, IR_TYPE) = (unsigned long)lty;
    IL(ld, IR_SRC1) = addr;
```

```
    IL(ld, IR_SRC2) = width;
    unsigned long v = kcc_ir_next_vreg();
    IL(ld, IR_DEST) = v;
    kcc_ir_emit(ld);
    return v;
}
```

□

860 · defined at 856, 859, 860, 862, 865, 872 · used in 902a

⟨ *Selfhost: ir exprs* 862 ⟩ + ≡

```
/* tiny emitters shared by the bit-field synthesis
    and compound assignment */
unsigned long kcc_ir_emit_const(unsigned long
   value) {
    void *inst = kcc_ir_alloc_instr(IR_CONST);
    IL(inst, IR_SRC1) = value;
    IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_long_u;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_emit_binop_typed(unsigned
   long op, unsigned long a, unsigned long b, void
    *ty) {
    void *inst = kcc_ir_alloc_instr(op);
    IL(inst, IR_TYPE) = (unsigned long)ty;
    IL(inst, IR_SRC1) = a;
    IL(inst, IR_SRC2) = b;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_emit_binop(unsigned long op,
   unsigned long a, unsigned long b) {
    return kcc_ir_emit_binop_typed(op, a, b,
   kcc_type_long_u);
}

unsigned long kcc_ir_compound_apply(unsigned long
   op, unsigned long old, void *L, unsigned long
   rhs, void *R) {
    if (op == IR_ADD || op == IR_SUB) {
        if ((unsigned long)L >= 0x1000 && IL(L,
   TY_KIND) == TY_PTR) {
            void *pointee = (void *)IL(L, TY_F0);
            if (pointee) {
                unsigned long sz = kcc_type_layout
   (pointee);
                if (sz > 1) {
                    unsigned long stride =
   kcc_ir_emit_const(sz);
                    rhs = kcc_ir_emit_binop(IR_MUL
   , rhs, stride);
                }
            }
        }
    }
    if (op == IR_ASHR || op == IR_SDIV || op ==
   IR_SMOD) {
        void *ct = (op == IR_ASHR) ?
   kcc_conv_promote(L) : kcc_conv_usual(L, R);
        if (ct) {
            unsigned long k = IL(ct, TY_KIND);
            if (k >= TY_BOOL && k <= TY_LLONG &&
   IL(ct, TY_F0) == 0) {
                if (op == IR_SDIV) op = IR_UDIV;
                else if (op == IR_SMOD) op =
   IR_UMOD;
                else op = IR_LSHR;
            }
        }
    }
    unsigned long lfp = kcc_ir_type_is_fp(L);
    unsigned long rfp = kcc_ir_type_is_fp(R);
    if (lfp || rfp) {
        void *C = lfp ? L : R;
        unsigned long old_c = kcc_ir_convert(old,
   L, C);
        unsigned long rhs_c = kcc_ir_convert(rhs,
   R, C);
        unsigned long res =
   kcc_ir_emit_binop_typed(op, old_c, rhs_c, C);
        return kcc_ir_convert(res, C, L);
    }
    unsigned long res = kcc_ir_emit_binop_typed(op
   , old, rhs, kcc_type_long_u);
    return kcc_ir_norm32(res, op, L, L);
}

unsigned long kcc_ir_bf_code(unsigned long
   container_bytes) {
    if (container_bytes == 4) return 4;
    if (container_bytes == 2) return 2;
    if (container_bytes == 1) return 1;
    return 0;
}

unsigned long kcc_ir_bf_mask(unsigned long width)
   {
    if (width >= 64) return (unsigned long)-1;
    return (1UL << width) - 1;
}

unsigned long kcc_ir_emit_load_t(unsigned long
   addr, unsigned long size, void *ty) {
    void *inst = kcc_ir_alloc_instr(IR_LOAD);
    IL(inst, IR_TYPE) = (unsigned long)ty;
    IL(inst, IR_SRC1) = addr;
    IL(inst, IR_SRC2) = size;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_emit_load(unsigned long addr,
    unsigned long size) {
    return kcc_ir_emit_load_t(addr, size,
   kcc_type_long_u);
}

void kcc_ir_emit_store_t(unsigned long value,
   unsigned long addr, unsigned long size, void *
   ty) {
    void *inst = kcc_ir_alloc_instr(IR_STORE);
    IL(inst, IR_TYPE) = (unsigned long)ty;
    IL(inst, IR_SRC1) = value;
    IL(inst, IR_SRC2) = addr;
    IL(inst, IR_DEST) = size;
    kcc_ir_emit(inst);
}

void kcc_ir_emit_store(unsigned long value,
   unsigned long addr, unsigned long size) {
```

```
    kcc_ir_emit_store_t(value, addr, size,
   kcc_type_long_u);
}

void *kcc_ir_bf_container_type(void *field_type) {
    if (field_type && (IL(field_type, TY_QUALS) &
   2)) {
        if (kcc_ir_vol_longu) return
   kcc_ir_vol_longu;
        void *nd = kcc_arena_alloc(TY_DESC_SIZE);
        void *src = kcc_type_long_u;
        IL(nd, 0) = IL(src, 0);
        IL(nd, 8) = IL(src, 8);
        IL(nd, 16) = IL(src, 16);
        IL(nd, 24) = IL(src, 24);
        IL(nd, 32) = IL(src, 32);
        IL(nd, 40) = IL(src, 40);
        IL(nd, TY_QUALS) = IL(nd, TY_QUALS) | 2;
        kcc_ir_vol_longu = nd;
        return nd;
    }
    return kcc_type_long_u;
}

unsigned long kcc_ir_type_layout_size(void *t) {
    if (!t) return 0;
    return kcc_type_layout(t);
}

unsigned long kcc_ir_bitfield_load(unsigned long
   addr, void *field) {
    unsigned long packed = IL(field, AST_F2);
    unsigned long width = packed & 0xffff;
    unsigned long lsb = packed >> 16;
    void *ft = kcc_sema_resolve_type((void *)IL(
   field, AST_F0));
    unsigned long code = kcc_ir_bf_code(
   kcc_ir_type_layout_size(ft));
    void *ct = kcc_ir_bf_container_type(ft);
    unsigned long val = kcc_ir_emit_load_t(addr,
   code, ct);
    if (lsb) {
        unsigned long c = kcc_ir_emit_const(lsb);
        val = kcc_ir_emit_binop(IR_LSHR, val, c);
    }
    unsigned long mc = kcc_ir_emit_const(
   kcc_ir_bf_mask(width));
    val = kcc_ir_emit_binop(IR_AND, val, mc);
    if (IL(ft, TY_KIND) == TY_ENUM) return val;
    if (kcc_sema_int_signedness(ft) != 1) return
   val;
    unsigned long s = 64 - width;
    unsigned long sc = kcc_ir_emit_const(s);
    val = kcc_ir_emit_binop(IR_SHL, val, sc);
    unsigned long sc2 = kcc_ir_emit_const(s);
    val = kcc_ir_emit_binop(IR_ASHR, val, sc2);
    return val;
}

unsigned long kcc_ir_bitfield_store(unsigned long
   addr, unsigned long value, void *field) {
    unsigned long packed = IL(field, AST_F2);
    unsigned long width = packed & 0xffff;
    unsigned long lsb = packed >> 16;
    void *ft = kcc_sema_resolve_type((void *)IL(
   field, AST_F0));
    unsigned long code = kcc_ir_bf_code(
   kcc_ir_type_layout_size(ft));
    void *ct = kcc_ir_bf_container_type(ft);
    unsigned long oldc = kcc_ir_emit_load_t(addr,
   code, ct);
    unsigned long mask = kcc_ir_bf_mask(width);
    unsigned long mc = kcc_ir_emit_const(mask);
    unsigned long vbits = kcc_ir_emit_binop(IR_AND
   , value, mc);
    if (lsb) {
        unsigned long lc = kcc_ir_emit_const(lsb);
        vbits = kcc_ir_emit_binop(IR_SHL, vbits,
   lc);
    }
    unsigned long clrc = kcc_ir_emit_const(~(mask
   << lsb));
    unsigned long cleared = kcc_ir_emit_binop(
   IR_AND, oldc, clrc);
    unsigned long newval = kcc_ir_emit_binop(IR_OR
   , cleared, vbits);
    void *ct2 = kcc_ir_bf_container_type(ft);
    kcc_ir_emit_store_t(newval, addr, code, ct2);
    return value;
}

unsigned long kcc_ir_emit_gep_const(unsigned long
   base, unsigned long off) {
    unsigned long ov = kcc_ir_emit_const(off);
    void *gep = kcc_ir_alloc_instr(IR_GEP);
    IL(gep, IR_TYPE) = (unsigned long)
   kcc_type_long_u;
    IL(gep, IR_SRC1) = base;
    IL(gep, IR_SRC2) = ov;
    unsigned long v = kcc_ir_next_vreg();
    IL(gep, IR_DEST) = v;
    kcc_ir_emit(gep);
    return v;
}

void kcc_ir_store_elem(unsigned long base,
   unsigned long off, unsigned long value,
   unsigned long width) {
    void *field = kcc_ir_member_field;
    if (field && IL(field, AST_F2)) {
        unsigned long caddr =
   kcc_ir_emit_gep_const(base, off);
        kcc_ir_bitfield_store(caddr, value, field)
   ;
    } else {
        kcc_ir_emit_indexed_store(base, off, value
   , width);
    }
}

void kcc_ir_emit_string_init(unsigned long base,
   const char *sp, unsigned long slen,
                                unsigned long alen,
   unsigned long zerofill) {
    if (zerofill) {
        unsigned long slots = (alen + 7) >> 3;
        unsigned long i = 0;
        while (i < slots) {
            unsigned long z = kcc_ir_emit_const(0)
   ;
            kcc_ir_emit_indexed_store(base, i <<
   3, z, 0);
            i++;
        }
    }
    unsigned long n = slen;
    if (alen < n) n = alen;
    unsigned long i = 0;
    while (i < n) {
        unsigned long b = kcc_ir_emit_const((
   unsigned char)sp[i]);
        kcc_ir_emit_indexed_store(base, i, b, 1);
```

```
        i++;
    }
}

unsigned long kcc_ir_emit_bool_norm(unsigned long
   vreg) {
    unsigned long z = kcc_ir_emit_const(0);
    return kcc_ir_emit_binop(IR_CMP_NE, vreg, z);
}

unsigned long kcc_ir_tok_to_binop(unsigned long
   tok) {
    if (tok == TOK_SUB_ASSIGN) return IR_SUB;
    if (tok == TOK_MUL_ASSIGN) return IR_MUL;
    if (tok == TOK_DIV_ASSIGN) return IR_SDIV;
    if (tok == TOK_MOD_ASSIGN) return IR_SMOD;
    if (tok == TOK_AND_ASSIGN) return IR_AND;
    if (tok == TOK_OR_ASSIGN) return IR_OR;
    if (tok == TOK_XOR_ASSIGN) return IR_XOR;
    if (tok == TOK_SHL_ASSIGN) return IR_SHL;
    if (tok == TOK_SHR_ASSIGN) return IR_ASHR;
    return IR_ADD;
    /* TOK_ADD_ASSIGN and default */
}

unsigned long kcc_ir_bf_trunc_value(unsigned long
   value, void *field) {
    unsigned long width = IL(field, AST_F2) & 0
   xffff;
    void *ft = kcc_sema_resolve_type((void *)IL(
   field, AST_F0));
    unsigned long mc = kcc_ir_emit_const(
   kcc_ir_bf_mask(width));
    value = kcc_ir_emit_binop(IR_AND, value, mc);
    if (IL(ft, TY_KIND) == TY_ENUM) return value;
    if (kcc_sema_int_signedness(ft) != 1) return
   value;
    unsigned long s = 64 - width;
    unsigned long sc = kcc_ir_emit_const(s);
    value = kcc_ir_emit_binop(IR_SHL, value, sc);
    unsigned long sc2 = kcc_ir_emit_const(s);
    value = kcc_ir_emit_binop(IR_ASHR, value, sc2)
   ;
    return value;
}

unsigned long kcc_ir_assign_through(unsigned long
   addr, unsigned long rhs, unsigned long op_tok,
                                       void *field,
   void *ltype, void *rtype) {
    unsigned long is_bf = 0;
    if (field && IL(field, AST_F2)) is_bf = 1;
    if (op_tok == TOK_ASSIGN) {
        if (is_bf) {
            unsigned long stored =
   kcc_ir_bitfield_store(addr, rhs, field);
            (void)stored;
            return kcc_ir_bf_trunc_value(rhs,
   field);
        }
        unsigned long sz = kcc_ir_store_width(
   ltype);
        kcc_ir_emit_store(rhs, addr, sz);
        return rhs;
    }
    /* compound: load-modify-store */
    unsigned long old;
    if (is_bf) old = kcc_ir_bitfield_load(addr,
   field);
    else old = kcc_ir_emit_load(addr,
   kcc_ir_load_width(ltype));
    unsigned long op = kcc_ir_tok_to_binop(op_tok)
   ;
    unsigned long nv = kcc_ir_compound_apply(op,
   old, ltype, rhs, rtype);
    if (is_bf) {
        kcc_ir_bitfield_store(addr, nv, field);
        return kcc_ir_bf_trunc_value(nv, field);
    }
    kcc_ir_emit_store(nv, addr, kcc_ir_store_width
   (ltype));
    return nv;
}
```

□

862 · defined at 856, 859, 860, 862, 865, 872 · used in 902a

75.6. Identifiers, Operators, and Unary Expressions

Loading a variable resolves local (alloca), static-local (mangled global), and file-scope cases, with arrays and functions decaying to their address rather than loading. Binary operators scale pointer arithmetic, remap signed opcodes to unsigned by the operands' common type, take an FP path when any operand is floating, and short-circuit `&&`/`||` through fresh blocks. When the *left* operand of a `&&`/`||` is an integer constant expression, the fold happens at build time instead—`0 && X` yields a constant 0 and never builds `X`, `1 || X` a constant 1—so a dead right operand is dropped rather than emitted behind a run-time branch. This matters because the kernel guards asm-bearing idioms this way: `nf_hook`'s `__builtin_constant_p(pf) && __builtin_constant_p(ho` must vanish when `pf`/`hook` are run-time, or the static-key jump table would emit `.quad %reg` against the array-indexed key and the assembler would reject the register relocation. Unary handles negation, complement, logical-not, address-of (dispatched by operand shape), and dereference. Logical-not lowers to a compare against zero typed at the operand's own width—`double` for a floating operand, a 64-bit compare for a `long` or pointer, else 32-bit—so that `!x` on a wide value whose low 32 bits happen to be zero (the kernel's `find_bit` loop tests exactly such a masked word) still reports true only when the whole word is zero.

`__int128` arithmetic. A `__int128` rvalue is materialised as the *address of a sixteen-byte stack slot* — a two-eightbyte `IR_ALLOCA` holding the value little-endian, low half at offset 0 and high half at offset 8 — exactly the representation the aggregate paths already use for a two-word struct. Two operators are lowered, which together are precisely the kernel's `math64.h` helpers `mul_u64_u32_shr` and `mul_u64_u64_shr` (`(u64)(((unsigned __int128)a * b) >> s)`): an unsigned multiply and a logical right shift, both computed with ordinary 64-bit IR operations on the two halves so no register-pair opcode or back-end change is needed. The multiply is the schoolbook $64 \times 64 \to 128$ expansion (split each operand into 32-bit limbs, four partial products, recombine with carries) plus the two cross terms $a_{lo}b_{hi} + a_{hi}b_{lo}$ folded into the high word; its low 128 bits are correct for signed and unsigned alike. The shift is branchless: it forms the within-a-word

shift of the pair by $n \bmod 64$ (guarding the hi $\ll (64-n)$ spill against the $n \equiv 0$ case that would otherwise be a shift by 64), then selects between that and the ≥ 64 result with an all-ones/zero mask derived from bit 6 of the count, since the IR has no select opcode. A widening cast zero- or sign-fills the high half; a narrowing cast back to an integer simply loads the low half. This is a self-hosting-port (Linux/x86-64) capability: the assembly reference still rejects __int128 arithmetic (Appendix 3.1), as it does the other kernel-only builtins.

⟨ *Selfhost: ir exprs* 865 ⟩ +≡

```
unsigned long kcc_ir_build_ident(void *e) {
    char *name = (char *)IL(e, AST_F0);
    if (!name) return 0;
    unsigned long keylen = kcc_strlen(name);
    unsigned long v = (unsigned long)kcc_ht_get(
  kcc_ir_locals, name, keylen);
    if (v) {
        unsigned long alloca;
        if (v & 0x8000000000000000UL) {
            alloca = (v & ~0x8000000000000000UL) -
   1;
        } else {
            alloca = v - 1;
            void *ft = (void *)IL(e, AST_F3);
            if (ft) {
                unsigned long k = IL(ft, TY_KIND);
                if (k == TY_PTR || k == TY_ARRAY)
  return alloca;  /* array decay */
            }
        }
        void *ft = (void *)IL(e, AST_F3);
        unsigned long width = kcc_ir_load_width(ft
  );
        void *lty = ((unsigned long)ft >= 0x1000
  && (IL(ft, TY_QUALS) & 2))
                    ? ft : kcc_type_int_s;
        void *ld = kcc_ir_alloc_instr(IR_LOAD);
        IL(ld, IR_TYPE) = (unsigned long)lty;
        IL(ld, IR_SRC1) = alloca;
        IL(ld, IR_SRC2) = width;
        unsigned long rv = kcc_ir_next_vreg();
        IL(ld, IR_DEST) = rv;
        kcc_ir_emit(ld);
        return rv;
    }
    /* global path */
    char *symname = name;
    unsigned long is_array = 0;
    unsigned long resolved = 0;
    if (kcc_ir_statics) {
        void *mangled = kcc_ht_get(kcc_ir_statics,
    name, keylen);
        if (mangled) {
            symname = (char *)mangled;
            resolved = 1;
            if (kcc_ir_static_types) {
                void *st = kcc_ht_get(
  kcc_ir_static_types, name, keylen);
                if (st) {
                    unsigned long k = IL(st,
  TY_KIND);
                    if (k == TY_ARRAY || k ==
  TY_FUNC) is_array = 1;
                }
            }
        }
    }
    if (!resolved) {
        void *sym = kcc_symtab_lookup(name, keylen
);
        if (sym) {
            if (IL(sym, SYM_KIND) ==
SYM_ENUM_CONST) {
                void *c = kcc_ir_alloc_instr(
IR_CONST);
                IL(c, IR_TYPE) = (unsigned long)
kcc_type_int_s;
                IL(c, IR_SRC1) = IL(sym, SYM_VALUE
);
                unsigned long rv =
kcc_ir_next_vreg();
                IL(c, IR_DEST) = rv;
                kcc_ir_emit(c);
                return rv;
            }
            void *st = (void *)IL(sym, SYM_TYPE);
            if (st) {
                unsigned long k = IL(st, TY_KIND);
                if (k == TY_ARRAY || k == TY_FUNC)
    is_array = 1;
            }
        }
    }
    /* Using an always_inline function's name as a
    value---never calling it, only
       taking its address, as in `.less =
mod_tree_less' in a latch_tree_ops
       table---cannot be inlined, so the function
must be emitted out-of-line.
       The call path records non-inlinable calls
the same way; a bare reference
       reaches here instead (a call extracts the
callee name directly and never
       builds the designator as an ident). */
    if (kcc_always_inline_fns &&
        kcc_ht_get(kcc_always_inline_fns, name,
keylen)) {
        if (!kcc_inl_need_out) kcc_inl_need_out =
kcc_ht_init(32);
        kcc_ht_put(kcc_inl_need_out, name, keylen,
    (void *)1);
    }
    if (is_array) {
        void *g = kcc_ir_alloc_instr(IR_GLOBAL_REF
);
        IL(g, IR_SRC1) = (unsigned long)symname;
        IL(g, IR_TYPE) = (unsigned long)
kcc_type_int_s;
        unsigned long rv = kcc_ir_next_vreg();
        IL(g, IR_DEST) = rv;
        kcc_ir_emit(g);
        return rv;
    }
    void *ft = (void *)IL(e, AST_F3);
    unsigned long width = kcc_ir_load_width(ft);
    void *lty;
    if ((unsigned long)ft >= 0x1000 &&
        ((IL(ft, TY_QUALS) & 2) || IL(ft, TY_KIND)
    == TY_FLOAT || IL(ft, TY_KIND) == TY_DOUBLE))
        lty = ft;
    else
        lty = kcc_type_int_s;
    void *g = kcc_ir_alloc_instr(IR_GLOBAL_REF);
    IL(g, IR_SRC1) = (unsigned long)symname;
    IL(g, IR_TYPE) = (unsigned long)kcc_type_int_s
;
    unsigned long addr = kcc_ir_next_vreg();
```

```
    IL(g, IR_DEST) = addr;
    kcc_ir_emit(g);
    void *ld = kcc_ir_alloc_instr(IR_LOAD);
    IL(ld, IR_TYPE) = (unsigned long)lty;
    IL(ld, IR_SRC1) = addr;
    IL(ld, IR_SRC2) = width;
    unsigned long rv = kcc_ir_next_vreg();
    IL(ld, IR_DEST) = rv;
    kcc_ir_emit(ld);
    return rv;
}

/* 1 iff a relational comparison of these two
   operands is unsigned, applying
   the integer promotions then the usual
   arithmetic conversions' sign rule. */
unsigned long kcc_ir_cmp_unsigned(void *lhs_node,
   void *rhs_node) {
    if (!lhs_node) return 0;
    void *lt = (void *)IL(lhs_node, AST_F3);
    if (!lt) return 0;
    if (!rhs_node) return 0;
    void *rt = (void *)IL(rhs_node, AST_F3);
    if (!rt) return 0;
    if (IL(lt, TY_KIND) - 2 > 5) return 0;
   /* not an integer kind */
    if (IL(rt, TY_KIND) - 2 > 5) return 0;
    unsigned long sl = IL(lt, TY_SIZE), gl = IL(lt
   , TY_F0);
    if (sl < 4) { sl = 4; gl = 1; }
   /* promote to signed int */
    unsigned long sr = IL(rt, TY_SIZE), gr = IL(rt
   , TY_F0);
    if (sr < 4) { sr = 4; gr = 1; }
    if (sl > sr) return gl == 0 ? 1 : 0;
   /* wider operand's sign wins */
    if (sl < sr) return gr == 0 ? 1 : 0;
    if (gl == 0 || gr == 0) return 1;
   /* equal rank: unsigned if either is */
    return 0;
}

/* Re-canonicalise a width-growing op's result to
   32 bits when both operands
   are <=int-width integers (SEXT for a signed
   result, ZEXT for unsigned). */
unsigned long kcc_ir_norm32(unsigned long vreg,
   unsigned long op, void *ltype, void *rtype) {
    if (!(op == IR_ADD || op == IR_SUB || op ==
   IR_MUL || op == IR_SHL ||
          op == IR_NEG || op == IR_NOT || op ==
   IR_AND || op == IR_OR || op == IR_XOR))
        return vreg;
    if (!ltype) return vreg;
    if (IL(ltype, TY_KIND) - 2 > 3) return vreg;
   /* lhs not <=32-bit integer */
    if (!rtype) return vreg;
    if (IL(rtype, TY_KIND) - 2 > 3) return vreg;
    unsigned long ext;
   /* IR_SEXT or IR_ZEXT */
    if (IL(ltype, TY_F0) == 0 && IL(ltype, TY_SIZE
   ) >= 4) {
        ext = IR_ZEXT;
    } else if (op == IR_SHL) {
        ext = IR_SEXT;
   /* shift: left operand alone decides */
    } else if (IL(rtype, TY_F0)) {
        ext = IR_SEXT;
    } else if (IL(rtype, TY_SIZE) >= 4) {
        ext = IR_ZEXT;
    } else {
        ext = IR_SEXT;
    }
    void *inst = kcc_ir_alloc_instr(ext);
    IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    IL(inst, IR_SRC1) = vreg;
    IL(inst, IR_SRC2) = 4;
   /* source width 32-bit */
    unsigned long r = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = r;
    kcc_ir_emit(inst);
    return r;
}

/* scale a vreg by an element size: const(int) +
   mul(int) */
static unsigned long bin_scale(unsigned long v,
   unsigned long size) {
    void *c = kcc_ir_alloc_instr(IR_CONST);
    IL(c, IR_TYPE) = (unsigned long)kcc_type_int_s
   ;
    IL(c, IR_SRC1) = size;
    unsigned long cv = kcc_ir_next_vreg();
    IL(c, IR_DEST) = cv;
    kcc_ir_emit(c);
    void *m = kcc_ir_alloc_instr(IR_MUL);
    IL(m, IR_TYPE) = (unsigned long)kcc_type_int_s
   ;
    IL(m, IR_SRC1) = v;
    IL(m, IR_SRC2) = cv;
    unsigned long r = kcc_ir_next_vreg();
    IL(m, IR_DEST) = r;
    kcc_ir_emit(m);
    return r;
}

/* (a - b) / sizeof(*lhs), all in signed long */
static unsigned long bin_ptr_diff(void *e,
   unsigned long a, unsigned long b) {
    void *lt = (void *)IL((void *)IL(e, AST_F1),
   AST_F3);
    void *pointee = lt ? (void *)IL(lt, TY_F0) :
   0;
    unsigned long stride = 1;
    if (pointee) { unsigned long s =
   kcc_type_layout(pointee); if (s) stride = s; }
    void *sub = kcc_ir_alloc_instr(IR_SUB);
    IL(sub, IR_TYPE) = (unsigned long)
   kcc_type_long_s;
    IL(sub, IR_SRC1) = a;
    IL(sub, IR_SRC2) = b;
    unsigned long diff = kcc_ir_next_vreg();
    IL(sub, IR_DEST) = diff;
    kcc_ir_emit(sub);
    void *c = kcc_ir_alloc_instr(IR_CONST);
    IL(c, IR_TYPE) = (unsigned long)
   kcc_type_long_s;
    IL(c, IR_SRC1) = stride;
    unsigned long sv = kcc_ir_next_vreg();
    IL(c, IR_DEST) = sv;
    kcc_ir_emit(c);
    void *d = kcc_ir_alloc_instr(IR_SDIV);
    IL(d, IR_TYPE) = (unsigned long)
   kcc_type_long_s;
    IL(d, IR_SRC1) = diff;
    IL(d, IR_SRC2) = sv;
    unsigned long r = kcc_ir_next_vreg();
    IL(d, IR_DEST) = r;
    kcc_ir_emit(d);
    return r;
}
```

```
/* floating operation: promote integer operands to
    double (rounding to single
   when the other operand is float), emit typed
   double, round result if float */
static unsigned long bin_fp_emit(void *e, unsigned
    long op, unsigned long a, unsigned long b) {
    void *lt = (void *)IL((void *)IL(e, AST_F1),
   AST_F3);
    void *rt = (void *)IL((void *)IL(e, AST_F2),
   AST_F3);
    if (!kcc_ir_type_is_fp(lt)) {
        a = kcc_ir_emit_int_to_fp(a, lt);
        if (rt && IL(rt, TY_KIND) == TY_FLOAT) a =
    kcc_ir_emit_fpround(a);
    }
    if (!kcc_ir_type_is_fp(rt)) {
        b = kcc_ir_emit_int_to_fp(b, rt);
        if (lt && IL(lt, TY_KIND) == TY_FLOAT) b =
    kcc_ir_emit_fpround(b);
    }
    void *inst = kcc_ir_alloc_instr(op);
    IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_double;
    IL(inst, IR_SRC1) = a;
    IL(inst, IR_SRC2) = b;
    unsigned long res = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = res;
    kcc_ir_emit(inst);
    void *rty = (void *)IL(e, AST_F3);
    if (rty && IL(rty, TY_KIND) == TY_FLOAT) res =
    kcc_ir_emit_fpround(res);
    return res;
}

static unsigned long bin_short_and(void *e) {
    /* Fold a compile-time-constant left operand
   at build time, exactly as gcc
       evaluates the guard: `0 && X' is 0 with X
   never built; `k && X' (k!=0) is
       `(X != 0)'.  This is what lets the kernel
   drop a
       `__builtin_constant_p(n) &&
   static_key_false(...)' branch when n is a
       run-time value.  Without it the dead static
   -key asm still reaches the
       assembler, and with the key an array
   element `&nf_hooks_needed[pf][hook]'
       (nf_hook, netfilter) it emits `.quad %reg'
   into __jump_table --- which as
       rejects: "invalid R_X86_64_64 relocation
   against register". */
    void *lhs_e = (void *)IL(e, AST_F1);
    if (kcc_expr_is_ice(lhs_e)) {
        if (kcc_eval_const_int(lhs_e) == 0) return
    kcc_ir_emit_const(0);
        return kcc_ir_emit_bool_norm(
   kcc_ir_build_expr((void *)IL(e, AST_F2)));
    }
    void *tmp = kcc_ir_alloc_instr(IR_ALLOCA);
    IL(tmp, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    unsigned long tmpv = kcc_ir_next_vreg();
    IL(tmp, IR_DEST) = tmpv;
    kcc_ir_emit(tmp);
    unsigned long lhs = kcc_ir_build_expr((void *)
   IL(e, AST_F1));
    void *rhs_bb = kcc_ir_alloc_bb();
    void *short_bb = kcc_ir_alloc_bb();
    void *merge_bb = kcc_ir_alloc_bb();
    void *bc = kcc_ir_alloc_instr(IR_BR_COND);
    IL(bc, IR_DEST) = lhs;
    IL(bc, IR_SRC1) = IL(rhs_bb, BB_ID);
    IL(bc, IR_SRC2) = IL(short_bb, BB_ID);
    kcc_ir_emit(bc);
    IL(kcc_ir_cur_bb, BB_FLAGS) = IL(kcc_ir_cur_bb
   , BB_FLAGS) | BB_FLAG_TERMINATED;
    kcc_ir_set_bb(short_bb);
    void *c0 = kcc_ir_alloc_instr(IR_CONST);
    IL(c0, IR_SRC1) = 0;
    IL(c0, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    unsigned long zv = kcc_ir_next_vreg();
    IL(c0, IR_DEST) = zv;
    kcc_ir_emit(c0);
    void *st0 = kcc_ir_alloc_instr(IR_STORE);
    IL(st0, IR_SRC1) = zv;
    IL(st0, IR_SRC2) = tmpv;
    kcc_ir_emit(st0);
    void *br0 = kcc_ir_alloc_instr(IR_BR);
    IL(br0, IR_SRC1) = IL(merge_bb, BB_ID);
    kcc_ir_emit(br0);
    IL(kcc_ir_cur_bb, BB_FLAGS) = IL(kcc_ir_cur_bb
   , BB_FLAGS) | BB_FLAG_TERMINATED;
    kcc_ir_set_bb(rhs_bb);
    unsigned long rv = kcc_ir_build_expr((void *)
   IL(e, AST_F2));
    rv = kcc_ir_emit_bool_norm(rv);
    void *st1 = kcc_ir_alloc_instr(IR_STORE);
    IL(st1, IR_SRC1) = rv;
    IL(st1, IR_SRC2) = tmpv;
    kcc_ir_emit(st1);
    void *br1 = kcc_ir_alloc_instr(IR_BR);
    IL(br1, IR_SRC1) = IL(merge_bb, BB_ID);
    kcc_ir_emit(br1);
    IL(kcc_ir_cur_bb, BB_FLAGS) = IL(kcc_ir_cur_bb
   , BB_FLAGS) | BB_FLAG_TERMINATED;
    kcc_ir_set_bb(merge_bb);
    void *ld = kcc_ir_alloc_instr(IR_LOAD);
    IL(ld, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    IL(ld, IR_SRC1) = tmpv;
    unsigned long res = kcc_ir_next_vreg();
    IL(ld, IR_DEST) = res;
    kcc_ir_emit(ld);
    return res;
}

static unsigned long bin_short_or(void *e) {
    /* Symmetric compile-time fold: `k || X' (k
   !=0) is 1 with X never built;
       `0 || X' is `(X != 0)'. */
    void *lhs_e = (void *)IL(e, AST_F1);
    if (kcc_expr_is_ice(lhs_e)) {
        if (kcc_eval_const_int(lhs_e) != 0) return
    kcc_ir_emit_const(1);
        return kcc_ir_emit_bool_norm(
   kcc_ir_build_expr((void *)IL(e, AST_F2)));
    }
    void *tmp = kcc_ir_alloc_instr(IR_ALLOCA);
    IL(tmp, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    unsigned long tmpv = kcc_ir_next_vreg();
    IL(tmp, IR_DEST) = tmpv;
    kcc_ir_emit(tmp);
    unsigned long lhs = kcc_ir_build_expr((void *)
   IL(e, AST_F1));
    void *short_bb = kcc_ir_alloc_bb();
    void *rhs_bb = kcc_ir_alloc_bb();
    void *merge_bb = kcc_ir_alloc_bb();
    void *bc = kcc_ir_alloc_instr(IR_BR_COND);
```

```
    IL(bc, IR_DEST) = lhs;
    IL(bc, IR_SRC1) = IL(short_bb, BB_ID);
    IL(bc, IR_SRC2) = IL(rhs_bb, BB_ID);
    kcc_ir_emit(bc);
    IL(kcc_ir_cur_bb, BB_FLAGS) = IL(kcc_ir_cur_bb
   , BB_FLAGS) | BB_FLAG_TERMINATED;
    kcc_ir_set_bb(short_bb);
    void *c1 = kcc_ir_alloc_instr(IR_CONST);
    IL(c1, IR_SRC1) = 1;
    IL(c1, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    unsigned long ov = kcc_ir_next_vreg();
    IL(c1, IR_DEST) = ov;
    kcc_ir_emit(c1);
    void *st0 = kcc_ir_alloc_instr(IR_STORE);
    IL(st0, IR_SRC1) = ov;
    IL(st0, IR_SRC2) = tmpv;
    kcc_ir_emit(st0);
    void *br0 = kcc_ir_alloc_instr(IR_BR);
    IL(br0, IR_SRC1) = IL(merge_bb, BB_ID);
    kcc_ir_emit(br0);
    IL(kcc_ir_cur_bb, BB_FLAGS) = IL(kcc_ir_cur_bb
   , BB_FLAGS) | BB_FLAG_TERMINATED;
    kcc_ir_set_bb(rhs_bb);
    unsigned long rv = kcc_ir_build_expr((void *)
   IL(e, AST_F2));
    rv = kcc_ir_emit_bool_norm(rv);
    void *st1 = kcc_ir_alloc_instr(IR_STORE);
    IL(st1, IR_SRC1) = rv;
    IL(st1, IR_SRC2) = tmpv;
    kcc_ir_emit(st1);
    void *br1 = kcc_ir_alloc_instr(IR_BR);
    IL(br1, IR_SRC1) = IL(merge_bb, BB_ID);
    kcc_ir_emit(br1);
    IL(kcc_ir_cur_bb, BB_FLAGS) = IL(kcc_ir_cur_bb
   , BB_FLAGS) | BB_FLAG_TERMINATED;
    kcc_ir_set_bb(merge_bb);
    void *ld = kcc_ir_alloc_instr(IR_LOAD);
    IL(ld, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    IL(ld, IR_SRC1) = tmpv;
    unsigned long res = kcc_ir_next_vreg();
    IL(ld, IR_DEST) = res;
    kcc_ir_emit(ld);
    return res;
}

/* --- __int128 arithmetic ---
   A 128-bit value is the address of a fresh two-
   slot (16-byte) alloca, low
   half at offset 0 and high half at offset 8.
   All arithmetic below is built
   from ordinary 64-bit IR ops on the two halves.
   */
static unsigned long kcc_ir_i128_slot(void) {
    void *a = kcc_ir_alloc_instr(IR_ALLOCA);
    IL(a, IR_TYPE) = (unsigned long)
   kcc_type_int128_u;
    IL(a, IR_SRC1) = 2;                          /*
   two eight-byte slots */
    unsigned long avreg = kcc_ir_next_vreg();
    IL(a, IR_DEST) = avreg;
    kcc_ir_emit(a);
    return avreg;
}
static void kcc_ir_i128_set(unsigned long slot,
   unsigned long lo, unsigned long hi) {
    kcc_ir_emit_indexed_store(slot, 0, lo, 0);
    kcc_ir_emit_indexed_store(slot, 8, hi, 0);
}

/* Width code 0 is a plain 8-byte integer load (
   kcc_ir_load_width); a literal
   8 would instead select the single-float form in
    the back ends. */
static unsigned long kcc_ir_i128_lo(unsigned long
   addr) { return kcc_ir_emit_load(addr, 0); }
static unsigned long kcc_ir_i128_hi(unsigned long
   addr) {
    return kcc_ir_emit_load(kcc_ir_emit_gep_const(
   addr, 8), 0);
}
/* The 128-bit slot address of a value already
   typed __int128.  An lvalue (an
   identifier, member, subscript, or dereference
   ---possibly parenthesised) is its
   own slot, so we take its address; only an
   rvalue (a cast, or a 128-bit
   arithmetic result) is built directly, since
   those already yield a slot address
   and building `kcc_ir_build_expr' on an lvalue
   would instead load its low half
   as a scalar and then dereference that value as
   if it were the slot pointer. */
static unsigned long kcc_ir_i128_addr(void *node)
   {
    void *n = node;
    while (n && IL(n, AST_TAG) == AST_PAREN_EXPR)
   n = (void *)IL(n, AST_F0);
    unsigned long tag = n ? IL(n, AST_TAG) : 0;
    if (tag == AST_IDENT_EXPR || tag == AST_MEMBER
    || tag == AST_SUBSCRIPT ||
        (tag == AST_UNARY_EXPR && IL(n, AST_F0) ==
    TOK_STAR)) {
        void *u = kcc_ast_alloc(AST_UNARY_EXPR);
        IL(u, AST_F0) = TOK_AMP;
        IL(u, AST_F1) = (unsigned long)n;
        IL(u, AST_F2) = 1;
        return kcc_ir_build_expr(u);
    }
    return kcc_ir_build_expr(node);
}
/* Build a subexpression as a 128-bit slot address
   : a value already typed
   __int128 yields the slot directly; a narrower
   integer is widened (zero- or
   sign-filled high half) into a fresh slot. */
static unsigned long kcc_ir_i128_operand(void *
   node) {
    void *ty = (void *)IL(node, AST_F3);
    if (ty && (unsigned long)ty >= 0x1000 && IL(ty
   , TY_KIND) == TY_INT128)
        return kcc_ir_i128_addr(node);
    unsigned long v = kcc_ir_build_expr(node);
    unsigned long slot = kcc_ir_i128_slot();
    unsigned long hi = kcc_ir_emit_const(0);
    if (ty && (unsigned long)ty >= 0x1000 && IL(ty
   , TY_KIND) != TY_PTR && IL(ty, TY_F0) != 0)
        hi = kcc_ir_emit_binop(IR_ASHR, v,
   kcc_ir_emit_const(63));
    kcc_ir_i128_set(slot, v, hi);
    return slot;
}
/* Low 128 bits of A*B (correct for signed and
   unsigned operands). */
static unsigned long kcc_ir_i128_mul(unsigned long
    A, unsigned long B) {
    unsigned long a_lo = kcc_ir_i128_lo(A), a_hi =
    kcc_ir_i128_hi(A);
    unsigned long b_lo = kcc_ir_i128_lo(B), b_hi =
    kcc_ir_i128_hi(B);
```

```
    unsigned long M   = kcc_ir_emit_const(0
xFFFFFFFF);
    unsigned long c32 = kcc_ir_emit_const(32);
    unsigned long alo = kcc_ir_emit_binop(IR_AND,
a_lo, M);
    unsigned long ahi = kcc_ir_emit_binop(IR_LSHR,
 a_lo, c32);
    unsigned long blo = kcc_ir_emit_binop(IR_AND,
b_lo, M);
    unsigned long bhi = kcc_ir_emit_binop(IR_LSHR,
 b_lo, c32);
    unsigned long ll = kcc_ir_emit_binop(IR_MUL,
alo, blo);
    unsigned long lh = kcc_ir_emit_binop(IR_MUL,
alo, bhi);
    unsigned long hl = kcc_ir_emit_binop(IR_MUL,
ahi, blo);
    unsigned long hh = kcc_ir_emit_binop(IR_MUL,
ahi, bhi);
    /* mid = (ll>>32) + (lh & M) + (hl & M) */
    unsigned long mid = kcc_ir_emit_binop(IR_ADD,
        kcc_ir_emit_binop(IR_ADD,
kcc_ir_emit_binop(IR_LSHR, ll, c32),

kcc_ir_emit_binop(IR_AND, lh, M)),
        kcc_ir_emit_binop(IR_AND, hl, M));
    unsigned long res_lo = kcc_ir_emit_binop(IR_OR
,
        kcc_ir_emit_binop(IR_AND, ll, M),
        kcc_ir_emit_binop(IR_SHL, mid, c32));
    /* high 64 of a_lo*b_lo */
    unsigned long hi_ll = kcc_ir_emit_binop(IR_ADD
,
        kcc_ir_emit_binop(IR_ADD, hh,
kcc_ir_emit_binop(IR_LSHR, lh, c32)),
        kcc_ir_emit_binop(IR_ADD,
kcc_ir_emit_binop(IR_LSHR, hl, c32),

kcc_ir_emit_binop(IR_LSHR, mid, c32)));
    unsigned long cross = kcc_ir_emit_binop(IR_ADD
,
        kcc_ir_emit_binop(IR_MUL, a_lo, b_hi),
        kcc_ir_emit_binop(IR_MUL, a_hi, b_lo));
    unsigned long res_hi = kcc_ir_emit_binop(
IR_ADD, hi_ll, cross);
    unsigned long slot = kcc_ir_i128_slot();
    kcc_ir_i128_set(slot, res_lo, res_hi);
    return slot;
}
/* Logical right shift of the 128-bit value at A
   by the 64-bit count n, for
   n in [0,127].  Branchless: no select opcode
   exists. */
static unsigned long kcc_ir_i128_lshr(unsigned
   long A, unsigned long n) {
    unsigned long lo = kcc_ir_i128_lo(A), hi =
kcc_ir_i128_hi(A);
    unsigned long c0 = kcc_ir_emit_const(0);
    unsigned long c63 = kcc_ir_emit_const(63);
    unsigned long c64 = kcc_ir_emit_const(64);
    unsigned long ones = kcc_ir_emit_const(0
xFFFFFFFFFFFFFFFFUL);
    unsigned long n6 = kcc_ir_emit_binop(IR_AND, n
, c63);          /* n & 63 */
    /* spill = (n6!=0) ? (hi << (64-n6)) : 0  -
   avoids a shift by 64 */
    unsigned long sh = kcc_ir_emit_binop(IR_AND,
kcc_ir_emit_binop(IR_SUB, c64, n6), c63);
    unsigned long spill_raw = kcc_ir_emit_binop(
IR_SHL, hi, sh);
    unsigned long ne = kcc_ir_emit_binop(IR_CMP_NE
, n6, c0);       /* 0/1 */
    unsigned long nzmask = kcc_ir_emit_binop(
IR_SUB, c0, ne);      /* 0 or all-ones */
    unsigned long spill = kcc_ir_emit_binop(IR_AND
, spill_raw, nzmask);
    unsigned long s_lo = kcc_ir_emit_binop(IR_OR,
kcc_ir_emit_binop(IR_LSHR, lo, n6), spill);
    unsigned long s_hi = kcc_ir_emit_binop(IR_LSHR
, hi, n6);
    /* select the >=64 result (lo=hi>>n6, hi=0)
   via a mask from bit 6 of n */
    unsigned long ge = kcc_ir_emit_binop(IR_AND,
kcc_ir_emit_binop(IR_LSHR, n, kcc_ir_emit_const
(6)),

    kcc_ir_emit_const(1));
    unsigned long gemask = kcc_ir_emit_binop(
IR_SUB, c0, ge);      /* all-ones if n>=64 */
    unsigned long notmask = kcc_ir_emit_binop(
IR_XOR, gemask, ones);
    unsigned long r_lo = kcc_ir_emit_binop(IR_OR,
        kcc_ir_emit_binop(IR_AND, s_hi, gemask),
        kcc_ir_emit_binop(IR_AND, s_lo, notmask));
    unsigned long r_hi = kcc_ir_emit_binop(IR_AND,
    s_hi, notmask);
    unsigned long slot = kcc_ir_i128_slot();
    kcc_ir_i128_set(slot, r_lo, r_hi);
    return slot;
}
/* Dispatch a binary operator whose result type is
    __int128.  sema has already
   restricted these to unsigned multiply and
   logical right shift. */
static unsigned long kcc_ir_i128_binary(void *e) {
    unsigned long op_tok = IL(e, AST_F0);
    unsigned long A = kcc_ir_i128_operand((void *)
IL(e, AST_F1));
    if (op_tok == TOK_SHR) {
        unsigned long nsh = kcc_ir_build_expr((
void *)IL(e, AST_F2));
        return kcc_ir_i128_lshr(A, nsh);
    }
    /* TOK_STAR */
    unsigned long B = kcc_ir_i128_operand((void *)
IL(e, AST_F2));
    return kcc_ir_i128_mul(A, B);
}
/* Equality of two 128-bit values: both halves
   must match.  The result is an
   ordinary `int' (0/1), so the caller dispatches
   on the operands' type, not the
   result's.  `!=' is the complement. */
static unsigned long kcc_ir_i128_cmp(void *e,
   unsigned long op_tok) {
    unsigned long A = kcc_ir_i128_operand((void *)
IL(e, AST_F1));
    unsigned long B = kcc_ir_i128_operand((void *)
IL(e, AST_F2));
    unsigned long lo_eq = kcc_ir_emit_binop(
IR_CMP_EQ, kcc_ir_i128_lo(A), kcc_ir_i128_lo(B)
);
    unsigned long hi_eq = kcc_ir_emit_binop(
IR_CMP_EQ, kcc_ir_i128_hi(A), kcc_ir_i128_hi(B)
);
    unsigned long eq = kcc_ir_emit_binop(IR_AND,
lo_eq, hi_eq);
    if (op_tok == TOK_NE) return kcc_ir_emit_binop
(IR_XOR, eq, kcc_ir_emit_const(1));
    return eq;
```

```
}
/* Whether an expression node's static type is
   __int128. */
static unsigned long kcc_ir_is_i128(void *node) {
    void *ty = (void *)IL(node, AST_F3);
    return ty && (unsigned long)ty >= 0x1000 && IL
   (ty, TY_KIND) == TY_INT128;
}

unsigned long kcc_ir_build_binary(void *e) {
    unsigned long op_tok = IL(e, AST_F0);
    { void *rty0 = (void *)IL(e, AST_F3);
      if (rty0 && (unsigned long)rty0 >= 0x1000 &&
    IL(rty0, TY_KIND) == TY_INT128)
          return kcc_ir_i128_binary(e); }
    /* A 128-bit equality test yields a plain int,
    so it is not caught above;
       detect it from the operands (the kernel's
   try_cmpxchg128). */
    if ((op_tok == TOK_EQ || op_tok == TOK_NE) &&
        (kcc_ir_is_i128((void *)IL(e, AST_F1)) ||
   kcc_ir_is_i128((void *)IL(e, AST_F2))))
        return kcc_ir_i128_cmp(e, op_tok);
    if (op_tok == TOK_AND) return bin_short_and(e)
   ;
    if (op_tok == TOK_OR) return bin_short_or(e);
    unsigned long a = kcc_ir_build_expr((void *)IL
   (e, AST_F1));
    unsigned long b = kcc_ir_build_expr((void *)IL
   (e, AST_F2));
    unsigned long op;
    unsigned long addsub = 0;
    if (op_tok == TOK_PLUS) { op = IR_ADD; addsub
   = 1; }
    else if (op_tok == TOK_MINUS) { op = IR_SUB;
   addsub = 1; }
    else if (op_tok == TOK_STAR) op = IR_MUL;
    else if (op_tok == TOK_SLASH) op = IR_SDIV;
    else if (op_tok == TOK_PERCENT) op = IR_SMOD;
    else if (op_tok == TOK_AMP) op = IR_AND;
    else if (op_tok == TOK_PIPE) op = IR_OR;
    else if (op_tok == TOK_CARET) op = IR_XOR;
    else if (op_tok == TOK_SHL) op = IR_SHL;
    else if (op_tok == TOK_SHR) op = IR_ASHR;
    else if (op_tok == TOK_EQ) op = IR_CMP_EQ;
    else if (op_tok == TOK_NE) op = IR_CMP_NE;
    else if (op_tok == TOK_LT) op = IR_CMP_SLT;
    else if (op_tok == TOK_LE) op = IR_CMP_SLE;
    else if (op_tok == TOK_GT) op = IR_CMP_SGT;
    else if (op_tok == TOK_GE) op = IR_CMP_SGE;
    else { op = IR_ADD; addsub = 1; }

    if (addsub) {
        void *lt = (void *)IL((void *)IL(e, AST_F1
   ), AST_F3);
        unsigned long lhs_ptr = 0;
        if (lt) { unsigned long k = IL(lt, TY_KIND
   ); if (k == TY_PTR || k == TY_ARRAY) lhs_ptr =
   1; }
        if (lhs_ptr) {
            if (op == IR_SUB) {
                void *rt = (void *)IL((void *)IL(e
   , AST_F2), AST_F3);
                if (rt) { unsigned long rk = IL(rt
   , TY_KIND);
                    if (rk == TY_PTR || rk ==
   TY_ARRAY) return bin_ptr_diff(e, a, b); }
            }
            void *pointee = (void *)IL(lt, TY_F0);
            if (pointee) { unsigned long sz =
kcc_type_layout(pointee); if (sz > 1) b =
bin_scale(b, sz); }
        } else {
            void *rt = (void *)IL((void *)IL(e,
AST_F2), AST_F3);
            if (rt && (IL(rt, TY_KIND) == TY_PTR
|| IL(rt, TY_KIND) == TY_ARRAY)) {
                void *pointee = (void *)IL(rt,
TY_F0);
                if (pointee) { unsigned long sz =
kcc_type_layout(pointee); if (sz > 1) a =
bin_scale(a, sz); }
            }
        }
    }

    /* FP path if the result or either operand is
floating */
    void *rty = (void *)IL(e, AST_F3);
    unsigned long fp = kcc_ir_type_is_fp(rty);
    if (!fp) fp = kcc_ir_type_is_fp((void *)IL((
void *)IL(e, AST_F1), AST_F3));
    if (!fp) fp = kcc_ir_type_is_fp((void *)IL((
void *)IL(e, AST_F2), AST_F3));
    if (fp) return bin_fp_emit(e, op, a, b);

    /* integer opcode signedness remap */
    if (op >= IR_CMP_SLT && op <= IR_CMP_SGE) {
        if (kcc_ir_cmp_unsigned((void *)IL(e,
AST_F1), (void *)IL(e, AST_F2))) {
            if (op == IR_CMP_SLT) op = IR_CMP_ULT;
            else if (op == IR_CMP_SLE) op =
IR_CMP_ULE;
            else if (op == IR_CMP_SGT) { unsigned
long t = a; a = b; b = t; op = IR_CMP_ULT; }
            else { unsigned long t = a; a = b; b =
 t; op = IR_CMP_ULE; }
        }
    } else if (op == IR_SDIV || op == IR_SMOD ||
op == IR_ASHR) {
        void *rt = (void *)IL(e, AST_F3);
        if (rt && IL(rt, TY_F0) == 0) {
            if (op == IR_SDIV) op = IR_UDIV;
            else if (op == IR_SMOD) op = IR_UMOD;
            else op = IR_LSHR;
        }
    }

    void *inst = kcc_ir_alloc_instr(op);
    IL(inst, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    IL(inst, IR_SRC1) = a;
    IL(inst, IR_SRC2) = b;
    unsigned long is_cmp = (op >= IR_CMP_EQ && op
<= IR_CMP_ULE);
    unsigned long is_divmod = (op >= IR_SDIV && op
    <= IR_UMOD);
    if (is_cmp || is_divmod) {
        unsigned long lw = 8, rw = 8;
        void *lt = (void *)IL((void *)IL(e, AST_F1
), AST_F3);
        if (lt) lw = IL(lt, TY_SIZE);
        void *rt = (void *)IL((void *)IL(e, AST_F2
), AST_F3);
        if (rt) rw = IL(rt, TY_SIZE);
        unsigned long w = (rw > lw) ? rw : lw;
        if (w > 4) IL(inst, IR_TYPE) = (unsigned
long)kcc_type_long_s;
    }
```

```
    unsigned long r = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = r;
    kcc_ir_emit(inst);
    void *lt = (void *)IL((void *)IL(e, AST_F1),
   AST_F3);
    void *rt = (void *)IL((void *)IL(e, AST_F2),
   AST_F3);
    return kcc_ir_norm32(r, op, lt, rt);
}

unsigned long kcc_ir_build_unary(void *e) {
    unsigned long op = IL(e, AST_F0);
    if (op == TOK_INC || op == TOK_DEC) return
   kcc_ir_build_inc_dec(e);
    if (op == TOK_AMP) {
        void *operand = (void *)IL(e, AST_F1);
        if (!operand) return 0;
        while (operand && IL(operand, AST_TAG) ==
   AST_PAREN_EXPR)
            operand = (void *)IL(operand, AST_F0);
        if (!operand) return 0;
        unsigned long otag = IL(operand, AST_TAG);
        if (otag == AST_SUBSCRIPT) return
   kcc_ir_build_subscript_addr(operand);
        if (otag == AST_MEMBER) return
   kcc_ir_build_member_addr(operand);
        if (otag == AST_COMPOUND_LIT) return
   kcc_ir_build_compound_lit(operand);
        /* &(*p) is just p---the address-of and
   dereference cancel.  Needed by
           memory-constraint asm operands (`"m"(*
   addr)`), where the address of
           the pointed-to object is the pointer
   value itself. */
        if (otag == AST_UNARY_EXPR && IL(operand,
   AST_F0) == TOK_STAR)
            return kcc_ir_build_expr((void *)IL(
   operand, AST_F1));
        if (otag != AST_IDENT_EXPR) return 0;
        char *name = (char *)IL(operand, AST_F0);
        if (!name) return 0;
        kcc_ir_local_const_kill(name);   /* &x may
    alias-modify x */
        unsigned long keylen = kcc_strlen(name);
        unsigned long v = (unsigned long)
   kcc_ht_get(kcc_ir_locals, name, keylen);
        if (v) return (v & ~0x8000000000000000UL)
   - 1;
        /* global (or static-local hoisted to a
   mangled global) */
        char *symname = name;
        if (kcc_ir_statics) {
            void *mangled = kcc_ht_get(
   kcc_ir_statics, name, keylen);
            if (mangled) symname = (char *)mangled
   ;
        }
        void *g = kcc_ir_alloc_instr(IR_GLOBAL_REF
   );
        IL(g, IR_SRC1) = (unsigned long)symname;
        IL(g, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        unsigned long rv = kcc_ir_next_vreg();
        IL(g, IR_DEST) = rv;
        kcc_ir_emit(g);
        return rv;
    }
    unsigned long v = kcc_ir_build_expr((void *)IL
   (e, AST_F1));
    if (op == TOK_MINUS) {
        void *operand = (void *)IL(e, AST_F1);
        unsigned long is_fp = 0;
        void *fpty = 0;
        if (operand && IL(operand, AST_TAG) ==
   AST_FLOAT_LIT) { is_fp = 1; fpty =
   kcc_type_double; }
        else if (operand) {
            void *ot = (void *)IL(operand, AST_F3)
   ;
            if ((unsigned long)ot >= 0x1000) {
                unsigned long k = IL(ot, TY_KIND);
                if (k >= TY_FLOAT && k <=
   TY_LDOUBLE) { is_fp = 1; fpty = ot; }
            }
        }
        if (is_fp) {
            void *inst = kcc_ir_alloc_instr(IR_NEG
   );
            IL(inst, IR_SRC1) = v;
            IL(inst, IR_TYPE) = (unsigned long)
   fpty;
            unsigned long r = kcc_ir_next_vreg();
            IL(inst, IR_DEST) = r;
            kcc_ir_emit(inst);
            return r;
        }
        void *inst = kcc_ir_alloc_instr(IR_NEG);
        IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        IL(inst, IR_SRC1) = v;
        unsigned long r = kcc_ir_next_vreg();
        IL(inst, IR_DEST) = r;
        kcc_ir_emit(inst);
        void *ot = (void *)IL(operand, AST_F3);
        return kcc_ir_norm32(r, IR_NEG, ot, ot);
    }
    if (op == TOK_TILDE) {
        void *inst = kcc_ir_alloc_instr(IR_NOT);
        IL(inst, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        IL(inst, IR_SRC1) = v;
        unsigned long r = kcc_ir_next_vreg();
        IL(inst, IR_DEST) = r;
        kcc_ir_emit(inst);
        void *ot = (void *)IL((void *)IL(e, AST_F1
   ), AST_F3);
        return kcc_ir_norm32(r, IR_NOT, ot, ot);
    }
    if (op == TOK_BANG) {
        void *operand = (void *)IL(e, AST_F1);
        void *cty = kcc_type_int_s;
        if (operand) {
            void *ot = (void *)IL(operand, AST_F3)
   ;
            if (ot) {
                unsigned long k = IL(ot, TY_KIND);
                /* Compare against zero at the
   operand's own width: a 64-bit
                   value (long or pointer) whose
   low 32 bits happen to be zero
                   must still test true only when
   the whole word is zero. */
                if (k >= TY_FLOAT && k <=
   TY_LDOUBLE) cty = kcc_type_double;
                else if (IL(ot, TY_SIZE) > 4) cty
   = kcc_type_long_s;
            }
        }
        void *c = kcc_ir_alloc_instr(IR_CONST);
        IL(c, IR_TYPE) = (unsigned long)cty;
        IL(c, IR_SRC1) = 0;
```

```
        unsigned long zv = kcc_ir_next_vreg();
        IL(c, IR_DEST) = zv;
        kcc_ir_emit(c);
        void *cmp = kcc_ir_alloc_instr(IR_CMP_EQ);
        IL(cmp, IR_TYPE) = (unsigned long)cty;
        IL(cmp, IR_SRC1) = v;
        IL(cmp, IR_SRC2) = zv;
        unsigned long r = kcc_ir_next_vreg();
        IL(cmp, IR_DEST) = r;
        kcc_ir_emit(cmp);
        return r;
    }
    if (op == TOK_STAR) {
        void *inst = kcc_ir_alloc_instr(IR_LOAD);
        void *pt = (void *)IL(e, AST_F3);
        if (pt) {
            IL(inst, IR_TYPE) = (unsigned long)pt;
            IL(inst, IR_SRC2) = kcc_ir_load_width(
    pt);
        } else {
            IL(inst, IR_TYPE) = (unsigned long)
    kcc_type_int_s;
            IL(inst, IR_SRC2) = 0;
        }
        IL(inst, IR_SRC1) = v;
        unsigned long r = kcc_ir_next_vreg();
        IL(inst, IR_DEST) = r;
        kcc_ir_emit(inst);
        return r;
    }
    return v;
    /* default: operand as-is */
}
```

□

865 · defined at 856, 859, 860, 862, 865, 872 · used in 902a

⟨ *Selfhost: ir exprs* 872 ⟩ + ≡

```
/* ++/-- : load-modify-store in the operand's own
   type; postfix returns the
   old value, prefix the new.  Bit-fields go
   through the masked RMW. */
unsigned long kcc_ir_build_inc_dec(void *e) {
    void *operand = (void *)IL(e, AST_F1);
    while (operand && IL(operand, AST_TAG) ==
   AST_PAREN_EXPR) {
        operand = (void *)IL(operand, AST_F0);
        IL(e, AST_F1) = (unsigned long)operand;
    }
    unsigned long op = IL(e, AST_F0);
    unsigned long prefix = IL(e, AST_F2);
    operand = (void *)IL(e, AST_F1);
    if (!operand) return 0;
    unsigned long tag = IL(operand, AST_TAG);
    unsigned long addr;
    void *bf = 0;
    if (tag == AST_MEMBER) {
        addr = kcc_ir_build_member_addr(operand);
        void *field = kcc_ir_member_field;
        if (field && IL(field, AST_F2)) bf = field
    ;
    } else if (tag == AST_SUBSCRIPT) {
        addr = kcc_ir_build_subscript_addr(operand
    );
    } else if (tag == AST_UNARY_EXPR) {
        if (IL(operand, AST_F0) != TOK_STAR)
    return 0;
        addr = kcc_ir_build_expr((void *)IL(
    operand, AST_F1));
    } else if (tag == AST_IDENT_EXPR) {
        char *name = (char *)IL(operand, AST_F0);
        if (!name) return 0;
        kcc_ir_local_const_kill(name);   /* ++/--
    changes the value */
        unsigned long keylen = kcc_strlen(name);
        unsigned long v = (unsigned long)
    kcc_ht_get(kcc_ir_locals, name, keylen);
        if (v) {
            addr = (v & ~0x8000000000000000UL) -
    1;
        } else {
            char *symname = name;
            if (kcc_ir_statics) {
                void *mangled = kcc_ht_get(
    kcc_ir_statics, name, keylen);
                if (mangled) symname = (char *)
    mangled;
            }
            void *g = kcc_ir_alloc_instr(
    IR_GLOBAL_REF);
            IL(g, IR_SRC1) = (unsigned long)
    symname;
            IL(g, IR_TYPE) = (unsigned long)
    kcc_type_int_s;
            addr = kcc_ir_next_vreg();
            IL(g, IR_DEST) = addr;
            kcc_ir_emit(g);
        }
    } else {
        return 0;
    }
    if (bf) {
        unsigned long old = kcc_ir_bitfield_load(
    addr, bf);
        unsigned long one = kcc_ir_emit_const(1);
        unsigned long nw = kcc_ir_emit_binop(op ==
     TOK_INC ? IR_ADD : IR_SUB, old, one);
        kcc_ir_bitfield_store(addr, nw, bf);
        return prefix ? nw : old;
    }
    void *wt = kcc_type_int_s;
    unsigned long step = 1;
    unsigned long is_fp = 0;
    void *ot = (void *)IL(operand, AST_F3);
    if ((unsigned long)ot >= 0x1000) {
        unsigned long k = IL(ot, TY_KIND);
        if (k == TY_PTR) {
            wt = ot;
            void *pointee = (void *)IL(ot, TY_F0);
            if (pointee) { unsigned long s =
    kcc_type_layout(pointee); if (s) step = s; }
        } else if ((k >= TY_BOOL && k <= TY_LLONG)
     || k == TY_ENUM) {
            wt = ot;
        } else if (k == TY_FLOAT || k == TY_DOUBLE
    ) {
            wt = ot; is_fp = 1;
        }
    }
    void *ld = kcc_ir_alloc_instr(IR_LOAD);
    IL(ld, IR_TYPE) = (unsigned long)wt;
    IL(ld, IR_SRC1) = addr;
    IL(ld, IR_SRC2) = kcc_ir_load_width(wt);
    unsigned long old = kcc_ir_next_vreg();
    IL(ld, IR_DEST) = old;
    kcc_ir_emit(ld);
    unsigned long stepv;
    if (is_fp) {
        unsigned long one = kcc_ir_emit_const(1);
        stepv = kcc_ir_convert(one, kcc_type_int_s
    , wt);
```

```
    } else {
        void *c = kcc_ir_alloc_instr(IR_CONST);
        IL(c, IR_TYPE) = (unsigned long)wt;
        IL(c, IR_SRC1) = step;
        stepv = kcc_ir_next_vreg();
        IL(c, IR_DEST) = stepv;
        kcc_ir_emit(c);
    }
    void *bop = kcc_ir_alloc_instr(op == TOK_INC ?
    IR_ADD : IR_SUB);
    IL(bop, IR_TYPE) = (unsigned long)wt;
    IL(bop, IR_SRC1) = old;
    IL(bop, IR_SRC2) = stepv;
    unsigned long nw = kcc_ir_next_vreg();
    IL(bop, IR_DEST) = nw;
    kcc_ir_emit(bop);
    unsigned long k = IL(wt, TY_KIND);
    if (k >= TY_BOOL && k <= TY_LLONG) {
        unsigned long sz = IL(wt, TY_SIZE);
        if (sz < 8) {
            unsigned long ext = IL(wt, TY_F0) ?
   IR_SEXT : IR_ZEXT;
            void *ei = kcc_ir_alloc_instr(ext);
            IL(ei, IR_TYPE) = (unsigned long)wt;
            IL(ei, IR_SRC1) = nw;
            IL(ei, IR_SRC2) = sz;
            nw = kcc_ir_next_vreg();
            IL(ei, IR_DEST) = nw;
            kcc_ir_emit(ei);
        }
    }
    void *st = kcc_ir_alloc_instr(IR_STORE);
    IL(st, IR_TYPE) = (unsigned long)wt;
    IL(st, IR_SRC1) = nw;
    IL(st, IR_SRC2) = addr;
    IL(st, IR_DEST) = kcc_ir_store_width(wt);
    kcc_ir_emit(st);
    return prefix ? nw : old;
}
```

□

872 · defined at 856, 859, 860, 862, 865, 872 · used in 902a

75.7. Aggregates and Declarations

Struct copies walk the object in descending power-of-two chunks for exactly TY_SIZE bytes; build_struct_addr yields the address of a struct-valued expression for by-value passing and return. When that expression is a bare identifier naming a *static-local* aggregate, its address must resolve through the same mangled sl_ name that its storage is emitted under—the member-access path already does this, and the by-value argument path must match it, or passing such a struct whole (the kernel's str_spec handed to string_nocheck) relocates against a raw name that nothing defines. The recursive initializer descends nested aggregates, honouring designator chains and brace elision, over a zero-filled region.

⟨ *Selfhost: ir aggregates* 873 ⟩ ≡

```
void kcc_ir_emit_struct_copy(unsigned long src,
   unsigned long dst, unsigned long size) {
    unsigned long off = 0;
    while ((long)(size - off) > 0) {
        unsigned long rem = size - off;
        unsigned long chunk, code;
        if (rem >= 8) { chunk = 8; code = 0; }
        else if (rem >= 4) { chunk = 4; code = 4;
   }
        else if (rem >= 2) { chunk = 2; code = 2;
   }
        else { chunk = 1; code = 1; }
        void *c = kcc_ir_alloc_instr(IR_CONST);
        IL(c, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        IL(c, IR_SRC1) = off;
        unsigned long idx = kcc_ir_next_vreg();
        IL(c, IR_DEST) = idx;
        kcc_ir_emit(c);
        void *g1 = kcc_ir_alloc_instr(IR_GEP);
        IL(g1, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        IL(g1, IR_SRC1) = src;
        IL(g1, IR_SRC2) = idx;
        unsigned long se = kcc_ir_next_vreg();
        IL(g1, IR_DEST) = se;
        kcc_ir_emit(g1);
        void *ld = kcc_ir_alloc_instr(IR_LOAD);
        IL(ld, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        IL(ld, IR_SRC1) = se;
        IL(ld, IR_SRC2) = code;
        unsigned long val = kcc_ir_next_vreg();
        IL(ld, IR_DEST) = val;
        kcc_ir_emit(ld);
        void *g2 = kcc_ir_alloc_instr(IR_GEP);
        IL(g2, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        IL(g2, IR_SRC1) = dst;
        IL(g2, IR_SRC2) = idx;
        unsigned long de = kcc_ir_next_vreg();
        IL(g2, IR_DEST) = de;
        kcc_ir_emit(g2);
        void *st = kcc_ir_alloc_instr(IR_STORE);
        IL(st, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        IL(st, IR_SRC1) = val;
        IL(st, IR_SRC2) = de;
        IL(st, IR_DEST) = code;
        kcc_ir_emit(st);
        off += chunk;
    }
}

unsigned long kcc_ir_build_struct_addr(void *e) {
    if (!e) return 0;
    if (IL(e, AST_TAG) == AST_PAREN_EXPR) {
        e = (void *)IL(e, AST_F0);
        if (!e) return 0;
    }
    unsigned long tag = IL(e, AST_TAG);
    if (tag == AST_IDENT_EXPR) {
        char *name = (char *)IL(e, AST_F0);
        unsigned long v = (unsigned long)
   kcc_ht_get(kcc_ir_locals, name, kcc_strlen(name
   ));
        if (v) return (v & ~0x8000000000000000UL)
   - 1;
        /* a static-local aggregate passed whole (
   by value) reaches its storage
           through the same mangled `sl_' name as
   a member access does, so the
           argument's address must map through
   kcc_ir_statics too---otherwise
           `string_nocheck(..., str_spec)' on a `
   static const struct printf_spec
```

```
        str_spec' relocates against the raw `
   str_spec' that nothing defines. */
        char *symname = name;
        if (kcc_ir_statics) {
            void *mangled = kcc_ht_get(
   kcc_ir_statics, name, kcc_strlen(name));
            if (mangled) symname = (char *)mangled
   ;
        }
        void *g = kcc_ir_alloc_instr(IR_GLOBAL_REF
   );
        IL(g, IR_SRC1) = (unsigned long)symname;
        IL(g, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        unsigned long rv = kcc_ir_next_vreg();
        IL(g, IR_DEST) = rv;
        kcc_ir_emit(g);
        return rv;
    }
    if (tag == AST_CALL_EXPR) return
   kcc_ir_build_call(e);
    if (tag == AST_MEMBER) return
   kcc_ir_build_member_addr(e);
    if (tag == AST_SUBSCRIPT) return
   kcc_ir_build_subscript_addr(e);
    if (tag == AST_COMPOUND_LIT) return
   kcc_ir_build_compound_lit(e);
    if (tag == AST_COMMA_EXPR) {
        kcc_ir_build_expr((void *)IL(e, AST_F0));
        return kcc_ir_build_struct_addr((void *)IL
   (e, AST_F1));
    }
    if (tag == AST_ASSIGN_EXPR) return
   kcc_ir_build_assign(e);
    if (tag == AST_UNARY_EXPR) {
        if (IL(e, AST_F0) == TOK_STAR) return
   kcc_ir_build_expr((void *)IL(e, AST_F1));
        return kcc_ir_build_expr(e);
    }
    return kcc_ir_build_expr(e);
   /* pointer operand → address */
}

unsigned long kcc_ir_initlist_count(void *list) {
    void *el = (void *)IL(list, AST_F0);
    unsigned long idx = 0, max = 0;
    while (el) {
        if (IL(el, AST_TAG) == AST_DESIGNATOR && !
   IL(el, AST_F1))
            idx = kcc_eval_const_int((void *)IL(el
   , AST_F0)); /* [i]= : evaluate the index expr
   */
        idx++;
        if (idx > max) max = idx;
        el = (void *)IL(el, AST_F4);
    }
    return max;
}

void kcc_ir_emit_indexed_store(unsigned long base,
    unsigned long off, unsigned long value,
   unsigned long width) {
    void *c = kcc_ir_alloc_instr(IR_CONST);
    IL(c, IR_TYPE) = (unsigned long)kcc_type_int_s
   ;
    IL(c, IR_SRC1) = off;
    unsigned long ov = kcc_ir_next_vreg();
    IL(c, IR_DEST) = ov;
    kcc_ir_emit(c);
    void *g = kcc_ir_alloc_instr(IR_GEP);
    IL(g, IR_TYPE) = (unsigned long)kcc_type_int_s
   ;
    IL(g, IR_SRC1) = base;
    IL(g, IR_SRC2) = ov;
    unsigned long av = kcc_ir_next_vreg();
    IL(g, IR_DEST) = av;
    kcc_ir_emit(g);
    void *st = kcc_ir_alloc_instr(IR_STORE);
    IL(st, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    IL(st, IR_SRC1) = value;
    IL(st, IR_SRC2) = av;
    IL(st, IR_DEST) = width;
    kcc_ir_emit(st);
}
```

□

873 · defined at 873, 874 · used in 902a

A designated initializer names its target member with `kcc_ir_field_index`, which walks a struct or union's field list and returns the zero-based position of the named member (so the positional offset machinery of `kcc_ir_subobj` can be reused), or -1. An *anonymous* member—an unnamed `struct`/`union` whose own members are promoted into the enclosing type (C17 6.7.2.1p13)—carries a `NULL` name, so the by-name scan must skip it rather than hand `NULL` to `kcc_strcmp`. Skipping alone would lose the promoted names, so `kcc_ir_type_has_member` answers, recursively, whether a name is reachable through a type's anonymous members, and `kcc_ir_anon_containing` returns the position of the anonymous member that holds it. `kcc_ir_resolve_designator` uses the pair to *descend*: when a `.name` is not a direct field it steps into the anonymous member that contains it—accumulating that member's byte offset and continuing from its type—until the name resolves to a direct field. The first anonymous member entered is recorded as `first_pos` so the positional cursor resumes at the right outer slot after the designated element. A name that is genuinely absent still yields -1 and the legacy out-of-range path, unchanged.

⟨ *Selfhost: ir aggregates* 874 ⟩ + ≡

```
long kcc_ir_field_index(void *agg, const char *
   name) {
    void *sd = (void *)IL(agg, TY_F0);
    if (!sd) return -1;
    void *field = (void *)IL(sd, AST_F1);
    long idx = 0;
    while (field) {
        const char *fn = (const char *)IL(field,
   AST_F1);
        if (fn && kcc_strcmp(fn, name) == 0)
   return idx;
        field = (void *)IL(field, AST_F4);
        idx++;
    }
    return -1;
}

unsigned long kcc_ir_type_has_member(void *ty,
   const char *name) {
    if (!ty) return 0;
    unsigned long k = IL(ty, TY_KIND);
    if (k != TY_STRUCT && k != TY_UNION) return 0;
    void *sd = (void *)IL(ty, TY_F0);
    if (!sd) return 0;
```

```
    void *field = (void *)IL(sd, AST_F1);
    while (field) {
        const char *fn = (const char *)IL(field,
   AST_F1);
        if (fn) {
            if (kcc_strcmp(fn, name) == 0) return
   1;
        } else {
            void *fts = (void *)IL(field, AST_F0);
            void *aty = fts ?
   kcc_sema_resolve_type(fts) : 0;
            if (kcc_ir_type_has_member(aty, name))
    return 1;
        }
        field = (void *)IL(field, AST_F4);
    }
    return 0;
}

long kcc_ir_anon_containing(void *agg, const char
   *name) {
    void *sd = (void *)IL(agg, TY_F0);
    if (!sd) return -1;
    void *field = (void *)IL(sd, AST_F1);
    long idx = 0;
    while (field) {
        const char *fn = (const char *)IL(field,
   AST_F1);
        if (!fn) {
            void *fts = (void *)IL(field, AST_F0);
            void *aty = fts ?
   kcc_sema_resolve_type(fts) : 0;
            if (kcc_ir_type_has_member(aty, name))
    return idx;
        }
        field = (void *)IL(field, AST_F4);
        idx++;
    }
    return -1;
}

void *kcc_ir_subobj(void *agg, unsigned long idx,
   unsigned long *out_off) {
    kcc_ir_member_field = 0;
    if (!agg) { *out_off = idx << 3; return 0; }
    unsigned long k = IL(agg, TY_KIND);
    if (k == TY_ARRAY) {
        void *et = (void *)IL(agg, TY_F0);
        *out_off = idx * kcc_type_layout(et);
        return et;
    }
    if (k == TY_STRUCT || k == TY_UNION) {
        kcc_type_layout(agg);
        void *sd = (void *)IL(agg, TY_F0);
        if (!sd) { *out_off = 0; return 0; }
        void *field = (void *)IL(sd, AST_F1);
        while (field && idx) { field = (void *)IL(
   field, AST_F4); idx--; }
        if (!field) { *out_off = 0; return 0; }
        kcc_ir_member_field = field;
        *out_off = IL(field, AST_F3);
        void *fts = (void *)IL(field, AST_F0);
        return fts ? kcc_sema_resolve_type(fts) :
   0;
    }
    *out_off = idx * kcc_type_layout(agg);
    return agg;
   /* scalar sub-object is itself */
}

struct desig_res kcc_ir_resolve_designator(void *
   agg, unsigned long base_off, void *desig) {
    void *cur_type = agg;
    unsigned long cur_off = base_off;
    unsigned long first_pos = (unsigned long)-1;
    void *leaf_field = 0;
    while (desig) {
        unsigned long pos;
        unsigned long outer_pos;
        if (IL(desig, AST_F1)) {
            const char *nm = (const char *)IL(
   desig, AST_F0);
            long p = kcc_ir_field_index(cur_type,
   nm);
            /* A `.name` that is not a direct
   member of cur_type but lives inside
               an unnamed struct/union member is
   reached by first stepping into
               that member (C17 6.7.2.1p13):
   descend, accumulating its offset,
               until the name resolves to a direct
    field. */
            while (p < 0) {
                long ap = kcc_ir_anon_containing(
   cur_type, nm);
                if (ap < 0) break;
                unsigned long aoff;
                void *asub = kcc_ir_subobj(
   cur_type, (unsigned long)ap, &aoff);
                cur_off += aoff;
                cur_type = asub;
                if (first_pos == (unsigned long)
   -1) first_pos = (unsigned long)ap;
                p = kcc_ir_field_index(cur_type,
   nm);
            }
            pos = (unsigned long)p;
            outer_pos = pos;
        } else {
            pos = kcc_eval_const_int((void *)IL(
   desig, AST_F0)); /* [i] index expr */
            outer_pos = pos;
        }
        unsigned long off;
        void *sub = kcc_ir_subobj(cur_type, pos, &
   off);
        cur_off += off;
        cur_type = sub;
        leaf_field = kcc_ir_member_field;
        if (first_pos == (unsigned long)-1)
   first_pos = outer_pos;
        desig = (void *)IL(desig, AST_F2);
    }
    struct desig_res r;
    r.off = cur_off; r.sub_type = cur_type; r.
   first_pos = first_pos; r.field = leaf_field;
    return r;
}

unsigned long kcc_ir_agg_count(void *ty) {
    if (!ty) return 1;
    unsigned long k = IL(ty, TY_KIND);
    if (k == TY_ARRAY) return IL(ty, TY_F1);
    if (k == TY_STRUCT) {
        void *sd = (void *)IL(ty, TY_F0);
        if (!sd) return 1;
        void *field = (void *)IL(sd, AST_F1);
        unsigned long n = 0;
        while (field) { n++; field = (void *)IL(
   field, AST_F4); }
        return n;
    }
    return 1;
```

```
}

void kcc_ir_elem_off_width(void *ty, unsigned long
    idx, unsigned long *out_off, unsigned long *
   out_width) {
    kcc_ir_member_field = 0;
    if (!ty) { *out_off = idx << 3; *out_width =
   0; return; }
    unsigned long k = IL(ty, TY_KIND);
    if (k == TY_STRUCT || k == TY_UNION) {
        kcc_type_layout(ty);
        void *sd = (void *)IL(ty, TY_F0);
        if (!sd) { *out_off = 0; *out_width = 0;
   return; }
        void *field = (void *)IL(sd, AST_F1);
        while (field && idx) { field = (void *)IL(
   field, AST_F4); idx--; }
        if (!field) { *out_off = 0; *out_width =
   0; return; }
        kcc_ir_member_field = field;
        *out_off = IL(field, AST_F3);
        void *fts = (void *)IL(field, AST_F0);
        *out_width = fts ? kcc_ir_store_width(
   kcc_sema_resolve_type(fts)) : 0;
        return;
    }
    *out_off = idx * kcc_type_layout(ty);
    *out_width = kcc_ir_store_width(ty);
}

void kcc_ir_fill_one(void *ty, unsigned long base,
    unsigned long off, void *node, void *field) {
    if (!node) return;
    if (IL(node, AST_TAG) == AST_INIT_LIST) {
        kcc_ir_fill_list(ty, base, off, (void *)IL
   (node, AST_F0));
        return;
    }
    if (ty) {
        unsigned long k = IL(ty, TY_KIND);
        if (k == TY_STRUCT || k == TY_UNION || k
   == TY_ARRAY) {
            kcc_ir_fill(ty, base, off, node);
            return;
        }
    }
    unsigned long val = kcc_ir_build_expr(node);
    void *from = (void *)IL(node, AST_F3);
    if (from && ty) val = kcc_ir_convert(val, from
   , ty);
    kcc_ir_member_field = field;
    kcc_ir_store_elem(base, off, val,
   kcc_ir_store_width(ty));
}

void *kcc_ir_fill(void *ty, unsigned long base,
   unsigned long off, void *elem) {
    if (!elem) return elem;
    unsigned long agg = 0;
    if (ty) {
        unsigned long k = IL(ty, TY_KIND);
        if (k == TY_STRUCT || k == TY_UNION || k
   == TY_ARRAY) agg = 1;
    }
    if (!agg) {
        kcc_ir_fill_one(ty, base, off, elem, 0);
        return (void *)IL(elem, AST_F4);
    }
    unsigned long etag = IL(elem, AST_TAG);
    if (etag != AST_INIT_LIST) {
        if (etag == AST_STR_LIT && IL(ty, TY_KIND)
    == TY_ARRAY) {
            unsigned long cap = kcc_type_layout(ty
   );
            const char *sp = (const char *)IL(elem
   , AST_F0);
            unsigned long len = IL(elem, AST_F1);
            if (cap && len > cap) len = cap;
            unsigned long i = 0;
            while (i < len) {
                unsigned long c =
   kcc_ir_emit_const((unsigned char)sp[i]);
                kcc_ir_emit_indexed_store(base,
   off + i, c, 1);
                i++;
            }
            return (void *)IL(elem, AST_F4);
        }
        if (!kcc_type_layout(ty)) {
    /* zero-size aggregate element */
            kcc_ir_build_struct_addr(elem);
    /* evaluate for side effects */
            return (void *)IL(elem, AST_F4);
        }
        void *et = (void *)IL(elem, AST_F3);
        if (et) {
            unsigned long ek = IL(et, TY_KIND);
            if (ek == TY_STRUCT || ek == TY_UNION)
    {
                unsigned long dst =
   kcc_ir_emit_gep_const(base, off);
                unsigned long src =
   kcc_ir_build_struct_addr(elem);
                kcc_ir_emit_struct_copy(src, dst,
   kcc_type_layout(ty));
                return (void *)IL(elem, AST_F4);
            }
        }
    }
    unsigned long count = kcc_ir_agg_count(ty);
    unsigned long cursor = 0;
    while (elem && cursor < count) {
        unsigned long so;
        void *sub = kcc_ir_subobj(ty, cursor, &so)
   ;
        elem = kcc_ir_fill(sub, base, off + so,
   elem);
        cursor++;
    }
    return elem;
}

void kcc_ir_fill_list(void *ty, unsigned long base
   , unsigned long base_off, void *head) {
    void *elem = head;
    unsigned long cursor = 0;
    while (elem) {
        if (IL(elem, AST_TAG) == AST_DESIGNATOR) {
            if (IL(elem, AST_F1) == 2) {
    /* GNU array range [lo ... hi] = v */
                unsigned long lo =
   kcc_eval_const_int((void *)IL(elem, AST_F0));
                unsigned long hi =
   kcc_eval_const_int((void *)IL(elem, AST_F2));
                void *val = (void *)IL(elem,
   AST_F3);
                for (unsigned long pos = lo; pos
   <= hi; pos++) {
                    unsigned long off;
                    void *sub = kcc_ir_subobj(ty,
   pos, &off);
                    kcc_ir_fill_one(sub, base,
```

```
    base_off + off, val, kcc_ir_member_field);
                }
                cursor = hi + 1;
                elem = (void *)IL(elem, AST_F4);
                continue;
            }
            struct desig_res dr =
    kcc_ir_resolve_designator(ty, base_off, elem);
            cursor = dr.first_pos;
            kcc_ir_fill_one(dr.sub_type, base, dr.
    off, (void *)IL(elem, AST_F3), dr.field);
            cursor++;
            elem = (void *)IL(elem, AST_F4);
            continue;
        }
        unsigned long off;
        void *sub = kcc_ir_subobj(ty, cursor, &off
    );
        unsigned long abs = base_off + off;
        void *field = kcc_ir_member_field;
        if (field && !IL(field, AST_F1) && IL(
    field, AST_F2)) {   /* unnamed bit-field */
            cursor++;
            continue;
        }
        if (!sub) {
    /* past the aggregate */
            elem = (void *)IL(elem, AST_F4);
            cursor++;
            continue;
        }
        unsigned long elide = 0;
        if (IL(elem, AST_TAG) != AST_INIT_LIST) {
            unsigned long k = IL(sub, TY_KIND);
            if (k == TY_STRUCT || k == TY_UNION ||
     k == TY_ARRAY) elide = 1;
        }
        if (elide) {
            elem = kcc_ir_fill(sub, base, abs,
    elem);
        } else {
            kcc_ir_fill_one(sub, base, abs, elem,
    field);
            elem = (void *)IL(elem, AST_F4);
        }
        cursor++;
    }
}

unsigned long kcc_ir_build_compound_lit(void *cl)
    {
    void *ty = kcc_sema_resolve_type((void *)IL(cl
    , AST_F0));
    if (!ty) return 0;
    unsigned long slots = (kcc_type_layout(ty) +
    7) >> 3;
    if (!slots) slots = 1;
    void *a = kcc_ir_alloc_instr(IR_ALLOCA);
    IL(a, IR_TYPE) = (unsigned long)ty;
    IL(a, IR_SRC1) = slots;
    unsigned long base = kcc_ir_next_vreg();
    IL(a, IR_DEST) = base;
    kcc_ir_emit(a);
    void *c = kcc_ir_alloc_instr(IR_CONST);
    IL(c, IR_TYPE) = (unsigned long)kcc_type_int_s
    ;
    IL(c, IR_SRC1) = 0;
    unsigned long zv = kcc_ir_next_vreg();
    IL(c, IR_DEST) = zv;
    kcc_ir_emit(c);
    unsigned long i = 0;
    while (i < slots) { kcc_ir_emit_indexed_store(
    base, i << 3, zv, 0); i++; }
    void *init = (void *)IL(cl, AST_F1);
    if (init) {
        if (IL(init, AST_TAG) == AST_INIT_LIST)
            kcc_ir_fill_list(ty, base, 0, (void *)
    IL(init, AST_F0));
        else
            kcc_ir_fill_one(ty, base, 0, init, 0);
    }
    return base;
}

const char *kcc_ir_static_name(void *decl) {
    char *b = kcc_arena_alloc(24);
    b[0] = 's'; b[1] = 'l'; b[2] = '_';
    /* The decl address is an arena offset that
    resets each compilation, so two
       files' statics can collide; xor in the per-
    file salt to make the name
       unique across a whole-kernel link yet
    stable within this unit. */
    unsigned long v = (unsigned long)decl ^
    kcc_tu_salt;
    char *p = b + 3;
    long shift = 60;
    while (shift >= 0) {
        unsigned long nib = (v >> shift) & 0xf;
        *p = (nib < 10) ? (char)('0' + nib) : (
    char)('a' + (nib - 10));
        p++;
        shift -= 4;
    }
    *p = 0;
    return b;
}

/* Resolve every `&&label' buried in an
   initializer to its label's basic block
   NOW, while this function's label map is live,
   stashing the BB in the node
   (f1) with f2 as a resolved-flag.  A static
   jumptable's initialiser runs later
   as module-init code, in another function's
   context where the label map has
   been reset---the stashed BB lets IR_LADDR still
    name the right block. */
static void kcc_ir_stash_label_addrs(void *n) {
    if (!n) return;
    unsigned long tag = IL(n, AST_TAG);
    if (tag == AST_LABEL_ADDR) {
        void *bb = kcc_ir_label_bb((const char *)
    IL(n, AST_F0));
        if (bb) { IL(n, AST_F1) = (unsigned long)
    bb; IL(n, AST_F2) = 1; }
    } else if (tag == AST_INIT_LIST) {
        for (void *e = (void *)IL(n, AST_F0); e; e
    = (void *)IL(e, AST_F4))
            kcc_ir_stash_label_addrs(e);
    } else if (tag == AST_DESIGNATOR) {
        kcc_ir_stash_label_addrs((void *)IL(n,
    AST_F3));
    } else if (tag == AST_PAREN_EXPR) {
        kcc_ir_stash_label_addrs((void *)IL(n,
    AST_F0));
    } else if (tag == AST_CAST_EXPR) {
        kcc_ir_stash_label_addrs((void *)IL(n,
    AST_F1));
    }
}

void kcc_ir_build_var_decl(void *decl) {
```

```
 void *name = (void *)IL(decl, AST_F1);
 if (!name) return;
 unsigned long sc = IL(decl, AST_F3);
 unsigned long keylen = kcc_strlen((const char
*)name);
 if (sc == 3) {
/* static: hoist to a global */
     const char *mangled = kcc_ir_static_name(
decl);
     kcc_ht_put(kcc_ir_statics, name, keylen, (
void *)mangled);
     kcc_vec_push(kcc_ir_static_decls, &decl);
     if (IL(decl, AST_F2))
kcc_ir_stash_label_addrs((void *)IL(decl,
AST_F2));
     void *tn = (void *)IL(decl, AST_F0);
     if (!tn) return;
     void *rt = kcc_sema_resolve_type(tn);
     if (!rt) return;
     kcc_ht_put(kcc_ir_static_types, name,
keylen, rt);
     return;
 }
 if (sc == 2) {
/* block-scope extern */
     kcc_ir_locals_put((const char *)name,
keylen, 0);
     return;
 }
 void *tn = (void *)IL(decl, AST_F0);
 void *ty = tn ? kcc_sema_resolve_type(tn) : 0;
 if (!ty) ty = kcc_type_int_s;
 unsigned long ptrflag = (IL(ty, TY_KIND) ==
TY_PTR) ? 1 : 0;
 unsigned long slots = 1;
 void *alloca_type = ty;
 unsigned long k = IL(ty, TY_KIND);
 if (k == TY_ARRAY) {
     unsigned long len = IL(ty, TY_F1);
     void *et = (void *)IL(ty, TY_F0);
     if (!len) {
         void *init = (void *)IL(decl, AST_F2);
         if (init && IL(init, AST_TAG) ==
AST_INIT_LIST) len = kcc_ir_initlist_count(init
);
     }
     alloca_type = et;
     if (len) slots = (len * kcc_type_layout(et
) + 7) >> 3;
     else slots = 0;
 } else if (k == TY_STRUCT || k == TY_UNION) {
     slots = kcc_ir_type_slots(ty);
 }
 void *a = kcc_ir_alloc_instr(IR_ALLOCA);
 IL(a, IR_TYPE) = (unsigned long)alloca_type;
 IL(a, IR_SRC1) = slots;
 unsigned long base = kcc_ir_next_vreg();
 IL(a, IR_DEST) = base;
 kcc_ir_emit_entry(a);
 unsigned long lv = base + 1;
 if (ptrflag) lv = lv | 0x8000000000000000UL;
 kcc_ir_locals_put((const char *)name, keylen,
lv);
 void *init = (void *)IL(decl, AST_F2);
 if (!init) return;
 unsigned long itag = IL(init, AST_TAG);
 if (itag == AST_INIT_LIST) {
     void *aggty = kcc_sema_resolve_type((void
*)IL(decl, AST_F0));
     void *c = kcc_ir_alloc_instr(IR_CONST);
     IL(c, IR_TYPE) = (unsigned long)
kcc_type_int_s;
     IL(c, IR_SRC1) = 0;
     unsigned long zv = kcc_ir_next_vreg();
     IL(c, IR_DEST) = zv;
     kcc_ir_emit(c);
     unsigned long i = 0;
     while (i < slots) {
kcc_ir_emit_indexed_store(base, i << 3, zv, 0);
 i++; }
     kcc_ir_fill_list(aggty, base, 0, (void *)
IL(init, AST_F0));
     return;
 }
 if (itag == AST_STR_LIT) {
     void *rt = kcc_sema_resolve_type((void *)
IL(decl, AST_F0));
     if (rt && IL(rt, TY_KIND) == TY_ARRAY) {
         const char *sp = (const char *)IL(init
, AST_F0);
         unsigned long slen = IL(init, AST_F1);
         unsigned long alen = IL(rt, TY_F1);
         if (!alen) alen = slen + 1;
         kcc_ir_emit_string_init(base, sp, slen
, alen, 1);
         return;
     }
 }
 if (k == TY_STRUCT || k == TY_UNION) {
     unsigned long src =
kcc_ir_build_struct_addr(init);
     kcc_ir_emit_struct_copy(src, base,
kcc_type_layout(ty));
     return;
 }
 void *from = (void *)IL(init, AST_F3);
 unsigned long val = kcc_ir_build_expr(init);
 val = kcc_ir_convert(val, from, ty);
 void *st = kcc_ir_alloc_instr(IR_STORE);
 IL(st, IR_TYPE) = (unsigned long)ty;
 IL(st, IR_SRC1) = val;
 IL(st, IR_SRC2) = base;
 IL(st, IR_DEST) = kcc_ir_store_width(ty);
 kcc_ir_emit(st);
 /* Record a scalar-integer local whose
initialiser is an integer constant
    expression, so an `"i"' inline-asm operand
naming it (the kernel's
    `__auto_type __flags = ...' passed to WARN'
s `"i"(flags)') folds to the
    constant instead of being materialised in a
 register.  x86 only. */
 if (kcc_target) {
     unsigned long kk = IL(ty, TY_KIND);
     if (((kk >= TY_BOOL && kk <= TY_LLONG) ||
kk == TY_ENUM) &&
         kcc_expr_is_ice(init))
         kcc_ir_local_const_set((const char *)
name, keylen, lv,
                                (unsigned long)
kcc_eval_const_int(init));
 }
}
```

□

874 · defined at 873, 874 · used in 902a

75.8. Control Flow

Each control-flow builder allocates fresh blocks and wires them with IR_BR/IR_BR_COND, marking a block terminated after its terminator so nothing is appended past a branch. switch scans its body twice (allocate label blocks, emit the dispatch chain) then builds the body once so case/default labels place themselves. A case whose label carries a high bound in f3 is a GNU range and dispatches through two signed comparisons (>= LOW then <= HIGH) with a middle block, rather than the single equality test of an ordinary case. The constant evaluator folds integer constant expressions for case values and array bounds, including offsetof-style null-based address constants: is_ice accepts &(((T*)0)->m) and eval_const_int reduces it to the member's byte offset (via eval_addr), so a bound such as char b[sizeof(P)-offsetof(P,tail)] lays out at its true size rather than being rejected as a variable-length array. It also folds the bit-counting builtins—__builtin_clz/ctz/popcount/ffs and their l/ll widths—and the ?: operator, because the kernel derives array bounds through ilog2 and BITS_TO_LONGS, which expand to __builtin_constant_p(n) ? (63 - __builtin_clzll(n)) : without folding the clz call and the ternary, such a bound looks variable and the array is wrongly rejected as a VLA.

When the controlling expression is itself an integer constant expression, the whole dispatch is dead weight: only one arm can ever run, yet emitting every arm is exactly what forces kcc to lower the kernel's size-dispatched inline asm. __raw_cmpxchg switches on a compile-time-constant sizeof and each arm carries a differently-sized cmpxchg; GCC inlines with a literal size, folds the switch, and emits only the live arm, whereas kcc—not inlining—would render the whole switch including the byte-wide cmpxchgb on a wider register. So build_switch folds a constant switch the way build_if folds a constant condition: it evaluates the control value, finds the matching case (or default, or nothing), and builds only that arm as straight-line code, honouring fall-through into later arms and stopping at the first break.

The fold applies only when the body is a single compound whose case and default labels are all direct children—is_flat confirms a deep scan finds no label nested inside an arm—so that fall-through is a simple walk of the sibling list and no nested label is ever built without its dispatch block. Anything more intricate falls back to the ordinary two-scan lowering, which is always correct.

Dropping the untaken if arm has one exception: a label inside it may be the target of an asm goto (or an ordinary goto) from live code. The kernel's atomic __GEN_RMWcc plants precisely if (0) { cc_label: c = true; } and jumps to cc_label out of the asm on the condition flag—the label is reachable *only* through that jump, never through normal control. Labels are pre-allocated blocks (prescan_labels), but a block is emitted only once set_bb appends it to the function's block list, and that happens when its label statement is *built*. Silently dropping the arm therefore leaves the block referenced by the asm but never placed, so the assembler sees an undefined .Lkccbb target. build_if guards against this: if the dropped arm contains any label (kcc_stmt_has_label, the same traversal prescan_labels uses), it is lowered into an unreachable block—entered only by the jump, falling through to the continuation—so the label is placed and its code emitted while normal flow steps straight past it.

build_for lowers the three clauses in the obvious order, with one subtlety in the initialiser: a for-init declaration may bind several objects at once (for (T a, *b = …; …)), and parse_decl chains the extra declarators on f4 exactly as it does inside a block. Building only the head node would give the siblings no storage, so the initialiser is walked as an f4 chain (a plain expression init has no chain and runs the loop once). The type checker walks the same chain for the same reason; the kernel's scoped_guard relies on it, expanding to for (CLASS(…)(args), *done = NULL; !done; …).

⟨ *Selfhost: ir control* 879 ⟩ ≡

```
static void ir_brcond(unsigned long cond, void *tb
    , void *fb) {
     void *bc = kcc_ir_alloc_instr(IR_BR_COND);
     IL(bc, IR_DEST) = cond;
     IL(bc, IR_SRC1) = IL(tb, BB_ID);
     IL(bc, IR_SRC2) = IL(fb, BB_ID);
     kcc_ir_emit(bc);
     IL(kcc_ir_cur_bb, BB_FLAGS) = IL(kcc_ir_cur_bb
    , BB_FLAGS) | BB_FLAG_TERMINATED;
}

/* Does a statement subtree contain a label?
   Mirrors kcc_ir_prescan_labels'
   traversal.  A constant-folded `if' drops its
   untaken branch, but a goto or
   asm-goto elsewhere can still target a label
   buried in that branch --- the
   kernel's __GEN_RMWcc plants exactly `if (0) {
   cc_label: c = 1; }` --- so such
   a branch must still be lowered even though
   normal control never enters it. */
static int kcc_stmt_has_label(void *s) {
     if (!s) return 0;
     unsigned long tag = IL(s, AST_TAG);
     if (tag == AST_LABEL_STMT) return 1;
     if (tag == AST_COMPOUND) {
         for (void *it = (void *)IL(s, AST_F0); it;
     it = (void *)IL(it, AST_F4))
             if (kcc_stmt_has_label(it)) return 1;
         return 0;
     }
     if (tag == AST_IF_STMT)
         return kcc_stmt_has_label((void *)IL(s,
    AST_F1)) ||
                kcc_stmt_has_label((void *)IL(s,
    AST_F2));
     if (tag == AST_WHILE_STMT || tag ==
    AST_SWITCH_STMT || tag == AST_CASE_STMT)
         return kcc_stmt_has_label((void *)IL(s,
    AST_F1));
     if (tag == AST_DO_STMT || tag ==
    AST_DEFAULT_STMT)
         return kcc_stmt_has_label((void *)IL(s,
    AST_F0));
     if (tag == AST_FOR_STMT)
         return kcc_stmt_has_label((void *)IL(s,
    AST_F3));
     return 0;
}
```

```
void kcc_ir_build_if(void *s) {
    /* A compile-time-constant condition selects
   one branch; build only that
       one and drop the other entirely.  The
   kernel's size-dispatched inline
       asm depends on this: the untaken branch (e.
   g. a byte `cmpxchgb` reached
       only when `size==1`) would otherwise be
   lowered and emit an assembler
       error even though it can never run. */
    void *cond_e = (void *)IL(s, AST_F0);
    if (kcc_expr_is_ice(cond_e)) {
        unsigned long v = kcc_eval_const_int(
   cond_e);
        void *taken   = v ? (void *)IL(s, AST_F1)
   : (void *)IL(s, AST_F2);
        void *dropped = v ? (void *)IL(s, AST_F2)
   : (void *)IL(s, AST_F1);
        if (dropped && kcc_stmt_has_label(dropped)
   ) {
            /* The untaken branch is unreachable
   by normal flow but holds a label
               an asm-goto (or goto) targets.
   Lower it into an unreachable block
               so the label is placed (
   kcc_ir_set_bb appends it to the emit list)
               and its code emitted; normal flow
   jumps straight past it. */
            void *cont_bb = kcc_ir_alloc_bb();
            if (taken) kcc_ir_build_stmt(taken);
            kcc_ir_branch_to(cont_bb);
            kcc_ir_set_bb(kcc_ir_alloc_bb());   /*
    only the label's jump enters here */
            kcc_ir_build_stmt(dropped);
            kcc_ir_branch_to(cont_bb);
            kcc_ir_set_bb(cont_bb);
        } else if (taken) {
            kcc_ir_build_stmt(taken);
        }
        return;
    }
    unsigned long cond = kcc_ir_build_expr(cond_e)
   ;
    void *then_bb = kcc_ir_alloc_bb();
    void *else_node = (void *)IL(s, AST_F2);
    if (else_node) {
        void *else_bb = kcc_ir_alloc_bb();
        void *merge_bb = kcc_ir_alloc_bb();
        ir_brcond(cond, then_bb, else_bb);
        kcc_ir_set_bb(then_bb);
        kcc_ir_build_stmt((void *)IL(s, AST_F1));
        kcc_ir_branch_to(merge_bb);
        kcc_ir_set_bb(else_bb);
        kcc_ir_build_stmt(else_node);
        kcc_ir_branch_to(merge_bb);
        kcc_ir_set_bb(merge_bb);
    } else {
        void *merge_bb = kcc_ir_alloc_bb();
        ir_brcond(cond, then_bb, merge_bb);
        kcc_ir_set_bb(then_bb);
        kcc_ir_build_stmt((void *)IL(s, AST_F1));
        kcc_ir_branch_to(merge_bb);
        kcc_ir_set_bb(merge_bb);
    }
}

void kcc_ir_build_while(void *s) {
    void *cond_bb = kcc_ir_alloc_bb();
    void *body_bb = kcc_ir_alloc_bb();
    void *exit_bb = kcc_ir_alloc_bb();
    kcc_ir_branch_to(cond_bb);
    kcc_ir_set_bb(cond_bb);
    unsigned long cond = kcc_ir_build_expr((void
   *)IL(s, AST_F0));
    ir_brcond(cond, body_bb, exit_bb);
    kcc_loop_push(IL(exit_bb, BB_ID), IL(cond_bb,
   BB_ID));
    kcc_ir_set_bb(body_bb);
    kcc_ir_build_stmt((void *)IL(s, AST_F1));
    kcc_loop_pop();
    kcc_ir_branch_to(cond_bb);
    kcc_ir_set_bb(exit_bb);
}

void kcc_ir_build_do(void *s) {
    void *body_bb = kcc_ir_alloc_bb();
    void *cond_bb = kcc_ir_alloc_bb();
    void *exit_bb = kcc_ir_alloc_bb();
    kcc_ir_branch_to(body_bb);
    kcc_loop_push(IL(exit_bb, BB_ID), IL(cond_bb,
   BB_ID));
    kcc_ir_set_bb(body_bb);
    kcc_ir_build_stmt((void *)IL(s, AST_F0));
    kcc_loop_pop();
    kcc_ir_branch_to(cond_bb);
    kcc_ir_set_bb(cond_bb);
    unsigned long cond = kcc_ir_build_expr((void
   *)IL(s, AST_F1));
    ir_brcond(cond, body_bb, exit_bb);
    kcc_ir_set_bb(exit_bb);
}

void kcc_ir_build_for(void *s) {
    /* Walk the init's f4 chain: a for-init
   declaration may bind several objects
       (`for (T a, *b = ...; ...)'), chained as a
   block's declarators are.
       Building only the head would leave the
   siblings without storage. */
    for (void *init = (void *)IL(s, AST_F0); init;
    init = (void *)IL(init, AST_F4))
        kcc_ir_build_stmt(init);
    void *cond_bb = kcc_ir_alloc_bb();
    void *body_bb = kcc_ir_alloc_bb();
    void *incr_bb = kcc_ir_alloc_bb();
    void *exit_bb = kcc_ir_alloc_bb();
    kcc_ir_branch_to(cond_bb);
    kcc_ir_set_bb(cond_bb);
    void *cnode = (void *)IL(s, AST_F1);
    if (cnode) {
        unsigned long cond = kcc_ir_build_expr(
   cnode);
        ir_brcond(cond, body_bb, exit_bb);
    } else {
        kcc_ir_branch_to(body_bb);
    }
    kcc_loop_push(IL(exit_bb, BB_ID), IL(incr_bb,
   BB_ID));
    kcc_ir_set_bb(body_bb);
    void *body = (void *)IL(s, AST_F3);
    if (body) kcc_ir_build_stmt(body);
    kcc_loop_pop();
    kcc_ir_branch_to(incr_bb);
    kcc_ir_set_bb(incr_bb);
    void *incr = (void *)IL(s, AST_F2);
    if (incr) kcc_ir_build_expr(incr);
    kcc_ir_branch_to(cond_bb);
    kcc_ir_set_bb(exit_bb);
}
```

```
/* The kernel computes array bounds (ilog2/
   const_ilog2/BITS_TO_LONGS, hash
   orders) through the bit-counting builtins, so
   they must fold in a constant
   expression.  kcc_call_foldable_builtin
   recognises the call; kcc_call_builtin_value
   evaluates it from the constant argument. */
static unsigned long kcc_clz_w(unsigned long x,
   unsigned long width) {
    x &= (width >= 64) ? ~0UL : ((1UL << width) -
   1);
    if (x == 0) return width;
    unsigned long n = 0;
    while (!(x & (1UL << (width - 1)))) { x <<= 1;
    n++; }
    return n;
}
static unsigned long kcc_ctz_w(unsigned long x,
   unsigned long width) {
    x &= (width >= 64) ? ~0UL : ((1UL << width) -
   1);
    if (x == 0) return width;
    unsigned long n = 0;
    while (!(x & 1UL)) { x >>= 1; n++; }
    return n;
}
static unsigned long kcc_popcount_w(unsigned long
   x) {
    unsigned long n = 0;
    while (x) { n += x & 1UL; x >>= 1; }
    return n;
}
unsigned long kcc_call_foldable_builtin(void *n) {
    void *callee = (void *)IL(n, AST_F0);
    if (!callee || IL(callee, AST_TAG) !=
   AST_IDENT_EXPR) return 0;
    const char *m = (const char *)IL(callee,
   AST_F0);
    if (!m) return 0;
    if (kcc_strcmp(m, "__builtin_clz") == 0 ||
   kcc_strcmp(m, "__builtin_clzl") == 0 ||
        kcc_strcmp(m, "__builtin_clzll") == 0 ||
   kcc_strcmp(m, "__builtin_ctz") == 0 ||
        kcc_strcmp(m, "__builtin_ctzl") == 0 ||
   kcc_strcmp(m, "__builtin_ctzll") == 0 ||
        kcc_strcmp(m, "__builtin_popcount") == 0
   || kcc_strcmp(m, "__builtin_popcountl") == 0 ||
        kcc_strcmp(m, "__builtin_popcountll") == 0
    || kcc_strcmp(m, "__builtin_ffs") == 0 ||
        kcc_strcmp(m, "__builtin_ffsl") == 0 ||
   kcc_strcmp(m, "__builtin_ffsll") == 0)
        return 1;
    return 0;
}
unsigned long kcc_call_builtin_value(void *n) {
    const char *m = (const char *)IL((void *)IL(n,
    AST_F0), AST_F0);  /* "__builtin_..." */
    unsigned long x = kcc_eval_const_int((void *)
   IL(n, AST_F1));
    unsigned long len = kcc_strlen(m);
    unsigned long w = (m[len - 1] == 'l') ? 64 :
   32;   /* an l/ll suffix widens to long */
    if (m[10] == 'c' && m[11] == 'l') return
   kcc_clz_w(x, w);           /* __builtin_clz*
   */
    if (m[10] == 'c' && m[11] == 't') return
   kcc_ctz_w(x, w);           /* __builtin_ctz*
   */
    if (m[10] == 'p') return kcc_popcount_w(x);
                        /* __builtin_popcount* */
    return x ? kcc_ctz_w(x, w) + 1 : 0;
                                /* __builtin_ffs*  */
}

unsigned long kcc_eval_const_int(void *n) {
    if (!n) return 0;
    unsigned long tag = IL(n, AST_TAG);
    if (tag == AST_INT_LIT || tag == AST_CHAR_LIT)
    return IL(n, AST_F0);
    if (tag == AST_CALL_EXPR && kcc_call_is_bcp(n)
   )
        return kcc_expr_is_ice((void *)IL(n,
   AST_F1)) ? 1 : 0;   /* __builtin_constant_p(x)
   */
    if (tag == AST_CALL_EXPR &&
   kcc_call_foldable_builtin(n))
        return kcc_call_builtin_value(n);
                    /* __builtin_clz/ctz/popcount/
   ffs */
    if (tag == AST_CALL_EXPR &&
   kcc_call_is_choose_expr(n)) {
        void *a0 = (void *)IL(n, AST_F1);
        void *a1 = a0 ? (void *)IL(a0, AST_F4) :
   0;
        void *a2 = a1 ? (void *)IL(a1, AST_F4) :
   0;
        return kcc_eval_const_int(a0) ?
   kcc_eval_const_int(a1) : kcc_eval_const_int(a2)
   ;
    }
    if (tag == AST_COND_EXPR)
        return kcc_eval_const_int((void *)IL(n,
   AST_F0))
                ? kcc_eval_const_int((void *)IL(n,
   AST_F1))
                : kcc_eval_const_int((void *)IL(n,
   AST_F2));
    if (tag == AST_PAREN_EXPR) return
   kcc_eval_const_int((void *)IL(n, AST_F0));
    if (tag == AST_CAST_EXPR) return
   kcc_eval_const_int((void *)IL(n, AST_F1));
    if (tag == AST_SIZEOF_EXPR) {
        void *op = (void *)IL(n, AST_F0);
        if (!op) return 0;
        void *ty;
        if (IL(n, AST_F1)) ty =
   kcc_sema_resolve_type(op);
        else { ty = (void *)IL(op, AST_F3); if (!
   ty) ty = kcc_sema_expr(op); }
        if (!ty) return 0;
        return kcc_type_layout(ty);
    }
    if (tag == AST_TYPES_COMPAT) {
        void *t1 = kcc_sema_resolve_type((void *)
   IL(n, AST_F0));
        void *t2 = kcc_sema_resolve_type((void *)
   IL(n, AST_F1));
        return kcc_type_compatible(t1, t2);
    }
    if (tag == AST_IDENT_EXPR) {
        char *name = (char *)IL(n, AST_F0);
        if (!name) return 0;
        void *sym = kcc_symtab_lookup(name,
   kcc_strlen(name));
        if (!sym) return 0;
        if (IL(sym, SYM_KIND) != SYM_ENUM_CONST)
   return 0;
        return IL(sym, SYM_VALUE);
    }
    if (tag == AST_UNARY_EXPR) {
        unsigned long op = IL(n, AST_F0);
```

```
    if (op == TOK_AMP) return kcc_eval_addr((
void *)IL(n, AST_F1));  /* offsetof */
    long v = (long)kcc_eval_const_int((void *)
IL(n, AST_F1));
    if (op == TOK_MINUS) return (unsigned long
)(-v);
    if (op == TOK_TILDE) return (unsigned long
)(~v);
    if (op == TOK_BANG) return v == 0 ? 1 : 0;
    return (unsigned long)v;
 }
 if (tag == AST_BINARY_EXPR) {
    unsigned long op = IL(n, AST_F0);
    void *o1 = (void *)IL(n, AST_F1), *o2 = (
void *)IL(n, AST_F2);
    unsigned long ua = kcc_eval_const_int(o1);
    unsigned long ub = kcc_eval_const_int(o2);
    long a = (long)ua, b = (long)ub;
    /* Division, modulo, the right shift, and
the relational comparisons
       differ by signedness.  Fold them
unsigned when the usual arithmetic
       conversions make the operation unsigned
 (kcc_ir_cmp_unsigned reads the
       operands' cached types), or a 64-bit
unsigned constant such as
       `(~(size_t)0) / 4' folds to a wrong
signed value (-1/4 = 0) and its
       comparison then picks the wrong branch
---exactly what broke Lua's
       `luaM_limitN' on the x86-64 self-host,
whose `opcodes' limit came out
       -1.  When the operand types are
unavailable kcc_ir_cmp_unsigned returns
       0 and the historical signed fold stands
, so no signed case regresses. */
    unsigned long uns = kcc_ir_cmp_unsigned(o1
, o2);
    if (op == TOK_PLUS) return (unsigned long)
(a + b);
    if (op == TOK_MINUS) return (unsigned long
)(a - b);
    if (op == TOK_STAR) return (unsigned long)
(a * b);
    if (op == TOK_SLASH) return b ? (uns ? ua
/ ub : (unsigned long)(a / b)) : 0;
    if (op == TOK_PERCENT) return b ? (uns ?
ua % ub : (unsigned long)(a % b)) : 0;
    /* Left-shift in the unsigned domain: a
left shift of a negative signed
       value is undefined behaviour (\stdref
{6.5.7}), and computing on the
       two's-complement bit pattern yields the
 value every target actually
       wants.  The right shift follows the
operand signedness---logical for an
       unsigned value, arithmetic for a signed
 `x >> 1' on a negative bound. */
    if (op == TOK_SHL) return (unsigned long)a
 << (b & 63);
    if (op == TOK_SHR) return uns ? (ua >> (ub
 & 63)) : (unsigned long)(a >> b);
    if (op == TOK_AMP) return (unsigned long)(
a & b);
    if (op == TOK_PIPE) return (unsigned long)
(a | b);
    if (op == TOK_CARET) return (unsigned long
)(a ^ b);
    /* Relational operators yield 0/1, signed
or unsigned per the operands'
       common type: the signed path is the
common `int' case (`if (x < 0)', a
       GNU array-range bound); the unsigned
path is what a `size_t' bound
       needs and what folds Lua's limit
correctly. */
    if (op == TOK_LT) return (uns ? ua <  ub :
 a <  b) ? 1 : 0;
    if (op == TOK_GT) return (uns ? ua >  ub :
 a >  b) ? 1 : 0;
    if (op == TOK_LE) return (uns ? ua <= ub :
 a <= b) ? 1 : 0;
    if (op == TOK_GE) return (uns ? ua >= ub :
 a >= b) ? 1 : 0;
    if (op == TOK_EQ) return a == b ? 1 : 0;
    if (op == TOK_NE) return a != b ? 1 : 0;
    if (op == TOK_AND) return (a && b) ? 1 :
0;
    if (op == TOK_OR)  return (a || b) ? 1 :
0;
    return 0;
 }
 return 0;
}

/* The integer value of the address of an lvalue
   whose base is a compile-time
   integer (an `offsetof`-style null-based address
    constant).  For `p->m` the
   base pointer's value is added to the member's
   byte offset; for `s.m` the
   address of `s` is; `&*p` collapses to the value
    of `p`. */
unsigned long kcc_eval_addr(void *n) {
 if (!n) return 0;
 unsigned long tag = IL(n, AST_TAG);
 if (tag == AST_PAREN_EXPR) return
kcc_eval_addr((void *)IL(n, AST_F0));
 if (tag == AST_MEMBER) {
    void *obj = (void *)IL(n, AST_F0);
    unsigned long base = IL(n, AST_F2) ?
kcc_eval_const_int(obj)  /* p->m */
                                    :
kcc_eval_addr(obj);       /* s.m  */
    void *st = (void *)IL(obj, AST_F3);
    if (!st) st = kcc_sema_expr(obj);  /* base
 may be untyped during array-bound eval */
    if (st && IL(st, TY_KIND) == TY_PTR) st =
(void *)IL(st, TY_F0);
    unsigned long off = 0;
    if (st && (IL(st, TY_KIND) == TY_STRUCT ||
 IL(st, TY_KIND) == TY_UNION)) {
       kcc_type_layout(st);
       struct kcc_member_res mr =
kcc_type_find_member(st, (const char *)IL(n,
AST_F1));
       if (mr.field) off = mr.off;
    }
    return base + off;
 }
 if (tag == AST_UNARY_EXPR && IL(n, AST_F0) ==
TOK_STAR)
    return kcc_eval_const_int((void *)IL(n,
AST_F1));
 return 0;
}

/* Whether the address of an lvalue reduces to an
   integer constant expression:
   its ultimate base must be a constant pointer,
   as in `&(((T*)0)->m)`. */
unsigned long kcc_addr_is_iconst(void *n) {
 if (!n) return 0;
```

```
    unsigned long tag = IL(n, AST_TAG);
    if (tag == AST_PAREN_EXPR) return
   kcc_addr_is_iconst((void *)IL(n, AST_F0));
    if (tag == AST_MEMBER)
        return IL(n, AST_F2) ? kcc_expr_is_ice((
   void *)IL(n, AST_F0))    /* p->m */
                             : kcc_addr_is_iconst
   ((void *)IL(n, AST_F0)); /* s.m  */
    if (tag == AST_UNARY_EXPR && IL(n, AST_F0) ==
   TOK_STAR)
        return kcc_expr_is_ice((void *)IL(n,
   AST_F1));
    return 0;
}

unsigned long kcc_expr_is_ice(void *n) {
    if (!n) return 1;
    unsigned long tag = IL(n, AST_TAG);
    if (tag == AST_INT_LIT || tag == AST_CHAR_LIT
   ||
        tag == AST_SIZEOF_EXPR || tag ==
   AST_ALIGNOF_EXPR ||
        tag == AST_TYPES_COMPAT) return 1;   /*
   __builtin_types_compatible_p → 0/1 */
    if (tag == AST_PAREN_EXPR) return
   kcc_expr_is_ice((void *)IL(n, AST_F0));
    if (tag == AST_CAST_EXPR) return
   kcc_expr_is_ice((void *)IL(n, AST_F1));
    if (tag == AST_UNARY_EXPR) {
        unsigned long op = IL(n, AST_F0);
        if (op == TOK_AMP) return
   kcc_addr_is_iconst((void *)IL(n, AST_F1));  /*
   offsetof */
        if (op == TOK_STAR) return 0;
                             /* deref */
        return kcc_expr_is_ice((void *)IL(n,
   AST_F1));
    }
    if (tag == AST_BINARY_EXPR) {
        if (!kcc_expr_is_ice((void *)IL(n, AST_F1)
   )) return 0;
        return kcc_expr_is_ice((void *)IL(n,
   AST_F2));
    }
    if (tag == AST_COND_EXPR) {
        void *cond = (void *)IL(n, AST_F0);
        if (!kcc_expr_is_ice(cond)) return 0;
        /* A constant condition selects one branch
   ; only that branch need be a
           constant expression (\stdref{6.6}), so
   the kernel's
           `__builtin_constant_p(n) ? <const> :
   __ilog2_u32(n)` is constant even
           though its else branch is a function
   call. */
        return kcc_eval_const_int(cond) ?
   kcc_expr_is_ice((void *)IL(n, AST_F1))
                                        :
   kcc_expr_is_ice((void *)IL(n, AST_F2));
    }
    if (tag == AST_IDENT_EXPR) {
        char *name = (char *)IL(n, AST_F0);
        if (!name) return 0;
        void *sym = kcc_symtab_lookup(name,
   kcc_strlen(name));
        if (!sym) return 0;
        if (IL(sym, SYM_KIND) == SYM_ENUM_CONST)
   return 1;
        return 0;
    }
    /* __builtin_constant_p(x) is itself a
   constant (0 or 1), so it is an
       integer constant expression regardless of x
   .  The kernel gates whole
       code paths on it (`__builtin_constant_p(nr)
    ? ... : ...`). */
    if (tag == AST_CALL_EXPR && kcc_call_is_bcp(n)
   ) return 1;
    /* A foldable bit-counting builtin is an
   integer constant expression when its
       argument is (the kernel's ilog2/
   BITS_TO_LONGS array bounds). */
    if (tag == AST_CALL_EXPR &&
   kcc_call_foldable_builtin(n))
        return kcc_expr_is_ice((void *)IL(n,
   AST_F1));
    /* __builtin_choose_expr(cond, a, b): an ICE
   when the (ICE) condition selects
       an operand that is itself an ICE; the
   unchosen operand is not examined. */
    if (tag == AST_CALL_EXPR &&
   kcc_call_is_choose_expr(n)) {
        void *a0 = (void *)IL(n, AST_F1);
        void *a1 = a0 ? (void *)IL(a0, AST_F4) :
   0;
        void *a2 = a1 ? (void *)IL(a1, AST_F4) :
   0;
        if (!a0 || !kcc_expr_is_ice(a0)) return 0;
        void *pick = kcc_eval_const_int(a0) ? a1 :
    a2;
        return pick ? kcc_expr_is_ice(pick) : 0;
    }
    return 0;
}

/* True when n is a call to __builtin_constant_p.
   */
unsigned long kcc_call_is_bcp(void *n) {
    void *callee = (void *)IL(n, AST_F0);
    if (!callee || IL(callee, AST_TAG) !=
   AST_IDENT_EXPR) return 0;
    char *cn = (char *)IL(callee, AST_F0);
    return cn && kcc_strcmp(cn, "
   __builtin_constant_p") == 0;
}

/* __builtin_choose_expr(cond, a, b) is a compile-
   time selection: the result is
   the chosen argument and the unchosen one is not
    evaluated.  Sema rewrites the
   call node into its pick, but a constant array
   size is resolved before that
   runs, so the constant-expression predicates
   recognise it directly too---the
   kernel's `max_t(...)' (behind e.g.
   TCPUDP_MIB_MAX, the size of proc.c's
   snmp buffer) expands to one, and without this
   the fixed array is wrongly
   rejected as a VLA. */
unsigned long kcc_call_is_choose_expr(void *n) {
    void *callee = (void *)IL(n, AST_F0);
    if (!callee || IL(callee, AST_TAG) !=
   AST_IDENT_EXPR) return 0;
    char *cn = (char *)IL(callee, AST_F0);
    return cn && kcc_strcmp(cn, "
   __builtin_choose_expr") == 0;
}

/* Allocate a block for every case/default label
   reachable in a switch body. */
void kcc_ir_sw_scan(void *stmt) {
    if (!stmt) return;
```

```
    unsigned long tag = IL(stmt, AST_TAG);
    if (tag == AST_CASE_STMT) {
        IL(stmt, AST_F2) = (unsigned long)
  kcc_ir_alloc_bb();
        kcc_ir_sw_scan((void *)IL(stmt, AST_F1));
    } else if (tag == AST_DEFAULT_STMT) {
        void *bb = kcc_ir_alloc_bb();
        IL(stmt, AST_F1) = (unsigned long)bb;
        kcc_sw_default_bb = bb;
        kcc_ir_sw_scan((void *)IL(stmt, AST_F0));
    } else if (tag == AST_COMPOUND) {
        void *it = (void *)IL(stmt, AST_F0);
        while (it) { kcc_ir_sw_scan(it); it = (
  void *)IL(it, AST_F4); }
    } else if (tag == AST_IF_STMT) {
        kcc_ir_sw_scan((void *)IL(stmt, AST_F1));
        kcc_ir_sw_scan((void *)IL(stmt, AST_F2));
    } else if (tag == AST_WHILE_STMT || tag ==
  AST_LABEL_STMT) {
        kcc_ir_sw_scan((void *)IL(stmt, AST_F1));
    } else if (tag == AST_DO_STMT) {
        kcc_ir_sw_scan((void *)IL(stmt, AST_F0));
    } else if (tag == AST_FOR_STMT) {
        kcc_ir_sw_scan((void *)IL(stmt, AST_F3));
    }
}

/* Emit the dispatch comparison for every case, in
    source order. */
void kcc_ir_sw_disp(void *stmt) {
    if (!stmt) return;
    unsigned long tag = IL(stmt, AST_TAG);
    if (tag == AST_CASE_STMT && IL(stmt, AST_F3))
  {
        /* GNU case range `case LO ... HI:` - take
    the case body iff the switch
           value is in [LO, HI] (signed): dispatch
    as cond>=LO then cond<=HI,
           with a middle block between the two
  comparisons. */
        unsigned long lo = kcc_eval_const_int((
  void *)IL(stmt, AST_F0));
        unsigned long hi = kcc_eval_const_int((
  void *)IL(stmt, AST_F3));
        void *lc = kcc_ir_alloc_instr(IR_CONST);
        IL(lc, IR_TYPE) = (unsigned long)
  kcc_type_int_s;
        IL(lc, IR_SRC1) = lo;
        unsigned long lcv = kcc_ir_next_vreg();
        IL(lc, IR_DEST) = lcv;
        kcc_ir_emit(lc);
        void *ge = kcc_ir_alloc_instr(IR_CMP_SGE);
        IL(ge, IR_TYPE) = (unsigned long)
  kcc_type_int_s;
        IL(ge, IR_SRC1) = kcc_sw_cond_vreg;
        IL(ge, IR_SRC2) = lcv;
        unsigned long gev = kcc_ir_next_vreg();
        IL(ge, IR_DEST) = gev;
        kcc_ir_emit(ge);
        void *mid_bb = kcc_ir_alloc_bb();
        void *next_bb = kcc_ir_alloc_bb();
        void *bc1 = kcc_ir_alloc_instr(IR_BR_COND)
  ;
        IL(bc1, IR_DEST) = gev;
        IL(bc1, IR_SRC1) = IL(mid_bb, BB_ID);
        IL(bc1, IR_SRC2) = IL(next_bb, BB_ID);
        kcc_ir_emit(bc1);
        IL(kcc_ir_cur_bb, BB_FLAGS) = IL(
  kcc_ir_cur_bb, BB_FLAGS) | BB_FLAG_TERMINATED;
        kcc_ir_set_bb(mid_bb);
        void *hc = kcc_ir_alloc_instr(IR_CONST);
        IL(hc, IR_TYPE) = (unsigned long)
  kcc_type_int_s;
        IL(hc, IR_SRC1) = hi;
        unsigned long hcv = kcc_ir_next_vreg();
        IL(hc, IR_DEST) = hcv;
        kcc_ir_emit(hc);
        void *le = kcc_ir_alloc_instr(IR_CMP_SLE);
        IL(le, IR_TYPE) = (unsigned long)
  kcc_type_int_s;
        IL(le, IR_SRC1) = kcc_sw_cond_vreg;
        IL(le, IR_SRC2) = hcv;
        unsigned long lev = kcc_ir_next_vreg();
        IL(le, IR_DEST) = lev;
        kcc_ir_emit(le);
        void *bc2 = kcc_ir_alloc_instr(IR_BR_COND)
  ;
        IL(bc2, IR_DEST) = lev;
        IL(bc2, IR_SRC1) = IL((void *)IL(stmt,
  AST_F2), BB_ID);  /* case bb */
        IL(bc2, IR_SRC2) = IL(next_bb, BB_ID);
        kcc_ir_emit(bc2);
        IL(kcc_ir_cur_bb, BB_FLAGS) = IL(
  kcc_ir_cur_bb, BB_FLAGS) | BB_FLAG_TERMINATED;
        kcc_ir_set_bb(next_bb);
        kcc_ir_sw_disp((void *)IL(stmt, AST_F1));
    } else if (tag == AST_CASE_STMT) {
        unsigned long cv = kcc_eval_const_int((
  void *)IL(stmt, AST_F0));
        void *c = kcc_ir_alloc_instr(IR_CONST);
        IL(c, IR_TYPE) = (unsigned long)
  kcc_type_int_s;
        IL(c, IR_SRC1) = cv;
        unsigned long cvreg = kcc_ir_next_vreg();
        IL(c, IR_DEST) = cvreg;
        kcc_ir_emit(c);
        void *cmp = kcc_ir_alloc_instr(IR_CMP_EQ);
        IL(cmp, IR_TYPE) = (unsigned long)
  kcc_type_int_s;
        IL(cmp, IR_SRC1) = kcc_sw_cond_vreg;
        IL(cmp, IR_SRC2) = cvreg;
        unsigned long cmpv = kcc_ir_next_vreg();
        IL(cmp, IR_DEST) = cmpv;
        kcc_ir_emit(cmp);
        void *next_bb = kcc_ir_alloc_bb();
        void *bc = kcc_ir_alloc_instr(IR_BR_COND);
        IL(bc, IR_DEST) = cmpv;
        IL(bc, IR_SRC1) = IL((void *)IL(stmt,
  AST_F2), BB_ID);   /* case bb */
        IL(bc, IR_SRC2) = IL(next_bb, BB_ID);
        kcc_ir_emit(bc);
        IL(kcc_ir_cur_bb, BB_FLAGS) = IL(
  kcc_ir_cur_bb, BB_FLAGS) | BB_FLAG_TERMINATED;
        kcc_ir_set_bb(next_bb);
        kcc_ir_sw_disp((void *)IL(stmt, AST_F1));
    } else if (tag == AST_DEFAULT_STMT) {
        kcc_ir_sw_disp((void *)IL(stmt, AST_F0));
    } else if (tag == AST_COMPOUND) {
        void *it = (void *)IL(stmt, AST_F0);
        while (it) { kcc_ir_sw_disp(it); it = (
  void *)IL(it, AST_F4); }
    } else if (tag == AST_IF_STMT) {
        kcc_ir_sw_disp((void *)IL(stmt, AST_F1));
        kcc_ir_sw_disp((void *)IL(stmt, AST_F2));
    } else if (tag == AST_WHILE_STMT || tag ==
  AST_LABEL_STMT) {
        kcc_ir_sw_disp((void *)IL(stmt, AST_F1));
    } else if (tag == AST_DO_STMT) {
        kcc_ir_sw_disp((void *)IL(stmt, AST_F0));
    } else if (tag == AST_FOR_STMT) {
```

```
        kcc_ir_sw_disp((void *)IL(stmt, AST_F3));
    }
}

/* Deep count of case/default labels in a switch
   body, recursing exactly where
   kcc_ir_sw_scan does (and, like it, NOT into a
   nested switch, whose labels
   belong to that switch).  Used to detect labels
   buried inside an arm. */
static unsigned long kcc_sw_count_labels(void *
   stmt) {
    if (!stmt) return 0;
    unsigned long tag = IL(stmt, AST_TAG);
    if (tag == AST_CASE_STMT)
        return 1 + kcc_sw_count_labels((void *)IL(
   stmt, AST_F1));
    if (tag == AST_DEFAULT_STMT)
        return 1 + kcc_sw_count_labels((void *)IL(
   stmt, AST_F0));
    if (tag == AST_COMPOUND) {
        unsigned long n = 0;
        for (void *it = (void *)IL(stmt, AST_F0);
   it; it = (void *)IL(it, AST_F4))
            n += kcc_sw_count_labels(it);
        return n;
    }
    if (tag == AST_IF_STMT)
        return kcc_sw_count_labels((void *)IL(stmt
   , AST_F1))
             + kcc_sw_count_labels((void *)IL(stmt
   , AST_F2));
    if (tag == AST_WHILE_STMT || tag ==
   AST_LABEL_STMT)
        return kcc_sw_count_labels((void *)IL(stmt
   , AST_F1));
    if (tag == AST_DO_STMT)
        return kcc_sw_count_labels((void *)IL(stmt
   , AST_F0));
    if (tag == AST_FOR_STMT)
        return kcc_sw_count_labels((void *)IL(stmt
   , AST_F3));
    return 0;
}

/* A switch body is foldable when it is a compound
    whose case/default labels are
   all top-level label chains (so fall-through is
   a walk of the sibling list),
   whose first item is a label (no statement
   precedes the first arm), and which
   declares no variable directly in the switch
   scope (only inside an arm's own
   braces)---the last guard means skipping an
   earlier arm can never omit a
   declaration a built arm relies on. */
static int kcc_sw_foldable_body(void *body) {
    if (!body || IL(body, AST_TAG) != AST_COMPOUND
   ) return 0;
    void *first = (void *)IL(body, AST_F0);
    if (!first) return 0;
    unsigned long ft = IL(first, AST_TAG);
    if (ft != AST_CASE_STMT && ft !=
   AST_DEFAULT_STMT) return 0;
    unsigned long chain_total = 0;
    for (void *it = first; it; it = (void *)IL(it,
    AST_F4)) {
        void *node = it;
        while (node) {
            unsigned long nt = IL(node, AST_TAG);
            if (nt == AST_CASE_STMT) { chain_total
   ++; node = (void *)IL(node, AST_F1); }
            else if (nt == AST_DEFAULT_STMT) {
   chain_total++; node = (void *)IL(node, AST_F0);
    }
            else break;
        }
        if (node && IL(node, AST_TAG) ==
   AST_VAR_DECL) return 0;  /* bare decl in switch
    scope */
    }
    return chain_total == kcc_sw_count_labels(body
   );  /* else a label is buried in an arm */
}

/* Fold a switch whose control value is a constant
   : build only the arm that
   value selects (or default, or nothing), as
   straight-line code with the
   ordinary case/default label handling---no
   dispatch, so the unselected arms
   (and their differently-sized inline asm) are
   never emitted.  Returns 1 when
   it folded, 0 to fall back to the general two-
   scan lowering. */
static int kcc_ir_try_fold_switch(void *s) {
    void *cond = (void *)IL(s, AST_F0);
    if (!kcc_expr_is_ice(cond)) return 0;
    void *body = (void *)IL(s, AST_F1);
    if (!kcc_sw_foldable_body(body)) return 0;
    long v = (long)kcc_eval_const_int(cond);
    void *start = 0;
    void *deflt = 0;
    for (void *it = (void *)IL(body, AST_F0); it
   && !start; it = (void *)IL(it, AST_F4)) {
        void *node = it;
        while (node) {
            unsigned long nt = IL(node, AST_TAG);
            if (nt == AST_CASE_STMT) {
                long lo = (long)kcc_eval_const_int
   ((void *)IL(node, AST_F0));
                if (IL(node, AST_F3)) {
                    long hi = (long)
   kcc_eval_const_int((void *)IL(node, AST_F3));
                    if (v >= lo && v <= hi) {
   start = it; break; }
                } else if (v == lo) { start = it;
   break; }
                node = (void *)IL(node, AST_F1);
            } else if (nt == AST_DEFAULT_STMT) {
                if (!deflt) deflt = it;
                node = (void *)IL(node, AST_F0);
            } else break;
        }
    }
    if (!start) start = deflt;                  /*
   no case matched: default, or nothing */
    void *exit_bb = kcc_ir_alloc_bb();
    unsigned long cont = 0;
    if (kcc_loop_depth) cont = kcc_loop_stack[(
   kcc_loop_depth - 1) * 2 + 1];
    kcc_loop_push(IL(exit_bb, BB_ID), cont);    /*
    break targets the switch exit */
    for (void *it = start; it; it = (void *)IL(it,
    AST_F4)) {
        kcc_ir_build_stmt(it);                  /*
    label f2/f1 are 0: builds transparently */
        if (IL(kcc_ir_cur_bb, BB_FLAGS) &
   BB_FLAG_TERMINATED) break;  /* a break/return
   ended the arm */
    }
    kcc_loop_pop();
```

```
    kcc_ir_branch_to(exit_bb);
    kcc_ir_set_bb(exit_bb);
    return 1;
}

void kcc_ir_build_switch(void *s) {
    if (kcc_ir_try_fold_switch(s)) return;
    kcc_sw_cond_vreg = kcc_ir_build_expr((void *)
   IL(s, AST_F0));
    void *exit_bb = kcc_ir_alloc_bb();
    kcc_sw_default_bb = 0;
    kcc_ir_sw_scan((void *)IL(s, AST_F1));
    void *default_bb = kcc_sw_default_bb;
    kcc_ir_sw_disp((void *)IL(s, AST_F1));
    kcc_ir_branch_to(default_bb ? default_bb :
   exit_bb);
    unsigned long cont = 0;
    if (kcc_loop_depth) cont = kcc_loop_stack[(
   kcc_loop_depth - 1) * 2 + 1];
    kcc_loop_push(IL(exit_bb, BB_ID), cont);
    kcc_ir_build_stmt((void *)IL(s, AST_F1));
    kcc_loop_pop();
    kcc_ir_branch_to(exit_bb);
    kcc_ir_set_bb(exit_bb);
}

void kcc_loop_push(unsigned long break_id,
   unsigned long continue_id) {
    unsigned long d = kcc_loop_depth;
    kcc_loop_stack[d * 2] = break_id;
    kcc_loop_stack[d * 2 + 1] = continue_id;
    kcc_loop_depth = d + 1;
}

void kcc_loop_pop(void) {
    kcc_loop_depth = kcc_loop_depth - 1;
}

void kcc_ir_build_break(void *s) {
    (void)s;
    unsigned long tgt = kcc_loop_stack[(
   kcc_loop_depth - 1) * 2];
    void *br = kcc_ir_alloc_instr(IR_BR);
    IL(br, IR_SRC1) = tgt;
    kcc_ir_emit(br);
    IL(kcc_ir_cur_bb, BB_FLAGS) = IL(kcc_ir_cur_bb
   , BB_FLAGS) | BB_FLAG_TERMINATED;
}

void kcc_ir_build_continue(void *s) {
    (void)s;
    unsigned long tgt = kcc_loop_stack[(
   kcc_loop_depth - 1) * 2 + 1];
    void *br = kcc_ir_alloc_instr(IR_BR);
    IL(br, IR_SRC1) = tgt;
    kcc_ir_emit(br);
    IL(kcc_ir_cur_bb, BB_FLAGS) = IL(kcc_ir_cur_bb
   , BB_FLAGS) | BB_FLAG_TERMINATED;
}
```

□

879 · used in 902a

75.9. Assignment, Calls, Casts

Assignment stores through an lvalue (variable, member, subscript, dereference), using the shared `assign_through` for address-based targets and a whole-object copy for aggregates. `build_call` classifies each argument for AAPCS64, reserves a hidden result buffer for an aggregate return, and dispatches direct or indirect. The cast builder delegates FP conversions to `convert` and re-canonicalises integer width and signedness.

Before any of that, `build_call` intercepts the GNU builtins the kernel relies on. `__builtin_expect` is a branch hint that reduces to its value; `__builtin_constant_p` is a compile-time predicate; `__builtin_unreachable` annotates a dead point and—since kcc does not exploit undefined behaviour—collapses to a harmless dummy value. The `__builtin_bswap16/32/64` family reverses byte order through a portable IR gather (`kcc_ir_bswap`), so no new opcode or per-target support is needed and the low n bytes carry the answer. The bit-scan family—`__builtin_clz`, `ctz`, `popcount`, and `ffs` (each with 32-bit and 64-bit `l`/`ll` widths)— also lowers to portable bit-twiddling rather than a per-target intrinsic, so it stays freestanding-correct with no new opcode. Population count is the classic SWAR fold; trailing zeros is the population count of the mask below the lowest set bit ($x \,\&\, -x$, minus one); leading zeros smears the top set bit downward and subtracts its population count from the width; and `ffs` is one past the trailing-zero count, multiplied by a nonzero predicate so a zero input yields zero without a branch. The remaining names have an exact library counterpart—`__builtin_trap` to `abort` and `__builtin_abs/labs/llabs` to their unprefixed forms—which is correct while the milestone links against a C library.

Variadic access has two lowerings; `build_call` reaches them by name, accepting both the short `va_start`/`va_end` operators and their GNU `__builtin_va_start`/`__builtin_va_end` spellings (a freestanding `<stdarg.h>` expands the `va_*` macros to the `__builtin_` forms). On ARM64 a `va_list` is a bare pointer that walks the homogeneous save area, so `va_start` records the frame base and `va_arg` bumps the cursor by eight. On x86-64 the System V ABI splits arguments across the integer and vector banks, so `va_start` materialises the four-field `va_list` struct (`gp_offset`, `fp_offset`, overflow pointer, register-save pointer) and `va_arg` selects register-versus-overflow *branchlessly* (`addr = overflow + c*(reg_addr - overflow)`, where `c` is the in-range predicate) and advances the matching counter. Because the requested type governs which bank to read, `va_arg` resolves the unresolved type-name node to a `Type` first, so a `double` takes the vector-bank path rather than being misread from the integer save area.

`va_copy(dst, src)` must hand the destination an *independent* snapshot so that draining one list leaves the other untouched. The two targets diverge here because they mutate different memory. On ARM64 the `va_list` object *is* the cursor—`va_arg` writes the bumped pointer back into the variable, never into what it points at—so copying the eight-byte cursor value already yields two lists that advance separately. On x86-64 the object is a pointer to the four-field save-area struct that `va_arg` rewrites in place; duplicating just the pointer would alias the counters. So the x86-64 path allocates a fresh four-slot struct, copies the four fields across, and stores the new struct's address into the destination—the kernel's `va_copy` in its `vsnprintf` bounds pass then walks its private copy.

⟨ *Selfhost: ir assigncall* 886 ⟩ ≡

```
unsigned long kcc_ir_build_assign(void *e) {
    /* parentheses transparent on the target */
    void *lhs = (void *)IL(e, AST_F1);
    while (lhs && IL(lhs, AST_TAG) ==
  AST_PAREN_EXPR) {
        lhs = (void *)IL(lhs, AST_F0);
        IL(e, AST_F1) = (unsigned long)lhs;
    }
    void *ltype = (void *)IL(e, AST_F3);
    unsigned long op = IL(e, AST_F0);

    /* aggregate simple assignment → whole-object
  copy */
    if (op == TOK_ASSIGN && lhs) {
        void *lt = (void *)IL(lhs, AST_F3);
        if (lt && (IL(lt, TY_KIND) == TY_STRUCT ||
    IL(lt, TY_KIND) == TY_UNION)) {
            void *aggty = lt;
            /* destination address */
            unsigned long dst;
            unsigned long dtag = IL(lhs, AST_TAG);
            if (dtag == AST_MEMBER) dst =
  kcc_ir_build_member_addr(lhs);
            else if (dtag == AST_SUBSCRIPT) dst =
  kcc_ir_build_subscript_addr(lhs);
            else if (dtag == AST_UNARY_EXPR) {
                if (IL(lhs, AST_F0) != TOK_STAR)
  return 0;
                dst = kcc_ir_build_expr((void *)IL
  (lhs, AST_F1));
            } else if (dtag == AST_IDENT_EXPR) {
                char *name = (char *)IL(lhs,
  AST_F0);
                unsigned long v = (unsigned long)
  kcc_ht_get(kcc_ir_locals, name, kcc_strlen(name
  ));
                if (v) dst = (v & ~0
  x8000000000000000UL) - 1;
                else {
                    void *g = kcc_ir_alloc_instr(
  IR_GLOBAL_REF);
                    IL(g, IR_SRC1) = (unsigned
  long)name;
                    IL(g, IR_TYPE) = (unsigned
  long)kcc_type_int_s;
                    dst = kcc_ir_next_vreg();
                    IL(g, IR_DEST) = dst;
                    kcc_ir_emit(g);
                }
            } else return 0;
            /* source address (peel parens,
  evaluate comma LHS) */
            void *src_node = (void *)IL(e, AST_F2)
  ;
            if (!src_node) return 0;
            while (1) {
                unsigned long st = IL(src_node,
  AST_TAG);
                if (st == AST_PAREN_EXPR) {
  src_node = (void *)IL(src_node, AST_F0); if (!
  src_node) return 0; continue; }
                if (st == AST_COMMA_EXPR) {
                    kcc_ir_build_expr((void *)IL(
  src_node, AST_F0));
                    src_node = (void *)IL(src_node
  , AST_F1);
                    if (!src_node) return 0;
                    continue;
                }
                break;
            }
            unsigned long src;
            unsigned long st = IL(src_node,
  AST_TAG);
            if (st == AST_ASSIGN_EXPR) src =
  kcc_ir_build_assign(src_node);
            else if (st == AST_CALL_EXPR) src =
  kcc_ir_build_call(src_node);
            else if (st == AST_MEMBER) src =
  kcc_ir_build_member_addr(src_node);
            else if (st == AST_SUBSCRIPT) src =
  kcc_ir_build_subscript_addr(src_node);
            else if (st == AST_VA_ARG) src =
  kcc_ir_build_va_arg(src_node);
            else if (st == AST_UNARY_EXPR) {
                if (IL(src_node, AST_F0) !=
  TOK_STAR) return 0;
                src = kcc_ir_build_expr((void *)IL
  (src_node, AST_F1));
            } else if (st == AST_IDENT_EXPR) {
                char *name = (char *)IL(src_node,
  AST_F0);
                unsigned long v = (unsigned long)
  kcc_ht_get(kcc_ir_locals, name, kcc_strlen(name
  ));
                if (v) src = (v & ~0
  x8000000000000000UL) - 1;
                else {
                    void *g = kcc_ir_alloc_instr(
  IR_GLOBAL_REF);
                    IL(g, IR_SRC1) = (unsigned
  long)name;
                    IL(g, IR_TYPE) = (unsigned
  long)kcc_type_int_s;
                    src = kcc_ir_next_vreg();
                    IL(g, IR_DEST) = src;
                    kcc_ir_emit(g);
                }
            } else return 0;
            kcc_ir_emit_struct_copy(src, dst,
  kcc_type_layout(aggty));
            return dst;
        }
    }

    /* scalar RHS */
    void *rhs_node = (void *)IL(e, AST_F2);
    void *rtype = (void *)IL(rhs_node, AST_F3);
    unsigned long rhs = kcc_ir_build_expr(rhs_node
  );
    if (op == TOK_ASSIGN) rhs = kcc_ir_convert(rhs
  , rtype, ltype);

    if (!lhs) return rhs;
    unsigned long tag = IL(lhs, AST_TAG);
    if (tag == AST_SUBSCRIPT) {
        unsigned long addr =
  kcc_ir_build_subscript_addr(lhs);
        return kcc_ir_assign_through(addr, rhs, op
  , 0, (void *)IL(lhs, AST_F3), rtype);
    }
    if (tag == AST_MEMBER) {
        unsigned long addr =
  kcc_ir_build_member_addr(lhs);
        void *field = kcc_ir_member_field;
        return kcc_ir_assign_through(addr, rhs, op
  , field, (void *)IL(lhs, AST_F3), rtype);
    }
```

```
    if (tag == AST_UNARY_EXPR) {
        if (IL(lhs, AST_F0) != TOK_STAR) return
   rhs;
        void *pty = (void *)IL(lhs, AST_F3);
       /* type of *expr */
        unsigned long addr = kcc_ir_build_expr((
   void *)IL(lhs, AST_F1));
        return kcc_ir_assign_through(addr, rhs, op
   , 0, pty, rtype);
    }
    if (tag != AST_IDENT_EXPR) return rhs;

    char *name = (char *)IL(lhs, AST_F0);
    if (!name) return rhs;
    kcc_ir_local_const_kill(name);   /* reassigned
   : value no longer constant */
    unsigned long v = (unsigned long)kcc_ht_get(
   kcc_ir_locals, name, kcc_strlen(name));
    unsigned long addr;
    if (v) {
        addr = (v & ~0x8000000000000000UL) - 1;
    } else {
        char *symname = name;
        if (kcc_ir_statics) {
            void *mangled = kcc_ht_get(
   kcc_ir_statics, name, kcc_strlen(name));
            if (mangled) symname = (char *)mangled
   ;
        }
        void *g = kcc_ir_alloc_instr(IR_GLOBAL_REF
   );
        IL(g, IR_SRC1) = (unsigned long)symname;
        IL(g, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        addr = kcc_ir_next_vreg();
        IL(g, IR_DEST) = addr;
        kcc_ir_emit(g);
    }
    unsigned long value = rhs;
    if (op != TOK_ASSIGN) {
        void *ld = kcc_ir_alloc_instr(IR_LOAD);
        IL(ld, IR_TYPE) = (unsigned long)ltype;
        IL(ld, IR_SRC1) = addr;
        IL(ld, IR_SRC2) = kcc_ir_load_width(ltype)
   ;
        unsigned long old = kcc_ir_next_vreg();
        IL(ld, IR_DEST) = old;
        kcc_ir_emit(ld);
        unsigned long bop = kcc_ir_tok_to_binop(op
   );
        value = kcc_ir_compound_apply(bop, old,
   ltype, rhs, rtype);
    }
    void *st = kcc_ir_alloc_instr(IR_STORE);
    IL(st, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    IL(st, IR_SRC1) = value;
    IL(st, IR_SRC2) = addr;
    IL(st, IR_DEST) = kcc_ir_store_width(ltype);
    kcc_ir_emit(st);
    return value;
}

/* __builtin_bswapN reverses the byte order of an
   n-byte value.  It lowers to a
   target-independent gather: byte i of the input
   (isolated with a shift and an
   AND) is deposited at position n-1-i, and the n
   terms are OR-ed together.  The
   result occupies the low n bytes with zero above
   , so a narrower result type
   truncates correctly without extra work. */
static unsigned long kcc_ir_bswap(unsigned long
   val, unsigned long nbytes) {
    unsigned long result = 0;
    unsigned long have = 0;
    for (unsigned long i = 0; i < nbytes; i++) {
        unsigned long term = val;
        if (i) {
            unsigned long sh = kcc_ir_emit_const(i
    * 8);
            term = kcc_ir_emit_binop(IR_LSHR, term
   , sh);
        }
        unsigned long m = kcc_ir_emit_const(0xff);
        term = kcc_ir_emit_binop(IR_AND, term, m);
        unsigned long outpos = (nbytes - 1 - i) *
   8;
        if (outpos) {
            unsigned long sh2 = kcc_ir_emit_const(
   outpos);
            term = kcc_ir_emit_binop(IR_SHL, term,
    sh2);
        }
        if (!have) { result = term; have = 1; }
        else result = kcc_ir_emit_binop(IR_OR,
   result, term);
    }
    return result;
}

/* Population count of the low nbits of a value,
   SWAR-style: pairwise sums fold
   up through 1/2/4-bit fields, then a multiply by
    0x01..01 broadcasts the byte
   sums into the top byte, which a final shift
   brings down.  Everything is a
   plain binop, so it needs no new opcode and
   lowers identically on every
   target.  The width mask keeps a 32-bit count
   from seeing stray high bits. */
static unsigned long kcc_ir_popcount(unsigned long
    val, unsigned long nbits) {
    unsigned long full = (nbits >= 64) ? 0
   xffffffffffffffffUL
                                       : ((1UL <<
   nbits) - 1);
    if (nbits < 64) {
        unsigned long fm = kcc_ir_emit_const(full)
   ;
        val = kcc_ir_emit_binop(IR_AND, val, fm);
    }
    unsigned long m1 = kcc_ir_emit_const(0
   x5555555555555555UL & full);
    unsigned long m2 = kcc_ir_emit_const(0
   x3333333333333333UL & full);
    unsigned long m4 = kcc_ir_emit_const(0
   x0f0f0f0f0f0f0f0fUL & full);
    unsigned long h01 = kcc_ir_emit_const(0
   x0101010101010101UL & full);
    unsigned long c1 = kcc_ir_emit_const(1);
    unsigned long t = kcc_ir_emit_binop(IR_LSHR,
   val, c1);
    t = kcc_ir_emit_binop(IR_AND, t, m1);
    val = kcc_ir_emit_binop(IR_SUB, val, t);
    unsigned long a = kcc_ir_emit_binop(IR_AND,
   val, m2);
    unsigned long c2 = kcc_ir_emit_const(2);
    unsigned long b = kcc_ir_emit_binop(IR_LSHR,
   val, c2);
    b = kcc_ir_emit_binop(IR_AND, b, m2);
    val = kcc_ir_emit_binop(IR_ADD, a, b);
```

```
    unsigned long c4 = kcc_ir_emit_const(4);
    unsigned long s = kcc_ir_emit_binop(IR_LSHR,
   val, c4);
    val = kcc_ir_emit_binop(IR_ADD, val, s);
    val = kcc_ir_emit_binop(IR_AND, val, m4);
    val = kcc_ir_emit_binop(IR_MUL, val, h01);
    unsigned long sh = kcc_ir_emit_const(nbits -
   8);
    val = kcc_ir_emit_binop(IR_LSHR, val, sh);
    unsigned long ff = kcc_ir_emit_const(0xff);
    return kcc_ir_emit_binop(IR_AND, val, ff);
}

/* Count trailing zeros = popcount of the mask
   below the lowest set bit.
   val & -val isolates that bit; subtracting one
   turns the zeros beneath it into
   ones.  Undefined at zero (as the builtin is),
   where it yields nbits. */
static unsigned long kcc_ir_ctz(unsigned long val,
    unsigned long nbits) {
    if (nbits < 64) {
        unsigned long fm = kcc_ir_emit_const((1UL
   << nbits) - 1);
        val = kcc_ir_emit_binop(IR_AND, val, fm);
    }
    unsigned long zero = kcc_ir_emit_const(0);
    unsigned long neg = kcc_ir_emit_binop(IR_SUB,
   zero, val);
    unsigned long low = kcc_ir_emit_binop(IR_AND,
   val, neg);
    unsigned long one = kcc_ir_emit_const(1);
    unsigned long m = kcc_ir_emit_binop(IR_SUB,
   low, one);
    return kcc_ir_popcount(m, nbits);
}

/* Count leading zeros: smear the highest set bit
   down to fill every lower bit,
   so popcount then gives one past the top bit's
   index; nbits minus that is the
   leading-zero count.  Undefined at zero, where
   it yields nbits. */
static unsigned long kcc_ir_clz(unsigned long val,
    unsigned long nbits) {
    if (nbits < 64) {
        unsigned long fm = kcc_ir_emit_const((1UL
   << nbits) - 1);
        val = kcc_ir_emit_binop(IR_AND, val, fm);
    }
    for (unsigned long s = 1; s < nbits; s <<= 1)
   {
        unsigned long sh = kcc_ir_emit_const(s);
        unsigned long r = kcc_ir_emit_binop(
   IR_LSHR, val, sh);
        val = kcc_ir_emit_binop(IR_OR, val, r);
    }
    unsigned long pc = kcc_ir_popcount(val, nbits)
   ;
    unsigned long nb = kcc_ir_emit_const(nbits);
    return kcc_ir_emit_binop(IR_SUB, nb, pc);
}

/* Find first set = one past the trailing-zero
   count, or zero for a zero input.
   Multiplying by the (nonzero?) predicate folds
   the zero case in branchlessly. */
static unsigned long kcc_ir_ffs(unsigned long val,
    unsigned long nbits) {
    if (nbits < 64) {
        unsigned long fm = kcc_ir_emit_const((1UL
   << nbits) - 1);
        val = kcc_ir_emit_binop(IR_AND, val, fm);
    }
    unsigned long t = kcc_ir_ctz(val, nbits);
    unsigned long one = kcc_ir_emit_const(1);
    unsigned long tp1 = kcc_ir_emit_binop(IR_ADD,
   t, one);
    unsigned long zero = kcc_ir_emit_const(0);
    unsigned long nz = kcc_ir_emit_binop(IR_CMP_NE
   , val, zero);
    return kcc_ir_emit_binop(IR_MUL, tp1, nz);
}

/* --- always_inline expansion ---
   An `always_inline` function is inlined at each
   call by building a clone of its
   body with the parameters replaced by the call's
    arguments.  This is what makes
   a constant the CALLER knows fold through the
   callee---the kernel's
   `static_cpu_has(FEATURE)` needs the constant
   feature bit substituted into a
   `1 << (bit & 7)` `"i"` inline-asm operand,
   which GCC only accepts once it is a
   compile-time constant.  The registry is filled
   by the parser as each
   always_inline definition closes (
   kcc_register_always_inline).  This first
   phase handles a `{ return EXPR; }` body; the
   clone declines (returns 0) for
   any construct it does not model, so an
   unhandled callee simply falls back to a
   real call and can never be miscompiled.

   Because substitution is textual, an argument
   that carries a side effect (a
   function call, `p++`, an assignment) can NOT be
    pasted in directly---it might
   be evaluated twice or, if a folded branch drops
    the parameter, not at all.
   try_inline binds each such argument to a one-
   shot temporary declared at the top
   of the expansion and substitutes the temporary
   instead; only plain identifiers
   and constants stay textual, so a constant a `
   __builtin_constant_p` guard needs
   still folds.  This is precisely what lets the
   size-dispatched bit ops
   (arch_set_bit / arch_clear_bit) inline over a `
   per_cpu` address expression
   rather than degrade to an out-of-line call
   whose asm body can never be emitted
   standalone. */
void kcc_register_always_inline(const char *name,
   void *fndecl) {
    if (!kcc_always_inline_fns)
   kcc_always_inline_fns = kcc_ht_init(64);
    kcc_ht_put(kcc_always_inline_fns, name,
   kcc_strlen(name), fndecl);
}

/* Labels are function-scoped, so every label the
   inlined body defines is renamed
   with a per-inline-instance suffix, or two
   expansions of the same callee in one
   caller would collide.  The renamed name is only
    ever an internal hash-table key
   (blocks emit as numbered .Lkccbb labels), so
   the suffix need not be a legal C
   identifier.  The return conversion targets (`
   __r'/`__end') live in these two
   globals for the duration of one body clone. */
static unsigned long kcc_inl_inst = 0;
```

```
static const char *kcc_inl_ret_name = 0;   /* the
   __r variable's name, 0 if void */
static const char *kcc_inl_end_name = 0;   /* the
   __end label's name */

static const char *kcc_inl_lbl(const char *name) {
    unsigned long nl = kcc_strlen(name);
    char *buf = (char *)kcc_arena_alloc(nl + 24);
    unsigned long i = 0;
    while (i < nl) { buf[i] = name[i]; i++; }
    buf[i++] = '$';
    unsigned long v = kcc_inl_inst;
    char tmp[20]; unsigned long tn = 0;
    if (v == 0) tmp[tn++] = '0';
    while (v) { tmp[tn++] = (char)('0' + (v % 10))
   ; v /= 10; }
    while (tn) buf[i++] = tmp[--tn];
    buf[i] = 0;
    return (const char *)kcc_strtab_intern(buf, i)
   ;
}

/* Classifies whether an argument is safe to
   substitute TEXTUALLY: it may be
   copied into the body more than once (or dropped
    by a folded branch), so a
   textual argument must be free of side effects.
    A `safe' one is kept as an
   expression, letting a constant fold through `
   __builtin_constant_p'; an unsafe
   one is instead bound once to a temporary by the
    caller (kcc_ir_try_inline). */
static int kcc_inl_arg_safe(void *a) {
    if (!a) return 1;
    unsigned long tag = IL(a, AST_TAG);
    if (tag == AST_INT_LIT || tag == AST_CHAR_LIT
   || tag == AST_FLOAT_LIT ||
        tag == AST_STR_LIT || tag ==
   AST_IDENT_EXPR)
        return 1;
    if (tag == AST_PAREN_EXPR)
        return kcc_inl_arg_safe((void *)IL(a,
   AST_F0));
    if (tag == AST_CAST_EXPR)
        return kcc_inl_arg_safe((void *)IL(a,
   AST_F1));
    if (tag == AST_UNARY_EXPR) {
        unsigned long op = IL(a, AST_F0);
        if (op == TOK_INC || op == TOK_DEC) return
    0;
        return kcc_inl_arg_safe((void *)IL(a,
   AST_F1));
    }
    if (tag == AST_BINARY_EXPR)
        return kcc_inl_arg_safe((void *)IL(a,
   AST_F1)) &&
               kcc_inl_arg_safe((void *)IL(a,
   AST_F2));
    if (tag == AST_SUBSCRIPT)
        return kcc_inl_arg_safe((void *)IL(a,
   AST_F0)) &&
               kcc_inl_arg_safe((void *)IL(a,
   AST_F1));
    if (tag == AST_MEMBER)
        return kcc_inl_arg_safe((void *)IL(a,
   AST_F0));
    if (tag == AST_COND_EXPR)
        return kcc_inl_arg_safe((void *)IL(a,
   AST_F0)) &&
               kcc_inl_arg_safe((void *)IL(a,
   AST_F1)) &&
               kcc_inl_arg_safe((void *)IL(a,
   AST_F2));
    return 0;                                  /*
    calls, assignments, comma: assume unsafe */
}

/* Shallow node copy: same tag and all five fields
   , no sibling link. */
static void *kcc_inl_dup(void *node) {
    void *d = kcc_ast_alloc(IL(node, AST_TAG));
    IL(d, AST_LINE) = IL(node, AST_LINE); IL(d,
   AST_COL) = IL(node, AST_COL);
    IL(d, AST_F0) = IL(node, AST_F0); IL(d, AST_F1
   ) = IL(node, AST_F1);
    IL(d, AST_F2) = IL(node, AST_F2); IL(d, AST_F3
   ) = IL(node, AST_F3);
    IL(d, AST_F4) = 0;
    return d;
}

static void *kcc_inl_clone(void *node, void *
   params, void *args);

/* Clone an f4-linked argument list; (void *)-1
   signals an unclonable element. */
static void *kcc_inl_clone_list(void *head, void *
   params, void *args) {
    void *first = 0, *last = 0;
    for (void *el = head; el; el = (void *)IL(el,
   AST_F4)) {
        void *c = kcc_inl_clone(el, params, args);
        if (!c) return (void *)-1;
        if (!first) { first = c; last = c; }
        else { IL(last, AST_F4) = (unsigned long)c
   ; last = c; }
    }
    return first;
}

static void *kcc_inl_clone(void *node, void *
   params, void *args) {
    if (!node) return 0;
    unsigned long tag = IL(node, AST_TAG);
    if (tag == AST_INT_LIT || tag == AST_CHAR_LIT
   || tag == AST_FLOAT_LIT ||
        tag == AST_STR_LIT)
        return kcc_inl_dup(node);
    if (tag == AST_IDENT_EXPR) {
        const char *nm = (const char *)IL(node,
   AST_F0);
        void *p = params, *a = args;
        while (p && a) {
            const char *pn = (const char *)IL(p,
   AST_F1);
            if (pn && kcc_strcmp(pn, nm) == 0) {
                void *ac = kcc_inl_clone(a, 0, 0);
   /* clone the argument */
                if (!ac) return 0;
                /* Bind through the parameter's
   type: an argument of a narrower
                   type would otherwise give the
   operand the wrong width (a `long'
                   bit number materialised as %r9d
    clashes with a `btrq').  A cast
                   of a constant is still a
   constant, so this preserves folding. */
                void *pty = (void *)IL(p, AST_F0);
                if (!pty) return ac;
                void *cst = kcc_ast_alloc(
   AST_CAST_EXPR);
                IL(cst, AST_F0) = (unsigned long)
```

```
pty;
            IL(cst, AST_F1) = (unsigned long)
ac;
            IL(cst, AST_F3) = (unsigned long)
kcc_sema_resolve_type(pty);
            return cst;
        }
        p = (void *)IL(p, AST_F4); a = (void
*)IL(a, AST_F4);
    }
    return kcc_inl_dup(node);
 /* a global / enum reference */
}
 if (tag == AST_BINARY_EXPR || tag ==
AST_ASSIGN_EXPR || tag == AST_COMMA_EXPR) {
    void *l = kcc_inl_clone((void *)IL(node,
AST_F1), params, args);
    void *r = kcc_inl_clone((void *)IL(node,
AST_F2), params, args);
    if (!l || !r) return 0;
    void *d = kcc_inl_dup(node);
    IL(d, AST_F1) = (unsigned long)l; IL(d,
AST_F2) = (unsigned long)r;
    return d;
}
 if (tag == AST_UNARY_EXPR || tag ==
AST_PAREN_EXPR) {
    unsigned long fld = (tag == AST_UNARY_EXPR
) ? AST_F1 : AST_F0;
    void *o = kcc_inl_clone((void *)IL(node,
fld), params, args);
    if (!o) return 0;
    void *d = kcc_inl_dup(node);
    IL(d, fld) = (unsigned long)o;
    return d;
}
 if (tag == AST_CAST_EXPR) {
 /* F0 = type (shared), F1 = expr */
    void *o = kcc_inl_clone((void *)IL(node,
AST_F1), params, args);
    if (!o) return 0;
    void *d = kcc_inl_dup(node);
    IL(d, AST_F1) = (unsigned long)o;
    return d;
}
 if (tag == AST_COND_EXPR) {
    void *c = kcc_inl_clone((void *)IL(node,
AST_F0), params, args);
    void *t = kcc_inl_clone((void *)IL(node,
AST_F1), params, args);
    void *f = kcc_inl_clone((void *)IL(node,
AST_F2), params, args);
    if (!c || !t || !f) return 0;
    void *d = kcc_inl_dup(node);
    IL(d, AST_F0) = (unsigned long)c; IL(d,
AST_F1) = (unsigned long)t;
    IL(d, AST_F2) = (unsigned long)f;
    return d;
}
 if (tag == AST_SUBSCRIPT) {
    void *b = kcc_inl_clone((void *)IL(node,
AST_F0), params, args);
    void *i = kcc_inl_clone((void *)IL(node,
AST_F1), params, args);
    if (!b || !i) return 0;
    void *d = kcc_inl_dup(node);
    IL(d, AST_F0) = (unsigned long)b; IL(d,
AST_F1) = (unsigned long)i;
    return d;
}
 if (tag == AST_MEMBER) {
 /* F0 = obj, F1 = name, F2 = arrow */
    void *o = kcc_inl_clone((void *)IL(node,
AST_F0), params, args);
    if (!o) return 0;
    void *d = kcc_inl_dup(node);
    IL(d, AST_F0) = (unsigned long)o;
    return d;
}
 if (tag == AST_SIZEOF_EXPR || tag ==
AST_ALIGNOF_EXPR) {
    void *d = kcc_inl_dup(node);
    if (!IL(node, AST_F1)) {
 /* F1 set = a type operand (kept) */
        void *o = kcc_inl_clone((void *)IL(
node, AST_F0), params, args);
        if (!o) return 0;
        IL(d, AST_F0) = (unsigned long)o;
    }
    return d;
}
 if (tag == AST_CALL_EXPR) {
    void *cal = kcc_inl_clone((void *)IL(node,
 AST_F0), params, args);
    if (!cal) return 0;
    void *al = kcc_inl_clone_list((void *)IL(
node, AST_F1), params, args);
    if (al == (void *)-1) return 0;
    void *d = kcc_inl_dup(node);
    IL(d, AST_F0) = (unsigned long)cal; IL(d,
AST_F1) = (unsigned long)al;
    return d;
}
 /* --- statements (the general body path) ---
*/
 if (tag == AST_EXPR_STMT || tag ==
AST_STMT_EXPR) {
    void *o = kcc_inl_clone((void *)IL(node,
AST_F0), params, args);
    if (!o) return 0;
    void *d = kcc_inl_dup(node);
    IL(d, AST_F0) = (unsigned long)o;
    return d;
}
 if (tag == AST_COMPOUND) {
    void *lst = kcc_inl_clone_list((void *)IL(
node, AST_F0), params, args);
    if (lst == (void *)-1) return 0;
    void *d = kcc_inl_dup(node);
    IL(d, AST_F0) = (unsigned long)lst;
    return d;
}
 if (tag == AST_RETURN_STMT) {
    /* `return X;' -> `{ __r = X; goto __end;
}' (or just the goto for void). */
    void *g = kcc_ast_alloc(AST_GOTO_STMT);
    IL(g, AST_F0) = (unsigned long)
kcc_inl_end_name;
    void *seq = g;
    void *xe = (void *)IL(node, AST_F0);
    if (xe && kcc_inl_ret_name) {
        void *xc = kcc_inl_clone(xe, params,
args);
        if (!xc) return 0;
        void *lhs = kcc_ast_alloc(
AST_IDENT_EXPR);
        IL(lhs, AST_F0) = (unsigned long)
kcc_inl_ret_name;
        void *asn = kcc_ast_alloc(
AST_ASSIGN_EXPR);
```

```
            IL(asn, AST_F0) = TOK_ASSIGN;
            IL(asn, AST_F1) = (unsigned long)lhs;
            IL(asn, AST_F2) = (unsigned long)xc;
            void *es = kcc_ast_alloc(AST_EXPR_STMT
);
            IL(es, AST_F0) = (unsigned long)asn;
            IL(es, AST_F4) = (unsigned long)g;
            seq = es;
        }
        void *comp = kcc_ast_alloc(AST_COMPOUND);
        IL(comp, AST_F0) = (unsigned long)seq;
        return comp;
    }
    if (tag == AST_IF_STMT) {
        void *c = kcc_inl_clone((void *)IL(node,
AST_F0), params, args);
        void *t = kcc_inl_clone((void *)IL(node,
AST_F1), params, args);
        if (!c || !t) return 0;
        void *el = 0;
        if (IL(node, AST_F2)) {
            el = kcc_inl_clone((void *)IL(node,
AST_F2), params, args);
            if (!el) return 0;
        }
        void *d = kcc_inl_dup(node);
        IL(d, AST_F0) = (unsigned long)c; IL(d,
AST_F1) = (unsigned long)t;
        IL(d, AST_F2) = (unsigned long)el;
        return d;
    }
    if (tag == AST_WHILE_STMT || tag ==
AST_DO_STMT || tag == AST_SWITCH_STMT) {
        void *a0 = kcc_inl_clone((void *)IL(node,
AST_F0), params, args);
        void *a1 = kcc_inl_clone((void *)IL(node,
AST_F1), params, args);
        if (!a0 || !a1) return 0;
        void *d = kcc_inl_dup(node);
        IL(d, AST_F0) = (unsigned long)a0; IL(d,
AST_F1) = (unsigned long)a1;
        return d;
    }
    if (tag == AST_CASE_STMT) {
      /* F0 = value, F1 = body */
        void *a0 = kcc_inl_clone((void *)IL(node,
AST_F0), params, args);
        void *a1 = kcc_inl_clone((void *)IL(node,
AST_F1), params, args);
        if (!a0 || !a1) return 0;
        void *d = kcc_inl_dup(node);
        IL(d, AST_F0) = (unsigned long)a0; IL(d,
AST_F1) = (unsigned long)a1;
        return d;
    }
    if (tag == AST_FOR_STMT) {
        void *c[4]; unsigned long fld[4] = {AST_F0
, AST_F1, AST_F2, AST_F3};
        for (int k = 0; k < 4; k++) {
            void *src = (void *)IL(node, fld[k]);
            c[k] = src ? kcc_inl_clone(src, params
, args) : 0;
            if (src && !c[k]) return 0;
        }
        void *d = kcc_inl_dup(node);
        for (int k = 0; k < 4; k++) IL(d, fld[k])
= (unsigned long)c[k];
        return d;
    }
    if (tag == AST_DEFAULT_STMT) {
      /* F0 = labelled statement */
        /* A `default:' carries its statement in
F0 (kcc_parse_default), not F1.
           Cloning F1 left F0 as kcc_inl_dup's
shallow copy---the ORIGINAL body
           node, shared with the un-inlined AST---
and building that shared node
           corrupted state and crashed (kernel
switch(sizeof) fallbacks spell
           `default: __bad(); break;').  Clone the
 real body in F0. */
        void *a0 = kcc_inl_clone((void *)IL(node,
AST_F0), params, args);
        if (!a0) return 0;
        void *d = kcc_inl_dup(node);
        IL(d, AST_F0) = (unsigned long)a0;
        return d;
    }
    if (tag == AST_LABEL_STMT) {
     /* rename the label per instance */
        void *a1 = kcc_inl_clone((void *)IL(node,
AST_F1), params, args);
        if (!a1) return 0;
        void *d = kcc_inl_dup(node);
        IL(d, AST_F0) = (unsigned long)kcc_inl_lbl
((const char *)IL(node, AST_F0));
        IL(d, AST_F1) = (unsigned long)a1;
        return d;
    }
    if (tag == AST_GOTO_STMT) {
        void *d = kcc_inl_dup(node);
        IL(d, AST_F0) = (unsigned long)kcc_inl_lbl
((const char *)IL(node, AST_F0));
        return d;
    }
    if (tag == AST_BREAK_STMT || tag ==
AST_CONTINUE_STMT || tag == AST_NULL_STMT)
        return kcc_inl_dup(node);
    if (tag == AST_VAR_DECL) {
      /* a local: block-scoped, name kept */
        void *d = kcc_inl_dup(node);
        if (IL(node, AST_F2)) {
            void *ini = kcc_inl_clone((void *)IL(
node, AST_F2), params, args);
            if (!ini) return 0;
            IL(d, AST_F2) = (unsigned long)ini;
        }
        return d;
    }
    if (tag == AST_ASM_STMT) {
        void *d = kcc_inl_dup(node);
      /* f0 template, f1 length kept */
        void *ops = 0, *last = 0;
        for (void *op = (void *)IL(node, AST_F2);
op; op = (void *)IL(op, AST_F4)) {
            void *ve = kcc_inl_clone((void *)IL(op
, AST_F2), params, args);
            if (!ve) return 0;
            void *oc = kcc_inl_dup(op);
      /* f0 constraint, f1 [name] kept */
            IL(oc, AST_F2) = (unsigned long)ve;
            if (!ops) { ops = oc; last = oc; }
            else { IL(last, AST_F4) = (unsigned
long)oc; last = oc; }
        }
        IL(d, AST_F2) = (unsigned long)ops;
        /* goto labels: keep F0 (the template-
match name for %l[name]) but resolve
           through the renamed label via the F1
alias, so no template surgery. */
        void *gls = 0; last = 0;
```

```
        for (void *l = (void *)IL(node, AST_F3); l
    ; l = (void *)IL(l, AST_F4)) {
            void *lc = kcc_inl_dup(l);
            IL(lc, AST_F1) = (unsigned long)
    kcc_inl_lbl((const char *)IL(l, AST_F0));
            IL(lc, AST_F2) = 0;
            if (!gls) { gls = lc; last = lc; }
            else { IL(last, AST_F4) = (unsigned
    long)lc; last = lc; }
        }
        IL(d, AST_F3) = (unsigned long)gls;
        return d;
     }
     return 0;
      /* unhandled: decline to inline */
}

/* If `name` is a registered always_inline
    function with a `{ return E; }` body
    and a matching argument count, build E with
    parameters replaced by the call
    arguments and report its value; return 0 to
    fall back to a real call. */
static unsigned long kcc_inl_depth = 0;          /*
     live inline-expansion nesting */
static int kcc_ir_try_inline(void *call, const
    char *name, unsigned long *out) {
     if (!kcc_always_inline_fns) return 0;
     void *fn = (void *)kcc_ht_get(
    kcc_always_inline_fns, name, kcc_strlen(name));
     if (!fn) return 0;
     void *body = (void *)IL(fn, AST_F3);
     if (!body || IL(body, AST_TAG) != AST_COMPOUND
    ) return 0;
     /* Bound the expansion: building an inlined
    body recursively inlines the
        always_inline calls inside it, and the
    kernel's tracing macros nest these
        deeply enough (or mutually) to overflow the
     compiler's own stack.  Beyond
        a generous depth fall back to a real call
    ---the outer levels are already
        inlined, so a rare deep call is harmless.
    */
     if (kcc_inl_depth >= 40) return 0;
     void *params = (void *)(IL(fn, AST_F2) & ~3UL)
    ; /* strip variadic + linkage flags */
     /* A `(void)' parameter list is a single void-
    typed placeholder, not a real
        parameter; drop it so a no-argument callee
    (get_current, rep_nop,
        native_irq_enable, ... --- the pervasive
    inline-asm kernel primitives)
        matches its zero-argument call instead of
    failing the np==na test. */
     if (params && !IL(params, AST_F4)) {
         void *pt = (void *)IL(params, AST_F0);
         void *rt = pt ? kcc_sema_resolve_type(pt)
    : 0;
         if (rt && IL(rt, TY_KIND) == TY_VOID)
    params = 0;
     }
     void *args = (void *)IL(call, AST_F1);
     unsigned long np = 0, na = 0;
     for (void *p = params; p; p = (void *)IL(p,
    AST_F4)) np++;
     for (void *a = args; a; a = (void *)IL(a,
    AST_F4)) na++;
     if (np != na) return 0;
       /* no variadic inlining */

     /* An argument with a side effect (a call, an
    increment, an assignment) cannot
        be substituted textually: kcc_inl_clone
    copies it to every use of the
        parameter, so a body mentioning the
    parameter twice would evaluate it twice,
        and one dropping it (a folded-away branch)
    would lose its effect.  Bind each
        such argument to a fresh temporary,
    evaluated exactly once at the top of the
        expansion, and substitute a reference to
    that temporary.  Plain identifiers
        and constants stay textual so `
    __builtin_constant_p(nr)' still sees the
        literal and selects the constant-immediate
    asm branch --- the whole point of
        the bit-op helpers.  This is what lets
    arch_set_bit / arch_clear_bit inline
        when the address is a `per_cpu' pointer
    expression (a call) rather than a
        bare variable, instead of falling back to
    an out-of-line call that the asm
        body can never satisfy standalone. */
     void *sargs = args;
     void *tmpdecls = 0, *tmplast = 0;
     unsigned long any_unsafe = 0;
     for (void *a = args; a; a = (void *)IL(a,
    AST_F4))
         if (!kcc_inl_arg_safe(a)) { any_unsafe =
    1; break; }
     if (any_unsafe) {
         void *sfirst = 0, *slast = 0, *p = params,
     *a = args;
         while (p && a) {
             void *use;
             if (!kcc_inl_arg_safe(a)) {
                 void *ac = kcc_inl_clone(a, 0, 0);
       /* the argument, in caller scope */
                 if (!ac) return 0;
                 const char *tn = kcc_inl_lbl("
    __inl_a");
                 void *vd = kcc_ast_alloc(
    AST_VAR_DECL);
                 IL(vd, AST_F0) = IL(p, AST_F0);
       /* the parameter's type node */
                 IL(vd, AST_F1) = (unsigned long)tn
    ;
                 IL(vd, AST_F2) = (unsigned long)ac
    ;  /* PTYPE __inl_aN = <arg>; */
                 if (!tmpdecls) { tmpdecls = vd;
    tmplast = vd; }
                 else { IL(tmplast, AST_F4) = (
    unsigned long)vd; tmplast = vd; }
                 use = kcc_ast_alloc(AST_IDENT_EXPR
    );
                 IL(use, AST_F0) = (unsigned long)
    tn;
             } else {
                 use = kcc_inl_dup(a);
       /* textual, sibling link cleared */
             }
             if (!sfirst) { sfirst = use; slast =
    use; }
             else { IL(slast, AST_F4) = (unsigned
    long)use; slast = use; }
             p = (void *)IL(p, AST_F4); a = (void
    *)IL(a, AST_F4);
         }
         sargs = sfirst;
     }

     kcc_inl_inst++;
     /* Fast path: a lone `return EXPR;' body
```

```
    inlines to EXPR with no wrapper---only
        when no argument needed a temporary, since
    a bare expression has nowhere to
        declare one. */
     void *s0 = (void *)IL(body, AST_F0);
     if (!any_unsafe && s0 && !IL(s0, AST_F4) && IL
    (s0, AST_TAG) == AST_RETURN_STMT && IL(s0,
    AST_F0)) {
         void *clone = kcc_inl_clone((void *)IL(s0,
     AST_F0), params, args);
         if (!clone) return 0;
         kcc_inl_depth++;
         *out = kcc_ir_build_expr(clone);
         kcc_inl_depth--;
         return 1;
     }

     /* General path: clone the body, turning each
    `return X' into an assignment to
        a fresh __r plus `goto __end', and evaluate
     it as
        `({ RETTYPE __r; <body>; __end: ; __r; })'.
      build_stmt_expr registers the
        labels and takes the value from the
    trailing __r. */
     void *rettype = (void *)IL(fn, AST_F0);
     const char *retn = kcc_inl_lbl("__inl_r");
     const char *endn = kcc_inl_lbl("__inl_end");
     kcc_inl_ret_name = rettype ? retn : 0;
     kcc_inl_end_name = endn;
     void *bclone = kcc_inl_clone(body, params,
    sargs);
     kcc_inl_ret_name = 0; kcc_inl_end_name = 0;
     if (!bclone) return 0;

     void *first = (void *)IL(bclone, AST_F0);
     /* `__end: ;' followed by the value item */
     void *lbl = kcc_ast_alloc(AST_LABEL_STMT);
     IL(lbl, AST_F0) = (unsigned long)endn;
     IL(lbl, AST_F1) = (unsigned long)kcc_ast_alloc
    (AST_NULL_STMT);
     void *valstmt = 0;
     if (rettype) {
         void *rid = kcc_ast_alloc(AST_IDENT_EXPR);
         IL(rid, AST_F0) = (unsigned long)retn;
         valstmt = kcc_ast_alloc(AST_EXPR_STMT);
         IL(valstmt, AST_F0) = (unsigned long)rid;
     }
     IL(lbl, AST_F4) = (unsigned long)valstmt;
     if (first) {
         void *t = first;
         while (IL(t, AST_F4)) t = (void *)IL(t,
    AST_F4);
         IL(t, AST_F4) = (unsigned long)lbl;
     } else {
         first = lbl;
     }
     if (rettype) {
       /* prepend `RETTYPE __r;' */
         void *rdecl = kcc_ast_alloc(AST_VAR_DECL);
         IL(rdecl, AST_F0) = (unsigned long)rettype
    ;
         IL(rdecl, AST_F1) = (unsigned long)retn;
         IL(rdecl, AST_F4) = (unsigned long)first;
         first = rdecl;
     }
     if (tmpdecls) {
       /* declare the arg temporaries first */
         IL(tmplast, AST_F4) = (unsigned long)first
    ;
         first = tmpdecls;
     }
     IL(bclone, AST_F0) = (unsigned long)first;
     void *se = kcc_ast_alloc(AST_STMT_EXPR);
     IL(se, AST_F0) = (unsigned long)bclone;
     kcc_inl_depth++;
     *out = kcc_ir_build_stmt_expr(se);
     kcc_inl_depth--;
     return 1;
}

/* __builtin_{mul,add,sub}_overflow(a, b, *res):
    compute a OP b at the result
    type, store it through *res, and return whether
     the true result did not fit.
    The kernel uses these for size arithmetic,
    almost always with same-typed
    operands, so the operation is done at the
    result type's width.  Multiply
    overflow is the reliable divide-back identity
    res/a != b (exact for unsigned;
    the a==0 case is folded in by dividing by a+(a
    ==0) so the division is never by
    zero and the a!=0 guard zeroes the result
    anyway).  Add/subtract use the
    classic wrap (unsigned) and sign-agreement (
    signed) tests. */
static unsigned long kcc_ir_build_ovf_builtin(void
    *e, unsigned long aop) {
     void *a0 = (void *)IL(e, AST_F1);
     void *a1 = a0 ? (void *)IL(a0, AST_F4) : 0;
     void *a2 = a1 ? (void *)IL(a1, AST_F4) : 0;
     if (!a2) return kcc_ir_emit_const(0);
     void *pty = (void *)IL(a2, AST_F3);
       /* pointer to the result */
     void *rT = (pty && IL(pty, TY_KIND) == TY_PTR)
     ? (void *)IL(pty, TY_F0) : 0;
     if (!rT || IL(rT, TY_KIND) < TY_BOOL || IL(rT,
     TY_KIND) > TY_LLONG)
         rT = kcc_type_long_u;
     unsigned long av = kcc_ir_build_expr(a0);
     unsigned long bv = kcc_ir_build_expr(a1);
     unsigned long raddr = kcc_ir_build_expr(a2);
     av = kcc_ir_convert(av, (void *)IL(a0, AST_F3)
    , rT);
     bv = kcc_ir_convert(bv, (void *)IL(a1, AST_F3)
    , rT);
     unsigned long signd = IL(rT, TY_F0) ? 1 : 0;
     unsigned long prod = kcc_ir_emit_binop_typed(
    aop, av, bv, rT);
      /* Force the result to the destination width:
    a narrower multiply leaves the
        full product in the register, and the
    divide-back check would then never
        see the overflow.  Truncating (via convert
    from 64-bit) also sign-extends
        for a signed destination, which the signed
    divide/compares below need. */
     prod = kcc_ir_convert(prod, kcc_type_long_u,
    rT);
     kcc_ir_emit_store_t(prod, raddr,
    kcc_ir_load_width(rT), rT);
     unsigned long zero = kcc_ir_emit_const(0);
     if (aop == IR_MUL) {
         unsigned long aeqz =
    kcc_ir_emit_binop_typed(IR_CMP_EQ, av, zero, rT
    );
         unsigned long d = kcc_ir_emit_binop_typed(
    IR_ADD, av, aeqz, rT);
         unsigned long q = kcc_ir_emit_binop_typed(
    signd ? IR_SDIV : IR_UDIV, prod, d, rT);
         unsigned long qneb =
    kcc_ir_emit_binop_typed(IR_CMP_NE, q, bv, rT);
         unsigned long anz =
```

```
    kcc_ir_emit_binop_typed(IR_CMP_NE, av, zero, rT
    );
        return kcc_ir_emit_binop(IR_AND, anz, qneb
    );
     }
     if (!signd)
       /* add: res<a; sub: a<b */
        return (aop == IR_ADD)
            ? kcc_ir_emit_binop_typed(IR_CMP_ULT,
    prod, av, rT)
            : kcc_ir_emit_binop_typed(IR_CMP_ULT,
    av, bv, rT);
     unsigned long x1 = (aop == IR_ADD)
       /* add: (a^res)&(b^res)<0 */
        ? kcc_ir_emit_binop_typed(IR_XOR, av, prod
    , rT)   /* sub: (a^b)&(a^res)<0 */
        : kcc_ir_emit_binop_typed(IR_XOR, av, bv,
    rT);
     unsigned long x2 = (aop == IR_ADD)
        ? kcc_ir_emit_binop_typed(IR_XOR, bv, prod
    , rT)
        : kcc_ir_emit_binop_typed(IR_XOR, av, prod
    , rT);
     unsigned long ad = kcc_ir_emit_binop_typed(
    IR_AND, x1, x2, rT);
     return kcc_ir_emit_binop_typed(IR_CMP_SLT, ad,
     zero, rT);
}

unsigned long kcc_ir_build_call(void *e) {
    void *callee = (void *)IL(e, AST_F0);
    if (!callee) return 0;
    unsigned long indirect = 0;
    const char *name = 0;
    if (IL(callee, AST_TAG) == AST_IDENT_EXPR) {
        name = (const char *)IL(callee, AST_F0);
        if (kcc_strcmp(name, "va_start") == 0 ||
            kcc_strcmp(name, "__builtin_va_start")
     == 0) return kcc_ir_build_va_start(e);
        if (kcc_strcmp(name, "va_end") == 0 ||
            kcc_strcmp(name, "__builtin_va_end")
    == 0) return 0;
        if (kcc_strcmp(name, "va_copy") == 0 ||
            kcc_strcmp(name, "__builtin_va_copy")
    == 0) return kcc_ir_build_va_copy(e);
        /* Common GNU builtins the kernel leans on
     pervasively:
           __builtin_expect(x,c) is a branch hint
    -> just x; constant_p is a
           compile-time predicate; the mem*
    builtins are the libc entry points. */
        if (kcc_strcmp(name, "__builtin_expect")
    == 0)
            return kcc_ir_build_expr((void *)IL(e,
     AST_F1));
        if (kcc_strcmp(name, "__builtin_constant_p
    ") == 0)
            return kcc_ir_emit_const(
    kcc_expr_is_ice((void *)IL(e, AST_F1)) ? 1 : 0)
    ;
        /* __builtin_bswapN: reverse byte order
    with a portable IR gather. */
        if (kcc_strcmp(name, "__builtin_bswap16")
    == 0)
            return kcc_ir_bswap(kcc_ir_build_expr
    ((void *)IL(e, AST_F1)), 2);
        if (kcc_strcmp(name, "__builtin_bswap32")
    == 0)
            return kcc_ir_bswap(kcc_ir_build_expr
    ((void *)IL(e, AST_F1)), 4);
        if (kcc_strcmp(name, "__builtin_bswap64")
    == 0)
            return kcc_ir_bswap(kcc_ir_build_expr
    ((void *)IL(e, AST_F1)), 8);
        /* __builtin_unreachable() marks a point
    control never reaches; with no
           undefined-behaviour exploitation a
    plain fall-through is safe, so it
           yields a dummy value and emits nothing.
     */
        if (kcc_strcmp(name, "
    __builtin_unreachable") == 0)
            return kcc_ir_emit_const(0);
        /* Bit-scan builtins lower to portable bit
    -twiddling (like bswap), so
           they stay freestanding-correct without
    a new opcode. clz/ctz on the
           `l`/`ll` widths operate on 64 bits, the
     plain form on 32. */
        if (kcc_strcmp(name, "__builtin_clz") ==
    0)
            return kcc_ir_clz(kcc_ir_build_expr((
    void *)IL(e, AST_F1)), 32);
        if (kcc_strcmp(name, "__builtin_clzl") ==
    0 ||
            kcc_strcmp(name, "__builtin_clzll") ==
     0)
            return kcc_ir_clz(kcc_ir_build_expr((
    void *)IL(e, AST_F1)), 64);
        if (kcc_strcmp(name, "__builtin_ctz") ==
    0)
            return kcc_ir_ctz(kcc_ir_build_expr((
    void *)IL(e, AST_F1)), 32);
        if (kcc_strcmp(name, "__builtin_ctzl") ==
    0 ||
            kcc_strcmp(name, "__builtin_ctzll") ==
     0)
            return kcc_ir_ctz(kcc_ir_build_expr((
    void *)IL(e, AST_F1)), 64);
        if (kcc_strcmp(name, "__builtin_popcount")
     == 0)
            return kcc_ir_popcount(
    kcc_ir_build_expr((void *)IL(e, AST_F1)), 32);
        if (kcc_strcmp(name, "__builtin_popcountl"
    ) == 0 ||
            kcc_strcmp(name, "__builtin_popcountll
    ") == 0)
            return kcc_ir_popcount(
    kcc_ir_build_expr((void *)IL(e, AST_F1)), 64);
        if (kcc_strcmp(name, "__builtin_ffs") ==
    0)
            return kcc_ir_ffs(kcc_ir_build_expr((
    void *)IL(e, AST_F1)), 32);
        if (kcc_strcmp(name, "__builtin_ffsl") ==
    0 ||
            kcc_strcmp(name, "__builtin_ffsll") ==
     0)
            return kcc_ir_ffs(kcc_ir_build_expr((
    void *)IL(e, AST_F1)), 64);
        /* __builtin_object_size(ptr, type): the
    number of bytes from ptr to the
           end of its object, as a compile-time
    constant, or---when that cannot
           be determined, which is always so here
    since kcc does no object-size
           analysis (matching gcc at -O0)---the "
    unknown" answer: (size_t)-1 for
           the maximizing types 0/1, 0 for the
    minimizing types 2/3.  The pointer
           argument is never evaluated for side
    effects.  The kernel's
```

```
        FORTIFY_SOURCE checks read exactly this
 fallback. */
    if (kcc_strcmp(name, "
__builtin_object_size") == 0 ||
        kcc_strcmp(name, "
__builtin_dynamic_object_size") == 0) {
        void *sz0 = (void *)IL(e, AST_F1);
        void *sz1 = sz0 ? (void *)IL(sz0,
AST_F4) : 0;
        unsigned long ost = sz1 ?
kcc_eval_const_int(sz1) : 0;
        return kcc_ir_emit_const((ost & 2) ? 0
 : (unsigned long)-1);
    }
    /* __builtin_prefetch(addr, ...) is a
cache hint with no observable
        effect; evaluate the address (for any
side effects) and yield nothing. */
    if (kcc_strcmp(name, "__builtin_prefetch")
 == 0) {
        void *pa = (void *)IL(e, AST_F1);
        if (pa) kcc_ir_build_expr(pa);
        return kcc_ir_emit_const(0);
    }
    /* On x86 a return address needs no
demangling, so these are identity. */
    if (kcc_strcmp(name, "
__builtin_extract_return_addr") == 0 ||
        kcc_strcmp(name, "
__builtin_frob_return_addr") == 0)
        return kcc_ir_build_expr((void *)IL(e,
 AST_F1));
    /* __builtin_frame_address(n) /
__builtin_return_address(n): the frame
        pointer of the n-th enclosing frame,
and the return address saved in
        it.  The level is a constant; codegen
walks the saved-rbp chain. */
    if (kcc_strcmp(name, "
__builtin_frame_address") == 0 ||
        kcc_strcmp(name, "
__builtin_return_address") == 0) {
        void *lv = (void *)IL(e, AST_F1);
        unsigned long lvl = lv ?
kcc_eval_const_int(lv) : 0;
        unsigned long fop = (name[10] == 'f')
? IR_FRAME_ADDR : IR_RET_ADDR;
        void *inst = kcc_ir_alloc_instr(fop);
        IL(inst, IR_SRC1) = lvl;
        IL(inst, IR_TYPE) = (unsigned long)
kcc_type_long_u;
        unsigned long v = kcc_ir_next_vreg();
        IL(inst, IR_DEST) = v;
        kcc_ir_emit(inst);
        return v;
    }
    /* Overflow-checked arithmetic: store the
result through the third
        argument and yield whether it
overflowed. */
    if (kcc_strcmp(name, "
__builtin_mul_overflow") == 0)
        return kcc_ir_build_ovf_builtin(e,
IR_MUL);
    if (kcc_strcmp(name, "
__builtin_add_overflow") == 0)
        return kcc_ir_build_ovf_builtin(e,
IR_ADD);
    if (kcc_strcmp(name, "
__builtin_sub_overflow") == 0)
        return kcc_ir_build_ovf_builtin(e,
IR_SUB);
    /* __builtin_trap() must terminate; lower
it to a call to abort. */
    if (kcc_strcmp(name, "__builtin_trap") ==
0) name = "abort";
    /* abs family has an exact libc
counterpart (valid while the milestone
        links against a C library). */
    else if (kcc_strcmp(name, "__builtin_abs")
 == 0) name = "abs";
    else if (kcc_strcmp(name, "__builtin_labs"
) == 0) name = "labs";
    else if (kcc_strcmp(name, "__builtin_llabs
") == 0) name = "llabs";
    else if (kcc_strcmp(name, "
__builtin_memcpy") == 0) name = "memcpy";
    else if (kcc_strcmp(name, "
__builtin_memmove") == 0) name = "memmove";
    else if (kcc_strcmp(name, "
__builtin_memset") == 0) name = "memset";
    else if (kcc_strcmp(name, "
__builtin_strlen") == 0) name = "strlen";
    else if (kcc_strcmp(name, "
__builtin_strcpy") == 0) name = "strcpy";
    else if (kcc_strcmp(name, "
__builtin_strcmp") == 0) name = "strcmp";
    unsigned long v = (unsigned long)
kcc_ht_get(kcc_ir_locals, name, kcc_strlen(name
));
    /* An always_inline callee (not shadowed
by a local of the same name) is
        expanded in place rather than called.
*/
    if (!v) {
        unsigned long inl;
        if (kcc_ir_try_inline(e, name, &inl))
return inl;
        /* A registered always_inline callee
we could NOT inline falls through
            to a plain `call name'; since it is
 never emitted standalone and is
            usually static, record it for the
out-of-line pass. */
        if (kcc_always_inline_fns &&
            kcc_ht_get(kcc_always_inline_fns,
name, kcc_strlen(name))) {
            if (!kcc_inl_need_out)
kcc_inl_need_out = kcc_ht_init(32);
            kcc_ht_put(kcc_inl_need_out, name,
 kcc_strlen(name), (void *)1);
        }
    }
    if (v) indirect = 1;
    else {
        void *ct = (void *)IL(callee, AST_F3);
        if (ct && IL(ct, TY_KIND) == TY_PTR)
indirect = 1;
    }
 } else {
    indirect = 1;
 }

 /* count args */
 unsigned long argc = 0;
 void *a = (void *)IL(e, AST_F1);
 while (a) { argc++; a = (void *)IL(a, AST_F4);
 }

 /* aggregate return-by-value: reserve a result
 buffer */
 unsigned long sret = 0;
```

```
    /* buffer vreg + 1, 0 = none */
  void *rt = (void *)IL(e, AST_F3);
  if (rt && (IL(rt, TY_KIND) == TY_STRUCT || IL(
rt, TY_KIND) == TY_UNION)) {
      unsigned long slots = kcc_ir_type_slots(rt
);
      void *al = kcc_ir_alloc_instr(IR_ALLOCA);
      IL(al, IR_TYPE) = (unsigned long)rt;
      IL(al, IR_SRC1) = slots;
      unsigned long bv = kcc_ir_next_vreg();
      IL(al, IR_DEST) = bv;
      kcc_ir_emit(al);
      sret = bv + 1;
  }

  unsigned long total = argc + (sret ? 1 : 0);
  unsigned long *args = kcc_arena_alloc((1 +
total * 2) * 8);

  /* first_stack_idx */
  unsigned long fsi = 1;
  void *ct = (void *)IL(callee, AST_F3);
  if (ct) {
      unsigned long k = IL(ct, TY_KIND);
      void *ft = 0;
      if (k == TY_PTR) { ft = (void *)IL(ct,
TY_F0); }
      else if (k == TY_FUNC) ft = ct;
      if (ft && IL(ft, TY_KIND) == TY_FUNC) {
          unsigned long nfixed = IL(ft, TY_QUALS
);
          if (nfixed) { fsi = nfixed; if (fsi >
8) fsi = 8; }
      }
  }
  args[0] = total | (fsi << 32);

  /* parameter cursor for prototyped conversion/
classification */
  void *pcur = 0;
  if (ct) {
      unsigned long k = IL(ct, TY_KIND);
      void *ft = 0;
      if (k == TY_FUNC) ft = ct;
      else if (k == TY_PTR) { void *p = (void *)
IL(ct, TY_F0); if (p && IL(p, TY_KIND) ==
TY_FUNC) ft = p; }
      if (ft) pcur = (void *)IL(ft, TY_F1);
  }

  a = (void *)IL(e, AST_F1);
  unsigned long i = 0;
  while (i < argc) {
      void *ptype = 0;
      if (pcur) {
          void *ptn = (void *)IL(pcur, AST_F0);
          if (ptn) ptype = kcc_sema_resolve_type
(ptn);
          pcur = (void *)IL(pcur, AST_F4);
      }
      void *cls_type = ptype ? ptype : (void *)
IL(a, AST_F3);
      unsigned long cls = kcc_ir_classify_type(
cls_type);
      unsigned long vreg;
      if (cls >= 2) {
          vreg = kcc_ir_build_struct_addr(a);
      } else {
          vreg = kcc_ir_build_expr(a);
          if (ptype) vreg = kcc_ir_convert(vreg,
 (void *)IL(a, AST_F3), ptype);
      }
      args[1 + i * 2] = vreg;
      args[2 + i * 2] = cls;
      a = (void *)IL(a, AST_F4);
      i++;
  }
  if (sret) {
      args[1 + argc * 2] = sret - 1;
      args[2 + argc * 2] = 5;
    /* class X8 */
  }

  void *inst;
  if (indirect) {
      /* peel parens and a dereference of a
function pointer */
      void *ce = callee;
      while (1) {
          unsigned long t = IL(ce, AST_TAG);
          if (t == AST_PAREN_EXPR) { void *in =
(void *)IL(ce, AST_F0); if (!in) break; ce = in
; continue; }
          if (t == AST_UNARY_EXPR && IL(ce,
AST_F0) == TOK_STAR) {
              void *dt = (void *)IL(ce, AST_F3);
              if (dt && IL(dt, TY_KIND) ==
TY_FUNC) {
                  void *op = (void *)IL(ce,
AST_F1);
                  if (!op) break;
                  ce = op; continue;
              }
          }
          break;
      }
      unsigned long fnptr = kcc_ir_build_expr(ce
);
      inst = kcc_ir_alloc_instr(IR_CALL_IND);
      IL(inst, IR_SRC1) = fnptr;
  } else {
      inst = kcc_ir_alloc_instr(IR_CALL);
      IL(inst, IR_SRC1) = (unsigned long)name;
  }
  IL(inst, IR_SRC2) = (unsigned long)args;
  IL(inst, IR_TYPE) = rt ? (unsigned long)rt : (
unsigned long)kcc_type_int_s;
  if (sret) IL(inst, IR_DEST) = sret - 1;
  else IL(inst, IR_DEST) = kcc_ir_next_vreg();
  kcc_ir_emit(inst);
  if (sret) return sret - 1;
  return IL(inst, IR_DEST);
}

unsigned long kcc_ir_ident_alloca(void *ident) {
  if (!ident) return (unsigned long)-1;
  unsigned long tag = IL(ident, AST_TAG);
  if (tag == AST_SUBSCRIPT) return
kcc_ir_build_subscript_addr(ident);
  if (tag == AST_MEMBER) return
kcc_ir_build_member_addr(ident);
  if (tag != AST_IDENT_EXPR) return (unsigned
long)-1;
  char *name = (char *)IL(ident, AST_F0);
  if (!name) return (unsigned long)-1;
  unsigned long v = (unsigned long)kcc_ht_get(
kcc_ir_locals, name, kcc_strlen(name));
  if (v) return (v & ~0x8000000000000000UL) - 1;
  /* file-scope va_list: its global-ref address
*/
  void *g = kcc_ir_alloc_instr(IR_GLOBAL_REF);
```

```
    IL(g, IR_SRC1) = (unsigned long)name;
    IL(g, IR_TYPE) = (unsigned long)kcc_type_int_s
   ;
    unsigned long rv = kcc_ir_next_vreg();
    IL(g, IR_DEST) = rv;
    kcc_ir_emit(g);
    return rv;
}

/* small IR emitters for the x86-64 System V
   va_list lowering */
static unsigned long ir_vc(unsigned long v) {
    void *n = kcc_ir_alloc_instr(IR_CONST);
    IL(n, IR_TYPE) = (unsigned long)
   kcc_type_long_s;
    IL(n, IR_SRC1) = v;
    unsigned long d = kcc_ir_next_vreg(); IL(n,
   IR_DEST) = d; kcc_ir_emit(n);
    return d;
}
static unsigned long ir_v2(unsigned long op,
   unsigned long a, unsigned long b) {
    void *n = kcc_ir_alloc_instr(op);
    IL(n, IR_TYPE) = (unsigned long)
   kcc_type_long_s;
    IL(n, IR_SRC1) = a; IL(n, IR_SRC2) = b;
    unsigned long d = kcc_ir_next_vreg(); IL(n,
   IR_DEST) = d; kcc_ir_emit(n);
    return d;
}
static unsigned long ir_vload(unsigned long addr,
   void *ty) {
    void *n = kcc_ir_alloc_instr(IR_LOAD);
    IL(n, IR_TYPE) = (unsigned long)ty;
    IL(n, IR_SRC1) = addr;
    unsigned long d = kcc_ir_next_vreg(); IL(n,
   IR_DEST) = d; kcc_ir_emit(n);
    return d;
}
static void ir_vstore(unsigned long val, unsigned
   long addr, void *ty) {
    void *n = kcc_ir_alloc_instr(IR_STORE);
    IL(n, IR_TYPE) = (unsigned long)ty;
    IL(n, IR_SRC1) = val; IL(n, IR_SRC2) = addr;
    kcc_ir_emit(n);
}

/* Branchless System V va_arg: the register-save
   area (rdi-r9 at +0, xmm0-7 at
   +48) and the stack overflow area are picked
   without a jump ---
   addr = overflow + c*(reg_addr - overflow), c =
   (offset < limit). */
static unsigned long ir_va_arg_sysv(unsigned long
   ap, int fp) {
    unsigned long off_field = fp ? 8 : 0;
    unsigned long base_off  = fp ? 48 : 0;
    unsigned long limit     = fp ? 128 : 48;
    unsigned long reg_adv   = fp ? 16 : 8;
    unsigned long S      = ir_vload(ap,
   kcc_type_long_s);              /* &va_list */
    unsigned long S_off  = ir_v2(IR_ADD, S, ir_vc(
   off_field));
    unsigned long off    = ir_vload(S_off,
   kcc_type_long_s);
    unsigned long c      = ir_v2(IR_CMP_ULT, off,
   ir_vc(limit));       /* 0/1 */
    unsigned long rs     = ir_vload(ir_v2(IR_ADD,
   S, ir_vc(24)), kcc_type_long_s);
    unsigned long ra     = ir_v2(IR_ADD, ir_v2(
   IR_ADD, rs, ir_vc(base_off)), off);
    unsigned long S_ov   = ir_v2(IR_ADD, S, ir_vc
   (16));
    unsigned long ov     = ir_vload(S_ov,
   kcc_type_long_s);
    unsigned long diff   = ir_v2(IR_SUB, ra, ov);
    unsigned long addr   = ir_v2(IR_ADD, ov, ir_v2
   (IR_MUL, c, diff));
    unsigned long val    = ir_vload(addr, fp ?
   kcc_type_double : kcc_type_long_s);
    ir_vstore(ir_v2(IR_ADD, off, ir_v2(IR_MUL, c,
   ir_vc(reg_adv))), S_off, kcc_type_long_s);
    ir_vstore(ir_v2(IR_ADD, ov, ir_v2(IR_MUL,
   ir_v2(IR_SUB, ir_vc(1), c), ir_vc(8))), S_ov,
   kcc_type_long_s);
    return val;
}

unsigned long kcc_ir_build_va_start(void *call) {
    unsigned long ap = kcc_ir_ident_alloca((void
   *)IL(call, AST_F1));
    if (kcc_target) {                    /* x86
   -64: build the System V va_list struct */
        unsigned long np = IL(kcc_ir_cur_func,
   IRF_PARAMS);
        void *cls = (void *)IL(kcc_ir_cur_func,
   IRF_PARAM_CLS);
        unsigned long ngp = 0, nfp = 0;
        for (unsigned long p = 0; p < np; p++) {
            unsigned long cl = cls ? ((unsigned
   long *)cls)[p] : 0;
            if (cl == 1) nfp++; else ngp++;
        }
        if (ngp > 6) ngp = 6;
        if (nfp > 8) nfp = 8;
        void *a = kcc_ir_alloc_instr(IR_ALLOCA);
        /* 32-byte struct (4 slots) */
        IL(a, IR_TYPE) = (unsigned long)
   kcc_type_long_s;
        IL(a, IR_SRC1) = 4;
        unsigned long S = kcc_ir_next_vreg();
        IL(a, IR_DEST) = S;
        kcc_ir_emit(a);
        void *vs = kcc_ir_alloc_instr(IR_VA_START)
   ;
        IL(vs, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
        IL(vs, IR_SRC1) = S;
        IL(vs, IR_SRC2) = (nfp << 8) | ngp;
        unsigned long d = kcc_ir_next_vreg();
        IL(vs, IR_DEST) = d;
        kcc_ir_emit(vs);
        ir_vstore(S, ap, kcc_type_long_s);
        return S;
    }
    unsigned long F = IL(kcc_ir_cur_func,
   IRF_PARAMS);
    void *vs = kcc_ir_alloc_instr(IR_VA_START);
    IL(vs, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    IL(vs, IR_SRC2) = F;
    unsigned long base = kcc_ir_next_vreg();
    IL(vs, IR_DEST) = base;
    kcc_ir_emit(vs);
    void *st = kcc_ir_alloc_instr(IR_STORE);
    IL(st, IR_TYPE) = (unsigned long)
   kcc_type_int_s;
    IL(st, IR_SRC1) = base;
    IL(st, IR_SRC2) = ap;
    kcc_ir_emit(st);
    return base;
}
```

```
unsigned long kcc_ir_build_va_copy(void *call) {
    void *dst = (void *)IL(call, AST_F1);
    void *src = dst ? (void *)IL(dst, AST_F4) : 0;
    unsigned long ap_dst = kcc_ir_ident_alloca(dst
);
    unsigned long ap_src = kcc_ir_ident_alloca(src
);
    unsigned long S_src = ir_vload(ap_src,
kcc_type_long_s);
    if (kcc_target) {                    /* x86
-64: deep-copy the four-field struct */
        void *a = kcc_ir_alloc_instr(IR_ALLOCA);
        IL(a, IR_TYPE) = (unsigned long)
kcc_type_long_s;
        IL(a, IR_SRC1) = 4;
        unsigned long S_dst = kcc_ir_next_vreg();
        IL(a, IR_DEST) = S_dst;
        kcc_ir_emit(a);
        for (unsigned long k = 0; k < 4; k++) {
            unsigned long off = ir_vc(k * 8);
            unsigned long v = ir_vload(ir_v2(
IR_ADD, S_src, off), kcc_type_long_s);
            ir_vstore(v, ir_v2(IR_ADD, S_dst, off)
, kcc_type_long_s);
        }
        ir_vstore(S_dst, ap_dst, kcc_type_long_s);
        return 0;
    }
    ir_vstore(S_src, ap_dst, kcc_type_long_s);
/* ARM64: the cursor is the whole state */
    return 0;
}

unsigned long kcc_ir_build_va_arg(void *va) {
    unsigned long ap = kcc_ir_ident_alloca((void
*)IL(va, AST_F0));
    if (kcc_target) {                    /* x86-64
    System V */
        /* AST_F1 is the unresolved type-name node
; resolve it to a Type before
           reading its kind, so float/double
arguments take the %xmm path. */
        void *ty = kcc_sema_resolve_type((void *)
IL(va, AST_F1));
        int fp = ty && (IL(ty, TY_KIND) >=
TY_FLOAT && IL(ty, TY_KIND) <= TY_LDOUBLE);
        return ir_va_arg_sysv(ap, fp);
    }
    void *ld1 = kcc_ir_alloc_instr(IR_LOAD);
    IL(ld1, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    IL(ld1, IR_SRC1) = ap;
    unsigned long p = kcc_ir_next_vreg();
    IL(ld1, IR_DEST) = p;
    kcc_ir_emit(ld1);
    void *ld2 = kcc_ir_alloc_instr(IR_LOAD);
    IL(ld2, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    IL(ld2, IR_SRC1) = p;
    unsigned long val = kcc_ir_next_vreg();
    IL(ld2, IR_DEST) = val;
    kcc_ir_emit(ld2);
    void *c = kcc_ir_alloc_instr(IR_CONST);
    IL(c, IR_TYPE) = (unsigned long)kcc_type_int_s
;
    IL(c, IR_SRC1) = 8;
    unsigned long c8 = kcc_ir_next_vreg();
    IL(c, IR_DEST) = c8;
    kcc_ir_emit(c);
    void *add = kcc_ir_alloc_instr(IR_ADD);
    IL(add, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    IL(add, IR_SRC1) = p;
    IL(add, IR_SRC2) = c8;
    unsigned long p2 = kcc_ir_next_vreg();
    IL(add, IR_DEST) = p2;
    kcc_ir_emit(add);
    void *st = kcc_ir_alloc_instr(IR_STORE);
    IL(st, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    IL(st, IR_SRC1) = p2;
    IL(st, IR_SRC2) = ap;
    kcc_ir_emit(st);
    return val;
}

unsigned long kcc_ir_build_string_lit(void *lit) {
    void *inst = kcc_ir_alloc_instr(IR_STRING_LIT)
;
    IL(inst, IR_SRC1) = IL(lit, AST_F0);
    IL(inst, IR_SRC2) = IL(lit, AST_F1);
    IL(inst, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    unsigned long v = kcc_ir_next_vreg();
    IL(inst, IR_DEST) = v;
    kcc_ir_emit(inst);
    return v;
}

unsigned long kcc_ir_build_cond_expr(void *e) {
    /* Constant condition: evaluate only the taken
    arm (and lower only it).
       The kernel's `__builtin_constant_p(nr) ?
constant_test_bit(nr,a)
       : variable_test_bit(nr,a)` folds here - for
    a runtime `nr` the predicate
       is 0, so the constant-only arm (whose asm
needs an immediate) is dropped. */
    void *cc = (void *)IL(e, AST_F0);
    if (kcc_expr_is_ice(cc)) {
        if (kcc_eval_const_int(cc)) {
            void *ta = (void *)IL(e, AST_F1);
            return ta ? kcc_ir_build_expr(ta) :
kcc_ir_build_expr(cc);
        }
        return kcc_ir_build_expr((void *)IL(e,
AST_F2));
    }
    void *tmp = kcc_ir_alloc_instr(IR_ALLOCA);
    IL(tmp, IR_TYPE) = (unsigned long)
kcc_type_int_s;
    unsigned long slot = kcc_ir_next_vreg();
    IL(tmp, IR_DEST) = slot;
    kcc_ir_emit(tmp);
    unsigned long cond = kcc_ir_build_expr((void
*)IL(e, AST_F0));
    void *then_bb = kcc_ir_alloc_bb();
    void *else_bb = kcc_ir_alloc_bb();
    void *merge_bb = kcc_ir_alloc_bb();
    ir_brcond(cond, then_bb, else_bb);
    kcc_ir_set_bb(then_bb);
    /* GNU `a ?: b': the omitted middle operand (
AST_F1 == 0) reuses the
       already-computed condition value, so `a' is
    evaluated exactly once. */
    void *then_ast = (void *)IL(e, AST_F1);
    unsigned long tv = then_ast ?
kcc_ir_build_expr(then_ast) : cond;
    void *st1 = kcc_ir_alloc_instr(IR_STORE);
    IL(st1, IR_TYPE) = (unsigned long)
```

```
    kcc_type_int_s;
     IL(st1, IR_SRC1) = tv;
     IL(st1, IR_SRC2) = slot;
     kcc_ir_emit(st1);
     kcc_ir_branch_to(merge_bb);
     kcc_ir_set_bb(else_bb);
     unsigned long ev = kcc_ir_build_expr((void *)
    IL(e, AST_F2));
     void *st2 = kcc_ir_alloc_instr(IR_STORE);
     IL(st2, IR_TYPE) = (unsigned long)
    kcc_type_int_s;
     IL(st2, IR_SRC1) = ev;
     IL(st2, IR_SRC2) = slot;
     kcc_ir_emit(st2);
     kcc_ir_branch_to(merge_bb);
     kcc_ir_set_bb(merge_bb);
     void *ld = kcc_ir_alloc_instr(IR_LOAD);
     IL(ld, IR_TYPE) = (unsigned long)
    kcc_type_int_s;
     IL(ld, IR_SRC1) = slot;
     unsigned long res = kcc_ir_next_vreg();
     IL(ld, IR_DEST) = res;
     kcc_ir_emit(ld);
     return res;
}

unsigned long kcc_ir_build_cast(void *cast) {
     void *inner = (void *)IL(cast, AST_F1);
     unsigned long src = kcc_ir_build_expr(inner);
     void *to = (void *)IL(cast, AST_F3);
     if (!to) return src;
     void *from = (void *)IL(inner, AST_F3);
     if (!from) return src;
     /* __int128 casts.  Widening an integer into a
     __int128 fills a fresh
        16-byte slot (low half = src, high half
    zero- or sign-extended) and
        yields the slot address.  Narrowing a
    __int128 back to an integer loads
        its low half, then resizes if the target is
     under 8 bytes. */
     if (IL(to, TY_KIND) == TY_INT128) {
         if (IL(from, TY_KIND) == TY_INT128) return
     src;   /* already a slot */
         unsigned long slot = kcc_ir_i128_slot();
         unsigned long hi = kcc_ir_emit_const(0);
         if (IL(from, TY_KIND) != TY_PTR && IL(from
    , TY_F0) != 0)
             hi = kcc_ir_emit_binop(IR_ASHR, src,
    kcc_ir_emit_const(63));
         kcc_ir_i128_set(slot, src, hi);
         return slot;
     }
     if (IL(from, TY_KIND) == TY_INT128) {
         unsigned long lo = kcc_ir_emit_load(src,
    0);   /* 0 = 8-byte integer load */
         unsigned long ts0 = IL(to, TY_SIZE);
         if (ts0 >= 8) return lo;
         void *inst = kcc_ir_alloc_instr(IL(to,
    TY_F0) ? IR_SEXT : IR_ZEXT);
         IL(inst, IR_TYPE) = (unsigned long)to;
         IL(inst, IR_SRC1) = lo;
         IL(inst, IR_SRC2) = ts0;
         unsigned long r0 = kcc_ir_next_vreg();
         IL(inst, IR_DEST) = r0;
         kcc_ir_emit(inst);
         return r0;
     }
     if (kcc_ir_type_is_fp(to) || kcc_ir_type_is_fp
    (from))
         return kcc_ir_convert(src, from, to);
     /* integer resize */
     if (IL(to, TY_KIND) == TY_BOOL) return
    kcc_ir_emit_bool_norm(src);
     unsigned long ts = IL(to, TY_SIZE);
     unsigned long fs = IL(from, TY_SIZE);
     if (ts == fs) {
         if (ts >= 8) return src;
         unsigned long tsign = IL(to, TY_F0);
         if (tsign == IL(from, TY_F0)) return src;
       /* same width & sign */
         void *inst = kcc_ir_alloc_instr(tsign ?
    IR_SEXT : IR_ZEXT);
         IL(inst, IR_TYPE) = (unsigned long)to;
         IL(inst, IR_SRC1) = src;
         IL(inst, IR_SRC2) = ts;
         unsigned long r = kcc_ir_next_vreg();
         IL(inst, IR_DEST) = r;
         kcc_ir_emit(inst);
         return r;
     }
     /* different width: re-canonicalise to target
    width and signedness */
     void *inst = kcc_ir_alloc_instr(IL(to, TY_F0)
    ? IR_SEXT : IR_ZEXT);
     IL(inst, IR_TYPE) = (unsigned long)to;
     IL(inst, IR_SRC1) = src;
     IL(inst, IR_SRC2) = ts;
     unsigned long r = kcc_ir_next_vreg();
     IL(inst, IR_DEST) = r;
     kcc_ir_emit(inst);
     return r;
}
```

□

886 · used in 902a

75.10. Dump and Validation

The `--dump-ir` printer and the structural validator round out the module. Neither is exercised by `difftest` (the dump runs only under a debug flag, and the validator returns 0 for well-formed IR), but both are exported symbols the driver links against, so the port must define them.

⟨ *Selfhost: ir dump* 900 ⟩ ≡

```
static const char *ir_op_name(unsigned long op) {
     if (op == IR_ADD) return "add";
     if (op == IR_SUB) return "sub";
     if (op == IR_MUL) return "mul";
     if (op == IR_SDIV) return "sdiv";
     if (op == IR_UDIV) return "udiv";
     if (op == IR_SMOD) return "smod";
     if (op == IR_UMOD) return "umod";
     if (op == IR_NEG) return "neg";
     if (op == IR_AND) return "and";
     if (op == IR_OR) return "or";
     if (op == IR_XOR) return "xor";
     if (op == IR_SHL) return "shl";
     if (op == IR_LSHR) return "lshr";
     if (op == IR_ASHR) return "ashr";
     if (op == IR_NOT) return "not";
     if (op == IR_CMP_EQ) return "cmp_eq";
     if (op == IR_CMP_NE) return "cmp_ne";
     if (op == IR_CMP_SLT) return "cmp_slt";
     if (op == IR_CMP_SLE) return "cmp_sle";
     if (op == IR_CMP_SGT) return "cmp_sgt";
     if (op == IR_CMP_SGE) return "cmp_sge";
```

```
    if (op == IR_CMP_ULT) return "cmp_ult";
    if (op == IR_CMP_ULE) return "cmp_ule";
    if (op == IR_SEXT) return "sext";
    if (op == IR_ZEXT) return "zext";
    if (op == IR_TRUNC) return "trunc";
    if (op == IR_BITCAST) return "bitcast";
    if (op == IR_SITOFP) return "sitofp";
    if (op == IR_UITOFP) return "uitofp";
    if (op == IR_FPTOSI) return "fptosi";
    return "???";
}

void kcc_ir_dump_type_short(void *t) {
    if ((unsigned long)t < 0x1000) { kcc_io_puts("
   ?type"); return; }
    if (IL(t, TY_QUALS) & 2) kcc_io_puts("volatile
    ");
    unsigned long k = IL(t, TY_KIND);
    const char *s;
    if (k == TY_VOID) s = "void";
    else if (k == TY_BOOL) s = "i1";
    else if (k == TY_CHAR) s = "i8";
    else if (k == TY_SHORT) s = "i16";
    else if (k == TY_INT || k == TY_ENUM) s = "i32
   ";
    else if (k == TY_LONG || k == TY_LLONG) s = "
   i64";
    else if (k == TY_FLOAT) s = "f32";
    else if (k == TY_DOUBLE) s = "f64";
    else if (k == TY_PTR || k == TY_ARRAY || k ==
   TY_FUNC) s = "ptr";
    else if (k == TY_STRUCT || k == TY_UNION) s =
   "struct";
    else s = "?type";
    kcc_io_puts(s);
}

void kcc_ir_dump_instr(void *i) {
    unsigned long op = IL(i, IR_OP);
    kcc_io_puts("    ");
    if (!(op == IR_STORE || op == IR_BR || op ==
   IR_BR_COND || op == IR_RET || op == IR_NOP)) {
        kcc_io_putc('%'); kcc_io_putu(IL(i,
   IR_DEST)); kcc_io_puts(" = ");
    }
    void *ty = (void *)IL(i, IR_TYPE);
    unsigned long s1 = IL(i, IR_SRC1), s2 = IL(i,
   IR_SRC2);
    if (op == IR_LOAD) {
        kcc_io_puts("load ");
   kcc_ir_dump_type_short(ty); kcc_io_puts(", ptr
   %"); kcc_io_putu(s1);
    } else if (op == IR_STORE) {
        kcc_io_puts("store ");
   kcc_ir_dump_type_short(ty);
        kcc_io_puts(" %"); kcc_io_putu(s1);
   kcc_io_puts(", ptr %"); kcc_io_putu(s2);
    } else if (op == IR_ALLOCA) {
        kcc_io_puts("alloca ");
   kcc_ir_dump_type_short(ty);
    } else if (op == IR_GEP) {
        kcc_io_puts("gep %"); kcc_io_putu(s1);
   kcc_io_puts(", %"); kcc_io_putu(s2);
    } else if (op == IR_BR) {
        kcc_io_puts("br bb"); kcc_io_putu(s1);
    } else if (op == IR_BR_COND) {
        kcc_io_puts("br_cond %"); kcc_io_putu(IL(i
   , IR_DEST));
        kcc_io_puts(", bb"); kcc_io_putu(s1);
   kcc_io_puts(", bb"); kcc_io_putu(s2);
    } else if (op == IR_RET) {
        kcc_io_puts("ret ");
   kcc_ir_dump_type_short(ty);
        if (ty && IL(ty, TY_KIND) != TY_VOID) {
   kcc_io_puts(" %"); kcc_io_putu(s1); }
    } else if (op == IR_CALL) {
        kcc_io_puts("call ");
   kcc_ir_dump_type_short(ty);
        kcc_io_puts(" @"); if (s1) kcc_io_puts((
   const char *)s1);
    } else if (op == IR_CALL_IND) {
        kcc_io_puts("call_ind ");
   kcc_ir_dump_type_short(ty); kcc_io_puts(" %");
   kcc_io_putu(s1);
    } else if (op == IR_CONST) {
        kcc_io_puts("const ");
   kcc_ir_dump_type_short(ty); kcc_io_putc(' ');
   kcc_io_putu(s1);
    } else if (op == IR_NOP) {
        kcc_io_puts("nop");
    } else if (op == IR_GLOBAL_REF) {
        kcc_io_puts("global_ref ");
   kcc_ir_dump_type_short(ty);
        kcc_io_puts(" @"); if (s1) kcc_io_puts((
   const char *)s1);
    } else if (op == IR_STRING_LIT) {
        kcc_io_putc('"');
        const char *d = (const char *)s1;
        unsigned long j = 0;
        while (j < s2) { kcc_io_putc((unsigned
   char)d[j]); j++; }
        kcc_io_putc('"');
    } else if (op == IR_NEG || op == IR_NOT || op
   == IR_SEXT || op == IR_ZEXT ||
               op == IR_TRUNC || op == IR_BITCAST
   || op == IR_SITOFP ||
               op == IR_UITOFP || op == IR_FPTOSI)
    {
        kcc_io_puts(ir_op_name(op)); kcc_io_putc('
    ');
        kcc_ir_dump_type_short(ty); kcc_io_puts("
   %"); kcc_io_putu(s1);
    } else {
        kcc_io_puts(ir_op_name(op)); kcc_io_putc('
    ');
        kcc_ir_dump_type_short(ty);
        kcc_io_puts(" %"); kcc_io_putu(s1);
   kcc_io_puts(", %"); kcc_io_putu(s2);
    }
    kcc_io_putc('\n');
}

void kcc_ir_dump(void) {
    void *vec = kcc_ir_funcs;
    if (!vec) return;
    unsigned long n = IL(vec, VEC_COUNT);
    unsigned long i = 0;
    while (i < n) {
        void **slot = kcc_vec_get(vec, i);
        void *f = *slot;
        kcc_io_puts("func @");
        kcc_io_puts((const char *)IL(f, IRF_NAME))
   ;
        kcc_io_puts("() -> ");
        kcc_ir_dump_type_short((void *)IL(f,
   IRF_RETTYPE));
        kcc_io_puts(":\n");
        void *bvec = (void *)IL(f, IRF_BLOCKS);
        if (bvec) {
```

```
            unsigned long bn = IL(bvec, VEC_COUNT)
    ;
            unsigned long j = 0;
            while (j < bn) {
                void **bslot = kcc_vec_get(bvec, j
    );
                void *bb = *bslot;
                kcc_io_puts("  bb"); kcc_io_putu(
    IL(bb, BB_ID)); kcc_io_puts(":\n");
                void *ins = (void *)IL(bb,
    BB_FIRST);
                while (ins) { kcc_ir_dump_instr(
    ins); ins = (void *)IL(ins, IR_NEXT); }
                j++;
            }
            kcc_io_putc('\n');
        }
        i++;
    }
    kcc_io_flush();
}

unsigned long kcc_ir_validate(void) {
    void *vec = kcc_ir_funcs;
    if (!vec) return 0;
    unsigned long n = IL(vec, VEC_COUNT);
    unsigned long i = 0;
    while (i < n) {
        void **slot = kcc_vec_get(vec, i);
        void *f = *slot;
        const char *msg = 0;
        void *bb = (void *)IL(f, IRF_ENTRY);
        while (bb && !msg) {
            void *ins = (void *)IL(bb, BB_FIRST);
            unsigned long cnt = 0;
            while (ins) {
                cnt++;
                if (cnt > 0x100000) { msg = "kcc:
    internal error: cyclic instruction list in
    function "; break; }
                unsigned long op = IL(ins, IR_OP);
                if (op == 0 || op > IR_RET_ADDR) {
     msg = "kcc: internal error: invalid IR opcode
    in function "; break; }
                ins = (void *)IL(ins, IR_NEXT);
            }
            if (!msg && IL(bb, BB_FIRST) && !(IL(
    bb, BB_FLAGS) & BB_FLAG_TERMINATED))
                msg = "kcc: internal error:
    unterminated basic block in function ";
            bb = (void *)IL(bb, BB_NEXT);
        }
        if (msg) {
            kcc_io_puts(msg);
            void *nm = (void *)IL(f, IRF_NAME);
            if (nm) kcc_io_puts((const char *)nm);
            kcc_io_puts("\n");
            return 1;
        }
        i++;
    }
    return 0;
}
```

□

900 · used in 902a

75.11. Composition and Verification

The complete C module assembles the fragments in dependency order; the verify script compiles it with kcc, links it in place of the assembly `ir.o`, and runs the byte-exact `difftest` plus `diagtest` and the code-generation suite.

⟨ *Selfhost: ir* 902a ⟩ ≡

⟨ *Selfhost: ir preamble* 837 ⟩
⟨ *Selfhost: ir design* 840 ⟩
⟨ *Selfhost: ir spine* 841 ⟩
⟨ *Selfhost: ir exprs* 856, … ⟩
⟨ *Selfhost: ir aggregates* 873, … ⟩
⟨ *Selfhost: ir control* 879 ⟩
⟨ *Selfhost: ir assigncall* 886 ⟩
⟨ *Selfhost: ir dump* 900 ⟩

□

902a · used in 36p

⟨ *Selfhost: ir verify* 902b ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-ir - compile kcc's IR builder in C
    with kcc, relink, run
# difftest + diagtest + codegen.  The IR
   determines every downstream byte, so
# difftest byte-comparing against clang is the
   strongest possible oracle.
set -euo pipefail
SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "
echo "[1/4] compiling $BASE/selfhost/ir.c with kcc
   "
$BASE/kcc -c "$SH/ir.c" -o "$SH/ir_c.o" >/dev/null
echo "[2/4] linking $BASE/kcc_selfhost_ir (ir.o ->
    C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "ir" ]; then OBJ="$OBJ $SH/ir_c.o"
   ; else OBJ="$OBJ $BASE/src/${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_ir $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64
cp $BASE/kcc "$SH/kcc_asm_backup_ir"
cp $BASE/kcc_selfhost_ir $BASE/kcc
echo "[3/4] difftest (byte-compare) + diagtest"
set +e
bash $BASE/tests/difftest.sh >/$BASE/ir_difftest.
   log 2>&1; dstatus=$?
tail -1 /$BASE/ir_difftest.log
bash $BASE/tests/diagtest.sh >/$BASE/ir_diag.log
   2>&1; diagst=$?
tail -1 /$BASE/ir_diag.log
set -e
if [ $dstatus -ne 0 ] || [ $diagst -ne 0 ]; then
   cp "$SH/kcc_asm_backup_ir" $BASE/kcc; exit 1;
   fi
echo "[4/4] codegen suite with the self-hosted IR
   builder"
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_ir" $BASE/kcc
exit $status
```

□

902b · used in 37a · uses _main 621, 629

76. Self-Hosting: The Entry Point in C

The driver's `_main` (Chapter 44) was the last piece of the compiler still written in assembly other than the raw-syscall wrappers. It is pure control flow—initialise the subsystems, compare `argv[1]` against each mode flag, run the corresponding prefix of the pipeline—so porting it to C costs nothing in fidelity and buys two things at once. First, it finishes the self-hosting effort: every line of the compiler that is not a `svc/syscall` instruction is now C that kcc compiles. Second, it is exactly the entry point the second target needs. A C function `main` becomes the symbol `_main` that Darwin's `libSystem` start calls, and it is also the `main` that the Linux runtime's `_start` (Chapter 39) calls after laying out `argc/argv/envp` from the stack—so one C source is the entry on both platforms. The environment pointer, which the driver hands to the linker through `execve`, is recovered the same way the assembly did: it sits immediately past `argv`'s terminating null, at `argv + argc + 1`.

Because `main` drives the entire compiler, the port is verified functionally: swap the kcc-compiled `main.o` in for the assembly one and the whole test corpus—`difftest`'s byte-exact comparison against clang, the code-generation suite, and the diagnostic suite—must still pass, which it can only do if the dispatch is faithful.

76.1. Entry Point and Mode Dispatch

`main` initialises the subsystems, then dispatches on the arguments. The diagnostic modes (`--dump-tokens`, `--dump-ast`, `--check`, `--dump-ir`, …) are selected by `argv[1]`; anything else is a compile (`-c` anywhere) or a compile-and-link.

Two behaviours make the driver a usable gcc-style `CC=`. First, `-E` (preprocess only) is recognised *anywhere* in the command line and takes priority: `kcc_run_preprocess` registers the target, include directories (`-I`), macros (`-D`), and force-includes (`-include`) through the shared `kcc_driver_register_flags`—so a `#include` resolves exactly as it would when compiling, not silently vanishing—then preprocesses the source operand it returns (the last non-flag word, or `-` for standard input), and `kcc_pp_emit_text` writes the preprocessed token stream back as C *source text*—token spellings with line-preserving newlines and no line markers—rather than kcc's token dump, because that is the form a build system's compiler probe (Kbuild's `cc-version.sh`) reads. Second, the startup banner is written to `stderr`, keeping `stdout` clean for that probe. The same faithful emission also drives `kcc_assemble_source`: an assembly input (`.S`/`.s`) named on a `-c` line is preprocessed straight into `<out>.s`—by pointing `kcc_io_fd` at the file for the duration—and handed to the system assembler, so kcc builds the kernel's hand-written assembly, not only its C.

⟨ *Selfhost: main* 905 ⟩ ≡

```
/* subsystem init, front-end phases, sema, IR,
   driver, syscalls */
void kcc_arena_init(void);
void kcc_lex_kw_init(void);
void kcc_pp_add_default_includes(const char *path)
   ;
void kcc_io_puts(const char *);
void kcc_io_err(const char *);
void kcc_io_flush(void);
int kcc_strcmp(const char *, const char *);
void kcc_sys_exit(int);
void kcc_source_open(const char *);
void kcc_source_encode(void);
void kcc_source_splice(void);
void kcc_lex_init(void *base, unsigned long size);
void kcc_lex_dump_all(void);
unsigned long kcc_diag_had_errors(void);
void kcc_pp_init(void);
void kcc_pp_dump_all(void);
void kcc_strtab_init(void);
void kcc_typedef_init(void);
void *kcc_parse(void);
void kcc_ast_dump(void);
void kcc_types_init(void);
void kcc_symtab_init(void);
void kcc_sema_check(void *tu);
void kcc_sema_dump_types(void *tu);
void kcc_ir_init(void);
void kcc_ir_build(void *tu);
void kcc_opt_run(void);
void kcc_ir_dump(void);
void kcc_driver_compile_all(int argc, char **argv)
   ;
void kcc_driver_link_all(int argc, char **argv);
const char *kcc_driver_register_flags(long argc,
   char **argv);
void kcc_driver_apply_force_includes(void);
extern void *kcc_src_base;
extern unsigned long kcc_src_size;
extern char **kcc_envp;
   /* defined by the driver */
extern unsigned long kcc_target;
   /* set by --target=x86_64[-linux] */
extern unsigned long kcc_x86_os;
void kcc_pp_next(void);
/* For assembling `.S'/`.s' sources: preprocess-to
   -file then spawn the system as. */
unsigned long kcc_strlen(const char *);
void *kcc_arena_alloc(long);
long kcc_sys_open(const char *path, long flags,
   long mode);
long kcc_sys_close(long fd);
int kcc_driver_spawn(const char *path, char *const
    argv[]);
extern long kcc_io_fd;
   /* 1 (stdout) unless redirected */
extern unsigned long kcc_lex_asm;
   /* relax `'' for .S/.s inputs */
void kcc_io_write(const char *buf, unsigned long
   len);
extern unsigned long kcc_tok_type, kcc_tok_len,
   kcc_tok_line;
extern const char *kcc_tok_start;
extern unsigned long kcc_pp_lead_ws;
   /* token had leading source whitespace */

/* gcc-style -E: emit the preprocessed token
   stream as C source TEXT (not kcc's
   token dump).  Each token's spelling is written
```

```
   verbatim; a newline is emitted
   whenever the source line advances.  On the same
    line a space is written only
   where the source \emph{separated} the tokens.
   The preprocessor reports that per
   token as \texttt{kcc\_pp\_lead\_ws}: normally
   the byte before the token, but at a
   macro-expansion boundary---where that byte
   lives in a macro buffer and the source
   gap is invisible---the leading whitespace of
   the invocation, carried through so
   \texttt{= VAL} prints \texttt{= 42}, matching
   gcc's PREV\_WHITE flag.  Reproducing
   the spacing rather than forcing a space between
    every pair is what lets the output
   feed an assembler: \texttt{.macro} must not
   become `\texttt{. macro}', a
   `\texttt{672:}' label must not split, and `\
   texttt{\%rax}' must stay joined.  A few
   would-fuse pairs still get a synthesised space
   even when the source (or the macro
   body) had none: two word tokens (\texttt{.
   balign}+\texttt{16}), two punctuators that
   would join into a longer operator (\texttt
   {--}/\texttt{>>}/\texttt{\&\&}), a pp-number
   before a \texttt{+}/\texttt{-}/\texttt{.} that
   could extend it (\texttt{1e}+\texttt{+5}
   $\to$ \texttt{1e+5}, \texttt{3}+\texttt{.5}),
   in asm mode a word then a
   \texttt{\%}/\texttt{\$}/\texttt{(} operand
   starter (a mnemonic and its \texttt{\%reg})
   or a symbol then a string/char literal.  The
   number case fires only at an expansion
   boundary---detected by the current token not
   abutting the previous one's end---so a
   body-internal \texttt{5+3} stays verbatim, as
   in gcc.  Line markers are omitted
   (matching gcc's -P).  Defined here, in the
   driver, over kcc_pp_next and the
   spacing flag it publishes. */
static int kcc_word_char(char c) {
    return (c >= 'A' && c <= 'Z') || (c >= 'a' &&
   c <= 'z') ||
           (c >= '0' && c <= '9') || c == '_';
}

/* Would a token ending in `a' and one starting
   with `b', written adjacent, lex
   as a single longer pp-token?  The two-character
    C punctuators (`--', `>>',
   `&&', `+=', `->', ...); a mirror of the
   preprocessor's own paste table, kept
   local so the emitter stays self-contained.
   `/'+`/' is deliberately absent:
   after phase 3 those are two operators, and a
   computed include path such as
   `asm/trace//x86_fpu.h' must keep its `//'
   rather than gain a stray space. */
static int kcc_emit_pastes(char a, char b) {
    switch (a) {
    case '+': return b == '+' || b == '=';
    case '-': return b == '-' || b == '>' || b ==
   '=';
    case '<': return b == '<' || b == '=' || b ==
   ':' || b == '%';
    case '>': return b == '>' || b == '=';
    case '=': case '!': case '*': case '/': case '
   %': case '^': return b == '=';
    case '&': return b == '&' || b == '=';
    case '|': return b == '|' || b == '=';
    case '#': return b == '#';
    case ':': return b == '>';
    case '.': return b == '.';
    default:  return 0;
    }
}

static void kcc_pp_emit_text(void) {
    unsigned long prev_line = 0;
    char prev_last = 0;
    unsigned long prev_type = 0;
    const char *prev_end = 0;
   /* end of the last token emitted */
    for (;;) {
        kcc_pp_next();
        if (kcc_tok_type == 0) break;
   /* TOK_EOF */
        unsigned long line = kcc_tok_line;
        if (prev_line == 0) prev_line = line;
        else if (line != prev_line) {
            /* One newline on any line change.
   Line numbers are NOT globally
               monotonic---each #include resets to
   ~1, so catching up by count
               (`while prev<line') would merge
   every header's tail into the
               includer's line and, for a .S file,
    swallow a `.macro's `.endm'.
               A single newline per change puts
   every source line on its own
               output line, which is all the
   assembler (or a C re-parse) needs. */
            kcc_io_puts("\n");
            prev_line = line;
        }
        else {
            /* A space is emitted where the source
    separated the tokens.  The
               preprocessor publishes that as
   kcc_pp_lead_ws: for a plain token it
               is the byte before it, but across a
    macro-expansion boundary---where
               that byte lives in a macro buffer
   ---it is the leading whitespace of
               the invocation, carried through so
   `= VAL' prints `= 42' and not
               `=42'.  Two adjacent word tokens
   still get a forced space even when
               the source had none, since fusing
   them would forge a wrong token
               (`.balign'+`16'->`.balign16'); a
   word then a `%'/`$'/`(' operand
               starter needs the same only in
   assembly (a mnemonic and its `%reg',
               `$imm', or `('), never in C where
   they are always distinct tokens. */
            const char *s = kcc_tok_start;
            char b = s[0];
            int space = (int)kcc_pp_lead_ws;
            if (!space && kcc_word_char(prev_last)
    &&
                (kcc_word_char(b) ||
                 (kcc_lex_asm && (b == '%' || b ==
    '$' || b == '('))))
                space = 1;
            /* Two punctuators that would fuse
   into one longer token if abutted
               (`-'+`-'->`--', `>'+`>'->`>>',
   `&'+`&'->`&&') must stay apart
               across a macro-expansion boundary,
   where the source gap that kept
               them separate lives in another
   buffer and s[-1] cannot see it.
               The kernel plants exactly this: `
   MAX_NICE - MIN_NICE' with
               `MIN_NICE' expanding to `-20' would
```

```
     otherwise print `19 --20',
                which a re-parse reads as a post-
   decrement. */
             if (!space && kcc_emit_pastes(
   prev_last, b))
                 space = 1;
             /* A pp-number abutting `+'/`-'/`.'
   can fuse into ONE number---a
                `.'-fraction (`3'+`.5'->`3.5'), or
   an exponent sign when the number
                ends in e/E/p/P (`1e'+`+5'->`1e+5',
    `0x1p'+`+3').  This only happens
                across a macro-expansion boundary:
   two tokens contiguous in the same
                buffer (`5+3' in an object-like
   body) already lexed apart and stay
                verbatim.  A boundary is exactly
   where the current token does not
                abut the previous one's end.  gcc
   spaces the whole class (matching
                its conservative PREV_WHITE avoid-
   paste); we do too rather than test
                the last char, keeping `= P3+1' as
   `3 +1'. */
             if (!space && kcc_tok_start !=
   prev_end && prev_type == 2 /* TOK_NUMBER */ &&
                 (b == '+' || b == '-' || b == '.')
   )
                 space = 1;
             /* In assembly a directive or symbol
   abutting a string/char operand
                must stay separated: ELFNOTE's `.
   asciz CONFIG_BUILD_SALT' with an
                empty build salt expands to `.asciz
    ""', and glued to `.asciz""'
                the assembler reads an unterminated
    string and loses the note's
                `4484' label.  A C encoding-
   prefixed literal (L"...", u8"...") is a
                single token that never arrives
   here split, so gate this on asm
                mode to leave C output byte-for-
   byte unchanged. */
             if (!space && kcc_lex_asm &&
   kcc_word_char(prev_last) &&
                 (b == '"' || b == '\''))
                 space = 1;
             if (space) kcc_io_puts(" ");
         }
         kcc_io_write(kcc_tok_start, kcc_tok_len);
         prev_last = kcc_tok_start[kcc_tok_len -
   1];
         prev_type = kcc_tok_type;
         prev_end = kcc_tok_start + kcc_tok_len;
     }
     kcc_io_puts("\n");
     kcc_io_flush();
}

/* Terminal paths, mirroring the assembly's .
   Lmain_exit / _error_exit / usage. */
static void exit_ok(void)          { kcc_io_flush
   (); kcc_sys_exit(0); }
static void die_usage(const char *m){ kcc_io_err(m
   ); kcc_sys_exit(1); }
static void finish(void) {
     if (kcc_diag_had_errors()) { kcc_io_flush();
   kcc_sys_exit(1); }
     kcc_io_flush(); kcc_sys_exit(0);
}

/* Translation phases 1-2 then lexer init on the
   source at a given path.  An
   assembly input (`.S'/`.s') puts the lexer in
   relaxed mode so a stray `'' in a
   GAS `#'/`;' comment or a bare `'a' char does
   not read as an unterminated
   character constant. */
static void open_lex_path(const char *path) {
     unsigned long lp = kcc_strlen(path);
     kcc_lex_asm = (lp >= 2 && path[lp - 2] == '.'
   &&
                    (path[lp - 1] == 'S' || path[lp
    - 1] == 's')) ? 1 : 0;
     kcc_source_open(path);
     kcc_source_encode();
    /* phase 1: trigraphs */
     kcc_source_splice();
    /* phase 2: line splicing */
     kcc_lex_init(kcc_src_base, kcc_src_size);
}
static void open_lex(char **argv) { open_lex_path(
   argv[2]); }

/* Preprocess-only mode (`-E`): register the
   command line's target, include
   directories, macros, and force-includes exactly
    as the compile path does---so
   the headers a Kbuild `-nostdinc -I... -include
   ...` line names actually
   resolve---then preprocess the source operand it
    returns and emit the token
   text.  Without registering `-I`/`-D`/`-include`
    here, every `#include` would
   fail to find its header and the output would be
    the unexpanded source. */
static void kcc_run_preprocess(int argc, char **
   argv) {
     const char *src = kcc_driver_register_flags(
   argc, argv);
     if (!src) die_usage("kcc: error: -E requires a
     source file\n");
     open_lex_path(src);
     kcc_pp_init();
     kcc_driver_apply_force_includes();
     kcc_pp_emit_text();
     finish();
}

/* Assemble a `.S'/`.s' source into an object, the
    way `gcc -c foo.S' does: a
   `.S' is run through the C preprocessor (
   emitting faithful assembler text into
   <out>.s), a `.s' is already preprocessed;
   either is handed to the system
   assembler.  Called from the compile loop for
   every assembly input, so a
   Kbuild line naming head_64.S resolves its
   headers and macros exactly as a C
   compile would.  Flags (-I/-D/-include) are
   already registered by the caller;
   we mirror the compile path's pp setup (init
   then force-includes).  Output is
   redirected to <out>.s by pointing kcc_io_fd at
   it for the duration. */
int kcc_assemble_source(const char *src, const
   char *out) {
     unsigned long ls = kcc_strlen(src);
     const char *asmpath = src;
     if (src[ls - 1] == 'S') {
    /* .S: preprocess first */
         unsigned long lo = kcc_strlen(out);
         char *tmp = (char *)kcc_arena_alloc((long)
   (lo + 3));
```

```
        unsigned long j = 0;
        for (; j < lo; j++) tmp[j] = out[j];
        tmp[lo] = '.'; tmp[lo + 1] = 's'; tmp[lo +
    2] = 0;
        long fd = kcc_sys_open(tmp, 0x601, 0x1a4);
    /* O_WRONLY|O_CREAT|O_TRUNC, 0644 */
        if (fd < 0) { kcc_io_err("kcc: error:
   cannot create assembler output\n"); return 1; }
        kcc_io_flush();
    /* flush any pending stdout */
        long saved = kcc_io_fd;
        kcc_io_fd = fd;
        open_lex_path(src);
        kcc_pp_init();
        kcc_driver_apply_force_includes();
        kcc_pp_emit_text();
    /* writes preprocessed text to fd */
        kcc_io_fd = saved;
        kcc_sys_close(fd);
        if (kcc_diag_had_errors()) return 1;
        asmpath = tmp;
    }
    char *av[6];
    av[0] = (char *)"as";
    av[1] = (char *)"--64";
    av[2] = (char *)"-o";
    av[3] = (char *)out;
    av[4] = (char *)asmpath;
    av[5] = 0;
    return kcc_driver_spawn("/usr/bin/as", av) ? 1
    : 0;
}

int main(int argc, char **argv) {
    /* envp sits past argv's terminating NULL,
   exactly as the assembly computes
       it; the driver passes it on to the linker
   via execve. */
    kcc_envp = argv + argc + 1;

    kcc_arena_init();
    kcc_lex_kw_init();
    /* register dirname(argv[0])+"include" once,
   before any file, so the shipped
       hosted headers resolve with no -I. */
    kcc_pp_add_default_includes(argv[0]);
    /* The startup banner is a diagnostic, so it
   goes to stderr---keeping stdout
       clean for `-E` output that build-system
   compiler probes capture. */
    kcc_io_err("kcc v⟨ version ?⟩ - a formal C17
   compiler for ARM64\n");

    if (argc < 2) {
        kcc_io_puts("usage: kcc [--dump-tokens|-E
   |--dump-ast|--dump-types|--check|--dump-ir|-c <
   src> -o <out>] <source.c>\n");
        exit_ok();
    }

    /* -E (preprocess only) may appear anywhere in
    a gcc/Kbuild command line;
       when present it takes priority and emits
   preprocessed source text. */
    for (int i = 1; i < argc; i++) {
        if (kcc_strcmp(argv[i], "-E") == 0) {
   kcc_run_preprocess(argc, argv); }
    }

    const char *a1 = argv[1];
    if (kcc_strcmp(a1, "--dump-tokens") == 0) {
        if (argc < 3) die_usage("kcc: error: --
dump-tokens requires a source file\n");
        open_lex(argv);
        kcc_lex_dump_all();
        finish();
    }
    if (kcc_strcmp(a1, "--dump-ast") == 0) {
        if (argc < 3) die_usage("kcc: error: --
dump-ast requires a source file\n");
        open_lex(argv);
        kcc_pp_init();
        kcc_strtab_init();
        kcc_typedef_init();
        kcc_parse();
        kcc_ast_dump();
        finish();
    }
    if (kcc_strcmp(a1, "--dump-types") == 0) {
        if (argc < 3) die_usage("kcc: error: --
dump-types requires a source file\n");
        open_lex(argv);
        kcc_pp_init();
        kcc_strtab_init();
        kcc_typedef_init();
        void *tu = kcc_parse();
        kcc_types_init();
        kcc_symtab_init();
        kcc_sema_check(tu);
        kcc_sema_dump_types(tu);
        finish();
    }
    if (kcc_strcmp(a1, "--check") == 0) {
        if (argc < 3) die_usage("kcc: error: --
check requires a source file\n");
        open_lex(argv);
        kcc_pp_init();
        kcc_strtab_init();
        kcc_typedef_init();
        void *tu = kcc_parse();
        kcc_types_init();
        kcc_symtab_init();
        kcc_sema_check(tu);
        kcc_io_flush();
        if (kcc_diag_had_errors()) kcc_sys_exit(1)
;
        kcc_io_puts("semantic check passed\n");
        exit_ok();
    }
    if (kcc_strcmp(a1, "--dump-ir") == 0) {
        if (argc < 3) die_usage("kcc: error: --
dump-ir requires a source file\n");
        open_lex(argv);
        kcc_pp_init();
        kcc_strtab_init();
        kcc_typedef_init();
        void *tu = kcc_parse();
        kcc_types_init();
        kcc_symtab_init();
        kcc_sema_check(tu);
        if (kcc_diag_had_errors()) { kcc_io_flush
(); kcc_sys_exit(1); }
        kcc_ir_init();
        kcc_ir_build(tu);
        kcc_opt_run();
        kcc_ir_dump();
        finish();
    }
    /* Compile-only mode: "-c" anywhere in the
   argument list, not only as the
```

```
        first argument.  gcc-driven builds (Kbuild)
     lead with the compiler flags
        and end with `-c -o out.o in.c`, so a fixed
     argv[1] test would miss it and
        fall through to link mode, feeding the
    object flags to ld. */
     for (int i = 1; i < argc; i++) {
         if (kcc_strcmp(argv[i], "-c") == 0) {
             kcc_driver_compile_all(argc, argv);
    /* one object per source */
             exit_ok();
         }
     }

     /* Not a flag: the argument is a source file.
     Compile every source and link
        the resulting objects into an executable (
    the default mode). */
     kcc_driver_link_all(argc, argv);
     exit_ok();
     return 0;
    /* unreachable */
}
```

□

905 · used in 37b

76.2. Composition and Verification

The assembly chapter routes the `Selfhost: main` fragment above into `gen/selfhost/main.c`; nothing rewraps it here. What remains is to *verify* that entry point, which `make selfhost-main` does: it compiles `main.c` with kcc in place of the assembly `main.o`, relinks the compiler, and runs the difftest, diagtest, and codegen sweeps.

⟨ *Selfhost: main verify* 909 ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-main - compile kcc's entry point
   in C with kcc, relink in place
# of the assembly main.o, and run difftest +
   diagtest + codegen.  Because main
# drives the whole compiler, a clean sweep proves
   the dispatch is faithful.
set -euo pipefail
SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "
echo "[1/4] compiling $BASE/selfhost/main.c with
   kcc"
$BASE/kcc -c "$SH/main.c" -o "$SH/main_c.o" >/dev/
   null
echo "[2/4] linking $BASE/kcc_selfhost_main (main.
   o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "main" ]; then OBJ="$OBJ $SH/
   main_c.o"; else OBJ="$OBJ $BASE/src/${f}.o"; fi
done
ld -o $BASE/kcc_selfhost_main $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64
cp $BASE/kcc "$SH/kcc_asm_backup_main"
cp $BASE/kcc_selfhost_main $BASE/kcc
echo "[3/4] difftest + diagtest"
set +e
bash $BASE/tests/difftest.sh >/$BASE/main_difftest
   .log 2>&1; dstatus=$?
tail -1 /$BASE/main_difftest.log
bash $BASE/tests/diagtest.sh >/$BASE/main_diag.log
    2>&1; diagst=$?
tail -1 /$BASE/main_diag.log
set -e
if [ $dstatus -ne 0 ] || [ $diagst -ne 0 ]; then
   cp "$SH/kcc_asm_backup_main" $BASE/kcc; exit 1;
    fi
echo "[4/4] codegen suite with the self-hosted
   entry point"
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_main" $BASE/kcc
exit $status
```

□

909 · used in 37c · uses _main 621, 629

77. Self-Hosting: The Lexer in C

The compiler's lexer (Chapter 16) was, until now, the one runtime module still only in assembly: the earlier "lexer self-host" verified a *standalone* tokenizer against clang, not a drop-in for the module whose `kcc_lex_next` the preprocessor and every parser stage call. Porting it to C closes that gap. The lexer is a byte scanner—skip whitespace and comments while counting newlines, classify the first character, and hand off to the identifier, number, string, character, or punctuator scanner—so the port is mechanical, and it is verified the strongest way: because the token stream determines every downstream byte, swapping the kcc-compiled `lexer.o` in and getting `difftest`'s byte-exact comparison against clang proves the scan is faithful token-for-token.

The lexer keeps its state in globals—the cursor, the buffer end, the line number and line start, and the current token's type, text, and location—all carried at their assembly displacements so the other modules see the same symbols. A keyword identifier is recognised by looking its text up in the keyword table (populated by `lexer_tables`); every other scan sets the token type directly.

One wrinkle in classification: an identifier-start byte that is `L`, `u`, or `U` may not begin an identifier at all—followed by a quote it is the encoding prefix of a character constant (`L'x'`) or a string literal (`L"..."`, `u"..."`, `U"..."`, and the two-byte `u8"..."`). The dispatcher peeks the next byte (and the one after, for `u8`) and, on a quote, drops the prefix and scans the literal body. kcc's byte-oriented model has no wide or UTF-16 element type, so the prefix is only advisory and the body lexes as an ordinary narrow literal—but recognising it is essential: the kernel's `efi.h` passes `L"SecureBoot"` to a variable getter, and lexing the `L` as a bare identifier splits the call apart into "expected '(' ".

77.1. State, Init, and the Scanners

One piece of state is set by the driver, not the scanners: `kcc_lex_asm` marks the current input as assembly (a `.S`/`.s` file being preprocessed for `as`). In that mode the character scanner does not treat a `'` as necessarily opening a constant—GAS writes a bare `'a` and, far more often, an apostrophe rides inside a `#`/`;` line comment (`arch/x86/kernel/head_64.S`'s `# don't loop`)—so an otherwise unterminated `'` is emitted as a lone one-byte token and lexing resumes at the next character, leaving the assembler's comment intact instead of raising "unterminated character constant". For a `.c` compile the flag is 0 and the strict C rule stands.

⟨ *Selfhost: lexcore* 911 ⟩ ≡

```
/* token type codes the lexer itself emits (
   keyword codes 10-59 come from the
   keyword table's stored value, so they are not
   needed here). */
enum {
    TOK_EOF=0, TOK_IDENT=1, TOK_NUMBER=2,
   TOK_CHAR_LIT=3, TOK_STRING_LIT=4,
    TOK_UNKNOWN=5,
    TOK_LPAREN=60, TOK_RPAREN=61, TOK_LBRACE=62,
   TOK_RBRACE=63, TOK_LBRACKET=64,
    TOK_RBRACKET=65, TOK_SEMI=66, TOK_COMMA=67,
   TOK_DOT=68, TOK_ARROW=69,
    TOK_INC=70, TOK_DEC=71, TOK_AMP=72, TOK_STAR
   =73, TOK_PLUS=74, TOK_MINUS=75,
    TOK_TILDE=76, TOK_BANG=77, TOK_SLASH=78,
   TOK_PERCENT=79, TOK_SHL=80,
    TOK_SHR=81, TOK_LT=82, TOK_GT=83, TOK_LE=84,
   TOK_GE=85, TOK_EQ=86, TOK_NE=87,
    TOK_CARET=88, TOK_PIPE=89, TOK_AND=90, TOK_OR
   =91, TOK_QUESTION=92,
    TOK_COLON=93, TOK_ASSIGN=94, TOK_MUL_ASSIGN
   =95, TOK_DIV_ASSIGN=96,
    TOK_MOD_ASSIGN=97, TOK_ADD_ASSIGN=98,
   TOK_SUB_ASSIGN=99, TOK_SHL_ASSIGN=100,
    TOK_SHR_ASSIGN=101, TOK_AND_ASSIGN=102,
   TOK_XOR_ASSIGN=103, TOK_OR_ASSIGN=104,
    TOK_HASH=105, TOK_DHASH=106, TOK_ELLIPSIS=107
};

/* other modules */
void *kcc_ht_get(void *ht, const void *key,
   unsigned long keylen);
void kcc_diag_error(unsigned long line, unsigned
   long col, const char *msg);
const char *kcc_lex_type_name(unsigned long type);
void kcc_io_putu(unsigned long);
void kcc_io_putc(unsigned long);
void kcc_io_puts(const char *);
void kcc_io_write(const void *, unsigned long);

/* lexer state (this module is its definition site
   ) */
char *kcc_lex_cursor = 0;
char *kcc_lex_end = 0;
unsigned long kcc_lex_line = 1;
char *kcc_lex_line_start = 0;
unsigned long kcc_tok_type = 0;
char *kcc_tok_start = 0;
unsigned long kcc_tok_len = 0;
unsigned long kcc_tok_line = 0;
unsigned long kcc_tok_col = 0;
void *kcc_kw_table = 0;
   /* filled by lexer_tables */
unsigned long kcc_lex_asm = 0;
   /* input is `.S'/`.s': relax `'' */
unsigned long kcc_lex_no_comments = 0;
   /* lexing a macro-expansion buffer:

       `//'/`/*' formed by phase-4 token

      pasting are operators, not comments */

static int is_id_start(unsigned c) {
    return c == '_' || (c >= 'a' && c <= 'z') || (
   c >= 'A' && c <= 'Z');
}
static int is_id_cont(unsigned c) {
    return is_id_start(c) || (c >= '0' && c <= '9'
   );
}

void kcc_lex_init(char *base, unsigned long size)
   {
    kcc_lex_cursor = base;
```

```
    kcc_lex_end = base + size;
    kcc_lex_line = 1;
    kcc_lex_line_start = base;
}

/* tok_line = current line; tok_col = cursor -
   line_start + 1 */
static void kcc_lex_record_loc(void) {
    kcc_tok_line = kcc_lex_line;
    kcc_tok_col = (unsigned long)(kcc_lex_cursor -
    kcc_lex_line_start) + 1;
}

static void kcc_lex_scan_ident(void) {
    char *p = kcc_lex_cursor;
    char *end = kcc_lex_end;
    char *start = p;
    for (;;) {
        p++;
        if (p >= end) break;
        if (!is_id_cont((unsigned char)*p)) break;
    }
    kcc_lex_cursor = p;
    kcc_tok_start = start;
    kcc_tok_len = (unsigned long)(p - start);
    kcc_tok_type = TOK_IDENT;
    if (kcc_kw_table) {
        void *r = kcc_ht_get(kcc_kw_table, start,
   (unsigned long)(p - start));
        if (r) kcc_tok_type = (unsigned long)r;
    /* a keyword: its stored code */
    }
}

static void kcc_lex_scan_number(void) {
    char *p = kcc_lex_cursor;
    char *end = kcc_lex_end;
    char *start = p;
    for (;;) {
        p++;
        if (p >= end) break;
        unsigned char c = (unsigned char)*p;
        if (c >= '0' && c <= '9') continue;
        if ((c >= 'a' && c <= 'z') || (c >= 'A' &&
    c <= 'Z')) {
            if (c == 'e' || c == 'E' || c == 'p'
   || c == 'P') {   /* exponent sign */
                if (p + 1 < end) {
                    unsigned char n = (unsigned
   char)p[1];
                    if (n == '+' || n == '-') { p
   ++; continue; }
                }
            }
            continue;
        }
        if (c == '.' || c == '_') continue;
        break;
    }
    kcc_lex_cursor = p;
    kcc_tok_start = start;
    kcc_tok_len = (unsigned long)(p - start);
    kcc_tok_type = TOK_NUMBER;
}

static void kcc_lex_scan_string(void) {
    char *p = kcc_lex_cursor;
    char *end = kcc_lex_end;
    char *start = p;
    int unterm = 0;
    p++;
    /* skip opening " */
    for (;;) {
        if (p >= end) { unterm = 1; break; }
        unsigned char c = (unsigned char)*p;
        if (c == '"') { p++; break; }
        if (c == '\\') { p += 2; continue; }
    /* escape: skip 2 */
        if (c == '\n') { unterm = 1; break; }
        p++;
    }
    kcc_lex_cursor = p;
    if (unterm) kcc_diag_error(kcc_tok_line,
   kcc_tok_col, "unterminated string literal");
    kcc_tok_start = start;
    kcc_tok_len = (unsigned long)(p - start);
    kcc_tok_type = TOK_STRING_LIT;
}

static void kcc_lex_scan_char(void) {
    char *p = kcc_lex_cursor;
    char *end = kcc_lex_end;
    char *start = p;
    int unterm = 0;
    p++;
    /* skip opening ' */
    for (;;) {
        if (p >= end) { unterm = 1; break; }
        unsigned char c = (unsigned char)*p;
        if (c == '\'') { p++; break; }
        if (c == '\\') { p += 2; continue; }
        if (c == '\n') { unterm = 1; break; }
        p++;
    }
    /* Preprocessing a `.S' source (kcc_lex_asm):
   a `'' need not open a character
       constant.  GAS writes a bare `'a' char and,
    more commonly, `#'/`;' line
       comments carry apostrophes (`# don't loop')
   .  A C lexer would scan the rest
       of the line as an unterminated constant;
   instead emit the lone `'' as a
       one-byte token and resume at the next
   character, so the assembler (which
       owns the comment) sees the line intact. */
    if (unterm && kcc_lex_asm) {
        kcc_lex_cursor = start + 1;
        kcc_tok_start = start;
        kcc_tok_len = 1;
        kcc_tok_type = TOK_UNKNOWN;
        return;
    }
    kcc_lex_cursor = p;
    if (unterm) kcc_diag_error(kcc_tok_line,
   kcc_tok_col, "unterminated character constant")
   ;
    kcc_tok_start = start;
    kcc_tok_len = (unsigned long)(p - start);
    kcc_tok_type = TOK_CHAR_LIT;
}

/* longest-match punctuator scan (\stdref{6.4.6})
   */
static void kcc_lex_scan_punct(void) {
    char *p = kcc_lex_cursor;
    char *end = kcc_lex_end;
    char *start = p;
    unsigned char c = (unsigned char)*p;
    unsigned char c2 = (p + 1 < end) ? (unsigned
   char)p[1] : 0;
    unsigned long type;
    unsigned long n = 1;
    if (c == '(') type = TOK_LPAREN;
    else if (c == ')') type = TOK_RPAREN;
    else if (c == '{') type = TOK_LBRACE;
```

```
    else if (c == '}') type = TOK_RBRACE;
    else if (c == '[') type = TOK_LBRACKET;
    else if (c == ']') type = TOK_RBRACKET;
    else if (c == ';') type = TOK_SEMI;
    else if (c == ',') type = TOK_COMMA;
    else if (c == '~') type = TOK_TILDE;
    else if (c == '?') type = TOK_QUESTION;
    else if (c == ':') type = TOK_COLON;
    else if (c == '+') {
        type = TOK_PLUS;
        if (c2 == '+') { type = TOK_INC; n = 2; }
        else if (c2 == '=') { type =
TOK_ADD_ASSIGN; n = 2; }
    } else if (c == '-') {
        type = TOK_MINUS;
        if (c2 == '-') { type = TOK_DEC; n = 2; }
        else if (c2 == '=') { type =
TOK_SUB_ASSIGN; n = 2; }
        else if (c2 == '>') { type = TOK_ARROW; n
= 2; }
    } else if (c == '*') {
        type = TOK_STAR;
        if (c2 == '=') { type = TOK_MUL_ASSIGN; n
= 2; }
    } else if (c == '/') {
        type = TOK_SLASH;
        if (c2 == '=') { type = TOK_DIV_ASSIGN; n
= 2; }
    } else if (c == '%') {
        type = TOK_PERCENT;
        if (c2 == '=') { type = TOK_MOD_ASSIGN; n
= 2; }
    } else if (c == '<') {
        type = TOK_LT;
        if (c2 == '<') {
            type = TOK_SHL; n = 2;
            if (p + 2 < end && (unsigned char)p[2]
 == '=') { type = TOK_SHL_ASSIGN; n = 3; }
        } else if (c2 == '=') { type = TOK_LE; n =
 2; }
    } else if (c == '>') {
        type = TOK_GT;
        if (c2 == '>') {
            type = TOK_SHR; n = 2;
            if (p + 2 < end && (unsigned char)p[2]
 == '=') { type = TOK_SHR_ASSIGN; n = 3; }
        } else if (c2 == '=') { type = TOK_GE; n =
 2; }
    } else if (c == '=') {
        type = TOK_ASSIGN;
        if (c2 == '=') { type = TOK_EQ; n = 2; }
    } else if (c == '!') {
        type = TOK_BANG;
        if (c2 == '=') { type = TOK_NE; n = 2; }
    } else if (c == '&') {
        type = TOK_AMP;
        if (c2 == '&') { type = TOK_AND; n = 2; }
        else if (c2 == '=') { type =
TOK_AND_ASSIGN; n = 2; }
    } else if (c == '|') {
        type = TOK_PIPE;
        if (c2 == '|') { type = TOK_OR; n = 2; }
        else if (c2 == '=') { type = TOK_OR_ASSIGN
; n = 2; }
    } else if (c == '^') {
        type = TOK_CARET;
        if (c2 == '=') { type = TOK_XOR_ASSIGN; n
= 2; }
    } else if (c == '#') {
        type = TOK_HASH;
        if (c2 == '#') { type = TOK_DHASH; n = 2;
    }
    } else if (c == '.') {
        type = TOK_DOT;
        if (c2 == '.' && p + 2 < end && (unsigned
char)p[2] == '.') { type = TOK_ELLIPSIS; n = 3;
    }
    } else {
        type = TOK_UNKNOWN;
    /* unrecognized character */
    }
    kcc_lex_cursor = p + n;
    kcc_tok_type = type;
    kcc_tok_start = start;
    kcc_tok_len = n;
}

void kcc_lex_next(void) {
    char *p = kcc_lex_cursor;
    char *end = kcc_lex_end;
    for (;;) {
        if (p >= end) {
    /* EOF (cursor pinned to end) */
            kcc_lex_record_loc();
            kcc_tok_type = TOK_EOF;
            kcc_lex_cursor = end;
            kcc_tok_start = p;
            kcc_tok_len = 0;
            return;
        }
        unsigned char c = (unsigned char)*p;
        if (c == ' ' || c == '\t' || c == '\r' ||
c == '\v' || c == '\f') { p++; continue; }
        if (c == '\n') { p++; kcc_lex_line++;
kcc_lex_line_start = p; continue; }
        if (c == '/' && !kcc_lex_no_comments) {
            if (p + 1 < end) {
                unsigned char c2 = (unsigned char)
p[1];
                if (c2 == '/') {
    /* line comment */
                    p += 2;
                    while (p < end && *p != '\n')
p++;
                    continue;
                }
                if (c2 == '*') {
    /* block comment */
                    p += 2;
                    int unterm = 0;
                    for (;;) {
                        if (p >= end) { unterm =
1; break; }
                        unsigned char cc = (
unsigned char)*p;
                        if (cc == '\n') { p++;
kcc_lex_line++; kcc_lex_line_start = p;
continue; }
                        if (cc == '*') {
                            if (p + 1 < end && (
unsigned char)p[1] == '/') { p += 2; break; }
                            p++;
                            continue;
                        }
                        p++;
                    }
                    if (unterm) {
                        kcc_diag_error(
kcc_lex_line, 1, "unterminated block comment");
                        kcc_lex_record_loc();
                        kcc_tok_type = TOK_EOF;
                        kcc_lex_cursor = end;
```

```
                    kcc_tok_start = p;
                    kcc_tok_len = 0;
                    return;
                }
                continue;
            }
        }
        break;
 /* a lone '/' → punctuator */
    }
    break;
 /* significant character */
 }
 /* dispatch: publish cursor, record location,
classify */
 kcc_lex_cursor = p;
 kcc_lex_record_loc();
 unsigned char c = (unsigned char)*p;
 if (is_id_start(c)) {
    /* L'x' / u'x' / U'x' prefixed character
constant: drop the prefix */
    if ((c == 'L' || c == 'u' || c == 'U') &&
p + 1 < end && (unsigned char)p[1] == '\'') {
        kcc_lex_cursor = p + 1;
        kcc_lex_scan_char();
        return;
    }
    /* L"..." / u"..." / U"..." / u8"..."
prefixed string literal: drop the
       encoding prefix and lex the body as an
ordinary string.  kcc's
       byte-oriented model carries no wide/UTF
-16 element type, so the prefix
       is advisory---the kernel's efi.h passes
 `L"SecureBoot"' to a variable
       getter, and without this the `L' lexes
as an identifier and the string
       as a separate token, derailing the call
 into "expected `)'". */
    if ((c == 'L' || c == 'u' || c == 'U') &&
p + 1 < end) {
        if ((unsigned char)p[1] == '"') {
            kcc_lex_cursor = p + 1;
            kcc_lex_scan_string();
            return;
        }
        if (c == 'u' && (unsigned char)p[1] ==
 '8' &&
            p + 2 < end && (unsigned char)p[2]
 == '"') {
            kcc_lex_cursor = p + 2;
            kcc_lex_scan_string();
            return;
        }
    }
    kcc_lex_scan_ident();
    return;
 }
 if (c >= '0' && c <= '9') {
kcc_lex_scan_number(); return; }
 if (c == '"') { kcc_lex_scan_string(); return;
 }
 if (c == '\'') { kcc_lex_scan_char(); return;
}
 if (c == '.') {
    if (p + 1 < end) {
        unsigned char c2 = (unsigned char)p
[1];
        if (c2 >= '0' && c2 <= '9') {
kcc_lex_scan_number(); return; }
    }
    kcc_lex_scan_punct();
        return;
    }
    kcc_lex_scan_punct();
}

void kcc_lex_dump_token(void) {
    kcc_io_putu(kcc_tok_line);
    kcc_io_putc(':');
    kcc_io_putu(kcc_tok_col);
    kcc_io_putc(' ');
    kcc_io_puts(kcc_lex_type_name(kcc_tok_type));
    kcc_io_putc(' ');
    kcc_io_putc('\'');
    kcc_io_write(kcc_tok_start, kcc_tok_len);
    kcc_io_putc('\'');
    kcc_io_putc('\n');
}

void kcc_lex_dump_all(void) {
    for (;;) {
        kcc_lex_next();
        if (kcc_tok_type == TOK_EOF) break;
        kcc_lex_dump_token();
    }
}
□
```

911 · used in 37d

77.2. Verification

⟨ *Selfhost: lexcore verify* 914 ⟩ ≡

```
#!/usr/bin/env bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# make selfhost-lexer-core - compile the compiler'
   s lexer in C with kcc, relink
# in place of the assembly lexer.o, and run
   difftest + diagtest + codegen.  The
# token stream drives everything, so a byte-exact
   difftest proves the scan.
set -euo pipefail
SDK=$(xcrun --show-sdk-path)
SH=$BASE/selfhost
sources="main syscalls arena strings io hashtab
   vec strtab source lexer \
lexer_tables diag preprocessor parser expressions
   statements declarations \
types symtab conversions sema ir opt emit_arm64
   emit_x86 linker codegen emit_macho dwarf driver
   "
echo "[1/4] compiling $BASE/selfhost/lexer_core.c
   with kcc"
$BASE/kcc -c "$SH/lexer_core.c" -o "$SH/
   lexer_core_c.o" >/dev/null
echo "[2/4] linking $BASE/kcc_selfhost_lexer (
   lexer.o -> C)"
OBJ=""
for f in $sources; do
    if [ "$f" = "lexer" ]; then OBJ="$OBJ $SH/
   lexer_core_c.o"; else OBJ="$OBJ $BASE/src/${f}.
   o"; fi
done
ld -o $BASE/kcc_selfhost_lexer $OBJ -lSystem -
   syslibroot "$SDK" -e _main -arch arm64
cp $BASE/kcc "$SH/kcc_asm_backup_lexer"
cp $BASE/kcc_selfhost_lexer $BASE/kcc
echo "[3/4] difftest + diagtest"
set +e
bash $BASE/tests/difftest.sh >/$BASE/
   lexer_difftest.log 2>&1; dstatus=$?
```

```
tail -1 /$BASE/lexer_difftest.log
bash $BASE/tests/diagtest.sh >/$BASE/lexer_diag.
   log 2>&1; diagst=$?
tail -1 /$BASE/lexer_diag.log
set -e
if [ $dstatus -ne 0 ] || [ $diagst -ne 0 ]; then
   cp "$SH/kcc_asm_backup_lexer" $BASE/kcc; exit
   1; fi

echo "[4/4] codegen suite with the self-hosted
   lexer"
set +e
bash $BASE/tests/test_codegen.sh
status=$?
cp "$SH/kcc_asm_backup_lexer" $BASE/kcc
exit $status
```

□

914 · used in 37e · uses _main 621, 629

78. Differential Testing

The code-generation suite of Chapter 47 checks each program against an exit code computed by hand. That is a hard ceiling: a test can only be as complex as a value one is willing to work out on paper. Differential testing lifts the ceiling. It compiles each program with kcc *and* with `clang`, runs both, and compares—using `clang` as an oracle for what the program *should* do. No expected value is written down; the reference compiler supplies it. This is how the self-hosting tokenizer of Chapter 77 was validated, generalised here into a standing suite.

The programs are freestanding: each is an `int main(void)` whose return value becomes the process exit code, linked with entry `_main` and no C runtime—the same shape kcc's own tests use, and one `clang` honours too. Because an exit code is a single byte, programs return small results (or reduce modulo a small number); a genuine miscompilation practically never lands on an exact multiple of 256 away from the correct answer, so agreement on the byte is agreement in fact.

Each program lands in one of three buckets. **ok**: both compilers agree. **mismatch**: both compile and run but disagree—a code-generation defect, the bug this suite exists to catch. **gap**: `clang` compiles it but kcc cannot—a missing language feature, reported but not counted as a failure, since it maps the frontier of what kcc accepts rather than a wrong answer.

78.1. The Runner

The runner compiles with the oracle first: if `clang` rejects a program the program itself is wrong, and it is skipped. Otherwise it records `clang`'s exit code, then tries kcc. A kcc compile failure is a gap; a successful compile is linked, run, and its exit code compared against the oracle's.

⟨ *Difftest: runner* 917 ⟩ ≡

```
#!/bin/bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# kcc differential test suite (vs clang) -
   generated by weft, do not edit
set -u

PASS=0
FAIL=0
GAP=0
XFAIL=0
TOTAL=0
KCC="$BASE/kcc"
TMP="/$BASE/kcc_difftest_$$"
SDK=$(xcrun --show-sdk-path 2>/dev/null)

mkdir -p "$TMP"
trap 'rm -rf "$TMP"' EXIT

# Link an object freestanding and run it; echo the
    exit code, or "L" if the
# link fails (kcc emitted an object the system
   linker rejects).
link_run() {
    if ld "$1" -o "$2" -lSystem -syslibroot "$SDK"
    -e _main -arch arm64 2>/dev/null; then
        "$2" >/dev/null 2>&1
        echo $?
    else
        echo "L"
    fi
}

difftest() {
    local name="$1"
    local src="$2"
    TOTAL=$((TOTAL + 1))
    printf '%s\n' "$src" > "$TMP/d.c"

    # Oracle: clang must accept the program, else
   it is not valid C.
    if ! clang -c "$TMP/d.c" -o "$TMP/dc.o" 2>/dev
   /null; then
        printf "  %-28s SKIP (clang rejects)\n" "
   $name"
        return
    fi
    local crc
    crc=$(link_run "$TMP/dc.o" "$TMP/dc")

    # kcc: a compile failure is a feature gap, not
    a wrong answer.
    if ! $KCC -c "$TMP/d.c" -o "$TMP/dk.o" >/dev/
   null 2>&1; then
        GAP=$((GAP + 1))
        printf "  %-28s GAP (kcc cannot compile)\n
   " "$name"
        return
    fi
    local krc
    krc=$(link_run "$TMP/dk.o" "$TMP/dk")

    if [ "$krc" = "$crc" ]; then
        PASS=$((PASS + 1))
        printf "  %-28s ok (%s)\n" "$name" "$krc"
    else
        FAIL=$((FAIL + 1))
        printf "  %-28s MISMATCH kcc=%s clang=%s\n
   " "$name" "$krc" "$crc"
    fi
}

# Like difftest, but the divergence is a
   documented, expected limitation.
# It is reported but never fails the build.  If it
    unexpectedly agrees, the
# limitation has been resolved and the case should
    graduate to difftest.
difftest_xfail() {
    local name="$1"
    local src="$2"
    local reason="$3"
    TOTAL=$((TOTAL + 1))
    printf '%s\n' "$src" > "$TMP/d.c"
    if ! clang -c "$TMP/d.c" -o "$TMP/dc.o" 2>/dev
   /null; then
        printf "  %-28s SKIP (clang rejects)\n" "
   $name"
        return
    fi
    local crc
    crc=$(link_run "$TMP/dc.o" "$TMP/dc")
    if ! $KCC -c "$TMP/d.c" -o "$TMP/dk.o" >/dev/
   null 2>&1; then
```

```
        GAP=$((GAP + 1))
        printf "  %-28s GAP (kcc cannot compile)\n
   " "$name"
        return
    fi
    local krc
    krc=$(link_run "$TMP/dk.o" "$TMP/dk")
    if [ "$krc" = "$crc" ]; then
        printf "  %-28s XPASS (divergence resolved
   : %s)\n" "$name" "$reason"
    else
        XFAIL=$((XFAIL + 1))
        printf "  %-28s xfail: %s (kcc=%s clang=%s
   )\n" "$name" "$reason" "$krc" "$crc"
    fi
}

echo "kcc differential test suite (vs clang)"
echo "======================================"
```

□

917 · used in 37k · uses _main 621, 629

78.2. Arithmetic and Bit Manipulation

Integer arithmetic is where an all-spill backend's shuttling of values through scratch registers is most likely to drop or corrupt a value. These programs stress operator precedence, signed and unsigned division and remainder (well-defined truncation toward zero in C), shifts, and the bitwise operators—keeping every result in defined territory.

⟨ *Difftest: arithmetic* 918a ⟩ ≡

```
echo ""
echo "--- Arithmetic and Bits ---"

difftest "precedence-1" \
    "int main(void){ return 2 + 3 * 4 - 6 / 2; }"
difftest "precedence-2" \
    "int main(void){ return (2 + 3) * (4 - 1) % 7;
    }"
difftest "signed-div-neg" \
    "int main(void){ int a = -17; int b = 5;
   return 100 + a / b; }"
difftest "signed-mod-neg" \
    "int main(void){ int a = -17; int b = 5;
   return 100 + a % b; }"
difftest "unsigned-div" \
    "int main(void){ unsigned int a = 4000000000;
   unsigned int b = 7; return (int)(a / b % 200);
   }"
difftest "shifts" \
    "int main(void){ int a = 1; return (a << 5) +
   (256 >> 3); }"
difftest "bitwise-mix" \
    "int main(void){ int a = 0xF0; int b = 0x3C;
   return (a & b) | (a ^ b); }"
difftest "chained-cmp" \
    "int main(void){ int a = 5; int b = 7; int c =
    5; return (a < b) + (a == c) + (b > c); }"
difftest "unary-neg-not" \
    "int main(void){ int a = 5; return (-a + 10) +
    (~a & 7) + (!0); }"
```

□

918a · used in 37k

78.3. Control Flow

Branch lowering, loop back-edges, and `switch` dispatch are exercised here: nested conditionals, each loop form, fall-through, and `break`/`continue`.

⟨ *Difftest: control* 918b ⟩ ≡

```
echo ""
echo "--- Control Flow ---"

difftest "nested-if" \
    "int main(void){ int x = 7; if (x > 5) { if (x
    < 10) return 1; return 2; } return 3; }"
difftest "while-sum" \
    "int main(void){ int s = 0; int i = 1; while (
   i <= 10) { s = s + i; i = i + 1; } return s; }"
difftest "do-while" \
    "int main(void){ int n = 0; int i = 0; do { n
   = n + 2; i = i + 1; } while (i < 6); return n;
   }"
difftest "for-product" \
    "int main(void){ int p = 1; int i = 1; for (i
   = 1; i <= 5; i = i + 1) p = p * i; return p %
   200; }"
difftest "switch-fallthrough" \
    "int main(void){ int x = 2; int r = 0; switch
   (x) { case 1: r = r + 1; case 2: r = r + 10;
   case 3: r = r + 100; break; default: r = 255; }
    return r; }"
difftest "switch-case-range" \
    "int f(int x){ switch (x) { case 0 ... 7:
   return 1; case 10 ... 20: return 2; default:
   return 0; } } int main(void){ return f(3)*100 +
    f(15)*10 + f(8) + f(20); }"
difftest "switch-case-range-char" \
    "int cls(char c){ switch (c) { case '0' ...
   '9': return 1; case 'a' ... 'z': case 'A' ... '
   Z': return 2; case '_': return 3; default:
   return 0; } } int main(void){ return cls('5')
   *100 + cls('Q')*10 + cls('_') + cls('#')*9; }"
difftest "break-continue" \
    "int main(void){ int s = 0; int i = 0; while (
   i < 20) { i = i + 1; if (i == 5) continue; if (
   i == 12) break; s = s + i; } return s; }"
difftest "nested-loops" \
    "int main(void){ int c = 0; int i = 0; for (i
   = 0; i < 5; i = i + 1) { int j = 0; for (j = 0;
    j < 5; j = j + 1) c = c + 1; } return c; }"
# A local declared inside a switch block, whose
   address escapes, must have its
# storage reserved in the entry block: dispatching
    straight to a later case
# (here case 2) enters the block PAST the
   declaration.  When kcc emitted the
# alloca at the declaration point, the direct
   entry left the address vreg
# undefined and stores through it crashed -- this
   is exactly SQLite's
# OP_OpenRead handler, whose locals are declared
   at case OP_ReopenIdx.
difftest "switch-into-scope-addr-local" \
    "int sink; int probe(int w, int v){ int r =
   -1; switch (w) { case 1: { int acc; sink = (int
   )(long)&acc; acc = 111; if (acc == v) { r = acc
   ; break; } case 2: acc = v; r = acc; } } return
    r; } int main(void){ return (probe(2, 55) +
   probe(1, 111)) & 255; }"
```

□

918b · used in 37k

78.4. Functions and Recursion

Calling convention, argument passing, and return values are exercised by recursion (single and mutual) and multi-argument calls.

⟨ *Difftest: functions* 919a ⟩ ≡

```
echo ""
echo "--- Functions and Recursion ---"

difftest "factorial" \
    "int fact(int n){ if (n <= 1) return 1; return
    n * fact(n - 1); } int main(void){ return fact
   (5) % 200; }"
difftest "fib" \
    "int fib(int n){ if (n < 2) return n; return
   fib(n-1) + fib(n-2); } int main(void){ return
   fib(11); }"
difftest "mutual-rec" \
    "int is_even(int n); int is_odd(int n){ if (n
   == 0) return 0; return is_even(n - 1); } int
   is_even(int n){ if (n == 0) return 1; return
   is_odd(n - 1); } int main(void){ return is_even
   (10) + is_odd(7); }"
difftest "many-args" \
    "int sum6(int a, int b, int c, int d, int e,
   int f){ return a + b + c + d + e + f; } int
   main(void){ return sum6(1, 2, 3, 4, 5, 6); }"
difftest "nine-args" \
    "int s(int a,int b,int c,int d,int e,int f,int
    g,int h,int i){ return a+b+c+d+e+f+g+h+i; }
   int main(void){ return s(1,2,3,4,5,6,7,8,9); }"
difftest "twelve-args" \
    "int s(int a,int b,int c,int d,int e,int f,int
    g,int h,int i,int j,int k,int l){ return a+b+c
   +d+e+f+g+h+i+j+k+l; } int main(void){ return s
   (1,2,3,4,5,6,7,8,9,10,11,12); }"
difftest "stack-args-positional" \
    "int f(int a,int b,int c,int d,int e,int g,int
    h,int i,int j,int k){ return a*100 + j*10 + k;
    } int main(void){ return f
   (1,2,3,4,5,6,7,8,9,10); }"
difftest "stack-args-recursive" \
    "int s(int n,int a,int b,int c,int d,int e,int
    f,int g,int h){ if (n == 0) return a+b+c+d+e+f
   +g+h; return s(n-1,a,b,c,d,e,f,g,h); } int main
   (void){ return s(3,1,2,3,4,5,6,7,8); }"
difftest "vararg-sum" \
    "#include <stdarg.h>
int sum(int n, ...){ va_list ap; va_start(ap, n);
   int s = 0; int i = 0; for (i = 0; i < n; i = i
   + 1) s = s + va_arg(ap, int); va_end(ap);
   return s; } int main(void){ return sum(4, 10,
   20, 30, 40); }"
difftest "vararg-max" \
    "#include <stdarg.h>
int maxv(int n, ...){ va_list ap; va_start(ap, n);
    int m = va_arg(ap, int); int i = 1; for (i =
   1; i < n; i = i + 1){ int x = va_arg(ap, int);
   if (x > m) m = x; } va_end(ap); return m; } int
    main(void){ return maxv(5, 3, 17, 9, 42, 11);
   }"
difftest "vararg-two-fixed" \
    "#include <stdarg.h>
int f(int a, int b, ...){ va_list ap; va_start(ap,
    b); int s = a + b; int i = 0; for (i = 0; i <
   3; i = i + 1) s = s + va_arg(ap, int); va_end(
   ap); return s; } int main(void){ return f(1, 2,
    10, 20, 30); }"
difftest "vararg-three-fixed" \
    "#include <stdarg.h>
int g(int a, int b, int c, ...){ va_list ap;
   va_start(ap, c); int s = a + b + c; int i = 0;
   for (i = 0; i < 4; i = i + 1) s = s + va_arg(ap
   , int); va_end(ap); return s; } int main(void){
    return g(1, 2, 3, 4, 5, 6, 7); }"
difftest "vararg-builtin-spelling" \
    "typedef __builtin_va_list va_list;
int sum(int n, ...){ va_list ap;
   __builtin_va_start(ap, n); int s = 0; int i =
   0; for (i = 0; i < n; i = i + 1) s = s +
   __builtin_va_arg(ap, int); __builtin_va_end(ap)
   ; return s; } int main(void){ return sum(4, 10,
    20, 30, 40); }"
difftest "accumulate" \
    "int step(int acc, int x){ return acc + x * x;
    } int main(void){ int acc = 0; int i = 1;
   while (i <= 6) { acc = step(acc, i); i = i + 1;
    } return acc % 250; }"
difftest "regalloc-call-crossing" \
    "int g(int x){ return x * 2 + 1; } int main(
   void){ int a = 17, b = 23; int t = a * b + 5;
   int u = g(a) + g(b); int v = t - u + a * b;
   return (t + u + v) % 250; }"
```

□

919a · used in 37k

78.5. Pointers, Arrays, and Structs

Aggregate access and pointer arithmetic—subscripting, member access through pointers, address-of, and passing addresses to functions—are the patterns most sensitive to the address-computation paths.

⟨ *Difftest: aggregates* 919b ⟩ ≡

```
echo ""
echo "--- Pointers, Arrays, Structs ---"

difftest "array-sum" \
    "int main(void){ int a[6]; int i = 0; for (i =
    0; i < 6; i = i + 1) a[i] = i * i; int s = 0;
   for (i = 0; i < 6; i = i + 1) s = s + a[i];
   return s; }"
difftest "array-init-list" \
    "int main(void){ int a[5] = {3, 1, 4, 1, 5};
   return a[0] + a[2] + a[4]; }"
difftest "pointer-swap" \
    "void swap(int *a, int *b){ int t = *a; *a = *
   b; *b = t; } int main(void){ int x = 10; int y
   = 3; swap(&x, &y); return x - y; }"
difftest "struct-fields" \
    "struct P { int x; int y; }; int main(void){
   struct P p; p.x = 12; p.y = 30; return p.x + p.
   y; }"
difftest "struct-ptr" \
    "struct P { int x; int y; }; int dist(struct P
    *p){ return p->x + p->y; } int main(void){
   struct P p; p.x = 40; p.y = 2; return dist(&p);
    }"
difftest "struct-array" \
    "struct P { int a; int b; }; int main(void){
   struct P v[3]; int i = 0; for (i = 0; i < 3; i
   = i + 1) { v[i].a = i; v[i].b = i * 10; }
   return v[2].a + v[2].b; }"
difftest "addr-of-elem" \
```

```
    "void set(int *p, int v){ *p = v; } int main(
   void){ int a[4]; int i = 0; for (i = 0; i < 4;
   i = i + 1) set(&a[i], i + 1); return a[0] + a
   [1] + a[2] + a[3]; }"
difftest "ptr-to-ptr" \
    "int main(void){ int x = 42; int *p = &x; int
   **pp = &p; **pp = 7; return x; }"
difftest "array-2d" \
    "int main(void){ int a[3][3]; int i; int j;
   for (i = 0; i < 3; i = i + 1) for (j = 0; j <
   3; j = j + 1) a[i][j] = i * 3 + j; return a
   [2][2]; }"
# A struct/union-VALUE element in a brace
   initializer is a whole-object copy,
# not brace elision: `struct A a = {2, b}` copies
   all of the struct b into a.b.
# Elision would splice b's fields one scalar at a
   time and drop everything past
# the first, so a nested member came back zero.
# A 16-byte struct passed BY VALUE consumes two
   argument registers, so with
# enough following scalar args the last one spills
    to the stack.  The callee's
# overflow read must key off the argument index,
   not its running register-slot
# count (which counts the struct as two), or it
   reads the wrong stack word.
difftest "struct-byval-arg-spill" \
    "struct s { int f[4]; }; int foo(struct s s,
   int a, int b, int c, int d, int e, int g){
   return s.f[3] + g; } int main(void){ struct s s
    = {1,2,3,4}; return (foo(s, 10,20,30,40,50,
   70) ) & 255; }"

# A local union is as large as its widest member;
   initializing it must allocate
# and zero the WHOLE object, not just the first
   member.  A `union { int i; int
# j[4]; } u = {0}` left j[2]/j[3] (past the first
   member) as stack garbage.
difftest "union-init-zeroes-tail" \
    "union U { int i; int j[4]; }; int main(void){
    union U u = {0}; return u.j[0] + u.j[1] + u.j
   [2] + u.j[3]; }"
difftest "union-init-empty" \
    "union U { int i; int j[4]; }; int main(void){
    union U u = {}; int s = 0; for (int k = 0; k <
    4; k++) s += u.j[k]; return s; }"
difftest "init-struct-value-element" \
    "struct B { int x; int y; }; struct A { int z;
    struct B b; }; int main(void){ struct B b =
   {4,5}; struct A a = {2, b}; return a.z*100 + a.
   b.x*10 + a.b.y; }"
difftest "init-array-of-struct-values" \
    "struct P { int a; int b; }; int main(void){
   struct P p = {3,4}; struct P v[2] = {p, {5,6}};
    return v[0].a*1000 + v[0].b*100 + v[1].a*10 +
   v[1].b; }"
# An unnamed bit-field (int :4) reserves layout
   bits but takes NO initializer
# (C17 6.7.9): {2,3,4} must fill a, b, c -- not
   the anonymous padding member.
difftest "bitfield-init-skip-anon" \
    "struct { int a:4; int :4; int b:4; int c:4; }
    x = { 2,3,4 }; int main(void){ return x.a*100
   + x.b*10 + x.c; }"
# ++/-- on a bit-field must go through the shift-
   and-mask RMW, not a container-
# wide +/-1 (which would hit the field at bit
   offset 0). `--b.j` must decrement
# j, not i.
difftest "bitfield-inc-dec" \
    "struct { unsigned int i:6, j:11, k:15; } b;
   int main(void){ b.i=51; b.j=636; b.k=31278; b.i
   ++; --b.j; b.k++; return (b.i==52) && (b.j
   ==635) && (b.k==31279); }"
# A zero-size aggregate element (empty struct
   member from a call) copies no
# bytes, but the call's side effect must still run
    (C17 6.5.2.2/6.7.9).
difftest "zero-size-struct-init-side-effect" \
    "int ii; typedef struct {} R; typedef struct {
    R raw; } S; R mk(void){ R r; ii++; return r; }
    int main(void){ S lock = (S){ .raw = mk() };
   return ii; }"

# A LOCAL aggregate initialized with an array-
   valued element must decay the
# array to its address to fill a pointer member (`
   S s = {arr, ...}`).  Sema did
# not type list elements, so the array element was
    loaded as an int and the
# pointer member pointed at garbage.  Covers plain
   , designated, and array-of-ptr.
difftest "local-init-array-decay" \
    "struct S { int *p; int n; }; int main(void){
   int a[3] = {10,20,30}; struct S s = {a, 3};
   return s.p[0] + s.p[2] + s.n; }"
difftest "local-init-array-decay-designated" \
    "struct S { int n; int *p; }; int main(void){
   int a[2] = {5,6}; struct S s = {.p = a}; s.p[0]
    = 42; return a[0] + s.n * 0; }"
difftest "local-init-array-of-ptrs" \
    "int main(void){ int a[2] = {7,8}; int b[2] =
   {9,10}; int *pa[2] = {a, b}; return pa[0][1] +
   pa[1][0]; }"
# A char-ARRAY struct member (or a row of a 2-D
   char array) initialized with a
# string literal copies the string's BYTES, not
   its address (which is what a
# scalar store would leave, as for a char* member)
   .
difftest "struct-char-array-member-string" \
    "struct A { char c; char p[3]; } a = { 'v', \"
   cq\" }; int main(void){ return a.c==( 'v') && a
   .p[0]=='c' && a.p[1]=='q' && a.p[2]==0; }"
difftest "char-array-2d-string-rows" \
    "char m[2][3] = { \"ab\", \"cd\" }; int main(
   void){ return m[0][0]=='a' && m[0][1]=='b' && m
   [1][0]=='c' && m[1][1]=='d'; }"
# A trailing flexible array member initialized
   with a string grows the object's
# storage and copies the string's bytes into the
   tail.
difftest "flexible-array-member-string" \
    "struct S { char c; char p[]; } a = { 'o', \"
   wx\" }; int main(void){ return a.c=='o' && a.p
   [0]=='w' && a.p[1]=='x' && a.p[2]==0; }"
difftest "array-2d-nonsquare" \
    "int main(void){ int a[2][5]; int i; int j;
   for (i = 0; i < 2; i = i + 1) for (j = 0; j <
   5; j = j + 1) a[i][j] = i * 5 + j; return a
   [1][4]; }"
difftest "array-3d" \
    "int main(void){ int a[2][2][2]; a[1][1][1] =
   42; a[0][1][0] = 5; return a[1][1][1] + a
   [0][1][0]; }"
difftest "matrix-multiply" \
    "int main(void){ int a[2][2]; int b[2][2]; a
   [0][0]=1; a[0][1]=2; a[1][0]=3; a[1][1]=4; b
   [0][0]=5; b[0][1]=6; b[1][0]=7; b[1][1]=8; int
   c = a[0][0]*b[0][0] + a[0][1]*b[1][0]; return c
   ; }"
```

```
difftest "sizeof-2d-array" \
    "int main(void){ int a[3][4]; return (int)(
   sizeof(a) / sizeof(a[0][0])); }"
difftest "sret-basic" \
    "struct P { int x; int y; }; struct P make(int
    a, int b){ struct P p; p.x = a; p.y = b;
   return p; } int main(void){ struct P q = make
   (15, 27); return q.x + q.y; }"
difftest "sret-zero-arg" \
    "struct P { int x; int y; }; struct P make(
   void){ struct P p; p.x = 10; p.y = 20; return p
   ; } int main(void){ struct P q = make(); return
    q.x + q.y; }"
difftest "sret-three-fields" \
    "struct P { int a; int b; int c; }; struct P
   make(void){ struct P p; p.a = 1; p.b = 2; p.c =
    3; return p; } int main(void){ struct P q =
   make(); return q.a + q.b + q.c; }"
difftest "sret-assign" \
    "struct P { int x; int y; }; struct P make(int
    a){ struct P p; p.x = a; p.y = a * 2; return p
   ; } int main(void){ struct P q; q = make(7);
   return q.x + q.y; }"
difftest "sret-assign-global" \
    "struct P { int x; int y; }; struct P g;
   struct P make(int a){ struct P p; p.x = a; p.y
   = a * 3; return p; } int main(void){ g = make
   (5); return g.x + g.y; }"
difftest "sret-member-of-temp" \
    "struct P { int x; int y; }; struct P make(
   void){ struct P p; p.x = 5; p.y = 9; return p;
   } int main(void){ return make().y; }"
difftest "sret-passthrough" \
    "struct P { int x; int y; }; struct P mk(int a
   ){ struct P p; p.x = a; p.y = a + 1; return p;
   } struct P wrap(int a){ return mk(a); } int
   main(void){ struct P q = wrap(20); return q.x +
    q.y; }"
difftest "sret-five-args" \
    "struct P { int x; int y; }; struct P make(int
    a, int b, int c, int d, int e){ struct P p; p.
   x = a + b + c; p.y = d + e; return p; } int
   main(void){ struct P q = make(1, 2, 3, 4, 5);
   return q.x + q.y; }"
difftest "sarg-basic" \
    "struct P { int x; int y; }; int sum(struct P
   p){ return p.x + p.y; } int main(void){ struct
   P q; q.x = 3; q.y = 4; return sum(q); }"
difftest "sarg-value-isolation" \
    "struct P { int x; int y; }; int f(struct P p)
   { p.x = 99; return p.x + p.y; } int main(void){
    struct P q; q.x = 3; q.y = 4; int r = f(q);
   return r + q.x; }"
difftest "sarg-three-fields" \
    "struct P { int a; int b; int c; }; int sum(
   struct P p){ return p.a + p.b + p.c; } int main
   (void){ struct P q; q.a = 1; q.b = 2; q.c = 3;
   return sum(q); }"
difftest "sarg-two-structs" \
    "struct P { int x; int y; }; int add(struct P
   a, struct P b){ return a.x + a.y + b.x + b.y; }
    int main(void){ struct P p; p.x = 1; p.y = 2;
   struct P q; q.x = 3; q.y = 4; return add(p, q);
    }"
difftest "sarg-mixed" \
    "struct P { int x; int y; }; int f(int a,
   struct P p, int b){ return a + p.x + p.y + b; }
    int main(void){ struct P q; q.x = 5; q.y = 6;
   return f(10, q, 20); }"
difftest "sarg-pass-and-return" \
    "struct P { int x; int y; }; struct P dbl(
   struct P p){ struct P r; r.x = p.x * 2; r.y = p
   .y * 2; return r; } int main(void){ struct P q;
    q.x = 3; q.y = 4; struct P d = dbl(q); return
   d.x + d.y; }"
difftest "sarg-temp" \
    "struct P { int x; int y; }; struct P mk(int a
   ){ struct P p; p.x = a; p.y = a + 1; return p;
   } int use(struct P p){ return p.x + p.y; } int
   main(void){ return use(mk(10)); }"
```

□

919b · used in 37k

The floating-point cases exercise the `double` pipeline end to end (§22.10): compile-time constant evaluation, the four arithmetic operations, ordered comparisons, and conversions in both directions, including through variables, globals, arrays, loops, and function boundaries. Every observable value is an `int` (the exit code), so the truncating `(int)` cast is always the final step.

78.6. Struct Layout

These cases pin down the byte-accurate aggregate layout (§A.12). `clang` is the oracle for `sizeof` and alignment, so a struct whose size is padded to the wrong boundary—or a field read at the wrong offset or width—diverges immediately. They cover padding between members of different sizes, the final tail padding that rounds a struct up to its alignment, arrays of odd-sized structs (whose stride is not a multiple of eight), signed narrow fields that must sign-extend on load, pointer arithmetic that must step by the real element size, and nested aggregates.

⟨ *Difftest: struct layout* 921 ⟩ ≡

```
echo ""
echo "--- Struct Layout ---"

difftest "sizeof-struct-2int" \
    "struct P { int x; int y; }; int main(void){
   return sizeof(struct P); }"
difftest "sizeof-struct-3int" \
    "struct P { int a; int b; int c; }; int main(
   void){ return sizeof(struct P); }"
difftest "sizeof-struct-char-int" \
    "struct P { char c; int x; }; int main(void){
   return sizeof(struct P); }"
difftest "sizeof-struct-mixed" \
    "struct P { char a; char b; int c; char d; };
   int main(void){ return sizeof(struct P); }"
difftest "sizeof-struct-short" \
    "struct P { short a; char b; }; int main(void)
   { return sizeof(struct P); }"
difftest "struct-pad-access" \
    "struct P { char c; int x; }; int main(void){
   struct P p; p.c = 7; p.x = 100; return p.c + p.
   x; }"
difftest "struct-char-signed-field" \
    "struct P { char c; int x; }; int main(void){
   struct P p; p.c = -1; p.x = 5; return p.c < 0 ?
    42 : 0; }"
difftest "struct-short-field-sum" \
    "struct P { short a; short b; int c; }; int
   main(void){ struct P p; p.a = 300; p.b = 12; p.
```

```
   c = 7; return p.a + p.b + p.c; }"
difftest "struct-array-oddsize" \
    "struct P { int a; int b; int c; }; int main(
   void){ struct P v[3]; int i = 0; for (i = 0; i
   < 3; i = i + 1){ v[i].a = i; v[i].b = i + 1; v[
   i].c = i + 2; } return v[2].a + v[2].b + v[2].c
    + sizeof(v); }"
# Arrow access on a global ARRAY (`g->m` == `g[0].
   m`): the array name is already
# the element-0 address, so member access must not
    load it as a pointer.  A
# static initializer taking the address of such a
   member exercises the same path.
difftest "global-array-arrow" \
    "struct F { int a; int b; }; struct F g[2] =
   {{5,6},{7,8}}; int main(void){ return g->b +
   (&(g->a))[0]; }"
difftest "global-array-member-addr-init" \
    "struct F { int id; char *n; }; struct F t[] =
    {{1,\"a\"},{2,\"b\"}}; int *q = (int *)&((t+1)
   ->id); int *r = (int *)&((t)->id); int main(
   void){ return *q * 10 + *r; }"
difftest "struct-array-stride-ptrdiff" \
    "struct P { int a; int b; int c; }; int main(
   void){ struct P v[4]; return (int)(&v[3] - &v
   [0]); }"
difftest "struct-nested-layout" \
    "struct In { int a; int b; }; struct Out { int
    x; struct In in; int y; }; int main(void){
   struct Out o; o.x = 1; o.in.a = 2; o.in.b = 3;
   o.y = 4; return o.x + o.in.a + o.in.b + o.y +
   sizeof(struct Out); }"
difftest "sizeof-int-array" \
    "int main(void){ int a[7]; return sizeof(a); }
   "
difftest "sizeof-char-array" \
    "int main(void){ char a[10]; return sizeof(a);
    }"
difftest "char-array-packing" \
    "int main(void){ char s[4]; s[0] = 65; s[1] =
   66; s[2] = 67; s[3] = 0; return s[0] + s[1] + s
   [2] + sizeof(s); }"
difftest "int-ptr-stride" \
    "int main(void){ int a[5]; int *p = a; int i =
    0; for (i = 0; i < 5; i = i + 1) *(p + i) = i
   * 2; return a[4] + (int)(&a[4] - a); }"
difftest "union-alias" \
    "union U { int i; int j; }; int main(void){
   union U u; u.i = 42; return u.j; }"
difftest "union-overlap" \
    "union U { int a; char b; }; int main(void){
   union U u; u.a = 300; return u.b; }"
difftest "union-array-member" \
    "union U { int i; int arr[2]; }; int main(void
   ){ union U u; u.arr[0] = 5; u.arr[1] = 9;
   return u.i; }"
difftest "struct-array-member" \
    "struct S { int n; int a[3]; }; int main(void)
   { struct S s; s.a[0]=1; s.a[1]=2; s.a[2]=3;
   return s.a[0]+s.a[1]+s.a[2]; }"
difftest "union-in-struct" \
    "struct S { int tag; union { int i; char c; }
   u; }; int main(void){ struct S s; s.tag = 1; s.
   u.i = 7; return s.tag + s.u.i; }"
difftest "char-sign-extend" \
    "int main(void){ char c = (char)200; int x = (
   int)c; return x < 0 ? 1 : 0; }"
difftest "signed-char-neg" \
    "int main(void){ signed char c = -5; int i = c
   ; return i + 10; }"
difftest "short-narrow" \
    "int main(void){ short s = 1000; s = s + 100;
   return s; }"
```

□

921 · used in 37k

⟨ *Difftest: floating point* 922 ⟩ ≡

```
echo ""
echo "--- Floating Point ---"

difftest "fp-const-arith" \
    "int main(void){ double a = 1.5; double b =
   2.25; return (int)((a + b) * 2); }"
difftest "fp-sub-scale" \
    "int main(void){ double a = 3.5, b = 1.25;
   return (int)((a - b) * 100); }"
difftest "fp-div" \
    "int main(void){ double a = 10.0, b = 4.0;
   return (int)(a / b * 10); }"
difftest "fp-int-to-double" \
    "int main(void){ int n = 7; double d = n;
   return (int)(d * 2); }"
difftest "fp-double-to-int" \
    "int main(void){ double d = 9.87; return (int)
   d; }"
difftest "fp-mixed-promote" \
    "int main(void){ int n = 3; double r = n +
   1.5; return (int)(r * 2); }"
difftest "fp-compare" \
    "int main(void){ double a = 1.1; double b =
   2.2; return a < b ? 10 : 20; }"
difftest "fp-assign-convert" \
    "int main(void){ double d; int n = 5; d = n;
   int i; double e = 7.9; i = e; return (int)(d *
   3) + i; }"
difftest "fp-global" \
    "double g = 3.5; int main(void){ g = g + 1.5;
   return (int)g; }"
difftest "fp-array" \
    "int main(void){ double a[3]; a[0] = 1.5; a[1]
    = 2.5; a[2] = 3.0; return (int)(a[0] + a[1] +
   a[2]); }"
# A negative floating literal in an aggregate
   initializer negates in the FP
# unit; it once fell to the integer path and two's
   -complement-negated the double
# bit pattern (-4.0 became -1.0).
difftest "fp-array-neg-init" \
    "double m[3] = {1.5, -4.0, 2.0}; int main(void
   ){ return (int)((m[0] + m[1] + m[2]) * 100); }"
difftest "fp-float-array-neg-init" \
    "float m[2] = {-2.5f, 4.0f}; int main(void){
   return (int)((m[0] + m[1]) * 100); }"
difftest "fp-array-neg-mixed" \
    "double m[4] = {-1.0, 2.0, -3.0, 4.0}; int
   main(void){ double s=0; int i; for(i=0;i<4;i++)
    s+=m[i]; return (int)(s*10); }"
difftest "fp-loop-accumulate" \
    "int main(void){ double s = 0.0; int i; for (i
    = 0; i < 5; i = i + 1) s = s + 1.5; return (
   int)s; }"
difftest "fp-exponent-literal" \
    "int main(void){ double d = 1.5e3; return (int
   )(d / 100); }"
difftest "fp-function" \
    "int scale(double x){ return (int)(x * 2); }
   int main(void){ return scale(3.5); }"
difftest "fp-return" \
```

```
    "double half(double x){ return x / 2.0; } int
   main(void){ return (int)(half(9.0) * 2); }"
difftest "float-precision-literal" \
    "int main(void){ float f = 16777217.0f; return
    f == 16777216.0f ? 1 : 0; }"
difftest "float-precision-convert" \
    "int main(void){ double d = 16777217.0; float
   f = d; return f == 16777216.0f ? 1 : 0; }"
difftest "float-precision-cast" \
    "int main(void){ return (float)16777217.0 ==
   16777216.0f ? 1 : 0; }"
difftest "float-sizeof" \
    "int main(void){ return (int)(sizeof(float) +
   sizeof(1.5f)); }"
difftest "float-single-round-chain" \
    "int main(void){ volatile float a=1.0f,b=3.0f,
   c=7.0f; float r=a/b*c; union { float f;
   unsigned u; } v; v.f=r; return v.u & 0xFF; }"
difftest "float-single-round-sum" \
    "int main(void){ volatile float a=1.1f,c=2.2f,
   d=3.3f,e=0.7f; float r=a*c+d*e; union { float f
   ; unsigned u; } v; v.f=r; return v.u & 0xFF; }"
# Hexadecimal floating constants (\S\ref{sec:parse
   -float}, \stdref{6.4.4.2}):
# a hex mantissa scaled by a binary exponent.  0x1
   .fp1 = 3.875 exactly, so the
# reinterpreted low bit must match; a wrong parse
   drifts the mantissa.
difftest "hex-float-value" \
    "int main(void){ double x = 0x1.fp1; return x
   == 3.875 ? 7 : 0; }"
difftest "hex-float-bits" \
    "int main(void){ double x = 0x1.8p0; union {
   double d; unsigned long u; } v; v.d = x; return
    (int)(v.u >> 48); }"
# A decimal fraction that is exactly representable
    must parse to its exact
# double.  Digit-by-digit division drifts a ULP
   low (3.875 -> 0x400eff...ff);
# accumulating one integer mantissa and dividing
   once is correctly rounded.
difftest "decimal-exact-fraction" \
    "int main(void){ double x = 3.875; union {
   double d; unsigned long u; } v; v.d = x; return
    v.u == 0x400f000000000000UL ? 1 : 0; }"
# Logical NOT of a floating operand is `x == 0.0`,
    a FLOATING compare (FCMP),
# not an integer test of the raw bit pattern: !1.0
   f is 0, and 1.0f's bits
# (0x3F800000) are nonzero, so a raw-integer test
   came out backwards.
difftest "lognot-float-nonzero" \
    "int main(void){ float c = 1.0f; return !c; }"
difftest "lognot-float-zero" \
    "int main(void){ float c = 0.0f; return !c; }"
difftest "lognot-double-in-expr" \
    "int test(float c){ return !!c * 7LL == 0; }
   int main(void){ return test(1.0f); }"
difftest "float-arith" \
    "int main(void){ float f = 1.5f, g = 2.25f;
   return (int)((f + g) * 8); }"
difftest "float-func" \
    "float sq(float x){ return x * x; } int main(
   void){ return (int)sq(7.0f); }"
difftest "float-int-regalloc" \
    "int main(void){ int a = 3, b = 5, c = 7, d =
   11; int e = a*b + c*d - a*c + b*d; double x =
   3.5; int di = (int)(x * (a + b + c + d));
   return (e % 100) + (di % 100); }"
difftest "global-double-init-int" \
    "double g = 100; int main(void){ return (int)g
    + (g < 1); }"
difftest "global-float-init-int" \
    "float g = 5; int main(void){ return (int)(g *
    2); }"
difftest "global-double-read" \
    "double g = 2.5; int main(void){ return (int)(
   g * 4); }"
difftest "struct-double-member" \
    "struct S { double d; int t; }; struct S s = {
    3.5, 9 }; int main(void){ return (int)s.d + s.
   t; }"
```

□

922 · used in 37k

78.7. Operators and Edge Cases

The remaining operators—ternary, comma, compound assignment, increment/decrement, short-circuit logicals—and a few numeric edge cases round out the corpus.

⟨ *Difftest: operators* 923 ⟩ ≡

```
echo ""
echo "--- Operators and Edge Cases ---"

# Unsigned 64-bit -> floating point must use UCVTF
   , not SCVTF: a value with its
# top bit set (0xFFFF...FF) becomes ~1.8e19, not
   -1.
difftest "unsigned64-to-double" \
    "int main(void){ unsigned long long u = 0
   xFFFFFFFFFFFFFFFFULL; double d = (double)u;
   return d > 1.8e19 && d < 1.9e19; }"
difftest "unsigned-to-double-arith" \
    "int main(void){ unsigned long long u = 0
   x8000000000000000ULL; double d = u + 0.0;
   return d > 9.2e18 && d < 9.3e18; }"
# && and || yield the int 0 or 1 (C17
   6.5.13/6.5.14), never the raw operand:
# 4 && 2 is 1, not 2; 0 || 2 is 1, not 2.
difftest "logical-and-or-normalize" \
    "int main(void){ int c = 4; int e = 2; return
   (c && e) * 100 + (c || 0) * 10 + (0 || e); }"
# The value of a bit-field assignment is the
   TRUNCATED stored value, not the raw
# RHS: (signed int:11) = 1081 yields 1081-2048 =
   -967, so (l.m=j)==j is false.
difftest "bitfield-assign-result-trunc" \
    "struct { signed int m:11; } l; int main(void)
   { int j = 1081; return (l.m = j) == j; }"

# A narrow UNSIGNED operand promotes to signed int
   , so `/`, `%`, and `/=` on it
# are SIGNED even though the operand was unsigned:
    50 / (short)-5 == -10, not 0.
# The signedness must come from the promoted
   operation type, not the raw operand
# or the compound-assign lvalue.  Genuine unsigned
   -int division stays unsigned.
difftest "uchar-div-signed-promote" \
    "int main(void){ unsigned char x = 50; short y
    = -5; return (unsigned char)(x / y); }"
# An unsigned divide/remainder runs at the
   operation's own width. `int % unsigned`
# converts to unsigned INT (32-bit): -13 % 61u =
   44, not (2^64-13)%61. And an
# unsigned LONG remainder must stay 64-bit: (
   unsigned long)-13 % 61 = 3.
```

```
difftest "int-mod-unsigned-int-width" \
    "int main(void){ int i = -13; unsigned u = 61;
    return (int)(i % u); }"
difftest "unsigned-long-mod-width" \
    "int main(void){ unsigned long a = 0
   xFFFFFFFFFFFFFFF3UL, b = 61; return (int)(a % b
   ); }"
difftest "unsigned-int-div-negative" \
    "int main(void){ int i = -100; unsigned u = 7;
    return (int)(i / u); }"
difftest "uchar-divassign-signed-promote" \
    "int main(void){ unsigned char x = 50; short y
    = -5; x /= y; return x; }"
difftest "uchar-modassign-signed-promote" \
    "int main(void){ unsigned char x = 53; short y
    = -5; x %= y; return x; }"
# Bitwise AND/OR/XOR pass their operands' upper
   bits straight through, so mixing
# a sign-extended operand into an unsigned result
   must re-canonicalise (ZEXT) the
# result -- otherwise a later widen or 64-bit
   compare sees dirty upper bits.
# `0u ^ (short)-1` is the unsigned int 0xFFFFFFFF,
    positive when widened to long.
# A chained assignment yields, at each link, the
   source CONVERTED to that
# lvalue's type -- not the raw source.  `ul = us =
    l2 = -1` must narrow -1 to
# unsigned short (0xFFFF) before widening it to ul
   ; storing the raw -1 into all
# three left ul = -1.  The implicit int-to-int
   conversion must materialise.
difftest "chained-assign-narrowing" \
    "int main(void){ unsigned short us; long long
   l2; unsigned long long ul; short s2; ul = us =
   l2 = s2 = -1; return ul == 65535 ? 1 : 0; }"
difftest "chained-assign-truncate" \
    "int main(void){ int a; unsigned char c; a = c
    = 300; return a; }"
difftest "xor-mixed-width-unsigned" \
    "int main(void){ volatile short s = -1; return
    (0LL > (0u ^ s)) ? 1 : 0; }"
difftest "or-mixed-width-widen" \
    "int main(void){ short s = -1; long long v =
   (0u | s); return v > 0 ? 1 : 0; }"
difftest "uint-divassign-stays-unsigned" \
    "int main(void){ unsigned x = 50; unsigned s =
    4294967291u; x /= s; return (int)x; }"
# A struct-returning call REUSES its sret-buffer
   alloca as the call's DEST vreg;
# the linear-scan allocator must not treat that
   write as a fresh definition, or
# it re-allocates the vreg at the call site to a
   register freed since the alloca
# --- clobbering a still-live local (here `a`,
   needed as the next call's arg)
# whose address the alloca already emitted into
   the original register.  gcc.c-
# torture 20030613-1 in miniature: struct returned
   /copied through a temporary.
difftest "struct-return-regalloc-live-arg" \
    "struct S { long x, y; }; static struct S id(
   struct S s){ struct S r; r.x=s.x; r.y=s.y;
   return r; } static struct S pw(struct S s, int
   n){ struct S a=s; while(--n>0) a=id(a); return
   a; } int main(void){ struct S x; x.x=-7; x.y
   =-7; struct S a=pw(x,2); struct S b=id(pw(a,2))
   ; return b.x==b.y ? 0 : 1; }"
# A struct DEFINED together with a pointer
   declarator --- `struct S {...} *t;`,
# the ubiquitous `struct Node {...} *head;` form
   --- is still a definition of the
# tag.  If the tag is not registered (its layout
   computed), a later `struct S`
# resolves to a member-less descriptor, so field
   access and aggregate inits read
# and write the wrong offsets.  gcc.c-torture
   pr49419 turned on exactly this.
difftest "struct-def-with-pointer-declarator" \
    "struct S { int w, x, y; } *t; int main(void){
    struct S b[3] = { {1,1,2}, {0,0,0}, {0,0,0} };
    int *p = (int*)b; int s = 0; for (int i=0;i<9;
   i++) s = s*10 + p[i]; return s == 112000000 ? 0
    : 1; }"
# A nested block introduces a name scope: an inner
    declaration shadows an outer
# local of the same name only for the block, and
   the outer binding is restored
# on exit.  The IR builder's →namevreg map must be
    scoped, or the inner vreg
# leaks out and the outer read sees the inner
   value.
difftest "block-scope-shadow-restore" \
    "int main(void){ int v = 4; { int v = 5; if (v
    != 5) return 1; } return v == 4 ? 0 : 2; }"
# A block-scope `extern int v;` refers to the FILE
   -SCOPE object, not the
# enclosing local of the same name; the reference
   must lower to a global.
difftest "block-scope-extern-shadows-local" \
    "int v = 3; int main(void){ int v = 4; {
   extern int v; if (v != 3) return 1; } return v
   == 4 ? 0 : 2; }"
# Each brace-list element is assigned to its
   object (6.7.9), so an integer
# element of a double array must be CONVERTED (int
   ->double), not stored as raw
# integer bits.  `double a[] = {1,2,3}` must hold
   1.0/2.0/3.0.  gcc.c-torture
# pr42691 depended on this (`double table[2] = {
   inf, 23 }`).
difftest "double-array-int-element-convert" \
    "int main(void){ double a[3] = { 1, 2, 3 };
   int n = 5; double b[2] = { n, 7 }; return (a
   [0]==1.0 && a[2]==3.0 && b[0]==5.0 && b
   [1]==7.0) ? 0 : 1; }"
# `++`/`--` on a floating lvalue must step by 1.0
   through the FP unit, not do an
# integer +1 on the raw double bits (which turned
   0.0 into a denormal, so
# `for (double x=0; x<20; x++)` spun forever --
   gcc.c-torture 20000731-1).
difftest "float-inc-dec" \
    "int main(void){ double x = 0; int n = 0; for
   (x = 0; x < 20; x++) n++; float f = 2.5f; f++;
   double d = 10.0; double old = d--; return (n
   ==20 && x==20.0 && f==3.5f && old==10.0 && d
   ==9.0) ? 0 : 1; }"
# A <=16B struct-returning call unpacks its result
    into the DEST buffer AFTER
# the call.  When the result is DISCARDED, that
   buffer's address must still
# survive the call (callee-saved), or the post-
   call unpack stores through a
# clobbered pointer and faults.  gcc.c-torture
   920908-1 (a variadic function
# returning a struct, called for effect) crashed
   on exactly this.
difftest "discarded-struct-return-buffer-survives-
   call" \
    "#include <stdarg.h>
typedef struct { int a; } T; T f(int x, ...){
   va_list ap; T r; va_start(ap,x); r = va_arg(ap,
   T); va_end(ap); return r; } int main(void){ T v
   ; v.a = 42; f(1, v); return 0; }"
```

```
# When an integer and a `float` mix, the common type is float (6.3.1.8): the
# integer rounds to single precision before the operation.  `16777217L != (float)16777217e0` is FALSE because the long rounds to 16777216.0f, exactly
# as the float does.  Promoting the integer only to double (exact) made it
# compare unequal.  gcc.c-torture 920710-1.
difftest "int-float-common-type-rounds-to-single" \
    "int main(void){ return (16777217L != (float)16777217e0) ? 1 : 0; }"
# An empty struct/union (GNU extension) has size 0: returning one carries no
# value, so the callee must load nothing and the caller unpack nothing.  Loading
# from the 0-byte object's address dereferenced garbage and faulted.  gcc.c-
# torture pr23324 (a function returning `union {}`) crashed on exactly this.
difftest "empty-union-return" \
    "union E {}; static union E g = {}; union E f(int x){ (void)x; return g; } int main(void){ union E r; r = f(5); (void)r; return (int)sizeof(union E); }"
# A struct assignment whose right side is a comma (or parenthesised) expression
# still copies the struct: `g = (1, s)` copies `s`.  The struct-copy path
# dispatched on the RHS shape and matched no case for a comma/paren node, so the
# copy was silently skipped and the destination kept its old value.  Found by
# csmith differential testing (a silent miscompile, UB-free).
difftest "struct-assign-comma-rhs" \
    "struct S { unsigned short a; unsigned long long b; }; static struct S x = { 7, 0xAABBCCDDEEFF }, y = { 0, 0 }; int main(void){ y = (1, 2, x); return (y.a == 7 && y.b == 0xAABBCCDDEEFF) ? 0 : 1; }"
# A chained struct assignment `a = b = c` copies c into b and then b into a.  The
# struct-copy path had no case for an ASSIGN_EXPR right side, so the outer copy
# was silently skipped and `a` kept its old value.  Found by csmith differential
# testing (a silent miscompile, UB-free per MSan + ASan/UBSan).
difftest "struct-assign-chained" \
    "struct S { int a; long b; }; static struct S w = {0,0}, x = {0,0}, y = {0,0}, z = {9, 0x123456789}; int main(void){ w = x = y = z; return (w.a==9 && w.b==0x123456789 && x.a==9 && y.b==0x123456789) ? 0 : 1; }"
# A struct/union passed BY VALUE whose argument is a comma or nested assignment
# must copy the aggregate.  The argument-address builder had no case for a comma
# or assign-expr, so `f((x, s))` / `f((a = s))` read the aggregate loaded as a
# scalar and used it as an address -- a fault.  Found by csmith (bitfields on).
difftest "struct-arg-comma-assign" \
    "struct S { int a; long b; }; static struct S d = {0,0}, s = {5, 0x99}; static int t; static int take(struct S x){ return x.a + (int)x.b; } int main(void){ int r1 = take((t = 1, s)); int r2 = take((d = s)); return (r1 == 158 && r2 == 158 && d.a == 5) ? 0 : 1; }"
# GNU __attribute__((...)) (and __extension__, and the __x__ spellings of standard
# keywords) must be accepted wherever they appear, or real system headers do not
# parse.  kcc consumes and ignores them (layout attributes packed/aligned are a
# later refinement); the point here is that the surrounding declarations parse and
# run.  clang accepts the same, so this is a valid differential test.
difftest "gnu-attributes-parse" \
    "extern int f(int) __attribute__((const));
struct __attribute__((unused)) S { int x __attribute__((aligned(4))); };
__extension__ typedef long long ll;
static __inline__ __attribute__((always_inline)) int g(__const int x __attribute__((unused))){ return x + 1; }
int f(int y){ return y; }
int main(void){ struct S s; s.x = 41; ll v = 100; return (g(s.x) == 42 && f(0) == 0 && v == 100) ? 0 : 1; }"
# A GNU attribute between a pointer type and the declarator name (kernel headers'
# `char * __must_check f(...)` after __must_check expands).
difftest "gnu-attr-mid-declarator" \
    "char * __attribute__((warn_unused_result)) pick(char *a){ return a; }
int __attribute__((unused)) mid(void){ return 5; }
int main(void){ char b[3]; b[0] = 37; return (pick(b)[0] == 37 && mid() == 5) ? 0 : 1; }"
# GNU __builtin_types_compatible_p: a compile-time 0/1, plus a file-scope
# _Static_assert built from it (the kernel's __same_type sanity checks in
# bitops.h -- the single construct that stopped ~76% of a kernel .c sample).
difftest "builtin-types-compatible" \
    "int a; long b;
_Static_assert(__builtin_types_compatible_p(typeof(a), typeof(a)), \"same\");
_Static_assert(!__builtin_types_compatible_p(typeof(a), typeof(b)), \"diff\");
int main(void){ return __builtin_types_compatible_p(int, int)*100 + __builtin_types_compatible_p(int, long)*10 + __builtin_types_compatible_p(unsigned, unsigned); }"
# GNU `, ##__VA_ARGS__` comma elision: the trailing comma vanishes when the
# variadic argument is empty (the kernel's static_assert(expr) macro).
difftest "gnu-va-comma-elision" \
    "int add3(int a,int b,int c){ return a*100+b*10+c; }
#define M(a, ...) add3(a, ##__VA_ARGS__)
#define ONE(a, ...) sink(a, ##__VA_ARGS__)
int sink(int a){ return a; }
int main(void){ return M(1, 2, 3) - ONE(123) * 0 - 123; }"
# GNU __alignof__ / __alignof: aliases for _Alignof (kernel kstrtox.h).
difftest "gnu-alignof-spelling" \
    "_Static_assert(__alignof__(unsigned long) == __alignof__(unsigned long long), \"a\");
int main(void){ return (int)(__alignof__(long)*10 + __alignof(int)); }"
# GNU __builtin_offsetof (the kernel's offsetof): single, nested, and used in an
```

```
# integer context (kernel ptrace.h MAX_REG_OFFSET)
    .
difftest "gnu-builtin-offsetof" \
    "struct R { long a; long ss; int f; }; struct
    N { int x; struct R r; char arr[4]; };
int main(void){ return (int)(__builtin_offsetof(
    struct R, ss) + __builtin_offsetof(struct R, f)
     + __builtin_offsetof(struct N, r.ss) +
    __builtin_offsetof(struct N, arr[2])); }"
# A function call nested inside a __builtin_expect
    argument.  __builtin_expect is
# an undeclared bare name taken as an implicit
    function; the inner call must not
# clobber that "implicit callee" status (kernel
    likely()/unlikely() wrap calls
# like static_key_count(k) this way - jump_label.h
    and 13 other lib/ files).
difftest "builtin-expect-nested-call" \
    "static int f(int *p){ return *p; }
int main(void){ int x = 7, y = 0;
  if (__builtin_expect(!!(f(&x) > 0), 0)) y += 10;
  if (__builtin_expect(f(&x) == 7, 1)) y += 3;
  return (y == 13) ? 0 : 1; }"
# __FILE__ predefined macro (C17 6.10.8.1):
    expands to the current source path
# as a string literal.  The path itself differs
    between build environments, so
# the check is path-independent - the difftest
    harness compiles both kcc and
# clang on a file named d.c, so __FILE__ is a non-
    empty string ending in ".c".
# The kernel reaches __FILE__ through WARN()/BUG()
    /pr_* in ~half of all files.
difftest "builtin-file-macro" \
    "int main(void){ const char *f = __FILE__; int
    n = 0; while (f[n]) n++;
     return (n >= 2 && f[n-1] == 'c' && f[n-2] ==
    '.') ? 0 : 1; }"
# Constant-condition folding: an `if`, `?:`, or `
    __builtin_constant_p` whose
# controlling expression is a compile-time
    constant selects one arm, and only
# that arm is lowered (the kernel gates size-
    dispatched inline asm on exactly
# this).  __builtin_constant_p is 1 for a constant
    , 0 for a runtime value.
difftest "constant-condition-fold" \
    "int main(void){ int r = 0; int n = 3;
       if (1) r += 3; else r += 100;
       if (0) r += 100; else r += 4;
       r += (1 ? 5 : 100); r += (0 ? 100 : 6);
       r += __builtin_constant_p(42) ? 7 : 100;
       r += __builtin_constant_p(n) ? 100 : 8;
       return r - (3+4+5+6+7+8); }"
# Constant-switch folding: a switch on a compile-
    time-constant value selects a
# single arm (or default, or none), building only
    that arm with fall-through and
# break honoured.  The kernel's __raw_cmpxchg
    switches on a constant `sizeof`
# and each arm has a differently-sized cmpxchg;
    folding emits only the live one.
# Covers exact match, fall-through, default, a
    case-label chain, and no-match.
difftest "constant-switch-fold" \
    "int main(void){ int r = 0;
       switch (1) { case 1: r += 1; break; case 2:
    r += 100; break; default: r += 100; }
       switch (2) { case 1: r += 100; case 2: r +=
    20; case 3: r += 40; break; default: r += 100;
    }
       switch (9) { case 1: r += 100; break;
    default: r += 3; }
       switch (5) { case 4: case 5: r += 5; break;
     default: r += 100; }
       switch (7) { case 1: r += 100; break; }
       return r - (1+60+3+5); }"
# The # stringize operator (C17 6.10.3.2) keeps
    adjacent tokens adjacent: a
# single space appears only where the source had
    whitespace, none between tokens
# written together like `a.b` or `.section`.  The
    kernel's EXPORT_SYMBOL builds
# an `asm(".section ...")` template via
    __stringify, so `.section` splitting into
# `. section` makes the emitted asm unassemblable.
difftest "stringize-token-adjacency" \
    "#define S1(x) #x
#define S(x) S1(x)
int main(void){ const char *p = S(a.b); const char
    *q = S(a b);
  return (p[0]=='a' && p[1]=='.' && p[2]=='b' && p
    [3]==0 &&
          q[0]=='a' && q[1]==' ' && q[2]=='b' && q
    [3]==0) ? 0 : 1; }"
# A comment in a macro body must be removed before
    the body is stored (phase 3),
# or an apostrophe/quote inside it (as in the
    kernel's `interval_tree` doc
# comment "a node's interval") flips the argument
    substitutor into char/string
# mode and copies the rest of the body verbatim,
    leaving parameters and ## after
# it unexpanded.  Here the comment holds an odd
    apostrophe between two uses of X.
difftest "macro-body-comment-apostrophe" \
    "#define M(X) int X##_a(void){ return 0; } /*
    a node's interval: X can't be null */ int X##_b
    (void){ return 7; }
M(foo)
int main(void){ return foo_a() + foo_b() - 7; }"
# A single `typedef` may introduce several names,
    and a later declarator may be a
# pointer -- the kernel's Dhrystone `typedef
    struct {...} Rec_Type, *Rec_Pointer;`.
# Each name past the first must be registered as a
    typedef, not left to trip the
# next declaration.  Covers a struct typedef with
    a pointer second name and a
# plain `typedef int A, *B;`.
difftest "typedef-multiple-declarators" \
    "typedef struct rec { int x; } Rec, *RecP;
typedef int A, *B;
int main(void){ Rec r; r.x = 5; RecP p = &r; A a =
    3; B b = &a; return p->x - a + *b - 5; }"
# GNU __auto_type infers a variable's type from
    its initializer (`__auto_type x
# = e` means `typeof(e) x = e`).  The kernel's
    WARN/min/max macros use it, so it
# must parse as a declaration both at file scope
    and inside a block.  Covers
# integer, long (width from the initializer), and
    pointer inference.
difftest "auto-type" \
    "__auto_type g = 100;
int main(void){ __auto_type x = 5; __auto_type y =
    x + 2L; __auto_type z = &x;
  return (int)(x + y) - 12 + (*z - 5) + (g - 100);
    }"
# GNU array-range designator `[lo ... hi] = v`
    fills every element in the range
# (the kernel's lib/atomic64.c seeds its lock
    table `[0 ... (NR_LOCKS-1)]`), and
```

```
# a designator index is a constant EXPRESSION, not just a literal, so [NR-1] and
# the range bounds must be folded.  Covers a scalar range and a struct range.
difftest "designator-range-and-const-index" \
    "#define NR 4
struct s { int a; int b; };
static struct s g[NR] = { [0 ... (NR - 1)] = { .a = 7, .b = 3 } };
int main(void){ int v[4] = { [NR - 2] = 9 };
  int sum = v[2];
  for (int i = 0; i < NR; i++) sum += g[i].a + g[i].b;
  return (sum == 9 + 4*(7+3)) ? 0 : 1; }"
# A designator index may name an ENUM CONSTANT (`[NLA_U32] = ...` fills kernel
# validation tables).  An enum constant is not in the symbol table until sema,
# so the index cannot be folded during the parse - it is kept as an expression
# and evaluated when the initializer is lowered.  Covers a single enum index, an
# enum in a range bound, and enum-index array-length inference (`arr[]`).
difftest "enum-constant-designator-index" \
    "enum { E0, E1, E2, E3, ELAST };
static int arr[ELAST] = { [E0] = 10, [E2] = 30, [E1 ... E1] = 20 };
static int inf[] = { [E3] = 7 };
int main(void){
  if (arr[0]!=10 || arr[1]!=20 || arr[2]!=30 || arr[3]!=0) return 1;
  if (sizeof(inf)/sizeof(inf[0]) != 4 || inf[3]!=7) return 2;
  return 0; }"
# A block may open with a GNU local label declaration `__label__ name;` so a
# nested `goto` inside a statement-expression can name a block-local target.
# The kernel's wait_event/RCU macros expand to exactly this shape.  kcc scopes
# labels per function, so the declaration is consumed and the label binds
# through the ordinary goto machinery.  Covers a `__label__` at the head of a
# statement-expression with a forward goto to it.
difftest "local-label-declaration" \
    "int pick(int x){
       return ({ __label__ done; int r = 1; if (x > 0) goto done; r = 2; done: r; });
     }
     int main(void){ return (pick(5) == 1 && pick(-1) == 2) ? 0 : 1; }"
# An always_inline function is expanded at its call site with the arguments
# substituted for the parameters (the kernel marks static_cpu_has and many small
# accessors this way).  A `{ return EXPR; }` body inlines to EXPR; the result
# must match a plain call for constant and non-constant arguments alike, and a
# nested inline call must expand too.
difftest "always-inline-expr" \
    "static inline __attribute__((always_inline))
   int add1(int x){ return x + 1; }
     static inline __attribute__((always_inline))
   int dbl(int x){ return x + x; }
     static inline __attribute__((always_inline))
   int mix(int a,int b){ return add1(a) + dbl(b); }
     int main(void){ int y = 20;
       if (add1(41) != 42) return 1;
       if (dbl(y) != 40) return 2;
       if (mix(9, 3) != (9+1)+(3+3)) return 3;
       return 0; }"
# The integer constant-expression evaluator must handle relational and logical
# operators (done signed), or a constant condition like `if ((-5) < 0)' folds to
# false---which then broke always_inline bodies once a constant argument made the
# condition constant.
difftest "constant-comparison-fold" \
    "int main(void){
       if (((-5) < 0) ? 0 : 1) return 1;
       if (!(3 >= 3) || (2 == 2 ? 0 : 1)) return 2;
       int a[(1 < 2) + (5 >= 5) + (7 != 5) + 1];
       return (sizeof(a)/sizeof(a[0]) == 4) ? 0 : 3; }"
# An always_inline body with control flow and several returns inlines as a
# statement-expression: each `return X' becomes an assignment plus a jump to a
# per-instance end label, so a caller's constant argument still selects the right
# branch.  A plain call gives the same answer, so this holds on both ports.
difftest "always-inline-control-flow" \
    "static inline __attribute__((always_inline))
   int clamp(int x,int lo,int hi){
       if (x < lo) return lo;
       if (x > hi) return hi;
       return x; }
     int main(void){
       if (clamp(5,0,10) != 5) return 1;
       if (clamp(-3,0,10) != 0) return 2;
       if (clamp(99,0,10) != 10) return 3;
       int v = -7; if (clamp(v,0,10) != 0) return 4;
       return 0; }"
# A `.name` designator may target a member promoted from an anonymous
# struct/union (C17 6.7.2.1p13) - pervasive in kernel headers (qspinlock's
# `union { atomic_t val; struct { u8 locked, pending; }; }` initialized by name).
# The field-by-name search must skip the unnamed member (NULL name - never hand
# it to strcmp) yet still descend into it to reach the promoted name, at the
# right byte offset.  Covers a name in an anonymous union, a name nested one
# level deeper through an anonymous struct, and overlap via the union alias.
difftest "anonymous-member-designator" \
    "struct S { int a; union { int x; struct { short lo, hi; }; }; int b; };
static struct S s = { .a = 1, .hi = 0x1234, .b = 2 };
int main(void){
  if (s.a != 1 || s.b != 2) return 1;
  if (s.hi != 0x1234 || s.lo != 0) return 2;
  return (s.x == (0x1234 << 16)) ? 0 : 3; }"
# A directive skipped inside an inactive #if group whose trailing block comment
# spans a newline.  Skipping must consume the whole comment; otherwise the
# cursor is left mid-comment and the apostrophe in "that's" is lexed as a char
# constant (the kernel re-includes uapi/linux/openat2.h, whose RESOLVE_CACHED
# #define carries exactly such a comment, so its guarded body is skipped).
difftest "skip-directive-block-comment" \
```

```
    "#if 0
#define FOO 1 /* only complete if it can be
        done. May return -EAGAIN if that's not
        possible. */
#endif
int main(void){ return 0; }"
# A function-like macro whose parameters are
   spelled like keywords.  In
# translation phase 4 there are no keywords (C17
   6.4.1), so `int` and `struct`
# are ordinary parameter names; they must bind and
    substitute in the body.  The
# kernel's TRACE_EVENT(name, proto, args, struct,
   assign, print) relies on this.
difftest "macro-keyword-params" \
    "#define ADD(int, struct) ((int) + (struct))
int main(void){ return ADD(3, 4) == 7 ? 0 : 1; }"
# GNU attribute trailing a continuation declarator
   , before its '=' (the kernel's
# DEFINE_LOCK_GUARD_0 emits `T a = ..., *b
   __attribute__((__unused__)) = &a;`).
difftest "gnu-attr-trailing-declarator" \
    "int main(void){ int a = 5, *b __attribute__((
   __unused__)) = &a; int c __attribute__((
   __unused__)) = 3; return *b + c - 8; }"
# Function-TYPE typedef, incl. the parenthesised-
   name form the kernel's percpu.h
# uses (`typedef int (pcpu_fc_cpu_distance_fn_t)(
   unsigned, unsigned);`).
difftest "function-type-typedef" \
    "typedef int fn_t(int); typedef int (fn2_t)(
   int);
int sq(int a){ return a*a; } int prototype(fn_t cb
   , int x);
int main(void){ fn_t *p = sq; fn2_t *q = sq;
   return p(6) + q(1); }"
# GNU typeof: a type-specifier that is the type of
    a parenthesised type-name or
# expression.  Used in declarations, casts, and
   sizeof; __typeof__ is the alias.
difftest "gnu-typeof" \
    "struct S { int a; long b; }; static struct S
   g = { 1, 0x99 }; int main(void){ int x = 5;
   typeof(x) y = x + 1; __typeof__(&x) p = &x;
   typeof(g) c = g; unsigned u = 4000000000u;
   typeof(u) v = u; return (y == 6 && *p == 5 && c
   .b == 0x99 && v == 4000000000u && sizeof(typeof
   (double)) == 8) ? 0 : 1; }"
# typeof of an *expression* must keep that
   expression's real type, not silently
# fall back to int.  These cases would all pass
   with an int fallback if the
# operands were int, so each names a wider type: `
   double` (8 bytes, distinct
# codegen) and a `long` sum.  The block-local uses
    also exercise resolving the
# operand name after its scope is gone (see the
   memoisation in resolve-type).
difftest "gnu-typeof-expr-type" \
    "int main(void){ double d = 2.5; __typeof__(d)
    x = d; typeof(d + d) s = d + d; long a = 3, b
   = 4; __typeof__(a * b) p = a * b; return (
   sizeof(x) == 8 && (int)(s * 2.0) == 10 &&
   sizeof(p) == 8 && p == 12) ? 0 : 1; }"
# GNU statement expressions `({ ... })`: the value
    is the final expression-
# statement.  Covers a plain value, an
   intermediate declaration whose local
# feeds the result, a loop that accumulates, and
   the canonical typeof-based
# `max` idiom written inline (statement expression
    + typeof together).
difftest "gnu-statement-expr" \
    "int main(void){ int a = ({ int t = 3; t * t +
    1; }); int b = ({ 5; }); int c = ({ int s = 0;
    for (int i = 0; i < 4; i++) s += i; s; }); int
    m = ({ __typeof__(a) _x = a; __typeof__(b) _y
   = b; _x > _y ? _x : _y; }); return (a == 10 &&
   b == 5 && c == 6 && m == 10) ? 0 : 1; }"
# The identifier-led expression statement `a > b ?
    a : b;` must parse its whole
# operator chain and trailing ternary --- a bare
   multiplicative-then-additive
# chain (`t * t + 1;`) and a relational-then-
   ternary both begin with an
# identifier consumed for label look-ahead.
difftest "expr-stmt-operator-chain" \
    "int main(void){ int t = 3, r = 0; t * t + 1;
   r = t * t + 1; int m; t > r ? (m = t) : (m = r)
   ; return (r == 10 && m == 10) ? 0 : 1; }"
# __attribute__((packed)): no inter-member padding
   , aggregate alignment 1.  Both
# the before-tag and after-brace spellings, and a
   packed struct nested inside
# another, must match clang's sizes and offsets
   exactly; the unaligned int must
# also read and write back correctly at run time.
difftest "gnu-packed-layout" \
    "struct __attribute__((packed)) P { char c;
   int i; short s; }; struct Q { char c; int i; }
   __attribute__((packed)); struct __attribute__((
   packed)) N { char c; struct P p; }; int main(
   void){ struct P p; p.c = 1; p.i = 0x12345678; p
   .s = 0x2222; unsigned long oi = (unsigned long)
   ((char*)&p.i - (char*)&p); return (sizeof(
   struct P) == 7 && oi == 1 && sizeof(struct Q)
   == 5 && sizeof(struct N) == 8 && p.i == 0
   x12345678 && p.s == 0x2222) ? 0 : 1; }"
# __attribute__((aligned(N))) raises the aggregate
   's size and alignment.
difftest "gnu-aligned-attribute" \
    "struct __attribute__((aligned(16))) A { int x
   ; }; int main(void){ return (sizeof(struct A)
   == 16 && _Alignof(struct A) == 16) ? 0 : 1; }"
# A STORE's IR_DEST holds its access-size code,
   not a vreg.  The spill-slot
# colourer read IR_DEST as a freshly-defined vreg
   for every instruction, so a
# narrow store's access-size code (e.g. 2 for a
   16-bit store) posed as "vreg 2"
# and stole a slot mid-block --- under enough
   register pressure a local's
# address and the value stored into it aliased one
    slot, and `1 = 1` became a
# store of 1 THROUGH the value 1 (a write to
   address 1) and faulted.  Found by
# csmith differential testing; reduced.  The
   colourer must skip non-defining
# opcodes exactly as pass 1 does.
difftest "spill-slot-store-access-code-not-vreg" \
    "#include <stdint.h>
static int16_t g_182 = -3; static int16_t *
   volatile g_181 = &g_182;
static int16_t func_1(void){ int8_t *l_13[10];
   const uint16_t l_15 = 1UL; int32_t l_56 = 0
   x9123EBAAL; uint32_t l_94[8] = {0x275CA956L,0
   x275CA956L,0x275CA956L,0x275CA956L,0x275CA956L
   ,0x275CA956L,0x275CA956L,0x275CA956L}; int32_t
   l_106 = 3L, l_110 = 0x16EC97F7L, l_124 = 0
   x55A8A431L, l_125 = (-9L); int32_t l_126[2][6]
   = {{6L,0xE54C48CFL,0xE54C48CFL,6L,7L,6L},{6L,7L
   ,6L,0xE54C48CFL,0xE54C48CFL,6L}}; uint8_t l_178
    = 253UL; int32_t *l_229 = &l_125, *l_232 = &
   l_124, *l_233 = &l_106, *l_234 = &l_110; return
    (*g_181); }
```

```
int main(void){ return func_1() == -3 ? 0 : 1; }"
# A UNION returns by value exactly like a struct: the call allocates a result
# buffer and the callee passes the union's ADDRESS (not a scalar load of member
# zero).  Both were only wired for TY_STRUCT, so a union return unpacked x0 into
# a never-materialised buffer (fault) and the callee `return u` loaded 4 bytes
# as an int.  Also: a value-returning function that falls off its end with the
# result discarded must not fault.  Found by csmith differential testing.
difftest "union-return-by-value" \
    "#include <stdint.h>
union U { uint64_t a; int b; }; union U mk(int x){ union U u; u.a = x; return u; } int main(void){ union U r = mk(42); return r.a == 42 ? 0 : 1; }"

# Strength reduction: mul/udiv/umod by powers of two become shifts/masks.
# The signed div/mod and negative cases must stay numerically correct
# (they are NOT reduced, precisely because a shift would round differently).
difftest "strength-unsigned-mul" \
    "unsigned f(unsigned x){ return x*8 + x*16; } int main(void){ return (int)(f(3) % 200); }"
difftest "strength-unsigned-div" \
    "unsigned f(unsigned x){ return x/4 + x/8; } int main(void){ return (int)f(100); }"
difftest "strength-unsigned-mod" \
    "unsigned f(unsigned x){ return x%16 + x%4; } int main(void){ return (int)f(103); }"
difftest "strength-signed-mul" \
    "int f(int x){ return x*32; } int main(void){ return f(-3) + 100; }"
difftest "strength-signed-divmod-neg" \
    "int main(void){ int a=-17; return (a/4)*10 + (a%8) + 100; }"
difftest "strength-mul-zero" \
    "int f(int x){ return x*0 + 7; } int main(void){ return f(999); }"
difftest "strength-nonpow2" \
    "unsigned f(unsigned x){ return x*3 + x/5 + x%7; } int main(void){ return (int)f(100); }"

# Post-increment of a GLOBAL pointer through a store (`*b++ = v`): the address
# must load in the pointer's 64-bit width and step by sizeof(pointee), not 1.
# The global path once clobbered the AST-node register the shared load-modify-
# store tail reads its operand type from, so it loaded the pointer as a 32-bit
# int, stepped by 1, and stored through a truncated address (SIGSEGV).
difftest "global-ptr-postinc-store" \
    "int buf[4]; int *b = buf; int main(void){ *b++ = 10; *b++ = 20; return (b - buf) * 100 + buf[0] + buf[1]; }"
difftest "global-ptr-postinc-neg-index" \
    "int buf[4]; int *b = buf; int main(void){ *b++ = 7; return b[-1] == 7 ? 42 : 0; }"

# ptr - ptr is a pointer DIFFERENCE, typed ptrdiff_t (a signed integer), not a
# pointer.  If the result kept a pointer type, a surrounding `(p-q) + n` scaled
# `n` by the element size, so `(p-a)+a[2]` added `a[2]*sizeof(int)`.
difftest "ptrdiff-in-arith" \
    "int a[8] = {0,0,3,4}; int main(void){ int *p = a + 2; return (p - a) + a[2] + a[3]; }"

# Read-only const globals (M5): a const integer scalar/array with a literal
# initializer is emitted as read-only bytes in __TEXT rather than __common
# storage.  These verify the byte image reads back correctly (the write-fault
# property is out of band for a differential runtime test).
difftest "rodata-const-array" \
    "static const int t[] = {0,1,2,3,5,6,7,9,10,11}; int main(void){ int s=0; for(int i=0;i<10;i++) s+=t[i]; return s; }"
difftest "rodata-const-scalar" \
    "const int k = 1234; int main(void){ return k % 200; }"
difftest "rodata-const-char" \
    "static const char c[] = {'A','B','C','D'}; int main(void){ return c[0]+c[3]; }"
difftest "rodata-const-index-var" \
    "static const int t[4] = {11,22,33,44}; int at(int i){ return t[i]; } int main(void){ return at(2) - at(0); }"
difftest "rodata-const-partial" \
    "static const int t[6] = {9,8,7}; int main(void){ int s=0; for(int i=0;i<6;i++) s+=t[i]; return s; }"
difftest "rodata-const-neg-int" \
    "static const int t[] = {-1,2,-3,4}; int main(void){ int s=0; for(int i=0;i<4;i++) s+=t[i]; return s + 100; }"
difftest "rodata-const-double" \
    "static const double t[] = {1.5,2.25,-4.0}; int main(void){ double s=0; for(int i=0;i<3;i++) s+=t[i]; return (int)(s*100); }"
difftest "rodata-const-float" \
    "static const float t[] = {0.5f,1.25f,-2.0f}; int main(void){ float s=0; for(int i=0;i<3;i++) s+=t[i]; return (int)(s*100); }"
difftest "rodata-const-double-scalar" \
    "const double pi = 3.14159; int main(void){ return (int)(pi*1000); }"
difftest "rodata-const-bitflags" \
    "static const int f[]={1<<0,1<<1,1<<2,1<<3,(1<<4)|1}; int main(void){ int s=0; for(int i=0;i<5;i++) s+=f[i]; return s; }"
difftest "rodata-const-arith" \
    "static const int t[]={2*3,10%3,0xFF&0x0F,100/4}; int main(void){ int s=0; for(int i=0;i<4;i++) s+=t[i]; return s; }"
difftest "rodata-const-enum" \
    "enum{A=5,B=7,C=100}; static const int t[]={A,B,C,A+B}; int main(void){ int s=0; for(int i=0;i<4;i++) s+=t[i]; return s; }"
difftest "rodata-const-struct" \
    "struct P{int a;int b;int c;}; static const struct P p={10,20,30}; int main(void){ return p.a+p.b+p.c; }"
difftest "rodata-const-struct-mixed" \
    "struct Q{int a;double b;char c;}; static const struct Q q={7,2.5,65}; int main(void){ return q.a + (int)(q.b*2) + q.c; }"
difftest "rodata-const-struct-partial" \
    "struct R{int a;int b;int c;}; static const struct R r={100}; int main(void){ return r.a + r.b + r.c; }"
```

```
difftest "rodata-const-struct-array" \
    "struct P{int x;int y;}; static const struct P
    t[]={{1,2},{3,4},{5,6}}; int main(void){ int s
   =0; for(int i=0;i<3;i++) s+=t[i].x*t[i].y;
   return s; }"
difftest "rodata-const-struct-array-mixed" \
    "struct Q{int a;double b;}; static const
   struct Q q[]={{1,1.5},{2,2.5}}; int main(void){
    return q[0].a + (int)(q[0].b*10) + q[1].a + (
   int)(q[1].b*10); }"
difftest "rodata-const-struct-array-partial" \
    "struct P{int x;int y;}; static const struct P
    t[3]={{7,8}}; int main(void){ return t[0].x+t
   [0].y+t[1].x+t[2].y; }"
# Const pointer tables (M5 Stage 2): go to
   __DATA_CONST,__const with a
# relocation per slot binding it to the string
   literal; NULL slots stay zero.
difftest "rodata-ptrarr" \
    "static const char *const o[]={\"ab\",\"cd
   \",\"ef\"}; int main(void){ int s=0; for(int i
   =0;i<3;i++) s+=o[i][0]; return s; }"
difftest "rodata-ptrarr-null" \
    "static const char *const o[]={\"x\",\"yz
   \",0}; int main(void){ int n=0; for(int i=0;o[i
   ];i++) n++; return n*10 + o[1][1]; }"
difftest "rodata-ptrarr-extern" \
    "const char *const msgs[]={\"hello\",\"world
   \"}; int main(void){ return msgs[0][0]+msgs
   [1][0]; }"

difftest "ternary-chain" \
    "int main(void){ int x = 3; return x == 1 ? 10
    : x == 2 ? 20 : x == 3 ? 30 : 40; }"
difftest "comma" \
    "int main(void){ int a = 0; int b = (a = 5, a
   + 3); return a + b; }"
difftest "compound-assign" \
    "int main(void){ int a = 10; a += 5; a *= 2; a
    -= 3; a /= 4; return a; }"
difftest "inc-dec" \
    "int main(void){ int a = 5; int b = a++; int c
    = ++a; return a + b + c; }"
difftest "short-circuit" \
    "int side(int *n){ *n = *n + 1; return 1; }
   int main(void){ int n = 0; if (0 && side(&n)) n
    = n + 100; if (1 || side(&n)) n = n + 10;
   return n; }"
difftest "sizeof-mix" \
    "int main(void){ return sizeof(int) + sizeof(
   char) + sizeof(long); }"
difftest "char-arith" \
    "int main(void){ char c = 'A'; int d = c + 1;
   return d; }"
difftest "global-counter" \
    "int g = 100; void bump(void){ g = g + 7; }
   int main(void){ bump(); bump(); bump(); return
   g % 200; }"
difftest "static-local" \
    "int next(void){ static int c = 0; c = c + 1;
   return c; } int main(void){ next(); next();
   return next() + next(); }"
```

□

923 · used in 37k

78.8. Deeper Combinations

A second batch reaches for feature *combinations*: signed comparisons and shifts of negatives, whole-`struct` assignment, nested structs, arrays of pointers, multi-level indirection, register pressure from many live locals, explicit narrowing casts, and a function pointer stored in a `struct` field—the last a known frontier that surfaces as a gap.

⟨ *Difftest: combinations* 930 ⟩ ≡

```
echo ""
echo "--- Deeper Combinations ---"

difftest "neg-compare" \
    "int main(void){ int a = -5; int b = 3; return
    (a < b) * 100 + (a <= b) * 10 + (b > a); }"
difftest "ashr-negative" \
    "int main(void){ int a = -64; return (a >> 2)
   + 200; }"
difftest "mod-signs" \
    "int main(void){ return (17 % 5) + (-17 % 5) +
    (17 % -5) + 100; }"
difftest "nested-struct" \
    "struct I { int x; int y; }; struct O { struct
    I p; int z; }; int main(void){ struct O o; o.p
   .x = 1; o.p.y = 2; o.z = 3; return o.p.x + o.p.
   y + o.z; }"
difftest "struct-assign" \
    "struct P { int a; int b; int c; }; int main(
   void){ struct P x; x.a = 5; x.b = 6; x.c = 7;
   struct P y; y = x; return y.a + y.b + y.c; }"
difftest "array-of-ptr" \
    "int main(void){ int a = 1; int b = 2; int c =
    3; int *p[3]; p[0] = &a; p[1] = &b; p[2] = &c;
    int s = 0; int i = 0; while (i < 3) { s = s +
   *p[i]; i = i + 1; } return s; }"
difftest "triple-deref" \
    "int main(void){ int x = 9; int *p = &x; int
   **q = &p; int ***r = &q; ***r = 42; return x; }
   "
difftest "deep-arith" \
    "int main(void){ int a = 2; int b = 3; int c =
    4; int d = 5; return ((a + b) * (c + d) - (a *
    b)) / (c - a) % 50; }"
difftest "many-locals" \
    "int main(void){ int a=1; int b=2; int c=3;
   int d=4; int e=5; int f=6; int g=7; int h=8;
   int i=9; int j=10; return a+b+c+d+e+f+g+h+i+j;
   }"
difftest "string-index" \
    "int main(void){ char *s = \"hello\"; return s
   [0] + s[4]; }"
difftest "cast-narrow" \
    "int main(void){ int a = 300; char c = (char)a
   ; return (int)c + 100; }"
difftest "fnptr-in-struct" \
    "struct Ops { int (*f)(int, int); }; int add(
   int a, int b){ return a + b; } int main(void){
   struct Ops o; o.f = add; return o.f(20, 22); }"
```

□

930 · used in 37k

78.9. Language Features

A broad sweep across language constructs, several of which the harness first flagged as bugs and which are now fixed:

enumerators with explicit values, `sizeof` of a whole array (which must not decay to a pointer), the comma operator in a `for` clause (a parser register bug that hung the loop), and pointer difference. The rest exercise `goto`, dense `switch`, deep recursion, eight-argument calls, pointer comparison, and a spread of operators. Two cases pin the C11 anonymous member: a nameless `union` and a nameless `struct` whose fields are reached directly through the enclosing object, which must agree byte-for-byte with the reference compiler.

⟨ *Difftest: language* 931 ⟩ ≡

```
echo ""
echo "--- Language Features ---"

difftest "define-multiline-comment" \
    "#define X 5 /* comment that\nspans two lines
   */\nint main(void){ return X - 5; }"
# #pragma push_macro / pop_macro save and restore
   a macro around a local
# redefinition; after the pop, X is 10 again
   despite the intervening define.
difftest "pragma-push-pop-macro" \
    "#define X 10\n#pragma push_macro(\"X\")\n#
   undef X\n#define X 20\n#pragma pop_macro(\"X\")
   \nint main(void){ return X; }"
difftest "nested-struct-tag" \
    "struct Hash { int count; struct _ht { int n;
   } *ht; }; int main(void){ struct _ht e; e.n =
   42; return e.n - 42; }"
# A struct defined inline in an ARRAY (or
   qualified) declaration must register
# its tag, so a later reference resolves to the
   struct -- not the int fallback.
# SQLite's `static const struct EncName {...}
   encnames[]` relies on this.
difftest "inline-struct-in-array-decl" \
    "static const struct EncName { char *z; int e;
    } tbl[] = {{\"a\",1},{\"b\",2},{0,0}}; int
   main(void){ const struct EncName *p; int s=0;
   for(p=&tbl[0]; p->z; p++) s+=p->e; return s; }"
# A `static const` local declared inside a nested
   block (here a switch case)
# must get its read-only storage AND initializer
   emitted -- the codegen pre-pass
# has to recurse into block bodies, not just scan
   the function's top level.  This
# is SQLite's PRAGMA-encoding pattern (`static
   const struct EncName {...}
# encnames[]` inside a switch case).  (A non-const
    static in a nested block links
# but its runtime initializer is a separate, still
   -top-level-only pass.)
difftest "nested-static-const-local" \
    "int pick(int k){ switch(k){ case 1: { static
   const int a[3] = {10,20,30}; return a[k]; }
   default: { static const int b[2] = {7,9};
   return b[0]; } } } int main(void){ return pick
   (1) + pick(0); }"
difftest "va-arg-many-fixed-params" \
    "#include <stdarg.h>\nint f(int p1,int p2,int
   p3,int p4,int p5,int p6,int p7,int p8,int p9
   ,...){va_list ap;va_start(ap,p9);int a=va_arg(
   ap,int),b=va_arg(ap,int);va_end(ap);return p1+
   p9+a+b;}\nint main(void){return f
   (1,2,3,4,5,6,7,8,9,100,200)-310;}"
difftest "va-arg-global-valist" \
    "#include <stdarg.h>\nva_list gap;\nint g(int
   n,...){va_start(gap,n);int r=va_arg(gap,int)+
   va_arg(gap,int);va_end(gap);return r;}\nint
   main(void){return g(2,3,4)-7;}"
difftest "va-arg-struct-assign" \
    "#include <stdarg.h>\nstruct tiny{int c;};\
   nint sum(int n,...){int t=0;struct tiny x;
   va_list ap;va_start(ap,n);for(int i=0;i<n;i++){
   x=va_arg(ap,struct tiny);t+=x.c;}va_end(ap);
   return t;}\nint main(void){struct tiny a={10},b
   ={11},c={12};return sum(3,a,b,c)-33;}"
difftest "parenthesized-func-name" \
    "static int (add)(int a, int b){ return a + b;
    } int (main)(void){ return add(3,4) - 7; }"
difftest "stdint-signed-mins" \
    "#include <stdint.h>\nint main(void){ return (
   INT8_MIN==-128) && (INT16_MIN==-32768) && (
   INT32_MIN==(-2147483647-1)) ? 0 : 1; }"
difftest "block-scoped-tag" \
    "struct T{int x;}; int main(void){ struct T v;
    { struct T {int z;} w; w.z=9; } v.x=2; return
   v.x - 2; }"
difftest "prefixed-char-literal" \
    "int main(void){ return (L'A' == 65) + (u'B'
   == 66) + (U'\\n' == 10) + 7; }"
difftest "variadic-macro" \
    "#define CALL(F,...) F(__VA_ARGS__)\nint add3(
   int a,int b,int c){return a+b+c;}\nint main(
   void){ return CALL(add3, 10, 20, 12); }"
difftest "variadic-macro-empty" \
    "#define SUM(base,...) (base)\nint main(void){
    return SUM(7); }"
difftest "anon-union-member" \
    "struct S{int a; union{int b1;int b2;};}; int
   main(void){ struct S s; s.a=5; s.b1=12; return
   s.b2 - 12 + s.a; }"
difftest "anon-struct-member" \
    "struct S{int a; struct{int c1;int c2;};}; int
    main(void){ struct S s; s.a=3; s.c1=10; s.c2
   =2; return s.c1 + s.c2 - s.a - 2; }"
difftest "proto-in-declarator-list" \
    "int f(int a), g(int a), v; int f(int a){
   return a;} int g(int a){return a*2;} int main(
   void){ v=3; return f(1)+g(1)-3-v; }"
difftest "array-param-qualifiers" \
    "int sum(int n, int a[static 1]){int s=0;int i
   ;for(i=0;i<n;i=i+1)s=s+a[i];return s;} int main
   (void){ int v[3]; v[0]=10;v[1]=20;v[2]=12;
   return sum(3,v)-42; }"
difftest "cmp-int-vs-uint-max" \
    "int main(void){ int x=0; x=~x; if(x!=0
   xffffffffU) return 1; if(x>=0) return 2;
   unsigned u=x; if(u!=4294967295U) return 3;
   return 0; }"
difftest "cmp-signed-neg-vs-unsigned" \
    "int main(void){ int a=-1; unsigned b=1;
   return (a>b)?7:0; }"
difftest "cmp-uint-vs-long-signed" \
    "int main(void){ unsigned a=1; long b=-1;
   return (a>b)?0:9; }"
difftest "cmp-int-vs-ulong-unsigned" \
    "int main(void){ int c=-1; unsigned long d=1;
   return (c<d)?9:0; }"
difftest "cmp-promoted-uchar-schar" \
    "int main(void){ unsigned char uc=200; signed
   char sc=-1; return (uc>sc)?0:9; }"
difftest "shift-type-left-operand" \
    "int main(void){ return ((-1 << (unsigned)1) <
    0) ? 0 : 9; }"
difftest "shift-neg-widen-long" \
```

```
    "int main(void){ long b = (-1 << (unsigned)1);
    return (b == -2) ? 0 : 9; }"
difftest "unary-minus-narrow-unsigned" \
    "int main(void){ return (-(unsigned short)1 <
   0) ? 0 : 9; }"
difftest "cast-signed-to-wider-unsigned" \
    "signed char c=(signed char)0xff; int main(
   void){ return (((unsigned short)c ^ 0)==65535)
   && ((int)(signed char)200==-56) && ((short)
   70000==4464) ? 0 : 1; }"
difftest "predec-unsigned-char-wrap" \
    "int main(void){ unsigned int i=0; unsigned
   char z=0; do{++i;}while(--z>0); return i - 256;
    }"
difftest "preinc-narrow-wrap-value" \
    "int main(void){ unsigned char z=0; int r=--z;
    signed char s=127; int t=++s; return (r==255
   && t==-128) ? 0 : 1; }"
difftest "bitfield-promote-signed" \
    "struct X{unsigned int u3:3; signed int s31
   :31; unsigned int u32:32;}; struct X x; int
   main(void){ return ((x.u3-2)<0) && ((x.s31-2)
   <0) && ((x.u32-2)>=0) ? 0 : 1; }"
difftest "enum-bitfield-unsigned" \
    "enum E{LOW=148,HIGH}; struct S{enum E c:8;};
   int main(void){ struct S s; s.c=HIGH; return (
   int)s.c - 149; }"
difftest "array-of-struct-elided-length" \
    "struct P{int a,b,c;}; struct P arr[] =
   {1,2,3, 4,5,6, 7,8,9}; int main(void){ return (
   int)(sizeof(arr)/sizeof(arr[0])) - 3 + arr[2].a
    - 7; }"
difftest "func-ptr-cast" \
    "int add(int a,int b){return a+b;} int main(
   void){ int (*f)(int,int)=add; int (*g)(int,int)
   =(int (*)(int,int))f; return g(3,4) - 7; }"
difftest "sizeof-func-ptr-type" \
    "int main(void){ return (int)sizeof(void (*)(
   void)) - 8; }"

difftest "builtin-line-macro" \
    "int main(void){ int a = __LINE__; return a +
   __LINE__; }"
difftest "line-directive-macro" \
    "#define L 500\n#line L\nint main(void){
   return __LINE__ - 501; }"
difftest "compound-literal-addr" \
    "struct S{int a;int b;}; struct S *g = &(
   struct S){1,2}; int main(void){ return g->a + g
   ->b - 3; }"
difftest "compound-literal-local" \
    "struct S{int a;int b;}; int sum(struct S s){
   return s.a+s.b;} int main(void){ struct S x = (
   struct S){7,9}; return sum(x) + sum((struct S)
   {2,3}) - 21; }"
difftest "compound-literal-member" \
    "struct S{int a;int b;}; int main(void){
   return (struct S){3,4}.b - 4; }"
difftest "compound-literal-subscript" \
    "int main(void){ return (int[]){10,20,30}[1] -
    20; }"
difftest "ptr-to-array-declarator" \
    "int main(void){ char arr[2][4]; char (*p)[4];
    p=arr; arr[1][3]=2; return (p[1][3]!=2)?1:0; }
   "
difftest "enum-in-struct-member" \
    "struct { enum { X = 5 } e; } s; int main(void
   ){ return X; }"
difftest "enum-explicit" \
    "enum E { A, B, C, D = 10, E2 }; int main(void
   ){ return A + B + C + D + E2; }"
difftest "enum-negative" \
    "enum E { X = -1, Y, Z = 100, W }; int main(
   void){ return X + Y + Z + W + 100; }"
difftest "sizeof-array-count" \
    "int main(void){ int a[10]; return (int)(
   sizeof(a) / sizeof(a[0])); }"
difftest "sizeof-array-bytes" \
    "int main(void){ char a[25]; return (int)
   sizeof(a); }"
difftest "for-comma-update" \
    "int main(void){ int s = 0; int i; for (i = 0;
    i < 5; i = i + 1, s = s + 10) { } return s; }"
difftest "for-comma-both" \
    "int main(void){ int s = 0; int i; int j; for
   (i = 0, j = 10; i < j; i = i + 1, j = j - 1) {
   s = s + 1; } return s; }"
difftest "ptr-diff-index" \
    "int main(void){ int a[10]; int *p = &a[2];
   int *q = &a[7]; return (int)(q - p); }"
difftest "ptr-diff-strlen" \
    "int slen(char *s){ char *p = s; while (*p) p
   = p + 1; return (int)(p - s); } int main(void){
    return slen(\"hello world\"); }"
difftest "goto-loop" \
    "int main(void){ int i = 0; int s = 0; loop:
   if (i >= 10) goto end; s = s + i; i = i + 1;
   goto loop; end: return s; }"
difftest "goto-forward" \
    "int main(void){ int x = 5; if (x > 0) goto
   pos; return 1; pos: return 42; }"
difftest "goto-cleanup" \
    "int f(int n){ int r = 0; if (n < 0) goto done
   ; r = n * 2; done: return r; } int main(void){
   return f(21); }"
difftest "goto-break-out" \
    "int main(void){ int s = 0; int i = 0; while (
   i < 5) { if (i == 3) goto out; s = s + i; i = i
    + 1; } out: return s + 10; }"
difftest "switch-dense" \
    "int f(int x){ switch (x) { case 0: return 10;
    case 1: return 20; case 2: return 30; case 3:
   return 40; default: return 99; } } int main(
   void){ return f(2) + f(5); }"
difftest "nested-switch" \
    "int main(void){ int a = 1; int b = 2; int r =
    0; switch (a) { case 1: switch (b) { case 2: r
    = 7; break; default: r = 1; } break; default:
   r = 0; } return r; }"
difftest "ackermann" \
    "int ack(int m, int n){ if (m == 0) return n +
    1; if (n == 0) return ack(m - 1, 1); return
   ack(m - 1, ack(m, n - 1)); } int main(void){
   return ack(2, 3); }"
difftest "eight-args" \
    "int f(int a, int b, int c, int d, int e, int
   g, int h, int i){ return a + b + c + d + e + g
   + h + i; } int main(void){ return f(1, 2, 3, 4,
    5, 6, 7, 8); }"
difftest "ptr-compare" \
    "int main(void){ int a[5]; int *p = a; int *e
   = a + 5; int c = 0; while (p < e) { c = c + 1;
   p = p + 1; } return c; }"
difftest "assign-chain" \
    "int main(void){ int a; int b; int c; a = b =
   c = 7; return a + b + c; }"
difftest "hex-octal" \
    "int main(void){ int a = 0xFF; int b = 0777;
   return a + b; }"
```

```
difftest "main-implicit-return" \
    "int main(void){ int x = 7; x = x + 1; }"
difftest "multi-declarator-global" \
    "int a = 1, b = 2, c = 3; int main(void){
   return a + b + c; }"
difftest "multi-declarator-global-noinit" \
    "int a, b, c; int main(void){ a = 4; b = 5; c
   = 6; return a + b + c; }"
difftest "copyprop-chain" \
    "long f(long a){ long x = a; long y = x; long
   z = y; return x + y + z; } int main(void){
   return (int)f(7); }"
difftest "copyprop-alias-invalidate" \
    "long f(void){ long v = 5; long *p = &v; long
   a = v; *p = 9; long b = v; return a * 10 + b; }
    int main(void){ return (int)f(); }"
difftest "copyprop-reassign" \
    "long f(long a){ long x = a; long b = x; x = a
    + 100; long c = x; return b + c; } int main(
   void){ return (int)f(1); }"
difftest "copyprop-across-call" \
    "long id(long z){ return z; } long f(long a){
   long x = a; long b = id(a); long c = x; return
   b + c; } int main(void){ return (int)f(4); }"
difftest "char-escapes" \
    "int main(void){ char n = '\\n'; char t = '\\t
   '; char z = '\\0'; return n + t + z; }"
```

□

931 · used in 37k

78.10. Integer Width

The first divergence this suite caught was a real one and is now fixed. kcc's all-spill model keeps every value in a 64-bit slot and computes with 64-bit instructions, so it once left an `int` or `unsigned int` result un-truncated: a computation that wrapped past 2^{32} kept the extra high bits and disagreed with `clang`. `_kcc_ir_norm32` re-canonicalises the result of each width-growing operation to its declared width—sign-extending a signed result, zero-extending an unsigned one. It is called from all three sites that produce such results: ordinary binary operators (`build_binary`), compound assignment for both locals and globals (`build_assign`), and unary negation and complement (`build_unary`). The tests below pin the behaviour down: unsigned wraparound on `+`, `-`, `*`, `<<`, compound `+=`/`*=`, and unary `-`/`~`; unsigned comparison and division after a wrap; the usual-arithmetic-conversion case where a signed operand meets an unsigned one; and—as guards against over-eager masking—`long` and pointer computations that must stay 64-bit. The `xfail` machinery remains for the next genuine limitation; at present there are none.

⟨ *Difftest: divergences* 933 ⟩ ≡

```
echo ""
echo "--- Integer Width (32-bit truncation) ---"

difftest "u32-wrap-sub" \
    "int main(void){ unsigned int a = 0; a = a -
   1; return (int)(a % 251); }"
difftest "u32-wrap-add" \
    "int main(void){ unsigned int a = 0xFFFFFFFF;
   a = a + 1; return (int)(a % 100); }"
difftest "u32-overflow-sum" \
    "int main(void){ unsigned int a = 4000000000;
   unsigned int b = 1000000000; return (int)((a +
   b) % 1000); }"
difftest "u32-mul-overflow" \
    "int main(void){ unsigned int a = 100000;
   unsigned int b = 100000; return (int)((a * b) %
    1000); }"
difftest "u32-shl-overflow" \
    "int main(void){ unsigned int a = 0xFFFFFFFF;
   return (int)((a << 4) % 100); }"
difftest "u32-cmp-after-wrap" \
    "int main(void){ unsigned int a = 0; a = a -
   1; unsigned int b = 5; return a > b; }"
difftest "uac-signed-unsigned" \
    "int main(void){ int a = -1; unsigned int b =
   10; return (int)((a + b) % 100); }"
difftest "u32-compound-add" \
    "int main(void){ unsigned int a = 0xFFFFFFFF;
   a += 1; return (int)(a % 100); }"
difftest "u32-compound-mul" \
    "int main(void){ unsigned int a = 100000; a *=
    100000; return (int)(a % 1000); }"
difftest "u32-compound-global" \
    "unsigned int g; int main(void){ g = 0
   xFFFFFFFF; g += 1; return (int)(g % 100); }"
difftest "u32-negate" \
    "int main(void){ unsigned int a = 5; unsigned
   int b = -a; return (int)(b % 251); }"
difftest "u32-complement" \
    "int main(void){ unsigned int a = 0; unsigned
   int b = ~a; return (int)(b % 251); }"
difftest "long-stays-64" \
    "int main(void){ long a = 4000000000; long b =
    1000000000; return (int)((a + b) % 1000); }"
difftest "ptr-arith-untouched" \
    "int main(void){ int a[10]; int i = 0; while (
   i < 10) { a[i] = i; i = i + 1; } int *p = a;
   return *(p + 7); }"
difftest "desig-array-local" \
    "int main(void){ int a[6] = { [4] = 44, [0] =
   1, 2 }; return a[0]+a[1]+a[4]; }"
difftest "desig-struct-local" \
    "struct P { int x; int y; int z; }; int main(
   void){ struct P p = { .y = 20, .x = 10 };
   return p.x + p.y + p.z; }"
difftest "desig-array-global" \
    "int g[6] = { [5] = 9, [1] = 11 }; int main(
   void){ return g[1] + g[5] + g[0]; }"
difftest "desig-struct-global" \
    "struct P { int a; int b; int c; }; struct P g
    = { .c = 3, .a = 1 }; int main(void){ return g
   .a*100 + g.b*10 + g.c; }"
difftest "desig-mixed" \
    "int main(void){ int m[5] = { 1, 2, [4] = 5 };
    struct { int a, b, c; } s = { 100, .c = 30 };
   return m[0]+m[1]+m[4]+s.a+s.c; }"
difftest "desig-array-inferred" \
    "int main(void){ int a[] = { [3] = 7, [1] = 2
   }; return a[0] + a[1] + a[3]; }"
difftest "bitfield-pack" \
    "struct F { unsigned a:3; unsigned b:5;
   unsigned c:1; }; int main(void){ struct F f; f.
   a=5; f.b=20; f.c=1; return f.a + f.b + f.c + (
   int)sizeof(struct F); }"
difftest "bitfield-signed" \
    "struct M { int x:5; unsigned y:10; }; int
   main(void){ struct M m; m.x=-3; m.y=1000;
   return m.x + (int)m.y; }"
difftest "bitfield-cross" \
```

```
    "struct C { unsigned p:30; unsigned q:6; };
   int main(void){ struct C c; c.p=1000000; c.q
   =40; return (int)(c.p % 1000) + c.q + (int)
   sizeof(struct C); }"
difftest "bitfield-truncate" \
    "struct T { unsigned a:3; unsigned c:8; }; int
    main(void){ struct T t; t.a=100; t.c=300;
   return t.a + t.c; }"
difftest "bitfield-init" \
    "struct F { unsigned a:3; unsigned b:5;
   unsigned c:1; }; int main(void){ struct F f = {
    5, 20, 1 }; return f.a + f.b + f.c; }"
difftest "bitfield-desig" \
    "struct F { unsigned a:3; unsigned b:5;
   unsigned c:4; }; int main(void){ struct F f = {
    .c = 7, .a = 2 }; return f.a*100 + f.b*10 + f.
   c; }"
difftest "bitfield-global" \
    "struct F { unsigned a:3; unsigned b:5; };
   struct F g = { 6, 25 }; int main(void){ return
   g.a + g.b; }"
difftest "bitfield-volatile" \
    "struct S { volatile unsigned f:3; volatile
   int g:5; int tail; }; int main(void){ struct S
   s; s.tail=0; s.f=5; s.g=10; s.f=s.f+1; return s
   .f*10 + s.g; }"
difftest "compound-member" \
    "struct S { int x; int y; }; int main(void){
   struct S a; a.x=10; a.y=3; a.x+=5; a.y*=4; a.x
   -=2; return a.x + a.y; }"
difftest "compound-subscript" \
    "int main(void){ int m[3]={1,2,3}; m[0]+=m[2];
    m[1]*=10; m[2]<<=2; return m[0]+m[1]+m[2]; }"
difftest "compound-deref" \
    "int main(void){ int z=20; int *p=&z; *p+=7; *
   p-=2; *p%=6; return z; }"
difftest "compound-bitfield" \
    "struct B { unsigned a:4; unsigned b:8; }; int
    main(void){ struct B f; f.a=4; f.b=100; f.a
   +=3; f.b-=50; f.a|=8; return f.a*1000 + f.b; }"
difftest "compound-nested" \
    "struct S { int arr[4]; int v; }; int main(
   void){ struct S a; a.v=100; a.arr[2]=7; a.v/=4;
    a.arr[2]<<=3; a.v&=30; return a.v + a.arr[2];
   }"
difftest "bool-normalize" \
    "int main(void){ _Bool a=5; _Bool b=(7&4);
   _Bool c=0; _Bool d=256; return a+b+c+d; }"
difftest "bool-cast" \
    "int main(void){ int x=3; return (_Bool)x + (
   int)(_Bool)0 + (int)(_Bool)99; }"
difftest "bool-global" \
    "_Bool g = 42; int main(void){ return g; }"
difftest "bool-runtime" \
    "int main(void){ _Bool a; a = 7; a = a && 1;
   return a; }"
difftest "char-array-string" \
    "int main(void){ char s[] = \"ABC\"; return s
   [0]+s[1]+s[2]+s[3]; }"
difftest "char-array-sized" \
    "int main(void){ char s[8] = \"Hi\"; return s
   [0]+s[1]+s[2]+s[7]; }"
difftest "char-array-global" \
    "char g[] = \"xy\"; int main(void){ return g
   [0]+g[1]+g[2]; }"
difftest "char-array-index" \
    "int main(void){ char s[] = \"hello\"; int n
   =0; int i=0; while(s[i]){ n+=s[i]; i++; }
   return n % 200; }"
difftest "sizeof-inferred-int" \
    "int main(void){ int a[] = {1,2,3,4,5}; return
    sizeof(a)/sizeof(int); }"
difftest "sizeof-inferred-char" \
    "int main(void){ char s[] = \"hello\"; return
   (int)sizeof(s); }"
difftest "sizeof-inferred-desig" \
    "int main(void){ int d[] = {[5]=1, [2]=9};
   return (int)(sizeof(d)/sizeof(int)); }"
difftest "sizeof-inferred-global" \
    "char g[] = \"global\"; int ga[] =
   {10,20,30,40}; int main(void){ return (int)
   sizeof(g) + (int)(sizeof(ga)/sizeof(int)); }"
```

□

933 · used in 37k

78.11. Known Divergences

The sweep's last open construct—a `struct` returned *by value*—has now been lowered (§29.6) and graduated to the real `sret-*` tests above, following the same path `goto` (§29.4) and multi-dimensional arrays (§29.9.4) took once implemented. No expected divergence remains: every program the corpus exercises either agrees with clang or is an honest feature gap that kcc refuses to compile. The `difftest_xfail` harness stays in place so the next discovered divergence has a home.

⟨ *Difftest: gaps* 934a ⟩ ≡

```
echo ""
echo "--- Known Divergences (Stage 9 features) ---
   "
echo "  (none - the struct return-by-value
   divergence has been resolved)"
```

□

934a · used in 37k

78.12. Results

The suite fails only on a mismatch—a genuine wrong answer. Gaps (features kcc cannot yet compile) and expected divergences (§78.11) are tallied and reported so both frontiers stay visible, but neither fails the build.

⟨ *Difftest: results* 934b ⟩ ≡

```
echo ""
echo "========================================"
printf "diff results: %d ok, %d mismatch, %d gap,
   %d xfail (of %d)\n" \
    "$PASS" "$FAIL" "$GAP" "$XFAIL" "$TOTAL"
if [ "$FAIL" -ne 0 ]; then
    exit 1
fi
```

□

934b · used in 37k

79. ABI Interoperability Testing

The differential harness of Chapter 78 compiles each program *entirely* with kcc or entirely with `clang`, so it validates observable behaviour but is blind to the calling convention: a program that is internally self-consistent passes regardless of whether its register usage matches AAPCS64. To check the ABI itself we must compile part of a program with kcc and part with `clang`, link the two objects together, and run the result. If kcc's idea of where a `double` argument, a small `struct`, or a struct return value lives disagrees with `clang`'s, the mixed binary computes the wrong answer.

Each case provides a *caller* translation unit and a *callee* translation unit. The oracle is the all-`clang` build; the test then rebuilds one of the two units with kcc (both directions are exercised) and compares. Because exit codes are eight bits, we compare against the oracle's exit code rather than a hand-computed value.

⟨ *Interop: runner* 935a ⟩ ≡

```
#!/bin/bash
BASE="$(cd "$(dirname "$0")/.." && pwd)"
# kcc<->clang ABI interoperability suite -
   generated by weft, do not edit
set -u
PASS=0
FAIL=0
TOTAL=0
KCC="$BASE/kcc"
TMP="/$BASE/kcc_interop_$$"
mkdir -p "$TMP"
trap 'rm -rf "$TMP"' EXIT

# interop name caller_src callee_src
# Builds the all-clang oracle, then rebuilds the
   caller with kcc and (a second
# case) the callee with kcc, linking each against
   the clang-built other half.
interop() {
    local name="$1" caller="$2" callee="$3"
    printf '%s\n' "$caller" > "$TMP/caller.c"
    printf '%s\n' "$callee" > "$TMP/callee.c"

    # Oracle: both halves with clang.
    if ! clang -o "$TMP/oracle" "$TMP/caller.c" "
   $TMP/callee.c" 2>/dev/null; then
        printf "  %-26s SKIP (clang rejects)\n" "
   $name"
        return
    fi
    "$TMP/oracle" >/dev/null 2>&1
    local orc=$?

    clang -c "$TMP/callee.c" -o "$TMP/callee_clang
   .o" 2>/dev/null
    clang -c "$TMP/caller.c" -o "$TMP/caller_clang
   .o" 2>/dev/null

    local dir
    for dir in caller callee; do
        TOTAL=$((TOTAL + 1))
        if ! $KCC -c "$TMP/$dir.c" -o "$TMP/$dir""
   _kcc.o" >/dev/null 2>&1; then
            FAIL=$((FAIL + 1))
            printf "  %-26s FAIL (%s-kcc: compile)
   \n" "$name" "$dir"
            continue
        fi
        local other a b
        if [ "$dir" = caller ]; then a="$TMP/
   caller_kcc.o"; b="$TMP/callee_clang.o";
        else a="$TMP/caller_clang.o"; b="$TMP/
   callee_kcc.o"; fi
        if ! clang -o "$TMP/mixed" "$a" "$b" 2>/
   dev/null; then
            FAIL=$((FAIL + 1))
            printf "  %-26s FAIL (%s-kcc: link)\n"
    "$name" "$dir"
            continue
        fi
        "$TMP/mixed" >/dev/null 2>&1
        local mrc=$?
        if [ "$mrc" = "$orc" ]; then
            PASS=$((PASS + 1))
            printf "  %-26s ok (%s-kcc = %s)\n" "
   $name" "$dir" "$mrc"
        else
            FAIL=$((FAIL + 1))
            printf "  %-26s MISMATCH (%s-kcc=%s
   clang=%s)\n" "$name" "$dir" "$mrc" "$orc"
        fi
    done
}

echo "=== kcc<->clang ABI interoperability ==="
```

□

935a · used in 37l

The cases cover the parts of AAPCS64 that differ from kcc's earlier uniform convention: floating arguments and returns in `d` registers, small structs passed and returned in `x0`–`x1` (including a 16-byte struct argument that arrives in two registers and is reconstructed by the callee), a large struct argument passed by reference, and a large struct return through the `x8` indirect-result pointer.

⟨ *Interop: cases* 935b ⟩ ≡

```
interop "double-arg" \
    "double addd(double a, double b); int main(
   void){ return (int)(addd(1.5, 2.25) * 10); }" \
    "double addd(double a, double b){ return a + b
   ; }"
interop "five-doubles" \
    "double f(double a,double b,double c,double d,
   double e); int main(void){ return (int)f
   (1.0,2.0,3.0,4.0,5.0); }" \
    "double f(double a,double b,double c,double d,
   double e){ return a+b+c+d+e; }"
interop "mixed-int-double" \
    "double g(int a,double b,int c,double d); int
   main(void){ return (int)g(10,1.5,20,2.5); }" \
    "double g(int a,double b,int c,double d){
   return a+b+c+d; }"
interop "struct8-arg" \
    "struct P{int x;int y;}; int sump(struct P p);
    int main(void){ struct P q; q.x=10; q.y=20;
   return sump(q); }" \
```

```
    "struct P{int x;int y;}; int sump(struct P p){
    return p.x+p.y; }"
interop "struct16-arg" \
    "struct Q{long a;long b;}; long sq(struct Q q)
   ; int main(void){ struct Q q; q.a=100; q.b=23;
   return (int)sq(q); }" \
    "struct Q{long a;long b;}; long sq(struct Q q)
   { return q.a+q.b; }"
interop "struct8-ret" \
    "struct P{int x;int y;}; struct P makep(int a,
   int b); int main(void){ struct P q=makep(7,5);
   return q.x*q.y; }" \
    "struct P{int x;int y;}; struct P makep(int a,
   int b){ struct P p; p.x=a; p.y=b; return p; }"
interop "structbig-ret" \
    "struct B{int a;int b;int c;int d;int e;};
   struct B mk(int v); int main(void){ struct B b=
   mk(3); return b.a+b.b+b.c+b.d+b.e; }" \
    "struct B{int a;int b;int c;int d;int e;};
   struct B mk(int v){ struct B b; b.a=v;b.b=v;b.c
   =v;b.d=v;b.e=v; return b; }"
interop "structbig-arg" \
    "struct B{int a;int b;int c;int d;int e;}; int
    sb(struct B b); int main(void){ struct B b; b.
   a=1;b.b=2;b.c=3;b.d=4;b.e=5; return sb(b); }" \
    "struct B{int a;int b;int c;int d;int e;}; int
    sb(struct B b){ return b.a+b.b+b.c+b.d+b.e; }"
□
```

935b · used in 37l

⟨ *Interop: results* 936 ⟩ ≡

```
echo ""
echo "=== interop: $PASS ok, $FAIL fail (of $TOTAL
   ) ==="
[ "$FAIL" -eq 0 ]
□
```

936 · used in 37l

A. Development Roadmap

This appendix enumerates the complete sequence of implementation phases that take kcc from a bootable skeleton (Phase 0) to a self-hosting, conforming C17 compiler (Phase 8). Each phase lists the `.weft` files to be created or extended, the new tangled output files, and the acceptance criteria that must hold before the phase is considered complete.

A.1. Phase 0 — Project Skeleton

Goal. Prove the literate infrastructure works end to end: `make build` produces a native ARM64 binary that boots, prints a banner, and exits cleanly. No C source is compiled yet.

weft files.

- `kcc.weft` — master file, LaTeX preamble, includes.
- `literate/assembly.weft` — output routing (all output directives live here).
- `literate/foundations/01-normative-corpus.weft` — standards and citation conventions (prose only).
- `literate/foundations/02-conformance-model.weft` — conformance obligations (prose only).
- `literate/machine/07-arm64-conventions.weft` — register map, Apple ABI, `FUNC`/`ENDFUNC`/`SYSCALL` macros.
- `literate/machine/08-syscalls.weft` — `exit`, `write`, `read`, `open`, `close`, `mmap` wrappers.
- `literate/machine/09-memory-management.weft` — arena allocator (bump pointer over `mmap` chunks).
- `literate/machine/11-string-operations.weft` — `strlen`, `memcpy`, `memcmp`, `memset`.
- `literate/machine/12-io.weft` — buffered file I/O (4 KiB static buffer, flush/putc/puts).
- `literate/driver/43-main.weft` — `_main` entry point, argument parsing, banner.

Tangled output. `main.s`, `syscalls.s`, `arena.s`, `strings.s`, `io.s`.

Acceptance.

1. `make tangle` succeeds with no errors.
2. `make build` assembles and links with no errors.
3. `gen/kcc` prints its banner and exits 0.
4. `gen/kcc hello.c` echoes the source file name.

Status. Complete.

A.2. Phase 1 — Data Structures and Lexer Foundation

Goal. Read a C source file from disk and tokenize it into a stream of C17 preprocessing tokens (§6.4).

weft files.

- `literate/machine/10-data-structures.weft` — hash table (open addressing, FNV-1a) and dynamic vector, both arena-allocated.
- `literate/machine/14-source-manager.weft` — `mmap`-based file reading, source location tracking (file/line/column as a packed 64-bit value).
- `literate/frontend/17-lexer.weft` — character classification tables, tokenizer for all C17 token classes: identifiers, keywords, integer/floating constants, string/char literals, punctuators.

New tangled files. `hashtab.s`, `vec.s`, `strtab.s`, `source.s`, `lexer.s`, `lexer_tables.s`.

Acceptance.

1. `gen/kcc --dump-tokens hello.c` prints every token with its source location.
2. All 44 C17 keywords are recognized.
3. Integer, floating, character, and string literals are correctly classified (value parsing deferred to Phase 5).
4. Unterminated strings and unrecognized characters produce diagnostics on stderr.

Status. Complete.

A.3. Phase 2 — Complete Lexer and Diagnostics

Goal. Finish the tokenizer to full C17 conformance and build the diagnostic engine required by §5.1.1.3.

weft files.

- `literate/machine/13-diagnostics.weft` — diagnostic engine: error/warning/note classification, source-location printing ("file:line:col: error: ..."), error-count tracking, fatal-error limit.
- `literate/frontend/15-source-encoding.weft` — Phase 1 of translation: trigraph replacement, source character set mapping (§5.1.1.2 phase 1).
- `literate/frontend/16-line-splicing.weft` — Phase 2 of translation: backslash-newline splicing (§5.1.1.2 phase 2).
- `literate/verification/44-test-harness.weft` — test runner: discover tests, run each, report pass/fail.
- `literate/verification/45-conformance-tests.weft` — initial lexer conformance tests.

New tangled files. `diag.s`, `test_runner.s`, `test_lexer.s`.

Acceptance.

1. Diagnostics follow the format `file:line:col: error: message`.
2. Trigraphs and line splicing are handled before tokenization.
3. `make test` runs the test suite and reports results.
4. The lexer correctly tokenizes the GCC torture test suite's lexer-visible subset (no preprocessing yet).

Status. Complete.

A.4. Phase 3 — Preprocessor

Goal. Implement the full C17 preprocessor (§6.10), covering translation phases 3 and 4.

weft files.

- `literate/frontend/18-preprocessor.weft` — macro definition (`#define`), macro expansion (object-like and function-like with stringification and token pasting), conditional compilation (`#if`/`#ifdef`/`#elif`/`#else`/`#endif`), file inclusion (`#include`), line control (`#line`), error directive (`#error`), pragma handling (`#pragma`, including `_Pragma` operator), null directive.
- `literate/frontend/19-string-literals.weft` — Phase 5: escape sequence processing, execution charset mapping.
- `literate/frontend/20-concatenation.weft` — Phase 6: adjacent string literal concatenation.

New tangled files. `preprocessor.s`.

Acceptance.

1. `gen/kcc -E file.c` outputs the preprocessed token stream (translation phases 1–6 complete).
2. Recursive and self-referential macros terminate correctly per §6.10.3.4.
3. `#include` searches `-I` directories and the system include path.
4. String concatenation across macro boundaries works.
5. Predefined macros (`__FILE__`, `__LINE__`, `__DATE__`, `__TIME__`, `__STDC__`, `__STDC_VERSION__`) are set correctly.

Status. Complete (core directives: `#define` object-like and function-like, `#include` of a quoted header resolved relative to the including file, `#if`/`#ifdef`/`#endif`, `#error`; predefined macros). A `-I` system search path remains for Phase 9.

A.5. Phase 4 — Parser

Goal. Build a recursive-descent parser that consumes the preprocessed token stream and produces a typed abstract syntax tree (AST) for every C17 construct.

weft files.

- `literate/frontend/21-parser.weft` — parser core: recursive descent infrastructure, AST node allocation (arena), AST node types, error recovery (synchronization tokens).
- `literate/frontend/22-declarations.weft` — declaration parsing: declaration specifiers, declarators (including abstract and function), the typedef hack (lexer feedback), `struct`/`union`/`enum` definitions, `_Static_assert`.
- `literate/frontend/23-expressions.weft` — expression parsing: precedence climbing for all C17 operators (15 precedence levels), postfix operators (call, subscript, member access, increment/decrement), unary operators, cast expressions, compound literals, `_Generic` selection, `sizeof`/`_Alignof`.
- `literate/frontend/24-statements.weft` — statement parsing: compound, `if`/`else`, `switch`/`case`/`default`, `while`, `do`/`while`, `for`, `goto`, `continue`, `break`, `return`, labeled statements, expression statements.

New tangled files. `parser.s`, `declarations.s`, `expressions.s`, `statements.s`.

Acceptance.

1. `gen/kcc --dump-ast file.c` prints the AST in a readable tree format.
2. All syntactically valid C17 programs parse without error.
3. Syntax errors produce useful diagnostics with the offending token highlighted.
4. The typedef ambiguity is resolved correctly (e.g., `T * x;` parsed as declaration when `T` is a typedef name).
5. `sizeof(int)` and `sizeof x` both parse.

Status. Complete.

A.6. Phase 5 — Type System and Semantic Analysis

Goal. Implement the C17 type system and semantic analysis pass that decorates the AST with type information and checks all constraints.

weft files.

- `literate/semantics/25-symbol-table.weft` — four namespaces (§6.2.3): ordinary identifiers, tags, labels, struct/union members. Scope management: file, function, block, function-prototype scopes. Linkage resolution (external, internal, none).

- `literate/semantics/26-type-system.weft` — internal type representation: basic types, qualified types, pointer, array (fixed, variable-length, incomplete), function types, struct/union/enum types. Type compatibility (§6.2.7) and composite type construction.
- `literate/semantics/27-type-conversions.weft` — integer promotions (§6.3.1.1), usual arithmetic conversions (§6.3.1.8), implicit conversions, explicit casts, pointer conversions.
- `literate/semantics/28-type-checking.weft` — constraint checking for every expression operator per §6.5: operand type requirements, result types, lvalue-to-rvalue conversion, array-to-pointer decay, function-to-pointer decay.
- `literate/semantics/29-initializers.weft` — initializer processing (§6.7.9): designated initializers, brace elision, string initializers for arrays, recursive struct/union/array initialization.
- `literate/semantics/30-constant-expressions.weft` — compile-time evaluation (§6.6): integer constant expressions, arithmetic constant expressions, address constants.

New tangled files. `symtab.s`, `types.s`, `conversions.s`, `sema.s`.

Acceptance.

1. Type errors produce clear diagnostics citing the relevant Standard clause.
2. `gen/kcc --dump-types file.c` prints every expression with its resolved type.
3. Implicit conversions (integer promotions, UAC, pointer decay) are inserted automatically.
4. Struct member access chains resolve correctly through anonymous structs/unions.
5. `enum` constants are typed as `int` per §6.7.2.2.
6. Initializer lists with designators and brace elision produce the correct initialized object.

Status. Complete (core type checking for integer types, pointers, functions; `--dump-types` and `--check` flags).

A.7. Phase 6 — Intermediate Representation and Optimization

Goal. Lower the typed AST into a linear intermediate representation (IR) with virtual registers, and implement essential optimization passes.

weft files.

- `literate/ir/31-ir-design.weft` — IR design: linear IR with virtual registers, basic blocks, control-flow graph, SSA-like properties (each virtual register assigned once). IR instruction set: arithmetic, comparison, branch, call, load, store, alloca, phi, cast.
- `literate/ir/32-ir-builder.weft` — AST-to-IR lowering: expression evaluation, short-circuit boolean operators, `switch` lowering (jump table or if-chain), loop lowering, struct passing/returning.
- `literate/ir/33-ir-validation.weft` — IR well-formedness checks: every use dominated by its definition, consistent types, correct basic-block termination, reachability analysis.
- `literate/optimization/34-ir-passes.weft` — pass infrastructure: pass manager, iteration control, analysis caching.
- `literate/optimization/35-peephole.weft` — algebraic simplifications: constant folding, strength reduction, identity elimination (`x+0`, `x*1`).
- `literate/optimization/36-dce.weft` — dead code elimination: remove instructions whose results are never used, remove unreachable basic blocks.

New tangled files. `ir.s`, `opt.s`.

Acceptance.

1. `gen/kcc --dump-ir file.c` prints the IR in a readable textual format.
2. IR validation passes with no errors for all valid programs.
3. Constant expressions evaluate at compile time.
4. Dead stores and unused computations are eliminated.
5. The IR correctly models all C17 control flow, including `goto`, `switch` fall-through, and `setjmp`/`longjmp` (treated as opaque calls).

Status. Complete (constant folding, DCE, pass manager; `--dump-ir` flag).

A.8. Phase 7 — ARM64 Backend

Goal. Generate native ARM64 machine code and emit Mach-O object files directly — no external assembler needed. This is the self-referential core: ARM64 assembly generating ARM64 machine code.

Strategy. Phase 7a uses a simplified *stack-slot allocator* where every virtual register lives on the stack. Scratch registers `x9`, `x10`, `x11` shuttle values between stack slots and ALU operations. This produces correct but suboptimal code quickly. Phase 7b (future) will introduce linear-scan register allocation.

weft files.

- `literate/backend/37-codegen.weft` — IR-to-ARM64 lowering: stack-slot strategy, instruction dispatch for all IR opcodes (arithmetic, logical, compare, memory, branch, call, return), prologue/epilogue emission, BB label tracking, forward-branch patching.

- `literate/backend/38-emit-arm64.weft` — ARM64 instruction binary encoding: code buffer management, data processing (register and immediate forms), logical operations, compare and conditional select (CSINC), load/store (unsigned offset, STP/LDP), branch (B, BL, B.cond, CBZ, CBNZ, RET), ADRP/ADD pair for relocations.
- `literate/backend/39-emit-macho.weft` — Mach-O object file emission: header, load commands (`LC_SEGMENT_64`, `LC_SYMTAB`, `LC_DYSYMTAB`), `__text` section, symbol table (nlist_64), string table, relocation entries (`ARM64_RELOC_BRANCH26`).
- `literate/backend/40-linker-info.weft` — relocation and symbol table management: add/lookup symbols, add relocations, extern deduplication.
- `literate/driver/42-driver.weft` — `-c` flag dispatch: pipeline orchestration from source through lex, parse, sema, IR, codegen, to Mach-O output.

New tangled files. `codegen.s`, `emit_arm64.s`, `emit_macho.s`, `linker.s`, `driver.s`.

Acceptance.

1. `gen/kcc -c file.c -o file.o` produces a valid Mach-O object file.
2. `ld file.o -o file -lSystem ...` links the object file into a working executable.
3. Generated code passes the C17 conformance test suite (integer arithmetic, control flow, function calls, structs, pointers, arrays).
4. Calling convention is correct: kcc-compiled code can call system libraries and vice versa.
5. The object file is accepted by Apple's `ld` linker without warnings.

Status. Complete. All IR opcodes have codegen handlers; constants, arithmetic, locals, control flow, loops, function calls, global variables, break/continue, string literals, `printf`, bitwise/comparison/logical operators, pointers, cast, negation, unsigned operations, and type extensions all compile, link, and run correctly. Valid Mach-O with `LC_BUILD_VERSION`.

Completed milestones.

- ✓ Return constants, arithmetic, locals, `if`/`else`.
- ✓ `while`, `do`/`while`, `for` (with explicit `i = i + 1`).
- ✓ Function calls: multi-argument, variadic, external (`printf`).
- ✓ Global variables via `ADRP+ADD` relocation.
- ✓ String literals in `__text` section with cstring fixups.
- ✓ Bitwise operators, comparisons, logical `&&`/`||`, unary `!`.
- ✓ Pointers (address-of, dereference), cast expressions, negation.
- ✓ Comma operator, `break`/`continue`, nested calls.
- ✓ Post/pre increment and decrement (`i++`, `++i`, `i--`, `--i`): load-modify-store sequence for both locals and globals.
- ✓ Compound assignment operators (`+=`, `-=`, `*=`, `/=`, etc.) for both local and global variables.
- ✓ Ternary operator (`?:`): alloca rendezvous pattern ensures both branches store to a shared slot; merge block loads the result.
- ✓ `switch`/`case`/`default`: three-pass lowering (create BBs, dispatch chain, bodies) with C fall-through semantics and `break` via the loop stack.
- ✓ `sizeof`: lowered to `IR_CONST` with the operand type's `TY_SIZE`; both type and expression forms supported.
- ✓ Arrays (`a[i]`): `IR_ALLOCA` with slot count, `IR_GEP` for address arithmetic, frame pre-scan for alloca storage layout.
- ✓ Structs (`s.x`): field-count alloca, field-index lookup via AST field chain walk, `IR_GEP` for member access. Fixed sema to register struct tags from `AST_TYPE_STRUCT` wrappers and to look up tags for forward references.
- ✓ Multi-variable declarations (`int a=3, b=4;`): compound-statement appender now walks the `F4` tail of multi-declarator chains so that subsequent items link after the last declarator instead of overwriting the chain.
- ✓ `IR_TRUNC` and `IR_BITCAST`: codegen handlers for type-narrowing casts (mask to target width) and same-width reinterpretation. Cast expressions in the IR builder now emit `IR_TRUNC` when the target type is narrower than the source.
- ✓ Remaining IR opcodes: `IR_UMOD` (unsigned modulo via `UDIV+MSUB`), `IR_CMP_ULT` and `IR_CMP_ULE` (unsigned comparisons via `CSET` with `LO`/`LS` conditions), `IR_SEXT` (sign extension via hardcoded `SBFM` instruction words), `IR_ZEXT` (zero extension via hardcoded `UBFM` instruction words). The IR builder's binary handler remaps signed opcodes to unsigned variants when the left operand's type has `TY_F0`=0.

A.9. Phase 8 — Self-Hosting

Goal. Demonstrate that kcc can compile a non-trivial C program — ideally a C subset of itself or an equivalent program — and produce correct output. This is the ultimate proof that the compiler is correct and complete.

Strategy. Phase 8 proceeds in three stages. Stage 8a fixes missing C features and hardens the backend by compiling increasingly complex programs. Stage 8b writes a C implementation of a kcc subsystem (the lexer or the arena allocator) and compiles it with kcc itself. Stage 8c performs differential testing against `clang` and, if feasible, a

full bootstrap where kcc compiles a stage-1 C compiler that then compiles itself.

Acceptance.

1. At least one non-trivial C program (> 1000 lines) compiles and runs correctly under kcc.
2. Differential testing against `clang` reveals no behavioral discrepancies on conforming programs.
3. The §J.3 implementation-defined behavior document is complete (Appendix A).
4. The UB policy matrix is complete (Appendix B).
5. `make pdf` produces a typeset document that serves as a comprehensive textbook on both compiler construction and ARM64 systems programming.

Status. Stage 8a in progress. 201 codegen tests passing. The compiler handles integer arithmetic, control flow, loops, function calls (direct and indirect), structs (nested, arrays), pointers, pointer-to-pointer and arrays of pointers, arrays, brace-enclosed array and struct initializers (local and file-scope), static locals, global variables, enums, switch/case, sizeof, character literals, string literals, void functions, forward-declared prototypes, function pointers, the preprocessor on the `-c` path (`#define`, `#ifdef`, and `#include` of a source-relative header), and multi-file compilation (several translation units in one invocation, sharing functions and `extern` globals across objects). A 1,130-line C test suite compiles and runs correctly: hash tables, arena allocators, recursive-descent parsers, linked-list manipulation, tree evaluators, and the sieve of Eratosthenes. Stage 8b has begun: kcc's own arena allocator, re-written in C and compiled by kcc, replaces the hand-written assembly arena in a `kcc_selfhost` binary that passes all 201 codegen tests (Chapter 48); and a 250-line C tokenizer, compiled by kcc, self-tests green and matches `clang` (Chapter 77), a program whose size flushed out two codegen bugs now fixed.

Completed milestones.

- ✓ All-spill backend: every IR opcode has a codegen handler; valid Mach-O with `LC_BUILD_VERSION`.
- ✓ Pointer arithmetic: scale factor clamped to `max(TY_SIZE, 8)` for the all-spill model.
- ✓ Struct support: nested struct access (recursive `build_member_addr`), struct arrays with correct `TY_SIZE/8` alloca slots.
- ✓ Global initializers: emit `IR_GLOBAL_REF` + `IR_STORE` at top of `main()` entry block.
- ✓ Local array decay: `build_ident` detects arrays via bit-63 pointer tag + AST field check, returns alloca address without loading.
- ✓ Function pointers: grouped-declarator parsing `(*name)(params)`, `IR_CALL_IND` opcode with `BLR` emission, function-to-pointer decay, DCE marking, linker symbol precedence fix.
- ✓ Void functions, forward-declared prototypes, character literals, and pointer-to-pointer: verified end to end by direct compilation. The `IR_RET` handler already distinguishes `void`, prototypes are skipped by the IR builder, and multi-level indirection lowers correctly.
- ✓ Arrays of pointers (`int *a[N]`, `char **argv`): the pointer-variable tag is now taken from the declared kind rather than the array element, so a pointer-array is indexed from its alloca base via GEP instead of being mis-lowered as a scalar pointer.
- ✓ Brace-enclosed initializer lists: positional `IR_GEP`+`IR_STORE` per element for both arrays and flat structs, implicit array bounds inferred from the element count, zero-fill of the unspecified tail, and file-scope lookup tables via `_kcc_ir_emit_global_inits` (BSS supplies the zero image). String initializers for char arrays and designated initializers remain for Phase 9.
- ✓ Static locals: hoisted to a per-declaration global named `sl_`
  metanode address by `_kcc_ir_static_name`, so no stack alloca is emitted. A per-function `statics` table redirects identifier and assignment references onto the mangled symbol (shadowing any like-named file-scope global), and a second pass in `_kcc_ir_emit_global_inits` emits the one-time initialiser at program start. Nested-block statics and static aggregates remain for Phase 9.
- ✓ Preprocessor on the `-c` path: object- and function-like `#define`, `#ifdef`/`#ifndef` conditionals, and `#include` of a quoted header resolved relative to the including file's directory (then the working directory). `_kcc_sys_open` was fixed to collapse the kernel's carry-flag `errno` to -1 so a missing header produces a clean diagnostic instead of crashing on a bogus descriptor. A `-I` system search path is left to Phase 9.
- ✓ Multi-file compilation: `_kcc_driver_compile` now returns a status so `_kcc_driver_compile_all` can loop it over every source argument, writing one object per input (`.c`→`.o`). `extern` data is honoured: an `extern` declaration reserves no storage, and a reference to a global not defined in the current unit is emitted as an undefined external symbol (via a `defined_globals` set feeding the `IR_GLOBAL_REF` handler's choice between `linker_global_sym` and `linker_extern`), so two objects can share a global without colliding. Automatic linking (invoking `ld`) is left to Phase 9. A later refinement (Phase 9) closed the one hole this left: a translation unit that does not define `main` now initialises its own file-scope globals through a synthesised `kcc.module_init` function registered in `(__DATA,__mod_init_func)`, which `dyld` runs before `main` (§29.1.1, §40.3.2); previously such a unit's initializers, being emitted as stores injected into `main`, never ran.

✓ 201 codegen tests covering all features (11 Phase 8 test groups: nested structs, struct arrays, global init, pointer arithmetic, pointer arrays, initializer lists, static locals, preprocessor, multi-file, self-hosting patterns, function pointers).

Remaining work — Stage 8a: missing C features.
The following features are needed before kcc can compile a realistic subsystem of itself.

- **Typedef support.** The parser recognizes `typedef` but codegen has not been stress-tested with typedef'd structs and function pointer types.
- **String initializers for char arrays.** `char s[] = "abc"` must copy the literal bytes into the array rather than storing the string pointer.
- **Automatic linking.** kcc now emits one object per source, but the resulting `.o` files are still handed to `ld` by the caller; invoking the linker from the driver is a Phase 9 convenience.

Pointer-to-pointer, arrays of pointers, void functions, forward-declared prototypes, character literals, brace-enclosed initializer lists (array and flat-struct, local and file-scope), static locals, and multi-file compilation (multiple translation units with shared `extern` data) were originally listed here; all are now implemented and covered by the codegen suite (see the completed milestones above).

Stage 8b: subsystem in C.

✓ The kcc arena allocator is re-implemented in C (Chapter 48), a faithful port of the assembly arena of Chapter 8 with the same exported symbols and 24-byte chunk layout.

✓ `make selfhost` compiles it with kcc (`kcc -c arena.c -o arena_c.o`), relinks a `kcc_selfhost` binary with the hand-written `arena.o` replaced by the C-compiled object, and runs the full code-generation suite against it.

✓ Behaviour is identical: `kcc_selfhost` passes all 201 codegen tests, exercising the C arena's fast and slow paths across every allocation the compiler makes. kcc has compiled a correctness-critical piece of itself.

✓ A branch-heavy tokenizer in C (Chapter 77) is compiled by kcc, self-tests green, and agrees with `clang` (`make selfhost-lexer`). Building it exposed and fixed two real codegen bugs a 250-line program reaches but the unit tests do not: the basic-block label table overflowed on large functions (fixed-size open-addressed hash → sized to the block count), and address-of assumed a bare identifier (`&a[i]` now dispatches on operand shape).

- Remaining: port more of the front-end and, eventually, compile enough of kcc to bootstrap.

Stage 8c: validation.

✓ Differential testing (Chapter 78, `make difftest`): a 97-program corpus is compiled by both kcc and `clang`, run freestanding, and compared with `clang` as the oracle. kcc agrees on every program with zero miscompilations and no remaining expected divergence. The sweep drove a string of fixes—function pointers in `struct` fields, 32-bit integer wraparound (`_kcc_ir_norm32`), enumerators with explicit values, `sizeof` of a whole array, the comma operator in a `for` clause *and* labels in a compound (two distinct callee-saved-register bugs, in `parse_expr` and `parse_stmt`), pointer difference, `goto`/labels lowered end to end (§29.4), multi-dimensional arrays laid out flat with a recursive slot count (`_kcc_ir_type_slots`), and—last of the sweep's open constructs—`struct` return by value via a hidden result pointer (§29.6), which uncovered a latent `(void)`-parameter miscount along the way. The harness needs no hand-computed expected values, so it scales to programs far more complex than the code-generation suite's.

- Bootstrap test: write a minimal C compiler (or expression evaluator) in C, compile it with kcc, then use it to compile itself. Compare the two binaries.
- Performance characterization: measure compilation speed and generated-code quality against `clang -O0`.
- Appendix A: § J.3 implementation-defined behavior.
- Appendix B: undefined-behavior policy matrix.

A.10. Phase 9 — Production Compiler

Goal. Elevate kcc from a working subset compiler to a conforming, efficient, and usable C17 implementation suitable for compiling real-world programs. Phase 8 proves correctness on a significant codebase; Phase 9 fills every remaining gap in the language, the ABI, the optimizer, and the tooling.

> **North Star**
> A conforming C17 compiler a developer can actually use: it compiles real-world code, reports *every* error clearly, produces debuggable executables, and generates efficient code—while remaining a single, self-hosting literate program.

Phase 9 is where the compiler crosses from "proven correct on a large codebase" to "pleasant to use." The five thematic stages and their current standing:

Stage	Theme	Where it stands
9a	Complete type system	`double`/`float`/unions done; `_Bool`,
9b	Full ABI compliance	AAPCS64, struct-by-value, variadic;
9c	Register allocation	local linear-scan, caller & callee-save
9d	Optimization	CSE, copy-prop, strength reduction,
9e	Tooling & diagnostics	auto-link, headers, errors, `-Wall`, DW

Legend: ✓ DONE shipped and tested ⊙ PARTIAL partially landed ○ OPEN not begun. *Impact / effort:* ●●● high ●●○ medium ●○○ low.

Strategy. Phase 9 is organized into five *thematic* stages. Stage 9a completes the C17 type system (floating point, unions, qualifiers). Stage 9b makes the calling convention fully ABI-compliant (struct-by-value, stack arguments, variadic definitions). Stage 9c replaces the all-spill model with a real register allocator. Stage 9d adds optimization passes that bring generated-code quality within striking distance of `clang -O1`. Stage 9e delivers the tooling and diagnostics expected of a production compiler. These groupings organize the *material*; they are no longer the order of work. With 9a–9c largely landed, the remaining open items are re-prioritized by impact toward a genuinely *usable* compiler in the go-forward plan below (§A.10.1)—diagnostics and high-frequency language features first, optimization and breadth later.

Status. Stage 9a is well advanced—`double` floating point (§22.10), `unions`, and single-precision `float` (rounded at its boundaries) are done; `_Bool`, qualifiers, `long double`, and `_Generic` remain. Stage 9b has struct pass/return by value working (§29.3), a byte-accurate § 6.7.2.1 struct/union layout with standard padding and alignment (so `sizeof` and member offsets are conformant), more-than-eight-argument stack passing, variadic function definitions (§29.9.13), and now a fully AAPCS64-compliant calling convention—floating arguments and returns in `d0`–`d7`, small structs passed and returned in `x0`–`x1` (a 9–16-byte struct argument arrives in two registers and is rebuilt by the callee), large struct arguments by reference, and large struct returns through the `x8` indirect-result pointer—validated in both directions against `clang` by a mixed-object interoperability suite (Chapter 79), which passes on all sixteen checks. kcc's Stage 9b ABI work is therefore complete: its objects link and run correctly with `clang`'s. Stage 9c's local register allocator has landed (§35.9); Stages 9d–9e (optimization, tooling) are largely open. A precise ledger of what is done and what is left appears in the production-readiness assessment (§A.12).

A.10.1. Reprioritized Go-Forward Plan

The thematic stages above group the material; this list is the authoritative *order* in which the remaining open items should be tackled. It is re-ordered by impact toward a usable production compiler rather than by stage number, on one principle: *remove what stops a user from using kcc in practice before adding polish or breadth*. Correctness and self-hosting are already in hand, so the leverage now is in usability and coverage, not speed.

#	Open item	Impact	Effort	Home
1	Multi-error recovery & diagnostics ✓ DONE	●●●	●●○	9e
2	`-Wall` warnings ✓ DONE	●●○	●○○	9e
3	Designated initializers ✓ DONE	●●●	●●○	2.9a
4	Bit-fields ✓ DONE	●●●	●●○	9a
5	_Bool normalize, `char s[]="…"` ✓ DONE	●●○	●○○	9a
6	DWARF debug info ✓ DONE	●●○	●●●	9e
7	Harden vs. malformed input ⊙ PARTIAL	●●○	●●○	9e
8	Optimizer & global/FP reg-alloc	●●○	●●●	9c–9d
9	Tail C17 (_Generic, `long double`)	●○○	●●○	9a
10	Second target (x86-64 / ELF)	●○○	●●●	new

The table is the scan; the notes below are the *why*.

1. **Multi-error recovery and precise diagnostics** (Stage 9e). *Essentially complete.* The single biggest usability jump. Parser recovery turned out to be already working—`kcc_parse_sync` skips to the next `;`/`}`/EOF, the driver defers its error check until after parse *and* sema, and one compile already reports several syntax errors with line and column. The real gap was that semantic analysis emitted *no* diagnostics at all (it silently accepted undeclared names, so the failure surfaced only at link time). Now landed: an "`use of undeclared identifier 'metaname'`" error for any bare identifier used as a value—kept lenient only in call position, for implicit function declarations—and specific "`expected 'X'`" messages for the common punctuators (was a bare "expected token"). A negative-test harness (`make diagtest`, §12.9) guards it. A second round added the semantic checks that share the same quoted-message helper: calling a non-function, member access on a non-aggregate, a nonexistent member ("`no member named 'X'`"), and redefinition of a block-scope identifier or parameter (file-scope tentative definitions, prototype-then-definition, and inner-scope shadowing are all correctly left alone). A third round added the two pieces named here as the remaining work. *Richer messages:* every diagnostic now prints the offending source line followed by a ^ caret under the reported column—the `clang`/`gcc` presentation—rendered straight from the live source buffer and written so that a missing buffer or an out-of-range line simply omits the snippet rather than faulting. Adding the caret immediately exposed a latent one-token location error (an identifier node was stamped with the *following* token's column because it was allocated after the token was consumed); fixing the allocation order put the caret exactly under the name. *Type-mismatch on return:* the two constraints of § 6.8.6.4—no value returned from a `void` function, and no bare `return` in a non-`void` function—are now enforced as errors, which is standard-conformant (both are *shall* constraints) and, because no valid program violates them, costs nothing in self-hosting or the differential corpus. `diagtest` now runs eighteen cases, and kcc matches `clang`'s error *count* on mixed syntax/semantic inputs. *Deliberately deferred to item 2:* pointer/integer conversion mismatches, which `clang` itself treats as `-Wall` warnings rather than errors, and whose false-positive risk against pointer-heavy systems C argues for the warning machinery, not the error path.

2. **`-Wall`-equivalent warnings** (Stage 9e). *Core complete.* The whole family sits behind a single default-off `-Wall` flag (`_kcc_warn_all`, set while the driver scans arguments), and warnings go through the warning path of the diagnostic engine, which counts but does not abort—so a compile without `-Wall` is byte-for-byte what it was before, and a warning never

turns a valid program into a failed build. Two of the canonical warnings are landed. *Implicit function declaration:* an undeclared name used in call position, previously accepted silently for legacy leniency, now draws `implicit declaration of function ' metaname'` under `-Wall` (the compile still proceeds). *Unused variable:* a block-scope local that is declared but never referenced draws `unused variable ' metaname'`, checked as each block and `for` scope is left; a variable counts as referenced the moment it appears in any expression, so the check never fires on a genuinely-used name, and parameters—which live in the function scope, not the body's—are excluded, matching `clang`'s placement of unused parameters in `-Wextra`. A dedicated `make warntest` harness checks all three properties at once: the warning appears with `-Wall`, the program still compiles, and the very same source is silent without the flag. A third warning, `-Wreturn-type`, is now landed on top of a genuine control-flow analysis (§12.9 describes the harness; `kcc_sema_falls_through` the analysis): a non-void function is flagged only when control can be *proven* to run off its end, so an exhaustive `if`/`else`, an infinite loop, an exhaustive returning `switch`, and a tail call to a `_Noreturn` function are all correctly left quiet, and `main` is exempt. A fourth, `-Wsign-compare`, flags a relational or equality comparison that mixes a signed and an unsigned integer—the `u < -1` surprise—while staying quiet when an operand is a known non-negative constant, so the ubiquitous `i < 10` loop test draws nothing. The fifth, `-Wint-conversion`, completes the canonical set: mixing an integer and a pointer across an assignment, initialiser, or `return` without a cast (`int *p = 5`) is flagged, with the integer constant zero exempted as the null pointer constant and compound assignment left to pointer arithmetic. All five warnings are false-positive-free, carry the same source-line caret as the errors, and are guarded by `make warntest`. *Optional later:* `-Wparentheses`, `-Wuninitialized`, and argument-position conversion checks once the function type descriptor carries its parameter types.

3. **Designated initializers** (Stage 9a). *Landed.* Pervasive in real systems C (`{.field = v}`, `[i] = v`); without them a great deal of ordinary code simply will not compile. A single-level designator now reuses the positional lowering unchanged: the write loop keeps a running position and a designator merely resets it (an array index directly, a member name through `kcc_ir_field_index`), after which plain elements resume from the new position exactly as §6.7.9 prescribes. Local and file-scope aggregates, structs, unions, arrays, sparse and out-of-order lists, and mixed positional/designated forms all agree with `clang` in the differential suite. *Remaining:* nested designator chains (`.a.b`, `[i].f`).

4. **Bit-fields** (Stage 9a). *Landed.* Equally pervasive in systems and device-register headers. `_kcc_type_layout` now packs consecutive same-type bit-fields into one container least-significant-bit first, recording each field's container offset and packed (`bit_offset, width`) on its node; reads and writes are *synthesised* from ordinary shifts, masks, and an or—no new backend instructions—and the read-modify-write is reused by the initializer path so positional, designated, local, and file-scope bit-field initialisation all agree with `clang` (packing, signed sign-extension, container crossing, and truncation included). *Not modelled:* mixed-type packing and zero-width separators.

5. **`_Bool` normalization and `char s[] = "..."`** (Stage 9a). *Landed.* Two cheap, high-frequency gaps. A conversion to `_Bool` now normalises any non-zero value to `1` (an `x != 0` folded into `_kcc_ir_convert`, the integer cast path, and file-scope initialisation), and a `char` array initialised from a string literal copies the literal's bytes into its storage—zero-filling or relying on BSS for the terminating NUL—rather than storing a pointer, while a `char*` target still correctly takes the address. `sizeof` of an implicitly-sized array (`char s[] = "abc"`, `int a[] = {1,2,3}`) is correct too: `_kcc_sema_infer_array_len` completes the array type's length from its initializer before the type is resolved, so every use sees the real length. Only the multi-dimensional case (`int m[][3]`) remains, awaiting nested-aggregate initialization—a gap present for explicit sizes as well.

✓ **DWARF debug information** (Stage 9e, done). The function-granularity line table and `subprogram` DIEs now land under `-g` (Chapter 42)—additive, emitting `__debug_*` sections without touching code generation. Statement-level rows and variable DIEs are the follow-ups that unlock full `lldb`/`gdb` stepping and inspection.

6. **Hardening against malformed input** (new, *in progress*). A production compiler must reject adversarial or truncated source cleanly rather than crash. Ad-hoc fuzzing closed the first crashes: the lexer cleared its cursor at end-of-file (a null cursor slipped under the end-of-buffer bound check and faulted—this alone crashed `#`, `#include`, and `#define` at EOF); the Mach-O output buffer was a fixed 64 KB that overflowed on large functions (now sized to the object's content with a capacity guard); and the add/subtract-immediate encoders overflowed their 12-bit field into the shift bit on frames or stack-slot offsets past 4095, emitting a tagged ADDG/SUBG that trapped at run time (now split into a shifted-high plus low part). A follow-up closed the last frame bound: `ldr`/`str` to a stack slot beyond the scaled 12-bit reach (32760 bytes) now materialises the slot address in a scratch register, so a function with an arbitrarily large frame—tens of thousands of spilled temporaries—compiles correctly. What remains is not correctness but speed: very large symbol counts run in super-linear time.

7. **Optimization and global register allocation** (Stages 9c–9d, *in progress*). Cross-block (global) allocation now runs in every function—loop-free bod-

ies keep an escaping value in a register through `if`/`else`, `switch`, and the short-circuit/ternary operators ($\sim$22% fewer frame accesses on a loop-free corpus), while a function with a back-edge gives each escaping value a callee-saved register for the whole function—correct across the loop without liveness because every vreg is single-def ($\sim$36% fewer frame accesses on a loopy corpus). On the IR side, constant folding now covers binary, unary (`NEG`/`NOT`), and—load-bearing—the `SEXT`/`ZEXT`/`TRUNC` width casts the front end inserts on every integer operation, so a whole constant expression collapses to one value instead of stalling at the first cast ($\sim$55% fewer instructions on a constant-expression corpus). Enabling the unary fold surfaced and fixed a dead-code-elimination bug—its mark pass read the stale operands of already-deleted `NOP` shells, keeping dead code alive and corrupting a type field—and hardened `--dump-ir` to survive malformed IR. Common subexpression elimination now runs too: a local value-numbering pass that reuses an earlier pure computation, and crucially an earlier *load* of the same address (invalidated by any intervening store or call), so the pervasive reload-a-local pattern collapses ($\sim$19% fewer instructions on a redundant-expression corpus). *Mem2reg*—promoting scalar `alloca` locals to registers—now lands too, and does so *unified* with the allocator rather than as the competing pre-reservation that made the first attempt net-negative. The insight: the allocator already gives an `alloca`'s vreg a register (today it holds the slot address), so a scalar local whose address never escapes needs no new register—that same register is simply reinterpreted as holding the local's contents, and the loads and stores become register moves. Because nothing new is reserved, promotion adds zero register pressure, and a local is promoted exactly when the shared scan already had a register for it (the excess stays in memory, as before). A promoted local also vacates its frame slot, so loop bodies become memory-free. Floating-point values now participate in allocation as well, in a *dedicated* register bank: a second linear scan—an exact mirror of the integer one—hands floating vregs `d16`–`d31` (caller-saved) or `d8`–`d15` (callee-saved, spilled by the prologue), so they neither steal the general registers nor round-trip through `FMOV` on an FP-only chain. What remains is quality, not enablement: a tighter loop reservation (real liveness) and the remaining classic IR passes (copy propagation, strength reduction).

8. **Tail C17 features—`_Generic`, `long double`, `volatile`** (Stage 9a). `volatile` is now honoured for every lvalue access—scalar, global, pointer dereference, array element, struct/union member, and (as of milestone M1) a volatile-declared bit-field, whose synthesised read-modify-write now carries the volatile bit on its container load and store—a soundness requirement once the optimizer was added: it must not merge or drop a volatile load. The qualifier reaches the IR at each access and CSE/DCE respect it. (What remains is propagating `volatile` from a qualified *object* through member access, a gap shared by all members, not bit-fields specifically.) `_Generic` is now selected at type-check time—the controlling expression is typed but not evaluated, and the compatible association's expression is the one lowered. Integer constants are now fully typed per §6.4.4.1: the suffix (`3L` → `long`, `0u` → `unsigned`, up to `unsigned long long`) fixes a floor, and value-directed widening then promotes a constant whose magnitude overflows that floor—2147483648 → `long`, `0xFFFFFFFF` → `unsigned int`—climbing the suffix-and-base-dependent list, so `_Generic` and `sizeof` agree with the Standard on literal operands. Finally `long double` is honoured per the Apple ARM64 ABI, under which it shares `double`'s 64-bit representation: its descriptor is aliased to `double`, so it lowers, sizes, and passes identically and no floating-point path needed a new kind. With this the tail C17 surface is complete—diagnostics, `-Wall`, designated initialisers, bit-fields, `_Bool`, `volatile`, `_Generic`, integer-constant typing, and `long double` all land.

9. **A second target** (new). Retargeting to x86-64 or Linux/ELF is a strategic multiplier but an entire new backend or object writer; undertake it only if cross-platform support becomes an explicit goal.

A.10.2. Optional Features — The Four Conditional Headers

Of the twenty-nine standard headers, kcc now supplies twenty-five: the nine *freestanding* headers embedded in the compiler and sixteen *hosted* bridges shipped as files under `include/` and found on the `-I` search path (§17.16.2). The four that remain absent are not an oversight and not a matter of writing more headers. They are the language's only *conditional features* (§6.10.8.3), which the Standard expressly permits an implementation to omit *provided it says so*—and kcc does, predefining `__STDC_NO_ATOMICS__`, `__STDC_NO_THREADS__`, `__STDC_NO_COMPLEX__`, and `__STDC_NO_VLA__`. The compiler is therefore fully conforming in the freestanding sense without them. What each would *cost*, should hosted-complete C17 ever become a goal, is a question of compiler and runtime work, not header text—each is a language or library subsystem.

Feature	What it requires	Eff
`tgmath.h` (real-only)	`_Generic` header + libm `f`/`l` protos	low
`threads.h`	pthreads shim + TLS (`_Thread_local`)	me
`stdatomic.h`	`_Atomic` type + LSE/LL-SC selection	me
`complex.h`	complex type class + conversions + ABI	hig

`<complex.h>` and `_Complex` — the largest. A complex value is a pair of floats with its own arithmetic. It needs a new type class in the type system (`float`/`double _Complex`) and an extension of the usual arithmetic conversions (§6.3.1.8) so that a real operand promotes to complex; a parser specifier and the `I` macro; codegen for addition and subtraction (componentwise, easy), multiplication ($ac - bd$ and $ad + bc$), and division with the NaN/infinity rules of § Annex G (hard); and the AAPCS64

mapping, under which `double _Complex` travels as a two-member aggregate in `v0/v1` and returns there. The library functions (`cabs`, `csqrt`, `cexp`) then bridge to `libSystem`, but only once the type matches that ABI exactly. This is weeks of work, and it is the prerequisite for a *conforming* `<tgmath.h>`.

`<tgmath.h>` — the cheapest, with an asterisk. Type-generic macros dispatch `sqrt(x)` to `sqrtf/sqrt/sqrtl` by argument type. Because kcc already has `_Generic`, a *real-only* `<tgmath.h>` is nearly free: a header of `_Generic` macros plus prototypes for the `f/l` variants in `libm`. A *conforming* `<tgmath.h>`, however, must also dispatch to the complex variants, so full support depends on `<complex.h>`.

`<stdatomic.h>` and `_Atomic` — medium to high. The `_Atomic` qualifier and specifier form (`_Atomic T` and `_Atomic(T)`) enter the type system, and—the real work—codegen must emit genuine atomic instructions. Apple Silicon offers the LSE atomics (`CAS`, `LDADD`, `SWP`) alongside the load/store-exclusive (`LDXR/STXR`) loop, and each memory order (acquire/release/seq_cst) selects the matching barrier variant. The header (`atomic_*`, `memory_order`, `atomic_flag`, the `..._LOCK_FREE` macros) is the small part; a new family of ordered atomic operations in the instruction selector is the large one. Faking them as plain operations would be a lie the moment real concurrency appears.

`<threads.h>` — mostly a runtime shim. The subtlety here is that macOS never shipped the C11 threads library—only pthreads—so there is no `libSystem` symbol to bridge to (the system `clang` likewise lacks `<threads.h>`, exactly as it lacks `<uchar.h>`). Support therefore means *writing an implementation*: wrapping `thrd_create` onto `pthread_create`, `mtx_*` onto `pthread_mutex_*`, and so on—real code, not a header—plus `_Thread_local`/TLS, which kcc does not yet model.

Recommendation. None of the four is required, and none should be stubbed: a stub would falsely advertise support the `__STDC_NO_*` macros correctly deny. Were kcc to pursue hosted completeness, the order by value is `complex` first (the most C-language-like, and it unlocks `tgmath`), then `stdatomic` (real concurrency); `threads` is largely glue to pthreads, and a real-only `tgmath` is a quick, non-conforming convenience. All of it is orthogonal to, and lower priority than, the second target (§A.10.1, item 10).

A.10.3. Stage 9a — Complete Type System

The backend began with integer types only; `double` floating point is now in place. This stage adds every remaining C17 type.

- ✓ **Floating point (`double`).** Implemented end to end. The lexer classifies a floating constant and `_kcc_parse_atof` evaluates it to an IEEE-754 `double` at compile time on the host FPU (§22.10); sema types it `double` and the usual arithmetic conversions already promote toward it. The IR gains `IR_SITOFP/IR_FPTOSI` plus a shared `_kcc_ir_convert`; `build_binary` promotes integer operands and tags floating results `double`, and that type tag is the backend's signal to use the FP unit. Codegen adds FADD/FSUB/FMUL/FDIV, FCMP (reusing the integer comparison tail, since ordered comparisons leave `NZCV` exactly as the signed conditions test), SCVTF/FCVTZS, and SIMD&FP LDR/STR. A `double` occupies the same 8-byte spill slot as any value—no constant pool is needed, the 64-bit pattern is materialised by the ordinary `IR_CONST` path, and because arguments and returns travel through that slot (via `x0`–`x7`) they interoperate for kcc-to-kcc calls. Verified by 14 `fp-*` differential tests.
- ✓ **Single-precision `float`.** A floating constant with an `f/F` suffix is rounded to single precision at compile time (on the host FPU) and typed `float`; a run-time conversion *to* `float`—an assignment, a cast, or an `int`→`float`—emits `IR_FPROUND`, which the backend lowers to `FCVT s,d` followed by `FCVT d,s` to truncate the value to single precision. So `16777217.0f == 16777216.0f` and `(float)d` agree with `clang` (eight `float-*` differential tests). Arithmetic runs in `double` registers, but as of milestone M1 a binary operation whose result type is `float` appends an `IR_FPROUND`, so each `float` sub-expression is rounded to single precision as it is produced—`FLT_EVAL_METHOD` 0, bit-exact with `clang` on chains like `a/b*c`. Only the `s0`–`s7` argument convention for `clang` float-argument interop remains. `long double` is lowered as `double` per the Apple ARM64 ABI (identical 64-bit representation).
- ✓ **Unions.** Done: `_kcc_type_layout` lays a union out with every member at offset 0 and `TY_SIZE` equal to the largest member (rounded to its alignment), and member access, classification, and pass/return share the struct paths. Fixing a subscript on an aggregate *member* array (`u.arr[i]` / `s.arr[i]`) along the way—it had been (mis)loaded as a pointer—closed the last union gap. Six `union-*`/aggregate differential tests pass.

- **`_Bool`.** `TY_BOOL` exists as a singleton. The IR builder must insert `IR_TRUNC` to 1 bit on assignment (any non-zero value becomes 1) and `IR_ZEXT` on promotion.
- **Type qualifiers.** `const` objects should land in `__const` section. `volatile` loads and stores must not be eliminated by DCE or reordered by future optimization passes. `restrict` can be ignored initially (it is a hint, not a constraint).
- **`_Alignof` and `_Alignas`.** Lower `_Alignof` to `IR_CONST` with the type's alignment. `_Alignas` adjusts alloca padding and `__data`-section alignment.
- **Bit-fields.** Struct members declared with a `: width` need sub-byte layout in `_kcc_type_layout` (packing consecutive fields into storage units) and masked

load/store codegen to read and write the field within its unit.

- **`char` array from a string literal.** `char s[] = "abc"` (and a sized `char s[8] = "abc"`) must copy the literal's bytes into the array's storage, not store a pointer to it.
- **Designated initializers.** Struct and array initializers with `.field = val` and `[index] = val` syntax, including nested designators.
- **Compound literals.** `(type){init}` creates a temporary with block scope; lower to an anonymous alloca + initializer stores.
- **`_Generic`.** Type-dispatch at compile time; the sema pass selects the matching association and replaces the `_Generic` node with the selected expression.

A.10.4. Stage 9b — Full ABI Compliance

The calling convention now handles struct pass and return by value (both checked below), but still passes at most eight arguments in `x0`–`x7` and uses kcc's own uniform slot convention rather than the exact AAPCS64 register/stack rules. This stage closes the remaining gaps toward full AAPCS64 compliance.

- ✓ **Stack arguments.** Done: a call with more than eight arguments loads the first eight into `x0`–`x7` and pushes the rest onto the stack (extending the same push the Apple variadic ABI already used—argument k at offset $(k-1)\times 8$ below the pre-call stack pointer), and the callee's prologue reads parameters nine-and-later back from `[x29, #frame_size + (k-1)×8]` into their vreg slots. This is kcc's uniform slot convention (self-consistent for kcc-to-kcc calls), not the exact AAPCS64 slot ordering; five differential tests up to twelve arguments, including a recursive stack-argument passthrough, agree with `clang`. A single integer/floating register file is used (kcc passes `double` bit patterns through the general registers), so there is no separate eight-d-register count.
- ✓ **Struct return by value.** Implemented (§29.6) with a *hidden result pointer*: the caller allocates the destination and passes its address as a trailing argument; the callee copies the aggregate there and returns `void`. Because kcc lays every struct field in its own 8-byte slot, this one convention covers structs of any size uniformly (rather than packing $\leq$16-byte structs into `x0`–`x1`), and it is self-consistent for kcc-to-kcc calls, which is all the corpus requires. A later refinement can adopt the exact AAPCS64 register-packing and `x8` rules for interoperation with `clang` object files.
- ✓ **Struct pass by value.** Implemented (§29.3) as the mirror image of struct return: the caller passes the *address* of the argument object in a single register (via `_kcc_ir_build_struct_addr`) and the callee's `build_param` copies the N slots into its own parameter storage, giving true by-value semantics. Each struct argument consumes one register, so this composes with the hidden result pointer and with scalar arguments. As with the return path, this is the uniform all-spill convention rather than AAPCS64's $\leq$16-byte register packing; adopting the exact register/stack rules is deferred to `clang`-interop work.
- ✓ **Variadic function definitions.** Done (§29.9.13): `va_list`, `va_start`, `va_arg`, and `va_end` are provided as compiler built-ins rather than a header (a missing `#include <stdarg.h>` is silently skipped). Since the call lowering already pushes every argument onto the stack, `va_list` is an opaque eight-byte cursor; `va_start(ap, last)` lowers to an `IR_VA_START` instruction that the backend turns into `ap = x29 + frame_size + (F-1)×8` (with F the fixed parameter count); `va_arg` reads the eight-byte slot at the cursor and advances it; `va_end` is a no-op. `va_arg`'s type operand required a small parser special case. Three differential tests—including one with two fixed parameters—agree with `clang`. Because kcc stores each argument in a uniform eight-byte slot, no register save area is needed.
- ✓ **Correct struct member alignment.** Done (§A.12): `_kcc_type_layout` places each member at the next offset that satisfies its alignment, sets `TY_SIZE` to the size padded up to the aggregate's alignment, and records `TY_ALIGN`. Addressing is now byte-accurate throughout—the `IR_GEP` instruction adds a byte offset (arrays pre-scale the index by the real element size, members pass a constant offset), narrow field loads/stores carry a width-and-sign override, and pointer arithmetic steps by the exact pointee size. `sizeof(struct {int x; int y;})` is now 8, and 15 `struct-layout` differential tests agree with `clang` on size, padding, odd-sized array strides, and signed narrow fields. Only the AAPCS64 register-*packing* of small structs into argument registers remains (deferred to `clang`-interop work); the in-memory layout itself now matches the platform.

A.10.5. Stage 9c — Register Allocation

The all-spill model generates roughly $3\times$ as many load/store instructions as necessary. This stage introduces register allocation to eliminate redundant memory traffic.

- ✓ **Local linear-scan allocator** (§35.9). A conservative pass allocates block-local, integer-scalar temporaries to two banks: the caller-saved scratch registers `x12`–`x15` for call-free values, and the callee-saved registers `x19`–`x27` for values whose live range crosses a call. Two walks over the IR compute per-vreg live information (mirroring the DCE use-classification, and detecting call-crossing with a monotonic call counter) and then run the scan; `load_vreg`/`store_vreg` consult the resulting map, with the FP helpers bridging a register-resident value to a `d` register via FMOV. Each claimed callee-saved register is spilled in the prologue to a save area *above* the vreg slots (so no other offset moves) and restored

at every return. An unallocated vreg falls back to exact all-spill behaviour, so the change is transparent. Measured frame-memory traffic on arithmetic-heavy functions drops by roughly one third to two thirds.

- **Global (cross-block) ranges.** Extend beyond block-local liveness with real live-in/live-out dataflow, so a temporary spanning several blocks (or a loop) can stay in a register.

- **Spill heuristics.** When registers are exhausted, spill the vreg with the longest remaining interval rather than declining to allocate; coalesce adjacent load/store pairs.

- **FP register allocation.** *Done.* Floating vregs get a *dedicated* bank via a second linear scan mirroring the integer one—`d16`–`d31` (caller-saved) for call-free values and `d8`–`d15` (callee-saved, spilled by the prologue) for call-crossing and loop-carried ones, with `d0`–`d7` left for arguments and scratch. They no longer compete with integer values for the general registers, and an FP-only chain avoids the FMOV round-trip. (The all-in-FMOV helpers remain as the bridge for the paths that materialise a floating constant or spill a double to memory.)

A.10.6. Stage 9d — Optimization

Go-forward priority: deferred (§A.10.1, item 8). kcc already emits correct code, so these passes are quality, not enablement, and come after diagnostics and the high-frequency language gaps.

Phase 6 provides constant folding and dead code elimination. This stage adds passes that substantially improve code quality.

- **Common subexpression elimination (CSE).** Hash each instruction's opcode + operands; reuse an existing vreg instead of recomputing an identical value.

- **Copy propagation.** Replace uses of `IR_LOAD` with the original vreg when the address has not been stored to since the load.

- **Strength reduction.** Convert multiplications by powers of two into shifts; convert divisions by constants into multiply-high sequences.

- **Basic-block reordering.** Place fall-through successors physically after their predecessors to eliminate unconditional branches.

- **Instruction combining.** Fuse `CMP`+`B.cond` into `CBZ`/`CBNZ` where possible; fold `ADD`+`LDR` into addressing modes.

- **Tail-call optimization.** Replace `BL`+`RET` with `B` when the callee's frame is compatible.

- **Loop-invariant code motion (LICM).** Hoist instructions whose operands do not change inside a loop to the loop's preheader block.

A.10.7. Stage 9e — Tooling and Diagnostics

Go-forward priority: highest (§A.10.1, items 1–2, 6–7). Error recovery and warnings are the top of the reprioritized plan—the biggest usability jump—followed by DWARF and input hardening; only automatic linking and freestanding headers here are already done.

A production compiler needs more than correct code generation.

- ✓ **DWARF debug information.** Done (Chapter 42): under `-g` the emitter writes `__debug_abbrev`, `__debug_info` (a `compile_unit` DIE plus one `subprogram` DIE per function, names inlined as `DW_FORM_string`), and `__debug_line` (a function-granularity line-number program) into a non-loaded `__DWARF` `LC_SEGMENT_64` whose VM range is placed past `__TEXT` and whose file span sits between the code and the LINKEDIT region. `dwarfdump --verify` reports no errors and the `-g` object still links. Statement-level rows and relocated addresses (so `lldb` can map a *linked* binary) are the next refinements.

- **Warning system.** Implement at least `-Wall`-equivalent diagnostics: unused variables, implicit int conversions, missing return statements, unreachable code, shadowed declarations, comparison of signed and unsigned.

- **Error recovery.** The parser currently aborts on the first error. Add synchronization-point recovery (skip to next `;` or `}`) to report multiple errors per compilation.

- **Command-line interface.** Full flag parsing: `-O0`/`-O1`/`-O2` (controls which optimization passes run), `-Wall`/`-Werror`, `-I path` (include search path), `-D name=val` (command-line defines), `-S` (emit assembly text), `-E` (preprocess only, already implemented), `-o output`.

- ✓ **Automatic linking.** Done (§43.3): invoked without `-c`, the driver compiles each source to an object, then `fork`s and `execve`s `/usr/bin/ld` (new process syscalls in Chapter 7) with `-lSystem -syslibroot metasdk -arch arm64 -e _main`, `wait4`s for it, removes the intermediate objects, and exits with the linker's status. The SDK root comes from `$SDKROOT` or the standard locations. `kcc hello.c -o hello` now produces a runnable executable—including one that calls `libSystem`'s `printf`—in one step; eight end-to-end driver tests (§43.5, `make e2e`), three of them driving the built-in headers (§17.16.2), exercise it.

- ✓ **System header compatibility.** `#include <stdio.h>` (and `stdlib`, `string`, `math`, `ctype`, `stddef`, `stdbool`, `stdint`, `stdarg`, `assert`) are served from a compiler-embedded freestanding header set rather than a file: the include machinery hands back a resident C snippet of prototypes, typedefs, and macros (§17.16.2) and `libSystem` resolves the symbols. This exposed—and forced the

fix of—a pre-existing variadic-ABI limitation: multi-fixed-argument variadic calls (`fprintf`, `sprintf`, `snprintf`) now lay out the Apple variadic stack from the prototype's fixed-parameter count instead of assuming one. The standard streams `stdin`/`stdout`/`stderr`—dylib-imported *data*—also work now: the back end reaches every external data symbol through the GOT (ADRP `GOTPAGE` + LDR `GOTPAGEOFF`) rather than a direct ADRP/ADD, so `ld` binds the slot to `libSystem`'s `__stdoutp` and friends.

- **Source-level profiling.** Instrument function entry/exit for -pg-style profiling or basic timing output.

Acceptance milestones. (✓ met ⊙ partial ○ open)

- ⊙ The full C17 type system: floating point and unions are in; _Bool, qualifiers, bit-fields, designated initializers, and _Generic remain.
- ✓ The calling convention matches AAPCS64: struct-by-value, stack arguments, and variadic definitions all work, `clang`-validated.
- ⊙ Register allocation beats the all-spill baseline: a local linear-scan has landed (memory traffic down a third to two thirds); the $2\times$ code-size target awaits global allocation.
- ○ `lldb` can debug kcc-compiled programs with source-level breakpoints and variable inspection.
- ○ The compiler accepts at least two -O levels and produces measurably better code at -O1 than -O0.
- ✓ `gen/kcc hello.c -o hello` compiles, links, and runs without a manual `ld` invocation.
- ⊙ kcc compiles a > 5000-line program (its own source) to a correct executable; the remaining "*debuggable*" clause is gated on the DWARF milestone above.

A.11. Summary

Phase	Status	Milestone	Key Output
0	✓	Skeleton boots	`gen/kcc` runs
1	✓	Lexer	`--dump-tokens`
2	✓	Diagnostics + tests	`make test`
3	✓	Preprocessor	`-E`
4	✓	Parser	`--dump-ast`
5	✓	Type system	`--dump-types`
6	✓	IR + optimization	`--dump-ir`
7	✓	Backend	`-c file.c -o file.o`
8	✓	Self-hosting	Differential testing
9	⊙	Production	`kcc hello.c -o hello`

Phase 9 is under way: its ABI and register-allocation stages have landed and the compiler self-hosts and auto-links; diagnostics, DWARF, and the remaining language and optimization work are what stand between it and "production-ready" (§A.10.1).

Total: 45+ `.weft` source files (growing to ∼55 by Phase 9), approximately 30 tangled assembly files, producing a single native `gen/kcc` binary that compiles C17 to ARM64 Mach-O — with no dependency on any existing C compiler.

A.12. Production Readiness

"Is kcc a *100% functional, production-ready* C compiler?" splits into two independent questions. The first is **language conformance**: does it accept and correctly translate all of C17? The second is **production engineering**: optimization, portability, tooling, and robustness—everything a team depends on *beyond* a correct answer. kcc scores very differently on the two, and being honest about the gap matters more than any single headline number.

What is already solid (and verified). The integer, pointer, and aggregate core of C is complete and, crucially, *checked against an oracle*: the differential harness (§**??**) compiles 155 programs with both kcc and `clang` and finds *zero* miscompilations. That core includes all control flow (`if`/`while`/`do`/`for`/`switch`/`goto`), functions and recursion, pointers and pointer-to-pointer, arrays (including multi-dimensional), structs (field access, nesting, both pass and return by value, and now a byte-accurate §6.7.2.1 layout with correct `sizeof`, padding, and alignment), enumerations, function pointers, `typedef`, `static` locals, initializer lists, the object- and function-like preprocessor with source-relative `#include`, multi-file compilation, and now `double` floating point.

Language conformance — what remains. The remaining language gaps are individually small but collectively keep kcc short of full C17:

Feature	Status	Note
standard struct layout	✓	padding/align, 15 diff. test
`double` floating point	✓	complete, 14 diff. tests
single-precision `float`	✓	per-op single rounding (M
`long double`	✓	Apple ABI: aliased to doub
`union`	✓	offset-0 layout; 6 diff. tests
bit-fields	✓	LSB-first packing; 7 diff. t
`_Bool`	✓	any non-zero → 1
`char` signedness	✓	byte-copy init; 8 diff. tests
designated initialisers	○	single-level `[i]=`/`.f=`; nest
`volatile` semantics	✓	honoured for every lvalue a
variadic definitions	✓	`va_arg` builtins, 3 diff. tes
`_Generic`	✓	selected at type-check time

The struct-layout gap that used to hide behind otherwise-correct results—every field on its own 8-byte slot, so `sizeof(struct {int x; int y;})` was 16—is now **closed**. `_kcc_type_layout` lays aggregates out byte-accurately with standard padding and alignment, addressing became byte-granular (byte-offset `IR_GEP`, real array/pointer strides, width- and sign-correct narrow loads and stores), and `sizeof` now matches the Standard. What remains for `clang` object-file interoperability is narrower:

`.weft` Files 01, 02, 07–09, 11, 12, 43
10, 14, "17."
13, 15, 16, 44, 45
18, 19, 20
21–24
25–30
31–36
37–41, 47
46–49
50–52

the AAPCS64 rules for *passing* small structs packed into `x0`–`x1` rather than by hidden pointer, plus the `d0`–`d7` floating-point argument registers. The in-memory layout itself is now the platform layout.

Production engineering — what remains. Once the larger axis, this is now mostly closed; what remains is quality-oriented—an optimizer, global register allocation, input hardening, and a second target—rather than missing capability:

Concern	Status	Note
register allocation	✓	linear-scan; cross-block loop-free & loopy; mem2reg (scalar locals)
optimizer / CSE / FP r.a.	○	const-fold+DCE+CSE+mem2reg+FP-alloc (dedicated d-bank); copy-prop/streng
> 8 arguments on the stack	✓	self-consistent, 5 diff. tests
AAPCS64 register convention	✓	FP/small+large struct/x8; interop 16/16
second target (x86-64/ELF)		ARM64 + Mach-O only
automatic linking (`ld`)	✓	`kcc x.c -o x; make e2e`
freestanding sys. headers	✓	built-in <stdio.h> etc.; GOT-indirect data
multi-error recovery	✓	reports every error; carets; `make diagtest`
warnings (`-Wall`)	✓	5 canonical warnings; `make warntest`
debug info (DWARF)	✓	`-g`; func-granularity; verify-clean + links
harden vs. malformed input	✓	crash classes fixed; hash-table resize kills the large-symbol $O(n^2)$ hang
full `-O`/`-I`/`-D` CLI	○	partial

Bottom line. As a *correct compiler for the integer/pointer/aggregate subset of C, plus `double` and single-precision `float`*, kcc is in good shape and unusually well-tested for a hand-written assembly project: the differential suite finds zero miscompilations against `clang` (187/0), the mixed-object interoperability suite is green in both directions (16/16), and the compiler self-hosts to a byte-identical binary. As a *complete C17 implementation* it is nearly all the way there: with `unions`, single-precision `float`, bit-fields, single-level designated initializers, `_Bool` normalization, `char s[] = "..."`, `_Generic` selection, full integer-constant typing, `volatile` semantics, and `long double` (Apple-ABI aliased to `double`) now done—and, as of milestone M1 (Appendix B), volatile bit-fields, nested designator chains with brace elision, and per-operation single-precision `float` rounding as well—the language axis is essentially closed. The one bounded piece still missing is propagating `volatile` from a qualified *object* through member access, which is not specific to bit-fields.

The toolchain axis, previously the weakest, has closed most of its gaps. Standard struct layout, more-than-eight stack arguments, variadic function definitions, and a fully AAPCS64-compliant calling convention (floating values in `d` registers, small structs in `x0`–`x1`, 9–16-byte struct arguments in two registers, large struct returns via `x8`) are done and cross-checked against `clang`. The driver links automatically—`kcc hello.c -o hello` produces a runnable executable in one step (§43.3)—serving free-standing system headers (`#include <stdio.h>` yields real prototypes, imported data reached GOT-indirect). The front end now recovers past the first error and reports every diagnostic with a `clang`-style caret (`make diagtest`), `-Wall` enables five canonical warnings (`make warntest`), a linear-scan register allocator uses both caller- and callee-saved banks and keeps values in registers across blocks in loop-free functions, and `-g` emits verify-clean, linkable DWARF (Chapter 42). Ad-hoc fuzzing also closed the front end's crash classes (truncated input, oversized output, large frames).

What still separates kcc from an *industrial* toolchain is now narrow and quality-oriented rather than a matter of missing capability: there is no IR optimizer, register allocation does not yet cover loop-carried or floating values, so generated code is correct but not fast; and there is a single target (ARM64/Mach-O). The shortest credible path from here runs through extending register allocation to loop-carried and floating values, then IR optimization passes (CSE, copy propagation, strength reduction) for code quality. Everything after that—a second target, statement-level DWARF, an optimizer rivalling `clang -O1`—is refinement rather than feasibility.

B. Roadmap: The Road to a Second Target

kcc today is a correct, self-hosting C17 compiler for one target (ARM64 / Mach-O) that compiles and runs real software—Lua 5.4.7's entire official suite, zlib 1.3.1 end to end, cJSON byte-identical to `clang`. The natural next ambition is a *second* target. This appendix argues that a second target should not be started until the shared core is finished and proven, and lays out precisely what that means. (The argument held: by the time the x86-64 target was actually begun, the target-neutral milestones M1–M5 were complete—see the revised M7 status in §B.9 for how the second target was then grown, skeleton-first, ahead of the interface-carving step M6.)

B.1. The Guiding Principle

The retargeting playbook (Chapter 2) establishes that the IR is the *neutral seam*: roughly seventy percent of the compiler—the preprocessor, parser, semantic analysis, IR construction, the optimizer, and the IR verifier—is target-independent and is reused verbatim by every backend. Only the last thirty percent (instruction selection, register banks, ABI, object emission) is target-specific.

That split dictates the order of work by one rule:

> **Finish the shared seventy percent before you split.**
>
> Every defect in the target-neutral core that survives into a two-target world costs *twice*: it must be re-diagnosed through the ambiguity "is this the shared frontend or the new backend?", and every fix must be re-validated against two code generators and two test baselines. A feature added to the IR optimizer *before* the split is written once and inherited free by the second target; the same feature added *after* must be proven correct on both.

There is a second, quieter reason. The ARM64 backend is not just *a* target—once a second target exists it becomes the *oracle*: the differential reference (`clang`-validated, byte-identical on real code) against which the new backend's output is checked. An oracle must be complete and trustworthy before it is relied upon. So the pre-split work is of exactly two kinds: **(A)** complete and prove the shared core, and **(B)** polish ARM64 into a dependable oracle.

B.2. Milestones at a Glance

Six milestones (M1–M6) precede the second target (M7). M1–M4 are target-neutral (kind A); M5 is ARM64 oracle-hardening (kind B); M6 is the structural work that *enables* the split. The table is the scan; the sections are the detail.

#	Milestone
M1	Finish the C17 frontend (designators, volatile bit-fields, real `flo`
M2	Harden the IR verifier
M3	Systematic conformance battery
M4	Broaden the IR optimizer (strength reduction + copy propagatio
M5	Write-protect read-only data (stray writes fault, not corrupt)
M6	Carve the target interface (route ARM64 through it)
M7	Second target (x86-64 / Mach-O)

Effort: ●●● high ●●○ medium ●○○ low. M1–M5 may proceed in parallel; M6 must come last of the six, because it is validated by re-running the now-complete M1–M5 suites through the refactored ARM64 path with byte-identical output.

B.3. M1 — Finish the C17 Frontend

Goal. Close the remaining language gaps so the shared frontend is genuinely conforming before it feeds two backends. Each of these is IR-*construction* work—it lives above the seam and is reused by every target.

Work items.

- **Nested designated initializers** (§6.7.9): `.a.b[2] = v` and brace-elision across aggregate levels. Single-level `.f=/[i]=` is already done; the recursion over sub-objects is the gap.
- **Volatile bit-fields** *(landed)*: the synthesised read–modify–write of a `volatile`-declared bit-field now stamps the volatile bit on its container load and store (`_kcc_ir_bf_container_type` feeding the type-taking `_kcc_ir_emit_load_t/_store_t`), so CSE and DCE treat each access as a mandatory side effect—parity with a volatile scalar member.
- **A real single-precision `float` model** *(landed)*. Values still live as `double` in the SIMD registers—a uniform backend model—but a binary operation whose *result* type is `float` now appends an `IR_FPROUND`, rounding that sub-expression to single precision as it is produced. This gives the `FLT_EVAL_METHOD` 0 semantics Apple ARM64 and `clang` use—`a/b*c` matches `clang` to the last bit—while a comparison (`int` result) and a mixed `float`/`double` expression (full precision) are correctly left alone. Settling this *now* avoids re-baselining two backends later.

weft files. `27b-sema-declarations` and `32c-ir-aggregates` (designators); `32b-ir-expressions` and `37c-codegen-dispatch` (volatile bit-fields; the value model is neutral, the final load/store width is where ARM64 realizes it); `28-conversions` and the IR float opcodes (`float` model).

Acceptance. The differential corpus (`make difftest`) gains cases for each feature and stays byte-identical to `clang`; self-host stays byte-identical; no existing suite regresses.

Status. In progress. Nested designator chains (`.p.a`, `.arr[1]`, `[0].x`) and brace elision (`int m[2][2] = {1,2,3,4}`) are *landed*, at both block and file scope,

through a single recursive initializer (`_kcc_ir_fill_list`) shared by the local and global paths and validated element-for-element against `clang`. Volatile-declared bit-fields now carry the volatile bit on their container access, at parity with volatile scalar members. Single-precision `float` arithmetic now rounds each `float` sub-expression once (`FLT_EVAL_METHOD 0`), bit-exact with `clang`. With this, **M1 is complete**; the residual (propagating `volatile` from a qualified *object* through member access) is a general member-access matter, not specific to this milestone.

B.4. M2 — Harden the IR Verifier

Goal. Make the IR verifier (Chapter 30) strong enough to reject malformed IR *before* it reaches any backend. With one code generator, a backend crash and an IR defect look alike; with two, a verifier that catches the defect at the seam is what keeps a frontend bug from masquerading as a backend bug.

Work items *(landed).* The verifier (`_kcc_ir_validate`, §30) now walks every function, block, and instruction and enforces three invariants whose violation is undefined back-end behaviour: valid opcodes (an uninitialised or corrupted `IR_OP`—the shape of the stale-`NOP` bug—is rejected), terminated blocks (a non-empty block that would let code generation fall through is rejected), and finite instruction lists (a cyclic `IR_NEXT` is reported, not spun on). It is wired into the driver as a mandatory pass between the optimizer and the back end, so it also catches an optimization pass that corrupts the IR. Dataflow invariants that need dominance (strict def-before-use) or that kcc's pre-register-allocation IR does not maintain (single static assignment) are deliberately left to a later dataflow-aware pass, to avoid false rejections.

weft files. `33-ir-validation` (real verifier replacing the stub); `42-driver` (mandatory call before code generation).

Acceptance. The verifier accepts all IR the current suites produce—every codegen, difftest, self-host, interop, e2e case, plus all of Lua and zlib, compiles with no false rejection—and a controlled fault (tightening the opcode ceiling) confirms it does fire and abort the compile with a diagnostic naming the function.

Status. Done.

B.5. M3 — Systematic Conformance Battery

Goal. Prove the shared frontend and IR are correct on the *uncommon* cases that no hand-picked real program happened to exercise—*while there is still only one target*, so a failure localizes unambiguously to the core.

Work items.

- *(done)* Run the `c-testsuite` curated corpus (220 single-file programs, each with an expected exit code and stdout) through kcc as an external oracle, and record a baseline: at the start **181/220** passed (24 compile gaps, 15 miscompiles); after the fixes below, **184/220**, and every bug the corpus surfaced that lies in the shared front end or IR was fixed directly.
- *(future)* A `csmith` random-differential harness (compile with kcc and `clang`, compare runtime output, shrink divergences) is the natural next escalation; it needs the external generator installed and is left as a follow-on.

Per project policy (§no-fuzz-harness) this uses *external* generators as oracles and fixes each surfaced bug directly; it does not add a bespoke in-tree fuzzing suite. Each fix instead gains a targeted differential regression test, and the existing suites stay green.

Bugs the corpus surfaced and fixed (all in the target-neutral core, so both targets inherit the fix): **(1)** reaching the closing brace of `main` now returns 0 (§5.1.2.2.3) instead of a garbage register; **(2)** a file-scope floating variable initialised from an integer constant (`double d = 100;`) now converts to the floating representation instead of storing the integer bits; **(3)** reading a `float`/`double` *global* or struct member now emits a floating load carrying the real type, not an integer load; **(4)** a multi-declarator external declaration (`int a, b;`) now registers *every* declarator, not just the first.

weft files. `21-parser` (multi-declarator tail), `32-ir-builder` (main-returns-0, global scalar init conversion), `32b-ir-expressions` (floating global/member load type), `49-differential-testing` (seven new regression cases).

Acceptance. A recorded c-testsuite baseline, every core-attributable divergence fixed with a regression test, and the full kcc suite plus Lua and zlib still green. Remaining corpus failures are tracked feature gaps (variadic `char` promotion, some predefined macros, a few control-flow corners), not silent miscompiles.

Status. Done (c-testsuite corpus); `csmith` follow-on deferred.

B.6. M4 — Broaden the IR Optimizer

Goal. Raise generated-code quality with passes that are *entirely* IR-to-IR, so the second target inherits every one of them at no additional cost. This is the milestone where the "write once, reused free" economics of the guiding principle pay off most.

Work items. On top of the present constant folding, CSE, and DCE, the fixed-point pipeline gains two new IR-to-IR passes.

(landed) **Strength reduction and algebraic simplification** (`_kcc_opt_strength`, §32.2): multiply, unsigned divide, and unsigned remainder by a power of two become a shift or a mask ($x*2^k \to x<<k$, $x /_u 2^k \to x>>_u k$, $x \%_u 2^k \to x\&(2^k-1)$), and $x*0$/$x\&0$ fold to zero; signed `SDIV`/`SMOD` and every floating operation are deliberately left alone.

(landed) **Copy and constant propagation** (`_kcc_opt_copyprop`, §34): because kcc keeps locals in memory, "copies" live as store/load pairs, so the pass does *store-to-load forwarding*—a load that reads exactly what a preceding store wrote takes the stored value (or constant) directly, and DCE removes the dead load. It is airtight by three conditions rather than by alias or dominance analysis: the store and load sit in one straight-line block with no store, call, or `va_start` between them (memory is provably unchanged);

both are the natural 8-byte access (no truncation or re-extension can perturb the value); and substitution is intra-block (the map resets at each block boundary). Constants stored then loaded are forwarded by the same mechanism, and constant *folding* already propagates within a block through its value map, so global constant propagation is substantially covered. *(future)* Sparse-conditional constant propagation across the CFG and small-function inlining remain; the inliner (IR cloning, vreg remapping, param/argument binding, callee-alloca hoisting) is the largest dataflow addition and is deferred.

weft files. 35-peephole (`Opt: strength reduction`), 36a-cse (`_kcc_opt_copyprop`), 34-ir-passes (pipeline), `assembly.weft` (routing).

Acceptance. Behaviour preserved on the whole deterministic corpus—codegen 222, difftest 208 (runtime-equal to `clang`, including negative signed operands and a store-then-alias-store-then-load that must *not* forward), self-host 222, and zlib (example self-test and a gzip round-trip) all green. Metric: a benchmark of nine multiplies, three divides, and four remainders by powers of two compiles to **zero** `mul` and `udiv` machine instructions. (The Lua suite, an additional oracle, is presently blocked by an environment-dependent flake unrelated to these passes—the historical commit that first passed it fails identically today—so the deterministic suites and zlib stand in.)

Status. Done. Both IR-to-IR passes have landed—strength reduction and copy & constant propagation—and the acceptance metric is met: the benchmark of multiplies, unsigned divides, and unsigned remainders by powers of two compiles to *zero* `mul` and `udiv` machine instructions (each becomes a shift or a mask), with the deterministic corpus, self-host, and zlib all green and the Lua suite (its layout flake now non-reproducing) again passing as an extra oracle. Sparse-conditional constant propagation across the CFG and function inlining are tracked as *future escalations beyond this milestone*—valuable, but not open parts of M4's definition of done, exactly as the `csmith` harness is a deferred follow-on to M3 rather than a gap in it.

B.7. M5 — Write-Protect Read-Only Data (Make Stray Writes Fault)

Kind. This is the roadmap's first *kind-B* milestone: it does not extend the shared core but polishes ARM64 into a trustworthy oracle, by converting a whole class of *silent* miscompile into an immediate, deterministic fault. An oracle that can corrupt data without complaint is not yet dependable enough to validate a second backend against.

The defect. kcc's object emitter places *all* program data into a single writable `__common` section in `__DATA`. A build of Lua makes this concrete: the `static const` lookup tables of `luaL_checkoption` (`opts[]`/`optsnum[]`), ordinary mutable globals, and zero-initialised BSS all share one writable segment (`vmaddr 0x100134000`, one `__common` block), with no `__DATA_CONST`/`__const` for read-only statics and no distinct `__bss`. Nothing that should be read-only is write-protected. This is what turns every codegen *stray write*—the recurring bug family of an undersized `alloca` (Bug 38), a mis-coloured spill slot (Bug 32), or an 8-byte load/store through a 4-byte slot (Bug 45)—from a hard fault into a *silent* corruption: the wild store lands on a neighbouring object and is discovered only much later, as scrambled data, at a site arbitrarily far from the cause.

The exemplar (Lua `all.lua:109`). Under `_port=true`, `collectgarbage("count")` intermittently returns `boolean false` instead of a number, and the entire `collectgarbage` option dispatch is scrambled (`"collect"`→`nil`, `"count"`→`false`, `"isrunning"`→a number) while every unrelated operation—`math.floor`, `string.format`, `("count"):len()`—stays correct. Disassembling the two functions proves the dispatch *code* is right: it calls `luaL_checkoption`, indexes `optsnum[idx]` with an `ldrsw`, and switches on the result with a correct if-else chain. So the fault is not logic; it is a foreign write onto the `opts[]`/`optsnum[]` tables the path reads. And it is *layout-sensitive*: kcc's build is deterministic, yet whether the stray write collides with those tables depends on the final binary layout, so shifting any single translation unit—or merely changing the linker—moves the collision and flips the bug on or off. The commit that historically first passed the whole Lua suite reproduces the failure in some environments and passes it in others (under both kcc's own linker and `clang`'s ld today); the current build passes. A heisenbug of this shape resists the ordinary tools: any injected `printf` or assert perturbs the layout and masks it, and it cannot be reproduced on demand. *Write protection is what makes it catchable*—with the tables in a read-only page, the offending store faults at its own instruction and `lldb` names the culprit function on the spot, instead of the investigator chasing distant scrambled data.

Mechanism (what reconnaissance established). The fix is larger than a section retag, because of two facts about kcc's back end. First, kcc emits a single `MH_OBJECT` and *delegates final linking to the system `ld`*; the read-only page protection of `__DATA_CONST` (the `SG_READ_ONLY` segment flag) is therefore `ld`'s job, not kcc's. kcc only has to *name* the section (`__TEXT,__const` or `__DATA_CONST,__const`) that `ld` already places in a read-only segment—there is no segment-protection code to write. Second, and harder: kcc stores *no* initialiser bytes in the object. The `__common` section is `S_ZEROFILL`, and *every* global initialiser—scalars and aggregates alike—is executed at load time as runtime stores from a synthesised `kcc.module_init` constructor. A store into a read-only page faults, so a `const` object cannot be both runtime-stored and write-protected. Making it read-only therefore requires *also* laying its initialiser bytes into a real (`S_REGULAR`) file-backed section at compile time and suppressing its `module_init` stores. A third fact eases one worry: `const` is currently dropped in semantics (§30 neighbours), but it can be read straight from the declaration AST at global pre-registration, so the shared type descriptors—and the identity comparisons that depend on them—stay untouched.

Work items (staged).

- **Stage 1.** A `const`-qualified static whose initialiser is a non-relocatable compile-time constant (integer/float scalars and aggregates of them—the `optsnum[]` victim class) is emitted into read-only `__TEXT` *(landed)*. Rather than mint a new `section_`-

64, it reuses the existing read-only tail that already carries string literals (the growable `_kcc_cstr_buf` appended to `__text`): its bytes are evaluated at compile time from the literal initialiser, a section-1 symbol is minted at that offset with a fixup (value patched to $code_size + offset$) and an extern-table entry so every `IR_GLOBAL_REF` dedups to it, and its `module_init` stores are suppressed. A stray write into such a table now faults (`SIGBUS`) at the store instead of silently corrupting a neighbour. Extended to `float`/`double` elements (a floating literal's double bit pattern, narrowed by a compile-time `fcvt` for `float`), to negated literals (`-4`, `-2.5`), and to integer constant *expressions*—bit-flag and masked-literal tables (`{1<<0, 1<<1}`) and `enum` tables (`{STATE_A, STATE_B}`) fold through `_kcc_eval_const_int`. A non-array `struct` of scalar fields is also laid out field-by-field at each member's offset (mixed `{int, double, char}` records included), and to a 1-D array of such structs (each element laid out at its stride). Multi-dimensional arrays of structs, structs with aggregate fields, and non-literal floating initialisers still take the writable path and are a later slice.

- **Stage 2** *(landed)*. A `const` array of const pointers initialised with string literals (`static const char *const opts[]`—the other half of the Lua victim) now goes to `__DATA_CONST,__const`. Because a pointer must be relocated at load it cannot live in immutable `__TEXT`; instead the emitter adds a real `section_64` (modelled on `__mod_init_func`) holding one zero 8-byte slot per element, emits each string into the read-only cstring buffer (`_kcc_emit_cstring`), and records an `ARM64_RELOC_UNSIGNED` per non-NULL slot binding it to that string symbol; `ld`/dyld apply the fix-ups and *then* make the page read-only. A laundered write to such a table faults with `SIGBUS`. The whole layout addition is gated on a per-object flag, so an object with no const pointer table is byte-unchanged.

- **Stage 3 (optional, re-scoped).** The original aim—split zero-initialised BSS from initialised data "so a BSS overrun cannot walk into a live object"—turns out *not* to be achievable as a write-protection measure in kcc's model. kcc emits one segment and delegates linking to `ld`; in the final image *all* mutable data (initialised globals and BSS alike) lands in the single writable `__DATA` segment. Even after separating an initialised `__DATA,__data` section from the `__common` BSS, the two remain contiguous in that one rw segment, so an overrun between them still does not fault—catching it would require a *guard page between separate segments*, i.e. taking over final linking, which is out of scope. What *is* achievable and valuable is the split itself as a *data-model* improvement (orthogonal to write protection): emit simple initialised mutable globals as file-backed `__DATA,__data` bytes (reusing the Stage-1 compile-time evaluator) instead of `kcc.module_init` runtime stores—standard, inspectable, faster startup, and it sidesteps the whole `module_init` init-bug class (the negative-float-in-array miscompile fixed during Stage 1 lived there). Because that is orthogonal to M5's write-protection theme and touches every initialised global, it is better tracked as its own item than folded into this milestone.

weft files. `37-codegen` (`const` detection at `_kcc_preregister_globals` + a new compile-time initialiser-to-bytes data buffer), `40-linker-info` (per-symbol section decision in `_kcc_linker_global_sym`, a new `_kcc_rodata_size` beside `_kcc_globals_size`), `39-emit-macho` (the new `section_64` header, its real file bytes, and the load-command/file-offset arithmetic), `32-ir-builder` (suppress the `module_init`/`main` init stores for a const object). Note: *not* segment protection—the system `ld` owns that.

Acceptance. A `static const` object lives in a read-only page (a deliberate write to one faults); the whole deterministic corpus—codegen, difftest, self-host—plus Lua and zlib stay green (re-baselined, since moving data to a new section legitimately changes the emitted bytes); and a controlled stray write into a `const` table now aborts at the store, with `lldb` reporting the writing instruction, instead of surfacing as distant corruption.

Status. Done (Stages 1 and 2; Stage 3 re-scoped out as architecturally impractical for write protection—see its item above). A `const` integer/floating scalar, array, struct, or array-of-struct whose initialiser is a literal, a negated literal, an integer constant expression, or an `enum` constant is write-protected in read-only `__TEXT`; and a `const` array of const pointers to string literals goes to read-only `__DATA_CONST,__const` with per-slot relocations. A laundered write to either faults with `SIGBUS` at the store. The whole corpus stays green (codegen 222, difftest 232 with twenty-one `rodata-*` cases, self-host 222, Lua `final OK` with *both* `optsnum[]` and `opts[]` now in read-only memory, zlib self-test); multi-TU `extern const` references and the `module_init` path are verified, and an object with no const pointer table is byte-unchanged. Remaining: only the optional Stage 3 and a few marginal Stage 1 shapes (non-literal-floating initialisers, aggregate-field/multi-dimensional structs). Being kind B, this milestone may proceed in parallel with M1–M4 and need not precede the seam refactor—but doing it *early* would have turned three past heisenbugs (Bugs 32, 38, 45) into one-line faults at the store site, which is the standing argument for scheduling oracle-hardening sooner rather than later.

B.8. M6 — Carve the Target Interface

Goal. Perform the structural refactor that turns "the ARM64 backend" into "the first instance of a target interface," so a second target slots in beside it rather than forking the backend. This is the step that literally *enables* M7.

Work items. Following the retargeting playbook (Chapter 2): express register banks as *descriptors* (so the linear-scan allocator becomes bank-parameterized rather than ARM64-wired), lift the ABI into a *table* (argument/return classification, callee-saved sets, alignment), and route instruction selection, prologue/epilogue, and object emission through a *target vtable*. Add the `--target` selection

plumbing to the driver. Throughout, ARM64 remains the sole concrete target and must keep passing through the new indirection.

weft files. `37/37a/37b/37c-codegen` (behind the vtable), `38/39-emit` and `40-linker` (object-format parameterization), `42-driver` (`--target`); the interface itself is a new backend-infrastructure fragment.

Acceptance. The *whole* suite—most importantly self-host and `difftest`—stays **byte-identical** after the refactor: a pure structural change to ARM64's path must produce the exact same bytes it did before. This is why M6 comes last of the six: it is validated by re-running the by-now-complete M1–M5 suites through the refactored path.

Status. Open. The playbook (Chapter 2) already specifies the interface shape; this milestone is its implementation for ARM64.

B.9. M7 — The Second Target (Hand-off)

With M1–M6 complete, adding x86-64 / Mach-O (then ELF / Linux) is the thirty-percent problem the playbook describes: an instruction-selection table, a register-bank descriptor, an ABI table, and an object emitter, validated differentially against the ARM64 oracle and run under Rosetta / qemu. The mechanics of M7 live in the retargeting playbook (Chapter 2).

Status (in progress, *skeleton-first*). The second target has in fact been *started* ahead of M6, by a deliberate change of plan. Rather than carve the full target interface first and then populate it, the x86-64 back end was grown as a separate, self-contained emitter (Chapter 38) that walks the same optimized, target-neutral IR and prints x86-64 assembly for the system assembler—the playbook's *assembly-emission* strategy. This proves the seam end to end at every step while leaving ARM64 byte-identical (it stays the default; the driver branches only under `--target=x86_64`). Five rungs have landed: rung one (`const` + `ret`, the whole pipeline proven with `int main(void){return N;}`), rung two (the integer ALU and the memory operations—`alloca`/`load`/`store`/`gep`—via an all-spill `%rbp` frame), rung three (comparisons, branches, and System V calls, including recursion and stack-passed arguments), rung four (data emission—mutable file-scope globals and string literals—plus the GOT-indirect global reference that lets a program call `printf` and link in one step), and rung five (read-only data: `const` globals, `static` locals, and struct/array images become fixed byte tables in the constant section). Together they compile the scalar language and more: zlib's `deflate`/`inflate` core, compiled by kcc for x86-64 and linked against the C library, performs a byte-exact round-trip. What remains for the *code generator* is floating point and by-value aggregates; and M6 proper—refactoring ARM64 to *share* this target's interface rather than run beside it—is still the eventual consolidation.

B.9.1. Two Milestones for Linux

The x86-64 emitter answers "can kcc *write* code for another machine?". Running on Linux asks two sharper questions, and it helps to name them apart, because they are reached by different work and the first is a prerequisite for neither more nor less than a useful cross-compiler.

Milestone A — a program kcc compiles runs natively on x86/Linux. Here kcc is the cross-compiler and the Linux machine merely runs the result. Three things differ from macOS, and all three live below the language, in the *runtime floor* the program stands on: the system-call mechanism (Linux uses the `syscall` instruction with its own call numbers, not `svc #0x80` with Mach numbers), the process entry point (an ELF `_start`, not a Mach-O `_main`), and the final link (a static image with no `libSystem`). None of these is the compiler's front or middle end; each is a thin, hand-written slab of platform assembly. Chapter 39 is exactly that slab for Linux/x86-64—an ELF `_start` and the `kcc_sys_*` wrappers—and `make meta-a` is its proof: a two-file C program, compiled by kcc under `--target=x86_64-linux`, linked against that runtime with no C library, maps arena memory, calls across a translation unit, prints, and exits with the status the harness expects, running on a Linux kernel. This half of Milestone A is *closed*.

Milestone B — kcc itself runs, and re-compiles itself, on x86/Linux. This is the true trophy of the second target, and it is gated not on the code generator but on the compiler's own *portability as source*. Today kcc is largely ARM64 assembly, which no x86 machine can run. The self-hosting chapters (beginning at Chapter 48) are the answer: one subsystem at a time, the assembly is rewritten in C, compiled by kcc, and linked back in place of the original object, verified byte-for-byte by re-running the whole suite through the swapped-in implementation. Each such port also compiles cleanly for x86-64, so with every subsystem the compiler's source grows more target-neutral by construction. When the last subsystem is C, the Linux kcc follows for free: its source is built—by a stock C compiler, or by kcc's own x86 back end—and linked against the same runtime floor Milestone A established. Nothing new is invented at that point; the two tracks simply meet. The north star beyond both, compiling the Linux kernel itself, is then a matter of breadth—headers, inline assembly, the last dark corners of the language—rather than of any missing seam.

The divergence from the "M6 before M7" order was a judgement that a running, differentially-tested skeleton is a better foundation for the interface than a carved-but-empty one.

B.10. Summary

The order is not a preference; it falls out of the seam. M1–M4 finish and prove the target-neutral seventy percent while a single target still makes every failure unambiguous. M5 write-protects read-only data so the ARM64 oracle turns stray writes into hard faults at the store instead of silent, layout-sensitive corruption—the oracle must not lie before it is trusted to judge a second backend. M6 carves the interface with ARM64 as the only occupant, proven by

byte-identical output. Only then does M7 introduce a second occupant—by which point the shared core is complete, the optimizer is inherited free, the verifier guards the seam, and ARM64 is a trustworthy oracle. *Finish the shared part before you split*; the rest is instruction tables.

../Makefile defined at 38k
hello.c defined at 38j
include/assert.h defined at 32e
include/ctype.h defined at 32c
include/errno.h defined at 32g
include/fcntl.h defined at 32p
include/fenv.h defined at 32l
include/inttypes.h defined at 32h
include/locale.h defined at 32k
include/math.h defined at 32d
include/setjmp.h defined at 32j
include/signal.h defined at 32i
include/stdio.h defined at 31g
include/stdlib.h defined at 32a
include/string.h defined at 32b
include/time.h defined at 32f
include/uchar.h defined at 32o
include/unistd.h defined at 32q
include/wchar.h defined at 32m
include/wctype.h defined at 32n
repro/01-image.sh defined at 38c
repro/02-build-kcc.sh defined at 38d
repro/03-kernel.sh defined at 38e
repro/04-initramfs.sh defined at 38f
repro/05-boot.sh defined at 38g
repro/Dockerfile defined at 38b
repro/run-all.sh defined at 38h
runtime/demo_main.c defined at 37h
runtime/demo_util.c defined at 37g
runtime/meta_a_verify.sh defined at 37i
runtime/meta_c_verify.sh defined at 37j
runtime/os_linux.s defined at 37f
selfhost/arena.c defined at 33c
selfhost/codegen.c defined at 36n
selfhost/codegen_verify.sh defined at 36o
selfhost/conversions.c defined at 34m
selfhost/conversions_verify.sh defined at 34n
selfhost/declarations.c defined at 36b
selfhost/declarations_verify.sh defined at 36c
selfhost/diag.c defined at 34i
selfhost/diag_verify.sh defined at 34j
selfhost/driver.c defined at 35p
selfhost/driver_verify.sh defined at 36a
selfhost/dwarf.c defined at 36d
selfhost/dwarf_verify.sh defined at 36e
selfhost/emit_arm64.c defined at 36f
selfhost/emit_arm64_verify.sh defined at 36g
selfhost/emit_macho.c defined at 35j
selfhost/emit_macho_verify.sh defined at 35k
selfhost/emit_x86.c defined at 36j
selfhost/emit_x86_verify.sh defined at 36k
selfhost/expressions.c defined at 35l
selfhost/expressions_verify.sh defined at 35m
selfhost/hashtab.c defined at 34a
selfhost/hashtab_verify.sh defined at 34b
selfhost/io.c defined at 34e
selfhost/io_verify.sh defined at 34f
selfhost/ir.c defined at 36p
selfhost/ir_verify.sh defined at 37a
selfhost/lexer.c defined at 33e
selfhost/lexer_core.c defined at 37d
selfhost/lexer_core_verify.sh defined at 37e
selfhost/lexer_tables.c defined at 35d
selfhost/lexer_tables_verify.sh defined at 35e
selfhost/lexer_verify.sh defined at 33f
selfhost/linker.c defined at 35f
selfhost/linker_verify.sh defined at 35g
selfhost/main.c defined at 37b
selfhost/main_verify.sh defined at 37c
selfhost/opt.c defined at 35b
selfhost/opt_verify.sh defined at 35c
selfhost/parser.c defined at 35n
selfhost/parser_verify.sh defined at 35o
selfhost/preprocessor.c defined at 36l
selfhost/preprocessor_verify.sh defined at 36m
selfhost/selfhost.sh defined at 33d
selfhost/sema.c defined at 36h
selfhost/sema_verify.sh defined at 36i
selfhost/source.c defined at 34g
selfhost/source_verify.sh defined at 34h
selfhost/statements.c defined at 35h
selfhost/statements_verify.sh defined at 35i
selfhost/strings.c defined at 33i
selfhost/strings_verify.sh defined at 33j
selfhost/strtab.c defined at 34c
selfhost/strtab_verify.sh defined at 34d
selfhost/symtab.c defined at 34k
selfhost/symtab_verify.sh defined at 34l
selfhost/types.c defined at 34o
selfhost/types_verify.sh defined at 35a
selfhost/vec.c defined at 33g
selfhost/vec_verify.sh defined at 33h
src/arena.s defined at 27c
src/codegen.s defined at 31c
src/conversions.s defined at 29f
src/declarations.s defined at 29c
src/diag.s defined at 28b
src/driver.s defined at 31f
src/dwarf.s defined at 31e
src/emit_arm64.s defined at 30d
src/emit_macho.s defined at 31d
src/emit_x86.s defined at 31a
src/expressions.s defined at 29a
src/hashtab.s defined at 27f
src/io.s defined at 27e
src/ir.s defined at 30b
src/lexer.s defined at 28d
src/lexer_tables.s defined at 28e
src/linker.s defined at 31b
src/main.s defined at 27a
src/opt.s defined at 30c
src/parser.s defined at 28g
src/preprocessor.s defined at 28f
src/sema.s defined at 30a
src/source.s defined at 28c
src/statements.s defined at 29b
src/strings.s defined at 27d
src/strtab.s defined at 28a
src/symtab.s defined at 29e
src/syscalls.s defined at 27b
src/types.s defined at 29d
src/vec.s defined at 27g
tests/diagtest.sh defined at 37n
tests/difftest.sh defined at 37k
tests/e2e.sh defined at 37m
tests/interop.sh defined at 37l
tests/test_codegen.sh defined at 33b
tests/test_lexer.s defined at 33a
tests/test_runner.s defined at 32r
tests/warntest.sh defined at 38a
VERSION defined at 38i

⟨ *Arena: alloc fast path* 58a ⟩ used in 27c
⟨ *Arena: alloc link new chunk* 58c ⟩ used in 58b
⟨ *Arena: alloc slow path* 58b ⟩ used in 27c
⟨ *Arena: constants* 57a ⟩ used in 27c
⟨ *Arena: data* 59 ⟩ used in 27c
⟨ *Arena: init* 57b ⟩ used in 27c
⟨ *Arena: init chunk header* 57d ⟩ used in 57b
⟨ *Arena: init mmap call* 57c ⟩ used in 57b

⟨ ARM64: common macros 51 ⟩ used in 27abcdefg, 28abcdefg, 29abcdef, 30abcd, 31abcdef, 32r, 33a
⟨ Backend: code buffer 525b, 526a ⟩ used in 30d
⟨ Backend: constants 525a ⟩ used in 30d
⟨ Backend: emit data 533c ⟩ used in 30d
⟨ Backend: encode adrp 533b ⟩ used in 30d
⟨ Backend: encode arithmetic imm 527a ⟩ used in 30d
⟨ Backend: encode arithmetic reg 526b ⟩ used in 30d
⟨ Backend: encode branch 532ab ⟩ used in 30d
⟨ Backend: encode compare 528b ⟩ used in 30d
⟨ Backend: encode floating point 530b ⟩ used in 30d
⟨ Backend: encode logical 527b, 528a ⟩ used in 30d
⟨ Backend: encode memory 529ab, 530a ⟩ used in 30d
⟨ Backend: encode system 533a ⟩ used in 30d
⟨ CodeGen: asm 522b ⟩ used in 31c
⟨ CodeGen: compile all 472, 473 ⟩ used in 31c
⟨ CodeGen: compile function 493, 496, 498ab ⟩ used in 31c
⟨ CodeGen: constants 463a ⟩ used in 31c
⟨ CodeGen: data 479 ⟩ used in 31c
⟨ CodeGen: emit strings 475, 476, 477ab, 478 ⟩ used in 31c
⟨ CodeGen: floating point 511 ⟩ used in 31c
⟨ CodeGen: init 463b ⟩ used in 31c
⟨ CodeGen: instruction dispatch 504, 505, 506, 508, 509, 510, 513, 516, 517, 518, 521 ⟩ used in 31c
⟨ CodeGen: large immediate adjust 495 ⟩ used in 31c
⟨ CodeGen: marshal args 502 ⟩ used in 31c
⟨ CodeGen: patch branches 474 ⟩ used in 31c
⟨ CodeGen: preregister globals 465, 468a, 469 ⟩ used in 31c
⟨ CodeGen: register allocation 481 ⟩ used in 31c
⟨ CodeGen: rodata write LE 468b ⟩ used in 469
⟨ CodeGen: slot colouring 490 ⟩ used in 31c
⟨ CodeGen: vreg slot offset 499ab ⟩ used in 31c
⟨ Concat: adjacent strings 165 ⟩ used in 28f
⟨ Concat: data 166 ⟩ used in 28f
⟨ Conv: array decay 296a ⟩ used in 29f
⟨ Conv: func decay 296b ⟩ used in 29f
⟨ Conv: implicit 297b ⟩ used in 29f
⟨ Conv: integer promotions 295a ⟩ used in 29f
⟨ Conv: lvalue 297a ⟩ used in 29f
⟨ Conv: usual arithmetic 295b ⟩ used in 29f
⟨ Decl: classify layout attribute 186a ⟩ used in 184b
⟨ Decl: data 205b ⟩ used in 29c
⟨ Decl: fold layout attrs into x25/x26 200a ⟩ used in 198, 199b
⟨ Decl: is type token 202 ⟩ used in 29c
⟨ Decl: parse abstract 196 ⟩ used in 29c
⟨ Decl: parse declaration 181, 182, 184a, 191a ⟩ used in 29c
⟨ Decl: parse declarator 190 ⟩ used in 29c
⟨ Decl: parse direct declarator 191b, 192, 195 ⟩ used in 29c
⟨ Decl: parse enum 200b ⟩ used in 29c
⟨ Decl: parse init 203 ⟩ used in 29c
⟨ Decl: parse specifiers 184b, 186b, 188, 189 ⟩ used in 29c
⟨ Decl: parse static assert 205a ⟩ used in 29c
⟨ Decl: parse struct 198, 199ab ⟩ used in 29c
⟨ Decl: parse type name 201 ⟩ used in 29c
⟨ Diag: caret 75 ⟩ used in 28b
⟨ Diag: constants 73a ⟩ used in 28bg, 29e, 30a
⟨ Diag: data 77a ⟩ used in 28b
⟨ Diag: emit 73c ⟩ used in 28b
⟨ Diag: error 76a ⟩ used in 28b
⟨ Diag: had errors 76d ⟩ used in 28b
⟨ Diag: note 76c ⟩ used in 28b
⟨ Diag: putu stderr 73b ⟩ used in 28b
⟨ Diag: reset 76e ⟩ used in 28b
⟨ Diag: tests 77b ⟩ used in 37n
⟨ Diag: warning 76b ⟩ used in 28b
⟨ Difftest: aggregates 919b ⟩ used in 37k
⟨ Difftest: arithmetic 918a ⟩ used in 37k
⟨ Difftest: combinations 930 ⟩ used in 37k
⟨ Difftest: control 918b ⟩ used in 37k
⟨ Difftest: divergences 933 ⟩ used in 37k
⟨ Difftest: floating point 922 ⟩ used in 37k
⟨ Difftest: functions 919a ⟩ used in 37k
⟨ Difftest: gaps 934a ⟩ used in 37k
⟨ Difftest: language 931 ⟩ used in 37k
⟨ Difftest: operators 923 ⟩ used in 37k
⟨ Difftest: results 934b ⟩ used in 37k
⟨ Difftest: runner 917 ⟩ used in 37k
⟨ Difftest: struct layout 921 ⟩ used in 37k
⟨ Driver: compile 603, 606ab, 607, 608ab, 609 ⟩ used in 31f
⟨ Driver: data 615 ⟩ used in 31f
⟨ Driver: link 611 ⟩ used in 31f
⟨ DWARF: build 596 ⟩ used in 31e
⟨ DWARF: constants 595a ⟩ used in 31e
⟨ DWARF: data 601 ⟩ used in 31e
⟨ DWARF: emit 595b ⟩ used in 31e
⟨ DWARF: macho 600 ⟩ used in 31e
⟨ E2E: runner 616b ⟩ used in 37m
⟨ Emit-asm: buffer 535b ⟩ used in 30d
⟨ Emit-asm: data 535a ⟩ used in 30d
⟨ Emit-asm: render 537, 543 ⟩ used in 30d
⟨ Encoding: translate 85 ⟩ used in 28c
⟨ Expr: data 223b ⟩ used in 29a
⟨ Expr: parse arg list 223a ⟩ used in 29a
⟨ Expr: parse assign 208a ⟩ used in 29a
⟨ Expr: parse binary 209a ⟩ used in 29a
⟨ Expr: parse cast 211 ⟩ used in 29a
⟨ Expr: parse conditional 208b ⟩ used in 29a
⟨ Expr: parse expression 207b ⟩ used in 29a
⟨ Expr: parse float 220 ⟩ used in 29a
⟨ Expr: parse postfix 212, 214 ⟩ used in 29a
⟨ Expr: parse primary 215, 218, 219 ⟩ used in 29a
⟨ Expr: parse unary 209b, 210 ⟩ used in 29a
⟨ Expr: precedence table 207a ⟩ used in 29a
⟨ Hash: constants 61b ⟩ used in 27f, 30a
⟨ Hash: FNV-1a 61a ⟩ used in 27f
⟨ Hash: get 62 ⟩ used in 27f
⟨ Hash: init 61c ⟩ used in 27f
⟨ Hash: put 63 ⟩ used in 27f
⟨ Header: assert 156b ⟩ used in 32e
⟨ Header: ctype 155c ⟩ used in 32c
⟨ Header: errno 156d ⟩ used in 32g
⟨ Header: fcntl 158e ⟩ used in 32p
⟨ Header: fenv 158a ⟩ used in 32l
⟨ Header: inttypes 157a ⟩ used in 32h
⟨ Header: locale 157d ⟩ used in 32k
⟨ Header: math 156a ⟩ used in 32d
⟨ Header: setjmp 157c ⟩ used in 32j
⟨ Header: signal 157b ⟩ used in 32i
⟨ Header: stdio 154 ⟩ used in 31g
⟨ Header: stdlib 155a ⟩ used in 32a
⟨ Header: string 155b ⟩ used in 32b
⟨ Header: time 156c ⟩ used in 32f
⟨ Header: uchar 158d ⟩ used in 32o
⟨ Header: unistd 158f ⟩ used in 32q
⟨ Header: wchar 158b ⟩ used in 32m
⟨ Header: wctype 158c ⟩ used in 32n
⟨ Interop: cases 935b ⟩ used in 37l
⟨ Interop: results 936 ⟩ used in 37l
⟨ Interop: runner 935a ⟩ used in 37l
⟨ IO: data 69a ⟩ used in 27e
⟨ IO: err 71b ⟩ used in 27e
⟨ IO: flush 69b ⟩ used in 27e
⟨ IO: putc 69c ⟩ used in 27e
⟨ IO: puts 70a ⟩ used in 27e
⟨ IO: putu 71a ⟩ used in 27e
⟨ IO: write 70b ⟩ used in 27e
⟨ IR: access width 349b ⟩ used in 30b
⟨ IR: aggregate init helpers 407, 413, 414 ⟩ used in 30b
⟨ IR: alloc basic block 303b ⟩ used in 30b
⟨ IR: alloc function 303c ⟩ used in 30b

⟨ IR: alloc instruction 303a ⟩ used in 30b
⟨ IR: bitfield access 359 ⟩ used in 30b
⟨ IR: build 307, 308, 312, 314ab, 316 ⟩ used in 30b
⟨ IR: build asm operands 325 ⟩ used in 30b
⟨ IR: build assign 387 ⟩ used in 30b
⟨ IR: build binary 369, 374, 375, 376 ⟩ used in 30b
⟨ IR: build break 434b, 435a ⟩ used in 30b
⟨ IR: build call 393 ⟩ used in 30b
⟨ IR: build cast 402b ⟩ used in 30b
⟨ IR: build conditional 400c ⟩ used in 30b
⟨ IR: build continue 435b ⟩ used in 30b
⟨ IR: build do 423b ⟩ used in 30b
⟨ IR: build expression 340, 342 ⟩ used in 30b
⟨ IR: build for 424 ⟩ used in 30b
⟨ IR: build function 317 ⟩ used in 30b
⟨ IR: build goto 321 ⟩ used in 30b
⟨ IR: build ident 367 ⟩ used in 30b
⟨ IR: build if 421c ⟩ used in 30b
⟨ IR: build increment/decrement 382, 383, 384 ⟩ used in 30b
⟨ IR: build int literal 343 ⟩ used in 30b
⟨ IR: build member 355, 357 ⟩ used in 30b
⟨ IR: build param 320 ⟩ used in 30b
⟨ IR: build return 326 ⟩ used in 30b
⟨ IR: build sizeof 347 ⟩ used in 30b
⟨ IR: build statement 322 ⟩ used in 30b
⟨ IR: build string literal 400b ⟩ used in 30b
⟨ IR: build subscript 349a, 350, 353a ⟩ used in 30b
⟨ IR: build switch 426 ⟩ used in 30b
⟨ IR: build unary 378b ⟩ used in 30b
⟨ IR: build var decl 417b ⟩ used in 30b
⟨ IR: build while 423a ⟩ used in 30b
⟨ IR: builder data 328 ⟩ used in 30b
⟨ IR: constants 301 ⟩ used in 30bc, 31ace
⟨ IR: data 305c ⟩ used in 30b
⟨ IR: dump 334 ⟩ used in 30b
⟨ IR: dump instruction 335, 336, 337, 338 ⟩ used in 30b
⟨ IR: dump type short 339 ⟩ used in 30b
⟨ IR: emit 304 ⟩ used in 30b
⟨ IR: init 302 ⟩ used in 30b
⟨ IR: next vreg 305a ⟩ used in 30b
⟨ IR: set current bb 305b ⟩ used in 30b
⟨ IR: static local helper 417a ⟩ used in 30b
⟨ IR: struct address helper 405 ⟩ used in 30b
⟨ IR: struct copy helper 404 ⟩ used in 30b
⟨ IR: validate 441 ⟩ used in 30b
⟨ IR: variadic builtins 398 ⟩ used in 30b
⟨ Lexer: data 104b ⟩ used in 28d
⟨ Lexer: dump all 100a ⟩ used in 28d
⟨ Lexer: dump token 99 ⟩ used in 28d
⟨ Lexer: init 90a ⟩ used in 28d
⟨ Lexer: keyword data 103 ⟩ used in 28e
⟨ Lexer: keyword init 104a ⟩ used in 28e
⟨ Lexer: next token 90b, 91ab ⟩ used in 28d
⟨ Lexer: record location 92 ⟩ used in 28d
⟨ Lexer: scan char 95 ⟩ used in 28d
⟨ Lexer: scan identifier 93 ⟩ used in 28d
⟨ Lexer: scan number 94a ⟩ used in 28d
⟨ Lexer: scan punctuator 96, 97ab, 98 ⟩ used in 28d
⟨ Lexer: scan string 94b ⟩ used in 28d
⟨ Lexer: token constants 89 ⟩ used in 28dfg, 29abcdef, 30ab, 33a
⟨ Lexer: token names 100b, 101ab ⟩ used in 28e
⟨ Linker: add reloc 592b ⟩ used in 31b
⟨ Linker: add symbol 592a ⟩ used in 31b
⟨ Linker: constants 591a ⟩ used in 30d, 31bcd
⟨ Linker: data 594 ⟩ used in 31b
⟨ Linker: global sym 593 ⟩ used in 31b
⟨ Linker: init 591b ⟩ used in 31b
⟨ Linker: lookup or add extern 592c ⟩ used in 31b
⟨ MachO: constants 577a ⟩ used in 31d
⟨ MachO: data 590 ⟩ used in 31d
⟨ MachO: finalize 580, 581, 582, 583, 585, 586, 587, 588, 589ab ⟩ used in 31d
⟨ MachO: output buffer 577b, 578, 579 ⟩ used in 31d
⟨ Main: data 625 ⟩ used in 27a
⟨ Main: entry point 621, 622, 623, 624 ⟩ used in 27a
⟨ Opt: common subexpression elimination 457 ⟩ used in 30c
⟨ Opt: constant fold 447, 448, 449 ⟩ used in 30c
⟨ Opt: dead code elimination 453ab, 454 ⟩ used in 30c
⟨ Opt: pass manager 445ab ⟩ used in 30c
⟨ Opt: strength reduction 450 ⟩ used in 30c
⟨ Parser: AST alloc 168a ⟩ used in 28g
⟨ Parser: AST constants 167 ⟩ used in 28g, 29abcdef, 30ab, 31ac
⟨ Parser: data 178ab, 179 ⟩ used in 28g
⟨ Parser: dump AST 173, 174ab, 175ab, 176 ⟩ used in 28g
⟨ Parser: dump helpers 177 ⟩ used in 28g
⟨ Parser: error recovery 169 ⟩ used in 28g
⟨ Parser: helpers 168b ⟩ used in 28g
⟨ Parser: translation unit 172 ⟩ used in 28g
⟨ Parser: typedef table 170ab, 171 ⟩ used in 28g
⟨ PP: conditional 135, 136, 139ab, 140 ⟩ used in 28f
⟨ PP: constants 107a ⟩ used in 28f
⟨ PP: context 111, 112 ⟩ used in 28f
⟨ PP: data 116 ⟩ used in 28f
⟨ PP: define 117, 118 ⟩ used in 28f
⟨ PP: directive dispatch 109, 110 ⟩ used in 28f
⟨ PP: dump 115 ⟩ used in 28f
⟨ PP: error directive 159b ⟩ used in 28f
⟨ PP: eval 141, 142, 143, 144, 145ab, 146, 147ab ⟩ used in 28f
⟨ PP: expand 121, 123, 125ab, 127, 130, 131 ⟩ used in 28f
⟨ PP: helpers 113 ⟩ used in 28f
⟨ PP: include 151 ⟩ used in 28f
⟨ PP: include path 150 ⟩ used in 28f
⟨ PP: init 107b ⟩ used in 28f
⟨ PP: next token 108 ⟩ used in 28f
⟨ PP: pragma 119b ⟩ used in 28f
⟨ PP: predefined 132 ⟩ used in 28f
⟨ PP: resolve include 148 ⟩ used in 28f
⟨ PP: undef 119a ⟩ used in 28f
⟨ Preprocessor: builtin headers 153 ⟩ used in 28f
⟨ Repro: 01 build image 13a ⟩ used in 38c
⟨ Repro: 02 build kcc 13b ⟩ used in 38d
⟨ Repro: 03 build kernel 15a ⟩ used in 38e
⟨ Repro: 04 initramfs 15b ⟩ used in 38f
⟨ Repro: 05 boot 16 ⟩ used in 38g
⟨ Repro: Dockerfile 12 ⟩ used in 38b
⟨ Repro: run all 17 ⟩ used in 38h
⟨ Runtime-linux: constants 571a ⟩ used in 573a
⟨ Runtime-linux: demo main 573c ⟩ used in 37h
⟨ Runtime-linux: demo util 573b ⟩ used in 37g
⟨ Runtime-linux: file 573a ⟩ used in 37f
⟨ Runtime-linux: fileio 572a ⟩ used in 573a
⟨ Runtime-linux: meta-a verify 573d ⟩ used in 37i
⟨ Runtime-linux: meta-c verify 574 ⟩ used in 37j
⟨ Runtime-linux: mmap 572b ⟩ used in 573a
⟨ Runtime-linux: process 571c ⟩ used in 573a
⟨ Runtime-linux: start 571b ⟩ used in 573a
⟨ Selfhost: arena 640a ⟩ used in 33c
⟨ Selfhost: arena alloc 639c ⟩ used in 640a
⟨ Selfhost: arena init 639b ⟩ used in 640a
⟨ Selfhost: arena preamble 639a ⟩ used in 640a
⟨ Selfhost: cg asm 835 ⟩ used in 836a
⟨ Selfhost: cg color 827a ⟩ used in 836a
⟨ Selfhost: cg dispatch 831 ⟩ used in 836a
⟨ Selfhost: cg frame 827b ⟩ used in 836a
⟨ Selfhost: cg func 829 ⟩ used in 836a
⟨ Selfhost: cg init 821a ⟩ used in 836a
⟨ Selfhost: cg preamble 819 ⟩ used in 836a
⟨ Selfhost: cg prereg 823 ⟩ used in 836a
⟨ Selfhost: cg regalloc 824 ⟩ used in 836a
⟨ Selfhost: cg rodata 821b ⟩ used in 836a

⟨ Selfhost: cg strings 830 ⟩ used in 836a
⟨ Selfhost: codegen 836a ⟩ used in 36n
⟨ Selfhost: codegen verify 836b ⟩ used in 36o
⟨ Selfhost: conversions 668b ⟩ used in 34m
⟨ Selfhost: conversions arith 667b ⟩ used in 668b
⟨ Selfhost: conversions decay 667c ⟩ used in 668b
⟨ Selfhost: conversions implicit 668a ⟩ used in 668b
⟨ Selfhost: conversions preamble 667a ⟩ used in 668b
⟨ Selfhost: conversions verify 668c ⟩ used in 34n
⟨ Selfhost: declarations 738a ⟩ used in 36b
⟨ Selfhost: declarations abstract 733 ⟩ used in 738a
⟨ Selfhost: declarations aggregate 734 ⟩ used in 738a
⟨ Selfhost: declarations attrs 728 ⟩ used in 738a
⟨ Selfhost: declarations declarator 731 ⟩ used in 738a
⟨ Selfhost: declarations direct 732 ⟩ used in 738a
⟨ Selfhost: declarations preamble 727 ⟩ used in 738a
⟨ Selfhost: declarations rest 735 ⟩ used in 738a
⟨ Selfhost: declarations specs 730 ⟩ used in 738a
⟨ Selfhost: declarations verify 738b ⟩ used in 36c
⟨ Selfhost: diag 662b ⟩ used in 34i
⟨ Selfhost: diag caret 661b ⟩ used in 662b
⟨ Selfhost: diag emit 662a ⟩ used in 662b
⟨ Selfhost: diag preamble 661a ⟩ used in 662b
⟨ Selfhost: diag verify 662c ⟩ used in 34j
⟨ Selfhost: driver 725a ⟩ used in 35p
⟨ Selfhost: driver args 721 ⟩ used in 725a
⟨ Selfhost: driver compile 720 ⟩ used in 725a
⟨ Selfhost: driver compile-all 722 ⟩ used in 725a
⟨ Selfhost: driver link 724 ⟩ used in 725a
⟨ Selfhost: driver preamble 719 ⟩ used in 725a
⟨ Selfhost: driver spawn 723 ⟩ used in 725a
⟨ Selfhost: driver verify 725b ⟩ used in 36a
⟨ Selfhost: dwarf 742a ⟩ used in 36d
⟨ Selfhost: dwarf abbrev 740a ⟩ used in 742a
⟨ Selfhost: dwarf build 741b ⟩ used in 742a
⟨ Selfhost: dwarf info 740b ⟩ used in 742a
⟨ Selfhost: dwarf line 741a ⟩ used in 742a
⟨ Selfhost: dwarf preamble 739 ⟩ used in 742a
⟨ Selfhost: dwarf verify 742b ⟩ used in 36e
⟨ Selfhost: emit-macho 702a ⟩ used in 35j
⟨ Selfhost: emit-macho auxcmds 700b ⟩ used in 702a
⟨ Selfhost: emit-macho body 701 ⟩ used in 702a
⟨ Selfhost: emit-macho header 699 ⟩ used in 702a
⟨ Selfhost: emit-macho layout 698 ⟩ used in 702a
⟨ Selfhost: emit-macho preamble 697a ⟩ used in 702a
⟨ Selfhost: emit-macho sections 700a ⟩ used in 702a
⟨ Selfhost: emit-macho verify 702b ⟩ used in 35k
⟨ Selfhost: emit-macho writers 697b ⟩ used in 702a
⟨ Selfhost: emit_arm64 750b ⟩ used in 36f
⟨ Selfhost: emit_arm64 as-buffer 748a ⟩ used in 750b
⟨ Selfhost: emit_arm64 as-render 748b, 750a ⟩ used in 750b
⟨ Selfhost: emit_arm64 codebuf 747 ⟩ used in 750b
⟨ Selfhost: emit_arm64 encoders 745b ⟩ used in 750b
⟨ Selfhost: emit_arm64 preamble 745a ⟩ used in 750b
⟨ Selfhost: emit_arm64 verify 751 ⟩ used in 36g
⟨ Selfhost: emit_x86 792a ⟩ used in 36j
⟨ Selfhost: emit_x86 data 784 ⟩ used in 792a
⟨ Selfhost: emit_x86 dispatch 774 ⟩ used in 792a
⟨ Selfhost: emit_x86 frame 781 ⟩ used in 792a
⟨ Selfhost: emit_x86 general-init 789 ⟩ used in 792a
⟨ Selfhost: emit_x86 helpers 771 ⟩ used in 792a
⟨ Selfhost: emit_x86 preamble 769 ⟩ used in 792a
⟨ Selfhost: emit_x86 render 791 ⟩ used in 792a
⟨ Selfhost: emit_x86 rodata 787b ⟩ used in 792a
⟨ Selfhost: emit_x86 shapes 772 ⟩ used in 792a
⟨ Selfhost: emit_x86 stmt-expr static storage 787a ⟩ used in 784
⟨ Selfhost: emit_x86 verify 792b ⟩ used in 36k
⟨ Selfhost: expressions 710a ⟩ used in 35l
⟨ Selfhost: expressions core 704 ⟩ used in 710a
⟨ Selfhost: expressions literals 708 ⟩ used in 710a
⟨ Selfhost: expressions postfix 706 ⟩ used in 710a
⟨ Selfhost: expressions preamble 703 ⟩ used in 710a
⟨ Selfhost: expressions primary 707 ⟩ used in 710a
⟨ Selfhost: expressions unary 705 ⟩ used in 710a
⟨ Selfhost: expressions verify 710b ⟩ used in 35m
⟨ Selfhost: hashtab 650c ⟩ used in 34a
⟨ Selfhost: hashtab fnv1a 649b ⟩ used in 650c
⟨ Selfhost: hashtab get 649d ⟩ used in 650c
⟨ Selfhost: hashtab grow 650a ⟩ used in 650c
⟨ Selfhost: hashtab init 649c ⟩ used in 650c
⟨ Selfhost: hashtab preamble 649a ⟩ used in 650c
⟨ Selfhost: hashtab put 650b ⟩ used in 650c
⟨ Selfhost: hashtab verify 651 ⟩ used in 34b
⟨ Selfhost: io 656b ⟩ used in 34e
⟨ Selfhost: io err 656a ⟩ used in 656b
⟨ Selfhost: io flush 655b ⟩ used in 656b
⟨ Selfhost: io preamble 655a ⟩ used in 656b
⟨ Selfhost: io putc 655c ⟩ used in 656b
⟨ Selfhost: io puts 655d ⟩ used in 656b
⟨ Selfhost: io putu 655f ⟩ used in 656b
⟨ Selfhost: io verify 656c ⟩ used in 34f
⟨ Selfhost: io write 655e ⟩ used in 656b
⟨ Selfhost: ir 902a ⟩ used in 36p
⟨ Selfhost: ir aggregates 873, 874 ⟩ used in 902a
⟨ Selfhost: ir assigncall 886 ⟩ used in 902a
⟨ Selfhost: ir control 879 ⟩ used in 902a
⟨ Selfhost: ir design 840 ⟩ used in 902a
⟨ Selfhost: ir dump 900 ⟩ used in 902a
⟨ Selfhost: ir exprs 856, 859, 860, 862, 865, 872 ⟩ used in 902a
⟨ Selfhost: ir preamble 837 ⟩ used in 902a
⟨ Selfhost: ir spine 841 ⟩ used in 902a
⟨ Selfhost: ir verify 902b ⟩ used in 37a
⟨ Selfhost: lexcore 911 ⟩ used in 37d
⟨ Selfhost: lexcore verify 914 ⟩ used in 37e
⟨ Selfhost: lexer 643c ⟩ used in 33e
⟨ Selfhost: lexer all 643a ⟩ used in 643c
⟨ Selfhost: lexer classify 641c ⟩ used in 643c
⟨ Selfhost: lexer cursor 642a ⟩ used in 643c
⟨ Selfhost: lexer main 643b ⟩ used in 643c
⟨ Selfhost: lexer next 642b ⟩ used in 643c
⟨ Selfhost: lexer state 641b ⟩ used in 643c
⟨ Selfhost: lexer tokens 641a ⟩ used in 643c
⟨ Selfhost: lexer verify 644 ⟩ used in 33f
⟨ Selfhost: lexer-tables 684a ⟩ used in 35d
⟨ Selfhost: lexer-tables kw 683a ⟩ used in 684a
⟨ Selfhost: lexer-tables names 683b ⟩ used in 684a
⟨ Selfhost: lexer-tables verify 684b ⟩ used in 35e
⟨ Selfhost: linker 686b ⟩ used in 35f
⟨ Selfhost: linker add 685c ⟩ used in 686b
⟨ Selfhost: linker externs 686a ⟩ used in 686b
⟨ Selfhost: linker init 685b ⟩ used in 686b
⟨ Selfhost: linker preamble 685a ⟩ used in 686b
⟨ Selfhost: linker verify 686c ⟩ used in 35g
⟨ Selfhost: main 905 ⟩ used in 37b
⟨ Selfhost: main verify 909 ⟩ used in 37c
⟨ Selfhost: opt 681b ⟩ used in 35b
⟨ Selfhost: opt constfold 676 ⟩ used in 681b
⟨ Selfhost: opt copyprop 680a ⟩ used in 681b
⟨ Selfhost: opt cse 678 ⟩ used in 681b
⟨ Selfhost: opt dce 680b ⟩ used in 681b
⟨ Selfhost: opt preamble 675 ⟩ used in 681b
⟨ Selfhost: opt run 681a ⟩ used in 681b
⟨ Selfhost: opt strength 677 ⟩ used in 681b
⟨ Selfhost: opt verify 681c ⟩ used in 35c
⟨ Selfhost: parser 717a ⟩ used in 35n
⟨ Selfhost: parser core 714a ⟩ used in 717a
⟨ Selfhost: parser dump 715 ⟩ used in 717a
⟨ Selfhost: parser preamble 713 ⟩ used in 717a
⟨ Selfhost: parser typedef 714b ⟩ used in 717a
⟨ Selfhost: parser verify 717b ⟩ used in 35o

⟨ *Selfhost: pp builtin* 814 ⟩ used in 818a
⟨ *Selfhost: pp conditional* 807 ⟩ used in 818a
⟨ *Selfhost: pp core* 796 ⟩ used in 818a
⟨ *Selfhost: pp define* 799b ⟩ used in 818a
⟨ *Selfhost: pp dispatch* 799a ⟩ used in 818a
⟨ *Selfhost: pp eval* 809 ⟩ used in 818a
⟨ *Selfhost: pp expand* 800 ⟩ used in 818a
⟨ *Selfhost: pp helpers* 798 ⟩ used in 818a
⟨ *Selfhost: pp include* 811 ⟩ used in 818a
⟨ *Selfhost: pp preamble* 795 ⟩ used in 818a
⟨ *Selfhost: pp predefined* 806 ⟩ used in 818a
⟨ *Selfhost: pp strlit* 813 ⟩ used in 818a
⟨ *Selfhost: pp substitute* 803 ⟩ used in 818a
⟨ *Selfhost: preprocessor* 818a ⟩ used in 36l
⟨ *Selfhost: preprocessor verify* 818b ⟩ used in 36m
⟨ *Selfhost: sema* 768a ⟩ used in 36h
⟨ *Selfhost: sema binunasgn* 763 ⟩ used in 768a
⟨ *Selfhost: sema callmembercast* 764 ⟩ used in 768a
⟨ *Selfhost: sema checks* 762b ⟩ used in 768a
⟨ *Selfhost: sema declarations* 758 ⟩ used in 768a
⟨ *Selfhost: sema dump* 767 ⟩ used in 768a
⟨ *Selfhost: sema expr* 760 ⟩ used in 768a
⟨ *Selfhost: sema exprident* 762a ⟩ used in 768a
⟨ *Selfhost: sema layout* 757 ⟩ used in 768a
⟨ *Selfhost: sema preamble* 753 ⟩ used in 768a
⟨ *Selfhost: sema resolve* 755 ⟩ used in 768a
⟨ *Selfhost: sema returnpath* 766 ⟩ used in 768a
⟨ *Selfhost: sema statements* 756 ⟩ used in 768a
⟨ *Selfhost: sema verify* 768b ⟩ used in 36i
⟨ *Selfhost: source* 659a ⟩ used in 34g
⟨ *Selfhost: source encode* 658a ⟩ used in 659a
⟨ *Selfhost: source open* 657b ⟩ used in 659a
⟨ *Selfhost: source preamble* 657a ⟩ used in 659a
⟨ *Selfhost: source splice* 658b ⟩ used in 659a
⟨ *Selfhost: source verify* 659b ⟩ used in 34h
⟨ *Selfhost: statements* 694b ⟩ used in 35h
⟨ *Selfhost: statements asm* 691b ⟩ used in 694b
⟨ *Selfhost: statements compound* 692 ⟩ used in 694b
⟨ *Selfhost: statements dispatch* 690 ⟩ used in 694b
⟨ *Selfhost: statements exprrest* 691a ⟩ used in 694b
⟨ *Selfhost: statements jumps* 694a ⟩ used in 694b
⟨ *Selfhost: statements preamble* 689 ⟩ used in 694b
⟨ *Selfhost: statements verify* 694c ⟩ used in 35i
⟨ *Selfhost: strings* 648a ⟩ used in 33i
⟨ *Selfhost: strings memcmp* 647d ⟩ used in 648a
⟨ *Selfhost: strings memcpy* 647b ⟩ used in 648a
⟨ *Selfhost: strings memset* 647c ⟩ used in 648a
⟨ *Selfhost: strings strcmp* 647e ⟩ used in 648a
⟨ *Selfhost: strings strlen* 647a ⟩ used in 648a
⟨ *Selfhost: strings verify* 648b ⟩ used in 33j
⟨ *Selfhost: strtab* 653a ⟩ used in 34c
⟨ *Selfhost: strtab verify* 653b ⟩ used in 34d
⟨ *Selfhost: symtab* 664c ⟩ used in 34k
⟨ *Selfhost: symtab labels* 664b ⟩ used in 664c
⟨ *Selfhost: symtab ordinary* 663c ⟩ used in 664c
⟨ *Selfhost: symtab preamble* 663a ⟩ used in 664c
⟨ *Selfhost: symtab scopes* 663b ⟩ used in 664c
⟨ *Selfhost: symtab tags* 664a ⟩ used in 664c
⟨ *Selfhost: symtab verify* 665 ⟩ used in 34l
⟨ *Selfhost: types* 672b ⟩ used in 34o
⟨ *Selfhost: types aggregates* 670b ⟩ used in 672b
⟨ *Selfhost: types constructors* 670a ⟩ used in 672b
⟨ *Selfhost: types dump* 672a ⟩ used in 672b
⟨ *Selfhost: types init* 669b ⟩ used in 672b
⟨ *Selfhost: types preamble* 669a ⟩ used in 672b
⟨ *Selfhost: types predicates* 671b ⟩ used in 672b
⟨ *Selfhost: types specmask* 671a ⟩ used in 672b
⟨ *Selfhost: types verify* 673 ⟩ used in 35a
⟨ *Selfhost: vec* 646a ⟩ used in 33g
⟨ *Selfhost: vec get* 645d ⟩ used in 646a
⟨ *Selfhost: vec init* 645b ⟩ used in 646a
⟨ *Selfhost: vec preamble* 645a ⟩ used in 646a
⟨ *Selfhost: vec push* 645c ⟩ used in 646a
⟨ *Selfhost: vec verify* 646b ⟩ used in 33h
⟨ *Selfhost: verify* 640b ⟩ used in 33d
⟨ *Sema: check* 259a ⟩ used in 30a
⟨ *Sema: check declaration* 264, 269 ⟩ used in 30a
⟨ *Sema: check expression* 274, 276 ⟩ used in 30a
⟨ *Sema: check statement* 271, 272, 273 ⟩ used in 30a
⟨ *Sema: data* 263 ⟩ used in 30a
⟨ *Sema: dump types* 262 ⟩ used in 30a
⟨ *Sema: expr assign* 287 ⟩ used in 30a
⟨ *Sema: expr binary* 284 ⟩ used in 30a
⟨ *Sema: expr call* 289 ⟩ used in 30a
⟨ *Sema: expr cast* 292a ⟩ used in 30a
⟨ *Sema: expr ident* 277 ⟩ used in 30a
⟨ *Sema: expr literal* 283 ⟩ used in 30a
⟨ *Sema: expr member* 290 ⟩ used in 30a
⟨ *Sema: expr sizeof* 292b ⟩ used in 30a
⟨ *Sema: expr unary* 286 ⟩ used in 30a
⟨ *Sema: layout bit-field* 268 ⟩ used in 267
⟨ *Sema: resolve type* 259b ⟩ used in 30a
⟨ *Sema: return path* 281 ⟩ used in 30a
⟨ *Sema: struct layout* 267 ⟩ used in 30a
⟨ *Source: constants* 81a ⟩ used in 28c
⟨ *Source: data* 82 ⟩ used in 28c
⟨ *Source: open file* 81b ⟩ used in 28c
⟨ *Splice: translate* 87 ⟩ used in 28c
⟨ *Stmt: data* 235 ⟩ used in 29b
⟨ *Stmt: parse asm* 227 ⟩ used in 29b
⟨ *Stmt: parse compound* 230 ⟩ used in 29b
⟨ *Stmt: parse do* 232b ⟩ used in 29b
⟨ *Stmt: parse for* 233 ⟩ used in 29b
⟨ *Stmt: parse goto* 234a ⟩ used in 29b
⟨ *Stmt: parse if* 231a ⟩ used in 29b
⟨ *Stmt: parse labeled* 234c ⟩ used in 29b
⟨ *Stmt: parse return* 234b ⟩ used in 29b
⟨ *Stmt: parse statement* 225, 226, 228 ⟩ used in 29b
⟨ *Stmt: parse switch* 231b ⟩ used in 29b
⟨ *Stmt: parse while* 232a ⟩ used in 29b
⟨ *String: memcmp* 67c ⟩ used in 27d
⟨ *String: memcpy* 67b ⟩ used in 27d
⟨ *String: memset* 67d ⟩ used in 27d
⟨ *String: strcmp* 68 ⟩ used in 27d
⟨ *String: strlen* 67a ⟩ used in 27d
⟨ *StringLit: data* 163 ⟩ used in 28f
⟨ *StringLit: process* 161, 162ab ⟩ used in 28f
⟨ *Strtab: data* 66c ⟩ used in 28a
⟨ *Strtab: init* 66a ⟩ used in 28a
⟨ *Strtab: intern* 66b ⟩ used in 28a
⟨ *Symtab: constants* 253a ⟩ used in 29e, 30ab
⟨ *Symtab: data* 258 ⟩ used in 29e
⟨ *Symtab: declare* 254b ⟩ used in 29e
⟨ *Symtab: declare tag* 256a ⟩ used in 29e
⟨ *Symtab: init* 253b ⟩ used in 29e
⟨ *Symtab: label scope* 257ab ⟩ used in 29e
⟨ *Symtab: lookup* 255 ⟩ used in 29e
⟨ *Symtab: lookup tag* 256b ⟩ used in 29e
⟨ *Symtab: pop scope* 254a ⟩ used in 29e
⟨ *Symtab: push scope* 253c ⟩ used in 29e
⟨ *Syscall: close* 54b ⟩ used in 27b
⟨ *Syscall: constants* 53a ⟩ used in 27bc, 28c
⟨ *Syscall: exit* 53b ⟩ used in 27b
⟨ *Syscall: lseek* 54d ⟩ used in 27b
⟨ *Syscall: mmap* 54c ⟩ used in 27b
⟨ *Syscall: open* 54a ⟩ used in 27b
⟨ *Syscall: process* 54e ⟩ used in 27b
⟨ *Syscall: read* 53d ⟩ used in 27b
⟨ *Syscall: write* 53c ⟩ used in 27b
⟨ *Test: assert helpers* 630a ⟩ used in 32r

⟨ *Test: codegen arrays* 353b ⟩ used in 33b
⟨ *Test: codegen compound* 392 ⟩ used in 33b
⟨ *Test: codegen constants* 522a ⟩ used in 33b
⟨ *Test: codegen control flow* 425 ⟩ used in 33b
⟨ *Test: codegen enums* 435d ⟩ used in 33b
⟨ *Test: codegen fnptr* 439 ⟩ used in 33b
⟨ *Test: codegen functions* 400a ⟩ used in 33b
⟨ *Test: codegen global init* 436c ⟩ used in 33b
⟨ *Test: codegen increment* 386 ⟩ used in 33b
⟨ *Test: codegen init lists* 437b ⟩ used in 33b
⟨ *Test: codegen locals* 420 ⟩ used in 33b
⟨ *Test: codegen logical* 378a ⟩ used in 33b
⟨ *Test: codegen misc* 435c ⟩ used in 33b
⟨ *Test: codegen multi-var* 421a ⟩ used in 33b
⟨ *Test: codegen multifile* 616a ⟩ used in 33b
⟨ *Test: codegen nested structs* 436a ⟩ used in 33b
⟨ *Test: codegen pointer arrays* 437a ⟩ used in 33b
⟨ *Test: codegen pointers* 381 ⟩ used in 33b
⟨ *Test: codegen preprocessor* 159a ⟩ used in 33b
⟨ *Test: codegen ptr arithmetic* 436d ⟩ used in 33b
⟨ *Test: codegen results* 638 ⟩ used in 33b
⟨ *Test: codegen runner* 637 ⟩ used in 33b
⟨ *Test: codegen self-hosting* 438b ⟩ used in 33b
⟨ *Test: codegen sizeof* 348 ⟩ used in 33b
⟨ *Test: codegen static locals* 438a ⟩ used in 33b
⟨ *Test: codegen struct arrays* 436b ⟩ used in 33b
⟨ *Test: codegen structs* 366 ⟩ used in 33b
⟨ *Test: codegen switch* 434a ⟩ used in 33b
⟨ *Test: codegen ternary* 402a ⟩ used in 33b
⟨ *Test: codegen unsigned* 421b ⟩ used in 33b
⟨ *Test: lex comments* 634b ⟩ used in 33a
⟨ *Test: lex data* 636b ⟩ used in 33a
⟨ *Test: lex keywords* 633a ⟩ used in 33a
⟨ *Test: lex line splice* 635b ⟩ used in 33a
⟨ *Test: lex numbers* 633b ⟩ used in 33a
⟨ *Test: lex punctuators* 634a ⟩ used in 33a
⟨ *Test: lex strings* 633c ⟩ used in 33a
⟨ *Test: lex trigraphs* 635a ⟩ used in 33a
⟨ *Test: registration* 636a ⟩ used in 33a
⟨ *Test: runner data* 630b ⟩ used in 32r
⟨ *Test: runner entry* 629 ⟩ used in 32r
⟨ *Tutorial: hello world* 6 ⟩ used in 38j
⟨ *Types: compatible* 247a ⟩ used in 29df, 30abc, 31ac
⟨ *Types: constants* 239a ⟩ used in 29df, 30abc, 31ac
⟨ *Types: data* 250b ⟩ used in 29d
⟨ *Types: dump* 249ab, 250a ⟩ used in 29d
⟨ *Types: init* 239b, 240, 241 ⟩ used in 29d
⟨ *Types: is arithmetic* 248a ⟩ used in 29d
⟨ *Types: is integer* 247b ⟩ used in 29d
⟨ *Types: is scalar* 248b ⟩ used in 29d
⟨ *Types: make array* 243a ⟩ used in 29d
⟨ *Types: make enum* 244 ⟩ used in 29d
⟨ *Types: make function* 243b ⟩ used in 29d
⟨ *Types: make pointer* 242ab ⟩ used in 29d
⟨ *Types: make struct* 243c ⟩ used in 29d
⟨ *Types: rank* 248c ⟩ used in 29d
⟨ *Types: resolve specmask* 245ab, 246 ⟩ used in 29d
⟨ *Vec: constants* 64a ⟩ used in 27g, 28g, 29e, 30abcd, 31abcde
⟨ *Vec: get* 65b ⟩ used in 27g
⟨ *Vec: init* 64b ⟩ used in 27g
⟨ *Vec: push* 65a ⟩ used in 27g
⟨ *version* ? ⟩ used in 38i, 625, 905
⟨ *Warn: tests* 78 ⟩ used in 38a
⟨ *X86: data* 546, 547ab ⟩ used in 31a
⟨ *X86: render* 548ab, 549, 550, 551, 552ab, 554, 559, 560ab, 562, 563, 564, 567, 568ab ⟩ used in 31a

`_kcc_abst_splice`: 196.
`_kcc_addr_is_iconst`: 426.
`_kcc_arena_alloc`: 58a, 61c, 63, 64b, 65a, 66b, 85, 113, 117, 118, 119b, 121, 123, 125b, 127, 132, 136, 148, 150, 151, 159b, 161, 165, 168a, 239b, 240, 241, 242a, 243abc, 244, 254b, 256a, 259b, 277, 302, 303abc, 343, 359, 393, 417a, 447, 450, 453a, 457, 463b, 465, 472, 475, 476, 477b, 478, 481, 490, 493, 498a, 518, 521, 522b, 525b, 535b, 537, 577b, 586, 591b, 596, 603, 609, 611.
`_kcc_arena_current`: 57d, 58ac, 59.
`_kcc_arena_init`: 57b, 621, 629.
`_kcc_ast_alloc`: 168a, 172, 184b, 188, 189, 190, 191ab, 192, 195, 196, 199ab, 200b, 203, 205a, 207b, 208ab, 209ab, 210, 211, 212, 214, 215, 218, 225, 226, 227, 228, 230, 231ab, 232ab, 233, 234abc, 277, 325.
`_kcc_ast_dump`: 173, 174ab, 175ab, 176, 622.
`_kcc_ast_indent`: 177, 178a.
`_kcc_ast_indent_dec`: 174ab, 175ab, 176, 177.
`_kcc_ast_indent_inc`: 174ab, 175ab, 176, 177.
`_kcc_ast_n_alignof_expr`: 179.
`_kcc_ast_n_assign_expr`: 179.
`_kcc_ast_n_binary_expr`: 179.
`_kcc_ast_n_break_stmt`: 179.
`_kcc_ast_n_call_expr`: 179.
`_kcc_ast_n_case_stmt`: 179.
`_kcc_ast_n_cast_expr`: 179.
`_kcc_ast_n_char_lit`: 179.
`_kcc_ast_n_comma_expr`: 179.
`_kcc_ast_n_compound`: 179.
`_kcc_ast_n_compound_lit`: 179.
`_kcc_ast_n_cond_expr`: 179.
`_kcc_ast_n_continue_stmt`: 179.
`_kcc_ast_n_default_stmt`: 179.
`_kcc_ast_n_do_stmt`: 179.
`_kcc_ast_n_enumerator`: 179.
`_kcc_ast_n_enum_decl`: 179.
`_kcc_ast_n_expr_stmt`: 179.
`_kcc_ast_n_field_decl`: 179.
`_kcc_ast_n_for_stmt`: 179.
`_kcc_ast_n_func_decl`: 179.
`_kcc_ast_n_generic`: 179.
`_kcc_ast_n_generic_assoc`: 179.
`_kcc_ast_n_goto_stmt`: 179.
`_kcc_ast_n_ident_expr`: 179.
`_kcc_ast_n_if_stmt`: 179.
`_kcc_ast_n_init_list`: 179.
`_kcc_ast_n_int_lit`: 179.
`_kcc_ast_n_label_stmt`: 179.
`_kcc_ast_n_member`: 179.
`_kcc_ast_n_null_stmt`: 179.
`_kcc_ast_n_param_decl`: 179.
`_kcc_ast_n_paren_expr`: 179.
`_kcc_ast_n_return_stmt`: 179.
`_kcc_ast_n_sizeof_expr`: 179.
`_kcc_ast_n_static_assert`: 179.
`_kcc_ast_n_struct_decl`: 179.
`_kcc_ast_n_str_lit`: 179.
`_kcc_ast_n_subscript`: 179.
`_kcc_ast_n_switch_stmt`: 179.
`_kcc_ast_n_trans_unit`: 179.
`_kcc_ast_n_typedef_decl`: 179.
`_kcc_ast_n_type_array`: 179.
`_kcc_ast_n_type_builtin`: 179.
`_kcc_ast_n_type_enum`: 179.
`_kcc_ast_n_type_func`: 179.
`_kcc_ast_n_type_name`: 179.
`_kcc_ast_n_type_ptr`: 179.
`_kcc_ast_n_type_quals`: 179.
`_kcc_ast_n_type_struct`: 179.
`_kcc_ast_n_type_union`: 179.
`_kcc_ast_n_unary_expr`: 179.
`_kcc_ast_n_union_decl`: 179.
`_kcc_ast_n_var_decl`: 179.

_kcc_ast_n_va_arg: 179.
_kcc_ast_n_while_stmt: 179.
_kcc_ast_print_indent: 173, 177.
_kcc_ast_print_name_field: 174ab, 175ab, 176, 177.
_kcc_ast_tag_name: 173, 177.
_kcc_cg_asm_extended: 504, 522b.
_kcc_cg_asm_subst: 522b.
_kcc_cg_asm_validate: 522b.
_kcc_cg_callee_mask: 479, 481, 499b.
_kcc_cg_callee_transfer: 493, 499b, 517.
_kcc_cg_color_slots: 481, 490.
_kcc_cg_defined_globals: 463b, 465, 479, 521.
_kcc_cg_emit_add_imm: 495, 513, 517.
_kcc_cg_emit_frame_ldst: 495, 496, 499b.
_kcc_cg_emit_sub_imm: 493, 495.
_kcc_cg_fpregmap: 479, 481, 490, 499b.
_kcc_cg_fp_callee_mask: 479, 481, 499b.
_kcc_cg_fp_save_base: 479, 493, 499b.
_kcc_cg_fp_save_count: 479, 481, 493.
_kcc_cg_frame_size: 463b, 479, 493, 513, 517.
_kcc_cg_load_vreg: 499b, 502, 506, 508, 509, 510, 511, 513, 516, 517, 518, 522b.
_kcc_cg_load_vreg_fp: 499b, 502, 506, 511, 513, 517.
_kcc_cg_marshal_args: 502, 518.
_kcc_cg_max_slot: 479, 481, 490, 493, 513.
_kcc_cg_normalize_ret: 518.
_kcc_cg_preregister_funcs: 472, 473, 498b.
_kcc_cg_prereg_static: 465.
_kcc_cg_prereg_stmt: 465.
_kcc_cg_promoted: 479, 481, 490, 493, 499b.
_kcc_cg_regalloc: 481, 493.
_kcc_cg_regmap: 479, 481, 490, 499b.
_kcc_cg_save_base: 479, 493, 499b.
_kcc_cg_save_count: 479, 481, 493.
_kcc_cg_slotmap: 479, 481, 490, 499a.
_kcc_cg_slot_offset: 496, 499a, 499b.
_kcc_cg_store_vreg: 477b, 499b, 505, 506, 508, 509, 510, 511, 513, 518, 521.
_kcc_cg_store_vreg_fp: 496, 499b, 506, 511, 513, 518.
_kcc_cg_vreg_count: 479, 493.
_kcc_codegen_all: 473, 603.
_kcc_codegen_emit_strings: 478, 603.
_kcc_codegen_func: 473, 493.
_kcc_codegen_init: 463b, 603.
_kcc_codegen_instr: 498a, 504.
_kcc_codegen_patch_branches: 474, 498b.
_kcc_codegen_string_lit: 477a, 521.
_kcc_code_buf: 474, 478, 525b, 526a, 533c, 537, 586.
_kcc_code_cap: 478, 525b, 533c.
_kcc_code_current_pos: 477b, 478, 493, 498a, 504, 516, 518, 521, 522b, 526a.
_kcc_code_emit32: 477b, 493, 495, 496, 499b, 502, 504, 505, 506, 508, 509, 510, 511, 513, 516, 517, 518, 521, 522b, 525b.
_kcc_code_init: 525b, 603.
_kcc_code_patch32: 474, 526a.
_kcc_code_pos: 478, 525b, 526a, 533c, 537, 577b, 580, 588, 596.
_kcc_conv_array_decay: 277, 296a.
_kcc_conv_func_decay: 277, 296b.
_kcc_conv_implicit: 297b.
_kcc_conv_lvalue: 297a.
_kcc_conv_promote: 284, 286, 295a, 295b, 359.
_kcc_conv_usual: 284, 295b, 359.
_kcc_cse_canon: 457.
_kcc_cse_canon_ops: 457.
_kcc_cstr_buf: 463b, 469, 475, 476, 477a, 478, 479.
_kcc_cstr_count: 463b, 476, 477b, 479.
_kcc_cstr_fixups: 463b, 469, 476, 477b, 478, 479.
_kcc_cstr_pos: 463b, 469, 475, 476, 477a, 478, 479, 537.
_kcc_cstr_reserve: 469, 475, 476, 477a.
_kcc_diag_caret: 73c, 75.
_kcc_diag_colon: 73c, 77a.
_kcc_diag_colon_space: 73c, 77a.
_kcc_diag_emit: 73c, 76abc, 169.
_kcc_diag_error: 76a, 91a, 94b, 95, 108, 111, 118, 125a, 135, 139ab, 151, 159b, 188, 259b, 273, 276, 277, 284, 286, 287, 289, 290.
_kcc_diag_error_count: 73c, 76de, 77a.
_kcc_diag_had_errors: 76d, 603, 622, 623.
_kcc_diag_newline: 73c, 75, 77a.
_kcc_diag_note: 76c.
_kcc_diag_reset: 76e, 629.
_kcc_diag_str_error: 73c, 77a.
_kcc_diag_str_note: 73c, 77a.
_kcc_diag_str_warning: 73c, 77a.
_kcc_diag_too_many: 73c, 77a.
_kcc_diag_unknown_file: 73c, 77a.
_kcc_diag_warning: 76b, 110, 277, 281, 284, 287.
_kcc_diag_warn_count: 73c, 76e, 77a.
_kcc_dir_exists: 611.
_kcc_driver_compile: 603, 609, 611.
_kcc_driver_compile_all: 609, 624.
_kcc_driver_D: 606b, 609, 611.
_kcc_driver_emit_asm_object: 603.
_kcc_driver_I: 606a, 609, 611.
_kcc_driver_link_all: 611, 621.
_kcc_driver_mflag: 607, 608b.
_kcc_driver_obj_name: 609, 611.
_kcc_driver_sdkroot: 611.
_kcc_driver_skipopt: 608b, 609, 611.
_kcc_driver_spawn: 603, 611.
_kcc_driver_x86_object: 603.
_kcc_driver_xflag: 608a, 608b.
_kcc_emit_cstring: 469, 476.
_kcc_emit_rodata_global: 465, 469.
_kcc_emit_rodata_ptrarr: 465, 469.
_kcc_emit_struct_fields: 469.
_kcc_enc_add_imm: 477b, 493, 495, 510, 518, 521, 527a.
_kcc_enc_add_reg: 506, 513, 526b.
_kcc_enc_adrp: 477b, 521, 533b.
_kcc_enc_and_reg: 508, 510, 527b.
_kcc_enc_asr_reg: 508, 528a.
_kcc_enc_b: 474, 516, 532a.
_kcc_enc_bl: 518, 532a.
_kcc_enc_blr: 518, 532b.
_kcc_enc_brk: 533a.
_kcc_enc_b_cond: 532b.
_kcc_enc_cbnz: 516, 532b.
_kcc_enc_cbz: 532b.
_kcc_enc_cmp_reg: 509, 528b.
_kcc_enc_cset: 509, 511, 528b.
_kcc_enc_eor_reg: 508, 527b.
_kcc_enc_fcmp: 511, 530b.
_kcc_enc_fcvtzs: 511, 530b.
_kcc_enc_fcvt_d_s: 511, 513, 530b.
_kcc_enc_fcvt_s_d: 511, 513, 530b.
_kcc_enc_fp_dp2: 511, 530b.
_kcc_enc_ldp_post: 517, 529b.
_kcc_enc_ldr_fp: 495, 530b.
_kcc_enc_ldr_fp_s: 513, 530b.
_kcc_enc_ldr_imm: 495, 502, 513, 517, 521, 529a.
_kcc_enc_lsl_reg: 508, 528a.
_kcc_enc_lsr_reg: 508, 528a.
_kcc_enc_movk: 505, 510, 513, 527a.
_kcc_enc_movz: 505, 510, 513, 527a.
_kcc_enc_mov_reg: 499b, 513, 530a.
_kcc_enc_msub: 506, 526b.
_kcc_enc_mul: 506, 526b.
_kcc_enc_mvn: 508, 527b.

_kcc_enc_nop: 504, 522b, 533a.
_kcc_enc_orr_reg: 508, 527b.
_kcc_enc_ret: 517, 532b.
_kcc_enc_scvtf: 511, 530b.
_kcc_enc_sdiv: 506, 526b.
_kcc_enc_stp_pre: 493, 529b.
_kcc_enc_str_fp: 495, 530b.
_kcc_enc_str_fp_s: 513, 530b.
_kcc_enc_str_imm: 495, 496, 502, 513, 518, 522b, 529a.
_kcc_enc_sub_imm: 495, 502, 527a.
_kcc_enc_sub_reg: 506, 526b.
_kcc_enc_ucvtf: 511, 530b.
_kcc_enc_udiv: 506, 526b.
_kcc_envp: 611, 615, 621.
_kcc_err_empty: 81b, 82.
_kcc_err_mmap: 81b, 82.
_kcc_err_open: 81b, 82.
_kcc_err_read: 81b, 82.
_kcc_err_unterm_char: 95, 104b.
_kcc_err_unterm_comment: 91a, 104b.
_kcc_err_unterm_string: 94b, 104b.
_kcc_eval_addr: 426.
_kcc_eval_const_int: 259b, 269, 308, 340, 407, 414, 426, 468a.
_kcc_expr_is_ice: 259b, 393, 426.
_kcc_fnv1a: 61a, 62, 63.
_kcc_getenv: 603, 611.
_kcc_globals_size: 543, 580, 588, 593, 594.
_kcc_global_const_ptrarr: 308, 312, 465, 562.
_kcc_global_const_rodata: 308, 312, 465, 562, 563.
_kcc_ht_get: 62, 66b, 93, 119b, 121, 135, 136, 146, 171, 255, 256b, 257b, 321, 322, 350, 355, 367, 378b, 382, 383, 387, 393, 398, 405, 474, 516, 521, 592c, 593.
_kcc_ht_init: 61c, 66a, 104a, 107b, 170ab, 253c, 257a, 302, 312, 316, 317, 463b, 493, 591b.
_kcc_ht_put: 63, 66b, 104a, 118, 119ab, 132, 171, 254b, 256a, 257b, 312, 320, 321, 322, 417b, 465, 469, 472, 498a, 592c, 593.
_kcc_io_buf: 69a, 69bc, 70ab.
_kcc_io_cap: 69a, 69c.
_kcc_io_err: 71b, 73c, 75, 81b, 522b, 554, 577b, 579, 607, 609, 611, 624.
_kcc_io_fd: 69a, 69b.
_kcc_io_flush: 69b, 69c, 70ab, 262, 334, 554, 603, 607, 609, 611, 622, 623, 624, 629.
_kcc_io_pos: 69a, 69bc, 70ab.
_kcc_io_puts: 70a, 99, 173, 175ab, 177, 249ab, 250a, 262, 334, 335, 336, 337, 338, 339, 441, 621, 623, 624, 629.
_kcc_io_putu: 71a, 99, 173, 175b, 250a, 334, 335, 336, 337, 338, 629.
_kcc_io_write: 70b, 71a, 99, 175b.
_kcc_ir_agg_count: 407, 414.
_kcc_ir_alloc_bb: 303b, 316, 317, 321, 322, 376, 400c, 421c, 423ab, 424, 426.
_kcc_ir_alloc_func: 303c, 316, 317.
_kcc_ir_alloc_instr: 303a, 312, 314b, 316, 317, 320, 321, 322, 326, 343, 347, 350, 353a, 355, 357, 359, 367, 369, 375, 376, 378b, 383, 384, 387, 393, 398, 400bc, 402b, 404, 405, 407, 413, 417b, 421c, 423ab, 424, 426, 435ab, 450.
_kcc_ir_anon_containing: 407, 414.
_kcc_ir_asm_addr_of: 325.
_kcc_ir_asm_lower_operands: 322, 325.
_kcc_ir_bb_counter: 302, 303b, 305c.
_kcc_ir_branch_to: 321, 322, 342, 426.
_kcc_ir_build: 307, 603, 623.
_kcc_ir_build_alignof: 340, 347.
_kcc_ir_build_assign: 340, 387, 405.
_kcc_ir_build_binary: 340, 369.
_kcc_ir_build_break: 322, 435a.
_kcc_ir_build_call: 340, 387, 393, 405.
_kcc_ir_build_cast: 340, 402b.
_kcc_ir_build_compound_lit: 340, 378b, 405, 413.
_kcc_ir_build_cond_expr: 340, 400c.
_kcc_ir_build_continue: 322, 435b.
_kcc_ir_build_do: 322, 423b.
_kcc_ir_build_expr: 314b, 322, 325, 326, 340, 342, 350, 355, 369, 376, 378b, 382, 387, 393, 400c, 402b, 405, 407, 417b, 421c, 423ab, 424, 426.
_kcc_ir_build_float_lit: 340, 343.
_kcc_ir_build_for: 322, 424.
_kcc_ir_build_func: 307, 317.
_kcc_ir_build_ident: 340, 367.
_kcc_ir_build_if: 322, 421c.
_kcc_ir_build_inc_dec: 378b, 382.
_kcc_ir_build_int_lit: 340, 343.
_kcc_ir_build_member: 340, 357.
_kcc_ir_build_member_addr: 328, 350, 355, 357, 378b, 382, 387, 398, 405.
_kcc_ir_build_param: 317, 320.
_kcc_ir_build_return: 322, 326.
_kcc_ir_build_sizeof: 340, 347.
_kcc_ir_build_stmt: 317, 322, 342, 421c, 423ab, 424, 426.
_kcc_ir_build_stmt_expr: 340, 342.
_kcc_ir_build_string_lit: 340, 400b.
_kcc_ir_build_struct_addr: 314b, 326, 393, 405, 414, 417b.
_kcc_ir_build_subscript: 340, 353a.
_kcc_ir_build_subscript_addr: 350, 353a, 355, 378b, 382, 387, 398, 405.
_kcc_ir_build_switch: 322, 426.
_kcc_ir_build_unary: 340, 378b.
_kcc_ir_build_var_decl: 322, 350, 417b.
_kcc_ir_build_va_arg: 340, 387, 398.
_kcc_ir_build_va_start: 393, 398.
_kcc_ir_build_while: 322, 423a.
_kcc_ir_classify_type: 317, 326, 343, 393.
_kcc_ir_cmp_unsigned: 369, 374.
_kcc_ir_convert: 314b, 326, 343, 359, 384, 387, 393, 402b, 407, 417b.
_kcc_ir_cur_bb: 302, 304, 305b, 305c, 316, 317, 321, 326, 376, 400c, 421c, 423ab, 424, 426, 435ab.
_kcc_ir_cur_func: 302, 304, 305ab, 305c, 316, 317, 326, 398.
_kcc_ir_dump: 334, 623.
_kcc_ir_dump_instr: 334, 335.
_kcc_ir_dump_type_short: 334, 336, 337, 338, 339.
_kcc_ir_elem_off_width: 359, 407, 414.
_kcc_ir_emit: 304, 312, 314b, 316, 317, 320, 321, 322, 326, 343, 347, 350, 353a, 355, 357, 359, 367, 369, 375, 376, 378b, 383, 384, 387, 393, 398, 400bc, 402b, 404, 405, 407, 413, 417b, 421c, 423ab, 424, 426, 435ab.
_kcc_ir_emit_aggregate_init: 314a, 314b.
_kcc_ir_emit_entry: 304, 417b.
_kcc_ir_emit_fpround: 343, 369.
_kcc_ir_emit_fptosi: 343.
_kcc_ir_emit_global_inits: 312, 316, 317.
_kcc_ir_emit_indexed_store: 320, 359, 407, 413, 414, 417b.
_kcc_ir_emit_init_stores: 312, 314b.
_kcc_ir_emit_int_to_fp: 343, 369.
_kcc_ir_emit_modinit: 307, 316.
_kcc_ir_emit_sitofp: 343.
_kcc_ir_emit_struct_copy: 314b, 320, 326, 387, 404, 414, 417b.
_kcc_ir_emit_uitofp: 343.
_kcc_ir_field_index: 407, 414.
_kcc_ir_fill: 407, 414.
_kcc_ir_fill_list: 314a, 407, 413, 414, 417b.
_kcc_ir_fill_one: 407, 413, 414.
_kcc_ir_fill_param_classes: 317, 343.

_kcc_ir_funcs: 302, 305c, 316, 317, 334, 441, 445a, 472, 473, 560b, 596.
_kcc_ir_ident_alloca: 398.
_kcc_ir_init: 302, 603, 623.
_kcc_ir_initlist_count: 277, 407, 414, 417b.
_kcc_ir_labels: 305c, 316, 317, 321.
_kcc_ir_label_bb: 321, 322, 342.
_kcc_ir_load_width: 349b, 353a, 357, 359, 367, 378b, 384, 387, 518.
_kcc_ir_locals: 302, 305c, 316, 317, 320, 322, 350, 355, 367, 378b, 382, 387, 393, 398, 405.
_kcc_ir_loop_stack: 302, 305c.
_kcc_ir_next_vreg: 305a, 312, 317, 320, 343, 347, 350, 353a, 355, 357, 359, 367, 369, 375, 376, 378b, 383, 384, 387, 393, 398, 400bc, 402b, 404, 405, 407, 413, 417b, 426.
_kcc_ir_norm32: 359, 369, 375, 378b.
_kcc_ir_op_add: 328, 336.
_kcc_ir_op_alloca: 328, 338.
_kcc_ir_op_and: 328, 336.
_kcc_ir_op_ashr: 328, 336.
_kcc_ir_op_bitcast: 328, 337.
_kcc_ir_op_br: 328, 338.
_kcc_ir_op_br_cond: 328, 338.
_kcc_ir_op_call: 328, 338.
_kcc_ir_op_call_ind: 328, 338.
_kcc_ir_op_cmp_eq: 328, 336.
_kcc_ir_op_cmp_ne: 328, 336.
_kcc_ir_op_cmp_sge: 328, 336.
_kcc_ir_op_cmp_sgt: 328, 336.
_kcc_ir_op_cmp_sle: 328, 336.
_kcc_ir_op_cmp_slt: 328, 336.
_kcc_ir_op_cmp_ule: 328, 336.
_kcc_ir_op_cmp_ult: 328, 336.
_kcc_ir_op_const: 328, 338.
_kcc_ir_op_gep: 328, 338.
_kcc_ir_op_global_ref: 328, 338.
_kcc_ir_op_load: 328, 338.
_kcc_ir_op_lshr: 328, 336.
_kcc_ir_op_mul: 328, 336.
_kcc_ir_op_neg: 328, 337.
_kcc_ir_op_nop: 328, 338.
_kcc_ir_op_not: 328, 337.
_kcc_ir_op_or: 328, 336.
_kcc_ir_op_ret: 328, 338.
_kcc_ir_op_sdiv: 328, 336.
_kcc_ir_op_sext: 328, 337.
_kcc_ir_op_shl: 328, 336.
_kcc_ir_op_smod: 328, 336.
_kcc_ir_op_store: 328, 338.
_kcc_ir_op_string_lit: 328, 338.
_kcc_ir_op_sub: 328, 336.
_kcc_ir_op_trunc: 328, 337.
_kcc_ir_op_udiv: 328, 336.
_kcc_ir_op_umod: 328, 336.
_kcc_ir_op_xor: 328, 336.
_kcc_ir_op_zext: 328, 337.
_kcc_ir_prescan_labels: 317, 321, 342.
_kcc_ir_resolve_designator: 407, 414.
_kcc_ir_scope_jdepth: 305c, 316, 322.
_kcc_ir_scope_journal: 305c, 322.
_kcc_ir_scope_sp: 305c, 316, 322.
_kcc_ir_scope_stack: 305c, 322.
_kcc_ir_set_bb: 305b, 316, 317, 322, 342, 376, 400c, 421c, 423ab, 424, 426.
_kcc_ir_statics: 305c, 312, 316, 317, 350, 355, 367, 378b, 383, 387, 417b.
_kcc_ir_static_name: 312, 417a, 417b, 465, 563.
_kcc_ir_static_types: 305c, 312, 316, 317, 350, 367, 417b.
_kcc_ir_store_width: 314b, 320, 340, 349b, 359, 384, 387, 407, 417b.
_kcc_ir_str_arrow: 328, 334.
_kcc_ir_str_bb: 328, 334.
_kcc_ir_str_builtin_constant_p: 393, 398.
_kcc_ir_str_builtin_expect: 393, 398.
_kcc_ir_str_builtin_va_end: 393, 398.
_kcc_ir_str_builtin_va_start: 393, 398.
_kcc_ir_str_colon_nl: 328, 334.
_kcc_ir_str_comma: 328, 336, 338.
_kcc_ir_str_comma_bb: 328, 338.
_kcc_ir_str_eq: 328, 335.
_kcc_ir_str_func: 328, 334.
_kcc_ir_str_indent: 328, 335.
_kcc_ir_str_ptr_pct: 328, 338.
_kcc_ir_str_unknown: 328, 335.
_kcc_ir_str_va_end: 393, 398.
_kcc_ir_str_va_start: 393, 398.
_kcc_ir_subobj: 407, 414.
_kcc_ir_sw_disp: 426.
_kcc_ir_sw_scan: 426.
_kcc_ir_type_has_member: 407, 414.
_kcc_ir_type_is_fp: 343, 359, 369, 402b, 554.
_kcc_ir_type_slots: 320, 349a, 393, 417b.
_kcc_ir_ty_f32: 328, 339.
_kcc_ir_ty_f64: 328, 339.
_kcc_ir_ty_i1: 328, 339.
_kcc_ir_ty_i16: 328, 339.
_kcc_ir_ty_i32: 328, 339.
_kcc_ir_ty_i64: 328, 339.
_kcc_ir_ty_i8: 328, 339.
_kcc_ir_ty_ptr: 328, 339.
_kcc_ir_ty_struct: 328, 339.
_kcc_ir_ty_unknown: 328, 339.
_kcc_ir_ty_void: 328, 339.
_kcc_ir_validate: 441, 603.
_kcc_is_float_literal: 215, 220.
_kcc_kw_s_alignas: 103.
_kcc_kw_s_alignof: 103.
_kcc_kw_s_atomic: 103.
_kcc_kw_s_auto: 103.
_kcc_kw_s_bool: 103.
_kcc_kw_s_break: 103.
_kcc_kw_s_case: 103.
_kcc_kw_s_char: 103.
_kcc_kw_s_complex: 103.
_kcc_kw_s_const: 103.
_kcc_kw_s_continue: 103.
_kcc_kw_s_default: 103.
_kcc_kw_s_do: 103.
_kcc_kw_s_double: 103.
_kcc_kw_s_else: 103.
_kcc_kw_s_enum: 103.
_kcc_kw_s_extern: 103.
_kcc_kw_s_float: 103.
_kcc_kw_s_for: 103.
_kcc_kw_s_generic: 103.
_kcc_kw_s_goto: 103.
_kcc_kw_s_if: 103.
_kcc_kw_s_imaginary: 103.
_kcc_kw_s_inline: 103.
_kcc_kw_s_int: 103.
_kcc_kw_s_long: 103.
_kcc_kw_s_noreturn: 103.
_kcc_kw_s_register: 103.
_kcc_kw_s_restrict: 103.
_kcc_kw_s_return: 103.
_kcc_kw_s_sassert: 103.
_kcc_kw_s_short: 103.
_kcc_kw_s_signed: 103.
_kcc_kw_s_sizeof: 103.
_kcc_kw_s_static: 103.

_kcc_kw_s_struct: 103.
_kcc_kw_s_switch: 103.
_kcc_kw_s_tlocal: 103.
_kcc_kw_s_typedef: 103.
_kcc_kw_s_union: 103.
_kcc_kw_s_unsigned: 103.
_kcc_kw_s_void: 103.
_kcc_kw_s_volatile: 103.
_kcc_kw_s_while: 103.
_kcc_kw_table: 93, 104a, 104b.
_kcc_lex_cursor: 90ab, 91b, 92, 93, 94ab, 95, 96, 104b, 107a, 111, 112, 113, 117, 123, 136, 139a, 147a, 151.
_kcc_lex_dump_all: 100a, 622.
_kcc_lex_dump_token: 99, 100a, 115.
_kcc_lex_end: 90ab, 93, 94ab, 95, 96, 104b, 107a, 111, 112, 113, 117, 123, 136, 139a, 147a, 151.
_kcc_lex_init: 90a, 151, 603, 622, 623, 630a.
_kcc_lex_kw_init: 104a, 621, 629.
_kcc_lex_line: 90ab, 91a, 92, 104b, 107a, 111, 112, 113, 116, 121, 136.
_kcc_lex_line_start: 90ab, 91a, 92, 104b, 107a, 111, 112, 113.
_kcc_lex_next: 90b, 100a, 108, 109, 117, 119ab, 123, 135, 136, 139a, 140, 141, 142, 143, 144, 145ab, 146, 147a, 151, 630a.
_kcc_lex_type_name: 99, 100b, 175ab.
_kcc_linker_add_reloc: 477b, 518, 521, 592b.
_kcc_linker_add_sym: 469, 472, 476, 477b, 592a, 592c, 593.
_kcc_linker_extern: 518, 521, 592c.
_kcc_linker_extern_ht: 469, 472, 591b, 592c, 593, 594.
_kcc_linker_global_sym: 465, 521, 593.
_kcc_linker_init: 591b, 603.
_kcc_linker_relocs: 537, 577b, 580, 587, 591b, 592b, 594.
_kcc_linker_syms: 478, 498b, 537, 543, 577b, 580, 581, 586, 588, 589ab, 591b, 592ac, 593, 594.
_kcc_loop_depth: 328, 426, 434b, 435ab.
_kcc_loop_pop: 423ab, 424, 426, 434b.
_kcc_loop_push: 423ab, 424, 426, 434b.
_kcc_loop_stack: 328, 426, 434b, 435ab.
_kcc_macho_buf: 577b, 578, 579, 589b, 590.
_kcc_macho_cap: 577b, 579, 590.
_kcc_macho_fd: 577b, 589b, 590.
_kcc_macho_finalize: 580, 603.
_kcc_macho_init: 577b, 603.
_kcc_macho_pad_to: 579, 586.
_kcc_macho_pos: 577b, 578, 579, 589b, 590.
_kcc_macho_write16: 578, 588, 589a.
_kcc_macho_write32: 578, 582, 583, 585, 587, 588, 589a, 600.
_kcc_macho_write64: 578, 582, 583, 586, 588, 589a, 600.
_kcc_macho_write8: 578, 588, 589ab.
_kcc_macho_write_bytes: 579, 586, 589b, 600.
_kcc_modinit_fileoff: 302, 305c, 580, 582, 583.
_kcc_modinit_present: 302, 305c, 316, 567, 580, 582, 583, 586, 587, 588.
_kcc_modinit_sym: 302, 305c, 472, 543, 587.
_kcc_opt_constfold: 445b, 447.
_kcc_opt_copyprop: 445b, 457.
_kcc_opt_cse: 445b, 457.
_kcc_opt_dce: 445b, 453a.
_kcc_opt_run: 445a, 603, 623.
_kcc_opt_run_func: 445a, 445b.
_kcc_parse: 172, 603, 622, 623.
_kcc_parse_abstract_declarator: 196, 201.
_kcc_parse_advance: 168b.
_kcc_parse_arg_list: 212, 223a.
_kcc_parse_asm_clobbers: 227.
_kcc_parse_asm_extended: 226, 227.
_kcc_parse_asm_opsec: 227.
_kcc_parse_assign_expr: 191b, 192, 196, 203, 207b, 208a, 212, 218, 223a, 227, 228.
_kcc_parse_atof: 215, 220.
_kcc_parse_atoi: 215, 219.
_kcc_parse_binary_expr: 208b, 209a, 228.
_kcc_parse_builtin_offsetof: 212.
_kcc_parse_case: 226, 234c.
_kcc_parse_cast_expr: 209ab, 211.
_kcc_parse_compound: 181, 211, 225, 230.
_kcc_parse_cond_expr: 199a, 200b, 205a, 208a, 208b, 228, 234c.
_kcc_parse_decl: 172, 181, 230, 233.
_kcc_parse_declarator: 181, 182, 190, 192, 195, 199a.
_kcc_parse_decl_specs: 181, 186b, 195, 199a, 201.
_kcc_parse_default: 226, 234c.
_kcc_parse_designated_element: 203.
_kcc_parse_direct_declarator: 190, 191b.
_kcc_parse_do: 225, 232b.
_kcc_parse_enum: 188, 200b.
_kcc_parse_error: 168b, 169, 215.
_kcc_parse_error_flag: 169, 178a.
_kcc_parse_expect: 168b, 181, 182, 184ab, 186b, 191b, 192, 196, 199ab, 200b, 203, 205a, 208b, 210, 211, 212, 218, 226, 227, 228, 230, 231ab, 232ab, 233, 234abc.
_kcc_parse_expr: 184b, 207b, 208b, 210, 211, 212, 218, 226, 228, 231ab, 232ab, 233, 234b.
_kcc_parse_expr_rest: 226, 228.
_kcc_parse_for: 225, 233.
_kcc_parse_goto: 226, 234a.
_kcc_parse_if: 225, 231a.
_kcc_parse_initializer: 182, 203, 211.
_kcc_parse_is_type_token: 184b, 202, 210, 211, 230, 233.
_kcc_parse_match: 168b.
_kcc_parse_param_decl: 192, 195, 196.
_kcc_parse_peek: 168b.
_kcc_parse_postfix_expr: 209b, 212.
_kcc_parse_postfix_tail: 211, 212, 226.
_kcc_parse_primary_expr: 212, 215.
_kcc_parse_return: 226, 234b.
_kcc_parse_skip_attributes: 181, 182, 184b, 186b, 190, 195, 198, 199ab, 200b.
_kcc_parse_static_assert: 184a, 205a.
_kcc_parse_stmt: 225, 226, 230, 231ab, 232ab, 233, 234c.
_kcc_parse_struct_or_union: 188, 198.
_kcc_parse_switch: 225, 231b.
_kcc_parse_sync: 169, 184a.
_kcc_parse_typedef_funcfix: 182, 184a, 191a.
_kcc_parse_typeof: 184b, 188.
_kcc_parse_type_name: 184b, 201, 210, 211, 212, 218.
_kcc_parse_unary_expr: 209b, 210, 211.
_kcc_parse_while: 225, 232a.
_kcc_pp_add_cmdline_define: 132, 606b.
_kcc_pp_add_default_includes: 150, 621.
_kcc_pp_add_include_dir: 150, 606a.
_kcc_pp_apply_cmdline_defines: 132.
_kcc_pp_cond_depth: 107b, 108, 116, 136, 139ab.
_kcc_pp_cond_stack: 108, 116, 139ab.
_kcc_pp_ctx_depth: 107b, 108, 111, 112, 116, 123, 125b.
_kcc_pp_ctx_stack: 111, 112, 116.
_kcc_pp_dir_line: 109, 116, 118, 125a, 135, 139ab, 140, 151, 159b.
_kcc_pp_do_pragma: 110, 119b.
_kcc_pp_dump_all: 115, 622.
_kcc_pp_emit_string: 127, 131.
_kcc_pp_err_def_name: 116, 118.
_kcc_pp_err_def_param: 116, 118.
_kcc_pp_err_depth: 111, 116.
_kcc_pp_err_ifdef_name: 116, 135.
_kcc_pp_err_include: 116, 151.
_kcc_pp_err_stray_elif: 116, 139a.
_kcc_pp_err_stray_else: 116, 139b.

_kcc_pp_err_stray_endif: 116, 139b.
_kcc_pp_err_unknown_dir: 110, 116.
_kcc_pp_err_unterm_args: 116, 125a.
_kcc_pp_err_unterm_cond: 108, 116.
_kcc_pp_eval_add: 144.
_kcc_pp_eval_band: 142.
_kcc_pp_eval_bor: 141, 142.
_kcc_pp_eval_bxor: 142.
_kcc_pp_eval_expr: 136, 139a, 141, 145b, 147a.
_kcc_pp_eval_mul: 144.
_kcc_pp_eval_shift: 143, 144.
_kcc_pp_eval_ternary: 141.
_kcc_pp_expand_all_args: 125a, 125b.
_kcc_pp_expand_arg: 125b, 136.
_kcc_pp_has_pending: 107b, 108, 116, 125b.
_kcc_pp_init: 107b, 603, 622, 623.
_kcc_pp_macros: 107b, 116, 118, 119ab, 121, 132, 135, 136, 146.
_kcc_pp_next: 108, 115, 125b, 136, 165, 168b, 169, 172, 181, 182, 184ab, 186b, 188, 190, 191b, 192, 196, 198, 199a, 200b, 203, 205a, 207b, 208ab, 209ab, 210, 211, 212, 214, 215, 218, 223a, 225, 226, 227, 228, 230, 231ab, 232ab, 233, 234abc.
_kcc_pp_pragma_name: 119b.
_kcc_pp_prev_line: 107b, 108, 111, 112, 116, 125b, 151.
_kcc_pp_pre_c17val: 116, 132.
_kcc_pp_pre_hosted: 116, 132.
_kcc_pp_pre_one: 116, 132.
_kcc_pp_pre_stdc: 116, 132.
_kcc_pp_pre_stdc_ver: 116, 132.
_kcc_pp_s_define: 109, 116.
_kcc_pp_s_defined: 116, 136, 146.
_kcc_pp_s_elif: 109, 116, 140.
_kcc_pp_s_else: 109, 116, 140.
_kcc_pp_s_endif: 109, 116, 140.
_kcc_pp_s_error: 109, 116.
_kcc_pp_s_if: 109, 116, 140.
_kcc_pp_s_ifdef: 109, 116, 140.
_kcc_pp_s_ifndef: 109, 116, 140.
_kcc_pp_s_include: 109, 116.
_kcc_pp_s_line: 109, 116.
_kcc_pp_s_pragma: 109, 116.
_kcc_pp_s_undef: 109, 116.
_kcc_pp_try_expand: 108, 121, 132.
_kcc_prec_lookup: 207a, 209a, 228.
_kcc_preregister_globals: 465, 603.
_kcc_ra_disq: 481.
_kcc_ra_use: 481.
_kcc_rodata_aos_ok: 308.
_kcc_rodata_binop_ok: 308.
_kcc_rodata_const_fileoff: 479, 580, 582, 583.
_kcc_rodata_const_intexpr: 308.
_kcc_rodata_const_present: 463b, 469, 479, 580, 582, 583, 586, 587.
_kcc_rodata_const_relocs: 463b, 469, 479, 580, 583, 587.
_kcc_rodata_const_size: 463b, 469, 479, 580, 582, 583, 586.
_kcc_rodata_is_lit: 308.
_kcc_rodata_leaf_bits: 468a, 469, 564.
_kcc_rodata_struct_ok: 308.
_kcc_round_to_float: 215, 220.
_kcc_sema_callee_ctx: 263, 277, 289.
_kcc_sema_callee_implicit: 263, 277, 289.
_kcc_sema_check: 259a, 603, 623.
_kcc_sema_check_decl: 259a, 264, 269, 273.
_kcc_sema_check_stmt: 264, 271, 272, 273, 274.
_kcc_sema_cur_ret: 263, 264, 273, 281.
_kcc_sema_dump_types: 262, 623.
_kcc_sema_expr: 259b, 264, 271, 272, 273, 274, 276, 284, 286, 287, 289, 290, 292ab, 426.
_kcc_sema_expr_generic: 274, 276.
_kcc_sema_expr_member: 274, 290.
_kcc_sema_redef_pre: 263, 277.
_kcc_sema_resolve_type: 259b, 262, 264, 267, 269, 274, 276, 277, 290, 292ab, 308, 312, 314b, 317, 320, 340, 343, 347, 359, 393, 407, 413, 417b, 426, 465, 469, 562, 563, 564.
_kcc_sema_str_builtin_va_list: 259b, 263.
_kcc_sema_str_colon: 262, 263.
_kcc_sema_str_eq: 262, 263.
_kcc_sema_str_func: 262, 263.
_kcc_sema_str_nl: 262, 263.
_kcc_sema_str_typedef: 262, 263.
_kcc_sema_str_var: 262, 263.
_kcc_sema_str_va_list: 259b, 263.
_kcc_source_encode: 85, 151, 603, 622, 623, 630a.
_kcc_source_open: 81b, 151, 603, 622, 623.
_kcc_source_splice: 87, 151, 603, 622, 623, 630a.
_kcc_src_base: 75, 81b, 82, 85, 87, 107a, 111, 112, 151, 603, 622, 623, 630a.
_kcc_src_newline: 81b, 82.
_kcc_src_path: 73c, 81b, 82, 107a, 111, 112, 121, 136, 148, 151, 596, 630a.
_kcc_src_size: 75, 81b, 82, 85, 87, 107a, 111, 112, 151, 603, 622, 623, 630a.
_kcc_strlit_gather: 165, 215, 226, 227.
_kcc_strlit_process: 161, 165, 215.
_kcc_strtab: 66ab, 66c.
_kcc_strtab_init: 66a, 603, 622, 623.
_kcc_strtab_intern: 66b, 188, 191b, 198, 200b, 203, 205a, 212, 214, 215, 226, 234a.
_kcc_str_builtin_va_list: 171, 179.
_kcc_str_va_list: 171, 179.
_kcc_symtab_current_ht: 255, 277.
_kcc_symtab_declare: 254b, 264, 269.
_kcc_symtab_declare_tag: 256a, 269.
_kcc_symtab_init: 253b, 603, 623.
_kcc_symtab_label_add: 257b, 273.
_kcc_symtab_label_begin: 257a, 264.
_kcc_symtab_label_end: 257a, 264.
_kcc_symtab_label_lookup: 257b.
_kcc_symtab_lookup: 255, 259b, 277, 292b, 308, 350, 355, 367, 426.
_kcc_symtab_lookup_current: 255, 277.
_kcc_symtab_lookup_tag: 256b, 259b, 269.
_kcc_symtab_pop_scope: 254a, 264, 272.
_kcc_symtab_push_scope: 253b, 253c, 264, 272.
_kcc_sym_depth: 253bc, 254ab, 256a, 258, 269, 277.
_kcc_sym_labels: 253b, 257ab, 258.
_kcc_sym_ordinary: 253bc, 254ab, 255, 258.
_kcc_sym_tags: 253bc, 254a, 256ab, 258.
_kcc_target: 326, 546, 568b, 603, 609, 611.
_kcc_test_expect_tok: 630a, 633abc, 634ab, 635ab.
_kcc_test_lex_setup: 630a, 633abc, 634ab, 635ab.
_kcc_test_path: 630a, 630b.
_kcc_test_table: 629, 636a.
_kcc_tn_alignas: 100b, 101b.
_kcc_tn_alignof: 100b, 101b.
_kcc_tn_amp: 101a, 101b.
_kcc_tn_ampamp: 101a, 101b.
_kcc_tn_ampequal: 101a, 101b.
_kcc_tn_arrow: 101a, 101b.
_kcc_tn_atomic: 100b, 101b.
_kcc_tn_auto: 100b, 101b.
_kcc_tn_bool: 100b, 101b.
_kcc_tn_break: 100b, 101b.
_kcc_tn_caret: 101a, 101b.
_kcc_tn_caretequal: 101a, 101b.
_kcc_tn_case: 100b, 101b.
_kcc_tn_char: 100b, 101b.

_kcc_tn_char_lit: 100b, 101b.
_kcc_tn_colon: 101a, 101b.
_kcc_tn_comma: 101a, 101b.
_kcc_tn_complex: 100b, 101b.
_kcc_tn_const: 100b, 101b.
_kcc_tn_continue: 100b, 101b.
_kcc_tn_default: 100b, 101b.
_kcc_tn_do: 100b, 101b.
_kcc_tn_dot: 101a, 101b.
_kcc_tn_double: 100b, 101b.
_kcc_tn_ellipsis: 101a, 101b.
_kcc_tn_else: 100b, 101b.
_kcc_tn_enum: 100b, 101b.
_kcc_tn_eof: 100b, 101b.
_kcc_tn_equal: 101a, 101b.
_kcc_tn_equalequal: 101a, 101b.
_kcc_tn_exclaim: 101a, 101b.
_kcc_tn_exclaimequal: 101a, 101b.
_kcc_tn_extern: 100b, 101b.
_kcc_tn_float: 100b, 101b.
_kcc_tn_for: 100b, 101b.
_kcc_tn_generic: 100b, 101b.
_kcc_tn_goto: 100b, 101b.
_kcc_tn_greater: 101a, 101b.
_kcc_tn_greaterequal: 101a, 101b.
_kcc_tn_greatergreater: 101a, 101b.
_kcc_tn_greatergreaterequal: 101a, 101b.
_kcc_tn_hash: 101a, 101b.
_kcc_tn_hashhash: 101a, 101b.
_kcc_tn_identifier: 100b, 101b.
_kcc_tn_if: 100b, 101b.
_kcc_tn_imaginary: 100b, 101b.
_kcc_tn_inline: 100b, 101b.
_kcc_tn_int: 100b, 101b.
_kcc_tn_int128: 100b, 101b.
_kcc_tn_lbrace: 101a, 101b.
_kcc_tn_lbracket: 101a, 101b.
_kcc_tn_less: 101a, 101b.
_kcc_tn_lessequal: 101a, 101b.
_kcc_tn_lessless: 101a, 101b.
_kcc_tn_lesslessequal: 101a, 101b.
_kcc_tn_long: 100b, 101b.
_kcc_tn_lparen: 101a, 101b.
_kcc_tn_minus: 101a, 101b.
_kcc_tn_minusequal: 101a, 101b.
_kcc_tn_minusminus: 101a, 101b.
_kcc_tn_noreturn: 100b, 101b.
_kcc_tn_number: 100b, 101b.
_kcc_tn_percent: 101a, 101b.
_kcc_tn_percentequal: 101a, 101b.
_kcc_tn_pipe: 101a, 101b.
_kcc_tn_pipeequal: 101a, 101b.
_kcc_tn_pipepipe: 101a, 101b.
_kcc_tn_plus: 101a, 101b.
_kcc_tn_plusequal: 101a, 101b.
_kcc_tn_plusplus: 101a, 101b.
_kcc_tn_question: 101a, 101b.
_kcc_tn_rbrace: 101a, 101b.
_kcc_tn_rbracket: 101a, 101b.
_kcc_tn_register: 100b, 101b.
_kcc_tn_restrict: 100b, 101b.
_kcc_tn_return: 100b, 101b.
_kcc_tn_rparen: 101a, 101b.
_kcc_tn_semi: 101a, 101b.
_kcc_tn_short: 100b, 101b.
_kcc_tn_signed: 100b, 101b.
_kcc_tn_sizeof: 100b, 101b.
_kcc_tn_slash: 101a, 101b.
_kcc_tn_slashequal: 101a, 101b.
_kcc_tn_star: 101a, 101b.
_kcc_tn_starequal: 101a, 101b.
_kcc_tn_static: 100b, 101b.
_kcc_tn_static_assert: 100b, 101b.
_kcc_tn_string_lit: 100b, 101b.
_kcc_tn_struct: 100b, 101b.
_kcc_tn_switch: 100b, 101b.
_kcc_tn_thread_local: 100b, 101b.
_kcc_tn_tilde: 101a, 101b.
_kcc_tn_typedef: 100b, 101b.
_kcc_tn_union: 100b, 101b.
_kcc_tn_unknown: 100b, 101b.
_kcc_tn_unsigned: 100b, 101b.
_kcc_tn_void: 100b, 101b.
_kcc_tn_volatile: 100b, 101b.
_kcc_tn_while: 100b, 101b.
_kcc_tok_col: 92, 94b, 95, 99, 104b, 168a, 169, 188.
_kcc_tok_len: 91b, 93, 94ab, 95, 98, 99, 104b, 109, 117, 119ab, 121, 125b, 135, 136, 140, 145b, 146, 151, 165, 184b, 186a, 188, 191b, 198, 200b, 202, 203, 205a, 212, 214, 215, 226, 230, 234a.
_kcc_tok_line: 92, 94b, 95, 99, 104b, 108, 109, 111, 140, 168a, 169, 188, 254b.
_kcc_tok_start: 91b, 93, 94ab, 95, 98, 99, 104b, 109, 117, 119ab, 121, 123, 125b, 135, 136, 140, 145b, 146, 151, 165, 184b, 186a, 188, 191b, 198, 200b, 202, 203, 205a, 212, 214, 215, 226, 230, 234a.
_kcc_tok_type: 91b, 93, 94ab, 95, 98, 99, 100a, 104b, 108, 109, 115, 117, 119ab, 123, 125b, 135, 136, 140, 141, 142, 143, 144, 145ab, 146, 151, 165, 168b, 169, 172, 181, 182, 184ab, 186b, 190, 191b, 192, 195, 196, 198, 199a, 200b, 201, 202, 203, 207b, 208ab, 209ab, 210, 211, 212, 215, 218, 223a, 225, 226, 227, 228, 230, 231a, 233, 234bc, 630a.
_kcc_typedef_add: 171, 182, 184a.
_kcc_typedef_init: 170a, 603, 622, 623.
_kcc_typedef_lookup: 171, 188, 202.
_kcc_typedef_pop_scope: 170b, 230.
_kcc_typedef_push_scope: 170b, 230.
_kcc_typedef_scopes: 170ab, 171, 178a.
_kcc_types_init: 239b, 603, 623.
_kcc_type_align: 267.
_kcc_type_array: 243a, 259b, 292b.
_kcc_type_bool: 239b, 246, 250b.
_kcc_type_char_s: 240, 246, 250b, 283, 292b.
_kcc_type_char_u: 240, 246, 250b.
_kcc_type_compatible: 247a, 276, 426.
_kcc_type_double: 241, 246, 250b, 283, 295b, 343, 369, 378b.
_kcc_type_dump: 249a, 250a, 262.
_kcc_type_enum: 244, 259b, 269.
_kcc_type_find_member: 290, 355, 426.
_kcc_type_float: 241, 246, 250b, 283, 295b, 343.
_kcc_type_from_specmask: 245a, 259b.
_kcc_type_func: 243b, 259b, 264.
_kcc_type_int128_s: 240, 246, 250b.
_kcc_type_int128_u: 240, 246, 250b.
_kcc_type_int_s: 240, 246, 250b, 259b, 269, 274, 276, 277, 283, 284, 286, 289, 290, 295a, 312, 314b, 316, 317, 343, 350, 353a, 355, 357, 367, 369, 375, 376, 378b, 383, 384, 387, 393, 398, 400bc, 404, 405, 407, 413, 417b, 426, 450.
_kcc_type_int_u: 240, 246, 250b, 283, 290.
_kcc_type_is_arithmetic: 248a, 248b, 297b.
_kcc_type_is_integer: 247b, 248a, 297b.
_kcc_type_is_scalar: 248b.
_kcc_type_layout: 243a, 267, 268, 269, 290, 308, 314b, 320, 326, 343, 347, 349a, 350, 355, 359, 369, 384, 387, 407, 413, 414, 417b, 426, 465, 469, 562, 563, 564.
_kcc_type_ldouble: 241, 246, 250b, 295b.
_kcc_type_llong_s: 240, 246, 250b, 283.
_kcc_type_llong_u: 240, 246, 250b, 283.

_kcc_type_long_s: 240, 246, 250b, 283, 284, 369, 378b.
_kcc_type_long_u: 240, 246, 250b, 274, 283, 292b, 347, 359.
_kcc_type_param_adjust: 242b, 264, 320, 343.
_kcc_type_ptr: 242a, 242b, 259b, 276, 283, 286, 296ab.
_kcc_type_rank: 248c, 295b.
_kcc_type_short_s: 240, 246, 250b.
_kcc_type_short_u: 240, 246, 250b.
_kcc_type_struct: 199b, 243c, 259b, 269.
_kcc_type_union: 243c, 259b, 269.
_kcc_type_void: 239b, 246, 250b, 274, 326.
_kcc_ty_str_arr_open: 250a, 250b.
_kcc_ty_str_bool: 249b, 250b.
_kcc_ty_str_char: 249b, 250b.
_kcc_ty_str_const: 249a, 250b.
_kcc_ty_str_double: 249b, 250b.
_kcc_ty_str_enum: 250a, 250b.
_kcc_ty_str_float: 249b, 250b.
_kcc_ty_str_func: 250a, 250b.
_kcc_ty_str_int: 249b, 250b.
_kcc_ty_str_ldouble: 249b, 250b.
_kcc_ty_str_llong: 249b, 250b.
_kcc_ty_str_long: 249b, 250b.
_kcc_ty_str_null: 249b, 250b.
_kcc_ty_str_ptr: 250a, 250b.
_kcc_ty_str_short: 249b, 250b.
_kcc_ty_str_struct: 250a, 250b.
_kcc_ty_str_uchar: 249b, 250b.
_kcc_ty_str_uint: 249b, 250b.
_kcc_ty_str_ullong: 249b, 250b.
_kcc_ty_str_ulong: 249b, 250b.
_kcc_ty_str_union: 250a, 250b.
_kcc_ty_str_unknown: 249a, 250b.
_kcc_ty_str_ushort: 249b, 250b.
_kcc_ty_str_void: 249b, 250b.
_kcc_ty_str_volatile: 249a, 250b.
_kcc_vec_get: 65b, 171, 254b, 255, 256ab, 257b, 334, 441, 445a, 447, 448, 453b, 454, 457, 472, 473, 474, 478, 481, 490, 493, 498a, 559, 560b, 568a, 580, 581, 586, 587, 588, 589ab, 596.
_kcc_vec_init: 64b, 170a, 253b, 568b.
_kcc_vec_push: 65a, 170ab, 253c, 257a, 305b, 316, 317, 469, 476, 477b, 516, 554, 592ab.
_kcc_warn_all: 77a, 277, 281, 284, 287, 608b, 609, 611.
_kcc_x86_render: 568b, 603.
_kcc_x86_tu: 546, 562, 563, 568b, 603.
_main: 38k, 611, 615, 621, 629, 637, 640b, 644, 646b, 648b, 651, 653b, 656c, 659b, 662c, 665, 668c, 673, 681c, 684b, 686c, 694c, 702b, 710b, 717b, 719, 725b, 738b, 742b, 751, 768b, 792b, 818b, 836b, 902b, 909, 914, 917.
_test_lex_comments: 634b, 636a.
_test_lex_keywords: 633a, 636a.
_test_lex_line_splice: 635b, 636a.
_test_lex_numbers: 633b, 636a.
_test_lex_punctuators: 634a, 636a.
_test_lex_strings: 633c, 636a.
_test_lex_trigraphs: 635a, 636a.

A compiler you can read.

kcc is a formal C17 compiler written entirely in ARM64 assembly, set down as one literate program in weft — a metalanguage that weaves prose and code together, every instruction explained before it is written.

It checks C against the Standard, optimises it, allocates registers, and emits native machine code. It compiles itself to a byte-identical binary, and cross-compiles x86-64 for Linux and macOS. It builds and runs Lua, SQLite, and DOOM — and boots a Linux kernel.

kcc AT A GLANCE

- Formal C17, checked against the Standard
- Written in ARM64 assembly · self-hosting
- Cross-compiles to x86-64 · Linux and macOS
- Runs Lua · SQLite · DOOM · the Linux kernel

The case for Literate Driven Development: that software built in the open, with an AI collaborator, and explained as it is made, can be more certain and more humane — and can aspire to be a work of art.

Learn more about Literate Driven Development
literatedriven.com

ISBN — TO BE ASSIGNED

Cuauhtémoc Pacheco

kcc 1.0.0

co-authored with an AI · Claude · Independently published